The Ultimate High-ROI LeetCode Patterns Guide

Part 1: 41 patterns · 632 problems
Part 2: 50 extra topics · 210 more problems for practice
one problem per page · statements, examples, figures, Python solutions and follow-ups

Pattern list and problem order: Automedon / ultimate-leetcode-patterns (github.com/Automedon/ultimate-leetcode-patterns).
Problem statements, figures and solutions: doocs / leetcode (github.com/doocs/leetcode), CC BY-SA 4.0.
LeetCode is a trademark of LeetCode LLC; problems link back to leetcode.com.

How to use this guide

41 patterns, 20 problems each, all free — no Premium garbage. Sorted top to bottom by how often each pattern actually shows up in interviews. Hash Map and Sliding Window first because they're everywhere. Segment Tree and Fenwick Tree last because almost nobody asks them. Inside each pattern: Easy to Hard, so you're not getting wrecked by a Hard problem before you've even nailed the basic version of the trick.*

How to actually use this:

Don't grind all 820. That's a waste of your time and most of this list won't come up. Drill the first 10-15 patterns until you don't have to think — you see the problem, your hands already know the code. Everything below that is bonus coverage for if your interviewer likes to go off-script. Speed matters more than breadth here: 45 minutes isn't a lot of time, and if you're still figuring out syntax halfway through, you're not getting to the part where you actually look good.

Hard truths about the process:

Most loops are decided on the first problem.

Speed and clean code beat a late correct answer.

Follow-ups exist to catch memorized solutions.

You're graded against other candidates that week, not a fixed bar.

HOW THIS PDF IS ORGANISED

- Every pattern is a chapter. It opens with a one-page note (when to use it, how to spot it, a small reusable template) and the list of its 20 problems.
- Every problem starts on its own page: the full statement with examples, figures and constraints, then every solution as Python 3 with the reasoning and the complexity.
- Inside a pattern the problems go from Easy to Hard. Some problems belong to more than one pattern. Each problem is printed once, in the first pattern that lists it (its header says where else it appears); the other chapter lists it too, with a link to that page.
- A ★ marks the top 7 of every chapter: Medium or Hard problems chosen because they cover the chapter's variations, so if you can solve them cold, the other problems of the chapter are routine. A chapter or Part 2 topic with 7 or fewer Medium and Hard problems has all of them starred. Do the starred problems first, then the rest.
- Click anything in the contents, or use Prev / Contents / Next at the bottom of every page. The PDF outline (bookmarks panel) lists all patterns and problems.
- Solutions: the doocs/leetcode Python 3 solutions. Premium problems are included, with their statements and solutions.
- Part 1 is the 41 patterns. Part 2 is extra topics and more practice: the problems of AlgoMaster's 600 list that Part 1 does not have, in AlgoMaster's own groups (each group page also lists the group's problems that are already in Part 1).

Contents

Click a part, a pattern, or a problem on the pattern's first page.

PART 1 · The 41 patterns			6
#	Pattern	E / M / H ★ top	Page
1	Hash Map / Hash Set	8 / 10 / 2 ★ 7	7
2	Sliding Window	1 / 16 / 3 ★ 7	68
3	Two Pointers	9 / 9 / 2 ★ 7	127
4	Binary Search	3 / 15 / 2 ★ 7	183
5	Prefix Sum	3 / 17 / 0 ★ 7	239
6	String Manipulation / Parsing	4 / 13 / 3 ★ 7	291
7	String Matching / Pattern Search	5 / 6 / 9 ★ 7	348
8	Stack	6 / 13 / 1 ★ 7	387
9	Monotonic Stack / Queue (Specific Variants)	1 / 13 / 6 ★ 7	432
10	Queue / Deque	4 / 12 / 4 ★ 7	477
11	Tree DFS (Preorder / Inorder / Postorder)	10 / 9 / 1 ★ 7	529
12	Tree BFS (Level Order)	4 / 16 / 0 ★ 7	595
13	Dynamic Programming - 1D	1 / 19 / 0 ★ 7	673
14	Recursion / Divide and Conquer	1 / 10 / 9 ★ 7	724
15	Backtracking / Subsets / Combinations / Permutations	0 / 18 / 2 ★ 7	779
16	Graph DFS	1 / 18 / 1 ★ 7	841
17	Graph BFS	0 / 14 / 6 ★ 7	923
18	Matrix / Grid Traversal	4 / 15 / 1 ★ 7	996
19	Heap / Priority Queue	4 / 14 / 2 ★ 7	1037
20	Intervals (Merge / Insert / Overlap)	0 / 13 / 7 ★ 7	1086
21	Greedy	2 / 15 / 3 ★ 7	1146
22	Dynamic Programming - 2D / Grid	0 / 12 / 8 ★ 7	1171
23	Sorting	6 / 14 / 0 ★ 7	1229
24	Fast & Slow Pointers (Linked List)	6 / 14 / 0 ★ 7	1267
25	Linked List Reversal / Manipulation	4 / 14 / 2 ★ 7	1324
26	Binary Search Tree (BST)	8 / 12 / 0 ★ 7	1392
27	Bit Manipulation	8 / 12 / 0 ★ 7	1458
28	Dynamic Programming - Knapsack Style	0 / 14 / 6 ★ 7	1503
29	Design Problems	1 / 16 / 3 ★ 7	1544
30	Topological Sort	0 / 13 / 7 ★ 7	1594
31	Union-Find (Disjoint Set)	1 / 13 / 6 ★ 7	1660
32	Trie (Prefix Tree)	2 / 11 / 7 ★ 7	1725
33	Cyclic Sort	9 / 9 / 2 ★ 7	1759
34	Math / Number Theory	10 / 10 / 0 ★ 7	1792
35	Sliding Window Maximum / Minimum (Specific Variants)	2 / 13 / 5 ★ 7	1835
36	Simulation / Array Rearrangement	4 / 15 / 1 ★ 7	1854
37	Two Heaps	3 / 10 / 7 ★ 7	1904
38	K-way Merge	3 / 11 / 6 ★ 7	1922
39	Game Theory / Minimax	3 / 12 / 5 ★ 7	1945
40	Segment Tree	0 / 7 / 13 ★ 7	2005

PART 1 · The 41 patterns			6
#	Pattern	E / M / H ★ top	Page
41	Binary Indexed Tree (Fenwick Tree)	0 / 10 / 10 ★ 7	2042

PART 2 · Extra topics and more practice			2076
#	Topic	E / M / H ★ top	Page
42	Arrays	5 / 4 / 0 ★ 4	2078
43	Strings	3 / 4 / 1 ★ 5	2099
44	Bit Manipulation	2 / 0 / 0 (no Medium / Hard)	2119
45	Hash Tables	4 / 4 / 0 ★ 4	2124
46	Two Pointers	3 / 2 / 0 ★ 2	2145
47	Prefix Sum	2 / 3 / 0 ★ 3	2160
48	Sliding Window - Fixed Size	0 / 1 / 1 ★ 2	2174
49	Sliding Window - Dynamic Size	0 / 3 / 0 ★ 3	2181
50	Kadane's Algorithm	0 / 5 / 0 ★ 5	2189
51	Matrix (2D Array)	0 / 1 / 0 ★ 1	2202
52	Linked List	0 / 3 / 0 ★ 3	2206
53	Stacks	1 / 3 / 1 ★ 4	2217
54	Monotonic Stack	1 / 3 / 2 ★ 5	2234
55	Queues	1 / 1 / 0 ★ 1	2251
56	Monotonic Queue	0 / 2 / 1 ★ 3	2257
57	Bucket Sort	0 / 0 / 1 ★ 1	2267
58	Recursion	0 / 0 / 2 ★ 2	2271
59	Divide and Conquer	1 / 2 / 0 ★ 2	2278
60	Binary Search	4 / 4 / 1 ★ 5	2293
61	Backtracking	0 / 0 / 1 ★ 1	2318
62	Tree Traversal - Level Order	0 / 1 / 0 ★ 1	2322
63	Tree Traversal - Pre Order	3 / 4 / 0 ★ 4	2327
64	Tree Traversal - In Order	1 / 0 / 0 (no Medium / Hard)	2352
65	Tree Traversal - Post-Order	0 / 4 / 0 ★ 4	2356
66	BST / Ordered Set	0 / 4 / 0 ★ 4	2371
67	Tries	0 / 3 / 1 ★ 4	2384
68	Heaps	0 / 1 / 0 ★ 1	2397
69	Intervals	2 / 3 / 1 ★ 4	2401
70	Data Structure Design	1 / 2 / 6 ★ 7	2418
71	Greedy	2 / 4 / 2 ★ 6	2453
72	Depth First Search (DFS)	0 / 2 / 2 ★ 4	2477
73	Breadth First Search (BFS)	0 / 3 / 1 ★ 4	2494
74	Topological Sort	0 / 0 / 1 ★ 1	2511
75	Union Find	0 / 3 / 1 ★ 4	2515
76	Minimum Spanning Tree	0 / 2 / 3 ★ 5	2532
77	Shortest Path	0 / 6 / 3 ★ 7	2553
78	Eulerian Circuit	0 / 0 / 3 ★ 3	2592
79	1-D DP	2 / 3 / 1 ★ 4	2601

PART 2 · Extra topics and more practice			2076
#	Topic	E / M / H ★ top	Page
80	Longest Increasing Subsequence (LIS)	0 / 2 / 0 ★ 2	2621
81	2D Grid DP	0 / 2 / 8 ★ 7	2628
82	String DP	0 / 2 / 3 ★ 5	2662
83	Tree / Graph DP	0 / 1 / 2 ★ 3	2678
84	Bitmask DP	0 / 3 / 1 ★ 4	2692
85	Digit DP	0 / 1 / 2 ★ 3	2707
86	Probability DP	0 / 3 / 0 ★ 3	2717
87	State Machine DP	0 / 1 / 2 ★ 3	2728
88	Maths / Geometry	0 / 5 / 1 ★ 6	2740
89	String Matching	0 / 2 / 1 ★ 3	2758
90	Binary Indexed Tree / Segment Tree	0 / 1 / 2 ★ 3	2767
91	Line Sweep	0 / 0 / 1 ★ 1	2782

PART 1

The 41 Patterns

The high-ROI patterns, with every problem worked in full

- 41 pattern chapters, 632 different problems (each printed once, in the first pattern that lists it; a problem that belongs to two patterns is a link in the second chapter).
- Each chapter starts with a one-page note: when to use the pattern, how to spot it, and a small template to keep in your hands.
- Every problem has its own page: full statement, examples and figures, the Python 3 solutions with the reasoning, and the follow-up code an interviewer may ask next.
- Problems go from Easy to Hard inside a chapter. Use Prev / Contents / Next at the bottom of every page.
- Want more practice after these? Part 2 covers the extra topics and the problems that are not in these 41 chapters.

PATTERN 1 OF 41

Hash Map / Hash Set

20 problems · 8 Easy · 10 Medium · 2 Hard

THE IDEA

You need to know whether you have seen something before, how many times, or where. Trade memory for time: one pass that stores what you saw turns an O(n^2) search into O(n).

Signals: pairs that add up to a target, duplicates, frequency counts, grouping by a key, first unique item, prefix totals stored in a dictionary.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
seen = {}                       # value -> index (or count)
for i, x in enumerate(nums):
    need = target - x           # what would complete the answer?
    if need in seen:
        return [seen[need], i]
    seen[x] = i
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Two Sum	Easy	LC 1	8
2	Contains Duplicate	Easy	LC 217	11
3	Valid Anagram	Easy	LC 242	14
4	Isomorphic Strings	Easy	LC 205	17
5	Word Pattern	Easy	LC 290	20
6	Ransom Note	Easy	LC 383	23
7	Longest Palindrome	Easy	LC 409	25
8	Number of Equivalent Domino Pairs	Easy	LC 1128	28
9	★ **Group Anagrams**	Medium	LC 49	31
10	★ **Top K Frequent Elements**	Medium	LC 347	34
11	★ **Longest Consecutive Sequence**	Medium	LC 128	37
12	★ **Subarray Sum Equals K**	Medium	LC 560	40
13	Find the Duplicate Number	Medium	LC 287	43
14	★ **Four Sum II**	Medium	LC 454	46
15	Brick Wall	Medium	LC 554	49
16	Check If a String Contains All Binary Codes of Size K	Medium	LC 1461	52
17	Continuous Subarray Sum	Medium	LC 523	55
18	★ **Insert Delete GetRandom O(1)**	Medium	LC 380	58
19	Subarrays with K Different Integers	Hard	LC 992	62
20	★ **First Missing Positive**	Hard	LC 41	65

PATTERN 1 · HASH MAP / HASH SET · PROBLEM 1 OF 20

1. Two Sum

Easy · Array, Hash Table

leetcode.com/problems/two-sum

PROBLEM

You are given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to* `target`.

You may assume that each input would have ***exactly* one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

```
Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].
```

Example 2:

```
Input: nums = [3,2,4], target = 6
Output: [1,2]
```

Example 3:

```
Input: nums = [3,3], target = 6
Output: [0,1]
```

Constraints:

- `2 <= nums.length <= 10^4`
- `-10^9 <= nums[i] <= 10^9`
- `-10^9 <= target <= 10^9`
- **Only one valid answer exists.**

Follow-up:

Can you come up with an algorithm that is less than

$O(n^2)$

time complexity?

SOLUTION (PYTHON 3)

Solution 1: Hash Table

How to think about it

The first idea is a nested loop: pick x, then scan for target-x. Correct, but $O(n^2)$. $n \le 10^4$ might pass, yet the follow-up asks for faster. The bottleneck is repeatedly searching for the complement among numbers already seen. A hash table makes that lookup expected O(1). Walk once, storing value → index: when we see x, ask whether target-x already appeared; if yes, return the two indices; if not, store x. Look up before inserting. Insert-then-lookup would reuse the same index on [3,3], target=6. Lookup-first guarantees the match is an earlier index.

We can use a hash table d to store each element and its corresponding index.

Traverse the array nums, for the current element nums[i], we first check if target - nums[i] is in the hash table d. If it is in d, it means the target value has been found, and we return the indices of target - nums[i] and i.

Time complexity is O(n), and space complexity is O(n), where n is the length of the array nums.

Python 3

```python
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        d = {}
        for i, x in enumerate(nums):
            if (y := target - x) in d:
                return [d[y], i]
            d[x] = i
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The array is already sorted
# 2. Return every pair of values that adds up to the
# target, without repeats
# 3. Numbers arrive one at a time and you must answer 'is
# there a pair that adds up to V?'


# ============================================================
# FOLLOW-UP 1: The array is already sorted
# ============================================================

# ---- follow-up 1: function ----
# Answer: Two pointers, one at each end, in O(n) time and
# O(1) space: a sum that is too small moves the left
# pointer up, a sum that is too big moves the right
# pointer down.
# Time: O(n).  Space: O(1).
def two_sum_sorted(numbers, target):
    left, right = 0, len(numbers) - 1
    while left < right:
        total = numbers[left] + numbers[right]
        if total == target:
            return [left, right]
        if total < target:
            left += 1
        else:
            right -= 1
    return []


# ============================================================
# FOLLOW-UP 2: Return every pair of values that adds up to
# the target, without repeats
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count each value, then for every value x look
# for target - x. Take a pair only once (x < y), and a
# pair of equal values only when that value appears twice.
# Time: O(n).  Space: O(n).
def all_pairs(nums, target):
    count = {}
    for x in nums:
        count[x] = count.get(x, 0) + 1
    pairs = []
    for x in count:
        y = target - x
        if y in count and x < y:
            pairs.append((x, y))
        elif x == y and count[x] >= 2:
            pairs.append((x, x))
    return pairs
```

```
# ============================================================
# FOLLOW-UP 3: Numbers arrive one at a time and you must
# answer 'is there a pair that adds up to V?'
# ============================================================
```

```
# ---- follow-up 3: function ----
# Answer: Keep a dictionary of how many times each number
# was added. add() is O(1); find() looks at each stored
# number once, O(n).
# Time: add O(1), find O(n).  Space: O(n).
class TwoSumStream:
    def __init__(self):
        self.count = {}

    def add(self, number):
        self.count[number] = self.count.get(number, 0) + 1

    def find(self, value):
        for x in self.count:
            y = value - x
            if y != x and y in self.count:
                return True
            if y == x and self.count[x] > 1:
                return True
        return False
```

PATTERN 1 · HASH MAP / HASH SET · PROBLEM 2 OF 20

217. Contains Duplicate

Easy · Array, Hash Table, Sorting

leetcode.com/problems/contains-duplicate

PROBLEM

Given an integer array `nums`, return `true` if any value appears **at least twice** in the array, and return `false` if every element is distinct.

Example 1:

Input: nums = [1,2,3,1]

Output: true

Explanation:

The element 1 occurs at the indices 0 and 3.

Example 2:

Input: nums = [1,2,3,4]

Output: false

Explanation:

All elements are distinct.

Example 3:

Input: nums = [1,1,1,3,3,4,3,2,4,2]

Output: true

Constraints:

- `1 <= nums.length <= 10^5`
- `-10^9 <= nums[i] <= 10^9`

SOLUTIONS (PYTHON 3)

Solution 1: Sorting

> **How to think about it**
>
> Comparing every pair works but is slow for large n. Equal values become adjacent after sorting. Sort the array, then check whether any two neighbors are equal.

First, we sort the array `nums`.

Then, we traverse the array. If there are two adjacent elements that are the same, it means that there are duplicate elements in the array, and we directly return `true`.

Otherwise, when the traversal ends, we return `false`.

The time complexity is O(n × log n), where n is the length of the array `nums`.

Python 3

```python
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        return any(a == b for a, b in pairwise(sorted(nums)))
```

Solution 2: Hash Table

> **How to think about it**
>
> Sorting is O(nlog n) and reorders the array. Membership alone can be tracked with a hash set in one pass. A value that is already in the set is a duplicate.

We traverse the array and record the elements that have appeared in the hash table s. If an element appears for the second time, it means that there are duplicate elements in the array, and we directly return `true`.

The time complexity is O(n), and the space complexity is O(n), where n is the length of the array `nums`.

Python 3

```
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        return len(set(nums)) < len(nums)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The two equal numbers must be at most k positions
# apart
# 2. The numbers only have to be close (difference at most
# t) and at most k positions apart
# 3. Use O(1) extra space


# ============================================================
# FOLLOW-UP 1: The two equal numbers must be at most k
# positions apart
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember the last index of every value; a repeat
# only counts when the distance to its last index is at
# most k.
# Time: O(n).  Space: O(n).
def contains_nearby_duplicate(nums, k):
    last_seen = {}
    for i in range(len(nums)):
        x = nums[i]
        if x in last_seen and i - last_seen[x] <= k:
            return True
        last_seen[x] = i
    return False


# ============================================================
# FOLLOW-UP 2: The numbers only have to be close
# (difference at most t) and at most k positions apart
# ============================================================

# ---- follow-up 2: function ----
# Answer: Put every number into a bucket of width t + 1:
# two numbers in the same bucket are always close enough,
# and only the two neighbouring buckets can hold another
# close number. Keep only the last k numbers in the
# buckets.
# Time: O(n).  Space: O(min(n, k)).
def contains_nearby_almost_duplicate(nums, index_diff, value_diff):
    width = value_diff + 1
    buckets = {}                    # bucket number -> the one number in it
    for i in range(len(nums)):
        x = nums[i]
        b = x // width
        if b in buckets:
```

```
            return True
        if b - 1 in buckets and x - buckets[b - 1] <= value_diff:
            return True
        if b + 1 in buckets and buckets[b + 1] - x <= value_diff:
            return True
        buckets[b] = x
        if i >= index_diff:          # the oldest number leaves the window
            del buckets[nums[i - index_diff] // width]
    return False


# ============================================================
# FOLLOW-UP 3: Use O(1) extra space
```

```
# ============================================================

# ---- follow-up 3: function ----
# Answer: Sort the array in place; equal numbers become
# neighbours. O(n log n) time, and the input order is
# lost.
# Time: O(n log n).  Space: O(1) extra.
def contains_duplicate_no_extra_space(nums):
    nums.sort()
    for i in range(1, len(nums)):
        if nums[i] == nums[i - 1]:
            return True
    return False
```

PATTERN 1 · HASH MAP / HASH SET · PROBLEM 3 OF 20

242. Valid Anagram

Easy · Hash Table, String, Sorting

leetcode.com/problems/valid-anagram

Also appears in pattern 7 String Matching / Pattern Search

PROBLEM

Given two strings `s` and `t`, return `true` if `t` is an anagram of `s`, and `false` otherwise.

Example 1:

Input: s = "anagram", t = "nagaram"

Output: true

Example 2:

Input: s = "rat", t = "car"

Output: false

Constraints:

- `1 <= s.length, t.length <= 5 * 10^4`
- `s` and `t` consist of lowercase English letters.

Follow up: What if the inputs contain Unicode characters? How would you adapt your solution to such a case?

SOLUTIONS (PYTHON 3)

Solution 1: Counting

> **How to think about it**
>
> Anagrams have identical character counts. Unequal lengths fail immediately. Count s, then decrement while scanning t; a negative count means they differ.

We first determine whether the length of the two strings is equal. If they are not equal, the characters in the two strings must be different, so return `false`.

Otherwise, we use a hash table or an array of length 26 to record the number of times each character appears in the string s, and then traverse the other string t. Each time we traverse a character, we subtract the number of times the corresponding character appears in the hash table by one. If the number of times after subtraction is less than 0, the number of times the character appears in the two strings is different, return `false`. If after traversing the two strings, all the character counts in the hash table are 0, it means that the characters in the two strings appear the same number of times, return `true`.

The time complexity is O(n), the space complexity is O(C), where n is the length of the string; and C is the size of the character set, which is C=26 in this problem.

Python 3

```python
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False
        cnt = Counter(s)
        for c in t:
            cnt[c] -= 1
            if cnt[c] < 0:
                return False
        return True
```

Solution 2

> **How to think about it**
> The first method maintains the table by hand. Comparing two counters is shorter and has the same complexity.

Python 3

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        return Counter(s) == Counter(t)
```

Complexity. Time: O(n). Space: O(C), C = 26 letters.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The strings can contain any Unicode characters
# 2. Minimum number of letters to replace in t to make it
# an anagram of s
# 3. Every start position in a long text where an anagram
# of a pattern begins


# ============================================================
# FOLLOW-UP 1: The strings can contain any Unicode
# characters
# ============================================================

# ---- follow-up 1: function ----
# Answer: A 26-slot array no longer works; count with a
# dictionary keyed by the character.
# Time: O(n).  Space: O(s), s = distinct characters.
def is_anagram_any_characters(s, t):
    if len(s) != len(t):
        return False
    count = {}
    for ch in s:
        count[ch] = count.get(ch, 0) + 1
    for ch in t:
        if count.get(ch, 0) == 0:
            return False
        count[ch] -= 1
    return True


# ============================================================
# FOLLOW-UP 2: Minimum number of letters to replace in t
# to make it an anagram of s
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the letters of s minus the letters of t;
# the surplus of s is the number of letters of t that must
# be replaced. Both strings have the same length.
# Time: O(n).  Space: O(s).
def min_steps_to_anagram(s, t):
    count = {}
    for ch in s:
        count[ch] = count.get(ch, 0) + 1
    for ch in t:
        count[ch] = count.get(ch, 0) - 1
    steps = 0
    for value in count.values():
        if value > 0:
            steps += value
    return steps
```

```
# ============================================================
# FOLLOW-UP 3: Every start position in a long text where
# an anagram of a pattern begins
# ============================================================

# ---- follow-up 3: function ----
# Answer: Slide a window the size of the pattern over the
```

```
# text, adding the new letter and removing the old one,
# and compare its counts with the pattern's counts.
# Time: O(n * s).  Space: O(s).
def find_anagram_starts(text, pattern):
    need = {}
    for ch in pattern:
        need[ch] = need.get(ch, 0) + 1
    window = {}
    size = len(pattern)
    starts = []
    for i in range(len(text)):
        window[text[i]] = window.get(text[i], 0) + 1
        if i >= size:                              # the letter that left the window
            old = text[i - size]
            window[old] -= 1
            if window[old] == 0:
                del window[old]
        if window == need:
            starts.append(i - size + 1)
    return starts
```

205. Isomorphic Strings

Easy · Hash Table, String

leetcode.com/problems/isomorphic-strings

PROBLEM

Given two strings `s` and `t`, *determine if they are isomorphic*.

Two strings `s` and `t` are isomorphic if the characters in `s` can be replaced to get `t`.

All occurrences of a character must be replaced with another character while preserving the order of characters. No two characters may map to the same character, but a character may map to itself.

Example 1:

Input: s = "egg", t = "add"

Output: true

Explanation:

The strings `s` and `t` can be made identical by:

- Mapping `'e'` to `'a'`.
- Mapping `'g'` to `'d'`.

Example 2:

Input: s = "f11", t = "b23"

Output: false

Explanation:

The strings `s` and `t` can not be made identical as `'1'` needs to be mapped to both `'2'` and `'3'`.

Example 3:

Input: s = "paper", t = "title"

Output: true

Constraints:

- `1 <= s.length <= 5 * 10^4`
- `t.length == s.length`
- `s` and `t` consist of any valid ascii character.

SOLUTIONS (PYTHON 3)

Solution 1: Hash Table or Array

> **How to think about it**
>
> Isomorphism needs a bijection between characters of s and t; a one-way map misses many-to-one collisions. The strings have equal length and a finite alphabet, so one pass suffices. We keep d_1 and d_2 for s→ t and t→ s. A conflict with an existing mapping fails; otherwise we update and continue.

We can use two hash tables or arrays d_1 and d_2 to record the character mapping relationship between s and t.

Traverse s and t, if the corresponding character mapping relationships in d_1 and d_2 are different, return `false`, otherwise update the corresponding character mapping relationships in d_1 and d_2. After the traversal is complete, it means that s and t are isomorphic, and return `true`.

The time complexity is O(n) and the space complexity is O(C). Where n is the length of the string s; and C is the size of the character set, which is C = 256 in this problem.

Python 3

```python
class Solution:
    def isIsomorphic(self, s: str, t: str) -> bool:
        d1 = {}
        d2 = {}
        for a, b in zip(s, t):
            if (a in d1 and d1[a] != b) or (b in d2 and d2[b] != a):
                return False
            d1[a] = b
            d2[b] = a
        return True
```

Solution 2

> **How to think about it**
>
> Two hash maps are correct but pay hashing overhead. The alphabet has size 256, so arrays suffice. Store the last index of each character as a shared timestamp: if the recorded indices differ, the mapping is inconsistent.

Python 3

```python
class Solution:
    def isIsomorphic(self, s: str, t: str) -> bool:
        d1, d2 = [0] * 256, [0] * 256
        for i, (a, b) in enumerate(zip(s, t), 1):
            a, b = ord(a), ord(b)
            if d1[a] != d2[b]:
                return False
            d1[a] = d2[b] = i
        return True
```

Complexity. Time: O(n). Space: O(C), C = 256 characters.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the actual letter mapping, or say there is
# none
# 2. Group a list of words so that every group has the
# same letter pattern
# 3. Which words in a list match a given pattern word?


# ============================================================
# FOLLOW-UP 1: Return the actual letter mapping, or say
# there is none
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep one dictionary for each direction and check
# both before adding a pair: a letter may map to only one
# letter, and two letters may not map to the same one.
# Time: O(n).  Space: O(s), s = distinct characters.
def isomorphic_mapping(s, t):
    forward = {}
    backward = {}
    for i in range(len(s)):
        a, b = s[i], t[i]
        if a in forward and forward[a] != b:
            return None
        if b in backward and backward[b] != a:
            return None
        forward[a] = b
        backward[b] = a
    return forward
```

```
# ============================================================
# FOLLOW-UP 2: Group a list of words so that every group
# has the same letter pattern
# ============================================================

# ---- follow-up 2: helper functions ----
def shape(word):
    first_seen = {}
    code = []
    for ch in word:
        if ch not in first_seen:
            first_seen[ch] = len(first_seen)
        code.append(first_seen[ch])
    return tuple(code)

# ---- follow-up 2: function ----
# Answer: Give each word a signature: replace every letter
# by the order in which it first appeared (abb and xzz
# both become 0, 1, 1). Words with the same signature are
# isomorphic.
# Time: O(N * L), N words of length L.  Space: O(N * L).
def group_by_shape(words):
    groups = {}
    for word in words:
        key = shape(word)
        if key not in groups:
            groups[key] = []
        groups[key].append(word)
    return list(groups.values())


# ============================================================
# FOLLOW-UP 3: Which words in a list match a given pattern
# word?
# ============================================================

# ---- follow-up 3: function ----
# Answer: Compute the signature of the pattern once and
# keep the words with the same signature.
# Time: O(N * L).  Space: O(L).
def find_and_replace_pattern(words, pattern):
    target = shape(pattern)
    result = []
    for word in words:
        if shape(word) == target:
            result.append(word)
    return result
```

PATTERN 1 · HASH MAP / HASH SET · PROBLEM 5 OF 20

290. Word Pattern

Easy · Hash Table, String

leetcode.com/problems/word-pattern

PROBLEM

Given a `pattern` and a string `s`, find if `s` follows the same pattern.

Here **follow** means a full match, such that there is a bijection between a letter in `pattern` and a **non-empty** word in `s`. Specifically:

- Each letter in `pattern` maps to **exactly** one unique word in `s`.
- Each unique word in `s` maps to **exactly** one letter in `pattern`.
- No two letters map to the same word, and no two words map to the same letter.

Example 1:

Input: pattern = "abba", s = "dog cat cat dog"

Output: true

Explanation:

The bijection can be established as:

- `'a'` maps to `"dog"`.
- `'b'` maps to `"cat"`.

Example 2:

Input: pattern = "abba", s = "dog cat cat fish"

Output: false

Example 3:

Input: pattern = "aaaa", s = "dog cat cat dog"

Output: false

Constraints:

- `1 <= pattern.length <= 300`
- `pattern` contains only lower-case English letters.
- `1 <= s.length <= 3000`
- `s` contains only lowercase English letters and spaces `' '`.
- `s` **does not contain** any leading or trailing spaces.
- All the words in `s` are separated by a **single space**.

SOLUTIONS (PYTHON 3)

Solution 1: Hash Table

> **How to think about it**
>
> The pattern and the words must be bijections; unequal lengths fail. As with isomorphic strings, two maps record char-to-word and word-to-char. A conflict with an existing mapping fails.

First, we split the string s into a word array ws with spaces. If the length of pattern and ws is not equal, return `false` directly. Otherwise, we use two hash tables d_1 and d_2 to record the correspondence between each character and word in pattern and ws.

Then, we traverse pattern and ws. For each character a and word b, if there is a mapping for a in d_1, and the mapped word is not b, or there is a mapping for b in d_2, and the mapped character is not a, return `false`. Otherwise, we add the mapping of a and b to d_1 and d_2 respectively.

After the traversal, return `true`.

The time complexity is O(m + n) and the space complexity is O(m + n). Here m and n are the length of pattern and string s.

Python 3

```
class Solution:
    def wordPattern(self, pattern: str, s: str) -> bool:
        ws = s.split()
        if len(pattern) != len(ws):
            return False
        d1 = {}
        d2 = {}
        for a, b in zip(pattern, ws):
            if (a in d1 and d1[a] != b) or (b in d2 and d2[b] != a):
                return False
            d1[a] = b
            d2[b] = a
        return True
```

Solution 2

> **How to think about it**
>
> The two maps can be replaced by checking that the number of distinct pattern characters equals the number of distinct words, then keeping a one-way map.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```
class Solution:
    def wordPattern(self, pattern: str, s: str) -> bool:
        words = s.split()
        if len(pattern) != len(words) or len(set(pattern)) != len(set(words)):
            return False
        table = {}
        for letter, word in zip(pattern, words):
            if table.setdefault(letter, word) != word:
                return False
        return True
```

Complexity. Time: O(n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Compare first-occurrence positions instead of keeping
# two maps
# 2. The text has no spaces: the words must be found as
# well (Word Pattern II)


# ============================================================
# FOLLOW-UP 1: Compare first-occurrence positions instead
# of keeping two maps
# ============================================================

# ---- follow-up 1: function ----
# Answer: A letter and a word belong together exactly when
# they first appeared at the same position, so one
# comparison per position is enough.
# Time: O(n).  Space: O(n).
def word_pattern_by_first_position(pattern, s):
```

```
    words = s.split()
    if len(words) != len(pattern):
        return False
    first_letter = {}
    first_word = {}
    for i in range(len(words)):
        if first_letter.setdefault(pattern[i], i) != first_word.setdefault(words[i], i):
            return False
    return True


# ============================================================
# FOLLOW-UP 2: The text has no spaces: the words must be
# found as well (Word Pattern II)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Backtracking: for the next pattern letter either
# reuse its word (the text must continue with it) or try
# every possible new word, making sure no other letter
# already uses that word.
# Time: exponential: about O(len(s)^len(pattern)) in the
# worst case.  Space: O(len(s) + len(pattern)).
def word_pattern_match(pattern, s):
    word_of = {}
    used = set()

    def go(p, i):
        if p == len(pattern):
            return i == len(s)
        letter = pattern[p]
        if letter in word_of:
            word = word_of[letter]
            if not s.startswith(word, i):
                return False
            return go(p + 1, i + len(word))
        for end in range(i + 1, len(s) + 1):
            word = s[i:end]
            if word in used:
                continue
            word_of[letter] = word
            used.add(word)
```

```
            if go(p + 1, end):
                return True
            del word_of[letter]                     # undo and try a longer word
            used.discard(word)
        return False

    return go(0, 0)
```

PATTERN 1 · HASH MAP / HASH SET · PROBLEM 6 OF 20

383. Ransom Note

Easy · Hash Table, String, Counting

leetcode.com/problems/ransom-note

PROBLEM

Given two strings `ransomNote` and `magazine`, return `true` *if* `ransomNote` *can be constructed by using the letters from* `magazine` *and* `false` *otherwise*.

Each letter in `magazine` can only be used once in `ransomNote`.

Example 1:

```
Input: ransomNote = "a", magazine = "b"
Output: false
```

Example 2:

```
Input: ransomNote = "aa", magazine = "ab"
Output: false
```

Example 3:

```
Input: ransomNote = "aa", magazine = "aab"
Output: true
```

Constraints:

- `1 <= ransomNote.length, magazine.length <= 10^5`
- `ransomNote` and `magazine` consist of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Hash Table or Array

> **How to think about it**
>
> Can `ransomNote` be formed from `magazine` letters? A multiset inclusion. Count magazine frequencies, then decrement along the note; a negative count means shortage. The alphabet has size 26.

We can use a hash table or an array cnt of length 26 to record the number of times each character appears in the string `magazine`. Then traverse the string `ransomNote`, for each character c in it, we decrease the number of c by 1 in cnt. If the number of c is less than 0 after the decrease, it means that the number of c in `magazine` is not enough, so it cannot be composed of `ransomNote`, just return false.

Otherwise, after the traversal, it means that each character in `ransomNote` can be found in `magazine`. Therefore, return true.

The time complexity is O(m + n), and the space complexity is O(C). Where m and n are the lengths of the strings `ransomNote` and `magazine` respectively; and C is the size of the character set, which is 26 in this question.

Python 3

```python
class Solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        cnt = Counter(magazine)
        for c in ransomNote:
            cnt[c] -= 1
            if cnt[c] < 0:
                return False
        return True
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The magazine is huge and arrives as a stream: you
# cannot store it
# 2. How many whole copies of the note can be cut out of
# the magazine?


# ============================================================
# FOLLOW-UP 1: The magazine is huge and arrives as a
# stream: you cannot store it
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count only the letters the note needs, read the
# magazine once, and stop as soon as every needed letter
# has been found.
# Time: O(r + m), r = note length, m = magazine length.
# Space: O(s), s = distinct letters.
def can_construct_streaming(ransom_note, magazine_letters):
    need = {}
    missing = 0
    for ch in ransom_note:
        need[ch] = need.get(ch, 0) + 1
        missing += 1
    if missing == 0:
        return True
    for ch in magazine_letters:                  # any iterable, read once
        if need.get(ch, 0) > 0:
            need[ch] -= 1
            missing -= 1
            if missing == 0:
                return True
    return False


# ============================================================
# FOLLOW-UP 2: How many whole copies of the note can be
# cut out of the magazine?
# ============================================================

# ---- follow-up 2: function ----
# Answer: For every letter the note needs, divide how many
# the magazine has by how many one note uses; the smallest
# result is the answer.
# Time: O(r + m).  Space: O(s).
def copies_of_note(note, magazine):
    need = {}
    for ch in note:
        need[ch] = need.get(ch, 0) + 1
    have = {}
    for ch in magazine:
        have[ch] = have.get(ch, 0) + 1
    fewest = None
    for ch in need:
        copies = have.get(ch, 0) // need[ch]
        if fewest is None or copies < fewest:
            fewest = copies
    if fewest is None:
        return 0                                 # an empty note: nothing to count
    return fewest
```

PATTERN 1 · HASH MAP / HASH SET · PROBLEM 7 OF 20

409. Longest Palindrome

Easy · Greedy, Hash Table, String

leetcode.com/problems/longest-palindrome

PROBLEM

Given a string `s` which consists of lowercase or uppercase letters, return the length of the **longest palindrome** that can be built with those letters.

Letters are **case sensitive**, for example, `"Aa"` is not considered a palindrome.

Example 1:

```
Input: s = "abccccdd"
Output: 7
Explanation: One longest palindrome that can be built is "dccaccd", whose length is 7.
```

Example 2:

```
Input: s = "a"
Output: 1
Explanation: The longest palindrome that can be built is "a", whose length is 1.
```

Constraints:

- `1 <= s.length <= 2000`
- `s` consists of lowercase **and/or** uppercase English letters only.

SOLUTIONS (PYTHON 3)

Solution 1: Counting

How to think about it

A palindrome allows at most one odd-count character; the rest must pair. We do not need to try permutations, only how many pairs each character can give. After counting, add v//2× 2 for every frequency v. If the total is still shorter than |s|, one leftover character can sit in the center. Taking pairs first, then the optional center, yields a maximum feasible length.

A valid palindrome string can have at most one character that appears an odd number of times, and the rest of the characters appear an even number of times.

Therefore, we can first traverse the string s, count the number of occurrences of each character, and record it in an array or hash table cnt.

Then, we traverse cnt, for each count v, we divide v by 2, take the integer part, multiply by 2, and add it to the answer ans.

Finally, if the answer is less than the length of the string s, we increment the answer by one and return ans.

The time complexity is O(n + |Σ|), and the space complexity is O(|Σ|). Where n is the length of the string s, and |Σ| is the size of the character set. In this problem, |Σ| = 128.

Python 3

```python
class Solution:
    def longestPalindrome(self, s: str) -> int:
        cnt = Counter(s)
        ans = sum(v // 2 * 2 for v in cnt.values())
        ans += int(ans < len(s))
        return ans
```

Solution 2: Bit Manipulation + Counting

> **How to think about it**
>
> Solution 1 counts, then sums. XORing a parity flag while scanning and tracking the number of odd characters cnt gives n-cnt+1 (keep one center) or n. The second pass over the frequency table disappears; space is still the alphabet size.

We can use an array or hash table odd to record whether each character in string s appears an odd number of times, and an integer variable cnt to record the number of characters that appear an odd number of times.

We iterate through the string s. For each character c, we flip odd[c], i.e., 0 → 1, 1 → 0. If odd[c] changes from 0 to 1, then we increment cnt by one; if odd[c] changes from 1 to 0, then we decrement cnt by one.

Finally, if cnt is greater than 0, the answer is n - cnt + 1, otherwise, the answer is n.

The time complexity is O(n), and the space complexity is O(|Σ|). Where n is the length of the string s, and |Σ| is the size of the character set. In this problem, |Σ| = 128.

Python 3

```python
class Solution:
    def longestPalindrome(self, s: str) -> int:
        odd = defaultdict(int)
        cnt = 0
        for c in s:
            odd[c] ^= 1
            cnt += 1 if odd[c] else -1
        return len(s) - cnt + 1 if cnt else len(s)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the palindrome itself, not only its length
# 2. Can the letters be rearranged into a palindrome at
# all?
# 3. The letters must keep their order: longest
# palindromic subsequence


# ============================================================
# FOLLOW-UP 1: Return the palindrome itself, not only its
# length
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use every pair of equal letters: half of each
# pair goes on the left, the mirror image on the right,
# and one letter with an odd leftover can sit in the
# middle.
# Time: O(n).  Space: O(n).
def build_longest_palindrome(s):
    count = {}
    for ch in s:
        count[ch] = count.get(ch, 0) + 1
    half = ""
    middle = ""
    for ch in count:
        half += ch * (count[ch] // 2)
        if count[ch] % 2 == 1 and middle == "":
            middle = ch
    return half + middle + half[::-1]


# ============================================================
# FOLLOW-UP 2: Can the letters be rearranged into a
# palindrome at all?
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Only if at most one letter appears an odd number
# of times.
# Time: O(n).  Space: O(s), s = distinct characters.
def can_form_palindrome(s):
    count = {}
    for ch in s:
        count[ch] = count.get(ch, 0) + 1
    odd = 0
    for value in count.values():
        if value % 2 == 1:
            odd += 1
    return odd <= 1


# ============================================================
# FOLLOW-UP 3: The letters must keep their order: longest
# palindromic subsequence
# ============================================================

# ---- follow-up 3: function ----
# Answer: No longer a counting problem: dp[i][j] is the
# best length inside s[i..j]. Equal ends add 2 to the
```

```
# inside answer; otherwise drop one end.
# Time: O(n^2).  Space: O(n^2).
def longest_palindromic_subsequence(s):
    n = len(s)
    if n == 0:
        return 0
    dp = []
    for i in range(n):
        dp.append([0] * n)
    for i in range(n - 1, -1, -1):
        dp[i][i] = 1
        for j in range(i + 1, n):
            if s[i] == s[j]:
                dp[i][j] = dp[i + 1][j - 1] + 2
            elif dp[i + 1][j] >= dp[i][j - 1]:
                dp[i][j] = dp[i + 1][j]
            else:
                dp[i][j] = dp[i][j - 1]
    return dp[0][n - 1]
```

1128. Number of Equivalent Domino Pairs

Easy · Array, Hash Table, Counting

leetcode.com/problems/number-of-equivalent-domino-pairs

PROBLEM

Given a list of `dominoes`, `dominoes[i] = [a, b]` is **equivalent to** `dominoes[j] = [c, d]` if and only if either (`a == c` and `b == d`), or (`a == d` and `b == c`) - that is, one domino can be rotated to be equal to another domino.

Return *the number of pairs* `(i, j)` *for which* `0 <= i < j < dominoes.length`, *and* `dominoes[i]` ***is equivalent to*** `dominoes[j]`.

Example 1:

```
Input: dominoes = [[1,2],[2,1],[3,4],[5,6]]
Output: 1
```

Example 2:

```
Input: dominoes = [[1,2],[1,2],[1,1],[1,2],[2,2]]
Output: 3
```

Constraints:

- `1 <= dominoes.length <= 4 * 10^4`
- `dominoes[i].length == 2`
- `1 <= dominoes[i][j] <= 9`

SOLUTION (PYTHON 3)

Solution 1: Counting

> **How to think about it**
>
> Pairs treat [a,b] and [b,a] as the same. Scanning all earlier tiles for each new one is quadratic. Pack min(a,b) in the tens digit and max(a,b) in the ones, a key in 0..99. Add the current count of that key to the answer, then increment, so each tile pairs only with equivalents already seen.

We can concatenate the two numbers of each domino in order of size to form a two-digit number, so that equivalent dominoes can be concatenated into the same two-digit number. For example, both `[1, 2]` and `[2, 1]` are concatenated into the two-digit number `12`, and both `[3, 4]` and `[4, 3]` are concatenated into the two-digit number `34`.

Then we traverse all the dominoes, using an array cnt of length 100 to record the number of occurrences of each two-digit number. For each domino, the two-digit number we concatenate is x, then the answer will increase by cnt[x], and then we add 1 to the value of cnt[x]. Continue to traverse the next domino, and we can count the number of all equivalent domino pairs.

The time complexity is O(n), and the space complexity is O(C). Here, n is the number of dominoes, and C is the maximum number of two-digit numbers concatenated in the dominoes, which is 100.

Python 3

```python
class Solution:
    def numEquivDominoPairs(self, dominoes: List[List[int]]) -> int:
        cnt = Counter()
        ans = 0
        for a, b in dominoes:
            x = a * 10 + b if a < b else b * 10 + a
            ans += cnt[x]
            cnt[x] += 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the groups of equivalent dominoes, not the
# number of pairs
# 2. Dominoes arrive one at a time: report the number of
# pairs after each one
# 3. The numbers are single digits and speed matters:
# avoid tuples


# ============================================================
# FOLLOW-UP 1: Return the groups of equivalent dominoes,
# not the number of pairs
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use the same key (smaller number, bigger number)
# but keep the dominoes of each key in a list.
# Time: O(n).  Space: O(n).
def group_equivalent_dominoes(dominoes):
    groups = {}
    for a, b in dominoes:
        key = (a, b) if a <= b else (b, a)
        if key not in groups:
            groups[key] = []
        groups[key].append([a, b])
    return list(groups.values())


# ============================================================
# FOLLOW-UP 2: Dominoes arrive one at a time: report the
# number of pairs after each one
# ============================================================

# ---- follow-up 2: function ----
# Answer: A new domino pairs up with every earlier
# equivalent domino, so add the count of its key before
# increasing that count.
# Time: O(1) per domino.  Space: O(n).
class DominoCounter:
    def __init__(self):
        self.seen = {}
        self.pairs = 0

    def add(self, a, b):
        key = (a, b) if a <= b else (b, a)
        self.pairs += self.seen.get(key, 0)
        self.seen[key] = self.seen.get(key, 0) + 1
        return self.pairs


# ============================================================
# FOLLOW-UP 3: The numbers are single digits and speed
# matters: avoid tuples
# ============================================================

# ---- follow-up 3: function ----
# Answer: Encode the domino as one number, smaller * 10 +
# bigger, and count with an array of 100 slots. A group of
# c equal dominoes makes c * (c - 1) / 2 pairs.
# Time: O(n).  Space: O(1).
```

```
def num_equiv_domino_pairs_encoded(dominoes):
    count = [0] * 100
    for a, b in dominoes:
```

```
            if a > b:
                a, b = b, a
            count[a * 10 + b] += 1
        pairs = 0
        for c in count:
            pairs += c * (c - 1) // 2
        return pairs
```

PATTERN 1 · HASH MAP / HASH SET · PROBLEM 9 OF 20

49. Group Anagrams

Medium · Array, Hash Table, String, Sorting

leetcode.com/problems/group-anagrams

★ **TOP 7 PROBLEM** in Hash Map / Hash Set: master this one first

Also appears in pattern 6 String Manipulation / Parsing

PROBLEM

Given an array of strings `strs`, group the anagrams together. You can return the answer in **any order**.

Example 1:

Input: strs = ["eat","tea","tan","ate","nat","bat"]

Output: [["bat"],["nat","tan"],["ate","eat","tea"]]

Explanation:

- There is no string in strs that can be rearranged to form `"bat"`.
- The strings `"nat"` and `"tan"` are anagrams as they can be rearranged to form each other.
- The strings `"ate"`, `"eat"`, and `"tea"` are anagrams as they can be rearranged to form each other.

Example 2:

Input: strs = [""]

Output: [[""]]

Example 3:

Input: strs = ["a"]

Output: [["a"]]

Constraints:

- `1 <= strs.length <= 10^4`
- `0 <= strs[i].length <= 100`
- `strs[i]` consists of lowercase English letters.

SOLUTIONS (PYTHON 3)

Solution 1: Hash Table

How to think about it

The first idea is pairwise anagram checks: sort each string and compare. Correct, but $n \le 10^4$ and $k \le 100$ make $O(n^2 \cdot k \log k)$ too slow. Pairwise comparison is the bottleneck. Anagrams share one sorted form — that string is the group id. Use the sorted string as key and a list of originals as value. One pass into a hash table clusters them; no pairwise matching.

1. Traverse the string array, sort each string in **character dictionary order** to get a new string.
2. Use the new string as `key` and `[str]` as `value`, and store them in the hash table (`HashMap<String, List<String>>`).
3. When encountering the same `key` during subsequent traversal, add it to the corresponding `value`.

Take `strs = ["eat", "tea", "tan", "ate", "nat", "bat"]` as an example. At the end of the traversal, the state of the hash table is:

key	value
`"aet"`	`["eat", "tea", "ate"]`

key	value
"ant"	["tan", "nat"]
"abt"	["bat"]

Finally, return the `value` list of the hash table.

The time complexity is O(n× k× log k), where n and k are the lengths of the string array and the maximum length of the string, respectively.

Python 3

```
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        d = defaultdict(list)
        for s in strs:
            k = ''.join(sorted(s))
            d[k].append(s)
        return list(d.values())
```

Solution 2: Counting

> **How to think about it**
>
> Solution 1 sorts every string, O(k log k) each. The alphabet is 26 lowercase letters; with k ≤ 100 the log k factor is waste. What it lacks is a cheaper key. Count each letter and use the length-26 tuple as the key. Same hash grouping, linear per string.

We can also change the sorting part in Solution 1 to counting, that is, use the characters in each string s and their occurrence times as `key`, and use the string s as `value` to store in the hash table.

The time complexity is O(n× (k + C)), where n and k are the lengths of the string array and the maximum length of the string, respectively, and C is the size of the character set. In this problem, C = 26.

Python 3

```
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        d = defaultdict(list)
        for s in strs:
            cnt = [0] * 26
            for c in s:
                cnt[ord(c) - ord('a')] += 1
            d[tuple(cnt)].append(s)
        return list(d.values())
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Sorting every word is too slow for long words
# 2. Words arrive one at a time: return the group the new
# word joins
# 3. Find the largest group of anagrams


# ============================================================
# FOLLOW-UP 1: Sorting every word is too slow for long
# words
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use the 26 letter counts as the key instead of
# the sorted word: O(L) per word instead of O(L log L)
# (lowercase letters only).
# Time: O(N * L).  Space: O(N * L).
```

```
def group_anagrams_by_count(words):
    groups = {}
    for word in words:
        counts = [0] * 26
        for ch in word:
            counts[ord(ch) - ord("a")] += 1
        key = tuple(counts)
        if key not in groups:
            groups[key] = []
        groups[key].append(word)
    return list(groups.values())


# ============================================================
# FOLLOW-UP 2: Words arrive one at a time: return the
# group the new word joins
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the dictionary between calls and look the
# word up by its key.
# Time: O(L log L) per word.  Space: O(N * L).
class AnagramGroups:
    def __init__(self):
        self.groups = {}

    def add(self, word):
        key = "".join(sorted(word))
        if key not in self.groups:
            self.groups[key] = []
        self.groups[key].append(word)
        return self.groups[key]


# ============================================================
# FOLLOW-UP 3: Find the largest group of anagrams
# ============================================================

# ---- follow-up 3: function ----
# Answer: Group as usual and keep the biggest list.
# Time: O(N * L log L).  Space: O(N * L).
def largest_anagram_group(words):
    groups = {}
```

```
    for word in words:
        key = "".join(sorted(word))
        if key not in groups:
            groups[key] = []
        groups[key].append(word)
    best = []
    for group in groups.values():
        if len(group) > len(best):
            best = group
    return best
```

PATTERN 1 · HASH MAP / HASH SET · PROBLEM 10 OF 20

347. Top K Frequent Elements

Medium · Array, Hash Table, Divide and Conquer, Bucket Sort, Counting, Quickselect, Sorting, Heap (Priority Queue)

leetcode.com/problems/top-k-frequent-elements

★ **TOP 7 PROBLEM** in Hash Map / Hash Set, Heap / Priority Queue: master this one first

Also appears in pattern 19 Heap / Priority Queue, 38 K-way Merge

PROBLEM

Given an integer array `nums` and an integer `k`, return *the* `k` *most frequent elements*. You may return the answer in **any order**.

Example 1:

Input: nums = [1,1,1,2,2,3], k = 2

Output: [1,2]

Example 2:

Input: nums = [1], k = 1

Output: [1]

Example 3:

Input: nums = [1,2,1,2,1,2,3,1,3,2], k = 2

Output: [1,2]

Constraints:

- `1 <= nums.length <= 10^5`
- `-10^4 <= nums[i] <= 10^4`
- `k` is in the range `[1, the number of unique elements in the array]`.
- It is **guaranteed** that the answer is **unique**.

Follow up: Your algorithm's time complexity must be better than `O(n log n)`, where n is the array's size.

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Priority Queue (Min Heap)

How to think about it

The k most frequent values. Full sort is O(nlog n). Only the top k frequencies matter. Count, then select the k largest frequencies. `Counter.most_common(k)` is a heap selection in O(nlog k).

We can use a hash table cnt to count the occurrence of each element, and then use a min heap (priority queue) to store the top k frequent elements.

First, we traverse the array once to count the occurrence of each element. Then, we iterate through the hash table, storing each element and its count into the min heap. If the size of the min heap exceeds k, we pop the top element of the heap to ensure the heap size is always k.

Finally, we pop the elements from the min heap one by one and place them into the result array.

The time complexity is O(n log k), and the space complexity is O(k). Here, n is the length of the array.

Python 3

```
class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        cnt = Counter(nums)
        return [x for x, _ in cnt.most_common(k)]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Better than O(n log n): O(n)
# 2. Top K frequent WORDS, ties broken alphabetically
# 3. Numbers keep arriving: be able to answer 'top K now'
# at any moment


# ============================================================
# FOLLOW-UP 1: Better than O(n log n): O(n)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Bucket sort by frequency: a number can appear at
# most n times, so put each number in the bucket of its
# count and read the buckets from the highest count down.
# Time: O(n).  Space: O(n).
def top_k_frequent_bucket(nums, k):
    count = {}
    for x in nums:
        count[x] = count.get(x, 0) + 1
    buckets = []
    for i in range(len(nums) + 1):
        buckets.append([])
    for x in count:
        buckets[count[x]].append(x)
    result = []
    for frequency in range(len(nums), 0, -1):
        for x in buckets[frequency]:
            result.append(x)
            if len(result) == k:
                return result
    return result


# ============================================================
# FOLLOW-UP 2: Top K frequent WORDS, ties broken
# alphabetically
# ============================================================

# ---- follow-up 2: function ----
# Answer: Sort by (negative count, word): the highest
# count comes first and equal counts fall back to
# alphabetical order.
# Time: O(W + U log U), U = distinct words.  Space: O(U).
def top_k_frequent_words(words, k):
    count = {}
    for word in words:
        count[word] = count.get(word, 0) + 1
    pairs = []
    for word in count:
        pairs.append((-count[word], word))
    pairs.sort()
    result = []
    for i in range(min(k, len(pairs))):
        result.append(pairs[i][1])
    return result


# ============================================================
# FOLLOW-UP 3: Numbers keep arriving: be able to answer
```

```
# 'top K now' at any moment
# ============================================================
```

```python
# ---- follow-up 3: function ----
# Answer: Keep the counts between calls and take the K
# largest with a heap when asked: O(n log k) per question,
# O(1) per new number.
# Time: add O(1), top O(U log k).  Space: O(U).
import heapq


class TopKTracker:
    def __init__(self):
        self.count = {}

    def add(self, x):
        self.count[x] = self.count.get(x, 0) + 1

    def top(self, k):
        return heapq.nlargest(k, self.count, key=self.count.get)
```

128. Longest Consecutive Sequence

Medium · Union Find, Array, Hash Table

leetcode.com/problems/longest-consecutive-sequence

★ **TOP 7 PROBLEM** in Hash Map / Hash Set: master this one first

Also appears in pattern 31 Union-Find (Disjoint Set)

PROBLEM

Given an unsorted array of integers `nums`, return *the length of the longest consecutive elements sequence.*

You must write an algorithm that runs in `O(n)` time.

Example 1:

```
Input: nums = [100,4,200,1,3,2]
Output: 4
Explanation: The longest consecutive elements sequence is [1, 2, 3, 4]. Therefore its length is 4.
```

Example 2:

```
Input: nums = [0,3,7,2,5,8,4,6,0,1]
Output: 9
```

Example 3:

```
Input: nums = [1,0,1,2]
Output: 3
```

Constraints:

- `0 <= nums.length <= 10^5`
- `-10^9 <= nums[i] <= 10^9`

SOLUTIONS (PYTHON 3)

Solution 1: Hash Table

How to think about it

Sorting and scanning runs is O(nlog n); the problem asks for O(n). n ≤ 10^5. Put the values in a hash set. From each x still in the set, walk right until the run breaks, store that length at the start, and let earlier starts concatenate. Each number enters and leaves the set once.

We can use a hash table s to store all the elements in the array, a variable ans to record the length of the longest consecutive sequence, and a hash table d to record the length of the consecutive sequence each element x belongs to.

Next, we iterate through each element x in the array, using a temporary variable y to record the maximum value of the current consecutive sequence, initially y = x. Then, we continuously try to match y+1, y+2, y+3, … until we can no longer match. During this process, we remove the matched elements from the hash table s. The length of the consecutive sequence that the current element x belongs to is d[x] = d[y] + y - x, and then we update the answer ans = max(ans, d[x]).

After the iteration, we return the answer ans.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        s = set(nums)
        ans = 0
        d = defaultdict(int)
        for x in nums:
            y = x
            while y in s:
                s.remove(y)
                y += 1
            d[x] = d[y] + y - x
            ans = max(ans, d[x])
        return ans
```

Solution 2: Hash Table (Optimization)

> **How to think about it**
>
> Solution 1 stores each run's length so later starts can concatenate. If we only grow a run when x-1 is absent, x is already a start, so the extra map d is unnecessary. The code is shorter and still touches each number a constant number of times.

Similar to Solution 1, we use a hash table s to store all the elements in the array and a variable ans to record the length of the longest consecutive sequence. However, we no longer use a hash table d to record the length of the consecutive sequence each element x belongs to. During the iteration, we skip elements where x-1 is also in the hash table s. If x-1 is in the hash table s, then x is definitely not the start of a consecutive sequence, so we can directly skip x.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        s = set(nums)
        ans = 0
        for x in s:
            if x - 1 not in s:
                y = x + 1
                while y in s:
                    y += 1
                ans = max(ans, y - x)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the sequence itself, not its length
# 2. Numbers arrive one at a time: report the longest run
# after each one


# ============================================================
# FOLLOW-UP 1: Return the sequence itself, not its length
# ============================================================

# ---- follow-up 1: function ----
# Answer: Start only at numbers that have no predecessor
# in the set, walk upwards, and remember the start of the
# longest run.
# Time: O(n).  Space: O(n).
def longest_consecutive_run(nums):
    seen = set(nums)
    best_start = None
```

```
    best_length = 0
    for x in seen:
        if x - 1 not in seen:                    # x starts a run
            end = x
            while end + 1 in seen:
                end += 1
            if end - x + 1 > best_length:
                best_length = end - x + 1
                best_start = x
    if best_start is None:
        return []
    return list(range(best_start, best_start + best_length))


# ============================================================
# FOLLOW-UP 2: Numbers arrive one at a time: report the
# longest run after each one
# ============================================================

# ---- follow-up 2: function ----
# Answer: Store the run length at both ends of every run.
# A new number joins the run on its left and the run on
# its right, and updates the two outer ends.
# Time: O(1) per number.  Space: O(n).
class ConsecutiveStream:
    def __init__(self):
        self.run = {}                  # end value -> length of the run that has this value as an end
        self.best = 0

    def add(self, x):
        if x in self.run:
            return self.best
        left = self.run.get(x - 1, 0)
        right = self.run.get(x + 1, 0)
        total = left + right + 1
        self.run[x] = total
        self.run[x - left] = total
        self.run[x + right] = total
        if total > self.best:
            self.best = total
        return self.best
```

PATTERN 1 · HASH MAP / HASH SET · PROBLEM 12 OF 20

560. Subarray Sum Equals K

Medium · Array, Hash Table, Prefix Sum

leetcode.com/problems/subarray-sum-equals-k

★ **TOP 7 PROBLEM** in Hash Map / Hash Set, Prefix Sum: master this one first

Also appears in pattern 5 Prefix Sum

PROBLEM

Given an array of integers `nums` and an integer `k`, return *the total number of subarrays whose sum equals to* `k`.

A subarray is a contiguous **non-empty** sequence of elements within an array.

Example 1:

```
Input: nums = [1,1,1], k = 2
Output: 2
```

Example 2:

```
Input: nums = [1,2,3], k = 3
Output: 2
```

Constraints:

- `1 <= nums.length <= 2 * 10^4`
- `-1000 <= nums[i] <= 1000`
- `-10^7 <= k <= 10^7`

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Prefix Sum

How to think about it

Count subarrays whose sum is k. Pairwise endpoints are $O(n^2)$, tight for $n \le 2 \times 10^4$. A prefix s needs earlier prefixes equal to s-k. A hash map stores prefix frequencies, seeded with cnt[0]=1 for the empty prefix. Query first, then insert, so the current index is not reused.

We define a hash table `cnt` to store the number of times the prefix sum of the array `nums` appears. Initially, we set the value of `cnt[0]` to `1`, indicating that the prefix sum `0` appears once.

We traverse the array `nums`, calculate the prefix sum `s`, then add the value of `cnt[s - k]` to the answer, and increase the value of `cnt[s]` by `1`.

After the traversal, we return the answer.

The time complexity is `O(n)`, and the space complexity is `O(n)`. Where `n` is the length of the array `nums`.

Python 3

```python
class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        cnt = Counter({0: 1})
        ans = s = 0
        for x in nums:
            s += x
            ans += cnt[s - k]
            cnt[s] += 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Longest subarray with sum K
# 2. Count the subarrays whose sum is divisible by K
# 3. Return the positions of one subarray with sum K


# ============================================================
# FOLLOW-UP 1: Longest subarray with sum K
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember only the FIRST index at which each
# prefix sum appeared: the earliest earlier prefix gives
# the longest subarray.
# Time: O(n).  Space: O(n).
def longest_subarray_with_sum(nums, k):
    first_index = {0: -1}
    total = 0
    best = 0
    for i in range(len(nums)):
        total += nums[i]
        if total - k in first_index and i - first_index[total - k] > best:
            best = i - first_index[total - k]
        if total not in first_index:
            first_index[total] = i
    return best


# ============================================================
# FOLLOW-UP 2: Count the subarrays whose sum is divisible
# by K
# ============================================================

# ---- follow-up 2: function ----
# Answer: Two prefix sums with the same remainder modulo K
# enclose a subarray whose sum is divisible by K, so count
# prefix remainders instead of prefix sums.
# Time: O(n).  Space: O(k).
def subarrays_divisible_by_k(nums, k):
    seen = {0: 1}
    remainder = 0
    count = 0
    for x in nums:
        remainder = (remainder + x) % k
        count += seen.get(remainder, 0)
        seen[remainder] = seen.get(remainder, 0) + 1
    return count


# ============================================================
# FOLLOW-UP 3: Return the positions of one subarray with
# sum K
# ============================================================

# ---- follow-up 3: function ----
# Answer: Same prefix-sum dictionary, but store the index:
# the first time total - k was seen gives the start.
# Time: O(n).  Space: O(n).
def find_subarray_with_sum(nums, k):
    first_index = {0: -1}
```

```
    total = 0
    for i in range(len(nums)):
        total += nums[i]
        if total - k in first_index:
```

```
            return (first_index[total - k] + 1, i)
        if total not in first_index:
            first_index[total] = i
    return None
```

PATTERN 1 · HASH MAP / HASH SET · PROBLEM 13 OF 20

287. Find the Duplicate Number

Medium · Bit Manipulation, Array, Two Pointers, Binary Search, Floyd Cycle Detection, Pigeonhole Principle

leetcode.com/problems/find-the-duplicate-number

★ **TOP 7 PROBLEM** in Fast & Slow Pointers (Linked List), Cyclic Sort: master this one first

Also appears in pattern 24 Fast & Slow Pointers (Linked List), 33 Cyclic Sort

PROBLEM

Given an array of integers `nums` containing `n + 1` integers where each integer is in the range `[1, n]` inclusive.

There is only **one repeated number** in `nums`, return *this repeated number*.

You must solve the problem **without** modifying the array `nums` and using only constant extra space.

Example 1:

```
Input: nums = [1,3,4,2,2]
Output: 2
```

Example 2:

```
Input: nums = [3,1,3,4,2]
Output: 3
```

Example 3:

```
Input: nums = [3,3,3,3,3]
Output: 3
```

Constraints:

- `1 <= n <= 10^5`
- `nums.length == n + 1`
- `1 <= nums[i] <= n`
- All the integers in `nums` appear only **once** except for **precisely one integer** which appears **two or more** times.

Follow up:

- How can we prove that at least one duplicate number must exist in `nums`?
- Can you solve the problem in linear runtime complexity?

SOLUTION (PYTHON 3)

Solution 1: Binary Search

How to think about it

We cannot mutate the array or use a hash table. By pigeonhole, if more than x values lie in [1,x], the duplicate is there. Binary-search the value range: count how many entries are ≤ mid; if that count exceeds mid, search the left half, otherwise the right.

We can observe that if the number of elements in [1,..x] is greater than x, then the duplicate number must be in [1,..x], otherwise the duplicate number must be in [x+1,..n].

Therefore, we can use binary search to find x, and check whether the number of elements in [1,..x] is greater than x at each iteration. This way, we can determine which interval the duplicate number is in, and narrow down the search range until we find the duplicate number.

The time complexity is O(n × log n), where n is the length of the array nums. The space complexity is O(1).

Python 3

```python
class Solution:
    def findDuplicate(self, nums: List[int]) -> int:
        def f(x: int) -> bool:
            return sum(v <= x for v in nums) > x

        return bisect_left(range(len(nums)), True, key=f)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Use O(1) extra space and do not change the array
# (Floyd's cycle method)
# 2. Binary search on the value instead of on the array
# 3. Several numbers appear twice (values 1..n): find all
# of them


# ============================================================
# FOLLOW-UP 1: Use O(1) extra space and do not change the
# array (Floyd's cycle method)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Treat nums[i] as 'the next index'. Because a
# value repeats, two indexes lead into the same place, so
# the walk has a cycle, and the cycle's entrance is the
# duplicate: find where the slow and fast walkers meet,
# then walk from the start and from the meeting point at
# the same speed.
# Time: O(n).  Space: O(1).
def find_duplicate_floyd(nums):
    slow = nums[0]
    fast = nums[nums[0]]
    while slow != fast:
        slow = nums[slow]
        fast = nums[nums[fast]]
    finder = 0
    while finder != slow:
        finder = nums[finder]
        slow = nums[slow]
    return slow


# ============================================================
# FOLLOW-UP 2: Binary search on the value instead of on
# the array
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count how many numbers are at most mid. If more
# than mid of them are, the duplicate is in 1..mid;
# otherwise it is above. O(n log n) time, O(1) space, no
# changes to the array.
# Time: O(n log n).  Space: O(1).
def find_duplicate_binary_search(nums):
    low, high = 1, len(nums) - 1
    while low < high:
        mid = (low + high) // 2
        count = 0
        for x in nums:
            if x <= mid:
                count += 1
        if count > mid:
            high = mid
        else:
```

```
            low = mid + 1
    return low
```

```
# ============================================================
# FOLLOW-UP 3: Several numbers appear twice (values 1..n):
# find all of them
# ============================================================

# ---- follow-up 3: function ----
# Answer: Use the array itself as the marker: for each
# value, flip the sign of the number at index value - 1;
# meeting an already negative number means the value is a
# second copy.
# Time: O(n).  Space: O(1) extra (the input is changed).
def find_all_duplicates(nums):
    result = []
    for x in nums:
        index = abs(x) - 1
        if nums[index] < 0:
            result.append(abs(x))
        else:
            nums[index] = -nums[index]
    return result
```

PATTERN 1 · HASH MAP / HASH SET · PROBLEM 14 OF 20

454. 4Sum II

Medium · Array, Hash Table

leetcode.com/problems/4sum-ii

★ **TOP 7 PROBLEM** in Hash Map / Hash Set: master this one first

PROBLEM

Given four integer arrays `nums1`, `nums2`, `nums3`, and `nums4` all of length `n`, return the number of tuples `(i, j, k, l)` such that:

- `0 <= i, j, k, l < n`
- `nums1[i] + nums2[j] + nums3[k] + nums4[l] == 0`

Example 1:

```
Input: nums1 = [1,2], nums2 = [-2,-1], nums3 = [-1,2], nums4 = [0,2]
Output: 2
Explanation:
The two tuples are:
1. (0, 0, 0, 1) -> nums1[0] + nums2[0] + nums3[0] + nums4[1] = 1 + (-2) + (-1) + 2 = 0
2. (1, 1, 0, 0) -> nums1[1] + nums2[1] + nums3[0] + nums4[0] = 2 + (-1) + (-1) + 0 = 0
```

Example 2:

```
Input: nums1 = [0], nums2 = [0], nums3 = [0], nums4 = [0]
Output: 1
```

Constraints:

- `n == nums1.length`
- `n == nums2.length`
- `n == nums3.length`
- `n == nums4.length`
- `1 <= n <= 200`
- -2^{28} `<= nums1[i], nums2[i], nums3[i], nums4[i] <=` 2^{28}

SOLUTION (PYTHON 3)

Solution 1: Hash Table

How to think about it

Four nested loops are $O(n^4)$ and $n \le 200$ is too large. A zero sum splits as $(a+b)+(c+d)=0$. Count every pair sum from nums1 and nums2, then enumerate c,d and add cnt[-(c+d)]. The $O(n^2)$ pair enumeration turns a 4-tuple search into a hash lookup.

We can add the elements a and b in arrays nums1 and nums2 respectively, and store all possible sums in a hash table cnt, where the key is the sum of the two numbers, and the value is the count of the sum.

Then we iterate through the elements c and d in arrays nums3 and nums4, let c+d be the target value, then the answer is the cumulative sum of cnt[-(c+d)].

The time complexity is $O(n^2)$, and the space complexity is $O(n^2)$, where n is the length of the array.

Python 3

```python
class Solution:
    def fourSumCount(
        self, nums1: List[int], nums2: List[int], nums3: List[int], nums4: List[int]
    ) -> int:
        cnt = Counter(a + b for a in nums1 for b in nums2)
        return sum(cnt[-(c + d)] for c in nums3 for d in nums4)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The four numbers come from ONE array and the
# quadruplets must be unique
# 2. K lists instead of four


# ============================================================
# FOLLOW-UP 1: The four numbers come from ONE array and
# the quadruplets must be unique
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort, fix the first two numbers with two loops
# (skipping equal neighbours), and find the other two with
# two pointers.
# Time: O(n^3).  Space: O(1) extra.
def four_sum(nums, target):
    nums.sort()
    n = len(nums)
    result = []
    for i in range(n - 3):
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        for j in range(i + 1, n - 2):
            if j > i + 1 and nums[j] == nums[j - 1]:
                continue
            left, right = j + 1, n - 1
            while left < right:
                total = nums[i] + nums[j] + nums[left] + nums[right]
                if total == target:
                    result.append([nums[i], nums[j], nums[left], nums[right]])
                    left += 1
                    right -= 1
                    while left < right and nums[left] == nums[left - 1]:
                        left += 1
                    while left < right and nums[right] == nums[right + 1]:
                        right -= 1
                elif total < target:
                    left += 1
                else:
                    right -= 1
    return result


# ============================================================
# FOLLOW-UP 2: K lists instead of four
# ============================================================

# ---- follow-up 2: helper functions ----
def all_sums(lists):
    sums = {0: 1}                                   # sum -> number of ways
    for values in lists:
        bigger = {}
        for s in sums:
            for v in values:
                bigger[s + v] = bigger.get(s + v, 0) + sums[s]
```

```
        sums = bigger
    return sums

# ---- follow-up 2: function ----
```

```
# Answer: Meet in the middle: build a dictionary of every
# possible sum of the first half of the lists (with how
# many ways), then look up target - sum for every sum of
# the second half.
# Time: O(n^(k/2)) for k lists of n numbers.  Space:
# O(n^(k/2)).
def k_sum_count(lists, target):
    half = len(lists) // 2
    left = all_sums(lists[:half])
    right = all_sums(lists[half:])
    total = 0
    for s in left:
        total += left[s] * right.get(target - s, 0)
    return total
```

554. Brick Wall

Medium · Array, Hash Table

leetcode.com/problems/brick-wall

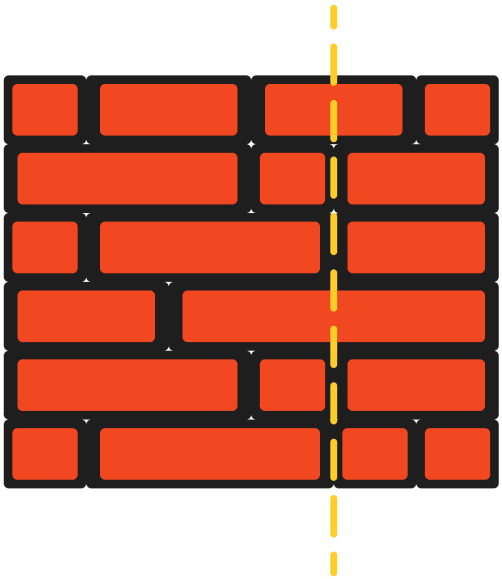
PROBLEM

There is a rectangular brick wall in front of you with `n` rows of bricks. The `i`th row has some number of bricks each of the same height (i.e., one unit) but they can be of different widths. The total width of each row is the same.

Draw a vertical line from the top to the bottom and cross the least bricks. If your line goes through the edge of a brick, then the brick is not considered as crossed. You cannot draw a line just along one of the two vertical edges of the wall, in which case the line will obviously cross no bricks.

Given the 2D array `wall` that contains the information about the wall, return *the minimum number of crossed bricks after drawing such a vertical line*.

Example 1:



```
Input: wall = [[1,2,2,1],[3,1,2],[1,3,2],[2,4],[3,1,2],[1,3,1,1]]
Output: 2
```

Example 2:

```
Input: wall = [[1],[1],[1]]
Output: 3
```

Constraints:

- `n == wall.length`
- `1 <= n <= 10`^{4}
- `1 <= wall[i].length <= 10`^{4}
- `1 <= sum(wall[i].length) <= 2 * 10`^{4}
- `sum(wall[i])` is the same for each row `i`.
- `1 <= wall[i][j] <= 2`^{31} `- 1`

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Prefix Sum

How to think about it

A vertical line that never cuts a brick's interior crosses (rows - rows aligned on that gap) bricks. Recounting rows per gap repeats work. Prefix-sum each row except the last brick and count gap positions. The most frequent gap is crossed least often; the answer is rows minus that frequency. The wall's edges are excluded, or the line would cross zero bricks. A gap is a sum of widths up to 2^{31}-1, so it can exceed 2^{31}-1. Java and C++ keep that sum in a 64-bit integer; a 32-bit sum treats positions 2^{32} apart as one gap.

We can use a hash table cnt to record the prefix sum of each row except for the last brick. The key is the value of the prefix sum, and the value is the number of times the prefix sum appears.

Traverse each row, and for each brick in the current row, add it to the current prefix sum and update cnt.

Finally, we traverse cnt to find the prefix sum that appears the most times, which represents the situation where the least number of bricks are crossed. The final answer is the number of rows in the brick wall minus the number of bricks crossed.

The time complexity is O(m × n), and the space complexity is O(n). Here, m and n are the number of rows and the number of bricks in the brick wall, respectively.

Python 3

```python
class Solution:
    def leastBricks(self, wall: List[List[int]]) -> int:
        cnt = Counter()
        for row in wall:
            s = 0
            for x in row[:-1]:
                s += x
                cnt[s] += 1
        return len(wall) - max(cnt.values(), default=0)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the position of the best vertical line, not
# only how many bricks it crosses
# 2. How many bricks does a line at a given position
# cross?


# ============================================================
# FOLLOW-UP 1: Return the position of the best vertical
# line, not only how many bricks it crosses
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same counting of brick edges, but remember which
# edge position has the highest count.
# Time: O(B), B = total number of bricks.  Space: O(B).
def best_cut_position(wall):
    edges = {}
    best_position = None
    best_count = 0
    for row in wall:
        position = 0
        for i in range(len(row) - 1):            # the last edge is the wall's side: not allowed
            position += row[i]
            edges[position] = edges.get(position, 0) + 1
            if edges[position] > best_count:
                best_count = edges[position]
                best_position = position
```

```
    return best_position


# ============================================================
# FOLLOW-UP 2: How many bricks does a line at a given
# position cross?
# ============================================================

# ---- follow-up 2: function ----
# Answer: For each row add up brick widths until the sum
# reaches or passes the position: a row is crossed unless
# a brick edge sits exactly there. Bricks may be huge, so
# never step one unit at a time.
# Time: O(B).  Space: O(1).
def bricks_crossed_at(wall, x):
    crossed = 0
    for row in wall:
        position = 0
        on_edge = False
        for brick in row:
            position += brick
            if position == x:
                on_edge = True
                break
            if position > x:
                break
        if not on_edge:
            crossed += 1
    return crossed
```

1461. Check If a String Contains All Binary Codes of Size K

Medium · Bit Manipulation, Hash Table, String, Directed Acyclic Graph, Hash Function, Rolling Hash

leetcode.com/problems/check-if-a-string-contains-all-binary-codes-of-size-k

PROBLEM

Given a binary string `s` and an integer `k`, return `true` *if every binary code of length* `k` *is a substring of* `s`. Otherwise, return `false`.

Example 1:

```
Input: s = "00110110", k = 2
Output: true
Explanation: The binary codes of length 2 are "00", "01", "10" and "11". They can be all found as substrings at indices 0, 1, 3 and 2 respectively.
```

Example 2:

```
Input: s = "0110", k = 1
Output: true
Explanation: The binary codes of length 1 are "0" and "1", it is clear that both exist as a substring.
```

Example 3:

```
Input: s = "0110", k = 2
Output: false
Explanation: The binary code "00" is of length 2 and does not exist in the array.
```

Constraints:

- `1 <= s.length <= 5 * 10^5`
- `s[i]` is either `'0'` or `'1'`.
- `1 <= k <= 20`

SOLUTIONS (PYTHON 3)

Solution 1: Hash Table

> **How to think about it**
>
> There are 2^k binary strings of length k, and k≤ 20. If s has fewer than 2^k windows, fail; otherwise collect every length-k substring and test the set size.

First, for a string s of length n, the number of substrings of length k is n - k + 1. If n - k + 1 < 2^k, then there must exist a binary string of length k that is not a substring of s, so we return `false`.

Next, we traverse the string s and store all substrings of length k in a set ss. Finally, we check if the size of the set ss is equal to 2^k.

The time complexity is O(n × k), and the space complexity is O(n). Here, n is the length of the string s.

Python 3

```python
class Solution:
    def hasAllCodes(self, s: str, k: int) -> bool:
        n = len(s)
        m = 1 << k
        if n - k + 1 < m:
            return False
        ss = {s[i : i + k] for i in range(n - k + 1)}
        return len(ss) == m
```

Solution 2: Sliding Window

> **How to think about it**
>
> Method 1 copies k characters per window. Treat the window as an integer, shift in the new bit and drop the high bit, and insert in O(1).

In Solution 1, we stored all distinct substrings of length k, and processing each substring requires O(k) time. We can instead use a sliding window, where each time we add the latest character, we remove the leftmost character from the window. During this process, we use an integer x to store the substring.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the string s.

Python 3

```python
class Solution:
    def hasAllCodes(self, s: str, k: int) -> bool:
        n = len(s)
        m = 1 << k
        if n - k + 1 < m:
            return False
        ss = set()
        x = int(s[:k], 2)
        ss.add(x)
        for i in range(k, n):
            a = int(s[i - k]) << (k - 1)
            b = int(s[i])
            x = (x - a) << 1 | b
            ss.add(x)
        return len(ss) == m
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. K is large and building substrings is too slow or too
# big
# 2. List the codes of size K that are missing


# ============================================================
# FOLLOW-UP 1: K is large and building substrings is too
# slow or too big
# ============================================================

# ---- follow-up 1: function ----
# Answer: Roll a number: shift left, add the new bit, and
# keep only the last k bits with a mask. A table of 2^k
# flags replaces the set of strings.
# Time: O(n + 2^k).  Space: O(2^k).
def has_all_codes_rolling(s, k):
    need = 1 << k
    if len(s) - k + 1 < need:
        return False
    seen = [False] * need
    mask = need - 1
    value = 0
    found = 0
    for i in range(len(s)):
        bit = 1 if s[i] == "1" else 0
        value = ((value << 1) | bit) & mask
        if i >= k - 1 and not seen[value]:
            seen[value] = True
            found += 1
            if found == need:
                return True
    return False
```

```
# ============================================================
# FOLLOW-UP 2: List the codes of size K that are missing
# ============================================================

# ---- follow-up 2: helper functions ----
def to_binary(number, k):
    bits = ""
    for i in range(k):
        bits = str(number % 2) + bits
        number //= 2
    return bits

# ---- follow-up 2: function ----
# Answer: Collect the substrings that do occur, then walk
# through all 2^k codes and report those that are absent.
# Time: O(n * k + 2^k * k).  Space: O(2^k * k).
def missing_codes(s, k):
    present = set()
    for i in range(len(s) - k + 1):
        present.add(s[i:i + k])
    missing = []
    for number in range(1 << k):
        code = to_binary(number, k)
        if code not in present:
            missing.append(code)
```

```
    return missing
```

523. Continuous Subarray Sum

Medium · Array, Hash Table, Math, Prefix Sum, Pigeonhole Principle

leetcode.com/problems/continuous-subarray-sum

★ **TOP 7 PROBLEM** in Prefix Sum: master this one first

Also appears in pattern 5 Prefix Sum

PROBLEM

Given an integer array nums and an integer k, return `true` *if* `nums` *has a* ***good subarray*** *or* `false` *otherwise*.

A **good subarray** is a subarray where:

- its length is **at least two**, and
- the sum of the elements of the subarray is a multiple of `k`.

Note that:

- A **subarray** is a contiguous part of the array.
- An integer `x` is a multiple of `k` if there exists an integer `n` such that `x = n * k`. `0` is **always** a multiple of `k`.

Example 1:

```
Input: nums = [23,2,4,6,7], k = 6
Output: true
Explanation: [2, 4] is a continuous subarray of size 2 whose elements sum up to 6.
```

Example 2:

```
Input: nums = [23,2,6,4,7], k = 6
Output: true
Explanation: [23, 2, 6, 4, 7] is an continuous subarray of size 5 whose elements sum up to 42.
42 is a multiple of 6 because 42 = 7 * 6 and 7 is an integer.
```

Example 3:

```
Input: nums = [23,2,6,4,7], k = 13
Output: false
```

Constraints:

- `1 <= nums.length <= 10^5`
- `0 <= nums[i] <= 10^9`
- `0 <= sum(nums[i]) <= 2^31 - 1`
- `1 <= k <= 2^31 - 1`

SOLUTION (PYTHON 3)

Solution 1: Prefix Sum + Hash Table

How to think about it

We need a subarray of length at least 2 whose sum is a multiple of k. Checking all pairs is $O(n^2)$ and too slow for $n \le 10^5$. Equal prefix sums modulo k mean the interior sum is divisible by k. Store the first index of each remainder; a later repeat with gap greater than 1 is a hit. Remainder 0 is seeded at -1 so prefixes from the start are covered.

According to the problem description, if there exist two positions i and j (j < i) where the remainders of the prefix sums modulo k are the same, then the sum of the subarray nums[j+1..i] is a multiple of k.

Therefore, we can use a hash table to store the first occurrence of each remainder of the prefix sum modulo k. Initially, we store a key-value pair (0, -1) in the hash table, indicating that the remainder 0 of the prefix sum 0 appears at position -1.

As we iterate through the array, we calculate the current prefix sum's remainder modulo k. If the current prefix sum's remainder modulo k has not appeared in the hash table, we store the current prefix sum's remainder modulo k and its corresponding position in the hash table. Otherwise, if the current prefix sum's remainder modulo k has already appeared in the hash table at position j, then we have found a subarray nums[j+1..i] that meets the conditions, and thus return True.

After completing the iteration, if no subarray meeting the conditions is found, we return False.

The time complexity is O(n), and the space complexity is O(n), where n is the length of the array nums.

Python 3

```python
class Solution:
    def checkSubarraySum(self, nums: List[int], k: int) -> bool:
        d = {0: -1}
        s = 0
        for i, x in enumerate(nums):
            s = (s + x) % k
            if s not in d:
                d[s] = i
            elif i - d[s] > 1:
                return True
        return False
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the start and end of one such subarray
# 2. Find the LONGEST such subarray (length at least 2)
# 3. k can also be 0 (the sum must then be exactly 0)


# ============================================================
# FOLLOW-UP 1: Return the start and end of one such
# subarray
# ============================================================

# ---- follow-up 1: function ----
# Answer: Store the first index of every remainder;
# meeting the same remainder again at least two positions
# later means the numbers in between add up to a multiple
# of k.
# Time: O(n).  Space: O(min(n, k)).
def find_multiple_subarray(nums, k):
    first_index = {0: -1}
    remainder = 0
    for i in range(len(nums)):
        remainder = (remainder + nums[i]) % k
        if remainder in first_index:
            if i - first_index[remainder] >= 2:
                return (first_index[remainder] + 1, i)
        else:
            first_index[remainder] = i
    return None


# ============================================================
# FOLLOW-UP 2: Find the LONGEST such subarray (length at
# least 2)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the first index of each remainder and
# measure the distance every time the remainder repeats.
# Time: O(n).  Space: O(min(n, k)).
```

```
def longest_multiple_subarray(nums, k):
    first_index = {0: -1}
    remainder = 0
    best = 0
    for i in range(len(nums)):
        remainder = (remainder + nums[i]) % k
        if remainder in first_index:
            length = i - first_index[remainder]
            if length >= 2 and length > best:
                best = length
        else:
            first_index[remainder] = i
    return best


# ========================================================
# FOLLOW-UP 3: k can also be 0 (the sum must then be
# exactly 0)
# ========================================================

# ---- follow-up 3: function ----
```

```
# Answer: With k = 0 compare the plain prefix sums instead
# of their remainders; everything else stays the same.
# Time: O(n).  Space: O(min(n, k)) (O(n) when k = 0).
def check_subarray_sum_allow_zero(nums, k):
    first_index = {0: -1}
    total = 0
    for i in range(len(nums)):
        total += nums[i]
        key = total
        if k != 0:
            key = total % k
        if key in first_index:
            if i - first_index[key] >= 2:
                return True
        else:
            first_index[key] = i
    return False
```

PATTERN 1 · HASH MAP / HASH SET · PROBLEM 18 OF 20

380. Insert Delete GetRandom O(1)

Medium · Design, Array, Hash Table, Math, Randomized

leetcode.com/problems/insert-delete-getrandom-o1

★ **TOP 7 PROBLEM** in Hash Map / Hash Set, Design Problems: master this one first

Also appears in pattern 29 Design Problems

PROBLEM

Implement the `RandomizedSet` class:

- `RandomizedSet()` Initializes the `RandomizedSet` object.
- `bool insert(int val)` Inserts an item `val` into the set if not present. Returns `true` if the item was not present, `false` otherwise.
- `bool remove(int val)` Removes an item `val` from the set if present. Returns `true` if the item was present, `false` otherwise.
- `int getRandom()` Returns a random element from the current set of elements (it's guaranteed that at least one element exists when this method is called). Each element must have the **same probability** of being returned.

You must implement the functions of the class such that each function works in **average** `O(1)` time complexity.

Example 1:

```
Input
["RandomizedSet", "insert", "remove", "insert", "getRandom", "remove", "insert", "getRandom"]
[[], [1], [2], [2], [], [1], [2], []]
Output
[null, true, false, true, 2, true, false, 2]

Explanation
RandomizedSet randomizedSet = new RandomizedSet();
randomizedSet.insert(1); // Inserts 1 to the set. Returns true as 1 was inserted successfully.
randomizedSet.remove(2); // Returns false as 2 does not exist in the set.
randomizedSet.insert(2); // Inserts 2 to the set, returns true. Set now contains [1,2].
randomizedSet.getRandom(); // getRandom() should return either 1 or 2 randomly.
randomizedSet.remove(1); // Removes 1 from the set, returns true. Set now contains [2].
randomizedSet.insert(2); // 2 was already in the set, so return false.
randomizedSet.getRandom(); // Since 2 is the only number in the set, getRandom() will always return 2.
```

Constraints:

- -2^{31} <= val <= $2^{31} - 1$
- At most $2 * 10^5$ calls will be made to `insert`, `remove`, and `getRandom`.
- There will be **at least one** element in the data structure when `getRandom` is called.

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Dynamic List

How to think about it

A set with insert, delete, and uniform random, all O(1). A hash map alone has no index; an array alone deletes in O(n). The array holds values, the map holds indices. Insert appends; delete swaps with the last element, pops, and repairs that index. Random is `choice` on the array.

We define a dynamic list q to store the elements in the set, and a hash table d to store the index of each element in q.

When inserting an element, if the element already exists in the hash table d, return `false` directly; otherwise, we insert the element into the end of the dynamic list q, and insert the element and its index in q into the hash table d at the same time,

and finally return `true`.

When deleting an element, if the element does not exist in the hash table d, return `false` directly; otherwise, we obtain the index of the element in the list q from the hash table, then swap the last element q[-1] in the list q with q[i], and then update the index of q[-1] in the hash table to i. Then delete the last element in q, and remove the element from the hash table at the same time, and finally return `true`.

When getting a random element, we can randomly select an element from the dynamic list q and return it.

Time complexity O(1), space complexity O(n), where n is the number of elements in the set.

Python 3

```
class RandomizedSet:
    def __init__(self):
        self.d = {}
        self.q = []

    def insert(self, val: int) -> bool:
        if val in self.d:
            return False
        self.d[val] = len(self.q)
        self.q.append(val)
        return True

    def remove(self, val: int) -> bool:
        if val not in self.d:
            return False
        i = self.d[val]
        self.d[self.q[-1]] = i
        self.q[i] = self.q[-1]
        self.q.pop()
        self.d.pop(val)
        return True

    def getRandom(self) -> int:
        return choice(self.q)


# Your RandomizedSet object will be instantiated and called as such:
# obj = RandomizedSet()
# param_1 = obj.insert(val)
# param_2 = obj.remove(val)
# param_3 = obj.getRandom()
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Duplicates are allowed (the same value can be
# inserted several times)
# 2. Pick a value with probability proportional to its
# weight
# 3. getRandom must not repeat a value until every value
# has been returned once


# ============================================================
# FOLLOW-UP 1: Duplicates are allowed (the same value can
# be inserted several times)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the list of values plus, for every value,
# the SET of positions where it sits. Removing swaps one
# position of the value with the last list element, and
# fixes both position sets.
# Time: O(1) per operation.  Space: O(n).
```

```
import random


class RandomizedCollection:
    def __init__(self):
        self.items = []
        self.where = {}                          # value -> set of positions in self.items

    def insert(self, val):
        is_new = val not in self.where or len(self.where[val]) == 0
        if val not in self.where:
            self.where[val] = set()
        self.where[val].add(len(self.items))
        self.items.append(val)
        return is_new

    def remove(self, val):
        if val not in self.where or len(self.where[val]) == 0:
            return False
        index = self.where[val].pop()
        last_index = len(self.items) - 1
        last = self.items[last_index]
        if index != last_index:
            self.items[index] = last             # move the last value into the hole
            self.where[last].discard(last_index)
            self.where[last].add(index)
        self.items.pop()
        return True

    def getRandom(self):
        return self.items[random.randrange(len(self.items))]


# ============================================================
# FOLLOW-UP 2: Pick a value with probability proportional
# to its weight
# ============================================================

# ---- follow-up 2: function ----
# Answer: Add the weights up and pick a random point below
```

```
# the total; the value whose running total passes the
# point is the answer. O(n) per pick; a Fenwick tree makes
# it O(log n).
# Time: insert and remove O(1), get_random O(n).  Space:
# O(n).
import random


class WeightedRandomSet:
    def __init__(self):
        self.weights = {}

    def insert(self, val, weight):
        self.weights[val] = weight

    def remove(self, val):
        if val in self.weights:
            del self.weights[val]

    def get_random(self):
        total = 0
        for w in self.weights.values():
            total += w
        point = random.random() * total
        running = 0
        for val in self.weights:
            running += self.weights[val]
            if point < running:
```

```
                return val
        return None


# ============================================================
# FOLLOW-UP 3: getRandom must not repeat a value until
# every value has been returned once
# ============================================================

# ---- follow-up 3: function ----
# Answer: Keep a boundary in the list: the elements before
# it are not yet returned. Pick a random one from them,
# swap it to the boundary, and move the boundary down.
# Time: O(1) per call.  Space: O(n).
import random


class NoRepeatRandom:
    def __init__(self, values):
        self.items = list(values)
        self.left = len(self.items)               # items[:left] have not been returned in this round

    def get_random(self):
        if self.left == 0:
            self.left = len(self.items)           # everything was returned: start a new round
        i = random.randrange(self.left)
        self.left -= 1
        self.items[i], self.items[self.left] = self.items[self.left], self.items[i]
        return self.items[self.left]
```

PATTERN 1 · HASH MAP / HASH SET · PROBLEM 19 OF 20

992. Subarrays with K Different Integers

Hard · Array, Hash Table, Counting, Sliding Window

leetcode.com/problems/subarrays-with-k-different-integers

Also appears in pattern 2 Sliding Window

PROBLEM

Given an integer array `nums` and an integer `k`, return *the number of **good subarrays** of* `nums`.

A **good array** is an array where the number of different integers in that array is exactly `k`.

- For example, `[1,2,3,1,2]` has `3` different integers: `1`, `2`, and `3`.

A **subarray** is a **contiguous** part of an array.

Example 1:

```
Input: nums = [1,2,1,2,3], k = 2
Output: 7
Explanation: Subarrays formed with exactly 2 different integers: [1,2], [2,1], [1,2], [2,3], [1,2,1],
[2,1,2], [1,2,1,2]
```

Example 2:

```
Input: nums = [1,2,1,3,4], k = 3
Output: 3
Explanation: Subarrays formed with exactly 3 different integers: [1,2,1,3], [2,1,3], [1,3,4].
```

Constraints:

- `1 <= nums.length <= 2 * 10^4`
- `1 <= nums[i], k <= nums.length`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Count subarrays with exactly k distinct values. n≤ 2× 10^4, so enumeration is too slow. The number of distinct values is monotone in the window, so “at most k” is a sliding window. Exactly k equals the leftmost index for “at most k” minus that for “at most k-1”, summed over every right end.

Python 3

```python
class Solution:
    def subarraysWithKDistinct(self, nums: List[int], k: int) -> int:
        def f(k):
            pos = [0] * len(nums)
            cnt = Counter()
            j = 0
            for i, x in enumerate(nums):
                cnt[x] += 1
                while len(cnt) > k:
                    cnt[nums[j]] -= 1
                    if cnt[nums[j]] == 0:
                        cnt.pop(nums[j])
                    j += 1
                pos[i] = j
            return pos

        return sum(a - b for a, b in zip(f(k - 1), f(k)))
```

Complexity. Time: O(n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The longest subarray with at most K different
# integers
# 2. Count the subarrays with AT LEAST K different
# integers


# ========================================================
# FOLLOW-UP 1: The longest subarray with at most K
# different integers
# ========================================================

# ---- follow-up 1: function ----
# Answer: Sliding window: grow on the right, and shrink
# from the left whenever there are more than K different
# values.
# Time: O(n).  Space: O(k).
def longest_with_at_most_k_distinct(nums, k):
    count = {}
    left = 0
    best = 0
    for right in range(len(nums)):
        count[nums[right]] = count.get(nums[right], 0) + 1
        while len(count) > k:
            count[nums[left]] -= 1
            if count[nums[left]] == 0:
                del count[nums[left]]
            left += 1
        if right - left + 1 > best:
            best = right - left + 1
    return best


# ========================================================
# FOLLOW-UP 2: Count the subarrays with AT LEAST K
# different integers
# ========================================================

# ---- follow-up 2: helper functions ----
def count_at_most(nums, k):
    if k <= 0:
        return 0
    count = {}
    left = 0
    total = 0
    for right in range(len(nums)):
        count[nums[right]] = count.get(nums[right], 0) + 1
        while len(count) > k:
            count[nums[left]] -= 1
            if count[nums[left]] == 0:
                del count[nums[left]]
            left += 1
        total += right - left + 1
    return total

# ---- follow-up 2: function ----
# Answer: All subarrays minus those with at most K - 1
# different integers. Counting 'at most' with a sliding
# window adds right - left + 1 new subarrays at every
# right end.
```

```
# Time: O(n).  Space: O(k).
def count_at_least_k_distinct(nums, k):
```

```
    n = len(nums)
    everything = n * (n + 1) // 2
    return everything - count_at_most(nums, k - 1)
```

PATTERN 1 · HASH MAP / HASH SET · PROBLEM 20 OF 20

41. First Missing Positive

Hard · Array, Hash Table

leetcode.com/problems/first-missing-positive

★ **TOP 7 PROBLEM** in Hash Map / Hash Set, Cyclic Sort: master this one first

Also appears in pattern 33 Cyclic Sort

PROBLEM

Given an unsorted integer array `nums`. Return the *smallest positive integer* that is *not present* in `nums`.

You must implement an algorithm that runs in `O(n)` time and uses `O(1)` auxiliary space.

Example 1:

```
Input: nums = [1,2,0]
Output: 3
Explanation: The numbers in the range [1,2] are all in the array.
```

Example 2:

```
Input: nums = [3,4,-1,1]
Output: 2
Explanation: 1 is in the array but 2 is missing.
```

Example 3:

```
Input: nums = [7,8,9,11,12]
Output: 1
Explanation: The smallest positive integer 1 is missing.
```

Constraints:

- `1 <= nums.length <= 10^5`
- `-2^31 <= nums[i] <= 2^31 - 1`

SOLUTION (PYTHON 3)

Solution 1: In-place Swap

How to think about it

The first idea is a hash set of seen numbers, then probe 1, 2, 3, …. Correct and O(n) time, but O(n) extra space. The problem asks for O(n) time and constant extra space; $n \le 10^5$ makes the set fail the space bound. The bottleneck is recording whether 1..n appear with extra memory. The missing positive must lie in [1, n+1], so we only care about 1..n — the array indices themselves can be that table. Swap value x to index x-1 when $x \in [1, n]$. In-place swap turns the array into a hash; one more scan finds the first hole.

We assume the length of the array nums is n, then the smallest positive integer must be in the range [1, .., n + 1]. We can traverse the array and swap each number x to its correct position, that is, the position x - 1. If x is not in the range [1, n + 1], then we can ignore it.

After the traversal, we traverse the array again. If i+1 is not equal to nums[i], then i+1 is the smallest positive integer we are looking for.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Python 3

```python
class Solution:
    def firstMissingPositive(self, nums: List[int]) -> int:
        n = len(nums)
        for i in range(n):
            while 1 <= nums[i] <= n and nums[i] != nums[nums[i] - 1]:
                j = nums[i] - 1
                nums[i], nums[j] = nums[j], nums[i]
        for i in range(n):
            if nums[i] != i + 1:
                return i + 1
        return n + 1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The smallest missing NON-NEGATIVE number (0 counts)
# 2. The array is sorted and has no duplicates (non-
# negative): O(log n)
# 3. You may not change the input array


# ============================================================
# FOLLOW-UP 1: The smallest missing NON-NEGATIVE number (0
# counts)
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same swapping trick, with value v belonging
# at index v instead of v - 1.
# Time: O(n).  Space: O(1).
def first_missing_non_negative(nums):
    n = len(nums)
    i = 0
    while i < n:
        v = nums[i]
        if 0 <= v < n and nums[v] != v:
            nums[i], nums[v] = nums[v], nums[i]
        else:
            i += 1
    for i in range(n):
        if nums[i] != i:
            return i
    return n


# ============================================================
# FOLLOW-UP 2: The array is sorted and has no duplicates
# (non-negative): O(log n)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Binary search for the first index whose value is
# not equal to the index: everything before it is in
# place, everything after it has a gap.
# Time: O(log n).  Space: O(1).
def first_missing_in_sorted(nums):
    low, high = 0, len(nums)
    while low < high:
        mid = (low + high) // 2
        if nums[mid] == mid:
            low = mid + 1
        else:
            high = mid
    return low
```

```
# ============================================================
# FOLLOW-UP 3: You may not change the input array
# ============================================================

# ---- follow-up 3: function ----
# Answer: Put the numbers in a set (O(n) extra space) and
# count up from 1 until a number is missing.
# Time: O(n).  Space: O(n).
```

```
def first_missing_positive_no_change(nums):
    seen = set(nums)
    candidate = 1
    while candidate in seen:
        candidate += 1
    return candidate
```

PATTERN 2 OF 41

Sliding Window

20 problems · 1 Easy · 16 Medium · 3 Hard

THE IDEA

A contiguous piece of an array or string that grows on the right and shrinks on the left. Both ends only move forward, so the whole scan is O(n).

Signals: longest / shortest / count of subarrays or substrings that satisfy a condition, at most K distinct, sum at least S, fixed length K.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
left = 0
for right in range(len(s)):
    add(s[right])                       # grow the window
    while window_is_bad():              # shrink until it is valid again
        remove(s[left])
        left += 1
    best = max(best, right - left + 1)
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Best Time to Buy and Sell Stock	Easy	LC 121	69
2	★ **Longest Substring Without Repeating Characters**	Medium	LC 3	72
3	★ **Minimum Size Subarray Sum**	Medium	LC 209	75
4	★ **Longest Repeating Character Replacement**	Medium	LC 424	78
5	★ **Permutation in String**	Medium	LC 567	81
6	Find All Anagrams in a String	Medium	LC 438	84
7	Fruit Into Baskets	Medium	LC 904	87
8	★ **Max Consecutive Ones III**	Medium	LC 1004	91
9	Subarray Product Less Than K	Medium	LC 713	94
10	Longest Subarray of 1's After Deleting One Element	Medium	LC 1493	97
11	Maximum Number of Vowels in a Substring of Given Length	Medium	LC 1456	101
12	Count Number of Nice Subarrays	Medium	LC 1248	103
13	Binary Subarrays With Sum	Medium	LC 930	105
14	Maximum Erasure Value	Medium	LC 1695	108
15	Grumpy Bookstore Owner	Medium	LC 1052	111
16	Frequency of the Most Frequent Element	Medium	LC 1838	114
17	Minimum Number of Flips to Make the Binary String Alternating	Medium	LC 1888	118
18	★ **Minimum Window Substring**	Hard	LC 76	121
19	★ **Sliding Window Maximum**	Hard	LC 239	124
20	Subarrays with K Different Integers (see page)	Hard	LC 992	62

PATTERN 2 · SLIDING WINDOW · PROBLEM 1 OF 20

121. Best Time to Buy and Sell Stock

Easy · Array, Dynamic Programming

leetcode.com/problems/best-time-to-buy-and-sell-stock

PROBLEM

You are given an array `prices` where `prices[i]` is the price of a given stock on the i^{th} day.

You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

Return *the maximum profit you can achieve from this transaction*. If you cannot achieve any profit, return `0`.

Example 1:

```
Input: prices = [7,1,5,3,6,4]
Output: 5
Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.
Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.
```

Example 2:

```
Input: prices = [7,6,4,3,1]
Output: 0
Explanation: In this case, no transactions are done and the max profit = 0.
```

Constraints:

- `1 <= prices.length <=` 10^5
- `0 <= prices[i] <=` 10^4

SOLUTION (PYTHON 3)

Solution 1: Enumerate + Maintain the Minimum Value of the Prefix

> **How to think about it**
>
> One buy and one sell; trying every pair is $O(n^2)$ and n can reach 10^5. For a fixed sell day the best buy is the minimum price before it. Scan while keeping the prefix minimum, update the answer by today's price minus that minimum, then fold today into the prefix.

We can enumerate each element of the array nums as the selling price. Then we need to find a minimum value in front of it as the purchase price to maximize the profit.

Therefore, we use a variable mi to maintain the prefix minimum value of the array nums. Then we traverse the array nums and for each element v, calculate the difference between it and the minimum value mi in front of it, and update the answer to the maximum of the difference. Then update mi = min(mi, v). Continue to traverse the array nums until the traversal ends.

Finally, return the answer.

The time complexity is O(n), where n is the length of the array nums. The space complexity is O(1).

Python 3

```python
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        ans, mi = 0, inf
        for v in prices:
            ans = max(ans, v - mi)
            mi = min(mi, v)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the days to buy and to sell, not only the
# profit
# 2. You may buy and sell as many times as you like (one
# share at a time)
# 3. At most two transactions


# ============================================================
# FOLLOW-UP 1: Return the days to buy and to sell, not
# only the profit
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same single pass: remember on which day the
# lowest price so far happened, and when selling today
# beats the best profit, store both days.
# Time: O(n).  Space: O(1).
def best_days_to_trade(prices):
    if len(prices) < 2:
        return 0, None, None
    lowest_day = 0
    best_profit, buy_day, sell_day = 0, None, None
    for day in range(1, len(prices)):
        if prices[day] < prices[lowest_day]:
            lowest_day = day
        elif prices[day] - prices[lowest_day] > best_profit:
            best_profit = prices[day] - prices[lowest_day]
            buy_day, sell_day = lowest_day, day
    return best_profit, buy_day, sell_day


# ============================================================
# FOLLOW-UP 2: You may buy and sell as many times as you
# like (one share at a time)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Every rise from one day to the next can be
# collected: add every positive difference between
# neighbouring days.
# Time: O(n).  Space: O(1).
def max_profit_unlimited(prices):
    profit = 0
    for day in range(1, len(prices)):
        if prices[day] > prices[day - 1]:
            profit += prices[day] - prices[day - 1]
    return profit


# ============================================================
# FOLLOW-UP 3: At most two transactions
# ============================================================

# ---- follow-up 3: helper functions ----
def max_of(a, b):
    if a > b:
        return a
    return b
```

```
# ---- follow-up 3: function ----
# Answer: Track four numbers while scanning: the best
# balance after the first buy, after the first sell, after
# the second buy and after the second sell. Each one is
# built from the one before it.
```

```
# Time: O(n).  Space: O(1).
def max_profit_two_transactions(prices):
    buy1 = buy2 = float("-inf")
    sell1 = sell2 = 0
    for price in prices:
        buy1 = max_of(buy1, -price)
        sell1 = max_of(sell1, buy1 + price)
        buy2 = max_of(buy2, sell1 - price)
        sell2 = max_of(sell2, buy2 + price)
    return sell2
```

PATTERN 2 · SLIDING WINDOW · PROBLEM 2 OF 20

3. Longest Substring Without Repeating Characters

Medium · Hash Table, String, Sliding Window

leetcode.com/problems/longest-substring-without-repeating-characters

★ **TOP 7 PROBLEM** in Sliding Window: master this one first

PROBLEM

Given a string `s`, find the length of the **longest substring** without duplicate characters.

Example 1:

```
Input: s = "abcabcbb"
Output: 3
Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.
```

Example 2:

```
Input: s = "bbbbb"
Output: 1
Explanation: The answer is "b", with the length of 1.
```

Example 3:

```
Input: s = "pwwkew"
Output: 3
Explanation: The answer is "wke", with the length of 3.
Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.
```

Constraints:

- `0 <= s.length <= 10^5`
- `s` consists of English letters, digits, symbols and spaces.

SOLUTION (PYTHON 3)

Solution 1: Sliding Window

How to think about it

The first idea is to try every substring and test uniqueness. Correct, but at least $O(n^2)$. $n \le 10^5$ will time out. The waste is overlap: once [l,r] has no repeats, adding a character c on the right only needs to fix whether c broke the window—not a full rescan. A duplicate means the valid answer cannot keep the extra c on the left, so l must move right. A count table telling whether cnt[c] > 1 is enough to decide shrinking; after c appears once, the window is the longest unique substring ending at r.

We can use two pointers l and r to maintain a sliding window that always satisfies the condition of having no repeating characters within the window. Initially, both l and r point to the first character of the string. We use a hash table or an array of length 128 called cnt to record the number of occurrences of each character, where cnt[c] represents the number of occurrences of character c.

Next, we move the right pointer r one step at a time. Each time we move it, we increment the value of cnt[s[r]] by 1, and then check if the value of cnt[s[r]] is greater than 1 within the current window [l, r]. If it is greater than 1, it means there are repeating characters within the current window, and we need to move the left pointer l until there are no repeating characters within the window. Then, we update the answer ans = max(ans, r - l + 1).

Finally, we return the answer ans.

The time complexity is O(n), where n is the length of the string. The space complexity is O(|Σ|), where Σ represents the character set, and the size of Σ is 128.

Python 3

```python
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        cnt = Counter()
        ans = l = 0
        for r, c in enumerate(s):
            cnt[c] += 1
            while cnt[c] > 1:
                cnt[s[l]] -= 1
                l += 1
            ans = max(ans, r - l + 1)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the substring itself, not only its length
# 2. Allow at most K different characters instead of none
# repeated


# ============================================================
# FOLLOW-UP 1: Return the substring itself, not only its
# length
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember the last index of every character. When
# the current character was seen inside the window, move
# the window start just after that earlier copy.
# Time: O(n).  Space: O(s), s = distinct characters.
def longest_unique_substring(s):
    last_index = {}
    start = 0
    best_start, best_length = 0, 0
    for i in range(len(s)):
        ch = s[i]
        if ch in last_index and last_index[ch] >= start:
            start = last_index[ch] + 1
        last_index[ch] = i
        if i - start + 1 > best_length:
            best_length = i - start + 1
            best_start = start
    return s[best_start:best_start + best_length]


# ============================================================
# FOLLOW-UP 2: Allow at most K different characters
# instead of none repeated
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the characters inside the window; while
# there are more than K different ones, shrink from the
# left.
# Time: O(n).  Space: O(k).
def longest_with_k_distinct(s, k):
    count = {}
    left = 0
    best = 0
    for right in range(len(s)):
        count[s[right]] = count.get(s[right], 0) + 1
        while len(count) > k:
```

```
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        if right - left + 1 > best:
            best = right - left + 1
    return best
```

PATTERN 2 · SLIDING WINDOW · PROBLEM 3 OF 20

209. Minimum Size Subarray Sum

Medium · Array, Binary Search, Prefix Sum, Sliding Window

leetcode.com/problems/minimum-size-subarray-sum

★ **TOP 7 PROBLEM** in Sliding Window: master this one first

PROBLEM

Given an array of positive integers `nums` and a positive integer `target`, return *the **minimal length** of a subarray whose sum is greater than or equal to* `target`. If there is no such subarray, return `0` instead.

Example 1:

```
Input: target = 7, nums = [2,3,1,2,4,3]
Output: 2
Explanation: The subarray [4,3] has the minimal length under the problem constraint.
```

Example 2:

```
Input: target = 4, nums = [1,4,4]
Output: 1
```

Example 3:

```
Input: target = 11, nums = [1,1,1,1,1,1,1,1]
Output: 0
```

Constraints:

- `1 <= target <=` 10^9
- `1 <= nums.length <=` 10^5
- `1 <= nums[i] <=` 10^4

Follow up:

If you have figured out the

`O(n)`

solution, try coding another solution of which the time complexity is

`O(n log(n))`

.

SOLUTIONS (PYTHON 3)

Solution 1: Prefix Sum + Binary Search

How to think about it

Summing every subarray is O(n^2). All elements are positive, so prefix sums are monotone and the shortest right end for a fixed left end can be binary-searched. After building s, for each s[i] we find the least j with s[j] ≥ s[i]+target and update the length by j-i.

First, we preprocess the prefix sum array s of the array nums, where s[i] represents the sum of the first i elements of the array nums. Since all elements in the array nums are positive integers, the array s is also monotonically increasing. Also, we initialize the answer ans = n + 1, where n is the length of the array nums.

Next, we traverse the prefix sum array s. For each element s[i], we can find the smallest index j that satisfies s[j] ≥ s[i] + target by binary search. If j ≤ n, it means that there exists a subarray that satisfies the condition, and we can update the answer, i.e., ans = min(ans, j - i).

Finally, if ans ≤ n, it means that there exists a subarray that satisfies the condition, return ans, otherwise return 0.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def minSubArrayLen(self, target: int, nums: List[int]) -> int:
        n = len(nums)
        s = list(accumulate(nums, initial=0))
        ans = n + 1
        for i, x in enumerate(s):
            j = bisect_left(s, x + target)
            if j <= n:
                ans = min(ans, j - i)
        return ans if ans <= n else 0
```

Solution 2: Two Pointers

> **How to think about it**
>
> Prefix sums still use O(n) space and a log factor. Positivity makes the window sum grow with the right end and shrink when the left end moves right. Two pointers therefore suffice: add on the right, and while the sum is at least target, shrink from the left and record the shortest length.

We can use two pointers j and i to maintain a window, where the sum of all elements in the window is less than target. Initially, j = 0, and the answer ans = n + 1, where n is the length of the array nums.

Next, the pointer i starts to move to the right from 0, moving one step each time. We add the element corresponding to the pointer i to the window and update the sum of the elements in the window. If the sum of the elements in the window is greater than or equal to target, it means that the current subarray satisfies the condition, and we can update the answer, i.e., ans = min(ans, i - j + 1). Then we continuously remove the element nums[j] from the window until the sum of the elements in the window is less than target, and then repeat the above process.

Finally, if ans ≤ n, it means that there exists a subarray that satisfies the condition, return ans, otherwise return 0.

The time complexity is O(n), and the space complexity is O(1). Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def minSubArrayLen(self, target: int, nums: List[int]) -> int:
        l = s = 0
        ans = inf
        for r, x in enumerate(nums):
            s += x
            while s >= target:
                ans = min(ans, r - l + 1)
                s -= nums[l]
                l += 1
        return 0 if ans == inf else ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The numbers can be negative (shortest subarray with
# sum at least K)
# 2. Return where the shortest subarray starts and ends


# ========================================================
# FOLLOW-UP 1: The numbers can be negative (shortest
# subarray with sum at least K)
# ========================================================
```

```
# ---- follow-up 1: function ----
# Answer: A plain window no longer works because adding a
# number can lower the sum. Use prefix sums and a deque of
# candidate start positions whose prefix sums increase:
# pop from the front while the subarray is long enough,
# and pop from the back any start that is no better than
# the new one.
# Time: O(n).  Space: O(n).
from collections import deque


def shortest_subarray_with_negatives(nums, k):
    prefix = [0]
    for x in nums:
        prefix.append(prefix[-1] + x)
    candidates = deque()
    best = len(nums) + 1
    for i in range(len(prefix)):
        while candidates and prefix[i] - prefix[candidates[0]] >= k:
            length = i - candidates.popleft()
            if length < best:
                best = length
        while candidates and prefix[candidates[-1]] >= prefix[i]:
            candidates.pop()
        candidates.append(i)
    if best > len(nums):
        return -1
    return best


# ============================================================
# FOLLOW-UP 2: Return where the shortest subarray starts
# and ends
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same window, but save the two positions whenever
# a shorter window is found.
# Time: O(n).  Space: O(1).
def shortest_subarray_positions(target, nums):
    left = 0
    total = 0
    best = None
    for right in range(len(nums)):
        total += nums[right]
        while total >= target:
            if best is None or right - left + 1 < best[1] - best[0] + 1:
                best = (left, right)
            total -= nums[left]
            left += 1
    return best
```

PATTERN 2 · SLIDING WINDOW · PROBLEM 4 OF 20

424. Longest Repeating Character Replacement

Medium · Hash Table, String, Sliding Window

leetcode.com/problems/longest-repeating-character-replacement

★ **TOP 7 PROBLEM** in Sliding Window: master this one first

PROBLEM

You are given a string `s` and an integer `k`. You can choose any character of the string and change it to any other uppercase English character. You can perform this operation at most `k` times.

Return *the length of the longest substring containing the same letter you can get after performing the above operations*.

Example 1:

```
Input: s = "ABAB", k = 2
Output: 4
Explanation: Replace the two 'A's with two 'B's or vice versa.
```

Example 2:

```
Input: s = "AABABBA", k = 1
Output: 4
Explanation: Replace the one 'A' in the middle with 'B' and form "AABBBBA".
The substring "BBBB" has the longest repeating letters, which is 4.
There may exists other ways to achieve this answer too.
```

Constraints:

- `1 <= s.length <= 10^5`
- `s` consists of only uppercase English letters.
- `0 <= k <= s.length`

SOLUTION (PYTHON 3)

Solution 1: Two Pointers

How to think about it

A window can be made uniform with at most k changes iff its length minus the majority count is at most k. Trying every pair of ends is O(n^2). Expand the right end while tracking the maximum frequency mx. When r-l+1-mx>k, shift the left end by one. The longest legal window has length n-l. There is no need to decrease mx on a left move: we only want the maximum length, and an overestimate of mx only makes the test stricter.

We use a hash table `cnt` to count the occurrence of each character in the string, and two pointers `l` and `r` to maintain a sliding window, such that the size of the window minus the count of the most frequent character does not exceed k.

We iterate through the string, updating the right boundary `r` of the window each time, updating the count of characters within the window, and updating the maximum count `mx` of the characters that have appeared. When the size of the window minus `mx` is greater than k, we need to shrink the left boundary `l` of the window, updating the count of characters within the window, until the size of the window minus `mx` is no longer greater than k.

Finally, the answer is n - l, where n is the length of the string.

The time complexity is O(n), and the space complexity is O(|Σ|). Where n is the length of the string, and |Σ| is the size of the character set. In this problem, the character set is uppercase English letters, so |Σ| = 26.

Python 3

```python
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        cnt = Counter()
        l = mx = 0
        for r, c in enumerate(s):
            cnt[c] += 1
            mx = max(mx, cnt[c])
            if r - l + 1 - mx > k:
                cnt[s[l]] -= 1
                l += 1
        return len(s) - l
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the longest window itself
# 2. Fewest replacements so that some window of a fixed
# length L is one repeated letter


# ============================================================
# FOLLOW-UP 1: Return the longest window itself
# ============================================================

# ---- follow-up 1: helper functions ----
def most_frequent(count):
    # the biggest number in a dictionary of counts
    best = 0
    for value in count.values():
        if value > best:
            best = value
    return best

# ---- follow-up 1: function ----
# Answer: Keep the window valid with a shrink loop: the
# window needs (length - count of its most frequent
# letter) replacements. Remember where the longest valid
# window started.
# Time: O(n * s), s = distinct letters.  Space: O(s).
def longest_replaced_window(s, k):
    count = {}
    left = 0
    best_start, best_length = 0, 0
    for right in range(len(s)):
        count[s[right]] = count.get(s[right], 0) + 1
        while right - left + 1 - most_frequent(count) > k:
            count[s[left]] -= 1
            left += 1
        if right - left + 1 > best_length:
            best_length = right - left + 1
            best_start = left
    return s[best_start:best_start + best_length]


# ============================================================
# FOLLOW-UP 2: Fewest replacements so that some window of
# a fixed length L is one repeated letter
# ============================================================

# ---- follow-up 2: function ----
# Answer: Slide a window of exactly L letters; for each
# window the cost is L minus its most frequent letter
# count. Take the smallest cost.
# Time: O(n * s).  Space: O(s).
def min_replacements_for_window(s, length):
```

```
    count = {}
    best = length
    for right in range(len(s)):
        count[s[right]] = count.get(s[right], 0) + 1
        if right >= length:
            count[s[right - length]] -= 1
        if right >= length - 1:
            cost = length - most_frequent(count)
            if cost < best:
```

```
                best = cost
    return best
```

PATTERN 2 · SLIDING WINDOW · PROBLEM 5 OF 20

567. Permutation in String

Medium · Hash Table, Two Pointers, String, Sliding Window

leetcode.com/problems/permutation-in-string

★ **TOP 7 PROBLEM** in Sliding Window: master this one first

Also appears in pattern 7 String Matching / Pattern Search

PROBLEM

Given two strings `s1` and `s2`, return `true` if `s2` contains a permutation of `s1`, or `false` otherwise.

In other words, return `true` if one of `s1`'s permutations is the substring of `s2`.

Example 1:

```
Input: s1 = "ab", s2 = "eidbaooo"
Output: true
Explanation: s2 contains one permutation of s1 ("ba").
```

Example 2:

```
Input: s1 = "ab", s2 = "eidboaoo"
Output: false
```

Constraints:

- `1 <= s1.length, s2.length <=` 10^4
- `s1` and `s2` consist of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Sliding Window

How to think about it

A permutation of s1 is a window of length |s1| with the same counts. Sorting every window is wasteful. A sliding window stores the remaining counts versus s1 and need is how many letters are still off. Add the right end, drop the left, and succeed when need=0. Fixed length, one scan.

We use an array cnt to record the characters and their counts that need to be matched, and a variable need to record the number of different characters that still need to be matched. Initially, cnt contains the character counts from the string s1, and need is the number of different characters in s1.

Then we traverse the string s2. For each character, we decrement its corresponding value in cnt. If the decremented value equals 0, it means the current character's count in s1 is satisfied, and we decrement need. If the current index i is greater than or equal to the length of s1, we need to increment the corresponding value in cnt for s2[i-s1]. If the incremented value equals 1, it means the current character's count in s1 is no longer satisfied, and we increment need. During the traversal, if the value of need equals 0, it means all character counts are satisfied, and we have found a valid substring, so we return true.

Otherwise, if the traversal ends without finding a valid substring, we return false.

The time complexity is O(m + n), where m and n are the lengths of strings s1 and s2, respectively. The space complexity is O(|Σ|), where Σ is the character set. In this problem, the character set is lowercase letters, so the space complexity is constant.

Python 3

```
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        cnt = Counter(s1)
        need = len(cnt)
        m = len(s1)
        for i, c in enumerate(s2):
            cnt[c] -= 1
            if cnt[c] == 0:
                need -= 1
            if i >= m:
                cnt[s2[i - m]] += 1
                if cnt[s2[i - m]] == 1:
                    need += 1
            if need == 0:
                return True
        return False
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the start of the first window that is a
# permutation of s1, or -1
# 2. Allow ONE letter of the window to be different from a
# permutation


# ============================================================
# FOLLOW-UP 1: Return the start of the first window that
# is a permutation of s1, or -1
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the letter counts of the current window and
# compare them with s1's counts at every position.
# Time: O(n * s), s = distinct letters (a table comparison
# per step).  Space: O(s).
def first_permutation_start(s1, s2):
    need = {}
    for ch in s1:
        need[ch] = need.get(ch, 0) + 1
    window = {}
    size = len(s1)
    for i in range(len(s2)):
        window[s2[i]] = window.get(s2[i], 0) + 1
        if i >= size:
            old = s2[i - size]
            window[old] -= 1
            if window[old] == 0:
                del window[old]
        if window == need:
            return i - size + 1
    return -1


# ============================================================
# FOLLOW-UP 2: Allow ONE letter of the window to be
# different from a permutation
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count how many letters of the window are surplus
# compared with s1 (the sum of the positive differences).
# A window is close enough when at most one letter is
# surplus.
# Time: O(n * s).  Space: O(s).
```

```
def has_near_permutation(s1, s2):
    need = {}
    for ch in s1:
        need[ch] = need.get(ch, 0) + 1
    window = {}
    size = len(s1)
    for i in range(len(s2)):
        window[s2[i]] = window.get(s2[i], 0) + 1
        if i >= size:
            window[s2[i - size]] -= 1
        if i >= size - 1:
            surplus = 0
            for ch in window:
                extra = window[ch] - need.get(ch, 0)
```

```
                if extra > 0:
                    surplus += extra
            if surplus <= 1:
                return True
    return False
```

PATTERN 2 · SLIDING WINDOW · PROBLEM 6 OF 20

438. Find All Anagrams in a String

Medium · Hash Table, String, Sliding Window

leetcode.com/problems/find-all-anagrams-in-a-string

★ **TOP 7 PROBLEM** in String Matching / Pattern Search: master this one first

Also appears in pattern 7 String Matching / Pattern Search

PROBLEM

Given two strings `s` and `p`, return an array of all the start indices of `p`'s anagrams in `s`. You may return the answer in **any order**.

Example 1:

```
Input: s = "cbaebabacd", p = "abc"
Output: [0,6]
Explanation:
The substring with start index = 0 is "cba", which is an anagram of "abc".
The substring with start index = 6 is "bac", which is an anagram of "abc".
```

Example 2:

```
Input: s = "abab", p = "ab"
Output: [0,1,2]
Explanation:
The substring with start index = 0 is "ab", which is an anagram of "ab".
The substring with start index = 1 is "ba", which is an anagram of "ab".
The substring with start index = 2 is "ab", which is an anagram of "ab".
```

Constraints:

- `1 <= s.length, p.length <= 3 * 10^4`
- `s` and `p` consist of lowercase English letters.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

An anagram is a same-length substring with the same letter counts. Sorting every window is too slow. Keep a window of length n=|p| and compare its counter with p's. Prefill n-1 characters, then on each step add the right character, compare, and drop the left one. The window length stays n, so only two 26-slot counters are compared.

Python 3

```python
class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        m, n = len(s), len(p)
        ans = []
        if m < n:
            return ans
        cnt1 = Counter(p)
        cnt2 = Counter(s[: n - 1])
        for i in range(n - 1, m):
            cnt2[s[i]] += 1
            if cnt1 == cnt2:
                ans.append(i - n + 1)
            cnt2[s[i - n + 1]] -= 1
        return ans
```

Complexity. Time: O(m * C), m = len(s), C = 26 (comparing two counters). Space: O(C).

Solution 2

> **How to think about it**
>
> Solution 1 compares the whole counter each time. After adding the right character, shrink the left end while any letter exceeds p's quota. When every letter is within quota and the length equals n, the window is an anagram. The O(|Σ|) equality test becomes a length check; the scan is still linear.

Python 3

```python
class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        m, n = len(s), len(p)
        ans = []
        if m < n:
            return ans
        cnt1 = Counter(p)
        cnt2 = Counter()
        j = 0
        for i, c in enumerate(s):
            cnt2[c] += 1
            while cnt2[c] > cnt1[c]:
                cnt2[s[j]] -= 1
                j += 1
            if i - j + 1 == n:
                ans.append(j)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Faster slide: O(1) work per step instead of comparing
# whole count tables
# 2. The text is a stream: report each start as soon as it
# is known


# ============================================================
# FOLLOW-UP 1: Faster slide: O(1) work per step instead of
# comparing whole count tables
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep one number, matches, the count of letters
# whose window count equals the pattern's count. Only the
# letter that enters and the letter that leaves can change
# it.
# Time: O(n).  Space: O(s), s = distinct letters.
def find_anagram_starts_fast(text, pattern):
    need = {}
    for ch in pattern:
        need[ch] = need.get(ch, 0) + 1
    window = {}
    matches = 0                                  # letters whose counts agree
    size = len(pattern)
    starts = []
    for i in range(len(text)):
        entering = text[i]
        if entering in need:
            window[entering] = window.get(entering, 0) + 1
            if window[entering] == need[entering]:
                matches += 1
            elif window[entering] == need[entering] + 1:
                matches -= 1                     # it was matching a moment ago
```

```
        if i >= size:
            leaving = text[i - size]
            if leaving in need:
                window[leaving] -= 1
                if window[leaving] == need[leaving]:
                    matches += 1
                elif window[leaving] == need[leaving] - 1:
                    matches -= 1
        if matches == len(need):
            starts.append(i - size + 1)
    return starts


# ========================================================
# FOLLOW-UP 2: The text is a stream: report each start as
# soon as it is known
# ========================================================

# ---- follow-up 2: function ----
# Answer: Use a generator that reads one character at a
# time and keeps only the window; nothing else is stored.
# Time: O(n * m), m = len(pattern).  Space: O(m).
def anagram_starts_stream(characters, pattern):
    need = {}
    for ch in pattern:
        need[ch] = need.get(ch, 0) + 1
```

```
    window = []                                 # the last len(pattern) characters
    position = -1
    for ch in characters:
        position += 1
        window.append(ch)
        if len(window) > len(pattern):
            window.pop(0)
        if len(window) == len(pattern):
            counts = {}
            for c in window:
                counts[c] = counts.get(c, 0) + 1
            if counts == need:
                yield position - len(pattern) + 1
```

PATTERN 2 · SLIDING WINDOW · PROBLEM 7 OF 20

904. Fruit Into Baskets

Medium · Array, Hash Table, Sliding Window

leetcode.com/problems/fruit-into-baskets

PROBLEM

You are visiting a farm that has a single row of fruit trees arranged from left to right. The trees are represented by an integer array `fruits` where `fruits[i]` is the **type** of fruit the i^{th} tree produces.

You want to collect as much fruit as possible. However, the owner has some strict rules that you must follow:

- You only have **two** baskets, and each basket can only hold a **single type** of fruit. There is no limit on the amount of fruit each basket can hold.
- Starting from any tree of your choice, you must pick **exactly one fruit** from **every** tree (including the start tree) while moving to the right. The picked fruits must fit in one of your baskets.
- Once you reach a tree with fruit that cannot fit in your baskets, you must stop.

Given the integer array `fruits`, return *the **maximum** number of fruits you can pick*.

Example 1:

```
Input: fruits = [1,2,1]
Output: 3
Explanation: We can pick from all 3 trees.
```

Example 2:

```
Input: fruits = [0,1,2,2]
Output: 3
Explanation: We can pick from trees [1,2,2].
If we had started at the first tree, we would only pick from trees [0,1].
```

Example 3:

```
Input: fruits = [1,2,3,2,2]
Output: 4
Explanation: We can pick from trees [2,3,2,2].
If we had started at the first tree, we would only pick from trees [1,2].
```

Constraints:

- `1 <= fruits.length <= 10^5`
- `0 <= fruits[i] < fruits.length`

SOLUTIONS (PYTHON 3)

Solution 1: Hash Table + Sliding Window

How to think about it

We need the longest subarray with at most two fruit types. n≤ 10^5, so enumerating endpoints and recounting types is too slow. After the right end absorbs a fruit, the left end must move if the window has more than two types. A counter tracks frequencies; the number of keys is the number of types. Shrink from the left while that exceeds 2, and keep the maximum window length.

We use a hash table cnt to maintain the types and corresponding quantities of fruits in the current window, and use two pointers j and i to maintain the left and right boundaries of the window.

We traverse the fruits array, add the current fruit x to the window, i.e., cnt[x]++, then judge whether the types of fruits in the current window exceed 2. If it exceeds 2, we need to move the left boundary j of the window to the right until the types of

fruits in the window do not exceed 2. Then we update the answer, i.e., ans = max(ans, i - j + 1).

After the traversal ends, we can get the final answer.

```
1 2 3 2 2 1 4
^   ^
j   i


1 2 3 2 2 1 4
  ^ ^
  j i


1 2 3 2 2 1 4
  ^     ^
  j     i
```

The time complexity is O(n), and the space complexity is O(1). Here, n is the length of the fruits array.

Python 3

```python
class Solution:
    def totalFruit(self, fruits: List[int]) -> int:
        cnt = Counter()
        ans = j = 0
        for i, x in enumerate(fruits):
            cnt[x] += 1
            while len(cnt) > 2:
                y = fruits[j]
                cnt[y] -= 1
                if cnt[y] == 0:
                    cnt.pop(y)
                j += 1
            ans = max(ans, i - j + 1)
        return ans
```

Solution 2: Monotonic Variable-Length Sliding Window

> **How to think about it**
>
> Method 1 shrinks the window and updates the answer on every step, yet only the maximum length is required, so shorter historical windows can be ignored. Grow the window monotonically: when a third type appears, advance the left end once. The final n-j is the largest feasible window.

In Solution 1, we find that the window size sometimes increases and sometimes decreases, which requires us to update the answer each time.

But what this problem actually asks for is the maximum number of fruits, that is, the "largest" window. We don't need to shrink the window, we just need to let the window monotonically increase. So the code omits the operation of updating the answer each time, and only needs to return the size of the window as the answer after the traversal ends.

The time complexity is O(n), and the space complexity is O(n), where n is the length of the fruits array.

Python 3

```
class Solution:
    def totalFruit(self, fruits: List[int]) -> int:
        cnt = Counter()
        j = 0
        for x in fruits:
            cnt[x] += 1
            if len(cnt) > 2:
                y = fruits[j]
                cnt[y] -= 1
                if cnt[y] == 0:
                    cnt.pop(y)
                j += 1
        return len(fruits) - j
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. K baskets instead of two
# 2. Return the first and last tree of the best stretch


# ============================================================
# FOLLOW-UP 1: K baskets instead of two
# ============================================================

# ---- follow-up 1: function ----
# Answer: The window may hold at most K different fruit
# types: shrink from the left while it holds more.
# Time: O(n).  Space: O(k).
def total_fruit_k_baskets(fruits, k):
    count = {}
    left = 0
    best = 0
    for right in range(len(fruits)):
        count[fruits[right]] = count.get(fruits[right], 0) + 1
        while len(count) > k:
            count[fruits[left]] -= 1
            if count[fruits[left]] == 0:
                del count[fruits[left]]
            left += 1
        if right - left + 1 > best:
            best = right - left + 1
    return best


# ============================================================
# FOLLOW-UP 2: Return the first and last tree of the best
# stretch
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same window; store the two positions whenever
# the window beats the best length so far.
# Time: O(n).  Space: O(1).
def best_stretch(fruits):
    count = {}
    left = 0
    best = (0, -1)                                  # (first tree, last tree)
    for right in range(len(fruits)):
        count[fruits[right]] = count.get(fruits[right], 0) + 1
        while len(count) > 2:
            count[fruits[left]] -= 1
            if count[fruits[left]] == 0:
                del count[fruits[left]]
            left += 1
```

```
        if right - left > best[1] - best[0]:
            best = (left, right)
    return best
```

PATTERN 2 · SLIDING WINDOW · PROBLEM 8 OF 20

1004. Max Consecutive Ones III

Medium · Array, Binary Search, Prefix Sum, Sliding Window

leetcode.com/problems/max-consecutive-ones-iii

★ **TOP 7 PROBLEM** in Sliding Window: master this one first

PROBLEM

Given a binary array `nums` and an integer `k`, return *the maximum number of consecutive* `1`*'s in the array if you can flip at most* `k` `0`'s.

Example 1:

```
Input: nums = [1,1,1,0,0,0,1,1,1,1,0], k = 2
Output: 6
Explanation: [1,1,1,0,0,1,1,1,1,1,1]
Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.
```

Example 2:

```
Input: nums = [0,0,1,1,0,0,1,1,1,0,1,1,0,0,0,1,1,1,1], k = 3
Output: 10
Explanation: [0,0,1,1,1,1,1,1,1,1,1,1,0,0,0,1,1,1,1]
Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.
```

Constraints:

- `1 <= nums.length <= 10^5`
- `nums[i]` is either 0 or 1.
- `0 <= k <= nums.length`

SOLUTION (PYTHON 3)

Solution 1: Sliding Window

How to think about it

Checking every subarray for its number of zeros is O(n^2), which fails for n≤ 10^5. Flipping at most k zeros to maximize consecutive ones is the same as the longest window that contains at most k zeros. When the right end advances and the zero count exceeds k, the left end must advance to restore feasibility. Only the maximum length is required, so the window is allowed to grow monotonically: the left end moves at most one step per iteration. We keep l and cnt for the current window. After the right end visits every index, n-l is the length of the longest feasible window.

We can iterate through the array, using a variable cnt to record the current number of 0s in the window. When cnt > k, we move the left boundary of the window to the right by one position.

After the iteration ends, the length of the window is the maximum number of consecutive 1s.

Note that in the process above, we do not need to loop to move the left boundary of the window to the right. Instead, we directly move the left boundary to the right by one position. This is because the problem asks for the maximum number of consecutive 1s, so the length of the window will only increase, not decrease. Therefore, we do not need to loop to move the left boundary to the right.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Similar problems:

- 487. Max Consecutive Ones II
- 2024. Maximize the Confusion of an Exam

Python 3

```python
class Solution:
    def longestOnes(self, nums: List[int], k: int) -> int:
        l = cnt = 0
        for x in nums:
            cnt += x ^ 1
            if cnt > k:
                cnt -= nums[l] ^ 1
                l += 1
        return len(nums) - l
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return where the longest window is
# 2. Numbers arrive one by one: report the best length
# after each


# ============================================================
# FOLLOW-UP 1: Return where the longest window is
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same window with at most k zeros; remember the
# start and end of the longest one.
# Time: O(n).  Space: O(1).
def longest_ones_positions(nums, k):
    zeros = 0
    left = 0
    best = (0, -1)
    for right in range(len(nums)):
        if nums[right] == 0:
            zeros += 1
        while zeros > k:
            if nums[left] == 0:
                zeros -= 1
            left += 1
        if right - left > best[1] - best[0]:
            best = (left, right)
    return best


# ============================================================
# FOLLOW-UP 2: Numbers arrive one by one: report the best
# length after each
# ============================================================

# ---- follow-up 2: function ----
# Answer: The window only ever moves forward, so one
# object that keeps the zero count and the left end can
# answer after every new number.
# Time: O(1) amortised per number.  Space: O(n).
class LongestOnesStream:
    def __init__(self, k):
        self.k = k
        self.items = []
        self.left = 0
        self.zeros = 0
        self.best = 0

    def add(self, value):
        self.items.append(value)
        if value == 0:
            self.zeros += 1
        while self.zeros > self.k:
```

```
            if self.items[self.left] == 0:
                self.zeros -= 1
            self.left += 1
        length = len(self.items) - self.left
        if length > self.best:
            self.best = length
        return self.best
```

PATTERN 2 · SLIDING WINDOW · PROBLEM 9 OF 20

713. Subarray Product Less Than K

Medium · Array, Binary Search, Prefix Sum, Sliding Window

leetcode.com/problems/subarray-product-less-than-k

Also appears in pattern 35 Sliding Window Maximum / Minimum (Specific Variants)

PROBLEM

You are given an array of integers `nums` and an integer `k`.

Return the number of contiguous subarrays where the **product** of all the elements in the **subarray** is **strictly less** than `k`.

Example 1:

```
Input: nums = [10,5,2,6], k = 100
Output: 8
Explanation: The 8 subarrays that have product less than 100 are:
[10], [5], [2], [6], [10, 5], [5, 2], [2, 6], [5, 2, 6]
Note that [10, 5, 2] is not included as the product of 100 is not strictly less than k.
```

Example 2:

```
Input: nums = [1,2,3], k = 0
Output: 0
```

Constraints:

- `1 <= nums.length <= 3 * 10^4`
- `1 <= nums[i] <= 1000`
- `0 <= k <= 10^6`

SOLUTION (PYTHON 3)

Solution 1: Two Pointers

How to think about it

Count contiguous subarrays whose product is strictly less than k. n can be 3× 10^4, so enumerating both ends is O(n^2). All values are positive, so the window product is monotone as the right end grows. Once it reaches k, only advancing the left end restores a legal window—each pointer moves at most once. Keep product p and left l. After multiplying by x, divide out nums[l] while p≥ k. Then r-l+1 subarrays end at r. One pass suffices.

We can use two pointers to maintain a sliding window, where the product of all elements in the window is less than k.

Define two pointers l and r pointing to the left and right boundaries of the sliding window, initially l = r = 0. We use a variable p to record the product of all elements in the window, initially p = 1.

Each time, we move r one step to the right, adding the element x pointed to by r to the window, and update p = p × x. Then, if p ≥ k, we move l one step to the right in a loop and update p = p ÷ nums[l] until p < k or l > r. Thus, the number of contiguous subarrays ending at r with a product less than k is r - l + 1. We then add this number to the answer and continue moving r until r reaches the end of the array.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Python 3

```
class Solution:
    def numSubarrayProductLessThanK(self, nums: List[int], k: int) -> int:
        ans = l = 0
        p = 1
        for r, x in enumerate(nums):
            p *= x
            while l <= r and p >= k:
                p //= nums[l]
                l += 1
            ans += r - l + 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the length of the longest subarray whose
# product is less than k
# 2. List the subarrays themselves


# ============================================================
# FOLLOW-UP 1: Return the length of the longest subarray
# whose product is less than k
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same window: shrink while the product is too big
# and remember the longest window.
# Time: O(n).  Space: O(1).
def longest_product_below(nums, k):
    if k <= 1:
        return 0
    product = 1
    left = 0
    best = 0
    for right in range(len(nums)):
        product *= nums[right]
        while product >= k:
            product //= nums[left]
            left += 1
        if right - left + 1 > best:
            best = right - left + 1
    return best


# ============================================================
# FOLLOW-UP 2: List the subarrays themselves
# ============================================================

# ---- follow-up 2: function ----
# Answer: For each right end, every start from left to
# right works, so the subarrays ending there are
# nums[left..right], nums[left+1..right], ... (the output
# itself can be large).
# Time: O(n^2) in the worst case (the output itself can be
# that large).  Space: O(n^2) for the output.
def products_below_k_subarrays(nums, k):
    result = []
    if k <= 1:
        return result
    product = 1
    left = 0
    for right in range(len(nums)):
        product *= nums[right]
        while product >= k:
```

```
            product //= nums[left]
            left += 1
        for start in range(left, right + 1):
            result.append(nums[start:right + 1])
    return result
```

PATTERN 2 · SLIDING WINDOW · PROBLEM 10 OF 20

1493. Longest Subarray of 1's After Deleting One Element

Medium · Array, Dynamic Programming, Sliding Window

leetcode.com/problems/longest-subarray-of-1s-after-deleting-one-element

PROBLEM

You are given a binary array `nums`, you should delete **one** element from it.

Return the size of the **longest non-empty subarray** containing only 1's in the resulting array. Return 0 if there is no such **subarray**.

Example 1:

```
Input: nums = [1,1,0,1]
Output: 3
Explanation: After deleting the number in position 2, [1,1,1] contains 3 numbers with value of 1's.
```

Example 2:

```
Input: nums = [0,1,1,1,0,1,1,0,1]
Output: 5
Explanation: After deleting the number in position 4, [0,1,1,1,1,1,0,1] longest subarray with value of
1's is [1,1,1,1,1].
```

Example 3:

```
Input: nums = [1,1,1]
Output: 2
Explanation: You must delete one element.
```

Constraints:

- `1 <= nums.length <= 10^5`
- `nums[i]` is either 0 or 1.

SOLUTIONS (PYTHON 3)

Solution 1: Enumeration

> **How to think about it**
>
> Exactly one element is deleted, so the answer is the ones immediately left of i plus those immediately right. n≤ 10^5. Precompute longest ones ending before i and starting after i, then maximize over the deleted index.

We can enumerate each position i to be deleted, then calculate the number of consecutive 1s on the left and right, and finally take the maximum value.

Specifically, we use two arrays left and right of length n+1, where left[i] represents the number of consecutive 1s ending with nums[i-1], and right[i] represents the number of consecutive 1s starting with nums[i].

The final answer is $\max_{0 \le i < n} \{left[i] + right[i+1]\}$.

The time complexity is O(n), and the space complexity is O(n). Where n is the length of the array nums.

Python 3

```python
class Solution:
    def longestSubarray(self, nums: List[int]) -> int:
        n = len(nums)
        left = [0] * (n + 1)
        right = [0] * (n + 1)
        for i, x in enumerate(nums, 1):
            if x:
                left[i] = left[i - 1] + 1
        for i in range(n - 1, -1, -1):
            if nums[i]:
                right[i] = right[i + 1] + 1
        return max(left[i] + right[i + 1] for i in range(n))
```

Solution 2: Two Pointers

> **How to think about it**
>
> Method 1 stores two arrays. Equivalently, find the longest window with at most one 0; after deleting that 0 (or one 1) the length is i-j. Expand the right end and shrink while the zero count exceeds 1.

The problem is actually asking us to find the longest subarray that contains at most one 0. The remaining length after deleting one element from this subarray is the answer.

Therefore, we can use two pointers j and i to point to the left and right boundaries of the subarray, initially j = 0, i = 0. In addition, we use a variable cnt to record the number of 0s in the subarray.

Next, we move the right pointer i. If nums[i] = 0, then cnt is incremented by 1. When cnt > 1, we need to move the left pointer j until cnt ≤ 1. Then, we update the answer, i.e., ans = max(ans, i - j). Continue to move the right pointer i until i reaches the end of the array.

The time complexity is O(n), where n is the length of the array nums. The space complexity is O(1).

Python 3

```python
class Solution:
    def longestSubarray(self, nums: List[int]) -> int:
        ans = 0
        cnt = j = 0
        for i, x in enumerate(nums):
            cnt += x ^ 1
            while cnt > 1:
                cnt -= nums[j] ^ 1
                j += 1
            ans = max(ans, i - j)
        return ans
```

Solution 3: Two Pointers (Optimization)

> **How to think about it**
>
> Method 2 may shrink the window when zeros exceed one. We only need the maximum length, so move the left end once whenever the count is too large and keep the window non-decreasing. The answer is n-l-1.

In Solution 2, we move the left pointer in a loop until cnt ≤ 1. Since the problem asks for the longest subarray, it means we don't need to reduce the length of the subarray. Therefore, if cnt > 1, we only move the left pointer once, and the right pointer continues to move to the right. This ensures that the length of the subarray does not decrease.

Finally, the answer we return is n - l - 1, where l is the position of the left pointer.

The time complexity is O(n), where n is the length of the array nums. The space complexity is O(1).

Python 3

```python
class Solution:
    def longestSubarray(self, nums: List[int]) -> int:
        cnt = l = 0
        for x in nums:
            cnt += x ^ 1
            if cnt > 1:
                cnt -= nums[l] ^ 1
                l += 1
        return len(nums) - l - 1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Delete up to K zeros instead of one element
# 2. O(1) memory with no window: remember only the
# previous and the current run of ones


# ============================================================
# FOLLOW-UP 1: Delete up to K zeros instead of one element
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep a window with at most K zeros; after
# deleting those zeros the run of ones has (window length
# - zeros in window) elements.
# Time: O(n).  Space: O(1).
def longest_ones_after_deleting_zeros(nums, k):
    zeros = 0
    left = 0
    best = 0
    for right in range(len(nums)):
        if nums[right] == 0:
            zeros += 1
        while zeros > k:
            if nums[left] == 0:
                zeros -= 1
            left += 1
        ones = right - left + 1 - zeros
        if ones > best:
            best = ones
    return best


# ============================================================
# FOLLOW-UP 2: O(1) memory with no window: remember only
# the previous and the current run of ones
# ============================================================

# ---- follow-up 2: function ----
# Answer: A deleted zero joins the run before it and the
# run after it, so the answer is the best sum of two
# neighbouring runs.
# Time: O(n).  Space: O(1).
def longest_after_deleting_one_runs(nums):
    previous = 0
    current = 0
    best = 0
    saw_zero = False
    for x in nums:
        if x == 1:
            current += 1
        else:
            saw_zero = True
            if previous + current > best:
```

```
                best = previous + current
            previous = current
            current = 0
    if previous + current > best:
        best = previous + current
    if not saw_zero:
        return len(nums) - 1                    # one element must still be deleted
```

```
    return best
```

PATTERN 2 · SLIDING WINDOW · PROBLEM 11 OF 20

1456. Maximum Number of Vowels in a Substring of Given Length

Medium · String, Sliding Window

leetcode.com/problems/maximum-number-of-vowels-in-a-substring-of-given-length

PROBLEM

Given a string `s` and an integer `k`, return *the maximum number of vowel letters in any substring of* `s` *with length* `k`.

Vowel letters in English are `'a'`, `'e'`, `'i'`, `'o'`, and `'u'`.

Example 1:

```
Input: s = "abciiidef", k = 3
Output: 3
Explanation: The substring "iii" contains 3 vowel letters.
```

Example 2:

```
Input: s = "aeiou", k = 2
Output: 2
Explanation: Any substring of length 2 contains 2 vowels.
```

Example 3:

```
Input: s = "leetcode", k = 3
Output: 2
Explanation: "lee", "eet" and "ode" contain 2 vowels.
```

Constraints:

- `1 <= s.length <= 10^5`
- `s` consists of lowercase English letters.
- `1 <= k <= s.length`

SOLUTION (PYTHON 3)

Solution 1: Sliding Window

> **How to think about it**
>
> We want the most vowels in any length-k window. n≤ 10^5, so count the first k characters, then add/remove one character as the window slides.

First, we count the number of vowels in the first k characters, denoted as cnt, and initialize the answer ans as cnt.

Then we start traversing the string from k. For each iteration, we add the current character to the window. If the current character is a vowel, we increment cnt. We remove the first character from the window. If the removed character is a vowel, we decrement cnt. Then, we update the answer ans = max(ans, cnt).

After the traversal, we return the answer.

The time complexity is O(n), where n is the length of the string s. The space complexity is O(1).

Python 3

```python
class Solution:
    def maxVowels(self, s: str, k: int) -> int:
        vowels = set("aeiou")
        ans = cnt = sum(c in vowels for c in s[:k])
        for i in range(k, len(s)):
            cnt += int(s[i] in vowels) - int(s[i - k] in vowels)
            ans = max(ans, cnt)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the substring with the most vowels
# 2. Count the windows of length k that contain at least m
# vowels


# ============================================================
# FOLLOW-UP 1: Return the substring with the most vowels
# ============================================================

# ---- follow-up 1: function ----
# Answer: Slide the fixed window, add the new letter and
# remove the old one, and remember where the best count
# happened.
# Time: O(n).  Space: O(1).
def substring_with_most_vowels(s, k):
    vowels = "aeiou"
    count = 0
    best_count = -1
    best_start = 0
    for i in range(len(s)):
        if s[i] in vowels:
            count += 1
        if i >= k and s[i - k] in vowels:
            count -= 1
        if i >= k - 1 and count > best_count:
            best_count = count
            best_start = i - k + 1
    return s[best_start:best_start + k]


# ============================================================
# FOLLOW-UP 2: Count the windows of length k that contain
# at least m vowels
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same slide, and add one every time the vowel
# count reaches m.
# Time: O(n).  Space: O(1).
def windows_with_at_least_vowels(s, k, m):
    vowels = "aeiou"
    count = 0
    windows = 0
    for i in range(len(s)):
        if s[i] in vowels:
            count += 1
        if i >= k and s[i - k] in vowels:
            count -= 1
        if i >= k - 1 and count >= m:
            windows += 1
    return windows
```

PATTERN 2 · SLIDING WINDOW · PROBLEM 12 OF 20

1248. Count Number of Nice Subarrays

Medium · Array, Hash Table, Math, Prefix Sum, Sliding Window

leetcode.com/problems/count-number-of-nice-subarrays

Also appears in pattern 5 Prefix Sum

PROBLEM

Given an array of integers `nums` and an integer `k`. A continuous subarray is called **nice** if there are `k` odd numbers on it.

Return *the number of **nice** sub-arrays*.

Example 1:

```
Input: nums = [1,1,2,1,1], k = 3
Output: 2
Explanation: The only sub-arrays with 3 odd numbers are [1,1,2,1] and [1,2,1,1].
```

Example 2:

```
Input: nums = [2,4,6], k = 1
Output: 0
Explanation: There are no odd numbers in the array.
```

Example 3:

```
Input: nums = [2,2,2,1,2,2,1,2,2,2], k = 2
Output: 16
```

Constraints:

- `1 <= nums.length <= 50000`
- `1 <= nums[i] <= 10^5`
- `1 <= k <= nums.length`

SOLUTION (PYTHON 3)

Solution 1: Prefix Sum + Array or Hash Table

> **How to think about it**
>
> We want subarrays with exactly k odds, n ≤ 5× 10^4. Treating odds as 1 and evens as 0, these are intervals whose prefix sums differ by k. While scanning we keep the odd count t, look up how often t-k has appeared, then record t. The prefix sum turns “exactly k odds” into one map query.

The problem asks for the number of subarrays that contain exactly k odd numbers. We can calculate the number of odd numbers t in each prefix array and record it in an array or hash table cnt. For each prefix array, we only need to find the number of prefix arrays with t-k odd numbers, which is the number of subarrays ending with the current prefix array.

The time complexity is O(n), and the space complexity is O(n). Where n is the length of the array nums.

Python 3

```python
class Solution:
    def numberOfSubarrays(self, nums: List[int], k: int) -> int:
        cnt = Counter({0: 1})
        ans = t = 0
        for v in nums:
            t += v & 1
            ans += cnt[t - k]
            cnt[t] += 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Constant-extra-space counting from the positions of
# the odd numbers
# 2. Longest subarray with exactly k odd numbers


# ============================================================
# FOLLOW-UP 1: Constant-extra-space counting from the
# positions of the odd numbers
# ============================================================

# ---- follow-up 1: function ----
# Answer: Write down the index of every odd number. A
# subarray with exactly k odd numbers uses k neighbouring
# odd positions; the number of ways to choose its left end
# is the gap before the first one, and its right end the
# gap after the last one.
# Time: O(n).  Space: O(n).
def nice_subarrays_by_positions(nums, k):
    odds = [-1]                                  # a sentinel before the array
    for i in range(len(nums)):
        if nums[i] % 2 == 1:
            odds.append(i)
    odds.append(len(nums))                       # a sentinel after the array
    count = 0
    for i in range(1, len(odds) - k):
        left_choices = odds[i] - odds[i - 1]
        right_choices = odds[i + k] - odds[i + k - 1]
        count += left_choices * right_choices
    return count


# ============================================================
# FOLLOW-UP 2: Longest subarray with exactly k odd numbers
# ============================================================

# ---- follow-up 2: function ----
# Answer: Use the same positions: the subarray can stretch
# from just after the odd number before the group of k to
# just before the odd number after it.
# Time: O(n).  Space: O(n).
def longest_nice_subarray(nums, k):
    odds = [-1]
    for i in range(len(nums)):
        if nums[i] % 2 == 1:
            odds.append(i)
    odds.append(len(nums))
    best = 0
    for i in range(1, len(odds) - k):
        length = odds[i + k] - odds[i - 1] - 1
        if length > best:
            best = length
    return best
```

PATTERN 2 · SLIDING WINDOW · PROBLEM 13 OF 20

930. Binary Subarrays With Sum

Medium · Array, Hash Table, Prefix Sum, Sliding Window

leetcode.com/problems/binary-subarrays-with-sum

Also appears in pattern 5 Prefix Sum

PROBLEM

Given a binary array `nums` and an integer `goal`, return *the number of non-empty **subarrays** with a sum* `goal`.

A **subarray** is a contiguous part of the array.

Example 1:

```
Input: nums = [1,0,1,0,1], goal = 2
Output: 4
Explanation: The 4 subarrays are bolded and underlined below:
[1,0,1,0,1]
[1,0,1,0,1]
[1,0,1,0,1]
[1,0,1,0,1]
```

Example 2:

```
Input: nums = [0,0,0,0,0], goal = 0
Output: 15
```

Constraints:

- `1 <= nums.length <= 3 * 10^4`
- `nums[i]` is either `0` or `1`.
- `0 <= goal <= nums.length`

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Count subarrays of a 0/1 array whose sum is goal. n is large, so enumerating intervals is too slow. When the prefix sum is s, the number of earlier prefixes equal to s-goal is the number of good subarrays ending here. A counter of prefix frequencies, starting at cnt[0]=1, answers each step in O(1).

Python 3

```python
class Solution:
    def numSubarraysWithSum(self, nums: List[int], goal: int) -> int:
        cnt = Counter({0: 1})
        ans = s = 0
        for v in nums:
            s += v
            ans += cnt[s - goal]
            cnt[s] += 1
        return ans
```

Solution 2

How to think about it

Method 1 uses linear extra space. Because the array is non-negative, the window sum is monotone in the left end. Two left pointers keep the first index whose sum exceeds goal and the first whose sum is at least goal; their difference is the number of windows with sum exactly goal.

Python 3

```python
class Solution:
    def numSubarraysWithSum(self, nums: List[int], goal: int) -> int:
        i1 = i2 = s1 = s2 = j = ans = 0
        n = len(nums)
        while j < n:
            s1 += nums[j]
            s2 += nums[j]
            while i1 <= j and s1 > goal:
                s1 -= nums[i1]
                i1 += 1
            while i2 <= j and s2 >= goal:
                s2 -= nums[i2]
                i2 += 1
            ans += i2 - i1
            j += 1
        return ans
```

Complexity. Time: O(n). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Longest subarray whose sum equals the goal
# 2. Use a sliding window: count the subarrays with sum AT
# MOST the goal, then subtract


# ============================================================
# FOLLOW-UP 1: Longest subarray whose sum equals the goal
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the first index of every prefix sum; the
# earliest earlier prefix gives the longest subarray.
# Time: O(n).  Space: O(n).
def longest_binary_subarray_with_sum(nums, goal):
    first_index = {0: -1}
    total = 0
    best = 0
    for i in range(len(nums)):
        total += nums[i]
        if total - goal in first_index and i - first_index[total - goal] > best:
            best = i - first_index[total - goal]
        if total not in first_index:
            first_index[total] = i
    return best


# ============================================================
# FOLLOW-UP 2: Use a sliding window: count the subarrays
# with sum AT MOST the goal, then subtract
# ============================================================

# ---- follow-up 2: function ----
# Answer: A window can count 'at most' because every
# shorter start also works; exactly = at most(goal) - at
# most(goal - 1). It needs the numbers to be 0 or 1 (never
```

```
# negative).
# Time: O(n).  Space: O(1).
def count_sum_at_most(nums, goal):
    if goal < 0:
        return 0
    left = 0
    total = 0
    count = 0
    for right in range(len(nums)):
        total += nums[right]
        while total > goal:
            total -= nums[left]
            left += 1
        count += right - left + 1
    return count
```

PATTERN 2 · SLIDING WINDOW · PROBLEM 14 OF 20

1695. Maximum Erasure Value

Medium · Array, Hash Table, Sliding Window

leetcode.com/problems/maximum-erasure-value

Also appears in pattern 35 Sliding Window Maximum / Minimum (Specific Variants)

PROBLEM

You are given an array of positive integers `nums` and want to erase a subarray containing **unique elements**. The **score** you get by erasing the subarray is equal to the **sum** of its elements.

Return *the **maximum score** you can get by erasing **exactly one** subarray.*

An array `b` is called to be a subarray of `a` if it forms a contiguous subsequence of `a`, that is, if it is equal to `a[l],a[l+1],...,a[r]` for some `(l,r)`.

Example 1:

```
Input: nums = [4,2,4,5,6]
Output: 17
Explanation: The optimal subarray here is [2,4,5,6].
```

Example 2:

```
Input: nums = [5,2,1,2,5,2,1,2,5]
Output: 8
Explanation: The optimal subarray here is [5,2,1] or [1,2,5].
```

Constraints:

- `1 <= nums.length <= 10^5`
- `1 <= nums[i] <= 10^4`

SOLUTIONS (PYTHON 3)

Solution 1: Array or Hash Table + Prefix Sum

How to think about it

We want the maximum sum of a contiguous subarray of distinct values. n is 10^5; the last-seen index pushes the left end j past duplicates, and prefix sums give the range sum. Array d stores the latest index of each value. On v, set j=max(j,d[v]) and update with s[i]-s[j].

We use an array or hash table d to record the last occurrence position of each number, and use a prefix sum array s to record the sum from the starting point to the current position. We use a variable j to record the left endpoint of the current non-repeating subarray.

We iterate through the array. For each number v, if d[v] exists, we update j to max(j, d[v]), which ensures that the current non-repeating subarray does not contain v. Then we update the answer to max(ans, s[i] - s[j]), and finally update d[v] to i.

The time complexity is O(n), and the space complexity is O(n), where n is the length of the array nums.

Python 3

```python
class Solution:
    def maximumUniqueSubarray(self, nums: List[int]) -> int:
        d = [0] * (max(nums) + 1)
        s = list(accumulate(nums, initial=0))
        ans = j = 0
        for i, v in enumerate(nums, 1):
            j = max(j, d[v])
            ans = max(ans, s[i] - s[j])
            d[v] = i
        return ans
```

Solution 2: Two Pointers (Sliding Window)

> **How to think about it**
>
> Solution 1 uses prefix sums and an index table. A sliding window can store the live distinct sum: shrink from the left on a duplicate and skip the prefix array.

The problem is essentially asking us to find the longest subarray where all elements are distinct. We can use two pointers i and j to point to the left and right boundaries of the subarray, initially i = 0 and j = 0. Additionally, we use a hash table vis to record the elements in the subarray.

We iterate through the array. For each number x, if x is in vis, we continuously remove nums[i] from vis until x is no longer in vis. This way, we find a subarray that contains no duplicate elements. We add x to vis, update the subarray sum s, and then update the answer ans = max(ans, s).

After the iteration, we can get the maximum subarray sum.

The time complexity is O(n), and the space complexity is O(n), where n is the length of the array nums.

Python 3

```python
class Solution:
    def maximumUniqueSubarray(self, nums: List[int]) -> int:
        vis = set()
        ans = s = i = 0
        for x in nums:
            while x in vis:
                y = nums[i]
                s -= y
                vis.remove(y)
                i += 1
            vis.add(x)
            s += x
            ans = max(ans, s)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subarray itself
# 2. The numbers may be negative


# ============================================================
# FOLLOW-UP 1: Return the subarray itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same window of unique numbers with a running
# sum; keep the positions of the best sum.
# Time: O(n).  Space: O(n).
def best_unique_subarray(nums):
```

```
    in_window = set()
    left = 0
    total = 0
    best_sum, best_range = 0, (0, -1)
    for right in range(len(nums)):
        while nums[right] in in_window:
            in_window.remove(nums[left])
            total -= nums[left]
            left += 1
        in_window.add(nums[right])
        total += nums[right]
        if total > best_sum:
            best_sum = total
            best_range = (left, right)
    return best_range


# ============================================================
# FOLLOW-UP 2: The numbers may be negative
# ============================================================

# ---- follow-up 2: function ----
# Answer: The best subarray ending at a position may now
# be shorter than the whole unique window. With prefix
# sums the sum of nums[s..i] is prefix[i + 1] - prefix[s],
# so keep a deque of the start positions s that are still
# allowed, ordered by increasing prefix value; the front
# is the best start.
# Time: O(n).  Space: O(n).
from collections import deque


def max_unique_sum_with_negatives(nums):
    prefix = [0]
    for x in nums:
        prefix.append(prefix[-1] + x)
    last_index = {}
    window_start = 0
    starts = deque()                                # allowed starts, prefix values increasing
    best = None
    for i in range(len(nums)):
        if nums[i] in last_index and last_index[nums[i]] >= window_start:
            window_start = last_index[nums[i]] + 1
        last_index[nums[i]] = i
        while starts and prefix[starts[-1]] >= prefix[i]:
            starts.pop()
```

```
        starts.append(i)
        while starts[0] < window_start:
            starts.popleft()
        value = prefix[i + 1] - prefix[starts[0]]
        if best is None or value > best:
            best = value
    return best
```

1052. Grumpy Bookstore Owner

Medium · Array, Sliding Window

leetcode.com/problems/grumpy-bookstore-owner

Also appears in pattern 35 Sliding Window Maximum / Minimum (Specific Variants)

PROBLEM

There is a bookstore owner that has a store open for `n` minutes. You are given an integer array `customers` of length `n` where `customers[i]` is the number of the customers that enter the store at the start of the i^{th} minute and all those customers leave after the end of that minute.

During certain minutes, the bookstore owner is grumpy. You are given a binary array grumpy where `grumpy[i]` is `1` if the bookstore owner is grumpy during the i^{th} minute, and is `0` otherwise.

When the bookstore owner is grumpy, the customers entering during that minute are not **satisfied**. Otherwise, they are satisfied.

The bookstore owner knows a secret technique to remain **not grumpy** for `minutes` consecutive minutes, but this technique can only be used **once**.

Return the **maximum** number of customers that can be *satisfied* throughout the day.

Example 1:

Input: customers = [1,0,1,2,1,1,7,5], grumpy = [0,1,0,1,0,1,0,1], minutes = 3

Output: 16

Explanation:

The bookstore owner keeps themselves not grumpy for the last 3 minutes.

The maximum number of customers that can be satisfied = 1 + 1 + 1 + 1 + 7 + 5 = 16.

Example 2:

Input: customers = [1], grumpy = [0], minutes = 1

Output: 1

Constraints:

- `n == customers.length == grumpy.length`
- `1 <= minutes <= n <= 2 * 10^4`
- `0 <= customers[i] <= 1000`
- `grumpy[i]` is either `0` or `1`.

SOLUTION (PYTHON 3)

Solution 1: Sliding Window

How to think about it

Customers in calm minutes always count; the technique covers only a window of minutes angry minutes. n≤ 2× 10^4 calls for one pass. Sum the calm flow, then take the maximum angry flow over every window of length minutes. The window adds customers[i]· grumpy[i] and drops the value that leaves. The answer is the calm sum plus that maximum.

According to the problem description, we only need to count the number of customers when the boss is not angry tot, and add the maximum number of customers when the boss is angry within a sliding window of size `minutes` mx.

We define a variable cnt to record the number of customers when the boss is angry within the sliding window, the initial value is the number of customers when the boss is angry in the first `minutes`. Then we traverse the array, each time we move the sliding window, we update the value of cnt, and at the same time update the value of mx.

Finally, return tot + mx.

The time complexity is O(n), where n is the length of the array `customers`. The space complexity is O(1).

Python 3

```python
class Solution:
    def maxSatisfied(
        self, customers: List[int], grumpy: List[int], minutes: int
    ) -> int:
        mx = cnt = sum(c * g for c, g in zip(customers[:minutes], grumpy))
        for i in range(minutes, len(customers)):
            cnt += customers[i] * grumpy[i]
            cnt -= customers[i - minutes] * grumpy[i - minutes]
            mx = max(mx, cnt)
        return sum(c * (g ^ 1) for c, g in zip(customers, grumpy)) + mx
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the minute at which the owner should start the
# technique
# 2. Answer many different technique lengths quickly


# ============================================================
# FOLLOW-UP 1: Return the minute at which the owner should
# start the technique
# ============================================================

# ---- follow-up 1: function ----
# Answer: Compute the extra customers saved by a window of
# that length starting at every minute, and return the
# start of the best window.
# Time: O(n).  Space: O(1).
def best_technique_start(customers, grumpy, minutes):
    extra = 0
    best_extra = 0
    best_start = 0
    for i in range(len(customers)):
        if grumpy[i] == 1:
            extra += customers[i]
        if i >= minutes and grumpy[i - minutes] == 1:
            extra -= customers[i - minutes]
        if i >= minutes - 1 and extra > best_extra:
            best_extra = extra
            best_start = i - minutes + 1
    return best_start


# ============================================================
# FOLLOW-UP 2: Answer many different technique lengths
# quickly
# ============================================================

# ---- follow-up 2: function ----
# Answer: Prefix sums of the customers lost to grumpiness:
# the gain of any window is one subtraction, so each
# question costs O(n) instead of a new pass.
# Time: O(n).  Space: O(n).
def best_gain_for_length(customers, grumpy, minutes):
    lost = [0]
    for i in range(len(customers)):
        lost.append(lost[-1] + (customers[i] if grumpy[i] == 1 else 0))
    best = 0
    for start in range(len(customers) - minutes + 1):
        gain = lost[start + minutes] - lost[start]
        if gain > best:
```

```
            best = gain
    return best
```

PATTERN 2 · SLIDING WINDOW · PROBLEM 16 OF 20

1838. Frequency of the Most Frequent Element

Medium · Greedy, Array, Binary Search, Prefix Sum, Sorting, Sliding Window

leetcode.com/problems/frequency-of-the-most-frequent-element

Also appears in pattern 35 Sliding Window Maximum / Minimum (Specific Variants)

PROBLEM

The **frequency** of an element is the number of times it occurs in an array.

You are given an integer array `nums` and an integer `k`. In one operation, you can choose an index of `nums` and increment the element at that index by `1`.

Return *the **maximum possible frequency** of an element after performing **at most** `k` operations*.

Example 1:

```
Input: nums = [1,2,4], k = 5
Output: 3
Explanation: Increment the first element three times and the second element two times to make nums = [4,4,4].
4 has a frequency of 3.
```

Example 2:

```
Input: nums = [1,4,8,13], k = 5
Output: 2
Explanation: There are multiple optimal solutions:
- Increment the first element three times to make nums = [4,4,8,13]. 4 has a frequency of 2.
- Increment the second element four times to make nums = [1,8,8,13]. 8 has a frequency of 2.
- Increment the third element five times to make nums = [1,4,13,13]. 13 has a frequency of 2.
```

Example 3:

```
Input: nums = [3,9,6], k = 2
Output: 1
```

Constraints:

- `1 <= nums.length <= 10^5`
- `1 <= nums[i] <= 10^5`
- `1 <= k <= 10^5`

SOLUTIONS (PYTHON 3)

Solution 1: Sorting + Prefix Sum + Binary Search

How to think about it

We may increment elements at most k times and want the highest achievable frequency. The target must be some original value, and the raised elements form a contiguous prefix of the sorted array. Brute-forcing every right endpoint is too slow for $n \le 10^5$. After sorting, feasibility is monotone in the window length, so we binary-search the frequency. Prefix sums test whether some window of that length can be raised to its rightmost value with cost at most k.

According to the problem description, we can draw three conclusions:

1. After several operations, the element with the highest frequency in the array must be an element in the original array. Why? Suppose the elements operated are $a_1, a_2, \ldots, a_m$, where the maximum is a_m. These elements have all been changed to the same value x, where $x \ge a_m$. Then we can also change these elements all to a_m, and the number of operations will not increase.

2. The elements operated must be a continuous subarray in the sorted array.
3. If a frequency m satisfies the condition, then all m' < m also satisfy the condition. This inspires us to consider using binary search to find the maximum frequency that satisfies the condition.

Therefore, we can sort the array nums and then calculate the prefix sum array s of the sorted array, where s[i] represents the sum of the first i elements.

Next, we define the left boundary of the binary search as l = 1, and the right boundary as r = n. For each binary search, we take the middle value m = (l + r + 1) / 2, and then check whether there exists a continuous subarray of length m such that all elements in the subarray can be changed to an element in the array, and the number of operations does not exceed k. If such a subarray exists, we can update the left boundary l to m, otherwise update the right boundary r to m - 1.

Finally, return the left boundary l.

The time complexity is O(n × log n), and the space complexity is O(n). Where n is the length of the array nums.

Python 3

```python
class Solution:
    def maxFrequency(self, nums: List[int], k: int) -> int:
        def check(m: int) -> bool:
            for i in range(m, n + 1):
                if nums[i - 1] * m - (s[i] - s[i - m]) <= k:
                    return True
            return False

        n = len(nums)
        nums.sort()
        s = list(accumulate(nums, initial=0))
        l, r = 1, n
        while l < r:
            mid = (l + r + 1) >> 1
            if check(mid):
                l = mid
            else:
                r = mid - 1
        return l
```

Solution 2: Sorting + Two Pointers

> **How to think about it**
>
> Solution 1 needs prefix sums and a binary search. The window cost grows with the right end, so two pointers suffice: when i advances we raise the window to nums[i], and we move j while the cost exceeds k. One scan yields the longest valid window.

We can also use two pointers to maintain a sliding window, where all elements in the window can be changed to the maximum value in the window. The number of operations for the elements in the window is s, and s ≤ k.

Initially, we set the left pointer j to point to the first element of the array, and the right pointer i also points to the first element of the array. Next, we move the right pointer i each time, changing all elements in the window to nums[i]. At this time, the number of operations to be increased is (nums[i] - nums[i - 1]) × (i - j). If this number of operations exceeds k, then we need to move the left pointer j until the number of operations for the elements in the window does not exceed k. Then, we update the answer to the maximum length of the window.

The time complexity is O(n log n), and the space complexity is O(log n). Where n is the length of the array nums.

Python 3

```python
class Solution:
    def maxFrequency(self, nums: List[int], k: int) -> int:
        nums.sort()
        ans = 1
        s = j = 0
        for i in range(1, len(nums)):
            s += (nums[i] - nums[i - 1]) * (i - j)
            while s > k:
                s -= nums[i] - nums[j]
                j += 1
            ans = max(ans, i - j + 1)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the value that reaches the highest frequency
# 2. Each operation may also DECREASE a number by one


# ============================================================
# FOLLOW-UP 1: Return the value that reaches the highest
# frequency
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same sorted window; remember the largest number
# of the window (the value everything is raised to)
# whenever the window is the biggest so far.
# Time: O(n log n).  Space: O(n).
def most_frequent_value_after_increments(nums, k):
    nums = sorted(nums)
    left = 0
    total = 0
    best_length = 0
    best_value = None
    for right in range(len(nums)):
        total += nums[right]
        while nums[right] * (right - left + 1) - total > k:
            total -= nums[left]
            left += 1
        if right - left + 1 > best_length:
            best_length = right - left + 1
            best_value = nums[right]
    return best_value, best_length


# ============================================================
# FOLLOW-UP 2: Each operation may also DECREASE a number
# by one
# ============================================================

# ---- follow-up 2: function ----
# Answer: Make the window equal to its median: it needs
# the least total change. With prefix sums the cost of a
# window is one formula, and a longer window never costs
# less, so two pointers still work.
# Time: O(n log n).  Space: O(n).
def max_frequency_up_or_down(nums, k):
    nums = sorted(nums)
    prefix = [0]
    for x in nums:
        prefix.append(prefix[-1] + x)

    def cost(left, right):
```

```
        middle = (left + right) // 2
        below = nums[middle] * (middle - left) - (prefix[middle] - prefix[left])
        above = (prefix[right + 1] - prefix[middle + 1]) - nums[middle] * (right - middle)
        return below + above

    left = 0
    best = 0
    for right in range(len(nums)):
        while cost(left, right) > k:
            left += 1
```

```
        if right - left + 1 > best:
            best = right - left + 1
    return best
```

PATTERN 2 · SLIDING WINDOW · PROBLEM 17 OF 20

1888. Minimum Number of Flips to Make the Binary String Alternating

Medium · String, Dynamic Programming, Sliding Window

leetcode.com/problems/minimum-number-of-flips-to-make-the-binary-string-alternating

Also appears in pattern 35 Sliding Window Maximum / Minimum (Specific Variants)

PROBLEM

You are given a binary string `s`. You are allowed to perform two types of operations on the string in any sequence:

- **Type-1: Remove** the character at the start of the string `s` and **append** it to the end of the string.
- **Type-2: Pick** any character in `s` and **flip** its value, i.e., if its value is `'0'` it becomes `'1'` and vice-versa.

Return *the **minimum** number of **type-2** operations you need to perform such that* `s` *becomes **alternating**.*

The string is called **alternating** if no two adjacent characters are equal.

- For example, the strings `"010"` and `"1010"` are alternating, while the string `"0100"` is not.

Example 1:

```
Input: s = "111000"
Output: 2
Explanation: Use the first operation two times to make s = "100011".
Then, use the second operation on the third and sixth elements to make s = "101010".
```

Example 2:

```
Input: s = "010"
Output: 0
Explanation: The string is already alternating.
```

Example 3:

```
Input: s = "1110"
Output: 1
Explanation: Use the second operation on the second element to make s = "1010".
```

Constraints:

- `1 <= s.length <= 10^5`
- `s[i]` is either `'0'` or `'1'`.

SOLUTION (PYTHON 3)

Solution 1: Sliding Window

How to think about it

Type 1 makes the string circular; type 2 flips bits to an alternating pattern. Comparing every rotation bit by bit is $O(n^2)$ and too slow for $n \le 10^5$. Only two targets exist, and their costs sum to n. A window of length n on the circular string is one rotation; slide it while updating the mismatch count and keep min(cnt,n-cnt).

We notice that operation 1 effectively turns the string into a cycle, and operation 2 makes a substring of length n within the cycle into an alternating binary string.

Therefore, we only need to enumerate each substring of length n, calculate the cost to make it an alternating binary string, and take the minimum.

We can pre-calculate the number of differences between string s and the two types of alternating binary strings, denoted as cnt. The cost to make s into the first type of alternating binary string is cnt, and the cost to make s into the second type is n - cnt. We initialize ans = min(cnt, n - cnt).

Next, we enumerate each substring of length n and update the value of cnt. For each position i, we subtract the difference of s[i] from the first type of alternating binary string, and add the difference of s[i] to the second type. We update ans = min(ans, cnt, n - cnt).

Finally, return ans.

The time complexity is O(n), where n is the length of the string. The space complexity is O(1).

Python 3

```python
class Solution:
    def minFlips(self, s: str) -> int:
        n = len(s)
        target = "01"
        cnt = sum(c != target[i & 1] for i, c in enumerate(s))
        ans = min(cnt, n - cnt)
        for i in range(n):
            cnt -= s[i] != target[i & 1]
            cnt += s[i] != target[(i + n) & 1]
            ans = min(ans, cnt, n - cnt)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Only flips are allowed, no rotations
# 2. Which rotation (how many characters moved from the
# front to the back) needs the fewest flips?


# ============================================================
# FOLLOW-UP 1: Only flips are allowed, no rotations
# ============================================================

# ---- follow-up 1: helper functions ----
def min_of(a, b):
    if a < b:
        return a
    return b

# ---- follow-up 1: function ----
# Answer: An alternating string is either 0101... or
# 1010...; count the positions that differ from each and
# take the smaller count.
# Time: O(n).  Space: O(1).
def min_flips_no_rotation(s):
    differs_from_zero_start = 0
    for i in range(len(s)):
        expected = "0" if i % 2 == 0 else "1"
        if s[i] != expected:
            differs_from_zero_start += 1
    return min_of(differs_from_zero_start, len(s) - differs_from_zero_start)


# ============================================================
# FOLLOW-UP 2: Which rotation (how many characters moved
# from the front to the back) needs the fewest flips?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Double the string and slide a window of the
# original length over it, keeping the mismatch count
# against 0101... up to date; the count against 1010... is
```

```
# the length minus it.
# Time: O(n).  Space: O(n).
def best_rotation(s):
    n = len(s)
    doubled = s + s
    mismatch = 0                                    # window against the pattern 0101...
    best_flips, best_shift = n + 1, 0
    for i in range(len(doubled)):
        expected = "0" if i % 2 == 0 else "1"
        if doubled[i] != expected:
            mismatch += 1
        if i >= n:
            expected_old = "0" if (i - n) % 2 == 0 else "1"
            if doubled[i - n] != expected_old:
                mismatch -= 1
        if i >= n - 1:
            flips = min_of(mismatch, n - mismatch)
            if flips < best_flips:
                best_flips = flips
                best_shift = i - n + 1
    return best_shift, best_flips
```

PATTERN 2 · SLIDING WINDOW · PROBLEM 18 OF 20

76. Minimum Window Substring

Hard · Hash Table, String, Sliding Window

leetcode.com/problems/minimum-window-substring

★ **TOP 7 PROBLEM** in Sliding Window: master this one first

Also appears in pattern 6 String Manipulation / Parsing, 7 String Matching / Pattern Search, 35 Sliding Window Maximum / Minimum (Specific Variants)

PROBLEM

Given two strings `s` and `t` of lengths `m` and `n` respectively, return *the* ***minimum window substring*** *of* `s` *such that every character in* `t` *(****including duplicates****) is included in the window*. If there is no such substring, return *the empty string* `""`.

The testcases will be generated such that the answer is **unique**.

Example 1:

```
Input: s = "ADOBECODEBANC", t = "ABC"
Output: "BANC"
Explanation: The minimum window substring "BANC" includes 'A', 'B', and 'C' from string t.
```

Example 2:

```
Input: s = "a", t = "a"
Output: "a"
Explanation: The entire string s is the minimum window.
```

Example 3:

```
Input: s = "a", t = "aa"
Output: ""
Explanation: Both 'a's from t must be included in the window.
Since the largest window of s only has one 'a', return empty string.
```

Constraints:

- `m == s.length`
- `n == t.length`
- `1 <= m, n <=` 10^5
- `s` and `t` consist of uppercase and lowercase English letters.

Follow up: Could you find an algorithm that runs in `O(m + n)` time?

SOLUTION (PYTHON 3)

Solution 1: Counting + Two Pointers

How to think about it

Enumerate every substring of s and test coverage of t: O(m^2) windows, m ≤ 10^5 will TLE. The follow-up wants O(m+n). Once a window covers, extra characters on the left can go; if it does not cover, only the right end can grow. need stores t's quota, window stores the window, and cnt counts how many required occurrences are met, so coverage is O(1). Scan s with the right pointer; while valid, shrink the left and record the shortest span.

We use a hash table or array need to count the occurrences of each character in string t, and another hash table or array window to count the occurrences of each character in the sliding window. Additionally, we define two pointers l and r to point to the left and right boundaries of the window, a variable cnt to represent how many characters from t are already included in the window, and variables k and mi to represent the starting position and length of the minimum window substring.

We traverse the string s from left to right. For the current character s[r]:

- We add it to the window, i.e., window[s[r]] = window[s[r]] + 1. If need[s[r]] ≥ window[s[r]], it means s[r] is a "necessary character", and we increment cnt by one.
- If cnt equals the length of t, it means the window already contains all characters from t, and we can try to update the starting position and length of the minimum window substring. If r - l + 1 < mi, it means the current window represents a shorter substring, so we update mi = r - l + 1 and k = l.
- Then, we try to move the left boundary l. If need[s[l]] ≥ window[s[l]], it means s[l] is a "necessary character", and moving the left boundary will remove s[l] from the window. Therefore, we need to decrement cnt by one, update window[s[l]] = window[s[l]] - 1, and move l one position to the right.
- If cnt is not equal to the length of t, it means the window does not yet contain all characters from t, so we do not need to move the left boundary. Instead, we move r one position to the right and continue traversing.

After the traversal, if no minimum window substring is found, return an empty string. Otherwise, return s[k:k+mi].

The time complexity is O(m + n), and the space complexity is O(|Σ|). Here, m and n are the lengths of strings s and t, respectively; and |Σ| is the size of the character set, which is 128 in this problem.

Python 3

```python
class Solution:
    def minWindow(self, s: str, t: str) -> str:
        need = Counter(t)
        window = Counter()
        cnt = l = 0
        k, mi = -1, inf
        for r, c in enumerate(s):
            window[c] += 1
            if need[c] >= window[c]:
                cnt += 1
            while cnt == len(t):
                if r - l + 1 < mi:
                    mi = r - l + 1
                    k = l
                if need[s[l]] >= window[s[l]]:
                    cnt -= 1
                window[s[l]] -= 1
                l += 1
        return "" if k < 0 else s[k : k + mi]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return every minimum-length window, not only the
# first
# 2. Shortest window that contains at least K different
# characters


# ========================================================
# FOLLOW-UP 1: Return every minimum-length window, not
# only the first
# ========================================================

# ---- follow-up 1: function ----
# Answer: Keep the usual window, but whenever a valid
# window ties the shortest length so far, add it to the
# list; when it is shorter, start the list again.
# Time: O(n + W * L), W reported windows of length L.
# Space: O(s + W * L).
def all_minimum_windows(s, t):
    need = {}
    for ch in t:
        need[ch] = need.get(ch, 0) + 1
```

```
    have = {}
    satisfied = 0
    left = 0
    best_length = len(s) + 1
    windows = []
    for right in range(len(s)):
        ch = s[right]
        have[ch] = have.get(ch, 0) + 1
        if ch in need and have[ch] == need[ch]:
            satisfied += 1
        while satisfied == len(need):
            length = right - left + 1
            if length < best_length:
                best_length = length
                windows = [s[left:right + 1]]
            elif length == best_length:
                windows.append(s[left:right + 1])
            out = s[left]
            have[out] -= 1
            if out in need and have[out] < need[out]:
                satisfied -= 1
            left += 1
    return windows


# ============================================================
# FOLLOW-UP 2: Shortest window that contains at least K
# different characters
# ============================================================

# ---- follow-up 2: function ----
# Answer: No target string: grow the window, and while it
# still has at least K different characters keep shrinking
# it from the left.
# Time: O(n^2) in the worst case (slicing); O(n) to find
# the window.  Space: O(k + n).
def shortest_window_with_k_distinct(s, k):
    count = {}
```

```
    left = 0
    best = None
    for right in range(len(s)):
        count[s[right]] = count.get(s[right], 0) + 1
        while len(count) >= k:
            if best is None or right - left + 1 < len(best):
                best = s[left:right + 1]
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
    return best
```

PATTERN 2 · SLIDING WINDOW · PROBLEM 19 OF 20

239. Sliding Window Maximum

Hard · Queue, Array, Sliding Window, Monotonic Queue, Heap (Priority Queue), Range Query

leetcode.com/problems/sliding-window-maximum

★ **TOP 7 PROBLEM** in Sliding Window, Queue / Deque, Sliding Window Maximum / Minimum (Specific Variants): master this one first

Also appears in pattern 9 Monotonic Stack / Queue (Specific Variants), 10 Queue / Deque, 35 Sliding Window Maximum / Minimum (Specific Variants)

PROBLEM

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Return *the max sliding window*.

Example 1:

```
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [3,3,5,5,6,7]
Explanation:
Window position Max
--------------- -----
[1 3 -1] -3 5 3 6 7 3
 1 [3 -1 -3] 5 3 6 7 3
 1 3 [-1 -3 5] 3 6 7 5
 1 3 -1 [-3 5 3] 6 7 5
 1 3 -1 -3 [5 3 6] 7 6
 1 3 -1 -3 5 [3 6 7] 7
```

Example 2:

```
Input: nums = [1], k = 1
Output: [1]
```

Constraints:

- $1 <= nums.length <= 10^5$
- $-10^4 <= nums[i] <= 10^4$
- $1 <= k <= nums.length$

SOLUTIONS (PYTHON 3)

Solution 1: Priority Queue (Max-Heap)

How to think about it

A linear scan inside every window is O(nk). A max-heap reports the current maximum in logarithmic time; stale indices are popped only when they reach the top. Load the first k-1 values, then push each new index, drop expired tops, and record the top.

We can use a priority queue (max-heap) to maintain the maximum value in the sliding window.

First, add the first k-1 elements to the priority queue. Then, starting from the k-th element, add the new element to the priority queue and check if the top element of the heap is out of the window. If it is, remove the top element. Then, add the top element of the heap to the result array.

The time complexity is O(n × log k), and the space complexity is O(k). Here, n is the length of the array.

Python 3

```python
class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        q = [(-v, i) for i, v in enumerate(nums[: k - 1])]
        heapify(q)
        ans = []
        for i in range(k - 1, len(nums)):
            heappush(q, (-nums[i], i))
            while q[0][1] <= i - k:
                heappop(q)
            ans.append(-q[0][0])
        return ans
```

Solution 2: Monotonic Queue

> **How to think about it**
>
> The heap may hold stale indices and still pays a log factor. Only a decreasing sequence of candidates can become the window maximum. A monotonic queue of indices drops the front when it leaves the window and the back when it is ≤ the new value; the front is the maximum, in linear time.

To find the maximum value in a sliding window, a common method is to use a monotonic queue.

We can maintain a queue q that is monotonically decreasing from the front to the back, storing the indices of the elements. As we traverse the array nums, for the current element nums[i], we first check if the front element of the queue is out of the window. If it is, we remove the front element. Then, we compare the current element nums[i] with the elements at the back of the queue. If the elements at the back are less than or equal to the current element, we remove them until the element at the back is greater than the current element or the queue is empty. Then, we add the index of the current element to the queue. At this point, the front element of the queue is the maximum value of the current sliding window. Note that we add the front element of the queue to the result array when the index i is greater than or equal to k-1.

The time complexity is O(n), and the space complexity is O(k). Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        q = deque()
        ans = []
        for i, x in enumerate(nums):
            if q and i - q[0] >= k:
                q.popleft()
            while q and nums[q[-1]] <= x:
                q.pop()
            q.append(i)
            if i >= k - 1:
                ans.append(nums[q[0]])
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Longest subarray whose maximum minus minimum is at
# most a limit
# 2. The numbers are a stream: produce each window's
# maximum as soon as the window is full


# ========================================================
# FOLLOW-UP 1: Longest subarray whose maximum minus
# minimum is at most a limit
# ========================================================
```

```
# ---- follow-up 1: function ----
# Answer: Keep two deques over the current window: one
# with the candidates for the maximum (values decreasing)
# and one for the minimum (values increasing). Shrink the
# window from the left while max - min exceeds the limit.
# Time: O(n).  Space: O(n).
from collections import deque


def longest_within_limit(nums, limit):
    biggest = deque()                           # indexes, values decreasing
    smallest = deque()                          # indexes, values increasing
    left = 0
    best = 0
    for right in range(len(nums)):
        while biggest and nums[biggest[-1]] <= nums[right]:
            biggest.pop()
        biggest.append(right)
        while smallest and nums[smallest[-1]] >= nums[right]:
            smallest.pop()
        smallest.append(right)
        while nums[biggest[0]] - nums[smallest[0]] > limit:
            left += 1
            if biggest[0] < left:
                biggest.popleft()
            if smallest[0] < left:
                smallest.popleft()
        if right - left + 1 > best:
            best = right - left + 1
    return best


# ============================================================
# FOLLOW-UP 2: The numbers are a stream: produce each
# window's maximum as soon as the window is full
# ============================================================

# ---- follow-up 2: function ----
# Answer: A generator holds only the deque of candidate
# indexes, so memory stays at the window size.
# Time: O(1) amortised per number.  Space: O(k).
from collections import deque


def window_maximums_stream(numbers, k):
    candidates = deque()                        # (index, value), values decreasing
    index = -1
    for value in numbers:
        index += 1
        while candidates and candidates[-1][1] <= value:
            candidates.pop()
        candidates.append((index, value))
        if candidates[0][0] <= index - k:
            candidates.popleft()
        if index >= k - 1:
            yield candidates[0][1]
```

PATTERN 3 OF 41

Two Pointers

20 problems · 9 Easy · 9 Medium · 2 Hard

THE IDEA

Two indexes walk over the data (towards each other, or in the same direction) so each element is looked at a constant number of times. Usually needs sorted input or a partition rule.

Signals: sorted array and a pair or triple with a sum, remove or compact in place, palindrome checks, merge two sorted lists, container with most water.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
i, j = 0, len(a) - 1
while i < j:
    total = a[i] + a[j]
    if total == target:
        return [i, j]
    if total < target:
        i += 1          # need more: move the small side up
    else:
        j -= 1          # need less: move the big side down
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Valid Palindrome	Easy	LC 125	128
2	Reverse String	Easy	LC 344	131
3	Move Zeroes	Easy	LC 283	133
4	Remove Element	Easy	LC 27	135
5	Squares of a Sorted Array	Easy	LC 977	138
6	Remove Duplicates from Sorted Array	Easy	LC 26	141
7	Valid Palindrome II	Easy	LC 680	144
8	Backspace String Compare	Easy	LC 844	147
9	★ **Two Sum II - Input Array Is Sorted**	Medium	LC 167	150
10	★ **Remove Duplicates from Sorted Array II**	Medium	LC 80	153
11	★ **Sort Colors**	Medium	LC 75	156
12	Rotate Array	Medium	LC 189	159
13	★ **4Sum**	Medium	LC 18	161
14	Sort Array By Parity II	Easy	LC 922	164
15	★ **Container With Most Water**	Medium	LC 11	166
16	Partition Labels	Medium	LC 763	169
17	★ **Boats to Save People**	Medium	LC 881	172
18	Number of Subsequences That Satisfy the Given Sum Condition	Medium	LC 1498	174
19	★ **Trapping Rain Water**	Hard	LC 42	177
20	Minimum Window Subsequence	Hard	LC 727	180

PATTERN 3 · TWO POINTERS · PROBLEM 1 OF 20

125. Valid Palindrome

Easy · Two Pointers, String

leetcode.com/problems/valid-palindrome

PROBLEM

A phrase is a **palindrome** if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.

Given a string `s`, return `true` *if it is a* ***palindrome****, or* `false` *otherwise*.

Example 1:

```
Input: s = "A man, a plan, a canal: Panama"
Output: true
Explanation: "amanaplanacanalpanama" is a palindrome.
```

Example 2:

```
Input: s = "race a car"
Output: false
Explanation: "raceacar" is not a palindrome.
```

Example 3:

```
Input: s = " "
Output: true
Explanation: s is an empty string "" after removing non-alphanumeric characters.
Since an empty string reads the same forward and backward, it is a palindrome.
```

Constraints:

- `1 <= s.length <= 2 * 10^5`
- `s` consists only of printable ASCII characters.

SOLUTION (PYTHON 3)

Solution 1: Two Pointers

How to think about it

Check whether the string is a palindrome after ignoring case and non-alphanumeric characters. Filtering into a new string and reversing works at n ≤ 2× 10^5, but uses O(n) extra space. A palindrome only needs matching pairs. Two pointers walk inward: skip illegal characters, compare legal ones case-insensitively. One pass, constant space.

We use two pointers i and j to point to the two ends of the string s, and then loop through the following process until i ≥ j:

1. If s[i] is not a letter or a number, move the pointer i one step to the right and continue to the next loop.
2. If s[j] is not a letter or a number, move the pointer j one step to the left and continue to the next loop.
3. If the lowercase form of s[i] and s[j] are not equal, return `false`.
4. Otherwise, move the pointer i one step to the right and the pointer j one step to the left, and continue to the next loop.

At the end of the loop, return `true`.

The time complexity is O(n), where n is the length of the string s. The space complexity is O(1).

Python 3

```python
class Solution:
    def isPalindrome(self, s: str) -> bool:
        i, j = 0, len(s) - 1
        while i < j:
            if not s[i].isalnum():
                i += 1
            elif not s[j].isalnum():
                j -= 1
            elif s[i].lower() != s[j].lower():
                return False
            else:
                i, j = i + 1, j - 1
        return True
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Fewest letters to INSERT to turn the string into a
# palindrome
# 2. Find the longest palindromic substring


# ============================================================
# FOLLOW-UP 1: Fewest letters to INSERT to turn the string
# into a palindrome
# ============================================================

# ---- follow-up 1: function ----
# Answer: Every letter that is not part of the longest
# palindromic subsequence needs a partner, so the answer
# is length minus the length of that subsequence (a table
# over ranges).
# Time: O(n^2).  Space: O(n^2).
def min_insertions_for_palindrome(s):
    n = len(s)
    if n == 0:
        return 0
    best = []
    for i in range(n):
        best.append([0] * n)
    for i in range(n - 1, -1, -1):
        best[i][i] = 1
        for j in range(i + 1, n):
            if s[i] == s[j]:
                best[i][j] = best[i + 1][j - 1] + 2
            elif best[i + 1][j] >= best[i][j - 1]:
                best[i][j] = best[i + 1][j]
            else:
                best[i][j] = best[i][j - 1]
    return n - best[0][n - 1]


# ============================================================
# FOLLOW-UP 2: Find the longest palindromic substring
# ============================================================

# ---- follow-up 2: function ----
# Answer: Expand outwards from every possible centre (a
# letter, or the gap between two letters) while both ends
# match.
# Time: O(n^2).  Space: O(1).
def longest_palindromic_substring(s):
    best_start, best_length = 0, 0
    for centre in range(2 * len(s) - 1):
        left = centre // 2
```

```
        right = left + centre % 2                # odd centre: one letter, even centre: two letters
        while left >= 0 and right < len(s) and s[left] == s[right]:
            left -= 1
            right += 1
        length = right - left - 1
        if length > best_length:
            best_length = length
            best_start = left + 1
    return s[best_start:best_start + best_length]
```

PATTERN 3 · TWO POINTERS · PROBLEM 2 OF 20

344. Reverse String

Easy · Two Pointers, String

leetcode.com/problems/reverse-string

PROBLEM

Write a function that reverses a string. The input string is given as an array of characters `s`.

You must do this by modifying the input array in-place with `O(1)` extra memory.

Example 1:

```
Input: s = ["h","e","l","l","o"]
Output: ["o","l","l","e","h"]
```

Example 2:

```
Input: s = ["H","a","n","n","a","h"]
Output: ["h","a","n","n","a","H"]
```

Constraints:

- `1 <= s.length <= 10^5`
- `s[i]` is a printable ascii character.

SOLUTION (PYTHON 3)

Solution 1: Two Pointers

> **How to think about it**
>
> Reverse a character array in place. A second buffer uses O(n) space. Swapping ends is enough. Two pointers i,j start at the ends, swap, and move inward until they meet. One pass, constant extra space.

We use two pointers i and j, initially pointing to the start and end of the array respectively. Each time, we swap the elements at i and j, then move i forward and j backward, until i and j meet.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Python 3

```python
class Solution:
    def reverseString(self, s: List[str]) -> None:
        i, j = 0, len(s) - 1
        while i < j:
            s[i], s[j] = s[j], s[i]
            i, j = i + 1, j - 1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Reverse the order of the words, in place, on a list
# of characters
# 2. Reverse only the letters and keep every other
# character where it is


# ============================================================
# FOLLOW-UP 1: Reverse the order of the words, in place,
# on a list of characters
# ============================================================
```

```python
# ---- follow-up 1: helper functions ----
def reverse_part(chars, left, right):
    while left < right:
        chars[left], chars[right] = chars[right], chars[left]
        left += 1
        right -= 1

# ---- follow-up 1: function ----
# Answer: Reverse the whole list, then reverse every word
# again so each word reads correctly.
# Time: O(n).  Space: O(1).
def reverse_words_in_place(chars):
    reverse_part(chars, 0, len(chars) - 1)
    start = 0
    for i in range(len(chars) + 1):
        if i == len(chars) or chars[i] == " ":
            reverse_part(chars, start, i - 1)
            start = i + 1


# ========================================================
# FOLLOW-UP 2: Reverse only the letters and keep every
# other character where it is
# ========================================================

# ---- follow-up 2: function ----
# Answer: Two pointers that skip characters which are not
# letters and swap the letters they stop on.
# Time: O(n).  Space: O(n).
def reverse_only_letters(s):
    chars = list(s)
    left, right = 0, len(chars) - 1
    while left < right:
        if not chars[left].isalpha():
            left += 1
        elif not chars[right].isalpha():
            right -= 1
        else:
            chars[left], chars[right] = chars[right], chars[left]
            left += 1
            right -= 1
    return "".join(chars)
```

PATTERN 3 · TWO POINTERS · PROBLEM 3 OF 20

283. Move Zeroes

Easy · Array, Two Pointers

leetcode.com/problems/move-zeroes

Also appears in pattern 33 Cyclic Sort

PROBLEM

Given an integer array `nums`, move all `0`'s to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

Example 1:

```
Input: nums = [0,1,0,3,12]
Output: [1,3,12,0,0]
```

Example 2:

```
Input: nums = [0]
Output: [0]
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

Follow up:

Could you minimize the total number of operations done?

SOLUTION (PYTHON 3)

Solution 1: Two Pointers

> **How to think about it**
>
> Zeros must go to the end while nonzeros keep their order. Let k be the next slot for a nonzero. Each nonzero is swapped with nums[k] and k advances, so the prefix of length k is the original nonzero sequence.

We use a pointer k to record the current position to insert, initially k = 0.

Then we iterate through the array nums, and each time we encounter a non-zero number, we swap it with nums[k] and increment k by 1.

This way, we can ensure that the first k elements of nums are non-zero, and their relative order is the same as in the original array.

The time complexity is O(n), where n is the length of the array nums. The space complexity is O(1).

Python 3

```python
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        k = 0
        for i, x in enumerate(nums):
            if x:
                nums[k], nums[i] = nums[i], nums[k]
                k += 1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Move any given value to the end (not only zero),
# keeping the order of the rest
# 2. Do the fewest possible writes


# ============================================================
# FOLLOW-UP 1: Move any given value to the end (not only
# zero), keeping the order of the rest
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same write pointer: copy every element that
# is not the value forward, then fill the tail with the
# value.
# Time: O(n).  Space: O(1).
def move_value_to_end(nums, value):
    write = 0
    for read in range(len(nums)):
        if nums[read] != value:
            nums[write] = nums[read]
            write += 1
    for i in range(write, len(nums)):
        nums[i] = value


# ============================================================
# FOLLOW-UP 2: Do the fewest possible writes
# ============================================================

# ---- follow-up 2: function ----
# Answer: Only swap when the two positions differ: when
# the array has no zero before a number there is nothing
# to write, so the write pointer and the read pointer stay
# equal.
# Time: O(n).  Space: O(1).
def move_zeroes_fewest_writes(nums):
    write = 0
    writes = 0
    for read in range(len(nums)):
        if nums[read] != 0:
            if read != write:
                nums[write], nums[read] = nums[read], nums[write]
                writes += 1
            write += 1
    return writes
```

PATTERN 3 · TWO POINTERS · PROBLEM 4 OF 20

27. Remove Element

Easy · Array, Two Pointers

leetcode.com/problems/remove-element

PROBLEM

Given an integer array `nums` and an integer `val`, remove all occurrences of `val` in `nums` **in-place**. The order of the elements may be changed. Then return *the number of elements in* `nums` *which are not equal to* `val`.

Consider the number of elements in `nums` which are not equal to `val` be `k`, to get accepted, you need to do the following things:

- Change the array `nums` such that the first `k` elements of `nums` contain the elements which are not equal to `val`. The remaining elements of `nums` are not important as well as the size of `nums`.
- Return `k`.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int val = ...; // Value to remove
int[] expectedNums = [...]; // The expected answer with correct length.
                            // It is sorted with no values equaling val.

int k = removeElement(nums, val); // Calls your implementation

assert k == expectedNums.length;
sort(nums, 0, k); // Sort the first k elements of nums
for (int i = 0; i < actualLength; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

```
Input: nums = [3,2,2,3], val = 3
Output: 2, nums = [2,2,_,_]
Explanation: Your function should return k = 2, with the first two elements of nums being 2.
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Example 2:

```
Input: nums = [0,1,2,2,3,0,4,2], val = 2
Output: 5, nums = [0,1,4,0,3,_,_,_]
Explanation: Your function should return k = 5, with the first five elements of nums containing 0, 0,
1, 3, and 4.
Note that the five elements can be returned in any order.
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Constraints:

- `0 <= nums.length <= 100`
- `0 <= nums[i] <= 50`
- `0 <= val <= 100`

SOLUTION (PYTHON 3)

Solution 1: One Pass

How to think about it

The first idea is a fresh array of everything not equal to val. n ≤ 100 would pass, but the problem asks for an in-place rewrite, so the extra allocation is unnecessary. The bottleneck is "deleting" with something like erase: later elements shift again and again, worst case $O(n^2)$. The judge only cares that the first k slots hold every non-val value; order may change. So overwrite the front of the array with the values we keep. k counts how many we have written: when x ≠ val, store it at nums[k] and increment k. One pass, O(1) extra space.

We use the variable k to record the number of elements that are not equal to val.

Traverse the array nums, if the current element x is not equal to val, then assign x to nums[k], and increment k by 1.

Finally, return k.

The time complexity is O(n) and the space complexity is O(1), where n is the length of the array nums.

Python 3

```
class Solution:
    def removeElement(self, nums: List[int], val: int) -> int:
        k = 0
        for x in nums:
            if x != val:
                nums[k] = x
                k += 1
        return k
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The wanted value is rare, so avoid shifting
# everything: the order does not matter
# 2. Remove every value that appears in a given set


# ============================================================
# FOLLOW-UP 1: The wanted value is rare, so avoid shifting
# everything: the order does not matter
# ============================================================

# ---- follow-up 1: function ----
# Answer: Overwrite each unwanted element with the last
# element and shrink the array: at most one write per
# removed element.
# Time: O(n).  Space: O(1).
def remove_element_swap_with_last(nums, val):
    i = 0
    size = len(nums)
    while i < size:
        if nums[i] == val:
            nums[i] = nums[size - 1]
            size -= 1
        else:
            i += 1
    return size


# ============================================================
# FOLLOW-UP 2: Remove every value that appears in a given
# set
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Same write pointer, but the keep-or-remove test
# is a set lookup.
# Time: O(n).  Space: O(1).
def remove_values_in_set(nums, unwanted):
    write = 0
    for read in range(len(nums)):
        if nums[read] not in unwanted:
            nums[write] = nums[read]
            write += 1
    return write
```

PATTERN 3 · TWO POINTERS · PROBLEM 5 OF 20

977. Squares of a Sorted Array

Easy · Array, Two Pointers, Sorting

leetcode.com/problems/squares-of-a-sorted-array

PROBLEM

Given an integer array `nums` sorted in **non-decreasing** order, return *an array of **the squares of each number** sorted in non-decreasing order*.

Example 1:

```
Input: nums = [-4,-1,0,3,10]
Output: [0,1,9,16,100]
Explanation: After squaring, the array becomes [16,1,0,9,100].
After sorting, it becomes [0,1,9,16,100].
```

Example 2:

```
Input: nums = [-7,-3,2,3,11]
Output: [4,9,9,49,121]
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^4 <= nums[i] <= 10^4`
- `nums` is sorted in **non-decreasing** order.

Follow up:

Squaring each element and sorting the new array is very trivial, could you find an

`O(n)`

solution using a different approach?

SOLUTION (PYTHON 3)

Solution 1: Two Pointers

> **How to think about it**
>
> A sorted array may contain negatives, so the squares must be reordered. Squaring then sorting is O(nlog n). After squaring, the ends are large and the middle is small, so two pointers pick the larger square and write from the back (or append then reverse), in linear time.

Since the array nums is already sorted in non-decreasing order, the square values of the negative numbers in the array are decreasing, and the square values of the positive numbers are increasing. We can use two pointers, each pointing to the ends of the array. Each time we compare the square values of the elements pointed to by the two pointers, we put the larger square value at the end of the result array.

The time complexity is O(n), where n is the length of the array nums. Ignoring the space consumption of the answer array, the space complexity is O(1).

Python 3

```python
class Solution:
    def sortedSquares(self, nums: List[int]) -> List[int]:
        ans = []
        i, j = 0, len(nums) - 1
        while i <= j:
            a = nums[i] * nums[i]
            b = nums[j] * nums[j]
            if a > b:
                ans.append(a)
                i += 1
            else:
                ans.append(b)
                j -= 1
        return ans[::-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Apply a quadratic a*x*x + b*x + c to a sorted array
# and return it sorted
# 2. Only the K smallest squares are needed


# ============================================================
# FOLLOW-UP 1: Apply a quadratic a*x*x + b*x + c to a
# sorted array and return it sorted
# ============================================================

# ---- follow-up 1: helper functions ----
def quadratic(x, a, b, c):
    return a * x * x + b * x + c

# ---- follow-up 1: function ----
# Answer: The values form a valley when a is positive
# (smallest in the middle) and a hill when a is negative,
# so the two ends hold the extremes: fill the result from
# the back (a > 0) or from the front (a < 0) with two
# pointers. A linear function is just a reversal if b is
# negative.
# Time: O(n).  Space: O(n).
def sort_transformed_array(nums, a, b, c):
    result = [0] * len(nums)
    left, right = 0, len(nums) - 1
    if a >= 0:
        index = len(nums) - 1                    # biggest values come from the ends
        while left <= right:
            low = quadratic(nums[left], a, b, c)
            high = quadratic(nums[right], a, b, c)
            if low > high:
                result[index] = low
                left += 1
            else:
                result[index] = high
                right -= 1
            index -= 1
    else:
        index = 0                                # smallest values come from the ends
        while left <= right:
            low = quadratic(nums[left], a, b, c)
            high = quadratic(nums[right], a, b, c)
            if low < high:
                result[index] = low
                left += 1
            else:
                result[index] = high
```

```
                right -= 1
            index += 1
    return result


# ========================================================
# FOLLOW-UP 2: Only the K smallest squares are needed
# ========================================================

# ---- follow-up 2: function ----
# Answer: Start at the border between the negative and the
# non-negative numbers and walk outwards, always taking
```

```
# the smaller square of the two pointers; stop after K
# numbers.
# Time: O(n + k).  Space: O(k).
def k_smallest_squares(nums, k):
    right = 0
    while right < len(nums) and nums[right] < 0:
        right += 1
    left = right - 1
    result = []
    while len(result) < k and (left >= 0 or right < len(nums)):
        if right >= len(nums) or (left >= 0 and nums[left] * nums[left] < nums[right] * nums[right]):
            result.append(nums[left] * nums[left])
            left -= 1
        else:
            result.append(nums[right] * nums[right])
            right += 1
    return result
```

PATTERN 3 · TWO POINTERS · PROBLEM 6 OF 20

26. Remove Duplicates from Sorted Array

Easy · Array, Two Pointers

leetcode.com/problems/remove-duplicates-from-sorted-array

PROBLEM

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**.

Consider the number of *unique elements* in `nums` to be `k□□□□□□□`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in **sorted order**. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

```
Input: nums = [1,1,2]
Output: 2, nums = [1,2,_]
Explanation: Your function should return k = 2, with the first two elements of nums being 1 and 2
respectively.
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Example 2:

```
Input: nums = [0,0,1,1,1,2,2,3,3,4]
Output: 5, nums = [0,1,2,3,4,_,_,_,_,_]
Explanation: Your function should return k = 5, with the first five elements of nums being 0, 1, 2, 3,
and 4 respectively.
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Constraints:

- `1 <= nums.length <= 3 * 10^4`
- `-100 <= nums[i] <= 100`
- `nums` is sorted in **non-decreasing** order.

SOLUTION (PYTHON 3)

Solution 1: Single Pass

How to think about it

The first idea is a hash set of seen values, then write them back. n ≤ 3× 10^4 would pass, but the problem wants an in-place rewrite, so the extra table is wasteful. The bottleneck is the "have we seen this" query. The array is non-decreasing, so equals sit next to each other and a global lookup is unnecessary. Compare x with the last value already written. If it differs (or nothing has been written yet), write it at index k; otherwise skip. k is both the write pointer and the unique length. One pass, O(1) extra space.

We use a variable k to record the current length of the processed array. Initially, k=0 represents an empty array.

Then we traverse the array from left to right. For each element x we encounter, if k=0 or x ≠ nums[k-1], we place x in the position of nums[k], and then increment k by 1. Otherwise, x is the same as nums[k-1], so we skip this element. Continue to traverse until the entire array is traversed.

In this way, when the traversal ends, the first k elements in nums are the answer we are looking for, and k is the length of the answer.

The time complexity is O(n), and the space complexity is O(1). Here, n is the length of the array.

Supplement:

The original problem requires that the same number appear at most once. We can extend it to keep at most k identical numbers.

- Since the same number can be kept at most k times, we can directly keep the first k elements of the original array;
- For the following numbers, the premise of being able to keep them is: the current number x is compared with the last kth element of the previously retained numbers. If they are different, keep them, otherwise skip them.

Similar problems:

- 80. Remove Duplicates from Sorted Array II

Python 3

```python
class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        k = 0
        for x in nums:
            if k == 0 or x != nums[k - 1]:
                nums[k] = x
                k += 1
        return k
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The array is NOT sorted: keep the first copy of each
# value, in order
# 2. Remove every value that appears more than once,
# keeping only the values that appear exactly once


# ========================================================
# FOLLOW-UP 1: The array is NOT sorted: keep the first
# copy of each value, in order
# ========================================================

# ---- follow-up 1: function ----
# Answer: A set of the values already kept replaces the
# 'compare with the previous element' test.
# Time: O(n).  Space: O(n).
def remove_duplicates_unsorted(nums):
```

```
    seen = set()
    write = 0
    for read in range(len(nums)):
        if nums[read] not in seen:
            seen.add(nums[read])
            nums[write] = nums[read]
            write += 1
    return write


# ============================================================
# FOLLOW-UP 2: Remove every value that appears more than
# once, keeping only the values that appear exactly once
# ============================================================

# ---- follow-up 2: function ----
# Answer: Look at the neighbours on both sides: an element
# is kept only when it differs from the previous one and
# from the next one.
# Time: O(n).  Space: O(1).
def keep_only_unique_values(nums):
    write = 0
    for i in range(len(nums)):
        same_as_before = i > 0 and nums[i] == nums[i - 1]
        same_as_after = i + 1 < len(nums) and nums[i] == nums[i + 1]
        if not same_as_before and not same_as_after:
            nums[write] = nums[i]
            write += 1
    return write
```

PATTERN 3 · TWO POINTERS · PROBLEM 7 OF 20

680. Valid Palindrome II

Easy · Greedy, Two Pointers, String

leetcode.com/problems/valid-palindrome-ii

PROBLEM

Given a string `s`, return `true` *if the* `s` *can be palindrome after deleting* ***at most one*** *character from it*.

Example 1:

```
Input: s = "aba"
Output: true
```

Example 2:

```
Input: s = "abca"
Output: true
Explanation: You could delete the character 'c'.
```

Example 3:

```
Input: s = "abc"
Output: false
```

Constraints:

- `1 <= s.length <= 10^5`
- `s` consists of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Two Pointers

How to think about it

Delete at most one character to make a palindrome. Trying every deletion is quadratic for $n \le 10^5$. Two pointers meet a mismatch at most once: check the remaining interval after skipping the left or the right character. The extra scan is still linear.

We use two pointers to point to the left and right ends of the string, respectively. Each time, we check whether the characters pointed to by the two pointers are the same. If they are not the same, we check whether the string is a palindrome after deleting the character corresponding to the left pointer, or we check whether the string is a palindrome after deleting the character corresponding to the right pointer. If the characters pointed to by the two pointers are the same, we move both pointers towards the middle by one position, until the two pointers meet.

If we have not encountered a situation where the characters pointed to by the pointers are different by the end of the traversal, then the string itself is a palindrome, and we return `true`.

The time complexity is O(n), where n is the length of the string s. The space complexity is O(1).

Python 3

```python
class Solution:
    def validPalindrome(self, s: str) -> bool:
        def check(i, j):
            while i < j:
                if s[i] != s[j]:
                    return False
                i, j = i + 1, j - 1
            return True

        i, j = 0, len(s) - 1
        while i < j:
            if s[i] != s[j]:
                return check(i, j - 1) or check(i + 1, j)
            i, j = i + 1, j - 1
        return True
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Delete at most K characters
# 2. Which index should be deleted?


# ============================================================
# FOLLOW-UP 1: Delete at most K characters
# ============================================================

# ---- follow-up 1: function ----
# Answer: A string can be made a palindrome with at most K
# deletions exactly when length minus its longest
# palindromic subsequence is at most K.
# Time: O(n^2).  Space: O(n^2).
def palindrome_with_k_deletions(s, k):
    n = len(s)
    if n == 0:
        return True
    best = []
    for i in range(n):
        best.append([0] * n)
    for i in range(n - 1, -1, -1):
        best[i][i] = 1
        for j in range(i + 1, n):
            if s[i] == s[j]:
                best[i][j] = best[i + 1][j - 1] + 2
            elif best[i + 1][j] >= best[i][j - 1]:
                best[i][j] = best[i + 1][j]
            else:
                best[i][j] = best[i][j - 1]
    return n - best[0][n - 1] <= k


# ============================================================
# FOLLOW-UP 2: Which index should be deleted?
# ============================================================

# ---- follow-up 2: helper functions ----
def is_palindrome_range(s, left, right):
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1
    return True

# ---- follow-up 2: function ----
```

```
# Answer: Walk in from both ends; at the first mismatch
# try removing the left letter or the right letter and see
# which side leaves a palindrome. Return -1 if the string
# is already a palindrome and -2 if no single deletion
# helps.
# Time: O(n).  Space: O(1).
def index_to_delete(s):
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            if is_palindrome_range(s, left + 1, right):
                return left
            if is_palindrome_range(s, left, right - 1):
                return right
            return -2
        left += 1
        right -= 1
    return -1
```

PATTERN 3 · TWO POINTERS · PROBLEM 8 OF 20

844. Backspace String Compare

Easy · Stack, Two Pointers, String, Simulation

leetcode.com/problems/backspace-string-compare

Also appears in pattern 8 Stack

PROBLEM

Given two strings `s` and `t`, return `true` *if they are equal when both are typed into empty text editors*. `'#'` means a backspace character.

Note that after backspacing an empty text, the text will continue empty.

Example 1:

```
Input: s = "ab#c", t = "ad#c"
Output: true
Explanation: Both s and t become "ac".
```

Example 2:

```
Input: s = "ab##", t = "c#d#"
Output: true
Explanation: Both s and t become "".
```

Example 3:

```
Input: s = "a#c", t = "b"
Output: false
Explanation: s becomes "c" while t becomes "b".
```

Constraints:

- `1 <= s.length, t.length <= 200`
- `s` and `t` only contain lowercase letters and `'#'` characters.

Follow up: Can you solve it in `O(n)` time and `O(1)` space?

SOLUTION (PYTHON 3)

Solution 1

How to think about it

We compare two strings after applying backspaces. Building the cleaned strings uses extra linear space; the intended scan works on the originals. Walk right to left, letting a skip counter consume `#` and the characters they delete, then compare the next live characters. If one side runs out first, they differ.

Python 3

```python
class Solution:
    def backspaceCompare(self, s: str, t: str) -> bool:
        i, j, skip1, skip2 = len(s) - 1, len(t) - 1, 0, 0
        while i >= 0 or j >= 0:
            while i >= 0:
                if s[i] == '#':
                    skip1 += 1
                    i -= 1
                elif skip1:
                    skip1 -= 1
                    i -= 1
                else:
                    break
            while j >= 0:
                if t[j] == '#':
                    skip2 += 1
                    j -= 1
                elif skip2:
                    skip2 -= 1
                    j -= 1
                else:
                    break
            if i >= 0 and j >= 0:
                if s[i] != t[j]:
                    return False
            elif i >= 0 or j >= 0:
                return False
            i, j = i - 1, j - 1
        return True
```

Complexity. Time: O(n + m). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Use O(1) extra space (no stack)
# 2. Return the text that is left after all backspaces


# ============================================================
# FOLLOW-UP 1: Use O(1) extra space (no stack)
# ============================================================

# ---- follow-up 1: helper functions ----
def next_valid_index(text, index):
    # the position of the next letter that survives the backspaces, walking left (or -1)
    skip = 0
    while index >= 0:
        if text[index] == "#":
            skip += 1
        elif skip > 0:
            skip -= 1
        else:
            return index
        index -= 1
    return -1

# ---- follow-up 1: function ----
# Answer: Walk both strings from the END. A '#' means
# 'skip one more letter'; skip as many letters as pending
# backspaces, then compare the next real letters.
# Time: O(n + m).  Space: O(1).
def backspace_compare_constant_space(s, t):
    i = next_valid_index(s, len(s) - 1)
    j = next_valid_index(t, len(t) - 1)
    while i >= 0 and j >= 0:
```

```
        if s[i] != t[j]:
            return False
        i = next_valid_index(s, i - 1)
        j = next_valid_index(t, j - 1)
    return i < 0 and j < 0


# ============================================================
# FOLLOW-UP 2: Return the text that is left after all
# backspaces
# ============================================================

# ---- follow-up 2: function ----
# Answer: A stack of letters: a letter is pushed, a '#'
# pops one if there is one.
# Time: O(n).  Space: O(n).
def final_text(typed):
    stack = []
    for ch in typed:
        if ch == "#":
            if stack:
                stack.pop()
        else:
            stack.append(ch)
    return "".join(stack)
```

PATTERN 3 · TWO POINTERS · PROBLEM 9 OF 20

167. Two Sum II - Input Array Is Sorted

Medium · Array, Two Pointers, Binary Search

leetcode.com/problems/two-sum-ii-input-array-is-sorted

★ **TOP 7 PROBLEM** in Two Pointers: master this one first

PROBLEM

You are given a **1-indexed** array of integers `numbers` that is already **sorted in non-decreasing order**.

Find **two** numbers such that they add up to a specific `target` number. Let these two numbers be `numbers[index1]` and `numbers[index2]` where `1 <= index1 < index2 <= numbers.length`.

Return the indices of the two numbers `index1` and `index2` as an integer array `[index1, index2]` of length 2.

The tests are generated such that there is **exactly one solution**. You **may not** use the same element twice.

Your solution must use only constant extra space.

Example 1:

```
Input: numbers = [2,7,11,15], target = 9
Output: [1,2]
Explanation: The sum of 2 and 7 is 9. Therefore, index1 = 1, index2 = 2. We return [1, 2].
```

Example 2:

```
Input: numbers = [2,3,4], target = 6
Output: [1,3]
Explanation: The sum of 2 and 4 is 6. Therefore index1 = 1, index2 = 3. We return [1, 3].
```

Example 3:

```
Input: numbers = [-1,0], target = -1
Output: [1,2]
Explanation: The sum of -1 and 0 is -1. Therefore index1 = 1, index2 = 2. We return [1, 2].
```

Constraints:

- `2 <= numbers.length <= 3 * 10^4`
- `-1000 <= numbers[i] <= 1000`
- `numbers` is sorted in **non-decreasing order**.
- `-1000 <= target <= 1000`
- The tests are generated such that there is **exactly one solution**.

SOLUTIONS (PYTHON 3)

Solution 1: Binary Search

How to think about it

Two sum on a sorted array, 1-based indices, exactly one pair. A hash map uses O(n) space. For each numbers[i] binary-search target-numbers[i] on the right. n≤ 3× 10^4, so O(nlog n) time and O(1) space.

Note that the array is sorted in non-decreasing order, so for each `numbers[i]`, we can find the position of `target - numbers[i]` by binary search, and return [i + 1, j + 1] if it exists.

The time complexity is O(n × log n), where n is the length of the array `numbers`. The space complexity is O(1).

Python 3

```python
class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        n = len(numbers)
        for i in range(n - 1):
            x = target - numbers[i]
            j = bisect_left(numbers, x, lo=i + 1)
            if j < n and numbers[j] == x:
                return [i + 1, j + 1]
```

Solution 2: Two Pointers

> **How to think about it**
>
> Solution 1 binary-searches at every left index. Two pointers from the ends: a small sum moves the left pointer right, a large sum moves the right pointer left. Sorted order cannot miss the unique pair, and the time becomes O(n).

We define two pointers i and j, which point to the first element and the last element of the array respectively. Each time we calculate numbers[i] + numbers[j]. If the sum is equal to the target value, return [i + 1, j + 1] directly. If the sum is less than the target value, move i to the right by one position, and if the sum is greater than the target value, move j to the left by one position.

The time complexity is O(n), where n is the length of the array `numbers`. The space complexity is O(1).

Python 3

```python
class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        i, j = 0, len(numbers) - 1
        while i < j:
            x = numbers[i] + numbers[j]
            if x == target:
                return [i + 1, j + 1]
            if x < target:
                i += 1
            else:
                j -= 1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Find all unique triplets that add up to zero (3Sum)
# 2. The triplet whose sum is CLOSEST to the target
# 3. Count the pairs whose sum is LESS than the target


# ============================================================
# FOLLOW-UP 1: Find all unique triplets that add up to
# zero (3Sum)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort, fix the first number, and find the other
# two with the two-pointer method, skipping equal
# neighbours to avoid repeated triplets.
# Time: O(n^2).  Space: O(n) for the sorted copy.
def three_sum(nums):
    nums = sorted(nums)
    result = []
    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        left, right = i + 1, len(nums) - 1
        while left < right:
```

```
            total = nums[i] + nums[left] + nums[right]
            if total == 0:
                result.append([nums[i], nums[left], nums[right]])
                left += 1
                right -= 1
                while left < right and nums[left] == nums[left - 1]:
                    left += 1
            elif total < 0:
                left += 1
            else:
                right -= 1
    return result


# ============================================================
# FOLLOW-UP 2: The triplet whose sum is CLOSEST to the
# target
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same sort and two pointers; keep the sum with
# the smallest distance to the target and stop at once on
# an exact hit.
# Time: O(n^2).  Space: O(n).
def three_sum_closest(nums, target):
    nums = sorted(nums)
    best = nums[0] + nums[1] + nums[2]
    for i in range(len(nums) - 2):
        left, right = i + 1, len(nums) - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if abs(total - target) < abs(best - target):
                best = total
            if total == target:
                return total
            if total < target:
```

```
                left += 1
            else:
                right -= 1
    return best


# ============================================================
# FOLLOW-UP 3: Count the pairs whose sum is LESS than the
# target
# ============================================================

# ---- follow-up 3: function ----
# Answer: When the smallest and the biggest of the current
# range add up to less than the target, every number
# between them also works with the smallest one.
# Time: O(n log n).  Space: O(n).
def count_pairs_below(nums, target):
    nums = sorted(nums)
    left, right = 0, len(nums) - 1
    count = 0
    while left < right:
        if nums[left] + nums[right] < target:
            count += right - left
            left += 1
        else:
            right -= 1
    return count
```

PATTERN 3 · TWO POINTERS · PROBLEM 10 OF 20

80. Remove Duplicates from Sorted Array II

Medium · Array, Two Pointers

leetcode.com/problems/remove-duplicates-from-sorted-array-ii

★ **TOP 7 PROBLEM** in Two Pointers: master this one first

PROBLEM

Given an integer array `nums` sorted in **non-decreasing order**, remove some duplicates **in-place** such that each unique element appears **at most twice**. The **relative order** of the elements should be kept the **same**.

Since it is impossible to change the length of the array in some languages, you must instead have the result be placed in the **first part** of the array `nums`. More formally, if there are `k` elements after removing the duplicates, then the first `k` elements of `nums` should hold the final result. It does not matter what you leave beyond the first `k` elements.

Return `k` *after placing the final result in the first* `k` *slots of* `nums`.

Do **not** allocate extra space for another array. You must do this by **modifying the input array in-place** with O(1) extra memory.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

```
Input: nums = [1,1,1,2,2,3]
Output: 5, nums = [1,1,2,2,3,_]
Explanation: Your function should return k = 5, with the first five elements of nums being 1, 1, 2, 2
and 3 respectively.
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Example 2:

```
Input: nums = [0,0,1,1,1,1,2,3,3]
Output: 7, nums = [0,0,1,1,2,3,3,_,_]
Explanation: Your function should return k = 7, with the first seven elements of nums being 0, 0, 1, 1,
2, 3 and 3 respectively.
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Constraints:

- $1 <= nums.length <= 3 * 10^4$
- $-10^4 <= nums[i] <= 10^4$
- `nums` is sorted in **non-decreasing** order.

SOLUTION (PYTHON 3)

Solution 1: Single Pass

How to think about it

Copy into a new array, at most two of each value: correct, but the problem wants in-place O(1) extra space. $n \le 3\times 10^4$, so one pass. Sorted runs are contiguous; problem 26 keeps one copy, here we keep two. Let k be the length of the kept prefix: x may be written iff fewer than two are kept, or it differs from the second-to-last kept value — otherwise it would be a third copy. Write at nums[k], bump k, return k.

We use a variable k to record the current length of the array that has been processed. Initially, k=0, representing an empty array.

Then we traverse the array from left to right. For each element x we traverse, if k < 2 or x ≠ nums[k-2], we put x in the position of nums[k], and then increment k by 1. Otherwise, x is the same as nums[k-2], we directly skip this element. Continue to traverse until the entire array is traversed.

In this way, when the traversal ends, the first k elements in nums are the answer we want, and k is the length of the answer.

The time complexity is O(n), and the space complexity is O(1). Here, n is the length of the array.

Supplement:

The original problem requires that the same number appears at most 2 times. We can extend it to keep at most k identical numbers.

- Since the same number can be kept at most k times, we can directly keep the first k elements of the original array;
- For the later numbers, the premise of being able to keep them is: the current number x compares with the kth element from the end of the previously kept numbers. If they are different, keep it, otherwise skip it.

Similar problems:

- 26. Remove Duplicates from Sorted Array

Python 3

```python
class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        k = 0
        for x in nums:
            if k < 2 or x != nums[k - 2]:
                nums[k] = x
                k += 1
        return k
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Allow at most K copies of each value
# 2. The array is not sorted: keep only the first K copies
# of every value


# ============================================================
# FOLLOW-UP 1: Allow at most K copies of each value
# ============================================================

# ---- follow-up 1: function ----
# Answer: A number may be written when it differs from the
# number K places behind the write position (that number
# would be its (K+1)-th copy).
# Time: O(n).  Space: O(1).
def remove_duplicates_at_most_k(nums, k):
    write = 0
    for read in range(len(nums)):
```

```
        if write < k or nums[read] != nums[write - k]:
            nums[write] = nums[read]
            write += 1
    return write


# ========================================================
# FOLLOW-UP 2: The array is not sorted: keep only the
# first K copies of every value
# ========================================================

# ---- follow-up 2: function ----
# Answer: Count how many copies of each value have been
# kept so far.
# Time: O(n).  Space: O(n).
def keep_first_k_copies(nums, k):
    kept = {}
    write = 0
    for read in range(len(nums)):
        value = nums[read]
        if kept.get(value, 0) < k:
            kept[value] = kept.get(value, 0) + 1
            nums[write] = value
            write += 1
    return write
```

PATTERN 3 · TWO POINTERS · PROBLEM 11 OF 20

75. Sort Colors

Medium · Array, Two Pointers, Bubble Sort, Sorting, Quick Sort

leetcode.com/problems/sort-colors

★ **TOP 7 PROBLEM** in Two Pointers, Sorting, Cyclic Sort: master this one first

Also appears in pattern 23 Sorting, 33 Cyclic Sort

PROBLEM

You are given an array `nums` with `n` objects colored red, white, or blue, sort them **in-place** so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

Input: nums = [2,0,2,1,1,0]

Output: [0,0,1,1,2,2]

Explanation:

The array has two 0s, two 1s, and two 2s. Sorting them in-place places all 0s first, then all 1s, then all 2s.

Example 2:

Input: nums = [2,0,1]

Output: [0,1,2]

Explanation:

The array has one each of 0, 1, and 2, arranged in-place in the order 0, 1, 2.

Constraints:

- `n == nums.length`
- `1 <= n <= 300`
- `nums[i]` is either 0, 1, or 2.

Follow up: Could you come up with a one-pass algorithm using only constant extra space?

SOLUTION (PYTHON 3)

Solution 1: Three Pointers

How to think about it

Count 0,1,2 then overwrite: two passes. Sorting would also work for n ≤ 300. The problem forbids library sort, and the follow-up wants one scan and O(1) space. Three values only need a partition into all 0s, all 1s, and all 2s. Let i and j bound the 0s and 2s already placed, and let k scan the unknown middle. A swap toward 2 brings an unseen value, so k stays; a swap toward 0 brings a value from the scanned range, so k advances too. One pass finishes the three segments.

We define three pointers i, j, and k. Pointer i is used to point to the rightmost boundary of the elements with a value of 0 in the array, and pointer j is used to point to the leftmost boundary of the elements with a value of 2 in the array. Initially, i=-1, j=n. Pointer k is used to point to the current element being traversed, initially k=0.

When k < j, we perform the following operations:

- If nums[k] = 0, then swap it with nums[i+1], then increment both i and k by 1;
- If nums[k] = 2, then swap it with nums[j-1], then decrement j by 1;

- If nums[k] = 1, then increment k by 1.

After the traversal, the elements in the array are divided into three parts: [0,i], [i+1,j-1] and [j,n-1].

The time complexity is O(n), where n is the length of the array. Only one traversal of the array is needed. The space complexity is O(1).

Python 3

```
class Solution:
    def sortColors(self, nums: List[int]) -> None:
        i, j, k = -1, len(nums), 0
        while k < j:
            if nums[k] == 0:
                i += 1
                nums[i], nums[k] = nums[k], nums[i]
                k += 1
            elif nums[k] == 2:
                j -= 1
                nums[j], nums[k] = nums[k], nums[j]
            else:
                k += 1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. K different colours instead of three
# 2. Only two colours (0 and 1): one pass, two pointers


# ============================================================
# FOLLOW-UP 1: K different colours instead of three
# ============================================================

# ---- follow-up 1: function ----
# Answer: Counting sort: count each colour, then write the
# colours back in order. O(n + k) time, O(k) extra space.
# Time: O(n + k).  Space: O(k).
def sort_k_colors(nums, k):
    count = [0] * k
    for x in nums:
        count[x] += 1
    index = 0
    for colour in range(k):
        for repeat in range(count[colour]):
            nums[index] = colour
            index += 1


# ============================================================
# FOLLOW-UP 2: Only two colours (0 and 1): one pass, two
# pointers
# ============================================================

# ---- follow-up 2: function ----
# Answer: Left stops on a 1 that is out of place and right
# on a 0 that is out of place; swap them.
# Time: O(n).  Space: O(1).
def sort_two_colors(nums):
    left, right = 0, len(nums) - 1
    while left < right:
        if nums[left] == 0:
            left += 1
        elif nums[right] == 1:
            right -= 1
        else:
            nums[left], nums[right] = nums[right], nums[left]
            left += 1
```

```
            right -= 1
```

PATTERN 3 · TWO POINTERS · PROBLEM 12 OF 20

189. Rotate Array

Medium · Array, Math, Two Pointers

leetcode.com/problems/rotate-array

★ **TOP 7 PROBLEM** in Simulation / Array Rearrangement: master this one first

Also appears in pattern 36 Simulation / Array Rearrangement

PROBLEM

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

Example 1:

```
Input: nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
Explanation:
rotate 1 steps to the right: [7,1,2,3,4,5,6]
rotate 2 steps to the right: [6,7,1,2,3,4,5]
rotate 3 steps to the right: [5,6,7,1,2,3,4]
```

Example 2:

```
Input: nums = [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]
Explanation:
rotate 1 steps to the right: [99,-1,-100,3]
rotate 2 steps to the right: [3,99,-1,-100]
```

Constraints:

- `1 <= nums.length <= 10^5`
- `-2^31 <= nums[i] <= 2^31 - 1`
- `0 <= k <= 10^5`

Follow up:

- Try to come up with as many solutions as you can. There are at least **three** different ways to solve this problem.
- Could you do it in-place with `O(1)` extra space?

SOLUTION (PYTHON 3)

Solution 1: Reverse three times

How to think about it

Rotate right by k. A second array is O(n) space; n≤ 10^5, and the follow-up wants in-place. A right rotate by k moves the last k items to the front. Reverse the whole array, then reverse the first k and the last n-k. Three reversals, only swaps.

We can assume the length of the array is n and calculate the actual number of steps needed by taking the module of k and n, which is k mod n.

Next, let us reverse three times to get the final result:

1. Reverse the entire array.
2. Reverse the first k elements.
3. Reverse the last n - k elements.

For example, for the array [1, 2, 3, 4, 5, 6, 7], k = 3, n = 7, k mod n = 3.

1. In the first reverse, reverse the entire array. We get [7, 6, 5, 4, 3, 2, 1].
2. In the second reverse, reverse the first k elements. We get [5, 6, 7, 4, 3, 2, 1].

3. In the third reverse, reverse the last n - k elements. We get [5, 6, 7, 1, 2, 3, 4], which is the final result.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Python 3

```python
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        def reverse(i: int, j: int):
            while i < j:
                nums[i], nums[j] = nums[j], nums[i]
                i, j = i + 1, j - 1

        n = len(nums)
        k %= n
        reverse(0, n - 1)
        reverse(0, k - 1)
        reverse(k, n - 1)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Rotate to the LEFT by k
# 2. Is one string a rotation of another?


# ============================================================
# FOLLOW-UP 1: Rotate to the LEFT by k
# ============================================================

# ---- follow-up 1: helper functions ----
def reverse_part(nums, left, right):
    while left < right:
        nums[left], nums[right] = nums[right], nums[left]
        left += 1
        right -= 1

# ---- follow-up 1: function ----
# Answer: Rotating left by k is the same as rotating right
# by n - k; with three reversals: reverse the first k,
# reverse the rest, reverse everything.
# Time: O(n).  Space: O(1).
def rotate_left(nums, k):
    n = len(nums)
    if n == 0:
        return
    k %= n
    reverse_part(nums, 0, k - 1)
    reverse_part(nums, k, n - 1)
    reverse_part(nums, 0, n - 1)


# ============================================================
# FOLLOW-UP 2: Is one string a rotation of another?
# ============================================================

# ---- follow-up 2: function ----
# Answer: If b is a rotation of a, then b appears inside a
# + a, and the lengths are equal.
# Time: O(n).  Space: O(n).
def is_rotation(a, b):
    if len(a) != len(b):
        return False
    return b in a + a
```

PATTERN 3 · TWO POINTERS · PROBLEM 13 OF 20

18. 4Sum

Medium · Array, Two Pointers, Sorting

leetcode.com/problems/4sum

★ **TOP 7 PROBLEM** in Two Pointers: master this one first

PROBLEM

Given an array `nums` of `n` integers, return *an array of all the **unique** quadruplets* `[nums[a], nums[b], nums[c], nums[d]]` such that:

- `0 <= a, b, c, d < n`
- `a`, `b`, `c`, and `d` are **distinct**.
- `nums[a] + nums[b] + nums[c] + nums[d] == target`

You may return the answer in **any order**.

Example 1:

```
Input: nums = [1,0,-1,0,-2,2], target = 0
Output: [[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]
```

Example 2:

```
Input: nums = [2,2,2,2,2], target = 8
Output: [[2,2,2,2]]
```

Constraints:

- `1 <= nums.length <= 200`
- `-10^9 <= nums[i] <= 10^9`
- `-10^9 <= target <= 10^9`

SOLUTION (PYTHON 3)

Solution 1: Sorting + Double Pointers

How to think about it

The first idea is four nested loops plus a set. With n≤ 200, $O(n^4)$ is tight, and deduplication is messy. The bottleneck is the same as 3Sum: after fixing two numbers, the rest is two-sum. Brute force is $O(n^2)$; sorted two pointers are O(n). After sorting, duplicates sit together, so skipping equal i and j keeps quadruplets unique. So we enumerate two indices and squeeze the rest with a pair of pointers, $O(n^3)$, with no hash table.

We notice that the problem requires us to find non-repeating quadruplets. Therefore, we can first sort the array, which makes it easy to skip duplicate elements.

Next, we enumerate the first two elements of the quadruplet, nums[i] and nums[j], where i < j. During the enumeration process, we skip duplicate nums[i] and nums[j]. Then, we use two pointers k and l to point to the two ends behind nums[i] and nums[j]. Let x = nums[i] + nums[j] + nums[k] + nums[l], we compare x with target and perform the following operations:

- If x < target, then update k = k + 1 to get a larger x;
- If x > target, then update l = l - 1 to get a smaller x;
- Otherwise, it means that a quadruplet (nums[i], nums[j], nums[k], nums[l]) is found. Add it to the answer, then we update the pointers k and l, and skip all duplicate elements to prevent the answer from containing duplicate quadruplets, and continue to find the next quadruplet.

The time complexity is $O(n^3)$, and the space complexity is O(log n). Here, n is the length of the array.

Python 3

```python
class Solution:
    def fourSum(self, nums: List[int], target: int) -> List[List[int]]:
        n = len(nums)
        ans = []
        if n < 4:
            return ans
        nums.sort()
        for i in range(n - 3):
            if i and nums[i] == nums[i - 1]:
                continue
            for j in range(i + 1, n - 2):
                if j > i + 1 and nums[j] == nums[j - 1]:
                    continue
                k, l = j + 1, n - 1
                while k < l:
                    x = nums[i] + nums[j] + nums[k] + nums[l]
                    if x < target:
                        k += 1
                    elif x > target:
                        l -= 1
                    else:
                        ans.append([nums[i], nums[j], nums[k], nums[l]])
                        k, l = k + 1, l - 1
                        while k < l and nums[k] == nums[k - 1]:
                            k += 1
                        while k < l and nums[l] == nums[l + 1]:
                            l -= 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. K numbers instead of four (kSum)
# 2. Count the quadruplets (index sets) that add up to the
# target


# ========================================================
# FOLLOW-UP 1: K numbers instead of four (kSum)
# ========================================================

# ---- follow-up 1: function ----
# Answer: Recursion: sort once; for k > 2 fix the first
# number and solve (k - 1)Sum on the rest; for k = 2 use
# two pointers. Skip equal numbers at every level to avoid
# repeated answers.
# Time: O(n^(k-1)).  Space: O(k) recursion (plus the
# answers).
def k_sum(nums, target, k):
    nums = sorted(nums)

    def solve(start, target, k):
        result = []
        if k == 2:
            left, right = start, len(nums) - 1
            while left < right:
                total = nums[left] + nums[right]
                if total == target:
                    result.append([nums[left], nums[right]])
                    left += 1
                    right -= 1
                    while left < right and nums[left] == nums[left - 1]:
                        left += 1
                elif total < target:
                    left += 1
```

```
                else:
                    right -= 1
            return result
        for i in range(start, len(nums) - k + 1):
            if i > start and nums[i] == nums[i - 1]:
                continue
            for rest in solve(i + 1, target - nums[i], k - 1):
                result.append([nums[i]] + rest)
        return result

    return solve(0, target, k)


# ============================================================
# FOLLOW-UP 2: Count the quadruplets (index sets) that add
# up to the target
# ============================================================

# ---- follow-up 2: function ----
# Answer: Counting does not need uniqueness: store how
# many index pairs give every sum, and combine pairs that
# use four different indexes (process the second pair only
# with indexes after the first pair).
# Time: O(n^2).  Space: O(n^2).
def count_quadruplets(nums, target):
    pair_sums = {}
```

```
    count = 0
    for j in range(len(nums)):
        for k in range(j + 1, len(nums)):
            count += pair_sums.get(target - nums[j] - nums[k], 0)
        for i in range(j):                          # pairs that end before j are safe to use later
            s = nums[i] + nums[j]
            pair_sums[s] = pair_sums.get(s, 0) + 1
    return count
```

PATTERN 3 · TWO POINTERS · PROBLEM 14 OF 20

922. Sort Array By Parity II

Easy · Array, Two Pointers, Sorting

leetcode.com/problems/sort-array-by-parity-ii

PROBLEM

Given an array of integers `nums`, half of the integers in `nums` are **odd**, and the other half are **even**.

Sort the array so that whenever `nums[i]` is odd, `i` is **odd**, and whenever `nums[i]` is even, `i` is **even**.

Return *any answer array that satisfies this condition*.

Example 1:

```
Input: nums = [4,2,5,7]
Output: [4,5,2,7]
Explanation: [4,7,2,5], [2,5,4,7], [2,7,4,5] would also have been accepted.
```

Example 2:

```
Input: nums = [2,3]
Output: [2,3]
```

Constraints:

- `2 <= nums.length <= 2 * 10^4`
- `nums.length` is even.
- Half of the integers in `nums` are even.
- `0 <= nums[i] <= 1000`

Follow Up: Could you solve it in-place?

SOLUTION (PYTHON 3)

Solution 1: Two Pointers

> **How to think about it**
>
> Even indices must hold evens and odd indices odds, and the two kinds are equally many. An extra array would work, but we can swap in place. i walks even indices; when that slot is odd, j walks odd indices until an even is found and they swap. j only increases, so the pass is linear.

We use two pointers i and j to point to even and odd indices, respectively. Initially, i = 0 and j = 1.

When i points to an even index, if nums[i] is odd, we need to find an odd index j such that nums[j] is even, and then swap nums[i] and nums[j]. Continue traversing until i reaches the end of the array.

The time complexity is O(n), where n is the length of the array nums. The space complexity is O(1).

Python 3

```python
class Solution:
    def sortArrayByParityII(self, nums: List[int]) -> List[int]:
        n, j = len(nums), 1
        for i in range(0, n, 2):
            if nums[i] % 2:
                while nums[j] % 2:
                    j += 2
                nums[i], nums[j] = nums[j], nums[i]
        return nums
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Put all the even numbers first (no alternating rule)
# 2. Keep the original relative order of the evens and of
# the odds


# ============================================================
# FOLLOW-UP 1: Put all the even numbers first (no
# alternating rule)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Two pointers from the ends: swap an odd number
# on the left with an even number on the right.
# Time: O(n).  Space: O(1).
def sort_array_by_parity(nums):
    left, right = 0, len(nums) - 1
    while left < right:
        if nums[left] % 2 == 0:
            left += 1
        elif nums[right] % 2 == 1:
            right -= 1
        else:
            nums[left], nums[right] = nums[right], nums[left]
            left += 1
            right -= 1
    return nums


# ============================================================
# FOLLOW-UP 2: Keep the original relative order of the
# evens and of the odds
# ============================================================

# ---- follow-up 2: function ----
# Answer: Swapping breaks the order, so collect the evens
# and the odds into two lists in one pass and put them
# back at the even and the odd positions.
# Time: O(n).  Space: O(n).
def alternate_parity_stable(nums):
    evens = []
    odds = []
    for x in nums:
        if x % 2 == 0:
            evens.append(x)
        else:
            odds.append(x)
    result = []
    for i in range(len(evens)):
        result.append(evens[i])
        result.append(odds[i])
    return result
```

PATTERN 3 · TWO POINTERS · PROBLEM 15 OF 20

11. Container With Most Water

Medium · Greedy, Array, Two Pointers

leetcode.com/problems/container-with-most-water

★ **TOP 7 PROBLEM** in Two Pointers: master this one first

PROBLEM

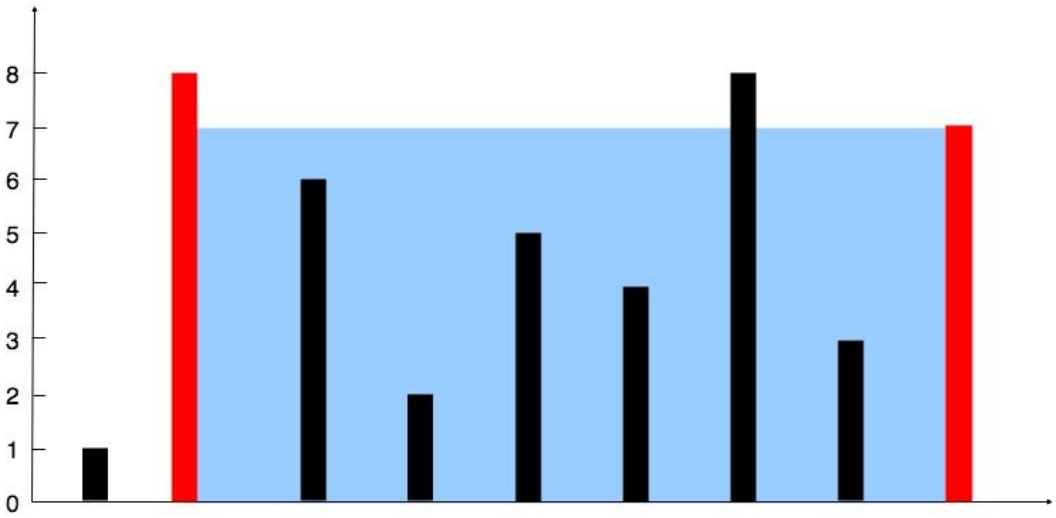
You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the i^{th} line are `(i, 0)` and `(i, height[i])`.

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return *the maximum amount of water a container can store*.

Notice that you may not slant the container.

Example 1:




```
Input: height = [1,8,6,2,5,4,8,3,7]
Output: 49
Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the
max area of water (blue section) the container can contain is 49.
```

Example 2:

```
Input: height = [1,1]
Output: 1
```

Constraints:

- `n == height.length`
- `2 <= n <= 10^5`
- `0 <= height[i] <= 10^4`

SOLUTION (PYTHON 3)

Solution 1: Two Pointers

How to think about it

The first idea is to try every pair (i,j) and take min(height[i],height[j])×(j-i). Correct, but O(n^2). n ≤ 10^5 will time out. The bottleneck is treating every pair independently and ignoring that height is fixed by the shorter line. Start from the widest interval [0,n-1]: moving the taller end inward shrinks the width while the height is still capped by the shorter line, so the area cannot improve. We must drop the shorter line and look for a taller one. So we shrink from both ends, always moving the shorter side. Each index is visited at most once.

We use two pointers l and r to point to the left and right ends of the array, respectively, i.e., l = 0 and r = n - 1, where n is the length of the array.

Next, we use a variable ans to record the maximum capacity of the container, initially set to 0.

Then, we start a loop. In each iteration, we calculate the current capacity of the container, i.e., min(height[l], height[r]) × (r - l), and compare it with ans, assigning the larger value to ans. Then, we compare the values of height[l] and height[r]. If height[l] < height[r], moving the r pointer will not improve the result because the height of the container is determined by the shorter vertical line, so we move the l pointer. Otherwise, we move the r pointer.

After the iteration, we return ans.

The time complexity is O(n), where n is the length of the array height. The space complexity is O(1).

Python 3

```python
class Solution:
    def maxArea(self, height: List[int]) -> int:
        l, r = 0, len(height) - 1
        ans = 0
        while l < r:
            t = min(height[l], height[r]) * (r - l)
            ans = max(ans, t)
            if height[l] < height[r]:
                l += 1
            else:
                r -= 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the two bar positions as well as the area


# ============================================================
# FOLLOW-UP 1: Return the two bar positions as well as the
# area
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same two pointers; remember the pair that gave
# the best area. Always move the pointer at the shorter
# bar: moving the taller one can only make the area
# smaller.
# Time: O(n).  Space: O(1).
def best_container(height):
    left, right = 0, len(height) - 1
    best_area = 0
    best_pair = (0, 0)
    while left < right:
        shorter = height[left]
        if height[right] < shorter:
```

```
            shorter = height[right]
        area = shorter * (right - left)
        if area > best_area:
            best_area = area
            best_pair = (left, right)
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1
    return best_area, best_pair
```

PATTERN 3 · TWO POINTERS · PROBLEM 16 OF 20

763. Partition Labels

Medium · Greedy, Hash Table, Two Pointers, String

leetcode.com/problems/partition-labels

★ **TOP 7 PROBLEM** in Greedy: master this one first

Also appears in pattern 20 Intervals (Merge / Insert / Overlap), 21 Greedy

PROBLEM

You are given a string `s`. We want to partition the string into as many parts as possible so that each letter appears in at most one part. For example, the string `"ababcc"` can be partitioned into `["abab", "cc"]`, but partitions such as `["aba", "bcc"]` or `["ab", "ab", "cc"]` are invalid.

Note that the partition is done so that after concatenating all the parts in order, the resultant string should be `s`.

Return *a list of integers representing the size of these parts*.

Example 1:

```
Input: s = "ababcbacadefegdehijhklij"
Output: [9,7,8]
Explanation:
The partition is "ababcbaca", "defegde", "hijhklij".
This is a partition so that each letter appears in at most one part.
A partition like "ababcbacadefegde", "hijhklij" is incorrect, because it splits s into less parts.
```

Example 2:

```
Input: s = "eccbbbbdec"
Output: [10]
```

Constraints:

- `1 <= s.length <= 500`
- `s` consists of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Greedy

How to think about it

Split s into as many parts as possible so no letter appears in two parts. A letter's last index is how far its part must reach. Scan left to right, extending the current end by last[c]. When the scan index meets that end, the part is closed. Build last, then cut in one pass. O(n).

We first use an array or hash table last to record the last occurrence of each letter in the string s.

Next, we use a greedy approach to partition the string into as many segments as possible.

Traverse the string s from left to right, while maintaining the start index j and end index i of the current segment, both initially set to 0.

For each letter c visited, get the last occurrence position last[c]. Since the end index of the current segment must not be less than last[c], let mx = max(mx, last[c]).

When visiting the index mx, it means the current segment ends. The index range of the current segment is [j,.. i], and the length is i - j + 1. We add this length to the result array. Then set j = i + 1 and continue to find the next segment.

Repeat the above process until the string traversal is complete to get the lengths of all segments.

Time complexity is O(n), and space complexity is O(|Σ|). Where n is the length of the string s, and |Σ| is the size of the character set. In this problem, |Σ| = 26.

Python 3

```python
class Solution:
    def partitionLabels(self, s: str) -> List[int]:
        last = {c: i for i, c in enumerate(s)}
        mx = j = 0
        ans = []
        for i, c in enumerate(s):
            mx = max(mx, last[c])
            if mx == i:
                ans.append(i - j + 1)
                j = i + 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return where each part starts and ends instead of its
# size
# 2. See it as merging intervals: each letter covers the
# range from its first to its last position


# ============================================================
# FOLLOW-UP 1: Return where each part starts and ends
# instead of its size
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same scan with the last position of every
# letter; when the current position reaches the farthest
# last position, close the part and start the next one
# right after it.
# Time: O(n).  Space: O(s), s = distinct letters.
def partition_ranges(s):
    last = {}
    for i in range(len(s)):
        last[s[i]] = i
    ranges = []
    start = 0
    farthest = 0
    for i in range(len(s)):
        if last[s[i]] > farthest:
            farthest = last[s[i]]
        if i == farthest:
            ranges.append((start, i))
            start = i + 1
    return ranges


# ============================================================
# FOLLOW-UP 2: See it as merging intervals: each letter
# covers the range from its first to its last position
# ============================================================

# ---- follow-up 2: function ----
# Answer: Build one interval per letter and merge the
# overlapping ones; every merged interval is one part.
# Time: O(n + s log s).  Space: O(s).
def partition_by_merging(s):
    first = {}
    last = {}
    for i in range(len(s)):
        if s[i] not in first:
```

```
            first[s[i]] = i
        last[s[i]] = i
    intervals = []
    for ch in first:
        intervals.append((first[ch], last[ch]))
    intervals.sort()
    merged = []
    for start, end in intervals:
        if merged and start <= merged[-1][1]:
            if end > merged[-1][1]:
                merged[-1][1] = end
        else:
```

```
            merged.append([start, end])
    sizes = []
    for start, end in merged:
        sizes.append(end - start + 1)
    return sizes
```

PATTERN 3 · TWO POINTERS · PROBLEM 17 OF 20

881. Boats to Save People

Medium · Greedy, Array, Two Pointers, Sorting, Timsort

leetcode.com/problems/boats-to-save-people

★ **TOP 7 PROBLEM** in Two Pointers: master this one first

Also appears in pattern 21 Greedy, 23 Sorting

PROBLEM

You are given an array `people` where `people[i]` is the weight of the i^{th} person, and an **infinite number of boats** where each boat can carry a maximum weight of `limit`. Each boat carries at most two people at the same time, provided the sum of the weight of those people is at most `limit`.

Return *the minimum number of boats to carry every given person*.

Example 1:

```
Input: people = [1,2], limit = 3
Output: 1
Explanation: 1 boat (1, 2)
```

Example 2:

```
Input: people = [3,2,2,1], limit = 3
Output: 3
Explanation: 3 boats (1, 2), (2) and (3)
```

Example 3:

```
Input: people = [3,5,3,4], limit = 5
Output: 4
Explanation: 4 boats (3), (3), (4), (5)
```

Constraints:

- `1 <= people.length <= 5 * 10^4`
- `1 <= people[i] <= limit <= 3 * 10^4`

SOLUTION (PYTHON 3)

Solution 1: Greedy + Two Pointers

How to think about it

Each boat holds at most two people whose weights sum to at most limit. $n \le 5 \cdot 10^4$, so pair the heaviest with the lightest when possible. Sort and use two pointers: if the ends fit, both board; otherwise the heavier one goes alone. Each step uses one boat until the pointers cross.

After sorting, use two pointers to point to the beginning and end of the array respectively. Each time, compare the sum of the elements pointed to by the two pointers with `limit`. If it is less than or equal to `limit`, then both pointers move one step towards the middle. Otherwise, only the right pointer moves. Accumulate the answer.

The time complexity is $O(n \times \log n)$, and the space complexity is $O(\log n)$. Here, n is the length of the array `people`.

Python 3

```python
class Solution:
    def numRescueBoats(self, people: List[int], limit: int) -> int:
        people.sort()
        ans = 0
        i, j = 0, len(people) - 1
        while i <= j:
            if people[i] + people[j] <= limit:
                i += 1
            j -= 1
            ans += 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return which people share a boat


# ============================================================
# FOLLOW-UP 1: Return which people share a boat
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same greedy two pointers over the sorted
# weights: the heaviest person always leaves, and the
# lightest comes along when the two fit.
# Time: O(n log n).  Space: O(n).
def boat_assignments(people, limit):
    weights = sorted(people)
    left, right = 0, len(weights) - 1
    boats = []
    while left <= right:
        if left < right and weights[left] + weights[right] <= limit:
            boats.append((weights[left], weights[right]))
            left += 1
        else:
            boats.append((weights[right],))
        right -= 1
    return boats
```

1498. Number of Subsequences That Satisfy the Given Sum Condition

Medium · Array, Two Pointers, Binary Search, Sorting

leetcode.com/problems/number-of-subsequences-that-satisfy-the-given-sum-condition

PROBLEM

You are given an array of integers `nums` and an integer `target`.

Return *the number of **non-empty** subsequences of* `nums` *such that the sum of the minimum and maximum element on it is less or equal to* `target`. Since the answer may be too large, return it **modulo** $10^9 + 7$.

Example 1:

```
Input: nums = [3,5,6,7], target = 9
Output: 4
Explanation: There are 4 subsequences that satisfy the condition.
[3] -> Min value + max value <= target (3 + 3 <= 9)
[3,5] -> (3 + 5 <= 9)
[3,5,6] -> (3 + 6 <= 9)
[3,6] -> (3 + 6 <= 9)
```

Example 2:

```
Input: nums = [3,3,6,8], target = 10
Output: 6
Explanation: There are 6 subsequences that satisfy the condition. (nums can have repeated numbers).
[3] , [3] , [3,3], [3,6] , [3,6] , [3,3,6]
```

Example 3:

```
Input: nums = [2,3,3,4,6,7], target = 12
Output: 61
Explanation: There are 63 non-empty subsequences, two of them do not satisfy the condition ([6,7],
[7]).
Number of valid subsequences (63 - 2 = 61).
```

Constraints:

- $1 \le$ `nums.length` $\le 10^5$
- $1 \le$ `nums[i]` $\le 10^6$
- $1 \le$ `target` $\le 10^6$

SOLUTION (PYTHON 3)

Solution 1: Sorting + Binary Search

> **How to think about it**
>
> A subsequence is judged only by its min and max, so sorting does not change those pairs. n≤ 10^5. Fix the minimum nums[i], binary-search the largest legal nums[j], and multiply by 2^{j-i} for free choices in between. Precompute powers of two. Once 2· nums[i]>target, larger minima are impossible.

Since the problem is about subsequences and involves the sum of the minimum and maximum elements, we can first sort the array nums.

Then we enumerate the minimum element nums[i]. For each nums[i], we can find the maximum element nums[j] in nums[i + 1] to nums[n - 1] such that nums[i] + nums[j] ≤ target. The number of valid subsequences in this case is $2^{j - i}$, where $2^{j - i}$ represents all possible subsequences from nums[i + 1] to nums[j]. We sum up the counts of all such subsequences.

The time complexity is O(n × log n), and the space complexity is O(n), where n is the length of the array nums.

Python 3

```python
class Solution:
    def numSubseq(self, nums: List[int], target: int) -> int:
        mod = 10**9 + 7
        nums.sort()
        n = len(nums)
        f = [1] + [0] * n
        for i in range(1, n + 1):
            f[i] = f[i - 1] * 2 % mod
        ans = 0
        for i, x in enumerate(nums):
            if x * 2 > target:
                break
            j = bisect_right(nums, target - x, i + 1) - 1
            ans = (ans + f[j - i]) % mod
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count the subsequences whose minimum plus maximum is
# AT LEAST the target
# 2. The size of the LARGEST subsequence with minimum +
# maximum at most the target


# ============================================================
# FOLLOW-UP 1: Count the subsequences whose minimum plus
# maximum is AT LEAST the target
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count all non-empty subsequences and subtract
# the ones with minimum + maximum below the target; or run
# the two pointers the other way: for a fixed maximum,
# count how many smaller numbers can be its partner.
# Time: O(n log n).  Space: O(n).
def count_subsequences_min_max_at_least(nums, target):
    modulo = 10 ** 9 + 7
    nums = sorted(nums)
    n = len(nums)
    powers = [1] * (n + 1)
    for i in range(1, n + 1):
        powers[i] = powers[i - 1] * 2 % modulo
    below = 0                                   # subsequences with min + max < target
    left, right = 0, n - 1
    while left <= right:
        if nums[left] + nums[right] < target:
            below = (below + powers[right - left]) % modulo
            left += 1
        else:
            right -= 1
    return (powers[n] - 1 - below) % modulo


# ============================================================
# FOLLOW-UP 2: The size of the LARGEST subsequence with
# minimum + maximum at most the target
# ============================================================

# ---- follow-up 2: function ----
# Answer: For each smallest element the window may extend
# while the biggest still fits; every element in between
# can be included, so the best size is the widest window.
```

```python
# Time: O(n log n).  Space: O(n).
def longest_subsequence_min_max_at_most(nums, target):
    nums = sorted(nums)
    right = len(nums) - 1
    best = 0
    for left in range(len(nums)):
        while right >= left and nums[left] + nums[right] > target:
            right -= 1
        if right < left:
            break
        if right - left + 1 > best:
            best = right - left + 1
    return best
```

PATTERN 3 · TWO POINTERS · PROBLEM 19 OF 20

42. Trapping Rain Water

Hard · Stack, Array, Two Pointers, Dynamic Programming, Monotonic Stack

leetcode.com/problems/trapping-rain-water

★ **TOP 7 PROBLEM** in Two Pointers, Monotonic Stack / Queue (Specific Variants): master this one first

Also appears in pattern 9 Monotonic Stack / Queue (Specific Variants)

PROBLEM

Given `n` non-negative integers representing an elevation map where the width of each bar is `1`, compute how much water it can trap after raining.

Example 1:

```
Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]
Output: 6
Explanation: The above elevation map (black section) is represented by array
[0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.
```

Example 2:

```
Input: height = [4,2,0,3,2,5]
Output: 9
```

Constraints:

- `n == height.length`
- `1 <= n <= 2 * 10^4`
- `0 <= height[i] <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

The first idea is, for each i, scan left and right for the tallest bars; water is min(leftMax, rightMax) - height[i]. Correct, but $O(n^2)$. $n \le 2 \times 10^4$ can time out. The bottleneck is recomputing left/right maxima at every i. Those maxima only depend on one side and can be precomputed in linear time. Store, at each index, the highest bar to the left (inclusive) and to the right (inclusive), then sum. The water level is the shorter of the two walls — that is the DP state.

We define left[i] as the height of the highest bar to the left of and including the position at index i, and right[i] as the height of the highest bar to the right of and including the position at index i. Therefore, the amount of rainwater that can be trapped at index i is min(left[i], right[i]) - height[i]. We traverse the array to calculate left[i] and right[i], and the final answer is $\sum_{i=0}^{n-1}$ min(left[i], right[i]) - height[i].

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array.

Python 3

```
class Solution:
    def trap(self, height: List[int]) -> int:
        n = len(height)
        left = [height[0]] * n
        right = [height[-1]] * n
        for i in range(1, n):
            left[i] = max(left[i - 1], height[i])
            right[n - i - 1] = max(right[n - i], height[n - i - 1])
        return sum(min(l, r) - h for l, r, h in zip(left, right, height))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Use O(1) extra space instead of two helper arrays
# 2. Return how much water sits on top of every bar
# 3. Trap water on a 2D height map (Trapping Rain Water
# II)


# ============================================================
# FOLLOW-UP 1: Use O(1) extra space instead of two helper
# arrays
# ============================================================

# ---- follow-up 1: function ----
# Answer: Two pointers from the ends, each side
# remembering the tallest bar it has seen. The side with
# the smaller tallest bar is the limit, so water on that
# side can be computed immediately.
# Time: O(n).  Space: O(1).
def trap_two_pointers(height):
    left, right = 0, len(height) - 1
    left_tallest = right_tallest = 0
    water = 0
    while left < right:
        if height[left] < height[right]:
            if height[left] >= left_tallest:
                left_tallest = height[left]
            else:
                water += left_tallest - height[left]
            left += 1
        else:
            if height[right] >= right_tallest:
                right_tallest = height[right]
            else:
                water += right_tallest - height[right]
            right -= 1
    return water


# ============================================================
# FOLLOW-UP 2: Return how much water sits on top of every
# bar
# ============================================================

# ---- follow-up 2: function ----
# Answer: The water above a bar is the smaller of the
# tallest bar to its left and the tallest bar to its
# right, minus the bar itself (never below zero).
# Time: O(n).  Space: O(n).
def water_per_bar(height):
    n = len(height)
    tallest_left = [0] * n
    tallest_right = [0] * n
    for i in range(n):
```

```
        previous = tallest_left[i - 1] if i > 0 else 0
        tallest_left[i] = height[i] if height[i] > previous else previous
    for i in range(n - 1, -1, -1):
        previous = tallest_right[i + 1] if i + 1 < n else 0
        tallest_right[i] = height[i] if height[i] > previous else previous
    result = []
    for i in range(n):
```

```
        level = tallest_left[i] if tallest_left[i] < tallest_right[i] else tallest_right[i]
        result.append(level - height[i])
    return result


# ============================================================
# FOLLOW-UP 3: Trap water on a 2D height map (Trapping
# Rain Water II)
# ============================================================

# ---- follow-up 3: function ----
# Answer: Start from the border cells in a min-heap.
# Always take the lowest border cell: it is the current
# water wall, so each unvisited neighbour holds (wall -
# its height) water if it is lower, and joins the border
# with height max(wall, own height).
# Time: O(m * n * log(m * n)).  Space: O(m * n).
import heapq


def trap_rain_water_2d(height_map):
    rows, cols = len(height_map), len(height_map[0])
    if rows < 3 or cols < 3:
        return 0
    seen = []
    for r in range(rows):
        seen.append([False] * cols)
    heap = []
    for r in range(rows):
        for c in range(cols):
            if r == 0 or c == 0 or r == rows - 1 or c == cols - 1:
                heapq.heappush(heap, (height_map[r][c], r, c))
                seen[r][c] = True
    water = 0
    while heap:
        wall, r, c = heapq.heappop(heap)
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and not seen[nr][nc]:
                seen[nr][nc] = True
                if height_map[nr][nc] < wall:
                    water += wall - height_map[nr][nc]
                    heapq.heappush(heap, (wall, nr, nc))
                else:
                    heapq.heappush(heap, (height_map[nr][nc], nr, nc))
    return water
```

PATTERN 3 · TWO POINTERS · PROBLEM 20 OF 20

727. Minimum Window Subsequence

Hard · String, Dynamic Programming, Sliding Window

leetcode.com/problems/minimum-window-subsequence

PROBLEM

Given strings `s1` and `s2`, return *the minimum contiguous substring part of* `s1`*, so that* `s2` *is a subsequence of the part*.

If there is no such window in `s1` that covers all characters in `s2`, return the empty string `""`. If there are multiple such minimum-length windows, return the one with the **left-most starting index**.

Example 1:

```
Input: s1 = "abcdebdde", s2 = "bde"
Output: "bcde"
Explanation:
"bcde" is the answer because it occurs before "bdde" which has the same length.
"deb" is not a smaller window because the elements of s2 in the window must occur in order.
```

Example 2:

```
Input: s1 = "jmeqksfrsdcmsiwvaovztaqenprpvnbstl", s2 = "u"
Output: ""
```

Constraints:

- `1 <= s1.length <= 2 * 10^4`
- `1 <= s2.length <= 100`
- `s1` and `s2` consist of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

Find the shortest window of s_1 that contains s_2 as a subsequence. $|s_1| \le 2 \times 10^4$ and $|s_2| \le 100$ make "try every window" unattractive. When s_2's last character is matched, the window length is determined by where that match started. Threading the start index through the DP recovers every candidate. f[i][j] is that start: on a match it inherits f[i-1][j-1] (or i when j=1), otherwise f[i-1][j]. Whenever f[i][n]>0 we update the shortest slice.

We define f[i][j] to represent the starting position of the shortest substring of the first i characters of string s1 that contains the first j characters of string s2. If it does not exist, it is 0.

We can derive the state transition equation:

f[i][j] = cases i, & j = 1 and s1[i-1] = s2[j] \ f[i - 1][j - 1], & j > 1 and s1[i-1] = s2[j-1] \ f[i - 1][j], & s1[i-1] ≠ s2[j-1] cases

Next, we only need to traverse s1. If f[i][n] > 0, update the starting position and length of the shortest substring. Finally, return the shortest substring.

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the lengths of strings s1 and s2, respectively.

Python 3

```python
class Solution:
    def minWindow(self, s1: str, s2: str) -> str:
        m, n = len(s1), len(s2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for i, a in enumerate(s1, 1):
            for j, b in enumerate(s2, 1):
                if a == b:
                    f[i][j] = i if j == 1 else f[i - 1][j - 1]
                else:
                    f[i][j] = f[i - 1][j]
        p, k = 0, m + 1
        for i, a in enumerate(s1, 1):
            if a == s2[n - 1] and f[i][n]:
                j = f[i][n] - 1
                if i - j < k:
                    k = i - j
                    p = j
        return "" if k > m else s1[p : p + k]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return every shortest window, not only the first


# ============================================================
# FOLLOW-UP 1: Return every shortest window, not only the
# first
# ============================================================

# ---- follow-up 1: function ----
# Answer: For each place where the forward scan finishes
# matching t, scan backwards to find the latest start that
# still matches: that gives the tightest window ending
# there. Collect the windows of the smallest length.
# Time: O(n * m), n = len(s), m = len(t).  Space: O(W *
# L), W windows of length L.
def all_min_windows_subsequence(s, t):
    windows = []
    best_length = len(s) + 1
    if t == "":
        return windows
    i = 0
    while i < len(s):
        j = 0
        while i < len(s):                          # forward: match t as a subsequence
            if s[i] == t[j]:
                j += 1
                if j == len(t):
                    break
            i += 1
        if i == len(s):
            break
        end = i
        j = len(t) - 1
        while j >= 0:                              # backward: tighten the start
            if s[i] == t[j]:
                j -= 1
            i -= 1
        start = i + 1
        length = end - start + 1
        if length < best_length:
            best_length = length
            windows = [s[start:end + 1]]
        elif length == best_length:
```

```
            windows.append(s[start:end + 1])
        i = start + 1
    return windows
```

PATTERN 4 OF 41

Binary Search

20 problems · 3 Easy · 15 Medium · 2 Hard

THE IDEA

Cut the search space in half every step. It works on any yes/no condition that is false, false, ..., true, true (monotonic), not only on sorted arrays.

Signals: sorted or rotated array, 'minimum value such that ...', 'maximum value that still works', O(log n) required, answer lies in a numeric range.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
lo, hi = 0, len(a) - 1            # or the range of possible answers
while lo < hi:
    mid = (lo + hi) // 2
    if good(mid):                  # first index where the condition is true
        hi = mid
    else:
        lo = mid + 1
return lo
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Binary Search	Easy	LC 704	184
2	Search Insert Position	Easy	LC 35	187
3	Sqrt(x)	Easy	LC 69	190
4	★ **Find First and Last Position of Element in Sorted Array**	Medium	LC 34	192
5	★ **Search in Rotated Sorted Array**	Medium	LC 33	194
6	Search in Rotated Sorted Array II	Medium	LC 81	197
7	★ **Find Minimum in Rotated Sorted Array**	Medium	LC 153	200
8	Search a 2D Matrix	Medium	LC 74	202
9	★ **Koko Eating Bananas**	Medium	LC 875	206
10	Capacity To Ship Packages Within D Days	Medium	LC 1011	209
11	★ **Find Peak Element**	Medium	LC 162	212
12	Find the Smallest Divisor Given a Threshold	Medium	LC 1283	215
13	Kth Smallest Element in a Sorted Matrix	Medium	LC 378	217
14	Single Element in a Sorted Array	Medium	LC 540	220
15	Time Based Key-Value Store	Medium	LC 981	222
16	Minimum Number of Days to Make m Bouquets	Medium	LC 1482	225
17	Find K Closest Elements	Medium	LC 658	228
18	Peak Index in a Mountain Array	Medium	LC 852	231
19	★ **Median of Two Sorted Arrays**	Hard	LC 4	233
20	★ **Split Array Largest Sum**	Hard	LC 410	236

PATTERN 4 · BINARY SEARCH · PROBLEM 1 OF 20

704. Binary Search

Easy · Array, Binary Search

leetcode.com/problems/binary-search

PROBLEM

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [-1,0,3,5,9,12], target = 9
Output: 4
Explanation: 9 exists in nums and its index is 4
```

Example 2:

```
Input: nums = [-1,0,3,5,9,12], target = 2
Output: -1
Explanation: 2 does not exist in nums so return -1
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^4 < nums[i], target < 10^4`
- All the integers in `nums` are **unique**.
- `nums` is sorted in ascending order.

SOLUTION (PYTHON 3)

Solution 1: Binary Search

> **How to think about it**
>
> Search target in a strictly increasing array. n ≤ 10^4 would allow a scan, but each comparison can discard half of a sorted range. If nums[mid] ≥ target, the answer is not to the right, so r moves to mid; otherwise it lies past mid, so l becomes mid+1. When l=r we sit on the first value that is not smaller than the target, then compare once. Time O(log n), extra space O(1).

We define the left boundary l=0 and the right boundary r=n-1 for binary search.

In each iteration, we calculate the middle position mid=(l+r)/2, then compare the size of nums[mid] and target.

- If nums[mid] ≥ target, it means target is in the left half, so we move the right boundary r to mid;
- Otherwise, it means target is in the right half, so we move the left boundary l to mid+1.

The loop ends when l<r, at this point nums[l] is the target value we are looking for. If nums[l]=target, return l; otherwise, return -1.

The time complexity is O(log n), where n is the length of the array nums. The space complexity is O(1).

Python 3

```python
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        l, r = 0, len(nums) - 1
        while l < r:
            mid = (l + r) >> 1
            if nums[mid] >= target:
                r = mid
            else:
                l = mid + 1
        return l if nums[l] == target else -1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The array has duplicates: find the first and the last
# position of a value
# 2. The array's length is unknown (you can only read
# element i, and a read past the end gives a huge number)


# ============================================================
# FOLLOW-UP 1: The array has duplicates: find the first
# and the last position of a value
# ============================================================

# ---- follow-up 1: function ----
# Answer: Two searches that keep going after a hit: lower
# bound = first index with a value >= target, upper bound
# = first index with a value > target. The last position
# is upper bound - 1.
# Time: O(log n).  Space: O(1).
def lower_bound(nums, target):
    low, high = 0, len(nums)
    while low < high:
        mid = (low + high) // 2
        if nums[mid] < target:
            low = mid + 1
        else:
            high = mid
    return low


def upper_bound(nums, target):
    low, high = 0, len(nums)
    while low < high:
        mid = (low + high) // 2
        if nums[mid] <= target:
            low = mid + 1
        else:
            high = mid
    return low


# ============================================================
# FOLLOW-UP 2: The array's length is unknown (you can only
# read element i, and a read past the end gives a huge
# number)
# ============================================================

# ---- follow-up 2: function ----
# Answer: First find a range that must contain the target
# by doubling the right end (1, 2, 4, 8, ...), then binary
# search inside it.
# Time: O(log p), p = position of the target.  Space:
# O(1).
```

```
def search_unknown_length(read, target):
    # read(i) returns the i-th element, or a very large number past the end
    right = 1
    while read(right) < target:
        right *= 2
    left = right // 2
    while left <= right:
        mid = (left + right) // 2
        value = read(mid)
        if value == target:
            return mid
        if value < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

PATTERN 4 · BINARY SEARCH · PROBLEM 2 OF 20

35. Search Insert Position

Easy · Array, Binary Search

leetcode.com/problems/search-insert-position

PROBLEM

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [1,3,5,6], target = 5
Output: 2
```

Example 2:

```
Input: nums = [1,3,5,6], target = 2
Output: 1
```

Example 3:

```
Input: nums = [1,3,5,6], target = 7
Output: 4
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^4 <= nums[i] <= 10^4`
- `nums` contains **distinct** values sorted in **ascending** order.
- `-10^4 <= target <= 10^4`

SOLUTIONS (PYTHON 3)

Solution 1: Binary Search

How to think about it

The first idea is to scan left to right until the first index ≥ target. Correct, and n ≤ 10^4 would pass, but the array is strictly increasing, so a linear scan wastes the order. The insertion point is exactly the first index not smaller than target — a standard lower bound. Keep a half-open interval [l,r): if nums[mid] ≥ target the answer lies in the left half (including mid), otherwise it lies to the right of mid. When l=r, l is the insertion index.

Since the array nums is already sorted, we can use the binary search method to find the insertion position of the target value target.

The time complexity is O(log n), and the space complexity is O(1). Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        l, r = 0, len(nums)
        while l < r:
            mid = (l + r) >> 1
            if nums[mid] >= target:
                r = mid
            else:
                l = mid + 1
        return l
```

Solution 2: Binary Search (Built-in Function)

> **How to think about it**
>
> Method 1 is already O(log n); what it still lacks is only that we need not write the binary search by hand. Languages already expose a lower bound: Python's `bisect_left`, C++'s `lower_bound`, Java's `Arrays.binarySearch` (returning -i-1 when absent). The meaning matches Method 1; we just call the builtin.

We can also directly use the built-in function for binary search.

The time complexity is O(log n), where n is the length of the array nums. The space complexity is O(1).

Python 3

```
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        return bisect_left(nums, target)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Count how many times a value appears, in O(log n)
# 2. Return the element closest to the target


# ============================================================
# FOLLOW-UP 1: Count how many times a value appears, in
# O(log n)
# ============================================================

# ---- follow-up 1: helper functions ----
def first_not_less(nums, target):
    low, high = 0, len(nums)
    while low < high:
        mid = (low + high) // 2
        if nums[mid] < target:
            low = mid + 1
        else:
            high = mid
    return low

# ---- follow-up 1: function ----
# Answer: The number of copies is the position after the
# last copy minus the position of the first copy: two
# binary searches.
# Time: O(log n).  Space: O(1).
def count_occurrences(nums, target):
    return first_not_less(nums, target + 1) - first_not_less(nums, target)


# ============================================================
# FOLLOW-UP 2: Return the element closest to the target
# ============================================================

# ---- follow-up 2: function ----
# Answer: The insert position tells which two elements are
# the candidates: the one just before it and the one at
# it. Compare their distances.
# Time: O(log n).  Space: O(1).
def closest_value(nums, target):
    position = first_not_less(nums, target)
    if position == 0:
        return nums[0]
    if position == len(nums):
        return nums[-1]
    before, after = nums[position - 1], nums[position]
```

```
    if target - before <= after - target:
        return before
    return after
```

PATTERN 4 · BINARY SEARCH · PROBLEM 3 OF 20

69. Sqrt(x)

Easy · Math, Binary Search, Newton's Method

leetcode.com/problems/sqrtx

Also appears in pattern 34 Math / Number Theory

PROBLEM

Given a non-negative integer `x`, return *the square root of* `x` *rounded down to the nearest integer*. The returned integer should be **non-negative** as well.

You **must not use** any built-in exponent function or operator.

- For example, do not use `pow(x, 0.5)` in c++ or `x ** 0.5` in python.

Example 1:

```
Input: x = 4
Output: 2
Explanation: The square root of 4 is 2, so we return 2.
```

Example 2:

```
Input: x = 8
Output: 2
Explanation: The square root of 8 is 2.82842..., and since we round it down to the nearest integer, 2
is returned.
```

Constraints:

- $0 <= x <= 2^{31} - 1$

SOLUTION (PYTHON 3)

Solution 1: Binary Search

> **How to think about it**
>
> The first idea is to scan i from 0 to x and keep the largest i with $i^2 \le x$. x can be 2^{31}-1, so a linear scan is too slow, i^2 overflows easily, and `pow` is forbidden. The answer is monotone: if $mid^2 \le x$, a larger candidate might still work; otherwise we must go left. So we binary-search [0, x]. Compare with mid > x / mid to avoid overflow. Use the upper mid and shrink toward the largest feasible value; the left endpoint is the integer square root.

We define the left boundary of the binary search as l = 0 and the right boundary as r = x, then we search for the square root within the range [l, r].

In each step of the search, we find the middle value mid = (l + r + 1) / 2. If mid > x / mid, it means the square root is within the range [l, mid - 1], so we set r = mid - 1. Otherwise, it means the square root is within the range [mid, r], so we set l = mid.

After the search ends, we return l.

The time complexity is O(log x), and the space complexity is O(1).

Python 3

```python
class Solution:
    def mySqrt(self, x: int) -> int:
        l, r = 0, x
        while l < r:
            mid = (l + r + 1) >> 1
            if mid > x // mid:
                r = mid - 1
            else:
                l = mid
        return l
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The k-th integer root (round down) instead of the
# square root
# 2. The square root with a given number of decimal places


# ============================================================
# FOLLOW-UP 1: The k-th integer root (round down) instead
# of the square root
# ============================================================

# ---- follow-up 1: function ----
# Answer: Binary search the answer: the largest r with r
# to the power k at most x; the test is monotone.
# Time: O(k log x).  Space: O(1).
def integer_kth_root(x, k):
    low, high = 0, x
    while low < high:
        mid = (low + high + 1) // 2
        if mid ** k <= x:
            low = mid
        else:
            high = mid - 1
    return low


# ============================================================
# FOLLOW-UP 2: The square root with a given number of
# decimal places
# ============================================================

# ---- follow-up 2: function ----
# Answer: Scale first: the root of x with d decimals is
# the integer root of x times 100...0 (10 to the 2d)
# divided by 10 to the d. Integers keep the answer exact.
# Time: O(log x + p), p = decimal places.  Space: O(1).
def sqrt_with_decimals(x, places):
    scaled = x * 10 ** (2 * places)
    low, high = 0, scaled
    while low < high:
        mid = (low + high + 1) // 2
        if mid * mid <= scaled:
            low = mid
        else:
            high = mid - 1
    whole = low // 10 ** places
    fraction = low % 10 ** places
    return str(whole) + "." + str(fraction).rjust(places, "0")
```

PATTERN 4 · BINARY SEARCH · PROBLEM 4 OF 20

34. Find First and Last Position of Element in Sorted Array

Medium · Array, Binary Search

leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array

★ **TOP 7 PROBLEM** in Binary Search: master this one first

PROBLEM

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given `target` value.

If `target` is not found in the array, return `[-1, -1]`.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [5,7,7,8,8,10], target = 8
Output: [3,4]
```

Example 2:

```
Input: nums = [5,7,7,8,8,10], target = 6
Output: [-1,-1]
```

Example 3:

```
Input: nums = [], target = 0
Output: [-1,-1]
```

Constraints:

- `0 <= nums.length <=` 10^5
- -10^9 `<= nums[i] <=` 10^9
- `nums` is a non-decreasing array.
- -10^9 `<= target <=` 10^9

SOLUTION (PYTHON 3)

Solution 1: Binary Search

How to think about it

The first idea is a linear scan that records the first and last hits. Correct, but $n \le 10^5$ and the problem requires O(log n). Expanding outward from one match is still O(n) in the worst case. The array is non-decreasing, so the two endpoints are separate binary-search questions; we need not find both in one pass. The left endpoint is the first index ≥ target; the right endpoint is the first index ≥ target+1, minus one. Two lower-bound queries suffice; if they land on the same index, target is absent.

We can perform two binary searches to find the left boundary and the right boundary.

The time complexity is O(log n), where n is the length of the array nums. The space complexity is O(1).

Python 3

```python
class Solution:
    def searchRange(self, nums: List[int], target: int) -> List[int]:
        l = bisect_left(nums, target)
        r = bisect_left(nums, target + 1)
        return [-1, -1] if l == r else [l, r - 1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. How many elements lie in a value range [low_value,
# high_value]?
# 2. Answer many queries (value -> first and last
# position) fast


# ============================================================
# FOLLOW-UP 1: How many elements lie in a value range
# [low_value, high_value]?
# ============================================================

# ---- follow-up 1: helper functions ----
def first_not_less(nums, target):
    low, high = 0, len(nums)
    while low < high:
        mid = (low + high) // 2
        if nums[mid] < target:
            low = mid + 1
        else:
            high = mid
    return low

# ---- follow-up 1: function ----
# Answer: Two lower-bound searches: the first position
# with a value >= low_value and the first position with a
# value > high_value; the difference is the count.
# Time: O(log n).  Space: O(1).
def count_in_range(nums, low_value, high_value):
    return first_not_less(nums, high_value + 1) - first_not_less(nums, low_value)


# ============================================================
# FOLLOW-UP 2: Answer many queries (value -> first and
# last position) fast
# ============================================================

# ---- follow-up 2: function ----
# Answer: Binary search per query is O(log n); if the same
# array answers thousands of queries, one pass can store
# the first and last index of every value in a dictionary
# for O(1) answers.
# Time: O(n) to build, O(1) per query.  Space: O(n).
def build_position_table(nums):
    table = {}
    for i in range(len(nums)):
        if nums[i] not in table:
            table[nums[i]] = [i, i]
        else:
            table[nums[i]][1] = i
    return table
```

PATTERN 4 · BINARY SEARCH · PROBLEM 5 OF 20

33. Search in Rotated Sorted Array

Medium · Array, Binary Search

leetcode.com/problems/search-in-rotated-sorted-array

★ **TOP 7 PROBLEM** in Binary Search: master this one first

PROBLEM

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly left rotated** at an unknown index `k` (`1 <= k < nums.length`) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be left rotated by `3` indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return *the index of* `target` *if it is in* `nums`*, or* `-1` *if it is not in* `nums`.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4
```

Example 2:

```
Input: nums = [4,5,6,7,0,1,2], target = 3
Output: -1
```

Example 3:

```
Input: nums = [1], target = 0
Output: -1
```

Constraints:

- `1 <= nums.length <= 5000`
- `-10^4 <= nums[i] <= 10^4`
- All values of `nums` are **unique**.
- `nums` is an ascending array that is possibly rotated.
- `-10^4 <= target <= 10^4`

SOLUTION (PYTHON 3)

Solution 1: Binary Search

How to think about it

The first idea is a left-to-right scan. Correct, but n ≤ 5000 and the problem requires O(log n). After the rotation the array is no longer fully sorted, so ordinary binary search cannot tell which half holds target. The key is that any cut leaves at least one half monotonic. Comparing nums[0] with nums[mid] tells us whether the left half is sorted; then we check whether target lies in that sorted range and discard the other half. So we can still binary-search: the sorted half decides keep-or-drop, and the unsorted half is cut again in the next round.

We use binary search to divide the array into two parts, [left,.. mid] and [mid + 1,.. right]. At this point, we can find that one part must be sorted.

Therefore, we can determine whether target is in this part based on the sorted part:

- If the elements in the range [0,.. mid] form a sorted array:
  - If nums[0] ≤ target ≤ nums[mid], then our search range can be narrowed down to [left,.. mid];
  - Otherwise, search in [mid + 1,.. right];
- If the elements in the range [mid + 1, n - 1] form a sorted array:
  - If nums[mid] < target ≤ nums[n - 1], then our search range can be narrowed down to [mid + 1,.. right];
  - Otherwise, search in [left,.. mid].

The termination condition for binary search is left ≥ right. If at the end we find that nums[left] is not equal to target, it means that there is no element with a value of target in the array, and we return -1. Otherwise, we return the index left.

The time complexity is O(log n), where n is the length of the array nums. The space complexity is O(1).

Python 3

```python
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        n = len(nums)
        left, right = 0, n - 1
        while left < right:
            mid = (left + right) >> 1
            if nums[0] <= nums[mid]:
                if nums[0] <= target <= nums[mid]:
                    right = mid
                else:
                    left = mid + 1
            else:
                if nums[mid] < target <= nums[n - 1]:
                    left = mid + 1
                else:
                    right = mid
        return left if nums[left] == target else -1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. How many times was the sorted array rotated?
# 2. Search by pretending the array is sorted again


# ============================================================
# FOLLOW-UP 1: How many times was the sorted array
# rotated?
# ============================================================

# ---- follow-up 1: function ----
# Answer: The rotation count is the index of the smallest
# element. Compare the middle with the last element: if
# the middle is bigger, the smallest is to its right.
# Time: O(log n).  Space: O(1).
def rotation_count(nums):
    low, high = 0, len(nums) - 1
    while low < high:
        mid = (low + high) // 2
        if nums[mid] > nums[high]:
            low = mid + 1
        else:
            high = mid
    return low


# ============================================================
# FOLLOW-UP 2: Search by pretending the array is sorted
# again
# ============================================================
```

```python
# ---- follow-up 2: helper functions ----
def rotation_index(nums):
    low, high = 0, len(nums) - 1
    while low < high:
        mid = (low + high) // 2
        if nums[mid] > nums[high]:
            low = mid + 1
        else:
            high = mid
    return low

# ---- follow-up 2: function ----
# Answer: With the rotation count r known, position i of
# the sorted order lives at index (i + r) mod n, so an
# ordinary binary search works through that mapping.
# Time: O(log n).  Space: O(1).
def search_rotated_with_offset(nums, target):
    n = len(nums)
    if n == 0:
        return -1
    offset = rotation_index(nums)
    low, high = 0, n - 1
    while low <= high:
        mid = (low + high) // 2
        real = (mid + offset) % n
        if nums[real] == target:
            return real
        if nums[real] < target:
            low = mid + 1
```

```python
        else:
            high = mid - 1
    return -1
```

PATTERN 4 · BINARY SEARCH · PROBLEM 6 OF 20

81. Search in Rotated Sorted Array II

Medium · Array, Binary Search

leetcode.com/problems/search-in-rotated-sorted-array-ii

PROBLEM

There is an integer array `nums` sorted in non-decreasing order (not necessarily with **distinct** values).

Before being passed to your function, `nums` is **rotated** at an unknown pivot index `k` (`0 <= k < nums.length`) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,4,4,5,6,6,7]` might be rotated at pivot index `5` and become `[4,5,6,6,7,0,1,2,4,4]`.

Given the array `nums` **after** the rotation and an integer `target`, return `true` *if* `target` *is in* `nums`*, or* `false` *if it is not in* `nums`.

You must decrease the overall operation steps as much as possible.

Example 1:

```
Input: nums = [2,5,6,0,0,1,2], target = 0
Output: true
```

Example 2:

```
Input: nums = [2,5,6,0,0,1,2], target = 3
Output: false
```

Constraints:

- `1 <= nums.length <= 5000`
- -10^4 `<= nums[i] <=` 10^4
- `nums` is guaranteed to be rotated at some pivot.
- -10^4 `<= target <=` 10^4

Follow up: This problem is similar to Search in Rotated Sorted Array, but `nums` may contain **duplicates**. Would this affect the runtime complexity? How and why?

SOLUTION (PYTHON 3)

Solution 1: Binary Search

How to think about it

The first idea is a linear scan: compare each element with target. Correct, but the problem asks to minimize operations, and the array is two sorted halves after rotation. n ≤ 5000 would pass a scan, yet that ignores the order. The bottleneck is not using “at least one half is sorted.” Ordinary binary search needs a fully sorted range; comparing nums[mid] with nums[r] tells us which side is ordered, then whether target lies in that half. Unlike Search in Rotated Sorted Array, duplicates are allowed. When nums[mid] = nums[r] we cannot tell which side is ordered, so we only decrement r. If the whole range is equal this degrades to O(n) — the unavoidable cost of duplicates.

We define the left boundary of the binary search as l = 0 and the right boundary as r = n - 1, where n is the length of the array.

Each time during the binary search, we get the current midpoint mid = (l + r) / 2.

- If nums[mid] > nums[r], it means [l, mid] is ordered. If nums[l] ≤ target ≤ nums[mid], it means target is in [l, mid]. Otherwise, target is in [mid + 1, r].

- If nums[mid] < nums[r], it means [mid + 1, r] is ordered. If nums[mid] < target ≤ nums[r], it means target is in [mid + 1, r]. Otherwise, target is in [l, mid].
- If nums[mid] = nums[r], it means the elements nums[mid] and nums[r] are equal. In this case, we cannot determine which interval target is in, so we can only decrease r by 1.

After the binary search, if nums[l] = target, it means the target value target exists in the array. Otherwise, it does not exist.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Python 3

```
class Solution:
    def search(self, nums: List[int], target: int) -> bool:
        n = len(nums)
        l, r = 0, n - 1
        while l < r:
            mid = (l + r) >> 1
            if nums[mid] > nums[r]:
                if nums[l] <= target <= nums[mid]:
                    r = mid
                else:
                    l = mid + 1
            elif nums[mid] < nums[r]:
                if nums[mid] < target <= nums[r]:
                    l = mid + 1
                else:
                    r = mid
            else:
                r -= 1
        return nums[l] == target
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Find the minimum when duplicates are allowed
# 2. Return the index of the target, not just whether it
# exists


# ============================================================
# FOLLOW-UP 1: Find the minimum when duplicates are
# allowed
# ============================================================

# ---- follow-up 1: function ----
# Answer: When the middle equals the last element you
# cannot tell which side the minimum is on, so shrink the
# range by one from the right; the worst case becomes
# O(n).
# Time: O(log n) on average, O(n) in the worst case (many
# duplicates).  Space: O(1).
def find_min_with_duplicates(nums):
    low, high = 0, len(nums) - 1
    while low < high:
        mid = (low + high) // 2
        if nums[mid] > nums[high]:
            low = mid + 1
        elif nums[mid] < nums[high]:
            high = mid
        else:
            high -= 1
    return nums[low]


# ============================================================
# FOLLOW-UP 2: Return the index of the target, not just
# whether it exists
```

```
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same search; when the left, middle and right
# values are equal, move both ends inwards.
# Time: O(log n) on average, O(n) in the worst case.
# Space: O(1).
def search_index_with_duplicates(nums, target):
    low, high = 0, len(nums) - 1
    while low <= high:
        mid = (low + high) // 2
        if nums[mid] == target:
            return mid
        if nums[low] == nums[mid] == nums[high]:
            low += 1
            high -= 1
        elif nums[low] <= nums[mid]:                    # the left half is sorted
            if nums[low] <= target < nums[mid]:
                high = mid - 1
            else:
                low = mid + 1
        else:                                           # the right half is sorted
            if nums[mid] < target <= nums[high]:
                low = mid + 1
            else:
                high = mid - 1
```

```
    return -1
```

PATTERN 4 · BINARY SEARCH · PROBLEM 7 OF 20

153. Find Minimum in Rotated Sorted Array

Medium · Array, Binary Search

leetcode.com/problems/find-minimum-in-rotated-sorted-array

★ **TOP 7 PROBLEM** in Binary Search: master this one first

PROBLEM

Suppose an array of length `n` sorted in ascending order is **rotated** between `1` and `n` times. For example, the array `nums = [0,1,2,4,5,6,7]` might become:

- `[4,5,6,7,0,1,2]` if it was rotated `4` times.
- `[0,1,2,4,5,6,7]` if it was rotated `7` times.

Notice that **rotating** an array `[a[0], a[1], a[2], ..., a[n-1]]` 1 time results in the array `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]`.

Given the sorted rotated array `nums` of **unique** elements, return *the minimum element of this array*.

You must write an algorithm that runs in `O(log n) time`.

Example 1:

```
Input: nums = [3,4,5,1,2]
Output: 1
Explanation: The original array was [1,2,3,4,5] rotated 3 times.
```

Example 2:

```
Input: nums = [4,5,6,7,0,1,2]
Output: 0
Explanation: The original array was [0,1,2,4,5,6,7] and it was rotated 4 times.
```

Example 3:

```
Input: nums = [11,13,15,17]
Output: 11
Explanation: The original array was [11,13,15,17] and it was rotated 4 times.
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5000`
- `-5000 <= nums[i] <= 5000`
- All the integers of `nums` are **unique**.
- `nums` is sorted and rotated between `1` and `n` times.

SOLUTION (PYTHON 3)

Solution 1: Binary Search

How to think about it

A rotated sorted array with distinct values; a linear scan is O(n) and n≤ 5000 is fine, but order allows log n. The minimum splits the array into two increasing runs. Compare the mid to the last value: larger means the min is on the right, otherwise on the left (including mid). Shrink to one index.

We can use binary search to solve this problem.

First, we define two pointers l and r pointing to the start and end of the array respectively. Then we enter a loop until l is no longer less than r.

In each iteration, we calculate the middle position mid and compare nums[mid] with nums[n-1]. If nums[mid] is greater than nums[n-1], the minimum value is to the right of mid, so we update l to mid + 1. Otherwise, the minimum value is at mid or to its left, so we update r to mid. When the loop ends, pointer l will point to the minimum value, and we return nums[l].

The time complexity is O(log n), where n is the length of array nums. The space complexity is O(1).

Python 3

```python
class Solution:
    def findMin(self, nums: List[int]) -> int:
        l, r = 0, len(nums) - 1
        while l < r:
            mid = (l + r) >> 1
            if nums[mid] > nums[-1]:
                l = mid + 1
            else:
                r = mid
        return nums[l]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Find the maximum instead
# 2. The k-th smallest element of the rotated array


# ============================================================
# FOLLOW-UP 1: Find the maximum instead
# ============================================================

# ---- follow-up 1: function ----
# Answer: The maximum sits just before the minimum (or at
# the end when the array is not rotated).
# Time: O(log n).  Space: O(1).
def find_max_rotated(nums):
    low, high = 0, len(nums) - 1
    while low < high:
        mid = (low + high) // 2
        if nums[mid] > nums[high]:
            low = mid + 1
        else:
            high = mid
    return nums[(low - 1) % len(nums)]


# ============================================================
# FOLLOW-UP 2: The k-th smallest element of the rotated
# array
# ============================================================

# ---- follow-up 2: function ----
# Answer: Once the position of the minimum is known, the
# sorted order starts there and wraps around: the k-th
# smallest is at (minimum position + k - 1) mod n.
# Time: O(log n).  Space: O(1).
def kth_smallest_in_rotated(nums, k):
    low, high = 0, len(nums) - 1
    while low < high:
        mid = (low + high) // 2
        if nums[mid] > nums[high]:
            low = mid + 1
        else:
            high = mid
    return nums[(low + k - 1) % len(nums)]
```

PATTERN 4 · BINARY SEARCH · PROBLEM 8 OF 20

74. Search a 2D Matrix

Medium · Array, Binary Search, Matrix

leetcode.com/problems/search-a-2d-matrix

PROBLEM

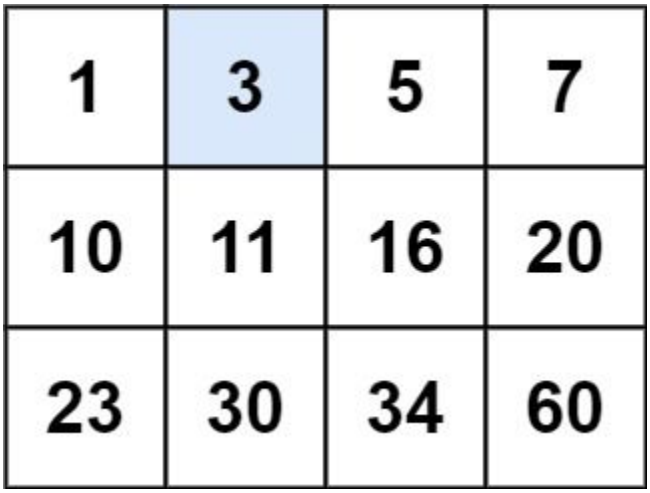
You are given an `m x n` integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `true` *if* `target` *is in* `matrix` *or* `false` *otherwise*.

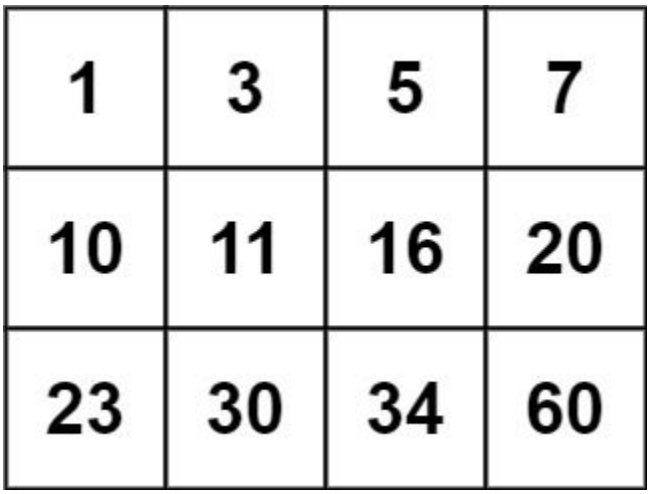
You must write a solution in `O(log(m * n))` time complexity.

Example 1:




```
Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: true
```

Example 2:




```
Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13
Output: false
```

Constraints:

- `m == matrix.length`

- `n == matrix[i].length`
- `1 <= m, n <= 100`
- -10^4 `<= matrix[i][j], target <=` 10^4

SOLUTIONS (PYTHON 3)

Solution 1: Binary Search

> **How to think about it**
>
> A full scan is O(mn). m,n ≤ 100 would pass, but the problem asks for O(log(mn)). Two dimensions hide a single binary search. Each row increases, and the last of row i is less than the first of row i+1, so concatenating rows is a sorted array of length mn. Logical index mid maps to matrix[floor(mid/n)][mid mod n], so we binary-search [0,mn-1] without materializing the flatten.

We can logically unfold the two-dimensional matrix and then perform binary search.

The time complexity is O(log(m × n)), where m and n are the number of rows and columns of the matrix, respectively. The space complexity is O(1).

Python 3

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        left, right = 0, m * n - 1
        while left < right:
            mid = (left + right) >> 1
            x, y = divmod(mid, n)
            if matrix[x][y] >= target:
                right = mid
            else:
                left = mid + 1
        return matrix[left // n][left % n] == target
```

Solution 2: Search from the Bottom Left or Top Right

> **How to think about it**
>
> Method 1 flattens the matrix and binary-searches, which is O(log(mn)), but it maps mid to coordinates and needs the last of one row to be less than the first of the next. Without that index map, row/column monotonicity lets a corner walk drop a whole row or column per comparison. The code is simpler; the cost is O(m+n) time.

Here, we start searching from the bottom left corner and move towards the top right direction. We compare the current element matrix[i][j] with target:

- If matrix[i][j] = target, we have found the target value and return `true`.
- If matrix[i][j] > target, all elements to the right of the current position in this row are greater than target, so we should move the pointer i upwards, i.e., i = i - 1.
- If matrix[i][j] < target, all elements above the current position in this column are less than target, so we should move the pointer j to the right, i.e., j = j + 1.

If we still can't find target after the search, return `false`.

The time complexity is O(m + n), where m and n are the number of rows and columns of the matrix, respectively. The space complexity is O(1).

Python 3

```python
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        i, j = m - 1, 0
        while i >= 0 and j < n:
            if matrix[i][j] == target:
                return True
            if matrix[i][j] > target:
                i -= 1
            else:
                j += 1
        return False
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Rows and columns are each sorted, but a row does not
# start above the end of the previous row (Search a 2D
# Matrix II)
# 2. Count the numbers that are at most x in such a matrix


# ============================================================
# FOLLOW-UP 1: Rows and columns are each sorted, but a row
# does not start above the end of the previous row (Search
# a 2D Matrix II)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Start at the top-right corner: if it is bigger
# than the target, drop its column; if it is smaller, drop
# its row. Each step removes a whole line: O(rows +
# columns).
# Time: O(m + n).  Space: O(1).
def search_matrix_staircase(matrix, target):
    row, col = 0, len(matrix[0]) - 1
    while row < len(matrix) and col >= 0:
        if matrix[row][col] == target:
            return True
        if matrix[row][col] > target:
            col -= 1
        else:
            row += 1
    return False


# ============================================================
# FOLLOW-UP 2: Count the numbers that are at most x in
# such a matrix
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same staircase walk from the bottom-left
# corner: when the cell is at most x, the whole column
# above it also counts.
# Time: O(m + n).  Space: O(1).
def count_at_most(matrix, x):
    row, col = len(matrix) - 1, 0
    count = 0
    while row >= 0 and col < len(matrix[0]):
        if matrix[row][col] <= x:
            count += row + 1
            col += 1
        else:
            row -= 1
```

```
    return count
```

PATTERN 4 · BINARY SEARCH · PROBLEM 9 OF 20

875. Koko Eating Bananas

Medium · Array, Binary Search

leetcode.com/problems/koko-eating-bananas

★ **TOP 7 PROBLEM** in Binary Search: master this one first

PROBLEM

Koko loves to eat bananas. There are `n` piles of bananas, the `i`th pile has `piles[i]` bananas. The guards have gone and will come back in `h` hours.

Koko can decide her bananas-per-hour eating speed of `k`. Each hour, she chooses some pile of bananas and eats `k` bananas from that pile. If the pile has less than `k` bananas, she eats all of them instead and will not eat any more bananas during this hour.

Koko likes to eat slowly but still wants to finish eating all the bananas before the guards return.

Return *the minimum integer* `k` *such that she can eat all the bananas within* `h` *hours*.

Example 1:

```
Input: piles = [3,6,7,11], h = 8
Output: 4
```

Example 2:

```
Input: piles = [30,11,23,4,20], h = 5
Output: 30
```

Example 3:

```
Input: piles = [30,11,23,4,20], h = 6
Output: 23
```

Constraints:

- `1 <= piles.length <= 10`^{4}
- `piles.length <= h <= 10`^{9}
- `1 <= piles[i] <= 10`^{9}

SOLUTION (PYTHON 3)

Solution 1: Binary Search

How to think about it

A larger speed k finishes sooner; we want the smallest k that finishes within h hours. h and pile sizes reach 10^9, so we binary-search the monotone predicate “can finish in h hours”. Search [1,max piles]; the check sums ceil(x/k) over piles. The first feasible speed is the answer.

We notice that if Koko can eat all the bananas at a speed of k within h hours, then she can also eat all the bananas at a speed of k' > k within h hours. This shows monotonicity, so we can use binary search to find the smallest k that satisfies the condition.

We define the left boundary of the binary search as l = 1, and the right boundary as r = max(piles). For each binary search, we take the middle value mid = (l + r)/2, and then calculate the time s required to eat bananas at a speed of mid. If s ≤ h, it means that the speed of mid can meet the condition, and we update the right boundary r to mid; otherwise, we update the left boundary l to mid + 1. Finally, when l = r, we find the smallest k that satisfies the condition.

The time complexity is O(n × log M), where n and M are the length and maximum value of the array piles respectively. The space complexity is O(1).

Python 3

```python
class Solution:
    def minEatingSpeed(self, piles: List[int], h: int) -> int:
        def check(k: int) -> bool:
            return sum((x + k - 1) // k for x in piles) <= h

        return 1 + bisect_left(range(1, max(piles) + 1), True, key=check)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The general recipe: binary search on the answer with
# a yes/no test
# 2. Return the minimum speed AND the hours that speed
# needs


# ============================================================
# FOLLOW-UP 1: The general recipe: binary search on the
# answer with a yes/no test
# ============================================================

# ---- follow-up 1: function ----
# Answer: When 'is this speed enough?' flips from no to
# yes exactly once as the speed grows, search the speed
# range with the test as the comparison: this template
# solves a whole family of problems.
# Time: O(log(high - low)) calls to the test.  Space:
# O(1).
def smallest_value_that_works(low, high, works):
    # works(x) is False for small x and True from some point on
    while low < high:
        mid = (low + high) // 2
        if works(mid):
            high = mid
        else:
            low = mid + 1
    return low


# ============================================================
# FOLLOW-UP 2: Return the minimum speed AND the hours that
# speed needs
# ============================================================

# ---- follow-up 2: function ----
# Answer: Run the search, then count the hours once at the
# answer; if the guard returns before she can even visit
# every pile, there is no answer.
# Time: O(n log max(piles)).  Space: O(1).
def max_pile(piles):
    biggest = 0
    for pile in piles:
        if pile > biggest:
            biggest = pile
    return biggest


def hours_needed(piles, speed):
    hours = 0
    for pile in piles:
        hours += (pile + speed - 1) // speed
    return hours
```

```
def min_speed_and_hours(piles, h):
    if h < len(piles):
        return None
    low, high = 1, max_pile(piles)
    while low < high:
        mid = (low + high) // 2
        if hours_needed(piles, mid) <= h:
            high = mid
        else:
            low = mid + 1
    return low, hours_needed(piles, low)
```

1011. Capacity To Ship Packages Within D Days

Medium · Array, Binary Search

leetcode.com/problems/capacity-to-ship-packages-within-d-days

PROBLEM

A conveyor belt has packages that must be shipped from one port to another within `days` days.

The i^{th} package on the conveyor belt has a weight of `weights[i]`. Each day, we load the ship with packages on the conveyor belt (in the order given by `weights`). We may not load more weight than the maximum weight capacity of the ship.

Return the least weight capacity of the ship that will result in all the packages on the conveyor belt being shipped within `days` days.

Example 1:

```
Input: weights = [1,2,3,4,5,6,7,8,9,10], days = 5
Output: 15
Explanation: A ship capacity of 15 is the minimum to ship all the packages in 5 days like this:
1st day: 1, 2, 3, 4, 5
2nd day: 6, 7
3rd day: 8
4th day: 9
5th day: 10

Note that the cargo must be shipped in the order given, so using a ship of capacity 14 and splitting
the packages into parts like (2, 3, 4, 5), (1, 6, 7), (8), (9), (10) is not allowed.
```

Example 2:

```
Input: weights = [3,2,2,4,1,4], days = 3
Output: 6
Explanation: A ship capacity of 6 is the minimum to ship all the packages in 3 days like this:
1st day: 3, 2
2nd day: 2, 4
3rd day: 1, 4
```

Example 3:

```
Input: weights = [1,2,3,1,1], days = 4
Output: 3
Explanation:
1st day: 1
2nd day: 2
3rd day: 3
4th day: 1, 1
```

Constraints:

- `1 <= days <= weights.length <= 5 * 10^4`
- `1 <= weights[i] <= 500`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Capacity is at least the heaviest package and at most the total weight. Testing every value in that range is too slow: $n \le 5 \times 10^4$ and the sum can be in the millions. Larger capacity never increases the number of days, so feasibility is monotonic and the least valid capacity can be found by binary search. For a candidate x we pack left to right and start a new day when x is exceeded. We bisect the first true value of this check on $[\max w_i, \sum w_i]$.

Python 3

```
class Solution:
    def shipWithinDays(self, weights: List[int], days: int) -> int:
        def check(mx):
            ws, cnt = 0, 1
            for w in weights:
                ws += w
                if ws > mx:
                    cnt += 1
                    ws = w
            return cnt <= days

        left, right = max(weights), sum(weights) + 1
        return left + bisect_left(range(left, right), True, key=check)
```

Complexity. Time: O(n * log(sum(weights))). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the actual plan: which packages go on which
# day
# 2. The packages do not have to stay in order


# ============================================================
# FOLLOW-UP 1: Return the actual plan: which packages go
# on which day
# ============================================================

# ---- follow-up 1: helper functions ----
def max_weight(weights):
    heaviest = 0
    for w in weights:
        if w > heaviest:
            heaviest = w
    return heaviest


def sum_weights(weights):
    total = 0
    for w in weights:
        total += w
    return total

# ---- follow-up 1: function ----
# Answer: Find the smallest capacity, then fill the days
# greedily in order with that capacity.
# Time: O(n log S), S = total weight.  Space: O(n) for the
# plan.
def days_needed(weights, capacity):
    days = 1
    load = 0
    for w in weights:
```

```
        if load + w > capacity:
            days += 1
            load = 0
        load += w
    return days


def shipping_plan(weights, days):
    low, high = max_weight(weights), sum_weights(weights)
    while low < high:
        mid = (low + high) // 2
        if days_needed(weights, mid) <= days:
            high = mid
        else:
            low = mid + 1
    plan = [[]]
    load = 0
    for w in weights:
        if load + w > low:
            plan.append([])
            load = 0
        plan[-1].append(w)
        load += w
    return low, plan
```

```
# ============================================================
# FOLLOW-UP 2: The packages do not have to stay in order
# ============================================================

# ---- follow-up 2: function ----
# Answer: Then the best possible capacity is limited only
# by the total weight and the heaviest package: at least
# the ceiling of total / days, and at least the heaviest
# weight (a lower bound; reaching it exactly is a bin-
# packing problem).
# Time: O(n).  Space: O(1).
def capacity_lower_bound(weights, days):
    total = sum_weights(weights)
    by_total = (total + days - 1) // days
    heaviest = max_weight(weights)
    if heaviest > by_total:
        return heaviest
    return by_total
```

PATTERN 4 · BINARY SEARCH · PROBLEM 11 OF 20

162. Find Peak Element

Medium · Array, Binary Search

leetcode.com/problems/find-peak-element

★ **TOP 7 PROBLEM** in Binary Search: master this one first

PROBLEM

A peak element is an element that is strictly greater than its neighbors.

Given a **0-indexed** integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to **any of the peaks**.

You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in `O(log n)` time.

Example 1:

```
Input: nums = [1,2,3,1]
Output: 2
Explanation: 3 is a peak element and your function should return the index number 2.
```

Example 2:

```
Input: nums = [1,2,1,3,5,6,4]
Output: 5
Explanation: Your function can return either index number 1 where the peak element is 2, or index
number 5 where the peak element is 6.
```

Constraints:

- `1 <= nums.length <= 1000`
- -2^{31} `<= nums[i] <=` $2^{31} - 1$
- `nums[i] != nums[i + 1]` for all valid `i`.

SOLUTION (PYTHON 3)

Solution 1: Binary Search

How to think about it

Neighbors differ and the sentinels are -∞, so a peak exists. A linear scan works for n≤ 1000; the problem wants O(log n). Compare mid to its right neighbor: a smaller right side means a peak in the left half (including mid), otherwise in the right half. Shrink until the two ends meet.

We define the left boundary of binary search as left=0 and the right boundary as right=n-1, where n is the length of the array. In each step of binary search, we find the middle element mid of the current interval, and compare the values of mid and its right neighbor mid+1:

- If the value of mid is greater than the value of mid+1, there exists a peak element on the left side, and we update the right boundary right to mid.
- Otherwise, there exists a peak element on the right side, and we update the left boundary left to mid+1.
- Finally, when the left boundary left is equal to the right boundary right, we have found the peak element of the array.

The time complexity is O(log n), where n is the length of the array nums. Each step of binary search can reduce the search interval by half, so the time complexity is O(log n). The space complexity is O(1).

Python 3

```python
class Solution:
    def findPeakElement(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) >> 1
            if nums[mid] > nums[mid + 1]:
                right = mid
            else:
                left = mid + 1
        return left
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return every peak, not just one
# 2. Find a peak in a 2D grid (bigger than its four
# neighbours) in O(rows log columns)


# ========================================================
# FOLLOW-UP 1: Return every peak, not just one
# ========================================================

# ---- follow-up 1: function ----
# Answer: One pass: an element is a peak when it is bigger
# than both neighbours (a missing neighbour counts as
# minus infinity). O(n), because all peaks must be listed.
# Time: O(n).  Space: O(1) extra.
def all_peaks(nums):
    peaks = []
    for i in range(len(nums)):
        left_ok = i == 0 or nums[i] > nums[i - 1]
        right_ok = i == len(nums) - 1 or nums[i] > nums[i + 1]
        if left_ok and right_ok:
            peaks.append(i)
    return peaks


# ========================================================
# FOLLOW-UP 2: Find a peak in a 2D grid (bigger than its
# four neighbours) in O(rows log columns)
# ========================================================

# ---- follow-up 2: function ----
# Answer: Take the middle column and find its largest
# cell; if the cell to its left is bigger, a peak exists
# in the left half, if the right one is bigger, in the
# right half; otherwise the cell is a peak.
# Time: O(rows * log cols).  Space: O(1).
def find_peak_2d(grid):
    rows, cols = len(grid), len(grid[0])
    low, high = 0, cols - 1
    while low <= high:
        mid = (low + high) // 2
        best_row = 0
        for r in range(rows):
            if grid[r][mid] > grid[best_row][mid]:
                best_row = r
        left = grid[best_row][mid - 1] if mid > 0 else -1
        right = grid[best_row][mid + 1] if mid < cols - 1 else -1
        if grid[best_row][mid] > left and grid[best_row][mid] > right:
            return [best_row, mid]
        if left > grid[best_row][mid]:
            high = mid - 1
        else:
```

```
            low = mid + 1
    return [-1, -1]
```

PATTERN 4 · BINARY SEARCH · PROBLEM 12 OF 20

1283. Find the Smallest Divisor Given a Threshold

Medium · Array, Binary Search

leetcode.com/problems/find-the-smallest-divisor-given-a-threshold

PROBLEM

Given an array of integers `nums` and an integer `threshold`, we will choose a positive integer `divisor`, divide all the array by it, and sum the division's result. Find the **smallest** `divisor` such that the result mentioned above is less than or equal to `threshold`.

Each result of the division is rounded to the nearest integer greater than or equal to that element. (For example: `7/3 = 3` and `10/2 = 5`).

The test cases are generated so that there will be an answer.

Example 1:

```
Input: nums = [1,2,5,9], threshold = 6
Output: 5
Explanation: We can get a sum to 17 (1+2+5+9) if the divisor is 1.
If the divisor is 4 we can get a sum of 7 (1+1+2+3) and if the divisor is 5 the sum will be 5
(1+1+1+2).
```

Example 2:

```
Input: nums = [44,22,33,11,1], threshold = 5
Output: 44
```

Constraints:

- `1 <= nums.length <= 5 * 10^4`
- `1 <= nums[i] <= 10^6`
- `nums.length <= threshold <= 10^6`

SOLUTION (PYTHON 3)

Solution 1: Binary Search

How to think about it

A larger divisor only decreases the sum of ceilings, so the predicate is monotone. $n \le 5\times 10^4$ forbids trying every divisor. We binary-search the smallest v in [1,max nums] such that $\sum \text{ceil}(\text{nums}_i/v) \le \text{threshold}$. Each check is one linear scan.

Notice that for number v, if the sum of results of dividing each number in nums by v is less than or equal to threshold, then all values greater than v satisfy the condition. There is a monotonicity, so we can use binary search to find the smallest v that satisfies the condition.

We define the left boundary of the binary search l=1, r=max(nums). Each time we take mid=(l+r)/2, calculate the sum of the results of dividing each number in nums by mid s, if s is less than or equal to threshold, then it means that mid satisfies the condition, we will update r to mid, otherwise we will update l to mid+1.

Finally, return l.

The time complexity is O(n × log M), where n is the length of the array nums and M is the maximum value in the array nums. The space complexity is O(1).

Python 3

```
class Solution:
    def smallestDivisor(self, nums: List[int], threshold: int) -> int:
        def f(v: int) -> bool:
            v += 1
            return sum((x + v - 1) // v for x in nums) <= threshold

        return bisect_left(range(max(nums)), True, key=f) + 1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Divide with rounding DOWN (floor) and keep the sum at
# most the threshold
# 2. The LARGEST divisor for which the rounded-up sum is
# still at least the threshold


# ========================================================
# FOLLOW-UP 1: Divide with rounding DOWN (floor) and keep
# the sum at most the threshold
# ========================================================

# ---- follow-up 1: function ----
# Answer: The sum of floors also falls as the divisor
# grows, so the same search works with a different test.
# Time: O(n log max(nums)).  Space: O(1).
def smallest_divisor_floor(nums, threshold):
    low, high = 1, max(nums) + 1
    while low < high:
        mid = (low + high) // 2
        total = 0
        for x in nums:
            total += x // mid
        if total <= threshold:
            high = mid
        else:
            low = mid + 1
    return low


# ========================================================
# FOLLOW-UP 2: The LARGEST divisor for which the rounded-
# up sum is still at least the threshold
# ========================================================

# ---- follow-up 2: function ----
# Answer: Now the test is true for small divisors and
# false for big ones, so search for the last true value
# instead of the first.
# Time: O(n log max(nums)).  Space: O(1).
def largest_divisor_ceil_at_least(nums, threshold):
    low, high = 1, max(nums)
    best = 0
    while low <= high:
        mid = (low + high) // 2
        total = 0
        for x in nums:
            total += (x + mid - 1) // mid
        if total >= threshold:
            best = mid
            low = mid + 1
        else:
            high = mid - 1
    return best
```

PATTERN 4 · BINARY SEARCH · PROBLEM 13 OF 20

378. Kth Smallest Element in a Sorted Matrix

Medium · Array, Binary Search, Matrix, Sorting, Heap (Priority Queue)

leetcode.com/problems/kth-smallest-element-in-a-sorted-matrix

★ **TOP 7 PROBLEM** in K-way Merge: master this one first

Also appears in pattern 38 K-way Merge

PROBLEM

Given an `n x n matrix` where each of the rows and columns is sorted in ascending order, return *the* k^{th} *smallest element in the matrix*.

Note that it is the k^{th} smallest element **in the sorted order**, not the k^{th} **distinct** element.

You must find a solution with a memory complexity better than $O(n^2)$.

Example 1:

```
Input: matrix = [[1,5,9],[10,11,13],[12,13,15]], k = 8
Output: 13
Explanation: The elements in the matrix are [1,5,9,10,11,12,13,13,15], and the 8th smallest number is 13
```

Example 2:

```
Input: matrix = [[-5]], k = 1
Output: -5
```

Constraints:

- `n == matrix.length == matrix[i].length`
- `1 <= n <= 300`
- -10^9 `<= matrix[i][j] <=` 10^9
- All the rows and columns of `matrix` are **guaranteed** to be sorted in **non-decreasing order**.
- `1 <= k <=` n^2

Follow up:

- Could you solve the problem with a constant memory (i.e., $O(1)$ memory complexity)?
- Could you solve the problem in $O(n)$ time complexity? The solution may be too advanced for an interview but you may find reading this paper fun.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Rows and columns increase; find the k-th smallest. Flattening and sorting is $O(n^2 \log n)$. The answer lies in [matrix[0][0], matrix[n-1][n-1]], so binary-search the value. `check(mid)` counts entries ≤ mid from the bottom-left in O(n) by monotonicity. If the count is ≥ k, shrink the right bound. The smallest feasible mid is the answer.

Python 3

```python
class Solution:
    def kthSmallest(self, matrix: List[List[int]], k: int) -> int:
        def check(matrix, mid, k, n):
            count = 0
            i, j = n - 1, 0
            while i >= 0 and j < n:
                if matrix[i][j] <= mid:
                    count += i + 1
                    j += 1
                else:
                    i -= 1
            return count >= k

        n = len(matrix)
        left, right = matrix[0][0], matrix[n - 1][n - 1]
        while left < right:
            mid = (left + right) >> 1
            if check(matrix, mid, k, n):
                right = mid
            else:
                left = mid + 1
        return left
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Use a heap instead of binary search on values (k-way
# merge)
# 2. The k-th smallest SUM of one number from each of two
# sorted arrays


# ============================================================
# FOLLOW-UP 1: Use a heap instead of binary search on
# values (k-way merge)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Each row is a sorted list: keep a min-heap with
# the first element of each row, pop k times and push the
# next element of the row that was popped. O(k log n).
# Time: O((n + k) log n).  Space: O(n).
import heapq


def kth_smallest_heap(matrix, k):
    heap = []
    for r in range(len(matrix)):
        heapq.heappush(heap, (matrix[r][0], r, 0))
    value = None
    for step in range(k):
        value, r, c = heapq.heappop(heap)
        if c + 1 < len(matrix[r]):
            heapq.heappush(heap, (matrix[r][c + 1], r, c + 1))
    return value


# ============================================================
# FOLLOW-UP 2: The k-th smallest SUM of one number from
# each of two sorted arrays
# ============================================================

# ---- follow-up 2: function ----
# Answer: Start with the pair of smallest numbers in a
# heap; popping a pair (i, j) pushes (i + 1, j) and (i, j
```

```
# + 1), skipping pairs already pushed.
# Time: O(k log k).  Space: O(k).
import heapq


def kth_smallest_pair_sum(a, b, k):
    heap = [(a[0] + b[0], 0, 0)]
    seen = {(0, 0)}
    total = None
    for step in range(k):
        total, i, j = heapq.heappop(heap)
        if i + 1 < len(a) and (i + 1, j) not in seen:
            seen.add((i + 1, j))
            heapq.heappush(heap, (a[i + 1] + b[j], i + 1, j))
        if j + 1 < len(b) and (i, j + 1) not in seen:
            seen.add((i, j + 1))
            heapq.heappush(heap, (a[i] + b[j + 1], i, j + 1))
    return total
```

PATTERN 4 · BINARY SEARCH · PROBLEM 14 OF 20

540. Single Element in a Sorted Array

Medium · Array, Binary Search

leetcode.com/problems/single-element-in-a-sorted-array

PROBLEM

You are given a sorted array consisting of only integers where every element appears exactly twice, except for one element which appears exactly once.

Return *the single element that appears only once*.

Your solution must run in `O(log n)` time and `O(1)` space.

Example 1:

```
Input: nums = [1,1,2,3,3,4,4,8,8]
Output: 2
```

Example 2:

```
Input: nums = [3,3,7,7,10,11,11]
Output: 10
```

Constraints:

- `1 <= nums.length <= 10^5`
- `0 <= nums[i] <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: Binary Search

> **How to think about it**
>
> All elements appear in adjacent pairs except one singleton, and we must find it in O(log n). Pairs occupy even-odd indices. If `mid` still matches `mid xor 1`, the singleton is to the right; otherwise it lies on the left (including `mid`). XOR avoids separate even/odd cases.

The given array nums is sorted, and we need to find the element that appears only once in O(log n) time. Therefore, we consider using binary search to solve this problem.

We define the left boundary of the binary search as l = 0 and the right boundary as r = n - 1, where n is the length of the array.

In each step, we take the middle position mid = (l + r) / 2. If the index mid is even, we should compare nums[mid] with nums[mid + 1]. If the index mid is odd, we should compare nums[mid] with nums[mid - 1]. Therefore, we can uniformly compare nums[mid] with nums[mid xor 1], where xor denotes the XOR operation.

If nums[mid] ≠ nums[mid xor 1], then the answer is in [l, mid], so we set r = mid. If nums[mid] = nums[mid xor 1], then the answer is in [mid + 1, r], so we set l = mid + 1. We continue the binary search until l = r, at which point nums[l] is the element that appears only once.

The time complexity is O(log n), where n is the length of the array nums. The space complexity is O(1).

Python 3

```
class Solution:
    def singleNonDuplicate(self, nums: List[int]) -> int:
        l, r = 0, len(nums) - 1
        while l < r:
            mid = (l + r) >> 1
            if nums[mid] != nums[mid ^ 1]:
                r = mid
            else:
                l = mid + 1
        return nums[l]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The array is NOT sorted
# 2. Sorted, and every other number appears THREE times


# ============================================================
# FOLLOW-UP 1: The array is NOT sorted
# ============================================================

# ---- follow-up 1: function ----
# Answer: Binary search needs the order; without it use
# XOR: equal numbers cancel and the single one is left
# (O(n) time, O(1) space).
# Time: O(n).  Space: O(1).
def single_number_xor(nums):
    result = 0
    for x in nums:
        result ^= x
    return result


# ============================================================
# FOLLOW-UP 2: Sorted, and every other number appears
# THREE times
# ============================================================

# ---- follow-up 2: function ----
# Answer: Groups of three start at positions that are
# multiples of 3 until the single element. Look at the
# group start at the middle: if its first two numbers are
# equal the group is intact, so the single element is
# later; otherwise it is at or before this group.
# Time: O(log n).  Space: O(1).
def single_in_sorted_triples(nums):
    low, high = 0, len(nums) - 1
    while low < high:
        mid = (low + high) // 2
        mid -= mid % 3                              # the start of a group of three
        if mid + 1 < len(nums) and nums[mid] == nums[mid + 1]:
            low = mid + 3
        else:
            high = mid
    return nums[low]
```

PATTERN 4 · BINARY SEARCH · PROBLEM 15 OF 20

981. Time Based Key-Value Store

Medium · Design, Hash Table, String, Binary Search

leetcode.com/problems/time-based-key-value-store

★ **TOP 7 PROBLEM** in Design Problems: master this one first

Also appears in pattern 29 Design Problems

PROBLEM

Design a time-based key-value data structure that can store multiple values for the same key at different time stamps and retrieve the key's value at a certain timestamp.

Implement the `TimeMap` class:

- `TimeMap()` Initializes the object of the data structure.
- `void set(String key, String value, int timestamp)` Stores the key `key` with the value `value` at the given time `timestamp`.
- `String get(String key, int timestamp)` Returns a value such that `set` was called previously, with `timestamp_prev <= timestamp`. If there are multiple such values, it returns the value associated with the largest `timestamp_prev`. If there are no values, it returns `""`.

Example 1:

```
Input
["TimeMap", "set", "get", "get", "set", "get", "get"]
[[], ["foo", "bar", 1], ["foo", 1], ["foo", 3], ["foo", "bar2", 4], ["foo", 4], ["foo", 5]]
Output
[null, null, "bar", "bar", null, "bar2", "bar2"]

Explanation
TimeMap timeMap = new TimeMap();
timeMap.set("foo", "bar", 1); // store the key "foo" and value "bar" along with timestamp = 1.
timeMap.get("foo", 1); // return "bar"
timeMap.get("foo", 3); // return "bar", since there is no value corresponding to foo at timestamp 3
and timestamp 2, then the only value is at timestamp 1 is "bar".
timeMap.set("foo", "bar2", 4); // store the key "foo" and value "bar2" along with timestamp = 4.
timeMap.get("foo", 4); // return "bar2"
timeMap.get("foo", 5); // return "bar2"
```

Constraints:

- `1 <= key.length, value.length <= 100`
- `key` and `value` consist of lowercase English letters and digits.
- `1 <= timestamp <= 10^7`
- All the timestamps `timestamp` of `set` are strictly increasing.
- At most `2 * 10^5` calls will be made to `set` and `get`.

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Ordered Set (or Binary Search)

How to think about it

`set` timestamps are strictly increasing, so each key's history is already sorted. `get` wants the latest value at a time ≤ the query. A linear scan is too slow over 2× 10^5 calls. Store (timestamp,value) lists per key and binary-search the upper bound.

We can use a hash table kvt to record key-value pairs, where the key is the string key and the value is an ordered set. Each element in the set is a tuple (timestamp, value), representing the value value corresponding to the key key at the timestamp timestamp.

When we need to query the value corresponding to the key key at the timestamp timestamp, we can use the ordered set to find the largest timestamp timestamp' such that timestamp' ≤ timestamp, and then return the corresponding value.

In terms of time complexity, for the set operation, since the insertion operation of the hash table has a time complexity of O(1), the time complexity is O(1). For the get operation, since the lookup operation of the hash table has a time complexity of O(1) and the lookup operation of the ordered set has a time complexity of O(log n), the time complexity is O(log n). The space complexity is O(n), where n is the number of set operations.

Python 3

```
class TimeMap:
    def __init__(self):
        self.ktv = defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.ktv[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        if key not in self.ktv:
            return ''
        tv = self.ktv[key]
        i = bisect_right(tv, (timestamp, chr(127)))
        return tv[i - 1][1] if i else ''


# Your TimeMap object will be instantiated and called as such:
# obj = TimeMap()
# obj.set(key,value,timestamp)
# param_2 = obj.get(key,timestamp)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return all values of a key between two timestamps
# 2. Keys can be deleted and values overwritten at an
# existing timestamp


# ============================================================
# FOLLOW-UP 1: Return all values of a key between two
# timestamps
# ============================================================

# ---- follow-up 1: function ----
# Answer: The timestamps of a key are sorted, so two
# binary searches give the slice between them.
# Time: set O(1), get_range O(log n + r), r = values
# returned.  Space: O(n).
import bisect


class TimeMapRange:
    def __init__(self):
        self.times = {}
        self.values = {}

    def set(self, key, value, timestamp):
        if key not in self.times:
            self.times[key] = []
            self.values[key] = []
        self.times[key].append(timestamp)
        self.values[key].append(value)
```

```
    def get_range(self, key, start, end):
        if key not in self.times:
            return []
        low = bisect.bisect_left(self.times[key], start)
        high = bisect.bisect_right(self.times[key], end)
        return self.values[key][low:high]


# ============================================================
# FOLLOW-UP 2: Keys can be deleted and values overwritten
# at an existing timestamp
# ============================================================

# ---- follow-up 2: function ----
# Answer: Deleting a key just forgets its two lists.
# Writing at a timestamp that already exists replaces the
# value found by a binary search instead of appending.
# Time: set O(n) (insert into a list), get O(log n),
# delete O(1).  Space: O(n).
import bisect


class TimeMapEditable:
    def __init__(self):
        self.times = {}
        self.values = {}

    def set(self, key, value, timestamp):
        if key not in self.times:
```

```
            self.times[key] = []
            self.values[key] = []
        position = bisect.bisect_left(self.times[key], timestamp)
        if position < len(self.times[key]) and self.times[key][position] == timestamp:
            self.values[key][position] = value          # overwrite
        else:
            self.times[key].insert(position, timestamp)
            self.values[key].insert(position, value)

    def get(self, key, timestamp):
        if key not in self.times:
            return ""
        position = bisect.bisect_right(self.times[key], timestamp)
        if position == 0:
            return ""
        return self.values[key][position - 1]

    def delete(self, key):
        if key in self.times:
            del self.times[key]
            del self.values[key]
```

1482. Minimum Number of Days to Make m Bouquets

Medium · Array, Binary Search

leetcode.com/problems/minimum-number-of-days-to-make-m-bouquets

PROBLEM

You are given an integer array `bloomDay`, an integer `m` and an integer `k`.

You want to make `m` bouquets. To make a bouquet, you need to use `k` **adjacent flowers** from the garden.

The garden consists of `n` flowers, the i^{th} flower will bloom in the `bloomDay[i]` and then can be used in **exactly one** bouquet.

Return the **minimum** number of days you need to wait to be able to make m bouquets from the garden. If it is impossible to make m bouquets return -1.

Example 1:

```
Input: bloomDay = [1,10,3,10,2], m = 3, k = 1
Output: 3
Explanation: Let us see what happened in the first three days. x means flower bloomed and _ means
flower did not bloom in the garden.
We need 3 bouquets each should contain 1 flower.
After day 1: [x, _, _, _, _] // we can only make one bouquet.
After day 2: [x, _, _, _, x] // we can only make two bouquets.
After day 3: [x, _, x, _, x] // we can make 3 bouquets. The answer is 3.
```

Example 2:

```
Input: bloomDay = [1,10,3,10,2], m = 3, k = 2
Output: -1
Explanation: We need 3 bouquets each has 2 flowers, that means we need 6 flowers. We only have 5
flowers so it is impossible to get the needed bouquets and we return -1.
```

Example 3:

```
Input: bloomDay = [7,7,7,7,12,7,7], m = 2, k = 3
Output: 12
Explanation: We need 2 bouquets each should have 3 flowers.
Here is the garden after the 7 and 12 days:
After day 7: [x, x, x, x, _, x, x]
We can make one bouquet of the first three flowers that bloomed. We cannot make another bouquet from
the last three flowers that bloomed because they are not adjacent.
After day 12: [x, x, x, x, x, x, x]
It is obvious that we can make two bouquets in different ways.
```

Constraints:

- `bloomDay.length == n`
- `1 <= n <= 10^5`
- `1 <= bloomDay[i] <= 10^9`
- `1 <= m <= 10^6`
- `1 <= k <= n`

SOLUTION (PYTHON 3)

Solution 1: Binary Search

> **How to think about it**
>
> More days only make it easier to form m bouquets. n≤ 10^5 and bloom days reach 10^9, so binary-search the day. The check counts adjacent blooms of length k. If no day works, return -1.

According to the problem description, if a day t can satisfy making m bouquets, then for any t' > t, it can also satisfy making m bouquets. Therefore, we can use binary search to find the minimum day that satisfies making m bouquets.

Let mx be the maximum blooming day in the garden. Next, we define the left boundary of the binary search as l = 1 and the right boundary as r = mx + 1.

Then, we perform binary search. For each middle value mid = (l + r)/2, we check if it is possible to make m bouquets. If it is possible, we update the right boundary r to mid; otherwise, we update the left boundary l to mid + 1.

Finally, when l = r, the binary search ends. At this point, if l > mx, it means it is not possible to make m bouquets, and we return -1; otherwise, we return l.

Therefore, the problem is reduced to checking if a day days can make m bouquets.

We can use a function check(days) to determine if it is possible to make m bouquets. Specifically, we traverse each flower in the garden from left to right. If the blooming day of the current flower is less than or equal to days, we add the current flower to the current bouquet; otherwise, we clear the current bouquet. When the number of flowers in the current bouquet equals k, we increment the bouquet count and clear the current bouquet. Finally, we check if the bouquet count is greater than or equal to m. If it is, it means it is possible to make m bouquets; otherwise, it is not possible.

The time complexity is O(n × log M), where n and M are the number of flowers in the garden and the maximum blooming day, respectively. In this problem, M ≤ 10^9. The space complexity is O(1).

Python 3

```python
class Solution:
    def minDays(self, bloomDay: List[int], m: int, k: int) -> int:
        def check(days: int) -> int:
            cnt = cur = 0
            for x in bloomDay:
                cur = cur + 1 if x <= days else 0
                if cur == k:
                    cnt += 1
                    cur = 0
            return cnt >= m

        mx = max(bloomDay)
        l = bisect_left(range(mx + 2), True, key=check)
        return -1 if l > mx else l
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. How many bouquets can be made by a given day?
# 2. The earliest day on which the FIRST bouquet can be
# made


# ========================================================
# FOLLOW-UP 1: How many bouquets can be made by a given
# day?
# ========================================================

# ---- follow-up 1: function ----
# Answer: Walk the garden: a flower that has bloomed
# extends the current run of adjacent flowers; every k
```

```
# adjacent bloomed flowers make one bouquet and the run
# restarts.
# Time: O(n).  Space: O(1).
def bouquets_by_day(bloom_day, k, day):
    bouquets = 0
    run = 0
    for bloom in bloom_day:
        if bloom <= day:
            run += 1
            if run == k:
                bouquets += 1
                run = 0
        else:
            run = 0
    return bouquets


# ============================================================
# FOLLOW-UP 2: The earliest day on which the FIRST bouquet
# can be made
# ============================================================

# ---- follow-up 2: helper functions ----
def bouquets_on_day(bloom_day, k, day):
    bouquets = 0
    run = 0
    for bloom in bloom_day:
        if bloom <= day:
            run += 1
            if run == k:
                bouquets += 1
                run = 0
        else:
            run = 0
    return bouquets


def min_value(values):
    smallest = values[0]
    for v in values:
        if v < smallest:
            smallest = v
    return smallest


def max_value(values):
```

```
    biggest = values[0]
    for v in values:
        if v > biggest:
            biggest = v
    return biggest

# ---- follow-up 2: function ----
# Answer: That is the case m = 1 of the same search: the
# first day with at least one bouquet.
# Time: O(n log(max - min)).  Space: O(1).
def first_bouquet_day(bloom_day, k):
    if len(bloom_day) < k:
        return -1
    low, high = min_value(bloom_day), max_value(bloom_day)
    while low < high:
        mid = (low + high) // 2
        if bouquets_on_day(bloom_day, k, mid) >= 1:
            high = mid
        else:
            low = mid + 1
    return low
```

PATTERN 4 · BINARY SEARCH · PROBLEM 17 OF 20

658. Find K Closest Elements

Medium · Array, Two Pointers, Binary Search, Sorting, Sliding Window, Heap (Priority Queue)

leetcode.com/problems/find-k-closest-elements

Also appears in pattern 38 K-way Merge

PROBLEM

Given a **sorted** integer array `arr`, two integers `k` and `x`, return the `k` closest integers to `x` in the array. The result should also be sorted in ascending order.

An integer `a` is closer to `x` than an integer `b` if:

- `|a - x| < |b - x|`, or
- `|a - x| == |b - x|` and `a < b`

Example 1:

Input: arr = [1,2,3,4,5], k = 4, x = 3

Output: [1,2,3,4]

Example 2:

Input: arr = [1,1,2,3,4,5], k = 4, x = -1

Output: [1,1,2,3]

Constraints:

- `1 <= k <= arr.length`
- `1 <= arr.length <= 10^4`
- `arr` is sorted in **ascending** order.
- `-10^4 <= arr[i], x <= 10^4`

SOLUTIONS (PYTHON 3)

Solution 1: Sort

How to think about it

We need the k closest values to x, reported in sorted order. Sorting by distance then taking k is O(nlog n). Sort by |v-x|, keep k elements, and sort those by value.

Python 3

```python
class Solution:
    def findClosestElements(self, arr: List[int], k: int, x: int) -> List[int]:
        arr.sort(key=lambda v: abs(v - x))
        return sorted(arr[:k])
```

Solution 2: Two Pointers

How to think about it

The answer is a contiguous slice. Shrink the farther endpoint until the window has length k; the slice is already sorted.

The k elements closest to x in a sorted array form one contiguous subarray.

Set pointers l and r at the two ends. Compare x - arr[l] with arr[r - 1] - x and drop the farther end until r - l = k.

The time complexity is O(n).

Python 3

```python
class Solution:
    def findClosestElements(self, arr: List[int], k: int, x: int) -> List[int]:
        l, r = 0, len(arr)
        while r - l > k:
            if x - arr[l] <= arr[r - 1] - x:
                r -= 1
            else:
                l += 1
        return arr[l:r]
```

Solution 3: Binary Search

> **How to think about it**
>
> Two pointers are linear. The best left bound in [0,n-k] is monotone: compare x-arr[mid] with arr[mid+k]-x and binary-search it.

On top of method 2, binary-search the left boundary of the window of length k over [0, n - k]. At mid, move the window left when x - arr[mid] ≤ arr[mid + k] - x, and right otherwise.

The time complexity is O(log n).

Python 3

```python
class Solution:
    def findClosestElements(self, arr: List[int], k: int, x: int) -> List[int]:
        left, right = 0, len(arr) - k
        while left < right:
            mid = (left + right) >> 1
            if x - arr[mid] <= arr[mid + k] - x:
                right = mid
            else:
                left = mid + 1
        return arr[left : left + k]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The array is not sorted
# 2. Grow outwards from the position where x would be
# inserted (two pointers)


# ============================================================
# FOLLOW-UP 1: The array is not sorted
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort by (distance to x, value) and take the
# first k, then return them sorted; or keep the k best in
# a heap while scanning.
# Time: O(n log n).  Space: O(n).
def k_closest_unsorted(nums, k, x):
    pairs = []
    for value in nums:
        distance = value - x
        if distance < 0:
            distance = -distance
        pairs.append((distance, value))
    pairs.sort()
    chosen = []
    for i in range(k):
        chosen.append(pairs[i][1])
    chosen.sort()
```

```
    return chosen


# ============================================================
# FOLLOW-UP 2: Grow outwards from the position where x
# would be inserted (two pointers)
# ============================================================

# ---- follow-up 2: helper functions ----
def insert_position(nums, x):
    low, high = 0, len(nums)
    while low < high:
        mid = (low + high) // 2
        if nums[mid] < x:
            low = mid + 1
        else:
            high = mid
    return low

# ---- follow-up 2: function ----
# Answer: Binary search the insert position, then
# repeatedly take the closer of the two neighbours; ties
# prefer the smaller value.
# Time: O(log n + k).  Space: O(1) extra.
def k_closest_expand(nums, k, x):
    right = insert_position(nums, x)
    left = right - 1
    for step in range(k):
        if left < 0:
            right += 1
        elif right >= len(nums):
            left -= 1
        elif x - nums[left] <= nums[right] - x:
```

```
            left -= 1
        else:
            right += 1
    return nums[left + 1:right]
```

PATTERN 4 · BINARY SEARCH · PROBLEM 18 OF 20

852. Peak Index in a Mountain Array

Medium · Array, Binary Search, Ternary Search

leetcode.com/problems/peak-index-in-a-mountain-array

PROBLEM

You are given an integer **mountain** array `arr` of length `n` where the values increase to a **peak element** and then decrease.

Return the index of the peak element.

Your task is to solve it in `O(log(n))` time complexity.

Example 1:

Input: arr = [0,1,0]

Output: 1

Example 2:

Input: arr = [0,2,1,0]

Output: 1

Example 3:

Input: arr = [0,10,5,2]

Output: 1

Constraints:

- `3 <= arr.length <= 10^5`
- `0 <= arr[i] <= 10^6`
- `arr` is **guaranteed** to be a mountain array.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

The array rises strictly then falls strictly; we need the peak index. A linear scan works, but n≤ 10^5 and the peak test is monotone: left of the peak arr[i]<arr[i+1], right of it the opposite. Binary search [1,n-2]: if arr[mid]>arr[mid+1] the peak is in the left half (including mid), otherwise in the right. The left endpoint is the peak.

Python 3

```python
class Solution:
    def peakIndexInMountainArray(self, arr: List[int]) -> int:
        left, right = 1, len(arr) - 2
        while left < right:
            mid = (left + right) >> 1
            if arr[mid] > arr[mid + 1]:
                right = mid
            else:
                left = mid + 1
        return left
```

Complexity. Time: O(log n). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Search for a value in a mountain array


# ============================================================
# FOLLOW-UP 1: Search for a value in a mountain array
# ============================================================

# ---- follow-up 1: helper functions ----
def find_peak(mountain):
    low, high = 0, len(mountain) - 1
    while low < high:
        mid = (low + high) // 2
        if mountain[mid] < mountain[mid + 1]:
            low = mid + 1
        else:
            high = mid
    return low

# ---- follow-up 1: function ----
# Answer: Find the peak first, then binary search the
# rising part (ascending) and, if needed, the falling part
# (descending).
# Time: O(log n).  Space: O(1).
def search_mountain(mountain, target):
    peak = find_peak(mountain)
    low, high = 0, peak                              # rising: ascending order
    while low <= high:
        mid = (low + high) // 2
        if mountain[mid] == target:
            return mid
        if mountain[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    low, high = peak + 1, len(mountain) - 1          # falling: descending order
    while low <= high:
        mid = (low + high) // 2
        if mountain[mid] == target:
            return mid
        if mountain[mid] > target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

PATTERN 4 · BINARY SEARCH · PROBLEM 19 OF 20

4. Median of Two Sorted Arrays

Hard · Array, Binary Search, Divide and Conquer

leetcode.com/problems/median-of-two-sorted-arrays

★ **TOP 7 PROBLEM** in Binary Search: master this one first

Also appears in pattern 14 Recursion / Divide and Conquer

PROBLEM

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return **the median** of the two sorted arrays.

The overall run time complexity should be `O(log (m+n))`.

Example 1:

```
Input: nums1 = [1,3], nums2 = [2]
Output: 2.00000
Explanation: merged array = [1,2,3] and median is 2.
```

Example 2:

```
Input: nums1 = [1,2], nums2 = [3,4]
Output: 2.50000
Explanation: merged array = [1,2,3,4] and median is (2 + 3) / 2 = 2.5.
```

Constraints:

- `nums1.length == m`
- `nums2.length == n`
- `0 <= m <= 1000`
- `0 <= n <= 1000`
- `1 <= m + n <= 2000`
- -10^6 `<= nums1[i], nums2[i] <=` 10^6

SOLUTION (PYTHON 3)

Solution 1: Divide and Conquer

How to think about it

Merging the two sorted arrays and then reading the median is the direct approach, in O(m+n). m,n ≤ 10^3 would pass, but the problem asks for O(log(m+n)), so a linear scan is out. The bottleneck: the median only cares about the one or two middle positions after a merge. Smaller values need only be counted, not listed. Both arrays are sorted, so comparing each side's floor(k/2)-th element tells us which side's first half cannot contain the k-th smallest, and we can drop that half at once. So we never build a merged array; we search for the k-th remaining element. The median is the average of the floor((m+n+1)/2)-th and the floor((m+n+2)/2)-th, covering odd and even length with the same code. If one side has fewer than floor(k/2) elements, treat it as +∞ and discard from the other side.

The problem requires the time complexity of the algorithm to be O(log (m + n)), so we cannot directly traverse the two arrays, but need to use the binary search method.

If m + n is odd, then the median is the floor((m + n + 1)/2)-th number; if m + n is even, then the median is the average of the floor((m + n + 1)/2)-th and the floor((m + n + 2)/2)-th numbers. In fact, we can unify it as the average of the floor((m + n + 1)/2)-th and the floor((m + n + 2)/2)-th numbers.

Therefore, we can design a function f(i, j, k), which represents the k-th smallest number in the interval [i, m) of array nums1 and the interval [j, n) of array nums2. The median is the average of f(0, 0, floor((m + n + 1)/2)) and f(0, 0, floor((m + n + 2)/2)).

The implementation idea of the function f(i, j, k) is as follows:

- If i ≥ m, it means that the interval [i, m) of array nums1 is empty, so directly return nums2[j + k - 1];
- If j ≥ n, it means that the interval [j, n) of array nums2 is empty, so directly return nums1[i + k - 1];
- If k = 1, it means to find the first number, so just return the minimum of nums1[i] and nums2[j];
- Otherwise, we find the floor(k/2)-th number in the two arrays, denoted as x and y. (Note, if a certain array does not have the floor(k/2)-th number, then we regard the floor(k/2)-th number as +∞.) Compare the size of x and y:
  - If x ≤ y, it means that the floor(k/2)-th number of array nums1 cannot be the k-th smallest number, so we can exclude the interval [i, i + floor(k/2)) of array nums1, and recursively call f(i + floor(k/2), j, k - floor(k/2)).
  - If x > y, it means that the floor(k/2)-th number of array nums2 cannot be the k-th smallest number, so we can exclude the interval [j, j + floor(k/2)) of array nums2, and recursively call f(i, j + floor(k/2), k - floor(k/2)).

The time complexity is O(log(m + n)), and the space complexity is O(log(m + n)). Here, m and n are the lengths of arrays nums1 and nums2 respectively.

Python 3

```python
class Solution:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
        def f(i: int, j: int, k: int) -> int:
            if i >= m:
                return nums2[j + k - 1]
            if j >= n:
                return nums1[i + k - 1]
            if k == 1:
                return min(nums1[i], nums2[j])
            p = k // 2
            x = nums1[i + p - 1] if i + p - 1 < m else inf
            y = nums2[j + p - 1] if j + p - 1 < n else inf
            return f(i + p, j, k - p) if x < y else f(i, j + p, k - p)

        m, n = len(nums1), len(nums2)
        a = f(0, 0, (m + n + 1) // 2)
        b = f(0, 0, (m + n + 2) // 2)
        return (a + b) / 2
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Find the k-th smallest element of two sorted arrays
# 2. The k-th smallest element across K sorted lists


# ============================================================
# FOLLOW-UP 1: Find the k-th smallest element of two
# sorted arrays
# ============================================================

# ---- follow-up 1: function ----
# Answer: Compare the (k/2)-th elements of both arrays:
# the smaller one and everything before it cannot be the
# answer, so throw that part away and look for the (k -
# removed)-th element. The median is a k-th element call
# (or two).
# Time: O(log k) steps but O(n log k) copying because of
# the slices.  Space: O(n).
def kth_of_two(a, b, k):
    # k counts from 1
    if len(a) > len(b):
        a, b = b, a
    if not a:
        return b[k - 1]
    if k == 1:
        return a[0] if a[0] < b[0] else b[0]
```

```
    take_a = k // 2 if k // 2 < len(a) else len(a)
    take_b = k - take_a
    if a[take_a - 1] <= b[take_b - 1]:
        return kth_of_two(a[take_a:], b, k - take_a)
    return kth_of_two(a, b[take_b:], k - take_b)


# ============================================================
# FOLLOW-UP 2: The k-th smallest element across K sorted
# lists
# ============================================================

# ---- follow-up 2: helper functions ----
def min_of_firsts(lists):
    smallest = None
    for numbers in lists:
        if numbers and (smallest is None or numbers[0] < smallest):
            smallest = numbers[0]
    return smallest


def max_of_lasts(lists):
    biggest = None
    for numbers in lists:
        if numbers and (biggest is None or numbers[-1] > biggest):
            biggest = numbers[-1]
    return biggest

# ---- follow-up 2: function ----
# Answer: Binary search the VALUE: for a guess v count how
# many numbers are at most v in every list with a binary
# search each; the answer is the smallest v whose count is
# at least k.
# Time: O(K log n log R), K lists, R = value range.
```

```
# Space: O(1).
import bisect


def kth_across_lists(lists, k):
    low = min_of_firsts(lists)
    high = max_of_lasts(lists)
    while low < high:
        mid = (low + high) // 2
        count = 0
        for numbers in lists:
            count += bisect.bisect_right(numbers, mid)
        if count >= k:
            high = mid
        else:
            low = mid + 1
    return low
```

PATTERN 4 · BINARY SEARCH · PROBLEM 20 OF 20

410. Split Array Largest Sum

Hard · Greedy, Array, Binary Search, Dynamic Programming, Prefix Sum

leetcode.com/problems/split-array-largest-sum

★ **TOP 7 PROBLEM** in Binary Search: master this one first

PROBLEM

Given an integer array `nums` and an integer `k`, split `nums` into `k` non-empty subarrays such that the largest sum of any subarray is **minimized**.

Return *the minimized largest sum of the split*.

A **subarray** is a contiguous part of the array.

Example 1:

```
Input: nums = [7,2,5,10,8], k = 2
Output: 18
Explanation: There are four ways to split nums into two subarrays.
The best way is to split it into [7,2,5] and [10,8], where the largest sum among the two subarrays is
only 18.
```

Example 2:

```
Input: nums = [1,2,3,4,5], k = 2
Output: 9
Explanation: There are four ways to split nums into two subarrays.
The best way is to split it into [1,2,3] and [4,5], where the largest sum among the two subarrays is
only 9.
```

Constraints:

- `1 <= nums.length <= 1000`
- `0 <= nums[i] <= 10^6`
- `1 <= k <= min(50, nums.length)`

SOLUTION (PYTHON 3)

Solution 1: Binary Search

How to think about it

Enumerating every way to place k-1 cuts is too large. A larger cap on a subarray sum only makes a feasible split easier, so feasibility is monotone in the cap. Binary-search the cap mid. A greedy left-to-right accumulation starts a new piece whenever adding the next value would exceed mid, and we ask whether at most k pieces suffice. The search range is [max(nums),∑ nums]. The minimum feasible cap is the answer; combinatorial search becomes a linear check.

We notice that the larger the maximum sum of the subarrays, the fewer the number of subarrays. When there is a maximum sum of the subarrays that meets the condition, then a larger maximum sum of the subarrays will definitely meet the condition. This means that we can perform a binary search for the maximum sum of the subarrays to find the smallest value that meets the condition.

We define the left boundary of the binary search as left = max(nums), and the right boundary as right = sum(nums). Then for each step of the binary search, we take the middle value mid = floor((left + right)/2), and then determine whether there is a way to split the array so that the maximum sum of the subarrays does not exceed mid. If there is, it means that mid might be the smallest value that meets the condition, so we adjust the right boundary to mid. Otherwise, we adjust the left boundary to mid + 1.

How do we determine whether there is a way to split the array so that the maximum sum of the subarrays does not exceed mid? We can use a greedy method, traverse the array from left to right, and add the elements of the array to the subarray one by one. If the current sum of the subarray is greater than mid, then we add the current element to the next subarray. If we can split the array into no more than k subarrays, and the maximum sum of each subarray does not exceed mid, then mid is the smallest value that meets the condition. Otherwise, mid does not meet the condition.

The time complexity is O(n × log m), and the space complexity is O(1). Here, n and m are the length of the array and the sum of all elements in the array, respectively.

Python 3

```python
class Solution:
    def splitArray(self, nums: List[int], k: int) -> int:
        def check(mx):
            s, cnt = inf, 0
            for x in nums:
                s += x
                if s > mx:
                    s = x
                    cnt += 1
            return cnt <= k

        left, right = max(nums), sum(nums)
        return left + bisect_left(range(left, right + 1), True, key=check)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the split itself (the parts), not only the
# smallest largest sum
# 2. Solve it with dynamic programming instead (useful to
# see why the search works)


# ============================================================
# FOLLOW-UP 1: Return the split itself (the parts), not
# only the smallest largest sum
# ============================================================

# ---- follow-up 1: function ----
# Answer: Find the answer with the search, then cut the
# array greedily: close a part just before its sum would
# pass the answer.
# Time: O(n log S), S = total sum.  Space: O(n).
def parts_needed(nums, limit):
    parts = 1
    current = 0
    for x in nums:
        if current + x > limit:
            parts += 1
            current = 0
        current += x
    return parts


def split_array_parts(nums, m):
    low = 0
    high = 0
    for x in nums:
        if x > low:
            low = x
        high += x
    while low < high:
        mid = (low + high) // 2
        if parts_needed(nums, mid) <= m:
            high = mid
```

```
        else:
            low = mid + 1
    parts = [[]]
    current = 0
    for x in nums:
        if current + x > low:
            parts.append([])
            current = 0
        parts[-1].append(x)
        current += x
    return low, parts


# ============================================================
# FOLLOW-UP 2: Solve it with dynamic programming instead
# (useful to see why the search works)
# ============================================================

# ---- follow-up 2: function ----
# Answer: best[i][j] = the smallest possible largest sum
# when the first i numbers are cut into j parts: try every
```

```
# place for the last cut. O(n * n * m), slower, but it
# always gives the exact answer.
# Time: O(n^2 * m).  Space: O(n * m).
def split_array_dp(nums, m):
    n = len(nums)
    prefix = [0]
    for x in nums:
        prefix.append(prefix[-1] + x)
    big = float("inf")
    best = []
    for i in range(n + 1):
        best.append([big] * (m + 1))
    best[0][0] = 0
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            for cut in range(j - 1, i):
                last_part = prefix[i] - prefix[cut]
                worst = best[cut][j - 1]
                if last_part > worst:
                    worst = last_part
                if worst < best[i][j]:
                    best[i][j] = worst
    return best[n][m]
```

PATTERN 5 OF 41

Prefix Sum

20 problems · 3 Easy · 17 Medium · 0 Hard

THE IDEA

Pre-compute running totals once so the sum of any range is one subtraction. Pairs well with a hash map for 'subarray with sum K'.

Signals: many range-sum queries, subarray sum equals K, product of the rest, balance points, 2D region sums.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
prefix = [0]
for x in nums:
    prefix.append(prefix[-1] + x)
# sum of nums[l..r] inclusive
range_sum = prefix[r + 1] - prefix[l]
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Range Sum Query - Immutable	Easy	LC 303	240
2	Find Pivot Index	Easy	LC 724	243
3	Find the Highest Altitude	Easy	LC 1732	246
4	★ **Range Sum Query 2D - Immutable**	Medium	LC 304	248
5	★ **Subarray Sum Equals K** (see page)	Medium	LC 560	40
6	★ **Maximum Size Subarray Sum Equals k — substitute: Contiguous Array**	Medium	LC 525	252
7	★ **Product of Array Except Self**	Medium	LC 238	255
8	★ **Continuous Subarray Sum** (see page)	Medium	LC 523	55
9	Maximum Subarray	Medium	LC 53	258
10	★ **Path Sum III**	Medium	LC 437	262
11	Count Number of Nice Subarrays (see page)	Medium	LC 1248	103
12	Binary Subarrays With Sum (see page)	Medium	LC 930	105
13	Number of Sub-arrays With Odd Sum	Medium	LC 1524	266
14	Minimum Value to Get Positive Step by Step Sum	Medium	LC 1413	268
15	Maximum Sum of Two Non-Overlapping Subarrays	Medium	LC 1031	271
16	Range Sum Query - Mutable	Medium	LC 307	274
17	★ **Make Sum Divisible by P**	Medium	LC 1590	279
18	Matrix Block Sum	Medium	LC 1314	282
19	Count Square Submatrices with All Ones	Medium	LC 1277	285
20	Number of Ways to Split Array	Medium	LC 2270	288

PATTERN 5 · PREFIX SUM · PROBLEM 1 OF 20

303. Range Sum Query - Immutable

Easy · Design, Array, Prefix Sum

leetcode.com/problems/range-sum-query-immutable

PROBLEM

Given an integer array `nums`, handle multiple queries of the following type:

1. Calculate the **sum** of the elements of `nums` between indices `left` and `right` **inclusive** where `left <= right`.

Implement the `NumArray` class:

- `NumArray(int[] nums)` Initializes the object with the integer array `nums`.
- `int sumRange(int left, int right)` Returns the **sum** of the elements of `nums` between indices `left` and `right` **inclusive** (i.e. `nums[left] + nums[left + 1] + ... + nums[right]`).

Example 1:

```
Input
["NumArray", "sumRange", "sumRange", "sumRange"]
[[[-2, 0, 3, -5, 2, -1]], [0, 2], [2, 5], [0, 5]]
Output
[null, 1, -1, -3]

Explanation
NumArray numArray = new NumArray([-2, 0, 3, -5, 2, -1]);
numArray.sumRange(0, 2); // return (-2) + 0 + 3 = 1
numArray.sumRange(2, 5); // return 3 + (-5) + 2 + (-1) = -1
numArray.sumRange(0, 5); // return (-2) + 0 + 3 + (-5) + 2 + (-1) = -3
```

Constraints:

- $1 \le nums.length \le 10^4$
- $-10^5 \le nums[i] \le 10^5$
- $0 \le left \le right < nums.length$
- At most 10^4 calls will be made to `sumRange`.

SOLUTION (PYTHON 3)

Solution 1: Prefix Sum

How to think about it

The array never changes, yet many range sums are requested. Summing [left,right] from scratch is O(n) per query and does not scale. A prefix array s stores the sum of the first i elements, so sum(left,right)=s[right+1]-s[left]. One linear build makes every later query O(1). Because the array is immutable, no update structure is needed.

We create a prefix sum array s of length n + 1, where s[i] represents the prefix sum of the first i elements, that is, s[i] = $\sum_{j=0}^{i-1}$ nums[j]. Therefore, the sum of the elements between the indices [left, right] can be expressed as s[right + 1] - s[left].

The time complexity for initializing the prefix sum array s is O(n), and the time complexity for querying is O(1). The space complexity is O(n).

Python 3

```
class NumArray:
    def __init__(self, nums: List[int]):
        self.s = list(accumulate(nums, initial=0))

    def sumRange(self, left: int, right: int) -> int:
        return self.s[right + 1] - self.s[left]


# Your NumArray object will be instantiated and called as such:
# obj = NumArray(nums)
# param_1 = obj.sumRange(left,right)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Range MINIMUM queries instead of sums
# 2. Range XOR queries


# ============================================================
# FOLLOW-UP 1: Range MINIMUM queries instead of sums
# ============================================================

# ---- follow-up 1: function ----
# Answer: Minimums cannot be subtracted, so use a sparse
# table: table[j][i] is the minimum of the 2^j numbers
# starting at i. A query takes the minimum of two
# overlapping blocks that cover the range.
# Time: build O(n log n), query O(log n).  Space: O(n log
# n).
class RangeMinimum:
    def __init__(self, nums):
        self.table = [list(nums)]
        j = 1
        while (1 << j) <= len(nums):
            previous = self.table[j - 1]
            row = []
            for i in range(len(nums) - (1 << j) + 1):
                a, b = previous[i], previous[i + (1 << (j - 1))]
                row.append(a if a < b else b)
            self.table.append(row)
            j += 1

    def query(self, left, right):
        length = right - left + 1
        j = 0
        while (1 << (j + 1)) <= length:
            j += 1
        a = self.table[j][left]
        b = self.table[j][right - (1 << j) + 1]
        return a if a < b else b


# ============================================================
# FOLLOW-UP 2: Range XOR queries
# ============================================================

# ---- follow-up 2: function ----
# Answer: XOR is its own inverse, so a prefix XOR array
# works exactly like prefix sums: the XOR of nums[l..r] is
# prefix[r + 1] ^ prefix[l].
# Time: build O(n), query O(1).  Space: O(n).
class RangeXor:
    def __init__(self, nums):
        self.prefix = [0]
```

```
        for x in nums:
            self.prefix.append(self.prefix[-1] ^ x)

    def query(self, left, right):
        return self.prefix[right + 1] ^ self.prefix[left]
```

PATTERN 5 · PREFIX SUM · PROBLEM 2 OF 20

724. Find Pivot Index

Easy · Array, Prefix Sum

leetcode.com/problems/find-pivot-index

PROBLEM

Given an array of integers `nums`, calculate the **pivot index** of this array.

The **pivot index** is the index where the sum of all the numbers **strictly** to the left of the index is equal to the sum of all the numbers **strictly** to the index's right.

If the index is on the left edge of the array, then the left sum is `0` because there are no elements to the left. This also applies to the right edge of the array.

Return *the **leftmost pivot index***. If no such index exists, return `-1`.

Example 1:

```
Input: nums = [1,7,3,6,5,6]
Output: 3
Explanation:
The pivot index is 3.
Left sum = nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11
Right sum = nums[4] + nums[5] = 5 + 6 = 11
```

Example 2:

```
Input: nums = [1,2,3]
Output: -1
Explanation:
There is no index that satisfies the conditions in the problem statement.
```

Example 3:

```
Input: nums = [2,1,-1]
Output: 0
Explanation:
The pivot index is 0.
Left sum = 0 (no elements to the left of index 0)
Right sum = nums[1] + nums[2] = 1 + -1 = 0
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-1000 <= nums[i] <= 1000`

Note: This question is the same as 1991: https://leetcode.com/problems/find-the-middle-index-in-array/

SOLUTION (PYTHON 3)

Solution 1: Prefix Sum

How to think about it

Find an index whose left sum equals its right sum. Recomputing both sides at every i is $O(n^2)$ for $n \le 10^4$. The two sides plus nums[i] make the total S, so 2·left+nums[i]=S. One left-to-right pass suffices. Start right at the total, subtract x before comparing with left, then add x to the left. Extra space O(1).

We define a variable left to represent the sum of elements to the left of index i in the array nums, and a variable right to represent the sum of elements to the right of index i in the array nums. Initially, left = 0, right = $\sum_{i=0}^{n-1}$ nums[i].

We traverse the array nums. For the current number x being traversed, we update right = right - x. At this point, if left = right, it indicates that the current index i is the middle position, and we can return it directly. Otherwise, we update left = left + x and continue to traverse the next number.

If the middle position is not found by the end of the traversal, return -1.

The time complexity is O(n), and the space complexity is O(1). Here, n is the length of the array nums.

Similar Problems:

- 1991. Find the Middle Index in Array
- 2574. Left and Right Sum Differences

Python 3

```
class Solution:
    def pivotIndex(self, nums: List[int]) -> int:
        left, right = 0, sum(nums)
        for i, x in enumerate(nums):
            right -= x
            if left == right:
                return i
            left += x
        return -1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return every pivot index, not only the leftmost
# 2. Cut the array into two non-empty parts with equal
# sums: return where to cut


# ============================================================
# FOLLOW-UP 1: Return every pivot index, not only the
# leftmost
# ============================================================

# ---- follow-up 1: function ----
# Answer: Compute the total once; at every index the left
# sum is known, and the right sum is total - left - the
# number itself.
# Time: O(n).  Space: O(1) extra.
def all_pivot_indexes(nums):
    total = 0
    for x in nums:
        total += x
    pivots = []
    left = 0
    for i in range(len(nums)):
        if left == total - left - nums[i]:
            pivots.append(i)
        left += nums[i]
    return pivots


# ============================================================
# FOLLOW-UP 2: Cut the array into two non-empty parts with
# equal sums: return where to cut
# ============================================================

# ---- follow-up 2: function ----
# Answer: After the first i numbers the left part has sum
# prefix; it equals the right part when prefix * 2 =
# total.
# Time: O(n).  Space: O(1).
def equal_split_position(nums):
```

```
    total = 0
    for x in nums:
        total += x
    left = 0
    for i in range(len(nums) - 1):
        left += nums[i]
        if left * 2 == total:
            return i + 1                        # the first part is nums[:i + 1]
    return -1
```

1732. Find the Highest Altitude

Easy · Array, Prefix Sum

leetcode.com/problems/find-the-highest-altitude

PROBLEM

There is a biker going on a road trip. The road trip consists of `n + 1` points at various altitudes. The biker starts his trip on point `0` with altitude equal `0`.

You are given an integer array `gain` of length `n` where `gain[i]` is the **net gain in altitude** between points `i` and `i + 1` for all (`0 <= i < n`). Return *the **highest altitude** of a point.*

Example 1:

```
Input: gain = [-5,1,5,0,-7]
Output: 1
Explanation: The altitudes are [0,-5,-4,1,1,-6]. The highest is 1.
```

Example 2:

```
Input: gain = [-4,-3,-2,-1,4,3,2]
Output: 0
Explanation: The altitudes are [0,-4,-7,-9,-10,-6,-3,-1]. The highest is 0.
```

Constraints:

- `n == gain.length`
- `1 <= n <= 100`
- `-100 <= gain[i] <= 100`

SOLUTION (PYTHON 3)

Solution 1: Prefix Sum (Difference Array)

> **How to think about it**
>
> gain stores consecutive altitude differences from a start of 0. The highest point is the maximum prefix sum of that difference array. Accumulate gain from 0 and keep the running maximum.

We assume the altitude of each point is h_i. Since gain[i] represents the altitude difference between the ith point and the (i + 1)th point, we have gain[i] = $h_{i+1} - h_i$. Therefore:

$$\sum_{i=0}^{n-1} \text{gain}[i] = h_1 - h_0 + h_2 - h_1 + \ldots + h_n - h_{n-1} = h_n - h_0 = h_n$$

which implies:

$$h_{i+1} = \sum_{j=0}^{i} \text{gain}[j]$$

We can see that the altitude of each point can be calculated through the prefix sum. Therefore, we only need to traverse the array once, find the maximum value of the prefix sum, which is the highest altitude.

In fact, the gain array in the problem is a difference array. The prefix sum of the difference array gives the original altitude array. Then find the maximum value of the original altitude array.

The time complexity is O(n), and the space complexity is O(1). Here, n is the length of the array `gain`.

Python 3

```python
class Solution:
    def largestAltitude(self, gain: List[int]) -> int:
        return max(accumulate(gain, initial=0))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the altitude after every step and the step
# where the highest point is reached
# 2. How many times does the path cross sea level
# (altitude 0)?


# ============================================================
# FOLLOW-UP 1: Return the altitude after every step and
# the step where the highest point is reached
# ============================================================

# ---- follow-up 1: function ----
# Answer: The altitudes are the running sums of the gains
# (starting at 0). Remember the first position of the
# maximum.
# Time: O(n).  Space: O(n).
def altitude_profile(gain):
    altitude = 0
    altitudes = [0]
    best_step = 0
    for i in range(len(gain)):
        altitude += gain[i]
        altitudes.append(altitude)
        if altitude > altitudes[best_step]:
            best_step = i + 1
    return altitudes, best_step


# ============================================================
# FOLLOW-UP 2: How many times does the path cross sea
# level (altitude 0)?
# ============================================================

# ---- follow-up 2: function ----
# Answer: A crossing happens when the altitude changes
# sign between two steps (or touches zero and leaves to
# the other side); count the places where consecutive non-
# zero altitudes have different signs.
# Time: O(n).  Space: O(1).
def sea_level_crossings(gain):
    altitude = 0
    side = 0                                          # -1 below, +1 above, 0 not decided yet
    crossings = 0
    for g in gain:
        altitude += g
        if altitude == 0:
            continue
        new_side = 1 if altitude > 0 else -1
        if side != 0 and new_side != side:
            crossings += 1
        side = new_side
    return crossings
```

PATTERN 5 · PREFIX SUM · PROBLEM 4 OF 20

304. Range Sum Query 2D - Immutable

Medium · Design, Array, Matrix, Prefix Sum

leetcode.com/problems/range-sum-query-2d-immutable

★ **TOP 7 PROBLEM** in Prefix Sum: master this one first

Also appears in pattern 41 Binary Indexed Tree (Fenwick Tree)

PROBLEM

Given a 2D matrix `matrix`, handle multiple queries of the following type:

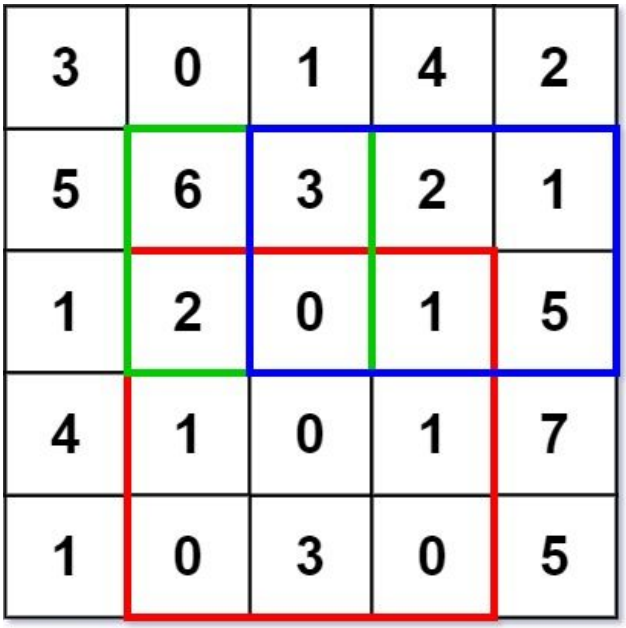
- Calculate the **sum** of the elements of `matrix` inside the rectangle defined by its **upper left corner** `(row1, col1)` and **lower right corner** `(row2, col2)`.

Implement the `NumMatrix` class:

- `NumMatrix(int[][] matrix)` Initializes the object with the integer matrix `matrix`.
- `int sumRegion(int row1, int col1, int row2, int col2)` Returns the **sum** of the elements of `matrix` inside the rectangle defined by its **upper left corner** `(row1, col1)` and **lower right corner** `(row2, col2)`.

You must design an algorithm where `sumRegion` works on `O(1)` time complexity.

Example 1:




```
Input
["NumMatrix", "sumRegion", "sumRegion", "sumRegion"]
[[[[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]]], [2, 1, 4, 3],
[1, 1, 2, 2], [1, 2, 2, 4]]
Output
[null, 8, 11, 12]

Explanation
NumMatrix numMatrix = new NumMatrix([[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1,
7], [1, 0, 3, 0, 5]]);
numMatrix.sumRegion(2, 1, 4, 3); // return 8 (i.e sum of the red rectangle)
numMatrix.sumRegion(1, 1, 2, 2); // return 11 (i.e sum of the green rectangle)
numMatrix.sumRegion(1, 2, 2, 4); // return 12 (i.e sum of the blue rectangle)
```

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 200`
- -10^4 `<= matrix[i][j] <=` 10^4
- `0 <= row1 <= row2 < m`
- `0 <= col1 <= col2 < n`
- At most 10^4 calls will be made to `sumRegion`.

SOLUTION (PYTHON 3)

Solution 1: Two-dimensional Prefix Sum

> **How to think about it**
>
> The matrix is immutable and many sub-rectangle sums are requested. Adding cells on the fly per query does not scale. A 2D prefix s[i+1][j+1] stores the sum of the rectangle from the origin to (i,j). Inclusion-exclusion then yields any target in four lookups: add the bottom-right prefix, subtract the top and left strips, and add back the overlapping corner. After an O(mn) build each query is O(1).

We use s[i + 1][j + 1] to represent the sum of all elements in the upper left part of the ith row and jth column, where indices i and j both start from 0. We can get the following prefix sum formula:

$$s[i + 1][j + 1] = s[i + 1][j] + s[i][j + 1] - s[i][j] + nums[i][j]$$

Then, the sum of the elements of the rectangle with (x_1, y_1) and (x_2, y_2) as the upper left corner and lower right corner respectively is:

$$s[x_2 + 1][y_2 + 1] - s[x_2 + 1][y_1] - s[x_1][y_2 + 1] + s[x_1][y_1]$$

In the initialization method, we preprocess the prefix sum array s, and in the query method, we directly return the result of the above formula.

The time complexity for initializing is O(m × n), and the time complexity for querying is O(1). The space complexity is O(m × n).

Python 3

```
class NumMatrix:
    def __init__(self, matrix: List[List[int]]):
        m, n = len(matrix), len(matrix[0])
        self.s = [[0] * (n + 1) for _ in range(m + 1)]
        for i, row in enumerate(matrix):
            for j, v in enumerate(row):
                self.s[i + 1][j + 1] = (
                    self.s[i][j + 1] + self.s[i + 1][j] - self.s[i][j] + v
                )

    def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:
        return (
            self.s[row2 + 1][col2 + 1]
            - self.s[row2 + 1][col1]
            - self.s[row1][col2 + 1]
            + self.s[row1][col1]
        )


# Your NumMatrix object will be instantiated and called as such:
# obj = NumMatrix(matrix)
# param_1 = obj.sumRegion(row1,col1,row2,col2)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Cells change over time (2D Fenwick tree)
# 2. The k x k square with the largest sum


# ============================================================
# FOLLOW-UP 1: Cells change over time (2D Fenwick tree)
# ============================================================

# ---- follow-up 1: function ----
# Answer: A Fenwick tree in two dimensions: update adds a
# difference to O(log R * log C) cells, and the sum of the
# rectangle from the corner is a double loop that strips
# the lowest bit of both indexes.
# Time: build O(m * n * log m * log n), update and query
# O(log m * log n).  Space: O(m * n).
class Fenwick2D:
    def __init__(self, matrix):
        self.rows = len(matrix)
        self.cols = len(matrix[0])
        self.tree = []
        for r in range(self.rows + 1):
            self.tree.append([0] * (self.cols + 1))
        self.values = []
        for r in range(self.rows):
            self.values.append([0] * self.cols)
        for r in range(self.rows):
            for c in range(self.cols):
                self.update(r, c, matrix[r][c])

    def update(self, row, col, value):
        delta = value - self.values[row][col]
        self.values[row][col] = value
        r = row + 1
        while r <= self.rows:
            c = col + 1
            while c <= self.cols:
                self.tree[r][c] += delta
                c += c & -c
            r += r & -r

    def corner_sum(self, row, col):
        total = 0
        r = row + 1
        while r > 0:
            c = col + 1
            while c > 0:
                total += self.tree[r][c]
                c -= c & -c
            r -= r & -r
        return total

    def sum_region(self, r1, c1, r2, c2):
        return (self.corner_sum(r2, c2) - self.corner_sum(r1 - 1, c2)
                - self.corner_sum(r2, c1 - 1) + self.corner_sum(r1 - 1, c1 - 1))


# ============================================================
# FOLLOW-UP 2: The k x k square with the largest sum
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Build the 2D prefix sums once, then every k x k
# square costs four lookups.
```

```
# Time: O(m * n).  Space: O(m * n).
def best_square_sum(matrix, k):
    rows, cols = len(matrix), len(matrix[0])
    prefix = []
    for r in range(rows + 1):
        prefix.append([0] * (cols + 1))
    for r in range(rows):
        for c in range(cols):
            prefix[r + 1][c + 1] = matrix[r][c] + prefix[r][c + 1] + prefix[r + 1][c] - prefix[r][c]
    best = None
    for r in range(rows - k + 1):
        for c in range(cols - k + 1):
            total = prefix[r + k][c + k] - prefix[r][c + k] - prefix[r + k][c] + prefix[r][c]
            if best is None or total > best:
                best = total
    return best
```

PATTERN 5 · PREFIX SUM · PROBLEM 6 OF 20

525. Contiguous Array

Medium · Array, Hash Table, Prefix Sum

leetcode.com/problems/contiguous-array

★ **TOP 7 PROBLEM** in Prefix Sum: master this one first

PROBLEM

Given a binary array `nums`, return *the maximum length of a contiguous subarray with an equal number of* `0` *and* `1`.

Example 1:

```
Input: nums = [0,1]
Output: 2
Explanation: [0, 1] is the longest contiguous subarray with an equal number of 0 and 1.
```

Example 2:

```
Input: nums = [0,1,0]
Output: 2
Explanation: [0, 1] (or [1, 0]) is a longest contiguous subarray with equal number of 0 and 1.
```

Example 3:

```
Input: nums = [0,1,1,1,1,1,0,0,0]
Output: 6
Explanation: [1,1,1,0,0,0] is the longest contiguous subarray with equal number of 0 and 1.
```

Constraints:

- `1 <= nums.length <= 10^5`
- `nums[i]` is either `0` or `1`.

SOLUTION (PYTHON 3)

Solution 1: Prefix Sum + Hash Table

How to think about it

We want the longest subarray with equally many 0s and 1s. Checking all pairs is $O(n^2)$ and too slow for $n \le 10^5$. Treat 0 as -1; a zero-sum subarray is balanced. Equal prefix sums bound such a segment. Store the first index of each prefix; a later repeat updates the length. Seed 0 at -1 so prefixes from the start are covered.

According to the problem description, we can treat 0s in the array as -1. In this way, when encountering a 0, the prefix sum s will decrease by one, and when encountering a 1, the prefix sum s will increase by one. Therefore, suppose the prefix sum s is equal at indices j and i, where j < i, then the subarray from index j + 1 to i has an equal number of 0s and 1s.

We use a hash table to store all prefix sums and their first occurrence indices. Initially, we map the prefix sum of 0 to -1.

As we iterate through the array, we calculate the prefix sum s. If s is already in the hash table, then we have found a subarray with a sum of 0, and its length is i - d[s], where d[s] is the index where s first appeared in the hash table. If s is not in the hash table, we store s and its index i in the hash table.

The time complexity is O(n), and the space complexity is O(n), where n is the length of the array.

Python 3

```python
class Solution:
    def findMaxLength(self, nums: List[int]) -> int:
        d = {0: -1}
        ans = s = 0
        for i, x in enumerate(nums):
            s += 1 if x else -1
            if s in d:
                ans = max(ans, i - d[s])
            else:
                d[s] = i
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the positions of the longest balanced subarray
# 2. Equal numbers of 0s, 1s and 2s
# 3. Count the balanced subarrays


# ============================================================
# FOLLOW-UP 1: Return the positions of the longest
# balanced subarray
# ============================================================

# ---- follow-up 1: function ----
# Answer: Treat 0 as -1 and keep the first index of every
# running balance; a repeated balance encloses a balanced
# subarray.
# Time: O(n).  Space: O(n).
def longest_balanced_positions(nums):
    first_index = {0: -1}
    balance = 0
    best = (0, -1)
    for i in range(len(nums)):
        balance += 1 if nums[i] == 1 else -1
        if balance in first_index:
            start = first_index[balance] + 1
            if i - start > best[1] - best[0]:
                best = (start, i)
        else:
            first_index[balance] = i
    return best


# ============================================================
# FOLLOW-UP 2: Equal numbers of 0s, 1s and 2s
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep TWO running differences, (ones - zeros,
# twos - ones); the same pair of differences at two
# positions means the part in between is balanced.
# Time: O(n).  Space: O(n).
def longest_equal_zeros_ones_twos(nums):
    first_index = {(0, 0): -1}
    zeros = ones = twos = 0
    best = 0
    for i in range(len(nums)):
        if nums[i] == 0:
            zeros += 1
        elif nums[i] == 1:
            ones += 1
        else:
            twos += 1
```

```
        key = (ones - zeros, twos - ones)
        if key in first_index:
            if i - first_index[key] > best:
                best = i - first_index[key]
        else:
            first_index[key] = i
    return best
```

```
# ============================================================
# FOLLOW-UP 3: Count the balanced subarrays
# ============================================================

# ---- follow-up 3: function ----
# Answer: Count how many times each running balance has
# appeared; every earlier equal balance is the start of
# one balanced subarray.
# Time: O(n).  Space: O(n).
def count_balanced_subarrays(nums):
    seen = {0: 1}
    balance = 0
    count = 0
    for x in nums:
        balance += 1 if x == 1 else -1
        count += seen.get(balance, 0)
        seen[balance] = seen.get(balance, 0) + 1
    return count
```

PATTERN 5 · PREFIX SUM · PROBLEM 7 OF 20

238. Product of Array Except Self

Medium · Array, Prefix Sum

leetcode.com/problems/product-of-array-except-self

★ **TOP 7 PROBLEM** in Prefix Sum: master this one first

PROBLEM

Given an integer array `nums`, return *an array* `answer` *such that* `answer[i]` *is equal to the product of all the elements of* `nums` *except* `nums[i]`.

The product of any prefix or suffix of `nums` is **guaranteed** to fit in a **32-bit** integer.

You must write an algorithm that runs in `O(n)` time and without using the division operation.

Example 1:

```
Input: nums = [1,2,3,4]
Output: [24,12,8,6]
```

Example 2:

```
Input: nums = [-1,1,0,-3,3]
Output: [0,0,9,0,0]
```

Constraints:

- `2 <= nums.length <= 10^5`
- `-30 <= nums[i] <= 30`
- The input is generated such that `answer[i]` is **guaranteed** to fit in a **32-bit** integer.

Follow up: Can you solve the problem in `O(1)` extra space complexity? (The output array **does not** count as extra space for space complexity analysis.)

SOLUTION (PYTHON 3)

Solution 1: Two Passes

How to think about it

The product except nums[i] is the left product times the right product. Two extra arrays for prefixes and suffixes use more than constant space. Write prefix products into the answer left to right, then multiply a running suffix from the right, using only the output array.

We define two variables left and right to represent the product of all elements to the left and right of the current element, respectively. Initially, left = 1 and right = 1. We define an answer array ans of length n.

First, we traverse the array from left to right. For the i-th element, we update ans[i] with left, then multiply left by nums[i].

Next, we traverse the array from right to left. For the i-th element, we update ans[i] to ans[i] × right, then multiply right by nums[i].

After the traversal, we return the answer array ans.

The time complexity is O(n), where n is the length of the array nums. Ignoring the space consumption of the answer array, the space complexity is O(1).

Python 3

```python
class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        ans = [0] * n
        left = right = 1
        for i, x in enumerate(nums):
            ans[i] = left
            left *= x
        for i in range(n - 1, -1, -1):
            ans[i] *= right
            right *= nums[i]
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Division is allowed: handle the zeros
# 2. O(1) extra space (the answer array does not count)


# ============================================================
# FOLLOW-UP 1: Division is allowed: handle the zeros
# ============================================================

# ---- follow-up 1: function ----
# Answer: With no zero, divide the total product by each
# number. With one zero, only the zero's own position gets
# the product of the others. With two or more zeros, every
# answer is 0.
# Time: O(n).  Space: O(1) extra.
def product_except_self_with_division(nums):
    zeros = 0
    product_of_non_zero = 1
    for x in nums:
        if x == 0:
            zeros += 1
        else:
            product_of_non_zero *= x
    answer = []
    for x in nums:
        if zeros > 1:
            answer.append(0)
        elif zeros == 1:
            answer.append(product_of_non_zero if x == 0 else 0)
        else:
            answer.append(product_of_non_zero // x)
    return answer


# ============================================================
# FOLLOW-UP 2: O(1) extra space (the answer array does not
# count)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Write the products of everything to the left
# into the answer, then walk from the right with one
# running variable that holds the product of everything to
# the right.
# Time: O(n).  Space: O(1) extra.
def product_except_self_constant_space(nums):
    n = len(nums)
    answer = [1] * n
    left = 1
    for i in range(n):
```

```
        answer[i] = left
        left *= nums[i]
    right = 1
    for i in range(n - 1, -1, -1):
        answer[i] *= right
        right *= nums[i]
    return answer
```

PATTERN 5 · PREFIX SUM · PROBLEM 9 OF 20

53. Maximum Subarray

Medium · Array, Divide and Conquer, Dynamic Programming

leetcode.com/problems/maximum-subarray

Also appears in pattern 13 Dynamic Programming - 1D, 14 Recursion / Divide and Conquer

PROBLEM

Given an integer array `nums`, find the subarray with the largest sum, and return *its sum*.

Example 1:

```
Input: nums = [-2,1,-3,4,-1,2,1,-5,4]
Output: 6
Explanation: The subarray [4,-1,2,1] has the largest sum 6.
```

Example 2:

```
Input: nums = [1]
Output: 1
Explanation: The subarray [1] has the largest sum 1.
```

Example 3:

```
Input: nums = [5,4,-1,7,8]
Output: 23
Explanation: The subarray [5,4,-1,7,8] has the largest sum 23.
```

Constraints:

- `1 <= nums.length <= 10^5`
- `-10^4 <= nums[i] <= 10^4`

Follow up: If you have figured out the `O(n)` solution, try coding another solution using the **divide and conquer** approach, which is more subtle.

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

The first idea is to enumerate every pair of endpoints and sum, in $O(n^2)$ or $O(n^3)$. $n \le 10^5$ will time out. The waste is recomputing overlapping subarrays. With the right endpoint fixed at i, the best left end either continues the previous segment (if its sum is positive) or starts over at i. So let f[i] be the best sum ending at i. It depends only on f[i-1], so one rolling variable suffices; the answer is the max along the way.

We define f[i] to represent the maximum sum of a contiguous subarray ending at element nums[i]. Initially, f[0] = nums[0]. The final answer we seek is $\max_{0 \le i < n} f[i]$.

Consider f[i] for $i \ge 1$. Its state transition equation is:

$$f[i] = \max(f[i - 1] + nums[i], nums[i])$$

That is:

$$f[i] = \max(f[i - 1], 0) + nums[i]$$

Since f[i] is only related to f[i - 1], we can use a single variable f to maintain the current value of f[i] and perform the state transition. The answer is $\max_{0 \le i < n} f$.

The time complexity is O(n), where n is the length of the array nums. The space complexity is O(1).

Python 3

```python
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        ans = f = nums[0]
        for x in nums[1:]:
            f = max(f, 0) + x
            ans = max(ans, f)
        return ans
```

Solution 2: Divide and Conquer

> **How to think about it**
>
> Solution 1 is already O(n) time and O(1) space. The follow-up asks for divide and conquer, but DP's recurrence is linear in the right endpoint and does not split into disjoint halves. What it lacks is another observation: the best subarray lies entirely on the left, entirely on the right, or crosses the midpoint. Crossing means a max suffix on the left plus a max prefix on the right. Time becomes O(n log n); we use it for the follow-up, not for speed.

Split the array at the midpoint. The answer is the max of the left half, the right half, and the best subarray that crosses the midpoint (max suffix of the left plus max prefix of the right).

The time complexity is O(n log n) and the space complexity is O(log n), where n is the length of the array.

Python 3

```python
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        def crossMaxSub(nums, left, mid, right):
            lsum = rsum = 0
            lmx = rmx = -inf
            for i in range(mid, left - 1, -1):
                lsum += nums[i]
                lmx = max(lmx, lsum)
            for i in range(mid + 1, right + 1):
                rsum += nums[i]
                rmx = max(rmx, rsum)
            return lmx + rmx

        def maxSub(nums, left, right):
            if left == right:
                return nums[left]
            mid = (left + right) >> 1
            lsum = maxSub(nums, left, mid)
            rsum = maxSub(nums, mid + 1, right)
            csum = crossMaxSub(nums, left, mid, right)
            return max(lsum, rsum, csum)

        left, right = 0, len(nums) - 1
        return maxSub(nums, left, right)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subarray (its positions) that gives the
# maximum sum
# 2. The array is circular (the subarray may wrap around
# the end)
# 3. The subarray must contain at least k numbers


# ========================================================
# FOLLOW-UP 1: Return the subarray (its positions) that
# gives the maximum sum
# ========================================================
```

```
# ---- follow-up 1: function ----
# Answer: Kadane's scan: extend the running subarray while
# its sum is not negative, otherwise start again at the
# current number. Remember where the best one started and
# ended.
# Time: O(n).  Space: O(1).
def max_subarray_positions(nums):
    best_sum = nums[0]
    best_range = (0, 0)
    current = nums[0]
    start = 0
    for i in range(1, len(nums)):
        if current < 0:
            current = nums[i]
            start = i
        else:
            current += nums[i]
        if current > best_sum:
            best_sum = current
            best_range = (start, i)
    return best_sum, best_range


# ============================================================
# FOLLOW-UP 2: The array is circular (the subarray may
# wrap around the end)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Either the best subarray does not wrap (ordinary
# Kadane), or it does, and then the part left out in the
# middle is the MINIMUM subarray: total - minimum
# subarray. Take the larger, except when all numbers are
# negative.
# Time: O(n).  Space: O(1).
def max_circular_subarray(nums):
    total = 0
    best_max = nums[0]
    best_min = nums[0]
    current_max = 0
    current_min = 0
    for x in nums:
        total += x
        current_max = x if current_max < 0 else current_max + x
        current_min = x if current_min > 0 else current_min + x
        if current_max > best_max:
            best_max = current_max
        if current_min < best_min:
            best_min = current_min
    if best_max < 0:
        return best_max                          # all negative: wrapping cannot help
    wrapped = total - best_min
    return best_max if best_max > wrapped else wrapped


# ============================================================
# FOLLOW-UP 3: The subarray must contain at least k
# numbers
# ============================================================

# ---- follow-up 3: function ----
# Answer: Use prefix sums: for each end position, subtract
# the smallest prefix among the starts that leave at least
# k numbers.
# Time: O(n).  Space: O(n).
def max_subarray_at_least_k(nums, k):
    prefix = [0]
```

```
    for x in nums:
        prefix.append(prefix[-1] + x)
    best = None
    smallest_prefix = None
    for end in range(k, len(prefix)):
        candidate = prefix[end - k]
        if smallest_prefix is None or candidate < smallest_prefix:
            smallest_prefix = candidate
        total = prefix[end] - smallest_prefix
        if best is None or total > best:
            best = total
    return best
```

PATTERN 5 · PREFIX SUM · PROBLEM 10 OF 20

437. Path Sum III

Medium · Tree, Depth-First Search, Binary Tree

leetcode.com/problems/path-sum-iii

★ **TOP 7 PROBLEM** in Prefix Sum, Tree DFS (Preorder / Inorder / Postorder): master this one first

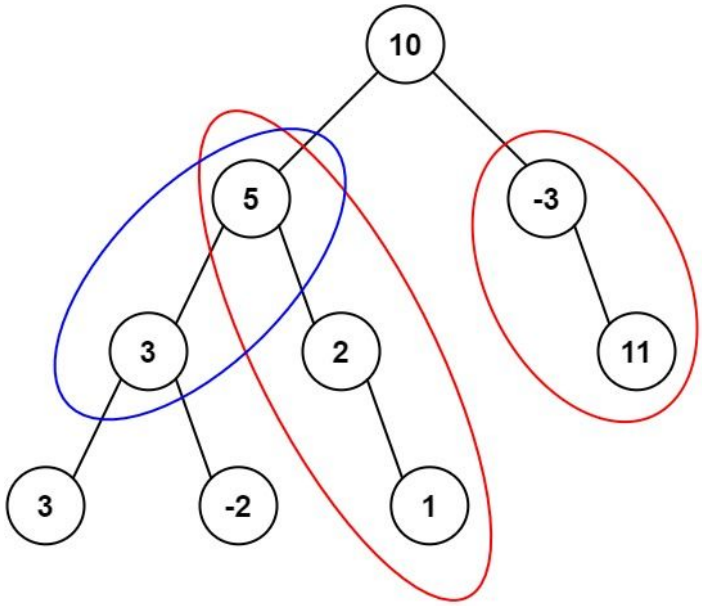
Also appears in pattern 11 Tree DFS (Preorder / Inorder / Postorder)

PROBLEM

Given the `root` of a binary tree and an integer `targetSum`, return *the number of paths where the sum of the values along the path equals* `targetSum`.

The path does not need to start or end at the root or a leaf, but it must go downwards (i.e., traveling only from parent nodes to child nodes).

Example 1:




```
Input: root = [10,5,-3,3,2,null,11,3,-2,null,1], targetSum = 8
Output: 3
Explanation: The paths that sum to 8 are shown.
```

Example 2:

```
Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22
Output: 3
```

Constraints:

- The number of nodes in the tree is in the range `[0, 1000]`.
- -10^9 `<= Node.val <=` 10^9
- `-1000 <= targetSum <= 1000`

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Prefix Sum + Recursion

How to think about it

Paths must go downward. Trying every ancestor then walking down is O(n^2). A path summing to targetSum is a current prefix minus some ancestor prefix. Accumulate the prefix s in preorder and store prefix frequencies on the root-to-node path. cnt[s-targetSum] counts valid paths ending here. Undo s on the way back so another branch does not see it. cnt[0]=1 accounts for a path that starts at the root. Lookup before insert so the current node is not used twice.

We can use the idea of prefix sums to recursively traverse the binary tree while using a hash table cnt to count the occurrences of each prefix sum along the path from the root to the current node.

We design a recursive function dfs(node, s), where node represents the current node being traversed, and s represents the prefix sum along the path from the root to the current node. The return value of the function is the number of paths ending at node or its subtree nodes with a sum equal to targetSum. The final answer is dfs(root, 0).

The recursive process of dfs(node, s) is as follows:

- If the current node node is null, return 0.
- Calculate the prefix sum s along the path from the root to the current node.
- Use cnt[s - targetSum] to represent the number of paths ending at the current node with a sum equal to targetSum. Here, cnt[s - targetSum] is the count of prefix sums equal to s - targetSum in cnt.
- Increment the count of the prefix sum s by 1, i.e., cnt[s] = cnt[s] + 1.
- Recursively traverse the left and right child nodes of the current node by calling dfs(node.left, s) and dfs(node.right, s), and add their return values.
- After the return value is calculated, decrement the count of the prefix sum s by 1, i.e., cnt[s] = cnt[s] - 1.
- Finally, return the result.

The time complexity is O(n), and the space complexity is O(n), where n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def pathSum(self, root: Optional[TreeNode], targetSum: int) -> int:
        def dfs(node, s):
            if node is None:
                return 0
            s += node.val
            ans = cnt[s - targetSum]
            cnt[s] += 1
            ans += dfs(node.left, s)
            ans += dfs(node.right, s)
            cnt[s] -= 1
            return ans

        cnt = Counter({0: 1})
        return dfs(root, 0)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the paths themselves, not just how many
# 2. The tree is very deep: no recursion


# ============================================================
# FOLLOW-UP 1: Return the paths themselves, not just how
# many
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: Start a downward walk from every node, keeping
# the path so far; record the path whenever the running
# sum hits the target (and keep going, since negative
# values may continue the path).
# Time: O(n * h) plus the size of the output.  Space:
# O(h).
def paths_with_sum(root, target):
    found = []

    def walk_down(node, path, total):
        if node is None:
            return
        path.append(node.val)
        total += node.val
        if total == target:
            found.append(list(path))
        walk_down(node.left, path, total)
        walk_down(node.right, path, total)
        path.pop()

    def visit_every_node(node):
        if node is None:
            return
        walk_down(node, [], 0)
        visit_every_node(node.left)
        visit_every_node(node.right)

    visit_every_node(root)
    return found


# ============================================================
# FOLLOW-UP 2: The tree is very deep: no recursion
# ============================================================

# ---- follow-up 2: function ----
# Answer: Use an explicit stack of (node, state) entries:
# when a node is first seen add its prefix sum and push it
# again as an exit marker; when the marker comes back,
# undo the change.
# Time: O(n).  Space: O(n).
def count_paths_iterative(root, target):
```

```
    if root is None:
        return 0
    prefix_count = {0: 1}
    count = 0
```

```
    stack = [(root, 0, False)]                    # (node, sum before this node, leaving?)
    while stack:
        node, before, leaving = stack.pop()
        total = before + node.val
        if leaving:
            prefix_count[total] -= 1
            continue
        count += prefix_count.get(total - target, 0)
        prefix_count[total] = prefix_count.get(total, 0) + 1
        stack.append((node, before, True))
        if node.right:
            stack.append((node.right, total, False))
        if node.left:
            stack.append((node.left, total, False))
    return count
```

PATTERN 5 · PREFIX SUM · PROBLEM 13 OF 20

1524. Number of Sub-arrays With Odd Sum

Medium · Array, Math, Dynamic Programming, Prefix Sum

leetcode.com/problems/number-of-sub-arrays-with-odd-sum

PROBLEM

Given an array of integers `arr`, return *the number of subarrays with an **odd** sum*.

Since the answer can be very large, return it modulo $10^9 + 7$.

Example 1:

```
Input: arr = [1,3,5]
Output: 4
Explanation: All subarrays are [[1],[1,3],[1,3,5],[3],[3,5],[5]]
All sub-arrays sum are [1,4,9,3,8,5].
Odd sums are [1,9,3,5] so the answer is 4.
```

Example 2:

```
Input: arr = [2,4,6]
Output: 0
Explanation: All subarrays are [[2],[2,4],[2,4,6],[4],[4,6],[6]]
All sub-arrays sum are [2,6,12,4,10,6].
All sub-arrays have even sum and the answer is 0.
```

Example 3:

```
Input: arr = [1,2,3,4,5,6,7]
Output: 16
```

Constraints:

- $1 <= arr.length <= 10^5$
- $1 <= arr[i] <= 100$

SOLUTION (PYTHON 3)

Solution 1: Prefix Sum + Counter

How to think about it

Count subarrays whose sum is odd. There are $O(n^2)$ of them and $n \le 10^5$, so explicit sums fail. The parity of a subarray sum is the parity of the difference of two prefix sums. An odd prefix pairs with every previous even prefix; an even prefix pairs with every previous odd one. While scanning we keep those two counters, add the matching count for the current prefix, then update. Reduce the answer modulo 10^9+7.

We define an array cnt of length 2 as a counter, where cnt[0] and cnt[1] represent the number of subarrays with even and odd prefix sums, respectively. Initially, cnt[0] = 1 and cnt[1] = 0.

Next, we maintain the current prefix sum s, initially s = 0.

Traverse the array arr, for each element x encountered, add the value of x to s, then based on the parity of s, add the value of cnt[s mod 2 xor 1] to the answer, and then increment the value of cnt[s mod 2] by 1.

After the traversal, we get the answer. Note the modulo operation for the answer.

Time complexity is O(n), where n is the length of the array arr. Space complexity is O(1).

Python 3

```python
class Solution:
    def numOfSubarrays(self, arr: List[int]) -> int:
        mod = 10**9 + 7
        cnt = [1, 0]
        ans = s = 0
        for x in arr:
            s += x
            ans = (ans + cnt[s & 1 ^ 1]) % mod
            cnt[s & 1] += 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count the subarrays with an EVEN sum
# 2. Count the subarrays whose sum leaves remainder r when
# divided by m


# ========================================================
# FOLLOW-UP 1: Count the subarrays with an EVEN sum
# ========================================================

# ---- follow-up 1: function ----
# Answer: Same idea: two prefix sums with the same parity
# enclose an even-sum subarray. Count the prefix sums that
# are even and odd so far.
# Time: O(n).  Space: O(1).
def even_sum_subarrays(nums):
    even_prefixes = 1                               # the empty prefix
    odd_prefixes = 0
    parity = 0
    count = 0
    for x in nums:
        parity = (parity + x) % 2
        if parity == 0:
            count += even_prefixes
            even_prefixes += 1
        else:
            count += odd_prefixes
            odd_prefixes += 1
    return count


# ========================================================
# FOLLOW-UP 2: Count the subarrays whose sum leaves
# remainder r when divided by m
# ========================================================

# ---- follow-up 2: function ----
# Answer: Count how often each prefix remainder occurred;
# a subarray has remainder r when (current - earlier) mod
# m = r, so look up (current - r) mod m.
# Time: O(n).  Space: O(m).
def subarrays_with_remainder(nums, m, r):
    seen = {0: 1}
    remainder = 0
    count = 0
    for x in nums:
        remainder = (remainder + x) % m
        count += seen.get((remainder - r) % m, 0)
        seen[remainder] = seen.get(remainder, 0) + 1
    return count
```

PATTERN 5 · PREFIX SUM · PROBLEM 14 OF 20

1413. Minimum Value to Get Positive Step by Step Sum

Easy · Array, Prefix Sum

leetcode.com/problems/minimum-value-to-get-positive-step-by-step-sum

PROBLEM

Given an array of integers `nums`, you start with an initial **positive** value *startValue*.

In each iteration, you calculate the step by step sum of *startValue* plus elements in `nums` (from left to right).

Return the minimum **positive** value of *startValue* such that the step by step sum is never less than 1.

Example 1:

```
Input: nums = [-3,2,-3,4,2]
Output: 5
Explanation: If you choose startValue = 4, in the third iteration your step by step sum is less than 1.
step by step sum
startValue = 4 | startValue = 5 | nums
  (4 -3 ) = 1 | (5 -3 ) = 2 | -3
  (1 +2 ) = 3 | (2 +2 ) = 4 | 2
  (3 -3 ) = 0 | (4 -3 ) = 1 | -3
  (0 +4 ) = 4 | (1 +4 ) = 5 | 4
  (4 +2 ) = 6 | (5 +2 ) = 7 | 2
```

Example 2:

```
Input: nums = [1,2]
Output: 1
Explanation: Minimum start value should be positive.
```

Example 3:

```
Input: nums = [1,-2,-3]
Output: 5
```

Constraints:

- `1 <= nums.length <= 100`
- `-100 <= nums[i] <= 100`

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Every running sum must stay at least 1. If the start value is x, then x plus every prefix is ≥ 1, so x≥ 1-minprefix. Also x≥ 1. n≤ 100. One pass tracks the prefix and its minimum t; the answer is max(1,1-t).

Python 3

```python
class Solution:
    def minStartValue(self, nums: List[int]) -> int:
        s, t = 0, inf
        for num in nums:
            s += num
            t = min(t, s)
        return max(1, 1 - t)
```

Complexity. Time: O(n). Space: O(1).

Solution 2

> **How to think about it**
>
> Method 1 tracks the minimum on the fly. Building all prefixes with `accumulate` and then taking min uses the same formula, only with an explicit prefix array.

Python 3

```python
class Solution:
    def minStartValue(self, nums: List[int]) -> int:
        s = list(accumulate(nums))
        return 1 if min(s) >= 0 else abs(min(s)) + 1
```

Complexity. Time: O(n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The running sum must never drop below a given floor T
# (not only below 1)
# 2. Where does the running sum reach its lowest point?


# ============================================================
# FOLLOW-UP 1: The running sum must never drop below a
# given floor T (not only below 1)
# ============================================================

# ---- follow-up 1: function ----
# Answer: The start value must lift the lowest running sum
# (after at least one step) up to T; the start value
# itself must still be a positive integer.
# Time: O(n).  Space: O(1).
def min_start_value(nums, floor):
    total = 0
    lowest = None
    for x in nums:
        total += x
        if lowest is None or total < lowest:
            lowest = total
    if lowest is None:
        return 1
    start = floor - lowest
    if start < 1:
        return 1
    return start


# ============================================================
# FOLLOW-UP 2: Where does the running sum reach its lowest
# point?
# ============================================================

# ---- follow-up 2: function ----
# Answer: The lowest prefix sum and the first index where
# it happens are the places where the walk is in most
# danger.
# Time: O(n).  Space: O(1).
def lowest_point(nums):
    total = 0
    lowest = 0
    position = -1                                   # -1 means: before the first number
    for i in range(len(nums)):
        total += nums[i]
        if total < lowest:
            lowest = total
```

```
            position = i
    return lowest, position
```

1031. Maximum Sum of Two Non-Overlapping Subarrays

Medium · Array, Dynamic Programming, Sliding Window

leetcode.com/problems/maximum-sum-of-two-non-overlapping-subarrays

Also appears in pattern 10 Queue / Deque

PROBLEM

Given an integer array `nums` and two integers `firstLen` and `secondLen`, return *the maximum sum of elements in two non-overlapping **subarrays** with lengths* `firstLen` *and* `secondLen`.

The array with length `firstLen` could occur before or after the array with length `secondLen`, but they have to be non-overlapping.

A **subarray** is a **contiguous** part of an array.

Example 1:

```
Input: nums = [0,6,5,2,2,5,1,9,4], firstLen = 1, secondLen = 2
Output: 20
Explanation: One choice of subarrays is [9] with length 1, and [6,5] with length 2.
```

Example 2:

```
Input: nums = [3,8,1,3,2,1,8,9,0], firstLen = 3, secondLen = 2
Output: 29
Explanation: One choice of subarrays is [3,8,1] with length 3, and [8,9] with length 2.
```

Example 3:

```
Input: nums = [2,1,5,6,0,9,5,0,3,8], firstLen = 4, secondLen = 3
Output: 31
Explanation: One choice of subarrays is [5,6,0,9] with length 4, and [0,3,8] with length 3.
```

Constraints:

- `1 <= firstLen, secondLen <= 1000`
- `2 <= firstLen + secondLen <= 1000`
- `firstLen + secondLen <= nums.length <= 1000`
- `0 <= nums[i] <= 1000`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Enumerating both segments is O(n^2) and tight at n≤ 1000. After one segment is fixed, the other only needs the best fixed-length window on the remaining side. Prefix sums give any window sum in O(1). We handle “ firstLen on the left” and “ secondLen on the left” separately, keeping the best left window in t as the right window slides. The answer is the larger of the two scans.

Python 3

```
class Solution:
    def maxSumTwoNoOverlap(self, nums: List[int], firstLen: int, secondLen: int) -> int:
        n = len(nums)
        s = list(accumulate(nums, initial=0))
        ans = t = 0
        i = firstLen
        while i + secondLen - 1 < n:
            t = max(t, s[i] - s[i - firstLen])
            ans = max(ans, t + s[i + secondLen] - s[i])
            i += 1
        t = 0
        i = secondLen
        while i + firstLen - 1 < n:
            t = max(t, s[i] - s[i - secondLen])
            ans = max(ans, t + s[i + firstLen] - s[i])
            i += 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Three non-overlapping subarrays of the same length k:
# return the starts with the maximum total


# ============================================================
# FOLLOW-UP 1: Three non-overlapping subarrays of the same
# length k: return the starts with the maximum total
# ============================================================

# ---- follow-up 1: function ----
# Answer: Compute the window sums. For the middle window,
# combine the best window to its left and the best window
# to its right; keep the starting positions of those best
# windows.
# Time: O(n).  Space: O(n).
def max_sum_three_subarrays(nums, k):
    windows = []
    current = 0
    for i in range(len(nums)):
        current += nums[i]
        if i >= k:
            current -= nums[i - k]
        if i >= k - 1:
            windows.append(current)
    count = len(windows)
    best_left = [0] * count
    best = 0
    for i in range(count):
        if windows[i] > windows[best]:
            best = i
        best_left[i] = best
    best_right = [0] * count
    best = count - 1
    for i in range(count - 1, -1, -1):
        if windows[i] >= windows[best]:
            best = i
        best_right[i] = best
    answer = None
    best_total = -1
    for middle in range(k, count - k):
        left, right = best_left[middle - k], best_right[middle + k]
        total = windows[left] + windows[middle] + windows[right]
        if total > best_total:
            best_total = total
```

```
            answer = [left, middle, right]
    return answer
```

PATTERN 5 · PREFIX SUM · PROBLEM 16 OF 20

307. Range Sum Query - Mutable

Medium · Design, Binary Indexed Tree, Segment Tree, Array, Divide and Conquer, Sqrt Decomposition

leetcode.com/problems/range-sum-query-mutable

★ **TOP 7 PROBLEM** in Segment Tree, Binary Indexed Tree (Fenwick Tree): master this one first

Also appears in pattern 40 Segment Tree, 41 Binary Indexed Tree (Fenwick Tree)

PROBLEM

Given an integer array `nums`, handle multiple queries of the following types:

1. **Update** the value of an element in `nums`.
2. Calculate the **sum** of the elements of `nums` between indices `left` and `right` **inclusive** where `left <= right`.

Implement the `NumArray` class:

- `NumArray(int[] nums)` Initializes the object with the integer array `nums`.
- `void update(int index, int val)` **Updates** the value of `nums[index]` to be `val`.
- `int sumRange(int left, int right)` Returns the **sum** of the elements of `nums` between indices `left` and `right` **inclusive** (i.e. `nums[left] + nums[left + 1] + ... + nums[right]`).

Example 1:

```
Input
["NumArray", "sumRange", "update", "sumRange"]
[[[1, 3, 5]], [0, 2], [1, 2], [0, 2]]
Output
[null, 9, null, 8]

Explanation
NumArray numArray = new NumArray([1, 3, 5]);
numArray.sumRange(0, 2); // return 1 + 3 + 5 = 9
numArray.update(1, 2); // nums = [1, 2, 5]
numArray.sumRange(0, 2); // return 1 + 2 + 5 = 8
```

Constraints:

- `1 <= nums.length <= 3 * 10^4`
- `-100 <= nums[i] <= 100`
- `0 <= index < nums.length`
- `-100 <= val <= 100`
- `0 <= left <= right < nums.length`
- At most `3 * 10^4` calls will be made to `update` and `sumRange`.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Unlike problem 303, a point update must stay cheap together with range sums. Rebuilding a prefix array after every write, or scanning on each query, cannot keep both operations logarithmic. A Fenwick tree stores prefix sums via lowbit: update walks x+=x&-x, query walks x-=x&-x. A range is the difference of two prefixes; an update reads the old value and writes the delta. The constructor inserts each index once.

Python 3

```python
class BinaryIndexedTree:
    __slots__ = ["n", "c"]

    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x: int, delta: int):
        while x <= self.n:
            self.c[x] += delta
            x += x & -x

    def query(self, x: int) -> int:
        s = 0
        while x > 0:
            s += self.c[x]
            x -= x & -x
        return s


class NumArray:
    __slots__ = ["tree"]

    def __init__(self, nums: List[int]):
        self.tree = BinaryIndexedTree(len(nums))
        for i, v in enumerate(nums, 1):
            self.tree.update(i, v)

    def update(self, index: int, val: int) -> None:
        prev = self.sumRange(index, index)
        self.tree.update(index + 1, val - prev)

    def sumRange(self, left: int, right: int) -> int:
        return self.tree.query(right + 1) - self.tree.query(left)


# Your NumArray object will be instantiated and called as such:
# obj = NumArray(nums)
# obj.update(index,val)
# param_2 = obj.sumRange(left,right)
```

Complexity. Time: build O(n log n), update and sumRange O(log n) each. Space: O(n).

Solution 2

How to think about it

A Fenwick tree is tailored to prefix sums and is less natural for arbitrary segment aggregates. A segment tree splits [l,r] at the midpoint; each internal node stores its interval sum, and a point update touches only the O(log n) nodes on the root-to-leaf path. A query decomposes [left,right] into O(log n) node intervals. The asymptotics match Method 1; the explicit children make later extensions (e.g. range extrema) straightforward.

Python 3

```python
class Node:
    __slots__ = ["l", "r", "v"]

    def __init__(self):
        self.l = self.r = self.v = 0


class SegmentTree:
    __slots__ = ["nums", "tr"]

    def __init__(self, nums):
        self.nums = nums
        n = len(nums)
        self.tr = [Node() for _ in range(n << 2)]
        self.build(1, 1, n)

    def build(self, u, l, r):
        self.tr[u].l, self.tr[u].r = l, r
        if l == r:
            self.tr[u].v = self.nums[l - 1]
            return
        mid = (l + r) >> 1
        self.build(u << 1, l, mid)
        self.build(u << 1 | 1, mid + 1, r)
        self.pushup(u)

    def modify(self, u, x, v):
        if self.tr[u].l == x and self.tr[u].r == x:
            self.tr[u].v = v
            return
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        if x <= mid:
            self.modify(u << 1, x, v)
        else:
            self.modify(u << 1 | 1, x, v)
        self.pushup(u)

    def query(self, u, l, r):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            return self.tr[u].v
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        result = 0
        if l <= mid:
            result += self.query(u << 1, l, r)
        if r > mid:
            result += self.query(u << 1 | 1, l, r)
        return result

    def pushup(self, u):
        self.tr[u].v = self.tr[u << 1].v + self.tr[u << 1 | 1].v


class NumArray:
    __slots__ = ["tree"]

    def __init__(self, nums: List[int]):
        self.tree = SegmentTree(nums)

    def update(self, index: int, val: int) -> None:
        self.tree.modify(1, index + 1, val)
```

```python
    def sumRange(self, left: int, right: int) -> int:
        return self.tree.query(1, left + 1, right + 1)


# Your NumArray object will be instantiated and called as such:
```

```
# obj = NumArray(nums)
# obj.update(index,val)
# param_2 = obj.sumRange(left,right)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Use a segment tree instead of a Fenwick tree
# 2. Add a value to a whole range, and ask for a single
# element


# ============================================================
# FOLLOW-UP 1: Use a segment tree instead of a Fenwick
# tree
# ============================================================

# ---- follow-up 1: function ----
# Answer: A tree where every node stores the sum of a
# range: a point update fixes the O(log n) nodes above the
# leaf, and a query combines O(log n) nodes.
# Time: build O(n), update and sum_range O(log n).  Space:
# O(n).
class SegmentTree:
    def __init__(self, nums):
        self.n = len(nums)
        self.tree = [0] * (2 * self.n)
        for i in range(self.n):
            self.tree[self.n + i] = nums[i]
        for i in range(self.n - 1, 0, -1):
            self.tree[i] = self.tree[2 * i] + self.tree[2 * i + 1]

    def update(self, index, value):
        i = index + self.n
        self.tree[i] = value
        while i > 1:
            i //= 2
            self.tree[i] = self.tree[2 * i] + self.tree[2 * i + 1]

    def sum_range(self, left, right):
        total = 0
        i, j = left + self.n, right + self.n + 1
        while i < j:
            if i % 2 == 1:
                total += self.tree[i]
                i += 1
            if j % 2 == 1:
                j -= 1
                total += self.tree[j]
            i //= 2
            j //= 2
        return total


# ============================================================
# FOLLOW-UP 2: Add a value to a whole range, and ask for a
# single element
# ============================================================

# ---- follow-up 2: function ----
# Answer: Store the DIFFERENCES between neighbours in a
# Fenwick tree: adding to a range changes two differences,
# and an element is the prefix sum of the differences up
# to it.
# Time: range_add and point_query O(log n).  Space: O(n).
class RangeAddPointQuery:
```

```
    def __init__(self, size):
        self.size = size
        self.tree = [0] * (size + 2)

    def add_to_tree(self, i, delta):
        i += 1
        while i <= self.size + 1:
            self.tree[i] += delta
            i += i & -i

    def range_add(self, left, right, value):
        self.add_to_tree(left, value)
        self.add_to_tree(right + 1, -value)

    def point_query(self, index):
        i = index + 1
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & -i
        return total
```

PATTERN 5 · PREFIX SUM · PROBLEM 17 OF 20

1590. Make Sum Divisible by P

Medium · Array, Hash Table, Prefix Sum

leetcode.com/problems/make-sum-divisible-by-p

★ **TOP 7 PROBLEM** in Prefix Sum: master this one first

PROBLEM

Given an array of positive integers `nums`, remove the **smallest** subarray (possibly **empty**) such that the **sum** of the remaining elements is divisible by `p`. It is **not** allowed to remove the whole array.

Return *the length of the smallest subarray that you need to remove, or* `-1` *if it's impossible*.

A **subarray** is defined as a contiguous block of elements in the array.

Example 1:

```
Input: nums = [3,1,4,2], p = 6
Output: 1
Explanation: The sum of the elements in nums is 10, which is not divisible by 6. We can remove the
subarray [4], and the sum of the remaining elements is 6, which is divisible by 6.
```

Example 2:

```
Input: nums = [6,3,5,2], p = 9
Output: 2
Explanation: We cannot remove a single element to get a sum divisible by 9. The best way is to remove
the subarray [5,2], leaving us with [6,3] with sum 9.
```

Example 3:

```
Input: nums = [1,2,3], p = 3
Output: 0
Explanation: Here the sum is 6. which is already divisible by 3. Thus we do not need to remove
anything.
```

Constraints:

- `1 <= nums.length <= 10^5`
- `1 <= nums[i] <= 10^9`
- `1 <= p <= 10^9`

SOLUTION (PYTHON 3)

Solution 1: Prefix Sum + Hash Table

How to think about it

Delete the shortest subarray so the remaining sum is divisible by p. n≤ 10^5 rules out enumerating intervals. If the total is k modulo p, the deleted piece must be k modulo p as well. Scan prefix sums modulo p and remember the last index of each residue. At residue cur, look up (cur-k) mod p; the gap is a candidate deletion. Deleting the entire array is forbidden.

First, we calculate the sum of all elements in the array nums modulo p, denoted as k. If k is 0, it means the sum of all elements in the array nums is a multiple of p, so we directly return 0.

If k is not 0, we need to find the shortest subarray such that removing this subarray makes the sum of the remaining elements modulo p equal to 0.

We can traverse the array nums, maintaining the current prefix sum modulo p, denoted as cur. We use a hash table last to record the last occurrence of each prefix sum modulo p.

If there exists a subarray ending at nums[i] such that removing this subarray makes the sum of the remaining elements modulo p equal to 0, we need to find a previous prefix sum modulo p equal to target at position j such that (target + k - cur) mod p = 0. If found, we can remove the subarray nums[j+1,..i] to make the sum of the remaining elements modulo p equal to 0.

Therefore, if there exists a target = (cur - k + p) mod p, we can update the answer to min(ans, i - j). Then, we update last[cur] to i. We continue traversing the array nums until the end to get the answer.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def minSubarray(self, nums: List[int], p: int) -> int:
        k = sum(nums) % p
        if k == 0:
            return 0
        last = {0: -1}
        cur = 0
        ans = len(nums)
        for i, x in enumerate(nums):
            cur = (cur + x) % p
            target = (cur - k + p) % p
            if target in last:
                ans = min(ans, i - last[target])
            last[cur] = i
        return -1 if ans == len(nums) else ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the positions of the shortest subarray to
# remove
# 2. Only a prefix or a suffix may be removed


# ========================================================
# FOLLOW-UP 1: Return the positions of the shortest
# subarray to remove
# ========================================================

# ---- follow-up 1: function ----
# Answer: Same prefix-remainder dictionary, but store the
# length together with its start: the removed part is
# nums[start .. i].
# Time: O(n).  Space: O(min(n, p)).
def shortest_removal_positions(nums, p):
    total = 0
    for x in nums:
        total += x
    need = total % p
    if need == 0:
        return None                                  # nothing to remove
    last_index = {0: -1}
    remainder = 0
    best = None
    for i in range(len(nums)):
        remainder = (remainder + nums[i]) % p
        wanted = (remainder - need) % p
        if wanted in last_index:
            start = last_index[wanted] + 1
            if best is None or i - start < best[1] - best[0]:
                best = (start, i)
        last_index[remainder] = i
    return best
```

```
# ============================================================
# FOLLOW-UP 2: Only a prefix or a suffix may be removed
# ============================================================

# ---- follow-up 2: function ----
# Answer: Try every prefix length and every suffix length:
# the removed part must have the remainder total mod p.
# Time: O(n).  Space: O(1).
def shortest_prefix_or_suffix_removal(nums, p):
    total = 0
    for x in nums:
        total += x
    need = total % p
    if need == 0:
        return 0
    best = None
    running = 0
    for length in range(1, len(nums) + 1):
        running += nums[length - 1]
        if running % p == need and (best is None or length < best):
            best = length
    running = 0
    for length in range(1, len(nums) + 1):
```

```
        running += nums[len(nums) - length]
        if running % p == need and (best is None or length < best):
            best = length
    return best if best is not None else -1
```

PATTERN 5 · PREFIX SUM · PROBLEM 18 OF 20

1314. Matrix Block Sum

Medium · Array, Matrix, Prefix Sum

leetcode.com/problems/matrix-block-sum

PROBLEM

Given a `m x n` matrix `mat` and an integer `k`, return *a matrix* `answer` *where each* `answer[i][j]` *is the sum of all elements* `mat[r][c]` *for*:

- `i - k <= r <= i + k`,
- `j - k <= c <= j + k`, and
- `(r, c)` is a valid position in the matrix.

Example 1:

```
Input: mat = [[1,2,3],[4,5,6],[7,8,9]], k = 1
Output: [[12,21,16],[27,45,33],[24,39,28]]
```

Example 2:

```
Input: mat = [[1,2,3],[4,5,6],[7,8,9]], k = 2
Output: [[45,45,45],[45,45,45],[45,45,45]]
```

Constraints:

- `m == mat.length`
- `n == mat[i].length`
- `1 <= m, n, k <= 100`
- `1 <= mat[i][j] <= 100`

SOLUTION (PYTHON 3)

Solution 1: Two-Dimensional Prefix Sum

How to think about it

Each cell asks for the sum of a square neighborhood. Summing on the fly is $O(mnk^2)$ when k is large. The same rectangles are requested many times; a 2D prefix reduces any rectangle to four corners. Build s[i][j] as the sum of the top-left i × j block, clamp each window to the matrix, and evaluate $s[x_2+1][y_2+1]-s[x_1][y_2+1]-s[x_2+1][y_1]+s[x_1][y_1]$.

This problem is a template for two-dimensional prefix sum.

We define s[i][j] as the sum of the elements in the first i rows and the first j columns of the matrix mat. The calculation formula for s[i][j] is:

$$s[i][j] = s[i-1][j] + s[i][j-1] - s[i-1][j-1] + mat[i-1][j-1]$$

In this way, we can quickly calculate the sum of elements in any rectangular area through the s array.

For a rectangular area with the upper left coordinate (x_1, y_1) and the lower right coordinate (x_2, y_2), we can calculate the sum of its elements through the s array:

$$s[x_2+1][y_2+1] - s[x_1][y_2+1] - s[x_2+1][y_1] + s[x_1][y_1]$$

The time complexity is O(m × n), and the space complexity is O(m × n). Where m and n are the number of rows and columns in the matrix, respectively.

Python 3

```python
class Solution:
    def matrixBlockSum(self, mat: List[List[int]], k: int) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        s = [[0] * (n + 1) for _ in range(m + 1)]
        for i, row in enumerate(mat, 1):
            for j, x in enumerate(row, 1):
                s[i][j] = s[i - 1][j] + s[i][j - 1] - s[i - 1][j - 1] + x
        ans = [[0] * n for _ in range(m)]
        for i in range(m):
            for j in range(n):
                x1, y1 = max(i - k, 0), max(j - k, 0)
                x2, y2 = min(m - 1, i + k), min(n - 1, j + k)
                ans[i][j] = (
                    s[x2 + 1][y2 + 1] - s[x1][y2 + 1] - s[x2 + 1][y1] + s[x1][y1]
                )
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Answer one rectangle query: the sum of the cells
# between two corners
# 2. Which centre cell has the largest block sum for a
# given radius?


# ============================================================
# FOLLOW-UP 1: Answer one rectangle query: the sum of the
# cells between two corners
# ============================================================

# ---- follow-up 1: function ----
# Answer: With the 2D prefix sums built once, any
# rectangle is four lookups (add the big one, subtract the
# strips above and to the left, add back the corner that
# was subtracted twice).
# Time: build O(m * n), each query O(1).  Space: O(m * n).
def build_prefix(matrix):
    rows, cols = len(matrix), len(matrix[0])
    prefix = []
    for r in range(rows + 1):
        prefix.append([0] * (cols + 1))
    for r in range(rows):
        for c in range(cols):
            prefix[r + 1][c + 1] = matrix[r][c] + prefix[r][c + 1] + prefix[r + 1][c] - prefix[r][c]
    return prefix


def rectangle_sum(prefix, r1, c1, r2, c2):
    return prefix[r2 + 1][c2 + 1] - prefix[r1][c2 + 1] - prefix[r2 + 1][c1] + prefix[r1][c1]


# ============================================================
# FOLLOW-UP 2: Which centre cell has the largest block sum
# for a given radius?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Build the prefix sums once and clamp every block
# to the matrix, then keep the best cell.
# Time: O(m * n).  Space: O(m * n).
def best_block_centre(matrix, radius):
    prefix = build_prefix(matrix)
    rows, cols = len(matrix), len(matrix[0])
    best_sum = None
```

```
    best_cell = (0, 0)
    for r in range(rows):
        for c in range(cols):
            r1 = r - radius if r - radius > 0 else 0
            c1 = c - radius if c - radius > 0 else 0
            r2 = r + radius if r + radius < rows - 1 else rows - 1
            c2 = c + radius if c + radius < cols - 1 else cols - 1
            total = rectangle_sum(prefix, r1, c1, r2, c2)
            if best_sum is None or total > best_sum:
                best_sum = total
                best_cell = (r, c)
    return best_cell, best_sum
```

PATTERN 5 · PREFIX SUM · PROBLEM 19 OF 20

1277. Count Square Submatrices with All Ones

Medium · Array, Dynamic Programming, Matrix

leetcode.com/problems/count-square-submatrices-with-all-ones

Also appears in pattern 22 Dynamic Programming - 2D / Grid, 28 Dynamic Programming - Knapsack Style

PROBLEM

Given a `m * n` matrix of ones and zeros, return how many **square** submatrices have all ones.

Example 1:

```
Input: matrix =
[
  [0,1,1,1],
  [1,1,1,1],
  [0,1,1,1]
]
Output: 15
Explanation:
There are 10 squares of side 1.
There are 4 squares of side 2.
There is 1 square of side 3.
Total number of squares = 10 + 4 + 1 = 15.
```

Example 2:

```
Input: matrix =
[
  [1,0,1],
  [1,1,0],
  [1,1,0]
]
Output: 7
Explanation:
There are 6 squares of side 1.
There is 1 square of side 2.
Total number of squares = 6 + 1 = 7.
```

Constraints:

- `1 <= arr.length <= 300`
- `1 <= arr[0].length <= 300`
- `0 <= arr[i][j] <= 1`

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

> **How to think about it**
>
> We count all-1 squares. m,n ≤ 300, so enumerating squares is O(n^3). The largest square cornered at (i,j) is limited by the squares at the top, left, and top-left cells. f[i][j] is that side length: if the cell is 1, take the min of those three plus one. Each such square contributes f[i][j] squares (sides 1… f). DP turns counting into one fill.

We define f[i][j] as the side length of the square submatrix with (i,j) as the bottom-right corner. Initially f[i][j] = 0, and the answer is $\sum_{i,j}$ f[i][j].

Consider how to perform state transition for f[i][j].

- When matrix[i][j] = 0, we have f[i][j] = 0.

- When matrix[i][j] = 1, the value of state f[i][j] depends on the values of the three positions above, left, and top-left:
  - If i = 0 or j = 0, then f[i][j] = 1.
  - Otherwise f[i][j] = min(f[i-1][j-1], f[i-1][j], f[i][j-1]) + 1.

The answer is $\sum_{i,j} f[i][j]$.

Time complexity O(m × n), space complexity O(m × n). Where m and n are the number of rows and columns of the matrix respectively.

Python 3

```
class Solution:
    def countSquares(self, matrix: List[List[int]]) -> int:
        m, n = len(matrix), len(matrix[0])
        f = [[0] * n for _ in range(m)]
        ans = 0
        for i, row in enumerate(matrix):
            for j, v in enumerate(row):
                if v == 0:
                    continue
                if i == 0 or j == 0:
                    f[i][j] = 1
                else:
                    f[i][j] = min(f[i - 1][j - 1], f[i - 1][j], f[i][j - 1]) + 1
                ans += f[i][j]
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The side of the largest all-ones square (Maximal
# Square)
# 2. The largest all-ones RECTANGLE


# ============================================================
# FOLLOW-UP 1: The side of the largest all-ones square
# (Maximal Square)
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same table: side[r][c] is the side of the
# biggest square of ones whose bottom-right corner is (r,
# c). It is 1 + the smallest of the three neighbours
# above, left and diagonal. The answer is the largest
# value.
# Time: O(m * n).  Space: O(m * n).
def largest_square_side(matrix):
    rows, cols = len(matrix), len(matrix[0])
    side = []
    for r in range(rows):
        side.append([0] * cols)
    best = 0
    for r in range(rows):
        for c in range(cols):
            if matrix[r][c] == 1:
                if r == 0 or c == 0:
                    side[r][c] = 1
                else:
                    smallest = side[r - 1][c]
                    if side[r][c - 1] < smallest:
                        smallest = side[r][c - 1]
                    if side[r - 1][c - 1] < smallest:
                        smallest = side[r - 1][c - 1]
                    side[r][c] = smallest + 1
                if side[r][c] > best:
                    best = side[r][c]
```

```
    return best


# ============================================================
# FOLLOW-UP 2: The largest all-ones RECTANGLE
# ============================================================

# ---- follow-up 2: helper functions ----
def largest_in_histogram(heights):
    stack = []                                      # indexes of bars with increasing heights
    best = 0
    for i in range(len(heights) + 1):
        current = heights[i] if i < len(heights) else 0
        while stack and heights[stack[-1]] >= current:
            height = heights[stack.pop()]
            left = stack[-1] if stack else -1
            area = height * (i - left - 1)
            if area > best:
                best = area
        stack.append(i)
    return best
```

```
# ---- follow-up 2: function ----
# Answer: Treat every row as the base of a histogram of
# consecutive ones above it, and find the largest
# rectangle in each histogram with a stack of increasing
# bars.
# Time: O(m * n).  Space: O(n).
def largest_rectangle_of_ones(matrix):
    cols = len(matrix[0])
    heights = [0] * cols
    best = 0
    for row in matrix:
        for c in range(cols):
            if row[c] == 1:
                heights[c] += 1
            else:
                heights[c] = 0
        area = largest_in_histogram(heights)
        if area > best:
            best = area
    return best
```

2270. Number of Ways to Split Array

Medium · Array, Prefix Sum

leetcode.com/problems/number-of-ways-to-split-array

PROBLEM

You are given a **0-indexed** integer array `nums` of length `n`.

`nums` contains a **valid split** at index `i` if the following are true:

- The sum of the first `i + 1` elements is **greater than or equal to** the sum of the last `n - i - 1` elements.
- There is **at least one** element to the right of `i`. That is, `0 <= i < n - 1`.

Return *the number of **valid splits** in* `nums`.

Example 1:

```
Input: nums = [10,4,-8,7]
Output: 2
Explanation:
There are three ways of splitting nums into two non-empty parts:
- Split nums at index 0. Then, the first part is [10], and its sum is 10. The second part is [4,-8,7],
and its sum is 3. Since 10 >= 3, i = 0 is a valid split.
- Split nums at index 1. Then, the first part is [10,4], and its sum is 14. The second part is [-8,7],
and its sum is -1. Since 14 >= -1, i = 1 is a valid split.
- Split nums at index 2. Then, the first part is [10,4,-8], and its sum is 6. The second part is [7],
and its sum is 7. Since 6 < 7, i = 2 is not a valid split.
Thus, the number of valid splits in nums is 2.
```

Example 2:

```
Input: nums = [2,3,1,0]
Output: 2
Explanation:
There are two valid splits in nums:
- Split nums at index 1. Then, the first part is [2,3], and its sum is 5. The second part is [1,0], and
its sum is 1. Since 5 >= 1, i = 1 is a valid split.
- Split nums at index 2. Then, the first part is [2,3,1], and its sum is 6. The second part is [0], and
its sum is 0. Since 6 >= 0, i = 2 is a valid split.
```

Constraints:

- `2 <= nums.length <= 10^5`
- `-10^5 <= nums[i] <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: Prefix Sum

> **How to think about it**
>
> A valid split needs the left sum at least the right sum, and a non-empty right part. n ≤ 10^5 forbids summing from scratch. The two sides add to the total s, so the test is prefix t ≥ s-t. Compute s, then scan the first n-1 elements, accumulating t and counting.

First, we calculate the total sum s of the array nums. Then, we traverse the first n-1 elements of the array nums, using the variable t to record the prefix sum. If t ≥ s - t, we increment the answer by one.

After the traversal, we return the answer.

The time complexity is O(n), where n is the length of the array nums. The space complexity is O(1).

Python 3

```
class Solution:
    def waysToSplitArray(self, nums: List[int]) -> int:
        s = sum(nums)
        ans = t = 0
        for x in nums[:-1]:
            t += x
            ans += t >= s - t
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Cut the array into THREE parts with equal sums: how
# many ways?
# 2. Return the positions of the valid splits instead of
# the count


# ============================================================
# FOLLOW-UP 1: Cut the array into THREE parts with equal
# sums: how many ways?
# ============================================================

# ---- follow-up 1: function ----
# Answer: The total must be divisible by 3, and each part
# must sum to total / 3. Walk through the prefix sums:
# every position whose prefix equals 2/3 of the total can
# end the second part, and it pairs with every earlier
# position whose prefix equals 1/3 of the total.
# Time: O(n).  Space: O(1).
def ways_to_split_into_three(nums):
    total = 0
    for x in nums:
        total += x
    if total % 3 != 0 or len(nums) < 3:
        return 0
    third = total // 3
    prefix = 0
    ends_of_first = 0                               # cut positions where the first part already sums to
third
    ways = 0
    for i in range(len(nums) - 1):
        prefix += nums[i]
        if prefix == 2 * third and i >= 1:
            ways += ends_of_first                   # a second cut here, first cuts before
        if prefix == third:
            ends_of_first += 1
    return ways


# ============================================================
# FOLLOW-UP 2: Return the positions of the valid splits
# instead of the count
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same prefix sums; store every index where the
# left sum is at least the right sum.
# Time: O(n).  Space: O(1) extra.
def valid_split_positions(nums):
    total = 0
    for x in nums:
        total += x
    positions = []
    left = 0
```

```
    for i in range(len(nums) - 1):
        left += nums[i]
        if left >= total - left:
            positions.append(i)
    return positions
```

PATTERN 6 OF 41

String Manipulation / Parsing

20 problems · 4 Easy · 13 Medium · 3 Hard

THE IDEA

Walk the string once with an index (or a stack) and handle each kind of character. Most bugs are edge cases: signs, spaces, overflow, empty input.

Signals: convert or validate text (atoi, roman numerals), reverse words, compress, add or multiply numbers stored as strings, simplify a path.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
i, n = 0, len(s)
while i < n and s[i] == " ":        # skip leading spaces
    i += 1
sign = 1
if i < n and s[i] in "+-":
    sign = -1 if s[i] == "-" else 1
    i += 1
value = 0
while i < n and s[i].isdigit():
    value = value * 10 + int(s[i])
    i += 1
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Valid Parentheses	Easy	LC 20	292
2	Longest Common Prefix	Easy	LC 14	295
3	Repeated Substring Pattern	Easy	LC 459	297
4	Implement strStr() — substitute: Find the Index of the First Occurrence in a String	Easy	LC 28	299
5	★ **String to Integer (atoi)**	Medium	LC 8	303
6	Zigzag Conversion	Medium	LC 6	307
7	Group Anagrams (see page)	Medium	LC 49	31
8	★ **Multiply Strings**	Medium	LC 43	310
9	Compare Version Numbers	Medium	LC 165	313
10	Decode Ways	Medium	LC 91	316
11	★ **Simplify Path**	Medium	LC 71	320
12	Word Break	Medium	LC 139	323
13	★ **Longest Palindromic Substring**	Medium	LC 5	327
14	Count and Say	Medium	LC 38	330
15	Group Shifted Strings	Medium	LC 249	333
16	Reorganize String	Medium	LC 767	335
17	★ **Decode String**	Medium	LC 394	338
18	★ **Text Justification**	Hard	LC 68	341
19	★ **Basic Calculator**	Hard	LC 224	344
20	Minimum Window Substring (see page)	Hard	LC 76	121

PATTERN 6 · STRING MANIPULATION / PARSING · PROBLEM 1 OF 20

20. Valid Parentheses

Easy · Stack, String, Parentheses

leetcode.com/problems/valid-parentheses

Also appears in pattern 8 Stack

PROBLEM

Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: s = "()"

Output: true

Example 2:

Input: s = "()[]{}"

Output: true

Example 3:

Input: s = "(]"

Output: false

Example 4:

Input: s = "([])"

Output: true

Example 5:

Input: s = "([)]"

Output: false

Constraints:

- `1 <= s.length <= 10^4`
- `s` consists of parentheses only `'()[]{}'`.

SOLUTION (PYTHON 3)

Solution 1: Stack

How to think about it

Repeatedly deleting adjacent pairs (), [], and {} until nothing is left is correct, but each pass rescans the string. With n ≤ 10^4, deeply nested input falls to O(n^2) and can easily time out. The bottleneck is that a match is more than two neighboring characters. A later opener must close first, while an earlier one stays open. Once the brackets cross, as in ([)], deleting neighbors cannot reduce the string to empty. Matching is therefore last-in, first-out: the only legal partner of the current closer is the most recent unmatched opener. A stack keeps those unfinished matches. An opener pushes its closer, so the closer allowed next sits on top, and a closer is compared only with that top. A mismatch means the type or the order is already wrong, and a non-empty stack at the end means some opener never closed.

We use a hash table d to map each opening bracket to its closing bracket, and a stack stk to store closers that have not been matched yet. Scan s from left to right. When the current character is an opening bracket, push the corresponding closer from d onto stk. When it is a closing bracket, return `false` if stk is empty or the popped top is different from that character.

After the scan, return `true` if stk is empty: every pair has been closed in the right type and order. If the stack still holds a closer, some opening bracket was never matched, so return `false`.

The time complexity is O(n), and the space complexity is O(n), where n is the length of s.

Python 3

```python
class Solution:
    def isValid(self, s: str) -> bool:
        stk = []
        d = {'(': ')', '[': ']', '{': '}'}
        for c in s:
            if c in d:
                stk.append(d[c])
            elif not stk or stk.pop() != c:
                return False
        return not stk
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the position of the first bracket that breaks
# the rule
# 2. The length of the longest valid parentheses substring
# 3. The string may contain '*' that can be '(', ')' or
# empty


# ============================================================
# FOLLOW-UP 1: Return the position of the first bracket
# that breaks the rule
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep a stack of (bracket, position). A closing
# bracket that does not match the top, or that meets an
# empty stack, is the first error; leftover opening
# brackets are errors at their own position.
# Time: O(n).  Space: O(n).
def first_bracket_error(s):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for i in range(len(s)):
        ch = s[i]
        if ch in "([{":
            stack.append((ch, i))
        elif ch in pairs:
            if not stack or stack[-1][0] != pairs[ch]:
                return i
            stack.pop()
    if stack:
        return stack[0][1]
    return -1                                       # the string is valid


# ============================================================
# FOLLOW-UP 2: The length of the longest valid parentheses
# substring
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep a stack of positions with the position just
```

```
# before the current valid run at the bottom. A closing
# bracket pops; if the stack becomes empty it becomes the
# new bottom, otherwise the run length is the distance to
# the new top.
# Time: O(n).  Space: O(n).
def longest_valid_parentheses(s):
    stack = [-1]
    best = 0
    for i in range(len(s)):
        if s[i] == "(":
            stack.append(i)
        else:
            stack.pop()
            if not stack:
                stack.append(i)
            elif i - stack[-1] > best:
                best = i - stack[-1]
    return best
```

```
# ============================================================
# FOLLOW-UP 3: The string may contain '*' that can be '(',
# ')' or empty
# ============================================================

# ---- follow-up 3: function ----
# Answer: Track the lowest and the highest possible number
# of open brackets. '(' raises both, ')' lowers both, '*'
# lowers the low end and raises the high end. The high end
# must never drop below zero, and the low end is kept at
# zero or more; the string is valid if the low end reaches
# 0 at the end.
# Time: O(n).  Space: O(1).
def check_valid_string(s):
    low = 0
    high = 0
    for ch in s:
        if ch == "(":
            low += 1
            high += 1
        elif ch == ")":
            low -= 1
            high -= 1
        else:
            low -= 1
            high += 1
        if high < 0:
            return False
        if low < 0:
            low = 0
    return low == 0
```

PATTERN 6 · STRING MANIPULATION / PARSING · PROBLEM 2 OF 20

14. Longest Common Prefix

Easy · Trie, Array, String

leetcode.com/problems/longest-common-prefix

PROBLEM

Write a function to find the longest common prefix string amongst an array of strings.

If there is no common prefix, return an empty string `""`.

Example 1:

```
Input: strs = ["flower","flow","flight"]
Output: "fl"
```

Example 2:

```
Input: strs = ["dog","racecar","car"]
Output: ""
Explanation: There is no common prefix among the input strings.
```

Constraints:

- `1 <= strs.length <= 200`
- `0 <= strs[i].length <= 200`
- `strs[i]` consists of only lowercase English letters if it is non-empty.

SOLUTION (PYTHON 3)

Solution 1: Character Comparison

How to think about it

The first idea is to take the first string as the prefix and shrink it against each later string. n,m≤ 200, so even a triple loop would pass. A trie works too, but it is heavier than this problem needs. The prefix only ever shrinks: once a column disagrees, a longer prefix cannot exist. Align the strings vertically; position i can extend the prefix only if every string matches there. So we compare column by column against strs[0] and return as soon as a string is too short or a character differs. No trie.

We use the first string strs[0] as a benchmark, and compare whether the i-th character of the subsequent strings is the same as the i-th character of strs[0]. If they are the same, we continue to compare the next character. Otherwise, we return the first i characters of strs[0].

If the traversal ends, it means that the first i characters of all strings are the same, and we return strs[0].

The time complexity is O(n × m), where n and m are the length of the string array and the minimum length of the strings, respectively. The space complexity is O(1).

Python 3

```python
class Solution:
    def longestCommonPrefix(self, strs: List[str]) -> str:
        for i in range(len(strs[0])):
            for s in strs[1:]:
                if len(s) <= i or s[i] != strs[0][i]:
                    return s[:i]
        return strs[0]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The longest common SUFFIX
# 2. The longest prefix shared by at least K of the words


# ============================================================
# FOLLOW-UP 1: The longest common SUFFIX
# ============================================================

# ---- follow-up 1: function ----
# Answer: Compare the words from their ends; or reverse
# every word, find the common prefix, and reverse the
# answer.
# Time: O(S), S = total number of characters.  Space:
# O(L), L = length of the suffix.
def longest_common_suffix(words):
    if not words:
        return ""
    suffix = ""
    shortest = len(words[0])
    for word in words:
        if len(word) < shortest:
            shortest = len(word)
    for offset in range(1, shortest + 1):
        ch = words[0][-offset]
        for word in words:
            if word[-offset] != ch:
                return suffix
        suffix = ch + suffix
    return suffix


# ============================================================
# FOLLOW-UP 2: The longest prefix shared by at least K of
# the words
# ============================================================

# ---- follow-up 2: function ----
# Answer: Sort the words: words with a long common prefix
# are neighbours. For every block of K consecutive words,
# the common prefix of the whole block is the common
# prefix of its first and last word.
# Time: O(N * L * log N) (sorting the words).  Space: O(1)
# extra.
def common_prefix_of_two(a, b):
    i = 0
    while i < len(a) and i < len(b) and a[i] == b[i]:
        i += 1
    return a[:i]


def longest_prefix_shared_by_k(words, k):
    words = sorted(words)
    best = ""
    for start in range(len(words) - k + 1):
        prefix = common_prefix_of_two(words[start], words[start + k - 1])
        if len(prefix) > len(best):
            best = prefix
    return best
```

PATTERN 6 · STRING MANIPULATION / PARSING · PROBLEM 3 OF 20

459. Repeated Substring Pattern

Easy · String, String Matching, KMP, Extended KMP

leetcode.com/problems/repeated-substring-pattern

Also appears in pattern 7 String Matching / Pattern Search

PROBLEM

Given a string `s`, check if it can be constructed by taking a substring of it and appending multiple copies of the substring together.

Example 1:

```
Input: s = "abab"
Output: true
Explanation: It is the substring "ab" twice.
```

Example 2:

```
Input: s = "aba"
Output: false
```

Example 3:

```
Input: s = "abcabcabcabc"
Output: true
Explanation: It is the substring "abc" four times or the substring "abcabc" twice.
```

Constraints:

- `1 <= s.length <= 10^4`
- `s` consists of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

We ask whether s is a proper prefix repeated. Trying every prefix length is O(n^2). Build s+s and search for s starting at index 1. A hit before n means s lines up inside the concatenation, hence a period exists. Starting at 1 skips the trivial match at 0; a hit at n is only the middle copy, so there is no smaller period.

Python 3

```python
class Solution:
    def repeatedSubstringPattern(self, s: str) -> bool:
        return (s + s).index(s, 1) < len(s)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the smallest repeating unit
# 2. How many times is the unit repeated?


# ========================================================
# FOLLOW-UP 1: Return the smallest repeating unit
# ========================================================
```

```python
# ---- follow-up 1: helper functions ----
def prefix_function(p):
    border = [0] * len(p)
    k = 0
    for i in range(1, len(p)):
        while k > 0 and p[i] != p[k]:
            k = border[k - 1]
        if p[i] == p[k]:
            k += 1
        border[i] = k
    return border

# ---- follow-up 1: function ----
# Answer: The prefix function tells the longest border (a
# prefix that is also a suffix). The shortest period is
# length - border, and the string is a repetition only
# when that period divides the length.
# Time: O(n).  Space: O(n).
def smallest_repeating_unit(s):
    if not s:
        return s
    period = len(s) - prefix_function(s)[-1]
    if len(s) % period == 0:
        return s[:period]
    return s


# ============================================================
# FOLLOW-UP 2: How many times is the unit repeated?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Divide the length by the length of the smallest
# repeating unit.
# Time: O(n).  Space: O(n).
def repeat_count(s):
    if not s:
        return 0
    return len(s) // len(smallest_repeating_unit(s))
```

PATTERN 6 · STRING MANIPULATION / PARSING · PROBLEM 4 OF 20

28. Find the Index of the First Occurrence in a String

Easy · Two Pointers, String, String Matching, KMP, Boyer–Moore, Extended KMP

leetcode.com/problems/find-the-index-of-the-first-occurrence-in-a-string

Also appears in pattern 7 String Matching / Pattern Search

PROBLEM

Given two strings `needle` and `haystack`, return the index of the first occurrence of `needle` in `haystack`, or `-1` if `needle` is not part of `haystack`.

Example 1:

```
Input: haystack = "sadbutsad", needle = "sad"
Output: 0
Explanation: "sad" occurs at index 0 and 6.
The first occurrence is at index 0, so we return 0.
```

Example 2:

```
Input: haystack = "leetcode", needle = "leeto"
Output: -1
Explanation: "leeto" did not occur in "leetcode", so we return -1.
```

Constraints:

- `1 <= haystack.length, needle.length <= 10^4`
- `haystack` and `needle` consist of only lowercase English characters.

SOLUTIONS (PYTHON 3)

Solution 1: Traversal

How to think about it

The first idea is to try every start i in haystack and test whether the length-m slice equals needle. With n,m ≤ 10^4, the worst case O((n-m)m) is about 10^8 and usually passes. The bottleneck is paying m character comparisons at many near-matches. Smarter matchers amortize toward linear time, but this size does not force them yet. We only need the first hit; a mismatch just moves on to the next i. So scan i from 0 to n-m, return i on equality, else -1. Extra space is O(1).

We compare the string `needle` with each character of the string `haystack` as the starting point. If we find a matching index, we return it directly.

Assuming the length of the string `haystack` is n and the length of the string `needle` is m, the time complexity is O((n-m) × m), and the space complexity is O(1).

Python 3

```python
class Solution:
    def strStr(self, haystack: str, needle: str) -> int:
        n, m = len(haystack), len(needle)
        for i in range(n - m + 1):
            if haystack[i : i + m] == needle:
                return i
        return -1
```

Solution 2: Rabin-Karp String Matching Algorithm

> **How to think about it**
>
> Solution 1 may compare all m characters at every start, approaching O(nm). We want the next window to reuse work from the last one. A fixed-length substring can be rolling-hashed: add the incoming character, drop the outgoing one, update in O(1). When the window hash equals needle's hash, compare the raw strings to rule out a collision. One scan of haystack then costs expected O(n+m).

The Rabin-Karp algorithm essentially uses a sliding window combined with a hash function to compare the hashes of fixed-length strings, which can reduce the time complexity of comparing whether two strings are the same to O(1).

Assuming the length of the string `haystack` is n and the length of the string `needle` is m, the time complexity is O(n+m), and the space complexity is O(1).

Python 3

```python
class Solution:
    def strStr(self, haystack: str, needle: str) -> int:
        n, m = len(haystack), len(needle)
        mod = (1 << 31) - 1
        target = sha = 0
        multi = 1
        for i in range(m):
            target = (target * 256 + ord(needle[i])) % mod
        for _ in range(1, m):
            multi = multi * 256 % mod
        left = 0
        for right in range(n):
            sha = (sha * 256 + ord(haystack[right])) % mod
            if right - left + 1 < m:
                continue
            if sha == target and haystack[left : right + 1] == needle:
                return left
            sha = (sha - ord(haystack[left]) * multi % mod + mod) % mod
            left += 1
        return -1
```

Solution 3: KMP

> **How to think about it**
>
> Solution 2 makes a window comparison expected O(1), but it still hashes modulo a prime and must verify the raw strings on a collision. We want a worst-case linear scan without hashing. After a mismatch we need not rewind haystack to the start of the window. The prefix function of needle stores the longest proper border of the matched prefix, so we know where in the pattern to resume. Build next for needle, then scan haystack once, falling back only along next. Time O(n+m) and extra space O(m).

Compute the prefix function next of needle, where next[i] is the longest proper border of needle[0..i]. Scan haystack: equal characters grow the match length, and a mismatch jumps it to next[j-1]. When the match length reaches m, return the start index i-m+1.

The time complexity is O(n+m) and the space complexity is O(m), where n and m are the lengths of haystack and needle.

Python 3

```python
class Solution:
    def strStr(self, haystack: str, needle: str) -> int:
        n, m = len(haystack), len(needle)
        nxt = [0] * m
        j = 0
        for i in range(1, m):
            while j and needle[i] != needle[j]:
                j = nxt[j - 1]
            if needle[i] == needle[j]:
                j += 1
            nxt[i] = j
        j = 0
        for i, ch in enumerate(haystack):
            while j and ch != needle[j]:
                j = nxt[j - 1]
            if ch == needle[j]:
                j += 1
            if j == m:
                return i - m + 1
        return -1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Find EVERY occurrence in O(n + m) (KMP)
# 2. A rolling hash (Rabin-Karp)


# ============================================================
# FOLLOW-UP 1: Find EVERY occurrence in O(n + m) (KMP)
# ============================================================

# ---- follow-up 1: helper functions ----
def prefix_function(p):
    border = [0] * len(p)
    k = 0
    for i in range(1, len(p)):
        while k > 0 and p[i] != p[k]:
            k = border[k - 1]
        if p[i] == p[k]:
            k += 1
        border[i] = k
    return border

# ---- follow-up 1: function ----
# Answer: Build the prefix function of the needle once.
# While scanning the haystack, a mismatch falls back to
# the longest border instead of restarting, so no
# character of the haystack is read twice.
# Time: O(n + m).  Space: O(m).
def find_all_occurrences(haystack, needle):
    if needle == "":
        return []
    border = prefix_function(needle)
    positions = []
    k = 0
    for i in range(len(haystack)):
        while k > 0 and haystack[i] != needle[k]:
            k = border[k - 1]
        if haystack[i] == needle[k]:
            k += 1
        if k == len(needle):
            positions.append(i - len(needle) + 1)
            k = border[k - 1]
    return positions
```

```python
# ============================================================
# FOLLOW-UP 2: A rolling hash (Rabin-Karp)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Hash the needle and every window of the haystack
# with a hash that updates in O(1) when the window slides;
# compare the characters only when the hashes are equal.
# Time: O(n + m) on average, O(n * m) in the worst case.
# Space: O(1).
def rabin_karp(haystack, needle):
    n, m = len(haystack), len(needle)
    if m == 0:
        return 0
    if m > n:
        return -1
```

```python
    base, mod = 256, 1000000007
    high = pow(base, m - 1, mod)                    # the weight of the leftmost character
    target = 0
    window = 0
    for i in range(m):
        target = (target * base + ord(needle[i])) % mod
        window = (window * base + ord(haystack[i])) % mod
    for start in range(n - m + 1):
        if window == target and haystack[start:start + m] == needle:
            return start
        if start + m < n:
            window = (window - ord(haystack[start]) * high) % mod
            window = (window * base + ord(haystack[start + m])) % mod
    return -1
```

PATTERN 6 · STRING MANIPULATION / PARSING · PROBLEM 5 OF 20

8. String to Integer (atoi)

Medium · String

leetcode.com/problems/string-to-integer-atoi

★ **TOP 7 PROBLEM** in String Manipulation / Parsing: master this one first

PROBLEM

Implement the `myAtoi(string s)` function, which converts a string to a 32-bit signed integer.

The algorithm for `myAtoi(string s)` is as follows:

1. **Whitespace**: Ignore any leading whitespace (`" "`).
2. **Signedness**: Determine the sign by checking if the next character is `'-'` or `'+'`, assuming positivity if neither present.
3. **Conversion**: Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.
4. **Rounding**: If the integer is out of the 32-bit signed integer range $[-2^{31}, 2^{31} - 1]$, then round the integer to remain in the range. Specifically, integers less than -2^{31} should be rounded to -2^{31}, and integers greater than $2^{31} - 1$ should be rounded to $2^{31} - 1$.

Return the integer as the final result.

Note: You must not use any built-in library function that converts a string to a number (for example, `atoi`, `stoi`, `Integer.parseInt`, `int()`, `parseInt`, `Number()`). Perform the conversion manually.

Example 1:

Input: s = "42"

Output: 42

Explanation:

```
The underlined characters are what is read in and the caret is the current reader position.
Step 1: "42" (no characters read because there is no leading whitespace)
         ^
Step 2: "42" (no characters read because there is neither a '-' nor '+')
         ^
Step 3: "42" ("42" is read in)
           ^
```

Example 2:

Input: s = " -042"

Output: -42

Explanation:

```
Step 1: " -042" (leading whitespace is read and ignored)
            ^
Step 2: " -042" ('-' is read, so the result should be negative)
             ^
Step 3: " -042" ("042" is read in, leading zeros ignored in the result)
               ^
```

Example 3:

Input: s = "1337c0d3"

Output: 1337

Explanation:

```
Step 1: "1337c0d3" (no characters read because there is no leading whitespace)
         ^
Step 2: "1337c0d3" (no characters read because there is neither a '-' nor '+')
         ^
Step 3: "1337c0d3" ("1337" is read in; reading stops because the next character is a non-digit)
            ^
```

Example 4:

Input: s = "0-1"

Output: 0

Explanation:

```
Step 1: "0-1" (no characters read because there is no leading whitespace)
         ^
Step 2: "0-1" (no characters read because there is neither a '-' nor '+')
         ^
Step 3: "0-1" ("0" is read in; reading stops because the next character is a non-digit)
          ^
```

Example 5:

Input: s = "words and 987"

Output: 0

Explanation:

Reading stops at the first non-digit character 'w'.

Constraints:

- `0 <= s.length <= 200`
- `s` consists of English letters (lower-case and upper-case), digits (`0 - 9`), `' '`, `'+'`, `'-'`, and `'.'`.

SOLUTION (PYTHON 3)

Solution 1: Traverse the String

How to think about it

A single left-to-right scan is enough; n ≤ 200 is not a complexity issue. The easy mistakes are the order of rules: skip leading spaces, consume at most one sign, then take consecutive digits, stop at the first non-digit, and clamp to 32 bits. Without 64-bit integers we cannot finish the product and clip later. We accumulate the absolute value, comparing against floor((2^{31}-1)/10) before multiplying by 10, and saturate to 2^{31}-1 or -2^{31} on overflow. Both signs share that upper bound; the sign is applied at the end. An empty string or all spaces has no digits, so we return 0.

First, we determine whether the string is empty. If it is, we directly return 0.

Otherwise, we need to traverse the string, skip the leading spaces, and determine whether the first non-space character is a positive or negative sign.

Then we traverse the following characters. If it is a digit, we judge whether adding this digit will cause integer overflow. If it does, we return the result according to the positive or negative sign. Otherwise, we add the digit to the result. We continue to traverse the following characters until we encounter a non-digit character or the traversal ends.

After the traversal ends, we return the result according to the positive or negative sign.

The time complexity is O(n), where n is the length of the string. We only need to process all characters in turn. The space complexity is O(1).

Python 3

```
class Solution:
    def myAtoi(self, s: str) -> int:
        if not s:
            return 0
        n = len(s)
        if n == 0:
            return 0
        i = 0
        while s[i] == ' ':
            i += 1
            if i == n:
                return 0
        sign = -1 if s[i] == '-' else 1
        if s[i] in ['-', '+']:
            i += 1
        res, flag = 0, (2**31 - 1) // 10
        while i < n:
            if not s[i].isdigit():
                break
            c = int(s[i])
            if res > flag or (res == flag and c > 7):
                return 2**31 - 1 if sign > 0 else -(2**31)
            res = res * 10 + c
            i += 1
        return sign * res
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Parse a number in a given base (2 to 36)
# 2. Parse a decimal number with a fractional part (like
# -12.75)


# ============================================================
# FOLLOW-UP 1: Parse a number in a given base (2 to 36)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same scan, but a digit is worth its position in
# '0123456789abcdefghijklmnopqrstuvwxyz'; stop at the
# first character that is not a digit of that base.
# Time: O(n).  Space: O(1).
def parse_in_base(s, base):
    digits = "0123456789abcdefghijklmnopqrstuvwxyz"
    i = 0
    while i < len(s) and s[i] == " ":
        i += 1
    sign = 1
    if i < len(s) and s[i] in "+-":
        if s[i] == "-":
            sign = -1
        i += 1
    value = 0
    while i < len(s):
        position = digits.find(s[i].lower())
        if position == -1 or position >= base:
            break
        value = value * base + position
        i += 1
    return sign * value


# ============================================================
# FOLLOW-UP 2: Parse a decimal number with a fractional
```

```
# part (like -12.75)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Read the whole part as before, then after a '.'
# read the digits and add each one divided by ten to the
# power of its position.
# Time: O(n).  Space: O(1).
def parse_decimal(s):
    i = 0
    while i < len(s) and s[i] == " ":
        i += 1
    sign = 1
    if i < len(s) and s[i] in "+-":
        if s[i] == "-":
            sign = -1
        i += 1
    whole = 0
    while i < len(s) and s[i].isdigit():
        whole = whole * 10 + (ord(s[i]) - ord("0"))
        i += 1
    fraction = 0.0
    scale = 0.1
```

```
    if i < len(s) and s[i] == ".":
        i += 1
        while i < len(s) and s[i].isdigit():
            fraction += (ord(s[i]) - ord("0")) * scale
            scale /= 10
            i += 1
    return sign * (whole + fraction)
```

PATTERN 6 · STRING MANIPULATION / PARSING · PROBLEM 6 OF 20

6. Zigzag Conversion

Medium · String

leetcode.com/problems/zigzag-conversion

PROBLEM

The string "PAYPALISHIRING" is written in a zigzag pattern on a given number of rows like this: (you may want to display this pattern in a fixed font for better legibility)

```
P A H N
A P L S I I G
Y I R
```

And then read line by line: "PAHNAPLSIIGYIR"

Write the code that will take a string and make this conversion given a number of rows:

```
string convert(string s, int numRows);
```

Example 1:

```
Input: s = "PAYPALISHIRING", numRows = 3
Output: "PAHNAPLSIIGYIR"
```

Example 2:

```
Input: s = "PAYPALISHIRING", numRows = 4
Output: "PINALSIGYAHRPI"
Explanation:
P I N
A L S I G
Y A H R
P I
```

Example 3:

```
Input: s = "A", numRows = 1
Output: "A"
```

Constraints:

- `1 <= s.length <= 1000`
- `s` consists of English letters (lower-case and upper-case), `','` and `'.'`.
- `1 <= numRows <= 1000`

SOLUTION (PYTHON 3)

Solution 1: Simulation

How to think about it

Drawing the 2D grid and reading by row is correct, but most cells are empty. n ≤ 10^3 would pass, yet the unused "column" dimension is wasted. Only the row of each character, and order within a row, affect the answer. The row index goes 0,1,…,numRows-1 and then back, reversing at the top or bottom. So we keep numRows lists and flip a direction k at the boundaries. k starts at -1, so the first row immediately turns it downward. When numRows=1, top and bottom are the same row; we must return s as is, or the direction would bounce in place.

We use a 2D array g to simulate the process of arranging the string in a zigzag pattern, where g[i][j] represents the character at row i and column j. Initially, i = 0. We also define a direction variable k, initially k = -1, which means moving

upwards.

We traverse the string s from left to right. For each character c, we append it to g[i]. If i = 0 or i = numRows - 1, it means the current character is at a turning point in the zigzag pattern, so we reverse the value of k, i.e., k = -k. Then, we update i to i + k, which means moving up or down one row. Continue traversing the next character until the end of the string s. Finally, we return the concatenation of all rows in g as the result.

The time complexity is O(n) and the space complexity is O(n), where n is the length of the string s.

Python 3

```python
class Solution:
    def convert(self, s: str, numRows: int) -> str:
        if numRows == 1:
            return s
        g = [[] for _ in range(numRows)]
        i, k = 0, -1
        for c in s:
            g[i].append(c)
            if i == 0 or i == numRows - 1:
                k = -k
            i += k
        return ''.join(chain(*g))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Convert back: from the zigzag reading order to the
# original text


# ============================================================
# FOLLOW-UP 1: Convert back: from the zigzag reading order
# to the original text
# ============================================================

# ---- follow-up 1: function ----
# Answer: Work out which row every original position falls
# in (the row number bounces between 0 and rows - 1),
# count the characters per row, cut the zigzag string into
# row pieces of those sizes, and read the rows in the
# bouncing order.
# Time: O(n).  Space: O(n).
def zigzag_row_of_each_position(length, rows):
    row_of = []
    row = 0
    step = 1
    for i in range(length):
        row_of.append(row)
        if rows > 1:
            if row == 0:
                step = 1
            elif row == rows - 1:
                step = -1
            row += step
    return row_of


def decode_zigzag(zigzag, rows):
    row_of = zigzag_row_of_each_position(len(zigzag), rows)
    sizes = [0] * rows
    for row in row_of:
        sizes[row] += 1
    pieces = []
    start = 0
    for size in sizes:
        pieces.append(zigzag[start:start + size])
```

```
        start += size
    used = [0] * rows
    original = []
    for row in row_of:
        original.append(pieces[row][used[row]])
        used[row] += 1
    return "".join(original)
```

43. Multiply Strings

Medium · Math, String, Simulation

leetcode.com/problems/multiply-strings

★ **TOP 7 PROBLEM** in String Manipulation / Parsing: master this one first

PROBLEM

Given two non-negative integers `num1` and `num2` represented as strings, return the product of `num1` and `num2`, also represented as a string.

Note: You must not use any built-in BigInteger library or convert the inputs to integer directly.

Example 1:

```
Input: num1 = "2", num2 = "3"
Output: "6"
```

Example 2:

```
Input: num1 = "123", num2 = "456"
Output: "56088"
```

Constraints:

- `1 <= num1.length, num2.length <= 200`
- `num1` and `num2` consist of digits only.
- Both `num1` and `num2` do not contain any leading zero, except the number `0` itself.

SOLUTION (PYTHON 3)

Solution 1: Simulating Mathematical Multiplication

How to think about it

The first idea is convert both strings to integers and multiply. Lengths go to 200, so native integers overflow; the problem also forbids converting directly. The bottleneck is big-integer multiplication. Grade-school arithmetic says the product of digit i and digit j contributes to a fixed place; carries can wait. The product is at most m+n digits, so an array of that length is the scratch paper: accumulate, then carry. That is cheaper than a full bigint library.

Assume the lengths of num1 and num2 are m and n respectively, then the length of their product can be at most m + n.

The proof is as follows:

- If num1 and num2 both take the minimum value, then their product is $10^{m-1} \times 10^{n-1} = 10^{m+n-2}$, with a length of m + n - 1.
- If num1 and num2 both take the maximum value, then their product is $(10^m - 1) \times (10^n - 1) = 10^{m+n} - 10^m - 10^n + 1$, with a length of m + n.

Therefore, we can apply for an array of length m + n to store each digit of the product.

From the least significant digit to the most significant digit, we calculate each digit of the product in turn, and finally convert the array into a string.

Note to check whether the most significant digit is 0, if it is, remove it.

The time complexity is O(m × n), and the space complexity is O(m + n). Here, m and n are the lengths of num1 and num2 respectively.

Python 3

```
class Solution:
    def multiply(self, num1: str, num2: str) -> str:
        if num1 == "0" or num2 == "0":
            return "0"
        m, n = len(num1), len(num2)
        arr = [0] * (m + n)
        for i in range(m - 1, -1, -1):
            a = int(num1[i])
            for j in range(n - 1, -1, -1):
                b = int(num2[j])
                arr[i + j + 1] += a * b
        for i in range(m + n - 1, 0, -1):
            arr[i - 1] += arr[i] // 10
            arr[i] %= 10
        i = 0 if arr[0] else 1
        return "".join(str(x) for x in arr[i:])
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Add two non-negative numbers given as strings
# 2. Divide a big number (a string) by a small integer and
# return quotient and remainder


# ============================================================
# FOLLOW-UP 1: Add two non-negative numbers given as
# strings
# ============================================================

# ---- follow-up 1: function ----
# Answer: Add digit by digit from the right with a carry,
# exactly like on paper.
# Time: O(max(m, n)).  Space: O(max(m, n)).
def add_strings(a, b):
    i, j = len(a) - 1, len(b) - 1
    carry = 0
    digits = []
    while i >= 0 or j >= 0 or carry:
        total = carry
        if i >= 0:
            total += ord(a[i]) - ord("0")
            i -= 1
        if j >= 0:
            total += ord(b[j]) - ord("0")
            j -= 1
        digits.append(str(total % 10))
        carry = total // 10
    digits.reverse()
    return "".join(digits)


# ============================================================
# FOLLOW-UP 2: Divide a big number (a string) by a small
# integer and return quotient and remainder
# ============================================================

# ---- follow-up 2: function ----
# Answer: Long division: bring down one digit at a time,
# the quotient digit is the current value divided by the
# divisor, and the rest carries into the next step.
# Time: O(n).  Space: O(n).
def divide_string_by_int(number, divisor):
    remainder = 0
    quotient = []
```

```
    for ch in number:
        remainder = remainder * 10 + (ord(ch) - ord("0"))
        quotient.append(str(remainder // divisor))
        remainder %= divisor
    text = "".join(quotient).lstrip("0")
    if text == "":
        text = "0"
    return text, remainder
```

PATTERN 6 · STRING MANIPULATION / PARSING · PROBLEM 9 OF 20

165. Compare Version Numbers

Medium · Two Pointers, String

leetcode.com/problems/compare-version-numbers

PROBLEM

Given two **version strings**, `version1` and `version2`, compare them. A version string consists of **revisions** separated by dots `'.'`. The **value of the revision** is its **integer conversion** ignoring leading zeros.

To compare version strings, compare their revision values in **left-to-right order**. If one of the version strings has fewer revisions, treat the missing revision values as `0`.

Return the following:

- If `version1 < version2`, return -1.
- If `version1 > version2`, return 1.
- Otherwise, return 0.

Example 1:

Input: version1 = "1.2", version2 = "1.10"

Output: -1

Explanation:

version1's second revision is "2" and version2's second revision is "10": 2 < 10, so version1 < version2.

Example 2:

Input: version1 = "1.01", version2 = "1.001"

Output: 0

Explanation:

Ignoring leading zeroes, both "01" and "001" represent the same integer "1".

Example 3:

Input: version1 = "1.0", version2 = "1.0.0.0"

Output: 0

Explanation:

version1 has less revisions, which means every missing revision are treated as "0".

Constraints:

- `1 <= version1.length, version2.length <= 500`
- `version1` and `version2` only contain digits and `'.'`.
- `version1` and `version2` **are valid version numbers**.
- All the given revisions in `version1` and `version2` can be stored in a **32-bit integer**.

SOLUTION (PYTHON 3)

Solution 1: Two Pointers

How to think about it

Compare revision numbers; missing parts are 0 and leading zeros do not count. Splitting into integer arrays uses extra space. Lengths are at most 500. Two pointers scan together, finish a part at each dot, and treat a missing side as 0 until they differ or both end.

Traverse both strings simultaneously using two pointers i and j, which point to the current positions in each string, starting with i = j = 0.

Each time, extract the corresponding revision numbers from both strings, denoted as a and b. Compare a and b: if a < b, return -1; if a > b, return 1; if a = b, continue to compare the next pair of revision numbers.

The time complexity is O(max(m, n)), and the space complexity is O(1), where m and n are the lengths of the two strings.

Python 3

```python
class Solution:
    def compareVersion(self, version1: str, version2: str) -> int:
        m, n = len(version1), len(version2)
        i = j = 0
        while i < m or j < n:
            a = b = 0
            while i < m and version1[i] != '.':
                a = a * 10 + int(version1[i])
                i += 1
            while j < n and version2[j] != '.':
                b = b * 10 + int(version2[j])
                j += 1
            if a != b:
                return -1 if a < b else 1
            i, j = i + 1, j + 1
        return 0
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Find the newest version in a list
# 2. Is a version inside the range [lowest, highest]?


# ============================================================
# FOLLOW-UP 1: Find the newest version in a list
# ============================================================

# ---- follow-up 1: helper functions ----
def compare_versions(a, b):
    x = a.split(".")
    y = b.split(".")
    for i in range(max_len(x, y)):
        p = int(x[i]) if i < len(x) else 0
        q = int(y[i]) if i < len(y) else 0
        if p != q:
            return 1 if p > q else -1
    return 0


def max_len(x, y):
    if len(x) > len(y):
        return len(x)
    return len(y)

# ---- follow-up 1: function ----
# Answer: Compare two versions with the same function as
# the main question and keep the larger one.
# Time: O(V * L), V versions of length L.  Space: O(L).
def newest_version(versions):
    best = versions[0]
    for v in versions:
        if compare_versions(v, best) > 0:
            best = v
    return best
```

```
# ============================================================
# FOLLOW-UP 2: Is a version inside the range [lowest,
# highest]?
# ============================================================

# ---- follow-up 2: function ----
# Answer: It is when it is not older than the lowest and
# not newer than the highest.
# Time: O(L).  Space: O(L).
def version_in_range(version, lowest, highest):
    return compare_versions(version, lowest) >= 0 and compare_versions(version, highest) <= 0
```

PATTERN 6 · STRING MANIPULATION / PARSING · PROBLEM 10 OF 20

91. Decode Ways

Medium · String, Dynamic Programming

leetcode.com/problems/decode-ways

★ **TOP 7 PROBLEM** in Dynamic Programming - 1D: master this one first

Also appears in pattern 13 Dynamic Programming - 1D

PROBLEM

You have intercepted a secret message encoded as a string of numbers. The message is **decoded** via the following mapping:

```
"1" -> 'A'
"2" -> 'B'
...
"25" -> 'Y'
"26" -> 'Z'
```

However, while decoding the message, you realize that there are many different ways you can decode the message because some codes are contained in other codes (`"2"` and `"5"` vs `"25"`).

For example, `"11106"` can be decoded into:

- `"AAJF"` with the grouping `(1, 1, 10, 6)`
- `"KJF"` with the grouping `(11, 10, 6)`
- The grouping `(1, 11, 06)` is invalid because `"06"` is not a valid code (only `"6"` is valid).

Note: there may be strings that are impossible to decode.

Given a string s containing only digits, return the **number of ways** to **decode** it. If the entire string cannot be decoded in any valid way, return `0`.

The test cases are generated so that the answer fits in a **32-bit** integer.

Example 1:

Input: s = "12"

Output: 2

Explanation:

"12" could be decoded as "AB" (1 2) or "L" (12).

Example 2:

Input: s = "226"

Output: 3

Explanation:

"226" could be decoded as "BZ" (2 26), "VF" (22 6), or "BBF" (2 2 6).

Example 3:

Input: s = "06"

Output: 0

Explanation:

"06" cannot be mapped to "F" because of the leading zero ("6" is different from "06"). In this case, the string is not a valid encoding, so return 0.

Constraints:

- `1 <= s.length <= 100`
- `s` contains only digits and may contain leading zero(s).

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming

> **How to think about it**
>
> The first idea is recursion: at index i, decode s[i] alone (if not 0), or pair it with the previous digit into 10– 26. n ≤ 100, but without memoization the tree is exponential because the same prefix is decoded again and again. The bottleneck is that the number of ways to decode the first i characters depends only on shorter prefixes, and a valid split is only “one digit / two digits”. That is climbing stairs with encoding constraints. So let f[i] be the number of ways for the first i characters. The empty string has one way; 0 cannot stand alone; only 10– 26 can be a pair. Scan once instead of enumerating every partition.

We define f[i] to represent the number of decoding methods for the first i characters of the string. Initially, f[0]=1, and the rest f[i]=0.

Consider how f[i] transitions.

- If the ith character (i.e., s[i-1]) forms a code on its own, it corresponds to one decoding method, i.e., f[i]=f[i-1]. The premise is s[i-1] ≠ 0.
- If the string formed by the i-1th character and the ith character is within the range [1,26], then they can be treated as a whole, corresponding to one decoding method, i.e., f[i] = f[i] + f[i-2]. The premise is s[i-2] ≠ 0, and s[i-2]s[i-1] is within the range [1,26].

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the string.

Python 3

```python
class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        f = [1] + [0] * n
        for i, c in enumerate(s, 1):
            if c != "0":
                f[i] = f[i - 1]
            if i > 1 and s[i - 2] != "0" and int(s[i - 2 : i]) <= 26:
                f[i] += f[i - 2]
        return f[n]
```

Solution 2: Optimized Dynamic Programming

> **How to think about it**
>
> Solution 1 is already correct. Each f[i] only needs f[i-1] and f[i-2], so the full array is unnecessary. Two rolling variables cut space to O(1); time is still a single pass.

We notice that the state f[i] is only related to f[i-1] and f[i-2]. Therefore, we can use two variables to replace these states, reducing the space complexity from O(n) to O(1). The time complexity remains O(n).

Python 3

```python
class Solution:
    def numDecodings(self, s: str) -> int:
        f, g = 0, 1
        for i, c in enumerate(s, 1):
            h = g if c != "0" else 0
            if i > 1 and s[i - 2] != "0" and int(s[i - 2 : i]) <= 26:
                h += f
            f, g = g, h
        return g
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. List every decoding instead of counting them
# 2. The string may contain '*' that stands for any digit
# 1 to 9 (count modulo 10^9 + 7)


# ============================================================
# FOLLOW-UP 1: List every decoding instead of counting
# them
# ============================================================

# ---- follow-up 1: function ----
# Answer: Backtracking: at each position take one digit
# (1-9) or two digits (10-26) and move on; the path of
# letters at the end of the string is one decoding.
# Time: exponential in the worst case (the number of
# decodings), O(n * 2^n).  Space: O(n) recursion plus the
# results.
def all_decodings(s):
    results = []

    def go(i, path):
        if i == len(s):
            results.append("".join(path))
            return
        if s[i] == "0":
            return
        path.append(chr(ord("A") + int(s[i]) - 1))
        go(i + 1, path)
        path.pop()
        if i + 1 < len(s) and 10 <= int(s[i:i + 2]) <= 26:
            path.append(chr(ord("A") + int(s[i:i + 2]) - 1))
            go(i + 2, path)
            path.pop()

    go(0, [])
    return results


# ============================================================
# FOLLOW-UP 2: The string may contain '*' that stands for
# any digit 1 to 9 (count modulo 10^9 + 7)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same table, but a single '*' gives 9 choices,
# and a pair involving '*' gives a fixed number of
# choices: '**' = 15, '1*' = 9, '2*' = 6, '*d' = 2 when d
# is at most 6 and 1 otherwise.
# Time: O(n).  Space: O(n).
def num_decodings_with_star(s):
    mod = 10 ** 9 + 7
    n = len(s)
    ways = [0] * (n + 1)
    ways[0] = 1
    for i in range(1, n + 1):
        c = s[i - 1]
        if c == "*":
            single = 9
        elif c != "0":
            single = 1
        else:
            single = 0
        total = single * ways[i - 1]
```

```
        if i >= 2:
            p = s[i - 2]
            if p == "*" and c == "*":
                pair = 15
            elif p == "*":
                pair = 2 if c <= "6" else 1
            elif p == "1":
                pair = 9 if c == "*" else 1
            elif p == "2":
                if c == "*":
                    pair = 6
                else:
                    pair = 1 if c <= "6" else 0
            else:
                pair = 0
            total += pair * ways[i - 2]
        ways[i] = total % mod
    return ways[n]
```

71. Simplify Path

Medium · Stack, String

leetcode.com/problems/simplify-path

★ **TOP 7 PROBLEM** in String Manipulation / Parsing: master this one first

Also appears in pattern 8 Stack

PROBLEM

You are given an *absolute* path for a Unix-style file system, which always begins with a slash `'/'`. Your task is to transform this absolute path into its **simplified canonical path**.

The *rules* of a Unix-style file system are as follows:

- A single period `'.'` represents the current directory.
- A double period `'..'` represents the previous/parent directory.
- Multiple consecutive slashes such as `'//'` and `'///'` are treated as a single slash `'/'`.
- Any sequence of periods that does **not match** the rules above should be treated as a **valid directory or file name**. For example, `'...'` and `'....'` are valid directory or file names.

The simplified canonical path should follow these *rules*:

- The path must start with a single slash `'/'`.
- Directories within the path must be separated by exactly one slash `'/'`.
- The path must not end with a slash `'/'`, unless it is the root directory.
- The path must not have any single or double periods (`'.'` and `'..'`) used to denote current or parent directories.

Return the **simplified canonical path**.

Example 1:

Input: path = "/home/"

Output: "/home"

Explanation:

The trailing slash should be removed.

Example 2:

Input: path = "/home//foo/"

Output: "/home/foo"

Explanation:

Multiple consecutive slashes are replaced by a single one.

Example 3:

Input: path = "/home/user/Documents/../Pictures"

Output: "/home/user/Pictures"

Explanation:

A double period `".."` refers to the directory up a level (the parent directory).

Example 4:

Input: path = "/../"

Output: "/"

Explanation:

Going one level up from the root directory is not possible.

Example 5:

Input: path = "/.../a/../b/c/../d/./"

Output: "/.../b/d"

Explanation:

`"..."` is a valid name for a directory in this problem.

Constraints:

- `1 <= path.length <= 3000`
- `path` consists of English letters, digits, period `'.'`, slash `'/'` or `'_'`.
- `path` is a valid absolute Unix path.

SOLUTION (PYTHON 3)

Solution 1: Stack

> **How to think about it**
>
> The first idea is repeated string replace of `'//'`, `'/./'`, and `'/../'`. n ≤ 3000 might pass, but stacked slashes and `'..'` interact, so the replace order is hard to get right. What we need is undo: a valid name must be erasable by `'..'`, while empty segments and `'.'` do not move us. That is last-in, first-out, so a stack holds the directories from the root to here. Split on `'/'`, process each piece, then join into the canonical path. We must not pop above the root, so the stack is checked before popping.

We first split the path into a number of substrings split by `'/'`. Then, we traverse each substring and perform the following operations based on the content of the substring:

- If the substring is empty or `'.'`, no operation is performed because `'.'` represents the current directory.
- If the substring is `'..'`, the top element of the stack is popped, because `'..'` represents the parent directory.
- If the substring is other strings, the substring is pushed into the stack, because the substring represents the subdirectory of the current directory.

Finally, we concatenate all the elements in the stack from the bottom to the top of the stack to form a string, which is the simplified canonical path.

The time complexity is O(n) and the space complexity is O(n), where n is the length of the path.

Python 3

```python
class Solution:
    def simplifyPath(self, path: str) -> str:
        stk = []
        for s in path.split('/'):
            if not s or s == '.':
                continue
            if s == '..':
                if stk:
                    stk.pop()
            else:
                stk.append(s)
        return '/' + '/'.join(stk)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Resolve a relative path against a current directory
# 2. The relative path that leads from one directory to
# another


# ============================================================
# FOLLOW-UP 1: Resolve a relative path against a current
# directory
# ============================================================

# ---- follow-up 1: function ----
# Answer: Start the stack from the parts of the current
# directory and apply '.' and '..' on top of it; an
# absolute path ignores the current directory.
# Time: O(L).  Space: O(L).
def resolve_path(current, path):
    parts = []
    if not path.startswith("/"):
        for name in current.split("/"):
            if name != "":
                parts.append(name)
    for name in path.split("/"):
        if name == "" or name == ".":
            continue
        if name == "..":
            if parts:
                parts.pop()
        else:
            parts.append(name)
    return "/" + "/".join(parts)


# ============================================================
# FOLLOW-UP 2: The relative path that leads from one
# directory to another
# ============================================================

# ---- follow-up 2: function ----
# Answer: Split both directories into names, drop the
# names they share, then go up once ('..') for every name
# left in the first one and down through the names left in
# the second.
# Time: O(L).  Space: O(L).
def relative_path(source, destination):
    a = []
    for name in source.split("/"):
        if name != "":
            a.append(name)
    b = []
    for name in destination.split("/"):
        if name != "":
            b.append(name)
    common = 0
    while common < len(a) and common < len(b) and a[common] == b[common]:
        common += 1
    steps = [".."] * (len(a) - common) + b[common:]
    if not steps:
        return "."
    return "/".join(steps)
```

PATTERN 6 · STRING MANIPULATION / PARSING · PROBLEM 12 OF 20

139. Word Break

Medium · Trie, Memoization, Array, Hash Table, String, Dynamic Programming, Brute-Force Search

leetcode.com/problems/word-break

★ **TOP 7 PROBLEM** in Dynamic Programming - 1D: master this one first

Also appears in pattern 13 Dynamic Programming - 1D, 32 Trie (Prefix Tree)

PROBLEM

Given a string `s` and a dictionary of strings `wordDict`, return `true` if `s` can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

```
Input: s = "leetcode", wordDict = ["leet","code"]
Output: true
Explanation: Return true because "leetcode" can be segmented as "leet code".
```

Example 2:

```
Input: s = "applepenapple", wordDict = ["apple","pen"]
Output: true
Explanation: Return true because "applepenapple" can be segmented as "apple pen apple".
Note that you are allowed to reuse a dictionary word.
```

Example 3:

```
Input: s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]
Output: false
```

Constraints:

- `1 <= s.length <= 300`
- `1 <= wordDict.length <= 1000`
- `1 <= wordDict[i].length <= 20`
- `s` and `wordDict[i]` consist of only lowercase English letters.
- All the strings of `wordDict` are **unique**.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Can s be segmented into dictionary words? |s|≤ 300; backtracking repeats the same suffixes. f[i] means the prefix s[0..i) can be split: try a cut j where f[j] is true and s[j..i) is in the dictionary. A hash set makes the membership test expected O(1).

Python 3

```python
class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        words = set(wordDict)
        n = len(s)
        f = [True] + [False] * n
        for i in range(1, n + 1):
            f[i] = any(f[j] and s[j:i] in words for j in range(i))
        return f[n]
```

Solution 2

> **How to think about it**
>
> Solution 1 slices a substring and hashes it at every cut, which is O(n) per test. A trie lets us walk from index i along existing edges only, so useless slices disappear and the time becomes $O(n^2)$.

Python 3

```python
class Trie:
    def __init__(self):
        self.children: List[Trie | None] = [None] * 26
        self.isEnd = False

    def insert(self, w: str):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if not node.children[idx]:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.isEnd = True


class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        trie = Trie()
        for w in wordDict:
            trie.insert(w)
        n = len(s)
        f = [False] * (n + 1)
        f[n] = True
        for i in range(n - 1, -1, -1):
            node = trie
            for j in range(i, n):
                idx = ord(s[j]) - ord('a')
                if not node.children[idx]:
                    break
                node = node.children[idx]
                if node.isEnd and f[j + 1]:
                    f[i] = True
                    break
        return f[0]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return every possible sentence (Word Break II)
# 2. The smallest number of dictionary words that spell
# the string
# 3. Count the number of ways to split the string into
# dictionary words
```

```
# ============================================================
# FOLLOW-UP 1: Return every possible sentence (Word Break
# II)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Recursion with a dictionary of results per start
# position: for each start try every dictionary word that
# matches there, and join it with every sentence of the
# rest.
# Time: O(2^n) in the worst case (the number of
# sentences).  Space: O(2^n).
def word_break_sentences(s, word_dict):
    words = set(word_dict)
    memo = {}

    def sentences_from(start):
        if start == len(s):
            return [""]
        if start in memo:
            return memo[start]
        result = []
        for end in range(start + 1, len(s) + 1):
            word = s[start:end]
            if word in words:
                for rest in sentences_from(end):
                    result.append(word if rest == "" else word + " " + rest)
        memo[start] = result
        return result

    return sentences_from(0)


# ============================================================
# FOLLOW-UP 2: The smallest number of dictionary words
# that spell the string
# ============================================================

# ---- follow-up 2: function ----
# Answer: Table over prefixes: fewest[i] is the smallest
# number of words for the first i letters, built from
# every word that ends at i.
# Time: O(n^3) (n^2 substrings, each hashed in O(n)).
# Space: O(n).
def min_words_to_spell(s, word_dict):
    words = set(word_dict)
    big = len(s) + 1
    fewest = [big] * (len(s) + 1)
    fewest[0] = 0
    for end in range(1, len(s) + 1):
        for start in range(end):
            if s[start:end] in words and fewest[start] + 1 < fewest[end]:
                fewest[end] = fewest[start] + 1
    if fewest[len(s)] == big:
        return -1
    return fewest[len(s)]


# ============================================================
# FOLLOW-UP 3: Count the number of ways to split the
# string into dictionary words
# ============================================================

# ---- follow-up 3: function ----
# Answer: The same table, but add the ways instead of
# taking a minimum.
# Time: O(n^3).  Space: O(n).
def count_word_break_ways(s, word_dict):
```

```
    words = set(word_dict)
    ways = [0] * (len(s) + 1)
    ways[0] = 1
    for end in range(1, len(s) + 1):
        for start in range(end):
            if s[start:end] in words:
                ways[end] += ways[start]
    return ways[len(s)]
```

PATTERN 6 · STRING MANIPULATION / PARSING · PROBLEM 13 OF 20

5. Longest Palindromic Substring

Medium · Two Pointers, String, Dynamic Programming, Manacher

leetcode.com/problems/longest-palindromic-substring

★ **TOP 7 PROBLEM** in String Manipulation / Parsing, String Matching / Pattern Search: master this one first

Also appears in pattern 7 String Matching / Pattern Search

PROBLEM

Given a string `s`, return *the longest palindromic substring* in `s`.

Example 1:

```
Input: s = "babad"
Output: "bab"
Explanation: "aba" is also a valid answer.
```

Example 2:

```
Input: s = "cbbd"
Output: "bb"
```

Constraints:

- `1 <= s.length <= 1000`
- `s` consist of only digits and English letters.

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

Checking every s[i..j] from the ends is $O(n^3)$. $n \le 10^3$ is tight and often times out. The waste is rescanning each substring. A palindrome has optimal substructure: s[i..j] is a palindrome iff the two ends match and the inner s[i+1..j-1] already is one. Single characters are palindromes for free. So we fill an interval table in dependency order instead of scanning repeatedly. f[i][j] reads a shorter inner cell, so that cell must already be known; while filling, k and mx remember the start and length of the best palindrome so far.

We define f[i][j] to represent whether the string s[i..j] is a palindrome, initially f[i][j] = true.

Next, we define variables k and mx, where k represents the starting position of the longest palindrome, and mx represents the length of the longest palindrome. Initially, k = 0, mx = 1.

Considering f[i][j], if s[i] = s[j], then f[i][j] = f[i + 1][j - 1]; otherwise, f[i][j] = false. If f[i][j] = true and mx < j - i + 1, then we update k = i, mx = j - i + 1.

Since f[i][j] depends on f[i + 1][j - 1], we need to ensure that i + 1 is before j - 1, so we need to enumerate i from large to small, and enumerate j from small to large.

The time complexity is $O(n^2)$, and the space complexity is $O(n^2)$. Here, n is the length of the string s.

Python 3

```python
class Solution:
    def longestPalindrome(self, s: str) -> str:
        n = len(s)
        f = [[True] * n for _ in range(n)]
        k, mx = 0, 1
        for i in range(n - 2, -1, -1):
            for j in range(i + 1, n):
                f[i][j] = False
                if s[i] == s[j]:
                    f[i][j] = f[i + 1][j - 1]
                    if f[i][j] and mx < j - i + 1:
                        k, mx = i, j - i + 1
        return s[k : k + mx]
```

Solution 2: Enumerate Palindrome Midpoint

How to think about it

Solution 1 is already $O(n^2)$ time, but it still pays $O(n^2)$ space. Each check only needs the inner layer, not the whole table. A palindrome expands symmetrically from a center: grow while the two ends match. We must try both odd centers (i,i) and even centers (i,i+1), or we miss strings like `"bb"`. After expanding, recover the start from center i and length t. Extra space drops to O(1).

We can enumerate the midpoint of the palindrome, spread to both sides, and find the longest palindrome.

The time complexity is $O(n^2)$, and the space complexity is $O(1)$. Here, n is the length of the string s.

Python 3

```python
class Solution:
    def longestPalindrome(self, s: str) -> str:
        def f(l, r):
            while l >= 0 and r < n and s[l] == s[r]:
                l, r = l - 1, r + 1
            return r - l - 1

        n = len(s)
        start, mx = 0, 1
        for i in range(n):
            a = f(i, i)
            b = f(i, i + 1)
            t = max(a, b)
            if mx < t:
                mx = t
                start = i - ((t - 1) >> 1)
        return s[start : start + mx]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Linear time: Manacher's algorithm
# 2. Count ALL palindromic substrings


# ========================================================
# FOLLOW-UP 1: Linear time: Manacher's algorithm
# ========================================================

# ---- follow-up 1: helper functions ----
def min_of(a, b):
    if a < b:
        return a
    return b
```

```
# ---- follow-up 1: function ----
# Answer: Insert a separator between all characters so
# every palindrome has odd length, and keep for every
# centre the radius of its palindrome. Reuse the mirror
# centre's radius inside the rightmost palindrome found so
# far: the centres are visited once and the right edge
# never moves back.
# Time: O(n).  Space: O(n).
def manacher_longest_palindrome(s):
    t = "#" + "#".join(s) + "#"
    radius = [0] * len(t)
    centre = right = 0
    for i in range(len(t)):
        if i < right:
            radius[i] = min_of(right - i, radius[2 * centre - i])
        while i - radius[i] - 1 >= 0 and i + radius[i] + 1 < len(t) and t[i - radius[i] - 1] == t[i +
radius[i] + 1]:
            radius[i] += 1
        if i + radius[i] > right:
            centre, right = i, i + radius[i]
    best = 0
    best_centre = 0
    for i in range(len(t)):
        if radius[i] > best:
            best = radius[i]
            best_centre = i
    start = (best_centre - best) // 2
    return s[start:start + best]


# ============================================================
# FOLLOW-UP 2: Count ALL palindromic substrings
# ============================================================

# ---- follow-up 2: function ----
# Answer: Expand around each of the 2n - 1 centres and
# count one palindrome for every successful expansion
# step.
# Time: O(n^2).  Space: O(1).
def count_palindromic_substrings(s):
    count = 0
    for centre in range(2 * len(s) - 1):
        left = centre // 2
        right = left + centre % 2
        while left >= 0 and right < len(s) and s[left] == s[right]:
            count += 1
            left -= 1
            right += 1
    return count
```

38. Count and Say

Medium · String

leetcode.com/problems/count-and-say

PROBLEM

The **count-and-say** sequence is a sequence of digit strings defined by the recursive formula:

- `countAndSay(1) = "1"`
- `countAndSay(n)` is the run-length encoding of `countAndSay(n - 1)`.

Run-length encoding (RLE) is a string compression method that works by replacing each maximal group of consecutive identical characters with the concatenation of the length of the group followed by the character itself. For example, to compress the string `"3322251"` we replace `"33"` with `"23"`, replace `"222"` with `"32"`, replace `"5"` with `"15"`, and replace `"1"` with `"11"`. Thus the compressed string becomes `"23321511"`.

Given a positive integer `n`, return *the* n^{th} *element of the* ***count-and-say*** *sequence*.

Example 1:

Input: n = 4

Output: "1211"

Explanation:

```
countAndSay(1) = "1"
countAndSay(2) = RLE of "1" = "11"
countAndSay(3) = RLE of "11" = "21"
countAndSay(4) = RLE of "21" = "1211"
```

Example 2:

Input: n = 1

Output: "1"

Explanation:

This is the base case.

Constraints:

- `1 <= n <= 30`

Follow up:

Could you solve it iteratively?

SOLUTION (PYTHON 3)

Solution 1: Simulation

> **How to think about it**
>
> The definition itself is “the n-th term describes the (n-1)-th”, so there is no cheaper closed form. n ≤ 30, so we can start from `"1"` and iterate n-1 times. Each term is a run-length encoding of the previous one: a streak of the same character becomes “count + character”. Two pointers walk the current string s and append each run’s length and character to a new string. The follow-up asks for an iterative solution, which also avoids a recursion stack.

The task requires outputting the appearance sequence of the n-th item, where the n-th item is the description of the n-1-th item in the sequence. Therefore, we iterate n-1 times. In each iteration, we use fast and slow pointers, denoted as j and i respectively, to record the current character's position and the position of the next character that is not equal to the current character. We then update the sequence of the previous item to be j-i occurrences of the current character.

Time Complexity:

1. The outer loop runs `n - 1` times, iterating to generate the "Count and Say" sequence up to the nth term.
2. The inner while loop iterates through each character in the current string s and counts the consecutive occurrences of the same character.
3. The inner while loop runs in O(m) time, where m is the length of the current string s.

Overall, the time complexity is O(n × m), where n is the input parameter representing the term to generate, and m is the maximum length of the string in the sequence.

Space Complexity: O(m).

Python 3

```python
class Solution:
    def countAndSay(self, n: int) -> str:
        s = '1'
        for _ in range(n - 1):
            i = 0
            t = []
            while i < len(s):
                j = i
                while j < len(s) and s[j] == s[i]:
                    j += 1
                t.append(str(j - i))
                t.append(str(s[i]))
                i = j
            s = ''.join(t)
        return s
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Say one term out loud: describe a given digit string
# 2. Go backwards: which term produced a given term?


# ============================================================
# FOLLOW-UP 1: Say one term out loud: describe a given
# digit string
# ============================================================

# ---- follow-up 1: function ----
# Answer: Group equal neighbouring digits: every group
# becomes its length followed by its digit.
# Time: O(n).  Space: O(n).
def say(term):
    result = []
    i = 0
    while i < len(term):
        j = i
        while j < len(term) and term[j] == term[i]:
            j += 1
        result.append(str(j - i) + term[i])
        i = j
    return "".join(result)


# ============================================================
# FOLLOW-UP 2: Go backwards: which term produced a given
# term?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Read the term as (count, digit) pairs and expand
# every pair. The term must have an even length.
# Time: O(n).  Space: O(n).
```

```
def unsay(term):
    if len(term) % 2 != 0:
        return None
    result = []
    for i in range(0, len(term), 2):
        result.append(term[i + 1] * int(term[i]))
    return "".join(result)
```

249. Group Shifted Strings

Medium · Array, Hash Table, String

leetcode.com/problems/group-shifted-strings

Also appears in pattern 7 String Matching / Pattern Search

PROBLEM

Perform the following shift operations on a string:

- **Right shift**: Replace every letter with the **successive** letter of the English alphabet, where 'z' is replaced by 'a'. For example, `"abc"` can be right-shifted to `"bcd"` or `"xyz"` can be right-shifted to `"yza"`.
- **Left shift**: Replace every letter with the **preceding** letter of the English alphabet, where 'a' is replaced by 'z'. For example, `"bcd"` can be left-shifted to `"abc"` or `"yza"` can be left-shifted to `"xyz"`.

We can keep shifting the string in both directions to form an **endless shifting sequence**.

- For example, shift `"abc"` to form the sequence: `... <-> "abc" <-> "bcd" <-> ... <-> "xyz" <-> "yza" <-> .... <-> "zab" <-> "abc" <-> ...`

You are given an array of strings `strings`, group together all `strings[i]` that belong to the same shifting sequence. You may return the answer in **any order**.

Example 1:

Input: strings = ["abc","bcd","acef","xyz","az","ba","a","z"]

Output: [["acef"],["a","z"],["abc","bcd","xyz"],["az","ba"]]

Example 2:

Input: strings = ["a"]

Output: [["a"]]

Constraints:

- `1 <= strings.length <= 200`
- `1 <= strings[i].length <= 50`
- `strings[i]` consists of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Hash Table

> **How to think about it**
>
> Strings in one shift group share the same sequence of letter gaps. Translating each string so that it starts with a yields a grouping key. A hash map buckets by that canonical form, and the buckets are the answer.

We use a hash table g to store each string after shifting and with the first character as 'a'. That is, g[t] represents the set of all strings that become t after shifting.

We iterate through each string. For each string, we calculate its shifted string t, and then add it to g[t].

Finally, we take out all the values in g, which is the answer.

The time complexity is O(L) and the space complexity is O(L), where L is the sum of the lengths of all strings.

Python 3

```
class Solution:
    def groupStrings(self, strings: List[str]) -> List[List[str]]:
        g = defaultdict(list)
        for s in strings:
            diff = ord(s[0]) - ord("a")
            t = []
            for c in s:
                c = ord(c) - diff
                if c < ord("a"):
                    c += 26
                t.append(chr(c))
            g["".join(t)].append(s)
        return list(g.values())
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. How many letters must string a be shifted to become
# string b (or -1)?
# 2. Count the groups instead of listing them


# ============================================================
# FOLLOW-UP 1: How many letters must string a be shifted
# to become string b (or -1)?
# ============================================================

# ---- follow-up 1: function ----
# Answer: Strings of the same length are shifts of each
# other when every position differs by the same amount
# modulo 26.
# Time: O(n).  Space: O(1).
def shift_amount(a, b):
    if len(a) != len(b):
        return -1
    if a == "":
        return 0
    amount = (ord(b[0]) - ord(a[0])) % 26
    for i in range(len(a)):
        if (ord(b[i]) - ord(a[i])) % 26 != amount:
            return -1
    return amount


# ============================================================
# FOLLOW-UP 2: Count the groups instead of listing them
# ============================================================

# ---- follow-up 2: function ----
# Answer: Give every string a key made of the differences
# between neighbouring letters modulo 26; the number of
# different keys is the number of groups.
# Time: O(N * L).  Space: O(N * L).
def count_shift_groups(strings):
    keys = set()
    for s in strings:
        key = []
        for i in range(1, len(s)):
            key.append((ord(s[i]) - ord(s[i - 1])) % 26)
        keys.add((len(s), tuple(key)))
    return len(keys)
```

767. Reorganize String

Medium · Greedy, Hash Table, String, Counting, Sorting, Heap (Priority Queue)

leetcode.com/problems/reorganize-string

★ **TOP 7 PROBLEM** in Heap / Priority Queue: master this one first

Also appears in pattern 19 Heap / Priority Queue, 37 Two Heaps

PROBLEM

Given a string `s`, rearrange the characters of `s` so that any two adjacent characters are not the same.

Return *any possible rearrangement of* `s` *or return* `""` *if not possible*.

Example 1:

```
Input: s = "aab"
Output: "aba"
```

Example 2:

```
Input: s = "aaab"
Output: ""
```

Constraints:

- `1 <= s.length <= 500`
- `s` consists of lowercase English letters.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Rearrange so no two adjacent characters match. If one letter exceeds ceil(n/2), it is impossible. Place the most frequent letters on even indices first, then restart at index 1, which separates copies. Fill from `most_common` into a buffer, wrapping i to 1 when it passes n.

Python 3

```python
class Solution:
    def reorganizeString(self, s: str) -> str:
        n = len(s)
        cnt = Counter(s)
        mx = max(cnt.values())
        if mx > (n + 1) // 2:
            return ''
        i = 0
        ans = [None] * n
        for k, v in cnt.most_common():
            while v:
                ans[i] = k
                v -= 1
                i += 2
                if i >= n:
                    i = 1
        return ''.join(ans)
```

Complexity. Time: O(n + C log C), C = number of distinct letters. Space: O(n).

Solution 2

How to think about it

Even-index placement is specific to distance 1. The same greedy as “rearrange k apart” uses a max-heap plus a cooldown queue of length k=2. Pop the current most frequent letter, enqueue it, and return it to the heap after k steps. If the built string is short, fail.

Python 3

```python
class Solution:
    def reorganizeString(self, s: str) -> str:
        return self.rearrangeString(s, 2)

    def rearrangeString(self, s: str, k: int) -> str:
        h = [(-v, c) for c, v in Counter(s).items()]
        heapify(h)
        q = deque()
        ans = []
        while h:
            v, c = heappop(h)
            v *= -1
            ans.append(c)
            q.append((v - 1, c))
            if len(q) >= k:
                w, c = q.popleft()
                if w:
                    heappush(h, (-w, c))
        return "" if len(ans) != len(s) else "".join(ans)
```

Complexity. Time: O(n log C), C = number of distinct letters. Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The same letter must be at least K positions apart
# 2. Only say whether it is possible


# ============================================================
# FOLLOW-UP 1: The same letter must be at least K
# positions apart
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use a max-heap by remaining count. Take the most
# frequent letter, put it in the answer, and park it in a
# waiting queue for K steps; when it leaves the queue it
# goes back in the heap if letters remain.
# Time: O(n log s), s = distinct letters.  Space: O(n).
import heapq
from collections import deque


def rearrange_k_distance_apart(s, k):
    if k <= 1:
        return s
    count = {}
    for ch in s:
        count[ch] = count.get(ch, 0) + 1
    heap = []
    for ch in count:
        heapq.heappush(heap, (-count[ch], ch))
    waiting = deque()
    result = []
    while heap:
        remaining, ch = heapq.heappop(heap)
```

```
        result.append(ch)
        waiting.append((remaining + 1, ch))
        if len(waiting) >= k:
            back, back_ch = waiting.popleft()
            if back < 0:
                heapq.heappush(heap, (back, back_ch))
    if len(result) == len(s):
        return "".join(result)
    return ""


# ============================================================
# FOLLOW-UP 2: Only say whether it is possible
# ============================================================

# ---- follow-up 2: function ----
# Answer: No rebuilding is needed: it is possible exactly
# when the most frequent letter appears at most (length +
# 1) / 2 times.
# Time: O(n).  Space: O(s).
def can_reorganize(s):
    count = {}
    most = 0
    for ch in s:
        count[ch] = count.get(ch, 0) + 1
        if count[ch] > most:
            most = count[ch]
```

```
    return most <= (len(s) + 1) // 2
```

PATTERN 6 · STRING MANIPULATION / PARSING · PROBLEM 17 OF 20

394. Decode String

Medium · Stack, Recursion, String

leetcode.com/problems/decode-string

★ **TOP 7 PROBLEM** in String Manipulation / Parsing, Stack: master this one first

Also appears in pattern 8 Stack

PROBLEM

Given an encoded string, return its decoded string.

The encoding rule is: `k[encoded_string]`, where the `encoded_string` inside the square brackets is being repeated exactly `k` times. Note that `k` is guaranteed to be a positive integer.

You may assume that the input string is always valid; there are no extra white spaces, square brackets are well-formed, etc. Furthermore, you may assume that the original data does not contain any digits and that digits are only for those repeat numbers, `k`. For example, there will not be input like `3a` or `2[4]`.

The test cases are generated so that the length of the output will never exceed 10^5.

Example 1:

```
Input: s = "3[a]2[bc]"
Output: "aaabcbc"
```

Example 2:

```
Input: s = "3[a2[c]]"
Output: "accaccacc"
```

Example 3:

```
Input: s = "2[abc]3[cd]ef"
Output: "abcabccdcdcdef"
```

Constraints:

- `1 <= s.length <= 30`
- `s` consists of lowercase English letters, digits, and square brackets `'[]'`.
- `s` is guaranteed to be **a valid** input.
- All the integers in `s` are in the range `[1, 300]`.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Decode nested k[encoded] repeats. Recursion works; a stack can store the multiplier and the outer string together. Digits build `num`; `[` pushes the count and the current result; `]` pops and concatenates the repeated segment; letters append. The stack expands inside-out.

Python 3

```python
class Solution:
    def decodeString(self, s: str) -> str:
        s1, s2 = [], []
        num, res = 0, ''
        for c in s:
            if c.isdigit():
                num = num * 10 + int(c)
            elif c == '[':
                s1.append(num)
                s2.append(res)
                num, res = 0, ''
            elif c == ']':
                res = s2.pop() + res * s1.pop()
            else:
                res += c
        return res
```

Complexity. Time: O(L), L = length of the decoded string (each character is copied once per nesting level). Space: O(L).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Find the character at a given position of the decoded
# string without building it
# 2. Check that an encoded string is well formed


# ============================================================
# FOLLOW-UP 1: Find the character at a given position of
# the decoded string without building it
# ============================================================

# ---- follow-up 1: function ----
# Answer: Work out the decoded length of every piece
# first; then walk down into the piece that contains the
# position, reducing the position modulo the length of one
# repetition.
# Time: O(n^2) in the worst case (the length of a group is
# recomputed).  Space: O(n).
def char_at(s, index):
    # parse into nested groups, then skip whole repetitions instead of expanding them
    stack = []
    current = []                                    # list of items: a letter or a (times, items) group
    number = 0
    for ch in s:
        if ch.isdigit():
            number = number * 10 + int(ch)
        elif ch == "[":
            stack.append((current, number))
            current = []
            number = 0
        elif ch == "]":
            before, times = stack.pop()
            before.append((times, current))
            current = before
        else:
            current.append(ch)

    def length(items):
        total = 0
        for item in items:
            if isinstance(item, str):
                total += 1
            else:
                total += item[0] * length(item[1])
        return total
```

```
    def find(items, position):
        for item in items:
            if isinstance(item, str):
                if position == 0:
                    return item
                position -= 1
            else:
                one = length(item[1])
                if position < item[0] * one:
                    return find(item[1], position % one)
                position -= item[0] * one
        return None

    return find(current, index)
```

```
# ============================================================
# FOLLOW-UP 2: Check that an encoded string is well formed
# ============================================================

# ---- follow-up 2: function ----
# Answer: Run the same stack scan and check: brackets
# balance, every '[' has a number before it, and numbers
# are not followed by anything but a letter, a digit or
# '['.
# Time: O(n).  Space: O(1).
def is_valid_encoding(s):
    depth = 0
    previous = ""
    for ch in s:
        if ch == "[":
            if not previous.isdigit():
                return False
            depth += 1
        elif ch == "]":
            depth -= 1
            if depth < 0 or previous == "[":
                return False
        elif ch.isdigit():
            pass
        elif not ch.isalpha():
            return False
        if previous.isdigit() and not (ch.isdigit() or ch == "["):
            return False
        previous = ch
    return depth == 0 and not previous.isdigit()
```

68. Text Justification

Hard · Array, String, Simulation

leetcode.com/problems/text-justification

★ **TOP 7 PROBLEM** in String Manipulation / Parsing: master this one first

PROBLEM

Given an array of strings `words` and a width `maxWidth`, format the text such that each line has exactly `maxWidth` characters and is fully (left and right) justified.

You should pack your words in a greedy approach; that is, pack as many words as you can in each line. Pad extra spaces `' '` when necessary so that each line has exactly `maxWidth` characters.

Extra spaces between words should be distributed as evenly as possible. If the number of spaces on a line does not divide evenly between words, the empty slots on the left will be assigned more spaces than the slots on the right.

For the last line of text, it should be left-justified, and no extra space is inserted between words.

Note:

- A word is defined as a character sequence consisting of non-space characters only.
- Each word's length is guaranteed to be greater than `0` and not exceed `maxWidth`.
- The input array `words` contains at least one word.

Example 1:

```
Input: words = ["This", "is", "an", "example", "of", "text", "justification."], maxWidth = 16
Output:
[
   "This is an",
   "example of text",
   "justification. "
]
```

Example 2:

```
Input: words = ["What","must","be","acknowledgment","shall","be"], maxWidth = 16
Output:
[
  "What must be",
  "acknowledgment ",
  "shall be "
]
Explanation: Note that the last line is "shall be " instead of "shall be", because the last line must
be left-justified instead of fully-justified.
Note that the second line is also left-justified because it contains only one word.
```

Example 3:

```
Input: words = ["Science","is","what","we","understand","well","enough","to","explain","to","a","compu
ter.","Art","is","everything","else","we","do"], maxWidth = 20
Output:
[
  "Science is what we",
  "understand well",
  "enough to explain to",
  "a computer. Art is",
  "everything else we",
  "do "
]
```

Constraints:

- `1 <= words.length <= 300`
- `1 <= words[i].length <= 20`
- `words[i]` consists of only English letters and symbols.
- `1 <= maxWidth <= 100`
- `words[i].length <= maxWidth`

SOLUTION (PYTHON 3)

Solution 1: Simulation

How to think about it

The first idea is to pack as many words as possible into a line, then pad with spaces. There are at most 300 words and the width is at most 100, so a linear scan is enough; no backtracking. The bottleneck is not runtime but the spacing rules: the last line and a one-word line must be left-aligned; every other line must spread extra spaces as evenly as possible, leftover spaces going to the left gaps first. So we greedily take the words that fit, then branch on “last line / single word” versus even distribution. Process lines from left to right.

We can simulate the process according to the problem's requirements. Note that if it is the last line, or if there is only one word in the line, then we should align to the left. Otherwise, we should distribute the spaces evenly.

The time complexity is O(L), and the space complexity is O(L). Here, L is the sum of the lengths of all words.

Python 3

```python
class Solution:
    def fullJustify(self, words: List[str], maxWidth: int) -> List[str]:
        ans = []
        i, n = 0, len(words)
        while i < n:
            t = []
            cnt = len(words[i])
            t.append(words[i])
            i += 1
            while i < n and cnt + 1 + len(words[i]) <= maxWidth:
                cnt += 1 + len(words[i])
                t.append(words[i])
                i += 1
            if i == n or len(t) == 1:
                left = ' '.join(t)
                right = ' ' * (maxWidth - len(left))
                ans.append(left + right)
                continue
            space_width = maxWidth - (cnt - len(t) + 1)
            w, m = divmod(space_width, len(t) - 1)
            row = []
            for j, s in enumerate(t[:-1]):
                row.append(s)
                row.append(' ' * (w + (1 if j < m else 0)))
            row.append(t[-1])
            ans.append(''.join(row))
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Left-justify every line (no stretching of spaces)
# 2. Break the lines so that the total 'raggedness' (sum
# of squares of the trailing spaces) is smallest


# ============================================================
# FOLLOW-UP 1: Left-justify every line (no stretching of
# spaces)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Pack words greedily with single spaces; pad each
# line on the right with spaces to the full width.
# Time: O(total number of characters).  Space: O(total
# number of characters).
def left_justify(words, max_width):
    lines = []
    line = ""
    for word in words:
        if line == "":
            line = word
        elif len(line) + 1 + len(word) <= max_width:
            line += " " + word
        else:
            lines.append(line + " " * (max_width - len(line)))
            line = word
    if line != "":
        lines.append(line + " " * (max_width - len(line)))
    return lines


# ============================================================
# FOLLOW-UP 2: Break the lines so that the total
# 'raggedness' (sum of squares of the trailing spaces) is
# smallest
# ============================================================

# ---- follow-up 2: function ----
# Answer: Greedy filling is not optimal. best[i] = the
# least cost for the first i words; try every start of the
# last line, where the cost is the square of its unused
# width (the last line is free).
# Time: O(n * k), k = most words on one line.  Space:
# O(n).
def min_raggedness(words, max_width):
    n = len(words)
    big = float("inf")
    best = [big] * (n + 1)
    best[0] = 0
    for end in range(1, n + 1):
        length = -1
        for start in range(end - 1, -1, -1):
            length += len(words[start]) + 1
            if length > max_width:
                break
            spare = max_width - length
            cost = 0 if end == n else spare * spare
            if best[start] + cost < best[end]:
                best[end] = best[start] + cost
```

```
    return best[n]
```

PATTERN 6 · STRING MANIPULATION / PARSING · PROBLEM 19 OF 20

224. Basic Calculator

Hard · Stack, Recursion, Math, String

leetcode.com/problems/basic-calculator

★ **TOP 7 PROBLEM** in String Manipulation / Parsing, Stack: master this one first

Also appears in pattern 8 Stack

PROBLEM

Given a string `s` representing a valid expression, implement a basic calculator to evaluate it, and return *the result of the evaluation*.

Note: You are **not** allowed to use any built-in function which evaluates strings as mathematical expressions, such as `eval()`.

Example 1:

```
Input: s = "1 + 1"
Output: 2
```

Example 2:

```
Input: s = " 2-1 + 2 "
Output: 3
```

Example 3:

```
Input: s = "(1+(4+5+2)-3)+(6+8)"
Output: 23
```

Constraints:

- `1 <= s.length <= 3 * 10^5`
- `s` consists of digits, `'+'`, `'-'`, `'('`, `')'`, and `' '`.
- `s` represents a valid expression.
- `'+'` is **not** used as a unary operation (i.e., `"+1"` and `"+(2 + 3)"` is invalid).
- `'-'` could be used as a unary operation (i.e., `"-1"` and `"-(2 + 3)"` is valid).
- There will be no two consecutive operators in the input.
- Every number and running calculation will fit in a signed 32-bit integer.

SOLUTION (PYTHON 3)

Solution 1: Stack

How to think about it

Plus, minus, and parentheses prevent a single left-to-right sum. A parenthesized fragment is its own accumulation, so the outer total and sign must be saved on entry. A stack stores ans and sign at (and resets them; at) we pop and add the inner result times the outer sign. Digits join ans with the current sign.

We use a stack stk to save the current calculation result and operator, a variable sign to save the current sign, and a variable ans to save the final calculation result.

Next, we traverse each character of the string s:

- If the current character is a number, we use a loop to read the following consecutive numbers, and then add or subtract it to ans according to the current sign.
- If the current character is `'+'`, we change the variable sign to positive.

- If the current character is `'-'`, we change the variable sign to negative.
- If the current character is `'('`, we push the current ans and sign into the stack, and reset them to empty and 1, and start to calculate the new ans and sign.
- If the current character is `')'`, we pop the top two elements of the stack, one is the operator, and the other is the number calculated before the bracket. We multiply the current number by the operator, and add the previous number to get the new ans.

After traversing the string s, we return ans.

The time complexity is O(n), and the space complexity is O(n), where n is the length of the string s.

Python 3

```python
class Solution:
    def calculate(self, s: str) -> int:
        stk = []
        ans, sign = 0, 1
        i, n = 0, len(s)
        while i < n:
            if s[i].isdigit():
                x = 0
                j = i
                while j < n and s[j].isdigit():
                    x = x * 10 + int(s[j])
                    j += 1
                ans += sign * x
                i = j - 1
            elif s[i] == "+":
                sign = 1
            elif s[i] == "-":
                sign = -1
            elif s[i] == "(":
                stk.append(ans)
                stk.append(sign)
                ans, sign = 0, 1
            elif s[i] == ")":
                ans = stk.pop() * ans + stk.pop()
            i += 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Support * and / as well, with parentheses (Basic
# Calculator III)
# 2. Evaluate an expression that contains variables


# ============================================================
# FOLLOW-UP 1: Support * and / as well, with parentheses
# (Basic Calculator III)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Recursive descent: an expression is terms joined
# by + and -, a term is factors joined by * and /, and a
# factor is a number, a minus sign followed by a factor,
# or a parenthesised expression (which calls the
# expression function again).
# Time: O(n).  Space: O(n).
def tokenize(s):
    tokens = []
    i = 0
    while i < len(s):
        ch = s[i]
        if ch.isdigit():
```

```
            number = 0
            while i < len(s) and s[i].isdigit():
                number = number * 10 + int(s[i])
                i += 1
            tokens.append(number)
            continue
        if ch != " ":
            tokens.append(ch)
        i += 1
    return tokens


def calculate_with_all_operators(s):
    tokens = tokenize(s)
    position = [0]

    def next_is(characters):
        return position[0] < len(tokens) and isinstance(tokens[position[0]], str) and tokens[position[0]]
in characters

    def parse_factor():
        token = tokens[position[0]]
        position[0] += 1
        if token == "(":
            value = parse_expression()
            position[0] += 1                        # skip the closing bracket
            return value
        if token == "-":
            return -parse_factor()
        return token

    def parse_term():
        value = parse_factor()
        while next_is("*/"):
            operator = tokens[position[0]]
            position[0] += 1
            other = parse_factor()
```

```
            if operator == "*":
                value = value * other
            else:
                value = int(value / other)          # division truncates toward zero
        return value

    def parse_expression():
        value = parse_term()
        while next_is("+-"):
            operator = tokens[position[0]]
            position[0] += 1
            other = parse_term()
            if operator == "+":
                value = value + other
            else:
                value = value - other
        return value

    return parse_expression()


# ============================================================
# FOLLOW-UP 2: Evaluate an expression that contains
# variables
# ============================================================

# ---- follow-up 2: function ----
# Answer: Replace each variable by its value from a
# dictionary while scanning (read the whole name, then
# look it up); the rest of the scan is the same.
# Time: O(n).  Space: O(n).
```

```
def calculate_with_variables(s, values):
    result = 0
    sign = 1
    stack = []
    i = 0
    while i < len(s):
        ch = s[i]
        if ch.isdigit():
            number = 0
            while i < len(s) and s[i].isdigit():
                number = number * 10 + int(s[i])
                i += 1
            result += sign * number
            continue
        if ch.isalpha():
            j = i
            while j < len(s) and s[j].isalnum():
                j += 1
            result += sign * values[s[i:j]]
            i = j
            continue
        if ch == "+":
            sign = 1
        elif ch == "-":
            sign = -1
        elif ch == "(":
            stack.append(result)
            stack.append(sign)
            result = 0
```

```
            sign = 1
        elif ch == ")":
            outer_sign = stack.pop()
            outer_result = stack.pop()
            result = outer_result + outer_sign * result
        i += 1
    return result
```

PATTERN 7 OF 41

String Matching / Pattern Search

20 problems · 5 Easy · 6 Medium · 9 Hard

THE IDEA

Find a pattern inside a text faster than checking every start position: prefix function (KMP), Z-array, rolling hash, or a trie.

Signals: find needle in haystack, repeated substring, shortest palindrome, longest prefix that is also a suffix, many pattern queries.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```python
def prefix_function(p):
    pi = [0] * len(p)
    k = 0
    for i in range(1, len(p)):
        while k and p[i] != p[k]:
            k = pi[k - 1]               # fall back to a shorter border
        if p[i] == p[k]:
            k += 1
        pi[i] = k
    return pi
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Implement strStr() — substitute: Find the Index of the First Occurrence in a String (see page)	Easy	LC 28	299
2	Repeated Substring Pattern (see page)	Easy	LC 459	297
3	Valid Anagram (see page)	Easy	LC 242	14
4	Shortest Distance to a Character	Easy	LC 821	349
5	Index Pairs of a String	Easy	LC 1065	351
6	★ **Find All Anagrams in a String** (see page)	Medium	LC 438	84
7	Permutation in String (see page)	Medium	LC 567	81
8	★ **Longest Palindromic Substring** (see page)	Medium	LC 5	327
9	Group Shifted Strings (see page)	Medium	LC 249	333
10	Camelcase Matching	Medium	LC 1023	354
11	Number of Matching Subsequences	Medium	LC 792	357
12	★ **Shortest Palindrome**	Hard	LC 214	360
13	★ **Wildcard Matching**	Hard	LC 44	363
14	★ **Regular Expression Matching**	Hard	LC 10	367
15	★ **Longest Happy Prefix**	Hard	LC 1392	371
16	Find All Good Strings — substitute: Distinct Echo Substrings	Hard	LC 1316	374
17	Minimum Window Substring (see page)	Hard	LC 76	121
18	★ **Stream of Characters**	Hard	LC 1032	376
19	Palindrome Pairs	Hard	LC 336	380
20	Concatenated Words	Hard	LC 472	384

821. Shortest Distance to a Character

Easy · Array, Two Pointers, String

leetcode.com/problems/shortest-distance-to-a-character

PROBLEM

Given a string `s` and a character `c` that occurs in `s`, return *an array of integers* `answer` *where* `answer.length == s.length` *and* `answer[i]` *is the* ***distance*** *from index* `i` *to the* ***closest*** *occurrence of character* `c` *in* `s`.

The **distance** between two indices `i` and `j` is `abs(i - j)`, where `abs` is the absolute value function.

Example 1:

```
Input: s = "loveleetcode", c = "e"
Output: [3,2,1,0,1,0,0,1,2,2,1,0]
Explanation: The character 'e' appears at indices 3, 5, 6, and 11 (0-indexed).
The closest occurrence of 'e' for index 0 is at index 3, so the distance is abs(0 - 3) = 3.
The closest occurrence of 'e' for index 1 is at index 3, so the distance is abs(1 - 3) = 2.
For index 4, there is a tie between the 'e' at index 3 and the 'e' at index 5, but the distance is
still the same: abs(4 - 3) == abs(4 - 5) = 1.
The closest occurrence of 'e' for index 8 is at index 6, so the distance is abs(8 - 6) = 2.
```

Example 2:

```
Input: s = "aaab", c = "b"
Output: [3,2,1,0]
```

Constraints:

- `1 <= s.length <= 10^4`
- `s[i]` and `c` are lowercase English letters.
- It is guaranteed that `c` occurs at least once in `s`.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Each index needs its distance to the nearest c. Scanning both sides from every position is quadratic and unnecessary for n≤ 10^4. The nearest c is either to the left or to the right. A left-to-right pass records the latest c; a right-to-left pass keeps the closer of the two. Two linear sweeps fill the answer.

Python 3

```python
class Solution:
    def shortestToChar(self, s: str, c: str) -> List[int]:
        n = len(s)
        ans = [n] * n
        pre = -inf
        for i, ch in enumerate(s):
            if ch == c:
                pre = i
            ans[i] = min(ans[i], i - pre)
        suf = inf
        for i in range(n - 1, -1, -1):
            if s[i] == c:
                suf = i
            ans[i] = min(ans[i], suf - i)
        return ans
```

Complexity. Time: O(n). Space: O(1) extra (the answer itself is O(n)).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Also return WHICH position is the nearest occurrence
# 2. The distance to the nearest of SEVERAL characters


# ============================================================
# FOLLOW-UP 1: Also return WHICH position is the nearest
# occurrence
# ============================================================

# ---- follow-up 1: function ----
# Answer: Two passes over the string, left to right and
# right to left, each remembering the last position of the
# character; keep the closer of the two.
# Time: O(n).  Space: O(n).
def nearest_positions(s, c):
    n = len(s)
    nearest = [-1] * n
    distance = [n + 1] * n
    last = -n - 1
    for i in range(n):
        if s[i] == c:
            last = i
        if i - last < distance[i]:
            distance[i] = i - last
            nearest[i] = last
    last = 2 * n + 1
    for i in range(n - 1, -1, -1):
        if s[i] == c:
            last = i
        if last - i < distance[i]:
            distance[i] = last - i
            nearest[i] = last
    return nearest


# ============================================================
# FOLLOW-UP 2: The distance to the nearest of SEVERAL
# characters
# ============================================================

# ---- follow-up 2: function ----
# Answer: Treat every position holding one of the
# characters as a source; one pass from each side keeps
# the distance to the last source seen.
# Time: O(n).  Space: O(n).
def distance_to_nearest_of_set(s, characters):
    n = len(s)
    distance = [n + 1] * n
    last = None
    for i in range(n):
        if s[i] in characters:
            last = i
        if last is not None:
            distance[i] = i - last
    last = None
    for i in range(n - 1, -1, -1):
        if s[i] in characters:
            last = i
        if last is not None and last - i < distance[i]:
            distance[i] = last - i
    return distance
```

PATTERN 7 · STRING MATCHING / PATTERN SEARCH · PROBLEM 5 OF 20

1065. Index Pairs of a String

Easy · Trie, Array, String, Sorting, Aho-Corasick

leetcode.com/problems/index-pairs-of-a-string

Also appears in pattern 32 Trie (Prefix Tree)

PROBLEM

Given a string `text` and an array of strings `words`, return *an array of all index pairs* `[i, j]` *so that the substring* `text[i...j]` *is in* `words`.

Return the pairs `[i, j]` in sorted order (i.e., sort them by their first coordinate, and in case of ties sort them by their second coordinate).

Example 1:

```
Input: text = "thestoryofleetcodeandme", words = ["story","fleet","leetcode"]
Output: [[3,7],[9,13],[10,17]]
```

Example 2:

```
Input: text = "ababa", words = ["aba","ab"]
Output: [[0,1],[0,2],[2,3],[2,4]]
Explanation: Notice that matches can overlap, see "aba" is found in [0,2] and [2,4].
```

Constraints:

- `1 <= text.length <= 100`
- `1 <= words.length <= 20`
- `1 <= words[i].length <= 50`
- `text` and `words[i]` consist of lowercase English letters.
- All the strings of `words` are **unique**.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Every substring of text can be tested against the word list. Both sides are short, so a set of words plus all slices [i,j] is enough. The set answers in expected constant time, and enumerating indices already yields sorted pairs.

Python 3

```python
class Solution:
    def indexPairs(self, text: str, words: List[str]) -> List[List[int]]:
        words = set(words)
        n = len(text)
        return [
            [i, j] for i in range(n) for j in range(i, n) if text[i : j + 1] in words
        ]
```

Complexity. Time: O(n^3) in the worst case (n^2 slices of length up to n). Space: O(W), W = total length of the words.

Solution 2

How to think about it

Trying every slice wastes failed lookups on long text. Shared prefixes let us walk a trie from each start and stop at the first mismatch. Insert the words, then from each i extend the node to the right and record [i,j] at end marks.

Python 3

```python
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, word):
        node = self
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True


class Solution:
    def indexPairs(self, text: str, words: List[str]) -> List[List[int]]:
        trie = Trie()
        for w in words:
            trie.insert(w)
        n = len(text)
        ans = []
        for i in range(n):
            node = trie
            for j in range(i, n):
                idx = ord(text[j]) - ord('a')
                if node.children[idx] is None:
                    break
                node = node.children[idx]
                if node.is_end:
                    ans.append([i, j])
        return ans
```

Complexity. Time: O(W + n * m), m = length of the longest word. Space: O(W).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Thousands of words: find all matches with a trie of
# the words
# 2. Wrap every matched word in bold tags (merging
# touching or overlapping matches)


# ============================================================
# FOLLOW-UP 1: Thousands of words: find all matches with a
# trie of the words
# ============================================================

# ---- follow-up 1: function ----
# Answer: Build a trie of the words. From every start
# position walk down the trie along the text; every time
# you pass a node that ends a word, report the pair.
# Time: O(W + n * m), W = total letters of the words, m =
# longest word.  Space: O(W).
def index_pairs_with_trie(text, words):
    root = {}
    for word in words:
```

```
        node = root
        for ch in word:
            if ch not in node:
                node[ch] = {}
            node = node[ch]
        node["$"] = True                            # a word ends here
    pairs = []
    for start in range(len(text)):
        node = root
        for end in range(start, len(text)):
            if text[end] not in node:
                break
            node = node[text[end]]
            if "$" in node:
                pairs.append([start, end])
    return pairs


# ============================================================
# FOLLOW-UP 2: Wrap every matched word in bold tags
# (merging touching or overlapping matches)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Turn the pairs into intervals, merge the ones
# that overlap or touch, then insert <b> before the start
# and </b> after the end of every merged interval.
# Time: O(n * W') (find for every word), W' = total
# letters.  Space: O(n).
def add_bold_tags(text, words):
    bold = [False] * len(text)
    for word in words:
        start = text.find(word)
        while start != -1:
            for i in range(start, start + len(word)):
                bold[i] = True
            start = text.find(word, start + 1)
    result = []
    for i in range(len(text)):
```

```
        if bold[i] and (i == 0 or not bold[i - 1]):
            result.append("<b>")
        result.append(text[i])
        if bold[i] and (i == len(text) - 1 or not bold[i + 1]):
            result.append("</b>")
    return "".join(result)
```

1023. Camelcase Matching

Medium · Trie, Array, Two Pointers, String, String Matching

leetcode.com/problems/camelcase-matching

Also appears in pattern 32 Trie (Prefix Tree)

PROBLEM

Given an array of strings `queries` and a string `pattern`, return a boolean array `answer` where `answer[i]` is `true` if `queries[i]` matches `pattern`, and `false` otherwise.

A query word `queries[i]` matches `pattern` if you can insert lowercase English letters into the pattern so that it equals the query. You may insert a character at any position in pattern or you may choose not to insert any characters **at all**.

Example 1:

```
Input: queries = ["FooBar","FooBarTest","FootBall","FrameBuffer","ForceFeedBack"], pattern = "FB"
Output: [true,false,true,true,false]
Explanation: "FooBar" can be generated like this "F" + "oo" + "B" + "ar".
"FootBall" can be generated like this "F" + "oot" + "B" + "all".
"FrameBuffer" can be generated like this "F" + "rame" + "B" + "uffer".
```

Example 2:

```
Input: queries = ["FooBar","FooBarTest","FootBall","FrameBuffer","ForceFeedBack"], pattern = "FoBa"
Output: [true,false,true,false,false]
Explanation: "FooBar" can be generated like this "Fo" + "o" + "Ba" + "r".
"FootBall" can be generated like this "Fo" + "ot" + "Ba" + "ll".
```

Example 3:

```
Input: queries = ["FooBar","FooBarTest","FootBall","FrameBuffer","ForceFeedBack"], pattern = "FoBaT"
Output: [false,true,false,false,false]
Explanation: "FooBarTest" can be generated like this "Fo" + "o" + "Ba" + "r" + "T" + "est".
```

Constraints:

- `1 <= pattern.length, queries.length <= 100`
- `1 <= queries[i].length <= 100`
- `queries[i]` and `pattern` consist of English letters.

SOLUTION (PYTHON 3)

Solution 1: Two Pointers

> **How to think about it**
>
> Each query can be matched against the pattern in linear time; both the number of queries and the lengths are at most 100. Extra letters in a query may only be lowercase insertions, and every pattern character must appear in order. Two pointers walk the query and the pattern. The query pointer skips lowercase mismatches; running off the end or hitting a conflicting letter fails. After the pattern is consumed, the query suffix must be all lowercase. The check is applied to every query.

We can traverse every string in `queries` and check whether it matches `pattern` or not. If it matches, we add `true` to the answer array, otherwise we add `false`.

Next, we implement a function check(s, t) to check whether the string s matches the string t.

We can use two pointers i and j to traverse the two strings. If the characters pointed to by i and j are not the same and s[i] is a lowercase letter, then we move the pointer i to the next position.

If the pointer i has reached the end of the string s or the characters pointed to by i and j are not the same, we return `false`. Otherwise, we move both pointers i and j to the next position. When the pointer j reaches the end of the string t, we need to check if the remaining characters in the string s are all lowercase letters. If so, we return `true`, otherwise we return `false`.

Time complexity O(n × m), where n and m are the length of the array `queries` and the string `pattern` respectively.

Python 3

```python
class Solution:
    def camelMatch(self, queries: List[str], pattern: str) -> List[bool]:
        def check(s, t):
            m, n = len(s), len(t)
            i = j = 0
            while j < n:
                while i < m and s[i] != t[j] and s[i].islower():
                    i += 1
                if i == m or s[i] != t[j]:
                    return False
                i, j = i + 1, j + 1
            while i < m and s[i].islower():
                i += 1
            return i == m

        return [check(q, pattern) for q in queries]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return which positions of the query the pattern
# letters matched
# 2. Match a pattern against the INITIALS of the camel-
# case words (FB matches FooBar)


# ============================================================
# FOLLOW-UP 1: Return which positions of the query the
# pattern letters matched
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same greedy scan: every query letter that
# equals the next pattern letter is matched; the other
# letters must be lowercase.
# Time: O(n).  Space: O(n).
def camel_match_positions(query, pattern):
    positions = []
    j = 0
    for i in range(len(query)):
        if j < len(pattern) and query[i] == pattern[j]:
            positions.append(i)
            j += 1
        elif query[i].isupper():
            return None                              # an uppercase letter that is not in the pattern
    if j < len(pattern):
        return None
    return positions


# ============================================================
# FOLLOW-UP 2: Match a pattern against the INITIALS of the
# camel-case words (FB matches FooBar)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Cut the query into words at every uppercase
# letter, take the first letter of each word, and compare
# it with the pattern.
```

```
# Time: O(n).  Space: O(n).
def matches_camel_initials(query, pattern):
    initials = ""
    for i in range(len(query)):
        if i == 0 or query[i].isupper():
            initials += query[i]
    return initials == pattern
```

792. Number of Matching Subsequences

Medium · Trie, Array, Hash Table, String, Binary Search, Dynamic Programming, Sorting

leetcode.com/problems/number-of-matching-subsequences

Also appears in pattern 32 Trie (Prefix Tree)

PROBLEM

Given a string `s` and an array of strings `words`, return *the number of* `words[i]` *that is a subsequence of* `s`.

A **subsequence** of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- For example, `"ace"` is a subsequence of `"abcde"`.

Example 1:

```
Input: s = "abcde", words = ["a","bb","acd","ace"]
Output: 3
Explanation: There are three strings in words that are a subsequence of s: "a", "acd", "ace".
```

Example 2:

```
Input: s = "dsahjpjauf", words = ["ahjpjau","ja","ahbwzgqnuk","tnmlanowax"]
Output: 2
```

Constraints:

- `1 <= s.length <= 5 * 10^4`
- `1 <= words.length <= 5000`
- `1 <= words[i].length <= 50`
- `s` and `words[i]` consist of only lowercase English letters.

SOLUTIONS (PYTHON 3)

Solution 1: Bucketing

How to think about it

$|s| \le 5 \times 10^4$; a pointer scan per word rereads s. Bucket words by their next needed letter and, while scanning s, advance only that bucket. A finished word counts; otherwise the remainder goes to the bucket of its new head. Each character of each word is handled once.

Python 3

```python
class Solution:
    def numMatchingSubseq(self, s: str, words: List[str]) -> int:
        d = defaultdict(deque)
        for w in words:
            d[w[0]].append(w)
        ans = 0
        for c in s:
            for _ in range(len(d[c])):
                t = d[c].popleft()
                if len(t) == 1:
                    ans += 1
                else:
                    d[t[1]].append(t[1:])
        return ans
```

Complexity. Time: O(n + L), L = total length of the words. Space: O(k), k = number of words.

Solution 2: Bucketing (Index Pointers)

How to think about it

Solution 1 slices leftover strings. Store (word_index, matched_len) instead; same buckets, less allocation.

Python 3

```python
class Solution:
    def numMatchingSubseq(self, s: str, words: List[str]) -> int:
        d = defaultdict(deque)
        for i, w in enumerate(words):
            d[w[0]].append((i, 0))
        ans = 0
        for c in s:
            for _ in range(len(d[c])):
                i, j = d[c].popleft()
                j += 1
                if j == len(words[i]):
                    ans += 1
                else:
                    d[words[i][j]].append((i, j))
        return ans
```

Complexity. Time: O(n + L), L = total length of the words. Space: O(k), k = number of words.

Solution 3: Binary Search

How to think about it

Bucketing keeps all words live. Alternatively, store every index of each letter in s and binary-search the next position for each character of w. A test is O(|w|log n) and needs no queues.

Python 3

```python
class Solution:
    def numMatchingSubseq(self, s: str, words: List[str]) -> int:
        def check(w):
            i = -1
            for c in w:
                j = bisect_right(d[c], i)
                if j == len(d[c]):
                    return False
                i = d[c][j]
            return True

        d = defaultdict(list)
        for i, c in enumerate(s):
            d[c].append(i)
        return sum(check(w) for w in words)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Check all words in ONE pass over the text (waiting
# lists)
# 2. The longest dictionary word that is a subsequence of
# s (smallest alphabetically on ties)


# ========================================================
# FOLLOW-UP 1: Check all words in ONE pass over the text
# (waiting lists)
# ========================================================

# ---- follow-up 1: function ----
```

```
# Answer: Put every word into a bucket by the letter it is
# waiting for. Read the text letter by letter: the words
# waiting for this letter move forward one step, to the
# bucket of their next letter, or are finished.
# Time: O(n + L), L = total length of the words.  Space:
# O(k), k = number of words.
def count_subsequences_one_pass(s, words):
    waiting = {}
    for word in words:
        letter = word[0] if word else ""
        if letter not in waiting:
            waiting[letter] = []
        waiting[letter].append((word, 0))
    count = len(waiting.get("", []))
    for ch in s:
        if ch not in waiting:
            continue
        now = waiting[ch]
        waiting[ch] = []
        for word, position in now:
            position += 1
            if position == len(word):
                count += 1
            else:
                next_letter = word[position]
                if next_letter not in waiting:
                    waiting[next_letter] = []
                waiting[next_letter].append((word, position))
    return count


# ============================================================
# FOLLOW-UP 2: The longest dictionary word that is a
# subsequence of s (smallest alphabetically on ties)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Check each word with two pointers; keep the
# longest, and for equal lengths the alphabetically
# smaller one.
# Time: O(k * n).  Space: O(1).
def longest_word_subsequence(s, dictionary):
    best = ""
    for word in dictionary:
        i = 0
        for ch in s:
            if i < len(word) and ch == word[i]:
                i += 1
        if i == len(word):
            if len(word) > len(best) or (len(word) == len(best) and word < best):
                best = word
    return best
```

PATTERN 7 · STRING MATCHING / PATTERN SEARCH · PROBLEM 12 OF 20

214. Shortest Palindrome

Hard · String, String Matching, Manacher, Hash Function, Rolling Hash, KMP, Extended KMP

leetcode.com/problems/shortest-palindrome

★ **TOP 7 PROBLEM** in String Matching / Pattern Search: master this one first

PROBLEM

You are given a string `s`. You can convert `s` to a palindrome by adding characters in front of it.

Return *the shortest palindrome you can find by performing this transformation*.

Example 1:

```
Input: s = "aacecaaa"
Output: "aaacecaaa"
```

Example 2:

```
Input: s = "abcd"
Output: "dcbabcd"
```

Constraints:

- `0 <= s.length <= 5 * 10^4`
- `s` consists of lowercase English letters only.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

The fewest characters to prepend so that s becomes a palindrome are the reverse of the suffix after the longest palindromic prefix. Testing every prefix is slow. We roll a prefix hash and its reverse; equality updates the longest palindromic prefix. The leftover suffix is reversed and prepended.

Python 3

```python
class Solution:
    def shortestPalindrome(self, s: str) -> str:
        base = 131
        mod = 10**9 + 7
        n = len(s)
        prefix = suffix = 0
        mul = 1
        idx = 0
        for i, c in enumerate(s):
            prefix = (prefix * base + (ord(c) - ord('a') + 1)) % mod
            suffix = (suffix + (ord(c) - ord('a') + 1) * mul) % mod
            mul = (mul * base) % mod
            if prefix == suffix:
                idx = i + 1
        return s if idx == n else s[idx:][::-1] + s
```

Complexity. Time: O(n). Space: O(1) extra (the answer is O(n)).

Solution 2: KMP Algorithm

> **How to think about it**
> Polynomial hashes may collide. Concatenating s with its reverse and computing KMP next makes the last next value the longest palindromic prefix length. The same construction then runs in linear time without modular hashing.

According to the problem description, we need to reverse the string s to obtain the string rev, and then find the longest common part of the suffix of the string rev and the prefix of the string s. We can use the KMP algorithm to concatenate the string s and the string rev and find the longest common part of the longest prefix and the longest suffix.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the string s.

Python 3

```python
class Solution:
    def shortestPalindrome(self, s: str) -> str:
        t = s + "#" + s[::-1] + "$"
        n = len(t)
        next = [0] * n
        next[0] = -1
        i, j = 2, 0
        while i < n:
            if t[i - 1] == t[j]:
                j += 1
                next[i] = j
                i += 1
            elif j:
                j = next[j]
            else:
                next[i] = 0
                i += 1
        return s[::-1][: -next[-1]] + s
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Add characters at the END instead of the front
# 2. List the lengths of ALL palindromic prefixes


# ============================================================
# FOLLOW-UP 1: Add characters at the END instead of the
# front
# ============================================================

# ---- follow-up 1: helper functions ----
def prefix_function(p):
    border = [0] * len(p)
    k = 0
    for i in range(1, len(p)):
        while k > 0 and p[i] != p[k]:
            k = border[k - 1]
        if p[i] == p[k]:
            k += 1
        border[i] = k
    return border

# ---- follow-up 1: function ----
# Answer: Mirror the problem: find the longest palindromic
# SUFFIX; the characters before it, reversed, go to the
# end.
# Time: O(n).  Space: O(n).
def shortest_palindrome_by_appending(s):
    if s == "":
        return s
    reversed_s = s[::-1]
```

```
    border = prefix_function(reversed_s + "#" + s)[-1]      # the longest suffix of s that is a palindrome
    return s + s[:len(s) - border][::-1]


# ============================================================
# FOLLOW-UP 2: List the lengths of ALL palindromic
# prefixes
# ============================================================

# ---- follow-up 2: function ----
# Answer: The longest palindromic prefix length is the
# last value of the prefix function of s + '#' +
# reverse(s); the next shorter ones are found by following
# the borders (each border of that string is again a
# palindromic prefix).
# Time: O(n).  Space: O(n).
def palindromic_prefix_lengths(s):
    border = prefix_function(s + "#" + s[::-1])
    lengths = []
    k = border[-1]
    while k > 0:
        lengths.append(k)
        k = border[k - 1]
    return sorted(lengths)
```

44. Wildcard Matching

Hard · Greedy, Recursion, String, Dynamic Programming

leetcode.com/problems/wildcard-matching

★ **TOP 7 PROBLEM** in String Matching / Pattern Search: master this one first

PROBLEM

Given an input string (`s`) and a pattern (`p`), implement wildcard pattern matching with support for `'?'` and `'*'` where:

- `'?'` Matches any single character.
- `'*'` Matches any sequence of characters (including the empty sequence).

The matching should cover the **entire** input string (not partial).

Example 1:

```
Input: s = "aa", p = "a"
Output: false
Explanation: "a" does not match the entire string "aa".
```

Example 2:

```
Input: s = "aa", p = "*"
Output: true
Explanation: '*' matches any sequence.
```

Example 3:

```
Input: s = "cb", p = "?a"
Output: false
Explanation: '?' matches 'c', but the second letter is 'a', which does not match 'b'.
```

Constraints:

- `0 <= s.length, p.length <= 2000`
- `s` contains only lowercase English letters.
- `p` contains only lowercase English letters, `'?'` or `'*'`.

SOLUTIONS (PYTHON 3)

Solution 1: Memoization Search

How to think about it

The first idea is recurse character by character: literals and `?` match one, `*` eats empty or more. Correct, but without memo the `*` branches explode. |s|, |p| ≤ 2000 will time out. The bottleneck is asking the same suffix pair (i, j) over and over. The state is only “does s[i:] match p[j:]”; subproblems overlap heavily. Cache dfs(i, j). The three `*` moves (consume one, advance both, skip `*`) are transitions on that state. Memoization turns exponential search into O(mn).

We design a function dfs(i, j), which represents whether the string s starting from the i-th character matches the string p starting from the j-th character. The answer is dfs(0, 0).

The execution process of the function dfs(i, j) is as follows:

- If i ≥ len(s), then dfs(i, j) is true only when j ≥ len(p) or p[j] = '*' and dfs(i, j + 1) is true.
- If j ≥ len(p), then dfs(i, j) is false.
- If p[j] = '*', then dfs(i, j) is true if and only if dfs(i + 1, j) or dfs(i + 1, j + 1) or dfs(i, j + 1) is true.
- Otherwise, dfs(i, j) is true if and only if p[j] = '?' or s[i] = p[j] and dfs(i + 1, j + 1) is true.

To avoid repeated calculations, we use the method of memoization search and store the result of dfs(i, j) in a hash table.

The time complexity is O(m × n), and the space complexity is O(m × n). Where m and n are the lengths of the strings s and p, respectively.

Python 3

```python
class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        @cache
        def dfs(i: int, j: int) -> bool:
            if i >= len(s):
                return j >= len(p) or (p[j] == "*" and dfs(i, j + 1))
            if j >= len(p):
                return False
            if p[j] == "*":
                return dfs(i + 1, j) or dfs(i + 1, j + 1) or dfs(i, j + 1)
            return (p[j] == "?" or s[i] == p[j]) and dfs(i + 1, j + 1)

        return dfs(0, 0)
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> Solution 1 already fills this match table from shorter prefixes to longer ones. The f[i][j] below is that same table.

Solution 1 already fills this table from shorter prefixes to longer ones.

Define f[i][j] to represent whether the first i characters of string s match the first j characters of string p. Initially, f[0][0] = true, indicating that two empty strings are matching. For j ∈ [1, n], if p[j-1] = '*', then f[0][j] = f[0][j-1].

Next, we consider the case of i ∈ [1, m] and j ∈ [1, n]:

- If p[j-1] = '*', then f[i][j] = f[i-1][j] ∨ f[i][j-1] ∨ f[i-1][j-1].
- Otherwise, f[i][j] = (p[j-1] = '?' ∨ s[i-1] = p[j-1]) ∧ f[i-1][j-1].

The final answer is f[m][n].

The time complexity is O(m × n), and the space complexity is O(m × n). Where m and n are the lengths of the strings s and p, respectively.

Python 3

```python
class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m, n = len(s), len(p)
        f = [[False] * (n + 1) for _ in range(m + 1)]
        f[0][0] = True
        for j in range(1, n + 1):
            if p[j - 1] == "*":
                f[0][j] = f[0][j - 1]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if p[j - 1] == "*":
                    f[i][j] = f[i - 1][j] or f[i][j - 1] or f[i - 1][j - 1]
                else:
                    f[i][j] = f[i - 1][j - 1] and (
                        p[j - 1] == "?" or s[i - 1] == p[j - 1]
                    )
        return f[m][n]
```

Solution 3: Dynamic Programming

How to think about it

Matching one character at a time is the natural first idea: a literal or `?` consumes one character, and `*` consumes the empty string or any run. Both strings can have length 2000, and the first move on `*` is always dfs(i+1, j), so that chain is as deep as |s| and exceeds the default recursion limit. The same index pair is asked repeatedly, and whether two prefixes match depends only on shorter prefixes. Let f[i][j] mean the first i characters of s match the first j characters of p. The empty pair is true, and a run of `*` on the pattern inherits along j. Filling i and j from small to large works because a `*` reads the cell above, the cell to the left, and the diagonal, while a literal or `?` reads only the diagonal.

Define f[i][j] to represent whether the first i characters of string s match the first j characters of string p. Initially, f[0][0] = true, indicating that two empty strings are matching. For $j \in [1, n]$, if p[j-1] = '*', then f[0][j] = f[0][j-1].

Next, we consider the case of $i \in [1, m]$ and $j \in [1, n]$:

- If p[j-1] = '*', then f[i][j] = f[i-1][j] ∨ f[i][j-1] ∨ f[i-1][j-1].
- Otherwise, f[i][j] = (p[j-1] = '?' ∨ s[i-1] = p[j-1]) ∧ f[i-1][j-1].

The final answer is f[m][n].

The time complexity is O(m × n), and the space complexity is O(m × n). Where m and n are the lengths of the strings s and p, respectively.

Python 3

```python
class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m, n = len(s), len(p)
        f = [[False] * (n + 1) for _ in range(m + 1)]
        f[0][0] = True
        for j in range(1, n + 1):
            if p[j - 1] == "*":
                f[0][j] = f[0][j - 1]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if p[j - 1] == "*":
                    f[i][j] = f[i - 1][j] or f[i][j - 1] or f[i - 1][j - 1]
                else:
                    f[i][j] = f[i - 1][j - 1] and (
                        p[j - 1] == "?" or s[i - 1] == p[j - 1]
                    )
        return f[m][n]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. O(1) extra space: a greedy two-pointer match
# 2. Match many texts against one pattern: cut the pattern
# at the stars


# ============================================================
# FOLLOW-UP 1: O(1) extra space: a greedy two-pointer
# match
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember the last '*' and the text position it
# started matching from. On a mismatch go back to the
# star, let it swallow one more character, and continue.
# Time: O(n * m) in the worst case, O(n + m) typically.
# Space: O(1).
def is_match_greedy(s, p):
    i = j = 0
    star = -1
```

```
    mark = 0
    while i < len(s):
        if j < len(p) and (p[j] == s[i] or p[j] == "?"):
            i += 1
            j += 1
        elif j < len(p) and p[j] == "*":
            star = j
            mark = i
            j += 1
        elif star != -1:
            j = star + 1
            mark += 1
            i = mark
        else:
            return False
    while j < len(p) and p[j] == "*":
        j += 1
    return j == len(p)


# ========================================================
# FOLLOW-UP 2: Match many texts against one pattern: cut
# the pattern at the stars
# ========================================================

# ---- follow-up 2: function ----
# Answer: Split the pattern into pieces between the stars.
# The first piece must match the start, the last piece the
# end, and the middle pieces must appear in order; '?' can
# be handled by checking a piece character by character.
# Time: O(n * m) in the worst case.  Space: O(m).
def piece_matches_at(text, piece, position):
    if position + len(piece) > len(text):
        return False
    for k in range(len(piece)):
        if piece[k] != "?" and piece[k] != text[position + k]:
            return False
    return True
```

```
def match_by_segments(text, pattern):
    pieces = pattern.split("*")
    if len(pieces) == 1:
        return len(text) == len(pattern) and piece_matches_at(text, pattern, 0)
    first, last = pieces[0], pieces[-1]
    if len(text) < len(first) + len(last):
        return False
    if not piece_matches_at(text, first, 0):
        return False
    if not piece_matches_at(text, last, len(text) - len(last)):
        return False
    position = len(first)
    limit = len(text) - len(last)
    for piece in pieces[1:-1]:
        found = False
        while position + len(piece) <= limit:
            if piece_matches_at(text, piece, position):
                found = True
                position += len(piece)
                break
            position += 1
        if not found:
            return False
    return True
```

10. Regular Expression Matching

Hard · Recursion, String, Dynamic Programming

leetcode.com/problems/regular-expression-matching

★ **TOP 7 PROBLEM** in String Matching / Pattern Search: master this one first

PROBLEM

Given an input string `s` and a pattern `p`, implement regular expression matching with support for `'.'` and `'*'` where:

- `'.'` Matches any single character.
- `'*'` Matches zero or more of the preceding element.

Return a boolean indicating whether the matching covers the entire input string (not partial).

Example 1:

```
Input: s = "aa", p = "a"
Output: false
Explanation: "a" does not match the entire string "aa".
```

Example 2:

```
Input: s = "aa", p = "a*"
Output: true
Explanation: '*' means zero or more of the preceding element, 'a'. Therefore, by repeating 'a' once,
it becomes "aa".
```

Example 3:

```
Input: s = "ab", p = ".*"
Output: true
Explanation: ".*" means "zero or more (*) of any character (.)".
```

Constraints:

- `1 <= s.length <= 20`
- `1 <= p.length <= 20`
- `s` contains only lowercase English letters.
- `p` contains only lowercase English letters, `'.'`, and `'*'`.
- It is guaranteed for each appearance of the character `'*'`, there will be a previous valid character to match.

SOLUTIONS (PYTHON 3)

Solution 1: Memoization Search

How to think about it

Matching left to right and, at each `*`, trying 0,1,2,… repetitions is the natural search. s and p are at most length 20, so exponential time sometimes passes, but the same pair of positions is recomputed over and over. A `*` applies only to the preceding token: either match it 0 times (skip two pattern indices), or the current character fits that token (including `.`) and the same `*` consumes the next character of s. A plain character must be consumed one-for-one. The state is therefore “can suffix i of s match suffix j of p”. Search with memoization so exponential branches collapse to O(mn) pairs (i,j).

We design a function dfs(i, j), which indicates whether the i-th character of s matches the j-th character of p. The answer is dfs(0, 0).

The calculation process of the function dfs(i, j) is as follows:

- If j has reached the end of p, then if i has also reached the end of s, the match is successful, otherwise, the match fails.
- If the next character of j is `'*'`, we can choose to match 0 s[i] characters, which is dfs(i, j + 2). If i < m and s[i] matches p[j], we can choose to match 1 s[i] character, which is dfs(i + 1, j).
- If the next character of j is not `'*'`, then if i < m and s[i] matches p[j], it is dfs(i + 1, j + 1). Otherwise, the match fails.

During the process, we can use memoization search to avoid repeated calculations.

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the lengths of s and p respectively.

Python 3

```
class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        @cache
        def dfs(i, j):
            if j >= n:
                return i == m
            if j + 1 < n and p[j + 1] == '*':
                return dfs(i, j + 2) or (
                    i < m and (s[i] == p[j] or p[j] == '.') and dfs(i + 1, j)
                )
            return i < m and (s[i] == p[j] or p[j] == '.') and dfs(i + 1, j + 1)

        m, n = len(s), len(p)
        return dfs(0, 0)
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> Memoized Solution 1 is already O(mn), but it is still recursion: implicit stack depth and larger constants. The transitions have no aftereffect, so we can fill i,j in increasing order. f[i][j] is whether the first i characters of s match the first j of p. Empty matches empty. We also need i=0 so patterns like `a*` can match the empty string, which is why the outer loop starts from the empty prefix. The `*` cases are the same as Solution 1: skip two pattern characters, or consume one more character of s when it fits. After filling the table we read f[m][n].

We can convert the memoization search in Solution 1 into dynamic programming.

Define f[i][j] to represent whether the first i characters of string s match the first j characters of string p. The answer is f[m][n]. Initialize f[0][0] = true, indicating that the empty string and the empty regular expression match.

Similar to Solution 1, we can discuss different cases.

- If p[j - 1] is `'*'`, we can choose to match 0 s[i - 1] characters, which is f[i][j] = f[i][j - 2]. If s[i - 1] matches p[j - 2], we can choose to match 1 s[i - 1] character, which is f[i][j] = f[i][j] ∨ f[i - 1][j].
- If p[j - 1] is not `'*'`, then if s[i - 1] matches p[j - 1], it is f[i][j] = f[i - 1][j - 1]. Otherwise, the match fails.

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the lengths of s and p respectively.

Python 3

```
class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m, n = len(s), len(p)
        f = [[False] * (n + 1) for _ in range(m + 1)]
        f[0][0] = True
        for i in range(m + 1):
            for j in range(1, n + 1):
                if p[j - 1] == "*":
                    f[i][j] = f[i][j - 2]
                    if i > 0 and (p[j - 2] == "." or s[i - 1] == p[j - 2]):
                        f[i][j] |= f[i - 1][j]
                elif i > 0 and (p[j - 1] == "." or s[i - 1] == p[j - 1]):
                    f[i][j] = f[i - 1][j - 1]
        return f[m][n]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Also support '+' (one or more) and '?' (zero or one)
# 2. Bottom-up table instead of recursion


# ============================================================
# FOLLOW-UP 1: Also support '+' (one or more) and '?'
# (zero or one)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Treat the operator after a character as a repeat
# rule: '*' may repeat 0 or more times, '+' 1 or more, '?'
# 0 or 1. Try the options with recursion and remember
# results.
# Time: O(n * m).  Space: O(n * m).
def is_match_extended(s, p):
    memo = {}

    def match(i, j):
        if (i, j) in memo:
            return memo[(i, j)]
        if j == len(p):
            answer = i == len(s)
        else:
            first = i < len(s) and (p[j] == "." or p[j] == s[i])
            operator = p[j + 1] if j + 1 < len(p) else ""
            if operator == "*":
                answer = match(i, j + 2) or (first and match(i + 1, j))
            elif operator == "+":
                answer = first and (match(i + 1, j + 2) or match(i + 1, j))
            elif operator == "?":
                answer = match(i, j + 2) or (first and match(i + 1, j + 2))
            else:
                answer = first and match(i + 1, j + 1)
        memo[(i, j)] = answer
        return answer

    return match(0, 0)


# ============================================================
# FOLLOW-UP 2: Bottom-up table instead of recursion
# ============================================================

# ---- follow-up 2: function ----
# Answer: match[i][j] says whether the last i letters of s
# match the last j symbols of p; fill the table from the
# end of both strings.
# Time: O(n * m).  Space: O(n * m).
def is_match_table(s, p):
    n, m = len(s), len(p)
    match = []
    for i in range(n + 1):
        match.append([False] * (m + 1))
    match[n][m] = True
    for i in range(n, -1, -1):
        for j in range(m - 1, -1, -1):
            first = i < n and (p[j] == "." or p[j] == s[i])
            if j + 1 < m and p[j + 1] == "*":
                match[i][j] = match[i][j + 2] or (first and match[i + 1][j])
            else:
                match[i][j] = first and match[i + 1][j + 1]
```

```
    return match[0][0]
```

1392. Longest Happy Prefix

Hard · String, String Matching, Hash Function, Rolling Hash, KMP, Extended KMP

leetcode.com/problems/longest-happy-prefix

★ **TOP 7 PROBLEM** in String Matching / Pattern Search: master this one first

PROBLEM

A string is called a **happy prefix** if it is a **non-empty** prefix which is also a suffix (excluding itself).

Given a string `s`, return *the **longest happy prefix** of* `s`. Return an empty string `""` if no such prefix exists.

Example 1:

```
Input: s = "level"
Output: "l"
Explanation: s contains 4 prefix excluding itself ("l", "le", "lev", "leve"), and suffix ("l", "el", "vel", "evel"). The largest prefix which is also suffix is given by "l".
```

Example 2:

```
Input: s = "ababab"
Output: "abab"
Explanation: "abab" is the largest prefix which is also suffix. They can overlap in the original string.
```

Constraints:

- `1 <= s.length <= 10^5`
- `s` contains only lowercase English letters.

SOLUTIONS (PYTHON 3)

Solution 1: String Hashing

How to think about it

The longest proper prefix that is also a suffix. Comparing every length from short to long is quadratic for n ≤ 10^5. Starting from the longest proper prefix, s[:-i]==s[i:] yields that prefix as soon as it holds; otherwise the answer is empty.

String Hashing is a method to map a string of any length to a non-negative integer, with the probability of collision being almost zero. String hashing is used to calculate the hash value of a string, which allows for quick determination of whether two strings are equal.

We choose a fixed value BASE, and consider the string as a number in BASE radix, assigning a value greater than 0 to represent each character. Generally, the values we assign are much smaller than BASE. For example, for strings composed of lowercase letters, we can assign a=1, b=2, ..., z=26. We choose a fixed value MOD, and calculate the remainder of the BASE radix number divided by MOD, which is used as the hash value of the string.

Generally, we choose BASE=131 or BASE=13331, at which point the probability of hash value collision is extremely low. As long as the hash values of two strings are the same, we consider the two strings to be equal. Usually, MOD is chosen as 2^{64}. In C++, we can directly use the unsigned long long type to store this hash value. When calculating, we do not handle arithmetic overflow. When overflow occurs, it is equivalent to automatically taking the modulus of 2^{64}, which can avoid inefficient modulus operations.

Except for extremely specially constructed data, the above hash algorithm is unlikely to produce collisions. In general, the above hash algorithm can appear in the standard answers of the problem. We can also choose some appropriate values of BASE and MOD (such as large prime numbers), and perform several groups of hash operations. Only when the results are all the same do we consider the original strings to be equal, making it even more difficult to construct data that causes this hash to produce errors.

Python 3

```python
class Solution:
    def longestPrefix(self, s: str) -> str:
        for i in range(1, len(s)):
            if s[:-i] == s[i:]:
                return s[i:]
        return ''
```

Complexity. Time: O(n^2) (each slice comparison costs O(n)). Space: O(n).

Solution 2: KMP Algorithm

> **How to think about it**
>
> Comparing whole slices still scans Θ(n) characters per length. KMP's next table is exactly the longest proper prefix that equals a suffix. Appending a sentinel and reading next[-1] returns that length in linear time.

According to the problem description, we need to find the longest happy prefix of a string, which is the longest prefix of the string that is also a suffix of the string. We can use the KMP algorithm to solve this problem.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the string.

Python 3

```python
class Solution:
    def longestPrefix(self, s: str) -> str:
        s += "#"
        n = len(s)
        next = [0] * n
        next[0] = -1
        i, j = 2, 0
        while i < n:
            if s[i - 1] == s[j]:
                j += 1
                next[i] = j
                i += 1
            elif j:
                j = next[j]
            else:
                next[i] = 0
                i += 1
        return s[: next[-1]]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The longest prefix that appears at least TWICE
# anywhere in the string (overlap allowed)
# 2. How many times does each prefix occur in the whole
# string?


# ============================================================
# FOLLOW-UP 1: The longest prefix that appears at least
# TWICE anywhere in the string (overlap allowed)
# ============================================================

# ---- follow-up 1: helper functions ----
def prefix_function(p):
    border = [0] * len(p)
    k = 0
    for i in range(1, len(p)):
        while k > 0 and p[i] != p[k]:
            k = border[k - 1]
        if p[i] == p[k]:
```

```
            k += 1
        border[i] = k
    return border

# ---- follow-up 1: function ----
# Answer: Every prefix function value b means the prefix
# of length b ends at that position, so a prefix that
# appears again somewhere has a value somewhere in the
# array: take the largest value that is a border of an
# earlier position, which is simply the maximum of the
# prefix function (excluding the case of the whole
# string).
# Time: O(n).  Space: O(n).
def longest_repeated_prefix(s):
    border = prefix_function(s)
    best = 0
    for value in border:
        if value > best:
            best = value
    return s[:best]


# ============================================================
# FOLLOW-UP 2: How many times does each prefix occur in
# the whole string?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Every position i with border value b adds one
# occurrence to the prefix of length b. Then push the
# counts down the border chain: a prefix also occurs
# wherever a longer prefix that ends with it occurs.
# Time: O(n).  Space: O(n).
def prefix_occurrences(s):
    border = prefix_function(s)
    n = len(s)
    count = [0] * (n + 2)
    for i in range(n):
        count[border[i]] += 1
    for length in range(n, 0, -1):
        count[border[length - 1]] += count[length]
    result = []
    for length in range(1, n + 1):
        result.append(count[length] + 1)             # + 1 for the prefix itself
    return result
```

1316. Distinct Echo Substrings

Hard · Trie, String, Suffix Array, Suffix Tree, Hash Function, Rolling Hash, Suffix Automato

leetcode.com/problems/distinct-echo-substrings

PROBLEM

Return the number of **distinct** non-empty substrings of `text` that can be written as the concatenation of some string with itself (i.e. it can be written as `a + a` where `a` is some string).

Example 1:

```
Input: text = "abcabcabc"
Output: 3
Explanation: The 3 substrings are "abcabc", "bcabca" and "cabcab".
```

Example 2:

```
Input: text = "leetcodeleetcode"
Output: 2
Explanation: The 2 substrings are "ee" and "leetcodeleetcode".
```

Constraints:

- `1 <= text.length <= 2000`
- `text` has only lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Count distinct substrings of the form xx. For n ≤ 2000, comparing both halves of every even-length substring character-wise is nearly $O(n^3)$. Equality of two pieces is O(1) with string hashes. After a prefix-hash build, every even interval is tested by hashing its two halves; a set of those hashes is the number of distinct echo substrings.

Python 3

```python
class Solution:
    def distinctEchoSubstrings(self, text: str) -> int:
        def get(l, r):
            return (h[r] - h[l - 1] * p[r - l + 1]) % mod

        n = len(text)
        base = 131
        mod = int(1e9) + 7
        h = [0] * (n + 10)
        p = [1] * (n + 10)
        for i, c in enumerate(text):
            t = ord(c) - ord('a') + 1
            h[i + 1] = (h[i] * base) % mod + t
            p[i + 1] = (p[i] * base) % mod
        vis = set()
        for i in range(n - 1):
            for j in range(i + 1, n, 2):
                k = (i + j) >> 1
                a = get(i + 1, k + 1)
                b = get(k + 2, j + 1)
                if a == b:
                    vis.add(a)
        return len(vis)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the distinct echo substrings themselves
# 2. The LONGEST echo substring


# ============================================================
# FOLLOW-UP 1: Return the distinct echo substrings
# themselves
# ============================================================

# ---- follow-up 1: function ----
# Answer: For every length L and start i check whether the
# L characters at i equal the L characters after them;
# collect the substrings in a set to remove repeats.
# Time: O(n^3) (n^2 candidate lengths and positions, O(n)
# comparison).  Space: O(n^2) for the set.
def distinct_echo_substrings_list(text):
    found = set()
    n = len(text)
    for half in range(1, n // 2 + 1):
        for start in range(n - 2 * half + 1):
            if text[start:start + half] == text[start + half:start + 2 * half]:
                found.add(text[start:start + 2 * half])
    return sorted(found)


# ============================================================
# FOLLOW-UP 2: The LONGEST echo substring
# ============================================================

# ---- follow-up 2: function ----
# Answer: Check the halves from the longest possible
# length down; the first length that has an echo gives the
# answer.
# Time: O(n^3).  Space: O(n).
def longest_echo_substring(text):
    n = len(text)
    for half in range(n // 2, 0, -1):
        for start in range(n - 2 * half + 1):
            if text[start:start + half] == text[start + half:start + 2 * half]:
                return text[start:start + 2 * half]
    return ""
```

1032. Stream of Characters

Hard · Design, Trie, Array, String, Data Stream, Aho-Corasick

leetcode.com/problems/stream-of-characters

★ **TOP 7 PROBLEM** in String Matching / Pattern Search: master this one first

Also appears in pattern 32 Trie (Prefix Tree)

PROBLEM

Design an algorithm that accepts a stream of characters and checks if a suffix of these characters is a string of a given array of strings `words`.

For example, if `words = ["abc", "xyz"]` and the stream added the four characters (one by one) `'a'`, `'x'`, `'y'`, and `'z'`, your algorithm should detect that the suffix `"xyz"` of the characters `"axyz"` matches `"xyz"` from `words`.

Implement the `StreamChecker` class:

- `StreamChecker(String[] words)` Initializes the object with the strings array `words`.
- `boolean query(char letter)` Accepts a new character from the stream and returns `true` if any non-empty suffix from the stream forms a word that is in `words`.

Example 1:

```
Input
["StreamChecker", "query", "query", "query", "query", "query", "query", "query", "query", "query",
"query", "query", "query"]
[[["cd", "f", "kl"]], ["a"], ["b"], ["c"], ["d"], ["e"], ["f"], ["g"], ["h"], ["i"], ["j"], ["k"],
["l"]]
Output
[null, false, false, false, true, false, true, false, false, false, false, false, true]

Explanation
StreamChecker streamChecker = new StreamChecker(["cd", "f", "kl"]);
streamChecker.query("a"); // return False
streamChecker.query("b"); // return False
streamChecker.query("c"); // return False
streamChecker.query("d"); // return True, because 'cd' is in the wordlist
streamChecker.query("e"); // return False
streamChecker.query("f"); // return True, because 'f' is in the wordlist
streamChecker.query("g"); // return False
streamChecker.query("h"); // return False
streamChecker.query("i"); // return False
streamChecker.query("j"); // return False
streamChecker.query("k"); // return False
streamChecker.query("l"); // return True, because 'kl' is in the wordlist
```

Constraints:

- `1 <= words.length <= 2000`
- `1 <= words[i].length <= 200`
- `words[i]` consists of lowercase English letters.
- `letter` is a lowercase English letter.
- At most $4 * 10^4$ calls will be made to query.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Matching the whole stream against every word on each query repeats work: up to 4× 10^4 queries and a large total pattern length. We only need to know whether the stream currently ends with some word. Inserting the reversed words into a trie turns suffix queries into prefix walks. Words have length at most 200, so only a bounded tail of the stream matters. The constructor builds that reverse trie; each query appends the letter and walks at most 201 steps, succeeding at an end mark.

Python 3

```python
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, w: str):
        node = self
        for c in w[::-1]:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, w: List[str]) -> bool:
        node = self
        for c in w[::-1]:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                return False
            node = node.children[idx]
            if node.is_end:
                return True
        return False


class StreamChecker:
    def __init__(self, words: List[str]):
        self.trie = Trie()
        self.cs = []
        self.limit = 201
        for w in words:
            self.trie.insert(w)

    def query(self, letter: str) -> bool:
        self.cs.append(letter)
        return self.trie.search(self.cs[-self.limit :])


# Your StreamChecker object will be instantiated and called as such:
# obj = StreamChecker(words)
# param_1 = obj.query(letter)
```

Complexity. Time: build O(L), L = total length of the words; each query O(m), m = length of the longest word. Space: O(L).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Words can be added to and removed from the dictionary
# while the stream runs
# 2. Report ALL words that end at the newest letter


# ============================================================
# FOLLOW-UP 1: Words can be added to and removed from the
# dictionary while the stream runs
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep a reverse trie with a counter on the end
# node of each word. Removing a word decreases the
# counter; the query walks the recent letters backwards
# through the trie and succeeds while any word still ends
# on the way.
# Time: add_word and remove_word O(L); query O(m), m =
# longest word.  Space: O(total letters + stream length).
class StreamWithRemoval:
    def __init__(self, words):
        self.root = {}
        self.longest = 0
        self.letters = []
        for word in words:
            self.add_word(word)

    def add_word(self, word):
        node = self.root
        for ch in reversed(word):
            if ch not in node:
                node[ch] = {}
            node = node[ch]
        node["$"] = node.get("$", 0) + 1
        if len(word) > self.longest:
            self.longest = len(word)

    def remove_word(self, word):
        node = self.root
        for ch in reversed(word):
            if ch not in node:
                return
            node = node[ch]
        if node.get("$", 0) > 0:
            node["$"] -= 1

    def query(self, letter):
        self.letters.append(letter)
        node = self.root
        for i in range(len(self.letters) - 1, max(-1, len(self.letters) - 1 - self.longest), -1):
            ch = self.letters[i]
            if ch not in node:
                return False
            node = node[ch]
            if node.get("$", 0) > 0:
                return True
        return False


# ============================================================
```

```
# FOLLOW-UP 2: Report ALL words that end at the newest
# letter
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: The same backwards walk through the reverse
# trie, but collect every word whose end marker is passed
# (store the word itself on its end node).
# Time: query O(m + r), r = words reported.  Space:
# O(total letters + stream length).
class StreamReportAll:
    def __init__(self, words):
        self.root = {}
        self.longest = 0
        self.letters = []
        for word in words:
            node = self.root
            for ch in reversed(word):
                if ch not in node:
                    node[ch] = {}
                node = node[ch]
            node["$"] = word
            if len(word) > self.longest:
                self.longest = len(word)

    def query(self, letter):
        self.letters.append(letter)
        node = self.root
        found = []
        for i in range(len(self.letters) - 1, max(-1, len(self.letters) - 1 - self.longest), -1):
            ch = self.letters[i]
            if ch not in node:
                break
            node = node[ch]
            if "$" in node:
                found.append(node["$"])
        return found
```

PATTERN 7 · STRING MATCHING / PATTERN SEARCH · PROBLEM 19 OF 20

336. Palindrome Pairs

Hard · Trie, Array, Hash Table, String, Hash Function

leetcode.com/problems/palindrome-pairs

Also appears in pattern 32 Trie (Prefix Tree)

PROBLEM

You are given a **0-indexed** array of **unique** strings `words`.

A **palindrome pair** is a pair of integers `(i, j)` such that:

- `0 <= i, j < words.length`,
- `i != j`, and
- `words[i] + words[j]` (the concatenation of the two strings) is a palindrome.

Return *an array of all the **palindrome pairs** of* `words`.

You must write an algorithm with `O(sum of words[i].length)` runtime complexity.

Example 1:

```
Input: words = ["abcd","dcba","lls","s","sssll"]
Output: [[0,1],[1,0],[3,2],[2,4]]
Explanation: The palindromes are ["abcddcba","dcbaabcd","slls","llssssll"]
```

Example 2:

```
Input: words = ["bat","tab","cat"]
Output: [[0,1],[1,0]]
Explanation: The palindromes are ["battab","tabbat"]
```

Example 3:

```
Input: words = ["a",""]
Output: [[0,1],[1,0]]
Explanation: The palindromes are ["a","a"]
```

Constraints:

- `1 <= words.length <= 5000`
- `0 <= words[i].length <= 300`
- `words[i]` consists of lowercase English letters.

SOLUTIONS (PYTHON 3)

Solution 1: Hash Map

How to think about it

Two words should concatenate to a palindrome. Testing every ordered pair is too slow. Split a word: if one side is already a palindrome, the reverse of the other side, if present, completes a pair. Index every word in a hash map. For each cut of w, a palindromic prefix looks up the reversed suffix, and a palindromic suffix looks up the reversed prefix. The empty cut is used on only one side. The code checks both sides at every cut.

Python 3

```python
class Solution:
    def palindromePairs(self, words: List[str]) -> List[List[int]]:
        d = {w: i for i, w in enumerate(words)}
        ans = []
        for i, w in enumerate(words):
            for j in range(len(w) + 1):
                a, b = w[:j], w[j:]
                ra, rb = a[::-1], b[::-1]
                if ra in d and d[ra] != i and b == rb:
                    ans.append([i, d[ra]])
                if j and rb in d and d[rb] != i and a == ra:
                    ans.append([d[rb], i])
        return ans
```

Complexity. Time: O(n * m^2), m = length of the longest word. Space: O(n * m).

Solution 2

How to think about it

Method 1 builds a reversed string for every cut. Inserting words into a trie lets us walk the other word backwards and read an index at a node, without allocating the reverse. When the remaining side is a palindrome we take the stored index; the matches are the same.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```python
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.v = -1

    def insert(self, w: str, i: int) -> None:
        node = self
        for c in w:
            k = ord(c) - 97
            if node.children[k] is None:
                node.children[k] = Trie()
            node = node.children[k]
        node.v = i

    def search(self, w: str, i: int, j: int) -> int:
        # walk the letters w[j], w[j - 1], ..., w[i] (the word read backwards)
        node = self
        for k in range(j, i - 1, -1):
            idx = ord(w[k]) - 97
            if node.children[idx] is None:
                return -1
            node = node.children[idx]
        return node.v


class Solution:
    def palindromePairs(self, words: List[str]) -> List[List[int]]:
        def is_palindrome(w: str, i: int, j: int) -> bool:
            while i < j:
                if w[i] != w[j]:
                    return False
                i += 1
                j -= 1
            return True

        trie = Trie()
        for i, w in enumerate(words):
            trie.insert(w, i)
        ans = []
        for i, w in enumerate(words):
            m = len(w)
            for j in range(m + 1):
                if is_palindrome(w, j, m - 1):
                    k = trie.search(w, 0, j - 1)
                    if k != -1 and k != i:
                        ans.append([i, k])
                if j and is_palindrome(w, 0, j - 1):
                    k = trie.search(w, j, m - 1)
                    if k != -1 and k != i:
                        ans.append([k, i])
        return ans
```

Complexity. Time: O(n * m^2), m = length of the longest word. Space: O(n * m).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Find the partners for ONE given word
# 2. The longest palindrome that can be made by joining
# two different words


# ========================================================
# FOLLOW-UP 1: Find the partners for ONE given word
# ========================================================
```

```
# ---- follow-up 1: function ----
# Answer: Try both joins with every other word and keep
# the ones that read the same backwards: O(n * L). (The
# dictionary trick of the main question looks up the
# reverse of a piece of the word, which only finds
# partners that are not longer than the word, so for a
# single word a plain scan is the safe answer.)
# Time: O(N * L), N words of length L.  Space: O(L).
def is_palindrome(text):
    i, j = 0, len(text) - 1
    while i < j:
        if text[i] != text[j]:
            return False
        i += 1
        j -= 1
    return True


def partners_of(word, words):
    partners = []
    for other in words:
        if other == word:
            continue
        if is_palindrome(other + word):
            partners.append((other, word))
        if is_palindrome(word + other):
            partners.append((word, other))
    return partners


# ============================================================
# FOLLOW-UP 2: The longest palindrome that can be made by
# joining two different words
# ============================================================

# ---- follow-up 2: function ----
# Answer: Check every pair: only concatenations whose
# length beats the best so far need the palindrome test.
# Time: O(N^2 * L).  Space: O(L).
def longest_palindrome_pair(words):
    best = ""
    for i in range(len(words)):
        for j in range(len(words)):
            if i != j:
                joined = words[i] + words[j]
                if len(joined) > len(best) and is_palindrome(joined):
                    best = joined
    return best
```

472. Concatenated Words

Hard · Depth-First Search, Trie, Array, String, Dynamic Programming, Sorting

leetcode.com/problems/concatenated-words

Also appears in pattern 32 Trie (Prefix Tree)

PROBLEM

Given an array of strings `words` (**without duplicates**), return *all the **concatenated words** in the given list of* `words`.

A **concatenated word** is defined as a string that is comprised entirely of at least two shorter words (not necessarily distinct) in the given array.

Example 1:

```
Input: words = ["cat","cats","catsdogcats","dog","dogcatsdog","hippopotamuses","rat","ratcatdogcat"]
Output: ["catsdogcats","dogcatsdog","ratcatdogcat"]
Explanation: "catsdogcats" can be concatenated by "cats", "dog" and "cats";
"dogcatsdog" can be concatenated by "dog", "cats" and "dog";
"ratcatdogcat" can be concatenated by "rat", "cat", "dog" and "cat".
```

Example 2:

```
Input: words = ["cat","dog","catdog"]
Output: ["catdog"]
```

Constraints:

- `1 <= words.length <= 10^4`
- `1 <= words[i].length <= 30`
- `words[i]` consists of only lowercase English letters.
- All the strings of `words` are **unique**.
- `1 <= sum(words[i].length) <= 10^5`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

A concatenated word is at least two shorter dictionary words. Trying every split and a set lookup grows with both n and length. Sort by length and insert shorter words into a trie first. DFS the current word on the trie: at an end-of-word node recurse on the suffix; if the whole word splits, it is concatenated, otherwise insert it. Concatenated words stay out of the trie, since a longer word can always fall back to atomic pieces. The empty suffix is the success base case.

Python 3

```python
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, w):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True


class Solution:
    def findAllConcatenatedWordsInADict(self, words: List[str]) -> List[str]:
        def dfs(w):
            if not w:
                return True
            node = trie
            for i, c in enumerate(w):
                idx = ord(c) - ord('a')
                if node.children[idx] is None:
                    return False
                node = node.children[idx]
                if node.is_end and dfs(w[i + 1 :]):
                    return True
            return False

        trie = Trie()
        ans = []
        words.sort(key=lambda x: len(x))
        for w in words:
            if dfs(w):
                ans.append(w)
            else:
                trie.insert(w)
        return ans
```

Complexity. Time: O(n * m^2), m = length of the longest word. Space: O(n * m).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the longest concatenated word
# 2. Words made from EXACTLY two other words


# ============================================================
# FOLLOW-UP 1: Return the longest concatenated word
# ============================================================

# ---- follow-up 1: function ----
# Answer: Find all concatenated words with the usual
# check, then keep the longest.
# Time: O(N * L^3) (L^2 substrings, each hashed in O(L)),
# after an O(N log N) sort.  Space: O(N * L).
def can_be_built(word, smaller):
    if word == "":
        return False
    reachable = [False] * (len(word) + 1)
    reachable[0] = True
    for end in range(1, len(word) + 1):
        for start in range(end):
            if reachable[start] and word[start:end] in smaller:
```

```
                reachable[end] = True
                break
    return reachable[len(word)]


def longest_concatenated_word(words):
    words = sorted(words, key=len)
    seen = set()
    best = ""
    for word in words:
        if can_be_built(word, seen):
            if len(word) > len(best):
                best = word
        seen.add(word)
    return best


# ============================================================
# FOLLOW-UP 2: Words made from EXACTLY two other words
# ============================================================

# ---- follow-up 2: function ----
# Answer: Try every split of the word into a prefix and a
# suffix and check that both are in the list.
# Time: O(N * L^2).  Space: O(1) extra.
def words_from_exactly_two(words):
    known = set(words)
    result = []
    for word in words:
        for cut in range(1, len(word)):
            if word[:cut] in known and word[cut:] in known:
                result.append(word)
                break
    return result
```

PATTERN 8 OF 41

Stack

20 problems · 6 Easy · 13 Medium · 1 Hard

THE IDEA

Last in, first out. Use it when the most recent unfinished thing must be resolved first: brackets, nested structures, undo, expression evaluation.

Signals: valid parentheses, decode string, evaluate an expression, simplify a path, remove adjacent duplicates, next greater element.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
stack = []
for ch in s:
    if ch in "([{":
        stack.append(ch)
    else:
        if not stack or pairs[ch] != stack.pop():
            return False
return not stack
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Valid Parentheses (see page)	Easy	LC 20	292
2	Remove All Adjacent Duplicates in String	Easy	LC 1047	388
3	Make The String Great	Easy	LC 1544	390
4	Next Greater Element I	Easy	LC 496	392
5	Backspace String Compare (see page)	Easy	LC 844	147
6	Build a String From Another String — substitute: Baseball Game	Easy	LC 682	395
7	★ **Min Stack**	Medium	LC 155	398
8	★ **Evaluate Reverse Polish Notation**	Medium	LC 150	401
9	★ **Generate Parentheses**	Medium	LC 22	404
10	★ **Daily Temperatures**	Medium	LC 739	407
11	★ **Decode String** (see page)	Medium	LC 394	338
12	Asteroid Collision	Medium	LC 735	409
13	Simplify Path (see page)	Medium	LC 71	320
14	★ **Basic Calculator II**	Medium	LC 227	412
15	Remove All Adjacent Duplicates in String II	Medium	LC 1209	416
16	Score of Parentheses	Medium	LC 856	419
17	Exclusive Time of Functions	Medium	LC 636	422
18	Minimum Add to Make Parentheses Valid	Medium	LC 921	426
19	Validate Stack Sequences	Medium	LC 946	429
20	★ **Basic Calculator** (see page)	Hard	LC 224	344

PATTERN 8 · STACK · PROBLEM 2 OF 20

1047. Remove All Adjacent Duplicates In String

Easy · Stack, String

leetcode.com/problems/remove-all-adjacent-duplicates-in-string

PROBLEM

You are given a string `s` consisting of lowercase English letters. A **duplicate removal** consists of choosing two **adjacent** and **equal** letters and removing them.

We repeatedly make **duplicate removals** on `s` until we no longer can.

Return *the final string after all such duplicate removals have been made*. It can be proven that the answer is **unique**.

Example 1:

```
Input: s = "abbaca"
Output: "ca"
Explanation:
For example, in "abbaca" we could remove "bb" since the letters are adjacent and equal, and this is the only possible move. The result of this move is that the string is "aaca", of which only "aa" is possible, so the final string is "ca".
```

Example 2:

```
Input: s = "azxxzy"
Output: "ay"
```

Constraints:

- `1 <= s.length <= 10^5`
- `s` consists of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Repeatedly deleting adjacent equal pairs may rescan a string of length 10^5. After a deletion the new adjacency is local, so a stack can keep the compressed prefix. A character equal to the top is popped; otherwise it is pushed. The stack at the end is the fully reduced string.

Python 3

```python
class Solution:
    def removeDuplicates(self, s: str) -> str:
        stk = []
        for c in s:
            if stk and stk[-1] == c:
                stk.pop()
            else:
                stk.append(c)
        return ''.join(stk)
```

Complexity. Time: O(n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Characters arrive one at a time: always know the
# current reduced string
# 2. Count how many pairs were removed


# ============================================================
# FOLLOW-UP 1: Characters arrive one at a time: always
# know the current reduced string
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the stack between calls: a new character
# either cancels the top one or is pushed.
# Time: O(k) per add, k = current length (the string is
# rebuilt); O(1) if you only keep the stack.  Space: O(n).
class ReducingStream:
    def __init__(self):
        self.stack = []

    def add(self, ch):
        if self.stack and self.stack[-1] == ch:
            self.stack.pop()
        else:
            self.stack.append(ch)
        return "".join(self.stack)


# ============================================================
# FOLLOW-UP 2: Count how many pairs were removed
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same scan; every time the new character
# cancels the top one, count one removed pair.
# Time: O(n).  Space: O(n).
def count_removed_pairs(s):
    stack = []
    pairs = 0
    for ch in s:
        if stack and stack[-1] == ch:
            stack.pop()
            pairs += 1
        else:
            stack.append(ch)
    return pairs
```

PATTERN 8 · STACK · PROBLEM 3 OF 20

1544. Make The String Great

Easy · Stack, String

leetcode.com/problems/make-the-string-great

PROBLEM

Given a string `s` of lower and upper case English letters.

A good string is a string which doesn't have **two adjacent characters** `s[i]` and `s[i + 1]` where:

- `0 <= i <= s.length - 2`
- `s[i]` is a lower-case letter and `s[i + 1]` is the same letter but in upper-case or **vice-versa**.

To make the string good, you can choose **two adjacent** characters that make the string bad and remove them. You can keep doing this until the string becomes good.

Return *the string* after making it good. The answer is guaranteed to be unique under the given constraints.

Notice that an empty string is also good.

Example 1:

```
Input: s = "leEeetcode"
Output: "leetcode"
Explanation: In the first step, either you choose i = 1 or i = 2, both will result "leEeetcode" to be reduced to "leetcode".
```

Example 2:

```
Input: s = "abBAcC"
Output: ""
Explanation: We have many possible scenarios, and all lead to the same answer. For example:
"abBAcC" --> "aAcC" --> "cC" --> ""
"abBAcC" --> "abBA" --> "aA" --> ""
```

Example 3:

```
Input: s = "s"
Output: "s"
```

Constraints:

- `1 <= s.length <= 100`
- `s` contains only lower and upper case English letters.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

A “great” string has no adjacent letters that differ only in case. Repeated left-to-right deletions work for n≤ 100, but a deletion can create a new pair further left, forcing extra passes. A stack holds the cleaned prefix. If the new character differs from the top by 32 (exactly a case pair), pop; otherwise push. One linear scan produces the answer.

Python 3

```
class Solution:
    def makeGood(self, s: str) -> str:
        stk = []
        for c in s:
            if not stk or abs(ord(stk[-1]) - ord(c)) != 32:
                stk.append(c)
            else:
                stk.pop()
        return "".join(stk)
```

Complexity. Time: O(n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. How many characters were removed?
# 2. Any rule for which two neighbours cancel: pass the
# rule in


# ============================================================
# FOLLOW-UP 1: How many characters were removed?
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same stack scan; the answer is the original
# length minus the length of what is left.
# Time: O(n).  Space: O(n).
def removed_count(s):
    stack = []
    for ch in s:
        if stack and stack[-1] != ch and stack[-1].lower() == ch.lower():
            stack.pop()
        else:
            stack.append(ch)
    return len(s) - len(stack)


# ============================================================
# FOLLOW-UP 2: Any rule for which two neighbours cancel:
# pass the rule in
# ============================================================

# ---- follow-up 2: function ----
# Answer: Replace the hard-coded test by a function
# cancels(a, b); the stack scan stays the same.
# Time: O(n).  Space: O(n).
def reduce_with_rule(items, cancels):
    stack = []
    for item in items:
        if stack and cancels(stack[-1], item):
            stack.pop()
        else:
            stack.append(item)
    return stack
```

PATTERN 8 · STACK · PROBLEM 4 OF 20

496. Next Greater Element I

Easy · Stack, Array, Hash Table, Monotonic Stack

leetcode.com/problems/next-greater-element-i

Also appears in pattern 9 Monotonic Stack / Queue (Specific Variants)

PROBLEM

The **next greater element** of some element `x` in an array is the **first greater** element that is **to the right** of `x` in the same array.

You are given two **distinct 0-indexed** integer arrays `nums1` and `nums2`, where `nums1` is a subset of `nums2`.

For each `0 <= i < nums1.length`, find the index `j` such that `nums1[i] == nums2[j]` and determine the **next greater element** of `nums2[j]` in `nums2`. If there is no next greater element, then the answer for this query is `-1`.

Return *an array* `ans` *of length* `nums1.length` *such that* `ans[i]` *is the* ***next greater element*** *as described above.*

Example 1:

```
Input: nums1 = [4,1,2], nums2 = [1,3,4,2]
Output: [-1,3,-1]
Explanation: The next greater element for each value of nums1 is as follows:
- 4 is underlined in nums2 = [1,3,4,2]. There is no next greater element, so the answer is -1.
- 1 is underlined in nums2 = [1,3,4,2]. The next greater element is 3.
- 2 is underlined in nums2 = [1,3,4,2]. There is no next greater element, so the answer is -1.
```

Example 2:

```
Input: nums1 = [2,4], nums2 = [1,2,3,4]
Output: [3,-1]
Explanation: The next greater element for each value of nums1 is as follows:
- 2 is underlined in nums2 = [1,2,3,4]. The next greater element is 3.
- 4 is underlined in nums2 = [1,2,3,4]. There is no next greater element, so the answer is -1.
```

Constraints:

- `1 <= nums1.length <= nums2.length <= 1000`
- `0 <= nums1[i], nums2[i] <= 10^4`
- All integers in `nums1` and `nums2` are **unique**.
- All the integers of `nums1` also appear in `nums2`.

Follow up:

Could you find an

`O(nums1.length + nums2.length)`

solution?

SOLUTION (PYTHON 3)

Solution 1: Monotonic Stack

How to think about it

nums1 is a subset of nums2; we want each value's next greater element in nums2. Scanning rightward from every nums1 value is O(nm). Walk nums2 right to left with a decreasing stack: after popping smaller tops, the new top is the next greater value, stored in a map. Then look up nums1. Each value enters and leaves the stack once. Building the map on nums2 first avoids scanning it once per query.

We can traverse the array nums2 from right to left, maintaining a stack stk that is monotonically increasing from top to bottom. We use a hash table d to record the next greater element for each element.

When we encounter an element x, if the stack is not empty and the top element of the stack is less than x, we keep popping the top elements until the stack is empty or the top element is greater than or equal to x. At this point, if the stack is not empty, the top element of the stack is the next greater element for x. Otherwise, x has no next greater element.

Finally, we traverse the array nums1 and use the hash table d to get the answer.

The time complexity is O(m + n), and the space complexity is O(n). Here, m and n are the lengths of the arrays nums1 and nums2, respectively.

Python 3

```python
class Solution:
    def nextGreaterElement(self, nums1: List[int], nums2: List[int]) -> List[int]:
        stk = []
        d = {}
        for x in nums2[::-1]:
            while stk and stk[-1] < x:
                stk.pop()
            if stk:
                d[x] = stk[-1]
            stk.append(x)
        return [d.get(x, -1) for x in nums1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The array is circular (Next Greater Element II)
# 2. The next SMALLER element to the right


# ============================================================
# FOLLOW-UP 1: The array is circular (Next Greater Element
# II)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Go around the array twice (positions 0 to 2n -
# 1, reading the numbers modulo n) with a stack of
# positions whose next greater element is still unknown.
# Time: O(n).  Space: O(n).
def next_greater_circular(nums):
    n = len(nums)
    answer = [-1] * n
    stack = []
    for i in range(2 * n):
        value = nums[i % n]
        while stack and nums[stack[-1]] < value:
            answer[stack.pop()] = value
        if i < n:
            stack.append(i)
    return answer


# ============================================================
# FOLLOW-UP 2: The next SMALLER element to the right
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same stack, but pop while the new number is
# smaller than the number on top.
# Time: O(n).  Space: O(n).
def next_smaller(nums):
    answer = [-1] * len(nums)
    stack = []
```

```
    for i in range(len(nums)):
        while stack and nums[stack[-1]] > nums[i]:
            answer[stack.pop()] = nums[i]
        stack.append(i)
    return answer
```

682. Baseball Game

Easy · Stack, Array, Simulation

leetcode.com/problems/baseball-game

PROBLEM

You are keeping the scores for a baseball game with strange rules. At the beginning of the game, you start with an empty record.

You are given a list of strings `operations`, where `operations[i]` is the i^{th} operation you must apply to the record and is one of the following:

- An integer `x`.
  - Record a new score of `x`.
- `'+'`.
  - Record a new score that is the sum of the previous two scores.
- `'D'`.
  - Record a new score that is the double of the previous score.
- `'C'`.
  - Invalidate the previous score, removing it from the record.

Return *the sum of all the scores on the record after applying all the operations*.

The test cases are generated such that the answer and all intermediate calculations fit in a **32-bit** integer and that all operations are valid.

Example 1:

```
Input: ops = ["5","2","C","D","+"]
Output: 30
Explanation:
"5" - Add 5 to the record, record is now [5].
"2" - Add 2 to the record, record is now [5, 2].
"C" - Invalidate and remove the previous score, record is now [5].
"D" - Add 2 * 5 = 10 to the record, record is now [5, 10].
"+" - Add 5 + 10 = 15 to the record, record is now [5, 10, 15].
The total sum is 5 + 10 + 15 = 30.
```

Example 2:

```
Input: ops = ["5","-2","4","C","D","9","+","+"]
Output: 27
Explanation:
"5" - Add 5 to the record, record is now [5].
"-2" - Add -2 to the record, record is now [5, -2].
"4" - Add 4 to the record, record is now [5, -2, 4].
"C" - Invalidate and remove the previous score, record is now [5, -2].
"D" - Add 2 * -2 = -4 to the record, record is now [5, -2, -4].
"9" - Add 9 to the record, record is now [5, -2, -4, 9].
"+" - Add -4 + 9 = 5 to the record, record is now [5, -2, -4, 9, 5].
"+" - Add 9 + 5 = 14 to the record, record is now [5, -2, -4, 9, 5, 14].
The total sum is 5 + -2 + -4 + 9 + 5 + 14 = 27.
```

Example 3:

```
Input: ops = ["1","C"]
Output: 0
Explanation:
"1" - Add 1 to the record, record is now [1].
"C" - Invalidate and remove the previous score, record is now [].
Since the record is empty, the total sum is 0.
```

Constraints:

- `1 <= operations.length <= 1000`
- `operations[i]` is `"C"`, `"D"`, `"+"`, or a string representing an integer in the range $[-3 * 10^4, 3 * 10^4]$.
- For operation `"+"`, there will always be at least two previous scores on the record.
- For operations `"C"` and `"D"`, there will always be at least one previous score on the record.

SOLUTION (PYTHON 3)

Solution 1: Stack + Simulation

> **How to think about it**
>
> Each operation refers to the last score, the last two, or an undo, so recent values must stay accessible. A stack stores rounds: `+` sums the top two, `D` doubles the top, `C` pops, a number pushes. Sum the stack at the end.

We can use a stack to simulate this process.

Traverse operations, for each operation:

- If it is `+`, add the top two elements of the stack and push the result onto the stack;
- If it is `D`, multiply the top element of the stack by 2 and push the result onto the stack;
- If it is `C`, pop the top element of the stack;
- If it is a number, push the number onto the stack.

Finally, sum all the elements in the stack to get the answer.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of operations.

Python 3

```python
class Solution:
    def calPoints(self, operations: List[str]) -> int:
        stk = []
        for op in operations:
            if op == "+":
                stk.append(stk[-1] + stk[-2])
            elif op == "D":
                stk.append(stk[-1] << 1)
            elif op == "C":
                stk.pop()
            else:
                stk.append(int(op))
        return sum(stk)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the total after every operation
# 2. Add an operation 'U' that undoes the previous
# operation (of any kind)


# ========================================================
# FOLLOW-UP 1: Return the total after every operation
# ========================================================
```

```
# ---- follow-up 1: function ----
# Answer: Keep the list of valid scores as before and
# append the running sum of that list after each
# operation.
# Time: O(n^2) (the total is recomputed after every
# operation).  Space: O(n).
def totals_after_each_operation(operations):
    scores = []
    totals = []
    for op in operations:
        if op == "C":
            scores.pop()
        elif op == "D":
            scores.append(2 * scores[-1])
        elif op == "+":
            scores.append(scores[-1] + scores[-2])
        else:
            scores.append(int(op))
        total = 0
        for s in scores:
            total += s
        totals.append(total)
    return totals


# ============================================================
# FOLLOW-UP 2: Add an operation 'U' that undoes the
# previous operation (of any kind)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep a stack of snapshots of the score list:
# every operation pushes the new list, and 'U' drops the
# latest snapshot.
# Time: O(n^2) (the list is copied for every operation).
# Space: O(n^2).
def final_total_with_undo(operations):
    history = [[]]                                  # history[-1] is the current list of scores
    for op in operations:
        if op == "U":
            if len(history) > 1:
                history.pop()
            continue
        scores = list(history[-1])
        if op == "C":
            scores.pop()
        elif op == "D":
            scores.append(2 * scores[-1])
        elif op == "+":
            scores.append(scores[-1] + scores[-2])
        else:
            scores.append(int(op))
        history.append(scores)
    total = 0
    for s in history[-1]:
        total += s
    return total
```

PATTERN 8 · STACK · PROBLEM 7 OF 20

155. Min Stack

Medium · Stack, Design

leetcode.com/problems/min-stack

★ **TOP 7 PROBLEM** in Stack: master this one first

Also appears in pattern 29 Design Problems

PROBLEM

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the `MinStack` class:

- `MinStack()` initializes the stack object.
- `void push(int value)` pushes the element `value` onto the stack.
- `void pop()` removes the element on the top of the stack.
- `int top()` gets the top element of the stack.
- `int getMin()` retrieves the minimum element in the stack.

You must implement a solution with `O(1)` time complexity for each function.

Example 1:

```
Input
["MinStack","push","push","push","getMin","pop","top","getMin"]
[[],[-2],[0],[-3],[],[],[],[]]

Output
[null,null,null,null,-3,null,0,-2]

Explanation
MinStack minStack = new MinStack();
minStack.push(-2);
minStack.push(0);
minStack.push(-3);
minStack.getMin(); // return -3
minStack.pop();
minStack.top(); // return 0
minStack.getMin(); // return -2
```

Constraints:

- -2^{31} <= val <= $2^{31} - 1$
- Methods `pop`, `top` and `getMin` operations will always be called on **non-empty** stacks.
- At most $3 * 10^4$ calls will be made to `push`, `pop`, `top`, and `getMin`.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

getMin must be O(1) like push/pop. Scanning the stack is linear, and there are up to 3× 10^4 calls. The minimum changes only when a smaller value is pushed or the current min is popped. A parallel min-stack stores min(x, current min) on each push and pops in lockstep.

Python 3

```python
class MinStack:
    def __init__(self):
        self.stk1 = []
        self.stk2 = [inf]

    def push(self, val: int) -> None:
        self.stk1.append(val)
        self.stk2.append(min(val, self.stk2[-1]))

    def pop(self) -> None:
        self.stk1.pop()
        self.stk2.pop()

    def top(self) -> int:
        return self.stk1[-1]

    def getMin(self) -> int:
        return self.stk2[-1]


# Your MinStack object will be instantiated and called as such:
# obj = MinStack()
# obj.push(val)
# obj.pop()
# param_3 = obj.top()
# param_4 = obj.getMin()
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. A MAX stack: also pop the maximum element
# 2. Only ONE stack of numbers (no second stack) for the
# minimum


# ============================================================
# FOLLOW-UP 1: A MAX stack: also pop the maximum element
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep a second stack with the maximum so far for
# peekMax in O(1). popMax removes the top-most maximum:
# pop elements until it is found, then push the others
# back.
# Time: push, pop, top, peek_max O(1); pop_max O(n).
# Space: O(n).
class MaxStack:
    def __init__(self):
        self.items = []
        self.maximums = []

    def push(self, x):
        self.items.append(x)
        if not self.maximums or x >= self.maximums[-1]:
            self.maximums.append(x)
        else:
            self.maximums.append(self.maximums[-1])

    def pop(self):
        self.maximums.pop()
        return self.items.pop()

    def top(self):
        return self.items[-1]
```

```
    def peek_max(self):
        return self.maximums[-1]

    def pop_max(self):
        biggest = self.maximums[-1]
        buffer = []
        while self.items[-1] != biggest:
            buffer.append(self.pop())
        self.pop()
        while buffer:
            self.push(buffer.pop())
        return biggest


# ============================================================
# FOLLOW-UP 2: Only ONE stack of numbers (no second stack)
# for the minimum
# ============================================================

# ---- follow-up 2: function ----
# Answer: Store the difference between each pushed number
# and the minimum before it. A negative difference means
# the pushed number is the new minimum, and popping it
# restores the old minimum from the difference.
```

```
# Time: O(1) per operation.  Space: O(n).
class MinStackOneStack:
    def __init__(self):
        self.stack = []                             # differences from the minimum before the push
        self.minimum = 0

    def push(self, x):
        if not self.stack:
            self.stack.append(0)
            self.minimum = x
            return
        self.stack.append(x - self.minimum)
        if x < self.minimum:
            self.minimum = x

    def pop(self):
        difference = self.stack.pop()
        if difference < 0:
            self.minimum = self.minimum - difference     # restore the previous minimum

    def top(self):
        difference = self.stack[-1]
        if difference < 0:
            return self.minimum
        return self.minimum + difference

    def get_min(self):
        return self.minimum
```

PATTERN 8 · STACK · PROBLEM 8 OF 20

150. Evaluate Reverse Polish Notation

Medium · Stack, Array, Math

leetcode.com/problems/evaluate-reverse-polish-notation

★ **TOP 7 PROBLEM** in Stack: master this one first

PROBLEM

You are given an array of strings `tokens` that represents an arithmetic expression in a Reverse Polish Notation.

Evaluate the expression. Return *an integer that represents the value of the expression*.

Note that:

- The valid operators are `'+'`, `'-'`, `'*'`, and `'/'`.
- Each operand may be an integer or another expression.
- The division between two integers always **truncates toward zero**.
- There will not be any division by zero.
- The input represents a valid arithmetic expression in a reverse polish notation.
- The answer and all the intermediate calculations can be represented in a **32-bit** integer.

Example 1:

```
Input: tokens = ["2","1","+","3","*"]
Output: 9
Explanation: ((2 + 1) * 3) = 9
```

Example 2:

```
Input: tokens = ["4","13","5","/","+"]
Output: 6
Explanation: (4 + (13 / 5)) = 6
```

Example 3:

```
Input: tokens = ["10","6","9","3","+","-11","*","/","*","17","+","5","+"]
Output: 22
Explanation: ((10 * (6 / ((9 + 3) * -11))) + 17) + 5
= ((10 * (6 / (12 * -11))) + 17) + 5
= ((10 * (6 / -132)) + 17) + 5
= ((10 * 0) + 17) + 5
= (0 + 17) + 5
= 17 + 5
= 22
```

Constraints:

- `1 <= tokens.length <= 10^4`
- `tokens[i]` is either an operator: `"+"`, `"-"`, `"*"`, or `"/"`, or an integer in the range `[-200, 200]`.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Reverse Polish notation puts the operator after its operands, so evaluation order is already fixed and we need no parentheses or precedence. n≤ 10^4. Push numbers; on an operator pop two values, compute, and push back. The stack holds pending intermediates; one value remains.

Python 3

```python
import operator


class Solution:
    def evalRPN(self, tokens: List[str]) -> int:
        opt = {
            "+": operator.add,
            "-": operator.sub,
            "*": operator.mul,
            "/": operator.truediv,
        }
        s = []
        for token in tokens:
            if token in opt:
                s.append(int(opt[token](s.pop(-2), s.pop(-1))))
            else:
                s.append(int(token))
        return s[0]
```

Complexity. Time: O(n). Space: O(n).

Solution 2

> **How to think about it**
>
> Solution 1 maps operators to functions. This version mutates the stack top in place, branching on the four operators, and watches operand order and truncation toward zero. Same structure, no function table.

Python 3

```python
class Solution:
    def evalRPN(self, tokens: List[str]) -> int:
        nums = []
        for t in tokens:
            if len(t) > 1 or t.isdigit():
                nums.append(int(t))
            else:
                if t == "+":
                    nums[-2] += nums[-1]
                elif t == "-":
                    nums[-2] -= nums[-1]
                elif t == "*":
                    nums[-2] *= nums[-1]
                else:
                    nums[-2] = int(nums[-2] / nums[-1])
                nums.pop()
        return nums[0]
```

Complexity. Time: O(n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Convert a normal (infix) expression to reverse Polish
# notation (the shunting-yard method)
# 2. The expression contains variables


# ============================================================
# FOLLOW-UP 1: Convert a normal (infix) expression to
# reverse Polish notation (the shunting-yard method)
# ============================================================

# ---- follow-up 1: function ----
```

```
# Answer: Numbers go straight to the output. An operator
# waits on a stack; before it is pushed, every operator on
# the stack with the same or higher priority moves to the
# output. Parentheses hold operators back until the
# closing bracket.
# Time: O(n).  Space: O(n).
def infix_to_rpn(tokens):
    priority = {"+": 1, "-": 1, "*": 2, "/": 2}
    output = []
    operators = []
    for token in tokens:
        if token in priority:
            while operators and operators[-1] in priority and priority[operators[-1]] >= priority[token]:
                output.append(operators.pop())
            operators.append(token)
        elif token == "(":
            operators.append(token)
        elif token == ")":
            while operators[-1] != "(":
                output.append(operators.pop())
            operators.pop()
        else:
            output.append(token)
    while operators:
        output.append(operators.pop())
    return output


# ============================================================
# FOLLOW-UP 2: The expression contains variables
# ============================================================

# ---- follow-up 2: function ----
# Answer: Look a token up in a dictionary of values when
# it is not a number or an operator; the stack logic does
# not change.
# Time: O(n).  Space: O(n).
def eval_rpn_with_variables(tokens, values):
    stack = []
    for token in tokens:
        if token in ("+", "-", "*", "/"):
            b = stack.pop()
            a = stack.pop()
            if token == "+":
                stack.append(a + b)
            elif token == "-":
                stack.append(a - b)
            elif token == "*":
                stack.append(a * b)
            else:
                stack.append(int(a / b))
        elif token in values:
            stack.append(values[token])
        else:
            stack.append(int(token))
    return stack[0]
```

PATTERN 8 · STACK · PROBLEM 9 OF 20

22. Generate Parentheses

Medium · String, Dynamic Programming, Backtracking, Parentheses

leetcode.com/problems/generate-parentheses

★ **TOP 7 PROBLEM** in Stack: master this one first

Also appears in pattern 15 Backtracking / Subsets / Combinations / Permutations

PROBLEM

Given `n` pairs of parentheses, write a function to *generate all combinations of well-formed parentheses*.

Example 1:

```
Input: n = 3
Output: ["((()))","(()())","(())()","()(())","()()()"]
```

Example 2:

```
Input: n = 1
Output: ["()"]
```

Constraints:

- `1 <= n <= 8`

SOLUTION (PYTHON 3)

Solution 1: DFS + Pruning

How to think about it

The first idea is to enumerate every string of length 2n and keep the valid ones. n ≤ 8 gives 2^{16} candidates, so it would pass, but most prefixes are already illegal. The bottleneck is generate-then-check: once a prefix has more `)` than `(`, or either count exceeds n, no suffix can save it. A valid prefix only needs the two counts l and r: keep l ≥ r and neither above n. When l=r=n, record the string. DFS therefore tries appending `(` or `)` and prunes on those three inequalities. The search tree is much smaller than full enumeration; extra space is just the current string, O(n).

The range of n in the problem is [1, 8], so we can directly solve this problem through "brute force search + pruning".

We design a function dfs(l, r, t), where l and r represent the number of left and right brackets respectively, and t represents the current bracket sequence. Then we can get the following recursive structure:

- If l > n or r > n or l < r, then the current bracket combination t is invalid, return directly;
- If l = n and r = n, then the current bracket combination t is valid, add it to the answer array `ans`, and return directly;
- We can choose to add a left bracket, and recursively execute `dfs(l + 1, r, t + "(")`;
- We can also choose to add a right bracket, and recursively execute `dfs(l, r + 1, t + ")")`.

The time complexity is $O(2^{n\times 2} \times n)$, and the space complexity is O(n).

Python 3

```
class Solution:
    def generateParenthesis(self, n: int) -> List[str]:
        def dfs(l: int, r: int, t: str):
            if l > n or r > n or l < r:
                return
            if l == n and r == n:
                ans.append(t)
                return
            dfs(l + 1, r, t + "(")
            dfs(l, r + 1, t + ")")

        ans = []
        dfs(0, 0, "")
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Only count the valid sequences (do not build them)
# 2. Three kinds of brackets: (), [] and {}


# ============================================================
# FOLLOW-UP 1: Only count the valid sequences (do not
# build them)
# ============================================================

# ---- follow-up 1: function ----
# Answer: The number of valid sequences with n pairs is
# the Catalan number. Build it with a table: a valid
# sequence is '(' + a valid sequence of i pairs + ')' + a
# valid sequence of the remaining pairs.
# Time: O(n^2).  Space: O(n).
def count_valid_sequences(n):
    ways = [0] * (n + 1)
    ways[0] = 1
    for pairs in range(1, n + 1):
        for inside in range(pairs):
            ways[pairs] += ways[inside] * ways[pairs - 1 - inside]
    return ways[n]


# ============================================================
# FOLLOW-UP 2: Three kinds of brackets: (), [] and {}
# ============================================================

# ---- follow-up 2: function ----
# Answer: Backtracking with a stack of the kinds that are
# still open: at each step either open any of the three
# kinds (while the opened pairs are fewer than n) or close
# the kind on top of the stack.
# Time: O(3^n * Catalan(n) * n) (every result is built and
# copied).  Space: O(n) recursion plus the results.
def generate_with_three_kinds(n):
    pairs = {"(": ")", "[": "]", "{": "}"}
    results = []

    def build(current, open_stack, opened):
        if len(current) == 2 * n:
            results.append("".join(current))
            return
        if opened < n:
            for opener in pairs:
                current.append(opener)
                open_stack.append(opener)
```

```
                build(current, open_stack, opened + 1)
                open_stack.pop()
                current.pop()
        if open_stack:
            opener = open_stack.pop()
            current.append(pairs[opener])
            build(current, open_stack, opened)
            current.pop()
            open_stack.append(opener)

    build([], [], 0)
    return results
```

PATTERN 8 · STACK · PROBLEM 10 OF 20

739. Daily Temperatures

Medium · Stack, Array, Monotonic Stack

leetcode.com/problems/daily-temperatures

★ **TOP 7 PROBLEM** in Stack, Monotonic Stack / Queue (Specific Variants): master this one first

Also appears in pattern 9 Monotonic Stack / Queue (Specific Variants)

PROBLEM

Given an array of integers `temperatures` represents the daily temperatures, return *an array* `answer` *such that* `answer[i]` *is the number of days you have to wait after the* `i`th *day to get a warmer temperature*. If there is no future day for which this is possible, keep `answer[i] == 0` instead.

Example 1:

```
Input: temperatures = [73,74,75,71,69,72,76,73]
Output: [1,1,4,2,1,1,0,0]
```

Example 2:

```
Input: temperatures = [30,40,50,60]
Output: [1,1,1,0]
```

Example 3:

```
Input: temperatures = [30,60,90]
Output: [1,1,0]
```

Constraints:

- `1 <= temperatures.length <= 10`^{5}
- `30 <= temperatures[i] <= 100`

SOLUTION (PYTHON 3)

Solution 1: Monotonic Stack

How to think about it

For each day, the wait until a warmer day. n≤ 10^5 forbids a rightward scan from every index. This is next-greater-element. A monotonic stack of indices still waiting for a warmer day, scanned right to left, pops every temperature that is not strictly warmer; the new top is the answer. Temperatures increase from top to bottom. Each index is pushed and popped once, O(n).

This problem requires us to find the position of the first element greater than each element to its right, which is a typical application scenario for a monotonic stack.

We traverse the array temperatures from right to left, maintaining a stack stk that is monotonically increasing from top to bottom in terms of temperature. The stack stores the indices of the array elements. For each element temperatures[i], we continuously compare it with the top element of the stack. If the temperature corresponding to the top element of the stack is less than or equal to temperatures[i], we pop the top element of the stack in a loop until the stack is empty or the temperature corresponding to the top element of the stack is greater than temperatures[i]. At this point, the top element of the stack is the first element greater than temperatures[i] to its right, and the distance is stk.top() - i. We update the answer array accordingly. Then we push temperatures[i] onto the stack and continue traversing.

After the traversal, we return the answer array.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array temperatures.

Python 3

```python
class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        stk = []
        n = len(temperatures)
        ans = [0] * n
        for i in range(n - 1, -1, -1):
            while stk and temperatures[stk[-1]] <= temperatures[i]:
                stk.pop()
            if stk:
                ans[i] = stk[-1] - i
            stk.append(i)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. How many days back was the last WARMER day (previous
# warmer)?
# 2. How many days until a COLDER day?


# ============================================================
# FOLLOW-UP 1: How many days back was the last WARMER day
# (previous warmer)?
# ============================================================

# ---- follow-up 1: function ----
# Answer: Scan from left to right with a stack of
# decreasing temperatures; the top that is warmer than
# today is the previous warmer day.
# Time: O(n).  Space: O(n).
def days_since_warmer(temperatures):
    answer = [0] * len(temperatures)
    stack = []
    for i in range(len(temperatures)):
        while stack and temperatures[stack[-1]] <= temperatures[i]:
            stack.pop()
        if stack:
            answer[i] = i - stack[-1]
        stack.append(i)
    return answer


# ============================================================
# FOLLOW-UP 2: How many days until a COLDER day?
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same stack, but wait for a smaller
# temperature.
# Time: O(n).  Space: O(n).
def days_until_colder(temperatures):
    answer = [0] * len(temperatures)
    stack = []
    for i in range(len(temperatures)):
        while stack and temperatures[stack[-1]] > temperatures[i]:
            earlier = stack.pop()
            answer[earlier] = i - earlier
        stack.append(i)
    return answer
```

PATTERN 8 · STACK · PROBLEM 12 OF 20

735. Asteroid Collision

Medium · Stack, Array, Simulation

leetcode.com/problems/asteroid-collision

PROBLEM

We are given an array `asteroids` of integers representing asteroids in a row. The indices of the asteroid in the array represent their relative position in space.

For each asteroid, the absolute value represents its size, and the sign represents its direction (positive meaning right, negative meaning left). Each asteroid moves at the same speed.

Find out the state of the asteroids after all collisions. If two asteroids meet, the smaller one will explode. If both are the same size, both will explode. Two asteroids moving in the same direction will never meet.

Example 1:

```
Input: asteroids = [5,10,-5]
Output: [5,10]
Explanation: The 10 and -5 collide resulting in 10. The 5 and 10 never collide.
```

Example 2:

```
Input: asteroids = [8,-8]
Output: []
Explanation: The 8 and -8 collide exploding each other.
```

Example 3:

```
Input: asteroids = [10,2,-5]
Output: [10]
Explanation: The 2 and -5 collide resulting in -5. The 10 and -5 collide resulting in 10.
```

Example 4:

```
Input: asteroids = [3,5,-6,2,-1,4]
Output: [-6,2,4]
Explanation: The asteroid -6 makes the asteroid 3 and 5 explode, and then continues going left. On the
other side, the asteroid 2 destroys -1. Since 2 and 4 are both moving right, they never collide.
```

Constraints:

- `2 <= asteroids.length <= 10^4`
- `-1000 <= asteroids[i] <= 1000`
- `asteroids[i] != 0`

SOLUTION (PYTHON 3)

Solution 1: Stack

How to think about it

Collisions happen only when a right-moving rock meets a left-moving one. $n \le 10^4$. A stack holds the still-alive prefix, which is the natural surviving sequence. A positive rock cannot hit anything to its left, so it is pushed. A negative one fights the positive top: smaller tops explode, equal tops both vanish, a larger top destroys the new rock. Each rock is pushed and popped at most once, so the scan is O(n).

We traverse each asteroid x from left to right. Since each asteroid may collide with multiple asteroids before it, we consider using a stack to store.

- For the current asteroid, if x>0, it will definitely not collide with the previous asteroid, and we can directly push x into the stack.
- Otherwise, if the stack is not empty and the top element of the stack is greater than 0, and the top element of the stack is less than -x, then the top element of the stack corresponds to the asteroid will explode, we loop to the top element of the stack Pop out until the condition is not satisfied. At this time, if the top element of the stack is equal to -x, then the two asteroids will explode, and we only need to pop the top element of the stack; if the stack is empty, or the top element of the stack is less than 0, then the current asteroid will not collide, we will push x into the stack.

Finally, we return the elements in the stack as the answer.

The time complexity is O(n), and the space complexity is O(n). Where n is the length of the array asteroids.

Python 3

```python
class Solution:
    def asteroidCollision(self, asteroids: List[int]) -> List[int]:
        stk = []
        for x in asteroids:
            if x > 0:
                stk.append(x)
            else:
                while stk and stk[-1] > 0 and stk[-1] < -x:
                    stk.pop()
                if stk and stk[-1] == -x:
                    stk.pop()
                elif not stk or stk[-1] < 0:
                    stk.append(x)
        return stk
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count how many asteroids exploded and what total mass
# survives
# 2. When two asteroids have the same size they BOUNCE and
# both survive


# ============================================================
# FOLLOW-UP 1: Count how many asteroids exploded and what
# total mass survives
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same stack; every time an asteroid is
# removed or not added, count it.
# Time: O(n).  Space: O(n).
def collision_summary(asteroids):
    stack = []
    for a in asteroids:
        alive = True
        while alive and a < 0 and stack and stack[-1] > 0:
            if stack[-1] < -a:
                stack.pop()
            elif stack[-1] == -a:
                stack.pop()
                alive = False
            else:
                alive = False
        if alive:
            stack.append(a)
    mass = 0
    for a in stack:
        mass += abs(a)
    return len(asteroids) - len(stack), mass
```

```
# ============================================================
# FOLLOW-UP 2: When two asteroids have the same size they
# BOUNCE and both survive
# ============================================================

# ---- follow-up 2: function ----
# Answer: Only a strictly smaller asteroid is destroyed;
# on equal sizes both stay and the collision stops.
# Time: O(n).  Space: O(n).
def asteroid_collision_bounce(asteroids):
    stack = []
    for a in asteroids:
        alive = True
        while alive and a < 0 and stack and stack[-1] > 0:
            if stack[-1] < -a:
                stack.pop()
            elif stack[-1] == -a:
                alive = True
                break
            else:
                alive = False
        if alive:
            stack.append(a)
    return stack
```

PATTERN 8 · STACK · PROBLEM 14 OF 20

227. Basic Calculator II

Medium · Stack, Math, String

leetcode.com/problems/basic-calculator-ii

★ **TOP 7 PROBLEM** in Stack: master this one first

PROBLEM

Given a string `s` which represents an expression, *evaluate this expression and return its value*.

The integer division should truncate toward zero.

You may assume that the given expression is always valid. All intermediate results will be in the range of $[-2^{31}, 2^{31} - 1]$.

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as `eval()`.

Example 1:

```
Input: s = "3+2*2"
Output: 7
```

Example 2:

```
Input: s = " 3/2 "
Output: 1
```

Example 3:

```
Input: s = " 3+5 / 2 "
Output: 5
```

Constraints:

- $1 \le s.length \le 3 * 10^5$
- `s` consists of integers and operators `('+', '-', '*', '/')` separated by some number of spaces.
- `s` represents a valid expression.
- All the integers in the expression are non-negative integers in the range $[0, 2^{31} - 1]$.
- The answer is **guaranteed** to fit in a **32-bit integer**.

SOLUTION (PYTHON 3)

Solution 1: Stack

How to think about it

Multiplication and division bind tighter than addition, and there are no parentheses, so we cannot add every number immediately. Treat + and - as signed terms on a stack, and fold * and / into the top. sign remembers the previous operator; after a number is complete we push or update the top, then sum the stack.

We traverse the string s, and use a variable `sign` to record the operator before each number. For the first number, its previous operator is considered as a plus sign. Each time we traverse to the end of a number, we decide the calculation method based on `sign`:

- Plus sign: push the number into the stack;
- Minus sign: push the opposite number into the stack;
- Multiplication and division signs: calculate the number with the top element of the stack, and replace the top element of the stack with the calculation result.

After the traversal ends, the sum of the elements in the stack is the answer.

The time complexity is O(n), and the space complexity is O(n), where n is the length of the string s.

Python 3

```python
class Solution:
    def calculate(self, s: str) -> int:
        v, n = 0, len(s)
        sign = '+'
        stk = []
        for i, c in enumerate(s):
            if c.isdigit():
                v = v * 10 + int(c)
            if i == n - 1 or c in '+-*/':
                match sign:
                    case '+':
                        stk.append(v)
                    case '-':
                        stk.append(-v)
                    case '*':
                        stk.append(stk.pop() * v)
                    case '/':
                        stk.append(int(stk.pop() / v))
                sign = c
                v = 0
        return sum(stk)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Add the power operator ^ (it binds tighter than * and
# / and groups to the right)
# 2. Exact division: return a fraction instead of
# truncating


# ============================================================
# FOLLOW-UP 1: Add the power operator ^ (it binds tighter
# than * and / and groups to the right)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Recursive descent with one more level:
# expression (+ -), term (* /), power (a ^ b, right-
# grouping), number.
# Time: O(n).  Space: O(n).
def calculate_with_power(s):
    tokens = []
    i = 0
    while i < len(s):
        if s[i].isdigit():
            number = 0
            while i < len(s) and s[i].isdigit():
                number = number * 10 + int(s[i])
                i += 1
            tokens.append(number)
            continue
        if s[i] != " ":
            tokens.append(s[i])
        i += 1
    position = [0]

    def next_is(characters):
        return position[0] < len(tokens) and isinstance(tokens[position[0]], str) and tokens[position[0]]
in characters

    def parse_power():
```

```
        base = tokens[position[0]]
        position[0] += 1
        if next_is("^"):
            position[0] += 1
            return base ** parse_power()
        return base

    def parse_term():
        value = parse_power()
        while next_is("*/"):
            operator = tokens[position[0]]
            position[0] += 1
            other = parse_power()
            if operator == "*":
                value *= other
            else:
                value = int(value / other)
        return value

    def parse_expression():
        value = parse_term()
        while next_is("+-"):
            operator = tokens[position[0]]
```

```
            position[0] += 1
            other = parse_term()
            if operator == "+":
                value += other
            else:
                value -= other
        return value

    return parse_expression()


# ============================================================
# FOLLOW-UP 2: Exact division: return a fraction instead
# of truncating
# ============================================================

# ---- follow-up 2: function ----
# Answer: Use exact fractions for the numbers; the stack
# logic of the main question stays the same, but division
# no longer drops the remainder.
# Time: O(n).  Space: O(n).
from fractions import Fraction


def calculate_exact(s):
    stack = []
    number = 0
    operator = "+"
    s = s.replace(" ", "") + "+"
    for ch in s:
        if ch.isdigit():
            number = number * 10 + int(ch)
            continue
        if operator == "+":
            stack.append(Fraction(number))
        elif operator == "-":
            stack.append(Fraction(-number))
        elif operator == "*":
            stack.append(stack.pop() * number)
        else:
            stack.append(stack.pop() / number)
        operator = ch
        number = 0
    total = Fraction(0)
    for value in stack:
```

```
        total += value
    return total
```

1209. Remove All Adjacent Duplicates in String II

Medium · Stack, String

leetcode.com/problems/remove-all-adjacent-duplicates-in-string-ii

PROBLEM

You are given a string `s` and an integer `k`, a `k` **duplicate removal** consists of choosing `k` adjacent and equal letters from `s` and removing them, causing the left and the right side of the deleted substring to concatenate together.

We repeatedly make `k` **duplicate removals** on `s` until we no longer can.

Return *the final string after all such duplicate removals have been made*. It is guaranteed that the answer is **unique**.

Example 1:

```
Input: s = "abcd", k = 2
Output: "abcd"
Explanation: There's nothing to delete.
```

Example 2:

```
Input: s = "deeedbbcccbdaa", k = 3
Output: "aa"
Explanation:
First delete "eee" and "ccc", get "ddbbbdaa"
Then delete "bbb", get "dddaa"
Finally delete "ddd", get "aa"
```

Example 3:

```
Input: s = "pbbcggttciiippooaais", k = 2
Output: "ps"
```

Constraints:

- `1 <= s.length <= 10^5`
- `2 <= k <= 10^4`
- `s` only contains lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Stack

How to think about it

Repeatedly scanning and deleting k equal characters can rescan the string many times when $n \le 10^5$. Deletion is local: only adjacent runs merge, and the two sides may meet again after a deletion. A stack keeps surviving characters and their run lengths in left-to-right order. Matching the top increments the count; reaching k pops the run, i.e. one erasure. We store (char, count) and take the count modulo k so a full run pops immediately. After one pass the stack is the final string.

We can traverse the string s, maintaining a stack that stores the characters and their occurrence counts. When traversing to character c, if the character at the top of the stack is the same as c, we increment the count of the top element by one; otherwise, we push the character c and count 1 into the stack. When the count of the top element equals k, we pop the top element from the stack.

After traversing the string s, the elements remaining in the stack form the final result. We can pop the elements from the stack one by one, concatenate them into a string, and that's our answer.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the string s.

Python 3

```python
class Solution:
    def removeDuplicates(self, s: str, k: int) -> str:
        stk = []
        for c in s:
            if stk and stk[-1][0] == c:
                stk[-1][1] = (stk[-1][1] + 1) % k
                if stk[-1][1] == 0:
                    stk.pop()
            else:
                stk.append([c, 1])
        ans = [c * v for c, v in stk]
        return "".join(ans)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Remove every run of at least k equal letters (the
# whole run, not only k of them)
# 2. Return the groups that were removed, in order


# ============================================================
# FOLLOW-UP 1: Remove every run of at least k equal
# letters (the whole run, not only k of them)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep (letter, run length) pairs on the stack.
# When the run on top reaches k or more it is removed, but
# the next letter can still extend the run below it, so
# check again after each push.
# Time: O(n).  Space: O(n).
def remove_runs_at_least_k(s, k):
    stack = []                                      # [letter, run length]
    for ch in s:
        if stack and stack[-1][0] == ch:
            stack[-1][1] += 1
        else:
            stack.append([ch, 1])
    result = []
    for letter, length in stack:
        if length < k:
            result.append(letter * length)
    return "".join(result)


# ============================================================
# FOLLOW-UP 2: Return the groups that were removed, in
# order
# ============================================================

# ---- follow-up 2: function ----
# Answer: In the main scan, when a run reaches k, record
# that group and pop it.
# Time: O(n).  Space: O(n).
def removed_groups(s, k):
    stack = []
    removed = []
    for ch in s:
        if stack and stack[-1][0] == ch:
            stack[-1][1] += 1
            if stack[-1][1] == k:
                removed.append(ch * k)
                stack.pop()
        else:
```

```
            stack.append([ch, 1])
    return removed
```

PATTERN 8 · STACK · PROBLEM 16 OF 20

856. Score of Parentheses

Medium · Stack, String, Parentheses

leetcode.com/problems/score-of-parentheses

PROBLEM

Given a balanced parentheses string `s`, return *the **score** of the string*.

The **score** of a balanced parentheses string is based on the following rule:

- "`()`" has score `1`.
- `AB` has score `A + B`, where `A` and `B` are balanced parentheses strings.
- `(A)` has score `2 * A`, where `A` is a balanced parentheses string.

Example 1:

```
Input: s = "()"
Output: 1
```

Example 2:

```
Input: s = "(())"
Output: 2
```

Example 3:

```
Input: s = "()()"
Output: 2
```

Constraints:

- `2 <= s.length <= 50`
- `s` consists of only `'('` and `')'`.
- `s` is a balanced parentheses string.

SOLUTION (PYTHON 3)

Solution 1: Counting

> **How to think about it**
>
> The score doubles with nesting: an innermost () is 1. We need not build a tree or fully evaluate on a stack. Scan with depth d; when a () pair closes, add 1≪ d. Other closing parentheses only decrease depth.

By observing, we find that `()` is the only structure that contributes to the score, and the outer parentheses just add some multipliers to this structure. So, we only need to focus on `()`.

We use d to maintain the current depth of parentheses. For each `(`, we increase the depth by one, and for each `)`, we decrease the depth by one. When we encounter `()`, we add 2^d to the answer.

Let's take `(()(()))` as an example. We first find the two closed parentheses `()` inside, and then add the corresponding 2^d to the score. In fact, we are calculating the score of `(()) + ((()))`.

```
( ( ) ( ( ) ) )
  ^ ^   ^ ^

( ( ) ) + ( ( ( ) ) )
  ^ ^         ^ ^
```

The time complexity is O(n), and the space complexity is O(1). Here, n is the length of the string.

Related problems about parentheses:

- 678. Valid Parenthesis String
- 1021. Remove Outermost Parentheses
- 1096. Brace Expansion II
- 1249. Minimum Remove to Make Valid Parentheses
- 1541. Minimum Insertions to Balance a Parentheses String
- 2116. Check if a Parentheses String Can Be Valid

Python 3

```python
class Solution:
    def scoreOfParentheses(self, s: str) -> int:
        ans = d = 0
        for i, c in enumerate(s):
            if c == '(':
                d += 1
            else:
                d -= 1
                if s[i - 1] == '(':
                    ans += 1 << d
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. A formula instead of a stack: the score from the
# depths
# 2. The maximum nesting depth


# ============================================================
# FOLLOW-UP 1: A formula instead of a stack: the score
# from the depths
# ============================================================

# ---- follow-up 1: function ----
# Answer: Every pair '()' that is written directly
# contributes 2 to the power of its depth; add them up.
# Time: O(n).  Space: O(1).
def score_by_depth(s):
    depth = 0
    total = 0
    for i in range(len(s)):
        if s[i] == "(":
            depth += 1
        else:
            depth -= 1
            if s[i - 1] == "(":
                total += 2 ** depth
    return total


# ============================================================
# FOLLOW-UP 2: The maximum nesting depth
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count how deep the brackets go: add one for '('
# and subtract one for ')', and remember the highest
# value.
# Time: O(n).  Space: O(1).
def max_depth(s):
    depth = 0
    best = 0
```

```
    for ch in s:
        if ch == "(":
            depth += 1
            if depth > best:
                best = depth
        elif ch == ")":
            depth -= 1
    return best
```

636. Exclusive Time of Functions

Medium · Stack, Array

leetcode.com/problems/exclusive-time-of-functions

PROBLEM

On a **single-threaded** CPU, we execute a program containing `n` functions. Each function has a unique ID between 0 and `n - 1`.

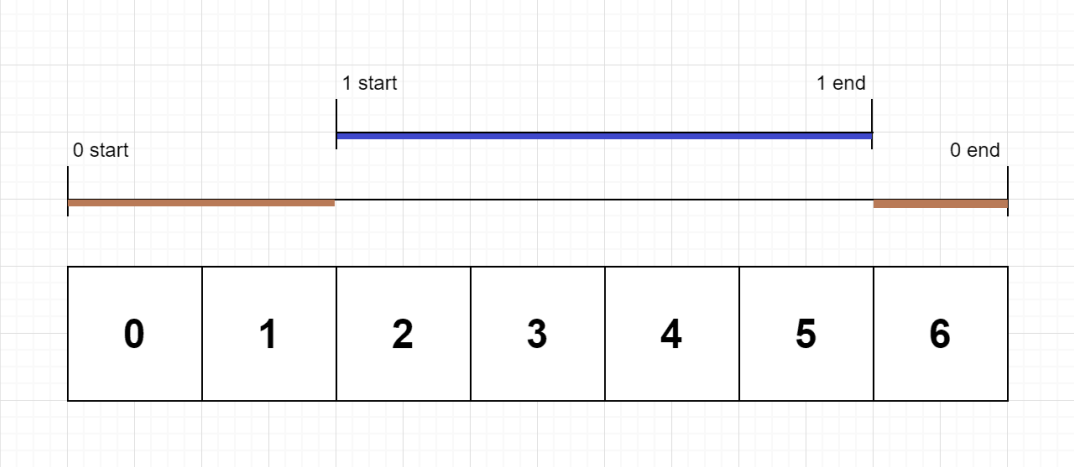
Function calls are **stored in a call stack**: when a function call starts, its ID is pushed onto the stack, and when a function call ends, its ID is popped off the stack. The function whose ID is at the top of the stack is **the current function being executed**. Each time a function starts or ends, we write a log with the ID, whether it started or ended, and the timestamp.

You are given a list `logs`, where `logs[i]` represents the i^{th} log message formatted as a string `"{function_id}:{"start" | "end"}:{timestamp}"`. For example, `"0:start:3"` means a function call with function ID 0 **started at the beginning** of timestamp 3, and `"1:end:2"` means a function call with function ID 1 **ended at the end** of timestamp 2. Note that a function can be called **multiple times, possibly recursively**.

A function's **exclusive time** is the sum of execution times for all function calls in the program. For example, if a function is called twice, one call executing for 2 time units and another call executing for 1 time unit, the **exclusive time** is `2 + 1 = 3`.

Return *the **exclusive time** of each function in an array, where the value at the* i^{th} *index represents the exclusive time for the function with ID* `i`.

Example 1:




```
Input: n = 2, logs = ["0:start:0","1:start:2","1:end:5","0:end:6"]
Output: [3,4]
Explanation:
Function 0 starts at the beginning of time 0, then it executes 2 for units of time and reaches the end of time 1.
Function 1 starts at the beginning of time 2, executes for 4 units of time, and ends at the end of time 5.
Function 0 resumes execution at the beginning of time 6 and executes for 1 unit of time.
So function 0 spends 2 + 1 = 3 units of total time executing, and function 1 spends 4 units of total time executing.
```

Example 2:

```
Input: n = 1, logs = ["0:start:0","0:start:2","0:end:5","0:start:6","0:end:6","0:end:7"]
Output: [8]
Explanation:
Function 0 starts at the beginning of time 0, executes for 2 units of time, and recursively calls
itself.
Function 0 (recursive call) starts at the beginning of time 2 and executes for 4 units of time.
Function 0 (initial call) resumes execution then immediately calls itself again.
Function 0 (2nd recursive call) starts at the beginning of time 6 and executes for 1 unit of time.
Function 0 (initial call) resumes execution at the beginning of time 7 and executes for 1 unit of time.
So function 0 spends 2 + 4 + 1 + 1 = 8 units of total time executing.
```

Example 3:

```
Input: n = 2, logs = ["0:start:0","0:start:2","0:end:5","1:start:6","1:end:6","0:end:7"]
Output: [7,1]
Explanation:
Function 0 starts at the beginning of time 0, executes for 2 units of time, and recursively calls
itself.
Function 0 (recursive call) starts at the beginning of time 2 and executes for 4 units of time.
Function 0 (initial call) resumes execution then immediately calls function 1.
Function 1 starts at the beginning of time 6, executes 1 unit of time, and ends at the end of time 6.
Function 0 resumes execution at the beginning of time 7 and executes for 1 unit of time.
So function 0 spends 2 + 4 + 1 = 7 units of total time executing, and function 1 spends 1 unit of total
time executing.
```

Constraints:

- `1 <= n <= 100`
- `2 <= logs.length <= 500`
- `0 <= function_id < n`
- `0 <= timestamp <= 10^9`
- No two start events will happen at the same timestamp.
- No two end events will happen at the same timestamp.
- Each function has an `"end"` log for each `"start"` log.

SOLUTION (PYTHON 3)

Solution 1: Stack + Simulation

> **How to think about it**
>
> Functions nest and we need exclusive time. Stepping the timeline second-by-second is impossible for large timestamps. A stack matches the nesting: on `start`, credit the top with [pre, cur); on `end`, credit [pre, cur] and set pre=cur+1. One pass over the logs.

We define a stack stk to store the identifiers of the currently executing functions. We also define an array ans to store the exclusive time of each function, initially setting the exclusive time of each function to 0. We use a variable pre to record the previous timestamp.

We traverse the log array. For each log entry, we first split it by colons to get the function identifier i, the operation type op, and the timestamp t.

If op is start, it means function i starts executing. We need to check if the stack is empty. If it is not empty, we add cur - pre to the exclusive time of the function at the top of the stack, then push i onto the stack and update pre to cur. If op is end, it means function i finishes executing. We add cur - pre + 1 to the exclusive time of the function at the top of the stack, then pop the top element from the stack and update pre to cur + 1.

Finally, we return the array ans.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the log array.

Python 3

```python
class Solution:
    def exclusiveTime(self, n: int, logs: List[str]) -> List[int]:
        stk = []
        ans = [0] * n
        pre = 0
        for log in logs:
            i, op, t = log.split(":")
            i, cur = int(i), int(t)
            if op[0] == "s":
                if stk:
                    ans[stk[-1]] += cur - pre
                stk.append(i)
                pre = cur
            else:
                ans[stk.pop()] += cur - pre + 1
                pre = cur + 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Inclusive time: count the time spent in called
# functions too
# 2. The deepest call depth reached


# ============================================================
# FOLLOW-UP 1: Inclusive time: count the time spent in
# called functions too
# ============================================================

# ---- follow-up 1: function ----
# Answer: Add the time of every call to every function
# that is on the stack at that moment, without subtracting
# what the callees used; with the same start/end events,
# the inclusive time of a call is simply end - start + 1.
# Time: O(n * d), d = deepest call depth.  Space: O(n).
def inclusive_times(n, logs):
    total = [0] * n
    stack = []
    for log in logs:
        function_id, kind, time = log.split(":")
        function_id, time = int(function_id), int(time)
        if kind == "start":
            stack.append((function_id, time))
        else:
            started_id, started_at = stack.pop()
            if function_id not in [item[0] for item in stack]:
                total[function_id] += time - started_at + 1      # skip time already counted by a
recursive outer call
    return total


# ============================================================
# FOLLOW-UP 2: The deepest call depth reached
# ============================================================

# ---- follow-up 2: function ----
# Answer: The size of the call stack at each start event;
# keep the biggest.
# Time: O(n).  Space: O(1).
def max_call_depth(logs):
    depth = 0
    best = 0
    for log in logs:
```

```
        kind = log.split(":")[1]
        if kind == "start":
            depth += 1
            if depth > best:
                best = depth
        else:
            depth -= 1
    return best
```

PATTERN 8 · STACK · PROBLEM 18 OF 20

921. Minimum Add to Make Parentheses Valid

Medium · Stack, Greedy, String, Parentheses

leetcode.com/problems/minimum-add-to-make-parentheses-valid

Also appears in pattern 21 Greedy

PROBLEM

A parentheses string is valid if and only if:

- It is the empty string,
- It can be written as `AB` (`A` concatenated with `B`), where `A` and `B` are valid strings, or
- It can be written as `(A)`, where `A` is a valid string.

You are given a parentheses string `s`. In one move, you can insert a parenthesis at any position of the string.

- For example, if `s = "())))"`, you can insert an opening parenthesis to be `"(**(**))))"` or a closing parenthesis to be `"())))**)**"`.

Return *the minimum number of moves required to make* `s` *valid*.

Example 1:

```
Input: s = "())"
Output: 1
```

Example 2:

```
Input: s = "((("
Output: 3
```

Constraints:

- `1 <= s.length <= 1000`
- `s[i]` is either `'('` or `')'`.

SOLUTIONS (PYTHON 3)

Solution 1: Greedy + Stack

> **How to think about it**
>
> We must add the fewest parentheses to make the string valid. Whatever remains on a matching stack needs one counterpart each. Push a left parenthesis; a right one pops a matching left if possible, otherwise it stays. The final stack size is the answer.

This problem is a classic parenthesis matching problem, which can be solved using "Greedy + Stack".

Iterate through each character c in the string s:

- If c is a left parenthesis, directly push c into the stack;
- If c is a right parenthesis, at this point if the stack is not empty, and the top element of the stack is a left parenthesis, then pop the top element of the stack, indicating a successful match; otherwise, push c into the stack.

After the iteration ends, the number of remaining elements in the stack is the number of parentheses that need to be added.

The time complexity is O(n), and the space complexity is O(n), where n is the length of the string s.

Python 3

```python
class Solution:
    def minAddToMakeValid(self, s: str) -> int:
        stk = []
        for c in s:
            if c == ')' and stk and stk[-1] == '(':
                stk.pop()
            else:
                stk.append(c)
        return len(stk)
```

Solution 2: Greedy + Counting

> **How to think about it**
>
> The stack only needs the count of unmatched left parentheses, not the characters. cnt tracks those lefts, unmatched rights go into ans, and the leftover cnt is added at the end, using constant extra space.

Solution 1 uses a stack to implement parenthesis matching, but we can also directly implement it through counting.

Define a variable `cnt` to represent the current number of left parentheses to be matched, and a variable `ans` to record the answer. Initially, both variables are set to 0.

Iterate through each character c in the string s:

- If c is a left parenthesis, increase the value of `cnt` by 1;
- If c is a right parenthesis, at this point if cnt > 0, it means that there are left parentheses that can be matched, so decrease the value of `cnt` by 1; otherwise, it means that the current right parenthesis cannot be matched, so increase the value of `ans` by 1.

After the iteration ends, add the value of `cnt` to `ans`, which is the answer.

The time complexity is O(n), and the space complexity is O(1), where n is the length of the string s.

Python 3

```python
class Solution:
    def minAddToMakeValid(self, s: str) -> int:
        ans = cnt = 0
        for c in s:
            if c == '(':
                cnt += 1
            elif cnt:
                cnt -= 1
            else:
                ans += 1
        ans += cnt
        return ans
```

Solution 3: Replace + recursion

> **How to think about it**
>
> A matched `()` pair does not affect the answer and can be deleted. Repeatedly replace one `()`; if the length does not change, nothing remains to pair and the length is the number of additions; otherwise recurse. n≤ 1000, so the replacements are affordable.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```python
class Solution:
    def minAddToMakeValid(self, s: str) -> int:
        # remove matched "()" pairs again and again; what is left is exactly the missing brackets
        while "()" in s:
            s = s.replace("()", "")
        return len(s)
```

Complexity. Time: O(n^2) in the worst case (each replace is O(n), up to n / 2 rounds). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the repaired string, not only the number of
# insertions
# 2. Minimum REMOVALS to make it valid: return the
# resulting string


# ============================================================
# FOLLOW-UP 1: Return the repaired string, not only the
# number of insertions
# ============================================================

# ---- follow-up 1: function ----
# Answer: Insert a '(' in front of every ')' that has no
# partner so far, and put one ')' at the end for every '('
# that is still open.
# Time: O(n).  Space: O(n).
def repair_by_adding(s):
    result = []
    open_count = 0
    for ch in s:
        if ch == "(":
            open_count += 1
            result.append(ch)
        else:
            if open_count == 0:
                result.append("(")
            else:
                open_count -= 1
            result.append(ch)
    result.append(")" * open_count)
    return "".join(result)


# ============================================================
# FOLLOW-UP 2: Minimum REMOVALS to make it valid: return
# the resulting string
# ============================================================

# ---- follow-up 2: function ----
# Answer: Mark the positions of unmatched ')' while
# scanning, and the unmatched '(' left on the stack at the
# end; build the answer without those positions.
# Time: O(n).  Space: O(n).
def repair_by_removing(s):
    remove = set()
    stack = []
    for i in range(len(s)):
        if s[i] == "(":
            stack.append(i)
        elif s[i] == ")":
            if stack:
                stack.pop()
            else:
                remove.add(i)
    for i in stack:
        remove.add(i)
    result = []
    for i in range(len(s)):
        if i not in remove:
            result.append(s[i])
    return "".join(result)
```

PATTERN 8 · STACK · PROBLEM 19 OF 20

946. Validate Stack Sequences

Medium · Stack, Array, Simulation

leetcode.com/problems/validate-stack-sequences

PROBLEM

Given two integer arrays `pushed` and `popped` each with distinct values, return `true` *if this could have been the result of a sequence of push and pop operations on an initially empty stack, or* `false` *otherwise.*

Example 1:

```
Input: pushed = [1,2,3,4,5], popped = [4,5,3,2,1]
Output: true
Explanation: We might do the following sequence:
push(1), push(2), push(3), push(4),
pop() -> 4,
push(5),
pop() -> 5, pop() -> 3, pop() -> 2, pop() -> 1
```

Example 2:

```
Input: pushed = [1,2,3,4,5], popped = [4,3,5,1,2]
Output: false
Explanation: 1 cannot be popped before 2.
```

Constraints:

- `1 <= pushed.length <= 1000`
- `0 <= pushed[i] <= 1000`
- All the elements of `pushed` are **unique**.
- `popped.length == pushed.length`
- `popped` is a permutation of `pushed`.

SOLUTION (PYTHON 3)

Solution 1: Stack Simulation

> **How to think about it**
>
> Check whether popped is a valid pop sequence for pushed. Values are unique, so push in given order and pop whenever the top matches the next needed pop. If every pop is consumed, the sequence is legal.

We iterate through the pushed array. For the current element x being iterated, we push it into the stack stk. Then, we check if the top element of the stack is equal to the next element to be popped in the popped array. If they are equal, we pop the top element from the stack and increment the index i of the next element to be popped in the popped array. Finally, if all elements can be popped in the order specified by the popped array, return true; otherwise, return false.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the pushed array.

Python 3

```python
class Solution:
    def validateStackSequences(self, pushed: List[int], popped: List[int]) -> bool:
        stk = []
        i = 0
        for x in pushed:
            stk.append(x)
            while stk and stk[-1] == popped[i]:
                stk.pop()
                i += 1
        return i == len(popped)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the sequence of push and pop operations that
# produces the pop order
# 2. List every possible pop order when 1..n are pushed in
# order


# ============================================================
# FOLLOW-UP 1: Return the sequence of push and pop
# operations that produces the pop order
# ============================================================

# ---- follow-up 1: function ----
# Answer: Simulate: push until the next wanted popped
# value is on top, then pop it. If the wanted value is
# buried below the top, the order is impossible.
# Time: O(n).  Space: O(n).
def stack_operations(pushed, popped):
    stack = []
    operations = []
    j = 0
    for x in pushed:
        stack.append(x)
        operations.append("push " + str(x))
        while stack and j < len(popped) and stack[-1] == popped[j]:
            operations.append("pop " + str(stack.pop()))
            j += 1
    if j == len(popped):
        return operations
    return None


# ============================================================
# FOLLOW-UP 2: List every possible pop order when 1..n are
# pushed in order
# ============================================================

# ---- follow-up 2: function ----
# Answer: Backtracking: at each step either push the next
# number or pop the top; the sequences of pops that finish
# with an empty stack are the valid orders (their count is
# the Catalan number).
# Time: O(n * Catalan(n)) (the number of pop orders).
# Space: O(n) recursion plus the results.
def all_pop_orders(n):
    results = []

    def go(next_number, stack, popped):
        if len(popped) == n:
            results.append(list(popped))
            return
        if next_number <= n:
```

```
            stack.append(next_number)
            go(next_number + 1, stack, popped)
            stack.pop()
        if stack:
            top = stack.pop()
            popped.append(top)
            go(next_number, stack, popped)
            popped.pop()
```

```
            stack.append(top)

    go(1, [], [])
    return results
```

PATTERN 9 OF 41

Monotonic Stack / Queue (Specific Variants)

20 problems · 1 Easy · 13 Medium · 6 Hard

THE IDEA

Keep a stack whose values only go up (or down). When a new value breaks the order, pop and resolve the popped ones: each element is pushed and popped once, so O(n).

Signals: next greater / smaller element, daily temperatures, largest rectangle in a histogram, trapping rain water, stock span.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
stack = []                          # indexes, values strictly decreasing
answer = [0] * len(a)
for i, x in enumerate(a):
    while stack and a[stack[-1]] < x:
        j = stack.pop()             # x is the next greater of a[j]
        answer[j] = i - j
    stack.append(i)
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Next Greater Element I (see page)	Easy	LC 496	392
2	★ **Daily Temperatures** (see page)	Medium	LC 739	407
3	★ **Next Greater Element II**	Medium	LC 503	433
4	★ **Remove K Digits**	Medium	LC 402	436
5	Remove Duplicate Letters	Medium	LC 316	438
6	★ **Sum of Subarray Minimums**	Medium	LC 907	440
7	Online Stock Span	Medium	LC 901	443
8	132 Pattern	Medium	LC 456	446
9	Car Fleet	Medium	LC 853	449
10	Next Greater Node In Linked List	Medium	LC 1019	452
11	Find the Most Competitive Subsequence	Medium	LC 1673	455
12	Maximum Width Ramp	Medium	LC 962	457
13	Sum of Subarray Ranges	Medium	LC 2104	459
14	Steps to Make Array Non-decreasing	Medium	LC 2289	463
15	★ **Largest Rectangle in Histogram**	Hard	LC 84	465
16	★ **Trapping Rain Water** (see page)	Hard	LC 42	177
17	Maximal Rectangle	Hard	LC 85	468
18	Sliding Window Maximum (see page)	Hard	LC 239	124
19	★ **Shortest Subarray with Sum at Least K**	Hard	LC 862	472
20	Constrained Subsequence Sum	Hard	LC 1425	474

503. Next Greater Element II

Medium · Stack, Array, Monotonic Stack

leetcode.com/problems/next-greater-element-ii

★ **TOP 7 PROBLEM** in Monotonic Stack / Queue (Specific Variants): master this one first

PROBLEM

Given a circular integer array `nums` (i.e., the next element of `nums[nums.length - 1]` is `nums[0]`), return *the **next greater number** for every element in* `nums`.

The **next greater number** of a number `x` is the first greater number to its traversing-order next in the array, which means you could search circularly to find its next greater number. If it doesn't exist, return `-1` for this number.

Example 1:

```
Input: nums = [1,2,1]
Output: [2,-1,2]
Explanation: The first 1's next greater number is 2;
The number 2 can't find next greater number.
The second 1's next greater number needs to search circularly, which is also 2.
```

Example 2:

```
Input: nums = [1,2,3,4,3]
Output: [2,3,4,-1,4]
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^9 <= nums[i] <= 10^9`

SOLUTION (PYTHON 3)

Solution 1: Monotonic Stack

How to think about it

A naive rightward scan for the next greater value is $O(n^2)$, tight for a circular array with $n \le 10^4$. Indices wrap after n. Walk right to left with a decreasing stack of candidates that have not yet been beaten. Two passes unroll the circle; modulo maps back to the original index. Each value enters and leaves the stack at most once.

The problem requires us to find the next greater element for each element. Therefore, we can traverse the array from back to front, which effectively turns the problem into finding the previous greater element. Additionally, since the array is circular, we can traverse the array twice.

Specifically, we start traversing the array from index n × 2 - 1, where n is the length of the array. Then, we let j = i mod n, where mod represents the modulo operation. If the stack is not empty and the top element of the stack is less than or equal to nums[j], then we continuously pop the top element of the stack until the stack is empty or the top element of the stack is greater than nums[j]. At this point, the top element of the stack is the previous greater element for nums[j], and we assign it to ans[j]. Finally, we push nums[j] onto the stack. We continue to the next element.

After the traversal is complete, we can obtain the array ans, which represents the next greater element for each element in the array nums.

The time complexity is O(n), and the space complexity is O(n), where n is the length of the array nums.

Python 3

```python
class Solution:
    def nextGreaterElements(self, nums: List[int]) -> List[int]:
        n = len(nums)
        ans = [-1] * n
        stk = []
        for i in range(n * 2 - 1, -1, -1):
            i %= n
            while stk and stk[-1] <= nums[i]:
                stk.pop()
            if stk:
                ans[i] = stk[-1]
            stk.append(nums[i])
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the POSITION of the next greater element
# instead of its value
# 2. The previous greater element in a circular array


# ============================================================
# FOLLOW-UP 1: Return the POSITION of the next greater
# element instead of its value
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same stack of positions; store the index of the
# number that popped them.
# Time: O(n).  Space: O(n).
def next_greater_position_circular(nums):
    n = len(nums)
    answer = [-1] * n
    stack = []
    for i in range(2 * n):
        value = nums[i % n]
        while stack and nums[stack[-1]] < value:
            answer[stack.pop()] = i % n
        if i < n:
            stack.append(i)
    return answer


# ============================================================
# FOLLOW-UP 2: The previous greater element in a circular
# array
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk the array backwards twice with the same
# stack: a number's previous greater (going around to the
# left) is the nearest bigger number on the stack.
# Time: O(n).  Space: O(n).
def previous_greater_circular(nums):
    n = len(nums)
    answer = [-1] * n
    stack = []
    for i in range(2 * n):
        value = nums[i % n]
        while stack and stack[-1] <= value:
            stack.pop()
        if i >= n and stack:
            answer[i % n] = stack[-1]
        stack.append(value)
```

```
    return answer
```

PATTERN 9 · MONOTONIC STACK / QUEUE (SPECIFIC VARIANTS) · PROBLEM 4 OF 20

402. Remove K Digits

Medium · Stack, Greedy, String, Monotonic Stack

leetcode.com/problems/remove-k-digits

★ **TOP 7 PROBLEM** in Monotonic Stack / Queue (Specific Variants): master this one first

Also appears in pattern 21 Greedy

PROBLEM

Given string num representing a non-negative integer `num`, and an integer `k`, return *the smallest possible integer after removing* `k` *digits from* `num`.

Example 1:

```
Input: num = "1432219", k = 3
Output: "1219"
Explanation: Remove the three digits 4, 3, and 2 to form the new number 1219 which is the smallest.
```

Example 2:

```
Input: num = "10200", k = 1
Output: "200"
Explanation: Remove the leading 1 and the number is 200. Note that the output must not contain leading zeroes.
```

Example 3:

```
Input: num = "10", k = 2
Output: "0"
Explanation: Remove all the digits from the number and it is left with nothing which is 0.
```

Constraints:

- `1 <= k <= num.length <= 10^5`
- `num` consists of only digits.
- `num` does not have any leading zeros except for the zero itself.

SOLUTION (PYTHON 3)

Solution 1: Greedy Algorithm

How to think about it

After deleting k digits we want the smallest remaining number. Equal-length integers are compared from the first differing digit, so larger digits on the left should be removed first. A non-decreasing stack from left to right pops the top whenever the current digit is smaller, spending one deletion. Then keep the leftover length and strip leading zeros. The scan has to go left to right: a smaller digit can occupy a higher place only after a larger left digit is gone.

Python 3

```python
class Solution:
    def removeKdigits(self, num: str, k: int) -> str:
        stk = []
        remain = len(num) - k
        for c in num:
            while k and stk and stk[-1] > c:
                stk.pop()
                k -= 1
            stk.append(c)
        return ''.join(stk[:remain]).lstrip('0') or '0'
```

Complexity. Time: O(n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Remove k digits to get the LARGEST possible number
# 2. Return the positions of the digits that were removed
# (smallest result)


# ============================================================
# FOLLOW-UP 1: Remove k digits to get the LARGEST possible
# number
# ============================================================

# ---- follow-up 1: function ----
# Answer: Flip the comparison: pop the top digit while it
# is smaller than the new digit and removals remain.
# Time: O(n).  Space: O(n).
def remove_k_digits_largest(num, k):
    stack = []
    for digit in num:
        while k > 0 and stack and stack[-1] < digit:
            stack.pop()
            k -= 1
        stack.append(digit)
    if k > 0:
        stack = stack[:len(stack) - k]
    return "".join(stack)


# ============================================================
# FOLLOW-UP 2: Return the positions of the digits that
# were removed (smallest result)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep positions on the stack instead of digits;
# every popped position and every position cut from the
# end is a removed digit.
# Time: O(n).  Space: O(n).
def removed_positions(num, k):
    stack = []                                      # positions of kept digits
    removed = []
    for i in range(len(num)):
        while k > 0 and stack and num[stack[-1]] > num[i]:
            removed.append(stack.pop())
            k -= 1
        stack.append(i)
    while k > 0 and stack:
        removed.append(stack.pop())
        k -= 1
    return sorted(removed)
```

316. Remove Duplicate Letters

Medium · Stack, Greedy, String, Monotonic Stack

leetcode.com/problems/remove-duplicate-letters

PROBLEM

Given a string `s`, remove duplicate letters so that every letter appears once and only once. You must make sure your result is **the smallest in lexicographical order** among all possible results.

Example 1:

```
Input: s = "bcabc"
Output: "abc"
```

Example 2:

```
Input: s = "cbacdcbc"
Output: "acdb"
```

Constraints:

- `1 <= s.length <= 10^4`
- `s` consists of lowercase English letters.

Note: This question is the same as 1081: https://leetcode.com/problems/smallest-subsequence-of-distinct-characters/

SOLUTIONS (PYTHON 3)

Solution 1: Stack

> **How to think about it**
>
> We want the lexicographically smallest subsequence that uses each present letter once. Searching all subsequences is too large. Record each letter's last index. Scan left to right: skip a letter already on the stack; otherwise pop a larger top while it still occurs later, then push. The monotone stack keeps the prefix small; last indices guarantee a popped letter can return.

We use an array `last` to record the last occurrence of each character, a stack to save the result string, and an array `vis` or an integer variable `mask` to record whether the current character is in the stack.

Traverse the string s, for each character c, if c is not in the stack, we need to check whether the top element of the stack is greater than c. If it is greater than c and the top element of the stack will appear later, we pop the top element of the stack and push c into the stack.

Finally, concatenate the elements in the stack into a string and return it as the result.

The time complexity is O(n), and the space complexity is O(n). Where n is the length of the string s.

Python 3

```python
class Solution:
    def removeDuplicateLetters(self, s: str) -> str:
        last = {c: i for i, c in enumerate(s)}
        stk = []
        vis = set()
        for i, c in enumerate(s):
            if c in vis:
                continue
            while stk and stk[-1] > c and last[stk[-1]] > i:
                vis.remove(stk.pop())
            stk.append(c)
            vis.add(c)
        return ''.join(stk)
```

Solution 2

How to think about it

Method 1 uses last indices to test leftover occurrences. Count frequencies first and decrement while scanning; pop the top while its remaining count is positive. The test is equivalent without storing last positions.

Python 3

```python
class Solution:
    def removeDuplicateLetters(self, s: str) -> str:
        count, in_stack = [0] * 128, [False] * 128
        stack = []
        for c in s:
            count[ord(c)] += 1
        for c in s:
            count[ord(c)] -= 1
            if in_stack[ord(c)]:
                continue
            while len(stack) and stack[-1] > c:
                peek = stack[-1]
                if count[ord(peek)] < 1:
                    break
                in_stack[ord(peek)] = False
                stack.pop()
            stack.append(c)
            in_stack[ord(c)] = True
        return ''.join(stack)
```

Complexity. Time: O(n). Space: O(C), C = 128 characters.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The LARGEST string that uses every distinct letter
# exactly once (a subsequence)


# ============================================================
# FOLLOW-UP 1: The LARGEST string that uses every distinct
# letter exactly once (a subsequence)
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same stack method with the opposite
# comparison: pop the top letter while it is smaller than
# the new letter and it appears again later.
# Time: O(n).  Space: O(s), s = distinct letters.
def largest_distinct_subsequence(s):
    last = {}
    for i in range(len(s)):
        last[s[i]] = i
    stack = []
    in_stack = set()
    for i in range(len(s)):
        ch = s[i]
        if ch in in_stack:
            continue
        while stack and stack[-1] < ch and last[stack[-1]] > i:
            in_stack.discard(stack.pop())
        stack.append(ch)
        in_stack.add(ch)
    return "".join(stack)
```

907. Sum of Subarray Minimums

Medium · Stack, Array, Dynamic Programming, Monotonic Stack

leetcode.com/problems/sum-of-subarray-minimums

★ **TOP 7 PROBLEM** in Monotonic Stack / Queue (Specific Variants): master this one first

Also appears in pattern 10 Queue / Deque, 35 Sliding Window Maximum / Minimum (Specific Variants)

PROBLEM

Given an array of integers arr, find the sum of `min(b)`, where `b` ranges over every (contiguous) subarray of `arr`. Since the answer may be large, return the answer **modulo** $10^9 + 7$.

Example 1:

```
Input: arr = [3,1,2,4]
Output: 17
Explanation:
Subarrays are [3], [1], [2], [4], [3,1], [1,2], [2,4], [3,1,2], [1,2,4], [3,1,2,4].
Minimums are 3, 1, 2, 4, 1, 1, 2, 1, 1, 1.
Sum is 17.
```

Example 2:

```
Input: arr = [11,81,94,43,3]
Output: 444
```

Constraints:

- `1 <= arr.length <= 3 * 10^4`
- `1 <= arr[i] <= 3 * 10^4`

SOLUTION (PYTHON 3)

Solution 1: Monotonic Stack

How to think about it

Summing the minimum of every subarray is quadratic if we enumerate intervals. Switch to contribution: multiply arr[i] by how often it is the minimum. Monotonic stacks find the previous strictly smaller value on the left and the next smaller-or-equal on the right, so equal values are credited once. The product of the two spans is the number of subarrays.

The problem asks for the sum of the minimum values of each subarray, which is equivalent to finding the number of subarrays for which each element arr[i] is the minimum, then multiplying by arr[i], and finally summing these up.

Therefore, the focus of the problem is to find the number of subarrays for which arr[i] is the minimum. For arr[i], we find the first position left[i] to its left that is less than arr[i], and the first position right[i] to its right that is less than or equal to arr[i]. The number of subarrays for which arr[i] is the minimum is (i - left[i]) × (right[i] - i).

Note, why do we find the first position right[i] to the right that is less than or equal to arr[i], rather than less than arr[i]? This is because if we find the first position right[i] to the right that is less than arr[i], it will lead to duplicate calculations.

Let's take an example to illustrate. For the following array:

The element at index 3 is 2, the first element to its left that is less than 2 is at index 0. If we find the first element to its right that is less than 2, we get index 7. That is, the subarray interval is (0, 7). Note that this is an open interval.

```
0 4 3 2 5 3 2 1
*     ^       *
```

In the same way, we can find the subarray interval for the element at index 6, and find that its subarray interval is also (0, 7). That is, the subarray intervals for the elements at index 3 and index 6 are the same. This leads to duplicate calculations.

```
0 4 3 2 5 3 2 1
*           ^ *
```

If we find the first element to its right that is less than or equal to its value, there will be no duplication, because the subarray interval for the element at index 3 becomes (0, 6), and the subarray interval for the element at index 6 is (0, 7), which are not the same.

Back to this problem, we just need to traverse the array, for each element arr[i], use a monotonic stack to find the first position left[i] to its left that is less than arr[i], and the first position right[i] to its right that is less than or equal to arr[i]. The number of subarrays for which arr[i] is the minimum is (i - left[i]) × (right[i] - i), then multiply by arr[i], and finally sum these up.

Be aware of data overflow and modulo operations.

The time complexity is O(n), and the space complexity is O(n), where n is the length of the array arr.

Python 3

```python
class Solution:
    def sumSubarrayMins(self, arr: List[int]) -> int:
        n = len(arr)
        left = [-1] * n
        right = [n] * n
        stk = []
        for i, v in enumerate(arr):
            while stk and arr[stk[-1]] >= v:
                stk.pop()
            if stk:
                left[i] = stk[-1]
            stk.append(i)

        stk = []
        for i in range(n - 1, -1, -1):
            while stk and arr[stk[-1]] > arr[i]:
                stk.pop()
            if stk:
                right[i] = stk[-1]
            stk.append(i)
        mod = 10**9 + 7
        return sum((i - left[i]) * (right[i] - i) * v for i, v in enumerate(arr)) % mod
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Sum of subarray MAXIMUMS
# 2. For every position: in how many subarrays is it the
# minimum?


# ============================================================
# FOLLOW-UP 1: Sum of subarray MAXIMUMS
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same method with the comparison reversed:
# for every number count the subarrays in which it is the
# maximum (distance to the previous greater-or-equal times
# distance to the next greater).
# Time: O(n).  Space: O(n).
def sum_of_subarray_maximums(nums):
    n = len(nums)
    left = [0] * n
    right = [0] * n
```

```
    stack = []
    for i in range(n):
        while stack and nums[stack[-1]] < nums[i]:
            stack.pop()
        left[i] = i - (stack[-1] if stack else -1)
        stack.append(i)
    stack = []
    for i in range(n - 1, -1, -1):
        while stack and nums[stack[-1]] <= nums[i]:
            stack.pop()
        right[i] = (stack[-1] if stack else n) - i
        stack.append(i)
    total = 0
    for i in range(n):
        total += nums[i] * left[i] * right[i]
    return total % (10 ** 9 + 7)


# ============================================================
# FOLLOW-UP 2: For every position: in how many subarrays
# is it the minimum?
# ============================================================

# ---- follow-up 2: function ----
# Answer: That count is the number of possible left ends
# times the number of possible right ends, which come from
# the distances to the previous smaller and the next
# smaller-or-equal numbers.
# Time: O(n).  Space: O(n).
def subarrays_where_minimum(nums):
    n = len(nums)
    left = [0] * n
    right = [0] * n
    stack = []
    for i in range(n):
        while stack and nums[stack[-1]] > nums[i]:
            stack.pop()
        left[i] = i - (stack[-1] if stack else -1)
        stack.append(i)
    stack = []
```

```
    for i in range(n - 1, -1, -1):
        while stack and nums[stack[-1]] >= nums[i]:
            stack.pop()
        right[i] = (stack[-1] if stack else n) - i
        stack.append(i)
    counts = []
    for i in range(n):
        counts.append(left[i] * right[i])
    return counts
```

901. Online Stock Span

Medium · Stack, Design, Data Stream, Monotonic Stack

leetcode.com/problems/online-stock-span

Also appears in pattern 29 Design Problems

PROBLEM

Design an algorithm that collects daily price quotes for some stock and returns **the span** of that stock's price for the current day.

The **span** of the stock's price in one day is the maximum number of consecutive days (starting from that day and going backward) for which the stock price was less than or equal to the price of that day.

- For example, if the prices of the stock in the last four days are `[7,2,1,2]` and the price of the stock today is 2, then the span of today is 4 because starting from today, the price of the stock was less than or equal to 2 for 4 consecutive days.
- Also, if the prices of the stock in the last four days is `[7,34,1,2]` and the price of the stock today is `8`, then the span of today is `3` because starting from today, the price of the stock was less than or equal `8` for `3` consecutive days.

Implement the `StockSpanner` class:

- `StockSpanner()` Initializes the object of the class.
- `int next(int price)` Returns the **span** of the stock's price given that today's price is `price`.

Example 1:

```
Input
["StockSpanner", "next", "next", "next", "next", "next", "next", "next"]
[[], [100], [80], [60], [70], [60], [75], [85]]
Output
[null, 1, 1, 1, 2, 1, 4, 6]

Explanation
StockSpanner stockSpanner = new StockSpanner();
stockSpanner.next(100); // return 1
stockSpanner.next(80); // return 1
stockSpanner.next(60); // return 1
stockSpanner.next(70); // return 2
stockSpanner.next(60); // return 1
stockSpanner.next(75); // return 4, because the last 4 prices (including today's price of 75) were
less than or equal to today's price.
stockSpanner.next(85); // return 6
```

Constraints:

- $1 <= price <= 10^5$
- At most 10^4 calls will be made to `next`.

SOLUTION (PYTHON 3)

Solution 1: Monotonic Stack

How to think about it

Scanning backward from today until a strictly higher price appears is quadratic over many queries. Spans already covered by an earlier price need not be recounted: if the stack top is at most today's price, its span can be merged in. A stack that is decreasing in price, storing (price, cnt), pops and accumulates those spans before pushing. Each price enters and leaves once, so each query is amortized constant time.

Based on the problem description, we know that for the current day's price price, we start from this price and look backwards to find the first price that is larger than this price. The difference in indices cnt between these two prices is the span of the current day's price.

This is actually a classic monotonic stack model, where we find the first element larger than the current element on the left.

We maintain a stack where the prices from the bottom to the top of the stack are monotonically decreasing. Each element in the stack is a (price, cnt) data pair, where price represents the price, and cnt represents the span of the current price.

When the price price appears, we compare it with the top element of the stack. If the price of the top element of the stack is less than or equal to price, we add the span cnt of the current day's price to the span of the top element of the stack, and then pop the top element of the stack. This continues until the price of the top element of the stack is greater than price, or the stack is empty.

Finally, we push (price, cnt) onto the stack and return cnt.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of times `next(price)` is called.

Python 3

```
class StockSpanner:
    def __init__(self):
        self.stk = []

    def next(self, price: int) -> int:
        cnt = 1
        while self.stk and self.stk[-1][0] <= price:
            cnt += self.stk.pop()[1]
        self.stk.append((price, cnt))
        return cnt


# Your StockSpanner object will be instantiated and called as such:
# obj = StockSpanner()
# param_1 = obj.next(price)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The span counts only the last K days
# 2. How many days ago was the last price that was HIGHER
# (0 if none)?


# ============================================================
# FOLLOW-UP 1: The span counts only the last K days
# ============================================================

# ---- follow-up 1: function ----
# Answer: Compute the span as before, then cap it at K.
# Time: O(1) amortised per call.  Space: O(n).
class BoundedStockSpanner:
    def __init__(self, k):
        self.k = k
        self.stack = []                              # (price, span) with decreasing prices

    def next(self, price):
        span = 1
        while self.stack and self.stack[-1][0] <= price:
            span += self.stack.pop()[1]
        self.stack.append((price, span))
        if span > self.k:
            return self.k
        return span


# ============================================================
```

```python
# FOLLOW-UP 2: How many days ago was the last price that
# was HIGHER (0 if none)?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep a stack of (price, day) with decreasing
# prices; after popping all prices not higher than today,
# the top is the last higher price.
# Time: O(1) amortised per call.  Space: O(n).
class DaysSinceHigher:
    def __init__(self):
        self.stack = []
        self.day = 0

    def next(self, price):
        self.day += 1
        while self.stack and self.stack[-1][0] <= price:
            self.stack.pop()
        answer = self.day - self.stack[-1][1] if self.stack else 0
        self.stack.append((price, self.day))
        return answer
```

456. 132 Pattern

Medium · Stack, Array, Binary Search, Ordered Set, Monotonic Stack

leetcode.com/problems/132-pattern

PROBLEM

Given an array of `n` integers `nums`, a **132 pattern** is a subsequence of three integers `nums[i]`, `nums[j]` and `nums[k]` such that `i < j < k` and `nums[i] < nums[k] < nums[j]`.

Return `true` *if there is a* ***132 pattern*** *in* `nums`*, otherwise, return* `false`*.*

Example 1:

```
Input: nums = [1,2,3,4]
Output: false
Explanation: There is no 132 pattern in the sequence.
```

Example 2:

```
Input: nums = [3,1,4,2]
Output: true
Explanation: There is a 132 pattern in the sequence: [1, 4, 2].
```

Example 3:

```
Input: nums = [-1,3,2,0]
Output: true
Explanation: There are three 132 patterns in the sequence: [-1, 3, 2], [-1, 3, 0] and [-1, 2, 0].
```

Constraints:

- `n == nums.length`
- `1 <= n <= 2 * 10^5`
- `-10^9 <= nums[i] <= 10^9`

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

We need i<j<k with nums[i]<nums[k]<nums[j]. Triple loops are O(n^3); fixing j and scanning both sides is still O(n^2). Walk right to left with a decreasing stack. vk is the largest value popped (a candidate nums[k] below the current peak). If the current value is less than vk, it can be nums[i]. The stack holds candidate peaks nums[j]. Update vk before pushing so k stays to the right of j.

Python 3

```python
class Solution:
    def find132pattern(self, nums: List[int]) -> bool:
        vk = -inf
        stk = []
        for x in nums[::-1]:
            if x < vk:
                return True
            while stk and stk[-1] < x:
                vk = stk.pop()
            stk.append(x)
        return False
```

Solution 2

How to think about it

Solution 1 is linear but tied to a right-to-left stack. We can also store the prefix minimum and, from the right, ask a Fenwick tree whether a value in (left[i],nums[i]) has been seen. After discretization the tree counts values already scanned on the right. A logarithmic factor appears; the structure is an interval counter rather than a monotone stack.

Python 3

```python
class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x, delta):
        while x <= self.n:
            self.c[x] += delta
            x += x & -x

    def query(self, x):
        s = 0
        while x:
            s += self.c[x]
            x -= x & -x
        return s


class Solution:
    def find132pattern(self, nums: List[int]) -> bool:
        s = sorted(set(nums))
        n = len(nums)
        left = [inf] * (n + 1)
        for i, x in enumerate(nums):
            left[i + 1] = min(left[i], x)
        tree = BinaryIndexedTree(len(s))
        for i in range(n - 1, -1, -1):
            x = bisect_left(s, nums[i]) + 1
            y = bisect_left(s, left[i]) + 1
            if x > y and tree.query(x - 1) > tree.query(y):
                return True
            tree.update(x, 1)
        return False
```

Complexity. Time: O(n log n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the three positions of a 132 pattern
# 2. Is there an increasing triplet (a 123 pattern)?


# ============================================================
# FOLLOW-UP 1: Return the three positions of a 132 pattern
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same right-to-left scan; keep with every
# candidate 'two' the position of the 'three' that made it
# and the position of the 'two' itself, then report them
# when a smaller 'one' appears.
# Time: O(n).  Space: O(n).
def find_132_positions(nums):
    stack = []                                      # positions, values decreasing from bottom to top
    best_two = None                                 # (value, position) of the largest valid '2'
    best_three = {}                                 # position of '2' -> position of '3'
```

```
    for i in range(len(nums) - 1, -1, -1):
        if best_two is not None and nums[i] < best_two[0]:
            return (i, best_three[best_two[1]], best_two[1])
        while stack and nums[stack[-1]] < nums[i]:
            popped = stack.pop()
            if best_two is None or nums[popped] > best_two[0]:
                best_two = (nums[popped], popped)
                best_three[popped] = i
        stack.append(i)
    return None


# ============================================================
# FOLLOW-UP 2: Is there an increasing triplet (a 123
# pattern)?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the two smallest candidates: the smallest
# value so far and the smallest value that has a smaller
# number before it. A number bigger than both completes
# the triplet.
# Time: O(n).  Space: O(1).
def increasing_triplet(nums):
    first = float("inf")
    second = float("inf")
    for x in nums:
        if x <= first:
            first = x
        elif x <= second:
            second = x
        else:
            return True
    return False
```

853. Car Fleet

Medium · Stack, Array, Sorting, Monotonic Stack

leetcode.com/problems/car-fleet

PROBLEM

There are `n` cars at given miles away from the starting mile 0, traveling to reach the mile `target`.

You are given two integer arrays `position` and `speed`, both of length `n`, where `position[i]` is the starting mile of the i^{th} car and `speed[i]` is the speed of the i^{th} car in miles per hour.

A car cannot pass another car, but it can catch up and then travel next to it at the speed of the slower car.

A **car fleet** is a single car or a group of cars driving next to each other. The speed of the car fleet is the **minimum** speed of any car in the fleet.

If a car catches up to a car fleet at the mile `target`, it will still be considered as part of the car fleet.

Return the number of car fleets that will arrive at the destination.

Example 1:

Input: target = 12, position = [10,8,0,5,3], speed = [2,4,1,1,3]

Output: 3

Explanation:

- The cars starting at 10 (speed 2) and 8 (speed 4) become a fleet, meeting each other at 12. The fleet forms at `target`.
- The car starting at 0 (speed 1) does not catch up to any other car, so it is a fleet by itself.
- The cars starting at 5 (speed 1) and 3 (speed 3) become a fleet, meeting each other at 6. The fleet moves at speed 1 until it reaches `target`.

Example 2:

Input: target = 10, position = [3], speed = [3]

Output: 1

Explanation:

There is only one car, hence there is only one fleet.

Example 3:

Input: target = 100, position = [0,2,4], speed = [4,2,1]

Output: 1

Explanation:

- The cars starting at 0 (speed 4) and 2 (speed 2) become a fleet, meeting each other at 4. The car starting at 4 (speed 1) travels to 5.
- Then, the fleet at 4 (speed 2) and the car at position 5 (speed 1) become one fleet, meeting each other at 6. The fleet moves at speed 1 until it reaches `target`.

Constraints:

- `n == position.length == speed.length`
- `1 <= n <= 10^5`
- `0 < target <= 10^6`
- `0 <= position[i] < target`
- All the values of `position` are **unique**.
- `0 < speed[i] <= 10^6`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

A car closer to the target cannot be passed; a faster car behind catches up and joins the same fleet. n≤ 10^5, so simulating overtakes is slow. Sort by position from the target backward and look at arrival times. A car slower than the fleet ahead starts a new fleet; otherwise it merges. The number of such leaders is the answer.

Python 3

```
class Solution:
    def carFleet(self, target: int, position: List[int], speed: List[int]) -> int:
        idx = sorted(range(len(position)), key=lambda i: position[i])
        ans = pre = 0
        for i in idx[::-1]:
            t = (target - position[i]) / speed[i]
            if t > pre:
                ans += 1
                pre = t
        return ans
```

Complexity. Time: O(n log n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the size of every fleet
# 2. When does the last fleet arrive?


# ============================================================
# FOLLOW-UP 1: Return the size of every fleet
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort the cars by position from the front. A car
# joins the fleet ahead of it when its arrival time is not
# later than that fleet's time; otherwise it starts a new
# fleet.
# Time: O(n log n).  Space: O(n).
def fleet_sizes(target, position, speed):
    cars = sorted(zip(position, speed), reverse=True)
    sizes = []
    slowest_time = 0.0
    for pos, spd in cars:
        time = (target - pos) / spd
        if time > slowest_time:
            sizes.append(1)
            slowest_time = time
        else:
            sizes[-1] += 1
    return sizes


# ============================================================
# FOLLOW-UP 2: When does the last fleet arrive?
# ============================================================

# ---- follow-up 2: function ----
# Answer: It is the largest arrival time among the leading
# cars of all fleets, i.e. the biggest 'time to the
# target' after merging.
# Time: O(n log n).  Space: O(n).
def last_arrival_time(target, position, speed):
```

```
    cars = sorted(zip(position, speed), reverse=True)
    slowest_time = 0.0
    for pos, spd in cars:
        time = (target - pos) / spd
        if time > slowest_time:
            slowest_time = time
    return slowest_time
```

1019. Next Greater Node In Linked List

Medium · Stack, Array, Linked List, Monotonic Stack

leetcode.com/problems/next-greater-node-in-linked-list

Also appears in pattern 24 Fast & Slow Pointers (Linked List)

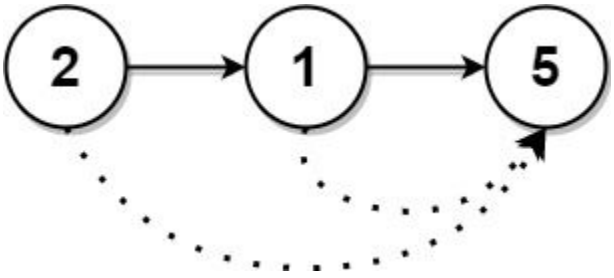
PROBLEM

You are given the `head` of a linked list with `n` nodes.

For each node in the list, find the value of the **next greater node**. That is, for each node, find the value of the first node that is next to it and has a **strictly larger** value than it.

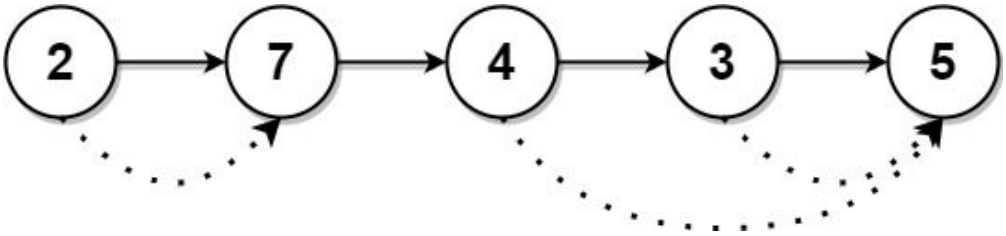
Return an integer array `answer` where `answer[i]` is the value of the next greater node of the i^{th} node (**1-indexed**). If the i^{th} node does not have a next greater node, set `answer[i] = 0`.

Example 1:




```
Input: head = [2,1,5]
Output: [5,5,0]
```

Example 2:




```
Input: head = [2,7,4,3,5]
Output: [7,0,5,5,0]
```

Constraints:

- The number of nodes in the list is `n`.
- `1 <= n <= 10^4`
- `1 <= Node.val <= 10^9`

SOLUTION (PYTHON 3)

Solution 1: Monotonic Stack

How to think about it

Scanning rightward from every node is $O(n^2)$ and tight at $n \le 10^4$. The query is the next greater element on the right, the same as the array problem. A decreasing stack from right to left holds candidates. The current value pops every top that is not strictly larger; the new top, if any, is the next greater value. We flatten the list into an array and fill the answer in one reverse pass.

The problem requires finding the next larger node for each node in the linked list, that is, finding the first node to the right of each node in the linked list that is larger than it. We first traverse the linked list and store the values in the linked list in an array nums. For each element in the array nums, we just need to find the first element to its right that is larger than it. The problem of finding the next larger element can be solved using a monotonic stack.

We traverse the array nums from back to front, maintaining a stack stk that is monotonically decreasing from the bottom to the top. During the traversal, if the top element of the stack is less than or equal to the current element, we loop to pop the top element of the stack until the top element of the stack is larger than the current element or the stack is empty.

If the stack is empty at this time, it means that the current element does not have a next larger element, otherwise, the next larger element of the current element is the top element of the stack, and we update the answer array ans. Then we push the current element into the stack and continue the traversal.

After the traversal is over, we return the answer array ans.

The time complexity is O(n), and the space complexity is O(n). Where n is the length of the linked list.

Python 3

```python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def nextLargerNodes(self, head: Optional[ListNode]) -> List[int]:
        nums = []
        while head:
            nums.append(head.val)
            head = head.next
        stk = []
        n = len(nums)
        ans = [0] * n
        for i in range(n - 1, -1, -1):
            while stk and stk[-1] <= nums[i]:
                stk.pop()
            if stk:
                ans[i] = stk[-1]
            stk.append(nums[i])
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The next SMALLER node value
# 2. Replace every node's value by its next greater value,
# in place (no new list of nodes)


# ============================================================
# FOLLOW-UP 1: The next SMALLER node value
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


def to_list(head):
    values = []
    while head:
        values.append(head.val)
        head = head.next
    return values
```

```
# ---- follow-up 1: function ----
# Answer: Copy the values to a list, then use the same
# stack with the comparison reversed.
# Time: O(n).  Space: O(n).
def next_smaller_nodes(head):
    values = to_list(head)
    answer = [0] * len(values)
    stack = []
    for i in range(len(values)):
        while stack and values[stack[-1]] > values[i]:
            answer[stack.pop()] = values[i]
        stack.append(i)
    return answer


# ============================================================
# FOLLOW-UP 2: Replace every node's value by its next
# greater value, in place (no new list of nodes)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk the list once keeping a stack of NODES
# waiting for a greater value; when a bigger value
# arrives, write it into the waiting nodes (the nodes keep
# their original values elsewhere, so store them first).
# Time: O(n).  Space: O(n).
def replace_with_next_greater(head):
    waiting = []                                    # nodes whose next greater value is still unknown
    node = head
    while node:
        while waiting and waiting[-1][0] < node.val:
            waiting.pop()[1].val = node.val
        waiting.append((node.val, node))
        node = node.next
    for value, pending in waiting:
        pending.val = 0
```

```
    return head
```

1673. Find the Most Competitive Subsequence

Medium · Stack, Greedy, Array, Monotonic Stack

leetcode.com/problems/find-the-most-competitive-subsequence

Also appears in pattern 39 Game Theory / Minimax

PROBLEM

Given an integer array `nums` and a positive integer `k`, return *the most **competitive** subsequence of* `nums` *of size* `k`.

An array's subsequence is a resulting sequence obtained by erasing some (possibly zero) elements from the array.

We define that a subsequence `a` is more **competitive** than a subsequence `b` (of the same length) if in the first position where `a` and `b` differ, subsequence `a` has a number **less** than the corresponding number in `b`. For example, `[1,3,4]` is more competitive than `[1,3,5]` because the first position they differ is at the final number, and `4` is less than `5`.

Example 1:

```
Input: nums = [3,5,2,6], k = 2
Output: [2,6]
Explanation: Among the set of every possible subsequence: {[3,5], [3,2], [3,6], [5,2], [5,6], [2,6]},
[2,6] is the most competitive.
```

Example 2:

```
Input: nums = [2,4,3,3,5,4,9,6], k = 4
Output: [2,3,3,4]
```

Constraints:

- `1 <= nums.length <= 10^5`
- `0 <= nums[i] <= 10^9`
- `1 <= k <= nums.length`

SOLUTION (PYTHON 3)

Solution 1: Stack

> **How to think about it**
>
> The most competitive subsequence of length k is the lexicographically smallest subsequence of that length. n is 10^5, so a monotone stack pops a larger top while enough elements remain to fill k. Push the current value when the stack is shorter than k; the stack is the answer.

We traverse the array `nums` from left to right, maintaining a stack `stk`. During the traversal, if the current element `nums[i]` is less than the top element of the stack, and the number of elements in the stack plus n-i is greater than k, then we pop the top element of the stack until the above condition is no longer satisfied. At this point, if the number of elements in the stack is less than k, then we push the current element into the stack.

After the traversal, the elements in the stack are the answer.

The time complexity is O(n), and the space complexity is O(k). Where n is the length of the array `nums`.

Python 3

```python
class Solution:
    def mostCompetitive(self, nums: List[int], k: int) -> List[int]:
        stk = []
        n = len(nums)
        for i, v in enumerate(nums):
            while stk and stk[-1] > v and len(stk) + n - i > k:
                stk.pop()
            if len(stk) < k:
                stk.append(v)
        return stk
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The LARGEST subsequence of size k
# 2. Return the positions of the chosen elements


# ============================================================
# FOLLOW-UP 1: The LARGEST subsequence of size k
# ============================================================

# ---- follow-up 1: function ----
# Answer: Flip the comparison of the stack method: pop a
# smaller top while enough numbers remain to still fill k
# places.
# Time: O(n).  Space: O(k).
def largest_subsequence(nums, k):
    stack = []
    for i in range(len(nums)):
        while stack and stack[-1] < nums[i] and len(stack) - 1 + len(nums) - i >= k:
            stack.pop()
        if len(stack) < k:
            stack.append(nums[i])
    return stack


# ============================================================
# FOLLOW-UP 2: Return the positions of the chosen elements
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep positions on the stack and compare the
# numbers at those positions.
# Time: O(n).  Space: O(k).
def competitive_positions(nums, k):
    stack = []
    for i in range(len(nums)):
        while stack and nums[stack[-1]] > nums[i] and len(stack) - 1 + len(nums) - i >= k:
            stack.pop()
        if len(stack) < k:
            stack.append(i)
    return stack
```

962. Maximum Width Ramp

Medium · Stack, Array, Two Pointers, Monotonic Stack

leetcode.com/problems/maximum-width-ramp

PROBLEM

A **ramp** in an integer array `nums` is a pair `(i, j)` for which `i < j` and `nums[i] <= nums[j]`. The **width** of such a ramp is `j - i`.

Given an integer array `nums`, return *the maximum width of a **ramp** in* `nums`. If there is no **ramp** in `nums`, return `0`.

Example 1:

```
Input: nums = [6,0,8,2,1,5]
Output: 4
Explanation: The maximum width ramp is achieved at (i, j) = (1, 5): nums[1] = 0 and nums[5] = 5.
```

Example 2:

```
Input: nums = [9,8,1,0,1,9,4,0,4,1]
Output: 7
Explanation: The maximum width ramp is achieved at (i, j) = (2, 9): nums[2] = 1 and nums[9] = 1.
```

Constraints:

- `2 <= nums.length <= 5 * 10^4`
- `0 <= nums[i] <= 5 * 10^4`

SOLUTION (PYTHON 3)

Solution 1: Monotonic Stack

> **How to think about it**
>
> Find the widest i<j with nums[i]≤ nums[j]. All pairs are quadratic. Useful left ends form a strictly decreasing sequence from the start; a later larger value is never a better left. A monotonic stack stores those candidates, then j walks from the right and pops every stack top that forms a ramp, keeping the maximum width.

According to the problem, we can find that the subsequence formed by all possible nums[i] must be monotonically decreasing. Why is that? Let's prove it by contradiction.

Suppose there exist $i_1<i_2$ and $nums[i_1]\le nums[i_2]$, then actually $nums[i_2]$ cannot be a candidate value, because $nums[i_1]$ is more to the left and would be a better value. Therefore, the subsequence formed by nums[i] must be monotonically decreasing, and i must start from 0.

We use a monotonically decreasing stack stk (from bottom to top) to store all possible nums[i]. Then we traverse j starting from the right boundary. If we encounter $nums[stk.top()]\le nums[j]$, it means a ramp is formed at this point. We cyclically pop the top elements from the stack and update ans.

The time complexity is O(n) and the space complexity is O(n), where n represents the length of nums.

Python 3

```python
class Solution:
    def maxWidthRamp(self, nums: List[int]) -> int:
        stk = []
        for i, v in enumerate(nums):
            if not stk or nums[stk[-1]] > v:
                stk.append(i)
        ans = 0
        for i in range(len(nums) - 1, -1, -1):
            while stk and nums[stk[-1]] <= nums[i]:
                ans = max(ans, i - stk.pop())
            if not stk:
                break
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the two positions of the widest ramp
# 2. A STRICT ramp (nums[i] < nums[j])


# ============================================================
# FOLLOW-UP 1: Return the two positions of the widest ramp
# ============================================================

# ---- follow-up 1: function ----
# Answer: Candidates for the left end are the strictly
# decreasing prefix minimums (kept on a stack); scan the
# right end from the back and pop every candidate it can
# pair with, remembering the widest pair.
# Time: O(n).  Space: O(n).
def widest_ramp_positions(nums):
    candidates = []
    for i in range(len(nums)):
        if not candidates or nums[i] < nums[candidates[-1]]:
            candidates.append(i)
    best = (0, 0)
    for j in range(len(nums) - 1, -1, -1):
        while candidates and nums[candidates[-1]] <= nums[j]:
            i = candidates.pop()
            if j - i > best[1] - best[0]:
                best = (i, j)
    return best


# ============================================================
# FOLLOW-UP 2: A STRICT ramp (nums[i] < nums[j])
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same method with '<' instead of '<=' when
# pairing; equal values no longer form a ramp.
# Time: O(n).  Space: O(n).
def widest_strict_ramp(nums):
    candidates = []
    for i in range(len(nums)):
        if not candidates or nums[i] < nums[candidates[-1]]:
            candidates.append(i)
    best = 0
    for j in range(len(nums) - 1, -1, -1):
        while candidates and nums[candidates[-1]] < nums[j]:
            i = candidates.pop()
            if j - i > best:
                best = j - i
    return best
```

2104. Sum of Subarray Ranges

Medium · Stack, Array, Monotonic Stack

leetcode.com/problems/sum-of-subarray-ranges

Also appears in pattern 35 Sliding Window Maximum / Minimum (Specific Variants)

PROBLEM

You are given an integer array `nums`. The **range** of a subarray of `nums` is the difference between the largest and smallest element in the subarray.

Return *the **sum of all** subarray ranges of* `nums`.

A subarray is a contiguous **non-empty** sequence of elements within an array.

Example 1:

```
Input: nums = [1,2,3]
Output: 4
Explanation: The 6 subarrays of nums are the following:
[1], range = largest - smallest = 1 - 1 = 0
[2], range = 2 - 2 = 0
[3], range = 3 - 3 = 0
[1,2], range = 2 - 1 = 1
[2,3], range = 3 - 2 = 1
[1,2,3], range = 3 - 1 = 2
So the sum of all ranges is 0 + 0 + 0 + 1 + 1 + 2 = 4.
```

Example 2:

```
Input: nums = [1,3,3]
Output: 4
Explanation: The 6 subarrays of nums are the following:
[1], range = largest - smallest = 1 - 1 = 0
[3], range = 3 - 3 = 0
[3], range = 3 - 3 = 0
[1,3], range = 3 - 1 = 2
[3,3], range = 3 - 3 = 0
[1,3,3], range = 3 - 1 = 2
So the sum of all ranges is 0 + 0 + 0 + 2 + 0 + 2 = 4.
```

Example 3:

```
Input: nums = [4,-2,-3,4,1]
Output: 59
Explanation: The sum of all subarray ranges of nums is 59.
```

Constraints:

- `1 <= nums.length <= 1000`
- -10^9 `<= nums[i] <=` 10^9

Follow-up: Could you find a solution with `O(n)` time complexity?

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

The sum of ranges is the sum of max-min over every subarray. With n≤ 1000, enumerating endpoints while maintaining the current max and min is $O(n^2)$ and acceptable. The inner scan need not restart: after fixing left index i, extending j only updates mi and mx and adds their difference. A double loop therefore accumulates every range.

Python 3

```python
class Solution:
    def subArrayRanges(self, nums: List[int]) -> int:
        ans, n = 0, len(nums)
        for i in range(n - 1):
            mi = mx = nums[i]
            for j in range(i + 1, n):
                mi = min(mi, nums[j])
                mx = max(mx, nums[j])
                ans += mx - mi
        return ans
```

Solution 2

How to think about it

Solution 1 becomes heavy for larger n. Each value's contribution as a subarray maximum (or minimum) is its value times the number of subarrays where it attains that extremum, so the range sum is "max contributions minus min contributions". A monotonic stack finds, for each index, the previous greater-or-equal and the next strictly greater positions, counting those subarrays in linear time. Negating the array and repeating yields the minimum side. We therefore implement f and return f(nums)+f([-v]).

Python 3

```python
class Solution:
    def subArrayRanges(self, nums: List[int]) -> int:
        def f(nums):
            stk = []
            n = len(nums)
            left = [-1] * n
            right = [n] * n
            for i, v in enumerate(nums):
                while stk and nums[stk[-1]] <= v:
                    stk.pop()
                if stk:
                    left[i] = stk[-1]
                stk.append(i)
            stk = []
            for i in range(n - 1, -1, -1):
                while stk and nums[stk[-1]] < nums[i]:
                    stk.pop()
                if stk:
                    right[i] = stk[-1]
                stk.append(i)
            return sum((i - left[i]) * (right[i] - i) * v for i, v in enumerate(nums))

        mx = f(nums)
        mi = f([-v for v in nums])
        return mx + mi
```

Complexity. Time: O(n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. O(n) instead of O(n squared): sum of maximums minus
# sum of minimums
# 2. Only subarrays of a fixed length k


# ============================================================
# FOLLOW-UP 1: O(n) instead of O(n squared): sum of
# maximums minus sum of minimums
# ============================================================

# ---- follow-up 1: function ----
# Answer: The range of a subarray is its maximum minus its
# minimum, so the total is (sum of subarray maximums) -
# (sum of subarray minimums), each computed with a
# monotonic stack that counts for every number the
# subarrays in which it is the extreme.
# Time: O(n).  Space: O(n).
def sum_of_extremes(nums, bigger):
    # sum over all subarrays of the maximum (bigger=True) or the minimum (bigger=False)
    n = len(nums)
    left = [0] * n
    right = [0] * n
    stack = []
    for i in range(n):
        while stack and ((nums[stack[-1]] < nums[i]) if bigger else (nums[stack[-1]] > nums[i])):
            stack.pop()
        left[i] = i - (stack[-1] if stack else -1)
        stack.append(i)
    stack = []
    for i in range(n - 1, -1, -1):
        while stack and ((nums[stack[-1]] <= nums[i]) if bigger else (nums[stack[-1]] >= nums[i])):
            stack.pop()
        right[i] = (stack[-1] if stack else n) - i
        stack.append(i)
    total = 0
    for i in range(n):
        total += nums[i] * left[i] * right[i]
    return total


def sum_of_subarray_ranges_fast(nums):
    return sum_of_extremes(nums, True) - sum_of_extremes(nums, False)


# ============================================================
# FOLLOW-UP 2: Only subarrays of a fixed length k
# ============================================================

# ---- follow-up 2: function ----
# Answer: Slide a window of size k and keep two deques
# (for the maximum and for the minimum); add max - min of
# every window.
# Time: O(n).  Space: O(k).
from collections import deque


def window_range_sum(nums, k):
    biggest = deque()
    smallest = deque()
```

```
    total = 0
    for i in range(len(nums)):
        while biggest and nums[biggest[-1]] <= nums[i]:
            biggest.pop()
```

```
        biggest.append(i)
        while smallest and nums[smallest[-1]] >= nums[i]:
            smallest.pop()
        smallest.append(i)
        if biggest[0] <= i - k:
            biggest.popleft()
        if smallest[0] <= i - k:
            smallest.popleft()
        if i >= k - 1:
            total += nums[biggest[0]] - nums[smallest[0]]
    return total
```

2289. Steps to Make Array Non-decreasing

Medium · Stack, Array, Linked List, Dynamic Programming, Monotonic Stack, Simulation

leetcode.com/problems/steps-to-make-array-non-decreasing

PROBLEM

You are given a **0-indexed** integer array `nums`. In one step, **remove** all elements `nums[i]` where `nums[i - 1] > nums[i]` for all `0 < i < nums.length`.

Return *the number of steps performed until* `nums` *becomes a* ***non-decreasing*** *array*.

Example 1:

```
Input: nums = [5,3,4,4,7,3,6,11,8,5,11]
Output: 3
Explanation: The following are the steps performed:
- Step 1: [5,3,4,4,7,3,6,11,8,5,11] becomes [5,4,4,7,6,11,11]
- Step 2: [5,4,4,7,6,11,11] becomes [5,4,7,11,11]
- Step 3: [5,4,7,11,11] becomes [5,7,11,11]
[5,7,11,11] is a non-decreasing array. Therefore, we return 3.
```

Example 2:

```
Input: nums = [4,5,7,7,13]
Output: 0
Explanation: nums is already a non-decreasing array. Therefore, we return 0.
```

Constraints:

- `1 <= nums.length <= 10^5`
- `1 <= nums[i] <= 10^9`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Each round deletes every element that is strictly smaller than its left neighbor; we want the number of rounds. n ≤ 10^5 forbids simulating rounds. The deletion time of an index is determined by how long a decreasing suffix to its right takes to be absorbed by a larger left value. A stack from the right holds indices not yet eaten. Popping updates dp[i] = max(dp[i]+1, dp[top]). The answer is max(dp).

Python 3

```python
class Solution:
    def totalSteps(self, nums: List[int]) -> int:
        stk = []
        ans, n = 0, len(nums)
        dp = [0] * n
        for i in range(n - 1, -1, -1):
            while stk and nums[i] > nums[stk[-1]]:
                dp[i] = max(dp[i] + 1, dp[stk.pop()])
            stk.append(i)
        return max(dp)
```

Complexity. Time: O(n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Show the array after every step (simulate)
# 2. O(n) with a stack: how many steps does each element
# survive?


# ============================================================
# FOLLOW-UP 1: Show the array after every step (simulate)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Each step removes every number that is smaller
# than the number before it (taken from the array as it
# was at the start of the step).
# Time: O(n^2) (one pass per step, and a copy of the array
# per step).  Space: O(n^2).
def steps_with_history(nums):
    history = [list(nums)]
    current = list(nums)
    while True:
        remaining = [current[0]] if current else []
        for i in range(1, len(current)):
            if current[i] >= current[i - 1]:
                remaining.append(current[i])
        if len(remaining) == len(current):
            return history
        history.append(remaining)
        current = remaining


# ============================================================
# FOLLOW-UP 2: O(n) with a stack: how many steps does each
# element survive?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep a stack of (value, steps it needs to be
# removed). A number removes every smaller number on the
# stack; its own survival time is the largest of their
# times plus one (0 if nothing was removed).
# Time: O(n).  Space: O(n).
def total_steps(nums):
    stack = []                                      # (value, steps until it is removed)
    answer = 0
    for x in nums:
        steps = 0
        while stack and stack[-1][0] <= x:
            popped_steps = stack.pop()[1]
            if popped_steps > steps:
                steps = popped_steps
        if stack:
            steps += 1
        else:
            steps = 0
        stack.append((x, steps))
        if steps > answer:
            answer = steps
    return answer
```

84. Largest Rectangle in Histogram

Hard · Stack, Array, Monotonic Stack, Range Query

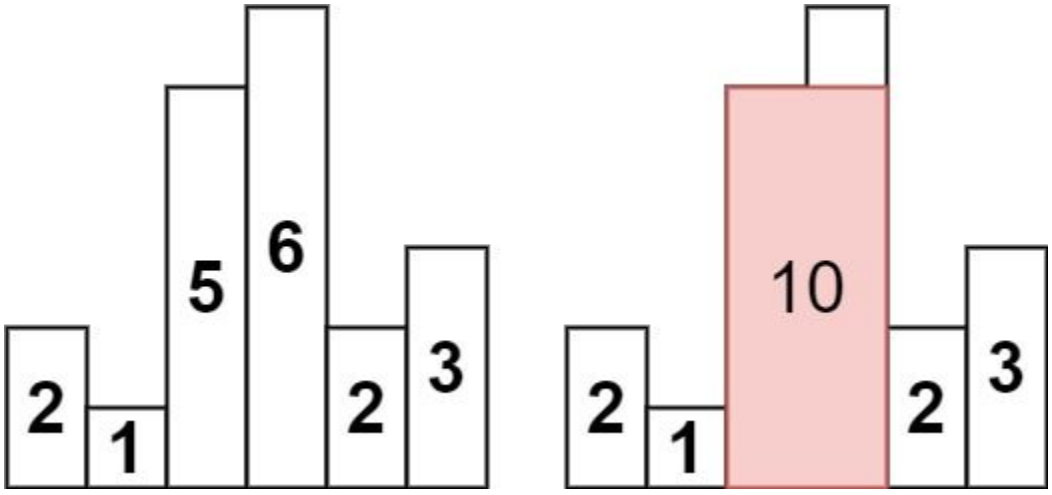
leetcode.com/problems/largest-rectangle-in-histogram

★ **TOP 7 PROBLEM** in Monotonic Stack / Queue (Specific Variants): master this one first

PROBLEM

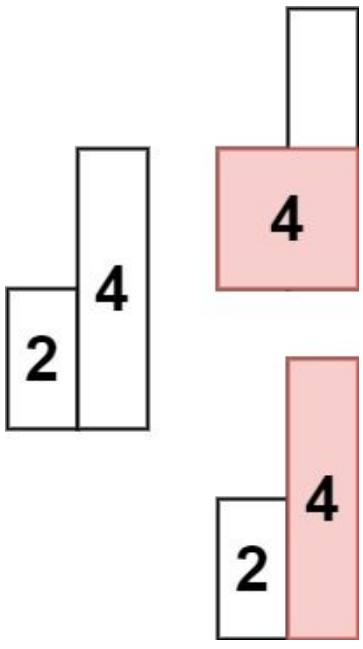
Given an array of integers `heights` representing the histogram's bar height where the width of each bar is `1`, return *the area of the largest rectangle in the histogram*.

Example 1:




```
Input: heights = [2,1,5,6,2,3]
Output: 10
Explanation: The above is a histogram where width of each bar is 1.
The largest rectangle is shown in the red area, which has an area = 10 units.
```

Example 2:



```
Input: heights = [2,4]
Output: 4
```

Constraints:

- `1 <= heights.length <= 10^5`
- `0 <= heights[i] <= 10^4`

SOLUTION (PYTHON 3)

Solution 1: Monotonic Stack

How to think about it

The first idea is enumerate every interval [l, r], take the min height, area (r - l + 1) × min. Correct, but $O(n^2)$. $n \le 10^5$ will time out. The bottleneck is recomputing the min over every range. Flip the enumeration: let bar i be the shortest bar of the rectangle; the width is the span to the nearest strictly shorter bars on both sides. “Nearest smaller on left/right” is the monotonic-stack model. Indices in the stack have increasing heights; a shorter bar pops others and becomes their right bound, while the new top is the left bound. Each bar enters and leaves once, so one linear pass.

We can enumerate the height h of each bar as the height of the rectangle. Using a monotonic stack, we find the index $left_i$, $right_i$ of the first bar with a height less than h to the left and right. The area of the rectangle at this time is h × ($right_i$-$left_i$-1). We can find the maximum value.

The time complexity is O(n), and the space complexity is O(n). Here, n represents the length of heights.

Common model of monotonic stack: Find the **nearest** number to the left/right of each number that is **larger/smaller** than it. Template:

```
stk = []
for i in range(n):
    while stk and check(stk[-1], i):
        stk.pop()
    stk.append(i)
```

Python 3

```
class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        n = len(heights)
        stk = []
        left = [-1] * n
        right = [n] * n
        for i, h in enumerate(heights):
            while stk and heights[stk[-1]] >= h:
                right[stk[-1]] = i
                stk.pop()
            if stk:
                left[i] = stk[-1]
            stk.append(i)
        return max(h * (right[i] - left[i] - 1) for i, h in enumerate(heights))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return where the largest rectangle is (left, right,
# height)
# 2. The largest SQUARE inside the histogram


# ========================================================
# FOLLOW-UP 1: Return where the largest rectangle is
# (left, right, height)
```

```
# ============================================================

# ---- follow-up 1: function ----
# Answer: In the stack method, when a bar is popped it is
# the height; its width runs from just after the new top
# of the stack to just before the current bar. Remember
# those bounds for the best area.
# Time: O(n).  Space: O(n).
def largest_rectangle_position(heights):
    stack = []
    best = (0, 0, 0, 0)                              # (area, left, right, height)
    for i in range(len(heights) + 1):
        current = heights[i] if i < len(heights) else 0
        while stack and heights[stack[-1]] >= current:
            height = heights[stack.pop()]
            left = stack[-1] + 1 if stack else 0
            area = height * (i - left)
            if area > best[0]:
                best = (area, left, i - 1, height)
        stack.append(i)
    return best[1], best[2], best[3]


# ============================================================
# FOLLOW-UP 2: The largest SQUARE inside the histogram
# ============================================================

# ---- follow-up 2: function ----
# Answer: At every popped bar the available rectangle is
# height x width; the biggest square inside it has a side
# equal to the smaller of the two.
# Time: O(n).  Space: O(n).
def largest_square_in_histogram(heights):
    stack = []
    best = 0
    for i in range(len(heights) + 1):
        current = heights[i] if i < len(heights) else 0
        while stack and heights[stack[-1]] >= current:
            height = heights[stack.pop()]
            left = stack[-1] + 1 if stack else 0
            width = i - left
            side = height if height < width else width
            if side > best:
                best = side
        stack.append(i)
    return best * best
```

85. Maximal Rectangle

Hard · Stack, Array, Dynamic Programming, Matrix, Monotonic Stack

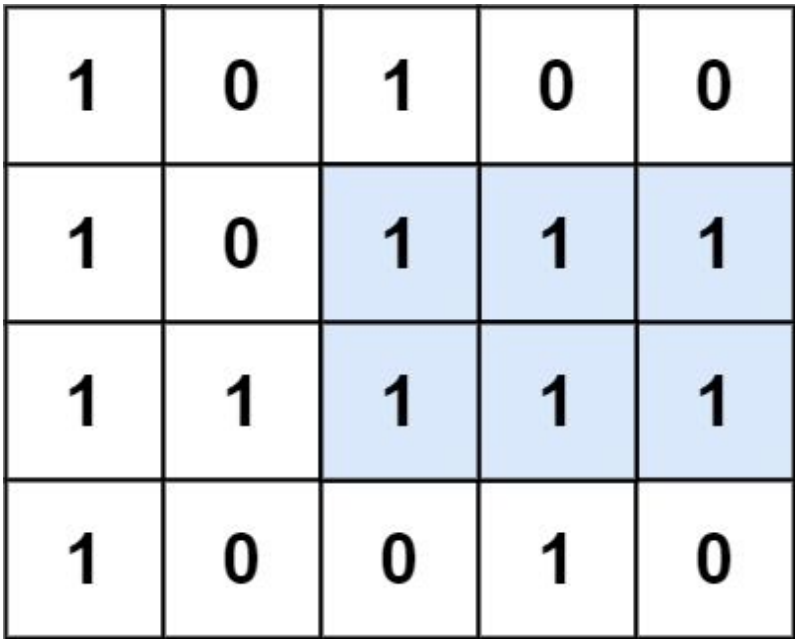
leetcode.com/problems/maximal-rectangle

Also appears in pattern 22 Dynamic Programming - 2D / Grid

PROBLEM

Given a `rows x cols` binary `matrix` filled with `0`'s and `1`'s, find the largest rectangle containing only `1`'s and return *its area*.

Example 1:




```
Input: matrix =
[["1","0","1","0","0"],["1","0","1","1","1"],["1","1","1","1","1"],["1","0","0","1","0"]]
Output: 6
Explanation: The maximal rectangle is shown in the above picture.
```

Example 2:

```
Input: matrix = [["0"]]
Output: 0
```

Example 3:

```
Input: matrix = [["1"]]
Output: 1
```

Constraints:

- `rows == matrix.length`
- `cols == matrix[i].length`
- `1 <= rows, cols <= 200`
- `matrix[i][j]` is `'0'` or `'1'`.

SOLUTION (PYTHON 3)

Solution 1: Monotonic Stack

> **How to think about it**
>
> The first idea is pick two cells as opposite corners and test whether the rectangle is all 1s. Correct, but $O(m^2 n^2)$. m, n ≤ 200 will time out. The bottleneck is rechecking 2D ranges. Observation: the count of consecutive 1s upward from row i, column j is exactly a histogram bar. Each row becomes “largest rectangle in histogram.” That is problem 84. Maintain heights[j] per row: increment on 1, reset on 0, then run the monotonic stack. Total time O(mn).

We can treat each row as the base of a histogram and calculate the maximum area of the histogram for each row.

Specifically, we maintain an array heights with the same length as the number of columns in the matrix, where heights[j] represents the height of the column at the j-th position with the current row as the base. For each row, we iterate through each column:

- If the current element is '1', increment heights[j] by 1.
- If the current element is '0', set heights[j] to 0.

Then, we use the monotonic stack algorithm to calculate the maximum rectangle area of the current histogram and update the answer.

The specific steps of the monotonic stack are as follows:

1. Initialize an empty stack stk to store the indices of the columns.
2. Initialize two arrays left and right, representing the index of the first column to the left and right of each column that is shorter than the current column.
3. Iterate through the heights array heights, first calculating the index of the first column to the left of each column that is shorter than the current column, and store it in left.
4. Then iterate through the heights array heights in reverse order, calculating the index of the first column to the right of each column that is shorter than the current column, and store it in right.
5. Finally, calculate the maximum rectangle area for each column and update the answer.

The time complexity is O(m × n), where m is the number of rows in matrix and n is the number of columns in matrix.

Python 3

```python
class Solution:
    def maximalRectangle(self, matrix: List[List[str]]) -> int:
        heights = [0] * len(matrix[0])
        ans = 0
        for row in matrix:
            for j, v in enumerate(row):
                if v == "1":
                    heights[j] += 1
                else:
                    heights[j] = 0
            ans = max(ans, self.largestRectangleArea(heights))
        return ans

    def largestRectangleArea(self, heights: List[int]) -> int:
        n = len(heights)
        stk = []
        left = [-1] * n
        right = [n] * n
        for i, h in enumerate(heights):
            while stk and heights[stk[-1]] >= h:
                stk.pop()
            if stk:
                left[i] = stk[-1]
            stk.append(i)
        stk = []
        for i in range(n - 1, -1, -1):
            h = heights[i]
            while stk and heights[stk[-1]] >= h:
                stk.pop()
            if stk:
                right[i] = stk[-1]
            stk.append(i)
        return max(h * (right[i] - left[i] - 1) for i, h in enumerate(heights))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the corners of the largest rectangle of ones


# ============================================================
# FOLLOW-UP 1: Return the corners of the largest rectangle
# of ones
# ============================================================

# ---- follow-up 1: function ----
# Answer: Build the histogram row by row as before; for
# every popped bar the rectangle's bottom row is the
# current row, its top row is row - height + 1, and its
# left and right columns come from the stack.
# Time: O(m * n).  Space: O(n).
def maximal_rectangle_corners(matrix):
    cols = len(matrix[0])
    heights = [0] * cols
    best = (0, None)
    for r in range(len(matrix)):
        for c in range(cols):
            if matrix[r][c] == "1" or matrix[r][c] == 1:
                heights[c] += 1
            else:
                heights[c] = 0
        stack = []
        for i in range(cols + 1):
            current = heights[i] if i < cols else 0
            while stack and heights[stack[-1]] >= current:
```

```
                height = heights[stack.pop()]
                left = stack[-1] + 1 if stack else 0
                area = height * (i - left)
                if area > best[0]:
                    best = (area, (r - height + 1, left, r, i - 1))
            stack.append(i)
    return best[1]
```

862. Shortest Subarray with Sum at Least K

Hard · Queue, Array, Binary Search, Prefix Sum, Sliding Window, Monotonic Queue, Heap (Priority Queue)

leetcode.com/problems/shortest-subarray-with-sum-at-least-k

★ **TOP 7 PROBLEM** in Monotonic Stack / Queue (Specific Variants), Queue / Deque, Sliding Window Maximum / Minimum (Specific Variants): master this one first

Also appears in pattern 10 Queue / Deque, 35 Sliding Window Maximum / Minimum (Specific Variants)

PROBLEM

Given an integer array `nums` and an integer `k`, return *the length of the shortest non-empty* ***subarray*** *of* `nums` *with a sum of at least* `k`. If there is no such **subarray**, return `-1`.

A **subarray** is a **contiguous** part of an array.

Example 1:

```
Input: nums = [1], k = 1
Output: 1
```

Example 2:

```
Input: nums = [1,2], k = 4
Output: -1
```

Example 3:

```
Input: nums = [2,-1,2], k = 3
Output: 3
```

Constraints:

- `1 <= nums.length <= 10^5`
- `-10^5 <= nums[i] <= 10^5`
- `1 <= k <= 10^9`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

The shortest subarray whose sum is at least k. Negatives break a sliding window, and n≤ 10^5 forbids quadratic enumeration. On prefix sums we want the smallest j-i with s[j]-s[i]≥ k. Keep a monotone increasing deque of indices: pop the front when it already yields ≥ k, and drop a tail that is at least the current prefix, because a later, smaller prefix is a better start.

Python 3

```python
class Solution:
    def shortestSubarray(self, nums: List[int], k: int) -> int:
        s = list(accumulate(nums, initial=0))
        q = deque()
        ans = inf
        for i, v in enumerate(s):
            while q and v - s[q[0]] >= k:
                ans = min(ans, i - q.popleft())
            while q and s[q[-1]] >= v:
                q.pop()
            q.append(i)
        return -1 if ans == inf else ans
```

Complexity. Time: O(n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the positions of the shortest subarray


# ============================================================
# FOLLOW-UP 1: Return the positions of the shortest
# subarray
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same deque of prefix-sum positions; when a
# window is found remember its start (the position taken
# from the front) and end.
# Time: O(n).  Space: O(n).
from collections import deque


def shortest_subarray_positions_with_negatives(nums, k):
    prefix = [0]
    for x in nums:
        prefix.append(prefix[-1] + x)
    candidates = deque()
    best = None
    for i in range(len(prefix)):
        while candidates and prefix[i] - prefix[candidates[0]] >= k:
            start = candidates.popleft()
            if best is None or i - start < best[1] - best[0] + 1:
                best = (start, i - 1)
        while candidates and prefix[candidates[-1]] >= prefix[i]:
            candidates.pop()
        candidates.append(i)
    return best
```

1425. Constrained Subsequence Sum

Hard · Queue, Array, Dynamic Programming, Sliding Window, Monotonic Queue, Heap (Priority Queue)

leetcode.com/problems/constrained-subsequence-sum

★ **TOP 7 PROBLEM** in Sliding Window Maximum / Minimum (Specific Variants): master this one first

Also appears in pattern 10 Queue / Deque, 35 Sliding Window Maximum / Minimum (Specific Variants), 37 Two Heaps

PROBLEM

Given an integer array `nums` and an integer `k`, return the maximum sum of a **non-empty** subsequence of that array such that for every two **consecutive** integers in the subsequence, `nums[i]` and `nums[j]`, where `i < j`, the condition `j - i <= k` is satisfied.

A *subsequence* of an array is obtained by deleting some number of elements (can be zero) from the array, leaving the remaining elements in their original order.

Example 1:

```
Input: nums = [10,2,-10,5,20], k = 2
Output: 37
Explanation: The subsequence is [10, 2, 5, 20].
```

Example 2:

```
Input: nums = [-1,-2,-3], k = 1
Output: -1
Explanation: The subsequence must be non-empty, so we choose the largest number.
```

Example 3:

```
Input: nums = [10,-2,-10,-5,20], k = 2
Output: 23
Explanation: The subsequence is [10, -2, -5, 20].
```

Constraints:

- `1 <= k <= nums.length <= 10^5`
- `-10^4 <= nums[i] <= 10^4`

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming + Monotonic Queue

How to think about it

Adjacent indices in the subsequence differ by at most k. n≤ 10^5, so scanning [i-k,i-1] for every i is O(nk). f[i]=nums[i]+max(0,$\max_{i-k\le j<i}$f[j]) is a sliding-window maximum on f. A decreasing deque of indices makes each transition amortized O(1). Values may be negative, so we may start a new subsequence at i. The answer is the maximum f[i].

We define f[i] to represent the maximum sum of the subsequence ending at nums[i] that meets the conditions. Initially, f[i] = 0, and the answer is $\max_{0 \le i < n} f(i)$.

We notice that the problem requires us to maintain the maximum value of a sliding window, which is a typical application scenario for a monotonic queue. We can use a monotonic queue to optimize the dynamic programming transition.

We maintain a monotonic queue q that is decreasing from the front to the back, storing the indices i. Initially, we add a sentinel 0 to the queue.

We traverse i from 0 to n - 1. For each i, we perform the following operations:

- If the front element q[0] satisfies i - q[0] > k, it means the front element is no longer within the sliding window, and we need to remove the front element from the queue;
- Then, we calculate f[i] = max(0, f[q[0]]) + nums[i], which means we add nums[i] to the sliding window to get the maximum subsequence sum;
- Next, we update the answer ans = max(ans, f[i]);
- Finally, we add i to the back of the queue and maintain the monotonicity of the queue. If f[q[back]] ≤ f[i], we need to remove the back element until the queue is empty or f[q[back]] > f[i].

The final answer is ans.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def constrainedSubsetSum(self, nums: List[int], k: int) -> int:
        q = deque([0])
        n = len(nums)
        f = [0] * n
        ans = -inf
        for i, x in enumerate(nums):
            while i - q[0] > k:
                q.popleft()
            f[i] = max(0, f[q[0]]) + x
            ans = max(ans, f[i])
            while q and f[q[-1]] <= f[i]:
                q.pop()
            q.append(i)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subsequence that gives the maximum sum
# 2. The same answer with a max-heap instead of a deque


# ============================================================
# FOLLOW-UP 1: Return the subsequence that gives the
# maximum sum
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep, for every position, the position of the
# previous chosen element (the one with the best value in
# the window of the last k positions); follow these links
# back from the best end.
# Time: O(n).  Space: O(n).
from collections import deque


def constrained_subsequence_positions(nums, k):
    n = len(nums)
    best_sum = [0] * n
    previous = [-1] * n
    window = deque()                                # positions, best_sum decreasing
    for i in range(n):
        while window and window[0] < i - k:
            window.popleft()
        best_sum[i] = nums[i]
        if window and best_sum[window[0]] > 0:
            best_sum[i] += best_sum[window[0]]
            previous[i] = window[0]
        while window and best_sum[window[-1]] <= best_sum[i]:
            window.pop()
        window.append(i)
```

```
    end = 0
    for i in range(n):
        if best_sum[i] > best_sum[end]:
            end = i
    positions = []
    while end != -1:
        positions.append(end)
        end = previous[end]
    positions.reverse()
    return best_sum[positions[-1]], positions


# ============================================================
# FOLLOW-UP 2: The same answer with a max-heap instead of
# a deque
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep (negative best sum, position) in a heap and
# discard the top while it lies outside the window of the
# last k positions: O(n log n), but simpler to write.
# Time: O(n log n).  Space: O(n).
import heapq


def constrained_subsequence_sum_heap(nums, k):
```

```
    heap = []
    answer = None
    for i in range(len(nums)):
        while heap and heap[0][1] < i - k:
            heapq.heappop(heap)
        best = nums[i]
        if heap and -heap[0][0] > 0:
            best += -heap[0][0]
        heapq.heappush(heap, (-best, i))
        if answer is None or best > answer:
            answer = best
    return answer
```

PATTERN 10 OF 41

Queue / Deque

20 problems · 4 Easy · 12 Medium · 4 Hard

THE IDEA

First in, first out for processing in arrival order, and a double-ended queue for O(1) work at both ends.

Signals: simulate a line, recent-calls counter, rotting or spreading level by level, circular buffers, sliding window needing both ends.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
from collections import deque
q = deque([start])
while q:
    x = q.popleft()                  # oldest first
    for nxt in next_states(x):
        q.append(nxt)
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Implement Queue using Stacks	Easy	LC 232	478
2	Implement Stack using Queues	Easy	LC 225	481
3	Moving Average from Data Stream — substitute: Number of Recent Calls	Easy	LC 933	484
4	Time Needed to Buy Tickets	Easy	LC 2073	487
5	★ **Design Circular Queue**	Medium	LC 622	490
6	★ **Design Circular Deque**	Medium	LC 641	494
7	★ **Dota2 Senate**	Medium	LC 649	499
8	Reveal Cards In Increasing Order	Medium	LC 950	502
9	Design Front Middle Back Queue	Medium	LC 1670	505
10	Maximum Number of Robots Within Budget	Medium	LC 2398	509
11	First Unique Number	Medium	LC 1429	511
12	★ **Design Snake Game**	Medium	LC 353	515
13	Sum of Subarray Minimums (see page)	Medium	LC 907	440
14	★ **Jump Game VI**	Medium	LC 1696	519
15	Find the Winner of the Circular Game	Medium	LC 1823	522
16	Maximum Sum of Two Non-Overlapping Subarrays (see page)	Medium	LC 1031	271
17	★ **Sliding Window Maximum** (see page)	Hard	LC 239	124
18	★ **Shortest Subarray with Sum at Least K** (see page)	Hard	LC 862	472
19	Constrained Subsequence Sum (see page)	Hard	LC 1425	474
20	Stamping The Sequence	Hard	LC 936	525

PATTERN 10 · QUEUE / DEQUE · PROBLEM 1 OF 20

232. Implement Queue using Stacks

Easy · Stack, Design, Queue

leetcode.com/problems/implement-queue-using-stacks

PROBLEM

Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (`push`, `peek`, `pop`, and `empty`).

Implement the `MyQueue` class:

- `void push(int x)` Pushes element x to the back of the queue.
- `int pop()` Removes the element from the front of the queue and returns it.
- `int peek()` Returns the element at the front of the queue.
- `boolean empty()` Returns `true` if the queue is empty, `false` otherwise.

Notes:

- You must use **only** standard operations of a stack, which means only `push to top`, `peek/pop from top`, `size`, and `is empty` operations are valid.
- Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.

Example 1:

```
Input
["MyQueue", "push", "push", "peek", "pop", "empty"]
[[], [1], [2], [], [], []]
Output
[null, null, null, 1, 1, false]

Explanation
MyQueue myQueue = new MyQueue();
myQueue.push(1); // queue is: [1]
myQueue.push(2); // queue is: [1, 2] (leftmost is front of the queue)
myQueue.peek(); // return 1
myQueue.pop(); // return 1, queue is [2]
myQueue.empty(); // return false
```

Constraints:

- `1 <= x <= 9`
- At most `100` calls will be made to `push`, `pop`, `peek`, and `empty`.
- All the calls to `pop` and `peek` are valid.

Follow-up: Can you implement the queue such that each operation is **amortized** `O(1)` time complexity? In other words, performing `n` operations will take overall `O(n)` time even if one of those operations may take longer.

SOLUTION (PYTHON 3)

Solution 1: Double Stack

How to think about it

A stack is LIFO and a queue is FIFO, so one stack cannot serve both ends. Push into stk1 and pop from stk2. When stk2 is empty, pour stk1 into it so the oldest value sits on top. Each element moves at most once, and dequeue is amortized constant.

We use two stacks, where `stk1` is used for enqueue, and another stack `stk2` is used for dequeue.

When enqueueing, we directly push the element into `stk1`. The time complexity is O(1).

When dequeueing, we first check whether `stk2` is empty. If it is empty, we pop all elements from `stk1` and push them into `stk2`, and then pop an element from `stk2`. If `stk2` is not empty, we directly pop an element from `stk2`. The amortized time complexity is O(1).

When getting the front element, we first check whether `stk2` is empty. If it is empty, we pop all elements from `stk1` and push them into `stk2`, and then get the top element from `stk2`. If `stk2` is not empty, we directly get the top element from `stk2`. The amortized time complexity is O(1).

When checking whether the queue is empty, we only need to check whether both stacks are empty. The time complexity is O(1).

Python 3

```
class MyQueue:
    def __init__(self):
        self.stk1 = []
        self.stk2 = []

    def push(self, x: int) -> None:
        self.stk1.append(x)

    def pop(self) -> int:
        self.move()
        return self.stk2.pop()

    def peek(self) -> int:
        self.move()
        return self.stk2[-1]

    def empty(self) -> bool:
        return not self.stk1 and not self.stk2

    def move(self):
        if not self.stk2:
            while self.stk1:
                self.stk2.append(self.stk1.pop())


# Your MyQueue object will be instantiated and called as such:
# obj = MyQueue()
# obj.push(x)
# param_2 = obj.pop()
# param_3 = obj.peek()
# param_4 = obj.empty()
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. A queue that can also return its MINIMUM in O(1)
# 2. A queue built from ONE stack (using recursion)


# ============================================================
# FOLLOW-UP 1: A queue that can also return its MINIMUM in
# O(1)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Every stack entry stores the value together with
# the minimum of its stack up to that entry. The queue's
# minimum is the smaller of the minimums on top of the two
# stacks.
# Time: push O(1), pop and peek O(1) amortised, get_min
# O(1).  Space: O(n).
class MinQueue:
```

```
    def __init__(self):
        self.inbox = []                                 # (value, minimum of the inbox up to here)
        self.outbox = []

    def push(self, x):
        smallest = x
        if self.inbox and self.inbox[-1][1] < smallest:
            smallest = self.inbox[-1][1]
        self.inbox.append((x, smallest))

    def pop(self):
        self.move_if_needed()
        return self.outbox.pop()[0]

    def peek(self):
        self.move_if_needed()
        return self.outbox[-1][0]

    def move_if_needed(self):
        if not self.outbox:
            while self.inbox:
                x = self.inbox.pop()[0]
                smallest = x
                if self.outbox and self.outbox[-1][1] < smallest:
                    smallest = self.outbox[-1][1]
                self.outbox.append((x, smallest))

    def get_min(self):
        candidates = []
        if self.inbox:
            candidates.append(self.inbox[-1][1])
        if self.outbox:
            candidates.append(self.outbox[-1][1])
        smallest = candidates[0]
        for c in candidates:
            if c < smallest:
                smallest = c
        return smallest


# ========================================================
# FOLLOW-UP 2: A queue built from ONE stack (using
```

```
# recursion)
# ========================================================

# ---- follow-up 2: function ----
# Answer: Pop the whole stack by recursion down to the
# last element, return that element for pop, and push the
# others back while unwinding. Every pop costs O(n), but
# no second stack is needed.
# Time: push O(1), pop O(n).  Space: O(n) (the recursion
# holds the elements).
class QueueWithOneStack:
    def __init__(self):
        self.stack = []

    def push(self, x):
        self.stack.append(x)

    def pop(self):
        top = self.stack.pop()
        if not self.stack:
            return top
        front = self.pop()                              # the recursion reaches the oldest element
        self.stack.append(top)
        return front
```

PATTERN 10 · QUEUE / DEQUE · PROBLEM 2 OF 20

225. Implement Stack using Queues

Easy · Stack, Design, Queue

leetcode.com/problems/implement-stack-using-queues

PROBLEM

Implement a last-in-first-out (LIFO) stack using only two queues. The implemented stack should support all the functions of a normal stack (`push`, `top`, `pop`, and `empty`).

Implement the `MyStack` class:

- `void push(int x)` Pushes element x to the top of the stack.
- `int pop()` Removes the element on the top of the stack and returns it.
- `int top()` Returns the element on the top of the stack.
- `boolean empty()` Returns `true` if the stack is empty, `false` otherwise.

Notes:

- You must use **only** standard operations of a queue, which means that only `push to back`, `peek/pop from front`, `size` and `is empty` operations are valid.
- Depending on your language, the queue may not be supported natively. You may simulate a queue using a list or deque (double-ended queue) as long as you use only a queue's standard operations.

Example 1:

```
Input
["MyStack", "push", "push", "top", "pop", "empty"]
[[], [1], [2], [], [], []]
Output
[null, null, null, 2, 2, false]

Explanation
MyStack myStack = new MyStack();
myStack.push(1);
myStack.push(2);
myStack.top(); // return 2
myStack.pop(); // return 2
myStack.empty(); // return False
```

Constraints:

- `1 <= x <= 9`
- At most `100` calls will be made to `push`, `pop`, `top`, and `empty`.
- All the calls to `pop` and `top` are valid.

Follow-up: Can you implement the stack using only one queue?

SOLUTION (PYTHON 3)

Solution 1: Two Queues

How to think about it

A queue inserts at the back and removes from the front, while a stack is LIFO. If each push rotates the new value to the front, pop and top become a dequeue. We enqueue x into q_2, move everything from q_1 behind it, and swap, so the front of q_1 is always the stack top.

We use two queues q_1 and q_2, where q_1 is used to store the elements in the stack, and q_2 is used to assist in implementing the stack operations.

- `push` operation: Push the element into q_2, then pop the elements in q_1 one by one and push them into q_2, finally swap the references of q_1 and q_2. The time complexity is O(n).
- `pop` operation: Directly pop the front element of q_1. The time complexity is O(1).
- `top` operation: Directly return the front element of q_1. The time complexity is O(1).
- `empty` operation: Check whether q_1 is empty. The time complexity is O(1).

The space complexity is O(n), where n is the number of elements in the stack.

Python 3

```
class MyStack:
    def __init__(self):
        self.q1 = deque()
        self.q2 = deque()

    def push(self, x: int) -> None:
        self.q2.append(x)
        while self.q1:
            self.q2.append(self.q1.popleft())
        self.q1, self.q2 = self.q2, self.q1

    def pop(self) -> int:
        return self.q1.popleft()

    def top(self) -> int:
        return self.q1[0]

    def empty(self) -> bool:
        return len(self.q1) == 0


# Your MyStack object will be instantiated and called as such:
# obj = MyStack()
# obj.push(x)
# param_2 = obj.pop()
# param_3 = obj.top()
# param_4 = obj.empty()
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Use only ONE queue
# 2. Make push O(1) and pop O(n) instead


# ============================================================
# FOLLOW-UP 1: Use only ONE queue
# ============================================================

# ---- follow-up 1: function ----
# Answer: After pushing x to the back, rotate the queue:
# move all the older elements behind it, so the newest
# element is at the front.
# Time: push O(n), pop, top and empty O(1).  Space: O(n).
from collections import deque


class StackWithOneQueue:
    def __init__(self):
        self.queue = deque()

    def push(self, x):
        self.queue.append(x)
        for i in range(len(self.queue) - 1):
            self.queue.append(self.queue.popleft())
```

```
    def pop(self):
        return self.queue.popleft()

    def top(self):
        return self.queue[0]

    def empty(self):
        return len(self.queue) == 0


# ============================================================
# FOLLOW-UP 2: Make push O(1) and pop O(n) instead
# ============================================================

# ---- follow-up 2: function ----
# Answer: push just appends. pop moves all but the last
# element to the back, then removes that last element.
# Time: push O(1), pop O(n), top O(n).  Space: O(n).
from collections import deque


class StackFastPush:
    def __init__(self):
        self.queue = deque()

    def push(self, x):
        self.queue.append(x)

    def pop(self):
        for i in range(len(self.queue) - 1):
            self.queue.append(self.queue.popleft())
        return self.queue.popleft()

    def top(self):
```

```
        value = self.pop()
        self.queue.append(value)
        return value
```

PATTERN 10 · QUEUE / DEQUE · PROBLEM 3 OF 20

933. Number of Recent Calls

Easy · Design, Queue, Data Stream

leetcode.com/problems/number-of-recent-calls

PROBLEM

You have a `RecentCounter` class which counts the number of recent requests within a certain time frame.

Implement the `RecentCounter` class:

- `RecentCounter()` Initializes the counter with zero recent requests.
- `int ping(int t)` Adds a new request at time `t`, where `t` represents some time in milliseconds, and returns the number of requests that have happened in the inclusive range `[t - 3000, t]`, that is, the new request plus every earlier request that is no more than `3000` milliseconds older.

It is **guaranteed** that every call to `ping` uses a strictly larger value of `t` than the previous call.

Example 1:

```
Input
["RecentCounter", "ping", "ping", "ping", "ping"]
[[], [1], [100], [3001], [3002]]
Output
[null, 1, 2, 3, 3]

Explanation
RecentCounter recentCounter = new RecentCounter();
recentCounter.ping(1); // requests = [1], range is [-2999,1], return 1
recentCounter.ping(100); // requests = [1, 100], range is [-2900,100], return 2
recentCounter.ping(3001); // requests = [1, 100, 3001], range is [1,3001], return 3
recentCounter.ping(3002); // requests = [1, 100, 3001, 3002], range is [2,3002], return 3
```

Constraints:

- $1 <= t <= 10^9$
- Each test case will call `ping` with **strictly increasing** values of `t`.
- At most 10^4 calls will be made to `ping`.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

t is strictly increasing, so a request older than t-3000 never returns. Store times in a queue and pop from the front while expired; the queue size is the answer.

Python 3

```python
class RecentCounter:
    def __init__(self):
        self.q = deque()

    def ping(self, t: int) -> int:
        self.q.append(t)
        while self.q[0] < t - 3000:
            self.q.popleft()
        return len(self.q)


# Your RecentCounter object will be instantiated and called as such:
# obj = RecentCounter()
# param_1 = obj.ping(t)
```

Complexity. Time: O(1) amortised per ping. Space: O(W), W = pings inside the 3000 ms window.

Solution 2

> **How to think about it**
>
> The queue is amortized linear. Because times are sorted, we may keep them in an array and binary-search the first index ≥ t-3000; the suffix length is the count. Extra space stays linear, and each query pays a logarithm.

Python 3

```python
class RecentCounter:
    def __init__(self):
        self.s = []

    def ping(self, t: int) -> int:
        self.s.append(t)
        return len(self.s) - bisect_left(self.s, t - 3000)


# Your RecentCounter object will be instantiated and called as such:
# obj = RecentCounter()
# param_1 = obj.ping(t)
```

Complexity. Time: O(log n) per ping. Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Calls can arrive out of order (timestamps are not
# increasing)
# 2. Count the recent calls of every user separately


# ============================================================
# FOLLOW-UP 1: Calls can arrive out of order (timestamps
# are not increasing)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the timestamps in a sorted list (insert at
# the right place with a binary search) and count the ones
# inside [t - 3000, t] with two binary searches.
# Time: O(n) per ping (inserting into a sorted list).
# Space: O(n).
import bisect


class RecentCounterUnordered:
```

```
    def __init__(self):
        self.times = []

    def ping(self, t):
        bisect.insort(self.times, t)
        low = bisect.bisect_left(self.times, t - 3000)
        high = bisect.bisect_right(self.times, t)
        return high - low


# ============================================================
# FOLLOW-UP 2: Count the recent calls of every user
# separately
# ============================================================

# ---- follow-up 2: function ----
# Answer: One queue per user, kept in a dictionary; each
# queue drops its old timestamps when that user pings.
# Time: O(1) amortised per ping.  Space: O(total pings in
# the window of every user).
from collections import deque


class RecentCounterPerUser:
    def __init__(self):
        self.queues = {}

    def ping(self, user, t):
        if user not in self.queues:
            self.queues[user] = deque()
        queue = self.queues[user]
        queue.append(t)
        while queue[0] < t - 3000:
            queue.popleft()
        return len(queue)
```

2073. Time Needed to Buy Tickets

Easy · Queue, Array, Simulation

leetcode.com/problems/time-needed-to-buy-tickets

PROBLEM

There are `n` people in a line queuing to buy tickets, where the `0`th person is at the **front** of the line and the `(n - 1)`th person is at the **back** of the line.

You are given a **0-indexed** integer array `tickets` of length `n` where the number of tickets that the `i`th person would like to buy is `tickets[i]`.

Each person takes **exactly 1 second** to buy a ticket. A person can only buy **1 ticket at a time** and has to go back to **the end** of the line (which happens **instantaneously**) in order to buy more tickets. If a person does not have any tickets left to buy, the person will **leave** the line.

Return the **time taken** for the person **initially** at position **k** (0-indexed) to finish buying tickets.

Example 1:

Input: tickets = [2,3,2], k = 2

Output: 6

Explanation:

- The queue starts as [2,3,2], where the kth person is underlined.
- After the person at the front has bought a ticket, the queue becomes [3,2,1] at 1 second.
- Continuing this process, the queue becomes [2,1,2] at 2 seconds.
- Continuing this process, the queue becomes [1,2,1] at 3 seconds.
- Continuing this process, the queue becomes [2,1] at 4 seconds. Note: the person at the front left the queue.
- Continuing this process, the queue becomes [1,1] at 5 seconds.
- Continuing this process, the queue becomes [1] at 6 seconds. The kth person has bought all their tickets, so return 6.

Example 2:

Input: tickets = [5,1,1,1], k = 0

Output: 8

Explanation:

- The queue starts as [5,1,1,1], where the kth person is underlined.
- After the person at the front has bought a ticket, the queue becomes [1,1,1,4] at 1 second.
- Continuing this process for 3 seconds, the queue becomes [4] at 4 seconds.
- Continuing this process for 4 seconds, the queue becomes [] at 8 seconds. The kth person has bought all their tickets, so return 8.

Constraints:

- `n == tickets.length`
- `1 <= n <= 100`
- `1 <= tickets[i] <= 100`
- `0 <= k < n`

SOLUTION (PYTHON 3)

Solution 1: Single Pass

> **How to think about it**
>
> Naively simulating the queue costs O(∑ tickets). When person k finishes, everyone ahead bought at most tickets[k] tickets and everyone behind at most tickets[k]-1. Sum min of that cap and each person's demand in one pass.

According to the problem description, when the k^{th} person finishes buying tickets, all the people in front of the k^{th} person will not buy more tickets than the k^{th} person, and all the people behind the k^{th} person will not buy more tickets than the k^{th} person minus 1.

Therefore, we can traverse the entire queue. For the i^{th} person, if $i \le k$, the time to buy tickets is min(tickets[i], tickets[k]); otherwise, the time to buy tickets is min(tickets[i], tickets[k] - 1). We sum the buying time for all people to get the result.

The time complexity is O(n), where n is the length of the queue. The space complexity is O(1).

Python 3

```python
class Solution:
    def timeRequiredToBuy(self, tickets: List[int], k: int) -> int:
        ans = 0
        for i, x in enumerate(tickets):
            ans += min(x, tickets[k] if i <= k else tickets[k] - 1)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The time at which EVERY person finishes
# 2. A person buys up to c tickets per turn (each ticket
# takes one second)


# ============================================================
# FOLLOW-UP 1: The time at which EVERY person finishes
# ============================================================

# ---- follow-up 1: function ----
# Answer: Person i finishes after everyone has bought
# min(their tickets, person i's tickets) tickets, except
# that people standing behind person i have not yet had
# their turn in the last round, so they buy one fewer.
# Time: O(n^2).  Space: O(n).
def finish_times(tickets):
    times = []
    for i in range(len(tickets)):
        total = 0
        for j in range(len(tickets)):
            if j <= i:
                total += tickets[j] if tickets[j] < tickets[i] else tickets[i]
            else:
                limit = tickets[i] - 1
                total += tickets[j] if tickets[j] < limit else limit
        times.append(total)
    return times


# ============================================================
# FOLLOW-UP 2: A person buys up to c tickets per turn
# (each ticket takes one second)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Simulate the queue: the person at the front buys
```

```
# min(c, tickets left), the time goes up by that many
# seconds, and the person returns to the back if tickets
# remain.
# Time: O(n + S / c), S = total tickets.  Space: O(n).
from collections import deque


def time_with_batch(tickets, k, per_turn):
    queue = deque()
    for i in range(len(tickets)):
        queue.append((i, tickets[i]))
    time = 0
    while queue:
        person, left = queue.popleft()
        bought = per_turn if left > per_turn else left
        time += bought
        left -= bought
        if person == k and left == 0:
            return time
        if left > 0:
            queue.append((person, left))
    return time
```

PATTERN 10 · QUEUE / DEQUE · PROBLEM 5 OF 20

622. Design Circular Queue

Medium · Design, Queue, Array, Linked List

leetcode.com/problems/design-circular-queue

★ **TOP 7 PROBLEM** in Queue / Deque: master this one first

PROBLEM

Design your implementation of the circular queue. The circular queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle, and the last position is connected back to the first position to make a circle. It is also called "Ring Buffer".

One of the benefits of the circular queue is that we can make use of the spaces in front of the queue. In a normal queue, once the queue becomes full, we cannot insert the next element even if there is a space in front of the queue. But using the circular queue, we can use the space to store new values.

Implement the `MyCircularQueue` class:

- `MyCircularQueue(k)` Initializes the object with the size of the queue to be `k`.
- `int Front()` Gets the front item from the queue. If the queue is empty, return `-1`.
- `int Rear()` Gets the last item from the queue. If the queue is empty, return `-1`.
- `boolean enQueue(int value)` Inserts an element into the circular queue. Return `true` if the operation is successful.
- `boolean deQueue()` Deletes an element from the circular queue. Return `true` if the operation is successful.
- `boolean isEmpty()` Checks whether the circular queue is empty or not.
- `boolean isFull()` Checks whether the circular queue is full or not.

You must solve the problem without using the built-in queue data structure in your programming language.

Example 1:

```
Input
["MyCircularQueue", "enQueue", "enQueue", "enQueue", "enQueue", "Rear", "isFull", "deQueue",
"enQueue", "Rear"]
[[3], [1], [2], [3], [4], [], [], [], [4], []]
Output
[null, true, true, true, false, 3, true, true, true, 4]

Explanation
MyCircularQueue myCircularQueue = new MyCircularQueue(3);
myCircularQueue.enQueue(1); // return True
myCircularQueue.enQueue(2); // return True
myCircularQueue.enQueue(3); // return True
myCircularQueue.enQueue(4); // return False
myCircularQueue.Rear(); // return 3
myCircularQueue.isFull(); // return True
myCircularQueue.deQueue(); // return True
myCircularQueue.enQueue(4); // return True
myCircularQueue.Rear(); // return 4
```

Constraints:

- `1 <= k <= 1000`
- `0 <= value <= 1000`
- At most `3000` calls will be made to `enQueue`, `deQueue`, `Front`, `Rear`, `isEmpty`, and `isFull`.

SOLUTION (PYTHON 3)

Solution 1: Array Simulation

How to think about it

A circular queue needs O(1) enqueue/dequeue on a fixed array and must tell empty from full. Head and tail alone collide when they wrap. Store size as well: write at (front+size) mod k, and read the rear at (front+size-1) mod k. Empty and full are just size checks.

We can use an array q of length k to simulate a circular queue, with a pointer front to record the position of the front element. Initially, the queue is empty, and front is 0. Additionally, we use a variable size to record the number of elements in the queue, initially size is 0.

When calling the `enQueue` method, we first check if the queue is full, i.e., size = k. If it is full, we return false. Otherwise, we insert the element at position (front + size) mod k, then size = size + 1, indicating that the number of elements in the queue has increased by 1. Finally, we return true.

When calling the `deQueue` method, we first check if the queue is empty, i.e., size = 0. If it is empty, we return false. Otherwise, we set front = (front + 1) mod k, indicating that the front element has been dequeued, then size = size - 1.

When calling the `Front` method, we first check if the queue is empty, i.e., size = 0. If it is empty, we return -1. Otherwise, we return q[front].

When calling the `Rear` method, we first check if the queue is empty, i.e., size = 0. If it is empty, we return -1. Otherwise, we return q[(front + size - 1) mod k].

When calling the `isEmpty` method, we simply check if size = 0.

When calling the `isFull` method, we simply check if size = k.

In terms of time complexity, the above operations all have a time complexity of O(1). The space complexity is O(k).

Python 3

```python
class MyCircularQueue:

    def __init__(self, k: int):
        self.q = [0] * k
        self.size = 0
        self.capacity = k
        self.front = 0

    def enQueue(self, value: int) -> bool:
        if self.isFull():
            return False
        self.q[(self.front + self.size) % self.capacity] = value
        self.size += 1
        return True

    def deQueue(self) -> bool:
        if self.isEmpty():
            return False
        self.front = (self.front + 1) % self.capacity
        self.size -= 1
        return True

    def Front(self) -> int:
        return -1 if self.isEmpty() else self.q[self.front]

    def Rear(self) -> int:
        if self.isEmpty():
            return -1
        return self.q[(self.front + self.size - 1) % self.capacity]

    def isEmpty(self) -> bool:
        return self.size == 0

    def isFull(self) -> bool:
        return self.size == self.capacity


# Your MyCircularQueue object will be instantiated and called as such:
# obj = MyCircularQueue(k)
# param_1 = obj.enQueue(value)
# param_2 = obj.deQueue()
# param_3 = obj.Front()
# param_4 = obj.Rear()
# param_5 = obj.isEmpty()
# param_6 = obj.isFull()
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. A ring buffer that overwrites the OLDEST element when
# it is full
# 2. A queue that grows when it is full


# ============================================================
# FOLLOW-UP 1: A ring buffer that overwrites the OLDEST
# element when it is full
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same array and two indexes; when a push finds
# the buffer full, move the head forward one step first
# (the oldest element is dropped).
# Time: push O(1), contents O(n).  Space: O(capacity).
class RingBuffer:
```

```
    def __init__(self, capacity):
        self.items = [0] * capacity
        self.head = 0
        self.size = 0

    def push(self, value):
        capacity = len(self.items)
        if self.size == capacity:
            self.head = (self.head + 1) % capacity          # drop the oldest
            self.size -= 1
        self.items[(self.head + self.size) % capacity] = value
        self.size += 1

    def contents(self):
        result = []
        for i in range(self.size):
            result.append(self.items[(self.head + i) % len(self.items)])
        return result


# ============================================================
# FOLLOW-UP 2: A queue that grows when it is full
# ============================================================

# ---- follow-up 2: function ----
# Answer: When the array is full, copy the elements in
# order into an array twice as big (so the head is back at
# index 0) and continue. Pushes stay O(1) on average.
# Time: push O(1) amortised, pop O(1).  Space: O(n).
class ResizableCircularQueue:
    def __init__(self):
        self.items = [0] * 2
        self.head = 0
        self.size = 0

    def push(self, value):
        if self.size == len(self.items):
            bigger = [0] * (2 * len(self.items))
            for i in range(self.size):
                bigger[i] = self.items[(self.head + i) % len(self.items)]
            self.items = bigger
            self.head = 0
        self.items[(self.head + self.size) % len(self.items)] = value
```

```
        self.size += 1

    def pop(self):
        value = self.items[self.head]
        self.head = (self.head + 1) % len(self.items)
        self.size -= 1
        return value
```

PATTERN 10 · QUEUE / DEQUE · PROBLEM 6 OF 20

641. Design Circular Deque

Medium · Design, Queue, Array, Linked List

leetcode.com/problems/design-circular-deque

★ **TOP 7 PROBLEM** in Queue / Deque: master this one first

Also appears in pattern 29 Design Problems

PROBLEM

Design your implementation of the circular double-ended queue (deque).

Implement the `MyCircularDeque` class:

- `MyCircularDeque(int k)` Initializes the deque with a maximum size of `k`.
- `boolean insertFront()` Adds an item at the front of Deque. Returns `true` if the operation is successful, or `false` otherwise.
- `boolean insertLast()` Adds an item at the rear of Deque. Returns `true` if the operation is successful, or `false` otherwise.
- `boolean deleteFront()` Deletes an item from the front of Deque. Returns `true` if the operation is successful, or `false` otherwise.
- `boolean deleteLast()` Deletes an item from the rear of Deque. Returns `true` if the operation is successful, or `false` otherwise.
- `int getFront()` Returns the front item from the Deque. Returns `-1` if the deque is empty.
- `int getRear()` Returns the last item from Deque. Returns `-1` if the deque is empty.
- `boolean isEmpty()` Returns `true` if the deque is empty, or `false` otherwise.
- `boolean isFull()` Returns `true` if the deque is full, or `false` otherwise.

Example 1:

```
Input
["MyCircularDeque", "insertLast", "insertLast", "insertFront", "insertFront", "getRear", "isFull",
"deleteLast", "insertFront", "getFront"]
[[3], [1], [2], [3], [4], [], [], [], [4], []]
Output
[null, true, true, true, false, 2, true, true, true, 4]

Explanation
MyCircularDeque myCircularDeque = new MyCircularDeque(3);
myCircularDeque.insertLast(1); // return True
myCircularDeque.insertLast(2); // return True
myCircularDeque.insertFront(3); // return True
myCircularDeque.insertFront(4); // return False, the queue is full.
myCircularDeque.getRear(); // return 2
myCircularDeque.isFull(); // return True
myCircularDeque.deleteLast(); // return True
myCircularDeque.insertFront(4); // return True
myCircularDeque.getFront(); // return 4
```

Constraints:

- `1 <= k <= 1000`
- `0 <= value <= 1000`
- At most `2000` calls will be made to `insertFront`, `insertLast`, `deleteFront`, `deleteLast`, `getFront`, `getRear`, `isEmpty`, `isFull`.

SOLUTION (PYTHON 3)

Solution 1: Array

How to think about it

A bounded deque needs O(1) inserts and deletes at both ends and a clear empty/full test. A linked list wastes pointers. Reuse a ring buffer with front and size: insert-front steps front backward, insert-last writes at (front+size) mod k.

We can use an array to implement the circular deque. We maintain a pointer front pointing to the front of the queue, a variable size representing the number of elements in the queue, and a variable capacity representing the queue's capacity. We use an array q to store the elements.

When insertFront is called, we first check if the queue is full; if so, return false. Otherwise, we move front one position forward (using modular arithmetic for circular wrapping), insert the new element at front, and increment size by 1.

When insertLast is called, we first check if the queue is full; if so, return false. Otherwise, we compute the insertion position (using front and size), insert the new element there, and increment size by 1.

When deleteFront is called, we first check if the queue is empty; if so, return false. Otherwise, we move front one position backward (using modular arithmetic for circular wrapping) and decrement size by 1.

When deleteLast is called, we first check if the queue is empty; if so, return false. Otherwise, we decrement size by 1.

When getFront is called, we first check if the queue is empty; if so, return -1. Otherwise, we return q[front].

When getRear is called, we first check if the queue is empty; if so, return -1. Otherwise, we compute the position of the rear element (using front and size) and return the element at that position.

When isEmpty is called, we check whether size equals 0.

When isFull is called, we check whether size equals capacity.

All operations above have a time complexity of O(1) and a space complexity of O(k), where k is the capacity of the deque.

Python 3

```
class MyCircularDeque:
    def __init__(self, k: int):
        """
        Initialize your data structure here. Set the size of the deque to be k.
        """
        self.q = [0] * k
        self.front = 0
        self.size = 0
        self.capacity = k

    def insertFront(self, value: int) -> bool:
        """
        Adds an item at the front of Deque. Return true if the operation is successful.
        """
        if self.isFull():
            return False
        if not self.isEmpty():
            self.front = (self.front - 1 + self.capacity) % self.capacity
        self.q[self.front] = value
        self.size += 1
        return True

    def insertLast(self, value: int) -> bool:
        """
        Adds an item at the rear of Deque. Return true if the operation is successful.
        """
        if self.isFull():
            return False
        idx = (self.front + self.size) % self.capacity
        self.q[idx] = value
        self.size += 1
        return True

    def deleteFront(self) -> bool:
        """
        Deletes an item from the front of Deque. Return true if the operation is successful.
        """
        if self.isEmpty():
            return False
        self.front = (self.front + 1) % self.capacity
        self.size -= 1
        return True

    def deleteLast(self) -> bool:
        """
        Deletes an item from the rear of Deque. Return true if the operation is successful.
        """
        if self.isEmpty():
            return False
        self.size -= 1
        return True

    def getFront(self) -> int:
        """
        Get the front item from the deque.
        """
        if self.isEmpty():
            return -1
        return self.q[self.front]
```

```
    def getRear(self) -> int:
        """
        Get the last item from the deque.
        """
        if self.isEmpty():
            return -1
        idx = (self.front + self.size - 1) % self.capacity
```

```
        return self.q[idx]

    def isEmpty(self) -> bool:
        """
        Checks whether the circular deque is empty or not.
        """
        return self.size == 0

    def isFull(self) -> bool:
        """
        Checks whether the circular deque is full or not.
        """
        return self.size == self.capacity


# Your MyCircularDeque object will be instantiated and called as such:
# obj = MyCircularDeque(k)
# param_1 = obj.insertFront(value)
# param_2 = obj.insertLast(value)
# param_3 = obj.deleteFront()
# param_4 = obj.deleteLast()
# param_5 = obj.getFront()
# param_6 = obj.getRear()
# param_7 = obj.isEmpty()
# param_8 = obj.isFull()
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Read the i-th element from the front, and rotate the
# deque by k places


# ========================================================
# FOLLOW-UP 1: Read the i-th element from the front, and
# rotate the deque by k places
# ========================================================

# ---- follow-up 1: function ----
# Answer: The i-th element sits at (head + i) mod
# capacity. A rotation by k moves k elements from one end
# to the other.
# Time: O(1) per operation, rotate_left O(k).  Space:
# O(capacity).
class IndexedCircularDeque:
    def __init__(self, capacity):
        self.items = [0] * capacity
        self.head = 0
        self.size = 0

    def push_back(self, value):
        self.items[(self.head + self.size) % len(self.items)] = value
        self.size += 1

    def push_front(self, value):
        self.head = (self.head - 1) % len(self.items)
        self.items[self.head] = value
        self.size += 1

    def pop_back(self):
        value = self.items[(self.head + self.size - 1) % len(self.items)]
        self.size -= 1
        return value

    def pop_front(self):
        value = self.items[self.head]
```

```
        self.head = (self.head + 1) % len(self.items)
        self.size -= 1
        return value

    def get(self, i):
        return self.items[(self.head + i) % len(self.items)]

    def rotate_left(self, k):
        for step in range(k % self.size if self.size else 0):
            self.push_back(self.pop_front())
```

PATTERN 10 · QUEUE / DEQUE · PROBLEM 7 OF 20

649. Dota2 Senate

Medium · Greedy, Queue, String

leetcode.com/problems/dota2-senate

★ **TOP 7 PROBLEM** in Queue / Deque: master this one first

PROBLEM

In the world of Dota2, there are two parties: the Radiant and the Dire.

The Dota2 senate consists of senators coming from two parties. Now the Senate wants to decide on a change in the Dota2 game. The voting for this change is a round-based procedure. In each round, each senator can exercise **one** of the two rights:

- **Ban one senator's right:** A senator can make another senator lose all his rights in this and all the following rounds.
- **Announce the victory:** If this senator found the senators who still have rights to vote are all from the same party, he can announce the victory and decide on the change in the game.

Given a string `senate` representing each senator's party belonging. The character `'R'` and `'D'` represent the Radiant party and the Dire party. Then if there are `n` senators, the size of the given string will be `n`.

The round-based procedure starts from the first senator to the last senator in the given order. This procedure will last until the end of voting. All the senators who have lost their rights will be skipped during the procedure.

Suppose every senator is smart enough and will play the best strategy for his own party. Predict which party will finally announce the victory and change the Dota2 game. The output should be `"Radiant"` or `"Dire"`.

Example 1:

```
Input: senate = "RD"
Output: "Radiant"
Explanation:
The first senator comes from Radiant and he can just ban the next senator's right in round 1.
And the second senator can't exercise any rights anymore since his right has been banned.
And in round 2, the first senator can just announce the victory since he is the only guy in the senate
who can vote.
```

Example 2:

```
Input: senate = "RDD"
Output: "Dire"
Explanation:
The first senator comes from Radiant and he can just ban the next senator's right in round 1.
And the second senator can't exercise any rights anymore since his right has been banned.
And the third senator comes from Dire and he can ban the first senator's right in round 1.
And in round 2, the third senator can just announce the victory since he is the only guy in the senate
who can vote.
```

Constraints:

- `n == senate.length`
- `1 <= n <= 10^4`
- `senate[i]` is either `'R'` or `'D'`.

SOLUTION (PYTHON 3)

Solution 1: Queue + Simulation

> **How to think about it**
>
> Senators ban the next opponent in order until one party is gone. Repeatedly deleting from the string is quadratic. Queue indices of each party. The smaller front acts first, pops the other, and re-enters at i+n for the next round. An empty queue loses.

We create two queues qr and qd to record the indices of the Radiant and Dire senators, respectively. Then we start the simulation, where in each round we dequeue one senator from each queue and perform different operations based on their factions:

- If the Radiant senator's index is less than the Dire senator's index, the Radiant senator can permanently ban the voting rights of the Dire senator. We add n to the Radiant senator's index and enqueue it back to the end of the queue, indicating that this senator will participate in the next round of voting.
- If the Dire senator's index is less than the Radiant senator's index, the Dire senator can permanently ban the voting rights of the Radiant senator. We add n to the Dire senator's index and enqueue it back to the end of the queue, indicating that this senator will participate in the next round of voting.

Finally, when there are only senators from one faction left in the queues, the senators from that faction win.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of senators.

Python 3

```python
class Solution:
    def predictPartyVictory(self, senate: str) -> str:
        qr = deque()
        qd = deque()
        for i, c in enumerate(senate):
            if c == "R":
                qr.append(i)
            else:
                qd.append(i)
        n = len(senate)
        while qr and qd:
            if qr[0] < qd[0]:
                qr.append(qr[0] + n)
            else:
                qd.append(qd[0] + n)
            qr.popleft()
            qd.popleft()
        return "Radiant" if qr else "Dire"
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. How many senators of the winning party are left when
# the vote ends?


# ============================================================
# FOLLOW-UP 1: How many senators of the winning party are
# left when the vote ends?
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use the same two queues of positions. Each time
# the front senators of both parties meet, the earlier one
# bans the other and returns to the back with its position
# plus n (it votes again in the next round). When one
# queue is empty, the size of the other is the answer.
```

```python
# Time: O(n).  Space: O(n).
from collections import deque


def winner_and_survivors(senate):
    radiant = deque()
    dire = deque()
    for i in range(len(senate)):
        if senate[i] == "R":
            radiant.append(i)
        else:
            dire.append(i)
    n = len(senate)
    while radiant and dire:
        r = radiant.popleft()
        d = dire.popleft()
        if r < d:
            radiant.append(r + n)
        else:
            dire.append(d + n)
    if radiant:
        return "Radiant", len(radiant)
    return "Dire", len(dire)
```

950. Reveal Cards In Increasing Order

Medium · Queue, Array, Sorting, Simulation

leetcode.com/problems/reveal-cards-in-increasing-order

PROBLEM

You are given an integer array `deck`. There is a deck of cards where every card has a unique integer. The integer on the i^{th} card is `deck[i]`.

You can order the deck in any order you want. Initially, all the cards start face down (unrevealed) in one deck.

You will do the following steps repeatedly until all cards are revealed:

1. Take the top card of the deck, reveal it, and take it out of the deck.
2. If there are still cards in the deck then put the next top card of the deck at the bottom of the deck.
3. If there are still unrevealed cards, go back to step 1. Otherwise, stop.

Return *an ordering of the deck that would reveal the cards in increasing order*.

Note that the first entry in the answer is considered to be the top of the deck.

Example 1:

```
Input: deck = [17,13,11,2,3,5,7]
Output: [2,13,3,11,5,17,7]
Explanation:
We get the deck in the order [17,13,11,2,3,5,7] (this order does not matter), and reorder it.
After reordering, the deck starts as [2,13,3,11,5,17,7], where 2 is the top of the deck.
We reveal 2, and move 13 to the bottom. The deck is now [3,11,5,17,7,13].
We reveal 3, and move 11 to the bottom. The deck is now [5,17,7,13,11].
We reveal 5, and move 17 to the bottom. The deck is now [7,13,11,17].
We reveal 7, and move 13 to the bottom. The deck is now [11,17,13].
We reveal 11, and move 17 to the bottom. The deck is now [13,17].
We reveal 13, and move 17 to the bottom. The deck is now [17].
We reveal 17.
Since all the cards revealed are in increasing order, the answer is correct.
```

Example 2:

```
Input: deck = [1,1000]
Output: [1,1000]
```

Constraints:

- `1 <= deck.length <= 1000`
- `1 <= deck[i] <= 10^6`
- All the values of `deck` are **unique**.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Revealing takes the front card and moves the new front to the back. Simulating forward needs the unknown start order, while the reveal should be increasing. Invert the process: from large to small, rotate the current back to the front (if the queue is nonempty) and insert the card at the front. The queue is the initial deck.

Python 3

```
class Solution:
    def deckRevealedIncreasing(self, deck: List[int]) -> List[int]:
        q = deque()
        for v in sorted(deck, reverse=True):
            if q:
                q.appendleft(q.pop())
            q.appendleft(v)
        return list(q)
```

Complexity. Time: O(n log n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Reveal one card, then move m cards to the bottom
# (instead of 1)
# 2. Does a given deck order reveal the cards in
# increasing order?


# ============================================================
# FOLLOW-UP 1: Reveal one card, then move m cards to the
# bottom (instead of 1)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Build the order backwards, or simulate the
# positions: play the process on the indexes 0..n-1 and
# give the sorted cards to the revealed positions in
# order.
# Time: O(n log n + n * m).  Space: O(n).
from collections import deque


def deck_order_for_pattern(deck, move):
    positions = deque(range(len(deck)))
    revealed_positions = []
    while positions:
        revealed_positions.append(positions.popleft())
        for i in range(move):
            if positions:
                positions.append(positions.popleft())
    answer = [0] * len(deck)
    for card, position in zip(sorted(deck), revealed_positions):
        answer[position] = card
    return answer


# ============================================================
# FOLLOW-UP 2: Does a given deck order reveal the cards in
# increasing order?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Simulate the process on the deck and check that
# the revealed cards come out sorted.
# Time: O(n).  Space: O(n).
from collections import deque


def reveals_increasing(deck):
    queue = deque(deck)
    revealed = []
    while queue:
        revealed.append(queue.popleft())
        if queue:
```

```
            queue.append(queue.popleft())
    for i in range(1, len(revealed)):
        if revealed[i] < revealed[i - 1]:
            return False
    return True
```

1670. Design Front Middle Back Queue

Medium · Design, Queue, Array, Linked List, Data Stream, Doubly-Linked List

leetcode.com/problems/design-front-middle-back-queue

PROBLEM

Design a queue that supports `push` and `pop` operations in the front, middle, and back.

Implement the `FrontMiddleBack` class:

- `FrontMiddleBack()` Initializes the queue.
- `void pushFront(int val)` Adds `val` to the **front** of the queue.
- `void pushMiddle(int val)` Adds `val` to the **middle** of the queue.
- `void pushBack(int val)` Adds `val` to the **back** of the queue.
- `int popFront()` Removes the **front** element of the queue and returns it. If the queue is empty, return `-1`.
- `int popMiddle()` Removes the **middle** element of the queue and returns it. If the queue is empty, return `-1`.
- `int popBack()` Removes the **back** element of the queue and returns it. If the queue is empty, return `-1`.

Notice that when there are **two** middle position choices, the operation is performed on the **frontmost** middle position choice. For example:

- Pushing `6` into the middle of `[1, 2, 3, 4, 5]` results in `[1, 2, 6, 3, 4, 5]`.
- Popping the middle from `[1, 2, 3, 4, 5, 6]` returns `3` and results in `[1, 2, 4, 5, 6]`.

Example 1:

```
Input:
["FrontMiddleBackQueue", "pushFront", "pushBack", "pushMiddle", "pushMiddle", "popFront", "popMiddle",
"popMiddle", "popBack", "popFront"]
[[], [1], [2], [3], [4], [], [], [], [], []]
Output:
[null, null, null, null, null, 1, 3, 4, 2, -1]

Explanation:
FrontMiddleBackQueue q = new FrontMiddleBackQueue();
q.pushFront(1); // [1]
q.pushBack(2); // [1, 2]
q.pushMiddle(3); // [1, 3, 2]
q.pushMiddle(4); // [1, 4, 3, 2]
q.popFront(); // return 1 -> [4, 3, 2]
q.popMiddle(); // return 3 -> [4, 2]
q.popMiddle(); // return 4 -> [2]
q.popBack(); // return 2 -> []
q.popFront(); // return -1 -> [] (The queue is empty)
```

Constraints:

- $1 <= val <= 10^9$
- At most `1000` calls will be made to `pushFront`, `pushMiddle`, `pushBack`, `popFront`, `popMiddle`, and `popBack`.

SOLUTION (PYTHON 3)

Solution 1: Two Deques

How to think about it

We need O(1) insert/delete at the front, middle, and back; finding the midpoint of one list is linear. Split into deques q_1 (front half) and q_2 (back half) with q_2 at most one longer than q_1. After an update, rebalance moves one element. The middle is the tail of q_1 when sizes are equal, otherwise the head of q_2.

We use two deques, where q_1 stores the first half, and q_2 stores the second half. The `rebalance` function is used to maintain the balance between the two queues, i.e., keeping the length of q_2 greater than or equal to the length of q_1, and the difference in length does not exceed 1.

In the `pushFront`, `pushMiddle`, and `pushBack` functions, we only need to add elements to q_1 or q_2, and call the `rebalance` function.

For the `popFront` function, we need to check whether q_1 and q_2 are empty. If both are empty, return -1. Otherwise, we need to check whether q_1 is empty. If not, pop the front element of q_1, otherwise pop the front element of q_2, and call the `rebalance` function.

For the `popMiddle` function, we need to check whether q_1 and q_2 are empty. If both are empty, return -1. Otherwise, we need to check whether the lengths of q_1 and q_2 are equal. If they are equal, pop the last element of q_1, otherwise pop the front element of q_2, and call the `rebalance` function.

For the `popBack` function, we only need to check whether q_2 is empty. If it is empty, return -1. Otherwise, pop the last element of q_2, and call the `rebalance` function.

The time complexity of the above operations is O(1), and the space complexity is O(n), where n is the number of elements in the queue.

Python 3

```
class FrontMiddleBackQueue:
    def __init__(self):
        self.q1 = deque()
        self.q2 = deque()

    def pushFront(self, val: int) -> None:
        self.q1.appendleft(val)
        self.rebalance()

    def pushMiddle(self, val: int) -> None:
        self.q1.append(val)
        self.rebalance()

    def pushBack(self, val: int) -> None:
        self.q2.append(val)
        self.rebalance()

    def popFront(self) -> int:
        if not self.q1 and not self.q2:
            return -1
        if self.q1:
            val = self.q1.popleft()
        else:
            val = self.q2.popleft()
        self.rebalance()
        return val

    def popMiddle(self) -> int:
        if not self.q1 and not self.q2:
            return -1
        if len(self.q1) == len(self.q2):
            val = self.q1.pop()
        else:
            val = self.q2.popleft()
        self.rebalance()
        return val

    def popBack(self) -> int:
        if not self.q2:
            return -1
        val = self.q2.pop()
        self.rebalance()
        return val

    def rebalance(self):
        if len(self.q1) > len(self.q2):
            self.q2.appendleft(self.q1.pop())
        if len(self.q2) > len(self.q1) + 1:
            self.q1.append(self.q2.popleft())


# Your FrontMiddleBackQueue object will be instantiated and called as such:
# obj = FrontMiddleBackQueue()
# obj.pushFront(val)
# obj.pushMiddle(val)
# obj.pushBack(val)
# param_4 = obj.popFront()
# param_5 = obj.popMiddle()
# param_6 = obj.popBack()
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Look at the middle element without removing it


# ============================================================
# FOLLOW-UP 1: Look at the middle element without removing
# it
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep two deques, the front half and the back
# half, with the back half never smaller than the front
# half and at most one element bigger. The middle is then
# at the end of the front half or the start of the back
# half.
# Time: O(1) per operation.  Space: O(n).
from collections import deque


class MiddleQueue:
    def __init__(self):
        self.front = deque()
        self.back = deque()

    def balance(self):
        while len(self.front) > len(self.back):
            self.back.appendleft(self.front.pop())
        while len(self.back) > len(self.front) + 1:
            self.front.append(self.back.popleft())

    def push_front(self, value):
        self.front.appendleft(value)
        self.balance()

    def push_back(self, value):
        self.back.append(value)
        self.balance()

    def push_middle(self, value):
        if len(self.front) < len(self.back):
            self.front.append(value)
        else:
            self.back.appendleft(value)
        self.balance()

    def peek_middle(self):
        if not self.back:
            return -1
        if len(self.front) == len(self.back):
            return self.front[-1]
        return self.back[0]
```

PATTERN 10 · QUEUE / DEQUE · PROBLEM 10 OF 20

2398. Maximum Number of Robots Within Budget

Hard · Queue, Array, Binary Search, Prefix Sum, Sliding Window, Monotonic Queue, Heap (Priority Queue)

leetcode.com/problems/maximum-number-of-robots-within-budget

★ **TOP 7 PROBLEM** in Sliding Window Maximum / Minimum (Specific Variants): master this one first

Also appears in pattern 35 Sliding Window Maximum / Minimum (Specific Variants)

PROBLEM

You have `n` robots. You are given two **0-indexed** integer arrays, `chargeTimes` and `runningCosts`, both of length `n`. The i^{th} robot costs `chargeTimes[i]` units to charge and costs `runningCosts[i]` units to run. You are also given an integer `budget`.

The **total cost** of running `k` chosen robots is equal to `max(chargeTimes) + k * sum(runningCosts)`, where `max(chargeTimes)` is the largest charge cost among the `k` robots and `sum(runningCosts)` is the sum of running costs among the `k` robots.

Return *the **maximum** number of **consecutive** robots you can run such that the total cost **does not** exceed* `budget`.

Example 1:

```
Input: chargeTimes = [3,6,1,3,4], runningCosts = [2,1,3,4,5], budget = 25
Output: 3
Explanation:
It is possible to run all individual and consecutive pairs of robots within budget.
To obtain answer 3, consider the first 3 robots. The total cost will be max(3,6,1) + 3 * sum(2,1,3) = 6
+ 3 * 6 = 24 which is less than 25.
It can be shown that it is not possible to run more than 3 consecutive robots within budget, so we
return 3.
```

Example 2:

```
Input: chargeTimes = [11,12,19], runningCosts = [10,8,7], budget = 19
Output: 0
Explanation: No robot can be run that does not exceed the budget, so we return 0.
```

Constraints:

- `chargeTimes.length == runningCosts.length == n`
- `1 <= n <= 5 * 10^4`
- `1 <= chargeTimes[i], runningCosts[i] <= 10^5`
- `1 <= budget <= 10^15`

SOLUTION (PYTHON 3)

Solution 1: Two Pointers + Monotonic Queue

How to think about it

A contiguous team costs the window's max charge time plus length times the sum of running costs. $n \le 5 \times 10^4$; scanning every window for the max is too slow. Cost grows with length, so the left end only moves right. Expand the right end with a monotonic deque for the max charge and a sum s for running costs. When over budget, drop stale indices and move left. The maximum width is the answer.

The problem is essentially finding the maximum value within a sliding window, which can be solved using a monotonic queue.

We only need to use binary search to enumerate the size of the window k, and find the largest k that satisfies the problem requirements.

In the implementation process, we don't actually need to perform binary search enumeration. We just need to change the fixed window to a non-fixed window with double pointers.

The time complexity is O(n), and the space complexity is O(n), where n is the number of robots in the problem.

Python 3

```python
class Solution:
    def maximumRobots(
        self, chargeTimes: List[int], runningCosts: List[int], budget: int
    ) -> int:
        q = deque()
        ans = s = l = 0
        for r, (t, c) in enumerate(zip(chargeTimes, runningCosts)):
            s += c
            while q and chargeTimes[q[-1]] <= t:
                q.pop()
            q.append(r)
            while q and (r - l + 1) * s + chargeTimes[q[0]] > budget:
                if q[0] == l:
                    q.popleft()
                s -= runningCosts[l]
                l += 1
            ans = max(ans, r - l + 1)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return where the longest affordable group of robots
# starts and ends


# ============================================================
# FOLLOW-UP 1: Return where the longest affordable group
# of robots starts and ends
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same sliding window with a deque for the maximum
# charge; remember the window when it is the longest so
# far.
# Time: O(n).  Space: O(n).
from collections import deque


def longest_affordable_positions(charge_times, running_costs, budget):
    window = deque()                                # positions, charge times decreasing
    total_cost = 0
    left = 0
    best = (0, -1)
    for right in range(len(charge_times)):
        while window and charge_times[window[-1]] <= charge_times[right]:
            window.pop()
        window.append(right)
        total_cost += running_costs[right]
        while left <= right and charge_times[window[0]] + (right - left + 1) * total_cost > budget:
            total_cost -= running_costs[left]
            if window[0] == left:
                window.popleft()
            left += 1
        if right - left > best[1] - best[0]:
            best = (left, right)
    return best
```

PATTERN 10 · QUEUE / DEQUE · PROBLEM 11 OF 20

1429. First Unique Number

Medium · Design, Queue, Array, Hash Table, Data Stream

leetcode.com/problems/first-unique-number

PROBLEM

You have a queue of integers, you need to retrieve the first unique integer in the queue.

Implement the `FirstUnique` class:

- `FirstUnique(int[] nums)` Initializes the object with the numbers in the queue.
- `int showFirstUnique()` returns the value of **the first unique** integer of the queue, and returns **-1** if there is no such integer.
- `void add(int value)` insert value to the queue.

Example 1:

```
Input:
["FirstUnique","showFirstUnique","add","showFirstUnique","add","showFirstUnique","add","showFirstUniqu
e"]
[[[2,3,5]],[],[5],[],[2],[],[3],[]]
Output:
[null,2,null,2,null,3,null,-1]
Explanation:
FirstUnique firstUnique = new FirstUnique([2,3,5]);
firstUnique.showFirstUnique(); // return 2
firstUnique.add(5); // the queue is now [2,3,5,5]
firstUnique.showFirstUnique(); // return 2
firstUnique.add(2); // the queue is now [2,3,5,5,2]
firstUnique.showFirstUnique(); // return 3
firstUnique.add(3); // the queue is now [2,3,5,5,2,3]
firstUnique.showFirstUnique(); // return -1
```

Example 2:

```
Input:
["FirstUnique","showFirstUnique","add","add","add","add","add","showFirstUnique"]
[[[7,7,7,7,7,7]],[],[7],[3],[3],[7],[17],[]]
Output:
[null,-1,null,null,null,null,null,17]
Explanation:
FirstUnique firstUnique = new FirstUnique([7,7,7,7,7,7]);
firstUnique.showFirstUnique(); // return -1
firstUnique.add(7); // the queue is now [7,7,7,7,7,7,7]
firstUnique.add(3); // the queue is now [7,7,7,7,7,7,7,3]
firstUnique.add(3); // the queue is now [7,7,7,7,7,7,7,3,3]
firstUnique.add(7); // the queue is now [7,7,7,7,7,7,7,3,3,7]
firstUnique.add(17); // the queue is now [7,7,7,7,7,7,7,3,3,7,17]
firstUnique.showFirstUnique(); // return 17
```

Example 3:

```
Input:
["FirstUnique","showFirstUnique","add","showFirstUnique"]
[[[809]],[],[809],[]]
Output:
[null,809,null,-1]
Explanation:
FirstUnique firstUnique = new FirstUnique([809]);
firstUnique.showFirstUnique(); // return 809
firstUnique.add(809); // the queue is now [809,809]
firstUnique.showFirstUnique(); // return -1
```

Constraints:

- `1 <= nums.length <= 10^5`
- `1 <= nums[i] <= 10^8`
- `1 <= value <= 10^8`
- At most `50000` calls will be made to `showFirstUnique` and `add`.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

After many `add`s we must report the oldest value that still occurs once. A counter alone loses order; a queue alone cannot delete a value that just became duplicate. Count frequencies and keep current uniques in an `OrderedDict`. On `add`, insert when the count becomes 1 and erase when it becomes 2. `showFirstUnique` returns the first remaining key.

Python 3

```python
class FirstUnique:
    def __init__(self, nums: List[int]):
        self.cnt = Counter(nums)
        self.unique = OrderedDict({v: 1 for v in nums if self.cnt[v] == 1})

    def showFirstUnique(self) -> int:
        return -1 if not self.unique else next(v for v in self.unique.keys())

    def add(self, value: int) -> None:
        self.cnt[value] += 1
        if self.cnt[value] == 1:
            self.unique[value] = 1
        elif value in self.unique:
            self.unique.pop(value)


# Your FirstUnique object will be instantiated and called as such:
# obj = FirstUnique(nums)
# param_1 = obj.showFirstUnique()
# obj.add(value)
```

Complexity. Time: constructor O(n), showFirstUnique and add O(1). Space: O(n).

Solution 2

How to think about it

Method 1 deletes from the ordered map as soon as a value repeats. We can instead append to a deque and, on query, pop front elements whose count is no longer one. Lazy deletion keeps the same answer with less bookkeeping.

Python 3

```python
class FirstUnique:
    def __init__(self, nums: List[int]):
        self.cnt = Counter(nums)
        self.q = deque(nums)

    def showFirstUnique(self) -> int:
        while self.q and self.cnt[self.q[0]] != 1:
            self.q.popleft()
        return -1 if not self.q else self.q[0]

    def add(self, value: int) -> None:
        self.cnt[value] += 1
        self.q.append(value)


# Your FirstUnique object will be instantiated and called as such:
# obj = FirstUnique(nums)
# param_1 = obj.showFirstUnique()
# obj.add(value)
```

Complexity. Time: constructor O(n), add O(1), showFirstUnique O(1) amortised. Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Support removing a number from the queue
# 2. The k-th unique number


# ========================================================
# FOLLOW-UP 1: Support removing a number from the queue
# ========================================================

# ---- follow-up 1: function ----
# Answer: Keep the count of every number and a queue of
# numbers in arrival order. Removing a number sets its
# count to 2 (so it is never unique again); the queue
# front is cleaned lazily.
# Time: add and remove O(1), show_first_unique O(1)
# amortised.  Space: O(n).
from collections import deque


class FirstUniqueWithRemoval:
    def __init__(self, nums):
        self.count = {}
        self.queue = deque()
        for x in nums:
            self.add(x)

    def add(self, value):
        self.count[value] = self.count.get(value, 0) + 1
        if self.count[value] == 1:
            self.queue.append(value)

    def remove(self, value):
        if value in self.count:
            self.count[value] = 2                    # never unique again

    def show_first_unique(self):
        while self.queue and self.count[self.queue[0]] > 1:
            self.queue.popleft()
        if self.queue:
            return self.queue[0]
        return -1
```

```
# ============================================================
# FOLLOW-UP 2: The k-th unique number
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk the arrival-order list and skip numbers
# that are no longer unique until the k-th unique one is
# found: O(n) per query.
# Time: add O(1), kth O(n).  Space: O(n).
class KthUnique:
    def __init__(self):
        self.count = {}
        self.order = []

    def add(self, value):
        self.count[value] = self.count.get(value, 0) + 1
        if self.count[value] == 1:
            self.order.append(value)

    def kth(self, k):
        seen = 0
        for value in self.order:
            if self.count[value] == 1:
                seen += 1
                if seen == k:
                    return value
        return -1
```

353. Design Snake Game

Medium · Design, Queue, Array, Hash Table, Simulation

leetcode.com/problems/design-snake-game

★ **TOP 7 PROBLEM** in Queue / Deque: master this one first

PROBLEM

Design a Snake game that is played on a device with screen size `height x width`. Play the game online if you are not familiar with the game.

The snake is initially positioned at the top left corner `(0, 0)` with a length of `1` unit.

You are given an array `food` where `food[i] = (r_i, c_i)` is the row and column position of a piece of food that the snake can eat. When a snake eats a piece of food, its length and the game's score both increase by `1`.

Each piece of food appears one by one on the screen, meaning the second piece of food will not appear until the snake eats the first piece of food.

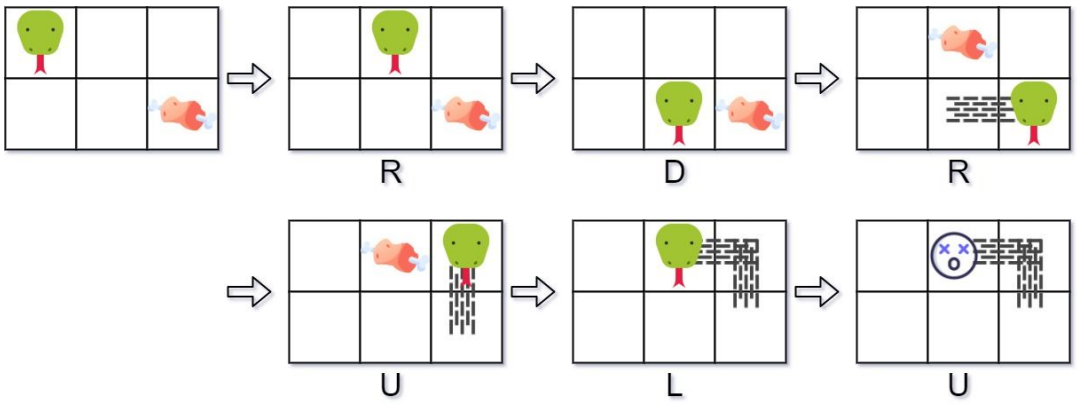
When a piece of food appears on the screen, it is **guaranteed** that it will not appear on a block occupied by the snake.

The game is over if the snake goes out of bounds (hits a wall) or if its head occupies a space that its body occupies **after** moving (i.e. a snake of length 4 cannot run into itself).

Implement the `SnakeGame` class:

- `SnakeGame(int width, int height, int[][] food)` Initializes the object with a screen of size `height x width` and the positions of the `food`.
- `int move(String direction)` Returns the score of the game after applying one `direction` move by the snake. If the game is over, return `-1`.

Example 1:



```
Input
["SnakeGame", "move", "move", "move", "move", "move", "move"]
[[3, 2, [[1, 2], [0, 1]]], ["R"], ["D"], ["R"], ["U"], ["L"], ["U"]]
Output
[null, 0, 0, 1, 1, 2, -1]

Explanation
SnakeGame snakeGame = new SnakeGame(3, 2, [[1, 2], [0, 1]]);
snakeGame.move("R"); // return 0
snakeGame.move("D"); // return 0
snakeGame.move("R"); // return 1, snake eats the first piece of food. The second piece of food appears
at (0, 1).
snakeGame.move("U"); // return 1
snakeGame.move("L"); // return 2, snake eats the second food. No more food appears.
snakeGame.move("U"); // return -1, game over because snake collides with border
```

Constraints:

- `1 <= width, height <= 10^4`
- `1 <= food.length <= 50`
- `food[i].length == 2`
- `0 <= r_i < height`
- `0 <= c_i < width`
- `direction.length == 1`
- `direction` is `'U'`, `'D'`, `'L'`, or `'R'`.
- At most `10^4` calls will be made to `move`.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Simulate snake: out of bounds or self-bite ends the game; food grows the body. The body is a queue with frequent head/tail updates. A deque stores the body (head at the front) and a set tests collisions. Compute the new head: -1 if out of bounds; on food, score and keep the tail, otherwise pop the tail. Then reject a body hit and push the head. Removing the tail first makes the old tail cell legal.

Python 3

```python
class SnakeGame:
    def __init__(self, width: int, height: int, food: List[List[int]]):
        self.m = height
        self.n = width
        self.food = food
        self.score = 0
        self.idx = 0
        self.q = deque([(0, 0)])
        self.vis = {(0, 0)}

    def move(self, direction: str) -> int:
        i, j = self.q[0]
        x, y = i, j
        match direction:
            case "U":
                x -= 1
            case "D":
                x += 1
            case "L":
                y -= 1
            case "R":
                y += 1
        if x < 0 or x >= self.m or y < 0 or y >= self.n:
            return -1
        if (
            self.idx < len(self.food)
            and x == self.food[self.idx][0]
            and y == self.food[self.idx][1]
        ):
            self.score += 1
            self.idx += 1
        else:
            self.vis.remove(self.q.pop())
        if (x, y) in self.vis:
            return -1
        self.q.appendleft((x, y))
        self.vis.add((x, y))
        return self.score


# Your SnakeGame object will be instantiated and called as such:
# obj = SnakeGame(width, height, food)
# param_1 = obj.move(direction)
```

Complexity. Time: O(1) per move. Space: O(L), L = length of the snake.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The snake wraps around the walls instead of dying,
# and there are obstacles


# ============================================================
# FOLLOW-UP 1: The snake wraps around the walls instead of
# dying, and there are obstacles
# ============================================================

# ---- follow-up 1: helper functions ----
def map_pairs(pairs):
    result = []
    for pair in pairs:
        result.append((pair[0], pair[1]))
    return result

# ---- follow-up 1: function ----
```

```
# Answer: The next head position is taken modulo the board
# size. The game ends only when the head would enter an
# obstacle or the snake's body (the tail cell is free if
# the snake does not eat).
# Time: O(1) per move.  Space: O(L + obstacles + food), L
# = snake length.
from collections import deque


class WrappingSnake:
    def __init__(self, width, height, food, obstacles):
        self.width = width
        self.height = height
        self.food = deque(food)
        self.obstacles = set(map_pairs(obstacles))
        self.body = deque([(0, 0)])
        self.cells = {(0, 0)}
        self.score = 0

    def move(self, direction):
        row, col = self.body[0]
        if direction == "U":
            row -= 1
        elif direction == "D":
            row += 1
        elif direction == "L":
            col -= 1
        else:
            col += 1
        head = (row % self.height, col % self.width)
        eating = bool(self.food) and tuple(self.food[0]) == head
        if not eating:
            tail = self.body.pop()
            self.cells.discard(tail)
        if head in self.cells or head in self.obstacles:
            return -1
        if eating:
            self.food.popleft()
            self.score += 1
        self.body.appendleft(head)
        self.cells.add(head)
        return self.score
```

PATTERN 10 · QUEUE / DEQUE · PROBLEM 14 OF 20

1696. Jump Game VI

Medium · Queue, Array, Dynamic Programming, Monotonic Queue, Heap (Priority Queue)

leetcode.com/problems/jump-game-vi

★ **TOP 7 PROBLEM** in Queue / Deque, Sliding Window Maximum / Minimum (Specific Variants): master this one first

Also appears in pattern 35 Sliding Window Maximum / Minimum (Specific Variants)

PROBLEM

You are given a **0-indexed** integer array `nums` and an integer `k`.

You are initially standing at index `0`. In one move, you can jump at most `k` steps forward without going outside the boundaries of the array. That is, you can jump from index `i` to any index in the range `[i + 1, min(n - 1, i + k)]` **inclusive**.

You want to reach the last index of the array (index `n - 1`). Your **score** is the **sum** of all `nums[j]` for each index `j` you visited in the array.

Return *the **maximum score** you can get*.

Example 1:

```
Input: nums = [1,-1,-2,4,-7,3], k = 2
Output: 7
Explanation: You can choose your jumps forming the subsequence [1,-1,4,3] (underlined above). The sum is 7.
```

Example 2:

```
Input: nums = [10,-5,-2,4,0,3], k = 3
Output: 17
Explanation: You can choose your jumps forming the subsequence [10,4,3] (underlined above). The sum is 17.
```

Example 3:

```
Input: nums = [1,-5,-20,4,-1,3,-6,-3], k = 2
Output: 0
```

Constraints:

- `1 <= nums.length, k <= 10^5`
- `-10^4 <= nums[i] <= 10^4`

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming + Monotonic Queue Optimization

How to think about it

From i we may jump into (i,i+k], adding the landing value, and want the best score at the end. Naive f[i]=nums[i]+max$_{i-k \le j < i}$ f[j] is O(nk) for n=10^5. A decreasing deque of indices holds the window maximum: f[q[0]] is the best predecessor; the back drops worse f values and the front drops indices that left the window.

We define f[i] as the maximum score when reaching index i. The value of f[i] can be transferred from f[j], where j satisfies i - k ≤ j ≤ i - 1. Therefore, we can use dynamic programming to solve this problem.

The state transition equation is:

$$f[i] = \max_{j \in [i - k, i - 1]} f[j] + nums[i]$$

We can use a monotonic queue to optimize the state transition equation. Specifically, we maintain a monotonically decreasing queue, which stores the index j, and the f[j] values corresponding to the indices in the queue are monotonically decreasing. When performing state transition, we only need to take out the index j at the front of the queue to get the maximum value of f[j], and then update the value of f[i] to f[j] + nums[i].

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array.

Python 3

```
class Solution:
    def maxResult(self, nums: List[int], k: int) -> int:
        n = len(nums)
        f = [0] * n
        q = deque([0])
        for i in range(n):
            if i - q[0] > k:
                q.popleft()
            f[i] = nums[i] + f[q[0]]
            while q and f[q[-1]] <= f[i]:
                q.pop()
            q.append(i)
        return f[-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the path that gives the best score
# 2. Minimum total cost to reach the last cell with jumps
# of at most k


# ============================================================
# FOLLOW-UP 1: Return the path that gives the best score
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep for every cell the previous cell of its
# best score (the best cell in the window of the last k
# cells); follow these links back from the last cell.
# Time: O(n).  Space: O(n).
from collections import deque


def best_jump_path(nums, k):
    n = len(nums)
    score = [0] * n
    previous = [-1] * n
    score[0] = nums[0]
    window = deque([0])                              # positions, scores decreasing
    for i in range(1, n):
        while window[0] < i - k:
            window.popleft()
        previous[i] = window[0]
        score[i] = score[window[0]] + nums[i]
        while window and score[window[-1]] <= score[i]:
            window.pop()
        window.append(i)
    path = []
    i = n - 1
    while i != -1:
        path.append(i)
        i = previous[i]
    path.reverse()
    return score[n - 1], path


# ============================================================
```

```
# FOLLOW-UP 2: Minimum total cost to reach the last cell
# with jumps of at most k
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same deque, but keep the SMALLEST cost on
# the window: pop from the back while the new cost is not
# smaller.
# Time: O(n).  Space: O(n).
from collections import deque


def min_cost_to_reach_end(costs, k):
    n = len(costs)
    best = [0] * n
    best[0] = costs[0]
    window = deque([0])
    for i in range(1, n):
```

```
        while window[0] < i - k:
            window.popleft()
        best[i] = best[window[0]] + costs[i]
        while window and best[window[-1]] >= best[i]:
            window.pop()
        window.append(i)
    return best[n - 1]
```

1823. Find the Winner of the Circular Game

Medium · Recursion, Queue, Array, Math, Simulation

leetcode.com/problems/find-the-winner-of-the-circular-game

Also appears in pattern 39 Game Theory / Minimax

PROBLEM

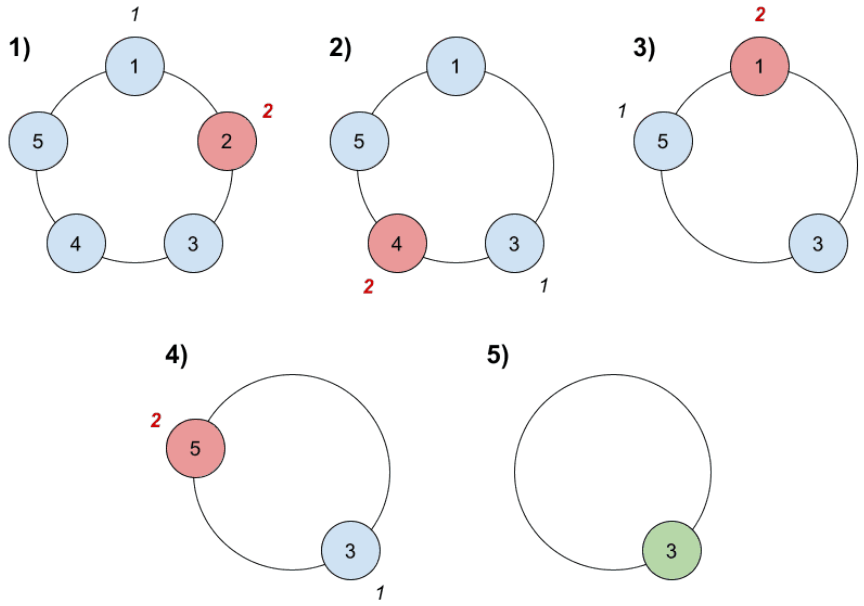
There are `n` friends that are playing a game. The friends are sitting in a circle and are numbered from `1` to `n` in **clockwise order**. More formally, moving clockwise from the `i`th friend brings you to the `(i+1)`th friend for `1 <= i < n`, and moving clockwise from the `n`th friend brings you to the `1`st friend.

The rules of the game are as follows:

1. **Start** at the `1`st friend.
2. Count the next `k` friends in the clockwise direction **including** the friend you started at. The counting wraps around the circle and may count some friends more than once.
3. The last friend you counted leaves the circle and loses the game.
4. If there is still more than one friend in the circle, go back to step `2` **starting** from the friend **immediately clockwise** of the friend who just lost and repeat.
5. Else, the last friend in the circle wins the game.

Given the number of friends, `n`, and an integer `k`, return *the winner of the game*.

Example 1:



```
Input: n = 5, k = 2
Output: 3
Explanation: Here are the steps of the game:
1) Start at friend 1.
2) Count 2 friends clockwise, which are friends 1 and 2.
3) Friend 2 leaves the circle. Next start is friend 3.
4) Count 2 friends clockwise, which are friends 3 and 4.
5) Friend 4 leaves the circle. Next start is friend 5.
6) Count 2 friends clockwise, which are friends 5 and 1.
7) Friend 1 leaves the circle. Next start is friend 3.
8) Count 2 friends clockwise, which are friends 3 and 5.
9) Friend 5 leaves the circle. Only friend 3 is left, so they are the winner.
```

Example 2:

```
Input: n = 6, k = 5
Output: 1
Explanation: The friends leave in this order: 5, 4, 6, 2, 3. The winner is friend 1.
```

Constraints:

- `1 <= k <= n <= 500`

Follow up:

Could you solve this problem in linear time with constant space?

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

n people stand in a circle and every k-th person is eliminated. n≤ 500, so a list simulation would pass, but counting k steps naively does not use the recurrence. The winner among n people is the winner among n-1 people shifted by k modulo n (treating 0 as n). The base case n=1 is person 1. The recurrence solves the game in O(n).

Python 3

```python
class Solution:
    def findTheWinner(self, n: int, k: int) -> int:
        if n == 1:
            return 1
        ans = (k + self.findTheWinner(n - 1, k)) % n
        return n if ans == 0 else ans
```

Solution 2: Simulation

How to think about it

Solution 1 is a compact index recurrence. We can instead keep the remaining people in a list or deque and repeatedly count to k and delete until one remains. With small n the simulation stays faithful to the statement and the 1-based labels.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```python
class Solution:
    def findTheWinner(self, n: int, k: int) -> int:
        friends = list(range(1, n + 1))
        i = 0
        while len(friends) > 1:
            i = (i + k - 1) % len(friends)
            friends.pop(i)
        return friends[0]
```

Complexity. Time: O(n^2) (removing from the middle of a list is O(n)). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the order in which the friends leave the
# circle
# 2. O(n) time and O(1) space with a formula


# ========================================================
# FOLLOW-UP 1: Return the order in which the friends leave
# the circle
# ========================================================

# ---- follow-up 1: function ----
# Answer: Simulate the circle with a list: move k - 1
# steps (modulo the size), remove that friend, and
# continue from the same position.
# Time: O(n^2) (removing from the middle of a list is
# O(n)).  Space: O(n).
def elimination_order(n, k):
    friends = list(range(1, n + 1))
    order = []
    position = 0
    while friends:
        position = (position + k - 1) % len(friends)
        order.append(friends.pop(position))
    return order


# ========================================================
# FOLLOW-UP 2: O(n) time and O(1) space with a formula
# ========================================================

# ---- follow-up 2: function ----
# Answer: The winner of a circle of size 1 is position 0.
# When the circle grows by one friend, the survivor's
# position moves k places forward, modulo the new size.
# Time: O(n).  Space: O(1).
def find_the_winner_formula(n, k):
    winner = 0
    for size in range(2, n + 1):
        winner = (winner + k) % size
    return winner + 1
```

PATTERN 10 · QUEUE / DEQUE · PROBLEM 20 OF 20

936. Stamping The Sequence

Hard · Stack, Greedy, Queue, String

leetcode.com/problems/stamping-the-sequence

PROBLEM

You are given two strings `stamp` and `target`. Initially, there is a string `s` of length `target.length` with all `s[i] == '?'`.

In one turn, you can place `stamp` over `s` and replace every letter in the `s` with the corresponding letter from `stamp`.

- For example, if `stamp = "abc"` and `target = "abcba"`, then `s` is `"?????"` initially. In one turn you can: Note that `stamp` must be fully contained in the boundaries of `s` in order to stamp (i.e., you cannot place `stamp` at index `3` of `s`).
  - place `stamp` at index `0` of `s` to obtain `"abc??"`,
  - place `stamp` at index `1` of `s` to obtain `"?abc?"`, or
  - place `stamp` at index `2` of `s` to obtain `"??abc"`.

We want to convert `s` to `target` using **at most** `10 * target.length` turns.

Return *an array of the index of the left-most letter being stamped at each turn*. If we cannot obtain `target` from `s` within `10 * target.length` turns, return an empty array.

Example 1:

```
Input: stamp = "abc", target = "ababc"
Output: [0,2]
Explanation: Initially s = "?????".
- Place stamp at index 0 to get "abc??".
- Place stamp at index 2 to get "ababc".
[1,0,2] would also be accepted as an answer, as well as some other answers.
```

Example 2:

```
Input: stamp = "abca", target = "aabcaca"
Output: [3,0,1]
Explanation: Initially s = "???????".
- Place stamp at index 3 to get "???abca".
- Place stamp at index 0 to get "abcabca".
- Place stamp at index 1 to get "aabcaca".
```

Constraints:

- `1 <= stamp.length <= target.length <= 1000`
- `stamp` and `target` consist of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Reverse Thinking + Topological Sorting

How to think about it

Forward stamping overwrites earlier marks, so it is hard to know when a stamp is legal. Work backward from target toward question marks: a window can be “unstamped” once every still-visible letter matches the stamp. The in-degree of a window is the number of mismatched letters; each position points at the windows that still need it. Windows of degree 0 enter a queue; unstamping them notifies neighbors. If every cell is covered, reverse the order to obtain a forward sequence.

If we operate on the sequence in a forward manner, it would be quite complicated because subsequent operations would overwrite previous ones. Therefore, we consider operating on the sequence in a reverse manner, i.e., starting from the

target string target and considering the process of turning target into ?????.

Let's denote the length of the stamp as m and the length of the target string as n. If we operate on the target string with the stamp, there are n-m+1 starting positions where the stamp can be placed. We can enumerate these n-m+1 starting positions and use a method similar to topological sorting to operate in reverse.

Firstly, we clarify that each starting position corresponds to a window of length m.

Next, we define the following data structures or variables:

- In-degree array indeg, where indeg[i] represents how many characters in the i-th window are different from the characters in the stamp. Initially, indeg[i]=m. If indeg[i]=0, it means that all characters in the i-th window are the same as those in the stamp, and we can place the stamp in the i-th window.
- Adjacency list g, where g[i] represents the set of all windows with different characters from the stamp on the i-th position of the target string target.
- Queue q, used to store the numbers of all windows with an in-degree of 0.
- Array vis, used to mark whether each position of the target string target has been covered.
- Array ans, used to store the answer.

Then, we perform topological sorting. In each step of the topological sorting, we take out the window number i at the head of the queue, and put i into the answer array ans. Then, we enumerate each position j in the stamp. If the j-th position in the i-th window has not been covered, we cover it and reduce the in-degree of all windows in the indeg array that are the same as the j-th position in the i-th window by 1. If the in-degree of a window becomes 0, we put it into the queue q for processing next time.

After the topological sorting is over, if every position of the target string target has been covered, then the answer array ans stores the answer we are looking for. Otherwise, the target string target cannot be covered, and we return an empty array.

The time complexity is O(n × (n - m + 1)), and the space complexity is O(n × (n - m + 1)). Here, n and m are the lengths of the target string target and the stamp, respectively.

Python 3

```python
class Solution:
    def movesToStamp(self, stamp: str, target: str) -> List[int]:
        m, n = len(stamp), len(target)
        indeg = [m] * (n - m + 1)
        q = deque()
        g = [[] for _ in range(n)]
        for i in range(n - m + 1):
            for j, c in enumerate(stamp):
                if target[i + j] == c:
                    indeg[i] -= 1
                    if indeg[i] == 0:
                        q.append(i)
                else:
                    g[i + j].append(i)
        ans = []
        vis = [False] * n
        while q:
            i = q.popleft()
            ans.append(i)
            for j in range(m):
                if not vis[i + j]:
                    vis[i + j] = True
                    for k in g[i + j]:
                        indeg[k] -= 1
                        if indeg[k] == 0:
                            q.append(k)
        return ans[::-1] if all(vis) else []
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Apply a list of stamp positions in order and return
# the resulting string (a checker)
# 2. Return the stamp positions in the order they must be
# applied (forward order)


# ============================================================
# FOLLOW-UP 1: Apply a list of stamp positions in order
# and return the resulting string (a checker)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Start with a row of '?' and for every position
# overwrite the next len(stamp) letters; the final string
# must equal the target.
# Time: O(m * s), m = stamps applied, s = stamp length.
# Space: O(n).
def apply_stamps(stamp, positions, length):
    result = ["?"] * length
    for start in positions:
        for i in range(len(stamp)):
            result[start + i] = stamp[i]
    return "".join(result)


# ============================================================
# FOLLOW-UP 2: Return the stamp positions in the order
# they must be applied (forward order)
# ============================================================

# ---- follow-up 2: helper functions ----
def can_stamp_here(target, stamp, start):
    # the window must contain at least one real letter, and every real letter must match the stamp
    real_letter = False
    for i in range(len(stamp)):
        if target[start + i] == "?":
            continue
        if target[start + i] != stamp[i]:
            return False
        real_letter = True
    return real_letter

# ---- follow-up 2: function ----
# Answer: The search finds the stamps in reverse order
# (the last stamp is found first), so reverse the list of
# positions before returning it.
# Time: O(n^2 * s) in the worst case (up to n passes, each
# trying every start).  Space: O(n).
def stamping_forward_order(stamp, target):
    letters = list(target)
    reverse_order = []
    changed = True
    while changed:
        changed = False
        for start in range(len(letters) - len(stamp) + 1):
            if can_stamp_here(letters, stamp, start):
                for i in range(len(stamp)):
                    letters[start + i] = "?"
                reverse_order.append(start)
```

```
                changed = True
    for ch in letters:
        if ch != "?":
            return []
```

```
    reverse_order.reverse()
    return reverse_order
```

PATTERN 11 OF 41

Tree DFS (Preorder / Inorder / Postorder)

20 problems · 10 Easy · 9 Medium · 1 Hard

THE IDEA

One recursive function that visits a node, asks its children, and reports upward. Choose the order: preorder passes information down, postorder combines answers coming up, inorder on a BST is sorted.

Signals: depth, diameter, path sums, same tree, subtree checks, lowest common ancestor, build or serialize a tree.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
def dfs(node):
    if node is None:
        return 0                        # base case
    left = dfs(node.left)
    right = dfs(node.right)
    best[0] = max(best[0], left + right)    # use both children here
    return 1 + max(left, right)           # report one chain upward
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Binary Tree Inorder Traversal	Easy	LC 94	530
2	Binary Tree Preorder Traversal	Easy	LC 144	535
3	Binary Tree Postorder Traversal	Easy	LC 145	540
4	Maximum Depth of Binary Tree	Easy	LC 104	545
5	Path Sum	Easy	LC 112	547
6	Diameter of Binary Tree	Easy	LC 543	550
7	Balanced Binary Tree	Easy	LC 110	553
8	Same Tree	Easy	LC 100	556
9	Symmetric Tree	Easy	LC 101	560
10	Invert Binary Tree	Easy	LC 226	563
11	★ **Path Sum II**	Medium	LC 113	565
12	★ **Lowest Common Ancestor of a Binary Tree**	Medium	LC 236	568
13	★ **Construct Binary Tree from Preorder and Inorder Traversal**	Medium	LC 105	571
14	★ **Flatten Binary Tree to Linked List**	Medium	LC 114	575
15	Sum Root to Leaf Numbers	Medium	LC 129	578
16	★ **Path Sum III** (see page)	Medium	LC 437	262
17	Count Good Nodes in Binary Tree	Medium	LC 1448	581
18	Binary Tree Pruning	Medium	LC 814	585
19	★ **All Nodes Distance K in Binary Tree**	Medium	LC 863	588
20	★ **Binary Tree Maximum Path Sum**	Hard	LC 124	591

94. Binary Tree Inorder Traversal

Easy · Stack, Tree, Depth-First Search, Binary Tree

leetcode.com/problems/binary-tree-inorder-traversal

PROBLEM

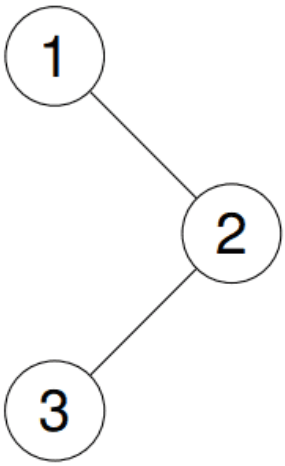
Given the `root` of a binary tree, return *the inorder traversal of its nodes' values*.

Example 1:

Input: root = [1,null,2,3]

Output: [1,3,2]

Explanation:

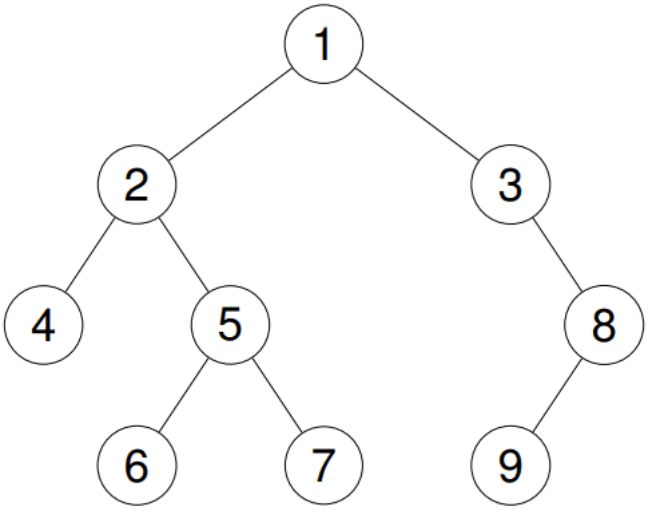


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [4,2,6,5,7,1,3,9,8]

Explanation:



Example 3:

Input: root = []

Output: []

Example 4:

Input: root = [1]

Output: [1]

Constraints:

- The number of nodes in the tree is in the range `[0, 100]`.
- `-100 <= Node.val <= 100`

Follow up:

Recursive solution is trivial, could you do it iteratively?

SOLUTIONS (PYTHON 3)

Solution 1: Recursive Traversal

> **How to think about it**
>
> Inorder is left, root, right. A tree is already recursive, so the first idea is recursion: finish the left subtree, record the root, then walk the right. n ≤ 100, so an O(n) call stack is fine. The follow-up is what asks for iteration.

We first recursively traverse the left subtree, then visit the root node, and finally recursively traverse the right subtree.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree, and the space complexity mainly depends on the stack space of the recursive call.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        def dfs(root):
            if root is None:
                return
            dfs(root.left)
            ans.append(root.val)
            dfs(root.right)

        ans = []
        dfs(root)
        return ans
```

Solution 2: Stack Implementation for Non-recursive Traversal

> **How to think about it**
>
> Solution 1 is correct; the follow-up drops the call stack. Recursion is “go left until you cannot, then pop, visit, and turn right”. An explicit stack simulates that: push while a left child exists, otherwise pop, visit, and move to the right child. Same order as inorder, still O(n) space.

The non-recursive approach is as follows:

1. Define a stack `stk`.
2. Push the left nodes of the tree into the stack in sequence.
3. When the left node is null, pop and process the top element of the stack.

4. Repeat steps 2-3.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree, and the space complexity mainly depends on the stack space.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        ans, stk = [], []
        while root or stk:
            if root:
                stk.append(root)
                root = root.left
            else:
                root = stk.pop()
                ans.append(root.val)
                root = root.right
        return ans
```

Solution 3: Morris Implementation for In-order Traversal

> **How to think about it**
>
> A stack still costs O(h). Morris notices that the successor of the rightmost node in the left subtree is the current root, and that right pointer is unused. Temporarily thread it to the root, walk down the left chain, then undo the link on the way back. The tree's null pointers become the stack, inorder is preserved, and extra space is O(1).

Morris traversal does not require a stack, so the space complexity is O(1). The core idea is:

Traverse the binary tree nodes,

1. If the left subtree of the current node `root` is null, **add the current node value to the result list** `ans`, and update the current node to `root.right`.
2. If the left subtree of the current node `root` is not null, find the rightmost node `prev` of the left subtree (which is the predecessor node of the `root` node in in-order traversal):
   - If the right subtree of the predecessor node `prev` is null, point the right subtree of the predecessor node to the current node `root`, and update the current node to `root.left`.
   - If the right subtree of the predecessor node `prev` is not null, **add the current node value to the result list** `ans`, then point the right subtree of the predecessor node to null (i.e., disconnect `prev` and `root`), and update the current node to `root.right`.
3. Repeat the above steps until the binary tree node is null, and the traversal ends.

The time complexity is O(n), and the space complexity is O(1). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        ans = []
        while root:
            if root.left is None:
                ans.append(root.val)
                root = root.right
            else:
                prev = root.left
                while prev.right and prev.right != root:
                    prev = prev.right
                if prev.right is None:
                    prev.right = root
                    root = root.left
                else:
                    ans.append(root.val)
                    prev.right = None
                    root = root.right
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. O(1) extra space: Morris traversal
# 2. Produce the values lazily, one at a time (an
# iterator)


# ============================================================
# FOLLOW-UP 1: O(1) extra space: Morris traversal
# ============================================================

# ---- follow-up 1: function ----
# Answer: Temporarily link the rightmost node of the left
# subtree back to the current node so you can climb back
# up without a stack; remove the link again when you come
# back.
# Time: O(n).  Space: O(1) extra (Morris).
def inorder_morris(root):
    result = []
    node = root
    while node:
        if node.left is None:
            result.append(node.val)
            node = node.right
        else:
            predecessor = node.left
            while predecessor.right and predecessor.right is not node:
                predecessor = predecessor.right
            if predecessor.right is None:
                predecessor.right = node             # a temporary link back up
                node = node.left
            else:
                predecessor.right = None             # remove the link
                result.append(node.val)
                node = node.right
    return result
```

```
# ============================================================
# FOLLOW-UP 2: Produce the values lazily, one at a time
# (an iterator)
# ============================================================

# ---- follow-up 2: function ----
# Answer: A generator keeps an explicit stack of the left
# spine; each next() pops one node, yields it, and pushes
# the left spine of its right child.
# Time: O(n).  Space: O(h).
def inorder_iterator(root):
    stack = []
    node = root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        yield node.val
        node = node.right
```

144. Binary Tree Preorder Traversal

Easy · Stack, Tree, Depth-First Search, Binary Tree

leetcode.com/problems/binary-tree-preorder-traversal

PROBLEM

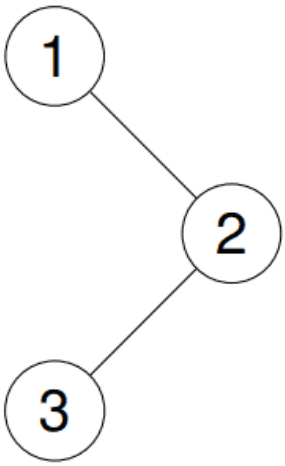
Given the `root` of a binary tree, return *the preorder traversal of its nodes' values*.

Example 1:

Input: root = [1,null,2,3]

Output: [1,2,3]

Explanation:

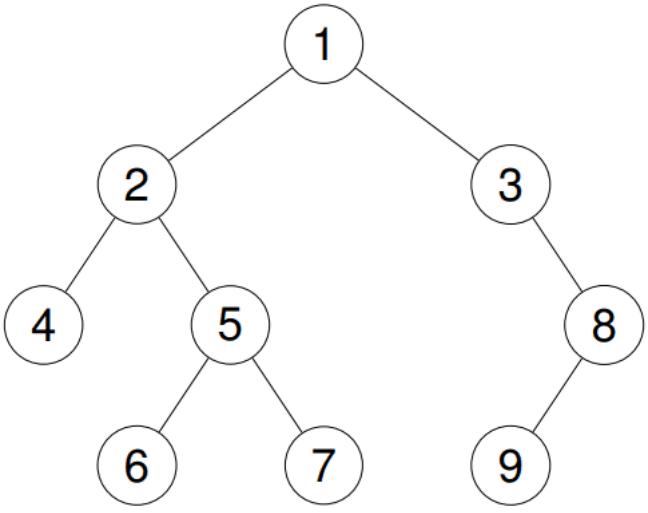


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [1,2,4,5,6,7,3,8,9]

Explanation:



Example 3:

Input: root = []

Output: []

Example 4:

Input: root = [1]

Output: [1]

Constraints:

- The number of nodes in the tree is in the range `[0, 100]`.
- `-100 <= Node.val <= 100`

Follow up: Recursive solution is trivial, could you do it iteratively?

SOLUTIONS (PYTHON 3)

Solution 1: Recursive Traversal

How to think about it

Preorder is root, left, right. A tree is recursive, so record the root and recurse on both children. n≤ 100, so the call stack is fine. The follow-up asks for iteration.

We first visit the root node, then recursively traverse the left and right subtrees.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree. The space complexity mainly depends on the stack space used for recursive calls.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def preorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        def dfs(root):
            if root is None:
                return
            ans.append(root.val)
            dfs(root.left)
            dfs(root.right)

        ans = []
        dfs(root)
        return ans
```

Solution 2: Stack Implementation for Non-Recursive Traversal

How to think about it

Solution 1 is already correct; the follow-up drops the call stack. An explicit stack pushes right then left, so popping visits root first. Same order, still O(n) space.

The idea of using a stack to implement non-recursive traversal is as follows:

1. Define a stack stk, and first push the root node into the stack.
2. If the stack is not empty, pop a node from the stack each time.
3. Process the node.
4. First push the right child of the node into the stack, then push the left child of the node into the stack (if there are child nodes).
5. Repeat steps 2-4.

6. Return the result.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree. The space complexity mainly depends on the stack space.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def preorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        ans = []
        if root is None:
            return ans
        stk = [root]
        while stk:
            node = stk.pop()
            ans.append(node.val)
            if node.right:
                stk.append(node.right)
            if node.left:
                stk.append(node.left)
        return ans
```

Solution 3: Morris Preorder Traversal

> **How to think about it**
>
> Solution 2 still uses O(h) space. Morris temporarily links the rightmost node of the left subtree to the current root, emits the root on the first visit, and clears the link on return. The tree's null pointers act as the stack, O(1) space.

Morris traversal does not require a stack, and its space complexity is O(1). The core idea is:

Traverse the binary tree nodes,

1. If the left subtree of the current node `root` is empty, add the current node value to the result list ans, and update the current node to `root.right`.
1. If the left subtree of the current node `root` is not empty, find the rightmost node `pre` of the left subtree (which is the predecessor of the `root` node in inorder traversal):
   - If the right subtree of the predecessor node `pre` is empty, add the current node value to the result list ans, then point the right subtree of the predecessor node to the current node `root`, and update the current node to `root.left`.
   - If the right subtree of the predecessor node `pre` is not empty, point the right subtree of the predecessor node to null (i.e., disconnect `pre` and `root`), and update the current node to `root.right`.
1. Repeat the above steps until the binary tree node is null, and the traversal ends.

The time complexity is O(n), where n is the number of nodes in the binary tree. The space complexity is O(1).

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def preorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        ans = []
        while root:
            if root.left is None:
                ans.append(root.val)
                root = root.right
            else:
                prev = root.left
                while prev.right and prev.right != root:
                    prev = prev.right
                if prev.right is None:
                    ans.append(root.val)
                    prev.right = root
                    root = root.left
                else:
                    prev.right = None
                    root = root.right
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Preorder of an N-ary tree (each node has a list of
# children)
# 2. Return every value together with its depth


# ============================================================
# FOLLOW-UP 1: Preorder of an N-ary tree (each node has a
# list of children)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use a stack and push the children in reverse
# order so the first child is processed first.
# Time: O(n).  Space: O(n).
def preorder_nary(root):
    result = []
    stack = [root] if root else []
    while stack:
        node = stack.pop()
        result.append(node.val)
        for child in reversed(node.children):
            stack.append(child)
    return result


# ============================================================
# FOLLOW-UP 2: Return every value together with its depth
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the depth next to each node on the stack.
# Time: O(n).  Space: O(h).
def preorder_with_depth(root):
    result = []
    stack = [(root, 0)] if root else []
    while stack:
```

```
        node, depth = stack.pop()
        result.append((node.val, depth))
        if node.right:
            stack.append((node.right, depth + 1))
        if node.left:
            stack.append((node.left, depth + 1))
    return result
```

145. Binary Tree Postorder Traversal

Easy · Stack, Tree, Depth-First Search, Binary Tree

leetcode.com/problems/binary-tree-postorder-traversal

PROBLEM

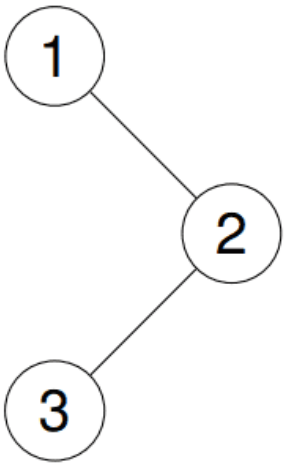
Given the `root` of a binary tree, return *the postorder traversal of its nodes' values*.

Example 1:

Input: root = [1,null,2,3]

Output: [3,2,1]

Explanation:

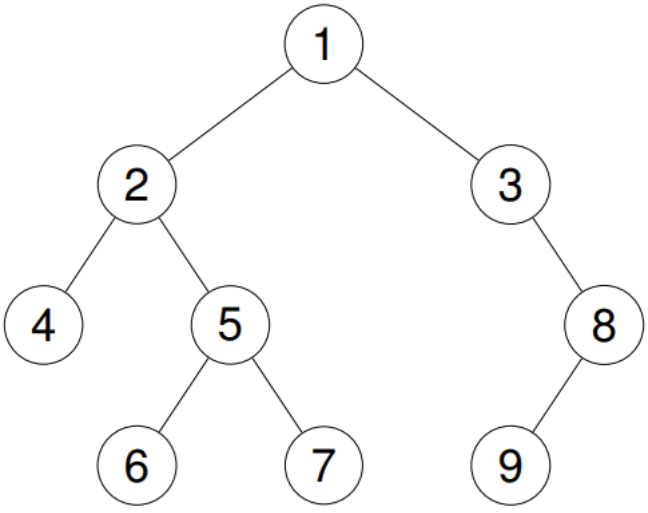


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [4,6,7,5,2,9,8,3,1]

Explanation:



Example 3:

Input: root = []

Output: []

Example 4:

Input: root = [1]

Output: [1]

Constraints:

- The number of the nodes in the tree is in the range `[0, 100]`.
- `-100 <= Node.val <= 100`

Follow up:

Recursive solution is trivial, could you do it iteratively?

SOLUTIONS (PYTHON 3)

Solution 1: Recursion

> **How to think about it**
>
> Postorder is left, right, root. Recurse on both children, then record the root. n≤ 100, so recursion is fine. The follow-up asks for iteration.

We first recursively traverse the left and right subtrees, then visit the root node.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree. The space complexity mainly depends on the stack space used for recursive calls.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def postorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        def dfs(root):
            if root is None:
                return
            dfs(root.left)
            dfs(root.right)
            ans.append(root.val)

        ans = []
        dfs(root)
        return ans
```

Solution 2: Stack Implementation for Postorder Traversal

> **How to think about it**
>
> Solution 1 uses the call stack. Root-right-left is preorder with children swapped; reversing it yields left-right-root. Traverse in that order with a stack and reverse the result, with no “second visit” flag.

The order of preorder traversal is: root, left, right. If we change the order of the left and right children, the order becomes: root, right, left. Finally, reversing the result gives us the postorder traversal result.

Therefore, the idea of using a stack to implement non-recursive traversal is as follows:

1. Define a stack stk, and first push the root node into the stack.
2. If the stack is not empty, pop a node from the stack each time.

3. Process the node.
4. First push the left child of the node into the stack, then push the right child of the node into the stack (if there are child nodes).
5. Repeat steps 2-4.
6. Reverse the result to get the postorder traversal result.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree. The space complexity mainly depends on the stack space.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def postorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        ans = []
        if root is None:
            return ans
        stk = [root]
        while stk:
            node = stk.pop()
            ans.append(node.val)
            if node.left:
                stk.append(node.left)
            if node.right:
                stk.append(node.right)
        return ans[::-1]
```

Solution 3: Morris Implementation for Postorder Traversal

> **How to think about it**
>
> Solution 2 still needs an O(n) stack and a reverse. Morris threads the tree in root-right-left order and reverses the output, so extra space becomes O(1).

Morris traversal does not require a stack, and its space complexity is O(1). The core idea is:

Traverse the binary tree nodes,

1. If the right subtree of the current node `root` is empty, add the current node value to the result list ans, and update the current node to `root.left`.
1. If the right subtree of the current node `root` is not empty, find the leftmost node `next` of the right subtree (which is the successor of the `root` node in inorder traversal):
   - If the left subtree of the successor node `next` is empty, add the current node value to the result list ans, then point the left subtree of the successor node to the current node `root`, and update the current node to `root.right`.
   - If the left subtree of the successor node `next` is not empty, point the left subtree of the successor node to null (i.e., disconnect `next` and `root`), and update the current node to `root.left`.
1. Repeat the above steps until the binary tree node is null, and the traversal ends.
1. Finally, return the reverse of the result list.

The idea of Morris postorder traversal is consistent with Morris preorder traversal, just change the "root-left-right" of preorder to "root-right-left", and finally reverse the result to become "left-right-root".

The time complexity is O(n), where n is the number of nodes in the binary tree. The space complexity is O(1).

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def postorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        ans = []
        while root:
            if root.right is None:
                ans.append(root.val)
                root = root.left
            else:
                next = root.right
                while next.left and next.left != root:
                    next = next.left
                if next.left != root:
                    ans.append(root.val)
                    next.left = root
                    root = root.right
                else:
                    next.left = None
                    root = root.left
        return ans[::-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Iterative postorder with ONE stack
# 2. Postorder of an N-ary tree


# ============================================================
# FOLLOW-UP 1: Iterative postorder with ONE stack
# ============================================================

# ---- follow-up 1: function ----
# Answer: Do a 'node, right, left' preorder with a stack
# and reverse the result: that is 'left, right, node'.
# Time: O(n).  Space: O(n).
def postorder_one_stack(root):
    result = []
    stack = [root] if root else []
    while stack:
        node = stack.pop()
        result.append(node.val)
        if node.left:
            stack.append(node.left)
        if node.right:
            stack.append(node.right)
    result.reverse()
    return result


# ============================================================
# FOLLOW-UP 2: Postorder of an N-ary tree
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same trick: visit 'node, then children from
# right to left', then reverse.
# Time: O(n).  Space: O(n).
def postorder_nary(root):
    result = []
```

```
    stack = [root] if root else []
    while stack:
        node = stack.pop()
        result.append(node.val)
        for child in node.children:
            stack.append(child)
    result.reverse()
    return result
```

104. Maximum Depth of Binary Tree

Easy · Tree, Depth-First Search, Breadth-First Search, Binary Tree

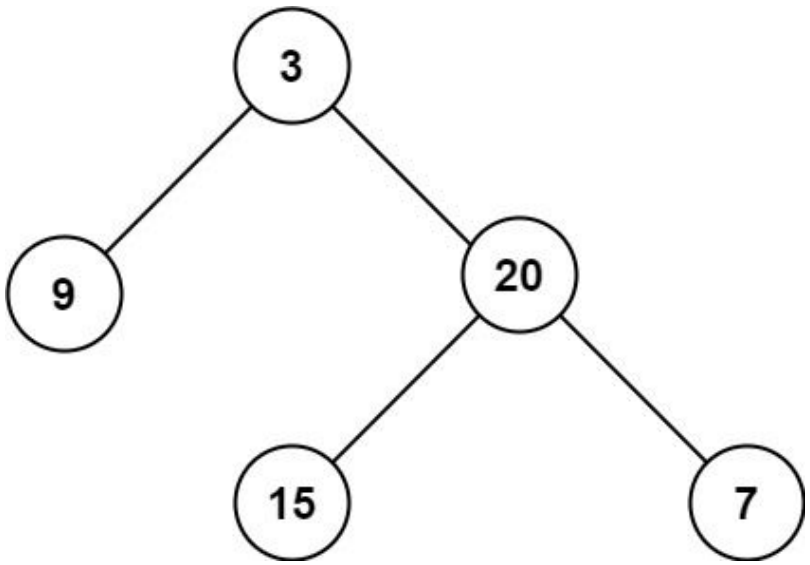
leetcode.com/problems/maximum-depth-of-binary-tree

PROBLEM

Given the `root` of a binary tree, return *its maximum depth*.

A binary tree's **maximum depth** is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: 3
```

Example 2:

```
Input: root = [1,null,2]
Output: 2
```

Constraints:

- The number of nodes in the tree is in the range `[0, 10^4]`.
- `-100 <= Node.val <= 100`

SOLUTION (PYTHON 3)

Solution 1: Recursion

> **How to think about it**
>
> Depth is the number of nodes on the longest root-to-leaf path. Level-order counting of layers works, and $n \le 10^4$ is fine. A tree is recursive: the depth is one plus the larger of the two subtree depths, and the empty tree has depth 0. Recurse on both children and take the max.

Recursively traverse the left and right subtrees, calculate the maximum depth of the left and right subtrees, and then take the maximum value plus 1.

The time complexity is O(n), where n is the number of nodes in the binary tree. Each node is traversed only once in the recursion.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maxDepth(self, root: TreeNode) -> int:
        if root is None:
            return 0
        l, r = self.maxDepth(root.left), self.maxDepth(root.right)
        return 1 + max(l, r)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The MINIMUM depth (distance to the nearest leaf)
# 2. The maximum depth of an N-ary tree


# ============================================================
# FOLLOW-UP 1: The MINIMUM depth (distance to the nearest
# leaf)
# ============================================================

# ---- follow-up 1: function ----
# Answer: A node with only one child is not a leaf, so the
# depth must continue on the side that exists. A level-by-
# level search stops at the first leaf.
# Time: O(n).  Space: O(w), w = widest level.
from collections import deque


def min_depth(root):
    if root is None:
        return 0
    queue = deque([(root, 1)])
    while queue:
        node, depth = queue.popleft()
        if node.left is None and node.right is None:
            return depth
        if node.left:
            queue.append((node.left, depth + 1))
        if node.right:
            queue.append((node.right, depth + 1))


# ============================================================
# FOLLOW-UP 2: The maximum depth of an N-ary tree
# ============================================================

# ---- follow-up 2: function ----
# Answer: One more than the deepest child; a node without
# children has depth 1.
# Time: O(n).  Space: O(h).
def max_depth_nary(root):
    if root is None:
        return 0
    deepest = 0
    for child in root.children:
        depth = max_depth_nary(child)
        if depth > deepest:
            deepest = depth
    return deepest + 1
```

112. Path Sum

Easy · Tree, Depth-First Search, Breadth-First Search, Binary Tree

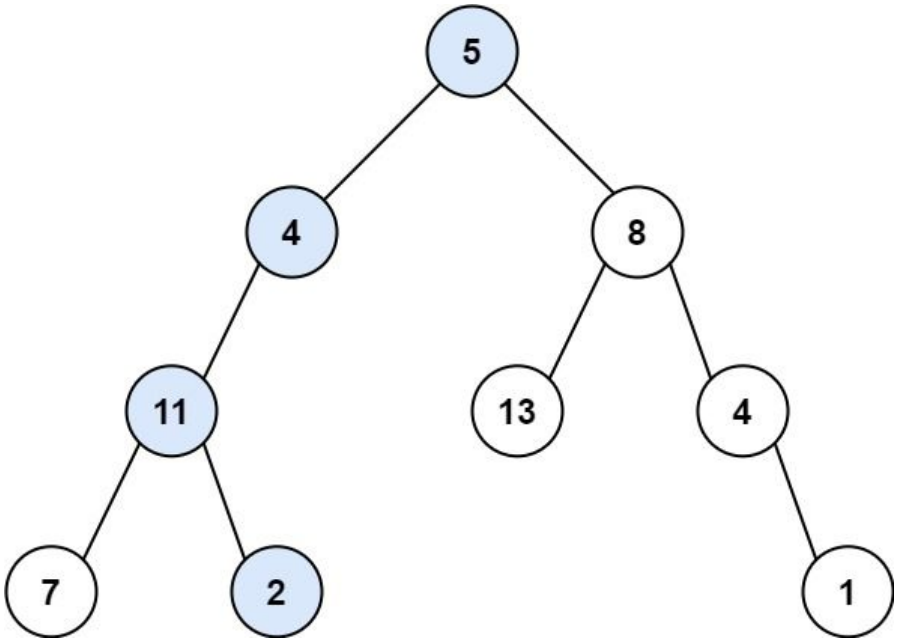
leetcode.com/problems/path-sum

PROBLEM

Given the `root` of a binary tree and an integer `targetSum`, return `true` if the tree has a **root-to-leaf** path such that adding up all the values along the path equals `targetSum`.

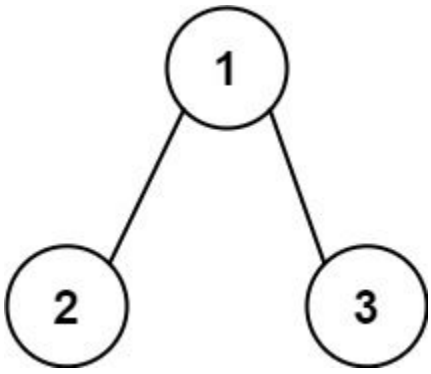
A **leaf** is a node with no children.

Example 1:




```
Input: root = [5,4,8,11,null,13,4,7,2,null,null,null,1], targetSum = 22
Output: true
Explanation: The root-to-leaf path with the target sum is shown.
```

Example 2:




```
Input: root = [1,2,3], targetSum = 5
Output: false
Explanation: There are two root-to-leaf paths in the tree:
(1 --> 2): The sum is 3.
(1 --> 3): The sum is 4.
There is no root-to-leaf path with sum = 5.
```

Example 3:

```
Input: root = [], targetSum = 0
Output: false
Explanation: Since the tree is empty, there are no root-to-leaf paths.
```

Constraints:

- The number of nodes in the tree is in the range `[0, 5000]`.
- `-1000 <= Node.val <= 1000`
- `-1000 <= targetSum <= 1000`

SOLUTION (PYTHON 3)

Solution 1: Recursion

> **How to think about it**
>
> We need a root-to-leaf path whose values sum to the target. Enumerating every path works for n ≤ 5000, but copying paths is extra work. Accumulate the running sum on the way down and test it only at a leaf. Either child succeeding is enough; the full path need not be stored.

Starting from the root node, recursively traverse the tree and update the value of the node to the path sum from the root node to that node. When you traverse to a leaf node, determine whether this path sum is equal to the target value. If it is equal, return `true`, otherwise return `false`.

The time complexity is O(n), where n is the number of nodes in the binary tree. Each node is visited once.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def hasPathSum(self, root: Optional[TreeNode], targetSum: int) -> bool:
        def dfs(root, s):
            if root is None:
                return False
            s += root.val
            if root.left is None and root.right is None and s == targetSum:
                return True
            return dfs(root.left, s) or dfs(root.right, s)

        return dfs(root, 0)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the root-to-leaf path itself
# 2. Count the root-to-leaf paths whose sum is the target


# ============================================================
# FOLLOW-UP 1: Return the root-to-leaf path itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Carry the list of values on the way down; at a
# leaf compare the running sum, and return the path as
# soon as one is found.
# Time: O(n).  Space: O(h).
```

```python
def path_with_sum(root, target):
    path = []

    def walk(node, remaining):
        if node is None:
            return False
        path.append(node.val)
        if node.left is None and node.right is None and remaining == node.val:
            return True
        if walk(node.left, remaining - node.val) or walk(node.right, remaining - node.val):
            return True
        path.pop()
        return False

    if walk(root, target):
        return path
    return None


# ========================================================
# FOLLOW-UP 2: Count the root-to-leaf paths whose sum is
# the target
# ========================================================

# ---- follow-up 2: function ----
# Answer: Same recursion, but add the counts of both sides
# instead of stopping at the first hit.
# Time: O(n).  Space: O(h).
def count_paths_to_leaf(root, target):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1 if root.val == target else 0
    return count_paths_to_leaf(root.left, target - root.val) + count_paths_to_leaf(root.right, target -
root.val)
```

543. Diameter of Binary Tree

Easy · Tree, Depth-First Search, Binary Tree, Tree DP

leetcode.com/problems/diameter-of-binary-tree

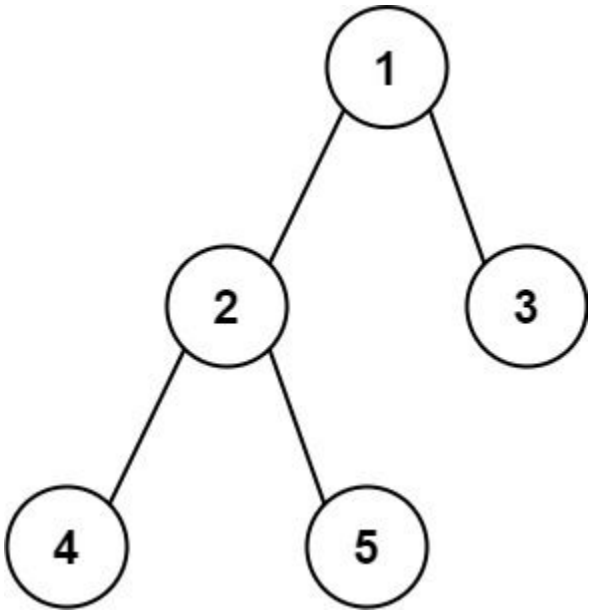
PROBLEM

Given the `root` of a binary tree, return *the length of the **diameter** of the tree*.

The **diameter** of a binary tree is the **length** of the longest path between any two nodes in a tree. This path may or may not pass through the `root`.

The **length** of a path between two nodes is represented by the number of edges between them.

Example 1:




```
Input: root = [1,2,3,4,5]
Output: 3
Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].
```

Example 2:

```
Input: root = [1,2]
Output: 1
```

Constraints:

- The number of nodes in the tree is in the range `[1, 10^4]`.
- `-100 <= Node.val <= 100`

SOLUTION (PYTHON 3)

Solution 1: Enumeration + DFS

How to think about it

The diameter is the longest number of edges between any two nodes. Pairwise paths are unrealistic. Any diameter passes through some node as the sum of that node's two subtree heights. DFS returns height and, on the way up, updates a global answer with l+r, then returns 1+max(l,r) to the parent. Each node is visited once.

We can enumerate each node of the binary tree, and for each node, calculate the maximum depth of its left and right subtrees, l and r, respectively. The diameter of the node is l + r. The maximum diameter among all nodes is the diameter of the binary tree.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def diameterOfBinaryTree(self, root: TreeNode) -> int:
        def dfs(root):
            if root is None:
                return 0
            nonlocal ans
            left, right = dfs(root.left), dfs(root.right)
            ans = max(ans, left + right)
            return 1 + max(left, right)

        ans = 0
        dfs(root)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the nodes of the longest path, not only its
# length
# 2. The diameter of an N-ary tree


# ============================================================
# FOLLOW-UP 1: Return the nodes of the longest path, not
# only its length
# ============================================================

# ---- follow-up 1: function ----
# Answer: Each call returns the longest downward path (as
# a list of nodes) from its node; the best path through a
# node joins its two best child paths.
# Time: O(n * h) (lists are joined at every node).  Space:
# O(h).
def diameter_path(root):
    best = [[]]

    def longest_down(node):
        if node is None:
            return []
        left = longest_down(node.left)
        right = longest_down(node.right)
        through = left[::-1] + [node.val] + right
        if len(through) > len(best[0]):
            best[0] = through
        if len(left) > len(right):
            return [node.val] + left
        return [node.val] + right

    longest_down(root)
    return best[0]


# ============================================================
```

```
# FOLLOW-UP 2: The diameter of an N-ary tree
# ============================================================

# ---- follow-up 2: function ----
# Answer: At each node take the two longest child depths:
# the longest path through the node is their sum.
# Time: O(n).  Space: O(h).
def diameter_nary(root):
    best = [0]

    def depth(node):
        top1 = top2 = 0
        for child in node.children:
            d = depth(child) + 1
            if d > top1:
                top2 = top1
                top1 = d
            elif d > top2:
                top2 = d
        if top1 + top2 > best[0]:
            best[0] = top1 + top2
        return top1
```

```
    if root:
        depth(root)
    return best[0]
```

110. Balanced Binary Tree

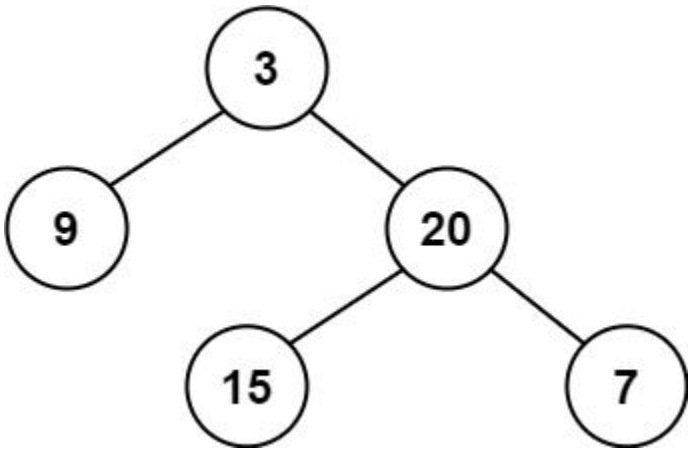
Easy · Tree, Depth-First Search, Binary Tree

leetcode.com/problems/balanced-binary-tree

PROBLEM

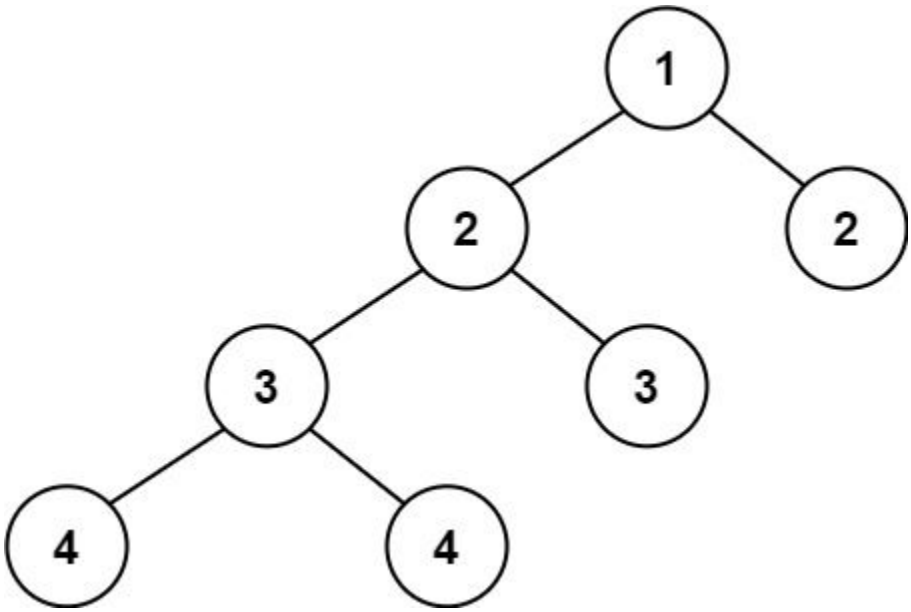
Given a binary tree, determine if it is **height-balanced**.

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: true
```

Example 2:




```
Input: root = [1,2,2,3,3,null,null,4,4]
Output: false
```

Example 3:

```
Input: root = []
Output: true
```

Constraints:

- The number of nodes in the tree is in the range `[0, 5000]`.
- -10^4 <= Node.val <= 10^4

SOLUTION (PYTHON 3)

Solution 1: Bottom-Up Recursion

How to think about it

Computing left and right heights from scratch at every node revisits the same subtrees and can hit $O(n^2)$; n ≤ 5000 is tight. Balance can be decided while computing height. A bottom-up pass returns the height, or a failure sentinel if a subtree is already unbalanced. One postorder walk answers both questions.

We define a function height(root) to calculate the height of a binary tree, with the following logic:

- If the binary tree root is null, return 0.
- Otherwise, recursively calculate the heights of the left and right subtrees, denoted as l and r respectively. If either l or r is -1, or the absolute difference between l and r is greater than 1, then return -1. Otherwise, return max(l, r) + 1.

Therefore, if the function height(root) returns -1, it means the binary tree root is not balanced. Otherwise, it is balanced.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        def height(root):
            if root is None:
                return 0
            l, r = height(root.left), height(root.right)
            if l == -1 or r == -1 or abs(l - r) > 1:
                return -1
            return 1 + max(l, r)

        return height(root) >= 0
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the balance factor (left height - right
# height) of every node
# 2. Find the deepest node whose subtree is unbalanced


# ========================================================
# FOLLOW-UP 1: Return the balance factor (left height -
# right height) of every node
# ========================================================

# ---- follow-up 1: function ----
# Answer: A post-order pass that returns the height and
# records the difference for each node.
# Time: O(n).  Space: O(h).
def balance_factors(root):
    factors = {}

    def height(node):
        if node is None:
```

```
            return 0
        left = height(node.left)
        right = height(node.right)
        factors[node.val] = left - right
        return 1 + max_of(left, right)

    height(root)
    return factors


def max_of(a, b):
    if a > b:
        return a
    return b


# ============================================================
# FOLLOW-UP 2: Find the deepest node whose subtree is
# unbalanced
# ============================================================

# ---- follow-up 2: function ----
# Answer: Return heights from the bottom up; the first
# node (in post-order) whose children's heights differ by
# more than 1 is a deepest unbalanced node.
# Time: O(n).  Space: O(h).
def deepest_unbalanced(root):
    found = [None]

    def height(node):
        if node is None:
            return 0
        left = height(node.left)
        right = height(node.right)
        if found[0] is None and abs(left - right) > 1:
            found[0] = node
        if left > right:
            return left + 1
        return right + 1
```

```
    height(root)
    return found[0]
```

100. Same Tree

Easy · Tree, Depth-First Search, Breadth-First Search, Binary Tree

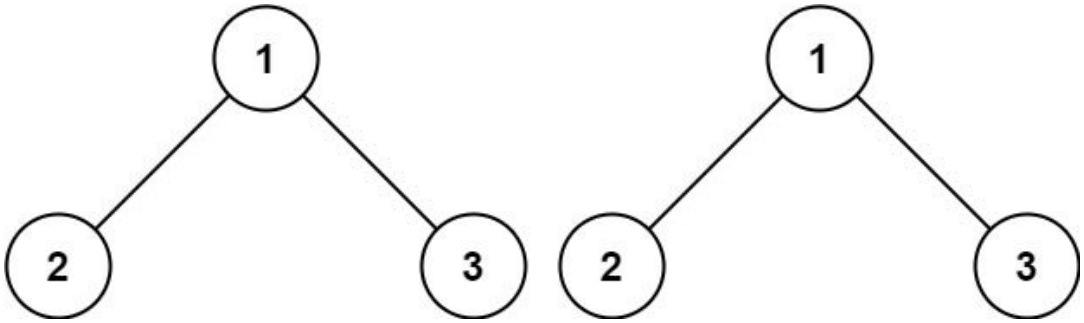
leetcode.com/problems/same-tree

PROBLEM

Given the roots of two binary trees `p` and `q`, write a function to check if they are the same or not.

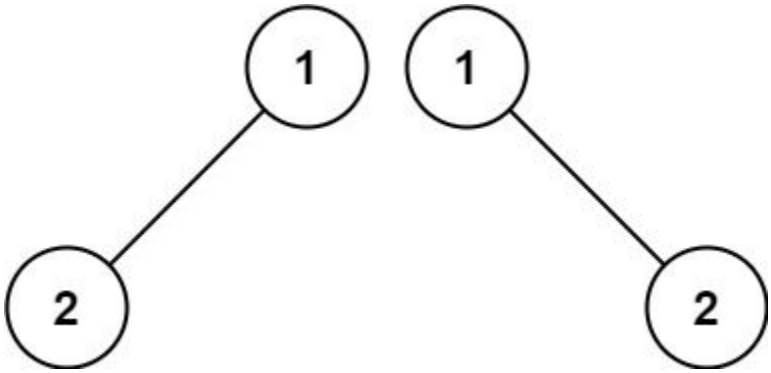
Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Example 1:



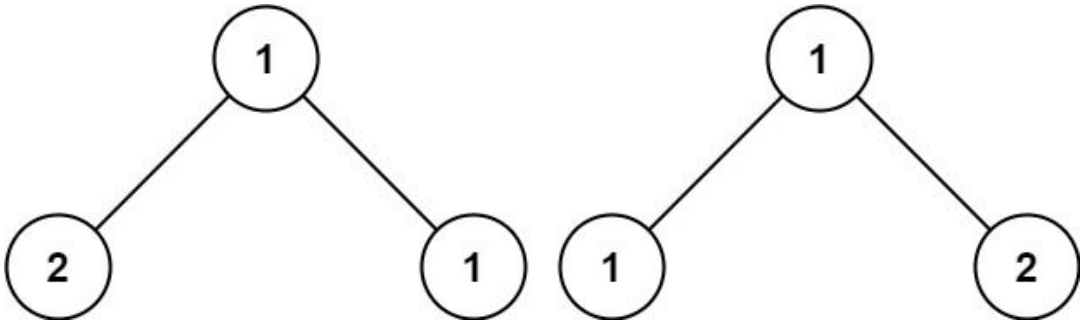

```
Input: p = [1,2,3], q = [1,2,3]
Output: true
```

Example 2:




```
Input: p = [1,2], q = [1,null,2]
Output: false
```

Example 3:




```
Input: p = [1,2,1], q = [1,1,2]
Output: false
```

Constraints:

- The number of nodes in both trees is in the range `[0, 100]`.
- -10^4 `<= Node.val <=` 10^4

SOLUTIONS (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> Whether two trees are the same depends on matching structure and values at every corresponding position. Collecting both traversals and comparing sequences still needs a sentinel encoding for missing children. n ≤ 100, so recursion is fine. A tree is defined by its root and two subtrees, so we only compare the current pair: both null means equal; one null or unequal values means not; otherwise recurse on the left children and the right children.

We can use the DFS recursive method to solve this problem.

First, determine whether the root nodes of the two binary trees are the same. If both root nodes are null, then the two binary trees are the same. If only one of the root nodes is null, then the two binary trees are definitely different. If both root nodes are not null, then determine whether their values are the same. If they are not the same, then the two binary trees are definitely different. If they are the same, then determine whether the left subtrees of the two binary trees are the same and whether the right subtrees are the same. The two binary trees are the same only when all the above conditions are met.

The time complexity is O(min(m, n)), and the space complexity is O(min(m, n)). Here, m and n are the number of nodes in the two binary trees, respectively. The space complexity mainly depends on the number of layers of recursive calls, which will not exceed the number of nodes in the smaller binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSameTree(self, p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
        if p == q:
            return True
        if p is None or q is None or p.val != q.val:
            return False
        return self.isSameTree(p.left, q.left) and self.isSameTree(p.right, q.right)
```

Solution 2: BFS

> **How to think about it**
>
> Solution 1 already produces the answer, at the cost of the call stack. An iterative version must visit both trees in lockstep. A pair of BFS queues can dequeue nodes together, check values and child presence, then enqueue matching children, so we no longer recurse.

We can also use the BFS iterative method to solve this problem.

First, add the root nodes of the two binary trees to two queues. Each time, take out one node from each of the two queues and perform the following comparison operations. If the values of the two nodes are not the same, then the structures of the two binary trees are definitely different. If the values of the two nodes are the same, then determine whether the child nodes of the two nodes are null. If only the left child node of one node is null, then the structures of the two binary trees are definitely different. If only the right child node of one node is null, then the structures of the two binary trees are definitely different. If the structures of the left and right child nodes are the same, then add the left and right child nodes of the two nodes to the two queues respectively. For the next iteration, take out one node from each of the two queues for comparison. When both queues are empty at the same time, it means that we have compared all the nodes, and the structures of the two binary trees are completely the same.

The time complexity is O(min(m, n)), and the space complexity is O(min(m, n)). Here, m and n are the number of nodes in the two binary trees, respectively. The space complexity mainly depends on the number of elements in the queue, which will not exceed the number of nodes in the smaller binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSameTree(self, p: TreeNode, q: TreeNode) -> bool:
        if p == q:
            return True
        if p is None or q is None:
            return False
        q1, q2 = deque([p]), deque([q])
        while q1 and q2:
            a, b = q1.popleft(), q2.popleft()
            if a.val != b.val:
                return False
            la, ra = a.left, a.right
            lb, rb = b.left, b.right
            if (la and not lb) or (lb and not la):
                return False
            if (ra and not rb) or (rb and not ra):
                return False
            if la:
                q1.append(la)
                q2.append(lb)
            if ra:
                q1.append(ra)
                q2.append(rb)
        return True
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Is one tree a SUBTREE of another (Subtree of Another
# Tree)?
# 2. Are the trees the same if left and right children may
# be swapped (flip equivalent)?


# ========================================================
# FOLLOW-UP 1: Is one tree a SUBTREE of another (Subtree
# of Another Tree)?
# ========================================================

# ---- follow-up 1: function ----
# Answer: Check 'same tree' at every node of the bigger
# tree.
# Time: O(n * m) (the same-tree check at every node).
# Space: O(h).
def is_same(a, b):
    if a is None and b is None:
        return True
    if a is None or b is None or a.val != b.val:
        return False
    return is_same(a.left, b.left) and is_same(a.right, b.right)


def is_subtree(root, sub):
    if sub is None:
        return True
```

```
    if root is None:
        return False
    return is_same(root, sub) or is_subtree(root.left, sub) or is_subtree(root.right, sub)


# ========================================================
# FOLLOW-UP 2: Are the trees the same if left and right
# children may be swapped (flip equivalent)?
# ========================================================

# ---- follow-up 2: function ----
# Answer: At each pair of nodes, the children must match
# either straight or crossed.
# Time: O(n) for distinct values (it can grow when many
# values are equal).  Space: O(h).
def flip_equivalent(a, b):
    if a is None and b is None:
        return True
    if a is None or b is None or a.val != b.val:
        return False
    straight = flip_equivalent(a.left, b.left) and flip_equivalent(a.right, b.right)
    crossed = flip_equivalent(a.left, b.right) and flip_equivalent(a.right, b.left)
    return straight or crossed
```

101. Symmetric Tree

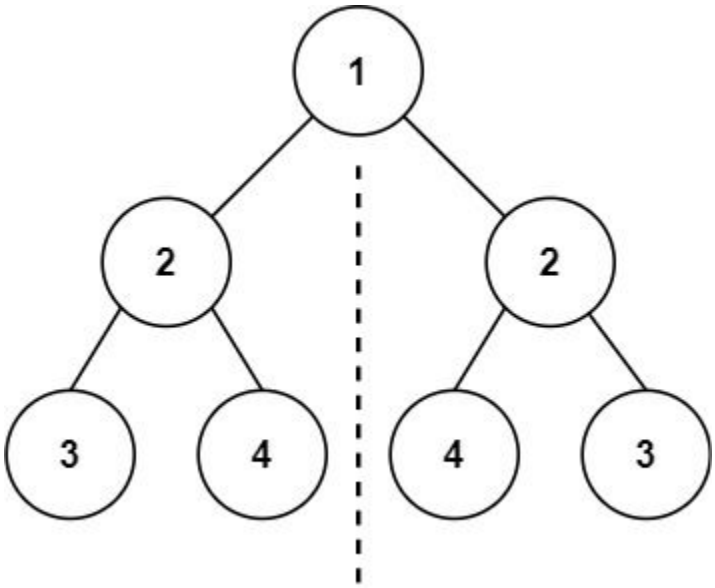
Easy · Tree, Depth-First Search, Breadth-First Search, Binary Tree

leetcode.com/problems/symmetric-tree

PROBLEM

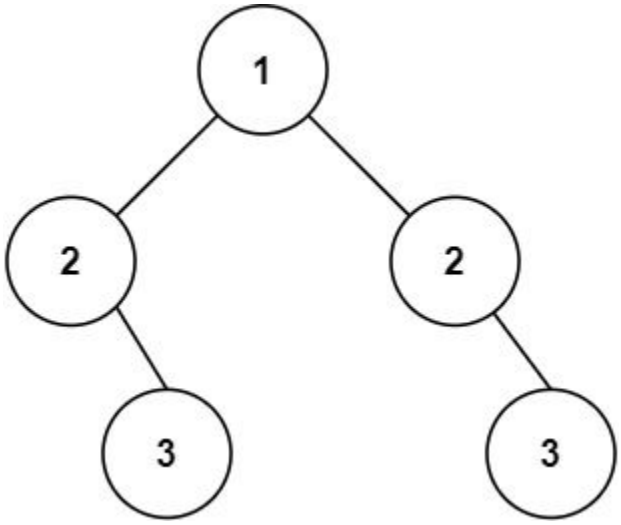
Given the `root` of a binary tree, *check whether it is a mirror of itself* (i.e., symmetric around its center).

Example 1:




```
Input: root = [1,2,2,3,4,4,3]
Output: true
```

Example 2:




```
Input: root = [1,2,2,null,3,null,3]
Output: false
```

Constraints:

- The number of nodes in the tree is in the range `[1, 1000]`.
- `-100 <= Node.val <= 100`

Follow up:

Could you solve it both recursively and iteratively?

SOLUTION (PYTHON 3)

Solution 1: Recursion

> **How to think about it**
>
> Symmetry means the left subtree mirrors the right, not that each subtree matches itself. The follow-up asks for both recursion and iteration; n ≤ 1000, so one pass is enough. A pair of nodes is symmetric iff their values match and the left child of one mirrors the right child of the other (and vice versa). We therefore start from the root's two children and recurse crosswise.

We design a function dfs(root1, root2) to determine whether two binary trees are symmetric. The answer is dfs(root.left, root.right).

The logic of the function dfs(root1, root2) is as follows:

- If both root1 and root2 are null, the two binary trees are symmetric, and we return `true`;
- If only one of root1 and root2 is null, or root1.val ≠ root2.val, we return `false`;
- Otherwise, we check whether the left subtree of root1 is symmetric with the right subtree of root2, and whether the right subtree of root1 is symmetric with the left subtree of root2, using recursion.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSymmetric(self, root: Optional[TreeNode]) -> bool:
        def dfs(root1: Optional[TreeNode], root2: Optional[TreeNode]) -> bool:
            if root1 == root2:
                return True
            if root1 is None or root2 is None or root1.val != root2.val:
                return False
            return dfs(root1.left, root2.right) and dfs(root1.right, root2.left)

        return dfs(root.left, root.right)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Iterative version with a queue
# 2. Is one tree the MIRROR IMAGE of another tree?


# ============================================================
# FOLLOW-UP 1: Iterative version with a queue
# ============================================================

# ---- follow-up 1: function ----
# Answer: Put pairs of nodes that must mirror each other
# in a queue; each step compares one pair and enqueues the
# pairs (left.left, right.right) and (left.right,
# right.left).
# Time: O(n).  Space: O(n).
from collections import deque
```

```
def is_symmetric_iterative(root):
    if root is None:
        return True
    queue = deque([(root.left, root.right)])
    while queue:
        a, b = queue.popleft()
        if a is None and b is None:
            continue
        if a is None or b is None or a.val != b.val:
            return False
        queue.append((a.left, b.right))
        queue.append((a.right, b.left))
    return True


# ============================================================
# FOLLOW-UP 2: Is one tree the MIRROR IMAGE of another
# tree?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Compare the first tree with the second one while
# crossing their children.
# Time: O(n).  Space: O(h).
def is_mirror(a, b):
    if a is None and b is None:
        return True
    if a is None or b is None or a.val != b.val:
        return False
    return is_mirror(a.left, b.right) and is_mirror(a.right, b.left)
```

226. Invert Binary Tree

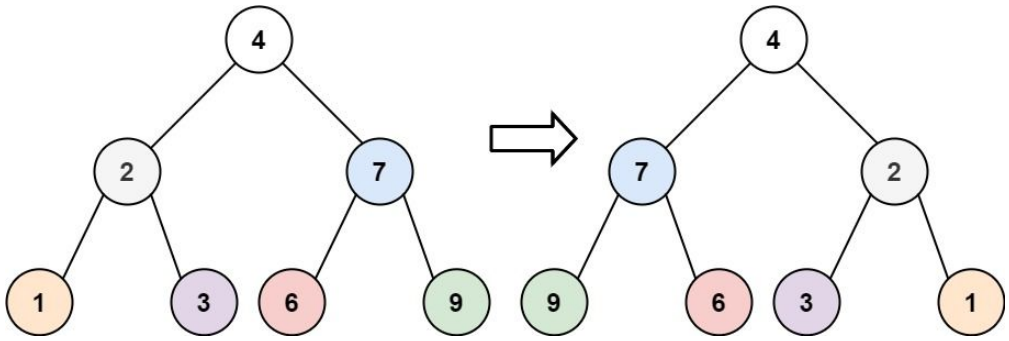
Easy · Tree, Depth-First Search, Breadth-First Search, Binary Tree

leetcode.com/problems/invert-binary-tree

PROBLEM

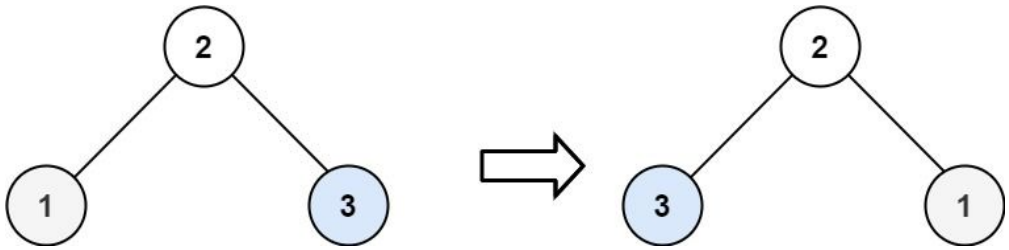
Given the `root` of a binary tree, invert the tree, and return *its root*.

Example 1:




```
Input: root = [4,2,7,1,3,6,9]
Output: [4,7,2,9,6,3,1]
```

Example 2:




```
Input: root = [2,1,3]
Output: [2,3,1]
```

Example 3:

```
Input: root = []
Output: []
```

Constraints:

- The number of nodes in the tree is in the range `[0, 100]`.
- `-100 <= Node.val <= 100`

SOLUTION (PYTHON 3)

Solution 1: Recursion

> **How to think about it**
>
> Inverting a tree means swapping every node's children and doing the same in both subtrees. One recursive walk is enough. Invert the two subtrees first, then swap the child pointers.

First, we check if root is null. If it is, we return null. Then, we recursively invert the left and right subtrees, set the inverted right subtree as the new left subtree, and set the inverted left subtree as the new right subtree. Finally, we return root.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        if root is None:
            return None
        l, r = self.invertTree(root.left), self.invertTree(root.right)
        root.left, root.right = r, l
        return root
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Iterative version (no recursion)
# 2. Return an inverted COPY and leave the original tree
# unchanged


# ============================================================
# FOLLOW-UP 1: Iterative version (no recursion)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Process nodes with a stack (or queue); swap the
# two children of every node you take out.
# Time: O(n).  Space: O(n).
def invert_iterative(root):
    stack = [root] if root else []
    while stack:
        node = stack.pop()
        node.left, node.right = node.right, node.left
        if node.left:
            stack.append(node.left)
        if node.right:
            stack.append(node.right)
    return root


# ============================================================
# FOLLOW-UP 2: Return an inverted COPY and leave the
# original tree unchanged
# ============================================================

# ---- follow-up 2: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 2: function ----
# Answer: Build new nodes: the copy's left child is the
# inverted copy of the original right child, and vice
# versa.
# Time: O(n).  Space: O(n) for the copy.
def inverted_copy(root):
    if root is None:
        return None
    return TreeNode(root.val, inverted_copy(root.right), inverted_copy(root.left))
```

PATTERN 11 · TREE DFS (PREORDER / INORDER / POSTORDER) · PROBLEM 11 OF 20

113. Path Sum II

Medium · Tree, Depth-First Search, Backtracking, Binary Tree

leetcode.com/problems/path-sum-ii

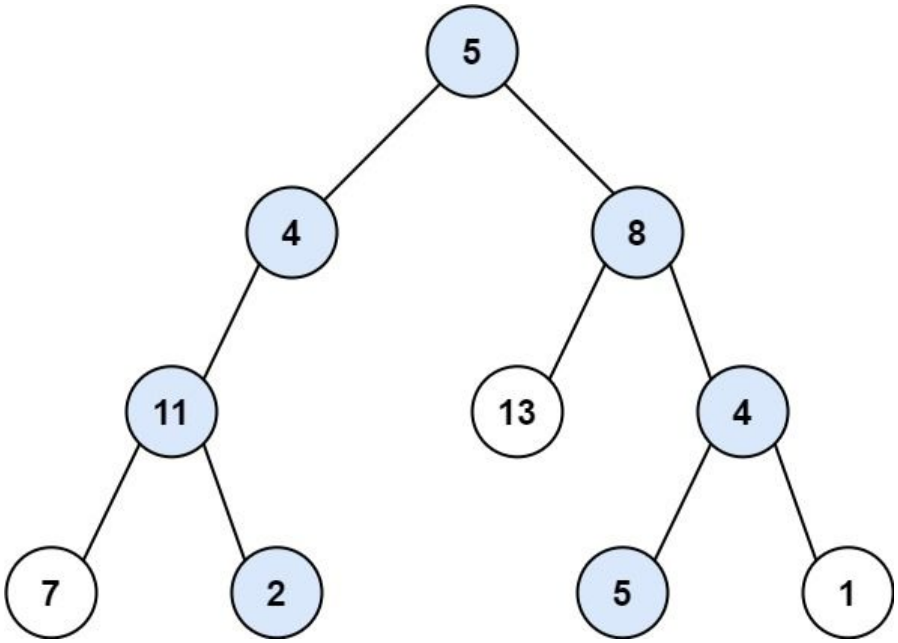
★ **TOP 7 PROBLEM** in Tree DFS (Preorder / Inorder / Postorder): master this one first

PROBLEM

Given the `root` of a binary tree and an integer `targetSum`, return *all **root-to-leaf** paths where the sum of the node values in the path equals* `targetSum`*. Each path should be returned as a list of the node **values**, not node references.*

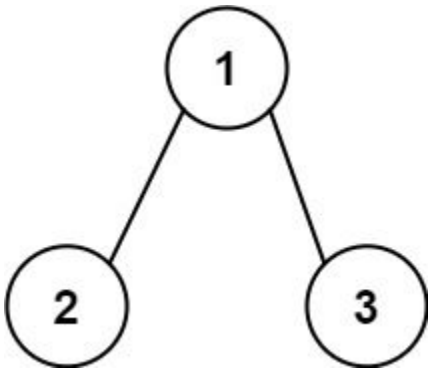
A **root-to-leaf** path is a path starting from the root and ending at any leaf node. A **leaf** is a node with no children.

Example 1:




```
Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22
Output: [[5,4,11,2],[5,8,4,5]]
Explanation: There are two paths whose sum equals targetSum:
5 + 4 + 11 + 2 = 22
5 + 8 + 4 + 5 = 22
```

Example 2:




```
Input: root = [1,2,3], targetSum = 5
Output: []
```

Example 3:

```
Input: root = [1,2], targetSum = 0
Output: []
```

Constraints:

- The number of nodes in the tree is in the range `[0, 5000]`.
- `-1000 <= Node.val <= 1000`
- `-1000 <= targetSum <= 1000`

SOLUTION (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> Unlike Path Sum, we must list every root-to-leaf path that sums to targetSum. n ≤ 5000; copying every path into the answer can cost O(n^2). DFS pushes the node onto a shared path and adds its value. At a matching leaf we copy the path; on the way back we pop, so both children reuse the same buffer.

We start from the root node, recursively traverse all paths from the root node to the leaf nodes, and record the path sum. When we traverse to a leaf node, if the current path sum equals `targetSum`, then we add this path to the answer.

The time complexity is O(n^2), where n is the number of nodes in the binary tree. The space complexity is O(n).

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def pathSum(self, root: Optional[TreeNode], targetSum: int) -> List[List[int]]:
        def dfs(root, s):
            if root is None:
                return
            s += root.val
            t.append(root.val)
            if root.left is None and root.right is None and s == targetSum:
                ans.append(t[:])
            dfs(root.left, s)
            dfs(root.right, s)
            t.pop()

        ans = []
        t = []
        dfs(root, 0)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return only the root-to-leaf path with the LARGEST
# sum
# 2. Without recursion


# ========================================================
# FOLLOW-UP 1: Return only the root-to-leaf path with the
# LARGEST sum
# ========================================================
```

```
# ---- follow-up 1: function ----
# Answer: Pass the path down; at each leaf compare its sum
# with the best one found so far.
# Time: O(n * h).  Space: O(h).
def max_sum_path(root):
    best = [None, []]

    def walk(node, path, total):
        if node is None:
            return
        path.append(node.val)
        total += node.val
        if node.left is None and node.right is None:
            if best[0] is None or total > best[0]:
                best[0] = total
                best[1] = list(path)
        walk(node.left, path, total)
        walk(node.right, path, total)
        path.pop()

    walk(root, [], 0)
    return best[0], best[1]


# ============================================================
# FOLLOW-UP 2: Without recursion
# ============================================================

# ---- follow-up 2: function ----
# Answer: A stack of (node, path so far, sum so far); at a
# leaf check the sum, otherwise push the children with the
# extended path.
# Time: O(n * h).  Space: O(n * h).
def paths_with_sum_iterative(root, target):
    found = []
    stack = [(root, [root.val], root.val)] if root else []
    while stack:
        node, path, total = stack.pop()
        if node.left is None and node.right is None and total == target:
            found.append(path)
        if node.right:
            stack.append((node.right, path + [node.right.val], total + node.right.val))
        if node.left:
            stack.append((node.left, path + [node.left.val], total + node.left.val))
    return found
```

236. Lowest Common Ancestor of a Binary Tree

Medium · Tree, Depth-First Search, Binary Tree, Lowest Common Ancestor, Binary Lifting

leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree

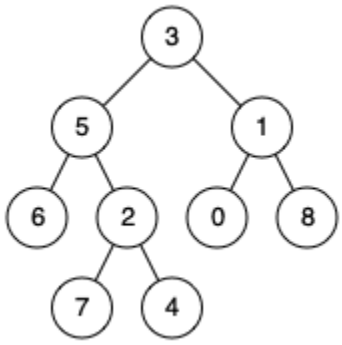
★ **TOP 7 PROBLEM** in Tree DFS (Preorder / Inorder / Postorder): master this one first

PROBLEM

Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree.

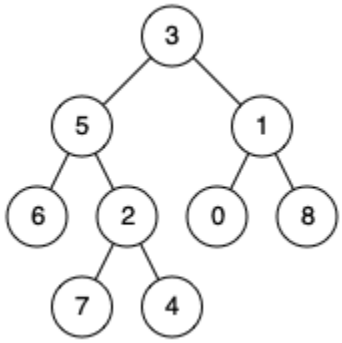
According to the definition of LCA on Wikipedia: "The lowest common ancestor is defined between two nodes `p` and `q` as the lowest node in `T` that has both `p` and `q` as descendants (where we allow **a node to be a descendant of itself**)."

Example 1:




```
Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
Output: 3
Explanation: The LCA of nodes 5 and 1 is 3.
```

Example 2:




```
Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4
Output: 5
Explanation: The LCA of nodes 5 and 4 is 5, since a node can be a descendant of itself according to the
LCA definition.
```

Example 3:

```
Input: root = [1,2], p = 1, q = 2
Output: 1
```

Constraints:

- The number of nodes in the tree is in the range `[2, 10^5]`.

- -10^9 <= Node.val <= 10^9
- All Node.val are **unique**.
- p != q
- p and q will exist in the tree.

SOLUTION (PYTHON 3)

Solution 1: Recursion

> **How to think about it**
>
> A general binary tree has no key order, so we cannot walk by value. The LCA is the deepest node that covers both p and q. A postorder recursion returns a found node from a subtree. If both sides are non-empty the current root is the LCA; otherwise we pass up the non-empty side.

We recursively traverse the binary tree:

If the current node is null or equals to p or q, then we return the current node;

Otherwise, we recursively traverse the left and right subtrees, and record the returned results as left and right. If both left and right are not null, it means that p and q are in the left and right subtrees respectively, so the current node is the nearest common ancestor; If only one of left and right is not null, we return the one that is not null.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None


class Solution:
    def lowestCommonAncestor(
        self, root: "TreeNode", p: "TreeNode", q: "TreeNode"
    ) -> "TreeNode":
        if root in (None, p, q):
            return root
        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)
        return root if left and right else (left or right)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. p or q may NOT be in the tree: return None then
# 2. The lowest common ancestor of SEVERAL nodes


# ============================================================
# FOLLOW-UP 1: p or q may NOT be in the tree: return None
# then
# ============================================================

# ---- follow-up 1: function ----
# Answer: The usual recursion returns a node as soon as it
# sees p or q, which can be wrong when the other one is
# missing. Count how many of the two were really found and
# only accept the answer when both were.
# Time: O(n).  Space: O(h).
def lca_if_both_exist(root, p, q):
```

```
    found = [0]

    def walk(node):
        if node is None:
            return None
        left = walk(node.left)
        right = walk(node.right)
        if node is p or node is q:
            found[0] += 1
            return node
        if left and right:
            return node
        return left or right

    answer = walk(root)
    if found[0] == 2:
        return answer
    return None


# ============================================================
# FOLLOW-UP 2: The lowest common ancestor of SEVERAL nodes
# ============================================================

# ---- follow-up 2: function ----
# Answer: Return a node when it is one of the targets, or
# when targets were found below it in two different
# children; with a set of targets the same recursion
# works.
# Time: O(n).  Space: O(h).
def lca_of_many(root, targets):
    def walk(node):
        if node is None:
            return None
        if node in targets:
            return node
        left = walk(node.left)
        right = walk(node.right)
        if left and right:
            return node
        return left or right

    return walk(root)
```

105. Construct Binary Tree from Preorder and Inorder Traversal

Medium · Tree, Array, Hash Table, Divide and Conquer, Binary Tree

leetcode.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal

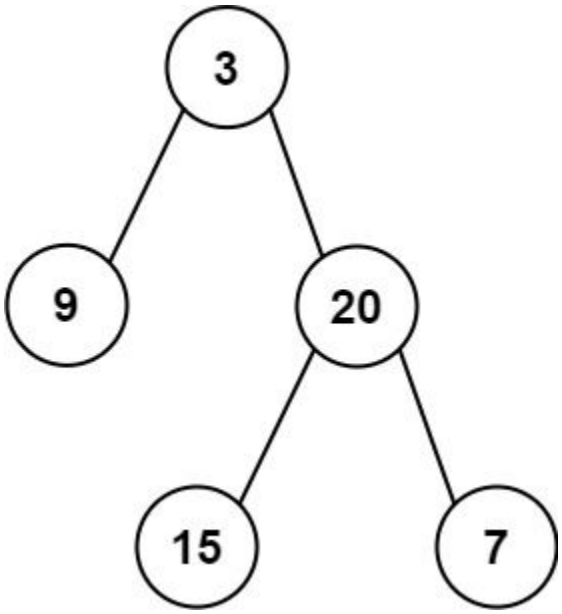
★ **TOP 7 PROBLEM** in Tree DFS (Preorder / Inorder / Postorder): master this one first

Also appears in pattern 14 Recursion / Divide and Conquer

PROBLEM

Given two integer arrays `preorder` and `inorder` where `preorder` is the preorder traversal of a binary tree and `inorder` is the inorder traversal of the same tree, construct and return *the binary tree*.

Example 1:




```
Input: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
Output: [3,9,20,null,null,15,7]
```

Example 2:

```
Input: preorder = [-1], inorder = [-1]
Output: [-1]
```

Constraints:

- `1 <= preorder.length <= 3000`
- `inorder.length == preorder.length`
- `-3000 <= preorder[i], inorder[i] <= 3000`
- `preorder` and `inorder` consist of **unique** values.
- Each value of `inorder` also appears in `preorder`.
- `preorder` is **guaranteed** to be the preorder traversal of the tree.
- `inorder` is **guaranteed** to be the inorder traversal of the tree.

SOLUTIONS (PYTHON 3)

Solution 1: Hash Table + Recursion

> **How to think about it**
>
> Preorder gives the root order; inorder splits each root into left and right subtrees. Scanning inorder for the root every time can cost $O(n^2)$; $n \le 3000$ is tight. Values are unique, so we map each inorder value to its index once. The first value of the current preorder slice is the root; everything left of it in inorder is the left subtree, the rest the right. Those lengths fix the next two recursive slices.

The first node preorder[0] in the pre-order sequence is the root node. We find the position k of the root node in the in-order sequence, which can divide the in-order sequence into the left subtree inorder[0..k] and the right subtree inorder[k+1..].

Through the intervals of the left and right subtrees, we can calculate the number of nodes in the left and right subtrees, assumed to be a and b. Then in the pre-order nodes, the a nodes after the root node are the left subtree, and the b nodes after that are the right subtree.

Therefore, we design a function dfs(i, j, n), where i and j represent the starting positions of the pre-order sequence and the in-order sequence, respectively, and n represents the number of nodes. The return value of the function is the binary tree constructed with preorder[i..i+n-1] as the pre-order sequence and inorder[j..j+n-1] as the in-order sequence.

The execution process of the function dfs(i, j, n) is as follows:

- If n ≤ 0, it means there are no nodes, return a null node.
- Take out the first node v = preorder[i] of the pre-order sequence as the root node, and then use the hash table d to find the position k of the root node in the in-order sequence. Then the number of nodes in the left subtree is k - j, and the number of nodes in the right subtree is n - k + j - 1.
- Recursively construct the left subtree l = dfs(i + 1, j, k - j) and the right subtree r = dfs(i + 1 + k - j, k + 1, n - k + j - 1).
- Finally, return the binary tree with v as the root node and l and r as the left and right subtrees, respectively.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        def dfs(i: int, j: int, n: int) -> Optional[TreeNode]:
            if n <= 0:
                return None
            v = preorder[i]
            k = d[v]
            l = dfs(i + 1, j, k - j)
            r = dfs(i + 1 + k - j, k + 1, n - k + j - 1)
            return TreeNode(v, l, r)

        d = {v: i for i, v in enumerate(inorder)}
        return dfs(0, 0, len(preorder))
```

Solution 2: Hash Table + Recursion (Duplicate Values)

> **How to think about it**
>
> Solution 1 needs unique values so one index locates the root. If values may repeat, a value can have several legal inorder positions. We store every index, try each one that falls in the current interval, recurse on both sides, and take the Cartesian product. This problem does not need that branch; it shows how to enumerate every feasible tree when duplicates are allowed.

This problem guarantees that node values are unique. If duplicates are allowed, the hash table should store every index of each value, and we enumerate every possible root position in the current range to build all valid binary trees.

The time and space complexities both depend on the number of valid trees (we build and store all of them) and are exponential in the worst case. The recursion stack uses O(n) extra space.

Python 3

```python
class Solution:
    def getBinaryTrees(self, preOrder: List[int], inOrder: List[int]) -> List[TreeNode]:
        def dfs(i: int, j: int, n: int) -> List[TreeNode]:
            if n <= 0:
                return [None]
            v = preOrder[i]
            ans = []
            for k in d[v]:
                if j <= k < j + n:
                    for l in dfs(i + 1, j, k - j):
                        for r in dfs(i + 1 + k - j, k + 1, n - 1 - (k - j)):
                            ans.append(TreeNode(v, l, r))
            return ans

        d = defaultdict(list)
        for i, x in enumerate(inOrder):
            d[x].append(i)
        return dfs(0, 0, len(preOrder))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Build the tree from INORDER and POSTORDER instead
# 2. Is a pair of sequences a valid preorder and inorder
# of ONE tree?


# ============================================================
# FOLLOW-UP 1: Build the tree from INORDER and POSTORDER
# instead
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: The last postorder value is the root; in
# inorder, everything left of it is the left subtree.
# Build the right subtree first, because postorder is read
# from the back (root, right, left).
# Time: O(n).  Space: O(n).
def build_from_inorder_postorder(inorder, postorder):
    position = {}
    for i in range(len(inorder)):
        position[inorder[i]] = i
    index = [len(postorder) - 1]

    def build(low, high):
        if low > high:
            return None
        value = postorder[index[0]]
        index[0] -= 1
        node = TreeNode(value)
        middle = position[value]
        node.right = build(middle + 1, high)
```

```
        node.left = build(low, middle - 1)
        return node

    return build(0, len(inorder) - 1)


# ============================================================
# FOLLOW-UP 2: Is a pair of sequences a valid preorder and
# inorder of ONE tree?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Try to build the tree; the sequences are valid
# exactly when the build never fails and uses every value
# once.
# Time: O(n log n).  Space: O(n).
def valid_preorder_inorder(preorder, inorder):
    if sorted(preorder) != sorted(inorder):
        return False
    position = {}
    for i in range(len(inorder)):
        position[inorder[i]] = i
```

```
    index = [0]

    def build(low, high):
        if low > high:
            return True
        if index[0] >= len(preorder):
            return False
        value = preorder[index[0]]
        index[0] += 1
        if value not in position or not (low <= position[value] <= high):
            return False
        middle = position[value]
        return build(low, middle - 1) and build(middle + 1, high)

    return build(0, len(inorder) - 1) and index[0] == len(preorder)
```

114. Flatten Binary Tree to Linked List

Medium · Stack, Tree, Depth-First Search, Linked List, Binary Tree

leetcode.com/problems/flatten-binary-tree-to-linked-list

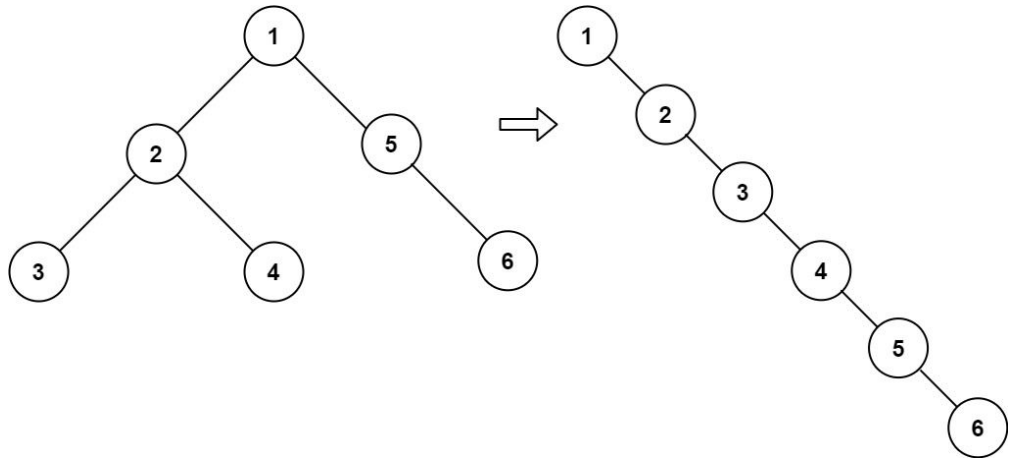
★ **TOP 7 PROBLEM** in Tree DFS (Preorder / Inorder / Postorder): master this one first

PROBLEM

Given the `root` of a binary tree, flatten the tree into a "linked list":

- The "linked list" should use the same `TreeNode` class where the `right` child pointer points to the next node in the list and the `left` child pointer is always `null`.
- The "linked list" should be in the same order as a **pre-order traversal** of the binary tree.

Example 1:




```
Input: root = [1,2,5,3,4,null,6]
Output: [1,null,2,null,3,null,4,null,5,null,6]
```

Example 2:

```
Input: root = []
Output: []
```

Example 3:

```
Input: root = [0]
Output: [0]
```

Constraints:

- The number of nodes in the tree is in the range `[0, 2000]`.
- `-100 <= Node.val <= 100`

Follow up:

Can you flatten the tree in-place (with

`O(1)`

extra space)?

SOLUTIONS (PYTHON 3)

Solution 1: Find Predecessor Node

> **How to think about it**
>
> A preorder flatten can build a new list with recursion or a stack, but the follow-up asks for O(1) extra space in place. n ≤ 2000. After the left subtree in preorder comes the original right subtree. The rightmost node of the left subtree is that predecessor: splice the old right onto it, move the left subtree to `right`, and clear `left`. Walk the right spine; no recursion stack is needed.

The visit order of preorder traversal is "root, left subtree, right subtree". After the last node of the left subtree is visited, the right subtree node of the root node will be visited next.

Therefore, for the current node, if its left child node is not null, we find the rightmost node of the left subtree as the predecessor node, and then assign the right child node of the current node to the right child node of the predecessor node. Then assign the left child node of the current node to the right child node of the current node, and set the left child node of the current node to null. Then take the right child node of the current node as the next node and continue processing until all nodes are processed.

The time complexity is O(n), where n is the number of nodes in the tree. The space complexity is O(1).

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def flatten(self, root: Optional[TreeNode]) -> None:
        """
        Do not return anything, modify root in-place instead.
        """
        while root:
            if root.left:
                pre = root.left
                while pre.right:
                    pre = pre.right
                pre.right = root.right
                root.right = root.left
                root.left = None
            root = root.right
```

Solution 2

> **How to think about it**
>
> Solution 1 already splices predecessors in O(1) space. This is the same idea with a different write-up: save both children, hang the left subtree on the right, attach the old right to its rightmost node, and step along `right`.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```
class Solution:
    def flatten(self, root: Optional[TreeNode]) -> None:
        while root:
            left, right = root.left, root.right
            root.left = None
            if left:
                root.right = left
                while left.right:
                    left = left.right
                left.right = right
            root = root.right
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Flatten with O(1) extra space and no recursion
# 2. Flatten into the INORDER order (a sorted list for a
# BST)


# ============================================================
# FOLLOW-UP 1: Flatten with O(1) extra space and no
# recursion
# ============================================================

# ---- follow-up 1: function ----
# Answer: For every node with a left child, find the
# rightmost node of the left subtree, hang the node's
# right subtree there, then move the left subtree to the
# right.
# Time: O(n).  Space: O(1).
def flatten_constant_space(root):
    node = root
    while node:
        if node.left:
            rightmost = node.left
            while rightmost.right:
                rightmost = rightmost.right
            rightmost.right = node.right
            node.right = node.left
            node.left = None
        node = node.right


# ============================================================
# FOLLOW-UP 2: Flatten into the INORDER order (a sorted
# list for a BST)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk the tree inorder and link every node to the
# next one through the right pointer.
# Time: O(n).  Space: O(h).
def flatten_inorder(root):
    dummy = TreeNodeLite()
    tail = [dummy]

    def walk(node):
        if node is None:
            return
        walk(node.left)
        node.left = None
        tail[0].right = node
        tail[0] = node
        walk(node.right)

    walk(root)
    return dummy.right


class TreeNodeLite:
    def __init__(self):
        self.right = None
```

129. Sum Root to Leaf Numbers

Medium · Tree, Depth-First Search, Binary Tree

leetcode.com/problems/sum-root-to-leaf-numbers

PROBLEM

You are given the `root` of a binary tree containing digits from `0` to `9` only.

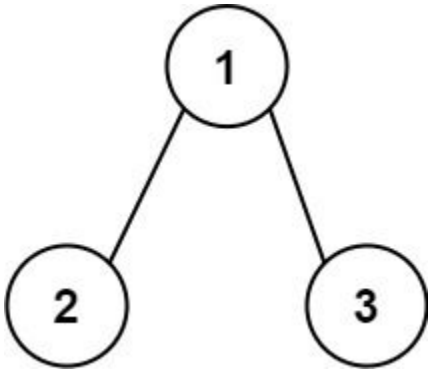
Each root-to-leaf path in the tree represents a number.

- For example, the root-to-leaf path `1 -> 2 -> 3` represents the number `123`.

Return *the total sum of all root-to-leaf numbers*. Test cases are generated so that the answer will fit in a **32-bit** integer.

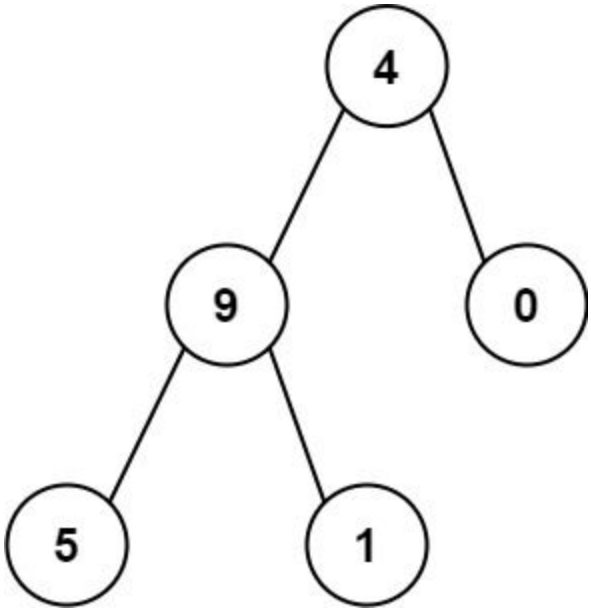
A **leaf** node is a node with no children.

Example 1:




```
Input: root = [1,2,3]
Output: 25
Explanation:
The root-to-leaf path 1->2 represents the number 12.
The root-to-leaf path 1->3 represents the number 13.
Therefore, sum = 12 + 13 = 25.
```

Example 2:



```
Input: root = [4,9,0,5,1]
Output: 1026
Explanation:
The root-to-leaf path 4->9->5 represents the number 495.
The root-to-leaf path 4->9->1 represents the number 491.
The root-to-leaf path 4->0 represents the number 40.
Therefore, sum = 495 + 491 + 40 = 1026.
```

Constraints:

- The number of nodes in the tree is in the range `[1, 1000]`.
- `0 <= Node.val <= 9`
- The depth of the tree will not exceed `10`.

SOLUTION (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> Each root-to-leaf path is an integer; we want their sum. Depth is at most 10, so there are few paths, but we need not build strings. Walking down, accumulate s × 10 + val. At a leaf that number is the contribution; add the two subtrees.

We can design a function dfs(root, s), which represents the sum of all path numbers from the current node root to the leaf nodes, given that the current path number is s. The answer is dfs(root, 0).

The calculation of the function dfs(root, s) is as follows:

- If the current node root is null, return 0.
- Otherwise, add the value of the current node to s, i.e., s = s × 10 + root.val.
- If the current node is a leaf node, return s.
- Otherwise, return dfs(root.left, s) + dfs(root.right, s).

The time complexity is O(n), and the space complexity is O(log n). Here, n is the number of nodes in the binary tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sumNumbers(self, root: Optional[TreeNode]) -> int:
        def dfs(root, s):
            if root is None:
                return 0
            s = s * 10 + root.val
            if root.left is None and root.right is None:
                return s
            return dfs(root.left, s) + dfs(root.right, s)

        return dfs(root, 0)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Add up the numbers formed by every root-to-NODE path
# (not only to leaves)
# 2. Without recursion


# ============================================================
# FOLLOW-UP 1: Add up the numbers formed by every root-to-
# NODE path (not only to leaves)
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same running number; add it at every node
# instead of only at leaves.
# Time: O(n).  Space: O(h).
def sum_all_node_paths(root):
    def walk(node, current):
        if node is None:
            return 0
        current = current * 10 + node.val
        return current + walk(node.left, current) + walk(node.right, current)

    return walk(root, 0)


# ============================================================
# FOLLOW-UP 2: Without recursion
# ============================================================

# ---- follow-up 2: function ----
# Answer: A stack of (node, number so far); at a leaf add
# the number to the total.
# Time: O(n).  Space: O(h).
def sum_numbers_iterative(root):
    total = 0
    stack = [(root, 0)] if root else []
    while stack:
        node, current = stack.pop()
        current = current * 10 + node.val
        if node.left is None and node.right is None:
            total += current
        if node.right:
            stack.append((node.right, current))
        if node.left:
            stack.append((node.left, current))
    return total
```

1448. Count Good Nodes in Binary Tree

Medium · Tree, Depth-First Search, Breadth-First Search, Binary Tree

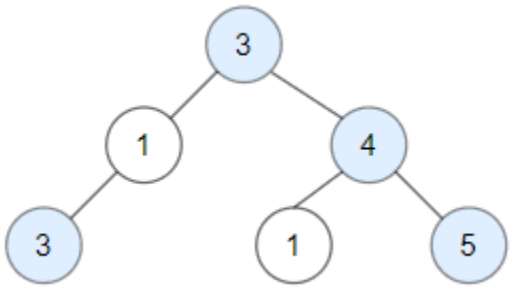
leetcode.com/problems/count-good-nodes-in-binary-tree

PROBLEM

Given a binary tree `root`, a node *X* in the tree is named **good** if in the path from root to *X* there are no nodes with a value *greater than* X.

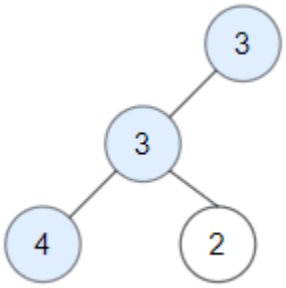
Return the number of **good** nodes in the binary tree.

Example 1:




```
Input: root = [3,1,4,3,null,1,5]
Output: 4
Explanation: Nodes in blue are good.
Root Node (3) is always a good node.
Node 4 -> (3,4) is the maximum value in the path starting from the root.
Node 5 -> (3,4,5) is the maximum value in the path
Node 3 -> (3,1,3) is the maximum value in the path.
```

Example 2:




```
Input: root = [3,3,null,4,2]
Output: 3
Explanation: Node 2 -> (3, 3, 2) is not good, because "3" is higher than it.
```

Example 3:

```
Input: root = [1]
Output: 1
Explanation: Root is considered as good.
```

Constraints:

- The number of nodes in the binary tree is in the range `[1, 10^5]`.
- Each node's value is between `[-10^4, 10^4]`.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

A node is good if nothing larger appears on the path from the root. n≤ 10^5. DFS carries the path maximum mx; count the node when its value is at least mx, then update mx.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def goodNodes(self, root: TreeNode) -> int:
        def dfs(root: TreeNode, mx: int):
            if root is None:
                return
            nonlocal ans
            if mx <= root.val:
                ans += 1
                mx = root.val
            dfs(root.left, mx)
            dfs(root.right, mx)

        ans = 0
        dfs(root, -1000000)
        return ans
```

Complexity. Time: O(n). Space: O(h), h = height of the tree.

Solution 2: Explicit Stack

How to think about it

A node is good when no larger value appears on the path from the root. With n up to 10^5, one walk that carries the path maximum mx is enough: count the node when its value is at least mx, then update mx. The walk enters the left child first. A left chain can be n nodes long, so recursion overflows before the count is finished. Each node is compared only with the maximum on its own path, so the two children do not depend on each other. The nodes still to visit fit on an explicit stack together with the mx from above. After a pop, a value at least mx is counted and mx becomes that value; the right child is pushed, then the left child, both with this mx, so the left child is handled first. The initial mx is below the lower bound -10^4, so the root is always good. Each node is pushed once.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def goodNodes(self, root: TreeNode) -> int:
        ans = 0
        stk = [(root, -1000000)]
        while stk:
            node, mx = stk.pop()
            if node is None:
                continue
            if mx <= node.val:
                ans += 1
                mx = node.val
            stk.append((node.right, mx))
            stk.append((node.left, mx))
        return ans
```

Complexity. Time: O(n). Space: O(h).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the good nodes (their values), not only how
# many
# 2. A node is good only when it is STRICTLY greater than
# every node before it on the path


# ============================================================
# FOLLOW-UP 1: Return the good nodes (their values), not
# only how many
# ============================================================

# ---- follow-up 1: function ----
# Answer: Carry the maximum on the path from the root; a
# node is good when its value is at least that maximum.
# Time: O(n).  Space: O(h) plus the output.
def good_node_values(root):
    good = []

    def walk(node, biggest):
        if node is None:
            return
        if node.val >= biggest:
            good.append(node.val)
            biggest = node.val
        walk(node.left, biggest)
        walk(node.right, biggest)

    if root:
        walk(root, root.val)
    return good


# ============================================================
# FOLLOW-UP 2: A node is good only when it is STRICTLY
# greater than every node before it on the path
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same walk, but the comparison is strict (the
# root is always good).
```

```python
# Time: O(n).  Space: O(h).
def count_strictly_good(root):
    def walk(node, biggest):
        if node is None:
            return 0
        count = 0
        if node.val > biggest:
            count = 1
            biggest = node.val
        return count + walk(node.left, biggest) + walk(node.right, biggest)

    if root is None:
        return 0
    return 1 + walk(root.left, root.val) + walk(root.right, root.val)
```

814. Binary Tree Pruning

Medium · Tree, Depth-First Search, Binary Tree

leetcode.com/problems/binary-tree-pruning

PROBLEM

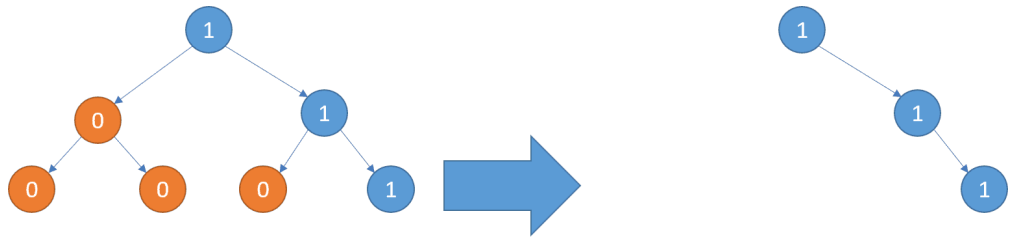
Given the `root` of a binary tree, return *the same tree where every subtree (of the given tree) not containing a* `1` *has been removed*.

A subtree of a node `node` is `node` plus every node that is a descendant of `node`.

Example 1:

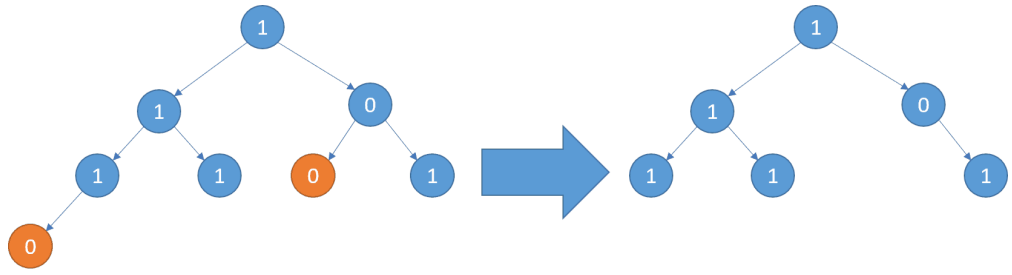
```
Input: root = [1,null,0,0,1]
Output: [1,null,0,null,1]
Explanation:
Only the red nodes satisfy the property "every subtree not containing a 1".
The diagram on the right represents the answer.
```

Example 2:




```
Input: root = [1,0,1,0,0,0,1]
Output: [1,null,1,null,1]
```

Example 3:




```
Input: root = [1,1,0,1,1,0,1,0]
Output: [1,1,0,1,1,null,1]
```

Constraints:

- The number of nodes in the tree is in the range `[1, 200]`.
- `Node.val` is either `0` or `1`.

SOLUTION (PYTHON 3)

Solution 1: Recursion

> **How to think about it**
>
> Every subtree that contains no 1 must be removed. A preorder walk cannot decide before seeing the children, so we prune left and right first, then drop the node if it became empty. After the recursive calls, a 0 with both children gone is deleted. The tree has at most 200 nodes, so one DFS suffices.

First, we check if the current node is null. If it is, we directly return the null node.

Otherwise, we recursively prune the left and right subtrees and reassign the pruned subtrees to the current node's left and right children. Then, we check if the current node's value is 0 and both its left and right children are null. If so, we return the null node; otherwise, we return the current node.

Time complexity is O(n), and space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def pruneTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        if root is None:
            return root
        root.left = self.pruneTree(root.left)
        root.right = self.pruneTree(root.right)
        if root.val == 0 and root.left == root.right:
            return None
        return root
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Remove every subtree whose values add up to zero
# 2. Keep only the nodes that lie on a path to a given
# value


# ============================================================
# FOLLOW-UP 1: Remove every subtree whose values add up to
# zero
# ============================================================

# ---- follow-up 1: function ----
# Answer: Return the sum of each subtree from the bottom
# up; a node is removed when its whole subtree sums to
# zero (after its children were pruned).
# Time: O(n).  Space: O(h).
def prune_zero_sum(root):
    def walk(node):
        # returns (the pruned node or None, the sum of the pruned subtree)
        if node is None:
            return None, 0
        node.left, left_sum = walk(node.left)
        node.right, right_sum = walk(node.right)
        total = node.val + left_sum + right_sum
        if total == 0:
            return None, 0
        return node, total
```

```
    return walk(root)[0]


# ============================================================
# FOLLOW-UP 2: Keep only the nodes that lie on a path to a
# given value
# ============================================================

# ---- follow-up 2: function ----
# Answer: A node stays when it holds the value or any of
# its subtrees keeps a node.
# Time: O(n).  Space: O(h).
def keep_paths_to_value(root, wanted):
    if root is None:
        return None
    root.left = keep_paths_to_value(root.left, wanted)
    root.right = keep_paths_to_value(root.right, wanted)
    if root.val == wanted or root.left or root.right:
        return root
    return None
```

863. All Nodes Distance K in Binary Tree

Medium · Tree, Depth-First Search, Breadth-First Search, Hash Table, Binary Tree

leetcode.com/problems/all-nodes-distance-k-in-binary-tree

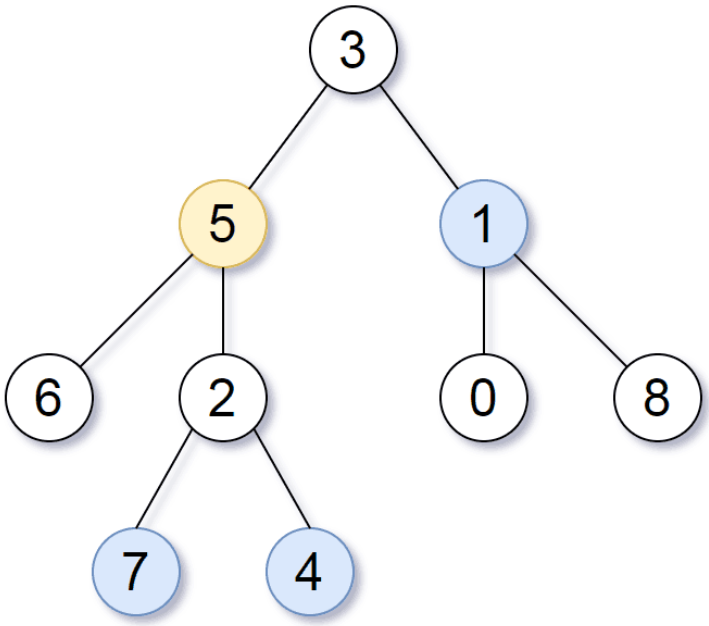
★ **TOP 7 PROBLEM** in Tree DFS (Preorder / Inorder / Postorder): master this one first

PROBLEM

Given the `root` of a binary tree, the value of a target node `target`, and an integer `k`, return *an array of the values of all nodes that have a distance* `k` *from the target node.*

You can return the answer in **any order**.

Example 1:




```
Input: root = [3,5,1,6,2,0,8,null,null,7,4], target = 5, k = 2
Output: [7,4,1]
Explanation: The nodes that are a distance 2 from the target node (with value 5) have values 7, 4, and
1.
```

Example 2:

```
Input: root = [1], target = 1, k = 3
Output: []
```

Constraints:

- The number of nodes in the tree is in the range `[1, 500]`.
- `0 <= Node.val <= 500`
- All the values `Node.val` are **unique**.
- `target` is the value of one of the nodes in the tree.
- `0 <= k <= 1000`

SOLUTION (PYTHON 3)

Solution 1: DFS + Hash Table

> **How to think about it**
>
> We want nodes at distance k from target. Child pointers alone cannot walk to a parent. With at most 500 nodes, record parents first, then walk k steps from target. The first DFS stores each parent; the second explores left, right, and parent, skipping the previous node, and collects values when the remaining distance is 0.

We first use DFS to traverse the entire tree and save each node's parent node in the hash table g.

Next, we use DFS again, starting from target, to search for nodes at a distance of k both upwards and downwards, and add them to the result array.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None


class Solution:
    def distanceK(self, root: TreeNode, target: TreeNode, k: int) -> List[int]:
        def dfs(root, fa):
            if root is None:
                return
            g[root] = fa
            dfs(root.left, root)
            dfs(root.right, root)

        def dfs2(root, fa, k):
            if root is None:
                return
            if k == 0:
                ans.append(root.val)
                return
            for nxt in (root.left, root.right, g[root]):
                if nxt != fa:
                    dfs2(nxt, root, k - 1)

        g = {}
        dfs(root, None)
        ans = []
        dfs2(target, None, k)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. All nodes at distance AT MOST k from the target
# 2. The distance between two nodes of the tree


# ============================================================
# FOLLOW-UP 1: All nodes at distance AT MOST k from the
# target
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same graph walk (parent links plus
```

```
# children), but collect every node reached within k steps
# instead of only those at exactly k.
# Time: O(n).  Space: O(n).
from collections import deque


def nodes_within_distance(root, target, k):
    parent = {root: None}
    stack = [root]
    while stack:
        node = stack.pop()
        for child in (node.left, node.right):
            if child:
                parent[child] = node
                stack.append(child)
    seen = {target}
    queue = deque([(target, 0)])
    result = []
    while queue:
        node, distance = queue.popleft()
        result.append(node.val)
        if distance == k:
            continue
        for neighbour in (node.left, node.right, parent[node]):
            if neighbour and neighbour not in seen:
                seen.add(neighbour)
                queue.append((neighbour, distance + 1))
    return result


# ============================================================
# FOLLOW-UP 2: The distance between two nodes of the tree
# ============================================================

# ---- follow-up 2: function ----
# Answer: Find the lowest common ancestor and add the
# depths of both nodes below it.
# Time: O(n).  Space: O(n).
def distance_between(root, a, b):
    depth = {}

    def lca(node, level):
        if node is None:
            return None
        depth[node] = level
        left = lca(node.left, level + 1)
        right = lca(node.right, level + 1)
        if node is a or node is b:
            return node
        if left and right:
            return node
        return left or right

    ancestor = lca(root, 0)
    return depth[a] + depth[b] - 2 * depth[ancestor]
```

124. Binary Tree Maximum Path Sum

Hard · Tree, Depth-First Search, Dynamic Programming, Binary Tree, Tree DP

leetcode.com/problems/binary-tree-maximum-path-sum

★ **TOP 7 PROBLEM** in Tree DFS (Preorder / Inorder / Postorder): master this one first

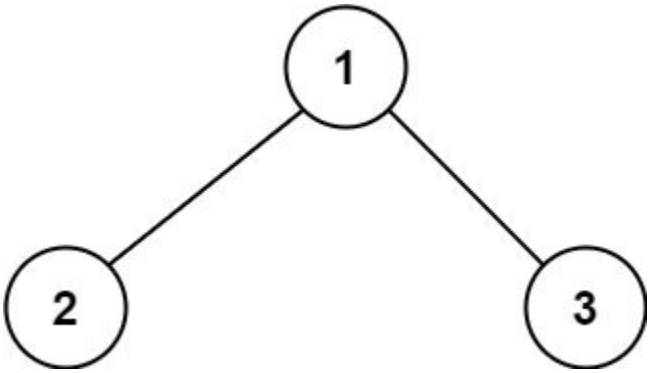
PROBLEM

A **path** in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence **at most once**. Note that the path does not need to pass through the root.

The **path sum** of a path is the sum of the node's values in the path.

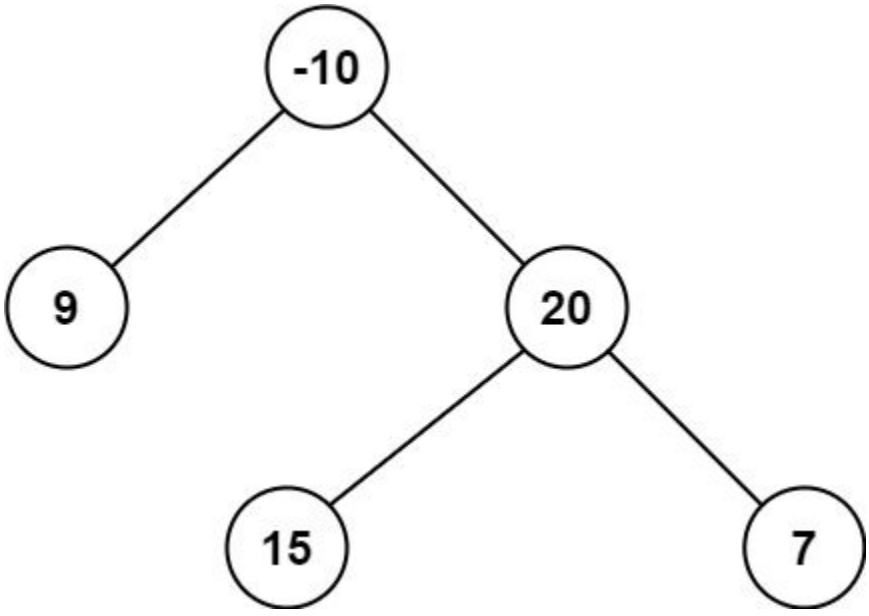
Given the `root` of a binary tree, return *the maximum **path sum** of any **non-empty** path*.

Example 1:




```
Input: root = [1,2,3]
Output: 6
Explanation: The optimal path is 2 -> 1 -> 3 with a path sum of 2 + 1 + 3 = 6.
```

Example 2:




```
Input: root = [-10,9,20,null,null,15,7]
Output: 42
Explanation: The optimal path is 15 -> 20 -> 7 with a path sum of 15 + 20 + 7 = 42.
```

Constraints:

- The number of nodes in the tree is in the range `[1, 3 * 10^4]`.
- `-1000 <= Node.val <= 1000`

SOLUTION (PYTHON 3)

Solution 1: Recursion

> **How to think about it**
>
> A path may start anywhere, need not pass the root, and may bend at a node to take both children. Enumerating paths is impossible at $n \le 3\times 10^4$. The value a node can pass upward is its value plus the better non-negative child chain. The sum that uses both children only updates the global answer and cannot propagate. Negative contributions are dropped by taking max(0,·) before returning.

When thinking about the classic routine of recursion problems in binary trees, we consider:

1. Termination condition (when to terminate recursion)
2. Recursively process the left and right subtrees
3. Merge the calculation results of the left and right subtrees

For this problem, we design a function dfs(root), which returns the maximum path sum of the binary tree with root as the root node.

The execution logic of the function dfs(root) is as follows:

If root does not exist, then dfs(root) returns 0;

Otherwise, we recursively calculate the maximum path sum of the left and right subtrees of root, denoted as left and right. If left is less than 0, then we set it to 0, similarly, if right is less than 0, then we set it to 0.

Then, we update the answer with root.val + left + right. Finally, the function returns root.val + max(left, right).

In the main function, we call dfs(root) to get the maximum path sum of each node, and the maximum value among them is the answer.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maxPathSum(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]) -> int:
            if root is None:
                return 0
            left = max(0, dfs(root.left))
            right = max(0, dfs(root.right))
            nonlocal ans
            ans = max(ans, root.val + left + right)
            return root.val + max(left, right)

        ans = -inf
        dfs(root)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the path itself, not only its sum
# 2. The path must start at the root (going downward)


# ============================================================
# FOLLOW-UP 1: Return the path itself, not only its sum
# ============================================================

# ---- follow-up 1: function ----
# Answer: Each call returns the best downward path
# starting at its node (a list); the best path through a
# node joins its best left path, the node and its best
# right path, ignoring paths with a negative sum.
# Time: O(n * h) (path sums are recomputed).  Space: O(h).
def max_path_sum_with_path(root):
    best = [None, []]

    def sum_of(values):
        total = 0
        for v in values:
            total += v
        return total

    def down(node):
        if node is None:
            return []
        left = down(node.left)
        right = down(node.right)
        if sum_of(left) < 0:
            left = []
        if sum_of(right) < 0:
            right = []
        through = left[::-1] + [node.val] + right
        total = sum_of(through)
        if best[0] is None or total > best[0]:
            best[0] = total
            best[1] = through
        if sum_of(left) >= sum_of(right):
            return [node.val] + left
        return [node.val] + right

    down(root)
    return best[0], best[1]


# ============================================================
# FOLLOW-UP 2: The path must start at the root (going
# downward)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Only one side is allowed: the best sum from the
# root is the node plus the better of its two children's
# best sums, ignoring negative ones.
# Time: O(n).  Space: O(h).
def max_root_down_sum(root):
    if root is None:
        return 0
    left = max_root_down_sum(root.left)
    right = max_root_down_sum(root.right)
    best_child = left if left > right else right
    if best_child < 0:
        best_child = 0
```

```
    return root.val + best_child
```

PATTERN 12 OF 41

Tree BFS (Level Order)

20 problems · 4 Easy · 16 Medium · 0 Hard

THE IDEA

Process the tree one level at a time with a queue; the queue length at the start of a level is the size of that level.

Signals: level order, zigzag, right side view, minimum depth, widest level, connect next pointers, average of each level.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
from collections import deque
q = deque([root]) if root else deque()
levels = []
while q:
    level = []
    for _ in range(len(q)):          # exactly one level
        node = q.popleft()
        level.append(node.val)
        if node.left:
            q.append(node.left)
        if node.right:
            q.append(node.right)
    levels.append(level)
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Average of Levels in Binary Tree	Easy	LC 637	596
2	Minimum Depth of Binary Tree	Easy	LC 111	600
3	Cousins in Binary Tree	Easy	LC 993	603
4	N-ary Tree Level Order Traversal	Easy	LC 429	607
5	★ **Binary Tree Level Order Traversal**	Medium	LC 102	611
6	Binary Tree Level Order Traversal II	Medium	LC 107	614
7	★ **Binary Tree Zigzag Level Order Traversal**	Medium	LC 103	617
8	★ **Binary Tree Right Side View**	Medium	LC 199	620
9	Populating Next Right Pointers in Each Node	Medium	LC 116	624
10	★ **Populating Next Right Pointers in Each Node II**	Medium	LC 117	628
11	★ **Maximum Width of Binary Tree**	Medium	LC 662	632
12	Find Largest Value in Each Tree Row	Medium	LC 515	637
13	★ **Binary Tree Vertical Order Traversal — substitute: Find Bottom Left Tree Value**	Medium	LC 513	640
14	Add One Row to Tree	Medium	LC 623	643
15	Level Order Traversal Distinct Values — substitute: Deepest Leaves Sum	Medium	LC 1302	647
16	Smallest String Starting From Leaf	Medium	LC 988	651
17	Binary Tree Coloring Game	Medium	LC 1145	655
18	Step-By-Step Directions From a Binary Tree Node to Another	Medium	LC 2096	658
19	★ **Check Completeness of a Binary Tree**	Medium	LC 958	666
20	Cousins in Binary Tree II	Medium	LC 2641	669

PATTERN 12 · TREE BFS (LEVEL ORDER) · PROBLEM 1 OF 20

637. Average of Levels in Binary Tree

Easy · Tree, Depth-First Search, Breadth-First Search, Binary Tree

leetcode.com/problems/average-of-levels-in-binary-tree

PROBLEM

Given the

`root`

of a binary tree, return

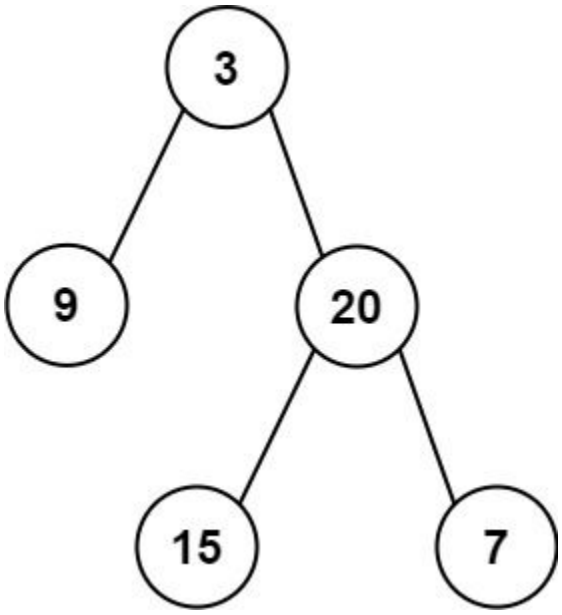
the average value of the nodes on each level in the form of an array

. Answers within

10^{-5}

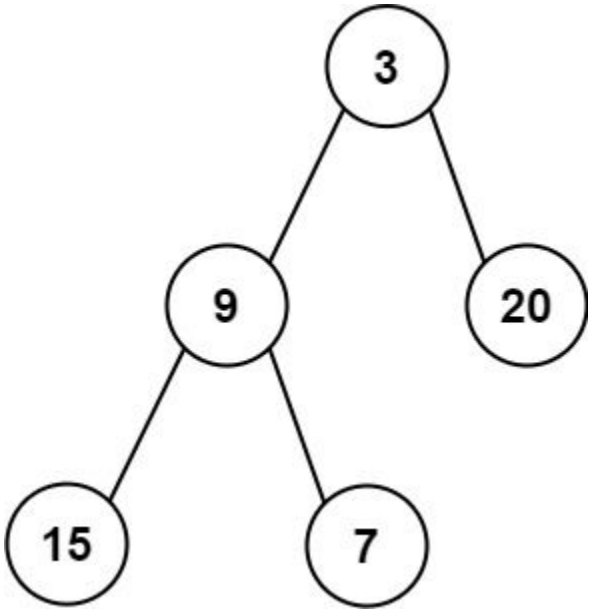
of the actual answer will be accepted.

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: [3.00000,14.50000,11.00000]
Explanation: The average value of nodes on level 0 is 3, on level 1 is 14.5, and on level 2 is 11.
Hence return [3, 14.5, 11].
```

Example 2:




```
Input: root = [3,9,20,15,7]
Output: [3.00000,14.50000,11.00000]
```

Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.
- $-2^{31} <= Node.val <= 2^{31} - 1$

SOLUTIONS (PYTHON 3)

Solution 1: BFS

> **How to think about it**
>
> Level averages match a level-order walk. One BFS is enough for $n \le 10^4$. Drain the current queue, divide the sum by its size, and enqueue children.

We can use the Breadth-First Search (BFS) method to traverse the nodes of each level and calculate the average value of each level.

Specifically, we define a queue q, initially adding the root node to the queue. Each time, we take out all the nodes in the queue, calculate their average value, add it to the answer array, and then add their child nodes to the queue. Repeat this process until the queue is empty.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def averageOfLevels(self, root: Optional[TreeNode]) -> List[float]:
        q = deque([root])
        ans = []
        while q:
            s, n = 0, len(q)
            for _ in range(n):
                root = q.popleft()
                s += root.val
                if root.left:
                    q.append(root.left)
                if root.right:
                    q.append(root.right)
            ans.append(s / n)
        return ans
```

Solution 2: DFS

> **How to think about it**
>
> BFS needs a queue. DFS can accumulate (sum, count) by depth and divide at the end, with the same result.

We can also use the Depth-First Search (DFS) method to calculate the average value of each level.

Specifically, we define an array s, where s[i] is a tuple representing the sum of node values and the number of nodes at the i-th level. We perform a depth-first search on the tree. For each node, we add the node's value to the corresponding s[i] and increment the node count by one. Finally, for each s[i], we calculate the average value and add it to the answer array.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def averageOfLevels(self, root: Optional[TreeNode]) -> List[float]:
        def dfs(root, i):
            if root is None:
                return
            if len(s) == i:
                s.append([root.val, 1])
            else:
                s[i][0] += root.val
                s[i][1] += 1
            dfs(root.left, i + 1)
            dfs(root.right, i + 1)

        s = []
        dfs(root, 0)
        return [a / b for a, b in s]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The averages of the levels of an N-ary tree
# 2. Which level has the largest sum (return the smallest
# such level, counting from 1)?


# ============================================================
# FOLLOW-UP 1: The averages of the levels of an N-ary tree
# ============================================================

# ---- follow-up 1: function ----
# Answer: Level by level with a queue; every node adds all
# its children to the next level.
# Time: O(n).  Space: O(w), w = widest level.
from collections import deque


def level_averages_nary(root):
    averages = []
    queue = deque([root]) if root else deque()
    while queue:
        total = 0
        size = len(queue)
        for i in range(size):
            node = queue.popleft()
            total += node.val
            for child in node.children:
                queue.append(child)
        averages.append(total / size)
    return averages


# ============================================================
# FOLLOW-UP 2: Which level has the largest sum (return the
# smallest such level, counting from 1)?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Compute the sum of each level and remember the
# first level with the biggest sum.
# Time: O(n).  Space: O(w).
from collections import deque


def level_with_max_sum(root):
    queue = deque([root]) if root else deque()
    best_level = 0
    best_sum = None
    level = 0
    while queue:
        level += 1
        total = 0
        for i in range(len(queue)):
            node = queue.popleft()
            total += node.val
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        if best_sum is None or total > best_sum:
            best_sum = total
            best_level = level
    return best_level
```

111. Minimum Depth of Binary Tree

Easy · Tree, Depth-First Search, Breadth-First Search, Binary Tree

leetcode.com/problems/minimum-depth-of-binary-tree

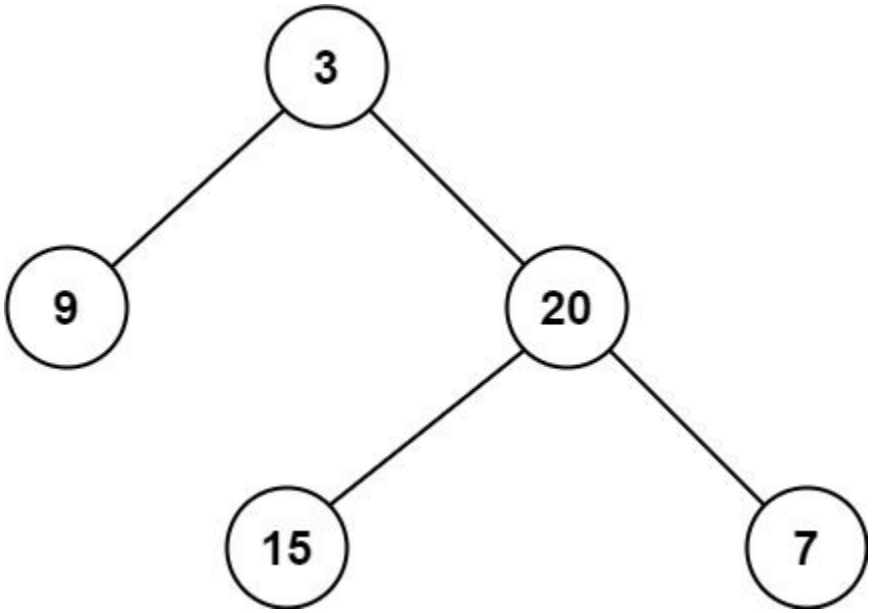
PROBLEM

Given a binary tree, find its minimum depth.

The minimum depth is the number of nodes along the shortest path from the root node down to the nearest leaf node.

Note: A leaf is a node with no children.

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: 2
```

Example 2:

```
Input: root = [2,null,3,null,4,null,5,null,6]
Output: 5
```

Constraints:

- The number of nodes in the tree is in the range `[0, 10^5]`.
- `-1000 <= Node.val <= 1000`

SOLUTIONS (PYTHON 3)

Solution 1: Recursion

How to think about it

Minimum depth is the number of nodes on the shortest root-to-leaf path. A node with only one child is not a leaf, so we must continue down the existing side; treating the missing child as depth 0 is wrong. n ≤ 10^5. Recursively: empty tree is 0; missing left means 1 plus the right minimum; missing right is symmetric; both children present means 1 plus the smaller of the two.

The termination condition for recursion is when the current node is null, at which point return 0. If one of the left or right subtrees of the current node is null, return the minimum depth of the non-null subtree plus 1. If neither the left nor right

subtree of the current node is null, return the smaller value of the minimum depths of the left and right subtrees plus 1.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def minDepth(self, root: Optional[TreeNode]) -> int:
        if root is None:
            return 0
        if root.left is None:
            return 1 + self.minDepth(root.right)
        if root.right is None:
            return 1 + self.minDepth(root.left)
        return 1 + min(self.minDepth(root.left), self.minDepth(root.right))
```

Solution 2: BFS

> **How to think about it**
>
> Solution 1 may walk an entire deep branch before seeing the nearest leaf. Minimum depth is a shortest path, so BFS can stop at the first leaf. That is faster in practice on skewed trees.

Use a queue to implement breadth-first search, initially adding the root node to the queue. Each time, take a node from the queue. If this node is a leaf node, directly return the current depth. If this node is not a leaf node, add all non-null child nodes of this node to the queue. Continue to search the next layer of nodes until a leaf node is found.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def minDepth(self, root: Optional[TreeNode]) -> int:
        if root is None:
            return 0
        q = deque([root])
        ans = 0
        while 1:
            ans += 1
            for _ in range(len(q)):
                node = q.popleft()
                if node.left is None and node.right is None:
                    return ans
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The minimum depth of an N-ary tree
# 2. Return the path from the root to the nearest leaf


# ============================================================
# FOLLOW-UP 1: The minimum depth of an N-ary tree
# ============================================================

# ---- follow-up 1: function ----
# Answer: Level by level; the first node without children
# ends the search.
# Time: O(n).  Space: O(w).
from collections import deque


def min_depth_nary(root):
    if root is None:
        return 0
    queue = deque([(root, 1)])
    while queue:
        node, depth = queue.popleft()
        if not node.children:
            return depth
        for child in node.children:
            queue.append((child, depth + 1))


# ============================================================
# FOLLOW-UP 2: Return the path from the root to the
# nearest leaf
# ============================================================

# ---- follow-up 2: function ----
# Answer: Remember each node's parent while searching,
# then walk back from the first leaf found.
# Time: O(n).  Space: O(n).
from collections import deque


def path_to_nearest_leaf(root):
    if root is None:
        return []
    parent = {root: None}
    queue = deque([root])
    while queue:
        node = queue.popleft()
        if node.left is None and node.right is None:
            path = []
            while node:
                path.append(node.val)
                node = parent[node]
            path.reverse()
            return path
        for child in (node.left, node.right):
            if child:
                parent[child] = node
                queue.append(child)
```

993. Cousins in Binary Tree

Easy · Tree, Depth-First Search, Breadth-First Search, Binary Tree

leetcode.com/problems/cousins-in-binary-tree

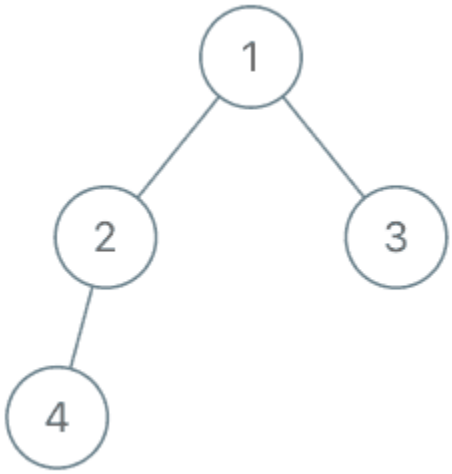
PROBLEM

Given the `root` of a binary tree with unique values and the values of two different nodes of the tree `x` and `y`, return `true` *if the nodes corresponding to the values* `x` *and* `y` *in the tree are* ***cousins****, or* `false` *otherwise.*

Two nodes of a binary tree are **cousins** if they have the same depth with different parents.

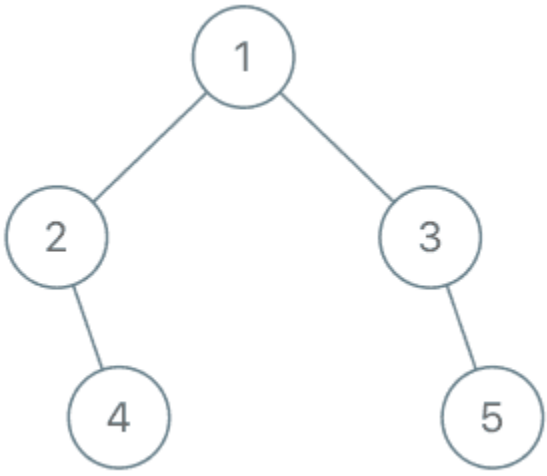
Note that in a binary tree, the root node is at the depth `0`, and children of each depth `k` node are at the depth `k + 1`.

Example 1:



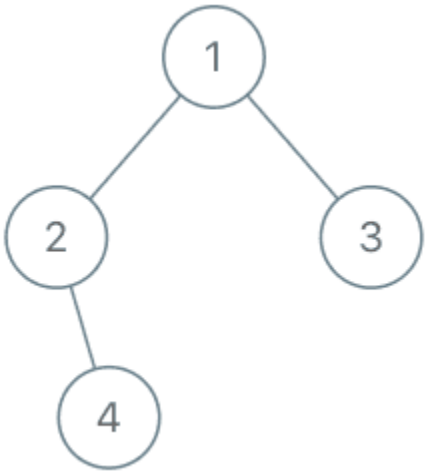

```
Input: root = [1,2,3,4], x = 4, y = 3
Output: false
```

Example 2:




```
Input: root = [1,2,3,null,4,null,5], x = 5, y = 4
Output: true
```

Example 3:




```
Input: root = [1,2,3,null,4], x = 2, y = 3
Output: false
```

Constraints:

- The number of nodes in the tree is in the range `[2, 100]`.
- `1 <= Node.val <= 100`
- Each node has a **unique** value.
- `x != y`
- `x` and `y` are exist in the tree.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Cousins share a depth and have different parents. Level-order the tree while carrying the parent; when x or y appears, record parent and depth, then compare after the walk.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isCousins(self, root: Optional[TreeNode], x: int, y: int) -> bool:
        q = deque([(root, None)])
        depth = 0
        p1 = p2 = None
        d1 = d2 = None
        while q:
            for _ in range(len(q)):
                node, parent = q.popleft()
                if node.val == x:
                    p1, d1 = parent, depth
                elif node.val == y:
                    p2, d2 = parent, depth
                if node.left:
                    q.append((node.left, node))
                if node.right:
                    q.append((node.right, node))
            depth += 1
        return p1 != p2 and d1 == d2
```

Complexity. Time: O(n). Space: O(n).

Solution 2

> **How to think about it**
>
> BFS proceeds layer by layer. DFS works as well: pass the parent and depth, record them for x and y, and test the cousin condition.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isCousins(self, root: Optional[TreeNode], x: int, y: int) -> bool:
        def dfs(root, parent, depth):
            if root is None:
                return
            if root.val == x:
                st[0] = (parent, depth)
            elif root.val == y:
                st[1] = (parent, depth)
            dfs(root.left, root, depth + 1)
            dfs(root.right, root, depth + 1)

        st = [None, None]
        dfs(root, None, 0)
        return st[0][0] != st[1][0] and st[0][1] == st[1][1]
```

Complexity. Time: O(n). Space: O(h).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return all the cousins of a node (same depth,
# different parent)
# 2. Are two values SIBLINGS (same parent)?


# ============================================================
# FOLLOW-UP 1: Return all the cousins of a node (same
# depth, different parent)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Find the depth and the parent of the target,
# then collect the nodes at that depth whose parent
# differs.
# Time: O(n).  Space: O(w).
from collections import deque


def cousins_of(root, target):
    queue = deque([(root, None)])
    while queue:
        level = []
        for i in range(len(queue)):
            node, parent = queue.popleft()
            level.append((node, parent))
            for child in (node.left, node.right):
                if child:
                    queue.append((child, node))
        for node, parent in level:
            if node.val == target:
                result = []
                for other, other_parent in level:
                    if other_parent is not parent:
                        result.append(other.val)
                return result
    return []


# ============================================================
# FOLLOW-UP 2: Are two values SIBLINGS (same parent)?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Look for a node whose two children are exactly x
# and y.
# Time: O(n).  Space: O(h).
def are_siblings(root, x, y):
    if root is None:
        return False
    if root.left and root.right:
        if {root.left.val, root.right.val} == {x, y}:
            return True
    return are_siblings(root.left, x, y) or are_siblings(root.right, x, y)
```

429. N-ary Tree Level Order Traversal

Medium · Tree, Breadth-First Search

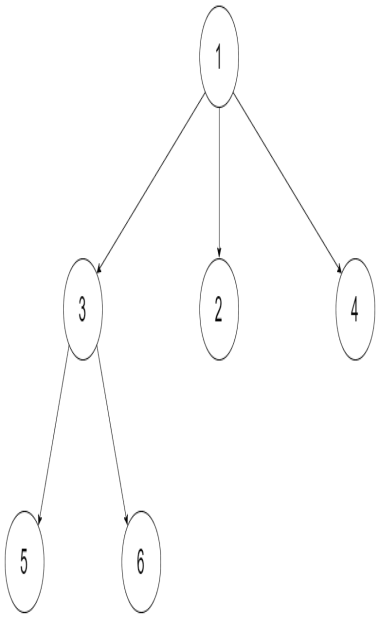
leetcode.com/problems/n-ary-tree-level-order-traversal

PROBLEM

Given an n-ary tree, return the *level order* traversal of its nodes' values.

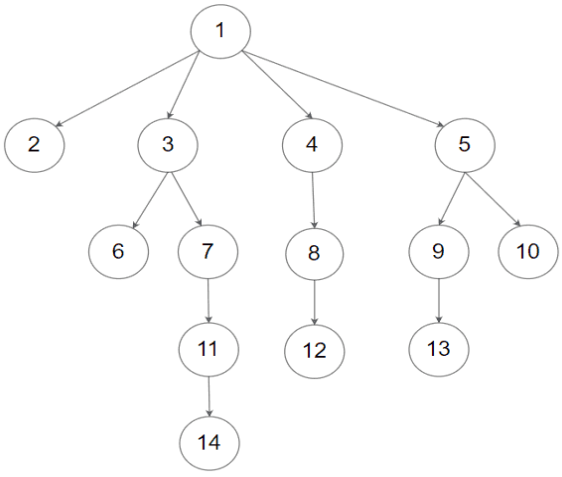
Nary-Tree input serialization is represented in their level order traversal, each group of children is separated by the null value (See examples).

Example 1:




```
Input: root = [1,null,3,2,4,null,5,6]
Output: [[1],[3,2,4],[5,6]]
```

Example 2:




```
Input: root =
[1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]
Output: [[1],[2,3,4,5],[6,7,8,9,10],[11,12,13],[14]]
```

Constraints:

- The height of the n-ary tree is less than or equal to `1000`
- The total number of nodes is between `[0, 10^4]`

SOLUTIONS (PYTHON 3)

Solution 1: BFS

> **How to think about it**
>
> Level order groups values by depth. A depth-first walk without a layer index would mix levels. A queue holds the current layer: snapshot its length, pop that many nodes into a row, and enqueue their children. An empty root yields an empty list. The snapshot of the queue length isolates one layer and keeps left-to-right order.

First, we check if the root node is null. If it is, we return an empty list directly.

Otherwise, we create a queue q and initially add the root node to the queue.

When the queue is not empty, we loop through the following operations:

1. Create an empty list t to store the values of the current level nodes.
2. For each node in the queue, add its value to t and add its child nodes to the queue.
3. Add t to the result list ans.

Finally, return the result list ans.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the N-ary tree.

Python 3

```
"""
# Definition for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children
"""


class Solution:
    def levelOrder(self, root: 'Node') -> List[List[int]]:
        ans = []
        if root is None:
            return ans
        q = deque([root])
        while q:
            t = []
            for _ in range(len(q)):
                root = q.popleft()
                t.append(root.val)
                q.extend(root.children)
            ans.append(t)
        return ans
```

Solution 2: DFS

> **How to think about it**
>
> Solution 1 needs a queue. Recursing with a depth i and appending to ans[i] (growing the list when needed) produces the same rows. The call stack carries the layer index; children are still written left to right.

We can use the Depth-First Search method to traverse the entire tree.

We define a helper function dfs(root, i), where root represents the current node, and i represents the current level.

In the dfs function, we first check if root is null. If it is, we return directly.

Otherwise, we check if the length of ans is less than or equal to i. If it is, it means that the current level has not been added to ans yet, so we need to add an empty list first. Then we add the value of root to ans[i].

Next, we traverse all child nodes of root. For each child node, we call dfs(child, i + 1).

In the main function, we call dfs(root, 0) and return ans.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the N-ary tree.

Python 3

```
"""
# Definition for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children
"""


class Solution:
    def levelOrder(self, root: 'Node') -> List[List[int]]:
        def dfs(root, i):
            if root is None:
                return
            if len(ans) <= i:
                ans.append([])
            ans[i].append(root.val)
            for child in root.children:
                dfs(child, i + 1)

        ans = []
        dfs(root, 0)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The same result with a recursive depth-first walk
# 2. Which level is every value on?


# ============================================================
# FOLLOW-UP 1: The same result with a recursive depth-
# first walk
# ============================================================

# ---- follow-up 1: function ----
# Answer: Pass the depth; append the value to
# levels[depth], adding a new list the first time a depth
# is seen.
# Time: O(n).  Space: O(h) plus the output.
def level_order_nary_dfs(root):
    levels = []

    def walk(node, depth):
        if depth == len(levels):
            levels.append([])
        levels[depth].append(node.val)
        for child in node.children:
            walk(child, depth + 1)

    if root:
        walk(root, 0)
    return levels


# ============================================================
```

```
# FOLLOW-UP 2: Which level is every value on?
# ============================================================

# ---- follow-up 2: function ----
# Answer: A queue of (node, level) pairs gives the level
# of every node while traversing.
# Time: O(n).  Space: O(n).
from collections import deque


def level_of_every_value(root):
    levels = {}
    queue = deque([(root, 0)]) if root else deque()
    while queue:
        node, level = queue.popleft()
        levels[node.val] = level
        for child in node.children:
            queue.append((child, level + 1))
    return levels
```

PATTERN 12 · TREE BFS (LEVEL ORDER) · PROBLEM 5 OF 20

102. Binary Tree Level Order Traversal

Medium · Tree, Breadth-First Search, Binary Tree

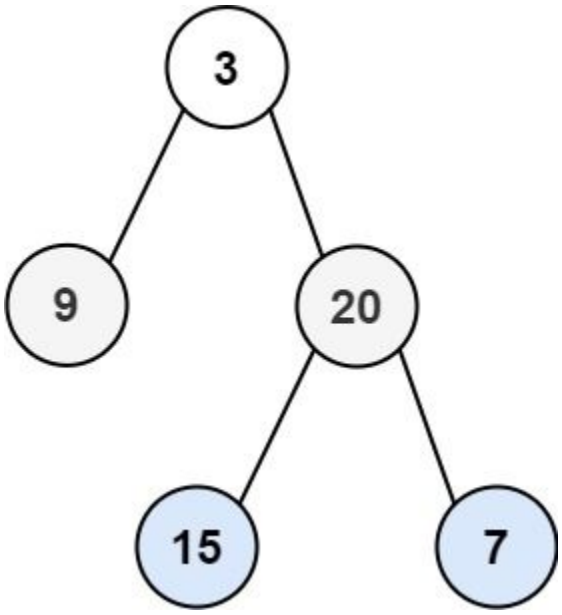
leetcode.com/problems/binary-tree-level-order-traversal

★ **TOP 7 PROBLEM** in Tree BFS (Level Order): master this one first

PROBLEM

Given the `root` of a binary tree, return *the level order traversal of its nodes' values*. (i.e., from left to right, level by level).

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: [[3],[9,20],[15,7]]
```

Example 2:

```
Input: root = [1]
Output: [[1]]
```

Example 3:

```
Input: root = []
Output: []
```

Constraints:

- The number of nodes in the tree is in the range `[0, 2000]`.
- `-1000 <= Node.val <= 1000`

SOLUTION (PYTHON 3)

Solution 1: BFS

How to think about it

We must emit values level by level. DFS can record depth and group later, but then left-to-right order needs extra work. n ≤ 2000, and a level holds at most O(n) nodes. BFS expands by level: the nodes currently in the queue are exactly one level. Dequeue them, collect values, and enqueue children to form the next level from left to right.

We can use the BFS method to solve this problem. First, enqueue the root node, then continuously perform the following operations until the queue is empty:

- Traverse all nodes in the current queue, store their values in a temporary array t, and then enqueue their child nodes.
- Store the temporary array t in the answer array.

Finally, return the answer array.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:
            return ans
        q = deque([root])
        while q:
            t = []
            for _ in range(len(q)):
                node = q.popleft()
                t.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(t)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Depth-first version (recursion with a depth argument)
# 2. On which level does a given value sit (and what are
# its level mates)?


# ============================================================
# FOLLOW-UP 1: Depth-first version (recursion with a depth
# argument)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Append every value to the list of its depth; a
# new list is added when the depth is seen for the first
# time.
# Time: O(n).  Space: O(h) plus the output.
def level_order_dfs(root):
    levels = []
```

```
    def walk(node, depth):
        if node is None:
            return
        if depth == len(levels):
            levels.append([])
        levels[depth].append(node.val)
        walk(node.left, depth + 1)
        walk(node.right, depth + 1)

    walk(root, 0)
    return levels


# ============================================================
# FOLLOW-UP 2: On which level does a given value sit (and
# what are its level mates)?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Traverse level by level and stop at the level
# that contains the value.
# Time: O(n).  Space: O(w).
from collections import deque


def level_of_value(root, value):
    queue = deque([root]) if root else deque()
    depth = 0
    while queue:
        level = []
        for i in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        if value in level:
            return depth, level
        depth += 1
    return -1, []
```

PATTERN 12 · TREE BFS (LEVEL ORDER) · PROBLEM 6 OF 20

107. Binary Tree Level Order Traversal II

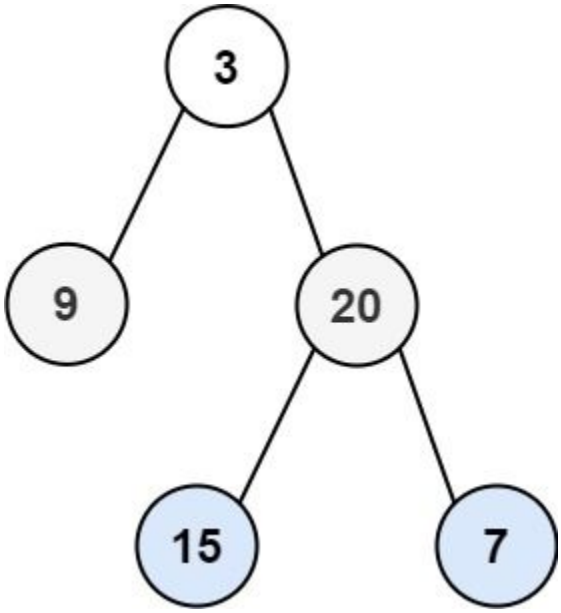
Medium · Tree, Breadth-First Search, Binary Tree

leetcode.com/problems/binary-tree-level-order-traversal-ii

PROBLEM

Given the `root` of a binary tree, return *the bottom-up level order traversal of its nodes' values*. (i.e., from left to right, level by level from leaf to root).

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: [[15,7],[9,20],[3]]
```

Example 2:

```
Input: root = [1]
Output: [[1]]
```

Example 3:

```
Input: root = []
Output: []
```

Constraints:

- The number of nodes in the tree is in the range `[0, 2000]`.
- `-1000 <= Node.val <= 1000`

SOLUTION (PYTHON 3)

Solution 1: BFS

How to think about it

This is ordinary level order, except the layers must be listed from the bottom up. Collect levels as usual and reverse the answer; there is no need to insert at the front while traversing. n ≤ 2000, so BFS by layer and a final reverse suffice.

We can use the BFS (Breadth-First Search) method to solve this problem. First, enqueue the root node, then continuously perform the following operations until the queue is empty:

- Traverse all nodes in the current queue, store their values in a temporary array t, and then enqueue their child nodes.
- Store the temporary array t in the answer array.

Finally, return the reversed answer array.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def levelOrderBottom(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:
            return ans
        q = deque([root])
        while q:
            t = []
            for _ in range(len(q)):
                node = q.popleft()
                t.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(t)
        return ans[::-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Bottom-up AND each level read from right to left
# 2. Only every second level (the odd levels, counting the
# root as level 1)


# ============================================================
# FOLLOW-UP 1: Bottom-up AND each level read from right to
# left
# ============================================================

# ---- follow-up 1: function ----
# Answer: Collect the levels top-down (each level written
# right to left) and reverse the list of levels at the
# end.
# Time: O(n).  Space: O(w).
from collections import deque


def bottom_up_right_to_left(root):
    levels = []
    queue = deque([root]) if root else deque()
    while queue:
        level = []
        for i in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.right:
                queue.append(node.right)
            if node.left:
```

```
                queue.append(node.left)
        levels.append(level)
    levels.reverse()
    return levels


# ============================================================
# FOLLOW-UP 2: Only every second level (the odd levels,
# counting the root as level 1)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Do the normal traversal and keep a level only
# when its number is odd.
# Time: O(n).  Space: O(w).
from collections import deque


def odd_levels(root):
    result = []
    queue = deque([root]) if root else deque()
    level_number = 0
    while queue:
        level_number += 1
        level = []
        for i in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
```

```
            if node.right:
                queue.append(node.right)
        if level_number % 2 == 1:
            result.append(level)
    return result
```

103. Binary Tree Zigzag Level Order Traversal

Medium · Tree, Breadth-First Search, Binary Tree

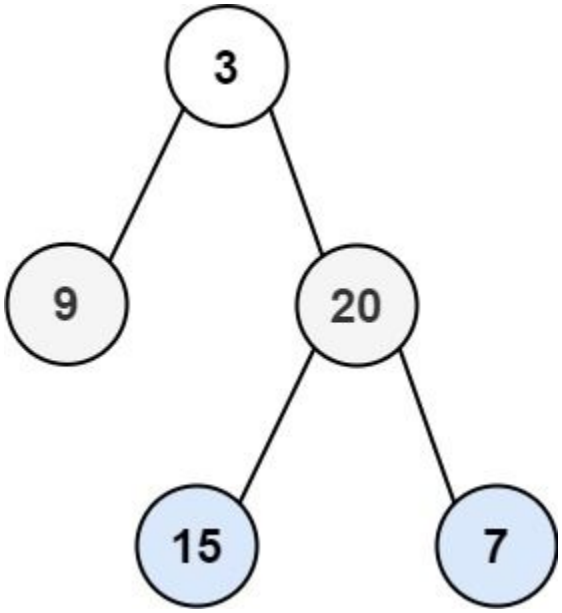
leetcode.com/problems/binary-tree-zigzag-level-order-traversal

★ **TOP 7 PROBLEM** in Tree BFS (Level Order): master this one first

PROBLEM

Given the `root` of a binary tree, return *the zigzag level order traversal of its nodes' values*. (i.e., from left to right, then right to left for the next level and alternate between).

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: [[3],[20,9],[15,7]]
```

Example 2:

```
Input: root = [1]
Output: [[1]]
```

Example 3:

```
Input: root = []
Output: []
```

Constraints:

- The number of nodes in the tree is in the range `[0, 2000]`.
- `-100 <= Node.val <= 100`

SOLUTION (PYTHON 3)

Solution 1: BFS

How to think about it

Unlike ordinary level order, odd levels stay left-to-right and even levels go right-to-left. Building each level and reversing some of them is still O(n). n ≤ 2000 is fine. We keep BFS and add a direction flag. After a level is collected, reverse it when the flag says right-to-left, then flip the flag. Children are always enqueued left to right, so the next level's visitation order is unchanged.

To implement zigzag level order traversal, we need to add a flag `left` on the basis of level order traversal. This flag is used to mark the order of the node values in the current level. If `left` is `true`, the node values of the current level are stored in the result array `ans` from left to right. If `left` is `false`, the node values of the current level are stored in the result array `ans` from right to left.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def zigzagLevelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:
            return ans
        q = deque([root])
        ans = []
        left = 1
        while q:
            t = []
            for _ in range(len(q)):
                node = q.popleft()
                t.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(t if left else t[::-1])
            left ^= 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Zigzag order for an N-ary tree
# 2. The zigzag order with a depth-first walk


# ============================================================
# FOLLOW-UP 1: Zigzag order for an N-ary tree
# ============================================================

# ---- follow-up 1: function ----
# Answer: Collect every level with its children in order
# and reverse every second level.
# Time: O(n).  Space: O(w).
from collections import deque
```

```
def zigzag_nary(root):
    result = []
    queue = deque([root]) if root else deque()
    left_to_right = True
    while queue:
        level = []
        for i in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            for child in node.children:
                queue.append(child)
        if not left_to_right:
            level.reverse()
        result.append(level)
        left_to_right = not left_to_right
    return result


# ============================================================
# FOLLOW-UP 2: The zigzag order with a depth-first walk
# ============================================================

# ---- follow-up 2: function ----
# Answer: Fill every level left to right while walking,
# then reverse the odd-numbered levels at the end.
# Time: O(n).  Space: O(h) plus the output.
def zigzag_dfs(root):
    levels = []

    def walk(node, depth):
        if node is None:
            return
        if depth == len(levels):
            levels.append([])
        levels[depth].append(node.val)
        walk(node.left, depth + 1)
        walk(node.right, depth + 1)

    walk(root, 0)
    for depth in range(1, len(levels), 2):
        levels[depth].reverse()
    return levels
```

199. Binary Tree Right Side View

Medium · Tree, Depth-First Search, Breadth-First Search, Binary Tree

leetcode.com/problems/binary-tree-right-side-view

★ **TOP 7 PROBLEM** in Tree BFS (Level Order): master this one first

PROBLEM

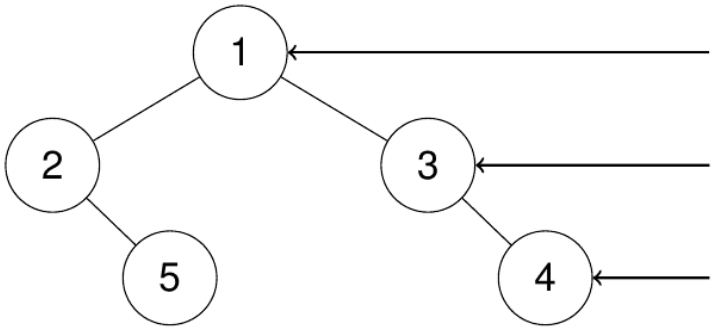
Given the `root` of a binary tree, imagine yourself standing on the **right side** of it, return *the values of the nodes you can see ordered from top to bottom*.

Example 1:

Input: root = [1,2,3,null,5,null,4]

Output: [1,3,4]

Explanation:

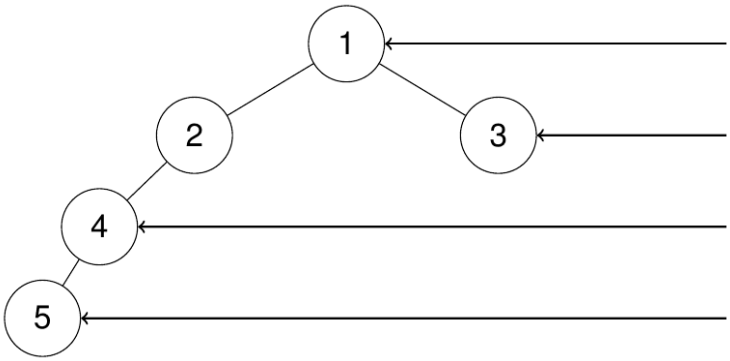


Example 2:

Input: root = [1,2,3,4,null,null,null,5]

Output: [1,3,4,5]

Explanation:



Example 3:

Input: root = [1,null,3]

Output: [1,3]

Example 4:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range `[0, 100]`.
- `-100 <= Node.val <= 100`

SOLUTIONS (PYTHON 3)

Solution 1: BFS

> **How to think about it**
>
> The right side view is the rightmost node of each level. n≤ 100. Enqueue the right child first so the front of the queue is that level's rightmost node; record it, then drain the level. No need to take the last element afterwards.

We can use breadth-first search (BFS) and define a queue q to store the nodes. We start by putting the root node into the queue. Each time, we take out all the nodes of the current level from the queue. For the current node, we first check if the right subtree exists; if it does, we put the right subtree into the queue. Then, we check if the left subtree exists; if it does, we put the left subtree into the queue. This way, the first node taken out from the queue each time is the rightmost node of that level.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        ans = []
        if root is None:
            return ans
        q = deque([root])
        while q:
            ans.append(q[0].val)
            for _ in range(len(q)):
                node = q.popleft()
                if node.right:
                    q.append(node.right)
                if node.left:
                    q.append(node.left)
        return ans
```

Solution 2: DFS

> **How to think about it**
>
> Solution 1 uses a queue. Recurse right then left with a depth: the first visit at that depth is the rightmost node. Space follows height rather than width.

Use DFS (depth-first search) to traverse the binary tree. Each time, traverse the right subtree first, then the left subtree. This way, the first node visited at each level is the rightmost node of that level.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        def dfs(root: Optional[TreeNode], depth: int) -> None:
            if root is None:
                return
            if len(ans) == depth:
                ans.append(root.val)
            dfs(root.right, depth + 1)
            dfs(root.left, depth + 1)

        ans = []
        dfs(root, 0)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Both the left AND the right side view in one pass
# 2. The TOP view (what you see looking down from above)


# ============================================================
# FOLLOW-UP 1: Both the left AND the right side view in
# one pass
# ============================================================

# ---- follow-up 1: function ----
# Answer: For every level keep its first and its last
# node.
# Time: O(n).  Space: O(w).
from collections import deque


def left_and_right_views(root):
    left, right = [], []
    queue = deque([root]) if root else deque()
    while queue:
        left.append(queue[0].val)
        right.append(queue[-1].val)
        for i in range(len(queue)):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return left, right


# ============================================================
# FOLLOW-UP 2: The TOP view (what you see looking down
# from above)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Give every node a horizontal distance (left
# child -1, right child +1) and keep the first node met at
# each distance in a level-order walk.
# Time: O(n + d log d), d = distinct columns.  Space:
# O(n).
```

```
from collections import deque


def top_view(root):
    first_at = {}
    queue = deque([(root, 0)]) if root else deque()
    while queue:
        node, distance = queue.popleft()
        if distance not in first_at:
            first_at[distance] = node.val
        if node.left:
            queue.append((node.left, distance - 1))
        if node.right:
            queue.append((node.right, distance + 1))
    return [first_at[d] for d in sorted(first_at)]
```

116. Populating Next Right Pointers in Each Node

Medium · Tree, Depth-First Search, Breadth-First Search, Linked List, Binary Tree

leetcode.com/problems/populating-next-right-pointers-in-each-node

PROBLEM

You are given a **perfect binary tree** where all leaves are on the same level, and every parent has two children. The binary tree has the following definition:

```
struct Node {
  int val;
  Node *left;
  Node *right;
  Node *next;
}
```

Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to `NULL`.

Initially, all next pointers are set to `NULL`.

Example 1:

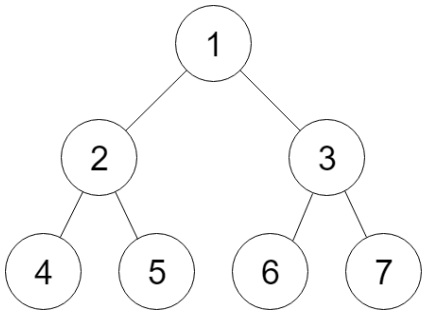


Figure A

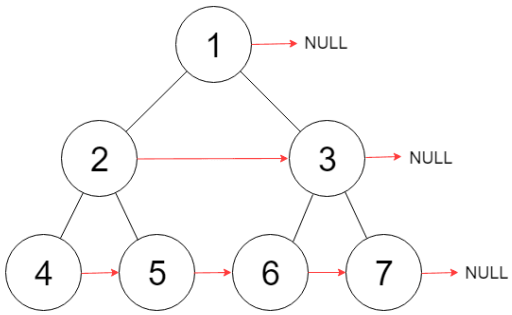


Figure B

```
Input: root = [1,2,3,4,5,6,7]
Output: [1,#,2,3,#,4,5,6,7,#]
Explanation: Given the above perfect binary tree (Figure A), your function should populate each next
pointer to point to its next right node, just like in Figure B. The serialized output is in level order
as connected by the next pointers, with '#' signifying the end of each level.
```

Example 2:

```
Input: root = []
Output: []
```

Constraints:

- The number of nodes in the tree is in the range `[0, 2^12 - 1]`.
- `-1000 <= Node.val <= 1000`

Follow-up:

- You may only use constant extra space.
- The recursive approach is fine. You may assume implicit stack space does not count as extra space for this problem.

SOLUTIONS (PYTHON 3)

Solution 1: BFS

How to think about it

The tree is perfect, so every level is full. BFS can wire next across each level. $n < 2^{12}$, and the queue is as wide as a level. The follow-up wants constant extra space; this method first gets the level-order linking right.

Use a queue for level order traversal, and each time you traverse a level, connect the nodes of the current level in order.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
"""
# Definition for a Node.
class Node:
    def __init__(self, val: int = 0, left: 'Node' = None, right: 'Node' = None, next: 'Node' = None):
        self.val = val
        self.left = left
        self.right = right
        self.next = next
"""


class Solution:
    def connect(self, root: "Optional[Node]") -> "Optional[Node]":
        if root is None:
            return root
        q = deque([root])
        while q:
            p = None
            for _ in range(len(q)):
                node = q.popleft()
                if p:
                    p.next = node
                p = node
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
        return root
```

Solution 2: DFS

How to think about it

Solution 1 stores a whole level in a queue. In a perfect tree, a node's two children are adjacent, and the right child is followed by the sibling's left child. Once the parent level is linked, those next pointers walk the next level, so recursion can wire children without a queue.

Use recursion for preorder traversal, and each time you traverse to a node, connect its left and right child nodes in order.

Specifically, we design a function dfs(left, right), which points the next pointer of the left node to the right node. In the function, we first check whether left and right are null. If both are not null, point left.next to right, and then recursively call dfs(left.left, left.right), dfs(left.right, right.left), dfs(right.left, right.right).

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
"""
# Definition for a Node.
class Node:
    def __init__(self, val: int = 0, left: 'Node' = None, right: 'Node' = None, next: 'Node' = None):
        self.val = val
        self.left = left
        self.right = right
        self.next = next
"""


class Solution:
    def connect(self, root: 'Optional[Node]') -> 'Optional[Node]':
        def dfs(left, right):
            if left is None or right is None:
                return
            left.next = right
            dfs(left.left, left.right)
            dfs(left.right, right.left)
            dfs(right.left, right.right)

        if root:
            dfs(root.left, root.right)
        return root
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Connect with a queue (O(n) extra space) instead of
# the pointers of the level above
# 2. Make each level circular: the last node points to the
# first


# ============================================================
# FOLLOW-UP 1: Connect with a queue (O(n) extra space)
# instead of the pointers of the level above
# ============================================================

# ---- follow-up 1: function ----
# Answer: Level by level, link every node to the one after
# it in the same level.
# Time: O(n).  Space: O(w).
from collections import deque


def connect_with_queue(root):
    queue = deque([root]) if root else deque()
    while queue:
        size = len(queue)
        for i in range(size):
            node = queue.popleft()
            if i < size - 1:
                node.next = queue[0]
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return root


# ============================================================
# FOLLOW-UP 2: Make each level circular: the last node
# points to the first
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: After linking a level, point its last node's
# next to the level's first node.
# Time: O(n).  Space: O(w).
from collections import deque


def connect_circular(root):
    queue = deque([root]) if root else deque()
    while queue:
        size = len(queue)
        first = queue[0]
        for i in range(size):
            node = queue.popleft()
            node.next = queue[0] if i < size - 1 else first
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return root
```

117. Populating Next Right Pointers in Each Node II

Medium · Tree, Depth-First Search, Breadth-First Search, Linked List, Binary Tree

leetcode.com/problems/populating-next-right-pointers-in-each-node-ii

★ **TOP 7 PROBLEM** in Tree BFS (Level Order): master this one first

PROBLEM

Given a binary tree

```
struct Node {
  int val;
  Node *left;
  Node *right;
  Node *next;
}
```

Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to `NULL`.

Initially, all next pointers are set to `NULL`.

Example 1:

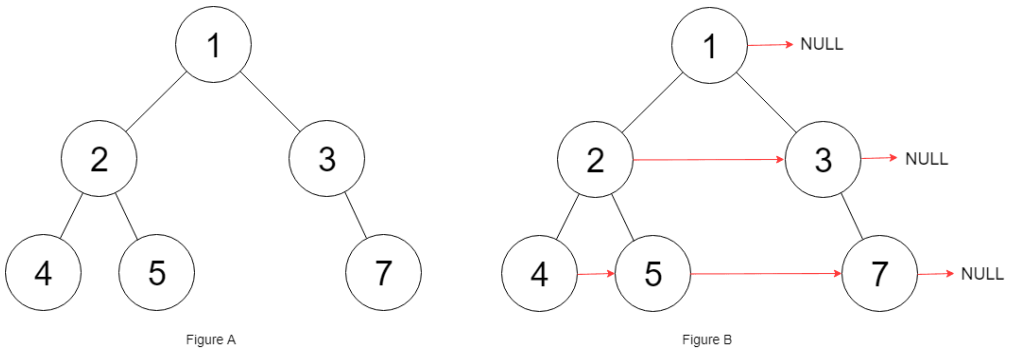


Figure A Figure B

```
Input: root = [1,2,3,4,5,null,7]
Output: [1,#,2,3,#,4,5,7,#]
Explanation: Given the above binary tree (Figure A), your function should populate each next pointer
to point to its next right node, just like in Figure B. The serialized output is in level order as
connected by the next pointers, with '#' signifying the end of each level.
```

Example 2:

```
Input: root = []
Output: []
```

Constraints:

- The number of nodes in the tree is in the range `[0, 6000]`.
- `-100 <= Node.val <= 100`

Follow-up:

- You may only use constant extra space.
- The recursive approach is fine. You may assume implicit stack space does not count as extra space for this problem.

SOLUTIONS (PYTHON 3)

Solution 1: BFS

> **How to think about it**
>
> Unlike the previous problem, the tree need not be perfect, so a left child does not always connect to a right child. Level-order is still correct: dequeue a level and link neighbors. n ≤ 6000.

We use a queue q for level order traversal. Each time we traverse a level, we connect the nodes of the current level in order.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
"""
# Definition for a Node.
class Node:
    def __init__(self, val: int = 0, left: 'Node' = None, right: 'Node' = None, next: 'Node' = None):
        self.val = val
        self.left = left
        self.right = right
        self.next = next
"""


class Solution:
    def connect(self, root: "Node") -> "Node":
        if root is None:
            return root
        q = deque([root])
        while q:
            p = None
            for _ in range(len(q)):
                node = q.popleft()
                if p:
                    p.next = node
                p = node
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
        return root
```

Solution 2: Space Optimization

> **How to think about it**
>
> Solution 1's queue is as wide as a level; the follow-up wants O(1) space. The current level is already a next list. While walking it we chain the next level's non-null children in order and remember that level's head, then step onto the new list. No queue.

The space complexity of Solution 1 is relatively high because it requires a queue to store the nodes of each level. We can implement it with constant space.

We define two pointers prev and next, which point to the previous node and the first node of the next level, respectively. When traversing the nodes of the current level, we string the nodes of the next level together and find the first node of the next level. After the current level is traversed, we assign the first node next of the next level to node and continue to traverse.

The time complexity is O(n), where n is the number of nodes in the binary tree. The space complexity is O(1).

Python 3

```
"""
# Definition for a Node.
class Node:
    def __init__(self, val: int = 0, left: 'Node' = None, right: 'Node' = None, next: 'Node' = None):
        self.val = val
        self.left = left
        self.right = right
        self.next = next
"""


class Solution:
    def connect(self, root: 'Node') -> 'Node':
        def modify(curr):
            nonlocal prev, next
            if curr is None:
                return
            next = next or curr
            if prev:
                prev.next = curr
            prev = curr

        node = root
        while node:
            prev = next = None
            while node:
                modify(node.left)
                modify(node.right)
                node = node.next
            node = next
        return root
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. O(1) extra space for ANY binary tree
# 2. Connect with a depth-first walk that remembers the
# last node of every depth


# ============================================================
# FOLLOW-UP 1: O(1) extra space for ANY binary tree
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use the next pointers of the level above to walk
# it; build the next level's chain with a dummy head and a
# tail pointer, skipping missing children.
# Time: O(n).  Space: O(1).
def connect_any_tree_constant_space(root):
    level_start = root
    while level_start:
        dummy_head = type("Dummy", (), {"next": None})()
        tail = dummy_head
        node = level_start
        while node:
            for child in (node.left, node.right):
                if child:
                    tail.next = child
                    tail = child
            node = node.next
        level_start = dummy_head.next
    return root
```

```
# ============================================================
# FOLLOW-UP 2: Connect with a depth-first walk that
# remembers the last node of every depth
# ============================================================

# ---- follow-up 2: function ----
# Answer: Go right to left or left to right while keeping,
# for each depth, the last node seen; link it to the new
# node.
# Time: O(n).  Space: O(h).
def connect_with_dfs(root):
    last_at_depth = {}

    def walk(node, depth):
        if node is None:
            return
        if depth in last_at_depth:
            last_at_depth[depth].next = node
        last_at_depth[depth] = node
        walk(node.left, depth + 1)
        walk(node.right, depth + 1)

    walk(root, 0)
    return root
```

662. Maximum Width of Binary Tree

Medium · Tree, Depth-First Search, Breadth-First Search, Binary Tree

leetcode.com/problems/maximum-width-of-binary-tree

★ **TOP 7 PROBLEM** in Tree BFS (Level Order): master this one first

PROBLEM

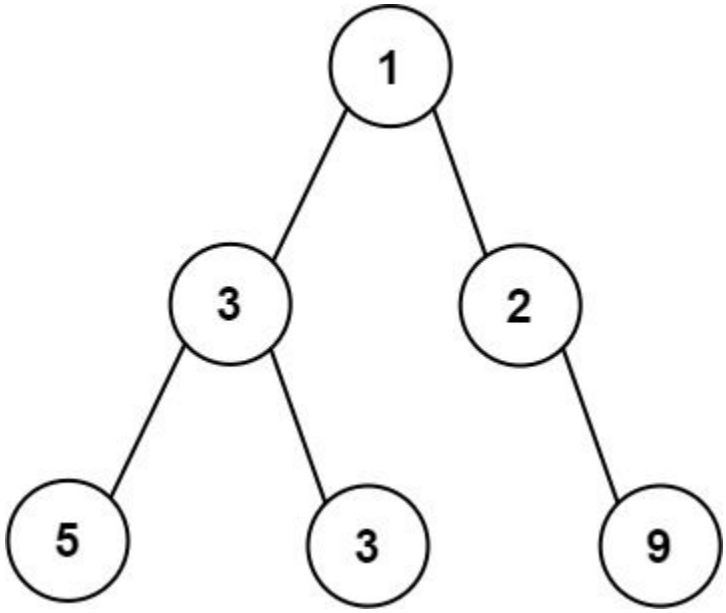
Given the `root` of a binary tree, return *the **maximum width** of the given tree*.

The **maximum width** of a tree is the maximum **width** among all levels.

The **width** of one level is defined as the length between the end-nodes (the leftmost and rightmost non-null nodes), where the null nodes between the end-nodes that would be present in a complete binary tree extending down to that level are also counted into the length calculation.

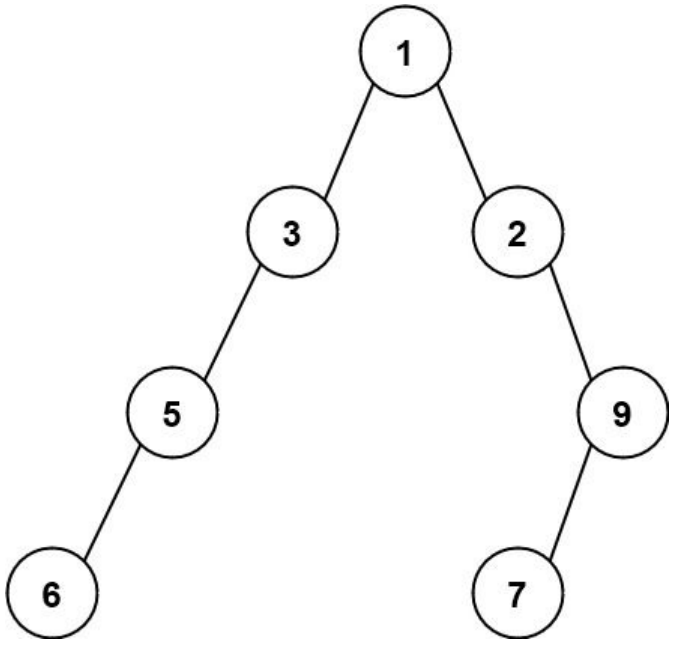
It is **guaranteed** that the answer will in the range of a **32-bit** signed integer.

Example 1:



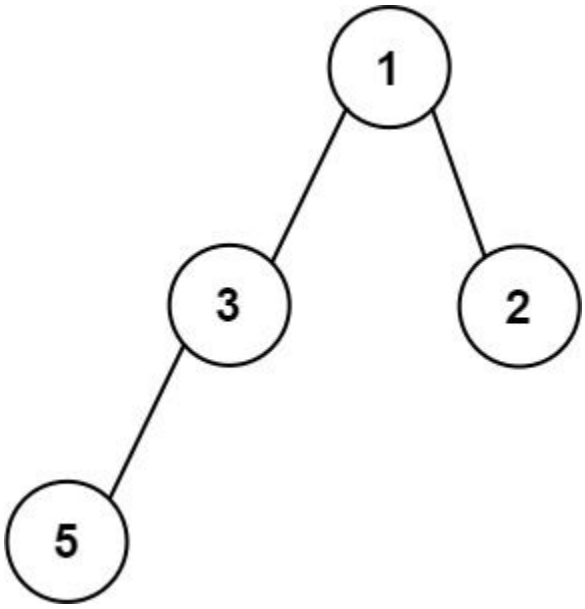

```
Input: root = [1,3,2,5,3,null,9]
Output: 4
Explanation: The maximum width exists in the third level with length 4 (5,3,null,9).
```

Example 2:




```
Input: root = [1,3,2,5,null,null,9,6,null,7]
Output: 7
Explanation: The maximum width exists in the fourth level with length 7
(6,null,null,null,null,null,7).
```

Example 3:




```
Input: root = [1,3,2,5]
Output: 2
Explanation: The maximum width exists in the second level with length 2 (3,2).
```

Constraints:

- The number of nodes in the tree is in the range `[1, 3000]`.
- `-100 <= Node.val <= 100`

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Width is the span between the leftmost and rightmost positions on a level, counting holes. Storing nodes alone hides those holes. Number the root 1, left i≪ 1, right i≪ 1|1. In BFS the width is last index minus first plus one; keep the maximum.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def widthOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        ans = 0
        q = deque([(root, 1)])
        while q:
            ans = max(ans, q[-1][1] - q[0][1] + 1)
            for _ in range(len(q)):
                root, i = q.popleft()
                if root.left:
                    q.append((root.left, i << 1))
                if root.right:
                    q.append((root.right, i << 1 | 1))
        return ans
```

Complexity. Time: O(n). Space: O(w), w = widest level.

Solution 2

How to think about it

BFS needs a queue. DFS records the first index seen at each depth and updates the width as i - t[depth] + 1 with the same numbering.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def widthOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        def dfs(root, depth, i):
            if root is None:
                return
            if len(t) == depth:
                t.append(i)
            else:
                nonlocal ans
                ans = max(ans, i - t[depth] + 1)
            dfs(root.left, depth + 1, i << 1)
            dfs(root.right, depth + 1, i << 1 | 1)

        ans = 1
        t = []
        dfs(root, 0, 1)
        return ans
```

Complexity. Time: O(n). Space: O(h).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Width counted by the nodes that really exist (no
# empty slots)
# 2. Return which level is the widest (with empty slots
# counted)


# ============================================================
# FOLLOW-UP 1: Width counted by the nodes that really
# exist (no empty slots)
# ============================================================

# ---- follow-up 1: function ----
# Answer: The width of a level is simply how many nodes it
# has: the size of the queue at the start of each level;
# keep the maximum.
# Time: O(n).  Space: O(w).
from collections import deque


def max_level_size(root):
    queue = deque([root]) if root else deque()
    best = 0
    while queue:
        size = len(queue)
        if size > best:
            best = size
        for i in range(size):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return best


# ============================================================
# FOLLOW-UP 2: Return which level is the widest (with
# empty slots counted)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Number the nodes like a heap (children 2i and 2i
# + 1), subtracting the first number of the level so the
# numbers stay small.
# Time: O(n).  Space: O(w).
from collections import deque


def widest_level(root):
    queue = deque([(root, 0)]) if root else deque()
    best_width = 0
    best_level = -1
    level = 0
    while queue:
        size = len(queue)
        first = queue[0][1]
        last = first
        for i in range(size):
            node, index = queue.popleft()
```

```
            index -= first
            last = index
```

```
            if node.left:
                queue.append((node.left, 2 * index))
            if node.right:
                queue.append((node.right, 2 * index + 1))
        if last + 1 > best_width:
            best_width = last + 1
            best_level = level
        level += 1
    return best_level, best_width
```

515. Find Largest Value in Each Tree Row

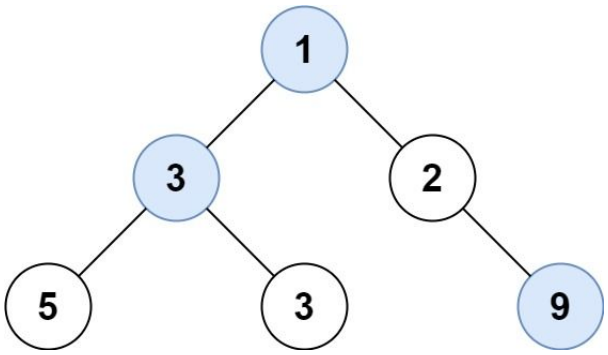
Medium · Tree, Depth-First Search, Breadth-First Search, Binary Tree

leetcode.com/problems/find-largest-value-in-each-tree-row

PROBLEM

Given the `root` of a binary tree, return *an array of the largest value in each row* of the tree **(0-indexed)**.

Example 1:




```
Input: root = [1,3,2,5,3,null,9]
Output: [1,3,9]
```

Example 2:

```
Input: root = [1,2,3]
Output: [1,3]
```

Constraints:

- The number of nodes in the tree will be in the range $[0, 10^4]$.
- $-2^{31} <= Node.val <= 2^{31} - 1$

SOLUTIONS (PYTHON 3)

Solution 1: BFS

How to think about it

The maximum of each level can be taken during a level-order walk. Enqueue the root, pop a whole level, track its maximum, and push children. Each finished level yields one answer. An empty tree returns an empty list. Time is linear in the number of nodes.

We define a queue q and put the root node into the queue. Each time, we take out all the nodes of the current level from the queue, find the maximum value, and then put all the nodes of the next level into the queue until the queue is empty.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def largestValues(self, root: Optional[TreeNode]) -> List[int]:
        ans = []
        if root is None:
            return ans
        q = deque([root])
        while q:
            x = -inf
            for _ in range(len(q)):
                node = q.popleft()
                x = max(x, node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(x)
        return ans
```

Solution 2

How to think about it

BFS is the natural level scan but needs a queue. DFS can carry the depth: append when the depth equals the answer length, otherwise take a maximum. Preorder creates a slot the first time a depth is seen. Extra space is the recursion depth; the values match BFS.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def largestValues(self, root: Optional[TreeNode]) -> List[int]:
        def dfs(root, curr):
            if root is None:
                return
            if curr == len(ans):
                ans.append(root.val)
            else:
                ans[curr] = max(ans[curr], root.val)
            dfs(root.left, curr + 1)
            dfs(root.right, curr + 1)

        ans = []
        dfs(root, 0)
        return ans
```

Complexity. Time: O(n). Space: O(h).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The SMALLEST value in each row
# 2. Largest value per row with a depth-first walk


# ============================================================
# FOLLOW-UP 1: The SMALLEST value in each row
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same level-order scan, keeping the minimum
# of each level.
# Time: O(n).  Space: O(w).
from collections import deque


def smallest_in_each_row(root):
    result = []
    queue = deque([root]) if root else deque()
    while queue:
        smallest = queue[0].val
        for i in range(len(queue)):
            node = queue.popleft()
            if node.val < smallest:
                smallest = node.val
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(smallest)
    return result


# ============================================================
# FOLLOW-UP 2: Largest value per row with a depth-first
# walk
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the biggest value seen so far for each
# depth in a list indexed by depth.
# Time: O(n).  Space: O(h).
def largest_in_each_row_dfs(root):
    best = []

    def walk(node, depth):
        if node is None:
            return
        if depth == len(best):
            best.append(node.val)
        elif node.val > best[depth]:
            best[depth] = node.val
        walk(node.left, depth + 1)
        walk(node.right, depth + 1)

    walk(root, 0)
    return best
```

513. Find Bottom Left Tree Value

Medium · Tree, Depth-First Search, Breadth-First Search, Binary Tree

leetcode.com/problems/find-bottom-left-tree-value

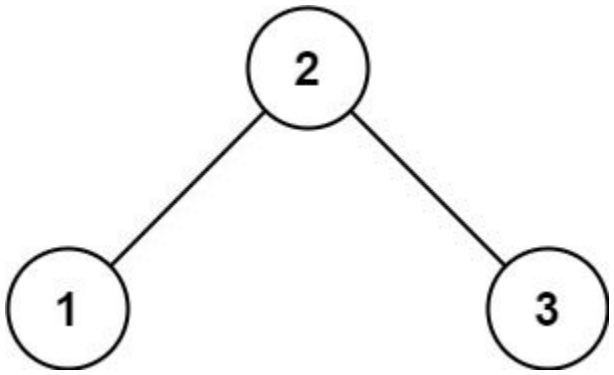
★ **TOP 7 PROBLEM** in Tree BFS (Level Order): master this one first

PROBLEM

You are given the `root` of a binary tree.

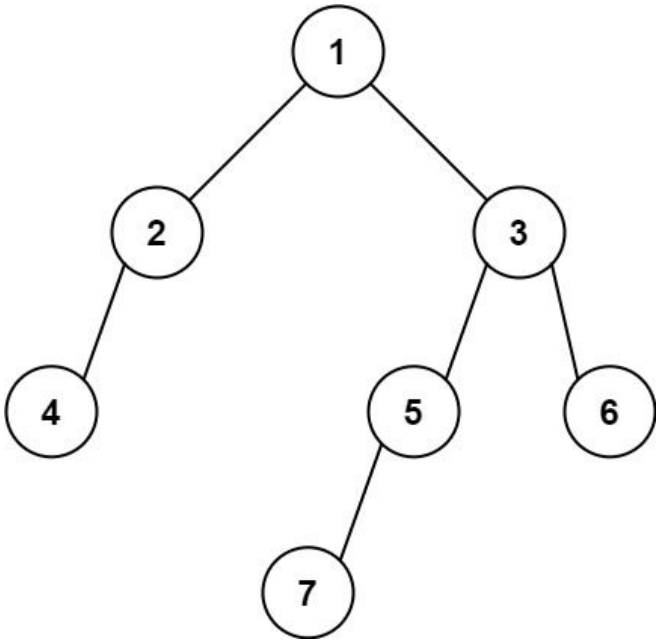
Return the **leftmost** value in the **last** row of the tree.

Example 1:




```
Input: root = [2,1,3]
Output: 1
Explanation: The last row is [1,3], so the leftmost value is 1.
```

Example 2:




```
Input: root = [1,2,3,4,null,5,6,null,null,7]
Output: 7
Explanation: The last row contains only the node 7.
```

Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.

- -2^{31} <= Node.val <= 2^{31} - 1

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

The bottom-left value is the first node of the last level in a level-order walk. In BFS the queue front is the leftmost node of the current level. Record the front at the start of each level and enqueue children. When the queue drains, the last recorded value is the answer. Level order processes the deepest level last.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findBottomLeftValue(self, root: Optional[TreeNode]) -> int:
        q = deque([root])
        ans = 0
        while q:
            ans = q[0].val
            for _ in range(len(q)):
                node = q.popleft()
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
        return ans
```

Complexity. Time: O(n). Space: O(w), w = widest level.

Solution 2

How to think about it

BFS uses an O(w) queue. DFS only needs to remember the first time a greater depth appears. A left-then-right preorder meets the leftmost node of each depth first; update the answer only when the depth is strictly larger. Extra space is the recursion stack, and the result matches level order.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findBottomLeftValue(self, root: Optional[TreeNode]) -> int:
        def dfs(root, curr):
            if root is None:
                return
            dfs(root.left, curr + 1)
            dfs(root.right, curr + 1)
            nonlocal ans, mx
            if mx < curr:
                mx = curr
                ans = root.val

        ans = mx = 0
        dfs(root, 1)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. All the values on the bottom row
# 2. The leftmost value of a given row


# ============================================================
# FOLLOW-UP 1: All the values on the bottom row
# ============================================================

# ---- follow-up 1: function ----
# Answer: The last level of the level-order traversal is
# the bottom row.
# Time: O(n).  Space: O(w).
from collections import deque


def bottom_row(root):
    last = []
    queue = deque([root]) if root else deque()
    while queue:
        last = []
        for i in range(len(queue)):
            node = queue.popleft()
            last.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return last


# ============================================================
# FOLLOW-UP 2: The leftmost value of a given row
# ============================================================

# ---- follow-up 2: function ----
# Answer: Traverse level by level and return the first
# node of the requested level.
# Time: O(n).  Space: O(w).
from collections import deque


def leftmost_of_row(root, row):
    queue = deque([root]) if root else deque()
    depth = 0
    while queue:
        if depth == row:
            return queue[0].val
        for i in range(len(queue)):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        depth += 1
    return None
```

623. Add One Row to Tree

Medium · Tree, Depth-First Search, Breadth-First Search, Binary Tree

leetcode.com/problems/add-one-row-to-tree

PROBLEM

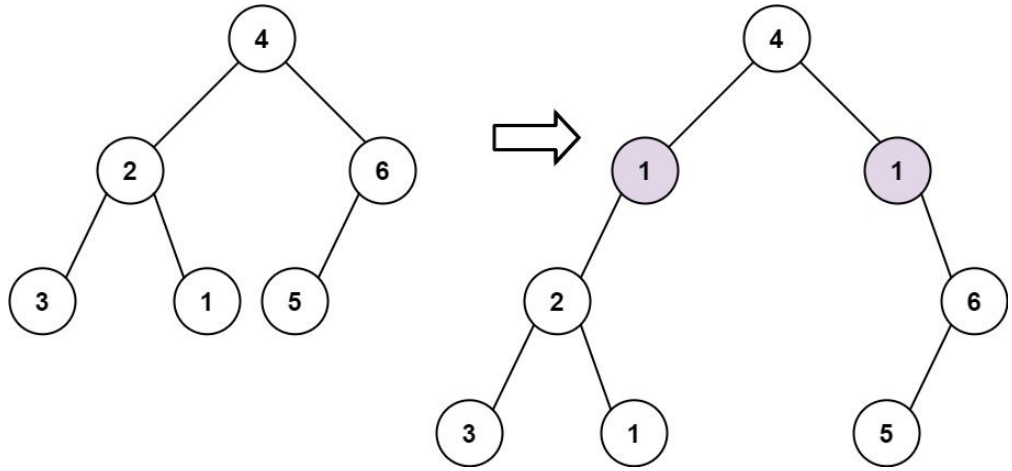
Given the `root` of a binary tree and two integers `val` and `depth`, add a row of nodes with value `val` at the given depth `depth`.

Note that the `root` node is at depth `1`.

The adding rule is:

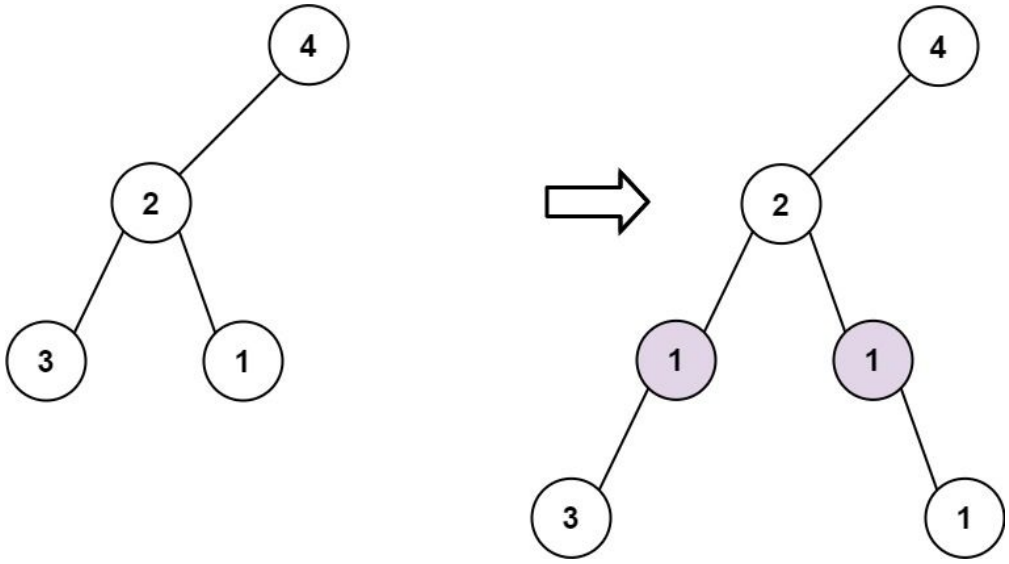
- Given the integer `depth`, for each not null tree node `cur` at the depth `depth - 1`, create two tree nodes with value `val` as `cur`'s left subtree root and right subtree root.
- `cur`'s original left subtree should be the left subtree of the new left subtree root.
- `cur`'s original right subtree should be the right subtree of the new right subtree root.
- If `depth == 1` that means there is no depth `depth - 1` at all, then create a tree node with value `val` as the new root of the whole original tree, and the original tree is the new root's left subtree.

Example 1:




```
Input: root = [4,2,6,3,1,5], val = 1, depth = 2
Output: [4,1,1,2,null,null,6,3,1,5]
```

Example 2:




```
Input: root = [4,2,null,3,1], val = 1, depth = 3
Output: [4,2,null,1,1,3,null,null,1]
```

Constraints:

- The number of nodes in the tree is in the range `[1, 10^4]`.
- The depth of the tree is in the range `[1, 10^4]`.
- `-100 <= Node.val <= 100`
- `-10^5 <= val <= 10^5`
- `1 <= depth <= the depth of tree + 1`

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

A new row must be inserted at depth depth, pushing the old nodes one level down. Rebuilding the whole tree is unnecessary. If depth=1, wrap the old root. Otherwise DFS to depth depth-1 and hang the previous children under the new nodes.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def addOneRow(
        self, root: Optional[TreeNode], val: int, depth: int
    ) -> Optional[TreeNode]:
        def dfs(root, d):
            if root is None:
                return
            if d == depth - 1:
                root.left = TreeNode(val, root.left, None)
                root.right = TreeNode(val, None, root.right)
                return
            dfs(root.left, d + 1)
            dfs(root.right, d + 1)

        if depth == 1:
            return TreeNode(val, root)
        dfs(root, 1)
        return root
```

Complexity. Time: O(n). Space: O(h).

Solution 2

> **How to think about it**
>
> DFS already attaches nodes at the parent layer. BFS reaches depth depth-1 with a queue and applies the same left/right splice, without recursion.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def addOneRow(
        self, root: Optional[TreeNode], val: int, depth: int
    ) -> Optional[TreeNode]:
        if depth == 1:
            return TreeNode(val, root)
        q = deque([root])
        i = 0
        while q:
            i += 1
            for _ in range(len(q)):
                node = q.popleft()
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
                if i == depth - 1:
                    node.left = TreeNode(val, node.left, None)
                    node.right = TreeNode(val, None, node.right)
        return root
```

Complexity. Time: O(n). Space: O(w), w = widest level.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Add a row of nodes at the BOTTOM of the tree (below
# every leaf)


# ============================================================
# FOLLOW-UP 1: Add a row of nodes at the BOTTOM of the
# tree (below every leaf)
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: Find the leaves and give each of them a new left
# and right child with the given value.
# Time: O(n).  Space: O(h).
def add_row_at_bottom(root, val):
    if root is None:
        return TreeNode(val)
    stack = [root]
    while stack:
        node = stack.pop()
        if node.left is None and node.right is None:
            node.left = TreeNode(val)
            node.right = TreeNode(val)
        else:
            if node.left:
                stack.append(node.left)
            if node.right:
                stack.append(node.right)
    return root
```

1302. Deepest Leaves Sum

Medium · Tree, Depth-First Search, Breadth-First Search, Binary Tree

leetcode.com/problems/deepest-leaves-sum

PROBLEM

Given the

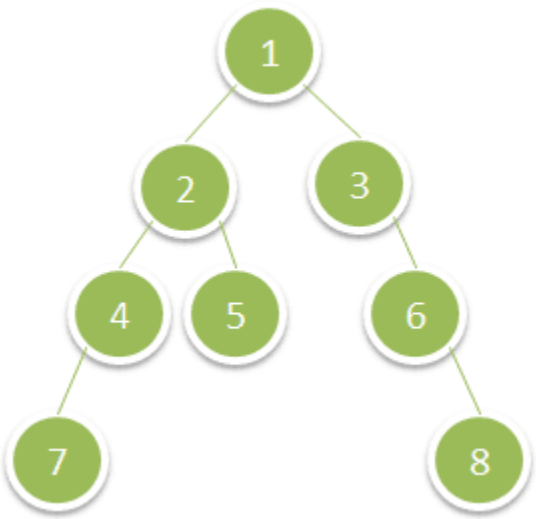
`root`

of a binary tree, return

the sum of values of its deepest leaves

.

Example 1:




```
Input: root = [1,2,3,4,5,null,6,7,null,null,null,null,8]
Output: 15
```

Example 2:

```
Input: root = [6,7,8,2,7,1,3,9,null,1,4,null,null,null,5]
Output: 19
```

Constraints:

- The number of nodes in the tree is in the range `[1, 10^4]`.
- `1 <= Node.val <= 100`

SOLUTIONS (PYTHON 3)

Solution 1: BFS

How to think about it

The sum of the deepest leaves can be obtained by a height pass plus a second walk, but two traversals are unnecessary. Level-order search already processes nodes by depth, so each layer's sum can be accumulated while dequeuing. The last nonempty layer is exactly the deepest one. We keep a queue of the current level and overwrite ans each time; when the queue is empty, ans is the answer.

We can use breadth-first search (BFS) to traverse the binary tree level by level, and calculate the sum of the node values at each level. After completing the traversal, return the sum of the node values at the last level.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def deepestLeavesSum(self, root: Optional[TreeNode]) -> int:
        q = deque([root])
        while q:
            ans = 0
            for _ in range(len(q)):
                node = q.popleft()
                ans += node.val
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
        return ans
```

Solution 2: DFS

> **How to think about it**
>
> BFS uses extra space proportional to the widest level. A recursive walk only needs the current depth: add the node when the depth matches the recorded maximum, and reset both the sum and the maximum when the depth is larger. One traversal then maintains the deepest-leaf sum.

We can use depth-first search (DFS) to recursively traverse the binary tree while keeping track of the current node's depth, the maximum depth, and the sum of the deepest leaf nodes. When visiting the current node, if the current node's depth equals the maximum depth, add the current node's value to the sum of the deepest leaf nodes. If the current node's depth is greater than the maximum depth, update the maximum depth to the current node's depth and update the sum of the deepest leaf nodes to the current node's value.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def deepestLeavesSum(self, root: Optional[TreeNode]) -> int:
        def dfs(root, i):
            nonlocal ans, mx
            if root is None:
                return
            if i == mx:
                ans += root.val
            elif i > mx:
                ans = root.val
                mx = i
            dfs(root.left, i + 1)
            dfs(root.right, i + 1)

        ans = mx = 0
        dfs(root, 1)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The lowest common ancestor of the deepest leaves
# 2. The sum of all the nodes on level k


# ============================================================
# FOLLOW-UP 1: The lowest common ancestor of the deepest
# leaves
# ============================================================

# ---- follow-up 1: function ----
# Answer: Return (depth, node) from every subtree: when
# both sides are equally deep the node itself is the
# answer, otherwise the deeper side's answer.
# Time: O(n).  Space: O(h).
def lca_of_deepest_leaves(root):
    def walk(node):
        if node is None:
            return 0, None
        left_depth, left_node = walk(node.left)
        right_depth, right_node = walk(node.right)
        if left_depth > right_depth:
            return left_depth + 1, left_node
        if right_depth > left_depth:
            return right_depth + 1, right_node
        return left_depth + 1, node

    return walk(root)[1]


# ============================================================
# FOLLOW-UP 2: The sum of all the nodes on level k
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk down level by level; add up the values of
# the requested level.
# Time: O(n).  Space: O(w).
from collections import deque
```

```
def sum_of_level(root, k):
    queue = deque([root]) if root else deque()
    depth = 0
    while queue:
        if depth == k:
            total = 0
            for node in queue:
                total += node.val
            return total
        for i in range(len(queue)):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        depth += 1
    return 0
```

988. Smallest String Starting From Leaf

Medium · Tree, Depth-First Search, String, Backtracking, Binary Tree

leetcode.com/problems/smallest-string-starting-from-leaf

PROBLEM

You are given the `root` of a binary tree where each node has a value in the range `[0, 25]` representing the letters `'a'` to `'z'`.

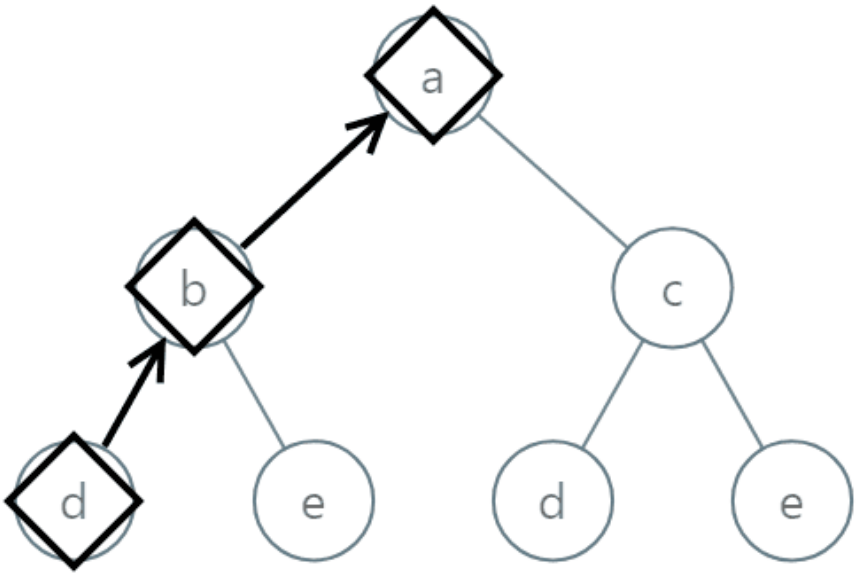
Return *the **lexicographically smallest** string that starts at a leaf of this tree and ends at the root*.

As a reminder, any shorter prefix of a string is **lexicographically smaller**.

- For example, `"ab"` is lexicographically smaller than `"aba"`.

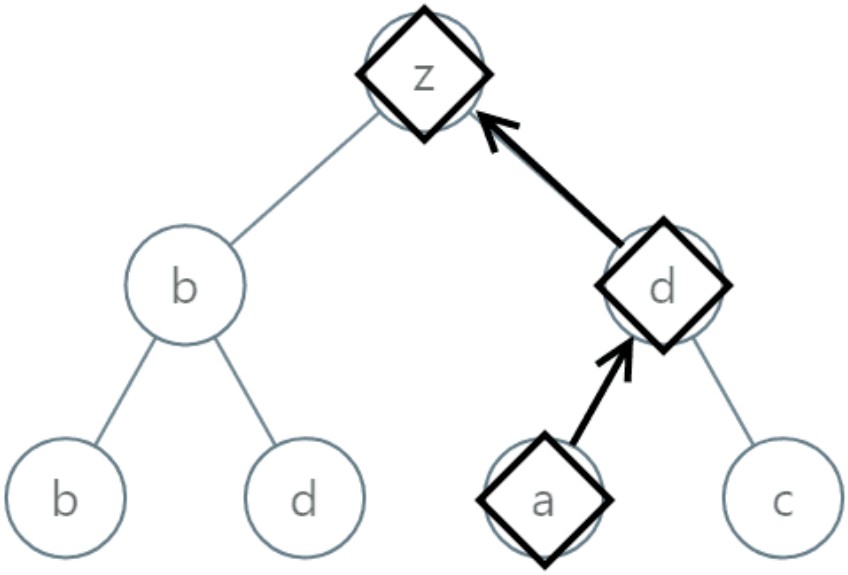
A leaf of a node is a node that has no children.

Example 1:



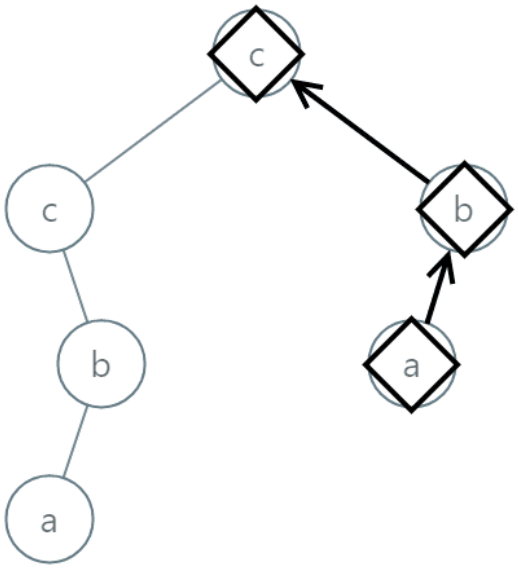

```
Input: root = [0,1,2,3,4,3,4]
Output: "dba"
```

Example 2:




```
Input: root = [25,1,3,1,3,0,2]
Output: "adz"
```

Example 3:




```
Input: root = [2,2,1,null,1,0,null,0]
Output: "abc"
```

Constraints:

- The number of nodes in the tree is in the range `[1, 8500]`.
- `0 <= Node.val <= 25`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

A leaf-to-root path is a string; we want the lexicographically smallest. At most 8500 nodes, so every root-to-leaf path can be enumerated. DFS pushes letters, and at a leaf the reversed path is compared with the answer; then the letter is popped.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def smallestFromLeaf(self, root: TreeNode) -> str:
        ans = chr(ord('z') + 1)

        def dfs(root, path):
            nonlocal ans
            if root:
                path.append(chr(ord('a') + root.val))
                if root.left is None and root.right is None:
                    ans = min(ans, ''.join(reversed(path)))
                dfs(root.left, path)
                dfs(root.right, path)
                path.pop()

        dfs(root, [])
        return ans
```

Complexity. Time: O(n * h) (a string of length up to h is built at every leaf). Space: O(h).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The LARGEST string from a leaf to the root


# ============================================================
# FOLLOW-UP 1: The LARGEST string from a leaf to the root
# ============================================================

# ---- follow-up 1: function ----
# Answer: Build the string of every leaf-to-root path and
# keep the biggest one.
# Time: O(n * h) (strings of length up to h are built).
# Space: O(h).
def largest_leaf_to_root_string(root):
    best = [None]

    def walk(node, path):
        if node is None:
            return
        path = chr(ord("a") + node.val) + path
        if node.left is None and node.right is None:
            if best[0] is None or path > best[0]:
                best[0] = path
        walk(node.left, path)
        walk(node.right, path)

    walk(root, "")
```

```
    return best[0]
```

1145. Binary Tree Coloring Game

Medium · Tree, Depth-First Search, Binary Tree

leetcode.com/problems/binary-tree-coloring-game

PROBLEM

Two players play a turn based game on a binary tree. We are given the `root` of this binary tree, and the number of nodes `n` in the tree. `n` is odd, and each node has a distinct value from `1` to `n`.

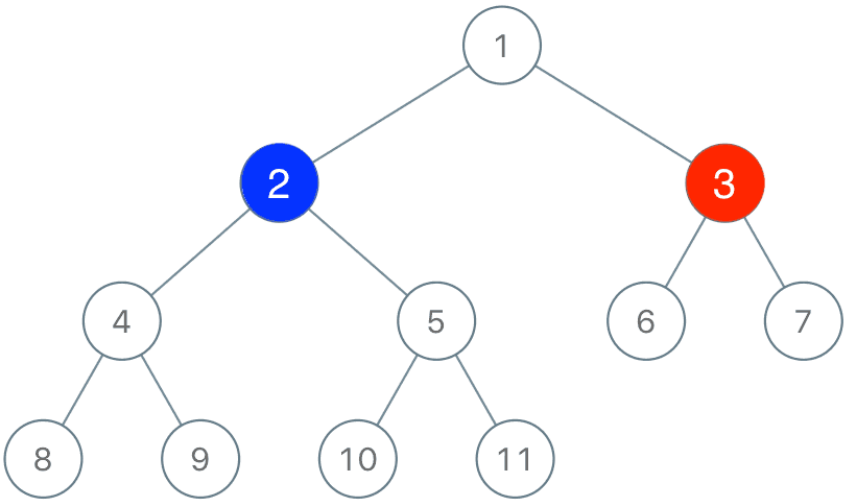
Initially, the first player names a value `x` with `1 <= x <= n`, and the second player names a value `y` with `1 <= y <= n` and `y != x`. The first player colors the node with value `x` red, and the second player colors the node with value `y` blue.

Then, the players take turns starting with the first player. In each turn, that player chooses a node of their color (red if player 1, blue if player 2) and colors an **uncolored** neighbor of the chosen node (either the left child, right child, or parent of the chosen node.)

If (and only if) a player cannot choose such a node in this way, they must pass their turn. If both players pass their turn, the game ends, and the winner is the player that colored more nodes.

You are the second player. If it is possible to choose such a `y` to ensure you win the game, return `true`. If it is not possible, return `false`.

Example 1:




```
Input: root = [1,2,3,4,5,6,7,8,9,10,11], n = 11, x = 3
Output: true
Explanation: The second player can choose the node with value 2.
```

Example 2:

```
Input: root = [1,2,3], n = 3, x = 1
Output: false
```

Constraints:

- The number of nodes in the tree is `n`.
- `1 <= x <= n <= 100`
- `n` is odd.
- 1 <= Node.val <= n

- All the values of the tree are **unique**.

SOLUTION (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> After the first player takes x, the tree splits into the left subtree, the right subtree, and the parent side. The second player colors the root of the largest of the three and owns that component. More than n/2 nodes there is a win. Finding x and counting the two subtrees yields all three sizes.

First, we use DFS to find the node where player 1's colored point x is located, denoted as node.

Next, we count the number of nodes in the left and right subtrees of node, denoted as l and r respectively, and the number of nodes in the direction of node's parent node is n - l - r - 1. As long as max(l, r, n - l - r - 1) > n/2, player 2 has a winning strategy.

The time complexity is O(n), and the space complexity is O(n). Here, n is the total number of nodes.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def btreeGameWinningMove(self, root: Optional[TreeNode], n: int, x: int) -> bool:
        def dfs(root):
            if root is None or root.val == x:
                return root
            return dfs(root.left) or dfs(root.right)

        def count(root):
            if root is None:
                return 0
            return 1 + count(root.left) + count(root.right)

        node = dfs(root)
        l, r = count(node.left), count(node.right)
        return max(l, r, n - l - r - 1) > n // 2
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The three regions around the first player's node
# (left, right and the rest)
# 2. Can the second player win? (needs a region bigger
# than half of the nodes)


# ============================================================
# FOLLOW-UP 1: The three regions around the first player's
# node (left, right and the rest)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count the nodes below the chosen node's left
# child, below its right child, and everything else (the
# part above it). The second player takes the biggest
# region.
# Time: O(n).  Space: O(h).
def count_nodes(node):
    if node is None:
```

```
        return 0
    return 1 + count_nodes(node.left) + count_nodes(node.right)


def find_node(root, value):
    if root is None or root.val == value:
        return root
    found = find_node(root.left, value)
    if found:
        return found
    return find_node(root.right, value)


def region_sizes(root, x):
    node = find_node(root, x)
    left = count_nodes(node.left)
    right = count_nodes(node.right)
    above = count_nodes(root) - left - right - 1
    return left, right, above


# ============================================================
# FOLLOW-UP 2: Can the second player win? (needs a region
# bigger than half of the nodes)
# ============================================================

# ---- follow-up 2: function ----
# Answer: The second player wins when the biggest region
# has more than n / 2 nodes.
# Time: O(n).  Space: O(h).
def second_player_wins(root, n, x):
    left, right, above = region_sizes(root, x)
    biggest = left
    if right > biggest:
        biggest = right
    if above > biggest:
        biggest = above
    return biggest > n - biggest
```

2096. Step-By-Step Directions From a Binary Tree Node to Another

Medium · Tree, Depth-First Search, String, Binary Tree, Lowest Common Ancestor, Binary Lifting

leetcode.com/problems/step-by-step-directions-from-a-binary-tree-node-to-another

PROBLEM

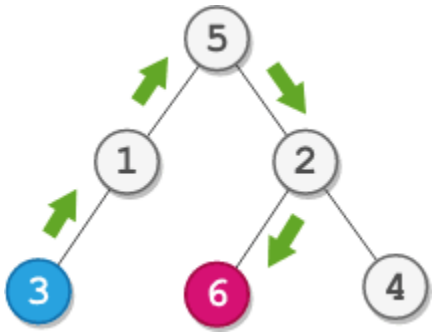
You are given the `root` of a **binary tree** with `n` nodes. Each node is uniquely assigned a value from `1` to `n`. You are also given an integer `startValue` representing the value of the start node `s`, and a different integer `destValue` representing the value of the destination node `t`.

Find the **shortest path** starting from node `s` and ending at node `t`. Generate step-by-step directions of such path as a string consisting of only the **uppercase** letters `'L'`, `'R'`, and `'U'`. Each letter indicates a specific direction:

- `'L'` means to go from a node to its **left child** node.
- `'R'` means to go from a node to its **right child** node.
- `'U'` means to go from a node to its **parent** node.

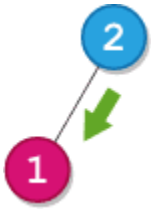
Return *the step-by-step directions of the **shortest path** from node `s` to node `t`.*

Example 1:




```
Input: root = [5,1,2,3,null,6,4], startValue = 3, destValue = 6
Output: "UURL"
Explanation: The shortest path is: 3 → 1 → 5 → 2 → 6.
```

Example 2:




```
Input: root = [2,1], startValue = 2, destValue = 1
Output: "L"
Explanation: The shortest path is: 2 → 1.
```

Constraints:

- The number of nodes in the tree is `n`.
- `2 <= n <= 10^5`
- `1 <= Node.val <= n`
- All the values in the tree are **unique**.
- `1 <= startValue, destValue <= n`

- `startValue != destValue`

SOLUTIONS (PYTHON 3)

Solution 1: Lowest Common Ancestor + DFS

> **How to think about it**
>
> The unique path goes through the LCA. Upward edges become `U`; the descent uses `L`/`R`. Three tree walks are fine for n ≤ 10^5. Find the LCA, DFS both directions from it, replace the start path by `U`s, and concatenate.

We can first find the lowest common ancestor of nodes startValue and destValue, denoted as node. Then, starting from node, we find the paths to startValue and destValue respectively. The path from startValue to node will consist of a number of Us, and the path from node to destValue will be the path. Finally, we concatenate these two paths.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right


class Solution:
    def getDirections(
        self, root: Optional[TreeNode], startValue: int, destValue: int
    ) -> str:
        def lca(node: Optional[TreeNode], p: int, q: int):
            if node is None or node.val in (p, q):
                return node
            left = lca(node.left, p, q)
            right = lca(node.right, p, q)
            if left and right:
                return node
            return left or right

        def dfs(node: Optional[TreeNode], x: int, path: List[str]):
            if node is None:
                return False
            if node.val == x:
                return True
            path.append("L")
            if dfs(node.left, x, path):
                return True
            path[-1] = "R"
            if dfs(node.right, x, path):
                return True
            path.pop()
            return False

        node = lca(root, startValue, destValue)

        path_to_start = []
        path_to_dest = []

        dfs(node, startValue, path_to_start)
        dfs(node, destValue, path_to_dest)

        return "U" * len(path_to_start) + "".join(path_to_dest)
```

Solution 2: Lowest Common Ancestor + DFS (Optimized)

> **How to think about it**
>
> Solution 1 finds the LCA then walks twice more. Paths from the root share a prefix; stripping it is exactly “up to the LCA then down.” The extra LCA search disappears. Two DFS strings, skip the common prefix of length i, emit (|start|-i) `U`s plus the destination suffix.

We can start from root, find the paths to startValue and destValue, denoted as pathToStart and pathToDest, respectively. Then, remove the longest common prefix of pathToStart and pathToDest. At this point, the length of pathToStart is the number of Us in the answer, and the path of pathToDest is the path in the answer. We just need to concatenate these two paths.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right


class Solution:
    def getDirections(
        self, root: Optional[TreeNode], startValue: int, destValue: int
    ) -> str:
        def dfs(node: Optional[TreeNode], x: int, path: List[str]):
            if node is None:
                return False
            if node.val == x:
                return True
            path.append("L")
            if dfs(node.left, x, path):
                return True
            path[-1] = "R"
            if dfs(node.right, x, path):
                return True
            path.pop()
            return False

        path_to_start = []
        path_to_dest = []

        dfs(root, startValue, path_to_start)
        dfs(root, destValue, path_to_dest)
        i = 0
        while (
            i < len(path_to_start)
            and i < len(path_to_dest)
            and path_to_start[i] == path_to_dest[i]
        ):
            i += 1
        return "U" * (len(path_to_start) - i) + "".join(path_to_dest[i:])
```

Solution 3: Explicit-Stack Lowest Common Ancestor

How to think about it

The shortest path goes through the lowest common ancestor. Every step above that ancestor is `U`, and the descent from the ancestor to the destination is `L` or `R`. Three linear walks fit $n \le 10^5$. Both the ancestor search and the direction search enter the left child first. A left chain can be n nodes long, so recursion overflows before the direction string is finished. The ancestor is known only after both subtrees return: the current node is the ancestor when both sides found a target, otherwise the non-empty side is passed upward. An explicit stack separates “expand both children” from “both children have returned,” and the direction search pushes a backtrack marker before the left child. A failed left subtree rewrites the last step to `R` and searches the right child, stopping when the target is found. The length of the start path is the number of `U`s, concatenated with the path from the ancestor to the destination.

An explicit stack finds the lowest common ancestor of startValue and destValue, denoted as node. State 0 expands the two children, and state 1 decides the ancestor after both sides return: the current node when both sides are non-empty, otherwise the non-empty side. The same stack then walks from node, recording `L` to the left and rewriting that step to `R` when the left subtree misses the target. The number of steps from startValue back to node is the number of `U`s, followed by the direction string from node to destValue.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right


class Solution:
    def getDirections(
        self, root: Optional[TreeNode], startValue: int, destValue: int
    ) -> str:
        def lca(root: Optional[TreeNode], p: int, q: int):
            ret = {}
            stk = [(root, 0)]
            while stk:
                node, state = stk.pop()
                if state == 0:
                    if node is None:
                        continue
                    if node.val in (p, q):
                        ret[id(node)] = node
                        continue
                    stk.append((node, 1))
                    stk.append((node.right, 0))
                    stk.append((node.left, 0))
                else:
                    left = ret.get(id(node.left)) if node.left is not None else None
                    right = ret.get(id(node.right)) if node.right is not None else None
                    if left and right:
                        ret[id(node)] = node
                    else:
                        ret[id(node)] = left or right
            return ret.get(id(root))

        def dfs(start: Optional[TreeNode], x: int, path: List[str]) -> bool:
            stk = [(start, 0)]
            while stk:
                node, state = stk.pop()
                if state == 0:
                    if node is None:
                        continue
                    if node.val == x:
                        return True
                    path.append('L')
                    stk.append((node, 1))
                    stk.append((node.left, 0))
                elif state == 1:
                    path[-1] = 'R'
                    stk.append((node, 2))
                    stk.append((node.right, 0))
                else:
                    path.pop()
            return False

        node = lca(root, startValue, destValue)
        path_to_start: List[str] = []
        path_to_dest: List[str] = []
        dfs(node, startValue, path_to_start)
        dfs(node, destValue, path_to_dest)
        return 'U' * len(path_to_start) + ''.join(path_to_dest)
```

Solution 4: Explicit-Stack Root Paths

How to think about it

Solution 1 still runs a separate ancestor search and then walks twice from that ancestor. Paths from the root share a prefix, and stripping it is the same as climbing back to the ancestor and then descending, so the dedicated ancestor search can be dropped. The two direction searches can still follow a chain of length n, so they use the same explicit stack: push a backtrack marker, then the left child, and rewrite the last step to `R` when the left subtree fails. After the common prefix of length i is removed, the answer is (|start|-i) `U`s plus the destination suffix.

An explicit stack walks from root to startValue and to destValue, producing pathToStart and pathToDest. A failed left subtree rewrites the last step to `R` before the right child is searched, so the strings match the previous depth-first order. After the longest common prefix is removed, the remaining length of pathToStart is the number of `U`s, and the remaining part of pathToDest is the downward path. Concatenate the two pieces.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right


class Solution:
    def getDirections(
        self, root: Optional[TreeNode], startValue: int, destValue: int
    ) -> str:
        def dfs(start: Optional[TreeNode], x: int, path: List[str]) -> bool:
            stk = [(start, 0)]
            while stk:
                node, state = stk.pop()
                if state == 0:
                    if node is None:
                        continue
                    if node.val == x:
                        return True
                    path.append('L')
                    stk.append((node, 1))
                    stk.append((node.left, 0))
                elif state == 1:
                    path[-1] = 'R'
                    stk.append((node, 2))
                    stk.append((node.right, 0))
                else:
                    path.pop()
            return False

        path_to_start: List[str] = []
        path_to_dest: List[str] = []
        dfs(root, startValue, path_to_start)
        dfs(root, destValue, path_to_dest)
        i = 0
        while (
            i < len(path_to_start)
            and i < len(path_to_dest)
            and path_to_start[i] == path_to_dest[i]
        ):
            i += 1
        return 'U' * (len(path_to_start) - i) + ''.join(path_to_dest[i:])
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Only the NUMBER of steps between the two nodes
# 2. The list of node values you pass on the way


# ============================================================
# FOLLOW-UP 1: Only the NUMBER of steps between the two
# nodes
# ============================================================

# ---- follow-up 1: function ----
# Answer: Find both root paths (strings of L and R), drop
# the common prefix; the remaining lengths added together
# are the distance.
# Time: O(n).  Space: O(h).
def path_from_root(root, value):
    path = []

    def walk(node):
        if node is None:
            return False
        if node.val == value:
            return True
        path.append("L")
        if walk(node.left):
            return True
        path.pop()
        path.append("R")
        if walk(node.right):
            return True
        path.pop()
        return False

    walk(root)
    return path


def steps_between(root, a, b):
    path_a = path_from_root(root, a)
    path_b = path_from_root(root, b)
    common = 0
    while common < len(path_a) and common < len(path_b) and path_a[common] == path_b[common]:
        common += 1
    return len(path_a) - common + len(path_b) - common


# ============================================================
# FOLLOW-UP 2: The list of node values you pass on the way
# ============================================================

# ---- follow-up 2: function ----
# Answer: Climb from the first node to the lowest common
# ancestor and then descend to the second node.
# Time: O(n).  Space: O(h).
def values_on_the_way(root, a, b):
    def root_path(value):
        result = []

        def walk(node):
            if node is None:
                return False
            result.append(node.val)
            if node.val == value:
                return True
```

```
            if walk(node.left) or walk(node.right):
                return True
            result.pop()
            return False

        walk(root)
        return result

    path_a = root_path(a)
    path_b = root_path(b)
    common = 0
    while common < len(path_a) and common < len(path_b) and path_a[common] == path_b[common]:
        common += 1
    up = path_a[common - 1:][::-1]
    down = path_b[common:]
    return up + down
```

PATTERN 12 · TREE BFS (LEVEL ORDER) · PROBLEM 19 OF 20

958. Check Completeness of a Binary Tree

Medium · Tree, Breadth-First Search, Binary Tree

leetcode.com/problems/check-completeness-of-a-binary-tree

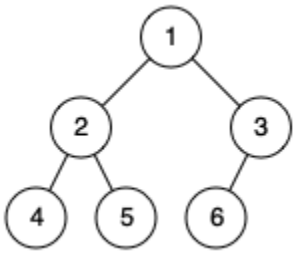
★ **TOP 7 PROBLEM** in Tree BFS (Level Order): master this one first

PROBLEM

Given the `root` of a binary tree, determine if it is a *complete binary tree*.

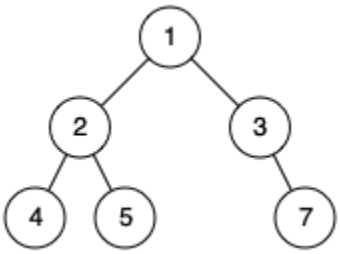
In a **complete binary tree**, every level, except possibly the last, is completely filled, and all nodes in the last level are as far left as possible. It can have between `1` and 2^h nodes inclusive at the last level `h`.

Example 1:




```
Input: root = [1,2,3,4,5,6]
Output: true
Explanation: Every level before the last is full (ie. levels with node-values {1} and {2, 3}), and all
nodes in the last level ({4, 5, 6}) are as far left as possible.
```

Example 2:




```
Input: root = [1,2,3,4,5,null,7]
Output: false
Explanation: The node with value 7 isn't as far left as possible.
```

Constraints:

- The number of nodes in the tree is in the range `[1, 100]`.
- `1 <= Node.val <= 1000`

SOLUTION (PYTHON 3)

Solution 1

> **How to think about it**
>
> In a complete tree, no non-null node may follow a null in level order. Enqueue null children as well; after the first null is popped, the rest of the queue must be null. The tree has at most 100 nodes, so one BFS decides it.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isCompleteTree(self, root: TreeNode) -> bool:
        q = deque([root])
        while q:
            node = q.popleft()
            if node is None:
                break
            q.append(node.left)
            q.append(node.right)
        return all(node is None for node in q)
```

Complexity. Time: O(n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Count the nodes of a COMPLETE tree in less than O(n)
# time
# 2. Insert a value into a complete tree so that it stays
# complete


# ============================================================
# FOLLOW-UP 1: Count the nodes of a COMPLETE tree in less
# than O(n) time
# ============================================================

# ---- follow-up 1: function ----
# Answer: Compare the height going only left with the
# height going only right: if equal the subtree is perfect
# (2^h - 1 nodes), otherwise count the two sides
# recursively. O(log squared n).
# Time: O(log^2 n).  Space: O(log n).
def count_complete_nodes(root):
    if root is None:
        return 0
    left_height = 0
    node = root
    while node:
        left_height += 1
        node = node.left
    right_height = 0
    node = root
    while node:
        right_height += 1
        node = node.right
    if left_height == right_height:
        return 2 ** left_height - 1
    return 1 + count_complete_nodes(root.left) + count_complete_nodes(root.right)
```

```
# ============================================================
# FOLLOW-UP 2: Insert a value into a complete tree so that
# it stays complete
# ============================================================

# ---- follow-up 2: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 2: function ----
# Answer: Keep the nodes in a list in level order; the
# parent of the new node is at index (size - 1) / 2 and
# the new node becomes its first free child.
# Time: constructor O(n), insert O(1).  Space: O(n).
from collections import deque


class CompleteTreeInserter:
    def __init__(self, root):
        self.root = root
        self.nodes = []
```

```
        queue = deque([root])
        while queue:
            node = queue.popleft()
            self.nodes.append(node)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

    def insert(self, val):
        new = TreeNode(val)
        parent = self.nodes[(len(self.nodes) - 1) // 2]
        if parent.left is None:
            parent.left = new
        else:
            parent.right = new
        self.nodes.append(new)
        return parent.val
```

2641. Cousins in Binary Tree II

Medium · Tree, Depth-First Search, Breadth-First Search, Hash Table, Binary Tree

leetcode.com/problems/cousins-in-binary-tree-ii

PROBLEM

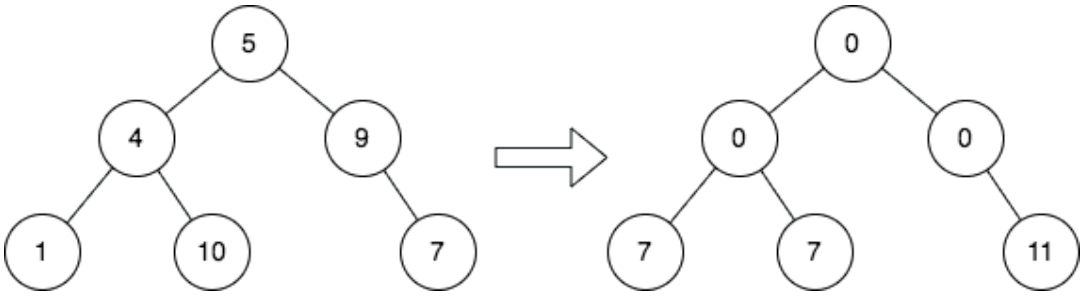
Given the `root` of a binary tree, replace the value of each node in the tree with the **sum of all its cousins' values**.

Two nodes of a binary tree are **cousins** if they have the same depth with different parents.

Return *the* `root` *of the modified tree*.

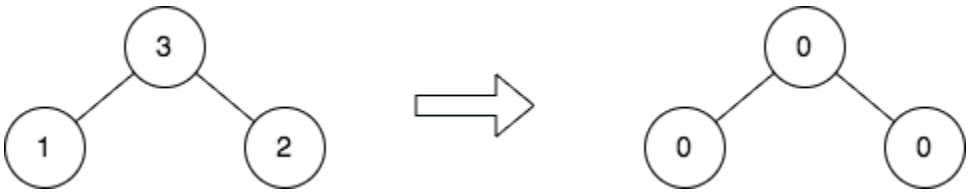
Note that the depth of a node is the number of edges in the path from the root node to it.

Example 1:




```
Input: root = [5,4,9,1,10,null,7]
Output: [0,0,0,7,7,null,11]
Explanation: The diagram above shows the initial binary tree and the binary tree after changing the
value of each node.
- Node with value 5 does not have any cousins so its sum is 0.
- Node with value 4 does not have any cousins so its sum is 0.
- Node with value 9 does not have any cousins so its sum is 0.
- Node with value 1 has a cousin with value 7 so its sum is 7.
- Node with value 10 has a cousin with value 7 so its sum is 7.
- Node with value 7 has cousins with values 1 and 10 so its sum is 11.
```

Example 2:




```
Input: root = [3,1,2]
Output: [0,0,0]
Explanation: The diagram above shows the initial binary tree and the binary tree after changing the
value of each node.
- Node with value 3 does not have any cousins so its sum is 0.
- Node with value 1 does not have any cousins so its sum is 0.
- Node with value 2 does not have any cousins so its sum is 0.
```

Constraints:

- The number of nodes in the tree is in the range $[1, 10^5]$.
- $1 <= Node.val <= 10^4$

SOLUTIONS (PYTHON 3)

Solution 1: Two DFS Traversals

> **How to think about it**
>
> Each node becomes the sum of its level minus itself and its sibling. Rescanning a level per node is too slow for $n \le 10^5$. The level sum depends only on depth, so it can be computed once. The first DFS fills s[depth]; the second writes, at the parent, the child-level total minus the two siblings. The root has no cousins and becomes 0.

We create a list s to record the sum of the node values at each level of the binary tree, where s[depth] represents the sum of the node values at the depth-th level (the root node is at level 0).

Next, we perform a DFS traversal to calculate the values in the array s. Then, we perform another DFS traversal to update the values of each node's children. The value of a child node is equal to the sum of the node values at its level minus the value of the child node and its sibling nodes.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def replaceValueInTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        def dfs1(root: Optional[TreeNode], depth: int):
            if root is None:
                return
            if len(s) <= depth:
                s.append(0)
            s[depth] += root.val
            dfs1(root.left, depth + 1)
            dfs1(root.right, depth + 1)

        def dfs2(root: Optional[TreeNode], depth: int):
            sub = (root.left.val if root.left else 0) + (
                root.right.val if root.right else 0
            )
            depth += 1
            if root.left:
                root.left.val = s[depth] - sub
                dfs2(root.left, depth)
            if root.right:
                root.right.val = s[depth] - sub
                dfs2(root.right, depth)

        s = []
        dfs1(root, 0)
        root.val = 0
        dfs2(root, 0)
        return root
```

Solution 2: Breadth-First Search (BFS)

> **How to think about it**
>
> Solution 1 uses two recursive passes and stores every level sum. In a BFS the current layer is already queued, so we sum the next layer, then subtract sibling pairs at each parent, dropping the extra array and the second DFS.

First, we update the root node's value to 0, and use a queue q to store all nodes at each level, initially enqueueing the root node.

Then, we traverse the queue, calculate the sum s of all child nodes' values at each level, then calculate the sum sub of each child node and its sibling nodes' values, and then update each child node's value to s - sub.

After the traversal ends, we return the root node.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def replaceValueInTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        root.val = 0
        q = [root]
        while q:
            t = []
            s = 0
            for node in q:
                if node.left:
                    t.append(node.left)
                    s += node.left.val
                if node.right:
                    t.append(node.right)
                    s += node.right.val
            for node in q:
                sub = (node.left.val if node.left else 0) + (
                    node.right.val if node.right else 0
                )
                if node.left:
                    node.left.val = s - sub
                if node.right:
                    node.right.val = s - sub
            q = t
        return root
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Replace every value with the sum of its SIBLINGS (the
# other child of the same parent)


# ============================================================
# FOLLOW-UP 1: Replace every value with the sum of its
# SIBLINGS (the other child of the same parent)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Process the tree top-down: when a parent is
# handled, read the old values of both children first,
# then give each child the old value of the other one. The
# root has no sibling.
# Time: O(n).  Space: O(w).
from collections import deque


def replace_with_sibling_sum(root):
    if root is None:
        return None
    root.val = 0
    queue = deque([root])
    while queue:
        node = queue.popleft()
```

```
        left_value = node.left.val if node.left else 0
        right_value = node.right.val if node.right else 0
        if node.left:
            node.left.val = right_value
            queue.append(node.left)
        if node.right:
            node.right.val = left_value
            queue.append(node.right)
    return root
```

PATTERN 13 OF 41

Dynamic Programming - 1D

20 problems · 1 Easy · 19 Medium · 0 Hard

THE IDEA

The answer for position i depends on a few earlier positions. Define what dp[i] means in one sentence, write the recurrence, fill left to right, and often keep only the last two or three values.

Signals: climbing stairs, house robber, decode ways, maximum subarray, longest increasing subsequence, word break, coin change as counting.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
dp = [0] * (n + 1)
dp[0] = 1                              # base case
for i in range(1, n + 1):
    dp[i] = dp[i - 1]                  # take one step
    if i >= 2:
        dp[i] += dp[i - 2]             # or two steps
return dp[n]
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Climbing Stairs	Easy	LC 70	674
2	★ **House Robber**	Medium	LC 198	677
3	House Robber II	Medium	LC 213	680
4	Maximum Subarray (see page)	Medium	LC 53	258
5	★ **Coin Change**	Medium	LC 322	683
6	★ **Longest Increasing Subsequence**	Medium	LC 300	686
7	★ **Word Break** (see page)	Medium	LC 139	323
8	★ **Decode Ways** (see page)	Medium	LC 91	316
9	Jump Game	Medium	LC 55	689
10	★ **Maximum Product Subarray**	Medium	LC 152	692
11	★ **Best Time to Buy and Sell Stock with Cooldown**	Medium	LC 309	694
12	Partition Equal Subset Sum	Medium	LC 416	698
13	Coin Change II	Medium	LC 518	701
14	Combination Sum IV	Medium	LC 377	704
15	Perfect Squares	Medium	LC 279	707
16	Wiggle Subsequence	Medium	LC 376	710
17	Number of Longest Increasing Subsequence	Medium	LC 673	712
18	Minimum Cost For Tickets	Medium	LC 983	716
19	Delete and Earn	Medium	LC 740	719
20	Maximum Length of Repeated Subarray	Medium	LC 718	722

PATTERN 13 · DYNAMIC PROGRAMMING - 1D · PROBLEM 1 OF 20

70. Climbing Stairs

Easy · Memoization, Math, Dynamic Programming

leetcode.com/problems/climbing-stairs

PROBLEM

You are climbing a staircase. It takes `n` steps to reach the top.

Each time you can either climb `1` or `2` steps. In how many distinct ways can you climb to the top?

Example 1:

```
Input: n = 2
Output: 2
Explanation: There are two ways to climb to the top.
1. 1 step + 1 step
2. 2 steps
```

Example 2:

```
Input: n = 3
Output: 3
Explanation: There are three ways to climb to the top.
1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step
```

Constraints:

- `1 <= n <= 45`

SOLUTIONS (PYTHON 3)

Solution 1: Recursion

How to think about it

The first idea is recursion: the last step is 1 or 2, so f(n) = f(n-1) + f(n-2). n ≤ 45, and a naive recursion re-expands the same subproblems, which explodes. The bottleneck is overlap. This is the Fibonacci sequence, so we can compute it bottom-up. f[i] depends only on the previous two terms, so two rolling variables suffice: O(n) time and O(1) extra space.

We define f[i] to represent the number of ways to climb to the i-th step, then f[i] can be transferred from f[i - 1] and f[i - 2], that is:

$$f[i] = f[i - 1] + f[i - 2]$$

The initial conditions are f[0] = 1 and f[1] = 1, that is, the number of ways to climb to the 0th step is 1, and the number of ways to climb to the 1st step is also 1.

The answer is f[n].

Since f[i] is only related to f[i - 1] and f[i - 2], we can use two variables a and b to maintain the current number of ways, reducing the space complexity to O(1).

The time complexity is O(n), and the space complexity is O(1).

Python 3

```
class Solution:
    def climbStairs(self, n: int) -> int:
        a, b = 0, 1
        for _ in range(n):
            a, b = b, a + b
        return b
```

Solution 2: Matrix Quick Power to Accelerate Recursion

> **How to think about it**
>
> Solution 1 is already O(n), which is enough for n ≤ 45, but a linear recurrence is still slow when n grows. A linear homogeneous recurrence is matrix multiplication: F_n comes from F_{n-1} times a fixed 2 × 2 transition. Fast matrix exponentiation compresses n steps into O(log n). Constants are larger; use it when the same recurrence needs to be sped up.

We set Fib(n) to represent a 1 × 2 matrix bmatrix F_n & F_{n-1} bmatrix, where F_n and F_{n-1} are the n-th and (n - 1)-th Fibonacci numbers respectively.

We hope to derive Fib(n) based on Fib(n-1) = bmatrix F_{n-1} & F_{n-2} bmatrix. That is to say, we need a matrix base, so that Fib(n - 1) × base = Fib(n), that is:

$$\text{bmatrix } F_{n-1} \text{ \& } F_{n-2} \text{ bmatrix} \times \text{base} = \text{bmatrix } F_n \text{ \& } F_{n-1} \text{ bmatrix}$$

Since $F_n = F_{n-1} + F_{n-2}$, the first column of the matrix base is:

$$\text{bmatrix } 1 \backslash 1 \text{ bmatrix}$$

The second column is:

$$\text{bmatrix } 1 \backslash 0 \text{ bmatrix}$$

Therefore:

$$\text{bmatrix } F_{n-1} \text{ \& } F_{n-2} \text{ bmatrix} \times \text{bmatrix1 \& 1} \backslash \text{1 \& 0bmatrix} = \text{bmatrix } F_n \text{ \& } F_{n-1} \text{ bmatrix}$$

We define the initial matrix res = bmatrix 1 & 1 bmatrix, then F_n is equal to the first element of the first row of the result matrix of res multiplied by base^{n-1}. We can solve it using matrix quick power.

The time complexity is O(log n), and the space complexity is O(1).

Python 3

```
import numpy as np


class Solution:
    def climbStairs(self, n: int) -> int:
        res = np.asmatrix([(1, 1)], np.dtype("O"))
        factor = np.asmatrix([(1, 1), (1, 0)], np.dtype("O"))
        n -= 1
        while n:
            if n & 1:
                res *= factor
            factor *= factor
            n >>= 1
        return res[0, 0]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. You may climb 1, 2, ... or k steps at a time
# 2. Some stairs are broken and cannot be stepped on


# ============================================================
# FOLLOW-UP 1: You may climb 1, 2, ... or k steps at a
# time
# ============================================================

# ---- follow-up 1: function ----
# Answer: ways[i] is the sum of the last k values of ways:
# keep a running sum of a window of size k instead of
# adding k numbers each time.
# Time: O(n).  Space: O(n).
def climb_stairs_k_steps(n, k):
    ways = [0] * (n + 1)
    ways[0] = 1
    window = 1                                          # sum of ways[i - k .. i - 1]
    for i in range(1, n + 1):
        ways[i] = window
        window += ways[i]
        if i - k >= 0:
            window -= ways[i - k]
    return ways[n]


# ============================================================
# FOLLOW-UP 2: Some stairs are broken and cannot be
# stepped on
# ============================================================

# ---- follow-up 2: function ----
# Answer: Set the number of ways to reach a broken stair
# to 0; every other stair adds the ways of the two stairs
# below.
# Time: O(n).  Space: O(n).
def climb_stairs_with_broken(n, broken):
    broken = set(broken)
    ways = [0] * (n + 1)
    ways[0] = 1
    for i in range(1, n + 1):
        if i in broken:
            continue
        ways[i] = ways[i - 1]
        if i >= 2:
            ways[i] += ways[i - 2]
    return ways[n]
```

PATTERN 13 · DYNAMIC PROGRAMMING - 1D · PROBLEM 2 OF 20

198. House Robber

Medium · Array, Dynamic Programming

leetcode.com/problems/house-robber

★ **TOP 7 PROBLEM** in Dynamic Programming - 1D: master this one first

PROBLEM

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and **it will automatically contact the police if two adjacent houses were broken into on the same night**.

Given an integer array `nums` representing the amount of money of each house, return *the maximum amount of money you can rob tonight **without alerting the police***.

Example 1:

```
Input: nums = [1,2,3,1]
Output: 4
Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
Total amount you can rob = 1 + 3 = 4.
```

Example 2:

```
Input: nums = [2,7,9,3,1]
Output: 12
Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1).
Total amount you can rob = 2 + 9 + 1 = 12.
```

Constraints:

- `1 <= nums.length <= 100`
- `0 <= nums[i] <= 400`

SOLUTIONS (PYTHON 3)

Solution 1: Memoization Search

How to think about it

Adjacent houses cannot both be robbed; maximize the take. Subset enumeration must enforce the gap, and overlapping suffixes repeat. From index i, rob and skip to i+2, or skip to i+1, and take the better. Memoization evaluates each index once.

We design a function dfs(i), which represents the maximum amount of money that can be stolen starting from the i-th house. Thus, the answer is dfs(0).

The execution process of the function dfs(i) is as follows:

- If i ≥ len(nums), it means all houses have been considered, and we directly return 0;
- Otherwise, consider stealing from the i-th house, then dfs(i) = nums[i] + dfs(i+2); if not stealing from the i-th house, then dfs(i) = dfs(i+1).
- Return max(nums[i] + dfs(i+2), dfs(i+1)).

To avoid repeated calculations, we use memoization search. The result of dfs(i) is saved in an array or hash table. Before each calculation, we first check if it has been calculated. If so, we directly return the result.

The time complexity is O(n), and the space complexity is O(n), where n is the length of the array.

Python 3

```python
class Solution:
    def rob(self, nums: List[int]) -> int:
        @cache
        def dfs(i: int) -> int:
            if i >= len(nums):
                return 0
            return max(nums[i] + dfs(i + 2), dfs(i + 1))

        return dfs(0)
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> Solution 1 is recursive. f[i] is the best total among the first i houses: rob house i and add f[i-2], or take f[i-1]. Fill the table bottom-up.

We define f[i] as the maximum total amount that can be robbed from the first i houses, initially f[0]=0, f[1]=nums[0].

Consider the case where i > 1, the ith house has two options:

- Do not rob the ith house, the total amount of robbery is f[i-1];
- Rob the ith house, the total amount of robbery is f[i-2]+nums[i-1];

Therefore, we can get the state transition equation:

$$f[i]= \text{cases } 0, \& \ i=0 \ \backslash \ nums[0], \& \ i=1 \ \backslash \ \max(f[i-1],f[i-2]+nums[i-1]), \& \ i > 1 \text{ cases}$$

The final answer is f[n], where n is the length of the array.

The time complexity is O(n), and the space complexity is O(n). Where n is the length of the array.

Python 3

```python
class Solution:
    def rob(self, nums: List[int]) -> int:
        n = len(nums)
        f = [0] * (n + 1)
        f[1] = nums[0]
        for i in range(2, n + 1):
            f[i] = max(f[i - 1], f[i - 2] + nums[i - 1])
        return f[n]
```

Solution 3: Dynamic Programming (Space Optimization)

> **How to think about it**
>
> f[i] in Solution 2 depends only on the previous two values, so two rolling variables cut space to O(1).

We notice that when i > 2, f[i] is only related to f[i-1] and f[i-2]. Therefore, we can use two variables instead of an array to reduce the space complexity to O(1).

Python 3

```python
class Solution:
    def rob(self, nums: List[int]) -> int:
        f = g = 0
        for x in nums:
            f, g = max(f, g), f + x
        return max(f, g)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return WHICH houses are robbed
# 2. Two robbed houses must be at least k apart


# ============================================================
# FOLLOW-UP 1: Return WHICH houses are robbed
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the whole table of best totals, then walk
# back from the end: if skipping the house gives the same
# total as the previous row, it was not robbed; otherwise
# rob it and jump two houses back.
# Time: O(n).  Space: O(n).
def houses_robbed(nums):
    n = len(nums)
    best = [0] * (n + 1)                            # best[i] = best total using the first i houses
    for i in range(1, n + 1):
        take = nums[i - 1] + (best[i - 2] if i >= 2 else 0)
        best[i] = take if take > best[i - 1] else best[i - 1]
    chosen = []
    i = n
    while i >= 1:
        take = nums[i - 1] + (best[i - 2] if i >= 2 else 0)
        if take >= best[i - 1] and take == best[i]:
            chosen.append(i - 1)
            i -= 2
        else:
            i -= 1
    chosen.reverse()
    return best[n], chosen


# ============================================================
# FOLLOW-UP 2: Two robbed houses must be at least k apart
# ============================================================

# ---- follow-up 2: function ----
# Answer: best[i] = the better of skipping house i, or
# robbing it and adding the best total of the houses up to
# i - k.
# Time: O(n).  Space: O(n).
def rob_at_least_k_apart(nums, k):
    n = len(nums)
    best = [0] * (n + 1)
    for i in range(1, n + 1):
        skip = best[i - 1]
        before = i - k if i - k > 0 else 0
        take = nums[i - 1] + best[before]
        best[i] = take if take > skip else skip
    return best[n]
```

213. House Robber II

Medium · Array, Dynamic Programming

leetcode.com/problems/house-robber-ii

PROBLEM

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are **arranged in a circle.** That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have a security system connected, and **it will automatically contact the police if two adjacent houses were broken into on the same night**.

Given an integer array `nums` representing the amount of money of each house, return *the maximum amount of money you can rob tonight **without alerting the police***.

Example 1:

```
Input: nums = [2,3,2]
Output: 3
Explanation: You cannot rob house 1 (money = 2) and then rob house 3 (money = 2), because they are
adjacent houses.
```

Example 2:

```
Input: nums = [1,2,3,1]
Output: 4
Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
Total amount you can rob = 1 + 3 = 4.
```

Example 3:

```
Input: nums = [1,2,3]
Output: 3
```

Constraints:

- `1 <= nums.length <= 100`
- `0 <= nums[i] <= 1000`

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

> **How to think about it**
>
> Houses form a circle, so the first and last cannot both be robbed, and the linear recurrence does not apply directly. We split into two linear instances—drop the first house, or drop the last—and take the better of the two. A single house is returned as-is.

The circular arrangement means that at most one of the first and last houses can be chosen for theft, so this circular arrangement problem can be reduced to two single-row house problems.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Python 3

```python
class Solution:
    def rob(self, nums: List[int]) -> int:
        def _rob(nums):
            f = g = 0
            for x in nums:
                f, g = max(f, g), f + x
            return max(f, g)

        if len(nums) == 1:
            return nums[0]
        return max(_rob(nums[1:]), _rob(nums[:-1]))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Houses stand in a TREE (a robbed house blocks its
# parent and children)
# 2. Return the houses robbed on the circle


# ============================================================
# FOLLOW-UP 1: Houses stand in a TREE (a robbed house
# blocks its parent and children)
# ============================================================

# ---- follow-up 1: function ----
# Answer: For every node return two numbers: the best
# total if the node is robbed and if it is not. If robbed,
# its children must be skipped; if not, each child may be
# robbed or skipped independently.
# Time: O(n).  Space: O(h).
def rob_tree(root):
    def walk(node):
        # returns (best if node is robbed, best if node is not robbed)
        if node is None:
            return 0, 0
        left_robbed, left_free = walk(node.left)
        right_robbed, right_free = walk(node.right)
        robbed = node.val + left_free + right_free
        left_best = left_robbed if left_robbed > left_free else left_free
        right_best = right_robbed if right_robbed > right_free else right_free
        return robbed, left_best + right_best

    robbed, free = walk(root)
    return robbed if robbed > free else free


# ============================================================
# FOLLOW-UP 2: Return the houses robbed on the circle
# ============================================================

# ---- follow-up 2: helper functions ----
def rob_line_with_houses(nums, offset):
    n = len(nums)
    best = [0] * (n + 1)
    for i in range(1, n + 1):
        take = nums[i - 1] + (best[i - 2] if i >= 2 else 0)
        best[i] = take if take > best[i - 1] else best[i - 1]
    chosen = []
    i = n
    while i >= 1:
        take = nums[i - 1] + (best[i - 2] if i >= 2 else 0)
        if take == best[i] and take >= best[i - 1]:
            chosen.append(i - 1 + offset)
            i -= 2
```

```
        else:
            i -= 1
    chosen.reverse()
    return best[n], chosen

# ---- follow-up 2: function ----
# Answer: Run the line version twice (without the first
# house and without the last house) and keep the better
# answer together with its houses.
```

```
# Time: O(n).  Space: O(n).
def houses_robbed_on_circle(nums):
    if len(nums) == 1:
        return nums[0], [0]
    without_first = rob_line_with_houses(nums[1:], 1)
    without_last = rob_line_with_houses(nums[:-1], 0)
    if without_first[0] >= without_last[0]:
        return without_first
    return without_last
```

PATTERN 13 · DYNAMIC PROGRAMMING - 1D · PROBLEM 5 OF 20

322. Coin Change

Medium · Breadth-First Search, Array, Dynamic Programming, Knapsack, Unbounded Knapsack

leetcode.com/problems/coin-change

★ **TOP 7 PROBLEM** in Dynamic Programming - 1D, Dynamic Programming - Knapsack Style: master this one first

Also appears in pattern 28 Dynamic Programming - Knapsack Style

PROBLEM

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return the **fewest** number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return -1.

You may assume that you have an infinite number of each kind of coin.

Example 1:

```
Input: coins = [1,2,5], amount = 11
Output: 3
Explanation: 11 = 5 + 5 + 1
```

Example 2:

```
Input: coins = [2], amount = 3
Output: -1
```

Example 3:

```
Input: coins = [1], amount = 0
Output: 0
```

Constraints:

- `1 <= coins.length <= 12`
- `1 <= coins[i] <= 2^31 - 1`
- `0 <= amount <= 10^4`

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming (Complete Knapsack)

How to think about it

Each coin may be used any number of times; we want the fewest coins summing to amount. This is unbounded knapsack. A raw DFS repeats sub-amounts. Let f[i][j] be the fewest coins using the first i types to make j. Skip type i via f[i-1][j]; if j≥ x, take f[i][j-x]+1. f[0][0]=0, the rest are ∞; return -1 if impossible.

We define f[i][j] as the minimum number of coins needed to make up the amount j using the first i types of coins. Initially, f[0][0] = 0, and the values of other positions are all positive infinity.

We can enumerate the quantity k of the last coin used, then we have:

$$f[i][j] = \min(f[i - 1][j], f[i - 1][j - x] + 1, \ldots, f[i - 1][j - k \times x] + k)$$

where x represents the face value of the i-th type of coin.

Let j = j - x, then we have:

$$f[i][j - x] = \min(f[i - 1][j - x], f[i - 1][j - 2 \times x] + 1, \ldots, f[i - 1][j - k \times x] + k - 1)$$

Substituting the second equation into the first one, we can get the following state transition equation:

$$f[i][j] = \min(f[i - 1][j], f[i][j - x] + 1)$$

The final answer is f[m][n].

The time complexity is O(m × n), and the space complexity is O(m × n). Where m and n are the number of types of coins and the total amount, respectively.

Python 3

```python
class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        m, n = len(coins), amount
        f = [[inf] * (n + 1) for _ in range(m + 1)]
        f[0][0] = 0
        for i, x in enumerate(coins, 1):
            for j in range(n + 1):
                f[i][j] = f[i - 1][j]
                if j >= x:
                    f[i][j] = min(f[i][j], f[i][j - x] + 1)
        return -1 if f[m][n] >= inf else f[m][n]
```

Solution 2: Optimized Dynamic Programming

> **How to think about it**
>
> Method 1 only reads f[i-1][j] and f[i][j-x]. Collapse to one array and iterate amounts upward so f[j-x] already includes the current coin. Space becomes O(amount) with the same recurrence.

We notice that f[i][j] is only related to f[i - 1][j] and f[i][j - x]. Therefore, we can optimize the two-dimensional array into a one-dimensional array, reducing the space complexity to O(n). The time complexity remains O(m × n).

Similar problems:

- 279. Perfect Squares

Python 3

```python
class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        n = amount
        f = [0] + [inf] * n
        for x in coins:
            for j in range(x, n + 1):
                f[j] = min(f[j], f[j - x] + 1)
        return -1 if f[n] >= inf else f[n]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the coins that make the amount
# 2. Each coin value has a LIMITED number of copies


# ============================================================
# FOLLOW-UP 1: Return the coins that make the amount
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember for every amount which coin gave its
# best count, then walk back from the target amount.
# Time: O(amount * c), c = number of coins.  Space:
# O(amount).
def coins_for_amount(coins, amount):
    big = amount + 1
    best = [big] * (amount + 1)
```

```
    last_coin = [0] * (amount + 1)
    best[0] = 0
    for a in range(1, amount + 1):
        for coin in coins:
            if coin <= a and best[a - coin] + 1 < best[a]:
                best[a] = best[a - coin] + 1
                last_coin[a] = coin
    if best[amount] == big:
        return None
    used = []
    while amount > 0:
        used.append(last_coin[amount])
        amount -= last_coin[amount]
    return used


# ============================================================
# FOLLOW-UP 2: Each coin value has a LIMITED number of
# copies
# ============================================================

# ---- follow-up 2: function ----
# Answer: Process the coin types one by one; for each type
# try using 0, 1, ... up to its available copies on top of
# the best results of the earlier types.
# Time: O(amount * c * k), k = copies per coin.  Space:
# O(amount).
def coin_change_limited(coins, counts, amount):
    big = amount + 1
    best = [big] * (amount + 1)
    best[0] = 0
    for k in range(len(coins)):
        updated = list(best)
        for a in range(amount + 1):
            for used in range(1, counts[k] + 1):
                if used * coins[k] > a:
                    break
                if best[a - used * coins[k]] + used < updated[a]:
                    updated[a] = best[a - used * coins[k]] + used
        best = updated
    if best[amount] >= big:
        return -1
    return best[amount]
```

PATTERN 13 · DYNAMIC PROGRAMMING - 1D · PROBLEM 6 OF 20

300. Longest Increasing Subsequence

Medium · Array, Binary Search, Dynamic Programming, Longest Increasing Subsequence

leetcode.com/problems/longest-increasing-subsequence

★ **TOP 7 PROBLEM** in Dynamic Programming - 1D: master this one first

Also appears in pattern 41 Binary Indexed Tree (Fenwick Tree)

PROBLEM

Given an integer array `nums`, return *the length of the longest **strictly increasing subsequence***.

Example 1:

```
Input: nums = [10,9,2,5,3,7,101,18]
Output: 4
Explanation: The longest increasing subsequence is [2,3,7,101], therefore the length is 4.
```

Example 2:

```
Input: nums = [0,1,0,3,2,3]
Output: 4
```

Example 3:

```
Input: nums = [7,7,7,7,7,7,7]
Output: 1
```

Constraints:

- `1 <= nums.length <= 2500`
- -10^4 `<= nums[i] <=` 10^4

Follow up: Can you come up with an algorithm that runs in `O(n log(n))` time complexity?

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

A subsequence must keep relative order and increase strictly. Enumerating all subsequences explodes with n; n ≤ 2500 still allows an $O(n^2)$ transfer by ending index. The LIS ending at nums[i] depends only on smaller values to its left. Define f[i] as that length, and for each j < i with nums[j] < nums[i] take max(f[j]+1). A singleton has length 1, and the answer is max f.

Python 3

```python
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        n = len(nums)
        f = [1] * n
        for i in range(1, n):
            for j in range(i):
                if nums[j] < nums[i]:
                    f[i] = max(f[i], f[j] + 1)
        return max(f)
```

Solution 2

How to think about it

Method 1 scans the left side linearly for the best f[j] below x, which costs $O(n^2)$. After discretizing values, that query is a prefix maximum on ranks, which a Fenwick tree maintains in O(log n). Scan each x in original order: query the prefix max strictly below x, then write x. Only left-hand information is used, so the recurrence matches Method 1 in O(n log n) time.

Python 3

```python
class BinaryIndexedTree:
    def __init__(self, n: int):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x: int, v: int):
        while x <= self.n:
            self.c[x] = max(self.c[x], v)
            x += x & -x

    def query(self, x: int) -> int:
        mx = 0
        while x:
            mx = max(mx, self.c[x])
            x -= x & -x
        return mx


class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        s = sorted(set(nums))
        m = len(s)
        tree = BinaryIndexedTree(m)
        for x in nums:
            x = bisect_left(s, x) + 1
            t = tree.query(x - 1) + 1
            tree.update(x, t)
        return tree.query(m)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. O(n log n): patience sorting
# 2. Return one longest increasing subsequence


# ============================================================
# FOLLOW-UP 1: O(n log n): patience sorting
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the smallest possible tail of an increasing
# subsequence of every length. A new number replaces the
# first tail that is not smaller (binary search), or
# extends the list.
# Time: O(n log n).  Space: O(n).
import bisect


def lis_length_fast(nums):
    tails = []
    for x in nums:
        position = bisect.bisect_left(tails, x)
        if position == len(tails):
            tails.append(x)
```

```
        else:
            tails[position] = x
    return len(tails)


# ========================================================
# FOLLOW-UP 2: Return one longest increasing subsequence
# ========================================================

# ---- follow-up 2: function ----
# Answer: Remember for every element its best length and
# the position of the previous element in that
# subsequence, then follow the links back from the best
# end.
# Time: O(n^2).  Space: O(n).
def lis_sequence(nums):
    n = len(nums)
    if n == 0:
        return []
    length = [1] * n
    previous = [-1] * n
    for i in range(n):
        for j in range(i):
            if nums[j] < nums[i] and length[j] + 1 > length[i]:
                length[i] = length[j] + 1
                previous[i] = j
    end = 0
    for i in range(n):
        if length[i] > length[end]:
            end = i
    sequence = []
    while end != -1:
        sequence.append(nums[end])
        end = previous[end]
    sequence.reverse()
    return sequence
```

PATTERN 13 · DYNAMIC PROGRAMMING - 1D · PROBLEM 9 OF 20

55. Jump Game

Medium · Greedy, Array, Dynamic Programming

leetcode.com/problems/jump-game

★ **TOP 7 PROBLEM** in Greedy: master this one first

Also appears in pattern 21 Greedy

PROBLEM

You are given an integer array `nums`. You are initially positioned at the array's **first index**, and each element in the array represents your maximum jump length at that position.

Return `true` *if you can reach the last index, or* `false` *otherwise*.

Example 1:

```
Input: nums = [2,3,1,1,4]
Output: true
Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.
```

Example 2:

```
Input: nums = [3,2,1,0,4]
Output: false
Explanation: You will always arrive at index 3 no matter what. Its maximum jump length is 0, which
makes it impossible to reach the last index.
```

Constraints:

- `1 <= nums.length <= 10^4`
- `0 <= nums[i] <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: Greedy

How to think about it

The first idea is DFS/BFS over reachable indices, or DP for whether each index is reachable. Correct, but worst-case $O(n^2)$. $n \le 10^4$ is tight, and we only care about the last index, not the path. The waste is expanding every jump. Reachable indices form a prefix: maintain the farthest reachable mx; if some i > mx, we are cut off. So we scan left to right, update mx with i + nums[i], and finishing the scan means the end is reachable.

We use a variable mx to maintain the farthest index that can currently be reached, initially mx = 0.

We traverse the array from left to right. For each position i we traverse, if mx < i, it means that the current position cannot be reached, so we directly return `false`. Otherwise, the farthest position that we can reach by jumping from position i is i+nums[i], we use i+nums[i] to update the value of mx, that is, mx = max(mx, i + nums[i]).

At the end of the traversal, we directly return `true`.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Similar problems:

- 45. Jump Game II
- 1024. Video Stitching
- 1326. Minimum Number of Taps to Open to Water a Garden

Python 3

```python
class Solution:
    def canJump(self, nums: List[int]) -> bool:
        mx = 0
        for i, x in enumerate(nums):
            if mx < i:
                return False
            mx = max(mx, i + x)
        return True
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The minimum number of jumps to reach the end (Jump
# Game II)
# 2. From index i you may jump exactly nums[i] to the left
# or to the right: can you reach a position with value 0?


# ============================================================
# FOLLOW-UP 1: The minimum number of jumps to reach the
# end (Jump Game II)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Greedy by levels: keep the end of the current
# jump's reach and the farthest position reachable from
# inside it; when the current reach ends, count one more
# jump.
# Time: O(n).  Space: O(1).
def min_jumps(nums):
    jumps = 0
    current_end = 0
    farthest = 0
    for i in range(len(nums) - 1):
        if i + nums[i] > farthest:
            farthest = i + nums[i]
        if i == current_end:
            jumps += 1
            current_end = farthest
    return jumps


# ============================================================
# FOLLOW-UP 2: From index i you may jump exactly nums[i]
# to the left or to the right: can you reach a position
# with value 0?
# ============================================================

# ---- follow-up 2: function ----
# Answer: A graph search from the start where every index
# has at most two neighbours.
# Time: O(n).  Space: O(n).
from collections import deque


def can_reach_zero(nums, start):
    seen = {start}
    queue = deque([start])
    while queue:
        i = queue.popleft()
        if nums[i] == 0:
            return True
        for j in (i + nums[i], i - nums[i]):
            if 0 <= j < len(nums) and j not in seen:
                seen.add(j)
```

```
                queue.append(j)
    return False
```

PATTERN 13 · DYNAMIC PROGRAMMING - 1D · PROBLEM 10 OF 20

152. Maximum Product Subarray

Medium · Array, Dynamic Programming

leetcode.com/problems/maximum-product-subarray

★ **TOP 7 PROBLEM** in Dynamic Programming - 1D: master this one first

PROBLEM

Given an integer array `nums`, find a subarray that has the largest product, and return *the product*.

The test cases are generated so that the answer will fit in a **32-bit** integer.

Note that the product of an array with a single element is the value of that element.

Example 1:

```
Input: nums = [2,3,-2,4]
Output: 6
Explanation: [2,3] has the largest product 6.
```

Example 2:

```
Input: nums = [-2,0,-1]
Output: 0
Explanation: The result cannot be 2, because [-2,-1] is not a subarray.
```

Constraints:

- `1 <= nums.length <= 2 * 10^4`
- `-10 <= nums[i] <= 10`
- The product of any subarray of `nums` is **guaranteed** to fit in a **32-bit** integer.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Kadane works for sums; products flip when a negative appears, and a zero resets the run. n≤ 2× 10^4. Keep the max and min product ending here; the new value may multiply either or start a new segment. The answer is the best max seen.

Python 3

```python
class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        ans = f = g = nums[0]
        for x in nums[1:]:
            ff, gg = f, g
            f = max(x, ff * x, gg * x)
            g = min(x, ff * x, gg * x)
            ans = max(ans, f)
        return ans
```

Complexity. Time: O(n). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subarray (its positions) with the maximum
# product


# ========================================================
# FOLLOW-UP 1: Return the subarray (its positions) with
# the maximum product
# ========================================================

# ---- follow-up 1: function ----
# Answer: Track the biggest and the smallest product
# ending at each position (a negative number swaps them)
# together with where each one started.
# Time: O(n).  Space: O(1).
def max_product_positions(nums):
    best_value = nums[0]
    best_range = (0, 0)
    big, big_start = nums[0], 0
    small, small_start = nums[0], 0
    for i in range(1, len(nums)):
        x = nums[i]
        candidates = [(x, i), (big * x, big_start), (small * x, small_start)]
        new_big = candidates[0]
        new_small = candidates[0]
        for value, start in candidates:
            if value > new_big[0]:
                new_big = (value, start)
            if value < new_small[0]:
                new_small = (value, start)
        big, big_start = new_big
        small, small_start = new_small
        if big > best_value:
            best_value = big
            best_range = (big_start, i)
    return best_value, best_range
```

309. Best Time to Buy and Sell Stock with Cooldown

Medium · Array, Dynamic Programming

leetcode.com/problems/best-time-to-buy-and-sell-stock-with-cooldown

★ **TOP 7 PROBLEM** in Dynamic Programming - 1D: master this one first

PROBLEM

You are given an array `prices` where `prices[i]` is the price of a given stock on the i^{th} day.

Find the maximum profit you can achieve. You may complete as many transactions as you like (i.e., buy one and sell one share of the stock multiple times) with the following restrictions:

- After you sell your stock, you cannot buy stock on the next day (i.e., cooldown one day).

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

```
Input: prices = [1,2,3,0,2]
Output: 3
Explanation: transactions = [buy, sell, cooldown, buy, sell]
```

Example 2:

```
Input: prices = [1]
Output: 0
```

Constraints:

- `1 <= prices.length <= 5000`
- `0 <= prices[i] <= 1000`

SOLUTIONS (PYTHON 3)

Solution 1: Memoization Search

How to think about it

We may trade many times, but a sell forces a one-day cooldown. A raw decision tree over day, holding, and cooldown revisits the same states. Compress to (i,j): starting at day i, whether we hold. Skip the day; if holding, sell and jump to i+2; if free, buy and start holding. Memoization evaluates each state once; the extra day after a sell is the cooldown.

We design a function dfs(i, j), which represents the maximum profit that can be obtained starting from the ith day with state j. The values of j are 0 and 1, respectively representing currently not holding a stock and holding a stock. The answer is dfs(0, 0).

The execution logic of the function dfs(i, j) is as follows:

If i ≥ n, it means that there are no more stocks to trade, so return 0;

Otherwise, we can choose not to trade, then dfs(i, j) = dfs(i + 1, j). We can also trade stocks. If j > 0, it means that we currently hold a stock and can sell it, then dfs(i, j) = prices[i] + dfs(i + 2, 0). If j = 0, it means that we currently do not hold a stock and can buy, then dfs(i, j) = -prices[i] + dfs(i + 1, 1). Take the maximum value as the return value of the function dfs(i, j).

The answer is dfs(0, 0).

To avoid repeated calculations, we use the method of memoization search, and use an array f to record the return value of dfs(i, j). If f[i][j] is not -1, it means that it has been calculated, and we can directly return f[i][j].

The time complexity is O(n), and the space complexity is O(n), where n is the length of the array prices.

Python 3

```python
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            if i >= len(prices):
                return 0
            ans = dfs(i + 1, j)
            if j:
                ans = max(ans, prices[i] + dfs(i + 2, 0))
            else:
                ans = max(ans, -prices[i] + dfs(i + 1, 1))
            return ans

        return dfs(0, 0)
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> Solution 1 fills a suffix table from the end. The same choices can be written forward: f[i][0/1] is the best profit after day i, free or holding. Free comes from staying free or selling today; holding comes from staying put or buying after cooldown, i.e. from f[i-2][0]. Fill left to right; the answer is free on the last day. Time stays O(n).

We can also use dynamic programming to solve this problem.

We define f[i][j] to represent the maximum profit that can be obtained on the ith day with state j. The values of j are 0 and 1, respectively representing currently not holding a stock and holding a stock. Initially, f[0][0] = 0, f[0][1] = -prices[0].

When i ≥ 1, if we currently do not hold a stock, then f[i][0] can be obtained by transitioning from f[i - 1][0] and f[i - 1][1] + prices[i], i.e., f[i][0] = max(f[i - 1][0], f[i - 1][1] + prices[i]). If we currently hold a stock, then f[i][1] can be obtained by transitioning from f[i - 1][1] and f[i - 2][0] - prices[i], i.e., f[i][1] = max(f[i - 1][1], f[i - 2][0] - prices[i]). The final answer is f[n - 1][0].

The time complexity is O(n), and the space complexity is O(n), where n is the length of the array prices.

Python 3

```python
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        n = len(prices)
        f = [[0] * 2 for _ in range(n)]
        f[0][1] = -prices[0]
        for i in range(1, n):
            f[i][0] = max(f[i - 1][0], f[i - 1][1] + prices[i])
            f[i][1] = max(f[i - 1][1], f[i - 2][0] - prices[i])
        return f[n - 1][0]
```

Solution 3: Dynamic Programming (Space Optimization)

> **How to think about it**
>
> Method 2 only reads i-1 and i-2, so the full table is unnecessary. Three rolling variables (free two days ago, free yesterday, holding yesterday) implement the same transfers in O(1) space.

The transition only needs the previous two days, so three variables are enough and the space complexity is O(1).

Python 3

```python
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        f, f0, f1 = 0, 0, -prices[0]
        for x in prices[1:]:
            f, f0, f1 = f0, max(f0, f1 + x), max(f1, f - x)
        return f0
```

Solution 4: Dynamic Programming

How to think about it

We may trade many times, but a sell forces a one-day cooldown. n ≤ 5000, so a raw decision tree is far too large. The state is the day and whether we hold. Skipping always calls the next day before it returns, so the chain has length n and overflows the stack. Later days are known if we walk backward. Let f[i][j] be the best profit from day i with holding flag j, and fill i from n-1 down to 0. Selling lands on i+2, which is the cooldown.

Let f[i][j] be the maximum profit starting from day i in state j. The values of j are 0 and 1, meaning we do not hold a stock and we hold a stock. The answer is f[0][0]. Days past the end contribute 0, so f[n][j] = f[n + 1][j] = 0.

Fill i from n - 1 down to 0. Doing nothing keeps f[i + 1][j]. If j > 0, we hold a stock and may sell it, earning prices[i] + f[i + 2][0]. The extra day is the cooldown. If j = 0, we may buy, earning -prices[i] + f[i + 1][1]. Take the larger value:

$$f[i][j] = \max(f[i + 1][j], prices[i] + f[i + 2][0])$$

when j > 0, and

$$f[i][j] = \max(f[i + 1][j], -prices[i] + f[i + 1][1])$$

when j = 0.

The time complexity is O(n), and the space complexity is O(n), where n is the length of the array prices.

Python 3

```python
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        n = len(prices)
        f = [[0] * 2 for _ in range(n + 2)]
        for i in range(n - 1, -1, -1):
            for j in range(2):
                ans = f[i + 1][j]
                if j:
                    ans = max(ans, prices[i] + f[i + 2][0])
                else:
                    ans = max(ans, -prices[i] + f[i + 1][1])
                f[i][j] = ans
        return f[0][0]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Every sale costs a transaction fee (no cooldown)
# 2. At most k transactions


# ============================================================
# FOLLOW-UP 1: Every sale costs a transaction fee (no
# cooldown)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Two states: holding a share, or holding none.
# Selling adds the price minus the fee; buying subtracts
# the price.
# Time: O(n).  Space: O(1).
def max_profit_with_fee(prices, fee):
    holding = float("-inf")
    free = 0
    for price in prices:
        new_holding = free - price
        if holding > new_holding:
            new_holding = holding
        new_free = holding + price - fee
        if free > new_free:
```

```
            new_free = free
        holding, free = new_holding, new_free
    return free


# ========================================================
# FOLLOW-UP 2: At most k transactions
# ========================================================

# ---- follow-up 2: function ----
# Answer: buy[j] and sell[j] hold the best balance after
# the j-th buy / sell; process the prices one by one and
# update every j.
# Time: O(n * k).  Space: O(k).
def max_profit_k_transactions(k, prices):
    if k >= len(prices) // 2:
        profit = 0
        for i in range(1, len(prices)):
            if prices[i] > prices[i - 1]:
                profit += prices[i] - prices[i - 1]
        return profit
    buy = [float("-inf")] * (k + 1)
    sell = [0] * (k + 1)
    for price in prices:
        for j in range(1, k + 1):
            if sell[j - 1] - price > buy[j]:
                buy[j] = sell[j - 1] - price
            if buy[j] + price > sell[j]:
                sell[j] = buy[j] + price
    return sell[k]
```

PATTERN 13 · DYNAMIC PROGRAMMING - 1D · PROBLEM 12 OF 20

416. Partition Equal Subset Sum

Medium · Array, Dynamic Programming, Knapsack, 0-1 Knapsack

leetcode.com/problems/partition-equal-subset-sum

★ **TOP 7 PROBLEM** in Dynamic Programming - Knapsack Style: master this one first

Also appears in pattern 28 Dynamic Programming - Knapsack Style

PROBLEM

Given an integer array `nums`, return `true` *if you can partition the array into two subsets such that the sum of the elements in both subsets is equal or* `false` *otherwise*.

Example 1:

```
Input: nums = [1,5,11,5]
Output: true
Explanation: The array can be partitioned as [1, 5, 5] and [11].
```

Example 2:

```
Input: nums = [1,2,3,5]
Output: false
Explanation: The array cannot be partitioned into equal sum subsets.
```

Constraints:

- `1 <= nums.length <= 200`
- `1 <= nums[i] <= 100`

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

Equal partition means a subset sums to half the total. An odd total is impossible. Subset enumeration is too large for n≤ 200. This is 0-1 knapsack with capacity m=s/2: f[i][j] is whether the first i numbers can make j, taking or skipping x. The n× m table fits the limits. Reject an odd sum first. The empty subset gives f[0][0]=true, which seeds the recurrence.

First, we calculate the total sum s of the array. If the total sum is odd, it cannot be divided into two subsets with equal sums, so we directly return `false`. If the total sum is even, we set the target subset sum to m = s/2. The problem is then transformed into: does there exist a subset whose element sum is m?

We define f[i][j] to represent whether it is possible to select several numbers from the first i numbers so that their sum is exactly j. Initially, f[0][0] = true and the rest f[i][j] = false. The answer is f[n][m].

Considering f[i][j], if we select the i-th number x, then f[i][j] = f[i - 1][j - x]. If we do not select the i-th number x, then f[i][j] = f[i - 1][j]. Therefore, the state transition equation is:

$$f[i][j] = f[i - 1][j] \text{ or } f[i - 1][j - x] \text{ if } j \geq x$$

The final answer is f[n][m].

The time complexity is O(m × n), and the space complexity is O(m × n). Where m and n are half of the total sum of the array and the length of the array, respectively.

Python 3

```python
class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        m, mod = divmod(sum(nums), 2)
        if mod:
            return False
        n = len(nums)
        f = [[False] * (m + 1) for _ in range(n + 1)]
        f[0][0] = True
        for i, x in enumerate(nums, 1):
            for j in range(m + 1):
                f[i][j] = f[i - 1][j] or (j >= x and f[i - 1][j - x])
        return f[n][m]
```

Solution 2: Dynamic Programming (Space Optimization)

> **How to think about it**
>
> Row i depends only on row i-1, and an item must not be reused, so update j downward and keep a one-dimensional array. Space drops from O(nm) to O(m).

We notice that in Solution 1, f[i][j] is only related to f[i - 1][·]. Therefore, we can compress the two-dimensional array into a one-dimensional array.

The time complexity is O(n × m), and the space complexity is O(m). Where n is the length of the array, and m is half of the total sum of the array.

Python 3

```python
class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        m, mod = divmod(sum(nums), 2)
        if mod:
            return False
        f = [True] + [False] * m
        for x in nums:
            for j in range(m, x - 1, -1):
                f[j] = f[j] or f[j - x]
        return f[m]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return one subset with half of the total
# 2. Split into two groups with the SMALLEST possible
# difference between their sums


# ============================================================
# FOLLOW-UP 1: Return one subset with half of the total
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember for every reachable sum the number that
# first made it reachable; walk back from the target sum.
# Time: O(n * S log S), S = half of the total.  Space:
# O(S).
def equal_subset(nums):
    total = 0
    for x in nums:
        total += x
    if total % 2 != 0:
        return None
    target = total // 2
    first_by = {0: None}                          # reachable sum -> the number that created it
```

```
    for x in nums:
        for s in sorted(first_by, reverse=True):
            if s + x <= target and s + x not in first_by:
                first_by[s + x] = x
    if target not in first_by:
        return None
    subset = []
    s = target
    while s > 0:
        subset.append(first_by[s])
        s -= first_by[s]
    return subset


# ============================================================
# FOLLOW-UP 2: Split into two groups with the SMALLEST
# possible difference between their sums
# ============================================================

# ---- follow-up 2: function ----
# Answer: The reachable sums up to half of the total tell
# how close one group can get to half; the answer is total
# - 2 * the best reachable sum.
# Time: O(n * S).  Space: O(S).
def min_subset_difference(nums):
    total = 0
    for x in nums:
        total += x
    half = total // 2
    reachable = [False] * (half + 1)
    reachable[0] = True
    for x in nums:
        for s in range(half, x - 1, -1):
            if reachable[s - x]:
                reachable[s] = True
    best = 0
    for s in range(half + 1):
```

```
        if reachable[s]:
            best = s
    return total - 2 * best
```

518. Coin Change II

Medium · Array, Dynamic Programming, Knapsack, Unbounded Knapsack

leetcode.com/problems/coin-change-ii

★ **TOP 7 PROBLEM** in Dynamic Programming - Knapsack Style: master this one first

Also appears in pattern 28 Dynamic Programming - Knapsack Style

PROBLEM

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return *the number of combinations that make up that amount*. If that amount of money cannot be made up by any combination of the coins, return `0`.

You may assume that you have an infinite number of each kind of coin.

The **final** answer is **guaranteed** to fit into a signed **32-bit** integer.

Example 1:

```
Input: amount = 5, coins = [1,2,5]
Output: 4
Explanation: there are four ways to make up the amount:
5=5
5=2+2+1
5=2+1+1+1
5=1+1+1+1+1
```

Example 2:

```
Input: amount = 3, coins = [2]
Output: 0
Explanation: the amount of 3 cannot be made up just with coins of 2.
```

Example 3:

```
Input: amount = 10, coins = [10]
Output: 1
```

Constraints:

- `1 <= coins.length <= 300`
- `1 <= coins[i] <= 5000`
- All the values of `coins` are **unique**.
- `0 <= amount <= 5000`

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming (Complete Knapsack)

How to think about it

We count combinations, not permutations. Generating sequences would overcount the same multiset. This is the unbounded-knapsack counting DP: iterate coins in the outer loop so each coin is added in one order. f[i][j] is the number of ways to make j with the first i coins: skip inherits f[i-1][j], take adds f[i][j-x]. f[0][0]=1 is the empty combination.

We define f[i][j] as the number of coin combinations to make up the amount j using the first i types of coins. Initially, f[0][0] = 1, and the values of other positions are all 0.

We can enumerate the quantity k of the last coin used, then we have equation one:

$$f[i][j] = f[i - 1][j] + f[i - 1][j - x] + f[i - 1][j - 2 \times x] + \ldots + f[i - 1][j - k \times x]$$

where x represents the face value of the i-th type of coin.

Let j = j - x, then we have equation two:

$$f[i][j - x] = f[i - 1][j - x] + f[i - 1][j - 2 \times x] + \ldots + f[i - 1][j - k \times x]$$

Substituting equation two into equation one, we can get the following state transition equation:

$$f[i][j] = f[i - 1][j] + f[i][j - x]$$

The final answer is f[m][n].

The time complexity is O(m × n), and the space complexity is O(m × n). Where m and n are the number of types of coins and the total amount, respectively.

Python 3

```python
class Solution:
    def change(self, amount: int, coins: List[int]) -> int:
        m, n = len(coins), amount
        f = [[0] * (n + 1) for _ in range(m + 1)]
        f[0][0] = 1
        for i, x in enumerate(coins, 1):
            for j in range(n + 1):
                f[i][j] = f[i - 1][j]
                if j >= x:
                    f[i][j] += f[i][j - x]
        return f[m][n]
```

Solution 2: Optimized Dynamic Programming

> **How to think about it**
>
> f[i][j] depends only on the previous row at j and the current row at j-x. A one-dimensional array, updated forward from x to `amount` for each coin, reuses the same coin's new states. Space drops to O(amount); the number of transitions is unchanged.

We notice that f[i][j] is only related to f[i - 1][j] and f[i][j - x]. Therefore, we can optimize the two-dimensional array into a one-dimensional array, reducing the space complexity to O(n). The time complexity remains O(m × n).

Python 3

```python
class Solution:
    def change(self, amount: int, coins: List[int]) -> int:
        n = amount
        f = [1] + [0] * n
        for x in coins:
            for j in range(x, n + 1):
                f[j] += f[j - x]
        return f[n]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. List the actual combinations of coins
# 2. Each coin may be used at most ONCE


# ============================================================
# FOLLOW-UP 1: List the actual combinations of coins
# ============================================================

# ---- follow-up 1: function ----
```

```python
# Answer: Backtracking over coin types in order, taking
# each coin 0 or more times, so every combination appears
# once.
# Time: exponential: O(number of combinations * length).
# Space: O(c) recursion plus the results.
def coin_combinations(amount, coins):
    results = []

    def go(index, remaining, chosen):
        if remaining == 0:
            results.append(list(chosen))
            return
        if index == len(coins):
            return
        coin = coins[index]
        count = 0
        while count * coin <= remaining:
            go(index + 1, remaining - count * coin, chosen + [coin] * count)
            count += 1

    go(0, amount, [])
    return results


# ============================================================
# FOLLOW-UP 2: Each coin may be used at most ONCE
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same table of ways, but loop the amounts
# downwards so a coin is not used twice.
# Time: O(n * amount).  Space: O(amount).
def ways_each_coin_once(amount, coins):
    ways = [0] * (amount + 1)
    ways[0] = 1
    for coin in coins:
        for a in range(amount, coin - 1, -1):
            ways[a] += ways[a - coin]
    return ways[amount]
```

377. Combination Sum IV

Medium · Array, Dynamic Programming

leetcode.com/problems/combination-sum-iv

Also appears in pattern 28 Dynamic Programming - Knapsack Style

PROBLEM

Given an array of **distinct** integers `nums` and a target integer `target`, return *the number of possible combinations that add up to* `target`.

The test cases are generated so that the answer can fit in a **32-bit** integer.

Example 1:

```
Input: nums = [1,2,3], target = 4
Output: 7
Explanation:
The possible combination ways are:
(1, 1, 1, 1)
(1, 1, 2)
(1, 2, 1)
(1, 3)
(2, 1, 1)
(2, 2)
(3, 1)
Note that different sequences are counted as different combinations.
```

Example 2:

```
Input: nums = [9], target = 3
Output: 0
```

Constraints:

- `1 <= nums.length <= 200`
- `1 <= nums[i] <= 1000`
- All the elements of `nums` are **unique**.
- `1 <= target <= 1000`

Follow up: What if negative numbers are allowed in the given array? How does it change the problem? What limitation we need to add to the question to allow negative numbers?

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

> **How to think about it**
>
> Count ordered ways to sum to target. An outer loop over items (combinations) misses permutations. Enumerate the sum first, then the last addend. f[i] is the number of permutations summing to i, f[0]=1. For each i and each x≤ i, add f[i-x]. Same shape as 322, counting instead of minimizing coins.

We define f[i] as the number of combinations that sum up to i. Initially, f[0] = 1, and the rest f[i] = 0. The final answer is f[target].

For f[i], we can enumerate each element x in the array. If i ≥ x, then f[i] = f[i] + f[i - x].

Finally, return f[target].

The time complexity is O(n × target), and the space complexity is O(target), where n is the length of the array.

Python 3

```python
class Solution:
    def combinationSum4(self, nums: List[int], target: int) -> int:
        f = [1] + [0] * target
        for i in range(1, target + 1):
            for x in nums:
                if i >= x:
                    f[i] += f[i - x]
        return f[target]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. List the ordered sequences instead of counting them
# 2. The sequences may use at most L numbers


# ============================================================
# FOLLOW-UP 1: List the ordered sequences instead of
# counting them
# ============================================================

# ---- follow-up 1: function ----
# Answer: Backtracking: at each step try every number that
# still fits; a sequence is complete when the remaining
# target is 0.
# Time: exponential: O(number of sequences * length).
# Space: O(target) recursion plus the results.
def ordered_sequences(nums, target):
    results = []

    def go(remaining, chosen):
        if remaining == 0:
            results.append(list(chosen))
            return
        for x in nums:
            if x <= remaining:
                chosen.append(x)
                go(remaining - x, chosen)
                chosen.pop()

    go(target, [])
    return results


# ============================================================
# FOLLOW-UP 2: The sequences may use at most L numbers
# ============================================================

# ---- follow-up 2: function ----
# Answer: Add the length to the state: ways[length][sum]
# counts sequences with exactly that many numbers; add up
# the lengths 1 to L.
# Time: O(L * target * n).  Space: O(L * target).
def sequences_with_length_limit(nums, target, limit):
    ways = []
    for length in range(limit + 1):
        ways.append([0] * (target + 1))
    ways[0][0] = 1
    total = 0
    for length in range(1, limit + 1):
        for s in range(target + 1):
            for x in nums:
                if x <= s:
                    ways[length][s] += ways[length - 1][s - x]
        total += ways[length][target]
```

```
    return total
```

PATTERN 13 · DYNAMIC PROGRAMMING - 1D · PROBLEM 15 OF 20

279. Perfect Squares

Medium · Breadth-First Search, Math, Dynamic Programming, Knapsack, Unbounded Knapsack

leetcode.com/problems/perfect-squares

Also appears in pattern 28 Dynamic Programming - Knapsack Style

PROBLEM

Given an integer `n`, return *the least number of perfect square numbers that sum to* `n`.

A **perfect square** is an integer that is the square of an integer; in other words, it is the product of some integer with itself. For example, `1`, `4`, `9`, and `16` are perfect squares while `3` and `11` are not.

Example 1:

```
Input: n = 12
Output: 3
Explanation: 12 = 4 + 4 + 4.
```

Example 2:

```
Input: n = 13
Output: 2
Explanation: 13 = 4 + 9.
```

Constraints:

- $1 <= n <= 10^4$

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming (Complete Knapsack)

How to think about it

Perfect squares may be reused, so the fewest that sum to n is an unbounded knapsack with items $1^2,\ldots,m^2$ where m=floor(√n). f[i][j] is the fewest squares among the first i kinds that sum to j: skip i^2, or take one more copy.

Python 3

```python
class Solution:
    def numSquares(self, n: int) -> int:
        m = int(sqrt(n))
        f = [[inf] * (n + 1) for _ in range(m + 1)]
        f[0][0] = 0
        for i in range(1, m + 1):
            for j in range(n + 1):
                f[i][j] = f[i - 1][j]
                if j >= i * i:
                    f[i][j] = min(f[i][j], f[i][j - i * i] + 1)
        return f[m][n]
```

Complexity. Time: O(n * sqrt(n)). Space: O(n * sqrt(n)).

Solution 2: Optimized Dynamic Programming

How to think about it

f[i][j] depends only on f[i-1][j] and f[i][j-i^2], so a 1-D array updated in increasing j is enough.

f[i][j] depends only on f[i - 1][j] and f[i][j - i^2], so the table can be rolled into a 1D array of space O(n). The time complexity stays O(m × n).

Python 3

```
class Solution:
    def numSquares(self, n: int) -> int:
        m = int(sqrt(n))
        f = [0] + [inf] * n
        for i in range(1, m + 1):
            for j in range(i * i, n + 1):
                f[j] = min(f[j], f[j - i * i] + 1)
        return f[n]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the squares that add up to n
# 2. O(sqrt n): the answer is always 1, 2, 3 or 4


# ============================================================
# FOLLOW-UP 1: Return the squares that add up to n
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember the last square used for every number
# and walk back from n.
# Time: O(n * sqrt(n)).  Space: O(n).
def squares_for(n):
    best = [0] + [n + 1] * n
    last = [0] * (n + 1)
    for i in range(1, n + 1):
        j = 1
        while j * j <= i:
            if best[i - j * j] + 1 < best[i]:
                best[i] = best[i - j * j] + 1
                last[i] = j * j
            j += 1
    used = []
    while n > 0:
        used.append(last[n])
        n -= last[n]
    return used


# ============================================================
# FOLLOW-UP 2: O(sqrt n): the answer is always 1, 2, 3 or
# 4
# ============================================================

# ---- follow-up 2: function ----
# Answer: By Lagrange's four-square theorem 4 always
# suffices. The answer is 1 for a perfect square, 4 when n
# has the form 4^a (8b + 7), 2 when it is a sum of two
# squares, otherwise 3.
# Time: O(sqrt(n)).  Space: O(1).
def num_squares_math(n):
    def is_square(x):
        r = int(x ** 0.5)
        while r * r > x:
            r -= 1
        while (r + 1) * (r + 1) <= x:
            r += 1
        return r * r == x

    if is_square(n):
        return 1
    m = n
    while m % 4 == 0:
```

```
        m //= 4
    if m % 8 == 7:
        return 4
    i = 1
    while i * i <= n:
        if is_square(n - i * i):
```

```
            return 2
        i += 1
    return 3
```

376. Wiggle Subsequence

Medium · Greedy, Array, Dynamic Programming

leetcode.com/problems/wiggle-subsequence

Also appears in pattern 21 Greedy

PROBLEM

A **wiggle sequence** is a sequence where the differences between successive numbers strictly alternate between positive and negative. The first difference (if one exists) may be either positive or negative. A sequence with one element and a sequence with two non-equal elements are trivially wiggle sequences.

- For example, `[1, 7, 4, 9, 2, 5]` is a **wiggle sequence** because the differences `(6, -3, 5, -7, 3)` alternate between positive and negative.
- In contrast, `[1, 4, 7, 2, 5]` and `[1, 7, 4, 5, 5]` are not wiggle sequences. The first is not because its first two differences are positive, and the second is not because its last difference is zero.

A **subsequence** is obtained by deleting some elements (possibly zero) from the original sequence, leaving the remaining elements in their original order.

Given an integer array `nums`, return *the length of the longest **wiggle subsequence** of* `nums`.

Example 1:

```
Input: nums = [1,7,4,9,2,5]
Output: 6
Explanation: The entire sequence is a wiggle sequence with differences (6, -3, 5, -7, 3).
```

Example 2:

```
Input: nums = [1,17,5,10,13,15,10,5,16,8]
Output: 7
Explanation: There are several subsequences that achieve this length.
One is [1, 17, 10, 13, 10, 16, 8] with differences (16, -7, 3, -3, 6, -8).
```

Example 3:

```
Input: nums = [1,2,3,4,5,6,7,8,9]
Output: 2
```

Constraints:

- `1 <= nums.length <= 1000`
- `0 <= nums[i] <= 1000`

Follow up: Could you solve this in `O(n)` time?

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

> **How to think about it**
>
> A wiggle subsequence alternates up and down. Enumerating subsequences is large. At an ending index we only distinguish a last up or last down. f[i] ends up, g[i] ends down. A smaller nums[j] extends a down into f; a larger one extends an up into g. The answer is the max of all f and g.

We define f[i] as the length of the wiggle sequence ending at the ith element with an upward trend, and g[i] as the length of the wiggle sequence ending at the ith element with a downward trend. Initially, f[0] = g[0] = 1 because when there is only one element, the length of the wiggle sequence is 1. Initialize the answer as 1.

For f[i], where i ≥ 1, we enumerate j in the range [0, i), if nums[j] < nums[i], it means that i can be appended after j to form an upward wiggle sequence, then f[i] = max(f[i], g[j] + 1); if nums[j] > nums[i], it means that i can be appended after j to form a downward wiggle sequence, then g[i] = max(g[i], f[j] + 1). Then we update the answer to max(f[i], g[i]).

Finally, we return the answer.

The time complexity is $O(n^2)$, and the space complexity is O(n). Where n is the length of the array nums.

Python 3

```python
class Solution:
    def wiggleMaxLength(self, nums: List[int]) -> int:
        n = len(nums)
        ans = 1
        f = [1] * n
        g = [1] * n
        for i in range(1, n):
            for j in range(i):
                if nums[j] < nums[i]:
                    f[i] = max(f[i], g[j] + 1)
                elif nums[j] > nums[i]:
                    g[i] = max(g[i], f[j] + 1)
            ans = max(ans, f[i], g[i])
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return one longest wiggle subsequence


# ============================================================
# FOLLOW-UP 1: Return one longest wiggle subsequence
# ============================================================

# ---- follow-up 1: function ----
# Answer: Walk through the numbers and keep a number when
# it changes the direction (up after down, or down after
# up); equal neighbours are skipped. For a run in the same
# direction, replace the last kept number by the more
# extreme one.
# Time: O(n).  Space: O(n).
def wiggle_sequence(nums):
    if not nums:
        return []
    result = [nums[0]]
    direction = 0                                   # 1 up, -1 down, 0 not decided
    for i in range(1, len(nums)):
        if nums[i] > result[-1]:
            if direction != 1:
                result.append(nums[i])
                direction = 1
            else:
                result[-1] = nums[i]
        elif nums[i] < result[-1]:
            if direction != -1:
                result.append(nums[i])
                direction = -1
            else:
                result[-1] = nums[i]
    return result
```

673. Number of Longest Increasing Subsequence

Medium · Binary Indexed Tree, Segment Tree, Array, Dynamic Programming, Longest Increasing Subsequence

leetcode.com/problems/number-of-longest-increasing-subsequence

Also appears in pattern 40 Segment Tree, 41 Binary Indexed Tree (Fenwick Tree)

PROBLEM

Given an integer array `nums`, return *the number of longest increasing subsequences.*

Notice that the sequence has to be **strictly** increasing.

Example 1:

```
Input: nums = [1,3,5,4,7]
Output: 2
Explanation: The two longest increasing subsequences are [1, 3, 4, 7] and [1, 3, 5, 7].
```

Example 2:

```
Input: nums = [2,2,2,2,2]
Output: 5
Explanation: The length of the longest increasing subsequence is 1, and there are 5 increasing
subsequences of length 1, so output 5.
```

Constraints:

- `1 <= nums.length <= 2000`
- -10^6 `<= nums[i] <=` 10^6
- The answer is guaranteed to fit inside a 32-bit integer.

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

We need both the LIS length and how many such sequences exist. n≤ 2000 allows O(n^2). f[i] and cnt[i] are the best length and count ending at i. Update or add when nums[j]<nums[i]. Sum cnt over positions that attain the global maximum length.

We define f[i] as the length of the longest increasing subsequence ending with nums[i], and cnt[i] as the number of longest increasing subsequences ending with nums[i]. Initially, f[i]=1, cnt[i]=1. Also, we define mx as the length of the longest increasing subsequence, and ans as the number of longest increasing subsequences.

For each nums[i], we enumerate all elements nums[j] in nums[0:i-1]. If nums[j] < nums[i], then nums[i] can be appended after nums[j] to form a longer increasing subsequence. If f[i] < f[j] + 1, it means the length of the longest increasing subsequence ending with nums[i] has increased, so we need to update f[i]=f[j]+1 and cnt[i]=cnt[j]. If f[i]=f[j]+1, it means we have found a longest increasing subsequence with the same length as before, so we need to increase cnt[i] by cnt[j]. Then, if mx < f[i], it means the length of the longest increasing subsequence has increased, so we need to update mx=f[i] and ans=cnt[i]. If mx=f[i], it means we have found a longest increasing subsequence with the same length as before, so we need to increase ans by cnt[i].

Finally, we return ans.

The time complexity is O(n^2), and the space complexity is O(n). Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def findNumberOfLIS(self, nums: List[int]) -> int:
        n = len(nums)
        f = [1] * n
        cnt = [1] * n
        mx = 0
        for i in range(n):
            for j in range(i):
                if nums[j] < nums[i]:
                    if f[i] < f[j] + 1:
                        f[i] = f[j] + 1
                        cnt[i] = cnt[j]
                    elif f[i] == f[j] + 1:
                        cnt[i] += cnt[j]
            if mx < f[i]:
                mx = f[i]
                ans = cnt[i]
            elif mx == f[i]:
                ans += cnt[i]
        return ans
```

Solution 2: Binary Indexed Tree

> **How to think about it**
>
> Scanning all previous indices is the bottleneck. After compressing values, a Fenwick tree stores the best length and count among smaller keys, so each step is O(log n).

We can use a binary indexed tree to maintain the length and count of the longest increasing subsequence in the prefix interval. We remove duplicates from the array nums and sort it to get the array arr. Then we enumerate each element x in nums, find the position i of x in the array arr by binary search, then query the length and count of the longest increasing subsequence in [1,i-1], denoted as v and cnt, then update the length and count of the longest increasing subsequence in [i] to v+1 and max(cnt,1). Finally, we query the length and count of the longest increasing subsequence in [1,m], where m is the length of the array arr, which is the answer.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the length of the array nums.

Python 3

```python
class BinaryIndexedTree:
    __slots__ = ["n", "c", "d"]

    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)
        self.d = [0] * (n + 1)

    def update(self, x, v, cnt):
        while x <= self.n:
            if self.c[x] < v:
                self.c[x] = v
                self.d[x] = cnt
            elif self.c[x] == v:
                self.d[x] += cnt
            x += x & -x

    def query(self, x):
        v = cnt = 0
        while x:
            if self.c[x] > v:
                v = self.c[x]
                cnt = self.d[x]
            elif self.c[x] == v:
                cnt += self.d[x]
            x -= x & -x
        return v, cnt


class Solution:
    def findNumberOfLIS(self, nums: List[int]) -> int:
        arr = sorted(set(nums))
        m = len(arr)
        tree = BinaryIndexedTree(m)
        for x in nums:
            i = bisect_left(arr, x) + 1
            v, cnt = tree.query(i - 1)
            tree.update(i, v + 1, max(cnt, 1))
        return tree.query(m)[1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count the longest NON-DECREASING subsequences
# 2. Count ALL increasing subsequences (any length of at
# least 1)


# ============================================================
# FOLLOW-UP 1: Count the longest NON-DECREASING
# subsequences
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same tables with 'nums[j] <= nums[i]' as the
# condition for extending.
# Time: O(n^2).  Space: O(n).
def count_longest_non_decreasing(nums):
    n = len(nums)
    length = [1] * n
    count = [1] * n
    for i in range(n):
        for j in range(i):
            if nums[j] <= nums[i]:
                if length[j] + 1 > length[i]:
```

```
                    length[i] = length[j] + 1
                    count[i] = count[j]
                elif length[j] + 1 == length[i]:
                    count[i] += count[j]
    longest = 0
    for value in length:
        if value > longest:
            longest = value
    total = 0
    for i in range(n):
        if length[i] == longest:
            total += count[i]
    return total


# ============================================================
# FOLLOW-UP 2: Count ALL increasing subsequences (any
# length of at least 1)
# ============================================================

# ---- follow-up 2: function ----
# Answer: ways[i] = 1 (the number alone) + the sum of
# ways[j] for earlier smaller numbers; add all ways[i].
# Time: O(n^2).  Space: O(n).
def count_increasing_subsequences(nums):
    ways = [1] * len(nums)
    for i in range(len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                ways[i] += ways[j]
    total = 0
    for w in ways:
        total += w
    return total
```

983. Minimum Cost For Tickets

Medium · Array, Dynamic Programming

leetcode.com/problems/minimum-cost-for-tickets

PROBLEM

You have planned some train traveling one year in advance. The days of the year in which you will travel are given as an integer array `days`. Each day is an integer from `1` to `365`.

Train tickets are sold in **three different ways**:

- a **1-day** pass is sold for `costs[0]` dollars,
- a **7-day** pass is sold for `costs[1]` dollars, and
- a **30-day** pass is sold for `costs[2]` dollars.

The passes allow that many days of consecutive travel.

- For example, if we get a **7-day** pass on day `2`, then we can travel for `7` days: `2`, `3`, `4`, `5`, `6`, `7`, and `8`.

Return *the minimum number of dollars you need to travel every day in the given list of days*.

Example 1:

```
Input: days = [1,4,6,7,8,20], costs = [2,7,15]
Output: 11
Explanation: For example, here is one way to buy passes that lets you travel your travel plan:
On day 1, you bought a 1-day pass for costs[0] = $2, which covered day 1.
On day 3, you bought a 7-day pass for costs[1] = $7, which covered days 3, 4, ..., 9.
On day 20, you bought a 1-day pass for costs[0] = $2, which covered day 20.
In total, you spent $11 and covered all the days of your travel.
```

Example 2:

```
Input: days = [1,2,3,4,5,6,7,8,9,10,30,31], costs = [2,7,15]
Output: 17
Explanation: For example, here is one way to buy passes that lets you travel your travel plan:
On day 1, you bought a 30-day pass for costs[2] = $15 which covered days 1, 2, ..., 30.
On day 31, you bought a 1-day pass for costs[0] = $2 which covered day 31.
In total, you spent $17 and covered all the days of your travel.
```

Constraints:

- `1 <= days.length <= 365`
- `1 <= days[i] <= 365`
- `days` is in strictly increasing order.
- `costs.length == 3`
- `1 <= costs[i] <= 1000`

SOLUTIONS (PYTHON 3)

Solution 1: Memoization Search + Binary Search

How to think about it

Each travel day must be covered by a 1-, 7-, or 30-day pass at minimum cost. After buying a pass on trip i, the next uncovered trip is found by binary search. dfs(i) takes the best of the three options. There are at most 365 trips, so memoization is enough.

We define a function dfs(i), which represents the minimum cost required from the i-th trip to the last trip. Thus, the answer is dfs(0).

The execution process of the function dfs(i) is as follows:

- If i ≥ n, it means all trips have ended, return 0;
- Otherwise, we need to consider three types of purchases: buying a 1-day pass, buying a 7-day pass, and buying a 30-day pass. We calculate the cost for these three purchasing methods separately and use binary search to find the index j of the next trip, then recursively call dfs(j), and finally return the minimum cost among these three purchasing methods.

To avoid repeated calculations, we use memoization search to save the results that have already been calculated.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n represents the number of trips.

Python 3

```python
class Solution:
    def mincostTickets(self, days: List[int], costs: List[int]) -> int:
        @cache
        def dfs(i: int) -> int:
            if i >= n:
                return 0
            ans = inf
            for c, v in zip(costs, valid):
                j = bisect_left(days, days[i] + v)
                ans = min(ans, c + dfs(j))
            return ans

        n = len(days)
        valid = [1, 7, 30]
        return dfs(0)
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> Searching by trip index needs binary search. We can instead DP on calendar days: f[i] is the cheapest way to finish day i. A non-travel day copies f[i-1]; a travel day takes the min over the three pass starts. The last day is at most 365.

Let's denote the last day in the days array as m. We can define an array f of length m + 1, where f[i] represents the minimum cost from day 1 to day i.

We can calculate the value of f[i] in increasing order of the dates in the days array, starting from day 1. If day i is a travel day, we can consider three purchasing options: buying a 1-day pass, buying a 7-day pass, and buying a 30-day pass. We calculate the cost for these three purchasing methods separately and take the minimum cost among these three as the value of f[i]. If day i is not a travel day, then f[i] = f[i - 1].

The final answer is f[m].

The time complexity is O(m), and the space complexity is O(m). Here, m represents the last day of travel.

Python 3

```python
class Solution:
    def mincostTickets(self, days: List[int], costs: List[int]) -> int:
        m = days[-1]
        f = [0] * (m + 1)
        valid = [1, 7, 30]
        j = 0
        for i in range(1, m + 1):
            if i == days[j]:
                f[i] = inf
                for c, v in zip(costs, valid):
                    f[i] = min(f[i], f[max(0, i - v)] + c)
                j += 1
            else:
                f[i] = f[i - 1]
        return f[m]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return which tickets to buy
# 2. Passes of ANY list of durations with their own prices


# ============================================================
# FOLLOW-UP 1: Return which tickets to buy
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember for each travel day which ticket gave
# the best cost, and walk back from the last day.
# Time: O(n).  Space: O(n).
def tickets_to_buy(days, costs):
    durations = [1, 7, 30]
    n = len(days)
    best = [0] * (n + 1)
    choice = [None] * (n + 1)
    for i in range(n - 1, -1, -1):
        best[i] = float("inf")
        for k in range(3):
            j = i
            while j < n and days[j] < days[i] + durations[k]:
                j += 1
            if costs[k] + best[j] < best[i]:
                best[i] = costs[k] + best[j]
                choice[i] = (k, j)
    plan = []
    i = 0
    while i < n:
        k, j = choice[i]
        plan.append((days[i], durations[k]))
        i = j
    return best[0], plan


# ============================================================
# FOLLOW-UP 2: Passes of ANY list of durations with their
# own prices
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same table; for every day try every pass:
# the next uncovered travel day is found by moving forward
# while days are inside the pass.
# Time: O(n * p), p = number of passes.  Space: O(n).
def min_cost_with_passes(days, passes):
    # passes is a list of (duration, price)
    n = len(days)
    best = [0] * (n + 1)
    for i in range(n - 1, -1, -1):
        best[i] = float("inf")
        for duration, price in passes:
            j = i
            while j < n and days[j] < days[i] + duration:
                j += 1
            if price + best[j] < best[i]:
                best[i] = price + best[j]
    return best[0]
```

740. Delete and Earn

Medium · Array, Hash Table, Dynamic Programming

leetcode.com/problems/delete-and-earn

PROBLEM

You are given an integer array `nums`. You want to maximize the number of points you get by performing the following operation any number of times:

- Pick any `nums[i]` and delete it to earn `nums[i]` points. Afterwards, you must delete **every** element equal to `nums[i] - 1` and **every** element equal to `nums[i] + 1`.

Return *the **maximum number of points** you can earn by applying the above operation some number of times*.

Example 1:

```
Input: nums = [3,4,2]
Output: 6
Explanation: You can perform the following operations:
- Delete 4 to earn 4 points. Consequently, 3 is also deleted. nums = [2].
- Delete 2 to earn 2 points. nums = [].
You earn a total of 6 points.
```

Example 2:

```
Input: nums = [2,2,3,3,3,4]
Output: 9
Explanation: You can perform the following operations:
- Delete a 3 to earn 3 points. All 2's and 4's are also deleted. nums = [3,3].
- Delete a 3 again to earn 3 points. nums = [3].
- Delete a 3 once more to earn 3 points. nums = [].
You earn a total of 9 points.
```

Constraints:

- `1 <= nums.length <= 2 * 10^4`
- `1 <= nums[i] <= 10^4`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Taking x earns every copy of x and forbids x-1 and x+1. n≤ 2× 10^4 and values ≤ 10^4, so searching each choice repeats work. Each value is all-or-nothing and neighbors conflict—the house-robber recurrence. Bucket scores into total[x], then walk the value line. Two rolling scalars are the best through i-2 and i-1; i takes max(first+total[i], second). Time follows the max value.

Python 3

```python
class Solution:
    def deleteAndEarn(self, nums: List[int]) -> int:
        mx = -inf
        for num in nums:
            mx = max(mx, num)
        total = [0] * (mx + 1)
        for num in nums:
            total[num] += num
        first = total[0]
        second = max(total[0], total[1])
        for i in range(2, mx + 1):
            cur = max(first + total[i], second)
            first = second
            second = cur
        return second
```

Complexity. Time: O(n + M), M = largest number. Space: O(M).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Deleting x also removes every number within distance
# k of x
# 2. Return the values that are deleted for points


# ========================================================
# FOLLOW-UP 1: Deleting x also removes every number within
# distance k of x
# ========================================================

# ---- follow-up 1: function ----
# Answer: Group the numbers by value (points per value);
# taking value v forbids values v - k .. v + k, so the
# best total up to v comes from skipping v or taking it
# plus the best total up to v - k - 1.
# Time: O(M), M = largest number.  Space: O(M).
def delete_and_earn_distance(nums, k):
    points = {}
    for x in nums:
        points[x] = points.get(x, 0) + x
    if not points:
        return 0
    biggest = max(points)
    best = [0] * (biggest + 2)
    for v in range(1, biggest + 1):
        skip = best[v - 1]
        before = v - k - 1
        take = points.get(v, 0) + (best[before] if before >= 0 else 0)
        best[v] = take if take > skip else skip
    return best[biggest]


# ========================================================
# FOLLOW-UP 2: Return the values that are deleted for
# points
# ========================================================

# ---- follow-up 2: function ----
# Answer: Walk back through the table: if taking the value
# gave the best total, record it and jump to the previous
# allowed value.
# Time: O(M).  Space: O(M).
def values_taken(nums):
    points = {}
    for x in nums:
```

```
        points[x] = points.get(x, 0) + x
    if not points:
        return []
    biggest = max(points)
    best = [0] * (biggest + 2)
    for v in range(1, biggest + 1):
        take = points.get(v, 0) + (best[v - 2] if v >= 2 else 0)
        best[v] = take if take > best[v - 1] else best[v - 1]
    taken = []
    v = biggest
    while v >= 1:
        take = points.get(v, 0) + (best[v - 2] if v >= 2 else 0)
        if points.get(v, 0) > 0 and take == best[v] and take >= best[v - 1]:
            taken.append(v)
```

```
            v -= 2
        else:
            v -= 1
    taken.reverse()
    return taken
```

718. Maximum Length of Repeated Subarray

Medium · Array, Binary Search, Dynamic Programming, Sliding Window, Hash Function, Rolling Hash

leetcode.com/problems/maximum-length-of-repeated-subarray

PROBLEM

Given two integer arrays `nums1` and `nums2`, return *the maximum length of a subarray that appears in **both** arrays*.

Example 1:

```
Input: nums1 = [1,2,3,2,1], nums2 = [3,2,1,4,7]
Output: 3
Explanation: The repeated subarray with maximum length is [3,2,1].
```

Example 2:

```
Input: nums1 = [0,0,0,0,0], nums2 = [0,0,0,0,0]
Output: 5
Explanation: The repeated subarray with maximum length is [0,0,0,0,0].
```

Constraints:

- `1 <= nums1.length, nums2.length <= 1000`
- `0 <= nums1[i], nums2[i] <= 100`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Find the longest common contiguous subarray. Lengths are 1000, so matching from every pair of starts is too slow, and LCS DP does not enforce contiguity. Contiguity means a suffix grows only when the current pair matches: it is one plus the suffix of the prefixes, otherwise zero. Let f[i][j] be the common suffix ending at nums1[i-1] and nums2[j-1]. Take the global maximum. Time O(mn).

Python 3

```python
class Solution:
    def findLength(self, nums1: List[int], nums2: List[int]) -> int:
        m, n = len(nums1), len(nums2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        ans = 0
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if nums1[i - 1] == nums2[j - 1]:
                    f[i][j] = f[i - 1][j - 1] + 1
                    ans = max(ans, f[i][j])
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the repeated subarray itself
# 2. The longest common SUBSEQUENCE (elements need not be
# next to each other)


# ============================================================
# FOLLOW-UP 1: Return the repeated subarray itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same table of common suffix lengths;
# remember where the longest one ends.
# Time: O(m * n).  Space: O(n).
def longest_common_subarray(a, b):
    best_length = 0
    best_end = 0
    previous = [0] * (len(b) + 1)
    for i in range(1, len(a) + 1):
        current = [0] * (len(b) + 1)
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                current[j] = previous[j - 1] + 1
                if current[j] > best_length:
                    best_length = current[j]
                    best_end = i
        previous = current
    return a[best_end - best_length:best_end]


# ============================================================
# FOLLOW-UP 2: The longest common SUBSEQUENCE (elements
# need not be next to each other)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Now a mismatch does not reset to 0: the table
# takes the better of dropping the last element of either
# array.
# Time: O(m * n).  Space: O(n).
def longest_common_subsequence(a, b):
    previous = [0] * (len(b) + 1)
    for i in range(1, len(a) + 1):
        current = [0] * (len(b) + 1)
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                current[j] = previous[j - 1] + 1
            elif previous[j] >= current[j - 1]:
                current[j] = previous[j]
            else:
                current[j] = current[j - 1]
        previous = current
    return previous[len(b)]
```

PATTERN 14 OF 41

Recursion / Divide and Conquer

20 problems · 1 Easy · 10 Medium · 9 Hard

THE IDEA

Split the problem into smaller copies of itself, solve each, and combine. Write the base case first and trust the recursive call.

Signals: merge sort, quick select, build a tree from traversals, power function, closest pair, 'solve left half and right half'.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
def solve(lo, hi):
    if lo >= hi:
        return base_case
    mid = (lo + hi) // 2
    left = solve(lo, mid)
    right = solve(mid + 1, hi)
    return combine(left, right)
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Convert Sorted Array to Binary Search Tree	Easy	LC 108	725
2	★ **Merge Sort — substitute: Sort an Array**	Medium	LC 912	729
3	★ **Pow(x, n)**	Medium	LC 50	732
4	Maximum Subarray (see page)	Medium	LC 53	258
5	★ **Different Ways to Add Parentheses**	Medium	LC 241	734
6	Kth Largest Element in an Array	Medium	LC 215	737
7	Construct Binary Tree from Preorder and Inorder Traversal (see page)	Medium	LC 105	571
8	Beautiful Array	Medium	LC 932	740
9	Unique Binary Search Trees II	Medium	LC 95	742
10	Maximum Score Words Formed by Letters	Medium	LC 1255	745
11	Super Pow	Medium	LC 372	748
12	★ **The Skyline Problem**	Hard	LC 218	750
13	★ **Count of Smaller Numbers After Self**	Hard	LC 315	753
14	Median of Two Sorted Arrays (see page)	Hard	LC 4	233
15	Reverse Pairs	Hard	LC 493	759
16	★ **Expression Add Operators**	Hard	LC 282	765
17	Strange Printer	Hard	LC 664	768
18	Minimum Cost to Merge Stones	Hard	LC 1000	770
19	★ **Burst Balloons**	Hard	LC 312	773
20	Count of Range Sum	Hard	LC 327	776

108. Convert Sorted Array to Binary Search Tree

Easy · Tree, Binary Search Tree, Array, Divide and Conquer, Binary Tree

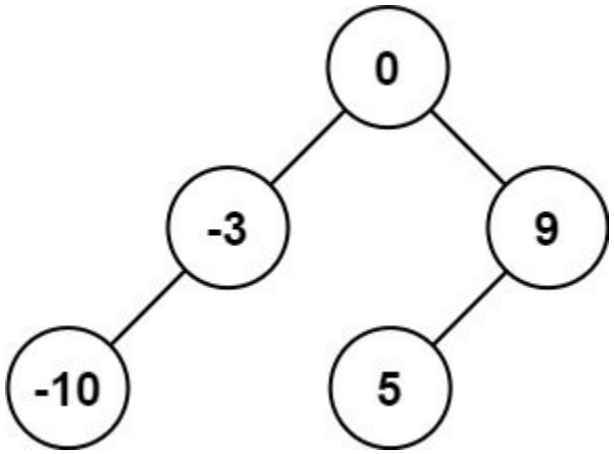
leetcode.com/problems/convert-sorted-array-to-binary-search-tree

Also appears in pattern 26 Binary Search Tree (BST)

PROBLEM

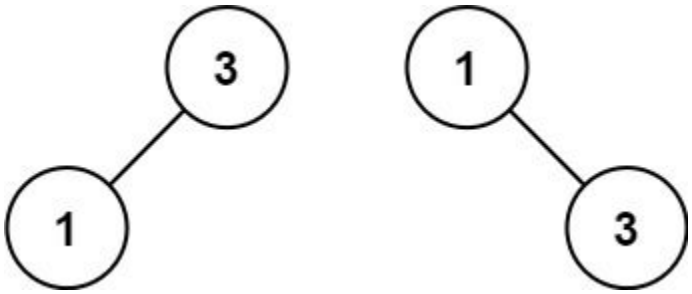
Given an integer array `nums` where the elements are sorted in **ascending order**, convert *it to a **height-balanced** binary search tree*.

Example 1:




```
Input: nums = [-10,-3,0,5,9]
Output: [0,-3,9,-10,null,5]
Explanation: [0,-10,5,null,-3,null,9] is also accepted:
```

Example 2:




```
Input: nums = [1,3]
Output: [3,1]
Explanation: [1,null,3] and [3,1] are both height-balanced BSTs.
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^4 <= nums[i] <= 10^4`
- `nums` is sorted in a **strictly increasing** order.

SOLUTION (PYTHON 3)

Solution 1: Binary Search + Recursion

How to think about it

A sorted array is already the inorder sequence of some BST. Always picking the leftmost or rightmost value as the root yields a chain of height n, which is not height-balanced. $n \le 10^4$. The midpoint of an interval leaves the two sides differing by at most one element, so the height stays balanced. Take the mid of [l,r] as the root and recurse on both halves.

We design a recursive function dfs(l, r), which represents that the values of the nodes to be constructed in the current binary search tree are within the index range [l, r] of the array nums. This function returns the root node of the constructed binary search tree.

The execution process of the function dfs(l, r) is as follows:

1. If l > r, it means the current array is empty, so return `null`.
2. If l ≤ r, take the element at index mid = floor((l + r)/2) of the array as the root node of the current binary search tree, where floor(x) denotes the floor function of x.
3. Recursively construct the left subtree of the current binary search tree, with the root node's value being the element at index mid - 1 of the array. The values of the nodes in the left subtree are within the index range [l, mid - 1] of the array.
4. Recursively construct the right subtree of the current binary search tree, with the root node's value being the element at index mid + 1 of the array. The values of the nodes in the right subtree are within the index range [mid + 1, r] of the array.
5. Return the root node of the current binary search tree.

The answer is the return value of the function dfs(0, n - 1).

The time complexity is O(n), and the space complexity is O(log n). Here, n is the length of the array nums.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sortedArrayToBST(self, nums: List[int]) -> Optional[TreeNode]:
        def dfs(l: int, r: int) -> Optional[TreeNode]:
            if l > r:
                return None
            mid = (l + r) >> 1
            return TreeNode(nums[mid], dfs(l, mid - 1), dfs(mid + 1, r))

        return dfs(0, len(nums) - 1)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The input is a SORTED LINKED LIST
# 2. Build a COMPLETE balanced BST (all levels full except
# maybe the last, filled from the left)


# ============================================================
# FOLLOW-UP 1: The input is a SORTED LINKED LIST
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
```

```
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 1: function ----
# Answer: Count the nodes, then build the tree in inorder:
# build the left half, take the next list node as the
# root, build the right half. The list pointer moves
# forward only once per node.
# Time: O(n).  Space: O(log n) recursion (plus the tree).
def sorted_list_to_bst(head):
    size = 0
    node = head
    while node:
        size += 1
        node = node.next
    current = [head]

    def build(count):
        if count == 0:
            return None
        left = build(count // 2)
        root = TreeNode(current[0].val)
        current[0] = current[0].next
        root.left = left
        root.right = build(count - count // 2 - 1)
        return root

    return build(size)


# ============================================================
# FOLLOW-UP 2: Build a COMPLETE balanced BST (all levels
# full except maybe the last, filled from the left)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Number the nodes like a heap (children 2i + 1
# and 2i + 2) and fill them in inorder, taking the sorted
# values one by one.
# Time: O(n).  Space: O(n).
```

```
def sorted_array_to_complete_bst(nums):
    nodes = []
    for value in nums:
        nodes.append(TreeNode(0))
    position = [0]

    def fill(i):
        if i >= len(nums):
            return
        fill(2 * i + 1)
        nodes[i].val = nums[position[0]]
        position[0] += 1
        fill(2 * i + 2)

    fill(0)
    for i in range(len(nums)):
        if 2 * i + 1 < len(nums):
            nodes[i].left = nodes[2 * i + 1]
        if 2 * i + 2 < len(nums):
            nodes[i].right = nodes[2 * i + 2]
```

```
    return nodes[0] if nodes else None
```

PATTERN 14 · RECURSION / DIVIDE AND CONQUER · PROBLEM 2 OF 20

912. Sort an Array

Medium · Array, Divide and Conquer, Bucket Sort, Counting Sort, Radix Sort, Sorting, Heap (Priority Queue), Merge Sort

leetcode.com/problems/sort-an-array

★ **TOP 7 PROBLEM** in Recursion / Divide and Conquer, Sorting: master this one first

Also appears in pattern 23 Sorting

PROBLEM

Given an array of integers `nums`, sort the array in ascending order and return it.

You must solve the problem **without using any built-in** functions in `O(nlog(n))` time complexity and with the smallest space complexity possible.

Example 1:

```
Input: nums = [5,2,3,1]
Output: [1,2,3,5]
Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).
```

Example 2:

```
Input: nums = [5,1,1,2,0,0]
Output: [0,0,1,1,2,5]
Explanation: Note that the values of nums are not necessarily unique.
```

Constraints:

- `1 <= nums.length <= 5 * 10^4`
- `-5 * 10^4 <= nums[i] <= 5 * 10^4`

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

n≤ 5× 10^4 needs an O(nlog n) sort; a naive quadratic pass is too slow. Three-way quicksort partitions around a random pivot into <, =, and > segments so duplicate keys do not degenerate, then recurses on the two sides.

Python 3

```python
class Solution:
    def sortArray(self, nums: List[int]) -> List[int]:
        def quick_sort(l, r):
            if l >= r:
                return
            x = nums[randint(l, r)]
            i, j, k = l - 1, r + 1, l
            while k < j:
                if nums[k] < x:
                    nums[i + 1], nums[k] = nums[k], nums[i + 1]
                    i, k = i + 1, k + 1
                elif nums[k] > x:
                    j -= 1
                    nums[j], nums[k] = nums[k], nums[j]
                else:
                    k = k + 1
            quick_sort(l, i)
            quick_sort(j, r)

        quick_sort(0, len(nums) - 1)
        return nums
```

Solution 2

> **How to think about it**
>
> Quicksort is fast on average but can unbalance and is not stable. Mergesort halves the range and merges in linear time, giving a guaranteed O(nlog n) bound at the cost of a linear buffer.

Python 3

```python
class Solution:
    def sortArray(self, nums: List[int]) -> List[int]:
        def merge_sort(l, r):
            if l >= r:
                return
            mid = (l + r) >> 1
            merge_sort(l, mid)
            merge_sort(mid + 1, r)
            i, j = l, mid + 1
            tmp = []
            while i <= mid and j <= r:
                if nums[i] <= nums[j]:
                    tmp.append(nums[i])
                    i += 1
                else:
                    tmp.append(nums[j])
                    j += 1
            if i <= mid:
                tmp.extend(nums[i : mid + 1])
            if j <= r:
                tmp.extend(nums[j : r + 1])
            for i in range(l, r + 1):
                nums[i] = tmp[i - l]

        merge_sort(0, len(nums) - 1)
        return nums
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Count the inversions (pairs i < j with nums[i] >
# nums[j]) while sorting
# 2. Quicksort with a three-way partition (fast with many
# equal numbers)


# ============================================================
# FOLLOW-UP 1: Count the inversions (pairs i < j with
# nums[i] > nums[j]) while sorting
# ============================================================

# ---- follow-up 1: function ----
# Answer: Merge sort: when an element of the right half is
# placed before the rest of the left half, it forms an
# inversion with every remaining left element.
# Time: O(n log n).  Space: O(n).
def sort_and_count(nums):
    if len(nums) <= 1:
        return list(nums), 0
    middle = len(nums) // 2
    left, left_count = sort_and_count(nums[:middle])
    right, right_count = sort_and_count(nums[middle:])
    merged = []
    i = j = 0
    count = left_count + right_count
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            count += len(left) - i
            j += 1
    merged += left[i:]
    merged += right[j:]
    return merged, count


# ============================================================
# FOLLOW-UP 2: Quicksort with a three-way partition (fast
# with many equal numbers)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Split the numbers into smaller than, equal to
# and larger than a pivot; only the smaller and larger
# parts are sorted recursively.
# Time: O(n log n) on average, O(n^2) in the worst case.
# Space: O(n).
def quicksort_three_way(nums):
    if len(nums) <= 1:
        return list(nums)
    pivot = nums[len(nums) // 2]
    smaller, equal, larger = [], [], []
    for x in nums:
        if x < pivot:
            smaller.append(x)
        elif x == pivot:
            equal.append(x)
        else:
            larger.append(x)
    return quicksort_three_way(smaller) + equal + quicksort_three_way(larger)
```

PATTERN 14 · RECURSION / DIVIDE AND CONQUER · PROBLEM 3 OF 20

50. Pow(x, n)

Medium · Recursion, Math

leetcode.com/problems/powx-n

★ **TOP 7 PROBLEM** in Recursion / Divide and Conquer, Math / Number Theory: master this one first

Also appears in pattern 34 Math / Number Theory

PROBLEM

Implement pow(x, n), which calculates `x` raised to the power `n` (i.e., x^n).

Example 1:

```
Input: x = 2.00000, n = 10
Output: 1024.00000
```

Example 2:

```
Input: x = 2.10000, n = 3
Output: 9.26100
```

Example 3:

```
Input: x = 2.00000, n = -2
Output: 0.25000
Explanation: 2-2 = 1/22 = 1/4 = 0.25
```

Constraints:

- `-100.0 < x < 100.0`
- -2^{31} `<= n <=` $2^{31}-1$
- `n` is an integer.
- Either `x` is not zero or `n > 0`.
- -10^4 `<=` x^n `<=` 10^4

SOLUTION (PYTHON 3)

Solution 1: Mathematics (Fast Powering)

How to think about it

The first idea is multiply x by itself |n| times. |n| reaches 2^{31}, so a linear loop times out; negative n also needs a reciprocal. The bottleneck is multiplying by one x at a time. $x^{2k} = (x^k)^2$ and $x^{2k+1} = x \cdot (x^k)^2$ — the exponent halves each step. Walk the bits of n: square the base, and when a bit is 1 multiply into the answer. For negative n, compute the positive power then take the reciprocal. Fast powering drops O(|n|) to O(log |n|).

The core idea of the fast powering algorithm is to decompose the exponent n into the sum of 1s on several binary bits, and then transform the nth power of x into the product of several powers of x.

The time complexity is O(log n), and the space complexity is O(1). Here, n is the exponent.

Python 3

```python
class Solution:
    def myPow(self, x: float, n: int) -> float:
        def qpow(a: float, n: int) -> float:
            ans = 1
            while n:
                if n & 1:
                    ans *= a
                a *= a
                n >>= 1
            return ans

        return qpow(x, n) if n >= 0 else 1 / qpow(x, -n)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Compute x to the power n modulo m for a huge n
# 2. The n-th Fibonacci number in O(log n) with matrix
# powers


# ============================================================
# FOLLOW-UP 1: Compute x to the power n modulo m for a
# huge n
# ============================================================

# ---- follow-up 1: function ----
# Answer: Square and multiply, reducing modulo m after
# every multiplication so the numbers stay small.
# Time: O(log n).  Space: O(1).
def power_mod(x, n, m):
    result = 1
    x %= m
    while n > 0:
        if n % 2 == 1:
            result = result * x % m
        x = x * x % m
        n //= 2
    return result


# ============================================================
# FOLLOW-UP 2: The n-th Fibonacci number in O(log n) with
# matrix powers
# ============================================================

# ---- follow-up 2: function ----
# Answer: The matrix [[1, 1], [1, 0]] raised to the n-th
# power holds Fibonacci numbers; raise it by squaring,
# like a number.
# Time: O(log n).  Space: O(1).
def multiply(a, b):
    return [[a[0][0] * b[0][0] + a[0][1] * b[1][0], a[0][0] * b[0][1] + a[0][1] * b[1][1]],
            [a[1][0] * b[0][0] + a[1][1] * b[1][0], a[1][0] * b[0][1] + a[1][1] * b[1][1]]]


def fibonacci(n):
    result = [[1, 0], [0, 1]]
    base = [[1, 1], [1, 0]]
    while n > 0:
        if n % 2 == 1:
            result = multiply(result, base)
        base = multiply(base, base)
        n //= 2
    return result[0][1]
```

241. Different Ways to Add Parentheses

Medium · Recursion, Memoization, Math, String, Dynamic Programming, Parentheses

leetcode.com/problems/different-ways-to-add-parentheses

★ **TOP 7 PROBLEM** in Recursion / Divide and Conquer: master this one first

PROBLEM

Given a string `expression` of numbers and operators, return *all possible results from computing all the different possible ways to group numbers and operators*. You may return the answer in **any order**.

The test cases are generated such that the output values fit in a 32-bit integer and the number of different results does not exceed 10^4.

Example 1:

```
Input: expression = "2-1-1"
Output: [0,2]
Explanation:
((2-1)-1) = 0
(2-(1-1)) = 2
```

Example 2:

```
Input: expression = "2*3-4*5"
Output: [-34,-14,-10,-10,10]
Explanation:
(2*(3-(4*5))) = -34
((2*3)-(4*5)) = -14
((2*(3-4))*5) = -10
(2*((3-4)*5)) = -10
(((2*3)-4)*5) = 10
```

Constraints:

- `1 <= expression.length <= 20`
- `expression` consists of digits and the operator `'+'`, `'-'`, and `'*'`.
- All the integer values in the input expression are in the range `[0, 99]`.
- The integer values in the input expression do not have a leading `'-'` or `'+'` denoting the sign.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Each parenthesization splits at some operator. Evaluating every split naively recomputes the same subexpressions. We cut at each operator, recurse for all left and right values, combine them, and memoize each substring.

Python 3

```python
class Solution:
    def diffWaysToCompute(self, expression: str) -> List[int]:
        @cache
        def dfs(exp):
            if exp.isdigit():
                return [int(exp)]
            ans = []
            for i, c in enumerate(exp):
                if c in '-+*':
                    left, right = dfs(exp[:i]), dfs(exp[i + 1 :])
                    for a in left:
                        for b in right:
                            if c == '-':
                                ans.append(a - b)
                            elif c == '+':
                                ans.append(a + b)
                            else:
                                ans.append(a * b)
            return ans

        return dfs(expression)
```

Complexity. Time: exponential: about O(4^n / sqrt(n)) (Catalan number of results). Space: the same order for the stored results.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Only count the ways
# 2. The DISTINCT results only


# ============================================================
# FOLLOW-UP 1: Only count the ways
# ============================================================

# ---- follow-up 1: function ----
# Answer: The number of results for an expression is the
# sum over every operator of (ways for the left part)
# times (ways for the right part). Remember the counts of
# parts already seen.
# Time: O(n^3) (n^2 substrings, O(n) splits each).  Space:
# O(n^2).
def count_ways(expression):
    memo = {}

    def count(text):
        if text in memo:
            return memo[text]
        total = 0
        for i in range(len(text)):
            if text[i] in "+-*":
                total += count(text[:i]) * count(text[i + 1:])
        if total == 0:
            total = 1                                   # a plain number
        memo[text] = total
        return total

    return count(expression)


# ============================================================
# FOLLOW-UP 2: The DISTINCT results only
# ============================================================

# ---- follow-up 2: function ----
# Answer: Collect the results of the two sides into sets
```

```
# and combine every pair; the set removes repeated values
# at every level.
# Time: exponential: up to about O(4^n / sqrt(n)) distinct
# results.  Space: the same order.
def distinct_results(expression):
    memo = {}

    def results(text):
        if text in memo:
            return memo[text]
        values = set()
        for i in range(len(text)):
            if text[i] in "+-*":
                for a in results(text[:i]):
                    for b in results(text[i + 1:]):
                        if text[i] == "+":
                            values.add(a + b)
                        elif text[i] == "-":
                            values.add(a - b)
                        else:
```

```
                            values.add(a * b)
        if not values:
            values.add(int(text))
        memo[text] = values
        return values

    return sorted(results(expression))
```

PATTERN 14 · RECURSION / DIVIDE AND CONQUER · PROBLEM 6 OF 20

215. Kth Largest Element in an Array

Medium · Array, Divide and Conquer, Quickselect, Sorting, Heap (Priority Queue)

leetcode.com/problems/kth-largest-element-in-an-array

★ **TOP 7 PROBLEM** in Heap / Priority Queue, Sorting: master this one first

Also appears in pattern 19 Heap / Priority Queue, 23 Sorting, 38 K-way Merge

PROBLEM

Given an integer array `nums` and an integer `k`, return *the* k^{th} *largest element in the array*.

Note that it is the k^{th} largest element in the sorted order, not the k^{th} distinct element.

Can you solve it without sorting?

Example 1:

```
Input: nums = [3,2,1,5,6,4], k = 2
Output: 5
```

Example 2:

```
Input: nums = [3,2,3,1,2,4,5,5,6], k = 4
Output: 4
```

Constraints:

- `1 <= k <= nums.length <=` 10^5
- -10^4 `<= nums[i] <=` 10^4

SOLUTIONS (PYTHON 3)

Solution 1: Quick Select

How to think about it

Full sorting finds the k-th largest but orders every other position as well. Quickselect only needs the side that contains that rank. After partitioning around a pivot we recurse only into the interval that holds the target. The implementation converts " k-th largest" into " (n-k)-th smallest".

Quick Select is an algorithm for finding the k^{th} largest or smallest element in an unsorted array. Its basic idea is to select a pivot element each time, dividing the array into two parts: one part contains elements smaller than the pivot, and the other part contains elements larger than the pivot. Then, based on the position of the pivot, it decides whether to continue the search on the left or right side until the k^{th} largest element is found.

The time complexity is O(n), and the space complexity is O(log n). Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        def quick_sort(l: int, r: int) -> int:
            if l == r:
                return nums[l]
            i, j = l - 1, r + 1
            x = nums[(l + r) >> 1]
            while i < j:
                while 1:
                    i += 1
                    if nums[i] >= x:
                        break
                while 1:
                    j -= 1
                    if nums[j] <= x:
                        break
                if i < j:
                    nums[i], nums[j] = nums[j], nums[i]
            if j < k:
                return quick_sort(j + 1, r)
            return quick_sort(l, j)

        n = len(nums)
        k = n - k
        return quick_sort(0, n - 1)
```

Solution 2: Priority Queue (Min Heap)

> **How to think about it**
>
> Quickselect is linear on average but quadratic in the worst case. A min-heap of size k is enough if we only need that order statistic. After one pass the heap stores the k largest values and its top is the answer, in O(nlog k) time.

We can maintain a min heap minQ of size k, and then iterate through the array nums, adding each element to the min heap. When the size of the min heap exceeds k, we pop the top element of the heap. This way, the final k elements in the min heap are the k largest elements in the array, and the top element of the heap is the k^{th} largest element.

The time complexity is O(nlog k), and the space complexity is O(k). Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        return nlargest(k, nums)[-1]
```

Solution 3: Counting Sort

> **How to think about it**
>
> The heap still pays a log factor. With a bounded value range we count frequencies, then walk values from high to low until k occurrences are gathered. Time is linear in n plus the value range, with no comparison sort.

We can use the idea of counting sort, counting the occurrence of each element in the array nums and recording it in a hash table cnt. Then, we iterate over the elements i from largest to smallest, subtracting the occurrence count cnt[i] each time, until k is less than or equal to 0. At this point, the element i is the k^{th} largest element in the array.

The time complexity is O(n + m), and the space complexity is O(n). Here, n is the length of the array nums, and m is the maximum value among the elements in nums.

Python 3

```
class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        cnt = Counter(nums)
        for i in count(max(cnt), -1):
            k -= cnt[i]
            if k <= 0:
                return i
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Quickselect: average O(n)
# 2. A min-heap of size k (works for a stream)


# ============================================================
# FOLLOW-UP 1: Quickselect: average O(n)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Partition around a pivot like quicksort, but
# continue only in the part that contains the wanted
# position.
# Time: O(n) on average, O(n^2) in the worst case.  Space:
# O(n).
import random


def quickselect_kth_largest(nums, k):
    target = len(nums) - k                          # the position in sorted order
    items = list(nums)
    low, high = 0, len(items) - 1
    while True:
        pivot = items[random.randint(low, high)]
        smaller = [x for x in items[low:high + 1] if x < pivot]
        equal = [x for x in items[low:high + 1] if x == pivot]
        bigger = [x for x in items[low:high + 1] if x > pivot]
        items[low:high + 1] = smaller + equal + bigger
        if target < low + len(smaller):
            high = low + len(smaller) - 1
        elif target < low + len(smaller) + len(equal):
            return pivot
        else:
            low = low + len(smaller) + len(equal)


# ============================================================
# FOLLOW-UP 2: A min-heap of size k (works for a stream)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep only the k biggest numbers seen so far in a
# min-heap; the smallest of them (the top) is the k-th
# largest.
# Time: O(n log k).  Space: O(k).
import heapq


def kth_largest_heap(nums, k):
    heap = []
    for x in nums:
        heapq.heappush(heap, x)
        if len(heap) > k:
            heapq.heappop(heap)
    return heap[0]
```

PATTERN 14 · RECURSION / DIVIDE AND CONQUER · PROBLEM 8 OF 20

932. Beautiful Array

Medium · Array, Math, Divide and Conquer

leetcode.com/problems/beautiful-array

PROBLEM

An array `nums` of length `n` is **beautiful** if:

- `nums` is a permutation of the integers in the range `[1, n]`.
- For every `0 <= i < j < n`, there is no index `k` with `i < k < j` where `2 * nums[k] == nums[i] + nums[j]`.

Given the integer `n`, return *any* ***beautiful*** *array* `nums` *of length* `n`. There will be at least one valid answer for the given `n`.

Example 1:

```
Input: n = 4
Output: [2,1,4,3]
```

Example 2:

```
Input: n = 5
Output: [3,1,2,5,4]
```

Constraints:

- `1 <= n <= 1000`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

A beautiful array forbids $2A_k = A_i + A_j$. The left side is even, so an odd/even pair A_i, A_j is always safe. Affine-map a smaller beautiful array into the odds and the evens and concatenate; the map preserves beauty, and divide-and-conquer builds 1..n.

Python 3

```python
class Solution:
    def beautifulArray(self, n: int) -> List[int]:
        if n == 1:
            return [1]
        left = self.beautifulArray((n + 1) >> 1)
        right = self.beautifulArray(n >> 1)
        left = [x * 2 - 1 for x in left]
        right = [x * 2 for x in right]
        return left + right
```

Complexity. Time: O(n log n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Check whether a given array is beautiful
# 2. Build it without recursion


# ============================================================
# FOLLOW-UP 1: Check whether a given array is beautiful
# ============================================================
```

```
# ---- follow-up 1: function ----
# Answer: For every pair i < j there must be no k between
# them with 2 * nums[k] = nums[i] + nums[j]: test with a
# position lookup.
# Time: O(n^2).  Space: O(n).
def is_beautiful(nums):
    position = {}
    for i in range(len(nums)):
        position[nums[i]] = i
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            total = nums[i] + nums[j]
            if total % 2 == 0:
                middle = total // 2
                if middle in position and i < position[middle] < j:
                    return False
    return True


# ============================================================
# FOLLOW-UP 2: Build it without recursion
# ============================================================

# ---- follow-up 2: function ----
# Answer: Start with [1] and repeat: map every number x to
# 2x - 1 (the odds) and 2x (the evens), writing the odds
# first, until the array has n numbers; drop the numbers
# above n.
# Time: O(n).  Space: O(n).
def beautiful_array_iterative(n):
    result = [1]
    while len(result) < n:
        result = [2 * x - 1 for x in result] + [2 * x for x in result]
    return [x for x in result if x <= n]
```

PATTERN 14 · RECURSION / DIVIDE AND CONQUER · PROBLEM 9 OF 20

95. Unique Binary Search Trees II

Medium · Tree, Binary Search Tree, Dynamic Programming, Backtracking, Binary Tree

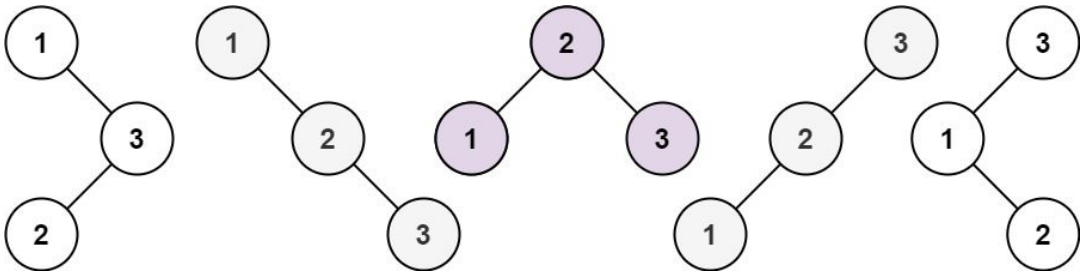
leetcode.com/problems/unique-binary-search-trees-ii

Also appears in pattern 26 Binary Search Tree (BST)

PROBLEM

Given an integer `n`, return *all the structurally unique **BST's** (binary search trees), which has exactly* `n` *nodes of unique values from* `1` *to* `n`. Return the answer in **any order**.

Example 1:




```
Input: n = 3
Output: [[1,null,2,null,3],[1,null,3,2],[2,1,3],[3,1,null,null,2],[3,2,null,1]]
```

Example 2:

```
Input: n = 1
Output: [[1]]
```

Constraints:

- `1 <= n <= 8`

SOLUTION (PYTHON 3)

Solution 1: DFS (Depth-First Search)

> **How to think about it**
>
> We need every BST on 1..n. Once the root is v, the left subtree uses only [1,v-1] and the right only [v+1,n]; the two sides are independent. Split by interval: enumerate the root, generate all left and right shapes, then take the Cartesian product. An empty interval is a null tree. n ≤ 8, Catalan numbers stay small, so search is enough.

We design a function dfs(i, j) that returns all feasible binary search trees composed of [i, j], so the answer is dfs(1, n).

The execution steps of the function dfs(i, j) are as follows:

1. If i > j, it means that there are no numbers to form a binary search tree at this time, so return a list consisting of a null node.
2. If i ≤ j, we enumerate the numbers v in [i, j] as the root node. The left subtree of the root node v is composed of [i, v - 1], and the right subtree is composed of [v + 1, j]. Finally, we take the Cartesian product of all combinations of the left and right subtrees, i.e., left × right, add the root node v, and get all binary search trees with v as the root node.

The time complexity is O(n × G(n)), and the space complexity is O(n × G(n)). Where G(n) is the Catalan number.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def generateTrees(self, n: int) -> List[Optional[TreeNode]]:
        def dfs(i: int, j: int) -> List[Optional[TreeNode]]:
            if i > j:
                return [None]
            ans = []
            for v in range(i, j + 1):
                left = dfs(i, v - 1)
                right = dfs(v + 1, j)
                for l in left:
                    for r in right:
                        ans.append(TreeNode(v, l, r))
            return ans

        return dfs(1, n)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Only count the unique trees
# 2. Count the unique BSTs with n nodes whose height is AT
# MOST h


# ============================================================
# FOLLOW-UP 1: Only count the unique trees
# ============================================================

# ---- follow-up 1: function ----
# Answer: With i as the root there are count(i - 1) left
# shapes and count(n - i) right shapes; add count(left) *
# count(right) over all roots (the Catalan numbers).
# Time: O(n^2).  Space: O(n).
def count_unique_bsts(n):
    count = [0] * (n + 1)
    count[0] = 1
    for nodes in range(1, n + 1):
        for left in range(nodes):
            count[nodes] += count[left] * count[nodes - 1 - left]
    return count[n]


# ============================================================
# FOLLOW-UP 2: Count the unique BSTs with n nodes whose
# height is AT MOST h
# ============================================================

# ---- follow-up 2: function ----
# Answer: A table over (number of nodes, height limit):
# the root splits the remaining nodes between a left and a
# right subtree that both have a height limit of h - 1.
# Time: O(h * n^2).  Space: O(h * n).
def count_bsts_height_at_most(n, h):
    ways = []
    for height in range(h + 1):
        ways.append([0] * (n + 1))
    for height in range(h + 1):
        ways[height][0] = 1
    for height in range(1, h + 1):
```

```
        for nodes in range(1, n + 1):
            for left in range(nodes):
                ways[height][nodes] += ways[height - 1][left] * ways[height - 1][nodes - 1 - left]
    return ways[h][n]
```

1255. Maximum Score Words Formed by Letters

Hard · Bit Manipulation, Array, Hash Table, String, Dynamic Programming, Backtracking, Bitmask, Counting

leetcode.com/problems/maximum-score-words-formed-by-letters

PROBLEM

Given a list of `words`, list of single `letters` (might be repeating) and `score` of every character.

Return the maximum score of **any** valid set of words formed by using the given letters (`words[i]` cannot be used two or more times).

It is not necessary to use all characters in `letters` and each letter can only be used once. Score of letters `'a'`, `'b'`, `'c'`, ... ,`'z'` is given by `score[0]`, `score[1]`, ... , `score[25]` respectively.

Example 1:

```
Input: words = ["dog","cat","dad","good"], letters = ["a","a","c","d","d","d","g","o","o"], score =
[1,0,9,5,0,0,3,0,0,0,0,0,0,2,0,0,0,0,0,0,0,0,0,0]
Output: 23
Explanation:
Score a=1, c=9, d=5, g=3, o=2
Given letters, we can form the words "dad" (5+1+5) and "good" (3+2+2+5) with a score of 23.
Words "dad" and "dog" only get a score of 21.
```

Example 2:

```
Input: words = ["xxxz","ax","bx","cx"], letters = ["z","a","b","c","x","x","x"], score =
[4,4,4,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,5,0,10]
Output: 27
Explanation:
Score a=4, b=4, c=4, x=5, z=10
Given letters, we can form the words "ax" (4+5), "bx" (4+5) and "cx" (4+5) with a score of 27.
Word "xxxz" only get a score of 25.
```

Example 3:

```
Input: words = ["leetcode"], letters = ["l","e","t","c","o","d"], score =
[0,0,1,1,1,0,0,0,0,0,0,1,0,0,1,0,0,0,0,1,0,0,0,0,0,0]
Output: 0
Explanation:
Letter "e" can only be used once.
```

Constraints:

- `1 <= words.length <= 14`
- `1 <= words[i].length <= 15`
- `1 <= letters.length <= 100`
- `letters[i].length == 1`
- `score.length == 26`
- `0 <= score[i] <= 10`
- `words[i]`, `letters[i]` contains only lower case English letters.

SOLUTION (PYTHON 3)

Solution 1: Binary Enumeration

How to think about it

There are at most 14 words, so 2^{14} subsets are enumerable. A subset is legal iff it does not exceed the stock in letters. We count the letter bank, then for each nonempty mask concatenate the chosen words, check the counts, and score legal ones. The masks exhaust the choices; the count test enforces the bank.

Given the small data range in the problem, we can use binary enumeration to enumerate all word combinations for the given word list. Then, we check whether each word combination meets the requirements of the problem. If it does, we calculate its score and finally take the word combination with the highest score.

First, we use a hash table or array cnt to record the number of occurrences of each letter in the alphabet letters.

Next, we use binary enumeration to enumerate all word combinations. Each bit in the binary represents whether each word in the word list is selected. If the ith bit is 1, it means the ith word is selected; otherwise, the ith word is not selected.

Then, we count the number of occurrences of each letter in the current word combination and record it in the hash table or array cur. If the number of occurrences of each letter in cur is not greater than the corresponding letter in cnt, it means the current word combination meets the requirements of the problem. We calculate the score of the current word combination and take the word combination with the highest score.

The time complexity is $O(2^n \times n \times M)$, and the space complexity is $O(C)$. Where n and M are the number of words in the word set and the maximum length of the word, respectively; and C is the number of letters in the alphabet, in this problem, C=26.

Python 3

```python
class Solution:
    def maxScoreWords(
        self, words: List[str], letters: List[str], score: List[int]
    ) -> int:
        cnt = Counter(letters)
        n = len(words)
        ans = 0
        for i in range(1 << n):
            cur = Counter(''.join([words[j] for j in range(n) if i >> j & 1]))
            if all(v <= cnt[c] for c, v in cur.items()):
                t = sum(v * score[ord(c) - ord('a')] for c, v in cur.items())
                ans = max(ans, t)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the words that give the best score
# 2. Each word may be used ANY number of times


# ========================================================
# FOLLOW-UP 1: Return the words that give the best score
# ========================================================

# ---- follow-up 1: function ----
# Answer: Backtracking over the words (take or skip) with
# the remaining letter counts; remember the best list.
# Time: O(2^w * L), w = number of words, L = word length.
# Space: O(w) recursion.
def best_words(words, letters, score):
    available = {}
    for ch in letters:
        available[ch] = available.get(ch, 0) + 1
```

```
    best = [0, []]

    def go(i, remaining, chosen, total):
        if total > best[0]:
            best[0] = total
            best[1] = list(chosen)
        if i == len(words):
            return
        for k in range(i, len(words)):
            need = {}
            for ch in words[k]:
                need[ch] = need.get(ch, 0) + 1
            if all(remaining.get(ch, 0) >= count for ch, count in need.items()):
                for ch, count in need.items():
                    remaining[ch] -= count
                chosen.append(words[k])
                gained = 0
                for ch in words[k]:
                    gained += score[ord(ch) - ord("a")]
                go(k + 1, remaining, chosen, total + gained)
                chosen.pop()
                for ch, count in need.items():
                    remaining[ch] += count

    go(0, available, [], 0)
    return best[0], best[1]


# ============================================================
# FOLLOW-UP 2: Each word may be used ANY number of times
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same backtracking, but after taking a word you
# may take it again (the next index is the same one).
# Time: exponential in the number of copies that fit.
# Space: O(depth) recursion.
def best_score_reusable(words, letters, score):
    available = {}
    for ch in letters:
        available[ch] = available.get(ch, 0) + 1
```

```
    def go(start, remaining):
        best = 0
        for k in range(start, len(words)):
            need = {}
            for ch in words[k]:
                need[ch] = need.get(ch, 0) + 1
            if all(remaining.get(ch, 0) >= count for ch, count in need.items()):
                for ch, count in need.items():
                    remaining[ch] -= count
                gained = 0
                for ch in words[k]:
                    gained += score[ord(ch) - ord("a")]
                total = gained + go(k, remaining)
                if total > best:
                    best = total
                for ch, count in need.items():
                    remaining[ch] += count
        return best

    return go(0, available)
```

PATTERN 14 · RECURSION / DIVIDE AND CONQUER · PROBLEM 11 OF 20

372. Super Pow

Medium · Math, Divide and Conquer, Euler's Totient Function, Euler's Theorem

leetcode.com/problems/super-pow

Also appears in pattern 34 Math / Number Theory

PROBLEM

Your task is to calculate a^b mod `1337` where `a` is a positive integer and `b` is an extremely large positive integer given in the form of an array.

Example 1:

Input: a = 2, b = [3]

Output: 8

Explanation:

The array `b = [3]` represents the exponent 3. Therefore, $a^3 = 2^3 = 8$, so the result is 8.

Example 2:

Input: a = 2, b = [1,0]

Output: 1024

Explanation:

The array `b = [1, 0]` represents the exponent 10. Therefore, $a^{10} = 2^{10} = 1024$, so the result is 1024.

Example 3:

Input: a = 1, b = [4,3,3,8,5,2]

Output: 1

Explanation:

The array `b = [4, 3, 3, 8, 5, 2]` represents a positive exponent. Since `a = 1`, any positive power of 1 is 1. Therefore, the result is 1.

Constraints:

- `1 <= a <= 2^31 - 1`
- `1 <= b.length <= 2000`
- `0 <= b[i] <= 9`
- `b` does not contain leading zeros.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

a is raised to a huge decimal b given as digits; return a^b mod 1337. Building b as one integer is awkward. Write $b=\ldots+e_i \cdot 10^i$. From low digit to high: multiply by a^e, then replace the base with a^{10}. Modular fast pow keeps each step O(log e).

Python 3

```
class Solution:
    def superPow(self, a: int, b: List[int]) -> int:
        mod = 1337
        ans = 1
        for e in b[::-1]:
            ans = ans * pow(a, e, mod) % mod
            a = pow(a, 10, mod)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. a to the power of a huge exponent given as a string
# of digits, modulo m
# 2. Only the LAST DIGIT of a to the power b


# ========================================================
# FOLLOW-UP 1: a to the power of a huge exponent given as
# a string of digits, modulo m
# ========================================================

# ---- follow-up 1: function ----
# Answer: Process the exponent digit by digit: a^(10 * e +
# d) = (a^e)^10 * a^d, each step done with fast modular
# exponentiation.
# Time: O(d), d = digits of the exponent (each step a
# constant number of multiplications).  Space: O(1).
def power_mod(x, n, m):
    result = 1
    x %= m
    while n > 0:
        if n % 2 == 1:
            result = result * x % m
        x = x * x % m
        n //= 2
    return result


def pow_mod_big_exponent(a, exponent_digits, m):
    result = 1
    for ch in exponent_digits:
        result = power_mod(result, 10, m) * power_mod(a, int(ch), m) % m
    return result


# ========================================================
# FOLLOW-UP 2: Only the LAST DIGIT of a to the power b
# ========================================================

# ---- follow-up 2: function ----
# Answer: The last digit repeats with a cycle of length at
# most 4 in the exponent, so reduce the exponent modulo 4
# (using 4 when it is 0 and b is positive).
# Time: O(1).  Space: O(1).
def last_digit_of_power(a, b):
    if b == 0:
        return 1
    last = a % 10
    cycle = b % 4
    if cycle == 0:
        cycle = 4
    return last ** cycle % 10
```

PATTERN 14 · RECURSION / DIVIDE AND CONQUER · PROBLEM 12 OF 20

218. The Skyline Problem

Hard · Binary Indexed Tree, Segment Tree, Array, Divide and Conquer, Ordered Set, Sorting, Sweep Line, Heap (Priority Queue)

leetcode.com/problems/the-skyline-problem

★ **TOP 7 PROBLEM** in Recursion / Divide and Conquer, Segment Tree: master this one first

Also appears in pattern 40 Segment Tree

PROBLEM

A city's **skyline** is the outer contour of the silhouette formed by all the buildings in that city when viewed from a distance. Given the locations and heights of all the buildings, return *the **skyline** formed by these buildings collectively*.

The geometric information of each building is given in the array `buildings` where `buildings[i] = [left_i, right_i, height_i]`:

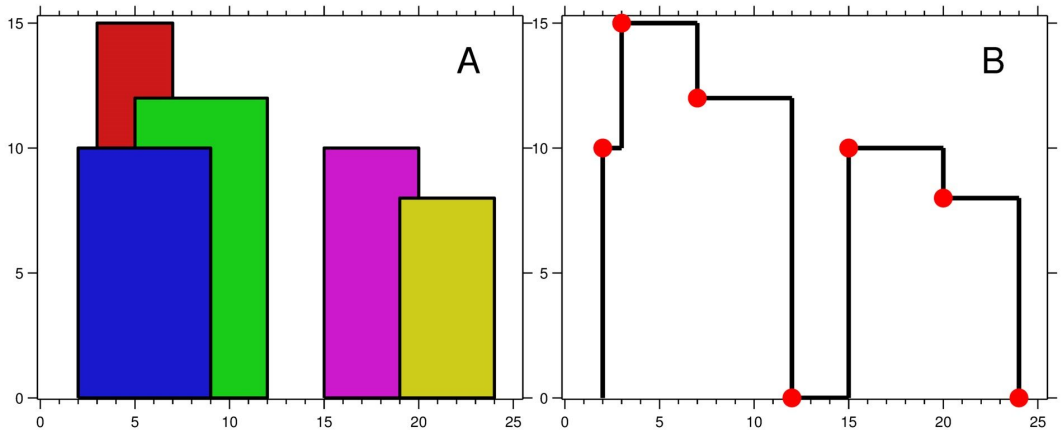
- `left_i` is the x coordinate of the left edge of the `i^th` building.
- `right_i` is the x coordinate of the right edge of the `i^th` building.
- `height_i` is the height of the `i^th` building.

You may assume all buildings are perfect rectangles grounded on an absolutely flat surface at height `0`.

The **skyline** should be represented as a list of "key points" **sorted by their x-coordinate** in the form `[[x_1,y_1],[x_2,y_2],...]`. Each key point is the left endpoint of some horizontal segment in the skyline except the last point in the list, which always has a y-coordinate `0` and is used to mark the skyline's termination where the rightmost building ends. Any ground between the leftmost and rightmost buildings should be part of the skyline's contour.

Note: There must be no consecutive horizontal lines of equal height in the output skyline. For instance, `[...,[2 3],[4 5],[7 5],[11 5],[12 7],...]` is not acceptable; the three lines of height 5 should be merged into one in the final output as such: `[...,[2 3],[4 5],[12 7],...]`

Example 1:




```
Input: buildings = [[2,9,10],[3,7,15],[5,12,12],[15,20,10],[19,24,8]]
Output: [[2,10],[3,15],[7,12],[12,0],[15,10],[20,8],[24,0]]
Explanation:
Figure A shows the buildings of the input.
Figure B shows the skyline formed by those buildings. The red points in figure B represent the key
points in the output list.
```

Example 2:

```
Input: buildings = [[0,2,3],[2,5,3]]
Output: [[0,3],[5,0]]
```

Constraints:

- `1 <= buildings.length <= 10^4`
- `0 <= left_i < right_i <= 2^31 - 1`
- `1 <= height_i <= 2^31 - 1`
- `buildings` is sorted by `left_i` in non-decreasing order.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Buildings occlude one another, so comparing every height at every x is not viable. The skyline can change only at left and right edges, which become sweep events. We sort those x-coordinates and keep a max-heap of buildings that still cover the sweep line. A key point is emitted when the height changes; adjacent equal heights are merged.

Python 3

```python
from queue import PriorityQueue


class Solution:
    def getSkyline(self, buildings: List[List[int]]) -> List[List[int]]:
        skys, lines, pq = [], [], PriorityQueue()
        for build in buildings:
            lines.extend([build[0], build[1]])
        lines.sort()
        city, n = 0, len(buildings)
        for line in lines:
            while city < n and buildings[city][0] <= line:
                pq.put([-buildings[city][2], buildings[city][0], buildings[city][1]])
                city += 1
            while not pq.empty() and pq.queue[0][2] <= line:
                pq.get()
            high = 0
            if not pq.empty():
                high = -pq.queue[0][0]
            if len(skys) > 0 and skys[-1][1] == high:
                continue
            skys.append([line, high])
        return skys
```

Complexity. Time: O(n log n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Divide and conquer: build the skylines of the two
# halves and merge them
# 2. The total area under the skyline


# ============================================================
# FOLLOW-UP 1: Divide and conquer: build the skylines of
# the two halves and merge them
# ============================================================

# ---- follow-up 1: function ----
# Answer: A skyline is a list of (x, height) points.
# Merging walks both lists in x order and keeps the higher
# of the two current heights, writing a point only when
# the combined height changes.
# Time: O(n log n).  Space: O(n).
```

```
def merge_skylines(left, right):
    result = []
    i = j = 0
    left_height = right_height = 0
    while i < len(left) or j < len(right):
        if j >= len(right) or (i < len(left) and left[i][0] < right[j][0]):
            x, left_height = left[i]
            i += 1
        elif i >= len(left) or right[j][0] < left[i][0]:
            x, right_height = right[j]
            j += 1
        else:
            x = left[i][0]
            left_height = left[i][1]
            right_height = right[j][1]
            i += 1
            j += 1
        height = left_height if left_height > right_height else right_height
        if not result or result[-1][1] != height:
            result.append([x, height])
    return result


def skyline_divide_and_conquer(buildings):
    if not buildings:
        return []
    if len(buildings) == 1:
        left, right, height = buildings[0]
        return [[left, height], [right, 0]]
    middle = len(buildings) // 2
    return merge_skylines(skyline_divide_and_conquer(buildings[:middle]),
skyline_divide_and_conquer(buildings[middle:]))


# ============================================================
# FOLLOW-UP 2: The total area under the skyline
# ============================================================

# ---- follow-up 2: function ----
# Answer: Between two neighbouring skyline points the
# height is constant, so add height times width for each
# stretch.
# Time: O(k), k = skyline points.  Space: O(1).
def skyline_area(skyline):
```

```
    area = 0
    for i in range(len(skyline) - 1):
        area += skyline[i][1] * (skyline[i + 1][0] - skyline[i][0])
    return area
```

PATTERN 14 · RECURSION / DIVIDE AND CONQUER · PROBLEM 13 OF 20

315. Count of Smaller Numbers After Self

Hard · Binary Indexed Tree, Segment Tree, Array, Binary Search, Divide and Conquer, Ordered Set, Treap, Merge Sort

leetcode.com/problems/count-of-smaller-numbers-after-self

★ **TOP 7 PROBLEM** in Recursion / Divide and Conquer, Segment Tree, Binary Indexed Tree (Fenwick Tree): master this one first

Also appears in pattern 40 Segment Tree, 41 Binary Indexed Tree (Fenwick Tree)

PROBLEM

Given an integer array `nums`, return *an integer array* `counts` *where* `counts[i]` *is the number of smaller elements to the right of* `nums[i]`.

Example 1:

```
Input: nums = [5,2,6,1]
Output: [2,1,1,0]
Explanation:
To the right of 5 there are 2 smaller elements (2 and 1).
To the right of 2 there is only 1 smaller element (1).
To the right of 6 there is 1 smaller element (1).
To the right of 1 there is 0 smaller element.
```

Example 2:

```
Input: nums = [-1]
Output: [0]
```

Example 3:

```
Input: nums = [-1,-1]
Output: [0,0]
```

Constraints:

- `1 <= nums.length <= 10^5`
- `-10^4 <= nums[i] <= 10^4`

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Count how many later elements are smaller. A nested scan is $O(n^2)$. The count is a prefix frequency over values already seen to the right. Walk right to left: discretize, insert the current rank into a Fenwick tree, then query ranks strictly below it. Insert-then-query x-1 still ignores equals. Reverse the answers to restore order.

Python 3

```python
class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x

    def update(self, x, delta):
        while x <= self.n:
            self.c[x] += delta
            x += BinaryIndexedTree.lowbit(x)

    def query(self, x):
        s = 0
        while x > 0:
            s += self.c[x]
            x -= BinaryIndexedTree.lowbit(x)
        return s


class Solution:
    def countSmaller(self, nums: List[int]) -> List[int]:
        alls = sorted(set(nums))
        m = {v: i for i, v in enumerate(alls, 1)}
        tree = BinaryIndexedTree(len(m))
        ans = []
        for v in nums[::-1]:
            x = m[v]
            tree.update(x, 1)
            ans.append(tree.query(x - 1))
        return ans[::-1]
```

Solution 2

> **How to think about it**
>
> The Fenwick tree is a prefix sum on the value domain. A segment tree splits that domain, supports point increment and range [1,x-1], and matches Method 1 with explicit interval nodes.

Python 3

```python
class Node:
    def __init__(self):
        self.l = 0
        self.r = 0
        self.v = 0


class SegmentTree:
    def __init__(self, n):
        self.tr = [Node() for _ in range(n << 2)]
        self.build(1, 1, n)

    def build(self, u, l, r):
        self.tr[u].l = l
        self.tr[u].r = r
        if l == r:
            return
        mid = (l + r) >> 1
        self.build(u << 1, l, mid)
        self.build(u << 1 | 1, mid + 1, r)

    def modify(self, u, x, v):
        if self.tr[u].l == x and self.tr[u].r == x:
            self.tr[u].v += v
            return
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        if x <= mid:
            self.modify(u << 1, x, v)
        else:
            self.modify(u << 1 | 1, x, v)
        self.pushup(u)

    def query(self, u, l, r):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            return self.tr[u].v
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        v = 0
        if l <= mid:
            v += self.query(u << 1, l, r)
        if r > mid:
            v += self.query(u << 1 | 1, l, r)
        return v

    def pushup(self, u):
        self.tr[u].v = self.tr[u << 1].v + self.tr[u << 1 | 1].v


class Solution:
    def countSmaller(self, nums: List[int]) -> List[int]:
        s = sorted(set(nums))
        m = {v: i for i, v in enumerate(s, 1)}
        tree = SegmentTree(len(s))
        ans = []
        for v in nums[::-1]:
            x = m[v]
            ans.append(tree.query(1, 1, x - 1))
            tree.modify(1, x, 1)
        return ans[::-1]
```

Complexity. Time: O(n log n). Space: O(n).

Solution 3: Merge Sort

> **How to think about it**
>
> The first two methods need discretization and an extra tree. During merge, when a left value is ≤ the current right value, exactly j consumed right elements are smaller and later; add j to that index. No value tree is required; time stays O(nlog n).

During the merge phase of merge sort, when a left element left[i] ≤ right[j], it means exactly j elements on the right side are smaller than left[i], so we accumulate j into the count of left[i].

Once all right elements are exhausted, all right elements are smaller than each remaining left element, so we accumulate the right array's full length into each remaining left element's count.

Note: In C++, merge sort on very large arrays may suffer Memory Limit Exceeded. Use a buffer array to avoid excessive memory allocations.

Complexity

- Time complexity: O(n log n), the standard time complexity for merge sort.
- Space complexity: O(n), the standard space complexity of recursion stack.

Python 3

```python
class Solution:
    def countSmaller(self, nums: list[int]) -> list[int]:
        self.right_smaller_counts = [0] * len(nums)

        nums_indices = [(num, idx) for idx, num in enumerate(nums)]
        self.merge_sort(nums_indices)

        return self.right_smaller_counts

    def combine_arrays(
        self,
        left_nums_indices: list[tuple[int, int]],
        right_nums_indices: list[tuple[int, int]],
    ) -> list[tuple[int, int]]:
        merged_nums_indices: list[tuple[int, int]] = []
        left_idx, right_idx = 0, 0

        while left_idx < len(left_nums_indices) and right_idx < len(right_nums_indices):
            if left_nums_indices[left_idx][0] <= right_nums_indices[right_idx][0]:
                # Iterated left side element finalizes its right smaller count.
                left_num_idx = left_nums_indices[left_idx][1]
                self.right_smaller_counts[left_num_idx] += right_idx

                merged_nums_indices.append(left_nums_indices[left_idx])
                left_idx += 1
                continue

            merged_nums_indices.append(right_nums_indices[right_idx])
            right_idx += 1

        while left_idx < len(left_nums_indices):
            # Iterated left side element finalizes its right smaller count.
            left_num_idx = left_nums_indices[left_idx][1]
            self.right_smaller_counts[left_num_idx] += len(right_nums_indices)

            merged_nums_indices.append(left_nums_indices[left_idx])
            left_idx += 1

        while right_idx < len(right_nums_indices):
            merged_nums_indices.append(right_nums_indices[right_idx])
            right_idx += 1

        return merged_nums_indices

    def merge_sort(self, nums_indices: list[tuple[int, int]]) -> list[tuple[int, int]]:
        if len(nums_indices) == 1:
            return nums_indices  # Single element.

        split_idx = len(nums_indices) // 2

        left_nums_indices = self.merge_sort(nums_indices[:split_idx])
        right_nums_indices = self.merge_sort(nums_indices[split_idx:])

        return self.combine_arrays(left_nums_indices, right_nums_indices)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count the LARGER numbers after each element
# 2. A Fenwick tree version for the smaller numbers


# ============================================================
# FOLLOW-UP 1: Count the LARGER numbers after each element
# ============================================================
```

```
# ---- follow-up 1: function ----
# Answer: Same method with the comparison reversed: count
# how many later numbers have a bigger rank.
# Time: O(n log n).  Space: O(n).
def count_larger_after_self(nums):
    ranks = {}
    for i, value in enumerate(sorted(set(nums))):
        ranks[value] = i + 1
    size = len(ranks)
    tree = [0] * (size + 2)

    def add(i):
        while i <= size:
            tree[i] += 1
            i += i & -i

    def prefix(i):
        total = 0
        while i > 0:
            total += tree[i]
            i -= i & -i
        return total

    result = [0] * len(nums)
    for i in range(len(nums) - 1, -1, -1):
        rank = ranks[nums[i]]
        result[i] = prefix(size) - prefix(rank)
        add(rank)
    return result


# ============================================================
# FOLLOW-UP 2: A Fenwick tree version for the smaller
# numbers
# ============================================================

# ---- follow-up 2: function ----
# Answer: Replace every number by its rank, walk from
# right to left, and ask the Fenwick tree how many smaller
# ranks were already inserted.
# Time: O(n log n).  Space: O(n).
def count_smaller_after_self_fenwick(nums):
    ranks = {}
    for i, value in enumerate(sorted(set(nums))):
        ranks[value] = i + 1
    size = len(ranks)
    tree = [0] * (size + 2)
    result = [0] * len(nums)
    for i in range(len(nums) - 1, -1, -1):
        rank = ranks[nums[i]]
        j = rank - 1
```

```
        total = 0
        while j > 0:
            total += tree[j]
            j -= j & -j
        result[i] = total
        j = rank
        while j <= size:
            tree[j] += 1
            j += j & -j
    return result
```

PATTERN 14 · RECURSION / DIVIDE AND CONQUER · PROBLEM 15 OF 20

493. Reverse Pairs

Hard · Binary Indexed Tree, Segment Tree, Array, Binary Search, Divide and Conquer, Ordered Set, Treap, Merge Sort

leetcode.com/problems/reverse-pairs

★ **TOP 7 PROBLEM** in Segment Tree, Binary Indexed Tree (Fenwick Tree): master this one first

Also appears in pattern 40 Segment Tree, 41 Binary Indexed Tree (Fenwick Tree)

PROBLEM

Given an integer array `nums`, return *the number of **reverse pairs** in the array*.

A **reverse pair** is a pair `(i, j)` where:

- `0 <= i < j < nums.length` and
- `nums[i] > 2 * nums[j]`.

Example 1:

```
Input: nums = [1,3,2,3,1]
Output: 2
Explanation: The reverse pairs are:
(1, 4) --> nums[1] = 3, nums[4] = 1, 3 > 2 * 1
(3, 4) --> nums[3] = 3, nums[4] = 1, 3 > 2 * 1
```

Example 2:

```
Input: nums = [2,4,3,5,1]
Output: 3
Explanation: The reverse pairs are:
(1, 4) --> nums[1] = 4, nums[4] = 1, 4 > 2 * 1
(2, 4) --> nums[2] = 3, nums[4] = 1, 3 > 2 * 1
(3, 4) --> nums[3] = 5, nums[4] = 1, 5 > 2 * 1
```

Constraints:

- $1 \le nums.length \le 5 * 10^4$
- $-2^{31} \le nums[i] \le 2^{31} - 1$

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

A reverse pair is i<j and nums[i]>2 nums[j]. A double loop is O(n^2). During mergesort both halves are sorted, so cross pairs can be counted before merging. Recurse on both halves, then for each right value advance a left pointer until nums[i]>2 nums[j]; the leftover left side all form pairs. Merge by raw value afterwards. Counting and merging stay separate: the count compares against 2 nums[j], the merge compares the original values.

Python 3

```python
class Solution:
    def reversePairs(self, nums: List[int]) -> int:
        def merge_sort(l, r):
            if l >= r:
                return 0
            mid = (l + r) >> 1
            ans = merge_sort(l, mid) + merge_sort(mid + 1, r)
            t = []
            i, j = l, mid + 1
            while i <= mid and j <= r:
                if nums[i] <= 2 * nums[j]:
                    i += 1
                else:
                    ans += mid - i + 1
                    j += 1
            i, j = l, mid + 1
            while i <= mid and j <= r:
                if nums[i] <= nums[j]:
                    t.append(nums[i])
                    i += 1
                else:
                    t.append(nums[j])
                    j += 1
            t.extend(nums[i : mid + 1])
            t.extend(nums[j : r + 1])
            nums[l : r + 1] = t
            return ans

        return merge_sort(0, len(nums) - 1)
```

Solution 2

How to think about it

Solution 1 rewrites the array. After discretizing nums and 2 nums, walk right to left, insert 2 nums[j] into a Fenwick tree, and query how many inserted values are less than nums[i]. The relative order of the original array is not needed beyond the right-to-left scan; the tree stores the 2 nums already seen.

Python 3

```python
class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x

    def update(self, x, delta):
        while x <= self.n:
            self.c[x] += delta
            x += BinaryIndexedTree.lowbit(x)

    def query(self, x):
        s = 0
        while x > 0:
            s += self.c[x]
            x -= BinaryIndexedTree.lowbit(x)
        return s


class Solution:
    def reversePairs(self, nums: List[int]) -> int:
        s = set()
        for num in nums:
            s.add(num)
            s.add(num * 2)
        alls = sorted(s)
        m = {v: i for i, v in enumerate(alls, 1)}
        ans = 0
        tree = BinaryIndexedTree(len(m))
        for num in nums[::-1]:
            ans += tree.query(m[num] - 1)
            tree.update(m[num * 2], 1)
        return ans
```

Complexity. Time: O(n log n). Space: O(n).

Solution 3

> **How to think about it**
>
> Replace the Fenwick tree with a segment tree: same discretization, insert 2 nums from the right, query the count below nums[i]. Same algebra, with explicit interval nodes.

Python 3

```python
class Node:
    def __init__(self):
        self.l = 0
        self.r = 0
        self.v = 0


class SegmentTree:
    def __init__(self, n):
        self.tr = [Node() for _ in range(4 * n)]
        self.build(1, 1, n)

    def build(self, u, l, r):
        self.tr[u].l = l
        self.tr[u].r = r
        if l == r:
            return
        mid = (l + r) >> 1
        self.build(u << 1, l, mid)
        self.build(u << 1 | 1, mid + 1, r)

    def modify(self, u, x, v):
        if self.tr[u].l == x and self.tr[u].r == x:
            self.tr[u].v += 1
            return
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        if x <= mid:
            self.modify(u << 1, x, v)
        else:
            self.modify(u << 1 | 1, x, v)
        self.pushup(u)

    def pushup(self, u):
        self.tr[u].v = self.tr[u << 1].v + self.tr[u << 1 | 1].v

    def query(self, u, l, r):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            return self.tr[u].v
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        v = 0
        if l <= mid:
            v += self.query(u << 1, l, r)
        if r > mid:
            v += self.query(u << 1 | 1, l, r)
        return v


class Solution:
    def reversePairs(self, nums: List[int]) -> int:
        s = set()
        for num in nums:
            s.add(num)
            s.add(num * 2)
        alls = sorted(s)
        m = {v: i for i, v in enumerate(alls, 1)}
        tree = SegmentTree(len(m))
        ans = 0
        for v in nums[::-1]:
            x = m[v]
            ans += tree.query(1, 1, x - 1)
            tree.modify(1, m[v * 2], 1)
        return ans
```

Complexity. Time: O(n log n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Count the plain inversions (nums[i] > nums[j], i < j)
# 2. Count the important reverse pairs with a Fenwick tree


# ============================================================
# FOLLOW-UP 1: Count the plain inversions (nums[i] >
# nums[j], i < j)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Merge sort: every time an element of the right
# half is placed before the remaining left elements, count
# those left elements.
# Time: O(n log n).  Space: O(n).
def count_inversions(nums):
    def sort(items):
        if len(items) <= 1:
            return items, 0
        middle = len(items) // 2
        left, a = sort(items[:middle])
        right, b = sort(items[middle:])
        merged = []
        i = j = 0
        count = a + b
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                merged.append(left[i])
                i += 1
            else:
                merged.append(right[j])
                count += len(left) - i
                j += 1
        merged += left[i:]
        merged += right[j:]
        return merged, count

    return sort(list(nums))[1]


# ============================================================
# FOLLOW-UP 2: Count the important reverse pairs with a
# Fenwick tree
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk from the left; for every number count how
# many earlier numbers are greater than twice... (more
# precisely, how many earlier values x have x > 2 *
# current). Compress all the values and twice the values
# into ranks first.
# Time: O(n log n).  Space: O(n).
def reverse_pairs_fenwick(nums):
    values = set()
    for x in nums:
        values.add(x)
        values.add(2 * x)
    ranks = {}
    for i, value in enumerate(sorted(values)):
        ranks[value] = i + 1
```

```
    size = len(ranks)
    tree = [0] * (size + 2)

    def add(i):
```

```
        while i <= size:
            tree[i] += 1
            i += i & -i

    def prefix(i):
        total = 0
        while i > 0:
            total += tree[i]
            i -= i & -i
        return total

    count = 0
    for x in nums:
        count += prefix(size) - prefix(ranks[2 * x])         # earlier numbers whose value is larger than
2x
        add(ranks[x])
    return count
```

PATTERN 14 · RECURSION / DIVIDE AND CONQUER · PROBLEM 16 OF 20

282. Expression Add Operators

Hard · Math, String, Backtracking

leetcode.com/problems/expression-add-operators

★ **TOP 7 PROBLEM** in Recursion / Divide and Conquer: master this one first

PROBLEM

Given a string `num` that contains only digits and an integer `target`, return ***all possibilities*** *to insert the binary operators* `'+'`, `'-'`, *and/or* `'*'` *between the digits of* `num` *so that the resultant expression evaluates to the* `target` *value.*

Note that operands in the returned expressions **should not** contain leading zeros.

Note that a number can contain multiple digits.

Example 1:

```
Input: num = "123", target = 6
Output: ["1*2*3","1+2+3"]
Explanation: Both "1*2*3" and "1+2+3" evaluate to 6.
```

Example 2:

```
Input: num = "232", target = 8
Output: ["2*3+2","2+3*2"]
Explanation: Both "2*3+2" and "2+3*2" evaluate to 8.
```

Example 3:

```
Input: num = "3456237490", target = 9191
Output: []
Explanation: There are no expressions that can be created from "3456237490" to evaluate to 9191.
```

Constraints:

- `1 <= num.length <= 10`
- `num` consists of only digits.
- -2^{31} `<= target <=` $2^{31} - 1$

SOLUTION (PYTHON 3)

Solution 1

How to think about it

We insert +, -, * or concatenate digits. Multiplication binds tighter than addition, and concatenated numbers may not have leading zeros. DFS keeps the last operand prev and the value curr. Plus and minus update curr directly; multiply rewinds the last term via curr-prev+prev×next.

Python 3

```python
class Solution:
    def addOperators(self, num: str, target: int) -> List[str]:
        ans = []

        def dfs(u, prev, curr, path):
            if u == len(num):
                if curr == target:
                    ans.append(path)
                return
            for i in range(u, len(num)):
                if i != u and num[u] == '0':
                    break
                next = int(num[u : i + 1])
                if u == 0:
                    dfs(i + 1, next, next, path + str(next))
                else:
                    dfs(i + 1, next, curr + next, path + "+" + str(next))
                    dfs(i + 1, -next, curr - next, path + "-" + str(next))
                    dfs(
                        i + 1,
                        prev * next,
                        curr - prev + prev * next,
                        path + "*" + str(next),
                    )

        dfs(0, 0, 0, "")
        return ans
```

Complexity. Time: O(n * 4^n). Space: O(n) recursion depth (plus the answers).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Only COUNT the expressions
# 2. Only + and - are allowed


# ============================================================
# FOLLOW-UP 1: Only COUNT the expressions
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same search, but add one at the end of every
# complete expression that reaches the target instead of
# building its text.
# Time: O(4^n).  Space: O(n) recursion.
def count_expressions(num, target):
    count = [0]

    def go(index, value, last):
        if index == len(num):
            if value == target:
                count[0] += 1
            return
        for end in range(index + 1, len(num) + 1):
            text = num[index:end]
            if len(text) > 1 and text[0] == "0":
                break
            current = int(text)
            if index == 0:
                go(end, current, current)
            else:
                go(end, value + current, current)
                go(end, value - current, -current)
                go(end, value - last + last * current, last * current)
```

```
    go(0, 0, 0)
    return count[0]


# ============================================================
# FOLLOW-UP 2: Only + and - are allowed
# ============================================================

# ---- follow-up 2: function ----
# Answer: Without * there is no need to remember the last
# term: the running value alone decides.
# Time: O(2^n * n).  Space: O(n) recursion plus the
# results.
def add_plus_minus(num, target):
    results = []

    def go(index, value, expression):
        if index == len(num):
            if value == target:
                results.append(expression)
            return
        for end in range(index + 1, len(num) + 1):
            text = num[index:end]
            if len(text) > 1 and text[0] == "0":
                break
            current = int(text)
```

```
            if index == 0:
                go(end, current, text)
            else:
                go(end, value + current, expression + "+" + text)
                go(end, value - current, expression + "-" + text)

    go(0, 0, "")
    return results
```

664. Strange Printer

Hard · String, Dynamic Programming

leetcode.com/problems/strange-printer

PROBLEM

There is a strange printer with the following two special properties:

- The printer can only print a sequence of **the same character** each time.
- At each turn, the printer can print new characters starting from and ending at any place and will cover the original existing characters.

Given a string `s`, return *the minimum number of turns the printer needed to print it*.

Example 1:

```
Input: s = "aaabbb"
Output: 2
Explanation: Print "aaa" first and then print "bbb".
```

Example 2:

```
Input: s = "aba"
Output: 2
Explanation: Print "aaa" first and then print "b" from the second place of the string, which will cover the existing character 'a'.
```

Constraints:

- `1 <= s.length <= 100`
- `s` consists of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

> **How to think about it**
>
> One print paints a contiguous segment with the same letter. n≤ 100 suits interval DP. f[i][j] is the fewest prints for s[i..j]. Equal endpoints reuse the last stroke; otherwise try every split. Fill shorter intervals first.

We define f[i][j] as the minimum operations to print s[i..j], with the initial value f[i][j]=∞, and the answer is f[0][n-1], where n is the length of string s.

Consider f[i][j], if s[i] = s[j], we can print s[j] when print s[i], so we can ignore s[j] and continue to print s[i+1..j-1]. If s[i] ≠ s[j], we need to print the substring separately, i.e. s[i..k] and s[k+1..j], where k ∈ [i,j). So we can have the following transition equation:

$$f[i][j]= \text{cases } 1, \& \text{ if } i=j \setminus f[i][j-1], \& \text{ if } s[i]=s[j] \setminus \min_{i \le k < j} \{f[i][k]+f[k+1][j]\}, \& \text{ otherwise cases}$$

We can enumerate i from large to small and j from small to large, so that we can ensure that f[i][j-1], f[i][k] and f[k+1][j] have been calculated when we calculate f[i][j].

The time complexity is $O(n^3)$ and the space complexity is $O(n^2)$. Where n is the length of string s.

Python 3

```python
class Solution:
    def strangePrinter(self, s: str) -> int:
        n = len(s)
        f = [[inf] * n for _ in range(n)]
        for i in range(n - 1, -1, -1):
            f[i][i] = 1
            for j in range(i + 1, n):
                if s[i] == s[j]:
                    f[i][j] = f[i][j - 1]
                else:
                    for k in range(i, j):
                        f[i][j] = min(f[i][j], f[i][k] + f[k + 1][j])
        return f[0][-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Top-down version with memoisation on the string
# without repeated neighbours


# ============================================================
# FOLLOW-UP 1: Top-down version with memoisation on the
# string without repeated neighbours
# ============================================================

# ---- follow-up 1: function ----
# Answer: First squeeze equal neighbours into one letter.
# Then turns(i, j) = turns(i, j - 1) + 1, or merge the
# print of s[j] with an earlier equal letter s[k]:
# turns(i, k) + turns(k + 1, j - 1) for every k with s[k]
# = s[j].
# Time: O(n^3).  Space: O(n^2).
def strange_printer_memo(s):
    letters = []
    for ch in s:
        if not letters or letters[-1] != ch:
            letters.append(ch)
    memo = {}

    def turns(i, j):
        if i > j:
            return 0
        if (i, j) in memo:
            return memo[(i, j)]
        best = turns(i, j - 1) + 1
        for k in range(i, j):
            if letters[k] == letters[j]:
                candidate = turns(i, k) + turns(k + 1, j - 1)
                if candidate < best:
                    best = candidate
        memo[(i, j)] = best
        return best

    return turns(0, len(letters) - 1)
```

1000. Minimum Cost to Merge Stones

Hard · Array, Dynamic Programming, Prefix Sum

leetcode.com/problems/minimum-cost-to-merge-stones

PROBLEM

There are `n` piles of `stones` arranged in a row. The i^{th} pile has `stones[i]` stones.

A move consists of merging exactly `k` **consecutive** piles into one pile, and the cost of this move is equal to the total number of stones in these `k` piles.

Return *the minimum cost to merge all piles of stones into one pile*. If it is impossible, return `-1`.

Example 1:

```
Input: stones = [3,2,4,1], k = 2
Output: 20
Explanation: We start with [3, 2, 4, 1].
We merge [3, 2] for a cost of 5, and we are left with [5, 4, 1].
We merge [4, 1] for a cost of 5, and we are left with [5, 5].
We merge [5, 5] for a cost of 10, and we are left with [10].
The total cost was 20, and this is the minimum possible.
```

Example 2:

```
Input: stones = [3,2,4,1], k = 3
Output: -1
Explanation: After any merge operation, there are 2 piles left, and we can't merge anymore. So the
task is impossible.
```

Example 3:

```
Input: stones = [3,5,1,2,6], k = 3
Output: 25
Explanation: We start with [3, 5, 1, 2, 6].
We merge [5, 1, 2] for a cost of 8, and we are left with [3, 8, 6].
We merge [3, 8, 6] for a cost of 17, and we are left with [17].
The total cost was 25, and this is the minimum possible.
```

Constraints:

- `n == stones.length`
- `1 <= n <= 30`
- `1 <= stones[i] <= 100`
- `2 <= k <= 30`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Enumerating every merge order is correct, but with n ≤ 30 the number of valid partitions grows too quickly to search. Each move only merges K consecutive piles, so the cost is determined by subproblems of the form “turn a contiguous segment into a given number of piles,” and the same interval is reached by many orders. Each merge reduces the pile count by K-1, so a single pile exists only when (n-1) mod (K-1)=0; otherwise the answer is -1. When it is feasible, merging [i,j] into k piles means some prefix becomes 1 pile and the suffix becomes k-1 piles; merging into 1 pile means first obtaining K piles and then paying the sum of that interval. We therefore fill f[i][j][k] by increasing interval length and keep a prefix sum s for O(1) range sums. The split h always uses f[i][h][1] on the left, matching the consecutive-merge constraint. The answer is f[1][n][1].

Python 3

```python
class Solution:
    def mergeStones(self, stones: List[int], K: int) -> int:
        n = len(stones)
        if (n - 1) % (K - 1):
            return -1
        s = list(accumulate(stones, initial=0))
        f = [[[inf] * (K + 1) for _ in range(n + 1)] for _ in range(n + 1)]
        for i in range(1, n + 1):
            f[i][i][1] = 0
        for l in range(2, n + 1):
            for i in range(1, n - l + 2):
                j = i + l - 1
                for k in range(1, K + 1):
                    for h in range(i, j):
                        f[i][j][k] = min(f[i][j][k], f[i][h][1] + f[h + 1][j][k - 1])
                f[i][j][1] = f[i][j][K] + s[j] - s[i - 1]
        return f[1][n][1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Merge exactly TWO piles at a time (k = 2)
# 2. Is it possible at all to merge into one pile with k
# at a time?


# ============================================================
# FOLLOW-UP 1: Merge exactly TWO piles at a time (k = 2)
# ============================================================

# ---- follow-up 1: function ----
# Answer: cost[i][j] = the cheapest way to merge piles
# i..j into one: split at every position and add the total
# number of stones in the range (the cost of the last
# merge).
# Time: O(n^3).  Space: O(n^2).
def merge_stones_two(stones):
    n = len(stones)
    prefix = [0]
    for x in stones:
        prefix.append(prefix[-1] + x)
    cost = []
    for i in range(n):
        cost.append([0] * n)
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            best = float("inf")
            for split in range(i, j):
```

```
                total = cost[i][split] + cost[split + 1][j]
                if total < best:
                    best = total
            cost[i][j] = best + prefix[j + 1] - prefix[i]
    return cost[0][n - 1] if n > 0 else 0


# ============================================================
# FOLLOW-UP 2: Is it possible at all to merge into one
# pile with k at a time?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Every merge removes k - 1 piles, so n - 1 must
# be a multiple of k - 1.
# Time: O(1).  Space: O(1).
def can_merge_to_one(n, k):
    return (n - 1) % (k - 1) == 0
```

312. Burst Balloons

Hard · Array, Dynamic Programming

leetcode.com/problems/burst-balloons

★ **TOP 7 PROBLEM** in Recursion / Divide and Conquer: master this one first

PROBLEM

You are given `n` balloons, indexed from `0` to `n - 1`. Each balloon is painted with a number on it represented by an array `nums`. You are asked to burst all the balloons.

If you burst the `i`th balloon, you will get `nums[i - 1] * nums[i] * nums[i + 1]` coins. If `i - 1` or `i + 1` goes out of bounds of the array, then treat it as if there is a balloon with a `1` painted on it.

Return *the maximum coins you can collect by bursting the balloons wisely*.

Example 1:

```
Input: nums = [3,1,5,8]
Output: 167
Explanation:
nums = [3,1,5,8] --> [3,5,8] --> [3,8] --> [8] --> []
coins = 3*1*5 + 3*5*8 + 1*3*8 + 1*8*1 = 167
```

Example 2:

```
Input: nums = [1,5]
Output: 10
```

Constraints:

- `n == nums.length`
- `1 <= n <= 300`
- `0 <= nums[i] <= 100`

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

The burst order changes neighbors, so searching by burst sequence revisits the same open intervals. Padding 1 at both ends fixes the multipliers on the boundary. If k is the last balloon burst in the open interval (i,j), the two sides do not interact and we add arr[i]· arr[k]· arr[j]. Let f[i][j] be the best score of clearing (i,j); fill by increasing width (i decreasing, j increasing). The answer is f[0][n+1].

Let's denote the length of the array `nums` as n. According to the problem description, we can add a 1 to both ends of the array `nums`, denoted as `arr`.

Then, we define f[i][j] as the maximum number of coins we can get by bursting all the balloons in the interval [i, j]. Therefore, the answer is f[0][n+1].

For f[i][j], we enumerate all positions k in the interval [i, j]. Suppose k is the last balloon to burst, then we can get the following state transition equation:

$$f[i][j] = \max(f[i][j], f[i][k] + f[k][j] + arr[i] \times arr[k] \times arr[j])$$

In implementation, since the state transition equation of f[i][j] involves f[i][k] and f[k][j], where i < k < j, we need to traverse i from large to small and j from small to large. This ensures that when calculating f[i][j], f[i][k] and f[k][j] have already been calculated.

Finally, we return f[0][n+1].

The time complexity is $O(n^3)$, and the space complexity is $O(n^2)$. Where n is the length of the array `nums`.

Python 3

```
class Solution:
    def maxCoins(self, nums: List[int]) -> int:
        n = len(nums)
        arr = [1] + nums + [1]
        f = [[0] * (n + 2) for _ in range(n + 2)]
        for i in range(n - 1, -1, -1):
            for j in range(i + 2, n + 2):
                for k in range(i + 1, j):
                    f[i][j] = max(f[i][j], f[i][k] + f[k][j] + arr[i] * arr[k] * arr[j])
        return f[0][-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the ORDER in which to burst the balloons
# 2. The same answer with recursion and memory (top-down)


# ============================================================
# FOLLOW-UP 1: Return the ORDER in which to burst the
# balloons
# ============================================================

# ---- follow-up 1: function ----
# Answer: Store, for every range, the balloon that is
# burst LAST; rebuild the order by recursion (the last
# balloon of a range is burst after the orders of its two
# sides).
# Time: O(n^3).  Space: O(n^2).
def burst_order(nums):
    values = [1] + list(nums) + [1]
    n = len(values)
    best = []
    last = []
    for i in range(n):
        best.append([0] * n)
        last.append([0] * n)
    for length in range(2, n):
        for left in range(n - length):
            right = left + length
            for k in range(left + 1, right):
                coins = values[left] * values[k] * values[right] + best[left][k] + best[k][right]
                if coins > best[left][right]:
                    best[left][right] = coins
                    last[left][right] = k

    def order(left, right):
        if right - left < 2:
            return []
        k = last[left][right]
        return order(left, k) + order(k, right) + [k - 1]

    return best[0][n - 1], order(0, n - 1)


# ============================================================
# FOLLOW-UP 2: The same answer with recursion and memory
# (top-down)
# ============================================================

# ---- follow-up 2: function ----
# Answer: coins(left, right) is the best result for the
```

```
# balloons strictly between the borders; try every balloon
# k as the LAST one burst there.
# Time: O(n^3).  Space: O(n^2).
def burst_balloons_memo(nums):
    values = [1] + list(nums) + [1]
    memo = {}

    def coins(left, right):
        if right - left < 2:
            return 0
        if (left, right) in memo:
```

```
            return memo[(left, right)]
        best = 0
        for k in range(left + 1, right):
            total = values[left] * values[k] * values[right] + coins(left, k) + coins(k, right)
            if total > best:
                best = total
        memo[(left, right)] = best
        return best

    return coins(0, len(values) - 1)
```

PATTERN 14 · RECURSION / DIVIDE AND CONQUER · PROBLEM 20 OF 20

327. Count of Range Sum

Hard · Binary Indexed Tree, Segment Tree, Array, Binary Search, Divide and Conquer, Ordered Set, Treap, Merge Sort

leetcode.com/problems/count-of-range-sum

★ **TOP 7 PROBLEM** in Binary Indexed Tree (Fenwick Tree): master this one first

Also appears in pattern 40 Segment Tree, 41 Binary Indexed Tree (Fenwick Tree)

PROBLEM

Given an integer array `nums` and two integers `lower` and `upper`, return *the number of range sums that lie in* `[lower, upper]` *inclusive*.

Range sum `S(i, j)` is defined as the sum of the elements in `nums` between indices `i` and `j` inclusive, where `i <= j`.

Example 1:

```
Input: nums = [-2,5,-1], lower = -2, upper = 2
Output: 3
Explanation: The three ranges are: [0,0], [2,2], and [0,2] and their respective sums are: -2, -1, 2.
```

Example 2:

```
Input: nums = [0], lower = 0, upper = 0
Output: 1
```

Constraints:

- `1 <= nums.length <= 10^5`
- `-2^31 <= nums[i] <= 2^31 - 1`
- `-10^5 <= lower <= upper <= 10^5`
- The answer is **guaranteed** to fit in a **32-bit** integer.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Count subarrays whose sum lies in [lower,upper]. A sum is a prefix difference; a double loop is $O(n^2)$. For prefix x we need how many earlier prefixes fall in [x-upper,x-lower]. Discretize all x, x-lower, and x-upper. A Fenwick tree stores frequencies of seen prefixes: query the interval, then insert x, so only earlier prefixes count.

Python 3

```python
class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x, v):
        while x <= self.n:
            self.c[x] += v
            x += x & -x

    def query(self, x):
        s = 0
        while x > 0:
            s += self.c[x]
            x -= x & -x
        return s


class Solution:
    def countRangeSum(self, nums: List[int], lower: int, upper: int) -> int:
        s = list(accumulate(nums, initial=0))
        arr = sorted(set(v for x in s for v in (x, x - lower, x - upper)))
        tree = BinaryIndexedTree(len(arr))
        ans = 0
        for x in s:
            l = bisect_left(arr, x - upper) + 1
            r = bisect_left(arr, x - lower) + 1
            ans += tree.query(r) - tree.query(l - 1)
            tree.update(bisect_left(arr, x) + 1, 1)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. A Fenwick tree instead of merge sort
# 2. All numbers are positive: two pointers are enough


# ============================================================
# FOLLOW-UP 1: A Fenwick tree instead of merge sort
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count pairs of prefix sums with lo <= later -
# earlier <= hi: compress all prefix sums, and for each
# prefix ask the Fenwick tree how many earlier prefix sums
# lie in [prefix - hi, prefix - lo].
# Time: O(n log n).  Space: O(n).
import bisect


def count_range_sum_fenwick(nums, lower, upper):
    prefix = [0]
    for x in nums:
        prefix.append(prefix[-1] + x)
    sorted_values = sorted(set(prefix))
    size = len(sorted_values)
    tree = [0] * (size + 2)

    def add(i):
        while i <= size:
            tree[i] += 1
            i += i & -i

    def count_up_to(i):
```

```
        total = 0
        while i > 0:
            total += tree[i]
            i -= i & -i
        return total

    total = 0
    for value in prefix:
        low_rank = bisect.bisect_left(sorted_values, value - upper)
        high_rank = bisect.bisect_right(sorted_values, value - lower)
        total += count_up_to(high_rank) - count_up_to(low_rank)
        add(bisect.bisect_left(sorted_values, value) + 1)
    return total


# ============================================================
# FOLLOW-UP 2: All numbers are positive: two pointers are
# enough
# ============================================================

# ---- follow-up 2: function ----
# Answer: Prefix sums are increasing, so for each end the
# valid starts form a contiguous range found by two
# pointers: O(n).
# Time: O(n).  Space: O(n).
def count_range_sum_positive(nums, lower, upper):
    prefix = [0]
    for x in nums:
```

```
        prefix.append(prefix[-1] + x)
    count = 0
    low = 0
    high = 0
    for end in range(1, len(prefix)):
        while prefix[end] - prefix[low] > upper:
            low += 1
        if high < low:
            high = low
        while high < end and prefix[end] - prefix[high] >= lower:
            high += 1
        count += high - low
    return count
```

PATTERN 15 OF 41

Backtracking / Subsets / Combinations / Permutations

20 problems · 0 Easy · 18 Medium · 2 Hard

THE IDEA

Build the answer one choice at a time: choose, recurse, undo. Prune a branch as soon as it cannot lead to a valid answer.

Signals: all subsets, combinations that sum to a target, permutations, N-Queens, word search, generate parentheses, sudoku.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```python
def backtrack(start, path):
    if is_complete(path):
        result.append(path[:])          # copy before storing
        return
    for i in range(start, len(options)):
        path.append(options[i])         # choose
        backtrack(i + 1, path)          # explore
        path.pop()                      # undo
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	★ **Subsets**	Medium	LC 78	780
2	Letter Case Permutation	Medium	LC 784	783
3	Generate Parentheses (see page)	Medium	LC 22	404
4	Combinations	Medium	LC 77	786
5	★ **Combination Sum**	Medium	LC 39	789
6	Subsets II	Medium	LC 90	793
7	★ **Permutations**	Medium	LC 46	796
8	Permutations II	Medium	LC 47	799
9	★ **Combination Sum II**	Medium	LC 40	802
10	Combination Sum III	Medium	LC 216	805
11	Letter Combinations of a Phone Number	Medium	LC 17	809
12	Beautiful Arrangement	Medium	LC 526	812
13	★ **Word Search**	Medium	LC 79	815
14	★ **Palindrome Partitioning**	Medium	LC 131	819
15	Restore IP Addresses	Medium	LC 93	822
16	Gray Code	Medium	LC 89	825
17	Factor Combinations	Medium	LC 254	827
18	Partition to K Equal Sum Subsets	Medium	LC 698	829
19	★ **N-Queens**	Hard	LC 51	834
20	Sudoku Solver	Hard	LC 37	837

PATTERN 15 · BACKTRACKING / SUBSETS / COMBINATIONS / PERMUTATIONS · PROBLEM 1 OF 20

78. Subsets

Medium · Bit Manipulation, Array, Backtracking

leetcode.com/problems/subsets

★ **TOP 7 PROBLEM** in Backtracking / Subsets / Combinations / Permutations, Bit Manipulation: master this one first

Also appears in pattern 27 Bit Manipulation

PROBLEM

Given an integer array `nums` of **unique** elements, return *all possible subsets (the power set)*.

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

Example 1:

```
Input: nums = [1,2,3]
Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]
```

Example 2:

```
Input: nums = [0]
Output: [[],[0]]
```

Constraints:

- `1 <= nums.length <= 10`
- `-10 <= nums[i] <= 10`
- All the numbers of `nums` are **unique**.

SOLUTIONS (PYTHON 3)

Solution 1: DFS (Backtracking)

How to think about it

A subset is a choose/skip for each element, 2^n of them. n ≤ 10, so listing all is fine. We could rebuild from each mask, or recurse. Recursion is the direct picture: from index i, skip nums[i] then take it and pop. At i=n copy the path into the answer. Distinct elements mean no duplicate subsets.

We design a function dfs(i), which represents starting the search from the ith element of the array for all subsets. The execution logic of the function dfs(i) is as follows:

- If i = n, it means the current search has ended. Add the current subset t to the answer array ans, and then return.
- Otherwise, we can choose not to select the current element and directly execute dfs(i + 1); or we can choose the current element, i.e., add the current element nums[i] to the subset t, and then execute dfs(i + 1). Note that we need to remove nums[i] from the subset t after executing dfs(i + 1) (backtracking).

In the main function, we call dfs(0), i.e., start searching all subsets from the first element of the array. Finally, return the answer array ans.

The time complexity is $O(n \times 2^n)$, and the space complexity is O(n). Here, n is the length of the array. There are a total of 2^n subsets, and each subset takes O(n) time to construct.

Python 3

```python
class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            if i == len(nums):
                ans.append(t[:])
                return
            dfs(i + 1)
            t.append(nums[i])
            dfs(i + 1)
            t.pop()

        ans = []
        t = []
        dfs(0)
        return ans
```

Solution 2: Binary Enumeration

> **How to think about it**
>
> Method 1 expands choose/skip on the call stack. n is tiny, so each subset can be a mask in $[0,2^n)$: bit i includes nums[i]. No recursion; the same $O(n \times 2^n)$ work.

We can also use the method of binary enumeration to get all subsets.

We can use 2^n binary numbers to represent all subsets of n elements. For the current binary number mask, if the ith bit is 1, it means that the ith element is selected, otherwise it means that the ith element is not selected.

The time complexity is $O(n \times 2^n)$, and the space complexity is $O(n)$. Here, n is the length of the array. There are a total of 2^n subsets, and each subset takes $O(n)$ time to construct.

Python 3

```python
class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        ans = []
        for mask in range(1 << len(nums)):
            t = [x for i, x in enumerate(nums) if mask >> i & 1]
            ans.append(t)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Generate all subsets without recursion (bitmasks)
# 2. Only the subsets of size k (combinations)


# ============================================================
# FOLLOW-UP 1: Generate all subsets without recursion
# (bitmasks)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Every number from 0 to 2^n - 1 is a subset: bit
# i says whether element i is in it.
# Time: O(n * 2^n).  Space: O(1) extra (the output is O(n
# * 2^n)).
def subsets_bitmask(nums):
    n = len(nums)
    result = []
    for mask in range(1 << n):
        subset = []
        for i in range(n):
```

```
            if (mask >> i) & 1:
                subset.append(nums[i])
        result.append(subset)
    return result


# ============================================================
# FOLLOW-UP 2: Only the subsets of size k (combinations)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Backtracking that stops as soon as the subset
# has k elements and prunes when there are not enough
# numbers left.
# Time: O(C(n, k) * k).  Space: O(k) plus the output.
def subsets_of_size(nums, k):
    result = []

    def go(start, current):
        if len(current) == k:
            result.append(list(current))
            return
        for i in range(start, len(nums)):
            if len(nums) - i < k - len(current):
                break
            current.append(nums[i])
            go(i + 1, current)
            current.pop()

    go(0, [])
    return result
```

784. Letter Case Permutation

Medium · Bit Manipulation, String, Backtracking

leetcode.com/problems/letter-case-permutation

PROBLEM

Given a string `s`, you can transform every letter individually to be lowercase or uppercase to create another string.

Return *a list of all possible strings we could create*. Return the output in **any order**.

Example 1:

```
Input: s = "a1b2"
Output: ["a1b2","a1B2","A1b2","A1B2"]
```

Example 2:

```
Input: s = "3z4"
Output: ["3z4","3Z4"]
```

Constraints:

- `1 <= s.length <= 12`
- `s` consists of lowercase English letters, uppercase English letters, and digits.

SOLUTIONS (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> Each letter may flip case. The string is short, so DFS enumerates. At a letter, first continue unchanged, then xor 32 and continue; digits have one branch. Collect at the end.

Since each letter in s can be converted to uppercase or lowercase, we can use the DFS (Depth-First Search) method to enumerate all possible cases.

Specifically, traverse the string s from left to right. For each letter encountered, you can choose to convert it to uppercase or lowercase, and then continue to traverse the subsequent letters. When you reach the end of the string, you get a conversion scheme and add it to the answer.

The method of converting case can be implemented using bitwise operations. For a letter, the difference between the ASCII codes of its lowercase and uppercase forms is 32, so we can achieve case conversion by XORing the ASCII code of the letter with 32.

The time complexity is $O(n \times 2^n)$, where n is the length of the string s. For each letter, we can choose to convert it to uppercase or lowercase, so there are 2^n conversion schemes in total. For each conversion scheme, we need $O(n)$ time to generate a new string.

Python 3

```python
class Solution:
    def letterCasePermutation(self, s: str) -> List[str]:
        def dfs(i: int) -> None:
            if i >= len(t):
                ans.append("".join(t))
                return
            dfs(i + 1)
            if t[i].isalpha():
                t[i] = chr(ord(t[i]) ^ 32)
                dfs(i + 1)

        t = list(s)
        ans = []
        dfs(0)
        return ans
```

Solution 2: Binary Enumeration

> **How to think about it**
>
> There are exactly 2^n strings if n letters appear. A bitmask can pick lower/upper for the j-th letter. For each mask, rewrite letters by that bit and copy digits.

For a letter, we can convert it to uppercase or lowercase. Therefore, for each letter, we can use a binary bit to represent its conversion scheme, where 1 represents lowercase and 0 represents uppercase.

First, we count the number of letters in the string s, denoted as n. Then, there are 2^n conversion schemes in total. We can use each bit of a binary number to represent the conversion scheme of each letter, enumerating from 0 to 2^n-1.

Specifically, we can use a variable i to represent the current binary number being enumerated, where the j-th bit of i represents the conversion scheme of the j-th letter. That is, the j-th bit of i being 1 means the j-th letter is converted to lowercase, and 0 means the j-th letter is converted to uppercase.

The time complexity is $O(n \times 2^n)$, where n is the length of the string s. For each letter, we can choose to convert it to uppercase or lowercase, so there are 2^n conversion schemes in total. For each conversion scheme, we need $O(n)$ time to generate a new string.

Python 3

```python
class Solution:
    def letterCasePermutation(self, s: str) -> List[str]:
        ans = []
        n = sum(c.isalpha() for c in s)
        for i in range(1 << n):
            j, t = 0, []
            for c in s:
                if c.isalpha():
                    c = c.lower() if (i >> j) & 1 else c.upper()
                    j += 1
                t.append(c)
            ans.append(''.join(t))
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Only count the permutations
# 2. Without recursion: grow the list one character at a
# time


# ============================================================
# FOLLOW-UP 1: Only count the permutations
# ============================================================

# ---- follow-up 1: function ----
# Answer: Every letter doubles the number of results: 2 to
# the power of the number of letters.
# Time: O(n).  Space: O(1).
def count_case_permutations(s):
    letters = 0
    for ch in s:
        if ch.isalpha():
            letters += 1
    return 2 ** letters


# ============================================================
# FOLLOW-UP 2: Without recursion: grow the list one
# character at a time
# ============================================================

# ---- follow-up 2: function ----
# Answer: Start with [''] and for every character extend
# every string with it (both cases for a letter).
# Time: O(2^L * n), L = number of letters.  Space: O(2^L *
# n).
def case_permutations_iterative(s):
    results = [""]
    for ch in s:
        if ch.isalpha():
            grown = []
            for text in results:
                grown.append(text + ch.lower())
                grown.append(text + ch.upper())
            results = grown
        else:
            results = [text + ch for text in results]
    return results
```

77. Combinations

Medium · Backtracking

leetcode.com/problems/combinations

PROBLEM

Given two integers `n` and `k`, return *all possible combinations of* `k` *numbers chosen from the range* `[1, n]`.

You may return the answer in **any order**.

Example 1:

```
Input: n = 4, k = 2
Output: [[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]]
Explanation: There are 4 choose 2 = 6 total combinations.
Note that combinations are unordered, i.e., [1,2] and [2,1] are considered to be the same combination.
```

Example 2:

```
Input: n = 1, k = 1
Output: [[1]]
Explanation: There is 1 choose 1 = 1 total combination.
```

Constraints:

- `1 <= n <= 20`
- `1 <= k <= n`

SOLUTIONS (PYTHON 3)

Solution 1: Backtracking (Two Ways)

How to think about it

The first idea is all 2^n subsets of {1,…,n}, keep those of length k. n ≤ 20 might pass, but most subsets have the wrong size. Order does not matter; each number is in or out, and we can stop a path once its length is k. So dfs(i) handles i: push, recurse i+1, pop, then skip. Enumerating the next j among leftovers is the other tree; both yield the same combinations. The code below is the first form.

We design a function dfs(i), which represents starting the search from number i, with the current search path as t, and the answer as ans.

The execution logic of the function dfs(i) is as follows:

- If the length of the current search path t equals k, then add the current search path to the answer and return.
- If i > n, it means the search has ended, return.
- Otherwise, we can choose to add the number i to the search path t, and then continue the search, i.e., execute dfs(i + 1), and then remove the number i from the search path t; or we do not add the number i to the search path t, and directly execute dfs(i + 1).

The above method is actually enumerating whether to select the current number or not, and then recursively searching the next number. We can also enumerate the next number j to be selected, where i ≤ j ≤ n. If the next number to be selected is j, then we add the number j to the search path t, and then continue the search, i.e., execute dfs(j + 1), and then remove the number j from the search path t.

In the main function, we start the search from number 1, i.e., execute dfs(1).

The time complexity is $O(C_n^k \times k)$, and the space complexity is O(k). Here, C_n^k represents the combination number.

Similar problems:

- 39. Combination Sum

- 40. Combination Sum II
- 216. Combination Sum III

Python 3

```python
class Solution:
    def combine(self, n: int, k: int) -> List[List[int]]:
        def dfs(i: int):
            if len(t) == k:
                ans.append(t[:])
                return
            if i > n:
                return
            t.append(i)
            dfs(i + 1)
            t.pop()
            dfs(i + 1)

        ans = []
        t = []
        dfs(1)
        return ans
```

Solution 2

How to think about it

Method 1 walks a choose/skip tree of depth n; skip branches still scan the rest of the numbers. Method 2 enumerates the next chosen j ∈ [i,n], so recursion depth equals how many we have picked, and the tree matches combinations of length k more closely. Same answers; the loop just expands the remaining candidates.

Python 3

```python
class Solution:
    def combine(self, n: int, k: int) -> List[List[int]]:
        def dfs(i: int):
            if len(t) == k:
                ans.append(t[:])
                return
            if i > n:
                return
            for j in range(i, n + 1):
                t.append(j)
                dfs(j + 1)
                t.pop()

        ans = []
        t = []
        dfs(1)
        return ans
```

Complexity. Time: O(C(n, k) * k). Space: O(k) (plus the answers).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Numbers may be chosen MORE than once (combinations
# with repetition)
# 2. The next combination in lexicographic order


# ============================================================
# FOLLOW-UP 1: Numbers may be chosen MORE than once
# (combinations with repetition)
# ============================================================
```

```
# ---- follow-up 1: function ----
# Answer: Same backtracking, but the next call may start
# at the same number again.
# Time: O(C(n + k - 1, k) * k).  Space: O(k) plus the
# output.
def combinations_with_repetition(n, k):
    result = []

    def go(start, current):
        if len(current) == k:
            result.append(list(current))
            return
        for x in range(start, n + 1):
            current.append(x)
            go(x, current)
            current.pop()

    go(1, [])
    return result


# ============================================================
# FOLLOW-UP 2: The next combination in lexicographic order
# ============================================================

# ---- follow-up 2: function ----
# Answer: Find the rightmost position that can still be
# increased (its value is below the largest it may have),
# increase it and reset everything after it to the
# smallest following values.
# Time: O(k).  Space: O(k).
def next_combination(combination, n):
    k = len(combination)
    items = list(combination)
    i = k - 1
    while i >= 0 and items[i] == n - k + i + 1:
        i -= 1
    if i < 0:
        return None                                  # this was the last combination
    items[i] += 1
    for j in range(i + 1, k):
        items[j] = items[j - 1] + 1
    return items
```

39. Combination Sum

Medium · Array, Backtracking

leetcode.com/problems/combination-sum

★ **TOP 7 PROBLEM** in Backtracking / Subsets / Combinations / Permutations: master this one first

PROBLEM

Given an array of **distinct** integers `candidates` and a target integer `target`, return *a list of all **unique combinations** of* `candidates` *where the chosen numbers sum to* `target`. You may return the combinations in **any order**.

The **same** number may be chosen from `candidates` an **unlimited number of times**. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to `target` is less than `150` combinations for the given input.

Example 1:

```
Input: candidates = [2,3,6,7], target = 7
Output: [[2,2,3],[7]]
Explanation:
2 and 3 are candidates, and 2 + 2 + 3 = 7. Note that 2 can be used multiple times.
7 is a candidate, and 7 = 7.
These are the only two combinations.
```

Example 2:

```
Input: candidates = [2,3,5], target = 8
Output: [[2,2,2,2],[2,3,3],[3,5]]
```

Example 3:

```
Input: candidates = [2], target = 1
Output: []
```

Constraints:

- `1 <= candidates.length <= 30`
- `2 <= candidates[i] <= 40`
- All elements of `candidates` are **distinct**.
- `1 <= target <= 40`

SOLUTIONS (PYTHON 3)

Solution 1: Sorting + Pruning + Backtracking

How to think about it

The first idea is to use each number any number of times and collect every sequence that sums to target. With n ≤ 30 and target ≤ 40, an unpruned search wastes paths that already exceed the target, and different orders of the same multiset would be counted twice. A combination ignores order, so each multiset should appear once. Sort first, and only take candidates from the current index rightward. After sorting, if the remainder s is already smaller than candidates[i], everything after is larger and the branch dies. dfs(i,s) enumerates j from i, and the recursive call stays at j so the same value may be reused.

We can first sort the array to facilitate pruning.

Next, we design a function dfs(i, s), which means starting the search from index i with a remaining target value of s. Here, i and s are both non-negative integers, the current search path is t, and the answer is ans.

In the function dfs(i, s), we first check whether s is 0. If it is, we add the current search path t to the answer ans, and then return. If s < candidates[i], it means that the elements of the current index and the following indices are all greater than the remaining target value s, and the path is invalid, so we return directly. Otherwise, we start the search from index i, and the search index range is $j \in [i, n)$, where n is the length of the array candidates. During the search, we add the element of the current index to the search path t, recursively call the function dfs(j, s - candidates[j]), and after the recursion ends, we remove the element of the current index from the search path t.

In the main function, we just need to call the function dfs(0, target) to get the answer.

The time complexity is $O(2^n \times n)$, and the space complexity is O(n). Here, n is the length of the array candidates. Due to pruning, the actual time complexity is much less than $O(2^n \times n)$.

Similar problems:

- 40. Combination Sum II
- 77. Combinations
- 216. Combination Sum III

Python 3

```python
class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]:
        def dfs(i: int, s: int):
            if s == 0:
                ans.append(t[:])
                return
            if s < candidates[i]:
                return
            for j in range(i, len(candidates)):
                t.append(candidates[j])
                dfs(j, s - candidates[j])
                t.pop()

        candidates.sort()
        t = []
        ans = []
        dfs(0, target)
        return ans
```

Solution 2: Sorting + Pruning + Backtracking(Another Form)

> **How to think about it**
>
> Method 1 already prunes correctly by looping over “who is next”. The other writing is choose-or-skip at the current index: skip goes to i+1; take stays at i so the same value may be reused. The search tree looks different; the answers do not. It is the same algorithm unfolded as a binary tree.

We can also change the implementation logic of the function dfs(i, s) to another form. In the function dfs(i, s), we first check whether s is 0. If it is, we add the current search path t to the answer ans, and then return. If $i \geq n$ or s < candidates[i], the path is invalid, so we return directly. Otherwise, we consider two situations, one is not selecting the element of the current index, that is, recursively calling the function dfs(i + 1, s), and the other is selecting the element of the current index, that is, recursively calling the function dfs(i, s - candidates[i]).

The time complexity is $O(2^n \times n)$, and the space complexity is O(n). Here, n is the length of the array candidates. Due to pruning, the actual time complexity is much less than $O(2^n \times n)$.

Python 3

```
class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]:
        def dfs(i: int, s: int):
            if s == 0:
                ans.append(t[:])
                return
            if i >= len(candidates) or s < candidates[i]:
                return
            dfs(i + 1, s)
            t.append(candidates[i])
            dfs(i, s - candidates[i])
            t.pop()

        candidates.sort()
        t = []
        ans = []
        dfs(0, target)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Each number may be used at most k times
# 2. Only COUNT the combinations


# ============================================================
# FOLLOW-UP 1: Each number may be used at most k times
# ============================================================

# ---- follow-up 1: function ----
# Answer: Backtracking that tracks how many times the
# current number was used and moves on to the next number
# after k uses.
# Time: exponential in target / smallest candidate.
# Space: O(target / smallest candidate) recursion plus the
# output.
def combination_sum_limited(candidates, target, k):
    result = []

    def go(index, remaining, current):
        if remaining == 0:
            result.append(list(current))
            return
        if index == len(candidates):
            return
        number = candidates[index]
        uses = 0
        while uses <= k and uses * number <= remaining:
            go(index + 1, remaining - uses * number, current + [number] * uses)
            uses += 1

    go(0, target, [])
    return result


# ============================================================
# FOLLOW-UP 2: Only COUNT the combinations
# ============================================================

# ---- follow-up 2: function ----
# Answer: A table over the target: for each candidate,
# ways[t] += ways[t - candidate] (candidates in the outer
# loop so the order does not matter).
# Time: O(n * target).  Space: O(target).
```

```
def count_combination_sum(candidates, target):
    ways = [0] * (target + 1)
    ways[0] = 1
    for number in candidates:
        for t in range(number, target + 1):
            ways[t] += ways[t - number]
    return ways[target]
```

90. Subsets II

Medium · Bit Manipulation, Array, Backtracking

leetcode.com/problems/subsets-ii

PROBLEM

Given an integer array `nums` that may contain duplicates, return *all possible subsets (the power set)*.

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

Example 1:

```
Input: nums = [1,2,2]
Output: [[],[1],[1,2],[1,2,2],[2],[2,2]]
```

Example 2:

```
Input: nums = [0]
Output: [[],[0]]
```

Constraints:

- `1 <= nums.length <= 10`
- `-10 <= nums[i] <= 10`

SOLUTIONS (PYTHON 3)

Solution 1: Sorting + DFS

How to think about it

The first idea is the same DFS as Subsets: take or skip each index. Correct, but duplicates make [1, 2] appear twice. n ≤ 10 is small, yet the solution set must be unique. The bottleneck is treating equal values at different indices as distinct choices. Sort so equals sit together; skip the same value on the “not take” branch so each multiset is generated once. After sorting, DFS: take i, then on the skip path jump over every later equal nums[i]. How many copies we take is decided by recursion depth, never as two identical subsets side by side.

We can first sort the array nums to facilitate deduplication.

Then, we design a function dfs(i), which represents the current search for subsets starting from the i-th element. The execution logic of the function dfs(i) is as follows:

If i ≥ n, it means all elements have been searched, add the current subset to the answer array, and end the recursion.

If i < n, add the i-th element to the subset, execute dfs(i + 1), then remove the i-th element from the subset. Next, we check if the i-th element is the same as the next element. If they are the same, skip the element in a loop until we find the first element different from the i-th element, then execute dfs(i + 1).

Finally, we only need to call dfs(0) and return the answer array.

The time complexity is $O(n \times 2^n)$, and the space complexity is $O(n)$. Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            if i == len(nums):
                ans.append(t[:])
                return
            t.append(nums[i])
            dfs(i + 1)
            x = t.pop()
            while i + 1 < len(nums) and nums[i + 1] == x:
                i += 1
            dfs(i + 1)

        nums.sort()
        ans = []
        t = []
        dfs(0)
        return ans
```

Solution 2: Sorting + Binary Enumeration

> **How to think about it**
>
> Solution 1 already deduplicates, but recursion pays a stack and the skip logic is easy to get wrong. n ≤ 10, so we can enumerate all 2^n masks. What it lacks is “take or skip” as bits. Bit i of mask means taking nums[i]; taking i without i - 1 when the values are equal skips an earlier copy — a duplicate, drop it. Same asymptotics, one loop.

Similar to Solution 1, we first sort the array nums to facilitate deduplication.

Next, we enumerate a binary number mask in the range $[0, 2^n)$, where the binary representation of mask is an n-bit bit string. If the i-th bit of mask is 1, it means selecting nums[i], and 0 means not selecting nums[i]. Note that if the (i - 1)-th bit of mask is 0 and nums[i] = nums[i - 1], it means that the i-th element is the same as the (i - 1)-th element in the current enumeration scheme. To avoid duplication, we skip this case. Otherwise, we add the subset corresponding to mask to the answer array.

After the enumeration, we return the answer array.

The time complexity is $O(n \times 2^n)$, and the space complexity is $O(n)$. Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        n = len(nums)
        ans = []
        for mask in range(1 << n):
            ok = True
            t = []
            for i in range(n):
                if mask >> i & 1:
                    if i and (mask >> (i - 1) & 1) == 0 and nums[i] == nums[i - 1]:
                        ok = False
                        break
                    t.append(nums[i])
            if ok:
                ans.append(t)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Only COUNT the distinct subsets
# 2. The distinct subsets of exactly size k


# ============================================================
# FOLLOW-UP 1: Only COUNT the distinct subsets
# ============================================================

# ---- follow-up 1: function ----
# Answer: Group equal numbers: with a copies of a value
# you can take 0..a of them, so the count is the product
# of (copies + 1) over the distinct values.
# Time: O(n).  Space: O(n).
def count_distinct_subsets(nums):
    count = {}
    for x in nums:
        count[x] = count.get(x, 0) + 1
    total = 1
    for copies in count.values():
        total *= copies + 1
    return total


# ============================================================
# FOLLOW-UP 2: The distinct subsets of exactly size k
# ============================================================

# ---- follow-up 2: function ----
# Answer: Sort the numbers and skip a number that equals
# the previous one at the same depth.
# Time: O(C(n, k) * k).  Space: O(k) plus the output.
def distinct_subsets_of_size(nums, k):
    nums = sorted(nums)
    result = []

    def go(start, current):
        if len(current) == k:
            result.append(list(current))
            return
        for i in range(start, len(nums)):
            if i > start and nums[i] == nums[i - 1]:
                continue
            current.append(nums[i])
            go(i + 1, current)
            current.pop()

    go(0, [])
    return result
```

PATTERN 15 · BACKTRACKING / SUBSETS / COMBINATIONS / PERMUTATIONS · PROBLEM 7 OF 20

46. Permutations

Medium · Array, Backtracking

leetcode.com/problems/permutations

★ **TOP 7 PROBLEM** in Backtracking / Subsets / Combinations / Permutations: master this one first

PROBLEM

Given an array `nums` of distinct integers, return all the possible permutations. You can return the answer in **any order**.

Example 1:

```
Input: nums = [1,2,3]
Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
```

Example 2:

```
Input: nums = [0,1]
Output: [[0,1],[1,0]]
```

Example 3:

```
Input: nums = [1]
Output: [[1]]
```

Constraints:

- `1 <= nums.length <= 6`
- `-10 <= nums[i] <= 10`
- All the integers of `nums` are **unique**.

SOLUTION (PYTHON 3)

Solution 1: DFS (Backtracking)

How to think about it

The first idea is a language builtin or nested loops. n ≤ 6 so n! fits, but n is not fixed — we cannot hard-code loop depth. Generating every permutation means “pick an unused number for this position”. Reusing a number breaks the permutation, so we must mark. DFS fills index i: try each unused value, place it, recurse, then undo. n is tiny; O(n · n!) matches the output size.

We design a function dfs(i) to represent that the first i positions have been filled, and now we need to fill the i+1 position. We enumerate all possible numbers, if this number has not been filled, we fill in this number, and then continue to fill the next position, until all positions are filled.

The time complexity is O(n × n!), where n is the length of the array. There are n! permutations in total, and each permutation takes O(n) time to construct.

Similar problems:

- 47. Permutations II

Python 3

```python
class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            if i >= n:
                ans.append(t[:])
                return
            for j, x in enumerate(nums):
                if not vis[j]:
                    vis[j] = True
                    t[i] = x
                    dfs(i + 1)
                    vis[j] = False

        n = len(nums)
        vis = [False] * n
        t = [0] * n
        ans = []
        dfs(0)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The k-th permutation in lexicographic order (without
# generating the others)
# 2. The NEXT permutation in lexicographic order


# ============================================================
# FOLLOW-UP 1: The k-th permutation in lexicographic order
# (without generating the others)
# ============================================================

# ---- follow-up 1: function ----
# Answer: The first number is decided by dividing k - 1 by
# (n - 1)!, the remainder decides the rest with the
# remaining numbers.
# Time: O(n^2).  Space: O(n).
def kth_permutation(n, k):
    numbers = list(range(1, n + 1))
    factorial = [1] * (n + 1)
    for i in range(1, n + 1):
        factorial[i] = factorial[i - 1] * i
    k -= 1
    result = []
    for position in range(n, 0, -1):
        index = k // factorial[position - 1]
        k %= factorial[position - 1]
        result.append(numbers.pop(index))
    return result


# ============================================================
# FOLLOW-UP 2: The NEXT permutation in lexicographic order
# ============================================================

# ---- follow-up 2: function ----
# Answer: Find the last position whose number is smaller
# than its successor, swap it with the smallest bigger
# number to its right, and reverse the tail.
# Time: O(n).  Space: O(n).
def next_permutation(nums):
    items = list(nums)
    i = len(items) - 2
    while i >= 0 and items[i] >= items[i + 1]:
```

```
        i -= 1
    if i >= 0:
        j = len(items) - 1
        while items[j] <= items[i]:
            j -= 1
        items[i], items[j] = items[j], items[i]
    items[i + 1:] = reversed(items[i + 1:])
    return items
```

47. Permutations II

Medium · Array, Backtracking, Sorting

leetcode.com/problems/permutations-ii

PROBLEM

Given a collection of numbers, `nums`, that might contain duplicates, return *all possible unique permutations* ***in any order***.

Example 1:

```
Input: nums = [1,1,2]
Output:
[[1,1,2],
 [1,2,1],
 [2,1,1]]
```

Example 2:

```
Input: nums = [1,2,3]
Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
```

Constraints:

- `1 <= nums.length <= 8`
- `-10 <= nums[i] <= 10`

SOLUTION (PYTHON 3)

Solution 1: Sorting + Backtracking

> **How to think about it**
>
> The first idea is the same backtracking as unique permutations, then drop duplicates with a set. Correct, but n ≤ 8 and repeated numbers clone whole subtrees; filtering afterwards wastes time and space. The bottleneck is those duplicate branches. Swapping two equal values at the same depth yields the same permutation — each value may be chosen only once per layer. Sort so equals sit together, then skip a candidate when the previous equal is still unused. Each distinct value expands once at a position; every generated permutation is unique.

We can first sort the array so that duplicate numbers are placed together, making it easier to remove duplicates.

Then, we design a function dfs(i), which represents the current number to be placed at the i-th position. The specific implementation of the function is as follows:

- If i = n, it means we have filled all positions, add the current permutation to the answer array, and then return.
- Otherwise, we enumerate the number nums[j] for the i-th position, where the range of j is [0, n - 1]. We need to ensure that nums[j] has not been used and is different from the previously enumerated number to ensure that the current permutation is not duplicated. If the conditions are met, we can place nums[j] and continue to recursively fill the next position by calling dfs(i + 1). After the recursive call ends, we need to mark nums[j] as unused to facilitate subsequent enumeration.

In the main function, we first sort the array, then call dfs(0) to start filling from the 0th position, and finally return the answer array.

The time complexity is O(n × n!), and the space complexity is O(n). Here, n is the length of the array. We need to perform n! enumerations, and each enumeration requires O(n) time to check for duplicates. Additionally, we need a marker array to mark whether each position has been used, so the space complexity is O(n).

Similar problems:

- 46. Permutations

Python 3

```python
class Solution:
    def permuteUnique(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            if i == n:
                ans.append(t[:])
                return
            for j in range(n):
                if vis[j] or (j and nums[j] == nums[j - 1] and not vis[j - 1]):
                    continue
                t[i] = nums[j]
                vis[j] = True
                dfs(i + 1)
                vis[j] = False

        n = len(nums)
        nums.sort()
        ans = []
        t = [0] * n
        vis = [False] * n
        dfs(0)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Only count the distinct permutations
# 2. Generate the distinct permutations in order, one
# after another, without recursion


# ============================================================
# FOLLOW-UP 1: Only count the distinct permutations
# ============================================================

# ---- follow-up 1: function ----
# Answer: With n numbers where value v appears c(v) times,
# there are n! / (c(a)! * c(b)! * ...) different
# permutations.
# Time: O(n).  Space: O(n).
def count_distinct_permutations(nums):
    def factorial(m):
        result = 1
        for i in range(2, m + 1):
            result *= i
        return result

    count = {}
    for x in nums:
        count[x] = count.get(x, 0) + 1
    total = factorial(len(nums))
    for copies in count.values():
        total //= factorial(copies)
    return total


# ============================================================
# FOLLOW-UP 2: Generate the distinct permutations in
# order, one after another, without recursion
# ============================================================

# ---- follow-up 2: helper functions ----
def next_permutation(nums):
    items = list(nums)
    i = len(items) - 2
    while i >= 0 and items[i] >= items[i + 1]:
```

```
        i -= 1
    if i < 0:
        return None
    j = len(items) - 1
    while items[j] <= items[i]:
        j -= 1
    items[i], items[j] = items[j], items[i]
    items[i + 1:] = reversed(items[i + 1:])
    return items

# ---- follow-up 2: function ----
# Answer: Start from the sorted array and apply the next-
# permutation step until it is no longer possible.
# Time: O(n * n!).  Space: O(n * n!) for the output.
def all_distinct_permutations(nums):
    current = sorted(nums)
    result = []
    while current is not None:
        result.append(current)
```

```
        current = next_permutation(current)
    return result
```

40. Combination Sum II

Medium · Array, Backtracking

leetcode.com/problems/combination-sum-ii

★ **TOP 7 PROBLEM** in Backtracking / Subsets / Combinations / Permutations: master this one first

PROBLEM

Given a collection of candidate numbers (`candidates`) and a target number (`target`), find all unique combinations in `candidates` where the candidate numbers sum to `target`.

Each number in `candidates` may only be used **once** in the combination.

Note: The solution set must not contain duplicate combinations.

Example 1:

```
Input: candidates = [10,1,2,7,6,1,5], target = 8
Output:
[
[1,1,6],
[1,2,5],
[1,7],
[2,6]
]
```

Example 2:

```
Input: candidates = [2,5,2,1,2], target = 5
Output:
[
[1,2,2],
[5]
]
```

Constraints:

- `1 <= candidates.length <= 100`
- `1 <= candidates[i] <= 50`
- `1 <= target <= 30`

SOLUTIONS (PYTHON 3)

Solution 1: Sorting + Pruning + Backtracking

How to think about it

Unlike the previous problem, each number may be used only once, and candidates may contain duplicates. Without deduping, the same multiset is found via different index combinations. n ≤ 100 and target ≤ 30 make pruning even more important. Sort so equal values sit together. At the same depth, if j > i and candidates[j] equals the previous value, that value has already been tried as “this level’s choice” and we skip it. The recursive call is dfs(j+1,…): this index cannot be reused. If the remainder s is already smaller than the current value, the whole branch stops.

We can first sort the array to facilitate pruning and skipping duplicate numbers.

Next, we design a function dfs(i, s), which means starting the search from index i with a remaining target value of s. Here, i and s are both non-negative integers, the current search path is t, and the answer is ans.

In the function dfs(i, s), we first check whether s is 0. If it is, we add the current search path t to the answer ans, and then return. If i ≥ n or s < candidates[i], the path is invalid, so we return directly. Otherwise, we start the search from index i, and the search index range is j ∈ [i, n), where n is the length of the array candidates. During the search, if j > i and candidates[j]

= candidates[j - 1], it means that the current number is the same as the previous number, we can skip the current number because the previous number has been searched. Otherwise, we add the current number to the search path t, recursively call the function dfs(j + 1, s - candidates[j]), and after the recursion ends, we remove the current number from the search path t.

We can also change the implementation logic of the function dfs(i, s) to another form. If we choose the current number, we add the current number to the search path t, then recursively call the function dfs(i + 1, s - candidates[i]), and after the recursion ends, we remove the current number from the search path t. If we do not choose the current number, we can skip all numbers that are the same as the current number, then recursively call the function dfs(j, s), where j is the index of the first number that is different from the current number.

In the main function, we just need to call the function dfs(0, target) to get the answer.

The time complexity is $O(2^n \times n)$, and the space complexity is $O(n)$. Here, n is the length of the array candidates. Due to pruning, the actual time complexity is much less than $O(2^n \times n)$.

Similar problems:

- 39. Combination Sum
- 77. Combinations
- 216. Combination Sum III

Python 3

```python
class Solution:
    def combinationSum2(self, candidates: List[int], target: int) -> List[List[int]]:
        def dfs(i: int, s: int):
            if s == 0:
                ans.append(t[:])
                return
            if i >= len(candidates) or s < candidates[i]:
                return
            for j in range(i, len(candidates)):
                if j > i and candidates[j] == candidates[j - 1]:
                    continue
                t.append(candidates[j])
                dfs(j + 1, s - candidates[j])
                t.pop()

        candidates.sort()
        ans = []
        t = []
        dfs(0, target)
        return ans
```

Solution 2: Sorting + Pruning + Backtracking(Another Form)

> **How to think about it**
>
> Method 1 skips same-depth duplicates inside the loop. We can instead choose or skip: taking x advances the index by one; skipping jumps over the whole run of x, so the same combination cannot reappear via different indices. Complexity is unchanged; deduping just moves onto the “skip” branch.

We can also change the implementation logic of the function dfs(i, s) to another form. If we choose the current number, we add the current number to the search path t, then recursively call the function dfs(i + 1, s - candidates[i]), and after the recursion ends, we remove the current number from the search path t. If we do not choose the current number, we can skip all numbers that are the same as the current number, then recursively call the function dfs(j, s), where j is the index of the first number that is different from the current number.

The time complexity is $O(2^n \times n)$, and the space complexity is $O(n)$. Here, n is the length of the array candidates. Due to pruning, the actual time complexity is much less than $O(2^n \times n)$.

Python 3

```python
class Solution:
    def combinationSum2(self, candidates: List[int], target: int) -> List[List[int]]:
        def dfs(i: int, s: int):
            if s == 0:
                ans.append(t[:])
                return
            if i >= len(candidates) or s < candidates[i]:
                return
            x = candidates[i]
            t.append(x)
            dfs(i + 1, s - x)
            t.pop()
            while i < len(candidates) and candidates[i] == x:
                i += 1
            dfs(i, s)

        candidates.sort()
        ans = []
        t = []
        dfs(0, target)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Only COUNT the distinct combinations


# ========================================================
# FOLLOW-UP 1: Only COUNT the distinct combinations
# ========================================================

# ---- follow-up 1: function ----
# Answer: Group equal numbers and run a table over the
# target where each group can be used 0 to c times.
# Time: O(u * target * c), u distinct values, c copies.
# Space: O(target).
def count_combination_sum_unique(candidates, target):
    count = {}
    for x in candidates:
        count[x] = count.get(x, 0) + 1
    ways = [0] * (target + 1)
    ways[0] = 1
    for value in count:
        updated = [0] * (target + 1)
        for t in range(target + 1):
            for used in range(count[value] + 1):
                if used * value > t:
                    break
                updated[t] += ways[t - used * value]
        ways = updated
    return ways[target]
```

216. Combination Sum III

Medium · Array, Backtracking

leetcode.com/problems/combination-sum-iii

PROBLEM

Find all valid combinations of `k` numbers that sum up to `n` such that the following conditions are true:

- Only numbers `1` through `9` are used.
- Each number is used **at most once**.

Return *a list of all possible valid combinations*. The list must not contain the same combination twice, and the combinations may be returned in any order.

Example 1:

```
Input: k = 3, n = 7
Output: [[1,2,4]]
Explanation:
1 + 2 + 4 = 7
There are no other valid combinations.
```

Example 2:

```
Input: k = 3, n = 9
Output: [[1,2,6],[1,3,5],[2,3,4]]
Explanation:
1 + 2 + 6 = 9
1 + 3 + 5 = 9
2 + 3 + 4 = 9
There are no other valid combinations.
```

Example 3:

```
Input: k = 4, n = 1
Output: []
Explanation: There are no valid combinations.
Using 4 different numbers in the range [1,9], the smallest sum we can get is 1+2+3+4 = 10 and since 10
> 1, there are no valid combination.
```

Constraints:

- `2 <= k <= 9`
- `1 <= n <= 60`

SOLUTIONS (PYTHON 3)

Solution 1: Pruning + Backtracking (Two Approaches)

> **How to think about it**
>
> We choose k numbers from 1 through 9 that sum to n. The universe is tiny, so we can enumerate. Decide include-or-skip in increasing order, and prune when the remaining sum, the count, or the next integer is invalid. dfs(i,s) considers integer i with remaining sum s: take it via dfs(i+1,s-i), or skip via dfs(i+1,s).

We design a function dfs(i, s), which represents that we are currently enumerating the number i, and there are still numbers with a sum of s to be enumerated. The current search path is t, and the answer is ans.

The execution logic of the function dfs(i, s) is as follows:

Approach One:

- If s = 0 and the length of the current search path t is k, it means that a group of answers has been found. Add t to ans and then return.
- If i > 9 or i > s or the length of the current search path t is greater than k, it means that the current search path cannot be the answer, so return directly.
- Otherwise, we can choose to add the number i to the search path t, and then continue to search, i.e., execute dfs(i + 1, s - i). After the search is completed, remove i from the search path t; we can also choose not to add the number i to the search path t, and directly execute dfs(i + 1, s).

Python 3

```
class Solution:
    def combinationSum3(self, k: int, n: int) -> List[List[int]]:
        def dfs(i: int, s: int):
            if s == 0:
                if len(t) == k:
                    ans.append(t[:])
                return
            if i > 9 or i > s or len(t) >= k:
                return
            t.append(i)
            dfs(i + 1, s - i)
            t.pop()
            dfs(i + 1, s)

        ans = []
        t = []
        dfs(1, n)
        return ans
```

Another approach:

- If s = 0 and the length of the current search path t is k, it means that a group of answers has been found. Add t to ans and then return.
- If i > 9 or i > s or the length of the current search path t is greater than k, it means that the current search path cannot be the answer, so return directly.
- Otherwise, we enumerate the next number j, i.e., $j \in [i, 9]$, add the number j to the search path t, and then continue to search, i.e., execute dfs(j + 1, s - j). After the search is completed, remove j from the search path t.

In the main function, we call dfs(1, n), i.e., start enumerating from the number 1, and the remaining numbers with a sum of n need to be enumerated. After the search is completed, we can get all the answers.

The time complexity is $O(C_9^k \times k)$, and the space complexity is O(k).

Python 3

```
class Solution:
    def combinationSum3(self, k: int, n: int) -> List[List[int]]:
        def dfs(i: int, s: int):
            if s == 0:
                if len(t) == k:
                    ans.append(t[:])
                return
            if i > 9 or i > s or len(t) >= k:
                return
            for j in range(i, 10):
                t.append(j)
                dfs(j + 1, s - j)
                t.pop()

        ans = []
        t = []
        dfs(1, n)
        return ans
```

Solution 2: Binary Enumeration

> **How to think about it**
>
> Backtracking uses a call stack. There are only 2^9 subsets of {1,…,9}, so a 9-bit mask can list them all. If the mask has k bits set and those integers sum to n, we record the subset.

We can use a binary integer of length 9 to represent the selection of numbers 1 to 9, where the i-th bit of the binary integer represents whether the number i + 1 is selected. If the i-th bit is 1, it means that the number i + 1 is selected, otherwise, it means that the number i + 1 is not selected.

We enumerate binary integers in the range of $[0, 2^9)$. For the currently enumerated binary integer mask, if the number of 1s in the binary representation of mask is k, and the sum of the numbers corresponding to 1 in the binary representation of mask is n, it means that the number selection scheme corresponding to mask is a group of answers. We can add the number selection scheme corresponding to mask to the answer.

The time complexity is $O(2^9 \times 9)$, and the space complexity is O(k).

Similar problems:

- 39. Combination Sum
- 40. Combination Sum II
- 77. Combinations

Python 3

```python
class Solution:
    def combinationSum3(self, k: int, n: int) -> List[List[int]]:
        ans = []
        for mask in range(1 << 9):
            if mask.bit_count() == k:
                t = [i + 1 for i in range(9) if mask >> i & 1]
                if sum(t) == n:
                    ans.append(t)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Use the numbers 1 to m (not only 1 to 9)
# 2. Only count the combinations


# ============================================================
# FOLLOW-UP 1: Use the numbers 1 to m (not only 1 to 9)
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same backtracking with a different upper
# limit.
# Time: O(C(m, k) * k).  Space: O(k) plus the output.
def combination_sum_range(k, target, m):
    result = []

    def go(start, remaining, current):
        if len(current) == k:
            if remaining == 0:
                result.append(list(current))
            return
        for x in range(start, m + 1):
            if x > remaining:
                break
            current.append(x)
            go(x + 1, remaining - x, current)
            current.pop()
```

```
    go(1, target, [])
    return result


# ============================================================
# FOLLOW-UP 2: Only count the combinations
# ============================================================

# ---- follow-up 2: function ----
# Answer: A table over (how many numbers were chosen,
# sum): add the numbers 1 to 9 one at a time, each at most
# once.
# Time: O(9 * k * target).  Space: O(k * target).
def count_combinations_k(k, target):
    ways = []
    for chosen in range(k + 1):
        ways.append([0] * (target + 1))
    ways[0][0] = 1
    for number in range(1, 10):
        for chosen in range(k, 0, -1):
            for total in range(target, number - 1, -1):
                ways[chosen][total] += ways[chosen - 1][total - number]
    return ways[k][target]
```

17. Letter Combinations of a Phone Number

Medium · Hash Table, String, Backtracking

leetcode.com/problems/letter-combinations-of-a-phone-number

PROBLEM

Given a string containing digits from `2-9` inclusive, return all possible letter combinations that the number could represent. Return the answer in **any order**.

A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.

Example 1:

```
Input: digits = "23"
Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]
```

Example 2:

```
Input: digits = "2"
Output: ["a","b","c"]
```

Constraints:

- `1 <= digits.length <= 4`
- `digits[i]` is a digit in the range `['2', '9']`.

SOLUTIONS (PYTHON 3)

Solution 1: Traversal

> **How to think about it**
>
> n≤ 4, so at most 4^n combinations; any enumeration would pass. The problem is a Cartesian product of letter groups. Nested loops cannot be written when n is not fixed. Start from the empty string: each new digit replaces the current prefixes with every concatenation against that digit's letters. That is the product expanded layer by layer. So we keep a mapping array and grow from a list that holds only the empty string. An empty input returns an empty list.

First, we use an array or hash table to store the letters corresponding to each digit. Then we traverse each digit, combine its corresponding letters with the previous results to get the new results.

The time complexity is $O(4^n)$, and the space complexity is $O(4^n)$. Here, n is the length of the input digits.

Python 3

```python
class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        if not digits:
            return []
        d = ["abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"]
        ans = [""]
        for i in digits:
            s = d[int(i) - 2]
            ans = [a + b for a in ans for b in s]
        return ans
```

Solution 2: DFS

> **How to think about it**
>
> Solution 1 already enumerates every answer, but each layer allocates a new prefix list, so intermediate strings stay $O(4^n)$. If we walk a single path and backtrack, extra space drops to O(n) (depth and path length). So we DFS by index: pick one letter for the current digit, go deeper, then undo. The combinations are the same; only the generation order changes from “expand by layer” to “one path to the end”.

We can use the method of depth-first search to enumerate all possible letter combinations. Suppose that a part of the letter combination has been generated, but some digits have not been exhausted. At this time, we take out the letters corresponding to the next digit, and then enumerate each letter corresponding to this digit one by one, add them to the letter combination that has been generated before, to form all possible combinations.

The time complexity is $O(4^n)$, and the space complexity is O(n). Here, n is the length of the input digits.

Python 3

```python
class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        def dfs(i: int):
            if i >= len(digits):
                ans.append("".join(t))
                return
            for c in d[int(digits[i]) - 2]:
                t.append(c)
                dfs(i + 1)
                t.pop()

        if not digits:
            return []
        d = ["abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"]
        ans = []
        t = []
        dfs(0)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Without recursion (a growing list)
# 2. Only the combinations that are words of a dictionary


# ============================================================
# FOLLOW-UP 1: Without recursion (a growing list)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Start with [''] and extend every string by each
# letter of the next digit.
# Time: O(4^n * n).  Space: O(4^n * n).
```

```
def letter_combinations_iterative(digits):
    letters = {"2": "abc", "3": "def", "4": "ghi", "5": "jkl", "6": "mno", "7": "pqrs", "8": "tuv", "9": "
wxyz"}
    if digits == "":
        return []
    results = [""]
    for digit in digits:
        grown = []
        for text in results:
            for ch in letters[digit]:
                grown.append(text + ch)
        results = grown
    return results


# ============================================================
# FOLLOW-UP 2: Only the combinations that are words of a
# dictionary
# ============================================================

# ---- follow-up 2: function ----
# Answer: Prune: stop extending a string when it is not a
# prefix of any dictionary word.
# Time: O(4^n * n) in the worst case (pruned by the
# prefixes).  Space: O(total dictionary letters).
def words_for_digits(digits, dictionary):
    letters = {"2": "abc", "3": "def", "4": "ghi", "5": "jkl", "6": "mno", "7": "pqrs", "8": "tuv", "9": "
wxyz"}
    prefixes = set()
    for word in dictionary:
        for i in range(len(word) + 1):
            prefixes.add(word[:i])
    words = set(dictionary)
    result = []

    def go(index, current):
        if current not in prefixes:
            return
        if index == len(digits):
            if current in words:
                result.append(current)
            return
        for ch in letters[digits[index]]:
            go(index + 1, current + ch)

    if digits:
        go(0, "")
    return result
```

526. Beautiful Arrangement

Medium · Bit Manipulation, Array, Dynamic Programming, Backtracking, Bitmask

leetcode.com/problems/beautiful-arrangement

PROBLEM

Suppose you have `n` integers labeled `1` through `n`. A permutation of those `n` integers `perm` (**1-indexed**) is considered a **beautiful arrangement** if for every `i` (`1 <= i <= n`), **either** of the following is true:

- `perm[i]` is divisible by `i`.
- `i` is divisible by `perm[i]`.

Given an integer `n`, return *the **number** of the **beautiful arrangements** that you can construct*.

Example 1:

```
Input: n = 2
Output: 2
Explanation:
The first beautiful arrangement is [1,2]:
    - perm[1] = 1 is divisible by i = 1
    - perm[2] = 2 is divisible by i = 2
The second beautiful arrangement is [2,1]:
    - perm[1] = 2 is divisible by i = 1
    - i = 2 is divisible by perm[2] = 1
```

Example 2:

```
Input: n = 1
Output: 1
```

Constraints:

- `1 <= n <= 15`

SOLUTIONS (PYTHON 3)

Solution 1: Backtracking

> **How to think about it**
>
> Count permutations where value j at position i divides or is divided by i. Full 15! search is impossible, but few values fit each position. Precompute the legal values per position, then backtrack by position while marking used numbers. Reaching n+1 counts one arrangement. The divisibility lists keep the search inside the feasible set.

Assign unused numbers to each position when the divisibility condition holds.

Python 3

```python
class Solution:
    def countArrangement(self, n: int) -> int:
        def dfs(i):
            nonlocal ans, n
            if i == n + 1:
                ans += 1
                return
            for j in match[i]:
                if not vis[j]:
                    vis[j] = True
                    dfs(i + 1)
                    vis[j] = False

        ans = 0
        vis = [False] * (n + 1)
        match = defaultdict(list)
        for i in range(1, n + 1):
            for j in range(1, n + 1):
                if j % i == 0 or i % j == 0:
                    match[i].append(j)

        dfs(1)
        return ans
```

Complexity. Time: O(n!) in the worst case (pruned heavily in practice). Space: O(n^2) for the match table.

Solution 2: State Compression DP

> **How to think about it**
>
> Backtracking still expands a permutation tree and repeats the same unused-set at the same position. With n ≤ 15 the used set fits in 2^n bits. f[i] is the number of ways to reach used-set i. The pop-count is the next position; try each unused j that divides that position. f[0]=1 and the full mask is the answer. Each subset is filled once.

f[i] is the number of ways to form the chosen-number mask i.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```python
class Solution:
    def countArrangement(self, n: int) -> int:
        size = 1 << n
        f = [0] * size                          # f[mask]: ways to fill the first popcount(mask) positions
with the numbers in mask
        f[0] = 1
        for mask in range(size):
            position = bin(mask).count("1") + 1
            for j in range(1, n + 1):
                if not (mask >> (j - 1)) & 1 and (position % j == 0 or j % position == 0):
                    f[mask | (1 << (j - 1))] += f[mask]
        return f[size - 1]
```

Complexity. Time: O(2^n * n). Space: O(2^n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count with a bitmask table (faster than plain
# backtracking)
# 2. List all the beautiful arrangements


# ========================================================
# FOLLOW-UP 1: Count with a bitmask table (faster than
# plain backtracking)
# ========================================================
```

```
# ---- follow-up 1: function ----
# Answer: ways[mask] = the number of ways to fill the
# first popcount(mask) positions using the numbers in
# mask; add a number to the next position when it divides,
# or is divided by, that position.
# Time: O(2^n * n).  Space: O(2^n).
def count_beautiful_arrangements(n):
    ways = [0] * (1 << n)
    ways[0] = 1
    for mask in range(1 << n):
        if ways[mask] == 0:
            continue
        position = 1
        for bit in range(n):
            if (mask >> bit) & 1:
                position += 1
        if position > n:
            continue
        for number in range(1, n + 1):
            if (mask >> (number - 1)) & 1:
                continue
            if number % position == 0 or position % number == 0:
                ways[mask | (1 << (number - 1))] += ways[mask]
    return ways[(1 << n) - 1]


# ============================================================
# FOLLOW-UP 2: List all the beautiful arrangements
# ============================================================

# ---- follow-up 2: function ----
# Answer: Backtracking: place numbers position by
# position, only those that divide or are divided by the
# position.
# Time: O(n!) in the worst case (pruned in practice).
# Space: O(n) recursion plus the output.
def all_beautiful_arrangements(n):
    result = []
    used = [False] * (n + 1)

    def go(position, current):
        if position > n:
            result.append(list(current))
            return
        for number in range(1, n + 1):
            if not used[number] and (number % position == 0 or position % number == 0):
                used[number] = True
                current.append(number)
                go(position + 1, current)
                current.pop()
                used[number] = False

    go(1, [])
    return result
```

79. Word Search

Medium · Depth-First Search, Array, String, Backtracking, Matrix

leetcode.com/problems/word-search

★ **TOP 7 PROBLEM** in Backtracking / Subsets / Combinations / Permutations, Matrix / Grid Traversal: master this one first

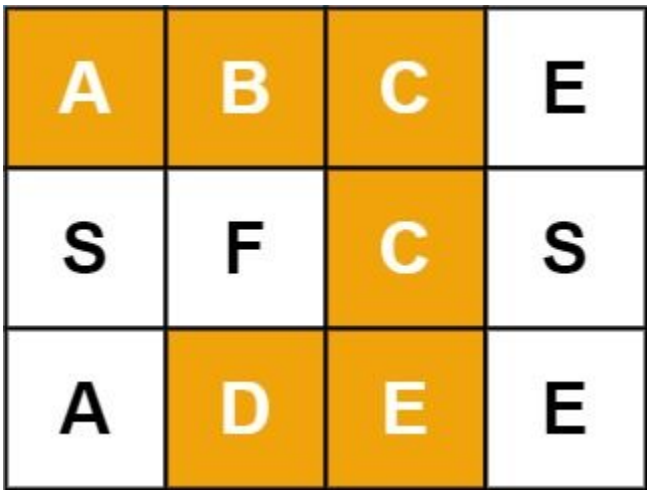
Also appears in pattern 18 Matrix / Grid Traversal

PROBLEM

Given an `m x n` grid of characters `board` and a string `word`, return `true` *if* `word` *exists in the grid*.

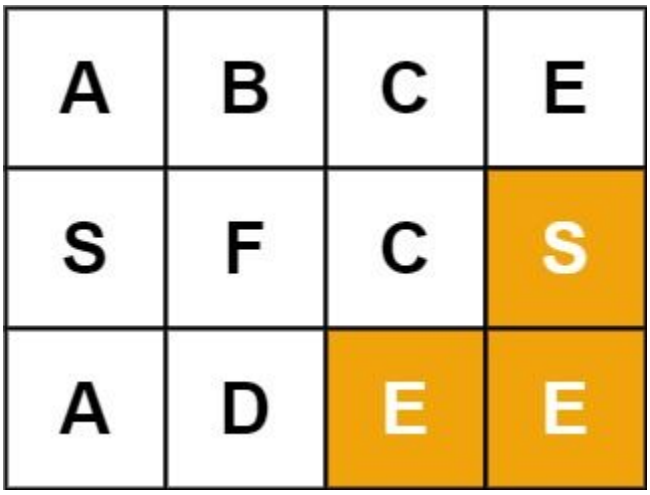
The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Example 1:



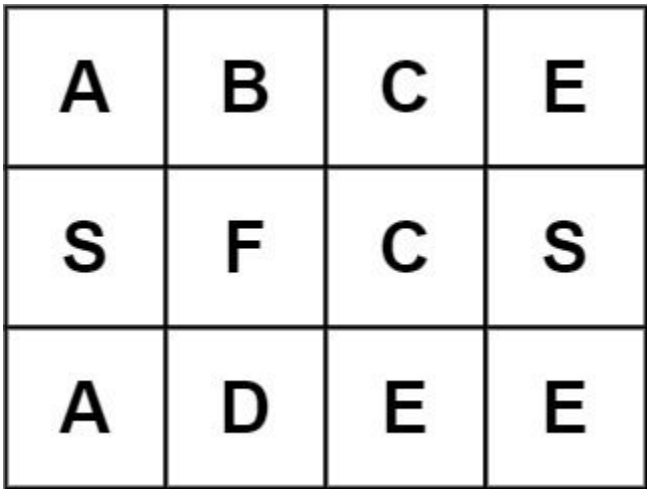

```
Input: board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCCED"
Output: true
```

Example 2:




```
Input: board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "SEE"
Output: true
```

Example 3:




```
Input: board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCB"
Output: false
```

Constraints:

- `m == board.length`
- `n = board[i].length`
- `1 <= m, n <= 6`
- `1 <= word.length <= 15`
- `board` and `word` consists of only lowercase and uppercase English letters.

Follow up: Could you use search pruning to make your solution faster with a larger `board`?

SOLUTION (PYTHON 3)

Solution 1: DFS (Backtracking)

How to think about it

From every cell, walk length-|word| paths and compare. At most 36 cells and |word| ≤ 15, unpruned search explodes. Match while walking, abort on mismatch, and never reuse a cell. Enumerate starts; dfs(i,j,k) means (i,j) should match word[k]. On match, write `'0'` to block re-entry, try four neighbors at k+1, then restore. Letters are never `'0'`, so no extra vis. Any successful start is enough.

We can enumerate each position (i, j) in the grid as the starting point of the search, and then start a depth-first search from the starting point. If we can search to the end of the word, it means the word exists, otherwise, it means the word does not exist.

Therefore, we design a function dfs(i, j, k), which represents whether we can successfully search from the (i, j) position of the grid, starting from the kth character of the word. The execution steps of the function dfs(i, j, k) are as follows:

- If k = |word|-1, it means that we have searched to the last character of the word. At this time, we only need to judge whether the character at the (i, j) position of the grid is equal to word[k]. If they are equal, it means the word exists, otherwise, it means the word does not exist. Whether the word exists or not, there is no need to continue to search, so return the result directly.
- Otherwise, if the word[k] character is not equal to the character at the (i, j) position of the grid, it means that the search failed this time, so return `false` directly.
- Otherwise, we temporarily store the character at the (i, j) position of the grid in c, and then modify the character at this position to a special character `'0'`, indicating that the character at this position has been used to prevent it from being reused in subsequent searches. Then we start from the up, down, left, and right directions of the (i, j) position to search for the k+1th character in the grid. If any direction is successful, it means the search is successful, otherwise, it means the search failed. At this time, we need to restore the character at the (i, j) position of the grid, that is, put c back to the

(i, j) position (backtracking).

In the main function, we enumerate each position (i, j) in the grid as the starting point. If calling dfs(i, j, 0) returns `true`, it means the word exists, otherwise, it means the word does not exist, so return `false`.

The time complexity is $O(m \times n \times 3^k)$, and the space complexity is $O(\min(m \times n, k))$. Here, m and n are the number of rows and columns of the grid, respectively; and k is the length of the string word.

Python 3

```
class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        def dfs(i: int, j: int, k: int) -> bool:
            if k == len(word) - 1:
                return board[i][j] == word[k]
            if board[i][j] != word[k]:
                return False
            c = board[i][j]
            board[i][j] = "0"
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                x, y = i + a, j + b
                ok = 0 <= x < m and 0 <= y < n and board[x][y] != "0"
                if ok and dfs(x, y, k + 1):
                    return True
            board[i][j] = c
            return False

        m, n = len(board), len(board[0])
        return any(dfs(i, j, 0) for i in range(m) for j in range(n))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Search for MANY words at once (Word Search II) with a
# trie
# 2. Count how many different paths spell the word


# ============================================================
# FOLLOW-UP 1: Search for MANY words at once (Word Search
# II) with a trie
# ============================================================

# ---- follow-up 1: function ----
# Answer: Put all the words in a trie and walk the grid
# once: a path that leaves the trie is abandoned, and a
# node that ends a word records it.
# Time: O(m * n * 3^L), L = longest word (pruned by the
# trie).  Space: O(W) for the trie.
def find_words(board, words):
    root = {}
    for word in words:
        node = root
        for ch in word:
            if ch not in node:
                node[ch] = {}
            node = node[ch]
        node["$"] = word
    rows, cols = len(board), len(board[0])
    found = []

    def go(r, c, node):
        ch = board[r][c]
        if ch not in node:
            return
        next_node = node[ch]
        if "$" in next_node:
```

```
            found.append(next_node.pop("$"))
        board[r][c] = "#"
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] != "#":
                go(nr, nc, next_node)
        board[r][c] = ch

    for r in range(rows):
        for c in range(cols):
            go(r, c, root)
    return found


# ============================================================
# FOLLOW-UP 2: Count how many different paths spell the
# word
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same search, but do not stop at the first
# success: add one for every path that reaches the end of
# the word.
# Time: O(m * n * 3^L).  Space: O(L).
def count_word_paths(board, word):
```

```
    rows, cols = len(board), len(board[0])
    count = [0]

    def go(r, c, index):
        if board[r][c] != word[index]:
            return
        if index == len(word) - 1:
            count[0] += 1
            return
        saved = board[r][c]
        board[r][c] = "#"
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols:
                go(nr, nc, index + 1)
        board[r][c] = saved

    for r in range(rows):
        for c in range(cols):
            go(r, c, 0)
    return count[0]
```

131. Palindrome Partitioning

Medium · String, Dynamic Programming, Backtracking

leetcode.com/problems/palindrome-partitioning

★ **TOP 7 PROBLEM** in Backtracking / Subsets / Combinations / Permutations: master this one first

PROBLEM

Given a string `s`, partition `s` such that every substring of the partition is a **palindrome**. Return *all possible palindrome partitioning of* `s`.

Example 1:

```
Input: s = "aab"
Output: [["a","a","b"],["aa","b"]]
```

Example 2:

```
Input: s = "a"
Output: [["a"]]
```

Constraints:

- `1 <= s.length <= 16`
- `s` contains only lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Preprocessing + DFS (Backtracking)

How to think about it

List every way to cut s into palindromic pieces. n≤ 16, so at most 2^{n-1} partitions; backtracking is fine. Checking palindromes at each cut rescans the same spans. Precompute f[i][j] whether s[i..j] is a palindrome, then try the next cut only when f[i][j] is true.

We can use dynamic programming to preprocess whether any substring in the string is a palindrome, i.e., f[i][j] indicates whether the substring s[i..j] is a palindrome.

Next, we design a function dfs(i), which represents starting from the i-th character of the string and partitioning it into several palindromic substrings, with the current partition scheme being t.

If i = |s|, it means the partitioning is complete, and we add t to the answer array and then return.

Otherwise, we can start from i and enumerate the end position j from small to large. If s[i..j] is a palindrome, we add s[i..j] to t, then continue to recursively call dfs(j+1). When backtracking, we need to pop s[i..j].

The time complexity is $O(n \times 2^n)$, and the space complexity is $O(n^2)$. Here, n is the length of the string.

Python 3

```python
class Solution:
    def partition(self, s: str) -> List[List[str]]:
        def dfs(i: int):
            if i == n:
                ans.append(t[:])
                return
            for j in range(i, n):
                if f[i][j]:
                    t.append(s[i : j + 1])
                    dfs(j + 1)
                    t.pop()

        n = len(s)
        f = [[True] * n for _ in range(n)]
        for i in range(n - 1, -1, -1):
            for j in range(i + 1, n):
                f[i][j] = s[i] == s[j] and f[i + 1][j - 1]
        ans = []
        t = []
        dfs(0)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The minimum number of cuts so that every part is a
# palindrome
# 2. Only count the partitions


# ============================================================
# FOLLOW-UP 1: The minimum number of cuts so that every
# part is a palindrome
# ============================================================

# ---- follow-up 1: function ----
# Answer: cuts[i] = the fewest cuts for the first i
# letters; try every palindromic last part, using a table
# that tells which substrings are palindromes.
# Time: O(n^2).  Space: O(n^2).
def min_cuts(s):
    n = len(s)
    is_pal = []
    for i in range(n):
        is_pal.append([False] * n)
    for i in range(n - 1, -1, -1):
        for j in range(i, n):
            if s[i] == s[j] and (j - i < 2 or is_pal[i + 1][j - 1]):
                is_pal[i][j] = True
    cuts = [0] * (n + 1)
    for end in range(1, n + 1):
        cuts[end] = end - 1
        for start in range(end):
            if is_pal[start][end - 1]:
                candidate = 0 if start == 0 else cuts[start] + 1
                if candidate < cuts[end]:
                    cuts[end] = candidate
    return cuts[n] if n > 0 else 0


# ============================================================
# FOLLOW-UP 2: Only count the partitions
# ============================================================

# ---- follow-up 2: function ----
```

```
# Answer: ways[i] = the number of ways to partition the
# first i letters: add ways[start] for every palindromic
# part that ends at i.
# Time: O(n^3).  Space: O(n).
def count_palindrome_partitions(s):
    n = len(s)
    ways = [0] * (n + 1)
    ways[0] = 1
    for end in range(1, n + 1):
        for start in range(end):
            part = s[start:end]
            if part == part[::-1]:
                ways[end] += ways[start]
    return ways[n]
```

93. Restore IP Addresses

Medium · String, Backtracking

leetcode.com/problems/restore-ip-addresses

PROBLEM

A **valid IP address** consists of exactly four integers separated by single dots. Each integer is between `0` and `255` (**inclusive**) and cannot have leading zeros.

- For example, `"0.1.2.201"` and `"192.168.1.1"` are **valid** IP addresses, but `"0.011.255.245"`, `"192.168.1.312"` and `"192.168@1.1"` are **invalid** IP addresses.

Given a string `s` containing only digits, return *all possible valid IP addresses that can be formed by inserting dots into* `s`. You are **not** allowed to reorder or remove any digits in `s`. You may return the valid IP addresses in **any** order.

Example 1:

```
Input: s = "25525511135"
Output: ["255.255.11.135","255.255.111.35"]
```

Example 2:

```
Input: s = "0000"
Output: ["0.0.0.0"]
```

Example 3:

```
Input: s = "101023"
Output: ["1.0.10.23","1.0.102.3","10.1.0.23","10.10.2.3","101.0.2.3"]
```

Constraints:

- `1 <= s.length <= 20`
- `s` consists of digits only.

SOLUTION (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> An IP is exactly four segments, each 0– 255 with no leading zeros. n ≤ 20, so enumerating three cuts works, but leading zeros, overflow, and segment count scatter across nested loops. Backtracking is the natural shape: from index i, try 1–3 digits as the next segment, and recurse if valid. Collect only when we have four segments and the string is consumed; stop if we already have four or run past the end. Prune illegal prefixes early.

We define a function dfs(i), which represents the list of IP addresses that can be formed starting from the ith position of string s.

The execution steps of function dfs(i) are as follows:

If i is greater than or equal to the length of string s, it means that we have completed the splicing of four segments of the IP address. At this point, we need to check whether it meets the requirements of the four segments of the IP address. If it does, add the current IP to the answer.

If i is less than the length of string s, it means that we still need to splice a segment of the IP address. At this point, we need to determine the value of this segment of the IP address. If the value is greater than 255, or the current position i is 0 and the value of several positions after i is greater than 0, it means that it does not meet the requirements, so we return directly. Otherwise, add it to the IP address list, and continue to search for the next segment of the IP address.

The time complexity is $O(n \times 3^4)$, and the space complexity is $O(n)$. Here, n is the length of string s.

Python 3

```python
class Solution:
    def restoreIpAddresses(self, s: str) -> List[str]:
        def check(i: int, j: int) -> int:
            if s[i] == "0" and i != j:
                return False
            return 0 <= int(s[i : j + 1]) <= 255

        def dfs(i: int):
            if i >= n and len(t) == 4:
                ans.append(".".join(t))
                return
            if i >= n or len(t) >= 4:
                return
            for j in range(i, min(i + 3, n)):
                if check(i, j):
                    t.append(s[i : j + 1])
                    dfs(j + 1)
                    t.pop()

        n = len(s)
        ans = []
        t = []
        dfs(0)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count the valid addresses only
# 2. Check whether a string is a valid IPv4 address


# ============================================================
# FOLLOW-UP 1: Count the valid addresses only
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same search, adding one when four valid
# parts use all the digits.
# Time: O(1) (at most 81 ways to cut).  Space: O(1).
def count_ip_addresses(s):
    def go(index, parts):
        if parts == 4:
            return 1 if index == len(s) else 0
        total = 0
        for length in range(1, 4):
            part = s[index:index + length]
            if len(part) < length:
                break
            if len(part) > 1 and part[0] == "0":
                continue
            if int(part) > 255:
                continue
            total += go(index + length, parts + 1)
        return total

    return go(0, 0)


# ============================================================
# FOLLOW-UP 2: Check whether a string is a valid IPv4
# address
# ============================================================

# ---- follow-up 2: function ----
```

```
# Answer: Split at the dots: exactly four parts, each made
# of digits only, without leading zeros, with a value from
# 0 to 255.
# Time: O(n).  Space: O(n).
def is_valid_ipv4(text):
    parts = text.split(".")
    if len(parts) != 4:
        return False
    for part in parts:
        if part == "" or not part.isdigit():
            return False
        if len(part) > 1 and part[0] == "0":
            return False
        if int(part) > 255:
            return False
    return True
```

89. Gray Code

Medium · Bit Manipulation, Math, Backtracking

leetcode.com/problems/gray-code

Also appears in pattern 27 Bit Manipulation

PROBLEM

An **n-bit gray code sequence** is a sequence of 2^n integers where:

- Every integer is in the **inclusive** range `[0, 2^n - 1]`,
- The first integer is `0`,
- An integer appears **no more than once** in the sequence,
- The binary representation of every pair of **adjacent** integers differs by **exactly one bit**, and
- The binary representation of the **first** and **last** integers differs by **exactly one bit**.

Given an integer `n`, return *any valid **n-bit gray code sequence***.

Example 1:

```
Input: n = 2
Output: [0,1,3,2]
Explanation:
The binary representation of [0,1,3,2] is [00,01,11,10].
- 00 and 01 differ by one bit
- 01 and 11 differ by one bit
- 11 and 10 differ by one bit
- 10 and 00 differ by one bit
[0,2,3,1] is also a valid gray code sequence, whose binary representation is [00,10,11,01].
- 00 and 10 differ by one bit
- 10 and 11 differ by one bit
- 11 and 01 differ by one bit
- 01 and 00 differ by one bit
```

Example 2:

```
Input: n = 1
Output: [0,1]
```

Constraints:

- `1 <= n <= 16`

SOLUTION (PYTHON 3)

Solution 1: Binary to Gray Code Conversion

> **How to think about it**
>
> The first idea is backtrack: start at 0, flip an unused bit each time, until 2^n numbers. Correct, and n ≤ 16 would pass, but we must dedup and also make first and last differ by one bit. The bottleneck is maintaining "adjacent codes differ by one bit" by hand. Binary-reflected Gray code has a closed form: i maps to i xor (i ≫ 1). That formula already guarantees adjacent integers differ by one bit, and 0 versus 2^n - 1 differs only in the top bit. So just map [0, 2^n) — no search.

Gray code is a type of encoding method that we often encounter in engineering. Its basic feature is that only one bit of binary number is different between any two adjacent codes.

The rule for converting binary code to binary Gray code is to keep the highest bit of the binary code as the highest bit of the Gray code, and the second highest bit of the Gray code is the XOR of the highest bit and the second highest bit of the binary code. The calculation of the remaining bits of the Gray code is similar to the second highest bit.

Assume that a binary number is represented as $B_{n-1}B_{n-2}...B_2B_1B_0$, and its Gray code is represented as $G_{n-1}G_{n-2}...G_2G_1G_0$. The highest bit is retained, so $G_{n-1} = B_{n-1}$; and the other bits $G_i = B_{i+1}$ xor B_i, where i=0,1,2..,n-2.

Therefore, for an integer x, we can use the function gray(x) to get its Gray code:

```
int gray(x) {
    return x ^ (x >> 1);
}
```

We directly map the integers $[0,..2^n - 1]$ to the corresponding Gray codes to get the answer array.

The time complexity is $O(2^n)$, where n is the integer given in the problem. Ignoring the space consumption of the answer, the space complexity is O(1).

Python 3

```
class Solution:
    def grayCode(self, n: int) -> List[int]:
        return [i ^ (i >> 1) for i in range(1 << n)]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Convert a number to its Gray code and back
# 2. A Gray code sequence that starts at a given number


# ========================================================
# FOLLOW-UP 1: Convert a number to its Gray code and back
# ========================================================

# ---- follow-up 1: function ----
# Answer: The Gray code of x is x XOR (x shifted right by
# one). To go back, XOR the bits from the top down: keep a
# running XOR.
# Time: O(log x).  Space: O(1).
def to_gray(x):
    return x ^ (x >> 1)


def from_gray(g):
    x = 0
    while g:
        x ^= g
        g >>= 1
    return x


# ========================================================
# FOLLOW-UP 2: A Gray code sequence that starts at a given
# number
# ========================================================

# ---- follow-up 2: function ----
# Answer: Generate the normal sequence and XOR every value
# with the start; neighbours still differ in one bit (XOR
# with a constant keeps that), and the first value becomes
# the start.
# Time: O(2^n).  Space: O(2^n).
def gray_code_from(n, start):
    sequence = []
    for i in range(1 << n):
        sequence.append((i ^ (i >> 1)) ^ start)
    return sequence
```

254. Factor Combinations

Medium · Backtracking, Prime Factorization

leetcode.com/problems/factor-combinations

PROBLEM

Numbers can be regarded as the product of their factors.

- For example, `8 = 2 x 2 x 2 = 2 x 4`.

Given an integer `n`, return *all possible combinations of its factors*. You may return the answer in **any order**.

Note that the factors should be in the range `[2, n - 1]`.

Example 1:

```
Input: n = 1
Output: []
```

Example 2:

```
Input: n = 12
Output: [[2,6],[3,4],[2,2,3]]
```

Example 3:

```
Input: n = 37
Output: []
```

Constraints:

- $1 <= n <= 10^7$

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Factorizations of n with at least two factors should be generated in nondecreasing order to avoid permutations. Enumerate factors from the current minimum i up to √n. dfs(n,i) records the chosen factors plus leftover n, then tries each j≥ i that divides n and recurses on n/j.

Python 3

```python
class Solution:
    def getFactors(self, n: int) -> List[List[int]]:
        def dfs(n, i):
            if t:
                ans.append(t + [n])
            j = i
            while j * j <= n:
                if n % j == 0:
                    t.append(j)
                    dfs(n // j, j)
                    t.pop()
                j += 1

        t = []
        ans = []
        dfs(n, 2)
        return ans
```

Complexity. Time: about O(F * sqrt(n)), F = number of factorisations. Space: O(log n) recursion depth (plus the answers).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Only count the factor combinations
# 2. Prime factorization: list the primes of n with their
# exponents


# ============================================================
# FOLLOW-UP 1: Only count the factor combinations
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same recursion with factors in increasing
# order; add one for every completed product.
# Time: about O(F * sqrt(n)), F = number of
# factorisations.  Space: O(log n) recursion.
def count_factorizations(n):
    def go(remaining, smallest):
        total = 0
        factor = smallest
        while factor * factor <= remaining:
            if remaining % factor == 0:
                total += 1 + go(remaining // factor, factor)
            factor += 1
        return total

    return go(n, 2)


# ============================================================
# FOLLOW-UP 2: Prime factorization: list the primes of n
# with their exponents
# ============================================================

# ---- follow-up 2: function ----
# Answer: Divide by 2, 3, 4... while the divisor squared
# is at most the remaining number; what is left over
# (above 1) is a prime.
# Time: O(sqrt(n)).  Space: O(log n).
def prime_factorization(n):
    factors = []
    divisor = 2
    while divisor * divisor <= n:
        count = 0
        while n % divisor == 0:
            n //= divisor
            count += 1
        if count > 0:
            factors.append((divisor, count))
        divisor += 1
    if n > 1:
        factors.append((n, 1))
    return factors
```

698. Partition to K Equal Sum Subsets

Medium · Bit Manipulation, Memoization, Array, Dynamic Programming, Backtracking, Bitmask

leetcode.com/problems/partition-to-k-equal-sum-subsets

Also appears in pattern 28 Dynamic Programming - Knapsack Style

PROBLEM

Given an integer array `nums` and an integer `k`, return `true` if it is possible to divide this array into `k` non-empty subsets whose sums are all equal.

Example 1:

```
Input: nums = [4,3,2,3,5,2,1], k = 4
Output: true
Explanation: It is possible to divide it into 4 subsets (5), (1, 4), (2,3), (2,3) with equal sums.
```

Example 2:

```
Input: nums = [1,2,3,4], k = 3
Output: false
```

Constraints:

- `1 <= k <= nums.length <= 16`
- `1 <= nums[i] <= 10^4`
- The frequency of each element is in the range `[1, 4]`.

SOLUTIONS (PYTHON 3)

Solution 1: DFS + Pruning

> **How to think about it**
>
> Partition into k equal-sum subsets. If the total is not divisible by k, fail. n≤ 16 allows DFS if we cut symmetric fillings. Target s, array cur holds bucket sums. Place numbers largest-first: skip a bucket that would exceed s or that matches the previous bucket. Success when every number is placed.

According to the problem description, we need to partition the array nums into k subsets such that the sum of each subset is equal. Therefore, we first sum all the elements in nums. If the total sum cannot be divided by k, it means we cannot partition the array into k subsets, and we return false early.

If the total sum can be divided by k, let's denote the expected sum of each subset as s. Then, we create an array cur of length k to represent the current sum of each subset.

We sort the array nums in descending order (to reduce the number of searches), and then start from the first element, trying to add it to each subset in cur one by one. If adding nums[i] to a subset cur[j] makes the subset's sum exceed s, it means it cannot be placed in that subset, and we can skip it. Additionally, if cur[j] is equal to cur[j - 1], it means we have already completed the search for cur[j - 1], and we can skip the current search.

If we can add all elements to cur, it means we can partition the array into k subsets, and we return true.

Python 3

```python
class Solution:
    def canPartitionKSubsets(self, nums: List[int], k: int) -> bool:
        def dfs(i: int) -> bool:
            if i == len(nums):
                return True
            for j in range(k):
                if j and cur[j] == cur[j - 1]:
                    continue
                cur[j] += nums[i]
                if cur[j] <= s and dfs(i + 1):
                    return True
                cur[j] -= nums[i]
            return False

        s, mod = divmod(sum(nums), k)
        if mod:
            return False
        cur = [0] * k
        nums.sort(reverse=True)
        return dfs(0)
```

Complexity. Time: O(k^n) in the worst case (pruned in practice). Space: O(n + k).

Solution 2: State Compression + Memoization

> **How to think about it**
>
> Bucket DFS can still revisit the same used set. A mask state plus the current bucket remainder t memoizes the search. Sorting ascending lets us break once t+v>s.

Similar to Solution 1, we first check whether the array nums can be partitioned into k subsets. If it cannot be divided by k, we directly return false.

Let s be the expected sum of each subset, and let state represent the current partitioning state of the elements. For the i-th number, if the i-th bit of state is 0, it means the i-th element has not been partitioned.

Our goal is to form k subsets with a sum of s from all elements. Let t be the current sum of the subset. When the i-th element is not partitioned:

- If t + nums[i] > s, it means the i-th element cannot be added to the current subset. Since we sort the array nums in ascending order, all elements from position i onwards cannot be added to the current subset, and we directly return false.
- Otherwise, add the i-th element to the current subset, change the state to state | 2^i, and continue searching for unpartitioned elements. Note that if t + nums[i] = s, it means we can form a subset with a sum of s. The next step is to reset t to zero (which can be achieved by (t + nums[i]) mod s) and continue partitioning the next subset.

To avoid repeated searches, we use an array f of length 2^n to record the search results for each state. The array f has three possible values:

- `0` indicates that the current state has not been searched yet;
- `-1`: indicates that the current state cannot be partitioned into k subsets;
- `1`: indicates that the current state can be partitioned into k subsets.

The time complexity is $O(n \times 2^n)$, and the space complexity is $O(2^n)$. Here, n represents the length of the array nums. For each state, we need to traverse the array nums, which has a time complexity of O(n). The total number of states is 2^n, so the overall time complexity is $O(n \times 2^n)$.

Python 3

```python
class Solution:
    def canPartitionKSubsets(self, nums: List[int], k: int) -> bool:
        @cache
        def dfs(state, t):
            if state == mask:
                return True
            for i, v in enumerate(nums):
                if (state >> i) & 1:
                    continue
                if t + v > s:
                    break
                if dfs(state | 1 << i, (t + v) % s):
                    return True
            return False

        s, mod = divmod(sum(nums), k)
        if mod:
            return False
        nums.sort()
        mask = (1 << len(nums)) - 1
        return dfs(0, 0)
```

Solution 3: Dynamic Programming

> **How to think about it**
>
> Memoization is top-down. Bottom-up, f[i] says whether mask i is reachable and cur[i] is the current bucket fill. Try unused j to set f[i|2^j] without recursion.

We can use dynamic programming to solve this problem.

We define f[i] to represent whether there exists k subsets that meet the requirements when the current selected numbers' state is i. Initially, f[0] = true, and the answer is f[2^n - 1], where n is the length of the array nums. Additionally, we define cur[i] to represent the sum of the last subset when the current selected numbers' state is i.

We enumerate the states i in the range [0, 2^n]. For each state i, if f[i] is false, we skip it. Otherwise, we enumerate any number nums[j] in the array nums. If cur[i] + nums[j] > s, we break the enumeration loop because the subsequent numbers are larger and cannot be placed in the current subset. Otherwise, if the j-th bit of the binary representation of i is 0, it means the current nums[j] has not been selected. We can place it in the current subset, change the state to i | 2^j, update cur[i | 2^j] = (cur[i] + nums[j]) mod s, and set f[i | 2^j] = true.

Finally, we return f[2^n - 1].

The time complexity is $O(n \times 2^n)$, and the space complexity is $O(2^n)$. Here, n represents the length of the array nums.

Python 3

```python
class Solution:
    def canPartitionKSubsets(self, nums: List[int], k: int) -> bool:
        s = sum(nums)
        if s % k:
            return False
        s //= k
        nums.sort()
        n = len(nums)
        f = [False] * (1 << n)
        cur = [0] * (1 << n)
        f[0] = True
        for i in range(1 << n):
            if not f[i]:
                continue
            for j in range(n):
                if cur[i] + nums[j] > s:
                    break
                if (i >> j & 1) == 0:
                    if not f[i | 1 << j]:
                        cur[i | 1 << j] = (cur[i] + nums[j]) % s
                        f[i | 1 << j] = True
        return f[-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. A bitmask table instead of plain backtracking
# 2. Return the k subsets themselves


# ============================================================
# FOLLOW-UP 1: A bitmask table instead of plain
# backtracking
# ============================================================

# ---- follow-up 1: function ----
# Answer: best[mask] = the sum of the numbers in the
# current partly filled bucket when the numbers in mask
# are used (or -1 if impossible); add one more number when
# it still fits.
# Time: O(2^n * n).  Space: O(2^n).
def can_partition_bitmask(nums, k):
    total = 0
    for x in nums:
        total += x
    if total % k != 0:
        return False
    target = total // k
    n = len(nums)
    filled = [-1] * (1 << n)                          # the sum inside the unfinished bucket
    filled[0] = 0
    for mask in range(1 << n):
        if filled[mask] == -1:
            continue
        for i in range(n):
            if (mask >> i) & 1:
                continue
            if filled[mask] + nums[i] <= target:
                next_mask = mask | (1 << i)
                filled[next_mask] = (filled[mask] + nums[i]) % target
    return filled[(1 << n) - 1] == 0


# ============================================================
# FOLLOW-UP 2: Return the k subsets themselves
```

```
# ============================================================

# ---- follow-up 2: function ----
# Answer: Backtracking that puts each number into a bucket
# whose sum stays at most the target; skip a bucket when
# an earlier bucket has the same sum (it would give the
# same result).
# Time: O(k^n) in the worst case (pruned in practice).
# Space: O(n + k).
def partition_into_subsets(nums, k):
    total = 0
    for x in nums:
        total += x
    if total % k != 0:
        return None
    target = total // k
    items = sorted(nums, reverse=True)
    buckets = []
    sums = [0] * k
    for i in range(k):
```

```
        buckets.append([])

    def go(index):
        if index == len(items):
            return True
        tried = set()
        for b in range(k):
            if sums[b] + items[index] <= target and sums[b] not in tried:
                tried.add(sums[b])
                sums[b] += items[index]
                buckets[b].append(items[index])
                if go(index + 1):
                    return True
                buckets[b].pop()
                sums[b] -= items[index]
        return False

    if go(0):
        return buckets
    return None
```

51. N-Queens

Hard · Array, Backtracking, Algorithm X

leetcode.com/problems/n-queens

★ **TOP 7 PROBLEM** in Backtracking / Subsets / Combinations / Permutations: master this one first

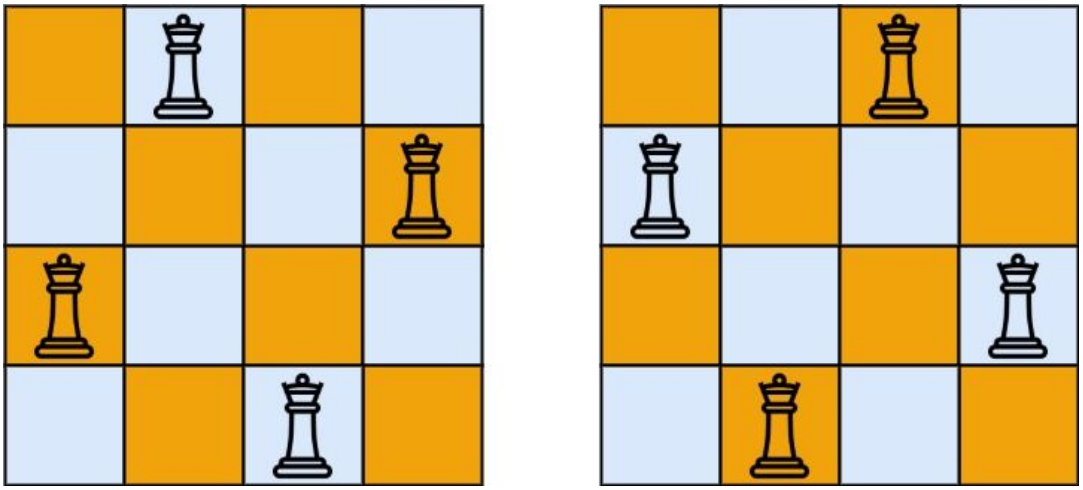
PROBLEM

The **n-queens** puzzle is the problem of placing `n` queens on an `n x n` chessboard such that no two queens attack each other.

Given an integer `n`, return *all distinct solutions to the **n-queens puzzle***. You may return the answer in **any order**.

Each solution contains a distinct board configuration of the n-queens' placement, where `'Q'` and `'.'` both indicate a queen and an empty space, respectively.

Example 1:



```
Input: n = 4
Output: [[".Q..","...Q","Q...","..Q."],["..Q.","Q...","...Q",".Q.."]]
Explanation: There exist two distinct solutions to the 4-queens puzzle as shown above
```

Example 2:

```
Input: n = 1
Output: [["Q"]]
```

Constraints:

- `1 <= n <= 9`

SOLUTION (PYTHON 3)

Solution 1: DFS (Backtracking)

How to think about it

The first idea is to try every column permutation: put the queen in row i at column p_i, then check diagonals. n ≤ 9, so 9! is searchable, but we only discover a conflict after the board is full, and we still have to build the string grid. The waste is placing everything, then validating. One queen per row is mandatory; conflicts are only the same column or the same diagonal. Column, i+j, and n-i+j can all be marked in O(1). So we DFS row by row: place and mark only when legal, then backtrack. The board g flips `'Q'`/`'.'` in lockstep; at row n we copy one answer.

We define three arrays col, dg, and udg to represent whether there is a queen in the column, the main diagonal, and the anti-diagonal, respectively. If there is a queen at position (i, j), then col[j], dg[i + j], and udg[n - i + j] are all 1. In addition, we use an array g to record the current state of the chessboard, where all elements in g are initially `'.'`.

Next, we define a function dfs(i), which represents placing queens starting from the ith row.

In dfs(i), if i = n, it means that we have completed the placement of all queens. We put the current g into the answer array and end the recursion.

Otherwise, we enumerate each column j of the current row. If there is no queen at position (i, j), that is, col[j], dg[i + j], and udg[n - i + j] are all 0, then we can place a queen, that is, change g[i][j] to `'Q'`, and set col[j], dg[i + j], and udg[n - i + j] to 1. Then we continue to search the next row, that is, call dfs(i + 1). After the recursion ends, we need to change g[i][j] back to `'.'` and set col[j], dg[i + j], and udg[n - i + j] to 0.

In the main function, we call dfs(0) to start recursion, and finally return the answer array.

The time complexity is $O(n^2 \times n!)$, and the space complexity is O(n). Here, n is the integer given in the problem.

Python 3

```python
class Solution:
    def solveNQueens(self, n: int) -> List[List[str]]:
        def dfs(i: int):
            if i == n:
                ans.append(["".join(row) for row in g])
                return
            for j in range(n):
                if col[j] + dg[i + j] + udg[n - i + j] == 0:
                    g[i][j] = "Q"
                    col[j] = dg[i + j] = udg[n - i + j] = 1
                    dfs(i + 1)
                    col[j] = dg[i + j] = udg[n - i + j] = 0
                    g[i][j] = "."

        ans = []
        g = [["."] * n for _ in range(n)]
        col = [0] * n
        dg = [0] * (n << 1)
        udg = [0] * (n << 1)
        dfs(0)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count the solutions with bitmasks (fast)
# 2. Return just ONE solution quickly


# ============================================================
# FOLLOW-UP 1: Count the solutions with bitmasks (fast)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Three bitmasks mark the columns and the two
# diagonals that are attacked; the free squares of the
# next row are the bits that are not marked.
# Time: O(n!) (heavily pruned in practice).  Space: O(n).
def count_queens(n):
    full = (1 << n) - 1

    def go(columns, diagonal_1, diagonal_2):
        if columns == full:
            return 1
        count = 0
        free = full & ~(columns | diagonal_1 | diagonal_2)
        while free:
```

```
            bit = free & -free                      # the lowest free square
            free -= bit
            count += go(columns | bit, ((diagonal_1 | bit) << 1) & full, (diagonal_2 | bit) >> 1)
        return count

    return go(0, 0, 0)


# ============================================================
# FOLLOW-UP 2: Return just ONE solution quickly
# ============================================================

# ---- follow-up 2: function ----
# Answer: Stop the search as soon as the first complete
# placement is found.
# Time: O(n!) in the worst case.  Space: O(n).
def one_queens_solution(n):
    placement = []
    columns = set()
    slash = set()
    backslash = set()

    def go(row):
        if row == n:
            return True
        for col in range(n):
            if col in columns or row + col in slash or row - col in backslash:
                continue
            columns.add(col)
            slash.add(row + col)
            backslash.add(row - col)
            placement.append(col)
            if go(row + 1):
                return True
            placement.pop()
            columns.remove(col)
            slash.remove(row + col)
```

```
            backslash.remove(row - col)
        return False

    if go(0):
        return placement
    return None
```

37. Sudoku Solver

Hard · Array, Hash Table, Backtracking, Matrix, Algorithm X, Dancing Links

leetcode.com/problems/sudoku-solver

PROBLEM

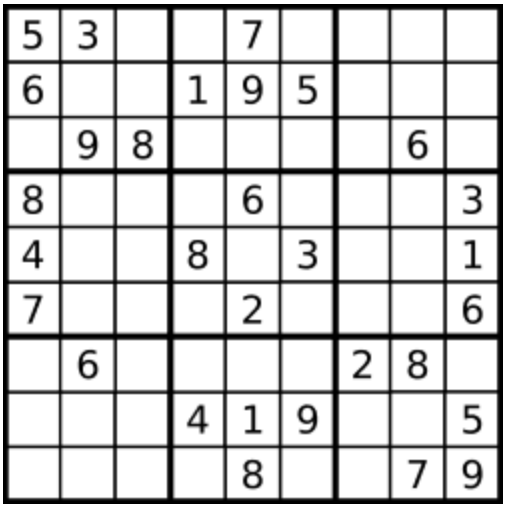
Write a program to solve a Sudoku puzzle by filling the empty cells.

A sudoku solution must satisfy **all of the following rules**:

1. Each of the digits `1-9` must occur exactly once in each row.
2. Each of the digits `1-9` must occur exactly once in each column.
3. Each of the digits `1-9` must occur exactly once in each of the 9 `3x3` sub-boxes of the grid.

The `'.'` character indicates empty cells.

Example 1:



5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9


```
Input: board = [["5","3",".",".","7",".",".",".","."],["6",".",".","1","9","5",".",".","."],[".","9","8",".",".",".",".",".","6","."],["8",".",".",".",".","6",".",".",".","3"],["4",".",".","8",".","3",".",".","1"],["7",".",".",".","2",".",".",".",".","6"],[".","6",".",".",".",".",".","2","8","."],[".",".",".","4","1","9",".",".",".","5"],[".",".",".",".",".","8",".",".","7","9"]]
Output: [["5","3","4","6","7","8","9","1","2"],["6","7","2","1","9","5","3","4","8"],["1","9","8","3","4","2","5","6","7"],["8","5","9","7","6","1","4","2","3"],["4","2","6","8","5","3","7","9","1"],["7","1","3","9","2","4","8","5","6"],["9","6","1","5","3","7","2","8","4"],["2","8","7","4","1","9","6","3","5"],["3","4","5","2","8","6","1","7","9"]]
Explanation: The input board is shown above and the only valid solution is shown below:
```


Constraints:

- `board.length == 9`
- `board[i].length == 9`
- `board[i][j]` is a digit or `'.'`.
- It is **guaranteed** that the input board has only one solution.

SOLUTION (PYTHON 3)

Solution 1: Backtracking

> **How to think about it**
>
> There can be dozens of empty cells and 9 choices each, so brute force is impossible. We must stop as soon as a row, column, or box is violated. As in Valid Sudoku, the bottleneck is repeatedly asking whether a digit is already used. Record the given digits in row, col, and block, then search only the empty-cell list t. dfs(k) fills the k-th empty cell: try each unused v, write it, and recurse. When a solution is found, ok cuts off further tries. Backtracking restores the occupancy flags; digits on the successful path stay on the board.

We use arrays row, col, and box to record whether each number has appeared in each row, each column, and each 3x3 sub-box, respectively. If the number i has appeared in row r, column c, or the b-th 3x3 sub-box, then row[r][i], col[c][i], and box[b][i] are all set to true.

We iterate over every empty cell in the board and enumerate the possible numbers v that can be filled in. If v has not appeared in the current row, column, or 3x3 sub-box, we can try filling in v and continue searching for the next empty cell. If we reach the end and all cells are filled, it means we have found a valid solution.

Python 3

```python
class Solution:
    def solveSudoku(self, board: List[List[str]]) -> None:
        def dfs(k):
            nonlocal ok
            if k == len(t):
                ok = True
                return
            i, j = t[k]
            for v in range(9):
                if row[i][v] == col[j][v] == block[i // 3][j // 3][v] == False:
                    row[i][v] = col[j][v] = block[i // 3][j // 3][v] = True
                    board[i][j] = str(v + 1)
                    dfs(k + 1)
                    row[i][v] = col[j][v] = block[i // 3][j // 3][v] = False
                if ok:
                    return

        row = [[False] * 9 for _ in range(9)]
        col = [[False] * 9 for _ in range(9)]
        block = [[[False] * 9 for _ in range(3)] for _ in range(3)]
        t = []
        ok = False
        for i in range(9):
            for j in range(9):
                if board[i][j] == '.':
                    t.append((i, j))
                else:
                    v = int(board[i][j]) - 1
                    row[i][v] = col[j][v] = block[i // 3][j // 3][v] = True
        dfs(0)
```

Complexity. Time: O(9^m), m = number of empty cells (pruned heavily in practice). Space: O(m).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Check whether a (partly filled) board is valid
# 2. Count the solutions (stop after a limit, e.g. 2, to
# check uniqueness)


# ============================================================
# FOLLOW-UP 1: Check whether a (partly filled) board is
# valid
# ============================================================

# ---- follow-up 1: function ----
# Answer: Every row, column and 3 x 3 box may contain each
# digit at most once (empty cells are '.').
# Time: O(81).  Space: O(81).
def is_valid_sudoku(board):
    rows = [set() for i in range(9)]
    cols = [set() for i in range(9)]
    boxes = [set() for i in range(9)]
    for r in range(9):
        for c in range(9):
            digit = board[r][c]
            if digit == ".":
                continue
            box = (r // 3) * 3 + c // 3
            if digit in rows[r] or digit in cols[c] or digit in boxes[box]:
                return False
            rows[r].add(digit)
            cols[c].add(digit)
            boxes[box].add(digit)
    return True


# ============================================================
# FOLLOW-UP 2: Count the solutions (stop after a limit,
# e.g. 2, to check uniqueness)
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same backtracking, but do not stop at the
# first solution: count it and keep searching until the
# limit is reached.
# Time: exponential: about O(9^m), m = empty cells.
# Space: O(m).
def count_sudoku_solutions(board, limit):
    count = [0]

    def candidates(r, c):
        used = set()
        for i in range(9):
            used.add(board[r][i])
            used.add(board[i][c])
        br, bc = r // 3 * 3, c // 3 * 3
        for i in range(3):
            for j in range(3):
                used.add(board[br + i][bc + j])
        return [d for d in "123456789" if d not in used]

    def go():
        if count[0] >= limit:
```

```
            return
        best = None
        for r in range(9):
            for c in range(9):
```

```
                if board[r][c] == ".":
                    options = candidates(r, c)
                    if best is None or len(options) < len(best[2]):
                        best = (r, c, options)
        if best is None:
            count[0] += 1
            return
        r, c, options = best
        for d in options:
            board[r][c] = d
            go()
            board[r][c] = "."

    go()
    return count[0]
```

PATTERN 16 OF 41

Graph DFS

20 problems · 1 Easy · 18 Medium · 1 Hard

THE IDEA

Go as deep as possible along one path before backing up. Mark visited nodes so each is processed once.

Signals: connected components, clone a graph, detect a cycle, all paths, number of provinces, flood fill, bipartite check.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
def dfs(u):
    seen.add(u)
    for v in graph[u]:
        if v not in seen:
            dfs(v)
components = 0
for u in range(n):
    if u not in seen:
        dfs(u)
        components += 1
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Flood Fill	Easy	LC 733	842
2	★ **Number of Islands**	Medium	LC 200	846
3	Max Area of Island	Medium	LC 695	851
4	★ **Clone Graph**	Medium	LC 133	854
5	★ **Pacific Atlantic Water Flow**	Medium	LC 417	858
6	★ **Surrounded Regions**	Medium	LC 130	862
7	★ **Course Schedule**	Medium	LC 207	866
8	Number of Provinces	Medium	LC 547	869
9	Number of Enclaves	Medium	LC 1020	873
10	Keys and Rooms	Medium	LC 841	879
11	Reorder Routes to Make All Paths Lead to the City Zero	Medium	LC 1466	883
12	All Paths From Source to Target	Medium	LC 797	888
13	★ **Find Eventual Safe States**	Medium	LC 802	891
14	Number of Closed Islands	Medium	LC 1254	895
15	Time Needed to Inform All Employees	Medium	LC 1376	899
16	Most Stones Removed with Same Row or Column	Medium	LC 947	903
17	Detonate the Maximum Bombs	Medium	LC 2101	908
18	Count Sub Islands	Medium	LC 1905	912
19	Path with Maximum Probability	Medium	LC 1514	916
20	★ **Making A Large Island**	Hard	LC 827	920

PATTERN 16 · GRAPH DFS · PROBLEM 1 OF 20

733. Flood Fill

Easy · Depth-First Search, Breadth-First Search, Array, Matrix

leetcode.com/problems/flood-fill

Also appears in pattern 18 Matrix / Grid Traversal

PROBLEM

You are given an image represented by an `m x n` grid of integers `image`, where `image[i][j]` represents the pixel value of the image. You are also given three integers `sr`, `sc`, and `color`. Your task is to perform a **flood fill** on the image starting from the pixel `image[sr][sc]`.

To perform a **flood fill**:

1. Begin with the starting pixel and change its color to `color`.
2. Perform the same process for each pixel that is **directly adjacent** (pixels that share a side with the original pixel, either horizontally or vertically) and shares the **same color** as the starting pixel.
3. Keep **repeating** this process by checking neighboring pixels of the *updated* pixels and modifying their color if it matches the original color of the starting pixel.
4. The process **stops** when there are **no more** adjacent pixels of the original color to update.

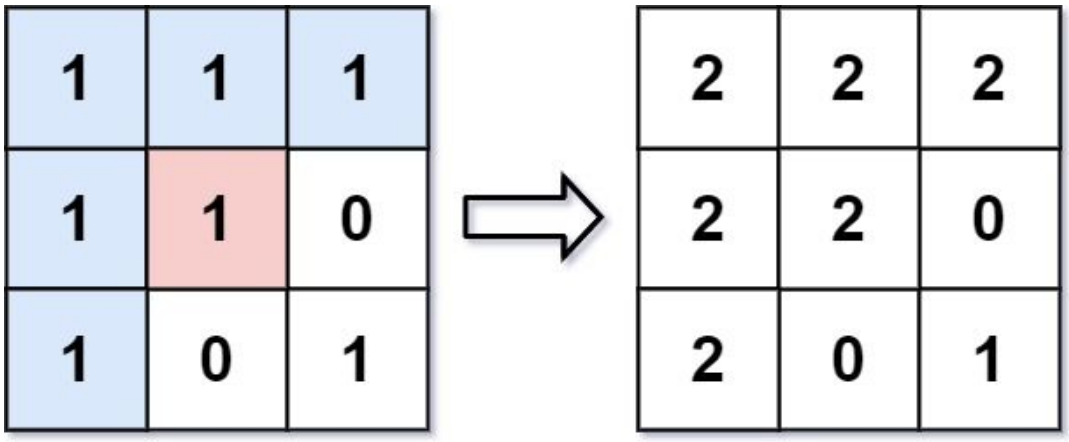
Return the **modified** image after performing the flood fill.

Example 1:

Input: image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, color = 2

Output: [[2,2,2],[2,2,0],[2,0,1]]

Explanation:



From the center of the image with position `(sr, sc) = (1, 1)` (i.e., the red pixel), all pixels connected by a path of the same color as the starting pixel (i.e., the blue pixels) are colored with the new color.

Note the bottom corner is **not** colored 2, because it is not horizontally or vertically connected to the starting pixel.

Example 2:

Input: image = [[0,0,0],[0,0,0]], sr = 0, sc = 0, color = 0

Output: [[0,0,0],[0,0,0]]

Explanation:

The starting pixel is already colored with 0, which is the same as the target color. Therefore, no changes are made to the image.

Constraints:

- `m == image.length`
- `n == image[i].length`
- `1 <= m, n <= 50`
- `0 <= image[i][j], color < 2^16`
- `0 <= sr < m`
- `0 <= sc < n`

SOLUTIONS (PYTHON 3)

Solution 1: DFS

How to think about it

Recolor the 4-connected component of the start pixel. The grid is small. If the start already has the target color we must not recurse, or we loop. The component is a grid graph. DFS recolors every neighbor that still equals the original color oc; the new color is the visited mark. Record oc and recurse from (sr,sc) only when it differs from color.

We denote the initial pixel's color as oc. If oc is not equal to the target color color, we start a depth-first search from (sr, sc) to change the color of all eligible pixels to the target color.

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the number of rows and columns of the 2D array image, respectively.

Python 3

```
class Solution:
    def floodFill(
        self, image: List[List[int]], sr: int, sc: int, color: int
    ) -> List[List[int]]:
        def dfs(i: int, j: int):
            image[i][j] = color
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < len(image) and 0 <= y < len(image[0]) and image[x][y] == oc:
                    dfs(x, y)

        oc = image[sr][sc]
        if oc != color:
            dirs = (-1, 0, 1, 0, -1)
            dfs(sr, sc)
        return image
```

Solution 2: BFS

How to think about it

Solution 1 is recursive; a large component uses O(mn) stack. The same flood can proceed level by level. BFS from the start: pop a pixel, recolor neighbors that still equal oc, and enqueue them. Same asymptotics, no call stack.

We first check if the initial pixel's color is equal to the target color. If it is, we return the original image directly. Otherwise, we can use the breadth-first search method, starting from (sr, sc), to change the color of all eligible pixels to the target color.

Specifically, we define a queue q and add the initial pixel (sr, sc) to the queue. Then, we continuously take pixels (i, j) from the queue, change their color to the target color, and add the pixels in the four directions (up, down, left, right) that have the same original color as the initial pixel to the queue. When the queue is empty, we have completed the flood fill.

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the number of rows and columns of the 2D array image, respectively.

Python 3

```python
class Solution:
    def floodFill(
        self, image: List[List[int]], sr: int, sc: int, color: int
    ) -> List[List[int]]:
        if image[sr][sc] == color:
            return image
        q = deque([(sr, sc)])
        oc = image[sr][sc]
        image[sr][sc] = color
        dirs = (-1, 0, 1, 0, -1)
        while q:
            i, j = q.popleft()
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < len(image) and 0 <= y < len(image[0]) and image[x][y] == oc:
                    q.append((x, y))
                    image[x][y] = color
        return image
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Spread in all 8 directions (diagonals too)
# 2. Return how many cells were repainted


# ============================================================
# FOLLOW-UP 1: Spread in all 8 directions (diagonals too)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use eight neighbour offsets instead of four.
# Time: O(m * n).  Space: O(m * n).
def flood_fill_8(image, sr, sc, color):
    old = image[sr][sc]
    if old == color:
        return image
    rows, cols = len(image), len(image[0])
    stack = [(sr, sc)]
    image[sr][sc] = color
    while stack:
        r, c = stack.pop()
        for dr in (-1, 0, 1):
            for dc in (-1, 0, 1):
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols and image[nr][nc] == old:
                    image[nr][nc] = color
                    stack.append((nr, nc))
    return image


# ============================================================
# FOLLOW-UP 2: Return how many cells were repainted
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the cells while spreading.
# Time: O(m * n).  Space: O(m * n).
def flood_fill_count(image, sr, sc, color):
    old = image[sr][sc]
    if old == color:
        return 0
    rows, cols = len(image), len(image[0])
    stack = [(sr, sc)]
    image[sr][sc] = color
```

```
    count = 0
    while stack:
        r, c = stack.pop()
        count += 1
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and image[nr][nc] == old:
                image[nr][nc] = color
                stack.append((nr, nc))
    return count
```

PATTERN 16 · GRAPH DFS · PROBLEM 2 OF 20

200. Number of Islands

Medium · Depth-First Search, Breadth-First Search, Union Find, Array, Matrix

leetcode.com/problems/number-of-islands

★ **TOP 7 PROBLEM** in Graph DFS, Matrix / Grid Traversal: master this one first

Also appears in pattern 18 Matrix / Grid Traversal

PROBLEM

Given an `m x n` 2D binary grid `grid` which represents a map of `'1'`s (land) and `'0'`s (water), return *the number of islands*.

An **island** is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

```
Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
Output: 1
```

Example 2:

```
Input: grid = [
  ["1","1","0","0","0"],
  ["1","1","0","0","0"],
  ["0","0","1","0","0"],
  ["0","0","0","1","1"]
]
Output: 3
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 300`
- `grid[i][j]` is `'0'` or `'1'`.

SOLUTIONS (PYTHON 3)

Solution 1: DFS

How to think about it

Counting every cell with value 1 as its own island would double-count 4-connected land. With m,n ≤ 300, a full scan is acceptable, but each connected component must be cleared in one search. Adjacent land cells form one component, so a DFS from any unseen land cell covers the whole island. We scan the grid, start DFS at each 1, rewrite visited land to 0, and increment the answer once per search.

We can use depth-first search (DFS) to traverse each island. We iterate through each cell (i, j) in the grid. If the cell's value is '1', it means we have found a new island. We can start a DFS from this cell, marking all connected land cells as '0' to avoid duplicate counting. Each time we find a new island, we increment the island count by 1.

The time complexity is O(m × n), and the space complexity is O(m × n). Where m and n are the number of rows and columns in the grid, respectively.

Python 3

```python
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def dfs(i, j):
            grid[i][j] = '0'
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and grid[x][y] == '1':
                    dfs(x, y)

        ans = 0
        dirs = (-1, 0, 1, 0, -1)
        m, n = len(grid), len(grid[0])
        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    dfs(i, j)
                    ans += 1
        return ans
```

Solution 2: BFS

> **How to think about it**
>
> Solution 1 already clears each component with an explicit stack. The same 4-connectivity can be expanded with a queue. The scan order matches Solution 1. The extra space is still the size of the frontier.

We can also use breadth-first search (BFS) to traverse each island. We iterate through each cell (i, j) in the grid. If the cell's value is '1', it means we have found a new island. We can start a BFS from this cell, marking all connected land cells as '0' to avoid duplicate counting. Each time we find a new island, we increment the island count by 1.

The specific BFS process is as follows:

1. Enqueue the starting cell (i, j) and mark its value as '0'.
2. While the queue is not empty, perform the following operations:
   - Dequeue a cell p.
   - Iterate through the four adjacent cells (x, y) of p. If (x, y) is within the grid bounds and its value is '1', enqueue it and mark its value as '0'.

The time complexity is O(m × n), and the space complexity is O(m × n). Where m and n are the number of rows and columns in the grid, respectively.

Python 3

```python
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def bfs(i, j):
            grid[i][j] = '0'
            q = deque([(i, j)])
            while q:
                i, j = q.popleft()
                for a, b in pairwise(dirs):
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n and grid[x][y] == '1':
                        q.append((x, y))
                        grid[x][y] = 0

        ans = 0
        dirs = (-1, 0, 1, 0, -1)
        m, n = len(grid), len(grid[0])
        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    bfs(i, j)
                    ans += 1
        return ans
```

Solution 3: Union-Find

> **How to think about it**
>
> If we want adjacency as an explicit partition, treat each land cell as a union-find node and union it with neighboring land. The number of islands is the number of components after the scan. Unions and finds are nearly linear; space remains proportional to the grid.

We can use the Union-Find data structure to solve this problem. We traverse each cell (i, j) in the grid, and if the cell's value is '1', we merge it with adjacent land cells. Finally, we count the number of distinct root nodes in the Union-Find structure, which represents the number of islands.

The time complexity is O(m × n × log (m × n)), and the space complexity is O(m × n). Where m and n are the number of rows and columns in the grid, respectively.

Python 3

```python
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        dirs = (0, 1, 0)
        m, n = len(grid), len(grid[0])
        p = list(range(m * n))
        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    for a, b in pairwise(dirs):
                        x, y = i + a, j + b
                        if x < m and y < n and grid[x][y] == '1':
                            p[find(i * n + j)] = find(x * n + y)
        return sum(
            grid[i][j] == '1' and i * n + j == find(i * n + j)
            for i in range(m)
            for j in range(n)
        )
```

Solution 4: Explicit Stack

> **How to think about it**
>
> Counting every cell with value 1 as its own island would double-count 4-connected land. With m,n ≤ 300, a full scan is acceptable, but each component must be cleared in one search. Recursing along a snake of 1000 land cells overflows the call stack, and a 300× 300 grid can hold that snake. Clearing an island does not need a return value. It only rewrites 4-connected 1s to 0. When a 1 is found, push it onto an explicit stack. Pop a cell and push each neighboring 1 after marking it. Each time a search starts, add one to the answer.

Scan every cell. When the value is `'1'`, flood the island with an explicit stack, rewrite connected land to `'0'`, and add one to the answer.

The time complexity is O(m × n), and the space complexity is O(m × n). Where m and n are the number of rows and columns in the grid, respectively.

Python 3

```
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def flood(i: int, j: int):
            stk = [(i, j)]
            grid[i][j] = '0'
            while stk:
                i, j = stk.pop()
                for a, b in pairwise(dirs):
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n and grid[x][y] == '1':
                        grid[x][y] = '0'
                        stk.append((x, y))

        ans = 0
        dirs = (-1, 0, 1, 0, -1)
        m, n = len(grid), len(grid[0])
        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    flood(i, j)
                    ans += 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Diagonal neighbours also connect land
# 2. Land cells are added one at a time: report the island
# count after each addition


# ============================================================
# FOLLOW-UP 1: Diagonal neighbours also connect land
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same search with eight directions.
# Time: O(m * n).  Space: O(m * n).
def count_islands_8(grid):
    rows, cols = len(grid), len(grid[0])
    seen = set()
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1" and (r, c) not in seen:
                count += 1
                stack = [(r, c)]
                seen.add((r, c))
                while stack:
                    x, y = stack.pop()
                    for dx in (-1, 0, 1):
                        for dy in (-1, 0, 1):
                            nx, ny = x + dx, y + dy
                            if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == "1" and (nx, ny) not
in seen:
                                seen.add((nx, ny))
                                stack.append((nx, ny))
    return count


# ============================================================
# FOLLOW-UP 2: Land cells are added one at a time: report
# the island count after each addition
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Union-find over the land cells: a new cell adds
# one island and every merge with a different neighbouring
# island removes one.
# Time: O(k * alpha(k)), k = number of additions.  Space:
# O(k).
def islands_after_each_addition(rows, cols, positions):
    parent = {}

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    count = 0
    result = []
    for r, c in positions:
        if (r, c) in parent:
            result.append(count)
            continue
```

```
        parent[(r, c)] = (r, c)
        count += 1
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if (nr, nc) in parent:
                a, b = find((r, c)), find((nr, nc))
                if a != b:
                    parent[a] = b
                    count -= 1
        result.append(count)
    return result
```

695. Max Area of Island

Medium · Depth-First Search, Breadth-First Search, Union Find, Array, Matrix

leetcode.com/problems/max-area-of-island

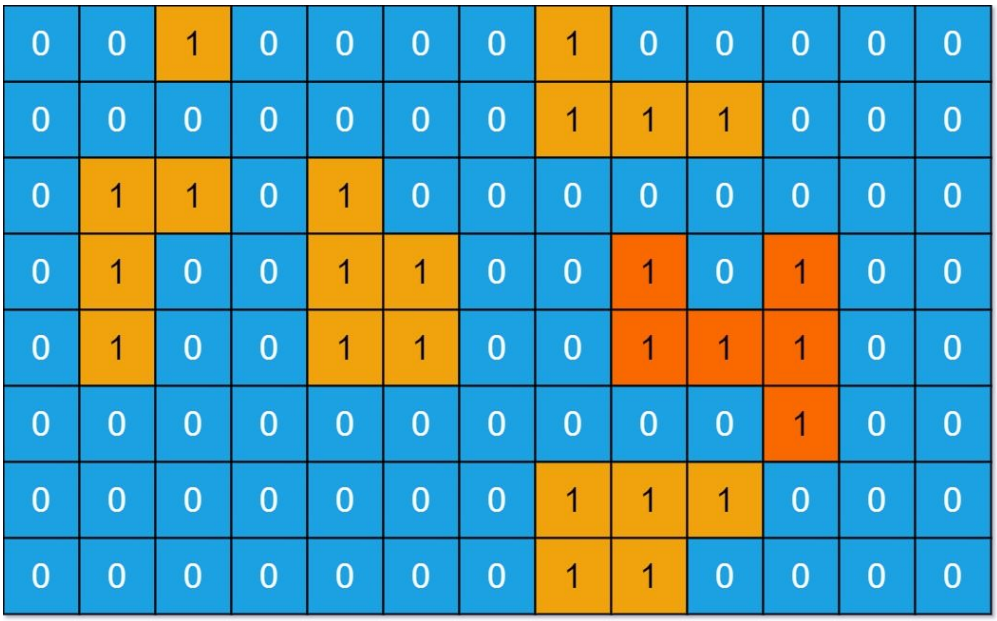
PROBLEM

You are given an `m x n` binary matrix `grid`. An island is a group of `1`'s (representing land) connected **4-directionally** (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

The **area** of an island is the number of cells with a value `1` in the island.

Return *the maximum **area** of an island in* `grid`. If there is no island, return `0`.

Example 1:




```
Input: grid = [[0,0,1,0,0,0,0,1,0,0,0,0,0],[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,1,1,0,1,0,0,0,0,0,0,0,0],[0,1,0,0,1,1,0,0,1,0,1,0,0],[0,1,0,0,1,1,0,0,1,1,1,0,0],[0,0,0,0,0,0,0,0,0,0,1,0,0],[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,0,0,0,0,0,0,1,1,0,0,0,0]]
Output: 6
Explanation: The answer is not 11, because the island must be connected 4-directionally.
```

Example 2:

```
Input: grid = [[0,0,0,0,0,0,0,0]]
Output: 0
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 50`
- `grid[i][j]` is either `0` or `1`.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Area is the count of 4-connected land cells. Each cell is visited at most once. DFS a land cell, set it to 0, and add the four recursive calls. The answer is the maximum over all starts.

Python 3

```
class Solution:
    def maxAreaOfIsland(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int) -> int:
            if grid[i][j] == 0:
                return 0
            ans = 1
            grid[i][j] = 0
            dirs = (-1, 0, 1, 0, -1)
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n:
                    ans += dfs(x, y)
            return ans

        m, n = len(grid), len(grid[0])
        return max(dfs(i, j) for i in range(m) for j in range(n))
```

Complexity. Time: O(m * n). Space: O(m * n) recursion in the worst case.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the cells of the biggest island
# 2. The areas of the k biggest islands


# ============================================================
# FOLLOW-UP 1: Return the cells of the biggest island
# ============================================================

# ---- follow-up 1: function ----
# Answer: Collect the cells of every island while
# searching and keep the biggest list.
# Time: O(m * n).  Space: O(m * n).
def biggest_island_cells(grid):
    rows, cols = len(grid), len(grid[0])
    seen = set()
    best = []
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1 and (r, c) not in seen:
                cells = []
                stack = [(r, c)]
                seen.add((r, c))
                while stack:
                    x, y = stack.pop()
                    cells.append((x, y))
                    for nx, ny in ((x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1)):
                        if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 1 and (nx, ny) not in
seen:
                            seen.add((nx, ny))
                            stack.append((nx, ny))
                if len(cells) > len(best):
                    best = cells
    return best
```

```
# ============================================================
# FOLLOW-UP 2: The areas of the k biggest islands
# ============================================================

# ---- follow-up 2: function ----
# Answer: Collect all island areas, sort them from big to
# small and take the first k.
# Time: O(m * n + I log I), I = number of islands.  Space:
# O(m * n).
def k_biggest_areas(grid, k):
    rows, cols = len(grid), len(grid[0])
    seen = set()
    areas = []
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1 and (r, c) not in seen:
                area = 0
                stack = [(r, c)]
                seen.add((r, c))
                while stack:
                    x, y = stack.pop()
                    area += 1
                    for nx, ny in ((x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1)):
                        if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 1 and (nx, ny) not in
seen:
                            seen.add((nx, ny))
```

```
                            stack.append((nx, ny))
                areas.append(area)
    areas.sort(reverse=True)
    return areas[:k]
```

PATTERN 16 · GRAPH DFS · PROBLEM 4 OF 20

133. Clone Graph

Medium · Depth-First Search, Breadth-First Search, Graph, Hash Table

leetcode.com/problems/clone-graph

★ **TOP 7 PROBLEM** in Graph DFS: master this one first

PROBLEM

Given a reference of a node in a **connected** undirected graph.

Return a **deep copy** (clone) of the graph.

Each node in the graph contains a value (`int`) and a list (`List[Node]`) of its neighbors.

```
class Node {
    public int val;
    public List<Node> neighbors;
}
```

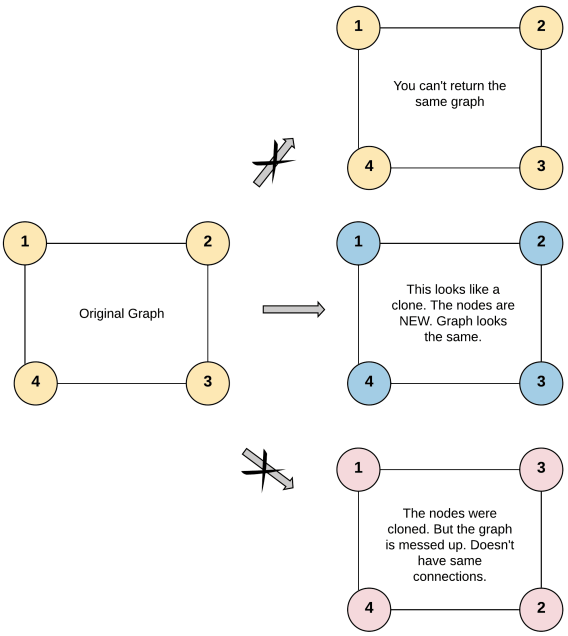
Test case format:

For simplicity, each node's value is the same as the node's index (1-indexed). For example, the first node with `val == 1`, the second node with `val == 2`, and so on. The graph is represented in the test case using an adjacency list.

An adjacency list is a collection of unordered **lists** used to represent a finite graph. Each list describes the set of neighbors of a node in the graph.

The given node will always be the first node with `val = 1`. You must return the **copy of the given node** as a reference to the cloned graph.

Example 1:




```
Input: adjList = [[2,4],[1,3],[2,4],[1,3]]
Output: [[2,4],[1,3],[2,4],[1,3]]
Explanation: There are 4 nodes in the graph.
1st node (val = 1)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).
2nd node (val = 2)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).
3rd node (val = 3)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).
4th node (val = 4)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).
```

Example 2:

```
Input: adjList = [[]]
Output: [[]]
Explanation: Note that the input contains one empty list. The graph consists of only one node with val = 1 and it does not have any neighbors.
```

Example 3:

```
Input: adjList = []
Output: []
Explanation: This an empty graph, it does not have any nodes.
```

Constraints:

- The number of nodes in the graph is in the range `[0, 100]`.
- `1 <= Node.val <= 100`
- `Node.val` is unique for each node.
- There are no repeated edges and no self-loops in the graph.
- The Graph is connected and all nodes can be visited starting from the given node.

SOLUTION (PYTHON 3)

Solution 1: Hash Table + DFS

How to think about it

Deep-copy a connected undirected graph; values are unique and n≤ 100. Copying values is not enough: neighbor pointers must refer to the new nodes, and cycles would loop if we recurse blindly. A map from original node to clone does it. On the first visit we create the clone and recurse on neighbors; a later visit returns the existing clone, which cuts cycles.

We use a hash table g to record the correspondence between each node in the original graph and its copy, and then perform depth-first search.

We define the function dfs(node), which returns the copy of the node. The process of dfs(node) is as follows:

- If node is null, then the return value of dfs(node) is null.
- If node is in g, then the return value of dfs(node) is g[node].
- Otherwise, we create a new node cloned and set the value of g[node] to cloned. Then, we traverse all the neighbor nodes nxt of node and add dfs(nxt) to the neighbor list of cloned.
- Finally, return cloned.

In the main function, we return dfs(node).

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes.

Python 3

```
"""
# Definition for a Node.
class Node:
    def __init__(self, val = 0, neighbors = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []
"""

from typing import Optional


class Solution:
    def cloneGraph(self, node: Optional["Node"]) -> Optional["Node"]:
        def dfs(node):
            if node is None:
                return None
            if node in g:
                return g[node]
            cloned = Node(node.val)
            g[node] = cloned
            for nxt in node.neighbors:
                cloned.neighbors.append(dfs(nxt))
            return cloned

        g = defaultdict()
        return dfs(node)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Clone without recursion (BFS)
# 2. Check that a copy is a real deep copy (same shape, no
# shared nodes)


# ============================================================
# FOLLOW-UP 1: Clone without recursion (BFS)
# ============================================================

# ---- follow-up 1: helper functions ----
class Node:
    def __init__(self, val=0, neighbors=None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []

# ---- follow-up 1: function ----
# Answer: A queue of original nodes and a dictionary
# original -> copy; copy each node's neighbour list when
# the node is taken out.
# Time: O(V + E).  Space: O(V).
from collections import deque


def clone_graph_bfs(node):
    if node is None:
        return None
    copies = {node: Node(node.val)}
    queue = deque([node])
    while queue:
        current = queue.popleft()
        for neighbour in current.neighbors:
            if neighbour not in copies:
                copies[neighbour] = Node(neighbour.val)
                queue.append(neighbour)
            copies[current].neighbors.append(copies[neighbour])
```

```
    return copies[node]


# ============================================================
# FOLLOW-UP 2: Check that a copy is a real deep copy (same
# shape, no shared nodes)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk both graphs together: values must match, no
# node of the copy may be a node of the original, and the
# same original node must always map to the same copy.
# Time: O(V + E).  Space: O(V).
def is_deep_copy(original, copy):
    mapping = {}
    stack = [(original, copy)]
    while stack:
        a, b = stack.pop()
        if a is b:
            return False
        if a in mapping:
            if mapping[a] is not b:
                return False
            continue
        mapping[a] = b
        if a.val != b.val or len(a.neighbors) != len(b.neighbors):
            return False
        for x, y in zip(a.neighbors, b.neighbors):
            stack.append((x, y))
    return True
```

PATTERN 16 · GRAPH DFS · PROBLEM 5 OF 20

417. Pacific Atlantic Water Flow

Medium · Depth-First Search, Breadth-First Search, Array, Matrix

leetcode.com/problems/pacific-atlantic-water-flow

★ **TOP 7 PROBLEM** in Graph DFS: master this one first

Also appears in pattern 18 Matrix / Grid Traversal

PROBLEM

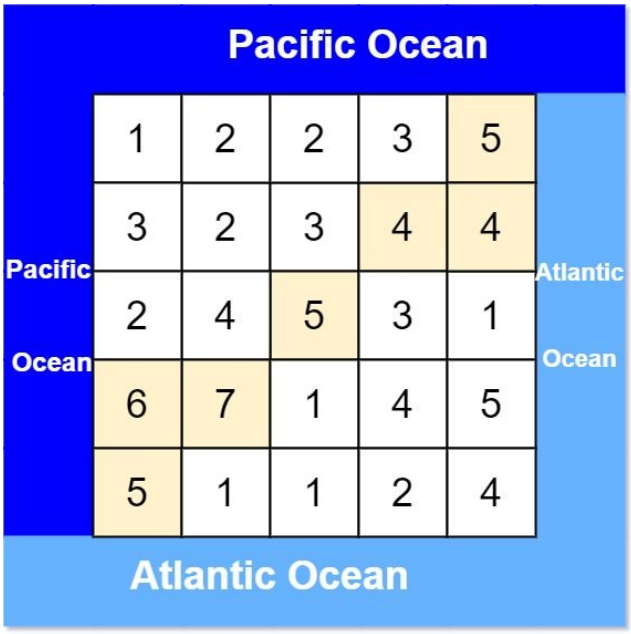
There is an `m x n` rectangular island that borders both the **Pacific Ocean** and **Atlantic Ocean**. The **Pacific Ocean** touches the island's left and top edges, and the **Atlantic Ocean** touches the island's right and bottom edges.

The island is partitioned into a grid of square cells. You are given an `m x n` integer matrix `heights` where `heights[r][c]` represents the **height above sea level** of the cell at coordinate `(r, c)`.

The island receives a lot of rain, and the rain water can flow to neighboring cells directly north, south, east, and west if the neighboring cell's height is **less than or equal to** the current cell's height. Water can flow from any cell adjacent to an ocean into the ocean.

Return *a **2D list** of grid coordinates* `result` *where* `result[i] = [r_i, c_i]` *denotes that rain water can flow from cell* `(r_i, c_i)` *to **both** the Pacific and Atlantic oceans*.

Example 1:



```
Input: heights = [[1,2,2,3,5],[3,2,3,4,4],[2,4,5,3,1],[6,7,1,4,5],[5,1,1,2,4]]
Output: [[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]]
Explanation: The following cells can flow to the Pacific and Atlantic oceans, as shown below:
[0,4]: [0,4] -> Pacific Ocean
       [0,4] -> Atlantic Ocean
[1,3]: [1,3] -> [0,3] -> Pacific Ocean
       [1,3] -> [1,4] -> Atlantic Ocean
[1,4]: [1,4] -> [1,3] -> [0,3] -> Pacific Ocean
       [1,4] -> Atlantic Ocean
[2,2]: [2,2] -> [1,2] -> [0,2] -> Pacific Ocean
       [2,2] -> [2,3] -> [2,4] -> Atlantic Ocean
[3,0]: [3,0] -> Pacific Ocean
       [3,0] -> [4,0] -> Atlantic Ocean
[3,1]: [3,1] -> [3,0] -> Pacific Ocean
       [3,1] -> [4,1] -> Atlantic Ocean
[4,0]: [4,0] -> Pacific Ocean
       [4,0] -> Atlantic Ocean
Note that there are other possible paths for these cells to flow to the Pacific and Atlantic oceans.
```

Example 2:

```
Input: heights = [[1]]
Output: [[0,0]]
Explanation: The water can flow from the only cell to the Pacific and Atlantic oceans.
```

Constraints:

- `m == heights.length`
- `n == heights[r].length`
- `1 <= m, n <= 200`
- `0 <= heights[r][c] <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: BFS

How to think about it

Walking downhill from every cell toward an ocean revisits the same cells. With m,n≤ 200 that duplication is costly. The reverse of flowing down is climbing to a neighbor that is at least as high. BFS from the Pacific border and from the Atlantic border; the intersection can reach both oceans. A neighbor is admissible only when heights[nx][ny]≥ heights[x][y]. Starting at a border guarantees a path back to that ocean.

We can start from the boundaries of the Pacific and Atlantic oceans and perform breadth-first search (BFS) respectively to find all cells that can flow to the Pacific and Atlantic oceans. Finally, we take the intersection of the two results, which represents cells that can flow to both the Pacific and Atlantic oceans.

Specifically, we define a queue q_1 to store all cells adjacent to the Pacific ocean, and define a boolean matrix vis_1 to record which cells can flow to the Pacific ocean. Similarly, we define queue q_2 and boolean matrix vis_2 to handle the Atlantic ocean. Initially, we add all cells adjacent to the Pacific ocean to queue q_1 and mark them as visited in vis_1. Similarly, we add all cells adjacent to the Atlantic ocean to queue q_2 and mark them as visited in vis_2.

Then, we perform BFS on q_1 and q_2 respectively. In each BFS, we dequeue a cell (x, y) from the queue and check its four adjacent cells (nx, ny). If an adjacent cell is within the matrix bounds, has not been visited, and its height is not less than the current cell's height (i.e., water can flow to that cell), we add it to the queue and mark it as visited.

Finally, we traverse the entire matrix to find cells that are marked as visited in both vis_1 and vis_2. These cells are our answer.

The time complexity is $O(m \times n)$ and the space complexity is $O(m \times n)$, where m and n are the number of rows and columns in the matrix, respectively.

Python 3

```python
class Solution:
    def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
        def bfs(q: Deque[Tuple[int, int]], vis: List[List[bool]]) -> None:
            while q:
                x, y = q.popleft()
                for dx, dy in pairwise(dirs):
                    nx, ny = x + dx, y + dy
                    if (
                        0 <= nx < m
                        and 0 <= ny < n
                        and not vis[nx][ny]
                        and heights[nx][ny] >= heights[x][y]
                    ):
                        vis[nx][ny] = True
                        q.append((nx, ny))

        m, n = len(heights), len(heights[0])
        vis1 = [[False] * n for _ in range(m)]
        vis2 = [[False] * n for _ in range(m)]
        q1: Deque[Tuple[int, int]] = deque()
        q2: Deque[Tuple[int, int]] = deque()
        dirs = (-1, 0, 1, 0, -1)

        for i in range(m):
            q1.append((i, 0))
            vis1[i][0] = True
            q2.append((i, n - 1))
            vis2[i][n - 1] = True

        for j in range(n):
            q1.append((0, j))
            vis1[0][j] = True
            q2.append((m - 1, j))
            vis2[m - 1][j] = True

        bfs(q1, vis1)
        bfs(q2, vis2)

        return [[i, j] for i in range(m) for j in range(n) if vis1[i][j] and vis2[i][j]]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Cells from which water reaches ONLY the Pacific
# 2. Water must be able to reach the ocean through cells
# that are strictly lower, never level


# ============================================================
# FOLLOW-UP 1: Cells from which water reaches ONLY the
# Pacific
# ============================================================

# ---- follow-up 1: helper functions ----
def reachable_from(heights, starts):
    rows, cols = len(heights), len(heights[0])
    seen = set(starts)
    stack = list(starts)
    while stack:
        r, c = stack.pop()
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and (nr, nc) not in seen and heights[nr][nc] >=
heights[r][c]:
                seen.add((nr, nc))
                stack.append((nr, nc))
```

```
    return seen

# ---- follow-up 1: function ----
# Answer: Compute the cells that reach each ocean with a
# search that climbs uphill from the border, then keep the
# Pacific-only ones.
# Time: O(m * n).  Space: O(m * n).
def pacific_only(heights):
    rows, cols = len(heights), len(heights[0])
    pacific = []
    atlantic = []
    for r in range(rows):
        pacific.append((r, 0))
        atlantic.append((r, cols - 1))
    for c in range(cols):
        pacific.append((0, c))
        atlantic.append((rows - 1, c))
    only = reachable_from(heights, pacific) - reachable_from(heights, atlantic)
    return sorted(only)


# ========================================================
# FOLLOW-UP 2: Water must be able to reach the ocean
# through cells that are strictly lower, never level
# ========================================================

# ---- follow-up 2: function ----
# Answer: The same uphill search with a strict comparison.
# Time: O(m * n).  Space: O(m * n).
def strict_pacific_atlantic(heights):
    rows, cols = len(heights), len(heights[0])

    def search(starts):
        seen = set(starts)
        stack = list(starts)
        while stack:
            r, c = stack.pop()
            for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
```

```
                if 0 <= nr < rows and 0 <= nc < cols and (nr, nc) not in seen and heights[nr][nc] >
heights[r][c]:
                    seen.add((nr, nc))
                    stack.append((nr, nc))
        return seen

    pacific = [(r, 0) for r in range(rows)] + [(0, c) for c in range(cols)]
    atlantic = [(r, cols - 1) for r in range(rows)] + [(rows - 1, c) for c in range(cols)]
    return sorted(search(pacific) & search(atlantic))
```

PATTERN 16 · GRAPH DFS · PROBLEM 6 OF 20

130. Surrounded Regions

Medium · Depth-First Search, Breadth-First Search, Union Find, Array, Matrix

leetcode.com/problems/surrounded-regions

★ **TOP 7 PROBLEM** in Graph DFS: master this one first

Also appears in pattern 18 Matrix / Grid Traversal

PROBLEM

You are given an `m x n` matrix `board` containing **letters** `'X'` and `'O'`, **capture regions** that are **surrounded**:

- **Connect**: A cell is connected to adjacent cells horizontally or vertically.
- **Region**: To form a region **connect every** `'O'` cell.
- **Surround**: A region is surrounded if none of the `'O'` cells in that region are on the edge of the board. Such regions are **completely enclosed** by `'X'` cells.

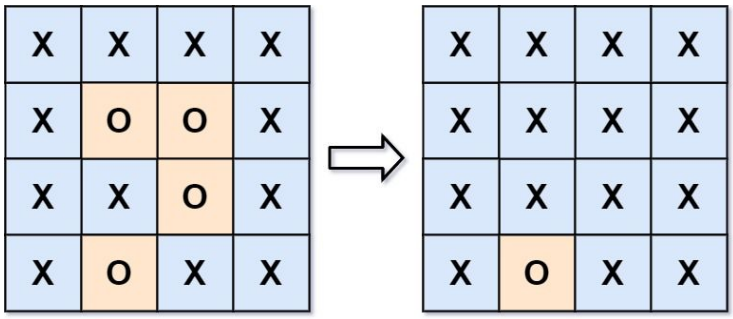
To capture a **surrounded region**, replace all `'O'`s with `'X'`s **in-place** within the original board. You do not need to return anything.

Example 1:

Input: board = [["X","X","X","X"],["X","O","O","X"],["X","X","O","X"],["X","O","X","X"]]

Output: [["X","X","X","X"],["X","X","X","X"],["X","X","X","X"],["X","O","X","X"]]

Explanation:



In the above diagram, the bottom region is not captured because it is on the edge of the board and cannot be surrounded.

Example 2:

Input: board = [["X"]]

Output: [["X"]]

Constraints:

- `m == board.length`
- `n == board[i].length`
- `1 <= m, n <= 200`
- `board[i][j]` is `'X'` or `'O'`.

SOLUTIONS (PYTHON 3)

Solution 1: Depth-First Search (DFS)

> **How to think about it**
>
> Surrounded `'O'`s become `'X'`; those connected to the border stay. m,n≤ 200. Starting from interior cells to ask “am I surrounded?” must flood a whole component and is easy to get wrong on the border. Mark the unsurrounded cells instead. DFS from every border `'O'`, paint those cells with a placeholder, flip remaining `'O'`s to `'X'`, then restore the placeholders.

We can start from the boundary of the matrix, taking each 'O' on the matrix boundary as a starting point, and perform depth-first search. All 'O's found in the search are replaced with '.'.

Then we traverse the matrix again, for each position:

- If it is '.', replace it with 'O';
- Otherwise, if it is 'O', replace it with 'X'.

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the number of rows and columns in the matrix, respectively.

Python 3

```
class Solution:
    def solve(self, board: List[List[str]]) -> None:
        def dfs(i: int, j: int):
            if not (0 <= i < m and 0 <= j < n and board[i][j] == "O"):
                return
            board[i][j] = "."
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                dfs(i + a, j + b)

        m, n = len(board), len(board[0])
        for i in range(m):
            dfs(i, 0)
            dfs(i, n - 1)
        for j in range(n):
            dfs(0, j)
            dfs(m - 1, j)
        for i in range(m):
            for j in range(n):
                if board[i][j] == ".":
                    board[i][j] = "O"
                elif board[i][j] == "O":
                    board[i][j] = "X"
```

Solution 2: Union-Find Set

> **How to think about it**
>
> Solution 1 uses DFS paint to mark border-connected cells. Union-find links adjacent `'O'`s and hooks every border `'O'` to a dummy node. Any `'O'` not in that component is surrounded. No recursion stack, and the connectivity is explicit.

We can also use a union-find set, connecting each 'O' on the matrix boundary with a super node m × n, and connecting each 'O' in the matrix with the 'O's above, below, left, and right of it.

Then we traverse this matrix, for each position, if it is 'O' and it is not connected to the super node, then we replace it with 'X'.

The time complexity is O(m × n × α(m × n)), and the space complexity is O(m × n). Here, m and n are the number of rows and columns in the matrix, respectively, and α is the inverse Ackermann function.

Python 3

```
class Solution:
    def solve(self, board: List[List[str]]) -> None:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        m, n = len(board), len(board[0])
        p = list(range(m * n + 1))
        for i in range(m):
            for j in range(n):
                if board[i][j] == "O":
                    if i in (0, m - 1) or j in (0, n - 1):
                        p[find(i * n + j)] = find(m * n)
                    else:
                        for a, b in pairwise((-1, 0, 1, 0, -1)):
                            x, y = i + a, j + b
                            if board[x][y] == "O":
                                p[find(x * n + y)] = find(i * n + j)
        for i in range(m):
            for j in range(n):
                if board[i][j] == "O" and find(i * n + j) != find(m * n):
                    board[i][j] = "X"
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Count how many regions of 'O' are surrounded
# 2. Solve it with union-find and a dummy 'border' node


# ============================================================
# FOLLOW-UP 1: Count how many regions of 'O' are
# surrounded
# ============================================================

# ---- follow-up 1: function ----
# Answer: Mark the 'O' cells connected to the border, then
# count the connected groups of the other 'O' cells.
# Time: O(m * n).  Space: O(m * n).
def count_surrounded_regions(board):
    rows, cols = len(board), len(board[0])
    seen = set()

    def spread(r, c):
        stack = [(r, c)]
        seen.add((r, c))
        while stack:
            x, y = stack.pop()
            for nx, ny in ((x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1)):
                if 0 <= nx < rows and 0 <= ny < cols and board[nx][ny] == "O" and (nx, ny) not in seen:
                    seen.add((nx, ny))
                    stack.append((nx, ny))

    for r in range(rows):
        for c in range(cols):
            if (r == 0 or c == 0 or r == rows - 1 or c == cols - 1) and board[r][c] == "O" and (r, c) not
in seen:
                spread(r, c)
    count = 0
    for r in range(rows):
        for c in range(cols):
            if board[r][c] == "O" and (r, c) not in seen:
                count += 1
                spread(r, c)
```

```
    return count


# ============================================================
# FOLLOW-UP 2: Solve it with union-find and a dummy
# 'border' node
# ============================================================

# ---- follow-up 2: function ----
# Answer: Join every 'O' with its neighbours and with a
# dummy node when it is on the border; the cells not
# joined to the dummy are surrounded.
# Time: O(m * n * alpha), essentially O(m * n).  Space:
# O(m * n).
def solve_with_union_find(board):
    rows, cols = len(board), len(board[0])
    parent = list(range(rows * cols + 1))
    border = rows * cols

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
```

```
            x = parent[x]
        return x

    def union(a, b):
        parent[find(a)] = find(b)

    for r in range(rows):
        for c in range(cols):
            if board[r][c] != "O":
                continue
            index = r * cols + c
            if r == 0 or c == 0 or r == rows - 1 or c == cols - 1:
                union(index, border)
            if r + 1 < rows and board[r + 1][c] == "O":
                union(index, index + cols)
            if c + 1 < cols and board[r][c + 1] == "O":
                union(index, index + 1)
    for r in range(rows):
        for c in range(cols):
            if board[r][c] == "O" and find(r * cols + c) != find(border):
                board[r][c] = "X"
    return board
```

PATTERN 16 · GRAPH DFS · PROBLEM 7 OF 20

207. Course Schedule

Medium · Depth-First Search, Breadth-First Search, Graph, Topological Sort, Directed Acyclic Graph

leetcode.com/problems/course-schedule

★ **TOP 7 PROBLEM** in Graph DFS, Topological Sort: master this one first

Also appears in pattern 30 Topological Sort

PROBLEM

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [a_i, b_i]` indicates that you **must** take course b_i first if you want to take course a_i.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:

```
Input: numCourses = 2, prerequisites = [[1,0]]
Output: true
Explanation: There are a total of 2 courses to take.
To take course 1 you should have finished course 0. So it is possible.
```

Example 2:

```
Input: numCourses = 2, prerequisites = [[1,0],[0,1]]
Output: false
Explanation: There are a total of 2 courses to take.
To take course 1 you should have finished course 0, and to take course 0 you should also have finished
course 1. So it is impossible.
```

Constraints:

- `1 <= numCourses <= 2000`
- `0 <= prerequisites.length <= 5000`
- `prerequisites[i].length == 2`
- $0 <= a_i, b_i < numCourses$
- All the pairs prerequisites[i] are **unique**.

SOLUTION (PYTHON 3)

Solution 1: Topological Sorting

How to think about it

Prerequisites form a directed graph; finishing every course is possible iff the graph is acyclic. Enumerating orders is not viable at the given size. Kahn's algorithm repeatedly takes a vertex of indegree 0 and decrements its successors. If every course enters the queue, there is no cycle.

For this problem, we can consider the courses as nodes in a graph, and prerequisites as edges in the graph. Thus, we can transform this problem into determining whether there is a cycle in the directed graph.

Specifically, we can use the idea of topological sorting. For each node with an in-degree of 0, we reduce the in-degree of its out-degree nodes by 1, until all nodes have been traversed.

If all nodes have been traversed, it means there is no cycle in the graph, and we can complete all courses; otherwise, we cannot complete all courses.

The time complexity is O(n + m), and the space complexity is O(n + m). Here, n and m are the number of courses and prerequisites respectively.

Python 3

```python
class Solution:
    def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
        g = [[] for _ in range(numCourses)]
        indeg = [0] * numCourses
        for a, b in prerequisites:
            g[b].append(a)
            indeg[a] += 1
        q = [i for i, x in enumerate(indeg) if x == 0]
        for i in q:
            numCourses -= 1
            for j in g[i]:
                indeg[j] -= 1
                if indeg[j] == 0:
                    q.append(j)
        return numCourses == 0
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return one cycle if the courses cannot be finished
# 2. A valid order of the courses with a depth-first
# search


# ============================================================
# FOLLOW-UP 1: Return one cycle if the courses cannot be
# finished
# ============================================================

# ---- follow-up 1: function ----
# Answer: Depth-first search with three colours: white
# (not visited), grey (on the current path) and black
# (finished). Meeting a grey course closes a cycle; the
# path from it to the current course is the cycle.
# Time: O(V + E).  Space: O(V + E).
def find_course_cycle(num_courses, prerequisites):
    after = [[] for i in range(num_courses)]
    for course, before in prerequisites:
        after[before].append(course)
    colour = [0] * num_courses                        # 0 white, 1 grey, 2 black
    path = []

    def visit(course):
        colour[course] = 1
        path.append(course)
        for nxt in after[course]:
            if colour[nxt] == 1:
                return path[path.index(nxt):] + [nxt]
            if colour[nxt] == 0:
                cycle = visit(nxt)
                if cycle:
                    return cycle
        path.pop()
        colour[course] = 2
        return None

    for course in range(num_courses):
        if colour[course] == 0:
            cycle = visit(course)
            if cycle:
                return cycle
    return None
```

```
# ============================================================
# FOLLOW-UP 2: A valid order of the courses with a depth-
# first search
# ============================================================

# ---- follow-up 2: function ----
# Answer: Finish a course after all the courses that
# depend on it were finished; the reversed finishing order
# is a valid course order (the graph must have no cycle).
# Time: O(V + E).  Space: O(V + E).
def course_order_dfs(num_courses, prerequisites):
    after = [[] for i in range(num_courses)]
    for course, before in prerequisites:
        after[before].append(course)
```

```
    colour = [0] * num_courses
    finished = []

    def visit(course):
        colour[course] = 1
        for nxt in after[course]:
            if colour[nxt] == 1:
                return False
            if colour[nxt] == 0 and not visit(nxt):
                return False
        colour[course] = 2
        finished.append(course)
        return True

    for course in range(num_courses):
        if colour[course] == 0 and not visit(course):
            return []
    finished.reverse()
    return finished
```

PATTERN 16 · GRAPH DFS · PROBLEM 8 OF 20

547. Number of Provinces

Medium · Depth-First Search, Breadth-First Search, Union Find, Graph

leetcode.com/problems/number-of-provinces

★ **TOP 7 PROBLEM** in Union-Find (Disjoint Set): master this one first

Also appears in pattern 31 Union-Find (Disjoint Set)

PROBLEM

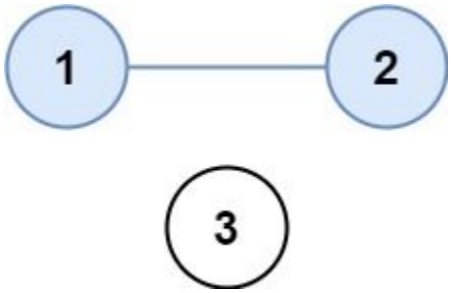
There are `n` cities. Some of them are connected, while some are not. If city `a` is connected directly with city `b`, and city `b` is connected directly with city `c`, then city `a` is connected indirectly with city `c`.

A **province** is a group of directly or indirectly connected cities and no other cities outside of the group.

You are given an `n x n` matrix `isConnected` where `isConnected[i][j] = 1` if the i^{th} city and the j^{th} city are directly connected, and `isConnected[i][j] = 0` otherwise.

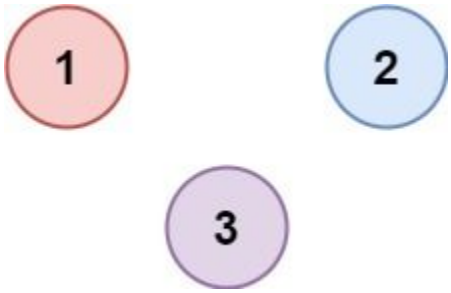
Return *the total number of **provinces***.

Example 1:




```
Input: isConnected = [[1,1,0],[1,1,0],[0,0,1]]
Output: 2
```

Example 2:




```
Input: isConnected = [[1,0,0],[0,1,0],[0,0,1]]
Output: 3
```

Constraints:

- `1 <= n <= 200`
- `n == isConnected.length`
- `n == isConnected[i].length`
- `isConnected[i][j]` is `1` or `0`.
- `isConnected[i][i] == 1`
- `isConnected[i][j] == isConnected[j][i]`

SOLUTIONS (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> A province is a connected component. The matrix tells which cities share an edge, so DFS or BFS from each unseen city paints one component. Scan cities, start a DFS at every unvisited index, and increment the answer. The visited array visits each city once.

We create an array vis to record whether each city has been visited.

Next, we traverse each city i. If the city has not been visited, we start a depth-first search from that city. Using the matrix isConnected, we find the cities directly connected to this city. These cities and the current city belong to the same province. We continue the depth-first search for these cities until all cities in the same province have been visited. This counts as one province, so we increment the answer ans by 1. Then, we move to the next unvisited city and repeat the process until all cities have been traversed.

Finally, return the answer.

The time complexity is $O(n^2)$, and the space complexity is $O(n)$. Here, n is the number of cities.

Python 3

```python
class Solution:
    def findCircleNum(self, isConnected: List[List[int]]) -> int:
        def dfs(i: int):
            vis[i] = True
            for j, x in enumerate(isConnected[i]):
                if not vis[j] and x:
                    dfs(j)

        n = len(isConnected)
        vis = [False] * n
        ans = 0
        for i in range(n):
            if not vis[i]:
                dfs(i)
                ans += 1
        return ans
```

Solution 2: Union-Find

> **How to think about it**
>
> DFS needs a recursion stack and a visited array. Union-find merges along edges: start with n components and decrement when an edge joins two roots. Iterate the upper triangle to avoid duplicate edges. Path compression keeps finds cheap. The remaining roots are the provinces.

We can also use the union-find data structure to maintain each connected component. Initially, each city belongs to a different connected component, so the number of provinces is n.

Next, we traverse the matrix isConnected. If there is a connection between two cities (i, j) and they belong to two different connected components, they will be merged into one connected component, and the number of provinces is decremented by 1.

Finally, return the number of provinces.

The time complexity is $O(n^2 \times \log n)$, and the space complexity is $O(n)$. Here, n is the number of cities, and $\log n$ is the time complexity of path compression in the union-find data structure.

Python 3

```
class Solution:
    def findCircleNum(self, isConnected: List[List[int]]) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(isConnected)
        p = list(range(n))
        ans = n
        for i in range(n):
            for j in range(i + 1, n):
                if isConnected[i][j]:
                    pa, pb = find(i), find(j)
                    if pa != pb:
                        p[pa] = pb
                        ans -= 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Solve it with union-find
# 2. The size of every province


# ============================================================
# FOLLOW-UP 1: Solve it with union-find
# ============================================================

# ---- follow-up 1: function ----
# Answer: Join the two cities of every connection; the
# number of provinces is the number of different leaders.
# Time: O(n^2 * alpha), essentially O(n^2).  Space: O(n).
def provinces_union_find(is_connected):
    n = len(is_connected)
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for i in range(n):
        for j in range(i + 1, n):
            if is_connected[i][j] == 1:
                parent[find(i)] = find(j)
    leaders = set()
    for i in range(n):
        leaders.add(find(i))
    return len(leaders)


# ============================================================
# FOLLOW-UP 2: The size of every province
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count how many cities have each leader.
# Time: O(n^2).  Space: O(n).
def province_sizes(is_connected):
    n = len(is_connected)
    seen = [False] * n
    sizes = []
```

```
    for start in range(n):
        if seen[start]:
            continue
        size = 0
        stack = [start]
        seen[start] = True
        while stack:
            city = stack.pop()
            size += 1
            for other in range(n):
                if is_connected[city][other] == 1 and not seen[other]:
                    seen[other] = True
                    stack.append(other)
        sizes.append(size)
    return sorted(sizes, reverse=True)
```

PATTERN 16 · GRAPH DFS · PROBLEM 9 OF 20

1020. Number of Enclaves

Medium · Depth-First Search, Breadth-First Search, Union Find, Array, Matrix

leetcode.com/problems/number-of-enclaves

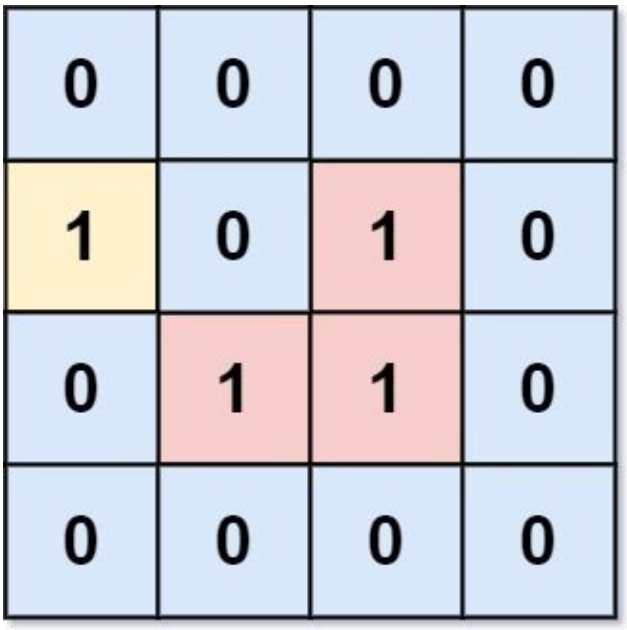
PROBLEM

You are given an `m x n` binary matrix `grid`, where `0` represents a sea cell and `1` represents a land cell.

A **move** consists of walking from one land cell to another adjacent (**4-directionally**) land cell or walking off the boundary of the `grid`.

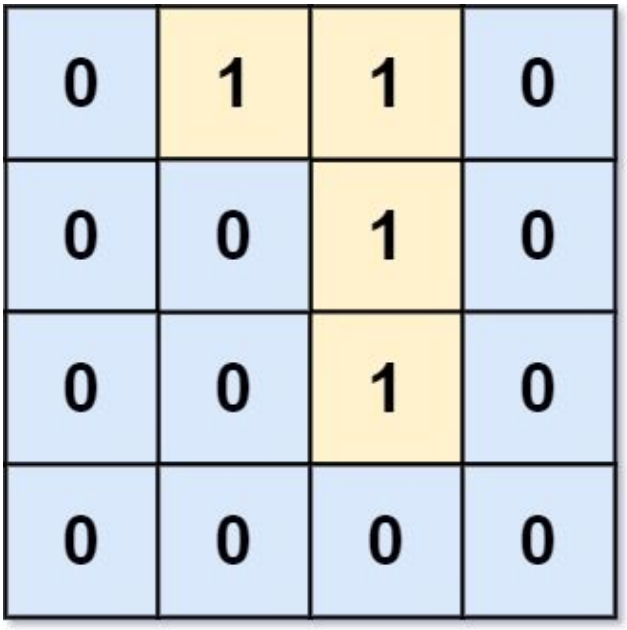
Return *the number of land cells in* `grid` *for which we cannot walk off the boundary of the grid in any number of* ***moves***.

Example 1:




```
Input: grid = [[0,0,0,0],[1,0,1,0],[0,1,1,0],[0,0,0,0]]
Output: 3
Explanation: There are three 1s that are enclosed by 0s, and one 1 that is not enclosed because its on the boundary.
```

Example 2:




```
Input: grid = [[0,1,1,0],[0,0,1,0],[0,0,1,0],[0,0,0,0]]
Output: 0
Explanation: All 1s are either on the boundary or can reach the boundary.
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 500`
- `grid[i][j]` is either `0` or `1`.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Testing whether each land cell can reach the border repeats searches over the same grid; m,n≤ 500 makes that wasteful. Enclaves are land that cannot reach the border, so they are the complement of border-connected land. Flooding from every 1 on the four borders removes all land that can leave. The remaining ones cannot. DFS performs the flood; the answer is the number of 1s left.

Python 3

```python
class Solution:
    def numEnclaves(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int):
            grid[i][j] = 0
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and grid[x][y]:
                    dfs(x, y)

        m, n = len(grid), len(grid[0])
        dirs = (-1, 0, 1, 0, -1)
        for j in range(n):
            if grid[0][j]:
                dfs(0, j)
            if grid[m - 1][j]:
                dfs(m - 1, j)
        for i in range(m):
            if grid[i][0]:
                dfs(i, 0)
            if grid[i][n - 1]:
                dfs(i, n - 1)
        return sum(sum(row) for row in grid)
```

Complexity. Time: O(m * n). Space: O(m * n).

Solution 2

How to think about it

Solution 1 already floods from the border with an explicit stack. Connectivity does not depend on search order, so a queue can clear the same cells by layers. Pop a cell and mark each neighboring 1. After the borders are processed, the remaining 1s are still the enclaves.

Python 3

```python
class Solution:
    def numEnclaves(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        q = deque()
        for i in range(m):
            for j in range(n):
                if grid[i][j] and (i == 0 or i == m - 1 or j == 0 or j == n - 1):
                    q.append((i, j))
                    grid[i][j] = 0
        dirs = (-1, 0, 1, 0, -1)
        while q:
            i, j = q.popleft()
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if x >= 0 and x < m and y >= 0 and y < n and grid[x][y]:
                    q.append((x, y))
                    grid[x][y] = 0
        return sum(v for row in grid for v in row)
```

Complexity. Time: O(m * n). Space: O(m * n).

Solution 3

How to think about it

Search still uses a stack or queue proportional to the component size. The same connectivity can be recorded with union-find: adjacent land is united, and every border land is united with a dummy exterior node. Cells that stay land and are not in the exterior component are enclaves, so we count those sizes.

Python 3

```
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa != pb:
            if self.size[pa] > self.size[pb]:
                self.p[pb] = pa
                self.size[pa] += self.size[pb]
            else:
                self.p[pa] = pb
                self.size[pb] += self.size[pa]


class Solution:
    def numEnclaves(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        uf = UnionFind(m * n + 1)
        dirs = (-1, 0, 1, 0, -1)
        for i, row in enumerate(grid):
            for j, v in enumerate(row):
                if v:
                    if i == 0 or i == m - 1 or j == 0 or j == n - 1:
                        uf.union(i * n + j, m * n)
                    else:
                        for a, b in pairwise(dirs):
                            x, y = i + a, j + b
                            if x >= 0 and x < m and y >= 0 and y < n and grid[x][y]:
                                uf.union(i * n + j, x * n + y)
        return sum(
            grid[i][j] == 1 and uf.find(i * n + j) != uf.find(m * n)
            for i in range(m)
            for j in range(n)
        )
```

Complexity. Time: O(m * n * alpha(m * n)), almost O(m * n). Space: O(m * n).

Solution 4: Explicit Stack

> **How to think about it**
>
> An enclave is land that cannot reach the border. Testing every land cell separately repeats the same search. With m,n≤ 500, the complement is the land connected to the border. Recursing along a border-connected snake of about a thousand cells exhausts the call stack before the flood finishes. The flood only needs to mark reachable cells as 0. It does not wait on a return value. An explicit stack holds the cells still to clear. Pop a cell and push each neighboring 1 after marking it. After all four borders are processed, the remaining 1s are enclaves.

Flood from every land cell on the four borders with an explicit stack, marking reachable cells as 0. The number of 1s left is the answer.

Python 3

```
class Solution:
    def numEnclaves(self, grid: List[List[int]]) -> int:
        def flood(i: int, j: int):
            stk = [(i, j)]
            grid[i][j] = 0
            while stk:
                i, j = stk.pop()
                for a, b in pairwise(dirs):
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n and grid[x][y]:
                        grid[x][y] = 0
                        stk.append((x, y))

        m, n = len(grid), len(grid[0])
        dirs = (-1, 0, 1, 0, -1)
        for j in range(n):
            for i in (0, m - 1):
                if grid[i][j]:
                    flood(i, j)
        for i in range(m):
            for j in (0, n - 1):
                if grid[i][j]:
                    flood(i, j)
        return sum(sum(row) for row in grid)
```

Complexity. Time: O(m * n). Space: O(m * n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The size of every enclave


# ============================================================
# FOLLOW-UP 1: The size of every enclave
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remove the land connected to the border, then
# measure each remaining group of land.
# Time: O(m * n).  Space: O(m * n).
def enclave_sizes(grid):
    rows, cols = len(grid), len(grid[0])
    seen = set()

    def spread(r, c):
        size = 0
        stack = [(r, c)]
        seen.add((r, c))
        while stack:
            x, y = stack.pop()
            size += 1
            for nx, ny in ((x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1)):
                if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 1 and (nx, ny) not in seen:
                    seen.add((nx, ny))
                    stack.append((nx, ny))
        return size

    for r in range(rows):
        for c in range(cols):
            if (r == 0 or c == 0 or r == rows - 1 or c == cols - 1) and grid[r][c] == 1 and (r, c) not in
seen:
                spread(r, c)
    sizes = []
    for r in range(rows):
        for c in range(cols):
```

```
            if grid[r][c] == 1 and (r, c) not in seen:
                sizes.append(spread(r, c))
    return sorted(sizes, reverse=True)
```

841. Keys and Rooms

Medium · Depth-First Search, Breadth-First Search, Graph

leetcode.com/problems/keys-and-rooms

PROBLEM

There are `n` rooms labeled from `0` to `n - 1` and all the rooms are locked except for room `0`. Your goal is to visit all the rooms. However, you cannot enter a locked room without having its key.

When you visit a room, you may find a set of **distinct keys** in it. Each key has a number on it, denoting which room it unlocks, and you can take all of them with you to unlock the other rooms.

Given an array `rooms` where `rooms[i]` is the set of keys that you can obtain if you visited room `i`, return `true` *if you can visit **all** the rooms, or* `false` *otherwise*.

Example 1:

```
Input: rooms = [[1],[2],[3],[]]
Output: true
Explanation:
We visit room 0 and pick up key 1.
We then visit room 1 and pick up key 2.
We then visit room 2 and pick up key 3.
We then visit room 3.
Since we were able to visit every room, we return true.
```

Example 2:

```
Input: rooms = [[1,3],[3,0,1],[2],[0]]
Output: false
Explanation: We can not enter room number 2 since the only key that unlocks it is in that room.
```

Constraints:

- `n == rooms.length`
- `2 <= n <= 1000`
- `0 <= rooms[i].length <= 1000`
- `1 <= sum(rooms[i].length) <= 3000`
- `0 <= rooms[i][j] < n`
- All the values of `rooms[i]` are **unique**.

SOLUTIONS (PYTHON 3)

Solution 1: Depth-First Search (DFS)

> **How to think about it**
>
> Rooms form a directed graph whose edges are keys. We ask whether 0 reaches every room. n≤ 1000, so one traversal is enough. DFS from 0 follows keys; success is |vis|=n.

We can use the Depth-First Search (DFS) method to traverse the entire graph, count the number of reachable nodes, and use an array `vis` to mark whether the current node has been visited to prevent repeated visits.

Finally, we count the number of visited nodes. If it is the same as the total number of nodes, it means that all nodes can be visited; otherwise, there are nodes that cannot be reached.

The time complexity is O(n + m), and the space complexity is O(n), where n is the number of nodes, and m is the number of edges.

Python 3

```python
class Solution:
    def canVisitAllRooms(self, rooms: List[List[int]]) -> bool:
        def dfs(i: int):
            if i in vis:
                return
            vis.add(i)
            for j in rooms[i]:
                dfs(j)

        vis = set()
        dfs(0)
        return len(vis) == len(rooms)
```

Solution 2: BFS

> **How to think about it**
>
> Solution 1 follows keys with an explicit stack. The same reachability is a queue: start at 0 and enqueue a room when a key to it is found and that room is still unvisited. The visited marks still drop duplicate keys, and the answer still compares the visited count with n.

We can also use the Breadth-First Search (BFS) method to traverse the entire graph. We use a hash table or an array `vis` to mark whether the current node has been visited to prevent repeated visits.

Specifically, we define a queue q, initially put node 0 into the queue, and then continuously traverse the queue. Each time we take out the front node i of the queue, if i has been visited, we skip it directly; otherwise, we mark it as visited, and then add the nodes that i can reach to the queue.

Finally, we count the number of visited nodes. If it is the same as the total number of nodes, it means that all nodes can be visited; otherwise, it means that there are unreachable nodes.

The time complexity is O(n + m), and the space complexity is O(n). Where n is the number of nodes, and m is the number of edges.

Python 3

```python
class Solution:
    def canVisitAllRooms(self, rooms: List[List[int]]) -> bool:
        vis = set()
        q = deque([0])
        while q:
            i = q.popleft()
            if i in vis:
                continue
            vis.add(i)
            q.extend(j for j in rooms[i])
        return len(vis) == len(rooms)
```

Solution 3: Explicit Stack

> **How to think about it**
>
> Rooms form a directed graph whose edges are keys. The question is whether room 0 reaches every room. With n ≤ 1000, one traversal is enough. Following a chain of keys by recursion uses a call depth equal to the room count and overflows Python once the chain reaches length 1000. A stack therefore holds the rooms still to open: a popped room is skipped when it was already visited, and otherwise it is marked and its keys are pushed. Duplicate keys may push the same room again, and the mark expands it only once. Every room is reachable when the visited count equals n.

Start at room 0 and walk the reachable rooms with an explicit stack. The stack begins with 0. When room i is popped, skip it if it has already been visited; otherwise mark it and push the room behind each key found in i. A repeated key only pushes that room again, and the later pop is skipped.

Finally, check whether every room was visited. If so, return true; otherwise some room cannot be reached.

The time complexity is O(n + m), and the space complexity is O(n), where n is the number of nodes, and m is the number of edges.

Python 3

```python
class Solution:
    def canVisitAllRooms(self, rooms: List[List[int]]) -> bool:
        n = len(rooms)
        vis = [False] * n
        stk = [0]
        while stk:
            i = stk.pop()
            if vis[i]:
                continue
            vis[i] = True
            for j in rooms[i]:
                stk.append(j)
        return all(vis)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the rooms that can NOT be visited
# 2. The order in which a depth-first visit opens the
# rooms


# ============================================================
# FOLLOW-UP 1: Return the rooms that can NOT be visited
# ============================================================

# ---- follow-up 1: function ----
# Answer: Search from room 0 and report the rooms that
# were never reached.
# Time: O(V + E).  Space: O(V).
def unreachable_rooms(rooms):
    seen = {0}
    stack = [0]
    while stack:
        room = stack.pop()
        for key in rooms[room]:
            if key not in seen:
                seen.add(key)
                stack.append(key)
    result = []
    for room in range(len(rooms)):
        if room not in seen:
            result.append(room)
    return result


# ============================================================
# FOLLOW-UP 2: The order in which a depth-first visit
# opens the rooms
# ============================================================

# ---- follow-up 2: function ----
# Answer: Record each room when it is first entered.
# Time: O(V + E).  Space: O(V).
def visit_order(rooms):
    order = []
    seen = set()

    def visit(room):
        seen.add(room)
        order.append(room)
        for key in rooms[room]:
```

```
            if key not in seen:
                visit(key)

    visit(0)
    return order
```

1466. Reorder Routes to Make All Paths Lead to the City Zero

Medium · Depth-First Search, Breadth-First Search, Graph

leetcode.com/problems/reorder-routes-to-make-all-paths-lead-to-the-city-zero

PROBLEM

There are `n` cities numbered from `0` to `n - 1` and `n - 1` roads such that there is only one way to travel between two different cities (this network form a tree). Last year, The ministry of transport decided to orient the roads in one direction because they are too narrow.

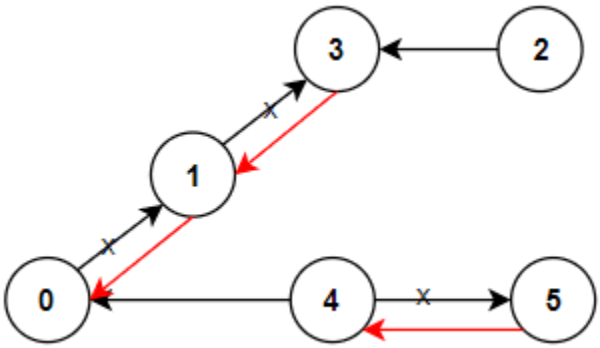
Roads are represented by `connections` where `connections[i] = [a_i, b_i]` represents a road from city `a_i` to city `b_i`.

This year, there will be a big event in the capital (city `0`), and many people want to travel to this city.

Your task consists of reorienting some roads such that each city can visit the city `0`. Return the **minimum** number of edges changed.

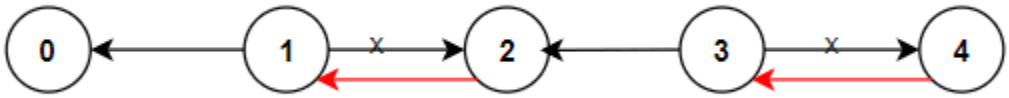
It's **guaranteed** that each city can reach city `0` after reorder.

Example 1:




```
Input: n = 6, connections = [[0,1],[1,3],[2,3],[4,0],[4,5]]
Output: 3
Explanation: Change the direction of edges show in red such that each node can reach the node 0 (capital).
```

Example 2:




```
Input: n = 5, connections = [[1,0],[1,2],[3,2],[3,4]]
Output: 2
Explanation: Change the direction of edges show in red such that each node can reach the node 0 (capital).
```

Example 3:

```
Input: n = 3, connections = [[1,0],[2,0]]
Output: 0
```

Constraints:

- `2 <= n <= 5 * 10^4`
- `connections.length == n - 1`

- `connections[i].length == 2`
- $0 <= a_i, b_i <= n - 1$
- $a_i != b_i$

SOLUTIONS (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> Ignoring direction, the graph is a tree; every city must reach 0. Walking outward from 0, an original forward edge must be reversed. Store cost 1 with the given direction and 0 with the reverse, then DFS the costs.

The route map given in the problem has n nodes and n-1 edges. If we ignore the direction of the edges, then these n nodes form a tree. The problem requires us to change the direction of some edges so that each node can reach node 0.

We might as well consider starting from node 0 and reaching all other nodes. The direction is opposite to the problem description, which means that when we build the graph, for the directed edge [a, b], we should regard it as the directed edge [b, a]. That is to say, if it is from a to b, we need to change the direction once; if it is from b to a, no direction change is needed.

Next, we only need to start from node 0, search all other nodes, and during the process, if we encounter an edge that needs to change direction, we accumulate the number of direction changes once.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the problem.

Python 3

```python
class Solution:
    def minReorder(self, n: int, connections: List[List[int]]) -> int:
        def dfs(a: int, fa: int) -> int:
            return sum(c + dfs(b, a) for b, c in g[a] if b != fa)

        g = [[] for _ in range(n)]
        for a, b in connections:
            g[a].append((b, 1))
            g[b].append((a, 0))
        return dfs(0, -1)
```

Solution 2: BFS

> **How to think about it**
>
> Method 1 already walks the same adjacency list with an explicit stack. BFS leaves 0 level by level and adds the edge cost the first time a neighbor is seen, so the visit order differs from the stack.

We can use the Breadth-First Search (BFS) method, starting from node 0, to search all other nodes. During the process, if we encounter an edge that requires a change of direction, we increment the count of direction changes.

The time complexity is O(n), and the space complexity is O(n). Where n is the number of nodes in the problem.

Python 3

```python
class Solution:
    def minReorder(self, n: int, connections: List[List[int]]) -> int:
        g = [[] for _ in range(n)]
        for a, b in connections:
            g[a].append((b, 1))
            g[b].append((a, 0))
        q = deque([0])
        vis = {0}
        ans = 0
        while q:
            a = q.popleft()
            for b, c in g[a]:
                if b not in vis:
                    vis.add(b)
                    q.append(b)
                    ans += c
        return ans
```

Solution 3: Explicit Stack

> **How to think about it**
>
> Ignoring direction, the graph is a tree. Walking outward from 0, an original forward edge must be reversed. n can reach 5×10^4. On a chain the first recursive call always follows the only child, so the depth is n and exceeds the default recursion limit. The cost of a tree edge is known as soon as the walk leaves its parent, so the total does not have to wait for the subtree to return. An explicit stack starts at 0. After a node is popped, each neighbor that is not the parent adds its edge cost and is pushed. A forward edge costs 1 and a reverse edge costs 0.

The route map given in the problem has n nodes and n-1 edges. If we ignore the direction of the edges, then these n nodes form a tree. The problem requires us to change the direction of some edges so that each node can reach node 0.

We might as well consider starting from node 0 and reaching all other nodes. The direction is opposite to the problem description, which means that when we build the graph, for the directed edge [a, b], we should regard it as the directed edge [b, a]. That is to say, if it is from a to b, we need to change the direction once; if it is from b to a, no direction change is needed.

An explicit stack walks the tree from node 0. After a node is popped, each neighbor that is not the parent adds its edge weight to the answer and is pushed. A forward edge has weight 1, and a reverse edge has weight 0.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the problem.

Python 3

```python
class Solution:
    def minReorder(self, n: int, connections: List[List[int]]) -> int:
        g = [[] for _ in range(n)]
        for a, b in connections:
            g[a].append((b, 1))
            g[b].append((a, 0))
        ans = 0
        stk = [(0, -1)]
        while stk:
            a, fa = stk.pop()
            for b, c in g[a]:
                if b != fa:
                    ans += c
                    stk.append((b, a))
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the roads that have to be reversed
# 2. For EVERY city as the capital: how many roads must be
# reversed?


# ============================================================
# FOLLOW-UP 1: Return the roads that have to be reversed
# ============================================================

# ---- follow-up 1: function ----
# Answer: Search from city 0 over the roads in both
# directions; a road that points away from city 0 (as
# given) must be reversed.
# Time: O(n).  Space: O(n).
def roads_to_reverse(n, connections):
    graph = [[] for i in range(n)]
    for a, b in connections:
        graph[a].append((b, True))                  # the road as given: a -> b (points away from a)
        graph[b].append((a, False))
    seen = {0}
    stack = [0]
    reverse = []
    while stack:
        city = stack.pop()
        for other, as_given in graph[city]:
            if other not in seen:
                seen.add(other)
                if as_given:
                    reverse.append([city, other])
                stack.append(other)
    return reverse


# ============================================================
# FOLLOW-UP 2: For EVERY city as the capital: how many
# roads must be reversed?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Compute the answer for city 0, then move the
# root along each road: moving the capital over a road
# that was pointing away from the old capital saves one
# reversal, otherwise it costs one.
# Time: O(n).  Space: O(n).
def reversals_for_every_capital(n, connections):
    graph = [[] for i in range(n)]
    for a, b in connections:
        graph[a].append((b, 1))                     # reversing costs 1 if we start at a
        graph[b].append((a, -1))
    answer = [0] * n
    seen = {0}
    stack = [0]
    while stack:
        city = stack.pop()
        for other, weight in graph[city]:
            if other not in seen:
                seen.add(other)
                if weight == 1:
                    answer[0] += 1
```

```
                stack.append(other)
    seen = {0}
    stack = [0]
    while stack:
```

```
        city = stack.pop()
        for other, weight in graph[city]:
            if other not in seen:
                seen.add(other)
                answer[other] = answer[city] - weight       # moving the capital along a road that pointed
away saves one reversal
                stack.append(other)
    return answer
```

797. All Paths From Source to Target

Medium · Depth-First Search, Breadth-First Search, Graph, Backtracking, Directed Acyclic Graph

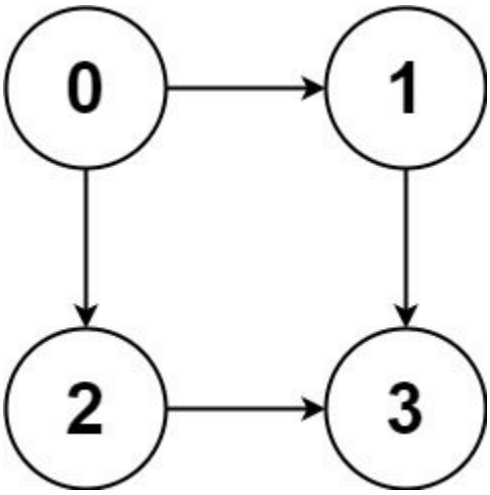
leetcode.com/problems/all-paths-from-source-to-target

PROBLEM

Given a directed acyclic graph (**DAG**) of `n` nodes labeled from `0` to `n - 1`, find all possible paths from node `0` to node `n - 1` and return them in **any order**.

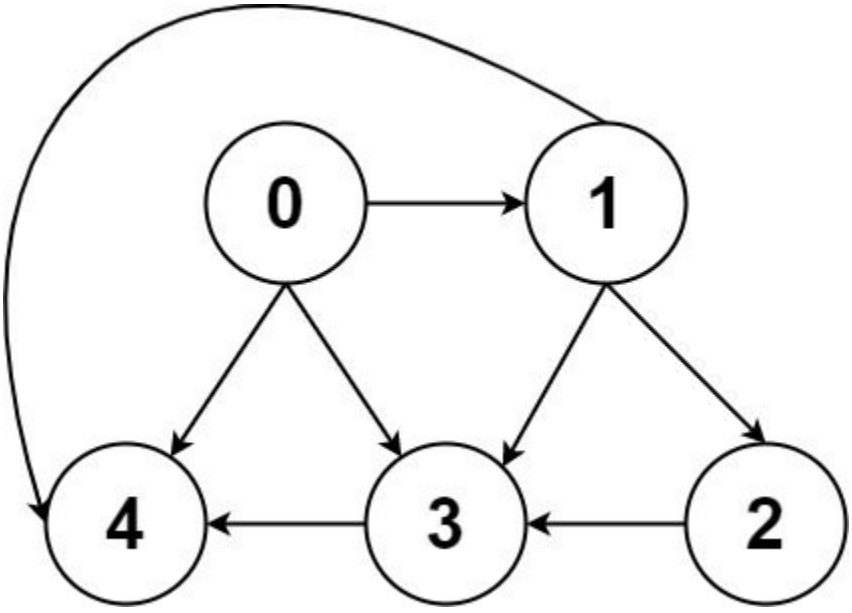
The graph is given as follows: `graph[i]` is a list of all nodes you can visit from node `i` (i.e., there is a directed edge from node `i` to node `graph[i][j]`).

Example 1:




```
Input: graph = [[1,2],[3],[3],[]]
Output: [[0,1,3],[0,2,3]]
Explanation: There are two paths: 0 -> 1 -> 3 and 0 -> 2 -> 3.
```

Example 2:




```
Input: graph = [[4,3,1],[3,2,4],[3],[4],[]]
Output: [[0,4],[0,3,4],[0,1,3,4],[0,1,2,3,4],[0,1,4]]
```

Constraints:

- `n == graph.length`
- `2 <= n <= 15`
- `0 <= graph[i][j] < n`
- `graph[i][j] != i` (i.e., there will be no self-loops).
- All the elements of `graph[i]` are **unique**.
- The input graph is **guaranteed** to be a **DAG**.

SOLUTIONS (PYTHON 3)

Solution 1: BFS

> **How to think about it**
>
> List every path from 0 to n-1 in a DAG. n≤ 15; all paths must be reported. BFS stores full paths; a finished path is recorded, otherwise each neighbor is appended and enqueued.

Start from node 0, keep paths in a queue, and record a path when it reaches n-1.

Python 3

```python
class Solution:
    def allPathsSourceTarget(self, graph: List[List[int]]) -> List[List[int]]:
        n = len(graph)
        q = deque([[0]])
        ans = []
        while q:
            path = q.popleft()
            u = path[-1]
            if u == n - 1:
                ans.append(path)
                continue
            for v in graph[u]:
                q.append(path + [v])
        return ans
```

Complexity. Time: O(2^n * n). Space: O(2^n * n).

Solution 2: DFS

> **How to think about it**
>
> BFS copies every prefix. DFS appends on one array, snapshots at the sink, and pops—fewer intermediate lists.

DFS from node 0 and backtrack after each path that reaches the target.

Python 3

```python
class Solution:
    def allPathsSourceTarget(self, graph: List[List[int]]) -> List[List[int]]:
        def dfs(t):
            if t[-1] == len(graph) - 1:
                ans.append(t[:])
                return
            for v in graph[t[-1]]:
                t.append(v)
                dfs(t)
                t.pop()

        ans = []
        dfs([0])
        return ans
```

Complexity. Time: O(2^n * n). Space: O(n) recursion (plus the answers).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Only count the paths
# 2. The shortest and the longest path (number of edges)
# in the DAG


# ============================================================
# FOLLOW-UP 1: Only count the paths
# ============================================================

# ---- follow-up 1: function ----
# Answer: Memoise the number of paths from every node to
# the target: it is the sum over the neighbours.
# Time: O(V + E).  Space: O(V).
def count_paths(graph):
    target = len(graph) - 1
    memo = {}

    def count(node):
        if node == target:
            return 1
        if node in memo:
            return memo[node]
        total = 0
        for nxt in graph[node]:
            total += count(nxt)
        memo[node] = total
        return total

    return count(0)


# ============================================================
# FOLLOW-UP 2: The shortest and the longest path (number
# of edges) in the DAG
# ============================================================

# ---- follow-up 2: function ----
# Answer: Memoise the best length from every node.
# Time: O(V + E).  Space: O(V).
def shortest_and_longest_path(graph):
    target = len(graph) - 1
    memo = {}

    def best(node):
        if node == target:
            return 0, 0
        if node in memo:
            return memo[node]
        shortest = None
        longest = None
        for nxt in graph[node]:
            s, l = best(nxt)
            if s is None:
                continue
            if shortest is None or s + 1 < shortest:
                shortest = s + 1
            if longest is None or l + 1 > longest:
                longest = l + 1
        memo[node] = (shortest, longest)
        return memo[node]

    return best(0)
```

PATTERN 16 · GRAPH DFS · PROBLEM 13 OF 20

802. Find Eventual Safe States

Medium · Depth-First Search, Breadth-First Search, Graph, Topological Sort, Kosaraju, Tarjan

leetcode.com/problems/find-eventual-safe-states

★ **TOP 7 PROBLEM** in Graph DFS, Topological Sort: master this one first

Also appears in pattern 30 Topological Sort

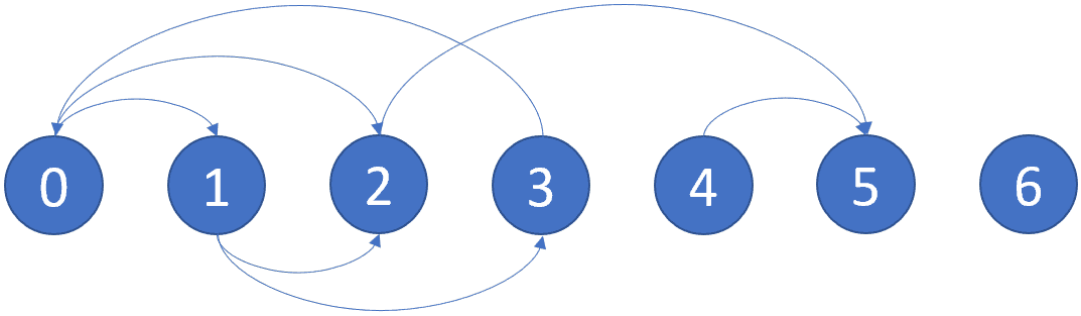
PROBLEM

There is a directed graph of `n` nodes with each node labeled from `0` to `n - 1`. The graph is represented by a **0-indexed** 2D integer array `graph` where `graph[i]` is an integer array of nodes adjacent to node `i`, meaning there is an edge from node `i` to each node in `graph[i]`.

A node is a **terminal node** if there are no outgoing edges. A node is a **safe node** if every possible path starting from that node leads to a **terminal node** (or another safe node).

Return *an array containing all the **safe nodes** of the graph*. The answer should be sorted in **ascending** order.

Example 1:




```
Input: graph = [[1,2],[2,3],[5],[0],[5],[],[]]
Output: [2,4,5,6]
Explanation: The given graph is shown above.
Nodes 5 and 6 are terminal nodes as there are no outgoing edges from either of them.
Every path starting at nodes 2, 4, 5, and 6 all lead to either node 5 or 6.
```

Example 2:

```
Input: graph = [[1,2,3,4],[1,2],[3,4],[0,4],[]]
Output: [4]
Explanation:
Only node 4 is a terminal node, and every path starting at node 4 leads to node 4.
```

Constraints:

- `n == graph.length`
- `1 <= n <= 10^4`
- `0 <= graph[i].length <= n`
- `0 <= graph[i][j] <= n - 1`
- `graph[i]` is sorted in a strictly increasing order.
- The graph may contain self-loops.
- The number of edges in the graph will be in the range `[1, 4 * 10^4]`.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

A node is eventually safe if every walk from it ends at a terminal and never enters a cycle. Searching from every node independently retraces the same chains. Reverse the edges so original out-degree becomes in-degree. Kahn deletion from in-degree 0 removes exactly the nodes that cannot reach a cycle, i.e. the safe nodes. One linear pass is enough for $n \leq 10^4$.

Python 3

```python
class Solution:
    def eventualSafeNodes(self, graph: List[List[int]]) -> List[int]:
        rg = defaultdict(list)
        indeg = [0] * len(graph)
        for i, vs in enumerate(graph):
            for j in vs:
                rg[j].append(i)
            indeg[i] = len(vs)
        q = deque([i for i, v in enumerate(indeg) if v == 0])
        while q:
            i = q.popleft()
            for j in rg[i]:
                indeg[j] -= 1
                if indeg[j] == 0:
                    q.append(j)
        return [i for i, v in enumerate(indeg) if v == 0]
```

Complexity. Time: O(n + e), e = number of edges. Space: O(n + e).

Solution 2

How to think about it

The topological approach builds a reverse graph and in-degrees. A three-color DFS on the original graph is enough: gray means the current path has a cycle; black after the recursion means every walk from that node is safe. No reverse graph is stored. The answer is the nodes whose DFS returns true, still in linear time.

Python 3

```python
class Solution:
    def eventualSafeNodes(self, graph: List[List[int]]) -> List[int]:
        def dfs(i):
            if color[i]:
                return color[i] == 2
            color[i] = 1
            for j in graph[i]:
                if not dfs(j):
                    return False
            color[i] = 2
            return True

        n = len(graph)
        color = [0] * n
        return [i for i in range(n) if dfs(i)]
```

Complexity. Time: O(n + e). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the UNSAFE nodes instead
# 2. The same answer by removing nodes: reverse the graph
# and use Kahn's algorithm


# ============================================================
# FOLLOW-UP 1: Return the UNSAFE nodes instead
# ============================================================

# ---- follow-up 1: function ----
# Answer: A node is unsafe when a cycle can be reached
# from it; the same colouring search finds them.
# Time: O(V + E).  Space: O(V).
def unsafe_nodes(graph):
    n = len(graph)
    colour = [0] * n                                # 0 new, 1 on the path, 2 safe
    unsafe = set()

    def safe(node):
        if colour[node] == 2:
            return True
        if colour[node] == 1 or node in unsafe:
            return False
        colour[node] = 1
        for nxt in graph[node]:
            if not safe(nxt):
                unsafe.add(node)
                return False
        colour[node] = 2
        return True

    for node in range(n):
        safe(node)
    return sorted(unsafe)


# ============================================================
# FOLLOW-UP 2: The same answer by removing nodes: reverse
# the graph and use Kahn's algorithm
# ============================================================

# ---- follow-up 2: function ----
# Answer: A node is safe when all its edges lead to safe
# nodes. Start from the terminal nodes and repeatedly mark
# nodes whose outgoing edges are all marked.
# Time: O(V + E).  Space: O(V + E).
from collections import deque


def safe_nodes_kahn(graph):
    n = len(graph)
    out_degree = [len(graph[i]) for i in range(n)]
    reverse = [[] for i in range(n)]
    for node in range(n):
        for nxt in graph[node]:
            reverse[nxt].append(node)
    queue = deque()
    for node in range(n):
        if out_degree[node] == 0:
```

```
            queue.append(node)
    safe = []
    while queue:
        node = queue.popleft()
```

```
        safe.append(node)
        for before in reverse[node]:
            out_degree[before] -= 1
            if out_degree[before] == 0:
                queue.append(before)
    return sorted(safe)
```

1254. Number of Closed Islands

Medium · Depth-First Search, Breadth-First Search, Union Find, Array, Matrix

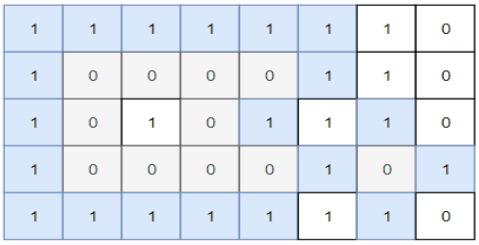
leetcode.com/problems/number-of-closed-islands

PROBLEM

Given a 2D `grid` consists of `0s` (land) and `1s` (water). An *island* is a maximal 4-directionally connected group of `0s` and a *closed island* is an island **totally** (all left, top, right, bottom) surrounded by `1s`.

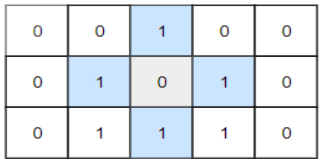
Return the number of *closed islands*.

Example 1:




```
Input: grid =
[[1,1,1,1,1,1,1,0],[1,0,0,0,0,1,1,0],[1,0,1,0,1,1,1,0],[1,0,0,0,0,1,0,1],[1,1,1,1,1,1,1,0]]
Output: 2
Explanation:
Islands in gray are closed because they are completely surrounded by water (group of 1s).
```

Example 2:




```
Input: grid = [[0,0,1,0,0],[0,1,0,1,0],[0,1,1,1,0]]
Output: 1
```

Example 3:

```
Input: grid = [[1,1,1,1,1,1,1],
               [1,0,0,0,0,0,1],
               [1,0,1,1,1,0,1],
               [1,0,1,0,1,0,1],
               [1,0,1,1,1,0,1],
               [1,0,0,0,0,0,1],
               [1,1,1,1,1,1,1]]
Output: 2
```

Constraints:

- `1 <= grid.length, grid[0].length <= 100`
- `0 <= grid[i][j] <=1`

SOLUTIONS (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> A closed island is a land component that never touches the border. m,n ≤ 100 allows flood fill. From each unseen land cell we DFS, painting water to avoid revisits, and count the component if every cell is interior. The return value ANDs "this cell is not on the border" with all neighbors. In-place writes serve as the visited mark.

We traverse the matrix, and for each piece of land, we perform a depth-first search to find all the land connected to it. Then we check if there is any land on the boundary. If there is, it is not a closed island; otherwise, it is a closed island, and we increment the answer by one.

Finally, we return the answer.

The time complexity is O(m × n), and the space complexity is O(m × n). Where m and n are the number of rows and columns in the matrix, respectively.

Python 3

```python
class Solution:
    def closedIsland(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int) -> int:
            res = int(0 < i < m - 1 and 0 < j < n - 1)
            grid[i][j] = 1
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and grid[x][y] == 0:
                    res &= dfs(x, y)
            return res

        m, n = len(grid), len(grid[0])
        dirs = (-1, 0, 1, 0, -1)
        return sum(grid[i][j] == 0 and dfs(i, j) for i in range(m) for j in range(n))
```

Solution 2: Union-Find

> **How to think about it**
>
> DFS uses a call stack and mutates the grid. Union-find links adjacent land and also links border cells to a dummy node; we count land whose root is itself and is not the dummy. Iterative unions avoid recursion; the meaning is still "land components off the border".

We can use a union-find set to maintain each piece of connected land.

We traverse the matrix, if the current position is on the boundary, we connect it with the virtual node m × n. If the current position is land, we connect it with the land below and to the right.

Then, we traverse the matrix again, for each piece of land, if its root node is itself, we increment the answer by one.

Finally, we return the answer.

The time complexity is O(m × n × α(m × n)), and the space complexity is O(m × n). Where m and n are the number of rows and columns in the matrix, respectively.

Python 3

```python
class UnionFind:
    def __init__(self, n: int):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x: int) -> int:
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a: int, b: int):
        pa, pb = self.find(a), self.find(b)
        if pa != pb:
            if self.size[pa] > self.size[pb]:
                self.p[pb] = pa
                self.size[pa] += self.size[pb]
            else:
                self.p[pa] = pb
                self.size[pb] += self.size[pa]


class Solution:
    def closedIsland(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        uf = UnionFind(m * n + 1)
        for i in range(m):
            for j in range(n):
                if i == 0 or i == m - 1 or j == 0 or j == n - 1:
                    uf.union(i * n + j, m * n)
                if grid[i][j] == 0:
                    if i < m - 1 and grid[i + 1][j] == 0:
                        uf.union(i * n + j, (i + 1) * n + j)
                    if j < n - 1 and grid[i][j + 1] == 0:
                        uf.union(i * n + j, i * n + j + 1)
        ans = 0
        for i in range(m):
            for j in range(n):
                ans += grid[i][j] == 0 and uf.find(i * n + j) == i * n + j
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The sizes of the closed islands


# ============================================================
# FOLLOW-UP 1: The sizes of the closed islands
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remove the land (0) that touches the border and
# measure each remaining group.
# Time: O(m * n).  Space: O(m * n).
def closed_island_sizes(grid):
    rows, cols = len(grid), len(grid[0])
    seen = set()

    def spread(r, c):
        # returns (size, touches_border) of the group of 0s containing (r, c)
        size = 0
        touches = False
        stack = [(r, c)]
        seen.add((r, c))
        while stack:
```

```
            x, y = stack.pop()
            size += 1
            if x == 0 or y == 0 or x == rows - 1 or y == cols - 1:
                touches = True
            for nx, ny in ((x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1)):
                if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 0 and (nx, ny) not in seen:
                    seen.add((nx, ny))
                    stack.append((nx, ny))
        return size, touches

    sizes = []
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 0 and (r, c) not in seen:
                size, touches = spread(r, c)
                if not touches:
                    sizes.append(size)
    return sorted(sizes, reverse=True)
```

1376. Time Needed to Inform All Employees

Medium · Tree, Depth-First Search, Breadth-First Search

leetcode.com/problems/time-needed-to-inform-all-employees

PROBLEM

A company has `n` employees with a unique ID for each employee from `0` to `n - 1`. The head of the company is the one with `headID`.

Each employee has one direct manager given in the `manager` array where `manager[i]` is the direct manager of the `i-th` employee, `manager[headID] = -1`. Also, it is guaranteed that the subordination relationships have a tree structure.

The head of the company wants to inform all the company employees of an urgent piece of news. He will inform his direct subordinates, and they will inform their subordinates, and so on until all employees know about the urgent news.

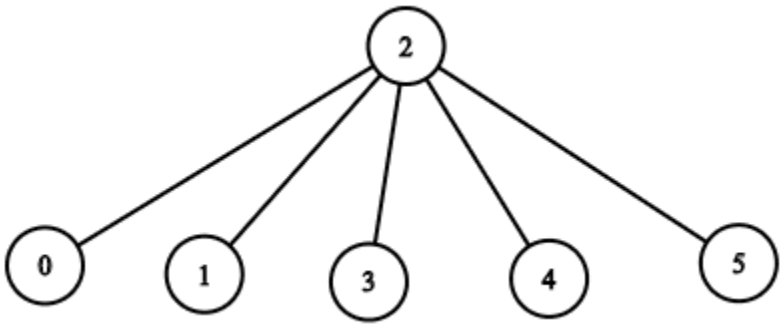
The `i-th` employee needs `informTime[i]` minutes to inform all of his direct subordinates (i.e., After informTime[i] minutes, all his direct subordinates can start spreading the news).

Return *the number of minutes* needed to inform all the employees about the urgent news.

Example 1:

```
Input: n = 1, headID = 0, manager = [-1], informTime = [0]
Output: 0
Explanation: The head of the company is the only employee in the company.
```

Example 2:




```
Input: n = 6, headID = 2, manager = [2,2,-1,2,2,2], informTime = [0,0,1,0,0,0]
Output: 1
Explanation: The head of the company with id = 2 is the direct manager of all the employees in the
company and needs 1 minute to inform them all.
The tree structure of the employees in the company is shown.
```

Constraints:

- `1 <= n <= 10^5`
- `0 <= headID < n`
- `manager.length == n`
- `0 <= manager[i] < n`
- `manager[headID] == -1`
- `informTime.length == n`
- `0 <= informTime[i] <= 1000`
- `informTime[i] == 0` if employee `i` has no subordinates.
- It is **guaranteed** that all the employees can be informed.

SOLUTIONS (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> In a tree of managers, a message starts at the head; an edge costs that employee's inform time. $n \le 10^5$ forbids summing a path per leaf. After building the child list, dfs(i) is the time for i to inform its whole subtree: informTime[i] plus the slowest dfs(j) among direct reports. The answer is dfs(headID).

We first build an adjacent list g according to the manager array, where g[i] represents all direct subordinates of employee i.

Next, we design a function dfs(i), which means the time required for employee i to notify all his subordinates (including direct subordinates and indirect subordinates), and then the answer is dfs(headID).

In function dfs(i), we need to traverse all direct subordinates j of i. For each subordinate, employee i needs to notify him, which takes informTime[i] time, and his subordinates need to notify their subordinates, which takes dfs(j) time. We take the maximum value of informTime[i] + dfs(j) as the return value of function dfs(i).

The time complexity is O(n), and the space complexity is O(n). Where n is the number of employees.

Python 3

```
class Solution:
    def numOfMinutes(
        self, n: int, headID: int, manager: List[int], informTime: List[int]
    ) -> int:
        def dfs(i: int) -> int:
            ans = 0
            for j in g[i]:
                ans = max(ans, dfs(j) + informTime[i])
            return ans

        g = defaultdict(list)
        for i, x in enumerate(manager):
            g[x].append(i)
        return dfs(headID)
```

Solution 2: Explicit Stack

> **How to think about it**
>
> The reporting lines form a tree rooted at the head, and an edge costs that employee's inform time. With $n \le 10^5$, recursing from the head to score every subtree is too deep: a chain makes the call depth n. The time for employee i to inform a whole subtree is i's own inform time plus the slowest direct report. A leaf has no reports, so that time is 0. The value depends only on results that are already known for the children. An explicit stack starts at headID and walks (employee, state) in postorder. On entry we push the exit marker and then the reports, and on exit we store the maximum of informTime[i] + time[j]. The head's time is the answer.

We first build an adjacent list g according to the manager array, where g[i] represents all direct subordinates of employee i.

The time for employee i to inform the whole subtree is informTime[i] plus the maximum time among the direct reports, or 0 when there are no reports. An explicit stack walks from headID in postorder and writes that time when the node is left. The answer is the head's time.

The time complexity is O(n), and the space complexity is O(n). Where n is the number of employees.

Python 3

```python
class Solution:
    def numOfMinutes(
        self, n: int, headID: int, manager: List[int], informTime: List[int]
    ) -> int:
        g = [[] for _ in range(n)]
        for i, x in enumerate(manager):
            if x != -1:
                g[x].append(i)
        time = [0] * n
        stk = [(headID, 0)]
        while stk:
            i, state = stk.pop()
            if state == 0:
                stk.append((i, 1))
                for j in g[i]:
                    stk.append((j, 0))
            else:
                ans = 0
                for j in g[i]:
                    ans = max(ans, time[j] + informTime[i])
                time[i] = ans
        return time[headID]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The time at which EVERY employee learns the news
# 2. Which employee is the last to learn the news?


# ============================================================
# FOLLOW-UP 1: The time at which EVERY employee learns the
# news
# ============================================================

# ---- follow-up 1: function ----
# Answer: A search from the head: an employee learns the
# news at the time of their manager plus the manager's
# inform time.
# Time: O(n).  Space: O(n).
def inform_times(n, head_id, manager, inform_time):
    subordinates = [[] for i in range(n)]
    for employee in range(n):
        if manager[employee] != -1:
            subordinates[manager[employee]].append(employee)
    times = [0] * n
    stack = [head_id]
    while stack:
        boss = stack.pop()
        for employee in subordinates[boss]:
            times[employee] = times[boss] + inform_time[boss]
            stack.append(employee)
    return times


# ============================================================
# FOLLOW-UP 2: Which employee is the last to learn the
# news?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Take the employee with the largest time from the
# previous follow-up.
# Time: O(n).  Space: O(n).
def last_to_learn(n, head_id, manager, inform_time):
```

```
    times = inform_times(n, head_id, manager, inform_time)
    last = head_id
    for employee in range(n):
        if times[employee] > times[last]:
            last = employee
    return last
```

947. Most Stones Removed with Same Row or Column

Medium · Depth-First Search, Union Find, Graph, Hash Table, Bipartite Graph

leetcode.com/problems/most-stones-removed-with-same-row-or-column

★ **TOP 7 PROBLEM** in Union-Find (Disjoint Set): master this one first

Also appears in pattern 31 Union-Find (Disjoint Set)

PROBLEM

On a 2D plane, we place `n` stones at some integer coordinate points. Each coordinate point may have at most one stone.

A stone can be removed if it shares either **the same row or the same column** as another stone that has not been removed.

Given an array `stones` of length `n` where `stones[i] = [x_i, y_i]` represents the location of the `i^th` stone, return *the largest possible number of stones that can be removed*.

Example 1:

```
Input: stones = [[0,0],[0,1],[1,0],[1,2],[2,1],[2,2]]
Output: 5
Explanation: One way to remove 5 stones is as follows:
1. Remove stone [2,2] because it shares the same row as [2,1].
2. Remove stone [2,1] because it shares the same column as [0,1].
3. Remove stone [1,2] because it shares the same row as [1,0].
4. Remove stone [1,0] because it shares the same column as [0,0].
5. Remove stone [0,1] because it shares the same row as [0,0].
Stone [0,0] cannot be removed since it does not share a row/column with another stone still on the
plane.
```

Example 2:

```
Input: stones = [[0,0],[0,2],[1,1],[2,0],[2,2]]
Output: 3
Explanation: One way to make 3 moves is as follows:
1. Remove stone [2,2] because it shares the same row as [2,0].
2. Remove stone [2,0] because it shares the same column as [0,0].
3. Remove stone [0,2] because it shares the same row as [0,0].
Stones [0,0] and [1,1] cannot be removed since they do not share a row/column with another stone still
on the plane.
```

Example 3:

```
Input: stones = [[0,0]]
Output: 0
Explanation: [0,0] is the only stone on the plane, so you cannot remove it.
```

Constraints:

- `1 <= stones.length <= 1000`
- `0 <= x_i, y_i <= 10^4`
- No two stones are at the same coordinate point.

SOLUTIONS (PYTHON 3)

Solution 1: Union-Find

> **How to think about it**
>
> Stones that share a row or column can reach each other. One stone remains per component, so the answer is n minus the number of components. Union every pair that shares a row or column; each successful union is one removable stone.

We can use a union-find data structure to maintain the relationships between stones. If two stones are in the same row or column, we consider them to be connected and use the union-find to link them together. In the end, we count how many connected components there are in the union-find, which corresponds to the number of stones that can remain. Therefore, the total number of stones that can be removed is the total number of stones minus the number of stones that can remain. We can also record the number of successful unions during the merge process, which equals the number of stones that can be removed.

The time complexity is $O(n^2 \times \alpha(n))$, and the space complexity is $O(n)$. Here, n is the number of stones.

Python 3

```python
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True


class Solution:
    def removeStones(self, stones: List[List[int]]) -> int:
        uf = UnionFind(len(stones))
        ans = 0
        for i, (x1, y1) in enumerate(stones):
            for j, (x2, y2) in enumerate(stones[:i]):
                if x1 == x2 or y1 == y2:
                    ans += uf.union(i, j)
        return ans
```

Solution 2: Union-Find (Optimized)

> **How to think about it**
>
> Method 1 links stone pairs and is quadratic. Treat rows and columns as elements: shift columns by an offset and union each stone's row with its column, so a shared row or column connects automatically. Remaining components equal the number of distinct row roots.

We can add an offset to the y-coordinates of the stones, allowing us to unify the x-coordinates and y-coordinates. Then, we use a union-find data structure to maintain the relationship between x-coordinates and y-coordinates.

We iterate through each stone, merging its x-coordinate with its y-coordinate.

Finally, we iterate through all the stones again, putting the root node of each stone's x-coordinate into a set. The number of elements in this set represents the number of stones that can remain. Therefore, the total number of stones that can be removed is the total number of stones minus the number of stones that can remain.

The time complexity is $O(n \times \alpha(m))$, and the space complexity is $O(m)$. Here, n and m represent the number of stones and the maximum value of the coordinates, respectively.

Python 3

```python
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True


class Solution:
    def removeStones(self, stones: List[List[int]]) -> int:
        m = 10001
        uf = UnionFind(m << 1)
        for x, y in stones:
            uf.union(x, y + m)
        return len(stones) - len({uf.find(x) for x, _ in stones})
```

Solution 3: Breadth-First Search

> **How to think about it**
>
> Without union-find, build a graph on shared rows or columns and BFS to count components. The implementation is direct; the worst-case number of edges is still quadratic.

Connect stones that share a row or a column. Each connected component can keep only one stone, so the answer is the number of stones minus the number of components.

Start a BFS from every unvisited stone to mark its component.

The time complexity is $O(n^2)$ and the space complexity is $O(n^2)$, where n is the number of stones.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```python
class Solution:
    def removeStones(self, stones: List[List[int]]) -> int:
        n = len(stones)
        graph = [[] for _ in range(n)]
        for i in range(n):
            for j in range(i + 1, n):
                if stones[i][0] == stones[j][0] or stones[i][1] == stones[j][1]:
                    graph[i].append(j)
                    graph[j].append(i)
        seen = [False] * n
        components = 0
        for start in range(n):
            if seen[start]:
                continue
            components += 1
            seen[start] = True
            queue = deque([start])
            while queue:
                i = queue.popleft()
                for j in graph[i]:
                    if not seen[j]:
                        seen[j] = True
                        queue.append(j)
        return n - components
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Solve it with union-find (join the row and the column
# of every stone)
# 2. List the groups of stones (the connected components)


# ============================================================
# FOLLOW-UP 1: Solve it with union-find (join the row and
# the column of every stone)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Treat each row and each column as a node; a
# stone joins its row node and its column node. The answer
# is the number of stones minus the number of connected
# groups.
# Time: O(n * alpha), essentially O(n).  Space: O(n).
def remove_stones_union_find(stones):
    parent = {}

    def find(x):
        parent.setdefault(x, x)
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for row, col in stones:
        parent[find(("row", row))] = find(("col", col))
    groups = set()
    for row, col in stones:
        groups.add(find(("row", row)))
    return len(stones) - len(groups)


# ============================================================
# FOLLOW-UP 2: List the groups of stones (the connected
# components)
```

```
# ============================================================

# ---- follow-up 2: function ----
# Answer: Two stones are in one group when a chain of
# stones sharing a row or a column joins them.
# Time: O(n^2).  Space: O(n).
def stone_groups(stones):
    n = len(stones)
    seen = [False] * n
    groups = []
    for start in range(n):
        if seen[start]:
            continue
        group = []
        stack = [start]
        seen[start] = True
        while stack:
            i = stack.pop()
            group.append(stones[i])
            for j in range(n):
                if not seen[j] and (stones[j][0] == stones[i][0] or stones[j][1] == stones[i][1]):
                    seen[j] = True
```

```
                    stack.append(j)
        groups.append(group)
    return groups
```

2101. Detonate the Maximum Bombs

Medium · Depth-First Search, Breadth-First Search, Graph, Geometry, Array, Math

leetcode.com/problems/detonate-the-maximum-bombs

PROBLEM

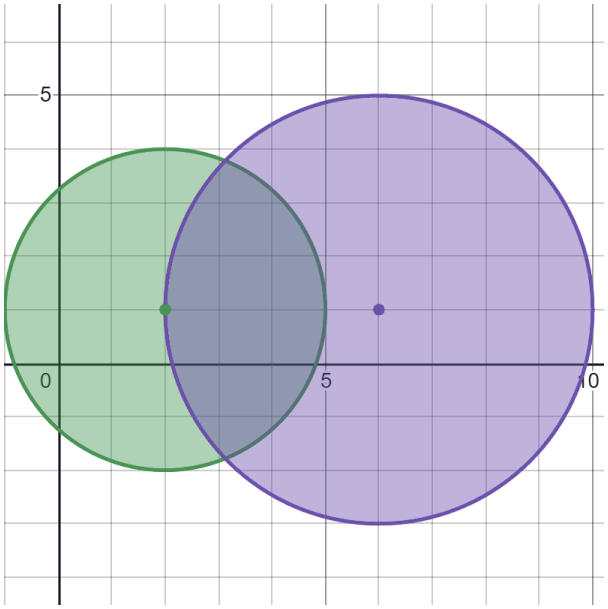
You are given a list of bombs. The **range** of a bomb is defined as the area where its effect can be felt. This area is in the shape of a **circle** with the center as the location of the bomb.

The bombs are represented by a **0-indexed** 2D integer array `bombs` where `bombs[i] = [x_i, y_i, r_i]`. `x_i` and `y_i` denote the X-coordinate and Y-coordinate of the location of the `i^th` bomb, whereas `r_i` denotes the **radius** of its range.

You may choose to detonate a **single** bomb. When a bomb is detonated, it will detonate **all bombs** that lie in its range. These bombs will further detonate the bombs that lie in their ranges.

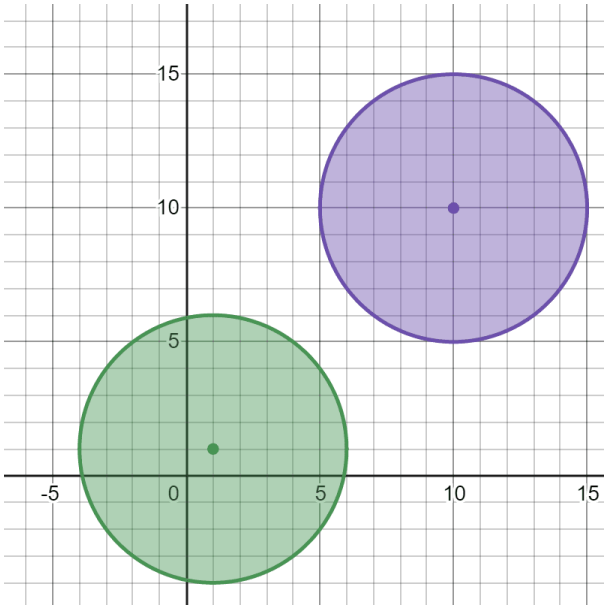
Given the list of `bombs`, return *the **maximum** number of bombs that can be detonated if you are allowed to detonate **only one** bomb.*

Example 1:



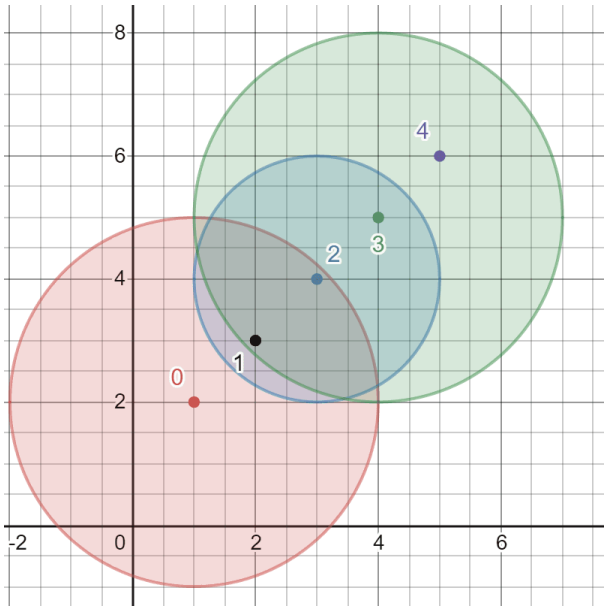

```
Input: bombs = [[2,1,3],[6,1,4]]
Output: 2
Explanation:
The above figure shows the positions and ranges of the 2 bombs.
If we detonate the left bomb, the right bomb will not be affected.
But if we detonate the right bomb, both bombs will be detonated.
So the maximum bombs that can be detonated is max(1, 2) = 2.
```

Example 2:



Input: bombs = [[1,1,5],[10,10,5]]
Output: 1
Explanation:
Detonating either bomb will not detonate the other bomb, so the maximum number of bombs that can be detonated is 1.

Example 3:



Input: bombs = [[1,2,3],[2,3,1],[3,4,2],[4,5,3],[5,6,4]]
Output: 5
Explanation:
The best bomb to detonate is bomb 0 because:
- Bomb 0 detonates bombs 1 and 2. The red circle denotes the range of bomb 0.
- Bomb 2 detonates bomb 3. The blue circle denotes the range of bomb 2.
- Bomb 3 detonates bomb 4. The green circle denotes the range of bomb 3.
Thus all 5 bombs are detonated.

Constraints:

- 1 <= bombs.length <= 100
- bombs[i].length == 3

- $1 <= x_i, y_i, r_i <= 10^5$

SOLUTION (PYTHON 3)

Solution 1: BFS

How to think about it

Detonation propagates along the “inside radius” relation, so we need the largest reachable set starting from one bomb. Treating the relation as undirected would miss one-way reachability when radii differ; rescanning all bombs on every detonation repeats the same distance work. With n≤ 100, building a directed graph g from every pair costs $O(n^2)$. A chain detonation is then a reachability search, which BFS solves in $O(n^2)$ per source. We therefore add edges by pairwise distance checks, then BFS from each bomb, returning n as soon as some search visits every vertex.

We define an array g of length n, where g[i] represents the indices of all bombs that can be triggered by bomb i within its explosion range.

Next, we iterate over all bombs. For two bombs (x_1, y_1, r_1) and (x_2, y_2, r_2), we calculate the distance between them dist = $\sqrt{((x_1 - x_2)^2 + (y_1 - y_2)^2)}$. If dist ≤ r_1, then bomb i can trigger bomb j within its explosion range, so we add j to g[i]. If dist ≤ r_2, then bomb j can trigger bomb i within its explosion range, so we add i to g[j].

Next, we iterate over all bombs. For each bomb k, we use breadth-first search to calculate the indices of all bombs that can be triggered by bomb k within its explosion range and record them. If the number of these bombs equals n, then we can trigger all bombs and directly return n. Otherwise, we record the number of these bombs and return the maximum value.

The time complexity is $O(n^3)$ and the space complexity is $O(n^2)$, where n is the number of bombs.

Python 3

```python
class Solution:
    def maximumDetonation(self, bombs: List[List[int]]) -> int:
        n = len(bombs)
        g = [[] for _ in range(n)]
        for i in range(n - 1):
            x1, y1, r1 = bombs[i]
            for j in range(i + 1, n):
                x2, y2, r2 = bombs[j]
                dist = hypot(x1 - x2, y1 - y2)
                if dist <= r1:
                    g[i].append(j)
                if dist <= r2:
                    g[j].append(i)
        ans = 0
        for k in range(n):
            vis = {k}
            q = [k]
            for i in q:
                for j in g[i]:
                    if j not in vis:
                        vis.add(j)
                        q.append(j)
            if len(vis) == n:
                return n
            ans = max(ans, len(vis))
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. How many bombs does each bomb set off?
# 2. Which bomb is the best one to detonate first?


# ============================================================
# FOLLOW-UP 1: How many bombs does each bomb set off?
# ============================================================

# ---- follow-up 1: function ----
# Answer: Build the graph (bomb i reaches bomb j when j is
# inside i's radius) and run a search from every bomb.
# Time: O(n^3) in the worst case (a search from every bomb
# over up to n^2 edges).  Space: O(n^2).
def detonation_counts(bombs):
    n = len(bombs)
    reaches = [[] for i in range(n)]
    for i in range(n):
        x1, y1, r1 = bombs[i]
        for j in range(n):
            if i != j:
                x2, y2, r2 = bombs[j]
                if (x1 - x2) ** 2 + (y1 - y2) ** 2 <= r1 * r1:
                    reaches[i].append(j)
    counts = []
    for start in range(n):
        seen = {start}
        stack = [start]
        while stack:
            i = stack.pop()
            for j in reaches[i]:
                if j not in seen:
                    seen.add(j)
                    stack.append(j)
        counts.append(len(seen))
    return counts


# ============================================================
# FOLLOW-UP 2: Which bomb is the best one to detonate
# first?
# ============================================================

# ---- follow-up 2: function ----
# Answer: The bomb with the biggest count.
# Time: O(n^3).  Space: O(n^2).
def best_bomb(bombs):
    counts = detonation_counts(bombs)
    best = 0
    for i in range(len(bombs)):
        if counts[i] > counts[best]:
            best = i
    return best, counts[best]
```

1905. Count Sub Islands

Medium · Depth-First Search, Breadth-First Search, Union Find, Array, Matrix

leetcode.com/problems/count-sub-islands

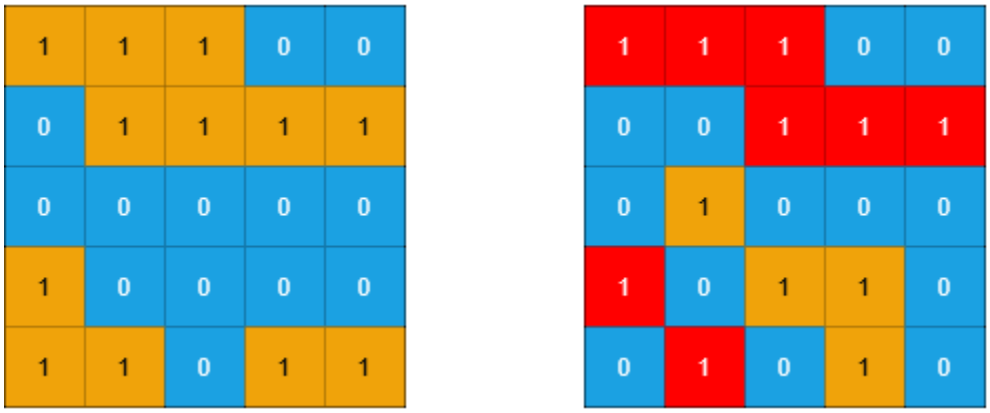
PROBLEM

You are given two `m x n` binary matrices `grid1` and `grid2` containing only `0`'s (representing water) and `1`'s (representing land). An **island** is a group of `1`'s connected **4-directionally** (horizontal or vertical). Any cells outside of the grid are considered water cells.

An island in `grid2` is considered a **sub-island** if there is an island in `grid1` that contains **all** the cells that make up **this** island in `grid2`.

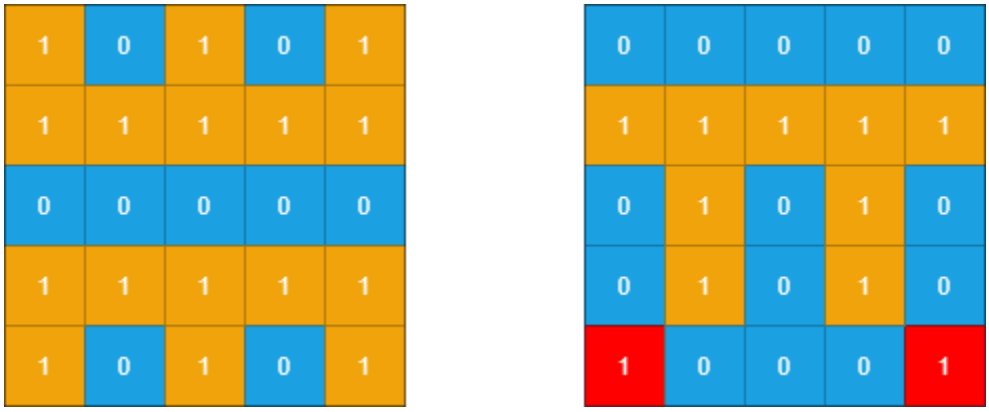
Return the ***number** of islands in `grid2` that are considered **sub-islands***.

Example 1:




```
Input: grid1 = [[1,1,1,0,0],[0,1,1,1,1],[0,0,0,0,0],[1,0,0,0,0],[1,1,0,1,1]], grid2 =
[[1,1,1,0,0],[0,0,1,1,1],[0,1,0,0,0],[1,0,1,1,0],[0,1,0,1,0]]
Output: 3
Explanation: In the picture above, the grid on the left is grid1 and the grid on the right is grid2.
The 1s colored red in grid2 are those considered to be part of a sub-island. There are three
sub-islands.
```

Example 2:




```
Input: grid1 = [[1,0,1,0,1],[1,1,1,1,1],[0,0,0,0,0],[1,1,1,1,1],[1,0,1,0,1]], grid2 =
[[0,0,0,0,0],[1,1,1,1,1],[0,1,0,1,0],[0,1,0,1,0],[1,0,0,0,1]]
Output: 2
Explanation: In the picture above, the grid on the left is grid1 and the grid on the right is grid2.
The 1s colored red in grid2 are those considered to be part of a sub-island. There are two sub-islands.
```

Constraints:

- `m == grid1.length == grid2.length`
- `n == grid1[i].length == grid2[i].length`
- `1 <= m, n <= 500`
- `grid1[i][j]` and `grid2[i][j]` are either `0` or `1`.

SOLUTIONS (PYTHON 3)

Solution 1: DFS

How to think about it

Each island of grid2 is a sub-island only if every land cell also lies on land in grid1. Checking cells in isolation cannot group them by island. One DFS (or BFS) walks a component, zeros grid2 to mark it visited, and ANDs the corresponding grid1 cells to decide whether the island is valid. A linear scan starts a search at every remaining 1 in grid2 and sums the return values.

We can traverse each cell (i, j) in the matrix `grid2`. If the value of the cell is 1, we start a depth-first search from this cell, set the value of all cells connected to this cell to 0, and record whether the corresponding cell in `grid1` is also 1 for all cells connected to this cell. If it is 1, it means that this cell is also an island in `grid1`, otherwise it is not. Finally, we count the number of sub-islands in `grid2`.

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the number of rows and columns of the matrices `grid1` and `grid2`, respectively.

Python 3

```
class Solution:
    def countSubIslands(self, grid1: List[List[int]], grid2: List[List[int]]) -> int:
        def dfs(i: int, j: int) -> int:
            ok = grid1[i][j]
            grid2[i][j] = 0
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and grid2[x][y] and not dfs(x, y):
                    ok = 0
            return ok

        m, n = len(grid1), len(grid1[0])
        dirs = (-1, 0, 1, 0, -1)
        return sum(dfs(i, j) for i in range(m) for j in range(n) if grid2[i][j])
```

Solution 2: Explicit Stack

How to think about it

An island of grid2 is a sub-island only when every land cell is also land in grid1. Cells that are not grouped by connectivity cannot be counted as one island. Recursing along a snake of 1000 land cells overflows the call stack, and m,n≤ 500 can hold that snake. The check only has to walk the whole island, zero grid2, and AND the matching grid1 cells. It does not need a recursive return. When a remaining 1 is found, push it onto an explicit stack. Pop a cell, AND it with grid1, and push each neighboring land cell after marking it. The sum of these results is the number of sub-islands.

Scan every cell of `grid2`. When it is still 1, walk that island with an explicit stack, set `grid2` to 0 along the way, and AND the matching `grid1` cells. A result of 1 means the island is a sub-island.

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the number of rows and columns of the matrices `grid1` and `grid2`, respectively.

Python 3

```
class Solution:
    def countSubIslands(self, grid1: List[List[int]], grid2: List[List[int]]) -> int:
        def flood(i: int, j: int) -> int:
            ok = 1
            grid2[i][j] = 0
            stk = [(i, j)]
            while stk:
                i, j = stk.pop()
                ok &= grid1[i][j]
                for a, b in pairwise(dirs):
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n and grid2[x][y]:
                        grid2[x][y] = 0
                        stk.append((x, y))
            return ok

        m, n = len(grid1), len(grid1[0])
        dirs = (-1, 0, 1, 0, -1)
        return sum(flood(i, j) for i in range(m) for j in range(n) if grid2[i][j])
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The sizes of the sub-islands
# 2. Count the islands of grid2 that overlap grid1 in at
# least one cell


# ========================================================
# FOLLOW-UP 1: The sizes of the sub-islands
# ========================================================

# ---- follow-up 1: function ----
# Answer: Search each island of grid2 and measure it when
# every one of its cells is land in grid1.
# Time: O(m * n).  Space: O(m * n).
def sub_island_sizes(grid1, grid2):
    rows, cols = len(grid2), len(grid2[0])
    seen = set()
    sizes = []
    for r in range(rows):
        for c in range(cols):
            if grid2[r][c] == 1 and (r, c) not in seen:
                size = 0
                inside = True
                stack = [(r, c)]
                seen.add((r, c))
                while stack:
                    x, y = stack.pop()
                    size += 1
                    if grid1[x][y] == 0:
                        inside = False
                    for nx, ny in ((x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1)):
                        if 0 <= nx < rows and 0 <= ny < cols and grid2[nx][ny] == 1 and (nx, ny) not in
seen:
                            seen.add((nx, ny))
                            stack.append((nx, ny))
                if inside:
                    sizes.append(size)
    return sorted(sizes, reverse=True)


# ========================================================
# FOLLOW-UP 2: Count the islands of grid2 that overlap
# grid1 in at least one cell
```

```
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same search; an island counts when at least one
# of its cells is also land in grid1.
# Time: O(m * n).  Space: O(m * n).
def count_overlapping_islands(grid1, grid2):
    rows, cols = len(grid2), len(grid2[0])
    seen = set()
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid2[r][c] == 1 and (r, c) not in seen:
                overlaps = False
                stack = [(r, c)]
                seen.add((r, c))
                while stack:
                    x, y = stack.pop()
```

```
                    if grid1[x][y] == 1:
                        overlaps = True
                    for nx, ny in ((x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1)):
                        if 0 <= nx < rows and 0 <= ny < cols and grid2[nx][ny] == 1 and (nx, ny) not in
seen:
                            seen.add((nx, ny))
                            stack.append((nx, ny))
                if overlaps:
                    count += 1
    return count
```

1514. Path with Maximum Probability

Medium · Graph, Array, Shortest Path, Dijkstra, Heap (Priority Queue)

leetcode.com/problems/path-with-maximum-probability

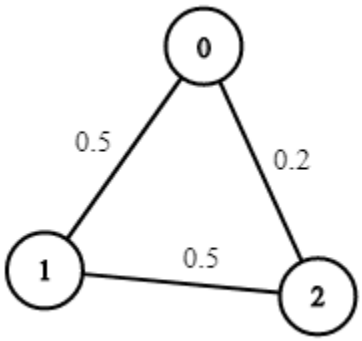
PROBLEM

You are given an undirected weighted graph of `n` nodes (0-indexed), represented by an edge list where `edges[i] = [a, b]` is an undirected edge connecting the nodes `a` and `b` with a probability of success of traversing that edge `succProb[i]`.

Given two nodes `start` and `end`, find the path with the maximum probability of success to go from `start` to `end` and return its success probability.

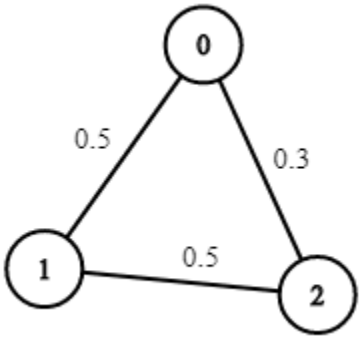
If there is no path from `start` to `end`, **return 0**. Your answer will be accepted if it differs from the correct answer by at most **1e-5**.

Example 1:



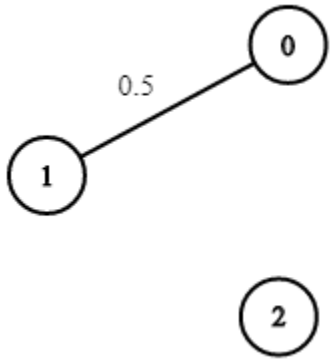

```
Input: n = 3, edges = [[0,1],[1,2],[0,2]], succProb = [0.5,0.5,0.2], start = 0, end = 2
Output: 0.25000
Explanation: There are two paths from start to end, one having a probability of success = 0.2 and the other has 0.5 * 0.5 = 0.25.
```

Example 2:




```
Input: n = 3, edges = [[0,1],[1,2],[0,2]], succProb = [0.5,0.5,0.3], start = 0, end = 2
Output: 0.30000
```

Example 3:




```
Input: n = 3, edges = [[0,1]], succProb = [0.5], start = 0, end = 2
Output: 0.00000
Explanation: There is no path between 0 and 2.
```

Constraints:

- `2 <= n <= 10^4`
- `0 <= start, end < n`
- `start != end`
- `0 <= a, b < n`
- `a != b`
- `0 <= succProb.length == edges.length <= 2*10^4`
- `0 <= succProb[i] <= 1`
- There is at most one edge between every two nodes.

SOLUTION (PYTHON 3)

Solution 1: Heap-Optimized Dijkstra Algorithm

How to think about it

We want the maximum success probability from start to end on an undirected graph whose edge weights multiply. n≤ 10^4 and m≤ 2× 10^4, so listing simple paths is impossible. Treating weights as the opposite of a cost, the problem is isomorphic to a shortest path: probabilities multiply and lie in (0,1], so the optimum has optimal substructure. A max-heap Dijkstra relaxes neighbors by the product of the current probability and the edge; storing negated values simulates the max-heap. The first time the destination is settled is the answer.

We can use Dijkstra's algorithm to find the shortest path, but here we modify it slightly to find the path with the maximum probability.

We use a priority queue (max-heap) pq to store the probability from the starting point to each node and the node's identifier. Initially, we set the probability of the starting point to 1 and the probabilities of the other nodes to 0, then add the starting point to pq.

In each iteration, we take out the node a with the highest probability from pq and its probability w. If the probability of node a is already greater than w, we can skip this node. Otherwise, we traverse all adjacent edges (a, b) of a. If the probability of b is less than the probability of a multiplied by the probability of (a, b), we update the probability of b and add b to pq.

Finally, we obtain the maximum probability from the starting point to the endpoint.

The time complexity is O(m × log m), and the space complexity is O(m). Here, m is the number of edges.

Python 3

```python
class Solution:
    def maxProbability(
        self,
        n: int,
        edges: List[List[int]],
        succProb: List[float],
        start_node: int,
        end_node: int,
    ) -> float:
        g: List[List[Tuple[int, float]]] = [[] for _ in range(n)]
        for (a, b), p in zip(edges, succProb):
            g[a].append((b, p))
            g[b].append((a, p))
        pq = [(-1, start_node)]
        dist = [0] * n
        dist[start_node] = 1
        while pq:
            w, a = heappop(pq)
            w = -w
            if dist[a] > w:
                continue
            for b, p in g[a]:
                if (t := w * p) > dist[b]:
                    dist[b] = t
                    heappush(pq, (-t, b))
        return dist[end_node]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the best path itself
# 2. The same answer with Bellman-Ford (repeat relaxing
# all edges)


# ============================================================
# FOLLOW-UP 1: Return the best path itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Dijkstra with a max-heap on the probability;
# remember the previous node of every improvement and walk
# back from the end.
# Time: O(E log V).  Space: O(V + E).
import heapq


def best_probability_path(n, edges, succ_prob, start, end):
    graph = [[] for i in range(n)]
    for i in range(len(edges)):
        a, b = edges[i]
        graph[a].append((b, succ_prob[i]))
        graph[b].append((a, succ_prob[i]))
    best = [0.0] * n
    best[start] = 1.0
    previous = [-1] * n
    heap = [(-1.0, start)]
    while heap:
        negative, node = heapq.heappop(heap)
        probability = -negative
        if probability < best[node]:
            continue
        if node == end:
            break
        for other, p in graph[node]:
```

```
            if probability * p > best[other]:
                best[other] = probability * p
                previous[other] = node
                heapq.heappush(heap, (-best[other], other))
    if best[end] == 0.0:
        return 0.0, []
    path = []
    node = end
    while node != -1:
        path.append(node)
        node = previous[node]
    path.reverse()
    return best[end], path


# ============================================================
# FOLLOW-UP 2: The same answer with Bellman-Ford (repeat
# relaxing all edges)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Relax every edge up to n - 1 times; it needs no
# heap and also works when the graph is given as a plain
# edge list.
```

```
# Time: O(V * E).  Space: O(V).
def max_probability_bellman_ford(n, edges, succ_prob, start, end):
    best = [0.0] * n
    best[start] = 1.0
    for round_number in range(n - 1):
        changed = False
        for i in range(len(edges)):
            a, b = edges[i]
            p = succ_prob[i]
            if best[a] * p > best[b]:
                best[b] = best[a] * p
                changed = True
            if best[b] * p > best[a]:
                best[a] = best[b] * p
                changed = True
        if not changed:
            break
    return best[end]
```

827. Making A Large Island

Hard · Depth-First Search, Breadth-First Search, Union Find, Array, Matrix

leetcode.com/problems/making-a-large-island

★ **TOP 7 PROBLEM** in Graph DFS: master this one first

Also appears in pattern 31 Union-Find (Disjoint Set)

PROBLEM

You are given an `n x n` binary matrix `grid`. You are allowed to change **at most one** `0` to be `1`.

Return *the size of the largest **island** in* `grid` *after applying this operation*.

An **island** is a 4-directionally connected group of `1`s.

Example 1:

```
Input: grid = [[1,0],[0,1]]
Output: 3
Explanation: Change one 0 to 1 and connect two 1s, then we get an island with area = 3.
```

Example 2:

```
Input: grid = [[1,1],[1,0]]
Output: 4
Explanation: Change the 0 to 1 and make the island bigger, only one island with area = 4.
```

Example 3:

```
Input: grid = [[1,1],[1,1]]
Output: 4
Explanation: Can't change any 0 to 1, only one island with area = 4.
```

Constraints:

- `n == grid.length`
- `n == grid[i].length`
- `1 <= n <= 500`
- `grid[i][j]` is either `0` or `1`.

SOLUTION (PYTHON 3)

Solution 1: DFS

How to think about it

We may flip at most one 0 and want the largest island. n≤ 500, so flooding from every 0 would revisit the same islands. DFS first labels each island and records its area. Then each 0 sums the distinct neighboring islands plus one. If the grid is all ones, the answer is the largest existing island.

We can assign a unique identifier to each connected component, using an array p to record the connected component each position belongs to, i.e., p[i][j] represents the connected component number of (i, j). Use an array cnt to record the size of each connected component, i.e., cnt[root] represents the size of the connected component root.

First, we traverse the entire matrix. For each position grid[i][j] = 1 and p[i][j] = 0, we perform a depth-first search on it, mark its connected component as root, and count the size of the connected component.

Then, we traverse the entire matrix again. For each position grid[i][j] = 0, we find the connected components of the four positions above, below, left, and right of it, add up the sizes of these connected components, and add 1 for the current position, to get the maximum island area after changing the current position to 1.

The time complexity is $O(n^2)$, and the space complexity is $O(n^2)$. Where n is the length of the sides of the matrix.

Python 3

```python
class Solution:
    def largestIsland(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int):
            p[i][j] = root
            cnt[root] += 1
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < n and 0 <= y < n and grid[x][y] and p[x][y] == 0:
                    dfs(x, y)

        n = len(grid)
        cnt = Counter()
        p = [[0] * n for _ in range(n)]
        dirs = (-1, 0, 1, 0, -1)
        root = 0
        for i, row in enumerate(grid):
            for j, x in enumerate(row):
                if x and p[i][j] == 0:
                    root += 1
                    dfs(i, j)
        ans = max(cnt.values() or [0])
        for i, row in enumerate(grid):
            for j, x in enumerate(row):
                if x == 0:
                    s = set()
                    for a, b in pairwise(dirs):
                        x, y = i + a, j + b
                        if 0 <= x < n and 0 <= y < n:
                            s.add(p[x][y])
                    ans = max(ans, sum(cnt[root] for root in s) + 1)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Which cell should be changed (return its position)?


# ============================================================
# FOLLOW-UP 1: Which cell should be changed (return its
# position)?
# ============================================================

# ---- follow-up 1: function ----
# Answer: Label every island with its size, then for each
# 0 add the sizes of the different islands around it plus
# one; remember the best cell.
# Time: O(n^2) for an n by n grid.  Space: O(n^2).
def best_cell_to_flip(grid):
    n = len(grid)
    label = {}
    size = {}
    next_id = 2
    for r in range(n):
        for c in range(n):
            if grid[r][c] == 1 and (r, c) not in label:
                stack = [(r, c)]
                label[(r, c)] = next_id
                count = 0
                while stack:
                    x, y = stack.pop()
                    count += 1
                    for nx, ny in ((x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1)):
```

```
                    if 0 <= nx < n and 0 <= ny < n and grid[nx][ny] == 1 and (nx, ny) not in label:
                        label[(nx, ny)] = next_id
                        stack.append((nx, ny))
            size[next_id] = count
            next_id += 1
    best = (0, None)
    for r in range(n):
        for c in range(n):
            if grid[r][c] == 0:
                around = set()
                for nx, ny in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
                    if (nx, ny) in label:
                        around.add(label[(nx, ny)])
                total = 1
                for island in around:
                    total += size[island]
                if best[1] is None or total > best[0]:
                    best = (total, (r, c))
    return best[1], best[0]
```

PATTERN 17 OF 41

Graph BFS

20 problems · 0 Easy · 14 Medium · 6 Hard

THE IDEA

Explore in rings around the start with a queue, so the first time you reach a node is by a shortest path (when every edge costs the same).

Signals: shortest path in an unweighted graph, minimum steps / moves, word ladder, rotting oranges, multi-source spreading, open the lock.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
from collections import deque
dist = {start: 0}
q = deque([start])
while q:
    u = q.popleft()
    for v in graph[u]:
        if v not in dist:
            dist[v] = dist[u] + 1
            q.append(v)
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	★ **Rotting Oranges**	Medium	LC 994	924
2	★ **01 Matrix**	Medium	LC 542	927
3	★ **Shortest Path in Binary Matrix**	Medium	LC 1091	931
4	★ **Open the Lock**	Medium	LC 752	934
5	Snakes and Ladders	Medium	LC 909	939
6	Nearest Exit from Entrance in Maze	Medium	LC 1926	943
7	As Far from Land as Possible	Medium	LC 1162	947
8	Shortest Path with Alternating Colors	Medium	LC 1129	950
9	Jump Game III	Medium	LC 1306	953
10	Minimum Genetic Mutation	Medium	LC 433	956
11	★ **Cheapest Flights Within K Stops**	Medium	LC 787	959
12	Shortest Bridge	Medium	LC 934	963
13	Get Watched Videos by Your Friends	Medium	LC 1311	966
14	Walls and Gates — substitute: Map of Highest Peak	Medium	LC 1765	970
15	★ **Word Ladder**	Hard	LC 127	974
16	Bus Routes	Hard	LC 815	978
17	★ **Shortest Path to Get All Keys**	Hard	LC 864	981
18	Minimum Number of Days to Disconnect Island	Hard	LC 1568	986
19	Shortest Path in a Grid with Obstacles Elimination	Hard	LC 1293	989
20	Word Ladder II	Hard	LC 126	993

PATTERN 17 · GRAPH BFS · PROBLEM 1 OF 20

994. Rotting Oranges

Medium · Breadth-First Search, Array, Matrix

leetcode.com/problems/rotting-oranges

★ **TOP 7 PROBLEM** in Graph BFS: master this one first

Also appears in pattern 18 Matrix / Grid Traversal

PROBLEM

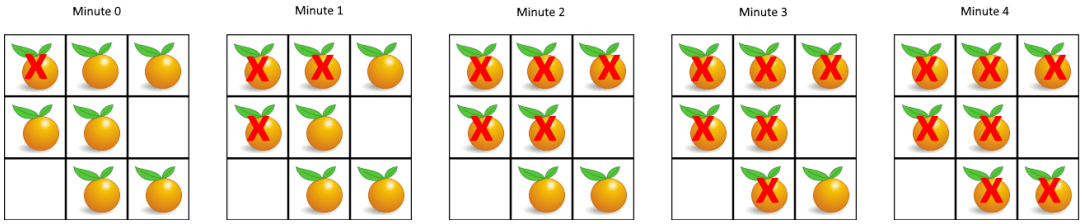
You are given an `m x n grid` where each cell can have one of three values:

- `0` representing an empty cell,
- `1` representing a fresh orange, or
- `2` representing a rotten orange.

Every minute, any fresh orange that is **4-directionally adjacent** to a rotten orange becomes rotten.

Return *the minimum number of minutes that must elapse until no cell has a fresh orange*. If *this is impossible, return* `-1`.

Example 1:




```
Input: grid = [[2,1,1],[1,1,0],[0,1,1]]
Output: 4
```

Example 2:

```
Input: grid = [[2,1,1],[0,1,1],[1,0,1]]
Output: -1
Explanation: The orange in the bottom left corner (row 2, column 0) is never rotten, because rotting
only happens 4-directionally.
```

Example 3:

```
Input: grid = [[0,2]]
Output: 0
Explanation: Since there are already no fresh oranges at minute 0, the answer is just 0.
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 10`
- `grid[i][j]` is `0`, `1`, or `2`.

SOLUTION (PYTHON 3)

Solution 1: BFS

How to think about it

Rot spreads to four-neighbors each minute; we want the time until every orange is rotten. Simultaneous multi-source spread is an unweighted shortest path. Enqueue every rotten orange and count the fresh ones, then BFS by layers. The layer that zeroes the fresh count is the answer; leftover fresh oranges yield -1.

First, we traverse the entire grid once, count the number of fresh oranges, denoted as cnt, and add the coordinates of all rotten oranges to the queue q.

Next, we perform a breadth-first search. In each round of the search, we let all the rotten oranges in the queue rot the fresh oranges in four directions, until the queue is empty or the number of fresh oranges is 0.

Finally, if the number of fresh oranges is 0, we return the current round number, otherwise, we return -1.

The time complexity is O(m × n), and the space complexity is O(m × n). Where m and n are the number of rows and columns of the grid, respectively.

Python 3

```python
class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        cnt = 0
        q = deque()
        for i, row in enumerate(grid):
            for j, x in enumerate(row):
                if x == 2:
                    q.append((i, j))
                elif x == 1:
                    cnt += 1
        ans = 0
        dirs = (-1, 0, 1, 0, -1)
        while q and cnt:
            ans += 1
            for _ in range(len(q)):
                i, j = q.popleft()
                for a, b in pairwise(dirs):
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n and grid[x][y] == 1:
                        grid[x][y] = 2
                        q.append((x, y))
                        cnt -= 1
                        if cnt == 0:
                            return ans
        return -1 if cnt else 0
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The minute at which EVERY fresh orange rots
# 2. Rot also spreads diagonally (8 directions): minutes
# until no fresh orange is left


# ============================================================
# FOLLOW-UP 1: The minute at which EVERY fresh orange rots
# ============================================================

# ---- follow-up 1: function ----
# Answer: The BFS level of a fresh orange is the minute it
# rots: record it in a grid (-1 for oranges that never
# rot).
```

```
# Time: O(m * n).  Space: O(m * n).
from collections import deque


def rotting_times(grid):
    rows, cols = len(grid), len(grid[0])
    time = []
    queue = deque()
    for r in range(rows):
        row = []
        for c in range(cols):
            if grid[r][c] == 2:
                row.append(0)
                queue.append((r, c))
            else:
                row.append(-1)
        time.append(row)
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1 and time[nr][nc] == -1:
                time[nr][nc] = time[r][c] + 1
                queue.append((nr, nc))
    return time


# ============================================================
# FOLLOW-UP 2: Rot also spreads diagonally (8 directions):
# minutes until no fresh orange is left
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same BFS with eight neighbours.
# Time: O(m * n).  Space: O(m * n).
from collections import deque


def minutes_to_rot_8(grid):
    rows, cols = len(grid), len(grid[0])
    queue = deque()
    fresh = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                queue.append((r, c, 0))
            elif grid[r][c] == 1:
                fresh += 1
    minutes = 0
    seen = set((r, c) for r, c, t in queue)
    while queue:
        r, c, t = queue.popleft()
        minutes = t
        for dr in (-1, 0, 1):
            for dc in (-1, 0, 1):
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1 and (nr, nc) not in seen:
                    seen.add((nr, nc))
                    fresh -= 1
                    queue.append((nr, nc, t + 1))
    if fresh > 0:
        return -1
    return minutes
```

PATTERN 17 · GRAPH BFS · PROBLEM 2 OF 20

542. 01 Matrix

Medium · Breadth-First Search, Array, Dynamic Programming, Matrix

leetcode.com/problems/01-matrix

★ **TOP 7 PROBLEM** in Graph BFS, Matrix / Grid Traversal: master this one first

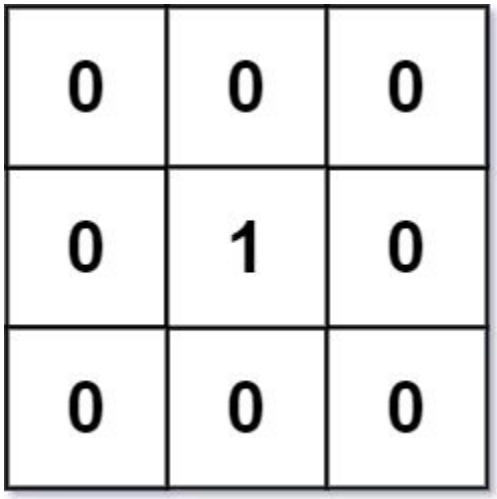
Also appears in pattern 18 Matrix / Grid Traversal

PROBLEM

Given an `m x n` binary matrix `mat`, return *the distance of the nearest* `0` *for each cell*.

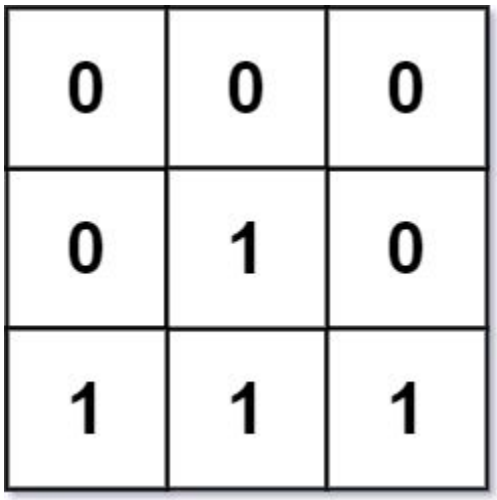
The distance between two cells sharing a common edge is `1`.

Example 1:




```
Input: mat = [[0,0,0],[0,1,0],[0,0,0]]
Output: [[0,0,0],[0,1,0],[0,0,0]]
```

Example 2:




```
Input: mat = [[0,0,0],[0,1,0],[1,1,1]]
Output: [[0,0,0],[0,1,0],[1,2,1]]
```

Constraints:

- `m == mat.length`
- `n == mat[i].length`
- `1 <= m, n <= 10^4`
- `1 <= m * n <= 10^4`
- `mat[i][j]` is either `0` or `1`.
- There is at least one `0` in `mat`.

Note: This question is the same as 1765: https://leetcode.com/problems/map-of-highest-peak/

SOLUTION (PYTHON 3)

Solution 1: BFS

> **How to think about it**
>
> Each 1 needs its distance to the nearest 0. A separate BFS from every 1 repeats O(mn) searches. Multi-source BFS: enqueue every 0 at distance 0 and expand to unseen neighbors. The first visit is the nearest 0. One queue pass covers the grid.

We create a matrix ans of the same size as mat and initialize all elements to -1.

Then, we traverse mat, adding the coordinates (i, j) of all 0 elements to the queue q, and setting ans[i][j] to 0.

Next, we use Breadth-First Search (BFS), removing an element (i, j) from the queue and traversing its four directions. If the element in that direction (x, y) satisfies $0 \le x < m$, $0 \le y < n$ and ans[x][y] = -1, then we set ans[x][y] to ans[i][j] + 1 and add (x, y) to the queue q.

Finally, we return ans.

The time complexity is $O(m \times n)$, and the space complexity is $O(m \times n)$. Here, m and n are the number of rows and columns in the matrix mat, respectively.

Python 3

```
class Solution:
    def updateMatrix(self, mat: List[List[int]]) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        ans = [[-1] * n for _ in range(m)]
        q = deque()
        for i, row in enumerate(mat):
            for j, x in enumerate(row):
                if x == 0:
                    ans[i][j] = 0
                    q.append((i, j))
        dirs = (-1, 0, 1, 0, -1)
        while q:
            i, j = q.popleft()
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and ans[x][y] == -1:
                    ans[x][y] = ans[i][j] + 1
                    q.append((x, y))
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Without BFS: two passes of dynamic programming
# 2. Some cells are walls (-1) that cannot be crossed


# ============================================================
# FOLLOW-UP 1: Without BFS: two passes of dynamic
# programming
# ============================================================

# ---- follow-up 1: function ----
# Answer: A cell's distance is 1 + the smallest neighbour
# distance. One pass from the top-left looks at the cells
# above and to the left; a second pass from the bottom-
# right looks at the cells below and to the right.
# Time: O(m * n).  Space: O(m * n).
def update_matrix_two_passes(mat):
    rows, cols = len(mat), len(mat[0])
    big = rows + cols
    dist = []
    for r in range(rows):
        dist.append([0 if mat[r][c] == 0 else big for c in range(cols)])
    for r in range(rows):
        for c in range(cols):
            if r > 0 and dist[r - 1][c] + 1 < dist[r][c]:
                dist[r][c] = dist[r - 1][c] + 1
            if c > 0 and dist[r][c - 1] + 1 < dist[r][c]:
                dist[r][c] = dist[r][c - 1] + 1
    for r in range(rows - 1, -1, -1):
        for c in range(cols - 1, -1, -1):
            if r < rows - 1 and dist[r + 1][c] + 1 < dist[r][c]:
                dist[r][c] = dist[r + 1][c] + 1
            if c < cols - 1 and dist[r][c + 1] + 1 < dist[r][c]:
                dist[r][c] = dist[r][c + 1] + 1
    return dist


# ============================================================
# FOLLOW-UP 2: Some cells are walls (-1) that cannot be
# crossed
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same multi-source BFS, but never enter a
# wall; cells that cannot reach a 0 keep the value -1.
# Time: O(m * n).  Space: O(m * n).
from collections import deque


def distance_to_zero_with_walls(mat):
    rows, cols = len(mat), len(mat[0])
    dist = []
    queue = deque()
    for r in range(rows):
        row = []
        for c in range(cols):
            if mat[r][c] == 0:
                row.append(0)
                queue.append((r, c))
            else:
```

```
                row.append(-1)
        dist.append(row)
    while queue:
        r, c = queue.popleft()
```

```
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and mat[nr][nc] == 1 and dist[nr][nc] == -1:
                dist[nr][nc] = dist[r][c] + 1
                queue.append((nr, nc))
    return dist
```

PATTERN 17 · GRAPH BFS · PROBLEM 3 OF 20

1091. Shortest Path in Binary Matrix

Medium · Breadth-First Search, Array, Matrix

leetcode.com/problems/shortest-path-in-binary-matrix

★ **TOP 7 PROBLEM** in Graph BFS: master this one first

Also appears in pattern 18 Matrix / Grid Traversal

PROBLEM

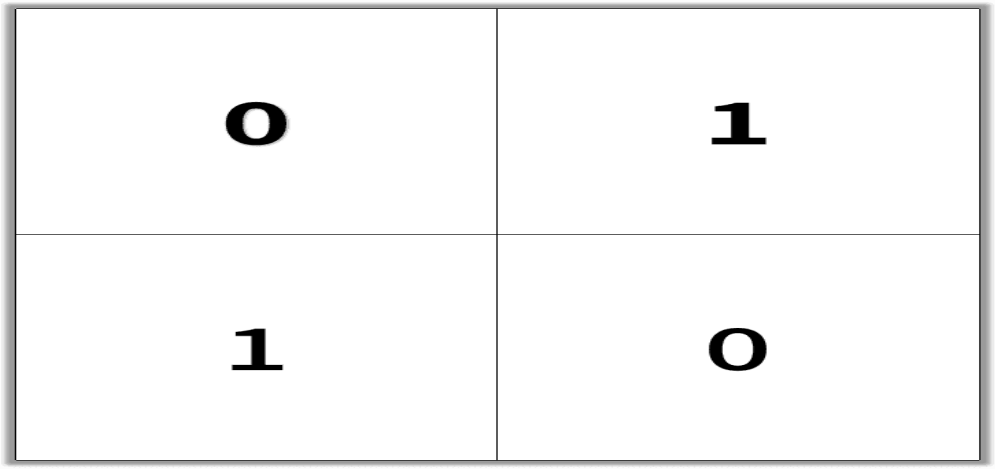
Given an `n x n` binary matrix `grid`, return *the length of the shortest **clear path** in the matrix*. If there is no clear path, return `-1`.

A **clear path** in a binary matrix is a path from the **top-left** cell (i.e., `(0, 0)`) to the **bottom-right** cell (i.e., `(n - 1, n - 1)`) such that:

- All the visited cells of the path are `0`.
- All the adjacent cells of the path are **8-directionally** connected (i.e., they are different and they share an edge or a corner).

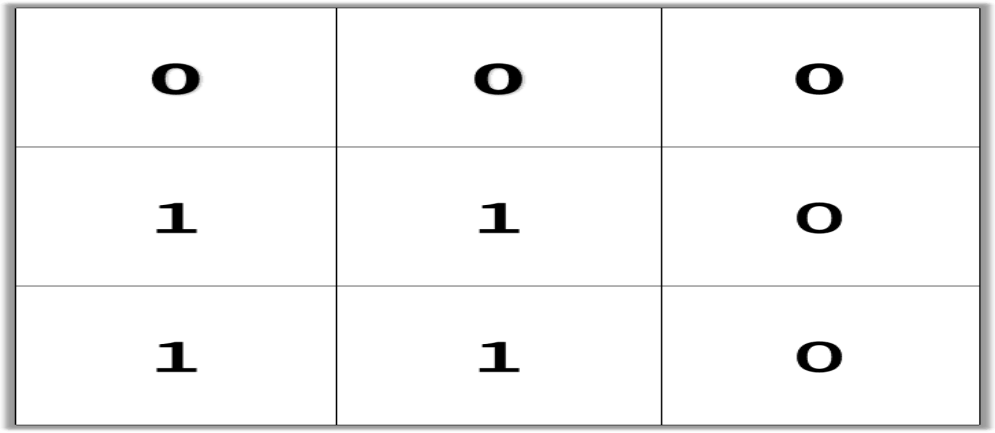
The **length of a clear path** is the number of visited cells of this path.

Example 1:




```
Input: grid = [[0,1],[1,0]]
Output: 2
```

Example 2:



```
Input: grid = [[0,0,0],[1,1,0],[1,1,0]]
Output: 4
```

Example 3:

```
Input: grid = [[1,0,0],[1,1,0],[1,1,0]]
Output: -1
```

Constraints:

- `n == grid.length`
- `n == grid[i].length`
- `1 <= n <= 100`
- `grid[i][j] is 0 or 1`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

The eight-connected path has unit weights, so BFS by layer is the shortest length. n≤ 100 lets us mark visited cells in the grid. A blocked origin has no path. Enqueue (0,0) and mark it 1. Each layer expands the eight neighbors that are still 0. Reaching (n-1,n-1) yields the current layer index. If the queue empties first, the answer is -1.

Python 3

```python
class Solution:
    def shortestPathBinaryMatrix(self, grid: List[List[int]]) -> int:
        if grid[0][0]:
            return -1
        n = len(grid)
        grid[0][0] = 1
        q = deque([(0, 0)])
        ans = 1
        while q:
            for _ in range(len(q)):
                i, j = q.popleft()
                if i == j == n - 1:
                    return ans
                for x in range(i - 1, i + 2):
                    for y in range(j - 1, j + 2):
                        if 0 <= x < n and 0 <= y < n and grid[x][y] == 0:
                            grid[x][y] = 1
                            q.append((x, y))
            ans += 1
        return -1
```

Complexity. Time: O(n^2). Space: O(n^2).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the path itself
# 2. Only 4 directions (no diagonal steps)


# ============================================================
# FOLLOW-UP 1: Return the path itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember the previous cell of every cell
```

```
# reached, and walk back from the bottom-right corner.
# Time: O(n^2).  Space: O(n^2).
from collections import deque


def shortest_path_cells(grid):
    n = len(grid)
    if grid[0][0] == 1 or grid[n - 1][n - 1] == 1:
        return []
    previous = {(0, 0): None}
    queue = deque([(0, 0)])
    while queue:
        r, c = queue.popleft()
        if (r, c) == (n - 1, n - 1):
            path = []
            while (r, c) is not None and (r, c) in previous:
                path.append((r, c))
                step = previous[(r, c)]
                if step is None:
                    break
                r, c = step
            path.reverse()
            return path
        for dr in (-1, 0, 1):
            for dc in (-1, 0, 1):
                nr, nc = r + dr, c + dc
                if 0 <= nr < n and 0 <= nc < n and grid[nr][nc] == 0 and (nr, nc) not in previous:
                    previous[(nr, nc)] = (r, c)
                    queue.append((nr, nc))
    return []


# ============================================================
# FOLLOW-UP 2: Only 4 directions (no diagonal steps)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Use four neighbours instead of eight.
# Time: O(n^2).  Space: O(n^2).
from collections import deque


def shortest_path_4(grid):
    n = len(grid)
    if grid[0][0] == 1 or grid[n - 1][n - 1] == 1:
        return -1
    queue = deque([(0, 0, 1)])
    seen = {(0, 0)}
    while queue:
```

```
        r, c, length = queue.popleft()
        if (r, c) == (n - 1, n - 1):
            return length
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < n and 0 <= nc < n and grid[nr][nc] == 0 and (nr, nc) not in seen:
                seen.add((nr, nc))
                queue.append((nr, nc, length + 1))
    return -1
```

PATTERN 17 · GRAPH BFS · PROBLEM 4 OF 20

752. Open the Lock

Medium · Breadth-First Search, Array, Hash Table, String, Bidirectional Search

leetcode.com/problems/open-the-lock

★ **TOP 7 PROBLEM** in Graph BFS: master this one first

PROBLEM

You have a lock in front of you with 4 circular wheels. Each wheel has 10 slots: `'0', '1', '2', '3', '4', '5', '6', '7', '8', '9'`. The wheels can rotate freely and wrap around: for example we can turn `'9'` to be `'0'`, or `'0'` to be `'9'`. Each move consists of turning one wheel one slot.

The lock initially starts at `'0000'`, a string representing the state of the 4 wheels.

You are given a list of `deadends` dead ends, meaning if the lock displays any of these codes, the wheels of the lock will stop turning and you will be unable to open it.

Given a `target` representing the value of the wheels that will unlock the lock, return the minimum total number of turns required to open the lock, or -1 if it is impossible.

Example 1:

```
Input: deadends = ["0201","0101","0102","1212","2002"], target = "0202"
Output: 6
Explanation:
A sequence of valid moves would be "0000" -> "1000" -> "1100" -> "1200" -> "1201" -> "1202" -> "0202".
Note that a sequence like "0000" -> "0001" -> "0002" -> "0102" -> "0202" would be invalid,
because the wheels of the lock become stuck after the display becomes the dead end "0102".
```

Example 2:

```
Input: deadends = ["8888"], target = "0009"
Output: 1
Explanation: We can turn the last wheel in reverse to move from "0000" -> "0009".
```

Example 3:

```
Input: deadends = ["8887","8889","8878","8898","8788","8988","7888","9888"], target = "8888"
Output: -1
Explanation: We cannot reach the target without getting stuck.
```

Constraints:

- `1 <= deadends.length <= 500`
- `deadends[i].length == 4`
- `target.length == 4`
- target **will not be** in the list `deadends`.
- `target` and `deadends[i]` consist of digits only.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

A 4-digit lock, one wheel per move, avoiding dead ends. 10^4 states and 8 neighbors each: unweighted shortest path is BFS. Dead ends and visited codes share a set. If `0000` is dead the answer is -1; if it is the target, 0. Expand by layer; neighbors flip one digit by ± 1 on a wheel. The first time we see the target is the distance.

Python 3

```python
class Solution:
    def openLock(self, deadends: List[str], target: str) -> int:
        def next(s):
            res = []
            s = list(s)
            for i in range(4):
                c = s[i]
                s[i] = '9' if c == '0' else str(int(c) - 1)
                res.append(''.join(s))
                s[i] = '0' if c == '9' else str(int(c) + 1)
                res.append(''.join(s))
                s[i] = c
            return res

        if target == '0000':
            return 0
        s = set(deadends)
        if '0000' in s:
            return -1
        q = deque([('0000')])
        s.add('0000')
        ans = 0
        while q:
            ans += 1
            for _ in range(len(q)):
                p = q.popleft()
                for t in next(p):
                    if t == target:
                        return ans
                    if t not in s:
                        q.append(t)
                        s.add(t)
        return -1
```

Complexity. Time: O(D * 10^D) with D = 4 wheels, so a constant. Space: O(10^D).

Solution 2

How to think about it

One-sided BFS may expand most of the cube. The two ends are known, so bidirectional BFS meets in the middle with smaller fronts. Always extend the smaller queue. A neighbor already seen from the other side yields step+1+m_2[t]. Dead ends are skipped on both sides.

Python 3

```python
class Solution:
    def openLock(self, deadends: List[str], target: str) -> int:
        def next(s):
            res = []
            s = list(s)
            for i in range(4):
                c = s[i]
                s[i] = '9' if c == '0' else str(int(c) - 1)
                res.append(''.join(s))
                s[i] = '0' if c == '9' else str(int(c) + 1)
                res.append(''.join(s))
                s[i] = c
            return res

        def extend(m1, m2, q):
            for _ in range(len(q)):
                p = q.popleft()
                step = m1[p]
                for t in next(p):
                    if t in s or t in m1:
                        continue
                    if t in m2:
                        return step + 1 + m2[t]
                    m1[t] = step + 1
                    q.append(t)
            return -1

        def bfs():
            m1, m2 = {"0000": 0}, {target: 0}
            q1, q2 = deque([('0000')]), deque([(target)])
            while q1 and q2:
                t = extend(m1, m2, q1) if len(q1) <= len(q2) else extend(m2, m1, q2)
                if t != -1:
                    return t
            return -1

        if target == '0000':
            return 0
        s = set(deadends)
        if '0000' in s:
            return -1
        return bfs()
```

Complexity. Time: O(D * 10^D) in the worst case, but much faster in practice (two searches meet in the middle). Space: O(10^D).

Solution 3

> **How to think about it**
>
> Bidirectional BFS still expands uniformly. The sum of circular digit distances to the target is an admissible heuristic, so A* pops the target first with dist+f as the key. f never exceeds the true remaining moves (ignoring dead ends). Skip dead codes; decrease-key via a dist map.

Python 3

```python
class Solution:
    def openLock(self, deadends: List[str], target: str) -> int:
        def next(s):
            res = []
            s = list(s)
            for i in range(4):
                c = s[i]
                s[i] = '9' if c == '0' else str(int(c) - 1)
                res.append(''.join(s))
                s[i] = '0' if c == '9' else str(int(c) + 1)
                res.append(''.join(s))
                s[i] = c
            return res

        def f(s):
            ans = 0
            for i in range(4):
                a = ord(s[i]) - ord('0')
                b = ord(target[i]) - ord('0')
                if a > b:
                    a, b = b, a
                ans += min(b - a, a + 10 - b)
            return ans

        if target == '0000':
            return 0
        s = set(deadends)
        if '0000' in s:
            return -1
        start = '0000'
        q = [(f(start), start)]
        dist = {start: 0}
        while q:
            _, state = heappop(q)
            if state == target:
                return dist[state]
            for t in next(state):
                if t in s:
                    continue
                if t not in dist or dist[t] > dist[state] + 1:
                    dist[t] = dist[state] + 1
                    heappush(q, (dist[t] + f(t), t))
        return -1
```

Complexity. Time: O(D * 10^D * log(10^D)) in the worst case with D = 4 wheels (A* only reorders the search). Space: O(10^D).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the sequence of combinations to turn through
# 2. Bidirectional BFS (search from both ends)


# ============================================================
# FOLLOW-UP 1: Return the sequence of combinations to turn
# through
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember the previous combination of each one
# reached and walk back from the target.
# Time: O(D * 10^D), D = 4 wheels.  Space: O(10^D).
from collections import deque


def lock_sequence(deadends, target):
```

```
    dead = set(deadends)
    if "0000" in dead:
        return None
    previous = {"0000": None}
    queue = deque(["0000"])
    while queue:
        code = queue.popleft()
        if code == target:
            path = []
            while code is not None:
                path.append(code)
                code = previous[code]
            path.reverse()
            return path
        for i in range(4):
            digit = int(code[i])
            for step in (1, -1):
                new = code[:i] + str((digit + step) % 10) + code[i + 1:]
                if new not in previous and new not in dead:
                    previous[new] = code
                    queue.append(new)
    return None


# ============================================================
# FOLLOW-UP 2: Bidirectional BFS (search from both ends)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Grow a search from the start and one from the
# target, always expanding the smaller side; stop when
# they meet. This explores far fewer combinations.
# Time: O(D * 10^D) in the worst case, much faster in
# practice.  Space: O(10^D).
def open_lock_bidirectional(deadends, target):
    dead = set(deadends)
    if "0000" in dead or target in dead:
        return -1
    if target == "0000":
        return 0
    front = {"0000"}
    back = {target}
```

```
    seen = {"0000", target}
    moves = 0
    while front and back:
        if len(front) > len(back):
            front, back = back, front
        moves += 1
        following = set()
        for code in front:
            for i in range(4):
                digit = int(code[i])
                for step in (1, -1):
                    new = code[:i] + str((digit + step) % 10) + code[i + 1:]
                    if new in back:
                        return moves
                    if new not in seen and new not in dead:
                        seen.add(new)
                        following.add(new)
        front = following
    return -1
```

909. Snakes and Ladders

Medium · Breadth-First Search, Array, Matrix

leetcode.com/problems/snakes-and-ladders

Also appears in pattern 36 Simulation / Array Rearrangement

PROBLEM

You are given an `n x n` integer matrix `board` where the cells are labeled from `1` to n^2 in a **Boustrophedon style** starting from the bottom left of the board (i.e. `board[n - 1][0]`) and alternating direction each row.

You start on square `1` of the board. In each move, starting from square `curr`, do the following:

- Choose a destination square `next` with a label in the range `[curr + 1, min(curr + 6, n^2)]`.
  - This choice simulates the result of a standard **6-sided die roll**: i.e., there are always at most 6 destinations, regardless of the size of the board.
- If `next` has a snake or ladder, you **must** move to the destination of that snake or ladder. Otherwise, you move to `next`.
- The game ends when you reach the square n^2.

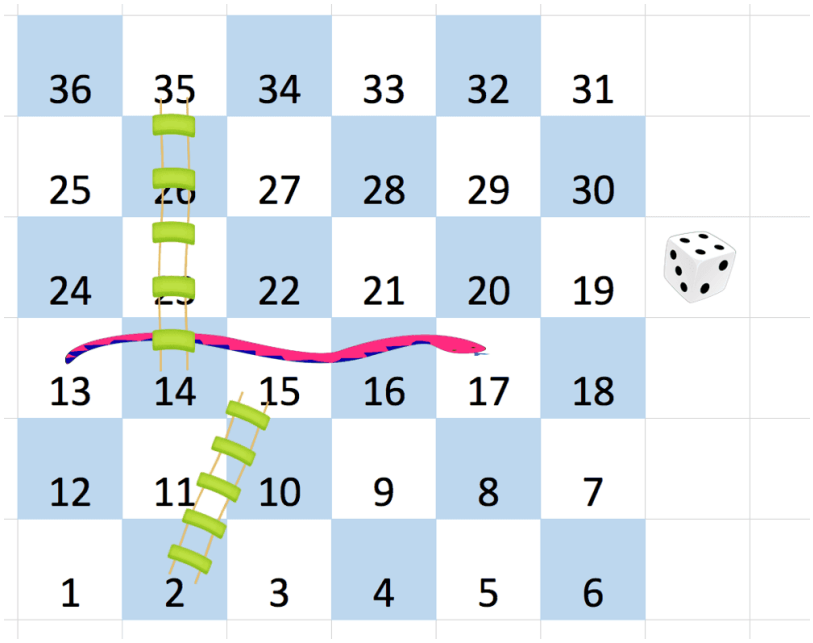
A board square on row `r` and column `c` has a snake or ladder if `board[r][c] != -1`. The destination of that snake or ladder is `board[r][c]`. Squares `1` and n^2 are not the starting points of any snake or ladder.

Note that you only take a snake or ladder at most once per dice roll. If the destination to a snake or ladder is the start of another snake or ladder, you do **not** follow the subsequent snake or ladder.

- For example, suppose the board is `[[-1,4],[-1,3]]`, and on the first move, your destination square is `2`. You follow the ladder to square `3`, but do **not** follow the subsequent ladder to `4`.

Return *the least number of dice rolls required to reach the square* n^2*. If it is not possible to reach the square, return* `-1`.

Example 1:



```
Input: board = [[-1,-1,-1,-1,-1,-1],[-1,-1,-1,-1,-1,-1],[-1,-1,-1,-1,-1,-1],[-1,35,-1,-1,13,-1],[-1,-1
,-1,-1,-1,-1],[-1,15,-1,-1,-1,-1]]
Output: 4
Explanation:
In the beginning, you start at square 1 (at row 5, column 0).
You decide to move to square 2 and must take the ladder to square 15.
You then decide to move to square 17 and must take the snake to square 13.
You then decide to move to square 14 and must take the ladder to square 35.
You then decide to move to square 36, ending the game.
This is the lowest possible number of moves to reach the last square, so return 4.
```

Example 2:

```
Input: board = [[-1,-1],[-1,3]]
Output: 1
```

Constraints:

- `n == board.length == board[i].length`
- `2 <= n <= 20`
- `board[i][j]` is either `-1` or in the range `[1, n^2]`.
- The squares labeled `1` and n^2 are not the starting points of any snake or ladder.

SOLUTION (PYTHON 3)

Solution 1: BFS

How to think about it

The board is at most 20× 20. The fewest die rolls is an unweighted shortest path. Labels snake through the grid; a snake or ladder teleports to its destination. BFS from 1 by layers, visiting each cell once. The layer that first reaches n^2 is the answer; if the queue empties, return -1.

We can use the Breadth-First Search (BFS) method, starting from the starting point, moving forward 1 to 6 steps each time, and then checking for snakes or ladders. If there are any, move to the destination of the snake or ladder; otherwise, move to the next square.

Specifically, we use a queue q to store the current reachable square numbers, initially putting number 1 into the queue. At the same time, we use a set vis to record the squares that have been reached to avoid revisiting them, initially adding number 1 to the set vis.

In each operation, we take out the square number x at the front of the queue. If x is the endpoint, we can return the current number of steps. Otherwise, we move x forward 1 to 6 steps, setting the new number as y. If y falls outside the board, we skip it directly. Otherwise, we need to find the row and column corresponding to y. Since the row numbers decrease from bottom to top, and the column numbers depend on the parity of the row, we need to perform some calculations to get the row and column corresponding to y.

If the square corresponding to y has a snake or ladder, we need to move to the destination of the snake or ladder, denoted as z. If z has not been visited, we add z to the queue and set, allowing us to continue the breadth-first search.

If we ultimately cannot reach the endpoint, we return -1.

The time complexity is $O(n^2)$, and the space complexity is $O(n^2)$. Here, n is the length of the side of the board.

Python 3

```
class Solution:
    def snakesAndLadders(self, board: List[List[int]]) -> int:
        n = len(board)
        q = deque([1])
        vis = {1}
        ans = 0
        m = n * n
        while q:
            for _ in range(len(q)):
                x = q.popleft()
                if x == m:
                    return ans
                for y in range(x + 1, min(x + 6, m) + 1):
                    i, j = divmod(y - 1, n)
                    if i & 1:
                        j = n - j - 1
                    i = n - i - 1
                    z = y if board[i][j] == -1 else board[i][j]
                    if z not in vis:
                        vis.add(z)
                        q.append(z)
            ans += 1
        return -1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the squares you land on along the shortest
# route
# 2. A die with d faces instead of 6


# ============================================================
# FOLLOW-UP 1: Return the squares you land on along the
# shortest route
# ============================================================

# ---- follow-up 1: helper functions ----
def square_to_cell(square, n):
    index = square - 1
    row = index // n
    col = index % n
    if row % 2 == 1:
        col = n - 1 - col
    return n - 1 - row, col

# ---- follow-up 1: function ----
# Answer: Remember the previous square of each square
# reached and walk back from the last square.
# Time: O(n^2 * 6).  Space: O(n^2).
from collections import deque


def snakes_ladders_route(board):
    n = len(board)
    last = n * n
    previous = {1: None}
    queue = deque([1])
    while queue:
        square = queue.popleft()
        if square == last:
            route = []
            while square is not None:
                route.append(square)
                square = previous[square]
```

```
            route.reverse()
            return route
        for roll in range(1, 7):
            nxt = square + roll
            if nxt > last:
                break
            r, c = square_to_cell(nxt, n)
            if board[r][c] != -1:
                nxt = board[r][c]
            if nxt not in previous:
                previous[nxt] = square
                queue.append(nxt)
    return None


# ============================================================
# FOLLOW-UP 2: A die with d faces instead of 6
# ============================================================

# ---- follow-up 2: function ----
# Answer: Only the number of moves from each square
```

```
# changes.
# Time: O(n^2 * d), d = faces.  Space: O(n^2).
from collections import deque


def snakes_ladders_with_dice(board, faces):
    n = len(board)
    last = n * n
    moves = {1: 0}
    queue = deque([1])
    while queue:
        square = queue.popleft()
        if square == last:
            return moves[square]
        for roll in range(1, faces + 1):
            nxt = square + roll
            if nxt > last:
                break
            r, c = square_to_cell(nxt, n)
            if board[r][c] != -1:
                nxt = board[r][c]
            if nxt not in moves:
                moves[nxt] = moves[square] + 1
                queue.append(nxt)
    return -1
```

1926. Nearest Exit from Entrance in Maze

Medium · Breadth-First Search, Array, Matrix

leetcode.com/problems/nearest-exit-from-entrance-in-maze

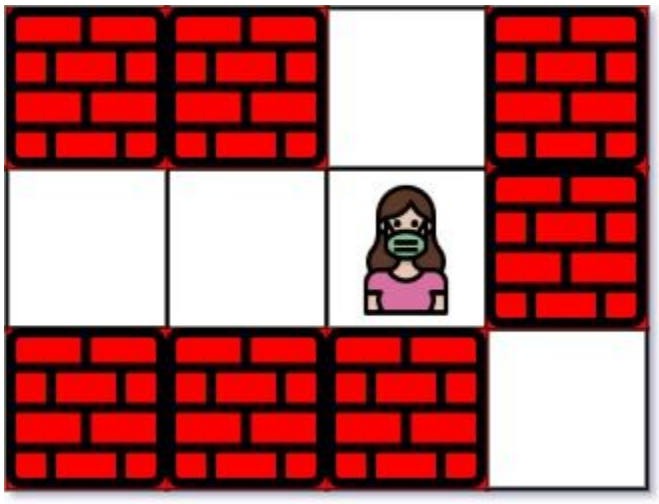
PROBLEM

You are given an `m x n` matrix `maze` (**0-indexed**) with empty cells (represented as `'.'`) and walls (represented as `'+'`). You are also given the `entrance` of the maze, where `entrance = [entrance_row, entrance_col]` denotes the row and column of the cell you are initially standing at.

In one step, you can move one cell **up**, **down**, **left**, or **right**. You cannot step into a cell with a wall, and you cannot step outside the maze. Your goal is to find the **nearest exit** from the `entrance`. An **exit** is defined as an **empty cell** that is at the **border** of the `maze`. The `entrance` **does not count** as an exit.

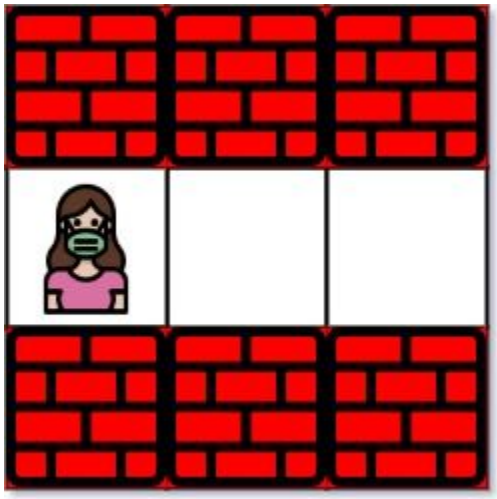
Return *the **number of steps** in the shortest path from the* `entrance` *to the nearest exit, or* `-1` *if no such path exists*.

Example 1:



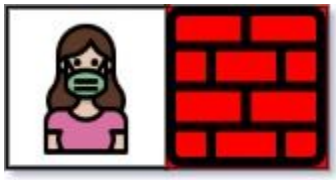
```
Input: maze = [["+","+",".","+"],[".",".",".","+"],["+","+","+","."]], entrance = [1,2]
Output: 1
Explanation: There are 3 exits in this maze at [1,0], [0,2], and [2,3].
Initially, you are at the entrance cell [1,2].
- You can reach [1,0] by moving 2 steps left.
- You can reach [0,2] by moving 1 step up.
It is impossible to reach [2,3] from the entrance.
Thus, the nearest exit is [0,2], which is 1 step away.
```

Example 2:



```
Input: maze = [["+","+","+"],[".",".","."],["+","+","+"]], entrance = [1,0]
Output: 2
Explanation: There is 1 exit in this maze at [1,2].
[1,0] does not count as an exit since it is the entrance cell.
Initially, you are at the entrance cell [1,0].
- You can reach [1,2] by moving 2 steps right.
Thus, the nearest exit is [1,2], which is 2 steps away.
```

Example 3:



```
Input: maze = [[".","+"]], entrance = [0,0]
Output: -1
Explanation: There are no exits in this maze.
```

Constraints:

- `maze.length == m`
- `maze[i].length == n`
- `1 <= m, n <= 100`
- `maze[i][j]` is either `'.'` or `'+'`.
- `entrance.length == 2`
- `0 <= entrance`$_{row}$ `< m`
- `0 <= entrance`$_{col}$ `< n`
- `entrance` will always be an empty cell.

SOLUTION (PYTHON 3)

Solution 1: BFS

How to think about it

The nearest exit is an unweighted shortest path. DFS does not guarantee minimality. The entrance is not an exit even if it lies on the border. BFS from the entrance, painting visited empty cells as walls. The first time a border empty cell is reached, the current distance is the answer; an empty queue means unreachable. Expanding four-neighbors level by level keeps the step count equal to distance.

We can start from the entrance and perform a breadth-first search (BFS). Each time we reach a new empty cell, we mark it as visited and add it to the queue until we find an empty cell on the boundary, then return the number of steps.

Specifically, we define a queue q, initially adding entrance to the queue. We define a variable ans to record the number of steps, initially set to 1. Then we start the BFS. In each round, we take out all elements from the queue and traverse them. For each element, we try to move in four directions. If the new position is an empty cell, we add it to the queue and mark it as visited. If the new position is an empty cell on the boundary, we return ans. If the queue is empty, we return -1. After this round of search, we increment ans by one and continue to the next round of search.

If we finish the traversal without finding an empty cell on the boundary, we return -1.

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the number of rows and columns in the maze, respectively.

Python 3

```
class Solution:
    def nearestExit(self, maze: List[List[str]], entrance: List[int]) -> int:
        m, n = len(maze), len(maze[0])
        i, j = entrance
        q = deque([(i, j)])
        maze[i][j] = "+"
        ans = 0
        while q:
            ans += 1
            for _ in range(len(q)):
                i, j = q.popleft()
                for a, b in [[0, -1], [0, 1], [-1, 0], [1, 0]]:
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n and maze[x][y] == ".":
                        if x == 0 or x == m - 1 or y == 0 or y == n - 1:
                            return ans
                        q.append((x, y))
                        maze[x][y] = "+"
        return -1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return WHICH exit is the nearest
# 2. How many different exits can be reached?


# ============================================================
# FOLLOW-UP 1: Return WHICH exit is the nearest
# ============================================================

# ---- follow-up 1: function ----
# Answer: The BFS stops at the first border cell (other
# than the entrance); report that cell together with the
# distance.
# Time: O(m * n).  Space: O(m * n).
from collections import deque


def nearest_exit_cell(maze, entrance):
    rows, cols = len(maze), len(maze[0])
    queue = deque([(entrance[0], entrance[1], 0)])
    seen = {(entrance[0], entrance[1])}
    while queue:
        r, c, steps = queue.popleft()
        if (r, c) != tuple(entrance) and (r == 0 or c == 0 or r == rows - 1 or c == cols - 1):
            return (r, c), steps
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and maze[nr][nc] == "." and (nr, nc) not in seen:
                seen.add((nr, nc))
```

```
                queue.append((nr, nc, steps + 1))
    return None


# ============================================================
# FOLLOW-UP 2: How many different exits can be reached?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Search the whole reachable area and count the
# border cells (not the entrance).
# Time: O(m * n).  Space: O(m * n).
from collections import deque


def count_reachable_exits(maze, entrance):
    rows, cols = len(maze), len(maze[0])
    queue = deque([(entrance[0], entrance[1])])
    seen = {(entrance[0], entrance[1])}
    exits = 0
    while queue:
        r, c = queue.popleft()
        if (r, c) != tuple(entrance) and (r == 0 or c == 0 or r == rows - 1 or c == cols - 1):
            exits += 1
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and maze[nr][nc] == "." and (nr, nc) not in seen:
                seen.add((nr, nc))
                queue.append((nr, nc))
    return exits
```

1162. As Far from Land as Possible

Medium · Breadth-First Search, Array, Dynamic Programming, Matrix

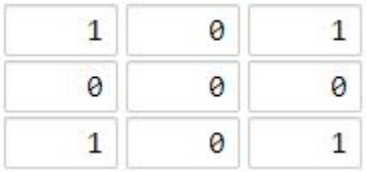
leetcode.com/problems/as-far-from-land-as-possible

PROBLEM

Given an `n x n grid` containing only values `0` and `1`, where `0` represents water and `1` represents land, find a water cell such that its distance to the nearest land cell is maximized, and return the distance. If no land or water exists in the grid, return `-1`.

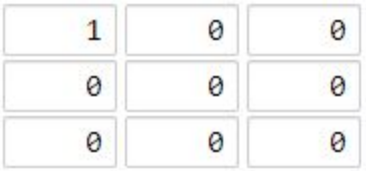
The distance used in this problem is the Manhattan distance: the distance between two cells `(x0, y0)` and `(x1, y1)` is `|x0 - x1| + |y0 - y1|`.

Example 1:




```
Input: grid = [[1,0,1],[0,0,0],[1,0,1]]
Output: 2
Explanation: The cell (1, 1) is as far as possible from all the land with distance 2.
```

Example 2:




```
Input: grid = [[1,0,0],[0,0,0],[0,0,0]]
Output: 4
Explanation: The cell (2, 2) is as far as possible from all the land with distance 4.
```

Constraints:

- `n == grid.length`
- `n == grid[i].length`
- `1 <= n <= 100`
- `grid[i][j]` is `0` or `1`

SOLUTION (PYTHON 3)

Solution 1: BFS

How to think about it

We want the maximum, over water cells, of the distance to the nearest land. A BFS from every water cell repeats visits. Multi-source BFS from all land cells grows one layer per step; the last water painted is that maximum. All-land or all-water grids have no answer, so return -1.

We can add all land cells to the queue q. If the queue is empty, or the number of elements in the queue equals the number of cells in the grid, it means that the grid contains only land or ocean, so return -1.

Otherwise, we start BFS from the land cells. Define the initial step count ans=-1.

In each round of search, we spread all cells in the queue in four directions. If a cell is an ocean cell, we mark it as a land cell and add it to the queue. After a round of spreading, we increase the step count by 1. Repeat this process until the queue is empty.

Finally, we return the step count ans.

The time complexity is $O(n^2)$, and the space complexity is $O(n^2)$. Here, n is the side length of the grid.

Python 3

```
class Solution:
    def maxDistance(self, grid: List[List[int]]) -> int:
        n = len(grid)
        q = deque((i, j) for i in range(n) for j in range(n) if grid[i][j])
        ans = -1
        if len(q) in (0, n * n):
            return ans
        dirs = (-1, 0, 1, 0, -1)
        while q:
            for _ in range(len(q)):
                i, j = q.popleft()
                for a, b in pairwise(dirs):
                    x, y = i + a, j + b
                    if 0 <= x < n and 0 <= y < n and grid[x][y] == 0:
                        grid[x][y] = 1
                        q.append((x, y))
            ans += 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the water cell that is farthest from land (and
# its distance)


# ============================================================
# FOLLOW-UP 1: Return the water cell that is farthest from
# land (and its distance)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Multi-source BFS from all the land cells; the
# last cell taken from the queue is a farthest water cell.
# Time: O(n^2).  Space: O(n^2).
from collections import deque


def farthest_water_cell(grid):
    n = len(grid)
    queue = deque()
    dist = {}
    for r in range(n):
        for c in range(n):
            if grid[r][c] == 1:
                dist[(r, c)] = 0
                queue.append((r, c))
    if len(queue) == 0 or len(queue) == n * n:
        return None
    last = None
    while queue:
        r, c = queue.popleft()
        last = (r, c)
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < n and 0 <= nc < n and (nr, nc) not in dist:
                dist[(nr, nc)] = dist[(r, c)] + 1
                queue.append((nr, nc))
```

```
    return last, dist[last]
```

1129. Shortest Path with Alternating Colors

Medium · Breadth-First Search, Graph

leetcode.com/problems/shortest-path-with-alternating-colors

PROBLEM

You are given an integer `n`, the number of nodes in a directed graph where the nodes are labeled from `0` to `n - 1`. Each edge is red or blue in this graph, and there could be self-edges and parallel edges.

You are given two arrays `redEdges` and `blueEdges` where:

- `redEdges[i] = [a_i, b_i]` indicates that there is a directed red edge from node a_i to node b_i in the graph, and
- `blueEdges[j] = [u_j, v_j]` indicates that there is a directed blue edge from node u_j to node v_j in the graph.

Return an array `answer` of length `n`, where each `answer[x]` is the length of the shortest path from node `0` to node `x` such that the edge colors alternate along the path, or `-1` if such a path does not exist.

Example 1:

```
Input: n = 3, redEdges = [[0,1],[1,2]], blueEdges = []
Output: [0,1,-1]
```

Example 2:

```
Input: n = 3, redEdges = [[0,1]], blueEdges = [[2,1]]
Output: [0,1,-1]
```

Constraints:

- `1 <= n <= 100`
- `0 <= redEdges.length, blueEdges.length <= 400`
- `redEdges[i].length == blueEdges[j].length == 2`
- `0 <= a_i, b_i, u_j, v_j < n`

SOLUTION (PYTHON 3)

Solution 1: BFS

> **How to think about it**
>
> Edges are red or blue and consecutive edges must differ, so a node may be visited twice with different incoming colors. The state is (node,last color). BFS from 0 with both virtual colors; the first time a node is reached is the shortest path. Adjacency is stored by color and the color is flipped on each hop so same-color edges are never taken.

The problem is essentially a shortest path problem, which we can consider solving using BFS.

First, we preprocess all the edges, categorizing all the edges by color and storing them in a multi-dimensional array g. Where g[0] stores all red edges, and g[1] stores all blue edges.

Next, we define the following data structures or variables:

- Queue q: used to store the currently searched node and the color of the current edge;
- Set vis: used to store the nodes that have been searched and the color of the current edge;
- Variable d: used to represent the current search level, i.e., the distance from the currently searched node to the starting point;
- Array ans: used to store the shortest distance from each node to the starting point. Initially, we initialize all elements in the ans array to -1, indicating that the distance from all nodes to the starting point is unknown.

We first enqueue the starting point 0 and the color of the starting edge 0 or 1, indicating that we start from the starting point and the current edge is red or blue.

Next, we start the BFS search. Each time we take out a node (i, c) from the queue, if the answer of the current node has not been updated, then we update the answer of the current node to the current level d, i.e., ans[i] = d. Then, we flip the color of the current edge c, i.e., if the current edge is red, we change it to blue, and vice versa. We take out all edges corresponding to the color, if the other end node j of the edge has not been searched, then we enqueue it.

After the search is over, return the answer array.

The time complexity is O(n + m), and the space complexity is O(n + m). Here, n and m are the number of nodes and edges, respectively.

Python 3

```
class Solution:
    def shortestAlternatingPaths(
        self, n: int, redEdges: List[List[int]], blueEdges: List[List[int]]
    ) -> List[int]:
        g = [defaultdict(list), defaultdict(list)]
        for i, j in redEdges:
            g[0][i].append(j)
        for i, j in blueEdges:
            g[1][i].append(j)
        ans = [-1] * n
        vis = set()
        q = deque([(0, 0), (0, 1)])
        d = 0
        while q:
            for _ in range(len(q)):
                i, c = q.popleft()
                if ans[i] == -1:
                    ans[i] = d
                vis.add((i, c))
                c ^= 1
                for j in g[c][i]:
                    if (j, c) not in vis:
                        q.append((j, c))
            d += 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the actual route to one target node
# 2. How many nodes can be reached by an alternating path
# at all?


# ============================================================
# FOLLOW-UP 1: Return the actual route to one target node
# ============================================================

# ---- follow-up 1: function ----
# Answer: The BFS state is (node, colour of the last
# edge); remember the previous state and walk back.
# Time: O(n + E).  Space: O(n + E).
from collections import deque


def alternating_route(n, red_edges, blue_edges, target):
    graph = {0: [[] for i in range(n)], 1: [[] for i in range(n)]}
    for a, b in red_edges:
        graph[0][a].append(b)
    for a, b in blue_edges:
        graph[1][a].append(b)
    previous = {(0, -1): None}
    queue = deque([(0, -1)])
    while queue:
        node, colour = queue.popleft()
```

```
        if node == target:
            route = []
            state = (node, colour)
            while state is not None:
                route.append(state[0])
                state = previous[state]
            route.reverse()
            return route
        for next_colour in (0, 1):
            if next_colour == colour:
                continue
            for other in graph[next_colour][node]:
                if (other, next_colour) not in previous:
                    previous[(other, next_colour)] = (node, colour)
                    queue.append((other, next_colour))
    return None


# ============================================================
# FOLLOW-UP 2: How many nodes can be reached by an
# alternating path at all?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Run the same search and count the different
# nodes that appear in any state.
# Time: O(n + E).  Space: O(n + E).
from collections import deque


def reachable_alternating(n, red_edges, blue_edges):
    graph = {0: [[] for i in range(n)], 1: [[] for i in range(n)]}
    for a, b in red_edges:
        graph[0][a].append(b)
    for a, b in blue_edges:
        graph[1][a].append(b)
    seen = {(0, -1)}
    queue = deque([(0, -1)])
    nodes = {0}
    while queue:
        node, colour = queue.popleft()
        for next_colour in (0, 1):
            if next_colour == colour:
                continue
            for other in graph[next_colour][node]:
                if (other, next_colour) not in seen:
                    seen.add((other, next_colour))
                    nodes.add(other)
                    queue.append((other, next_colour))
    return len(nodes)
```

PATTERN 17 · GRAPH BFS · PROBLEM 9 OF 20

1306. Jump Game III

Medium · Depth-First Search, Breadth-First Search, Array

leetcode.com/problems/jump-game-iii

PROBLEM

Given an array of non-negative integers `arr`, you are initially positioned at `start` index of the array. When you are at index `i`, you can jump to `i + arr[i]` or `i - arr[i]`, check if you can reach **any** index with value 0.

Notice that you can not jump outside of the array at any time.

Example 1:

```
Input: arr = [4,2,3,0,3,1,2], start = 5
Output: true
Explanation:
All possible ways to reach at index 3 with value 0 are:
index 5 -> index 4 -> index 1 -> index 3
index 5 -> index 6 -> index 4 -> index 1 -> index 3
```

Example 2:

```
Input: arr = [4,2,3,0,3,1,2], start = 0
Output: true
Explanation:
One possible way to reach at index 3 with value 0 is:
index 0 -> index 4 -> index 1 -> index 3
```

Example 3:

```
Input: arr = [3,0,2,1,2], start = 2
Output: false
Explanation: There is no way to reach at index 1 with value 0.
```

Constraints:

- `1 <= arr.length <= 5 * 10^4`
- `0 <= arr[i] < arr.length`
- `0 <= start < arr.length`

SOLUTION (PYTHON 3)

Solution 1: BFS

> **How to think about it**
>
> From start we jump to i ± arr[i] and ask whether a 0 is reachable. With $n \le 5 \times 10^4$, revisiting the same index loops forever. Each index needs to enter the queue at most once. The search is ordinary BFS on that graph: a dequeued 0 succeeds; otherwise we mark the cell visited (write -1) and enqueue the two in-range, unvisited jumps. The mark is the prune that keeps the work O(n).

We can use BFS to determine whether we can reach the index with a value of 0.

Define a queue q to store the currently reachable indices. Initially, enqueue the start index.

When the queue is not empty, take out the front index i of the queue. If arr[i] = 0, return `true`. Otherwise, mark the index i as visited. If i + arr[i] and i - arr[i] are within the array range and have not been visited, enqueue them and continue searching.

Finally, if the queue is empty, it means that we cannot reach the index with a value of 0, so return `false`.

The time complexity is O(n), and the space complexity is O(n). Where n is the length of the array.

Python 3

```python
class Solution:
    def canReach(self, arr: List[int], start: int) -> bool:
        q = deque([start])
        while q:
            i = q.popleft()
            if arr[i] == 0:
                return True
            x = arr[i]
            arr[i] = -1
            for j in (i + x, i - x):
                if 0 <= j < len(arr) and arr[j] >= 0:
                    q.append(j)
        return False
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the sequence of indexes that leads to a 0
# 2. How many different zeros can be reached?


# ============================================================
# FOLLOW-UP 1: Return the sequence of indexes that leads
# to a 0
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember where each index was reached from and
# walk back from the first 0 found.
# Time: O(n).  Space: O(n).
from collections import deque


def path_to_zero(arr, start):
    previous = {start: None}
    queue = deque([start])
    while queue:
        i = queue.popleft()
        if arr[i] == 0:
            path = []
            while i is not None:
                path.append(i)
                i = previous[i]
            path.reverse()
            return path
        for j in (i + arr[i], i - arr[i]):
            if 0 <= j < len(arr) and j not in previous:
                previous[j] = i
                queue.append(j)
    return None


# ============================================================
# FOLLOW-UP 2: How many different zeros can be reached?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Search everything reachable and count the zeros.
# Time: O(n).  Space: O(n).
def count_reachable_zeros(arr, start):
    seen = {start}
    stack = [start]
    zeros = 0
```

```
    while stack:
        i = stack.pop()
        if arr[i] == 0:
            zeros += 1
        for j in (i + arr[i], i - arr[i]):
            if 0 <= j < len(arr) and j not in seen:
                seen.add(j)
                stack.append(j)
    return zeros
```

433. Minimum Genetic Mutation

Medium · Breadth-First Search, Hash Table, String, Bidirectional Search

leetcode.com/problems/minimum-genetic-mutation

PROBLEM

A gene string can be represented by an 8-character long string, with choices from `'A'`, `'C'`, `'G'`, and `'T'`.

Suppose we need to investigate a mutation from a gene string `startGene` to a gene string `endGene` where one mutation is defined as one single character changed in the gene string.

- For example, `"AACCGGTT" --> "AACCGGTA"` is one mutation.

There is also a gene bank `bank` that records all the valid gene mutations. A gene must be in `bank` to make it a valid gene string.

Given the two gene strings `startGene` and `endGene` and the gene bank `bank`, return *the minimum number of mutations needed to mutate from* `startGene` *to* `endGene`. If there is no such a mutation, return `-1`.

Note that the starting point is assumed to be valid, so it might not be included in the bank.

Example 1:

```
Input: startGene = "AACCGGTT", endGene = "AACCGGTA", bank = ["AACCGGTA"]
Output: 1
```

Example 2:

```
Input: startGene = "AACCGGTT", endGene = "AAACGGTA", bank = ["AACCGGTA","AACCGCTA","AAACGGTA"]
Output: 2
```

Constraints:

- `0 <= bank.length <= 10`
- `startGene.length == endGene.length == bank[i].length == 8`
- `startGene`, `endGene`, and `bank[i]` consist of only the characters `['A', 'C', 'G', 'T']`.

SOLUTION (PYTHON 3)

Solution 1: BFS

How to think about it

A mutation flips one character and must land in the bank; we want the fewest steps. That is unweighted shortest path, so depth-first search does not give the minimum. BFS from the start. Each step scans unused bank strings that differ in exactly one position. The first time the end gene is dequeued is the answer; an empty queue means unreachable. The bank is tiny, so pairwise Hamming distance 1 is enough. A visited set prevents enqueueing a gene twice.

We define a queue `q` to store the current gene sequence and the number of changes, and a set `vis` to store the visited gene sequences. Initially, we add the starting gene sequence `start` to the queue `q` and the set `vis`.

Then, we continuously take out a gene sequence from the queue `q`. If this gene sequence equals the target gene sequence, we return the current number of changes. Otherwise, we iterate through the gene bank `bank`, calculate the difference value between the current gene sequence and the gene sequence in the gene bank. If the difference value is 1 and the gene sequence in the gene bank has not been visited, we add it to the queue `q` and the set `vis`.

If the queue `q` is empty, it means that the gene change cannot be completed, so we return -1.

The time complexity is $O(C \times n \times m)$, and the space complexity is $O(n \times m)$. Where n and m are the lengths of the gene sequence and the gene bank respectively, and C is the size of the character set of the gene sequence. In this problem, $C = 4$.

Python 3

```python
class Solution:
    def minMutation(self, startGene: str, endGene: str, bank: List[str]) -> int:
        q = deque([(startGene, 0)])
        vis = {startGene}
        while q:
            gene, depth = q.popleft()
            if gene == endGene:
                return depth
            for nxt in bank:
                diff = sum(a != b for a, b in zip(gene, nxt))
                if diff == 1 and nxt not in vis:
                    q.append((nxt, depth + 1))
                    vis.add(nxt)
        return -1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the sequence of mutations
# 2. The genes may have different lengths and a mutation
# can also insert or delete one letter


# ============================================================
# FOLLOW-UP 1: Return the sequence of mutations
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember the previous gene of each gene reached
# and walk back from the end gene.
# Time: O(N * L * 4), N genes, L length.  Space: O(N).
from collections import deque


def mutation_sequence(start, end, bank):
    valid = set(bank)
    if end not in valid:
        return None
    previous = {start: None}
    queue = deque([start])
    while queue:
        gene = queue.popleft()
        if gene == end:
            path = []
            while gene is not None:
                path.append(gene)
                gene = previous[gene]
            path.reverse()
            return path
        for i in range(len(gene)):
            for letter in "ACGT":
                if letter != gene[i]:
                    new = gene[:i] + letter + gene[i + 1:]
                    if new in valid and new not in previous:
                        previous[new] = gene
                        queue.append(new)
    return None


# ============================================================
# FOLLOW-UP 2: The genes may have different lengths and a
# mutation can also insert or delete one letter
# ============================================================

# ---- follow-up 2: function ----
```

```
# Answer: Add the insert and delete neighbours to the
# neighbour function; the BFS stays the same.
# Time: O(N * L^2).  Space: O(N).
from collections import deque


def min_edits_through_bank(start, end, bank):
    valid = set(bank)
    if end not in valid:
        return -1
    seen = {start}
    queue = deque([(start, 0)])
```

```
    while queue:
        gene, steps = queue.popleft()
        if gene == end:
            return steps
        neighbours = []
        for i in range(len(gene) + 1):
            for letter in "ACGT":
                neighbours.append(gene[:i] + letter + gene[i:])          # insert
        for i in range(len(gene)):
            neighbours.append(gene[:i] + gene[i + 1:])                   # delete
            for letter in "ACGT":
                neighbours.append(gene[:i] + letter + gene[i + 1:])      # replace
        for new in neighbours:
            if new in valid and new not in seen:
                seen.add(new)
                queue.append((new, steps + 1))
    return -1
```

787. Cheapest Flights Within K Stops

Medium · Depth-First Search, Breadth-First Search, Graph, Dynamic Programming, Shortest Path, Heap (Priority Queue)

leetcode.com/problems/cheapest-flights-within-k-stops

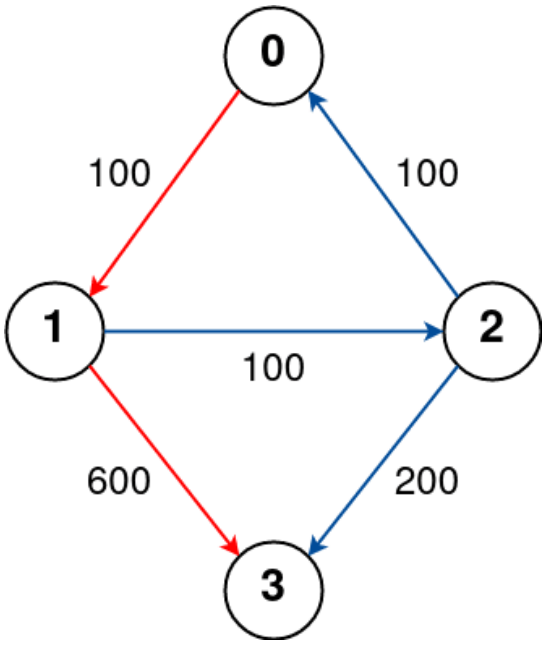
★ **TOP 7 PROBLEM** in Graph BFS: master this one first

PROBLEM

There are `n` cities connected by some number of flights. You are given an array `flights` where `flights[i] = [from`$_i$`, to`$_i$`, price`$_i$`]` indicates that there is a flight from city `from`$_i$ to city `to`$_i$ with cost `price`$_i$.

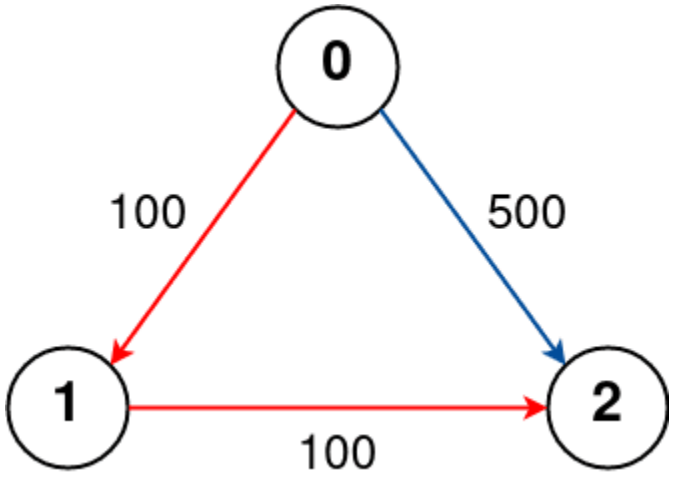
You are also given three integers `src`, `dst`, and `k`, return ***the cheapest price*** *from* `src` *to* `dst` *with at most* `k` *stops.* If there is no such route, return `-1`.

Example 1:



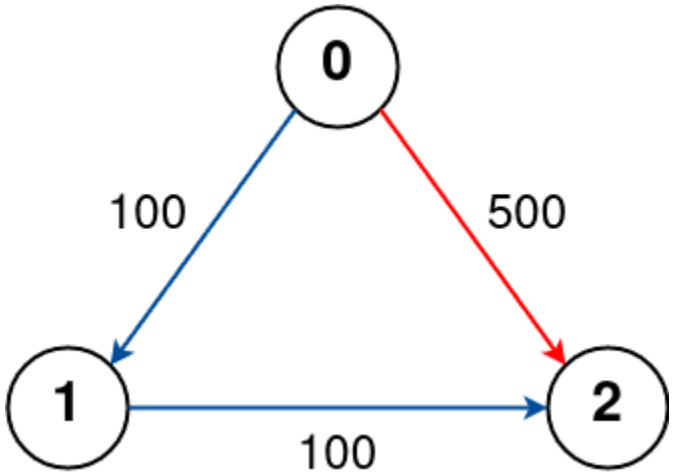

```
Input: n = 4, flights = [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]], src = 0, dst = 3, k = 1
Output: 700
Explanation:
The graph is shown above.
The optimal path with at most 1 stop from city 0 to 3 is marked in red and has cost 100 + 600 = 700.
Note that the path through cities [0,1,2,3] is cheaper but is invalid because it uses 2 stops.
```

Example 2:




```
Input: n = 3, flights = [[0,1,100],[1,2,100],[0,2,500]], src = 0, dst = 2, k = 1
Output: 200
Explanation:
The graph is shown above.
The optimal path with at most 1 stop from city 0 to 2 is marked in red and has cost 100 + 100 = 200.
```

Example 3:




```
Input: n = 3, flights = [[0,1,100],[1,2,100],[0,2,500]], src = 0, dst = 2, k = 0
Output: 500
Explanation:
The graph is shown above.
The optimal path with no stops from city 0 to 2 is marked in red and has cost 500.
```

Constraints:

- $2 <= n <= 100$
- $0 <= flights.length <= (n * (n - 1) / 2)$
- $flights[i].length == 3$
- $0 <= from_i, to_i < n$
- $from_i != to_i$
- $1 <= price_i <= 10^4$
- There will not be any multiple flights between two cities.
- $0 <= src, dst, k < n$
- $src != dst$

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Cheapest path with at most k stops. Dijkstra needs an extra “stops” dimension; Bellman-Ford with k+1 rounds is enough for n≤ 100. At most k stops means k+1 edges. Each round relaxes from a copy of the previous distances so one round is one edge. After k+1 rounds, infinite dist[dst] means unreachable.

Python 3

```python
class Solution:
    def findCheapestPrice(
        self, n: int, flights: List[List[int]], src: int, dst: int, k: int
    ) -> int:
        INF = 0x3F3F3F3F
        dist = [INF] * n
        dist[src] = 0
        for _ in range(k + 1):
            backup = dist.copy()
            for f, t, p in flights:
                dist[t] = min(dist[t], backup[f] + p)
        return -1 if dist[dst] == INF else dist[dst]
```

Complexity. Time: O(k * E), E = number of flights. Space: O(n).

Solution 2

How to think about it

Bellman-Ford scans every edge. Memoized dfs(u,k) is the min cost from u with k edges left: 0 at dst, inf when k runs out. The initial call uses k+1 edges. O(nk) states.

Python 3

```python
class Solution:
    def findCheapestPrice(
        self, n: int, flights: List[List[int]], src: int, dst: int, k: int
    ) -> int:
        @cache
        def dfs(u, k):
            if u == dst:
                return 0
            if k <= 0:
                return inf
            k -= 1
            ans = inf
            for v, p in g[u]:
                ans = min(ans, dfs(v, k) + p)
            return ans

        g = defaultdict(list)
        for u, v, p in flights:
            g[u].append((v, p))
        ans = dfs(src, k + 1)
        return -1 if ans >= inf else ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Bellman-Ford: relax all the flights k + 1 times
# 2. Return the cheapest route (the cities), not only its
# price


# ============================================================
# FOLLOW-UP 1: Bellman-Ford: relax all the flights k + 1
# times
# ============================================================

# ---- follow-up 1: function ----
# Answer: After round i, best[city] is the cheapest price
# using at most i flights; use a copy of the previous
# round so one round adds at most one flight.
# Time: O(k * E).  Space: O(n).
def cheapest_with_bellman_ford(n, flights, src, dst, k):
    big = float("inf")
    best = [big] * n
    best[src] = 0
    for round_number in range(k + 1):
        updated = list(best)
        for a, b, price in flights:
            if best[a] + price < updated[b]:
                updated[b] = best[a] + price
        best = updated
    if best[dst] == big:
        return -1
    return best[dst]


# ============================================================
# FOLLOW-UP 2: Return the cheapest route (the cities), not
# only its price
# ============================================================

# ---- follow-up 2: function ----
# Answer: Layer by layer, store with every price the route
# that produced it.
# Time: O(k * E * k) (the routes are copied).  Space: O(n
# * k).
def cheapest_route(n, flights, src, dst, k):
    big = float("inf")
    best = [(big, None)] * n
    best[src] = (0, [src])
    for round_number in range(k + 1):
        updated = list(best)
        for a, b, price in flights:
            if best[a][0] + price < updated[b][0]:
                updated[b] = (best[a][0] + price, best[a][1] + [b])
        best = updated
    if best[dst][0] == big:
        return -1, None
    return best[dst]
```

PATTERN 17 · GRAPH BFS · PROBLEM 12 OF 20

934. Shortest Bridge

Medium · Depth-First Search, Breadth-First Search, Array, Matrix

leetcode.com/problems/shortest-bridge

PROBLEM

You are given an `n x n` binary matrix `grid` where `1` represents land and `0` represents water.

An **island** is a 4-directionally connected group of `1`'s not connected to any other `1`'s. There are **exactly two islands** in `grid`.

You may change `0`'s to `1`'s to connect the two islands to form **one island**.

Return *the smallest number of `0`'s you must flip to connect the two islands*.

Example 1:

```
Input: grid = [[0,1],[1,0]]
Output: 1
```

Example 2:

```
Input: grid = [[0,1,0],[0,0,0],[0,0,1]]
Output: 2
```

Example 3:

```
Input: grid = [[1,1,1,1,1],[1,0,0,0,1],[1,0,1,0,1],[1,0,0,0,1],[1,1,1,1,1]]
Output: 1
```

Constraints:

- `n == grid.length == grid[i].length`
- `2 <= n <= 100`
- `grid[i][j]` is either `0` or `1`.
- There are exactly two islands in `grid`.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

There are exactly two islands; the fewest flips equal the shortest distance between them. DFS dumps one island into a queue and marks it visited, then BFS expands over water. The first time the other island is reached, the layer count is the answer.

Python 3

```
class Solution:
    def shortestBridge(self, grid: List[List[int]]) -> int:
        def dfs(i, j):
            q.append((i, j))
            grid[i][j] = 2
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < n and 0 <= y < n and grid[x][y] == 1:
                    dfs(x, y)

        n = len(grid)
        dirs = (-1, 0, 1, 0, -1)
        q = deque()
        i, j = next((i, j) for i in range(n) for j in range(n) if grid[i][j])
        dfs(i, j)
        ans = 0
        while 1:
            for _ in range(len(q)):
                i, j = q.popleft()
                for a, b in pairwise(dirs):
                    x, y = i + a, j + b
                    if 0 <= x < n and 0 <= y < n:
                        if grid[x][y] == 1:
                            return ans
                        if grid[x][y] == 0:
                            grid[x][y] = 2
                            q.append((x, y))
            ans += 1
```

Complexity. Time: O(n^2). Space: O(n^2).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the cells of the shortest bridge


# ============================================================
# FOLLOW-UP 1: Return the cells of the shortest bridge
# ============================================================

# ---- follow-up 1: function ----
# Answer: Colour the first island, grow outwards from it
# ring by ring, remember where each cell came from, and
# walk back from the first cell of the second island.
# Time: O(n^2).  Space: O(n^2).
from collections import deque


def bridge_cells(grid):
    n = len(grid)
    start = None
    for r in range(n):
        for c in range(n):
            if grid[r][c] == 1 and start is None:
                start = (r, c)
    island = {start}
    stack = [start]
    while stack:
        r, c = stack.pop()
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < n and 0 <= nc < n and grid[nr][nc] == 1 and (nr, nc) not in island:
                island.add((nr, nc))
                stack.append((nr, nc))
    previous = {cell: None for cell in island}
    queue = deque(island)
```

```
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < n and 0 <= nc < n and (nr, nc) not in previous:
                previous[(nr, nc)] = (r, c)
                if grid[nr][nc] == 1:
                    cells = []
                    cell = previous[(nr, nc)]
                    while cell is not None and cell not in island:
                        cells.append(cell)
                        cell = previous[cell]
                    cells.reverse()
                    return cells
                queue.append((nr, nc))
    return []
```

1311. Get Watched Videos by Your Friends

Medium · Breadth-First Search, Graph, Array, Hash Table, Sorting

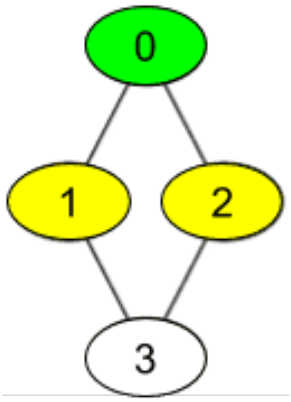
leetcode.com/problems/get-watched-videos-by-your-friends

PROBLEM

There are `n` people, each person has a unique *id* between `0` and `n-1`. Given the arrays `watchedVideos` and `friends`, where `watchedVideos[i]` and `friends[i]` contain the list of watched videos and the list of friends respectively for the person with `id = i`.

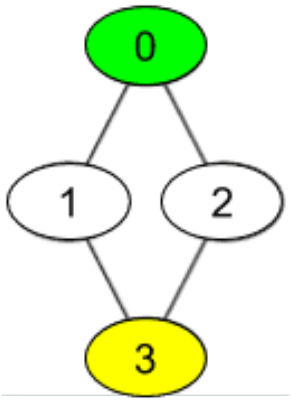
Level **1** of videos are all watched videos by your friends, level **2** of videos are all watched videos by the friends of your friends and so on. In general, the level `k` of videos are all watched videos by people with the shortest path **exactly** equal to `k` with you. Given your `id` and the `level` of videos, return the list of videos ordered by their frequencies (increasing). For videos with the same frequency order them alphabetically from least to greatest.

Example 1:




```
Input: watchedVideos = [["A","B"],["C"],["B","C"],["D"]], friends = [[1,2],[0,3],[0,3],[1,2]], id = 0, level = 1
Output: ["B","C"]
Explanation:
You have id = 0 (green color in the figure) and your friends are (yellow color in the figure):
Person with id = 1 -> watchedVideos = ["C"]
Person with id = 2 -> watchedVideos = ["B","C"]
The frequencies of watchedVideos by your friends are:
B -> 1
C -> 2
```

Example 2:



```
Input: watchedVideos = [["A","B"],["C"],["B","C"],["D"]], friends = [[1,2],[0,3],[0,3],[1,2]], id = 0, level = 2
Output: ["D"]
Explanation:
You have id = 0 (green color in the figure) and the only friend of your friends is the person with id = 3 (yellow color in the figure).
```

Constraints:

- `n == watchedVideos.length == friends.length`
- `2 <= n <= 100`
- `1 <= watchedVideos[i].length <= 100`
- `1 <= watchedVideos[i][j].length <= 8`
- `0 <= friends[i].length < n`
- `0 <= friends[i][j] < n`
- `0 <= id < n`
- `1 <= level < n`
- if `friends[i]` contains `j`, then `friends[j]` contains `i`

SOLUTION (PYTHON 3)

Solution 1: BFS

How to think about it

We need videos watched by friends exactly level hops away, ordered by frequency then name. Unbounded DFS does not stop on a precise layer and revisits people. level rounds of BFS from id leave exactly that layer in the queue. A counter tallies their videos; sorting by (cnt[v], v) produces the names. A visited set keeps a person from entering more than once.

We can use the Breadth-First Search (BFS) method to start from id and find all friends at a distance of level, then count the videos watched by these friends.

Specifically, we can use a queue q to store the friends at the current level. Initially, add id to the queue q. Use a hash table or a boolean array vis to record the friends that have already been visited. Then, perform level iterations, in each iteration dequeue all friends from the queue and enqueue their friends until all friends at distance level are found.

Next, we use a hash table cnt to count the videos watched by these friends and their frequencies. Finally, sort the key-value pairs in the hash table in ascending order by frequency, and if frequencies are the same, sort by video name in ascending order. Return the sorted list of video names.

Time complexity is $O(n + m + v \times \log v)$, and space complexity is $O(n + v)$. Here, n and m are the lengths of the arrays watchedVideos and friends, respectively, and v is the total number of videos watched by all friends.

Python 3

```python
class Solution:
    def watchedVideosByFriends(
        self,
        watchedVideos: List[List[str]],
        friends: List[List[int]],
        id: int,
        level: int,
    ) -> List[str]:
        q = deque([id])
        vis = {id}
        for _ in range(level):
            for _ in range(len(q)):
                i = q.popleft()
                for j in friends[i]:
                    if j not in vis:
                        vis.add(j)
                        q.append(j)
        cnt = Counter()
        for i in q:
            for v in watchedVideos[i]:
                cnt[v] += 1
        return sorted(cnt.keys(), key=lambda k: (cnt[k], k))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Which people are exactly at level k (friends of
# friends ...)?
# 2. The single most watched video at level k


# ============================================================
# FOLLOW-UP 1: Which people are exactly at level k
# (friends of friends ...)?
# ============================================================

# ---- follow-up 1: function ----
# Answer: A BFS over the friendship graph with levels;
# collect the people on level k.
# Time: O(n + E).  Space: O(n).
from collections import deque


def people_at_level(friends, person, level):
    seen = {person}
    current = [person]
    for step in range(level):
        following = []
        for p in current:
            for friend in friends[p]:
                if friend not in seen:
                    seen.add(friend)
                    following.append(friend)
        current = following
    return sorted(current)


# ============================================================
# FOLLOW-UP 2: The single most watched video at level k
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the videos watched by the people at that
# level and take the one with the highest count
# (alphabetical on ties).
```

```python
# Time: O(n + E + V), V = videos watched.  Space: O(n).
def most_watched_at_level(watched_videos, friends, person, level):
    seen = {person}
    current = [person]
    for step in range(level):
        following = []
        for p in current:
            for friend in friends[p]:
                if friend not in seen:
                    seen.add(friend)
                    following.append(friend)
        current = following
    count = {}
    for p in current:
        for video in watched_videos[p]:
            count[video] = count.get(video, 0) + 1
    best = None
    for video in count:
        if best is None or count[video] > count[best] or (count[video] == count[best] and video < best):
            best = video
```

```python
    return best
```

1765. Map of Highest Peak

Medium · Breadth-First Search, Array, Matrix

leetcode.com/problems/map-of-highest-peak

PROBLEM

You are given an integer matrix `isWater` of size `m x n` that represents a map of **land** and **water** cells.

- If `isWater[i][j] == 0`, cell `(i, j)` is a **land** cell.
- If `isWater[i][j] == 1`, cell `(i, j)` is a **water** cell.

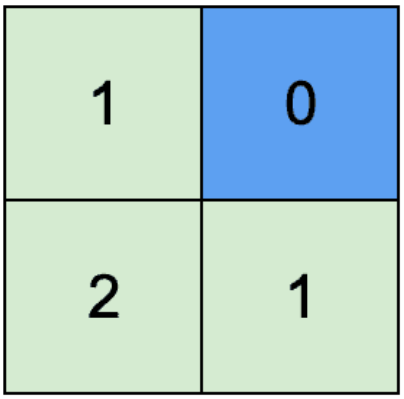
You must assign each cell a height in a way that follows these rules:

- The height of each cell must be non-negative.
- If the cell is a **water** cell, its height must be `0`.
- Any two adjacent cells must have an absolute height difference of **at most** `1`. A cell is adjacent to another cell if the former is directly north, east, south, or west of the latter (i.e., their sides are touching).

Find an assignment of heights such that the maximum height in the matrix is **maximized**.

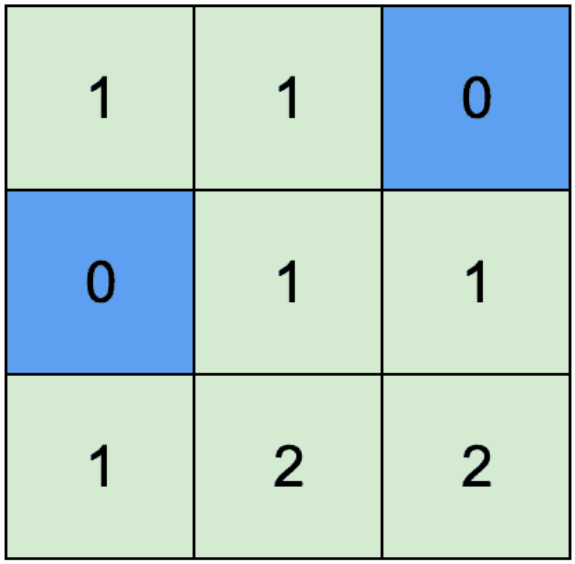
Return *an integer matrix* `height` *of size* `m x n` *where* `height[i][j]` *is cell* `(i, j)`*'s height. If there are multiple solutions, return* ***any*** *of them.*

Example 1:




```
Input: isWater = [[0,1],[0,0]]
Output: [[1,0],[2,1]]
Explanation: The image shows the assigned heights of each cell.
The blue cell is the water cell, and the green cells are the land cells.
```

Example 2:




```
Input: isWater = [[0,0,1],[1,0,0],[0,0,0]]
Output: [[1,1,0],[0,1,1],[1,2,2]]
Explanation: A height of 2 is the maximum possible height of any assignment.
Any height assignment that has a maximum height of 2 while still meeting the rules will also be
accepted.
```

Constraints:

- `m == isWater.length`
- `n == isWater[i].length`
- `1 <= m, n <= 1000`
- `isWater[i][j]` is `0` or `1`.
- There is at least **one** water cell.

Note: This question is the same as 542: https://leetcode.com/problems/01-matrix/

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Water cells must be height 0, adjacent heights differ by at most 1, and land should be as high as possible. The height is the distance to the nearest water. Multi-source BFS: enqueue every water cell at 0, and set each unseen neighbour to the current height plus one. The distance field is the height map.

Python 3

```python
class Solution:
    def highestPeak(self, isWater: List[List[int]]) -> List[List[int]]:
        m, n = len(isWater), len(isWater[0])
        ans = [[-1] * n for _ in range(m)]
        q = deque()
        for i, row in enumerate(isWater):
            for j, v in enumerate(row):
                if v:
                    q.append((i, j))
                    ans[i][j] = 0
        while q:
            i, j = q.popleft()
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and ans[x][y] == -1:
                    ans[x][y] = ans[i][j] + 1
                    q.append((x, y))
        return ans
```

Complexity. Time: O(m * n). Space: O(m * n).

Solution 2

> **How to think about it**
>
> Solution 2 is the same multi-source BFS, popping an explicit layer size. The algorithm and complexity are unchanged.

Python 3

```python
class Solution:
    def highestPeak(self, isWater: List[List[int]]) -> List[List[int]]:
        m, n = len(isWater), len(isWater[0])
        ans = [[-1] * n for _ in range(m)]
        q = deque()
        for i, row in enumerate(isWater):
            for j, v in enumerate(row):
                if v:
                    q.append((i, j))
                    ans[i][j] = 0
        while q:
            for _ in range(len(q)):
                i, j = q.popleft()
                for a, b in pairwise((-1, 0, 1, 0, -1)):
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n and ans[x][y] == -1:
                        ans[x][y] = ans[i][j] + 1
                        q.append((x, y))
        return ans
```

Complexity. Time: O(m * n). Space: O(m * n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Neighbouring cells may differ by at most d (not only
# 1)


# ========================================================
# FOLLOW-UP 1: Neighbouring cells may differ by at most d
# (not only 1)
# ========================================================

# ---- follow-up 1: function ----
# Answer: Every land cell is the source of a multi-source
```

```
# BFS; the height of a cell is d times its distance to the
# nearest water cell.
# Time: O(m * n).  Space: O(m * n).
from collections import deque


def highest_peak_with_step(is_water, step):
    rows, cols = len(is_water), len(is_water[0])
    height = []
    queue = deque()
    for r in range(rows):
        row = []
        for c in range(cols):
            if is_water[r][c] == 1:
                row.append(0)
                queue.append((r, c))
            else:
                row.append(-1)
        height.append(row)
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and height[nr][nc] == -1:
                height[nr][nc] = height[r][c] + step
                queue.append((nr, nc))
    return height
```

PATTERN 17 · GRAPH BFS · PROBLEM 15 OF 20

127. Word Ladder

Hard · Breadth-First Search, Hash Table, String, Bidirectional Search

leetcode.com/problems/word-ladder

★ **TOP 7 PROBLEM** in Graph BFS: master this one first

PROBLEM

A **transformation sequence** from word `beginWord` to word `endWord` using a dictionary `wordList` is a sequence of words `beginWord -> s1 -> s2 -> ... -> sk` such that:

- Every adjacent pair of words differs by a single letter.
- Every s_i for `1 <= i <= k` is in `wordList`. Note that `beginWord` does not need to be in `wordList`.
- s_k `== endWord`

Given two words, `beginWord` and `endWord`, and a dictionary `wordList`, return *the **number of words** in the **shortest transformation sequence** from* `beginWord` *to* `endWord`*, or* `0` *if no such sequence exists.*

Example 1:

```
Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log","cog"]
Output: 5
Explanation: One shortest transformation sequence is "hit" -> "hot" -> "dot" -> "dog" -> cog", which
is 5 words long.
```

Example 2:

```
Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log"]
Output: 0
Explanation: The endWord "cog" is not in wordList, therefore there is no valid transformation
sequence.
```

Constraints:

- `1 <= beginWord.length <= 10`
- `endWord.length == beginWord.length`
- `1 <= wordList.length <= 5000`
- `wordList[i].length == beginWord.length`
- `beginWord`, `endWord`, and `wordList[i]` consist of lowercase English letters.
- `beginWord != endWord`
- All the words in `wordList` are **unique**.

SOLUTIONS (PYTHON 3)

Solution 1: BFS

How to think about it

Find the shortest transformation; each step changes one letter and must stay in the word list. With up to 5000 words of length 10, enumerating paths is impossible. Unweighted shortest paths are BFS: try 26 letters at each position, enqueue hits, and remove them from the set so they are not visited twice.

BFS minimum step model. This problem can be solved with naive BFS, or it can be optimized with bidirectional BFS to reduce the search space and improve efficiency.

Bidirectional BFS is a common optimization method for BFS, with the main implementation ideas as follows:

1. Create two queues, q1 and q2, for "start -> end" and "end -> start" search directions, respectively.

2. Create two hash maps, m1 and m2, to record the visited nodes and their corresponding expansion times (steps).
3. During each search, prioritize the queue with fewer elements for search expansion. If a node visited from the other direction is found during the expansion, it means the shortest path has been found.
4. If one of the queues is empty, it means that the search in the current direction cannot continue, indicating that the start and end points are not connected, and there is no need to continue the search.

```
while q1 and q2:
    if len(q1) <= len(q2):
        # Prioritize the queue with fewer elements for expansion
        extend(m1, m2, q1)
    else:
        extend(m2, m1, q2)


def extend(m1, m2, q):
    # New round of expansion
    for _ in range(len(q)):
        p = q.popleft()
        step = m1[p]
        for t in next(p):
            if t in m1:
                # Already visited before
                continue
            if t in m2:
                # The other direction has been searched, indicating that a shortest path has been found
                return step + 1 + m2[t]
            q.append(t)
            m1[t] = step + 1
```

Python 3

```
class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str]) -> int:
        words = set(wordList)
        q = deque([beginWord])
        ans = 1
        while q:
            ans += 1
            for _ in range(len(q)):
                s = q.popleft()
                s = list(s)
                for i in range(len(s)):
                    ch = s[i]
                    for j in range(26):
                        s[i] = chr(ord('a') + j)
                        t = ''.join(s)
                        if t not in words:
                            continue
                        if t == endWord:
                            return ans
                        q.append(t)
                        words.remove(t)
                    s[i] = ch
        return 0
```

Complexity. Time: O(N * L^2) (26 letters is a constant), N = number of words, L = word length. Space: O(N * L).

Solution 2: Bidirectional BFS

> **How to think about it**
>
> Solution 1's frontier can grow wide on a large list. Expanding from both ends and always growing the smaller side meets in the middle at the shortest length, and the search is usually smaller.

Search from both ends until the two sides meet.

Python 3

```
class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str]) -> int:
        def extend(m1, m2, q):
            for _ in range(len(q)):
                s = q.popleft()
                step = m1[s]
                s = list(s)
                for i in range(len(s)):
                    ch = s[i]
                    for j in range(26):
                        s[i] = chr(ord('a') + j)
                        t = ''.join(s)
                        if t in m1 or t not in words:
                            continue
                        if t in m2:
                            return step + 1 + m2[t]
                        m1[t] = step + 1
                        q.append(t)
                    s[i] = ch
            return -1

        words = set(wordList)
        if endWord not in words:
            return 0
        q1, q2 = deque([beginWord]), deque([endWord])
        m1, m2 = {beginWord: 0}, {endWord: 0}
        while q1 and q2:
            t = extend(m1, m2, q1) if len(q1) <= len(q2) else extend(m2, m1, q2)
            if t != -1:
                return t + 1
        return 0
```

Complexity. Time: O(N * L^2) in the worst case, but much faster in practice (two searches meet in the middle). Space: O(N * L).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Bidirectional BFS
# 2. Return one shortest ladder (the words)


# ============================================================
# FOLLOW-UP 1: Bidirectional BFS
# ============================================================

# ---- follow-up 1: function ----
# Answer: Grow the search from the begin word and from the
# end word at the same time (always from the smaller side)
# until they meet: much less work on big dictionaries.
# Time: O(N * L^2) in the worst case, much faster in
# practice.  Space: O(N * L).
def ladder_length_bidirectional(begin, end, word_list):
    words = set(word_list)
    if end not in words:
        return 0
    front = {begin}
    back = {end}
    length = 1
    while front and back:
        if len(front) > len(back):
            front, back = back, front
        following = set()
        for word in front:
            for i in range(len(word)):
                for letter in "abcdefghijklmnopqrstuvwxyz":
                    new = word[:i] + letter + word[i + 1:]
```

```
                    if new in back:
                        return length + 1
                    if new in words:
                        words.discard(new)
                        following.add(new)
        front = following
        length += 1
    return 0


# ============================================================
# FOLLOW-UP 2: Return one shortest ladder (the words)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Remember the previous word of every word reached
# and walk back from the end word.
# Time: O(N * L^2).  Space: O(N * L).
from collections import deque


def one_ladder(begin, end, word_list):
    words = set(word_list)
    if end not in words:
        return []
    previous = {begin: None}
    queue = deque([begin])
    while queue:
        word = queue.popleft()
        if word == end:
```

```
            ladder = []
            while word is not None:
                ladder.append(word)
                word = previous[word]
            ladder.reverse()
            return ladder
        for i in range(len(word)):
            for letter in "abcdefghijklmnopqrstuvwxyz":
                new = word[:i] + letter + word[i + 1:]
                if new in words and new not in previous:
                    previous[new] = word
                    queue.append(new)
    return []
```

815. Bus Routes

Hard · Breadth-First Search, Array, Hash Table

leetcode.com/problems/bus-routes

PROBLEM

You are given an array `routes` representing bus routes where `routes[i]` is a bus route that the i^{th} bus repeats forever.

- For example, if `routes[0] = [1, 5, 7]`, this means that the 0^{th} bus travels in the sequence `1 -> 5 -> 7 -> 1 -> 5 -> 7 -> 1 -> ...` forever.

You will start at the bus stop `source` (You are not on any bus initially), and you want to go to the bus stop `target`. You can travel between bus stops by buses only.

Return *the least number of buses you must take to travel from* `source` *to* `target`. Return `-1` if it is not possible.

Example 1:

```
Input: routes = [[1,2,7],[3,6,7]], source = 1, target = 6
Output: 2
Explanation: The best strategy is take the first bus to the bus stop 7, then take the second bus to the bus stop 6.
```

Example 2:

```
Input: routes = [[7,12],[4,5,15],[6],[15,19],[9,12,13]], source = 15, target = 12
Output: -1
```

Constraints:

- `1 <= routes.length <= 500.`
- `1 <= routes[i].length <= 10^5`
- All the values of `routes[i]` are **unique**.
- `sum(routes[i].length) <= 10^5`
- `0 <= routes[i][j] < 10^6`
- `0 <= source, target < 10^6`

SOLUTION (PYTHON 3)

Solution 1: BFS

How to think about it

Riding one bus reaches every stop on that route; we want the fewest rides. Walking stop-to-stop would expand the same bus many times. The right BFS layer is a transfer. Build a stop-to-routes index. The queue stores a stop and the ride count; each route is expanded once, enqueueing its unseen stops. Every bus is processed at most once.

First, we check if source and target are the same. If they are, we directly return 0.

Next, we use a hash table g to build a mapping from stops to bus routes. For each bus route, we traverse all the stops it passes through and map each stop to that bus route, i.e., g[stop] represents all bus routes passing through stop stop.

Then, we check if source and target are in the stop mapping. If they are not, we return -1.

We use a queue q to perform a breadth-first search (BFS). Each element in the queue is a tuple (stop, busCount), representing the current stop stop and the number of buses taken to reach the current stop busCount.

We initialize a set visBus to record the bus routes that have been visited and a set visStop to record the stops that have been visited. Then, we add source to visStop and (source, 0) to the queue q.

Next, we start the BFS. While the queue q is not empty, we take out the first element from the queue, which is the current stop stop and the number of buses taken to reach the current stop busCount.

If the current stop stop is the target stop target, we return the number of buses taken to reach the target stop busCount.

Otherwise, we traverse all bus routes passing through the current stop. For each bus route, we traverse all stops on that route. If a stop nextStop has not been visited, we add it to visStop and add (nextStop, busCount + 1) to the queue q.

Finally, if we cannot reach the target stop, we return -1.

The time complexity is O(L), and the space complexity is O(L), where L is the total number of stops on all bus routes.

Python 3

```
class Solution:
    def numBusesToDestination(
        self, routes: List[List[int]], source: int, target: int
    ) -> int:
        if source == target:
            return 0
        g = defaultdict(list)
        for i, route in enumerate(routes):
            for stop in route:
                g[stop].append(i)
        if source not in g or target not in g:
            return -1
        q = [(source, 0)]
        vis_bus = set()
        vis_stop = {source}
        for stop, bus_count in q:
            if stop == target:
                return bus_count
            for bus in g[stop]:
                if bus not in vis_bus:
                    vis_bus.add(bus)
                    for next_stop in routes[bus]:
                        if next_stop not in vis_stop:
                            vis_stop.add(next_stop)
                            q.append((next_stop, bus_count + 1))
        return -1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the buses to take, in order


# ============================================================
# FOLLOW-UP 1: Return the buses to take, in order
# ============================================================

# ---- follow-up 1: function ----
# Answer: BFS over the buses: two buses are neighbours
# when they share a stop. Remember the previous bus and
# walk back from the first bus that reaches the target.
# Time: O(total stops * buses per stop).  Space: O(total
# stops).
from collections import deque


def buses_to_take(routes, source, target):
    if source == target:
        return []
    stop_buses = {}
    for bus in range(len(routes)):
        for stop in routes[bus]:
            stop_buses.setdefault(stop, []).append(bus)
    previous = {}
```

```
    queue = deque()
    for bus in stop_buses.get(source, []):
        previous[bus] = None
        queue.append(bus)
    while queue:
        bus = queue.popleft()
        if target in routes[bus]:
            path = []
            while bus is not None:
                path.append(bus)
                bus = previous[bus]
            path.reverse()
            return path
        for stop in routes[bus]:
            for other in stop_buses[stop]:
                if other not in previous:
                    previous[other] = bus
                    queue.append(other)
    return None
```

PATTERN 17 · GRAPH BFS · PROBLEM 17 OF 20

864. Shortest Path to Get All Keys

Hard · Bit Manipulation, Breadth-First Search, Array, Matrix

leetcode.com/problems/shortest-path-to-get-all-keys

★ **TOP 7 PROBLEM** in Graph BFS: master this one first

PROBLEM

You are given an `m x n` grid `grid` where:

- `'.'` is an empty cell.
- `'#'` is a wall.
- `'@'` is the starting point.
- Lowercase letters represent keys.
- Uppercase letters represent locks.

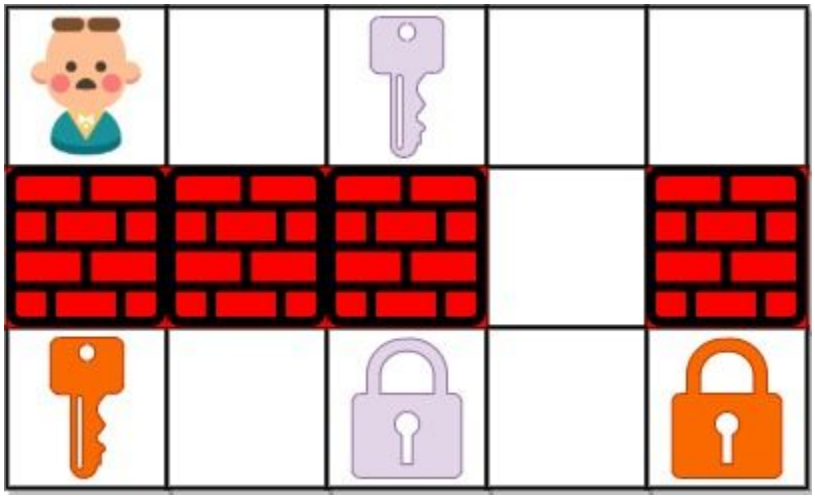
You start at the starting point and one move consists of walking one space in one of the four cardinal directions. You cannot walk outside the grid, or walk into a wall.

If you walk over a key, you can pick it up and you cannot walk over a lock unless you have its corresponding key.

For some `1 <= k <= 6`, there is exactly one lowercase and one uppercase letter of the first `k` letters of the English alphabet in the grid. This means that there is exactly one key for each lock, and one lock for each key; and also that the letters used to represent the keys and locks were chosen in the same order as the English alphabet.

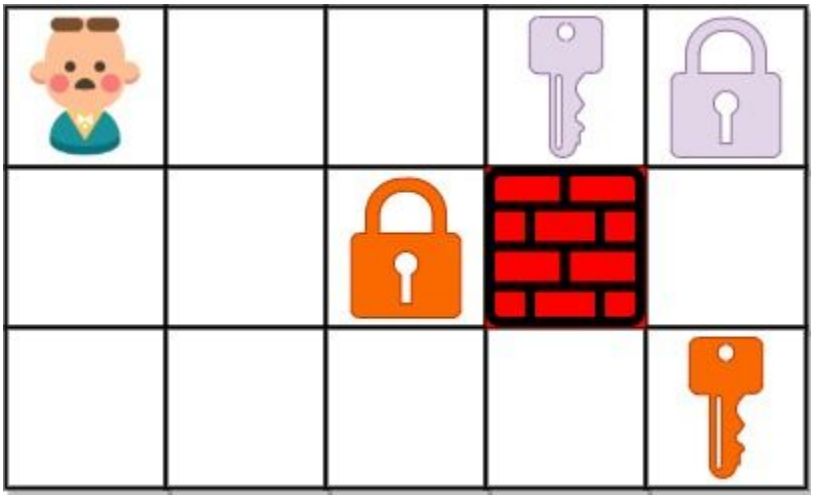
Return *the lowest number of moves to acquire all keys*. If it is impossible, return `-1`.

Example 1:



```
Input: grid = ["@.a..","###.#","b.A.B"]
Output: 8
Explanation: Note that the goal is to obtain all the keys not to open all the locks.
```

Example 2:



```
Input: grid = ["@..aA","..B#.","....b"]
Output: 6
```

Example 3:

```
Input: grid = ["@Aa"]
Output: -1
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 30`
- `grid[i][j]` is either an English letter, `'.'`, `'#'`, or `'@'`.
- There is exactly one `'@'` in the grid.
- The number of keys in the grid is in the range `[1, 6]`.
- Each key in the grid is **unique**.
- Each key in the grid has a matching lock.

SOLUTION (PYTHON 3)

Solution 1: State Compression + BFS

How to think about it

Collect every key; a lock blocks the path without its key. At most 6 keys on a 30× 30 grid, so (position, key mask) is a BFS state. (i,j,mask) is the cell and keys in hand. Walls and locked doors are skipped; a key sets a bit. The first full mask is the shortest path.

According to the problem description, we need to start from the initial position, move in four directions (up, down, left, right), collect all keys, and finally return the minimum number of moves required to collect all keys. If it is not possible to collect all keys, return -1.

First, we traverse the 2D grid to find the starting position (si, sj) and count the number of keys k.

Then, we can use Breadth-First Search (BFS) to solve this problem. Since the number of keys ranges from 1 to 6, we can use a binary number to represent the state of the keys, where the i-th bit being 1 indicates that the i-th key has been

collected, and 0 indicates that the i-th key has not been collected.

For example, in the following case, there are 4 bits set to 1, indicating that keys `'b', 'c', 'd', 'f'` have been collected.

```
1 0 1 1 1 0
^   ^ ^ ^
f   d c b
```

We define a queue q to store the current position and the state of the collected keys, i.e., (i, j, state), where (i, j) represents the current position, and state represents the state of the collected keys. The i-th bit of state being 1 indicates that the i-th key has been collected; otherwise, it indicates that the i-th key has not been collected.

Additionally, we define a hash table or array vis to record whether the current position and the state of the collected keys have been visited. If visited, there is no need to visit again. vis[i][j][state] indicates whether the position (i, j) and the state of the collected keys state have been visited.

We start from the initial position (si, sj), add it to the queue q, and set vis[si][sj][0] to true, indicating that the initial position and the state of the collected keys 0 have been visited.

During the BFS process, we take out a position (i, j, state) from the front of the queue and check whether the current position is the endpoint, i.e., whether the current position has collected all keys, which means the number of 1s in the binary representation of state is k. If so, we return the current number of steps as the answer.

Otherwise, we move from the current position in four directions (up, down, left, right). If we can move to the next position (x, y), we add (x, y, nxt) to the queue q, where nxt represents the state of the keys at the next position.

Here, (x, y) must first be within the grid range, i.e., $0 \le x < m$ and $0 \le y < n$. Secondly, if the position (x, y) is a wall, i.e., `grid[x][y] == '#'`, or the position (x, y) is a lock but we do not have the corresponding key, i.e., `grid[x][y] >= 'A' && grid[x][y] <= 'F' && (state >> (grid[x][y] - 'A') & 1) == 0)`, then we cannot move to the position (x, y)`. Otherwise, we can move to the position(x, y)`.

If the search ends and we have not collected all keys, return -1.

The time complexity is $O(m \times n \times 2^k)$, and the space complexity is $O(m \times n \times 2^k)$. Here, m and n are the number of rows and columns of the grid, respectively, and k is the number of keys.

Python 3

```python
class Solution:
    def shortestPathAllKeys(self, grid: List[str]) -> int:
        m, n = len(grid), len(grid[0])
        # Find the starting point (si, sj)
        si, sj = next((i, j) for i in range(m) for j in range(n) if grid[i][j] == '@')
        # Count the number of keys
        k = sum(v.islower() for row in grid for v in row)
        dirs = (-1, 0, 1, 0, -1)
        q = deque([(si, sj, 0)])
        vis = {(si, sj, 0)}
        ans = 0
        while q:
            for _ in range(len(q)):
                i, j, state = q.popleft()
                # If all keys are found, return the current step count
                if state == (1 << k) - 1:
                    return ans

                # Search in the four directions
                for a, b in pairwise(dirs):
                    x, y = i + a, j + b
                    nxt = state
                    # Within boundary limits
                    if 0 <= x < m and 0 <= y < n:
                        c = grid[x][y]
                        # It's a wall, or it's a lock but we don't have the key for it
                        if (
                            c == '#'
                            or c.isupper()
                            and (state & (1 << (ord(c) - ord('A')))) == 0
                        ):
                            continue
                        # It's a key
                        if c.islower():
                            # Update the state
                            nxt |= 1 << (ord(c) - ord('a'))
                        # If this state has not been visited, enqueue it
                        if (x, y, nxt) not in vis:
                            vis.add((x, y, nxt))
                            q.append((x, y, nxt))
            # Increment the step count
            ans += 1
        return -1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Only decide whether all the keys can be collected at
# all


# ============================================================
# FOLLOW-UP 1: Only decide whether all the keys can be
# collected at all
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same state search (position, keys held)
# without caring about the length: stop with True when the
# key set is complete.
# Time: O(m * n * 2^k).  Space: O(m * n * 2^k).
from collections import deque


def can_collect_all_keys(grid):
```

```
    rows, cols = len(grid), len(grid[0])
    total = 0
    start = None
    for r in range(rows):
        for c in range(cols):
            ch = grid[r][c]
            if ch == "@":
                start = (r, c)
            elif "a" <= ch <= "f":
                total += 1
    full = (1 << total) - 1
    seen = {(start[0], start[1], 0)}
    queue = deque([(start[0], start[1], 0)])
    while queue:
        r, c, keys = queue.popleft()
        if keys == full:
            return True
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
            ch = grid[nr][nc]
            if ch == "#":
                continue
            new_keys = keys
            if "a" <= ch <= "f":
                new_keys |= 1 << (ord(ch) - ord("a"))
            if "A" <= ch <= "F" and not (keys >> (ord(ch) - ord("A"))) & 1:
                continue
            if (nr, nc, new_keys) not in seen:
                seen.add((nr, nc, new_keys))
                queue.append((nr, nc, new_keys))
    return False
```

1568. Minimum Number of Days to Disconnect Island

Hard · Depth-First Search, Breadth-First Search, Array, Matrix, Articulation Point, Strongly Connected Component

leetcode.com/problems/minimum-number-of-days-to-disconnect-island

PROBLEM

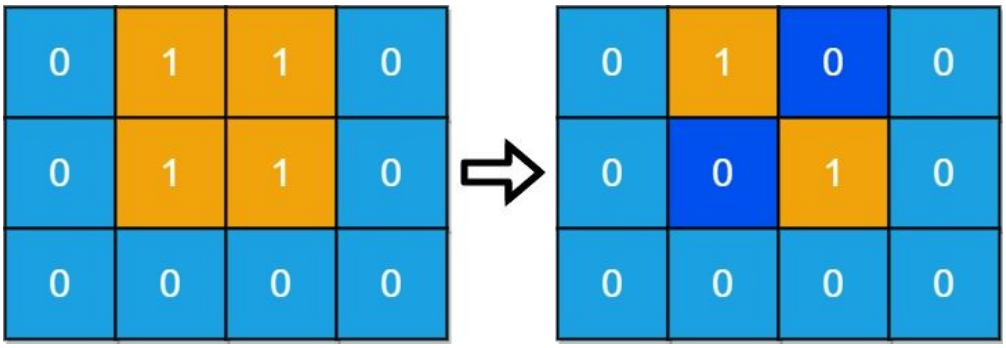
You are given an `m x n` binary grid `grid` where `1` represents land and `0` represents water. An **island** is a maximal **4-directionally** (horizontal or vertical) connected group of `1`'s.

The grid is said to be **connected** if we have **exactly one island**, otherwise is said **disconnected**.

In one day, we are allowed to change **any** single land cell `(1)` into a water cell `(0)`.

Return *the minimum number of days to disconnect the grid*.

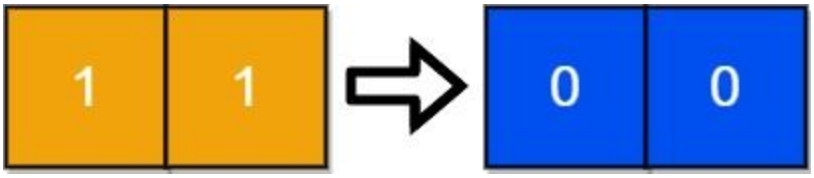
Example 1:




```
Input: grid = [[0,1,1,0],[0,1,1,0],[0,0,0,0]]

Output: 2
Explanation: We need at least 2 days to get a disconnected grid.
Change land grid[1][1] and grid[0][2] to water and get 2 disconnected island.
```

Example 2:




```
Input: grid = [[1,1]]
Output: 2
Explanation: Grid of full water is also disconnected ([[1,1]] -> [[0,0]]), 0 islands.
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 30`
- `grid[i][j]` is either `0` or `1`.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Each day one land cell may turn to water; we want the grid to stop containing exactly one island. The grid is small, and a stronger fact holds: any island can be split by removing at most two cells (for example two adjacent corner lands). Hence the answer is 0, 1, or 2. A DFS count that is not exactly one returns 0. Temporarily deleting each land cell and recounting returns 1 as soon as the island splits. Otherwise two deletions are required.

Python 3

```python
class Solution:
    def minDays(self, grid: List[List[int]]) -> int:
        if self.count(grid) != 1:
            return 0
        m, n = len(grid), len(grid[0])
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 1:
                    grid[i][j] = 0
                    if self.count(grid) != 1:
                        return 1
                    grid[i][j] = 1
        return 2

    def count(self, grid):
        def dfs(i, j):
            grid[i][j] = 2
            for a, b in [[0, -1], [0, 1], [1, 0], [-1, 0]]:
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and grid[x][y] == 1:
                    dfs(x, y)

        m, n = len(grid), len(grid[0])
        cnt = 0
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 1:
                    dfs(i, j)
                    cnt += 1
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 2:
                    grid[i][j] = 1
        return cnt
```

Complexity. Time: O((m * n)^2) (a full island count for every land cell). Space: O(m * n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Is the grid already disconnected (not exactly one
# island)?
# 2. Which single cells can be removed to disconnect the
# island?


# ============================================================
# FOLLOW-UP 1: Is the grid already disconnected (not
# exactly one island)?
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count the islands; the grid is connected only
```

```
# when there is exactly one.
# Time: O(m * n).  Space: O(m * n).
def count_islands(grid):
    rows, cols = len(grid), len(grid[0])
    seen = set()
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1 and (r, c) not in seen:
                count += 1
                stack = [(r, c)]
                seen.add((r, c))
                while stack:
                    x, y = stack.pop()
                    for nx, ny in ((x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1)):
                        if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 1 and (nx, ny) not in
seen:
                            seen.add((nx, ny))
                            stack.append((nx, ny))
    return count


def already_disconnected(grid):
    return count_islands(grid) != 1


# ============================================================
# FOLLOW-UP 2: Which single cells can be removed to
# disconnect the island?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Try removing each land cell and check whether
# the island count changes from one to something else.
# Time: O((m * n)^2).  Space: O(m * n).
def cells_that_disconnect(grid):
    cells = []
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == 1:
                grid[r][c] = 0
                if count_islands(grid) != 1:
                    cells.append((r, c))
                grid[r][c] = 1
    return cells
```

1293. Shortest Path in a Grid with Obstacles Elimination

Hard · Breadth-First Search, Array, Matrix

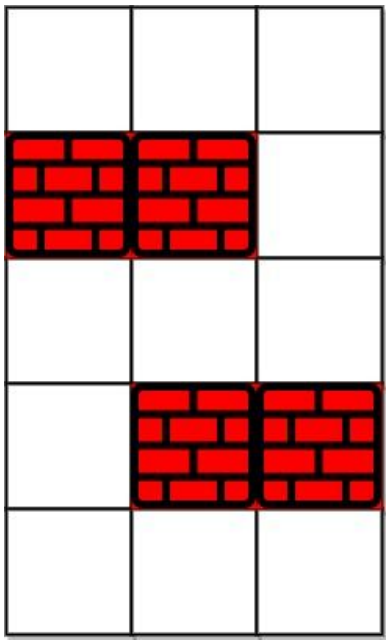
leetcode.com/problems/shortest-path-in-a-grid-with-obstacles-elimination

PROBLEM

You are given an `m x n` integer matrix `grid` where each cell is either `0` (empty) or `1` (obstacle). You can move up, down, left, or right from and to an empty cell in **one step**.

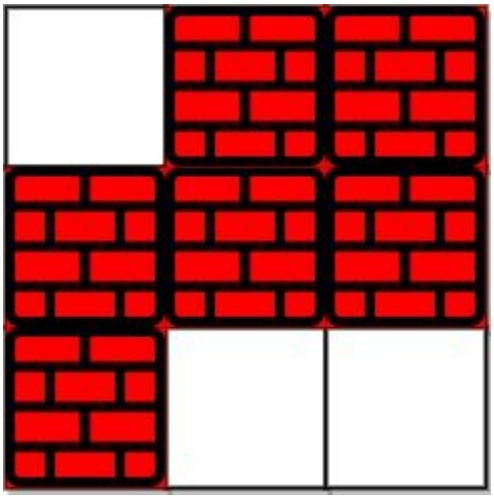
Return *the minimum number of **steps** to walk from the upper left corner* `(0, 0)` *to the lower right corner* `(m - 1, n - 1)` *given that you can eliminate **at most** `k` obstacles*. If it is not possible to find such walk return `-1`.

Example 1:



```
Input: grid = [[0,0,0],[1,1,0],[0,0,0],[0,1,1],[0,0,0]], k = 1
Output: 6
Explanation:
The shortest path without eliminating any obstacle is 10.
The shortest path with one obstacle elimination at position (3,2) is 6. Such path is (0,0) -> (0,1) -> (0,2) -> (1,2) -> (2,2) -> (3,2) -> (4,2).
```

Example 2:



```
Input: grid = [[0,1,1],[1,1,1],[1,0,0]], k = 1
Output: -1
Explanation: We need to eliminate at least two obstacles to find such a walk.
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 40`
- `1 <= k <= m * n`
- `grid[i][j]` is either `0` **or** `1`.
- `grid[0][0] == grid[m - 1][n - 1] == 0`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Shortest path on a grid, with at most k obstacles removable. m,n ≤ 40, so a state includes remaining removals. BFS layers are steps; if k already covers the Manhattan detour we return that length. The queue stores (i,j,k left). Empty cells keep k; obstacles use k-1. The same cell with different remaining k is another state. The first time we reach the end is shortest.

Python 3

```
class Solution:
    def shortestPath(self, grid: List[List[int]], k: int) -> int:
        m, n = len(grid), len(grid[0])
        if k >= m + n - 3:
            return m + n - 2
        q = deque([(0, 0, k)])
        vis = {(0, 0, k)}
        ans = 0
        while q:
            ans += 1
            for _ in range(len(q)):
                i, j, k = q.popleft()
                for a, b in [[0, -1], [0, 1], [1, 0], [-1, 0]]:
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n:
                        if x == m - 1 and y == n - 1:
                            return ans
                        if grid[x][y] == 0 and (x, y, k) not in vis:
                            q.append((x, y, k))
                            vis.add((x, y, k))
                        if grid[x][y] == 1 and k > 0 and (x, y, k - 1) not in vis:
                            q.append((x, y, k - 1))
                            vis.add((x, y, k - 1))
        return -1
```

Complexity. Time: O(m * n * k). Space: O(m * n * k).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the path, not only its length
# 2. Unlimited eliminations: the length is just the
# Manhattan distance


# ============================================================
# FOLLOW-UP 1: Return the path, not only its length
# ============================================================

# ---- follow-up 1: function ----
# Answer: The state is (row, column, obstacles removed so
# far); remember the previous state and walk back from the
# goal.
# Time: O(m * n * k).  Space: O(m * n * k).
from collections import deque


def path_with_eliminations(grid, k):
    rows, cols = len(grid), len(grid[0])
    start = (0, 0, 0)
    previous = {start: None}
    queue = deque([start])
    while queue:
        r, c, used = queue.popleft()
        if (r, c) == (rows - 1, cols - 1):
            path = []
            state = (r, c, used)
            while state is not None:
                path.append((state[0], state[1]))
                state = previous[state]
            path.reverse()
            return path
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols:
                new_used = used + grid[nr][nc]
                if new_used <= k and (nr, nc, new_used) not in previous:
```

```
                    previous[(nr, nc, new_used)] = (r, c, used)
                    queue.append((nr, nc, new_used))
    return None


# ============================================================
# FOLLOW-UP 2: Unlimited eliminations: the length is just
# the Manhattan distance
# ============================================================

# ---- follow-up 2: function ----
# Answer: When k is at least rows + cols - 3 you can walk
# straight through everything, so the answer is rows +
# cols - 2 without any search.
# Time: O(1).  Space: O(1).
def shortest_when_k_is_large(rows, cols, k):
    if k >= rows + cols - 3:
        return rows + cols - 2
    return None
```

PATTERN 17 · GRAPH BFS · PROBLEM 20 OF 20

126. Word Ladder II

Hard · Breadth-First Search, Hash Table, String, Backtracking, Bidirectional Search

leetcode.com/problems/word-ladder-ii

PROBLEM

A **transformation sequence** from word `beginWord` to word `endWord` using a dictionary `wordList` is a sequence of words `beginWord -> s1 -> s2 -> ... -> sk` such that:

- Every adjacent pair of words differs by a single letter.
- Every s_i for `1 <= i <= k` is in `wordList`. Note that `beginWord` does not need to be in `wordList`.
- s_k `== endWord`

Given two words, `beginWord` and `endWord`, and a dictionary `wordList`, return *all the **shortest transformation sequences** from* `beginWord` *to* `endWord`*, or an empty list if no such sequence exists. Each sequence should be returned as a list of the words* `[beginWord, s1, s2, ..., sk]`.

Example 1:

```
Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log","cog"]
Output: [["hit","hot","dot","dog","cog"],["hit","hot","lot","log","cog"]]
Explanation: There are 2 shortest transformation sequences:
"hit" -> "hot" -> "dot" -> "dog" -> "cog"
"hit" -> "hot" -> "lot" -> "log" -> "cog"
```

Example 2:

```
Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log"]
Output: []
Explanation: The endWord "cog" is not in wordList, therefore there is no valid transformation
sequence.
```

Constraints:

- `1 <= beginWord.length <= 5`
- `endWord.length == beginWord.length`
- `1 <= wordList.length <= 500`
- `wordList[i].length == beginWord.length`
- `beginWord`, `endWord`, and `wordList[i]` consist of lowercase English letters.
- `beginWord != endWord`
- All the words in `wordList` are **unique**.
- The **sum** of all shortest transformation sequences does not exceed 10^5.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

We need every shortest transformation, not just the length. Words have length at most 5 and the list at most 500, so the BFS depth is small, but the number of paths can be large. Unidirectional BFS expands level by level and records each word's predecessors. Only edges that keep the current distance are stored, so every recorded path is shortest. After the end word is reached, walk the predecessor sets backward to list them all.

Python 3

```python
class Solution:
    def findLadders(
        self, beginWord: str, endWord: str, wordList: List[str]
    ) -> List[List[str]]:
        def dfs(path, cur):
            if cur == beginWord:
                ans.append(path[::-1])
                return
            for precursor in prev[cur]:
                path.append(precursor)
                dfs(path, precursor)
                path.pop()

        ans = []
        words = set(wordList)
        if endWord not in words:
            return ans
        words.discard(beginWord)
        dist = {beginWord: 0}
        prev = defaultdict(set)
        q = deque([beginWord])
        found = False
        step = 0
        while q and not found:
            step += 1
            for i in range(len(q), 0, -1):
                p = q.popleft()
                s = list(p)
                for i in range(len(s)):
                    ch = s[i]
                    for j in range(26):
                        s[i] = chr(ord('a') + j)
                        t = ''.join(s)
                        if dist.get(t, 0) == step:
                            prev[t].add(p)
                        if t not in words:
                            continue
                        prev[t].add(p)
                        words.discard(t)
                        q.append(t)
                        dist[t] = step
                        if endWord == t:
                            found = True
                    s[i] = ch
        if found:
            path = [endWord]
            dfs(path, endWord)
        return ans
```

Complexity. Time: O(N * L^2) for the search, plus the number of shortest paths times their length to list them. Space: O(N * L) plus the paths.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count the shortest ladders without listing them


# ============================================================
# FOLLOW-UP 1: Count the shortest ladders without listing
# them
# ============================================================

# ---- follow-up 1: function ----
# Answer: BFS by layers storing how many shortest ways
# reach each word: the number of ways to a word is the sum
# over its previous-layer neighbours.
```

```
# Time: O(N * L^2).  Space: O(N).
def count_shortest_ladders(begin, end, word_list):
    words = set(word_list)
    if end not in words:
        return 0
    ways = {begin: 1}
    current = {begin}
    words.discard(begin)
    while current:
        following = {}
        for word in current:
            for i in range(len(word)):
                for letter in "abcdefghijklmnopqrstuvwxyz":
                    new = word[:i] + letter + word[i + 1:]
                    if new in words:
                        following[new] = following.get(new, 0) + ways[word]
        if end in following:
            return following[end]
        for word in following:
            words.discard(word)
            ways[word] = following[word]
        current = set(following)
    return 0
```

PATTERN 18 OF 41

Matrix / Grid Traversal

20 problems · 4 Easy · 15 Medium · 1 Hard

THE IDEA

A grid is a graph: each cell has up to four neighbours. Use the four-direction list and a bounds check, then DFS or BFS as needed.

Signals: number of islands, flood fill, surrounded regions, spiral order, rotate image, set matrix zeroes, word search.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
DIRS = ((1, 0), (-1, 0), (0, 1), (0, -1))
for dr, dc in DIRS:
    nr, nc = r + dr, c + dc
    if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
        visit(nr, nc)
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Island Perimeter	Easy	LC 463	997
2	Flood Fill (see page)	Easy	LC 733	842
3	Toeplitz Matrix	Easy	LC 766	1000
4	Transpose Matrix	Easy	LC 867	1003
5	★ **Number of Islands** (see page)	Medium	LC 200	846
6	Rotting Oranges (see page)	Medium	LC 994	924
7	★ **Spiral Matrix**	Medium	LC 54	1005
8	Spiral Matrix II	Medium	LC 59	1009
9	★ **Rotate Image**	Medium	LC 48	1012
10	★ **Set Matrix Zeroes**	Medium	LC 73	1015
11	★ **Word Search** (see page)	Medium	LC 79	815
12	Pacific Atlantic Water Flow (see page)	Medium	LC 417	858
13	Surrounded Regions (see page)	Medium	LC 130	862
14	★ **01 Matrix** (see page)	Medium	LC 542	927
15	Diagonal Traverse	Medium	LC 498	1019
16	★ **Game of Life**	Medium	LC 289	1022
17	Shortest Path in Binary Matrix (see page)	Medium	LC 1091	931
18	Number of Distinct Islands	Medium	LC 694	1026
19	Where Will the Ball Fall	Medium	LC 1706	1030
20	Unique Paths III	Hard	LC 980	1033

463. Island Perimeter

Easy · Depth-First Search, Breadth-First Search, Array, Matrix

leetcode.com/problems/island-perimeter

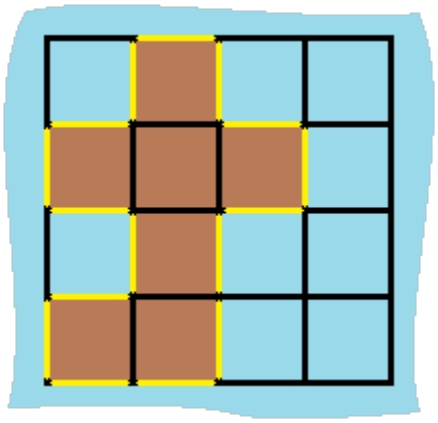
PROBLEM

You are given `row x col grid` representing a map where `grid[i][j] = 1` represents land and `grid[i][j] = 0` represents water.

Grid cells are connected **horizontally/vertically** (not diagonally). The `grid` is completely surrounded by water, and there is exactly one island (i.e., one or more connected land cells).

The island doesn't have "lakes", meaning the water inside isn't connected to the water around the island. One cell is a square with side length 1. The grid is rectangular, width and height don't exceed 100. Determine the perimeter of the island.

Example 1:



```
Input: grid = [[0,1,0,0],[1,1,1,0],[0,1,0,0],[1,1,0,0]]
Output: 16
Explanation: The perimeter is the 16 yellow stripes in the image above.
```

Example 2:

```
Input: grid = [[1]]
Output: 4
```

Example 3:

```
Input: grid = [[1,0]]
Output: 4
```

Constraints:

- `row == grid.length`
- `col == grid[i].length`
- `1 <= row, col <= 100`
- `grid[i][j]` is `0` or `1`.
- There is exactly one island in `grid`.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

The island is 4-connected; perimeter is 4 per land cell minus twice every shared edge. Checking all four neighbors needs bounds tests. Add 4 for each land cell; if the cell below or to the right is also land, that edge is shared, so subtract 2. Looking only down and right counts each inner edge once. No graph or DFS is required; one grid scan yields the perimeter.

Python 3

```python
class Solution:
    def islandPerimeter(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        ans = 0
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 1:
                    ans += 4
                    if i < m - 1 and grid[i + 1][j] == 1:
                        ans -= 2
                    if j < n - 1 and grid[i][j + 1] == 1:
                        ans -= 2
        return ans
```

Complexity. Time: O(m * n). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The OUTER perimeter only (do not count the edges of
# lakes inside the island)
# 2. How many land cells touch water or the border?


# ============================================================
# FOLLOW-UP 1: The OUTER perimeter only (do not count the
# edges of lakes inside the island)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Flood the water from outside the grid; every
# land edge that touches this outside water is part of the
# outer perimeter.
# Time: O(m * n).  Space: O(m * n).
def outer_perimeter(grid):
    rows, cols = len(grid), len(grid[0])
    outside = set()
    stack = []
    for r in range(-1, rows + 1):
        for c in range(-1, cols + 1):
            if r in (-1, rows) or c in (-1, cols):
                outside.add((r, c))
                stack.append((r, c))
    while stack:
        r, c = stack.pop()
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if -1 <= nr <= rows and -1 <= nc <= cols and (nr, nc) not in outside:
                if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                    continue
                outside.add((nr, nc))
                stack.append((nr, nc))
    perimeter = 0
    for r in range(rows):
```

```
        for c in range(cols):
            if grid[r][c] == 1:
                for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
                    if (nr, nc) in outside:
                        perimeter += 1
    return perimeter


# ============================================================
# FOLLOW-UP 2: How many land cells touch water or the
# border?
# ============================================================

# ---- follow-up 2: function ----
# Answer: A land cell touches water when at least one of
# its four neighbours is water or outside the grid.
# Time: O(m * n).  Space: O(1).
def coastal_land_cells(grid):
    rows, cols = len(grid), len(grid[0])
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
                    if not (0 <= nr < rows and 0 <= nc < cols) or grid[nr][nc] == 0:
```

```
                        count += 1
                        break
    return count
```

766. Toeplitz Matrix

Easy · Array, Matrix

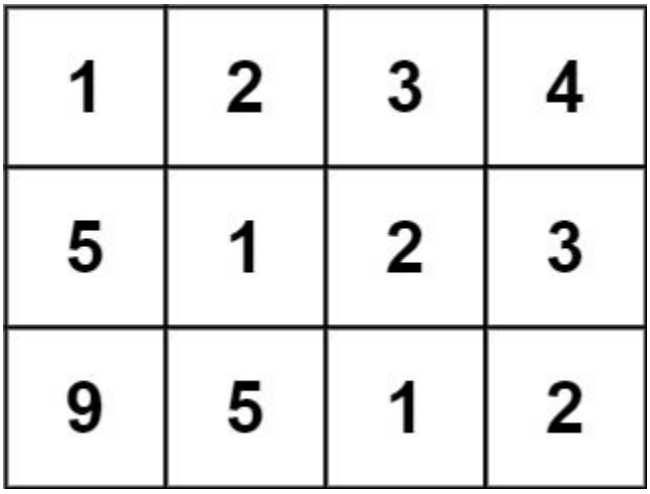
leetcode.com/problems/toeplitz-matrix

PROBLEM

Given an `m x n matrix`, return `true` *if the matrix is Toeplitz. Otherwise, return* `false`.

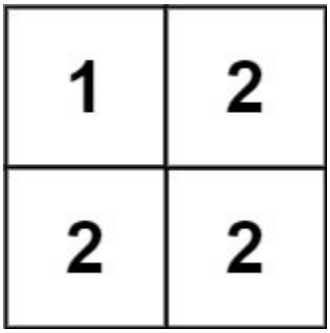
A matrix is **Toeplitz** if every diagonal from top-left to bottom-right has the same elements.

Example 1:




```
Input: matrix = [[1,2,3,4],[5,1,2,3],[9,5,1,2]]
Output: true
Explanation:
In the above grid, the diagonals are:
"[9]", "[5, 5]", "[1, 1, 1]", "[2, 2, 2]", "[3, 3]", "[4]".
In each diagonal all elements are the same, so the answer is True.
```

Example 2:




```
Input: matrix = [[1,2],[2,2]]
Output: false
Explanation:
The diagonal "[1, 2]" has different elements.
```

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 20`
- `0 <= matrix[i][j] <= 99`

Follow up:

- What if the `matrix` is stored on disk, and the memory is limited such that you can only load at most one row of the matrix into the memory at once?
- What if the `matrix` is so large that you can only load up a partial row into the memory at once?

SOLUTION (PYTHON 3)

Solution 1: Single Traversal

> **How to think about it**
>
> A Toeplitz matrix has equal diagonals, i.e. every cell equals its upper-left neighbor. The grid is at most 20× 20. Scan from (1,1); fail if matrix[i][j]≠ matrix[i-1][j-1]. The first row and column have no predecessor.

According to the problem description, the characteristic of a Toeplitz matrix is that each element is equal to the element in its upper left corner. Therefore, we only need to iterate through each element in the matrix and check if it is equal to the element in its upper left corner.

The time complexity is O(m × n), where m and n are the number of rows and columns of the matrix, respectively. The space complexity is O(1).

Python 3

```python
class Solution:
    def isToeplitzMatrix(self, matrix: List[List[int]]) -> bool:
        m, n = len(matrix), len(matrix[0])
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][j] != matrix[i - 1][j - 1]:
                    return False
        return True
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The matrix arrives one row at a time (it does not fit
# in memory)
# 2. Return the first cell that breaks the pattern


# ============================================================
# FOLLOW-UP 1: The matrix arrives one row at a time (it
# does not fit in memory)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep only the previous row: every number must
# equal the one up-left of it, i.e. row[i] == previous[i -
# 1].
# Time: O(m * n).  Space: O(n) (one row kept).
def is_toeplitz_streaming(rows):
    previous = None
    for row in rows:
        if previous is not None:
            for i in range(1, len(row)):
                if row[i] != previous[i - 1]:
                    return False
        previous = row
    return True


# ============================================================
# FOLLOW-UP 2: Return the first cell that breaks the
# pattern
```

```
# ============================================================

# ---- follow-up 2: function ----
# Answer: Compare each cell with its up-left neighbour and
# report the first mismatch in reading order.
# Time: O(m * n).  Space: O(1).
def first_toeplitz_violation(matrix):
    for r in range(1, len(matrix)):
        for c in range(1, len(matrix[0])):
            if matrix[r][c] != matrix[r - 1][c - 1]:
                return (r, c)
    return None
```

867. Transpose Matrix

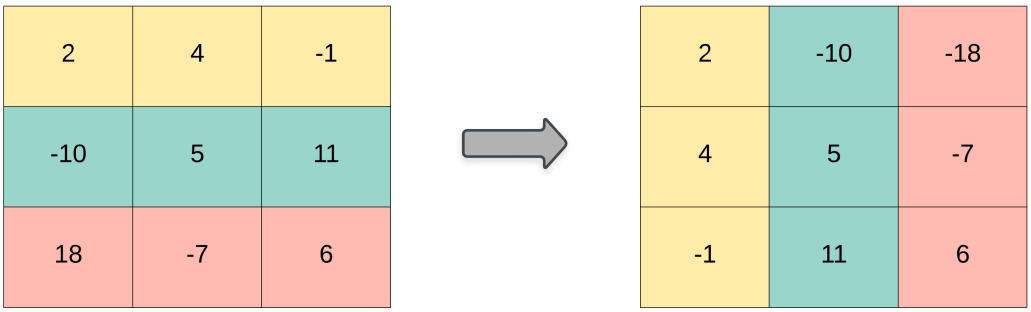
Easy · Array, Matrix, Simulation

leetcode.com/problems/transpose-matrix

PROBLEM

Given a 2D integer array `matrix`, return *the **transpose** of* `matrix`.

The **transpose** of a matrix is the matrix flipped over its main diagonal, switching the matrix's row and column indices.



Example 1:

```
Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [[1,4,7],[2,5,8],[3,6,9]]
```

Example 2:

```
Input: matrix = [[1,2,3],[4,5,6]]
Output: [[1,4],[2,5],[3,6]]
```

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 1000`
- `1 <= m * n <= 10^5`
- `-10^9 <= matrix[i][j] <= 10^9`

SOLUTION (PYTHON 3)

Solution 1: Simulation

> **How to think about it**
>
> A transpose swaps rows and columns. The matrix is small enough to build a new one; in-place swaps are unnecessary. zip over the matrix turns each column into a new row.

Let m be the number of rows and n be the number of columns in the matrix matrix. According to the definition of transpose, the transposed matrix ans will have n rows and m columns.

For any position (i, j) in ans, it corresponds to the position (j, i) in the matrix matrix. Therefore, we traverse each element in the matrix matrix and transpose it to the corresponding position in ans.

After the traversal, we return ans.

The time complexity is O(m × n), where m and n are the number of rows and columns in the matrix matrix, respectively. Ignoring the space consumption of the answer, the space complexity is O(1).

Python 3

```python
class Solution:
    def transpose(self, matrix: List[List[int]]) -> List[List[int]]:
        return [list(row) for row in zip(*matrix)]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Transpose a SQUARE matrix in place
# 2. The matrix is sparse (stored as a dictionary of non-
# zero cells)


# ============================================================
# FOLLOW-UP 1: Transpose a SQUARE matrix in place
# ============================================================

# ---- follow-up 1: function ----
# Answer: Swap every pair of cells that mirror each other
# across the diagonal.
# Time: O(n^2).  Space: O(1).
def transpose_in_place(matrix):
    n = len(matrix)
    for r in range(n):
        for c in range(r + 1, n):
            matrix[r][c], matrix[c][r] = matrix[c][r], matrix[r][c]
    return matrix


# ============================================================
# FOLLOW-UP 2: The matrix is sparse (stored as a
# dictionary of non-zero cells)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Swap the row and column in every key; work is
# proportional to the number of stored cells.
# Time: O(k), k = non-zero cells.  Space: O(k).
def transpose_sparse(cells):
    result = {}
    for (row, col), value in cells.items():
        result[(col, row)] = value
    return result
```

54. Spiral Matrix

Medium · Array, Matrix, Simulation

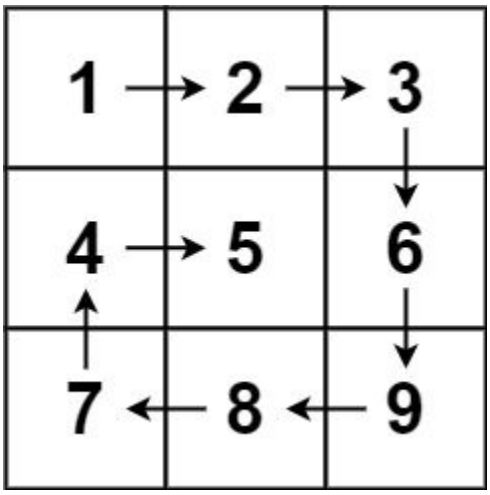
leetcode.com/problems/spiral-matrix

★ **TOP 7 PROBLEM** in Matrix / Grid Traversal: master this one first

PROBLEM

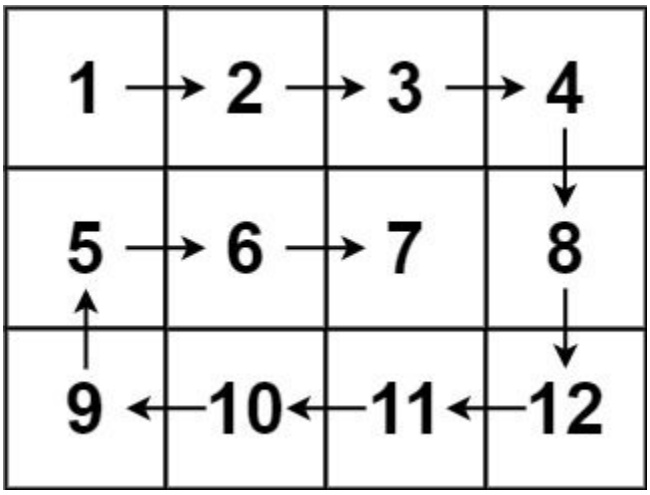
Given an `m x n matrix`, return *all elements of the* `matrix` *in spiral order*.

Example 1:




```
Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,2,3,6,9,8,7,4,5]
```

Example 2:




```
Input: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]
Output: [1,2,3,4,8,12,11,10,9,5,6,7]
```

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 10`
- `-100 <= matrix[i][j] <= 100`

SOLUTIONS (PYTHON 3)

Solution 1: Simulation

> **How to think about it**
>
> The first idea is to peel layers: four loops per layer for top, right, bottom, left, then shrink the bounds. m, n ≤ 10 would pass, but a leftover single row or column is easy to get wrong. The waste is hand-managing four edges. A spiral is: keep the current direction, and turn right when the next cell is out of bounds or already visited. So we cycle four directions and mark visited cells in vis, taking m × n steps.

We can simulate the entire traversal process. We use i and j to represent the row and column of the current element being visited, and k to represent the current direction. We use an array or hash table vis to record whether each element has been visited. Each time we visit an element, we mark it as visited, then move one step forward in the current direction. If moving forward results in an out-of-bounds condition or the element has already been visited, we change direction and continue moving forward until the entire matrix has been traversed.

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the number of rows and columns of the matrix, respectively.

Python 3

```python
class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        m, n = len(matrix), len(matrix[0])
        dirs = (0, 1, 0, -1, 0)
        vis = [[False] * n for _ in range(m)]
        i = j = k = 0
        ans = []
        for _ in range(m * n):
            ans.append(matrix[i][j])
            vis[i][j] = True
            x, y = i + dirs[k], j + dirs[k + 1]
            if x < 0 or x >= m or y < 0 or y >= n or vis[x][y]:
                k = (k + 1) % 4
            i += dirs[k]
            j += dirs[k + 1]
        return ans
```

Solution 2: Simulation (Space Optimization)

> **How to think about it**
>
> Solution 1 already walks the matrix in linear time, but vis still costs O(mn) extra space. Values lie in [-100, 100], so the visit mark can be written back into the cell. What it lacks is an in-place mark: add 300 to a visited cell, and treat > 100 as seen. Subtract 300 afterwards. Extra space drops to O(1).

Notice that the range of matrix element values is [-100, 100]. Therefore, we can add a large value, such as 300, to the visited elements. This way, we only need to check if the visited element is greater than 100, without needing extra space to record whether it has been visited. If we need to restore the original values of the visited elements, we can traverse the matrix again after the traversal is complete and subtract 300 from all elements.

The time complexity is O(m × n), where m and n are the number of rows and columns of the matrix, respectively. The space complexity is O(1).

Python 3

```python
class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        m, n = len(matrix), len(matrix[0])
        dirs = (0, 1, 0, -1, 0)
        i = j = k = 0
        ans = []
        for _ in range(m * n):
            ans.append(matrix[i][j])
            matrix[i][j] += 300
            x, y = i + dirs[k], j + dirs[k + 1]
            if x < 0 or x >= m or y < 0 or y >= n or matrix[x][y] > 100:
                k = (k + 1) % 4
            i += dirs[k]
            j += dirs[k + 1]
        for i in range(m):
            for j in range(n):
                matrix[i][j] -= 300
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Read the matrix counter-clockwise (down the first
# column first)
# 2. Return the cells in spiral order from the CENTRE
# outwards


# ========================================================
# FOLLOW-UP 1: Read the matrix counter-clockwise (down the
# first column first)
# ========================================================

# ---- follow-up 1: function ----
# Answer: Same four borders, but visit them in the order
# left column down, bottom row right, right column up, top
# row left.
# Time: O(m * n).  Space: O(1) extra (the output is O(m *
# n)).
def spiral_counter_clockwise(matrix):
    result = []
    if not matrix:
        return result
    top, bottom = 0, len(matrix) - 1
    left, right = 0, len(matrix[0]) - 1
    while top <= bottom and left <= right:
        for r in range(top, bottom + 1):
            result.append(matrix[r][left])
        left += 1
        for c in range(left, right + 1):
            result.append(matrix[bottom][c])
        bottom -= 1
        if left <= right:
            for r in range(bottom, top - 1, -1):
                result.append(matrix[r][right])
            right -= 1
        if top <= bottom:
            for c in range(right, left - 1, -1):
                result.append(matrix[top][c])
            top += 1
    return result


# ========================================================
# FOLLOW-UP 2: Return the cells in spiral order from the
```

```
# CENTRE outwards
# ============================================================

# ---- follow-up 2: function ----
# Answer: The reverse of the inward spiral (a reversed
# list), starting at the innermost cell.
# Time: O(m * n).  Space: O(m * n).
def spiral_outwards(matrix):
    result = []
    top, bottom = 0, len(matrix) - 1
    left, right = 0, len(matrix[0]) - 1
    while top <= bottom and left <= right:
        for c in range(left, right + 1):
            result.append(matrix[top][c])
        for r in range(top + 1, bottom + 1):
            result.append(matrix[r][right])
```

```
        if top < bottom:
            for c in range(right - 1, left - 1, -1):
                result.append(matrix[bottom][c])
        if left < right:
            for r in range(bottom - 1, top, -1):
                result.append(matrix[r][left])
        top += 1
        bottom -= 1
        left += 1
        right -= 1
    result.reverse()
    return result
```

59. Spiral Matrix II

Medium · Array, Matrix, Simulation

leetcode.com/problems/spiral-matrix-ii

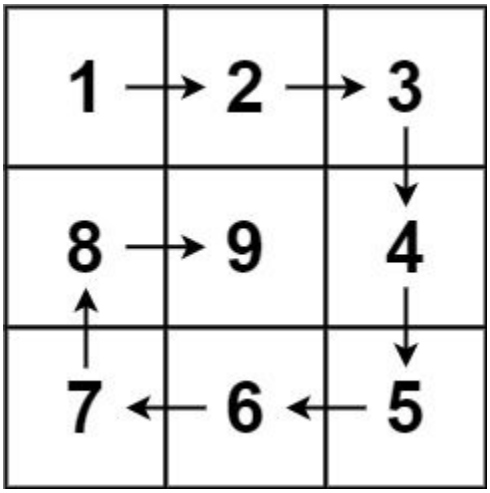
★ **TOP 7 PROBLEM** in Simulation / Array Rearrangement: master this one first

Also appears in pattern 36 Simulation / Array Rearrangement

PROBLEM

Given a positive integer `n`, generate an `n x n matrix` filled with elements from `1` to n^2 in spiral order.

Example 1:




```
Input: n = 3
Output: [[1,2,3],[8,9,4],[7,6,5]]
```

Example 2:

```
Input: n = 1
Output: [[1]]
```

Constraints:

- `1 <= n <= 20`

SOLUTION (PYTHON 3)

Solution 1: Simulation

How to think about it

The first idea is to fill by layers: four edges per layer, then shrink. n ≤ 20 would pass, with the same boundary bugs. The waste is hand-managing four edges. As in spiral read: keep the direction, turn right when the next cell is out of bounds or already filled. Empty cells are 0, filled cells are nonzero, so no extra vis. So cycle directions and write 1 through n^2.

We can directly simulate the process of generating the spiral matrix.

Define a 2D array ans to store the spiral matrix. Use i and j to represent the current row and column indices, and use k to represent the current direction index. dirs represents the mapping between direction indices and directions.

Starting from 1, fill each position in the matrix sequentially. After filling a position, calculate the row and column indices of the next position. If the next position is out of bounds or has already been filled, change the direction and then calculate the row and column indices of the next position.

The time complexity is $O(n^2)$, where n is the side length of the matrix. Ignoring the space consumption of the answer array, the space complexity is O(1).

Python 3

```python
class Solution:
    def generateMatrix(self, n: int) -> List[List[int]]:
        ans = [[0] * n for _ in range(n)]
        dirs = (0, 1, 0, -1, 0)
        i = j = k = 0
        for v in range(1, n * n + 1):
            ans[i][j] = v
            x, y = i + dirs[k], j + dirs[k + 1]
            if x < 0 or x >= n or y < 0 or y >= n or ans[x][y]:
                k = (k + 1) % 4
            i, j = i + dirs[k], j + dirs[k + 1]
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Fill a rectangle with m rows and n columns
# 2. Where does the number k sit in the n x n spiral
# (without building it)?


# ============================================================
# FOLLOW-UP 1: Fill a rectangle with m rows and n columns
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same four borders, with separate limits for the
# rows and the columns.
# Time: O(m * n).  Space: O(m * n).
def generate_spiral_rectangle(m, n):
    matrix = []
    for r in range(m):
        matrix.append([0] * n)
    top, bottom, left, right = 0, m - 1, 0, n - 1
    number = 1
    while top <= bottom and left <= right:
        for c in range(left, right + 1):
            matrix[top][c] = number
            number += 1
        top += 1
        for r in range(top, bottom + 1):
            matrix[r][right] = number
            number += 1
        right -= 1
        if top <= bottom:
            for c in range(right, left - 1, -1):
                matrix[bottom][c] = number
                number += 1
            bottom -= 1
        if left <= right:
            for r in range(bottom, top - 1, -1):
                matrix[r][left] = number
                number += 1
            left += 1
    return matrix


# ============================================================
# FOLLOW-UP 2: Where does the number k sit in the n x n
# spiral (without building it)?
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Walk the spiral with a position and a direction,
# turning when the next cell is outside or already used,
# until k steps are done: O(k) time and O(1) memory for
# the grid-less walk (the visited set is replaced by the
# shrinking borders).
# Time: O(n^2) in the worst case (walks the spiral until
# k).  Space: O(1).
def position_of_number(n, k):
    top, bottom, left, right = 0, n - 1, 0, n - 1
    number = 1
    while top <= bottom and left <= right:
        for c in range(left, right + 1):
```

```
            if number == k:
                return (top, c)
            number += 1
        top += 1
        for r in range(top, bottom + 1):
            if number == k:
                return (r, right)
            number += 1
        right -= 1
        for c in range(right, left - 1, -1):
            if top <= bottom:
                if number == k:
                    return (bottom, c)
                number += 1
        bottom -= 1
        for r in range(bottom, top - 1, -1):
            if left <= right:
                if number == k:
                    return (r, left)
                number += 1
        left += 1
    return None
```

48. Rotate Image

Medium · Array, Math, Matrix

leetcode.com/problems/rotate-image

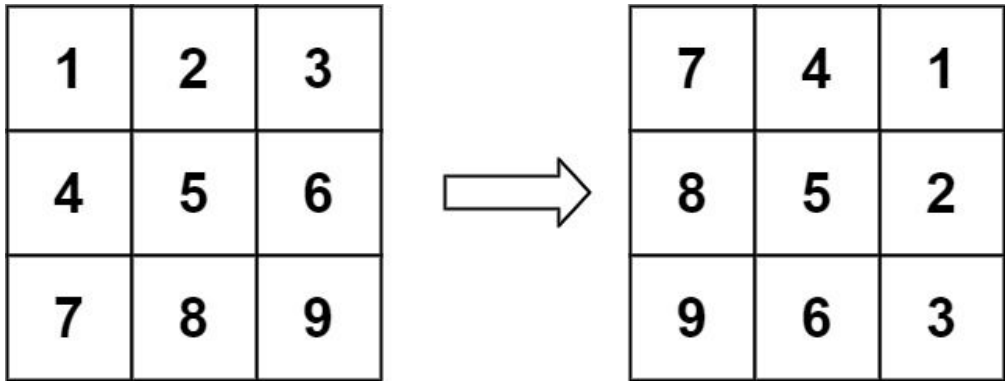
★ **TOP 7 PROBLEM** in Matrix / Grid Traversal: master this one first

PROBLEM

You are given an `n x n` 2D `matrix` representing an image, rotate the image by **90** degrees (clockwise).

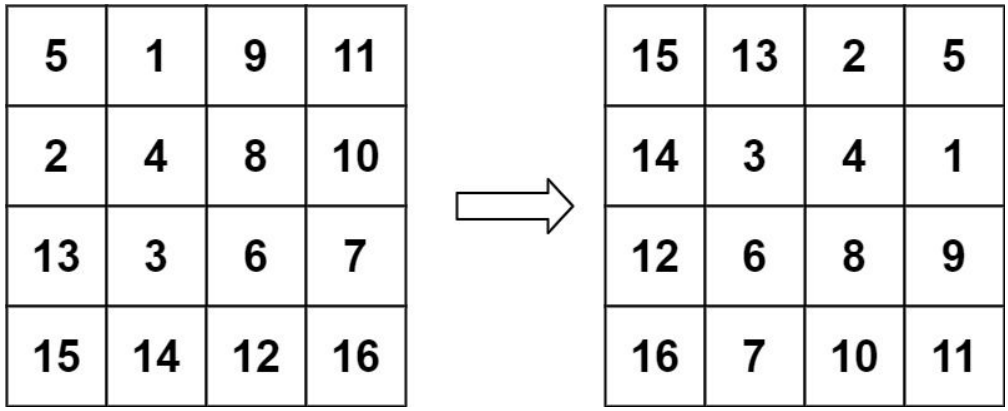
You have to rotate the image **in-place**, which means you have to modify the input 2D matrix directly. **DO NOT** allocate another 2D matrix and do the rotation.

Example 1:




```
Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [[7,4,1],[8,5,2],[9,6,3]]
```

Example 2:




```
Input: matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]
Output: [[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]
```

Constraints:

- `n == matrix.length == matrix[i].length`
- `1 <= n <= 20`
- `-1000 <= matrix[i][j] <= 1000`

SOLUTION (PYTHON 3)

Solution 1: In-place Rotation

> **How to think about it**
>
> The first idea is a new matrix with matrix[j][n-1-i] ← matrix[i][j]. Correct, O(n^2) time and space. n ≤ 20 fits, but the problem requires in-place. The extra matrix is the bottleneck. A 90° clockwise turn factors into two in-place flips: reverse upside-down, then transpose across the main diagonal. (i, j) goes to (n-1-i, j), then to (j, n-1-i) — the target. Two rounds of swaps, O(1) extra space.

According to the problem requirements, we need to rotate matrix[i][j] to matrix[j][n - i - 1].

We can first flip the matrix upside down, i.e., swap matrix[i][j] with matrix[n - i - 1][j], and then flip the matrix along the main diagonal, i.e., swap matrix[i][j] with matrix[j][i]. This way, we can rotate matrix[i][j] to matrix[j][n - i - 1].

The time complexity is O(n^2), where n is the side length of the matrix. The space complexity is O(1).

Python 3

```
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)
        for i in range(n >> 1):
            for j in range(n):
                matrix[i][j], matrix[n - i - 1][j] = matrix[n - i - 1][j], matrix[i][j]
        for i in range(n):
            for j in range(i):
                matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Rotate counter-clockwise
# 2. Rotate by k quarter turns


# ============================================================
# FOLLOW-UP 1: Rotate counter-clockwise
# ============================================================

# ---- follow-up 1: function ----
# Answer: Transpose the matrix, then reverse the order of
# the rows (instead of reversing every row).
# Time: O(n^2).  Space: O(1).
def rotate_counter_clockwise(matrix):
    n = len(matrix)
    for r in range(n):
        for c in range(r + 1, n):
            matrix[r][c], matrix[c][r] = matrix[c][r], matrix[r][c]
    matrix.reverse()
    return matrix


# ============================================================
# FOLLOW-UP 2: Rotate by k quarter turns
# ============================================================

# ---- follow-up 2: function ----
# Answer: Only k modulo 4 matters; apply a clockwise
# quarter turn that many times.
# Time: O(n^2).  Space: O(1).
def rotate_clockwise_once(matrix):
    n = len(matrix)
    for r in range(n):
        for c in range(r + 1, n):
```

```
            matrix[r][c], matrix[c][r] = matrix[c][r], matrix[r][c]
    for row in matrix:
        row.reverse()


def rotate_k_times(matrix, k):
    for turn in range(k % 4):
        rotate_clockwise_once(matrix)
    return matrix
```

73. Set Matrix Zeroes

Medium · Array, Hash Table, Matrix

leetcode.com/problems/set-matrix-zeroes

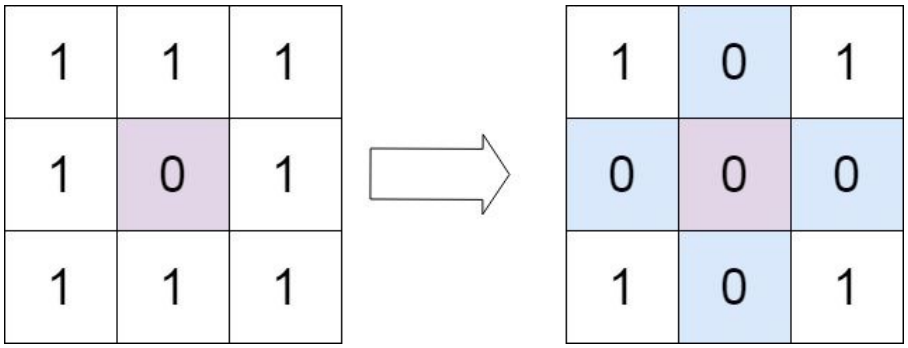
★ **TOP 7 PROBLEM** in Matrix / Grid Traversal: master this one first

PROBLEM

Given an `m x n` integer matrix `matrix`, if an element is `0`, set its entire row and column to `0`'s.

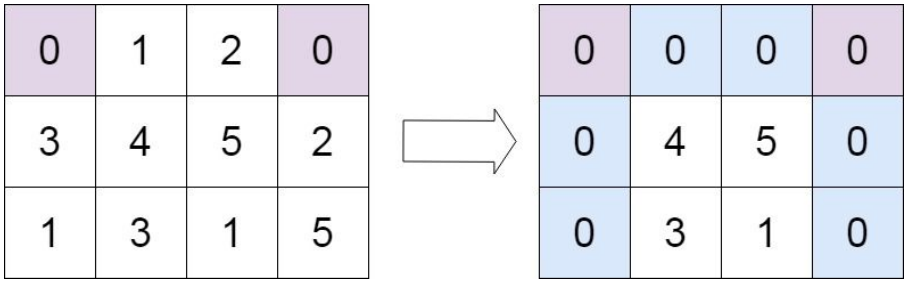
You must do it in place.

Example 1:




```
Input: matrix = [[1,1,1],[1,0,1],[1,1,1]]
Output: [[1,0,1],[0,0,0],[1,0,1]]
```

Example 2:




```
Input: matrix = [[0,1,2,0],[3,4,5,2],[1,3,1,5]]
Output: [[0,0,0,0],[0,4,5,0],[0,3,1,0]]
```

Constraints:

- `m == matrix.length`
- `n == matrix[0].length`
- `1 <= m, n <= 200`
- -2^{31} `<= matrix[i][j] <=` $2^{31} - 1$

Follow up:

- A straightforward solution using `O(mn)` space is probably a bad idea.
- A simple improvement uses `O(m + n)` space, but still not the best solution.
- Could you devise a constant space solution?

SOLUTIONS (PYTHON 3)

Solution 1: Array Mark

How to think about it

Zeroing a whole row and column as soon as we see a 0 wipes zeros we have not scanned yet, and we cannot tell original zeros from new ones. Copying an O(mn) matrix works, but the follow-up rejects that space. m,n ≤ 200, so two scans are fine. Record which rows and columns must be zeroed, then rewrite. An m-length row mark and an n-length column mark suffice: first pass only flags, second pass writes zeros. Extra space drops from O(mn) to O(m+n), still not the constant-space follow-up.

We use arrays `rows` and `cols` to mark the rows and columns to be cleared.

Then traverse the matrix again, and clear the elements in the rows and columns marked in `rows` and `cols`.

The time complexity is O(m× n), and the space complexity is O(m+n). Where m and n are the number of rows and columns of the matrix respectively.

Python 3

```
class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        m, n = len(matrix), len(matrix[0])
        row = [False] * m
        col = [False] * n
        for i in range(m):
            for j in range(n):
                if matrix[i][j] == 0:
                    row[i] = col[j] = True
        for i in range(m):
            for j in range(n):
                if row[i] or col[j]:
                    matrix[i][j] = 0
```

Solution 2: Mark in Place

How to think about it

Method 1 still spends O(m+n) on mark arrays. The follow-up wants O(1) extra space, and the first row and first column can play the role of row and col. Those strips are both data and marks, so we save with i0, j0 whether they themselves must be zeroed, update the interior, and only then clear the first row and column — otherwise the marks get overwritten too early.

In the first method, we use an additional array to mark the rows and columns to be cleared. In fact, we can also use the first row and first column of the matrix to mark them, without creating an additional array.

Since the first row and the first column are used to mark, their values may change due to the mark, so we need additional variables i0, j0 to mark whether the first row and the first column need to be cleared.

The time complexity is O(m× n), and the space complexity is O(1). Where m and n are the number of rows and columns of the matrix respectively.

Python 3

```python
class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        m, n = len(matrix), len(matrix[0])
        i0 = any(v == 0 for v in matrix[0])
        j0 = any(matrix[i][0] == 0 for i in range(m))
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][j] == 0:
                    matrix[i][0] = matrix[0][j] = 0
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][0] == 0 or matrix[0][j] == 0:
                    matrix[i][j] = 0
        if i0:
            for j in range(n):
                matrix[0][j] = 0
        if j0:
            for i in range(m):
                matrix[i][0] = 0
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Use O(1) extra space
# 2. How many rows and columns will be zeroed?


# ============================================================
# FOLLOW-UP 1: Use O(1) extra space
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use the first row and the first column as the
# markers for the other rows and columns, remembering
# separately whether the first row / column themselves
# must be zeroed.
# Time: O(m * n).  Space: O(1).
def set_zeroes_constant_space(matrix):
    rows, cols = len(matrix), len(matrix[0])
    first_row_zero = False
    first_col_zero = False
    for c in range(cols):
        if matrix[0][c] == 0:
            first_row_zero = True
    for r in range(rows):
        if matrix[r][0] == 0:
            first_col_zero = True
    for r in range(1, rows):
        for c in range(1, cols):
            if matrix[r][c] == 0:
                matrix[r][0] = 0
                matrix[0][c] = 0
    for r in range(1, rows):
        for c in range(1, cols):
            if matrix[r][0] == 0 or matrix[0][c] == 0:
                matrix[r][c] = 0
    if first_row_zero:
        for c in range(cols):
            matrix[0][c] = 0
    if first_col_zero:
        for r in range(rows):
            matrix[r][0] = 0
    return matrix
```

```
# ============================================================
# FOLLOW-UP 2: How many rows and columns will be zeroed?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Collect the rows and columns that contain a
# zero; their counts are the answer.
# Time: O(m * n).  Space: O(m + n).
def zeroed_lines(matrix):
    zero_rows = set()
    zero_cols = set()
    for r in range(len(matrix)):
        for c in range(len(matrix[0])):
            if matrix[r][c] == 0:
                zero_rows.add(r)
                zero_cols.add(c)
    return len(zero_rows), len(zero_cols)
```

498. Diagonal Traverse

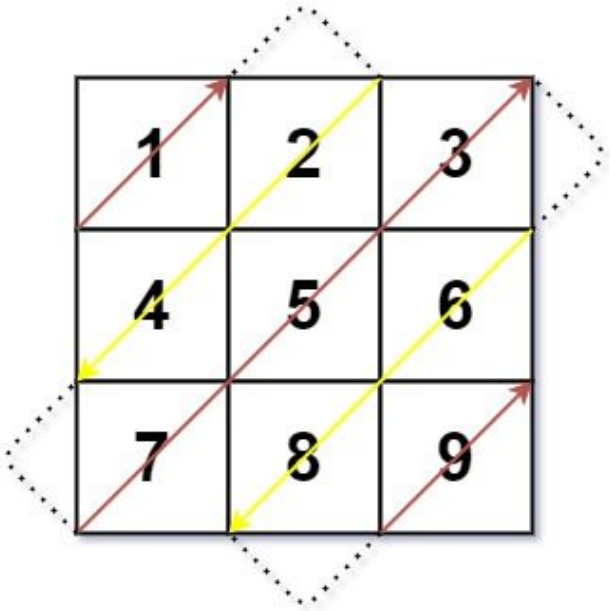
Medium · Array, Matrix, Simulation

leetcode.com/problems/diagonal-traverse

PROBLEM

Given an `m x n` matrix `mat`, return *an array of all the elements of the array in a diagonal order*.

Example 1:




```
Input: mat = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,2,4,7,5,3,6,8,9]
```

Example 2:

```
Input: mat = [[1,2],[3,4]]
Output: [1,2,3,4]
```

Constraints:

- `m == mat.length`
- `n == mat[i].length`
- `1 <= m, n <= 10^4`
- `1 <= m * n <= 10^4`
- `-10^5 <= mat[i][j] <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: Fixed Point Traversal

How to think about it

Traverse the matrix in zigzag diagonals. Simulating a direction and bouncing off edges needs many boundary cases. There are m+n-1 diagonals, each with a closed-form start. Diagonal k is collected top-right to bottom-left, starting at (0,k) when k<n and (k-n+1,n-1) otherwise. Even k is reversed to match the required zigzag. Always walking down-left and flipping on even k avoids switching a direction vector on the border.

For each round k, we fix the starting point from the top-right and traverse diagonally to the bottom-left to get t. If k is even, we reverse t.

The time complexity is O(m × n), and the space complexity is O(1). Ignoring the space used for the answer.

Python 3

```python
class Solution:
    def findDiagonalOrder(self, mat: List[List[int]]) -> List[int]:
        m, n = len(mat), len(mat[0])
        ans = []
        for k in range(m + n - 1):
            t = []
            i = 0 if k < n else k - n + 1
            j = k if k < n else n - 1
            while i < m and j >= 0:
                t.append(mat[i][j])
                i += 1
                j -= 1
            if k % 2 == 0:
                t = t[::-1]
            ans.extend(t)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Every diagonal read in the SAME direction (no zigzag)
# 2. The main-direction diagonals (r - c is constant),
# from the bottom-left corner


# ============================================================
# FOLLOW-UP 1: Every diagonal read in the SAME direction
# (no zigzag)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Group the cells by r + c; read every group from
# top to bottom.
# Time: O((m + n) * m).  Space: O(1) extra.
def diagonals_same_direction(matrix):
    rows, cols = len(matrix), len(matrix[0])
    result = []
    for total in range(rows + cols - 1):
        for r in range(rows):
            c = total - r
            if 0 <= c < cols:
                result.append(matrix[r][c])
    return result


# ============================================================
# FOLLOW-UP 2: The main-direction diagonals (r - c is
# constant), from the bottom-left corner
# ============================================================

# ---- follow-up 2: function ----
# Answer: Group the cells by r - c and go from the most
# negative to the most positive difference.
# Time: O(m * n).  Space: O(m * n).
def down_right_diagonals(matrix):
    rows, cols = len(matrix), len(matrix[0])
    result = []
    for difference in range(rows - 1, -cols, -1):
        r = difference if difference > 0 else 0
        c = r - difference
```

```
        diagonal = []
        while r < rows and c < cols:
            diagonal.append(matrix[r][c])
            r += 1
            c += 1
        result.append(diagonal)
    return result
```

289. Game of Life

Medium · Array, Matrix, Simulation

leetcode.com/problems/game-of-life

★ **TOP 7 PROBLEM** in Matrix / Grid Traversal, Simulation / Array Rearrangement: master this one first

Also appears in pattern 36 Simulation / Array Rearrangement

PROBLEM

According to Wikipedia's article: "The **Game of Life**, also known simply as **Life**, is a cellular automaton devised by the British mathematician John Horton Conway in 1970."

The board is made up of an `m x n` grid of cells, where each cell has an initial state: **live** (represented by a `1`) or **dead** (represented by a `0`). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules (taken from the above Wikipedia article):

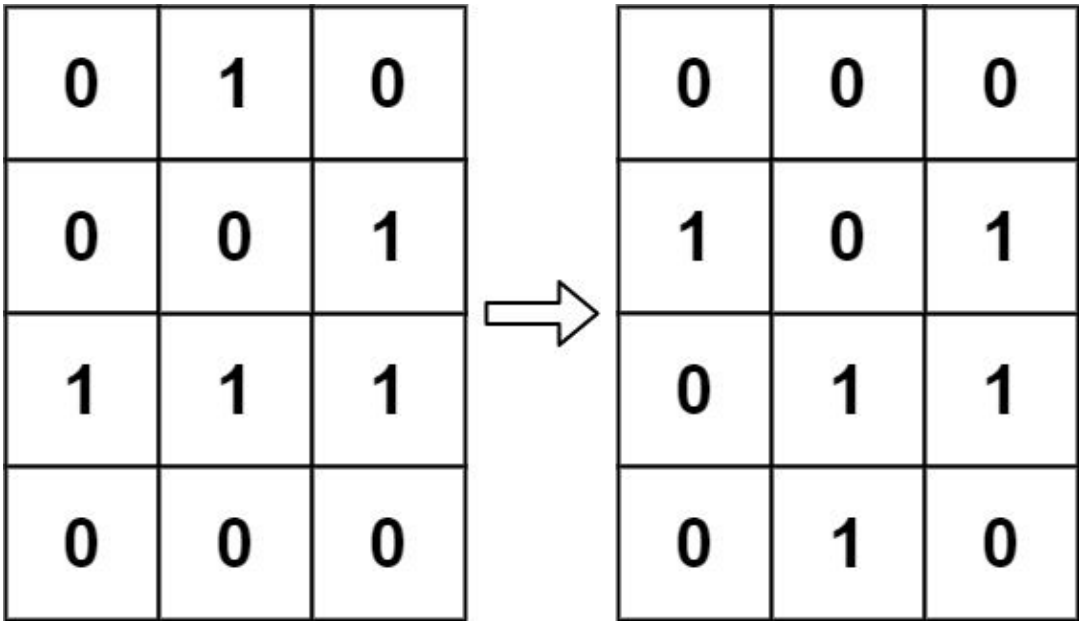
1. Any live cell with fewer than two live neighbors dies as if caused by under-population.
2. Any live cell with two or three live neighbors lives on to the next generation.
3. Any live cell with more than three live neighbors dies, as if by over-population.
4. Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

The next state of the board is determined by applying the above rules simultaneously to every cell in the current state of the `m x n` grid `board`. In this process, births and deaths occur **simultaneously**.

Given the current state of the `board`, **update** the `board` to reflect its next state.

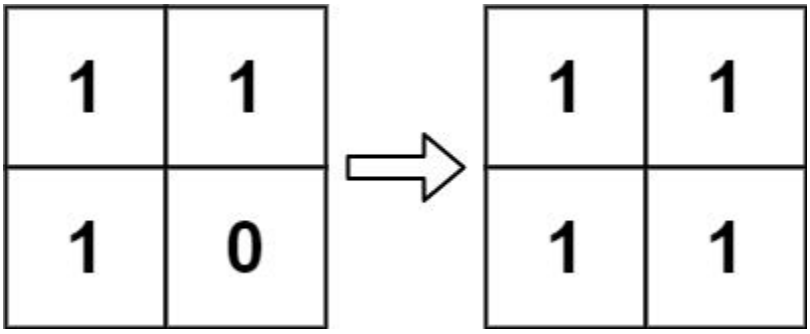
Note that you do not need to return anything.

Example 1:




```
Input: board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]
Output: [[0,0,0],[1,0,1],[0,1,1],[0,1,0]]
```

Example 2:




```
Input: board = [[1,1],[1,0]]
Output: [[1,1],[1,1]]
```

Constraints:

- `m == board.length`
- `n == board[i].length`
- `1 <= m, n <= 25`
- `board[i][j]` is `0` or `1`.

Follow up:

- Could you solve it in-place? Remember that the board needs to be updated simultaneously: You cannot update some cells first and then use their updated values to update other cells.
- In this question, we represent the board using a 2D array. In principle, the board is infinite, which would cause problems when the active area encroaches upon the border of the array (i.e., live cells reach the border). How would you address these problems?

SOLUTION (PYTHON 3)

Solution 1: In-place marking

How to think about it

The next state depends on the current eight neighbors; writing immediately would spoil cells not yet processed. Use sentinels: live-to-dead as 2, dead-to-live as -1, and treat positives as live when counting. A second pass maps 2 to 0 and -1 to 1, all in place.

Let's define two new states. State 2 indicates that the living cell becomes dead in the next state, and state -1 indicates that the dead cell becomes alive in the next state. Therefore, for the current grid we are traversing, if the grid is greater than 0, it means that the current grid is a living cell, otherwise it is a dead cell.

So we can traverse the entire board, for each grid, count the number of living neighbors around the grid, and use the variable live to represent it. If the current grid is a living cell, then when live < 2 or live > 3, the next state of the current grid is a dead cell, that is, state 2; if the current grid is a dead cell, then when live = 3, the next state of the current grid is an active cell, that is, state -1.

Finally, we traverse the board again, and update the grid with state 2 to a dead cell, and update the grid with state -1 to an active cell.

The time complexity is O(m × n), where m and n are the number of rows and columns of the board, respectively. We need to traverse the entire board. And the space complexity is O(1).

Python 3

```python
class Solution:
    def gameOfLife(self, board: List[List[int]]) -> None:
        m, n = len(board), len(board[0])
        for i in range(m):
            for j in range(n):
                live = -board[i][j]
                for x in range(i - 1, i + 2):
                    for y in range(j - 1, j + 2):
                        if 0 <= x < m and 0 <= y < n and board[x][y] > 0:
                            live += 1
                if board[i][j] and (live < 2 or live > 3):
                    board[i][j] = 2
                if board[i][j] == 0 and live == 3:
                    board[i][j] = -1
        for i in range(m):
            for j in range(n):
                if board[i][j] == 2:
                    board[i][j] = 0
                elif board[i][j] == -1:
                    board[i][j] = 1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. An infinite board: store only the live cells
# 2. Run n generations


# ============================================================
# FOLLOW-UP 1: An infinite board: store only the live
# cells
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count the live neighbours of every cell next to
# a live cell with a dictionary; apply the rules to those
# candidates.
# Time: O(L), L = live cells.  Space: O(L).
def life_step_infinite(live):
    counts = {}
    for r, c in live:
        for dr in (-1, 0, 1):
            for dc in (-1, 0, 1):
                if dr or dc:
                    cell = (r + dr, c + dc)
                    counts[cell] = counts.get(cell, 0) + 1
    new_live = set()
    for cell, count in counts.items():
        if count == 3 or (count == 2 and cell in live):
            new_live.add(cell)
    return new_live


# ============================================================
# FOLLOW-UP 2: Run n generations
# ============================================================

# ---- follow-up 2: function ----
# Answer: Apply one step n times; stop early when the
# board stops changing.
# Time: O(g * m * n), g generations.  Space: O(m * n).
def life_after_generations(board, generations):
    rows, cols = len(board), len(board[0])
    for step in range(generations):
        new_board = []
```

```
        for r in range(rows):
            row = []
            for c in range(cols):
                neighbours = 0
                for dr in (-1, 0, 1):
                    for dc in (-1, 0, 1):
                        if (dr or dc) and 0 <= r + dr < rows and 0 <= c + dc < cols:
                            neighbours += board[r + dr][c + dc]
                if board[r][c] == 1:
                    row.append(1 if neighbours in (2, 3) else 0)
                else:
                    row.append(1 if neighbours == 3 else 0)
            new_board.append(row)
        if new_board == board:
            break
        board = new_board
    return board
```

694. Number of Distinct Islands

Medium · Depth-First Search, Breadth-First Search, Union Find, Array, Hash Table, Matrix, Sorting, Hash Function

leetcode.com/problems/number-of-distinct-islands

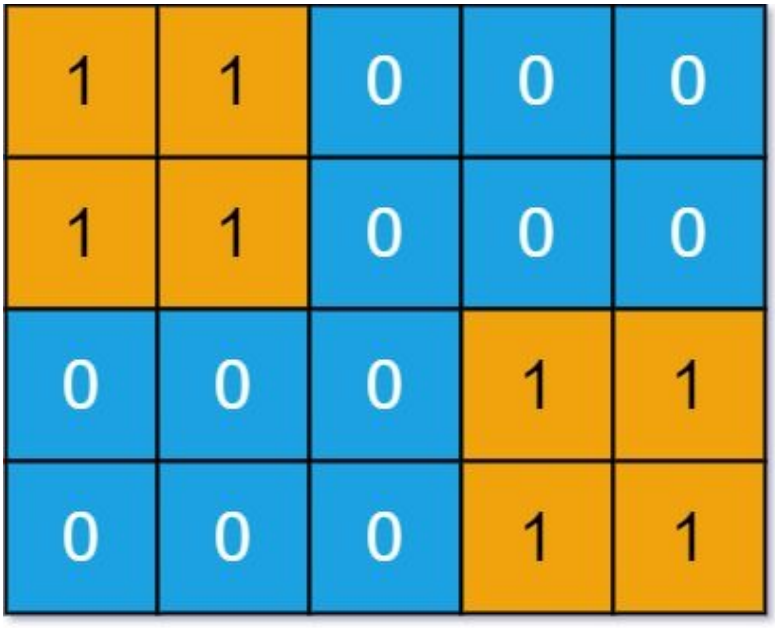
PROBLEM

You are given an `m x n` binary matrix `grid`. An island is a group of `1`'s (representing land) connected **4-directionally** (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

An island is considered to be the same as another if and only if one island can be translated (and not rotated or reflected) to equal the other.

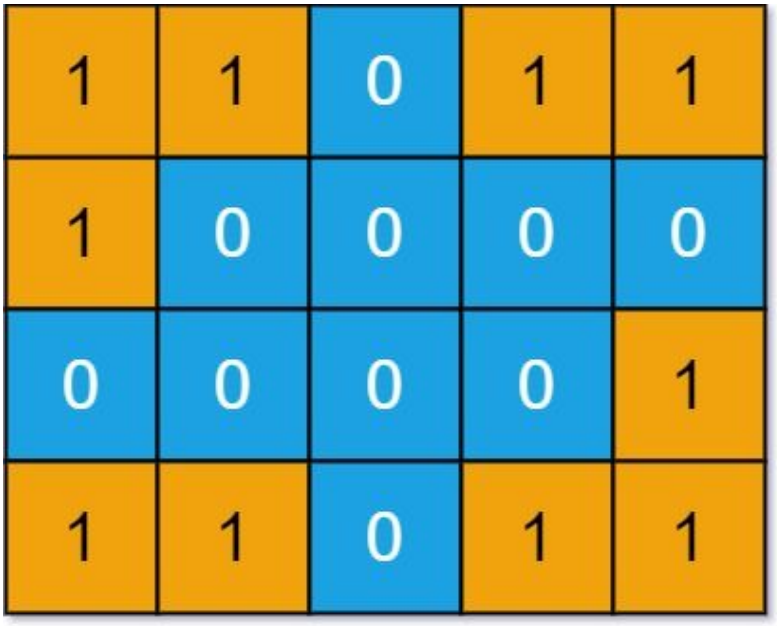
Return *the number of **distinct** islands*.

Example 1:




```
Input: grid = [[1,1,0,0,0],[1,1,0,0,0],[0,0,0,1,1],[0,0,0,1,1]]
Output: 1
```

Example 2:




```
Input: grid = [[1,1,0,1,1],[1,0,0,0,0],[0,0,0,0,1],[1,1,0,1,1]]
Output: 3
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 50`
- `grid[i][j]` is either `0` or `1`.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Islands are equal iff a translation matches them; rotation does not count. Absolute coordinates need a common origin. DFS records both the step direction and the backtrack. Distinct shapes yield distinct strings; a set of those strings is the answer. Mark land as 0 when visited.

Python 3

```
class Solution:
    def numDistinctIslands(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int, k: int):
            grid[i][j] = 0
            path.append(str(k))
            dirs = (-1, 0, 1, 0, -1)
            for h in range(1, 5):
                x, y = i + dirs[h - 1], j + dirs[h]
                if 0 <= x < m and 0 <= y < n and grid[x][y]:
                    dfs(x, y, h)
            path.append(str(-k))

        paths = set()
        path = []
        m, n = len(grid), len(grid[0])
        for i, row in enumerate(grid):
            for j, x in enumerate(row):
                if x:
                    dfs(i, j, 0)
                    paths.add("".join(path))
                    path.clear()
        return len(paths)
```

Complexity. Time: O(m * n). Space: O(m * n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Islands that are the same after ROTATION or
# REFLECTION also count as one (Distinct Islands II)
# 2. Return the distinct shapes themselves


# ============================================================
# FOLLOW-UP 1: Islands that are the same after ROTATION or
# REFLECTION also count as one (Distinct Islands II)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Describe every island by its cells; build all 8
# rotated / mirrored versions, move each to the origin and
# sort it; the smallest version is the island's canonical
# shape.
# Time: O(m * n) (eight transformations per island).
# Space: O(m * n).
def distinct_islands_with_rotation(grid):
    rows, cols = len(grid), len(grid[0])
    seen = set()
    shapes = set()
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1 and (r, c) not in seen:
                cells = []
                stack = [(r, c)]
                seen.add((r, c))
                while stack:
                    x, y = stack.pop()
                    cells.append((x, y))
                    for nx, ny in ((x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1)):
                        if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 1 and (nx, ny) not in
seen:
                            seen.add((nx, ny))
                            stack.append((nx, ny))
                shapes.add(canonical_shape(cells))
    return len(shapes)
```

```
def canonical_shape(cells):
    versions = []
    for transform in range(8):
        moved = []
        for x, y in cells:
            if transform >= 4:
                y = -y
            for turn in range(transform % 4):
                x, y = y, -x
            moved.append((x, y))
        low_x = min(p[0] for p in moved)
        low_y = min(p[1] for p in moved)
        versions.append(tuple(sorted((x - low_x, y - low_y) for x, y in moved)))
    return min(versions)


# ============================================================
# FOLLOW-UP 2: Return the distinct shapes themselves
# ============================================================

# ---- follow-up 2: function ----
```

```
# Answer: Store each island's cells relative to its first
# cell; shapes that are equal sets of offsets are the
# same.
# Time: O(m * n log(m * n)).  Space: O(m * n).
def distinct_shapes(grid):
    rows, cols = len(grid), len(grid[0])
    seen = set()
    shapes = set()
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1 and (r, c) not in seen:
                offsets = []
                stack = [(r, c)]
                seen.add((r, c))
                while stack:
                    x, y = stack.pop()
                    offsets.append((x - r, y - c))
                    for nx, ny in ((x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1)):
                        if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 1 and (nx, ny) not in
seen:
                            seen.add((nx, ny))
                            stack.append((nx, ny))
                shapes.add(tuple(sorted(offsets)))
    return sorted(shapes)
```

1706. Where Will the Ball Fall

Medium · Array, Matrix, Simulation

leetcode.com/problems/where-will-the-ball-fall

Also appears in pattern 22 Dynamic Programming - 2D / Grid

PROBLEM

You have a 2-D `grid` of size `m x n` representing a box, and you have `n` balls. The box is open on the top and bottom sides.

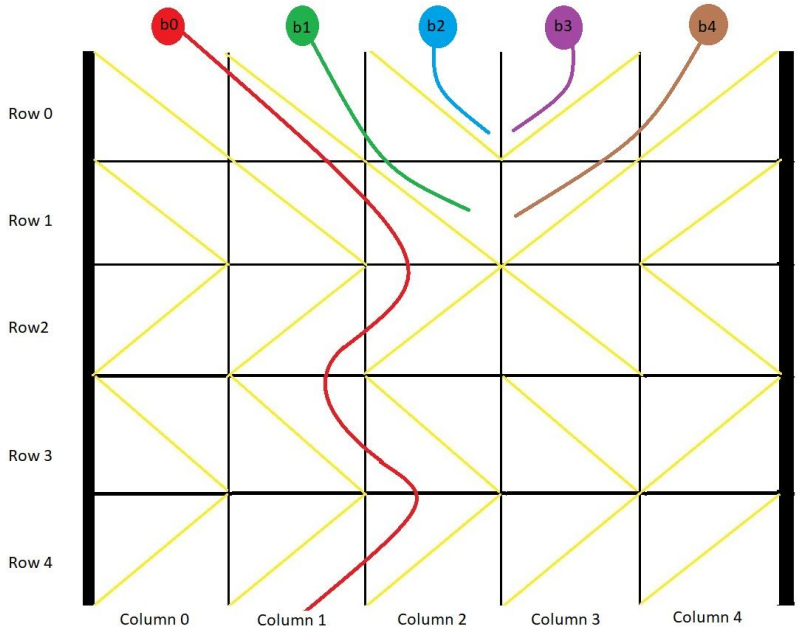
Each cell in the box has a diagonal board spanning two corners of the cell that can redirect a ball to the right or to the left.

- A board that redirects the ball to the right spans the top-left corner to the bottom-right corner and is represented in the grid as `1`.
- A board that redirects the ball to the left spans the top-right corner to the bottom-left corner and is represented in the grid as `-1`.

We drop one ball at the top of each column of the box. Each ball can get stuck in the box or fall out of the bottom. A ball gets stuck if it hits a "V" shaped pattern between two boards or if a board redirects the ball into either wall of the box.

Return *an array* `answer` *of size* `n` *where* `answer[i]` *is the column that the ball falls out of at the bottom after dropping the ball from the* `i`th *column at the top, or* `-1` *if the ball gets stuck in the box.*

Example 1:




```
Input: grid = [[1,1,1,-1,-1],[1,1,1,-1,-1],[-1,-1,-1,1,1],[1,1,1,1,-1],[-1,-1,-1,-1,-1]]
Output: [1,-1,-1,-1,-1]
Explanation: This example is shown in the photo.
Ball b0 is dropped at column 0 and falls out of the box at column 1.
Ball b1 is dropped at column 1 and will get stuck in the box between column 2 and 3 and row 1.
Ball b2 is dropped at column 2 and will get stuck on the box between column 2 and 3 and row 0.
Ball b3 is dropped at column 3 and will get stuck on the box between column 2 and 3 and row 0.
Ball b4 is dropped at column 4 and will get stuck on the box between column 2 and 3 and row 1.
```

Example 2:

```
Input: grid = [[-1]]
Output: [-1]
Explanation: The ball gets stuck against the left wall.
```

Example 3:

```
Input: grid = [[1,1,1,1,1,1],[-1,-1,-1,-1,-1,-1],[1,1,1,1,1,1],[-1,-1,-1,-1,-1,-1]]
Output: [0,1,2,3,4,-1]
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 100`
- `grid[i][j]` is `1` or `-1`.

SOLUTION (PYTHON 3)

Solution 1: Case Analysis + DFS

How to think about it

A ball is dropped from every top cell and boards steer it left or right. The grid is small enough to simulate each ball independently. A ball gets stuck in four cases: it is on the border and the board pushes it out, or two neighbouring boards form a V. Otherwise it slides diagonally to the next row. Let dfs(i,j) be the exit column from (i,j): return -1 if stuck, otherwise recurse to (i+1,j± 1). Reaching row m returns the current column.

We can use DFS to simulate the movement of the ball. Design a function dfs(i, j), which represents the column where the ball will fall when it starts from row i and column j. The ball will get stuck in the following cases:

1. The ball is in the leftmost column, and the cell's diagonal directs the ball to the left.
2. The ball is in the rightmost column, and the cell's diagonal directs the ball to the right.
3. The cell's diagonal directs the ball to the right, and the adjacent cell to the right directs the ball to the left.
4. The cell's diagonal directs the ball to the left, and the adjacent cell to the left directs the ball to the right.

If any of the above conditions are met, we can determine that the ball will get stuck and return -1. Otherwise, we can continue to recursively find the next position of the ball. Finally, if the ball reaches the last row, we can return the current column index.

The time complexity is O(m × n), and the space complexity is O(m). Here, m and n are the number of rows and columns of the grid, respectively.

Python 3

```python
class Solution:
    def findBall(self, grid: List[List[int]]) -> List[int]:
        def dfs(i: int, j: int) -> int:
            if i == m:
                return j
            if j == 0 and grid[i][j] == -1:
                return -1
            if j == n - 1 and grid[i][j] == 1:
                return -1
            if grid[i][j] == 1 and grid[i][j + 1] == -1:
                return -1
            if grid[i][j] == -1 and grid[i][j - 1] == 1:
                return -1
            return dfs(i + 1, j + 1) if grid[i][j] == 1 else dfs(i + 1, j - 1)

        m, n = len(grid), len(grid[0])
        return [dfs(0, j) for j in range(n)]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the whole path of one ball
# 2. How many balls get stuck?


# ============================================================
# FOLLOW-UP 1: Return the whole path of one ball
# ============================================================

# ---- follow-up 1: function ----
# Answer: Follow the ball row by row; its column changes
# by +1 for a board that slants down to the right (value
# 1) and by -1 for one that slants to the left (value -1).
# It is stuck when it meets a V shape or a wall.
# Time: O(m).  Space: O(m).
def ball_path(grid, start_column):
    cols = len(grid[0])
    column = start_column
    path = [column]
    for row in grid:
        direction = row[column]
        new_column = column + direction
        if new_column < 0 or new_column >= cols or row[new_column] != direction:
            return path, -1
        column = new_column
        path.append(column)
    return path, column


# ============================================================
# FOLLOW-UP 2: How many balls get stuck?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Run every ball and count the ones that do not
# reach the bottom.
# Time: O(m * n).  Space: O(1).
def count_stuck_balls(grid):
    stuck = 0
    for start in range(len(grid[0])):
        column = start
        for row in grid:
            direction = row[column]
            new_column = column + direction
            if new_column < 0 or new_column >= len(grid[0]) or row[new_column] != direction:
                stuck += 1
                break
            column = new_column
    return stuck
```

980. Unique Paths III

Hard · Bit Manipulation, Array, Backtracking, Hamiltonian Path, Matrix

leetcode.com/problems/unique-paths-iii

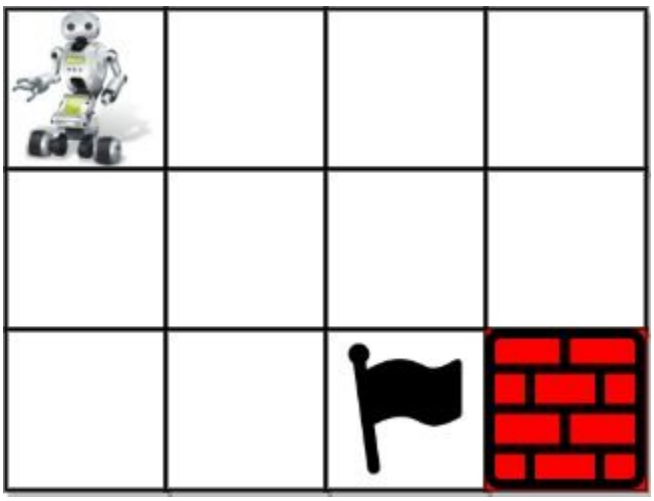
PROBLEM

You are given an `m x n` integer array `grid` where `grid[i][j]` could be:

- `1` representing the starting square. There is exactly one starting square.
- `2` representing the ending square. There is exactly one ending square.
- `0` representing empty squares we can walk over.
- `-1` representing obstacles that we cannot walk over.

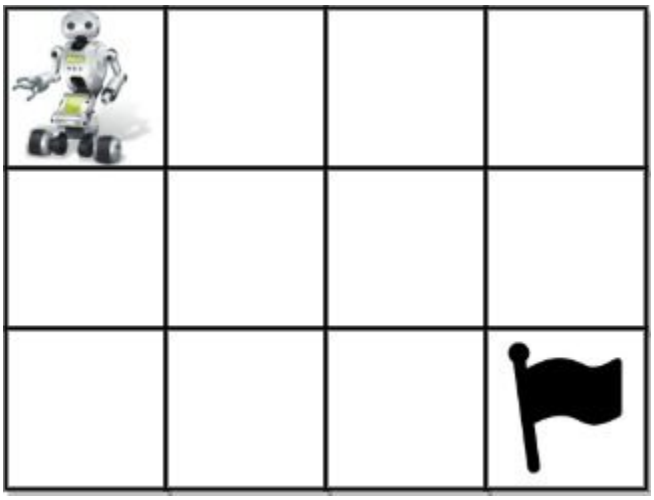
Return *the number of 4-directional walks from the starting square to the ending square, that walk over every non-obstacle square exactly once*.

Example 1:



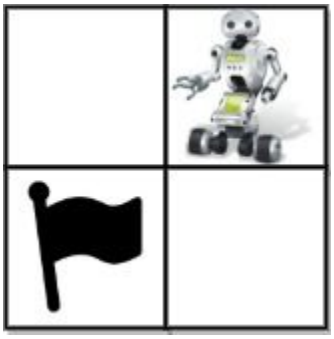
```
Input: grid = [[1,0,0,0],[0,0,0,0],[0,0,2,-1]]
Output: 2
Explanation: We have the following two paths:
1. (0,0),(0,1),(0,2),(0,3),(1,3),(1,2),(1,1),(1,0),(2,0),(2,1),(2,2)
2. (0,0),(1,0),(2,0),(2,1),(1,1),(0,1),(0,2),(0,3),(1,3),(1,2),(2,2)
```

Example 2:



```
Input: grid = [[1,0,0,0],[0,0,0,0],[0,0,0,2]]
Output: 4
Explanation: We have the following four paths:
1. (0,0),(0,1),(0,2),(0,3),(1,3),(1,2),(1,1),(1,0),(2,0),(2,1),(2,2),(2,3)
2. (0,0),(0,1),(1,1),(1,0),(2,0),(2,1),(2,2),(1,2),(0,2),(0,3),(1,3),(2,3)
3. (0,0),(1,0),(2,0),(2,1),(2,2),(1,2),(1,1),(0,1),(0,2),(0,3),(1,3),(2,3)
4. (0,0),(1,0),(2,0),(2,1),(1,1),(0,1),(0,2),(0,3),(1,3),(1,2),(2,2),(2,3)
```

Example 3:



```
Input: grid = [[0,1],[2,0]]
Output: 0
Explanation: There is no path that walks over every empty square exactly once.
Note that the starting and ending square can be anywhere in the grid.
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 20`
- `1 <= m * n <= 20`
- `-1 <= grid[i][j] <= 2`
- There is exactly one starting cell and one ending cell.

SOLUTION (PYTHON 3)

Solution 1: Backtracking

How to think about it

Walk from start to end visiting every empty cell exactly once. At most 20 cells, so backtracking fits. Count empties and find the start, then DFS in four directions with a visited set. A path that lands on the end counts only when the step count equals the number of empties plus one.

We can first traverse the entire grid, find the starting point (x, y), and count the number of blank spaces cnt.

Next, we can start searching from the starting point to get all the path numbers. We design a function dfs(i, j, k) to indicate that the path number is k and the starting point is (i, j).

In the function, we first determine whether the current cell is the end point. If it is, we determine whether k is equal to cnt + 1. If it is, the current path is a valid path, and 1 is returned, otherwise 0 is returned.

If the current cell is not the end point, we enumerate the four adjacent cells of the current cell. If the adjacent cell has not been visited, we mark the adjacent cell as visited, and then continue to search the path number from the adjacent cell. After the search is completed, we mark the adjacent cell as unvisited. After the search is completed, we return the sum of the path numbers of all adjacent cells.

Finally, we return the path number from the starting point, that is, dfs(x, y, 1).

The time complexity is $O(3^{m \times n})$, and the space complexity is $O(m \times n)$. Where m and n are the number of rows and columns of the grid, respectively.

Python 3

```python
class Solution:
    def uniquePathsIII(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int, k: int) -> int:
            if grid[i][j] == 2:
                return int(k == cnt + 1)
            ans = 0
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and (x, y) not in vis and grid[x][y] != -1:
                    vis.add((x, y))
                    ans += dfs(x, y, k + 1)
                    vis.remove((x, y))
            return ans

        m, n = len(grid), len(grid[0])
        start = next((i, j) for i in range(m) for j in range(n) if grid[i][j] == 1)
        dirs = (-1, 0, 1, 0, -1)
        cnt = sum(row.count(0) for row in grid)
        vis = {start}
        return dfs(*start, 0)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return one path that walks over every empty square
# exactly once
# 2. Count the paths from the start to the end that do NOT
# have to cover every square


# ============================================================
# FOLLOW-UP 1: Return one path that walks over every empty
# square exactly once
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same backtracking; stop and return the path
# as soon as a complete walk is found.
# Time: O(3^(m * n)) in the worst case.  Space: O(m * n).
def one_full_path(grid):
    rows, cols = len(grid), len(grid[0])
    empty = 0
    start = None
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                start = (r, c)
            if grid[r][c] != -1:
                empty += 1
    path = [start]
    visited = {start}

    def go(r, c):
        if grid[r][c] == 2:
            return len(path) == empty
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] != -1 and (nr, nc) not in visited:
                visited.add((nr, nc))
                path.append((nr, nc))
                if go(nr, nc):
                    return True
                path.pop()
```

```
                visited.remove((nr, nc))
        return False

    if go(start[0], start[1]):
        return path
    return None


# ============================================================
# FOLLOW-UP 2: Count the paths from the start to the end
# that do NOT have to cover every square
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same search, but add one for every arrival
# at the end square.
# Time: O(3^(m * n)) in the worst case.  Space: O(m * n).
def count_simple_paths(grid):
    rows, cols = len(grid), len(grid[0])
    start = None
    for r in range(rows):
```

```
        for c in range(cols):
            if grid[r][c] == 1:
                start = (r, c)
    visited = {start}

    def go(r, c):
        if grid[r][c] == 2:
            return 1
        total = 0
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] != -1 and (nr, nc) not in visited:
                visited.add((nr, nc))
                total += go(nr, nc)
                visited.remove((nr, nc))
        return total

    return go(start[0], start[1])
```

PATTERN 19 OF 41

Heap / Priority Queue

20 problems · 4 Easy · 14 Medium · 2 Hard

THE IDEA

Get the smallest or largest item in O(log n) while items keep arriving. For the K largest keep a min-heap of size K.

Signals: top K, kth largest, merge sorted lists, task scheduling, meeting rooms, running median, Dijkstra.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
import heapq
heap = []
for x in nums:
    heapq.heappush(heap, x)
    if len(heap) > k:
        heapq.heappop(heap)        # drop the smallest: keeps the K largest
return heap[0]                     # the kth largest
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Last Stone Weight	Easy	LC 1046	1038
2	Kth Largest Element in a Stream	Easy	LC 703	1040
3	The K Weakest Rows in a Matrix	Easy	LC 1337	1043
4	Take Gifts From the Richest Pile	Easy	LC 2558	1046
5	★ **Kth Largest Element in an Array** (see page)	Medium	LC 215	737
6	★ **Top K Frequent Elements** (see page)	Medium	LC 347	34
7	★ **K Closest Points to Origin**	Medium	LC 973	1048
8	★ **Task Scheduler**	Medium	LC 621	1052
9	★ **Reorganize String** (see page)	Medium	LC 767	335
10	Sort Characters By Frequency	Medium	LC 451	1055
11	Minimum Cost to Connect Sticks	Medium	LC 1167	1057
12	★ **Furthest Building You Can Reach**	Medium	LC 1642	1060
13	Single-Threaded CPU	Medium	LC 1834	1063
14	Maximum Subsequence Score	Medium	LC 2542	1066
15	Find the Kth Largest Integer in the Array	Medium	LC 1985	1069
16	Top K Frequent Words	Medium	LC 692	1071
17	Seat Reservation Manager	Medium	LC 1845	1074
18	Total Cost to Hire K Workers	Medium	LC 2462	1077
19	Maximum Performance of a Team	Hard	LC 1383	1080
20	★ **Process Tasks Using Servers**	Hard	LC 1882	1083

PATTERN 19 · HEAP / PRIORITY QUEUE · PROBLEM 1 OF 20

1046. Last Stone Weight

Easy · Array, Heap (Priority Queue)

leetcode.com/problems/last-stone-weight

Also appears in pattern 37 Two Heaps

PROBLEM

You are given an array of integers `stones` where `stones[i]` is the weight of the i^{th} stone.

We are playing a game with the stones. On each turn, we choose the **heaviest two stones** and smash them together. Suppose the heaviest two stones have weights `x` and `y` with `x <= y`. The result of this smash is:

- If `x == y`, both stones are destroyed, and
- If `x != y`, the stone of weight `x` is destroyed, and the stone of weight `y` has new weight `y - x`.

At the end of the game, there is **at most one** stone left.

Return *the weight of the last remaining stone*. If there are no stones left, return `0`.

Example 1:

```
Input: stones = [2,7,4,1,8,1]
Output: 1
Explanation:
We combine 7 and 8 to get 1 so the array converts to [2,4,1,1,1] then,
we combine 2 and 4 to get 2 so the array converts to [2,1,1,1] then,
we combine 2 and 1 to get 1 so the array converts to [1,1,1] then,
we combine 1 and 1 to get 0 so the array converts to [1] then that's the value of the last stone.
```

Example 2:

```
Input: stones = [1]
Output: 1
```

Constraints:

- `1 <= stones.length <= 30`
- `1 <= stones[i] <= 1000`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Each smash uses the two heaviest stones. Scanning for the maximum every time repeats comparisons; a heap extracts them in logarithmic time. Negating the weights turns a min-heap into a max-heap. Pop y≥ x and push y-x when they differ, until fewer than two stones remain. An empty heap is 0; otherwise the answer is the negated top.

Python 3

```python
class Solution:
    def lastStoneWeight(self, stones: List[int]) -> int:
        h = [-x for x in stones]
        heapify(h)
        while len(h) > 1:
            y, x = -heappop(h), -heappop(h)
            if x != y:
                heappush(h, x - y)
        return 0 if not h else -h[0]
```

Complexity. Time: O(n log n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the weight of every smash, in order
# 2. Choose which stones to smash to leave the SMALLEST
# possible remainder (Last Stone Weight II)


# ============================================================
# FOLLOW-UP 1: Return the weight of every smash, in order
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same max-heap (negated weights); record what is
# left after each smash.
# Time: O(n log n).  Space: O(n).
import heapq


def smash_log(stones):
    heap = [-s for s in stones]
    heapq.heapify(heap)
    log = []
    while len(heap) > 1:
        first = -heapq.heappop(heap)
        second = -heapq.heappop(heap)
        left = first - second
        log.append((first, second, left))
        if left > 0:
            heapq.heappush(heap, -left)
    return log


# ============================================================
# FOLLOW-UP 2: Choose which stones to smash to leave the
# SMALLEST possible remainder (Last Stone Weight II)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Smashing amounts to splitting the stones into
# two groups and subtracting their totals, so the best
# remainder is the smallest difference between two groups:
# a subset-sum table up to half of the total.
# Time: O(n * S), S = half of the total.  Space: O(S).
def last_stone_weight_min(stones):
    total = 0
    for s in stones:
        total += s
    half = total // 2
    reachable = [False] * (half + 1)
    reachable[0] = True
    for s in stones:
        for amount in range(half, s - 1, -1):
            if reachable[amount - s]:
                reachable[amount] = True
    best = 0
    for amount in range(half + 1):
        if reachable[amount]:
            best = amount
    return total - 2 * best
```

703. Kth Largest Element in a Stream

Easy · Tree, Design, Binary Search Tree, Binary Tree, Data Stream, Heap (Priority Queue)

leetcode.com/problems/kth-largest-element-in-a-stream

Also appears in pattern 37 Two Heaps

PROBLEM

You are part of a university admissions office and need to keep track of the `kth` highest test score from applicants in real-time. This helps to determine cut-off marks for interviews and admissions dynamically as new applicants submit their scores.

You are tasked to implement a class which, for a given integer `k`, maintains a stream of test scores and continuously returns the `k`th highest test score **after** a new score has been submitted. More specifically, we are looking for the `k`th highest score in the sorted list of all scores.

Implement the `KthLargest` class:

- `KthLargest(int k, int[] nums)` Initializes the object with the integer `k` and the stream of test scores `nums`.
- `int add(int val)` Adds a new test score `val` to the stream and returns the element representing the k^{th} largest element in the pool of test scores so far.

Example 1:

Input:
["KthLargest", "add", "add", "add", "add", "add"]
[[3, [4, 5, 8, 2]], [3], [5], [10], [9], [4]]

Output: [null, 4, 5, 5, 8, 8]

Explanation:

KthLargest kthLargest = new KthLargest(3, [4, 5, 8, 2]);
kthLargest.add(3); // return 4
kthLargest.add(5); // return 5
kthLargest.add(10); // return 5
kthLargest.add(9); // return 8
kthLargest.add(4); // return 8

Example 2:

Input:
["KthLargest", "add", "add", "add", "add"]
[[4, [7, 7, 7, 7, 8, 3]], [2], [10], [9], [9]]

Output: [null, 7, 7, 7, 8]

Explanation:

KthLargest kthLargest = new KthLargest(4, [7, 7, 7, 7, 8, 3]);

kthLargest.add(2); // return 7

kthLargest.add(10); // return 7

kthLargest.add(9); // return 7

kthLargest.add(9); // return 8

Constraints:

- $0 <= nums.length <= 10^4$
- $1 <= k <= nums.length + 1$
- $-10^4 <= nums[i] <= 10^4$

- -10^4 <= val <= 10^4
- At most 10^4 calls will be made to `add`.

SOLUTION (PYTHON 3)

Solution 1: Priority Queue (Min Heap)

> **How to think about it**
>
> We insert into a stream and must report the k-th largest after each add. Sorting or scanning the whole history on every query is too heavy when both n and the query count reach 10^4. Only the current top k values matter; anything smaller can never be the answer. In a size-k min-heap the top is the smallest of those k, i.e. the k-th largest overall. Push each new value and pop if the heap grows past k. Seeding from nums uses the same add. Each update is O(log k).

We maintain a priority queue (min heap) minQ.

Initially, we add the elements of the array nums to minQ one by one, ensuring that the size of minQ does not exceed k. The time complexity is O(n × log k).

Each time a new element is added, if the size of minQ exceeds k, we pop the top element of the heap to ensure that the size of minQ is k. The time complexity is O(log k).

In this way, the elements in minQ are the largest k elements in the array nums, and the top element of the heap is the k^{th} largest element.

The space complexity is O(k).

Python 3

```python
class KthLargest:

    def __init__(self, k: int, nums: List[int]):
        self.k = k
        self.min_q = []
        for x in nums:
            self.add(x)

    def add(self, val: int) -> int:
        heappush(self.min_q, val)
        if len(self.min_q) > self.k:
            heappop(self.min_q)
        return self.min_q[0]


# Your KthLargest object will be instantiated and called as such:
# obj = KthLargest(k, nums)
# param_1 = obj.add(val)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The k-th SMALLEST number of the stream
# 2. The k-th largest among only the LAST w numbers of the
# stream


# ========================================================
# FOLLOW-UP 1: The k-th SMALLEST number of the stream
# ========================================================

# ---- follow-up 1: function ----
# Answer: Keep a max-heap of the k smallest numbers (push
# negatives); the top is the k-th smallest.
# Time: O(log k) per add.  Space: O(k).
import heapq
```

```
class KthSmallest:
    def __init__(self, k):
        self.k = k
        self.heap = []

    def add(self, value):
        heapq.heappush(self.heap, -value)
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)
        if len(self.heap) < self.k:
            return None
        return -self.heap[0]


# ============================================================
# FOLLOW-UP 2: The k-th largest among only the LAST w
# numbers of the stream
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the last w numbers in a sorted list: insert
# each new number at its place (binary search), remove the
# number that leaves the window, and read the k-th from
# the end.
# Time: O(w) per add (inserting into and removing from
# sorted lists).  Space: O(w).
import bisect


class KthLargestInWindow:
    def __init__(self, k, window):
        self.k = k
        self.window = window
        self.recent = []                            # the numbers in arrival order
        self.ordered = []                           # the same numbers, sorted

    def add(self, value):
        self.recent.append(value)
        bisect.insort(self.ordered, value)
        if len(self.recent) > self.window:
            old = self.recent.pop(0)
            self.ordered.pop(bisect.bisect_left(self.ordered, old))
        if len(self.ordered) < self.k:
            return None
        return self.ordered[-self.k]
```

PATTERN 19 · HEAP / PRIORITY QUEUE · PROBLEM 3 OF 20

1337. The K Weakest Rows in a Matrix

Easy · Array, Binary Search, Matrix, Sorting, Heap (Priority Queue)

leetcode.com/problems/the-k-weakest-rows-in-a-matrix

Also appears in pattern 38 K-way Merge

PROBLEM

You are given an `m x n` binary matrix `mat` of `1`'s (representing soldiers) and `0`'s (representing civilians). The soldiers are positioned **in front** of the civilians. That is, all the `1`'s will appear to the **left** of all the `0`'s in each row.

A row `i` is **weaker** than a row `j` if one of the following is true:

- The number of soldiers in row `i` is less than the number of soldiers in row `j`.
- Both rows have the same number of soldiers and `i < j`.

Return *the indices of the* `k` ***weakest*** *rows in the matrix ordered from weakest to strongest*.

Example 1:

```
Input: mat =
[[1,1,0,0,0],
 [1,1,1,1,0],
 [1,0,0,0,0],
 [1,1,0,0,0],
 [1,1,1,1,1]],
k = 3
Output: [2,0,3]
Explanation:
The number of soldiers in each row is:
- Row 0: 2
- Row 1: 4
- Row 2: 1
- Row 3: 2
- Row 4: 5
The rows ordered from weakest to strongest are [2,0,3,1,4].
```

Example 2:

```
Input: mat =
[[1,0,0,0],
 [1,1,1,1],
 [1,0,0,0],
 [1,0,0,0]],
k = 2
Output: [0,2]
Explanation:
The number of soldiers in each row is:
- Row 0: 1
- Row 1: 4
- Row 2: 1
- Row 3: 1
The rows ordered from weakest to strongest are [0,2,3,1].
```

Constraints:

- `m == mat.length`
- `n == mat[i].length`
- `2 <= n, m <= 100`
- `1 <= k <= m`

- `matrix[i][j]` is either 0 or 1.

SOLUTION (PYTHON 3)

Solution 1

> **How to think about it**
>
> Rows are ordered by soldier count, then by index; we keep the first k. Ones sit left of zeros, so the count is the first zero. Bisecting the reversed row for 0 yields that count; sorting row indices by it and slicing k entries is the answer.

Python 3

```python
class Solution:
    def kWeakestRows(self, mat: List[List[int]], k: int) -> List[int]:
        m, n = len(mat), len(mat[0])
        ans = [n - bisect_right(row[::-1], 0) for row in mat]
        idx = list(range(m))
        idx.sort(key=lambda i: ans[i])
        return idx[:k]
```

Complexity. Time: O(m * log n + m * log m). Space: O(m).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Each row is sorted (soldiers first): count the
# soldiers with a binary search
# 2. The k STRONGEST rows


# ============================================================
# FOLLOW-UP 1: Each row is sorted (soldiers first): count
# the soldiers with a binary search
# ============================================================

# ---- follow-up 1: function ----
# Answer: The number of 1s in a row that starts with 1s is
# the position of the first 0, found by binary search: O(m
# log n) instead of O(m n).
# Time: O(m log n + m log m).  Space: O(m).
def count_soldiers(row):
    low, high = 0, len(row)
    while low < high:
        mid = (low + high) // 2
        if row[mid] == 1:
            low = mid + 1
        else:
            high = mid
    return low


def k_weakest_rows_fast(mat, k):
    pairs = []
    for i in range(len(mat)):
        pairs.append((count_soldiers(mat[i]), i))
    pairs.sort()
    result = []
    for j in range(k):
        result.append(pairs[j][1])
    return result


# ============================================================
# FOLLOW-UP 2: The k STRONGEST rows
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Sort by (soldiers descending, index ascending)
# and take the first k.
# Time: O(m * n + m log m).  Space: O(m).
def k_strongest_rows(mat, k):
    pairs = []
    for i in range(len(mat)):
        soldiers = 0
        for value in mat[i]:
            soldiers += value
        pairs.append((-soldiers, i))
    pairs.sort()
    result = []
    for j in range(k):
        result.append(pairs[j][1])
    return result
```

PATTERN 19 · HEAP / PRIORITY QUEUE · PROBLEM 4 OF 20

2558. Take Gifts From the Richest Pile

Easy · Array, Simulation, Heap (Priority Queue)

leetcode.com/problems/take-gifts-from-the-richest-pile

Also appears in pattern 37 Two Heaps

PROBLEM

You are given an integer array `gifts` denoting the number of gifts in various piles. Every second, you do the following:

- Choose the pile with the maximum number of gifts.
- If there is more than one pile with the maximum number of gifts, choose any.
- Reduce the number of gifts in the pile to the floor of the square root of the original number of gifts in the pile.

Return *the number of gifts remaining after* `k` *seconds.*

Example 1:

```
Input: gifts = [25,64,9,4,100], k = 4
Output: 29
Explanation:
The gifts are taken in the following way:
- In the first second, the last pile is chosen and 10 gifts are left behind.
- Then the second pile is chosen and 8 gifts are left behind.
- After that the first pile is chosen and 5 gifts are left behind.
- Finally, the last pile is chosen again and 3 gifts are left behind.
The final remaining gifts are [5,8,9,4,3], so the total number of gifts remaining is 29.
```

Example 2:

```
Input: gifts = [1,1,1,1], k = 4
Output: 4
Explanation:
In this case, regardless which pile you choose, you have to leave behind 1 gift in each pile.
That is, you can't take any pile with you.
So, the total gifts remaining are 4.
```

Constraints:

- $1 <= gifts.length <= 10^3$
- $1 <= gifts[i] <= 10^9$
- $1 <= k <= 10^3$

SOLUTION (PYTHON 3)

Solution 1: Priority Queue (Max Heap)

> **How to think about it**
>
> Each step replaces the richest pile with floor(√(·)), k times, then sums what remains. Scanning for the max each time works for the limits, but a heap yields it directly. A max-heap pops the top and pushes its integer square root, k times, then sums the heap. Python stores negations.

We can store the array gifts in a max heap, and then loop k times, each time taking out the top element of the heap, taking the square root of it, and putting the result back into the heap.

Finally, we add up all the elements in the heap as the answer.

The time complexity is O(n + k × log n), and the space complexity is O(n). Here, n is the length of the array gifts.

Python 3

```
class Solution:
    def pickGifts(self, gifts: List[int], k: int) -> int:
        h = [-v for v in gifts]
        heapify(h)
        for _ in range(k):
            heapreplace(h, -int(sqrt(-h[0])))
        return -sum(h)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Which pile is taken at each second?
# 2. Leave the CEILING of the square root instead of the
# floor


# ============================================================
# FOLLOW-UP 1: Which pile is taken at each second?
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep (negative count, pile index) in the heap
# and record the index of the pile popped at each second.
# Time: O((n + k) log n).  Space: O(n).
import heapq


def piles_taken(gifts, k):
    heap = []
    for i in range(len(gifts)):
        heapq.heappush(heap, (-gifts[i], i))
    order = []
    for second in range(k):
        negative, i = heapq.heappop(heap)
        order.append(i)
        remaining = int((-negative) ** 0.5)
        heapq.heappush(heap, (-remaining, i))
    return order


# ============================================================
# FOLLOW-UP 2: Leave the CEILING of the square root
# instead of the floor
# ============================================================

# ---- follow-up 2: function ----
# Answer: Only the number written back into the pile
# changes.
# Time: O((n + k) log n).  Space: O(n).
import heapq


def gifts_left_ceil(gifts, k):
    heap = [-g for g in gifts]
    heapq.heapify(heap)
    for second in range(k):
        biggest = -heapq.heappop(heap)
        root = int(biggest ** 0.5)
        while root * root < biggest:
            root += 1
        heapq.heappush(heap, -root)
    total = 0
    for value in heap:
        total += -value
    return total
```

973. K Closest Points to Origin

Medium · Geometry, Array, Math, Divide and Conquer, Quickselect, Sorting, Heap (Priority Queue), K-D Tree

leetcode.com/problems/k-closest-points-to-origin

★ **TOP 7 PROBLEM** in Heap / Priority Queue: master this one first

Also appears in pattern 38 K-way Merge

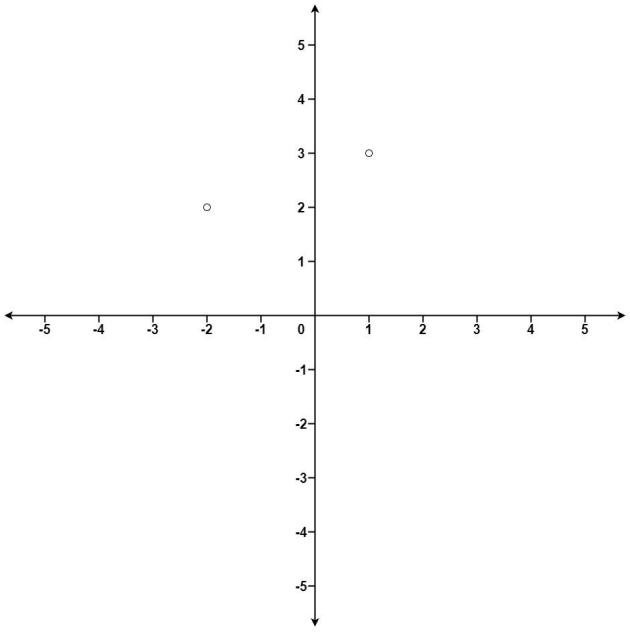
PROBLEM

Given an array of `points` where `points[i] = [x_i, y_i]` represents a point on the **X-Y** plane and an integer `k`, return the `k` closest points to the origin `(0, 0)`.

The distance between two points on the **X-Y** plane is the Euclidean distance (i.e., $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$).

You may return the answer in **any order**. The answer is **guaranteed** to be **unique** (except for the order that it is in).

Example 1:




```
Input: points = [[1,3],[-2,2]], k = 1
Output: [[-2,2]]
Explanation:
The distance between (1, 3) and the origin is sqrt(10).
The distance between (-2, 2) and the origin is sqrt(8).
Since sqrt(8) < sqrt(10), (-2, 2) is closer to the origin.
We only want the closest k = 1 points from the origin, so the answer is just [[-2,2]].
```

Example 2:

```
Input: points = [[3,3],[5,-1],[-2,4]], k = 2
Output: [[3,3],[-2,4]]
Explanation: The answer [[-2,4],[3,3]] would also be accepted.
```

Constraints:

- $1 <= k <= points.length <= 10^4$
- $-10^4 <= x_i, y_i <= 10^4$

SOLUTIONS (PYTHON 3)

Solution 1: Custom Sorting

> **How to think about it**
>
> Find the k points closest to the origin. Sorting by Euclidean distance and taking the first k is O(nlog n), which is fine for n≤ 10^4.

We sort all points by their distance from the origin in ascending order, and then take the first k points.

The time complexity is O(n log n), and the space complexity is O(log n). Here, n is the length of the array points.

Python 3

```
class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        points.sort(key=lambda p: hypot(p[0], p[1]))
        return points[:k]
```

Solution 2: Priority Queue (Max Heap)

> **How to think about it**
>
> A full sort also orders the farthest n-k points. A max-heap of size k keeps only the current closest k, in O(nlog k) time.

We can use a priority queue (max heap) to maintain the k closest points to the origin.

The time complexity is O(n × log k), and the space complexity is O(k). Here, n is the length of the array points.

Python 3

```
class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        max_q = []
        for i, (x, y) in enumerate(points):
            dist = math.hypot(x, y)
            heappush(max_q, (-dist, i))
            if len(max_q) > k:
                heappop(max_q)
        return [points[i] for _, i in max_q]
```

Solution 3: Binary Search

> **How to think about it**
>
> The heap still pays a logarithm. The number of points is monotone in the distance, so binary-search a threshold until at least k points lie inside, then collect every point whose distance is at most that critical value.

We notice that as the distance increases, the number of points increases as well. There exists a critical value such that the number of points before this value is less than or equal to k, and the number of points after this value is greater than k.

Therefore, we can use binary search to enumerate the distance. In each binary search iteration, we count the number of points whose distance is less than or equal to the current distance. If the count is greater than or equal to k, it indicates that the critical value is on the left side, so we set the right boundary equal to the current distance; otherwise, the critical value is on the right side, so we set the left boundary equal to the current distance plus one.

After the binary search is finished, we just need to return the points whose distance is less than or equal to the left boundary.

The time complexity is O(n × log M), and the space complexity is O(n). Here, n is the length of the array points, and M is the maximum value of the distance.

Python 3

```
class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        dist = [x * x + y * y for x, y in points]
        l, r = 0, max(dist)
        while l < r:
            mid = (l + r) >> 1
            cnt = sum(d <= mid for d in dist)
            if cnt >= k:
                r = mid
            else:
                l = mid + 1
        return [points[i] for i, d in enumerate(dist) if d <= l]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Quickselect: average O(n)
# 2. Points arrive in a stream: always report the k
# closest so far


# ============================================================
# FOLLOW-UP 1: Quickselect: average O(n)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Partition the points by distance around a random
# pivot and continue only on the side that contains the
# k-th position.
# Time: O(n) on average, O(n^2) in the worst case.  Space:
# O(n).
import random


def k_closest_quickselect(points, k):
    items = list(points)

    def distance(p):
        return p[0] * p[0] + p[1] * p[1]

    low, high = 0, len(items) - 1
    while low < high:
        pivot = distance(items[random.randint(low, high)])
        smaller, equal, bigger = [], [], []
        for p in items[low:high + 1]:
            d = distance(p)
            if d < pivot:
                smaller.append(p)
            elif d == pivot:
                equal.append(p)
            else:
                bigger.append(p)
        items[low:high + 1] = smaller + equal + bigger
        if k <= low + len(smaller):
            high = low + len(smaller) - 1
        elif k <= low + len(smaller) + len(equal):
            break
        else:
            low = low + len(smaller) + len(equal)
    return items[:k]


# ============================================================
# FOLLOW-UP 2: Points arrive in a stream: always report
# the k closest so far
```

```
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep a max-heap of size k by distance (push the
# negative distance); a new point replaces the farthest
# one when it is closer.
# Time: O(log k) per point.  Space: O(k).
import heapq
```

```
class ClosestPoints:
    def __init__(self, k):
        self.k = k
        self.heap = []

    def add(self, point):
        distance = point[0] * point[0] + point[1] * point[1]
        heapq.heappush(self.heap, (-distance, point))
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)

    def closest(self):
        result = []
        for negative, point in self.heap:
            result.append(point)
        return result
```

PATTERN 19 · HEAP / PRIORITY QUEUE · PROBLEM 8 OF 20

621. Task Scheduler

Medium · Greedy, Array, Hash Table, Counting, Sorting, Heap (Priority Queue)

leetcode.com/problems/task-scheduler

★ **TOP 7 PROBLEM** in Heap / Priority Queue: master this one first

Also appears in pattern 21 Greedy, 37 Two Heaps

PROBLEM

You are given an array of CPU `tasks`, each labeled with a letter from A to Z, and a number `n`. Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of **at least** `n` intervals between two tasks with the same label.

Return the **minimum** number of CPU intervals required to complete all tasks.

Example 1:

Input: tasks = ["A","A","A","B","B","B"], n = 2

Output: 8

Explanation: A possible sequence is: A -> B -> idle -> A -> B -> idle -> A -> B.

After completing task A, you must wait two intervals before doing A again. The same applies to task B. In the 3rd interval, neither A nor B can be done, so you idle. By the 4th interval, you can do A again as 2 intervals have passed.

Example 2:

Input: tasks = ["A","C","A","B","D","B"], n = 1

Output: 6

Explanation: A possible sequence is: A -> B -> C -> D -> A -> B.

With a cooling interval of 1, you can repeat a task after just one other task.

Example 3:

Input: tasks = ["A","A","A", "B","B","B"], n = 3

Output: 10

Explanation: A possible sequence is: A -> B -> idle -> idle -> A -> B -> idle -> idle -> A -> B.

There are only two types of tasks, A and B, which need to be separated by 3 intervals. This leads to idling twice between repetitions of these tasks.

Constraints:

- `1 <= tasks.length <= 10^4`
- `tasks[i]` is an uppercase English letter.
- `0 <= n <= 100`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Identical tasks must be at least n apart. Simulating the timeline is unnecessary once the bottleneck task is known. The most frequent count x builds (x-1) full gaps plus s tasks of that count at the end. If the total number of tasks is larger, idle slots fill, so the answer is max(m, (x-1)(n+1)+s).

Python 3

```
class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        cnt = Counter(tasks)
        x = max(cnt.values())
        s = sum(v == x for v in cnt.values())
        return max(len(tasks), (x - 1) * (n + 1) + s)
```

Complexity. Time: O(n). Space: O(C), C = 26 task types.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. O(n) formula without a heap
# 2. Return the actual schedule (with 'idle' for waiting)


# ============================================================
# FOLLOW-UP 1: O(n) formula without a heap
# ============================================================

# ---- follow-up 1: function ----
# Answer: The most frequent task forces the schedule:
# (max_count - 1) full cycles of n + 1 slots, plus one
# slot for every task that has the maximum count. Never
# less than the number of tasks.
# Time: O(n).  Space: O(C), C = 26 task types.
def least_interval_formula(tasks, n):
    count = {}
    for t in tasks:
        count[t] = count.get(t, 0) + 1
    most = 0
    for value in count.values():
        if value > most:
            most = value
    with_most = 0
    for value in count.values():
        if value == most:
            with_most += 1
    slots = (most - 1) * (n + 1) + with_most
    if slots < len(tasks):
        return len(tasks)
    return slots


# ============================================================
# FOLLOW-UP 2: Return the actual schedule (with 'idle' for
# waiting)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Simulate time: each second run the available
# task with the most copies left; a task becomes available
# again n seconds after it ran.
# Time: O(T log C), T = tasks, C = distinct tasks.  Space:
# O(T).
import heapq
from collections import deque


def task_schedule(tasks, n):
    count = {}
    for t in tasks:
        count[t] = count.get(t, 0) + 1
    heap = []
    for t in count:
        heapq.heappush(heap, (-count[t], t))
```

```
    waiting = deque()                                # (time it becomes available, left, task)
    schedule = []
    time = 0
    while heap or waiting:
        time += 1
```

```
        if heap:
            negative, task = heapq.heappop(heap)
            schedule.append(task)
            if negative + 1 < 0:
                waiting.append((time + n, negative + 1, task))
        else:
            schedule.append("idle")
        if waiting and waiting[0][0] <= time:
            ready_time, left, task = waiting.popleft()
            heapq.heappush(heap, (left, task))
    return schedule
```

PATTERN 19 · HEAP / PRIORITY QUEUE · PROBLEM 10 OF 20

451. Sort Characters By Frequency

Medium · Hash Table, String, Bucket Sort, Counting, Sorting, Heap (Priority Queue)

leetcode.com/problems/sort-characters-by-frequency

PROBLEM

Given a string `s`, sort it in **decreasing order** based on the **frequency** of the characters. The **frequency** of a character is the number of times it appears in the string.

Return *the sorted string*. If there are multiple answers, return *any of them*.

Example 1:

```
Input: s = "tree"
Output: "eert"
Explanation: 'e' appears twice while 'r' and 't' both appear once.
So 'e' must appear before both 'r' and 't'. Therefore "eetr" is also a valid answer.
```

Example 2:

```
Input: s = "cccaaa"
Output: "aaaccc"
Explanation: Both 'c' and 'a' appear three times, so both "cccaaa" and "aaaccc" are valid answers.
Note that "cacaca" is incorrect, as the same characters must be together.
```

Example 3:

```
Input: s = "Aabb"
Output: "bbAa"
Explanation: "bbaA" is also a valid answer, but "Aabb" is incorrect.
Note that 'A' and 'a' are treated as two different characters.
```

Constraints:

- `1 <= s.length <= 5 * 10^5`
- `s` consists of uppercase and lowercase English letters and digits.

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Sorting

> **How to think about it**
>
> Characters must be rewritten in decreasing frequency; ties may break arbitrarily. A bucket per count needs a bound on the count; the alphabet is small, so sorting the key–value pairs is enough. Count with a hash map, sort by frequency descending, and repeat each character v times. We sort distinct characters, not the whole string, so the log factor follows the alphabet size.

We use a hash table cnt to count the occurrences of each character in the string s. Then, we sort the key-value pairs in cnt in descending order by the number of occurrences. Finally, we concatenate the characters according to the sorted order.

The time complexity is O(n + k × log k), and the space complexity is O(n + k), where n is the length of the string s, and k is the number of distinct characters.

Python 3

```python
class Solution:
    def frequencySort(self, s: str) -> str:
        cnt = Counter(s)
        return ''.join(c * v for c, v in sorted(cnt.items(), key=lambda x: -x[1]))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Sort the WORDS of a sentence by how often they occur
# 2. O(n) bucket sort by frequency


# ============================================================
# FOLLOW-UP 1: Sort the WORDS of a sentence by how often
# they occur
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count each word and sort by (count descending,
# word); write each word as often as it occurs.
# Time: O(w log w), w = distinct words.  Space: O(w).
def sort_words_by_frequency(sentence):
    count = {}
    for word in sentence.split():
        count[word] = count.get(word, 0) + 1
    pairs = []
    for word in count:
        pairs.append((-count[word], word))
    pairs.sort()
    result = []
    for negative, word in pairs:
        for i in range(-negative):
            result.append(word)
    return " ".join(result)


# ============================================================
# FOLLOW-UP 2: O(n) bucket sort by frequency
# ============================================================

# ---- follow-up 2: function ----
# Answer: A character appears at most n times, so put each
# character in the bucket of its count and read the
# buckets from the highest down.
# Time: O(n).  Space: O(n).
def frequency_sort_buckets(s):
    count = {}
    for ch in s:
        count[ch] = count.get(ch, 0) + 1
    buckets = []
    for i in range(len(s) + 1):
        buckets.append([])
    for ch in count:
        buckets[count[ch]].append(ch)
    result = []
    for frequency in range(len(s), 0, -1):
        for ch in buckets[frequency]:
            result.append(ch * frequency)
    return "".join(result)
```

PATTERN 19 · HEAP / PRIORITY QUEUE · PROBLEM 11 OF 20

1167. Minimum Cost to Connect Sticks

Medium · Greedy, Array, Heap (Priority Queue)

leetcode.com/problems/minimum-cost-to-connect-sticks

Also appears in pattern 38 K-way Merge

PROBLEM

You have some number of sticks with positive integer lengths. These lengths are given as an array `sticks`, where `sticks[i]` is the length of the i^{th} stick.

You can connect any two sticks of lengths `x` and `y` into one stick by paying a cost of `x + y`. You must connect all the sticks until there is only one stick remaining.

Return *the minimum cost of connecting all the given sticks into one stick in this way*.

Example 1:

```
Input: sticks = [2,4,3]
Output: 14
Explanation: You start with sticks = [2,4,3].
1. Combine sticks 2 and 3 for a cost of 2 + 3 = 5. Now you have sticks = [5,4].
2. Combine sticks 5 and 4 for a cost of 5 + 4 = 9. Now you have sticks = [9].
There is only one stick left, so you are done. The total cost is 5 + 9 = 14.
```

Example 2:

```
Input: sticks = [1,8,3,5]
Output: 30
Explanation: You start with sticks = [1,8,3,5].
1. Combine sticks 1 and 3 for a cost of 1 + 3 = 4. Now you have sticks = [4,8,5].
2. Combine sticks 4 and 5 for a cost of 4 + 5 = 9. Now you have sticks = [9,8].
3. Combine sticks 9 and 8 for a cost of 9 + 8 = 17. Now you have sticks = [17].
There is only one stick left, so you are done. The total cost is 4 + 9 + 17 = 30.
```

Example 3:

```
Input: sticks = [5]
Output: 0
Explanation: There is only one stick, so you don't need to do anything. The total cost is 0.
```

Constraints:

- `1 <= sticks.length <= 10^4`
- `1 <= sticks[i] <= 10^4`

SOLUTION (PYTHON 3)

Solution 1: Greedy + Priority Queue (Min Heap)

> **How to think about it**
>
> The cost of joining two sticks is their sum, and the new stick is merged again, so longer sticks are charged repeatedly. Always joining the two shortest lengths limits how often large values reappear. A min-heap pops two sticks, pushes their sum, and accumulates the cost until one stick remains.

We can use a greedy approach, each time choosing the shortest two sticks to connect, which ensures the minimum cost of connection.

Therefore, we can use a priority queue (min heap) to maintain the current stick lengths. Each time, we take out two sticks from the priority queue to connect, then put the connected stick back into the priority queue, until there is only one stick left

in the priority queue.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the length of the array `sticks`.

Python 3

```
class Solution:
    def connectSticks(self, sticks: List[int]) -> int:
        heapify(sticks)
        ans = 0
        while len(sticks) > 1:
            z = heappop(sticks) + heappop(sticks)
            ans += z
            heappush(sticks, z)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the order of the connections (the cost of
# every step)
# 2. Join k sticks at a time (when possible)


# ============================================================
# FOLLOW-UP 1: Return the order of the connections (the
# cost of every step)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same greedy: always join the two shortest
# sticks; record each connection's cost.
# Time: O(n log n).  Space: O(n).
import heapq


def connection_costs(sticks):
    heap = list(sticks)
    heapq.heapify(heap)
    costs = []
    while len(heap) > 1:
        a = heapq.heappop(heap)
        b = heapq.heappop(heap)
        costs.append(a + b)
        heapq.heappush(heap, a + b)
    return costs


# ============================================================
# FOLLOW-UP 2: Join k sticks at a time (when possible)
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same greedy with the k shortest sticks each
# step; first add zero-length sticks so that (count - 1)
# is a multiple of (k - 1).
# Time: O(n log n).  Space: O(n).
import heapq


def min_cost_k_way(sticks, k):
    heap = list(sticks)
    while (len(heap) - 1) % (k - 1) != 0:
        heap.append(0)
    heapq.heapify(heap)
    total = 0
    while len(heap) > 1:
        group = 0
```

```
        for i in range(k):
            group += heapq.heappop(heap)
        total += group
        heapq.heappush(heap, group)
    return total
```

1642. Furthest Building You Can Reach

Medium · Greedy, Array, Heap (Priority Queue)

leetcode.com/problems/furthest-building-you-can-reach

★ **TOP 7 PROBLEM** in Heap / Priority Queue, Two Heaps: master this one first

Also appears in pattern 37 Two Heaps

PROBLEM

You are given an integer array `heights` representing the heights of buildings, some `bricks`, and some `ladders`.

You start your journey from building `0` and move to the next building by possibly using bricks or ladders.

While moving from building `i` to building `i+1` (**0-indexed**),

- If the current building's height is **greater than or equal** to the next building's height, you do **not** need a ladder or bricks.
- If the current building's height is **less than** the next building's height, you can either use **one ladder** or `(h[i+1] - h[i])` **bricks**.

Return the furthest building index (0-indexed) you can reach if you use the given ladders and bricks optimally.

Example 1:

```
Input: heights = [4,2,7,6,9,14,12], bricks = 5, ladders = 1
Output: 4
Explanation: Starting at building 0, you can follow these steps:
- Go to building 1 without using ladders nor bricks since 4 >= 2.
- Go to building 2 using 5 bricks. You must use either bricks or ladders because 2 < 7.
- Go to building 3 without using ladders nor bricks since 7 >= 6.
- Go to building 4 using your only ladder. You must use either bricks or ladders because 6 < 9.
It is impossible to go beyond building 4 because you do not have any more bricks or ladders.
```

Example 2:

```
Input: heights = [4,12,2,7,3,18,20,3,19], bricks = 10, ladders = 2
Output: 7
```

Example 3:

```
Input: heights = [14,3,19,3], bricks = 17, ladders = 0
Output: 3
```

Constraints:

- 1 <= heights.length <= 10^5
- 1 <= heights[i] <= 10^6
- 0 <= bricks <= 10^9
- 0 <= ladders <= heights.length

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Only upward climbs cost bricks or ladders, and ladders should cover the largest climbs. The input can be large, so the choice must be online. A min-heap stores climbs currently covered by ladders. When the heap exceeds the ladder count, pay the smallest climb with bricks. If bricks run out, stop. If bricks last the whole way, we reach the last building.

Python 3

```python
class Solution:
    def furthestBuilding(self, heights: List[int], bricks: int, ladders: int) -> int:
        h = []
        for i, a in enumerate(heights[:-1]):
            b = heights[i + 1]
            d = b - a
            if d > 0:
                heappush(h, d)
                if len(h) > ladders:
                    bricks -= heappop(h)
                    if bricks < 0:
                        return i
        return len(heights) - 1
```

Complexity. Time: O(n log L), L = number of ladders. Space: O(L).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Binary search on the answer instead of a heap
# 2. Return where the ladders were used


# ============================================================
# FOLLOW-UP 1: Binary search on the answer instead of a
# heap
# ============================================================

# ---- follow-up 1: function ----
# Answer: To check whether building i is reachable, take
# the climbs up to i, give the ladders to the biggest
# climbs, and pay the rest with bricks. Reachability is
# monotone, so binary search on i.
# Time: O(n log n log n)  (a sort for every binary search
# step).  Space: O(n).
def furthest_building_binary_search(heights, bricks, ladders):
    def reachable(target):
        climbs = []
        for i in range(target):
            if heights[i + 1] > heights[i]:
                climbs.append(heights[i + 1] - heights[i])
        climbs.sort()
```

```
        needed = len(climbs) - ladders
        total = 0
        for i in range(needed if needed > 0 else 0):
            total += climbs[i]
        return total <= bricks

    low, high = 0, len(heights) - 1
    while low < high:
        mid = (low + high + 1) // 2
        if reachable(mid):
            low = mid
        else:
            high = mid - 1
    return low


# ============================================================
# FOLLOW-UP 2: Return where the ladders were used
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same heap strategy, but also keep the index
# of each climb; the ladders end up on the biggest climbs.
# Time: O(n log L), L = ladders.  Space: O(L).
import heapq


def ladder_positions(heights, bricks, ladders):
    heap = []                                       # (climb, position) of the climbs covered by ladders
    reached = 0
    for i in range(len(heights) - 1):
        climb = heights[i + 1] - heights[i]
        if climb <= 0:
            reached = i + 1
            continue
        heapq.heappush(heap, (climb, i))
```

```
        if len(heap) > ladders:
            smallest, position = heapq.heappop(heap)
            bricks -= smallest
        if bricks < 0:
            return reached, sorted(p for c, p in heap)
        reached = i + 1
    return reached, sorted(p for c, p in heap)
```

PATTERN 19 · HEAP / PRIORITY QUEUE · PROBLEM 13 OF 20

1834. Single-Threaded CPU

Medium · Array, Sorting, Heap (Priority Queue)

leetcode.com/problems/single-threaded-cpu

★ **TOP 7 PROBLEM** in Two Heaps: master this one first

Also appears in pattern 37 Two Heaps

PROBLEM

You are given `n` tasks labeled from `0` to `n - 1` represented by a 2D integer array `tasks`, where `tasks[i] = [enqueueTime`$_i$`, processingTime`$_i$`]` means that the `i`□□□□□□th task will be available to process at `enqueueTime`$_i$ and will take `processingTime`$_i$ to finish processing.

You have a single-threaded CPU that can process **at most one** task at a time and will act in the following way:

- If the CPU is idle and there are no available tasks to process, the CPU remains idle.
- If the CPU is idle and there are available tasks, the CPU will choose the one with the **shortest processing time**. If multiple tasks have the same shortest processing time, it will choose the task with the smallest index.
- Once a task is started, the CPU will **process the entire task** without stopping.
- The CPU can finish a task then start a new one instantly.

Return the order in which the CPU will process the tasks.

Example 1:

```
Input: tasks = [[1,2],[2,4],[3,2],[4,1]]
Output: [0,2,3,1]
Explanation: The events go as follows:
- At time = 1, task 0 is available to process. Available tasks = {0}.
- Also at time = 1, the idle CPU starts processing task 0. Available tasks = {}.
- At time = 2, task 1 is available to process. Available tasks = {1}.
- At time = 3, task 2 is available to process. Available tasks = {1, 2}.
- Also at time = 3, the CPU finishes task 0 and starts processing task 2 as it is the shortest.
Available tasks = {1}.
- At time = 4, task 3 is available to process. Available tasks = {1, 3}.
- At time = 5, the CPU finishes task 2 and starts processing task 3 as it is the shortest. Available
tasks = {1}.
- At time = 6, the CPU finishes task 3 and starts processing task 1. Available tasks = {}.
- At time = 10, the CPU finishes task 1 and becomes idle.
```

Example 2:

```
Input: tasks = [[7,10],[7,12],[7,5],[7,4],[7,2]]
Output: [4,3,2,0,1]
Explanation: The events go as follows:
- At time = 7, all the tasks become available. Available tasks = {0,1,2,3,4}.
- Also at time = 7, the idle CPU starts processing task 4. Available tasks = {0,1,2,3}.
- At time = 9, the CPU finishes task 4 and starts processing task 3. Available tasks = {0,1,2}.
- At time = 13, the CPU finishes task 3 and starts processing task 2. Available tasks = {0,1}.
- At time = 18, the CPU finishes task 2 and starts processing task 0. Available tasks = {1}.
- At time = 28, the CPU finishes task 0 and starts processing task 1. Available tasks = {}.
- At time = 40, the CPU finishes task 1 and becomes idle.
```

Constraints:

- `1 <= tasks.length <= 10`^{5}
- `tasks[i] = [enqueueTime`$_i$`, processingTime`$_i$`]`
- `1 <= enqueueTime`$_i$`, processingTime`$_i$ `<= 10`^{9}

SOLUTION (PYTHON 3)

Solution 1: Sorting + Priority Queue (Min Heap)

How to think about it

An idle CPU picks the arrived task with the shortest processing time (then smallest index). Scanning every task each time is $O(n^2)$. With $n \le 10^5$ this will not pass. Sort by enqueue time and keep arrived tasks in a min-heap of (processingTime,index). Advance time to the next arrival or to the finish of the heap top, then push newly available tasks. The heap implements the scheduling rule directly.

First, we sort the tasks by `enqueueTime` in ascending order. Next, we use a priority queue (min heap) to maintain the currently executable tasks. The elements in the queue are `(processingTime, index)`, which represent the execution time and the index of the task. We also use a variable t to represent the current time, initially set to 0.

Next, we simulate the execution process of the tasks.

If the current queue is empty, it means there are no executable tasks at the moment. We update t to the larger value between the `enqueueTime` of the next task and the current time t. Then, we add all tasks with `enqueueTime` less than or equal to t to the queue.

Then, we take out a task from the queue, add its index to the answer array, and update t to the sum of the current time t and the execution time of the current task.

We repeat the above process until the queue is empty and all tasks have been added to the queue.

The time complexity is $O(n \times \log n)$, where n is the number of tasks.

Python 3

```python
class Solution:
    def getOrder(self, tasks: List[List[int]]) -> List[int]:
        for i, task in enumerate(tasks):
            task.append(i)
        tasks.sort()
        ans = []
        q = []
        n = len(tasks)
        i = t = 0
        while q or i < n:
            if not q:
                t = max(t, tasks[i][0])
            while i < n and tasks[i][0] <= t:
                heappush(q, (tasks[i][1], tasks[i][2]))
                i += 1
            pt, j = heappop(q)
            ans.append(j)
            t += pt
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the time at which each task finishes
# 2. Two CPUs that work in parallel


# ============================================================
# FOLLOW-UP 1: Return the time at which each task finishes
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same simulation; record the finish time of
# the task that was processed.
# Time: O(n log n).  Space: O(n).
```

```
import heapq


def finish_times(tasks):
    order = sorted(range(len(tasks)), key=lambda i: tasks[i][0])
    heap = []
    result = [0] * len(tasks)
    time = 0
    position = 0
    while position < len(order) or heap:
        if not heap and time < tasks[order[position]][0]:
            time = tasks[order[position]][0]
        while position < len(order) and tasks[order[position]][0] <= time:
            i = order[position]
            heapq.heappush(heap, (tasks[i][1], i))
            position += 1
        duration, i = heapq.heappop(heap)
        time += duration
        result[i] = time
    return result


# =======================================================
# FOLLOW-UP 2: Two CPUs that work in parallel
# =======================================================

# ---- follow-up 2: function ----
# Answer: Keep the time at which each CPU becomes free (a
# second heap); the next task goes to the CPU that is free
# first.
# Time: O(n log n).  Space: O(n).
import heapq


def two_cpu_finish_times(tasks):
    order = sorted(range(len(tasks)), key=lambda i: tasks[i][0])
    ready = []
    cpus = [0, 0]                                   # the time each CPU is free
    result = [0] * len(tasks)
    position = 0
    done = 0
    while done < len(tasks):
        cpus.sort()
        time = cpus[0]
        if not ready and position < len(order) and tasks[order[position]][0] > time:
            time = tasks[order[position]][0]
        while position < len(order) and tasks[order[position]][0] <= time:
```

```
            i = order[position]
            heapq.heappush(ready, (tasks[i][1], i))
            position += 1
        if not ready:
            continue
        duration, i = heapq.heappop(ready)
        cpus[0] = time + duration
        result[i] = cpus[0]
        done += 1
    return result
```

PATTERN 19 · HEAP / PRIORITY QUEUE · PROBLEM 14 OF 20

2542. Maximum Subsequence Score

Medium · Greedy, Array, Sorting, Heap (Priority Queue)

leetcode.com/problems/maximum-subsequence-score

Also appears in pattern 37 Two Heaps

PROBLEM

You are given two **0-indexed** integer arrays `nums1` and `nums2` of equal length `n` and a positive integer `k`. You must choose a **subsequence** of indices from `nums1` of length `k`.

For chosen indices $i_0, i_1, ..., i_{k-1}$, your **score** is defined as:

- The sum of the selected elements from `nums1` multiplied with the **minimum** of the selected elements from `nums2`.
- It can defined simply as: $(nums1[i_0] + nums1[i_1] + ... + nums1[i_{k-1}]) * \min(nums2[i_0], nums2[i_1], ..., nums2[i_{k-1}])$.

Return *the **maximum** possible score.*

A **subsequence** of indices of an array is a set that can be derived from the set $\{0, 1, ..., n-1\}$ by deleting some or no elements.

Example 1:

```
Input: nums1 = [1,3,3,2], nums2 = [2,1,3,4], k = 3
Output: 12
Explanation:
The four possible subsequence scores are:
- We choose the indices 0, 1, and 2 with score = (1+3+3) * min(2,1,3) = 7.
- We choose the indices 0, 1, and 3 with score = (1+3+2) * min(2,1,4) = 6.
- We choose the indices 0, 2, and 3 with score = (1+3+2) * min(2,3,4) = 12.
- We choose the indices 1, 2, and 3 with score = (3+3+2) * min(1,3,4) = 8.
Therefore, we return the max score, which is 12.
```

Example 2:

```
Input: nums1 = [4,2,3,1,1], nums2 = [7,5,10,9,6], k = 1
Output: 30
Explanation:
Choosing index 2 is optimal: nums1[2] * nums2[2] = 3 * 10 = 30 is the maximum possible score.
```

Constraints:

- `n == nums1.length == nums2.length`
- `1 <= n <=` 10^5
- `0 <= nums1[i], nums2[j] <=` 10^5
- `1 <= k <= n`

SOLUTION (PYTHON 3)

Solution 1: Sorting + Priority Queue (Min Heap)

How to think about it

A score is the sum of k chosen nums1 values times the minimum of the matching nums2 values. Enumerating k-subsets is impossible. If the minimum is fixed as a, we may only pick among indices with nums2≥ a. Scanning pairs in decreasing nums2 makes the current a the minimum, and all earlier pairs remain eligible. A min-heap keeps the k largest nums1 values; when it fills, multiply the sum by a and evict the smallest to make room.

Sort nums2 and nums1 in descending order according to nums2, then traverse from front to back, maintaining a min heap. The heap stores elements from nums1, and the number of elements in the heap does not exceed k. At the same time, maintain a variable s representing the sum of the elements in the heap, and continuously update the answer during the traversal process.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the length of the array nums1.

Python 3

```python
class Solution:
    def maxScore(self, nums1: List[int], nums2: List[int], k: int) -> int:
        nums = sorted(zip(nums2, nums1), reverse=True)
        q = []
        ans = s = 0
        for a, b in nums:
            s += b
            heappush(q, b)
            if len(q) == k:
                ans = max(ans, s * a)
                s -= heappop(q)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the chosen indices
# 2. You may choose AT MOST k indices (not exactly k)


# ========================================================
# FOLLOW-UP 1: Return the chosen indices
# ========================================================

# ---- follow-up 1: function ----
# Answer: Sort the pairs by the second array, descending;
# keep a min-heap of the k biggest first-array values seen
# so far, with their indices.
# Time: O(n log n).  Space: O(n).
import heapq


def best_subsequence_indices(nums1, nums2, k):
    order = sorted(range(len(nums1)), key=lambda i: -nums2[i])
    heap = []                                       # (value, index) of the chosen numbers
    total = 0
    best = (-1, [])
    for i in order:
        heapq.heappush(heap, (nums1[i], i))
        total += nums1[i]
        if len(heap) > k:
            value, index = heapq.heappop(heap)
            total -= value
        if len(heap) == k and total * nums2[i] > best[0]:
            best = (total * nums2[i], sorted(index for value, index in heap))
    return best


# ========================================================
# FOLLOW-UP 2: You may choose AT MOST k indices (not
# exactly k)
# ========================================================

# ---- follow-up 2: function ----
# Answer: The same sweep, but evaluate the score whenever
# the heap holds at least one number, not only when it
# holds exactly k.
# Time: O(n log n).  Space: O(k).
```

```
import heapq


def best_score_at_most_k(nums1, nums2, k):
    order = sorted(range(len(nums1)), key=lambda i: -nums2[i])
    heap = []
    total = 0
    best = 0
    for i in order:
        heapq.heappush(heap, nums1[i])
        total += nums1[i]
        if len(heap) > k:
            total -= heapq.heappop(heap)
        if total * nums2[i] > best:
            best = total * nums2[i]
    return best
```

1985. Find the Kth Largest Integer in the Array

Medium · Array, String, Divide and Conquer, Quickselect, Sorting, Heap (Priority Queue)

leetcode.com/problems/find-the-kth-largest-integer-in-the-array

PROBLEM

You are given an array of strings `nums` and an integer `k`. Each string in `nums` represents an integer without leading zeros.

Return *the string that represents the* k^{th} ***largest integer*** *in* `nums`.

Note: Duplicate numbers should be counted distinctly. For example, if `nums` is `["1","2","2"]`, `"2"` is the first largest integer, `"2"` is the second-largest integer, and `"1"` is the third-largest integer.

Example 1:

```
Input: nums = ["3","6","7","10"], k = 4
Output: "3"
Explanation:
The numbers in nums sorted in non-decreasing order are ["3","6","7","10"].
The 4th largest integer in nums is "3".
```

Example 2:

```
Input: nums = ["2","21","12","1"], k = 3
Output: "2"
Explanation:
The numbers in nums sorted in non-decreasing order are ["1","2","12","21"].
The 3rd largest integer in nums is "2".
```

Example 3:

```
Input: nums = ["0","0"], k = 2
Output: "0"
Explanation:
The numbers in nums sorted in non-decreasing order are ["0","0"].
The 2nd largest integer in nums is "0".
```

Constraints:

- `1 <= k <= nums.length <=` 10^4
- `1 <= nums[i].length <= 100`
- `nums[i]` consists of only digits.
- `nums[i]` will not have any leading zeros.

SOLUTION (PYTHON 3)

Solution 1: Sorting or Quickselect

> **How to think about it**
>
> The entries are decimal strings and must not be compared lexicographically. After parsing as integers we want the k-th largest; a heap selection is enough. nlargest(k,·) keyed by int returns that element at position k-1.

We can sort the strings in the nums array in descending order as integers, and then take the k-th element. Alternatively, we can use the quickselect algorithm to find the k-th largest integer.

The time complexity is O(n × log n) or O(n), where n is the length of the nums array. The space complexity is O(log n) or O(1).

Python 3

```
class Solution:
    def kthLargestNumber(self, nums: List[str], k: int) -> str:
        return nlargest(k, nums, key=lambda x: int(x))[k - 1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Sort big numbers given as strings correctly
# 2. Use a min-heap of size k with the same comparison


# ========================================================
# FOLLOW-UP 1: Sort big numbers given as strings correctly
# ========================================================

# ---- follow-up 1: function ----
# Answer: Compare by length first (a longer number without
# leading zeros is bigger), then alphabetically: sort by
# the pair (length, text).
# Time: O(n log n).  Space: O(n).
def kth_largest_number_string(nums, k):
    pairs = []
    for text in nums:
        pairs.append((len(text), text))
    pairs.sort(reverse=True)
    return pairs[k - 1][1]


# ========================================================
# FOLLOW-UP 2: Use a min-heap of size k with the same
# comparison
# ========================================================

# ---- follow-up 2: function ----
# Answer: Keep the k biggest numbers in a heap of (length,
# text) pairs; the top is the answer.
# Time: O(n log k).  Space: O(k).
import heapq


def kth_largest_number_heap(nums, k):
    heap = []
    for text in nums:
        heapq.heappush(heap, (len(text), text))
        if len(heap) > k:
            heapq.heappop(heap)
    return heap[0][1]
```

PATTERN 19 · HEAP / PRIORITY QUEUE · PROBLEM 16 OF 20

692. Top K Frequent Words

Medium · Trie, Array, Hash Table, String, Bucket Sort, Counting, Sorting, Heap (Priority Queue)

leetcode.com/problems/top-k-frequent-words

PROBLEM

Given an array of strings `words` and an integer `k`, return *the* `k` *most frequent strings*.

Return the answer **sorted** by **the frequency** from highest to lowest. Sort the words with the same frequency by their **lexicographical order**.

Example 1:

```
Input: words = ["i","love","leetcode","i","love","coding"], k = 2
Output: ["i","love"]
Explanation: "i" and "love" are the two most frequent words.
Note that "i" comes before "love" due to a lower alphabetical order.
```

Example 2:

```
Input: words = ["the","day","is","sunny","the","the","the","sunny","is","is"], k = 4
Output: ["the","is","sunny","day"]
Explanation: "the", "is", "sunny" and "day" are the four most frequent words, with the number of
occurrence being 4, 3, 2 and 1 respectively.
```

Constraints:

- `1 <= words.length <= 500`
- `1 <= words[i].length <= 10`
- `words[i]` consists of lowercase English letters.
- `k` is in the range `[1, The number of unique words[i]]`

Follow-up: Could you solve it in `O(n log(k))` time and `O(n)` extra space?

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Sorting

> **How to think about it**
>
> Return the k most frequent words, breaking ties lexicographically. A heap is O(nlog k), but a full sort is fine here. Count, sort keys by (-count, word), and take the first k.

We can use a hash table cnt to record the frequency of each word. Then, we sort the key-value pairs in the hash table by value, and if the values are the same, we sort by key.

Finally, we take the first k keys.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the number of words.

Python 3

```python
class Solution:
    def topKFrequent(self, words: List[str], k: int) -> List[str]:
        cnt = Counter(words)
        return sorted(cnt, key=lambda x: (-cnt[x], x))[:k]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. A heap of size k with a custom order (O(n log k))
# 2. Words arrive in a stream: answer 'top k now' at any
# moment


# ============================================================
# FOLLOW-UP 1: A heap of size k with a custom order (O(n
# log k))
# ============================================================

# ---- follow-up 1: function ----
# Answer: Wrap (count, word) in an entry whose order is:
# smaller count is 'worse', and for equal counts the
# alphabetically LATER word is 'worse'. Keep the k best
# entries in a min-heap of 'worst first' order.
# Time: O(n log k).  Space: O(n).
import heapq


class Entry:
    def __init__(self, count, word):
        self.count = count
        self.word = word

    def __lt__(self, other):
        if self.count != other.count:
            return self.count < other.count
        return self.word > other.word


def top_k_words_heap(words, k):
    count = {}
    for w in words:
        count[w] = count.get(w, 0) + 1
    heap = []
    for w in count:
        heapq.heappush(heap, Entry(count[w], w))
        if len(heap) > k:
            heapq.heappop(heap)
    result = []
    while heap:
        result.append(heapq.heappop(heap).word)
    result.reverse()
    return result


# ============================================================
# FOLLOW-UP 2: Words arrive in a stream: answer 'top k
# now' at any moment
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the counts between calls and build the
# answer from the dictionary when asked.
# Time: add O(1), top O(w log w).  Space: O(w).
class TopWordsStream:
    def __init__(self):
        self.count = {}
```

```
    def add(self, word):
        self.count[word] = self.count.get(word, 0) + 1

    def top(self, k):
        pairs = []
```

```
        for w in self.count:
            pairs.append((-self.count[w], w))
        pairs.sort()
        result = []
        for i in range(min(k, len(pairs))):
            result.append(pairs[i][1])
        return result
```

PATTERN 19 · HEAP / PRIORITY QUEUE · PROBLEM 17 OF 20

1845. Seat Reservation Manager

Medium · Design, Heap (Priority Queue)

leetcode.com/problems/seat-reservation-manager

Also appears in pattern 37 Two Heaps

PROBLEM

Design a system that manages the reservation state of `n` seats that are numbered from `1` to `n`.

Implement the `SeatManager` class:

- `SeatManager(int n)` Initializes a `SeatManager` object that will manage `n` seats numbered from `1` to `n`. All seats are initially available.
- `int reserve()` Fetches the **smallest-numbered** unreserved seat, reserves it, and returns its number.
- `void unreserve(int seatNumber)` Unreserves the seat with the given `seatNumber`.

Example 1:

```
Input
["SeatManager", "reserve", "reserve", "unreserve", "reserve", "reserve", "reserve", "reserve",
"unreserve"]
[[5], [], [], [2], [], [], [], [], [5]]
Output
[null, 1, 2, null, 2, 3, 4, 5, null]

Explanation
SeatManager seatManager = new SeatManager(5); // Initializes a SeatManager with 5 seats.
seatManager.reserve(); // All seats are available, so return the lowest numbered seat, which is 1.
seatManager.reserve(); // The available seats are [2,3,4,5], so return the lowest of them, which is 2.
seatManager.unreserve(2); // Unreserve seat 2, so now the available seats are [2,3,4,5].
seatManager.reserve(); // The available seats are [2,3,4,5], so return the lowest of them, which is 2.
seatManager.reserve(); // The available seats are [3,4,5], so return the lowest of them, which is 3.
seatManager.reserve(); // The available seats are [4,5], so return the lowest of them, which is 4.
seatManager.reserve(); // The only available seat is seat 5, so return 5.
seatManager.unreserve(5); // Unreserve seat 5, so now the available seats are [5].
```

Constraints:

- $1 <= n <= 10^5$
- $1 <= seatNumber <= n$
- For each call to `reserve`, it is guaranteed that there will be at least one unreserved seat.
- For each call to `unreserve`, it is guaranteed that `seatNumber` will be reserved.
- At most 10^5 calls **in total** will be made to `reserve` and `unreserve`.

SOLUTION (PYTHON 3)

Solution 1: Priority Queue (Min-Heap)

> **How to think about it**
>
> We must always reserve the smallest free seat and later release seats. Scanning a boolean array for the minimum free index is O(n) per call. Store free seat numbers in a min-heap: reserve pops the top, unreserve pushes the number back. Both updates are logarithmic.

We define a priority queue (min-heap) q to store all the available seat numbers. Initially, we add all seat numbers from 1 to n into q.

When calling the `reserve` method, we pop the top element from q, which is the smallest available seat number.

When calling the `unreserve` method, we add the seat number back into q.

In terms of time complexity, the initialization time complexity is O(n) or O(n × log n), and the time complexity of the `reserve` and `unreserve` methods is both O(log n). The space complexity is O(n).

Python 3

```
class SeatManager:
    def __init__(self, n: int):
        self.q = list(range(1, n + 1))

    def reserve(self) -> int:
        return heappop(self.q)

    def unreserve(self, seatNumber: int) -> None:
        heappush(self.q, seatNumber)


# Your SeatManager object will be instantiated and called as such:
# obj = SeatManager(n)
# param_1 = obj.reserve()
# obj.unreserve(seatNumber)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Reserve a SPECIFIC seat
# 2. Reserve k CONSECUTIVE seats (the lowest block)


# ============================================================
# FOLLOW-UP 1: Reserve a SPECIFIC seat
# ============================================================

# ---- follow-up 1: function ----
# Answer: A set of reserved seats plus a lazy heap: a seat
# that was taken directly is skipped when it reaches the
# top of the heap.
# Time: reserve and unreserve O(log n) amortised.  Space:
# O(n).
import heapq


class SeatManagerSpecific:
    def __init__(self, n):
        self.heap = list(range(1, n + 1))
        heapq.heapify(self.heap)
        self.taken = set()

    def reserve(self):
        while self.heap[0] in self.taken:
            self.taken.discard(heapq.heappop(self.heap))
        return heapq.heappop(self.heap)

    def reserve_seat(self, seat):
        self.taken.add(seat)

    def unreserve(self, seat):
        if seat in self.taken:
            self.taken.discard(seat)                # its old entry is still inside the heap
        else:
            heapq.heappush(self.heap, seat)


# ============================================================
# FOLLOW-UP 2: Reserve k CONSECUTIVE seats (the lowest
# block)
```

```
# ============================================================

# ---- follow-up 2: function ----
# Answer: Scan the seats in order and keep the first block
# of k free seats; a set of taken seats tells which are
# free.
# Time: reserve_block O(n), unreserve O(1).  Space: O(n).
class ConsecutiveSeats:
    def __init__(self, n):
        self.n = n
        self.taken = set()

    def reserve_block(self, k):
        run = 0
        for seat in range(1, self.n + 1):
            if seat in self.taken:
                run = 0
            else:
```

```
                run += 1
                if run == k:
                    block = list(range(seat - k + 1, seat + 1))
                    for s in block:
                        self.taken.add(s)
                    return block
        return None

    def unreserve(self, seat):
        self.taken.discard(seat)
```

2462. Total Cost to Hire K Workers

Medium · Array, Two Pointers, Simulation, Heap (Priority Queue)

leetcode.com/problems/total-cost-to-hire-k-workers

Also appears in pattern 37 Two Heaps

PROBLEM

You are given a **0-indexed** integer array `costs` where `costs[i]` is the cost of hiring the i^{th} worker.

You are also given two integers `k` and `candidates`. We want to hire exactly `k` workers according to the following rules:

- You will run `k` sessions and hire exactly one worker in each session.
- In each hiring session, choose the worker with the lowest cost from either the first `candidates` workers or the last `candidates` workers. Break the tie by the smallest index.
  - For example, if `costs = [3,2,7,7,1,2]` and `candidates = 2`, then in the first hiring session, we will choose the 4^{th} worker because they have the lowest cost `[3,2,7,7,1,2]`.
  - In the second hiring session, we will choose 1^{st} worker because they have the same lowest cost as 4^{th} worker but they have the smallest index `[3,2,7,7,2]`. Please note that the indexing may be changed in the process.
- If there are fewer than candidates workers remaining, choose the worker with the lowest cost among them. Break the tie by the smallest index.
- A worker can only be chosen once.

Return *the total cost to hire exactly* `k` *workers.*

Example 1:

```
Input: costs = [17,12,10,2,7,2,11,20,8], k = 3, candidates = 4
Output: 11
Explanation: We hire 3 workers in total. The total cost is initially 0.
- In the first hiring round we choose the worker from [17,12,10,2,7,2,11,20,8]. The lowest cost is 2,
and we break the tie by the smallest index, which is 3. The total cost = 0 + 2 = 2.
- In the second hiring round we choose the worker from [17,12,10,7,2,11,20,8]. The lowest cost is 2
(index 4). The total cost = 2 + 2 = 4.
- In the third hiring round we choose the worker from [17,12,10,7,11,20,8]. The lowest cost is 7
(index 3). The total cost = 4 + 7 = 11. Notice that the worker with index 3 was common in the first and
last four workers.
The total hiring cost is 11.
```

Example 2:

```
Input: costs = [1,2,4,1], k = 3, candidates = 3
Output: 4
Explanation: We hire 3 workers in total. The total cost is initially 0.
- In the first hiring round we choose the worker from [1,2,4,1]. The lowest cost is 1, and we break the
tie by the smallest index, which is 0. The total cost = 0 + 1 = 1. Notice that workers with index 1 and
2 are common in the first and last 3 workers.
- In the second hiring round we choose the worker from [2,4,1]. The lowest cost is 1 (index 2). The
total cost = 1 + 1 = 2.
- In the third hiring round there are less than three candidates. We choose the worker from the
remaining workers [2,4]. The lowest cost is 2 (index 0). The total cost = 2 + 2 = 4.
The total hiring cost is 4.
```

Constraints:

- `1 <= costs.length <= 10^5`
- `1 <= costs[i] <= 10^5`
- `1 <= k, candidates <= costs.length`

SOLUTION (PYTHON 3)

Solution 1: Priority Queue (Min Heap)

> **How to think about it**
>
> Each of k hires picks the cheapest among the first and last candidates workers; n≤ 10^5. If those two sides cover the array, just take the k global minima. Otherwise a heap holds both sides (with indices). After popping, push the next unused worker from that side. Stop refilling when the two pointers cross.

First, we check if candidates × 2 is greater than or equal to n. If it is, we directly return the sum of the costs of the first k smallest workers.

Otherwise, we use a min heap pq to maintain the costs of the first candidates workers and the last candidates workers.

We first add the costs and corresponding indices of the first candidates workers to the min heap pq, and then add the costs and corresponding indices of the last candidates workers to the min heap pq. We use two pointers l and r to point to the indices of the front and back candidates, initially l = candidates, r = n - candidates - 1.

Then we perform k iterations, each time taking the worker with the smallest cost from the min heap pq and adding its cost to the answer. If l > r, it means that all the front and back candidates have been selected, and we skip directly. Otherwise, if the index of the current worker is less than l, it means it is a worker from the front, we add the cost and index of the l-th worker to the min heap pq, and then increment l; otherwise, we add the cost and index of the r-th worker to the min heap pq, and then decrement r.

After the loop ends, we return the answer.

The time complexity is O(n × log n), and the space complexity is O(n). Where n is the length of the array costs.

Python 3

```python
class Solution:
    def totalCost(self, costs: List[int], k: int, candidates: int) -> int:
        n = len(costs)
        if candidates * 2 >= n:
            return sum(sorted(costs)[:k])
        pq = []
        for i, c in enumerate(costs[:candidates]):
            heappush(pq, (c, i))
        for i in range(n - candidates, n):
            heappush(pq, (costs[i], i))
        heapify(pq)
        l, r = candidates, n - candidates - 1
        ans = 0
        for _ in range(k):
            c, i = heappop(pq)
            ans += c
            if l > r:
                continue
            if i < l:
                heappush(pq, (costs[l], l))
                l += 1
            else:
                heappush(pq, (costs[r], r))
                r -= 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return which workers are hired


# ============================================================
# FOLLOW-UP 1: Return which workers are hired
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same two heaps (the first candidates and the
# last candidates), each with the worker's index; collect
# the index of every hired worker.
# Time: O((k + c) log c), c = candidates.  Space: O(c).
import heapq


def hired_workers(costs, k, candidates):
    n = len(costs)
    left = []
    right = []
    i, j = 0, n - 1
    while i < candidates and i <= j:
        heapq.heappush(left, (costs[i], i))
        i += 1
    while j >= n - candidates and j >= i:
        heapq.heappush(right, (costs[j], j))
        j -= 1
    hired = []
    for step in range(k):
        if right == [] or (left != [] and left[0] <= right[0]):
            cost, index = heapq.heappop(left)
            if i <= j:
                heapq.heappush(left, (costs[i], i))
                i += 1
        else:
            cost, index = heapq.heappop(right)
            if i <= j:
                heapq.heappush(right, (costs[j], j))
                j -= 1
        hired.append(index)
    return sorted(hired)
```

PATTERN 19 · HEAP / PRIORITY QUEUE · PROBLEM 19 OF 20

1383. Maximum Performance of a Team

Hard · Greedy, Array, Sorting, Heap (Priority Queue)

leetcode.com/problems/maximum-performance-of-a-team

★ **TOP 7 PROBLEM** in Two Heaps: master this one first

Also appears in pattern 37 Two Heaps

PROBLEM

You are given two integers `n` and `k` and two integer arrays `speed` and `efficiency` both of length `n`. There are `n` engineers numbered from `1` to `n`. `speed[i]` and `efficiency[i]` represent the speed and efficiency of the i^{th} engineer respectively.

Choose **at most** `k` different engineers out of the `n` engineers to form a team with the maximum **performance**.

The performance of a team is the sum of its engineers' speeds multiplied by the minimum efficiency among its engineers.

Return *the maximum performance of this team*. Since the answer can be a huge number, return it **modulo** $10^9 + 7$.

Example 1:

```
Input: n = 6, speed = [2,10,3,1,5,8], efficiency = [5,4,3,9,7,2], k = 2
Output: 60
Explanation:
We have the maximum performance of the team by selecting engineer 2 (with speed=10 and efficiency=4)
and engineer 5 (with speed=5 and efficiency=7). That is, performance = (10 + 5) * min(4, 7) = 60.
```

Example 2:

```
Input: n = 6, speed = [2,10,3,1,5,8], efficiency = [5,4,3,9,7,2], k = 3
Output: 68
Explanation:
This is the same example as the first but k = 3. We can select engineer 1, engineer 2 and engineer 5 to
get the maximum performance of the team. That is, performance = (2 + 10 + 5) * min(5, 4, 7) = 68.
```

Example 3:

```
Input: n = 6, speed = [2,10,3,1,5,8], efficiency = [5,4,3,9,7,2], k = 4
Output: 72
```

Constraints:

- $1 <= k <= n <= 10^5$
- `speed.length == n`
- `efficiency.length == n`
- $1 <= speed[i] <= 10^5$
- $1 <= efficiency[i] <= 10^8$

SOLUTION (PYTHON 3)

Solution 1

How to think about it

A team of at most k people scores (sum of speeds) times (minimum efficiency). n ≤ 10^5 forbids subsets. Scanning engineers from high efficiency to low makes the current efficiency the team's floor; we only keep the k fastest so far. A min-heap drops the slowest when the size hits k, and the speed sum times the current efficiency updates the answer.

Python 3

```python
class Solution:
    def maxPerformance(
        self, n: int, speed: List[int], efficiency: List[int], k: int
    ) -> int:
        t = sorted(zip(speed, efficiency), key=lambda x: -x[1])
        ans = tot = 0
        mod = 10**9 + 7
        h = []
        for s, e in t:
            tot += s
            ans = max(ans, tot * e)
            heappush(h, s)
            if len(h) == k:
                tot -= heappop(h)
        return ans % mod
```

Complexity. Time: O(n log n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the team members that give the best
# performance
# 2. The team must have EXACTLY k engineers


# ============================================================
# FOLLOW-UP 1: Return the team members that give the best
# performance
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same sweep over the engineers from the best
# efficiency down, with a min-heap of the k best speeds
# (and their indices); remember the members at the best
# moment.
# Time: O(n log n).  Space: O(n).
import heapq


def best_team(n, speed, efficiency, k):
    order = sorted(range(n), key=lambda i: -efficiency[i])
    heap = []
    total = 0
    best = (0, [])
    for i in order:
        heapq.heappush(heap, (speed[i], i))
        total += speed[i]
        if len(heap) > k:
            s, j = heapq.heappop(heap)
            total -= s
        if total * efficiency[i] > best[0]:
            best = (total * efficiency[i], sorted(j for s, j in heap))
    return best


# ============================================================
# FOLLOW-UP 2: The team must have EXACTLY k engineers
# ============================================================

# ---- follow-up 2: function ----
# Answer: Only accept a performance value when the heap
# holds exactly k engineers.
# Time: O(n log n).  Space: O(k).
import heapq
```

```
def best_team_exactly_k(n, speed, efficiency, k):
    order = sorted(range(n), key=lambda i: -efficiency[i])
    heap = []
    total = 0
    best = 0
    for i in order:
        heapq.heappush(heap, speed[i])
        total += speed[i]
        if len(heap) > k:
            total -= heapq.heappop(heap)
        if len(heap) == k and total * efficiency[i] > best:
            best = total * efficiency[i]
    return best
```

1882. Process Tasks Using Servers

Medium · Array, Heap (Priority Queue)

leetcode.com/problems/process-tasks-using-servers

★ **TOP 7 PROBLEM** in Heap / Priority Queue: master this one first

Also appears in pattern 37 Two Heaps

PROBLEM

You are given two **0-indexed** integer arrays `servers` and `tasks` of lengths `n` and `m` respectively. `servers[i]` is the **weight** of the `i`$^{\square\square\square\square\square\square th}$ server, and `tasks[j]` is the **time needed** to process the `j`$^{\square\square\square\square\square\square th}$ task **in seconds**.

Tasks are assigned to the servers using a **task queue**. Initially, all servers are free, and the queue is **empty**.

At second `j`, the `j`th task is **inserted** into the queue (starting with the `0`th task being inserted at second `0`). As long as there are free servers and the queue is not empty, the task in the front of the queue will be assigned to a free server with the **smallest weight**, and in case of a tie, it is assigned to a free server with the **smallest index**.

If there are no free servers and the queue is not empty, we wait until a server becomes free and immediately assign the next task. If multiple servers become free at the same time, then multiple tasks from the queue will be assigned **in order of insertion** following the weight and index priorities above.

A server that is assigned task `j` at second `t` will be free again at second `t + tasks[j]`.

Build an array `ans` of length `m`, where `ans[j]` is the **index** of the server the `j`$^{\square\square\square\square\square\square th}$ task will be assigned to.

Return *the array* `ans`.

Example 1:

```
Input: servers = [3,3,2], tasks = [1,2,3,2,1,2]
Output: [2,2,0,2,1,2]
Explanation: Events in chronological order go as follows:
- At second 0, task 0 is added and processed using server 2 until second 1.
- At second 1, server 2 becomes free. Task 1 is added and processed using server 2 until second 3.
- At second 2, task 2 is added and processed using server 0 until second 5.
- At second 3, server 2 becomes free. Task 3 is added and processed using server 2 until second 5.
- At second 4, task 4 is added and processed using server 1 until second 5.
- At second 5, all servers become free. Task 5 is added and processed using server 2 until second 7.
```

Example 2:

```
Input: servers = [5,1,4,3,2], tasks = [2,1,2,4,5,2,1]
Output: [1,4,1,4,1,3,2]
Explanation: Events in chronological order go as follows:
- At second 0, task 0 is added and processed using server 1 until second 2.
- At second 1, task 1 is added and processed using server 4 until second 2.
- At second 2, servers 1 and 4 become free. Task 2 is added and processed using server 1 until second
4.
- At second 3, task 3 is added and processed using server 4 until second 7.
- At second 4, server 1 becomes free. Task 4 is added and processed using server 1 until second 9.
- At second 5, task 5 is added and processed using server 3 until second 7.
- At second 6, task 6 is added and processed using server 2 until second 7.
```

Constraints:

- `servers.length == n`
- `tasks.length == m`
- `1 <= n, m <= 2 * 10`^{5}
- `1 <= servers[i], tasks[j] <= 2 * 10`^{5}

SOLUTION (PYTHON 3)

Solution 1: Priority Queue (Min-Heap)

How to think about it

Tasks arrive at times 0,1,2,…. A free server with the smallest weight (then smallest index) is chosen; if none is free we wait for the earliest one. Scanning all servers per task is too slow. An idle heap stores (weight,index); a busy heap stores (free time,weight,index). At task j move finished servers back, or if none are idle take the earliest busy server and chain the new task onto it. A chained free time is a sum of task lengths and reaches 4×10^{10}, so that field is a 64-bit integer.

We use a min-heap idle to maintain all idle servers, where each element is a tuple (x, i) representing the i-th server with weight x. We use another min-heap busy to maintain all busy servers, where each element is a tuple (w, s, i) representing the i-th server that will be idle at time w with weight s. Initially, we add all servers to idle.

Next, we iterate through all tasks. For the j-th task, we first remove all servers from busy that will be idle at or before time j and add them to idle. Then we take the server with the smallest weight from idle, add it to busy, and assign it to the j-th task. If idle is empty, we take the server with the earliest idle time from busy, add it to busy, and assign it to the j-th task.

After iterating through all tasks, we obtain the answer array ans.

The time complexity is O((n + m) log n), where n is the number of servers and m is the number of tasks. The space complexity is O(n). Here, n and m are the number of servers and tasks, respectively.

Similar problems:

- 2402. Meeting Rooms III

Python 3

```python
class Solution:
    def assignTasks(self, servers: List[int], tasks: List[int]) -> List[int]:
        idle = [(x, i) for i, x in enumerate(servers)]
        heapify(idle)
        busy = []
        ans = []
        for j, t in enumerate(tasks):
            while busy and busy[0][0] <= j:
                _, s, i = heappop(busy)
                heappush(idle, (s, i))
            if idle:
                s, i = heappop(idle)
                heappush(busy, (j + t, s, i))
            else:
                w, s, i = heappop(busy)
                heappush(busy, (w + t, s, i))
            ans.append(i)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the time at which every task finishes


# ========================================================
# FOLLOW-UP 1: Return the time at which every task
# finishes
# ========================================================

# ---- follow-up 1: function ----
# Answer: The same two heaps (free servers and busy
# servers by free-time); the finish time of a task is its
# start time plus its duration.
```

```
# Time: O((n + m) log n), n servers, m tasks.  Space: O(n
# + m).
import heapq


def task_finish_times(servers, tasks):
    free = []
    for i in range(len(servers)):
        heapq.heappush(free, (servers[i], i))
    busy = []
    finish = []
    time = 0
    for j in range(len(tasks)):
        time = max_of(time, j)
        while busy and busy[0][0] <= time:
            ready, weight, index = heapq.heappop(busy)
            heapq.heappush(free, (weight, index))
        if not free:
            time = busy[0][0]
            while busy and busy[0][0] <= time:
                ready, weight, index = heapq.heappop(busy)
                heapq.heappush(free, (weight, index))
        weight, index = heapq.heappop(free)
        heapq.heappush(busy, (time + tasks[j], weight, index))
        finish.append(time + tasks[j])
    return finish


def max_of(a, b):
    if a > b:
        return a
    return b
```

PATTERN 20 OF 41

Intervals (Merge / Insert / Overlap)

20 problems · 0 Easy · 13 Medium · 7 Hard

THE IDEA

Sort by start, then sweep: either the next interval overlaps the current one (merge) or it starts a new one.

Signals: merge intervals, insert interval, meeting rooms, minimum arrows, non-overlapping intervals, employee free time.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
intervals.sort(key=lambda x: x[0])
merged = [intervals[0]]
for start, end in intervals[1:]:
    if start <= merged[-1][1]:                    # overlaps the last one
        merged[-1][1] = max(merged[-1][1], end)
    else:
        merged.append([start, end])
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	★ **Merge Intervals**	Medium	LC 56	1087
2	★ **Insert Interval**	Medium	LC 57	1090
3	★ **Non-overlapping Intervals**	Medium	LC 435	1094
4	Minimum Number of Arrows to Burst Balloons	Medium	LC 452	1097
5	★ **Interval List Intersections**	Medium	LC 986	1099
6	My Calendar I	Medium	LC 729	1102
7	My Calendar II	Medium	LC 731	1105
8	★ **Car Pooling**	Medium	LC 1094	1109
9	Maximum Number of Events That Can Be Attended	Medium	LC 1353	1112
10	Video Stitching	Medium	LC 1024	1115
11	Remove Covered Intervals	Medium	LC 1288	1118
12	Partition Labels (see page)	Medium	LC 763	169
13	Teemo Attacking — substitute: Meeting Scheduler — substitute: Find Right Interval	Medium	LC 436	1120
14	★ **Employee Free Time**	Hard	LC 759	1122
15	★ **My Calendar III**	Hard	LC 732	1124
16	Minimum Interval to Include Each Query	Hard	LC 1851	1128
17	Range Module	Hard	LC 715	1131
18	Minimum Number of Taps to Open to Water a Garden	Hard	LC 1326	1136
19	Data Stream as Disjoint Intervals	Hard	LC 352	1139
20	Falling Squares	Hard	LC 699	1142

PATTERN 20 · INTERVALS (MERGE / INSERT / OVERLAP) · PROBLEM 1 OF 20

56. Merge Intervals

Medium · Array, Sorting, Quick Sort

leetcode.com/problems/merge-intervals

★ **TOP 7 PROBLEM** in Intervals (Merge / Insert / Overlap), Sorting: master this one first

Also appears in pattern 23 Sorting

PROBLEM

Given an array of `intervals` where `intervals[i] = [start_i, end_i]`, merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input*.

Example 1:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
Explanation: Since intervals [1,3] and [2,6] overlap, merge them into [1,6].
```

Example 2:

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
Explanation: Intervals [1,4] and [4,5] are considered overlapping.
```

Example 3:

```
Input: intervals = [[4,7],[1,4]]
Output: [[1,7]]
Explanation: Intervals [1,4] and [4,7] are considered overlapping.
```

Constraints:

- `1 <= intervals.length <= 10^4`
- `intervals[i].length == 2`
- `0 <= start_i <= end_i <= 10^4`

SOLUTIONS (PYTHON 3)

Solution 1: Sorting + One-pass Traversal

How to think about it

The first idea is to pick an interval and scan the rest for overlaps, merging until nothing changes. Correct, but worst-case $O(n^2)$ with a messy merge order. $n \le 10^4$ is tight. The waste is locating overlaps in unsorted input. After sorting by left endpoint, an interval can overlap only the interval we have not closed yet—later starts are larger, so they cannot skip over the middle and overlap again. So we sort, then scan once, keeping st, ed as the interval under merge.

We can sort the intervals in ascending order by the left endpoint, and then traverse the intervals for merging operations.

The specific merging operation is as follows.

First, we add the first interval to the answer. Then, we consider each subsequent interval in turn:

- If the right endpoint of the last interval in the answer array is less than the left endpoint of the current interval, it means that the two intervals will not overlap, so we can directly add the current interval to the end of the answer array;
- Otherwise, it means that the two intervals overlap. We need to use the right endpoint of the current interval to update the right endpoint of the last interval in the answer array, setting it to the larger of the two.

Finally, we return the answer array.

The time complexity is $O(n \times \log n)$, and the space complexity is $O(\log n)$. Here, n is the number of intervals.

Python 3

```python
class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        intervals.sort()
        ans = []
        st, ed = intervals[0]
        for s, e in intervals[1:]:
            if ed < s:
                ans.append([st, ed])
                st, ed = s, e
            else:
                ed = max(ed, e)
        ans.append([st, ed])
        return ans
```

Solution 2

> **How to think about it**
>
> Solution 1 is already O(n log n) and correct. It still keeps a separate st, ed, writes only when the interval closes, and appends once more at the end. What it lacks is storing the current interval in the answer: put the first interval into ans immediately, then either extend ans[-1]'s right end or append. Fewer variables, no final flush.

Python 3

```python
class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        intervals.sort()
        ans = [intervals[0]]
        for s, e in intervals[1:]:
            if ans[-1][1] < s:
                ans.append([s, e])
            else:
                ans[-1][1] = max(ans[-1][1], e)
        return ans
```

Solution 3

> **How to think about it**
>
> Solution 2 already mutates the last interval in the answer. The loop is still "look at one, patch if needed." What it lacks is merging by groups: fix left l, eat every overlapping interval in an inner loop while stretching r, then push [l, r] once. Already-written answers are never rewritten.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```python
class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        intervals.sort(key=lambda x: x[0])
        n = len(intervals)
        result = []
        i = 0
        while i < n:
            left, right = intervals[i]
            i += 1
            while i < n and right >= intervals[i][0]:
                right = max(right, intervals[i][1])
                i += 1
            result.append([left, right])
        return result
```

Complexity. Time: O(n log n). Space: O(log n) for the sort (plus the output).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The total length covered by the intervals
# 2. Intervals arrive one at a time: keep the merged list
# up to date


# ============================================================
# FOLLOW-UP 1: The total length covered by the intervals
# ============================================================

# ---- follow-up 1: function ----
# Answer: Merge first, then add up the lengths of the
# merged intervals.
# Time: O(n log n).  Space: O(n).
def merge(intervals):
    merged = []
    for start, end in sorted(intervals):
        if merged and start <= merged[-1][1]:
            if end > merged[-1][1]:
                merged[-1][1] = end
        else:
            merged.append([start, end])
    return merged


def covered_length(intervals):
    total = 0
    for start, end in merge(intervals):
        total += end - start
    return total


# ============================================================
# FOLLOW-UP 2: Intervals arrive one at a time: keep the
# merged list up to date
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the merged list sorted; a new interval is
# merged with every interval it overlaps (found with a
# scan or a binary search).
# Time: O(m) per add, m = merged intervals.  Space: O(m).
class MergedIntervals:
    def __init__(self):
        self.items = []

    def add(self, start, end):
        result = []
        placed = False
        for s, e in self.items:
            if e < start:
                result.append([s, e])
            elif s > end:
                if not placed:
                    result.append([start, end])
                    placed = True
                result.append([s, e])
            else:
                start = s if s < start else start
                end = e if e > end else end
        if not placed:
            result.append([start, end])
        self.items = result
        return self.items
```

PATTERN 20 · INTERVALS (MERGE / INSERT / OVERLAP) · PROBLEM 2 OF 20

57. Insert Interval

Medium · Array

leetcode.com/problems/insert-interval

★ **TOP 7 PROBLEM** in Intervals (Merge / Insert / Overlap): master this one first

PROBLEM

You are given an array of non-overlapping intervals `intervals` where `intervals[i] = [start_i, end_i]` represent the start and the end of the `i^th` interval and `intervals` is sorted in ascending order by `start_i`. You are also given an interval `newInterval = [start, end]` that represents the start and end of another interval.

Two intervals are considered overlapping if they share **at least** one point.

Insert `newInterval` into `intervals` such that `intervals` is still sorted in ascending order by `start_i` and `intervals` still does not have any overlapping intervals (merge overlapping intervals if necessary).

Return `intervals` *after the insertion*.

Note that you don't need to modify `intervals` in-place. You can make a new array and return it.

Example 1:

```
Input: intervals = [[1,3],[6,9]], newInterval = [2,5]
Output: [[1,5],[6,9]]
```

Example 2:

```
Input: intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]], newInterval = [4,8]
Output: [[1,2],[3,10],[12,16]]
Explanation: Because the new interval [4,8] overlaps with [3,5],[6,7],[8,10].
```

Constraints:

- `0 <= intervals.length <= 10^4`
- `intervals[i].length == 2`
- `0 <= start_i <= end_i <= 10^5`
- `intervals` is sorted by `start_i` in **ascending** order.
- `newInterval.length == 2`
- `0 <= start <= end <= 10^5`

SOLUTIONS (PYTHON 3)

Solution 1: Sorting + Interval Merging

How to think about it

The first idea is to insert newInterval into the list and reuse ordinary interval merging. n ≤ 10^4, so an O(n log n) sort will pass. The list is already sorted and non-overlapping, but reusing merge-intervals is the least extra code: append, sort, merge. Correct; it just ignores the given order.

We can first add the new interval `newInterval` to the interval list `intervals`, and then merge according to the regular method of interval merging.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the number of intervals.

Python 3

```python
class Solution:
    def insert(
        self, intervals: List[List[int]], newInterval: List[int]
    ) -> List[List[int]]:
        def merge(intervals: List[List[int]]) -> List[List[int]]:
            intervals.sort()
            ans = [intervals[0]]
            for s, e in intervals[1:]:
                if ans[-1][1] < s:
                    ans.append([s, e])
                else:
                    ans[-1][1] = max(ans[-1][1], e)
            return ans

        intervals.append(newInterval)
        return merge(intervals)
```

Solution 2: One-pass Traversal

> **How to think about it**
>
> Solution 1 sorts again and throws away the existing order, paying an extra log n. What it lacks is a single pass: write a current interval if it lies entirely left of the new one; if it lies entirely right, insert the new interval first; otherwise widen the new interval's ends. Append at the end if it was never inserted. Linear, no sort.

We can traverse the interval list `intervals`, let the current interval be `interval`, and there are three situations for each interval:

- The current interval is on the right side of the new interval, that is, newInterval[1] < interval[0]. At this time, if the new interval has not been added, then add the new interval to the answer, and then add the current interval to the answer.
- The current interval is on the left side of the new interval, that is, interval[1] < newInterval[0]. At this time, add the current interval to the answer.
- Otherwise, it means that the current interval and the new interval intersect. We take the minimum of the left endpoint of the current interval and the left endpoint of the new interval, and the maximum of the right endpoint of the current interval and the right endpoint of the new interval, as the left and right endpoints of the new interval, and then continue to traverse the interval list.

After the traversal, if the new interval has not been added, then add the new interval to the answer.

The time complexity is O(n), where n is the number of intervals. Ignoring the space consumption of the answer array, the space complexity is O(1).

Python 3

```python
class Solution:
    def insert(
        self, intervals: List[List[int]], newInterval: List[int]
    ) -> List[List[int]]:
        st, ed = newInterval
        ans = []
        insert = False
        for s, e in intervals:
            if ed < s:
                if not insert:
                    ans.append([st, ed])
                    insert = True
                ans.append([s, e])
            elif e < st:
                ans.append([s, e])
            else:
                st = min(st, s)
                ed = max(ed, e)
        if not insert:
            ans.append([st, ed])
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Remove (subtract) an interval from a list of disjoint
# intervals
# 2. Insert SEVERAL new intervals


# ============================================================
# FOLLOW-UP 1: Remove (subtract) an interval from a list
# of disjoint intervals
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the parts of every interval that lie
# outside the removed range: the part before it and the
# part after it.
# Time: O(n).  Space: O(n).
def remove_interval(intervals, remove_start, remove_end):
    result = []
    for start, end in intervals:
        if end <= remove_start or start >= remove_end:
            result.append([start, end])
        else:
            if start < remove_start:
                result.append([start, remove_start])
            if end > remove_end:
                result.append([remove_end, end])
    return result


# ============================================================
# FOLLOW-UP 2: Insert SEVERAL new intervals
# ============================================================

# ---- follow-up 2: function ----
# Answer: Insert them one by one, or add them all to the
# list and merge once.
# Time: O((n + k) log (n + k)).  Space: O(n + k).
def insert_many(intervals, new_intervals):
    merged = []
    for start, end in sorted(intervals + new_intervals):
        if merged and start <= merged[-1][1]:
```

```
            if end > merged[-1][1]:
                merged[-1][1] = end
        else:
            merged.append([start, end])
    return merged
```

PATTERN 20 · INTERVALS (MERGE / INSERT / OVERLAP) · PROBLEM 3 OF 20

435. Non-overlapping Intervals

Medium · Greedy, Array, Dynamic Programming, Sorting

leetcode.com/problems/non-overlapping-intervals

★ **TOP 7 PROBLEM** in Intervals (Merge / Insert / Overlap): master this one first

Also appears in pattern 21 Greedy

PROBLEM

Given an array of intervals `intervals` where `intervals[i] = [start_i, end_i]`, return *the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping*.

Note that intervals which only touch at a point are **non-overlapping**. For example, `[1, 2]` and `[2, 3]` are non-overlapping.

Example 1:

```
Input: intervals = [[1,2],[2,3],[3,4],[1,3]]
Output: 1
Explanation: [1,3] can be removed and the rest of the intervals are non-overlapping.
```

Example 2:

```
Input: intervals = [[1,2],[1,2],[1,2]]
Output: 2
Explanation: You need to remove two [1,2] to make the rest of the intervals non-overlapping.
```

Example 3:

```
Input: intervals = [[1,2],[2,3]]
Output: 0
Explanation: You don't need to remove any of the intervals since they're already non-overlapping.
```

Constraints:

- $1 <= intervals.length <= 10^5$
- `intervals[i].length == 2`
- $-5 * 10^4 <= start_i < end_i <= 5 * 10^4$

SOLUTION (PYTHON 3)

Solution 1: Sorting + Greedy

How to think about it

Removing the fewest intervals so the rest are disjoint is the same as keeping the largest set of non-overlapping intervals. Subset search is too large. Sort by right endpoint and keep an interval when it starts at or after the last kept end. The earliest finish leaves the most room, which is why we sort by the right end. The answer starts at n and decreases once per kept interval: total minus the size of a maximum compatible set.

We first sort the intervals in ascending order by their right boundary. We use a variable pre to record the right boundary of the previous interval and a variable ans to record the number of intervals that need to be removed. Initially, ans = intervals.length.

Then we iterate through the intervals. For each interval:

- If the left boundary of the current interval is greater than or equal to pre, it means that this interval does not need to be removed. We directly update pre to the right boundary of the current interval and decrement ans by one;
- Otherwise, it means that this interval needs to be removed, and we do not need to update pre and ans.

Finally, we return ans.

The time complexity is O(n × log n), and the space complexity is O(log n), where n is the number of intervals.

Python 3

```python
class Solution:
    def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int:
        intervals.sort(key=lambda x: x[1])
        ans = len(intervals)
        pre = -inf
        for l, r in intervals:
            if pre <= l:
                ans -= 1
                pre = r
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the intervals that must be removed
# 2. Every interval has a WEIGHT: the best total weight of
# non-overlapping intervals


# ============================================================
# FOLLOW-UP 1: Return the intervals that must be removed
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort by end; keep an interval when it starts at
# or after the end of the last kept one, otherwise it is
# removed.
# Time: O(n log n).  Space: O(n).
def intervals_to_remove(intervals):
    ordered = sorted(intervals, key=lambda x: x[1])
    removed = []
    last_end = None
    for start, end in ordered:
        if last_end is None or start >= last_end:
            last_end = end
        else:
            removed.append([start, end])
    return removed


# ============================================================
# FOLLOW-UP 2: Every interval has a WEIGHT: the best total
# weight of non-overlapping intervals
# ============================================================

# ---- follow-up 2: function ----
# Answer: Sort by end; best[i] = the best total using the
# first i intervals: either skip interval i, or take it
# and add the best total over intervals that end before it
# starts (found with a binary search).
# Time: O(n log n).  Space: O(n).
import bisect


def max_weight_schedule(intervals):
    # intervals are (start, end, weight)
    ordered = sorted(intervals, key=lambda x: x[1])
    ends = [x[1] for x in ordered]
    best = [0] * (len(ordered) + 1)
    for i in range(1, len(ordered) + 1):
        start, end, weight = ordered[i - 1]
        before = bisect.bisect_right(ends, start, 0, i - 1)
```

```
        take = weight + best[before]
        best[i] = take if take > best[i - 1] else best[i - 1]
    return best[len(ordered)]
```

PATTERN 20 · INTERVALS (MERGE / INSERT / OVERLAP) · PROBLEM 4 OF 20

452. Minimum Number of Arrows to Burst Balloons

Medium · Greedy, Array, Sorting

leetcode.com/problems/minimum-number-of-arrows-to-burst-balloons

Also appears in pattern 21 Greedy

PROBLEM

There are some spherical balloons taped onto a flat wall that represents the XY-plane. The balloons are represented as a 2D integer array `points` where `points[i] = [x_start, x_end]` denotes a balloon whose **horizontal diameter** stretches between x_{start} and x_{end}. You do not know the exact y-coordinates of the balloons.

Arrows can be shot up **directly vertically** (in the positive y-direction) from different points along the x-axis. A balloon with x_{start} and x_{end} is **burst** by an arrow shot at x if $x_{start} <= x <= x_{end}$. There is **no limit** to the number of arrows that can be shot. A shot arrow keeps traveling up infinitely, bursting any balloons in its path.

Given the array `points`, return *the **minimum** number of arrows that must be shot to burst all balloons*.

Example 1:

```
Input: points = [[10,16],[2,8],[1,6],[7,12]]
Output: 2
Explanation: The balloons can be burst by 2 arrows:
- Shoot an arrow at x = 6, bursting the balloons [2,8] and [1,6].
- Shoot an arrow at x = 11, bursting the balloons [10,16] and [7,12].
```

Example 2:

```
Input: points = [[1,2],[3,4],[5,6],[7,8]]
Output: 4
Explanation: One arrow needs to be shot for each balloon for a total of 4 arrows.
```

Example 3:

```
Input: points = [[1,2],[2,3],[3,4],[4,5]]
Output: 2
Explanation: The balloons can be burst by 2 arrows:
- Shoot an arrow at x = 2, bursting the balloons [1,2] and [2,3].
- Shoot an arrow at x = 4, bursting the balloons [3,4] and [4,5].
```

Constraints:

- $1 <= points.length <= 10^5$
- `points[i].length == 2`
- $-2^{31} <= x_{start} < x_{end} <= 2^{31} - 1$

SOLUTION (PYTHON 3)

Solution 1

How to think about it

One arrow bursts every balloon covering that x-coordinate; we want the fewest arrows. This is piercing all intervals with the fewest points. Sort by the right end. The current arrow sits on the last burst balloon's right end; if the next left end is past that point, shoot again and move the pin. Pinning the rightmost point of a kept interval covers as many later balloons as possible; sorting by the right end makes that greedy choice optimal.

Python 3

```python
class Solution:
    def findMinArrowShots(self, points: List[List[int]]) -> int:
        ans, last = 0, -inf
        for a, b in sorted(points, key=lambda x: x[1]):
            if a > last:
                ans += 1
                last = b
        return ans
```

Complexity. Time: O(n log n). Space: O(log n) for the sort.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the positions where the arrows are shot
# 2. Balloons are discs on a line... : each arrow bursts
# balloons within distance r of its position


# ============================================================
# FOLLOW-UP 1: Return the positions where the arrows are
# shot
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort by end; shoot at the end of the first
# balloon, and again at the end of the first balloon that
# this arrow misses.
# Time: O(n log n).  Space: O(n).
def arrow_positions(points):
    ordered = sorted(points, key=lambda p: p[1])
    arrows = []
    for start, end in ordered:
        if not arrows or start > arrows[-1]:
            arrows.append(end)
    return arrows


# ============================================================
# FOLLOW-UP 2: Balloons are discs on a line... : each
# arrow bursts balloons within distance r of its position
# ============================================================

# ---- follow-up 2: function ----
# Answer: Treat balloon x as the interval [x - r, x + r];
# the same greedy applies.
# Time: O(n log n).  Space: O(1) extra.
def arrows_for_points(xs, r):
    arrows = 0
    last_shot = None
    for x in sorted(xs):
        if last_shot is None or x - r > last_shot:
            arrows += 1
            last_shot = x + r
    return arrows
```

986. Interval List Intersections

Medium · Array, Two Pointers, Sweep Line

leetcode.com/problems/interval-list-intersections

★ **TOP 7 PROBLEM** in Intervals (Merge / Insert / Overlap): master this one first

PROBLEM

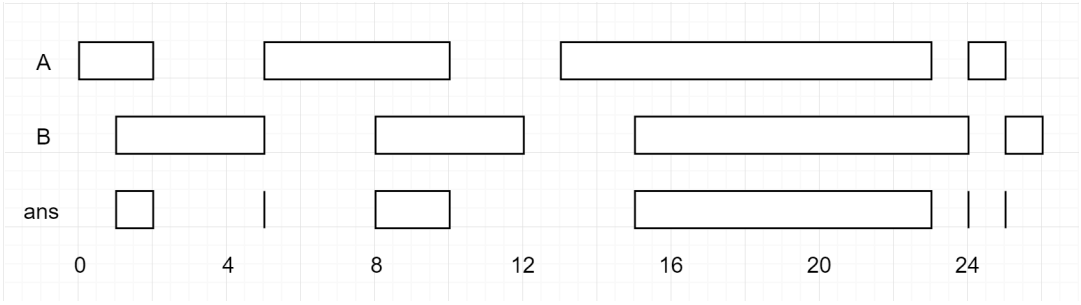
You are given two lists of closed intervals, `firstList` and `secondList`, where `firstList[i] = [start`$_i$`, end`$_i$`]` and `secondList[j] = [start`$_j$`, end`$_j$`]`. Each list of intervals is pairwise **disjoint** and in **sorted order**.

Return *the intersection of these two interval lists*.

A **closed interval** `[a, b]` (with `a <= b`) denotes the set of real numbers `x` with `a <= x <= b`.

The **intersection** of two closed intervals is a set of real numbers that are either empty or represented as a closed interval. For example, the intersection of `[1, 3]` and `[2, 4]` is `[2, 3]`.

Example 1:




```
Input: firstList = [[0,2],[5,10],[13,23],[24,25]], secondList = [[1,5],[8,12],[15,24],[25,26]]
Output: [[1,2],[5,5],[8,10],[15,23],[24,24],[25,25]]
```

Example 2:

```
Input: firstList = [[1,3],[5,9]], secondList = []
Output: []
```

Constraints:

- `0 <= firstList.length, secondList.length <= 1000`
- `firstList.length + secondList.length >= 1`
- `0 <= start`$_i$ `< end`$_i$ `<= 10`^{9}
- `end`$_i$ `< start`$_{i+1}$
- `0 <= start`$_j$ `< end`$_j$ `<= 10`^{9}
- `end`$_j$ `< start`$_{j+1}$

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Two lists of closed intervals are each sorted and internally disjoint; we want every intersection. Two pointers hold the current intervals. Their overlap is $[\max(s_1,s_2),\min(e_1,e_2)]$ and is recorded when nonempty. Advance the pointer whose interval ends first.

Python 3

```
class Solution:
    def intervalIntersection(
        self, firstList: List[List[int]], secondList: List[List[int]]
    ) -> List[List[int]]:
        i = j = 0
        ans = []
        while i < len(firstList) and j < len(secondList):
            s1, e1, s2, e2 = *firstList[i], *secondList[j]
            l, r = max(s1, s2), min(e1, e2)
            if l <= r:
                ans.append([l, r])
            if e1 < e2:
                i += 1
            else:
                j += 1
        return ans
```

Complexity. Time: O(m + n). Space: O(1) extra (the output is O(m + n)).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The UNION of two sorted interval lists
# 2. The intersection of K lists


# ============================================================
# FOLLOW-UP 1: The UNION of two sorted interval lists
# ============================================================

# ---- follow-up 1: function ----
# Answer: Merge the two lists by start and join
# overlapping intervals.
# Time: O((a + b) log (a + b)).  Space: O(a + b).
def union_of_two_lists(a, b):
    merged = []
    for start, end in sorted(a + b):
        if merged and start <= merged[-1][1]:
            if end > merged[-1][1]:
                merged[-1][1] = end
        else:
            merged.append([start, end])
    return merged


# ============================================================
# FOLLOW-UP 2: The intersection of K lists
# ============================================================

# ---- follow-up 2: helper functions ----
def intersect_two(a, b):
    result = []
    i = j = 0
    while i < len(a) and j < len(b):
        low = a[i][0] if a[i][0] > b[j][0] else b[j][0]
        high = a[i][1] if a[i][1] < b[j][1] else b[j][1]
        if low <= high:
            result.append([low, high])
        if a[i][1] < b[j][1]:
            i += 1
        else:
            j += 1
    return result

# ---- follow-up 2: function ----
# Answer: Intersect the first two lists, then intersect
```

```
# the result with the next list, and so on.
# Time: O(K * N), N = total size of all lists.  Space:
# O(N).
def intersect_many(lists):
    result = lists[0]
    for other in lists[1:]:
        result = intersect_two(result, other)
    return result
```

PATTERN 20 · INTERVALS (MERGE / INSERT / OVERLAP) · PROBLEM 6 OF 20

729. My Calendar I

Medium · Design, Segment Tree, Array, Binary Search, Ordered Set

leetcode.com/problems/my-calendar-i

Also appears in pattern 40 Segment Tree

PROBLEM

You are implementing a program to use as your calendar. We can add a new event if adding the event will not cause a **double booking**.

A **double booking** happens when two events have some non-empty intersection (i.e., some moment is common to both events.).

The event can be represented as a pair of integers `startTime` and `endTime` that represents a booking on the half-open interval `[startTime, endTime)`, the range of real numbers `x` such that `startTime <= x < endTime`.

Implement the `MyCalendar` class:

- `MyCalendar()` Initializes the calendar object.
- `boolean book(int startTime, int endTime)` Returns `true` if the event can be added to the calendar successfully without causing a **double booking**. Otherwise, return `false` and do not add the event to the calendar.

Example 1:

```
Input
["MyCalendar", "book", "book", "book"]
[[], [10, 20], [15, 25], [20, 30]]
Output
[null, true, false, true]

Explanation
MyCalendar myCalendar = new MyCalendar();
myCalendar.book(10, 20); // return True
myCalendar.book(15, 25); // return False, It can not be booked because time 15 is already booked by
another event.
myCalendar.book(20, 30); // return True, The event can be booked, as the first event takes every time
less than 20, but not including 20.
```

Constraints:

- `0 <= start < end <= 10^9`
- At most `1000` calls will be made to `book`.

SOLUTION (PYTHON 3)

Solution 1: Ordered Set

> **How to think about it**
>
> Bookings must not overlap; endpoints reach 10^9 and there are about 1000 calls. A linear scan works, but an ordered map finds the conflict in a log. Sorted by end time, the only interval that can meet [start,end) is the first whose end is greater than start. If that start is still less than end, they overlap. Store end↦start. bisect_right(start) yields that candidate; insert on success. O(log n) per book.

We can use an ordered set to store the schedule. An ordered set can perform insert, delete, and search operations in O(log n) time. The elements in the ordered set are sorted by the endTime of the schedule in ascending order.

When calling the book(start, end) method, we search for the first schedule in the ordered set with an end time greater than start. If it exists and its start time is less than end, it means there is a double booking, and we return false. Otherwise, we insert end as the key and start as the value into the ordered set and return true.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the number of schedules.

Python 3

```python
class MyCalendar:

    def __init__(self):
        self.sd = SortedDict()

    def book(self, start: int, end: int) -> bool:
        idx = self.sd.bisect_right(start)
        if idx < len(self.sd) and self.sd.values()[idx] < end:
            return False
        self.sd[end] = start
        return True


# Your MyCalendar object will be instantiated and called as such:
# obj = MyCalendar()
# param_1 = obj.book(start,end)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Cancel a booking
# 2. Find the earliest free slot of a given length after a
# given time


# ============================================================
# FOLLOW-UP 1: Cancel a booking
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the booked intervals in a list; cancelling
# removes the matching interval.
# Time: book O(n), cancel O(n).  Space: O(n).
class CalendarWithCancel:
    def __init__(self):
        self.booked = []

    def book(self, start, end):
        for s, e in self.booked:
            if start < e and s < end:
                return False
        self.booked.append((start, end))
        return True

    def cancel(self, start, end):
        if (start, end) in self.booked:
            self.booked.remove((start, end))
            return True
        return False


# ============================================================
# FOLLOW-UP 2: Find the earliest free slot of a given
# length after a given time
# ============================================================

# ---- follow-up 2: function ----
# Answer: Sort the bookings and try the time right after
# each booking (and the requested time itself).
# Time: O(n log n).  Space: O(1) extra.
def earliest_free_slot(booked, length, not_before):
    candidate = not_before
    for start, end in sorted(booked):
```

```
        if end <= candidate:
            continue
        if start >= candidate + length:
            break
        candidate = end
    return candidate
```

PATTERN 20 · INTERVALS (MERGE / INSERT / OVERLAP) · PROBLEM 7 OF 20

731. My Calendar II

Medium · Design, Segment Tree, Array, Binary Search, Ordered Set, Prefix Sum

leetcode.com/problems/my-calendar-ii

Also appears in pattern 40 Segment Tree

PROBLEM

You are implementing a program to use as your calendar. We can add a new event if adding the event will not cause a **triple booking**.

A **triple booking** happens when three events have some non-empty intersection (i.e., some moment is common to all the three events.).

The event can be represented as a pair of integers `startTime` and `endTime` that represents a booking on the half-open interval `[startTime, endTime)`, the range of real numbers `x` such that `startTime <= x < endTime`.

Implement the `MyCalendarTwo` class:

- `MyCalendarTwo()` Initializes the calendar object.
- `boolean book(int startTime, int endTime)` Returns `true` if the event can be added to the calendar successfully without causing a **triple booking**. Otherwise, return `false` and do not add the event to the calendar.

Example 1:

```
Input
["MyCalendarTwo", "book", "book", "book", "book", "book", "book"]
[[], [10, 20], [50, 60], [10, 40], [5, 15], [5, 10], [25, 55]]
Output
[null, true, true, true, false, true, true]

Explanation
MyCalendarTwo myCalendarTwo = new MyCalendarTwo();
myCalendarTwo.book(10, 20); // return True, The event can be booked.
myCalendarTwo.book(50, 60); // return True, The event can be booked.
myCalendarTwo.book(10, 40); // return True, The event can be double booked.
myCalendarTwo.book(5, 15); // return False, The event cannot be booked, because it would result in a
triple booking.
myCalendarTwo.book(5, 10); // return True, The event can be booked, as it does not use time 10 which is
already double booked.
myCalendarTwo.book(25, 55); // return True, The event can be booked, as the time in [25, 40) will be
double booked with the third event, the time [40, 50) will be single booked, and the time [50, 55) will
be double booked with the second event.
```

Constraints:

- $0 <= start < end <= 10^9$
- At most `1000` calls will be made to `book`.

SOLUTIONS (PYTHON 3)

Solution 1: Difference Array

How to think about it

Double booking is allowed, triple is not. Endpoints reach 10^9 with 1000 calls. Checking every pair plus a second list of doubles is clumsy. Coverage is a difference array: +1 at start, -1 at end. A prefix scan over sorted times rejects a booking if the sum exceeds 2, then rolls the two updates back. A sorted map stands in for the huge timeline. Each book scans the keys in O(n), total $O(n^2)$, which is fine for 1000 calls.

We can use the concept of a difference array to record the booking status at each time point. Then, we traverse all the time points and count the booking status at the current time point. If the number of bookings exceeds 2, we return false. Otherwise, we return true.

The time complexity is $O(n^2)$, and the space complexity is $O(n)$, where n is the number of bookings.

Python 3

```
class MyCalendarTwo:

    def __init__(self):
        self.sd = SortedDict()

    def book(self, startTime: int, endTime: int) -> bool:
        self.sd[startTime] = self.sd.get(startTime, 0) + 1
        self.sd[endTime] = self.sd.get(endTime, 0) - 1
        s = 0
        for v in self.sd.values():
            s += v
            if s > 2:
                self.sd[startTime] -= 1
                self.sd[endTime] += 1
                return False
        return True


# Your MyCalendarTwo object will be instantiated and called as such:
# obj = MyCalendarTwo()
# param_1 = obj.book(startTime,endTime)
```

Solution 2: Segment Tree

> **How to think about it**
>
> Solution 1 rescans every breakpoint. Range add plus range-max is a lazy segment tree. The universe is 10^9, so nodes are created on demand and store the max coverage. Query [start,end); reject if the max is already 2, otherwise add one. O(log n) per book.

A segment tree divides the entire interval into multiple non-contiguous subintervals, with the number of subintervals not exceeding log(width). To update the value of an element, only log(width) intervals need to be updated, and these intervals are all contained within a larger interval that includes the element. When modifying intervals, a **lazy mark** is used to ensure efficiency.

- Each node of the segment tree represents an interval;
- The segment tree has a unique root node representing the entire statistical range, such as [1, N];
- Each leaf node of the segment tree represents a unit interval of length 1, [x, x];
- For each internal node [l, r], its left child is [l, mid] and its right child is [mid + 1, r], where mid = floor((l + r) / 2) (i.e., floor division).

For this problem, the segment tree nodes maintain the following information:

1. The maximum number of bookings within the interval v
2. Lazy mark add

Since the time range is 10^9, which is very large, we use dynamic node creation.

The time complexity is $O(n \times \log n)$, and the space complexity is $O(n)$. Here, n is the number of bookings.

Python 3

```
class Node:
    def __init__(self, l: int, r: int):
        self.left = None
        self.right = None
        self.l = l
        self.r = r
        self.mid = (l + r) >> 1
        self.v = 0
        self.add = 0


class SegmentTree:
    def __init__(self):
        self.root = Node(1, 10**9 + 1)

    def modify(self, l: int, r: int, v: int, node: Node = None):
        if l > r:
            return
        if node is None:
            node = self.root
        if node.l >= l and node.r <= r:
            node.v += v
            node.add += v
            return
        self.pushdown(node)
        if l <= node.mid:
            self.modify(l, r, v, node.left)
        if r > node.mid:
            self.modify(l, r, v, node.right)
        self.pushup(node)

    def query(self, l: int, r: int, node: Node = None) -> int:
        if l > r:
            return 0
        if node is None:
            node = self.root
        if node.l >= l and node.r <= r:
            return node.v
        self.pushdown(node)
        v = 0
        if l <= node.mid:
            v = max(v, self.query(l, r, node.left))
        if r > node.mid:
            v = max(v, self.query(l, r, node.right))
        return v

    def pushup(self, node: Node):
        node.v = max(node.left.v, node.right.v)

    def pushdown(self, node: Node):
        if node.left is None:
            node.left = Node(node.l, node.mid)
        if node.right is None:
            node.right = Node(node.mid + 1, node.r)
        if node.add:
            node.left.v += node.add
            node.right.v += node.add
            node.left.add += node.add
            node.right.add += node.add
            node.add = 0
```

```
class MyCalendarTwo:
    def __init__(self):
        self.tree = SegmentTree()
```

```
    def book(self, startTime: int, endTime: int) -> bool:
        if self.tree.query(startTime + 1, endTime) >= 2:
            return False
        self.tree.modify(startTime + 1, endTime, 1)
        return True


# Your MyCalendarTwo object will be instantiated and called as such:
# obj = MyCalendarTwo()
# param_1 = obj.book(startTime,endTime)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. At most K bookings may overlap (a K-booking calendar)
# 2. How many bookings cover a given time?


# ============================================================
# FOLLOW-UP 1: At most K bookings may overlap (a K-booking
# calendar)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep a sorted map of changes (+1 at the start,
# -1 at the end); a new booking is accepted when no time
# inside it would reach k + 1 overlapping bookings.
# Time: O(n log n) per booking (copies and sorts the
# change map).  Space: O(n).
class KBookingCalendar:
    def __init__(self, k):
        self.k = k
        self.changes = {}

    def book(self, start, end):
        trial = dict(self.changes)
        trial[start] = trial.get(start, 0) + 1
        trial[end] = trial.get(end, 0) - 1
        active = 0
        for time in sorted(trial):
            active += trial[time]
            if active > self.k:
                return False
        self.changes = trial
        return True


# ============================================================
# FOLLOW-UP 2: How many bookings cover a given time?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Add up the changes up to that time.
# Time: O(n log n).  Space: O(1) extra.
def bookings_at(changes, time):
    active = 0
    for t in sorted(changes):
        if t > time:
            break
        active += changes[t]
    return active
```

PATTERN 20 · INTERVALS (MERGE / INSERT / OVERLAP) · PROBLEM 8 OF 20

1094. Car Pooling

Medium · Array, Prefix Sum, Sorting, Simulation, Heap (Priority Queue)

leetcode.com/problems/car-pooling

★ **TOP 7 PROBLEM** in Intervals (Merge / Insert / Overlap): master this one first

PROBLEM

There is a car with `capacity` empty seats. The vehicle only drives east (i.e., it cannot turn around and drive west).

You are given the integer `capacity` and an array `trips` where `trips[i] = [numPassengers`$_i$`, from`$_i$`, to`$_i$`]` indicates that the i^{th} trip has `numPassengers`$_i$ passengers and the locations to pick them up and drop them off are `from`$_i$ and `to`$_i$ respectively. The locations are given as the number of kilometers due east from the car's initial location.

Passengers are dropped off before new passengers are picked up at the same location. At every point along the route, the total number of passengers in the car must not exceed `capacity`.

Return `true` if it is possible to pick up and drop off all passengers for all the given trips, or `false` otherwise.

Example 1:

```
Input: trips = [[2,1,5],[3,3,7]], capacity = 4
Output: false
Explanation:
At kilometer 1, 2 passengers are picked up, so the car holds 2.
At kilometer 3, 3 more are picked up, so the car holds 5.
Since 5 > capacity = 4, the trips cannot all be completed.
```

Example 2:

```
Input: trips = [[2,1,5],[3,3,7]], capacity = 5
Output: true
Explanation:
At kilometer 1, the car holds 2 passengers.
At kilometer 3, the car holds 5 passengers.
At kilometer 5, the first 2 are dropped off, so the car holds 3.
At kilometer 7, the last 3 are dropped off, so the car holds 0.
The maximum occupancy is 5, which never exceeds capacity = 5.
```

Constraints:

- `1 <= trips.length <= 1000`
- `trips[i].length == 3`
- `1 <= numPassengers`$_i$ `<= 100`
- `0 <= from`$_i$ `< to`$_i$ `<= 1000`
- `1 <= capacity <= 10`^{5}

SOLUTION (PYTHON 3)

Solution 1: Difference Array

How to think about it

Each trip adds passengers on [from,to). We need the load never to exceed capacity. Locations are at most 1000, so a difference array is enough. Add at the start, subtract at the end, then prefix-sum and test every position against capacity. The array runs to the latest drop-off; empty stops keep the previous load.

We can use the idea of a difference array, adding the number of passengers to the starting point of each trip and subtracting from the end point. Finally, we just need to check whether the prefix sum of the difference array does not exceed the maximum passenger capacity of the car.

The time complexity is O(n), and the space complexity is O(M). Here, n is the number of trips, and M is the maximum end point in the trips. In this problem, M ≤ 1000.

Python 3

```
class Solution:
    def carPooling(self, trips: List[List[int]], capacity: int) -> bool:
        mx = max(e[2] for e in trips)
        d = [0] * (mx + 1)
        for x, f, t in trips:
            d[f] += x
            d[t] -= x
        return all(s <= capacity for s in accumulate(d))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. When is the car fullest, and how many passengers does
# it carry then?
# 2. The stops are numbered 0 to 1000: use a difference
# array


# ============================================================
# FOLLOW-UP 1: When is the car fullest, and how many
# passengers does it carry then?
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sweep the stops in order with a running count
# and remember the biggest value and the stop where it
# happened.
# Time: O(n log n).  Space: O(n).
def peak_load(trips):
    changes = {}
    for passengers, start, end in trips:
        changes[start] = changes.get(start, 0) + passengers
        changes[end] = changes.get(end, 0) - passengers
    load = 0
    best = (0, None)
    for stop in sorted(changes):
        load += changes[stop]
        if load > best[0]:
            best = (load, stop)
    return best


# ============================================================
# FOLLOW-UP 2: The stops are numbered 0 to 1000: use a
# difference array
# ============================================================

# ---- follow-up 2: function ----
# Answer: Add the passengers at the pick-up stop and
# subtract them at the drop-off stop in an array; a
# running sum gives the load at every stop.
# Time: O(n + 1001).  Space: O(1001).
def car_pooling_difference_array(trips, capacity):
    diff = [0] * 1002
    for passengers, start, end in trips:
        diff[start] += passengers
        diff[end] -= passengers
    load = 0
    for change in diff:
        load += change
        if load > capacity:
            return False
```

```
    return True
```

1353. Maximum Number of Events That Can Be Attended

Medium · Greedy, Array, Sorting, Heap (Priority Queue)

leetcode.com/problems/maximum-number-of-events-that-can-be-attended

Also appears in pattern 21 Greedy

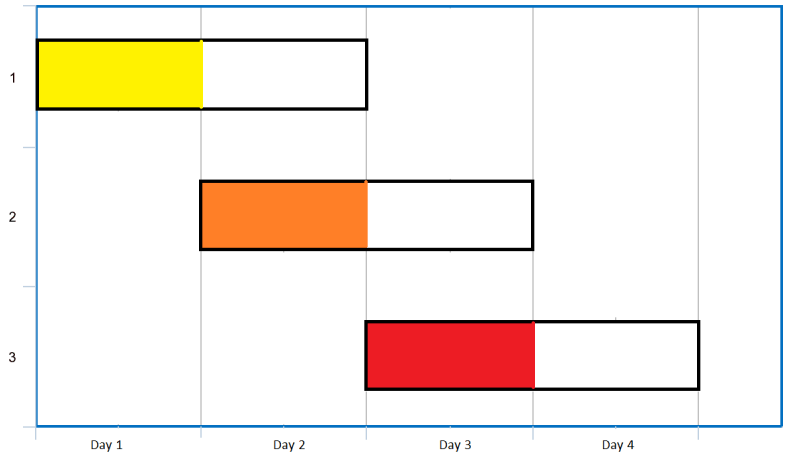
PROBLEM

You are given an array of `events` where `events[i] = [startDay`$_i$`, endDay`$_i$`]`. Every event `i` starts at `startDay`$_i$ and ends at `endDay`$_i$.

You can attend an event `i` at any day `d` where `startDay`$_i$ `<= d <=` `endDay`$_i$. You can only attend one event at any time `d`.

Return *the maximum number of events you can attend*.

Example 1:




```
Input: events = [[1,2],[2,3],[3,4]]
Output: 3
Explanation: You can attend all the three events.
One way to attend them all is as shown.
Attend the first event on day 1.
Attend the second event on day 2.
Attend the third event on day 3.
```

Example 2:

```
Input: events= [[1,2],[2,3],[3,4],[1,2]]
Output: 4
```

Constraints:

- `1 <= events.length <=` 10^5
- `events[i].length == 2`
- `1 <= startDay`$_i$ `<= endDay`$_i$ `<=` 10^5

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Greedy + Priority Queue

How to think about it

One event per day, each event occupying any day in [s,e]. n and the day range reach 10^5, so sorting by end and scanning days naively is tight. Sweep by day: push endings that start today into a min-heap, drop expired ones, and attend the earliest-ending event. Each day is given to the most urgent remaining event.

We use a hash table g to record the start and end times of each event. The key is the start time of the event, and the value is a list containing the end times of all events that start at that time. Two variables, l and r, are used to record the minimum start time and the maximum end time among all events.

For each time point s from l to r in increasing order, we perform the following steps:

1. Remove from the priority queue all events whose end time is less than the current time s.
2. Add the end times of all events that start at the current time s to the priority queue.
3. If the priority queue is not empty, take out the event with the earliest end time, increment the answer count, and remove this event from the priority queue.

In this way, we ensure that at each time point s, we always attend the event that ends the earliest, thus maximizing the number of events attended.

The time complexity is O(M × log n), and the space complexity is O(n), where M is the maximum end time and n is the number of events.

Python 3

```python
class Solution:
    def maxEvents(self, events: List[List[int]]) -> int:
        g = defaultdict(list)
        l, r = inf, 0
        for s, e in events:
            g[s].append(e)
            l = min(l, s)
            r = max(r, e)
        pq = []
        ans = 0
        for s in range(l, r + 1):
            while pq and pq[0] < s:
                heappop(pq)
            for e in g[s]:
                heappush(pq, e)
            if pq:
                heappop(pq)
                ans += 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return which event is attended on which day


# ============================================================
# FOLLOW-UP 1: Return which event is attended on which day
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same greedy with a heap of end days; record the
# event index with it.
# Time: O(n log n).  Space: O(n).
import heapq
```

```
def event_schedule(events):
    order = sorted(range(len(events)), key=lambda i: events[i][0])
    heap = []
    schedule = {}
    position = 0
    day = 0
    while position < len(order) or heap:
        if not heap:
            day = events[order[position]][0]
        while position < len(order) and events[order[position]][0] <= day:
            i = order[position]
            heapq.heappush(heap, (events[i][1], i))
            position += 1
        while heap and heap[0][0] < day:
            heapq.heappop(heap)
        if heap:
            end, i = heapq.heappop(heap)
            schedule[day] = i
            day += 1
    return schedule
```

1024. Video Stitching

Medium · Greedy, Array, Dynamic Programming

leetcode.com/problems/video-stitching

PROBLEM

You are given a series of video clips from a sporting event that lasted `time` seconds. These video clips can be overlapping with each other and have varying lengths.

Each video clip is described by an array `clips` where `clips[i] = [start_i, end_i]` indicates that the ith clip started at `start_i` and ended at `end_i`.

We can cut these clips into segments freely.

- For example, a clip `[0, 7]` can be cut into segments `[0, 1] + [1, 3] + [3, 7]`.

Return *the minimum number of clips needed so that we can cut the clips into segments that cover the entire sporting event* `[0, time]`. If the task is impossible, return `-1`.

Example 1:

```
Input: clips = [[0,2],[4,6],[8,10],[1,9],[1,5],[5,9]], time = 10
Output: 3
Explanation: We take the clips [0,2], [8,10], [1,9]; a total of 3 clips.
Then, we can reconstruct the sporting event as follows:
We cut [1,9] into segments [1,2] + [2,8] + [8,9].
Now we have segments [0,2] + [2,8] + [8,10] which cover the sporting event [0, 10].
```

Example 2:

```
Input: clips = [[0,1],[1,2]], time = 5
Output: -1
Explanation: We cannot cover [0,5] with only [0,1] and [1,2].
```

Example 3:

```
Input: clips =
[[0,1],[6,8],[0,2],[5,6],[0,4],[0,3],[6,7],[1,3],[4,7],[1,4],[2,5],[2,6],[3,4],[4,5],[5,7],[6,9]],
time = 9
Output: 3
Explanation: We can take clips [0,4], [4,7], and [6,9].
```

Constraints:

- `1 <= clips.length <= 100`
- `0 <= start_i <= end_i <= 100`
- `1 <= time <= 100`

SOLUTION (PYTHON 3)

Solution 1: Greedy

How to think about it

Both the number of clips and the time bound are at most 100, so a subset search could cover [0,time], but there are exponentially many subsets. For a fixed start we only need the clip with the farthest end, which is the jump-game setting. Store the farthest end last[i] of clips starting at i. A left-to-right scan keeps the reachable mx; if mx stalls at i, coverage is impossible. Crossing the previous segment end pre costs one extra clip. The scan yields the minimum number of clips or -1.

Note that if there are multiple sub-intervals with the same starting point, it is optimal to choose the one with the largest right endpoint.

Therefore, we can preprocess all sub-intervals. For each position i, calculate the largest right endpoint among all sub-intervals starting at i, and record it in the array last[i].

We define a variable `mx` to represent the farthest position that can currently be reached, a variable `ans` to represent the current minimum number of sub-intervals needed, and a variable `pre` to represent the right endpoint of the last used sub-interval.

Next, we start enumerating all positions i from 0, using last[i] to update `mx`. If after updating, mx = i, it means that the next position cannot be covered, so the task cannot be completed, return -1.

At the same time, we record the right endpoint `pre` of the last used sub-interval. If pre = i, it means that a new sub-interval needs to be used, so we add 1 to `ans` and update `pre` to `mx`.

After the traversal is over, return `ans`.

The time complexity is O(n+m), and the space complexity is O(m). Where n and m are the lengths of the `clips` array and the value of `time`, respectively.

Similar problems:

- 45. Jump Game II
- 55. Jump Game
- 1326. Minimum Number of Taps to Open to Water a Garden

Python 3

```
class Solution:
    def videoStitching(self, clips: List[List[int]], time: int) -> int:
        last = [0] * time
        for a, b in clips:
            if a < time:
                last[a] = max(last[a], b)
        ans = mx = pre = 0
        for i, v in enumerate(last):
            mx = max(mx, v)
            if mx <= i:
                return -1
            if pre == i:
                ans += 1
                pre = mx
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the clips that are used


# ============================================================
# FOLLOW-UP 1: Return the clips that are used
# ============================================================

# ---- follow-up 1: function ----
# Answer: Greedy: at each step take, among the clips
# starting inside the covered range, the one reaching
# farthest; remember it.
# Time: O(n log n).  Space: O(n).
def clips_used(clips, time):
    chosen = []
    covered = 0
    farthest = 0
    best = None
    ordered = sorted(clips)
```

```
    i = 0
    while covered < time:
        while i < len(ordered) and ordered[i][0] <= covered:
            if ordered[i][1] > farthest:
                farthest = ordered[i][1]
                best = ordered[i]
            i += 1
        if farthest <= covered:
            return None
        chosen.append(best)
        covered = farthest
    return chosen
```

1288. Remove Covered Intervals

Medium · Array, Sorting

leetcode.com/problems/remove-covered-intervals

PROBLEM

Given an array `intervals` where `intervals[i] = [l_i, r_i]` represent the interval `[l_i, r_i)`, remove all intervals that are covered by another interval in the list.

The interval `[a, b)` is covered by the interval `[c, d)` if and only if `c <= a` and `b <= d`.

Return *the number of remaining intervals*.

Example 1:

```
Input: intervals = [[1,4],[3,6],[2,8]]
Output: 2
Explanation: Interval [3,6] is covered by [2,8], therefore it is removed.
```

Example 2:

```
Input: intervals = [[1,4],[2,3]]
Output: 1
```

Constraints:

- `1 <= intervals.length <= 1000`
- `intervals[i].length == 2`
- `0 <= l_i < r_i <= 10^5`
- All the given intervals are **unique**.

SOLUTION (PYTHON 3)

Solution 1: Sorting

> **How to think about it**
>
> We drop intervals covered by another. Sorting by rising left end and, on ties, falling right end, an earlier interval cannot be covered by a later narrower one. We keep the largest right end seen: a strictly larger current right end is not covered. Sorting reduces 2-D containment to a 1-D right-end test.

We can sort the intervals in ascending order by their left endpoints, and if the left endpoints are the same, sort them in descending order by their right endpoints.

After sorting, we can traverse the intervals. If the right endpoint of the current interval is greater than the previous right endpoint, it means the current interval is not covered, and we increment the answer by one.

The time complexity is O(n × log n), and the space complexity is O(log n). Here, n is the number of intervals.

Python 3

```python
class Solution:
    def removeCoveredIntervals(self, intervals: List[List[int]]) -> int:
        intervals.sort(key=lambda x: (x[0], -x[1]))
        ans = 0
        pre = -inf
        for _, cur in intervals:
            if cur > pre:
                ans += 1
                pre = cur
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the intervals that remain
# 2. Return the intervals that are covered (removed)


# ============================================================
# FOLLOW-UP 1: Return the intervals that remain
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort by start (and longer first on ties) and
# keep an interval only when it ends after every earlier
# one.
# Time: O(n log n).  Space: O(n).
def remaining_intervals(intervals):
    ordered = sorted(intervals, key=lambda x: (x[0], -x[1]))
    kept = []
    farthest = 0
    for start, end in ordered:
        if end > farthest:
            kept.append([start, end])
            farthest = end
    return kept


# ============================================================
# FOLLOW-UP 2: Return the intervals that are covered
# (removed)
# ============================================================

# ---- follow-up 2: function ----
# Answer: The complement of the previous answer.
# Time: O(n log n).  Space: O(n).
def covered_intervals(intervals):
    ordered = sorted(intervals, key=lambda x: (x[0], -x[1]))
    covered = []
    farthest = 0
    for start, end in ordered:
        if end > farthest:
            farthest = end
        else:
            covered.append([start, end])
    return covered
```

PATTERN 20 · INTERVALS (MERGE / INSERT / OVERLAP) · PROBLEM 13 OF 20

436. Find Right Interval

Medium · Array, Binary Search, Sorting

leetcode.com/problems/find-right-interval

Also appears in pattern 37 Two Heaps

PROBLEM

You are given an array of `intervals`, where `intervals[i] = [start_i, end_i]` and each `start_i` is **unique**.

The **right interval** for an interval `i` is an interval `j` such that `start_j >= end_i` and `start_j` is **minimized**. Note that `i` may equal `j`.

Return *an array of **right interval** indices for each interval* `i`. If no **right interval** exists for interval `i`, then put `-1` at index `i`.

Example 1:

```
Input: intervals = [[1,2]]
Output: [-1]
Explanation: There is only one interval in the collection, so it outputs -1.
```

Example 2:

```
Input: intervals = [[3,4],[2,3],[1,2]]
Output: [-1,0,1]
Explanation: There is no right interval for [3,4].
The right interval for [2,3] is [3,4] since start0 = 3 is the smallest start that is >= end1 = 3.
The right interval for [1,2] is [2,3] since start1 = 2 is the smallest start that is >= end2 = 2.
```

Example 3:

```
Input: intervals = [[1,4],[2,3],[3,4]]
Output: [-1,2,-1]
Explanation: There is no right interval for [1,4] and [3,4].
The right interval for [2,3] is [3,4] since start2 = 3 is the smallest start that is >= end1 = 3.
```

Constraints:

- `1 <= intervals.length <= 2 * 10^4`
- `intervals[i].length == 2`
- `-10^6 <= start_i <= end_i <= 10^6`
- The start point of each interval is **unique**.

SOLUTION (PYTHON 3)

Solution 1: Sorting + Binary Search

> **How to think about it**
>
> Each interval wants the leftmost start that is at least its end. A linear scan per end is O(n^2). Sort (start,index) by start and, for every end, binary-search the first start ≥ that end. Sorted starts are monotone, so the first feasible start is the closest one. Storing the original index survives the sort.

We can store the start point and index of each interval into an array `arr`, and sort it by the start point. Then we iterate through the interval array, for each interval `[_, ed]`, we can use binary search to find the first interval whose start point is greater than or equal to `ed`, which is its right-side interval. If found, we store its index into the answer array, otherwise, we store `-1`.

The time complexity is O(n × log n), and the space complexity is O(n). Where n is the length of the intervals.

Python 3

```python
class Solution:
    def findRightInterval(self, intervals: List[List[int]]) -> List[int]:
        n = len(intervals)
        ans = [-1] * n
        arr = sorted((st, i) for i, (st, _) in enumerate(intervals))
        for i, (_, ed) in enumerate(intervals):
            j = bisect_left(arr, (ed, -inf))
            if j < n:
                ans[i] = arr[j][1]
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Find the LEFT interval: the interval with the largest
# end that is at most my start


# ============================================================
# FOLLOW-UP 1: Find the LEFT interval: the interval with
# the largest end that is at most my start
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort the intervals by end and binary search the
# last end that is at most the start.
# Time: O(n log n).  Space: O(n).
import bisect


def find_left_intervals(intervals):
    by_end = sorted((end, i) for i, (start, end) in enumerate(intervals))
    ends = [end for end, i in by_end]
    result = []
    for start, end in intervals:
        position = bisect.bisect_right(ends, start) - 1
        result.append(by_end[position][1] if position >= 0 else -1)
    return result
```

PATTERN 20 · INTERVALS (MERGE / INSERT / OVERLAP) · PROBLEM 14 OF 20

759. Employee Free Time

Hard · Array, Sorting, Sweep Line, Heap (Priority Queue)

leetcode.com/problems/employee-free-time

★ **TOP 7 PROBLEM** in Intervals (Merge / Insert / Overlap): master this one first

PROBLEM

We are given a list `schedule` of employees, which represents the working time for each employee.

Each employee has a list of non-overlapping `Intervals`, and these intervals are in sorted order.

Return the list of finite intervals representing **common, positive-length free time** for *all* employees, also in sorted order.

(Even though we are representing `Intervals` in the form `[x, y]`, the objects inside are `Intervals`, not lists or arrays. For example, `schedule[0][0].start = 1`, `schedule[0][0].end = 2`, and `schedule[0][0][0]` is not defined). Also, we wouldn't include intervals like [5, 5] in our answer, as they have zero length.

Example 1:

```
Input: schedule = [[[1,2],[5,6]],[[1,3]],[[4,10]]]
Output: [[3,4]]
Explanation: There are a total of three employees, and all common
free time intervals would be [-inf, 1], [3, 4], [10, inf].
We discard any intervals that contain inf as they aren't finite.
```

Example 2:

```
Input: schedule = [[[1,3],[6,7]],[[2,4]],[[2,5],[9,12]]]
Output: [[5,6],[7,9]]
```

Constraints:

- `1 <= schedule.length , schedule[i].length <= 50`
- `0 <= schedule[i].start < schedule[i].end <= 10^8`

SOLUTION (PYTHON 3)

Solution 1: Interval Merging

How to think about it

Find gaps when every employee is free. Each person's intervals are sorted, but the global list is not. The times when someone is busy are the union of all intervals; gaps of that union are common free time. Sort by start, merge overlaps, and emit [a.end, b.start) between consecutive merged intervals.

We can merge all employees' working time intervals into a single list, then sort and merge the overlapping intervals. Finally, we traverse the merged interval list to find the free time periods between adjacent intervals.

The time complexity is O(mn log(mn)) and the space complexity is O(mn), where m is the number of employees and n is the number of working intervals per employee.

Python 3

```
"""
# Definition for an Interval.
class Interval:
    def __init__(self, start: int = None, end: int = None):
        self.start = start
        self.end = end
"""


class Solution:
    def employeeFreeTime(self, schedule: "[[Interval]]") -> "[Interval]":
        intervals = []
        for e in schedule:
            intervals.extend(e)
        intervals.sort(key=lambda x: (x.start, x.end))
        merged = [intervals[0]]
        for x in intervals[1:]:
            if merged[-1].end < x.start:
                merged.append(x)
            else:
                merged[-1].end = max(merged[-1].end, x.end)
        ans = []
        for a, b in pairwise(merged):
            ans.append(Interval(a.end, b.start))
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The LONGEST common free time


# ============================================================
# FOLLOW-UP 1: The LONGEST common free time
# ============================================================

# ---- follow-up 1: function ----
# Answer: Merge all the working intervals and look at the
# gaps between them; keep the longest gap.
# Time: O(N log N), N = total intervals.  Space: O(N).
def longest_free_gap(schedules):
    intervals = []
    for employee in schedules:
        for start, end in employee:
            intervals.append((start, end))
    intervals.sort()
    best = None
    covered_until = intervals[0][1]
    for start, end in intervals[1:]:
        if start > covered_until:
            gap = (covered_until, start)
            if best is None or gap[1] - gap[0] > best[1] - best[0]:
                best = gap
        if end > covered_until:
            covered_until = end
    return best
```

PATTERN 20 · INTERVALS (MERGE / INSERT / OVERLAP) · PROBLEM 15 OF 20

732. My Calendar III

Hard · Design, Segment Tree, Binary Search, Ordered Set, Prefix Sum

leetcode.com/problems/my-calendar-iii

★ **TOP 7 PROBLEM** in Intervals (Merge / Insert / Overlap), Segment Tree: master this one first

Also appears in pattern 40 Segment Tree

PROBLEM

A `k`-booking happens when `k` events have some non-empty intersection (i.e., there is some time that is common to all `k` events.)

You are given some events `[startTime, endTime)`, after each given event, return an integer `k` representing the maximum `k`-booking between all the previous events.

Implement the `MyCalendarThree` class:

- `MyCalendarThree()` Initializes the object.
- `int book(int startTime, int endTime)` Returns an integer `k` representing the largest integer such that there exists a `k`-booking in the calendar.

Example 1:

```
Input
["MyCalendarThree", "book", "book", "book", "book", "book", "book"]
[[], [10, 20], [50, 60], [10, 40], [5, 15], [5, 10], [25, 55]]
Output
[null, 1, 1, 2, 3, 3, 3]

Explanation
MyCalendarThree myCalendarThree = new MyCalendarThree();
myCalendarThree.book(10, 20); // return 1
myCalendarThree.book(50, 60); // return 1
myCalendarThree.book(10, 40); // return 2
myCalendarThree.book(5, 15); // return 3
myCalendarThree.book(5, 10); // return 3
myCalendarThree.book(25, 55); // return 3
```

Constraints:

- `0 <= startTime < endTime <= 10^9`
- At most `400` calls will be made to `book`.

SOLUTION (PYTHON 3)

Solution 1: Segment Tree

How to think about it

After each booking we must report the current maximum overlap; at most 400 calls. A difference scan works but still walks every key. This is the same range-add / range-max problem as Calendar II, without the cap at 2. A dynamic segment tree updates the interval and queries the whole line. Nodes store the max and a lazy add. O(log n) per book.

A segment tree divides the entire interval into multiple non-contiguous subintervals, with the number of subintervals not exceeding log(width). To update the value of an element, we only need to update log(width) intervals, and these intervals are all contained within a larger interval that includes the element. When modifying intervals, we use **lazy propagation** to ensure efficiency.

- Each node of the segment tree represents an interval.

- The segment tree has a unique root node representing the entire range, such as [1, N].
- Each leaf node of the segment tree represents a unit interval of length 1, [x, x].
- For each internal node [l, r], its left child is [l, mid] and its right child is [mid + 1, r], where mid = floor((l + r) / 2) (i.e., floor division).

For this problem, the segment tree nodes maintain the following information:

1. The maximum number of times the interval has been booked, v.
2. Lazy propagation marker, add.

Since the time range is 10^9, which is very large, we use dynamic node creation.

The time complexity is O(n × log n), and the space complexity is O(n), where n is the number of bookings.

Python 3

```
class Node:
    def __init__(self, l, r):
        self.left = None
        self.right = None
        self.l = l
        self.r = r
        self.mid = (l + r) >> 1
        self.v = 0
        self.add = 0


class SegmentTree:
    def __init__(self):
        self.root = Node(1, int(1e9 + 1))

    def modify(self, l: int, r: int, v: int, node: Node = None):
        if l > r:
            return
        if node is None:
            node = self.root
        if node.l >= l and node.r <= r:
            node.v += v
            node.add += v
            return
        self.pushdown(node)
        if l <= node.mid:
            self.modify(l, r, v, node.left)
        if r > node.mid:
            self.modify(l, r, v, node.right)
        self.pushup(node)

    def query(self, l: int, r: int, node: Node = None) -> int:
        if l > r:
            return 0
        if node is None:
            node = self.root
        if node.l >= l and node.r <= r:
            return node.v
        self.pushdown(node)
        v = 0
        if l <= node.mid:
            v = max(v, self.query(l, r, node.left))
        if r > node.mid:
            v = max(v, self.query(l, r, node.right))
        return v

    def pushup(self, node: Node):
        node.v = max(node.left.v, node.right.v)

    def pushdown(self, node: Node):
        if node.left is None:
            node.left = Node(node.l, node.mid)
        if node.right is None:
            node.right = Node(node.mid + 1, node.r)
        if node.add:
            node.left.v += node.add
            node.right.v += node.add
            node.left.add += node.add
            node.right.add += node.add
            node.add = 0
```

```
class MyCalendarThree:
    def __init__(self):
        self.tree = SegmentTree()
```

```
    def book(self, start: int, end: int) -> int:
        self.tree.modify(start + 1, end, 1)
        return self.tree.query(1, int(1e9 + 1))


# Your MyCalendarThree object will be instantiated and called as such:
# obj = MyCalendarThree()
# param_1 = obj.book(start,end)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Cancel a booking
# 2. At WHICH time is the overlap the biggest?


# ========================================================
# FOLLOW-UP 1: Cancel a booking
# ========================================================

# ---- follow-up 1: function ----
# Answer: Remove the +1 / -1 changes of that booking from
# the sorted map; the maximum is recomputed by a sweep.
# Time: O(n log n) per book or cancel.  Space: O(n).
class MaxOverlapCalendar:
    def __init__(self):
        self.changes = {}

    def book(self, start, end):
        self.changes[start] = self.changes.get(start, 0) + 1
        self.changes[end] = self.changes.get(end, 0) - 1
        return self.max_overlap()

    def cancel(self, start, end):
        self.changes[start] = self.changes.get(start, 0) - 1
        self.changes[end] = self.changes.get(end, 0) + 1
        return self.max_overlap()

    def max_overlap(self):
        active = 0
        best = 0
        for time in sorted(self.changes):
            active += self.changes[time]
            if active > best:
                best = active
        return best


# ========================================================
# FOLLOW-UP 2: At WHICH time is the overlap the biggest?
# ========================================================

# ---- follow-up 2: function ----
# Answer: The same sweep, remembering the time at which
# the running count first reaches its maximum.
# Time: O(n log n).  Space: O(1) extra.
def busiest_time(changes):
    active = 0
    best = (0, None)
    for time in sorted(changes):
        active += changes[time]
        if active > best[0]:
            best = (active, time)
    return best
```

1851. Minimum Interval to Include Each Query

Hard · Array, Binary Search, Sorting, Sweep Line, Heap (Priority Queue)

leetcode.com/problems/minimum-interval-to-include-each-query

PROBLEM

You are given a 2D integer array `intervals`, where `intervals[i] = [left_i, right_i]` describes the i^{th} interval starting at `left_i` and ending at `right_i` **(inclusive)**. The **size** of an interval is defined as the number of integers it contains, or more formally `right_i - left_i + 1`.

You are also given an integer array `queries`. The answer to the j^{th} query is the **size of the smallest interval** `i` such that `left_i <= queries[j] <= right_i`. If no such interval exists, the answer is `-1`.

Return *an array containing the answers to the queries*.

Example 1:

```
Input: intervals = [[1,4],[2,4],[3,6],[4,4]], queries = [2,3,4,5]
Output: [3,3,1,4]
Explanation: The queries are processed as follows:
- Query = 2: The interval [2,4] is the smallest interval containing 2. The answer is 4 - 2 + 1 = 3.
- Query = 3: The interval [2,4] is the smallest interval containing 3. The answer is 4 - 2 + 1 = 3.
- Query = 4: The interval [4,4] is the smallest interval containing 4. The answer is 4 - 4 + 1 = 1.
- Query = 5: The interval [3,6] is the smallest interval containing 5. The answer is 6 - 3 + 1 = 4.
```

Example 2:

```
Input: intervals = [[2,3],[2,5],[1,8],[20,25]], queries = [2,19,5,22]
Output: [2,-1,4,6]
Explanation: The queries are processed as follows:
- Query = 2: The interval [2,3] is the smallest interval containing 2. The answer is 3 - 2 + 1 = 2.
- Query = 19: None of the intervals contain 19. The answer is -1.
- Query = 5: The interval [2,5] is the smallest interval containing 5. The answer is 5 - 2 + 1 = 4.
- Query = 22: The interval [20,25] is the smallest interval containing 22. The answer is 25 - 20 + 1 =
6.
```

Constraints:

- `1 <= intervals.length <= 10^5`
- `1 <= queries.length <= 10^5`
- `intervals[i].length == 2`
- `1 <= left_i <= right_i <= 10^7`
- `1 <= queries[j] <= 10^7`

SOLUTION (PYTHON 3)

Solution 1: Sorting + Offline Query + Priority Queue (Min Heap)

> **How to think about it**
>
> Each query wants the shortest interval covering that point. Both arrays can have size 10^5, so scanning all intervals per query is impossible. Sort queries offline by coordinate and intervals by left endpoint. A min-heap stores (length,right) of intervals that already started: push those whose left is at most the query, pop those whose right is too small. The heap top is the shortest cover of the current point.

We notice that the order of queries does not affect the answer, and the intervals involved do not change. Therefore, we consider sorting all queries in ascending order, and sorting all intervals in ascending order of the left endpoint.

We use a priority queue (min heap) pq to maintain all current intervals. Each element in the queue is a pair (v, r), representing an interval with length v and right endpoint r. Initially, the priority queue is empty. In addition, we define a pointer i that points to the current interval being traversed, and initially i=0.

We traverse each query (x, j) in ascending order and perform the following operations:

- If the pointer i has not traversed all intervals, and the left endpoint of the current interval [a, b] is less than or equal to x, then we add this interval to the priority queue and move the pointer i one step forward. Repeat this process.
- If the priority queue is not empty, and the right endpoint of the heap top element is less than x, then we pop the heap top element. Repeat this process.
- At this point, if the priority queue is not empty, then the heap top element is the smallest interval containing x. We add its length v to the answer array ans.

After the above process is over, we return the answer array ans.

The time complexity is O(n × log n + m × log m), and the space complexity is O(n + m). Where n and m are the lengths of the arrays `intervals` and `queries` respectively.

Python 3

```python
class Solution:
    def minInterval(self, intervals: List[List[int]], queries: List[int]) -> List[int]:
        n, m = len(intervals), len(queries)
        intervals.sort()
        queries = sorted((x, i) for i, x in enumerate(queries))
        ans = [-1] * m
        pq = []
        i = 0
        for x, j in queries:
            while i < n and intervals[i][0] <= x:
                a, b = intervals[i]
                heappush(pq, (b - a + 1, b))
                i += 1
            while pq and pq[0][1] < x:
                heappop(pq)
            if pq:
                ans[j] = pq[0][0]
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the interval itself for every query


# ============================================================
# FOLLOW-UP 1: Return the interval itself for every query
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same sweep with a heap; keep the interval
# (not only its size) together with its size.
# Time: O((n + q) log (n + q)).  Space: O(n + q).
import heapq


def smallest_interval_for_queries(intervals, queries):
    intervals = sorted(intervals)
    order = sorted(range(len(queries)), key=lambda i: queries[i])
    heap = []
    answer = [None] * len(queries)
    position = 0
    for i in order:
        q = queries[i]
        while position < len(intervals) and intervals[position][0] <= q:
            start, end = intervals[position]
```

```
            heapq.heappush(heap, (end - start + 1, start, end))
            position += 1
        while heap and heap[0][2] < q:
            heapq.heappop(heap)
        if heap:
            answer[i] = [heap[0][1], heap[0][2]]
    return answer
```

PATTERN 20 · INTERVALS (MERGE / INSERT / OVERLAP) · PROBLEM 17 OF 20

715. Range Module

Hard · Design, Segment Tree, Ordered Set

leetcode.com/problems/range-module

★ **TOP 7 PROBLEM** in Segment Tree: master this one first

Also appears in pattern 40 Segment Tree

PROBLEM

A Range Module is a module that tracks ranges of numbers. Design a data structure to track the ranges represented as **half-open intervals** and query about them.

A **half-open interval** `[left, right)` denotes all the real numbers `x` where `left <= x < right`.

Implement the `RangeModule` class:

- `RangeModule()` Initializes the object of the data structure.
- `void addRange(int left, int right)` Adds the **half-open interval** `[left, right)`, tracking every real number in that interval. Adding an interval that partially overlaps with currently tracked numbers should add any numbers in the interval `[left, right)` that are not already tracked.
- `boolean queryRange(int left, int right)` Returns `true` if every real number in the interval `[left, right)` is currently being tracked, and `false` otherwise.
- `void removeRange(int left, int right)` Stops tracking every real number currently being tracked in the **half-open interval** `[left, right)`.

Example 1:

```
Input
["RangeModule", "addRange", "removeRange", "queryRange", "queryRange", "queryRange"]
[[], [10, 20], [14, 16], [10, 14], [13, 15], [16, 17]]
Output
[null, null, null, true, false, true]

Explanation
RangeModule rangeModule = new RangeModule();
rangeModule.addRange(10, 20);
rangeModule.removeRange(14, 16);
rangeModule.queryRange(10, 14); // return True,(Every number in [10, 14) is being tracked)
rangeModule.queryRange(13, 15); // return False,(Numbers like 14, 14.03, 14.17 in [13, 15) are not
being tracked)
rangeModule.queryRange(16, 17); // return True, (The number 16 in [16, 17) is still being tracked,
despite the remove operation)
```

Constraints:

- $1 <= left < right <= 10^9$
- At most 10^4 calls will be made to `addRange`, `queryRange`, and `removeRange`.

SOLUTION (PYTHON 3)

Solution 1: Segment Tree

How to think about it

Maintain add, remove, and covered-query on half-open ranges with endpoints up to 10^9 and about 10^4 operations. Merging an explicit interval list can scan many fragments per update. These are range assign / range query operations. A segment tree handles them in logarithmic time; the universe is too large to allocate fully, so we create nodes on demand and push lazy assign marks. Each node stores whether its segment is fully covered. A fully contained update writes the mark; a query returns v on a covered node, otherwise pushes down and ands the two children. Each call is O(log n).

According to the problem description, we need to maintain a set of intervals, supporting operations of interval addition, deletion, and query. For the addition and deletion of intervals, we can use a segment tree to maintain the set of intervals.

The segment tree divides the entire interval into multiple non-continuous sub-intervals, the number of which does not exceed log(width). To update the value of an element, only log(width) intervals need to be updated, and these intervals are all included in a large interval containing the element. When modifying the interval, we need to use **lazy propagation** to ensure efficiency.

- Each node of the segment tree represents an interval;
- The segment tree has a unique root node, representing the entire statistical range, such as [1,N];
- Each leaf node of the segment tree represents an elementary interval of length 1, [x,x];
- For each internal node [l,r], its left child is [l,mid], and the right child is [mid+1,r], where mid=$\lfloor (l+r)/2 \rfloor$ (rounded down).

Due to the large data range of the problem, we can implement it with a dynamically allocated segment tree. A dynamically allocated segment tree means that we only allocate nodes when needed, instead of allocating all nodes at the beginning. This can save space, but it requires **lazy propagation** to maintain interval modification.

In terms of time complexity, the time complexity of each operation is O(log n). The space complexity is O(m × log n). Here, m is the number of operations, and n is the data range.

Python 3

```
class Node:
    __slots__ = ['left', 'right', 'add', 'v']

    def __init__(self):
        self.left = None
        self.right = None
        self.add = 0
        self.v = False


class SegmentTree:
    __slots__ = ['root']

    def __init__(self):
        self.root = Node()

    def modify(self, left, right, v, l=1, r=int(1e9), node=None):
        if node is None:
            node = self.root
        if l >= left and r <= right:
            if v == 1:
                node.add = 1
                node.v = True
            else:
                node.add = -1
                node.v = False
            return
        self.pushdown(node)
        mid = (l + r) >> 1
        if left <= mid:
            self.modify(left, right, v, l, mid, node.left)
        if right > mid:
            self.modify(left, right, v, mid + 1, r, node.right)
        self.pushup(node)

    def query(self, left, right, l=1, r=int(1e9), node=None):
        if node is None:
            node = self.root
        if l >= left and r <= right:
            return node.v
        self.pushdown(node)
        mid = (l + r) >> 1
        v = True
        if left <= mid:
            v = v and self.query(left, right, l, mid, node.left)
        if right > mid:
            v = v and self.query(left, right, mid + 1, r, node.right)
        return v

    def pushup(self, node):
        node.v = bool(node.left and node.left.v and node.right and node.right.v)

    def pushdown(self, node):
        if node.left is None:
            node.left = Node()
        if node.right is None:
            node.right = Node()
        if node.add:
            node.left.add = node.right.add = node.add
            node.left.v = node.add == 1
```

```
            node.right.v = node.add == 1
            node.add = 0


class RangeModule:
    def __init__(self):
```

```
        self.tree = SegmentTree()

    def addRange(self, left: int, right: int) -> None:
        self.tree.modify(left, right - 1, 1)

    def queryRange(self, left: int, right: int) -> bool:
        return self.tree.query(left, right - 1)

    def removeRange(self, left: int, right: int) -> None:
        self.tree.modify(left, right - 1, -1)


# Your RangeModule object will be instantiated and called as such:
# obj = RangeModule()
# obj.addRange(left,right)
# param_2 = obj.queryRange(left,right)
# obj.removeRange(left,right)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The total length that is currently tracked


# ========================================================
# FOLLOW-UP 1: The total length that is currently tracked
# ========================================================

# ---- follow-up 1: function ----
# Answer: Keep the tracked ranges as a sorted list of
# disjoint intervals; add up their lengths.
# Time: add_range, remove_range, query_range, total_length
# are each O(n).  Space: O(n).
class RangeModuleWithLength:
    def __init__(self):
        self.ranges = []

    def add_range(self, left, right):
        result = []
        placed = False
        for s, e in self.ranges:
            if e < left:
                result.append([s, e])
            elif s > right:
                if not placed:
                    result.append([left, right])
                    placed = True
                result.append([s, e])
            else:
                left = s if s < left else left
                right = e if e > right else right
        if not placed:
            result.append([left, right])
        self.ranges = result

    def remove_range(self, left, right):
        result = []
        for s, e in self.ranges:
            if e <= left or s >= right:
                result.append([s, e])
            else:
                if s < left:
                    result.append([s, left])
                if e > right:
                    result.append([right, e])
        self.ranges = result
```

```
    def query_range(self, left, right):
        for s, e in self.ranges:
            if s <= left and right <= e:
                return True
        return False

    def total_length(self):
        total = 0
        for s, e in self.ranges:
            total += e - s
        return total
```

1326. Minimum Number of Taps to Open to Water a Garden

Hard · Greedy, Array, Dynamic Programming

leetcode.com/problems/minimum-number-of-taps-to-open-to-water-a-garden

Also appears in pattern 21 Greedy

PROBLEM

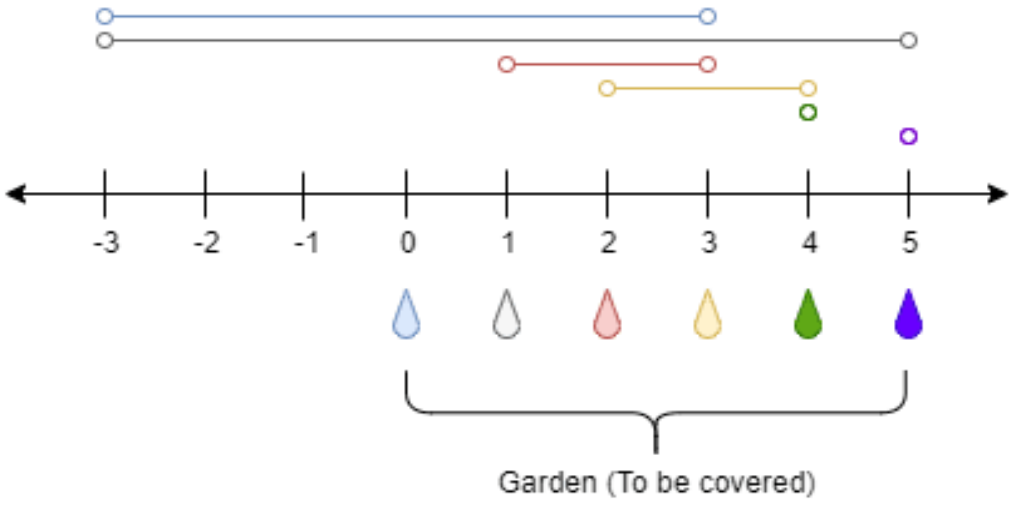
There is a one-dimensional garden on the x-axis. The garden starts at the point `0` and ends at the point `n`. (i.e., the length of the garden is `n`).

There are `n + 1` taps located at points `[0, 1, ..., n]` in the garden.

Given an integer `n` and an integer array `ranges` of length `n + 1` where `ranges[i]` (0-indexed) means the `i-th` tap can water the area `[i - ranges[i], i + ranges[i]]` if it was open.

Return *the minimum number of taps* that should be open to water the whole garden, If the garden cannot be watered return **-1**.

Example 1:




```
Input: n = 5, ranges = [3,4,1,1,0,0]
Output: 1
Explanation: The tap at point 0 can cover the interval [-3,3]
The tap at point 1 can cover the interval [-3,5]
The tap at point 2 can cover the interval [1,3]
The tap at point 3 can cover the interval [2,4]
The tap at point 4 can cover the interval [4,4]
The tap at point 5 can cover the interval [5,5]
Opening Only the second tap will water the whole garden [0,5]
```

Example 2:

```
Input: n = 3, ranges = [0,0,0,0]
Output: -1
Explanation: Even if you activate all the four taps you cannot water the whole garden.
```

Constraints:

- `1 <= n <= 10^4`
- `ranges.length == n + 1`
- `0 <= ranges[i] <= 100`

SOLUTION (PYTHON 3)

Solution 1: Greedy

How to think about it

Each tap covers $[i-r_i, i+r_i]$; we want the fewest taps that cover [0,n]. Subset search is impossible for $n \le 10^4$. Among taps that cover the same left end, the one that reaches farthest right is best, which is the jump-game setting: store in last[l] the farthest right reachable from l. Scanning positions, we keep the current reach mx and the previous segment end pre; hitting pre opens one more tap. If $mx \le i$ at any point, coverage fails.

We note that for all taps that can cover a certain left endpoint, choosing the tap that can cover the farthest right endpoint is optimal.

Therefore, we can preprocess the array ranges. For the i-th tap, it can cover the left endpoint l = max(0, i - ranges[i]) and the right endpoint r = i + ranges[i]. We calculate the position of the tap that can cover the left endpoint l with the farthest right endpoint and record it in the array last[i].

Then we define the following three variables:

- Variable ans represents the final answer, i.e., the minimum number of taps;
- Variable mx represents the farthest right endpoint that can currently be covered;
- Variable pre represents the farthest right endpoint covered by the previous tap.

We traverse all positions in the range [0, …, n-1]. For the current position i, we use last[i] to update mx, i.e., mx = max(mx, last[i]).

- If $mx \le i$, it means the next position cannot be covered, so we return -1.
- If pre = i, it means a new subinterval needs to be used, so we increment ans by 1 and update pre = mx.

After the traversal, we return ans.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the garden.

Similar problems:

- 45. Jump Game II
- 55. Jump Game
- 1024. Video Stitching

Python 3

```python
class Solution:
    def minTaps(self, n: int, ranges: List[int]) -> int:
        last = [0] * (n + 1)
        for i, x in enumerate(ranges):
            l, r = max(0, i - x), i + x
            last[l] = max(last[l], r)

        ans = mx = pre = 0
        for i in range(n):
            mx = max(mx, last[i])
            if mx <= i:
                return -1
            if pre == i:
                ans += 1
                pre = mx
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return which taps are opened


# ============================================================
# FOLLOW-UP 1: Return which taps are opened
# ============================================================

# ---- follow-up 1: function ----
# Answer: Convert every tap to the interval it waters and
# take the greedy farthest-reaching interval each time,
# remembering the tap.
# Time: O(n log n).  Space: O(n).
def taps_to_open(n, ranges):
    reach = []
    for i in range(n + 1):
        left = i - ranges[i] if i - ranges[i] > 0 else 0
        right = i + ranges[i] if i + ranges[i] < n else n
        reach.append((left, right, i))
    reach.sort()
    chosen = []
    covered = 0
    farthest = 0
    best = None
    i = 0
    while covered < n:
        while i < len(reach) and reach[i][0] <= covered:
            if reach[i][1] > farthest:
                farthest = reach[i][1]
                best = reach[i][2]
            i += 1
        if farthest <= covered:
            return None
        chosen.append(best)
        covered = farthest
    return chosen
```

352. Data Stream as Disjoint Intervals

Hard · Union Find, Design, Hash Table, Binary Search, Data Stream, Ordered Set

leetcode.com/problems/data-stream-as-disjoint-intervals

PROBLEM

Given a data stream input of non-negative integers $a_1, a_2, \ldots, a_n$, summarize the numbers seen so far as a list of disjoint intervals.

Implement the `SummaryRanges` class:

- `SummaryRanges()` Initializes the object with an empty stream.
- `void addNum(int value)` Adds the integer `value` to the stream.
- `int[][] getIntervals()` Returns a summary of the integers in the stream currently as a list of disjoint intervals $[start_i, end_i]$. The answer should be sorted by $start_i$.

Example 1:

```
Input
["SummaryRanges", "addNum", "getIntervals", "addNum", "getIntervals", "addNum", "getIntervals",
"addNum", "getIntervals", "addNum", "getIntervals"]
[[], [1], [], [3], [], [7], [], [2], [], [6], []]
Output
[null, null, [[1, 1]], null, [[1, 1], [3, 3]], null, [[1, 1], [3, 3], [7, 7]], null, [[1, 3], [7, 7]],
null, [[1, 3], [6, 7]]]

Explanation
SummaryRanges summaryRanges = new SummaryRanges();
summaryRanges.addNum(1); // arr = [1]
summaryRanges.getIntervals(); // return [[1, 1]]
summaryRanges.addNum(3); // arr = [1, 3]
summaryRanges.getIntervals(); // return [[1, 1], [3, 3]]
summaryRanges.addNum(7); // arr = [1, 3, 7]
summaryRanges.getIntervals(); // return [[1, 1], [3, 3], [7, 7]]
summaryRanges.addNum(2); // arr = [1, 2, 3, 7]
summaryRanges.getIntervals(); // return [[1, 3], [7, 7]]
summaryRanges.addNum(6); // arr = [1, 2, 3, 6, 7]
summaryRanges.getIntervals(); // return [[1, 3], [6, 7]]
```

Constraints:

- $0 <= value <= 10^4$
- At most $3 * 10^4$ calls will be made to `addNum` and `getIntervals`.
- At most 10^2 calls will be made to `getIntervals`.

Follow up: What if there are lots of merges and the number of disjoint intervals is small compared to the size of the data stream?

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Integers from a stream must merge into disjoint intervals. Sorting everything on each insert is heavy. An ordered map keyed by left endpoints lets an insert touch only neighbors. Bisect to the successor and predecessor: fill a two-sided gap by joining three intervals, extend one side if adjacent, otherwise insert a singleton. `getIntervals` lists the sorted values.

Python 3

```python
class SummaryRanges:
    def __init__(self):
        self.mp = SortedDict()

    def addNum(self, val: int) -> None:
        n = len(self.mp)
        ridx = self.mp.bisect_right(val)
        lidx = n if ridx == 0 else ridx - 1
        keys = self.mp.keys()
        values = self.mp.values()
        if (
            lidx != n
            and ridx != n
            and values[lidx][1] + 1 == val
            and values[ridx][0] - 1 == val
        ):
            self.mp[keys[lidx]][1] = self.mp[keys[ridx]][1]
            self.mp.pop(keys[ridx])
        elif lidx != n and val <= values[lidx][1] + 1:
            self.mp[keys[lidx]][1] = max(val, self.mp[keys[lidx]][1])
        elif ridx != n and val >= values[ridx][0] - 1:
            self.mp[keys[ridx]][0] = min(val, self.mp[keys[ridx]][0])
        else:
            self.mp[val] = [val, val]

    def getIntervals(self) -> List[List[int]]:
        return list(self.mp.values())


# # Your SummaryRanges object will be instantiated and called as such:
# # obj = SummaryRanges()
# # obj.addNum(val)
# # param_2 = obj.getIntervals()
```

Complexity. Time: addNum O(log n), getIntervals O(n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Numbers may also be removed from the set
# 2. How many numbers are in the set in total?


# ============================================================
# FOLLOW-UP 1: Numbers may also be removed from the set
# ============================================================

# ---- follow-up 1: function ----
# Answer: Removing a number from an interval splits it in
# two (or shrinks it); keep the intervals in a sorted
# list.
# Time: add, remove and get are each O(m), m = intervals.
# Space: O(m).
class IntervalSet:
    def __init__(self):
        self.intervals = []

    def add(self, value):
        result = []
        placed = False
        start = end = value
        for s, e in self.intervals:
            if e + 1 < value:
                result.append([s, e])
            elif s - 1 > value:
                if not placed:
```

```
                    result.append([start, end])
                    placed = True
                result.append([s, e])
            else:
                start = s if s < start else start
                end = e if e > end else end
        if not placed:
            result.append([start, end])
        self.intervals = result

    def remove(self, value):
        result = []
        for s, e in self.intervals:
            if s <= value <= e:
                if s < value:
                    result.append([s, value - 1])
                if value < e:
                    result.append([value + 1, e])
            else:
                result.append([s, e])
        self.intervals = result

    def get_intervals(self):
        return self.intervals


# ============================================================
# FOLLOW-UP 2: How many numbers are in the set in total?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Add up the sizes of the intervals.
```

```
# Time: O(m).  Space: O(1).
def total_numbers(intervals):
    total = 0
    for start, end in intervals:
        total += end - start + 1
    return total
```

699. Falling Squares

Hard · Segment Tree, Array, Ordered Set

leetcode.com/problems/falling-squares

★ **TOP 7 PROBLEM** in Segment Tree: master this one first

Also appears in pattern 40 Segment Tree

PROBLEM

There are several squares being dropped onto the X-axis of a 2D plane.

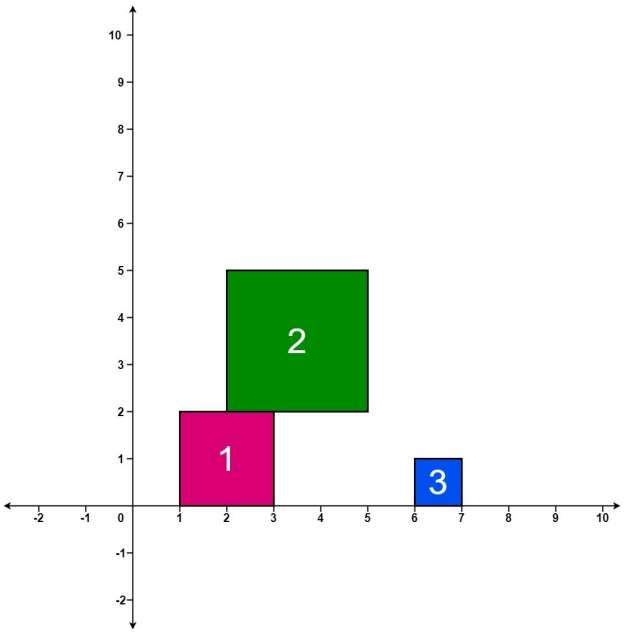
You are given a 2D integer array `positions` where `positions[i] = [left`$_i$`, sideLength`$_i$`]` represents the i^{th} square with a side length of `sideLength`$_i$ that is dropped with its left edge aligned with X-coordinate `left`$_i$.

Each square is dropped one at a time from a height above any landed squares. It then falls downward (negative Y direction) until it either lands **on the top side of another square** or **on the X-axis**. A square brushing the left/right side of another square does not count as landing on it. Once it lands, it freezes in place and cannot be moved.

After each square is dropped, you must record the **height of the current tallest stack of squares**.

Return *an integer array* `ans` *where* `ans[i]` *represents the height described above after dropping the* i^{th} *square*.

Example 1:




```
Input: positions = [[1,2],[2,3],[6,1]]
Output: [2,5,5]
Explanation:
After the first drop, the tallest stack is square 1 with a height of 2.
After the second drop, the tallest stack is squares 1 and 2 with a height of 5.
After the third drop, the tallest stack is still squares 1 and 2 with a height of 5.
Thus, we return an answer of [2, 5, 5].
```

Example 2:

```
Input: positions = [[100,100],[200,100]]
Output: [100,100]
Explanation:
After the first drop, the tallest stack is square 1 with a height of 100.
After the second drop, the tallest stack is either square 1 or square 2, both with heights of 100.
Thus, we return an answer of [100, 100].
Note that square 2 only brushes the right side of square 1, which does not count as landing on it.
```

Constraints:

- `1 <= positions.length <= 1000`
- `1 <= left_i <= 10^8`
- `1 <= sideLength_i <= 10^6`

SOLUTION (PYTHON 3)

Solution 1: Segment Tree

> **How to think about it**
>
> Squares drop onto the current skyline; after each drop we need the global max height. Coordinates reach 10^9, so an array is impossible. A dynamic segment tree stores range maxima: query [l,r] for the landing height, add the side length, and assign that height back with a lazy tag. Track the running maximum.

According to the problem description, we need to maintain a set of intervals that support modification and query operations. In this case, we can use a segment tree to solve the problem.

A segment tree divides the entire interval into multiple non-contiguous sub-intervals, with the number of sub-intervals not exceeding log(width), where width is the length of the interval. To update the value of an element, we only need to update log(width) intervals, and these intervals are all contained within a larger interval that includes the element. When modifying intervals, we need to use **lazy propagation** to ensure efficiency.

- Each node of the segment tree represents an interval;
- The segment tree has a unique root node representing the entire statistical range, such as [1, n];
- Each leaf node of the segment tree represents a primitive interval of length 1, [x, x];
- For each internal node [l, r], its left child is [l, mid], and its right child is [mid + 1, r], where mid = (l + r)/2;

For this problem, the information maintained by the segment tree nodes includes:

1. The maximum height v of the blocks in the interval
2. Lazy propagation marker add

Additionally, since the range of the number line is very large, up to 10^8, we use dynamic node creation.

In terms of time complexity, each query and modification has a time complexity of O(log n), and the total time complexity is O(n log n). The space complexity is O(n).

Python 3

```
class Node:
    def __init__(self, l, r):
        self.left = None
        self.right = None
        self.l = l
        self.r = r
        self.mid = (l + r) >> 1
        self.v = 0
        self.add = 0


class SegmentTree:
    def __init__(self):
        self.root = Node(1, int(1e9))

    def modify(self, l, r, v, node=None):
        if l > r:
            return
        if node is None:
            node = self.root
        if node.l >= l and node.r <= r:
            node.v = v
            node.add = v
            return
        self.pushdown(node)
        if l <= node.mid:
            self.modify(l, r, v, node.left)
        if r > node.mid:
            self.modify(l, r, v, node.right)
        self.pushup(node)

    def query(self, l, r, node=None):
        if l > r:
            return 0
        if node is None:
            node = self.root
        if node.l >= l and node.r <= r:
            return node.v
        self.pushdown(node)
        v = 0
        if l <= node.mid:
            v = max(v, self.query(l, r, node.left))
        if r > node.mid:
            v = max(v, self.query(l, r, node.right))
        return v

    def pushup(self, node):
        node.v = max(node.left.v, node.right.v)

    def pushdown(self, node):
        if node.left is None:
            node.left = Node(node.l, node.mid)
        if node.right is None:
            node.right = Node(node.mid + 1, node.r)
        if node.add:
            node.left.v = node.add
            node.right.v = node.add
            node.left.add = node.add
            node.right.add = node.add
            node.add = 0
```

```
class Solution:
    def fallingSquares(self, positions: List[List[int]]) -> List[int]:
        ans = []
        mx = 0
```

```
        tree = SegmentTree()
        for l, w in positions:
            r = l + w - 1
            h = tree.query(l, r) + w
            mx = max(mx, h)
            ans.append(mx)
            tree.modify(l, r, h)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Only the height of the final pile
# 2. How many squares land directly on the ground?


# ============================================================
# FOLLOW-UP 1: Only the height of the final pile
# ============================================================

# ---- follow-up 1: function ----
# Answer: The last answer of the list: after all squares
# have fallen, the height is the largest top of any
# square.
# Time: O(n^2).  Space: O(n).
def final_height(positions):
    placed = []                                     # (left, right, top)
    highest = 0
    for left, size in positions:
        right = left + size
        base = 0
        for l, r, top in placed:
            if l < right and left < r and top > base:
                base = top
        placed.append((left, right, base + size))
        if base + size > highest:
            highest = base + size
    return highest


# ============================================================
# FOLLOW-UP 2: How many squares land directly on the
# ground?
# ============================================================

# ---- follow-up 2: function ----
# Answer: A square lands on the ground when no earlier
# square overlaps its horizontal range.
# Time: O(n^2).  Space: O(n).
def squares_on_the_ground(positions):
    placed = []
    on_ground = 0
    for left, size in positions:
        right = left + size
        overlaps = False
        for l, r in placed:
            if l < right and left < r:
                overlaps = True
                break
        if not overlaps:
            on_ground += 1
        placed.append((left, right))
    return on_ground
```

PATTERN 21 OF 41

Greedy

20 problems · 2 Easy · 15 Medium · 3 Hard

THE IDEA

Make the best local choice at each step and never look back. It is only right when a local best choice can be proved safe (an exchange argument), so test it on small cases.

Signals: jump game, gas station, assign cookies, partition labels, minimum arrows, interval scheduling, buy and sell stock.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
farthest = 0
for i, jump in enumerate(nums):
    if i > farthest:
        return False                # stuck before reaching i
    farthest = max(farthest, i + jump)
return True
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Lemonade Change	Easy	LC 860	1147
2	Maximum Units on a Truck	Easy	LC 1710	1150
3	★ **Jump Game** (see page)	Medium	LC 55	689
4	★ **Jump Game II**	Medium	LC 45	1153
5	★ **Gas Station**	Medium	LC 134	1156
6	★ **Partition Labels** (see page)	Medium	LC 763	169
7	Task Scheduler (see page)	Medium	LC 621	1052
8	Non-overlapping Intervals (see page)	Medium	LC 435	1094
9	Minimum Number of Arrows to Burst Balloons (see page)	Medium	LC 452	1097
10	★ **Best Time to Buy and Sell Stock II**	Medium	LC 122	1159
11	Two City Scheduling	Medium	LC 1029	1162
12	Boats to Save People (see page)	Medium	LC 881	172
13	Greedy Florist — substitute: Maximum Number of Events That Can Be Attended (see page)	Medium	LC 1353	1112
14	Minimum Add to Make Parentheses Valid (see page)	Medium	LC 921	426
15	Remove K Digits (see page)	Medium	LC 402	436
16	★ **Queue Reconstruction by Height**	Medium	LC 406	1164
17	Wiggle Subsequence (see page)	Medium	LC 376	710
18	★ **Candy**	Hard	LC 135	1166
19	Minimum Number of Taps to Open to Water a Garden (see page)	Hard	LC 1326	1136
20	Course Schedule III	Hard	LC 630	1169

PATTERN 21 · GREEDY · PROBLEM 1 OF 20

860. Lemonade Change

Easy · Greedy, Array

leetcode.com/problems/lemonade-change

PROBLEM

At a lemonade stand, each lemonade costs `$5`. Customers are standing in a queue to buy from you and order one at a time (in the order specified by bills). Each customer will only buy one lemonade and pay with either a `$5`, `$10`, or `$20` bill. You must provide the correct change to each customer so that the net transaction is that the customer pays `$5`.

Note that you do not have any change in hand at first.

Given an integer array `bills` where `bills[i]` is the bill the `i`th customer pays, return `true` *if you can provide every customer with the correct change, or* `false` *otherwise*.

Example 1:

```
Input: bills = [5,5,5,10,20]
Output: true
Explanation:
From the first 3 customers, we collect three $5 bills in order.
From the fourth customer, we collect a $10 bill and give back a $5.
From the fifth customer, we give a $10 bill and a $5 bill.
Since all customers got correct change, we output true.
```

Example 2:

```
Input: bills = [5,5,10,10,20]
Output: false
Explanation:
From the first two customers in order, we collect two $5 bills.
For the next two customers in order, we collect a $10 bill and give back a $5 bill.
For the last customer, we can not give the change of $15 back because we only have two $10 bills.
Since not every customer received the correct change, the answer is false.
```

Constraints:

- `1 <= bills.length <= 10`^{5}
- `bills[i]` is either `5`, `10`, or `20`.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Bills are only 5, 10, and 20, and change must come from cash already taken. n≤ 10^5, so greedy: a 20 prefers one 10 plus one 5, otherwise three 5s. Track counts of 5s and 10s; a negative 5 count fails. The full history is unnecessary.

Python 3

```python
class Solution:
    def lemonadeChange(self, bills: List[int]) -> bool:
        five = ten = 0
        for v in bills:
            if v == 5:
                five += 1
            elif v == 10:
                ten += 1
                five -= 1
            else:
                if ten:
                    ten -= 1
                    five -= 1
                else:
                    five -= 3
            if five < 0:
                return False
        return True
```

Complexity. Time: O(n). Space: O(1).

Solution 2: One-liner

> **How to think about it**
>
> Method 1 already covers every change rule. Method 2 folds the same two counters into one every scan, branching on denomination with xor tests. The meaning is unchanged; only the implementation is a one-liner, still failing when the 5 count goes negative.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```python
class Solution:
    def lemonadeChange(self, bills: List[int]) -> bool:
        five = ten = 0
        for bill in bills:
            if bill == 5:
                five += 1
            elif bill == 10:
                five -= 1
                ten += 1
            elif ten:
                ten -= 1
                five -= 1
            else:
                five -= 3
            if five < 0:
                return False
        return True
```

Complexity. Time: O(n). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the first customer who does not get correct
# change
# 2. Bills of 5, 10, 20 AND 50, with a ticket price of 5


# ========================================================
# FOLLOW-UP 1: Return the first customer who does not get
# correct change
# ========================================================
```

```
# ---- follow-up 1: function ----
# Answer: The same bookkeeping of 5 and 10 bills; report
# the position of the first customer for whom no change
# can be given.
# Time: O(n).  Space: O(1).
def first_failed_customer(bills):
    fives = tens = 0
    for i in range(len(bills)):
        bill = bills[i]
        if bill == 5:
            fives += 1
        elif bill == 10:
            if fives == 0:
                return i
            fives -= 1
            tens += 1
        else:
            if tens > 0 and fives > 0:
                tens -= 1
                fives -= 1
            elif fives >= 3:
                fives -= 3
            else:
                return i
    return -1


# ============================================================
# FOLLOW-UP 2: Bills of 5, 10, 20 AND 50, with a ticket
# price of 5
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep a count of every bill and always give
# change with the biggest bills that fit (greedy works
# because every bill divides the next bigger one that is
# used).
# Time: O(n).  Space: O(1).
def lemonade_with_bigger_bills(bills):
    have = {5: 0, 10: 0, 20: 0, 50: 0}
    for bill in bills:
        change = bill - 5
        for coin in (50, 20, 10, 5):
            while change >= coin and have[coin] > 0:
                change -= coin
                have[coin] -= 1
        if change != 0:
            return False
        have[bill] += 1
```

```
    return True
```

PATTERN 21 · GREEDY · PROBLEM 2 OF 20

1710. Maximum Units on a Truck

Easy · Greedy, Array, Sorting

leetcode.com/problems/maximum-units-on-a-truck

PROBLEM

You are assigned to put some amount of boxes onto **one truck**. You are given a 2D array `boxTypes`, where `boxTypes[i] = [numberOfBoxes`$_i$`, numberOfUnitsPerBox`$_i$`]`:

- `numberOfBoxes`$_i$ is the number of boxes of type `i`.
- `numberOfUnitsPerBox`$_i$ is the number of units in each box of the type `i`.

You are also given an integer `truckSize`, which is the **maximum** number of **boxes** that can be put on the truck. You can choose any boxes to put on the truck as long as the number of boxes does not exceed `truckSize`.

Return *the **maximum** total number of **units** that can be put on the truck.*

Example 1:

```
Input: boxTypes = [[1,3],[2,2],[3,1]], truckSize = 4
Output: 8
Explanation: There are:
- 1 box of the first type that contains 3 units.
- 2 boxes of the second type that contain 2 units each.
- 3 boxes of the third type that contain 1 unit each.
You can take all the boxes of the first and second types, and one box of the third type.
The total number of units will be = (1 * 3) + (2 * 2) + (1 * 1) = 8.
```

Example 2:

```
Input: boxTypes = [[5,10],[2,5],[4,7],[3,9]], truckSize = 10
Output: 91
```

Constraints:

- `1 <= boxTypes.length <= 1000`
- `1 <= numberOfBoxes`$_i$`, numberOfUnitsPerBox`$_i$` <= 1000`
- `1 <= truckSize <= 10`^{6}

SOLUTIONS (PYTHON 3)

Solution 1: Greedy + Sorting

> **How to think about it**
>
> Capacity is measured in boxes, but the objective is units. Boxes of one type share the same unit count, so denser types should be taken first. Sort by units per box descending and load min(truckSize,count) of each type, decreasing remaining capacity until it is gone.

According to the problem, we should choose as many units as possible. Therefore, we first sort `boxTypes` in descending order of the number of units.

Then we traverse `boxTypes` from front to back, choose up to `truckSize` boxes, and accumulate the number of units.

The time complexity is O(n × log n), where n is the length of the two-dimensional array `boxTypes`.

Python 3

```python
class Solution:
    def maximumUnits(self, boxTypes: List[List[int]], truckSize: int) -> int:
        ans = 0
        for a, b in sorted(boxTypes, key=lambda x: -x[1]):
            ans += b * min(truckSize, a)
            truckSize -= a
            if truckSize <= 0:
                break
        return ans
```

Solution 2: Counting Sort

> **How to think about it**
>
> Solution 1 spends O(nlog n) on sorting. Units per box are at most 1000, so a counting array can store box counts by that value. Scan from 1000 down to 1 and load in the same greedy order, now in linear time.

We can also use the idea of counting sort, create an array cnt of length 1001, where cnt[b] represents the number of boxes with b units.

Then starting from the box with the maximum number of units, choose up to `truckSize` boxes, and accumulate the number of units.

The time complexity is O(M), where M is the maximum number of units. In this problem, M=1000.

Python 3

```python
class Solution:
    def maximumUnits(self, boxTypes: List[List[int]], truckSize: int) -> int:
        cnt = [0] * 1001
        for a, b in boxTypes:
            cnt[b] += a
        ans = 0
        for b in range(1000, 0, -1):
            a = cnt[b]
            if a:
                ans += b * min(truckSize, a)
                truckSize -= a
                if truckSize <= 0:
                    break
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return which box types are loaded and how many of
# each
# 2. The truck also has a WEIGHT limit and each box has a
# weight (a knapsack: greedy is no longer enough)


# ============================================================
# FOLLOW-UP 1: Return which box types are loaded and how
# many of each
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort by units per box; take whole box types
# while they fit and a partial last type.
# Time: O(n log n).  Space: O(n).
def loaded_boxes(box_types, truck_size):
    ordered = sorted(box_types, key=lambda b: -b[1])
    loaded = []
    space = truck_size
```

```
    for boxes, units in ordered:
        if space == 0:
            break
        take = boxes if boxes < space else space
        loaded.append((units, take))
        space -= take
    return loaded


# ========================================================
# FOLLOW-UP 2: The truck also has a WEIGHT limit and each
# box has a weight (a knapsack: greedy is no longer
# enough)
# ========================================================

# ---- follow-up 2: function ----
# Answer: Value per box and weight per box mean a two-
# dimensional choice; use a table over (boxes used,
# weight) instead of sorting.
# Time: O(n * T * W), T = truck size, W = weight limit.
# Space: O(T * W).
def max_units_with_weight(boxes, truck_size, max_weight):
    # boxes is a list of (units, weight), each box type given one box at a time
    best = []
    for count in range(truck_size + 1):
        best.append([0] * (max_weight + 1))
    for units, weight in boxes:
        for count in range(truck_size, 0, -1):
            for w in range(max_weight, weight - 1, -1):
                candidate = best[count - 1][w - weight] + units
                if candidate > best[count][w]:
                    best[count][w] = candidate
    answer = 0
    for count in range(truck_size + 1):
        for w in range(max_weight + 1):
            if best[count][w] > answer:
                answer = best[count][w]
    return answer
```

PATTERN 21 · GREEDY · PROBLEM 4 OF 20

45. Jump Game II

Medium · Greedy, Array, Dynamic Programming

leetcode.com/problems/jump-game-ii

★ **TOP 7 PROBLEM** in Greedy: master this one first

PROBLEM

You are given a **0-indexed** array of integers `nums` of length `n`. You are initially positioned at index 0.

Each element `nums[i]` represents the maximum length of a forward jump from index `i`. In other words, if you are at index `i`, you can jump to any index `(i + j)` where:

- `0 <= j <= nums[i]` and
- `i + j < n`

Return *the minimum number of jumps to reach index* `n - 1`. The test cases are generated such that you can reach index `n - 1`.

Example 1:

```
Input: nums = [2,3,1,1,4]
Output: 2
Explanation: The minimum number of jumps to reach the last index is 2. Jump 1 step from index 0 to 1, then 3 steps to the last index.
```

Example 2:

```
Input: nums = [2,3,0,1,4]
Output: 2
```

Constraints:

- `1 <= nums.length <= 10^4`
- `0 <= nums[i] <= 1000`
- It's guaranteed that you can reach `nums[n - 1]`.

SOLUTION (PYTHON 3)

Solution 1: Greedy Algorithm

How to think about it

The first idea is DP: f[i] is the fewest jumps to i, and from i we relax i+1..i+nums[i]. Correct, worst-case $O(n^2)$. $n \le 10^4$ may pass, but we only need the end, not a value at every index. The bottleneck is maintaining exact min jumps per position. Jump k covers a contiguous range; the farthest we can see inside it is the right end of jump k+1. Scan once, tracking this jump's boundary and the farthest the next jump can reach. Hitting the boundary forces a jump. That is BFS by layers without a queue.

We can use a variable mx to record the farthest position that can be reached from the current position, a variable last to record the position of the last jump, and a variable ans to record the number of jumps.

Next, we traverse each position i in [0,..n - 2]. For each position i, we can calculate the farthest position that can be reached from the current position through i + nums[i]. We use mx to record this farthest position, that is, mx = max(mx, i + nums[i]). Then, we check whether the current position has reached the boundary of the last jump, that is, i = last. If it has reached, then we need to make a jump, update last to mx, and increase the number of jumps ans by 1.

Finally, we return the number of jumps ans.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Similar problems:

- 55. Jump Game
- 1024. Video Stitching
- 1326. Minimum Number of Taps to Open to Water a Garden

Python 3

```
class Solution:
    def jump(self, nums: List[int]) -> int:
        ans = mx = last = 0
        for i, x in enumerate(nums[:-1]):
            mx = max(mx, i + x)
            if last == i:
                ans += 1
                last = mx
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the positions you land on
# 2. Count the number of different shortest jump sequences


# ============================================================
# FOLLOW-UP 1: Return the positions you land on
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember, for every position, from where it was
# reached by the fewest jumps (the earliest position that
# can reach it); follow these links back from the last
# position.
# Time: O(n * m), m = largest jump.  Space: O(n).
def jump_path(nums):
    n = len(nums)
    big = n + 1
    steps = [big] * n
    previous = [-1] * n
    steps[0] = 0
    for i in range(n):
        for j in range(i + 1, min(n, i + nums[i] + 1)):
            if steps[i] + 1 < steps[j]:
                steps[j] = steps[i] + 1
                previous[j] = i
    path = []
    position = n - 1
    while position != -1:
        path.append(position)
        position = previous[position]
    path.reverse()
    return path


# ============================================================
# FOLLOW-UP 2: Count the number of different shortest jump
# sequences
# ============================================================

# ---- follow-up 2: function ----
# Answer: Layer by layer: count[i] = the number of
# shortest ways to reach i; add the counts of all
# positions in the previous layer that can reach i.
# Time: O(n * m).  Space: O(n).
def count_shortest_jump_sequences(nums):
    n = len(nums)
```

```
    big = n + 1
    steps = [big] * n
    ways = [0] * n
    steps[0] = 0
    ways[0] = 1
    for i in range(n):
        for j in range(i + 1, min(n, i + nums[i] + 1)):
            if steps[i] + 1 < steps[j]:
                steps[j] = steps[i] + 1
                ways[j] = ways[i]
            elif steps[i] + 1 == steps[j]:
                ways[j] += ways[i]
    return ways[n - 1]
```

PATTERN 21 · GREEDY · PROBLEM 5 OF 20

134. Gas Station

Medium · Greedy, Array

leetcode.com/problems/gas-station

★ **TOP 7 PROBLEM** in Greedy: master this one first

PROBLEM

There are `n` gas stations along a circular route, where the amount of gas at the i^{th} station is `gas[i]`.

You have a car with an unlimited gas tank and it costs `cost[i]` of gas to travel from the i^{th} station to its next $(i + 1)^{th}$ station. You begin the journey with an empty tank at one of the gas stations.

Given two integer arrays `gas` and `cost`, return *the starting gas station's index if you can travel around the circuit once in the clockwise direction, otherwise return* `-1`. If there exists a solution, it is **guaranteed** to be **unique**.

Example 1:

```
Input: gas = [1,2,3,4,5], cost = [3,4,5,1,2]
Output: 3
Explanation:
Start at station 3 (index 3) and fill up with 4 unit of gas. Your tank = 0 + 4 = 4
Travel to station 4. Your tank = 4 - 1 + 5 = 8
Travel to station 0. Your tank = 8 - 2 + 1 = 7
Travel to station 1. Your tank = 7 - 3 + 2 = 6
Travel to station 2. Your tank = 6 - 4 + 3 = 5
Travel to station 3. The cost is 5. Your gas is just enough to travel back to station 3.
Therefore, return 3 as the starting index.
```

Example 2:

```
Input: gas = [2,3,4], cost = [3,4,3]
Output: -1
Explanation:
You can't start at station 0 or 1, as there is not enough gas to travel to the next station.
Let's start at station 2 and fill up with 4 unit of gas. Your tank = 0 + 4 = 4
Travel to station 0. Your tank = 4 - 3 + 2 = 3
Travel to station 1. Your tank = 3 - 3 + 3 = 3
You cannot travel back to station 2, as it requires 4 unit of gas but you only have 3.
Therefore, you can't travel around the circuit once no matter where you start.
```

Constraints:

- `n == gas.length == cost.length`
- `1 <= n <=` 10^5
- `0 <= gas[i], cost[i] <=` 10^4
- The input is generated such that the answer is unique.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

We need a starting station that completes the loop, if any. n≤ 10^5, so simulating from every start is O(n^2). If total gas is less than total cost there is no answer. A negative tank means the start must move earlier to cover that deficit. The code grows the end forward and the start backward from the last station, shifting i left whenever the tank drops below zero. Each station is used once; a non-negative tank at the end is the start.

Python 3

```python
class Solution:
    def canCompleteCircuit(self, gas: List[int], cost: List[int]) -> int:
        n = len(gas)
        i = j = n - 1
        cnt = s = 0
        while cnt < n:
            s += gas[j] - cost[j]
            cnt += 1
            j = (j + 1) % n
            while s < 0 and cnt < n:
                i -= 1
                s += gas[i] - cost[i]
                cnt += 1
        return -1 if s < 0 else i
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. List ALL the starting stations from which the trip
# can be completed
# 2. The smallest amount of gas the car needs in its tank
# at the start of station 0


# ============================================================
# FOLLOW-UP 1: List ALL the starting stations from which
# the trip can be completed
# ============================================================

# ---- follow-up 1: function ----
# Answer: Try each station (O(n squared)); or with prefix
# sums of gas minus cost, a start works when every prefix
# from it stays non-negative around the circle.
# Time: O(n^2).  Space: O(n).
def all_valid_starts(gas, cost):
    n = len(gas)
    starts = []
    for start in range(n):
        tank = 0
        ok = True
        for step in range(n):
            i = (start + step) % n
            tank += gas[i] - cost[i]
            if tank < 0:
                ok = False
                break
        if ok:
            starts.append(start)
    return starts


# ============================================================
# FOLLOW-UP 2: The smallest amount of gas the car needs in
# its tank at the start of station 0
# ============================================================

# ---- follow-up 2: function ----
# Answer: The lowest running balance of (gas - cost) while
# driving from station 0 tells how much extra is needed so
# the tank never drops below 0.
# Time: O(n).  Space: O(1).
def minimum_initial_gas(gas, cost):
    tank = 0
    lowest = 0
    for i in range(len(gas)):
```

```
        tank += gas[i] - cost[i]
        if tank < lowest:
            lowest = tank
    return -lowest
```

PATTERN 21 · GREEDY · PROBLEM 10 OF 20

122. Best Time to Buy and Sell Stock II

Medium · Greedy, Array, Dynamic Programming

leetcode.com/problems/best-time-to-buy-and-sell-stock-ii

★ **TOP 7 PROBLEM** in Greedy: master this one first

PROBLEM

You are given an integer array `prices` where `prices[i]` is the price of a given stock on the i^{th} day.

On each day, you may decide to buy and/or sell the stock. You can only hold **at most one** share of the stock at any time. However, you can sell and buy the stock multiple times on the **same day**, ensuring you never hold more than one share of the stock.

Find and return *the **maximum** profit you can achieve*.

Example 1:

```
Input: prices = [7,1,5,3,6,4]
Output: 7
Explanation: Buy on day 2 (price = 1) and sell on day 3 (price = 5), profit = 5-1 = 4.
Then buy on day 4 (price = 3) and sell on day 5 (price = 6), profit = 6-3 = 3.
Total profit is 4 + 3 = 7.
```

Example 2:

```
Input: prices = [1,2,3,4,5]
Output: 4
Explanation: Buy on day 1 (price = 1) and sell on day 5 (price = 5), profit = 5-1 = 4.
Total profit is 4.
```

Example 3:

```
Input: prices = [7,6,4,3,1]
Output: 0
Explanation: There is no way to make a positive profit, so we never buy the stock to achieve the
maximum profit of 0.
```

Constraints:

- `1 <= prices.length <= 3 * 10^4`
- `0 <= prices[i] <= 10^4`

SOLUTIONS (PYTHON 3)

Solution 1: Greedy Algorithm

How to think about it

We may trade as often as we like, but cannot hold two shares. Summing every adjacent uptick equals buying each valley and selling the next peak. n ≤ 3× 10^4. Walk consecutive days and add positive differences only.

Starting from the second day, if the stock price is higher than the previous day, buy on the previous day and sell on the current day to make a profit. If the stock price is lower than the previous day, do not buy or sell. In other words, buy and sell on all rising trading days, and do not trade on all falling trading days. The final profit will be the maximum.

The time complexity is O(n), where n is the length of the `prices` array. The space complexity is O(1).

Python 3

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        return sum(max(0, b - a) for a, b in pairwise(prices))
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> The greedy form has no explicit state, so it does not extend cleanly to limited transactions or a cooldown. f[i][0/1] is the best profit on day i holding or not; transitions are keep, buy today, stay flat, or sell today. Same answer as Solution 1, more general state.

We define f[i][j] as the maximum profit after trading on the ith day, where j indicates whether we currently hold the stock. When holding the stock, j=0, and when not holding the stock, j=1. The initial state is f[0][0]=-prices[0], and all other states are 0.

If we currently hold the stock, it may be that we held the stock the day before and do nothing today, i.e., f[i][0]=f[i-1][0]. Or it may be that we did not hold the stock the day before and bought the stock today, i.e., f[i][0]=f[i-1][1]-prices[i].

If we currently do not hold the stock, it may be that we did not hold the stock the day before and do nothing today, i.e., f[i][1]=f[i-1][1]. Or it may be that we held the stock the day before and sold the stock today, i.e., f[i][1]=f[i-1][0]+prices[i].

Therefore, we can write the state transition equation as:

$$\begin{cases} f[i][0]=\max(f[i-1][0],f[i-1][1]-prices[i]) \\ f[i][1]=\max(f[i-1][1],f[i-1][0]+prices[i]) \end{cases}$$

The final answer is f[n-1][1], where n is the length of the `prices` array.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the `prices` array.

Python 3

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        n = len(prices)
        f = [[0] * 2 for _ in range(n)]
        f[0][0] = -prices[0]
        for i in range(1, n):
            f[i][0] = max(f[i - 1][0], f[i - 1][1] - prices[i])
            f[i][1] = max(f[i - 1][1], f[i - 1][0] + prices[i])
        return f[n - 1][1]
```

Solution 3: Dynamic Programming (Space Optimization)

> **How to think about it**
>
> Day i in Solution 2 depends only on day i-1's two states, so two rolling variables cut space to O(1).

We can find that in Solution 2, the state of the ith day is only related to the state of the i-1th day. Therefore, we can use only two variables to maintain the state of the i-1th day, thereby optimizing the space complexity to O(1).

The time complexity is O(n), where n is the length of the `prices` array. The space complexity is O(1).

Python 3

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        n = len(prices)
        f = [-prices[0], 0]
        for i in range(1, n):
            g = [0] * 2
            g[0] = max(f[0], f[1] - prices[i])
            g[1] = max(f[1], f[0] + prices[i])
            f = g
        return f[1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the days on which to buy and sell
# 2. Every trade costs a fixed fee: when is a trade still
# worth it?


# ============================================================
# FOLLOW-UP 1: Return the days on which to buy and sell
# ============================================================

# ---- follow-up 1: function ----
# Answer: Every rising stretch is one trade: buy at its
# bottom and sell at its top.
# Time: O(n).  Space: O(k), k = trades.
def trade_days(prices):
    trades = []
    i = 0
    n = len(prices)
    while i < n - 1:
        while i < n - 1 and prices[i + 1] <= prices[i]:
            i += 1
        buy = i
        while i < n - 1 and prices[i + 1] > prices[i]:
            i += 1
        if i > buy:
            trades.append((buy, i))
    return trades


# ============================================================
# FOLLOW-UP 2: Every trade costs a fixed fee: when is a
# trade still worth it?
# ============================================================

# ---- follow-up 2: function ----
# Answer: A rising stretch is only traded when its profit
# is bigger than the fee; the careful way is the two-state
# scan (holding / not holding) that updates after each
# price.
# Time: O(n).  Space: O(1).
def max_profit_with_fee(prices, fee):
    holding = float("-inf")
    free = 0
    for price in prices:
        new_holding = free - price
        if holding > new_holding:
            new_holding = holding
        new_free = holding + price - fee
        if free > new_free:
            new_free = free
        holding, free = new_holding, new_free
    return free
```

PATTERN 21 · GREEDY · PROBLEM 11 OF 20

1029. Two City Scheduling

Medium · Greedy, Array, Hungarian Algorithm, Sorting, SSP

leetcode.com/problems/two-city-scheduling

PROBLEM

A company is planning to interview `2n` people. Given the array `costs` where `costs[i] = [aCost`$_i$`, bCost`$_i$`]`, the cost of flying the `i`th person to city `a` is `aCost`$_i$, and the cost of flying the `i`th person to city `b` is `bCost`$_i$.

Return *the minimum cost to fly every person to a city* such that exactly `n` people arrive in each city.

Example 1:

```
Input: costs = [[10,20],[30,200],[400,50],[30,20]]
Output: 110
Explanation:
The first person goes to city A for a cost of 10.
The second person goes to city A for a cost of 30.
The third person goes to city B for a cost of 50.
The fourth person goes to city B for a cost of 20.

The total minimum cost is 10 + 30 + 50 + 20 = 110 to have half the people interviewing in each city.
```

Example 2:

```
Input: costs = [[259,770],[448,54],[926,667],[184,139],[840,118],[577,469]]
Output: 1859
```

Example 3:

```
Input: costs = [[515,563],[451,713],[537,709],[343,819],[855,779],[457,60],[650,359],[631,42]]
Output: 3086
```

Constraints:

- `2 * n == costs.length`
- `2 <= costs.length <= 100`
- `costs.length` is even.
- `1 <= aCost`$_i$`, bCost`$_i$` <= 1000`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Choosing which n of 2n people fly to A is exponential if done by subsets. Sending everyone to B and then switching n people to A changes the cost by aCost-bCost. The people with the smallest (most negative) differences save the most when switched. Sorting by aCost-bCost and sending the first half to A and the rest to B is optimal. The answer is the corresponding half-and-half sum.

Python 3

```python
class Solution:
    def twoCitySchedCost(self, costs: List[List[int]]) -> int:
        costs.sort(key=lambda x: x[0] - x[1])
        n = len(costs) >> 1
        return sum(costs[i][0] + costs[i + n][1] for i in range(n))
```

Complexity. Time: O(n log n). Space: O(log n) for the sort.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return which people go to city A
# 2. Exactly k people go to city A (not half)


# ============================================================
# FOLLOW-UP 1: Return which people go to city A
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort by the difference (cost A - cost B); the
# first half goes to A, the rest to B.
# Time: O(n log n).  Space: O(n).
def people_to_city_a(costs):
    order = sorted(range(len(costs)), key=lambda i: costs[i][0] - costs[i][1])
    half = len(costs) // 2
    return sorted(order[:half])


# ============================================================
# FOLLOW-UP 2: Exactly k people go to city A (not half)
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same sort by difference; the k people with
# the smallest difference go to A.
# Time: O(n log n).  Space: O(n).
def min_cost_k_to_a(costs, k):
    order = sorted(range(len(costs)), key=lambda i: costs[i][0] - costs[i][1])
    total = 0
    for position in range(len(order)):
        person = order[position]
        if position < k:
            total += costs[person][0]
        else:
            total += costs[person][1]
    return total
```

PATTERN 21 · GREEDY · PROBLEM 16 OF 20

406. Queue Reconstruction by Height

Medium · Binary Indexed Tree, Segment Tree, Array, Sorting

leetcode.com/problems/queue-reconstruction-by-height

★ **TOP 7 PROBLEM** in Greedy, Sorting: master this one first

Also appears in pattern 23 Sorting, 41 Binary Indexed Tree (Fenwick Tree)

PROBLEM

You are given an array of people, `people`, which are the attributes of some people in a queue (not necessarily in order). Each `people[i] = [h_i, k_i]` represents the i^{th} person of height h_i with **exactly** k_i other people in front who have a height greater than or equal to h_i.

Reconstruct and return *the queue that is represented by the input array* `people`. The returned queue should be formatted as an array `queue`, where `queue[j] = [h_j, k_j]` is the attributes of the j^{th} person in the queue (`queue[0]` is the person at the front of the queue).

Example 1:

```
Input: people = [[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]]
Output: [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]]
Explanation:
Person 0 has height 5 with no other people taller or the same height in front.
Person 1 has height 7 with no other people taller or the same height in front.
Person 2 has height 5 with two persons taller or the same height in front, which is person 0 and 1.
Person 3 has height 6 with one person taller or the same height in front, which is person 1.
Person 4 has height 4 with four people taller or the same height in front, which are people 0, 1, 2, and 3.
Person 5 has height 7 with one person taller or the same height in front, which is person 1.
Hence [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]] is the reconstructed queue.
```

Example 2:

```
Input: people = [[6,0],[5,0],[4,0],[3,2],[2,2],[1,4]]
Output: [[4,0],[5,0],[2,2],[3,2],[1,4],[6,0]]
```

Constraints:

- `1 <= people.length <= 2000`
- $0 \le h_i \le 10^6$
- $0 \le k_i <$ `people.length`
- It is guaranteed that the queue can be reconstructed.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Each person [h,k] must have exactly k people of height at least h in front. Inserting shorter people first lets later taller people change those k counts. Sort by height descending, then k ascending, and insert each person at index k. Everyone already in the queue is taller or equal, so that index is exactly the required count, and earlier people are unaffected. Taller people occupy their slots first; a shorter insertion is invisible to them.

Python 3

```
class Solution:
    def reconstructQueue(self, people: List[List[int]]) -> List[List[int]]:
        people.sort(key=lambda x: (-x[0], x[1]))
        ans = []
        for p in people:
            ans.insert(p[1], p)
        return ans
```

Complexity. Time: O(n^2) (each insert into a list is O(n)). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Check that a queue satisfies the description
# 2. Build the queue without inserting into the middle of
# a list: shortest first into the free slots


# ============================================================
# FOLLOW-UP 1: Check that a queue satisfies the
# description
# ============================================================

# ---- follow-up 1: function ----
# Answer: For every person count how many people in front
# are at least as tall; it must equal k.
# Time: O(n^2).  Space: O(1).
def is_valid_queue(queue):
    for i in range(len(queue)):
        height, k = queue[i]
        count = 0
        for j in range(i):
            if queue[j][0] >= height:
                count += 1
        if count != k:
            return False
    return True


# ============================================================
# FOLLOW-UP 2: Build the queue without inserting into the
# middle of a list: shortest first into the free slots
# ============================================================

# ---- follow-up 2: function ----
# Answer: Process the people from the shortest to the
# tallest (taller-or-equal people with a bigger k first
# among equals): a person with k people in front who are
# at least as tall takes the (k + 1)-th free slot, because
# every person placed later is taller.
# Time: O(n^2).  Space: O(n).
def reconstruct_by_free_slots(people):
    ordered = sorted(people, key=lambda p: (p[0], -p[1]))
    slots = [None] * len(people)
    for height, k in ordered:
        free = 0
        for i in range(len(slots)):
            if slots[i] is None:
                if free == k:
                    slots[i] = [height, k]
                    break
                free += 1
    return slots
```

PATTERN 21 · GREEDY · PROBLEM 18 OF 20

135. Candy

Hard · Greedy, Array

leetcode.com/problems/candy

★ **TOP 7 PROBLEM** in Greedy: master this one first

PROBLEM

There are `n` children standing in a line.

Each child is assigned a rating value given in the integer array `ratings`.

You are giving candies to these children subjected to the following requirements:

- Each child must have **at least** one candy.
- Children with a **higher** rating get more candies than their neighbors.

Return the **minimum** number of candies you need to have to distribute the candies to the children.

Example 1:

```
Input: ratings = [1,0,2]
Output: 5
Explanation: You can allocate to the first, second and third child with 2, 1, 2 candies respectively.
```

Example 2:

```
Input: ratings = [1,2,2]
Output: 4
Explanation: You can allocate to the first, second and third child with 1, 2, 1 candies respectively.
The third child gets 1 candy because it satisfies the above two conditions.
```

Constraints:

- `1 <= n == ratings.length <= 5 * 10^4`
- `0 <= ratings[i] <= 5 * 10^4`

SOLUTION (PYTHON 3)

Solution 1: Two traversals

How to think about it

A higher-rated child must get more candy than a neighbor; minimize the total. n≤ 5× 10^4. Sorting by rating can work but equal ratings are fiddly. The left and right constraints are independent: a left-to-right pass enforces “more than the left neighbor”, a right-to-left pass enforces the right, and each child takes the max. Both increasing chains survive.

We initialize two arrays left and right, where left[i] represents the minimum number of candies the current child should get when the current child's score is higher than the left child's score, and right[i] represents the minimum number of candies the current child should get when the current child's score is higher than the right child's score. Initially, left[i]=1, right[i]=1.

We traverse the array from left to right once, and if the current child's score is higher than the left child's score, then left[i]=left[i-1]+1; similarly, we traverse the array from right to left once, and if the current child's score is higher than the right child's score, then right[i]=right[i+1]+1.

Finally, we traverse the array of scores once, and the minimum number of candies each child should get is the maximum of left[i] and right[i], and we add them up to get the answer.

Time complexity O(n), space complexity O(n). Where n is the length of the array of scores.

Python 3

```
class Solution:
    def candy(self, ratings: List[int]) -> int:
        n = len(ratings)
        left = [1] * n
        right = [1] * n
        for i in range(1, n):
            if ratings[i] > ratings[i - 1]:
                left[i] = left[i - 1] + 1
        for i in range(n - 2, -1, -1):
            if ratings[i] > ratings[i + 1]:
                right[i] = right[i + 1] + 1
        return sum(max(a, b) for a, b in zip(left, right))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return how many candies each child gets
# 2. O(1) extra space: count the slopes


# ============================================================
# FOLLOW-UP 1: Return how many candies each child gets
# ============================================================

# ---- follow-up 1: function ----
# Answer: Two passes: left to right (more than the left
# neighbour if rated higher), then right to left (take the
# larger of the two requirements).
# Time: O(n).  Space: O(n).
def candies_per_child(ratings):
    n = len(ratings)
    candies = [1] * n
    for i in range(1, n):
        if ratings[i] > ratings[i - 1]:
            candies[i] = candies[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if ratings[i] > ratings[i + 1] and candies[i] <= candies[i + 1]:
            candies[i] = candies[i + 1] + 1
    return candies


# ============================================================
# FOLLOW-UP 2: O(1) extra space: count the slopes
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk through the ratings: an ascent of length u
# and a descent of length d each add 1 + 2 + ... candies;
# the peak child gets the larger of the two slopes.
# Time: O(n).  Space: O(1).
def candy_constant_space(ratings):
    n = len(ratings)
    total = 1
    up = down = peak = 0
    for i in range(1, n):
        if ratings[i] > ratings[i - 1]:
            up += 1
            peak = up
            down = 0
            total += 1 + up
        elif ratings[i] == ratings[i - 1]:
            up = down = peak = 0
            total += 1
        else:
            up = 0
```

```
            down += 1
            total += 1 + down
            if peak >= down:
                total -= 1
    return total
```

PATTERN 21 · GREEDY · PROBLEM 20 OF 20

630. Course Schedule III

Hard · Greedy, Array, Sorting, Heap (Priority Queue)

leetcode.com/problems/course-schedule-iii

PROBLEM

There are `n` different online courses numbered from `1` to `n`. You are given an array `courses` where `courses[i] = [duration_i, lastDay_i]` indicate that the i^{th} course should be taken **continuously** for `duration_i` days and must be finished before or on `lastDay_i`.

You will start on the 1^{st} day and you cannot take two or more courses simultaneously.

Return *the maximum number of courses that you can take*.

Example 1:

```
Input: courses = [[100,200],[200,1300],[1000,1250],[2000,3200]]
Output: 3
Explanation:
There are totally 4 courses, but you can take 3 courses at most:
First, take the 1st course, it costs 100 days so you will finish it on the 100th day, and ready to take the next course on the 101st day.
Second, take the 3rd course, it costs 1000 days so you will finish it on the 1100th day, and ready to take the next course on the 1101st day.
Third, take the 2nd course, it costs 200 days so you will finish it on the 1300th day.
The 4th course cannot be taken now, since you will finish it on the 3300th day, which exceeds the closed date.
```

Example 2:

```
Input: courses = [[1,2]]
Output: 1
```

Example 3:

```
Input: courses = [[3,2],[4,3]]
Output: 0
```

Constraints:

- `1 <= courses.length <= 10^4`
- `1 <= duration_i, lastDay_i <= 10^4`

SOLUTION (PYTHON 3)

Solution 1: Greedy + Priority Queue (Max-Heap)

> **How to think about it**
>
> Each course has a duration and a deadline; searching subsets is impossible for n≤ 10^4. Sort by deadline and take every course. If the running time exceeds the deadline, drop the longest taken course (max-heap) to free room for later tighter ones. The heap size is the answer.

We can sort the courses in ascending order by their end time, and each time select the course with the earliest deadline to take.

If the total time s of the selected courses exceeds the end time last of the current course, we remove the course with the longest duration from the previously selected courses, until the constraint of the current course's end time is satisfied. Here we use a priority queue (max-heap) pq to maintain the durations of the currently selected courses, and each time we pop the course with the longest duration from the priority queue to remove it.

Finally, the number of elements in the priority queue is the maximum number of courses we can take.

The time complexity is O(n × log n) and the space complexity is O(n), where n is the number of courses.

Python 3

```
class Solution:
    def scheduleCourse(self, courses: List[List[int]]) -> int:
        courses.sort(key=lambda x: x[1])
        pq = []
        s = 0
        for duration, last in courses:
            heappush(pq, -duration)
            s += duration
            while s > last:
                s += heappop(pq)
        return len(pq)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the courses that are taken
# 2. Can ALL the courses be taken?


# ============================================================
# FOLLOW-UP 1: Return the courses that are taken
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same greedy with a max-heap of durations;
# keep the course indices in the heap and report those
# left at the end.
# Time: O(n log n).  Space: O(n).
import heapq


def courses_taken(courses):
    order = sorted(range(len(courses)), key=lambda i: courses[i][1])
    heap = []                                       # (negative duration, index)
    time = 0
    for i in order:
        duration, last_day = courses[i]
        heapq.heappush(heap, (-duration, i))
        time += duration
        if time > last_day:
            negative, longest = heapq.heappop(heap)
            time += negative
    return sorted(index for negative, index in heap)


# ============================================================
# FOLLOW-UP 2: Can ALL the courses be taken?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Take the courses in the order of their last
# days; all can be taken exactly when the running total
# never passes a course's last day.
# Time: O(n log n).  Space: O(1) extra.
def can_take_all(courses):
    time = 0
    for duration, last_day in sorted(courses, key=lambda c: c[1]):
        time += duration
        if time > last_day:
            return False
    return True
```

PATTERN 22 OF 41

Dynamic Programming - 2D / Grid

20 problems · 0 Easy · 12 Medium · 8 Hard

THE IDEA

A table where each cell is built from its neighbours above and to the left (or from two indexes). Often the table can shrink to one row.

Signals: unique paths, minimum path sum, edit distance, longest common subsequence, maximal square, interleaving strings.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
dp = [[0] * (m + 1) for _ in range(n + 1)]
for i in range(1, n + 1):
    for j in range(1, m + 1):
        if a[i - 1] == b[j - 1]:
            dp[i][j] = dp[i - 1][j - 1] + 1
        else:
            dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Unique Paths	Medium	LC 62	1172
2	Unique Paths II	Medium	LC 63	1175
3	★ **Minimum Path Sum**	Medium	LC 64	1179
4	★ **Longest Common Subsequence**	Medium	LC 1143	1182
5	★ **Maximal Square**	Medium	LC 221	1185
6	Triangle	Medium	LC 120	1188
7	★ **Interleaving String**	Medium	LC 97	1191
8	Minimum Falling Path Sum	Medium	LC 931	1195
9	Count Square Submatrices with All Ones (see page)	Medium	LC 1277	285
10	Maximum Points You Can Obtain from Cards	Medium	LC 1423	1199
11	Out of Boundary Paths	Medium	LC 576	1202
12	Where Will the Ball Fall (see page)	Medium	LC 1706	1030
13	★ **Edit Distance**	Hard	LC 72	1206
14	★ **Distinct Subsequences**	Hard	LC 115	1209
15	Dungeon Game	Hard	LC 174	1212
16	Maximal Rectangle (see page)	Hard	LC 85	468
17	Cherry Pickup	Hard	LC 741	1215
18	Cherry Pickup II	Hard	LC 1463	1219
19	Minimum Falling Path Sum II	Hard	LC 1289	1223
20	★ **Longest Increasing Path in a Matrix**	Hard	LC 329	1225

PATTERN 22 · DYNAMIC PROGRAMMING - 2D / GRID · PROBLEM 1 OF 20

62. Unique Paths

Medium · Math, Dynamic Programming, Combinatorics

leetcode.com/problems/unique-paths

PROBLEM

There is a robot on an `m x n` grid. The robot is initially located at the **top-left corner** (i.e., `grid[0][0]`). The robot tries to move to the **bottom-right corner** (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time.

Given the two integers `m` and `n`, return *the number of possible unique paths that the robot can take to reach the bottom-right corner*.

The test cases are generated so that the answer will be less than or equal to `2 * 10^9`.

Example 1:

```
Input: m = 3, n = 7
Output: 28
```

Example 2:

```
Input: m = 3, n = 2
Output: 3
Explanation: From the top-left corner, there are a total of 3 ways to reach the bottom-right corner:
1. Right -> Down -> Down
2. Down -> Down -> Right
3. Down -> Right -> Down
```

Constraints:

- `1 <= m, n <= 100`

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming

> **How to think about it**
>
> The first idea is to search: only right or down, enumerate every path. With m, n ≤ 100, the path count is combinatorial, so a raw search explodes. The bottleneck is revisiting the same cell. Paths into (i, j) come only from above or the left and do not overlap, so the subproblems add. Store that count in f[i][j] and fill in row-major order so the dependencies already exist. Start at 1; the bottom-right cell is the answer.

We define f[i][j] to represent the number of paths from the top left corner to (i, j), initially f[0][0] = 1, and the answer is f[m - 1][n - 1].

Consider f[i][j]:

- If i > 0, then f[i][j] can be reached by taking one step from f[i - 1][j], so f[i][j] = f[i][j] + f[i - 1][j];
- If j > 0, then f[i][j] can be reached by taking one step from f[i][j - 1], so f[i][j] = f[i][j] + f[i][j - 1].

Therefore, we have the following state transition equation:

$$f[i][j] = \begin{cases} 1 & i = 0, j = 0 \\ f[i - 1][j] + f[i][j - 1] & \text{otherwise} \end{cases}$$

The final answer is f[m - 1][n - 1].

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the number of rows and columns of the grid, respectively.

Python 3

```python
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        f = [[0] * n for _ in range(m)]
        f[0][0] = 1
        for i in range(m):
            for j in range(n):
                if i:
                    f[i][j] += f[i - 1][j]
                if j:
                    f[i][j] += f[i][j - 1]
        return f[-1][-1]
```

Solution 2: Dynamic Programming (Prefilled Borders)

> **How to think about it**
>
> Solution 1 tests “has an above / has a left” on every cell, so the border keeps taking extra branches. The first row can arrive only from the left, the first column only from above, and both are all 1. Prefill those borders and the interior adds unconditionally. Same complexity, cleaner code.

Fill the first row and first column with 1, then only compute interior cells f[i][j] = f[i-1][j] + f[i][j-1]. Time and space stay O(m × n).

Python 3

```python
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        f = [[1] * n for _ in range(m)]
        for i in range(1, m):
            for j in range(1, n):
                f[i][j] = f[i - 1][j] + f[i][j - 1]
        return f[-1][-1]
```

Solution 3: Dynamic Programming (Rolling Array)

> **How to think about it**
>
> The first two solutions keep a full m × n table. f[i][j] only needs the previous row f[i-1][j] and the left cell f[i][j-1], so the first dimension can go. After compressing to 1D, f[j] is still the previous row until we update it; add f[j-1] to get the current row. Space drops to O(n), time stays the same.

f[i][j] depends only on the previous row and the left cell, so a 1D array of length n is enough. The time complexity is O(m × n) and the space complexity is O(n).

Python 3

```python
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        f = [1] * n
        for _ in range(1, m):
            for j in range(1, n):
                f[j] += f[j - 1]
        return f[-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. A formula instead of a table
# 2. Paths that must pass through a given cell


# ============================================================
# FOLLOW-UP 1: A formula instead of a table
# ============================================================

# ---- follow-up 1: function ----
# Answer: Every path has m - 1 moves down and n - 1 moves
# right in some order, so the number of paths is the
# number of ways to choose where the down moves go: C(m +
# n - 2, m - 1), computed with a loop that keeps the value
# an integer.
# Time: O(min(m, n)).  Space: O(1).
def unique_paths_formula(m, n):
    result = 1
    k = m - 1 if m - 1 < n - 1 else n - 1
    for i in range(1, k + 1):
        result = result * (m + n - 2 - k + i) // i
    return result


# ============================================================
# FOLLOW-UP 2: Paths that must pass through a given cell
# ============================================================

# ---- follow-up 2: function ----
# Answer: Multiply the number of paths from the start to
# the cell by the number of paths from the cell to the
# end.
# Time: O(m * n).  Space: O(n).
def unique_paths_through(m, n, row, col):
    def paths(rows, cols):
        ways = [1] * cols
        for r in range(1, rows):
            for c in range(1, cols):
                ways[c] += ways[c - 1]
        return ways[cols - 1]

    return paths(row + 1, col + 1) * paths(m - row, n - col)
```

PATTERN 22 · DYNAMIC PROGRAMMING - 2D / GRID · PROBLEM 2 OF 20

63. Unique Paths II

Medium · Array, Dynamic Programming, Matrix

leetcode.com/problems/unique-paths-ii

PROBLEM

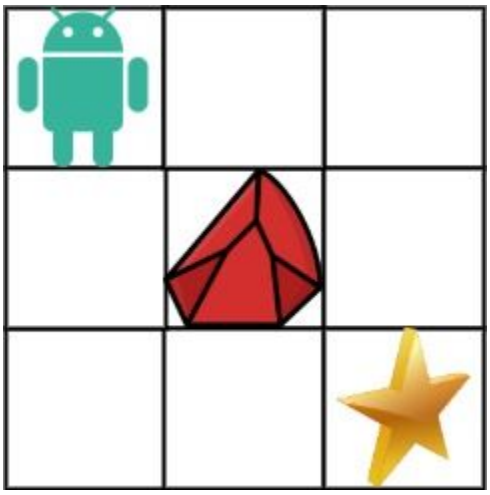
You are given an `m x n` integer array `grid`. There is a robot initially located at the **top-left corner** (i.e., `grid[0][0]`). The robot tries to move to the **bottom-right corner** (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time.

An obstacle and space are marked as `1` or `0` respectively in `grid`. A path that the robot takes cannot include **any** square that is an obstacle.

Return *the number of possible unique paths that the robot can take to reach the bottom-right corner*.

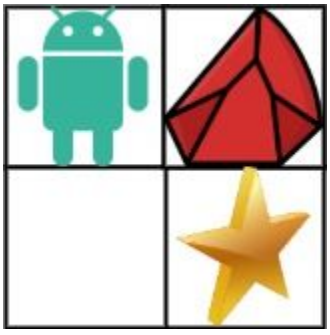
The testcases are generated so that the answer will be less than or equal to $2 * 10^9$.

Example 1:



```
Input: obstacleGrid = [[0,0,0],[0,1,0],[0,0,0]]
Output: 2
Explanation: There is one obstacle in the middle of the 3x3 grid above.
There are two ways to reach the bottom-right corner:
1. Right -> Right -> Down -> Down
2. Down -> Down -> Right -> Right
```

Example 2:



```
Input: obstacleGrid = [[0,1],[0,0]]
Output: 1
```

Constraints:

- `m == obstacleGrid.length`
- `n == obstacleGrid[i].length`
- `1 <= m, n <= 100`
- `obstacleGrid[i][j]` is `0` or `1`.

SOLUTIONS (PYTHON 3)

Solution 1: Memoization Search

> **How to think about it**
>
> As in Unique Paths, a raw search that only moves down or right explodes: m, n ≤ 100. Obstacles also break the "first row and column are all 1" shortcut. The bottleneck is expanding the same cell (i, j) many times. From here, out-of-bounds or an obstacle is 0, the destination is 1, otherwise the two branches add. Memoize dfs(i, j) so each cell is computed once; time and space are both O(mn).

We design a function dfs(i, j) to represent the number of paths from the grid (i, j) to the grid (m - 1, n - 1). Here, m and n are the number of rows and columns of the grid, respectively.

The execution process of the function dfs(i, j) is as follows:

- If i ≥ m or j ≥ n, or obstacleGrid[i][j] = 1, the number of paths is 0;
- If i = m - 1 and j = n - 1, the number of paths is 1;
- Otherwise, the number of paths is dfs(i + 1, j) + dfs(i, j + 1).

To avoid redundant calculations, we can use memoization.

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the number of rows and columns of the grid, respectively.

Python 3

```python
class Solution:
    def uniquePathsWithObstacles(self, obstacleGrid: List[List[int]]) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            if i >= m or j >= n or obstacleGrid[i][j]:
                return 0
            if i == m - 1 and j == n - 1:
                return 1
            return dfs(i + 1, j) + dfs(i, j + 1)

        m, n = len(obstacleGrid), len(obstacleGrid[0])
        return dfs(0, 0)
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> Solution 1 is already O(mn) after memoization, but it is still recursive, with a deeper implicit stack and larger constants. Obstacles cut the first row and column, so those borders are no longer all 1; filling a table bottom-up is more direct. f[i][j] is the number of paths from the start: 0 on an obstacle, otherwise from above and the left. Prefill the first row and column up to the first obstacle, then fill the interior—no recursion.

We can use a dynamic programming approach by defining a 2D array f, where f[i][j] represents the number of paths from the grid (0,0) to the grid (i,j).

We first initialize all values in the first column and the first row of f, then traverse the other rows and columns with two cases:

- If obstacleGrid[i][j] = 1, it means the number of paths is 0, so f[i][j] = 0;
- If obstacleGrid[i][j] = 0, then f[i][j] = f[i - 1][j] + f[i][j - 1].

Finally, return f[m - 1][n - 1].

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the number of rows and columns of the grid, respectively.

Python 3

```python
class Solution:
    def uniquePathsWithObstacles(self, obstacleGrid: List[List[int]]) -> int:
        m, n = len(obstacleGrid), len(obstacleGrid[0])
        f = [[0] * n for _ in range(m)]
        for i in range(m):
            if obstacleGrid[i][0] == 1:
                break
            f[i][0] = 1
        for j in range(n):
            if obstacleGrid[0][j] == 1:
                break
            f[0][j] = 1
        for i in range(1, m):
            for j in range(1, n):
                if obstacleGrid[i][j] == 0:
                    f[i][j] = f[i - 1][j] + f[i][j - 1]
        return f[-1][-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return one path (a list of cells)
# 2. Diagonal moves (down-right) are allowed too


# ============================================================
# FOLLOW-UP 1: Return one path (a list of cells)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Fill the table of path counts, then walk back
# from the end choosing a neighbour that has a path to it.
# Time: O(m * n).  Space: O(m * n).
def one_path_with_obstacles(grid):
    rows, cols = len(grid), len(grid[0])
    ways = []
    for r in range(rows):
        ways.append([0] * cols)
    if grid[0][0] == 1:
        return None
    ways[0][0] = 1
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                ways[r][c] = 0
            elif r > 0 or c > 0:
                if r > 0:
                    ways[r][c] += ways[r - 1][c]
                if c > 0:
                    ways[r][c] += ways[r][c - 1]
    if ways[rows - 1][cols - 1] == 0:
        return None
    path = [(rows - 1, cols - 1)]
    r, c = rows - 1, cols - 1
    while (r, c) != (0, 0):
        if r > 0 and ways[r - 1][c] > 0:
            r -= 1
        else:
            c -= 1
        path.append((r, c))
    path.reverse()
    return path
```

```
# ============================================================
# FOLLOW-UP 2: Diagonal moves (down-right) are allowed too
# ============================================================

# ---- follow-up 2: function ----
# Answer: Add the cell diagonally above-left to the sum.
# Time: O(m * n).  Space: O(m * n).
def paths_with_diagonals(grid):
    rows, cols = len(grid), len(grid[0])
    ways = []
    for r in range(rows):
        ways.append([0] * cols)
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                continue
            if r == 0 and c == 0:
                ways[r][c] = 1
                continue
            total = 0
            if r > 0:
                total += ways[r - 1][c]
            if c > 0:
                total += ways[r][c - 1]
            if r > 0 and c > 0:
                total += ways[r - 1][c - 1]
            ways[r][c] = total
    return ways[rows - 1][cols - 1]
```

64. Minimum Path Sum

Medium · Array, Dynamic Programming, Matrix

leetcode.com/problems/minimum-path-sum

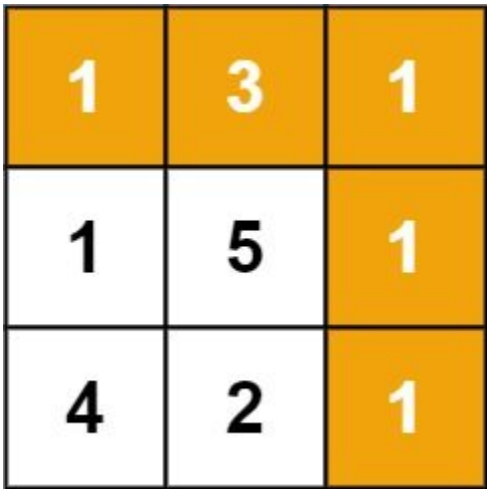
★ **TOP 7 PROBLEM** in Dynamic Programming - 2D / Grid: master this one first

PROBLEM

Given a `m x n grid` filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.

Example 1:




```
Input: grid = [[1,3,1],[1,5,1],[4,2,1]]
Output: 7
Explanation: Because the path 1 → 3 → 1 → 1 → 1 minimizes the sum.
```

Example 2:

```
Input: grid = [[1,2,3],[4,5,6]]
Output: 12
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 200`
- `0 <= grid[i][j] <= 200`

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

The first idea is to enumerate every path from the top left to the bottom right and take the smallest sum. m, n ≤ 200, so the number of paths is exponential. The bottleneck is recomputing overlapping prefixes. We may only move right or down, so the optimum into (i, j) is the better of the cell above and the cell to the left, plus the current value. Store that minimum in f[i][j]: the borders accumulate along a single edge; the interior takes a min then adds grid[i][j]. Fill by rows; the bottom-right cell is the answer.

We define f[i][j] to represent the minimum path sum from the top left corner to (i, j). Initially, f[0][0] = grid[0][0], and the answer is f[m - 1][n - 1].

Consider f[i][j]:

- If j = 0, then f[i][j] = f[i - 1][j] + grid[i][j];
- If i = 0, then f[i][j] = f[i][j - 1] + grid[i][j];
- If i > 0 and j > 0, then f[i][j] = min(f[i - 1][j], f[i][j - 1]) + grid[i][j].

Finally, return f[m - 1][n - 1].

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the number of rows and columns of the grid, respectively.

Python 3

```python
class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        f = [[0] * n for _ in range(m)]
        f[0][0] = grid[0][0]
        for i in range(1, m):
            f[i][0] = f[i - 1][0] + grid[i][0]
        for j in range(1, n):
            f[0][j] = f[0][j - 1] + grid[0][j]
        for i in range(1, m):
            for j in range(1, n):
                f[i][j] = min(f[i - 1][j], f[i][j - 1]) + grid[i][j]
        return f[-1][-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the path itself
# 2. You may move in all four directions


# ============================================================
# FOLLOW-UP 1: Return the path itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Fill the table of cheapest costs, then walk back
# from the end towards the neighbour with the smaller
# cost.
# Time: O(m * n).  Space: O(m * n).
def min_path(grid):
    rows, cols = len(grid), len(grid[0])
    cost = []
    for r in range(rows):
        cost.append([0] * cols)
    for r in range(rows):
        for c in range(cols):
            if r == 0 and c == 0:
                cost[r][c] = grid[r][c]
            elif r == 0:
                cost[r][c] = cost[r][c - 1] + grid[r][c]
            elif c == 0:
                cost[r][c] = cost[r - 1][c] + grid[r][c]
            else:
                best = cost[r - 1][c] if cost[r - 1][c] < cost[r][c - 1] else cost[r][c - 1]
                cost[r][c] = best + grid[r][c]
    path = [(rows - 1, cols - 1)]
    r, c = rows - 1, cols - 1
    while (r, c) != (0, 0):
        if r == 0:
            c -= 1
```

```
        elif c == 0:
            r -= 1
        elif cost[r - 1][c] <= cost[r][c - 1]:
            r -= 1
        else:
            c -= 1
        path.append((r, c))
    path.reverse()
    return cost[rows - 1][cols - 1], path


# ============================================================
# FOLLOW-UP 2: You may move in all four directions
# ============================================================

# ---- follow-up 2: function ----
# Answer: The cost of a cell no longer depends only on its
# top and left neighbour, so use Dijkstra's algorithm with
# a heap.
# Time: O(m * n log (m * n)).  Space: O(m * n).
import heapq


def min_path_four_directions(grid):
    rows, cols = len(grid), len(grid[0])
```

```
    best = {(0, 0): grid[0][0]}
    heap = [(grid[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if (r, c) == (rows - 1, cols - 1):
            return cost
        if cost > best[(r, c)]:
            continue
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols:
                new_cost = cost + grid[nr][nc]
                if (nr, nc) not in best or new_cost < best[(nr, nc)]:
                    best[(nr, nc)] = new_cost
                    heapq.heappush(heap, (new_cost, nr, nc))
    return None
```

1143. Longest Common Subsequence

Medium · String, Dynamic Programming, Longest Common Subsequence

leetcode.com/problems/longest-common-subsequence

★ **TOP 7 PROBLEM** in Dynamic Programming - 2D / Grid: master this one first

PROBLEM

Given two strings `text1` and `text2`, return *the length of their longest* ***common subsequence***. If there is no **common subsequence**, return `0`.

A **subsequence** of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- For example, `"ace"` is a subsequence of `"abcde"`.

A **common subsequence** of two strings is a subsequence that is common to both strings.

Example 1:

```
Input: text1 = "abcde", text2 = "ace"
Output: 3
Explanation: The longest common subsequence is "ace" and its length is 3.
```

Example 2:

```
Input: text1 = "abc", text2 = "abc"
Output: 3
Explanation: The longest common subsequence is "abc" and its length is 3.
```

Example 3:

```
Input: text1 = "abc", text2 = "def"
Output: 0
Explanation: There is no such common subsequence, so the result is 0.
```

Constraints:

- `1 <= text1.length, text2.length <= 1000`
- `text1` and `text2` consist of only lowercase English characters.

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

A common subsequence need not be contiguous; enumerating subsequences is too costly. f[i][j] is the LCS of the two prefixes: matching last characters add one to f[i-1][j-1], otherwise take the better of dropping one character. Filling by prefix length uses only shorter prefixes.

We define f[i][j] as the length of the longest common subsequence of the first i characters of text1 and the first j characters of text2. Therefore, the answer is f[m][n], where m and n are the lengths of text1 and text2, respectively.

If the ith character of text1 and the jth character of text2 are the same, then f[i][j] = f[i - 1][j - 1] + 1; if the ith character of text1 and the jth character of text2 are different, then f[i][j] = max(f[i - 1][j], f[i][j - 1]). The state transition equation is:

f[i][j] = cases f[i - 1][j - 1] + 1, & if text1[i - 1] = text2[j - 1] \ max(f[i - 1][j], f[i][j - 1]), & if text1[i - 1] ≠ text2[j - 1] cases

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the lengths of text1 and text2, respectively.

Python 3

```
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m, n = len(text1), len(text2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if text1[i - 1] == text2[j - 1]:
                    f[i][j] = f[i - 1][j - 1] + 1
                else:
                    f[i][j] = max(f[i - 1][j], f[i][j - 1])
        return f[m][n]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the common subsequence itself
# 2. The shortest string that has both texts as
# subsequences (shortest common supersequence)


# ============================================================
# FOLLOW-UP 1: Return the common subsequence itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Fill the table of lengths, then walk back from
# the end: equal letters belong to the answer, otherwise
# move towards the larger neighbour.
# Time: O(a * b).  Space: O(a * b).
def lcs_string(a, b):
    table = []
    for i in range(len(a) + 1):
        table.append([0] * (len(b) + 1))
    for i in range(1, len(a) + 1):
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                table[i][j] = table[i - 1][j - 1] + 1
            elif table[i - 1][j] >= table[i][j - 1]:
                table[i][j] = table[i - 1][j]
            else:
                table[i][j] = table[i][j - 1]
    letters = []
    i, j = len(a), len(b)
    while i > 0 and j > 0:
        if a[i - 1] == b[j - 1]:
            letters.append(a[i - 1])
            i -= 1
            j -= 1
        elif table[i - 1][j] >= table[i][j - 1]:
            i -= 1
        else:
            j -= 1
    letters.reverse()
    return "".join(letters)


# ============================================================
# FOLLOW-UP 2: The shortest string that has both texts as
# subsequences (shortest common supersequence)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Merge the two texts along their longest common
# subsequence: its letters are written once, and the
# letters between them from both texts are written in
```

```
# order.
# Time: O(a * b).  Space: O(a * b).
def shortest_common_supersequence(a, b):
    table = []
    for i in range(len(a) + 1):
        table.append([0] * (len(b) + 1))
    for i in range(1, len(a) + 1):
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
```

```
                table[i][j] = table[i - 1][j - 1] + 1
            elif table[i - 1][j] >= table[i][j - 1]:
                table[i][j] = table[i - 1][j]
            else:
                table[i][j] = table[i][j - 1]
    letters = []
    i, j = len(a), len(b)
    while i > 0 and j > 0:
        if a[i - 1] == b[j - 1]:
            letters.append(a[i - 1])
            i -= 1
            j -= 1
        elif table[i - 1][j] >= table[i][j - 1]:
            letters.append(a[i - 1])
            i -= 1
        else:
            letters.append(b[j - 1])
            j -= 1
    while i > 0:
        letters.append(a[i - 1])
        i -= 1
    while j > 0:
        letters.append(b[j - 1])
        j -= 1
    letters.reverse()
    return "".join(letters)
```

PATTERN 22 · DYNAMIC PROGRAMMING - 2D / GRID · PROBLEM 5 OF 20

221. Maximal Square

Medium · Array, Dynamic Programming, Matrix

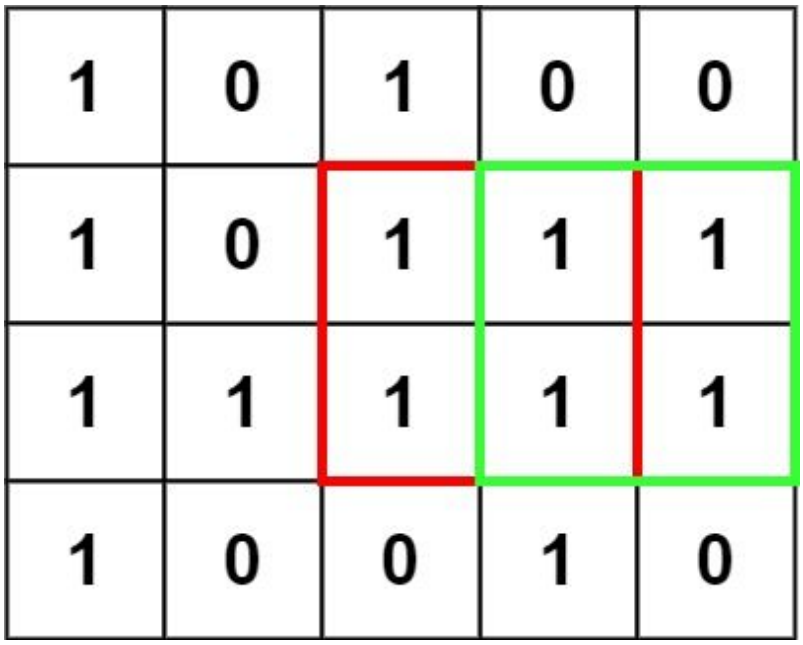
leetcode.com/problems/maximal-square

★ **TOP 7 PROBLEM** in Dynamic Programming - 2D / Grid: master this one first

PROBLEM

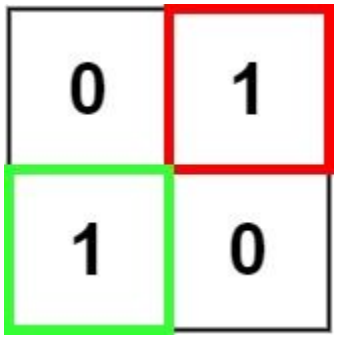
Given an `m x n` binary `matrix` filled with `0`'s and `1`'s, *find the largest square containing only* `1`'s *and return its area*.

Example 1:




```
Input: matrix =
[["1","0","1","0","0"],["1","0","1","1","1"],["1","1","1","1","1"],["1","0","0","1","0"]]
Output: 4
```

Example 2:




```
Input: matrix = [["0","1"],["1","0"]]
Output: 1
```

Example 3:

```
Input: matrix = [["0"]]
Output: 0
```

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 300`
- `matrix[i][j]` is `'0'` or `'1'`.

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

Checking a square above every bottom-right cell revisits many cells. The largest side ending at (i,j) is limited by the three sides ending at its left, upper, and upper-left neighbors. Let dp[i+1][j+1] be that side: 0 on a 0 cell, otherwise one plus the min of those three. The answer is the square of the largest side.

We define dp[i + 1][j + 1] as the maximum square side length with the lower right corner at index (i, j). The answer is the maximum value among all dp[i + 1][j + 1].

The state transition equation is:

dp[i + 1][j + 1] = cases 0 & if matrix[i][j] = '0' \ min(dp[i][j], dp[i][j + 1], dp[i + 1][j]) + 1 & if matrix[i][j] = '1' cases

The time complexity is O(m× n), and the space complexity is O(m× n). Where m and n are the number of rows and columns of the matrix, respectively.

Python 3

```
class Solution:
    def maximalSquare(self, matrix: List[List[str]]) -> int:
        m, n = len(matrix), len(matrix[0])
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        mx = 0
        for i in range(m):
            for j in range(n):
                if matrix[i][j] == '1':
                    dp[i + 1][j + 1] = min(dp[i][j + 1], dp[i + 1][j], dp[i][j]) + 1
                    mx = max(mx, dp[i + 1][j + 1])
        return mx * mx
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the position and side of the largest square
# 2. O(columns) extra memory


# ============================================================
# FOLLOW-UP 1: Return the position and side of the largest
# square
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same table of side lengths; remember where
# the largest value occurred: that cell is the bottom-
# right corner.
# Time: O(m * n).  Space: O(m * n).
def largest_square_position(matrix):
    rows, cols = len(matrix), len(matrix[0])
    side = []
    for r in range(rows):
        side.append([0] * cols)
    best = (0, None)
    for r in range(rows):
        for c in range(cols):
```

```
            if matrix[r][c] in ("1", 1):
                if r == 0 or c == 0:
                    side[r][c] = 1
                else:
                    smallest = side[r - 1][c]
                    if side[r][c - 1] < smallest:
                        smallest = side[r][c - 1]
                    if side[r - 1][c - 1] < smallest:
                        smallest = side[r - 1][c - 1]
                    side[r][c] = smallest + 1
                if side[r][c] > best[0]:
                    best = (side[r][c], (r - side[r][c] + 1, c - side[r][c] + 1))
    return best


# ============================================================
# FOLLOW-UP 2: O(columns) extra memory
# ============================================================

# ---- follow-up 2: function ----
# Answer: Only the previous row of the table is needed:
# keep one row and the diagonal value from the previous
# row in a variable.
# Time: O(m * n).  Space: O(n).
def largest_square_one_row(matrix):
    cols = len(matrix[0])
    row = [0] * (cols + 1)
    best = 0
    for r in range(len(matrix)):
        diagonal = 0
        for c in range(1, cols + 1):
            above = row[c]
            if matrix[r][c - 1] in ("1", 1):
                smallest = above
                if row[c - 1] < smallest:
                    smallest = row[c - 1]
                if diagonal < smallest:
                    smallest = diagonal
                row[c] = smallest + 1
                if row[c] > best:
                    best = row[c]
            else:
                row[c] = 0
            diagonal = above
    return best * best
```

120. Triangle

Medium · Array, Dynamic Programming

leetcode.com/problems/triangle

PROBLEM

Given a `triangle` array, return *the minimum path sum from top to bottom*.

For each step, you may move to an adjacent number of the row below. More formally, if you are on index `i` on the current row, you may move to either index `i` or index `i + 1` on the next row.

Example 1:

```
Input: triangle = [[2],[3,4],[6,5,7],[4,1,8,3]]
Output: 11
Explanation: The triangle looks like:
   2
  3 4
 6 5 7
4 1 8 3
The minimum path sum from top to bottom is 2 + 3 + 5 + 1 = 11 (underlined above).
```

Example 2:

```
Input: triangle = [[-10]]
Output: -10
```

Constraints:

- `1 <= triangle.length <= 200`
- `triangle[0].length == 1`
- `triangle[i].length == triangle[i - 1].length + 1`
- -10^4 `<= triangle[i][j] <=` 10^4

Follow up:

Could you do this using only

`O(n)`

extra space, where

`n`

is the total number of rows in the triangle?

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

Each step may only move to an adjacent cell on the next row; enumerating paths grows exponentially with the number of rows. Subproblems overlap: the best path from a cell depends only on the two cells below. Define f[i][j] bottom-up as the min path from that cell to the last row. Each cell takes the min of the two below plus itself; f[0][0] is the answer.

We define f[i][j] as the minimum path sum from the bottom of the triangle to position (i, j). Here, position (i, j) refers to the position in row i and column j of the triangle (both indexed from 0). We have the following state transition equation:

$$f[i][j] = \min(f[i + 1][j], f[i + 1][j + 1]) + triangle[i][j]$$

The answer is f[0][0].

Python 3

```
class Solution:
    def minimumTotal(self, triangle: List[List[int]]) -> int:
        n = len(triangle)
        f = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(n - 1, -1, -1):
            for j in range(i + 1):
                f[i][j] = min(f[i + 1][j], f[i + 1][j + 1]) + triangle[i][j]
        return f[0][0]
```

Complexity. Time: O(n^2), n = number of rows. Space: O(n^2).

Solution 2: Dynamic Programming (Space Optimization)

> **How to think about it**
>
> In Solution 1, f[i][j] depends only on the next row. Rolling a one-dimensional array upward cuts space from $O(n^2)$ to O(n), which matches the follow-up.

We notice that the state f[i][j] only depends on states f[i + 1][j] and f[i + 1][j + 1]. Therefore, we can use a one-dimensional array instead of a two-dimensional array, reducing the space complexity from $O(n^2)$ to O(n).

The time complexity is $O(n^2)$, and the space complexity is O(n), where n is the number of rows in the triangle.

Python 3

```
class Solution:
    def minimumTotal(self, triangle: List[List[int]]) -> int:
        n = len(triangle)
        f = [0] * (n + 1)
        for i in range(n - 1, -1, -1):
            for j in range(i + 1):
                f[j] = min(f[j], f[j + 1]) + triangle[i][j]
        return f[0]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the path of the minimum sum
# 2. The MAXIMUM path sum


# ============================================================
# FOLLOW-UP 1: Return the path of the minimum sum
# ============================================================

# ---- follow-up 1: function ----
# Answer: Fill the table from the bottom up and then walk
# down from the top choosing the smaller of the two cells
# below.
# Time: O(n^2), n = rows.  Space: O(n^2).
def triangle_path(triangle):
    n = len(triangle)
    best = []
    for row in triangle:
        best.append(list(row))
    for r in range(n - 2, -1, -1):
        for c in range(len(triangle[r])):
            smaller = best[r + 1][c] if best[r + 1][c] < best[r + 1][c + 1] else best[r + 1][c + 1]
            best[r][c] += smaller
    path = [triangle[0][0]]
    c = 0
    for r in range(1, n):
        if best[r][c + 1] < best[r][c]:
            c += 1
```

```
        path.append(triangle[r][c])
    return best[0][0], path


# ============================================================
# FOLLOW-UP 2: The MAXIMUM path sum
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same table with the larger of the two cells
# below.
# Time: O(n^2).  Space: O(n).
def triangle_max_path(triangle):
    best = list(triangle[-1])
    for r in range(len(triangle) - 2, -1, -1):
        for c in range(len(triangle[r])):
            bigger = best[c] if best[c] > best[c + 1] else best[c + 1]
            best[c] = triangle[r][c] + bigger
    return best[0]
```

97. Interleaving String

Medium · String, Dynamic Programming

leetcode.com/problems/interleaving-string

★ **TOP 7 PROBLEM** in Dynamic Programming - 2D / Grid: master this one first

PROBLEM

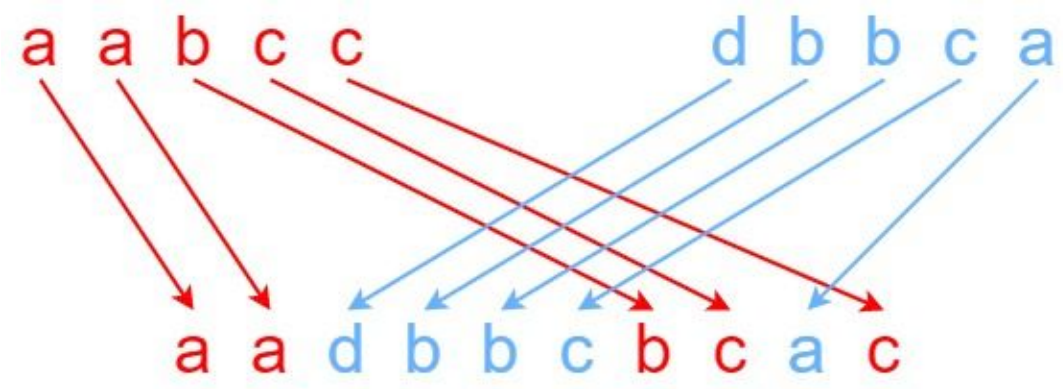
Given strings `s1`, `s2`, and `s3`, find whether `s3` is formed by an **interleaving** of `s1` and `s2`.

An **interleaving** of two strings `s` and `t` is a configuration where `s` and `t` are divided into `n` and `m` substrings respectively, such that:

- $s = s_1 + s_2 + ... + s_n$
- $t = t_1 + t_2 + ... + t_m$
- $|n - m| <= 1$
- The **interleaving** is $s_1 + t_1 + s_2 + t_2 + s_3 + t_3 + ...$ or $t_1 + s_1 + t_2 + s_2 + t_3 + s_3 + ...$

Note: `a + b` is the concatenation of strings `a` and `b`.

Example 1:




```
Input: s1 = "aabcc", s2 = "dbbca", s3 = "aadbbcbcac"
Output: true
Explanation: One way to obtain s3 is:
Split s1 into s1 = "aa" + "bc" + "c", and s2 into s2 = "dbbc" + "a".
Interleaving the two splits, we get "aa" + "dbbc" + "bc" + "a" + "c" = "aadbbcbcac".
Since s3 can be obtained by interleaving s1 and s2, we return true.
```

Example 2:

```
Input: s1 = "aabcc", s2 = "dbbca", s3 = "aadbbbaccc"
Output: false
Explanation: Notice how it is impossible to interleave s2 with any other string to obtain s3.
```

Example 3:

```
Input: s1 = "", s2 = "", s3 = ""
Output: true
```

Constraints:

- `0 <= s1.length, s2.length <= 100`
- `0 <= s3.length <= 200`
- `s1`, `s2`, and `s3` consist of lowercase English letters.

Follow up: Could you solve it using only `O(s2.length)` additional memory space?

SOLUTIONS (PYTHON 3)

Solution 1: Memoization Search

> **How to think about it**
>
> An interleaving keeps the relative order of s_1 and s_2, so lengths must match first. Brute force chooses the next char from s_1 or s_2 at every step and explodes. The state is only "how much of s_1 and s_2 we have used": s_3's index is k=i+j. A choice is legal iff $s_1[i]$ or $s_2[j]$ equals $s_3[k]$. Memoize (i,j) and there are O(mn) subproblems.

Let's denote the length of string s_1 as m and the length of string s_2 as n. If $m + n \neq |s_3|$, then s_3 is definitely not an interleaving string of s_1 and s_2, so we return `false`.

Next, we design a function dfs(i, j), which represents whether the remaining part of s_3 can be interleaved from the ith character of s_1 and the jth character of s_2. The answer is dfs(0, 0).

The calculation process of function dfs(i, j) is as follows:

If $i \geq m$ and $j \geq n$, it means that both s_1 and s_2 have been traversed, so we return `true`.

If $i < m$ and $s_1[i] = s_3[i + j]$, it means that the character $s_1[i]$ is part of $s_3[i + j]$. Therefore, we recursively call dfs(i + 1, j) to check whether the next character of s_1 can match the current character of s_2. If it can match, we return `true`.

Similarly, if $j < n$ and $s_2[j] = s_3[i + j]$, it means that the character $s_2[j]$ is part of $s_3[i + j]$. Therefore, we recursively call dfs(i, j + 1) to check whether the next character of s_2 can match the current character of s_1. If it can match, we return `true`.

Otherwise, we return `false`.

To avoid repeated calculations, we can use memoization search.

The time complexity is $O(m \times n)$, and the space complexity is $O(m \times n)$. Here, m and n are the lengths of strings s_1 and s_2 respectively.

Python 3

```python
class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        @cache
        def dfs(i: int, j: int) -> bool:
            if i >= m and j >= n:
                return True
            k = i + j
            if i < m and s1[i] == s3[k] and dfs(i + 1, j):
                return True
            if j < n and s2[j] == s3[k] and dfs(i, j + 1):
                return True
            return False

        m, n = len(s1), len(s2)
        if m + n != len(s3):
            return False
        return dfs(0, 0)
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> Solution 1 already has the transition. Bottom-up: f[i][j] means the first i chars of s_1 and first j of s_2 can form the first i+j of s_3. Empty-empty is true; fill by increasing i,j and drop the recursion stack.

We can convert the memoization search in Solution 1 into dynamic programming.

We define f[i][j] to represent whether the first i characters of string s_1 and the first j characters of string s_2 can interleave to form the first i + j characters of string s_3. When transitioning states, we can consider whether the current character is obtained from the last character of s_1 or the last character of s_2. Therefore, we have the state transition equation:

$$f[i][j] = \text{cases } f[i - 1][j] \ \& \text{ if } s_1[i - 1] = s_3[i + j - 1] \ \backslash \text{ or } f[i][j - 1] \ \& \text{ if } s_2[j - 1] = s_3[i + j - 1] \ \backslash \text{ false } \& \text{ otherwise cases}$$

where f[0][0] = true indicates that an empty string is an interleaving string of two empty strings.

The answer is f[m][n].

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the lengths of strings s_1 and s_2 respectively.

Python 3

```python
class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        m, n = len(s1), len(s2)
        if m + n != len(s3):
            return False
        f = [[False] * (n + 1) for _ in range(m + 1)]
        f[0][0] = True
        for i in range(m + 1):
            for j in range(n + 1):
                k = i + j - 1
                if i and s1[i - 1] == s3[k]:
                    f[i][j] = f[i - 1][j]
                if j and s2[j - 1] == s3[k]:
                    f[i][j] |= f[i][j - 1]
        return f[m][n]
```

Solution 3: Rolling Array Optimization

> **How to think about it**
>
> The follow-up asks for O(|s_2|) extra space. f[i][j] depends only on the previous row and the left cell, so one rolling array is enough — the O(n) bound in the problem.

We notice that f[i][j] is only related to f[i - 1][j] and f[i][j - 1]. Therefore, we can use a rolling array to reduce the space complexity to O(n). The time complexity remains O(m × n).

Python 3

```python
class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        m, n = len(s1), len(s2)
        if m + n != len(s3):
            return False
        f = [True] + [False] * n
        for i in range(m + 1):
            for j in range(n + 1):
                k = i + j - 1
                if i:
                    f[j] &= s1[i - 1] == s3[k]
                if j:
                    f[j] |= f[j - 1] and s2[j - 1] == s3[k]
        return f[n]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Count the number of different ways to interleave
# 2. Show which string every letter of s3 came from (one
# interleaving)


# ============================================================
# FOLLOW-UP 1: Count the number of different ways to
# interleave
# ============================================================

# ---- follow-up 1: function ----
```

```
# Answer: ways[i][j] = ways to build the first i + j
# letters of s3 from i letters of s1 and j letters of s2;
# add the two ways that the last letter could have come
# from.
# Time: O(a * b).  Space: O(a * b).
def count_interleavings(s1, s2, s3):
    if len(s1) + len(s2) != len(s3):
        return 0
    ways = []
    for i in range(len(s1) + 1):
        ways.append([0] * (len(s2) + 1))
    ways[0][0] = 1
    for i in range(len(s1) + 1):
        for j in range(len(s2) + 1):
            if i > 0 and s1[i - 1] == s3[i + j - 1]:
                ways[i][j] += ways[i - 1][j]
            if j > 0 and s2[j - 1] == s3[i + j - 1]:
                ways[i][j] += ways[i][j - 1]
    return ways[len(s1)][len(s2)]


# ========================================================
# FOLLOW-UP 2: Show which string every letter of s3 came
# from (one interleaving)
# ========================================================

# ---- follow-up 2: function ----
# Answer: Walk back through the table of possibilities
# from the end and record the source of each letter.
# Time: O(a * b).  Space: O(a * b).
def interleaving_sources(s1, s2, s3):
    if len(s1) + len(s2) != len(s3):
        return None
    ok = []
    for i in range(len(s1) + 1):
        ok.append([False] * (len(s2) + 1))
    ok[0][0] = True
    for i in range(len(s1) + 1):
        for j in range(len(s2) + 1):
            if i > 0 and ok[i - 1][j] and s1[i - 1] == s3[i + j - 1]:
                ok[i][j] = True
            if j > 0 and ok[i][j - 1] and s2[j - 1] == s3[i + j - 1]:
                ok[i][j] = True
    if not ok[len(s1)][len(s2)]:
        return None
    sources = []
    i, j = len(s1), len(s2)
    while i > 0 or j > 0:
```

```
        if i > 0 and ok[i - 1][j] and s1[i - 1] == s3[i + j - 1]:
            sources.append(1)
            i -= 1
        else:
            sources.append(2)
            j -= 1
    sources.reverse()
    return sources
```

931. Minimum Falling Path Sum

Medium · Array, Dynamic Programming, Matrix

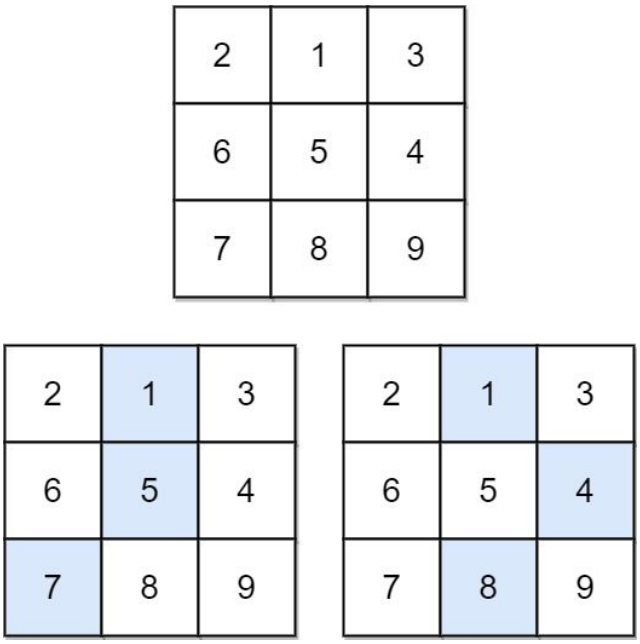
leetcode.com/problems/minimum-falling-path-sum

PROBLEM

Given an `n x n` array of integers `matrix`, return *the **minimum sum** of any **falling path** through* `matrix`.

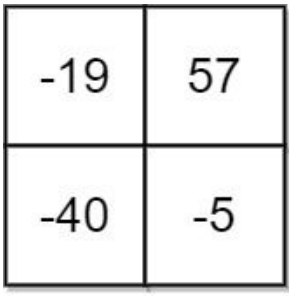
A **falling path** starts at any element in the first row and chooses the element in the next row that is either directly below or diagonally left/right. Specifically, the next element from position `(row, col)` will be `(row + 1, col - 1)`, `(row + 1, col)`, or `(row + 1, col + 1)`.

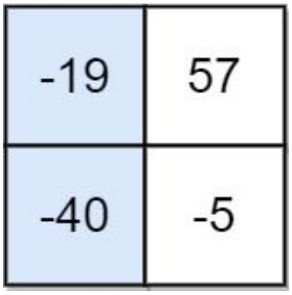
Example 1:




```
Input: matrix = [[2,1,3],[6,5,4],[7,8,9]]
Output: 13
Explanation: There are two falling paths with a minimum sum as shown.
```

Example 2:






```
Input: matrix = [[-19,57],[-40,-5]]
Output: -59
Explanation: The falling path with a minimum sum is shown.
```

Constraints:

- `n == matrix.length == matrix[i].length`
- `1 <= n <= 100`
- `-100 <= matrix[i][j] <= 100`

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming (Rolling Array)

How to think about it

A falling path may step only to the three adjacent cells in the next row, and n≤ 100. Enumerating paths is impossible. The best way to (i,j) depends only on the previous row at j-1,j,j+1. Compute this row by row and roll a one-dimensional array, using O(n) extra space.

Let f[i][j] be the minimum falling path sum that ends at row i, column j:

f[i][j] = matrix[i][j] + min { aligned & f[i - 1][j - 1], & j > 0 \ & f[i - 1][j], & 0 ≤ j < n \ & f[i - 1][j + 1], & j + 1 < n aligned .

The answer is $\min_{0 \le j < n} f[n - 1][j]$.

f[i][j] depends only on the previous row, so we keep two arrays f and g of length n.

The time complexity is $O(n^2)$ and the space complexity is O(n), where n is the side length of the matrix.

Python 3

```python
class Solution:
    def minFallingPathSum(self, matrix: List[List[int]]) -> int:
        n = len(matrix)
        f = [0] * n
        for row in matrix:
            g = [0] * n
            for j, x in enumerate(row):
                l, r = max(0, j - 1), min(n, j + 2)
                g[j] = min(f[l:r]) + x
            f = g
        return min(f)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the falling path itself
# 2. The MAXIMUM falling path sum


# ============================================================
# FOLLOW-UP 1: Return the falling path itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Fill the table of cheapest sums from the top,
# then walk back from the cheapest cell of the last row to
# the cheapest of the three cells above.
# Time: O(n^2).  Space: O(n^2).
def falling_path(matrix):
    n = len(matrix)
    best = []
    for row in matrix:
        best.append(list(row))
    for r in range(1, n):
        for c in range(n):
            smallest = best[r - 1][c]
            if c > 0 and best[r - 1][c - 1] < smallest:
                smallest = best[r - 1][c - 1]
            if c < n - 1 and best[r - 1][c + 1] < smallest:
                smallest = best[r - 1][c + 1]
            best[r][c] += smallest
    c = 0
    for i in range(n):
        if best[n - 1][i] < best[n - 1][c]:
            c = i
    total = best[n - 1][c]
    columns = [c]
    for r in range(n - 1, 0, -1):
        candidates = [c]
        if c > 0:
            candidates.append(c - 1)
        if c < n - 1:
            candidates.append(c + 1)
        c = min(candidates, key=lambda i: best[r - 1][i])
        columns.append(c)
    columns.reverse()
    return total, columns


# ============================================================
# FOLLOW-UP 2: The MAXIMUM falling path sum
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same table with the largest of the three
# cells above.
# Time: O(n^2).  Space: O(n).
def max_falling_path(matrix):
    n = len(matrix)
    best = list(matrix[0])
    for r in range(1, n):
        new_row = []
        for c in range(n):
            biggest = best[c]
            if c > 0 and best[c - 1] > biggest:
                biggest = best[c - 1]
            if c < n - 1 and best[c + 1] > biggest:
                biggest = best[c + 1]
```

```
            new_row.append(matrix[r][c] + biggest)
        best = new_row
    return max(best)
```

PATTERN 22 · DYNAMIC PROGRAMMING - 2D / GRID · PROBLEM 10 OF 20

1423. Maximum Points You Can Obtain from Cards

Medium · Array, Prefix Sum, Sliding Window

leetcode.com/problems/maximum-points-you-can-obtain-from-cards

Also appears in pattern 35 Sliding Window Maximum / Minimum (Specific Variants)

PROBLEM

There are several cards **arranged in a row**, and each card has an associated number of points. The points are given in the integer array `cardPoints`.

In one step, you can take one card from the beginning or from the end of the row. You have to take exactly `k` cards.

Your score is the sum of the points of the cards you have taken.

Given the integer array `cardPoints` and the integer `k`, return the *maximum score* you can obtain.

Example 1:

```
Input: cardPoints = [1,2,3,4,5,6,1], k = 3
Output: 12
Explanation: After the first step, your score will always be 1. However, choosing the rightmost card
first will maximize your total score. The optimal strategy is to take the three cards on the right,
giving a final score of 1 + 6 + 5 = 12.
```

Example 2:

```
Input: cardPoints = [2,2,2], k = 2
Output: 4
Explanation: Regardless of which two cards you take, your score will always be 4.
```

Example 3:

```
Input: cardPoints = [9,7,7,9,7,7,9], k = 7
Output: 55
Explanation: You have to take all the cards. Your score is the sum of points of all cards.
```

Constraints:

- `1 <= cardPoints.length <= 10^5`
- `1 <= cardPoints[i] <= 10^4`
- `1 <= k <= cardPoints.length`

SOLUTION (PYTHON 3)

Solution 1: Sliding Window

> **How to think about it**
>
> Taking k cards from the two ends means i from the left and k-i from the right. n≤ 10^5, so we cannot resumm each i. Start with the rightmost k cards, then replace the leftmost of those with the next left-end card, updating the sum in O(1) and keeping the maximum.

We can use a sliding window of length k to simulate this process.

Initially, we place the window at the end of the array, i.e., the k positions from index n-k to index n-1. The sum of the points of the cards in the window is denoted as s, and the initial value of the answer ans is also s.

Next, we consider the situation where we take 1, 2, ..., k cards from the beginning of the array in turn. Suppose the card taken is cardPoints[i]. Then we add it to s. Due to the length limit of the window is k, we need to subtract cardPoints[n-k+i] from s. In this way, we can calculate the sum of the points of the k cards taken and update the answer ans.

The time complexity is O(k), where k is the integer given in the problem. The space complexity is O(1).

Python 3

```python
class Solution:
    def maxScore(self, cardPoints: List[int], k: int) -> int:
        ans = s = sum(cardPoints[-k:])
        for i, x in enumerate(cardPoints[:k]):
            s += x - cardPoints[-k + i]
            ans = max(ans, s)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. How many cards are taken from the left in the best
# choice?
# 2. You may take FEWER than k cards (at most k) and some
# cards can be negative


# ============================================================
# FOLLOW-UP 1: How many cards are taken from the left in
# the best choice?
# ============================================================

# ---- follow-up 1: function ----
# Answer: The window of the cards that are NOT taken has
# length n - k; find the window with the smallest sum, and
# the number of cards taken from the left is the window's
# start.
# Time: O(n).  Space: O(1).
def best_left_count(card_points, k):
    n = len(card_points)
    window = n - k
    total = 0
    for x in card_points:
        total += x
    current = 0
    for i in range(window):
        current += card_points[i]
    smallest = current
    best_start = 0
    for start in range(1, k + 1):
        current += card_points[start + window - 1] - card_points[start - 1]
        if current < smallest:
            smallest = current
            best_start = start
    return best_start, total - smallest


# ============================================================
# FOLLOW-UP 2: You may take FEWER than k cards (at most k)
# and some cards can be negative
# ============================================================

# ---- follow-up 2: function ----
# Answer: Try every number of cards taken from the left
# and from the right (together at most k) with prefix sums
# and keep the best total.
# Time: O(k^2).  Space: O(n).
def max_points_at_most_k(card_points, k):
    n = len(card_points)
    prefix = [0]
    for x in card_points:
        prefix.append(prefix[-1] + x)
    best = 0
```

```
    for left in range(0, min(k, n) + 1):
        for right in range(0, min(k - left, n - left) + 1):
            total = prefix[left] + prefix[n] - prefix[n - right]
            if total > best:
                best = total
    return best
```

576. Out of Boundary Paths

Medium · Dynamic Programming

leetcode.com/problems/out-of-boundary-paths

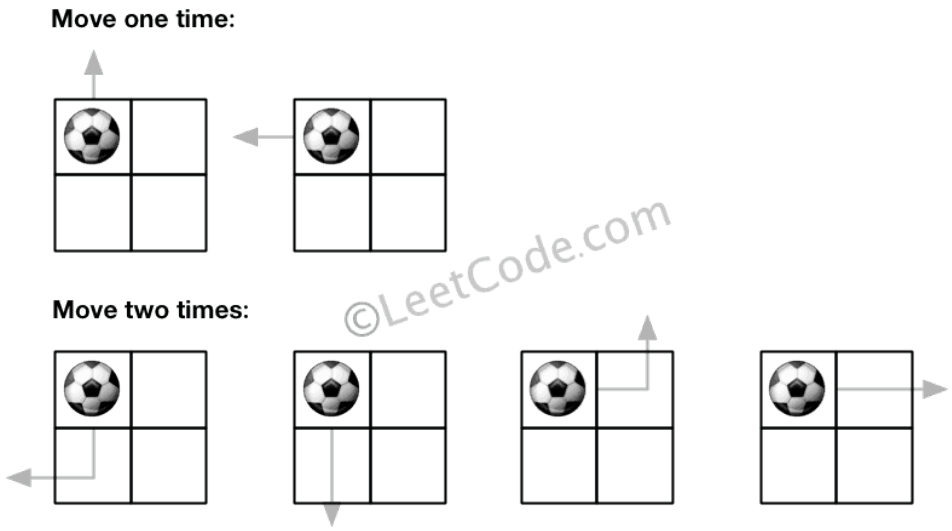
Also appears in pattern 36 Simulation / Array Rearrangement

PROBLEM

There is an `m x n` grid with a ball. The ball is initially at the position `[startRow, startColumn]`. You are allowed to move the ball to one of the four adjacent cells in the grid (possibly out of the grid crossing the grid boundary). You can apply **at most** `maxMove` moves to the ball.

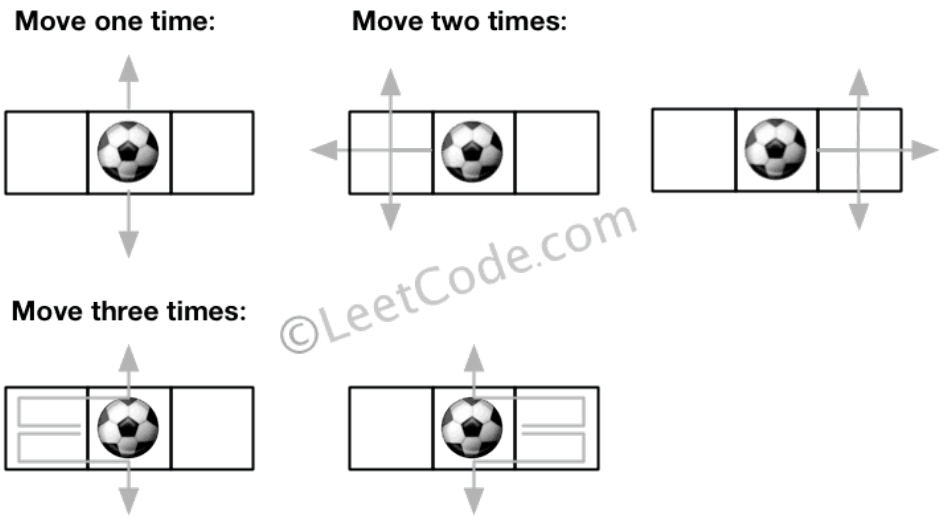
Given the five integers `m`, `n`, `maxMove`, `startRow`, `startColumn`, return the number of paths to move the ball out of the grid boundary. Since the answer can be very large, return it **modulo** $10^9 + 7$.

Example 1:




```
Input: m = 2, n = 2, maxMove = 2, startRow = 0, startColumn = 0
Output: 6
```

Example 2:




```
Input: m = 1, n = 3, maxMove = 3, startRow = 0, startColumn = 1
Output: 12
```

Constraints:

- `1 <= m, n <= 50`
- `0 <= maxMove <= 50`
- `0 <= startRow < m`
- `0 <= startColumn < n`

SOLUTION (PYTHON 3)

Solution 1: Memoization Search

How to think about it

From a cell we may step in four directions, at most k times, and we count paths that leave the grid. Unmemoized search repeats the same cell and remaining steps. dfs(i,j,k) is the number of out-of-bound paths from (i,j) with k steps left. Off-grid with k ≥ 0 scores 1; no steps left scores 0. Four-way transitions modulo 10^9+7. Each triple is computed once.

We define a function dfs(i, j, k) to represent the number of paths that can move out of the boundary starting from coordinates (i, j) with k steps remaining.

In the function dfs(i, j, k), we first handle the boundary cases. If the current coordinates (i, j) are out of the grid range, return 1 if k ≥ 0, otherwise return 0. If k ≤ 0, it means we are still within the grid but have no remaining moves, so return 0. Next, we iterate over the four directions, move to the next coordinates (x, y), then recursively call dfs(x, y, k - 1), and accumulate the results to the answer.

In the main function, we call dfs(startRow, startColumn, maxMove), which represents the number of paths that can move out of the boundary starting from the initial coordinates (startRow, startColumn) with maxMove steps remaining.

To avoid redundant calculations, we can use memoization.

The time complexity is O(m × n × k), and the space complexity is O(m × n × k). Here, m and n are the number of rows and columns of the grid, and k is the number of steps that can be moved, with k = maxMove ≤ 50.

Python 3

```python
class Solution:
    def findPaths(
        self, m: int, n: int, maxMove: int, startRow: int, startColumn: int
    ) -> int:
        @cache
        def dfs(i: int, j: int, k: int) -> int:
            if not 0 <= i < m or not 0 <= j < n:
                return int(k >= 0)
            if k <= 0:
                return 0
            ans = 0
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                ans = (ans + dfs(x, y, k - 1)) % mod
            return ans

        mod = 10**9 + 7
        dirs = (-1, 0, 1, 0, -1)
        return dfs(startRow, startColumn, maxMove)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count the paths that are still INSIDE the grid after
# exactly k moves
# 2. Count the paths that leave the grid at exactly move k
# (not earlier)


# ============================================================
# FOLLOW-UP 1: Count the paths that are still INSIDE the
# grid after exactly k moves
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same table, but count positions inside the
# grid after the last move instead of paths that left it.
# Time: O(k * m * n).  Space: O(m * n).
def paths_ending_inside(m, n, max_move, start_row, start_col):
    mod = 10 ** 9 + 7
    ways = []
    for r in range(m):
        ways.append([0] * n)
    ways[start_row][start_col] = 1
    for step in range(max_move):
        new_ways = []
        for r in range(m):
            new_ways.append([0] * n)
        for r in range(m):
            for c in range(n):
                if ways[r][c] == 0:
                    continue
                for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
                    if 0 <= nr < m and 0 <= nc < n:
                        new_ways[nr][nc] = (new_ways[nr][nc] + ways[r][c]) % mod
        ways = new_ways
    total = 0
    for r in range(m):
        for c in range(n):
            total = (total + ways[r][c]) % mod
    return total


# ============================================================
# FOLLOW-UP 2: Count the paths that leave the grid at
```

```
# exactly move k (not earlier)
# ============================================================

# ---- follow-up 2: function ----
# Answer: The ball must stay inside for k - 1 moves and
# leave on the last one: stop counting a path as soon as
# it leaves.
# Time: O(k * m * n).  Space: O(m * n).
def paths_leaving_at_move(m, n, moves, start_row, start_col):
    mod = 10 ** 9 + 7
    ways = []
    for r in range(m):
        ways.append([0] * n)
    ways[start_row][start_col] = 1
    for step in range(moves - 1):
        new_ways = []
        for r in range(m):
```

```
            new_ways.append([0] * n)
        for r in range(m):
            for c in range(n):
                for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
                    if 0 <= nr < m and 0 <= nc < n:
                        new_ways[nr][nc] = (new_ways[nr][nc] + ways[r][c]) % mod
        ways = new_ways
    total = 0
    for r in range(m):
        for c in range(n):
            for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
                if not (0 <= nr < m and 0 <= nc < n):
                    total = (total + ways[r][c]) % mod
    return total
```

72. Edit Distance

Medium · String, Dynamic Programming

leetcode.com/problems/edit-distance

★ **TOP 7 PROBLEM** in Dynamic Programming - 2D / Grid: master this one first

PROBLEM

Given two strings `word1` and `word2`, return *the minimum number of operations required to convert* `word1` *to* `word2`.

You have the following three operations permitted on a word:

- Insert a character
- Delete a character
- Replace a character

Example 1:

```
Input: word1 = "horse", word2 = "ros"
Output: 3
Explanation:
horse -> rorse (replace 'h' with 'r')
rorse -> rose (remove 'r')
rose -> ros (remove 'e')
```

Example 2:

```
Input: word1 = "intention", word2 = "execution"
Output: 5
Explanation:
intention -> inention (remove 't')
inention -> enention (replace 'i' with 'e')
enention -> exention (replace 'n' with 'x')
exention -> exection (replace 'n' with 'c')
exection -> execution (insert 'u')
```

Constraints:

- `0 <= word1.length, word2.length <= 500`
- `word1` and `word2` consist of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

The first idea is recursion: at each position of word1 try insert, delete, or replace, then take the best. Correct, but exponential. m,n ≤ 500 needs a polynomial. The bottleneck is overlapping subproblems — converting prefix i into prefix j is solved many times. Let f[i][j] be that minimum. Equal last characters inherit the diagonal; otherwise the three operations map to three neighbors, take min plus one. Empty-string borders are delete-all / insert-all. The answer is f[m][n].

We define f[i][j] as the minimum number of operations to convert word1 of length i to word2 of length j. f[i][0] = i, f[0][j] = j, $i \in [1, m]$, $j \in [0, n]$.

We consider f[i][j]:

- If word1[i - 1] = word2[j - 1], then we only need to consider the minimum number of operations to convert word1 of length i - 1 to word2 of length j - 1, so f[i][j] = f[i - 1][j - 1];

- Otherwise, we can consider insert, delete, and replace operations, then f[i][j] = min(f[i - 1][j], f[i][j - 1], f[i - 1][j - 1]) + 1.

Finally, we can get the state transition equation:

f[i][j] = cases i, & if j = 0 \ j, & if i = 0 \ f[i - 1][j - 1], & if word1[i - 1] = word2[j - 1] \ min(f[i - 1][j], f[i][j - 1], f[i - 1][j - 1]) + 1, & otherwise cases

Finally, we return f[m][n].

The time complexity is O(m × n), and the space complexity is O(m × n). m and n are the lengths of word1 and word2 respectively.

Python 3

```
class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for j in range(1, n + 1):
            f[0][j] = j
        for i, a in enumerate(word1, 1):
            f[i][0] = i
            for j, b in enumerate(word2, 1):
                if a == b:
                    f[i][j] = f[i - 1][j - 1]
                else:
                    f[i][j] = min(f[i - 1][j], f[i][j - 1], f[i - 1][j - 1]) + 1
        return f[m][n]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the sequence of edits
# 2. Only INSERT and DELETE are allowed (no replace)


# ============================================================
# FOLLOW-UP 1: Return the sequence of edits
# ============================================================

# ---- follow-up 1: function ----
# Answer: Walk back through the table of distances: a
# diagonal step is a match or a replace, a step up is a
# delete, a step left is an insert.
# Time: O(a * b).  Space: O(a * b).
def edit_operations(a, b):
    table = []
    for i in range(len(a) + 1):
        table.append([0] * (len(b) + 1))
    for i in range(len(a) + 1):
        table[i][0] = i
    for j in range(len(b) + 1):
        table[0][j] = j
    for i in range(1, len(a) + 1):
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                table[i][j] = table[i - 1][j - 1]
            else:
                best = table[i - 1][j - 1]
                if table[i - 1][j] < best:
                    best = table[i - 1][j]
                if table[i][j - 1] < best:
                    best = table[i][j - 1]
                table[i][j] = best + 1
    operations = []
    i, j = len(a), len(b)
    while i > 0 or j > 0:
        if i > 0 and j > 0 and a[i - 1] == b[j - 1]:
```

```
            i -= 1
            j -= 1
        elif i > 0 and j > 0 and table[i][j] == table[i - 1][j - 1] + 1:
            operations.append("replace " + a[i - 1] + " with " + b[j - 1])
            i -= 1
            j -= 1
        elif i > 0 and table[i][j] == table[i - 1][j] + 1:
            operations.append("delete " + a[i - 1])
            i -= 1
        else:
            operations.append("insert " + b[j - 1])
            j -= 1
    operations.reverse()
    return table[len(a)][len(b)], operations


# ============================================================
# FOLLOW-UP 2: Only INSERT and DELETE are allowed (no
# replace)
# ============================================================

# ---- follow-up 2: function ----
# Answer: The distance is the lengths minus twice the
```

```
# longest common subsequence.
# Time: O(a * b).  Space: O(b).
def edit_distance_insert_delete(a, b):
    previous = [0] * (len(b) + 1)
    for i in range(1, len(a) + 1):
        current = [0] * (len(b) + 1)
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                current[j] = previous[j - 1] + 1
            elif previous[j] >= current[j - 1]:
                current[j] = previous[j]
            else:
                current[j] = current[j - 1]
        previous = current
    return len(a) + len(b) - 2 * previous[len(b)]
```

115. Distinct Subsequences

Hard · String, Dynamic Programming

leetcode.com/problems/distinct-subsequences

★ **TOP 7 PROBLEM** in Dynamic Programming - 2D / Grid: master this one first

PROBLEM

Given two strings s and t, return *the number of distinct* ***subsequences*** *of* s *which equals* t.

The test cases are generated so that the answer fits on a 32-bit signed integer.

Example 1:

```
Input: s = "rabbbit", t = "rabbit"
Output: 3
Explanation:
As shown below, there are 3 ways you can generate "rabbit" from s.
rabbbit
rabbbit
rabbbit
```

Example 2:

```
Input: s = "babgbag", t = "bag"
Output: 5
Explanation:
As shown below, there are 5 ways you can generate "bag" from s.
babgbag
babgbag
babgbag
babgbag
babgbag
```

Constraints:

- `1 <= s.length, t.length <= 1000`
- `s` and `t` consist of English letters.

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

Count subsequences of s that equal t. Choosing to take or skip each character of s explodes when both lengths reach 1000; naive recursion repeats the same suffix pairs. The state is how much of s and t we have used. f[i][j] is the number of ways the first i characters of s form the first j of t. When the current characters match, we add the transition that consumes both. The empty t has exactly one empty subsequence.

We define f[i][j] as the number of schemes where the first i characters of string s form the first j characters of string t. Initially, f[i][0]=1 for all $i \in [0,m]$.

When i > 0, we consider the calculation of f[i][j]:

- When s[i-1] ≠ t[j-1], we cannot select s[i-1], so f[i][j]=f[i-1][j];
- Otherwise, we can select s[i-1], so f[i][j]=f[i-1][j-1].

Therefore, we have the following state transition equation:

$$f[i][j]=\{ \text{aligned } \&f[i-1][j], \&s[i-1] \neq t[j-1] \backslash \&f[i-1][j-1]+f[i-1][j], \&s[i-1]=t[j-1] \text{ aligned } .$$

The final answer is f[m][n], where m and n are the lengths of strings s and t respectively.

The time complexity is O(m × n), and the space complexity is O(m × n).

Python 3

```python
class Solution:
    def numDistinct(self, s: str, t: str) -> int:
        m, n = len(s), len(t)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(m + 1):
            f[i][0] = 1
        for i, a in enumerate(s, 1):
            for j, b in enumerate(t, 1):
                f[i][j] = f[i - 1][j]
                if a == b:
                    f[i][j] += f[i - 1][j - 1]
        return f[m][n]
```

Solution 2: Optimized Dynamic Programming

> **How to think about it**
>
> In Solution 1, f[i][j] depends only on the previous row. After collapsing to one dimension we must update j from the right, so f[j-1] is not overwritten too early. Space becomes O(|t|).

f[i][j] depends only on the previous row, so the first dimension can be dropped. Update j from large to small so f[j-1] is not overwritten. The space complexity becomes O(n).

Python 3

```python
class Solution:
    def numDistinct(self, s: str, t: str) -> int:
        n = len(t)
        f = [1] + [0] * n
        for a in s:
            for j in range(n, 0, -1):
                if a == t[j - 1]:
                    f[j] += f[j - 1]
        return f[n]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. O(len(t)) extra space
# 2. Return the positions of one occurrence of t inside s
# as a subsequence


# ============================================================
# FOLLOW-UP 1: O(len(t)) extra space
# ============================================================

# ---- follow-up 1: function ----
# Answer: Only the previous row is needed: update one row
# from the back so that every cell still sees the old
# values it needs.
# Time: O(s * t).  Space: O(t).
def num_distinct_one_row(s, t):
    ways = [0] * (len(t) + 1)
    ways[0] = 1
    for ch in s:
        for j in range(len(t), 0, -1):
            if t[j - 1] == ch:
                ways[j] += ways[j - 1]
    return ways[len(t)]
```

```python
# ============================================================
# FOLLOW-UP 2: Return the positions of one occurrence of t
# inside s as a subsequence
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk back through the table of counts and pick a
# letter of s whenever it can still lead to a complete
# match.
# Time: O(s * t).  Space: O(s * t).
def one_occurrence_positions(s, t):
    table = []
    for i in range(len(s) + 1):
        table.append([0] * (len(t) + 1))
    for i in range(len(s) + 1):
        table[i][0] = 1
    for i in range(1, len(s) + 1):
        for j in range(1, len(t) + 1):
            table[i][j] = table[i - 1][j]
            if s[i - 1] == t[j - 1]:
                table[i][j] += table[i - 1][j - 1]
    if table[len(s)][len(t)] == 0:
        return None
    positions = []
    i, j = len(s), len(t)
    while j > 0:
        if s[i - 1] == t[j - 1] and table[i - 1][j - 1] > 0:
            positions.append(i - 1)
            j -= 1
        i -= 1
    positions.reverse()
    return positions
```

174. Dungeon Game

Hard · Array, Dynamic Programming, Matrix

leetcode.com/problems/dungeon-game

PROBLEM

The demons had captured the princess and imprisoned her in **the bottom-right corner** of a `dungeon`. The `dungeon` consists of `m x n` rooms laid out in a 2D grid. Our valiant knight was initially positioned in **the top-left room** and must fight his way through `dungeon` to rescue the princess.

The knight has an initial health point represented by a positive integer. If at any point his health point drops to `0` or below, he dies immediately.

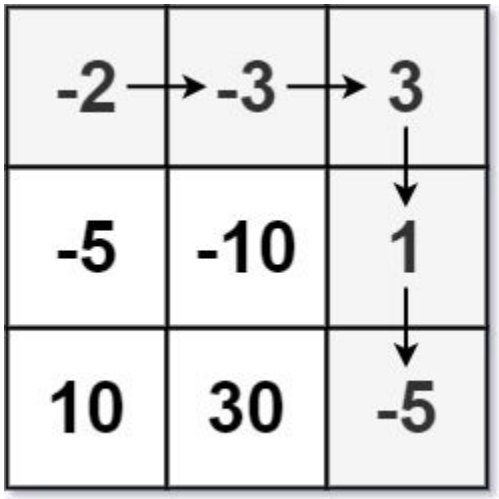
Some of the rooms are guarded by demons (represented by negative integers), so the knight loses health upon entering these rooms; other rooms are either empty (represented as 0) or contain magic orbs that increase the knight's health (represented by positive integers).

To reach the princess as quickly as possible, the knight decides to move only **rightward** or **downward** in each step.

Return *the knight's minimum initial health so that he can rescue the princess*.

Note that any room can contain threats or power-ups, even the first room the knight enters and the bottom-right room where the princess is imprisoned.

Example 1:




```
Input: dungeon = [[-2,-3,3],[-5,-10,1],[10,30,-5]]
Output: 7
Explanation: The initial health of the knight must be at least 7 if he follows the optimal path:
RIGHT-> RIGHT -> DOWN -> DOWN.
```

Example 2:

```
Input: dungeon = [[0]]
Output: 1
```

Constraints:

- `m == dungeon.length`
- `n == dungeon[i].length`
- `1 <= m, n <= 200`
- `-1000 <= dungeon[i][j] <= 1000`

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

> **How to think about it**
>
> The knight may only move right or down and must stay at least at 1 HP; we want the minimum starting HP. Enumerating paths on a 200× 200 grid is impossible. The HP needed ahead fixes the requirement here, so we work backward from the princess: next-cell need minus this cell's value, then at least 1. Take the cheaper of the two exits.

We define dp[i][j] as the minimum initial value needed from (i, j) to the end point. The value of dp[i][j] can be obtained from dp[i+1][j] and dp[i][j+1], that is:

$$dp[i][j] = \max(\min(dp[i+1][j], dp[i][j+1]) - dungeon[i][j], 1)$$

Initially, dp[m][n-1] and dp[m-1][n] are both 1, and the values at other positions are maximum.

The time complexity is O(m × n), and the space complexity is O(m × n). Where m and n are the number of rows and columns of the dungeon, respectively.

Python 3

```python
class Solution:
    def calculateMinimumHP(self, dungeon: List[List[int]]) -> int:
        m, n = len(dungeon), len(dungeon[0])
        dp = [[inf] * (n + 1) for _ in range(m + 1)]
        dp[m][n - 1] = dp[m - 1][n] = 1
        for i in range(m - 1, -1, -1):
            for j in range(n - 1, -1, -1):
                dp[i][j] = max(1, min(dp[i + 1][j], dp[i][j + 1]) - dungeon[i][j])
        return dp[0][0]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the path the knight should take
# 2. The knight's health can never be above a limit H
# (healing is capped)


# ============================================================
# FOLLOW-UP 1: Return the path the knight should take
# ============================================================

# ---- follow-up 1: function ----
# Answer: Fill the table of the minimum health needed from
# each room to the end, then walk from the start towards
# the neighbour that needs less health.
# Time: O(m * n).  Space: O(m * n).
def dungeon_path(dungeon):
    rows, cols = len(dungeon), len(dungeon[0])
    big = float("inf")
    need = []
    for r in range(rows + 1):
        need.append([big] * (cols + 1))
    need[rows][cols - 1] = 1
    need[rows - 1][cols] = 1
    for r in range(rows - 1, -1, -1):
        for c in range(cols - 1, -1, -1):
            smaller = need[r + 1][c] if need[r + 1][c] < need[r][c + 1] else need[r][c + 1]
            value = smaller - dungeon[r][c]
            need[r][c] = value if value > 1 else 1
    path = [(0, 0)]
    r = c = 0
    while (r, c) != (rows - 1, cols - 1):
        if need[r + 1][c] <= need[r][c + 1]:
```

```
            r += 1
        else:
            c += 1
        path.append((r, c))
    return need[0][0], path


# ============================================================
# FOLLOW-UP 2: The knight's health can never be above a
# limit H (healing is capped)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Now the walk forward matters: track the best
# health at each room as the largest health you can have
# when arriving, starting from H, and check whether the
# end is reachable alive.
# Time: O(m * n).  Space: O(m * n).
def can_survive_with_cap(dungeon, start_health, cap):
    rows, cols = len(dungeon), len(dungeon[0])
    best = []
    for r in range(rows):
        best.append([0] * cols)
    for r in range(rows):
        for c in range(cols):
            if r == 0 and c == 0:
                arriving = start_health
            else:
                arriving = 0
                if r > 0:
                    arriving = best[r - 1][c]
                if c > 0 and best[r][c - 1] > arriving:
                    arriving = best[r][c - 1]
            if arriving <= 0:
                best[r][c] = 0
                continue
            health = arriving + dungeon[r][c]
            if health > cap:
                health = cap
            best[r][c] = health if health > 0 else 0
    return best[rows - 1][cols - 1] > 0
```

741. Cherry Pickup

Hard · Array, Dynamic Programming, Matrix

leetcode.com/problems/cherry-pickup

PROBLEM

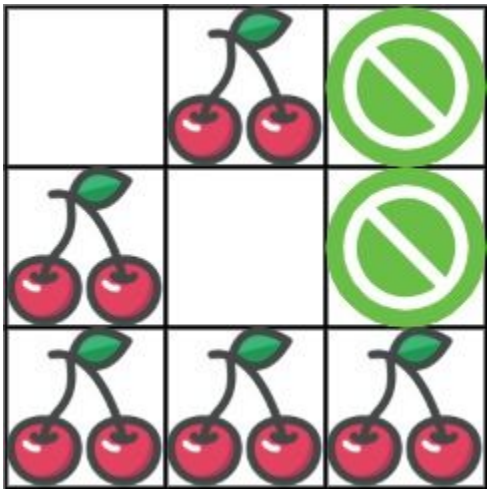
You are given an `n x n grid` representing a field of cherries, each cell is one of three possible integers.

- `0` means the cell is empty, so you can pass through,
- `1` means the cell contains a cherry that you can pick up and pass through, or
- `-1` means the cell contains a thorn that blocks your way.

Return *the maximum number of cherries you can collect by following the rules below*:

- Starting at the position `(0, 0)` and reaching `(n - 1, n - 1)` by moving right or down through valid path cells (cells with value `0` or `1`).
- After reaching `(n - 1, n - 1)`, returning to `(0, 0)` by moving left or up through valid path cells.
- When passing through a path cell containing a cherry, you pick it up, and the cell becomes an empty cell `0`.
- If there is no valid path between `(0, 0)` and `(n - 1, n - 1)`, then no cherries can be collected.

Example 1:



```
Input: grid = [[0,1,-1],[1,0,-1],[1,1,1]]
Output: 5
Explanation: The player started at (0, 0) and went down, down, right right to reach (2, 2).
4 cherries were picked up during this single trip, and the matrix becomes [[0,1,-1],[0,0,-1],[0,0,0]].
Then, the player went left, up, up, left to return home, picking up one more cherry.
The total number of cherries picked up is 5, and this is the maximum possible.
```

Example 2:

```
Input: grid = [[1,1,-1],[1,-1,1],[-1,1,1]]
Output: 0
```

Constraints:

- `n == grid.length`
- `n == grid[i].length`
- `1 <= n <= 50`
- `grid[i][j]` is `-1`, `0`, or `1`.
- `grid[0][0] != -1`

- `grid[n - 1][n - 1] != -1`

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

Walk to the far corner and back; n≤ 50. Two independent paths explode, and overlapping cherries are hard to charge only once. The round trip is two walks of k steps from the start. With a shared step count, columns are determined by rows, so the state is (k,i_1,i_2). A shared cell is counted once; a thorn is unreachable. $f[k][i_1][i_2]$ comes from the previous rows i_1 or i_1-1 and i_2 or i_2-1, plus the current cherries. The answer is max(0, f[2n-2][n-1][n-1]).

According to the problem description, the player starts from (0, 0), reaches (n-1, n-1), and then returns to the starting point (0, 0). We can consider the player as starting from (0, 0) to (n-1, n-1) twice.

Therefore, we define $f[k][i_1][i_2]$ as the maximum number of cherries that can be picked when both have walked k steps and reached $(i_1, k\text{-}i_1)$ and $(i_2, k\text{-}i_2)$ respectively. Initially, f[0][0][0] = grid[0][0]. The initial values of other $f[k][i_1][i_2]$ are negative infinity. The answer is max(0, f[2n-2][n-1][n-1]).

According to the problem description, we can get the state transition equation:

$$f[k][i_1][i_2] = \max(f[k-1][x_1][x_2] + t, f[k][i_1][i_2])$$

Where t represents the number of cherries at positions $(i_1, k\text{-}i_1)$ and $(i_2, k\text{-}i_2)$, and x_1, x_2 represent the previous step positions of $(i_1, k\text{-}i_1)$ and $(i_2, k\text{-}i_2)$ respectively.

The time complexity is $O(n^3)$, and the space complexity is $O(n^3)$. Where n is the side length of the grid.

Python 3

```python
class Solution:
    def cherryPickup(self, grid: List[List[int]]) -> int:
        n = len(grid)
        f = [[[-inf] * n for _ in range(n)] for _ in range((n << 1) - 1)]
        f[0][0][0] = grid[0][0]
        for k in range(1, (n << 1) - 1):
            for i1 in range(n):
                for i2 in range(n):
                    j1, j2 = k - i1, k - i2
                    if (
                        not 0 <= j1 < n
                        or not 0 <= j2 < n
                        or grid[i1][j1] == -1
                        or grid[i2][j2] == -1
                    ):
                        continue
                    t = grid[i1][j1]
                    if i1 != i2:
                        t += grid[i2][j2]
                    for x1 in range(i1 - 1, i1 + 1):
                        for x2 in range(i2 - 1, i2 + 1):
                            if x1 >= 0 and x2 >= 0:
                                f[k][i1][i2] = max(f[k][i1][i2], f[k - 1][x1][x2] + t)
        return max(0, f[-1][-1][-1])
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Only the trip to the bottom-right corner (no return
# trip): the most cherries
# 2. The path of the one-way trip


# ============================================================
# FOLLOW-UP 1: Only the trip to the bottom-right corner
# (no return trip): the most cherries
# ============================================================

# ---- follow-up 1: function ----
# Answer: A simple table: best[r][c] = cherries[r][c] +
# the better of the cell above and the cell to the left
# (thorns -1 block the way).
# Time: O(n^2).  Space: O(n^2).
def cherries_one_way(grid):
    n = len(grid)
    best = []
    for r in range(n):
        best.append([-1] * n)
    for r in range(n):
        for c in range(n):
            if grid[r][c] == -1:
                continue
            if r == 0 and c == 0:
                best[r][c] = grid[r][c]
                continue
            previous = -1
            if r > 0 and best[r - 1][c] > previous:
                previous = best[r - 1][c]
            if c > 0 and best[r][c - 1] > previous:
                previous = best[r][c - 1]
            if previous >= 0:
                best[r][c] = previous + grid[r][c]
    return best[n - 1][n - 1]


# ============================================================
# FOLLOW-UP 2: The path of the one-way trip
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk back from the corner through the neighbour
# with the larger table value.
# Time: O(n^2).  Space: O(n^2).
def cherry_path_one_way(grid):
    n = len(grid)
    best = []
    for r in range(n):
        best.append([-1] * n)
    for r in range(n):
        for c in range(n):
            if grid[r][c] == -1:
                continue
            if r == 0 and c == 0:
                best[r][c] = grid[r][c]
                continue
            previous = -1
            if r > 0 and best[r - 1][c] > previous:
```

```
                previous = best[r - 1][c]
            if c > 0 and best[r][c - 1] > previous:
                previous = best[r][c - 1]
            if previous >= 0:
```

```
                best[r][c] = previous + grid[r][c]
    if best[n - 1][n - 1] < 0:
        return None
    path = [(n - 1, n - 1)]
    r = c = n - 1
    while (r, c) != (0, 0):
        up = best[r - 1][c] if r > 0 else -1
        left = best[r][c - 1] if c > 0 else -1
        if up >= left:
            r -= 1
        else:
            c -= 1
        path.append((r, c))
    path.reverse()
    return path
```

1463. Cherry Pickup II

Hard · Array, Dynamic Programming, Matrix

leetcode.com/problems/cherry-pickup-ii

PROBLEM

You are given a `rows x cols` matrix `grid` representing a field of cherries where `grid[i][j]` represents the number of cherries that you can collect from the `(i, j)` cell.

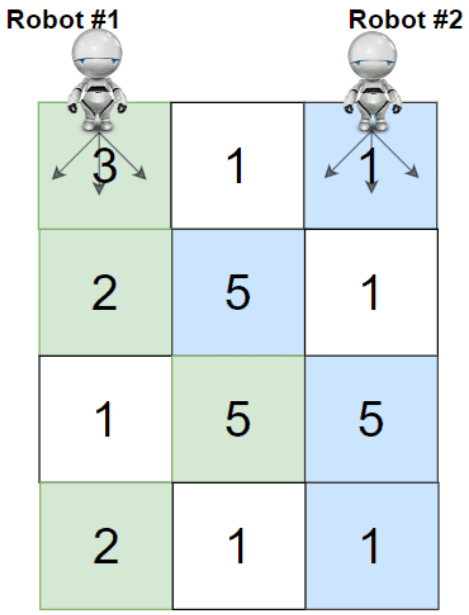
You have two robots that can collect cherries for you:

- **Robot #1** is located at the **top-left corner** `(0, 0)`, and
- **Robot #2** is located at the **top-right corner** `(0, cols - 1)`.

Return *the maximum number of cherries collection using both robots by following the rules below*:

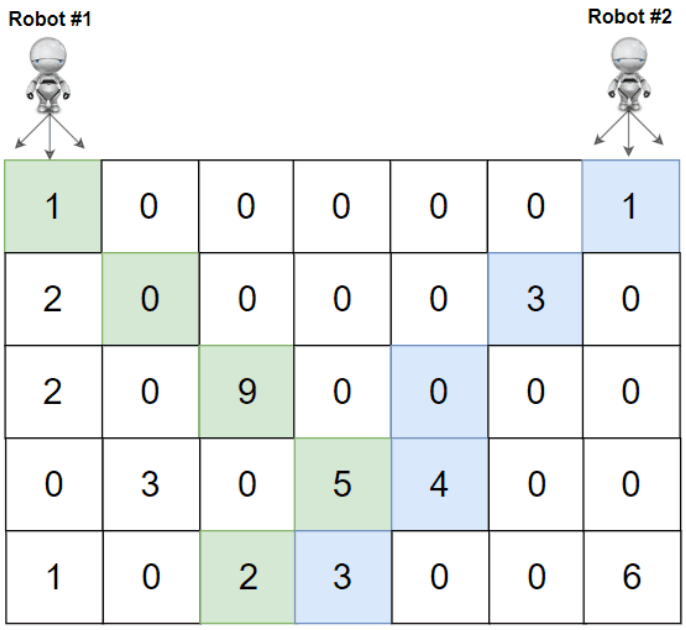
- From a cell `(i, j)`, robots can move to cell `(i + 1, j - 1)`, `(i + 1, j)`, or `(i + 1, j + 1)`.
- When any robot passes through a cell, It picks up all cherries, and the cell becomes an empty cell.
- When both robots stay in the same cell, only one takes the cherries.
- Both robots cannot move outside of the grid at any moment.
- Both robots should reach the bottom row in `grid`.

Example 1:




```
Input: grid = [[3,1,1],[2,5,1],[1,5,5],[2,1,1]]
Output: 24
Explanation: Path of robot #1 and #2 are described in color green and blue respectively.
Cherries taken by Robot #1, (3 + 2 + 5 + 2) = 12.
Cherries taken by Robot #2, (1 + 5 + 5 + 1) = 12.
Total of cherries: 12 + 12 = 24.
```

Example 2:




```
Input: grid = [[1,0,0,0,0,0,1],[2,0,0,0,0,3,0],[2,0,9,0,0,0,0],[0,3,0,5,4,0,0],[1,0,2,3,0,0,6]]
Output: 28
Explanation: Path of robot #1 and #2 are described in color green and blue respectively.
Cherries taken by Robot #1, (1 + 9 + 5 + 2) = 17.
Cherries taken by Robot #2, (1 + 3 + 4 + 3) = 11.
Total of cherries: 17 + 11 = 28.
```

Constraints:

- `rows == grid.length`
- `cols == grid[i].length`
- `2 <= rows, cols <= 70`
- `0 <= grid[i][j] <= 100`

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

Two robots move down a row together, each stepping left/stay/right. rows,cols≤ 70. After synchronizing rows, the state is the row and the two columns. f[i][j_1][j_2] is the best score with both robots on row i. A shared cell is counted once. Transition from the nine neighboring column pairs. Take the max on the last row.

We define f[i][j_1][j_2] as the maximum number of cherries that can be picked when the two robots are at positions j_1 and j_2 in the i-th row. Initially, f[0][0][n-1] = grid[0][0] + grid[0][n-1], and the other values are -1. The answer is $\max_{0 \le j_1, j_2 < n}$ f[m-1][j_1][j_2].

Consider f[i][j_1][j_2]. If $j_1 \neq j_2$, then the number of cherries that the robots can pick in the i-th row is grid[i][j_1] + grid[i][j_2]. If $j_1 = j_2$, then the number of cherries that the robots can pick in the i-th row is grid[i][j_1]. We can enumerate the previous state of the two robots f[i-1][y1][y2], where y_1, y_2 are the positions of the two robots in the (i-1)-th row, then $y_1 \in \{j_1-1, j_1, j_1+1\}$ and $y_2 \in \{j_2-1, j_2, j_2+1\}$. The state transition equation is as follows:

$$f[i][j_1][j_2] = \max_{y_1 \in \{j_1-1, j_1, j_1+1\},\ y_2 \in \{j_2-1, j_2, j_2+1\}} f[i-1][y_1][y_2] + \begin{cases} grid[i][j_1] + grid[i][j_2], & j_1 \neq j_2 \\ grid[i][j_1], & j_1 = j_2 \end{cases}$$

Where f[i-1][y_1][y_2] is ignored when it is -1.

The final answer is $\max_{0 \le j_1, j_2 < n}$ f[m-1][j_1][j_2].

The time complexity is $O(m \times n^2)$, and the space complexity is $O(m \times n^2)$. Where m and n are the number of rows and columns of the grid, respectively.

Python 3

```python
class Solution:
    def cherryPickup(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        f = [[[-1] * n for _ in range(n)] for _ in range(m)]
        f[0][0][n - 1] = grid[0][0] + grid[0][n - 1]
        for i in range(1, m):
            for j1 in range(n):
                for j2 in range(n):
                    x = grid[i][j1] + (0 if j1 == j2 else grid[i][j2])
                    for y1 in range(j1 - 1, j1 + 2):
                        for y2 in range(j2 - 1, j2 + 2):
                            if 0 <= y1 < n and 0 <= y2 < n and f[i - 1][y1][y2] != -1:
                                f[i][j1][j2] = max(f[i][j1][j2], f[i - 1][y1][y2] + x)
        return max(f[-1][j1][j2] for j1, j2 in product(range(n), range(n)))
```

Solution 2: Dynamic Programming (Space Optimization)

> **How to think about it**
>
> Layer i of method 1 depends only on layer i-1. Two n× n tables rolled in place drop a dimension without changing the transitions.

Notice that the calculation of f[i][j_1][j_2] is only related to f[i-1][y_1][y_2]. Therefore, we can use a rolling array to optimize the space complexity. After optimizing the space complexity, the time complexity is $O(n^2)$.

Python 3

```python
class Solution:
    def cherryPickup(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        f = [[-1] * n for _ in range(n)]
        g = [[-1] * n for _ in range(n)]
        f[0][n - 1] = grid[0][0] + grid[0][n - 1]
        for i in range(1, m):
            for j1 in range(n):
                for j2 in range(n):
                    x = grid[i][j1] + (0 if j1 == j2 else grid[i][j2])
                    for y1 in range(j1 - 1, j1 + 2):
                        for y2 in range(j2 - 1, j2 + 2):
                            if 0 <= y1 < n and 0 <= y2 < n and f[y1][y2] != -1:
                                g[j1][j2] = max(g[j1][j2], f[y1][y2] + x)
            f, g = g, f
        return max(f[j1][j2] for j1, j2 in product(range(n), range(n)))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. THREE robots instead of two


# ============================================================
# FOLLOW-UP 1: THREE robots instead of two
# ============================================================

# ---- follow-up 1: function ----
# Answer: The state is the row and the columns of all
# three robots; each moves to one of three columns, and a
# cell's cherries are counted once even if several robots
# stand on it.
# Time: O(rows * cols^3), 27 moves per state.  Space:
# O(cols^3).
def cherry_pickup_three(grid):
    rows, cols = len(grid), len(grid[0])
    best = {}
```

```
    best[(0, cols - 1, cols // 2)] = None

    def gain(r, positions):
        total = 0
        for c in set(positions):
            total += grid[r][c]
        return total

    current = {(0, cols - 1, cols // 2): gain(0, (0, cols - 1, cols // 2))}
    for r in range(1, rows):
        following = {}
        for (a, b, c), value in current.items():
            for da in (-1, 0, 1):
                for db in (-1, 0, 1):
                    for dc in (-1, 0, 1):
                        na, nb, nc = a + da, b + db, c + dc
                        if 0 <= na < cols and 0 <= nb < cols and 0 <= nc < cols:
                            total = value + gain(r, (na, nb, nc))
                            if (na, nb, nc) not in following or total > following[(na, nb, nc)]:
                                following[(na, nb, nc)] = total
        current = following
    best_value = 0
    for value in current.values():
        if value > best_value:
            best_value = value
    return best_value
```

1289. Minimum Falling Path Sum II

Hard · Array, Dynamic Programming, Matrix

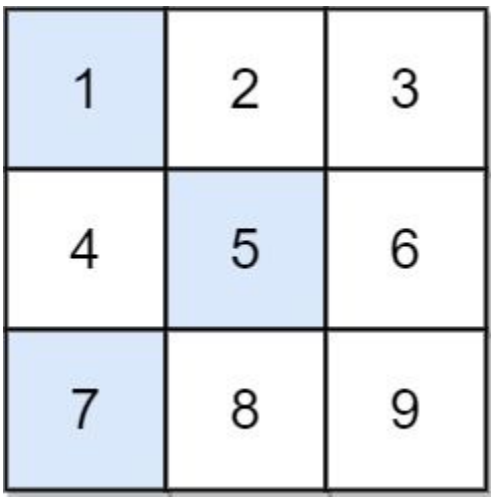
leetcode.com/problems/minimum-falling-path-sum-ii

PROBLEM

Given an `n x n` integer matrix `grid`, return *the minimum sum of a **falling path with non-zero shifts***.

A **falling path with non-zero shifts** is a choice of exactly one element from each row of `grid` such that no two elements chosen in adjacent rows are in the same column.

Example 1:




```
Input: grid = [[1,2,3],[4,5,6],[7,8,9]]
Output: 13
Explanation:
The possible falling paths are:
[1,5,9], [1,5,7], [1,6,7], [1,6,8],
[2,4,8], [2,4,9], [2,6,7], [2,6,8],
[3,4,8], [3,4,9], [3,5,7], [3,5,9]
The falling path with the smallest sum is [1,5,7], so the answer is 13.
```

Example 2:

```
Input: grid = [[7]]
Output: 7
```

Constraints:

- `n == grid.length == grid[i].length`
- `1 <= n <= 200`
- `-99 <= grid[i][j] <= 99`

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming (Rolling Array)

How to think about it

A falling path cannot reuse a column on the next row. n ≤ 200 allows $O(n^3)$. The best way to end row i in column j is the previous row's minimum excluding j, plus grid[i][j]. We keep only the previous n values and add “min except this column” in place. A rolling array drops the row dimension.

Let f[i][j] be the minimum path sum using the first i rows and ending in column j:

$$f[i][j] = \min_{k \neq j} f[i - 1][k] + grid[i - 1][j]$$

The answer is $\min_{0 \leq j < n} f[n][j]$. After rolling, only the last row remains, so this is the minimum of the 1D array f.

f[i][j] depends only on the previous row, so we keep two arrays f and g of length n.

The time complexity is $O(n^3)$ and the space complexity is O(n), where n is the number of rows.

Python 3

```
class Solution:
    def minFallingPathSum(self, grid: List[List[int]]) -> int:
        n = len(grid)
        f = [0] * n
        for row in grid:
            g = row[:]
            for i in range(n):
                g[i] += min((f[j] for j in range(n) if j != i), default=0)
            f = g
        return min(f)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the chosen columns of the cheapest path


# ============================================================
# FOLLOW-UP 1: Return the chosen columns of the cheapest
# path
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the two smallest values of each row with
# their columns: a cell takes the smallest value of the
# row above that is not in its own column. Walk back with
# the stored choices.
# Time: O(n^3).  Space: O(n^2).
def falling_path_columns(grid):
    n = len(grid)
    cost = []
    came_from = []
    for r in range(n):
        cost.append([0] * n)
        came_from.append([0] * n)
    for c in range(n):
        cost[0][c] = grid[0][c]
    for r in range(1, n):
        for c in range(n):
            best = None
            for k in range(n):
                if k != c and (best is None or cost[r - 1][k] < cost[r - 1][best]):
                    best = k
            came_from[r][c] = best
            cost[r][c] = grid[r][c] + cost[r - 1][best]
    column = 0
    for c in range(n):
        if cost[n - 1][c] < cost[n - 1][column]:
            column = c
    total = cost[n - 1][column]
    columns = [column]
    for r in range(n - 1, 0, -1):
        column = came_from[r][column]
        columns.append(column)
    columns.reverse()
    return total, columns
```

PATTERN 22 · DYNAMIC PROGRAMMING - 2D / GRID · PROBLEM 20 OF 20

329. Longest Increasing Path in a Matrix

Hard · Depth-First Search, Breadth-First Search, Graph, Topological Sort, Memoization, Array, Dynamic Programming, Directed Acyclic Graph, Matrix

leetcode.com/problems/longest-increasing-path-in-a-matrix

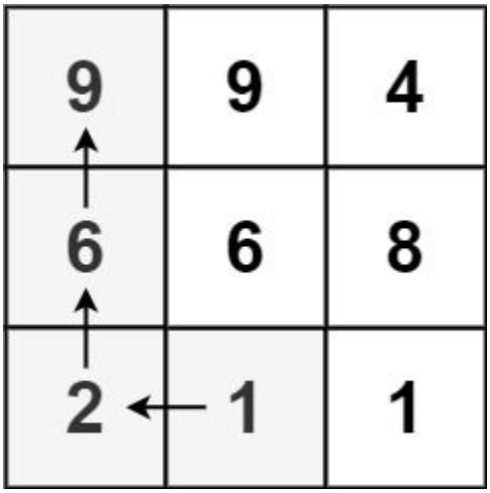
★ **TOP 7 PROBLEM** in Dynamic Programming - 2D / Grid: master this one first

PROBLEM

Given an `m x n` integers `matrix`, return *the length of the longest increasing path in* `matrix`.

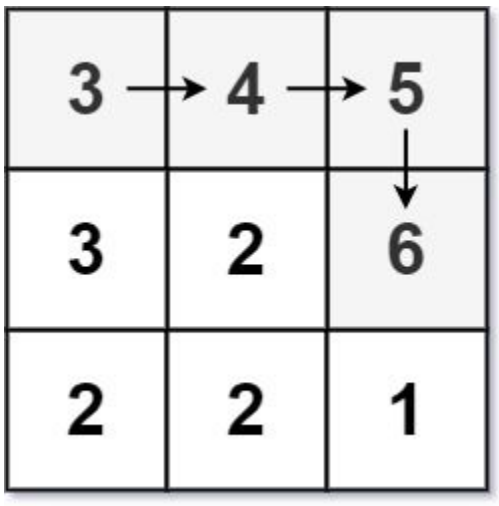
From each cell, you can either move in four directions: left, right, up, or down. You **may not** move **diagonally** or move **outside the boundary** (i.e., wrap-around is not allowed).

Example 1:




```
Input: matrix = [[9,9,4],[6,6,8],[2,1,1]]
Output: 4
Explanation: The longest increasing path is [1, 2, 6, 9].
```

Example 2:




```
Input: matrix = [[3,4,5],[3,2,6],[2,2,1]]
Output: 4
Explanation: The longest increasing path is [3, 4, 5, 6]. Moving diagonally is not allowed.
```

Example 3:

```
Input: matrix = [[1]]
Output: 1
```

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 200`
- `0 <= matrix[i][j] <= 2^31 - 1`

SOLUTION (PYTHON 3)

Solution 1: Memoization Search

> **How to think about it**
>
> Longest strictly increasing four-direction path. Plain DFS restarts from the same cell many times. Strict increase makes the graph acyclic. dfs(i,j) is one plus the best neighbor with a larger value. Memoization evaluates each cell once; the answer is the max over all starts.

We design a function dfs(i, j), which represents the length of the longest increasing path that can be obtained starting from the coordinate (i, j) in the matrix. The answer is $\max_{i,j} \text{dfs}(i, j)$.

The execution logic of the function dfs(i, j) is as follows:

- If (i, j) has been visited, directly return f(i, j);
- Otherwise, search (i, j), search the coordinates (x, y) in four directions. If $0 \le x < m$, $0 \le y < n$ and matrix[x][y] > matrix[i][j], then search (x, y). After the search is over, update f(i, j) to f(i, j) = max(f(i, j), f(x, y) + 1). Finally, return f(i, j).

The time complexity is $O(m \times n)$, and the space complexity is $O(m \times n)$. Where m and n are the number of rows and columns of the matrix, respectively.

Similar problems:

- 2328. Number of Increasing Paths in a Grid

Python 3

```python
class Solution:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            ans = 0
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and matrix[x][y] > matrix[i][j]:
                    ans = max(ans, dfs(x, y))
            return ans + 1

        m, n = len(matrix), len(matrix[0])
        return max(dfs(i, j) for i in range(m) for j in range(n))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the longest increasing path itself
# 2. The numbers on the path must increase by exactly 1
# each step


# ========================================================
# FOLLOW-UP 1: Return the longest increasing path itself
```

```
# ============================================================

# ---- follow-up 1: function ----
# Answer: Memoise the length from every cell, then follow,
# from the best start, the neighbour that continues the
# longest path.
# Time: O(m * n).  Space: O(m * n).
def longest_path_cells(matrix):
    rows, cols = len(matrix), len(matrix[0])
    memo = {}

    def length(r, c):
        if (r, c) in memo:
            return memo[(r, c)]
        best = 1
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and matrix[nr][nc] > matrix[r][c]:
                best = max_of(best, 1 + length(nr, nc))
        memo[(r, c)] = best
        return best

    start = (0, 0)
    for r in range(rows):
        for c in range(cols):
            if length(r, c) > length(start[0], start[1]):
                start = (r, c)
    path = [start]
    r, c = start
    while length(r, c) > 1:
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and matrix[nr][nc] > matrix[r][c] and length(nr, nc) ==
length(r, c) - 1:
                r, c = nr, nc
                path.append((r, c))
                break
    return path


def max_of(a, b):
    if a > b:
        return a
    return b


# ============================================================
# FOLLOW-UP 2: The numbers on the path must increase by
# exactly 1 each step
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same memoised search, but a step to a
# neighbour is only allowed when its value is exactly one
# bigger.
```

```
# Time: O(m * n).  Space: O(m * n).
def longest_consecutive_path(matrix):
    rows, cols = len(matrix), len(matrix[0])
    memo = {}

    def length(r, c):
        if (r, c) in memo:
            return memo[(r, c)]
        best = 1
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and matrix[nr][nc] == matrix[r][c] + 1:
                candidate = 1 + length(nr, nc)
                if candidate > best:
                    best = candidate
        memo[(r, c)] = best
```

```
        return best

    answer = 0
    for r in range(rows):
        for c in range(cols):
            if length(r, c) > answer:
                answer = length(r, c)
    return answer
```

PATTERN 23 OF 41

Sorting

20 problems · 6 Easy · 14 Medium · 0 Hard

THE IDEA

Sorting first makes the structure visible: equal items sit together, neighbours are the closest, and greedy or two-pointer scans become possible. Know the custom key and the counting-sort shortcut.

Signals: largest number, sort colors, merge sorted arrays, custom order, kth element, meeting rooms, sort by frequency.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
# sort by one key, then by another (negative number = descending)
people.sort(key=lambda p: (-p[0], p[1]))
# when values are small integers, count instead of compare
counts = [0] * (max_value + 1)
for x in nums:
    counts[x] += 1
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Merge Sorted Array	Easy	LC 88	1230
2	Sort Array By Parity	Easy	LC 905	1233
3	Relative Sort Array	Easy	LC 1122	1235
4	Sort the People	Easy	LC 2418	1238
5	Sort Array by Increasing Frequency	Easy	LC 1636	1240
6	Sort Integers by The Number of 1 Bits	Easy	LC 1356	1242
7	★ **Sort Colors** (see page)	Medium	LC 75	156
8	★ **Largest Number**	Medium	LC 179	1244
9	★ **Merge Intervals** (see page)	Medium	LC 56	1087
10	★ **Kth Largest Element in an Array** (see page)	Medium	LC 215	737
11	★ **Sort an Array** (see page)	Medium	LC 912	729
12	Wiggle Sort II	Medium	LC 324	1246
13	★ **Queue Reconstruction by Height** (see page)	Medium	LC 406	1164
14	★ **Maximum Gap**	Medium	LC 164	1249
15	Custom Sort String	Medium	LC 791	1252
16	H-Index	Medium	LC 274	1255
17	Minimum Increment to Make Array Unique	Medium	LC 945	1258
18	Pancake Sorting	Medium	LC 969	1261
19	Sort the Matrix Diagonally	Medium	LC 1329	1264
20	Boats to Save People (see page)	Medium	LC 881	172

PATTERN 23 · SORTING · PROBLEM 1 OF 20

88. Merge Sorted Array

Easy · Array, Two Pointers, Sorting

leetcode.com/problems/merge-sorted-array

Also appears in pattern 38 K-way Merge

PROBLEM

You are given two integer arrays `nums1` and `nums2`, sorted in **non-decreasing order**, and two integers `m` and `n`, representing the number of elements in `nums1` and `nums2` respectively.

Merge `nums1` and `nums2` into a single array sorted in **non-decreasing order**.

The final sorted array should not be returned by the function, but instead be *stored inside the array* `nums1`. To accommodate this, `nums1` has a length of `m + n`, where the first `m` elements denote the elements that should be merged, and the last `n` elements are set to `0` and should be ignored. `nums2` has a length of `n`.

Example 1:

```
Input: nums1 = [1,2,3,0,0,0], m = 3, nums2 = [2,5,6], n = 3
Output: [1,2,2,3,5,6]
Explanation: The arrays we are merging are [1,2,3] and [2,5,6].
The result of the merge is [1,2,2,3,5,6] with the underlined elements coming from nums1.
```

Example 2:

```
Input: nums1 = [1], m = 1, nums2 = [], n = 0
Output: [1]
Explanation: The arrays we are merging are [1] and [].
The result of the merge is [1].
```

Example 3:

```
Input: nums1 = [0], m = 0, nums2 = [1], n = 1
Output: [1]
Explanation: The arrays we are merging are [] and [1].
The result of the merge is [1].
Note that because m = 0, there are no elements in nums1. The 0 is only there to ensure the merge result
can fit in nums1.
```

Constraints:

- `nums1.length == m + n`
- `nums2.length == n`
- `0 <= m, n <= 200`
- `1 <= m + n <= 200`
- -10^9 `<= nums1[i], nums2[j] <=` 10^9

Follow up: Can you come up with an algorithm that runs in `O(m + n)` time?

SOLUTION (PYTHON 3)

Solution 1: Two Pointers

How to think about it

The first idea is allocate another array, merge from the front, copy back into nums1. Correct, and m + n ≤ 200 is tiny, but that uses O(m + n) extra space and is not in-place. The bottleneck is writing from the left: it overwrites values in nums1 we have not consumed. The spare slots sit at the tail. So merge from the back. The tail of nums1 is empty; placing the current larger value there never steps on unread elements. Pointers i, j walk the two ends, k writes the merged end, until nums2 is exhausted.

We use two pointers i and j pointing to the end of two arrays, and a pointer k pointing to the end of the merged array.

Every time we compare the two elements at the end of the two arrays, and move the larger one to the end of the merged array. Then we move the pointer one step forward, and repeat this process until the two pointers reach the start of the arrays.

The time complexity is O(m + n), where m and n are the lengths of two arrays. The space complexity is O(1).

Python 3

```
class Solution:
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        k = m + n - 1
        i, j = m - 1, n - 1
        while j >= 0:
            if i >= 0 and nums1[i] > nums2[j]:
                nums1[k] = nums1[i]
                i -= 1
            else:
                nums1[k] = nums2[j]
                j -= 1
            k -= 1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The UNION of two sorted arrays (every value once)
# 2. The INTERSECTION of two sorted arrays (keeping
# repeated values as often as both have them)


# ============================================================
# FOLLOW-UP 1: The UNION of two sorted arrays (every value
# once)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Merge with two pointers and skip a value when it
# equals the last value written.
# Time: O(a + b).  Space: O(a + b).
def union_of_sorted(a, b):
    result = []
    i = j = 0
    while i < len(a) or j < len(b):
        if j >= len(b) or (i < len(a) and a[i] <= b[j]):
            value = a[i]
            i += 1
        else:
            value = b[j]
            j += 1
        if not result or result[-1] != value:
            result.append(value)
    return result


# ============================================================
```

```
# FOLLOW-UP 2: The INTERSECTION of two sorted arrays
# (keeping repeated values as often as both have them)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk both arrays; equal numbers are taken,
# otherwise move the pointer of the smaller one.
# Time: O(a + b).  Space: O(min(a, b)).
def intersection_of_sorted(a, b):
    result = []
    i = j = 0
    while i < len(a) and j < len(b):
        if a[i] == b[j]:
            result.append(a[i])
            i += 1
            j += 1
        elif a[i] < b[j]:
            i += 1
        else:
            j += 1
    return result
```

PATTERN 23 · SORTING · PROBLEM 2 OF 20

905. Sort Array By Parity

Easy · Array, Two Pointers, Sorting

leetcode.com/problems/sort-array-by-parity

PROBLEM

Given an integer array `nums`, move all the even integers at the beginning of the array followed by all the odd integers.

Return ***any array*** *that satisfies this condition*.

Example 1:

```
Input: nums = [3,1,2,4]
Output: [2,4,3,1]
Explanation: The outputs [4,2,3,1], [2,4,1,3], and [4,2,1,3] would also be accepted.
```

Example 2:

```
Input: nums = [0]
Output: [0]
```

Constraints:

- `1 <= nums.length <= 5000`
- `0 <= nums[i] <= 5000`

SOLUTION (PYTHON 3)

Solution 1: Two Pointers

> **How to think about it**
>
> Even numbers should precede odds; relative order is free. An extra array would work, but the permutation can be done in place. Two pointers move inward: advance the left on an even, the right on an odd, otherwise swap. Each index is visited a constant number of times.

We use two pointers i and j to point to the beginning and end of the array respectively. When i < j, we perform the following operations.

- If nums[i] is even, then increment i by 1.
- If nums[j] is odd, then decrement j by 1.
- If nums[i] is odd and nums[j] is even, then swap nums[i] and nums[j]. Then increment i by 1, and decrement j by 1.

Finally, return the array nums.

The time complexity is O(n), where n is the length of the array nums. The space complexity is O(1).

Python 3

```python
class Solution:
    def sortArrayByParity(self, nums: List[int]) -> List[int]:
        i, j = 0, len(nums) - 1
        while i < j:
            if nums[i] % 2 == 0:
                i += 1
            elif nums[j] % 2 == 1:
                j -= 1
            else:
                nums[i], nums[j] = nums[j], nums[i]
                i, j = i + 1, j - 1
        return nums
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Keep the original order inside the evens and inside
# the odds
# 2. Group by the remainder modulo 3 (0, then 1, then 2)
# in place


# ============================================================
# FOLLOW-UP 1: Keep the original order inside the evens
# and inside the odds
# ============================================================

# ---- follow-up 1: function ----
# Answer: Collect the evens and the odds in two lists in
# one pass and join them.
# Time: O(n).  Space: O(n).
def sort_by_parity_stable(nums):
    evens = []
    odds = []
    for x in nums:
        if x % 2 == 0:
            evens.append(x)
        else:
            odds.append(x)
    return evens + odds


# ============================================================
# FOLLOW-UP 2: Group by the remainder modulo 3 (0, then 1,
# then 2) in place
# ============================================================

# ---- follow-up 2: function ----
# Answer: Three groups need the Dutch national flag
# method: keep a boundary for the zeros and one for the
# twos and move the pointer through the middle.
# Time: O(n).  Space: O(1).
def sort_by_mod_three(nums):
    low, i, high = 0, 0, len(nums) - 1
    while i <= high:
        r = nums[i] % 3
        if r == 0:
            nums[low], nums[i] = nums[i], nums[low]
            low += 1
            i += 1
        elif r == 2:
            nums[high], nums[i] = nums[i], nums[high]
            high -= 1
        else:
            i += 1
    return nums
```

PATTERN 23 · SORTING · PROBLEM 3 OF 20

1122. Relative Sort Array

Easy · Array, Hash Table, Bubble Sort, Counting Sort, Sorting, Quick Sort

leetcode.com/problems/relative-sort-array

PROBLEM

Given two arrays `arr1` and `arr2`, the elements of `arr2` are distinct, and all elements in `arr2` are also in `arr1`.

Sort the elements of `arr1` such that the relative ordering of items in `arr1` are the same as in `arr2`. Elements that do not appear in `arr2` should be placed at the end of `arr1` in **ascending** order.

Example 1:

```
Input: arr1 = [2,3,1,3,2,4,6,7,9,2,19], arr2 = [2,1,4,3,9,6]
Output: [2,2,2,1,4,3,3,9,6,7,19]
```

Example 2:

```
Input: arr1 = [28,6,22,8,44,17], arr2 = [22,28,8,6]
Output: [22,28,8,6,17,44]
```

Constraints:

- `1 <= arr1.length, arr2.length <= 1000`
- `0 <= arr1[i], arr2[i] <= 1000`
- All the elements of `arr2` are **distinct**.
- Each `arr2[i]` is in `arr1`.

SOLUTIONS (PYTHON 3)

Solution 1: Custom Sorting

How to think about it

The sort key is the relative order in arr2; values absent from arr2 follow, sorted by themselves. A map stores each arr2 index; the comparator uses that index if present and 1000+x otherwise, so one sort handles both groups. Values in arr2 are unique, so indices are a stable order.

First, we use a hash table pos to record the position of each element in array arr2. Then, we map each element in array arr1 to a tuple (pos.get(x, 1000 + x), x), and sort these tuples. Finally, we take out the second element of all tuples and return it.

The time complexity is O(n × log n + m), and the space complexity is O(n + m). Here, n and m are the lengths of arrays arr1 and arr2, respectively.

Python 3

```python
class Solution:
    def relativeSortArray(self, arr1: List[int], arr2: List[int]) -> List[int]:
        pos = {x: i for i, x in enumerate(arr2)}
        return sorted(arr1, key=lambda x: pos.get(x, 1000 + x))
```

Solution 2: Counting Sort

How to think about it

Method 1's comparison sort is O(nlog n). When the value range is small, count frequencies in arr1, emit values in arr2 order, then scan the leftovers by value, which is linear.

We can use the idea of counting sort. First, count the occurrence of each element in array arr1. Then, according to the order in array arr2, put the elements in arr1 into the answer array ans according to their occurrence. Finally, we traverse all elements in arr1 and put the elements that do not appear in arr2 in ascending order at the end of the answer array ans.

The time complexity is O(n + m), and the space complexity is O(n). Where n and m are the lengths of arrays arr1 and arr2 respectively.

Python 3

```
class Solution:
    def relativeSortArray(self, arr1: List[int], arr2: List[int]) -> List[int]:
        cnt = Counter(arr1)
        ans = []
        for x in arr2:
            ans.extend([x] * cnt[x])
            cnt.pop(x)
        mi, mx = min(arr1), max(arr1)
        for x in range(mi, mx + 1):
            ans.extend([x] * cnt[x])
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Numbers that are not in arr2 keep their ORIGINAL
# order at the end
# 2. Counting sort when the numbers are between 0 and 1000


# ============================================================
# FOLLOW-UP 1: Numbers that are not in arr2 keep their
# ORIGINAL order at the end
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count the numbers of arr2 first, then write the
# rest in the order they appear.
# Time: O(n + m).  Space: O(n).
def relative_sort_keep_order(arr1, arr2):
    position = {}
    for i in range(len(arr2)):
        position[arr2[i]] = i
    counts = {}
    rest = []
    for x in arr1:
        if x in position:
            counts[x] = counts.get(x, 0) + 1
        else:
            rest.append(x)
    result = []
    for x in arr2:
        result += [x] * counts.get(x, 0)
    return result + rest


# ============================================================
# FOLLOW-UP 2: Counting sort when the numbers are between
# 0 and 1000
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count every number, write the numbers of arr2
# first, then the remaining numbers in increasing order.
# Time: O(n + m + 1001).  Space: O(1001).
def relative_sort_counting(arr1, arr2):
    count = [0] * 1001
    for x in arr1:
```

```
        count[x] += 1
    result = []
    for x in arr2:
        result += [x] * count[x]
        count[x] = 0
    for x in range(1001):
        result += [x] * count[x]
    return result
```

PATTERN 23 · SORTING · PROBLEM 4 OF 20

2418. Sort the People

Easy · Array, Hash Table, String, Sorting

leetcode.com/problems/sort-the-people

PROBLEM

You are given an array of strings `names`, and an array `heights` that consists of **distinct** positive integers. Both arrays are of length `n`.

For each index `i`, `names[i]` and `heights[i]` denote the name and height of the i^{th} person.

Return `names` *sorted in* ***descending*** *order by the people's heights*.

Example 1:

```
Input: names = ["Mary","John","Emma"], heights = [180,165,170]
Output: ["Mary","Emma","John"]
Explanation: Mary is the tallest, followed by Emma and John.
```

Example 2:

```
Input: names = ["Alice","Bob","Bob"], heights = [155,185,150]
Output: ["Bob","Alice","Bob"]
Explanation: The first Bob is the tallest, followed by Alice and the second Bob.
```

Constraints:

- `n == names.length == heights.length`
- `1 <= n <= 10^3`
- `1 <= names[i].length <= 20`
- `1 <= heights[i] <= 10^5`
- `names[i]` consists of lower and upper case English letters.
- All the values of `heights` are distinct.

SOLUTIONS (PYTHON 3)

Solution 1: Sorting

How to think about it

With n≤ 10^3, sort names by descending height. Heights are unique, so the key is total. Sort indices by heights descending, then emit names[i].

According to the problem description, we can create an index array idx of length n, where idx[i]=i. Then we sort each index in idx in descending order according to the corresponding height in heights. Finally, we traverse each index i in the sorted idx and add names[i] to the answer array.

We can also create an array arr of length n, where each element is a tuple (heights[i], i). Then we sort arr in descending order by height. Finally, we traverse each element (heights[i], i) in the sorted arr and add names[i] to the answer array.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the length of the arrays names and heights.

Python 3

```python
class Solution:
    def sortPeople(self, names: List[str], heights: List[int]) -> List[str]:
        idx = list(range(len(heights)))
        idx.sort(key=lambda i: -heights[i])
        return [names[i] for i in idx]
```

Solution 2

> **How to think about it**
>
> Method 1 already sorts via an index array. Zipping height with name and sorting that pair descending removes the extra index array; the order is the same.

Python 3

```python
class Solution:
    def sortPeople(self, names: List[str], heights: List[int]) -> List[str]:
        return [name for _, name in sorted(zip(heights, names), reverse=True)]
```

Complexity. Time: O(n log n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Sort by height, and by NAME when the heights are
# equal
# 2. Ascending heights (shortest first)


# ============================================================
# FOLLOW-UP 1: Sort by height, and by NAME when the
# heights are equal
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort by the pair (-height, name).
# Time: O(n log n).  Space: O(n).
def sort_people_height_then_name(names, heights):
    pairs = []
    for i in range(len(names)):
        pairs.append((-heights[i], names[i]))
    pairs.sort()
    return [name for negative_height, name in pairs]


# ============================================================
# FOLLOW-UP 2: Ascending heights (shortest first)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Sort by the height ascending.
# Time: O(n log n).  Space: O(n).
def sort_people_ascending(names, heights):
    pairs = []
    for i in range(len(names)):
        pairs.append((heights[i], names[i]))
    pairs.sort()
    return [name for height, name in pairs]
```

PATTERN 23 · SORTING · PROBLEM 5 OF 20

1636. Sort Array by Increasing Frequency

Easy · Array, Hash Table, Sorting

leetcode.com/problems/sort-array-by-increasing-frequency

PROBLEM

Given an array of integers `nums`, sort the array in **increasing** order based on the frequency of the values. If multiple values have the same frequency, sort them in **decreasing** order.

Return the *sorted array*.

Example 1:

```
Input: nums = [1,1,2,2,2,3]
Output: [3,1,1,2,2,2]
Explanation: '3' has a frequency of 1, '1' has a frequency of 2, and '2' has a frequency of 3.
```

Example 2:

```
Input: nums = [2,3,1,3,2]
Output: [1,3,3,2,2]
Explanation: '2' and '3' both have a frequency of 2, so they are sorted in decreasing order.
```

Example 3:

```
Input: nums = [-1,1,-6,4,5,-6,1,4,1]
Output: [5,-1,4,4,-6,-6,1,1,1]
```

Constraints:

- `1 <= nums.length <= 100`
- `-100 <= nums[i] <= 100`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

The order is increasing frequency, then decreasing value on ties. n ≤ 100, so count then sort with a key. A counter (or a shifted array) yields cnt; the sort key is (cnt[x], -x).

Python 3

```python
class Solution:
    def frequencySort(self, nums: List[int]) -> List[int]:
        cnt = Counter(nums)
        return sorted(nums, key=lambda x: (cnt[x], -x))
```

Complexity. Time: O(n log n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Decreasing frequency, and ties by increasing value
# 2. Increasing frequency, and ties keep the ORIGINAL
# order (stable)


# ============================================================
# FOLLOW-UP 1: Decreasing frequency, and ties by
# increasing value
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort by the pair (-count, value).
# Time: O(n log n).  Space: O(n).
def sort_by_frequency_desc(nums):
    count = {}
    for x in nums:
        count[x] = count.get(x, 0) + 1
    return sorted(nums, key=lambda x: (-count[x], x))


# ============================================================
# FOLLOW-UP 2: Increasing frequency, and ties keep the
# ORIGINAL order (stable)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Python's sort is stable, so sorting only by the
# count keeps the original order of equal counts.
# Time: O(n log n).  Space: O(n).
def sort_by_frequency_stable(nums):
    count = {}
    for x in nums:
        count[x] = count.get(x, 0) + 1
    return sorted(nums, key=lambda x: count[x])
```

1356. Sort Integers by The Number of 1 Bits

Easy · Bit Manipulation, Array, Counting, Sorting

leetcode.com/problems/sort-integers-by-the-number-of-1-bits

PROBLEM

You are given an integer array `arr`. Sort the integers in the array in ascending order by the number of `1`'s in their binary representation and in case of two or more integers have the same number of `1`'s you have to sort them in ascending order.

Return *the array after sorting it*.

Example 1:

```
Input: arr = [0,1,2,3,4,5,6,7,8]
Output: [0,1,2,4,8,3,5,6,7]
Explantion: [0] is the only integer with 0 bits.
[1,2,4,8] all have 1 bit.
[3,5,6] have 2 bits.
[7] has 3 bits.
The sorted array by bits is [0,1,2,4,8,3,5,6,7]
```

Example 2:

```
Input: arr = [1024,512,256,128,64,32,16,8,4,2,1]
Output: [1,2,4,8,16,32,64,128,256,512,1024]
Explantion: All integers have 1 bit in the binary representation, you should just sort them in
ascending order.
```

Constraints:

- `1 <= arr.length <= 500`
- `0 <= arr[i] <= 10^4`

SOLUTION (PYTHON 3)

Solution 1: Custom Sorting

> **How to think about it**
>
> Order by the number of one-bits, then by the integer itself. The key (x.bit_count(), x) encodes both levels in one comparison.

We sort the array arr according to the requirements of the problem, that is, sort in ascending order according to the number of 1s in the binary representation. If there are multiple numbers with the same number of 1s in the binary representation, they must be sorted in ascending order by numerical value.

The time complexity is O(n log n), and the space complexity is O(n). Where n is the length of the array arr.

Python 3

```python
class Solution:
    def sortByBits(self, arr: List[int]) -> List[int]:
        return sorted(arr, key=lambda x: (x.bit_count(), x))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Count the 1 bits without converting to a string
# 2. The MORE one-bits the earlier (descending), ties by
# value


# ============================================================
# FOLLOW-UP 1: Count the 1 bits without converting to a
# string
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remove the lowest set bit again and again (x &
# (x - 1)) and count the removals.
# Time: O(n log n + n * b), b = bits per number.  Space:
# O(n).
def count_bits(x):
    count = 0
    while x:
        x &= x - 1
        count += 1
    return count


def sort_by_bits(arr):
    return sorted(arr, key=lambda x: (count_bits(x), x))


# ============================================================
# FOLLOW-UP 2: The MORE one-bits the earlier (descending),
# ties by value
# ============================================================

# ---- follow-up 2: function ----
# Answer: Sort by (-bits, value).
# Time: O(n log n + n * b).  Space: O(n).
def sort_by_bits_desc(arr):
    def bits(x):
        count = 0
        while x:
            x &= x - 1
            count += 1
        return count

    return sorted(arr, key=lambda x: (-bits(x), x))
```

PATTERN 23 · SORTING · PROBLEM 8 OF 20

179. Largest Number

Medium · Greedy, Array, String, Sorting

leetcode.com/problems/largest-number

★ **TOP 7 PROBLEM** in Sorting: master this one first

PROBLEM

Given a list of non-negative integers `nums`, arrange them such that they form the largest number and return it.

Since the result may be very large, so you need to return a string instead of an integer.

Example 1:

```
Input: nums = [10,2]
Output: "210"
```

Example 2:

```
Input: nums = [3,30,34,5,9]
Output: "9534330"
```

Constraints:

- `1 <= nums.length <= 100`
- `0 <= nums[i] <= 10^9`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Form the largest number by concatenation. Numeric order and plain lexicographic order both fail: 9 should precede 98 because 998>989. n≤ 100. Compare a+b with b+a to order two strings, sort by that, and join. If the first character is 0, every value was zero, so return "0".

Python 3

```python
class Solution:
    def largestNumber(self, nums: List[int]) -> str:
        nums = [str(v) for v in nums]
        nums.sort(key=cmp_to_key(lambda a, b: 1 if a + b < b + a else -1))
        return "0" if nums[0] == "0" else "".join(nums)
```

Complexity. Time: O(n log n * d), d = digits per number (at most 10). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The SMALLEST number that can be formed by joining the
# numbers
# 2. Why is comparing a + b with b + a correct? Check it
# by brute force on small inputs


# ========================================================
# FOLLOW-UP 1: The SMALLEST number that can be formed by
# joining the numbers
# ========================================================
```

```
# ---- follow-up 1: function ----
# Answer: Same comparison idea reversed: a goes before b
# when a + b is smaller than b + a as text (the first
# number must not be 0 unless all are 0).
# Time: O(n log n * d), d = digits per number.  Space: O(n
# * d).
from functools import cmp_to_key


def smallest_number(nums):
    def compare(a, b):
        if a + b < b + a:
            return -1
        if a + b > b + a:
            return 1
        return 0

    texts = sorted([str(x) for x in nums], key=cmp_to_key(compare))
    result = "".join(texts)
    if result[0] == "0":
        for i in range(len(texts)):
            if texts[i] != "0":
                texts[0], texts[i] = texts[i], texts[0]
                break
        result = "".join(texts)
    return result


# ============================================================
# FOLLOW-UP 2: Why is comparing a + b with b + a correct?
# Check it by brute force on small inputs
# ============================================================

# ---- follow-up 2: function ----
# Answer: The order 'a before b when a + b > b + a' is a
# consistent (transitive) order on strings, so sorting
# with it gives the best arrangement; a brute-force check
# over all permutations confirms it.
# Time: O(n! * n).  Space: O(n * d).
import itertools


def largest_number_brute_force(nums):
    best = ""
    for order in itertools.permutations([str(x) for x in nums]):
        text = "".join(order)
        if best == "" or (len(text) == len(best) and text > best):
            best = text
```

```
    return best
```

PATTERN 23 · SORTING · PROBLEM 12 OF 20

324. Wiggle Sort II

Medium · Greedy, Array, Divide and Conquer, Quickselect, Sorting

leetcode.com/problems/wiggle-sort-ii

Also appears in pattern 33 Cyclic Sort

PROBLEM

Given an integer array `nums`, reorder it such that `nums[0] < nums[1] > nums[2] < nums[3]...`.

You may assume the input array always has a valid answer.

Example 1:

```
Input: nums = [1,5,1,1,6,4]
Output: [1,6,1,5,1,4]
Explanation: [1,4,1,5,1,6] is also accepted.
```

Example 2:

```
Input: nums = [1,3,2,2,3,1]
Output: [2,3,1,3,1,2]
```

Constraints:

- `1 <= nums.length <= 5 * 10^4`
- `0 <= nums[i] <= 5000`
- It is guaranteed that there will be an answer for the given input `nums`.

Follow Up:

Can you do it in

`O(n)`

time and/or

in-place

with

`O(1)`

extra space?

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

We need nums[0]<nums[1]>nums[2]<… and may reorder in place. Sort, then fill even indices from the median downward and odd indices from the end downward. Reversed halves keep equal values from sitting next to each other.

Python 3

```
class Solution:
    def wiggleSort(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        arr = sorted(nums)
        n = len(arr)
        i, j = (n - 1) >> 1, n - 1
        for k in range(n):
            if k % 2 == 0:
                nums[k] = arr[i]
                i -= 1
            else:
                nums[k] = arr[j]
                j -= 1
```

Complexity. Time: O(n log n). Space: O(n).

Solution 2

> **How to think about it**
>
> Method 1 sorts in O(nlog n) and uses an extra array. Values lie in [0,5000]: count-sort, then emit large values onto odd indices first and even indices next. Peaks get large numbers in linear time.

Python 3

```
class Solution:
    def wiggleSort(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        bucket = [0] * 5001
        for v in nums:
            bucket[v] += 1
        n = len(nums)
        j = 5000
        for i in range(1, n, 2):
            while bucket[j] == 0:
                j -= 1
            nums[i] = j
            bucket[j] -= 1
        for i in range(0, n, 2):
            while bucket[j] == 0:
                j -= 1
            nums[i] = j
            bucket[j] -= 1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Wiggle sort I: only nums[0] <= nums[1] >= nums[2] <=
# ... (equal values are allowed)
# 2. Check whether an array is a valid wiggle sort II
# result (strict)


# ========================================================
# FOLLOW-UP 1: Wiggle sort I: only nums[0] <= nums[1] >=
# nums[2] <= ... (equal values are allowed)
# ========================================================

# ---- follow-up 1: function ----
# Answer: One pass: at every position look at the pair
```

```
# with the next one and swap them when they are in the
# wrong order for that position.
# Time: O(n).  Space: O(1).
def wiggle_sort_one(nums):
    for i in range(len(nums) - 1):
        if i % 2 == 0:
            if nums[i] > nums[i + 1]:
                nums[i], nums[i + 1] = nums[i + 1], nums[i]
        else:
            if nums[i] < nums[i + 1]:
                nums[i], nums[i + 1] = nums[i + 1], nums[i]
    return nums


# ============================================================
# FOLLOW-UP 2: Check whether an array is a valid wiggle
# sort II result (strict)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Every even index must be smaller than its
# neighbours and every odd index bigger than its
# neighbours.
# Time: O(n).  Space: O(1).
def is_wiggle_ii(nums):
    for i in range(len(nums) - 1):
        if i % 2 == 0 and not nums[i] < nums[i + 1]:
            return False
        if i % 2 == 1 and not nums[i] > nums[i + 1]:
            return False
    return True
```

PATTERN 23 · SORTING · PROBLEM 14 OF 20

164. Maximum Gap

Medium · Array, Bucket Sort, Radix Sort, Sorting, Pigeonhole Principle

leetcode.com/problems/maximum-gap

★ **TOP 7 PROBLEM** in Sorting: master this one first

PROBLEM

Given an integer array `nums`, return *the maximum difference between two successive elements in its sorted form*. If the array contains less than two elements, return `0`.

You must write an algorithm that runs in linear time and uses linear extra space.

Example 1:

```
Input: nums = [3,6,9,1]
Output: 3
Explanation: The sorted form of the array is [1,3,6,9], either (3,6) or (6,9) has the maximum
difference 3.
```

Example 2:

```
Input: nums = [10]
Output: 0
Explanation: The array contains less than 2 elements, therefore return 0.
```

Constraints:

- `1 <= nums.length <= 10^5`
- `0 <= nums[i] <= 10^9`

SOLUTION (PYTHON 3)

Solution 1: Bucket Sort

How to think about it

Maximum gap after sorting, in linear time. Comparison sorts are O(nlog n); n≤ 10^5. The max adjacent gap is at least (max-min)/(n-1). Bucket by that width: gaps inside a bucket are smaller than this lower bound, so the answer is between consecutive non-empty buckets (next min minus previous max). Each bucket stores only min and max.

Suppose nums has n elements. After sorting them into $nums_0 \le \dots \le nums_{n-1}$, the maximum adjacent gap maxGap satisfies

$$nums_{n-1} - nums_0 = \sum_{i=1}^{n-1}(nums_i - nums_{i-1}) \le maxGap \times (n-1).$$

Hence $maxGap \ge (nums_{n-1} - nums_0)/(n-1)$.

Use that lower bound as the bucket width, and at least 1 when every value is equal. Values that fall in the same bucket differ by less than maxGap, so a gap of size maxGap always crosses two buckets. Each bucket keeps only its minimum and its maximum, initialized to +∞ and -∞ to mark an empty bucket.

Place every value v into bucket floor((v - min) / bucketSize). Smaller indices then hold smaller values. Scan the buckets from left to right. The minimum of the current non-empty bucket and the maximum of the previous non-empty bucket are adjacent in sorted order; their difference updates the answer. There is no gap before the first non-empty bucket.

The time complexity is O(n), and the space complexity is O(n), where n is the length of nums.

Python 3

```python
class Solution:
    def maximumGap(self, nums: List[int]) -> int:
        n = len(nums)
        if n < 2:
            return 0
        mi, mx = min(nums), max(nums)
        bucket_size = max(1, (mx - mi) // (n - 1))
        bucket_count = (mx - mi) // bucket_size + 1
        buckets = [[inf, -inf] for _ in range(bucket_count)]
        for v in nums:
            i = (v - mi) // bucket_size
            buckets[i][0] = min(buckets[i][0], v)
            buckets[i][1] = max(buckets[i][1], v)
        ans = 0
        prev = inf
        for curmin, curmax in buckets:
            if curmin > curmax:
                continue
            ans = max(ans, curmin - prev)
            prev = curmax
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The MINIMUM gap between neighbours in sorted order
# 2. O(n) with buckets (pigeonhole)


# ============================================================
# FOLLOW-UP 1: The MINIMUM gap between neighbours in
# sorted order
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort and take the smallest difference of
# neighbours.
# Time: O(n log n).  Space: O(n).
def minimum_gap(nums):
    ordered = sorted(nums)
    best = None
    for i in range(1, len(ordered)):
        gap = ordered[i] - ordered[i - 1]
        if best is None or gap < best:
            best = gap
    return best


# ============================================================
# FOLLOW-UP 2: O(n) with buckets (pigeonhole)
# ============================================================

# ---- follow-up 2: function ----
# Answer: With n numbers between lo and hi the biggest gap
# is at least (hi - lo) / (n - 1), so buckets of that size
# never contain the biggest gap inside one bucket: it is
# between the maximum of one bucket and the minimum of the
# next non-empty bucket.
# Time: O(n).  Space: O(n).
def maximum_gap_buckets(nums):
    n = len(nums)
    if n < 2:
        return 0
    low, high = min(nums), max(nums)
    if low == high:
```

```
        return 0
    size = max(1, (high - low) // (n - 1))
    count = (high - low) // size + 1
    bucket_min = [None] * count
    bucket_max = [None] * count
    for x in nums:
        b = (x - low) // size
        if bucket_min[b] is None or x < bucket_min[b]:
            bucket_min[b] = x
        if bucket_max[b] is None or x > bucket_max[b]:
            bucket_max[b] = x
    best = 0
    previous_max = low
    for b in range(count):
        if bucket_min[b] is None:
            continue
        if bucket_min[b] - previous_max > best:
            best = bucket_min[b] - previous_max
        previous_max = bucket_max[b]
```

```
    return best
```

791. Custom Sort String

Medium · Hash Table, String, Sorting

leetcode.com/problems/custom-sort-string

PROBLEM

You are given two strings `order` and `s`. All the characters of `order` are **unique** and were sorted in some custom order previously.

Permute the characters of `s` so that they match the order that `order` was sorted. More specifically, if a character `x` occurs before a character `y` in `order`, then `x` should occur before `y` in the permuted string.

Return *any permutation of* `s` *that satisfies this property*.

Example 1:

Input: order = "cba", s = "abcd"

Output: "cbad"

Explanation: `"a"`, `"b"`, `"c"` appear in order, so the order of `"a"`, `"b"`, `"c"` should be `"c"`, `"b"`, and `"a"`.

Since `"d"` does not appear in `order`, it can be at any position in the returned string. `"dcba"`, `"cdba"`, `"cbda"` are also valid outputs.

Example 2:

Input: order = "bcafg", s = "abcd"

Output: "bcad"

Explanation: The characters `"b"`, `"c"`, and `"a"` from `order` dictate the order for the characters in `s`. The character `"d"` in `s` does not appear in `order`, so its position is flexible.

Following the order of appearance in `order`, `"b"`, `"c"`, and `"a"` from `s` should be arranged as `"b"`, `"c"`, `"a"`. `"d"` can be placed at any position since it's not in order. The output `"bcad"` correctly follows this rule. Other arrangements like `"dbca"` or `"bcda"` would also be valid, as long as `"b"`, `"c"`, `"a"` maintain their order.

Constraints:

- `1 <= order.length <= 26`
- `1 <= s.length <= 200`
- `order` and `s` consist of lowercase English letters.
- All the characters of `order` are **unique**.

SOLUTIONS (PYTHON 3)

Solution 1

> **How to think about it**
>
> Reorder s by the relative order in order. Sorting by the index in order does that. Missing letters get key 0 and gather at the front, which is allowed.

Python 3

```python
class Solution:
    def customSortString(self, order: str, s: str) -> str:
        d = {c: i for i, c in enumerate(order)}
        return ''.join(sorted(s, key=lambda x: d.get(x, 0)))
```

Complexity. Time: O(n log n). Space: O(n).

Solution 2

> **How to think about it**
>
> Sorting is nlog n. Count s, emit each order character that many times, then append the rest. Linear.

Python 3

```
class Solution:
    def customSortString(self, order: str, s: str) -> str:
        cnt = Counter(s)
        ans = []
        for c in order:
            ans.append(c * cnt[c])
            cnt[c] = 0
        for c, v in cnt.items():
            ans.append(c * v)
        return ''.join(ans)
```

Complexity. Time: O(n + m), m = len(order). Space: O(C), C = 26 letters (plus the answer).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Sort a LIST of words by a custom alphabet
# 2. Are the words already in sorted order under the
# custom alphabet (Verifying an Alien Dictionary)?


# ============================================================
# FOLLOW-UP 1: Sort a LIST of words by a custom alphabet
# ============================================================

# ---- follow-up 1: function ----
# Answer: Give every letter its position in the custom
# order and compare the words by the list of positions.
# Time: O(w * L log w), w words of length L.  Space: O(w *
# L).
def sort_words_custom(words, order):
    rank = {}
    for i in range(len(order)):
        rank[order[i]] = i
    def key(word):
        return [rank.get(ch, len(order)) for ch in word]
    return sorted(words, key=key)


# ============================================================
# FOLLOW-UP 2: Are the words already in sorted order under
# the custom alphabet (Verifying an Alien Dictionary)?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Compare every neighbouring pair letter by letter
# with the ranks; a word that is a prefix of the other
# must come first.
# Time: O(total letters).  Space: O(1).
def is_sorted_alien(words, order):
    rank = {}
    for i in range(len(order)):
        rank[order[i]] = i
    for k in range(len(words) - 1):
        a, b = words[k], words[k + 1]
        i = 0
        while i < len(a) and i < len(b) and a[i] == b[i]:
            i += 1
        if i == len(a):
```

```
            continue
        if i == len(b):
            return False
        if rank[a[i]] > rank[b[i]]:
            return False
    return True
```

274. H-Index

Medium · Array, Counting Sort, Sorting

leetcode.com/problems/h-index

PROBLEM

Given an array of integers `citations` where `citations[i]` is the number of citations a researcher received for their i^{th} paper, return *the researcher's h-index*.

According to the definition of h-index on Wikipedia: The h-index is defined as the maximum value of `h` such that the given researcher has published at least `h` papers that have each been cited at least `h` times.

Example 1:

```
Input: citations = [3,0,6,1,5]
Output: 3
Explanation: [3,0,6,1,5] means the researcher has 5 papers in total and each of them had received 3, 0, 6, 1, 5 citations respectively.
Since the researcher has 3 papers with at least 3 citations each and the remaining two with no more than 3 citations each, their h-index is 3.
```

Example 2:

```
Input: citations = [1,3,1]
Output: 1
```

Constraints:

- `n == citations.length`
- `1 <= n <= 5000`
- `0 <= citations[i] <= 1000`

SOLUTIONS (PYTHON 3)

Solution 1: Sorting

> **How to think about it**
>
> The h-index is the largest h such that at least h papers have ≥ h citations. After sorting descending, citations[h-1]≥ h is the test; search h from large to small.

We can sort the array `citations` in descending order. Then we enumerate the value h from large to small, if there is an h value satisfying citations[h-1] ≥ h, it means that there are at least h papers that have been cited at least h times, just return h directly. If we cannot find such an h value, it means that all the papers have not been cited, return 0.

Time complexity O(n × log n), space complexity O(log n). Here n is the length of the array `citations`.

Python 3

```python
class Solution:
    def hIndex(self, citations: List[int]) -> int:
        citations.sort(reverse=True)
        for h in range(len(citations), 0, -1):
            if citations[h - 1] >= h:
                return h
        return 0
```

Solution 2: Counting + Sum

> **How to think about it**
>
> Sorting is O(nlog n). h≤ n, so cap citations at n, count, and accumulate from high to low until s≥ h.

We can use an array cnt of length n+1, where cnt[i] represents the number of papers with the reference count of i. We traverse the array `citations` and treat the papers with the reference count greater than n as papers with a reference count of n. Then we use the reference count as the index and add 1 to the corresponding element of cnt for each paper. In this way, we have counted the number of papers for each reference count.

Then we enumerate the value h from large to small, and add the element value of cnt with the index of h to the variable s, where s represents the number of papers with a reference count greater than or equal to h. If s ≥ h, it means that at least h papers have been cited at least h times, just return h directly.

Time complexity O(n), space complexity O(n). Here n is the length of the array `citations`.

Python 3

```python
class Solution:
    def hIndex(self, citations: List[int]) -> int:
        n = len(citations)
        cnt = [0] * (n + 1)
        for x in citations:
            cnt[min(x, n)] += 1
        s = 0
        for h in range(n, -1, -1):
            s += cnt[h]
            if s >= h:
                return h
```

Solution 3: Binary Search

> **How to think about it**
>
> The predicate “at least h papers have ≥ h citations” is monotone, so we binary-search the largest feasible h by counting values ≥ mid.

We notice that if there is a h value that satisfies at least h papers are cited at least h times, then for any h'<h, at least h' papers are cited at least h' times. Therefore, we can use the binary search method to find the largest h such that at least h papers are cited at least h times.

We define the left boundary of binary search l=0 and the right boundary r=n. Each time we take mid = floor((l + r + 1)/2), where floor(x) represents floor x. Then we count the number of elements in array `citations` that are greater than or equal to mid, and denote it as s. If s ≥ mid, it means that at least mid papers are cited at least mid times. In this case, we change the left boundary l to mid. Otherwise, we change the right boundary r to mid-1. When the left boundary l is equal to the right boundary r, we find the largest h value, which is l or r.

Time complexity O(n × log n), where n is the length of array `citations`. Space complexity O(1).

Python 3

```python
class Solution:
    def hIndex(self, citations: List[int]) -> int:
        l, r = 0, len(citations)
        while l < r:
            mid = (l + r + 1) >> 1
            if sum(x >= mid for x in citations) >= mid:
                l = mid
            else:
                r = mid - 1
        return l
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The citations are already sorted: binary search
# (H-Index II)
# 2. O(n) with counting


# ============================================================
# FOLLOW-UP 1: The citations are already sorted: binary
# search (H-Index II)
# ============================================================

# ---- follow-up 1: function ----
# Answer: The h-index is the largest h with citations[n -
# h] >= h; search for the smallest position i with
# citations[i] >= n - i.
# Time: O(log n).  Space: O(1).
def h_index_sorted(citations):
    n = len(citations)
    low, high = 0, n
    while low < high:
        mid = (low + high) // 2
        if citations[mid] >= n - mid:
            high = mid
        else:
            low = mid + 1
    return n - low


# ============================================================
# FOLLOW-UP 2: O(n) with counting
# ============================================================

# ---- follow-up 2: function ----
# Answer: A paper cited more than n times counts as n
# citations; count the papers per citation value and walk
# down from n summing the counts until the sum reaches the
# value.
# Time: O(n).  Space: O(n).
def h_index_counting(citations):
    n = len(citations)
    count = [0] * (n + 1)
    for c in citations:
        count[c if c < n else n] += 1
    total = 0
    for h in range(n, -1, -1):
        total += count[h]
        if total >= h:
            return h
    return 0
```

PATTERN 23 · SORTING · PROBLEM 17 OF 20

945. Minimum Increment to Make Array Unique

Medium · Greedy, Array, Counting, Sorting

leetcode.com/problems/minimum-increment-to-make-array-unique

PROBLEM

You are given an integer array `nums`. In one move, you can pick an index `i` where `0 <= i < nums.length` and increment `nums[i]` by `1`.

Return *the minimum number of moves to make every value in* `nums` ***unique***.

The test cases are generated so that the answer fits in a 32-bit integer.

Example 1:

```
Input: nums = [1,2,2]
Output: 1
Explanation: After 1 move, the array could be [1, 2, 3].
```

Example 2:

```
Input: nums = [3,2,1,2,1,7]
Output: 6
Explanation: After 6 moves, the array could be [3, 4, 1, 2, 5, 7].
It can be shown that it is impossible for the array to have all unique values with 5 or less moves.
```

Constraints:

- `1 <= nums.length <= 10^5`
- `0 <= nums[i] <= 10^5`

SOLUTIONS (PYTHON 3)

Solution 1: Sorting + Greedy

> **How to think about it**
>
> Each move increments one element; we want all values distinct at minimum cost. After sorting, each x should occupy the next free slot y. Set y←max(y+1,x) and add y-x.

First, we sort the array nums, and use a variable y to record the current maximum value, initially y = -1.

Then, we iterate through the array nums. For each element x, we update y to max(y + 1, x), and accumulate the operation count y - x into the result.

After completing the iteration, we return the result.

The time complexity is O(n log n), and the space complexity is O(log n). Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def minIncrementForUnique(self, nums: List[int]) -> int:
        nums.sort()
        ans, y = 0, -1
        for x in nums:
            y = max(y + 1, x)
            ans += y - x
        return ans
```

Solution 2: Counting + Greedy

> **How to think about it**
>
> Sorting costs O(nlog n). With an upper bound m=max+n, a count array works: from low to high, the surplus cnt[i]-1 copies of i move to i+1, a linear sweep of the range.

According to the problem description, the maximum value of the result array m = max(nums) + len(nums). We can use a counting array cnt to record the occurrence count of each element.

Then, we iterate from 0 to m - 1. For each element i, if its occurrence count cnt[i] is greater than 1, then we add cnt[i] - 1 elements to i + 1, and accumulate the operation count into the result.

After completing the iteration, we return the result.

The time complexity is O(m), and the space complexity is O(m). Here, m is the length of the array nums plus the maximum value in the array.

Python 3

```python
class Solution:
    def minIncrementForUnique(self, nums: List[int]) -> int:
        m = max(nums) + len(nums)
        cnt = Counter(nums)
        ans = 0
        for i in range(m - 1):
            if (diff := cnt[i] - 1) > 0:
                cnt[i + 1] += diff
                ans += diff
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the final array after the increments
# 2. O(n + max) with counting (no sorting)


# ============================================================
# FOLLOW-UP 1: Return the final array after the increments
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort and raise every number to at least the
# previous number plus one.
# Time: O(n log n).  Space: O(n).
def make_unique(nums):
    ordered = sorted(nums)
    for i in range(1, len(ordered)):
        if ordered[i] <= ordered[i - 1]:
            ordered[i] = ordered[i - 1] + 1
    return ordered


# ============================================================
# FOLLOW-UP 2: O(n + max) with counting (no sorting)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk over the values from small to large; extra
# copies of a value are moved to the next value, so keep a
# running number of extras and add them up as moves.
# Time: O(n + max).  Space: O(n + max).
def min_increments_counting(nums):
    if not nums:
        return 0
    count = [0] * (max(nums) + len(nums) + 1)
```

```
    for x in nums:
        count[x] += 1
    moves = 0
    extras = 0
    for value in range(len(count)):
        extras += count[value]
        if extras > 0:
            extras -= 1                             # one copy stays at this value
            moves += extras                         # every remaining copy moves one step up
    return moves
```

PATTERN 23 · SORTING · PROBLEM 18 OF 20

969. Pancake Sorting

Medium · Greedy, Array, Two Pointers, Sorting

leetcode.com/problems/pancake-sorting

PROBLEM

Given an array of integers `arr`, sort the array by performing a series of **pancake flips**.

In one pancake flip we do the following steps:

- Choose an integer `k` where `1 <= k <= arr.length`.
- Reverse the sub-array `arr[0...k-1]` (**0-indexed**).

For example, if `arr = [3,2,1,4]` and we performed a pancake flip choosing `k = 3`, we reverse the sub-array `[3,2,1]`, so `arr = [1,2,3,4]` after the pancake flip at `k = 3`.

Return *an array of the* `k`*-values corresponding to a sequence of pancake flips that sort* `arr`. Any valid answer that sorts the array within `10 * arr.length` flips will be judged as correct.

Example 1:

```
Input: arr = [3,2,4,1]
Output: [4,2,4,3]
Explanation:
We perform 4 pancake flips, with k values 4, 2, 4, and 3.
Starting state: arr = [3, 2, 4, 1]
After 1st flip (k = 4): arr = [1, 4, 2, 3]
After 2nd flip (k = 2): arr = [4, 1, 2, 3]
After 3rd flip (k = 4): arr = [3, 2, 1, 4]
After 4th flip (k = 3): arr = [1, 2, 3, 4], which is sorted.
```

Example 2:

```
Input: arr = [1,2,3]
Output: []
Explanation: The input is already sorted, so there is no need to flip anything.
Note that other answers, such as [3, 3], would also be accepted.
```

Constraints:

- `1 <= arr.length <= 100`
- `1 <= arr[i] <= arr.length`
- All integers in `arr` are unique (i.e. `arr` is a permutation of the integers from `1` to `arr.length`).

SOLUTION (PYTHON 3)

Solution 1

How to think about it

A pancake flip reverses a prefix. The array is a permutation of 1..n, so we can place values from large to small: flip k to the front, then flip it to its final index k. Record the two prefix lengths. A suffix that is already placed is left untouched.

Python 3

```
class Solution:
    def pancakeSort(self, arr: List[int]) -> List[int]:
        def reverse(arr, j):
            i = 0
            while i < j:
                arr[i], arr[j] = arr[j], arr[i]
                i, j = i + 1, j - 1

        n = len(arr)
        ans = []
        for i in range(n - 1, 0, -1):
            j = i
            while j > 0 and arr[j] != i + 1:
                j -= 1
            if j < i:
                if j > 0:
                    ans.append(j + 1)
                    reverse(arr, j)
                ans.append(i + 1)
                reverse(arr, i)
        return ans
```

Complexity. Time: O(n^2). Space: O(1) extra (the answer is O(n)).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Check that a list of flips really sorts the array
# 2. Sort with the fewest flips for very small arrays
# (breadth-first search)


# ============================================================
# FOLLOW-UP 1: Check that a list of flips really sorts the
# array
# ============================================================

# ---- follow-up 1: function ----
# Answer: Apply each flip (reverse the first k numbers)
# and test the final order.
# Time: O(n * k), k = flips.  Space: O(n).
def apply_flips(arr, flips):
    items = list(arr)
    for k in flips:
        items[:k] = items[:k][::-1]
    return items


def flips_sort(arr, flips):
    result = apply_flips(arr, flips)
    for i in range(len(result) - 1):
        if result[i] > result[i + 1]:
            return False
    return True


# ============================================================
# FOLLOW-UP 2: Sort with the fewest flips for very small
# arrays (breadth-first search)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Every arrangement is a state; a flip is an edge.
# A breadth-first search from the start finds the minimum
# number of flips.
# Time: O(n! * n^2).  Space: O(n! * n).
```

```
from collections import deque


def min_flips_bfs(arr):
    start = tuple(arr)
    target = tuple(sorted(arr))
    seen = {start}
    queue = deque([(start, 0)])
    while queue:
        state, flips = queue.popleft()
        if state == target:
            return flips
        for k in range(2, len(state) + 1):
            new = state[:k][::-1] + state[k:]
            if new not in seen:
                seen.add(new)
                queue.append((new, flips + 1))
    return 0
```

PATTERN 23 · SORTING · PROBLEM 19 OF 20

1329. Sort the Matrix Diagonally

Medium · Array, Matrix, Sorting

leetcode.com/problems/sort-the-matrix-diagonally

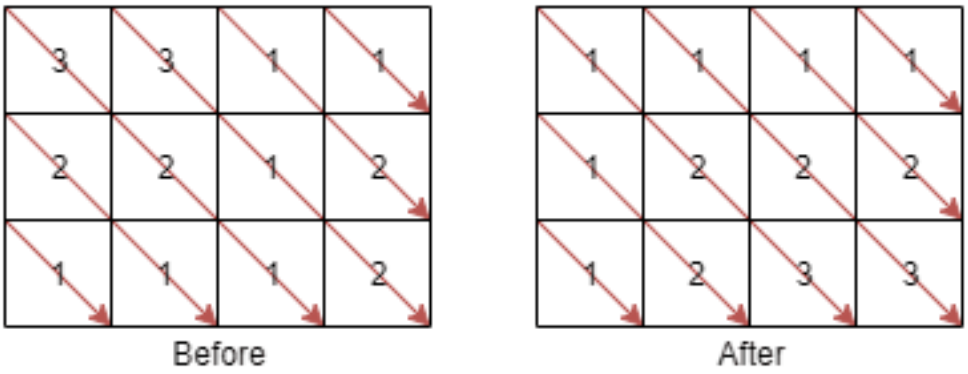
Also appears in pattern 38 K-way Merge

PROBLEM

A **matrix diagonal** is a diagonal line of cells starting from some cell in either the topmost row or leftmost column and going in the bottom-right direction until reaching the matrix's end. For example, the **matrix diagonal** starting from `mat[2][0]`, where `mat` is a `6 x 3` matrix, includes cells `mat[2][0]`, `mat[3][1]`, and `mat[4][2]`.

Given an `m x n` matrix `mat` of integers, sort each **matrix diagonal** in ascending order and return *the resulting matrix*.

Example 1:




```
Input: mat = [[3,3,1,1],[2,2,1,2],[1,1,1,2]]
Output: [[1,1,1,1],[1,2,2,2],[1,2,3,3]]
```

Example 2:

```
Input: mat =
[[11,25,66,1,69,7],[23,55,17,45,15,52],[75,31,36,44,58,8],[22,27,33,25,68,4],[84,28,14,11,5,50]]
Output:
[[5,17,4,1,52,7],[11,11,25,45,8,69],[14,23,25,44,58,15],[22,27,31,36,50,66],[84,28,75,33,55,68]]
```

Constraints:

- `m == mat.length`
- `n == mat[i].length`
- `1 <= m, n <= 100`
- `1 <= mat[i][j] <= 100`

SOLUTION (PYTHON 3)

Solution 1: Sorting

> **How to think about it**
>
> Cells on one diagonal share j-i. Extract each diagonal, sort it, and write it back. Mapping by m-i+j gives a nonnegative bucket; sorting those buckets in reverse and popping while scanning restores increasing order from top-left to bottom-right.

We can treat each diagonal of the matrix as an array, sort these arrays, and then fill the sorted elements back into the original matrix.

Specifically, we denote the number of rows in the matrix as m and the number of columns as n. Since any two elements (i_1, j_1) and (i_2, j_2) on the same diagonal satisfy $j_1 - i_1 = j_2 - i_2$, we can determine each diagonal based on the value of j - i. To ensure the value is positive, we add an offset m, that is, m - i + j.

Finally, we fill the sorted elements of each diagonal back into the original matrix.

The time complexity is O(m × n × log min(m, n)), and the space complexity is O(m × n). Where m and n are the number of rows and columns in the matrix, respectively.

Python 3

```python
class Solution:
    def diagonalSort(self, mat: List[List[int]]) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        g = [[] for _ in range(m + n)]
        for i, row in enumerate(mat):
            for j, x in enumerate(row):
                g[m - i + j].append(x)
        for e in g:
            e.sort(reverse=True)
        for i in range(m):
            for j in range(n):
                mat[i][j] = g[m - i + j].pop()
        return mat
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Sort the ANTI-diagonals (top-right to bottom-left)
# instead
# 2. Sort every diagonal in DESCENDING order


# ============================================================
# FOLLOW-UP 1: Sort the ANTI-diagonals (top-right to
# bottom-left) instead
# ============================================================

# ---- follow-up 1: function ----
# Answer: Cells on one anti-diagonal share the same r + c;
# collect them, sort and write them back.
# Time: O(m * n log min(m, n)).  Space: O(m * n).
def sort_anti_diagonals(mat):
    rows, cols = len(mat), len(mat[0])
    groups = {}
    for r in range(rows):
        for c in range(cols):
            groups.setdefault(r + c, []).append(mat[r][c])
    for key in groups:
        groups[key].sort()
    used = {}
    for r in range(rows):
        for c in range(cols):
            index = used.get(r + c, 0)
            mat[r][c] = groups[r + c][index]
            used[r + c] = index + 1
    return mat


# ============================================================
# FOLLOW-UP 2: Sort every diagonal in DESCENDING order
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same grouping by r - c, with the largest
# number first.
# Time: O(m * n log min(m, n)).  Space: O(m * n).
```

```
def sort_diagonals_descending(mat):
    rows, cols = len(mat), len(mat[0])
    groups = {}
    for r in range(rows):
        for c in range(cols):
            groups.setdefault(r - c, []).append(mat[r][c])
    for key in groups:
        groups[key].sort(reverse=True)
    used = {}
    for r in range(rows):
        for c in range(cols):
            index = used.get(r - c, 0)
            mat[r][c] = groups[r - c][index]
            used[r - c] = index + 1
    return mat
```

PATTERN 24 OF 41

Fast & Slow Pointers (Linked List)

20 problems · 6 Easy · 14 Medium · 0 Hard

THE IDEA

One pointer moves two steps, the other one step. If there is a cycle they meet; when the fast one reaches the end, the slow one is in the middle.

Signals: linked list cycle, start of the cycle, middle of a list, palindrome list, happy number, find the duplicate number.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
slow = fast = head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
    if slow is fast:
        return True          # cycle
return False
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Linked List Cycle	Easy	LC 141	1268
2	Middle of the Linked List	Easy	LC 876	1271
3	Happy Number	Easy	LC 202	1273
4	Palindrome Linked List	Easy	LC 234	1276
5	Intersection of Two Linked Lists	Easy	LC 160	1279
6	Remove Duplicates from Sorted List	Easy	LC 83	1283
7	★ **Linked List Cycle II**	Medium	LC 142	1286
8	★ **Remove Nth Node From End of List**	Medium	LC 19	1290
9	★ **Find the Duplicate Number** (see page)	Medium	LC 287	43
10	★ **Reorder List**	Medium	LC 143	1293
11	Maximum Twin Sum of a Linked List	Medium	LC 2130	1296
12	Split Linked List in Parts	Medium	LC 725	1300
13	Delete the Middle Node of a Linked List	Medium	LC 2095	1303
14	Rotate List	Medium	LC 61	1306
15	Convert Sorted List to Binary Search Tree	Medium	LC 109	1309
16	Linked List Components	Medium	LC 817	1312
17	Next Greater Node In Linked List (see page)	Medium	LC 1019	452
18	★ **Odd Even Linked List**	Medium	LC 328	1315
19	★ **Partition List**	Medium	LC 86	1318
20	★ **Sort List**	Medium	LC 148	1321

141. Linked List Cycle

Easy · Hash Table, Linked List, Two Pointers, Floyd Cycle Detection

leetcode.com/problems/linked-list-cycle

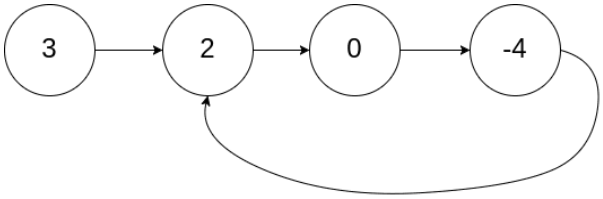
PROBLEM

Given `head`, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to. **Note that `pos` is not passed as a parameter**.

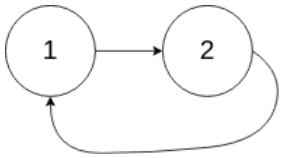
Return `true` *if there is a cycle in the linked list*. Otherwise, return `false`.

Example 1:




```
Input: head = [3,2,0,-4], pos = 1
Output: true
Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).
```

Example 2:




```
Input: head = [1,2], pos = 0
Output: true
Explanation: There is a cycle in the linked list, where the tail connects to the 0th node.
```

Example 3:

```
Input: head = [1], pos = -1
Output: false
Explanation: There is no cycle in the linked list.
```

Constraints:

- The number of the nodes in the list is in the range `[0, 10^4]`.
- `-10^5 <= Node.val <= 10^5`
- `pos` is `-1` or a **valid index** in the linked-list.

Follow up: Can you solve it using `O(1)` (i.e. constant) memory?

SOLUTIONS (PYTHON 3)

Solution 1: Hash Table

> **How to think about it**
>
> Detect a cycle. n≤ 10^4. Store visited nodes in a set; a repeat means a cycle, null means none. Simple, O(n) space. The follow-up asks for constant space.

We can traverse the linked list and use a hash table s to record each node. When a node appears for the second time, it indicates that there is a cycle, and we directly return `true`. Otherwise, when the linked list traversal ends, we return `false`.

The time complexity is O(n), and the space complexity is O(n), where n is the number of nodes in the linked list.

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None


class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        s = set()
        while head:
            if head in s:
                return True
            s.add(head)
            head = head.next
        return False
```

Solution 2: Fast and Slow Pointers

> **How to think about it**
>
> Solution 1 uses linear extra space. A fast pointer of two steps and a slow pointer of one must meet inside a cycle, or the fast pointer hits the end. Only two pointers.

We define two pointers, fast and slow, both initially pointing to head.

The fast pointer moves two steps at a time, and the slow pointer moves one step at a time, in a continuous loop. When the fast and slow pointers meet, it indicates that there is a cycle in the linked list. If the loop ends without the pointers meeting, it indicates that there is no cycle in the linked list.

The time complexity is O(n), and the space complexity is O(1), where n is the number of nodes in the linked list.

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None


class Solution:
    def hasCycle(self, head: ListNode) -> bool:
        slow = fast = head
        while fast and fast.next:
            slow, fast = slow.next, fast.next.next
            if slow == fast:
                return True
        return False
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the LENGTH of the cycle
# 2. Detect the cycle with a set of visited nodes (O(n)
# extra space)


# ============================================================
# FOLLOW-UP 1: Return the LENGTH of the cycle
# ============================================================

# ---- follow-up 1: function ----
# Answer: When the two pointers meet inside the cycle,
# keep moving one of them until it comes back to the
# meeting node, counting the steps.
# Time: O(n).  Space: O(1).
def cycle_length(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            length = 1
            node = slow.next
            while node is not slow:
                node = node.next
                length += 1
            return length
    return 0


# ============================================================
# FOLLOW-UP 2: Detect the cycle with a set of visited
# nodes (O(n) extra space)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk the list and remember every node seen;
# meeting a node again means a cycle. Simple, but it needs
# the memory the two-pointer method avoids.
# Time: O(n).  Space: O(n).
def has_cycle_with_set(head):
    seen = set()
    node = head
    while node:
        if node in seen:
            return True
        seen.add(node)
        node = node.next
    return False
```

876. Middle of the Linked List

Easy · Linked List, Two Pointers

leetcode.com/problems/middle-of-the-linked-list

PROBLEM

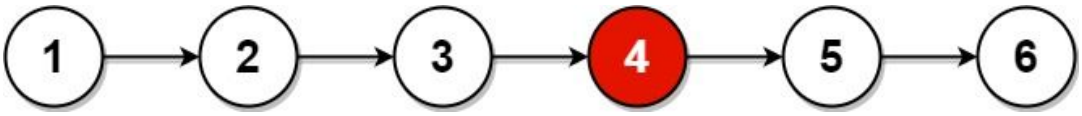
Given the `head` of a singly linked list, return *the middle node of the linked list*.

If there are two middle nodes, return **the second middle** node.

Example 1:

```
Input: head = [1,2,3,4,5]
Output: [3,4,5]
Explanation: The middle node of the list is node 3.
```

Example 2:




```
Input: head = [1,2,3,4,5,6]
Output: [4,5,6]
Explanation: Since the list has two middle nodes with values 3 and 4, we return the second one.
```

Constraints:

- The number of nodes in the list is in the range `[1, 100]`.
- `1 <= Node.val <= 100`

SOLUTION (PYTHON 3)

Solution 1: Fast and Slow Pointers

> **How to think about it**
>
> Return the middle node, the later one when the length is even. Counting the length then walking half needs two passes. A fast pointer moves two steps and a slow pointer one; when the fast pointer ends, the slow pointer is at the middle. One walk is enough.

We define two pointers fast and slow, both initially pointing to the head of the linked list.

The fast pointer fast moves two steps at a time, while the slow pointer slow moves one step at a time. When the fast pointer reaches the end of the linked list, the node pointed to by the slow pointer is the middle node.

The time complexity is O(n), where n is the length of the linked list. The space complexity is O(1).

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        slow = fast = head
        while fast and fast.next:
            slow, fast = slow.next, fast.next.next
        return slow
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. For an even length return the FIRST of the two middle
# nodes
# 2. Split the list into two halves


# ============================================================
# FOLLOW-UP 1: For an even length return the FIRST of the
# two middle nodes
# ============================================================

# ---- follow-up 1: function ----
# Answer: Start the fast pointer one step behind (compare
# fast.next and fast.next.next).
# Time: O(n).  Space: O(1).
def first_middle(head):
    slow = head
    fast = head
    while fast.next and fast.next.next:
        slow = slow.next
        fast = fast.next.next
    return slow


# ============================================================
# FOLLOW-UP 2: Split the list into two halves
# ============================================================

# ---- follow-up 2: function ----
# Answer: Find the first middle, then cut the list after
# it.
# Time: O(n).  Space: O(1).
def split_in_halves(head):
    if head is None:
        return None, None
    slow = fast = head
    while fast.next and fast.next.next:
        slow = slow.next
        fast = fast.next.next
    second = slow.next
    slow.next = None
    return head, second
```

202. Happy Number

Easy · Hash Table, Math, Two Pointers, Floyd Cycle Detection

leetcode.com/problems/happy-number

Also appears in pattern 34 Math / Number Theory

PROBLEM

Write an algorithm to determine if a number `n` is happy.

A **happy number** is a number defined by the following process:

- Starting with any positive integer, replace the number by the sum of the squares of its digits.
- Repeat the process until the number equals 1 (where it will stay), or it **loops endlessly in a cycle** which does not include 1.
- Those numbers for which this process **ends in 1** are happy.

Return `true` *if* `n` *is a happy number, and* `false` *if not*.

Example 1:

```
Input: n = 19
Output: true
Explanation:
12 + 92 = 82
82 + 22 = 68
62 + 82 = 100
12 + 02 + 02 = 1
```

Example 2:

```
Input: n = 2
Output: false
```

Constraints:

- $1 <= n <= 2^{31} - 1$

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Repeatedly replacing a number by the sum of squares of its digits follows the definition, but the sequence may cycle instead of reaching 1. The values quickly fall into a bounded range, so a hash set can record numbers already seen. A repeat means a cycle (not happy); reaching 1 means it is happy. Simulation and cycle checks run together.

Python 3

```python
class Solution:
    def isHappy(self, n: int) -> bool:
        vis = set()
        while n != 1 and n not in vis:
            vis.add(n)
            x = 0
            while n:
                n, v = divmod(n, 10)
                x += v * v
            n = x
        return n == 1
```

Complexity. Time: O(log n). Space: O(log n).

Solution 2

How to think about it

The hash set is correct but uses extra memory. The square-sum map is a deterministic iteration, so a cycle can be found without storing the full history. Floyd's pointers suffice: the slow pointer moves once and the fast pointer twice; if they meet at 1, the number is happy, and space becomes constant.

Python 3

```python
class Solution:
    def isHappy(self, n: int) -> bool:
        def next(x):
            y = 0
            while x:
                x, v = divmod(x, 10)
                y += v * v
            return y

        slow, fast = n, next(n)
        while slow != fast:
            slow, fast = next(slow), next(next(fast))
        return slow == 1
```

Complexity. Time: O(log n). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. In another base
# 2. How many steps does a happy number take to reach 1?


# ============================================================
# FOLLOW-UP 1: In another base
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same digit-square sum, with digits taken in
# the given base.
# Time: O(log n) per step, and the walk reaches 1 or loops
# within a bounded number of steps.  Space: O(1).
def is_happy_in_base(n, base):
    def step(x):
        total = 0
        while x > 0:
            digit = x % base
            total += digit * digit
            x //= base
        return total

    slow = n
    fast = step(n)
    while fast != 1 and slow != fast:
        slow = step(slow)
        fast = step(step(fast))
    return fast == 1


# ============================================================
# FOLLOW-UP 2: How many steps does a happy number take to
# reach 1?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Repeat the digit-square sum and count the steps
```

```
# (a seen-set stops an unhappy number).
# Time: O(log n) per step.  Space: O(k), k = steps.
def steps_to_happy(n):
    seen = set()
    steps = 0
    while n != 1:
        if n in seen:
            return -1
        seen.add(n)
        total = 0
        while n > 0:
            digit = n % 10
            total += digit * digit
            n //= 10
        n = total
        steps += 1
    return steps
```

234. Palindrome Linked List

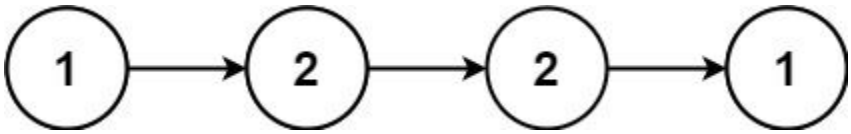
Easy · Stack, Recursion, Linked List, Two Pointers

leetcode.com/problems/palindrome-linked-list

PROBLEM

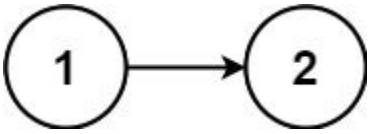
Given the `head` of a singly linked list, return `true` *if it is a palindrome or* `false` *otherwise*.

Example 1:




```
Input: head = [1,2,2,1]
Output: true
```

Example 2:




```
Input: head = [1,2]
Output: false
```

Constraints:

- The number of nodes in the list is in the range `[1, 10^5]`.
- `0 <= Node.val <= 9`

Follow up:

Could you do it in

`O(n)`

time and

`O(1)`

space?

SOLUTION (PYTHON 3)

Solution 1: Fast and Slow Pointers

> **How to think about it**
>
> Copying values into an array works, but we want constant extra space. A palindrome is symmetric about the midpoint, so we find the middle, reverse the second half, and compare. Slow/fast pointers locate the mid; the second half is reversed in place and compared with the first half.

We can use fast and slow pointers to find the middle of the linked list, then reverse the right half of the list. After that, we traverse both halves simultaneously, checking if the corresponding node values are equal. If any pair of values is unequal, it's not a palindrome linked list; otherwise, it is a palindrome linked list.

The time complexity is O(n), where n is the length of the linked list. The space complexity is O(1).

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def isPalindrome(self, head: Optional[ListNode]) -> bool:
        slow, fast = head, head.next
        while fast and fast.next:
            slow, fast = slow.next, fast.next.next
        pre, cur = None, slow.next
        while cur:
            t = cur.next
            cur.next = pre
            pre, cur = cur, t
        while pre:
            if pre.val != head.val:
                return False
            pre, head = pre.next, head.next
        return True
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Put the list back the way it was after the check
# 2. The list is a palindrome when you ignore one node (at
# most one node may differ)


# ============================================================
# FOLLOW-UP 1: Put the list back the way it was after the
# check
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 1: function ----
# Answer: Reverse the second half for the comparison and
# reverse it again before returning.
# Time: O(n).  Space: O(1).
def reverse_list(head):
    previous = None
    while head:
        following = head.next
        head.next = previous
        previous = head
        head = following
    return previous


def is_palindrome_restored(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    second = reverse_list(slow)
    first_half = head
    second_half = second
    result = True
    while second_half:
        if first_half.val != second_half.val:
```

```
            result = False
            break
        first_half = first_half.next
        second_half = second_half.next
    reverse_list(second)                            # restore the second half
    return result


# ============================================================
# FOLLOW-UP 2: The list is a palindrome when you ignore
# one node (at most one node may differ)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Compare from the ends using the values in an
# array; on the first mismatch try skipping the left or
# the right value.
# Time: O(n).  Space: O(n).
```

```
def almost_palindrome_list(head):
    values = []
    while head:
        values.append(head.val)
        head = head.next

    def is_pal(low, high):
        while low < high:
            if values[low] != values[high]:
                return False
            low += 1
            high -= 1
        return True

    low, high = 0, len(values) - 1
    while low < high:
        if values[low] != values[high]:
            return is_pal(low + 1, high) or is_pal(low, high - 1)
        low += 1
        high -= 1
    return True
```

160. Intersection of Two Linked Lists

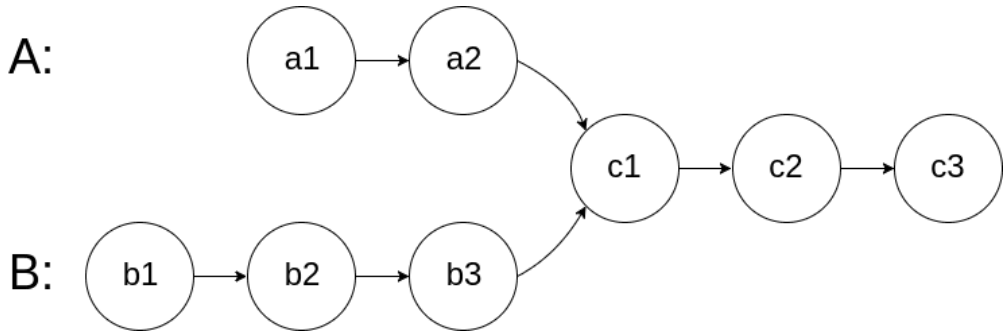
Easy · Hash Table, Linked List, Two Pointers

leetcode.com/problems/intersection-of-two-linked-lists

PROBLEM

Given the heads of two singly linked-lists `headA` and `headB`, return *the node at which the two lists intersect*. If the two linked lists have no intersection at all, return `null`.

For example, the following two linked lists begin to intersect at node `c1`:



The test cases are generated such that there are no cycles anywhere in the entire linked structure.

Note that the linked lists must **retain their original structure** after the function returns.

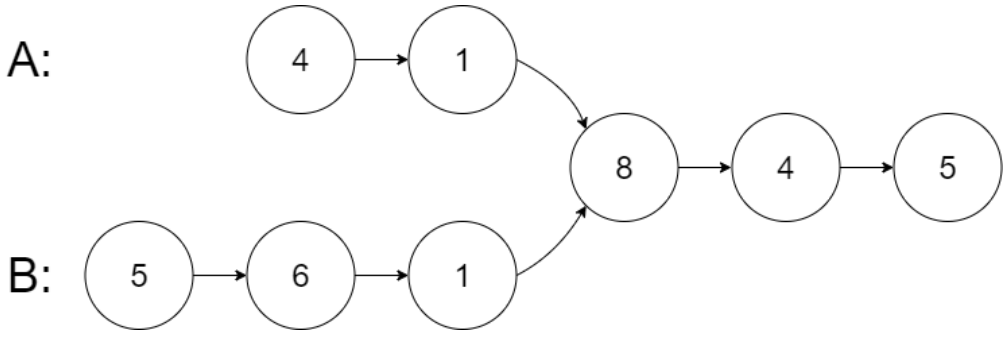
Custom Judge:

The inputs to the **judge** are given as follows (your program is **not** given these inputs):

- `intersectVal` - The value of the node where the intersection occurs. This is `0` if there is no intersected node.
- `listA` - The first linked list.
- `listB` - The second linked list.
- `skipA` - The number of nodes to skip ahead in `listA` (starting from the head) to get to the intersected node.
- `skipB` - The number of nodes to skip ahead in `listB` (starting from the head) to get to the intersected node.

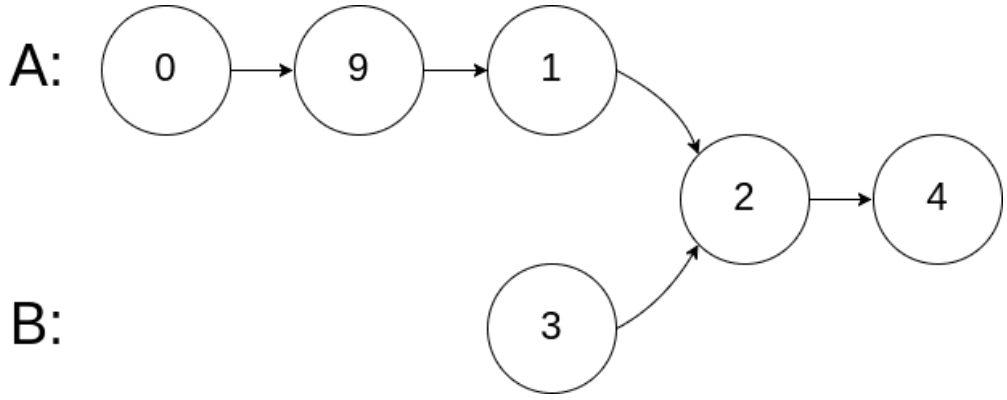
The judge will then create the linked structure based on these inputs and pass the two heads, `headA` and `headB` to your program. If you correctly return the intersected node, then your solution will be **accepted**.

Example 1:



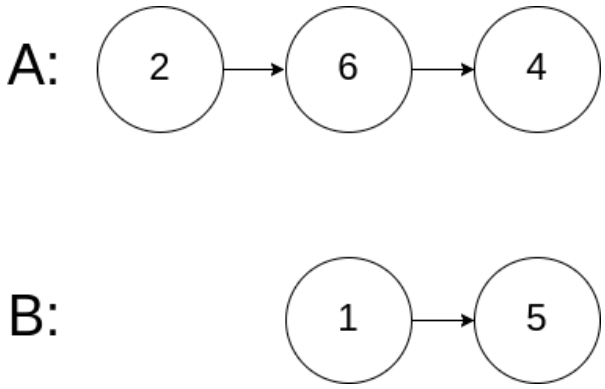
Input: intersectVal = 8, listA = [4,1,8,4,5], listB = [5,6,1,8,4,5], skipA = 2, skipB = 3
Output: Intersected at '8'
Explanation: The intersected node's value is 8 (note that this must not be 0 if the two lists intersect).
From the head of A, it reads as [4,1,8,4,5]. From the head of B, it reads as [5,6,1,8,4,5]. There are 2 nodes before the intersected node in A; There are 3 nodes before the intersected node in B.
- Note that the intersected node's value is not 1 because the nodes with value 1 in A and B (2nd node in A and 3rd node in B) are different node references. In other words, they point to two different locations in memory, while the nodes with value 8 in A and B (3rd node in A and 4th node in B) point to the same location in memory.

Example 2:



Input: intersectVal = 2, listA = [1,9,1,2,4], listB = [3,2,4], skipA = 3, skipB = 1
Output: Intersected at '2'
Explanation: The intersected node's value is 2 (note that this must not be 0 if the two lists intersect).
From the head of A, it reads as [1,9,1,2,4]. From the head of B, it reads as [3,2,4]. There are 3 nodes before the intersected node in A; There are 1 node before the intersected node in B.

Example 3:



Input: intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2
Output: No intersection
Explanation: From the head of A, it reads as [2,6,4]. From the head of B, it reads as [1,5]. Since the two lists do not intersect, intersectVal must be 0, while skipA and skipB can be arbitrary values.
Explanation: The two lists do not intersect, so return null.

Constraints:

- The number of nodes of `listA` is in the `m`.
- The number of nodes of `listB` is in the `n`.
- `1 <= m, n <= 3 * 10^4`
- `1 <= Node.val <= 10^5`
- `0 <= skipA <= m`

- `0 <= skipB <= n`
- `intersectVal` is `0` if `listA` and `listB` do not intersect.
- `intersectVal == listA[skipA] == listB[skipB]` if `listA` and `listB` intersect.

Follow up:

Could you write a solution that runs in

`O(m + n)`

time and use only

`O(1)`

memory?

SOLUTION (PYTHON 3)

Solution 1: Two Pointers

> **How to think about it**
>
> Find the first common node. Storing one list in a set is O(m+n) time and O(m) space; the follow-up wants O(1) memory. $m,n \leq 3 \times 10^4$. Each pointer walks its list then the other, so both travel a+b. They meet at the intersection, or both hit null. No need to measure lengths first.

We use two pointers a and b to point to the heads of the two linked lists headA and headB, respectively.

Traverse the linked lists simultaneously. When a reaches the end of headA, redirect it to the head of headB. Similarly, when b reaches the end of headB, redirect it to the head of headA.

If the two pointers meet, the node they point to is the first common node. If they do not meet, it means the two linked lists have no common nodes, and both pointers will point to `null`. Return either pointer.

The time complexity is O(m + n), where m and n are the lengths of the linked lists headA and headB, respectively. The space complexity is O(1).

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None


class Solution:
    def getIntersectionNode(self, headA: ListNode, headB: ListNode) -> ListNode:
        a, b = headA, headB
        while a != b:
            a = a.next if a else headB
            b = b.next if b else headA
        return a
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. How many nodes do the two lists share?
# 2. Find the intersection with a set (O(n) extra space)


# ============================================================
# FOLLOW-UP 1: How many nodes do the two lists share?
# ============================================================

# ---- follow-up 1: function ----
```

```
# Answer: Compute both lengths, advance the longer list by
# the difference, then walk together until they meet; the
# shared part is the rest.
# Time: O(a + b).  Space: O(1).
def shared_length(a, b):
    def length(node):
        count = 0
        while node:
            count += 1
            node = node.next
        return count

    la, lb = length(a), length(b)
    while la > lb:
        a = a.next
        la -= 1
    while lb > la:
        b = b.next
        lb -= 1
    while a is not b:
        a = a.next
        b = b.next
        la -= 1
    return la if a is not None else 0


# ============================================================
# FOLLOW-UP 2: Find the intersection with a set (O(n)
# extra space)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Put all the nodes of the first list in a set,
# then walk the second list until a node is in the set.
# Time: O(a + b).  Space: O(a).
def intersection_with_set(a, b):
    seen = set()
    while a:
        seen.add(a)
        a = a.next
    while b:
        if b in seen:
            return b
        b = b.next
    return None
```

83. Remove Duplicates from Sorted List

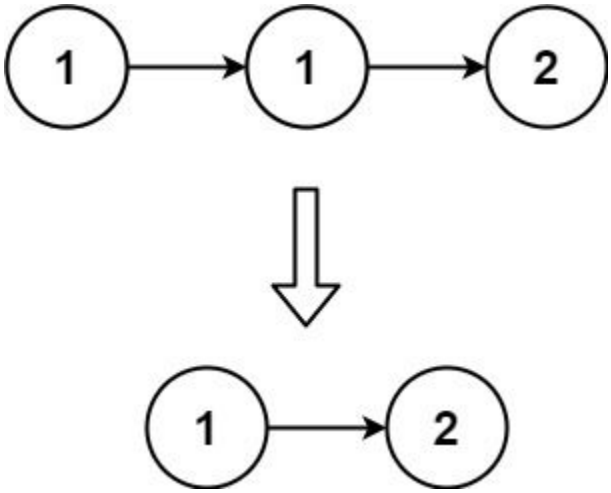
Easy · Linked List

leetcode.com/problems/remove-duplicates-from-sorted-list

PROBLEM

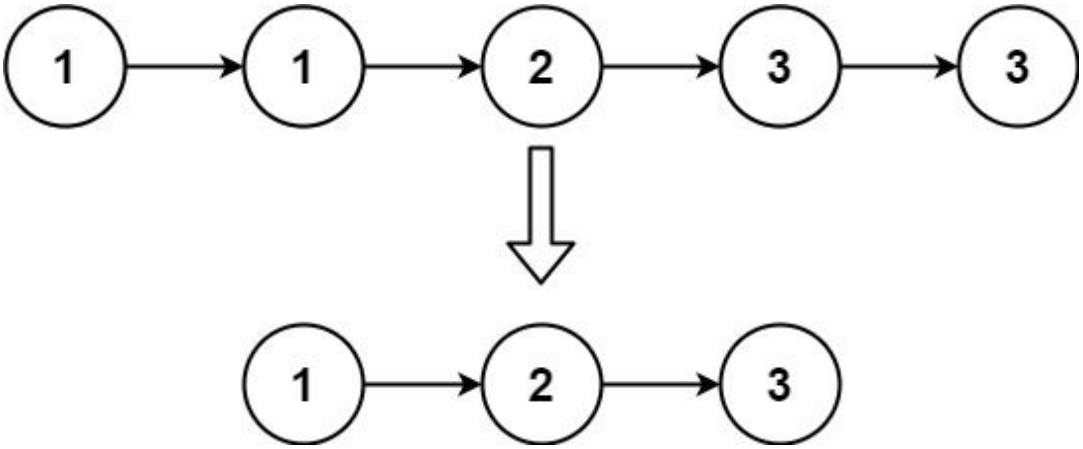
Given the `head` of a sorted linked list, *delete all duplicates such that each element appears only once*. Return *the linked list **sorted** as well*.

Example 1:




```
Input: head = [1,1,2]
Output: [1,2]
```

Example 2:




```
Input: head = [1,1,2,3,3]
Output: [1,2,3]
```

Constraints:

- The number of nodes in the list is in the range `[0, 300]`.
- `-100 <= Node.val <= 100`
- The list is guaranteed to be **sorted** in ascending order.

SOLUTION (PYTHON 3)

Solution 1: Single Pass

> **How to think about it**
>
> The first idea is dump values into a set and rebuild. Correct, and n ≤ 300 is tiny, but that ignores sorted adjacency and uses extra space. The bottleneck is not rewriting links in place. Duplicates sit next to each other, so skipping cur.next when values match is enough. Unlike the previous problem, we keep the first occurrence. A single pointer walk: drop the next node when equal, advance only when different. The head is always kept, so no dummy is needed.

We use a pointer cur to traverse the linked list. If the element corresponding to the current cur is the same as the element corresponding to cur.next, we set the next pointer of cur to point to the next node of cur.next. Otherwise, it means that the element corresponding to cur in the linked list is not duplicated, so we can move the cur pointer to the next node.

After the traversal ends, return the head node of the linked list.

The time complexity is O(n), where n is the length of the linked list. The space complexity is O(1).

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def deleteDuplicates(self, head: Optional[ListNode]) -> Optional[ListNode]:
        cur = head
        while cur and cur.next:
            if cur.val == cur.next.val:
                cur.next = cur.next.next
            else:
                cur = cur.next
        return head
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Remove EVERY value that appears more than once (keep
# only distinct values)
# 2. The list is NOT sorted: keep the first occurrence of
# each value


# ============================================================
# FOLLOW-UP 1: Remove EVERY value that appears more than
# once (keep only distinct values)
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 1: function ----
# Answer: Use a dummy head; skip a whole run of equal
# values when it has more than one node.
# Time: O(n).  Space: O(1).
def delete_all_duplicates(head):
    dummy = ListNode(0, head)
    previous = dummy
    node = head
    while node:
```

```
        if node.next and node.next.val == node.val:
            value = node.val
            while node and node.val == value:
                node = node.next
            previous.next = node
        else:
            previous = node
            node = node.next
    return dummy.next


# ============================================================
# FOLLOW-UP 2: The list is NOT sorted: keep the first
# occurrence of each value
# ============================================================

# ---- follow-up 2: function ----
# Answer: A set of the values already kept; unlink the
# nodes whose value was seen.
# Time: O(n).  Space: O(n).
def remove_duplicates_unsorted(head):
    seen = set()
    previous = None
    node = head
    while node:
        if node.val in seen:
            previous.next = node.next
        else:
            seen.add(node.val)
            previous = node
        node = node.next
    return head
```

142. Linked List Cycle II

Medium · Hash Table, Linked List, Two Pointers, Floyd Cycle Detection

leetcode.com/problems/linked-list-cycle-ii

★ **TOP 7 PROBLEM** in Fast & Slow Pointers (Linked List): master this one first

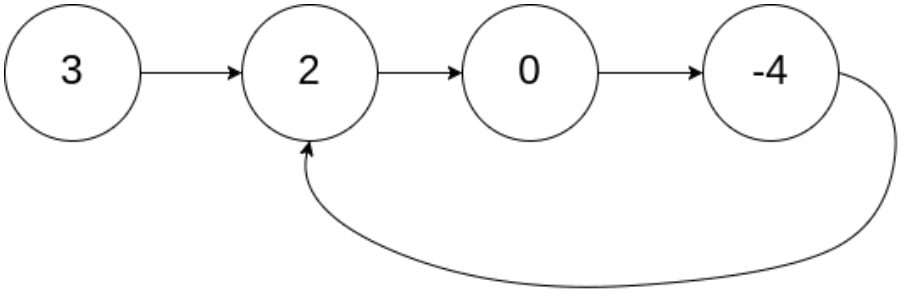
PROBLEM

Given the `head` of a linked list, return *the node where the cycle begins. If there is no cycle, return* `null`.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to (**0-indexed**). It is `-1` if there is no cycle. **Note that** `pos` **is not passed as a parameter**.

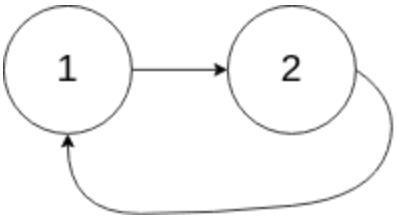
Do not modify the linked list.

Example 1:




```
Input: head = [3,2,0,-4], pos = 1
Output: tail connects to node index 1
Explanation: There is a cycle in the linked list, where tail connects to the second node.
```

Example 2:




```
Input: head = [1,2], pos = 0
Output: tail connects to node index 0
Explanation: There is a cycle in the linked list, where tail connects to the first node.
```

Example 3:

```
Input: head = [1], pos = -1
Output: no cycle
Explanation: There is no cycle in the linked list.
```

Constraints:

- The number of the nodes in the list is in the range `[0, 10^4]`.
- `-10^5 <= Node.val <= 10^5`

- `pos` is `-1` or a **valid index** in the linked-list.

Follow up: Can you solve it using `O(1)` (i.e. constant) memory?

SOLUTION (PYTHON 3)

Solution 1: Two Pointers

How to think about it

We must return the cycle entrance, not only detect it. A visited map works, but the follow-up wants O(1) space. n≤ 10^4. After the fast and slow pointers meet, send one pointer back to the head and walk both at the same speed; they meet at the entrance, because the head-to-entrance distance equals the meet-to-entrance distance around the cycle.

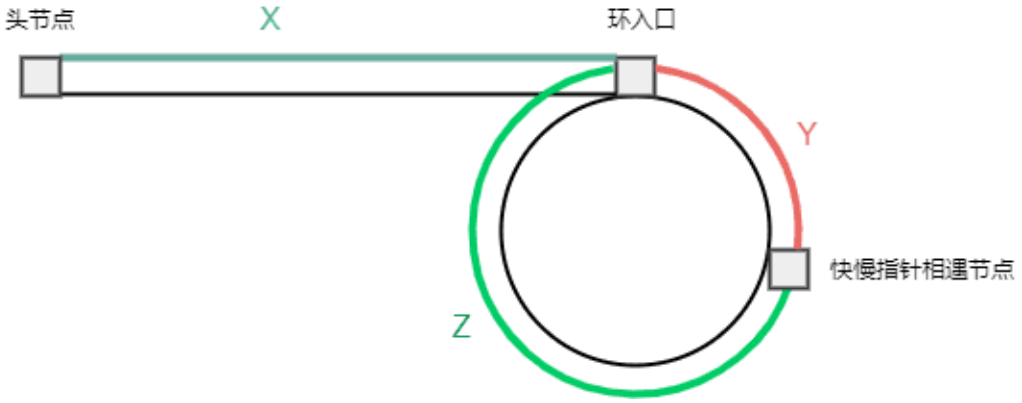
We first use the fast and slow pointers to judge whether the linked list has a ring. If there is a ring, the fast and slow pointers will definitely meet, and the meeting node must be in the ring.

If there is no ring, the fast pointer will reach the tail of the linked list first, and return `null` directly.

If there is a ring, we then define an answer pointer ans to point to the head of the linked list, and then let ans and the slow pointer move forward together, moving one step at a time, until ans and the slow pointer meet, and the meeting node is the ring entrance node.

Why can this find the entrance node of the ring?

Let's assume that the distance from the head node of the linked list to the entrance of the ring is x, the distance from the entrance of the ring to the meeting node is y, and the distance from the meeting node to the entrance of the ring is z. Then the distance traveled by the slow pointer is x + y, and the distance traveled by the fast pointer is x + y + k × (y + z), where k is the number of times the fast pointer goes around the ring.



Because the speed of the fast pointer is twice that of the slow pointer, it is 2 × (x + y) = x + y + k × (y + z), which can be deduced that x + y = k × (y + z), that is x = (k - 1) × (y + z) + z.

That is to say, if we define an answer pointer ans to point to the head of the linked list, and the ans and the slow pointer move forward together, they will definitely meet at the ring entrance.

The time complexity is O(n), where n is the number of nodes in the linked list. The space complexity is O(1).

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None


class Solution:
    def detectCycle(self, head: Optional[ListNode]) -> Optional[ListNode]:
        fast = slow = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow == fast:
                ans = head
                while ans != slow:
                    ans = ans.next
                    slow = slow.next
                return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Remove the cycle (make the last node point to None)
# 2. The length of the tail before the cycle and the
# length of the cycle


# ============================================================
# FOLLOW-UP 1: Remove the cycle (make the last node point
# to None)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Find the cycle start with the two-pointer
# method, then walk around the cycle to the node just
# before the start and cut it.
# Time: O(n).  Space: O(1).
def remove_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            break
    else:
        return head
    start = head
    while start is not slow:
        start = start.next
        slow = slow.next
    last = start
    while last.next is not start:
        last = last.next
    last.next = None
    return head


# ============================================================
# FOLLOW-UP 2: The length of the tail before the cycle and
# the length of the cycle
# ============================================================

# ---- follow-up 2: function ----
# Answer: The tail length is the number of steps from the
```

```
# head to the cycle start; the cycle length is one lap
# from the start.
# Time: O(n).  Space: O(1).
def tail_and_cycle_length(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            break
    else:
        return None
    start = head
    tail = 0
    while start is not slow:
        start = start.next
        slow = slow.next
```

```
        tail += 1
    lap = 1
    node = start.next
    while node is not start:
        node = node.next
        lap += 1
    return tail, lap
```

19. Remove Nth Node From End of List

Medium · Linked List, Two Pointers

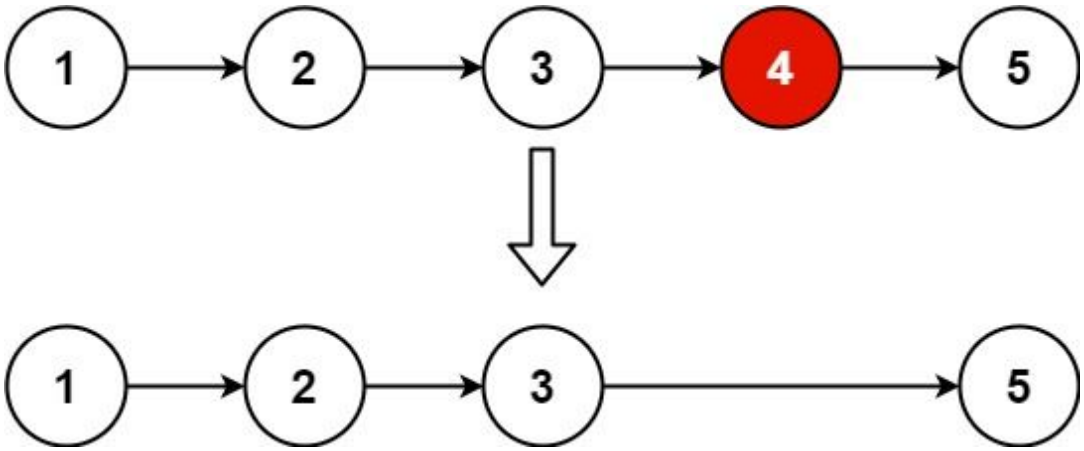
leetcode.com/problems/remove-nth-node-from-end-of-list

★ **TOP 7 PROBLEM** in Fast & Slow Pointers (Linked List): master this one first

PROBLEM

Given the `head` of a linked list, remove the n^{th} node from the end of the list and return its head.

Example 1:




```
Input: head = [1,2,3,4,5], n = 2
Output: [1,2,3,5]
```

Example 2:

```
Input: head = [1], n = 1
Output: []
```

Example 3:

```
Input: head = [1,2], n = 1
Output: [1]
```

Constraints:

- The number of nodes in the list is `sz`.
- `1 <= sz <= 30`
- `0 <= Node.val <= 100`
- `1 <= n <= sz`

Follow up: Could you do this in one pass?

SOLUTION (PYTHON 3)

Solution 1: Fast and Slow Pointers

How to think about it

The first idea is to count the length L, then walk L-n steps and delete. sz≤ 30 would pass, but the follow-up asks for one pass. Until we reach the end we do not know where the n-th node from the end is, and deletion still needs its predecessor. Give the fast pointer a head start of n steps, then move both; when fast hits the tail, slow sits on the predecessor. A dummy node makes deleting the head the same as deleting a middle node. So one walk with fast and slow locates and removes the node.

We define two pointers `fast` and `slow`, both initially pointing to the dummy head node of the linked list.

Next, the `fast` pointer moves forward n steps first, then `fast` and `slow` pointers move forward together until the `fast` pointer reaches the end of the linked list. At this point, the node pointed to by `slow.next` is the predecessor of the n-th node from the end, and we can delete it.

The time complexity is O(n), where n is the length of the linked list. The space complexity is O(1).

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def removeNthFromEnd(self, head: Optional[ListNode], n: int) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        fast = slow = dummy
        for _ in range(n):
            fast = fast.next
        while fast.next:
            slow, fast = slow.next, fast.next
        slow.next = slow.next.next
        return dummy.next
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Also return the value that was removed
# 2. Keep ONLY the last n nodes


# ========================================================
# FOLLOW-UP 1: Also return the value that was removed
# ========================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 1: function ----
# Answer: The same two-pointer gap of n nodes; read the
# value of the node before unlinking it.
# Time: O(n).  Space: O(1).
def remove_nth_from_end_with_value(head, n):
    dummy = ListNode(0, head)
    fast = slow = dummy
    for i in range(n):
        fast = fast.next
    while fast.next:
        fast = fast.next
        slow = slow.next
    removed = slow.next
```

```
    slow.next = removed.next
    return dummy.next, removed.val


# ============================================================
# FOLLOW-UP 2: Keep ONLY the last n nodes
# ============================================================

# ---- follow-up 2: function ----
# Answer: Find the (n + 1)-th node from the end and cut
# the list after it; return the rest.
# Time: O(n).  Space: O(1).
def keep_last_n(head, n):
    fast = head
    for i in range(n):
        if fast is None:
            return head
        fast = fast.next
    slow = head
    if fast is None:
        return head
    previous = None
    while fast:
        previous = slow
        fast = fast.next
        slow = slow.next
    return slow
```

143. Reorder List

Medium · Stack, Recursion, Linked List, Two Pointers

leetcode.com/problems/reorder-list

★ **TOP 7 PROBLEM** in Fast & Slow Pointers (Linked List): master this one first

PROBLEM

You are given the head of a singly linked-list. The list can be represented as:

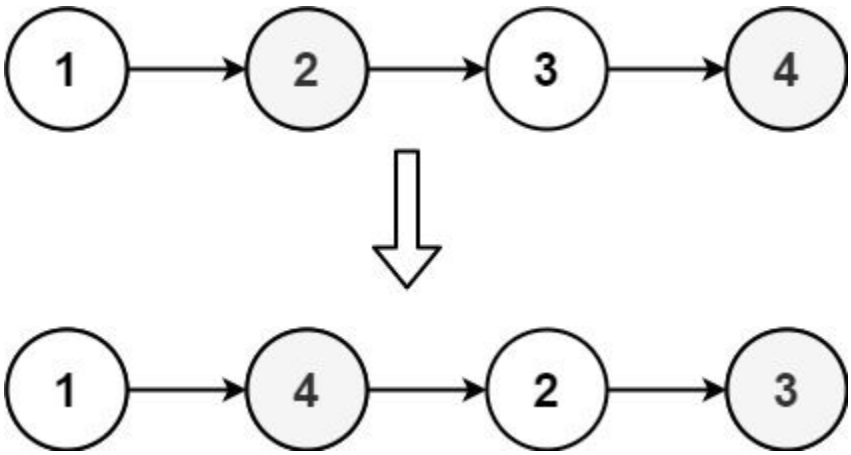
```
L0 → L1 → … → Ln - 1 → Ln
```

Reorder the list to be on the following form:

```
L0 → Ln → L1 → Ln - 1 → L2 → Ln - 2 → …
```

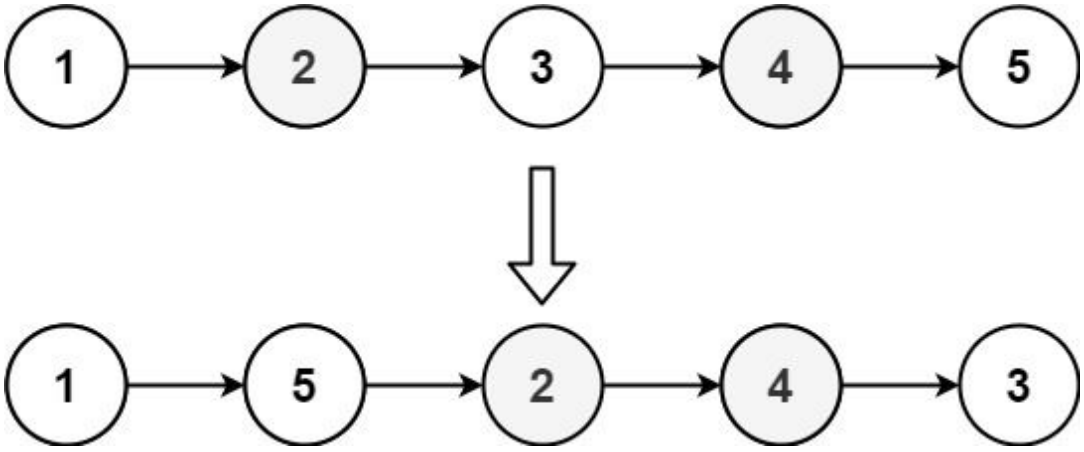
You may not modify the values in the list's nodes. Only nodes themselves may be changed.

Example 1:




```
Input: head = [1,2,3,4]
Output: [1,4,2,3]
```

Example 2:




```
Input: head = [1,2,3,4,5]
Output: [1,5,2,4,3]
```

Constraints:

- The number of nodes in the list is in the range `[1, 5 * 10^4]`.
- `1 <= Node.val <= 1000`

SOLUTION (PYTHON 3)

Solution 1: Fast and Slow Pointers + Reverse List + Merge Lists

How to think about it

Reorder as $L_0 \to L_n \to L_1 \to L_{n-1} \ldots$. An array of pointers is O(n) space. $n \leq 5 \times 10^4$. The second half must be reversed and woven into the first: find the midpoint with two pointers, cut, reverse the second half, and merge alternately. Three linear passes, in place.

We first use fast and slow pointers to find the midpoint of the linked list, then reverse the second half of the list, and finally merge the two halves.

The time complexity is O(n), where n is the length of the linked list. The space complexity is O(1).

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reorderList(self, head: Optional[ListNode]) -> None:
        fast = slow = head
        while fast.next and fast.next.next:
            slow = slow.next
            fast = fast.next.next

        cur = slow.next
        slow.next = None

        pre = None
        while cur:
            t = cur.next
            cur.next = pre
            pre, cur = cur, t
        cur = head

        while pre:
            t = pre.next
            pre.next = cur.next
            cur.next = pre
            cur, pre = pre.next, t
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Reorder with a stack (simpler, O(n) extra space)
# 2. Start with the LAST node: L(n), L(1), L(n-1), L(2),
# ...


# ============================================================
# FOLLOW-UP 1: Reorder with a stack (simpler, O(n) extra
# space)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Push all the nodes on a stack; then alternate
# between taking the next node from the front and the top
# of the stack, stopping when they meet.
```

```
# Time: O(n).  Space: O(n).
def reorder_with_stack(head):
    if head is None:
        return head
    stack = []
    node = head
    while node:
        stack.append(node)
        node = node.next
    node = head
    count = len(stack)
    for i in range(count // 2):
        last = stack.pop()
        following = node.next
        node.next = last
        last.next = following
        node = following
    node.next = None
    return head


# ============================================================
# FOLLOW-UP 2: Start with the LAST node: L(n), L(1),
# L(n-1), L(2), ...
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same method with the roles swapped: take the
# reversed second half first.
# Time: O(n).  Space: O(n).
def reorder_from_back(head):
    values = []
    node = head
    while node:
        values.append(node)
        node = node.next
    order = []
    low, high = 0, len(values) - 1
    while low <= high:
        order.append(values[high])
        if low != high:
            order.append(values[low])
        low += 1
        high -= 1
    for i in range(len(order) - 1):
```

```
        order[i].next = order[i + 1]
    order[-1].next = None
    return order[0]
```

2130. Maximum Twin Sum of a Linked List

Medium · Stack, Linked List, Two Pointers

leetcode.com/problems/maximum-twin-sum-of-a-linked-list

PROBLEM

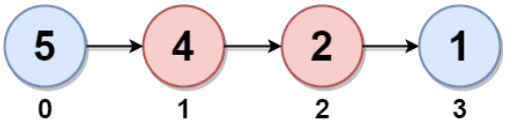
In a linked list of size `n`, where `n` is **even**, the i^{th} node (**0-indexed**) of the linked list is known as the **twin** of the $(n-1-i)^{th}$ node, if `0 <= i <= (n / 2) - 1`.

- For example, if `n = 4`, then node `0` is the twin of node `3`, and node `1` is the twin of node `2`. These are the only nodes with twins for `n = 4`.

The **twin sum** is defined as the sum of a node and its twin.

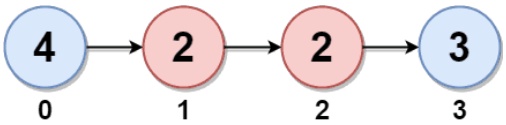
Given the `head` of a linked list with even length, return *the **maximum twin sum** of the linked list*.

Example 1:



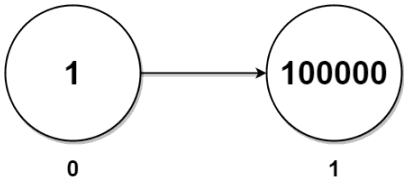

```
Input: head = [5,4,2,1]
Output: 6
Explanation:
Nodes 0 and 1 are the twins of nodes 3 and 2, respectively. All have twin sum = 6.
There are no other nodes with twins in the linked list.
Thus, the maximum twin sum of the linked list is 6.
```

Example 2:




```
Input: head = [4,2,2,3]
Output: 7
Explanation:
The nodes with twins present in this linked list are:
- Node 0 is the twin of node 3 having a twin sum of 4 + 3 = 7.
- Node 1 is the twin of node 2 having a twin sum of 2 + 2 = 4.
Thus, the maximum twin sum of the linked list is max(7, 4) = 7.
```

Example 3:




```
Input: head = [1,100000]
Output: 100001
Explanation:
There is only one node with a twin in the linked list having twin sum of 1 + 100000 = 100001.
```

Constraints:

- The number of nodes in the list is an **even** integer in the range `[2, 10^5]`.

- `1 <= Node.val <= 10^5`

SOLUTIONS (PYTHON 3)

Solution 1: Simulation

> **How to think about it**
>
> Twin nodes are values at symmetric indices; we need their maximum sum. A list does not support random access, and walking from the head for every pair would be quadratic. Copying values into an array makes index i pair with n-1-i. n is even and at most 10^5, so linear extra memory is acceptable. Store the values, then take the maximum of s[i]+s[-(i+1)] over the first half.

We can store the values of the nodes in the linked list into an array, then use two pointers pointing to the beginning and end of the array to calculate the twin sum for each pair of twin nodes. The maximum twin sum is the answer.

The time complexity is O(n) and the space complexity is O(n), where n is the number of nodes in the linked list.

Python 3

```python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def pairSum(self, head: Optional[ListNode]) -> int:
        s = []
        while head:
            s.append(head.val)
            head = head.next
        n = len(s)
        return max(s[i] + s[-(i + 1)] for i in range(n >> 1))
```

Solution 2

> **How to think about it**
>
> Solution 1 uses an O(n) array. Constant extra memory requires pairing on the list itself. A fast/slow split finds the midpoint; reverse the second half and walk it against the first half, taking pairwise sums. The reversal is in place, so the extra space is O(1).

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def pairSum(self, head: Optional[ListNode]) -> int:
        def reverse(head):
            dummy = ListNode()
            curr = head
            while curr:
                next = curr.next
                curr.next = dummy.next
                dummy.next = curr
                curr = next
            return dummy.next

        slow, fast = head, head.next
        while fast and fast.next:
            slow, fast = slow.next, fast.next.next
        pa = head
        q = slow.next
        slow.next = None
        pb = reverse(q)
        ans = 0
        while pa and pb:
            ans = max(ans, pa.val + pb.val)
            pa = pa.next
            pb = pb.next
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. O(1) extra space: reverse the second half
# 2. Return the twin pair with the biggest sum


# ============================================================
# FOLLOW-UP 1: O(1) extra space: reverse the second half
# ============================================================

# ---- follow-up 1: function ----
# Answer: Find the middle, reverse the second half in
# place, and walk both halves together adding the twins.
# Time: O(n).  Space: O(1).
def max_twin_sum_constant_space(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    previous = None
    while slow:
        following = slow.next
        slow.next = previous
        previous = slow
        slow = following
    best = 0
    first, second = head, previous
    while second:
        total = first.val + second.val
        if total > best:
            best = total
        first = first.next
        second = second.next
```

```
    return best


# ============================================================
# FOLLOW-UP 2: Return the twin pair with the biggest sum
# ============================================================

# ---- follow-up 2: function ----
# Answer: Collect the values in an array and compare each
# value with its twin from the other end.
# Time: O(n).  Space: O(n).
def best_twin_pair(head):
    values = []
    while head:
        values.append(head.val)
        head = head.next
    n = len(values)
    best = None
    for i in range(n // 2):
        total = values[i] + values[n - 1 - i]
        if best is None or total > best[0]:
            best = (total, values[i], values[n - 1 - i])
    return best
```

725. Split Linked List in Parts

Medium · Linked List

leetcode.com/problems/split-linked-list-in-parts

PROBLEM

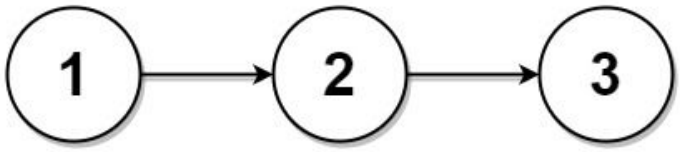
Given the `head` of a singly linked list and an integer `k`, split the linked list into `k` consecutive linked list parts.

The length of each part should be as equal as possible: no two parts should have a size differing by more than one. This may lead to some parts being null.

The parts should be in the order of occurrence in the input list, and parts occurring earlier should always have a size greater than or equal to parts occurring later.

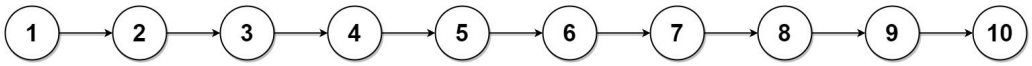
Return *an array of the* `k` *parts*.

Example 1:




```
Input: head = [1,2,3], k = 5
Output: [[1],[2],[3],[],[]]
Explanation:
The first element output[0] has output[0].val = 1, output[0].next = null.
The last element output[4] is null, but its string representation as a ListNode is [].
```

Example 2:




```
Input: head = [1,2,3,4,5,6,7,8,9,10], k = 3
Output: [[1,2,3,4],[5,6,7],[8,9,10]]
Explanation:
The input has been split into consecutive parts with size difference at most 1, and earlier parts are a
larger size than the later parts.
```

Constraints:

- The number of nodes in the list is in the range `[0, 1000]`.
- `0 <= Node.val <= 1000`
- `1 <= k <= 50`

SOLUTION (PYTHON 3)

Solution 1: Simulation

How to think about it

Split a list into k parts as evenly as possible, giving leftover nodes to the earlier parts. n≤ 1000, so count then cut. Each part gets floor(n/k) nodes, and the first n mod k parts get one extra. Unused slots stay null. A second walk advances the computed length, severs next, and records the next head. Two passes in all.

First, we traverse the linked list to obtain its length n, and then we calculate the average length cnt = floor(n/k) and the remainder mod = n mod k. For the first mod parts, each part has a length of cnt + 1, while the lengths of the remaining parts are cnt.

Next, we just need to traverse the linked list and split it into k parts.

The time complexity is O(n), and the space complexity is O(k). Here, n is the length of the linked list.

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def splitListToParts(
        self, head: Optional[ListNode], k: int
    ) -> List[Optional[ListNode]]:
        n = 0
        cur = head
        while cur:
            n += 1
            cur = cur.next
        cnt, mod = divmod(n, k)
        ans = [None] * k
        cur = head
        for i in range(k):
            if cur is None:
                break
            ans[i] = cur
            m = cnt + int(i < mod)
            for _ in range(1, m):
                cur = cur.next
            nxt = cur.next
            cur.next = None
            cur = nxt
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Split into parts of GIVEN sizes
# 2. Deal the nodes into k lists like cards (node 1 to
# list 1, node 2 to list 2, ...)


# ============================================================
# FOLLOW-UP 1: Split into parts of GIVEN sizes
# ============================================================

# ---- follow-up 1: function ----
# Answer: Walk the list, cutting after the requested
# number of nodes for each part.
# Time: O(n).  Space: O(k), k = parts.
def split_by_sizes(head, sizes):
```

```
    parts = []
    node = head
    for size in sizes:
        part_head = node
        previous = None
        for i in range(size):
            if node is None:
                break
            previous = node
            node = node.next
        if previous:
            previous.next = None
        parts.append(part_head if size > 0 else None)
    return parts


# ============================================================
# FOLLOW-UP 2: Deal the nodes into k lists like cards
# (node 1 to list 1, node 2 to list 2, ...)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep k tails and append every node to the next
# list in turn.
# Time: O(n).  Space: O(k).
def deal_into_lists(head, k):
    heads = [None] * k
    tails = [None] * k
    index = 0
    node = head
    while node:
        following = node.next
        node.next = None
        if heads[index] is None:
            heads[index] = node
        else:
            tails[index].next = node
        tails[index] = node
        index = (index + 1) % k
        node = following
    return heads
```

2095. Delete the Middle Node of a Linked List

Medium · Linked List, Two Pointers

leetcode.com/problems/delete-the-middle-node-of-a-linked-list

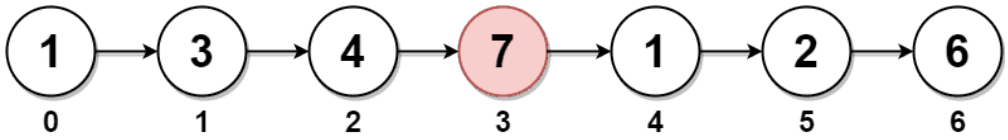
PROBLEM

You are given the `head` of a linked list. **Delete** the **middle node**, and return *the* `head` *of the modified linked list.*

The **middle node** of a linked list of size `n` is the $\lfloor n / 2 \rfloor^{th}$ node from the **start** using **0-based indexing**, where $\lfloor x \rfloor$ denotes the largest integer less than or equal to `x`.

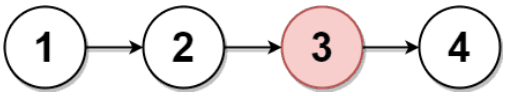
- For `n` = `1`, `2`, `3`, `4`, and `5`, the middle nodes are `0`, `1`, `1`, `2`, and `2`, respectively.

Example 1:



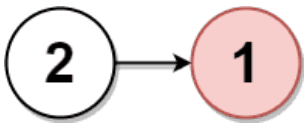

```
Input: head = [1,3,4,7,1,2,6]
Output: [1,3,4,1,2,6]
Explanation:
The above figure represents the given linked list. The indices of the nodes are written below.
Since n = 7, node 3 with value 7 is the middle node, which is marked in red.
We return the new list after removing this node.
```

Example 2:




```
Input: head = [1,2,3,4]
Output: [1,2,4]
Explanation:
The above figure represents the given linked list.
For n = 4, node 2 with value 3 is the middle node, which is marked in red.
```

Example 3:




```
Input: head = [2,1]
Output: [2]
Explanation:
The above figure represents the given linked list.
For n = 2, node 1 with value 1 is the middle node, which is marked in red.
Node 0 with value 2 is the only node remaining after removing node 1.
```

Constraints:

- The number of nodes in the list is in the range $[1, 10^5]$.
- $1 <= Node.val <= 10^5$

SOLUTION (PYTHON 3)

Solution 1: Fast and Slow Pointers

> **How to think about it**
>
> Delete the middle node (the later one if even length), which needs its predecessor. Slow starts at a dummy, fast at the head; two steps versus one leaves slow just before the middle. Rewire `slow.next`. Length 1 is handled because the dummy sits before the only node.

The fast and slow pointer technique is a common method used to solve problems related to linked lists. We can maintain two pointers, a slow pointer slow and a fast pointer fast. Initially, slow points to a dummy node, whose next pointer points to the head node head of the list, while fast points to the head node head.

Then, we move the slow pointer one position backward and the fast pointer two positions backward each time, until the fast pointer reaches the end of the list. At this point, the node next to the node pointed by the slow pointer is the middle node of the list. We can remove the middle node by setting the next pointer of the node pointed by the slow pointer to point to the next next node.

The time complexity is O(n), where n is the length of the list. The space complexity is O(1).

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def deleteMiddle(self, head: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        slow, fast = dummy, head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        slow.next = slow.next.next
        return dummy.next
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. For an even length delete BOTH middle nodes
# 2. Delete the node that is k places after the middle


# ============================================================
# FOLLOW-UP 1: For an even length delete BOTH middle nodes
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 1: function ----
# Answer: Count the nodes; the middle is index n // 2 for
# an odd length, and the two indexes n // 2 - 1 and n // 2
# for an even length. Walk to the node before them and
# unlink.
# Time: O(n).  Space: O(1).
def delete_middle_nodes(head):
    n = 0
    node = head
    while node:
```

```
        n += 1
        node = node.next
    if n <= 2:
        return None
    first = n // 2 - 1 if n % 2 == 0 else n // 2        # index of the first node to delete
    count = 2 if n % 2 == 0 else 1
    dummy = ListNode(0, head)
    previous = dummy
    for i in range(first):
        previous = previous.next
    for i in range(count):
        previous.next = previous.next.next
    return dummy.next


# ============================================================
# FOLLOW-UP 2: Delete the node that is k places after the
# middle
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the nodes, work out the position n // 2 +
# k of the target, walk to the node before it and unlink
# the target (nothing happens when the position is past
# the end).
# Time: O(n).  Space: O(1).
def delete_after_middle(head, k):
    n = 0
    node = head
    while node:
        n += 1
        node = node.next
    target = n // 2 + k
    if target >= n:
        return head
```

```
    if target == 0:
        return head.next
    previous = head
    for i in range(target - 1):
        previous = previous.next
    previous.next = previous.next.next
    return head
```

61. Rotate List

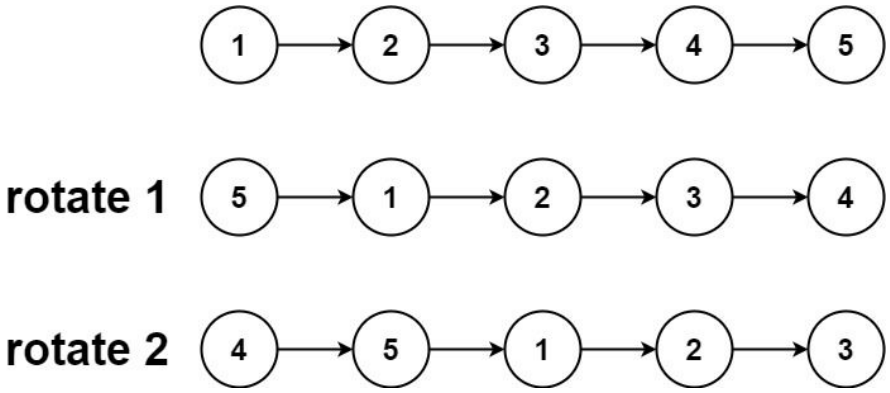
Medium · Linked List, Two Pointers

leetcode.com/problems/rotate-list

PROBLEM

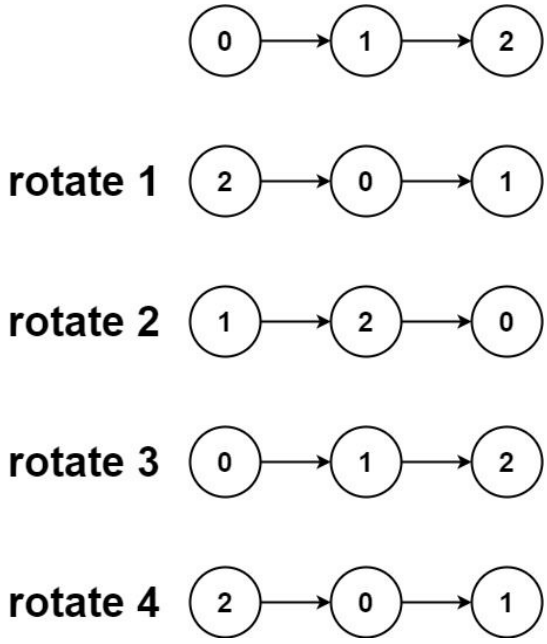
Given the `head` of a linked list, rotate the list to the right by `k` places.

Example 1:




```
Input: head = [1,2,3,4,5], k = 2
Output: [4,5,1,2,3]
```

Example 2:




```
Input: head = [0,1,2], k = 4
Output: [2,0,1]
```

Constraints:

- The number of nodes in the list is in the range `[0, 500]`.
- `-100 <= Node.val <= 100`
- `0 <= k <= 2 * 10^9`

SOLUTION (PYTHON 3)

Solution 1: Fast and Slow Pointers + Link List Concatenation

How to think about it

The first idea is to dump the list into an array, rotate, and rebuild. n ≤ 500 is fine, but k can be 2×10^9, so we cannot rotate one step at a time. The bottleneck is both the huge k and locating the new head without random access. Right-rotating k times equals rotating k mod n times; the new head is the k-th node from the tail. A gap of k between fast and slow pointers puts slow just before the new head when fast reaches the tail, so we never recount n-k. We only rewire a few pointers, in O(1) extra space.

First, we check whether the number of nodes in the linked list is less than 2. If so, we directly return head.

Otherwise, we first count the number of nodes n in the linked list, and then take the modulus of k by n to get the effective value of k.

If the effective value of k is 0, it means that the linked list does not need to be rotated, and we can directly return head.

Otherwise, we use fast and slow pointers, let the fast pointer move k steps first, and then let the fast and slow pointers move together until the fast pointer moves to the end of the linked list. At this time, the next node of the slow pointer is the new head node of the linked list.

Finally, we concatenate the linked list.

The time complexity is O(n), where n is the number of nodes in the linked list. The space complexity is O(1).

Python 3

```python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def rotateRight(self, head: Optional[ListNode], k: int) -> Optional[ListNode]:
        if head is None or head.next is None:
            return head
        cur, n = head, 0
        while cur:
            n += 1
            cur = cur.next
        k %= n
        if k == 0:
            return head
        fast = slow = head
        for _ in range(k):
            fast = fast.next
        while fast.next:
            fast, slow = fast.next, slow.next

        ans = slow.next
        slow.next = None
        fast.next = head
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Rotate to the LEFT by k
# 2. Rotate by k when k is gigantic (more than the list
# length many times over)


# ============================================================
# FOLLOW-UP 1: Rotate to the LEFT by k
# ============================================================

# ---- follow-up 1: function ----
# Answer: Rotating left by k is rotating right by length -
# k: find the new tail at position k - 1, and join the old
# tail to the old head.
# Time: O(n).  Space: O(1).
def rotate_left(head, k):
    if head is None or head.next is None:
        return head
    length = 1
    tail = head
    while tail.next:
        tail = tail.next
        length += 1
    k %= length
    if k == 0:
        return head
    new_tail = head
    for i in range(k - 1):
        new_tail = new_tail.next
    new_head = new_tail.next
    new_tail.next = None
    tail.next = head
    return new_head


# ============================================================
# FOLLOW-UP 2: Rotate by k when k is gigantic (more than
# the list length many times over)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Only k modulo the length matters; compute the
# length first.
# Time: O(n).  Space: O(1).
def rotate_right_big_k(head, k):
    if head is None or head.next is None:
        return head
    length = 1
    tail = head
    while tail.next:
        tail = tail.next
        length += 1
    k %= length
    if k == 0:
        return head
    new_tail = head
    for i in range(length - k - 1):
        new_tail = new_tail.next
    new_head = new_tail.next
    new_tail.next = None
```

```
    tail.next = head
    return new_head
```

109. Convert Sorted List to Binary Search Tree

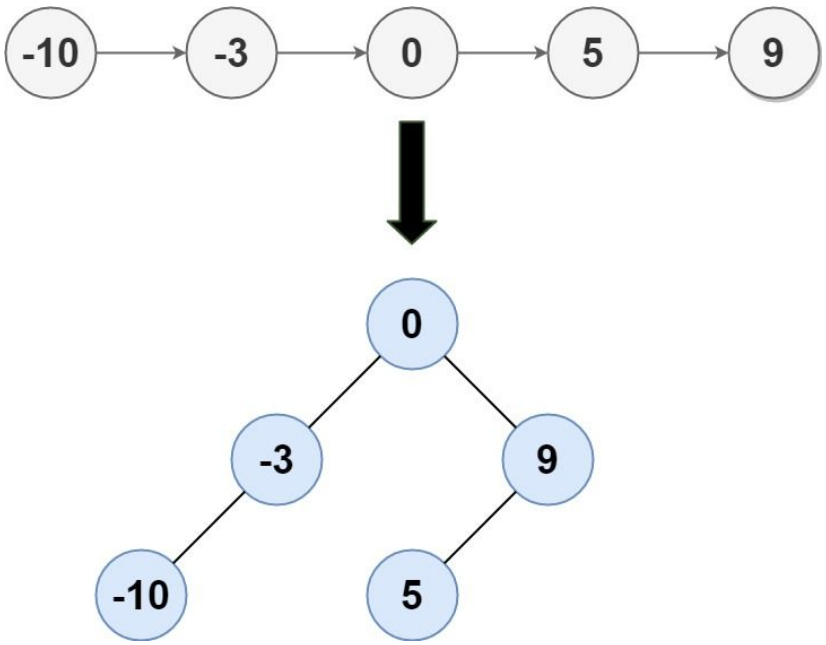
Medium · Tree, Binary Search Tree, Linked List, Divide and Conquer, Binary Tree

leetcode.com/problems/convert-sorted-list-to-binary-search-tree

PROBLEM

Given the `head` of a singly linked list where elements are sorted in **ascending order**, convert *it to a **height-balanced** binary search tree*.

Example 1:




```
Input: head = [-10,-3,0,5,9]
Output: [0,-3,9,-10,null,5]
Explanation: One possible answer is [0,-3,9,-10,null,5], which represents the shown height balanced BST.
```

Example 2:

```
Input: head = []
Output: []
```

Constraints:

- The number of nodes in `head` is in the range $[0, 2 * 10^4]$.
- -10^5 <= Node.val <= 10^5

SOLUTION (PYTHON 3)

Solution 1: DFS

How to think about it

A list cannot access the midpoint in O(1). Finding it with two pointers at every recursive step costs O(nlog n). There are up to 2× 10^4 nodes. A sorted list is still an inorder sequence. Flatten it into an array, then build a balanced BST as with a sorted array: midpoint as root, recurse on both sides. O(n) extra space buys O(1) midpoint access.

We first convert the linked list to an array nums, and then use depth-first search to construct the binary search tree.

We define a function dfs(i, j), where i and j represent the current interval [i, j]. Each time, we choose the number at the middle position mid of the interval as the root node, recursively construct the left subtree for the interval [i, mid - 1], and the right subtree for the interval [mid + 1, j]. Finally, we return the node corresponding to mid as the root node of the current subtree.

In the main function, we just need to call dfs(0, n - 1) and return the result.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the linked list.

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sortedListToBST(self, head: Optional[ListNode]) -> Optional[TreeNode]:
        def dfs(i: int, j: int) -> Optional[TreeNode]:
            if i > j:
                return None
            mid = (i + j) >> 1
            l, r = dfs(i, mid - 1), dfs(mid + 1, j)
            return TreeNode(nums[mid], l, r)

        nums = []
        while head:
            nums.append(head.val)
            head = head.next
        return dfs(0, len(nums) - 1)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Find the middle with slow and fast pointers each time
# (no array, O(n log n))


# ============================================================
# FOLLOW-UP 1: Find the middle with slow and fast pointers
# each time (no array, O(n log n))
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: The middle of the list is the root; recurse on
# the part before it and the part after it. Each level of
# recursion costs O(n) to find middles.
# Time: O(n log n).  Space: O(log n) recursion.
```

```
def sorted_list_to_bst_middle(head):
    if head is None:
        return None
    if head.next is None:
        return TreeNode(head.val)
    previous = None
    slow = fast = head
    while fast and fast.next:
        previous = slow
        slow = slow.next
        fast = fast.next.next
    previous.next = None
    root = TreeNode(slow.val)
    root.left = sorted_list_to_bst_middle(head)
    root.right = sorted_list_to_bst_middle(slow.next)
    return root
```

817. Linked List Components

Medium · Array, Hash Table, Linked List

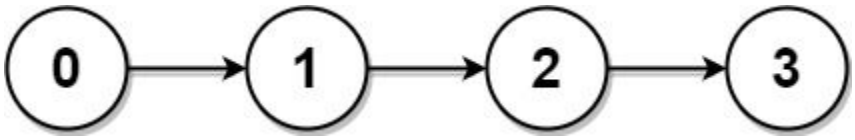
leetcode.com/problems/linked-list-components

PROBLEM

You are given the `head` of a linked list containing unique integer values and an integer array `nums` that is a subset of the linked list values.

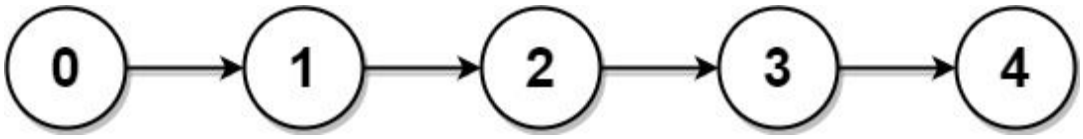
Return the number of **connected components** in `nums`. A connected component is a non-empty, maximal sequence of **consecutive** nodes in the linked list such that every node's value belongs to nums.

Example 1:




```
Input: head = [0,1,2,3], nums = [0,1,3]
Output: 2
Explanation: 0 and 1 are connected, so [0, 1] and [3] are the two connected components.
```

Example 2:




```
Input: head = [0,1,2,3,4], nums = [0,3,1,4]
Output: 2
Explanation: 0 and 1 are connected, 3 and 4 are connected, so [0, 1] and [3, 4] are the two connected
components.
```

Constraints:

- The number of nodes in the linked list is `n`.
- `1 <= n <= 10^4`
- `0 <= Node.val < n`
- All the values `Node.val` are **unique**.
- `1 <= nums.length <= n`
- `0 <= nums[i] < n`
- All the values of `nums` are **unique**.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

A component is a maximal run of nums values on the list. n≤ 10^4, so a set lookup plus one walk is enough; no graph is required. Skip nodes outside the set. Each time a run inside the set begins, count one component and advance past the whole run.

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def numComponents(self, head: Optional[ListNode], nums: List[int]) -> int:
        ans = 0
        s = set(nums)
        while head:
            while head and head.val not in s:
                head = head.next
            ans += head is not None
            while head and head.val in s:
                head = head.next
        return ans
```

Complexity. Time: O(n + m), m = len(nums). Space: O(m).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the components themselves (as lists of values)
# 2. The length of the longest component


# ============================================================
# FOLLOW-UP 1: Return the components themselves (as lists
# of values)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Walk the list; a run of consecutive nodes whose
# values are in the set is one component.
# Time: O(n + m).  Space: O(m).
def components_as_lists(head, nums):
    wanted = set(nums)
    components = []
    current = []
    while head:
        if head.val in wanted:
            current.append(head.val)
        elif current:
            components.append(current)
            current = []
        head = head.next
    if current:
        components.append(current)
    return components


# ============================================================
# FOLLOW-UP 2: The length of the longest component
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the run length while walking and keep the
# biggest.
# Time: O(n + m).  Space: O(m).
def longest_component(head, nums):
    wanted = set(nums)
    best = 0
    run = 0
    while head:
        if head.val in wanted:
            run += 1
```

```
            if run > best:
                best = run
        else:
            run = 0
        head = head.next
    return best
```

328. Odd Even Linked List

Medium · Linked List

leetcode.com/problems/odd-even-linked-list

★ **TOP 7 PROBLEM** in Fast & Slow Pointers (Linked List): master this one first

Also appears in pattern 25 Linked List Reversal / Manipulation

PROBLEM

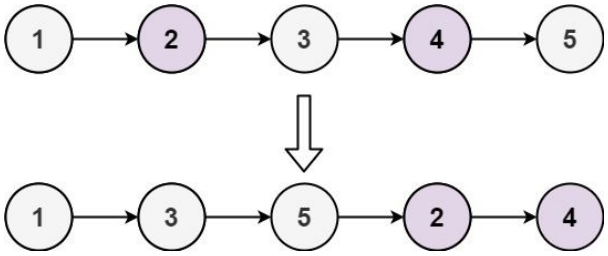
Given the `head` of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return *the reordered list*.

The **first** node is considered **odd**, and the **second** node is **even**, and so on.

Note that the relative order inside both the even and odd groups should remain as it was in the input.

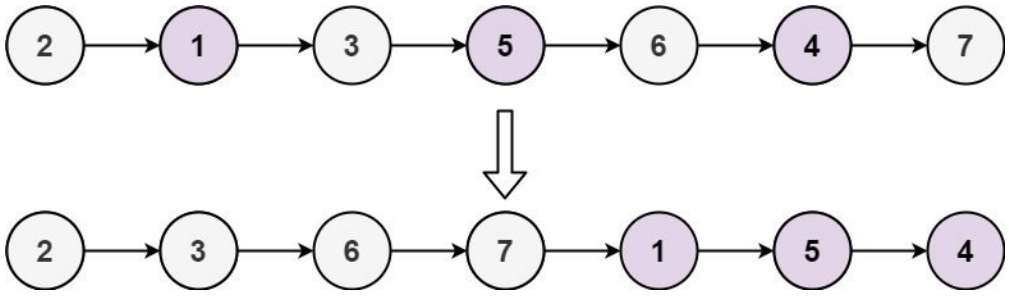
You must solve the problem in `O(1)` extra space complexity and `O(n)` time complexity.

Example 1:




```
Input: head = [1,2,3,4,5]
Output: [1,3,5,2,4]
```

Example 2:




```
Input: head = [2,1,3,5,6,4,7]
Output: [2,3,6,7,1,5,4]
```

Constraints:

- The number of nodes in the linked list is in the range `[0, 10^4]`.
- `-10^6 <= Node.val <= 10^6`

SOLUTION (PYTHON 3)

Solution 1: Single Pass

How to think about it

Keep odd-positioned nodes before even-positioned ones, preserving order. An extra list is easy; O(1) space is required. Let a and b trail the odd and even chains, and c remember the even head. Repeatedly splice b.next after a, then the new successor after b, until the even chain ends. Set a.next=c.

We can use two pointers a and b to represent the tail nodes of the odd and even nodes respectively. Initially, pointer a points to the head node head of the list, and pointer b points to the second node head.next of the list. In addition, we use a pointer c to point to the head node head.next of the even nodes, which is the initial position of pointer b.

We traverse the list, set pointer a to point to the next node of b, i.e., a.next = b.next, then move pointer a back by one position, i.e., a = a.next; set pointer b to point to the next node of a, i.e., b.next = a.next, then move pointer b back by one position, i.e., b = b.next. Continue to traverse until b reaches the end of the list.

Finally, we set the tail node a of the odd nodes to point to the head node c of the even nodes, i.e., a.next = c, then return the head node head of the list.

The time complexity is O(n), where n is the length of the list, and we need to traverse the list once. The space complexity is O(1). We only need to maintain a limited number of pointers.

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def oddEvenList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if head is None:
            return None
        a = head
        b = c = head.next
        while b and b.next:
            a.next = b.next
            a = a.next
            b.next = a.next
            b = b.next
        a.next = c
        return head
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Group the nodes by the PARITY OF THEIR VALUE (odd
# values first), keeping the order
# 2. Group the nodes by position modulo k (positions 0, k,
# 2k... first, then 1, k + 1, ...)


# ============================================================
# FOLLOW-UP 1: Group the nodes by the PARITY OF THEIR
# VALUE (odd values first), keeping the order
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

```
# ---- follow-up 1: function ----
# Answer: Two chains, one for odd and one for even values,
# joined at the end.
# Time: O(n).  Space: O(1).
def group_by_value_parity(head):
    odd_dummy = ListNode()
    even_dummy = ListNode()
    odd_tail, even_tail = odd_dummy, even_dummy
    while head:
        following = head.next
        head.next = None
        if head.val % 2 == 1:
            odd_tail.next = head
            odd_tail = head
        else:
            even_tail.next = head
            even_tail = head
        head = following
    odd_tail.next = even_dummy.next
    return odd_dummy.next


# ========================================================
# FOLLOW-UP 2: Group the nodes by position modulo k
# (positions 0, k, 2k... first, then 1, k + 1, ...)
# ========================================================

# ---- follow-up 2: function ----
# Answer: k chains, the node at position i goes to chain i
# mod k; join the chains in order.
# Time: O(n).  Space: O(k).
def group_by_position_mod_k(head, k):
    heads = [None] * k
    tails = [None] * k
    index = 0
    while head:
        following = head.next
        head.next = None
        chain = index % k
        if heads[chain] is None:
            heads[chain] = head
        else:
            tails[chain].next = head
        tails[chain] = head
        index += 1
        head = following
    first = None
    last = None
    for chain in range(k):
        if heads[chain] is None:
            continue
        if first is None:
            first = heads[chain]
        else:
            last.next = heads[chain]
        last = tails[chain]
    return first
```

86. Partition List

Medium · Linked List, Two Pointers

leetcode.com/problems/partition-list

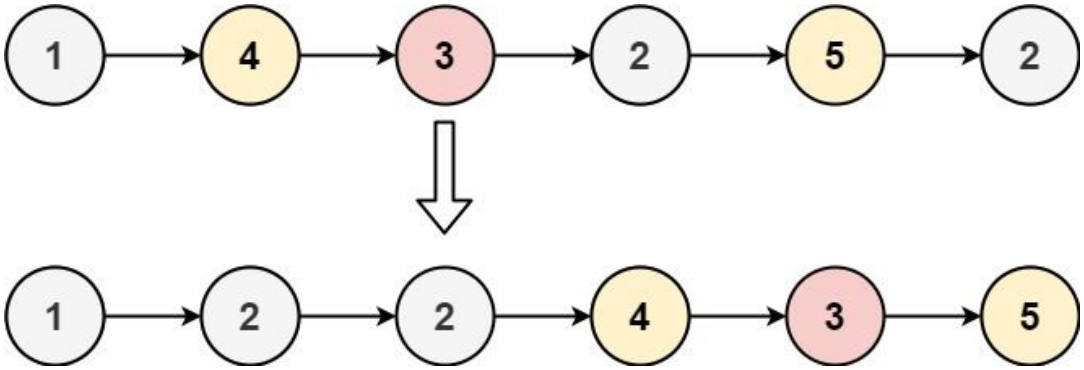
★ **TOP 7 PROBLEM** in Fast & Slow Pointers (Linked List): master this one first

PROBLEM

Given the `head` of a linked list and a value `x`, partition it such that all nodes **less than** `x` come before nodes **greater than or equal** to `x`.

You should **preserve** the original relative order of the nodes in each of the two partitions.

Example 1:




```
Input: head = [1,4,3,2,5,2], x = 3
Output: [1,2,2,4,3,5]
```

Example 2:

```
Input: head = [2,1], x = 2
Output: [1,2]
```

Constraints:

- The number of nodes in the list is in the range `[0, 200]`.
- `-100 <= Node.val <= 100`
- `-200 <= x <= 200`

SOLUTION (PYTHON 3)

Solution 1: Simulation

How to think about it

The first idea is two scans: collect nodes < x, then the rest, and rebuild. Correct, and n ≤ 200 is tiny, but that copies nodes or walks twice, and it is easy to scramble relative order. The bottleneck is splitting and reassembling instead of one pass. The problem only asks for a stable partition: order among the small nodes and among the large nodes both stay the same. So grow two lists in one walk and stitch the small tail to the large head. Tail pointers link original nodes — no copies; clear the large tail to avoid a cycle.

We create two linked lists l and r, one to store nodes less than x and the other to store nodes greater than or equal to x. Then we concatenate them.

The time complexity is O(n), where n is the length of the original linked list. The space complexity is O(1).

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def partition(self, head: Optional[ListNode], x: int) -> Optional[ListNode]:
        l = ListNode()
        r = ListNode()
        tl, tr = l, r
        while head:
            if head.val < x:
                tl.next = head
                tl = tl.next
            else:
                tr.next = head
                tr = tr.next
            head = head.next
        tr.next = None
        tl.next = r.next
        return l.next
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Three-way partition: smaller than x, EQUAL to x,
# bigger than x


# ============================================================
# FOLLOW-UP 1: Three-way partition: smaller than x, EQUAL
# to x, bigger than x
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 1: function ----
# Answer: Three chains with dummy heads, joined in order.
# Time: O(n).  Space: O(1).
def partition_three_way(head, x):
    small, equal, big = ListNode(), ListNode(), ListNode()
    small_tail, equal_tail, big_tail = small, equal, big
    while head:
        following = head.next
        head.next = None
        if head.val < x:
            small_tail.next = head
            small_tail = head
        elif head.val == x:
            equal_tail.next = head
            equal_tail = head
        else:
            big_tail.next = head
            big_tail = head
        head = following
    equal_head = equal.next
    big_head = big.next
    equal_tail.next = big_head                        # the chain of equal nodes ends in the bigger ones
    if equal_head is None:
        small_tail.next = big_head
    else:
```

```
        small_tail.next = equal_head
    return small.next
```

148. Sort List

Medium · Linked List, Two Pointers, Divide and Conquer, Sorting, Merge Sort

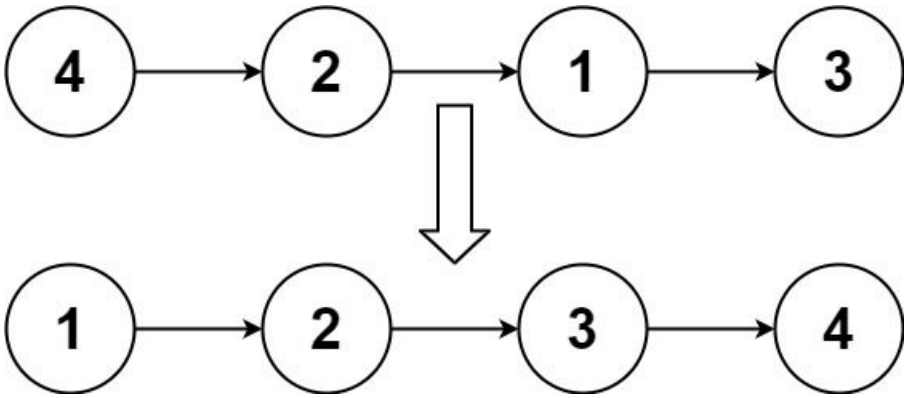
leetcode.com/problems/sort-list

★ **TOP 7 PROBLEM** in Fast & Slow Pointers (Linked List): master this one first

PROBLEM

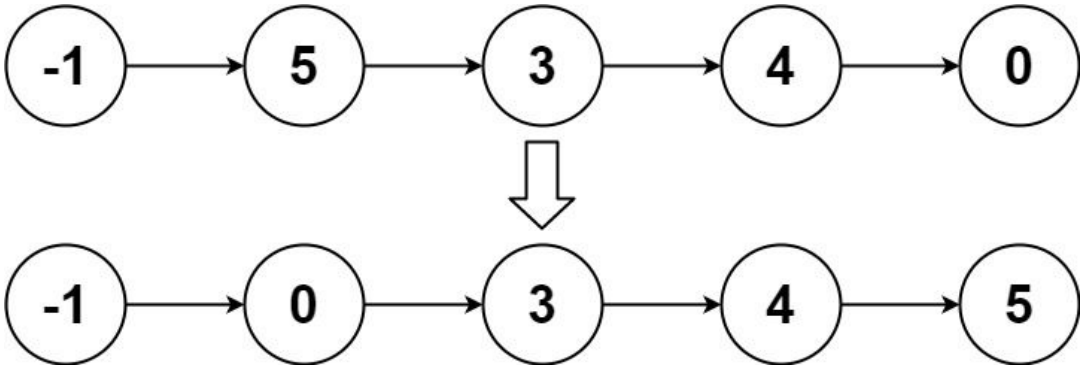
Given the `head` of a linked list, return *the list after sorting it in* ***ascending order***.

Example 1:




```
Input: head = [4,2,1,3]
Output: [1,2,3,4]
```

Example 2:




```
Input: head = [-1,5,3,4,0]
Output: [-1,0,3,4,5]
```

Example 3:

```
Input: head = []
Output: []
```

Constraints:

- The number of nodes in the list is in the range `[0, 5 * 10^4]`.
- `-10^5 <= Node.val <= 10^5`

Follow up: Can you sort the linked list in `O(n logn)` time and `O(1)` memory (i.e. constant space)?

SOLUTION (PYTHON 3)

Solution 1: Merge Sort

How to think about it

Sort a list; the follow-up wants O(nlog n) time and constant extra space. There is no random access, so quicksort swaps are awkward. n≤ 5× 10^4. Merge sort only needs sequential scans: split at the midpoint, sort both halves, merge. The recursion stack is O(log n).

We can use the merge sort approach to solve this problem.

First, we use the fast and slow pointers to find the middle of the linked list and break the list from the middle to form two separate sublists l1 and l2.

Then, we recursively sort l1 and l2, and finally merge l1 and l2 into a sorted linked list.

The time complexity is O(n × log n), and the space complexity is O(log n). Here, n is the length of the linked list.

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def sortList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if head is None or head.next is None:
            return head
        slow, fast = head, head.next
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        l1, l2 = head, slow.next
        slow.next = None
        l1, l2 = self.sortList(l1), self.sortList(l2)
        dummy = ListNode()
        tail = dummy
        while l1 and l2:
            if l1.val <= l2.val:
                tail.next = l1
                l1 = l1.next
            else:
                tail.next = l2
                l2 = l2.next
            tail = tail.next
        tail.next = l1 or l2
        return dummy.next
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Sort in DESCENDING order
# 2. A list of only 0s, 1s and 2s: sort it in one pass


# ============================================================
# FOLLOW-UP 1: Sort in DESCENDING order
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort ascending with merge sort and reverse, or
# merge with the opposite comparison.
# Time: O(n log n).  Space: O(log n) recursion.
class ListNode:
```

```
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


def merge_descending(a, b):
    dummy = ListNode()
    tail = dummy
    while a and b:
        if a.val >= b.val:
            tail.next = a
            a = a.next
        else:
            tail.next = b
            b = b.next
        tail = tail.next
    tail.next = a if a else b
    return dummy.next


def sort_list_descending(head):
    if head is None or head.next is None:
        return head
    slow = fast = head
    previous = None
    while fast and fast.next:
        previous = slow
        slow = slow.next
        fast = fast.next.next
    previous.next = None
    return merge_descending(sort_list_descending(head), sort_list_descending(slow))


# ============================================================
# FOLLOW-UP 2: A list of only 0s, 1s and 2s: sort it in
# one pass
# ============================================================

# ---- follow-up 2: function ----
# Answer: Three chains (for 0, 1 and 2) are joined in
# order, or count the values and rewrite them.
# Time: O(n).  Space: O(1).
def sort_zeros_ones_twos(head):
    counts = [0, 0, 0]
    node = head
    while node:
```

```
        counts[node.val] += 1
        node = node.next
    node = head
    for value in range(3):
        for i in range(counts[value]):
            node.val = value
            node = node.next
    return head
```

PATTERN 25 OF 41

Linked List Reversal / Manipulation

20 problems · 4 Easy · 14 Medium · 2 Hard

THE IDEA

Re-wire next pointers carefully: save the next node before you overwrite it. A dummy head removes the special case at the front.

Signals: reverse a list or part of it, reverse in groups of K, merge two lists, remove the nth node from the end, reorder list, add two numbers.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
prev, cur = None, head
while cur:
    nxt = cur.next        # save it first
    cur.next = prev       # turn the arrow around
    prev, cur = cur, nxt
return prev
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Reverse Linked List	Easy	LC 206	1325
2	Merge Two Sorted Lists	Easy	LC 21	1328
3	Remove Linked List Elements	Easy	LC 203	1332
4	Convert Binary Number in a Linked List to Integer	Easy	LC 1290	1335
5	★ **Reverse Linked List II**	Medium	LC 92	1338
6	★ **Swap Nodes in Pairs**	Medium	LC 24	1341
7	★ **Add Two Numbers**	Medium	LC 2	1344
8	Add Two Numbers II	Medium	LC 445	1347
9	★ **Copy List with Random Pointer**	Medium	LC 138	1351
10	Flatten a Multilevel Doubly Linked List	Medium	LC 430	1356
11	Design Linked List	Medium	LC 707	1360
12	Swapping Nodes in a Linked List	Medium	LC 1721	1365
13	Insertion Sort List	Medium	LC 147	1368
14	Plus One Linked List	Medium	LC 369	1371
15	Design a Stack With Increment Operation	Medium	LC 1381	1374
16	Odd Even Linked List (see page)	Medium	LC 328	1315
17	Merge In Between Linked Lists	Medium	LC 1669	1377
18	★ **LRU Cache**	Medium	LC 146	1380
19	★ **Reverse Nodes in k-Group**	Hard	LC 25	1385
20	★ **Merge k Sorted Lists**	Hard	LC 23	1389

PATTERN 25 · LINKED LIST REVERSAL / MANIPULATION · PROBLEM 1 OF 20

206. Reverse Linked List

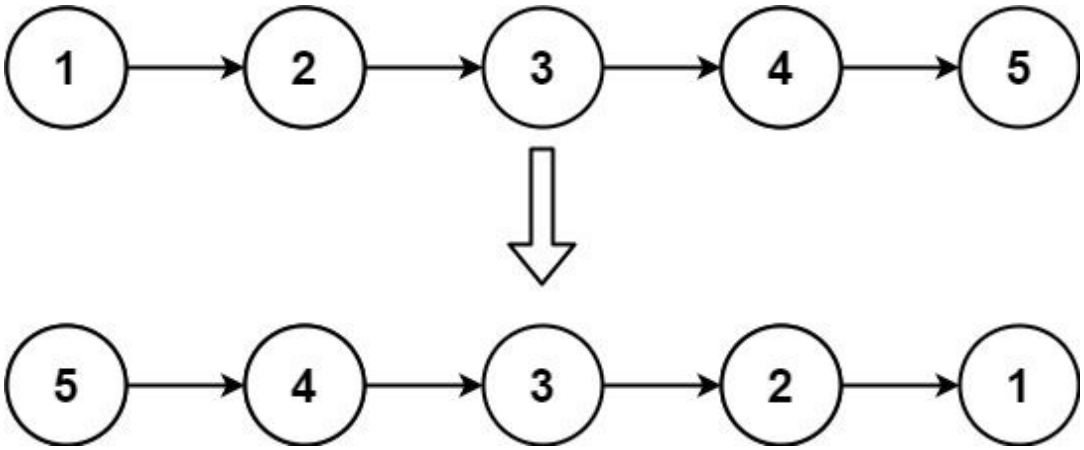
Easy · Recursion, Linked List

leetcode.com/problems/reverse-linked-list

PROBLEM

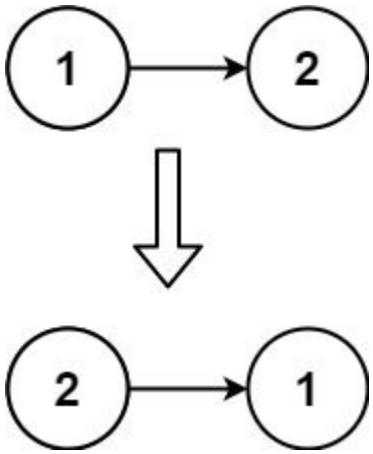
Given the `head` of a singly linked list, reverse the list, and return *the reversed list*.

Example 1:




```
Input: head = [1,2,3,4,5]
Output: [5,4,3,2,1]
```

Example 2:




```
Input: head = [1,2]
Output: [2,1]
```

Example 3:

```
Input: head = []
Output: []
```

Constraints:

- The number of nodes in the list is the range `[0, 5000]`.
- `-5000 <= Node.val <= 5000`

Follow up: A linked list can be reversed either iteratively or recursively. Could you implement both?

SOLUTIONS (PYTHON 3)

Solution 1: Head Insertion Method

> **How to think about it**
>
> Storing nodes in an array and rebuilding backwards works, but only pointer rewrites are needed. Head insertion reverses the list in one pass: each node is inserted immediately after a dummy. When the scan ends, dummy.next is the new head and extra space is constant.

We create a dummy node dummy, then traverse the linked list and insert each node after the dummy node. After traversal, return dummy.next.

The time complexity is O(n), where n is the length of the linked list. The space complexity is O(1).

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseList(self, head: ListNode) -> ListNode:
        dummy = ListNode()
        curr = head
        while curr:
            next = curr.next
            curr.next = dummy.next
            dummy.next = curr
            curr = next
        return dummy.next
```

Solution 2: Recursion

> **How to think about it**
>
> Head insertion is already linear. We can instead reverse the suffix recursively, then attach the current node behind the new tail. The pointer updates are shorter, at the cost of O(n) call-stack space.

We recursively reverse all nodes from the second node to the end of the list, then attach the head to the end of the reversed list.

The time complexity is O(n), and the space complexity is O(n). Where n is the length of the linked list.

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseList(self, head: ListNode) -> ListNode:
        if head is None or head.next is None:
            return head
        ans = self.reverseList(head.next)
        head.next.next = head
        head.next = None
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Reverse recursively
# 2. Reverse only the first k nodes


# ============================================================
# FOLLOW-UP 1: Reverse recursively
# ============================================================

# ---- follow-up 1: function ----
# Answer: Reverse the rest of the list first; then make
# the second node point back to the first node and cut the
# first node's own pointer.
# Time: O(n).  Space: O(n) recursion stack.
def reverse_recursive(head):
    if head is None or head.next is None:
        return head
    new_head = reverse_recursive(head.next)
    head.next.next = head
    head.next = None
    return new_head


# ============================================================
# FOLLOW-UP 2: Reverse only the first k nodes
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same loop, stopped after k nodes; the old
# head is then linked to the rest of the list.
# Time: O(k).  Space: O(1).
def reverse_first_k(head, k):
    previous = None
    node = head
    for i in range(k):
        if node is None:
            break
        following = node.next
        node.next = previous
        previous = node
        node = following
    head.next = node
    return previous
```

21. Merge Two Sorted Lists

Easy · Recursion, Linked List

leetcode.com/problems/merge-two-sorted-lists

Also appears in pattern 38 K-way Merge

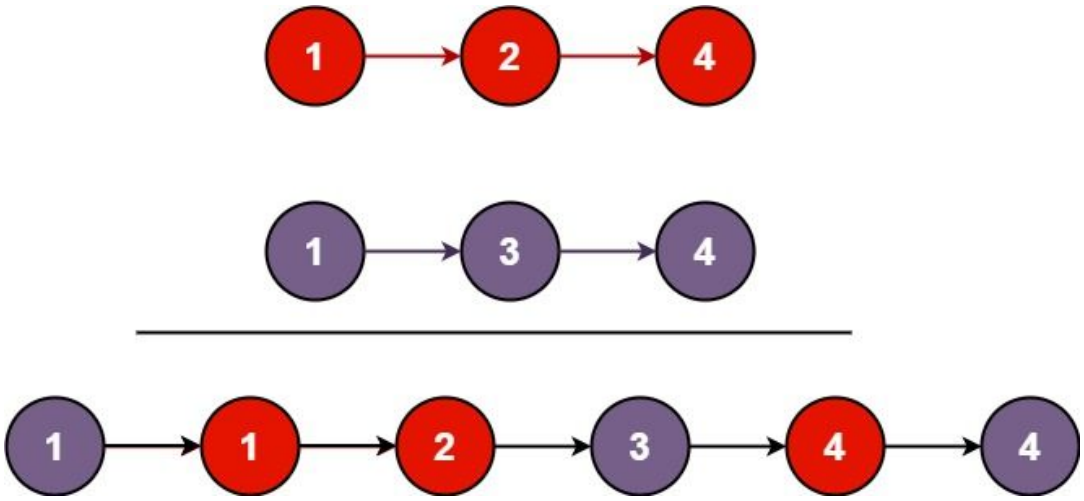
PROBLEM

You are given the heads of two sorted linked lists `list1` and `list2`.

Merge the two lists into one **sorted** list. The list should be made by splicing together the nodes of the first two lists.

Return *the head of the merged linked list*.

Example 1:




```
Input: list1 = [1,2,4], list2 = [1,3,4]
Output: [1,1,2,3,4,4]
```

Example 2:

```
Input: list1 = [], list2 = []
Output: []
```

Example 3:

```
Input: list1 = [], list2 = [0]
Output: [0]
```

Constraints:

- The number of nodes in both lists is in the range `[0, 50]`.
- `-100 <= Node.val <= 100`
- Both `list1` and `list2` are sorted in **non-decreasing** order.

SOLUTIONS (PYTHON 3)

Solution 1: Recursion

How to think about it

The first idea is to dump both lists into an array, sort, and rebuild. Total length is at most 100, so it would pass, but it throws away the fact that both lists are already sorted. The bottleneck is unnecessary reordering: each step only needs the smaller of the two heads. After taking it, what remains is still two sorted lists — the same problem, one node smaller. Compare the heads of l_1 and l_2; keep the smaller one and hang the merge of the rest on its next. When one list is empty, the other is the answer as-is. Recursion depth equals the number of nodes. Lengths here are tiny, so O(m+n) stack space is fine.

First, we judge whether the linked lists l_1 and l_2 are empty. If one of them is empty, we return the other linked list. Otherwise, we compare the head nodes of l_1 and l_2:

- If the value of the head node of l_1 is less than or equal to the value of the head node of l_2, we recursively call the function mergeTwoLists(l_1.next, l_2), and connect the head node of l_1 with the returned linked list head node, and return the head node of l_1.
- Otherwise, we recursively call the function mergeTwoLists(l_1, l_2.next), and connect the head node of l_2 with the returned linked list head node, and return the head node of l_2.

The time complexity is O(m + n), and the space complexity is O(m + n). Here, m and n are the lengths of the two linked lists respectively.

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeTwoLists(
        self, list1: Optional[ListNode], list2: Optional[ListNode]
    ) -> Optional[ListNode]:
        if list1 is None or list2 is None:
            return list1 or list2
        if list1.val <= list2.val:
            list1.next = self.mergeTwoLists(list1.next, list2)
            return list1
        else:
            list2.next = self.mergeTwoLists(list1, list2.next)
            return list2
```

Solution 2: Iteration

How to think about it

Solution 1 is correct, but the recursion stack is linear in the number of nodes. The lists here are short, yet the same choice can be made with constant extra space. The observation is unchanged: each step still compares the two heads and appends the smaller. A dummy dummy plus a tail pointer curr do that in a loop; when one list runs out, splice the other on. Time matches the recursive version; extra space drops to O(1).

We can also use iteration to implement the merging of two sorted linked lists.

First, we define a dummy head node dummy, then loop through the two linked lists, compare the head nodes of the two linked lists, add the smaller node to the end of dummy, until one of the linked lists is empty, then add the remaining part of the other linked list to the end of dummy.

Finally, return dummy.next.

The time complexity is O(m + n), where m and n are the lengths of the two linked lists respectively. Ignoring the space consumption of the answer linked list, the space complexity is O(1).

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeTwoLists(
        self, list1: Optional[ListNode], list2: Optional[ListNode]
    ) -> Optional[ListNode]:
        dummy = ListNode()
        curr = dummy
        while list1 and list2:
            if list1.val <= list2.val:
                curr.next = list1
                list1 = list1.next
            else:
                curr.next = list2
                list2 = list2.next
            curr = curr.next
        curr.next = list1 or list2
        return dummy.next
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Merge two DESCENDING lists into one descending list
# 2. Merge and drop the repeated values


# ============================================================
# FOLLOW-UP 1: Merge two DESCENDING lists into one
# descending list
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 1: function ----
# Answer: The same merge with the comparison reversed.
# Time: O(a + b).  Space: O(1).
def merge_descending(a, b):
    dummy = ListNode()
    tail = dummy
    while a and b:
        if a.val >= b.val:
            tail.next = a
            a = a.next
        else:
            tail.next = b
            b = b.next
        tail = tail.next
    tail.next = a if a else b
    return dummy.next


# ============================================================
# FOLLOW-UP 2: Merge and drop the repeated values
# ============================================================

# ---- follow-up 2: function ----
# Answer: Merge as usual but only append a node when its
# value differs from the last value appended.
```

```
# Time: O(a + b).  Space: O(1).
def merge_without_duplicates(a, b):
    dummy = ListNode()
    tail = dummy
    while a or b:
        if b is None or (a is not None and a.val <= b.val):
            node = a
            a = a.next
        else:
            node = b
            b = b.next
        if tail is dummy or tail.val != node.val:
            tail.next = node
            tail = node
    tail.next = None
    return dummy.next
```

203. Remove Linked List Elements

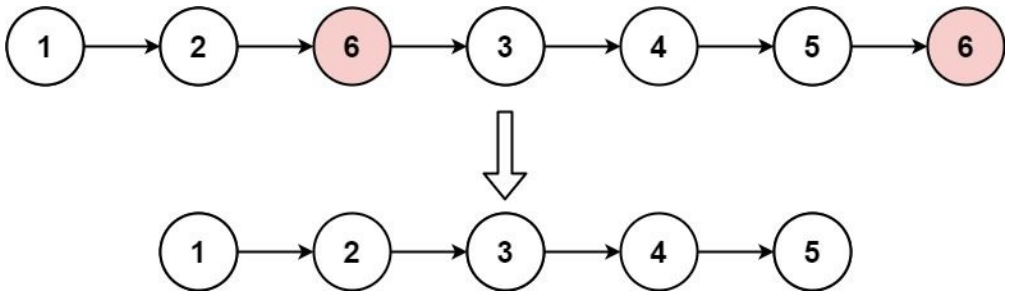
Easy · Recursion, Linked List

leetcode.com/problems/remove-linked-list-elements

PROBLEM

Given the `head` of a linked list and an integer `val`, remove all the nodes of the linked list that has `Node.val == val`, and return *the new head*.

Example 1:




```
Input: head = [1,2,6,3,4,5,6], val = 6
Output: [1,2,3,4,5]
```

Example 2:

```
Input: head = [], val = 1
Output: []
```

Example 3:

```
Input: head = [7,7,7,7], val = 7
Output: []
```

Constraints:

- The number of nodes in the list is in the range `[0, 10^4]`.
- `1 <= Node.val <= 50`
- `0 <= val <= 50`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Removing nodes with value val is a predecessor-pointer update, but the head itself may be deleted, so special-casing head is messy. A dummy node in front lets pre inspect its successor: skip it when the value matches, otherwise advance. Return dummy.next.

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def removeElements(self, head: ListNode, val: int) -> ListNode:
        dummy = ListNode(-1, head)
        pre = dummy
        while pre.next:
            if pre.next.val != val:
                pre = pre.next
            else:
                pre.next = pre.next.next
        return dummy.next
```

Complexity. Time: O(n). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Remove only the FIRST node with the value
# 2. Remove every node whose value is in a set


# ============================================================
# FOLLOW-UP 1: Remove only the FIRST node with the value
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 1: function ----
# Answer: Walk until the first match and unlink it, then
# stop.
# Time: O(n).  Space: O(1).
def remove_first_occurrence(head, val):
    dummy = ListNode(0, head)
    previous = dummy
    while previous.next:
        if previous.next.val == val:
            previous.next = previous.next.next
            break
        previous = previous.next
    return dummy.next


# ============================================================
# FOLLOW-UP 2: Remove every node whose value is in a set
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same pass with a set lookup instead of one
# comparison.
# Time: O(n).  Space: O(1).
def remove_values_in_set(head, unwanted):
    dummy = ListNode(0, head)
    previous = dummy
    while previous.next:
        if previous.next.val in unwanted:
            previous.next = previous.next.next
        else:
            previous = previous.next
```

```
    return dummy.next
```

1290. Convert Binary Number in a Linked List to Integer

Easy · Linked List, Math

leetcode.com/problems/convert-binary-number-in-a-linked-list-to-integer

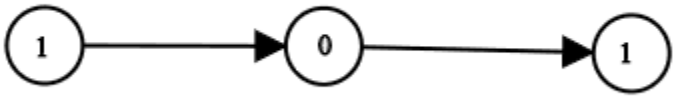
PROBLEM

Given `head` which is a reference node to a singly-linked list. The value of each node in the linked list is either `0` or `1`. The linked list holds the binary representation of a number.

Return the *decimal value* of the number in the linked list.

The **most significant bit** is at the head of the linked list.

Example 1:




```
Input: head = [1,0,1]
Output: 5
Explanation: (101) in base 2 = (5) in base 10
```

Example 2:

```
Input: head = [0]
Output: 0
```

Constraints:

- The Linked List is not empty.
- Number of nodes will not exceed `30`.
- Each node's value is either `0` or `1`.

SOLUTION (PYTHON 3)

Solution 1: Traverse the Linked List

> **How to think about it**
>
> The list is a binary number from high bit to low. Shifting the running value left and ORing the current bit accumulates the integer. Length is at most 30, so it fits. We need not collect bits first.

We use a variable ans to record the current decimal value, with an initial value of 0.

Traverse the linked list. For each node, left-shift ans by one bit, then perform a bitwise OR with the current node's value. After traversal, ans is the decimal value.

The time complexity is O(n), where n is the length of the linked list. The space complexity is O(1).

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def getDecimalValue(self, head: ListNode) -> int:
        ans = 0
        while head:
            ans = ans << 1 | head.val
            head = head.next
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The list holds the LEAST significant bit first
# 2. Add 1 to the binary number held in the list (most
# significant bit first)


# ============================================================
# FOLLOW-UP 1: The list holds the LEAST significant bit
# first
# ============================================================

# ---- follow-up 1: function ----
# Answer: Add each bit at its place value: the first node
# is 1, the next 2, then 4, ...
# Time: O(n).  Space: O(1).
def little_endian_to_int(head):
    value = 0
    place = 1
    while head:
        value += head.val * place
        place *= 2
        head = head.next
    return value


# ============================================================
# FOLLOW-UP 2: Add 1 to the binary number held in the list
# (most significant bit first)
# ============================================================

# ---- follow-up 2: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 2: function ----
# Answer: Find the last node that is 0 (or add a new
# leading node when there is none); set it to 1 and clear
# all the 1s after it.
# Time: O(n).  Space: O(1).
def add_one_binary(head):
    dummy = ListNode(0, head)
    last_zero = dummy
    node = head
    while node:
        if node.val == 0:
            last_zero = node
        node = node.next
    last_zero.val = 1
```

```
    node = last_zero.next
    while node:
        node.val = 0
        node = node.next
    if dummy.val == 1:
        return dummy
    return dummy.next
```

PATTERN 25 · LINKED LIST REVERSAL / MANIPULATION · PROBLEM 5 OF 20

92. Reverse Linked List II

Medium · Linked List

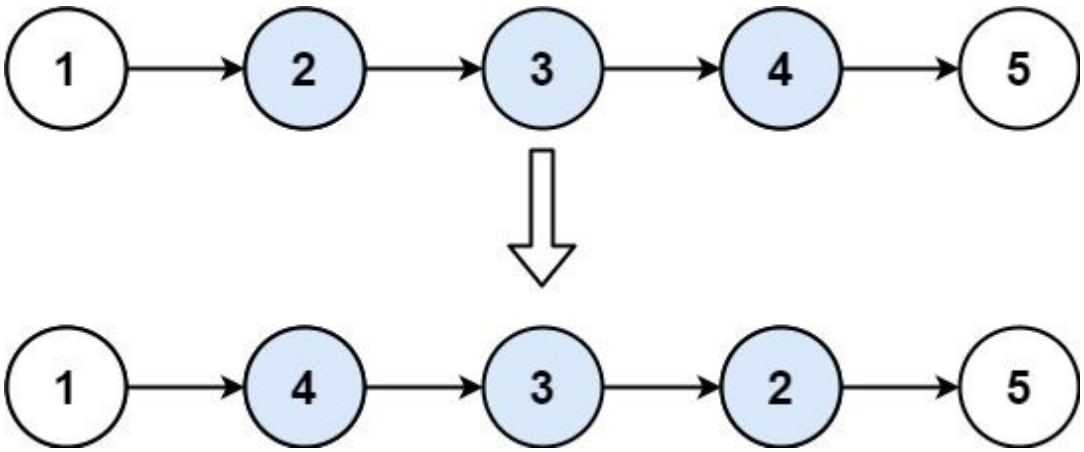
leetcode.com/problems/reverse-linked-list-ii

★ **TOP 7 PROBLEM** in Linked List Reversal / Manipulation: master this one first

PROBLEM

Given the `head` of a singly linked list and two integers `left` and `right` where `left <= right`, reverse the nodes of the list from position `left` to position `right`, and return *the reversed list*.

Example 1:




```
Input: head = [1,2,3,4,5], left = 2, right = 4
Output: [1,4,3,2,5]
```

Example 2:

```
Input: head = [5], left = 1, right = 1
Output: [5]
```

Constraints:

- The number of nodes in the list is `n`.
- `1 <= n <= 500`
- `-500 <= Node.val <= 500`
- `1 <= left <= right <= n`

Follow up:

Could you do it in one pass?

SOLUTION (PYTHON 3)

Solution 1: Simulation

How to think about it

We could cut out [left,right], reverse it, and splice it back, but left=1 replaces the head and we have to remember several cut points. The follow-up asks for one pass. The key: head-insert each node of the range right after the predecessor of left. That reverses in place. A dummy node covers left=1. Walk to left-1, then head-insert right-left+1 times.

Define a dummy head node `dummy`, pointing to the head node `head` of the linked list. Then define a pointer `pre` pointing to `dummy`. Start traversing the linked list from the dummy head node. When you traverse to the `left` node, point `pre` to this node. Then start traversing `right - left + 1` times from this node, and insert the nodes you traverse into the back of `pre`. Finally, return `dummy.next`.

The time complexity is O(n), and the space complexity is O(1). Here, n is the length of the linked list.

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseBetween(
        self, head: Optional[ListNode], left: int, right: int
    ) -> Optional[ListNode]:
        if head.next is None or left == right:
            return head
        dummy = ListNode(0, head)
        pre = dummy
        for _ in range(left - 1):
            pre = pre.next
        p, q = pre, pre.next
        cur = q
        for _ in range(right - left + 1):
            t = cur.next
            cur.next = pre
            pre, cur = cur, t
        p.next = pre
        q.next = cur
        return dummy.next
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Reverse several separate ranges (given as a list of
# non-overlapping pairs)


# ============================================================
# FOLLOW-UP 1: Reverse several separate ranges (given as a
# list of non-overlapping pairs)
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 1: function ----
# Answer: Apply the single-range reversal once for every
# range; ranges do not overlap, so the positions stay
# valid.
# Time: O(r * n) for r ranges (each range walks from the
# head).  Space: O(1).
def reverse_between(head, left, right):
    dummy = ListNode(0, head)
    before = dummy
    for i in range(left - 1):
        before = before.next
    current = before.next
    for i in range(right - left):
        moved = current.next
        current.next = moved.next
```

```
        moved.next = before.next
        before.next = moved
    return dummy.next


def reverse_ranges(head, ranges):
    for left, right in ranges:
        head = reverse_between(head, left, right)
    return head
```

24. Swap Nodes in Pairs

Medium · Recursion, Linked List

leetcode.com/problems/swap-nodes-in-pairs

★ **TOP 7 PROBLEM** in Linked List Reversal / Manipulation: master this one first

PROBLEM

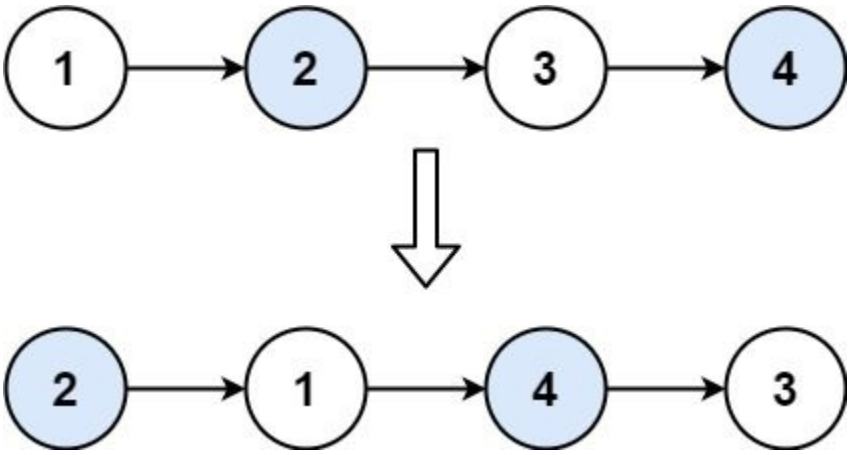
Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

Example 1:

Input: head = [1,2,3,4]

Output: [2,1,4,3]

Explanation:



Example 2:

Input: head = []

Output: []

Example 3:

Input: head = [1]

Output: [1]

Example 4:

Input: head = [1,2,3]

Output: [2,1,3]

Constraints:

- The number of nodes in the list is in the range `[0, 100]`.
- `0 <= Node.val <= 100`

SOLUTIONS (PYTHON 3)

Solution 1: Recursion

How to think about it

The first idea is to swap values of adjacent nodes. The problem forbids changing values, so we must rewire pointers. At most 100 nodes, so any correct linking will pass. The bottleneck is the link order: flip the first pair and then the rest, and it is easy to lose a successor. After one pair is swapped, the suffix starting at the third node is the same problem. The new head is always the original second node. So recurse on head.next.next first, then reverse the current pair onto that result. An empty list or a single node cannot form a pair and is returned as-is.

We can implement swapping two nodes in the linked list through recursion.

The termination condition of recursion is that there are no nodes in the linked list, or there is only one node in the linked list. At this time, swapping cannot be performed, so we directly return this node.

Otherwise, we recursively swap the linked list head.next.next, and let the swapped head node be t. Then we let p be the next node of head, and let p point to head, and head point to t, finally return p.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the linked list.

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def swapPairs(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if head is None or head.next is None:
            return head
        t = self.swapPairs(head.next.next)
        p = head.next
        p.next = head
        head.next = t
        return p
```

Solution 2: Iteration

How to think about it

Solution 1 is correct, but recursion depth tracks the number of pairs, so extra space is O(n). The same three pointer writes inside a pair can run in a loop. A dummy pins the list head. Pointers pre and cur rewrite the two nodes after pre, then advance by one pair. Time stays O(n); extra space drops to O(1).

We set a dummy head node dummy, initially pointing to head, and then set two pointers pre and cur, initially pre points to dummy, and cur points to head.

Next, we traverse the linked list. Each time we need to swap the two nodes after pre, so we first judge whether cur and cur.next are empty. If they are not empty, we perform the swap, otherwise we terminate the loop.

The time complexity is O(n), and the space complexity is O(1). Here, n is the length of the linked list.

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def swapPairs(self, head: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        pre, cur = dummy, head
        while cur and cur.next:
            t = cur.next
            cur.next = t.next
            t.next = cur
            pre.next = t
            pre, cur = cur, cur.next
        return dummy.next
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Swap recursively
# 2. Swap the VALUES only (do not move nodes)


# ============================================================
# FOLLOW-UP 1: Swap recursively
# ============================================================

# ---- follow-up 1: function ----
# Answer: Swap the first two nodes and let the recursion
# swap the pairs of the rest.
# Time: O(n).  Space: O(n) recursion stack.
def swap_pairs_recursive(head):
    if head is None or head.next is None:
        return head
    second = head.next
    head.next = swap_pairs_recursive(second.next)
    second.next = head
    return second


# ============================================================
# FOLLOW-UP 2: Swap the VALUES only (do not move nodes)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk two nodes at a time and exchange their
# values.
# Time: O(n).  Space: O(1).
def swap_pair_values(head):
    node = head
    while node and node.next:
        node.val, node.next.val = node.next.val, node.val
        node = node.next.next
    return head
```

2. Add Two Numbers

Medium · Recursion, Linked List, Math

leetcode.com/problems/add-two-numbers

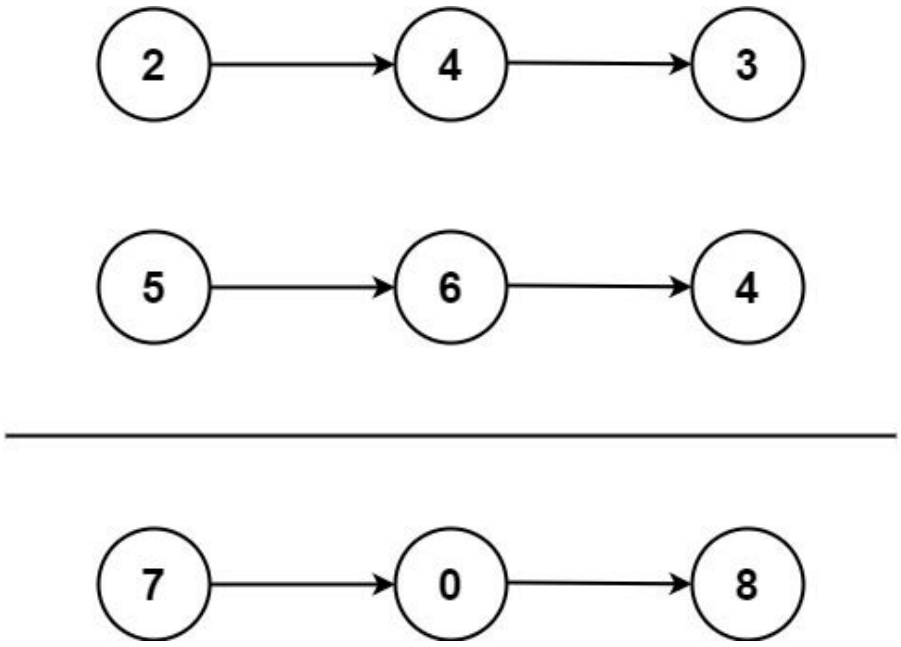
★ **TOP 7 PROBLEM** in Linked List Reversal / Manipulation: master this one first

PROBLEM

You are given two **non-empty** linked lists representing two non-negative integers. The digits are stored in **reverse order**, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:




```
Input: l1 = [2,4,3], l2 = [5,6,4]
Output: [7,0,8]
Explanation: 342 + 465 = 807.
```

Example 2:

```
Input: l1 = [0], l2 = [0]
Output: [0]
```

Example 3:

```
Input: l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]
Output: [8,9,9,9,0,0,0,1]
```

Constraints:

- The number of nodes in each linked list is in the range `[1, 100]`.
- `0 <= Node.val <= 9`
- It is guaranteed that the list represents a number that does not have leading zeros.

SOLUTION (PYTHON 3)

Solution 1: Simulation

How to think about it

The first idea is to convert both lists to integers and add. Length is at most 100, so big integers would pass, but the lists already store digits in reverse—this is column addition. Converting away from the list throws that structure away. The issue is not arithmetic volume: the lists may differ in length, and a final carry may remain. Each position is "the two digits (missing treated as 0) plus carry", then split into a new digit and a new carry. So we grow the answer from a dummy node and walk both lists together. The loop must continue while l_1, l_2, or carry remains; otherwise [9]+[1] drops the leading 1.

We traverse two linked lists l_1 and l_2 at the same time, and use the variable carry to indicate whether there is a carry.

Each time we traverse, we take out the current bit of the corresponding linked list, calculate the sum with the carry carry, and then update the value of the carry. Then we add the current bit to the answer linked list. If both linked lists are traversed, and the carry is 0, the traversal ends.

Finally, we return the head node of the answer linked list.

The time complexity is O(max (m, n)), where m and n are the lengths of the two linked lists. We need to traverse the entire position of the two linked lists, and each position only needs O(1) time. Ignoring the space consumption of the answer, the space complexity is O(1).

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def addTwoNumbers(
        self, l1: Optional[ListNode], l2: Optional[ListNode]
    ) -> Optional[ListNode]:
        dummy = ListNode()
        carry, curr = 0, dummy
        while l1 or l2 or carry:
            s = (l1.val if l1 else 0) + (l2.val if l2 else 0) + carry
            carry, val = divmod(s, 10)
            curr.next = ListNode(val)
            curr = curr.next
            l1 = l1.next if l1 else None
            l2 = l2.next if l2 else None
        return dummy.next
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Add K numbers given as lists (digits in reverse
# order)
# 2. Multiply a number (digits in reverse order) by a
# single digit


# ============================================================
# FOLLOW-UP 1: Add K numbers given as lists (digits in
# reverse order)
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
```

```
        self.val = val
        self.next = next

# ---- follow-up 1: function ----
# Answer: Add the lists one by one into a running total
# list, or add all the current digits of all lists at each
# position with one carry.
# Time: O(K * n), n = longest list.  Space: O(1) extra
# besides the result.
def add_many(lists):
    dummy = ListNode()
    tail = dummy
    carry = 0
    nodes = list(lists)
    while any(node is not None for node in nodes) or carry:
        total = carry
        for i in range(len(nodes)):
            if nodes[i] is not None:
                total += nodes[i].val
                nodes[i] = nodes[i].next
        tail.next = ListNode(total % 10)
        tail = tail.next
        carry = total // 10
    return dummy.next


# ============================================================
# FOLLOW-UP 2: Multiply a number (digits in reverse order)
# by a single digit
# ============================================================

# ---- follow-up 2: function ----
# Answer: Multiply each digit, add the carry, write the
# last digit, carry the rest.
# Time: O(n).  Space: O(1) extra besides the result.
def multiply_by_digit(head, digit):
    dummy = ListNode()
    tail = dummy
    carry = 0
    while head or carry:
        total = carry
        if head:
            total += head.val * digit
            head = head.next
        tail.next = ListNode(total % 10)
```

```
        tail = tail.next
        carry = total // 10
    return dummy.next
```

445. Add Two Numbers II

Medium · Stack, Linked List, Math

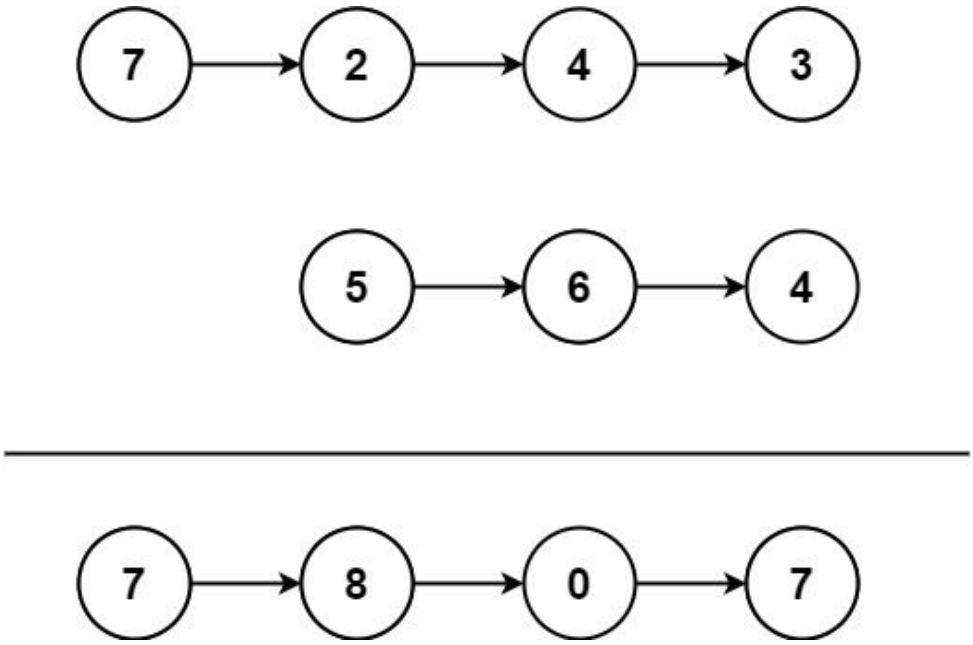
leetcode.com/problems/add-two-numbers-ii

PROBLEM

You are given two **non-empty** linked lists representing two non-negative integers. The most significant digit comes first and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:




```
Input: l1 = [7,2,4,3], l2 = [5,6,4]
Output: [7,8,0,7]
```

Example 2:

```
Input: l1 = [2,4,3], l2 = [5,6,4]
Output: [8,0,7]
```

Example 3:

```
Input: l1 = [0], l2 = [0]
Output: [0]
```

Constraints:

- The number of nodes in each linked list is in the range `[1, 100]`.
- `0 <= Node.val <= 9`
- It is guaranteed that the list represents a number that does not have leading zeros.

Follow up: Could you solve it without reversing the input lists?

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

The lists store the high digit first, so we cannot add from the heads as in Add Two Numbers. Reversing twice more is pointer-heavy. Push both lists onto stacks, pop the low digits, add with carry, and insert each new node at the front of the answer. A leftover carry becomes one more head insert. The stacks make the low end come out first; head insertion restores high-digit-first order without reversing the lists.

Python 3

```python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def addTwoNumbers(
        self, l1: Optional[ListNode], l2: Optional[ListNode]
    ) -> Optional[ListNode]:
        s1, s2 = [], []
        while l1:
            s1.append(l1.val)
            l1 = l1.next
        while l2:
            s2.append(l2.val)
            l2 = l2.next
        dummy = ListNode()
        carry = 0
        while s1 or s2 or carry:
            s = (0 if not s1 else s1.pop()) + (0 if not s2 else s2.pop()) + carry
            carry, val = divmod(s, 10)
            # node = ListNode(val, dummy.next)
            # dummy.next = node
            dummy.next = ListNode(val, dummy.next)
        return dummy.next
```

Complexity. Time: O(m + n). Space: O(m + n).

Solution 2

How to think about it

Solution 1 already adds via two stacks. Solution 2 is the same skeleton: pop, add, insert at the dummy head, until both stacks and the carry are gone.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```python
class Solution:
    def addTwoNumbers(self, l1: Optional[ListNode], l2: Optional[ListNode]) -> Optional[ListNode]:
        def to_stack(head: Optional[ListNode]) -> List[int]:
            stack = []
            while head:
                stack.append(head.val)
                head = head.next
            return stack

        s1, s2 = to_stack(l1), to_stack(l2)
        dummy = ListNode()
        carry = 0
        while s1 or s2 or carry:
            if s1:
                carry += s1.pop()
            if s2:
                carry += s2.pop()
            dummy.next = ListNode(carry % 10, dummy.next)      # put the new digit in front
            carry //= 10
        return dummy.next
```

Complexity. Time: O(m + n). Space: O(m + n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Compare two numbers given as lists (most significant
# digit first)
# 2. Subtract the smaller number from the bigger one


# ============================================================
# FOLLOW-UP 1: Compare two numbers given as lists (most
# significant digit first)
# ============================================================

# ---- follow-up 1: function ----
# Answer: A longer number (without leading zeros) is
# bigger; with equal lengths compare digit by digit from
# the front.
# Time: O(n).  Space: O(n).
def compare_numbers(a, b):
    def to_digits(node):
        digits = []
        while node:
            digits.append(node.val)
            node = node.next
        while len(digits) > 1 and digits[0] == 0:
            digits.pop(0)
        return digits

    x, y = to_digits(a), to_digits(b)
    if len(x) != len(y):
        return 1 if len(x) > len(y) else -1
    for i in range(len(x)):
        if x[i] != y[i]:
            return 1 if x[i] > y[i] else -1
    return 0


# ============================================================
# FOLLOW-UP 2: Subtract the smaller number from the bigger
# one
# ============================================================

# ---- follow-up 2: helper functions ----
```

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 2: function ----
# Answer: Put the digits on stacks, subtract from the last
# digit with a borrow, and build the answer list from the
# front.
# Time: O(n).  Space: O(n).
def subtract_numbers(a, b):
    def to_stack(node):
        stack = []
        while node:
            stack.append(node.val)
            node = node.next
        return stack

    x, y = to_stack(a), to_stack(b)
```

```
    result = None
    borrow = 0
    while x:
        top = x.pop() - borrow
        bottom = y.pop() if y else 0
        if top < bottom:
            top += 10
            borrow = 1
        else:
            borrow = 0
        result = ListNode(top - bottom, result)
    while result.next and result.val == 0:
        result = result.next
    return result
```

138. Copy List with Random Pointer

Medium · Hash Table, Linked List

leetcode.com/problems/copy-list-with-random-pointer

★ **TOP 7 PROBLEM** in Linked List Reversal / Manipulation: master this one first

PROBLEM

A linked list of length `n` is given such that each node contains an additional random pointer, which could point to any node in the list, or `null`.

Construct a **deep copy** of the list. The deep copy should consist of exactly `n` **brand new** nodes, where each new node has its value set to the value of its corresponding original node. Both the `next` and `random` pointer of the new nodes should point to new nodes in the copied list such that the pointers in the original list and copied list represent the same list state. **None of the pointers in the new list should point to nodes in the original list**.

For example, if there are two nodes `X` and `Y` in the original list, where `X.random --> Y`, then for the corresponding two nodes `x` and `y` in the copied list, `x.random --> y`.

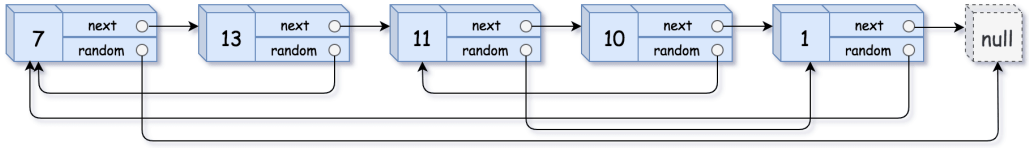
Return *the head of the copied linked list*.

The linked list is represented in the input/output as a list of `n` nodes. Each node is represented as a pair of `[val, random_index]` where:

- `val`: an integer representing `Node.val`
- `random_index`: the index of the node (range from `0` to `n-1`) that the `random` pointer points to, or `null` if it does not point to any node.

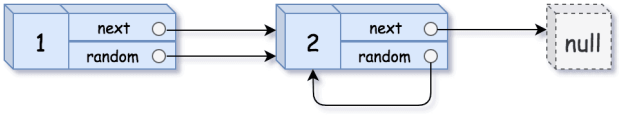
Your code will **only** be given the `head` of the original linked list.

Example 1:



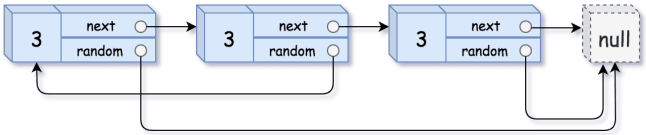

```
Input: head = [[7,null],[13,0],[11,4],[10,2],[1,0]]
Output: [[7,null],[13,0],[11,4],[10,2],[1,0]]
```

Example 2:




```
Input: head = [[1,1],[2,1]]
Output: [[1,1],[2,1]]
```

Example 3:




```
Input: head = [[3,null],[3,0],[3,null]]
Output: [[3,null],[3,0],[3,null]]
```

Constraints:

- `0 <= n <= 1000`
- -10^4 `<= Node.val <=` 10^4
- `Node.random` is `null` or is pointing to some node in the linked list.

SOLUTIONS (PYTHON 3)

Solution 1: Hash Table

> **How to think about it**
>
> Copy a list whose random pointer may refer to any node or null. If we copy next first, the random target may not exist yet. n≤ 1000. A first pass builds the new list along next and maps each original node to its copy; a second pass wires random through that map.

We can define a dummy head node dummy and use a pointer tail to point to the dummy head node. Then, we traverse the linked list, copying each node and storing the mapping between each node and its copy in a hash table d, while also connecting the next pointers of the copied nodes.

Next, we traverse the linked list again and use the mappings stored in the hash table to connect the random pointers of the copied nodes.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the linked list.

Python 3

```python
"""
# Definition for a Node.
class Node:
    def __init__(self, x: int, next: 'Node' = None, random: 'Node' = None):
        self.val = int(x)
        self.next = next
        self.random = random
"""


class Solution:
    def copyRandomList(self, head: "Optional[Node]") -> "Optional[Node]":
        d = {}
        dummy = tail = Node(0)
        cur = head
        while cur:
            node = Node(cur.val)
            tail.next = node
            tail = tail.next
            d[cur] = node
            cur = cur.next
        cur = head
        while cur:
            d[cur].random = d[cur.random] if cur.random else None
            cur = cur.next
        return dummy.next
```

Solution 2: Simulation (Space Optimization)

How to think about it

Solution 1's map is O(n) space. Insert each copy after its original so the pair is adjacent; the matching random node is cur.random.next. Split the list afterwards. No hash table.

In Solution 1, we used an additional hash table to store the mapping between the original nodes and the copied nodes. We can also achieve this without using extra space, as follows:

1. Traverse the original linked list, and for each node, create a new node and insert it between the original node and the original node's next node.
2. Traverse the linked list again, and set the random pointer of the new node based on the random pointer of the original node.
3. Finally, split the linked list into the original linked list and the copied linked list.

The time complexity is O(n), where n is the length of the linked list. Ignoring the space occupied by the answer linked list, the space complexity is O(1).

Python 3

```
"""
# Definition for a Node.
class Node:
    def __init__(self, x: int, next: 'Node' = None, random: 'Node' = None):
        self.val = int(x)
        self.next = next
        self.random = random
"""


class Solution:
    def copyRandomList(self, head: "Optional[Node]") -> "Optional[Node]":
        if head is None:
            return None
        cur = head
        while cur:
            node = Node(cur.val, cur.next)
            cur.next = node
            cur = node.next
        cur = head
        while cur:
            cur.next.random = cur.random.next if cur.random else None
            cur = cur.next.next
        cur = head
        ans = head.next
        while cur.next:
            node = cur.next
            cur.next = node.next
            cur = node
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. O(1) extra space: weave the copies into the original
# list
# 2. Check that a copy is a deep copy of the original


# ============================================================
# FOLLOW-UP 1: O(1) extra space: weave the copies into the
# original list
# ============================================================
```

```
# ---- follow-up 1: helper functions ----
class Node:
    def __init__(self, val=0, next=None, random=None):
        self.val = val
        self.next = next
        self.random = random

# ---- follow-up 1: function ----
# Answer: Put every copy right after its original node,
# set each copy's random pointer to the node after the
# original's random node, then split the two lists apart
# again.
# Time: O(n).  Space: O(1) extra besides the copy.
def copy_random_list_constant_space(head):
    if head is None:
        return None
    node = head
    while node:
        copy = Node(node.val, node.next)
        node.next = copy
        node = copy.next
    node = head
    while node:
        if node.random:
            node.next.random = node.random.next
        node = node.next.next
    new_head = head.next
    node = head
    while node:
        copy = node.next
        node.next = copy.next
        copy.next = copy.next.next if copy.next else None
        node = node.next
    return new_head


# ============================================================
# FOLLOW-UP 2: Check that a copy is a deep copy of the
# original
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk both lists together: values and the
# position of the random target must match, and no node
# may be shared.
# Time: O(n).  Space: O(n).
def is_valid_copy(original, copy):
    index = {}
    node = original
```

```
    position = 0
    while node:
        index[node] = position
        node = node.next
        position += 1
    copy_index = {}
    node = copy
    position = 0
    while node:
        copy_index[node] = position
        node = node.next
        position += 1
    a, b = original, copy
    while a and b:
        if a is b or a.val != b.val:
            return False
        a_random = index[a.random] if a.random else -1
        b_random = copy_index.get(b.random, -2) if b.random else -1
        if a_random != b_random:
```

```
            return False
        a, b = a.next, b.next
    return a is None and b is None
```

PATTERN 25 · LINKED LIST REVERSAL / MANIPULATION · PROBLEM 10 OF 20

430. Flatten a Multilevel Doubly Linked List

Medium · Depth-First Search, Linked List, Doubly-Linked List

leetcode.com/problems/flatten-a-multilevel-doubly-linked-list

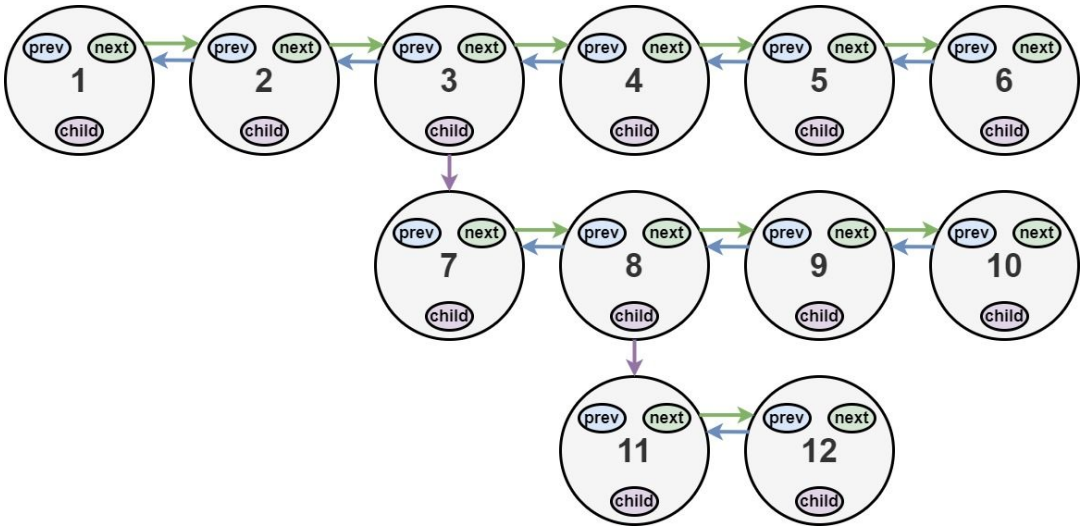
PROBLEM

You are given a doubly linked list, which contains nodes that have a next pointer, a previous pointer, and an additional **child pointer**. This child pointer may or may not point to a separate doubly linked list, also containing these special nodes. These child lists may have one or more children of their own, and so on, to produce a **multilevel data structure** as shown in the example below.

Given the `head` of the first level of the list, **flatten** the list so that all the nodes appear in a single-level, doubly linked list. Let `curr` be a node with a child list. The nodes in the child list should appear **after** `curr` and **before** `curr.next` in the flattened list.

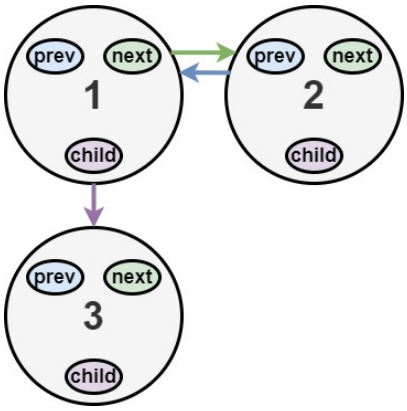
Return *the* `head` *of the flattened list. The nodes in the list must have* ***all*** *of their child pointers set to* `null`.

Example 1:




```
Input: head = [1,2,3,4,5,6,null,null,null,7,8,9,10,null,null,11,12]
Output: [1,2,3,7,8,11,12,9,10,4,5,6]
Explanation: The multilevel linked list in the input is shown.
After flattening the multilevel linked list it becomes:
```

Example 2:



```
Input: head = [1,2,null,3]
Output: [1,3,2]
Explanation: The multilevel linked list in the input is shown.
After flattening the multilevel linked list it becomes:
```

Example 3:

```
Input: head = []
Output: []
Explanation: There could be empty list in the input.
```

Constraints:

- The number of Nodes will not exceed `1000`.
- `1 <= Node.val <= 10^5`

How the multilevel linked list is represented in test cases:

We use the multilevel linked list from **Example 1** above:

```
 1---2---3---4---5---6--NULL
         |
         7---8---9---10--NULL
             |
             11--12--NULL
```

The serialization of each level is as follows:

```
[1,2,3,4,5,6,null]
[7,8,9,10,null]
[11,12,null]
```

To serialize all levels together, we will add nulls in each level to signify no node connects to the upper node of the previous level. The serialization becomes:

```
[1, 2, 3, 4, 5, 6, null]
             |
[null, null, 7, 8, 9, 10, null]
                   |
[ null, 11, 12, null]
```

Merging the serialization of each level and removing trailing nulls we obtain:

```
[1,2,3,4,5,6,null,null,null,7,8,9,10,null,null,11,12]
```

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Flattening a multilevel list is a preorder walk: child before the original next. Walking next first would append the child list at the very end. preorder(pre,cur) links the current node after its predecessor, saves the old next, flattens the child (the returned tail becomes the new predecessor), then flattens the saved next and clears child. A dummy node holds the head; afterwards the real head's prev is cleared. The original next must be saved before the child walk overwrites it.

Python 3

```
"""
# Definition for a Node.
class Node:
    def __init__(self, val, prev, next, child):
        self.val = val
        self.prev = prev
        self.next = next
        self.child = child
"""


class Solution:
    def flatten(self, head: 'Node') -> 'Node':
        def preorder(pre, cur):
            if cur is None:
                return pre
            cur.prev = pre
            pre.next = cur

            t = cur.next
            tail = preorder(cur, cur.child)
            cur.child = None
            return preorder(tail, t)

        if head is None:
            return None
        dummy = Node(0, None, head, None)
        preorder(dummy, head)
        dummy.next.prev = None
        return dummy.next
```

Complexity. Time: O(n). Space: O(d), d = deepest nesting level (O(n) in the worst case).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. How many levels deep does the structure go?
# 2. Flatten level by level (all of level 1, then all of
# level 2, ...)


# ============================================================
# FOLLOW-UP 1: How many levels deep does the structure go?
# ============================================================

# ---- follow-up 1: function ----
# Answer: Walk the list; every child pointer starts a
# level one deeper. Track the biggest depth.
# Time: O(n).  Space: O(d), d = depth.
def max_levels(head):
    best = 0
    stack = [(head, 1)]
    while stack:
        node, depth = stack.pop()
        while node:
            if depth > best:
                best = depth
            if node.child:
                stack.append((node.child, depth + 1))
            node = node.next
    return best


# ============================================================
# FOLLOW-UP 2: Flatten level by level (all of level 1,
# then all of level 2, ...)
```

```
# ============================================================

# ---- follow-up 2: function ----
# Answer: A breadth-first walk with a queue: collect the
# nodes of one level, queue their children for the next
# level.
# Time: O(n).  Space: O(w), w = widest level.
from collections import deque


def flatten_by_levels(head):
    order = []
    queue = deque([head] if head else [])
    while queue:
        node = queue.popleft()
        while node:
            order.append(node.val)
            if node.child:
                queue.append(node.child)
            node = node.next
    return order
```

707. Design Linked List

Medium · Design, Linked List

leetcode.com/problems/design-linked-list

PROBLEM

Design your implementation of the linked list. You can choose to use a singly or doubly linked list.
A node in a singly linked list should have two attributes: `val` and `next`. `val` is the value of the current node, and `next` is a pointer/reference to the next node.
If you want to use the doubly linked list, you will need one more attribute `prev` to indicate the previous node in the linked list. Assume all nodes in the linked list are **0-indexed**.

Implement the `MyLinkedList` class:

- `MyLinkedList()` Initializes the `MyLinkedList` object.
- `int get(int index)` Get the value of the `index`th node in the linked list. If the index is invalid, return `-1`.
- `void addAtHead(int val)` Add a node of value `val` before the first element of the linked list. After the insertion, the new node will be the first node of the linked list.
- `void addAtTail(int val)` Append a node of value `val` as the last element of the linked list.
- `void addAtIndex(int index, int val)` Add a node of value `val` before the `index`th node in the linked list. If `index` equals the length of the linked list, the node will be appended to the end of the linked list. If `index` is greater than the length, the node **will not be inserted**.
- `void deleteAtIndex(int index)` Delete the `index`th node in the linked list, if the index is valid.

Example 1:

```
Input
["MyLinkedList", "addAtHead", "addAtTail", "addAtIndex", "get", "deleteAtIndex", "get"]
[[], [1], [3], [1, 2], [1], [1], [1]]
Output
[null, null, null, null, 2, null, 3]

Explanation
MyLinkedList myLinkedList = new MyLinkedList();
myLinkedList.addAtHead(1);
myLinkedList.addAtTail(3);
myLinkedList.addAtIndex(1, 2); // linked list becomes 1->2->3
myLinkedList.get(1); // return 2
myLinkedList.deleteAtIndex(1); // now the linked list is 1->3
myLinkedList.get(1); // return 3
```

Constraints:

- `0 <= index, val <= 1000`
- Please do not use the built-in LinkedList library.
- At most `2000` calls will be made to `get`, `addAtHead`, `addAtTail`, `addAtIndex` and `deleteAtIndex`.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

We must implement index access, head/tail inserts, and indexed deletes without a library list. At most 2000 calls, so an O(n) walk from the head is fine. A singly linked list can reach index index only by walking predecessors. A dummy head unifies empty and non-empty updates; cnt makes bounds checks independent of the walk. Head and tail inserts reduce to addAtIndex: walk to the predecessor, then relink. Deletes do the same. Each operation is O(n).

Python 3

```python
class MyLinkedList:
    def __init__(self):
        self.dummy = ListNode()
        self.cnt = 0

    def get(self, index: int) -> int:
        if index < 0 or index >= self.cnt:
            return -1
        cur = self.dummy.next
        for _ in range(index):
            cur = cur.next
        return cur.val

    def addAtHead(self, val: int) -> None:
        self.addAtIndex(0, val)

    def addAtTail(self, val: int) -> None:
        self.addAtIndex(self.cnt, val)

    def addAtIndex(self, index: int, val: int) -> None:
        if index > self.cnt:
            return
        pre = self.dummy
        for _ in range(index):
            pre = pre.next
        pre.next = ListNode(val, pre.next)
        self.cnt += 1

    def deleteAtIndex(self, index: int) -> None:
        if index >= self.cnt:
            return
        pre = self.dummy
        for _ in range(index):
            pre = pre.next
        t = pre.next
        pre.next = t.next
        t.next = None
        self.cnt -= 1


# Your MyLinkedList object will be instantiated and called as such:
# obj = MyLinkedList()
# param_1 = obj.get(index)
# obj.addAtHead(val)
# obj.addAtTail(val)
# obj.addAtIndex(index,val)
# obj.deleteAtIndex(index)
```

Solution 2

How to think about it

Solution 1 allocates a node on every insert. The limit here is tiny, but repeated allocation dominates once the list grows large. Preallocate two arrays for values and successor indices, and treat an increasing idx as a node pool. Inserting claims the next slot and rewrites indices—the same linking as pointers. head stores the first index and cnt the length. Asymptotics match Solution 1 without per-operation allocation.

Python 3

```python
class MyLinkedList:
    def __init__(self):
        self.e = [0] * 1010
        self.ne = [0] * 1010
        self.idx = 0
        self.head = -1
        self.cnt = 0

    def get(self, index: int) -> int:
        if index < 0 or index >= self.cnt:
            return -1
        i = self.head
        for _ in range(index):
            i = self.ne[i]
        return self.e[i]

    def addAtHead(self, val: int) -> None:
        self.e[self.idx] = val
        self.ne[self.idx] = self.head
        self.head = self.idx
        self.idx += 1
        self.cnt += 1

    def addAtTail(self, val: int) -> None:
        self.addAtIndex(self.cnt, val)

    def addAtIndex(self, index: int, val: int) -> None:
        if index > self.cnt:
            return
        if index <= 0:
            self.addAtHead(val)
            return
        i = self.head
        for _ in range(index - 1):
            i = self.ne[i]
        self.e[self.idx] = val
        self.ne[self.idx] = self.ne[i]
        self.ne[i] = self.idx
        self.idx += 1
        self.cnt += 1

    def deleteAtIndex(self, index: int) -> None:
        if index < 0 or index >= self.cnt:
            return -1
        self.cnt -= 1
        if index == 0:
            self.head = self.ne[self.head]
            return
        i = self.head
        for _ in range(index - 1):
            i = self.ne[i]
        self.ne[i] = self.ne[self.ne[i]]


# Your MyLinkedList object will be instantiated and called as such:
# obj = MyLinkedList()
# param_1 = obj.get(index)
# obj.addAtHead(val)
# obj.addAtTail(val)
# obj.addAtIndex(index,val)
```

```python
# obj.deleteAtIndex(index)
```

Complexity. Time: get, addAtIndex, deleteAtIndex O(n); addAtHead O(1). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Add a reverse() operation
# 2. A tail pointer makes adding at the tail O(1)


# ============================================================
# FOLLOW-UP 1: Add a reverse() operation
# ============================================================

# ---- follow-up 1: function ----
# Answer: Reverse the node links in place; the head
# becomes the old last node.
# Time: add_at_tail O(n), reverse O(n).  Space: O(n).
class MyLinkedListWithReverse:
    class Node:
        def __init__(self, val):
            self.val = val
            self.next = None

    def __init__(self):
        self.head = None
        self.size = 0

    def add_at_tail(self, val):
        node = MyLinkedListWithReverse.Node(val)
        if self.head is None:
            self.head = node
        else:
            last = self.head
            while last.next:
                last = last.next
            last.next = node
        self.size += 1

    def reverse(self):
        previous = None
        node = self.head
        while node:
            following = node.next
            node.next = previous
            previous = node
            node = following
        self.head = previous

    def to_list(self):
        values = []
        node = self.head
        while node:
            values.append(node.val)
            node = node.next
        return values


# ============================================================
# FOLLOW-UP 2: A tail pointer makes adding at the tail
# O(1)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep a pointer to the last node and update it
```

```
# when the list grows or the last node is deleted.
# Time: add_at_tail O(1), delete_at_index O(n).  Space:
# O(n).
class MyLinkedListWithTail:
```

```
    class Node:
        def __init__(self, val):
            self.val = val
            self.next = None

    def __init__(self):
        self.head = None
        self.tail = None

    def add_at_tail(self, val):
        node = MyLinkedListWithTail.Node(val)
        if self.tail is None:
            self.head = self.tail = node
        else:
            self.tail.next = node
            self.tail = node

    def delete_at_index(self, index):
        previous = None
        node = self.head
        for i in range(index):
            if node is None:
                return
            previous = node
            node = node.next
        if node is None:
            return
        if previous is None:
            self.head = node.next
        else:
            previous.next = node.next
        if node is self.tail:
            self.tail = previous
```

1721. Swapping Nodes in a Linked List

Medium · Linked List, Two Pointers

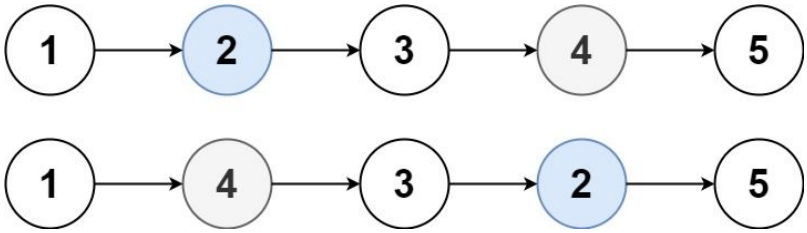
leetcode.com/problems/swapping-nodes-in-a-linked-list

PROBLEM

You are given the `head` of a linked list, and an integer `k`.

Return *the head of the linked list after **swapping** the values of the* k^{th} *node from the beginning and the* k^{th} *node from the end (the list is **1-indexed**).*

Example 1:




```
Input: head = [1,2,3,4,5], k = 2
Output: [1,4,3,2,5]
```

Example 2:

```
Input: head = [7,9,6,6,7,8,3,0,9,5], k = 5
Output: [7,9,6,6,8,7,3,0,9,5]
```

Constraints:

- The number of nodes in the list is `n`.
- `1 <= k <= n <=` 10^5
- `0 <= Node.val <= 100`

SOLUTION (PYTHON 3)

Solution 1: Two Pointers

> **How to think about it**
>
> We swap the values of the k-th node from the start and from the end. Measuring the length and walking twice works, but one pass is enough. Advance a fast pointer k-1 steps to the k-th node p, then move fast and slow together. When fast reaches the tail, slow is the k-th from the end. Swap the two values and return the head.

We can first use a fast pointer `fast` to find the kth node of the linked list, and use a pointer `p` to point to it. Then, we use a slow pointer `slow` to start from the head node of the linked list, and move both pointers forward at the same time. When the fast pointer reaches the last node of the linked list, the slow pointer `slow` points to the kth node from the end of the linked list, and we use a pointer `q` to point to it. At this point, we only need to swap the values of `p` and `q`.

The time complexity is O(n), where n is the length of the linked list. The space complexity is O(1).

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def swapNodes(self, head: Optional[ListNode], k: int) -> Optional[ListNode]:
        fast = slow = head
        for _ in range(k - 1):
            fast = fast.next
        p = fast
        while fast.next:
            fast, slow = fast.next, slow.next
        q = slow
        p.val, q.val = q.val, p.val
        return head
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Swap the NODES themselves (not just their values)
# 2. Swap the i-th node from the start with the i-th node
# from the end for EVERY i (reverse values, no extra list)


# ============================================================
# FOLLOW-UP 1: Swap the NODES themselves (not just their
# values)
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 1: function ----
# Answer: Find the two nodes and their predecessors, then
# re-link; take care when the nodes are neighbours or the
# same node.
# Time: O(n).  Space: O(1).
def swap_nodes_relink(head, k):
    dummy = ListNode(0, head)
    n = 0
    node = head
    while node:
        n += 1
        node = node.next
    i, j = k, n - k + 1
    if i == j:
        return head
    if i > j:
        i, j = j, i
    before_i = dummy
    for step in range(i - 1):
        before_i = before_i.next
    before_j = dummy
    for step in range(j - 1):
        before_j = before_j.next
    first, second = before_i.next, before_j.next
    if before_j is first:                              # neighbours
        first.next, second.next = second.next, first
        before_i.next = second
    else:
        before_i.next, before_j.next = second, first
```

```
        first.next, second.next = second.next, first.next
    return dummy.next


# ============================================================
# FOLLOW-UP 2: Swap the i-th node from the start with the
# i-th node from the end for EVERY i (reverse values, no
# extra list)
# ============================================================

# ---- follow-up 2: function ----
# Answer: That is reversing the list; do it by swapping
# values with a stack of values, or reverse the links.
# Time: O(n).  Space: O(n).
```

```
def mirror_values(head):
    values = []
    node = head
    while node:
        values.append(node.val)
        node = node.next
    node = head
    for value in reversed(values):
        node.val = value
        node = node.next
    return head
```

147. Insertion Sort List

Medium · Linked List, Sorting

leetcode.com/problems/insertion-sort-list

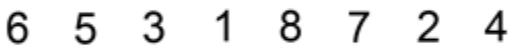
PROBLEM

Given the `head` of a singly linked list, sort the list using **insertion sort**, and return *the sorted list's head*.

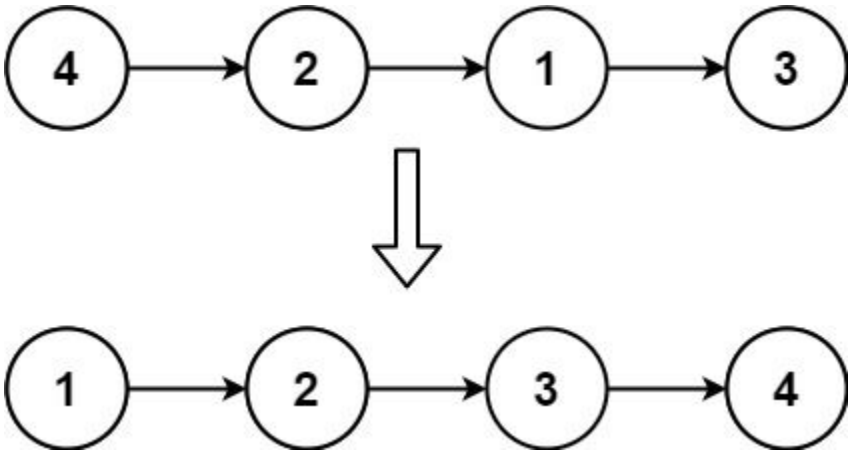
The steps of the **insertion sort** algorithm:

1. Insertion sort iterates, consuming one input element each repetition and growing a sorted output list.
2. At each iteration, insertion sort removes one element from the input data, finds the location it belongs within the sorted list and inserts it there.
3. It repeats until no input elements remain.

The following is a graphical example of the insertion sort algorithm. The partially sorted list (black) initially contains only the first element in the list. One element (red) is removed from the input data and inserted in-place into the sorted list with each iteration.

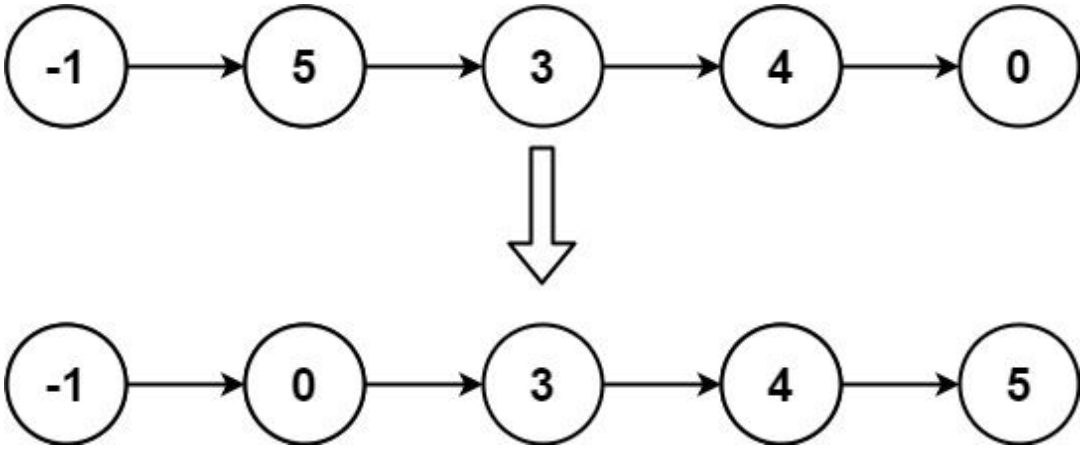


Example 1:




```
Input: head = [4,2,1,3]
Output: [1,2,3,4]
```

Example 2:




```
Input: head = [-1,5,3,4,0]
Output: [-1,0,3,4,5]
```

Constraints:

- The number of nodes in the list is in the range `[1, 5000]`.
- `-5000 <= Node.val <= 5000`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Insertion-sort a list. n≤ 5000, so quadratic time is fine. Sorting an array and rebuilding the list is not insertion sort. pre marks the end of the sorted prefix. If the current value is larger, extend the prefix; otherwise walk from a dummy to the first larger node and splice it in. The dummy removes the head-insertion special case.

Python 3

```python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def insertionSortList(self, head: ListNode) -> ListNode:
        if head is None or head.next is None:
            return head
        dummy = ListNode(head.val, head)
        pre, cur = dummy, head
        while cur:
            if pre.val <= cur.val:
                pre, cur = cur, cur.next
                continue
            p = dummy
            while p.next.val <= cur.val:
                p = p.next
            t = cur.next
            cur.next = p.next
            p.next = cur
            pre.next = t
            cur = t
        return dummy.next
```

Complexity. Time: O(n^2). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Sort in descending order
# 2. Count how many comparisons the insertion sort makes


# ============================================================
# FOLLOW-UP 1: Sort in descending order
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 1: function ----
# Answer: The same insertion into the sorted part, with
# the comparison reversed.
# Time: O(n^2).  Space: O(1).
def insertion_sort_descending(head):
    dummy = ListNode()
    node = head
    while node:
        following = node.next
        previous = dummy
        while previous.next and previous.next.val >= node.val:
            previous = previous.next
        node.next = previous.next
        previous.next = node
        node = following
    return dummy.next


# ============================================================
# FOLLOW-UP 2: Count how many comparisons the insertion
# sort makes
# ============================================================

# ---- follow-up 2: function ----
# Answer: Add one to a counter every time a node is
# compared with a node of the sorted part.
# Time: O(n^2).  Space: O(1).
def insertion_sort_comparisons(head):
    dummy = ListNode()
    comparisons = 0
    node = head
    while node:
        following = node.next
        previous = dummy
        while previous.next:
            comparisons += 1
            if previous.next.val >= node.val:
                break
            previous = previous.next
        node.next = previous.next
        previous.next = node
        node = following
    return comparisons
```

369. Plus One Linked List

Medium · Linked List, Math

leetcode.com/problems/plus-one-linked-list

PROBLEM

Given a non-negative integer represented as a linked list of digits, *plus one to the integer*.

The digits are stored such that the most significant digit is at the `head` of the list.

Example 1:

```
Input: head = [1,2,3]
Output: [1,2,4]
```

Example 2:

```
Input: head = [0]
Output: [1]
```

Constraints:

- The number of nodes in the linked list is in the range `[1, 100]`.
- `0 <= Node.val <= 9`
- The number represented by the linked list does not contain leading zeros except for the zero itself.

SOLUTION (PYTHON 3)

Solution 1: Linked List Traversal

> **How to think about it**
>
> Add one to a non-negative integer stored as a list. Materializing an array uses extra O(n) space. Carry only touches a trailing run of nines. A dummy head covers a new high digit. One scan remembers the last non-nine, increments it, and zeros the rest. If the dummy becomes 1, it is the new head.

We first set a dummy head node dummy, initially with a value of 0, and the successor node of dummy is the linked list head.

Next, we traverse the linked list starting from the dummy head node, find the last node that is not 9, increment its value by 1, and set the values of all nodes after this node to 0.

Finally, we check if the value of the dummy head node is 1. If it is 1, we return dummy; otherwise, we return the successor node of dummy.

The time complexity is O(n), where n is the length of the linked list. The space complexity is O(1).

Python 3

```python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def plusOne(self, head: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode(0, head)
        target = dummy
        while head:
            if head.val != 9:
                target = head
            head = head.next
        target.val += 1
        target = target.next
        while target:
            target.val = 0
            target = target.next
        return dummy if dummy.val else dummy.next
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Add any non-negative integer k to the number
# 2. Subtract one from the number (the number is at least
# 1)


# ========================================================
# FOLLOW-UP 1: Add any non-negative integer k to the
# number
# ========================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 1: function ----
# Answer: Put the digits on a stack (or reverse the list),
# add k digit by digit with a carry from the end, and
# build the result.
# Time: O(n + log k).  Space: O(n).
def add_integer(head, k):
    digits = []
    while head:
        digits.append(head.val)
        head = head.next
    result = None
    carry = k
    while digits or carry:
        carry += digits.pop() if digits else 0
        result = ListNode(carry % 10, result)
        carry //= 10
    return result if result else ListNode(0)


# ========================================================
# FOLLOW-UP 2: Subtract one from the number (the number is
# at least 1)
# ========================================================

# ---- follow-up 2: function ----
# Answer: Find the last non-zero digit, lower it by one
```

```
# and turn the zeros after it into 9s; remove a leading
# zero.
# Time: O(n).  Space: O(n).
def minus_one(head):
    digits = []
    node = head
    while node:
        digits.append(node)
        node = node.next
    i = len(digits) - 1
    while digits[i].val == 0:
        digits[i].val = 9
        i -= 1
    digits[i].val -= 1
    if head.val == 0 and head.next:
        return head.next
    return head
```

1381. Design a Stack With Increment Operation

Medium · Stack, Design, Array

leetcode.com/problems/design-a-stack-with-increment-operation

Also appears in pattern 29 Design Problems

PROBLEM

Design a stack that supports increment operations on its elements.

Implement the `CustomStack` class:

- `CustomStack(int maxSize)` Initializes the object with `maxSize` which is the maximum number of elements in the stack.
- `void push(int x)` Adds `x` to the top of the stack if the stack has not reached the `maxSize`.
- `int pop()` Pops and returns the top of the stack or `-1` if the stack is empty.
- `void inc(int k, int val)` Increments the bottom `k` elements of the stack by `val`. If there are less than `k` elements in the stack, increment all the elements in the stack.

Example 1:

```
Input
["CustomStack","push","push","pop","push","push","push","increment","increment","pop","pop","pop","pop"]
[[3],[1],[2],[],[2],[3],[4],[5,100],[2,100],[],[],[],[]]
Output
[null,null,null,2,null,null,null,null,null,103,202,201,-1]
Explanation
CustomStack stk = new CustomStack(3); // Stack is Empty []
stk.push(1); // stack becomes [1]
stk.push(2); // stack becomes [1, 2]
stk.pop(); // return 2 --> Return top of the stack 2, stack becomes [1]
stk.push(2); // stack becomes [1, 2]
stk.push(3); // stack becomes [1, 2, 3]
stk.push(4); // stack still [1, 2, 3], Do not add another elements as size is 4
stk.increment(5, 100); // stack becomes [101, 102, 103]
stk.increment(2, 100); // stack becomes [201, 202, 103]
stk.pop(); // return 103 --> Return top of the stack 103, stack becomes [201, 202]
stk.pop(); // return 202 --> Return top of the stack 202, stack becomes [201]
stk.pop(); // return 201 --> Return top of the stack 201, stack becomes []
stk.pop(); // return -1 --> Stack is empty return -1.
```

Constraints:

- `1 <= maxSize, x, k <= 1000`
- `0 <= val <= 100`
- At most `1000` calls will be made to each method of `increment`, `push` and `pop` each separately.

SOLUTION (PYTHON 3)

Solution 1: Array Simulation

How to think about it

increment adds val to the bottom k entries. Scanning the stack is fine for 10^3 calls, yet O(1) is possible: add[i] is a lazy increment for index i and below. The update lands on min(k,size)-1; a pop forwards that increment one slot down and clears it.

We can use an array stk to simulate the stack, and an integer i to represent the position of the next element to be pushed into the stack. In addition, we need another array add to record the cumulative increment value at each position.

When calling push(x), if i < maxSize, we put x into stk[i] and increment i by one.

When calling pop(), if i ≤ 0, it means the stack is empty, so we return -1. Otherwise, we decrement i by one, and the answer is stk[i] + add[i]. Then we add add[i] to add[i - 1], and set add[i] to zero. Finally, we return the answer.

When calling increment(k, val), if i > 0, we add val to add[min(i, k) - 1].

The time complexity is O(1), and the space complexity is O(n). Where n is the maximum capacity of the stack.

Python 3

```python
class CustomStack:
    def __init__(self, maxSize: int):
        self.stk = [0] * maxSize
        self.add = [0] * maxSize
        self.i = 0

    def push(self, x: int) -> None:
        if self.i < len(self.stk):
            self.stk[self.i] = x
            self.i += 1

    def pop(self) -> int:
        if self.i <= 0:
            return -1
        self.i -= 1
        ans = self.stk[self.i] + self.add[self.i]
        if self.i > 0:
            self.add[self.i - 1] += self.add[self.i]
        self.add[self.i] = 0
        return ans

    def increment(self, k: int, val: int) -> None:
        i = min(k, self.i) - 1
        if i >= 0:
            self.add[i] += val


# Your CustomStack object will be instantiated and called as such:
# obj = CustomStack(maxSize)
# obj.push(x)
# param_2 = obj.pop()
# obj.increment(k,val)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. O(1) increment with lazy increments
# 2. Add a decrement operation (subtract from the bottom k
# elements)


# ============================================================
# FOLLOW-UP 1: O(1) increment with lazy increments
# ============================================================

# ---- follow-up 1: function ----
# Answer: Do not touch the numbers at once: remember the
# increment at the position of the k-th element and pass
# it down to the element below when that element is
# popped.
# Time: push, pop and increment are each O(1).  Space:
# O(n).
class CustomStackLazy:
    def __init__(self, max_size):
```

```
        self.max_size = max_size
        self.items = []
        self.pending = []                           # pending[i] is added to items[0..i] when popped

    def push(self, x):
        if len(self.items) < self.max_size:
            self.items.append(x)
            self.pending.append(0)

    def pop(self):
        if not self.items:
            return -1
        add = self.pending.pop()
        value = self.items.pop() + add
        if self.pending:
            self.pending[-1] += add
        return value

    def increment(self, k, val):
        position = min_of(k, len(self.items)) - 1
        if position >= 0:
            self.pending[position] += val


def min_of(a, b):
    if a < b:
        return a
    return b


# ============================================================
# FOLLOW-UP 2: Add a decrement operation (subtract from
# the bottom k elements)
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same lazy idea with a negative value, or
# call increment with minus val.
# Time: push and pop O(1), decrement O(k).  Space: O(n).
class CustomStackWithDecrement:
    def __init__(self, max_size):
```

```
        self.max_size = max_size
        self.items = []

    def push(self, x):
        if len(self.items) < self.max_size:
            self.items.append(x)

    def pop(self):
        if self.items:
            return self.items.pop()
        return -1

    def decrement(self, k, val):
        for i in range(min(k, len(self.items))):
            self.items[i] -= val
```

1669. Merge In Between Linked Lists

Medium · Linked List

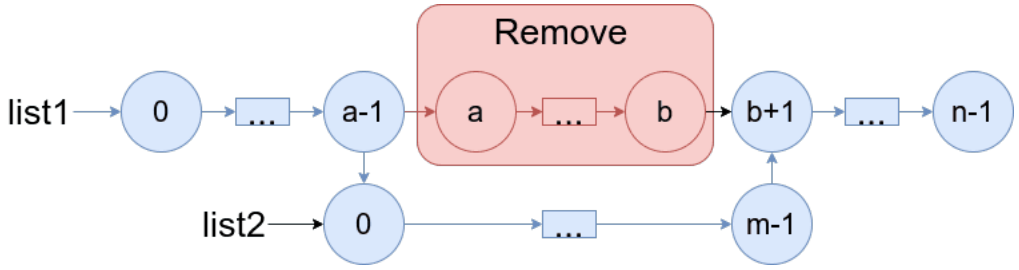
leetcode.com/problems/merge-in-between-linked-lists

PROBLEM

You are given two linked lists: `list1` and `list2` of sizes `n` and `m` respectively.

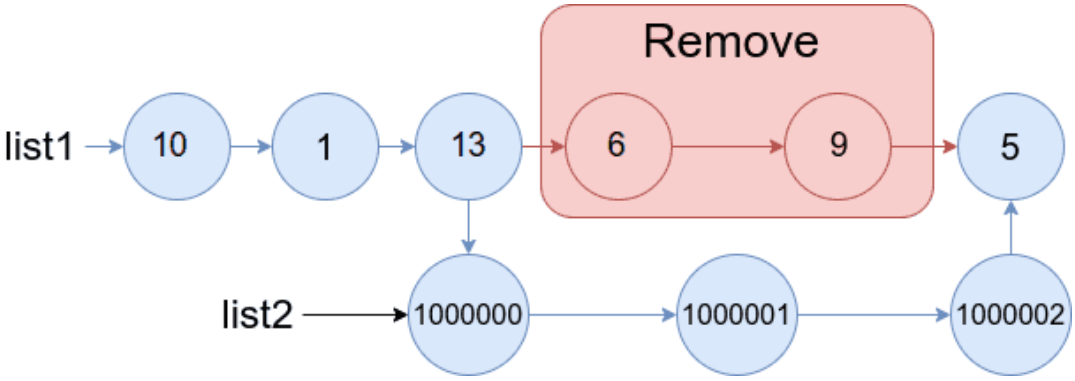
Remove `list1`'s nodes from the a^{th} node to the b^{th} node, and put `list2` in their place.

The blue edges and nodes in the following figure indicate the result:



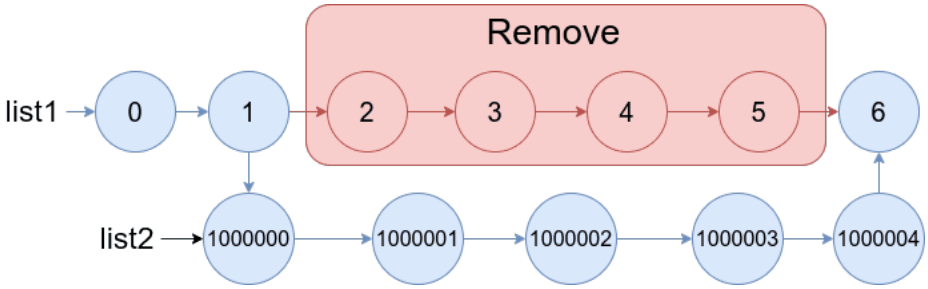
Build the result list and return its head.

Example 1:




```
Input: list1 = [10,1,13,6,9,5], a = 3, b = 4, list2 = [1000000,1000001,1000002]
Output: [10,1,13,1000000,1000001,1000002,5]
Explanation: We remove the nodes 3 and 4 and put the entire list2 in their place. The blue edges and
nodes in the above figure indicate the result.
```

Example 2:




```
Input: list1 = [0,1,2,3,4,5,6], a = 2, b = 5, list2 = [1000000,1000001,1000002,1000003,1000004]
Output: [0,1,1000000,1000001,1000002,1000003,1000004,6]
Explanation: The blue edges and nodes in the above figure indicate the result.
```

Constraints:

- `3 <= list1.length <= 10^4`

- `1 <= a <= b < list1.length - 1`
- `1 <= list2.length <= 10^4`

SOLUTION (PYTHON 3)

Solution 1: Simulation

> **How to think about it**
>
> Replace indices [a,b] of list1 with list2. Find the predecessor of a and node b, link the predecessor to list2's head, then list2's tail to b's successor. Two pointers walk the needed steps, a third scan finds list2's end, and b.next is cleared.

We can directly simulate the operations described in the problem.

In the implementation, we use two pointers p and q, both initially pointing to the head node of `list1`.

Then we move pointers p and q forward, until pointer p points to the node before the a-th node in `list1`, and pointer q points to the b-th node in `list1`. At this point, we set the `next` pointer of p to the head node of `list2`, and set the `next` pointer of the tail node of `list2` to the node pointed to by the `next` pointer of q. This completes the operation required by the problem.

The time complexity is O(m + n), and the space complexity is O(1). Where m and n are the lengths of `list1` and `list2` respectively.

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeInBetween(
        self, list1: ListNode, a: int, b: int, list2: ListNode
    ) -> ListNode:
        p = q = list1
        for _ in range(a - 1):
            p = p.next
        for _ in range(b):
            q = q.next
        p.next = list2
        while p.next:
            p = p.next
        p.next = q.next
        q.next = None
        return list1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Only remove the nodes from position a to b
# 2. Insert a list at a position (nothing removed)


# ============================================================
# FOLLOW-UP 1: Only remove the nodes from position a to b
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 1: function ----
```

```
# Answer: Walk to the node before position a, and link it
# to the node after position b.
# Time: O(b).  Space: O(1).
def remove_range(list1, a, b):
    dummy = ListNode(0, list1)
    before = dummy
    for i in range(a):
        before = before.next
    after = before
    for i in range(b - a + 2):
        after = after.next
    before.next = after
    return dummy.next


# ============================================================
# FOLLOW-UP 2: Insert a list at a position (nothing
# removed)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk to the node before the position, link it to
# the new list, and link the end of the new list to the
# rest.
# Time: O(position + len(list2)).  Space: O(1).
def insert_list_at(list1, position, list2):
    dummy = ListNode(0, list1)
    before = dummy
    for i in range(position):
        before = before.next
    rest = before.next
    before.next = list2
    tail = list2
    while tail.next:
        tail = tail.next
    tail.next = rest
    return dummy.next
```

146. LRU Cache

Medium · Design, Hash Table, Linked List, Doubly-Linked List

leetcode.com/problems/lru-cache

★ **TOP 7 PROBLEM** in Linked List Reversal / Manipulation, Design Problems: master this one first

Also appears in pattern 29 Design Problems

PROBLEM

Design a data structure that follows the constraints of a **Least Recently Used (LRU) cache**.

Implement the `LRUCache` class:

- `LRUCache(int capacity)` Initialize the LRU cache with **positive** size `capacity`.
- `int get(int key)` Return the value of the `key` if the key exists, otherwise return `-1`.
- `void put(int key, int value)` Update the value of the `key` if the `key` exists. Otherwise, add the `key-value` pair to the cache. If the number of keys exceeds the `capacity` from this operation, **evict** the least recently used key.

The functions `get` and `put` must each run in `O(1)` average time complexity.

Example 1:

```
Input
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
Output
[null, null, null, 1, null, -1, null, -1, 3, 4]

Explanation
LRUCache lRUCache = new LRUCache(2);
lRUCache.put(1, 1); // cache is {1=1}
lRUCache.put(2, 2); // cache is {1=1, 2=2}
lRUCache.get(1); // return 1
lRUCache.put(3, 3); // LRU key was 2, evicts key 2, cache is {1=1, 3=3}
lRUCache.get(2); // returns -1 (not found)
lRUCache.put(4, 4); // LRU key was 1, evicts key 1, cache is {4=4, 3=3}
lRUCache.get(1); // return -1 (not found)
lRUCache.get(3); // return 3
lRUCache.get(4); // return 4
```

Constraints:

- `1 <= capacity <= 3000`
- `0 <= key <=` 10^4
- `0 <= value <=` 10^5
- At most `2 * 10^5` calls will be made to `get` and `put`.

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Doubly Linked List

How to think about it

get and put must be O(1), evicting the least recently used key. A hash map finds keys in O(1) but does not keep recency; moving nodes in an array or singly linked list is O(n). Up to 2× 10^5 calls. Map keys to nodes, and keep a doubly linked list with most-recent at the head. On access, detach the node and insert it at the head; when over capacity, drop the tail's predecessor. Lookup and pointer updates are O(1).

We can implement an LRU (Least Recently Used) cache using a "hash table" and a "doubly linked list".

- Hash Table: Used to store the key and its corresponding node location.
- Doubly Linked List: Used to store node data, sorted by access time.

When accessing a node, if the node exists, we delete it from its original position and reinsert it at the head of the list. This ensures that the node stored at the tail of the list is the least recently used node. When the number of nodes exceeds the maximum cache space, we eliminate the node at the tail of the list.

When inserting a node, if the node exists, we delete it from its original position and reinsert it at the head of the list. If it does not exist, we first check if the cache is full. If it is full, we delete the node at the tail of the list and insert the new node at the head of the list.

The time complexity is O(1), and the space complexity is O(capacity).

Python 3

```python
class Node:
    def __init__(self, key: int = 0, val: int = 0):
        self.key = key
        self.val = val
        self.prev = None
        self.next = None


class LRUCache:

    def __init__(self, capacity: int):
        self.size = 0
        self.capacity = capacity
        self.cache = {}
        self.head = Node()
        self.tail = Node()
        self.head.next = self.tail
        self.tail.prev = self.head

    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1
        node = self.cache[key]
        self.remove_node(node)
        self.add_to_head(node)
        return node.val

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            node = self.cache[key]
            self.remove_node(node)
            node.val = value
            self.add_to_head(node)
        else:
            node = Node(key, value)
            self.cache[key] = node
            self.add_to_head(node)
            self.size += 1
            if self.size > self.capacity:
                node = self.tail.prev
                self.cache.pop(node.key)
                self.remove_node(node)
                self.size -= 1

    def remove_node(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def add_to_head(self, node):
        node.next = self.head.next
        node.prev = self.head
        self.head.next = node
        node.next.prev = node


# Your LRUCache object will be instantiated and called as such:
# obj = LRUCache(capacity)
# param_1 = obj.get(key)
# obj.put(key,value)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Entries EXPIRE after a time to live (TTL)
# 2. LFU: evict the LEAST FREQUENTLY used entry (the
# oldest one on ties)


# ============================================================
# FOLLOW-UP 1: Entries EXPIRE after a time to live (TTL)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Store the time of the last write with every
# entry; a get that finds an entry older than the TTL
# treats it as missing and deletes it.
# Time: get and put are each O(1).  Space: O(capacity).
from collections import OrderedDict


class LRUCacheWithTTL:
    def __init__(self, capacity, ttl):
        self.capacity = capacity
        self.ttl = ttl
        self.data = OrderedDict()                    # key -> (value, time written)

    def get(self, key, now):
        if key not in self.data:
            return -1
        value, written = self.data[key]
        if now - written > self.ttl:
            del self.data[key]
            return -1
        self.data.move_to_end(key)
        return value

    def put(self, key, value, now):
        if key in self.data:
            del self.data[key]
        self.data[key] = (value, now)
        if len(self.data) > self.capacity:
            self.data.popitem(last=False)


# ============================================================
# FOLLOW-UP 2: LFU: evict the LEAST FREQUENTLY used entry
# (the oldest one on ties)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep, for every entry, its use count, and for
# every count an ordered list of keys; remember the lowest
# count in use.
# Time: get and put are each O(1).  Space: O(capacity).
from collections import OrderedDict


class LFUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.values = {}
        self.counts = {}
        self.buckets = {}                            # count -> OrderedDict of keys (oldest first)
        self.lowest = 0

    def touch(self, key):
```

```
        count = self.counts[key]
        del self.buckets[count][key]
        if not self.buckets[count]:
            del self.buckets[count]
            if self.lowest == count:
                self.lowest += 1
        self.counts[key] = count + 1
        self.buckets.setdefault(count + 1, OrderedDict())[key] = None

    def get(self, key):
        if key not in self.values:
            return -1
        self.touch(key)
        return self.values[key]

    def put(self, key, value):
        if self.capacity == 0:
            return
        if key in self.values:
            self.values[key] = value
            self.touch(key)
            return
        if len(self.values) >= self.capacity:
            oldest, _ = self.buckets[self.lowest].popitem(last=False)
            if not self.buckets[self.lowest]:
                del self.buckets[self.lowest]
            del self.values[oldest]
            del self.counts[oldest]
        self.values[key] = value
        self.counts[key] = 1
        self.buckets.setdefault(1, OrderedDict())[key] = None
        self.lowest = 1
```

25. Reverse Nodes in k-Group

Hard · Recursion, Linked List

leetcode.com/problems/reverse-nodes-in-k-group

★ **TOP 7 PROBLEM** in Linked List Reversal / Manipulation: master this one first

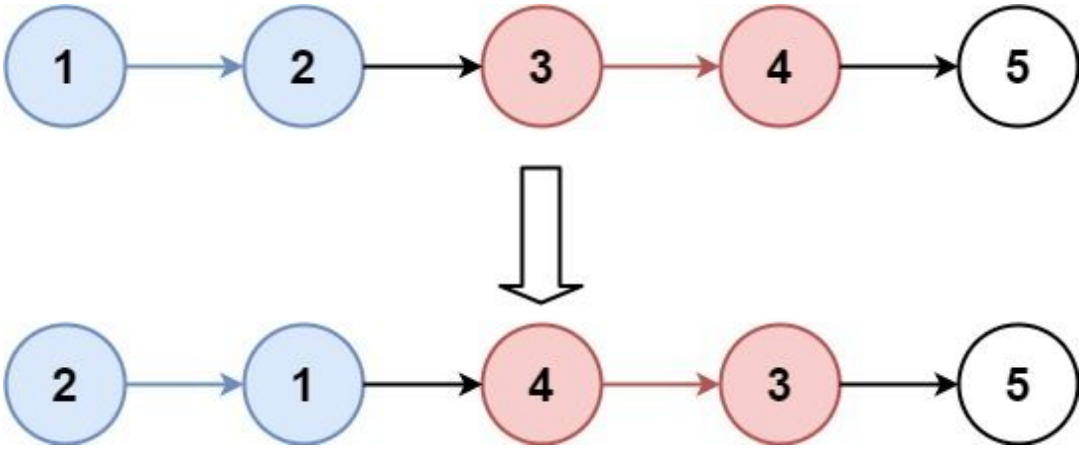
PROBLEM

Given the `head` of a linked list, reverse the nodes of the list `k` at a time, and return *the modified list*.

`k` is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of `k` then left-out nodes, in the end, should remain as it is.

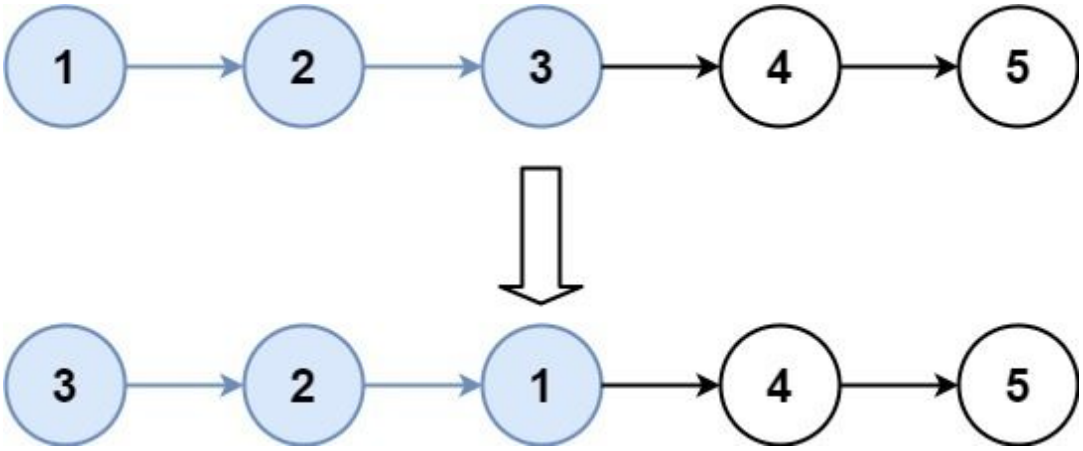
You may not alter the values in the list's nodes, only nodes themselves may be changed.

Example 1:




```
Input: head = [1,2,3,4,5], k = 2
Output: [2,1,4,3,5]
```

Example 2:




```
Input: head = [1,2,3,4,5], k = 3
Output: [3,2,1,4,5]
```

Constraints:

- The number of nodes in the list is `n`.
- `1 <= k <= n <= 5000`
- `0 <= Node.val <= 1000`

Follow-up: Can you solve the problem in `O(1)` extra memory space?

SOLUTION (PYTHON 3)

Solution 1: Simulation

How to think about it

The first idea is recursion by groups: reverse k nodes, then recurse on the rest. n ≤ 5000 would pass, but the follow-up asks for O(1) extra space, and a recursive stack grows with the number of groups. Two places are easy to get wrong: we must confirm a full group of k (otherwise leave the tail alone), and we must splice the reversed segment back without dropping a pointer. Inside a group the work is ordinary list reversal; between groups we only need the previous tail. Start from a dummy, walk k steps to probe, then detach, reverse, and reconnect; stop when a probe falls short. Probe, reverse, and relink are all pointer writes, so extra space is O(1).

We can simulate the entire reversal process according to the problem description.

First, we define a helper function reverse to reverse a linked list. Then, we define a dummy head node dummy and set its next pointer to head.

Next, we traverse the linked list, processing k nodes at a time. If the remaining nodes are fewer than k, we do not perform the reversal. Otherwise, we extract k nodes and call the reverse function to reverse these k nodes. Then, we connect the reversed linked list back to the original linked list. We continue to process the next k nodes until the entire linked list is traversed.

The time complexity is O(n), where n is the length of the linked list. The space complexity is O(1).

Python 3

```python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseKGroup(self, head: Optional[ListNode], k: int) -> Optional[ListNode]:
        def reverse(head: Optional[ListNode]) -> Optional[ListNode]:
            dummy = ListNode()
            cur = head
            while cur:
                nxt = cur.next
                cur.next = dummy.next
                dummy.next = cur
                cur = nxt
            return dummy.next

        dummy = pre = ListNode(next=head)
        while pre:
            cur = pre
            for _ in range(k):
                cur = cur.next
                if cur is None:
                    return dummy.next
            node = pre.next
            nxt = cur.next
            cur.next = None
            pre.next = reverse(node)
            node.next = nxt
            pre = node
        return dummy.next
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Also reverse the LAST group when it has fewer than k
# nodes
# 2. Reverse only every SECOND group of k nodes


# ============================================================
# FOLLOW-UP 1: Also reverse the LAST group when it has
# fewer than k nodes
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 1: function ----
# Answer: Reverse a group when it has k nodes, and also
# when the list ends early, in which case the group is
# simply whatever is left.
# Time: O(n).  Space: O(1).
def reverse_all_groups(head, k):
    dummy = ListNode(0, head)
    group_previous = dummy
    while group_previous.next:
        first = group_previous.next
        last = group_previous
        for i in range(k):
            if last.next is None:
                break
            last = last.next
        following = last.next
        previous, node = following, first
        while node is not following:
            nxt = node.next
            node.next = previous
            previous = node
            node = nxt
        group_previous.next = last
        group_previous = first
    return dummy.next


# ============================================================
# FOLLOW-UP 2: Reverse only every SECOND group of k nodes
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk group by group and reverse a group only
# when its number is even (counting from 1).
# Time: O(n).  Space: O(1).
def reverse_alternate_groups(head, k):
    dummy = ListNode(0, head)
    group_previous = dummy
    group_number = 0
    while group_previous.next:
        group_number += 1
        first = group_previous.next
        last = group_previous
```

```
        steps = 0
        while steps < k and last.next:
            last = last.next
            steps += 1
```

```
        if steps < k:
            break
        following = last.next
        if group_number % 2 == 0:
            previous, node = following, first
            while node is not following:
                nxt = node.next
                node.next = previous
                previous = node
                node = nxt
            group_previous.next = last
            group_previous = first
        else:
            group_previous = last
    return dummy.next
```

23. Merge k Sorted Lists

Hard · Linked List, Divide and Conquer, Heap (Priority Queue), Merge Sort, Tournament Sort

leetcode.com/problems/merge-k-sorted-lists

★ **TOP 7 PROBLEM** in Linked List Reversal / Manipulation, K-way Merge: master this one first

Also appears in pattern 38 K-way Merge

PROBLEM

You are given an array of `k` linked-lists `lists`, each linked-list is sorted in ascending order.

Merge all the linked-lists into one sorted linked-list and return it.

Example 1:

```
Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: [1,1,2,3,4,4,5,6]
Explanation: The linked-lists are:
[
  1->4->5,
  1->3->4,
  2->6
]
merging them into one sorted linked list:
1->1->2->3->4->4->5->6
```

Example 2:

```
Input: lists = []
Output: []
```

Example 3:

```
Input: lists = [[]]
Output: []
```

Constraints:

- `k == lists.length`
- `0 <= k <= 10^4`
- `0 <= lists[i].length <= 500`
- `-10^4 <= lists[i][j] <= 10^4`
- `lists[i]` is sorted in **ascending order**.
- The sum of `lists[i].length` will not exceed `10^4`.

SOLUTION (PYTHON 3)

Solution 1: Priority Queue (Min Heap)

How to think about it

The first idea is pairwise merge: merge list 1 with 2, then fold in 3, and so on. With $N \le 10^4$ nodes and $k \le 10^4$ lists, the worst case is O(Nk) and can time out. Dumping every node into an array and sorting also works, but ignores that each list is already sorted. The bottleneck is finding the smallest among k heads at every step. A linear scan is O(k) per node and does not fit. At any moment the next node is just the minimum of the current heads. A min-heap of those k heads pops the smallest and pushes its successor. Each node enters and leaves the heap once, so the time is O(Nlog k) and the heap holds at most k pointers.

We can create a min heap pq to maintain the head nodes of all linked lists. Each time, we take out the node with the smallest value from the min heap, add it to the end of the result linked list, and then add the next node of this node to the heap. Repeat the above steps until the heap is empty.

The time complexity is O(n × log k), and the space complexity is O(k). Here, n is the total number of all linked list nodes, and k is the number of linked lists given in the problem.

Python 3

```python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        setattr(ListNode, "__lt__", lambda a, b: a.val < b.val)
        pq = [head for head in lists if head]
        heapify(pq)
        dummy = cur = ListNode()
        while pq:
            node = heappop(pq)
            if node.next:
                heappush(pq, node.next)
            cur.next = node
            cur = cur.next
        return dummy.next
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Divide and conquer: merge the lists in pairs
# 2. Only the k smallest values of all the lists


# ============================================================
# FOLLOW-UP 1: Divide and conquer: merge the lists in
# pairs
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 1: function ----
# Answer: Merge list 1 with list 2, list 3 with list 4,
# and so on; repeat on the results: O(N log k) without a
# heap.
# Time: O(N log k), N = total nodes.  Space: O(1) extra
# besides the list of heads.
def merge_two(a, b):
    dummy = ListNode()
    tail = dummy
    while a and b:
        if a.val <= b.val:
            tail.next = a
            a = a.next
        else:
            tail.next = b
            b = b.next
        tail = tail.next
    tail.next = a if a else b
    return dummy.next
```

```
def merge_k_divide_and_conquer(lists):
    lists = [x for x in lists if x]
    if not lists:
        return None
    while len(lists) > 1:
        merged = []
        for i in range(0, len(lists), 2):
            if i + 1 < len(lists):
                merged.append(merge_two(lists[i], lists[i + 1]))
            else:
                merged.append(lists[i])
        lists = merged
    return lists[0]


# ============================================================
# FOLLOW-UP 2: Only the k smallest values of all the lists
# ============================================================

# ---- follow-up 2: function ----
# Answer: Use the heap of list heads and stop after k
# pops.
# Time: O(k + m log k), m = lists.  Space: O(m).
```

```
import heapq


def smallest_values(lists, k):
    heap = []
    for i in range(len(lists)):
        if lists[i]:
            heapq.heappush(heap, (lists[i].val, i))
    pointers = list(lists)
    result = []
    while heap and len(result) < k:
        value, i = heapq.heappop(heap)
        result.append(value)
        pointers[i] = pointers[i].next
        if pointers[i]:
            heapq.heappush(heap, (pointers[i].val, i))
    return result
```

PATTERN 26 OF 41

Binary Search Tree (BST)

20 problems · 8 Easy · 12 Medium · 0 Hard

THE IDEA

Left < node < right at every node, so search is O(height) and an inorder walk is sorted. Pass allowed (low, high) bounds down to validate.

Signals: validate a BST, kth smallest, lowest common ancestor in a BST, insert or delete, convert a sorted array, two sum in a BST.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
def valid(node, low, high):
    if node is None:
        return True
    if not (low < node.val < high):
        return False
    return valid(node.left, low, node.val) and valid(node.right, node.val, high)
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Search in a Binary Search Tree	Easy	LC 700	1393
2	Lowest Common Ancestor of a Binary Search Tree	Easy	LC 235	1396
3	Convert Sorted Array to Binary Search Tree (see page)	Easy	LC 108	725
4	Minimum Absolute Difference in BST	Easy	LC 530	1400
5	Range Sum of BST	Easy	LC 938	1404
6	Two Sum IV - Input is a BST	Easy	LC 653	1407
7	Find Mode in Binary Search Tree	Easy	LC 501	1412
8	Closest Binary Search Tree Value	Easy	LC 270	1416
9	★ **Validate Binary Search Tree**	Medium	LC 98	1419
10	Insert into a Binary Search Tree	Medium	LC 701	1422
11	★ **Delete Node in a BST**	Medium	LC 450	1425
12	★ **Kth Smallest Element in a BST**	Medium	LC 230	1428
13	★ **Convert BST to Greater Tree**	Medium	LC 538	1432
14	Balance a Binary Search Tree	Medium	LC 1382	1437
15	★ **Trim a Binary Search Tree**	Medium	LC 669	1441
16	★ **Recover Binary Search Tree**	Medium	LC 99	1445
17	Unique Binary Search Trees	Medium	LC 96	1449
18	Unique Binary Search Trees II (see page)	Medium	LC 95	742
19	★ **Binary Search Tree Iterator**	Medium	LC 173	1451
20	Construct Binary Search Tree from Preorder Traversal	Medium	LC 1008	1455

700. Search in a Binary Search Tree

Easy · Tree, Binary Search Tree, Binary Tree

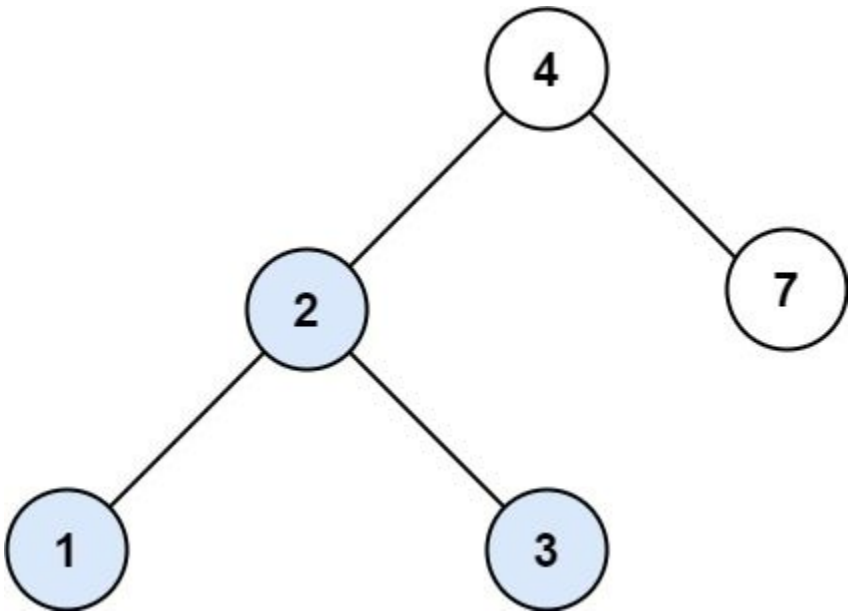
leetcode.com/problems/search-in-a-binary-search-tree

PROBLEM

You are given the `root` of a binary search tree (BST) and an integer `val`.

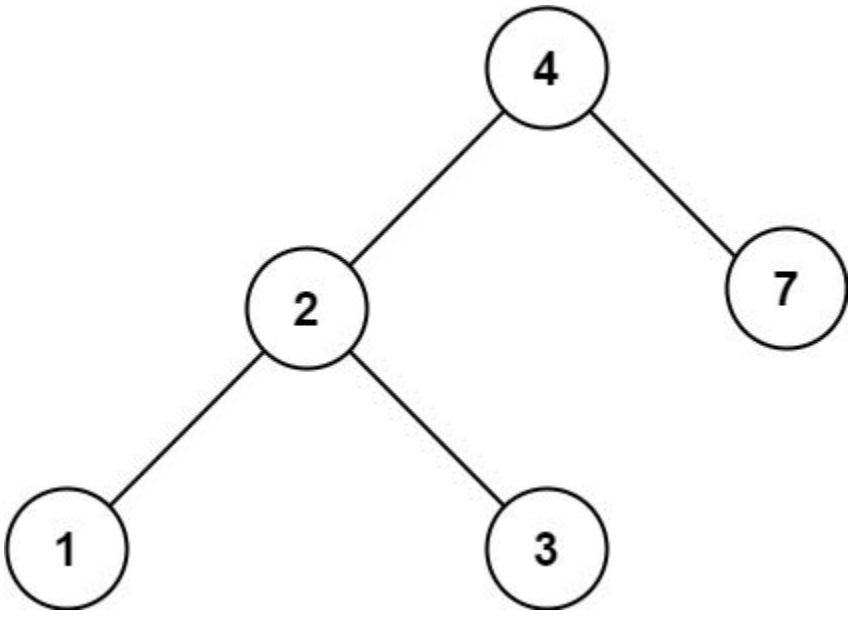
Find the node in the BST that the node's value equals `val` and return the subtree rooted with that node. If such a node does not exist, return `null`.

Example 1:




```
Input: root = [4,2,7,1,3], val = 2
Output: [2,1,3]
```

Example 2:



```
Input: root = [4,2,7,1,3], val = 5
Output: []
```

Constraints:

- The number of nodes in the tree is in the range `[1, 5000]`.
- `1 <= Node.val <= 10^7`
- `root` is a binary search tree.
- `1 <= val <= 10^7`

SOLUTION (PYTHON 3)

Solution 1: Recursion

How to think about it

We need the subtree rooted at the node whose value is val. n ≤ 5000 would allow a full scan, but that ignores the BST order. If the current value equals the target, that node is the answer; if it is larger, the target can only lie in the left subtree, otherwise only in the right. One comparison therefore discards an entire side. Recursion matches this walk: return the current node when it is empty or a hit, otherwise descend left or right. A degenerate chain still costs O(n) time and stack space.

We check if the current node is null or if the current node's value equals the target value. If so, we return the current node.

Otherwise, if the current node's value is greater than the target value, we recursively search the left subtree; otherwise, we recursively search the right subtree.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def searchBST(self, root: Optional[TreeNode], val: int) -> Optional[TreeNode]:
        if root is None or root.val == val:
            return root
        return (
            self.searchBST(root.left, val)
            if root.val > val
            else self.searchBST(root.right, val)
        )
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Search with a loop (no recursion)
# 2. Return the path of values from the root to the value


# ============================================================
# FOLLOW-UP 1: Search with a loop (no recursion)
# ============================================================

# ---- follow-up 1: function ----
# Answer: At every node go left when the value is smaller
# and right when it is bigger.
# Time: O(h).  Space: O(1).
def search_bst_iterative(root, val):
```

```
    node = root
    while node and node.val != val:
        if val < node.val:
            node = node.left
        else:
            node = node.right
    return node


# ============================================================
# FOLLOW-UP 2: Return the path of values from the root to
# the value
# ============================================================

# ---- follow-up 2: function ----
# Answer: Record the values you pass while searching; the
# path is empty if the value is missing.
# Time: O(h).  Space: O(h).
def path_to_value(root, val):
    path = []
    node = root
    while node:
        path.append(node.val)
        if node.val == val:
            return path
        node = node.left if val < node.val else node.right
    return []
```

235. Lowest Common Ancestor of a Binary Search Tree

Medium · Tree, Depth-First Search, Binary Search Tree, Binary Tree, Lowest Common Ancestor, Binary Lifting

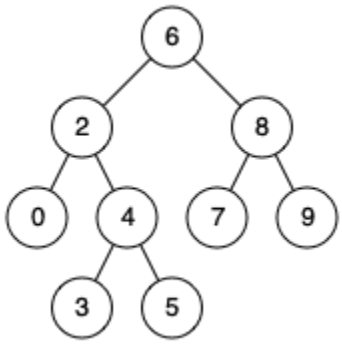
leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree

PROBLEM

Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

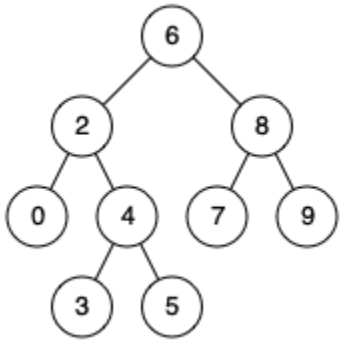
According to the definition of LCA on Wikipedia: "The lowest common ancestor is defined between two nodes `p` and `q` as the lowest node in `T` that has both `p` and `q` as descendants (where we allow **a node to be a descendant of itself**)."

Example 1:




```
Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8
Output: 6
Explanation: The LCA of nodes 2 and 8 is 6.
```

Example 2:




```
Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4
Output: 2
Explanation: The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself according to the
LCA definition.
```

Example 3:

```
Input: root = [2,1], p = 2, q = 1
Output: 2
```

Constraints:

- The number of nodes in the tree is in the range `[2, 10^5]`.
- `-10^9 <= Node.val <= 10^9`
- All `Node.val` are **unique**.

- `p != q`
- `p` and `q` will exist in the BST.

SOLUTIONS (PYTHON 3)

Solution 1: Iteration

> **How to think about it**
> In a BST the LCA of p and q is the fork between their values. Walk from the root: go left if both are smaller, right if both are larger, otherwise the current node is the fork. The walk is iterative and needs no call stack.

Starting from the root node, we traverse the tree. If the current node's value is less than both p and q values, it means that p and q should be in the right subtree of the current node, so we move to the right child. If the current node's value is greater than both p and q values, it means that p and q should be in the left subtree, so we move to the left child. Otherwise, it means the current node is the lowest common ancestor of p and q, so we return the current node.

The time complexity is O(n), where n is the number of nodes in the binary search tree. The space complexity is O(1).

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None


class Solution:
    def lowestCommonAncestor(
        self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode'
    ) -> 'TreeNode':
        while 1:
            if root.val < min(p.val, q.val):
                root = root.right
            elif root.val > max(p.val, q.val):
                root = root.left
            else:
                return root
```

Solution 2: Recursion

> **How to think about it**
> The same fork rule is a recursion: recurse into the side that contains both, otherwise return the current root. The idea matches the iteration with a shorter implementation.

We can also use a recursive approach to solve this problem.

We first check if the current node's value is less than both p and q values. If it is, we recursively traverse the right subtree. If the current node's value is greater than both p and q values, we recursively traverse the left subtree. Otherwise, it means the current node is the lowest common ancestor of p and q, so we return the current node.

The time complexity is O(n), and the space complexity is O(n). Where n is the number of nodes in the binary search tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None


class Solution:
    def lowestCommonAncestor(
        self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode'
    ) -> 'TreeNode':
        if root.val < min(p.val, q.val):
            return self.lowestCommonAncestor(root.right, p, q)
        if root.val > max(p.val, q.val):
            return self.lowestCommonAncestor(root.left, p, q)
        return root
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Iterative: O(1) extra space
# 2. The nodes may be missing from the tree: return None
# unless both exist


# ============================================================
# FOLLOW-UP 1: Iterative: O(1) extra space
# ============================================================

# ---- follow-up 1: function ----
# Answer: Walk down from the root: if both values are
# smaller go left, if both bigger go right, otherwise this
# node is the answer.
# Time: O(h).  Space: O(1).
def lca_bst_iterative(root, p, q):
    node = root
    while node:
        if p < node.val and q < node.val:
            node = node.left
        elif p > node.val and q > node.val:
            node = node.right
        else:
            return node
    return None


# ============================================================
# FOLLOW-UP 2: The nodes may be missing from the tree:
# return None unless both exist
# ============================================================

# ---- follow-up 2: function ----
# Answer: Find the LCA as above, then check that both
# values really are in the tree (search for each).
# Time: O(h).  Space: O(1).
def contains(root, val):
    node = root
    while node:
        if node.val == val:
            return True
        node = node.left if val < node.val else node.right
    return False
```

```
def lca_bst_if_both_exist(root, p, q):
    if not contains(root, p) or not contains(root, q):
        return None
    node = root
    while node:
        if p < node.val and q < node.val:
            node = node.left
        elif p > node.val and q > node.val:
            node = node.right
        else:
            return node
    return None
```

530. Minimum Absolute Difference in BST

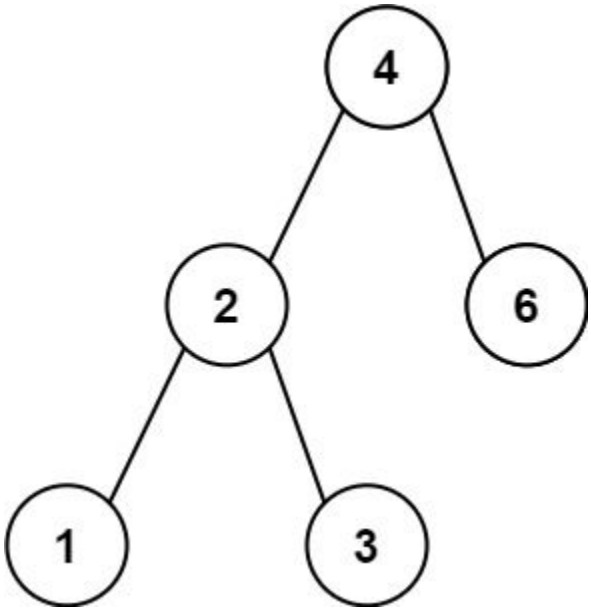
Easy · Tree, Depth-First Search, Breadth-First Search, Binary Search Tree, Binary Tree

leetcode.com/problems/minimum-absolute-difference-in-bst

PROBLEM

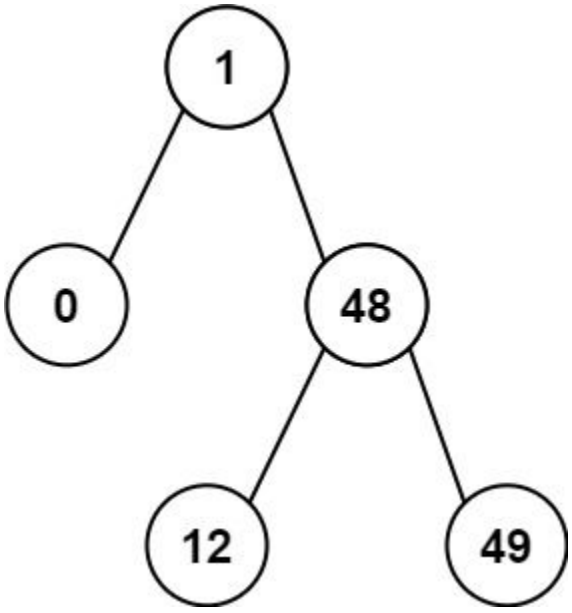
Given the `root` of a Binary Search Tree (BST), return *the minimum absolute difference between the values of any two different nodes in the tree*.

Example 1:




```
Input: root = [4,2,6,1,3]
Output: 1
```

Example 2:




```
Input: root = [1,0,48,null,null,12,49]
Output: 1
```

Constraints:

- The number of nodes in the tree is in the range `[2, 10^4]`.
- `0 <= Node.val <= 10^5`

Note: This question is the same as 783: https://leetcode.com/problems/minimum-distance-between-bst-nodes/

SOLUTIONS (PYTHON 3)

Solution 1: Inorder Traversal

How to think about it

The minimum pairwise difference on a general tree needs all pairs. In a BST, inorder is sorted, so the minimum lies between adjacent values. Walk inorder, keep the predecessor, and update with `val - pre`. Seed `pre` at -∞ so the first node is skipped. One inorder pass suffices.

The problem requires us to find the minimum difference between the values of any two nodes. Since the inorder traversal of a binary search tree is an increasing sequence, we only need to find the minimum difference between the values of two adjacent nodes in the inorder traversal.

We can use a recursive method to implement the inorder traversal. During the process, we use a variable pre to save the value of the previous node. This way, we can calculate the minimum difference between the values of two adjacent nodes during the traversal.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary search tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def getMinimumDifference(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]):
            if root is None:
                return
            dfs(root.left)
            nonlocal pre, ans
            ans = min(ans, root.val - pre)
            pre = root.val
            dfs(root.right)

        pre = -inf
        ans = inf
        dfs(root)
        return ans
```

Solution 2: Explicit-Stack Inorder

How to think about it

The minimum pairwise difference on a general tree needs all pairs. In a BST, inorder is sorted, so the minimum lies between adjacent values. Recursing inorder and subtracting the predecessor is correct on a short tree. The tree can contain 10^4 nodes. A left chain makes this walk recurse once per node and overflow the call stack. Each node only needs the previous value. The walk does not need a return value, as long as the left subtree is finished before the node and the right subtree comes after. An explicit stack performs that inorder walk. On entry it pushes an exit marker and the left child; on exit it updates the answer with the current value minus the predecessor and pushes the right child. The predecessor starts at -∞, so the first node does not become the answer.

The minimum difference between any two node values of a binary search tree is the minimum gap between adjacent values in its increasing inorder sequence.

An explicit stack walks the tree in that order, and pre stores the previous value, initially -∞. On entry we push an exit marker and the left child. On exit we update the answer with the current value minus pre, store the current value in pre, and push the right child.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary search tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def getMinimumDifference(self, root: Optional[TreeNode]) -> int:
        pre = -inf
        ans = inf
        stk = [(root, 0)]
        while stk:
            node, state = stk.pop()
            if node is None:
                continue
            if state == 0:
                stk.append((node, 1))
                stk.append((node.left, 0))
                continue
            ans = min(ans, node.val - pre)
            pre = node.val
            stk.append((node.right, 0))
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the pair of values that gives the smallest
# difference
# 2. The same question for a general binary tree (not a
# BST)


# ========================================================
# FOLLOW-UP 1: Return the pair of values that gives the
# smallest difference
# ========================================================

# ---- follow-up 1: function ----
# Answer: An inorder walk gives the values in order; the
# smallest difference is between two neighbours, so
# remember the best neighbouring pair.
# Time: O(n).  Space: O(h).
def closest_pair_in_bst(root):
    best = [None, None, None]                       # difference, smaller, bigger
    previous = [None]

    def walk(node):
        if node is None:
            return
        walk(node.left)
        if previous[0] is not None:
            difference = node.val - previous[0]
            if best[0] is None or difference < best[0]:
                best[0], best[1], best[2] = difference, previous[0], node.val
        previous[0] = node.val
```

```
        walk(node.right)

    walk(root)
    return best[1], best[2]


# ============================================================
# FOLLOW-UP 2: The same question for a general binary tree
# (not a BST)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Collect all the values, sort them and compare
# neighbours.
# Time: O(n log n).  Space: O(n).
def min_difference_any_tree(root):
    values = []
    stack = [root] if root else []
    while stack:
        node = stack.pop()
        values.append(node.val)
        if node.left:
            stack.append(node.left)
        if node.right:
            stack.append(node.right)
    values.sort()
    best = None
    for i in range(1, len(values)):
        difference = values[i] - values[i - 1]
        if best is None or difference < best:
```

```
            best = difference
    return best
```

938. Range Sum of BST

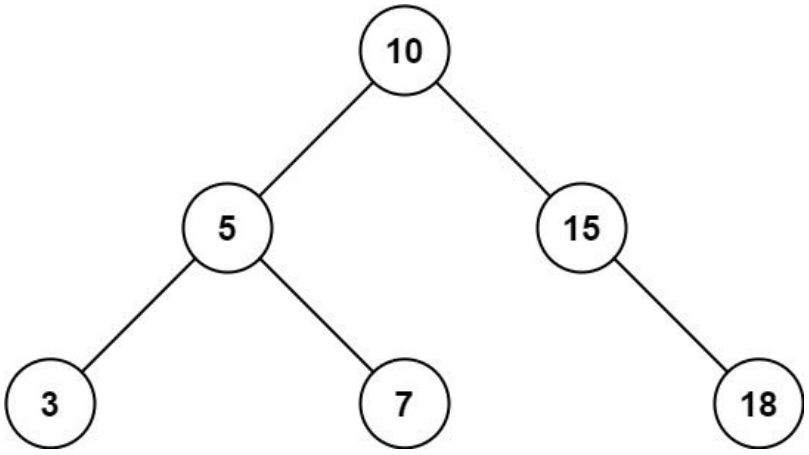
Easy · Tree, Depth-First Search, Binary Search Tree, Binary Tree

leetcode.com/problems/range-sum-of-bst

PROBLEM

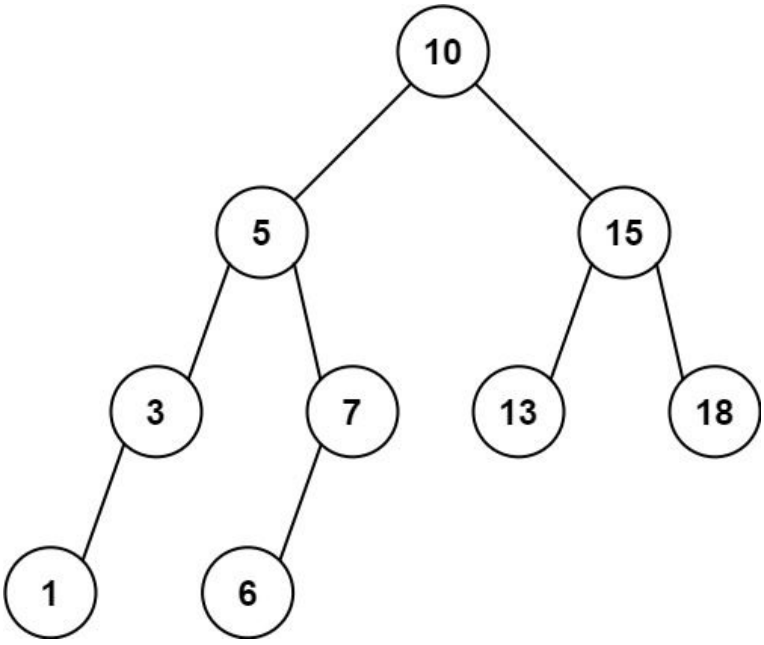
Given the `root` node of a binary search tree and two integers `low` and `high`, return *the sum of values of all nodes with a value in the* ***inclusive*** *range* `[low, high]`.

Example 1:




```
Input: root = [10,5,15,3,7,null,18], low = 7, high = 15
Output: 32
Explanation: Nodes 7, 10, and 15 are in the range [7, 15]. 7 + 10 + 15 = 32.
```

Example 2:




```
Input: root = [10,5,15,3,7,13,18,1,null,6], low = 6, high = 10
Output: 23
Explanation: Nodes 6, 7, and 10 are in the range [6, 10]. 6 + 7 + 10 = 23.
```

Constraints:

- The number of nodes in the tree is in the range `[1, 2 * 10^4]`.
- `1 <= Node.val <= 10^5`
- `1 <= low <= high <= 10^5`
- All `Node.val` are **unique**.

SOLUTION (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> Sum BST nodes whose values lie in [low,high]. A full traversal works, but the search-tree order prunes: skip the left subtree when the current value is already ≤ low, and skip the right when it is already ≥ high. Recurse and add nodes that fall inside the interval.

We design a function dfs(root), which represents the sum of the values of all nodes in the subtree with root as the root, and the values are within the range [low, high]. The answer is dfs(root).

The execution logic of the function dfs(root) is as follows:

- If root is null, return 0.
- If the value x of root is within the range [low, high], then the initial answer of the function dfs(root) is x, otherwise it is 0.
- If x > low, it means that there may be nodes in the left subtree of root with values within the range [low, high], so we need to recursively call dfs(root.left) and add the result to the answer.
- If x < high, it means that there may be nodes in the right subtree of root with values within the range [low, high], so we need to recursively call dfs(root.right) and add the result to the answer.
- Finally, return the answer.

The time complexity is O(n), and the space complexity is O(n). Where n is the number of nodes in the binary search tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rangeSumBST(self, root: Optional[TreeNode], low: int, high: int) -> int:
        def dfs(root: Optional[TreeNode]) -> int:
            if root is None:
                return 0
            x = root.val
            ans = x if low <= x <= high else 0
            if x > low:
                ans += dfs(root.left)
            if x < high:
                ans += dfs(root.right)
            return ans

        return dfs(root)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Count the nodes whose value is in the range
# 2. Return the values in the range in sorted order


# ============================================================
# FOLLOW-UP 1: Count the nodes whose value is in the range
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same pruned walk: skip the left subtree when
# the value is too small, skip the right subtree when it
# is too big.
# Time: O(n).  Space: O(h).
def count_in_range(root, low, high):
    if root is None:
        return 0
    if root.val < low:
        return count_in_range(root.right, low, high)
    if root.val > high:
        return count_in_range(root.left, low, high)
    return 1 + count_in_range(root.left, low, high) + count_in_range(root.right, low, high)


# ============================================================
# FOLLOW-UP 2: Return the values in the range in sorted
# order
# ============================================================

# ---- follow-up 2: function ----
# Answer: An inorder walk that skips the parts of the tree
# that cannot contain values in the range.
# Time: O(k + h), k = values in range.  Space: O(h + k).
def values_in_range(root, low, high):
    result = []

    def walk(node):
        if node is None:
            return
        if node.val > low:
            walk(node.left)
        if low <= node.val <= high:
            result.append(node.val)
        if node.val < high:
            walk(node.right)

    walk(root)
    return result
```

653. Two Sum IV - Input is a BST

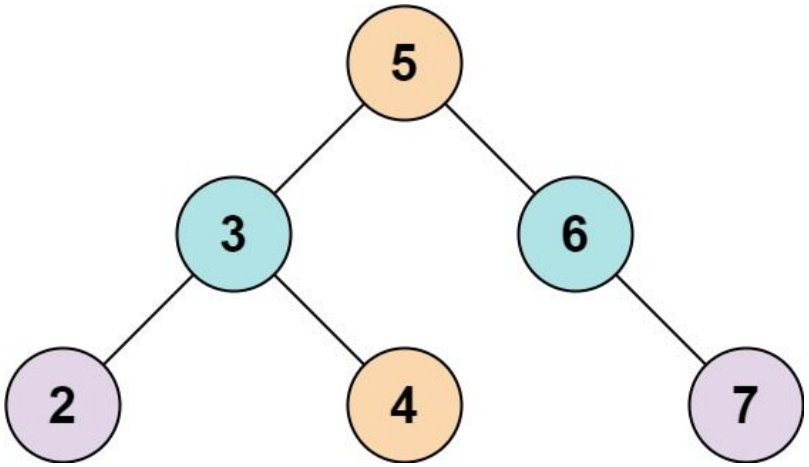
Easy · Tree, Depth-First Search, Breadth-First Search, Binary Search Tree, Hash Table, Two Pointers, Binary Tree

leetcode.com/problems/two-sum-iv-input-is-a-bst

PROBLEM

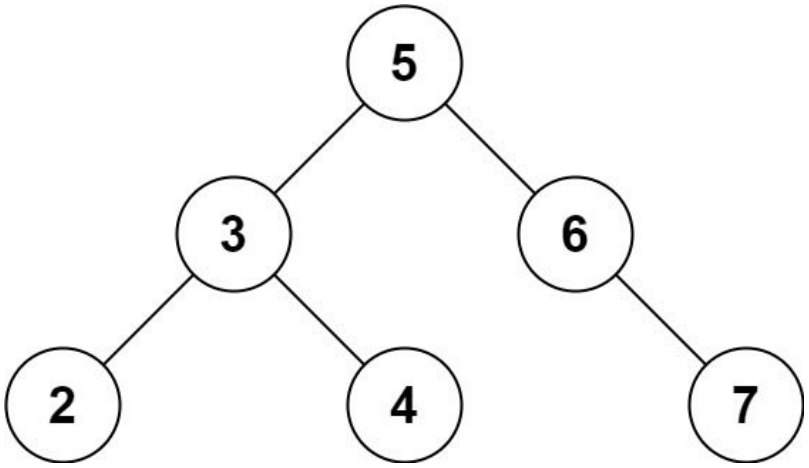
Given the `root` of a binary search tree and an integer `k`, return `true` *if there exist two elements in the BST such that their sum is equal to* `k`, *or* `false` *otherwise*.

Example 1:




```
Input: root = [5,3,6,2,4,null,7], k = 9
Output: true
```

Example 2:




```
Input: root = [5,3,6,2,4,null,7], k = 28
Output: false
```

Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.
- $-10^4 <= Node.val <= 10^4$
- `root` is guaranteed to be a **valid** binary search tree.
- $-10^5 <= k <= 10^5$

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Find two nodes summing to k. Inorder plus two pointers works after collecting values. A set of seen values during a walk is enough: if k-val is present, return true. The BST order is not required.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findTarget(self, root: Optional[TreeNode], k: int) -> bool:
        def dfs(root):
            if root is None:
                return False
            if k - root.val in vis:
                return True
            vis.add(root.val)
            return dfs(root.left) or dfs(root.right)

        vis = set()
        return dfs(root)
```

Complexity. Time: O(n). Space: O(n).

Solution 2

How to think about it

DFS can be replaced by BFS with the same seen-set test; only the traversal order changes.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findTarget(self, root: Optional[TreeNode], k: int) -> bool:
        q = deque([root])
        vis = set()
        while q:
            for _ in range(len(q)):
                node = q.popleft()
                if k - node.val in vis:
                    return True
                vis.add(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
        return False
```

Complexity. Time: O(n). Space: O(n).

Solution 3: Hash Set and Explicit Stack

How to think about it

Find two nodes whose values sum to k. Inorder plus two pointers works after collecting every value. A set of values already seen is enough: if k minus the current value is present, return true. On a short tree one depth-first walk does this, and the BST order is not required. The tree can contain 10^4 nodes. A left chain makes this walk recurse once per node and overflow the call stack. Each node depends only on values already visited. The children do not need to return before the node is checked. An explicit stack keeps that depth-first order. Pop a node, test k minus its value, then store the value and push the right child and the left child. The set belongs to this call only.

A hash set stores values already visited, and an explicit stack walks the tree depth-first. When a node is popped, return true if k minus its value is already in the set. Otherwise store the value and push the non-null right child and left child.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary search tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findTarget(self, root: Optional[TreeNode], k: int) -> bool:
        vis = set()
        stk = []
        if root is not None:
            stk.append(root)
        while stk:
            node = stk.pop()
            if k - node.val in vis:
                return True
            vis.add(node.val)
            if node.right is not None:
                stk.append(node.right)
            if node.left is not None:
                stk.append(node.left)
        return False
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the two values
# 2. O(h) extra space: two iterators, one walking up and
# one walking down


# ============================================================
# FOLLOW-UP 1: Return the two values
# ============================================================

# ---- follow-up 1: function ----
# Answer: Take the sorted values of the BST and use two
# pointers from both ends.
# Time: O(n).  Space: O(n).
def two_sum_bst_pair(root, k):
    values = []

    def walk(node):
        if node is None:
            return
        walk(node.left)
        values.append(node.val)
        walk(node.right)
```

```
    walk(root)
    left, right = 0, len(values) - 1
    while left < right:
        total = values[left] + values[right]
        if total == k:
            return values[left], values[right]
        if total < k:
            left += 1
        else:
            right -= 1
    return None


# ============================================================
# FOLLOW-UP 2: O(h) extra space: two iterators, one
# walking up and one walking down
# ============================================================

# ---- follow-up 2: function ----
# Answer: A forward iterator over the smallest values and
# a backward iterator over the biggest values replace the
# sorted array; each uses a stack of at most the height of
# the tree.
# Time: O(n).  Space: O(h).
def two_sum_bst_two_stacks(root, k):
    forward = []
    backward = []
    node = root
    while node:
        forward.append(node)
        node = node.left
    node = root
    while node:
        backward.append(node)
        node = node.right
```

```
    def next_forward():
        node = forward.pop()
        value = node.val
        node = node.right
        while node:
            forward.append(node)
            node = node.left
        return value

    def next_backward():
        node = backward.pop()
        value = node.val
        node = node.left
        while node:
            backward.append(node)
            node = node.right
        return value

    low = next_forward()
    high = next_backward()
    while low < high:
        total = low + high
        if total == k:
            return True
        if total < k:
            if not forward:
                return False
            low = next_forward()
        else:
            if not backward:
                return False
            high = next_backward()
```

```
    return False
```

501. Find Mode in Binary Search Tree

Easy · Tree, Depth-First Search, Binary Search Tree, Binary Tree

leetcode.com/problems/find-mode-in-binary-search-tree

PROBLEM

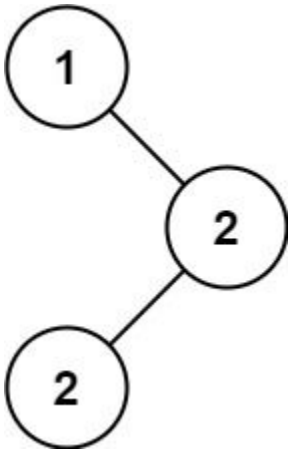
Given the `root` of a binary search tree (BST) with duplicates, return *all the mode(s) (i.e., the most frequently occurred element) in it*.

If the tree has more than one mode, return them in **any order**.

Assume a BST is defined as follows:

- The left subtree of a node contains only nodes with keys **less than or equal to** the node's key.
- The right subtree of a node contains only nodes with keys **greater than or equal to** the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:




```
Input: root = [1,null,2,2]
Output: [2]
```

Example 2:

```
Input: root = [0]
Output: [0]
```

Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.
- $-10^5 <= Node.val <= 10^5$

Follow up:

Could you do that without using any extra space? (Assume that the implicit stack space incurred due to recursion does not count).

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

The mode is the value with the highest frequency. Hashing every node is O(n) time and space and fits $n \le 10^4$, but ignores that the tree is a BST. Inorder yields a non-decreasing sequence, so equal values are adjacent. Track the predecessor, the current run length, and the best frequency: replace the answer when the run grows, append when it ties. One inorder pass collects every mode with O(h) extra space.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findMode(self, root: TreeNode) -> List[int]:
        def dfs(root):
            if root is None:
                return
            nonlocal mx, prev, ans, cnt
            dfs(root.left)
            cnt = cnt + 1 if prev == root.val else 1
            if cnt > mx:
                ans = [root.val]
                mx = cnt
            elif cnt == mx:
                ans.append(root.val)
            prev = root.val
            dfs(root.right)

        prev = None
        mx = cnt = 0
        ans = []
        dfs(root)
        return ans
```

Solution 2: Explicit-Stack Inorder

How to think about it

The mode is the value with the highest frequency. Hashing every node fits $n \le 10^4$ in time, but ignores that the tree is a BST. Inorder visits the left child first. A chain of 10^4 nodes recurses once per node and overflows the call stack. Inorder is non-decreasing, so equal values are adjacent. The predecessor, the current run length, and the best frequency are enough; the walk does not need a return value. An explicit stack performs the inorder walk. On entry it pushes an exit marker and the left child; on exit the run length updates the answer, and then the right child is pushed. A longer run replaces the answer, and a tie appends the value.

An inorder walk of a binary search tree is non-decreasing, so equal values are adjacent. An explicit stack carries that walk together with the previous value, the current run length, and the best frequency. On entry we push an exit marker and the left child. On exit the run grows by one when the value matches its predecessor and otherwise restarts at 1. A longer run replaces the answer, a tie appends the value, and then the right child is pushed.

The time complexity is O(n) and the space complexity is O(n), where n is the number of nodes in the binary search tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findMode(self, root: TreeNode) -> List[int]:
        prev = None
        mx = cnt = 0
        ans = []
        stk = [(root, 0)]
        while stk:
            node, state = stk.pop()
            if node is None:
                continue
            if state == 0:
                stk.append((node, 1))
                stk.append((node.left, 0))
                continue
            cnt = cnt + 1 if prev == node.val else 1
            if cnt > mx:
                ans = [node.val]
                mx = cnt
            elif cnt == mx:
                ans.append(node.val)
            prev = node.val
            stk.append((node.right, 0))
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The tree is a general binary tree: count with a
# dictionary
# 2. The k most frequent values of the BST (by inorder
# runs, no dictionary)


# ============================================================
# FOLLOW-UP 1: The tree is a general binary tree: count
# with a dictionary
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count every value in a dictionary and report the
# values that have the highest count.
# Time: O(n).  Space: O(n).
def modes_any_tree(root):
    count = {}
    stack = [root] if root else []
    while stack:
        node = stack.pop()
        count[node.val] = count.get(node.val, 0) + 1
        if node.left:
            stack.append(node.left)
        if node.right:
            stack.append(node.right)
    best = 0
    for value in count:
        if count[value] > best:
            best = count[value]
    return sorted(v for v in count if count[v] == best)
```

```
# ============================================================
# FOLLOW-UP 2: The k most frequent values of the BST (by
# inorder runs, no dictionary)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Equal values are neighbours in the inorder walk,
# so the lengths of the runs are the counts; sort the runs
# by length.
# Time: O(n + r log r), r = runs.  Space: O(n).
def k_most_frequent_in_bst(root, k):
    runs = []
    previous = [None]

    def walk(node):
        if node is None:
            return
        walk(node.left)
        if runs and previous[0] == node.val:
            runs[-1][1] += 1
        else:
            runs.append([node.val, 1])
        previous[0] = node.val
        walk(node.right)

    walk(root)
    runs.sort(key=lambda run: (-run[1], run[0]))
```

```
    return [value for value, length in runs[:k]]
```

270. Closest Binary Search Tree Value

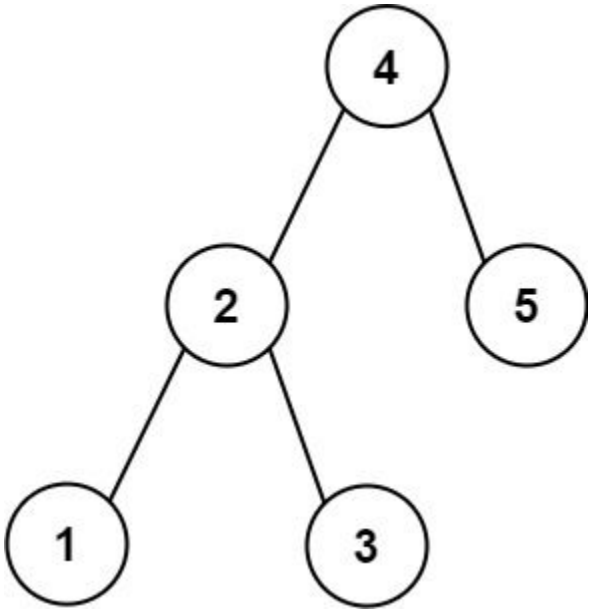
Easy · Tree, Depth-First Search, Binary Search Tree, Binary Search, Binary Tree

leetcode.com/problems/closest-binary-search-tree-value

PROBLEM

Given the `root` of a binary search tree and a `target` value, return *the value in the BST that is closest to the* `target`. If there are multiple answers, print the smallest.

Example 1:




```
Input: root = [4,2,5,1,3], target = 3.714286
Output: 4
```

Example 2:

```
Input: root = [1], target = 4.428571
Output: 1
```

Constraints:

- The number of nodes in the tree is in the range `[1, 10^4]`.
- `0 <= Node.val <= 10^9`
- `-10^9 <= target <= 10^9`

SOLUTIONS (PYTHON 3)

Solution 1: Recursion

How to think about it

We need not walk the whole BST: after comparing with target only one child can improve. Along the path we keep the closer value, breaking ties toward the smaller key. A recursion follows left or right accordingly.

We define a recursive function `dfs(node)`, which starts from the current node `node` and finds the node closest to the target value `target`. We can update the answer by comparing the absolute difference between the current node's value and the target value. If the target value is less than the current node's value, we recursively search the left subtree; otherwise, we recursively search the right subtree.

The time complexity is O(n), and the space complexity is O(n). Where n is the number of nodes in the binary search tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def closestValue(self, root: Optional[TreeNode], target: float) -> int:
        def dfs(node: Optional[TreeNode]):
            if node is None:
                return
            nxt = abs(target - node.val)
            nonlocal ans, diff
            if nxt < diff or (nxt == diff and node.val < ans):
                diff = nxt
                ans = node.val
            node = node.left if target < node.val else node.right
            dfs(node)

        ans = 0
        diff = inf
        dfs(root)
        return ans
```

Solution 2: Iteration

> **How to think about it**
>
> The same walk is an iteration and needs no call stack.

We can rewrite the recursive function in an iterative form, using a loop to simulate the recursive process. We start from the root node and check whether the absolute difference between the current node's value and the target value is less than the current minimum difference. If it is, we update the answer. Then, based on the size relationship between the target value and the current node's value, we decide to move to the left subtree or the right subtree. The loop ends when we traverse to a null node.

The time complexity is O(n), where n is the number of nodes in the binary search tree. The space complexity is O(1).

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def closestValue(self, root: Optional[TreeNode], target: float) -> int:
        ans, diff = root.val, inf
        while root:
            nxt = abs(root.val - target)
            if nxt < diff or (nxt == diff and root.val < ans):
                diff = nxt
                ans = root.val
            root = root.left if target < root.val else root.right
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The k closest values to the target
# 2. The closest value that is NOT bigger than the target
# (floor) and the closest that is not smaller (ceiling)


# ============================================================
# FOLLOW-UP 1: The k closest values to the target
# ============================================================

# ---- follow-up 1: function ----
# Answer: Take the values in inorder (sorted) order and
# use the two-pointer idea around the target, or a heap of
# size k.
# Time: O(n log n).  Space: O(n).
def k_closest_values(root, target, k):
    values = []

    def walk(node):
        if node is None:
            return
        walk(node.left)
        values.append(node.val)
        walk(node.right)

    walk(root)
    ordered = sorted(values, key=lambda v: abs(v - target))
    return sorted(ordered[:k])


# ============================================================
# FOLLOW-UP 2: The closest value that is NOT bigger than
# the target (floor) and the closest that is not smaller
# (ceiling)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk down once, remembering the last node that
# was on the right side (floor) and the last on the left
# side (ceiling).
# Time: O(h).  Space: O(1).
def floor_and_ceiling(root, target):
    floor = None
    ceiling = None
    node = root
    while node:
        if node.val == target:
            return node.val, node.val
        if node.val < target:
            floor = node.val
            node = node.right
        else:
            ceiling = node.val
            node = node.left
    return floor, ceiling
```

98. Validate Binary Search Tree

Medium · Tree, Depth-First Search, Binary Search Tree, Binary Tree

leetcode.com/problems/validate-binary-search-tree

★ **TOP 7 PROBLEM** in Binary Search Tree (BST): master this one first

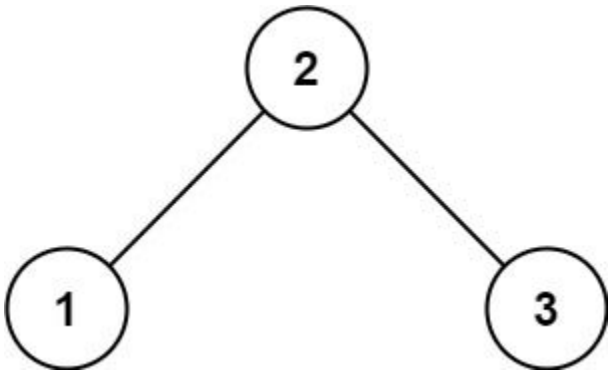
PROBLEM

Given the `root` of a binary tree, *determine if it is a valid binary search tree (BST)*.

A **valid BST** is defined as follows:

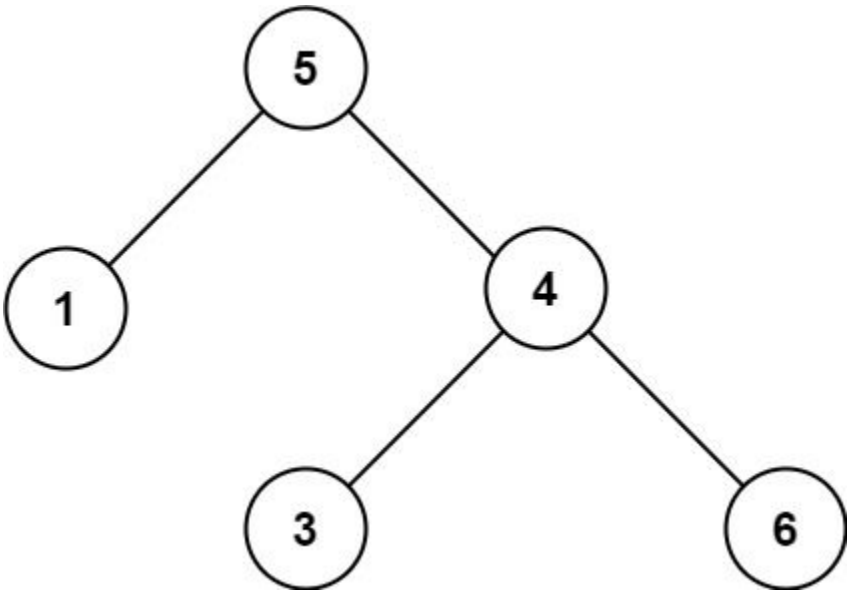
- The left subtree of a node contains only nodes with keys **strictly less than** the node's key.
- The right subtree of a node contains only nodes with keys **strictly greater than** the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:




```
Input: root = [2,1,3]
Output: true
```

Example 2:




```
Input: root = [5,1,4,null,null,3,6]
Output: false
Explanation: The root node's value is 5 but its right child's value is 4.
```

Constraints:

- The number of nodes in the tree is in the range `[1, 10^4]`.
- `-2^31 <= Node.val <= 2^31 - 1`

SOLUTION (PYTHON 3)

Solution 1: Recursion

> **How to think about it**
>
> Checking only "left < root < right" is not enough: the maximum in the left subtree must also be less than the root. We can pass an allowed range, or use the BST fact that inorder is strictly increasing. During inorder, remember the previous value; the current one must be larger. Recurse left, compare, recurse right — one pass finds a violation. An empty tree is valid.

We can perform a recursive in-order traversal on the binary tree. If the result of the traversal is strictly ascending, then this tree is a binary search tree.

Therefore, we use a variable `prev` to save the last node we traversed. Initially, `prev = -∞`. Then we recursively traverse the left subtree. If the left subtree is not a binary search tree, we directly return `False`. Otherwise, we check whether the value of the current node is greater than `prev`. If not, we return `False`. Otherwise, we update `prev` to the value of the current node, and then recursively traverse the right subtree.

The time complexity is O(n), and the space complexity is O(n). Where n is the number of nodes in the binary tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def dfs(root: Optional[TreeNode]) -> bool:
            if root is None:
                return True
            if not dfs(root.left):
                return False
            nonlocal prev
            if prev >= root.val:
                return False
            prev = root.val
            return dfs(root.right)

        prev = -inf
        return dfs(root)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Duplicates are allowed on the LEFT side (left <= node
# < right)
# 2. Iterative version (inorder with a stack)


# ============================================================
# FOLLOW-UP 1: Duplicates are allowed on the LEFT side
# (left <= node < right)
# ============================================================

# ---- follow-up 1: function ----
```

```python
# Answer: The same range check with one border inclusive.
# Time: O(n).  Space: O(h).
def is_valid_bst_duplicates_left(root):
    def check(node, low, high):
        if node is None:
            return True
        if low is not None and node.val <= low:
            return False
        if high is not None and node.val > high:
            return False
        return check(node.left, low, node.val) and check(node.right, node.val, high)

    return check(root, None, None)


# ============================================================
# FOLLOW-UP 2: Iterative version (inorder with a stack)
# ============================================================

# ---- follow-up 2: function ----
# Answer: An inorder walk must give strictly increasing
# values.
# Time: O(n).  Space: O(h).
def is_valid_bst_iterative(root):
    stack = []
    node = root
    previous = None
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        if previous is not None and node.val <= previous:
            return False
        previous = node.val
        node = node.right
    return True
```

701. Insert into a Binary Search Tree

Medium · Tree, Binary Search Tree, Binary Tree

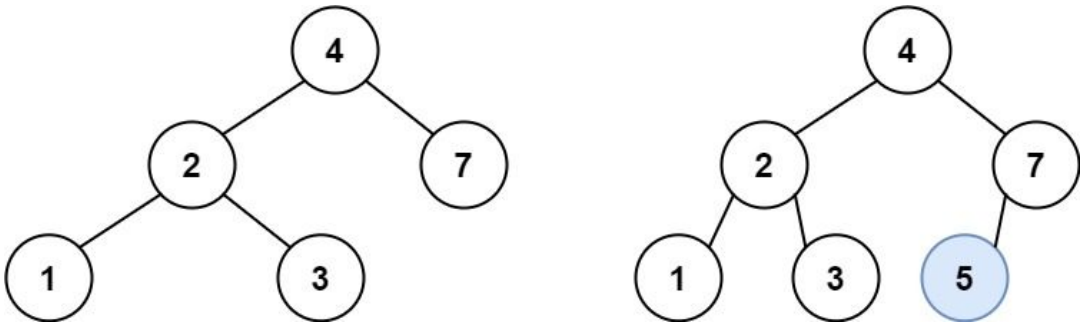
leetcode.com/problems/insert-into-a-binary-search-tree

PROBLEM

You are given the `root` node of a binary search tree (BST) and a `value` to insert into the tree. Return *the root node of the BST after the insertion*. It is **guaranteed** that the new value does not exist in the original BST.

Notice that there may exist multiple valid ways for the insertion, as long as the tree remains a BST after insertion. You can return **any of them**.

Example 1:




```
Input: root = [4,2,7,1,3], val = 5
Output: [4,2,7,1,3,5]
Explanation: Another accepted tree is:
```

Example 2:

```
Input: root = [40,20,60,10,30,50,70], val = 25
Output: [40,20,60,10,30,50,70,null,null,25]
```

Example 3:

```
Input: root = [4,2,7,1,3,null,null,null,null,null,null], val = 5
Output: [4,2,7,1,3,5]
```

Constraints:

- The number of nodes in the tree will be in the range `[0, 10^4]`.
- `-10^8 <= Node.val <= 10^8`
- All the values `Node.val` are **unique**.
- `-10^8 <= val <= 10^8`
- It's **guaranteed** that `val` does not exist in the original BST.

SOLUTION (PYTHON 3)

Solution 1: Recursion

How to think about it

Insert a fresh value into a BST and return the new root. n ≤ 10^4, so any correct insertion path is fine. BST order pins down a unique empty slot: walk by comparison and hang a new node there; no rotation is required. An empty tree becomes that single node. Recurse into the left or right child according to val, then attach the returned subtree. A chain still costs O(n) time and stack.

If the root node is null, we directly create a new node with the value val and return it.

If the root node's value is greater than val, we recursively insert val into the left subtree and update the root of the left subtree with the returned root node.

If the root node's value is less than val, we recursively insert val into the right subtree and update the root of the right subtree with the returned root node.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def insertIntoBST(self, root: Optional[TreeNode], val: int) -> Optional[TreeNode]:
        if root is None:
            return TreeNode(val)
        if root.val > val:
            root.left = self.insertIntoBST(root.left, val)
        else:
            root.right = self.insertIntoBST(root.right, val)
        return root
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Insert with a loop
# 2. Build a BST from a list of values


# ============================================================
# FOLLOW-UP 1: Insert with a loop
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: Walk down to the empty place where the value
# belongs and attach a new node there.
# Time: O(h).  Space: O(1).
def insert_iterative(root, val):
    if root is None:
        return TreeNode(val)
    node = root
    while True:
        if val < node.val:
            if node.left is None:
                node.left = TreeNode(val)
                break
            node = node.left
        else:
            if node.right is None:
                node.right = TreeNode(val)
                break
            node = node.right
    return root


# ============================================================
```

```
# FOLLOW-UP 2: Build a BST from a list of values
# ============================================================

# ---- follow-up 2: function ----
# Answer: Insert the values one after another (the shape
# depends on the order).
# Time: O(n * h).  Space: O(1) extra.
def build_bst(values):
    root = None
    for value in values:
        root = insert_iterative(root, value)
    return root
```

450. Delete Node in a BST

Medium · Tree, Binary Search Tree, Binary Tree

leetcode.com/problems/delete-node-in-a-bst

★ **TOP 7 PROBLEM** in Binary Search Tree (BST): master this one first

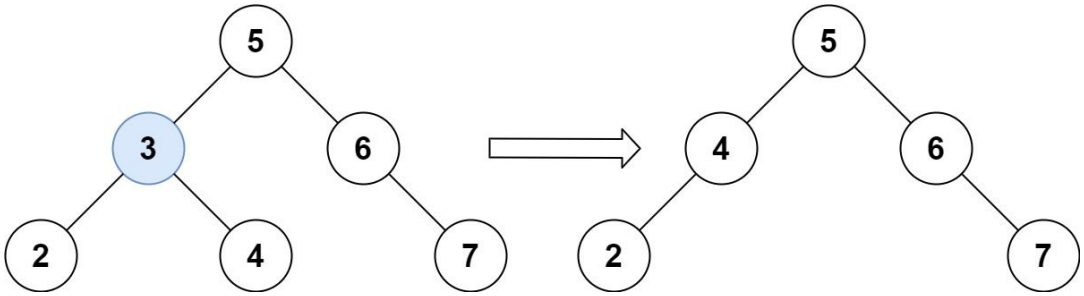
PROBLEM

Given a root node reference of a BST and a key, delete the node with the given key in the BST. Return *the **root node reference** (possibly updated) of the BST*.

Basically, the deletion can be divided into two stages:

1. Search for a node to remove.
2. If the node is found, delete the node.

Example 1:




```
Input: root = [5,3,6,2,4,null,7], key = 3
Output: [5,4,6,2,null,null,7]
Explanation: Given key to delete is 3. So we find the node with value 3 and delete it.
One valid answer is [5,4,6,2,null,null,7], shown in the above BST.
Please notice that another valid answer is [5,2,6,null,4,null,7] and it's also accepted.
```

Example 2:

```
Input: root = [5,3,6,2,4,null,7], key = 0
Output: [5,3,6,2,4,null,7]
Explanation: The tree does not contain a node with value = 0.
```

Example 3:

```
Input: root = [], key = 0
Output: []
```

Constraints:

- The number of nodes in the tree is in the range `[0, 10^4]`.
- `-10^5 <= Node.val <= 10^5`
- Each node has a **unique** value.
- `root` is a valid binary search tree.
- `-10^5 <= key <= 10^5`

Follow up: Could you solve it with time complexity `O(height of tree)`?

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Deletion must keep the BST ordered. A node with one child is replaced by that child; a node with two children needs a successor or predecessor. Recurse left or right by comparison. On a hit: return the right child if there is no left, the left child if there is no right; otherwise hang the whole left subtree off the leftmost node of the right subtree and return the right subtree. That leftmost node is the in-order successor and has no left child, so the in-order sequence stays sorted after one descent.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def deleteNode(self, root: Optional[TreeNode], key: int) -> Optional[TreeNode]:
        if root is None:
            return None
        if root.val > key:
            root.left = self.deleteNode(root.left, key)
            return root
        if root.val < key:
            root.right = self.deleteNode(root.right, key)
            return root
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left
        node = root.right
        while node.left:
            node = node.left
        node.left = root.left
        root = root.right
        return root
```

Complexity. Time: O(h), h = height of the tree. Space: O(h).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Replace a node with two children by its inorder
# PREDECESSOR (the biggest value of the left subtree)
# 2. Delete with a loop (no recursion)


# ============================================================
# FOLLOW-UP 1: Replace a node with two children by its
# inorder PREDECESSOR (the biggest value of the left
# subtree)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Copy the predecessor's value into the node and
# delete the predecessor from the left subtree.
# Time: O(h).  Space: O(h).
def delete_with_predecessor(root, key):
    if root is None:
        return None
    if key < root.val:
```

```
        root.left = delete_with_predecessor(root.left, key)
    elif key > root.val:
        root.right = delete_with_predecessor(root.right, key)
    else:
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left
        predecessor = root.left
        while predecessor.right:
            predecessor = predecessor.right
        root.val = predecessor.val
        root.left = delete_with_predecessor(root.left, predecessor.val)
    return root


# ============================================================
# FOLLOW-UP 2: Delete with a loop (no recursion)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Find the node and its parent, then replace the
# node by its only child or, with two children, by its
# successor.
# Time: O(h).  Space: O(1).
def delete_iterative(root, key):
    parent = None
    node = root
    while node and node.val != key:
        parent = node
        node = node.left if key < node.val else node.right
    if node is None:
        return root
    if node.left and node.right:
        successor_parent = node
        successor = node.right
        while successor.left:
            successor_parent = successor
            successor = successor.left
        node.val = successor.val
```

```
        parent = successor_parent
        node = successor
    child = node.left if node.left else node.right
    if parent is None:
        return child
    if parent.left is node:
        parent.left = child
    else:
        parent.right = child
    return root
```

230. Kth Smallest Element in a BST

Medium · Tree, Depth-First Search, Binary Search Tree, Binary Tree

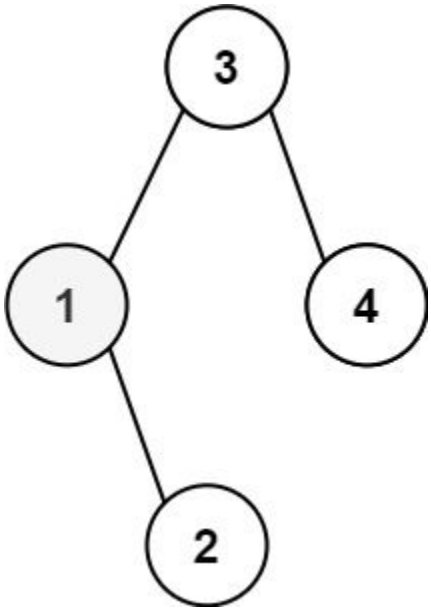
leetcode.com/problems/kth-smallest-element-in-a-bst

★ **TOP 7 PROBLEM** in Binary Search Tree (BST): master this one first

PROBLEM

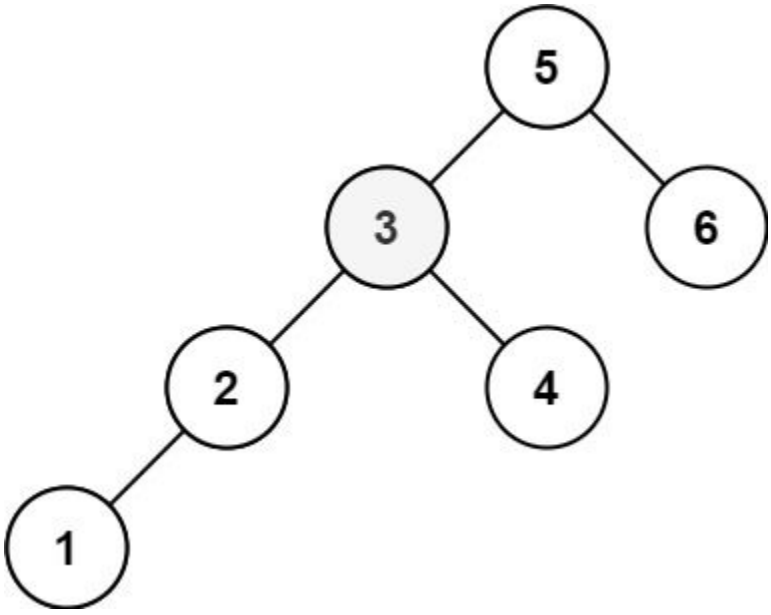
Given the `root` of a binary search tree, and an integer `k`, return *the* k^{th} *smallest value (**1-indexed**) of all the values of the nodes in the tree*.

Example 1:




```
Input: root = [3,1,4,null,2], k = 1
Output: 1
```

Example 2:



```
Input: root = [5,3,6,2,4,null,null,1], k = 3
Output: 3
```

Constraints:

- The number of nodes in the tree is `n`.
- `1 <= k <= n <= 10^4`
- `0 <= Node.val <= 10^4`

Follow up: If the BST is modified often (i.e., we can do insert and delete operations) and you need to find the kth smallest frequently, how would you optimize?

SOLUTIONS (PYTHON 3)

Solution 1

> **How to think about it**
>
> Inorder on a BST is sorted order. We only need the k-th visited node, not the full list. An explicit stack walks left, and each pop decrements k; when k hits 0 we have the answer.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        stk = []
        while root or stk:
            if root:
                stk.append(root)
                root = root.left
            else:
                root = stk.pop()
                k -= 1
                if k == 0:
                    return root.val
                root = root.right
```

Complexity. Time: O(h + k). Space: O(h).

Solution 2

> **How to think about it**
>
> One inorder pass is enough, but repeated queries on a static tree redo the left spine. After storing each subtree size, we compare the left size with k and walk left or right. Each query then follows a root-to-leaf path.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right


class BST:
    def __init__(self, root):
        self.cnt = Counter()
        self.root = root
        self.count(root)

    def kthSmallest(self, k):
        node = self.root
        while node:
            if self.cnt[node.left] == k - 1:
                return node.val
            if self.cnt[node.left] < k - 1:
                k -= self.cnt[node.left] + 1
                node = node.right
            else:
                node = node.left
        return 0

    def count(self, root):
        if root is None:
            return 0
        n = 1 + self.count(root.left) + self.count(root.right)
        self.cnt[root] = n
        return n


class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        bst = BST(root)
        return bst.kthSmallest(k)
```

Complexity. Time: build O(n), each query O(h). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The k-th LARGEST value
# 2. The tree changes often and k-th queries are frequent:
# store the size of every subtree


# ============================================================
# FOLLOW-UP 1: The k-th LARGEST value
# ============================================================

# ---- follow-up 1: function ----
# Answer: Walk the tree in reverse inorder (right, node,
# left) and stop at the k-th value.
# Time: O(h + k).  Space: O(h).
def kth_largest_in_bst(root, k):
    stack = []
    node = root
    while stack or node:
        while node:
            stack.append(node)
            node = node.right
        node = stack.pop()
        k -= 1
```

```
        if k == 0:
            return node.val
        node = node.left
    return None


# ============================================================
# FOLLOW-UP 2: The tree changes often and k-th queries are
# frequent: store the size of every subtree
# ============================================================

# ---- follow-up 2: function ----
# Answer: With the size of the left subtree known, each
# query goes down one path: O(h).
# Time: insert O(h), kth O(h).  Space: O(n).
class SizedNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
        self.size = 1


def insert_sized(root, val):
    if root is None:
        return SizedNode(val)
    root.size += 1
    if val < root.val:
        root.left = insert_sized(root.left, val)
    else:
        root.right = insert_sized(root.right, val)
    return root


def kth_smallest_sized(root, k):
    node = root
    while node:
```

```
        left_size = node.left.size if node.left else 0
        if k == left_size + 1:
            return node.val
        if k <= left_size:
            node = node.left
        else:
            k -= left_size + 1
            node = node.right
    return None
```

538. Convert BST to Greater Tree

Medium · Tree, Depth-First Search, Binary Search Tree, Binary Tree

leetcode.com/problems/convert-bst-to-greater-tree

★ **TOP 7 PROBLEM** in Binary Search Tree (BST): master this one first

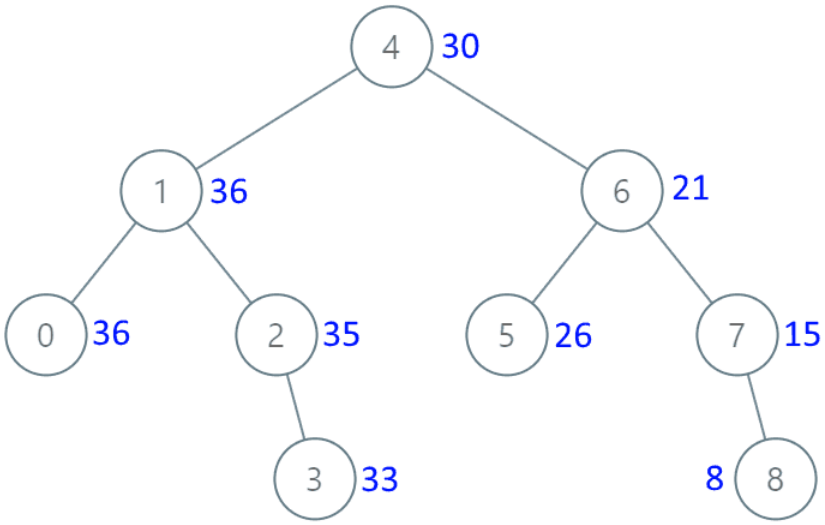
PROBLEM

Given the `root` of a Binary Search Tree (BST), convert it to a Greater Tree such that every key of the original BST is changed to the original key plus the sum of all keys greater than the original key in BST.

As a reminder, a *binary search tree* is a tree that satisfies these constraints:

- The left subtree of a node contains only nodes with keys **less than** the node's key.
- The right subtree of a node contains only nodes with keys **greater than** the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:




```
Input: root = [4,1,6,0,2,5,7,null,null,null,3,null,null,null,8]
Output: [30,36,21,36,35,26,15,null,null,null,33,null,null,null,8]
```

Example 2:

```
Input: root = [0,null,1]
Output: [1,null,1]
```

Constraints:

- The number of nodes in the tree is in the range `[0, 10^4]`.
- `-10^4 <= Node.val <= 10^4`
- All the values in the tree are **unique**.
- `root` is guaranteed to be a valid binary search tree.

Note: This question is the same as 1038: https://leetcode.com/problems/binary-search-tree-to-greater-sum-tree/

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Each node should become the sum of all keys not smaller than itself. In a BST those keys are exactly the nodes already seen in reverse inorder. Walk right-root-left, accumulate into s, and write s back. The right subtree is processed first, so larger keys are already included. One traversal.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def convertBST(self, root: TreeNode) -> TreeNode:
        def dfs(root):
            nonlocal s
            if root is None:
                return
            dfs(root.right)
            s += root.val
            root.val = s
            dfs(root.left)

        s = 0
        dfs(root)
        return root
```

Complexity. Time: O(n). Space: O(h).

Solution 2

How to think about it

Recursive reverse inorder uses an O(h) stack. Morris temporarily links a node's inorder predecessor (leftmost of the right subtree) to recover the parent without a stack. Thread when going right, accumulate and write on the return visit, then unthread and go left. The order is still right-root-left, with constant extra space.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def convertBST(self, root: TreeNode) -> TreeNode:
        s = 0
        node = root
        while root:
            if root.right is None:
                s += root.val
                root.val = s
                root = root.left
            else:
                next = root.right
                while next.left and next.left != root:
                    next = next.left
                if next.left is None:
                    next.left = root
                    root = root.right
                else:
                    s += root.val
                    root.val = s
                    next.left = None
                    root = root.left
        return node
```

Solution 3: Explicit Stack

How to think about it

Each node should become the sum of every key not smaller than itself. In a BST those keys are exactly the nodes already seen in reverse inorder. Recursing down the right child and writing the sum back is correct on a short tree. The tree can contain 10^4 nodes. A right chain makes this walk recurse once per node and overflow the call stack. The running sum depends only on the greater keys already visited. A node does not need a value returned from its children, as long as the right child is finished before the node and the left child comes after. An explicit stack performs that reverse inorder. On entry it pushes an exit marker and the right child; on exit it adds the node value into s, writes s back, and pushes the left child.

An explicit stack walks the binary search tree in the order right, root, left. On entry we push an exit marker and the right child. On exit we add the node value into s, write s back to the node, and push the left child.

The time complexity is O(n) and the space complexity is O(n), where n is the number of nodes in the binary search tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def convertBST(self, root: TreeNode) -> TreeNode:
        s = 0
        stk = [(root, 0)]
        while stk:
            node, state = stk.pop()
            if node is None:
                continue
            if state == 0:
                stk.append((node, 1))
                stk.append((node.right, 0))
                continue
            s += node.val
            node.val = s
            stk.append((node.left, 0))
        return root
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The SMALLER sum tree: every node becomes the sum of
# the nodes with smaller or equal keys
# 2. Convert in O(1) extra space (Morris traversal,
# reverse order)


# ============================================================
# FOLLOW-UP 1: The SMALLER sum tree: every node becomes
# the sum of the nodes with smaller or equal keys
# ============================================================

# ---- follow-up 1: function ----
# Answer: Walk the tree in normal inorder (left, node,
# right) with a running total.
# Time: O(n).  Space: O(h).
def convert_to_smaller_tree(root):
    total = 0
    stack = []
    node = root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        total += node.val
        node.val = total
        node = node.right
    return root


# ============================================================
# FOLLOW-UP 2: Convert in O(1) extra space (Morris
# traversal, reverse order)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Reverse Morris: thread from the leftmost node of
# the right subtree back to the current node, so no stack
# is needed.
```

```
# Time: O(n).  Space: O(1).
def convert_to_greater_morris(root):
    total = 0
    node = root
    while node:
        if node.right is None:
            total += node.val
            node.val = total
            node = node.left
        else:
            successor = node.right
            while successor.left and successor.left is not node:
                successor = successor.left
            if successor.left is None:
                successor.left = node
                node = node.right
            else:
                successor.left = None
                total += node.val
                node.val = total
```

```
                node = node.left
    return root
```

1382. Balance a Binary Search Tree

Medium · Greedy, Tree, Depth-First Search, Binary Search Tree, Divide and Conquer, Binary Tree

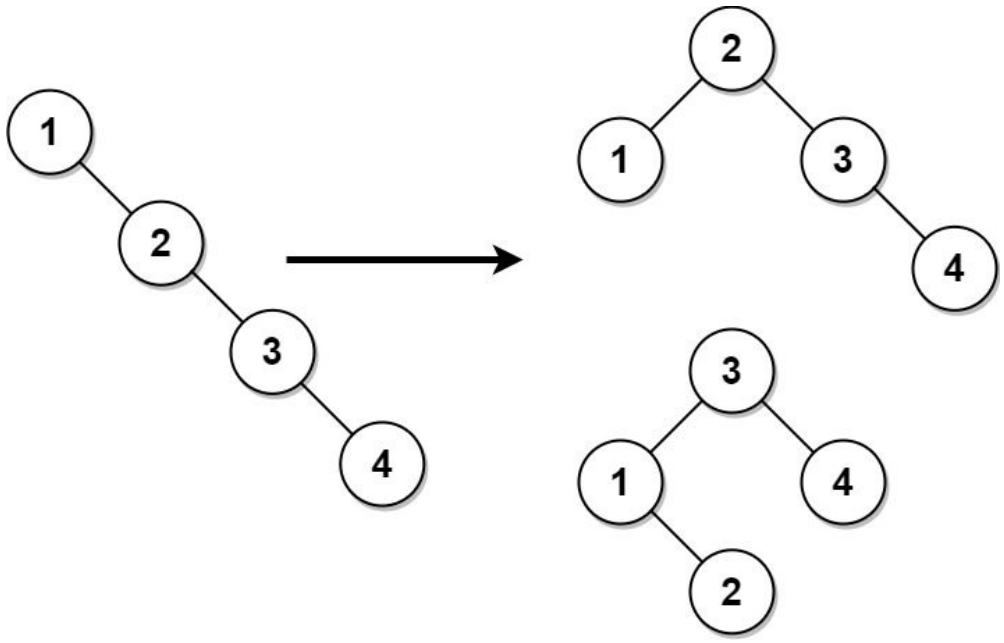
leetcode.com/problems/balance-a-binary-search-tree

PROBLEM

Given the `root` of a binary search tree, return *a **balanced** binary search tree with the same node values*. If there is more than one answer, return **any of them**.

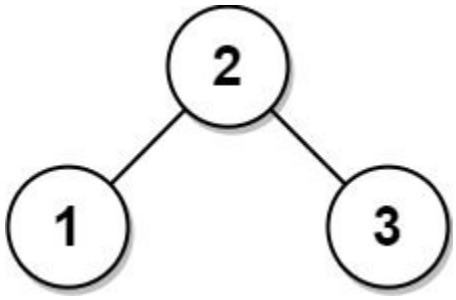
A binary search tree is **balanced** if the depth of the two subtrees of every node never differs by more than `1`.

Example 1:




```
Input: root = [1,null,2,null,3,null,4,null,null]
Output: [2,1,3,null,null,null,4]
Explanation: This is not the only correct answer, [3,1,4,null,2] is also correct.
```

Example 2:




```
Input: root = [2,1,3]
Output: [2,1,3]
```

Constraints:

- The number of nodes in the tree is in the range `[1, 10^4]`.
- `1 <= Node.val <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: In-order Traversal + Construct Balanced Binary Search Tree

> **How to think about it**
>
> Rebalance a BST without changing the inorder sequence. An inorder walk yields a sorted array; recursively taking the midpoint as the root builds a tree of height log n that is still a BST.

Since the original tree is a binary search tree, we can save the result of the in-order traversal in an array nums. Then we design a function build(i, j), which is used to construct a balanced binary search tree within the index range [i, j] in nums. The answer is build(0, |nums| - 1).

The execution logic of the function build(i, j) is as follows:

- If i > j, then the balanced binary search tree is empty, return an empty node;
- Otherwise, we take mid = (i + j) / 2 as the root node, then recursively build the left and right subtrees, and return the root node.

The time complexity is O(n), and the space complexity is O(n). Where n is the number of nodes in the binary search tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def balanceBST(self, root: TreeNode) -> TreeNode:
        def dfs(root: TreeNode):
            if root is None:
                return
            dfs(root.left)
            nums.append(root.val)
            dfs(root.right)

        def build(i: int, j: int) -> TreeNode:
            if i > j:
                return None
            mid = (i + j) >> 1
            left = build(i, mid - 1)
            right = build(mid + 1, j)
            return TreeNode(nums[mid], left, right)

        nums = []
        dfs(root)
        return build(0, len(nums) - 1)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Check whether a BST is already height-balanced
# 2. Rebuild by repeated insertion in a smart order


# ============================================================
# FOLLOW-UP 1: Check whether a BST is already height-
# balanced
# ============================================================

# ---- follow-up 1: function ----
# Answer: Return heights from the bottom up; the tree is
# balanced when the heights of the two children differ by
# at most 1 everywhere.
```

```
# Time: O(n).  Space: O(h).
def is_height_balanced(root):
    def height(node):
        if node is None:
            return 0
        left = height(node.left)
        if left == -1:
            return -1
        right = height(node.right)
        if right == -1 or abs(left - right) > 1:
            return -1
        return 1 + max_of(left, right)

    return height(root) != -1


def max_of(a, b):
    if a > b:
        return a
    return b


# ============================================================
# FOLLOW-UP 2: Rebuild by repeated insertion in a smart
# order
# ============================================================

# ---- follow-up 2: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 2: function ----
# Answer: Take the sorted values and insert the middle one
# first, then the middles of each half, which gives a
# balanced tree without recursion on nodes.
# Time: O(n log n).  Space: O(n).
from collections import deque


def balanced_by_insertion_order(sorted_values):
    root = None
    queue = deque([(0, len(sorted_values) - 1)])
    while queue:
```

```
        low, high = queue.popleft()
        if low > high:
            continue
        mid = (low + high) // 2
        root = insert_value(root, sorted_values[mid])
        queue.append((low, mid - 1))
        queue.append((mid + 1, high))
    return root


def insert_value(root, val):
    if root is None:
        return TreeNode(val)
    node = root
    while True:
        if val < node.val:
            if node.left is None:
                node.left = TreeNode(val)
                break
            node = node.left
        else:
            if node.right is None:
```

```
                node.right = TreeNode(val)
                break
            node = node.right
    return root
```

669. Trim a Binary Search Tree

Medium · Tree, Depth-First Search, Binary Search Tree, Binary Tree

leetcode.com/problems/trim-a-binary-search-tree

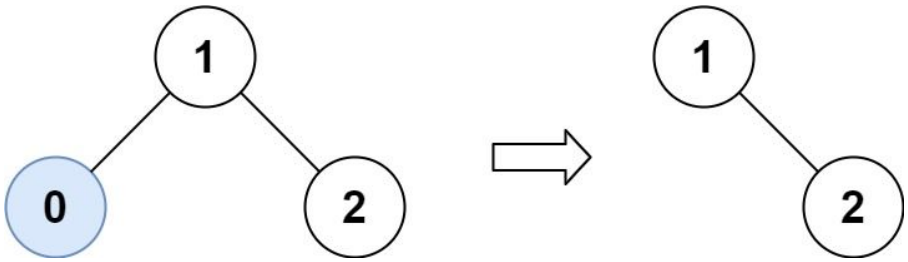
★ **TOP 7 PROBLEM** in Binary Search Tree (BST): master this one first

PROBLEM

Given the `root` of a binary search tree and the lowest and highest boundaries as `low` and `high`, trim the tree so that all its elements lies in `[low, high]`. Trimming the tree should **not** change the relative structure of the elements that will remain in the tree (i.e., any node's descendant should remain a descendant). It can be proven that there is a **unique answer**.

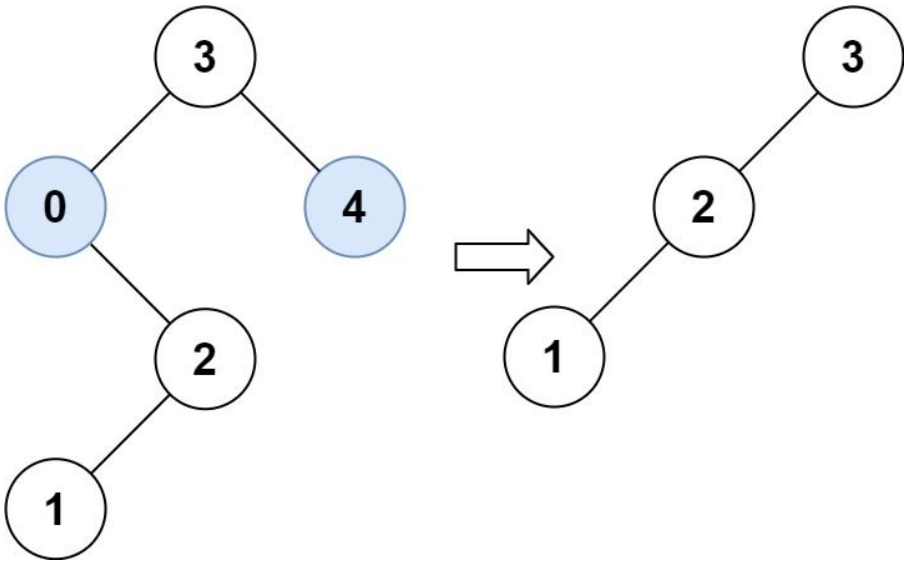
Return *the root of the trimmed binary search tree*. Note that the root may change depending on the given bounds.

Example 1:




```
Input: root = [1,0,2], low = 1, high = 2
Output: [1,null,2]
```

Example 2:




```
Input: root = [3,0,4,null,2,null,null,1], low = 1, high = 3
Output: [3,2,null,1]
```

Constraints:

- The number of nodes in the tree is in the range `[1, 10^4]`.
- `0 <= Node.val <= 10^4`
- The value of each node in the tree is **unique**.
- `root` is guaranteed to be a valid binary search tree.
- `0 <= low <= high <= 10^4`

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Drop BST nodes outside [low,high]. Order lets us discard a whole left or right subtree at once. If the root is too large, recurse only left; if too small, only right; otherwise trim both children and keep the root.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def trimBST(
        self, root: Optional[TreeNode], low: int, high: int
    ) -> Optional[TreeNode]:
        def dfs(root):
            if root is None:
                return root
            if root.val > high:
                return dfs(root.left)
            if root.val < low:
                return dfs(root.right)
            root.left = dfs(root.left)
            root.right = dfs(root.right)
            return root

        return dfs(root)
```

Complexity. Time: O(n). Space: O(h).

Solution 2

How to think about it

Recursion uses linear stack. Slide the root into range, then walk each side: replace an undersized left child by its right, an oversized right child by its left. Extra memory is constant.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def trimBST(
        self, root: Optional[TreeNode], low: int, high: int
    ) -> Optional[TreeNode]:
        while root and (root.val < low or root.val > high):
            root = root.left if root.val > high else root.right
        if root is None:
            return None
        node = root
        while node.left:
            if node.left.val < low:
                node.left = node.left.right
            else:
                node = node.left
        node = root
        while node.right:
            if node.right.val > high:
                node.right = node.right.left
            else:
                node = node.right
        return root
```

Complexity. Time: O(h). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Also report how many nodes were removed
# 2. Trim with a loop


# ============================================================
# FOLLOW-UP 1: Also report how many nodes were removed
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count the nodes before and after, or count every
# node that is dropped (the whole dropped subtrees).
# Time: O(n).  Space: O(h).
def trim_and_count(root, low, high):
    def size(node):
        if node is None:
            return 0
        return 1 + size(node.left) + size(node.right)

    before = size(root)

    def trim(node):
        if node is None:
            return None
        if node.val < low:
            return trim(node.right)
        if node.val > high:
            return trim(node.left)
        node.left = trim(node.left)
        node.right = trim(node.right)
        return node

    new_root = trim(root)
    return new_root, before - size(new_root)
```

```
# ============================================================
# FOLLOW-UP 2: Trim with a loop
# ============================================================

# ---- follow-up 2: function ----
# Answer: First move the root into the range, then trim
# the left side and the right side of the new root
# separately.
# Time: O(h).  Space: O(1).
def trim_iterative(root, low, high):
    while root and (root.val < low or root.val > high):
        root = root.right if root.val < low else root.left
    node = root
    while node:
        while node.left and node.left.val < low:
            node.left = node.left.right
        node = node.left
    node = root
    while node:
        while node.right and node.right.val > high:
            node.right = node.right.left
        node = node.right
    return root
```

99. Recover Binary Search Tree

Medium · Tree, Depth-First Search, Binary Search Tree, Binary Tree

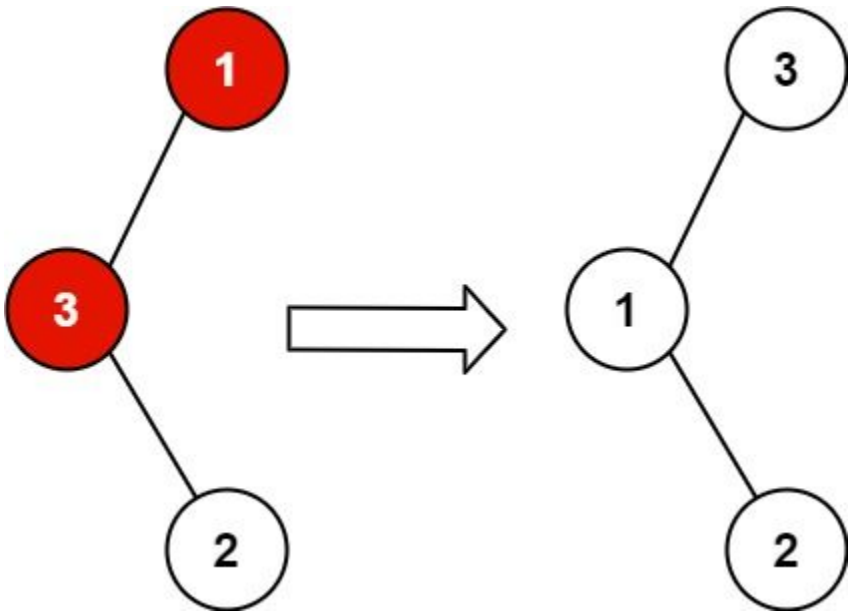
leetcode.com/problems/recover-binary-search-tree

★ **TOP 7 PROBLEM** in Binary Search Tree (BST): master this one first

PROBLEM

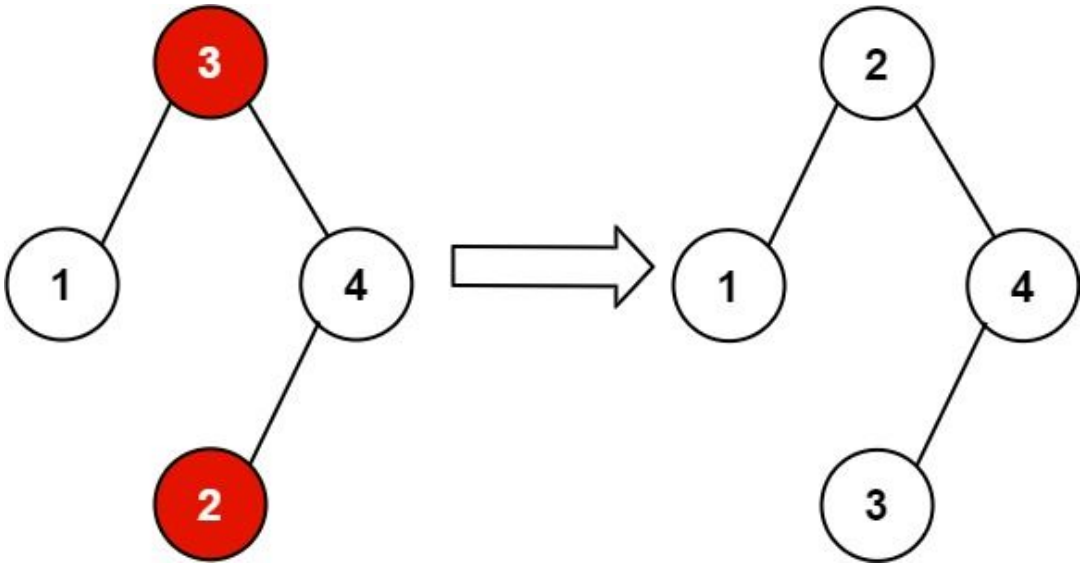
You are given the `root` of a binary search tree (BST), where the values of **exactly** two nodes of the tree were swapped by mistake. *Recover the tree without changing its structure*.

Example 1:




```
Input: root = [1,3,null,null,2]
Output: [3,1,null,null,2]
Explanation: 3 cannot be a left child of 1 because 3 > 1. Swapping 1 and 3 makes the BST valid.
```

Example 2:



```
Input: root = [3,1,4,null,null,2]
Output: [2,1,4,null,null,3]
Explanation: 2 cannot be in the right subtree of 3 because 2 < 3. Swapping 2 and 3 makes the BST valid.
```

Constraints:

- The number of nodes in the tree is in the range `[2, 1000]`.
- -2^{31} <= Node.val <= $2^{31} - 1$

Follow up:

A solution using

`O(n)`

space is pretty straight-forward. Could you devise a constant

`O(1)`

space solution?

SOLUTION (PYTHON 3)

Solution 1: In-order Traversal

How to think about it

Inorder of a BST is strictly increasing. After swapping two nodes, the sequence has one or two inversions: adjacent swap yields one; non-adjacent yields two (the earlier node of the first inversion and the later node of the second). Walk inorder, record those two nodes, and swap their values. No need to rebuild the tree. This solution uses recursive inorder (O(n) stack); Morris would meet the O(1) follow-up. The point of this method is to identify the swapped pair.

In-order traversal of a binary search tree results in an increasing sequence. If two nodes' values are mistakenly swapped, there will definitely be two reverse pairs in the sequence obtained from the in-order traversal. We use `first` and `second` to record the smaller and larger values of these two reverse pairs, respectively. Finally, swapping the values of these two nodes will correct the mistake.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary search tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def recoverTree(self, root: Optional[TreeNode]) -> None:
        """
        Do not return anything, modify root in-place instead.
        """

        def dfs(root):
            if root is None:
                return
            nonlocal prev, first, second
            dfs(root.left)
            if prev and prev.val > root.val:
                if first is None:
                    first = prev
                second = root
            prev = root
            dfs(root.right)

        prev = first = second = None
        dfs(root)
        first.val, second.val = second.val, first.val
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the two values that were swapped (without
# fixing the tree)
# 2. O(1) extra space with Morris traversal


# ============================================================
# FOLLOW-UP 1: Return the two values that were swapped
# (without fixing the tree)
# ============================================================

# ---- follow-up 1: function ----
# Answer: In the inorder walk, the first place where a
# value is smaller than its predecessor gives the first
# wrong node (the predecessor), and the last such place
# gives the second wrong node.
# Time: O(n).  Space: O(h).
def find_swapped_values(root):
    first = second = previous = None
    stack = []
    node = root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        if previous is not None and node.val < previous.val:
            if first is None:
                first = previous
            second = node
        previous = node
        node = node.right
    if first is None:
        return None
    return first.val, second.val


# ============================================================
# FOLLOW-UP 2: O(1) extra space with Morris traversal
# ============================================================

# ---- follow-up 2: function ----
# Answer: Do the same inorder check with threads instead
# of a stack.
# Time: O(n).  Space: O(1).
def recover_tree_morris(root):
    first = second = previous = None
    node = root
    while node:
        if node.left is None:
            if previous is not None and node.val < previous.val:
                if first is None:
                    first = previous
                second = node
            previous = node
            node = node.right
        else:
            predecessor = node.left
            while predecessor.right and predecessor.right is not node:
                predecessor = predecessor.right
```

```
            if predecessor.right is None:
                predecessor.right = node
                node = node.left
```

```
            else:
                predecessor.right = None
                if previous is not None and node.val < previous.val:
                    if first is None:
                        first = previous
                    second = node
                previous = node
                node = node.right
    if first:
        first.val, second.val = second.val, first.val
    return root
```

96. Unique Binary Search Trees

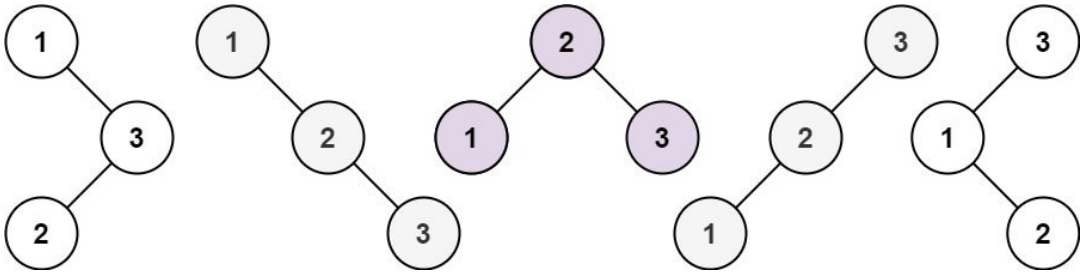
Medium · Tree, Binary Search Tree, Math, Dynamic Programming, Binary Tree

leetcode.com/problems/unique-binary-search-trees

PROBLEM

Given an integer `n`, return *the number of structurally unique **BST's** (binary search trees) which has exactly* `n` *nodes of unique values from* `1` *to* `n`.

Example 1:




```
Input: n = 3
Output: 5
```

Example 2:

```
Input: n = 1
Output: 1
```

Constraints:

- `1 <= n <= 19`

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

> **How to think about it**
>
> The previous problem lists every tree; here we only need the count. The structure is the same: pick a root, multiply left and right counts, sum over roots. Shapes collapse to “how many nodes”: any i consecutive integers yield the same number of BSTs. Let f[i] be the number of trees with i nodes, f[0]=1, and build from small i so each size is computed once.

We define f[i] to represent the number of binary search trees that can be generated from [1, i]. Initially, f[0] = 1, and the answer is f[n].

We can enumerate the number of nodes i, then the number of nodes in the left subtree $j \in [0, i - 1]$, and the number of nodes in the right subtree k = i - j - 1. The number of combinations of the number of nodes in the left subtree and the right subtree is f[j] × f[k], so $f[i] = \sum_{j=0}^{i-1} f[j] \times f[i - j - 1]$.

Finally, return f[n].

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes.

Python 3

```
class Solution:
    def numTrees(self, n: int) -> int:
        f = [1] + [0] * n
        for i in range(n + 1):
            for j in range(i):
                f[i] += f[j] * f[i - j - 1]
        return f[n]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. A closed formula: the Catalan number
# 2. How many BSTs have a given value r as the root
# (values 1..n)?


# ============================================================
# FOLLOW-UP 1: A closed formula: the Catalan number
# ============================================================

# ---- follow-up 1: function ----
# Answer: The number of BSTs with n nodes is C(2n, n) / (n
# + 1), computed with integer arithmetic.
# Time: O(n).  Space: O(1).
def num_trees_formula(n):
    result = 1
    for i in range(1, n + 1):
        result = result * (n + i) // i
    return result // (n + 1)


# ============================================================
# FOLLOW-UP 2: How many BSTs have a given value r as the
# root (values 1..n)?
# ============================================================

# ---- follow-up 2: function ----
# Answer: The left subtree uses the r - 1 smaller values
# and the right subtree the n - r bigger values; multiply
# their counts.
# Time: O(n^2).  Space: O(n).
def count_with_root(n, r):
    ways = [0] * (n + 1)
    ways[0] = 1
    for nodes in range(1, n + 1):
        for left in range(nodes):
            ways[nodes] += ways[left] * ways[nodes - 1 - left]
    return ways[r - 1] * ways[n - r]
```

173. Binary Search Tree Iterator

Medium · Stack, Tree, Design, Binary Search Tree, Binary Tree, Iterator

leetcode.com/problems/binary-search-tree-iterator

★ **TOP 7 PROBLEM** in Binary Search Tree (BST): master this one first

PROBLEM

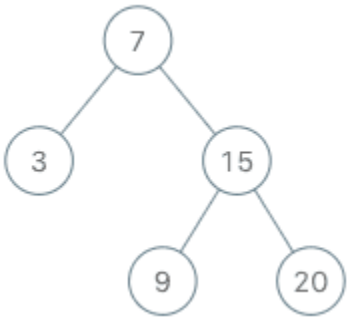
Implement the `BSTIterator` class that represents an iterator over the **in-order traversal** of a binary search tree (BST):

- `BSTIterator(TreeNode root)` Initializes an object of the `BSTIterator` class. The `root` of the BST is given as part of the constructor. The pointer should be initialized to a non-existent number smaller than any element in the BST.
- `boolean hasNext()` Returns `true` if there exists a number in the traversal to the right of the pointer, otherwise returns `false`.
- `int next()` Moves the pointer to the right, then returns the number at the pointer.

Notice that by initializing the pointer to a non-existent smallest number, the first call to `next()` will return the smallest element in the BST.

You may assume that `next()` calls will always be valid. That is, there will be at least a next number in the in-order traversal when `next()` is called.

Example 1:




```
Input
["BSTIterator", "next", "next", "hasNext", "next", "hasNext", "next", "hasNext", "next", "hasNext"]
[[[7, 3, 15, null, null, 9, 20]], [], [], [], [], [], [], [], [], []]
Output
[null, 3, 7, true, 9, true, 15, true, 20, false]

Explanation
BSTIterator bSTIterator = new BSTIterator([7, 3, 15, null, null, 9, 20]);
bSTIterator.next(); // return 3
bSTIterator.next(); // return 7
bSTIterator.hasNext(); // return True
bSTIterator.next(); // return 9
bSTIterator.hasNext(); // return True
bSTIterator.next(); // return 15
bSTIterator.hasNext(); // return True
bSTIterator.next(); // return 20
bSTIterator.hasNext(); // return False
```

Constraints:

- The number of nodes in the tree is in the range $[1, 10^5]$.
- $0 <= Node.val <= 10^6$
- At most 10^5 calls will be made to `hasNext`, and `next`.

Follow up:

- Could you implement `next()` and `hasNext()` to run in average `O(1)` time and use `O(h)` memory, where `h` is the height of the tree?

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Inorder on a BST is sorted. Up to 10^5 nodes and as many calls. A full inorder into an array makes next/hasNext a cursor move — amortized O(1) — but O(n) space. The follow-up wants O(h) memory.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class BSTIterator:
    def __init__(self, root: TreeNode):
        def inorder(root):
            if root:
                inorder(root.left)
                self.vals.append(root.val)
                inorder(root.right)

        self.cur = 0
        self.vals = []
        inorder(root)

    def next(self) -> int:
        res = self.vals[self.cur]
        self.cur += 1
        return res

    def hasNext(self) -> bool:
        return self.cur < len(self.vals)


# Your BSTIterator object will be instantiated and called as such:
# obj = BSTIterator(root)
# param_1 = obj.next()
# param_2 = obj.hasNext()
```

Solution 2

How to think about it

Solution 1 flattens the whole tree. An explicit stack simulates inorder: push the left spine at init; next pops, then pushes the left spine of the right child. At most h nodes sit on the stack, and each node is pushed and popped once.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class BSTIterator:
    def __init__(self, root: TreeNode):
        self.stack = []
        while root:
            self.stack.append(root)
            root = root.left

    def next(self) -> int:
        cur = self.stack.pop()
        node = cur.right
        while node:
            self.stack.append(node)
            node = node.left
        return cur.val

    def hasNext(self) -> bool:
        return len(self.stack) > 0


# Your BSTIterator object will be instantiated and called as such:
# obj = BSTIterator(root)
# param_1 = obj.next()
# param_2 = obj.hasNext()
```

Complexity. Time: next O(1) amortised, hasNext O(1). Space: O(h), h = height of the tree.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. A REVERSE iterator (from the biggest value down)
# 2. An iterator that can also go BACK (prev())


# ============================================================
# FOLLOW-UP 1: A REVERSE iterator (from the biggest value
# down)
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same stack idea with the roles of left and
# right swapped.
# Time: next O(1) amortised, has_next O(1).  Space: O(h).
class ReverseBSTIterator:
    def __init__(self, root):
        self.stack = []
        self.push_right(root)

    def push_right(self, node):
        while node:
            self.stack.append(node)
            node = node.right

    def next(self):
        node = self.stack.pop()
        self.push_right(node.left)
        return node.val

    def has_next(self):
        return len(self.stack) > 0
```

```
# ============================================================
# FOLLOW-UP 2: An iterator that can also go BACK (prev())
# ============================================================

# ---- follow-up 2: function ----
# Answer: Remember the values already returned in a list;
# prev() moves the position back, and next() walks forward
# through the list before asking the tree again.
# Time: next and prev O(1) amortised.  Space: O(n).
class BidirectionalBSTIterator:
    def __init__(self, root):
        self.stack = []
        self.seen = []
        self.position = -1
        self.push_left(root)

    def push_left(self, node):
        while node:
            self.stack.append(node)
            node = node.left

    def has_next(self):
        return self.position + 1 < len(self.seen) or len(self.stack) > 0

    def next(self):
        self.position += 1
        if self.position == len(self.seen):
```

```
            node = self.stack.pop()
            self.push_left(node.right)
            self.seen.append(node.val)
        return self.seen[self.position]

    def has_prev(self):
        return self.position > 0

    def prev(self):
        self.position -= 1
        return self.seen[self.position]
```

1008. Construct Binary Search Tree from Preorder Traversal

Medium · Stack, Tree, Binary Search Tree, Array, Binary Tree, Monotonic Stack

leetcode.com/problems/construct-binary-search-tree-from-preorder-traversal

PROBLEM

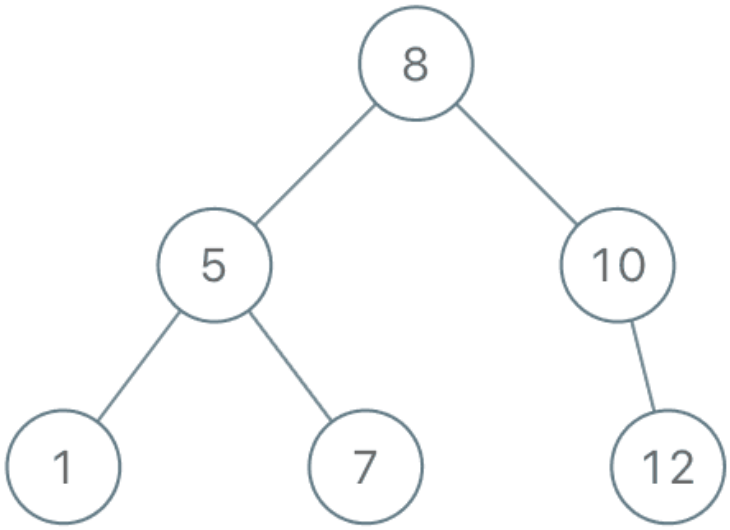
Given an array of integers preorder, which represents the **preorder traversal** of a BST (i.e., **binary search tree**), construct the tree and return *its root*.

It is **guaranteed** that there is always possible to find a binary search tree with the given requirements for the given test cases.

A **binary search tree** is a binary tree where for every node, any descendant of `Node.left` has a value **strictly less than** `Node.val`, and any descendant of `Node.right` has a value **strictly greater than** `Node.val`.

A **preorder traversal** of a binary tree displays the value of the node first, then traverses `Node.left`, then traverses `Node.right`.

Example 1:




```
Input: preorder = [8,5,1,7,10,12]
Output: [8,5,10,1,7,null,12]
```

Example 2:

```
Input: preorder = [1,3]
Output: [1,null,3]
```

Constraints:

- `1 <= preorder.length <= 100`
- `1 <= preorder[i] <= 1000`
- All the values of `preorder` are **unique**.

SOLUTION (PYTHON 3)

Solution 1: DFS + Binary Search

How to think about it

The first preorder value is the root; a linear scan finds the first larger key and splits the left and right subtrees. n≤ 100 makes $O(n^2)$ acceptable, yet each split walks the same ranges again. A BST preorder segment still has every left-subtree key below the root and every right-subtree key above it, so the first index greater than the root can be found by binary search. dfs(i,j) builds a node from preorder[i], bisects to the right-subtree start, and recurses on both sides in O(nlog n).

We design a function dfs(i, j) to construct a binary search tree from the nodes preorder[i] to preorder[j]. The answer is dfs(0, n - 1).

In dfs(i, j), we first construct the root node, which is preorder[i]. Then, we use binary search to find the first node greater than preorder[i] and get its index mid. We set dfs(i + 1, mid - 1) as the left subtree of the root node and dfs(mid, j) as the right subtree of the root node.

Finally, we return the root node.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the length of the array preorder.

Python 3

```python
class Solution:
    def bstFromPreorder(self, preorder: List[int]) -> Optional[TreeNode]:
        def dfs(i: int, j: int) -> Optional[TreeNode]:
            if i > j:
                return None
            root = TreeNode(preorder[i])
            l, r = i + 1, j + 1
            while l < r:
                mid = (l + r) >> 1
                if preorder[mid] > preorder[i]:
                    r = mid
                else:
                    l = mid + 1
            root.left = dfs(i + 1, l - 1)
            root.right = dfs(l, j)
            return root

        return dfs(0, len(preorder) - 1)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Build the BST from a POSTORDER traversal
# 2. Is a list a valid PREORDER of some BST? (without
# building it)


# ============================================================
# FOLLOW-UP 1: Build the BST from a POSTORDER traversal
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: Read the postorder from the end (root, then
```

```
# right subtree, then left subtree) with the same bounds
# idea.
# Time: O(n).  Space: O(h).
def bst_from_postorder(postorder):
    index = [len(postorder) - 1]

    def build(low, high):
        if index[0] < 0:
            return None
        value = postorder[index[0]]
        if value < low or value > high:
            return None
        index[0] -= 1
        node = TreeNode(value)
        node.right = build(value, high)
        node.left = build(low, value)
        return node

    return build(float("-inf"), float("inf"))


# ============================================================
# FOLLOW-UP 2: Is a list a valid PREORDER of some BST?
# (without building it)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the lower bound set by the last time you
# turned right: every later value must be bigger than it.
# Time: O(n).  Space: O(n).
def is_valid_preorder(preorder):
    stack = []
    lower = float("-inf")
    for value in preorder:
        if value <= lower:
            return False
        while stack and stack[-1] < value:
            lower = stack.pop()
        stack.append(value)
    return True
```

PATTERN 27 OF 41

Bit Manipulation

20 problems · 8 Easy · 12 Medium · 0 Hard

THE IDEA

Work on the binary digits directly. A few identities cover most problems: x & (x - 1) clears the lowest set bit, x ^ x = 0, 1 << k is a mask.

Signals: single number, count set bits, power of two, missing number, subsets by bitmask, add without plus, reverse bits.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
count = 0
while x:
    x &= x - 1            # remove the lowest set bit
    count += 1
single = 0
for n in nums:
    single ^= n           # pairs cancel out
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Single Number	Easy	LC 136	1459
2	Number of 1 Bits	Easy	LC 191	1461
3	Counting Bits	Easy	LC 338	1463
4	Missing Number	Easy	LC 268	1465
5	Reverse Bits	Easy	LC 190	1468
6	Find the Difference	Easy	LC 389	1470
7	Binary Number with Alternating Bits	Easy	LC 693	1472
8	XOR Operation in an Array	Easy	LC 1486	1474
9	★ **Single Number II**	Medium	LC 137	1476
10	★ **Single Number III**	Medium	LC 260	1480
11	★ **Sum of Two Integers**	Medium	LC 371	1483
12	★ **Bitwise AND of Numbers Range**	Medium	LC 201	1485
13	★ **Maximum XOR of Two Numbers in an Array**	Medium	LC 421	1487
14	Total Hamming Distance	Medium	LC 477	1490
15	UTF-8 Validation	Medium	LC 393	1492
16	Gray Code (see page)	Medium	LC 89	825
17	★ **Subsets** (see page)	Medium	LC 78	780
18	★ **Divide Two Integers**	Medium	LC 29	1495
19	Maximum Product of Word Lengths	Medium	LC 318	1498
20	Minimum Flips to Make a OR b Equal to c	Medium	LC 1318	1501

PATTERN 27 · BIT MANIPULATION · PROBLEM 1 OF 20

136. Single Number

Easy · Bit Manipulation, Array

leetcode.com/problems/single-number

Also appears in pattern 33 Cyclic Sort

PROBLEM

Given a **non-empty** array of integers `nums`, every element appears *twice* except for one. Find that single one.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:

Input: nums = [2,2,1]

Output: 1

Example 2:

Input: nums = [4,1,2,1,2]

Output: 4

Example 3:

Input: nums = [1]

Output: 1

Constraints:

- `1 <= nums.length <= 3 * 10^4`
- `-3 * 10^4 <= nums[i] <= 3 * 10^4`
- Each element in the array appears twice except for one element which appears only once.

SOLUTIONS (PYTHON 3)

Solution 1: Bitwise Operation

> **How to think about it**
>
> Every number appears twice except one; we need linear time and constant space. A frequency map uses O(n) space. XOR has xxor x=0, xxor 0=x, and is commutative, so pairing cancels and the leftover is the unique number.

The XOR operation has the following properties:

- Any number XOR 0 is still the original number, i.e., x xor 0 = x;
- Any number XOR itself is 0, i.e., x xor x = 0;

Performing XOR operation on all elements in the array will result in the number that only appears once.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Python 3

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        return reduce(xor, nums)
```

Solution 2

How to think about it

Solution 1 is already an XOR fold. This variant uses the language's reduction to do the same operation.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        answer = 0
        for x in nums:
            answer ^= x
        return answer
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The numbers 1..n appear once except one number that
# appears twice: find the repeated one
# 2. XOR of all the numbers from 0 to n in O(1)


# ============================================================
# FOLLOW-UP 1: The numbers 1..n appear once except one
# number that appears twice: find the repeated one
# ============================================================

# ---- follow-up 1: function ----
# Answer: XOR all the numbers of the array with all the
# numbers 1..n; every number cancels except the repeated
# one.
# Time: O(n).  Space: O(1).
def find_repeated(nums):
    n = len(nums) - 1
    result = 0
    for x in nums:
        result ^= x
    for i in range(1, n + 1):
        result ^= i
    return result


# ============================================================
# FOLLOW-UP 2: XOR of all the numbers from 0 to n in O(1)
# ============================================================

# ---- follow-up 2: function ----
# Answer: The XOR of 0..n repeats with period 4: n, 1, n +
# 1, 0 depending on n modulo 4.
# Time: O(1).  Space: O(1).
def xor_up_to(n):
    remainder = n % 4
    if remainder == 0:
        return n
    if remainder == 1:
        return 1
    if remainder == 2:
        return n + 1
    return 0
```

PATTERN 27 · BIT MANIPULATION · PROBLEM 2 OF 20

191. Number of 1 Bits

Easy · Bit Manipulation, Divide and Conquer

leetcode.com/problems/number-of-1-bits

PROBLEM

Given a positive integer `n`, write a function that returns the number of set bits in its binary representation (also known as the Hamming weight).

Example 1:

Input: n = 11

Output: 3

Explanation:

The input binary string **1011** has a total of three set bits.

Example 2:

Input: n = 128

Output: 1

Explanation:

The input binary string **10000000** has a total of one set bit.

Example 3:

Input: n = 2147483645

Output: 30

Explanation:

The input binary string **11111111111111111111111111111101** has a total of thirty set bits.

Constraints:

- `1 <= n <= 2^31 - 1`

Follow up:

If this function is called many times, how would you optimize it?

SOLUTIONS (PYTHON 3)

Solution 1

> **How to think about it**
>
> Count set bits. Checking every bit always looks at 32 positions. n&(n-1) clears the lowest 1, so the loop runs once per set bit.

Python 3

```python
class Solution:
    def hammingWeight(self, n: int) -> int:
        ans = 0
        while n:
            n &= n - 1
            ans += 1
        return ans
```

Complexity. Time: O(k), k = number of one bits (at most 32). Space: O(1).

Solution 2

> **How to think about it**
>
> Solution 1 uses n&(n-1). lowbit=n&-n isolates the lowest 1; subtract and repeat. Same idea, the form used in Fenwick trees.

Python 3

```python
class Solution:
    def hammingWeight(self, n: int) -> int:
        ans = 0
        while n:
            n -= n & -n
            ans += 1
        return ans
```

Complexity. Time: O(k), k = number of one bits (at most 32). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The Hamming distance between two numbers (the number
# of different bits)
# 2. The parity of a number (do the 1 bits make an even or
# an odd count?)


# ============================================================
# FOLLOW-UP 1: The Hamming distance between two numbers
# (the number of different bits)
# ============================================================

# ---- follow-up 1: function ----
# Answer: XOR the numbers: a bit is 1 exactly where they
# differ; count the ones.
# Time: O(b), b = 1 bits (at most 32).  Space: O(1).
def hamming_distance(a, b):
    x = a ^ b
    count = 0
    while x:
        x &= x - 1
        count += 1
    return count


# ============================================================
# FOLLOW-UP 2: The parity of a number (do the 1 bits make
# an even or an odd count?)
# ============================================================

# ---- follow-up 2: function ----
# Answer: XOR the bits into one bit; or fold the number in
# half repeatedly: x ^= x >> 16, then >> 8, >> 4, >> 2, >>
# 1.
# Time: O(1) (six shifts).  Space: O(1).
def parity(x):
    x ^= x >> 32
    x ^= x >> 16
    x ^= x >> 8
    x ^= x >> 4
    x ^= x >> 2
    x ^= x >> 1
    return x & 1
```

PATTERN 27 · BIT MANIPULATION · PROBLEM 3 OF 20

338. Counting Bits

Easy · Bit Manipulation, Dynamic Programming

leetcode.com/problems/counting-bits

PROBLEM

Given an integer `n`, return *an array* `ans` *of length* `n + 1` *such that for each* `i` (`0 <= i <= n`), `ans[i]` *is the* ***number of*** `1`***'s*** *in the binary representation of* `i`.

Do not solve it with built-in functions (i.e., like

`__builtin_popcount`

in C++).

Example 1:

```
Input: n = 2
Output: [0,1,1]
Explanation:
0 --> 0
1 --> 1
2 --> 10
```

Example 2:

```
Input: n = 5
Output: [0,1,1,2,1,2]
Explanation:
0 --> 0
1 --> 1
2 --> 10
3 --> 11
4 --> 100
5 --> 101
```

Constraints:

- `0 <= n <= 10^5`

Follow up:

- It is very easy to come up with a solution with a runtime of `O(n log n)`. Can you do it in linear time `O(n)` and possibly in a single pass?

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Count set bits of every integer in 0… n. Per-number bit scans cost O(log i). n≤ 10^5 still allows that, but we can do better later. Call `bit_count` (or repeated `lowbit`) on each i to fill the array.

Python 3

```python
class Solution:
    def countBits(self, n: int) -> List[int]:
        return [i.bit_count() for i in range(n + 1)]
```

Solution 2

How to think about it

Method 1 counts each integer from scratch. Clearing the lowest set bit yields i&(i-1), whose answer is already known, so ans[i]=ans[i&(i-1)]+1. Fill upward in O(n).

Python 3

```
class Solution:
    def countBits(self, n: int) -> List[int]:
        ans = [0] * (n + 1)
        for i in range(1, n + 1):
            ans[i] = ans[i & (i - 1)] + 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The total number of 1 bits in all the numbers from 0
# to n, in O(log n)
# 2. Return the numbers 0..n sorted by their number of 1
# bits (ties by value)


# ============================================================
# FOLLOW-UP 1: The total number of 1 bits in all the
# numbers from 0 to n, in O(log n)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Bit by bit: in the bit position with value 2^k
# the pattern 0...01...1 repeats every 2^(k+1) numbers,
# with 2^k ones in each period.
# Time: O(log n).  Space: O(1).
def total_one_bits(n):
    total = 0
    k = 0
    while (1 << k) <= n:
        period = 1 << (k + 1)
        full = (n + 1) // period
        total += full * (1 << k)
        rest = (n + 1) % period
        if rest > (1 << k):
            total += rest - (1 << k)
        k += 1
    return total


# ============================================================
# FOLLOW-UP 2: Return the numbers 0..n sorted by their
# number of 1 bits (ties by value)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Compute the counts with the table, then sort by
# the pair (count, value).
# Time: O(n log n).  Space: O(n).
def sort_by_one_bits(n):
    bits = [0] * (n + 1)
    for i in range(1, n + 1):
        bits[i] = bits[i >> 1] + (i & 1)
    numbers = list(range(n + 1))
    numbers.sort(key=lambda x: (bits[x], x))
    return numbers
```

PATTERN 27 · BIT MANIPULATION · PROBLEM 4 OF 20

268. Missing Number

Easy · Bit Manipulation, Array, Hash Table, Math, Binary Search, Sorting

leetcode.com/problems/missing-number

Also appears in pattern 33 Cyclic Sort

PROBLEM

Given an array `nums` containing `n` distinct numbers in the range `[0, n]`, return *the only number in the range that is missing from the array.*

Example 1:

Input: nums = [3,0,1]

Output: 2

Explanation:

`n = 3` since there are 3 numbers, so all numbers are in the range `[0,3]`. 2 is the missing number in the range since it does not appear in `nums`.

Example 2:

Input: nums = [0,1]

Output: 2

Explanation:

`n = 2` since there are 2 numbers, so all numbers are in the range `[0,2]`. 2 is the missing number in the range since it does not appear in `nums`.

Example 3:

Input: nums = [9,6,4,2,3,5,7,0,1]

Output: 8

Explanation:

`n = 9` since there are 9 numbers, so all numbers are in the range `[0,9]`. 8 is the missing number in the range since it does not appear in `nums`.

Constraints:

- `n == nums.length`
- `1 <= n <= 10^4`
- `0 <= nums[i] <= n`
- All the numbers of `nums` are **unique**.

Follow up: Could you implement a solution using only `O(1)` extra space complexity and `O(n)` runtime complexity?

SOLUTIONS (PYTHON 3)

Solution 1: Bitwise Operation

> **How to think about it**
>
> One value from [0,n] is missing. XOR the indices 1..n with the array entries; pairs cancel and the missing number remains.

The XOR operation has the following properties:

- Any number XOR 0 is still the original number, i.e., x xor 0 = x;
- Any number XOR itself is 0, i.e., x xor x = 0;

Therefore, we can traverse the array, perform XOR operation between each element and the numbers [0,..n], and the final result will be the missing number.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Python 3

```
class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        return reduce(xor, (i ^ v for i, v in enumerate(nums, 1)))
```

Solution 2: Mathematics

> **How to think about it**
>
> XOR avoids addition. Gauss's formula also works: n(n+1)/2 minus the array sum is the missing value.

We can also solve this problem using mathematics. By calculating the sum of [0,..n], subtracting the sum of all numbers in the array, we can obtain the missing number.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Python 3

```
class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        n = len(nums)
        return (1 + n) * n // 2 - sum(nums)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. TWO numbers are missing from 1..n
# 2. The array is sorted: find the missing number in O(log
# n)


# ============================================================
# FOLLOW-UP 1: TWO numbers are missing from 1..n
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sum and sum of squares (or XOR split by a
# differing bit): the sum of the two missing numbers and a
# bit that separates them identify both.
# Time: O(n).  Space: O(1).
def two_missing(nums, n):
    xor_all = 0
    for x in nums:
        xor_all ^= x
    for i in range(1, n + 1):
        xor_all ^= i                                # now xor_all = a ^ b for the missing a, b
    bit = xor_all & -xor_all
    a = 0
    b = 0
    for x in nums:
        if x & bit:
            a ^= x
        else:
            b ^= x
    for i in range(1, n + 1):
        if i & bit:
            a ^= i
        else:
            b ^= i
    return sorted([a, b])
```

```
# ============================================================
# FOLLOW-UP 2: The array is sorted: find the missing
# number in O(log n)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Binary search for the first index whose value is
# not equal to the index.
# Time: O(log n).  Space: O(1).
def missing_in_sorted(nums):
    low, high = 0, len(nums)
    while low < high:
        mid = (low + high) // 2
        if nums[mid] == mid:
            low = mid + 1
        else:
            high = mid
    return low
```

190. Reverse Bits

Easy · Bit Manipulation, Divide and Conquer

leetcode.com/problems/reverse-bits

PROBLEM

Reverse bits of a given 32 bits signed integer.

Example 1:

Input: n = 43261596

Output: 964176192

Explanation:

Integer	Binary
43261596	00000010100101000001111010011100
964176192	00111001011110000010100101000000

Example 2:

Input: n = 2147483644

Output: 1073741822

Explanation:

Integer	Binary
2147483644	01111111111111111111111111111100
1073741822	00111111111111111111111111111110

Constraints:

- `0 <= n <= 2^31 - 2`
- `n` is even.

Follow up: If this function is called many times, how would you optimize it?

SOLUTION (PYTHON 3)

Solution 1: Bit Manipulation

> **How to think about it**
>
> Reverse the 32 bits of an unsigned integer. Pull each bit and write it to the mirrored index. Scan 32 times from the low end: take n's lowest bit, write it at position 31-i, then shift n right. The width is fixed.

We can extract each bit of n from the lowest bit to the highest bit, and then place it at the corresponding position of ans.

For example, for the i-th bit, we can extract the i-th bit of n and place it at the (31 - i)-th bit of ans by (n & 1) ≪ (31 - i), and then right shift n by one bit.

The time complexity is O(log n), and the space complexity is O(1).

Python 3

```python
class Solution:
    def reverseBits(self, n: int) -> int:
        ans = 0
        for i in range(32):
            ans |= (n & 1) << (31 - i)
            n >>= 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Reverse the BYTES of a 32-bit number (swap the
# endianness)
# 2. Reverse the bits of MANY numbers quickly: cache the
# reversed bytes


# ============================================================
# FOLLOW-UP 1: Reverse the BYTES of a 32-bit number (swap
# the endianness)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Move each byte to its mirrored place with shifts
# and masks.
# Time: O(1).  Space: O(1).
def swap_endianness(x):
    return ((x & 0xFF) << 24) | ((x & 0xFF00) << 8) | ((x >> 8) & 0xFF00) | ((x >> 24) & 0xFF)


# ============================================================
# FOLLOW-UP 2: Reverse the bits of MANY numbers quickly:
# cache the reversed bytes
# ============================================================

# ---- follow-up 2: function ----
# Answer: Precompute the reversal of every byte (256
# values); a 32-bit number is then four table lookups.
# Time: O(1) per number after a one-time O(256) table.
# Space: O(256).
def build_byte_table():
    table = []
    for value in range(256):
        reversed_value = 0
        for i in range(8):
            reversed_value = (reversed_value << 1) | ((value >> i) & 1)
        table.append(reversed_value)
    return table


BYTE_TABLE = build_byte_table()


def reverse_bits_cached(x):
    return (BYTE_TABLE[x & 0xFF] << 24) | (BYTE_TABLE[(x >> 8) & 0xFF] << 16) | (BYTE_TABLE[(x >> 16) &
0xFF] << 8) | BYTE_TABLE[(x >> 24) & 0xFF]
```

389. Find the Difference

Easy · Bit Manipulation, Hash Table, String, Sorting

leetcode.com/problems/find-the-difference

Also appears in pattern 33 Cyclic Sort

PROBLEM

You are given two strings `s` and `t`.

String `t` is generated by random shuffling string `s` and then add one more letter at a random position.

Return the letter that was added to `t`.

Example 1:

```
Input: s = "abcd", t = "abcde"
Output: "e"
Explanation: 'e' is the letter that was added.
```

Example 2:

```
Input: s = "", t = "y"
Output: "y"
```

Constraints:

- `0 <= s.length <= 1000`
- `t.length == s.length + 1`
- `s` and `t` consist of lowercase English letters.

SOLUTIONS (PYTHON 3)

Solution 1: Counting

How to think about it

t is s shuffled plus one letter. A count difference finds it. Count s, decrement along t; the first negative frequency is the extra letter.

We can use a hash table or array cnt to count the occurrence of each character in string s, then traverse string t. For each character, we subtract its occurrence in cnt. If the corresponding count is negative, it means that the occurrence of this character in t is greater than in s, so this character is the added character.

The time complexity is O(n), and the space complexity is O(|Σ|), where n is the length of the string, and Σ represents the character set. Here the character set is all lowercase letters, so |Σ|=26.

Python 3

```python
class Solution:
    def findTheDifference(self, s: str, t: str) -> str:
        cnt = Counter(s)
        for c in t:
            cnt[c] -= 1
            if cnt[c] < 0:
                return c
```

Solution 2: Summation

> **How to think about it**
>
> Counting uses O(Σ) space. The difference of ASCII sums is the extra code point, in O(1) space.

We can sum the ASCII values of each character in string t, then subtract the sum of the ASCII values of each character in string s. The final result is the ASCII value of the added character.

The time complexity is O(n), where n is the length of the string. The space complexity is O(1).

Python 3

```
class Solution:
    def findTheDifference(self, s: str, t: str) -> str:
        a = sum(ord(c) for c in s)
        b = sum(ord(c) for c in t)
        return chr(b - a)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. A letter was REMOVED (not added): which one?
# 2. Which letter appears an ODD number of times in a
# string?


# ============================================================
# FOLLOW-UP 1: A letter was REMOVED (not added): which
# one?
# ============================================================

# ---- follow-up 1: function ----
# Answer: XOR the letters of both strings; the letter left
# is the removed one.
# Time: O(n).  Space: O(1).
def removed_letter(original, shorter):
    result = 0
    for ch in original:
        result ^= ord(ch)
    for ch in shorter:
        result ^= ord(ch)
    return chr(result)


# ============================================================
# FOLLOW-UP 2: Which letter appears an ODD number of times
# in a string?
# ============================================================

# ---- follow-up 2: function ----
# Answer: XOR all the letters: letters that appear an even
# number of times cancel out.
# Time: O(n).  Space: O(1).
def odd_letter(text):
    result = 0
    for ch in text:
        result ^= ord(ch)
    return chr(result)
```

693. Binary Number with Alternating Bits

Easy · Bit Manipulation

leetcode.com/problems/binary-number-with-alternating-bits

PROBLEM

Given a positive integer, check whether it has alternating bits: namely, if two adjacent bits will always have different values.

Example 1:

```
Input: n = 5
Output: true
Explanation: The binary representation of 5 is: 101
```

Example 2:

```
Input: n = 7
Output: false
Explanation: The binary representation of 7 is: 111.
```

Example 3:

```
Input: n = 11
Output: false
Explanation: The binary representation of 11 is: 1011.
```

Constraints:

- $1 <= n <= 2^{31} - 1$

SOLUTIONS (PYTHON 3)

Solution 1: Simulation

How to think about it

Check that neighboring bits strictly alternate. A bit-by-bit scan is enough. Read the lowest bit, shift, and fail if it equals the previous bit.

We cyclically right-shift n until it becomes 0, checking whether the binary bits of n appear alternately. If during the loop we find that 0 and 1 do not appear alternately, we directly return false. Otherwise, when the loop ends, we return true.

The time complexity is O(log n), and the space complexity is O(1).

Python 3

```python
class Solution:
    def hasAlternatingBits(self, n: int) -> bool:
        prev = -1
        while n:
            curr = n & 1
            if prev == curr:
                return False
            prev = curr
            n >>= 1
        return True
```

Solution 2: Bit Manipulation

> **How to think about it**
> The scan is O(log n). nxor(n≫ 1) becomes a run of ones iff bits alternate; then n&(n+1)=0 confirms it in constant time.

Assuming 01 appears alternately, we can convert all trailing bits to 1 through misaligned XOR. Adding 1 gives us a power of 2, which is a number n (where n has only one bit that is 1). Then, using n & (n + 1) can eliminate the last 1 bit.

At this point, check if it equals 0. If so, the assumption holds, and it is an alternating 01 sequence.

The time complexity is O(1), and the space complexity is O(1).

Python 3

```
class Solution:
    def hasAlternatingBits(self, n: int) -> bool:
        n ^= n >> 1
        return (n & (n + 1)) == 0
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Count the numbers from 1 to n whose bits alternate
# 2. The next bigger number with alternating bits


# ============================================================
# FOLLOW-UP 1: Count the numbers from 1 to n whose bits
# alternate
# ============================================================

# ---- follow-up 1: function ----
# Answer: There is exactly one alternating number of every
# length (it starts with a 1): 1, 2, 5, 10, 21, 42, ...
# Build them one from the other and count those that are
# at most n.
# Time: O(log n).  Space: O(log n).
def alternating_numbers_up_to(limit):
    numbers = []
    value = 1
    while value <= limit:
        numbers.append(value)
        value = value * 2 + (1 - (value & 1))        # append the opposite of the last bit
    return numbers


def count_alternating_up_to(n):
    return len(alternating_numbers_up_to(n))


# ============================================================
# FOLLOW-UP 2: The next bigger number with alternating
# bits
# ============================================================

# ---- follow-up 2: function ----
# Answer: Generate the alternating numbers in order and
# return the first one that is bigger than n.
# Time: O(log n).  Space: O(1).
def next_alternating(n):
    value = 1
    while value <= n:
        value = value * 2 + (1 - (value & 1))
    return value
```

PATTERN 27 · BIT MANIPULATION · PROBLEM 8 OF 20

1486. XOR Operation in an Array

Easy · Bit Manipulation, Math

leetcode.com/problems/xor-operation-in-an-array

PROBLEM

You are given an integer `n` and an integer `start`.

Define an array `nums` where `nums[i] = start + 2 * i` (**0-indexed**) and `n == nums.length`.

Return *the bitwise XOR of all elements of* `nums`.

Example 1:

```
Input: n = 5, start = 0
Output: 8
Explanation: Array nums is equal to [0, 2, 4, 6, 8] where (0 ^ 2 ^ 4 ^ 6 ^ 8) = 8.
Where "^" corresponds to bitwise XOR operator.
```

Example 2:

```
Input: n = 4, start = 3
Output: 8
Explanation: Array nums is equal to [3, 5, 7, 9] where (3 ^ 5 ^ 7 ^ 9) = 8.
```

Constraints:

- `1 <= n <= 1000`
- `0 <= start <= 1000`
- `n == nums.length`

SOLUTION (PYTHON 3)

Solution 1: Simulation

> **How to think about it**
>
> n≤ 1000. Emit start+2i and XOR the values together.

We can directly simulate to calculate the XOR result of all elements in the array.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Python 3

```python
class Solution:
    def xorOperation(self, n: int, start: int) -> int:
        return reduce(xor, ((start + 2 * i) for i in range(n)))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. O(1): the XOR of start + 2i for i = 0..n-1


# ============================================================
# FOLLOW-UP 1: O(1): the XOR of start + 2i for i = 0..n-1
# ============================================================

# ---- follow-up 1: helper functions ----
def xor_up_to(n):
```

```
    remainder = n % 4
    if remainder == 0:
        return n
    if remainder == 1:
        return 1
    if remainder == 2:
        return n + 1
    return 0

# ---- follow-up 1: function ----
# Answer: Write start = 2s + b: the numbers are 2(s + i) +
# b, so the answer is 2 * (XOR of s..s+n-1) + (b if n is
# odd), and the XOR of a range uses the prefix formula.
# Time: O(1).  Space: O(1).
def xor_operation_formula(n, start):
    s = start // 2
    b = start & 1
    range_xor = xor_up_to(s + n - 1) ^ xor_up_to(s - 1)
    answer = range_xor * 2
    if b == 1 and n % 2 == 1:
        answer += 1
    return answer
```

PATTERN 27 · BIT MANIPULATION · PROBLEM 9 OF 20

137. Single Number II

Medium · Bit Manipulation, Array

leetcode.com/problems/single-number-ii

★ **TOP 7 PROBLEM** in Bit Manipulation: master this one first

PROBLEM

Given an integer array `nums` where every element appears **three times** except for one, which appears **exactly once**. *Find the single element and return it*.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:

```
Input: nums = [2,2,3,2]
Output: 3
```

Example 2:

```
Input: nums = [0,1,0,1,0,1,99]
Output: 99
```

Constraints:

- `1 <= nums.length <= 3 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`
- Each element in `nums` appears exactly **three times** except for one element which appears **once**.

SOLUTIONS (PYTHON 3)

Solution 1: Bitwise Operation

How to think about it

Numbers appear three times except one; XOR is no longer enough because xxor xxor x=x. The follow-up still wants constant space. Count 1s per bit modulo 3: bits from the triples vanish, and the remainder is the answer. The sign bit is handled separately to avoid overflow.

We can enumerate each binary bit i, and for each binary bit, we calculate the sum of all numbers on that bit. If the sum of the numbers on that bit can be divided by 3, then the number that only appears once on that bit is 0, otherwise it is 1.

The time complexity is O(n × log M), where n and M are the length of the array and the range of elements in the array, respectively. The space complexity is O(1).

The C++ implementation adds INT_MIN for the sign bit instead of left-shifting a signed integer into overflow.

Python 3

```python
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        ans = 0
        for i in range(32):
            cnt = sum(num >> i & 1 for num in nums)
            if cnt % 3:
                if i == 31:
                    ans -= 1 << i
                else:
                    ans |= 1 << i
        return ans
```

Solution 2: Digital Circuit

> **How to think about it**
>
> Solution 1 rescans the array for every bit. Two integers a,b store each bit's count modulo 3; a truth table updates them as each number arrives, so one pass does the same counting.

We can use a more efficient method that uses digital circuits to simulate the above bitwise operation.

Each binary bit of an integer can only represent 2 states, 0 or 1. However, we need to represent the sum of the i-th bit of all integers traversed so far modulo 3. Therefore, we can use two integers a and b to represent it. There are three possible cases:

1. The i-th bit of integer a is 0 and the i-th bit of integer b is 0, which means the modulo 3 result is 0;
2. The i-th bit of integer a is 0 and the i-th bit of integer b is 1, which means the modulo 3 result is 1;
3. The i-th bit of integer a is 1 and the i-th bit of integer b is 0, which means the modulo 3 result is 2.

We use integer c to represent the number to be read in, and the truth table is as follows:

a_i	b_i	c_i	New a_i	New b_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	1	0
1	0	1	0	0

Based on the truth table, we can write the logical expression:

$$a_i = a_i' \, b_i \, c_i + a_i \, b_i' \, c_i'$$

and:

$$b_i = a_i' \, b_i' \, c_i + a_i' \, b_i \, c_i' = a_i' \, (b_i \text{ xor } c_i)$$

The final result is b, because when the binary bit of b is 1, it means that the number appears only once.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Python 3

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        a = b = 0
        for c in nums:
            aa = (~a & b & c) | (a & ~b & ~c)
            bb = ~a & (b ^ c)
            a, b = aa, bb
        return b
```

Solution 3: Set + Math

> **How to think about it**
>
> If extra space is allowed, the unique value is $(3\sum_{unique}-\sum_{all})/2$. Deduplicate, sum twice, and divide. Direct, but O(n) space.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        return (3 * sum(set(nums)) - sum(nums)) // 2
```

Solution 4: Bit Manipulation

How to think about it

The Solution 2 state machine collapses to two masks: ans and acc hold bits seen once and twice. They stay disjoint as x arrives; ans is the number that appeared once. Shorter code.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        ones = twos = 0
        for x in nums:
            ones ^= x & ~twos
            twos ^= x & ~ones
        return ones
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Every number appears k times except one that appears
# once
# 2. One pass with a small state machine (two counters of
# 2 bits)


# ============================================================
# FOLLOW-UP 1: Every number appears k times except one
# that appears once
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count how many numbers have each bit set; a bit
# of the answer is 1 when that count is not a multiple of
# k.
# Time: O(32 * n).  Space: O(1).
def single_number_k(nums, k):
    result = 0
    for bit in range(32):
        count = 0
        for x in nums:
            if (x >> bit) & 1:
                count += 1
        if count % k != 0:
            result |= 1 << bit
    if result >= 1 << 31:
        result -= 1 << 32
    return result


# ============================================================
# FOLLOW-UP 2: One pass with a small state machine (two
# counters of 2 bits)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep two numbers, ones and twos, that together
# count each bit's occurrences modulo 3; the answer is
# left in ones.
# Time: O(n).  Space: O(1).
def single_number_state_machine(nums):
    ones = 0
    twos = 0
    for x in nums:
        ones = (ones ^ x) & ~twos
        twos = (twos ^ x) & ~ones
```

```
    return ones
```

260. Single Number III

Medium · Bit Manipulation, Array

leetcode.com/problems/single-number-iii

★ **TOP 7 PROBLEM** in Bit Manipulation: master this one first

PROBLEM

Given an integer array `nums`, in which exactly two elements appear only once and all the other elements appear exactly twice. Find the two elements that appear only once. You can return the answer in **any order**.

You must write an algorithm that runs in linear runtime complexity and uses only constant extra space.

Example 1:

```
Input: nums = [1,2,1,3,2,5]
Output: [3,5]
Explanation: [5, 3] is also a valid answer.
```

Example 2:

```
Input: nums = [-1,0]
Output: [-1,0]
```

Example 3:

```
Input: nums = [0,1]
Output: [1,0]
```

Constraints:

- `2 <= nums.length <= 3 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`
- Each integer in `nums` will appear twice, only two integers will appear once.

SOLUTIONS (PYTHON 3)

Solution 1: Bitwise Operation

How to think about it

All but two values appear twice, so the XOR of the array is the XOR xs of those two. They differ, so xs has a 1 bit. Group by the lowbit of xs; the two targets land in different groups. XOR the group that has that bit to get a, then b=xsxor a.

The XOR operation has the following properties:

- Any number XOR 0 is still the original number, i.e., x xor 0 = x;
- Any number XOR itself is 0, i.e., x xor x = 0;

Since all numbers in the array appear twice except for two numbers, we can perform XOR operation on all numbers in the array to get the XOR result of the two numbers that only appear once.

Since these two numbers are not equal, there is at least one bit that is 1 in the XOR result. We can use the `lowbit` operation to find the lowest bit of 1 in the XOR result, and divide all numbers in the array into two groups based on whether this bit is 1 or not. This way, the two numbers that only appear once are separated into different groups.

Perform XOR operation on each group separately to obtain the two numbers a and b that only appear once.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Python 3

```
class Solution:
    def singleNumber(self, nums: List[int]) -> List[int]:
        xs = reduce(xor, nums)
        a = 0
        lb = xs & -xs
        for x in nums:
            if x & lb:
                a ^= x
        b = xs ^ a
        return [a, b]
```

Solution 2: Hash Table

> **How to think about it**
>
> The bit method is constant-space. With linear extra memory, a set toggles membership: insert on the first sighting, delete on the second. The remainder is the pair of unique numbers.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```
class Solution:
    def singleNumber(self, nums: List[int]) -> List[int]:
        seen = set()
        for x in nums:
            if x in seen:
                seen.remove(x)
            else:
                seen.add(x)
        return list(seen)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Find the two numbers with a hash set
# 2. The numbers may be negative: separate them with the
# lowest set bit of the XOR


# ============================================================
# FOLLOW-UP 1: Find the two numbers with a hash set
# ============================================================

# ---- follow-up 1: function ----
# Answer: Add a number to a set the first time and remove
# it the second time; what remains are the two singles.
# Time: O(n).  Space: O(n).
def two_singles_with_set(nums):
    seen = set()
    for x in nums:
        if x in seen:
            seen.remove(x)
        else:
            seen.add(x)
    return sorted(seen)


# ============================================================
# FOLLOW-UP 2: The numbers may be negative: separate them
# with the lowest set bit of the XOR
# ============================================================

# ---- follow-up 2: function ----
# Answer: The lowest set bit of a ^ b is a bit where the
```

```
# two singles differ; it splits all numbers into two
# groups, each holding exactly one single.
# Time: O(n).  Space: O(1).
def two_singles_xor(nums):
    xor_all = 0
    for x in nums:
        xor_all ^= x
    bit = xor_all & -xor_all
    a = 0
    b = 0
    for x in nums:
        if x & bit:
            a ^= x
        else:
            b ^= x
    return sorted([a, b])
```

PATTERN 27 · BIT MANIPULATION · PROBLEM 11 OF 20

371. Sum of Two Integers

Medium · Bit Manipulation, Math

leetcode.com/problems/sum-of-two-integers

★ **TOP 7 PROBLEM** in Bit Manipulation: master this one first

PROBLEM

Given two integers `a` and `b`, return *the sum of the two integers without using the operators* `+` *and* `-`.

Example 1:

```
Input: a = 1, b = 2
Output: 3
```

Example 2:

```
Input: a = 2, b = 3
Output: 5
```

Constraints:

- `-1000 <= a, b <= 1000`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Add without +/−. Bitwise: the sum without carry is XOR, the carry is AND shifted left. Repeat until the carry vanishes. Python ints are unbounded, so mask with 0xFFFFFFFF to 32 bits. If the sign bit is set, convert two's complement back to a negative.

Python 3

```python
class Solution:
    def getSum(self, a: int, b: int) -> int:
        a, b = a & 0xFFFFFFFF, b & 0xFFFFFFFF
        while b:
            carry = ((a & b) << 1) & 0xFFFFFFFF
            a, b = a ^ b, carry
        return a if a < 0x80000000 else ~(a ^ 0xFFFFFFFF)
```

Complexity. Time: O(1) (at most 32 rounds). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Subtract two integers without using - or +
# 2. Multiply two non-negative integers with shifts and
# additions only


# ============================================================
# FOLLOW-UP 1: Subtract two integers without using - or +
# ============================================================

# ---- follow-up 1: function ----
# Answer: a - b = a + (-b), and -b is the two's complement
# ~b + 1; use the same add loop for both additions (with a
```

```
# 32-bit mask in Python).
# Time: O(1) (at most 32 rounds).  Space: O(1).
def add_bits(a, b):
    mask = 0xFFFFFFFF
    while b != 0:
        a, b = (a ^ b) & mask, ((a & b) << 1) & mask
    if a > 0x7FFFFFFF:
        a = ~(a ^ mask)
    return a


def subtract_bits(a, b):
    return add_bits(a, add_bits(~b, 1))


# ============================================================
# FOLLOW-UP 2: Multiply two non-negative integers with
# shifts and additions only
# ============================================================

# ---- follow-up 2: function ----
# Answer: Russian peasant multiplication: when the lowest
# bit of b is 1 add a to the result; shift a left and b
# right.
# Time: O(log b).  Space: O(1).
def multiply_bits(a, b):
    result = 0
    while b > 0:
        if b & 1:
            result += a
        a <<= 1
        b >>= 1
    return result
```

201. Bitwise AND of Numbers Range

Medium · Bit Manipulation

leetcode.com/problems/bitwise-and-of-numbers-range

★ **TOP 7 PROBLEM** in Bit Manipulation: master this one first

PROBLEM

Given two integers `left` and `right` that represent the range `[left, right]`, return *the bitwise AND of all numbers in this range, inclusive*.

Example 1:

```
Input: left = 5, right = 7
Output: 4
```

Example 2:

```
Input: left = 0, right = 0
Output: 0
```

Example 3:

```
Input: left = 1, right = 2147483647
Output: 0
```

Constraints:

- `0 <= left <= right <= 2^31 - 1`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

AND-ing every integer in [left,right] is impossible when the span approaches 2^{31}. The bitwise AND of a contiguous range is their common binary prefix: lower bits are cleared by some number in the range. While left < right, clear the lowest set bit of right (right &= right-1) until right ≤ left. The remaining right is that common prefix.

Python 3

```python
class Solution:
    def rangeBitwiseAnd(self, left: int, right: int) -> int:
        while left < right:
            right &= right - 1
        return right
```

Complexity. Time: O(log right). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The bitwise OR of all the numbers in the range
# 2. The bitwise XOR of all the numbers in the range


# ========================================================
# FOLLOW-UP 1: The bitwise OR of all the numbers in the
# range
```

```
# ============================================================

# ---- follow-up 1: function ----
# Answer: All the bits below the highest bit in which left
# and right differ become 1; the common prefix stays.
# Time: O(log n).  Space: O(1).
def range_bitwise_or(left, right):
    shift = 0
    while left != right:
        left >>= 1
        right >>= 1
        shift += 1
    return (left << shift) | ((1 << shift) - 1)


# ============================================================
# FOLLOW-UP 2: The bitwise XOR of all the numbers in the
# range
# ============================================================

# ---- follow-up 2: function ----
# Answer: XOR of 0..right XOR XOR of 0..left - 1, each
# from the period-4 formula.
# Time: O(1).  Space: O(1).
def prefix_xor(n):
    remainder = n % 4
    if remainder == 0:
        return n
    if remainder == 1:
        return 1
    if remainder == 2:
        return n + 1
    return 0


def range_bitwise_xor(left, right):
    return prefix_xor(right) ^ prefix_xor(left - 1)
```

PATTERN 27 · BIT MANIPULATION · PROBLEM 13 OF 20

421. Maximum XOR of Two Numbers in an Array

Medium · Bit Manipulation, Trie, Array, Hash Table

leetcode.com/problems/maximum-xor-of-two-numbers-in-an-array

★ **TOP 7 PROBLEM** in Bit Manipulation, Trie (Prefix Tree): master this one first

Also appears in pattern 32 Trie (Prefix Tree)

PROBLEM

Given an integer array `nums`, return *the maximum result of* `nums[i] XOR nums[j]`, where `0 <= i <= j < n`.

Example 1:

```
Input: nums = [3,10,5,25,2,8]
Output: 28
Explanation: The maximum result is 5 XOR 25 = 28.
```

Example 2:

```
Input: nums = [14,70,53,83,49,91,36,80,92,51,66,70]
Output: 127
```

Constraints:

- `1 <= nums.length <= 2 * 10^5`
- `0 <= nums[i] <= 2^31 - 1`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Trying every pair is $O(n^2)$ and n can be 2×10^5. A large XOR wants the opposite bit as early as possible. Insert each number into a binary trie from the high bit. When querying x, prefer the child with the flipped bit so that bit of the answer becomes 1. Thirty-one bits cover the value range. Each query walks the trie once and is equivalent to choosing the best partner among the inserted numbers.

Python 3

```python
class Trie:
    __slots__ = ("children",)

    def __init__(self):
        self.children: List[Trie | None] = [None, None]

    def insert(self, x: int):
        node = self
        for i in range(30, -1, -1):
            v = x >> i & 1
            if node.children[v] is None:
                node.children[v] = Trie()
            node = node.children[v]

    def search(self, x: int) -> int:
        node = self
        ans = 0
        for i in range(30, -1, -1):
            v = x >> i & 1
            if node.children[v ^ 1]:
                ans |= 1 << i
                node = node.children[v ^ 1]
            else:
                node = node.children[v]
        return ans


class Solution:
    def findMaximumXOR(self, nums: List[int]) -> int:
        trie = Trie()
        for x in nums:
            trie.insert(x)
        return max(trie.search(x) for x in nums)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the pair that gives the maximum XOR
# 2. Queries: for a given number q, the maximum XOR of q
# with a number of the array


# ============================================================
# FOLLOW-UP 1: Return the pair that gives the maximum XOR
# ============================================================

# ---- follow-up 1: function ----
# Answer: Insert every number into a binary trie; for each
# number walk the trie choosing the opposite bit whenever
# possible, and remember the best partner.
# Time: O(32 * n).  Space: O(32 * n).
def best_xor_pair(nums):
    root = {}
    for x in nums:
        node = root
        for bit in range(31, -1, -1):
            b = (x >> bit) & 1
            if b not in node:
                node[b] = {}
            node = node[b]
        node["value"] = x
    best = (-1, None, None)
    for x in nums:
        node = root
        for bit in range(31, -1, -1):
```

```
            b = (x >> bit) & 1
            if (1 - b) in node:
                node = node[1 - b]
            else:
                node = node[b]
        partner = node["value"]
        if x ^ partner > best[0]:
            best = (x ^ partner, x, partner)
    return best


# ============================================================
# FOLLOW-UP 2: Queries: for a given number q, the maximum
# XOR of q with a number of the array
# ============================================================

# ---- follow-up 2: function ----
# Answer: Build the trie once; every query is one walk of
# 32 steps.
# Time: build O(32 * n), each query O(32).  Space: O(32 *
# n).
def build_trie(nums):
    root = {}
    for x in nums:
        node = root
        for bit in range(31, -1, -1):
            b = (x >> bit) & 1
            if b not in node:
                node[b] = {}
            node = node[b]
        node["value"] = x
```

```
    return root


def max_xor_with(root, q):
    node = root
    for bit in range(31, -1, -1):
        b = (q >> bit) & 1
        if (1 - b) in node:
            node = node[1 - b]
        else:
            node = node[b]
    return q ^ node["value"]
```

477. Total Hamming Distance

Medium · Bit Manipulation, Array, Math

leetcode.com/problems/total-hamming-distance

PROBLEM

The Hamming distance between two integers is the number of positions at which the corresponding bits are different.

Given an integer array `nums`, return *the sum of **Hamming distances** between all the pairs of the integers in* `nums`.

Example 1:

```
Input: nums = [4,14,2]
Output: 6
Explanation: In binary representation, the 4 is 0100, 14 is 1110, and 2 is 0010 (just
showing the four bits relevant in this case).
The answer will be:
HammingDistance(4, 14) + HammingDistance(4, 2) + HammingDistance(14, 2) = 2 + 2 + 2 = 6.
```

Example 2:

```
Input: nums = [4,14,4]
Output: 4
```

Constraints:

- `1 <= nums.length <= 10^4`
- `0 <= nums[i] <= 10^9`
- The answer for the given input will fit in a **32-bit** integer.

SOLUTION (PYTHON 3)

Solution 1: Bit Manipulation

How to think about it

Sum Hamming distances over every pair. XORing all pairs is O(n^2). Bits contribute independently. On bit i, if a numbers have a 1, then n-a have a 0, and the bit adds a(n-a). Sum over 32 bits. After splitting by bit, each pair that differs on that bit is counted once; pairs are never enumerated.

We enumerate each bit in the range [0, 31]. For the current enumerated bit i, we count the number of numbers where the i-th bit is 1, denoted as a. Therefore, the number of numbers where the i-th bit is 0 is b = n - a, where n is the length of the array. In this way, the sum of the Hamming distance on the i-th bit is a × b. We add the Hamming distances of all bits to get the answer.

The time complexity is O(n × log M), where n and M are the length of the array and the maximum value in the array, respectively. The space complexity is O(1).

Python 3

```python
class Solution:
    def totalHammingDistance(self, nums: List[int]) -> int:
        ans, n = 0, len(nums)
        for i in range(32):
            a = sum(x >> i & 1 for x in nums)
            b = n - a
            ans += a * b
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The sum of the AND of every pair
# 2. The sum of the OR of every pair


# ============================================================
# FOLLOW-UP 1: The sum of the AND of every pair
# ============================================================

# ---- follow-up 1: function ----
# Answer: Per bit position: a pair contributes 2^bit when
# both numbers have the bit, so add count * (count - 1) /
# 2 times 2^bit.
# Time: O(32 * n).  Space: O(1).
def sum_of_pair_ands(nums):
    total = 0
    for bit in range(32):
        ones = 0
        for x in nums:
            if (x >> bit) & 1:
                ones += 1
        total += (ones * (ones - 1) // 2) * (1 << bit)
    return total


# ============================================================
# FOLLOW-UP 2: The sum of the OR of every pair
# ============================================================

# ---- follow-up 2: function ----
# Answer: A pair contributes 2^bit unless both numbers
# have a 0 there: pairs minus pairs of zeros.
# Time: O(32 * n).  Space: O(1).
def sum_of_pair_ors(nums):
    n = len(nums)
    total = 0
    for bit in range(32):
        zeros = 0
        for x in nums:
            if not (x >> bit) & 1:
                zeros += 1
        pairs = n * (n - 1) // 2
        total += (pairs - zeros * (zeros - 1) // 2) * (1 << bit)
    return total
```

PATTERN 27 · BIT MANIPULATION · PROBLEM 15 OF 20

393. UTF-8 Validation

Medium · Bit Manipulation, Array

leetcode.com/problems/utf-8-validation

PROBLEM

Given an integer array `data` representing the data, return whether it is a valid **UTF-8** encoding (i.e. it translates to a sequence of valid UTF-8 encoded characters).

A character in **UTF8** can be from **1 to 4 bytes** long, subjected to the following rules:

1. For a **1-byte** character, the first bit is a `0`, followed by its Unicode code.
2. For an **n-bytes** character, the first `n` bits are all one's, the `n + 1` bit is `0`, followed by `n - 1` bytes with the most significant `2` bits being `10`.

This is how the UTF-8 encoding would work:

```
   Number of Bytes   | UTF-8 Octet Sequence
                     | (binary)
   --------------------+-----------------------------------------
            1 | 0xxxxxxx
            2 | 110xxxxx 10xxxxxx
            3 | 1110xxxx 10xxxxxx 10xxxxxx
            4 | 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
```

`x` denotes a bit in the binary form of a byte that may be either `0` or `1`.

Note: The input is an array of integers. Only the **least significant 8 bits** of each integer is used to store the data. This means each integer represents only 1 byte of data.

Example 1:

```
Input: data = [197,130,1]
Output: true
Explanation: data represents the octet sequence: 11000101 10000010 00000001.
It is a valid utf-8 encoding for a 2-bytes character followed by a 1-byte character.
```

Example 2:

```
Input: data = [235,140,4]
Output: false
Explanation: data represented the octet sequence: 11101011 10001100 00000100.
The first 3 bits are all one's and the 4th bit is 0 means it is a 3-bytes character.
The next byte is a continuation byte which starts with 10 and that's correct.
But the second continuation byte does not start with 10, so it is invalid.
```

Constraints:

- `1 <= data.length <= 2 * 10^4`
- `0 <= data[i] <= 255`

SOLUTION (PYTHON 3)

Solution 1: Single Pass

How to think about it

Validate a UTF-8 byte stream given as integers. The leading prefix dictates how many `10xxxxxx` bytes follow. One pass is enough. `cnt` is remaining continuation bytes, which must be `10xxxxxx`. Otherwise decode a 1–4-byte header and set `cnt`. Fail on a bad prefix; success requires `cnt=0` at the end.

We use a variable cnt to record the current number of bytes that need to be filled starting with 10, initially cnt = 0.

For each integer v in the array:

- If cnt > 0, then check if v starts with 10. If not, return `false`, otherwise decrement cnt.
- If the highest bit of v is 0, then cnt = 0.
- If the highest two bits of v are 110, then cnt = 1.
- If the highest three bits of v are 1110, then cnt = 2.
- If the highest four bits of v are 11110, then cnt = 3.
- Otherwise, return `false`.

Finally, if cnt = 0, return `true`, otherwise return `false`.

The time complexity is O(n), where n is the length of the array `data`. The space complexity is O(1).

Python 3

```
class Solution:
    def validUtf8(self, data: List[int]) -> bool:
        cnt = 0
        for v in data:
            if cnt > 0:
                if v >> 6 != 0b10:
                    return False
                cnt -= 1
            elif v >> 7 == 0:
                cnt = 0
            elif v >> 5 == 0b110:
                cnt = 1
            elif v >> 4 == 0b1110:
                cnt = 2
            elif v >> 3 == 0b11110:
                cnt = 3
            else:
                return False
        return cnt == 0
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Count how many characters the data contains (-1 if it
# is invalid)
# 2. Return the position of the first byte where the data
# becomes invalid


# ============================================================
# FOLLOW-UP 1: Count how many characters the data contains
# (-1 if it is invalid)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Read the leading bits of the first byte of each
# character to know how many continuation bytes follow.
# Time: O(n).  Space: O(1).
def count_utf8_characters(data):
    count = 0
    i = 0
    while i < len(data):
        byte = data[i] & 0xFF
        if byte >> 7 == 0:
            length = 1
        elif byte >> 5 == 0b110:
            length = 2
        elif byte >> 4 == 0b1110:
            length = 3
```

```
        elif byte >> 3 == 0b11110:
            length = 4
        else:
            return -1
        if i + length > len(data):
            return -1
        for j in range(1, length):
            if (data[i + j] & 0xFF) >> 6 != 0b10:
                return -1
        i += length
        count += 1
    return count


# ============================================================
# FOLLOW-UP 2: Return the position of the first byte where
# the data becomes invalid
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same scan; report the index where a lead
# byte is invalid or a continuation byte is missing.
# Time: O(n).  Space: O(1).
def first_invalid_position(data):
    i = 0
    while i < len(data):
        byte = data[i] & 0xFF
        if byte >> 7 == 0:
            length = 1
        elif byte >> 5 == 0b110:
            length = 2
        elif byte >> 4 == 0b1110:
            length = 3
```

```
        elif byte >> 3 == 0b11110:
            length = 4
        else:
            return i
        for j in range(1, length):
            if i + j >= len(data) or (data[i + j] & 0xFF) >> 6 != 0b10:
                return i + j
        i += length
    return -1
```

29. Divide Two Integers

Medium · Bit Manipulation, Math

leetcode.com/problems/divide-two-integers

★ **TOP 7 PROBLEM** in Bit Manipulation: master this one first

PROBLEM

Given two integers `dividend` and `divisor`, divide two integers **without** using multiplication, division, and mod operator.

The integer division should truncate toward zero, which means losing its fractional part. For example, `8.345` would be truncated to `8`, and `-2.7335` would be truncated to `-2`.

Return *the **quotient** after dividing* `dividend` *by* `divisor`.

Note: Assume we are dealing with an environment that could only store integers within the **32-bit** signed integer range: $[-2^{31}, 2^{31} - 1]$. For this problem, if the quotient is **strictly greater than** $2^{31} - 1$, then return $2^{31} - 1$, and if the quotient is **strictly less than** -2^{31}, then return -2^{31}.

Example 1:

```
Input: dividend = 10, divisor = 3
Output: 3
Explanation: 10/3 = 3.33333.. which is truncated to 3.
```

Example 2:

```
Input: dividend = 7, divisor = -3
Output: -2
Explanation: 7/-3 = -2.33333.. which is truncated to -2.
```

Constraints:

- $-2^{31} <= dividend, divisor <= 2^{31} - 1$
- `divisor != 0`

SOLUTION (PYTHON 3)

Solution 1: Simulation + Fast Power

How to think about it

The first idea is repeated subtraction: each time the dividend still covers the divisor, add 1 to the quotient. |a| can be about 2^{31}, so this times out, and we cannot use multiply, divide, or mod. The bottleneck is removing only 1× b per step. If we can remove 2^k × b at once, the quotient jumps in binary. That is the same idea as fast exponentiation: while the remainder still fits, left-shift b to double it, subtract the largest such chunk, and repeat on what is left. Record the sign separately and compute on negatives so negating a= -2^{31} cannot overflow. Special-case b=1 and a= -2^{31}, b=-1, clamping to $2^{31}-1$.

Division is essentially subtraction. The problem requires us to calculate the integer result after dividing two numbers, which is actually calculating how many divisors and a number less than the divisor constitute the dividend. However, only one subtraction can be done in one loop, which is too inefficient and will lead to timeout. This can be optimized by using the idea of fast power.

It should be noted that since the problem explicitly requires that only 32-bit signed integers can be used at most, the divisor and dividend need to be converted to negative numbers for calculation. Because converting to positive numbers may cause overflow, such as when the dividend is `INT32_MIN`, it will be greater than `INT32_MAX` when converted to a positive number.

Assuming the dividend is a and the divisor is b, the time complexity is O(log a × log b), and the space complexity is O(1).

Python 3

```
class Solution:
    def divide(self, a: int, b: int) -> int:
        if b == 1:
            return a
        if a == -(2**31) and b == -1:
            return 2**31 - 1
        sign = (a > 0 and b > 0) or (a < 0 and b < 0)
        a = -a if a > 0 else a
        b = -b if b > 0 else b
        ans = 0
        while a <= b:
            x = b
            cnt = 1
            while x >= (-(2**30)) and a <= (x << 1):
                x <<= 1
                cnt <<= 1
            a -= x
            ans += cnt
        return ans if sign else -ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The remainder (modulo) without using % or /
# 2. The quotient rounded UP (ceiling) instead of towards
# zero


# ============================================================
# FOLLOW-UP 1: The remainder (modulo) without using % or /
# ============================================================

# ---- follow-up 1: function ----
# Answer: Divide by repeated doubling as in the main
# solution; the remainder is what is left of the dividend
# after subtracting the quotient times the divisor.
# Time: O(log^2 n).  Space: O(1).
def remainder_without_division(dividend, divisor):
    sign = 1
    if dividend < 0:
        sign = -1
    a = abs(dividend)
    b = abs(divisor)
    while a >= b:
        multiple = b
        while multiple << 1 <= a:
            multiple <<= 1
        a -= multiple
    return sign * a


# ============================================================
# FOLLOW-UP 2: The quotient rounded UP (ceiling) instead
# of towards zero
# ============================================================

# ---- follow-up 2: function ----
# Answer: Compute the truncated quotient with the doubling
# method; add one when there is a remainder and the result
# is positive.
# Time: O(log^2 n).  Space: O(1).
def divide_ceiling(dividend, divisor):
    negative = (dividend < 0) != (divisor < 0)
    a = abs(dividend)
    b = abs(divisor)
```

```
    quotient = 0
    while a >= b:
        multiple = b
        power = 1
        while multiple << 1 <= a:
            multiple <<= 1
            power <<= 1
        a -= multiple
        quotient += power
    if negative:
        return -quotient
    if a > 0:
        quotient += 1
    return quotient
```

318. Maximum Product of Word Lengths

Medium · Bit Manipulation, Array, String

leetcode.com/problems/maximum-product-of-word-lengths

PROBLEM

Given a string array `words`, return *the maximum value of* `length(word[i]) * length(word[j])` *where the two words do not share common letters*. If no such two words exist, return `0`.

Example 1:

```
Input: words = ["abcw","baz","foo","bar","xtfn","abcdef"]
Output: 16
Explanation: The two words can be "abcw", "xtfn".
```

Example 2:

```
Input: words = ["a","ab","abc","d","cd","bcd","abcd"]
Output: 4
Explanation: The two words can be "ab", "cd".
```

Example 3:

```
Input: words = ["a","aa","aaa","aaaa"]
Output: 0
Explanation: No such pair of words.
```

Constraints:

- `2 <= words.length <= 1000`
- `1 <= words[i].length <= 1000`
- `words[i]` consists only of lowercase English letters.

SOLUTIONS (PYTHON 3)

Solution 1: Bit Manipulation

> **How to think about it**
>
> A product counts only when two words share no letter. Set intersection is O(σ) per pair; we need a cheaper disjointness test. Pack each word into a 26-bit mask; bitwise AND zero means disjoint. Check every pair i>j and keep the max product. n≤ 1000 allows $O(n^2)$.

The problem requires us to find two strings without common letters, so that their length product is maximized. We can represent each string with a binary number mask[i], where each bit of this binary number indicates whether the string contains a certain letter. If two strings do not have common letters, then the bitwise AND result of the two binary numbers corresponding to these strings is 0, that is, mask[i] & mask[j] = 0.

We traverse each string. For the current string words[i] we are traversing, we first calculate the corresponding binary number mask[i], and then traverse all strings words[j] where $j \in [0, i)$. We check whether mask[i] & mask[j] = 0 holds. If it holds, we update the answer to max(ans, |words[i]| × |words[j]|).

After the traversal, we return the answer.

The time complexity is $O(n^2 + L)$, and the space complexity is $O(n)$. Here, n is the length of the string array words, and L is the sum of the lengths of all strings in the string array.

Python 3

```python
class Solution:
    def maxProduct(self, words: List[str]) -> int:
        mask = [0] * len(words)
        ans = 0
        for i, s in enumerate(words):
            for c in s:
                mask[i] |= 1 << (ord(c) - ord("a"))
            for j, t in enumerate(words[:i]):
                if (mask[i] & mask[j]) == 0:
                    ans = max(ans, len(s) * len(t))
        return ans
```

Solution 2

> **How to think about it**
>
> Method 1 compares words that share a mask many times. Store the longest word per mask and compare only against seen masks. Duplicate alphabets collapse to one length.

Python 3

```python
class Solution:
    def maxProduct(self, words: List[str]) -> int:
        mask = defaultdict(int)
        ans = 0
        for s in words:
            a = len(s)
            x = 0
            for c in s:
                x |= 1 << (ord(c) - ord("a"))
            for y, b in mask.items():
                if (x & y) == 0:
                    ans = max(ans, a * b)
            mask[x] = max(mask[x], a)
        return ans
```

Complexity. Time: O(n^2 + L), L = total number of letters. Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the two words
# 2. The maximum SUM of the lengths of two words with no
# common letter


# ============================================================
# FOLLOW-UP 1: Return the two words
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same masks (one bit per letter); remember
# the pair with the best product.
# Time: O(L + n^2), L = total letters.  Space: O(n).
def best_word_pair(words):
    masks = []
    for word in words:
        mask = 0
        for ch in word:
            mask |= 1 << (ord(ch) - ord("a"))
        masks.append(mask)
    best = (0, None)
    for i in range(len(words)):
        for j in range(i + 1, len(words)):
```

```
            if masks[i] & masks[j] == 0:
                product = len(words[i]) * len(words[j])
                if product > best[0]:
                    best = (product, (words[i], words[j]))
    return best


# ============================================================
# FOLLOW-UP 2: The maximum SUM of the lengths of two words
# with no common letter
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same search with a sum instead of a product.
# Time: O(L + n^2).  Space: O(n).
def max_length_sum(words):
    masks = []
    for word in words:
        mask = 0
        for ch in word:
            mask |= 1 << (ord(ch) - ord("a"))
        masks.append(mask)
    best = 0
    for i in range(len(words)):
        for j in range(i + 1, len(words)):
            if masks[i] & masks[j] == 0:
                total = len(words[i]) + len(words[j])
                if total > best:
                    best = total
    return best
```

1318. Minimum Flips to Make a OR b Equal to c

Medium · Bit Manipulation

leetcode.com/problems/minimum-flips-to-make-a-or-b-equal-to-c

PROBLEM

Given 3 positives numbers `a`, `b` and `c`. Return the minimum flips required in some bits of `a` and `b` to make (`a` OR `b` == `c`). (bitwise OR operation).
Flip operation consists of change **any** single bit 1 to 0 or change the bit 0 to 1 in their binary representation.

Example 1:

```
Input: a = 2, b = 6, c = 5
Output: 3
Explanation: After flips a = 1 , b = 4 , c = 5 such that (a OR b == c)
```

Example 2:

```
Input: a = 4, b = 2, c = 7
Output: 1
```

Example 3:

```
Input: a = 1, b = 2, c = 3
Output: 0
```

Constraints:

- `1 <= a <= 10^9`
- `1 <= b <= 10^9`
- `1 <= c <= 10^9`

SOLUTION (PYTHON 3)

Solution 1: Bit Manipulation

> **How to think about it**
> We want the fewest bit flips so that av b=c. Bits are independent, so we never search over integers. If c's bit is 0, every 1 in a or b must flip; if it is 1, we flip once only when both bits are 0. Summing over 32 bits is enough.

We can enumerate each bit of the binary representation of a, b, and c, denoted as x, y, and z respectively. If the bitwise OR operation result of x and y is different from z, we then check if both x and y are 1. If so, we need to flip twice, otherwise, we only need to flip once. We accumulate all the required flip times.

The time complexity is O(log M), where M is the maximum value of the numbers in the problem. The space complexity is O(1).

Python 3

```python
class Solution:
    def minFlips(self, a: int, b: int, c: int) -> int:
        ans = 0
        for i in range(32):
            x, y, z = a >> i & 1, b >> i & 1, c >> i & 1
            ans += x + y if z == 0 else int(x == 0 and y == 0)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Any number of numbers: make the OR of the whole list
# equal to c
# 2. Make the bitwise AND of a and b equal to c


# ============================================================
# FOLLOW-UP 1: Any number of numbers: make the OR of the
# whole list equal to c
# ============================================================

# ---- follow-up 1: function ----
# Answer: Per bit: if c has the bit, at least one number
# must have it (flip one if none has it); if c does not
# have the bit, every number having it must be flipped.
# Time: O(32 * k), k numbers.  Space: O(1).
def min_flips_or_list(numbers, c):
    flips = 0
    for bit in range(32):
        ones = 0
        for x in numbers:
            if (x >> bit) & 1:
                ones += 1
        if (c >> bit) & 1:
            if ones == 0:
                flips += 1
        else:
            flips += ones
    return flips


# ============================================================
# FOLLOW-UP 2: Make the bitwise AND of a and b equal to c
# ============================================================

# ---- follow-up 2: function ----
# Answer: Per bit: when c has the bit both numbers need it
# (flip the missing ones); when c does not have it at
# least one number must lack it (flip one if both have
# it).
# Time: O(32).  Space: O(1).
def min_flips_and(a, b, c):
    flips = 0
    for bit in range(32):
        bit_a = (a >> bit) & 1
        bit_b = (b >> bit) & 1
        bit_c = (c >> bit) & 1
        if bit_c == 1:
            flips += (1 - bit_a) + (1 - bit_b)
        elif bit_a == 1 and bit_b == 1:
            flips += 1
    return flips
```

PATTERN 28 OF 41

Dynamic Programming - Knapsack Style

20 problems · 0 Easy · 14 Medium · 6 Hard

THE IDEA

Pick items to reach a capacity or target. 0/1 knapsack loops the capacity downwards (each item once); unbounded loops it upwards (items reusable).

Signals: subset sum, partition equal subset, coin change, combination sum IV, target sum, last stone weight II, ones and zeroes.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
dp = [False] * (target + 1)
dp[0] = True
for x in nums:                          # each item once
    for t in range(target, x - 1, -1):  # downwards so x is not reused
        dp[t] = dp[t] or dp[t - x]
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	★ **Partition Equal Subset Sum** (see page)	Medium	LC 416	698
2	★ **Target Sum**	Medium	LC 494	1504
3	★ **Coin Change** (see page)	Medium	LC 322	683
4	★ **Coin Change II** (see page)	Medium	LC 518	701
5	★ **Last Stone Weight II**	Medium	LC 1049	1507
6	★ **Ones and Zeroes**	Medium	LC 474	1510
7	Combination Sum IV (see page)	Medium	LC 377	704
8	Perfect Squares (see page)	Medium	LC 279	707
9	Partition to K Equal Sum Subsets (see page)	Medium	LC 698	829
10	Minimum Cost to Reach Destination in Time — substitute: Number of Dice Rolls With Target Sum	Medium	LC 1155	1513
11	Filling Bookcase Shelves	Medium	LC 1105	1516
12	Maximum Students Taking Exam — substitute: Shopping Offers	Medium	LC 638	1519
13	Form Largest Integer With Digit Costs	Medium	LC 1449	1522
14	Maximum Number of Points with Cost — substitute: Count Square Submatrices with All Ones (see page)	Medium	LC 1277	285
15	★ **Tallest Billboard**	Hard	LC 956	1525
16	Profitable Schemes	Hard	LC 879	1528
17	Matrix Block Sum — substitute: Minimum Number of Refueling Stops	Hard	LC 871	1531
18	Maximum Value of K Coins From Piles	Hard	LC 2218	1534
19	Closest Subsequence Sum	Hard	LC 1755	1537
20	Number of Ways to Stay in the Same Place After Some Steps	Hard	LC 1269	1541

PATTERN 28 · DYNAMIC PROGRAMMING - KNAPSACK STYLE · PROBLEM 2 OF 20

494. Target Sum

Medium · Array, Dynamic Programming, Backtracking, Knapsack, 0-1 Knapsack

leetcode.com/problems/target-sum

★ **TOP 7 PROBLEM** in Dynamic Programming - Knapsack Style: master this one first

PROBLEM

You are given an integer array `nums` and an integer `target`.

You want to build an **expression** out of nums by adding one of the symbols `'+'` and `'-'` before each integer in nums and then concatenate all the integers.

- For example, if `nums = [2, 1]`, you can add a `'+'` before `2` and a `'-'` before `1` and concatenate them to build the expression `"+2-1"`.

Return the number of different **expressions** that you can build, which evaluates to `target`.

Example 1:

```
Input: nums = [1,1,1,1,1], target = 3
Output: 5
Explanation: There are 5 ways to assign symbols to make the sum of nums be target 3.
-1 + 1 + 1 + 1 + 1 = 3
+1 - 1 + 1 + 1 + 1 = 3
+1 + 1 - 1 + 1 + 1 = 3
+1 + 1 + 1 - 1 + 1 = 3
+1 + 1 + 1 + 1 - 1 = 3
```

Example 2:

```
Input: nums = [1], target = 1
Output: 1
```

Constraints:

- `1 <= nums.length <= 20`
- `0 <= nums[i] <= 1000`
- `0 <= sum(nums[i]) <= 1000`
- `-1000 <= target <= 1000`

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

Assign + or - so the expression equals target. 2^n is tight for n≤ 20; the problem is a subset sum. If the negative part sums to x, then (s-x)-x=target, so x=(s-target)/2. Impossible when s<target or the difference is odd. f[i][j] is the number of ways for the first i numbers to sum to j. The empty subset gives f[0][0]=1. Take-or-skip is the usual 0-1 knapsack count.

Let's denote the sum of all elements in the array nums as s, and the sum of elements to which we assign a negative sign as x. Therefore, the sum of elements with a positive sign is s - x. We have:

$$(s - x) - x = \text{target} \Rightarrow x = (s - \text{target})/2$$

Since x ≥ 0 and x must be an integer, it follows that s ≥ target and s - target must be even. If these two conditions are not met, we directly return 0.

Next, we can transform the problem into: selecting several elements from the array nums such that the sum of these elements equals (s - target)/2. We are asked how many ways there are to make such a selection.

We can use dynamic programming to solve this problem. Define f[i][j] as the number of ways to select several elements from the first i elements of the array nums such that the sum of these elements equals j.

For nums[i - 1], we have two choices: to select or not to select. If we do not select nums[i - 1], then f[i][j] = f[i - 1][j]; if we do select nums[i - 1], then f[i][j] = f[i - 1][j - nums[i - 1]]. Therefore, the state transition equation is:

$$f[i][j] = f[i - 1][j] + f[i - 1][j - nums[i - 1]]$$

This selection is based on the premise that j ≥ nums[i - 1].

The final answer is f[m][n], where m is the length of the array nums, and n = (s - target)/2.

The time complexity is O(m × n), and the space complexity is O(m × n).

Python 3

```python
class Solution:
    def findTargetSumWays(self, nums: List[int], target: int) -> int:
        s = sum(nums)
        if s < target or (s - target) % 2:
            return 0
        m, n = len(nums), (s - target) // 2
        f = [[0] * (n + 1) for _ in range(m + 1)]
        f[0][0] = 1
        for i, x in enumerate(nums, 1):
            for j in range(n + 1):
                f[i][j] = f[i - 1][j]
                if j >= x:
                    f[i][j] += f[i - 1][j - x]
        return f[m][n]
```

Solution 2: Dynamic Programming (Space Optimization)

> **How to think about it**
>
> Row i depends only on row i-1. Update j downward in one array so an item is not reused; space follows the capacity.

We can observe that in the state transition equation of Solution 1, the value of f[i][j] is only related to f[i - 1][j] and f[i - 1][j - nums[i - 1]]. Therefore, we can eliminate the first dimension of the space and use only a one-dimensional array.

The time complexity is O(m × n), and the space complexity is O(n).

Python 3

```python
class Solution:
    def findTargetSumWays(self, nums: List[int], target: int) -> int:
        s = sum(nums)
        if s < target or (s - target) % 2:
            return 0
        n = (s - target) // 2
        f = [0] * (n + 1)
        f[0] = 1
        for x in nums:
            for j in range(n, x - 1, -1):
                f[j] += f[j - x]
        return f[n]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return every assignment of + and - signs that reaches
# the target
# 2. Count with a subset-sum table: the numbers with a +
# sign form a subset with sum (total + target) / 2


# ============================================================
# FOLLOW-UP 1: Return every assignment of + and - signs
# that reaches the target
# ============================================================

# ---- follow-up 1: function ----
# Answer: Backtracking: try + and - for each number and
# keep the sign strings whose total equals the target.
# Time: O(2^n * n).  Space: O(n).
def all_sign_assignments(nums, target):
    results = []

    def go(i, total, signs):
        if i == len(nums):
            if total == target:
                results.append("".join(signs))
            return
        signs.append("+")
        go(i + 1, total + nums[i], signs)
        signs.pop()
        signs.append("-")
        go(i + 1, total - nums[i], signs)
        signs.pop()

    go(0, 0, [])
    return results


# ============================================================
# FOLLOW-UP 2: Count with a subset-sum table: the numbers
# with a + sign form a subset with sum (total + target) /
# 2
# ============================================================

# ---- follow-up 2: function ----
# Answer: If P is the sum of the plus group and N of the
# minus group, P - N = target and P + N = total, so P =
# (total + target) / 2: count the subsets that reach P.
# Time: O(n * W), W = (total + target) / 2.  Space: O(W).
def count_by_subset_sum(nums, target):
    total = 0
    for x in nums:
        total += x
    if abs(target) > total or (total + target) % 2 != 0:
        return 0
    wanted = (total + target) // 2
    ways = [0] * (wanted + 1)
    ways[0] = 1
    for x in nums:
        for s in range(wanted, x - 1, -1):
            ways[s] += ways[s - x]
    return ways[wanted]
```

1049. Last Stone Weight II

Medium · Array, Dynamic Programming, Knapsack, 0-1 Knapsack

leetcode.com/problems/last-stone-weight-ii

★ **TOP 7 PROBLEM** in Dynamic Programming - Knapsack Style: master this one first

PROBLEM

You are given an array of integers `stones` where `stones[i]` is the weight of the i^{th} stone.

We are playing a game with the stones. On each turn, we choose any two stones and smash them together. Suppose the stones have weights `x` and `y` with `x <= y`. The result of this smash is:

- If `x == y`, both stones are destroyed, and
- If `x != y`, the stone of weight `x` is destroyed, and the stone of weight `y` has new weight `y - x`.

At the end of the game, there is **at most one** stone left.

Return *the smallest possible weight of the left stone*. If there are no stones left, return `0`.

Example 1:

```
Input: stones = [2,7,4,1,8,1]
Output: 1
Explanation:
We can combine 2 and 4 to get 2, so the array converts to [2,7,1,8,1] then,
we can combine 7 and 8 to get 1, so the array converts to [2,1,1,1] then,
we can combine 2 and 1 to get 1, so the array converts to [1,1,1] then,
we can combine 1 and 1 to get 0, so the array converts to [1], then that's the optimal value.
```

Example 2:

```
Input: stones = [31,26,33,21,40]
Output: 5
```

Constraints:

- `1 <= stones.length <= 30`
- `1 <= stones[i] <= 100`

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Each smash assigns opposite signs to two stones, so the remainder is the absolute difference of two subset sums. n≤ 30 but the total is only a few thousand, which is a 0-1 knapsack of capacity floor(s/2): pack as close to half as possible. dp[i][j] is the best weight of the first i stones that does not exceed j, taking or skipping stone i. The answer is s-2·dp[n][floor(s/2)].

Python 3

```python
class Solution:
    def lastStoneWeightII(self, stones: List[int]) -> int:
        s = sum(stones)
        m, n = len(stones), s >> 1
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            for j in range(n + 1):
                dp[i][j] = dp[i - 1][j]
                if stones[i - 1] <= j:
                    dp[i][j] = max(
                        dp[i][j], dp[i - 1][j - stones[i - 1]] + stones[i - 1]
                    )
        return s - 2 * dp[-1][-1]
```

Complexity. Time: O(m * S), S = sum of the stones. Space: O(m * S).

Solution 2: Optimized Dynamic Programming

> **How to think about it**
>
> Row i depends only on row i-1. A one-dimensional array updated from high capacity to low reuses the same transition without packing a stone twice, and the space becomes O(s).

dp[i][j] only depends on the previous row dp[i - 1][·], so we can drop the first dimension and enumerate the capacity from large to small, reducing the space complexity to O(sum).

Python 3

```python
class Solution:
    def lastStoneWeightII(self, stones: List[int]) -> int:
        s = sum(stones)
        m, n = len(stones), s >> 1
        dp = [0] * (n + 1)
        for v in stones:
            for j in range(n, v - 1, -1):
                dp[j] = max(dp[j], dp[j - v] + v)
        return s - dp[-1] * 2
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the two groups of stones with the smallest
# difference
# 2. Use one big integer as a bit set of the reachable
# sums


# ========================================================
# FOLLOW-UP 1: Return the two groups of stones with the
# smallest difference
# ========================================================

# ---- follow-up 1: function ----
# Answer: Remember for every reachable sum the stone that
# first made it reachable; walk back from the best sum up
# to half of the total to get one group; the rest is the
# other group.
# Time: O(n * S), S = half of the total (the reachable-sum
# dictionary is re-sorted for every stone).  Space: O(S).
def split_into_two_groups(stones):
    total = 0
    for s in stones:
        total += s
    half = total // 2
```

```
    made_by = {0: None}
    for s in stones:
        for amount in sorted(made_by, reverse=True):
            if amount + s <= half and amount + s not in made_by:
                made_by[amount + s] = s
    best = max(made_by)
    group = []
    amount = best
    while amount > 0:
        group.append(made_by[amount])
        amount -= made_by[amount]
    other = list(stones)
    for s in group:
        other.remove(s)
    return group, other


# ============================================================
# FOLLOW-UP 2: Use one big integer as a bit set of the
# reachable sums
# ============================================================

# ---- follow-up 2: function ----
# Answer: Bit i of the number says whether sum i is
# reachable; adding a stone is a shift and an OR.
# Time: O(n * S / w), w = machine word size, since each
# shift touches S bits.  Space: O(S).
def last_stone_weight_ii_bitset(stones):
    total = 0
    for s in stones:
        total += s
    reachable = 1
    for s in stones:
        reachable |= reachable << s
    best = total
    for amount in range(total // 2, -1, -1):
        if (reachable >> amount) & 1:
            return total - 2 * amount
    return best
```

474. Ones and Zeroes

Medium · Array, String, Dynamic Programming, Knapsack, 0-1 Knapsack

leetcode.com/problems/ones-and-zeroes

★ **TOP 7 PROBLEM** in Dynamic Programming - Knapsack Style: master this one first

PROBLEM

You are given an array of binary strings `strs` and two integers `m` and `n`.

Return *the size of the largest subset of* `strs` *such that there are* ***at most*** `m` `0`*'s and* `n` `1`*'s in the subset*.

A set `x` is a **subset** of a set `y` if all elements of `x` are also elements of `y`.

Example 1:

```
Input: strs = ["10","0001","111001","1","0"], m = 5, n = 3
Output: 4
Explanation: The largest subset with at most 5 0's and 3 1's is {"10", "0001", "1", "0"}, so the answer is 4.
Other valid but smaller subsets include {"0001", "1"} and {"10", "1", "0"}.
{"111001"} is an invalid subset because it contains 4 1's, greater than the maximum of 3.
```

Example 2:

```
Input: strs = ["10","0","1"], m = 1, n = 1
Output: 2
Explanation: The largest subset is {"0", "1"}, so the answer is 2.
```

Constraints:

- `1 <= strs.length <= 600`
- `1 <= strs[i].length <= 100`
- `strs[i]` consists only of digits `'0'` and `'1'`.
- `1 <= m, n <= 100`

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

Selecting the most strings under budgets of zeros and ones is 0-1 knapsack with two weights. Subset search is too large. f[i][j][k] is the best count using the first i strings, at most j zeros and k ones. Skip copies the previous row; take adds 1 when the budgets allow. Count 0/1 in the current string before filling the row so the transition uses that item's cost.

We define f[i][j][k] as the maximum number of strings that can be obtained from the first i strings using j zeros and k ones. Initially, f[i][j][k]=0, and the answer is f[sz][m][n], where sz is the length of the array strs.

For f[i][j][k], we have two choices:

- Do not select the i-th string, in which case f[i][j][k]=f[i-1][j][k];
- Select the i-th string, in which case f[i][j][k]=f[i-1][j-a][k-b]+1, where a and b are the number of zeros and ones in the i-th string, respectively.

We take the maximum of these two choices to obtain the value of f[i][j][k].

The final answer is f[sz][m][n].

The time complexity is O(sz × m × n), and the space complexity is O(sz × m × n), where sz is the length of the array strs, and m and n are the upper limits on the number of zeros and ones, respectively.

Python 3

```python
class Solution:
    def findMaxForm(self, strs: List[str], m: int, n: int) -> int:
        sz = len(strs)
        f = [[[0] * (n + 1) for _ in range(m + 1)] for _ in range(sz + 1)]
        for i, s in enumerate(strs, 1):
            a, b = s.count("0"), s.count("1")
            for j in range(m + 1):
                for k in range(n + 1):
                    f[i][j][k] = f[i - 1][j][k]
                    if j >= a and k >= b:
                        f[i][j][k] = max(f[i][j][k], f[i - 1][j - a][k - b] + 1)
        return f[sz][m][n]
```

Solution 2: Dynamic Programming (Space Optimization)

> **How to think about it**
>
> Row i depends only on row i-1. Update j and k downward and drop the first dimension. Space becomes O(mn), and a string cannot be taken twice.

We notice that the calculation of f[i][j][k] only depends on f[i-1][j][k] and f[i-1][j-a][k-b]. Therefore, we can eliminate the first dimension and optimize the space complexity to O(m × n).

Python 3

```python
class Solution:
    def findMaxForm(self, strs: List[str], m: int, n: int) -> int:
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for s in strs:
            a, b = s.count("0"), s.count("1")
            for i in range(m, a - 1, -1):
                for j in range(n, b - 1, -1):
                    f[i][j] = max(f[i][j], f[i - a][j - b] + 1)
        return f[m][n]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the chosen strings of the biggest subset
# 2. The strings have a value: maximise the total value
# instead of the count


# ========================================================
# FOLLOW-UP 1: Return the chosen strings of the biggest
# subset
# ========================================================

# ---- follow-up 1: function ----
# Answer: Keep, for every table cell, the list of strings
# that built it (simple, for small inputs).
# Time: O(L * m * n * L), the list copies cost up to L
# each.  Space: O(m * n * L).
def chosen_strings(strs, m, n):
    best = {}
    for zeros in range(m + 1):
        for ones in range(n + 1):
            best[(zeros, ones)] = []
    for s in strs:
        z = s.count("0")
        o = len(s) - z
        for zeros in range(m, z - 1, -1):
            for ones in range(n, o - 1, -1):
                candidate = best[(zeros - z, ones - o)] + [s]
```

```
                if len(candidate) > len(best[(zeros, ones)]):
                    best[(zeros, ones)] = candidate
    return best[(m, n)]


# ========================================================
# FOLLOW-UP 2: The strings have a value: maximise the
# total value instead of the count
# ========================================================

# ---- follow-up 2: function ----
# Answer: The same table, adding the value of the string
# instead of one.
# Time: O(L * m * n).  Space: O(m * n).
def max_value_subset(strs, values, m, n):
    best = []
    for zeros in range(m + 1):
        best.append([0] * (n + 1))
    for index in range(len(strs)):
        z = strs[index].count("0")
        o = len(strs[index]) - z
        for zeros in range(m, z - 1, -1):
            for ones in range(n, o - 1, -1):
                candidate = best[zeros - z][ones - o] + values[index]
                if candidate > best[zeros][ones]:
                    best[zeros][ones] = candidate
    return best[m][n]
```

1155. Number of Dice Rolls With Target Sum

Medium · Dynamic Programming

leetcode.com/problems/number-of-dice-rolls-with-target-sum

PROBLEM

You have `n` dice, and each dice has `k` faces numbered from `1` to `k`.

Given three integers `n`, `k`, and `target`, return *the number of possible ways (out of the* k^n *total ways) to roll the dice, so the sum of the face-up numbers equals* `target`. Since the answer may be too large, return it **modulo** $10^9 + 7$.

Example 1:

```
Input: n = 1, k = 6, target = 3
Output: 1
Explanation: You throw one die with 6 faces.
There is only one way to get a sum of 3.
```

Example 2:

```
Input: n = 2, k = 6, target = 7
Output: 6
Explanation: You throw two dice, each with 6 faces.
There are 6 ways to get a sum of 7: 1+6, 2+5, 3+4, 4+3, 5+2, 6+1.
```

Example 3:

```
Input: n = 30, k = 30, target = 500
Output: 222616187
Explanation: The answer must be returned modulo 109 + 7.
```

Constraints:

- `1 <= n, k <= 30`
- `1 <= target <= 1000`

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

Naively assigning n k-faced dice to sum to target is k^n. Let f[i][j] be the ways for i dice to sum to j; face h on the last die comes from f[i-1][j-h]. One way uses zero dice to make 0. Reduce modulo 10^9+7.

We define f[i][j] as the number of ways to get a sum of j using i dice. Then, we can obtain the following state transition equation:

$$f[i][j] = \sum_{h=1}^{\min(j,\ k)} f[i-1][j-h]$$

where h represents the number of points on the i-th die.

Initially, we have f[0][0] = 1, and the final answer is f[n][target].

The time complexity is O(n × k × target), and the space complexity is O(n × target).

Python 3

```python
class Solution:
    def numRollsToTarget(self, n: int, k: int, target: int) -> int:
        f = [[0] * (target + 1) for _ in range(n + 1)]
        f[0][0] = 1
        mod = 10**9 + 7
        for i in range(1, n + 1):
            for j in range(1, min(i * k, target) + 1):
                for h in range(1, min(j, k) + 1):
                    f[i][j] = (f[i][j] + f[i - 1][j - h]) % mod
        return f[n][target]
```

Solution 2: Dynamic Programming (Rolling Array)

> **How to think about it**
>
> Row i of method 1 depends only on row i-1. Two rolling arrays cut space from O(n× target) to O(target) with the same transition.

f[i][j] depends only on the previous row, so two arrays f and g of length target+1 are enough. The space complexity becomes O(target).

Python 3

```python
class Solution:
    def numRollsToTarget(self, n: int, k: int, target: int) -> int:
        f = [1] + [0] * target
        mod = 10**9 + 7
        for i in range(1, n + 1):
            g = [0] * (target + 1)
            for j in range(1, min(i * k, target) + 1):
                for h in range(1, min(j, k) + 1):
                    g[j] = (g[j] + f[j - h]) % mod
            f = g
        return f[target]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. O(n * target) with a running window sum
# 2. The number of rolls whose sum is AT MOST the target


# ============================================================
# FOLLOW-UP 1: O(n * target) with a running window sum
# ============================================================

# ---- follow-up 1: function ----
# Answer: ways[d][t] is the sum of ways[d - 1][t - 1 .. t
# - k]; keep that sum as a sliding window instead of
# adding k numbers each time.
# Time: O(n * target).  Space: O(target).
def dice_rolls_fast(n, k, target):
    mod = 10 ** 9 + 7
    ways = [0] * (target + 1)
    ways[0] = 1
    for dice in range(n):
        new_ways = [0] * (target + 1)
        window = 0
        for t in range(1, target + 1):
            window += ways[t - 1]
            if t - 1 - k >= 0:
                window -= ways[t - 1 - k]
            new_ways[t] = window % mod
        ways = new_ways
```

```
    return ways[target]


# ============================================================
# FOLLOW-UP 2: The number of rolls whose sum is AT MOST
# the target
# ============================================================

# ---- follow-up 2: function ----
# Answer: Add up the table for every sum from n to target.
# Time: O(n * target * k).  Space: O(target).
def dice_rolls_at_most(n, k, target):
    mod = 10 ** 9 + 7
    ways = [0] * (target + 1)
    ways[0] = 1
    for dice in range(n):
        new_ways = [0] * (target + 1)
        for t in range(target + 1):
            if ways[t] == 0:
                continue
            for face in range(1, k + 1):
                if t + face <= target:
                    new_ways[t + face] = (new_ways[t + face] + ways[t]) % mod
        ways = new_ways
    return sum(ways) % mod
```

1105. Filling Bookcase Shelves

Medium · Array, Dynamic Programming

leetcode.com/problems/filling-bookcase-shelves

PROBLEM

You are given an array `books` where `books[i] = [thickness`$_i$`, height`$_i$`]` indicates the thickness and height of the i^{th} book. You are also given an integer `shelfWidth`.

We want to place these books in order onto bookcase shelves that have a total width `shelfWidth`.

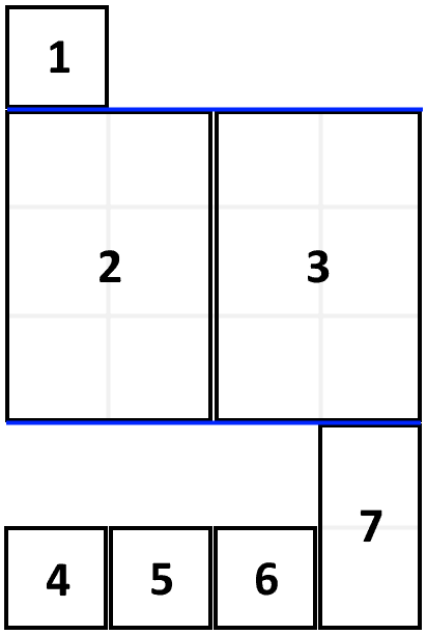
We choose some of the books to place on this shelf such that the sum of their thickness is less than or equal to `shelfWidth`, then build another level of the shelf of the bookcase so that the total height of the bookcase has increased by the maximum height of the books we just put down. We repeat this process until there are no more books to place.

Note that at each step of the above process, the order of the books we place is the same order as the given sequence of books.

- For example, if we have an ordered list of `5` books, we might place the first and second book onto the first shelf, the third book on the second shelf, and the fourth and fifth book on the last shelf.

Return *the minimum possible height that the total bookshelf can be after placing shelves in this manner*.

Example 1:




```
Input: books = [[1,1],[2,3],[2,3],[1,1],[1,1],[1,1],[1,2]], shelfWidth = 4
Output: 6
Explanation:
The sum of the heights of the 3 shelves is 1 + 3 + 2 = 6.
Notice that book number 2 does not have to be on the first shelf.
```

Example 2:

```
Input: books = [[1,3],[2,4],[3,2]], shelfWidth = 6
Output: 4
```

Constraints:

- `1 <= books.length <= 1000`
- `1 <= thickness`$_i$` <= shelfWidth <= 1000`

- $1 <= height_i <= 1000$

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

> **How to think about it**
>
> Each book may start a new shelf or share one with a prefix of earlier books; the partition count grows exponentially. With n≤ 1000, an $O(n^2)$ DP fits. Let f[i] be the minimum height for the first i books. The last shelf ends at books[i-1]; extend it backward, accumulating width, and stop once it exceeds shelfWidth. The shelf height is the max book height on that shelf, added to f[j-1]. Scanning backward keeps width monotonic.

We define f[i] as the minimum height for placing the first i books, initially f[0] = 0, and the answer is f[n].

Consider f[i], the last book is books[i - 1], its thickness is w, and its height is h.

- If this book is placed on a new layer alone, then f[i] = f[i - 1] + h;
- If this book can be placed on the same layer with the last few books in front, we enumerate the first book books[j-1] on the same layer from back to front, where j ∈ [1, i - 1], accumulate the thickness of the book to w, if w > shelfWidth, it means that books[j-1] can no longer be placed on the same layer with books[i-1], stop enumeration; otherwise, we update the maximum height h = max(h, books[j-1][1]) of the current layer, then f[i] = min(f[i], f[j - 1] + h).

The final answer is f[n].

The time complexity is $O(n^2)$, and the space complexity is O(n). Here, n is the length of the array books.

Python 3

```python
class Solution:
    def minHeightShelves(self, books: List[List[int]], shelfWidth: int) -> int:
        n = len(books)
        f = [0] * (n + 1)
        for i, (w, h) in enumerate(books, 1):
            f[i] = f[i - 1] + h
            for j in range(i - 1, 0, -1):
                w += books[j - 1][0]
                if w > shelfWidth:
                    break
                h = max(h, books[j - 1][1])
                f[i] = min(f[i], f[j - 1] + h)
        return f[n]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the shelves (which books go on which shelf)


# ============================================================
# FOLLOW-UP 1: Return the shelves (which books go on which
# shelf)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember the start of the last shelf for the
# best layout of each prefix, and walk back from the end.
# Time: O(n^2).  Space: O(n).
def bookcase_layout(books, shelf_width):
    n = len(books)
    big = float("inf")
    best = [big] * (n + 1)
    best[0] = 0
    start_of_last = [0] * (n + 1)
    for i in range(1, n + 1):
```

```
        width = 0
        height = 0
        for j in range(i, 0, -1):
            width += books[j - 1][0]
            if width > shelf_width:
                break
            if books[j - 1][1] > height:
                height = books[j - 1][1]
            if best[j - 1] + height < best[i]:
                best[i] = best[j - 1] + height
                start_of_last[i] = j - 1
    shelves = []
    i = n
    while i > 0:
        j = start_of_last[i]
        shelves.append(books[j:i])
        i = j
    shelves.reverse()
    return best[n], shelves
```

638. Shopping Offers

Medium · Bit Manipulation, Memoization, Array, Dynamic Programming, Backtracking, Bitmask, Knapsack, Unbounded Knapsack

leetcode.com/problems/shopping-offers

PROBLEM

In LeetCode Store, there are `n` items to sell. Each item has a price. However, there are some special offers, and a special offer consists of one or more different kinds of items with a sale price.

You are given an integer array `price` where `price[i]` is the price of the i^{th} item, and an integer array `needs` where `needs[i]` is the number of pieces of the i^{th} item you want to buy.

You are also given an array `special` where `special[i]` is of size `n + 1` where `special[i][j]` is the number of pieces of the j^{th} item in the i^{th} offer and `special[i][n]` (i.e., the last integer in the array) is the price of the i^{th} offer.

Return *the lowest price you have to pay for exactly certain items as given, where you could make optimal use of the special offers*. You are not allowed to buy more items than you want, even if that would lower the overall price. You could use any of the special offers as many times as you want.

Example 1:

```
Input: price = [2,5], special = [[3,0,5],[1,2,10]], needs = [3,2]
Output: 14
Explanation: There are two kinds of items, A and B. Their prices are $2 and $5 respectively.
In special offer 1, you can pay $5 for 3A and 0B
In special offer 2, you can pay $10 for 1A and 2B.
You need to buy 3A and 2B, so you may pay $10 for 1A and 2B (special offer #2), and $4 for 2A.
```

Example 2:

```
Input: price = [2,3,4], special = [[1,1,0,4],[2,2,1,9]], needs = [1,2,1]
Output: 11
Explanation: The price of A is $2, and $3 for B, $4 for C.
You may pay $4 for 1A and 1B, and $9 for 2A ,2B and 1C.
You need to buy 1A ,2B and 1C, so you may pay $4 for 1A and 1B (special offer #1), and $3 for 1B, $4
for 1C.
You cannot add more items, though only $9 for 2A ,2B and 1C.
```

Constraints:

- `n == price.length == needs.length`
- `1 <= n <= 6`
- `0 <= price[i], needs[i] <= 10`
- `1 <= special.length <= 100`
- `special[i].length == n + 1`
- `0 <= special[i][j] <= 50`
- The input is generated that at least one of `special[i][j]` is non-zero for `0 <= j <= n - 1`.

SOLUTION (PYTHON 3)

Solution 1: State Compression + Memoization Search

> **How to think about it**
>
> At most 6 item types and 10 of each, enumerating offer multiplicities revisits the same leftover list. Pack each need into 4 bits. Memoize on the leftover mask: pay retail, or apply any offer that still fits and recurse.

We notice that the number of types of items $n \le 6$ in the problem, and the quantity of each item needed does not exceed 10. We can use 4 binary bits to represent the quantity of each item needed. Thus, we only need at most $6 \times 4 = 24$ binary bits

to represent the entire shopping list.

First, we convert the shopping list needs into an integer mask, where the quantity of the i-th item needed is stored in the i × 4 to (i + 1) × 4 - 1 bits of mask. For example, when needs = [1, 2, 1], we have mask = 0b0001 0010 0001.

Then, we design a function dfs(cur), representing the minimum amount of money we need to spend when the current state of the shopping list is cur. Therefore, the answer is dfs(mask).

The calculation method of the function dfs(cur) is as follows:

- First, we calculate the cost of the current shopping list cur without using any bundles, denoted as ans.
- Then, we iterate through each bundle offer. If the current shopping list cur can use the bundle offer, i.e., the quantity of each item in cur is not less than that in the bundle offer, then we can try to use this bundle. We subtract the quantity of each item in the bundle offer from cur, obtaining a new shopping list nxt, then recursively calculate the minimum cost of nxt and add the price of the bundle offer[n], updating ans, i.e., ans = min(ans, offer[n] + dfs(nxt)).
- Finally, return ans.

To avoid repeated calculations, we use a hash table f to record the minimum cost corresponding to each state cur.

The time complexity is $O(n \times k \times m^n)$, where n represents the types of items, and k and m respectively represent the number of bundles and the maximum demand for each type of item. The space complexity is $O(n \times m^n)$.

Python 3

```
class Solution:
    def shoppingOffers(
        self, price: List[int], special: List[List[int]], needs: List[int]
    ) -> int:
        @cache
        def dfs(cur: int) -> int:
            ans = sum(p * (cur >> (i * bits) & 0xF) for i, p in enumerate(price))
            for offer in special:
                nxt = cur
                for j in range(len(needs)):
                    if (cur >> (j * bits) & 0xF) < offer[j]:
                        break
                    nxt -= offer[j] << (j * bits)
                else:
                    ans = min(ans, offer[-1] + dfs(nxt))
            return ans

        bits, mask = 4, 0
        for i, need in enumerate(needs):
            mask |= need << i * bits
        return dfs(mask)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return which special offers are used (and how many
# times)


# ========================================================
# FOLLOW-UP 1: Return which special offers are used (and
# how many times)
# ========================================================

# ---- follow-up 1: function ----
# Answer: The same memoised search, remembering the best
# choice for every needs vector.
# Time: O(S * prod(needs_i + 1)), S offers.  Space:
# O(prod(needs_i + 1)).
def offers_used(price, special, needs):
    memo = {}
```

```
    def best(needs):
        key = tuple(needs)
        if key in memo:
            return memo[key]
        cost = 0
        for i in range(len(needs)):
            cost += needs[i] * price[i]
        choice = None
        for index in range(len(special)):
            offer = special[index]
            new_needs = [needs[i] - offer[i] for i in range(len(needs))]
            if min(new_needs) < 0:
                continue
            candidate = offer[-1] + best(new_needs)[0]
            if candidate < cost:
                cost = candidate
                choice = index
        memo[key] = (cost, choice)
        return memo[key]

    used = []
    current = list(needs)
    total = best(current)[0]
    while True:
        cost, choice = best(current)
        if choice is None:
            break
        used.append(choice)
        current = [current[i] - special[choice][i] for i in range(len(current))]
    return total, used
```

1449. Form Largest Integer With Digits That Add up to Target

Hard · Array, Dynamic Programming, Knapsack, Unbounded Knapsack

leetcode.com/problems/form-largest-integer-with-digits-that-add-up-to-target

PROBLEM

Given an array of integers `cost` and an integer `target`, return *the **maximum** integer you can paint under the following rules*:

- The cost of painting a digit `(i + 1)` is given by `cost[i]` (**0-indexed**).
- The total cost used must be equal to `target`.
- The integer does not have `0` digits.

Since the answer may be very large, return it as a string. If there is no way to paint any integer given the condition, return `"0"`.

Example 1:

```
Input: cost = [4,3,2,5,6,7,2,5,5], target = 9
Output: "7772"
Explanation: The cost to paint the digit '7' is 2, and the digit '2' is 3. Then cost("7772") = 2*3+ 3*1
= 9. You could also paint "977", but "7772" is the largest number.
Digit cost
  1 -> 4
  2 -> 3
  3 -> 2
  4 -> 5
  5 -> 6
  6 -> 7
  7 -> 2
  8 -> 5
  9 -> 5
```

Example 2:

```
Input: cost = [7,6,5,5,5,6,8,7,8], target = 12
Output: "85"
Explanation: The cost to paint the digit '8' is 7, and the digit '5' is 5. Then cost("85") = 7 + 5 =
12.
```

Example 3:

```
Input: cost = [2,4,6,2,4,6,4,4,4], target = 5
Output: "0"
Explanation: It is impossible to paint any integer with total cost equal to target.
```

Constraints:

- `cost.length == 9`
- `1 <= cost[i], target <= 5000`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Digits 1– 9 have costs and may be reused to reach `target`. The lexicographically largest integer first maximizes length, then prefers larger digits. f[i][j] is the maximum length using the first i digits and exact cost j (unbounded knapsack). g[i][j] records whether digit i was taken so we can reconstruct from 9 downward. Impossible yields `0`.

Python 3

```
class Solution:
    def largestNumber(self, cost: List[int], target: int) -> str:
        f = [[-inf] * (target + 1) for _ in range(10)]
        f[0][0] = 0
        g = [[0] * (target + 1) for _ in range(10)]
        for i, c in enumerate(cost, 1):
            for j in range(target + 1):
                if j < c or f[i][j - c] + 1 < f[i - 1][j]:
                    f[i][j] = f[i - 1][j]
                    g[i][j] = j
                else:
                    f[i][j] = f[i][j - c] + 1
                    g[i][j] = j - c
        if f[9][target] < 0:
            return "0"
        ans = []
        i, j = 9, target
        while i:
            if j == g[i][j]:
                i -= 1
            else:
                ans.append(str(i))
                j = g[i][j]
        return "".join(ans)
```

Complexity. Time: O(9 * target). Space: O(9 * target).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The SMALLEST integer that can be formed with exactly
# the target cost
# 2. The digit 0 may be used too (but the number must not
# start with 0)


# ============================================================
# FOLLOW-UP 1: The SMALLEST integer that can be formed
# with exactly the target cost
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same table, but compare the formed numbers
# by (fewest digits, then smallest) and use the digits in
# increasing order.
# Time: O(9 * target * target), string compares cost up to
# target.  Space: O(target^2).
def smallest_integer(cost, target):
    best = [None] * (target + 1)
    best[0] = ""
    for t in range(1, target + 1):
        for digit in range(1, 10):
            c = cost[digit - 1]
            if c <= t and best[t - c] is not None:
```

```
                candidate = best[t - c] + str(digit)
                if best[t] is None or len(candidate) < len(best[t]) or (len(candidate) == len(best[t]) and
 candidate < best[t]):
                    best[t] = candidate
    if best[target] is None:
        return "0"
    return best[target]


# ============================================================
# FOLLOW-UP 2: The digit 0 may be used too (but the number
# must not start with 0)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Build the best string for every cost from all
# ten digits (zeros are fine inside), then force the first
# digit to be non-zero by choosing it separately.
# Time: O(10 * target * target log target).  Space:
# O(target^2).
def largest_integer_with_zero(cost_with_zero, target):
    # cost_with_zero[d] is the cost of digit d for d = 0..9
    def better(a, b):
        if b is None:
            return True
        if len(a) != len(b):
            return len(a) > len(b)
        return a > b

    best = [None] * (target + 1)
    best[0] = ""
    for t in range(1, target + 1):
        for digit in range(10):
            c = cost_with_zero[digit]
            if c <= t and best[t - c] is not None:
                candidate = "".join(sorted(best[t - c] + str(digit), reverse=True))
```

```
                if better(candidate, best[t]):
                    best[t] = candidate
    answer = None
    for digit in range(1, 10):
        c = cost_with_zero[digit]
        if c <= target and best[target - c] is not None:
            candidate = str(digit) + best[target - c]
            if better(candidate, answer):
                answer = candidate
    return answer or "0"
```

956. Tallest Billboard

Hard · Array, Dynamic Programming, Knapsack, 0-1 Knapsack, Meet in the Middle

leetcode.com/problems/tallest-billboard

★ **TOP 7 PROBLEM** in Dynamic Programming - Knapsack Style: master this one first

PROBLEM

You are installing a billboard and want it to have the largest height. The billboard will have two steel supports, one on each side. Each steel support must be an equal height.

You are given a collection of `rods` that can be welded together. For example, if you have rods of lengths `1`, `2`, and `3`, you can weld them together to make a support of length `6`.

Return *the largest possible height of your billboard installation*. If you cannot support the billboard, return `0`.

Example 1:

```
Input: rods = [1,2,3,6]
Output: 6
Explanation: We have two disjoint subsets {1,2,3} and {6}, which have the same sum = 6.
```

Example 2:

```
Input: rods = [1,2,3,4,5,6]
Output: 10
Explanation: We have two disjoint subsets {2,3,5} and {4,6}, which have the same sum = 10.
```

Example 3:

```
Input: rods = [1,2]
Output: 0
Explanation: The billboard cannot be supported, so we return 0.
```

Constraints:

- `1 <= rods.length <= 20`
- `1 <= rods[i] <= 1000`
- `sum(rods[i]) <= 5000`

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Each rod goes left, right, or is discarded; the two sides should match and be as tall as possible. 3^n assignments are too many. The useful state is the largest common height after considering rod i with current difference j. Three transitions cover the choices; a difference of 0 is a feasible height. Memoization cuts repeats.

Python 3

```
class Solution:
    def tallestBillboard(self, rods: List[int]) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            if i >= len(rods):
                return 0 if j == 0 else -inf
            ans = max(dfs(i + 1, j), dfs(i + 1, j + rods[i]))
            ans = max(ans, dfs(i + 1, abs(rods[i] - j)) + min(j, rods[i]))
            return ans

        return dfs(0, 0)
```

Complexity. Time: O(n * S), S = sum of the rods. Space: O(n * S).

Solution 2

How to think about it

The search becomes bottom-up: f[i][j] is the best common height using the first i rods with difference j. The transitions match the recursion, without call overhead. The answer is f[n][0].

Python 3

```
class Solution:
    def tallestBillboard(self, rods: List[int]) -> int:
        n = len(rods)
        s = sum(rods)
        f = [[-inf] * (s + 1) for _ in range(n + 1)]
        f[0][0] = 0
        t = 0
        for i, x in enumerate(rods, 1):
            t += x
            for j in range(t + 1):
                f[i][j] = f[i - 1][j]
                if j >= x:
                    f[i][j] = max(f[i][j], f[i - 1][j - x])
                if j + x <= t:
                    f[i][j] = max(f[i][j], f[i - 1][j + x] + x)
                if j < x:
                    f[i][j] = max(f[i][j], f[i - 1][x - j] + x - j)
        return f[n][0]
```

Complexity. Time: O(n * S), S = sum of the rods. Space: O(n * S).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Meet in the middle for many rods


# ============================================================
# FOLLOW-UP 1: Meet in the middle for many rods
# ============================================================

# ---- follow-up 1: function ----
# Answer: Split the rods into two halves; for each half
# compute every possible difference between the two
# supports with the best shorter height. Combine two
# halves whose differences cancel out.
# Time: O(3^(n/2)).  Space: O(3^(n/2)).
def tallest_billboard_meet_in_middle(rods):
    def halves(items):
        states = {0: 0}                                  # difference -> best height of the shorter support
        for rod in items:
```

```
            new_states = dict(states)
            for diff, shorter in states.items():
                taller = shorter + diff
                # put the rod on the taller support
                key = diff + rod
                new_states[key] = max_of(new_states.get(key, -1), shorter)
                # put the rod on the shorter support
                new_short = shorter + rod
                key2 = abs(taller - new_short)
                new_states[key2] = max_of(new_states.get(key2, -1), min_of(taller, new_short))
            states = new_states
        return states

    middle = len(rods) // 2
    left = halves(rods[:middle])
    right = halves(rods[middle:])
    best = 0
    for diff, shorter in left.items():
        if diff in right:
            total = shorter + right[diff] + diff
            if total > best:
                best = total
    return best


def max_of(a, b):
    if a > b:
        return a
    return b


def min_of(a, b):
    if a < b:
        return a
    return b
```

879. Profitable Schemes

Hard · Array, Dynamic Programming, Knapsack, 0-1 Knapsack

leetcode.com/problems/profitable-schemes

PROBLEM

There is a group of `n` members, and a list of various crimes they could commit. The i^{th} crime generates a `profit[i]` and requires `group[i]` members to participate in it. If a member participates in one crime, that member can't participate in another crime.

Let's call a **profitable scheme** any subset of these crimes that generates at least `minProfit` profit, and the total number of members participating in that subset of crimes is at most `n`.

Return the number of schemes that can be chosen. Since the answer may be very large, **return it modulo** $10^9 + 7$.

Example 1:

```
Input: n = 5, minProfit = 3, group = [2,2], profit = [2,3]
Output: 2
Explanation: To make a profit of at least 3, the group could either commit crimes 0 and 1, or just
crime 1.
In total, there are 2 schemes.
```

Example 2:

```
Input: n = 10, minProfit = 5, group = [2,3,5], profit = [6,7,8]
Output: 7
Explanation: To make a profit of at least 5, the group could commit any crimes, as long as they commit
one.
There are 7 possible schemes: (0), (1), (2), (0,1), (0,2), (1,2), and (0,1,2).
```

Constraints:

- `1 <= n <= 100`
- `0 <= minProfit <= 100`
- `1 <= group.length <= 100`
- `1 <= group[i] <= 100`
- `profit.length == group.length`
- `0 <= profit[i] <= 100`

SOLUTIONS (PYTHON 3)

Solution 1: recursion with memoization

How to think about it

Each crime may be taken or skipped under a people cap and a profit floor. The state is “item i, j people used, profit capped at minProfit”. dfs(i,j,k) skips the crime or takes it when people remain. Capping profit merges all states that already meet the threshold. At the end, k must equal minProfit.

We design a function dfs(i, j, k), which means that we start from the i-th job, and have chosen j employees, and the current profit is k, then the number of schemes in this case is dfs(0, 0, 0).

The execution process of function dfs(i, j, k) is as follows:

- If i = n, it means that all the jobs have been considered. If k ≥ minProfit, the number of schemes is 1, otherwise the number of schemes is 0;
- If i < n, we can choose not to choose the i-th job, then the number of schemes is dfs(i + 1, j, k); if j + group[i] ≤ n, we can also choose the i-th job, then the number of schemes is dfs(i + 1, j + group[i], min(k + profit[i], minProfit)). Here we limit

the profit upper limit to minProfit, because the profit exceeding minProfit has no effect on our answer.

Finally, return dfs(0, 0, 0).

In order to avoid repeated calculation, we can use the method of memoization. We use a three-dimensional array f to record all the results of dfs(i, j, k). When we calculate the value of dfs(i, j, k), we store it in f[i][j][k]. When we call dfs(i, j, k), if f[i][j][k] has been calculated, we return f[i][j][k] directly.

The time complexity is O(m × n × minProfit), and th e space complexity is O(m × n × minProfit). Here m and n are the number of jobs and employees, and minProfit is the minimum profit.

Python 3

```
class Solution:
    def profitableSchemes(
        self, n: int, minProfit: int, group: List[int], profit: List[int]
    ) -> int:
        @cache
        def dfs(i: int, j: int, k: int) -> int:
            if i >= len(group):
                return 1 if k == minProfit else 0
            ans = dfs(i + 1, j, k)
            if j + group[i] <= n:
                ans += dfs(i + 1, j + group[i], min(k + profit[i], minProfit))
            return ans % (10**9 + 7)

        return dfs(0, 0, 0)
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> The same three-dimensional state fills by item index. f[i][j][k] is the number of ways using the first i crimes, j people, and profit at least k. Skipping copies the previous item; taking adds f[i-1][j-x][max(0,k-p)]. The empty plan contributes 1 at profit 0. The answer is f[m][n][minProfit].

We define f[i][j][k] to be the number of schemes to make a profit of at least k with i jobs and j workers. Initially, we have f[0][j][0] = 1, which means that there is only one scheme to make a profit of 0 without any jobs.

For the i-th job, we can choose to work or not to work. If we do not work, then f[i][j][k] = f[i - 1][j][k]; if we work, then f[i][j][k] = f[i - 1][j - group[i - 1]][max(0, k - profit[i - 1])]. We need to enumerate j and k, and add up all the schemes.

The final answer is f[m][n][minProfit].

The time complexity is O(m × n × minProfit), and the space complexity is O(m × n × minProfit). Here m and n are the numbers of jobs and workers, and minProfit is the minimum profit to make.

Python 3

```
class Solution:
    def profitableSchemes(
        self, n: int, minProfit: int, group: List[int], profit: List[int]
    ) -> int:
        mod = 10**9 + 7
        m = len(group)
        f = [[[0] * (minProfit + 1) for _ in range(n + 1)] for _ in range(m + 1)]
        for j in range(n + 1):
            f[0][j][0] = 1
        for i, (x, p) in enumerate(zip(group, profit), 1):
            for j in range(n + 1):
                for k in range(minProfit + 1):
                    f[i][j][k] = f[i - 1][j][k]
                    if j >= x:
                        f[i][j][k] = (f[i][j][k] + f[i - 1][j - x][max(0, k - p)]) % mod
        return f[m][n][minProfit]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The MAXIMUM profit using at most n members
# 2. How many schemes reach EACH profit level (capped at
# min_profit)


# ============================================================
# FOLLOW-UP 1: The MAXIMUM profit using at most n members
# ============================================================

# ---- follow-up 1: function ----
# Answer: A knapsack over the members: best[m] is the
# biggest profit with at most m members used.
# Time: O(G * n).  Space: O(n).
def max_profit_with_members(n, group, profit):
    best = [0] * (n + 1)
    for i in range(len(group)):
        for m in range(n, group[i] - 1, -1):
            if best[m - group[i]] + profit[i] > best[m]:
                best[m] = best[m - group[i]] + profit[i]
    return best[n]


# ============================================================
# FOLLOW-UP 2: How many schemes reach EACH profit level
# (capped at min_profit)
# ============================================================

# ---- follow-up 2: function ----
# Answer: The usual table, keeping the whole row of counts
# instead of only the last one.
# Time: O(G * n * P), P = min_profit.  Space: O(n * P).
def schemes_per_profit(n, min_profit, group, profit):
    mod = 10 ** 9 + 7
    ways = []
    for m in range(n + 1):
        ways.append([0] * (min_profit + 1))
    ways[0][0] = 1
    for i in range(len(group)):
        for m in range(n, group[i] - 1, -1):
            for p in range(min_profit, -1, -1):
                target = min_profit if p + profit[i] > min_profit else p + profit[i]
                ways[m][target] = (ways[m][target] + ways[m - group[i]][p]) % mod
    result = [0] * (min_profit + 1)
    for m in range(n + 1):
        for p in range(min_profit + 1):
            result[p] = (result[p] + ways[m][p]) % mod
    return result
```

871. Minimum Number of Refueling Stops

Hard · Greedy, Array, Dynamic Programming, Heap (Priority Queue)

leetcode.com/problems/minimum-number-of-refueling-stops

PROBLEM

A car travels from a starting position to a destination which is `target` miles east of the starting position.

There are gas stations along the way. The gas stations are represented as an array `stations` where `stations[i] = [position`$_i$`, fuel`$_i$`]` indicates that the `i`th gas station is `position`$_i$ miles east of the starting position and has `fuel`$_i$ liters of gas.

The car starts with an infinite tank of gas, which initially has `startFuel` liters of fuel in it. It uses one liter of gas per one mile that it drives. When the car reaches a gas station, it may stop and refuel, transferring all the gas from the station into the car.

Return *the minimum number of refueling stops the car must make in order to reach its destination*. If it cannot reach the destination, return `-1`.

Note that if the car reaches a gas station with `0` fuel left, the car can still refuel there. If the car reaches the destination with `0` fuel left, it is still considered to have arrived.

Example 1:

```
Input: target = 1, startFuel = 1, stations = []
Output: 0
Explanation: We can reach the target without refueling.
```

Example 2:

```
Input: target = 100, startFuel = 1, stations = [[10,100]]
Output: -1
Explanation: We can not reach the target (or even the first gas station).
```

Example 3:

```
Input: target = 100, startFuel = 10, stations = [[10,60],[20,30],[30,30],[60,40]]
Output: 2
Explanation: We start with 10 liters of fuel.
We drive to position 10, expending 10 liters of fuel. We refuel from 0 liters to 60 liters of gas.
Then, we drive from position 10 to position 60 (expending 50 liters of fuel),
and refuel from 10 liters to 50 liters of gas. We then drive to and reach the target.
We made 2 refueling stops along the way, so we return 2.
```

Constraints:

- `1 <= target, startFuel <= 10`^{9}
- `0 <= stations.length <= 500`
- `1 <= position`$_i$ `< position`$_{i+1}$ `< target`
- `1 <= fuel`$_i$ `< 10`^{9}

SOLUTION (PYTHON 3)

Solution 1: Greedy + Priority Queue (Max-Heap)

> **How to think about it**
>
> Choose refuels to reach the target with as few stops as possible. The target is huge and there are ≤ 500 stations. When the tank cannot reach the next station, we should have refueled at the richest station already passed. Scan by position, pushing passed fuel into a max-heap; while the tank is negative, pop the largest fuel. An empty heap means unreachable. The target is treated as a final station.

We can use a priority queue (max-heap) pq to record the fuel amounts of all the gas stations we have passed. Each time the fuel is insufficient, we greedily take out the maximum fuel amount, which is the top element of pq, and accumulate the number of refuels ans. If pq is empty and the current fuel is still insufficient, it means we cannot reach the destination, and we return -1.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n represents the number of gas stations.

Python 3

```python
class Solution:
    def minRefuelStops(
        self, target: int, startFuel: int, stations: List[List[int]]
    ) -> int:
        pq = []
        ans = pre = 0
        stations.append([target, 0])
        for pos, fuel in stations:
            dist = pos - pre
            startFuel -= dist
            while startFuel < 0 and pq:
                startFuel -= heappop(pq)
                ans += 1
            if startFuel < 0:
                return -1
            heappush(pq, -fuel)
            pre = pos
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return which stations are used
# 2. Dynamic programming version: the farthest distance
# with exactly j stops


# ============================================================
# FOLLOW-UP 1: Return which stations are used
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same greedy with a max-heap of the fuel
# amounts, keeping the station index with each amount.
# Time: O(s log s), s = stations.  Space: O(s).
import heapq


def refuel_stops(target, start_fuel, stations):
    heap = []
    used = []
    fuel = start_fuel
    index = 0
    while fuel < target:
        while index < len(stations) and stations[index][0] <= fuel:
```

```
            heapq.heappush(heap, (-stations[index][1], index))
            index += 1
        if not heap:
            return None
        negative, station = heapq.heappop(heap)
        fuel += -negative
        used.append(station)
    return sorted(used)


# ============================================================
# FOLLOW-UP 2: Dynamic programming version: the farthest
# distance with exactly j stops
# ============================================================

# ---- follow-up 2: function ----
# Answer: best[j] is the farthest position reachable with
# j refuelings; a station can extend best[j] to best[j +
# 1] when it is reachable with j stops.
# Time: O(s^2).  Space: O(s).
def refuel_dp(target, start_fuel, stations):
    n = len(stations)
    best = [0] * (n + 1)
    best[0] = start_fuel
    for i in range(n):
        for j in range(i, -1, -1):
            if best[j] >= stations[i][0]:
                if best[j] + stations[i][1] > best[j + 1]:
                    best[j + 1] = best[j] + stations[i][1]
    for j in range(n + 1):
        if best[j] >= target:
            return j
    return -1
```

2218. Maximum Value of K Coins From Piles

Hard · Array, Dynamic Programming, Prefix Sum

leetcode.com/problems/maximum-value-of-k-coins-from-piles

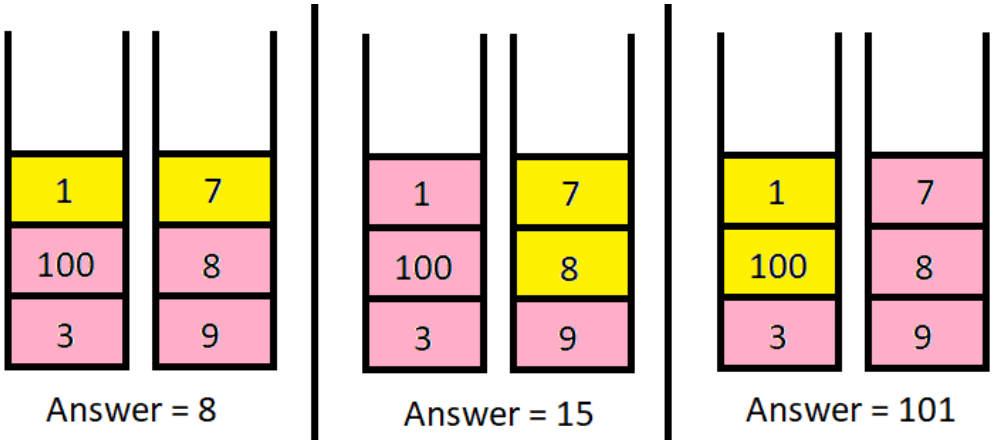
PROBLEM

There are `n` **piles** of coins on a table. Each pile consists of a **positive number** of coins of assorted denominations.

In one move, you can choose any coin on **top** of any pile, remove it, and add it to your wallet.

Given a list `piles`, where `piles[i]` is a list of integers denoting the composition of the i^{th} pile from **top to bottom**, and a positive integer `k`, return *the **maximum total value** of coins you can have in your wallet if you choose **exactly** `k` coins optimally*.

Example 1:




```
Input: piles = [[1,100,3],[7,8,9]], k = 2
Output: 101
Explanation:
The above diagram shows the different ways we can choose k coins.
The maximum total we can obtain is 101.
```

Example 2:

```
Input: piles = [[100],[100],[100],[100],[100],[100],[1,1,1,1,1,1,700]], k = 7
Output: 706
Explanation:
The maximum total can be obtained if we choose all coins from the last pile.
```

Constraints:

- `n == piles.length`
- `1 <= n <= 1000`
- `1 <= piles[i][j] <=` 10^5
- `1 <= k <= sum(piles[i].length) <= 2000`

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming (Grouped Knapsack)

How to think about it

Coins must be taken as a prefix of each pile, at most k in total. Searching allocations would recompute the same prefixes; n and k are up to a thousand and there are 2000 coins. Piles are independent groups: taking the first h coins of a pile is one item of size h. Let f[i][j] be the best value from the first i piles using j coins. Prefix-sum pile i into s[h] and relax f[i][j] by f[i-1][j-h]+s[h]. The work is proportional to k times the total number of coins.

We define f[i][j] as the maximum value sum of taking j coins from the first i piles. The answer is f[n][k], where n is the number of piles.

For the i-th pile, we can choose to take the first 0, 1, 2, …, k coins. We can use a prefix sum array s to quickly calculate the value sum of taking the first h coins.

The state transition equation is:

$$f[i][j] = \max(f[i][j], f[i - 1][j - h] + s[h])$$

where $0 \le h \le j$, and s[h] represents the value sum of taking the first h coins from the i-th pile.

The time complexity is $O(k \times L)$, and the space complexity is $O(n \times k)$. Here, L is the total number of coins, and n is the number of piles.

Python 3

```python
class Solution:
    def maxValueOfCoins(self, piles: List[List[int]], k: int) -> int:
        n = len(piles)
        f = [[0] * (k + 1) for _ in range(n + 1)]
        for i, nums in enumerate(piles, 1):
            s = list(accumulate(nums, initial=0))
            for j in range(k + 1):
                for h, w in enumerate(s):
                    if j < h:
                        break
                    f[i][j] = max(f[i][j], f[i - 1][j - h] + w)
        return f[n][k]
```

Solution 2: Dynamic Programming (Space Optimization)

How to think about it

Solution 1 only reads the previous row f[i-1][·], so the full table is unnecessary. After flattening f, capacity j must be enumerated downwards so the same pile is not used twice. Each pile is still prefix-summed, then j and h update f[j] in place. Time is unchanged; extra space drops to O(k).

We can observe that for the i-th pile, we only need to use f[i - 1][j] and f[i][j - h], so we can optimize the two-dimensional array to a one-dimensional array.

The time complexity is $O(k \times L)$, and the space complexity is $O(k)$.

Python 3

```python
class Solution:
    def maxValueOfCoins(self, piles: List[List[int]], k: int) -> int:
        f = [0] * (k + 1)
        for nums in piles:
            s = list(accumulate(nums, initial=0))
            for j in range(k, -1, -1):
                for h, w in enumerate(s):
                    if j < h:
                        break
                    f[j] = max(f[j], f[j - h] + w)
        return f[k]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return how many coins are taken from each pile


# ========================================================
# FOLLOW-UP 1: Return how many coins are taken from each
# pile
# ========================================================

# ---- follow-up 1: function ----
# Answer: Remember for every pile and count the number of
# coins that gave the best total, then walk back through
# the piles.
# Time: O(n * k * c), c = largest pile.  Space: O(n * k).
def coins_per_pile(piles, k):
    n = len(piles)
    best = []
    choice = []
    for i in range(n + 1):
        best.append([float("-inf")] * (k + 1))
        choice.append([0] * (k + 1))
    best[0][0] = 0
    for i in range(1, n + 1):
        prefix = [0]
        for coin in piles[i - 1]:
            prefix.append(prefix[-1] + coin)
        for total in range(k + 1):
            for take in range(0, min(total, len(piles[i - 1])) + 1):
                value = best[i - 1][total - take] + prefix[take]
                if value > best[i][total]:
                    best[i][total] = value
                    choice[i][total] = take
    counts = [0] * n
    remaining = k
    for i in range(n, 0, -1):
        counts[i - 1] = choice[i][remaining]
        remaining -= counts[i - 1]
    return best[n][k], counts
```

1755. Closest Subsequence Sum

Hard · Bit Manipulation, Array, Two Pointers, Dynamic Programming, Bitmask, Sorting, Meet in the Middle

leetcode.com/problems/closest-subsequence-sum

PROBLEM

You are given an integer array `nums` and an integer `goal`.

You want to choose a subsequence of `nums` such that the sum of its elements is the closest possible to `goal`. That is, if the sum of the subsequence's elements is `sum`, then you want to **minimize the absolute difference** `abs(sum - goal)`.

Return *the **minimum** possible value of* `abs(sum - goal)`.

Note that a subsequence of an array is an array formed by removing some elements **(possibly all or none)** of the original array.

Example 1:

```
Input: nums = [5,-7,3,5], goal = 6
Output: 0
Explanation: Choose the whole array as a subsequence, with a sum of 6.
This is equal to the goal, so the absolute difference is 0.
```

Example 2:

```
Input: nums = [7,-9,15,-2], goal = -5
Output: 1
Explanation: Choose the subsequence [7,-9,-2], with a sum of -4.
The absolute difference is abs(-4 - (-5)) = abs(1) = 1, which is the minimum.
```

Example 3:

```
Input: nums = [1,2,3], goal = -7
Output: 7
```

Constraints:

- `1 <= nums.length <= 40`
- `-10^7 <= nums[i] <= 10^7`
- `-10^9 <= goal <= 10^9`

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Find a subsequence sum closest to goal. n≤ 40 makes 2^n too large, but $2^{n/2}$ is acceptable. Split the array, DFS all subset sums on each half, and for every left sum binary-search goal-l in the sorted right sums, keeping the two nearest candidates.

Python 3

```
class Solution:
    def minAbsDifference(self, nums: List[int], goal: int) -> int:
        n = len(nums)
        left = set()
        right = set()

        self.getSubSeqSum(0, 0, nums[: n // 2], left)
        self.getSubSeqSum(0, 0, nums[n // 2 :], right)

        result = inf
        right = sorted(right)
        rl = len(right)

        for l in left:
            remaining = goal - l
            idx = bisect_left(right, remaining)

            if idx < rl:
                result = min(result, abs(remaining - right[idx]))

            if idx > 0:
                result = min(result, abs(remaining - right[idx - 1]))

        return result

    def getSubSeqSum(self, i: int, curr: int, arr: List[int], result: Set[int]):
        if i == len(arr):
            result.add(curr)
            return

        self.getSubSeqSum(i + 1, curr, arr, result)
        self.getSubSeqSum(i + 1, curr + arr[i], arr, result)
```

Complexity. Time: O(2^(n/2) * n). Space: O(2^(n/2)).

Solution 2

> **How to think about it**
>
> Solution 2 is the same meet-in-the-middle search; the DFS simply tries skip-then-take. The algorithm is unchanged, the code is tighter.

Python 3

```
class Solution:
    def minAbsDifference(self, nums: List[int], goal: int) -> int:
        def dfs(arr, res, i, s):
            if i == len(arr):
                res.add(s)
                return
            dfs(arr, res, i + 1, s)
            dfs(arr, res, i + 1, s + arr[i])

        n = len(nums)
        left, right = set(), set()
        dfs(nums[: n >> 1], left, 0, 0)
        dfs(nums[n >> 1 :], right, 0, 0)
        right = sorted(right)
        ans = inf
        for l in left:
            x = goal - l
            i = bisect_left(right, x)
            if i < len(right):
                ans = min(ans, abs(x - right[i]))
            if i:
                ans = min(ans, abs(x - right[i - 1]))
        return ans
```

Complexity. Time: O(2^(n/2) * n). Space: O(2^(n/2)).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subsequence whose sum is closest to the
# goal
# 2. Count the subsequences whose sum is exactly the goal


# ============================================================
# FOLLOW-UP 1: Return the subsequence whose sum is closest
# to the goal
# ============================================================

# ---- follow-up 1: function ----
# Answer: Meet in the middle: all subset sums of the left
# half and of the right half; for every left sum look for
# the right sum closest to goal - left sum (binary
# search), remembering the subsets.
# Time: O(2^(n/2) * n).  Space: O(2^(n/2) * n).
import bisect


def closest_subset(nums, goal):
    def all_subsets(items):
        subsets = [(0, [])]
        for x in items:
            subsets += [(s + x, chosen + [x]) for s, chosen in subsets]
        return subsets

    middle = len(nums) // 2
    left = all_subsets(nums[:middle])
    right = sorted(all_subsets(nums[middle:]), key=lambda pair: pair[0])
    sums = [pair[0] for pair in right]
    best = None
    for s, chosen in left:
        position = bisect.bisect_left(sums, goal - s)
        for p in (position - 1, position):
            if 0 <= p < len(right):
                total = s + right[p][0]
                if best is None or abs(total - goal) < abs(best[0] - goal):
                    best = (total, chosen + right[p][1])
    return best


# ============================================================
# FOLLOW-UP 2: Count the subsequences whose sum is exactly
# the goal
# ============================================================

# ---- follow-up 2: function ----
# Answer: Meet in the middle again: count the right-half
# sums in a dictionary and add the matching counts for
# every left sum.
# Time: O(2^(n/2)).  Space: O(2^(n/2)).
def count_subsets_with_sum(nums, goal):
    def all_sums(items):
        sums = [0]
        for x in items:
            sums += [s + x for s in sums]
        return sums

    middle = len(nums) // 2
```

```
    right_counts = {}
    for s in all_sums(nums[middle:]):
```

```
        right_counts[s] = right_counts.get(s, 0) + 1
    total = 0
    for s in all_sums(nums[:middle]):
        total += right_counts.get(goal - s, 0)
    return total
```

1269. Number of Ways to Stay in the Same Place After Some Steps

Hard · Dynamic Programming

leetcode.com/problems/number-of-ways-to-stay-in-the-same-place-after-some-steps

PROBLEM

You have a pointer at index `0` in an array of size `arrLen`. At each step, you can move 1 position to the left, 1 position to the right in the array, or stay in the same place (The pointer should not be placed outside the array at any time).

Given two integers `steps` and `arrLen`, return the number of ways such that your pointer is still at index `0` after **exactly** `steps` steps. Since the answer may be too large, return it **modulo** $10^9 + 7$.

Example 1:

```
Input: steps = 3, arrLen = 2
Output: 4
Explanation: There are 4 differents ways to stay at index 0 after 3 steps.
Right, Left, Stay
Stay, Right, Left
Right, Stay, Left
Stay, Stay, Stay
```

Example 2:

```
Input: steps = 2, arrLen = 4
Output: 2
Explanation: There are 2 differents ways to stay at index 0 after 2 steps
Right, Left
Stay, Stay
```

Example 3:

```
Input: steps = 4, arrLen = 2
Output: 8
```

Constraints:

- `1 <= steps <= 500`
- `1 <= arrLen <= 10^6`

SOLUTION (PYTHON 3)

Solution 1: Memoization Search

How to think about it

We walk steps steps on an array of length arrLen and return to the origin. arrLen may be 10^6, but steps ≤ 500, so reachable cells are at most steps. The count depends on position and remaining steps. dfs(i,j) is the number of ways from index i with j steps left to finish at 0; we may go left, right, or stay. If i>j we cannot return. Memoized states are O(steps^2).

We observe the data range of the problem and find that steps does not exceed 500, which means that we can only go to the right for up to 500 steps.

We can design a function dfs(i, j), which represents the number of schemes when we are currently at position i and the remaining steps are j. So the answer is dfs(0, steps).

The execution process of the function dfs(i, j) is as follows:

1. If i > j or i ≥ arrLen or i < 0 or j < 0, then return 0.
1. If i = 0 and j = 0, then the pointer has stopped in place and there are no remaining steps, so return 1.
1. Otherwise, we can choose to move one step to the left, one step to the right, or stay still, so return dfs(i - 1, j - 1) + dfs(i + 1, j - 1) + dfs(i, j - 1). Note the modulo operation of the answer.

During the process, we can use memoization search to avoid repeated calculations.

The time complexity is O(steps × steps), and the space complexity is O(steps × steps). Where steps is the number of steps given in the problem.

Python 3

```python
class Solution:
    def numWays(self, steps: int, arrLen: int) -> int:
        @cache
        def dfs(i, j):
            if i > j or i >= arrLen or i < 0 or j < 0:
                return 0
            if i == 0 and j == 0:
                return 1
            ans = 0
            for k in range(-1, 2):
                ans += dfs(i + k, j - 1)
                ans %= mod
            return ans

        mod = 10**9 + 7
        return dfs(0, steps)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. You MUST move at every step (no 'stay' option)
# 2. End at a given position instead of position 0


# ============================================================
# FOLLOW-UP 1: You MUST move at every step (no 'stay'
# option)
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same table without the 'stay in place'
# transition.
# Time: O(steps * min(steps, array_length)).  Space:
# O(min(steps, array_length)).
def ways_must_move(steps, array_length):
    mod = 10 ** 9 + 7
    limit = min_of(array_length, steps // 2 + 1)
    ways = [0] * limit
    ways[0] = 1
    for step in range(steps):
        new_ways = [0] * limit
        for position in range(limit):
            if ways[position] == 0:
                continue
            if position > 0:
                new_ways[position - 1] = (new_ways[position - 1] + ways[position]) % mod
            if position + 1 < limit:
                new_ways[position + 1] = (new_ways[position + 1] + ways[position]) % mod
        ways = new_ways
    return ways[0]


def min_of(a, b):
    if a < b:
```

```
        return a
    return b


# ============================================================
# FOLLOW-UP 2: End at a given position instead of position
# 0
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same table; read the answer from the target
# position (positions beyond steps cannot matter, but keep
# the array limit).
# Time: O(steps * array_length).  Space: O(array_length).
def ways_to_end_at(steps, array_length, end):
    mod = 10 ** 9 + 7
    ways = [0] * array_length
    ways[0] = 1
    for step in range(steps):
        new_ways = [0] * array_length
        for position in range(array_length):
            if ways[position] == 0:
                continue
            for move in (-1, 0, 1):
                nxt = position + move
```

```
                if 0 <= nxt < array_length:
                    new_ways[nxt] = (new_ways[nxt] + ways[position]) % mod
        ways = new_ways
    return ways[end]
```

PATTERN 29 OF 41

Design Problems

20 problems · 1 Easy · 16 Medium · 3 Hard

THE IDEA

Choose data structures so every operation meets its time limit: a dictionary for lookup, a doubly linked list or deque for order, a heap for priority.

Signals: LRU cache, LFU cache, min stack, implement a queue with stacks, insert / delete / getRandom in O(1), iterator, rate limiter.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```python
from collections import OrderedDict
class LRUCache:
    def __init__(self, capacity):
        self.cap, self.data = capacity, OrderedDict()
    def get(self, key):
        if key not in self.data:
            return -1
        self.data.move_to_end(key)              # most recent
        return self.data[key]
    def put(self, key, value):
        self.data[key] = value
        self.data.move_to_end(key)
        if len(self.data) > self.cap:
            self.data.popitem(last=False)       # drop the oldest
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Design HashMap	Easy	LC 706	1545
2	★ **LRU Cache** (see page)	Medium	LC 146	1380
3	Min Stack (see page)	Medium	LC 155	398
4	★ **Implement Trie (Prefix Tree)**	Medium	LC 208	1548
5	★ **Design Twitter**	Medium	LC 355	1552
6	★ **Insert Delete GetRandom O(1)** (see page)	Medium	LC 380	58
7	Design Underground System	Medium	LC 1396	1556
8	Design Browser History	Medium	LC 1472	1560
9	★ **Time Based Key-Value Store** (see page)	Medium	LC 981	222
10	Design Circular Deque (see page)	Medium	LC 641	494
11	Snapshot Array	Medium	LC 1146	1564
12	Design a Stack With Increment Operation (see page)	Medium	LC 1381	1374
13	Online Stock Span (see page)	Medium	LC 901	443
14	Design Add and Search Words Data Structure	Medium	LC 211	1567
15	Stock Price Fluctuation	Medium	LC 2034	1570
16	Design a Food Rating System	Medium	LC 2353	1574
17	Design Authentication Manager	Medium	LC 1797	1577
18	★ **LFU Cache**	Hard	LC 460	1581
19	★ **Find Median from Data Stream**	Hard	LC 295	1586
20	Serialize and Deserialize Binary Tree	Hard	LC 297	1589

PATTERN 29 · DESIGN PROBLEMS · PROBLEM 1 OF 20

706. Design HashMap

Easy · Design, Array, Hash Table, Linked List, Hash Function

leetcode.com/problems/design-hashmap

PROBLEM

Design a HashMap without using any built-in hash table libraries.

Implement the `MyHashMap` class:

- `MyHashMap()` initializes the object with an empty map.
- `void put(int key, int value)` inserts a `(key, value)` pair into the HashMap. If the `key` already exists in the map, update the corresponding `value`.
- `int get(int key)` returns the `value` to which the specified `key` is mapped, or `-1` if this map contains no mapping for the `key`.
- `void remove(key)` removes the `key` and its corresponding `value` if the map contains the mapping for the `key`.

Example 1:

```
Input
["MyHashMap", "put", "put", "get", "get", "put", "get", "remove", "get"]
[[], [1, 1], [2, 2], [1], [3], [2, 1], [2], [2], [2]]
Output
[null, null, null, 1, -1, null, 1, null, -1]

Explanation
MyHashMap myHashMap = new MyHashMap();
myHashMap.put(1, 1); // The map is now [[1,1]]
myHashMap.put(2, 2); // The map is now [[1,1], [2,2]]
myHashMap.get(1); // return 1, The map is now [[1,1], [2,2]]
myHashMap.get(3); // return -1 (i.e., not found), The map is now [[1,1], [2,2]]
myHashMap.put(2, 1); // The map is now [[1,1], [2,1]] (i.e., update the existing value)
myHashMap.get(2); // return 1, The map is now [[1,1], [2,1]]
myHashMap.remove(2); // remove the mapping for 2, The map is now [[1,1]]
myHashMap.get(2); // return -1 (i.e., not found), The map is now [[1,1]]
```

Constraints:

- $0 <= key, value <= 10^6$
- At most 10^4 calls will be made to `put`, `get`, and `remove`.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

We need a map whose keys and values lie in [0, 10^6], with at most 10^4 operations. As with the hash set, the key itself is a valid index. Allocate an array of length 10^6+1 filled with -1. put, get, and remove become an assignment, a read, and a write of -1. The sentinel -1 is outside the value range, so it cannot collide with a stored value. Space follows the universe, which is acceptable here.

Python 3

```
class MyHashMap:
    def __init__(self):
        self.data = [-1] * 1000001

    def put(self, key: int, value: int) -> None:
        self.data[key] = value

    def get(self, key: int) -> int:
        return self.data[key]

    def remove(self, key: int) -> None:
        self.data[key] = -1


# Your MyHashMap object will be instantiated and called as such:
# obj = MyHashMap()
# obj.put(key,value)
# param_2 = obj.get(key)
# obj.remove(key)
```

Complexity. Time: O(1) per operation. Space: O(M), M = 10^6 slots.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Grow the table when it gets too full (keep chains
# short)
# 2. List all keys, in the order they were first inserted


# ============================================================
# FOLLOW-UP 1: Grow the table when it gets too full (keep
# chains short)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count the entries; when entries / buckets passes
# 0.75, double the number of buckets and move every pair
# into its new bucket.
# Time: put, get and remove are O(1) amortised; growing
# costs O(n) occasionally.  Space: O(n).
class ResizingHashMap:
    def __init__(self):
        self.buckets = [[] for _ in range(8)]
        self.size = 0

    def put(self, key, value):
        bucket = self.buckets[key % len(self.buckets)]
        for pair in bucket:
            if pair[0] == key:
                pair[1] = value
                return
        bucket.append([key, value])
        self.size += 1
        if self.size > 0.75 * len(self.buckets):
            self.grow()

    def get(self, key):
        for pair in self.buckets[key % len(self.buckets)]:
            if pair[0] == key:
                return pair[1]
        return -1

    def remove(self, key):
        bucket = self.buckets[key % len(self.buckets)]
        for i in range(len(bucket)):
```

```
            if bucket[i][0] == key:
                bucket.pop(i)
                self.size -= 1
                return

    def grow(self):
        old = self.buckets
        self.buckets = [[] for _ in range(2 * len(old))]
        for bucket in old:
            for key, value in bucket:
                self.buckets[key % len(self.buckets)].append([key, value])


# ============================================================
# FOLLOW-UP 2: List all keys, in the order they were first
# inserted
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: A Python dictionary remembers the order keys
# were first inserted (changing a value keeps the
# position), so keep the pairs in a dictionary and list
# its keys.
# Time: put, get and remove O(1); keys O(n).  Space: O(n).
class OrderedHashMap:
    def __init__(self):
        self.data = {}

    def put(self, key, value):
        self.data[key] = value

    def get(self, key):
        return self.data.get(key, -1)

    def remove(self, key):
        if key in self.data:
            del self.data[key]

    def keys(self):
        return list(self.data.keys())
```

208. Implement Trie (Prefix Tree)

Medium · Design, Trie, Hash Table, String

leetcode.com/problems/implement-trie-prefix-tree

★ **TOP 7 PROBLEM** in Design Problems, Trie (Prefix Tree): master this one first

Also appears in pattern 32 Trie (Prefix Tree)

PROBLEM

A **trie** (pronounced as "try") or **prefix tree** is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. There are various applications of this data structure, such as autocomplete and spellchecker.

Implement the Trie class:

- `Trie()` Initializes the trie object.
- `void insert(String word)` Inserts the string `word` into the trie.
- `boolean search(String word)` Returns `true` if the string `word` is in the trie (i.e., was inserted before), and `false` otherwise.
- `boolean startsWith(String prefix)` Returns `true` if there is a previously inserted string `word` that has the prefix `prefix`, and `false` otherwise.

Example 1:

```
Input
["Trie", "insert", "search", "search", "startsWith", "insert", "search"]
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]
Output
[null, null, true, false, true, null, true]

Explanation
Trie trie = new Trie();
trie.insert("apple");
trie.search("apple"); // return True
trie.search("app"); // return False
trie.startsWith("app"); // return True
trie.insert("app");
trie.search("app"); // return True
```

Constraints:

- `1 <= word.length, prefix.length <= 2000`
- `word` and `prefix` consist only of lowercase English letters.
- At most `3 * 10^4` calls **in total** will be made to `insert`, `search`, and `startsWith`.

SOLUTION (PYTHON 3)

Solution 1: Trie (Prefix Tree)

How to think about it

Scanning every inserted word for each prefix query repeats work on shared prefixes. Letters are lowercase, so a 26-way tree can layer characters. Insertion walks (and creates) nodes and marks isEnd at the last one. Prefix search follows the same path; full-word search also checks the end flag.

Each node in the trie contains two parts:

1. An array of pointers to child nodes `children`. For this problem, the array length is 26, representing the number of lowercase English letters. `children[0]` corresponds to lowercase letter 'a', ..., and `children[25]` corresponds to

lowercase letter 'z'.

2. A boolean field `isEnd` indicating whether the node is the end of a string.

1. Insert a String

We start from the root of the trie and insert the string. For the child node corresponding to the current character, there are two cases:

- The child node exists. Move along the pointer to the child node and continue processing the next character.
- The child node does not exist. Create a new child node, record it in the corresponding position of the `children` array, then move along the pointer to the child node and continue searching for the next character.

Repeat the above steps until the last character of the string is processed, then mark the current node as the end of the string.

2. Search for a Prefix

We start from the root of the trie and search for the prefix. For the child node corresponding to the current character, there are two cases:

- The child node exists. Move along the pointer to the child node and continue searching for the next character.
- The child node does not exist. This means the trie does not contain the prefix, so return a null pointer.

Repeat the above steps until a null pointer is returned or the last character of the prefix is searched.

If we search to the end of the prefix, it means the trie contains the prefix. Additionally, if the `isEnd` of the node corresponding to the end of the prefix is true, it means the trie contains the string.

The time complexity for inserting a string is $O(m \times |\Sigma|)$, and the time complexity for searching a prefix is $O(m)$, where m is the length of the string and $|\Sigma|$ is the size of the character set (26 in this problem). The space complexity is $O(q \times m \times |\Sigma|)$, where q is the number of inserted strings.

Python 3

```python
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, word: str) -> None:
        node = self
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, word: str) -> bool:
        node = self._search_prefix(word)
        return node is not None and node.is_end

    def startsWith(self, prefix: str) -> bool:
        node = self._search_prefix(prefix)
        return node is not None

    def _search_prefix(self, prefix: str):
        node = self
        for c in prefix:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                return None
            node = node.children[idx]
        return node


# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)
# param_2 = obj.search(word)
# param_3 = obj.startsWith(prefix)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Delete a word
# 2. Count how many inserted words start with a prefix


# ============================================================
# FOLLOW-UP 1: Delete a word
# ============================================================

# ---- follow-up 1: function ----
# Answer: Walk down the word, then walk back up removing
# nodes that no longer lead to any word.
# Time: insert, search, starts_with and delete are each
# O(L), L = word length.  Space: O(total letters).
class TrieWithDelete:
    def __init__(self):
        self.children = {}
        self.is_word = False

    def insert(self, word):
        node = self
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieWithDelete()
            node = node.children[ch]
```

```
        node.is_word = True

    def search(self, word):
        node = self.find(word)
        return node is not None and node.is_word

    def starts_with(self, prefix):
        return self.find(prefix) is not None

    def find(self, text):
        node = self
        for ch in text:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node

    def delete(self, word):
        path = [self]
        for ch in word:
            if ch not in path[-1].children:
                return False
            path.append(path[-1].children[ch])
        if not path[-1].is_word:
            return False
        path[-1].is_word = False
        for i in range(len(word), 0, -1):
            node = path[i]
            if node.is_word or node.children:
                break
            del path[i - 1].children[word[i - 1]]
        return True


# ============================================================
```

```
# FOLLOW-UP 2: Count how many inserted words start with a
# prefix
# ============================================================

# ---- follow-up 2: function ----
# Answer: Store a counter in every node: how many inserted
# words pass through it.
# Time: insert and count_prefix are each O(L).  Space:
# O(total letters).
class TrieWithCounts:
    def __init__(self):
        self.children = {}
        self.passing = 0

    def insert(self, word):
        node = self
        node.passing += 1
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieWithCounts()
            node = node.children[ch]
            node.passing += 1

    def count_prefix(self, prefix):
        node = self
        for ch in prefix:
            if ch not in node.children:
                return 0
            node = node.children[ch]
        return node.passing
```

355. Design Twitter

Medium · Design, Hash Table, Linked List, Heap (Priority Queue)

leetcode.com/problems/design-twitter

★ **TOP 7 PROBLEM** in Design Problems: master this one first

PROBLEM

Design a simplified version of Twitter where users can post tweets, follow/unfollow another user, and is able to see the `10` most recent tweets in the user's news feed.

Implement the `Twitter` class:

- `Twitter()` Initializes your twitter object.
- `void postTweet(int userId, int tweetId)` Composes a new tweet with ID `tweetId` by the user `userId`. Each call to this function will be made with a unique `tweetId`.
- `List<Integer> getNewsFeed(int userId)` Retrieves the `10` most recent tweet IDs in the user's news feed. Each item in the news feed must be posted by users who the user followed or by the user themself. Tweets must be **ordered from most recent to least recent**.
- `void follow(int followerId, int followeeId)` The user with ID `followerId` started following the user with ID `followeeId`.
- `void unfollow(int followerId, int followeeId)` The user with ID `followerId` started unfollowing the user with ID `followeeId`.

Example 1:

```
Input
["Twitter", "postTweet", "getNewsFeed", "follow", "postTweet", "getNewsFeed", "unfollow",
"getNewsFeed"]
[[], [1, 5], [1], [1, 2], [2, 6], [1], [1, 2], [1]]
Output
[null, null, [5], null, null, [6, 5], null, [5]]

Explanation
Twitter twitter = new Twitter();
twitter.postTweet(1, 5); // User 1 posts a new tweet (id = 5).
twitter.getNewsFeed(1); // User 1's news feed should return a list with 1 tweet id -> [5]. return [5]
twitter.follow(1, 2); // User 1 follows user 2.
twitter.postTweet(2, 6); // User 2 posts a new tweet (id = 6).
twitter.getNewsFeed(1); // User 1's news feed should return a list with 2 tweet ids -> [6, 5]. Tweet id
6 should precede tweet id 5 because it is posted after tweet id 5.
twitter.unfollow(1, 2); // User 1 unfollows user 2.
twitter.getNewsFeed(1); // User 1's news feed should return a list with 1 tweet id -> [5], since user 1
is no longer following user 2.
```

Constraints:

- `1 <= userId, followerId, followeeId <= 500`
- `0 <= tweetId <= 10^4`
- All the tweets have **unique** IDs.
- At most `3 * 10^4` calls will be made to `postTweet`, `getNewsFeed`, `follow`, and `unfollow`.
- A user cannot follow himself.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Post, follow, and fetch the ten most recent tweets from a user and followees. A full scan of all tweets is wasteful. Store each user's tweets and follow set, and stamp tweets with a global clock. A feed gathers the latest ten from each relevant user and keeps the ten newest. Follow/unfollow only edit the set.

Python 3

```
class Twitter:
    def __init__(self):
        """
        Initialize your data structure here.
        """
        self.user_tweets = defaultdict(list)
        self.user_following = defaultdict(set)
        self.tweets = defaultdict()
        self.time = 0

    def postTweet(self, userId: int, tweetId: int) -> None:
        """
        Compose a new tweet.
        """
        self.time += 1
        self.user_tweets[userId].append(tweetId)
        self.tweets[tweetId] = self.time

    def getNewsFeed(self, userId: int) -> List[int]:
        """
        Retrieve the 10 most recent tweet ids in the user's news feed. Each item in the news feed must be
posted by users who the user followed or by the user herself. Tweets must be ordered from most recent to
least recent.
        """
        following = self.user_following[userId]
        users = set(following)
        users.add(userId)
        tweets = [self.user_tweets[u][::-1][:10] for u in users]
        tweets = sum(tweets, [])
        return nlargest(10, tweets, key=lambda tweet: self.tweets[tweet])

    def follow(self, followerId: int, followeeId: int) -> None:
        """
        Follower follows a followee. If the operation is invalid, it should be a no-op.
        """
        self.user_following[followerId].add(followeeId)

    def unfollow(self, followerId: int, followeeId: int) -> None:
        """
        Follower unfollows a followee. If the operation is invalid, it should be a no-op.
        """
        following = self.user_following[followerId]
        if followeeId in following:
            following.remove(followeeId)


# Your Twitter object will be instantiated and called as such:
# obj = Twitter()
# obj.postTweet(userId,tweetId)
# param_2 = obj.getNewsFeed(userId)
# obj.follow(followerId,followeeId)
# obj.unfollow(followerId,followeeId)
```

Complexity. Time: postTweet, follow, unfollow O(1); getNewsFeed O(F), F = people followed (it looks at 10 tweets of each). Space: O(T + U), T = tweets, U = follow pairs.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Delete a tweet
# 2. News feed with pages (page 0 is the 10 newest, page 1
# the next 10, ...)


# ============================================================
# FOLLOW-UP 1: Delete a tweet
# ============================================================

# ---- follow-up 1: function ----
# Answer: Every user keeps a list of (time, tweet id);
# deleting removes the pair, and the news feed simply no
# longer sees it.
# Time: post O(1), delete O(t), feed O(f * 10 log 10), f =
# followees.  Space: O(tweets + follows).
import heapq


class TwitterWithDelete:
    def __init__(self):
        self.time = 0
        self.tweets = {}
        self.following = {}

    def post_tweet(self, user_id, tweet_id):
        self.time += 1
        self.tweets.setdefault(user_id, []).append((self.time, tweet_id))

    def delete_tweet(self, user_id, tweet_id):
        mine = self.tweets.get(user_id, [])
        for i in range(len(mine)):
            if mine[i][1] == tweet_id:
                mine.pop(i)
                return True
        return False

    def get_news_feed(self, user_id):
        heap = []
        for person in self.following.get(user_id, set()) | {user_id}:
            for stamp, tweet_id in self.tweets.get(person, [])[-10:]:
                heapq.heappush(heap, (stamp, tweet_id))
                if len(heap) > 10:
                    heapq.heappop(heap)
        return [tweet_id for stamp, tweet_id in sorted(heap, reverse=True)]

    def follow(self, follower, followee):
        self.following.setdefault(follower, set()).add(followee)

    def unfollow(self, follower, followee):
        self.following.get(follower, set()).discard(followee)


# ============================================================
# FOLLOW-UP 2: News feed with pages (page 0 is the 10
# newest, page 1 the next 10, ...)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Collect the tweets of the user and the
```

```
# followees, sort by time, and slice out the page.
# Time: O(T log T), T = tweets of everyone followed.
# Space: O(T).
def news_feed_page(tweets, following, user_id, page, page_size=10):
```

```
    # tweets: user -> list of (time, tweet_id); following: user -> set of users
    everyone = set(following.get(user_id, set()))
    everyone.add(user_id)
    feed = []
    for person in everyone:
        feed += tweets.get(person, [])
    feed.sort(reverse=True)
    start = page * page_size
    return [tweet_id for stamp, tweet_id in feed[start:start + page_size]]
```

1396. Design Underground System

Medium · Design, Hash Table, String

leetcode.com/problems/design-underground-system

PROBLEM

An underground railway system is keeping track of customer travel times between different stations. They are using this data to calculate the average time it takes to travel from one station to another.

Implement the `UndergroundSystem` class:

- `void checkIn(int id, string stationName, int t)`
  - A customer with a card ID equal to `id`, checks in at the station `stationName` at time `t`.
  - A customer can only be checked into one place at a time.
- `void checkOut(int id, string stationName, int t)`
  - A customer with a card ID equal to `id`, checks out from the station `stationName` at time `t`.
- `double getAverageTime(string startStation, string endStation)`
  - Returns the average time it takes to travel from `startStation` to `endStation`.
  - The average time is computed from all the previous traveling times from `startStation` to `endStation` that happened **directly**, meaning a check in at `startStation` followed by a check out from `endStation`.
  - The time it takes to travel from `startStation` to `endStation` **may be different** from the time it takes to travel from `endStation` to `startStation`.
  - There will be at least one customer that has traveled from `startStation` to `endStation` before `getAverageTime` is called.

You may assume all calls to the `checkIn` and `checkOut` methods are consistent. If a customer checks in at time t_1 then checks out at time t_2, then $t_1 < t_2$. All events happen in chronological order.

Example 1:

Input
```
["UndergroundSystem","checkIn","checkIn","checkIn","checkOut","checkOut","checkOut","getAverageTime","
getAverageTime","checkIn","getAverageTime","checkOut","getAverageTime"]
[[],[45,"Leyton",3],[32,"Paradise",8],[27,"Leyton",10],[45,"Waterloo",15],[27,"Waterloo",20],[32,"Camb
ridge",22],["Paradise","Cambridge"],["Leyton","Waterloo"],[10,"Leyton",24],["Leyton","Waterloo"],[10,"
Waterloo",38],["Leyton","Waterloo"]]
```

Output
```
[null,null,null,null,null,null,null,14.00000,11.00000,null,11.00000,null,12.00000]
```

Explanation
```
UndergroundSystem undergroundSystem = new UndergroundSystem();
undergroundSystem.checkIn(45, "Leyton", 3);
undergroundSystem.checkIn(32, "Paradise", 8);
undergroundSystem.checkIn(27, "Leyton", 10);
undergroundSystem.checkOut(45, "Waterloo", 15); // Customer 45 "Leyton" -> "Waterloo" in 15-3 = 12
undergroundSystem.checkOut(27, "Waterloo", 20); // Customer 27 "Leyton" -> "Waterloo" in 20-10 = 10
undergroundSystem.checkOut(32, "Cambridge", 22); // Customer 32 "Paradise" -> "Cambridge" in 22-8 = 14
undergroundSystem.getAverageTime("Paradise", "Cambridge"); // return 14.00000. One trip "Paradise" ->
"Cambridge", (14) / 1 = 14
undergroundSystem.getAverageTime("Leyton", "Waterloo"); // return 11.00000. Two trips "Leyton" ->
"Waterloo", (10 + 12) / 2 = 11
undergroundSystem.checkIn(10, "Leyton", 24);
undergroundSystem.getAverageTime("Leyton", "Waterloo"); // return 11.00000
undergroundSystem.checkOut(10, "Waterloo", 38); // Customer 10 "Leyton" -> "Waterloo" in 38-24 = 14
undergroundSystem.getAverageTime("Leyton", "Waterloo"); // return 12.00000. Three trips "Leyton" ->
"Waterloo", (10 + 12 + 14) / 3 = 12
```

Example 2:

Input
```
["UndergroundSystem","checkIn","checkOut","getAverageTime","checkIn","checkOut","getAverageTime","chec
kIn","checkOut","getAverageTime"]
[[],[10,"Leyton",3],[10,"Paradise",8],["Leyton","Paradise"],[5,"Leyton",10],[5,"Paradise",16],["Leyton
","Paradise"],[2,"Leyton",21],[2,"Paradise",30],["Leyton","Paradise"]]
```

Output
```
[null,null,null,5.00000,null,null,5.50000,null,null,6.66667]
```

Explanation
```
UndergroundSystem undergroundSystem = new UndergroundSystem();
undergroundSystem.checkIn(10, "Leyton", 3);
undergroundSystem.checkOut(10, "Paradise", 8); // Customer 10 "Leyton" -> "Paradise" in 8-3 = 5
undergroundSystem.getAverageTime("Leyton", "Paradise"); // return 5.00000, (5) / 1 = 5
undergroundSystem.checkIn(5, "Leyton", 10);
undergroundSystem.checkOut(5, "Paradise", 16); // Customer 5 "Leyton" -> "Paradise" in 16-10 = 6
undergroundSystem.getAverageTime("Leyton", "Paradise"); // return 5.50000, (5 + 6) / 2 = 5.5
undergroundSystem.checkIn(2, "Leyton", 21);
undergroundSystem.checkOut(2, "Paradise", 30); // Customer 2 "Leyton" -> "Paradise" in 30-21 = 9
undergroundSystem.getAverageTime("Leyton", "Paradise"); // return 6.66667, (5 + 6 + 9) / 3 = 6.66667
```

Constraints:

- `1 <= id, t <= 10^6`
- `1 <= stationName.length, startStation.length, endStation.length <= 10`
- All strings consist of uppercase and lowercase English letters and digits.
- There will be at most `2 * 10^4` calls **in total** to `checkIn`, `checkOut`, and `getAverageTime`.
- Answers within `10^-5` of the actual value will be accepted.

SOLUTION (PYTHON 3)

Solution 1: Hash Table

> **How to think about it**
>
> Check-in, check-out, and the average time between two stations must be O(1). Map each id to its check-in time and station; on check-out, add the duration into a (start,end)↦(total,count) table. The average is that ratio.

We use two hash tables to store data:

- `ts`: Stores the passenger's id, check-in time, and check-in station. The key is the passenger's id, and the value is a tuple `(t, stationName)`.
- `d`: Stores the passenger's check-in station, check-out station, travel time, and number of trips. The key is a tuple `(startStation, endStation)`, and the value is a tuple `(totalTime, count)`.

When a passenger checks in, we store the passenger's id, check-in time, and check-in station in `ts`, i.e., `ts[id] = (t, stationName)`.

When a passenger checks out, we retrieve the passenger's check-in time and station `(t0, station)` from `ts`, then calculate the passenger's travel time t - t_0, and store the passenger's travel time and number of trips in `d`.

When we want to calculate a passenger's average travel time, we retrieve the passenger's total travel time and number of trips `(totalTime, count)` from `d`, then calculate the average travel time as totalTime / count.

The time complexity is O(1), and the space complexity is O(n). Where n is the number of passengers.

Python 3

```python
class UndergroundSystem:
    def __init__(self):
        self.ts = {}
        self.d = {}

    def checkIn(self, id: int, stationName: str, t: int) -> None:
        self.ts[id] = (t, stationName)

    def checkOut(self, id: int, stationName: str, t: int) -> None:
        t0, station = self.ts[id]
        x = self.d.get((station, stationName), (0, 0))
        self.d[(station, stationName)] = (x[0] + t - t0, x[1] + 1)

    def getAverageTime(self, startStation: str, endStation: str) -> float:
        x = self.d[(startStation, endStation)]
        return x[0] / x[1]


# Your UndergroundSystem object will be instantiated and called as such:
# obj = UndergroundSystem()
# obj.checkIn(id,stationName,t)
# obj.checkOut(id,stationName,t)
# param_3 = obj.getAverageTime(startStation,endStation)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Average time from ANY station to a given station
# 2. The pair of stations with the longest average trip


# ============================================================
# FOLLOW-UP 1: Average time from ANY station to a given
# station
# ============================================================

# ---- follow-up 1: function ----
```

```
# Answer: Keep one total and one count per end station as
# well.
# Time: check_in, check_out and average_to are each O(1).
# Space: O(riders + pairs).
class UndergroundByEnd:
    def __init__(self):
        self.riding = {}
        self.pair_totals = {}
        self.end_totals = {}

    def check_in(self, customer, station, time):
        self.riding[customer] = (station, time)

    def check_out(self, customer, station, time):
        start, began = self.riding.pop(customer)
        trip = time - began
        total, count = self.pair_totals.get((start, station), (0, 0))
        self.pair_totals[(start, station)] = (total + trip, count + 1)
        total, count = self.end_totals.get(station, (0, 0))
        self.end_totals[station] = (total + trip, count + 1)

    def average_to(self, station):
        total, count = self.end_totals[station]
        return total / count


# ============================================================
# FOLLOW-UP 2: The pair of stations with the longest
# average trip
# ============================================================

# ---- follow-up 2: function ----
# Answer: Scan the stored totals and keep the pair with
# the biggest average.
# Time: O(pairs).  Space: O(1).
def slowest_pair(pair_totals):
    # pair_totals: (start, end) -> (total_time, trip_count)
    best_pair = None
    best_average = -1
    for pair, (total, count) in pair_totals.items():
        average = total / count
        if average > best_average:
            best_average = average
            best_pair = pair
    return best_pair, best_average
```

1472. Design Browser History

Medium · Stack, Design, Array, Linked List, Data Stream, Doubly-Linked List

leetcode.com/problems/design-browser-history

PROBLEM

You have a **browser** of one tab where you start on the `homepage` and you can visit another `url`, get back in the history number of `steps` or move forward in the history number of `steps`.

Implement the `BrowserHistory` class:

- `BrowserHistory(string homepage)` Initializes the object with the `homepage` of the browser.
- `void visit(string url)` Visits `url` from the current page. It clears up all the forward history.
- `string back(int steps)` Move `steps` back in history. If you can only return `x` steps in the history and `steps > x`, you will return only `x` steps. Return the current `url` after moving back in history **at most** `steps`.
- `string forward(int steps)` Move `steps` forward in history. If you can only forward `x` steps in the history and `steps > x`, you will forward only `x` steps. Return the current `url` after forwarding in history **at most** `steps`.

Example:

```
Input:
["BrowserHistory","visit","visit","visit","back","back","forward","visit","forward","back","back"]
[["leetcode.com"],["google.com"],["facebook.com"],["youtube.com"],[1],[1],[1],["linkedin.com"],[2],[2]
,[7]]
Output:
[null,null,null,null,"facebook.com","google.com","facebook.com",null,"linkedin.com","google.com","leet
code.com"]

Explanation:
BrowserHistory browserHistory = new BrowserHistory("leetcode.com");
browserHistory.visit("google.com"); // You are in "leetcode.com". Visit "google.com"
browserHistory.visit("facebook.com"); // You are in "google.com". Visit "facebook.com"
browserHistory.visit("youtube.com"); // You are in "facebook.com". Visit "youtube.com"
browserHistory.back(1); // You are in "youtube.com", move back to "facebook.com" return "facebook.com"
browserHistory.back(1); // You are in "facebook.com", move back to "google.com" return "google.com"
browserHistory.forward(1); // You are in "google.com", move forward to "facebook.com" return
"facebook.com"
browserHistory.visit("linkedin.com"); // You are in "facebook.com". Visit "linkedin.com"
browserHistory.forward(2); // You are in "linkedin.com", you cannot move forward any steps.
browserHistory.back(2); // You are in "linkedin.com", move back two steps to "facebook.com" then to
"google.com". return "google.com"
browserHistory.back(7); // You are in "google.com", you can move back only one step to "leetcode.com".
return "leetcode.com"
```

Constraints:

- `1 <= homepage.length <= 20`
- `1 <= url.length <= 20`
- `1 <= steps <= 100`
- `homepage` and `url` consist of '.' or lower case English letters.
- At most `5000` calls will be made to `visit`, `back`, and `forward`.

SOLUTION (PYTHON 3)

Solution 1: Two Stacks

> **How to think about it**
>
> `visit` drops forward history; `back`/`forward` walk a timeline. stk1 holds the path to the current page, stk2 the forward pages. A visit pushes onto stk1 and clears stk2; back pops onto stk2, forward pops back.

We can use two stacks, stk1 and stk2, to store the back and forward pages, respectively. Initially, stk1 contains the homepage, and stk2 is empty.

When calling visit(url), we add url to stk1 and clear stk2. The time complexity is O(1).

When calling back(steps), we pop the top element from stk1 and push it to stk2. We repeat this operation steps times until the length of stk1 is 1 or steps is 0. Finally, we return the top element of stk1. The time complexity is O(steps).

When calling forward(steps), we pop the top element from stk2 and push it to stk1. We repeat this operation steps times until stk2 is empty or steps is 0. Finally, we return the top element of stk1. The time complexity is O(steps).

The space complexity is O(n), where n is the length of the browsing history.

Python 3

```python
class BrowserHistory:
    def __init__(self, homepage: str):
        self.stk1 = []
        self.stk2 = []
        self.visit(homepage)

    def visit(self, url: str) -> None:
        self.stk1.append(url)
        self.stk2.clear()

    def back(self, steps: int) -> str:
        while steps and len(self.stk1) > 1:
            self.stk2.append(self.stk1.pop())
            steps -= 1
        return self.stk1[-1]

    def forward(self, steps: int) -> str:
        while steps and self.stk2:
            self.stk1.append(self.stk2.pop())
            steps -= 1
        return self.stk1[-1]


# Your BrowserHistory object will be instantiated and called as such:
# obj = BrowserHistory(homepage)
# obj.visit(url)
# param_2 = obj.back(steps)
# param_3 = obj.forward(steps)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Limit the history to the most recent N pages
# 2. Make visit O(1) without slicing a list (linked list
# of pages)


# ========================================================
# FOLLOW-UP 1: Limit the history to the most recent N
# pages
# ========================================================
```

```python
# ---- follow-up 1: function ----
# Answer: Keep a list and a pointer; when a visit makes
# the list longer than N, drop the oldest page and move
# the pointer down by one.
# Time: visit O(n) (slicing), back and forward O(1), the
# pop(0) costs O(n).  Space: O(limit).
class LimitedBrowserHistory:
    def __init__(self, homepage, limit):
        self.limit = limit
        self.pages = [homepage]
        self.current = 0

    def visit(self, url):
        self.pages = self.pages[:self.current + 1]
        self.pages.append(url)
        self.current += 1
        if len(self.pages) > self.limit:
            self.pages.pop(0)
            self.current -= 1

    def back(self, steps):
        self.current = max(0, self.current - steps)
        return self.pages[self.current]

    def forward(self, steps):
        self.current = min(len(self.pages) - 1, self.current + steps)
        return self.pages[self.current]


# ============================================================
# FOLLOW-UP 2: Make visit O(1) without slicing a list
# (linked list of pages)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Pages are nodes with prev and next; visit just
# replaces next of the current node; back and forward walk
# the links.
# Time: visit O(1), back and forward O(steps).  Space:
# O(n).
class PageNode:
    def __init__(self, url):
        self.url = url
        self.prev = None
        self.next = None


class LinkedBrowserHistory:
    def __init__(self, homepage):
        self.current = PageNode(homepage)

    def visit(self, url):
        node = PageNode(url)
        node.prev = self.current
        self.current.next = node
        self.current = node

    def back(self, steps):
        while steps > 0 and self.current.prev is not None:
            self.current = self.current.prev
            steps -= 1
        return self.current.url

    def forward(self, steps):
        while steps > 0 and self.current.next is not None:
            self.current = self.current.next
            steps -= 1
        return self.current.url
```

1146. Snapshot Array

Medium · Design, Array, Hash Table, Binary Search, Persistent Data Structure

leetcode.com/problems/snapshot-array

PROBLEM

Implement a SnapshotArray that supports the following interface:

- `SnapshotArray(int length)` initializes an array-like data structure with the given length. **Initially, each element equals 0**.
- `void set(index, val)` sets the element at the given `index` to be equal to `val`.
- `int snap()` takes a snapshot of the array and returns the `snap_id`: the total number of times we called `snap()` minus `1`.
- `int get(index, snap_id)` returns the value at the given `index`, at the time we took the snapshot with the given `snap_id`

Example 1:

```
Input: ["SnapshotArray","set","snap","set","get"]
[[3],[0,5],[],[0,6],[0,0]]
Output: [null,null,0,null,5]
Explanation:
SnapshotArray snapshotArr = new SnapshotArray(3); // set the length to be 3
snapshotArr.set(0,5); // Set array[0] = 5
snapshotArr.snap(); // Take a snapshot, return snap_id = 0
snapshotArr.set(0,6);
snapshotArr.get(0,0); // Get the value of array[0] with snap_id = 0, return 5
```

Constraints:

- $1 <= length <= 5 * 10^4$
- $0 <= index < length$
- $0 <= val <= 10^9$
- $0 <= snap_id <$ (the total number of times we call `snap()`)
- At most $5 * 10^4$ calls will be made to `set`, `snap`, and `get`.

SOLUTION (PYTHON 3)

Solution 1: Array + Binary Search

> **How to think about it**
>
> Copying the whole array on every `snap` costs space proportional to length times snapshot count. Each index stores a history of (snap id,value); `set` appends the current id. `get` binary-searches the last change with id ≤ snap_id, or 0 if none.

We maintain an array of length `length`. Each element in the array is a list, which is used to store the value set each time and the corresponding snapshot ID.

When the `set` method is called, we add the value and snapshot ID to the list at the corresponding index. The time complexity is O(1).

When the `snap` method is called, we first increment the snapshot ID, then return the snapshot ID minus one. The time complexity is O(1).

When the `get` method is called, we use binary search to find the first snapshot ID greater than `snap_id` at the corresponding position, and then return the previous value. If it cannot be found, return 0. The time complexity is O(log n).

The space complexity is O(n).

Python 3

```python
class SnapshotArray:

    def __init__(self, length: int):
        self.arr = [[] for _ in range(length)]
        self.i = 0

    def set(self, index: int, val: int) -> None:
        self.arr[index].append((self.i, val))

    def snap(self) -> int:
        self.i += 1
        return self.i - 1

    def get(self, index: int, snap_id: int) -> int:
        i = bisect_left(self.arr[index], (snap_id, inf)) - 1
        return 0 if i < 0 else self.arr[index][i][1]


# Your SnapshotArray object will be instantiated and called as such:
# obj = SnapshotArray(length)
# obj.set(index,val)
# param_2 = obj.snap()
# param_3 = obj.get(index,snap_id)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Roll the array back to an earlier snapshot
# 2. Which indices changed between two snapshots


# ============================================================
# FOLLOW-UP 1: Roll the array back to an earlier snapshot
# ============================================================

# ---- follow-up 1: function ----
# Answer: Rollback to snapshot s: erase every change made
# after snapshot s, and continue numbering from s + 1.
# Time: set O(1), snap O(1), get O(log s), rollback
# O(total changes).  Space: O(changes).
import bisect


class SnapshotArrayWithRollback:
    def __init__(self, length):
        self.history = [[(0, 0)] for _ in range(length)]
        self.snap_id = 0

    def set(self, index, value):
        changes = self.history[index]
        if changes[-1][0] == self.snap_id:
            changes[-1] = (self.snap_id, value)
        else:
            changes.append((self.snap_id, value))

    def snap(self):
        self.snap_id += 1
        return self.snap_id - 1

    def get(self, index, snap_id):
        changes = self.history[index]
        position = bisect.bisect_right(changes, (snap_id, float("inf"))) - 1
        return changes[position][1]

    def rollback(self, snap_id):
```

```
        for changes in self.history:
            while len(changes) > 1 and changes[-1][0] > snap_id:
                changes.pop()
        self.snap_id = snap_id + 1


# ========================================================
# FOLLOW-UP 2: Which indices changed between two snapshots
# ========================================================

# ---- follow-up 2: function ----
# Answer: An index changed if its value at the later
# snapshot differs from its value at the earlier one.
# Time: O(length * log s).  Space: O(length).
def changed_indices(history, first_snap, second_snap):
    # history[i] is the sorted list of (snap_id, value) for index i
    def value_at(changes, snap_id):
        position = bisect.bisect_right(changes, (snap_id, float("inf"))) - 1
        return changes[position][1]

    result = []
```

```
    for index in range(len(history)):
        if value_at(history[index], first_snap) != value_at(history[index], second_snap):
            result.append(index)
    return result
```

PATTERN 29 · DESIGN PROBLEMS · PROBLEM 14 OF 20

211. Design Add and Search Words Data Structure

Medium · Depth-First Search, Design, Trie, String

leetcode.com/problems/design-add-and-search-words-data-structure

★ **TOP 7 PROBLEM** in Trie (Prefix Tree): master this one first

Also appears in pattern 32 Trie (Prefix Tree)

PROBLEM

Design a data structure that supports adding new words and finding if a string matches any previously added string.

Implement the `WordDictionary` class:

- `WordDictionary()` Initializes the object.
- `void addWord(word)` Adds `word` to the data structure, it can be matched later.
- `bool search(word)` Returns `true` if there is any string in the data structure that matches `word` or `false` otherwise. `word` may contain dots `'.'` where dots can be matched with any letter.

Example:

```
Input
["WordDictionary","addWord","addWord","addWord","search","search","search","search"]
[[],["bad"],["dad"],["mad"],["pad"],["bad"],[".ad"],["b.."]]
Output
[null,null,null,null,false,true,true,true]

Explanation
WordDictionary wordDictionary = new WordDictionary();
wordDictionary.addWord("bad");
wordDictionary.addWord("dad");
wordDictionary.addWord("mad");
wordDictionary.search("pad"); // return False
wordDictionary.search("bad"); // return True
wordDictionary.search(".ad"); // return True
wordDictionary.search("b.."); // return True
```

Constraints:

- `1 <= word.length <= 25`
- `word` in `addWord` consists of lowercase English letters.
- `word` in `search` consist of `'.'` or lowercase English letters.
- There will be at most `2` dots in `word` for `search` queries.
- At most 10^4 calls will be made to `addWord` and `search`.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Exact lookup could use a hash set, but . matches any letter, so scanning every word is awkward. Words use lowercase letters, which fit a trie. Add walks a 26-way path. Search follows a fixed edge on a letter and branches over every non-empty child on .; a match must end on a word node.

Python 3

```python
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False


class WordDictionary:
    def __init__(self):
        self.trie = Trie()

    def addWord(self, word: str) -> None:
        node = self.trie
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, word: str) -> bool:
        def search(word, node):
            for i in range(len(word)):
                c = word[i]
                idx = ord(c) - ord('a')
                if c != '.' and node.children[idx] is None:
                    return False
                if c == '.':
                    for child in node.children:
                        if child is not None and search(word[i + 1 :], child):
                            return True
                    return False
                node = node.children[idx]
            return node.is_end

        return search(word, self.trie)


# Your WordDictionary object will be instantiated and called as such:
# obj = WordDictionary()
# obj.addWord(word)
# param_2 = obj.search(word)
```

Complexity. Time: addWord O(L); search O(L) without dots, up to O(26^L) with dots. Space: O(total letters stored).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count how many stored words match a pattern with dots
# 2. List every stored word that matches a pattern with
# dots


# ============================================================
# FOLLOW-UP 1: Count how many stored words match a pattern
# with dots
# ============================================================

# ---- follow-up 1: function ----
# Answer: Search the trie and add up the counts of all
# word-end nodes reached.
# Time: add O(L), count_matches O(26^dots * L) in the
# worst case.  Space: O(total letters).
class WordDictionaryWithCount:
    def __init__(self):
        self.children = {}
        self.copies = 0
```

```
    def add_word(self, word):
        node = self
        for ch in word:
            if ch not in node.children:
                node.children[ch] = WordDictionaryWithCount()
            node = node.children[ch]
        node.copies += 1

    def count_matches(self, pattern):
        def go(node, i):
            if i == len(pattern):
                return node.copies
            if pattern[i] == ".":
                total = 0
                for child in node.children.values():
                    total += go(child, i + 1)
                return total
            if pattern[i] not in node.children:
                return 0
            return go(node.children[pattern[i]], i + 1)

        return go(self, 0)


# ============================================================
# FOLLOW-UP 2: List every stored word that matches a
# pattern with dots
# ============================================================

# ---- follow-up 2: function ----
# Answer: Search the trie carrying the letters walked so
# far, and collect the word when the pattern ends on a
# word node.
# Time: add O(L), matches O(26^dots * L) plus the output
# size.  Space: O(total letters).
class WordDictionaryList:
    def __init__(self):
        self.children = {}
        self.is_word = False
```

```
    def add_word(self, word):
        node = self
        for ch in word:
            if ch not in node.children:
                node.children[ch] = WordDictionaryList()
            node = node.children[ch]
        node.is_word = True

    def matches(self, pattern):
        found = []

        def go(node, i, so_far):
            if i == len(pattern):
                if node.is_word:
                    found.append(so_far)
                return
            if pattern[i] == ".":
                for ch in sorted(node.children):
                    go(node.children[ch], i + 1, so_far + ch)
            elif pattern[i] in node.children:
                go(node.children[pattern[i]], i + 1, so_far + pattern[i])

        go(self, 0, "")
        return found
```

2034. Stock Price Fluctuation

Medium · Design, Hash Table, Data Stream, Ordered Set, Heap (Priority Queue)

leetcode.com/problems/stock-price-fluctuation

PROBLEM

You are given a stream of **records** about a particular stock. Each record contains a **timestamp** and the corresponding **price** of the stock at that timestamp.

Unfortunately due to the volatile nature of the stock market, the records do not come in order. Even worse, some records may be incorrect. Another record with the same timestamp may appear later in the stream **correcting** the price of the previous wrong record.

Design an algorithm that:

- **Updates** the price of the stock at a particular timestamp, **correcting** the price from any previous records at the timestamp.
- Finds the **latest price** of the stock based on the current records. The **latest price** is the price at the latest timestamp recorded.
- Finds the **maximum price** the stock has been based on the current records.
- Finds the **minimum price** the stock has been based on the current records.

Implement the `StockPrice` class:

- `StockPrice()` Initializes the object with no price records.
- `void update(int timestamp, int price)` Updates the `price` of the stock at the given `timestamp`.
- `int current()` Returns the **latest price** of the stock.
- `int maximum()` Returns the **maximum price** of the stock.
- `int minimum()` Returns the **minimum price** of the stock.

Example 1:

```
Input
["StockPrice", "update", "update", "current", "maximum", "update", "maximum", "update", "minimum"]
[[], [1, 10], [2, 5], [], [], [1, 3], [], [4, 2], []]
Output
[null, null, null, 5, 10, null, 5, null, 2]

Explanation
StockPrice stockPrice = new StockPrice();
stockPrice.update(1, 10); // Timestamps are [1] with corresponding prices [10].
stockPrice.update(2, 5); // Timestamps are [1,2] with corresponding prices [10,5].
stockPrice.current(); // return 5, the latest timestamp is 2 with the price being 5.
stockPrice.maximum(); // return 10, the maximum price is 10 at timestamp 1.
stockPrice.update(1, 3); // The previous timestamp 1 had the wrong price, so it is updated to 3.
                         // Timestamps are [1,2] with corresponding prices [3,5].
stockPrice.maximum(); // return 5, the maximum price is 5 after the correction.
stockPrice.update(4, 2); // Timestamps are [1,2,4] with corresponding prices [3,5,2].
stockPrice.minimum(); // return 2, the minimum price is 2 at timestamp 4.
```

Constraints:

- $1 <= timestamp, price <= 10^9$
- At most 10^5 calls will be made **in total** to `update`, `current`, `maximum`, and `minimum`.
- `current`, `maximum`, and `minimum` will be called **only after** `update` has been called **at least once**.

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Ordered Set

How to think about it

We must correct past timestamps and query current, max, and min prices in 10^5 operations. A hash map alone cannot get extrema; a sorted set alone cannot overwrite by time. Map d stores time→price; ordered multiset ls stores live prices. On update, remove the old price if the time exists, insert the new one, and track the latest time last. Current is d[last]; extrema are the ends of ls.

We define the following data structures or variables:

- `d`: a hash table that stores the timestamp and the corresponding price;
- `ls`: an ordered set that stores all prices;
- `last`: the timestamp of the last update.

Then, we can perform the following operations:

- `update(timestamp, price)`: update the price corresponding to the timestamp `timestamp` to `price`. If `timestamp` already exists, we need to first remove its corresponding price from the ordered set, and then update it to `price`. Otherwise, we directly update it to `price`. Then, we need to update `last` to `max(last, timestamp)`. The time complexity is O(log n).
- `current()`: return the price corresponding to `last`. The time complexity is O(1).
- `maximum()`: return the maximum value in the ordered set. The time complexity is O(log n).
- `minimum()`: return the minimum value in the ordered set. The time complexity is O(log n).

The space complexity is O(n), where n is the number of `update` operations.

Python 3

```
class StockPrice:
    def __init__(self):
        self.d = {}
        self.ls = SortedList()
        self.last = 0

    def update(self, timestamp: int, price: int) -> None:
        if timestamp in self.d:
            self.ls.remove(self.d[timestamp])
        self.d[timestamp] = price
        self.ls.add(price)
        self.last = max(self.last, timestamp)

    def current(self) -> int:
        return self.d[self.last]

    def maximum(self) -> int:
        return self.ls[-1]

    def minimum(self) -> int:
        return self.ls[0]


# Your StockPrice object will be instantiated and called as such:
# obj = StockPrice()
# obj.update(timestamp,price)
# param_2 = obj.current()
# param_3 = obj.maximum()
# param_4 = obj.minimum()
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The highest price inside a time range
# 2. The k-th highest current price


# ============================================================
# FOLLOW-UP 1: The highest price inside a time range
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the timestamps sorted; binary search the
# range and take the maximum of the prices inside.
# Time: update O(n) (insort), highest_between O(n).
# Space: O(n).
import bisect


class StockPriceRange:
    def __init__(self):
        self.times = []
        self.price_at = {}

    def update(self, timestamp, price):
        if timestamp not in self.price_at:
            bisect.insort(self.times, timestamp)
        self.price_at[timestamp] = price

    def highest_between(self, start, end):
        low = bisect.bisect_left(self.times, start)
        high = bisect.bisect_right(self.times, end)
        best = None
        for t in self.times[low:high]:
            if best is None or self.price_at[t] > best:
                best = self.price_at[t]
        return best


# ============================================================
# FOLLOW-UP 2: The k-th highest current price
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep one sorted list of the current prices; a
# correction removes the old price and inserts the new
# one.
# Time: update O(n) (insort and pop), kth_highest O(1).
# Space: O(n).
import bisect


class StockPriceKth:
    def __init__(self):
        self.price_at = {}
        self.sorted_prices = []

    def update(self, timestamp, price):
        if timestamp in self.price_at:
            old = self.price_at[timestamp]
            self.sorted_prices.pop(bisect.bisect_left(self.sorted_prices, old))
        self.price_at[timestamp] = price
```

```
        bisect.insort(self.sorted_prices, price)

    def kth_highest(self, k):
        if k > len(self.sorted_prices):
```

```
            return None
        return self.sorted_prices[-k]
```

2353. Design a Food Rating System

Medium · Design, Array, Hash Table, String, Ordered Set, Heap (Priority Queue)

leetcode.com/problems/design-a-food-rating-system

PROBLEM

Design a food rating system that can do the following:

- **Modify** the rating of a food item listed in the system.
- Return the highest-rated food item for a type of cuisine in the system.

Implement the `FoodRatings` class:

- `FoodRatings(String[] foods, String[] cuisines, int[] ratings)` Initializes the system. The food items are described by `foods`, `cuisines` and `ratings`, all of which have a length of `n`.
  - `foods[i]` is the name of the i^{th} food,
  - `cuisines[i]` is the type of cuisine of the i^{th} food, and
  - `ratings[i]` is the initial rating of the i^{th} food.
- `void changeRating(String food, int newRating)` Changes the rating of the food item with the name `food`.
- `String highestRated(String cuisine)` Returns the name of the food item that has the highest rating for the given type of `cuisine`. If there is a tie, return the item with the **lexicographically smaller** name.

Note that a string `x` is lexicographically smaller than string `y` if `x` comes before `y` in dictionary order, that is, either `x` is a prefix of `y`, or if `i` is the first position such that `x[i] != y[i]`, then `x[i]` comes before `y[i]` in alphabetic order.

Example 1:

```
Input
["FoodRatings", "highestRated", "highestRated", "changeRating", "highestRated", "changeRating",
"highestRated"]
[[["kimchi", "miso", "sushi", "moussaka", "ramen", "bulgogi"], ["korean", "japanese", "japanese",
"greek", "japanese", "korean"], [9, 12, 8, 15, 14, 7]], ["korean"], ["japanese"], ["sushi", 16],
["japanese"], ["ramen", 16], ["japanese"]]
Output
[null, "kimchi", "ramen", null, "sushi", null, "ramen"]

Explanation
FoodRatings foodRatings = new FoodRatings(["kimchi", "miso", "sushi", "moussaka", "ramen", "bulgogi"],
["korean", "japanese", "japanese", "greek", "japanese", "korean"], [9, 12, 8, 15, 14, 7]);
foodRatings.highestRated("korean"); // return "kimchi"
                                    // "kimchi" is the highest rated korean food with a rating of 9.
foodRatings.highestRated("japanese"); // return "ramen"
                                      // "ramen" is the highest rated japanese food with a rating of
14.
foodRatings.changeRating("sushi", 16); // "sushi" now has a rating of 16.
foodRatings.highestRated("japanese"); // return "sushi"
                                      // "sushi" is the highest rated japanese food with a rating of
16.
foodRatings.changeRating("ramen", 16); // "ramen" now has a rating of 16.
foodRatings.highestRated("japanese"); // return "ramen"
                                      // Both "sushi" and "ramen" have a rating of 16.
                                      // However, "ramen" is lexicographically smaller than "sushi".
```

Constraints:

- $1 <= n <= 2 * 10^4$
- `n == foods.length == cuisines.length == ratings.length`
- `1 <= foods[i].length, cuisines[i].length <= 10`

- `foods[i]`, `cuisines[i]` consist of lowercase English letters.
- `1 <= ratings[i] <= 10^8`
- All the strings in `foods` are **distinct**.
- `food` will be the name of a food item in the system across all calls to `changeRating`.
- `cuisine` will be a type of cuisine of **at least one** food item in the system across all calls to `highestRated`.
- At most `2 * 10^4` calls **in total** will be made to `changeRating` and `highestRated`.

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Ordered Set

How to think about it

We query the highest-rated food of a cuisine (breaking ties lexicographically) and update ratings. Up to 2×10^4 calls make a linear scan too slow. Each cuisine keeps a sorted set of (-rating, food); a map stores each food's rating and cuisine. An update removes the old pair and inserts the new one; a query reads the first food name.

We can use a hash table d to store the foods for each cuisine, where the key is the cuisine and the value is an ordered set. Each element in the ordered set is a tuple (rating, food), sorted by rating in descending order, and if the ratings are the same, sorted by food name in lexicographical order.

We can also use a hash table g to store the rating and cuisine for each food. That is, g[food] = (rating, cuisine).

In the constructor, we iterate through foods, cuisines, and ratings, storing the rating and cuisine for each food in d and g.

In the changeRating function, we first get the original rating oldRating and cuisine cuisine of the food food, then update the rating of g[food] to newRating, remove (oldRating, food) from d[cuisine], and add (newRating, food) to d[cuisine].

In the highestRated function, we directly return the food name of the first element in d[cuisine].

In terms of time complexity, the constructor has a time complexity of O(n log n), where n is the number of foods. The other operations have a time complexity of O(log n). The space complexity is O(n).

Python 3

```python
class FoodRatings:

    def __init__(self, foods: List[str], cuisines: List[str], ratings: List[int]):
        self.d = defaultdict(SortedList)
        self.g = {}
        for food, cuisine, rating in zip(foods, cuisines, ratings):
            self.d[cuisine].add((-rating, food))
            self.g[food] = (rating, cuisine)

    def changeRating(self, food: str, newRating: int) -> None:
        oldRating, cuisine = self.g[food]
        self.g[food] = (newRating, cuisine)
        self.d[cuisine].remove((-oldRating, food))
        self.d[cuisine].add((-newRating, food))

    def highestRated(self, cuisine: str) -> str:
        return self.d[cuisine][0][1]


# Your FoodRatings object will be instantiated and called as such:
# obj = FoodRatings(foods, cuisines, ratings)
# obj.changeRating(food,newRating)
# param_2 = obj.highestRated(cuisine)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the top k foods of a cuisine
# 2. Remove a food from the menu


# ============================================================
# FOLLOW-UP 1: Return the top k foods of a cuisine
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep, per cuisine, a sorted list of (negative
# rating, name); changing a rating removes the old entry
# and inserts the new one.
# Time: change_rating O(n) per cuisine (insort and pop),
# top_k O(k).  Space: O(n).
import bisect


class FoodRatingsTopK:
    def __init__(self, foods, cuisines, ratings):
        self.info = {}
        self.by_cuisine = {}
        for i in range(len(foods)):
            self.info[foods[i]] = (cuisines[i], ratings[i])
            bisect.insort(self.by_cuisine.setdefault(cuisines[i], []), (-ratings[i], foods[i]))

    def change_rating(self, food, new_rating):
        cuisine, old = self.info[food]
        entries = self.by_cuisine[cuisine]
        entries.pop(bisect.bisect_left(entries, (-old, food)))
        bisect.insort(entries, (-new_rating, food))
        self.info[food] = (cuisine, new_rating)

    def top_k(self, cuisine, k):
        return [name for negative, name in self.by_cuisine[cuisine][:k]]


# ============================================================
# FOLLOW-UP 2: Remove a food from the menu
# ============================================================

# ---- follow-up 2: function ----
# Answer: Remove it from the lookup and from its cuisine
# list so it can never be the highest rated.
# Time: O(n).  Space: O(1) extra.
def remove_food(info, by_cuisine, food):
    # info: food -> (cuisine, rating); by_cuisine: cuisine -> sorted list of (-rating, name)
    cuisine, rating = info.pop(food)
    entries = by_cuisine[cuisine]
    entries.remove((-rating, food))
    if not entries:
        del by_cuisine[cuisine]
```

1797. Design Authentication Manager

Medium · Design, Hash Table, Linked List, Doubly-Linked List

leetcode.com/problems/design-authentication-manager

PROBLEM

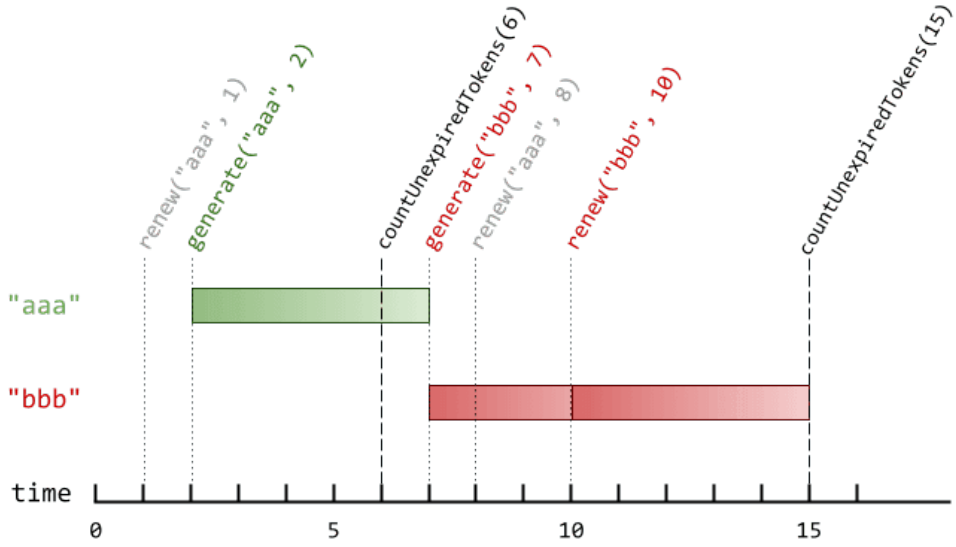
There is an authentication system that works with authentication tokens. For each session, the user will receive a new authentication token that will expire `timeToLive` seconds after the `currentTime`. If the token is renewed, the expiry time will be **extended** to expire `timeToLive` seconds after the (potentially different) `currentTime`.

Implement the `AuthenticationManager` class:

- `AuthenticationManager(int timeToLive)` constructs the `AuthenticationManager` and sets the `timeToLive`.
- `generate(string tokenId, int currentTime)` generates a new token with the given `tokenId` at the given `currentTime` in seconds.
- `renew(string tokenId, int currentTime)` renews the **unexpired** token with the given `tokenId` at the given `currentTime` in seconds. If there are no unexpired tokens with the given `tokenId`, the request is ignored, and nothing happens.
- `countUnexpiredTokens(int currentTime)` returns the number of **unexpired** tokens at the given currentTime.

Note that if a token expires at time `t`, and another action happens on time `t` (`renew` or `countUnexpiredTokens`), the expiration takes place **before** the other actions.

Example 1:



```
Input
["AuthenticationManager", "renew", "generate", "countUnexpiredTokens", "generate", "renew", "renew",
"countUnexpiredTokens"]
[[5], ["aaa", 1], ["aaa", 2], [6], ["bbb", 7], ["aaa", 8], ["bbb", 10], [15]]
Output
[null, null, null, 1, null, null, null, 0]

Explanation
AuthenticationManager authenticationManager = new AuthenticationManager(5); // Constructs the
AuthenticationManager with timeToLive = 5 seconds.
authenticationManager.renew("aaa", 1); // No token exists with tokenId "aaa" at time 1, so nothing
happens.
authenticationManager.generate("aaa", 2); // Generates a new token with tokenId "aaa" at time 2.
authenticationManager.countUnexpiredTokens(6); // The token with tokenId "aaa" is the only unexpired
one at time 6, so return 1.
authenticationManager.generate("bbb", 7); // Generates a new token with tokenId "bbb" at time 7.
authenticationManager.renew("aaa", 8); // The token with tokenId "aaa" expired at time 7, and 8 >= 7,
so at time 8 the renew request is ignored, and nothing happens.
authenticationManager.renew("bbb", 10); // The token with tokenId "bbb" is unexpired at time 10, so
the renew request is fulfilled and now the token will expire at time 15.
authenticationManager.countUnexpiredTokens(15); // The token with tokenId "bbb" expires at time 15,
and the token with tokenId "aaa" expired at time 7, so currently no token is unexpired, so return 0.
```

Constraints:

- `1 <= timeToLive <= 10^8`
- `1 <= currentTime <= 10^8`
- `1 <= tokenId.length <= 5`
- `tokenId` consists only of lowercase letters.
- All calls to `generate` will contain unique values of `tokenId`.
- The values of `currentTime` across all the function calls will be **strictly increasing**.
- At most `2000` calls will be made to all functions combined.

SOLUTION (PYTHON 3)

Solution 1: Hash Table

> **How to think about it**
>
> A token expires at currentTime+timeToLive after generate or renew. The operation count is modest, so a hash map of expiry times suffices. Generate writes an expiry; renew updates only if still valid; count scans how many expiries are still in the future.

We can simply maintain a hash table d, where the key is `tokenId` and the value is the expiration time.

- During the `generate` operation, we store `tokenId` as the key and `currentTime + timeToLive` as the value in the hash table d.
- During the `renew` operation, if `tokenId` is not in the hash table d, or `currentTime >= d[tokenId]`, we ignore this operation; otherwise, we update `d[tokenId]` to `currentTime + timeToLive`.
- During the `countUnexpiredTokens` operation, we traverse the hash table d and count the number of unexpired `tokenId`.

In terms of time complexity, both `generate` and `renew` operations have a time complexity of O(1), and the `countUnexpiredTokens` operation has a time complexity of O(n), where n is the number of key-value pairs in the hash table d.

The space complexity is O(n), where n is the number of key-value pairs in the hash table d.

Python 3

```python
class AuthenticationManager:
    def __init__(self, timeToLive: int):
        self.t = timeToLive
        self.d = defaultdict(int)

    def generate(self, tokenId: str, currentTime: int) -> None:
        self.d[tokenId] = currentTime + self.t

    def renew(self, tokenId: str, currentTime: int) -> None:
        if self.d[tokenId] <= currentTime:
            return
        self.d[tokenId] = currentTime + self.t

    def countUnexpiredTokens(self, currentTime: int) -> int:
        return sum(exp > currentTime for exp in self.d.values())


# Your AuthenticationManager object will be instantiated and called as such:
# obj = AuthenticationManager(timeToLive)
# obj.generate(tokenId,currentTime)
# obj.renew(tokenId,currentTime)
# param_3 = obj.countUnexpiredTokens(currentTime)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Revoke a token immediately
# 2. Count unexpired tokens quickly: delete expired ones
# lazily with a heap


# ============================================================
# FOLLOW-UP 1: Revoke a token immediately
# ============================================================

# ---- follow-up 1: function ----
# Answer: Delete the token from the dictionary; later
# calls to renew it find nothing and do nothing.
# Time: generate, renew and revoke O(1), count_unexpired
# O(n).  Space: O(n).
class AuthenticationManagerRevoke:
    def __init__(self, time_to_live):
        self.time_to_live = time_to_live
        self.expires_at = {}

    def generate(self, token_id, current_time):
        self.expires_at[token_id] = current_time + self.time_to_live

    def renew(self, token_id, current_time):
        if token_id in self.expires_at and self.expires_at[token_id] > current_time:
            self.expires_at[token_id] = current_time + self.time_to_live

    def revoke(self, token_id):
        if token_id in self.expires_at:
            del self.expires_at[token_id]

    def count_unexpired(self, current_time):
        count = 0
        for expiry in self.expires_at.values():
            if expiry > current_time:
                count += 1
        return count


# ============================================================
```

```
# FOLLOW-UP 2: Count unexpired tokens quickly: delete
# expired ones lazily with a heap
# ============================================================

# ---- follow-up 2: function ----
# Answer: Push (expiry, token) on every generate and
# renew; when counting, pop heap entries that are expired
# or out of date, and the dictionary size is the answer.
# Time: O(log n) amortised per call.  Space: O(n).
import heapq


class AuthenticationManagerHeap:
    def __init__(self, time_to_live):
        self.time_to_live = time_to_live
        self.expires_at = {}
        self.heap = []

    def clean(self, current_time):
        while self.heap and self.heap[0][0] <= current_time:
```

```
            expiry, token_id = heapq.heappop(self.heap)
            if self.expires_at.get(token_id) == expiry:
                del self.expires_at[token_id]

    def generate(self, token_id, current_time):
        self.expires_at[token_id] = current_time + self.time_to_live
        heapq.heappush(self.heap, (current_time + self.time_to_live, token_id))

    def renew(self, token_id, current_time):
        self.clean(current_time)
        if token_id in self.expires_at:
            self.generate(token_id, current_time)

    def count_unexpired(self, current_time):
        self.clean(current_time)
        return len(self.expires_at)
```

460. LFU Cache

Hard · Design, Hash Table, Linked List, Doubly-Linked List

leetcode.com/problems/lfu-cache

★ **TOP 7 PROBLEM** in Design Problems: master this one first

PROBLEM

Design and implement a data structure for a Least Frequently Used (LFU) cache.

Implement the `LFUCache` class:

- `LFUCache(int capacity)` Initializes the object with the `capacity` of the data structure.
- `int get(int key)` Gets the value of the `key` if the `key` exists in the cache. Otherwise, returns `-1`.
- `void put(int key, int value)` Update the value of the `key` if present, or inserts the `key` if not already present. When the cache reaches its `capacity`, it should invalidate and remove the **least frequently used** key before inserting a new item. For this problem, when there is a **tie** (i.e., two or more keys with the same frequency), the **least recently used** `key` would be invalidated.

To determine the least frequently used key, a **use counter** is maintained for each key in the cache. The key with the smallest **use counter** is the least frequently used key.

When a key is first inserted into the cache, its **use counter** is set to `1` (due to the `put` operation). The **use counter** for a key in the cache is incremented either a `get` or `put` operation is called on it.

The functions `get` and `put` must each run in `O(1)` average time complexity.

Example 1:

```
Input
["LFUCache", "put", "put", "get", "put", "get", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [3], [4, 4], [1], [3], [4]]
Output
[null, null, null, 1, null, -1, 3, null, -1, 3, 4]

Explanation
// cnt(x) = the use counter for key x
// cache=[] will show the last used order for tiebreakers (leftmost element is most recent)
LFUCache lfu = new LFUCache(2);
lfu.put(1, 1); // cache=[1,_], cnt(1)=1
lfu.put(2, 2); // cache=[2,1], cnt(2)=1, cnt(1)=1
lfu.get(1); // return 1
                // cache=[1,2], cnt(2)=1, cnt(1)=2
lfu.put(3, 3); // 2 is the LFU key because cnt(2)=1 is the smallest, invalidate 2.
                // cache=[3,1], cnt(3)=1, cnt(1)=2
lfu.get(2); // return -1 (not found)
lfu.get(3); // return 3
                // cache=[3,1], cnt(3)=2, cnt(1)=2
lfu.put(4, 4); // Both 1 and 3 have the same cnt, but 1 is LRU, invalidate 1.
                // cache=[4,3], cnt(4)=1, cnt(3)=2
lfu.get(1); // return -1 (not found)
lfu.get(3); // return 3
                // cache=[3,4], cnt(4)=1, cnt(3)=3
lfu.get(4); // return 4
                // cache=[4,3], cnt(4)=2, cnt(3)=3
```

Constraints:

- $1 <= capacity <= 10^4$
- $0 <= key <= 10^5$
- $0 <= value <= 10^9$

- At most $2 * 10^5$ calls will be made to `get` and `put`.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

LFU must get and put in O(1) and, when full, evict the least frequent key, breaking ties by LRU. One hash map finds a value but cannot track frequency and recency together. A key-to-node map gives lookup. A frequency-to-list map stores nodes of equal frequency, newest at the head. minFreq points at the victim list's tail. A hit moves the node onto the freq+1 list; if the old list is empty and was the minimum, bump minFreq. A brand-new key has frequency 1 and resets the minimum to 1.

Python 3

```
class Node:
    def __init__(self, key: int, value: int) -> None:
        self.key = key
        self.value = value
        self.freq = 1
        self.prev = None
        self.next = None


class DoublyLinkedList:
    def __init__(self) -> None:
        self.head = Node(-1, -1)
        self.tail = Node(-1, -1)
        self.head.next = self.tail
        self.tail.prev = self.head

    def add_first(self, node: Node) -> None:
        node.prev = self.head
        node.next = self.head.next
        self.head.next.prev = node
        self.head.next = node

    def remove(self, node: Node) -> Node:
        node.next.prev = node.prev
        node.prev.next = node.next
        node.next, node.prev = None, None
        return node

    def remove_last(self) -> Node:
        return self.remove(self.tail.prev)

    def is_empty(self) -> bool:
        return self.head.next == self.tail


class LFUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.min_freq = 0
        self.map = defaultdict(Node)
        self.freq_map = defaultdict(DoublyLinkedList)

    def get(self, key: int) -> int:
        if self.capacity == 0 or key not in self.map:
            return -1
        node = self.map[key]
        self.incr_freq(node)
        return node.value

    def put(self, key: int, value: int) -> None:
        if self.capacity == 0:
            return
        if key in self.map:
            node = self.map[key]
            node.value = value
            self.incr_freq(node)
            return
        if len(self.map) == self.capacity:
            ls = self.freq_map[self.min_freq]
            node = ls.remove_last()
            self.map.pop(node.key)
        node = Node(key, value)
        self.add_node(node)
        self.map[key] = node
        self.min_freq = 1
```

```
    def incr_freq(self, node: Node) -> None:
        freq = node.freq
        ls = self.freq_map[freq]
        ls.remove(node)
        if ls.is_empty():
            self.freq_map.pop(freq)
            if freq == self.min_freq:
                self.min_freq += 1
        node.freq += 1
        self.add_node(node)

    def add_node(self, node: Node) -> None:
        freq = node.freq
        ls = self.freq_map[freq]
        ls.add_first(node)
        self.freq_map[freq] = ls


# Your LFUCache object will be instantiated and called as such:
# obj = LFUCache(capacity)
# param_1 = obj.get(key)
# obj.put(key,value)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Tell the caller which key was evicted
# 2. Ask how many times a key has been used without
# counting as a use


# ============================================================
# FOLLOW-UP 1: Tell the caller which key was evicted
# ============================================================

# ---- follow-up 1: function ----
# Answer: Return the evicted key from put (or None) so it
# can be written to disk or logged.
# Time: get and put are each O(1).  Space: O(capacity).
from collections import OrderedDict


class LFUCacheEvictions:
    def __init__(self, capacity):
        self.capacity = capacity
        self.values = {}
        self.count = {}
        self.groups = {}
        self.lowest = 0

    def touch(self, key):
        c = self.count[key]
        del self.groups[c][key]
        if not self.groups[c]:
            del self.groups[c]
            if self.lowest == c:
                self.lowest += 1
        self.count[key] = c + 1
        self.groups.setdefault(c + 1, OrderedDict())[key] = None

    def get(self, key):
        if key not in self.values:
            return -1
        self.touch(key)
        return self.values[key]
```

```
    def put(self, key, value):
        if self.capacity == 0:
            return None
        if key in self.values:
            self.values[key] = value
            self.touch(key)
            return None
        evicted = None
        if len(self.values) == self.capacity:
            evicted, _ = self.groups[self.lowest].popitem(last=False)
            if not self.groups[self.lowest]:
                del self.groups[self.lowest]
            del self.values[evicted]
            del self.count[evicted]
        self.values[key] = value
        self.count[key] = 1
        self.groups.setdefault(1, OrderedDict())[key] = None
        self.lowest = 1
        return evicted
```

```
# ============================================================
# FOLLOW-UP 2: Ask how many times a key has been used
# without counting as a use
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the counter in the same dictionary and read
# it without calling touch.
# Time: O(1).  Space: O(1) extra.
def frequency_of(cache, key):
    # cache is the LFUCacheEvictions above; peeking must not change any count
    return cache.count.get(key, 0)
```

PATTERN 29 · DESIGN PROBLEMS · PROBLEM 19 OF 20

295. Find Median from Data Stream

Hard · Design, Two Pointers, Data Stream, Sorting, Heap (Priority Queue)

leetcode.com/problems/find-median-from-data-stream

★ **TOP 7 PROBLEM** in Design Problems, Two Heaps: master this one first

Also appears in pattern 37 Two Heaps

PROBLEM

The **median** is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

- For example, for `arr = [2,3,4]`, the median is `3`.
- For example, for `arr = [2,3]`, the median is `(2 + 3) / 2 = 2.5`.

Implement the MedianFinder class:

- `MedianFinder()` initializes the `MedianFinder` object.
- `void addNum(int num)` adds the integer `num` from the data stream to the data structure.
- `double findMedian()` returns the median of all elements so far. Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

```
Input
["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]
[[], [1], [2], [], [3], []]
Output
[null, null, null, 1.5, null, 2.0]

Explanation
MedianFinder medianFinder = new MedianFinder();
medianFinder.addNum(1); // arr = [1]
medianFinder.addNum(2); // arr = [1, 2]
medianFinder.findMedian(); // return 1.5 (i.e., (1 + 2) / 2)
medianFinder.addNum(3); // arr[1, 2, 3]
medianFinder.findMedian(); // return 2.0
```

Constraints:

- -10^5 `<= num <=` 10^5
- There will be at least one element in the data structure before calling `findMedian`.
- At most `5 *` 10^4 calls will be made to `addNum` and `findMedian`.

Follow up:

- If all integer numbers from the stream are in the range `[0, 100]`, how would you optimize your solution?
- If `99%` of all integer numbers from the stream are in the range `[0, 100]`, how would you optimize your solution?

SOLUTION (PYTHON 3)

Solution 1: Min Heap and Max Heap (Priority Queue)

How to think about it

Sorting after every insertion is too slow for a running median. A max-heap holds the lower half and a min-heap the upper half; the tops form the median. A new value goes into the max-heap then is moved to the min-heap, keeping the min-heap at most one larger. Even counts average the two tops; odd counts take the min-heap top.

We can use two heaps to maintain all the elements, a min heap minQ and a max heap maxQ, where the min heap minQ stores the larger half, and the max heap maxQ stores the smaller half.

When calling the `addNum` method, we first add the element to the max heap maxQ, then pop the top element of maxQ and add it to the min heap minQ. If at this time the size difference between minQ and maxQ is greater than 1, we pop the top element of minQ and add it to maxQ. The time complexity is O(log n).

When calling the `findMedian` method, if the size of minQ is equal to the size of maxQ, it means the total number of elements is even, and we can return the average value of the top elements of minQ and maxQ; otherwise, we return the top element of minQ. The time complexity is O(1).

The space complexity is O(n), where n is the number of elements.

Python 3

```
class MedianFinder:

    def __init__(self):
        self.minq = []
        self.maxq = []

    def addNum(self, num: int) -> None:
        heappush(self.minq, -heappushpop(self.maxq, -num))
        if len(self.minq) - len(self.maxq) > 1:
            heappush(self.maxq, -heappop(self.minq))

    def findMedian(self) -> float:
        if len(self.minq) == len(self.maxq):
            return (self.minq[0] - self.maxq[0]) / 2
        return self.minq[0]


# Your MedianFinder object will be instantiated and called as such:
# obj = MedianFinder()
# obj.addNum(num)
# param_2 = obj.findMedian()
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. All numbers are between 0 and 100
# 2. Support removing a number that was added


# ============================================================
# FOLLOW-UP 1: All numbers are between 0 and 100
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep a counter for every value and walk the 101
# counters to find the middle one or two.
# Time: add_num O(1), find_median O(101) = O(1).  Space:
# O(101) = O(1).
class MedianFinderSmallRange:
    def __init__(self):
        self.counts = [0] * 101
        self.total = 0

    def add_num(self, num):
        self.counts[num] += 1
        self.total += 1

    def kth(self, k):
        seen = 0
        for value in range(101):
            seen += self.counts[value]
            if seen >= k:
```

```
                return value

    def find_median(self):
        if self.total % 2 == 1:
            return float(self.kth(self.total // 2 + 1))
        return (self.kth(self.total // 2) + self.kth(self.total // 2 + 1)) / 2


# ============================================================
# FOLLOW-UP 2: Support removing a number that was added
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the numbers in a sorted list; add and
# remove with binary search and read the middle by index.
# Time: add_num and remove_num O(n), find_median O(1).
# Space: O(n).
import bisect


class MedianFinderWithRemove:
    def __init__(self):
        self.items = []

    def add_num(self, num):
        bisect.insort(self.items, num)

    def remove_num(self, num):
        position = bisect.bisect_left(self.items, num)
        if position < len(self.items) and self.items[position] == num:
            self.items.pop(position)
            return True
```

```
        return False

    def find_median(self):
        n = len(self.items)
        if n % 2 == 1:
            return float(self.items[n // 2])
        return (self.items[n // 2 - 1] + self.items[n // 2]) / 2
```

297. Serialize and Deserialize Binary Tree

Hard · Tree, Depth-First Search, Breadth-First Search, Design, String, Binary Tree

leetcode.com/problems/serialize-and-deserialize-binary-tree

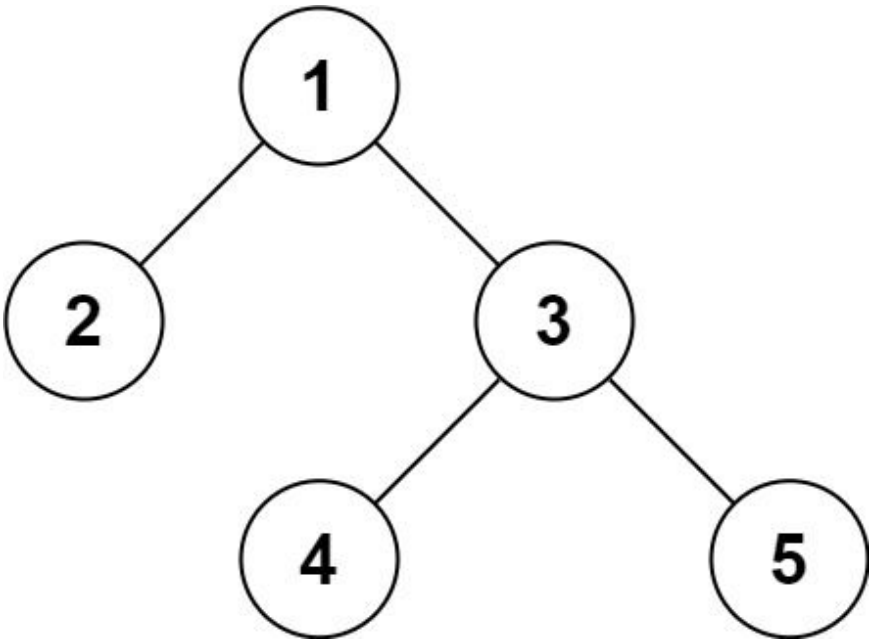
PROBLEM

Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

Clarification: The input/output format is the same as how LeetCode serializes a binary tree. You do not necessarily need to follow this format, so please be creative and come up with different approaches yourself.

Example 1:




```
Input: root = [1,2,3,null,null,4,5]
Output: [1,2,3,null,null,4,5]
```

Example 2:

```
Input: root = []
Output: []
```

Constraints:

- The number of nodes in the tree is in the range $[0, 10^4]$.
- $-1000 <= Node.val <= 1000$

SOLUTION (PYTHON 3)

Solution 1: Level Order Traversal

> **How to think about it**
>
> A general binary tree must record missing children; storing only present values loses shape. Level order writes # for nulls so parent–child slots can be rebuilt. Serialize by enqueueing both children; deserialize assigns the next two tokens as the front node's left and right.

We can use level order traversal to serialize the binary tree. Starting from the root node, we add the nodes of the binary tree to the queue in the order from top to bottom, from left to right. Then we dequeue the nodes in the queue one by one. If the node is not null, we add its value to the serialized string; otherwise, we add a special character `#`. Finally, we return the serialized string.

During deserialization, we split the serialized string by the delimiter to get a string array, and then add the elements in the string array to the queue in order. The elements in the queue are the nodes of the binary tree. We dequeue the elements from the queue one by one. If the element is not `#`, we convert it to an integer and use it as the value of the node, and then add the node to the queue; otherwise, we set it to `null`. Finally, we return the root node.

The time complexity is O(n), and the space complexity is O(n). Where n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None


class Codec:
    def serialize(self, root):
        """Encodes a tree to a single string.

        :type root: TreeNode
        :rtype: str
        """
        if root is None:
            return ""
        q = deque([root])
        ans = []
        while q:
            node = q.popleft()
            if node:
                ans.append(str(node.val))
                q.append(node.left)
                q.append(node.right)
            else:
                ans.append("#")
        return ",".join(ans)

    def deserialize(self, data):
        """Decodes your encoded data to tree.

        :type data: str
        :rtype: TreeNode
        """
        if not data:
            return None
        vals = data.split(",")
        root = TreeNode(int(vals[0]))
        q = deque([root])
        i = 1
        while q:
            node = q.popleft()
            if vals[i] != "#":
                node.left = TreeNode(int(vals[i]))
                q.append(node.left)
            i += 1
            if vals[i] != "#":
                node.right = TreeNode(int(vals[i]))
                q.append(node.right)
            i += 1
        return root


# Your Codec object will be instantiated and called as such:
# codec = Codec()
# codec.deserialize(codec.serialize(root))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. A binary SEARCH tree: make the text as short as
# possible
# 2. An N-ary tree


# ============================================================
# FOLLOW-UP 1: A binary SEARCH tree: make the text as
# short as possible
# ============================================================

# ---- follow-up 1: function ----
# Answer: A preorder list is enough because the order of
# the values already tells where each one goes (every
# value must stay inside the bounds of its ancestors).
# Time: O(n).  Space: O(h).
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def serialize_bst(root):
    values = []

    def go(node):
        if node is None:
            return
        values.append(str(node.val))
        go(node.left)
        go(node.right)

    go(root)
    return ",".join(values)


def deserialize_bst(data):
    if not data:
        return None
    values = [int(x) for x in data.split(",")]
    position = [0]

    def build(low, high):
        if position[0] == len(values):
            return None
        value = values[position[0]]
        if value < low or value > high:
            return None
        position[0] += 1
        node = TreeNode(value)
        node.left = build(low, value)
        node.right = build(value, high)
        return node

    return build(float("-inf"), float("inf"))


# ============================================================
# FOLLOW-UP 2: An N-ary tree
```

```
# ============================================================

# ---- follow-up 2: function ----
# Answer: Write each node as its value followed by how
```

```
# many children it has, in preorder; reading back, a node
# reads exactly that many children.
# Time: O(n).  Space: O(h).
class NaryNode:
    def __init__(self, val=0, children=None):
        self.val = val
        self.children = children if children is not None else []


def serialize_nary(root):
    parts = []

    def go(node):
        parts.append(str(node.val))
        parts.append(str(len(node.children)))
        for child in node.children:
            go(child)

    if root is not None:
        go(root)
    return " ".join(parts)


def deserialize_nary(data):
    if not data:
        return None
    tokens = data.split()
    position = [0]

    def build():
        value = int(tokens[position[0]])
        count = int(tokens[position[0] + 1])
        position[0] += 2
        node = NaryNode(value)
        for _ in range(count):
            node.children.append(build())
        return node

    return build()
```

PATTERN 30 OF 41

Topological Sort

20 problems · 0 Easy · 13 Medium · 7 Hard

THE IDEA

Order tasks so every prerequisite comes first. Kahn's algorithm repeatedly takes nodes with no remaining prerequisites; if some are never taken there is a cycle.

Signals: course schedule, alien dictionary, build order, parallel courses, sequence reconstruction, minimum height trees.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
from collections import deque
indeg = [0] * n
for a, b in edges:                    # b before a
    graph[b].append(a)
    indeg[a] += 1
q = deque(i for i in range(n) if indeg[i] == 0)
order = []
while q:
    u = q.popleft()
    order.append(u)
    for v in graph[u]:
        indeg[v] -= 1
        if indeg[v] == 0:
            q.append(v)
return len(order) == n
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	★ **Course Schedule** (see page)	Medium	LC 207	866
2	★ **Course Schedule II**	Medium	LC 210	1595
3	★ **Find Eventual Safe States** (see page)	Medium	LC 802	891
4	Course Schedule IV	Medium	LC 1462	1598
5	★ **Minimum Height Trees**	Medium	LC 310	1602
6	All Ancestors of a Node in a Directed Acyclic Graph	Medium	LC 2192	1605
7	★ **Parallel Courses**	Medium	LC 1136	1609
8	★ **Find All Possible Recipes from Given Supplies**	Medium	LC 2115	1613
9	Loud and Rich	Medium	LC 851	1616
10	Minimum Number of Vertices to Reach All Nodes	Medium	LC 1557	1619
11	Sequence Reconstruction	Medium	LC 444	1622
12	Number of Ways to Arrive at Destination	Medium	LC 1976	1625
13	Detect Cycles in 2D Grid	Medium	LC 1559	1629
14	Parallel Courses III	Hard	LC 2050	1633
15	Sort Items by Groups Respecting Dependencies	Hard	LC 1203	1637
16	Build a Matrix With Conditions	Hard	LC 2392	1641
17	★ **Largest Color Value in a Directed Graph**	Hard	LC 1857	1644
18	Maximum Employees to Be Invited to a Meeting	Hard	LC 2127	1648
19	Strange Printer II	Hard	LC 1591	1653
20	Sum of Scores of Built Strings — substitute: Eventual Safe States variant: Longest Cycle in a Graph	Hard	LC 2360	1657

PATTERN 30 · TOPOLOGICAL SORT · PROBLEM 2 OF 20

210. Course Schedule II

Medium · Depth-First Search, Breadth-First Search, Graph, Topological Sort

leetcode.com/problems/course-schedule-ii

★ **TOP 7 PROBLEM** in Topological Sort: master this one first

PROBLEM

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [a_i, b_i]` indicates that you **must** take course `b_i` first if you want to take course `a_i`.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return *the ordering of courses you should take to finish all courses*. If there are many valid answers, return **any** of them. If it is impossible to finish all courses, return **an empty array**.

Example 1:

```
Input: numCourses = 2, prerequisites = [[1,0]]
Output: [0,1]
Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course
0. So the correct course order is [0,1].
```

Example 2:

```
Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]
Output: [0,2,1,3]
Explanation: There are a total of 4 courses to take. To take course 3 you should have finished both
courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0.
So one correct course order is [0,1,2,3]. Another correct ordering is [0,2,1,3].
```

Example 3:

```
Input: numCourses = 1, prerequisites = []
Output: [0]
```

Constraints:

- `1 <= numCourses <= 2000`
- `0 <= prerequisites.length <= numCourses * (numCourses - 1)`
- `prerequisites[i].length == 2`
- `0 <= a_i, b_i < numCourses`
- `a_i != b_i`
- All the pairs `[a_i, b_i]` are **distinct**.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

As in the decision version, prerequisites form a directed graph; here we also need one valid order. The dequeue order of a topological sort is such a sequence. If fewer than numCourses vertices enter the queue, a cycle exists and we return an empty array.

Python 3

```python
class Solution:
    def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]:
        g = defaultdict(list)
        indeg = [0] * numCourses
        for a, b in prerequisites:
            g[b].append(a)
            indeg[a] += 1
        ans = []
        q = deque(i for i, x in enumerate(indeg) if x == 0)
        while q:
            i = q.popleft()
            ans.append(i)
            for j in g[i]:
                indeg[j] -= 1
                if indeg[j] == 0:
                    q.append(j)
        return ans if len(ans) == numCourses else []
```

Complexity. Time: O(V + E). Space: O(V + E).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return every valid order
# 2. Return the order that is smallest when read as a list
# (smallest course number first whenever there is a
# choice)


# ========================================================
# FOLLOW-UP 1: Return every valid order
# ========================================================

# ---- follow-up 1: function ----
# Answer: Backtracking over Kahn's algorithm: at each step
# try every course whose prerequisites are all taken.
# Time: O(V! * V) in the worst case (no prerequisites),
# plus O(V + E) setup.  Space: O(V + E + answers).
def all_course_orders(num_courses, prerequisites):
    waiting = [0] * num_courses
    unlocks = [[] for _ in range(num_courses)]
    for course, before in prerequisites:
        waiting[course] += 1
        unlocks[before].append(course)
    results = []
    order = []
    taken = [False] * num_courses

    def go():
        if len(order) == num_courses:
            results.append(list(order))
            return
        for course in range(num_courses):
            if not taken[course] and waiting[course] == 0:
                taken[course] = True
                order.append(course)
                for nxt in unlocks[course]:
                    waiting[nxt] -= 1
                go()
                for nxt in unlocks[course]:
                    waiting[nxt] += 1
                order.pop()
                taken[course] = False

    go()
    return results
```

```
# ============================================================
# FOLLOW-UP 2: Return the order that is smallest when read
# as a list (smallest course number first whenever there
# is a choice)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Kahn's algorithm with a min-heap instead of a
# queue.
# Time: O((V + E) log V).  Space: O(V + E).
import heapq


def smallest_course_order(num_courses, prerequisites):
    waiting = [0] * num_courses
    unlocks = [[] for _ in range(num_courses)]
    for course, before in prerequisites:
        waiting[course] += 1
        unlocks[before].append(course)
    heap = [c for c in range(num_courses) if waiting[c] == 0]
    heapq.heapify(heap)
    order = []
    while heap:
        course = heapq.heappop(heap)
        order.append(course)
        for nxt in unlocks[course]:
            waiting[nxt] -= 1
            if waiting[nxt] == 0:
                heapq.heappush(heap, nxt)
    if len(order) < num_courses:
        return []
    return order
```

1462. Course Schedule IV

Medium · Depth-First Search, Breadth-First Search, Graph, Topological Sort

leetcode.com/problems/course-schedule-iv

PROBLEM

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [a_i, b_i]` indicates that you **must** take course `a_i` first if you want to take course `b_i`.

- For example, the pair `[0, 1]` indicates that you have to take course `0` before you can take course `1`.

Prerequisites can also be **indirect**. If course `a` is a prerequisite of course `b`, and course `b` is a prerequisite of course `c`, then course `a` is a prerequisite of course `c`.

You are also given an array `queries` where `queries[j] = [u_j, v_j]`. For the `j^th` query, you should answer whether course `u_j` is a prerequisite of course `v_j` or not.

Return *a boolean array* `answer`*, where* `answer[j]` *is the answer to the* `j^th` *query.*

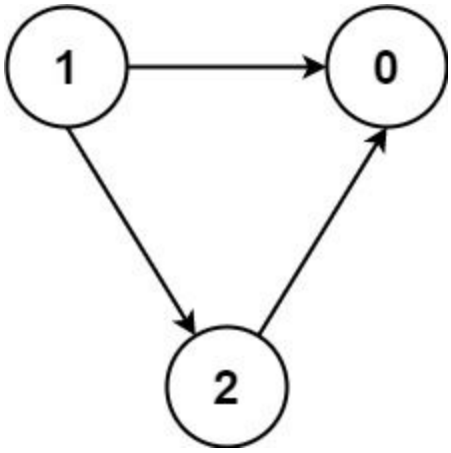
Example 1:

```
Input: numCourses = 2, prerequisites = [[1,0]], queries = [[0,1],[1,0]]
Output: [false,true]
Explanation: The pair [1, 0] indicates that you have to take course 1 before you can take course 0.
Course 0 is not a prerequisite of course 1, but the opposite is true.
```

Example 2:

```
Input: numCourses = 2, prerequisites = [], queries = [[1,0],[0,1]]
Output: [false,false]
Explanation: There are no prerequisites, and each course is independent.
```

Example 3:




```
Input: numCourses = 3, prerequisites = [[1,2],[1,0],[2,0]], queries = [[1,0],[1,2]]
Output: [true,true]
```

Constraints:

- `2 <= numCourses <= 100`

- `0 <= prerequisites.length <= (numCourses * (numCourses - 1) / 2)`
- `prerequisites[i].length == 2`
- `0 <= a_i, b_i <= numCourses - 1`
- `a_i != b_i`
- All the pairs `[a_i, b_i]` are **unique**.
- The prerequisites graph has no cycles.
- `1 <= queries.length <= 10^4`
- `0 <= u_i, v_i <= numCourses - 1`
- `u_i != v_i`

SOLUTIONS (PYTHON 3)

Solution 1: Floyd's Algorithm

> **How to think about it**
>
> Each query asks whether a is a prerequisite of b, i.e. reachability. With a modest number of courses, Floyd on a Boolean matrix closes the graph; each query is then O(1).

We create a 2D array f, where f[i][j] indicates whether node i can reach node j.

Next, we iterate through the prerequisites array prerequisites. For each item [a, b] in it, we set f[a][b] to true.

Then, we use Floyd's algorithm to compute the reachability between all pairs of nodes.

Specifically, we use three nested loops: first enumerating the intermediate node k, then the starting node i, and finally the ending node j. For each iteration, if node i can reach node k and node k can reach node j, then node i can also reach node j, and we set f[i][j] to true.

After computing the reachability between all pairs of nodes, for each query [a, b], we can directly return f[a][b].

The time complexity is $O(n^3)$, and the space complexity is $O(n^2)$, where n is the number of nodes.

Python 3

```python
class Solution:
    def checkIfPrerequisite(
        self, n: int, prerequisites: List[List[int]], queries: List[List[int]]
    ) -> List[bool]:
        f = [[False] * n for _ in range(n)]
        for a, b in prerequisites:
            f[a][b] = True
        for k in range(n):
            for i in range(n):
                for j in range(n):
                    if f[i][k] and f[k][j]:
                        f[i][j] = True
        return [f[a][b] for a, b in queries]
```

Solution 2: Topological Sorting

> **How to think about it**
>
> Method 1 pays $O(n^3)$ for an all-pairs closure. A topological pass unions each node's reachability into its successors, filling the same f[i][j] along the DAG.

Similar to Solution 1, we create a 2D array f, where f[i][j] indicates whether node i can reach node j. Additionally, we create an adjacency list g, where g[i] represents all successor nodes of node i, and an array indeg, where indeg[i] represents the in-degree of node i.

Next, we iterate through the prerequisites array prerequisites. For each item [a, b] in it, we update the adjacency list g and the in-degree array indeg.

Then, we use topological sorting to compute the reachability between all pairs of nodes.

We define a queue q, initially adding all nodes with an in-degree of 0 to the queue. Then, we continuously perform the following operations: remove the front node i from the queue, then iterate through all nodes j in g[i], setting f[i][j] to true. Next, we enumerate node h, and if f[h][i] is true, we also set f[h][j] to true. After this, we decrease the in-degree of j by 1. If the in-degree of j becomes 0, we add j to the queue.

After computing the reachability between all pairs of nodes, for each query [a, b], we can directly return f[a][b].

The time complexity is $O(n^3)$, and the space complexity is $O(n^2)$, where n is the number of nodes.

Python 3

```python
class Solution:
    def checkIfPrerequisite(
        self, n: int, prerequisites: List[List[int]], queries: List[List[int]]
    ) -> List[bool]:
        f = [[False] * n for _ in range(n)]
        g = [[] for _ in range(n)]
        indeg = [0] * n
        for a, b in prerequisites:
            g[a].append(b)
            indeg[b] += 1
        q = deque(i for i, x in enumerate(indeg) if x == 0)
        while q:
            i = q.popleft()
            for j in g[i]:
                f[i][j] = True
                for h in range(n):
                    f[h][j] = f[h][j] or f[h][i]
                indeg[j] -= 1
                if indeg[j] == 0:
                    q.append(j)
        return [f[a][b] for a, b in queries]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. List every (direct or indirect) prerequisite of one
# course
# 2. Many queries: one bitmask of ancestors per course


# ============================================================
# FOLLOW-UP 1: List every (direct or indirect)
# prerequisite of one course
# ============================================================

# ---- follow-up 1: function ----
# Answer: Search backwards from the course through the
# prerequisite edges.
# Time: O(V + E + k log k), k = prerequisites found.
# Space: O(V + E).
def all_prerequisites_of(num_courses, prerequisites, course):
    before = [[] for _ in range(num_courses)]
    for a, b in prerequisites:
        before[b].append(a)
    seen = set()
    stack = [course]
    while stack:
        current = stack.pop()
        for p in before[current]:
            if p not in seen:
                seen.add(p)
                stack.append(p)
    return sorted(seen)
```

```python

# ============================================================
# FOLLOW-UP 2: Many queries: one bitmask of ancestors per
# course
# ============================================================

# ---- follow-up 2: function ----
# Answer: Process the courses in topological order; the
# mask of a course is the OR of the masks of its
# prerequisites plus their own bits. A query is one bit
# test.
# Time: O((V + E) * V / w + q), w = machine word size.
# Space: O(V^2 / w).
def answer_queries_with_masks(num_courses, prerequisites, queries):
    waiting = [0] * num_courses
    unlocks = [[] for _ in range(num_courses)]
    for a, b in prerequisites:
        waiting[b] += 1
        unlocks[a].append(b)
    mask = [0] * num_courses
    queue = [c for c in range(num_courses) if waiting[c] == 0]
    for course in queue:
        for nxt in unlocks[course]:
            mask[nxt] |= mask[course] | (1 << course)
            waiting[nxt] -= 1
            if waiting[nxt] == 0:
                queue.append(nxt)
    return [bool((mask[b] >> a) & 1) for a, b in queries]
```

310. Minimum Height Trees

Medium · Depth-First Search, Breadth-First Search, Graph, Topological Sort

leetcode.com/problems/minimum-height-trees

★ **TOP 7 PROBLEM** in Topological Sort: master this one first

PROBLEM

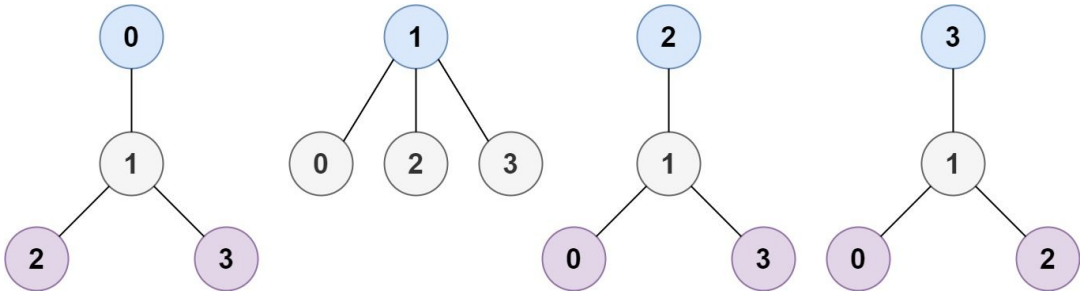
A tree is an undirected graph in which any two vertices are connected by *exactly* one path. In other words, any connected graph without simple cycles is a tree.

Given a tree of `n` nodes labelled from `0` to `n - 1`, and an array of `n - 1 edges` where `edges[i] = [a_i, b_i]` indicates that there is an undirected edge between the two nodes `a_i` and `b_i` in the tree, you can choose any node of the tree as the root. When you select a node `x` as the root, the result tree has height `h`. Among all possible rooted trees, those with minimum height (i.e. `min(h)`) are called **minimum height trees** (MHTs).

Return *a list of all **MHTs'** root labels*. You can return the answer in **any order**.

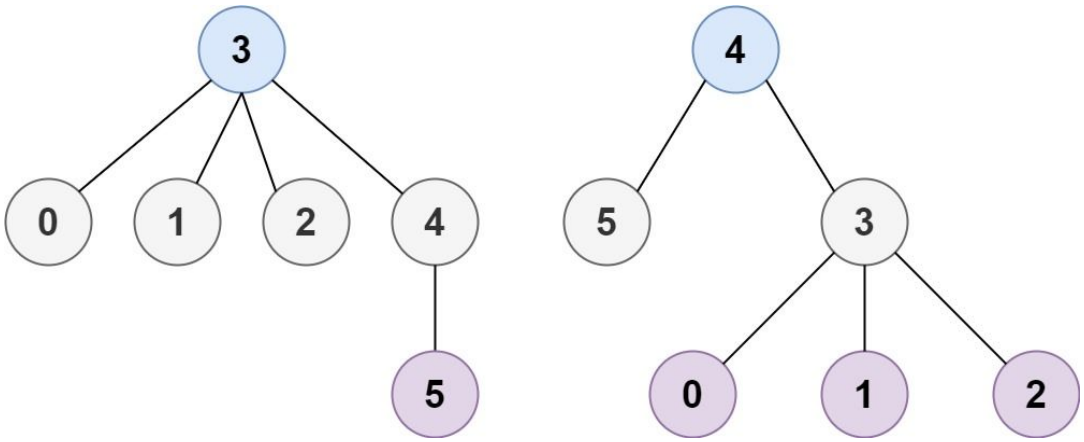
The **height** of a rooted tree is the number of edges on the longest downward path between the root and a leaf.

Example 1:




```
Input: n = 4, edges = [[1,0],[1,2],[1,3]]
Output: [1]
Explanation: As shown, the height of the tree is 1 when the root is the node with label 1 which is the only MHT.
```

Example 2:




```
Input: n = 6, edges = [[3,0],[3,1],[3,2],[3,4],[5,4]]
Output: [3,4]
```

Constraints:

- `1 <= n <= 2 * 10^4`
- `edges.length == n - 1`

- $0 <= a_i, b_i < n$
- $a_i != b_i$
- All the pairs (a_i, b_i) are distinct.
- The given input is **guaranteed** to be a tree and there will be **no repeated** edges.

SOLUTION (PYTHON 3)

Solution 1: Topological Sorting

> **How to think about it**
>
> Height is the distance from the root to the farthest leaf. BFS from every node is $O(n^2)$. The center of the tree diameter — one or two nodes — is exactly the set of minimum-height roots. Leaves have degree 1. Peel them inward together (a topological peel): each layer decrements neighbors, and new degree-1 nodes enqueue. The last remaining layer is the center. A single node is returned as-is.

If the tree only has one node, then this node is the root of the minimum height tree. We can directly return this node.

If the tree has multiple nodes, there must be leaf nodes. A leaf node is a node that only has one adjacent node. We can use topological sorting to peel off the leaf nodes from the outside to the inside. When we reach the last layer, the remaining nodes are the root nodes of the minimum height tree.

The time complexity is O(n) and the space complexity is O(n), where n is the number of nodes.

Python 3

```python
class Solution:
    def findMinHeightTrees(self, n: int, edges: List[List[int]]) -> List[int]:
        if n == 1:
            return [0]
        g = [[] for _ in range(n)]
        degree = [0] * n
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)
            degree[a] += 1
            degree[b] += 1
        q = deque(i for i in range(n) if degree[i] == 1)
        ans = []
        while q:
            ans.clear()
            for _ in range(len(q)):
                a = q.popleft()
                ans.append(a)
                for b in g[a]:
                    degree[b] -= 1
                    if degree[b] == 1:
                        q.append(b)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the minimum height itself
# 2. Return the longest path (diameter) of the tree


# ============================================================
# FOLLOW-UP 1: Return the minimum height itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Peel leaves layer by layer; the number of rounds
# needed to reach the last one or two nodes is the minimum
```

```
# height.
# Time: O(n).  Space: O(n).
def minimum_tree_height(n, edges):
    if n == 1:
        return 0
    neighbours = [set() for _ in range(n)]
    for a, b in edges:
        neighbours[a].add(b)
        neighbours[b].add(a)
    leaves = [i for i in range(n) if len(neighbours[i]) == 1]
    remaining = n
    rounds = 0
    while remaining > 2:
        remaining -= len(leaves)
        new_leaves = []
        for leaf in leaves:
            other = neighbours[leaf].pop()
            neighbours[other].remove(leaf)
            if len(neighbours[other]) == 1:
                new_leaves.append(other)
        leaves = new_leaves
        rounds += 1
    return rounds + (1 if remaining == 2 else 0)


# ============================================================
# FOLLOW-UP 2: Return the longest path (diameter) of the
# tree
# ============================================================

# ---- follow-up 2: function ----
# Answer: Breadth-first search from any node finds one end
# of the diameter; a second search from that end finds the
# other end and the path.
# Time: O(n).  Space: O(n).
def tree_diameter_path(n, edges):
    neighbours = [[] for _ in range(n)]
    for a, b in edges:
        neighbours[a].append(b)
        neighbours[b].append(a)

    def farthest(start):
        parent = {start: None}
        queue = [start]
        last = start
        for node in queue:
            last = node
            for nxt in neighbours[node]:
                if nxt not in parent:
                    parent[nxt] = node
                    queue.append(nxt)
        return last, parent

    end_one, parent = farthest(0)
    end_two, parent = farthest(end_one)
    path = []
    node = end_two
    while node is not None:
        path.append(node)
        node = parent[node]
    return path
```

2192. All Ancestors of a Node in a Directed Acyclic Graph

Medium · Depth-First Search, Breadth-First Search, Graph, Topological Sort, Directed Acyclic Graph

leetcode.com/problems/all-ancestors-of-a-node-in-a-directed-acyclic-graph

PROBLEM

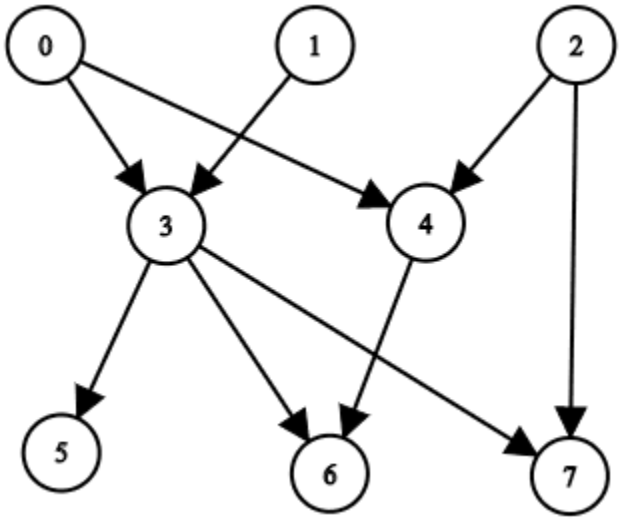
You are given a positive integer `n` representing the number of nodes of a **Directed Acyclic Graph** (DAG). The nodes are numbered from `0` to `n - 1` (**inclusive**).

You are also given a 2D integer array `edges`, where `edges[i] = [from`$_i$`, to`$_i$`]` denotes that there is a **unidirectional** edge from `from`$_i$ to `to`$_i$ in the graph.

Return *a list* `answer`*, where* `answer[i]` *is the* ***list of ancestors*** *of the* `i`th *node, sorted in* ***ascending order***.

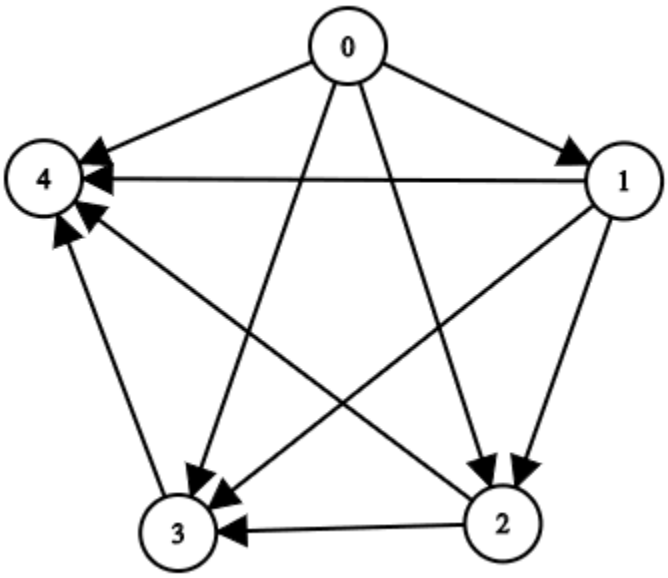
A node `u` is an **ancestor** of another node `v` if `u` can reach `v` via a set of edges.

Example 1:




```
Input: n = 8, edgeList = [[0,3],[0,4],[1,3],[2,4],[2,7],[3,5],[3,6],[3,7],[4,6]]
Output: [[],[],[],[0,1],[0,2],[0,1,3],[0,1,2,3,4],[0,1,2,3]]
Explanation:
The above diagram represents the input graph.
- Nodes 0, 1, and 2 do not have any ancestors.
- Node 3 has two ancestors 0 and 1.
- Node 4 has two ancestors 0 and 2.
- Node 5 has three ancestors 0, 1, and 3.
- Node 6 has five ancestors 0, 1, 2, 3, and 4.
- Node 7 has four ancestors 0, 1, 2, and 3.
```

Example 2:




```
Input: n = 5, edgeList = [[0,1],[0,2],[0,3],[0,4],[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]]
Output: [[],[0],[0,1],[0,1,2],[0,1,2,3]]
Explanation:
The above diagram represents the input graph.
- Node 0 does not have any ancestor.
- Node 1 has one ancestor 0.
- Node 2 has two ancestors 0 and 1.
- Node 3 has three ancestors 0, 1, and 2.
- Node 4 has four ancestors 0, 1, 2, and 3.
```

Constraints:

- `1 <= n <= 1000`
- `0 <= edges.length <= min(2000, n * (n - 1) / 2)`
- `edges[i].length == 2`
- $0 <= from_i, to_i <= n - 1$
- $from_i != to_i$
- There are no duplicate edges.
- The graph is **directed** and **acyclic**.

SOLUTION (PYTHON 3)

Solution 1: BFS

How to think about it

List every ancestor of each node in a DAG, sorted by id. Reverse DFS from every node repeats shared in-edges and still needs sorting. With n≤ 1000 we can walk descendants of each i and append i to those lists, which is already sorted by i. Build the adjacency list, BFS from every i, and record i on every reachable node. A visited set prevents re-enqueue. Each answer row is increasing because sources are enumerated in order.

First, we construct the adjacency list g based on the two-dimensional array edges, where g[i] represents all successor nodes of node i.

Then, we enumerate node i as the ancestor node from small to large, use BFS to search all successor nodes of node i, and add node i to the ancestor list of these successor nodes.

The time complexity is $O(n^2)$, and the space complexity is $O(n^2)$. Where n is the number of nodes.

Python 3

```python
class Solution:
    def getAncestors(self, n: int, edges: List[List[int]]) -> List[List[int]]:
        def bfs(s: int):
            q = deque([s])
            vis = {s}
            while q:
                i = q.popleft()
                for j in g[i]:
                    if j not in vis:
                        vis.add(j)
                        q.append(j)
                        ans[j].append(s)

        g = defaultdict(list)
        for u, v in edges:
            g[u].append(v)
        ans = [[] for _ in range(n)]
        for i in range(n):
            bfs(i)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the descendants of every node instead
# 2. Only count the ancestors of each node


# ============================================================
# FOLLOW-UP 1: Return the descendants of every node
# instead
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same idea on reversed edges.
# Time: O(V * (V + E)).  Space: O(V^2).
def all_descendants(n, edges):
    waiting = [0] * n
    came_from = [[] for _ in range(n)]
    for a, b in edges:
        waiting[a] += 1
        came_from[b].append(a)
    below = [set() for _ in range(n)]
    queue = [i for i in range(n) if waiting[i] == 0]
    for node in queue:
        for parent in came_from[node]:
            below[parent].add(node)
            below[parent] |= below[node]
            waiting[parent] -= 1
            if waiting[parent] == 0:
                queue.append(parent)
    return [sorted(s) for s in below]


# ============================================================
# FOLLOW-UP 2: Only count the ancestors of each node
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same walk, but only keep sizes of the ancestor
# sets.
# Time: O(V * (V + E)).  Space: O(V^2).
def ancestor_counts(n, edges):
    waiting = [0] * n
    goes_to = [[] for _ in range(n)]
```

```
    for a, b in edges:
        waiting[b] += 1
        goes_to[a].append(b)
    above = [set() for _ in range(n)]
    queue = [i for i in range(n) if waiting[i] == 0]
    for node in queue:
        for nxt in goes_to[node]:
            above[nxt].add(node)
            above[nxt] |= above[node]
            waiting[nxt] -= 1
            if waiting[nxt] == 0:
                queue.append(nxt)
    return [len(s) for s in above]
```

PATTERN 30 · TOPOLOGICAL SORT · PROBLEM 7 OF 20

1136. Parallel Courses

Medium · Graph, Topological Sort, Directed Acyclic Graph

leetcode.com/problems/parallel-courses

★ **TOP 7 PROBLEM** in Topological Sort: master this one first

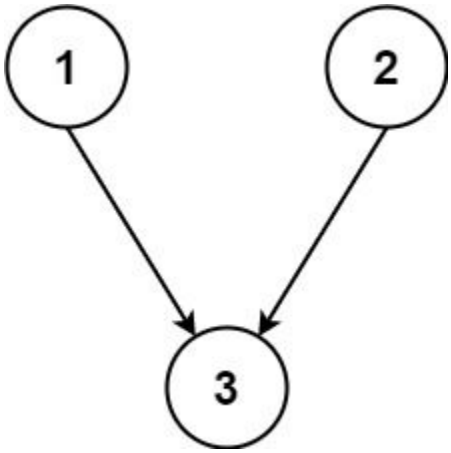
PROBLEM

You are given an integer `n`, which indicates that there are `n` courses labeled from `1` to `n`. You are also given an array `relations` where `relations[i] = [prevCourse`$_i$`, nextCourse`$_i$`]`, representing a prerequisite relationship between course `prevCourse`$_i$ and course `nextCourse`$_i$: course `prevCourse`$_i$ has to be taken before course `nextCourse`$_i$.

In one semester, you can take **any number** of courses as long as you have taken all the prerequisites in the **previous** semester for the courses you are taking.

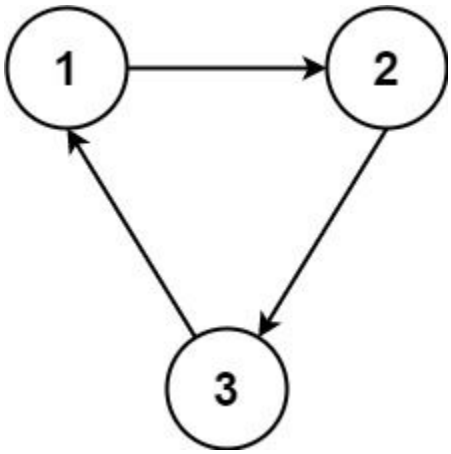
Return *the **minimum** number of semesters needed to take all courses*. If there is no way to take all the courses, return `-1`.

Example 1:




```
Input: n = 3, relations = [[1,3],[2,3]]
Output: 2
Explanation: The figure above represents the given graph.
In the first semester, you can take courses 1 and 2.
In the second semester, you can take course 3.
```

Example 2:



```
Input: n = 3, relations = [[1,2],[2,3],[3,1]]
Output: -1
Explanation: No course can be studied because they are prerequisites of each other.
```

Constraints:

- `1 <= n <= 5000`
- `1 <= relations.length <= 5000`
- `relations[i].length == 2`
- `1 <= prevCourse_i, nextCourse_i <= n`
- `prevCourse_i != nextCourse_i`
- All the pairs `[prevCourse_i, nextCourse_i]` are **unique**.

SOLUTION (PYTHON 3)

Solution 1: Topological Sorting

> **How to think about it**
>
> The number of semesters is the longest chain of prerequisites; a semester can take every course whose indegree is now 0. Layered Kahn topology: each layer is one semester. Leftover courses mean a cycle, so return -1.

We can first build a graph g to represent the prerequisite relationships between courses, and count the in-degree indeg of each course.

Then we enqueue the courses with an in-degree of 0 and start topological sorting. Each time, we dequeue a course from the queue, reduce the in-degree of the courses that it points to by 1, and if the in-degree becomes 0 after reduction, we enqueue that course. When the queue is empty, if there are still courses that have not been completed, it means that it is impossible to complete all courses, so we return -1. Otherwise, we return the number of semesters required to complete all courses.

The time complexity is O(n + m), and the space complexity is O(n + m). Here, n and m are the number of courses and the number of prerequisite relationships, respectively.

Python 3

```python
class Solution:
    def minimumSemesters(self, n: int, relations: List[List[int]]) -> int:
        g = defaultdict(list)
        indeg = [0] * n
        for prev, nxt in relations:
            prev, nxt = prev - 1, nxt - 1
            g[prev].append(nxt)
            indeg[nxt] += 1
        q = deque(i for i, v in enumerate(indeg) if v == 0)
        ans = 0
        while q:
            ans += 1
            for _ in range(len(q)):
                i = q.popleft()
                n -= 1
                for j in g[i]:
                    indeg[j] -= 1
                    if indeg[j] == 0:
                        q.append(j)
        return -1 if n else ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the semester in which each course is taken
# 2. Return the chain of courses that forces the answer


# ============================================================
# FOLLOW-UP 1: Return the semester in which each course is
# taken
# ============================================================

# ---- follow-up 1: function ----
# Answer: Kahn's algorithm in layers: the layer number is
# the semester.
# Time: O(n + m).  Space: O(n + m).
def semester_of_each_course(n, relations):
    waiting = [0] * (n + 1)
    unlocks = [[] for _ in range(n + 1)]
    for a, b in relations:
        waiting[b] += 1
        unlocks[a].append(b)
    semester = [0] * (n + 1)
    current = [c for c in range(1, n + 1) if waiting[c] == 0]
    for c in current:
        semester[c] = 1
    taken = 0
    while current:
        taken += len(current)
        following = []
        for c in current:
            for nxt in unlocks[c]:
                waiting[nxt] -= 1
                if waiting[nxt] == 0:
                    semester[nxt] = semester[c] + 1
                    following.append(nxt)
        current = following
    if taken < n:
        return None
    return semester[1:]


# ============================================================
# FOLLOW-UP 2: Return the chain of courses that forces the
# answer
# ============================================================

# ---- follow-up 2: function ----
# Answer: Remember, for every course, the prerequisite
# that put it in its semester, and walk back from a last-
# semester course.
# Time: O(n + m).  Space: O(n + m).
def longest_chain(n, relations):
    waiting = [0] * (n + 1)
    unlocks = [[] for _ in range(n + 1)]
    for a, b in relations:
        waiting[b] += 1
        unlocks[a].append(b)
    length = [1] * (n + 1)
    came_from = [0] * (n + 1)
    queue = [c for c in range(1, n + 1) if waiting[c] == 0]
    for c in queue:
```

```
        for nxt in unlocks[c]:
            if length[c] + 1 > length[nxt]:
                length[nxt] = length[c] + 1
                came_from[nxt] = c
```

```
            waiting[nxt] -= 1
            if waiting[nxt] == 0:
                queue.append(nxt)
    if len(queue) < n:
        return None
    last = 1
    for c in range(1, n + 1):
        if length[c] > length[last]:
            last = c
    chain = []
    while last != 0:
        chain.append(last)
        last = came_from[last]
    return chain[::-1]
```

2115. Find All Possible Recipes from Given Supplies

Medium · Graph, Topological Sort, Array, Hash Table, String, Directed Acyclic Graph

leetcode.com/problems/find-all-possible-recipes-from-given-supplies

★ **TOP 7 PROBLEM** in Topological Sort: master this one first

PROBLEM

You have information about `n` different recipes. You are given a string array `recipes` and a 2D string array `ingredients`. The `i`th recipe has the name `recipes[i]`, and you can **create** it if you have **all** the needed ingredients from `ingredients[i]`. A recipe can also be an ingredient for **other** recipes, i.e., `ingredients[i]` may contain a string that is in `recipes`.

You are also given a string array `supplies` containing all the ingredients that you initially have, and you have an infinite supply of all of them.

Return *a list of all the recipes that you can create.* You may return the answer in **any order**.

Note that two recipes may contain each other in their ingredients.

Example 1:

```
Input: recipes = ["bread"], ingredients = [["yeast","flour"]], supplies = ["yeast","flour","corn"]
Output: ["bread"]
Explanation:
We can create "bread" since we have the ingredients "yeast" and "flour".
```

Example 2:

```
Input: recipes = ["bread","sandwich"], ingredients = [["yeast","flour"],["bread","meat"]], supplies =
["yeast","flour","meat"]
Output: ["bread","sandwich"]
Explanation:
We can create "bread" since we have the ingredients "yeast" and "flour".
We can create "sandwich" since we have the ingredient "meat" and can create the ingredient "bread".
```

Example 3:

```
Input: recipes = ["bread","sandwich","burger"], ingredients =
[["yeast","flour"],["bread","meat"],["sandwich","meat","bread"]], supplies = ["yeast","flour","meat"]
Output: ["bread","sandwich","burger"]
Explanation:
We can create "bread" since we have the ingredients "yeast" and "flour".
We can create "sandwich" since we have the ingredient "meat" and can create the ingredient "bread".
We can create "burger" since we have the ingredient "meat" and can create the ingredients "bread" and
"sandwich".
```

Constraints:

- `n == recipes.length == ingredients.length`
- `1 <= n <= 100`
- `1 <= ingredients[i].length, supplies.length <= 100`
- `1 <= recipes[i].length, ingredients[i][j].length, supplies[k].length <= 10`
- `recipes[i], ingredients[i][j]`, and `supplies[k]` consist only of lowercase English letters.
- All the values of `recipes` and `supplies` combined are unique.
- Each `ingredients[i]` does not contain any duplicate values.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

A recipe is cookable once every ingredient is available from the initial supplies or from already cooked recipes, which is a directed dependency graph. Rechecking ingredients per recipe repeats work and mishandles chains. Point each ingredient to recipes that need it, with in-degree equal to the number of ingredients. Supplies are the sources; a recipe whose in-degree falls to zero can be cooked and used to unlock others. We build that graph and run Kahn's algorithm from supplies, appending a recipe when its in-degree hits zero.

Python 3

```python
class Solution:
    def findAllRecipes(
        self, recipes: List[str], ingredients: List[List[str]], supplies: List[str]
    ) -> List[str]:
        g = defaultdict(list)
        indeg = defaultdict(int)
        for a, b in zip(recipes, ingredients):
            for v in b:
                g[v].append(a)
            indeg[a] += len(b)
        q = supplies
        ans = []
        for i in q:
            for j in g[i]:
                indeg[j] -= 1
                if indeg[j] == 0:
                    ans.append(j)
                    q.append(j)
        return ans
```

Complexity. Time: O(n + m), n = recipes plus supplies, m = total ingredients. Space: O(n + m).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the recipes in an order you could cook them
# 2. For each recipe that cannot be made, name the
# ingredients that are missing


# ============================================================
# FOLLOW-UP 1: Return the recipes in an order you could
# cook them
# ============================================================

# ---- follow-up 1: function ----
# Answer: The queue order of Kahn's algorithm already is a
# valid cooking order.
# Time: O(R + I), I = total ingredients.  Space: O(R + I).
def cooking_order(recipes, ingredients, supplies):
    have = set(supplies)
    waiting = {}
    needed_by = {}
    for recipe, items in zip(recipes, ingredients):
        waiting[recipe] = 0
        for item in items:
            if item not in have:
                waiting[recipe] += 1
                needed_by.setdefault(item, []).append(recipe)
    queue = [r for r in recipes if waiting[r] == 0]
    for recipe in queue:
```

```
        for other in needed_by.get(recipe, []):
            waiting[other] -= 1
            if waiting[other] == 0:
                queue.append(other)
    return queue


# ============================================================
# FOLLOW-UP 2: For each recipe that cannot be made, name
# the ingredients that are missing
# ============================================================

# ---- follow-up 2: function ----
# Answer: A missing ingredient is neither a supply nor a
# recipe that can be made.
# Time: O(R + I + m log m).  Space: O(R + I).
def missing_ingredients(recipes, ingredients, supplies):
    possible = set(cooking_order(recipes, ingredients, supplies))
    have = set(supplies) | possible
    missing = {}
    for recipe, items in zip(recipes, ingredients):
        if recipe not in possible:
            missing[recipe] = sorted(set(i for i in items if i not in have))
    return missing
```

PATTERN 30 · TOPOLOGICAL SORT · PROBLEM 9 OF 20

851. Loud and Rich

Medium · Depth-First Search, Graph, Topological Sort, Array, Directed Acyclic Graph

leetcode.com/problems/loud-and-rich

PROBLEM

There is a group of `n` people labeled from `0` to `n - 1` where each person has a different amount of money and a different level of quietness.

You are given an array `richer` where `richer[i] = [a_i, b_i]` indicates that `a_i` has more money than `b_i` and an integer array `quiet` where `quiet[i]` is the quietness of the `i^th` person. All the given data in richer are **logically correct** (i.e., the data will not lead you to a situation where `x` is richer than `y` and `y` is richer than `x` at the same time).

Return *an integer array* `answer` *where* `answer[x] = y` *if* `y` *is the least quiet person (that is, the person* `y` *with the smallest value of* `quiet[y]`*) among all people who definitely have equal to or more money than the person* `x`.

Example 1:

```
Input: richer = [[1,0],[2,1],[3,1],[3,7],[4,3],[5,3],[6,3]], quiet = [3,2,5,4,6,1,7,0]
Output: [5,5,2,5,4,5,6,7]
Explanation:
answer[0] = 5.
Person 5 has more money than 3, which has more money than 1, which has more money than 0.
The only person who is quieter (has lower quiet[x]) is person 7, but it is not clear if they have more money than person 0.
answer[7] = 7.
Among all people that definitely have equal to or more money than person 7 (which could be persons 3, 4, 5, 6, or 7), the person who is the quietest (has lower quiet[x]) is person 7.
The other answers can be filled out with similar reasoning.
```

Example 2:

```
Input: richer = [], quiet = [0]
Output: [0]
```

Constraints:

- `n == quiet.length`
- `1 <= n <= 500`
- `0 <= quiet[i] < n`
- All the values of `quiet` are **unique**.
- `0 <= richer.length <= n * (n - 1) / 2`
- `0 <= a_i, b_i < n`
- `a_i != b_i`
- All the pairs of `richer` are **unique**.
- The observations in `richer` are all logically consistent.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

On the DAG of “richer than”, each person wants the quietest among themselves and all richer people. n≤ 500, so a fresh search per person retraces the same subgraphs. Point edges from poorer to richer and memoize DFS: start with oneself, then take the quieter answer among richer neighbors. Each node is computed once.

Python 3

```
class Solution:
    def loudAndRich(self, richer: List[List[int]], quiet: List[int]) -> List[int]:
        def dfs(i: int):
            if ans[i] != -1:
                return
            ans[i] = i
            for j in g[i]:
                dfs(j)
                if quiet[ans[j]] < quiet[ans[i]]:
                    ans[i] = ans[j]

        g = defaultdict(list)
        for a, b in richer:
            g[b].append(a)
        n = len(quiet)
        ans = [-1] * n
        for i in range(n):
            dfs(i)
        return ans
```

Complexity. Time: O(n + e), e = len(richer). Space: O(n + e).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Quietest person among everyone who is POORER than or
# equal to each person
# 2. Return all people who are at least as rich as the
# person, quietest first


# ============================================================
# FOLLOW-UP 1: Quietest person among everyone who is
# POORER than or equal to each person
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same search, along the edges in the other
# direction.
# Time: O(n + m).  Space: O(n + m).
def quietest_among_poorer(richer, quiet):
    n = len(quiet)
    poorer = [[] for _ in range(n)]
    for a, b in richer:
        poorer[a].append(b)
    answer = [-1] * n

    def go(person):
        if answer[person] != -1:
            return answer[person]
        answer[person] = person
        for other in poorer[person]:
            candidate = go(other)
            if quiet[candidate] < quiet[answer[person]]:
                answer[person] = candidate
        return answer[person]

    for person in range(n):
        go(person)
    return answer


# ============================================================
# FOLLOW-UP 2: Return all people who are at least as rich
# as the person, quietest first
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Collect everyone who can reach the person
# through the richer edges, then sort by quietness.
# Time: O(n + m + k log k).  Space: O(n + m).
def richer_people_by_quietness(richer, quiet, person):
    n = len(quiet)
    above = [[] for _ in range(n)]
    for a, b in richer:
        above[b].append(a)
    seen = {person}
    stack = [person]
    while stack:
        current = stack.pop()
        for other in above[current]:
            if other not in seen:
                seen.add(other)
                stack.append(other)
```

```
    return sorted(seen, key=lambda p: quiet[p])
```

1557. Minimum Number of Vertices to Reach All Nodes

Medium · Graph, Directed Acyclic Graph

leetcode.com/problems/minimum-number-of-vertices-to-reach-all-nodes

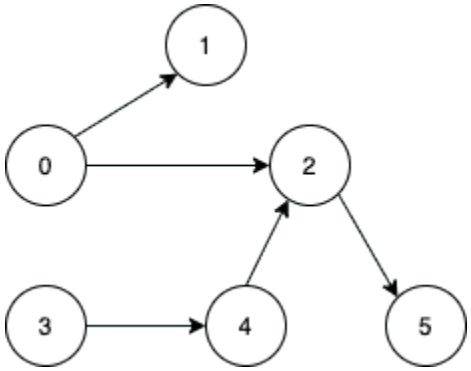
PROBLEM

Given a **directed acyclic graph**, with `n` vertices numbered from `0` to `n-1`, and an array `edges` where `edges[i] = [from_i, to_i]` represents a directed edge from node `from_i` to node `to_i`.

Find *the smallest set of vertices from which all nodes in the graph are reachable*. It's guaranteed that a unique solution exists.

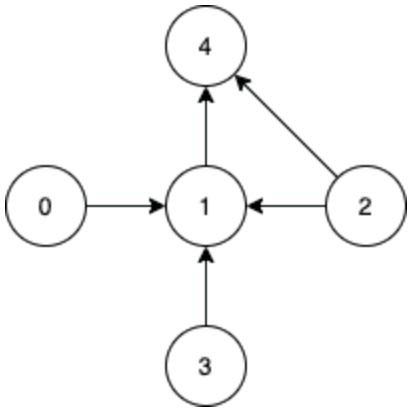
Notice that you can return the vertices in any order.

Example 1:




```
Input: n = 6, edges = [[0,1],[0,2],[2,5],[3,4],[4,2]]
Output: [0,3]
Explanation: It's not possible to reach all the nodes from a single vertex. From 0 we can reach
[0,1,2,5]. From 3 we can reach [3,4,2,5]. So we output [0,3].
```

Example 2:




```
Input: n = 5, edges = [[0,1],[2,1],[3,1],[1,4],[2,4]]
Output: [0,2,3]
Explanation: Notice that vertices 0, 3 and 2 are not reachable from any other node, so we must include
them. Also any of these vertices can reach nodes 1 and 4.
```

Constraints:

- `2 <= n <= 10^5`
- `1 <= edges.length <= min(10^5, n * (n - 1) / 2)`
- `edges[i].length == 2`

- $0 <= from_i, to_i < n$
- All pairs $(from_i, to_i)$ are distinct.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Choose as few vertices as possible so that every node is reachable from the set in a DAG. n and the number of edges reach 10^5, so we cannot enumerate subsets. A vertex of in-degree 0 cannot be reached from anyone else and must be chosen. Every other vertex has an in-edge and is therefore reachable from some source in the DAG. The answer is exactly the set of in-degree-zero vertices.

Python 3

```python
class Solution:
    def findSmallestSetOfVertices(self, n: int, edges: List[List[int]]) -> List[int]:
        cnt = Counter(t for _, t in edges)
        return [i for i in range(n) if cnt[i] == 0]
```

Complexity. Time: O(n + e). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The graph may contain cycles
# 2. The edges are undirected


# ============================================================
# FOLLOW-UP 1: The graph may contain cycles
# ============================================================

# ---- follow-up 1: function ----
# Answer: Take one node from every strongly connected
# component that has no incoming edge from another
# component (Kosaraju).
# Time: O(n + e).  Space: O(n + e).
def smallest_set_with_cycles(n, edges):
    forward = [[] for _ in range(n)]
    backward = [[] for _ in range(n)]
    for a, b in edges:
        forward[a].append(b)
        backward[b].append(a)

    seen = [False] * n
    finish_order = []
    for start in range(n):
        if seen[start]:
            continue
        seen[start] = True
        stack = [(start, 0)]
        while stack:
            node, i = stack.pop()
            if i < len(forward[node]):
                stack.append((node, i + 1))
                nxt = forward[node][i]
                if not seen[nxt]:
                    seen[nxt] = True
                    stack.append((nxt, 0))
            else:
                finish_order.append(node)

    component = [-1] * n
```

```
    count = 0
    for start in reversed(finish_order):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for nxt in backward[node]:
                if component[nxt] == -1:
                    component[nxt] = count
                    stack.append(nxt)
        count += 1

    has_incoming = [False] * count
    for a, b in edges:
        if component[a] != component[b]:
            has_incoming[component[b]] = True
    answer = []
    chosen = set()
```

```
    for node in range(n):
        c = component[node]
        if not has_incoming[c] and c not in chosen:
            chosen.add(c)
            answer.append(node)
    return answer


# ============================================================
# FOLLOW-UP 2: The edges are undirected
# ============================================================

# ---- follow-up 2: function ----
# Answer: One node per connected component is needed:
# count the components with a depth-first search.
# Time: O(n + e).  Space: O(n + e).
def smallest_set_undirected(n, edges):
    neighbours = [[] for _ in range(n)]
    for a, b in edges:
        neighbours[a].append(b)
        neighbours[b].append(a)
    seen = [False] * n
    starts = []
    for start in range(n):
        if seen[start]:
            continue
        starts.append(start)
        seen[start] = True
        stack = [start]
        while stack:
            node = stack.pop()
            for nxt in neighbours[node]:
                if not seen[nxt]:
                    seen[nxt] = True
                    stack.append(nxt)
    return starts
```

444. Sequence Reconstruction

Medium · Graph, Topological Sort, Array, Directed Acyclic Graph

leetcode.com/problems/sequence-reconstruction

PROBLEM

You are given an integer array `nums` of length `n` where `nums` is a permutation of the integers in the range `[1, n]`. You are also given a 2D integer array `sequences` where `sequences[i]` is a subsequence of `nums`.

Check if `nums` is the shortest possible and the only **supersequence**. The shortest **supersequence** is a sequence **with the shortest length** and has all `sequences[i]` as subsequences. There could be multiple valid **supersequences** for the given array `sequences`.

- For example, for `sequences = [[1,2],[1,3]]`, there are two shortest **supersequences**, `[1,2,3]` and `[1,3,2]`.
- While for `sequences = [[1,2],[1,3],[1,2,3]]`, the only shortest **supersequence** possible is `[1,2,3]`. `[1,2,3,4]` is a possible supersequence but not the shortest.

Return `true` *if* `nums` *is the only shortest* ***supersequence*** *for* `sequences`*, or* `false` *otherwise*.

A **subsequence** is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements.

Example 1:

```
Input: nums = [1,2,3], sequences = [[1,2],[1,3]]
Output: false
Explanation: There are two possible supersequences: [1,2,3] and [1,3,2].
The sequence [1,2] is a subsequence of both: [1,2,3] and [1,3,2].
The sequence [1,3] is a subsequence of both: [1,2,3] and [1,3,2].
Since nums is not the only shortest supersequence, we return false.
```

Example 2:

```
Input: nums = [1,2,3], sequences = [[1,2]]
Output: false
Explanation: The shortest possible supersequence is [1,2].
The sequence [1,2] is a subsequence of it: [1,2].
Since nums is not the shortest supersequence, we return false.
```

Example 3:

```
Input: nums = [1,2,3], sequences = [[1,2],[1,3],[2,3]]
Output: true
Explanation: The shortest possible supersequence is [1,2,3].
The sequence [1,2] is a subsequence of it: [1,2,3].
The sequence [1,3] is a subsequence of it: [1,2,3].
The sequence [2,3] is a subsequence of it: [1,2,3].
Since nums is the only shortest supersequence, we return true.
```

Constraints:

- `n == nums.length`
- `1 <= n <= 10^4`
- `nums` is a permutation of all the integers in the range `[1, n]`.
- `1 <= sequences.length <= 10^4`
- `1 <= sequences[i].length <= 10^4`
- `1 <= sum(sequences[i].length) <= 10^5`
- `1 <= sequences[i][j] <= n`
- All the arrays of `sequences` are **unique**.

- `sequences[i]` is a subsequence of `nums`.

SOLUTION (PYTHON 3)

Solution 1: Topological Sorting

> **How to think about it**
>
> We must decide whether nums is the unique shortest supersequence of the given sequences. Several topological orders would mean several supersequences. Turn consecutive pairs in each seq into directed edges and record indegrees. The queue may hold only one zero-indegree node at a time—two candidates mean the order is not unique. An empty queue at the end means every node was forced. We need not compare against nums position-wise: if a unique order exists, it has to be nums.

We can first traverse each subsequence `seq`. For each pair of adjacent elements a and b, we establish a directed edge a → b. At the same time, we count the in-degree of each node, and finally add all nodes with an in-degree of 0 to the queue.

When the number of nodes in the queue is equal to 1, we take out the head node i, remove i from the graph, and decrease the in-degree of all adjacent nodes of i by 1. If the in-degree of the adjacent nodes becomes 0 after decreasing, add these nodes to the queue. Repeat the above operation until the length of the queue is not 1. At this point, check whether the queue is empty. If it is not empty, it means there are multiple shortest supersequences, return `false`; if it is empty, it means there is only one shortest supersequence, return `true`.

The time complexity is O(n + m), and the space complexity is O(n + m). Where n and m are the number of nodes and edges, respectively.

Python 3

```python
class Solution:
    def sequenceReconstruction(
        self, nums: List[int], sequences: List[List[int]]
    ) -> bool:
        n = len(nums)
        g = [[] for _ in range(n)]
        indeg = [0] * n
        for seq in sequences:
            for a, b in pairwise(seq):
                a, b = a - 1, b - 1
                g[a].append(b)
                indeg[b] += 1
        q = deque(i for i, x in enumerate(indeg) if x == 0)
        while len(q) == 1:
            i = q.popleft()
            for j in g[i]:
                indeg[j] -= 1
                if indeg[j] == 0:
                    q.append(j)
        return len(q) == 0
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count how many sequences the given sequences can
# build
# 2. Return one valid sequence (the smallest in dictionary
# order)


# ========================================================
# FOLLOW-UP 1: Count how many sequences the given
# sequences can build
# ========================================================

# ---- follow-up 1: function ----
```

```
# Answer: Count the topological orders of the graph with a
# bitmask over the numbers taken so far (small n only).
# Time: O(2^n * n).  Space: O(2^n).
def count_reconstructions(n, sequences):
    before = [0] * n                          # bitmask of numbers that must come earlier
    for seq in sequences:
        for i in range(1, len(seq)):
            before[seq[i] - 1] |= 1 << (seq[i - 1] - 1)
    ways = {0: 1}
    for mask in range(1 << n):
        if mask not in ways:
            continue
        for x in range(n):
            if not (mask >> x) & 1 and (before[x] & mask) == before[x]:
                new_mask = mask | (1 << x)
                ways[new_mask] = ways.get(new_mask, 0) + ways[mask]
    return ways.get((1 << n) - 1, 0)


# ============================================================
# FOLLOW-UP 2: Return one valid sequence (the smallest in
# dictionary order)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Kahn's algorithm with a min-heap.
# Time: O((n + L) log n), L = total sequence length.
# Space: O(n + L).
import heapq


def smallest_reconstruction(n, sequences):
    waiting = [0] * (n + 1)
    unlocks = [set() for _ in range(n + 1)]
    for seq in sequences:
        for i in range(1, len(seq)):
            if seq[i] not in unlocks[seq[i - 1]]:
                unlocks[seq[i - 1]].add(seq[i])
                waiting[seq[i]] += 1
    heap = [x for x in range(1, n + 1) if waiting[x] == 0]
    heapq.heapify(heap)
    order = []
    while heap:
        x = heapq.heappop(heap)
        order.append(x)
        for nxt in unlocks[x]:
            waiting[nxt] -= 1
```

```
            if waiting[nxt] == 0:
                heapq.heappush(heap, nxt)
    if len(order) < n:
        return None
    return order
```

1976. Number of Ways to Arrive at Destination

Medium · Graph, Topological Sort, Dynamic Programming, Shortest Path, Dijkstra

leetcode.com/problems/number-of-ways-to-arrive-at-destination

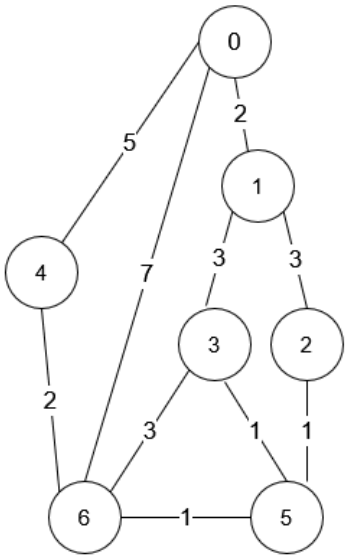
PROBLEM

You are in a city that consists of `n` intersections numbered from `0` to `n - 1` with **bi-directional** roads between some intersections. The inputs are generated such that you can reach any intersection from any other intersection and that there is at most one road between any two intersections.

You are given an integer `n` and a 2D integer array `roads` where `roads[i] = [u_i, v_i, time_i]` means that there is a road between intersections u_i and v_i that takes $time_i$ minutes to travel. You want to know in how many ways you can travel from intersection `0` to intersection `n - 1` in the **shortest amount of time**.

Return *the **number of ways** you can arrive at your destination in the **shortest amount of time***. Since the answer may be large, return it **modulo** $10^9 + 7$.

Example 1:




```
Input: n = 7, roads =
[[0,6,7],[0,1,2],[1,2,3],[1,3,3],[6,3,3],[3,5,1],[6,5,1],[2,5,1],[0,4,5],[4,6,2]]
Output: 4
Explanation: The shortest amount of time it takes to go from intersection 0 to intersection 6 is 7
minutes.
The four ways to get there in 7 minutes are:
- 0 → 6
- 0 → 4 → 6
- 0 → 1 → 2 → 5 → 6
- 0 → 1 → 3 → 5 → 6
```

Example 2:

```
Input: n = 2, roads = [[1,0,10]]
Output: 1
Explanation: There is only one way to go from intersection 0 to intersection 1, and it takes 10
minutes.
```

Constraints:

- `1 <= n <= 200`
- `n - 1 <= roads.length <= n * (n - 1) / 2`
- `roads[i].length == 3`
- $0 \le u_i, v_i \le n - 1$
- $1 \le time_i \le 10^9$
- $u_i \ne v_i$
- There is at most one road connecting any two intersections.
- You can reach any intersection from any other intersection.

SOLUTION (PYTHON 3)

Solution 1: Naive Dijkstra Algorithm

How to think about it

We need both the shortest path and the number of such paths. n≤ 200 makes dense Dijkstra $O(n^2)$ acceptable. When a strictly better distance appears, copy the predecessor's count; when the distance ties, add it. The destination count is taken modulo 10^9+7.

We define the following arrays:

- `g` represents the adjacency matrix of the graph. `g[i][j]` represents the shortest path length from point `i` to point `j`. Initially, all are infinity, while `g[0][0]` is 0. Then we traverse `roads` and update `g[u][v]` and `g[v][u]` to `t`.
- `dist[i]` represents the shortest path length from the starting point to point `i`. Initially, all are infinity, while `dist[0]` is 0.
- `f[i]` represents the number of shortest paths from the starting point to point `i`. Initially, all are 0, while `f[0]` is 1.
- `vis[i]` represents whether point `i` has been visited. Initially, all are `False`.

Then, we use the naive Dijkstra algorithm to find the shortest path length from the starting point to the end point, and record the number of shortest paths for each point during the process.

Finally, we return `f[n - 1]`. Since the answer may be very large, we need to take the modulus of $10^9 + 7$.

The time complexity is $O(n^2)$, and the space complexity is $O(n^2)$, where n is the number of points.

Python 3

```
class Solution:
    def countPaths(self, n: int, roads: List[List[int]]) -> int:
        g = [[inf] * n for _ in range(n)]
        for u, v, t in roads:
            g[u][v] = g[v][u] = t
        g[0][0] = 0
        dist = [inf] * n
        dist[0] = 0
        f = [0] * n
        f[0] = 1
        vis = [False] * n
        for _ in range(n):
            t = -1
            for j in range(n):
                if not vis[j] and (t == -1 or dist[j] < dist[t]):
                    t = j
            vis[t] = True
            for j in range(n):
                if j == t:
                    continue
                ne = dist[t] + g[t][j]
                if dist[j] > ne:
                    dist[j] = ne
                    f[j] = f[t]
                elif dist[j] == ne:
                    f[j] += f[t]
        mod = 10**9 + 7
        return f[-1] % mod
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return one fastest path
# 2. Among the fastest paths, how many use the fewest
# roads


# ========================================================
# FOLLOW-UP 1: Return one fastest path
# ========================================================

# ---- follow-up 1: function ----
# Answer: Dijkstra, remembering the previous city that
# gave the best time; walk back from the destination.
# Time: O(E log V).  Space: O(V + E).
import heapq


def one_fastest_path(n, roads):
    graph = [[] for _ in range(n)]
    for a, b, t in roads:
        graph[a].append((b, t))
        graph[b].append((a, t))
    best = [float("inf")] * n
    previous = [-1] * n
    best[0] = 0
    heap = [(0, 0)]
    while heap:
        d, node = heapq.heappop(heap)
        if d > best[node]:
            continue
        for nxt, t in graph[node]:
            if d + t < best[nxt]:
                best[nxt] = d + t
                previous[nxt] = node
```

```
                heapq.heappush(heap, (d + t, nxt))
    path = []
    node = n - 1
    while node != -1:
        path.append(node)
        node = previous[node]
    return path[::-1]


# ============================================================
# FOLLOW-UP 2: Among the fastest paths, how many use the
# fewest roads
# ============================================================

# ---- follow-up 2: function ----
# Answer: Dijkstra that compares (time, roads) pairs and
# counts the ways that give the best pair.
# Time: O(E log V).  Space: O(V + E).
def fewest_roads_among_fastest(n, roads):
    mod = 10 ** 9 + 7
    graph = [[] for _ in range(n)]
    for a, b, t in roads:
        graph[a].append((b, t))
        graph[b].append((a, t))
    inf = float("inf")
    best = [(inf, inf)] * n
```

```
    ways = [0] * n
    best[0] = (0, 0)
    ways[0] = 1
    heap = [(0, 0, 0)]
    while heap:
        d, hops, node = heapq.heappop(heap)
        if (d, hops) > best[node]:
            continue
        for nxt, t in graph[node]:
            candidate = (d + t, hops + 1)
            if candidate < best[nxt]:
                best[nxt] = candidate
                ways[nxt] = ways[node]
                heapq.heappush(heap, (candidate[0], candidate[1], nxt))
            elif candidate == best[nxt]:
                ways[nxt] = (ways[nxt] + ways[node]) % mod
    return ways[n - 1]
```

1559. Detect Cycles in 2D Grid

Medium · Depth-First Search, Breadth-First Search, Union Find, Array, Matrix

leetcode.com/problems/detect-cycles-in-2d-grid

PROBLEM

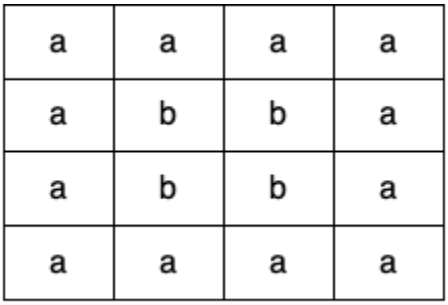
Given a 2D array of characters `grid` of size `m x n`, you need to find if there exists any cycle consisting of the **same value** in `grid`.

A cycle is a path of **length 4 or more** in the grid that starts and ends at the same cell. From a given cell, you can move to one of the cells adjacent to it - in one of the four directions (up, down, left, or right), if it has the **same value** of the current cell.

Also, you cannot move to the cell that you visited in your last move. For example, the cycle `(1, 1) -> (1, 2) -> (1, 1)` is invalid because from `(1, 2)` we visited `(1, 1)` which was the last visited cell.

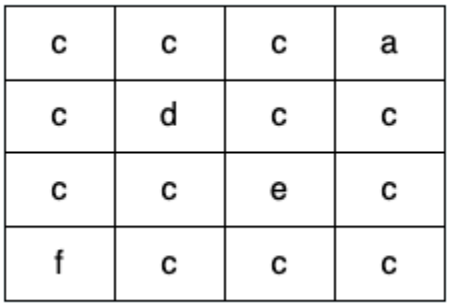
Return `true` if any cycle of the same value exists in `grid`, otherwise, return `false`.

Example 1:



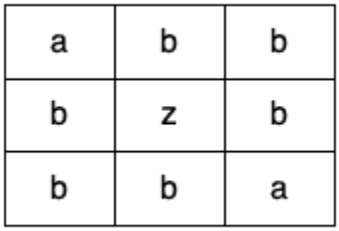

```
Input: grid = [["a","a","a","a"],["a","b","b","a"],["a","b","b","a"],["a","a","a","a"]]
Output: true
Explanation: There are two valid cycles shown in different colors in the image below:
```

Example 2:




```
Input: grid = [["c","c","c","a"],["c","d","c","c"],["c","c","e","c"],["f","c","c","c"]]
Output: true
Explanation: There is only one valid cycle highlighted in the image below:
```

Example 3:



```
Input: grid = [["a","b","b"],["b","z","b"],["b","b","a"]]
Output: false
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 500`
- `grid` consists only of lowercase English letters.

SOLUTIONS (PYTHON 3)

Solution 1: BFS

How to think about it

Detect a cycle of identical letters with length at least 4. The grid is small enough for one graph search, but stepping straight back to the parent must not count as a cycle. From every unvisited cell run BFS, walking only to same-letter neighbors and remembering the parent. A neighbor that is already visited and is not the parent closes a cycle. Each connected component is searched once.

We can traverse each cell in the 2D grid. For each cell, if the cell grid[i][j] has not been visited, we start a breadth-first search (BFS) from that cell. During the search, we need to record the parent node of each cell and the coordinates of the previous cell. If the value of the next cell is the same as the current cell, and it is not the previous cell, and it has already been visited, then it indicates the presence of a cycle, and we return true. After traversing all cells, if no cycle is found, we return false.

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the number of rows and columns of the 2D grid, respectively.

Python 3

```python
class Solution:
    def containsCycle(self, grid: List[List[str]]) -> bool:
        m, n = len(grid), len(grid[0])
        vis = [[False] * n for _ in range(m)]
        dirs = (-1, 0, 1, 0, -1)
        for i, row in enumerate(grid):
            for j, x in enumerate(row):
                if vis[i][j]:
                    continue
                vis[i][j] = True
                q = [(i, j, -1, -1)]
                while q:
                    x, y, px, py = q.pop()
                    for dx, dy in pairwise(dirs):
                        nx, ny = x + dx, y + dy
                        if 0 <= nx < m and 0 <= ny < n:
                            if grid[nx][ny] != grid[i][j] or (nx == px and ny == py):
                                continue
                            if vis[nx][ny]:
                                return True
                            vis[nx][ny] = True
                            q.append((nx, ny, x, y))
        return False
```

Solution 2: DFS

> **How to think about it**
>
> BFS needs an explicit queue. DFS uses the call stack for the same parent test and is shorter. The visited mark and the "visited non-parent means a cycle" rule stay unchanged; only the traversal order differs.

We can traverse each cell in the 2D grid. For each cell, if the cell grid[i][j] has not been visited, we start a depth-first search (DFS) from that cell. During the search, we need to record the parent node of each cell and the coordinates of the previous cell. If the value of the next cell is the same as the current cell, and it is not the previous cell, and it has already been visited, then it indicates the presence of a cycle, and we return true. After traversing all cells, if no cycle is found, we return false.

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the number of rows and columns of the 2D grid, respectively.

Python 3

```python
class Solution:
    def containsCycle(self, grid: List[List[str]]) -> bool:
        def dfs(x: int, y: int, px: int, py: int) -> bool:
            vis[x][y] = True
            for dx, dy in pairwise(dirs):
                nx, ny = x + dx, y + dy
                if 0 <= nx < m and 0 <= ny < n:
                    if grid[nx][ny] != grid[x][y] or (nx == px and ny == py):
                        continue
                    if vis[nx][ny] or dfs(nx, ny, x, y):
                        return True
            return False

        m, n = len(grid), len(grid[0])
        vis = [[False] * n for _ in range(m)]
        dirs = (-1, 0, 1, 0, -1)
        for i in range(m):
            for j in range(n):
                if vis[i][j]:
                    continue
                if dfs(i, j, -1, -1):
                    return True
        return False
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the cells of one cycle
# 2. Count the regions of equal letters that contain a
# cycle


# ============================================================
# FOLLOW-UP 1: Return the cells of one cycle
# ============================================================

# ---- follow-up 1: function ----
# Answer: Depth-first search with the parent cell
# remembered; reaching a visited cell that is not the
# parent closes a cycle, which is read back through the
# parent links.
# Time: O(R * C).  Space: O(R * C).
def find_one_cycle(grid):
    rows, cols = len(grid), len(grid[0])
    parent = {}
    for r0 in range(rows):
        for c0 in range(cols):
            if (r0, c0) in parent:
                continue
```

```
            parent[(r0, c0)] = None
            stack = [(r0, c0)]
            while stack:
                r, c = stack.pop()
                for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                    nr, nc = r + dr, c + dc
                    if not (0 <= nr < rows and 0 <= nc < cols) or grid[nr][nc] != grid[r][c]:
                        continue
                    if parent[(r, c)] == (nr, nc):
                        continue
                    if (nr, nc) in parent:
                        left = [(r, c)]
                        while parent[left[-1]] is not None:
                            left.append(parent[left[-1]])
                        right = [(nr, nc)]
                        while parent[right[-1]] is not None:
                            right.append(parent[right[-1]])
                        while left and right and left[-1] == right[-1]:
                            joint = left.pop()
                            right.pop()
                        return left + [joint] + right[::-1]
                    parent[(nr, nc)] = (r, c)
                    stack.append((nr, nc))
    return None


# ============================================================
# FOLLOW-UP 2: Count the regions of equal letters that
# contain a cycle
# ============================================================

# ---- follow-up 2: function ----
# Answer: Union-find the equal neighbours; a region has a
# cycle when some union joins two cells already in the
# same set.
# Time: O(R * C * alpha(R * C)).  Space: O(R * C).
def regions_with_cycle(grid):
```

```
    rows, cols = len(grid), len(grid[0])
    parent = list(range(rows * cols))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    cyclic_roots = set()
    for r in range(rows):
        for c in range(cols):
            for dr, dc in ((1, 0), (0, 1)):
                nr, nc = r + dr, c + dc
                if nr < rows and nc < cols and grid[nr][nc] == grid[r][c]:
                    a, b = find(r * cols + c), find(nr * cols + nc)
                    if a == b:
                        cyclic_roots.add(a)
                    else:
                        parent[a] = b
                        if a in cyclic_roots:
                            cyclic_roots.discard(a)
                            cyclic_roots.add(b)
    return len(set(find(x) for x in cyclic_roots))
```

2050. Parallel Courses III

Hard · Graph, Topological Sort, Array, Dynamic Programming, Directed Acyclic Graph

leetcode.com/problems/parallel-courses-iii

PROBLEM

You are given an integer `n`, which indicates that there are `n` courses labeled from `1` to `n`. You are also given a 2D integer array `relations` where `relations[j] = [prevCourse`$_j$`, nextCourse`$_j$`]` denotes that course `prevCourse`$_j$ has to be completed **before** course `nextCourse`$_j$ (prerequisite relationship). Furthermore, you are given a **0-indexed** integer array `time` where `time[i]` denotes how many **months** it takes to complete the `(i+1)`th course.

You must find the **minimum** number of months needed to complete all the courses following these rules:

- You may start taking a course at **any time** if the prerequisites are met.
- **Any number of courses** can be taken at the **same time**.

Return *the **minimum** number of months needed to complete all the courses*.

Note: The test cases are generated such that it is possible to complete every course (i.e., the graph is a directed acyclic graph).

Example 1:

```
Input: n = 3, relations = [[1,3],[2,3]], time = [3,2,5]
Output: 8
Explanation: The figure above represents the given graph and the time required to complete each
course.
We start course 1 and course 2 simultaneously at month 0.
Course 1 takes 3 months and course 2 takes 2 months to complete respectively.
Thus, the earliest time we can start course 3 is at month 3, and the total time required is 3 + 5 = 8
months.
```

Example 2:

```
Input: n = 5, relations = [[1,5],[2,5],[3,5],[3,4],[4,5]], time = [1,2,3,4,5]
Output: 12
Explanation: The figure above represents the given graph and the time required to complete each
course.
You can start courses 1, 2, and 3 at month 0.
You can complete them after 1, 2, and 3 months respectively.
Course 4 can be taken only after course 3 is completed, i.e., after 3 months. It is completed after 3 +
4 = 7 months.
Course 5 can be taken only after courses 1, 2, 3, and 4 have been completed, i.e., after max(1,2,3,7) =
7 months.
Thus, the minimum time needed to complete all the courses is 7 + 5 = 12 months.
```

Constraints:

- `1 <= n <= 5 * 10`^{4}
- `0 <= relations.length <= min(n * (n - 1) / 2, 5 * 10`^{4}`)`
- `relations[j].length == 2`
- `1 <= prevCourse`$_j$`, nextCourse`$_j$` <= n`
- `prevCourse`$_j$` != nextCourse`$_j$
- All the pairs `[prevCourse`$_j$`, nextCourse`$_j$`]` are **unique**.
- `time.length == n`
- `1 <= time[i] <= 10`^{4}
- The given graph is a directed acyclic graph.

SOLUTION (PYTHON 3)

Solution 1: Topological Sorting + Dynamic Programming

> **How to think about it**
>
> Courses have prerequisites and durations; the finish time is the heaviest path in the DAG. With $n \le 5 \times 10^4$ we cannot list paths. Sources start immediately and finish in their own `time`. In topological order, f[j] = max(f[j], f[i]+time[j]) once every predecessor i is done. The answer is the maximum f. A queue of zero in-degree nodes updates each course only after all prerequisites.

First, we construct a directed acyclic graph based on the given prerequisite course relationships, perform topological sorting on this graph, and then use dynamic programming to find the minimum time required to complete all courses according to the results of the topological sorting.

We define the following data structures or variables:

- Adjacency list g stores the directed acyclic graph, and an array indeg stores the in-degree of each node;
- Queue q stores all nodes with an in-degree of 0;
- Array f stores the earliest completion time of each node, initially f[i] = 0;
- Variable ans records the final answer, initially ans = 0;

When q is not empty, take out the head node i in turn, traverse each node j in g[i], update f[j] = max(f[j], f[i] + time[j]), update ans = max(ans, f[j]) at the same time, and reduce the in-degree of j by 1. If the in-degree of j is 0 at this time, add j to the queue q;

Finally, return ans.

The time complexity is O(m + n), and the space complexity is O(m + n). Where m is the length of the array relations.

Python 3

```python
class Solution:
    def minimumTime(self, n: int, relations: List[List[int]], time: List[int]) -> int:
        g = defaultdict(list)
        indeg = [0] * n
        for a, b in relations:
            g[a - 1].append(b - 1)
            indeg[b - 1] += 1
        q = deque()
        f = [0] * n
        ans = 0
        for i, (v, t) in enumerate(zip(indeg, time)):
            if v == 0:
                q.append(i)
                f[i] = t
                ans = max(ans, t)
        while q:
            i = q.popleft()
            for j in g[i]:
                f[j] = max(f[j], f[i] + time[j])
                ans = max(ans, f[j])
                indeg[j] -= 1
                if indeg[j] == 0:
                    q.append(j)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the courses on the critical path
# 2. How long can each course be delayed without delaying
# the whole degree (slack)


# ============================================================
# FOLLOW-UP 1: Return the courses on the critical path
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember which prerequisite finished last for
# every course and walk back from the course that finishes
# last.
# Time: O(n + m).  Space: O(n + m).
def critical_path(n, relations, time):
    waiting = [0] * (n + 1)
    unlocks = [[] for _ in range(n + 1)]
    for a, b in relations:
        waiting[b] += 1
        unlocks[a].append(b)
    finish = [0] * (n + 1)
    last_prerequisite = [0] * (n + 1)
    queue = []
    for c in range(1, n + 1):
        finish[c] = time[c - 1]
        if waiting[c] == 0:
            queue.append(c)
    for c in queue:
        for nxt in unlocks[c]:
            if finish[c] + time[nxt - 1] > finish[nxt]:
                finish[nxt] = finish[c] + time[nxt - 1]
                last_prerequisite[nxt] = c
            waiting[nxt] -= 1
            if waiting[nxt] == 0:
                queue.append(nxt)
    end = 1
    for c in range(1, n + 1):
        if finish[c] > finish[end]:
            end = c
    path = []
    while end != 0:
        path.append(end)
        end = last_prerequisite[end]
    return path[::-1]


# ============================================================
# FOLLOW-UP 2: How long can each course be delayed without
# delaying the whole degree (slack)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Compute the earliest start going forward and the
# latest start going backward; the slack is their
# difference.
# Time: O(n + m).  Space: O(n + m).
def slack_of_each_course(n, relations, time):
    waiting = [0] * (n + 1)
    unlocks = [[] for _ in range(n + 1)]
```

```
    for a, b in relations:
        waiting[b] += 1
        unlocks[a].append(b)
    earliest = [0] * (n + 1)
```

```
    order = [c for c in range(1, n + 1) if waiting[c] == 0]
    for c in order:
        for nxt in unlocks[c]:
            earliest[nxt] = max(earliest[nxt], earliest[c] + time[c - 1])
            waiting[nxt] -= 1
            if waiting[nxt] == 0:
                order.append(nxt)
    total = 0
    for c in range(1, n + 1):
        total = max(total, earliest[c] + time[c - 1])
    latest = [0] * (n + 1)
    for c in reversed(order):
        latest[c] = total - time[c - 1]
        for nxt in unlocks[c]:
            latest[c] = min(latest[c], latest[nxt] - time[c - 1])
    return [latest[c] - earliest[c] for c in range(1, n + 1)]
```

1203. Sort Items by Groups Respecting Dependencies

Hard · Depth-First Search, Breadth-First Search, Graph, Topological Sort, Directed Acyclic Graph

leetcode.com/problems/sort-items-by-groups-respecting-dependencies

PROBLEM

There are `n` items each belonging to zero or one of `m` groups where `group[i]` is the group that the `i`-th item belongs to and it's equal to `-1` if the `i`-th item belongs to no group. The items and the groups are zero indexed. A group can have no item belonging to it.

Return a sorted list of the items such that:

- The items that belong to the same group are next to each other in the sorted list.
- There are some relations between these items where `beforeItems[i]` is a list containing all the items that should come before the `i`-th item in the sorted array (to the left of the `i`-th item).

Return any solution if there is more than one solution and return an **empty list** if there is no solution.

Example 1:

Item	Group	Before
0	-1	
1	-1	6
2	1	5
3	0	6
4	0	3, 6
5	1	
6	0	
7	-1	

```
Input: n = 8, m = 2, group = [-1,-1,1,0,0,1,0,-1], beforeItems = [[],[6],[5],[6],[3,6],[],[],[]]
Output: [6,3,4,1,5,2,0,7]
```

Example 2:

```
Input: n = 8, m = 2, group = [-1,-1,1,0,0,1,0,-1], beforeItems = [[],[6],[5],[6],[3],[],[4],[]]
Output: []
Explanation: This is the same as example 1 except that 4 needs to be before 6 in the sorted list.
```

Constraints:

- `1 <= m <= n <= 3 * 10^4`
- `group.length == beforeItems.length == n`
- `-1 <= group[i] <= m - 1`
- `0 <= beforeItems[i].length <= n - 1`
- `0 <= beforeItems[i][j] <= n - 1`
- `i != beforeItems[i][j]`
- `beforeItems[i]` does not contain duplicates elements.

SOLUTION (PYTHON 3)

Solution 1: Topological Sorting

How to think about it

A single topological sort on items can split a group, so items from the same group need not stay together. With $n \le 3 \times 10^4$ we need linear-time graph construction. Constraints sit on two layers: groups precede other groups, and items precede other items inside a group. Ungrouped items become singleton groups. Cross-group dependencies become group edges; same-group dependencies become item edges. We topologically sort groups first, then items inside each group in that order. A cycle at either layer means no answer. The two graphs separate “groups stay contiguous” from “item order”, each handled by Kahn's algorithm.

First, we traverse the array group. For each project, if it does not belong to any group, we create a new group for it with the ID m, and increment m. This ensures that all projects belong to some group. Then, we use an array groupItems to record the projects contained in each group. The array index is the group ID, and the array value is the list of projects in the group.

Next, we need to build the graph. For each project, we need to build two types of graphs: one for the projects and one for the groups. We traverse the array group. For the current project i, its group is group[i]. We traverse beforeItems[i], and for each project j in it, if group[i] = group[j], it means that i and j belong to the same group. We add an edge j → i in the project graph. Otherwise, it means that i and j belong to different groups. We add an edge group[j] → group[i] in the group graph, and update the corresponding in-degree array.

Next, we perform topological sorting on the group graph to get the sorted group list groupOrder. If the length of the sorted list is not equal to the total number of groups, it means that the sorting cannot be completed, so we return an empty list. Otherwise, we traverse groupOrder, and for each group gi, we perform topological sorting on the project list groupItems[gi] to get the sorted project list itemOrder. If the length of the sorted list is not equal to the total number of projects in the group, it means that the sorting cannot be completed, so we return an empty list. Otherwise, we add the projects in itemOrder to the answer array, and finally return this answer array.

The time complexity is O(n + m), and the space complexity is O(n + m). Here, n and m are the total number of projects and groups, respectively.

Python 3

```python
class Solution:
    def sortItems(
        self, n: int, m: int, group: List[int], beforeItems: List[List[int]]
    ) -> List[int]:
        def topo_sort(degree, graph, items):
            q = deque(i for _, i in enumerate(items) if degree[i] == 0)
            res = []
            while q:
                i = q.popleft()
                res.append(i)
                for j in graph[i]:
                    degree[j] -= 1
                    if degree[j] == 0:
                        q.append(j)
            return res if len(res) == len(items) else []

        idx = m
        group_items = [[] for _ in range(n + m)]
        for i, g in enumerate(group):
            if g == -1:
                group[i] = idx
                idx += 1
            group_items[group[i]].append(i)

        item_degree = [0] * n
        group_degree = [0] * (n + m)
        item_graph = [[] for _ in range(n)]
        group_graph = [[] for _ in range(n + m)]
        for i, gi in enumerate(group):
            for j in beforeItems[i]:
                gj = group[j]
                if gi == gj:
                    item_degree[i] += 1
                    item_graph[j].append(i)
                else:
                    group_degree[gi] += 1
                    group_graph[gj].append(gi)

        group_order = topo_sort(group_degree, group_graph, range(n + m))
        if not group_order:
            return []
        ans = []
        for gi in group_order:
            items = group_items[gi]
            item_order = topo_sort(item_degree, item_graph, items)
            if len(items) != len(item_order):
                return []
            ans.extend(item_order)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Check that a given order of items is valid
# 2. Return only the order of the groups


# ============================================================
# FOLLOW-UP 1: Check that a given order of items is valid
# ============================================================

# ---- follow-up 1: function ----
# Answer: Every item must come after its before-items, and
# each group's items must sit next to each other.
# Time: O(n + D), D = total dependencies.  Space: O(n).
```

```
def is_valid_item_order(order, group, before_items):
    position = {}
    for i in range(len(order)):
        position[order[i]] = i
    for item in range(len(order)):
        for other in before_items[item]:
            if position[other] > position[item]:
                return False
    seen_groups = set()
    previous = None
    for item in order:
        g = group[item]
        if g == -1:
            previous = None
            continue
        if g != previous and g in seen_groups:
            return False
        seen_groups.add(g)
        previous = g
    return True


# ========================================================
# FOLLOW-UP 2: Return only the order of the groups
# ========================================================

# ---- follow-up 2: function ----
# Answer: Give every item with no group its own group, add
# an edge between groups for every dependency across
# groups, and sort the groups.
# Time: O(n + D).  Space: O(n + D).
def order_of_groups(n, m, group, before_items):
    group = list(group)
    total = m
    for i in range(n):
        if group[i] == -1:
            group[i] = total
            total += 1
    waiting = [0] * total
    unlocks = [set() for _ in range(total)]
    for item in range(n):
        for other in before_items[item]:
            if group[other] != group[item] and group[item] not in unlocks[group[other]]:
                unlocks[group[other]].add(group[item])
                waiting[group[item]] += 1
    queue = [g for g in range(total) if waiting[g] == 0]
    for g in queue:
```

```
        for nxt in unlocks[g]:
            waiting[nxt] -= 1
            if waiting[nxt] == 0:
                queue.append(nxt)
    if len(queue) < total:
        return []
    return queue
```

2392. Build a Matrix With Conditions

Hard · Graph, Topological Sort, Array, Directed Acyclic Graph, Matrix

leetcode.com/problems/build-a-matrix-with-conditions

PROBLEM

You are given a **positive** integer `k`. You are also given:

- a 2D integer array `rowConditions` of size `n` where `rowConditions[i] = [above`$_i$`, below`$_i$`]`, and
- a 2D integer array `colConditions` of size `m` where `colConditions[i] = [left`$_i$`, right`$_i$`]`.

The two arrays contain integers from `1` to `k`.

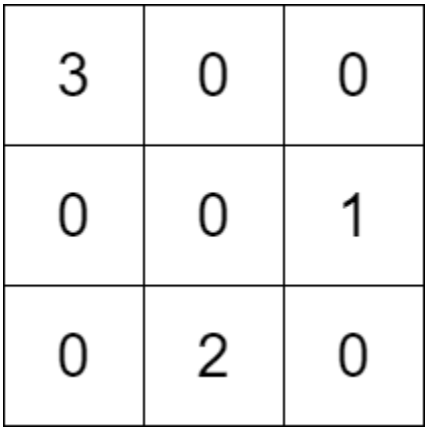
You have to build a `k x k` matrix that contains each of the numbers from `1` to `k` **exactly once**. The remaining cells should have the value `0`.

The matrix should also satisfy the following conditions:

- The number `above`$_i$ should appear in a **row** that is strictly **above** the row at which the number `below`$_i$ appears for all `i` from `0` to `n - 1`.
- The number `left`$_i$ should appear in a **column** that is strictly **left** of the column at which the number `right`$_i$ appears for all `i` from `0` to `m - 1`.

Return ***any** matrix that satisfies the conditions*. If no answer exists, return an empty matrix.

Example 1:




```
Input: k = 3, rowConditions = [[1,2],[3,2]], colConditions = [[2,1],[3,2]]
Output: [[3,0,0],[0,0,1],[0,2,0]]
Explanation: The diagram above shows a valid example of a matrix that satisfies all the conditions.
The row conditions are the following:
- Number 1 is in row 1, and number 2 is in row 2, so 1 is above 2 in the matrix.
- Number 3 is in row 0, and number 2 is in row 2, so 3 is above 2 in the matrix.
The column conditions are the following:
- Number 2 is in column 1, and number 1 is in column 2, so 2 is left of 1 in the matrix.
- Number 3 is in column 0, and number 2 is in column 1, so 3 is left of 2 in the matrix.
Note that there may be multiple correct answers.
```

Example 2:

```
Input: k = 3, rowConditions = [[1,2],[2,3],[3,1],[2,3]], colConditions = [[2,1]]
Output: []
Explanation: From the first two conditions, 3 has to be below 1 but the third conditions needs 3 to be
above 1 to be satisfied.
No matrix can satisfy all the conditions, so we return the empty matrix.
```

Constraints:

- $2 <= k <= 400$
- $1 <= rowConditions.length, colConditions.length <= 10^4$
- $rowConditions[i].length == colConditions[i].length == 2$
- $1 <= above_i, below_i, left_i, right_i <= k$
- $above_i != below_i$
- $left_i != right_i$

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Each of 1..k appears once; row and column conditions are order constraints. k ≤ 400, so treat them as directed edges and topological-sort. A cycle means impossible. Topo-sort rows and columns separately, then write each value at the paired indices. If either order is shorter than k, return an empty matrix.

Python 3

```
class Solution:
    def buildMatrix(
        self, k: int, rowConditions: List[List[int]], colConditions: List[List[int]]
    ) -> List[List[int]]:
        def f(cond):
            g = defaultdict(list)
            indeg = [0] * (k + 1)
            for a, b in cond:
                g[a].append(b)
                indeg[b] += 1
            q = deque([i for i, v in enumerate(indeg[1:], 1) if v == 0])
            res = []
            while q:
                for _ in range(len(q)):
                    i = q.popleft()
                    res.append(i)
                    for j in g[i]:
                        indeg[j] -= 1
                        if indeg[j] == 0:
                            q.append(j)
            return None if len(res) != k else res

        row = f(rowConditions)
        col = f(colConditions)
        if row is None or col is None:
            return []
        ans = [[0] * k for _ in range(k)]
        m = [0] * (k + 1)
        for i, v in enumerate(col):
            m[v] = i
        for i, v in enumerate(row):
            ans[i][m[v]] = v
        return ans
```

Complexity. Time: O(k^2 + r + c), r and c = number of row and column conditions. Space: O(k^2 + r + c).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Only tell whether a matrix exists
# 2. Let numbers share a row or column: use the fewest
# rows and columns


# ============================================================
# FOLLOW-UP 1: Only tell whether a matrix exists
# ============================================================

# ---- follow-up 1: function ----
# Answer: A matrix exists exactly when neither the row
# conditions nor the column conditions contain a cycle.
# Time: O(k + R + C), R, C = row and column conditions.
# Space: O(k + R + C).
def matrix_exists(k, row_conditions, col_conditions):
    def acyclic(conditions):
        waiting = [0] * (k + 1)
        unlocks = [[] for _ in range(k + 1)]
        for a, b in conditions:
            waiting[b] += 1
            unlocks[a].append(b)
        queue = [x for x in range(1, k + 1) if waiting[x] == 0]
        for x in queue:
            for nxt in unlocks[x]:
                waiting[nxt] -= 1
                if waiting[nxt] == 0:
                    queue.append(nxt)
        return len(queue) == k

    return acyclic(row_conditions) and acyclic(col_conditions)


# ============================================================
# FOLLOW-UP 2: Let numbers share a row or column: use the
# fewest rows and columns
# ============================================================

# ---- follow-up 2: function ----
# Answer: Give every number its level in the topological
# order (longest chain before it); the number of levels is
# the number of rows (or columns) needed.
# Time: O(k + conditions).  Space: O(k + conditions).
def levels_needed(k, conditions):
    waiting = [0] * (k + 1)
    unlocks = [[] for _ in range(k + 1)]
    for a, b in conditions:
        waiting[b] += 1
        unlocks[a].append(b)
    level = [0] * (k + 1)
    queue = [x for x in range(1, k + 1) if waiting[x] == 0]
    for x in queue:
        for nxt in unlocks[x]:
            level[nxt] = max(level[nxt], level[x] + 1)
            waiting[nxt] -= 1
            if waiting[nxt] == 0:
                queue.append(nxt)
    if len(queue) < k:
        return None
    return level[1:], max(level[1:]) + 1
```

1857. Largest Color Value in a Directed Graph

Hard · Graph, Topological Sort, Memoization, Hash Table, String, Dynamic Programming, Counting, Directed Acyclic Graph

leetcode.com/problems/largest-color-value-in-a-directed-graph

★ **TOP 7 PROBLEM** in Topological Sort: master this one first

PROBLEM

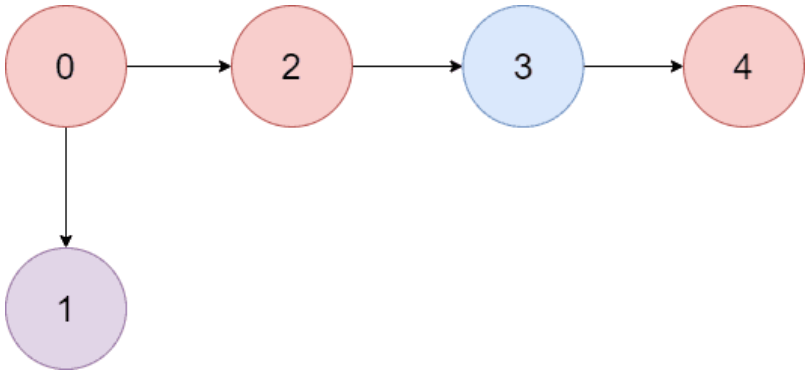
There is a **directed graph** of `n` colored nodes and `m` edges. The nodes are numbered from `0` to `n - 1`.

You are given a string `colors` where `colors[i]` is a lowercase English letter representing the **color** of the i^{th} node in this graph (**0-indexed**). You are also given a 2D array `edges` where `edges[j] = [a_j, b_j]` indicates that there is a **directed edge** from node a_j to node b_j.

A valid **path** in the graph is a sequence of nodes $x_1 \to x_2 \to x_3 \to \ldots \to x_k$ such that there is a directed edge from x_i to x_{i+1} for every $1 \le i < k$. The **color value** of the path is the number of nodes that are colored the **most frequently** occurring color along that path.

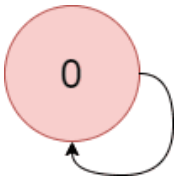
Return *the **largest color value** of any valid path in the given graph, or* `-1` *if the graph contains a cycle*.

Example 1:




```
Input: colors = "abaca", edges = [[0,1],[0,2],[2,3],[3,4]]
Output: 3
Explanation: The path 0 -> 2 -> 3 -> 4 contains 3 nodes that are colored "a" (red in the above image).
```

Example 2:




```
Input: colors = "a", edges = [[0,0]]
Output: -1
Explanation: There is a cycle from 0 to 0.
```

Constraints:

- `n == colors.length`
- `m == edges.length`
- `1 <= n <= 10^5`
- `0 <= m <= 10^5`
- `colors` consists of lowercase English letters.
- `0 <= a_j, b_j < n`

SOLUTION (PYTHON 3)

Solution 1: Topological Sort + Dynamic Programming

How to think about it

We need the maximum count of any single color on a directed path, or -1 if a cycle exists. There are exponentially many paths and $n,m \leq 10^5$. A topological order processes a node after all predecessors. dp[i][c] is the best count of color c on a path ending at i; we take a coordinate-wise maximum over in-edges and add the node's own color. If fewer than n nodes leave the queue, the graph has a cycle.

Calculate the in-degree of each node and perform a topological sort.

Define a 2D array dp, where dp[i][j] represents the number of nodes with color j on the path from the start node to node i.

From node i, traverse all outgoing edges i → j, and update dp[j][k] = max(dp[j][k], dp[i][k] + (c == k)), where c is the color of node j.

The answer is the maximum value in the dp array.

If there is a cycle in the graph, it is impossible to visit all nodes, so return -1.

The time complexity is O((n + m) × |Σ|), and the space complexity is O(m + n × |Σ|). Here, |Σ| is the size of the alphabet (26 in this case), and n and m are the number of nodes and edges, respectively.

Python 3

```python
class Solution:
    def largestPathValue(self, colors: str, edges: List[List[int]]) -> int:
        n = len(colors)
        indeg = [0] * n
        g = defaultdict(list)
        for a, b in edges:
            g[a].append(b)
            indeg[b] += 1
        q = deque()
        dp = [[0] * 26 for _ in range(n)]
        for i, v in enumerate(indeg):
            if v == 0:
                q.append(i)
                c = ord(colors[i]) - ord('a')
                dp[i][c] += 1
        cnt = 0
        ans = 1
        while q:
            i = q.popleft()
            cnt += 1
            for j in g[i]:
                indeg[j] -= 1
                if indeg[j] == 0:
                    q.append(j)
                c = ord(colors[j]) - ord('a')
                for k in range(26):
                    dp[j][k] = max(dp[j][k], dp[i][k] + (c == k))
                    ans = max(ans, dp[j][k])
        return -1 if cnt < n else ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Which colour reaches the largest value
# 2. Only paths that start at a given node


# ============================================================
# FOLLOW-UP 1: Which colour reaches the largest value
# ============================================================

# ---- follow-up 1: function ----
# Answer: Run the same table and remember the colour that
# gives the biggest count anywhere.
# Time: O(26 * (n + m)).  Space: O(26 * n).
def best_colour(colors, edges):
    n = len(colors)
    waiting = [0] * n
    unlocks = [[] for _ in range(n)]
    for a, b in edges:
        waiting[b] += 1
        unlocks[a].append(b)
    count = [[0] * 26 for _ in range(n)]
    queue = [i for i in range(n) if waiting[i] == 0]
    seen = 0
    best_value = 0
    best_letter = None
    for node in queue:
        seen += 1
        count[node][ord(colors[node]) - 97] += 1
        for letter in range(26):
            if count[node][letter] > best_value:
                best_value = count[node][letter]
                best_letter = chr(97 + letter)
        for nxt in unlocks[node]:
            for letter in range(26):
                count[nxt][letter] = max(count[nxt][letter], count[node][letter])
            waiting[nxt] -= 1
            if waiting[nxt] == 0:
                queue.append(nxt)
    if seen < n:
        return None
    return best_letter, best_value


# ============================================================
# FOLLOW-UP 2: Only paths that start at a given node
# ============================================================

# ---- follow-up 2: function ----
# Answer: Run the table only over the nodes reachable from
# the start.
# Time: O(26 * (n + m)).  Space: O(26 * n).
def largest_value_from(colors, edges, start):
    n = len(colors)
    unlocks = [[] for _ in range(n)]
    for a, b in edges:
        unlocks[a].append(b)
    reachable = {start}
    stack = [start]
    while stack:
        node = stack.pop()
```

```
        for nxt in unlocks[node]:
            if nxt not in reachable:
                reachable.add(nxt)
                stack.append(nxt)
```

```
    waiting = {node: 0 for node in reachable}
    for a, b in edges:
        if a in reachable:
            waiting[b] += 1
    if waiting[start] > 0:
        return -1
    count = {node: [0] * 26 for node in reachable}
    queue = [start]
    best = 0
    done = 0
    for node in queue:
        done += 1
        count[node][ord(colors[node]) - 97] += 1
        best = max(best, max(count[node]))
        for nxt in unlocks[node]:
            for letter in range(26):
                count[nxt][letter] = max(count[nxt][letter], count[node][letter])
            waiting[nxt] -= 1
            if waiting[nxt] == 0:
                queue.append(nxt)
    if done < len(reachable):
        return -1
    return best
```

2127. Maximum Employees to Be Invited to a Meeting

Hard · Depth-First Search, Graph, Topological Sort, Array, Dynamic Programming, Kosaraju, Tarjan

leetcode.com/problems/maximum-employees-to-be-invited-to-a-meeting

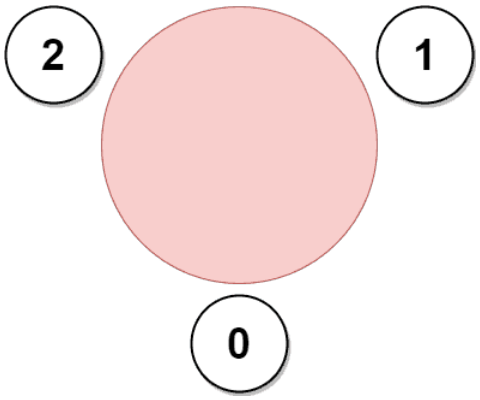
PROBLEM

A company is organizing a meeting and has a list of `n` employees, waiting to be invited. They have arranged for a large **circular** table, capable of seating **any number** of employees.

The employees are numbered from `0` to `n - 1`. Each employee has a **favorite** person and they will attend the meeting **only if** they can sit next to their favorite person at the table. The favorite person of an employee is **not** themself.

Given a **0-indexed** integer array `favorite`, where `favorite[i]` denotes the favorite person of the i^{th} employee, return *the **maximum number of employees** that can be invited to the meeting*.

Example 1:

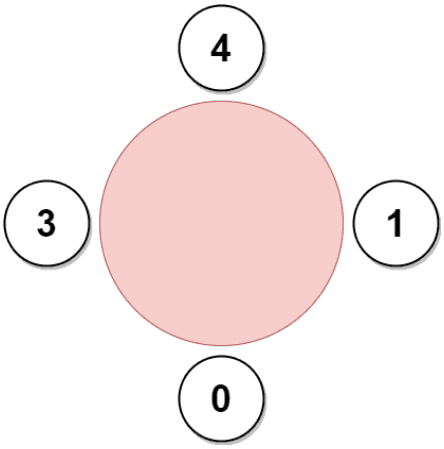



```
Input: favorite = [2,2,1,2]
Output: 3
Explanation:
The above figure shows how the company can invite employees 0, 1, and 2, and seat them at the round
table.
All employees cannot be invited because employee 2 cannot sit beside employees 0, 1, and 3,
simultaneously.
Note that the company can also invite employees 1, 2, and 3, and give them their desired seats.
The maximum number of employees that can be invited to the meeting is 3.
```

Example 2:

```
Input: favorite = [1,2,0]
Output: 3
Explanation:
Each employee is the favorite person of at least one other employee, and the only way the company can
invite them is if they invite every employee.
The seating arrangement will be the same as that in the figure given in example 1:
- Employee 0 will sit between employees 2 and 1.
- Employee 1 will sit between employees 0 and 2.
- Employee 2 will sit between employees 1 and 0.
The maximum number of employees that can be invited to the meeting is 3.
```

Example 3:




```
Input: favorite = [3,0,1,4,1]
Output: 4
Explanation:
The above figure shows how the company will invite employees 0, 1, 3, and 4, and seat them at the round table.
Employee 2 cannot be invited because the two spots next to their favorite employee 1 are taken.
So the company leaves them out of the meeting.
The maximum number of employees that can be invited to the meeting is 4.
```

Constraints:

- `n == favorite.length`
- `2 <= n <= 10^5`
- `0 <= favorite[i] <= n - 1`
- `favorite[i] != i`

SOLUTION (PYTHON 3)

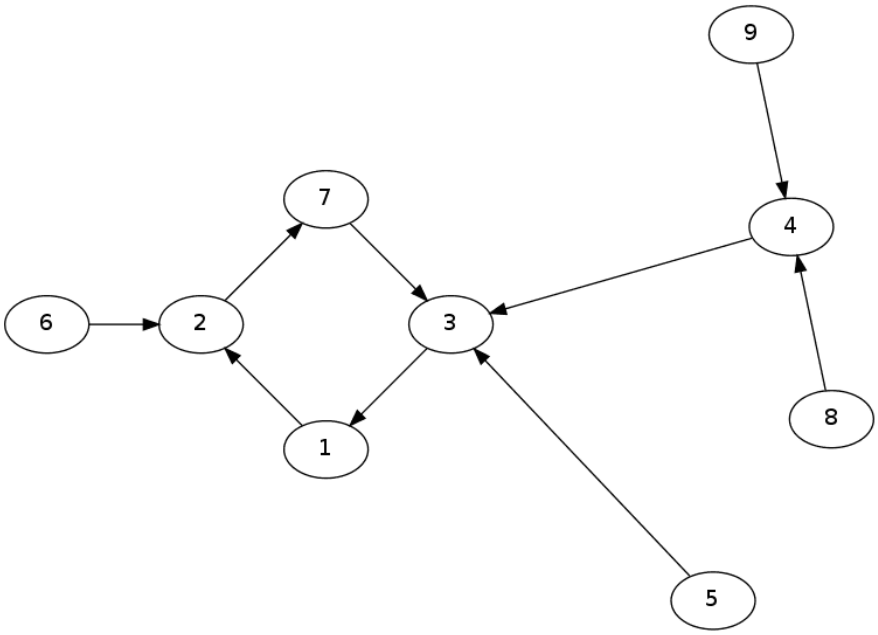
Solution 1: Maximum Cycle in Graph + Longest Chain

How to think about it

Each employee likes exactly one other person, so the graph is a disjoint union of inward rooted cycle-trees. A circular table requires mutual likes between neighbors: either a single cycle, or length-2 mutual pairs with incoming chains attached. Distinct cycles cannot be mixed. A cycle of length at least 3 can contribute only itself, so one candidate is the longest cycle. All 2-cycles may sit together, each extended by the longest incoming chain, which topological peeling computes as distances. We therefore take the maximum of the longest cycle and the sum of distances on all mutual pairs.

We observe that the employee's preference relationship in the problem can be regarded as a directed graph, which can be divided into multiple "base cycle inward trees". Each structure contains a cycle, and each node on the cycle is connected to a tree.

What is a "base cycle inward tree"? First, a base cycle tree is a directed graph with n nodes and n edges, and an inward tree means that in this directed graph, each node has exactly one outgoing edge. In this problem, each employee has exactly one favorite employee, so the constructed directed graph can be composed of multiple "base cycle inward trees".



For this problem, we can find the length of the maximum cycle in the graph. Here we only need to find the length of the largest cycle, because if there are multiple cycles, they are not connected to each other, which does not meet the problem requirements.

In addition, for the size of the cycle equal to 2, that is, there are two employees who like each other, then we can arrange these two employees together. If these two employees are each liked by other employees, then we only need to arrange the employees who like them next to them. If there are multiple such situations, we can arrange them all.

Therefore, the problem is actually equivalent to finding the length of the maximum cycle in the graph, and all cycles of length 2 plus their longest chain. The maximum of these two can be found. To find the longest chain to the cycle of length 2, we can use topological sorting.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array `favorite`.

Python 3

```python
class Solution:
    def maximumInvitations(self, favorite: List[int]) -> int:
        def max_cycle(fa: List[int]) -> int:
            n = len(fa)
            vis = [False] * n
            ans = 0
            for i in range(n):
                if vis[i]:
                    continue
                cycle = []
                j = i
                while not vis[j]:
                    cycle.append(j)
                    vis[j] = True
                    j = fa[j]
                for k, v in enumerate(cycle):
                    if v == j:
                        ans = max(ans, len(cycle) - k)
                        break
            return ans

        def topological_sort(fa: List[int]) -> int:
            n = len(fa)
            indeg = [0] * n
            dist = [1] * n
            for v in fa:
                indeg[v] += 1
            q = deque(i for i, v in enumerate(indeg) if v == 0)
            while q:
                i = q.popleft()
                dist[fa[i]] = max(dist[fa[i]], dist[i] + 1)
                indeg[fa[i]] -= 1
                if indeg[fa[i]] == 0:
                    q.append(fa[i])
            return sum(dist[i] for i, v in enumerate(fa) if i == fa[fa[i]])

        return max(max_cycle(favorite), topological_sort(favorite))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Only the biggest circle (a single cycle, no chains)
# 2. List the circles (all cycles of the favourite graph)


# ============================================================
# FOLLOW-UP 1: Only the biggest circle (a single cycle, no
# chains)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Walk the functional graph and take the longest
# cycle.
# Time: O(n).  Space: O(n).
def biggest_circle(favorite):
    n = len(favorite)
    state = [0] * n                       # 0 new, 1 on the current walk, 2 finished
    best = 0
    for start in range(n):
        if state[start] != 0:
            continue
        walk = []
        node = start
        while state[node] == 0:
            state[node] = 1
```

```
            walk.append(node)
            node = favorite[node]
        if state[node] == 1:
            best = max(best, len(walk) - walk.index(node))
        for x in walk:
            state[x] = 2
    return best


# ============================================================
# FOLLOW-UP 2: List the circles (all cycles of the
# favourite graph)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same walk; every time the walk runs into itself,
# the part from the repeated node on is one circle.
# Time: O(n).  Space: O(n).
def all_circles(favorite):
    n = len(favorite)
    state = [0] * n
    circles = []
    for start in range(n):
        if state[start] != 0:
            continue
        walk = []
        node = start
        while state[node] == 0:
            state[node] = 1
            walk.append(node)
            node = favorite[node]
        if state[node] == 1:
            circles.append(walk[walk.index(node):])
        for x in walk:
            state[x] = 2
```

```
    return circles
```

1591. Strange Printer II

Hard · Graph, Topological Sort, Array, Directed Acyclic Graph, Matrix

leetcode.com/problems/strange-printer-ii

PROBLEM

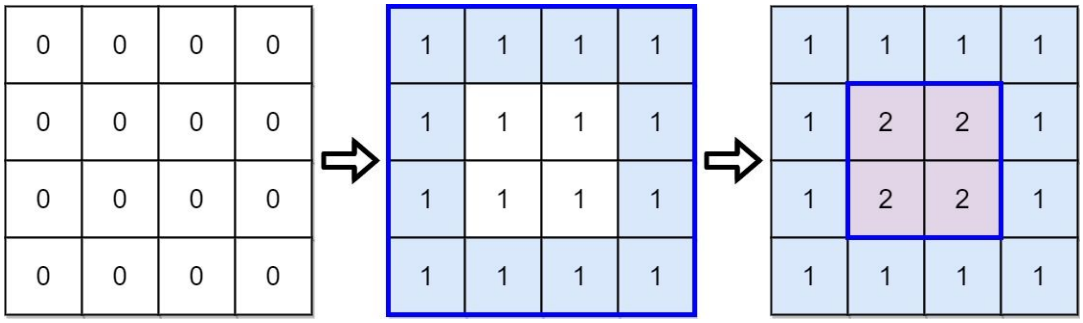
There is a strange printer with the following two special requirements:

- On each turn, the printer will print a solid rectangular pattern of a single color on the grid. This will cover up the existing colors in the rectangle.
- Once the printer has used a color for the above operation, **the same color cannot be used again**.

You are given a `m x n` matrix `targetGrid`, where `targetGrid[row][col]` is the color in the position `(row, col)` of the grid.

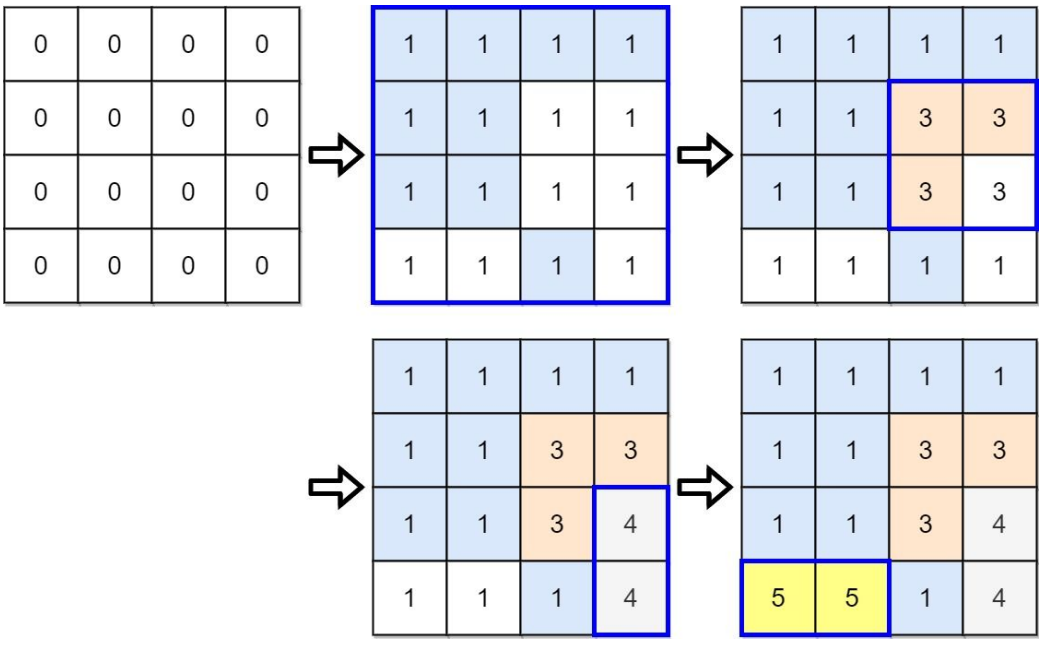
Return `true` *if it is possible to print the matrix* `targetGrid`*, otherwise, return* `false`.

Example 1:




```
Input: targetGrid = [[1,1,1,1],[1,2,2,1],[1,2,2,1],[1,1,1,1]]
Output: true
```

Example 2:




```
Input: targetGrid = [[1,1,1,1],[1,1,3,3],[1,1,3,4],[5,5,1,4]]
Output: true
```

Example 3:

```
Input: targetGrid = [[1,2,1],[2,1,2],[1,2,1]]
Output: false
Explanation: It is impossible to form targetGrid because it is not allowed to print the same color in
different turns.
```

Constraints:

- `m == targetGrid.length`
- `n == targetGrid[i].length`
- `1 <= m, n <= 60`
- `1 <= targetGrid[row][col] <= 60`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Each print lays down a fresh color as a rectangle that cannot be reused. The same color must occupy its bounding box, and later colors overwrite cells inside that box. For every color take the min/max row and column. Any other color c' inside that rectangle must have been printed later, so we add an edge c→ c'. The instance is printable iff this constraint graph is acyclic, which a topological sort decides.

Python 3 (written for this guide: the doocs page has an empty Python tab)

```python
class Solution:
    def isPrintable(self, targetGrid: List[List[int]]) -> bool:
        m, n = len(targetGrid), len(targetGrid[0])
        # bounding rectangle of every colour: [top, left, bottom, right]
        box = {}
        for i in range(m):
            for j in range(n):
                c = targetGrid[i][j]
                if c not in box:
                    box[c] = [i, j, i, j]
                else:
                    b = box[c]
                    b[0] = min(b[0], i)
                    b[1] = min(b[1], j)
                    b[2] = max(b[2], i)
                    b[3] = max(b[3], j)
        # edge a -> b: colour b appears inside the rectangle of a, so a must be printed BEFORE b
        after = {c: set() for c in box}
        indeg = {c: 0 for c in box}
        for a, (top, left, bottom, right) in box.items():
            for i in range(top, bottom + 1):
                for j in range(left, right + 1):
                    b = targetGrid[i][j]
                    if b != a and b not in after[a]:
                        after[a].add(b)
                        indeg[b] += 1
        # Kahn's algorithm: the order exists exactly when there is no cycle
        queue = [c for c in box if indeg[c] == 0]
        taken = 0
        while queue:
            a = queue.pop()
            taken += 1
            for b in after[a]:
                indeg[b] -= 1
                if indeg[b] == 0:
                    queue.append(b)
        return taken == len(box)
```

Complexity. Time: O(C * m * n), C = number of colours. Space: O(C^2).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return one valid order in which the colours were
# printed
# 2. If it cannot be printed, return colours that block
# each other (one cycle)


# ============================================================
# FOLLOW-UP 1: Return one valid order in which the colours
# were printed
# ============================================================

# ---- follow-up 1: function ----
# Answer: A colour must be printed before every other
# colour that shows up inside its bounding box; Kahn's
# order of that graph is a valid print order.
# Time: O(c * R * C), c = colours.  Space: O(c^2).
def print_order(target_grid):
    box = {}
    for r in range(len(target_grid)):
        for c in range(len(target_grid[0])):
            color = target_grid[r][c]
            if color not in box:
                box[color] = [r, c, r, c]
            else:
                b = box[color]
                b[0], b[1] = min(b[0], r), min(b[1], c)
                b[2], b[3] = max(b[2], r), max(b[3], c)
    after = {color: set() for color in box}
    for color, (r0, c0, r1, c1) in box.items():
        for r in range(r0, r1 + 1):
            for c in range(c0, c1 + 1):
                other = target_grid[r][c]
                if other != color:
                    after[color].add(other)
    waiting = {color: 0 for color in box}
    for color in after:
        for other in after[color]:
            waiting[other] += 1
    queue = [color for color in box if waiting[color] == 0]
    for color in queue:
        for other in after[color]:
            waiting[other] -= 1
            if waiting[other] == 0:
                queue.append(other)
    if len(queue) < len(box):
        return None
    return queue


# ============================================================
# FOLLOW-UP 2: If it cannot be printed, return colours
# that block each other (one cycle)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Run a depth-first search on the 'must be printed
# before' graph and report the first cycle found.
# Time: O(c * R * C).  Space: O(c^2).
def blocking_colours(target_grid):
```

```
    box = {}
    for r in range(len(target_grid)):
        for c in range(len(target_grid[0])):
            color = target_grid[r][c]
```

```
            if color not in box:
                box[color] = [r, c, r, c]
            else:
                b = box[color]
                b[0], b[1] = min(b[0], r), min(b[1], c)
                b[2], b[3] = max(b[2], r), max(b[3], c)
    after = {color: set() for color in box}
    for color, (r0, c0, r1, c1) in box.items():
        for r in range(r0, r1 + 1):
            for c in range(c0, c1 + 1):
                if target_grid[r][c] != color:
                    after[color].add(target_grid[r][c])
    state = {color: 0 for color in box}
    walk = []

    def go(color):
        state[color] = 1
        walk.append(color)
        for other in sorted(after[color]):
            if state[other] == 1:
                return walk[walk.index(other):]
            if state[other] == 0:
                found = go(other)
                if found:
                    return found
        walk.pop()
        state[color] = 2
        return None

    for color in sorted(box):
        if state[color] == 0:
            found = go(color)
            if found:
                return found
    return None
```

2360. Longest Cycle in a Graph

Hard · Depth-First Search, Breadth-First Search, Graph, Topological Sort, Kosaraju, Tarjan

leetcode.com/problems/longest-cycle-in-a-graph

PROBLEM

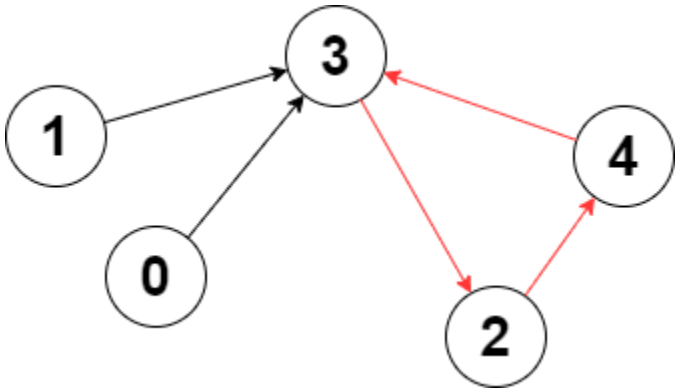
You are given a **directed** graph of `n` nodes numbered from `0` to `n - 1`, where each node has **at most one** outgoing edge.

The graph is represented with a given **0-indexed** array `edges` of size `n`, indicating that there is a directed edge from node `i` to node `edges[i]`. If there is no outgoing edge from node `i`, then `edges[i] == -1`.

Return *the length of the **longest** cycle in the graph*. If no cycle exists, return `-1`.

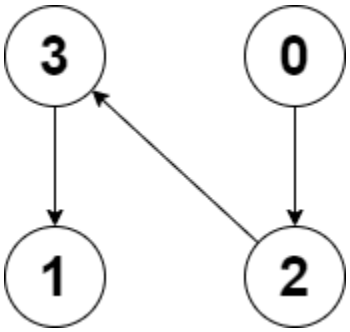
A cycle is a path that starts and ends at the **same** node.

Example 1:




```
Input: edges = [3,3,4,2,3]
Output: 3
Explanation: The longest cycle in the graph is the cycle: 2 -> 4 -> 3 -> 2.
The length of this cycle is 3, so 3 is returned.
```

Example 2:




```
Input: edges = [2,-1,3,1]
Output: -1
Explanation: There are no cycles in this graph.
```

Constraints:

- `n == edges.length`
- `2 <= n <= 10^5`
- `-1 <= edges[i] < n`
- `edges[i] != i`

SOLUTION (PYTHON 3)

Solution 1: Traverse Starting Points

> **How to think about it**
>
> Each node has at most one outgoing edge, so the graph is paths into cycles. n ≤ 10^5; walk the unique successor from every unvisited node. Mark nodes and record the path. Stopping at -1 means no cycle; stopping on a node of this path yields a cycle from that index to the end. Track the longest such length.

We can traverse each node in the range [0,..,n-1]. If a node has not been visited, we start from this node and search for adjacent nodes until we encounter a cycle or a node that has already been visited. If we encounter a cycle, we update the answer.

The time complexity is O(n) and the space complexity is O(n), where n is the number of nodes.

Similar problems:

- 2127. Maximum Employees to Be Invited to a Meeting

Python 3

```python
class Solution:
    def longestCycle(self, edges: List[int]) -> int:
        n = len(edges)
        vis = [False] * n
        ans = -1
        for i in range(n):
            if vis[i]:
                continue
            j = i
            cycle = []
            while j != -1 and not vis[j]:
                vis[j] = True
                cycle.append(j)
                j = edges[j]
            if j == -1:
                continue
            m = len(cycle)
            k = next((k for k in range(m) if cycle[k] == j), inf)
            ans = max(ans, m - k)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the nodes of the longest cycle
# 2. The SHORTEST cycle


# ============================================================
# FOLLOW-UP 1: Return the nodes of the longest cycle
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same walk, keeping the slice of the walk
# that forms the cycle.
# Time: O(n).  Space: O(n).
def longest_cycle_nodes(edges):
    n = len(edges)
    state = [0] * n
    best = []
    for start in range(n):
        if state[start] != 0:
            continue
        walk = []
```

```
        node = start
        while node != -1 and state[node] == 0:
            state[node] = 1
            walk.append(node)
            node = edges[node]
        if node != -1 and state[node] == 1:
            cycle = walk[walk.index(node):]
            if len(cycle) > len(best):
                best = cycle
        for x in walk:
            state[x] = 2
    return best


# ============================================================
# FOLLOW-UP 2: The SHORTEST cycle
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same walk, keeping the minimum cycle length
# instead of the maximum.
# Time: O(n).  Space: O(n).
def shortest_cycle(edges):
    n = len(edges)
    state = [0] * n
    best = -1
    for start in range(n):
        if state[start] != 0:
            continue
        walk = []
        node = start
        while node != -1 and state[node] == 0:
            state[node] = 1
            walk.append(node)
            node = edges[node]
        if node != -1 and state[node] == 1:
            length = len(walk) - walk.index(node)
            if best == -1 or length < best:
                best = length
```

```
        for x in walk:
            state[x] = 2
    return best
```

PATTERN 31 OF 41

Union-Find (Disjoint Set)

20 problems · 1 Easy · 13 Medium · 6 Hard

THE IDEA

Merge pairs into groups and ask who is together, in almost O(1) per operation, using path compression and union by size.

Signals: number of provinces, redundant connection, accounts merge, connected components with edges arriving, smallest string with swaps.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
parent = list(range(n))
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]      # path halving
        x = parent[x]
    return x
def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb:
        return False                       # already together (a cycle)
    parent[ra] = rb
    return True
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Find if Path Exists in Graph	Easy	LC 1971	1661
2	★ **Number of Provinces** (see page)	Medium	LC 547	869
3	★ **Redundant Connection**	Medium	LC 684	1667
4	★ **Accounts Merge**	Medium	LC 721	1671
5	★ **Most Stones Removed with Same Row or Column** (see page)	Medium	LC 947	903
6	★ **Number of Operations to Make Network Connected**	Medium	LC 1319	1674
7	Satisfiability of Equality Equations	Medium	LC 990	1677
8	★ **Smallest String With Swaps**	Medium	LC 1202	1680
9	Lexicographically Smallest Equivalent String	Medium	LC 1061	1683
10	Count Unreachable Pairs of Nodes in an Undirected Graph	Medium	LC 2316	1686
11	Minimize Malware Spread	Medium	LC 924	1690
12	Regions Cut By Slashes	Medium	LC 959	1694
13	Longest Consecutive Sequence (see page)	Medium	LC 128	37
14	Minimize the Maximum of Two Arrays — substitute: The Earliest Moment When Everyone Become Friends	Medium	LC 1101	1700
15	★ **Redundant Connection II**	Hard	LC 685	1704
16	Graph Connectivity With Threshold	Hard	LC 1627	1709
17	Making A Large Island (see page)	Hard	LC 827	920
18	Checking Existence of Edge Length Limited Paths	Hard	LC 1697	1713
19	Largest Component Size by Common Factor	Hard	LC 952	1718
20	Remove Max Number of Edges to Keep Graph Fully Traversable	Hard	LC 1579	1721

1971. Find if Path Exists in Graph

Easy · Depth-First Search, Breadth-First Search, Union Find, Graph

leetcode.com/problems/find-if-path-exists-in-graph

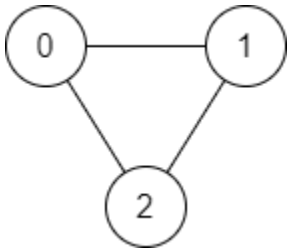
PROBLEM

There is a **bi-directional** graph with `n` vertices, where each vertex is labeled from `0` to `n - 1` (**inclusive**). The edges in the graph are represented as a 2D integer array `edges`, where each `edges[i] = [u_i, v_i]` denotes a bi-directional edge between vertex `u_i` and vertex `v_i`. Every vertex pair is connected by **at most one** edge, and no vertex has an edge to itself.

You want to determine if there is a **valid path** that exists from vertex `source` to vertex `destination`.

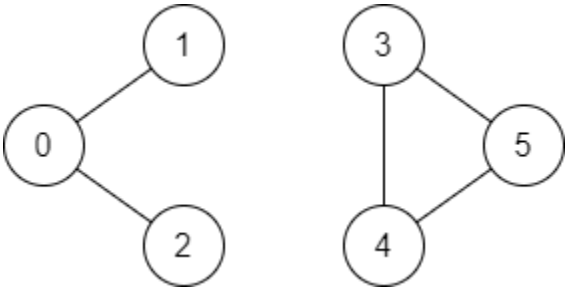
Given `edges` and the integers `n`, `source`, and `destination`, return `true` *if there is a* ***valid path*** *from* `source` *to* `destination`*, or* `false` *otherwise*.

Example 1:




```
Input: n = 3, edges = [[0,1],[1,2],[2,0]], source = 0, destination = 2
Output: true
Explanation: There are two paths from vertex 0 to vertex 2:
- 0 → 1 → 2
- 0 → 2
```

Example 2:




```
Input: n = 6, edges = [[0,1],[0,2],[3,5],[5,4],[4,3]], source = 0, destination = 5
Output: false
Explanation: There is no path from vertex 0 to vertex 5.
```

Constraints:

- `1 <= n <= 2 * 10^5`
- `0 <= edges.length <= 2 * 10^5`
- `edges[i].length == 2`
- `0 <= u_i, v_i <= n - 1`
- `u_i != v_i`
- `0 <= source, destination <= n - 1`
- There are no duplicate edges.
- There are no self edges.

SOLUTIONS (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> We only need connectivity between two vertices. After building the adjacency lists, DFS with a visited set reports success on reaching the destination, in linear time.

We first convert edges into an adjacency list g, then use DFS to determine whether there is a path from source to destination.

During the process, we use an array vis to record the vertices that have already been visited to avoid revisiting them.

The time complexity is O(n + m), and the space complexity is O(n + m). Here, n and m are the number of nodes and edges, respectively.

Python 3

```
class Solution:
    def validPath(
        self, n: int, edges: List[List[int]], source: int, destination: int
    ) -> bool:
        def dfs(i: int) -> bool:
            if i == destination:
                return True
            vis.add(i)
            for j in g[i]:
                if j not in vis and dfs(j):
                    return True
            return False

        g = [[] for _ in range(n)]
        for u, v in edges:
            g[u].append(v)
            g[v].append(u)
        vis = set()
        return dfs(source)
```

Solution 2: BFS

> **How to think about it**
>
> Method 1 expands with an explicit stack in depth-first order. A queue expands level by level with the same visited marks: the source is enqueued, and a dequeued vertex that is the destination succeeds. Unvisited neighbors are enqueued. An empty queue means the destination is unreachable.

We can also use BFS to determine whether there is a path from source to destination.

Specifically, we define a queue q, initially adding source to the queue. Additionally, we use a set vis to record the vertices that have already been visited to avoid revisiting them.

Next, we continuously take vertices i from the queue. If i = destination, it means there is a path from source to destination, and we return true. Otherwise, we traverse all adjacent vertices j of i. If j has not been visited, we add j to the queue q and mark j as visited.

Finally, if the queue is empty, it means there is no path from source to destination, and we return false.

The time complexity is O(n + m), and the space complexity is O(n + m). Here, n and m are the number of nodes and edges, respectively.

Python 3

```python
class Solution:
    def validPath(
        self, n: int, edges: List[List[int]], source: int, destination: int
    ) -> bool:
        g = [[] for _ in range(n)]
        for u, v in edges:
            g[u].append(v)
            g[v].append(u)
        q = deque([source])
        vis = {source}
        while q:
            i = q.popleft()
            if i == destination:
                return True
            for j in g[i]:
                if j not in vis:
                    vis.add(j)
                    q.append(j)
        return False
```

Solution 3: Union-Find

> **How to think about it**
>
> When the path itself is unused, union-find merges every edge and compares the two roots. There is no walk over the graph.

Union-Find is a tree-like data structure that, as the name suggests, is used to handle some disjoint set **merge** and **query** problems. It supports two operations:

1. Find: Determine which subset an element belongs to. The time complexity of a single operation is O(α(n)).
2. Union: Merge two subsets into one set. The time complexity of a single operation is O(α(n)).

For this problem, we can use the Union-Find set to merge the edges in `edges`, and then determine whether `source` and `destination` are in the same set.

The time complexity is O(n log n + m) or O(n α(n) + m), and the space complexity is O(n). Where n and m are the number of nodes and edges, respectively.

Python 3

```python
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True


class Solution:
    def validPath(
        self, n: int, edges: List[List[int]], source: int, destination: int
    ) -> bool:
        uf = UnionFind(n)
        for u, v in edges:
            uf.union(u, v)
        return uf.find(source) == uf.find(destination)
```

Solution 4: Explicit Stack

How to think about it

We only need connectivity between two vertices. With $n \le 2 \times 10^5$, an adjacency-list walk with a visited mark reports success on reaching the destination, in linear time. Recursing from the source into each neighbor reaches depth n on a chain and overflows the call stack. Connectivity does not depend on the order of neighbors. An explicit stack therefore returns immediately when the source is the destination. Otherwise a vertex is marked when it is pushed. After a pop, a neighbor that is the destination succeeds, and only unmarked neighbors are pushed. An empty stack means the destination is unreachable.

We first convert edges into an adjacency list g. If source is destination, we return true immediately. Otherwise an explicit stack starts at the source: a vertex is recorded in vis when it is pushed, a neighbor that is the destination returns true, and only unmarked neighbors are pushed. An empty stack means there is no path.

The time complexity is O(n + m), and the space complexity is O(n + m). Here, n and m are the number of nodes and edges, respectively.

Python 3

```
class Solution:
    def validPath(
        self, n: int, edges: List[List[int]], source: int, destination: int
    ) -> bool:
        if source == destination:
            return True
        g = [[] for _ in range(n)]
        for u, v in edges:
            g[u].append(v)
            g[v].append(u)
        vis = [False] * n
        vis[source] = True
        stk = [source]
        while stk:
            i = stk.pop()
            for j in g[i]:
                if j == destination:
                    return True
                if not vis[j]:
                    vis[j] = True
                    stk.append(j)
        return False
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Answer many 'is there a path' questions fast
# 2. Return the length of the shortest path (fewest edges)


# ============================================================
# FOLLOW-UP 1: Answer many 'is there a path' questions
# fast
# ============================================================

# ---- follow-up 1: function ----
# Answer: Join the edges once with union-find; each
# question is then two finds.
# Time: O((n + e + q) * alpha(n)).  Space: O(n).
def connected_queries(n, edges, queries):
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for a, b in edges:
        parent[find(a)] = find(b)
    return [find(a) == find(b) for a, b in queries]


# ============================================================
# FOLLOW-UP 2: Return the length of the shortest path
# (fewest edges)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Breadth-first search from the source.
# Time: O(n + e).  Space: O(n + e).
def shortest_path_length(n, edges, source, destination):
    neighbours = [[] for _ in range(n)]
    for a, b in edges:
        neighbours[a].append(b)
```

```
        neighbours[b].append(a)
    distance = {source: 0}
    queue = [source]
    for node in queue:
        if node == destination:
            return distance[node]
        for nxt in neighbours[node]:
            if nxt not in distance:
                distance[nxt] = distance[node] + 1
                queue.append(nxt)
    return -1
```

684. Redundant Connection

Medium · Depth-First Search, Breadth-First Search, Union Find, Graph

leetcode.com/problems/redundant-connection

★ **TOP 7 PROBLEM** in Union-Find (Disjoint Set): master this one first

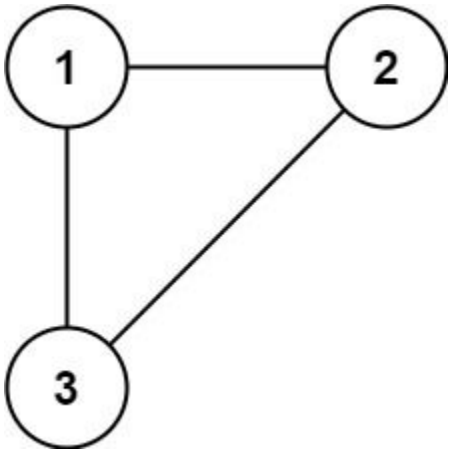
PROBLEM

In this problem, a tree is an **undirected graph** that is connected and has no cycles.

You are given a graph that started as a tree with `n` nodes labeled from `1` to `n`, with one additional edge added. The added edge has two **different** vertices chosen from `1` to `n`, and was not an edge that already existed. The graph is represented as an array `edges` of length `n` where `edges[i] = [a_i, b_i]` indicates that there is an edge between nodes a_i and b_i in the graph.

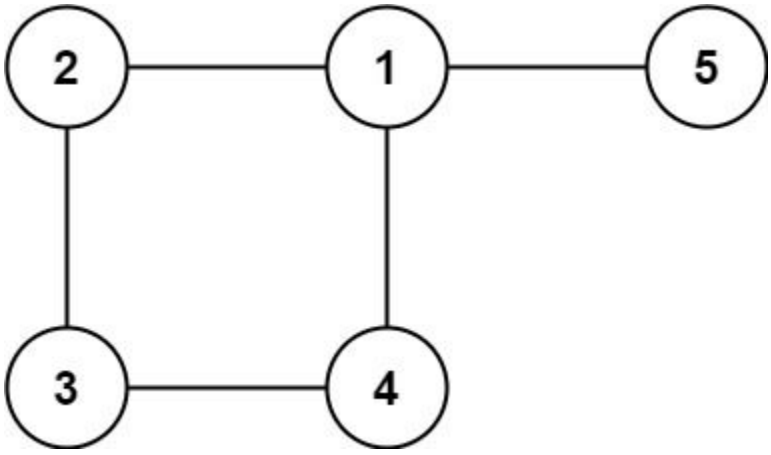
Return *an edge that can be removed so that the resulting graph is a tree of* `n` *nodes*. If there are multiple answers, return the answer that occurs last in the input.

Example 1:




```
Input: edges = [[1,2],[1,3],[2,3]]
Output: [2,3]
```

Example 2:




```
Input: edges = [[1,2],[2,3],[3,4],[1,4],[1,5]]
Output: [1,4]
```

Constraints:

- `n == edges.length`
- `3 <= n <= 1000`
- `edges[i].length == 2`
- `1 <= a_i < b_i <= edges.length`
- `a_i != b_i`
- There are no repeated edges.
- The given graph is connected.

SOLUTIONS (PYTHON 3)

Solution 1: Union-Find

> **How to think about it**
>
> An undirected tree plus one extra edge; delete the last extra edge in the input. Finding the cycle and ranking edges is heavier than needed. Union-Find the edges in order. The first pair already in one component is the redundant edge.

According to the problem description, we need to find an edge that can be removed so that the remaining part is a tree with n nodes. We can traverse each edge and determine whether the two nodes of this edge are in the same connected component. If they are in the same connected component, it means this edge is redundant and can be removed, so we directly return this edge. Otherwise, we merge the two nodes connected by this edge into the same connected component.

The time complexity is O(n log n), and the space complexity is O(n). Here, n is the number of edges.

Python 3

```
class Solution:
    def findRedundantConnection(self, edges: List[List[int]]) -> List[int]:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(len(edges)))
        for a, b in edges:
            pa, pb = find(a - 1), find(b - 1)
            if pa == pb:
                return [a, b]
            p[pa] = pb
```

Solution 2: Union-Find (Template Approach)

> **How to think about it**
>
> Method 1 inlines `find`. A `UnionFind` with union-by-size returns success/failure; the first failed union is the same edge, with slightly better constants.

Here is a template approach using Union-Find for your reference.

The time complexity is O(n α(n)), and the space complexity is O(n). Here, n is the number of edges, and α(n) is the inverse Ackermann function, which can be considered a very small constant.

Python 3

```python
class UnionFind:
    __slots__ = "p", "size"

    def __init__(self, n: int):
        self.p: List[int] = list(range(n))
        self.size: List[int] = [1] * n

    def find(self, x: int) -> int:
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True


class Solution:
    def findRedundantConnection(self, edges: List[List[int]]) -> List[int]:
        uf = UnionFind(len(edges))
        for a, b in edges:
            if not uf.union(a - 1, b - 1):
                return [a, b]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Several extra edges: return all edges that close a
# cycle when added in order
# 2. After every edge, how many connected groups are there


# ============================================================
# FOLLOW-UP 1: Several extra edges: return all edges that
# close a cycle when added in order
# ============================================================

# ---- follow-up 1: function ----
# Answer: Union-find; an edge whose two ends are already
# joined is redundant.
# Time: O(e * alpha(e)).  Space: O(e).
def all_redundant_edges(edges):
    parent = {}

    def find(x):
        parent.setdefault(x, x)
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    redundant = []
    for a, b in edges:
        ra, rb = find(a), find(b)
        if ra == rb:
            redundant.append([a, b])
        else:
```

```
            parent[ra] = rb
    return redundant


# ============================================================
# FOLLOW-UP 2: After every edge, how many connected groups
# are there
# ============================================================

# ---- follow-up 2: function ----
# Answer: Start with n groups and subtract one for each
# union that really joins two groups.
# Time: O(n + e * alpha(n)).  Space: O(n).
def groups_after_each_edge(n, edges):
    parent = list(range(n + 1))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    groups = n
    result = []
    for a, b in edges:
        ra, rb = find(a), find(b)
        if ra != rb:
            parent[ra] = rb
            groups -= 1
        result.append(groups)
    return result
```

721. Accounts Merge

Medium · Depth-First Search, Breadth-First Search, Union Find, Array, Hash Table, String, Sorting

leetcode.com/problems/accounts-merge

★ **TOP 7 PROBLEM** in Union-Find (Disjoint Set): master this one first

PROBLEM

Given a list of `accounts` where each element `accounts[i]` is a list of strings, where the first element `accounts[i][0]` is a name, and the rest of the elements are **emails** representing emails of the account.

Now, we would like to merge these accounts. Two accounts definitely belong to the same person if there is some common email to both accounts. Note that even if two accounts have the same name, they may belong to different people as people could have the same name. A person can have any number of accounts initially, but all of their accounts definitely have the same name.

After merging the accounts, return the accounts in the following format: the first element of each account is the name, and the rest of the elements are emails **in sorted order**. The accounts themselves can be returned in **any order**.

Example 1:

```
Input: accounts = [["John","johnsmith@mail.com","john_newyork@mail.com"],["John","johnsmith@mail.com",
"john00@mail.com"],["Mary","mary@mail.com"],["John","johnnybravo@mail.com"]]
Output: [["John","john00@mail.com","john_newyork@mail.com","johnsmith@mail.com"],["Mary","mary@mail.co
m"],["John","johnnybravo@mail.com"]]
Explanation:
The first and second John's are the same person as they have the common email "johnsmith@mail.com".
The third John and Mary are different people as none of their email addresses are used by other
accounts.
We could return these lists in any order, for example the answer [['Mary', 'mary@mail.com'], ['John',
'johnnybravo@mail.com'],
['John', 'john00@mail.com', 'john_newyork@mail.com', 'johnsmith@mail.com']] would still be accepted.
```

Example 2:

```
Input: accounts = [["Gabe","Gabe0@m.co","Gabe3@m.co","Gabe1@m.co"],["Kevin","Kevin3@m.co","Kevin5@m.co
","Kevin0@m.co"],["Ethan","Ethan5@m.co","Ethan4@m.co","Ethan0@m.co"],["Hanzo","Hanzo3@m.co","Hanzo1@m.
co","Hanzo0@m.co"],["Fern","Fern5@m.co","Fern1@m.co","Fern0@m.co"]]
Output: [["Ethan","Ethan0@m.co","Ethan4@m.co","Ethan5@m.co"],["Gabe","Gabe0@m.co","Gabe1@m.co","Gabe3@
m.co"],["Hanzo","Hanzo0@m.co","Hanzo1@m.co","Hanzo3@m.co"],["Kevin","Kevin0@m.co","Kevin3@m.co","Kevin
5@m.co"],["Fern","Fern0@m.co","Fern1@m.co","Fern5@m.co"]]
```

Constraints:

- `1 <= accounts.length <= 1000`
- `2 <= accounts[i].length <= 10`
- `1 <= accounts[i][j].length <= 30`
- `accounts[i][0]` consists of English letters.
- `accounts[i][j] (for j > 0)` is a valid email.

SOLUTION (PYTHON 3)

Solution 1: Union-Find + Hash Table

> **How to think about it**
>
> Accounts of the same person share emails and must be merged, then sorted. About 1000 accounts makes pairwise set comparison awkward, and names must stay attached. The connectivity is among accounts: two indices join if they share an email. Union-find on account ids, plus a map from email to the first account that used it, finds those edges. Group emails by root, take the root account's name, and sort. Path compression keeps the cost near linearithmic.

Based on the problem description, we can use a union-find data structure to merge accounts with the same email address. The specific steps are as follows:

First, we iterate through all the accounts. For the ith account, we iterate through all its email addresses. If an email address appears in the hash table d, we use the union-find to merge the account's index i with the previously appeared account's index; otherwise, we map this email address to the account's index i.

Next, we iterate through all the accounts again. For the ith account, we use the union-find to find its root node, and then add all the email addresses of that account to the hash table g, where the key is the root node, and the value is the account's email addresses.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the number of accounts.

Python 3

```python
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True


class Solution:
    def accountsMerge(self, accounts: List[List[str]]) -> List[List[str]]:
        uf = UnionFind(len(accounts))
        d = {}
        for i, (_, *emails) in enumerate(accounts):
            for email in emails:
                if email in d:
                    uf.union(i, d[email])
                else:
                    d[email] = i
        g = defaultdict(set)
        for i, (_, *emails) in enumerate(accounts):
            root = uf.find(i)
            g[root].update(emails)
        return [[accounts[root][0]] + sorted(emails) for root, emails in g.items()]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. How many different people are there
# 2. Return the merged account with the most emails


# ============================================================
# FOLLOW-UP 1: How many different people are there
# ============================================================

# ---- follow-up 1: function ----
# Answer: Union the emails of every account; the number of
# groups is the number of people.
# Time: O(L * alpha(L)), L = total emails.  Space: O(L).
def number_of_people(accounts):
    parent = {}

    def find(x):
        parent.setdefault(x, x)
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for account in accounts:
        first = find(account[1])
        for email in account[2:]:
            parent[find(email)] = first
    return len(set(find(email) for email in parent))


# ============================================================
# FOLLOW-UP 2: Return the merged account with the most
# emails
# ============================================================

# ---- follow-up 2: function ----
# Answer: Merge as in the question, then take the longest
# list.
# Time: O(L log L).  Space: O(L).
def account_with_most_emails(accounts):
    parent = {}
    owner = {}

    def find(x):
        parent.setdefault(x, x)
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for account in accounts:
        first = find(account[1])
        for email in account[1:]:
            owner[email] = account[0]
            parent[find(email)] = first
    groups = {}
    for email in parent:
        groups.setdefault(find(email), []).append(email)
    best = None
    for root, emails in groups.items():
```

```
        if best is None or len(emails) > len(best):
            best = emails
    return [owner[best[0]]] + sorted(best)
```

1319. Number of Operations to Make Network Connected

Medium · Depth-First Search, Breadth-First Search, Union Find, Graph

leetcode.com/problems/number-of-operations-to-make-network-connected

★ **TOP 7 PROBLEM** in Union-Find (Disjoint Set): master this one first

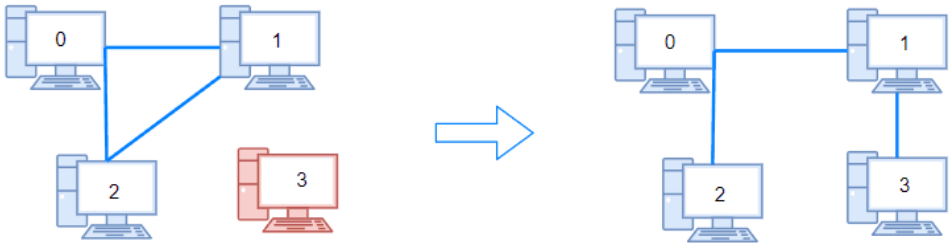
PROBLEM

There are `n` computers numbered from `0` to `n - 1` connected by ethernet cables `connections` forming a network where `connections[i] = [a_i, b_i]` represents a connection between computers a_i and b_i. Any computer can reach any other computer directly or indirectly through the network.

You are given an initial computer network `connections`. You can extract certain cables between two directly connected computers, and place them between any pair of disconnected computers to make them directly connected.

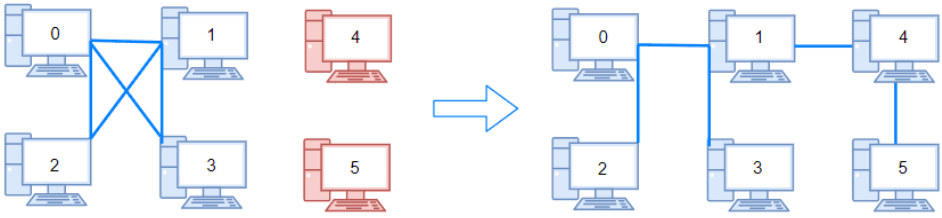
Return *the minimum number of times you need to do this in order to make all the computers connected*. If it is not possible, return `-1`.

Example 1:




```
Input: n = 4, connections = [[0,1],[0,2],[1,2]]
Output: 1
Explanation: Remove cable between computer 1 and 2 and place between computers 1 and 3.
```

Example 2:




```
Input: n = 6, connections = [[0,1],[0,2],[0,3],[1,2],[1,3]]
Output: 2
```

Example 3:

```
Input: n = 6, connections = [[0,1],[0,2],[0,3],[1,2]]
Output: -1
Explanation: There are not enough cables.
```

Constraints:

- `1 <= n <= 10^5`
- `1 <= connections.length <= min(n * (n - 1) / 2, 10^5)`
- `connections[i].length == 2`

- $0 <= a_i, b_i < n$
- $a_i != b_i$
- There are no repeated connections.
- No two computers are connected by more than one cable.

SOLUTION (PYTHON 3)

Solution 1: Union-Find

> **How to think about it**
>
> We may move existing cables so n machines become one network. With n,m up to 10^5, rebuilding the graph is the wrong tool. k components need at least k-1 spare edges to become a tree. Union-find over the edges: a link inside one component is redundant, otherwise we merge and decrease k. If the spare count is below k-1 the answer is -1; otherwise it is k-1.

We can use a union-find data structure to maintain the connectivity between computers. Traverse all connections, and for each connection (a, b), if a and b are already connected, then this connection is redundant, and we increment the count of redundant connections. Otherwise, we connect a and b, and decrement the number of connected components.

Finally, if the number of connected components minus one is greater than the number of redundant connections, it means we cannot connect all computers, so we return -1. Otherwise, we return the number of connected components minus one.

The time complexity is O(m × log n), and the space complexity is O(n). Here, n and m are the number of computers and the number of connections, respectively.

Python 3

```
class Solution:
    def makeConnected(self, n: int, connections: List[List[int]]) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        cnt = 0
        p = list(range(n))
        for a, b in connections:
            pa, pb = find(a), find(b)
            if pa == pb:
                cnt += 1
            else:
                p[pa] = pb
                n -= 1
        return -1 if n - 1 > cnt else n - 1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the actual moves: which cable to take and
# which two computers to join
# 2. Count how many computers are not connected to
# computer 0


# ========================================================
# FOLLOW-UP 1: Return the actual moves: which cable to
# take and which two computers to join
# ========================================================

# ---- follow-up 1: function ----
# Answer: Collect the spare cables (those closing a cycle)
# and the roots of the groups; connect group roots one
```

```
# after another with spare cables.
# Time: O((n + c) * alpha(n) + r log r), r = components.
# Space: O(n + c).
def cable_moves(n, connections):
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    spare = []
    for a, b in connections:
        ra, rb = find(a), find(b)
        if ra == rb:
            spare.append([a, b])
        else:
            parent[ra] = rb
    roots = sorted(set(find(x) for x in range(n)))
    if len(spare) < len(roots) - 1:
        return None
    moves = []
    for i in range(1, len(roots)):
        cable = spare.pop()
        moves.append((cable, [roots[0], roots[i]]))
    return moves


# ============================================================
# FOLLOW-UP 2: Count how many computers are not connected
# to computer 0
# ============================================================

# ---- follow-up 2: function ----
# Answer: Union all the cables and compare every root with
# the root of computer 0.
# Time: O((n + c) * alpha(n)).  Space: O(n).
def computers_not_with_zero(n, connections):
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for a, b in connections:
        parent[find(a)] = find(b)
    count = 0
    for x in range(n):
        if find(x) != find(0):
            count += 1
    return count
```

990. Satisfiability of Equality Equations

Medium · Union Find, Graph, Array, String

leetcode.com/problems/satisfiability-of-equality-equations

PROBLEM

You are given an array of strings `equations` that represent relationships between variables where each string `equations[i]` is of length `4` and takes one of two different forms: "x_i==y_i" or "x_i!=y_i".Here, x_i and y_i are lowercase letters (not necessarily different) that represent one-letter variable names.

Return `true` *if it is possible to assign integers to variable names so as to satisfy all the given equations, or* `false` *otherwise*.

Example 1:

```
Input: equations = ["a==b","b!=a"]
Output: false
Explanation: If we assign say, a = 1 and b = 1, then the first equation is satisfied, but not the second.
There is no way to assign the variables to satisfy both equations.
```

Example 2:

```
Input: equations = ["b==a","a==b"]
Output: true
Explanation: We could assign a = 1 and b = 1 to satisfy both equations.
```

Constraints:

- `1 <= equations.length <= 500`
- `equations[i].length == 4`
- `equations[i][0]` is a lowercase letter.
- `equations[i][1]` is either `'='` or `'!'`.
- `equations[i][2]` is `'='`.
- `equations[i][3]` is a lowercase letter.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

A set of a==b / a!=b equations must be simultaneously possible. Equality is transitive, so those edges should be united first. Union-find merges every `==`, then each `!=` is checked; if both sides share a component, the system is impossible.

Python 3

```
class Solution:
    def equationsPossible(self, equations: List[str]) -> bool:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(26))
        for e in equations:
            a, b = ord(e[0]) - ord('a'), ord(e[-1]) - ord('a')
            if e[1] == '=':
                p[find(a)] = find(b)
        for e in equations:
            a, b = ord(e[0]) - ord('a'), ord(e[-1]) - ord('a')
            if e[1] == '!' and find(a) == find(b):
                return False
        return True
```

Complexity. Time: O(n * alpha(26)), essentially O(n). Space: O(1) (26 letters).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the first equation that makes the list
# impossible
# 2. List the groups of letters that must be equal


# ========================================================
# FOLLOW-UP 1: Return the first equation that makes the
# list impossible
# ========================================================

# ---- follow-up 1: function ----
# Answer: Process the equations one at a time with union-
# find, keeping all != pairs so far; stop the first time
# any != pair becomes equal.
# Time: O(E^2 * alpha), the != pairs are rechecked after
# every equation.  Space: O(26 + E).
def first_conflict(equations):
    parent = {}

    def find(x):
        parent.setdefault(x, x)
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    different = []
    for i in range(len(equations)):
        a, op, b = equations[i][0], equations[i][1:3], equations[i][3]
        if op == "==":
            parent[find(a)] = find(b)
        else:
            different.append((a, b))
        for x, y in different:
            if find(x) == find(y):
                return i
    return -1


# ========================================================
# FOLLOW-UP 2: List the groups of letters that must be
# equal
# ========================================================
```

```
# ---- follow-up 2: function ----
# Answer: Union all the == pairs and collect the letters
# by root.
# Time: O(E * alpha + 26 log 26).  Space: O(26).
def equal_groups(equations):
    parent = {}

    def find(x):
        parent.setdefault(x, x)
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for eq in equations:
```

```
        a, b = eq[0], eq[3]
        find(a)
        find(b)
        if eq[1:3] == "==":
            parent[find(a)] = find(b)
    groups = {}
    for letter in parent:
        groups.setdefault(find(letter), []).append(letter)
    return sorted(sorted(g) for g in groups.values())
```

PATTERN 31 · UNION-FIND (DISJOINT SET) · PROBLEM 8 OF 20

1202. Smallest String With Swaps

Medium · Depth-First Search, Breadth-First Search, Union Find, Array, Hash Table, String, Sorting

leetcode.com/problems/smallest-string-with-swaps

★ **TOP 7 PROBLEM** in Union-Find (Disjoint Set): master this one first

PROBLEM

You are given a string `s`, and an array of pairs of indices in the string `pairs` where `pairs[i] = [a, b]` indicates 2 indices(0-indexed) of the string.

You can swap the characters at any pair of indices in the given `pairs` **any number of times**.

Return the lexicographically smallest string that `s` can be changed to after using the swaps.

Example 1:

```
Input: s = "dcab", pairs = [[0,3],[1,2]]
Output: "bacd"
Explaination:
Swap s[0] and s[3], s = "bcad"
Swap s[1] and s[2], s = "bacd"
```

Example 2:

```
Input: s = "dcab", pairs = [[0,3],[1,2],[0,2]]
Output: "abcd"
Explaination:
Swap s[0] and s[3], s = "bcad"
Swap s[0] and s[2], s = "acbd"
Swap s[1] and s[2], s = "abcd"
```

Example 3:

```
Input: s = "cba", pairs = [[0,1],[1,2]]
Output: "abc"
Explaination:
Swap s[0] and s[1], s = "bca"
Swap s[1] and s[2], s = "bac"
Swap s[0] and s[1], s = "abc"
```

Constraints:

- `1 <= s.length <= 10^5`
- `0 <= pairs.length <= 10^5`
- `0 <= pairs[i][0], pairs[i][1] < s.length`
- `s` only contains lower case English letters.

SOLUTION (PYTHON 3)

Solution 1: Union-Find

How to think about it

Trying all permutations of each swap is infeasible when both n and |pairs| reach 10^5. Swaps are transitive: indices form connected components, and characters inside a component may be rearranged freely. The lexicographically smallest string fills each component's indices with its characters in increasing order. Union-find builds the components; we collect characters by root, sort them in reverse, then pop the smallest character back to each index. Union-find identifies the groups; sorting supplies the optimal fill order.

We notice that the index pairs have transitivity, i.e., if a and b can be swapped, and b and c can be swapped, then a and c can also be swapped. Therefore, we can consider using a union-find data structure to maintain the connectivity of these index pairs, and sort the characters belonging to the same connected component in lexicographical order.

Finally, we traverse the string. For the character at the current position, we replace it with the smallest character in the connected component, then remove this character from the connected component, and continue to traverse the string.

The time complexity is $O(n \times \log n + m \times \alpha(m))$, and the space complexity is $O(n)$. Here, n and m are the length of the string and the number of index pairs, respectively, and α is the inverse Ackermann function.

Python 3

```python
class Solution:
    def smallestStringWithSwaps(self, s: str, pairs: List[List[int]]) -> str:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(s)
        p = list(range(n))
        for a, b in pairs:
            p[find(a)] = find(b)
        d = defaultdict(list)
        for i, c in enumerate(s):
            d[find(i)].append(c)
        for i in d.keys():
            d[i].sort(reverse=True)
        return "".join(d[find(i)].pop() for i in range(n))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The LARGEST string
# 2. How many independent groups of positions are there


# ============================================================
# FOLLOW-UP 1: The LARGEST string
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same groups, but put the biggest letters of each
# group first.
# Time: O(n log n + p * alpha(n)).  Space: O(n).
def largest_string_with_swaps(s, pairs):
    parent = list(range(len(s)))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for a, b in pairs:
        parent[find(a)] = find(b)
    positions = {}
    for i in range(len(s)):
        positions.setdefault(find(i), []).append(i)
    result = list(s)
    for spots in positions.values():
        letters = sorted((s[i] for i in spots), reverse=True)
        for spot, letter in zip(spots, letters):
            result[spot] = letter
    return "".join(result)
```

```
# ============================================================
# FOLLOW-UP 2: How many independent groups of positions
# are there
# ============================================================

# ---- follow-up 2: function ----
# Answer: The number of union-find roots.
# Time: O((n + p) * alpha(n)).  Space: O(n).
def number_of_swap_groups(length, pairs):
    parent = list(range(length))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for a, b in pairs:
        parent[find(a)] = find(b)
    return len(set(find(i) for i in range(length)))
```

1061. Lexicographically Smallest Equivalent String

Medium · Union Find, String

leetcode.com/problems/lexicographically-smallest-equivalent-string

PROBLEM

You are given two strings of the same length `s1` and `s2` and a string `baseStr`.

We say `s1[i]` and `s2[i]` are equivalent characters.

- For example, if `s1 = "abc"` and `s2 = "cde"`, then we have `'a' == 'c'`, `'b' == 'd'`, and `'c' == 'e'`.

Equivalent characters follow the usual rules of any equivalence relation:

- **Reflexivity:** `'a' == 'a'`.
- **Symmetry:** `'a' == 'b'` implies `'b' == 'a'`.
- **Transitivity:** `'a' == 'b'` and `'b' == 'c'` implies `'a' == 'c'`.

For example, given the equivalency information from `s1 = "abc"` and `s2 = "cde"`, `"acd"` and `"aab"` are equivalent strings of `baseStr = "eed"`, and `"aab"` is the lexicographically smallest equivalent string of `baseStr`.

Return *the lexicographically smallest equivalent string of* `baseStr` *by using the equivalency information from* `s1` *and* `s2`.

Example 1:

```
Input: s1 = "parker", s2 = "morris", baseStr = "parser"
Output: "makkek"
Explanation: Based on the equivalency information in s1 and s2, we can group their characters as
[m,p], [a,o], [k,r,s], [e,i].
The characters in each group are equivalent and sorted in lexicographical order.
So the answer is "makkek".
```

Example 2:

```
Input: s1 = "hello", s2 = "world", baseStr = "hold"
Output: "hdld"
Explanation: Based on the equivalency information in s1 and s2, we can group their characters as
[h,w], [d,e,o], [l,r].
So only the second letter 'o' in baseStr is changed to 'd', the answer is "hdld".
```

Example 3:

```
Input: s1 = "leetcode", s2 = "programs", baseStr = "sourcecode"
Output: "aauaaaaada"
Explanation: We group the equivalent characters in s1 and s2 as [a,o,e,r,s,c], [l,p], [g,t] and [d,m],
thus all letters in baseStr except 'u' and 'd' are transformed to 'a', the answer is "aauaaaaada".
```

Constraints:

- `1 <= s1.length, s2.length, baseStr <= 1000`
- `s1.length == s2.length`
- `s1`, `s2`, and `baseStr` consist of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Union Find

How to think about it

Equivalence is transitive. Each letter of baseStr should become the smallest letter in its class. Twenty-six letters fit in a disjoint-set union. When uniting s1[i] and s2[i] we attach the larger root to the smaller one, so the representative is the lexicographically smallest character. Each character of baseStr is replaced by its root.

We can use Union Find (Disjoint Set Union, DSU) to handle the equivalence relations between characters. Each character can be regarded as a node, and the equivalence relations can be seen as edges connecting these nodes. With Union Find, we can group all equivalent characters together and quickly find the representative element for each character during queries. When performing union operations, we always set the representative element to be the lexicographically smallest character. This ensures that the final string is the lexicographically smallest equivalent string.

The time complexity is $O((n + m) \times \log |\Sigma|)$ and the space complexity is $O(|\Sigma|)$, where n is the length of strings s1 and s2, m is the length of baseStr, and $|\Sigma|$ is the size of the character set, which is 26 in this problem.

Python 3

```
class Solution:
    def smallestEquivalentString(self, s1: str, s2: str, baseStr: str) -> str:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(26))
        for a, b in zip(s1, s2):
            x, y = ord(a) - ord("a"), ord(b) - ord("a")
            px, py = find(x), find(y)
            if px < py:
                p[py] = px
            else:
                p[px] = py
        return "".join(chr(find(ord(c) - ord("a")) + ord("a")) for c in baseStr)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The LARGEST equivalent string
# 2. List the groups of equivalent letters


# ============================================================
# FOLLOW-UP 1: The LARGEST equivalent string
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same union-find, but keep the largest letter as
# the root of each group.
# Time: O((n + m) * alpha(26)).  Space: O(26).
def largest_equivalent_string(s1, s2, base_str):
    parent = list(range(26))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for a, b in zip(s1, s2):
        ra, rb = find(ord(a) - 97), find(ord(b) - 97)
```

```
        if ra < rb:
            parent[ra] = rb
        else:
            parent[rb] = ra
    return "".join(chr(97 + find(ord(ch) - 97)) for ch in base_str)


# ============================================================
# FOLLOW-UP 2: List the groups of equivalent letters
# ============================================================

# ---- follow-up 2: function ----
# Answer: Collect the letters by root.
# Time: O(n * alpha(26) + 26 log 26).  Space: O(26).
def equivalent_letter_groups(s1, s2):
    parent = list(range(26))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    used = set()
    for a, b in zip(s1, s2):
        used.add(ord(a) - 97)
        used.add(ord(b) - 97)
        parent[find(ord(a) - 97)] = find(ord(b) - 97)
    groups = {}
    for x in used:
        groups.setdefault(find(x), []).append(chr(97 + x))
    return sorted(sorted(g) for g in groups.values())
```

2316. Count Unreachable Pairs of Nodes in an Undirected Graph

Medium · Depth-First Search, Breadth-First Search, Union Find, Graph

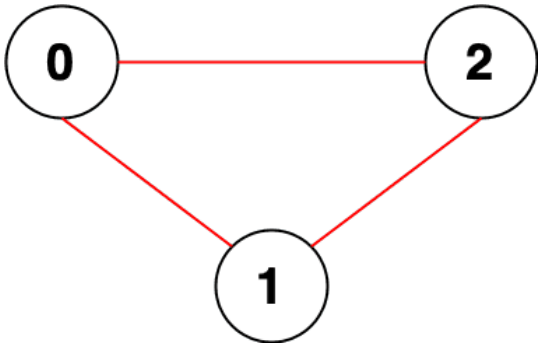
leetcode.com/problems/count-unreachable-pairs-of-nodes-in-an-undirected-graph

PROBLEM

You are given an integer `n`. There is an **undirected** graph with `n` nodes, numbered from `0` to `n - 1`. You are given a 2D integer array `edges` where `edges[i] = [a_i, b_i]` denotes that there exists an **undirected** edge connecting nodes `a_i` and `b_i`.

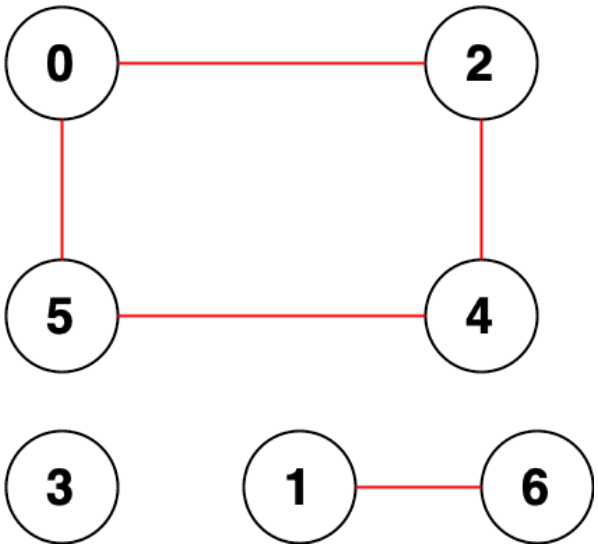
Return *the **number of pairs** of different nodes that are **unreachable** from each other*.

Example 1:




```
Input: n = 3, edges = [[0,1],[0,2],[1,2]]
Output: 0
Explanation: There are no pairs of nodes that are unreachable from each other. Therefore, we return 0.
```

Example 2:




```
Input: n = 7, edges = [[0,2],[0,5],[2,4],[1,6],[5,4]]
Output: 14
Explanation: There are 14 pairs of nodes that are unreachable from each other:
[[0,1],[0,3],[0,6],[1,2],[1,3],[1,4],[1,5],[2,3],[2,6],[3,4],[3,5],[3,6],[4,6],[5,6]].
Therefore, we return 14.
```

Constraints:

- $1 <= n <= 10^5$
- $0 <= edges.length <= 2 * 10^5$
- $edges[i].length == 2$
- $0 <= a_i, b_i < n$
- $a_i != b_i$
- There are no repeated edges.

SOLUTIONS (PYTHON 3)

Solution 1: DFS

How to think about it

Unreachable pairs sit in different components. $n \le 10^5$, so we cannot enumerate pairs. The count is the product of component sizes. DFS yields each component size t. Multiply by the sum s of earlier sizes, then add t into s. Each vertex and edge is visited once.

For any two nodes in an undirected graph, if there is a path between them, then they are mutually reachable.

Therefore, we can use depth-first search to find the number of nodes t in each connected component, and then multiply the current number of nodes t in the connected component by the number of nodes s in all previous connected components to obtain the number of unreachable node pairs in the current connected component, which is s × t. Then, we add t to s and continue to search for the next connected component until all connected components have been searched, and we can obtain the final answer.

The time complexity is O(n + m), and the space complexity is O(n + m). Here, n and m are the number of nodes and edges, respectively.

Python 3

```python
class Solution:
    def countPairs(self, n: int, edges: List[List[int]]) -> int:
        def dfs(i: int) -> int:
            if vis[i]:
                return 0
            vis[i] = True
            return 1 + sum(dfs(j) for j in g[i])

        g = [[] for _ in range(n)]
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)
        vis = [False] * n
        ans = s = 0
        for i in range(n):
            t = dfs(i)
            ans += s * t
            s += t
        return ans
```

Solution 2: Explicit Stack

How to think about it

Unreachable pairs sit in different components. With $n \le 10^5$ we cannot enumerate pairs. The count is the product of component sizes. Recursing from a node into each neighbor reaches depth equal to the component size on a chain. In Python the generator inside `sum` adds another frame, so a few hundred nodes already overflow the call stack. A component's size does not depend on visit order. Each component therefore uses an explicit stack: a node is marked when it is pushed, counted when it is popped, and only unmarked neighbors are pushed. Multiply the size t by the sum s of earlier sizes, then add t into s.

For any two nodes in an undirected graph, if there is a path between them, then they are mutually reachable.

An explicit stack finds the number of nodes t in each connected component. Starting from an unvisited node, we mark a node when pushing it, count it when popping it, and push only unmarked neighbors. Multiply t by the number of nodes s in all previous components to obtain the unreachable pairs s × t, then add t to s. Repeat until every node has been visited.

The time complexity is O(n + m), and the space complexity is O(n + m). Here, n and m are the number of nodes and edges, respectively.

Python 3

```python
class Solution:
    def countPairs(self, n: int, edges: List[List[int]]) -> int:
        def dfs(i: int) -> int:
            if vis[i]:
                return 0
            vis[i] = True
            stk = [i]
            cnt = 0
            while stk:
                u = stk.pop()
                cnt += 1
                for j in g[u]:
                    if not vis[j]:
                        vis[j] = True
                        stk.append(j)
            return cnt

        g = [[] for _ in range(n)]
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)
        vis = [False] * n
        ans = s = 0
        for i in range(n):
            t = dfs(i)
            ans += s * t
            s += t
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count the pairs that ARE reachable
# 2. List the sizes of the components, largest first


# ============================================================
# FOLLOW-UP 1: Count the pairs that ARE reachable
# ============================================================

# ---- follow-up 1: function ----
# Answer: Add up size * (size - 1) / 2 over the
# components.
# Time: O((n + e) * alpha(n)).  Space: O(n).
def reachable_pairs(n, edges):
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for a, b in edges:
        parent[find(a)] = find(b)
    size = {}
    for x in range(n):
        size[find(x)] = size.get(find(x), 0) + 1
```

```
    total = 0
    for s in size.values():
        total += s * (s - 1) // 2
    return total


# ============================================================
# FOLLOW-UP 2: List the sizes of the components, largest
# first
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the nodes per root and sort.
# Time: O((n + e) * alpha(n) + c log c).  Space: O(n).
def component_sizes(n, edges):
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for a, b in edges:
        parent[find(a)] = find(b)
    size = {}
    for x in range(n):
        size[find(x)] = size.get(find(x), 0) + 1
    return sorted(size.values(), reverse=True)
```

924. Minimize Malware Spread

Hard · Depth-First Search, Breadth-First Search, Union Find, Graph, Array, Hash Table

leetcode.com/problems/minimize-malware-spread

PROBLEM

You are given a network of `n` nodes represented as an `n x n` adjacency matrix `graph`, where the i^{th} node is directly connected to the j^{th} node if `graph[i][j] == 1`.

Some nodes `initial` are initially infected by malware. Whenever two nodes are directly connected, and at least one of those two nodes is infected by malware, both nodes will be infected by malware. This spread of malware will continue until no more nodes can be infected in this manner.

Suppose `M(initial)` is the final number of nodes infected with malware in the entire network after the spread of malware stops. We will remove **exactly one node** from `initial`.

Return the node that, if removed, would minimize `M(initial)`. If multiple nodes could be removed to minimize `M(initial)`, return such a node with **the smallest index**.

Note that if a node was removed from the `initial` list of infected nodes, it might still be infected later due to the malware spread.

Example 1:

```
Input: graph = [[1,1,0],[1,1,0],[0,0,1]], initial = [0,1]
Output: 0
```

Example 2:

```
Input: graph = [[1,0,0],[0,1,0],[0,0,1]], initial = [0,2]
Output: 0
```

Example 3:

```
Input: graph = [[1,1,1],[1,1,1],[1,1,1]], initial = [1,2]
Output: 1
```

Constraints:

- `n == graph.length`
- `n == graph[i].length`
- `2 <= n <= 300`
- `graph[i][j]` is `0` or `1`.
- `graph[i][j] == graph[j][i]`
- `graph[i][i] == 1`
- `1 <= initial.length <= n`
- `0 <= initial[i] <= n - 1`
- All the integers in `initial` are **unique**.

SOLUTION (PYTHON 3)

Solution 1: Union Find

How to think about it

Remove one initially infected node to minimize the final infection. Infection fills a whole component: if a component has two or more initial nodes, removing one still leaves it infected; only a component with a single initial node can be saved entirely. Union-find gives component sizes and the number of initials per root. Among nodes with cnt[root]=1, pick the one that saves the most nodes (smallest index on ties). If none exist, return the minimum of initial.

According to the problem description, if there are several nodes in the same connected component initially, there can be three situations:

1. None of these nodes are infected.
2. Only one node among these nodes is infected.
3. Multiple nodes among these nodes are infected.

What we need to consider is to minimize the number of infected nodes left after removing a certain infected node.

For situation 1, there are no infected nodes, so we don't need to consider it; for situation 2, only one node is infected, so after removing this node, the other nodes in this connected component will not be infected; for situation 3, multiple nodes are infected, so after removing any infected node, the other nodes in this connected component will still be infected. Therefore, we only need to consider situation 2.

We use a union find set uf to maintain the connectivity of nodes, a variable ans to record the answer, and a variable mx to record the maximum number of infections that can be reduced currently. Initially, ans = n, mx = 0.

Then we traverse the array initial, use a hash table or an array of length n named cnt to count the number of infected nodes in each connected component.

Next, we traverse the array initial again. For each node x, we find the root node root of its connected component. If there is only one infected node in this connected component, i.e., cnt[root] = 1, we update the answer. The update condition is that the number of nodes sz in this connected component is greater than mx or sz equals mx and the value of x is less than ans.

Finally, if ans has not been updated, it means that there are multiple infected nodes in all connected components, so we return the minimum value in initial, otherwise, we return ans.

The time complexity is $O(n^2 \times \alpha(n))$, and the space complexity is $O(n)$. Where n is the number of nodes, and $\alpha(n)$ is the inverse of the Ackermann function.

Python 3

```python
class UnionFind:
    __slots__ = "p", "size"

    def __init__(self, n: int):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x: int) -> int:
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True

    def get_size(self, root: int) -> int:
        return self.size[root]


class Solution:
    def minMalwareSpread(self, graph: List[List[int]], initial: List[int]) -> int:
        n = len(graph)
        uf = UnionFind(n)
        for i in range(n):
            for j in range(i + 1, n):
                graph[i][j] and uf.union(i, j)
        cnt = Counter(uf.find(x) for x in initial)
        ans, mx = n, 0
        for x in initial:
            root = uf.find(x)
            if cnt[root] > 1:
                continue
            sz = uf.get_size(root)
            if sz > mx or (sz == mx and x < ans):
                ans = x
                mx = sz
        return min(initial) if ans == n else ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. How many nodes end up infected if nothing is removed
# 2. For every node in initial, how many nodes would be
# saved by removing it


# ============================================================
# FOLLOW-UP 1: How many nodes end up infected if nothing
# is removed
# ============================================================

# ---- follow-up 1: function ----
# Answer: Join the graph; every component that contains at
# least one initially infected node is fully infected.
# Time: O(n^2 * alpha(n)).  Space: O(n).
def infected_without_removal(graph, initial):
```

```
    n = len(graph)
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for i in range(n):
        for j in range(i + 1, n):
            if graph[i][j] == 1:
                parent[find(i)] = find(j)
    size = {}
    for x in range(n):
        size[find(x)] = size.get(find(x), 0) + 1
    infected_roots = set(find(x) for x in initial)
    return sum(size[r] for r in infected_roots)


# ============================================================
# FOLLOW-UP 2: For every node in initial, how many nodes
# would be saved by removing it
# ============================================================

# ---- follow-up 2: function ----
# Answer: A node saves its whole component only when it is
# the only infected node there.
# Time: O(n^2 * alpha(n)).  Space: O(n).
def saved_by_each_removal(graph, initial):
    n = len(graph)
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for i in range(n):
        for j in range(i + 1, n):
            if graph[i][j] == 1:
                parent[find(i)] = find(j)
    size = {}
```

```
    for x in range(n):
        size[find(x)] = size.get(find(x), 0) + 1
    infected_in = {}
    for x in initial:
        infected_in[find(x)] = infected_in.get(find(x), 0) + 1
    saved = {}
    for x in initial:
        root = find(x)
        saved[x] = size[root] - 1 if infected_in[root] == 1 else 0
    return saved
```

959. Regions Cut By Slashes

Medium · Depth-First Search, Breadth-First Search, Union Find, Array, Hash Table, Matrix, Planar Graph

leetcode.com/problems/regions-cut-by-slashes

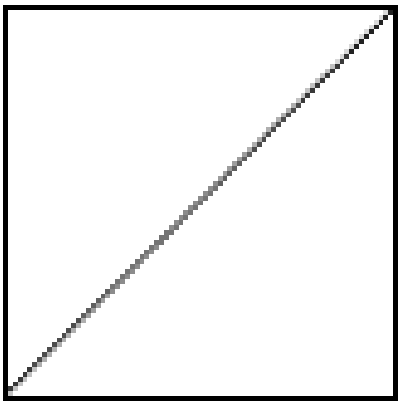
PROBLEM

An `n x n` grid is composed of `1 x 1` squares where each `1 x 1` square consists of a `'/'`, `'\'`, or blank space `' '`. These characters divide the square into contiguous regions.

Given the grid `grid` represented as a string array, return *the number of regions*.

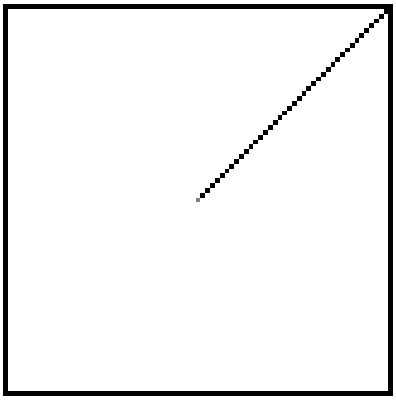
Note that backslash characters are escaped, so a `'\'` is represented as `'\\'`.

Example 1:



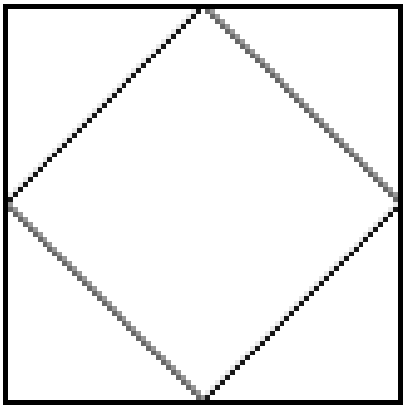
```
Input: grid = [" /","/ "]
Output: 2
```

Example 2:



```
Input: grid = [" /"," "]
Output: 1
```

Example 3:



```
Input: grid = ["/\\","\\/"]
Output: 5
Explanation: Recall that because \ characters are escaped, "\\/" refers to \/, and "/\\" refers to /\.
```

Constraints:

- `n == grid.length == grid[i].length`
- `1 <= n <= 30`
- `grid[i][j]` is either `'/'`, `'\'`, or `' '`.

SOLUTIONS (PYTHON 3)

Solution 1: Union-Find

How to think about it

Slashes cut the grid into regions, n≤ 30. Split each cell into four triangles. Merge pieces that share an open edge: inside a cell according to `'/'`, `'\\'`, or a blank, and between cells along the shared side. Union-find starts at $4n^2$ pieces and decreases on each successful union.

Python 3

```python
class Solution:
    def regionsBySlashes(self, grid: List[str]) -> int:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        def union(a, b):
            pa, pb = find(a), find(b)
            if pa != pb:
                p[pa] = pb
                nonlocal size
                size -= 1

        n = len(grid)
        size = n * n * 4
        p = list(range(size))
        for i, row in enumerate(grid):
            for j, v in enumerate(row):
                k = i * n + j
                if i < n - 1:
                    union(4 * k + 2, (k + n) * 4)
                if j < n - 1:
                    union(4 * k + 1, (k + 1) * 4 + 3)
                if v == '/':
                    union(4 * k, 4 * k + 3)
                    union(4 * k + 1, 4 * k + 2)
                elif v == '\\':
                    union(4 * k, 4 * k + 1)
                    union(4 * k + 2, 4 * k + 3)
                else:
                    union(4 * k, 4 * k + 1)
                    union(4 * k + 1, 4 * k + 2)
                    union(4 * k + 2, 4 * k + 3)
        return size
```

Complexity. Time: O(n^2 * alpha(n^2)), essentially O(n^2). Space: O(n^2).

Solution 2: DFS

> **How to think about it**
>
> Union-find treats a region as a set. We can instead refine each cell into a 2× 2 and DFS the empty squares; slashes become walls, and diagonal empties are entered only through a slash opening. The number of components is the answer.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```
class Solution:
    def regionsBySlashes(self, grid: List[str]) -> int:
        # every square becomes 2 x 2 cells; 0 = open, 1 and 2 mark the two diagonal directions, -1 =
visited
        size = len(grid)
        n = size * 2
        g = [[0] * n for _ in range(n)]
        for i in range(size):
            for j in range(size):
                y, x = i * 2, j * 2
                if grid[i][j] == "/":
                    g[y][x + 1] = g[y + 1][x] = 1
                elif grid[i][j] == "\\":
                    g[y][x] = g[y + 1][x + 1] = 2

        def valid(v: int) -> bool:
            return 0 <= v < n

        def dfs(i: int, j: int) -> None:
            if not valid(i) or not valid(j) or g[i][j]:
                return
            g[i][j] = -1
            dirs = [-1, 0, 1, 0, -1]
            neighbours = []
            for d in range(4):
                y, x = i + dirs[d], j + dirs[d + 1]
                if valid(y) and valid(x):
                    dfs(y, x)
                    neighbours.append(g[y][x])
                else:
                    neighbours.append(-1)
            top, right, bottom, left = neighbours
            if top == 1 and right == 1:
                dfs(i - 1, j + 1)
            if bottom == 1 and left == 1:
                dfs(i + 1, j - 1)
            if top == 2 and left == 2:
                dfs(i - 1, j - 1)
            if bottom == 2 and right == 2:
                dfs(i + 1, j + 1)

        regions = 0
        for i in range(n):
            for j in range(n):
                if g[i][j] == 0:
                    dfs(i, j)
                    regions += 1
        return regions
```

Complexity. Time: O(n^2). Space: O(n^2).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the size of every region, counted in quarter
# squares
# 2. The cell may also be a cross 'X' (both slashes)


# ============================================================
# FOLLOW-UP 1: Return the size of every region, counted in
# quarter squares
# ============================================================

# ---- follow-up 1: function ----
# Answer: Split every square into four triangles, join
```

```
# triangles that touch, and count the triangles per group.
# Time: O(n^2 * alpha(n^2) + n^2 log n^2).  Space: O(n^2).
def region_sizes_in_quarters(grid):
    n = len(grid)
    parent = list(range(4 * n * n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    def join(a, b):
        parent[find(a)] = find(b)

    # triangle numbers in a square: 0 top, 1 right, 2 bottom, 3 left
    for r in range(n):
        for c in range(n):
            base = 4 * (r * n + c)
            if grid[r][c] == " ":
                join(base, base + 1)
                join(base + 1, base + 2)
                join(base + 2, base + 3)
            elif grid[r][c] == "/":
                join(base, base + 3)
                join(base + 1, base + 2)
            else:
                join(base, base + 1)
                join(base + 2, base + 3)
            if r + 1 < n:
                join(base + 2, 4 * ((r + 1) * n + c))
            if c + 1 < n:
                join(base + 1, 4 * (r * n + c + 1) + 3)
    count = {}
    for x in range(4 * n * n):
        count[find(x)] = count.get(find(x), 0) + 1
    return sorted(count.values())


# ============================================================
# FOLLOW-UP 2: The cell may also be a cross 'X' (both
# slashes)
# ============================================================

# ---- follow-up 2: function ----
# Answer: A cross cuts the square into its four triangles,
# so join nothing inside that square.
```

```
# Time: O(n^2 * alpha(n^2)).  Space: O(n^2).
def regions_with_crosses(grid):
    n = len(grid)
    parent = list(range(4 * n * n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    def join(a, b):
        parent[find(a)] = find(b)

    for r in range(n):
        for c in range(n):
            base = 4 * (r * n + c)
            if grid[r][c] == " ":
                join(base, base + 1)
                join(base + 1, base + 2)
                join(base + 2, base + 3)
```

```
            elif grid[r][c] == "/":
                join(base, base + 3)
                join(base + 1, base + 2)
            elif grid[r][c] == "\\":
                join(base, base + 1)
                join(base + 2, base + 3)
            if r + 1 < n:
                join(base + 2, 4 * ((r + 1) * n + c))
            if c + 1 < n:
                join(base + 1, 4 * (r * n + c + 1) + 3)
    return len(set(find(x) for x in range(4 * n * n)))
```

1101. The Earliest Moment When Everyone Become Friends

Medium · Union Find, Array, Sorting

leetcode.com/problems/the-earliest-moment-when-everyone-become-friends

PROBLEM

There are n people in a social group labeled from `0` to `n - 1`. You are given an array `logs` where `logs[i] = [timestamp`$_i$`, x`$_i$`, y`$_i$`]` indicates that x_i and y_i will be friends at the time $timestamp_i$.

Friendship is **symmetric**. That means if `a` is friends with `b`, then `b` is friends with `a`. Also, person `a` is acquainted with a person `b` if `a` is friends with `b`, or `a` is a friend of someone acquainted with `b`.

Return *the earliest time for which every person became acquainted with every other person*. If there is no such earliest time, return `-1`.

Example 1:

```
Input: logs = [[20190101,0,1],[20190104,3,4],[20190107,2,3],[20190211,1,5],[20190224,2,4],[20190301,0,3],[20190312,1,2],[20190322,4,5]], n = 6
Output: 20190301
Explanation:
The first event occurs at timestamp = 20190101, and after 0 and 1 become friends, we have the following friendship groups [0,1], [2], [3], [4], [5].
The second event occurs at timestamp = 20190104, and after 3 and 4 become friends, we have the following friendship groups [0,1], [2], [3,4], [5].
The third event occurs at timestamp = 20190107, and after 2 and 3 become friends, we have the following friendship groups [0,1], [2,3,4], [5].
The fourth event occurs at timestamp = 20190211, and after 1 and 5 become friends, we have the following friendship groups [0,1,5], [2,3,4].
The fifth event occurs at timestamp = 20190224, and as 2 and 4 are already friends, nothing happens.
The sixth event occurs at timestamp = 20190301, and after 0 and 3 become friends, we all become friends.
```

Example 2:

```
Input: logs = [[0,2,0],[1,0,1],[3,0,3],[4,1,2],[7,3,1]], n = 4
Output: 3
Explanation: At timestamp = 3, all the persons (i.e., 0, 1, 2, and 3) become friends.
```

Constraints:

- `2 <= n <= 100`
- `1 <= logs.length <=` 10^4
- `logs[i].length == 3`
- `0 <=` $timestamp_i$ `<=` 10^9
- `0 <=` x_i`, `y_i `<= n - 1`
- x_i `!=` y_i
- All the values $timestamp_i$ are **unique**.
- All the pairs (x_i, y_i) occur at most one time in the input.

SOLUTIONS (PYTHON 3)

Solution 1: Sorting + Union-Find

How to think about it

We need the earliest time when all n people lie in one connected component. Logs arrive unordered, so sort them by timestamp and process in order. If the two people are still in different sets, union them and decrement the component count. Union-find makes the find/merge step nearly constant. When the count reaches 1, everyone is connected and that timestamp is the answer; if logs run out first, return -1.

We sort all the logs in ascending order by timestamp, then traverse the sorted logs. Using a union-find set, we check whether the two people in the current log are already friends. If they are not friends, we merge them into one friend circle, until everyone is in one friend circle, then return the timestamp of the current log.

If we have traversed all the logs and not everyone is in one friend circle, then return -1.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the number of logs.

Python 3

```python
class Solution:
    def earliestAcq(self, logs: List[List[int]], n: int) -> int:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(n))
        for t, x, y in sorted(logs):
            if find(x) == find(y):
                continue
            p[find(x)] = find(y)
            n -= 1
            if n == 1:
                return t
        return -1
```

Solution 2

How to think about it

Method 1 inlines path compression and does not union by size. Method 2 wraps a `UnionFind` with path compression and union-by-size; `union` reports whether a merge happened, and the main loop only tracks the component count. The moment everyone becomes friends is unchanged, with shallower trees.

Python 3

```python
class UnionFind:
    __slots__ = ('p', 'size')

    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x: int) -> int:
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True


class Solution:
    def earliestAcq(self, logs: List[List[int]], n: int) -> int:
        uf = UnionFind(n)
        for t, x, y in sorted(logs):
            if uf.union(x, y):
                n -= 1
                if n == 1:
                    return t
        return -1
```

Complexity. Time: O(m log m + m * alpha(n)), m = number of logs. Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Which two people's meeting finally united everyone
# 2. The earliest time two given people are connected
# through friends


# ============================================================
# FOLLOW-UP 1: Which two people's meeting finally united
# everyone
# ============================================================

# ---- follow-up 1: function ----
# Answer: Process the logs by time; the union that brings
# the number of groups down to one is the answer.
# Time: O(L log L + L * alpha(n)).  Space: O(n).
def final_friendship(logs, n):
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    groups = n
    for time, a, b in sorted(logs):
        ra, rb = find(a), find(b)
```

```
        if ra != rb:
            parent[ra] = rb
            groups -= 1
            if groups == 1:
                return time, a, b
    return None


# ============================================================
# FOLLOW-UP 2: The earliest time two given people are
# connected through friends
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same sweep, stop when the two given people share
# a root.
# Time: O(L log L + L * alpha(n)).  Space: O(n).
def earliest_moment_connected(logs, n, person_a, person_b):
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    if person_a == person_b:
        return 0
    for time, a, b in sorted(logs):
        parent[find(a)] = find(b)
        if find(person_a) == find(person_b):
            return time
    return -1
```

685. Redundant Connection II

Hard · Depth-First Search, Breadth-First Search, Union Find, Graph

leetcode.com/problems/redundant-connection-ii

★ **TOP 7 PROBLEM** in Union-Find (Disjoint Set): master this one first

PROBLEM

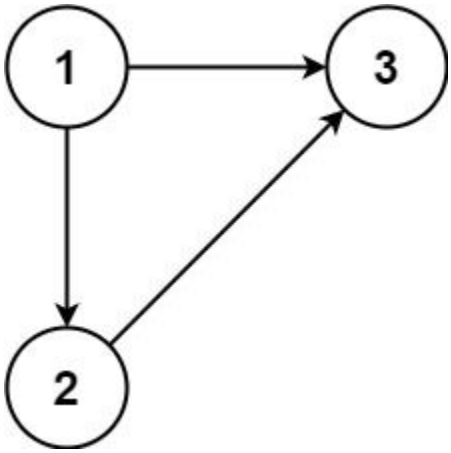
In this problem, a rooted tree is a **directed** graph such that, there is exactly one node (the root) for which all other nodes are descendants of this node, plus every node has exactly one parent, except for the root node which has no parents.

The given input is a directed graph that started as a rooted tree with `n` nodes (with distinct values from `1` to `n`), with one additional directed edge added. The added edge has two different vertices chosen from `1` to `n`, and was not an edge that already existed.

The resulting graph is given as a 2D-array of `edges`. Each element of `edges` is a pair $[u_i, v_i]$ that represents a **directed** edge connecting nodes u_i and v_i, where u_i is a parent of child v_i.

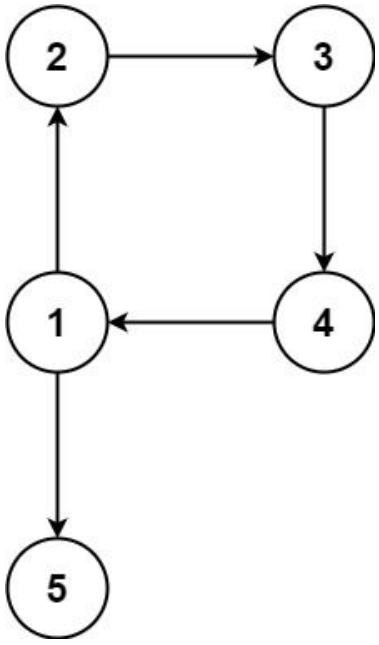
Return *an edge that can be removed so that the resulting graph is a rooted tree of* `n` *nodes*. If there are multiple answers, return the answer that occurs last in the given 2D-array.

Example 1:




```
Input: edges = [[1,2],[1,3],[2,3]]
Output: [2,3]
```

Example 2:




```
Input: edges = [[1,2],[2,3],[3,4],[4,1],[1,5]]
Output: [4,1]
```

Constraints:

- `n == edges.length`
- `3 <= n <= 1000`
- `edges[i].length == 2`
- `1 <= u_i, v_i <= n`
- `u_i != v_i`

SOLUTIONS (PYTHON 3)

Solution 1: Union-Find

How to think about it

A rooted tree plus one directed edge may create a node of in-degree 2, a cycle into the root, or both. Undirected union-find alone misses the double-parent case. Collect the two edges into an in-degree-2 node. Skip the later one and union-find: a cycle means the earlier edge must go. With no double parent, delete the first edge that closes a cycle.

According to the problem description, for a rooted tree, the in-degree of the root node is 0, and the in-degree of other nodes is 1. After adding an edge to the tree, there can be the following three scenarios:

1. The added edge points to a non-root node, and the in-degree of that node becomes 2. In this case, there is no directed cycle in the graph:

```
  1
 / \
v   v
2-->3
```

2. The added edge points to a non-root node, and the in-degree of that node becomes 2. In this case, there is a directed cycle in the graph:

```
  1
  |
  v
  2 <--> 3
```

3. The added edge points to the root node, and the in-degree of the root node becomes 1. In this case, there is a directed cycle in the graph, but there are no nodes with an in-degree of 2.

```
  1
  /^
v  \
2-->3
```

Therefore, we first calculate the in-degree of each node. If there exists a node with an in-degree of 2, we identify the two edges corresponding to that node, denoted as dup[0] and dup[1]. If deleting dup[1] results in the remaining edges not forming a tree, then dup[0] is the edge that needs to be deleted; otherwise, dup[1] is the edge that needs to be deleted.

If there are no nodes with an in-degree of 2, we traverse the array edges. For each edge (u, v), we use the union-find data structure to maintain connectivity between nodes. If u and v are already connected, it indicates that there is a directed cycle in the graph, and the current edge is the one that needs to be deleted.

The time complexity is O(n log n), and the space complexity is O(n), where n is the number of edges.

Python 3

```python
class Solution:
    def findRedundantDirectedConnection(self, edges: List[List[int]]) -> List[int]:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(edges)
        ind = [0] * n
        for _, v in edges:
            ind[v - 1] += 1
        dup = [i for i, (_, v) in enumerate(edges) if ind[v - 1] == 2]
        p = list(range(n))
        if dup:
            for i, (u, v) in enumerate(edges):
                if i == dup[1]:
                    continue
                pu, pv = find(u - 1), find(v - 1)
                if pu == pv:
                    return edges[dup[0]]
                p[pu] = pv
            return edges[dup[1]]
        for i, (u, v) in enumerate(edges):
            pu, pv = find(u - 1), find(v - 1)
            if pu == pv:
                return edges[i]
            p[pu] = pv
```

Solution 2: Union-Find (Template Approach)

> **How to think about it**
>
> The same case split, with path compression and union-by-size in a `UnionFind` class. The driver only handles `dup` and a failed union.

Here is a template approach using Union-Find for your reference.

The time complexity is O(n α(n)), and the space complexity is O(n). Here, n is the number of edges, and α(n) is the inverse Ackermann function, which can be considered a very small constant.

Python 3

```python
class UnionFind:
    __slots__ = "p", "size"

    def __init__(self, n: int):
        self.p: List[int] = list(range(n))
        self.size: List[int] = [1] * n

    def find(self, x: int) -> int:
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True


class Solution:
    def findRedundantDirectedConnection(self, edges: List[List[int]]) -> List[int]:
        n = len(edges)
        ind = [0] * n
        for _, v in edges:
            ind[v - 1] += 1
        dup = [i for i, (_, v) in enumerate(edges) if ind[v - 1] == 2]
        uf = UnionFind(n)
        if dup:
            for i, (u, v) in enumerate(edges):
                if i == dup[1]:
                    continue
                if not uf.union(u - 1, v - 1):
                    return edges[dup[0]]
            return edges[dup[1]]
        for i, (u, v) in enumerate(edges):
            if not uf.union(u - 1, v - 1):
                return edges[i]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Check that a directed graph is a rooted tree
# 2. List EVERY edge whose removal leaves a rooted tree


# ============================================================
# FOLLOW-UP 1: Check that a directed graph is a rooted
# tree
# ============================================================

# ---- follow-up 1: function ----
# Answer: Exactly one node has no parent, every other node
# has exactly one parent, and everything is reachable from
# the root.
# Time: O(n).  Space: O(n).
def is_rooted_tree(n, edges):
    parent_of = {}
    children = {}
    for a, b in edges:
```

```
        if b in parent_of:
            return False
        parent_of[b] = a
        children.setdefault(a, []).append(b)
    roots = [x for x in range(1, n + 1) if x not in parent_of]
    if len(roots) != 1:
        return False
    seen = {roots[0]}
    stack = [roots[0]]
    while stack:
        node = stack.pop()
        for nxt in children.get(node, []):
            if nxt not in seen:
                seen.add(nxt)
                stack.append(nxt)
    return len(seen) == n


# ============================================================
# FOLLOW-UP 2: List EVERY edge whose removal leaves a
# rooted tree
# ============================================================

# ---- follow-up 2: function ----
# Answer: Try removing each edge and test with the checker
# above.
# Time: O(n^2) (the tree check is repeated for every
# edge).  Space: O(n).
def removable_edges(edges):
    n = len(edges)
    result = []
    for i in range(len(edges)):
        rest = edges[:i] + edges[i + 1:]
        if is_rooted_tree(n, rest):
            result.append(edges[i])
    return result
```

1627. Graph Connectivity With Threshold

Hard · Union Find, Array, Math, Number Theory

leetcode.com/problems/graph-connectivity-with-threshold

PROBLEM

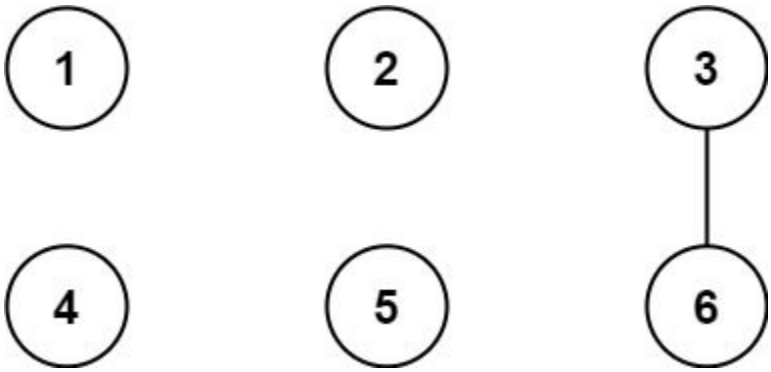
We have `n` cities labeled from `1` to `n`. Two different cities with labels `x` and `y` are directly connected by a bidirectional road if and only if `x` and `y` share a common divisor **strictly greater** than some `threshold`. More formally, cities with labels `x` and `y` have a road between them if there exists an integer `z` such that all of the following are true:

- `x % z == 0`,
- `y % z == 0`, and
- `z > threshold`.

Given the two integers, `n` and `threshold`, and an array of `queries`, you must determine for each `queries[i] = [a_i, b_i]` if cities `a_i` and `b_i` are connected directly or indirectly. (i.e. there is some path between them).

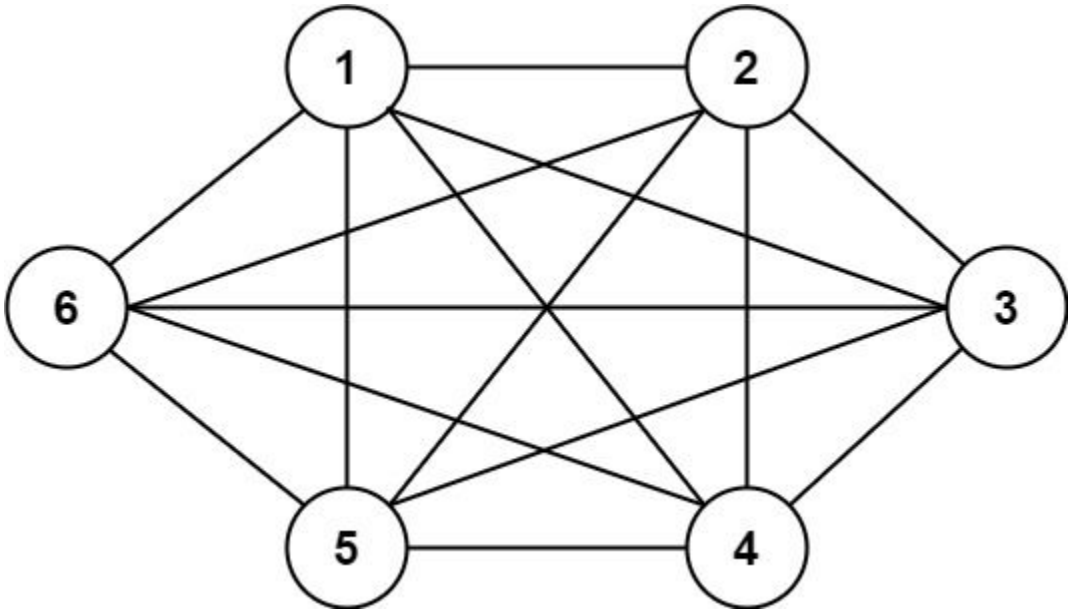
Return *an array* `answer`*, where* `answer.length == queries.length` *and* `answer[i]` *is* `true` *if for the* `i^th` *query, there is a path between* `a_i` *and* `b_i`*, or* `answer[i]` *is* `false` *if there is no path.*

Example 1:



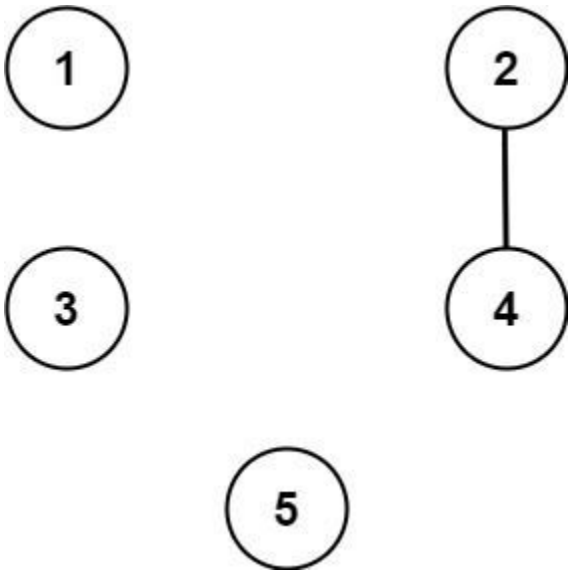

```
Input: n = 6, threshold = 2, queries = [[1,4],[2,5],[3,6]]
Output: [false,false,true]
Explanation: The divisors for each number:
1: 1
2: 1, 2
3: 1, 3
4: 1, 2, 4
5: 1, 5
6: 1, 2, 3, 6
Using the underlined divisors above the threshold, only cities 3 and 6 share a common divisor, so they
are the
only ones directly connected. The result of each query:
[1,4] 1 is not connected to 4
[2,5] 2 is not connected to 5
[3,6] 3 is connected to 6 through path 3--6
```

Example 2:



Input: n = 6, threshold = 0, queries = [[4,5],[3,4],[3,2],[2,6],[1,3]]
Output: [true,true,true,true,true]
Explanation: The divisors for each number are the same as the previous example. However, since the threshold is 0,
all divisors can be used. Since all numbers share 1 as a divisor, all cities are connected.

Example 3:



Input: n = 5, threshold = 1, queries = [[4,5],[4,5],[3,2],[2,3],[3,4]]
Output: [false,false,false,false,false]
Explanation: Only cities 2 and 4 share a common divisor 2 which is strictly greater than the threshold 1, so they are the only ones directly connected.
Please notice that there can be multiple queries for the same pair of nodes [x, y], and that the query [x, y] is equivalent to the query [y, x].

Constraints:

- $2 <= n <= 10^4$
- $0 <= threshold <= n$
- $1 <= queries.length <= 10^5$
- $queries[i].length == 2$

- $1 <= a_i, b_i <= cities$
- $a_i != b_i$

SOLUTION (PYTHON 3)

Solution 1: Union-Find

> **How to think about it**
>
> Two cities are connected if a path exists using edges whose gcd exceeds threshold. Testing gcd on every pair is too slow when both n and the number of queries are large. Each z above the threshold links all of its multiples, so unioning along multiples covers every direct edge. A disjoint-set unions z,2z,3z,… for z ∈ (threshold, n], and each query tests whether the two cities share a root.

We can enumerate z and its multiples, and use union-find to connect them. In this way, for each query [a, b], we only need to determine whether a and b are in the same connected component.

The time complexity is O(n × log n × (α(n) + q)), and the space complexity is O(n). Here, n and q are the number of nodes and queries, respectively, and α is the inverse function of the Ackermann function.

Python 3

```python
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True


class Solution:
    def areConnected(
        self, n: int, threshold: int, queries: List[List[int]]
    ) -> List[bool]:
        uf = UnionFind(n + 1)
        for a in range(threshold + 1, n + 1):
            for b in range(a + a, n + 1, a):
                uf.union(a, b)
        return [uf.find(a) == uf.find(b) for a, b in queries]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. How many separate groups of cities are there
# 2. Size of the biggest group


# =============================================================
# FOLLOW-UP 1: How many separate groups of cities are
# there
# =============================================================

# ---- follow-up 1: function ----
# Answer: Join each number to its multiples (above the
# threshold), then count roots.
# Time: O(n log n * alpha(n)).  Space: O(n).
def number_of_city_groups(n, threshold):
    parent = list(range(n + 1))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for divisor in range(threshold + 1, n + 1):
        for multiple in range(2 * divisor, n + 1, divisor):
            parent[find(multiple)] = find(divisor)
    return len(set(find(x) for x in range(1, n + 1)))


# =============================================================
# FOLLOW-UP 2: Size of the biggest group
# =============================================================

# ---- follow-up 2: function ----
# Answer: Count the cities per root.
# Time: O(n log n * alpha(n)).  Space: O(n).
def biggest_city_group(n, threshold):
    parent = list(range(n + 1))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for divisor in range(threshold + 1, n + 1):
        for multiple in range(2 * divisor, n + 1, divisor):
            parent[find(multiple)] = find(divisor)
    count = {}
    for x in range(1, n + 1):
        count[find(x)] = count.get(find(x), 0) + 1
    return max(count.values())
```

1697. Checking Existence of Edge Length Limited Paths

Hard · Union Find, Graph, Array, Two Pointers, Sorting

leetcode.com/problems/checking-existence-of-edge-length-limited-paths

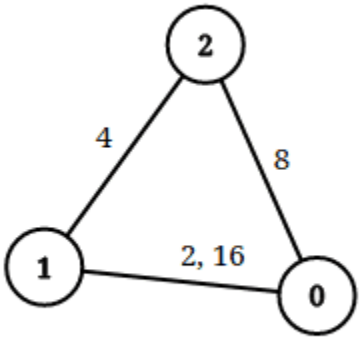
PROBLEM

An undirected graph of `n` nodes is defined by `edgeList`, where `edgeList[i] = [u_i, v_i, dis_i]` denotes an edge between nodes `u_i` and `v_i` with distance `dis_i`. Note that there may be **multiple** edges between two nodes.

Given an array `queries`, where `queries[j] = [p_j, q_j, limit_j]`, your task is to determine for each `queries[j]` whether there is a path between `p_j` and `q_j` such that each edge on the path has a distance **strictly less than** `limit_j`.

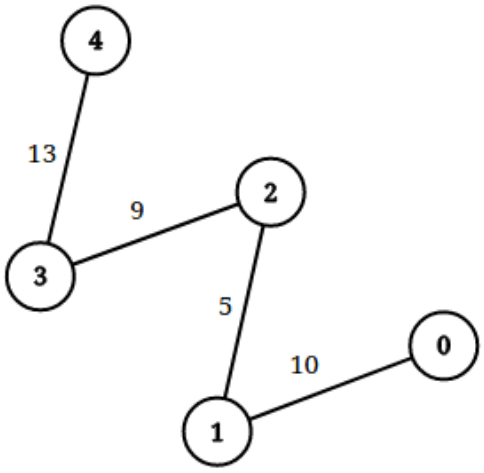
Return *a **boolean array*** `answer`*, where* `answer.length == queries.length` *and the* `j^th` *value of* `answer` *is* `true` *if there is a path for* `queries[j]` *is* `true`*, and* `false` *otherwise*.

Example 1:




```
Input: n = 3, edgeList = [[0,1,2],[1,2,4],[2,0,8],[1,0,16]], queries = [[0,1,2],[0,2,5]]
Output: [false,true]
Explanation: The above figure shows the given graph. Note that there are two overlapping edges between
0 and 1 with distances 2 and 16.
For the first query, between 0 and 1 there is no path where each distance is less than 2, thus we
return false for this query.
For the second query, there is a path (0 -> 1 -> 2) of two edges with distances less than 5, thus we
return true for this query.
```

Example 2:




```
Input: n = 5, edgeList = [[0,1,10],[1,2,5],[2,3,9],[3,4,13]], queries = [[0,4,14],[1,4,13]]
Output: [true,false]
Explanation: The above figure shows the given graph.
```

Constraints:

- $2 <= n <= 10^5$
- $1 <= edgeList.length, queries.length <= 10^5$
- $edgeList[i].length == 3$
- $queries[j].length == 3$
- $0 <= u_i, v_i, p_j, q_j <= n - 1$
- $u_i != v_i$
- $p_j != q_j$
- $1 <= dis_i, limit_j <= 10^9$
- There may be **multiple** edges between two nodes.

SOLUTION (PYTHON 3)

Solution 1: Offline Queries + Union-Find

How to think about it

Each query asks whether two vertices are connected by a path whose edges are all strictly below limit. Rebuilding a graph per query is too slow; queries are independent, so we go offline. Sort edges and queries by weight/limit. A two-pointer union-find adds every edge lighter than the current limit, then tests whether the two vertices share a root. Remember original query indices when writing the answer array.

According to the problem requirements, we need to judge each query queries[i], that is, to determine whether there is a path with edge weight less than or equal to limit between the two points a and b of the current query.

The connectivity of two points can be determined by a union-find set. Moreover, since the order of queries does not affect the result, we can sort all queries in ascending order by limit, and also sort all edges in ascending order by edge weight.

Then for each query, we start from the edge with the smallest weight, add all edges with weights strictly less than limit to the union-find set, and then use the query operation of the union-find set to determine whether the two points are connected.

The time complexity is O(m × log m + q × log q), where m and q are the number of edges and queries, respectively.

Python 3

```python
class Solution:
    def distanceLimitedPathsExist(
        self, n: int, edgeList: List[List[int]], queries: List[List[int]]
    ) -> List[bool]:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(n))
        edgeList.sort(key=lambda x: x[2])
        j = 0
        ans = [False] * len(queries)
        for i, (a, b, limit) in sorted(enumerate(queries), key=lambda x: x[1][2]):
            while j < len(edgeList) and edgeList[j][2] < limit:
                u, v, _ = edgeList[j]
                p[find(u)] = find(v)
                j += 1
            ans[i] = find(a) == find(b)
        return ans
```

Union-Find is a tree-like data structure that, as the name suggests, is used to handle some disjoint set **merge** and **query** problems. It supports two operations:

1. Find: Determine which subset an element belongs to. The time complexity of a single operation is O(α(n)).
2. Union: Merge two subsets into one set. The time complexity of a single operation is O(α(n)).

Here, α is the inverse Ackermann function, which grows extremely slowly. In other words, the average running time of its single operation can be considered a very small constant.

Below is a common template for Union-Find, which needs to be mastered proficiently. Where:

- `n` represents the number of nodes.
- `p` stores the parent node of each point. Initially, the parent node of each point is itself.
- `size` only makes sense when the node is an ancestor node, indicating the number of points in the set where the ancestor node is located.
- `find(x)` function is used to find the ancestor node of the set where x is located.
- `union(a, b)` function is used to merge the sets where a and b are located.

Python 3

```python
p = list(range(n))
size = [1] * n

def find(x):
    if p[x] != x:
        p[x] = find(p[x])
    return p[x]


def union(a, b):
    pa, pb = find(a), find(b)
    if pa == pb:
        return
    p[pa] = pb
    size[pb] += size[pa]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. For each query, the smallest limit that would make a
# path exist
# 2. Total length of a cheapest set of edges that connects
# everything


# ============================================================
# FOLLOW-UP 1: For each query, the smallest limit that
# would make a path exist
# ============================================================

# ---- follow-up 1: function ----
# Answer: Add edges from the shortest; the weight of the
# edge that first joins the two nodes is the answer.
# Time: O(E log E + Q * (n + E * alpha(n))).  Space: O(n +
# E).
def smallest_limit_per_query(n, edge_list, queries):
    edges = sorted(edge_list, key=lambda e: e[2])
    answers = []
    for p, q in queries:
        parent = list(range(n))

        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        if p == q:
            answers.append(0)
            continue
        found = -1
        for a, b, weight in edges:
            parent[find(a)] = find(b)
            if find(p) == find(q):
                found = weight
                break
        answers.append(found)
    return answers


# ============================================================
# FOLLOW-UP 2: Total length of a cheapest set of edges
# that connects everything
# ============================================================

# ---- follow-up 2: function ----
# Answer: Kruskal: sort the edges and keep every edge that
# joins two groups.
# Time: O(E log E + E * alpha(n)).  Space: O(n).
def minimum_spanning_total(n, edge_list):
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
```

```
    total = 0
    used = 0
    for a, b, weight in sorted(edge_list, key=lambda e: e[2]):
        ra, rb = find(a), find(b)
        if ra != rb:
```

```
            parent[ra] = rb
            total += weight
            used += 1
    if used < n - 1:
        return None
    return total
```

952. Largest Component Size by Common Factor

Hard · Union Find, Array, Hash Table, Math, Number Theory, Prime Factorization

leetcode.com/problems/largest-component-size-by-common-factor

PROBLEM

You are given an integer array of unique positive integers `nums`. Consider the following graph:

- There are `nums.length` nodes, labeled `nums[0]` to `nums[nums.length - 1]`,
- There is an undirected edge between `nums[i]` and `nums[j]` if `nums[i]` and `nums[j]` share a common factor greater than `1`.

Return *the size of the largest connected component in the graph*.

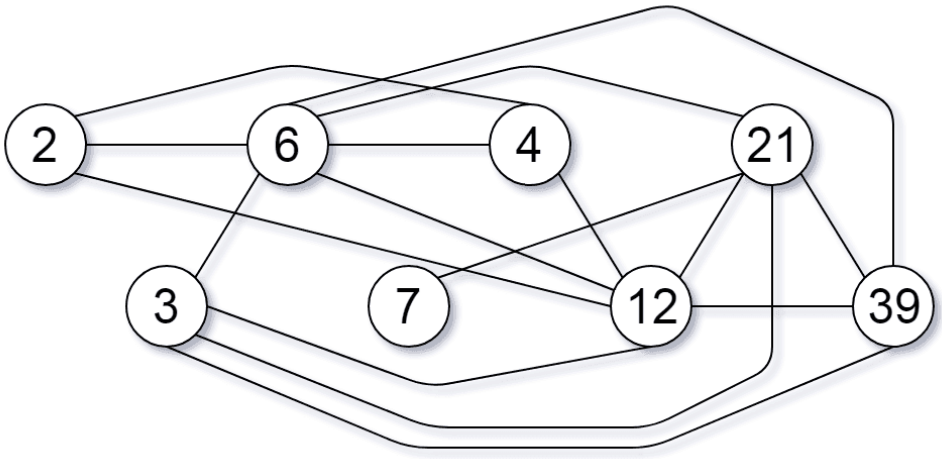
Example 1:

```
Input: nums = [4,6,15,35]
Output: 4
```

Example 2:

```
Input: nums = [20,50,9,63]
Output: 2
```

Example 3:




```
Input: nums = [2,3,6,7,4,12,21,39]
Output: 8
```

Constraints:

- `1 <= nums.length <= 2 * 10^4`
- `1 <= nums[i] <= 10^5`
- All the values of `nums` are **unique**.

SOLUTION (PYTHON 3)

Solution 1

> **How to think about it**
>
> Numbers sharing a factor greater than 1 are connected; we want the largest component. Pairwise gcd is quadratic and too slow for n≤ 2× 10^4. Union each value with all of its factors so numbers that share a factor meet. The answer is the highest frequency among the roots of the input values.

Python 3

```
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))

    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa != pb:
            self.p[pa] = pb

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]


class Solution:
    def largestComponentSize(self, nums: List[int]) -> int:
        uf = UnionFind(max(nums) + 1)
        for v in nums:
            i = 2
            while i <= v // i:
                if v % i == 0:
                    uf.union(v, i)
                    uf.union(v, v // i)
                i += 1
        return max(Counter(uf.find(v) for v in nums).values())
```

Complexity. Time: O(n * sqrt(M)), M = largest number. Space: O(M).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. List the groups of numbers (not just the biggest
# size)
# 2. Are two given numbers in the same component


# ========================================================
# FOLLOW-UP 1: List the groups of numbers (not just the
# biggest size)
# ========================================================

# ---- follow-up 1: function ----
# Answer: Join each number with each of its prime factors,
# then collect the numbers by root.
# Time: O(N * sqrt(M) * alpha + N log N), M = largest
# number.  Space: O(N).
```

```
def groups_by_common_factor(nums):
    parent = {}

    def find(x):
        parent.setdefault(x, x)
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for number in nums:
        x = number
        factor = 2
        while factor * factor <= x:
            if x % factor == 0:
                parent[find(number)] = find(factor)
                while x % factor == 0:
                    x //= factor
            factor += 1
        if x > 1:
            parent[find(number)] = find(x)
        find(number)
    groups = {}
    for number in nums:
        groups.setdefault(find(number), []).append(number)
    return sorted(sorted(g) for g in groups.values())


# ============================================================
# FOLLOW-UP 2: Are two given numbers in the same component
# ============================================================

# ---- follow-up 2: function ----
# Answer: Run the same joins and compare the roots.
# Time: O(N * sqrt(M) * alpha).  Space: O(N).
def same_component(nums, first, second):
    parent = {}

    def find(x):
        parent.setdefault(x, x)
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
```

```
    for number in nums:
        x = number
        factor = 2
        while factor * factor <= x:
            if x % factor == 0:
                parent[find(number)] = find(factor)
                while x % factor == 0:
                    x //= factor
            factor += 1
        if x > 1:
            parent[find(number)] = find(x)
    return find(first) == find(second)
```

1579. Remove Max Number of Edges to Keep Graph Fully Traversable

Hard · Union Find, Graph

leetcode.com/problems/remove-max-number-of-edges-to-keep-graph-fully-traversable

PROBLEM

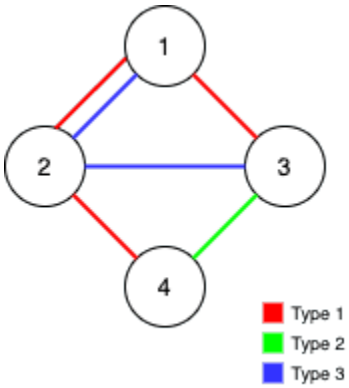
Alice and Bob have an undirected graph of `n` nodes and three types of edges:

- Type 1: Can be traversed by Alice only.
- Type 2: Can be traversed by Bob only.
- Type 3: Can be traversed by both Alice and Bob.

Given an array `edges` where `edges[i] = [type`$_i$`, u`$_i$`, v`$_i$`]` represents a bidirectional edge of type `type`$_i$ between nodes u_i and v_i, find the maximum number of edges you can remove so that after removing the edges, the graph can still be fully traversed by both Alice and Bob. The graph is fully traversed by Alice and Bob if starting from any node, they can reach all other nodes.

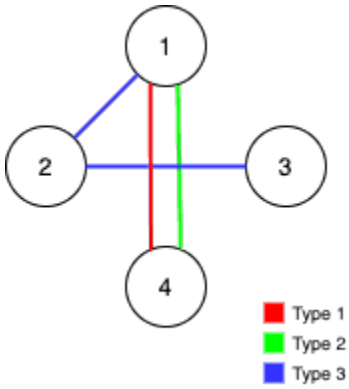
Return *the maximum number of edges you can remove, or return* `-1` *if Alice and Bob cannot fully traverse the graph.*

Example 1:



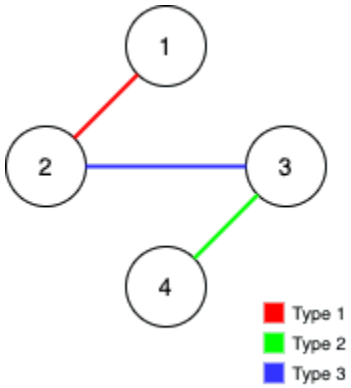

```
Input: n = 4, edges = [[3,1,2],[3,2,3],[1,1,3],[1,2,4],[1,1,2],[2,3,4]]
Output: 2
Explanation: If we remove the 2 edges [1,1,2] and [1,1,3]. The graph will still be fully traversable
by Alice and Bob. Removing any additional edge will not make it so. So the maximum number of edges we
can remove is 2.
```

Example 2:



```
Input: n = 4, edges = [[3,1,2],[3,2,3],[1,1,4],[2,1,4]]
Output: 0
Explanation: Notice that removing any edge will not make the graph fully traversable by Alice and Bob.
```

Example 3:




```
Input: n = 4, edges = [[3,2,3],[1,1,2],[2,3,4]]
Output: -1
Explanation: In the current graph, Alice cannot reach node 4 from the other nodes. Likewise, Bob
cannot reach 1. Therefore it's impossible to make the graph fully traversable.
```

Constraints:

- $1 <= n <= 10^5$
- $1 <= edges.length <= min(10^5, 3 * n * (n - 1) / 2)$
- $edges[i].length == 3$
- $1 <= type_i <= 3$
- $1 <= u_i < v_i <= n$
- All tuples $(type_i, u_i, v_i)$ are distinct.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Delete as many edges as possible while both Alice and Bob can still traverse the graph. n and the edge count reach 10^5. Type-3 edges serve both and should be kept first; exclusive edges only patch each person's graph. Two disjoint-set forests track the two reachability relations. Add type 3 first and discard a shared edge that would close a cycle. Then add types 1 and 2 the same way. A solution exists only when both forests have a single component.

Python 3

```python
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n
        self.cnt = n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        pa, pb = self.find(a - 1), self.find(b - 1)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        self.cnt -= 1
        return True


class Solution:
    def maxNumEdgesToRemove(self, n: int, edges: List[List[int]]) -> int:
        ufa = UnionFind(n)
        ufb = UnionFind(n)
        ans = 0
        for t, u, v in edges:
            if t == 3:
                if ufa.union(u, v):
                    ufb.union(u, v)
                else:
                    ans += 1
        for t, u, v in edges:
            if t == 1:
                ans += not ufa.union(u, v)
            if t == 2:
                ans += not ufb.union(u, v)
        return ans if ufa.cnt == 1 and ufb.cnt == 1 else -1
```

Complexity. Time: O(E * alpha(n)), essentially O(E). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the edges that are removed
# 2. Only check whether both people can traverse the whole
# graph


# ============================================================
# FOLLOW-UP 1: Return the edges that are removed
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep an edge only when it joins two groups in
# the right union-find (shared edges first); every other
# edge is removed.
# Time: O((n + E) * alpha(n) + R log R).  Space: O(n + E).
def edges_to_remove(n, edges):
    def make():
        parent = list(range(n + 1))
```

```
        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        return parent, find

    alice_parent, alice_find = make()
    bob_parent, bob_find = make()
    removed = []
    for i in range(len(edges)):
        kind, a, b = edges[i]
        if kind == 3:
            ra, rb = alice_find(a), alice_find(b)
            ba, bb = bob_find(a), bob_find(b)
            if ra == rb and ba == bb:
                removed.append(i)
            else:
                alice_parent[ra] = rb
                bob_parent[ba] = bb
    for i in range(len(edges)):
        kind, a, b = edges[i]
        if kind == 1:
            ra, rb = alice_find(a), alice_find(b)
            if ra == rb:
                removed.append(i)
            else:
                alice_parent[ra] = rb
        elif kind == 2:
            ba, bb = bob_find(a), bob_find(b)
            if ba == bb:
                removed.append(i)
            else:
                bob_parent[ba] = bb
    for find in (alice_find, bob_find):
        if len(set(find(x) for x in range(1, n + 1))) != 1:
            return None
    return sorted(removed)
```

```
# ============================================================
# FOLLOW-UP 2: Only check whether both people can traverse
# the whole graph
# ============================================================

# ---- follow-up 2: function ----
# Answer: Join the edges each person can use and check for
# one group each.
# Time: O((n + E) * alpha(n)).  Space: O(n).
def both_can_traverse(n, edges):
    def one_group(allowed):
        parent = list(range(n + 1))

        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        for kind, a, b in edges:
            if kind in allowed:
                parent[find(a)] = find(b)
        return len(set(find(x) for x in range(1, n + 1))) == 1

    return one_group((1, 3)) and one_group((2, 3))
```

PATTERN 32 OF 41

Trie (Prefix Tree)

20 problems · 2 Easy · 11 Medium · 7 Hard

THE IDEA

A tree of characters where each path from the root spells a prefix. Lookups cost the length of the word, and shared prefixes are stored once.

Signals: implement a trie, word search II, autocomplete, longest common prefix, replace words, maximum XOR of two numbers.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
root = {}
def insert(word):
    node = root
    for ch in word:
        node = node.setdefault(ch, {})
    node["$"] = True                # end of a word
def starts_with(prefix):
    node = root
    for ch in prefix:
        if ch not in node:
            return False
        node = node[ch]
    return True
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Longest Word in Dictionary	Easy	LC 720	1726
2	Count Pairs of Similar Strings — substitute: Index Pairs of a String (see page)	Easy	LC 1065	351
3	★ **Implement Trie (Prefix Tree)** (see page)	Medium	LC 208	1548
4	★ **Design Add and Search Words Data Structure** (see page)	Medium	LC 211	1567
5	Implement Trie II (Prefix Tree)	Medium	LC 1804	1729
6	★ **Replace Words**	Medium	LC 648	1733
7	★ **Map Sum Pairs**	Medium	LC 677	1736
8	★ **Maximum XOR of Two Numbers in an Array** (see page)	Medium	LC 421	1487
9	★ **Search Suggestions System**	Medium	LC 1268	1739
10	Word Break (see page)	Medium	LC 139	323
11	Camelcase Matching (see page)	Medium	LC 1023	354
12	Number of Matching Subsequences (see page)	Medium	LC 792	357
13	Short Encoding of Words	Medium	LC 820	1742
14	★ **Word Search II**	Hard	LC 212	1745
15	Concatenated Words (see page)	Hard	LC 472	384
16	Stream of Characters (see page)	Hard	LC 1032	376
17	Prefix and Suffix Search	Hard	LC 745	1749
18	Maximum XOR With an Element From Array	Hard	LC 1707	1753
19	Sum of Prefix Scores of Strings	Hard	LC 2416	1756
20	Palindrome Pairs (see page)	Hard	LC 336	380

PATTERN 32 · TRIE (PREFIX TREE) · PROBLEM 1 OF 20

720. Longest Word in Dictionary

Medium · Trie, Array, Hash Table, String, Sorting

leetcode.com/problems/longest-word-in-dictionary

PROBLEM

Given an array of strings `words` representing an English Dictionary, return *the longest word in* `words` *that can be built one character at a time by other words in* `words`.

If there is more than one possible answer, return the longest word with the smallest lexicographical order. If there is no answer, return the empty string.

Note that the word should be built from left to right with each additional character being added to the end of a previous word.

Example 1:

```
Input: words = ["w","wo","wor","worl","world"]
Output: "world"
Explanation: The word "world" can be built one character at a time by "w", "wo", "wor", and "worl".
```

Example 2:

```
Input: words = ["a","banana","app","appl","ap","apply","apple"]
Output: "apple"
Explanation: Both "apply" and "apple" can be built from other words in the dictionary. However,
"apple" is lexicographically smaller than "apply".
```

Constraints:

- `1 <= words.length <= 1000`
- `1 <= words[i].length <= 30`
- `words[i]` consists of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Trie

> **How to think about it**
>
> We want the longest word that can be built by adding one letter at a time from other dictionary words, breaking ties lexicographically. The input is small, but checking every prefix in a set is easy to get messy. That condition is exactly: every prefix of the word is itself in the dictionary. After inserting all words into a trie, a walk that requires every node to be a word-end decides it in one pass. Build the trie, then scan each word with search, keeping the longer word or the lexicographically smaller one of equal length. Time is linear in the total number of characters.

We can use a trie to store all the words, then traverse all the words to determine if the current word can be formed by adding one letter at a time from other words in the trie. Find the longest word that meets the condition and has the smallest lexicographical order.

The time complexity is O(L), and the space complexity is O(L), where L is the sum of the lengths of all words.

Python 3

```python
class Trie:
    def __init__(self):
        self.children: List[Optional[Trie]] = [None] * 26
        self.is_end = False

    def insert(self, w: str):
        node = self
        for c in w:
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, w: str) -> bool:
        node = self
        for c in w:
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
                return False
            node = node.children[idx]
            if not node.is_end:
                return False
        return True


class Solution:
    def longestWord(self, words: List[str]) -> str:
        trie = Trie()
        for w in words:
            trie.insert(w)
        ans = ""
        for w in words:
            if trie.search(w) and (
                len(ans) < len(w) or (len(ans) == len(w) and ans > w)
            ):
                ans = w
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return every longest buildable word (all ties,
# sorted)
# 2. How many words are buildable one letter at a time


# ========================================================
# FOLLOW-UP 1: Return every longest buildable word (all
# ties, sorted)
# ========================================================

# ---- follow-up 1: function ----
# Answer: Insert the words into a trie and walk only
# through nodes that end a word; keep the words of the
# greatest depth.
# Time: O(L + b log b), L = total letters, b = ties.
# Space: O(L).
class TrieNode:
    def __init__(self):
        self.children = {}
        self.word = None


def all_longest_buildable(words):
```

```
    root = TrieNode()
    for w in words:
        node = root
        for ch in w:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.word = w
    best = []
    stack = [root]
    while stack:
        node = stack.pop()
        for child in node.children.values():
            if child.word is not None:
                if not best or len(child.word) > len(best[0]):
                    best = [child.word]
                elif len(child.word) == len(best[0]):
                    best.append(child.word)
                stack.append(child)
    return sorted(best)


# ============================================================
# FOLLOW-UP 2: How many words are buildable one letter at
# a time
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the word nodes reachable through word
# nodes only.
# Time: O(L).  Space: O(L).
def count_buildable(words):
    root = TrieNode()
    for w in words:
        node = root
        for ch in w:
```

```
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.word = w
    count = 0
    stack = [root]
    while stack:
        node = stack.pop()
        for child in node.children.values():
            if child.word is not None:
                count += 1
                stack.append(child)
    return count
```

PATTERN 32 · TRIE (PREFIX TREE) · PROBLEM 5 OF 20

1804. Implement Trie II (Prefix Tree)

Medium · Design, Trie, Hash Table, String

leetcode.com/problems/implement-trie-ii-prefix-tree

PROBLEM

A **trie** (pronounced as "try") or **prefix tree** is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. There are various applications of this data structure, such as autocomplete and spellchecker.

Implement the Trie class:

- `Trie()` Initializes the trie object.
- `void insert(String word)` Inserts the string `word` into the trie.
- `int countWordsEqualTo(String word)` Returns the number of instances of the string `word` in the trie.
- `int countWordsStartingWith(String prefix)` Returns the number of strings in the trie that have the string `prefix` as a prefix.
- `void erase(String word)` Erases the string `word` from the trie.

Example 1:

```
Input
["Trie", "insert", "insert", "countWordsEqualTo", "countWordsStartingWith", "erase",
"countWordsEqualTo", "countWordsStartingWith", "erase", "countWordsStartingWith"]
[[], ["apple"], ["apple"], ["apple"], ["app"], ["apple"], ["apple"], ["app"], ["apple"], ["app"]]
Output
[null, null, null, 2, 2, null, 1, 1, null, 0]

Explanation
Trie trie = new Trie();
trie.insert("apple"); // Inserts "apple".
trie.insert("apple"); // Inserts another "apple".
trie.countWordsEqualTo("apple"); // There are two instances of "apple" so return 2.
trie.countWordsStartingWith("app"); // "app" is a prefix of "apple" so return 2.
trie.erase("apple"); // Erases one "apple".
trie.countWordsEqualTo("apple"); // Now there is only one instance of "apple" so return 1.
trie.countWordsStartingWith("app"); // return 1
trie.erase("apple"); // Erases "apple". Now the trie is empty.
trie.countWordsStartingWith("app"); // return 0
```

Constraints:

- `1 <= word.length, prefix.length <= 2000`
- `word` and `prefix` consist only of lowercase English letters.
- At most $3 * 10^4$ calls **in total** will be made to `insert`, `countWordsEqualTo`, `countWordsStartingWith`, and `erase`.
- It is guaranteed that for any function call to `erase`, the string `word` will exist in the trie.

SOLUTION (PYTHON 3)

Solution 1: Implement Trie with Array

> **How to think about it**
>
> A standard trie only answers existence. Here we also need the count of a word, the count of words with a given prefix, and deletion. Scanning every inserted string on each query would be too slow given the operation limits. Store two counters on every node: v for words ending here and pv for words that pass through. Insertion increments pv along the path and v at the end; queries walk to the node and read the corresponding counter; erasure decrements along the same path. Each operation stays linear in the word length.

Each node in the Trie includes three parts:

1. An array of pointers `children` pointing to child nodes. For this problem, the array length is 26, which is the number of lowercase English letters. `children[0]` corresponds to the lowercase letter a, ..., `children[25]` corresponds to the lowercase letter z.
1. An int variable `v`, representing the number of strings ending with this node.
1. An int variable `pv`, representing the number of strings with this node as the prefix node.

1. Insert String

We start from the root of the Trie and insert the string. For the child node corresponding to the current character, there are two cases:

- The child node exists. Move to the child node along the pointer and continue to process the next character.
- The child node does not exist. Create a new child node, record it in the corresponding position of the `children` array, then move to the child node along the pointer, and increase the `pv` value of the child node by 1. Continue to search for the next character.

Repeat the above steps until the last character of the string is processed, then increase the `v` value of the current node by 1.

The time complexity is O(n), where n is the length of the string.

2. Search Prefix

We start from the root of the Trie and search for the prefix. For the child node corresponding to the current character, there are two cases:

- The child node exists. Move to the child node along the pointer and continue to search for the next character.
- The child node does not exist. This means that the Trie does not contain this prefix, return a null pointer.

Repeat the above steps until a null pointer is returned or the last character of the prefix is searched.

The time complexity is O(n), where n is the length of the string.

3. Remove String

We start from the root node of the Trie, and sequentially reduce the `pv` value of the corresponding child node by 1, until the last character of the string is searched. Then reduce the `v` value of the current node by 1.

The time complexity is O(n), where n is the length of the string.

Python 3

```python
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.v = self.pv = 0

    def insert(self, word: str) -> None:
        node = self
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
            node.pv += 1
        node.v += 1

    def countWordsEqualTo(self, word: str) -> int:
        node = self.search(word)
        return 0 if node is None else node.v

    def countWordsStartingWith(self, prefix: str) -> int:
        node = self.search(prefix)
        return 0 if node is None else node.pv

    def erase(self, word: str) -> None:
        node = self
        for c in word:
            idx = ord(c) - ord('a')
            node = node.children[idx]
            node.pv -= 1
        node.v -= 1

    def search(self, word):
        node = self
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                return None
            node = node.children[idx]
        return node


# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)
# param_2 = obj.countWordsEqualTo(word)
# param_3 = obj.countWordsStartingWith(prefix)
# obj.erase(word)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Autocomplete: return the k smallest words that start
# with a prefix
# 2. Longest common prefix of all stored words


# ========================================================
# FOLLOW-UP 1: Autocomplete: return the k smallest words
# that start with a prefix
# ========================================================

# ---- follow-up 1: function ----
# Answer: Walk to the prefix node, then collect words in
# alphabetical order with a depth-first search that tries
# letters in sorted order and stops after k.
```

```
# Time: insert O(len), complete O(len + visited nodes * 26
# log 26).  Space: O(total letters).
class TrieWithComplete:
    def __init__(self):
        self.children = {}
        self.copies = 0

    def insert(self, word):
        node = self
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieWithComplete()
            node = node.children[ch]
        node.copies += 1

    def complete(self, prefix, k):
        node = self
        for ch in prefix:
            if ch not in node.children:
                return []
            node = node.children[ch]
        found = []

        def go(current, so_far):
            if len(found) == k:
                return
            if current.copies > 0:
                found.append(so_far)
            for ch in sorted(current.children):
                go(current.children[ch], so_far + ch)

        go(node, prefix)
        return found


# ============================================================
# FOLLOW-UP 2: Longest common prefix of all stored words
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk down from the root while the node has
# exactly one child and does not end a word.
# Time: O(length of the common prefix).  Space: O(length
# of the common prefix).
def longest_common_prefix_of_trie(root):
```

```
    prefix = []
    node = root
    while len(node.children) == 1 and node.copies == 0:
        ch = next(iter(node.children))
        prefix.append(ch)
        node = node.children[ch]
    return "".join(prefix)
```

PATTERN 32 · TRIE (PREFIX TREE) · PROBLEM 6 OF 20

648. Replace Words

Medium · Trie, Array, Hash Table, String

leetcode.com/problems/replace-words

★ **TOP 7 PROBLEM** in Trie (Prefix Tree): master this one first

PROBLEM

In English, we have a concept called **root**, which can be followed by some other word to form another longer word - let's call this word **derivative**. For example, when the **root** `"help"` is followed by the word `"ful"`, we can form a derivative `"helpful"`.

Given a `dictionary` consisting of many **roots** and a `sentence` consisting of words separated by spaces, replace all the derivatives in the sentence with the **root** forming it. If a derivative can be replaced by more than one **root**, replace it with the **root** that has **the shortest length**.

Return *the* `sentence` after the replacement.

Example 1:

```
Input: dictionary = ["cat","bat","rat"], sentence = "the cattle was rattled by the battery"
Output: "the cat was rat by the bat"
```

Example 2:

```
Input: dictionary = ["a","b","c"], sentence = "aadsfasf absbs bbab cadsfafs"
Output: "a a b c"
```

Constraints:

- `1 <= dictionary.length <= 1000`
- `1 <= dictionary[i].length <= 100`
- `dictionary[i]` consists of only lower-case letters.
- `1 <= sentence.length <= 10^6`
- `sentence` consists of only lower-case letters and spaces.
- The number of words in `sentence` is in the range `[1, 1000]`
- The length of each word in `sentence` is in the range `[1, 1000]`
- Every two consecutive words in `sentence` will be separated by exactly one space.
- `sentence` does not have leading or trailing spaces.

SOLUTION (PYTHON 3)

Solution 1: Trie

How to think about it

Each sentence word should be replaced by the shortest dictionary root. Testing every root on every word repeats prefixes. Insert roots into a trie and walk each word until the first `is_end`; that prefix is the shortest root.

We can use a trie to store all the roots in the dictionary. Define the trie node class Trie, which contains an array children of length 26 to store child nodes, and a boolean variable is_end to mark whether it is a complete root.

For each root, we insert it into the trie. For each word in the sentence, we search for its shortest root in the trie. If found, we replace the word; otherwise, we keep it unchanged.

The time complexity is O(n × |w| + L), and the space complexity is O(n × |w|), where n and |w| are the number of roots in the dictionary and the average length, respectively, and L is the total length of words in the sentence.

Python 3

```python
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, w: str) -> None:
        node = self
        for c in w:
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, w: str) -> str:
        node = self
        for i, c in enumerate(w, 1):
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
                return w
            node = node.children[idx]
            if node.is_end:
                return w[:i]
        return w


class Solution:
    def replaceWords(self, dictionary: List[str], sentence: str) -> str:
        trie = Trie()
        for w in dictionary:
            trie.insert(w)
        return " ".join(trie.search(w) for w in sentence.split())
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Replace each word with its LONGEST matching root
# 2. Also report how many words were replaced


# ============================================================
# FOLLOW-UP 1: Replace each word with its LONGEST matching
# root
# ============================================================

# ---- follow-up 1: function ----
# Answer: Walk the trie but remember the last root end
# seen; stop when the walk fails.
# Time: O(R + S), R = letters in roots, S = letters in the
# sentence.  Space: O(R + S).
def replace_with_longest_root(roots, sentence):
    trie = {}
    for root in roots:
        node = trie
        for ch in root:
            node = node.setdefault(ch, {})
        node["$"] = root

    def replace(word):
        node = trie
        best = word
        for ch in word:
            if ch not in node:
                break
            node = node[ch]
```

```
            if "$" in node:
                best = node["$"]
        return best

    return " ".join(replace(w) for w in sentence.split())


# ============================================================
# FOLLOW-UP 2: Also report how many words were replaced
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the words whose replacement differs from
# the word.
# Time: O(R + S).  Space: O(R + S).
def replace_and_count(roots, sentence):
    trie = {}
    for root in roots:
        node = trie
        for ch in root:
            node = node.setdefault(ch, {})
        node["$"] = root
    changed = 0
    result = []
    for word in sentence.split():
        node = trie
        replacement = word
        for ch in word:
            if ch not in node:
                break
```

```
            node = node[ch]
            if "$" in node:
                replacement = node["$"]
                break
        if replacement != word:
            changed += 1
        result.append(replacement)
    return " ".join(result), changed
```

PATTERN 32 · TRIE (PREFIX TREE) · PROBLEM 7 OF 20

677. Map Sum Pairs

Medium · Design, Trie, Hash Table, String

leetcode.com/problems/map-sum-pairs

★ **TOP 7 PROBLEM** in Trie (Prefix Tree): master this one first

PROBLEM

Design a map that allows you to do the following:

- Maps a string key to a given value.
- Returns the sum of the values that have a key with a prefix equal to a given string.

Implement the `MapSum` class:

- `MapSum()` Initializes the `MapSum` object.
- `void insert(String key, int val)` Inserts the `key-val` pair into the map. If the `key` already existed, the original `key-value` pair will be overridden to the new one.
- `int sum(string prefix)` Returns the sum of all the pairs' value whose `key` starts with the `prefix`.

Example 1:

```
Input
["MapSum", "insert", "sum", "insert", "sum"]
[[], ["apple", 3], ["ap"], ["app", 2], ["ap"]]
Output
[null, null, 3, null, 5]

Explanation
MapSum mapSum = new MapSum();
mapSum.insert("apple", 3);
mapSum.sum("ap"); // return 3 (apple = 3)
mapSum.insert("app", 2);
mapSum.sum("ap"); // return 5 (apple + app = 3 + 2 = 5)
```

Constraints:

- `1 <= key.length, prefix.length <= 50`
- `key` and `prefix` consist of only lowercase English letters.
- `1 <= val <= 1000`
- At most `50` calls will be made to `insert` and `sum`.

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Trie

How to think about it

`sum(prefix)` needs the total of keys with that prefix, and `insert` overwrites. Scanning every key is linear in the map size. Trie nodes store subtree sums. Insert adds Δ=val-old along the path; `sum` returns the node at the prefix. A hash map keeps the previous value.

We use a hash table d to store key-value pairs and a trie t to store the prefix sums of the key-value pairs. Each node in the trie contains two pieces of information:

- `val`: the total sum of the values of the key-value pairs with this node as the prefix
- `children`: an array of length 26 that stores the child nodes of this node

When inserting a key-value pair (key, val), we first check if the key exists in the hash table. If it does, the `val` of each node in the trie needs to subtract the original value of the key and then add the new value. If it does not exist, the `val` of each

node in the trie needs to add the new value.

When querying the prefix sum, we start from the root node of the trie and traverse the prefix string. If the current node's child nodes do not contain the character, it means the prefix does not exist in the trie, and we return 0. Otherwise, we continue to traverse the next character until we finish traversing the prefix string and return the `val` of the current node.

In terms of time complexity, the time complexity of inserting a key-value pair is O(n), where n is the length of the key. The time complexity of querying the prefix sum is O(m), where m is the length of the prefix.

The space complexity is O(n × m × C), where n and m are the number of keys and the maximum length of the keys, respectively; and C is the size of the character set, which is 26 in this problem.

Python 3

```
class Trie:
    def __init__(self):
        self.children: List[Trie | None] = [None] * 26
        self.val: int = 0

    def insert(self, w: str, x: int):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
            node.val += x

    def search(self, w: str) -> int:
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                return 0
            node = node.children[idx]
        return node.val


class MapSum:
    def __init__(self):
        self.d = defaultdict(int)
        self.tree = Trie()

    def insert(self, key: str, val: int) -> None:
        x = val - self.d[key]
        self.d[key] = val
        self.tree.insert(key, x)

    def sum(self, prefix: str) -> int:
        return self.tree.search(prefix)


# Your MapSum object will be instantiated and called as such:
# obj = MapSum()
# obj.insert(key,val)
# param_2 = obj.sum(prefix)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Delete a key
# 2. The largest value among the keys with a prefix


# ============================================================
# FOLLOW-UP 1: Delete a key
# ============================================================
```

```
# ---- follow-up 1: function ----
# Answer: Keep the value of each key in a dictionary;
# deleting subtracts that value along the key's path.
# Time: insert, delete, sum are each O(len).  Space:
# O(total letters).
class MapSumWithDelete:
    def __init__(self):
        self.value_of = {}
        self.root = {}

    def add(self, key, delta):
        node = self.root
        node["#"] = node.get("#", 0) + delta
        for ch in key:
            node = node.setdefault(ch, {})
            node["#"] = node.get("#", 0) + delta

    def insert(self, key, val):
        self.add(key, val - self.value_of.get(key, 0))
        self.value_of[key] = val

    def delete(self, key):
        if key in self.value_of:
            self.add(key, -self.value_of.pop(key))

    def sum(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node:
                return 0
            node = node[ch]
        return node.get("#", 0)


# ============================================================
# FOLLOW-UP 2: The largest value among the keys with a
# prefix
# ============================================================

# ---- follow-up 2: function ----
# Answer: Collect the keys of the prefix subtree and take
# the biggest value.
# Time: O(n * len).  Space: O(1) extra.
def max_value_with_prefix(value_of, prefix):
    best = None
    for key, value in value_of.items():
        if key.startswith(prefix):
            if best is None or value > best:
                best = value
    return best
```

PATTERN 32 · TRIE (PREFIX TREE) · PROBLEM 9 OF 20

1268. Search Suggestions System

Medium · Trie, Array, String, Binary Search, Sorting, Heap (Priority Queue)

leetcode.com/problems/search-suggestions-system

★ **TOP 7 PROBLEM** in Trie (Prefix Tree): master this one first

PROBLEM

You are given an array of strings `products` and a string `searchWord`.

Design a system that suggests at most three product names from `products` after each character of `searchWord` is typed. Suggested products should have common prefix with `searchWord`. If there are more than three products with a common prefix return the three lexicographically minimums products.

Return *a list of lists of the suggested products after each character of* `searchWord` *is typed*.

Example 1:

```
Input: products = ["mobile","mouse","moneypot","monitor","mousepad"], searchWord = "mouse"
Output: [["mobile","moneypot","monitor"],["mobile","moneypot","monitor"],["mouse","mousepad"],["mouse"
,"mousepad"],["mouse","mousepad"]]
Explanation: products sorted lexicographically = ["mobile","moneypot","monitor","mouse","mousepad"].
After typing m and mo all products match and we show user ["mobile","moneypot","monitor"].
After typing mou, mous and mouse the system suggests ["mouse","mousepad"].
```

Example 2:

```
Input: products = ["havana"], searchWord = "havana"
Output: [["havana"],["havana"],["havana"],["havana"],["havana"],["havana"]]
Explanation: The only word "havana" will be always suggested while typing the search word.
```

Constraints:

- `1 <= products.length <= 1000`
- `1 <= products[i].length <= 3000`
- `1 <= sum(products[i].length) <= 2 * 10^4`
- All the strings of `products` are **unique**.
- `products[i]` consists of lowercase English letters.
- `1 <= searchWord.length <= 1000`
- `searchWord` consists of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Sorting + Trie

How to think about it

After each extra letter we want the three lexicographically smallest products with that prefix. There are 1000 products: sort, then insert into a trie, each node keeping at most three indices in insertion order — the top three for that prefix. Walking searchWord maps those index lists back to names. Sorting makes insertion order lexicographic; the trie locates prefixes.

The problem requires that after each letter of the input `searchWord`, recommend up to three products from the `products` array that have the same prefix as `searchWord`. If there are more than three products with the same prefix that can be recommended, return the three with the smallest lexicographic order.

To find products with the same prefix, we can use a trie; to return the three products with the smallest lexicographic order, we can first sort the `products` array, and then store the indices of the sorted array in the trie.

Each node of the trie maintains the following information:

- `children`: This is an array of length 26, used to store the child nodes of the current node. `children[i]` represents the node whose character is `i + 'a'` among the child nodes of the current node.
- `v`: This is an array, used to store the indices of the characters in the `products` array among the child nodes of the current node, storing up to three indices.

During the search, we start from the root node of the trie, find the index array corresponding to each prefix, and store it in the result array. Finally, we only need to map each index in the index array to the `products` array.

The time complexity is O(L × log n + m), and the space complexity is O(L). Where L is the sum of the lengths of all strings in the `products` array, and n and m are the lengths of the `products` array and `searchWord`, respectively.

Python 3

```python
class Trie:
    def __init__(self):
        self.children: List[Union[Trie, None]] = [None] * 26
        self.v: List[int] = []

    def insert(self, w, i):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
            if len(node.v) < 3:
                node.v.append(i)

    def search(self, w):
        node = self
        ans = [[] for _ in range(len(w))]
        for i, c in enumerate(w):
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                break
            node = node.children[idx]
            ans[i] = node.v
        return ans


class Solution:
    def suggestedProducts(
        self, products: List[str], searchWord: str
    ) -> List[List[str]]:
        products.sort()
        trie = Trie()
        for i, w in enumerate(products):
            trie.insert(w, i)
        return [[products[i] for i in v] for v in trie.search(searchWord)]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Rank the suggestions by popularity instead of
# alphabet
# 2. Suggestions for the full typed word only (one trie
# walk)


# ============================================================
# FOLLOW-UP 1: Rank the suggestions by popularity instead
# of alphabet
# ============================================================

# ---- follow-up 1: function ----
```

```
# Answer: Give every product a weight; for each typed
# prefix take the three heaviest matches (ties
# alphabetical).
# Time: O(P log P + L * P * L), P products.  Space: O(P).
def suggestions_by_popularity(products, popularity, search_word):
    ranked = sorted(products, key=lambda p: (-popularity[p], p))
    result = []
    for i in range(1, len(search_word) + 1):
        prefix = search_word[:i]
        matches = [p for p in ranked if p.startswith(prefix)]
        result.append(matches[:3])
    return result


# ============================================================
# FOLLOW-UP 2: Suggestions for the full typed word only
# (one trie walk)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Store the three smallest words in every node
# while inserting; the answer is read off the last node.
# Time: O(P log P + total letters + L).  Space: O(total
# letters).
def final_suggestions(products, search_word):
    root = {}
    for product in sorted(products):
        node = root
        for ch in product:
            node = node.setdefault(ch, {})
            top = node.setdefault("#", [])
            if len(top) < 3:
                top.append(product)
    node = root
    for ch in search_word:
        if ch not in node:
            return []
        node = node[ch]
    return node["#"]
```

PATTERN 32 · TRIE (PREFIX TREE) · PROBLEM 13 OF 20

820. Short Encoding of Words

Medium · Trie, Array, Hash Table, String

leetcode.com/problems/short-encoding-of-words

PROBLEM

A **valid encoding** of an array of `words` is any reference string `s` and array of indices `indices` such that:

- `words.length == indices.length`
- The reference string `s` ends with the `'#'` character.
- For each index `indices[i]`, the **substring** of `s` starting from `indices[i]` and up to (but not including) the next `'#'` character is equal to `words[i]`.

Given an array of `words`, return *the **length of the shortest reference string** `s` possible of any **valid encoding** of* `words`.

Example 1:

```
Input: words = ["time", "me", "bell"]
Output: 10
Explanation: A valid encoding would be s = "time#bell#" and indices = [0, 2, 5].
words[0] = "time", the substring of s starting from indices[0] = 0 to the next '#' is underlined in
"time#bell#"
words[1] = "me", the substring of s starting from indices[1] = 2 to the next '#' is underlined in
"time#bell#"
words[2] = "bell", the substring of s starting from indices[2] = 5 to the next '#' is underlined in
"time#bell#"
```

Example 2:

```
Input: words = ["t"]
Output: 2
Explanation: A valid encoding would be s = "t#" and indices = [0].
```

Constraints:

- `1 <= words.length <= 2000`
- `1 <= words[i].length <= 7`
- `words[i]` consists of only lowercase letters.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

The encoding concatenates words with `#`. A shorter word that is a suffix of a longer one is already covered. Pairwise suffix checks waste work. Insert the reversed words into a trie so shared suffixes share a path. Only leaves need their own encoding (length plus one); internal nodes are covered by a longer word.

Python 3

```python
class Trie:
    def __init__(self) -> None:
        self.children = [None] * 26


class Solution:
    def minimumLengthEncoding(self, words: List[str]) -> int:
        root = Trie()
        for w in words:
            cur = root
            for c in w[::-1]:
                idx = ord(c) - ord("a")
                if cur.children[idx] == None:
                    cur.children[idx] = Trie()
                cur = cur.children[idx]
        return self.dfs(root, 1)

    def dfs(self, cur: Trie, l: int) -> int:
        isLeaf, ans = True, 0
        for i in range(26):
            if cur.children[i] != None:
                isLeaf = False
                ans += self.dfs(cur.children[i], l + 1)
        if isLeaf:
            ans += l
        return ans
```

Complexity. Time: O(L), L = total number of letters. Space: O(26 * L).

Solution 2

> **How to think about it**
>
> Method 1 builds the full trie, then DFS-counts leaves. Inserting reversed words from long to short lets longer words occupy paths first. If a later word's path already exists, it is a suffix of a longer word and contributes 0; otherwise it contributes length plus one. The answer is accumulated during insertion, so the second DFS is dropped.

Python 3

```python
class Trie:
    def __init__(self):
        self.children = [None] * 26

    def insert(self, w):
        node = self
        pref = True
        for c in w:
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
                node.children[idx] = Trie()
                pref = False
            node = node.children[idx]
        return 0 if pref else len(w) + 1


class Solution:
    def minimumLengthEncoding(self, words: List[str]) -> int:
        words.sort(key=lambda x: -len(x))
        trie = Trie()
        return sum(trie.insert(w[::-1]) for w in words)
```

Complexity. Time: O(L + n log n). Space: O(26 * L).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the shortest encoding string itself
# 2. Which words are swallowed (they are a suffix of
# another word)


# ============================================================
# FOLLOW-UP 1: Return the shortest encoding string itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep only the words that are not a suffix of
# another word, and join them with # after each.
# Time: O(L^2), because every suffix is sliced.  Space:
# O(L).
def shortest_encoding(words):
    unique = set(words)
    for word in words:
        for start in range(1, len(word)):
            unique.discard(word[start:])
    return "".join(w + "#" for w in sorted(unique))


# ============================================================
# FOLLOW-UP 2: Which words are swallowed (they are a
# suffix of another word)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Insert reversed words into a trie; a word is
# swallowed when its reversed path is a prefix of a longer
# reversed word.
# Time: O(L).  Space: O(L).
def swallowed_words(words):
    root = {}
    for word in set(words):
        node = root
        for ch in reversed(word):
            node = node.setdefault(ch, {})
    swallowed = []
    for word in sorted(set(words)):
        node = root
        for ch in reversed(word):
            node = node[ch]
        if node:
            swallowed.append(word)
    return swallowed
```

PATTERN 32 · TRIE (PREFIX TREE) · PROBLEM 14 OF 20

212. Word Search II

Hard · Trie, Array, String, Backtracking, Matrix

leetcode.com/problems/word-search-ii

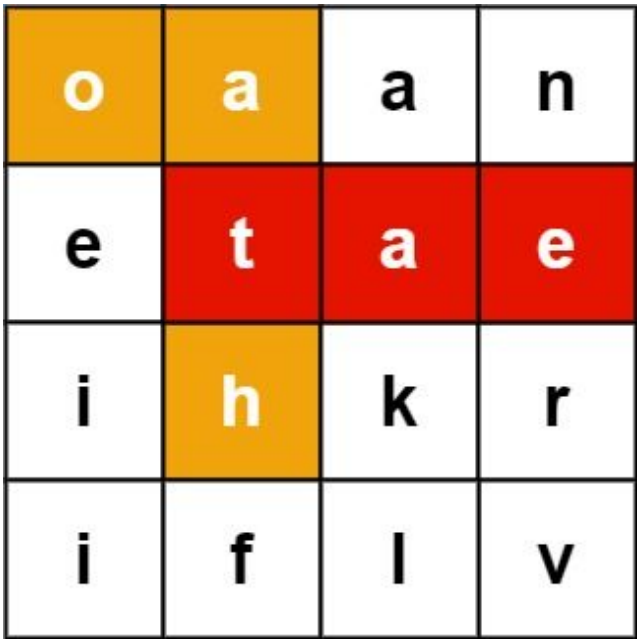
★ **TOP 7 PROBLEM** in Trie (Prefix Tree): master this one first

PROBLEM

Given an `m x n board` of characters and a list of strings `words`, return *all words on the board*.

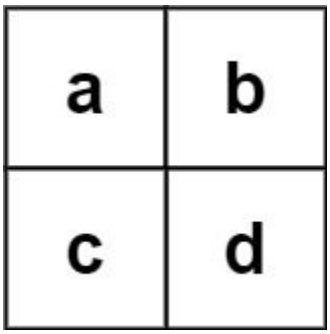
Each word must be constructed from letters of sequentially adjacent cells, where **adjacent cells** are horizontally or vertically neighboring. The same letter cell may not be used more than once in a word.

Example 1:




```
Input: board = [["o","a","a","n"],["e","t","a","e"],["i","h","k","r"],["i","f","l","v"]], words =
["oath","pea","eat","rain"]
Output: ["eat","oath"]
```

Example 2:




```
Input: board = [["a","b"],["c","d"]], words = ["abcb"]
Output: []
```

Constraints:

- `m == board.length`

- `n == board[i].length`
- `1 <= m, n <= 12`
- `board[i][j]` is a lowercase English letter.
- `1 <= words.length <= 3 * 10^4`
- `1 <= words[i].length <= 10`
- `words[i]` consists of lowercase English letters.
- All the strings of `words` are unique.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

A separate DFS per word repeats work on shared prefixes. Building a trie of words lets the board search walk the tree and prune dead branches. From each cell we expand to neighbors whose edge exists in the trie. Reaching a word node records the answer and sets ref to -1 so it is not reported again.

Python 3

```
class Trie:
    def __init__(self):
        self.children: List[Trie | None] = [None] * 26
        self.ref: int = -1

    def insert(self, w: str, ref: int):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.ref = ref


class Solution:
    def findWords(self, board: List[List[str]], words: List[str]) -> List[str]:
        def dfs(node: Trie, i: int, j: int):
            idx = ord(board[i][j]) - ord('a')
            if node.children[idx] is None:
                return
            node = node.children[idx]
            if node.ref >= 0:
                ans.append(words[node.ref])
                node.ref = -1
            c = board[i][j]
            board[i][j] = '#'
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and board[x][y] != '#':
                    dfs(node, x, y)
            board[i][j] = c

        tree = Trie()
        for i, w in enumerate(words):
            tree.insert(w, i)
        m, n = len(board), len(board[0])
        ans = []
        for i in range(m):
            for j in range(n):
                dfs(tree, i, j)
        return ans
```

Complexity. Time: O(m * n * 3^l), l = length of the longest word (pruned by the trie). Space: O(W) for the trie, W = total letters.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the path of cells used for each found word
# 2. Moves may also go diagonally (8 directions)


# ============================================================
# FOLLOW-UP 1: Return the path of cells used for each
# found word
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same trie search, keeping the cells of the
# current path and saving a copy when a word is completed.
# Time: O(W * L + R * C * 4^L) in the worst case.  Space:
# O(W * L + L).
def find_words_with_paths(board, words):
    root = {}
    for word in words:
        node = root
        for ch in word:
            node = node.setdefault(ch, {})
        node["$"] = word
    rows, cols = len(board), len(board[0])
    found = {}
    path = []

    def go(r, c, node):
        ch = board[r][c]
        if ch not in node:
            return
        node = node[ch]
        path.append((r, c))
        if "$" in node and node["$"] not in found:
            found[node["$"]] = list(path)
        board[r][c] = "#"
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] != "#":
                go(nr, nc, node)
        board[r][c] = ch
        path.pop()

    for r in range(rows):
        for c in range(cols):
            go(r, c, root)
    return found


# ============================================================
# FOLLOW-UP 2: Moves may also go diagonally (8 directions)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Only the list of directions changes.
# Time: O(W * L + R * C * 8^L) in the worst case.  Space:
# O(W * L + L).
def find_words_eight_directions(board, words):
    root = {}
    for word in words:
        node = root
```

```
        for ch in word:
            node = node.setdefault(ch, {})
        node["$"] = word
    rows, cols = len(board), len(board[0])
```

```
    found = set()

    def go(r, c, node):
        ch = board[r][c]
        if ch not in node:
            return
        node = node[ch]
        if "$" in node:
            found.add(node["$"])
        board[r][c] = "#"
        for dr in (-1, 0, 1):
            for dc in (-1, 0, 1):
                nr, nc = r + dr, c + dc
                if (dr or dc) and 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] != "#":
                    go(nr, nc, node)
        board[r][c] = ch

    for r in range(rows):
        for c in range(cols):
            go(r, c, root)
    return sorted(found)
```

745. Prefix and Suffix Search

Hard · Design, Trie, Array, Hash Table, String

leetcode.com/problems/prefix-and-suffix-search

PROBLEM

Design a special dictionary that searches the words in it by a prefix and a suffix.

Implement the `WordFilter` class:

- `WordFilter(string[] words)` Initializes the object with the `words` in the dictionary.
- `f(string pref, string suff)` Returns *the index of the word in the dictionary,* which has the prefix `pref` and the suffix `suff`. If there is more than one valid index, return **the largest** of them. If there is no such word in the dictionary, return `-1`.

Example 1:

```
Input
["WordFilter", "f"]
[[["apple"]], ["a", "e"]]
Output
[null, 0]
Explanation
WordFilter wordFilter = new WordFilter(["apple"]);
wordFilter.f("a", "e"); // return 0, because the word at index 0 has prefix = "a" and suffix = "e".
```

Constraints:

- `1 <= words.length <= 10^4`
- `1 <= words[i].length <= 7`
- `1 <= pref.length, suff.length <= 7`
- `words[i]`, `pref` and `suff` consist of lowercase English letters only.
- At most `10^4` calls will be made to the function `f`.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Return the largest index whose word has a given prefix and suffix. Words are at most length 7, so every prefix–suffix pair of every word can be stored, and a query is O(1). Later words overwrite earlier ones, so the map keeps the largest index. Empty prefix and suffix are stored as well. Keys are (pref,suff). f looks up or returns -1.

Python 3

```python
class WordFilter:
    def __init__(self, words: List[str]):
        self.d = {}
        for k, w in enumerate(words):
            n = len(w)
            for i in range(n + 1):
                a = w[:i]
                for j in range(n + 1):
                    b = w[j:]
                    self.d[(a, b)] = k

    def f(self, pref: str, suff: str) -> int:
        return self.d.get((pref, suff), -1)


# Your WordFilter object will be instantiated and called as such:
# obj = WordFilter(words)
# param_1 = obj.f(pref,suff)
```

Solution 2

How to think about it

Solution 1 stores every prefix–suffix pair. Two tries—one on the word, one on the reverse—record the indices that pass each node. A query fetches the two index lists (increasing by insert order) and walks them from the right for the largest common index. Lists stay short because words are short, without materializing every pair.

Python 3

```python
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.indexes = []

    def insert(self, word, i):
        node = self
        for c in word:
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
            node.indexes.append(i)

    def search(self, pref):
        node = self
        for c in pref:
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
                return []
            node = node.children[idx]
        return node.indexes


class WordFilter:
    def __init__(self, words: List[str]):
        self.p = Trie()
        self.s = Trie()
        for i, w in enumerate(words):
            self.p.insert(w, i)
            self.s.insert(w[::-1], i)

    def f(self, pref: str, suff: str) -> int:
        a = self.p.search(pref)
        b = self.s.search(suff[::-1])
        if not a or not b:
            return -1
        i, j = len(a) - 1, len(b) - 1
        while ~i and ~j:
            if a[i] == b[j]:
                return a[i]
            if a[i] > b[j]:
                i -= 1
            else:
                j -= 1
        return -1


# Your WordFilter object will be instantiated and called as such:
# obj = WordFilter(words)
# param_1 = obj.f(pref,suff)
```

Complexity. Time: build O(sum of word lengths); f O(p + s + N), p and s = prefix and suffix lengths, N = number of words. Space: O(sum of word lengths).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count how many words have the given prefix and suffix
# 2. New words can be added later


# ============================================================
# FOLLOW-UP 1: Count how many words have the given prefix
# and suffix
```

```python
# ============================================================

# ---- follow-up 1: function ----
# Answer: Store every (prefix, suffix) pair of every word
# in a dictionary of counts.
# Time: build O(W * L^3), count O(L).  Space: O(W * L^3).
class WordFilterCount:
    def __init__(self, words):
        self.counts = {}
        for word in set(words):
            for i in range(len(word) + 1):
                for j in range(len(word) + 1):
                    key = (word[:i], word[j:])
                    self.counts[key] = self.counts.get(key, 0) + 1

    def count(self, prefix, suffix):
        return self.counts.get((prefix, suffix), 0)


# ============================================================
# FOLLOW-UP 2: New words can be added later
# ============================================================

# ---- follow-up 2: function ----
# Answer: Store the pairs of the new word in the same
# dictionary with its index, keeping the biggest index.
# Time: add_word O(L^3), f O(L).  Space: O(W * L^3).
class WordFilterDynamic:
    def __init__(self):
        self.best = {}
        self.size = 0

    def add_word(self, word):
        for i in range(len(word) + 1):
            for j in range(len(word) + 1):
                self.best[(word[:i], word[j:])] = self.size
        self.size += 1

    def f(self, prefix, suffix):
        return self.best.get((prefix, suffix), -1)
```

1707. Maximum XOR With an Element From Array

Hard · Bit Manipulation, Trie, Array

leetcode.com/problems/maximum-xor-with-an-element-from-array

PROBLEM

You are given an array `nums` consisting of non-negative integers. You are also given a `queries` array, where `queries[i] = [x_i, m_i]`.

The answer to the `i^th` query is the maximum bitwise `XOR` value of `x_i` and any element of `nums` that does not exceed `m_i`. In other words, the answer is `max(nums[j] XOR x_i)` for all `j` such that `nums[j] <= m_i`. If all elements in `nums` are larger than `m_i`, then the answer is `-1`.

Return *an integer array* `answer` *where* `answer.length == queries.length` *and* `answer[i]` *is the answer to the* `i^th` *query.*

Example 1:

```
Input: nums = [0,1,2,3,4], queries = [[3,1],[1,3],[5,6]]
Output: [3,3,7]
Explanation:
1) 0 and 1 are the only two integers not greater than 1. 0 XOR 3 = 3 and 1 XOR 3 = 2. The larger of the two is 3.
2) 1 XOR 2 = 3.
3) 5 XOR 2 = 7.
```

Example 2:

```
Input: nums = [5,2,4,6,6,3], queries = [[12,4],[8,1],[6,3]]
Output: [15,-1,5]
```

Constraints:

- `1 <= nums.length, queries.length <= 10^5`
- `queries[i].length == 2`
- `0 <= nums[j], x_i, m_i <= 10^9`

SOLUTION (PYTHON 3)

Solution 1: Offline Query + Binary Trie

How to think about it

Each query asks for the maximum x_i xor nums[j] among values ≤ m_i. Scanning the array per query is O(nq) and fails for n,q≤ 10^5. Queries are independent of one another and of the order of nums. Sorting by m_i lets us insert eligible numbers monotonically into one structure. Sort nums and insert values ≤ m_i into a binary trie with a moving pointer. Walking opposite bits on the trie yields the maximum XOR; an empty trie answers -1.

From the problem description, we know that each query is independent and the result of the query is irrelevant to the order of elements in nums. Therefore, we consider sorting all queries in ascending order of m_i, and also sorting nums in ascending order.

Next, we use a binary trie to maintain the elements in nums. We use a pointer j to record the current elements in the trie, initially j=0. For each query $[x_i, m_i]$, we continuously insert elements from nums into the trie until nums[j] > m_i. At this point, we can query all elements not exceeding m_i in the trie, and we take the XOR value of the element with the maximum XOR value with x_i as the answer.

The time complexity is O(m × log m + n × (log n + log M)), and the space complexity is O(n × log M). Where m and n are the lengths of the arrays nums and queries respectively, and M is the maximum value in the array nums. In this problem, M ≤

10^9.

Python 3

```python
class Trie:
    __slots__ = ["children"]

    def __init__(self):
        self.children = [None] * 2

    def insert(self, x: int):
        node = self
        for i in range(30, -1, -1):
            v = x >> i & 1
            if node.children[v] is None:
                node.children[v] = Trie()
            node = node.children[v]

    def search(self, x: int) -> int:
        node = self
        ans = 0
        for i in range(30, -1, -1):
            v = x >> i & 1
            if node.children[v ^ 1]:
                ans |= 1 << i
                node = node.children[v ^ 1]
            elif node.children[v]:
                node = node.children[v]
            else:
                return -1
        return ans


class Solution:
    def maximizeXor(self, nums: List[int], queries: List[List[int]]) -> List[int]:
        trie = Trie()
        nums.sort()
        j, n = 0, len(queries)
        ans = [-1] * n
        for i, (x, m) in sorted(zip(range(n), queries), key=lambda x: x[1][1]):
            while j < len(nums) and nums[j] <= m:
                trie.insert(nums[j])
                j += 1
            ans[i] = trie.search(x)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. No limit on the element (plain maximum XOR with x)
# 2. The MINIMUM XOR with x


# ============================================================
# FOLLOW-UP 1: No limit on the element (plain maximum XOR
# with x)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Insert all numbers into a bit trie; at every bit
# prefer the opposite bit of x.
# Time: build O(31 * n), query O(31).  Space: O(31 * n).
def max_xor_with(nums, x, bits=30):
    root = {}
    for number in nums:
        node = root
        for b in range(bits, -1, -1):
```

```
            node = node.setdefault((number >> b) & 1, {})
    node = root
    result = 0
    for b in range(bits, -1, -1):
        bit = (x >> b) & 1
        if (1 - bit) in node:
            result |= 1 << b
            node = node[1 - bit]
        else:
            node = node[bit]
    return result


# ============================================================
# FOLLOW-UP 2: The MINIMUM XOR with x
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same trie, but prefer the SAME bit as x at
# every level.
# Time: build O(31 * n), query O(31).  Space: O(31 * n).
def min_xor_with(nums, x, bits=30):
    root = {}
    for number in nums:
        node = root
        for b in range(bits, -1, -1):
            node = node.setdefault((number >> b) & 1, {})
    node = root
    result = 0
    for b in range(bits, -1, -1):
        bit = (x >> b) & 1
        if bit in node:
            node = node[bit]
        else:
            result |= 1 << b
            node = node[1 - bit]
    return result
```

2416. Sum of Prefix Scores of Strings

Hard · Trie, Array, String, Counting

leetcode.com/problems/sum-of-prefix-scores-of-strings

PROBLEM

You are given an array `words` of size `n` consisting of **non-empty** strings.

We define the **score** of a string `term` as the **number** of strings `words[i]` such that `term` is a **prefix** of `words[i]`.

- For example, if `words = ["a", "ab", "abc", "cab"]`, then the score of `"ab"` is `2`, since `"ab"` is a prefix of both `"ab"` and `"abc"`.

Return *an array* `answer` *of size* `n` *where* `answer[i]` *is the* ***sum*** *of scores of every* ***non-empty*** *prefix of* `words[i]`.

Note that a string is considered as a prefix of itself.

Example 1:

```
Input: words = ["abc","ab","bc","b"]
Output: [5,4,3,2]
Explanation: The answer for each string is the following:
- "abc" has 3 prefixes: "a", "ab", and "abc".
- There are 2 strings with the prefix "a", 2 strings with the prefix "ab", and 1 string with the prefix
"abc".
The total is answer[0] = 2 + 2 + 1 = 5.
- "ab" has 2 prefixes: "a" and "ab".
- There are 2 strings with the prefix "a", and 2 strings with the prefix "ab".
The total is answer[1] = 2 + 2 = 4.
- "bc" has 2 prefixes: "b" and "bc".
- There are 2 strings with the prefix "b", and 1 string with the prefix "bc".
The total is answer[2] = 2 + 1 = 3.
- "b" has 1 prefix: "b".
- There are 2 strings with the prefix "b".
The total is answer[3] = 2.
```

Example 2:

```
Input: words = ["abcd"]
Output: [4]
Explanation:
"abcd" has 4 prefixes: "a", "ab", "abc", and "abcd".
Each prefix has a score of one, so the total is answer[0] = 1 + 1 + 1 + 1 = 4.
```

Constraints:

- `1 <= words.length <= 1000`
- `1 <= words[i].length <= 1000`
- `words[i]` consists of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Prefix Tree

> **How to think about it**
>
> A word's score is the sum of occurrence counts of all its prefixes. Scanning every word for every prefix is heavy when the total length reaches 10^6. Shared prefixes belong on a trie. Insert every word, incrementing cnt on each node. Then walk each word again and sum cnt. Time is linear in the total number of characters.

We can use a prefix tree to maintain all prefixes of the strings and count the occurrences of each prefix.

Define the prefix tree node structure `Trie`, which includes two properties:

- `children`: An array of length 26 used to store the current node's children.
- `cnt`: The occurrence count of the current node.

Define two methods for the prefix tree:

- `insert`: Inserts a string, adding its prefixes into the prefix tree.
- `search`: Searches for a string and returns the occurrence count of its prefixes.

We traverse all strings, inserting each string into the prefix tree. Then we traverse all strings again, calling the `search` method for each string and summing up the occurrence counts of each prefix.

Time complexity is O(L), and space complexity is O(L), where L is the total length of all strings.

Python 3

```python
class Trie:
    __slots__ = "children", "cnt"

    def __init__(self):
        self.children = [None] * 26
        self.cnt = 0

    def insert(self, w):
        node = self
        for c in w:
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
            node.cnt += 1

    def search(self, w):
        node = self
        ans = 0
        for c in w:
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
                return ans
            node = node.children[idx]
            ans += node.cnt
        return ans


class Solution:
    def sumPrefixScores(self, words: List[str]) -> List[int]:
        trie = Trie()
        for w in words:
            trie.insert(w)
        return [trie.search(w) for w in words]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Score of a new word that is not in the list
# 2. How many distinct non-empty prefixes are there in the
# list


# ============================================================
# FOLLOW-UP 1: Score of a new word that is not in the list
# ============================================================

# ---- follow-up 1: function ----
# Answer: Walk the trie of the list and add the counter of
# each visited node; stop when the walk leaves the trie.
# Time: build O(L), score O(len).  Space: O(L).
```

```
def build_prefix_counts(words):
    root = {}
    for word in words:
        node = root
        for ch in word:
            node = node.setdefault(ch, {})
            node["#"] = node.get("#", 0) + 1
    return root


def score_of_new_word(root, word):
    total = 0
    node = root
    for ch in word:
        if ch not in node:
            break
        node = node[ch]
        total += node["#"]
    return total


# ============================================================
# FOLLOW-UP 2: How many distinct non-empty prefixes are
# there in the list
# ============================================================

# ---- follow-up 2: function ----
# Answer: That is the number of nodes of the trie, apart
# from the root.
# Time: O(L).  Space: O(L).
def distinct_prefixes(words):
    root = {}
    count = 0
    for word in words:
        node = root
        for ch in word:
            if ch not in node:
                node[ch] = {}
                count += 1
            node = node[ch]
    return count
```

PATTERN 33 OF 41

Cyclic Sort

20 problems · 9 Easy · 9 Medium · 2 Hard

THE IDEA

When the numbers are in the range 1..n (or 0..n), put every number at its own index by swapping; whatever is left in the wrong place is the answer. O(n) time, O(1) extra space.

Signals: missing number, find all duplicates, first missing positive, set mismatch, array of n numbers in range 1..n.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
i = 0
while i < len(nums):
    j = nums[i] - 1                        # where nums[i] belongs
    if 0 <= j < len(nums) and nums[i] != nums[j]:
        nums[i], nums[j] = nums[j], nums[i]
    else:
        i += 1
for i, x in enumerate(nums):
    if x != i + 1:
        return i + 1                       # first misplaced position
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Missing Number (see page)	Easy	LC 268	1465
2	Find All Numbers Disappeared in an Array	Easy	LC 448	1760
3	Set Mismatch	Easy	LC 645	1762
4	Move Zeroes (see page)	Easy	LC 283	133
5	Degree of an Array	Easy	LC 697	1766
6	Number of Good Pairs	Easy	LC 1512	1769
7	Replace Elements with Greatest Element on Right Side	Easy	LC 1299	1771
8	Find the Difference (see page)	Easy	LC 389	1470
9	Single Number (see page)	Easy	LC 136	1459
10	★ **Find All Duplicates in an Array**	Medium	LC 442	1773
11	★ **Find the Duplicate Number** (see page)	Medium	LC 287	43
12	★ **Sort Colors** (see page)	Medium	LC 75	156
13	Minimum Number of Swaps to Make the String Balanced	Medium	LC 1963	1775
14	★ **Array Nesting**	Medium	LC 565	1778
15	★ **Maximum Swap**	Medium	LC 670	1781
16	Smallest Range II	Medium	LC 910	1784
17	Wiggle Sort II (see page)	Medium	LC 324	1246
18	Maximum Number of Integers to Choose From a Range I	Medium	LC 2554	1786
19	★ **First Missing Positive** (see page)	Hard	LC 41	65
20	★ **Couples Holding Hands**	Hard	LC 765	1789

PATTERN 33 · CYCLIC SORT · PROBLEM 2 OF 20

448. Find All Numbers Disappeared in an Array

Easy · Array, Hash Table

leetcode.com/problems/find-all-numbers-disappeared-in-an-array

PROBLEM

Given an array `nums` of `n` integers where `nums[i]` is in the range `[1, n]`, return *an array of all the integers in the range* `[1, n]` *that do not appear in* `nums`.

Example 1:

```
Input: nums = [4,3,2,7,8,2,3,1]
Output: [5,6]
```

Example 2:

```
Input: nums = [1,1]
Output: [2]
```

Constraints:

- `n == nums.length`
- `1 <= n <= 10^5`
- `1 <= nums[i] <= n`

Follow up: Could you do it without extra space and in `O(n)` runtime? You may assume the returned list does not count as extra space.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Values lie in [1,n]; missing numbers are a set difference. Store seen values in a hash set and scan 1..n. Linear time, O(n) extra space. The follow-up wants constant extra memory; this solution establishes correctness first.

Python 3

```python
class Solution:
    def findDisappearedNumbers(self, nums: List[int]) -> List[int]:
        s = set(nums)
        return [x for x in range(1, len(nums) + 1) if x not in s]
```

Solution 2

How to think about it

Solution 1 uses a set. Value x can negate index |x|-1 to mark “seen”; a still-positive slot i means i+1 is missing. Absolute value before indexing avoids treating a mark as a subscript.

Python 3

```python
class Solution:
    def findDisappearedNumbers(self, nums: List[int]) -> List[int]:
        for x in nums:
            i = abs(x) - 1
            if nums[i] > 0:
                nums[i] *= -1
        return [i + 1 for i in range(len(nums)) if nums[i] > 0]
```

Complexity. Time: O(n). Space: O(1) extra.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Some values may be outside 1..n: still return the
# missing numbers of 1..n
# 2. The smallest missing positive number (any integers)


# ============================================================
# FOLLOW-UP 1: Some values may be outside 1..n: still
# return the missing numbers of 1..n
# ============================================================

# ---- follow-up 1: function ----
# Answer: Cyclic sort that skips values outside the range.
# Time: O(n).  Space: O(1) extra.
def missing_with_outliers(nums):
    n = len(nums)
    i = 0
    while i < n:
        want = nums[i] - 1
        if 0 <= want < n and nums[want] != nums[i]:
            nums[i], nums[want] = nums[want], nums[i]
        else:
            i += 1
    return [i + 1 for i in range(n) if nums[i] != i + 1]


# ============================================================
# FOLLOW-UP 2: The smallest missing positive number (any
# integers)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same swapping; the first position that does not
# hold its own number gives the answer.
# Time: O(n).  Space: O(1) extra.
def first_missing_positive(nums):
    n = len(nums)
    i = 0
    while i < n:
        want = nums[i] - 1
        if 0 <= want < n and nums[want] != nums[i]:
            nums[i], nums[want] = nums[want], nums[i]
        else:
            i += 1
    for i in range(n):
        if nums[i] != i + 1:
            return i + 1
    return n + 1
```

PATTERN 33 · CYCLIC SORT · PROBLEM 3 OF 20

645. Set Mismatch

Easy · Bit Manipulation, Array, Hash Table, Sorting

leetcode.com/problems/set-mismatch

PROBLEM

You have a set of integers `s`, which originally contains all the numbers from `1` to `n`. Unfortunately, due to some error, one of the numbers in `s` got duplicated to another number in the set, which results in **repetition of one** number and **loss of another** number.

You are given an integer array `nums` representing the data status of this set after the error.

Find the number that occurs twice and the number that is missing and return *them in the form of an array*.

Example 1:

```
Input: nums = [1,2,2,4]
Output: [2,3]
```

Example 2:

```
Input: nums = [1,1]
Output: [1,2]
```

Constraints:

- `2 <= nums.length <= 10^4`
- `1 <= nums[i] <= 10^4`

SOLUTIONS (PYTHON 3)

Solution 1: Mathematics

> **How to think about it**
>
> One number in [1,n] is duplicated and one is missing. Sums identify both at once. Let s be the array sum, s_2 the unique-sum, s_1 the sum 1..n. The duplicate is s-s_2 and the missing is s_1-s_2.

We denote s_1 as the sum of all numbers from [1,..n], s_2 as the sum of the numbers in the array nums after removing duplicates, and s as the sum of the numbers in the array nums.

Then s - s_2 is the duplicate number, and s_1 - s_2 is the missing number.

The time complexity is O(n), and the space complexity is O(n), where n is the length of the array nums. Extra space is needed to store the array after de-duplication.

Python 3

```python
class Solution:
    def findErrorNums(self, nums: List[int]) -> List[int]:
        n = len(nums)
        s1 = (1 + n) * n // 2
        s2 = sum(set(nums))
        s = sum(nums)
        return [s - s2, s1 - s2]
```

Solution 2: Hash Table

> **How to think about it**
>
> The sum method needs a set. A frequency map makes the duplicate (2) and the missing (0) explicit.

We can also use a more intuitive method, using a hash table cnt to count the occurrence of each number in the array nums.

Next, iterate through $x \in [1, n]$, if cnt[x] = 2, then x is the duplicate number, if cnt[x] = 0, then x is the missing number.

The time complexity is O(n), and the space complexity is O(n), where n is the length of the array nums.

Python 3

```python
class Solution:
    def findErrorNums(self, nums: List[int]) -> List[int]:
        cnt = Counter(nums)
        n = len(nums)
        ans = [0] * 2
        for x in range(1, n + 1):
            if cnt[x] == 2:
                ans[0] = x
            if cnt[x] == 0:
                ans[1] = x
        return ans
```

Solution 3: Bit Operation

> **How to think about it**
>
> The first two methods use linear extra memory. XOR of nums and 1..n yields axor b; split by the lowest set bit to recover a and b, then see which one appears in the array.

According to the properties of the XOR operation, for integer x, we have x xor x = 0 and x xor 0 = x. Therefore, if we perform the XOR operation on all elements in the array nums and all numbers $i \in [1, n]$, we can eliminate the numbers that appear twice, leaving only the XOR result of the missing number and the duplicate number, i.e., xs = a xor b.

Since these two numbers are not equal, there must be at least one bit in the XOR result that is 1. We find the lowest bit of 1 in the XOR result through the lowbit operation, and then divide all numbers in the array nums and all numbers $i \in [1, n]$ into two groups according to whether this bit is 1. In this way, the two numbers are divided into different groups. The XOR result of one group of numbers is a, and the XOR result of the other group is b. These two numbers are the answers we are looking for.

Next, we only need to determine which of a and b is the duplicate number and which is the missing number. Therefore, iterate through the array nums, for the traversed number x, if x=a, then a is the duplicate number, return [a, b], otherwise, at the end of the iteration, return [b, a].

The time complexity is O(n), where n is the length of the array nums. The space complexity is O(1), only using a constant size of extra space.

Python 3

```python
class Solution:
    def findErrorNums(self, nums: List[int]) -> List[int]:
        xs = 0
        for i, x in enumerate(nums, 1):
            xs ^= i ^ x
        a = 0
        lb = xs & -xs
        for i, x in enumerate(nums, 1):
            if i & lb:
                a ^= i
            if x & lb:
                a ^= x
        b = xs ^ a
        for x in nums:
            if x == a:
                return [a, b]
        return [b, a]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Several numbers can be repeated or missing: return
# both lists
# 2. Find them with XOR only (no extra memory, no
# swapping)


# ============================================================
# FOLLOW-UP 1: Several numbers can be repeated or missing:
# return both lists
# ============================================================

# ---- follow-up 1: function ----
# Answer: Cyclic sort; positions holding the wrong number
# show the missing value, and the values sitting there are
# the repeated ones.
# Time: O(n).  Space: O(n) for the answer lists.
def all_repeated_and_missing(nums):
    n = len(nums)
    i = 0
    while i < n:
        want = nums[i] - 1
        if nums[want] != nums[i]:
            nums[i], nums[want] = nums[want], nums[i]
        else:
            i += 1
    repeated = []
    missing = []
    for i in range(n):
        if nums[i] != i + 1:
            repeated.append(nums[i])
            missing.append(i + 1)
    return repeated, missing


# ============================================================
# FOLLOW-UP 2: Find them with XOR only (no extra memory,
# no swapping)
# ============================================================

# ---- follow-up 2: function ----
# Answer: XOR of all numbers with 1..n equals repeated ^
# missing; its lowest set bit splits all numbers into two
# groups that separate the two values.
# Time: O(n^2) in the worst case (the check `a in nums`
```

```
# scans the list), otherwise O(n).  Space: O(1).
def mismatch_with_xor(nums):
    n = len(nums)
    both = 0
    for i in range(n):
        both ^= nums[i] ^ (i + 1)
    bit = both & -both
    a = 0
    b = 0
    for i in range(n):
        for x in (nums[i], i + 1):
            if x & bit:
                a ^= x
            else:
                b ^= x
```

```
    if a in nums:
        return [a, b]
    return [b, a]
```

PATTERN 33 · CYCLIC SORT · PROBLEM 5 OF 20

697. Degree of an Array

Easy · Array, Hash Table

leetcode.com/problems/degree-of-an-array

PROBLEM

Given a non-empty array of non-negative integers `nums`, the **degree** of this array is defined as the maximum frequency of any one of its elements.

Your task is to find the smallest possible length of a (contiguous) subarray of `nums`, that has the same degree as `nums`.

Example 1:

```
Input: nums = [1,2,2,3,1]
Output: 2
Explanation:
The input array has a degree of 2 because both elements 1 and 2 appear twice.
Of the subarrays that have the same degree:
[1, 2, 2, 3, 1], [1, 2, 2, 3], [2, 2, 3, 1], [1, 2, 2], [2, 2, 3], [2, 2]
The shortest length is 2. So return 2.
```

Example 2:

```
Input: nums = [1,2,2,3,1,4,2]
Output: 6
Explanation:
The degree is 3 because the element 2 is repeated 3 times.
So [2,2,3,1,4,2] is the shortest subarray, therefore returning 6.
```

Constraints:

- `nums.length` will be between 1 and 50,000.
- `nums[i]` will be an integer between 0 and 49,999.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

The degree is the maximum frequency. A shortest subarray with the same degree must cover the first and last occurrence of some mode. Count frequencies and record endpoints. For values that attain the degree, take the minimum right-left+1.

Python 3

```python
class Solution:
    def findShortestSubArray(self, nums: List[int]) -> int:
        cnt = Counter(nums)
        degree = cnt.most_common()[0][1]
        left, right = {}, {}
        for i, v in enumerate(nums):
            if v not in left:
                left[v] = i
            right[v] = i
        ans = inf
        for v in nums:
            if cnt[v] == degree:
                t = right[v] - left[v] + 1
                if ans > t:
                    ans = t
        return ans
```

Complexity. Time: O(n). Space: O(n).

Solution 2

> **How to think about it**
>
> Method 1 records endpoints in one pass. We can also collect the modes first and scan first/last indices per mode. The helpers are longer, the complexity is the same.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```
class Solution:
    def findShortestSubArray(self, nums: List[int]) -> int:
        count = Counter(nums)
        degree = max(count.values())
        answer = len(nums)
        for value, c in count.items():
            if c == degree:
                first = nums.index(value)
                last = len(nums) - 1 - nums[::-1].index(value)
                answer = min(answer, last - first + 1)
        return answer
```

Complexity. Time: O(n * c), c = number of values that reach the degree. Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the shortest subarray itself
# 2. List all values that reach the degree


# ============================================================
# FOLLOW-UP 1: Return the shortest subarray itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember the first and last position of every
# value; for the values with the highest count pick the
# one with the shortest span.
# Time: O(n).  Space: O(d), d = distinct values.
def shortest_subarray_with_degree(nums):
    first = {}
    last = {}
    count = {}
    for i in range(len(nums)):
        if nums[i] not in first:
            first[nums[i]] = i
        last[nums[i]] = i
        count[nums[i]] = count.get(nums[i], 0) + 1
    degree = max(count.values())
    best = None
    for value in count:
        if count[value] == degree:
            if best is None or last[value] - first[value] < last[best] - first[best]:
                best = value
    return nums[first[best]:last[best] + 1]


# ============================================================
# FOLLOW-UP 2: List all values that reach the degree
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the values and keep those with the maximum
# count.
```

```
# Time: O(n + d log d).  Space: O(d).
def values_with_degree(nums):
    count = {}
    for x in nums:
        count[x] = count.get(x, 0) + 1
    degree = max(count.values())
    return sorted(x for x in count if count[x] == degree)
```

1512. Number of Good Pairs

Easy · Array, Hash Table, Math, Counting

leetcode.com/problems/number-of-good-pairs

PROBLEM

Given an array of integers `nums`, return *the number of **good pairs***.

A pair `(i, j)` is called *good* if `nums[i] == nums[j]` and `i < j`.

Example 1:

```
Input: nums = [1,2,3,1,1,3]
Output: 4
Explanation: There are 4 good pairs (0,3), (0,4), (3,4), (2,5) 0-indexed.
```

Example 2:

```
Input: nums = [1,1,1,1]
Output: 6
Explanation: Each pair in the array are good.
```

Example 3:

```
Input: nums = [1,2,3]
Output: 0
```

Constraints:

- `1 <= nums.length <= 100`
- `1 <= nums[i] <= 100`

SOLUTION (PYTHON 3)

Solution 1: Counting

How to think about it

Count index pairs with i<j and nums[i]=nums[j]. A double loop would pass for n≤ 100, but it rescans earlier equals on every step. The current x forms one pair with each previous x. A frequency map of values already seen lets us add that count and then increment, so one pass suffices.

Traverse the array, and for each element x, count how many elements before it are equal to x. This count represents the number of good pairs formed by x and the previous elements. After traversing the entire array, we obtain the answer.

The time complexity is O(n), and the space complexity is O(C). Here, n is the length of the array, and C is the range of values in the array. In this problem, C = 101.

Python 3

```python
class Solution:
    def numIdenticalPairs(self, nums: List[int]) -> int:
        ans = 0
        cnt = Counter()
        for x in nums:
            ans += cnt[x]
            cnt[x] += 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the pairs themselves
# 2. Count pairs i < j with nums[i] - nums[j] == k


# ============================================================
# FOLLOW-UP 1: Return the pairs themselves
# ============================================================

# ---- follow-up 1: function ----
# Answer: Group the indices by value and list every pair
# inside each group.
# Time: O(n + P log P), P = pairs.  Space: O(n + P).
def good_pair_list(nums):
    positions = {}
    for i in range(len(nums)):
        positions.setdefault(nums[i], []).append(i)
    pairs = []
    for group in positions.values():
        for a in range(len(group)):
            for b in range(a + 1, len(group)):
                pairs.append((group[a], group[b]))
    return sorted(pairs)


# ============================================================
# FOLLOW-UP 2: Count pairs i < j with nums[i] - nums[j] ==
# k
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk left to right and count how many earlier
# numbers equal nums[j] + k.
# Time: O(n).  Space: O(n).
def pairs_with_difference(nums, k):
    seen = {}
    total = 0
    for x in nums:
        total += seen.get(x + k, 0)
        seen[x] = seen.get(x, 0) + 1
    return total
```

PATTERN 33 · CYCLIC SORT · PROBLEM 7 OF 20

1299. Replace Elements with Greatest Element on Right Side

Easy · Array

leetcode.com/problems/replace-elements-with-greatest-element-on-right-side

PROBLEM

Given an array `arr`, replace every element in that array with the greatest element among the elements to its right, and replace the last element with `-1`.

After doing so, return the array.

Example 1:

```
Input: arr = [17,18,5,4,6,1]
Output: [18,6,6,6,1,-1]
Explanation:
- index 0 --> the greatest element to the right of index 0 is index 1 (18).
- index 1 --> the greatest element to the right of index 1 is index 4 (6).
- index 2 --> the greatest element to the right of index 2 is index 4 (6).
- index 3 --> the greatest element to the right of index 3 is index 4 (6).
- index 4 --> the greatest element to the right of index 4 is index 5 (1).
- index 5 --> there are no elements to the right of index 5, so we put -1.
```

Example 2:

```
Input: arr = [400]
Output: [-1]
Explanation: There are no elements to the right of index 0.
```

Constraints:

- `1 <= arr.length <= 10^4`
- `1 <= arr[i] <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: Reverse Traversal

How to think about it

Each index becomes the maximum to its right; the last is -1. Scanning the suffix from the left repeats work. Walking right to left, we keep a suffix max mx, write the old mx into the cell, then update mx with the original value. One reverse pass, O(1) extra space.

We use a variable mx to record the maximum value to the right of the current position, initially mx = -1.

Then we traverse the array from right to left. For each position i, we denote the current value as x, update the current position's value to mx, and then update mx = max(mx, x).

Finally, return the original array.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Python 3

```python
class Solution:
    def replaceElements(self, arr: List[int]) -> List[int]:
        mx = -1
        for i in reversed(range(len(arr))):
            x = arr[i]
            arr[i] = mx
            mx = max(mx, x)
        return arr
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Replace with the SMALLEST element on the right
# 2. Replace with the first element on the right that is
# strictly greater (or -1)


# ============================================================
# FOLLOW-UP 1: Replace with the SMALLEST element on the
# right
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same right-to-left walk, keeping the
# minimum.
# Time: O(n).  Space: O(1).
def replace_with_smallest_on_right(arr):
    smallest = -1
    for i in range(len(arr) - 1, -1, -1):
        current = arr[i]
        arr[i] = smallest
        if smallest == -1 or current < smallest:
            smallest = current
    return arr


# ============================================================
# FOLLOW-UP 2: Replace with the first element on the right
# that is strictly greater (or -1)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk from the right with a stack of candidates,
# popping the ones that are not greater.
# Time: O(n).  Space: O(n).
def replace_with_next_greater(arr):
    stack = []
    result = [-1] * len(arr)
    for i in range(len(arr) - 1, -1, -1):
        while stack and stack[-1] <= arr[i]:
            stack.pop()
        if stack:
            result[i] = stack[-1]
        stack.append(arr[i])
    return result
```

PATTERN 33 · CYCLIC SORT · PROBLEM 10 OF 20

442. Find All Duplicates in an Array

Medium · Array, Hash Table, Sorting

leetcode.com/problems/find-all-duplicates-in-an-array

★ **TOP 7 PROBLEM** in Cyclic Sort: master this one first

PROBLEM

Given an integer array `nums` of length `n` where all the integers of `nums` are in the range `[1, n]` and each integer appears **at most twice**, return *an array of all the integers that appears **twice***.

You must write an algorithm that runs in `O(n)` time and uses only *constant* auxiliary space, excluding the space needed to store the output

Example 1:

```
Input: nums = [4,3,2,7,8,2,3,1]
Output: [2,3]
```

Example 2:

```
Input: nums = [1,1,2]
Output: [1]
```

Example 3:

```
Input: nums = [1]
Output: []
```

Constraints:

- `n == nums.length`
- `1 <= n <= 10^5`
- `1 <= nums[i] <= n`
- Each element in `nums` appears **once** or **twice**.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Values lie in [1,n] and appear at most twice; the follow-up wants linear time and constant extra memory. A hash set finds duplicates but uses O(n) space. Swap v into index v-1 (cycle sort). Afterwards a value that is not i+1 at index i is a leftover duplicate. The swap loop stops when nums[i]=nums[nums[i]-1], so a duplicate collides with the value already in place and never cycles forever.

Python 3

```python
class Solution:
    def findDuplicates(self, nums: List[int]) -> List[int]:
        for i in range(len(nums)):
            while nums[i] != nums[nums[i] - 1]:
                nums[nums[i] - 1], nums[i] = nums[i], nums[nums[i] - 1]
        return [v for i, v in enumerate(nums) if v != i + 1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Use cyclic sort (swap into place) instead of negation
# 2. Each number appears one, two or three times: return
# those appearing exactly twice


# ============================================================
# FOLLOW-UP 1: Use cyclic sort (swap into place) instead
# of negation
# ============================================================

# ---- follow-up 1: function ----
# Answer: After sorting by swapping, a number sitting at
# the wrong place is a duplicate.
# Time: O(n + k log k).  Space: O(k) for the answer.
def duplicates_by_swapping(nums):
    i = 0
    while i < len(nums):
        want = nums[i] - 1
        if nums[want] != nums[i]:
            nums[i], nums[want] = nums[want], nums[i]
        else:
            i += 1
    return sorted(set(nums[i] for i in range(len(nums)) if nums[i] != i + 1))


# ============================================================
# FOLLOW-UP 2: Each number appears one, two or three
# times: return those appearing exactly twice
# ============================================================

# ---- follow-up 2: function ----
# Answer: Add n to the slot of each number as a counter
# (read the count with integer division).
# Time: O(n).  Space: O(1) extra.
def appearing_exactly_twice(nums):
    n = len(nums)
    for x in nums:
        nums[(x - 1) % n] += n
    return [i + 1 for i in range(n) if (nums[i] - 1) // n == 2]
```

PATTERN 33 · CYCLIC SORT · PROBLEM 13 OF 20

1963. Minimum Number of Swaps to Make the String Balanced

Medium · Stack, Greedy, Two Pointers, String, Parentheses

leetcode.com/problems/minimum-number-of-swaps-to-make-the-string-balanced

PROBLEM

You are given a **0-indexed** string `s` of **even** length `n`. The string consists of **exactly** `n / 2` opening brackets `'['` and `n / 2` closing brackets `']'`.

A string is called **balanced** if and only if:

- It is the empty string, or
- It can be written as `AB`, where both `A` and `B` are **balanced** strings, or
- It can be written as `[C]`, where `C` is a **balanced** string.

You may swap the brackets at **any** two indices **any** number of times.

Return *the **minimum** number of swaps to make* `s` ***balanced***.

Example 1:

```
Input: s = "][]["
Output: 1
Explanation: You can make the string balanced by swapping index 0 with index 3.
The resulting string is "[[]]".
```

Example 2:

```
Input: s = "]]][[["
Output: 2
Explanation: You can do the following to make the string balanced:
- Swap index 0 with index 4. s = "[]][][".
- Swap index 1 with index 5. s = "[[][]]".
The resulting string is "[[][]]".
```

Example 3:

```
Input: s = "[]"
Output: 0
Explanation: The string is already balanced.
```

Constraints:

- `n == s.length`
- `2 <= n <= 10^6`
- `n` is even.
- `s[i]` is either `'['` or `']'`.
- The number of opening brackets `'['` equals `n / 2`, and the number of closing brackets `']'` equals `n / 2`.

SOLUTION (PYTHON 3)

Solution 1: Greedy

> **How to think about it**
>
> Any two brackets may be swapped. After matching, the leftover is some `]`s followed by `[`s. If x left brackets remain unmatched, each swap of the two ends removes two of them, so floor((x+1)/2) swaps suffice. One counter computes x.

We use a variable x to record the current number of unmatched left brackets. We traverse the string s, for each character c:

- If c is a left bracket, then we increment x by one;

- If c is a right bracket, then we need to check whether x is greater than zero. If it is, we match the current right bracket with the nearest unmatched left bracket on the left, i.e., decrement x by one.

After the traversal, we will definitely get a string of the form `"]]]...[[[..."`. We then greedily swap the brackets at both ends each time, which can eliminate 2 unmatched left brackets at a time. Therefore, the total number of swaps needed is floor((x + 1)/2).

The time complexity is O(n), where n is the length of the string s. The space complexity is O(1).

Python 3

```
class Solution:
    def minSwaps(self, s: str) -> int:
        x = 0
        for c in s:
            if c == "[":
                x += 1
            elif x:
                x -= 1
        return (x + 1) >> 1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the swaps (index pairs) that fix the string
# 2. Only ADJACENT swaps are allowed: the minimum number
# needed


# ============================================================
# FOLLOW-UP 1: Return the swaps (index pairs) that fix the
# string
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use two pointers: the left pointer finds an
# unmatched ']' and the right pointer the last '['; swap
# them.
# Time: O(n).  Space: O(n).
def balancing_swaps(s):
    chars = list(s)
    swaps = []
    open_count = 0
    j = len(chars) - 1
    for i in range(len(chars)):
        if chars[i] == "[":
            open_count += 1
        elif open_count > 0:
            open_count -= 1
        else:
            while chars[j] != "[":
                j -= 1
            chars[i], chars[j] = chars[j], chars[i]
            swaps.append((i, j))
            open_count += 1
    return swaps


# ============================================================
# FOLLOW-UP 2: Only ADJACENT swaps are allowed: the
# minimum number needed
# ============================================================

# ---- follow-up 2: function ----
# Answer: Greedy: whenever the next ']' would go negative,
# bring the nearest '[' to this position with adjacent
# swaps and count the distance.
```

```
# Time: O(n^2) in the worst case.  Space: O(n).
def adjacent_swaps_to_balance(s):
    chars = list(s)
    total = 0
    balance = 0
    for i in range(len(chars)):
        if chars[i] == "[":
            balance += 1
        else:
            balance -= 1
        if balance < 0:
            j = i + 1
            while chars[j] != "[":
                j += 1
            while j > i:
                chars[j], chars[j - 1] = chars[j - 1], chars[j]
```

```
                total += 1
                j -= 1
            balance = 1
    return total
```

PATTERN 33 · CYCLIC SORT · PROBLEM 14 OF 20

565. Array Nesting

Medium · Depth-First Search, Array

leetcode.com/problems/array-nesting

★ **TOP 7 PROBLEM** in Cyclic Sort: master this one first

PROBLEM

You are given an integer array `nums` of length `n` where `nums` is a permutation of the numbers in the range `[0, n - 1]`.

You should build a set `s[k] = {nums[k], nums[nums[k]], nums[nums[nums[k]]], ... }` subjected to the following rule:

- The first element in `s[k]` starts with the selection of the element `nums[k]` of `index = k`.
- The next element in `s[k]` should be `nums[nums[k]]`, and then `nums[nums[nums[k]]]`, and so on.
- We stop adding right before a duplicate element occurs in `s[k]`.

Return *the longest length of a set* `s[k]`.

Example 1:

```
Input: nums = [5,4,0,3,1,6,2]
Output: 4
Explanation:
nums[0] = 5, nums[1] = 4, nums[2] = 0, nums[3] = 3, nums[4] = 1, nums[5] = 6, nums[6] = 2.
One of the longest sets s[k]:
s[0] = {nums[0], nums[5], nums[6], nums[2]} = {5, 6, 2, 0}
```

Example 2:

```
Input: nums = [0,1,2]
Output: 1
```

Constraints:

- $1 <= nums.length <= 10^5$
- `0 <= nums[i] < nums.length`
- All the values of `nums` are **unique**.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

A[i] points to A[A[i]], partitioning the array into disjoint cycles. A nest is a cycle. Walking from every index would revisit the same cycle. A visited array marks seen indices; start a walk only from an unseen index and count the cycle. Disjointness visits each index once. The longest cycle is the answer.

Python 3

```python
class Solution:
    def arrayNesting(self, nums: List[int]) -> int:
        n = len(nums)
        vis = [False] * n
        res = 0
        for i in range(n):
            if vis[i]:
                continue
            cur, m = nums[i], 1
            vis[cur] = True
            while nums[cur] != nums[i]:
                cur = nums[cur]
                m += 1
                vis[cur] = True
            res = max(res, m)
        return res
```

Complexity. Time: O(n). Space: O(n).

Solution 2

> **How to think about it**
>
> Solution 1 uses an O(n) visited array. Values already lie in [0,n-1], so the sentinel n can mark a visited cell in place. Walk the cycle, write n into each cell, and count. A stored n means this index was already processed. Extra space becomes constant; the cycles are the same.

Python 3

```python
class Solution:
    def arrayNesting(self, nums: List[int]) -> int:
        ans, n = 0, len(nums)
        for i in range(n):
            cnt = 0
            while nums[i] != n:
                j = nums[i]
                nums[i] = n
                i = j
                cnt += 1
            ans = max(ans, cnt)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the longest cycle itself
# 2. How many cycles does the permutation have


# ============================================================
# FOLLOW-UP 1: Return the longest cycle itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Walk each unvisited cycle and keep the longest
# one.
# Time: O(n).  Space: O(n).
def longest_cycle_of_permutation(nums):
    seen = [False] * len(nums)
    best = []
    for start in range(len(nums)):
        if seen[start]:
            continue
        cycle = []
```

```
        x = start
        while not seen[x]:
            seen[x] = True
            cycle.append(nums[x])
            x = nums[x]
        if len(cycle) > len(best):
            best = cycle
    return best


# ============================================================
# FOLLOW-UP 2: How many cycles does the permutation have
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count how many times a new unvisited start is
# found.
# Time: O(n).  Space: O(n).
def number_of_cycles(nums):
    seen = [False] * len(nums)
    count = 0
    for start in range(len(nums)):
        if not seen[start]:
            count += 1
            x = start
            while not seen[x]:
                seen[x] = True
                x = nums[x]
    return count
```

PATTERN 33 · CYCLIC SORT · PROBLEM 15 OF 20

670. Maximum Swap

Medium · Greedy, Math

leetcode.com/problems/maximum-swap

★ **TOP 7 PROBLEM** in Cyclic Sort: master this one first

PROBLEM

You are given an integer `num`. You can swap two digits at most once to get the maximum valued number.

Return *the maximum valued number you can get*.

Example 1:

```
Input: num = 2736
Output: 7236
Explanation: Swap the number 2 and the number 7.
```

Example 2:

```
Input: num = 9973
Output: 9973
Explanation: No swap.
```

Constraints:

- $0 <= num <= 10^8$

SOLUTIONS (PYTHON 3)

Solution 1: Greedy Algorithm

How to think about it

Swap at most two digits to maximize the number. Trying every pair is unnecessary. From the right, record the index of the largest digit to the right (inclusive). From the left, swap at the first s[i]<s[d[i]]. That raises the highest possible place.

First, we convert the number into a string s. Then, we traverse the string s from right to left, using an array or hash table d to record the position of the maximum number to the right of each number (it can be the position of the number itself).

Next, we traverse d from left to right. If s[i] < s[d[i]], we swap them and exit the traversal process.

Finally, we convert the string s back into a number, which is the answer.

The time complexity is O(log M), and the space complexity is O(log M). Here, M is the range of the number num.

Python 3

```python
class Solution:
    def maximumSwap(self, num: int) -> int:
        s = list(str(num))
        n = len(s)
        d = list(range(n))
        for i in range(n - 2, -1, -1):
            if s[i] <= s[d[i + 1]]:
                d[i] = d[i + 1]
        for i, j in enumerate(d):
            if s[i] < s[j]:
                s[i], s[j] = s[j], s[i]
                break
        return int(''.join(s))
```

Solution 2: Space Optimized Greedy

How to think about it

Method 1 stores a right-max index array. One right-to-left scan can keep the best seen digit and the leftmost profitable swap, without the extra array.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```
class Solution:
    def maximumSwap(self, num: int) -> int:
        digits = list(str(num))
        low = high = maybe_high = -1
        for i in range(len(digits) - 1, -1, -1):
            if maybe_high == -1 or digits[i] > digits[maybe_high]:
                maybe_high = i
            if i < maybe_high and digits[i] < digits[maybe_high]:
                low, high = i, maybe_high
        if low != -1 and high != -1 and low < high:
            digits[low], digits[high] = digits[high], digits[low]
        return int("".join(digits))
```

Complexity. Time: O(d), d = number of digits. Space: O(d).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Swap once to get the SMALLEST number (no leading
# zero)
# 2. At most k swaps to get the largest number


# ============================================================
# FOLLOW-UP 1: Swap once to get the SMALLEST number (no
# leading zero)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Mirror image: find the first digit that has a
# smaller digit after it and swap with the last smallest
# one (skipping zero for the first position).
# Time: O(d), d = digits.  Space: O(d).
def minimum_swap(num):
    digits = list(str(num))
    last = {}
    for i in range(len(digits)):
        last[digits[i]] = i
    for i in range(len(digits)):
        for d in "0123456789":
            if d < digits[i] and d in last and last[d] > i and not (i == 0 and d == "0"):
                digits[i], digits[last[d]] = digits[last[d]], digits[i]
                return int("".join(digits))
    return num


# ============================================================
# FOLLOW-UP 2: At most k swaps to get the largest number
# ============================================================

# ---- follow-up 2: function ----
# Answer: Try all swap sequences level by level, keeping
# each distinct string once.
# Time: O(k * S * d^2), S = distinct strings reached (up
# to d!).  Space: O(S * d).
def maximum_with_k_swaps(num, k):
    level = {str(num)}
    for _ in range(k):
```

```
        next_level = set(level)
        for text in level:
            for i in range(len(text)):
                for j in range(i + 1, len(text)):
                    swapped = list(text)
                    swapped[i], swapped[j] = swapped[j], swapped[i]
                    next_level.add("".join(swapped))
        level = next_level
    return int(max(level))
```

PATTERN 33 · CYCLIC SORT · PROBLEM 16 OF 20

910. Smallest Range II

Medium · Greedy, Array, Math, Sorting

leetcode.com/problems/smallest-range-ii

PROBLEM

You are given an integer array `nums` and an integer `k`.

For each index `i` where `0 <= i < nums.length`, change `nums[i]` to be either `nums[i] + k` or `nums[i] - k`.

The **score** of `nums` is the difference between the maximum and minimum elements in `nums`.

Return *the minimum **score** of* `nums` *after changing the values at each index*.

Example 1:

```
Input: nums = [1], k = 0
Output: 0
Explanation: The score is max(nums) - min(nums) = 1 - 1 = 0.
```

Example 2:

```
Input: nums = [0,10], k = 2
Output: 6
Explanation: Change nums to be [2, 8]. The score is max(nums) - min(nums) = 8 - 2 = 6.
```

Example 3:

```
Input: nums = [1,3,6], k = 3
Output: 3
Explanation: Change nums to be [4, 6, 3]. The score is max(nums) - min(nums) = 6 - 3 = 3.
```

Constraints:

- `1 <= nums.length <= 10^4`
- `0 <= nums[i] <= 10^4`
- `0 <= k <= 10^4`

SOLUTION (PYTHON 3)

Solution 1: Greedy + Enumeration

How to think about it

Each value becomes +k or -k; 2^n assignments are impossible. After sorting, smaller numbers should take +k and larger ones -k, so an optimal split is a prefix of +k and a suffix of -k. For each cut i, the new minimum is min(nums[0]+k, nums[i]-k) and the new maximum is max(nums[i-1]+k, nums[-1]-k). Keep the smallest gap, including the original one.

According to the problem requirements, we need to find the minimum difference between the maximum and minimum values in the array. Each element can be increased or decreased by k, so we can divide the elements in the array into two parts, one part increased by k and the other part decreased by k. Therefore, we should decrease the larger values in the array by k and increase the smaller values by k to ensure the minimum difference between the maximum and minimum values.

Therefore, we can first sort the array, then enumerate each element in the array, divide it into two parts, one part increased by k and the other part decreased by k, and calculate the difference between the maximum and minimum values. Finally, take the minimum value among all differences.

The time complexity is $O(n \times \log n)$, and the space complexity is $O(\log n)$, where n is the length of the array.

Python 3

```python
class Solution:
    def smallestRangeII(self, nums: List[int], k: int) -> int:
        nums.sort()
        ans = nums[-1] - nums[0]
        for i in range(1, len(nums)):
            mi = min(nums[0] + k, nums[i] - k)
            mx = max(nums[i - 1] + k, nums[-1] - k)
            ans = min(ans, mx - mi)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Each number may change by ANY amount from -k to k:
# the smallest possible range
# 2. Return the array that reaches the best +k/-k answer


# ============================================================
# FOLLOW-UP 1: Each number may change by ANY amount from
# -k to k: the smallest possible range
# ============================================================

# ---- follow-up 1: function ----
# Answer: If each number may change by any amount in [-k,
# k], the range shrinks by 2k at most, but not below zero.
# Time: O(1) after finding min and max, O(n).  Space:
# O(1).
def smallest_range_any_shift(nums, k):
    return max(0, max(nums) - min(nums) - 2 * k)


# ============================================================
# FOLLOW-UP 2: Return the array that reaches the best
# +k/-k answer
# ============================================================

# ---- follow-up 2: function ----
# Answer: Sort the numbers; try every split point where
# the smaller numbers get +k and the bigger ones get -k,
# and return the best split and its array.
# Time: O(n log n).  Space: O(n).
def array_for_best_split(nums, k):
    nums = sorted(nums)
    best_value = nums[-1] - nums[0]
    best_split = 0
    for i in range(len(nums) - 1):
        high = max(nums[-1] - k, nums[i] + k)
        low = min(nums[0] + k, nums[i + 1] - k)
        if high - low < best_value:
            best_value = high - low
            best_split = i + 1
    changed = []
    for i in range(len(nums)):
        if i < best_split:
            changed.append(nums[i] + k)
        else:
            changed.append(nums[i] - k)
    return best_value, changed
```

PATTERN 33 · CYCLIC SORT · PROBLEM 18 OF 20

2554. Maximum Number of Integers to Choose From a Range I

Medium · Greedy, Array, Hash Table, Binary Search, Sorting

leetcode.com/problems/maximum-number-of-integers-to-choose-from-a-range-i

PROBLEM

You are given an integer array `banned` and two integers `n` and `maxSum`. You are choosing some number of integers following the below rules:

- The chosen integers have to be in the range `[1, n]`.
- Each integer can be chosen **at most once**.
- The chosen integers should not be in the array `banned`.
- The sum of the chosen integers should not exceed `maxSum`.

Return *the **maximum** number of integers you can choose following the mentioned rules*.

Example 1:

```
Input: banned = [1,6,5], n = 5, maxSum = 6
Output: 2
Explanation: You can choose the integers 2 and 4.
2 and 4 are from the range [1, 5], both did not appear in banned, and their sum is 6, which did not
exceed maxSum.
```

Example 2:

```
Input: banned = [1,2,3,4,5,6,7], n = 8, maxSum = 1
Output: 0
Explanation: You cannot choose any integer while following the mentioned conditions.
```

Example 3:

```
Input: banned = [11], n = 7, maxSum = 50
Output: 7
Explanation: You can choose the integers 1, 2, 3, 4, 5, 6, and 7.
They are from the range [1, 7], all did not appear in banned, and their sum is 28, which did not exceed
maxSum.
```

Constraints:

- `1 <= banned.length <= 10^4`
- `1 <= banned[i], n <= 10^4`
- `1 <= maxSum <= 10^9`

SOLUTIONS (PYTHON 3)

Solution 1: Greedy + Enumeration

> **How to think about it**
>
> Pick as many distinct integers from [1,n] as possible, skipping banned, with sum at most maxSum. Smaller values leave more room, so take them first. For n≤ 10^4, walk i=1,2,…, skip banned values via a set, and stop when the next i would exceed the remaining sum.

We use the variable s to represent the sum of the currently selected integers, and the variable ans to represent the number of currently selected integers. We convert the array `banned` into a hash table for easy determination of whether a certain integer is not selectable.

Next, we start enumerating the integer i from 1. If s + i ≤ maxSum and i is not in `banned`, then we can select the integer i, and add i and 1 to s and ans respectively.

Finally, we return ans.

The time complexity is O(n), and the space complexity is O(n). Where n is the given integer.

Python 3

```
class Solution:
    def maxCount(self, banned: List[int], n: int, maxSum: int) -> int:
        ans = s = 0
        ban = set(banned)
        for i in range(1, n + 1):
            if s + i > maxSum:
                break
            if i not in ban:
                ans += 1
                s += i
        return ans
```

Solution 2: Greedy + Binary Search

> **How to think about it**
>
> Solution 1 is linear in n and fails when n is huge. Banned values split [1,n] into contiguous gaps whose prefix sums are arithmetic; binary search finds how many we can still afford. Fill gaps from the left until the budget is gone.

If n is very large, the enumeration in Method One will time out.

We can add 0 and n + 1 to the array `banned`, deduplicate the array `banned`, remove elements greater than n+1, and then sort it.

Next, we enumerate every two adjacent elements i and j in the array `banned`. The range of selectable integers is [i + 1, j - 1]. We use binary search to enumerate the number of elements we can select in this range, find the maximum number of selectable elements, and then add it to ans. At the same time, we subtract the sum of these elements from `maxSum`. If `maxSum` is less than 0, we break the loop. Return the answer.

The time complexity is O(n × log n), and the space complexity is O(n). Where n is the length of the array `banned`.

Similar problems:

- 2557. Maximum Number of Integers to Choose From a Range II

Python 3

```
class Solution:
    def maxCount(self, banned: List[int], n: int, maxSum: int) -> int:
        banned.extend([0, n + 1])
        ban = sorted(x for x in set(banned) if x < n + 2)
        ans = 0
        for i, j in pairwise(ban):
            left, right = 0, j - i - 1
            while left < right:
                mid = (left + right + 1) >> 1
                if (i + 1 + i + mid) * mid // 2 <= maxSum:
                    left = mid
                else:
                    right = mid - 1
            ans += left
            maxSum -= (i + 1 + i + left) * left // 2
            if maxSum <= 0:
                break
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the numbers that are chosen
# 2. n is up to a billion: count without looping over
# every number


# ============================================================
# FOLLOW-UP 1: Return the numbers that are chosen
# ============================================================

# ---- follow-up 1: function ----
# Answer: Greedily take the smallest allowed numbers while
# the sum stays within maxSum.
# Time: O(n + b).  Space: O(b + c), c = chosen.
def chosen_numbers(banned, n, max_sum):
    blocked = set(banned)
    chosen = []
    total = 0
    for x in range(1, n + 1):
        if x in blocked:
            continue
        if total + x > max_sum:
            break
        chosen.append(x)
        total += x
    return chosen


# ============================================================
# FOLLOW-UP 2: n is up to a billion: count without looping
# over every number
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk the sorted banned numbers; between two
# banned numbers take a block of consecutive numbers using
# binary search on the block size.
# Time: O(b log b + b log n).  Space: O(b).
def count_chosen_huge(banned, n, max_sum):
    cuts = [0] + sorted(set(b for b in banned if b <= n)) + [n + 1]
    count = 0
    for i in range(len(cuts) - 1):
        low, high = cuts[i] + 1, cuts[i + 1] - 1
        if low > high:
            continue
        left, right = 0, high - low + 1
        while left < right:
            middle = (left + right + 1) // 2
            block_sum = middle * (low + low + middle - 1) // 2
            if block_sum <= max_sum:
                left = middle
            else:
                right = middle - 1
        count += left
        max_sum -= left * (low + low + left - 1) // 2
        if left < high - low + 1:
            break
    return count
```

PATTERN 33 · CYCLIC SORT · PROBLEM 20 OF 20

765. Couples Holding Hands

Hard · Greedy, Depth-First Search, Breadth-First Search, Union Find, Graph

leetcode.com/problems/couples-holding-hands

★ **TOP 7 PROBLEM** in Cyclic Sort: master this one first

PROBLEM

There are `n` couples sitting in `2n` seats arranged in a row and want to hold hands.

The people and seats are represented by an integer array `row` where `row[i]` is the ID of the person sitting in the i^{th} seat. The couples are numbered in order, the first couple being `(0, 1)`, the second couple being `(2, 3)`, and so on with the last couple being `(2n - 2, 2n - 1)`.

Return *the minimum number of swaps so that every couple is sitting side by side*. A swap consists of choosing any two people, then they stand up and switch seats.

Example 1:

```
Input: row = [0,2,1,3]
Output: 1
Explanation: We only need to swap the second (row[1]) and third (row[2]) person.
```

Example 2:

```
Input: row = [3,2,0,1]
Output: 0
Explanation: All couples are already seated side by side.
```

Constraints:

- `2n == row.length`
- `2 <= n <= 30`
- `0 <= row[i] < 2n`
- All the elements of `row` are **unique**.

SOLUTION (PYTHON 3)

Solution 1: Union-Find

How to think about it

Seat couples side by side with the fewest swaps. The count equals a permutation of couple-ids. Each sofa maps two people to couple numbers; those couples are unioned. A cycle of y couples needs y-1 swaps. Union-find on n couple ids; the answer is n minus the number of roots.

We can assign a number to each pair of couples. Person with number 0 and 1 corresponds to couple 0, person with number 2 and 3 corresponds to couple 1, and so on. In other words, the person corresponding to row[i] has a couple number of floor(row[i]/2).

If there are k pairs of couples who are seated incorrectly with respect to each other, i.e., if k pairs of couples are in the same permutation cycle, it will take k-1 swaps for all of them to be seated correctly.

Why? Consider the following: we first adjust the positions of a couple to their correct seats. After this, the problem reduces from k couples to k-1 couples. This process continues, and when k = 1, the number of swaps required is 0. Therefore, if k pairs of couples are in the wrong positions, we need k-1 swaps.

Thus, we only need to traverse the array once, use union-find to determine how many permutation cycles there are. Suppose there are x cycles, and the size of each cycle (in terms of couple pairs) is y_1, y_2, …, y_x. The number of swaps required is y_1-1 + y_2-1 + … + y_x-1 = y_1 + y_2 + … + y_x - x = n - x.

The time complexity is O(n × α(n)), and the space complexity is O(n), where α(n) is the inverse Ackermann function, which can be considered a very small constant.

Python 3

```
class Solution:
    def minSwapsCouples(self, row: List[int]) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(row) >> 1
        p = list(range(n))
        for i in range(0, len(row), 2):
            a, b = row[i] >> 1, row[i + 1] >> 1
            p[find(a)] = find(b)
        return n - sum(i == find(i) for i in range(n))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the swaps (which two seats are swapped)
# 2. Count the swaps with union-find on the couples (n
# minus the number of groups)


# ============================================================
# FOLLOW-UP 1: Return the swaps (which two seats are
# swapped)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Go through the pairs of seats; if a pair is not
# a couple, find the partner of the first person and swap
# it into the second seat.
# Time: O(n).  Space: O(n).
def couple_swaps(row):
    row = list(row)
    where = {}
    for i in range(len(row)):
        where[row[i]] = i
    swaps = []
    for i in range(0, len(row), 2):
        partner = row[i] ^ 1
        if row[i + 1] != partner:
            j = where[partner]
            where[row[i + 1]] = j
            where[partner] = i + 1
            row[i + 1], row[j] = row[j], row[i + 1]
            swaps.append((i + 1, j))
    return swaps


# ============================================================
# FOLLOW-UP 2: Count the swaps with union-find on the
# couples (n minus the number of groups)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Treat every pair of seats as a node and join the
# two couples sitting there; each group of size s needs s
# - 1 swaps.
# Time: O(n * alpha(n)).  Space: O(n).
def couple_swaps_union_find(row):
    n = len(row) // 2
    parent = list(range(n))
```

```
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for i in range(0, len(row), 2):
        a, b = row[i] // 2, row[i + 1] // 2
        parent[find(a)] = find(b)
    return n - len(set(find(x) for x in range(n)))
```

PATTERN 34 OF 41

Math / Number Theory

20 problems · 10 Easy · 10 Medium · 0 Hard

THE IDEA

Look for a formula, a modular identity, or a number-theory tool: gcd, primes (sieve), powers by squaring, digit tricks.

Signals: count primes, power(x, n), happy number, factorial trailing zeroes, ugly number, excel sheet column, fraction to decimal.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```python
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def power(x, n):                # fast exponent: O(log n)
    result = 1
    while n:
        if n & 1:
            result *= x
        x *= x
        n >>= 1
    return result
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Palindrome Number	Easy	LC 9	1793
2	Sqrt(x) (see page)	Easy	LC 69	190
3	Roman to Integer	Easy	LC 13	1796
4	Excel Sheet Column Number	Easy	LC 171	1799
5	Excel Sheet Column Title	Easy	LC 168	1801
6	Greatest Common Divisor of Strings	Easy	LC 1071	1803
7	Happy Number (see page)	Easy	LC 202	1273
8	Add Digits	Easy	LC 258	1806
9	Ugly Number	Easy	LC 263	1808
10	Self Dividing Numbers	Easy	LC 728	1810
11	★ **Reverse Integer**	Medium	LC 7	1812
12	★ **Pow(x, n)** (see page)	Medium	LC 50	732
13	★ **Integer to Roman**	Medium	LC 12	1815
14	★ **Count Primes**	Medium	LC 204	1818
15	★ **Factorial Trailing Zeroes**	Medium	LC 172	1821
16	★ **Fraction to Recurring Decimal**	Medium	LC 166	1824
17	★ **Ugly Number II**	Medium	LC 264	1827
18	Super Pow (see page)	Medium	LC 372	748
19	Water and Jug Problem	Medium	LC 365	1830
20	Nth Digit	Medium	LC 400	1833

PATTERN 34 · MATH / NUMBER THEORY · PROBLEM 1 OF 20

9. Palindrome Number

Easy · Math

leetcode.com/problems/palindrome-number

PROBLEM

Given an integer `x`, return `true` if `x` is a **palindrome**, and `false` otherwise.

Example 1:

```
Input: x = 121
Output: true
Explanation: 121 reads as 121 from left to right and from right to left.
```

Example 2:

```
Input: x = -121
Output: false
Explanation: From left to right, it reads -121. From right to left, it becomes 121-. Therefore it is
not a palindrome.
```

Example 3:

```
Input: x = 10
Output: false
Explanation: Reads 01 from right to left. Therefore it is not a palindrome.
```

Constraints:

- $-2^{31} <= x <= 2^{31} - 1$

Follow up:

Could you solve it without converting the integer to a string?

SOLUTION (PYTHON 3)

Solution 1: Reverse Half of the Number

How to think about it

Converting to a string and comparing with its reverse is the obvious idea; the follow-up asks us not to. Reversing the whole x and comparing would also overflow 32-bit integers. We only need half: keep appending x's last digit onto y until y is no longer smaller than the remaining prefix. A negative number cannot be a palindrome; a nonzero number ending in 0 would drop that zero when reversed, so those are false as well. The loop stops when the two halves have the same length. Even length compares x with y; odd length leaves the middle digit on y, so we compare x with y/10.

First, we determine special cases:

- If x < 0, then x is not a palindrome, directly return `false`;
- If x > 0 and the last digit of x is 0, then x is not a palindrome, directly return `false`;
- If the last digit of x is not 0, then x might be a palindrome, continue the following steps.

We reverse the second half of x and compare it with the first half. If they are equal, then x is a palindrome, otherwise, x is not a palindrome.

For example, for x = 1221, we can reverse the second half from "21" to "12" and compare it with the first half "12". Since they are equal, we know that x is a palindrome.

Let's see how to reverse the second half.

For the number 1221, if we perform 1221 mod 10, we will get the last digit 1. To get the second last digit, we can first remove the last digit from 1221 by dividing by 10, 1221 / 10 = 122, then get the remainder of the previous result divided by 10, 122 mod 10 = 2, to get the second last digit.

If we continue this process, we will get more reversed digits.

By continuously multiplying the last digit to the variable y, we can get the number in reverse order.

In the code implementation, we can repeatedly "take out" the last digit of x and "add" it to the end of y, loop until y ≥ x. If at this time x = y, or x = y / 10, then x is a palindrome.

The time complexity is $O(\log_{10}(n))$, where n is x. For each iteration, we will divide the input by 10, so the time complexity is $O(\log_{10}(n))$. The space complexity is O(1).

Python 3

```python
class Solution:
    def isPalindrome(self, x: int) -> bool:
        if x < 0 or (x and x % 10 == 0):
            return False
        y = 0
        while y < x:
            y = y * 10 + x % 10
            x //= 10
        return x in (y, y // 10)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Is the number a palindrome in base b
# 2. The smallest palindrome that is greater than or equal
# to x


# ============================================================
# FOLLOW-UP 1: Is the number a palindrome in base b
# ============================================================

# ---- follow-up 1: function ----
# Answer: Collect the digits of the number in base b and
# compare them with their reverse.
# Time: O(log x).  Space: O(log x).
def is_palindrome_in_base(x, base):
    if x < 0:
        return False
    digits = []
    while x > 0:
        digits.append(x % base)
        x //= base
    return digits == digits[::-1]


# ============================================================
# FOLLOW-UP 2: The smallest palindrome that is greater
# than or equal to x
# ============================================================

# ---- follow-up 2: function ----
# Answer: Try x, then x + 1, ... (palindromes are dense
# enough for this to be short for normal sizes).
# Time: O(g * log x), g = distance to the next palindrome.
# Space: O(log x).
def next_palindrome(x):
    while True:
        text = str(x)
        if text == text[::-1]:
            return x
```

```
        x += 1
```

13. Roman to Integer

Easy · Hash Table, Math, String

leetcode.com/problems/roman-to-integer

PROBLEM

Roman numerals are represented by seven different symbols: `I`, `V`, `X`, `L`, `C`, `D` and `M`.

```
Symbol Value
I 1
V 5
X 10
L 50
C 100
D 500
M 1000
```

For example, `2` is written as `II` in Roman numeral, just two ones added together. `12` is written as `XII`, which is simply `X + II`. The number `27` is written as `XXVII`, which is `XX + V + II`.

Roman numerals are usually written largest to smallest from left to right. However, the numeral for four is not `IIII`. Instead, the number four is written as `IV`. Because the one is before the five we subtract it making four. The same principle applies to the number nine, which is written as `IX`. There are six instances where subtraction is used:

- `I` can be placed before `V` (5) and `X` (10) to make 4 and 9.
- `X` can be placed before `L` (50) and `C` (100) to make 40 and 90.
- `C` can be placed before `D` (500) and `M` (1000) to make 400 and 900.

Given a roman numeral, convert it to an integer.

Example 1:

```
Input: s = "III"
Output: 3
Explanation: III = 3.
```

Example 2:

```
Input: s = "LVIII"
Output: 58
Explanation: L = 50, V= 5, III = 3.
```

Example 3:

```
Input: s = "MCMXCIV"
Output: 1994
Explanation: M = 1000, CM = 900, XC = 90 and IV = 4.
```

Constraints:

- `1 <= s.length <= 15`
- `s` contains only the characters `('I', 'V', 'X', 'L', 'C', 'D', 'M')`.
- It is **guaranteed** that `s` is a valid roman numeral in the range `[1, 3999]`.

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Simulation

How to think about it

The first idea is a scan with special cases for the six subtractive pairs (IV, IX, …) and addition elsewhere. |s|≤ 15, so it would pass, but the exceptions are easy to miss. The annoyance is deciding whether the current character is added or subtracted. A character is subtracted if and only if it is smaller than the next one; the last character is always added. We do not need to list the six pairs. So we store each character's value in a hash table, compare neighbors to choose the sign, and add the last character.

First, we use a hash table d to record the numerical value corresponding to each character. Then, we traverse the string s from left to right. If the numerical value corresponding to the current character is less than the numerical value corresponding to the character on the right, we subtract the numerical value corresponding to the current character. Otherwise, we add the numerical value corresponding to the current character.

The time complexity is O(n), and the space complexity is O(m). Here, n and m are the length of the string s and the size of the character set, respectively.

Python 3

```
class Solution:
    def romanToInt(self, s: str) -> int:
        d = {'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}
        return sum((-1 if d[a] < d[b] else 1) * d[a] for a, b in pairwise(s)) + d[s[-1]]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Check that a string is a valid roman numeral
# 2. Add two roman numerals and answer in roman


# ============================================================
# FOLLOW-UP 1: Check that a string is a valid roman
# numeral
# ============================================================

# ---- follow-up 1: function ----
# Answer: A valid numeral matches the thousands, hundreds,
# tens and ones patterns, in that order.
# Time: O(n) (the pattern is bounded).  Space: O(1).
import re


def is_valid_roman(s):
    pattern = "^M{0,3}(CM|CD|D?C{0,3})(XC|XL|L?X{0,3})(IX|IV|V?I{0,3})$"
    return len(s) > 0 and re.match(pattern, s) is not None


# ============================================================
# FOLLOW-UP 2: Add two roman numerals and answer in roman
# ============================================================

# ---- follow-up 2: function ----
# Answer: Convert both to numbers, add, and convert back.
# Time: O(n + 13 + result length).  Space: O(1).
def roman_to_number(s):
    value = {"I": 1, "V": 5, "X": 10, "L": 50, "C": 100, "D": 500, "M": 1000}
    total = 0
    for i in range(len(s)):
        if i + 1 < len(s) and value[s[i]] < value[s[i + 1]]:
            total -= value[s[i]]
```

```
        else:
            total += value[s[i]]
    return total


def number_to_roman(num):
    table = [(1000, "M"), (900, "CM"), (500, "D"), (400, "CD"), (100, "C"), (90, "XC"),
             (50, "L"), (40, "XL"), (10, "X"), (9, "IX"), (5, "V"), (4, "IV"), (1, "I")]
    result = ""
    for value, symbol in table:
        while num >= value:
            result += symbol
            num -= value
    return result


def add_romans(a, b):
    return number_to_roman(roman_to_number(a) + roman_to_number(b))
```

171. Excel Sheet Column Number

Easy · Math, String

leetcode.com/problems/excel-sheet-column-number

PROBLEM

Given a string `columnTitle` that represents the column title as appears in an Excel sheet, return *its corresponding column number*.

For example:

```
A -> 1
B -> 2
C -> 3
...
Z -> 26
AA -> 27
AB -> 28
...
```

Example 1:

```
Input: columnTitle = "A"
Output: 1
```

Example 2:

```
Input: columnTitle = "AB"
Output: 28
```

Example 3:

```
Input: columnTitle = "ZY"
Output: 701
```

Constraints:

- `1 <= columnTitle.length <= 7`
- `columnTitle` consists only of uppercase English letters.
- `columnTitle` is in the range `["A", "FXSHRXW"]`.

SOLUTION (PYTHON 3)

Solution 1: Base Conversion

> **How to think about it**
>
> Title to number is the inverse of problem 168: each letter is a digit in 1… 26. Fold left to right with ans=ans× 26 + (c-'A'+1), the same pattern as parsing a decimal string.

The column name in Excel is a representation in base 26. For example, "AB" represents the column number 1 × 26 + 2 = 28.

Therefore, we can iterate through the string `columnTitle`, convert each character to its corresponding value, and then calculate the result.

The time complexity is O(n), where n is the length of the string `columnTitle`. The space complexity is O(1).

Python 3

```
class Solution:
    def titleToNumber(self, columnTitle: str) -> int:
        ans = 0
        for c in map(ord, columnTitle):
            ans = ans * 26 + c - ord("A") + 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Add two column titles and answer as a title
# 2. List all column titles from one title to another


# ============================================================
# FOLLOW-UP 1: Add two column titles and answer as a title
# ============================================================

# ---- follow-up 1: function ----
# Answer: Convert both to numbers, add, and convert back
# to letters.
# Time: O(n + log value).  Space: O(log value).
def title_to_number(title):
    number = 0
    for ch in title:
        number = number * 26 + (ord(ch) - ord("A") + 1)
    return number


def number_to_title(number):
    letters = []
    while number > 0:
        number -= 1
        letters.append(chr(ord("A") + number % 26))
        number //= 26
    return "".join(reversed(letters))


def add_columns(a, b):
    return number_to_title(title_to_number(a) + title_to_number(b))


# ============================================================
# FOLLOW-UP 2: List all column titles from one title to
# another
# ============================================================

# ---- follow-up 2: function ----
# Answer: Convert both ends to numbers and convert every
# number in between.
# Time: O(k * L), k = columns in the range, L = title
# length.  Space: O(k * L).
def columns_between(first, last):
    return [number_to_title(x) for x in range(title_to_number(first), title_to_number(last) + 1)]
```

PATTERN 34 · MATH / NUMBER THEORY · PROBLEM 5 OF 20

168. Excel Sheet Column Title

Easy · Math, String

leetcode.com/problems/excel-sheet-column-title

PROBLEM

Given an integer `columnNumber`, return *its corresponding column title as it appears in an Excel sheet*.

For example:

```
A -> 1
B -> 2
C -> 3
...
Z -> 26
AA -> 27
AB -> 28
...
```

Example 1:

```
Input: columnNumber = 1
Output: "A"
```

Example 2:

```
Input: columnNumber = 28
Output: "AB"
```

Example 3:

```
Input: columnNumber = 701
Output: "ZY"
```

Constraints:

- `1 <= columnNumber <= 2^31 - 1`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Map a positive integer to an Excel column title: base 26 with A… Z as 1… 26, and no zero. Ordinary base 26 would emit 0 instead of Z on a zero remainder. Subtract 1 before mod 26, map the remainder to A– Z, continue with the quotient, and reverse.

Python 3

```python
class Solution:
    def convertToTitle(self, columnNumber: int) -> str:
        res = []
        while columnNumber:
            columnNumber -= 1
            res.append(chr(ord('A') + columnNumber % 26))
            columnNumber //= 26
        return ''.join(res[::-1])
```

Complexity. Time: O(log n). Space: O(log n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Use a different alphabet size (for example only the
# letters A to E)
# 2. Zero based columns (0 is A, 25 is Z, 26 is AA)


# ============================================================
# FOLLOW-UP 1: Use a different alphabet size (for example
# only the letters A to E)
# ============================================================

# ---- follow-up 1: function ----
# Answer: It is the same bijective numbering with the base
# changed from 26 to the alphabet size.
# Time: O(log n).  Space: O(log n).
def title_in_alphabet(number, alphabet):
    base = len(alphabet)
    letters = []
    while number > 0:
        number -= 1
        letters.append(alphabet[number % base])
        number //= base
    return "".join(reversed(letters))


# ============================================================
# FOLLOW-UP 2: Zero based columns (0 is A, 25 is Z, 26 is
# AA)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Add one and use the usual conversion.
# Time: O(log n).  Space: O(log n).
def zero_based_title(index):
    number = index + 1
    letters = []
    while number > 0:
        number -= 1
        letters.append(chr(ord("A") + number % 26))
        number //= 26
    return "".join(reversed(letters))
```

1071. Greatest Common Divisor of Strings

Easy · Math, String, Greatest Common Divisor, Euclidean Algorithm

leetcode.com/problems/greatest-common-divisor-of-strings

PROBLEM

For two strings `s` and `t`, we say "`t` divides `s`" if and only if `s = t + t + t + ... + t + t` (i.e., `t` is concatenated with itself one or more times).

Given two strings `str1` and `str2`, return *the largest string* `x` *such that* `x` *divides both* `str1` *and* `str2`.

Example 1:

Input: str1 = "ABCABC", str2 = "ABC"

Output: "ABC"

Example 2:

Input: str1 = "ABABAB", str2 = "ABAB"

Output: "AB"

Example 3:

Input: str1 = "LEET", str2 = "CODE"

Output: ""

Example 4:

Input: str1 = "AAAAAB", str2 = "AAA"

Output: ""

Constraints:

- `1 <= str1.length, str2.length <= 1000`
- `str1` and `str2` consist of English uppercase letters.

SOLUTIONS (PYTHON 3)

Solution 1: Enumeration

How to think about it

A common divisor string must tile both inputs, so its length divides both lengths. Trying prefixes from the shorter length downward is enough for m,n≤ 1000. Each candidate t=str1[:i] is repeated until it matches the target length and compared. The first t that tiles both strings is the longest; otherwise the answer is empty.

Enumerate candidate prefixes t from the shorter string length downward, and check whether repeating t can produce str1 and str2. The first valid t is the longest gcd string.

The time complexity is O((m + n) × min(m, n)), and the space complexity is O(m + n), where m and n are the lengths of the two strings.

Python 3

```python
class Solution:
    def gcdOfStrings(self, str1: str, str2: str) -> str:
        def check(a, b):
            c = ""
            while len(c) < len(b):
                c += a
            return c == b

        for i in range(min(len(str1), len(str2)), 0, -1):
            t = str1[:i]
            if check(t, str1) and check(t, str2):
                return t
        return ''
```

Solution 2: Math

> **How to think about it**
>
> Enumeration still tries many lengths. If a common divisor exists then $s_1+s_2=s_2+s_1$, and the longest length is $\gcd(|s_1|,|s_2|)$. We reject unequal concatenations and otherwise return the prefix of s_1 of that gcd length.

If a gcd string exists, then $s_1+s_2=s_2+s_1$. In that case the length of the longest gcd string is $\gcd(|s_1|,|s_2|)$.

The time complexity is O(m + n), and the space complexity is O(m + n), where m and n are the lengths of the two strings.

Python 3

```python
class Solution:
    def gcdOfStrings(self, str1: str, str2: str) -> str:
        if str1 + str2 != str2 + str1:
            return ''
        n = gcd(len(str1), len(str2))
        return str1[:n]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The greatest common divisor string of a whole list
# 2. How many different strings divide both strings


# ============================================================
# FOLLOW-UP 1: The greatest common divisor string of a
# whole list
# ============================================================

# ---- follow-up 1: function ----
# Answer: Reduce the list pair by pair with the two-string
# answer.
# Time: O(k * L), L = longest string.  Space: O(L).
def gcd_of_strings_list(strs):
    def gcd(a, b):
        while b:
            a, b = b, a % b
        return a

    def pair(s, t):
        if s + t != t + s:
            return ""
        return s[:gcd(len(s), len(t))]

    answer = strs[0]
    for s in strs[1:]:
        answer = pair(answer, s)
```

```
        if answer == "":
            return ""
    return answer


# ============================================================
# FOLLOW-UP 2: How many different strings divide both
# strings
# ============================================================

# ---- follow-up 2: function ----
# Answer: If the strings commute, the biggest divisor is
# the prefix of length gcd; a shorter divisor must repeat
# to form exactly that prefix, and its length must divide
# the gcd.
# Time: O(g + D * g), D = divisors of g.  Space: O(g).
def count_common_divisor_strings(str1, str2):
    def gcd(a, b):
        while b:
            a, b = b, a % b
        return a

    if str1 + str2 != str2 + str1:
        return 0
    g = gcd(len(str1), len(str2))
    biggest = str1[:g]
    count = 0
    for d in range(1, g + 1):
        if g % d == 0 and biggest[:d] * (g // d) == biggest:
            count += 1
    return count
```

PATTERN 34 · MATH / NUMBER THEORY · PROBLEM 8 OF 20

258. Add Digits

Easy · Math, Number Theory, Simulation

leetcode.com/problems/add-digits

PROBLEM

Given an integer `num`, repeatedly add all its digits until the result has only one digit, and return it.

Example 1:

```
Input: num = 38
Output: 2
Explanation: The process is
38 --> 3 + 8 --> 11
11 --> 1 + 1 --> 2
Since 2 has only one digit, return it.
```

Example 2:

```
Input: num = 0
Output: 0
```

Constraints:

- `0 <= num <= 2^31 - 1`

Follow up: Could you do it without any loop/recursion in `O(1)` runtime?

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Repeated digit sums yield the digital root. For a non-negative integer it is 0 when num=0, and (num-1) mod 9+1 otherwise.

Python 3

```python
class Solution:
    def addDigits(self, num: int) -> int:
        return 0 if num == 0 else (num - 1) % 9 + 1
```

Complexity. Time: O(1). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The number is given as a very long string
# 2. How many steps does it take to reach one digit


# ============================================================
# FOLLOW-UP 1: The number is given as a very long string
# ============================================================

# ---- follow-up 1: function ----
# Answer: The digital root only depends on the number
# modulo 9; add the digits (mod 9) one by one.
# Time: O(n).  Space: O(1).
def digital_root_of_text(text):
```

```
    total = 0
    for ch in text:
        total += int(ch)
    if total == 0:
        return 0
    if total % 9 == 0:
        return 9
    return total % 9


# ============================================================
# FOLLOW-UP 2: How many steps does it take to reach one
# digit
# ============================================================

# ---- follow-up 2: function ----
# Answer: Repeat the digit sum and count the rounds.
# Time: O(log num) per step, and the number shrinks fast.
# Space: O(1).
def steps_to_single_digit(num):
    steps = 0
    while num >= 10:
        total = 0
        while num > 0:
            total += num % 10
            num //= 10
        num = total
        steps += 1
    return steps
```

PATTERN 34 · MATH / NUMBER THEORY · PROBLEM 9 OF 20

263. Ugly Number

Easy · Math

leetcode.com/problems/ugly-number

PROBLEM

An **ugly number** is a *positive* integer which does not have a prime factor other than 2, 3, and 5.

Given an integer `n`, return `true` *if* `n` *is an* ***ugly number***.

Example 1:

```
Input: n = 6
Output: true
Explanation: 6 = 2 × 3
```

Example 2:

```
Input: n = 1
Output: true
Explanation: 1 has no prime factors.
```

Example 3:

```
Input: n = 14
Output: false
Explanation: 14 is not ugly since it includes the prime factor 7.
```

Constraints:

- $-2^{31} <= n <= 2^{31} - 1$

SOLUTION (PYTHON 3)

Solution 1

How to think about it

An ugly number's prime factors are only 2,3,5. Reject non-positives; otherwise divide out those three primes and test whether 1 remains.

Python 3

```python
class Solution:
    def isUgly(self, n: int) -> bool:
        if n < 1:
            return False
        for x in [2, 3, 5]:
            while n % x == 0:
                n //= x
        return n == 1
```

Complexity. Time: O(log n). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Ugly with a custom list of prime factors
# 2. Return how many times 2, 3 and 5 divide the number


# ============================================================
# FOLLOW-UP 1: Ugly with a custom list of prime factors
# ============================================================

# ---- follow-up 1: function ----
# Answer: Divide by each allowed prime as often as
# possible; ugly means 1 is left.
# Time: O(p + log n).  Space: O(1).
def is_ugly_with_primes(n, primes):
    if n <= 0:
        return False
    for p in primes:
        while n % p == 0:
            n //= p
    return n == 1


# ============================================================
# FOLLOW-UP 2: Return how many times 2, 3 and 5 divide the
# number
# ============================================================

# ---- follow-up 2: function ----
# Answer: Divide out each prime and count; the leftover
# must be 1.
# Time: O(log n).  Space: O(1).
def ugly_exponents(n):
    if n <= 0:
        return None
    exponents = []
    for p in (2, 3, 5):
        count = 0
        while n % p == 0:
            n //= p
            count += 1
        exponents.append(count)
    if n != 1:
        return None
    return exponents
```

728. Self Dividing Numbers

Easy · Math

leetcode.com/problems/self-dividing-numbers

PROBLEM

A **self-dividing number** is a number that is divisible by every digit it contains.

- For example, `128` is **a self-dividing number** because `128 % 1 == 0`, `128 % 2 == 0`, and `128 % 8 == 0`.

A **self-dividing number** is not allowed to contain the digit zero.

Given two integers `left` and `right`, return *a list of all the* ***self-dividing numbers*** *in the range* `[left, right]` (both **inclusive**).

Example 1:

```
Input: left = 1, right = 22
Output: [1,2,3,4,5,6,7,8,9,11,12,15,22]
```

Example 2:

```
Input: left = 47, right = 85
Output: [48,55,66,77]
```

Constraints:

- `1 <= left <= right <= 10^4`

SOLUTION (PYTHON 3)

Solution 1: Simulation

> **How to think about it**
>
> The range is at most 10^4, and testing a number against its own digits is a few divisions. Enumerating the interval is fine. A zero digit cannot divide, so those numbers fail immediately; otherwise x must be divisible by each digit. Walk digits of y=x; reject on a 0 or a failed modulo, otherwise keep x.

We define a function check(x) to determine whether x is a self-dividing number. The implementation idea of the function is as follows:

We use y to record the value of x, and then continuously divide y by 10 until y is 0. During this process, we check whether the last digit of y is 0, or whether x cannot be divided by the last digit of y. If either of these conditions is met, then x is not a self-dividing number, and we return false. Otherwise, after traversing all the digits, we return true.

Finally, we traverse all the numbers in the interval [left, right], and for each number, we call check(x). If it returns true, we add this number to the answer array.

The time complexity is $O(n \times \log_{10} M)$, where n is the number of elements in the interval [left, right], and M = right, which is the maximum value in the interval.

Python 3

```python
class Solution:
    def selfDividingNumbers(self, left: int, right: int) -> List[int]:
        def check(x: int) -> bool:
            y = x
            while y:
                if y % 10 == 0 or x % (y % 10):
                    return False
                y //= 10
            return True

        return [x for x in range(left, right + 1) if check(x)]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count the self dividing numbers in a range
# 2. Numbers divisible by the SUM of their digits (Harshad
# numbers) in a range


# ========================================================
# FOLLOW-UP 1: Count the self dividing numbers in a range
# ========================================================

# ---- follow-up 1: function ----
# Answer: Test each number in the range.
# Time: O((right - left) * log right).  Space: O(1).
def count_self_dividing(left, right):
    count = 0
    for x in range(left, right + 1):
        y = x
        good = True
        while y > 0:
            digit = y % 10
            if digit == 0 or x % digit != 0:
                good = False
                break
            y //= 10
        if good:
            count += 1
    return count


# ========================================================
# FOLLOW-UP 2: Numbers divisible by the SUM of their
# digits (Harshad numbers) in a range
# ========================================================

# ---- follow-up 2: function ----
# Answer: Compute the digit sum and test divisibility.
# Time: O((right - left) * log right).  Space: O(k) for
# the answer.
def harshad_numbers(left, right):
    result = []
    for x in range(left, right + 1):
        total = 0
        y = x
        while y > 0:
            total += y % 10
            y //= 10
        if total > 0 and x % total == 0:
            result.append(x)
    return result
```

7. Reverse Integer

Medium · Math

leetcode.com/problems/reverse-integer

★ **TOP 7 PROBLEM** in Math / Number Theory: master this one first

PROBLEM

Given a signed 32-bit integer `x`, return `x` *with its digits reversed*. If reversing `x` causes the value to go outside the signed 32-bit integer range `[-2^31, 2^31 - 1]`, then return `0`.

Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

Example 1:

```
Input: x = 123
Output: 321
```

Example 2:

```
Input: x = -123
Output: -321
```

Example 3:

```
Input: x = 120
Output: 21
```

Constraints:

- `-2^31 <= x <= 2^31 - 1`

SOLUTION (PYTHON 3)

Solution 1: Mathematics

How to think about it

Reversing via a string is the obvious idea, but the environment is assumed not to hold 64-bit integers. We cannot compute in a wider type and check overflow afterwards. Peeling the last digit and appending it is ans ← 10·ans+y. Before multiplying by 10, we must know whether the next value would leave [-2^{31}, 2^{31}-1]; if it would, return 0. That check collapses to whether ans still lies in [floor(mi/10), floor(mx/10)]. Inside the range, one more digit is safe; outside, it has already overflowed.

Let's denote mi and mx as -2^{31} and $2^{31} - 1$ respectively, then the reverse result of x, ans, needs to satisfy mi ≤ ans ≤ mx.

We can continuously take the remainder of x to get the last digit y of x, and add y to the end of ans. Before adding y, we need to check if ans overflows. That is, check whether ans × 10 + y is within the range [mi, mx].

If x > 0, it needs to satisfy ans × 10 + y ≤ mx, that is, ans × 10 + y ≤ floor(mx/10) × 10 + 7. Rearranging gives (ans - floor(mx/10)) × 10 ≤ 7 - y.

Next, we discuss the conditions for the inequality to hold:

- When ans < floor(mx/10), the inequality obviously holds;
- When ans = floor(mx/10), the necessary and sufficient condition for the inequality to hold is y ≤ 7. If ans = floor(mx/10) and we can still add numbers, it means that the number is at the highest digit, that is, y must not exceed 2, therefore, the inequality must hold;
- When ans > floor(mx/10), the inequality obviously does not hold.

In summary, when x > 0, the necessary and sufficient condition for the inequality to hold is ans ≤ floor(mx/10).

Similarly, when x < 0, the necessary and sufficient condition for the inequality to hold is ans ≥ floor(mi/10).

Therefore, we can check whether ans overflows by checking whether ans is within the range [floor(mi/10), floor(mx/10)]. If it overflows, return 0. Otherwise, add y to the end of ans, and then remove the last digit of x, that is, x ← floor(x/10).

The time complexity is O(log |x|), where |x| is the absolute value of x. The space complexity is O(1).

Python 3

```
class Solution:
    def reverse(self, x: int) -> int:
        ans = 0
        mi, mx = -(2**31), 2**31 - 1
        while x:
            if ans < mi // 10 + 1 or ans > mx // 10:
                return 0
            y = x % 10
            if x < 0 and y > 0:
                y -= 10
            ans = ans * 10 + y
            x = (x - y) // 10
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Reverse a number given as a string (keep the leading
# zeros that appear after reversing)
# 2. Use a different word size (for example 16 bits)


# ============================================================
# FOLLOW-UP 1: Reverse a number given as a string (keep
# the leading zeros that appear after reversing)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Reverse the digits as text; keep the sign in
# front.
# Time: O(n).  Space: O(n).
def reverse_text_number(text):
    sign = ""
    if text[0] == "-":
        sign = "-"
        text = text[1:]
    return sign + text[::-1]


# ============================================================
# FOLLOW-UP 2: Use a different word size (for example 16
# bits)
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same reversal, but compare with the limits
# of the given number of bits.
# Time: O(log x).  Space: O(1).
def reverse_with_bits(x, bits):
    low = -(2 ** (bits - 1))
    high = 2 ** (bits - 1) - 1
    sign = -1 if x < 0 else 1
    x = abs(x)
    reversed_number = 0
    while x > 0:
        reversed_number = reversed_number * 10 + x % 10
        x //= 10
    reversed_number *= sign
    if reversed_number < low or reversed_number > high:
```

```
        return 0
    return reversed_number
```

12. Integer to Roman

Medium · Hash Table, Math, String

leetcode.com/problems/integer-to-roman

★ **TOP 7 PROBLEM** in Math / Number Theory: master this one first

PROBLEM

Seven different symbols represent Roman numerals with the following values:

Symbol	Value
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

Roman numerals are formed by appending the conversions of decimal place values from highest to lowest. Converting a decimal place value into a Roman numeral has the following rules:

- If the value does not start with 4 or 9, select the symbol of the maximal value that can be subtracted from the input, append that symbol to the result, subtract its value, and convert the remainder to a Roman numeral.
- If the value starts with 4 or 9 use the **subtractive form** representing one symbol subtracted from the following symbol, for example, 4 is 1 (`I`) less than 5 (`V`): `IV` and 9 is 1 (`I`) less than 10 (`X`): `IX`. Only the following subtractive forms are used: 4 (`IV`), 9 (`IX`), 40 (`XL`), 90 (`XC`), 400 (`CD`) and 900 (`CM`).
- Only powers of 10 (`I`, `X`, `C`, `M`) can be appended consecutively at most 3 times to represent multiples of 10. You cannot append 5 (`V`), 50 (`L`), or 500 (`D`) multiple times. If you need to append a symbol 4 times use the **subtractive form**.

Given an integer, convert it to a Roman numeral.

Example 1:

Input: num = 3749

Output: "MMMDCCXLIX"

Explanation:

```
3000 = MMM as 1000 (M) + 1000 (M) + 1000 (M)
 700 = DCC as 500 (D) + 100 (C) + 100 (C)
  40 = XL as 10 (X) less of 50 (L)
   9 = IX as 1 (I) less of 10 (X)
Note: 49 is not 1 (I) less of 50 (L) because the conversion is based on decimal places
```

Example 2:

Input: num = 58

Output: "LVIII"

Explanation:

```
50 = L
 8 = VIII
```

Example 3:

Input: num = 1994

Output: "MCMXCIV"

Explanation:

```
1000 = M
 900 = CM
  90 = XC
   4 = IV
```

Constraints:

- `1 <= num <= 3999`

SOLUTION (PYTHON 3)

Solution 1: Greedy

How to think about it

The first idea is a per-place lookup (thousands, hundreds, tens, ones), with extra branches for 4 and 9. 1 ≤ num ≤ 3999, so it would pass, but the special cases are easy to miss. The mess is keeping “additive” and “subtractive” rules apart. If we treat CM=900, CD=400, XC=90, and so on as first-class denominations, Roman numerals become a fixed list of values from large to small. The representation in this range is unique, so taking the largest denomination that still fits is always legal. We scan that table greedily, subtracting and appending until num becomes 0.

We can first list all possible symbols cs and their corresponding values vs, then enumerate each value vs[i] from large to small. Each time, we use as many symbols cs[i] corresponding to this value as possible, until the number num becomes 0.

The time complexity is O(m), and the space complexity is O(m). Here, m is the number of symbols.

Python 3

```python
class Solution:
    def intToRoman(self, num: int) -> str:
        cs = ('M', 'CM', 'D', 'CD', 'C', 'XC', 'L', 'XL', 'X', 'IX', 'V', 'IV', 'I')
        vs = (1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1)
        ans = []
        for c, v in zip(cs, vs):
            while num >= v:
                num -= v
                ans.append(c)
        return ''.join(ans)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Old style numerals without subtraction (4 is IIII, 9
# is VIIII)
# 2. Numbers of 4000 and more: write the thousands part in
# brackets


# ============================================================
# FOLLOW-UP 1: Old style numerals without subtraction (4
# is IIII, 9 is VIIII)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Drop the subtractive pairs from the table.
# Time: O(1) (a fixed number of symbols).  Space: O(1).
def to_additive_roman(num):
    table = [(1000, "M"), (500, "D"), (100, "C"), (50, "L"), (10, "X"), (5, "V"), (1, "I")]
    result = ""
    for value, symbol in table:
        while num >= value:
```

```
            result += symbol
            num -= value
    return result


# ============================================================
# FOLLOW-UP 2: Numbers of 4000 and more: write the
# thousands part in brackets
# ============================================================

# ---- follow-up 2: function ----
# Answer: Write num // 1000 as a numeral in brackets (a
# thousand times bigger), then the remainder.
# Time: O(1).  Space: O(1).
def to_roman_large(num):
    table = [(1000, "M"), (900, "CM"), (500, "D"), (400, "CD"), (100, "C"), (90, "XC"),
             (50, "L"), (40, "XL"), (10, "X"), (9, "IX"), (5, "V"), (4, "IV"), (1, "I")]

    def small(n):
        result = ""
        for value, symbol in table:
            while n >= value:
                result += symbol
                n -= value
        return result

    if num < 4000:
        return small(num)
    return "(" + small(num // 1000) + ")" + small(num % 1000)
```

PATTERN 34 · MATH / NUMBER THEORY · PROBLEM 14 OF 20

204. Count Primes

Medium · Array, Math, Enumeration, Number Theory, Primality Test, Sieve, Sieve of Eratosthenes

leetcode.com/problems/count-primes

★ **TOP 7 PROBLEM** in Math / Number Theory: master this one first

PROBLEM

Given an integer `n`, return *the number of prime numbers that are strictly less than* `n`.

Example 1:

```
Input: n = 10
Output: 4
Explanation: There are 4 prime numbers less than 10, they are 2, 3, 5, 7.
```

Example 2:

```
Input: n = 0
Output: 0
```

Example 3:

```
Input: n = 1
Output: 0
```

Constraints:

- `0 <= n <= 5 * 10^6`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Trial division up to √x counts primes, but n can be 5× 10^6, so repeated tests are slow. If x is prime, its multiples 2x,3x,… are composite. The Sieve of Eratosthenes marks those multiples as we scan upward, then counts unmarked values. Each composite is crossed off by a factor, in about O(nloglog n) time.

Python 3

```python
class Solution:
    def countPrimes(self, n: int) -> int:
        primes = [True] * n
        ans = 0
        for i in range(2, n):
            if primes[i]:
                ans += 1
                for j in range(i + i, n, i):
                    primes[j] = False
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Primes between a and b (b is large, a to b is short)
# 2. Factorise many numbers fast (smallest prime factor
# table)


# ============================================================
# FOLLOW-UP 1: Primes between a and b (b is large, a to b
# is short)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Segmented sieve: find primes up to sqrt(b), then
# cross out their multiples inside [a, b].
# Time: O(sqrt(b) log log b + (b - a) log log b).  Space:
# O(sqrt(b) + (b - a)).
def primes_in_range(a, b):
    if b < 2:
        return []
    limit = int(b ** 0.5) + 1
    small = [True] * (limit + 1)
    small[0] = small[1] = False
    for i in range(2, limit + 1):
        if small[i]:
            for j in range(i * i, limit + 1, i):
                small[j] = False
    is_prime = [True] * (b - a + 1)
    for p in range(2, limit + 1):
        if small[p]:
            start = max(p * p, ((a + p - 1) // p) * p)
            for multiple in range(start, b + 1, p):
                is_prime[multiple - a] = False
    result = []
    for x in range(max(a, 2), b + 1):
        if is_prime[x - a]:
            result.append(x)
    return result


# ============================================================
# FOLLOW-UP 2: Factorise many numbers fast (smallest prime
# factor table)
# ============================================================

# ---- follow-up 2: function ----
# Answer: A sieve that stores the smallest prime factor of
# every number; factorising is repeated division by it.
# Time: build O(n log log n), factorise O(log x).  Space:
# O(n).
def build_smallest_factor(limit):
    smallest = list(range(limit + 1))
    for i in range(2, int(limit ** 0.5) + 1):
        if smallest[i] == i:
            for j in range(i * i, limit + 1, i):
                if smallest[j] == j:
                    smallest[j] = i
    return smallest


def factorise(smallest, x):
```

```
    factors = []
    while x > 1:
        factors.append(smallest[x])
        x //= smallest[x]
```

```
    return factors
```

172. Factorial Trailing Zeroes

Medium · Math

leetcode.com/problems/factorial-trailing-zeroes

★ **TOP 7 PROBLEM** in Math / Number Theory: master this one first

PROBLEM

Given an integer `n`, return *the number of trailing zeroes in* `n!`.

Note that `n! = n * (n - 1) * (n - 2) * ... * 3 * 2 * 1`.

Example 1:

```
Input: n = 3
Output: 0
Explanation: 3! = 6, no trailing zero.
```

Example 2:

```
Input: n = 5
Output: 1
Explanation: 5! = 120, one trailing zero.
```

Example 3:

```
Input: n = 0
Output: 0
```

Constraints:

- `0 <= n <= 10^4`

Follow up: Could you write a solution that works in logarithmic time complexity?

SOLUTION (PYTHON 3)

Solution 1: Mathematics

How to think about it

Trailing zeros of n! come from factors 10=2× 5. There are always more twos than fives, so the count is the number of factors 5 in [1,n]. Computing n! overflows even at n≤ 10^4. The follow-up wants logarithmic time: repeatedly replace n by floor(n/5) to add the contributions of 5,5^2,5^3,….

The problem is actually asking how many factors of 5 are there in [1,n].

Let's take 130 as an example for analysis:

1. Divide by 5 for the first time, get 26, indicating that there are 26 numbers containing the factor 5;
2. Divide by 5 for the second time, get 5, indicating that there are 5 numbers containing the factor 5^2;
3. Divide by 5 for the third time, get 1, indicating that there is 1 number containing the factor 5^3;
4. Sum up to get the count of all factors of 5 in [1,n].

The time complexity is O(log n), and the space complexity is O(1).

Python 3

```python
class Solution:
    def trailingZeroes(self, n: int) -> int:
        ans = 0
        while n:
            n //= 5
            ans += n
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Smallest n such that n! has exactly k trailing zeros
# (or none)
# 2. Trailing zeros of n! in another base b


# ============================================================
# FOLLOW-UP 1: Smallest n such that n! has exactly k
# trailing zeros (or none)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Trailing zeros never decrease with n, so binary
# search for the first n that has at least k and check it
# has exactly k.
# Time: O(log^2 k).  Space: O(1).
def smallest_factorial_with_zeros(k):
    def zeros(n):
        total = 0
        while n > 0:
            n //= 5
            total += n
        return total

    low, high = 0, 5 * (k + 1)
    while low < high:
        middle = (low + high) // 2
        if zeros(middle) >= k:
            high = middle
        else:
            low = middle + 1
    if zeros(low) == k:
        return low
    return -1


# ============================================================
# FOLLOW-UP 2: Trailing zeros of n! in another base b
# ============================================================

# ---- follow-up 2: function ----
# Answer: Factor b into primes; for each prime p^e count
# how often p divides n! and divide by e; the smallest
# value wins.
# Time: O(sqrt(b) + log n).  Space: O(1).
def trailing_zeros_in_base(n, base):
    best = None
    p = 2
    while p * p <= base:
        if base % p == 0:
            e = 0
            while base % p == 0:
                base //= p
                e += 1
            count = 0
```

```
            m = n
            while m > 0:
                m //= p
                count += m
            if best is None or count // e < best:
                best = count // e
        p += 1
    if base > 1:
        count = 0
        m = n
        while m > 0:
            m //= base
            count += m
        if best is None or count < best:
            best = count
    return best
```

166. Fraction to Recurring Decimal

Medium · Hash Table, Math, String

leetcode.com/problems/fraction-to-recurring-decimal

★ **TOP 7 PROBLEM** in Math / Number Theory: master this one first

PROBLEM

Given two integers representing the `numerator` and `denominator` of a fraction, return *the fraction in string format*.

If the fractional part is repeating, enclose the repeating part in parentheses

If multiple answers are possible, return **any of them**.

It is **guaranteed** that the length of the answer string is less than 10^4 for all the given inputs.

Note that if the fraction can be represented as a *finite length string*, you **must** return it.

Example 1:

```
Input: numerator = 1, denominator = 2
Output: "0.5"
```

Example 2:

```
Input: numerator = 2, denominator = 1
Output: "2"
```

Example 3:

```
Input: numerator = 4, denominator = 333
Output: "0.(012)"
```

Constraints:

- -2^{31} <= numerator, denominator <= 2^{31} - 1
- denominator != 0

SOLUTION (PYTHON 3)

Solution 1: Mathematics + Hash Table

How to think about it

Write a fraction as a decimal and wrap the repeating part. Long division repeats iff a remainder repeats. Handle the sign and the integer part first. During the fraction, map each remainder to the first index it appeared; a repeat inserts parentheses there. Remainder 0 is a terminating decimal.

First, we check if the numerator is 0. If it is, we return `"0"` directly.

Next, we check if the numerator and denominator have different signs. If they do, the result is negative, and we set the first character of the result to `"-"`.

Then we take the absolute values of the numerator and denominator, denoted as a and b. Since the range of the numerator and denominator is [-2^{31}, 2^{31} - 1], taking the absolute value directly may cause overflow, so we convert both a and b to long integers.

Next, we calculate the integer part, which is the integer part of a divided by b, convert it to a string, and add it to the result. Then we take the remainder of a divided by b, denoted as a.

If a is 0, it means the result is an integer, and we return the result directly.

Next, we calculate the decimal part. We use a hash table d to record the length of the result corresponding to each remainder. We continuously multiply a by 10, then add the integer part of a divided by b to the result, then take the remainder of a divided by b, denoted as a. If a is 0, it means the result is a finite decimal, and we return the result directly. If a has appeared in the hash table, it means the result is a recurring decimal. We find the starting position of the cycle, insert the result into parentheses, and then return the result.

The time complexity is O(l), and the space complexity is O(l), where l is the length of the result. In this problem, $l < 10^4$.

Python 3

```
class Solution:
    def fractionToDecimal(self, numerator: int, denominator: int) -> str:
        if numerator == 0:
            return "0"
        ans = []
        neg = (numerator > 0) ^ (denominator > 0)
        if neg:
            ans.append("-")
        a, b = abs(numerator), abs(denominator)
        ans.append(str(a // b))
        a %= b
        if a == 0:
            return "".join(ans)
        ans.append(".")
        d = {}
        while a:
            d[a] = len(ans)
            a *= 10
            ans.append(str(a // b))
            a %= b
            if a in d:
                ans.insert(d[a], "(")
                ans.append(")")
                break
        return "".join(ans)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Only the length of the repeating part
# 2. The same in another base (digits 0 to base - 1)


# ============================================================
# FOLLOW-UP 1: Only the length of the repeating part
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same long division, remembering the position
# where each remainder first appeared.
# Time: O(d), d = denominator.  Space: O(d).
def period_length(numerator, denominator):
    remainder = abs(numerator) % abs(denominator)
    seen = {}
    position = 0
    while remainder != 0 and remainder not in seen:
        seen[remainder] = position
        remainder = remainder * 10 % abs(denominator)
        position += 1
    if remainder == 0:
        return 0
    return position - seen[remainder]


# ============================================================
# FOLLOW-UP 2: The same in another base (digits 0 to base
```

```
# - 1)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Long division, multiplying the remainder by the
# base instead of by 10.
# Time: O(d * log).  Space: O(d).
def fraction_in_base(numerator, denominator, base):
    digits = "0123456789abcdefghijklmnopqrstuvwxyz"
    sign = "-" if (numerator < 0) != (denominator < 0) and numerator != 0 else ""
    numerator, denominator = abs(numerator), abs(denominator)
    whole = numerator // denominator
    remainder = numerator % denominator
    integer_part = ""
    if whole == 0:
        integer_part = "0"
    while whole > 0:
        integer_part = digits[whole % base] + integer_part
        whole //= base
    if remainder == 0:
        return sign + integer_part
    fraction = []
    seen = {}
    while remainder != 0 and remainder not in seen:
        seen[remainder] = len(fraction)
        remainder *= base
        fraction.append(digits[remainder // denominator])
        remainder %= denominator
    if remainder == 0:
        return sign + integer_part + "." + "".join(fraction)
    start = seen[remainder]
    return sign + integer_part + "." + "".join(fraction[:start]) + "(" + "".join(fraction[start:]) + ")"
```

PATTERN 34 · MATH / NUMBER THEORY · PROBLEM 17 OF 20

264. Ugly Number II

Medium · Hash Table, Math, Dynamic Programming, Heap (Priority Queue)

leetcode.com/problems/ugly-number-ii

★ **TOP 7 PROBLEM** in Math / Number Theory, K-way Merge: master this one first

Also appears in pattern 38 K-way Merge

PROBLEM

An **ugly number** is a positive integer whose prime factors are limited to `2`, `3`, and `5`.

Given an integer `n`, return *the* n^{th} ***ugly number***.

Example 1:

```
Input: n = 10
Output: 12
Explanation: [1, 2, 3, 4, 5, 6, 8, 9, 10, 12] is the sequence of the first 10 ugly numbers.
```

Example 2:

```
Input: n = 1
Output: 1
Explanation: 1 has no prime factors, therefore all of its prime factors are limited to 2, 3, and 5.
```

Constraints:

- `1 <= n <= 1690`

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Testing every integer by division wastes non-ugly values when n is large. Every ugly number is a previous one times 2, 3, or 5. A min-heap pops the next ugly number and pushes its products, with a set to skip duplicates. The n-th pop is the answer.

Python 3

```python
class Solution:
    def nthUglyNumber(self, n: int) -> int:
        h = [1]
        vis = {1}
        ans = 1
        for _ in range(n):
            ans = heappop(h)
            for v in [2, 3, 5]:
                nxt = ans * v
                if nxt not in vis:
                    vis.add(nxt)
                    heappush(h, nxt)
        return ans
```

Complexity. Time: O(n log n). Space: O(n).

Solution 2

How to think about it

The heap plus set uses extra memory and a log factor. Three pointers mark the next candidates times 2,3,5; we write the min into dp and advance every pointer that produced it, avoiding duplicates.

Python 3

```python
class Solution:
    def nthUglyNumber(self, n: int) -> int:
        dp = [1] * n
        p2 = p3 = p5 = 0
        for i in range(1, n):
            next2, next3, next5 = dp[p2] * 2, dp[p3] * 3, dp[p5] * 5
            dp[i] = min(next2, next3, next5)
            if dp[i] == next2:
                p2 += 1
            if dp[i] == next3:
                p3 += 1
            if dp[i] == next5:
                p5 += 1
        return dp[-1]
```

Complexity. Time: O(n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Use a heap instead of three pointers
# 2. How many ugly numbers are there up to a limit


# ============================================================
# FOLLOW-UP 1: Use a heap instead of three pointers
# ============================================================

# ---- follow-up 1: function ----
# Answer: Pop the smallest number n times, pushing its
# multiples by 2, 3 and 5, and skipping duplicates.
# Time: O(n log n).  Space: O(n).
import heapq


def nth_ugly_by_heap(n):
    heap = [1]
    seen = {1}
    number = 1
    for _ in range(n):
        number = heapq.heappop(heap)
        for factor in (2, 3, 5):
            if number * factor not in seen:
                seen.add(number * factor)
                heapq.heappush(heap, number * factor)
    return number


# ============================================================
# FOLLOW-UP 2: How many ugly numbers are there up to a
# limit
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk through the ugly numbers in order and stop
# when one passes the limit.
# Time: O(count).  Space: O(count).
def count_ugly_up_to(limit):
```

```
    ugly = [1]
    i2 = i3 = i5 = 0
    while True:
        nxt = min(ugly[i2] * 2, ugly[i3] * 3, ugly[i5] * 5)
        if nxt > limit:
            return len(ugly)
        ugly.append(nxt)
        if nxt == ugly[i2] * 2:
            i2 += 1
        if nxt == ugly[i3] * 3:
            i3 += 1
        if nxt == ugly[i5] * 5:
            i5 += 1
```

365. Water and Jug Problem

Medium · Depth-First Search, Breadth-First Search, Math, Greatest Common Divisor, Euclidean Algorithm, Extended Euclidean Algorithm, Bézout's Identity

leetcode.com/problems/water-and-jug-problem

PROBLEM

You are given two jugs with capacities `x` liters and `y` liters. You have an infinite water supply. Return whether the total amount of water in both jugs may reach `target` using the following operations:

- Fill either jug completely with water.
- Completely empty either jug.
- Pour water from one jug into another until the receiving jug is full, or the transferring jug is empty.

Example 1:

Input: x = 3, y = 5, target = 4

Output: true

Explanation:

Follow these steps to reach a total of 4 liters:

1. Fill the 5-liter jug (0, 5).
2. Pour from the 5-liter jug into the 3-liter jug, leaving 2 liters (3, 2).
3. Empty the 3-liter jug (0, 2).
4. Transfer the 2 liters from the 5-liter jug to the 3-liter jug (2, 0).
5. Fill the 5-liter jug again (2, 5).
6. Pour from the 5-liter jug into the 3-liter jug until the 3-liter jug is full. This leaves 4 liters in the 5-liter jug (3, 4).
7. Empty the 3-liter jug. Now, you have exactly 4 liters in the 5-liter jug (0, 4).

Reference: The Die Hard example.

Example 2:

Input: x = 2, y = 6, target = 5

Output: false

Example 3:

Input: x = 1, y = 2, target = 3

Output: true

Explanation: Fill both jugs. The total amount of water in both jugs is equal to 3 now.

Constraints:

- `1 <= x, y, target <= 10^3`

SOLUTION (PYTHON 3)

Solution 1: DFS

How to think about it

Two jugs of capacities x,y; can we measure z? Bézout's identity says z must be a multiple of gcd(x,y). A state search also decides it. There are O(xy) states. dfs(i,j) is the current pair. A seen state fails; a jug or their sum equal to z succeeds. Otherwise fill, empty, or pour. Start from (0,0).

Let's denote jug1Capacity as x, jug2Capacity as y, and targetCapacity as z.

Next, we design a function dfs(i, j), which represents whether we can get z liters of water when there are i liters of water in jug1 and j liters of water in jug2.

The execution process of the function dfs(i, j) is as follows:

- If (i, j) has been visited, return false.
- If i = z or j = z or i + j = z, return true.
- If we can get z liters of water by filling jug1 or jug2, or emptying jug1 or jug2, return true.
- If we can get z liters of water by pouring water from jug1 into jug2, or pouring water from jug2 into jug1, return true.

The answer is dfs(0, 0).

The time complexity is O(x + y), and the space complexity is O(x + y). Here, x and y are the sizes of jug1Capacity and jug2Capacity respectively.

Python 3

```
class Solution:
    def canMeasureWater(self, x: int, y: int, z: int) -> bool:
        def dfs(i: int, j: int) -> bool:
            if (i, j) in vis:
                return False
            vis.add((i, j))
            if i == z or j == z or i + j == z:
                return True
            if dfs(x, j) or dfs(i, y) or dfs(0, j) or dfs(i, 0):
                return True
            a = min(i, y - j)
            b = min(j, x - i)
            return dfs(i - a, j + a) or dfs(i + b, j - b)

        vis = set()
        return dfs(0, 0)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the shortest list of pours
# 2. Any number of jugs


# ============================================================
# FOLLOW-UP 1: Return the shortest list of pours
# ============================================================

# ---- follow-up 1: function ----
# Answer: Breadth-first search over (water in jug 1, water
# in jug 2); every state remembers how it was reached.
# Time: O(x * y).  Space: O(x * y).
def jug_steps(x, y, target):
    start = (0, 0)
    came_from = {start: None}
    queue = [start]
    for a, b in queue:
        if a == target or b == target or a + b == target:
            steps = []
            state = (a, b)
            while state is not None:
                steps.append(state)
                state = came_from[state]
            return steps[::-1]
        pour_ab = min(a, y - b)
        pour_ba = min(b, x - a)
        for nxt in ((x, b), (a, y), (0, b), (a, 0), (a - pour_ab, b + pour_ab), (a + pour_ba, b -
pour_ba)):
            if nxt not in came_from:
```

```
                came_from[nxt] = (a, b)
                queue.append(nxt)
    return None


# ============================================================
# FOLLOW-UP 2: Any number of jugs
# ============================================================

# ---- follow-up 2: function ----
# Answer: The target is reachable when it is at most the
# total capacity and divisible by the gcd of all jug
# sizes.
# Time: O(k log c).  Space: O(1).
def can_measure_with_jugs(capacities, target):
    def gcd(a, b):
        while b:
            a, b = b, a % b
        return a

    g = 0
    for c in capacities:
        g = gcd(g, c)
    if target > sum(capacities):
        return False
    return target % g == 0
```

400. Nth Digit

Medium · Math, Binary Search

leetcode.com/problems/nth-digit

PROBLEM

Given an integer `n`, return the n^{th} digit of the infinite integer sequence `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, ...]`.

Example 1:

```
Input: n = 3
Output: 3
```

Example 2:

```
Input: n = 11
Output: 0
Explanation: The 11th digit of the sequence 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, ... is a 0, which is part of the number 10.
```

Constraints:

- $1 <= n <= 2^{31} - 1$

SOLUTION (PYTHON 3)

Solution 1: Mathematics

> **How to think about it**
>
> Building the infinite concatenation and indexing the n-th character is impossible: n can reach 2^{31}-1. Numbers with k digits contribute a closed-form count: $9\times 10^{k-1}$ of them, hence $k\times 9\times 10^{k-1}$ digits. Subtract those blocks while increasing k until n lands inside the current length, then recover the integer and the offset inside it. The subtraction must finish before locating a concrete number; otherwise a global rank cannot become an index inside one integer.

The smallest and largest integers with k digits are 10^{k-1} and 10^k-1 respectively, so the total number of digits for k-digit numbers is $k \times 9 \times 10^{k-1}$.

We use k to represent the number of digits of the current number, and cnt to represent the total number of numbers with the current number of digits. Initially, k=1, cnt=9.

Each time we subtract cnt × k from n, when n is less than or equal to cnt × k, it means that the number corresponding to n is within the range of numbers with the current number of digits. At this time, we can calculate the corresponding number.

The specific method is to first calculate which number of the current number of digits corresponds to n, and then calculate which digit of this number it is, so as to get the number on this digit.

The time complexity is $O(\log_{10} n)$.

Python 3

```python
class Solution:
    def findNthDigit(self, n: int) -> int:
        k, cnt = 1, 9
        while k * cnt < n:
            n -= k * cnt
            k += 1
            cnt *= 10
        num = 10 ** (k - 1) + (n - 1) // k
        idx = (n - 1) % k
        return int(str(num)[idx])
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Where does a number first show up: the position of
# its first digit
# 2. The nth digit of the sequence of EVEN numbers
# 2468101214...


# ============================================================
# FOLLOW-UP 1: Where does a number first show up: the
# position of its first digit
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count the digits written before it: all shorter
# numbers plus the digits of the numbers before it of the
# same length.
# Time: O(log x).  Space: O(1).
def position_of_number(x):
    length = len(str(x))
    position = 1
    for shorter in range(1, length):
        position += 9 * 10 ** (shorter - 1) * shorter
    position += (x - 10 ** (length - 1)) * length
    return position


# ============================================================
# FOLLOW-UP 2: The nth digit of the sequence of EVEN
# numbers 2468101214...
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same block idea over even numbers: there are 4
# (2,4,6,8), 45 (10 to 98), ... of each length.
# Time: O(log n).  Space: O(1).
def nth_digit_of_evens(n):
    length = 1
    count = 4
    start = 2
    while n > count * length:
        n -= count * length
        length += 1
        start = 10 ** (length - 1)
        count = 45 * 10 ** (length - 2) if length > 1 else 4
        if start % 2 == 1:
            start += 1
    number = start + 2 * ((n - 1) // length)
    return int(str(number)[(n - 1) % length])
```

PATTERN 35 OF 41

Sliding Window Maximum / Minimum (Specific Variants)

20 problems · 2 Easy · 13 Medium · 5 Hard

THE IDEA

A window that needs its maximum or minimum at every step: keep a deque of candidate indexes whose values are in decreasing (or increasing) order.

Signals: sliding window maximum, shortest subarray with sum at least K, longest subarray with limit, jump game VI.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
from collections import deque
dq = deque()                          # indexes, values decreasing
for i, x in enumerate(nums):
    while dq and nums[dq[-1]] <= x:
        dq.pop()                      # smaller ones can never be the max again
    dq.append(i)
    if dq[0] <= i - k:
        dq.popleft()                  # left the window
    if i >= k - 1:
        result.append(nums[dq[0]])
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Maximum Average Subarray I	Easy	LC 643	1836
2	Maximum Sum of Almost Unique Subarray — substitute: Minimum Recolors to Get K Consecutive Black Blocks	Easy	LC 2379	1838
3	★ **Jump Game VI** (see page)	Medium	LC 1696	519
4	★ **Maximum Number of Robots Within Budget** (see page)	Medium	LC 2398	509
5	Sum of Subarray Minimums (see page)	Medium	LC 907	440
6	Sum of Subarray Ranges (see page)	Medium	LC 2104	459
7	Continuous Subarrays — substitute: Maximum Side Length of a Square with Sum Less than or Equal to Threshold	Medium	LC 1292	1840
8	Subarray Product Less Than K (see page)	Medium	LC 713	94
9	★ **Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit**	Medium	LC 1438	1843
10	Frequency of the Most Frequent Element (see page)	Medium	LC 1838	114
11	Grumpy Bookstore Owner (see page)	Medium	LC 1052	111
12	Maximum Points You Can Obtain from Cards (see page)	Medium	LC 1423	1199
13	Minimum Number of Flips to Make the Binary String Alternating (see page)	Medium	LC 1888	118
14	Maximum Erasure Value (see page)	Medium	LC 1695	108
15	Find the Longest Equal Subarray	Medium	LC 2831	1848
16	★ **Sliding Window Maximum** (see page)	Hard	LC 239	124
17	★ **Shortest Subarray with Sum at Least K** (see page)	Hard	LC 862	472
18	★ **Constrained Subsequence Sum** (see page)	Hard	LC 1425	474
19	Minimum Window Substring (see page)	Hard	LC 76	121
20	★ **Count Subarrays With Fixed Bounds**	Hard	LC 2444	1851

PATTERN 35 · SLIDING WINDOW MAXIMUM / MINIMUM (SPECIFIC VARIANTS) · PROBLEM 1 OF 20

643. Maximum Average Subarray I

Easy · Array, Sliding Window

leetcode.com/problems/maximum-average-subarray-i

PROBLEM

You are given an integer array `nums` consisting of `n` elements, and an integer `k`.

Find a contiguous subarray whose **length is equal to** `k` that has the maximum average value and return *this value*. Any answer with a calculation error less than 10^{-5} will be accepted.

Example 1:

```
Input: nums = [1,12,-5,-6,50,3], k = 4
Output: 12.75000
Explanation: Maximum average is (12 - 5 - 6 + 50) / 4 = 51 / 4 = 12.75
```

Example 2:

```
Input: nums = [5], k = 1
Output: 5.00000
```

Constraints:

- `n == nums.length`
- `1 <= k <= n <=` 10^5
- -10^4 `<= nums[i] <=` 10^4

SOLUTION (PYTHON 3)

Solution 1: Sliding Window

How to think about it

The maximum average of a window of length k is the maximum sum. Recomputing each window is O(nk) for n≤ 10^5. Slide a length-k sum: add the entering value, drop the leaving one, and divide the best sum by k.

We maintain a sliding window of length k, and for each window, we calculate the sum s of the numbers within the window. We take the maximum sum s as the answer.

The time complexity is O(n), where n is the length of the array nums. The space complexity is O(1).

Python 3

```python
class Solution:
    def findMaxAverage(self, nums: List[int], k: int) -> float:
        ans = s = sum(nums[:k])
        for i in range(k, len(nums)):
            s += nums[i] - nums[i - k]
            ans = max(ans, s)
        return ans / k
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The window length may be any length of at least k
# 2. The smallest average of a window of length k, with
# its start


# ============================================================
# FOLLOW-UP 1: The window length may be any length of at
# least k
# ============================================================

# ---- follow-up 1: function ----
# Answer: Binary search the average: a window of length >=
# k with average >= m exists when some window has a non-
# negative sum after subtracting m from every number.
# Time: O(60 * n) = O(n) (60 rounds of binary search).
# Space: O(1).
def max_average_at_least_k(nums, k):
    def can_reach(m):
        total = 0
        for i in range(k):
            total += nums[i] - m
        if total >= 0:
            return True
        before = 0
        smallest_before = 0
        for i in range(k, len(nums)):
            total += nums[i] - m
            before += nums[i - k] - m
            if before < smallest_before:
                smallest_before = before
            if total - smallest_before >= 0:
                return True
        return False

    low, high = min(nums), max(nums)
    for _ in range(60):
        middle = (low + high) / 2
        if can_reach(middle):
            low = middle
        else:
            high = middle
    return low


# ============================================================
# FOLLOW-UP 2: The smallest average of a window of length
# k, with its start
# ============================================================

# ---- follow-up 2: function ----
# Answer: Slide the window and keep the smallest sum.
# Time: O(n).  Space: O(1).
def min_average_window(nums, k):
    total = sum(nums[:k])
    best_total = total
    best_start = 0
    for i in range(k, len(nums)):
        total += nums[i] - nums[i - k]
        if total < best_total:
            best_total = total
            best_start = i - k + 1
    return best_total / k, best_start
```

2379. Minimum Recolors to Get K Consecutive Black Blocks

Easy · String, Sliding Window

leetcode.com/problems/minimum-recolors-to-get-k-consecutive-black-blocks

PROBLEM

You are given a **0-indexed** string `blocks` of length `n`, where `blocks[i]` is either `'W'` or `'B'`, representing the color of the i^{th} block. The characters `'W'` and `'B'` denote the colors white and black, respectively.

You are also given an integer `k`, which is the desired number of **consecutive** black blocks.

In one operation, you can **recolor** a white block such that it becomes a black block.

Return *the **minimum** number of operations needed such that there is at least **one** occurrence of `k` consecutive black blocks.*

Example 1:

```
Input: blocks = "WBBWWBBWBW", k = 7
Output: 3
Explanation:
One way to achieve 7 consecutive black blocks is to recolor the 0th, 3rd, and 4th blocks
so that blocks = "BBBBBBBWBW".
It can be shown that there is no way to achieve 7 consecutive black blocks in less than 3 operations.
Therefore, we return 3.
```

Example 2:

```
Input: blocks = "WBWBBBW", k = 2
Output: 0
Explanation:
No changes need to be made, since 2 consecutive black blocks already exist.
Therefore, we return 0.
```

Constraints:

- `n == blocks.length`
- `1 <= n <= 100`
- `blocks[i]` is either `'W'` or `'B'`.
- `1 <= k <= n`

SOLUTION (PYTHON 3)

Solution 1: Sliding Window

How to think about it

Painting a window of length k all black costs its number of whites. We only need every window, not a search over colorings. Count whites in the first k cells, then slide, adding and removing the ends, and keep the minimum.

We observe that what the problem actually asks for is the minimum number of white blocks in a sliding window of size k.

Therefore, we only need to traverse the string blocks, use a variable cnt to count the number of white blocks in the current window, and then use a variable ans to maintain the minimum value.

After the traversal ends, we can get the answer.

The time complexity is O(n), where n is the length of the string blocks. The space complexity is O(1).

Python 3

```
class Solution:
    def minimumRecolors(self, blocks: str, k: int) -> int:
        ans = cnt = blocks[:k].count('W')
        for i in range(k, len(blocks)):
            cnt += blocks[i] == 'W'
            cnt -= blocks[i - k] == 'W'
            ans = min(ans, cnt)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return where the best window starts
# 2. The blocks form a circle


# ============================================================
# FOLLOW-UP 1: Return where the best window starts
# ============================================================

# ---- follow-up 1: function ----
# Answer: Slide a window of length k counting the white
# blocks; remember the start of the window with the
# fewest.
# Time: O(n).  Space: O(1).
def best_window_start(blocks, k):
    whites = 0
    for i in range(k):
        if blocks[i] == "W":
            whites += 1
    best = whites
    start = 0
    for i in range(k, len(blocks)):
        if blocks[i] == "W":
            whites += 1
        if blocks[i - k] == "W":
            whites -= 1
        if whites < best:
            best = whites
            start = i - k + 1
    return start, best


# ============================================================
# FOLLOW-UP 2: The blocks form a circle
# ============================================================

# ---- follow-up 2: function ----
# Answer: Slide over the string written twice, with
# windows starting in the first copy.
# Time: O(n).  Space: O(n).
def min_recolors_circular(blocks, k):
    doubled = blocks + blocks
    whites = 0
    for i in range(k):
        if doubled[i] == "W":
            whites += 1
    best = whites
    for i in range(k, len(blocks) + k - 1):
        if doubled[i] == "W":
            whites += 1
        if doubled[i - k] == "W":
            whites -= 1
        best = min(best, whites)
    return best
```

1292. Maximum Side Length of a Square with Sum Less than or Equal to Threshold

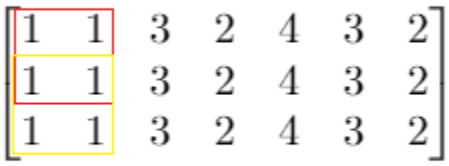
Medium · Array, Binary Search, Matrix, Prefix Sum

leetcode.com/problems/maximum-side-length-of-a-square-with-sum-less-than-or-equal-to-threshold

PROBLEM

Given a `m x n` matrix `mat` and an integer `threshold`, return *the maximum side-length of a square with a sum less than or equal to* `threshold` *or return* `0` *if there is no such square*.

Example 1:




```
Input: mat = [[1,1,3,2,4,3,2],[1,1,3,2,4,3,2],[1,1,3,2,4,3,2]], threshold = 4
Output: 2
Explanation: The maximum side length of square with sum less than or equal to 4 is 2 as shown.
```

Example 2:

```
Input: mat = [[2,2,2,2,2],[2,2,2,2,2],[2,2,2,2,2],[2,2,2,2,2],[2,2,2,2,2]], threshold = 1
Output: 0
```

Constraints:

- `m == mat.length`
- `n == mat[i].length`
- `1 <= m, n <= 300`
- `0 <= mat[i][j] <= 10^4`
- `0 <= threshold <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: 2D Prefix Sum + Binary Search

How to think about it

A square's sum grows with its side (entries are non-negative), so feasibility is monotone. m,n ≤ 300; enumerating sides and corners with an $O(k^2)$ sum is too slow. A 2-D prefix makes every square O(1), and we binary-search the side. The check scans top-left corners and compares the prefix sum to the threshold. The search returns the largest feasible side.

We can precompute a 2D prefix sum array s, where s[i + 1][j + 1] represents the sum of elements in the matrix mat from (0, 0) to (i, j). With this, we can calculate the sum of elements in any square region in O(1) time.

Next, we can use binary search to find the maximum side length. We enumerate the side length k of the square, and then iterate through all possible top-left positions (i, j) of the square. We can calculate the sum of elements v for the square. If v ≤ threshold, it indicates that there exists a square region with side length k whose sum is less than or equal to the threshold;

otherwise, no such square exists for the current k.

The time complexity is O(m × n × log min(m, n)), and the space complexity is O(m × n).

Python 3

```python
class Solution:
    def maxSideLength(self, mat: List[List[int]], threshold: int) -> int:
        def check(k: int) -> bool:
            for i in range(m - k + 1):
                for j in range(n - k + 1):
                    v = s[i + k][j + k] - s[i][j + k] - s[i + k][j] + s[i][j]
                    if v <= threshold:
                        return True
            return False

        m, n = len(mat), len(mat[0])
        s = [[0] * (n + 1) for _ in range(m + 1)]
        for i, row in enumerate(mat, 1):
            for j, x in enumerate(row, 1):
                s[i][j] = s[i - 1][j] + s[i][j - 1] - s[i - 1][j - 1] + x
        l, r = 0, min(m, n)
        while l < r:
            mid = (l + r + 1) >> 1
            if check(mid):
                l = mid
            else:
                r = mid - 1
        return l
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the top left corner of such a largest square
# 2. Count the squares of a given side whose sum is at
# most the threshold


# ============================================================
# FOLLOW-UP 1: Return the top left corner of such a
# largest square
# ============================================================

# ---- follow-up 1: function ----
# Answer: Prefix sums; for the best side length, search
# the first square whose sum is within the threshold.
# Time: O(min(m, n) * m * n) in the worst case.  Space:
# O(m * n).
def largest_square_position(mat, threshold):
    rows, cols = len(mat), len(mat[0])
    prefix = [[0] * (cols + 1) for _ in range(rows + 1)]
    for r in range(rows):
        for c in range(cols):
            prefix[r + 1][c + 1] = mat[r][c] + prefix[r][c + 1] + prefix[r + 1][c] - prefix[r][c]

    def square_sum(r, c, side):
        return prefix[r + side][c + side] - prefix[r][c + side] - prefix[r + side][c] + prefix[r][c]

    best = (0, None)
    for side in range(1, min(rows, cols) + 1):
        found = None
        for r in range(rows - side + 1):
            for c in range(cols - side + 1):
                if square_sum(r, c, side) <= threshold:
                    found = (r, c)
                    break
            if found:
                break
```

```
        if found:
            best = (side, found)
        else:
            break
    return best


# ============================================================
# FOLLOW-UP 2: Count the squares of a given side whose sum
# is at most the threshold
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same prefix sums, counting every position.
# Time: O(m * n).  Space: O(m * n).
def count_squares_of_side(mat, side, threshold):
    rows, cols = len(mat), len(mat[0])
    prefix = [[0] * (cols + 1) for _ in range(rows + 1)]
    for r in range(rows):
        for c in range(cols):
            prefix[r + 1][c + 1] = mat[r][c] + prefix[r][c + 1] + prefix[r + 1][c] - prefix[r][c]
    count = 0
    for r in range(rows - side + 1):
        for c in range(cols - side + 1):
```

```
            total = prefix[r + side][c + side] - prefix[r][c + side] - prefix[r + side][c] + prefix[r][c]
            if total <= threshold:
                count += 1
    return count
```

1438. Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit

Medium · Queue, Array, Ordered Set, Sliding Window, Monotonic Queue, Heap (Priority Queue)

leetcode.com/problems/longest-continuous-subarray-with-absolute-diff-less-than-or-equal-to-limit

★ **TOP 7 PROBLEM** in Sliding Window Maximum / Minimum (Specific Variants): master this one first

PROBLEM

You are given an array of integers `nums` and an integer `limit`.

Return the size of the **longest non-empty subarray** such that the **absolute** difference between **every pair of elements** in this **subarray** is **less than or equal** to `limit`.

Example 1:

Input: nums = [8,2,4,7], limit = 4

Output: 2

Explanation:

Checking every subarray, where the absolute difference is taken over all pairs of its elements:
`[8]` with maximum absolute diff `|8-8| = 0 <= 4`.
`[8,2]` with maximum absolute diff `|8-2| = 6 > 4`.
`[8,2,4]` with maximum absolute diff `|8-2| = 6 > 4`.
`[8,2,4,7]` with maximum absolute diff `|8-2| = 6 > 4`.
`[2]` with maximum absolute diff `|2-2| = 0 <= 4`.
`[2,4]` with maximum absolute diff `|4-2| = 2 <= 4`.
`[2,4,7]` with maximum absolute diff `|7-2| = 5 > 4`. Note that the pair `(2, 7)` is not adjacent, but it is still compared.
`[4]` with maximum absolute diff `|4-4| = 0 <= 4`.
`[4,7]` with maximum absolute diff `|7-4| = 3 <= 4`.
`[7]` with maximum absolute diff `|7-7| = 0 <= 4`.
Therefore, the size of the longest subarray is 2.

Example 2:

Input: nums = [10,1,2,4,7,2], limit = 5

Output: 4

Explanation:

The subarray `[2,4,7,2]` is valid because every pair of its elements differs by at most `5`, the largest being `|7-2| = 5`.

It cannot be extended to the left, since `[1,2,4,7,2]` contains the pair `(1, 7)` with `|7-1| = 6 > 5`. Therefore, the size of the longest subarray is 4.

Example 3:

Input: nums = [4,2,2,2,4,4,2,2], limit = 0

Output: 3

Explanation:

Since `limit = 0`, every pair of elements in the subarray must be equal, so only runs of identical values qualify.

The longest such run is `[2,2,2]`, of size 3.

Constraints:

- $1 <= nums.length <= 10^5$
- $1 <= nums[i] <= 10^9$

- $0 <= limit <= 10^9$

SOLUTIONS (PYTHON 3)

Solution 1: Ordered Set + Sliding Window

> **How to think about it**
>
> The max–min gap of a subarray does not decrease as the window grows. n≤ 10^5 rules out all intervals. For each right end, move the left end until max-min≤ limit. An ordered multiset exposes the current extrema; remove the leftmost value when the window is illegal. The longest valid window is the answer.

We can enumerate each position as the right endpoint of the subarray, and find the leftmost left endpoint corresponding to it, such that the difference between the maximum and minimum values in the interval does not exceed limit. During the process, we use an ordered set to maintain the maximum and minimum values within the window.

The time complexity is O(n log n), and the space complexity is O(n). Here, n is the length of the array `nums`.

Python 3

```
class Solution:
    def longestSubarray(self, nums: List[int], limit: int) -> int:
        sl = SortedList()
        ans = j = 0
        for i, x in enumerate(nums):
            sl.add(x)
            while sl[-1] - sl[0] > limit:
                sl.remove(nums[j])
                j += 1
            ans = max(ans, i - j + 1)
        return ans
```

Solution 2: Binary Search + Sliding Window

> **How to think about it**
>
> Method 1 shrinks per right end. Length is monotone: if some window of length mid is valid, shorter ones exist. Binary-search the length and test a candidate with two monotonic queues. The time is still O(nlog n); the longest-window search becomes a decision problem.

We notice that if a subarray of length k satisfies the condition, then a subarray of length k' < k also satisfies the condition. This shows a monotonicity, therefore, we can use binary search to find the longest subarray that satisfies the condition.

We define the left boundary of the binary search as l = 0, and the right boundary as r = n. For each mid = (l + r + 1)/2, we check whether there exists a subarray of length mid that satisfies the condition. If it exists, we update l = mid, otherwise we update r = mid - 1. The problem is transformed into whether there exists a subarray of length mid in the array that satisfies the condition, which is actually to find the difference between the maximum and minimum values in the sliding window does not exceed limit. We can use two monotonic queues to maintain the maximum and minimum values in the window respectively.

The time complexity is O(n × log n), and the space complexity is O(n). Where n is the length of the array nums.

Python 3

```python
class Solution:
    def longestSubarray(self, nums: List[int], limit: int) -> int:
        def check(k: int) -> bool:
            min_q = deque()
            max_q = deque()
            for i, x in enumerate(nums):
                if min_q and i - min_q[0] + 1 > k:
                    min_q.popleft()
                if max_q and i - max_q[0] + 1 > k:
                    max_q.popleft()
                while min_q and nums[min_q[-1]] >= x:
                    min_q.pop()
                while max_q and nums[max_q[-1]] <= x:
                    max_q.pop()
                min_q.append(i)
                max_q.append(i)
                if i >= k - 1 and nums[max_q[0]] - nums[min_q[0]] <= limit:
                    return True
            return False

        l, r = 1, len(nums)
        while l < r:
            mid = (l + r + 1) >> 1
            if check(mid):
                l = mid
            else:
                r = mid - 1
        return l
```

Solution 3: Sliding Window + Deque

> **How to think about it**
>
> Methods 1 and 2 pay a logarithmic factor. Two monotonic deques maintain the window max and min in amortized O(1), so expanding the right end and shrinking the left end is linear.

We can use a deque to maintain the maximum and minimum values within the window. We maintain two deques, one for storing the indices of the maximum values and the other for the minimum values within the window. Define two pointers l and r to point to the left and right boundaries of the window, respectively.

Each time we move the right boundary r to the right, we check if the element corresponding to the tail index of the maximum value deque is less than the current element. If it is, we dequeue the tail element until the element corresponding to the tail of the maximum value deque is not less than the current element. Similarly, we check if the element corresponding to the tail index of the minimum value deque is greater than the current element. If it is, we dequeue the tail element until the element corresponding to the tail of the minimum value deque is not greater than the current element. Then, we enqueue the current element's index.

If the difference between the elements at the front of the maximum value deque and the minimum value deque is greater than limit, then we move the left boundary l to the right. If the element at the front of the maximum value deque is less than l, we dequeue the front element of the maximum value deque. Similarly, if the element at the front of the minimum value deque is less than l, we dequeue the front element of the minimum value deque.

The answer is n - l.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array nums.

Python 3

```
class Solution:
    def longestSubarray(self, nums: List[int], limit: int) -> int:
        maxq = deque()
        minq = deque()
        l, n = 0, len(nums)
        for r, x in enumerate(nums):
            while maxq and nums[maxq[-1]] < x:
                maxq.pop()
            while minq and nums[minq[-1]] > x:
                minq.pop()
            maxq.append(r)
            minq.append(r)
            if nums[maxq[0]] - nums[minq[0]] > limit:
                l += 1
                if maxq[0] < l:
                    maxq.popleft()
                if minq[0] < l:
                    minq.popleft()
        return n - l
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subarray itself
# 2. Count ALL subarrays whose max minus min is at most
# the limit


# ============================================================
# FOLLOW-UP 1: Return the subarray itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same two monotonic queues; remember where the
# longest window starts.
# Time: O(n).  Space: O(n).
from collections import deque


def longest_subarray_itself(nums, limit):
    biggest = deque()
    smallest = deque()
    left = 0
    best_length = 0
    best_start = 0
    for right in range(len(nums)):
        while biggest and nums[biggest[-1]] <= nums[right]:
            biggest.pop()
        biggest.append(right)
        while smallest and nums[smallest[-1]] >= nums[right]:
            smallest.pop()
        smallest.append(right)
        while nums[biggest[0]] - nums[smallest[0]] > limit:
            left += 1
            if biggest[0] < left:
                biggest.popleft()
            if smallest[0] < left:
                smallest.popleft()
        if right - left + 1 > best_length:
            best_length = right - left + 1
            best_start = left
    return nums[best_start:best_start + best_length]


# ============================================================
```

```
# FOLLOW-UP 2: Count ALL subarrays whose max minus min is
# at most the limit
# ============================================================

# ---- follow-up 2: function ----
# Answer: For every right end, every start from the
# window's left up to right works, so add the window
# length.
# Time: O(n).  Space: O(n).
def count_subarrays_within_limit(nums, limit):
    biggest = deque()
    smallest = deque()
    left = 0
    total = 0
    for right in range(len(nums)):
        while biggest and nums[biggest[-1]] <= nums[right]:
            biggest.pop()
        biggest.append(right)
        while smallest and nums[smallest[-1]] >= nums[right]:
            smallest.pop()
        smallest.append(right)
        while nums[biggest[0]] - nums[smallest[0]] > limit:
            left += 1
            if biggest[0] < left:
                biggest.popleft()
            if smallest[0] < left:
                smallest.popleft()
        total += right - left + 1
    return total
```

2831. Find the Longest Equal Subarray

Medium · Array, Hash Table, Binary Search, Sliding Window

leetcode.com/problems/find-the-longest-equal-subarray

PROBLEM

You are given a **0-indexed** integer array `nums` and an integer `k`.

A subarray is called **equal** if all of its elements are equal. Note that the empty subarray is an **equal** subarray.

Return *the length of the **longest** possible equal subarray after deleting **at most** `k` elements from* `nums`.

A **subarray** is a contiguous, possibly empty sequence of elements within an array.

Example 1:

```
Input: nums = [1,3,2,3,1,3], k = 3
Output: 3
Explanation: It's optimal to delete the elements at index 2 and index 4.
After deleting them, nums becomes equal to [1, 3, 3, 3].
The longest equal subarray starts at i = 1 and ends at j = 3 with length equal to 3.
It can be proven that no longer equal subarrays can be created.
```

Example 2:

```
Input: nums = [1,1,2,2,1,1], k = 2
Output: 4
Explanation: It's optimal to delete the elements at index 2 and index 3.
After deleting them, nums becomes equal to [1, 1, 1, 1].
The array itself is an equal subarray, so the answer is 4.
It can be proven that no longer equal subarrays can be created.
```

Constraints:

- `1 <= nums.length <= 10^5`
- `1 <= nums[i] <= nums.length`
- `0 <= k <= nums.length`

SOLUTIONS (PYTHON 3)

Solution 1: Hash Table + Two Pointers

> **How to think about it**
>
> An equal subarray may delete at most k other values. A window is illegal once the count of non-majority elements exceeds k. A frequency map tracks the peak count mx; shrink from the left when r-l+1-mx>k. The answer is that peak.

We use two pointers to maintain a monotonically variable length window, and a hash table to maintain the number of occurrences of each element in the window.

The number of all elements in the window minus the number of the most frequently occurring element in the window is the number of elements that need to be deleted from the window.

Each time, we add the element pointed to by the right pointer to the window, then update the hash table, and also update the number of the most frequently occurring element in the window. When the number of elements that need to be deleted from the window exceeds k, we move the left pointer once, and then update the hash table.

After the traversal, return the number of the most frequently occurring element.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array.

Python 3

```python
class Solution:
    def longestEqualSubarray(self, nums: List[int], k: int) -> int:
        cnt = Counter()
        l = 0
        mx = 0
        for r, x in enumerate(nums):
            cnt[x] += 1
            mx = max(mx, cnt[x])
            if r - l + 1 - mx > k:
                cnt[nums[l]] -= 1
                l += 1
        return mx
```

Solution 2: Hash Table + Two Pointers (Method 2)

> **How to think about it**
>
> Method 1 keeps frequencies of every value in one window. Grouping indices by value lets each two-pointer scan count only the gaps between occurrences of that value, with the same linear bound.

We can use a hash table g to maintain the index list of each element.

Next, we enumerate each element as the equal value element. We take out the index list ids of this element from the hash table g. Then we define two pointers l and r to maintain a window, so that the number of elements in the window minus the number of equal value elements does not exceed k. Therefore, we only need to find the largest window that meets the condition.

The time complexity is O(n), and the space complexity is O(n). Where n is the length of the array.

Python 3

```python
class Solution:
    def longestEqualSubarray(self, nums: List[int], k: int) -> int:
        g = defaultdict(list)
        for i, x in enumerate(nums):
            g[x].append(i)
        ans = 0
        for ids in g.values():
            l = 0
            for r in range(len(ids)):
                while ids[r] - ids[l] - (r - l) > k:
                    l += 1
                ans = max(ans, r - l + 1)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Which value gives the longest equal subarray
# 2. The longest equal subarray you can get for EACH value


# ============================================================
# FOLLOW-UP 1: Which value gives the longest equal
# subarray
# ============================================================

# ---- follow-up 1: function ----
# Answer: Run the window for every value separately and
# remember the value of the best.
# Time: O(n).  Space: O(n).
def best_value_for_equal_subarray(nums, k):
    positions = {}
```

```
    for i in range(len(nums)):
        positions.setdefault(nums[i], []).append(i)
    best_length = 0
    best_value = None
    for value, spots in positions.items():
        left = 0
        for right in range(len(spots)):
            while spots[right] - spots[left] - (right - left) > k:
                left += 1
            if right - left + 1 > best_length:
                best_length = right - left + 1
                best_value = value
    return best_value, best_length


# ============================================================
# FOLLOW-UP 2: The longest equal subarray you can get for
# EACH value
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same window, storing the best length per
# value.
# Time: O(n).  Space: O(n).
def best_length_for_each_value(nums, k):
    positions = {}
    for i in range(len(nums)):
        positions.setdefault(nums[i], []).append(i)
    answer = {}
    for value, spots in positions.items():
        left = 0
        best = 0
        for right in range(len(spots)):
            while spots[right] - spots[left] - (right - left) > k:
                left += 1
            best = max(best, right - left + 1)
        answer[value] = best
    return answer
```

2444. Count Subarrays With Fixed Bounds

Hard · Queue, Array, Sliding Window, Monotonic Queue

leetcode.com/problems/count-subarrays-with-fixed-bounds

★ **TOP 7 PROBLEM** in Sliding Window Maximum / Minimum (Specific Variants): master this one first

PROBLEM

You are given an integer array `nums` and two integers `minK` and `maxK`.

A **fixed-bound subarray** of `nums` is a subarray that satisfies the following conditions:

- The **minimum** value in the subarray is equal to `minK`.
- The **maximum** value in the subarray is equal to `maxK`.

Return *the **number** of fixed-bound subarrays*.

A **subarray** is a **contiguous** part of an array.

Example 1:

```
Input: nums = [1,3,5,2,7,5], minK = 1, maxK = 5
Output: 2
Explanation: The fixed-bound subarrays are [1,3,5] and [1,3,5,2].
```

Example 2:

```
Input: nums = [1,1,1,1], minK = 1, maxK = 1
Output: 10
Explanation: Every subarray of nums is a fixed-bound subarray. There are 10 possible subarrays.
```

Constraints:

- `2 <= nums.length <= 10^5`
- `1 <= nums[i], minK, maxK <= 10^6`

SOLUTION (PYTHON 3)

Solution 1: Enumerate the Right Endpoint

How to think about it

At $n \le 10^5$, a fixed-bound subarray stays inside [minK,maxK] and contains both extrema. For a right end i, the left end must be after the last out-of-range index and at most the earlier of the last minK and maxK. Keep those three indices k, j_1, j_2 and add $\max(0, \min(j_1, j_2) - k)$. Each right end is O(1).

According to the problem description, we know that all elements of a bounded subarray are within the range [minK, maxK], and the minimum value must be minK, while the maximum value must be maxK.

We iterate through the array nums and count the number of bounded subarrays with nums[i] as the right endpoint. Then, we sum up all the counts.

The specific implementation logic is as follows:

1. Maintain the index k of the most recent element that is not within the range [minK, maxK], initialized to -1. The left endpoint of the current element nums[i] must be greater than k.
2. Maintain the most recent index j_1 where the value is minK and the most recent index j_2 where the value is maxK, both initialized to -1. The left endpoint of the current element nums[i] must be less than or equal to $\min(j_1, j_2)$.
3. Based on the above, the number of bounded subarrays with the current element as the right endpoint is $\max(0, \min(j_1, j_2) - k)$. Accumulate all these counts to get the result.

The time complexity is O(n), where n is the length of the array nums. The space complexity is O(1).

Python 3

```python
class Solution:
    def countSubarrays(self, nums: List[int], minK: int, maxK: int) -> int:
        j1 = j2 = k = -1
        ans = 0
        for i, v in enumerate(nums):
            if v < minK or v > maxK:
                k = i
            if v == minK:
                j1 = i
            if v == maxK:
                j2 = i
            ans += max(0, min(j1, j2) - k)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count subarrays whose elements are all between the
# bounds (min >= minK and max <= maxK)
# 2. Count subarrays whose minimum is exactly minK (no
# condition on the maximum)


# ============================================================
# FOLLOW-UP 1: Count subarrays whose elements are all
# between the bounds (min >= minK and max <= maxK)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Cut the array at elements outside the range; a
# piece of length L has L * (L + 1) / 2 subarrays.
# Time: O(n).  Space: O(1).
def count_within_bounds(nums, min_k, max_k):
    total = 0
    run = 0
    for x in nums:
        if min_k <= x <= max_k:
            run += 1
            total += run
        else:
            run = 0
    return total


# ============================================================
# FOLLOW-UP 2: Count subarrays whose minimum is exactly
# minK (no condition on the maximum)
# ============================================================

# ---- follow-up 2: function ----
# Answer: For every right end, count the starts after the
# last element smaller than minK and not after the last
# minK.
# Time: O(n).  Space: O(1).
def count_with_min_exactly(nums, min_k):
    total = 0
    last_small = -1
    last_min = -1
    for i in range(len(nums)):
        if nums[i] < min_k:
            last_small = i
            last_min = -1
        elif nums[i] == min_k:
            last_min = i
        if last_min != -1:
```

```
            total += last_min - last_small
    return total
```

PATTERN 36 OF 41

Simulation / Array Rearrangement

20 problems · 4 Easy · 15 Medium · 1 Hard

THE IDEA

Do exactly what the statement says, but choose the representation so each step is simple. Rearrangements often use two indexes and an in-place trick.

Signals: rotate array, spiral matrix, game of life, move zeroes, sort colors, rearrange by sign, set matrix zeroes.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
write = 0
for read in range(len(nums)):
    if nums[read] != 0:
        nums[write] = nums[read]      # compact the non-zeros
        write += 1
for i in range(write, len(nums)):
    nums[i] = 0
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Pascal's Triangle	Easy	LC 118	1855
2	Reshape the Matrix	Easy	LC 566	1857
3	Image Smoother	Easy	LC 661	1860
4	Plus One	Easy	LC 66	1864
5	★ **Rotate Array** (see page)	Medium	LC 189	159
6	★ **Spiral Matrix II** (see page)	Medium	LC 59	1009
7	★ **Game of Life** (see page)	Medium	LC 289	1022
8	★ **Candy Crush**	Medium	LC 723	1866
9	★ **Robot Bounded In Circle**	Medium	LC 1041	1870
10	Spiral Matrix III	Medium	LC 885	1874
11	Design a Number Container System	Medium	LC 2349	1877
12	Snakes and Ladders (see page)	Medium	LC 909	939
13	★ **Minesweeper**	Medium	LC 529	1880
14	Out of Boundary Paths (see page)	Medium	LC 576	1202
15	Diagonal Traverse II	Medium	LC 1424	1884
16	Cyclically Rotating a Grid	Medium	LC 1914	1886
17	Maximum Matrix Sum	Medium	LC 1975	1890
18	Magic Squares In Grid	Medium	LC 840	1893
19	★ **Rotating the Box**	Medium	LC 1861	1896
20	Stamping the Grid	Hard	LC 2132	1900

118. Pascal's Triangle

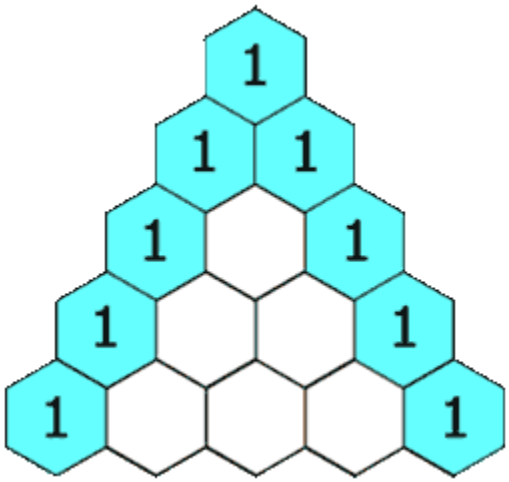
Easy · Array, Dynamic Programming

leetcode.com/problems/pascals-triangle

PROBLEM

Given an integer `numRows`, return the first numRows of **Pascal's triangle**.

In **Pascal's triangle**, each number is the sum of the two numbers directly above it as shown:



Example 1:

```
Input: numRows = 5
Output: [[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]]
```

Example 2:

```
Input: numRows = 1
Output: [[1]]
```

Constraints:

- `1 <= numRows <= 30`

SOLUTION (PYTHON 3)

Solution 1: Simulation

> **How to think about it**
>
> Row i is formed by adding adjacent entries of row i-1, with 1s on both ends. numRows≤ 30, so building row by row is enough. Each row depends only on the previous one.

We first create an answer array f, then set the first row of f to [1]. Next, starting from the second row, the first and last elements of each row are 1, and for other elements f[i][j] = f[i - 1][j - 1] + f[i - 1][j].

The time complexity is $O(n^2)$, where n is the given number of rows. Ignoring the space consumption of the answer, the space complexity is O(1).

Python 3

```
class Solution:
    def generate(self, numRows: int) -> List[List[int]]:
        f = [[1]]
        for i in range(numRows - 1):
            g = [1] + [a + b for a, b in pairwise(f[-1])] + [1]
            f.append(g)
        return f
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Only row number r using O(r) memory
# 2. How many odd numbers are in row r


# ============================================================
# FOLLOW-UP 1: Only row number r using O(r) memory
# ============================================================

# ---- follow-up 1: function ----
# Answer: Build the row from the previous one in place,
# going from right to left.
# Time: O(r^2).  Space: O(r).
def pascal_row(row_index):
    row = [1] * (row_index + 1)
    for i in range(2, row_index + 1):
        for j in range(i - 1, 0, -1):
            row[j] += row[j - 1]
    return row


# ============================================================
# FOLLOW-UP 2: How many odd numbers are in row r
# ============================================================

# ---- follow-up 2: function ----
# Answer: By Lucas' theorem, the count is 2 to the power
# of the number of one bits in r.
# Time: O(log r).  Space: O(1).
def odd_entries_in_row(row_index):
    ones = 0
    while row_index > 0:
        ones += row_index & 1
        row_index >>= 1
    return 2 ** ones
```

566. Reshape the Matrix

Easy · Array, Matrix, Simulation

leetcode.com/problems/reshape-the-matrix

PROBLEM

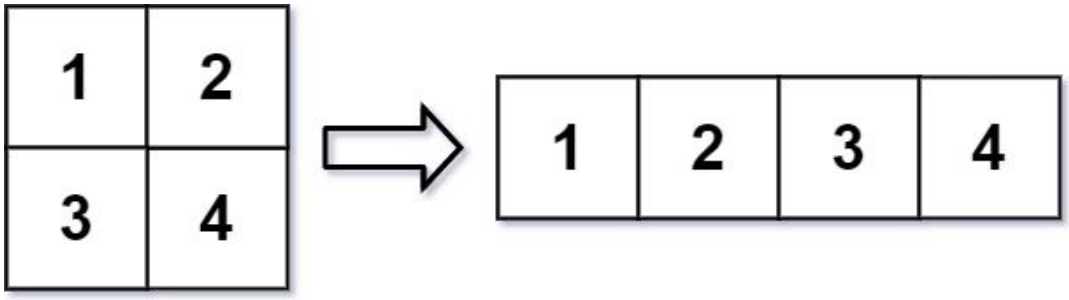
In MATLAB, there is a handy function called `reshape` which can reshape an `m x n` matrix into a new one with a different size `r x c` keeping its original data.

You are given an `m x n` matrix `mat` and two integers `r` and `c` representing the number of rows and the number of columns of the wanted reshaped matrix.

The reshaped matrix should be filled with all the elements of the original matrix in the same row-traversing order as they were.

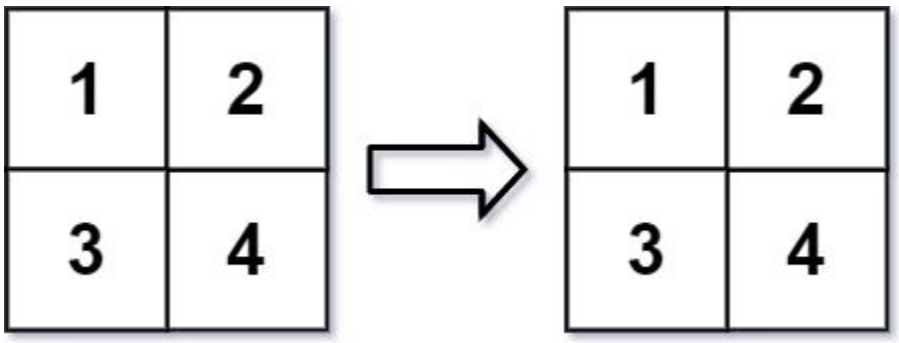
If the `reshape` operation with given parameters is possible and legal, output the new reshaped matrix; Otherwise, output the original matrix.

Example 1:




```
Input: mat = [[1,2],[3,4]], r = 1, c = 4
Output: [[1,2,3,4]]
```

Example 2:




```
Input: mat = [[1,2],[3,4]], r = 2, c = 4
Output: [[1,2],[3,4]]
```

Constraints:

- `m == mat.length`
- `n == mat[i].length`
- `1 <= m, n <= 100`
- `-1000 <= mat[i][j] <= 1000`
- `1 <= r, c <= 300`

SOLUTIONS (PYTHON 3)

Solution 1: Simulation

> **How to think about it**
>
> Reshape is a row-major relayout into r × c. If the product of dimensions differs, return the original matrix. A linear index i reads `mat[i // n][i % n]` and writes `ans[i // c][i % c]`. One pass copies every element.

First, we get the number of rows and columns of the original matrix, denoted as m and n respectively. If m × n ≠ r × c, then the matrix cannot be reshaped, and we return the original matrix directly.

Otherwise, we create a new matrix with r rows and c columns. Starting from the first element of the original matrix, we traverse all elements in row-major order and place the traversed elements into the new matrix in order.

After traversing all elements of the original matrix, we get the answer.

The time complexity is O(m × n), where m and n are the number of rows and columns of the original matrix, respectively. Ignoring the space consumption of the answer, the space complexity is O(1).

Python 3

```python
class Solution:
    def matrixReshape(self, mat: List[List[int]], r: int, c: int) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        if m * n != r * c:
            return mat
        ans = [[0] * c for _ in range(r)]
        for i in range(m * n):
            ans[i // c][i % c] = mat[i // n][i % n]
        return ans
```

Solution 2

> **How to think about it**
>
> Solution 1 already maps through a linear index. Solution 2 writes the same formula while iterating the destination, so only the loop order changes. The correspondence is identical; there is no new asymptotic gain.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```python
class Solution:
    def matrixReshape(self, mat: List[List[int]], r: int, c: int) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        if m * n != r * c:
            return mat
        ans = [[0] * c for _ in range(r)]
        for i in range(r * c):
            ans[i // c][i % c] = mat[i // n][i % n]
        return ans
```

Complexity. Time: O(r * c). Space: O(r * c).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Read and write column by column instead of row by row
# 2. Flatten a matrix of any depth into one list


# ========================================================
# FOLLOW-UP 1: Read and write column by column instead of
# row by row
# ========================================================
```

```
# ---- follow-up 1: function ----
# Answer: Walk the old matrix by columns and fill the new
# one by columns.
# Time: O(m * n).  Space: O(m * n).
def reshape_by_columns(mat, r, c):
    rows, cols = len(mat), len(mat[0])
    if rows * cols != r * c:
        return mat
    result = [[0] * c for _ in range(r)]
    for k in range(rows * cols):
        old_col, old_row = divmod(k, rows)
        new_col, new_row = divmod(k, r)
        result[new_row][new_col] = mat[old_row][old_col]
    return result


# ============================================================
# FOLLOW-UP 2: Flatten a matrix of any depth into one list
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk the nested lists and collect the numbers.
# Time: O(N * d), N = elements, d = nesting depth.  Space:
# O(N).
def flatten(items):
    result = []
    for item in items:
        if isinstance(item, list):
            result += flatten(item)
        else:
            result.append(item)
    return result
```

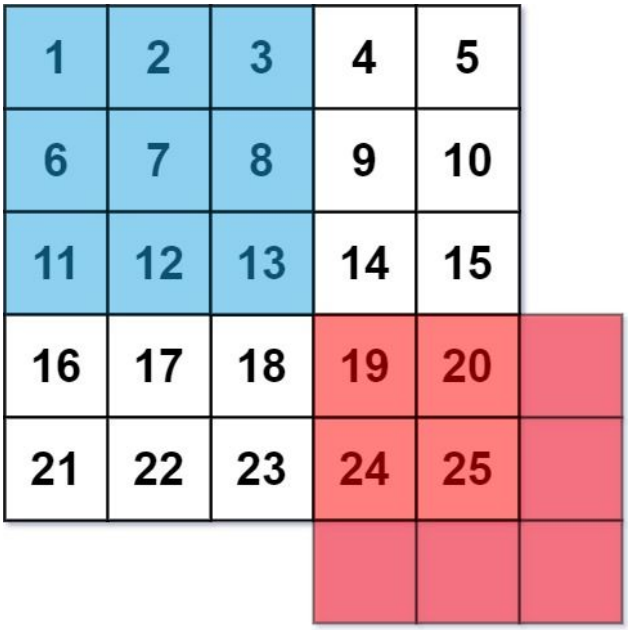
661. Image Smoother

Easy · Array, Matrix

leetcode.com/problems/image-smoother

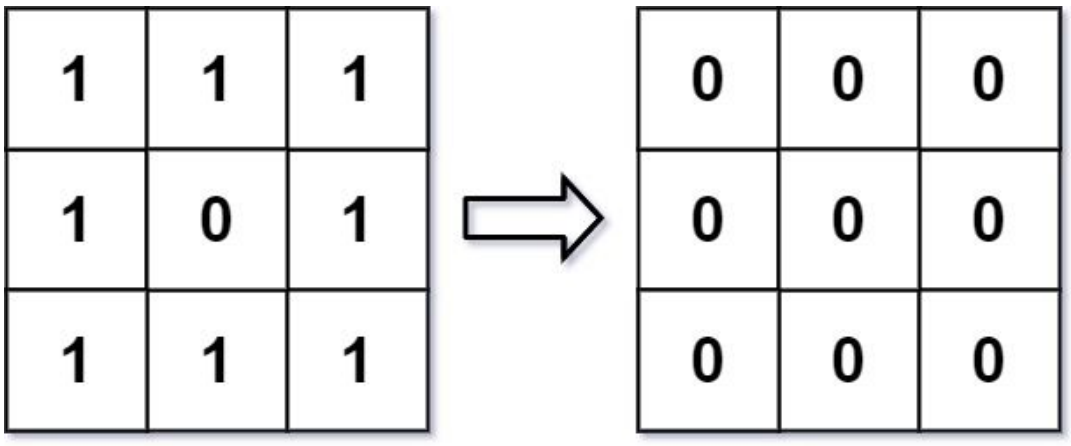
PROBLEM

An **image smoother** is a filter of the size `3 x 3` that can be applied to each cell of an image by rounding down the average of the cell and the eight surrounding cells (i.e., the average of the nine cells in the blue smoother). If one or more of the surrounding cells of a cell is not present, we do not consider it in the average (i.e., the average of the four cells in the red smoother).



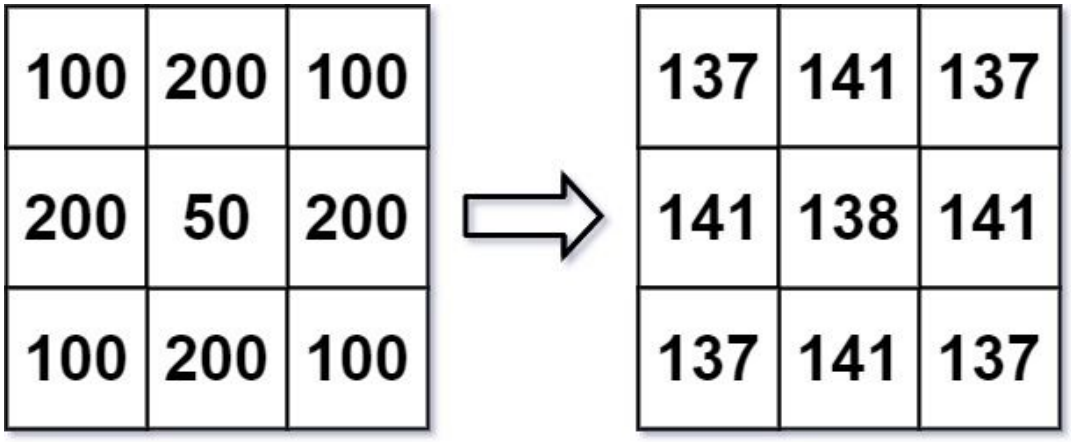
Given an `m x n` integer matrix `img` representing the grayscale of an image, return *the image after applying the smoother on each cell of it*.

Example 1:




```
Input: img = [[1,1,1],[1,0,1],[1,1,1]]
Output: [[0,0,0],[0,0,0],[0,0,0]]
Explanation:
For the points (0,0), (0,2), (2,0), (2,2): floor(3/4) = floor(0.75) = 0
For the points (0,1), (1,0), (1,2), (2,1): floor(5/6) = floor(0.83333333) = 0
For the point (1,1): floor(8/9) = floor(0.88888889) = 0
```

Example 2:




```
Input: img = [[100,200,100],[200,50,200],[100,200,100]]
Output: [[137,141,137],[141,138,141],[137,141,137]]
Explanation:
For the points (0,0), (0,2), (2,0), (2,2): floor((100+200+200+50)/4) = floor(137.5) = 137
For the points (0,1), (1,0), (1,2), (2,1): floor((200+200+50+200+100+100)/6) = floor(141.666667) = 141
For the point (1,1): floor((50+200+200+200+200+100+100+100+100)/9) = floor(138.888889) = 138
```

Constraints:

- `m == img.length`
- `n == img[i].length`
- `1 <= m, n <= 200`
- `0 <= img[i][j] <= 255`

SOLUTION (PYTHON 3)

Solution 1: Direct Traversal

How to think about it

Each cell becomes the floor-average of itself and its (up to eight) neighbors. A 200× 200 grid can be scanned directly. For (i,j) sum the in-bound cells of [i-1,i+1]×[j-1,j+1] into a new matrix so in-place writes do not affect neighbors.

We create a 2D array ans of size m × n, where ans[i][j] represents the smoothed value of the cell in the i-th row and j-th column of the image.

For ans[i][j], we traverse the cell in the i-th row and j-th column of img and its surrounding 8 cells, calculate their sum s and count cnt, then compute the average value s / cnt and store it in ans[i][j].

After the traversal, we return ans.

The time complexity is O(m × n), where m and n are the number of rows and columns of img, respectively. Ignoring the space consumption of the answer array, the space complexity is O(1).

Python 3

```python
class Solution:
    def imageSmoother(self, img: List[List[int]]) -> List[List[int]]:
        m, n = len(img), len(img[0])
        ans = [[0] * n for _ in range(m)]
        for i in range(m):
            for j in range(n):
                s = cnt = 0
                for x in range(i - 1, i + 2):
                    for y in range(j - 1, j + 2):
                        if 0 <= x < m and 0 <= y < n:
                            cnt += 1
                            s += img[x][y]
                ans[i][j] = s // cnt
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Change the image in place (no second grid)
# 2. A bigger filter: square of size (2k + 1)


# ============================================================
# FOLLOW-UP 1: Change the image in place (no second grid)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Store the old value in the low 8 bits and the
# new value in the next bits; shift at the end.
# Time: O(m * n).  Space: O(1) extra.
def smooth_in_place(img):
    rows, cols = len(img), len(img[0])
    for r in range(rows):
        for c in range(cols):
            total = 0
            count = 0
            for dr in (-1, 0, 1):
                for dc in (-1, 0, 1):
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < rows and 0 <= nc < cols:
                        total += img[nr][nc] & 255
                        count += 1
            img[r][c] |= (total // count) << 8
    for r in range(rows):
        for c in range(cols):
            img[r][c] >>= 8
    return img


# ============================================================
# FOLLOW-UP 2: A bigger filter: square of size (2k + 1)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Average over all cells within k steps in both
# directions.
# Time: O(m * n * k^2).  Space: O(m * n).
def smooth_with_radius(img, k):
    rows, cols = len(img), len(img[0])
    result = [[0] * cols for _ in range(rows)]
    for r in range(rows):
        for c in range(cols):
            total = 0
            count = 0
            for nr in range(max(0, r - k), min(rows, r + k + 1)):
```

```
                for nc in range(max(0, c - k), min(cols, c + k + 1)):
                    total += img[nr][nc]
                    count += 1
            result[r][c] = total // count
    return result
```

66. Plus One

Easy · Array, Math

leetcode.com/problems/plus-one

PROBLEM

You are given a **large integer** represented as an integer array `digits`, where each `digits[i]` is the i^{th} digit of the integer. The digits are ordered from most significant to least significant in left-to-right order. The large integer does not contain any leading `0`'s.

Increment the large integer by one and return *the resulting array of digits*.

Example 1:

```
Input: digits = [1,2,3]
Output: [1,2,4]
Explanation: The array represents the integer 123.
Incrementing by one gives 123 + 1 = 124.
Thus, the result should be [1,2,4].
```

Example 2:

```
Input: digits = [4,3,2,1]
Output: [4,3,2,2]
Explanation: The array represents the integer 4321.
Incrementing by one gives 4321 + 1 = 4322.
Thus, the result should be [4,3,2,2].
```

Example 3:

```
Input: digits = [9]
Output: [1,0]
Explanation: The array represents the integer 9.
Incrementing by one gives 9 + 1 = 10.
Thus, the result should be [1,0].
```

Constraints:

- `1 <= digits.length <= 100`
- `0 <= digits[i] <= 9`
- `digits` does not contain any leading `0`'s.

SOLUTION (PYTHON 3)

Solution 1: Simulation

How to think about it

The first idea is to join the digits into an integer and add one. n ≤ 100, so the value overflows ordinary integer types. The bottleneck is a carry that may run all the way to the most significant digit. Adding one never produces a carry other than 1: walk from the least significant digit; stop at the first digit that is not 9; only an all-9s array needs a leading 1. So we simulate from right to left and return as soon as a digit does not carry, without converting the whole number into another representation.

We start traversing from the last element of the array, add one to the current element, and then take the modulus by 10. If the result is not 0, it means that there is no carry for the current element, and we can directly return the array. Otherwise, the current element is 0 and needs to be carried over. We continue to traverse the previous element and repeat the above operation. If we still haven't returned after traversing the array, it means that all elements in the array are 0, and we need to insert a 1 at the beginning of the array.

The time complexity is O(n), where n is the length of the array. Ignoring the space consumption of the answer, the space complexity is O(1).

Python 3

```python
class Solution:
    def plusOne(self, digits: List[int]) -> List[int]:
        n = len(digits)
        for i in range(n - 1, -1, -1):
            digits[i] += 1
            digits[i] %= 10
            if digits[i] != 0:
                return digits
        return [1] + digits
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Add one to a number written in base b
# 2. Add two numbers given as digit lists


# ========================================================
# FOLLOW-UP 1: Add one to a number written in base b
# ========================================================

# ---- follow-up 1: function ----
# Answer: The same carry walk, using the base instead of
# 10.
# Time: O(n).  Space: O(n).
def plus_one_in_base(digits, base):
    digits = list(digits)
    for i in range(len(digits) - 1, -1, -1):
        if digits[i] + 1 < base:
            digits[i] += 1
            return digits
        digits[i] = 0
    return [1] + digits


# ========================================================
# FOLLOW-UP 2: Add two numbers given as digit lists
# ========================================================

# ---- follow-up 2: function ----
# Answer: Add from the right with a carry.
# Time: O(max(a, b)).  Space: O(max(a, b)).
def add_digit_lists(a, b):
    result = []
    carry = 0
    i, j = len(a) - 1, len(b) - 1
    while i >= 0 or j >= 0 or carry:
        total = carry
        if i >= 0:
            total += a[i]
            i -= 1
        if j >= 0:
            total += b[j]
            j -= 1
        result.append(total % 10)
        carry = total // 10
    return result[::-1]
```

723. Candy Crush

Medium · Array, Two Pointers, Matrix, Simulation

leetcode.com/problems/candy-crush

★ **TOP 7 PROBLEM** in Simulation / Array Rearrangement: master this one first

PROBLEM

This question is about implementing a basic elimination algorithm for Candy Crush.

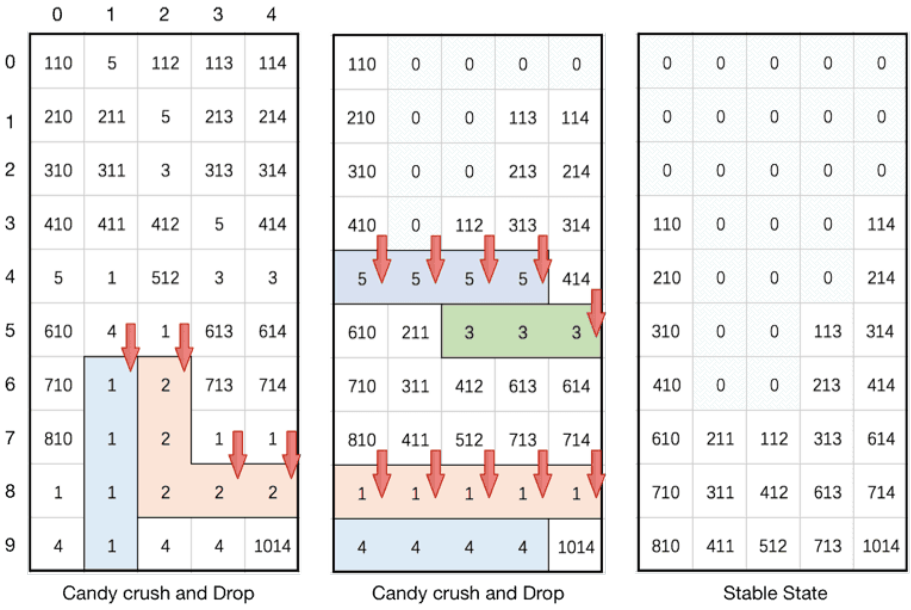
Given an `m x n` integer array `board` representing the grid of candy where `board[i][j]` represents the type of candy. A value of `board[i][j] == 0` represents that the cell is empty.

The given board represents the state of the game following the player's move. Now, you need to restore the board to a stable state by crushing candies according to the following rules:

- If three or more candies of the same type are adjacent vertically or horizontally, crush them all at the same time - these positions become empty.
- After crushing all candies simultaneously, if an empty space on the board has candies on top of itself, then these candies will drop until they hit a candy or bottom at the same time. No new candies will drop outside the top boundary.
- After the above steps, there may exist more candies that can be crushed. If so, you need to repeat the above steps.
- If there does not exist more candies that can be crushed (i.e., the board is stable), then return the current board.

You need to perform the above rules until the board becomes stable, then return *the stable board*.

Example 1:




```
Input: board = [[110,5,112,113,114],[210,211,5,213,214],[310,311,3,313,314],[410,411,412,5,414],[5,1,5
12,3,3],[610,4,1,613,614],[710,1,2,713,714],[810,1,2,1,1],[1,1,2,2,2],[4,1,4,4,1014]]
Output: [[0,0,0,0,0],[0,0,0,0,0],[0,0,0,0,0],[110,0,0,0,114],[210,0,0,0,214],[310,0,0,113,314],[410,0,
0,213,414],[610,211,112,313,614],[710,311,412,613,714],[810,411,512,713,1014]]
```

Example 2:

```
Input: board = [[1,3,5,5,2],[3,4,3,3,1],[3,2,4,5,2],[2,4,4,5,5],[1,4,4,1,1]]
Output: [[1,3,0,0,0],[3,4,0,5,2],[3,2,0,3,1],[2,4,0,5,2],[1,4,3,1,1]]
```

Constraints:

- `m == board.length`
- `n == board[i].length`
- `3 <= m, n <= 50`
- `1 <= board[i][j] <= 2000`

SOLUTION (PYTHON 3)

Solution 1: Simulation

> **How to think about it**
>
> Crush every run of three or more equal candies, drop, and repeat until stable. The board is small enough to simulate. Crushing immediately would disturb other runs that should vanish in the same step. The rule is: mark all current runs, then drop once. Mark doomed cells with a negative value (compare by absolute value) using length-3 windows on rows and columns. If anything was marked, pack positives downward in each column, fill zeros, and repeat.

We can traverse the matrix row by row and column by column to find three consecutive identical elements and mark them as negative numbers. If marking is successful, we need to move the elements in the matrix down until no elements can move down.

The time complexity is $O(m^2 \times n^2)$, where m and n are the number of rows and columns of the matrix, respectively. The space complexity is O(1).

Python 3

```python
class Solution:
    def candyCrush(self, board: List[List[int]]) -> List[List[int]]:
        m, n = len(board), len(board[0])
        run = True
        while run:
            run = False
            for i in range(m):
                for j in range(2, n):
                    if board[i][j] and abs(board[i][j]) == abs(board[i][j - 1]) == abs(
                        board[i][j - 2]
                    ):
                        run = True
                        board[i][j] = board[i][j - 1] = board[i][j - 2] = -abs(
                            board[i][j]
                        )
            for j in range(n):
                for i in range(2, m):
                    if board[i][j] and abs(board[i][j]) == abs(board[i - 1][j]) == abs(
                        board[i - 2][j]
                    ):
                        run = True
                        board[i][j] = board[i - 1][j] = board[i - 2][j] = -abs(
                            board[i][j]
                        )
            if run:
                for j in range(n):
                    k = m - 1
                    for i in range(m - 1, -1, -1):
                        if board[i][j] > 0:
                            board[k][j] = board[i][j]
                            k -= 1
                    while k >= 0:
                        board[k][j] = 0
                        k -= 1
        return board
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Count how many candies are crushed in total
# 2. Crush runs of m or more instead of 3


# ============================================================
# FOLLOW-UP 1: Count how many candies are crushed in total
# ============================================================

# ---- follow-up 1: function ----
# Answer: Run the same simulation and count the cells that
# become 0 in each round.
# Time: O((m * n)^2) in the worst case (O(m * n) per pass,
# up to m * n / 3 passes).  Space: O(m * n).
def candies_crushed(board):
    rows, cols = len(board), len(board[0])
    total = 0
    while True:
        marked = set()
        for r in range(rows):
            for c in range(cols - 2):
                if board[r][c] != 0 and board[r][c] == board[r][c + 1] == board[r][c + 2]:
                    marked |= {(r, c), (r, c + 1), (r, c + 2)}
        for c in range(cols):
            for r in range(rows - 2):
                if board[r][c] != 0 and board[r][c] == board[r + 1][c] == board[r + 2][c]:
                    marked |= {(r, c), (r + 1, c), (r + 2, c)}
        if not marked:
            return total
        total += len(marked)
        for r, c in marked:
            board[r][c] = 0
        for c in range(cols):
            column = [board[r][c] for r in range(rows) if board[r][c] != 0]
            column = [0] * (rows - len(column)) + column
            for r in range(rows):
                board[r][c] = column[r]


# ============================================================
# FOLLOW-UP 2: Crush runs of m or more instead of 3
# ============================================================

# ---- follow-up 2: function ----
# Answer: Find runs by walking each row and column and
# marking runs of length >= m.
# Time: O((m * n)^2) in the worst case.  Space: O(m * n).
def crush_runs_of(board, m):
    rows, cols = len(board), len(board[0])
    while True:
        marked = set()
        for r in range(rows):
            c = 0
            while c < cols:
                end = c
                while end + 1 < cols and board[r][end + 1] == board[r][c]:
                    end += 1
                if board[r][c] != 0 and end - c + 1 >= m:
                    for k in range(c, end + 1):
                        marked.add((r, k))
                c = end + 1
        for c in range(cols):
            r = 0
            while r < rows:
```

```
            end = r
            while end + 1 < rows and board[end + 1][c] == board[r][c]:
                end += 1
            if board[r][c] != 0 and end - r + 1 >= m:
                for k in range(r, end + 1):
                    marked.add((k, c))
            r = end + 1
    if not marked:
        return board
    for r, c in marked:
        board[r][c] = 0
    for c in range(cols):
        column = [board[r][c] for r in range(rows) if board[r][c] != 0]
        column = [0] * (rows - len(column)) + column
        for r in range(rows):
            board[r][c] = column[r]
```

1041. Robot Bounded In Circle

Medium · Math, String, Simulation

leetcode.com/problems/robot-bounded-in-circle

★ **TOP 7 PROBLEM** in Simulation / Array Rearrangement: master this one first

PROBLEM

On an infinite plane, a robot initially stands at `(0, 0)` and faces north. Note that:

- The **north direction** is the positive direction of the y-axis.
- The **south direction** is the negative direction of the y-axis.
- The **east direction** is the positive direction of the x-axis.
- The **west direction** is the negative direction of the x-axis.

The robot can receive one of three instructions:

- `"G"`: go straight 1 unit.
- `"L"`: turn 90 degrees to the left (i.e., anti-clockwise direction).
- `"R"`: turn 90 degrees to the right (i.e., clockwise direction).

The robot performs the `instructions` given in order, and repeats them forever.

Return `true` if and only if there exists a circle in the plane such that the robot never leaves the circle.

Example 1:

```
Input: instructions = "GGLLGG"
Output: true
Explanation: The robot is initially at (0, 0) facing the north direction.
"G": move one step. Position: (0, 1). Direction: North.
"G": move one step. Position: (0, 2). Direction: North.
"L": turn 90 degrees anti-clockwise. Position: (0, 2). Direction: West.
"L": turn 90 degrees anti-clockwise. Position: (0, 2). Direction: South.
"G": move one step. Position: (0, 1). Direction: South.
"G": move one step. Position: (0, 0). Direction: South.
Repeating the instructions, the robot goes into the cycle: (0, 0) --> (0, 1) --> (0, 2) --> (0, 1) --> (0, 0).
Based on that, we return true.
```

Example 2:

```
Input: instructions = "GG"
Output: false
Explanation: The robot is initially at (0, 0) facing the north direction.
"G": move one step. Position: (0, 1). Direction: North.
"G": move one step. Position: (0, 2). Direction: North.
Repeating the instructions, keeps advancing in the north direction and does not go into cycles.
Based on that, we return false.
```

Example 3:

```
Input: instructions = "GL"
Output: true
Explanation: The robot is initially at (0, 0) facing the north direction.
"G": move one step. Position: (0, 1). Direction: North.
"L": turn 90 degrees anti-clockwise. Position: (0, 1). Direction: West.
"G": move one step. Position: (-1, 1). Direction: West.
"L": turn 90 degrees anti-clockwise. Position: (-1, 1). Direction: South.
"G": move one step. Position: (-1, 0). Direction: South.
"L": turn 90 degrees anti-clockwise. Position: (-1, 0). Direction: East.
"G": move one step. Position: (0, 0). Direction: East.
"L": turn 90 degrees anti-clockwise. Position: (0, 0). Direction: North.
Repeating the instructions, the robot goes into the cycle: (0, 0) --> (0, 1) --> (-1, 1) --> (-1, 0)
--> (0, 0).
Based on that, we return true.
```

Constraints:

- `1 <= instructions.length <= 100`
- `instructions[i]` is `'G'`, `'L'` or, `'R'`.

SOLUTION (PYTHON 3)

Solution 1: Simulation

How to think about it

The program repeats forever, so we cannot simulate infinitely many steps. One cycle's net displacement and heading decide boundedness: back at the origin, or no longer facing north, later cycles stay in a loop. k∈[0,3] is the heading and dist counts steps on the four axes. Left adds one, right adds three, and a go increments the current axis. After one cycle we accept if north equals south and east equals west, or if k≠ 0.

We can simulate the robot's movement. Use a variable k to represent the robot's direction, initialized to 0, which means the robot is facing north. The variable k can take values in the range [0, 3], representing the robot facing north, west, south, and east, respectively. Additionally, we use an array dist of length 4 to record the distance the robot travels in the four directions, initialized to [0, 0, 0, 0].

Traverse the instruction string instructions. If the current instruction is `'L'`, the robot turns west, i.e., k = (k + 1) mod 4; if the instruction is `'R'`, the robot turns east, i.e., k = (k + 3) mod 4; otherwise, the robot moves one step in the current direction, i.e., dist[k]++.

If the given instruction string instructions is executed once and the robot returns to the origin, i.e., dist[0] = dist[2] and dist[1] = dist[3], then the robot will definitely enter a loop. This is because no matter how many times the instructions are repeated, the robot always returns to the origin, so it must enter a loop.

If the given instruction string instructions is executed once and the robot does not return to the origin, suppose the robot is at (x, y) and its direction is k.

- If k=0, i.e., the robot is facing north, then after executing the instructions a second time, the coordinate change is (x, y); after executing the instructions a third time, the coordinate change is still (x, y)... Accumulating these changes, the robot will eventually reach (n × x, n × y), where n is a positive integer. Since the robot does not return to the origin, i.e., x ≠ 0 or y ≠ 0, it follows that n × x ≠ 0 or n × y ≠ 0, so the robot will not enter a loop;
- If k=1, i.e., the robot is facing west, then after executing the instructions a second time, the coordinate change is (-y, x); after executing the instructions a third time, the coordinate change is (-x, -y); after executing the instructions a fourth time, the coordinate change is (y, -x). Accumulating these coordinate changes, we find that the robot will eventually return to the origin (0, 0);
- If k=2, i.e., the robot is facing south, then after executing the instructions a second time, the coordinate change is (-x, -y). Accumulating these two coordinate changes, we find that the robot will eventually return to the origin (0, 0);
- If k=3, i.e., the robot is facing east, then after executing the instructions a second time, the coordinate change is (y, -x); after executing the instructions a third time, the coordinate change is (-x, -y); after executing the instructions a fourth time, the coordinate change is (-y, x). Accumulating these coordinate changes, we find that the robot will eventually

return to the origin (0, 0).

In conclusion, if the given instruction string instructions is executed once and the robot returns to the origin, or if the robot's direction is different from the initial direction, then the robot will definitely enter a loop.

The time complexity is O(n), and the space complexity is O(1), where n is the length of the instruction string instructions.

Python 3

```
class Solution:
    def isRobotBounded(self, instructions: str) -> bool:
        k = 0
        dist = [0] * 4
        for c in instructions:
            if c == 'L':
                k = (k + 1) % 4
            elif c == 'R':
                k = (k + 3) % 4
            else:
                dist[k] += 1
        return (dist[0] == dist[2] and dist[1] == dist[3]) or k != 0
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Where is the robot after k repeats of the
# instructions
# 2. How many different squares does the robot visit in
# the first n steps


# ============================================================
# FOLLOW-UP 1: Where is the robot after k repeats of the
# instructions
# ============================================================

# ---- follow-up 1: function ----
# Answer: Simulate one pass to get the displacement and
# turn, then repeat k times (or use the 4-cycle of the
# turns).
# Time: O(k * L), L = instruction length.  Space: O(1).
def position_after_repeats(instructions, k):
    x, y = 0, 0
    dx, dy = 0, 1
    for _ in range(k):
        for ch in instructions:
            if ch == "G":
                x += dx
                y += dy
            elif ch == "L":
                dx, dy = -dy, dx
            else:
                dx, dy = dy, -dx
    return x, y


# ============================================================
# FOLLOW-UP 2: How many different squares does the robot
# visit in the first n steps
# ============================================================

# ---- follow-up 2: function ----
# Answer: Simulate and keep a set of visited squares.
# Time: O(n).  Space: O(n).
def distinct_squares_visited(instructions, n):
    x, y = 0, 0
    dx, dy = 0, 1
    seen = {(0, 0)}
```

```
    steps = 0
    while steps < n:
        for ch in instructions:
            if steps == n:
                break
            if ch == "G":
                x += dx
                y += dy
                seen.add((x, y))
            elif ch == "L":
                dx, dy = -dy, dx
            else:
                dx, dy = dy, -dx
            steps += 1
    return len(seen)
```

PATTERN 36 · SIMULATION / ARRAY REARRANGEMENT · PROBLEM 10 OF 20

885. Spiral Matrix III

Medium · Array, Matrix, Simulation

leetcode.com/problems/spiral-matrix-iii

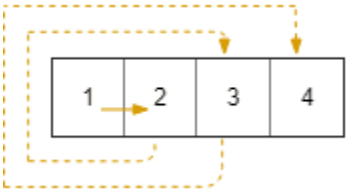
PROBLEM

You start at the cell `(rStart, cStart)` of an `rows x cols` grid facing east. The northwest corner is at the first row and column in the grid, and the southeast corner is at the last row and column.

You will walk in a clockwise spiral shape to visit every position in this grid. Whenever you move outside the grid's boundary, we continue our walk outside the grid (but may return to the grid boundary later.). Eventually, we reach all `rows * cols` spaces of the grid.

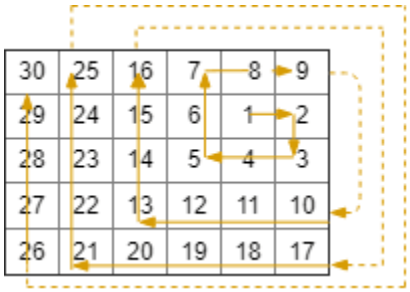
Return *an array of coordinates representing the positions of the grid in the order you visited them.*

Example 1:




```
Input: rows = 1, cols = 4, rStart = 0, cStart = 0
Output: [[0,0],[0,1],[0,2],[0,3]]
```

Example 2:




```
Input: rows = 5, cols = 6, rStart = 1, cStart = 4
Output: [[1,4],[1,5],[2,5],[2,4],[2,3],[1,3],[0,3],[0,4],[0,5],[3,5],[3,4],[3,3],[3,2],[2,2],[1,2],[0,
2],[4,5],[4,4],[4,3],[4,2],[4,1],[3,1],[2,1],[1,1],[0,1],[4,0],[3,0],[2,0],[1,0],[0,0]]
```

Constraints:

- `1 <= rows, cols <= 100`
- `0 <= rStart < rows`
- `0 <= cStart < cols`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Walk the grid in an east-south-west-north spiral from a given start. Step lengths are 1,1,2,2,3,3,…. The grid is at most 100× 100, so simulation is enough. Advance by the current direction and length, recording in-bound cells. After east and south the length grows by one, and again after west and north. Stop when rows· cols cells are collected.

Python 3

```python
class Solution:
    def spiralMatrixIII(
        self, rows: int, cols: int, rStart: int, cStart: int
    ) -> List[List[int]]:
        ans = [[rStart, cStart]]
        if rows * cols == 1:
            return ans
        k = 1
        while True:
            for dr, dc, dk in [[0, 1, k], [1, 0, k], [0, -1, k + 1], [-1, 0, k + 1]]:
                for _ in range(dk):
                    rStart += dr
                    cStart += dc
                    if 0 <= rStart < rows and 0 <= cStart < cols:
                        ans.append([rStart, cStart])
                        if len(ans) == rows * cols:
                            return ans
            k += 2
```

Complexity. Time: O(max(rows, cols)^2). Space: O(rows * cols) for the answer.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Walk counterclockwise instead
# 2. At which step is a given cell visited


# ============================================================
# FOLLOW-UP 1: Walk counterclockwise instead
# ============================================================

# ---- follow-up 1: function ----
# Answer: Turn left instead of right: the direction order
# becomes east, north, west, south.
# Time: O(max(R, C)^2).  Space: O(R * C).
def counterclockwise_spiral(rows, cols, r_start, c_start):
    directions = [(0, 1), (-1, 0), (0, -1), (1, 0)]
    result = [[r_start, c_start]]
    r, c = r_start, c_start
    step = 1
    d = 0
    while len(result) < rows * cols:
        for _ in range(2):
            dr, dc = directions[d % 4]
            for _ in range(step):
                r += dr
                c += dc
                if 0 <= r < rows and 0 <= c < cols:
                    result.append([r, c])
            d += 1
        step += 1
    return result


# ============================================================
# FOLLOW-UP 2: At which step is a given cell visited
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk the clockwise spiral and count the steps
# until the cell is reached.
# Time: O(max(R, C)^2).  Space: O(1) extra.
def step_of_cell(rows, cols, r_start, c_start, target_r, target_c):
    directions = [(0, 1), (1, 0), (0, -1), (-1, 0)]
    r, c = r_start, c_start
```

```
    visited = 1
    if (r, c) == (target_r, target_c):
        return 0
    step = 1
    d = 0
    while visited < rows * cols:
        for _ in range(2):
            dr, dc = directions[d % 4]
            for _ in range(step):
                r += dr
                c += dc
                if 0 <= r < rows and 0 <= c < cols:
                    if (r, c) == (target_r, target_c):
                        return visited
                    visited += 1
            d += 1
        step += 1
```

```
    return -1
```

2349. Design a Number Container System

Medium · Design, Hash Table, Ordered Set, Heap (Priority Queue)

leetcode.com/problems/design-a-number-container-system

PROBLEM

Design a number container system that can do the following:

- **Insert** or **Replace** a number at the given index in the system.
- **Return** the smallest index for the given number in the system.

Implement the `NumberContainers` class:

- `NumberContainers()` Initializes the number container system.
- `void change(int index, int number)` Fills the container at `index` with the `number`. If there is already a number at that `index`, replace it.
- `int find(int number)` Returns the smallest index for the given `number`, or `-1` if there is no index that is filled by `number` in the system.

Example 1:

```
Input
["NumberContainers", "find", "change", "change", "change", "change", "find", "change", "find"]
[[], [10], [2, 10], [1, 10], [3, 10], [5, 10], [10], [1, 20], [10]]
Output
[null, -1, null, null, null, null, 1, null, 2]

Explanation
NumberContainers nc = new NumberContainers();
nc.find(10); // There is no index that is filled with number 10. Therefore, we return -1.
nc.change(2, 10); // Your container at index 2 will be filled with number 10.
nc.change(1, 10); // Your container at index 1 will be filled with number 10.
nc.change(3, 10); // Your container at index 3 will be filled with number 10.
nc.change(5, 10); // Your container at index 5 will be filled with number 10.
nc.find(10); // Number 10 is at the indices 1, 2, 3, and 5. Since the smallest index that is filled
with 10 is 1, we return 1.
nc.change(1, 20); // Your container at index 1 will be filled with number 20. Note that index 1 was
filled with 10 and then replaced with 20.
nc.find(10); // Number 10 is at the indices 2, 3, and 5. The smallest index that is filled with 10 is
2. Therefore, we return 2.
```

Constraints:

- $1 \le index, number \le 10^9$
- At most 10^5 calls will be made **in total** to `change` and `find`.

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Ordered Set

> **How to think about it**
>
> We change the number at an index and query the smallest index of a number. Up to 10^5 operations rule out linear scans. Map d stores the value at each index; a sorted set g[number] stores its indices. A change removes the old index and inserts the new one; find reads the first element of the set.

We use a hash table d to record the mapping relationship between indices and numbers, and another hash table g to record the set of indices corresponding to each number. Here, we can use an ordered set to store the indices, which allows us to conveniently find the smallest index.

When calling the `change` method, we first check if the index already exists. If it does, we remove the original number from its corresponding index set and then add the new number to the corresponding index set. The time complexity is O(log n).

When calling the `find` method, we simply return the first element of the index set corresponding to the number. The time complexity is O(1).

The space complexity is O(n), where n is the number of numbers.

Python 3

```
class NumberContainers:

    def __init__(self):
        self.d = {}
        self.g = defaultdict(SortedSet)

    def change(self, index: int, number: int) -> None:
        if index in self.d:
            old_number = self.d[index]
            self.g[old_number].remove(index)
        self.d[index] = number
        self.g[number].add(index)

    def find(self, number: int) -> int:
        ids = self.g[number]
        return ids[0] if ids else -1


# Your NumberContainers object will be instantiated and called as such:
# obj = NumberContainers()
# obj.change(index,number)
# param_2 = obj.find(number)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Remove the number at an index
# 2. Return the k smallest indices holding a number


# ============================================================
# FOLLOW-UP 1: Remove the number at an index
# ============================================================

# ---- follow-up 1: function ----
# Answer: Delete the index from the number's set; the next
# change can then use that index again.
# Time: change O(log n), remove O(1), find O(log n)
# amortised.  Space: O(n).
import heapq


class NumberContainersWithRemove:
    def __init__(self):
        self.number_at = {}
        self.heaps = {}

    def change(self, index, number):
        self.number_at[index] = number
        heapq.heappush(self.heaps.setdefault(number, []), index)

    def remove(self, index):
        if index in self.number_at:
            del self.number_at[index]

    def find(self, number):
        heap = self.heaps.get(number, [])
```

```
        while heap and self.number_at.get(heap[0]) != number:
            heapq.heappop(heap)
        return heap[0] if heap else -1


# ============================================================
# FOLLOW-UP 2: Return the k smallest indices holding a
# number
# ============================================================

# ---- follow-up 2: function ----
# Answer: Pop from the cleaned heap k times and push the
# good ones back.
# Time: O(k log n) amortised.  Space: O(n).
def smallest_k_indices(container, number, k):
    heap = container.heaps.get(number, [])
    taken = []
    while heap and len(taken) < k:
        index = heapq.heappop(heap)
        if container.number_at.get(index) == number and (not taken or taken[-1] != index):
            taken.append(index)
    for index in taken:
        heapq.heappush(heap, index)
    return taken
```

529. Minesweeper

Medium · Depth-First Search, Breadth-First Search, Array, Matrix

leetcode.com/problems/minesweeper

★ **TOP 7 PROBLEM** in Simulation / Array Rearrangement: master this one first

PROBLEM

Let's play the minesweeper game (Wikipedia, online game)!

You are given an `m x n` char matrix `board` representing the game board where:

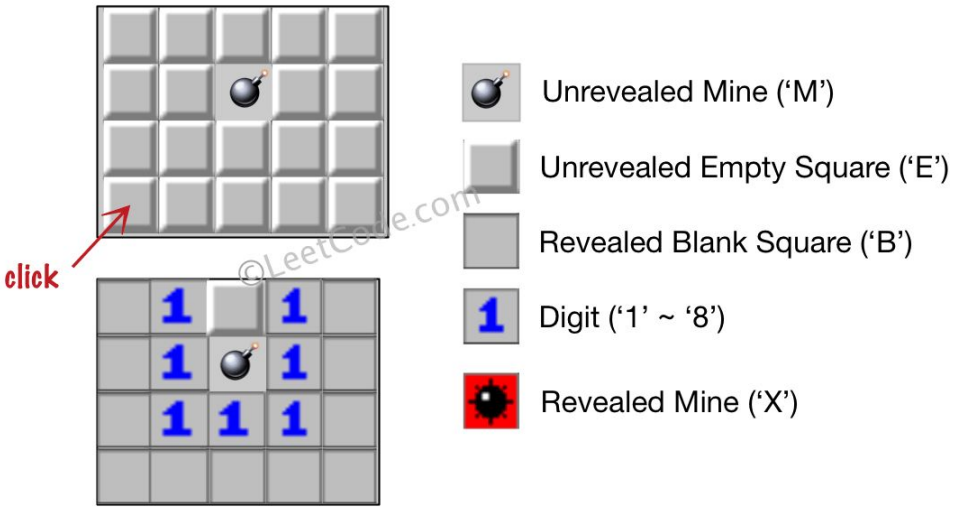
- `'M'` represents an unrevealed mine,
- `'E'` represents an unrevealed empty square,
- `'B'` represents a revealed blank square that has no adjacent mines (i.e., above, below, left, right, and all 4 diagonals),
- digit (`'1'` to `'8'`) represents how many mines are adjacent to this revealed square, and
- `'X'` represents a revealed mine.

You are also given an integer array `click` where `click = [click_r, click_c]` represents the next click position among all the unrevealed squares (`'M'` or `'E'`).

Return *the board after revealing this position according to the following rules*:

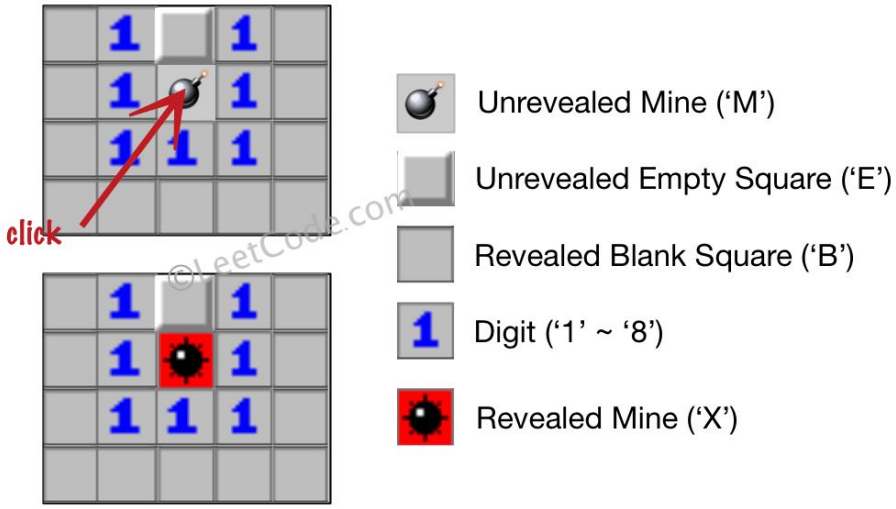
1. If a mine `'M'` is revealed, then the game is over. You should change it to `'X'`.
2. If an empty square `'E'` with no adjacent mines is revealed, then change it to a revealed blank `'B'` and all of its adjacent unrevealed squares should be revealed recursively.
3. If an empty square `'E'` with at least one adjacent mine is revealed, then change it to a digit (`'1'` to `'8'`) representing the number of adjacent mines.
4. Return the board when no more squares will be revealed.

Example 1:




```
Input: board =
[["E","E","E","E","E"],["E","E","M","E","E"],["E","E","E","E","E"],["E","E","E","E","E"]], click =
[3,0]
Output: [["B","1","E","1","B"],["B","1","M","1","B"],["B","1","1","1","B"],["B","B","B","B","B"]]
```

Example 2:




```
Input: board =
[["B","1","E","1","B"],["B","1","M","1","B"],["B","1","1","1","B"],["B","B","B","B","B"]], click =
[1,2]
Output: [["B","1","E","1","B"],["B","1","X","1","B"],["B","1","1","1","B"],["B","B","B","B","B"]]
```

Constraints:

- `m == board.length`
- `n == board[i].length`
- `1 <= m, n <= 50`
- `board[i][j]` is either `'M'`, `'E'`, `'B'`, or a digit from `'1'` to `'8'`.
- `click.length == 2`
- $0 \le click_r < m$
- $0 \le click_c < n$
- $board[click_r][click_c]$ is either `'M'` or `'E'`.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

A click on a mine becomes `X`. Otherwise count adjacent mines: write the count, or write `B` and flood neighboring empties. Only the click region needs to be revealed. If the cell is `M`, mark `X`. Otherwise DFS: count neighbors, write the digit and stop if any mine, else write `B` and recurse on adjacent `E` cells. Each cell expands at most once.

Python 3

```
class Solution:
    def updateBoard(self, board: List[List[str]], click: List[int]) -> List[List[str]]:
        def dfs(i: int, j: int):
            cnt = 0
            for x in range(i - 1, i + 2):
                for y in range(j - 1, j + 2):
                    if 0 <= x < m and 0 <= y < n and board[x][y] == "M":
                        cnt += 1
            if cnt:
                board[i][j] = str(cnt)
            else:
                board[i][j] = "B"
                for x in range(i - 1, i + 2):
                    for y in range(j - 1, j + 2):
                        if 0 <= x < m and 0 <= y < n and board[x][y] == "E":
                            dfs(x, y)

        m, n = len(board), len(board[0])
        i, j = click
        if board[i][j] == "M":
            board[i][j] = "X"
        else:
            dfs(i, j)
        return board
```

Complexity. Time: O(m * n). Space: O(m * n) recursion in the worst case.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Build the finished board of numbers from the mine
# positions
# 2. Has the player won (every square without a mine is
# revealed)


# ============================================================
# FOLLOW-UP 1: Build the finished board of numbers from
# the mine positions
# ============================================================

# ---- follow-up 1: function ----
# Answer: For every empty square count the mines around
# it.
# Time: O(m * n * 8).  Space: O(m * n).
def number_board(board):
    rows, cols = len(board), len(board[0])
    result = [row[:] for row in board]
    for r in range(rows):
        for c in range(cols):
            if board[r][c] == "M":
                continue
            count = 0
            for dr in (-1, 0, 1):
                for dc in (-1, 0, 1):
                    nr, nc = r + dr, c + dc
                    if (dr or dc) and 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] == "M":
                        count += 1
            result[r][c] = str(count) if count else "B"
    return result


# ============================================================
# FOLLOW-UP 2: Has the player won (every square without a
# mine is revealed)
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Check that no unrevealed 'E' remains and no mine
# is revealed.
# Time: O(m * n).  Space: O(1).
def has_won(board):
    for row in board:
        for cell in row:
            if cell == "E" or cell == "X":
                return False
    return True
```

1424. Diagonal Traverse II

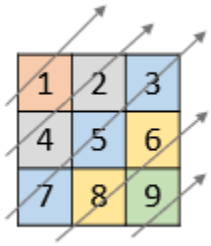
Medium · Array, Sorting, Heap (Priority Queue)

leetcode.com/problems/diagonal-traverse-ii

PROBLEM

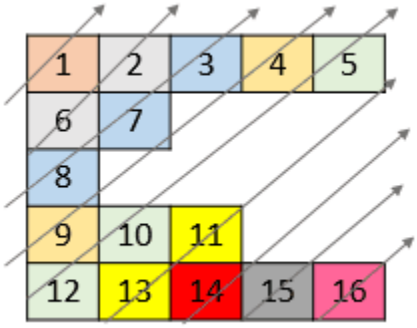
Given a 2D integer array `nums`, return *all elements of* `nums` *in diagonal order as shown in the below images*.

Example 1:




```
Input: nums = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,4,2,7,5,3,8,6,9]
```

Example 2:




```
Input: nums = [[1,2,3,4,5],[6,7],[8],[9,10,11],[12,13,14,15,16]]
Output: [1,6,2,8,7,3,9,4,12,10,5,13,11,14,15,16]
```

Constraints:

- `1 <= nums.length <= 10^5`
- `1 <= nums[i].length <= 10^5`
- `1 <= sum(nums[i].length) <= 10^5`
- `1 <= nums[i][j] <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: Sorting

How to think about it

Rows have unequal lengths and there are up to 10^5 entries, so walking diagonals with raw indices is awkward. Entries on one diagonal share i+j; diagonals increase by i+j, and within a diagonal j increases. Store triples (i+j,j,v), sort, and emit the values.

We observe that:

- The value of i + j is the same for each diagonal;
- The value of i + j for the next diagonal is greater than that of the previous diagonal;
- Within the same diagonal, the value of i + j is the same, and the value of j increases from small to large.

Therefore, we store all numbers in the form of (i, j, nums[i][j]) into arr, and then sort according to the first two items. Finally, return the array composed of the values at index 2 of all elements in arr.

The time complexity is O(n × log n), where n is the number of elements in the array nums. The space complexity is O(n).

Python 3

```
class Solution:
    def findDiagonalOrder(self, nums: List[List[int]]) -> List[int]:
        arr = []
        for i, row in enumerate(nums):
            for j, v in enumerate(row):
                arr.append((i + j, j, v))
        arr.sort()
        return [v[2] for v in arr]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Group the numbers by the diagonals that run down and
# to the right (same row minus column), right-most
# diagonal first
# 2. The largest number on every diagonal (r + c is the
# same)


# ============================================================
# FOLLOW-UP 1: Group the numbers by the diagonals that run
# down and to the right (same row minus column), right-
# most diagonal first
# ============================================================

# ---- follow-up 1: function ----
# Answer: Group by the difference row - column and read
# each group from the top.
# Time: O(N + D log D), D = diagonals.  Space: O(N).
def traverse_by_difference(nums):
    groups = {}
    for r in range(len(nums)):
        for c in range(len(nums[r])):
            groups.setdefault(r - c, []).append(nums[r][c])
    result = []
    for difference in sorted(groups):
        result += groups[difference]
    return result


# ============================================================
# FOLLOW-UP 2: The largest number on every diagonal (r + c
# is the same)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the maximum per r + c.
# Time: O(N + D log D).  Space: O(D).
def max_of_each_diagonal(nums):
    best = {}
    for r in range(len(nums)):
        for c in range(len(nums[r])):
            if r + c not in best or nums[r][c] > best[r + c]:
                best[r + c] = nums[r][c]
    return [best[k] for k in sorted(best)]
```

1914. Cyclically Rotating a Grid

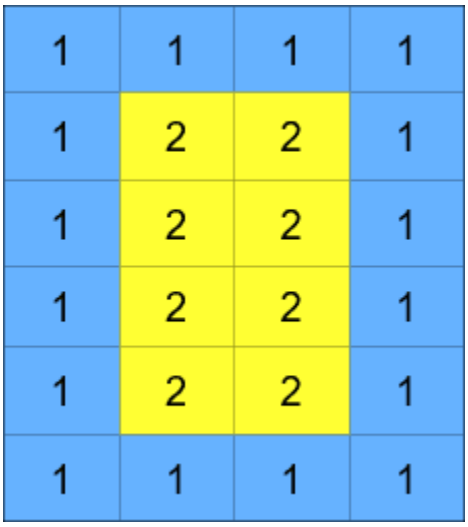
Medium · Array, Matrix, Simulation

leetcode.com/problems/cyclically-rotating-a-grid

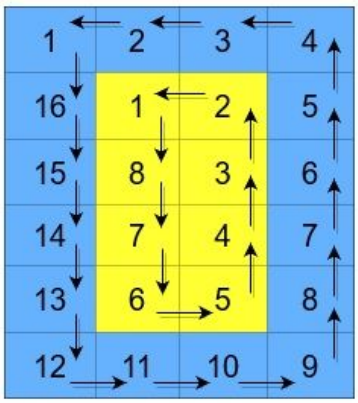
PROBLEM

You are given an `m x n` integer matrix `grid`, where `m` and `n` are both **even** integers, and an integer `k`.

The matrix is composed of several layers, which is shown in the below image, where each color is its own layer:



A cyclic rotation of the matrix is done by cyclically rotating **each layer** in the matrix. To cyclically rotate a layer once, each element in the layer will take the place of the adjacent element in the **counter-clockwise** direction. An example rotation is shown below:

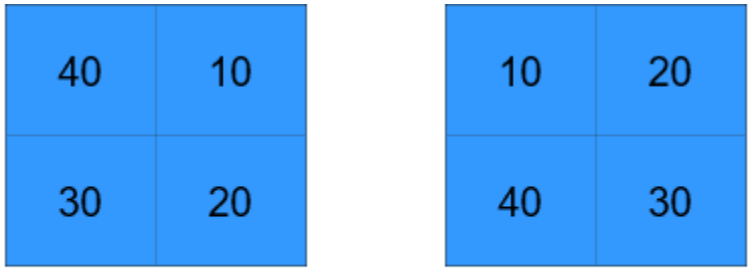


Before rotation

After rotation with k = 1

Return *the matrix after applying* `k` *cyclic rotations to it*.

Example 1:



Before Any Rotations **After One Rotation**

```
Input: grid = [[40,10],[30,20]], k = 1
Output: [[10,20],[40,30]]
Explanation: The figures above represent the grid at every state.
```

Example 2:

```
Input: grid = [[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,16]], k = 2
Output: [[3,4,8,12],[2,11,10,16],[1,7,6,15],[5,9,13,14]]
Explanation: The figures above represent the grid at every state.
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `2 <= m, n <= 50`
- Both `m` and `n` are **even** integers.
- `1 <= grid[i][j] <= 5000`
- `1 <= k <= 10^9`

SOLUTION (PYTHON 3)

Solution 1: Layer-by-Layer Simulation

How to think about it

Layers are disjoint cycles and k may exceed a cycle length, so stepping cell by cell wastes work. Flatten each layer clockwise, reduce k modulo its length, then write back. Collect top, right, bottom, and left in that order and restore in the same order. Layers are independent, so the total time is linear in the grid size.

First, we compute the number of layers in the matrix, denoted by p, and then simulate the cyclic rotation layer by layer from the outside to the inside.

For each layer, we traverse clockwise and append the elements on the top, right, bottom, and left edges to an array nums in order. Let the length of nums be l. Next, we take k mod l. Then, starting from index k in the array, we write the elements back to the matrix along the top, right, bottom, and left edges in order.

The time complexity is O(m × n), and the space complexity is O(m + n), where m and n are the number of rows and columns of the matrix, respectively.

Python 3

```python
class Solution:
    def rotateGrid(self, grid: List[List[int]], k: int) -> List[List[int]]:
        def rotate(p: int, k: int):
            nums = []
            for j in range(p, n - p - 1):
                nums.append(grid[p][j])
            for i in range(p, m - p - 1):
                nums.append(grid[i][n - p - 1])
            for j in range(n - p - 1, p, -1):
                nums.append(grid[m - p - 1][j])
            for i in range(m - p - 1, p, -1):
                nums.append(grid[i][p])
            k %= len(nums)
            if k == 0:
                return
            nums = nums[k:] + nums[:k]
            k = 0
            for j in range(p, n - p - 1):
                grid[p][j] = nums[k]
                k += 1
            for i in range(p, m - p - 1):
                grid[i][n - p - 1] = nums[k]
                k += 1
            for j in range(n - p - 1, p, -1):
                grid[m - p - 1][j] = nums[k]
                k += 1
            for i in range(m - p - 1, p, -1):
                grid[i][p] = nums[k]
                k += 1

        m, n = len(grid), len(grid[0])
        for p in range(min(m, n) >> 1):
            rotate(p, k)
        return grid
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Rotate clockwise instead
# 2. The sum of every layer


# ============================================================
# FOLLOW-UP 1: Rotate clockwise instead
# ============================================================

# ---- follow-up 1: function ----
# Answer: Rotate every layer the other way: use a negative
# shift.
# Time: O(m * n).  Space: O(m + n) per layer.
def rotate_grid_clockwise(grid, k):
    rows, cols = len(grid), len(grid[0])
    layers = min(rows, cols) // 2
    for layer in range(layers):
        top, left = layer, layer
        bottom, right = rows - 1 - layer, cols - 1 - layer
        cells = []
        for c in range(left, right + 1):
            cells.append((top, c))
        for r in range(top + 1, bottom + 1):
            cells.append((r, right))
        for c in range(right - 1, left - 1, -1):
            cells.append((bottom, c))
        for r in range(bottom - 1, top, -1):
            cells.append((r, left))
```

```
        values = [grid[r][c] for r, c in cells]
        shift = k % len(cells)
        for i in range(len(cells)):
            r, c = cells[(i + shift) % len(cells)]
            grid[r][c] = values[i]
    return grid


# ============================================================
# FOLLOW-UP 2: The sum of every layer
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk the border cells of each layer and add
# them.
# Time: O(m * n).  Space: O(m + n) per layer.
def layer_sums(grid):
    rows, cols = len(grid), len(grid[0])
    sums = []
    for layer in range((min(rows, cols) + 1) // 2):
        top, left = layer, layer
        bottom, right = rows - 1 - layer, cols - 1 - layer
        seen = set()
        for c in range(left, right + 1):
            seen.add((top, c))
            seen.add((bottom, c))
        for r in range(top, bottom + 1):
            seen.add((r, left))
            seen.add((r, right))
        sums.append(sum(grid[r][c] for r, c in seen))
    return sums
```

1975. Maximum Matrix Sum

Medium · Greedy, Array, Matrix

leetcode.com/problems/maximum-matrix-sum

PROBLEM

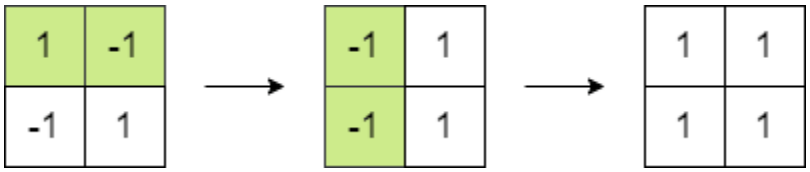
You are given an `n x n` integer `matrix`. You can do the following operation **any** number of times:

- Choose any two **adjacent** elements of `matrix` and **multiply** each of them by `-1`.

Two elements are considered **adjacent** if and only if they share a **border**.

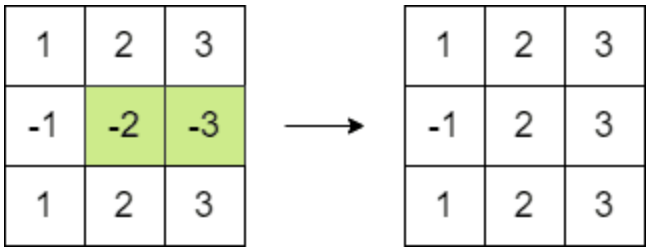
Your goal is to **maximize** the summation of the matrix's elements. Return *the **maximum** sum of the matrix's elements using the operation mentioned above.*

Example 1:




```
Input: matrix = [[1,-1],[-1,1]]
Output: 4
Explanation: We can follow the following steps to reach sum equals 4:
- Multiply the 2 elements in the first row by -1.
- Multiply the 2 elements in the first column by -1.
```

Example 2:




```
Input: matrix = [[1,2,3],[-1,-2,-3],[1,2,3]]
Output: 16
Explanation: We can follow the following step to reach sum equals 16:
- Multiply the 2 last elements in the second row by -1.
```

Constraints:

- `n == matrix.length == matrix[i].length`
- `2 <= n <= 250`
- `-10^5 <= matrix[i][j] <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: Greedy

How to think about it

Flipping two adjacent signs moves a minus around the grid. A zero or an even number of minuses can be cleared; an odd count leaves exactly one. The answer is the sum of absolute values, minus twice the smallest absolute value when the minus count is odd.

If there is a zero in the matrix, or the number of negative numbers in the matrix is even, then the maximum sum is the sum of the absolute values of all elements in the matrix.

Otherwise, if there are an odd number of negative numbers in the matrix, there will be one negative number left in the end. We choose the number with the smallest absolute value and make it negative, so that the final sum is maximized.

The time complexity is O(m × n), where m and n are the number of rows and columns of the matrix, respectively. The space complexity is O(1).

Python 3

```python
class Solution:
    def maxMatrixSum(self, matrix: List[List[int]]) -> int:
        mi = inf
        s = cnt = 0
        for row in matrix:
            for x in row:
                cnt += x < 0
                y = abs(x)
                mi = min(mi, y)
                s += y
        return s if cnt % 2 == 0 else s - mi * 2
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Only neighbours in the same row can be flipped
# together
# 2. Which cell stays negative in the best answer


# ============================================================
# FOLLOW-UP 1: Only neighbours in the same row can be
# flipped together
# ============================================================

# ---- follow-up 1: function ----
# Answer: Rows become independent: in every row, flip so
# that all numbers are positive if the number of negatives
# is even, otherwise one number with the smallest absolute
# value stays negative.
# Time: O(m * n).  Space: O(1).
def max_sum_row_flips(matrix):
    total = 0
    for row in matrix:
        negatives = 0
        smallest = float("inf")
        for x in row:
            total += abs(x)
            if x < 0:
                negatives += 1
            smallest = min(smallest, abs(x))
        if negatives % 2 == 1:
            total -= 2 * smallest
    return total


# ============================================================
# FOLLOW-UP 2: Which cell stays negative in the best
# answer
# ============================================================

# ---- follow-up 2: function ----
# Answer: If the count of negatives is odd, it is the cell
# with the smallest absolute value; otherwise none.
# Time: O(m * n).  Space: O(1).
def cell_left_negative(matrix):
```

```
    negatives = 0
    best = None
    for r in range(len(matrix)):
        for c in range(len(matrix[r])):
            if matrix[r][c] < 0:
                negatives += 1
            if best is None or abs(matrix[r][c]) < abs(matrix[best[0]][best[1]]):
                best = (r, c)
    if negatives % 2 == 0:
        return None
    return best
```

840. Magic Squares In Grid

Medium · Array, Hash Table, Math, Matrix

leetcode.com/problems/magic-squares-in-grid

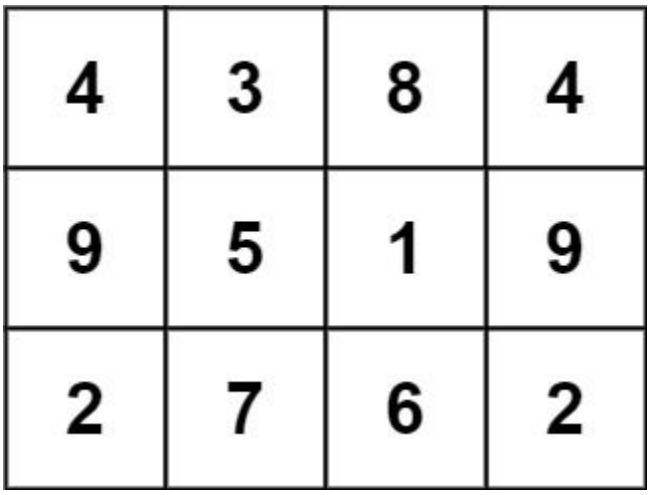
PROBLEM

A `3 x 3` **magic square** is a `3 x 3` grid filled with distinct numbers **from** 1 **to** 9 such that each row, column, and both diagonals all have the same sum.

Given a `row x col grid` of integers, how many `3 x 3` magic square subgrids are there?

Note: while a magic square can only contain numbers from 1 to 9, `grid` may contain numbers up to 15.

Example 1:




```
Input: grid = [[4,3,8,4],[9,5,1,9],[2,7,6,2]]
Output: 1
Explanation:
The following subgrid is a 3 x 3 magic square:

while this one is not:

In total, there is only one magic square inside the given grid.
```

Example 2:

```
Input: grid = [[8]]
Output: 0
```

Constraints:

- `row == grid.length`
- `col == grid[i].length`
- `1 <= row, col <= 10`
- `0 <= grid[i][j] <= 15`

SOLUTION (PYTHON 3)

Solution 1: Enumeration

> **How to think about it**
>
> A magic square uses 1… 9 once each and equal line sums. The grid is at most 10× 10, so every 3× 3 window can be checked. Collect the set and the row, column, and diagonal sums. Duplicates, out-of-range values, or unequal sums reject the window. Count the valid ones.

We directly enumerate the top-left coordinates (i, j) of each 3 × 3 submatrix, then check whether the submatrix satisfies the "magic square" property. If so, we increment the answer by one. After the enumeration, we return the answer.

The time complexity is O(m × n), where m and n are the number of rows and columns of the matrix, respectively. The space complexity is O(1).

Python 3

```python
class Solution:
    def numMagicSquaresInside(self, grid: List[List[int]]) -> int:
        def check(i: int, j: int) -> int:
            if i + 3 > m or j + 3 > n:
                return 0
            s = set()
            row = [0] * 3
            col = [0] * 3
            a = b = 0
            for x in range(i, i + 3):
                for y in range(j, j + 3):
                    v = grid[x][y]
                    if v < 1 or v > 9:
                        return 0
                    s.add(v)
                    row[x - i] += v
                    col[y - j] += v
                    if x - i == y - j:
                        a += v
                    if x - i == 2 - (y - j):
                        b += v
            if len(s) != 9 or a != b:
                return 0
            if any(x != a for x in row) or any(x != a for x in col):
                return 0
            return 1

        m, n = len(grid), len(grid[0])
        return sum(check(i, j) for i in range(m) for j in range(n))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Squares of any size k
# 2. Return the top left corners of the 3 by 3 magic
# squares


# ========================================================
# FOLLOW-UP 1: Squares of any size k
# ========================================================

# ---- follow-up 1: function ----
# Answer: Check every k by k window: all rows, columns and
# both diagonals share the same sum and the numbers are 1
# to k * k, each once.
# Time: O((R - k + 1) * (C - k + 1) * k^2 log k).  Space:
```

```
# O(k^2).
def count_magic_squares_of_size(grid, k):
    rows, cols = len(grid), len(grid[0])
    count = 0
    for r in range(rows - k + 1):
        for c in range(cols - k + 1):
            numbers = sorted(grid[r + i][c + j] for i in range(k) for j in range(k))
            if numbers != list(range(1, k * k + 1)):
                continue
            target = sum(grid[r][c:c + k])
            ok = True
            for i in range(k):
                if sum(grid[r + i][c:c + k]) != target:
                    ok = False
                if sum(grid[r + j][c + i] for j in range(k)) != target:
                    ok = False
            if sum(grid[r + i][c + i] for i in range(k)) != target:
                ok = False
            if sum(grid[r + i][c + k - 1 - i] for i in range(k)) != target:
                ok = False
            if ok:
                count += 1
    return count


# ============================================================
# FOLLOW-UP 2: Return the top left corners of the 3 by 3
# magic squares
# ============================================================

# ---- follow-up 2: function ----
# Answer: Check every 3 by 3 window and collect its
# corner.
# Time: O(R * C).  Space: O(1) extra besides the answer.
def magic_square_corners(grid):
    rows, cols = len(grid), len(grid[0])
    corners = []
    for r in range(rows - 2):
        for c in range(cols - 2):
            cells = [grid[r + i][c + j] for i in range(3) for j in range(3)]
            if sorted(cells) != list(range(1, 10)):
                continue
            lines = [sum(grid[r + i][c:c + 3]) for i in range(3)]
            lines += [sum(grid[r + i][c + j] for i in range(3)) for j in range(3)]
            lines.append(grid[r][c] + grid[r + 1][c + 1] + grid[r + 2][c + 2])
```

```
            lines.append(grid[r][c + 2] + grid[r + 1][c + 1] + grid[r + 2][c])
            if len(set(lines)) == 1:
                corners.append((r, c))
    return corners
```

1861. Rotating the Box

Medium · Array, Two Pointers, Matrix

leetcode.com/problems/rotating-the-box

★ **TOP 7 PROBLEM** in Simulation / Array Rearrangement: master this one first

PROBLEM

You are given an `m x n` matrix of characters `boxGrid` representing a side-view of a box. Each cell of the box is one of the following:

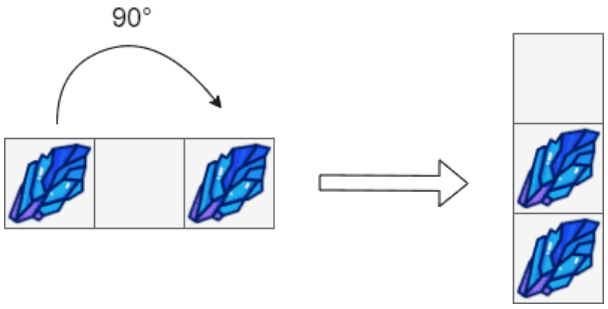
- A stone `'#'`
- A stationary obstacle `'*'`
- Empty `'.'`

The box is rotated **90 degrees clockwise**, causing some of the stones to fall due to gravity. Each stone falls down until it lands on an obstacle, another stone, or the bottom of the box. Gravity **does not** affect the obstacles' positions, and the inertia from the box's rotation **does not** affect the stones' horizontal positions.

It is **guaranteed** that each stone in `boxGrid` rests on an obstacle, another stone, or the bottom of the box.

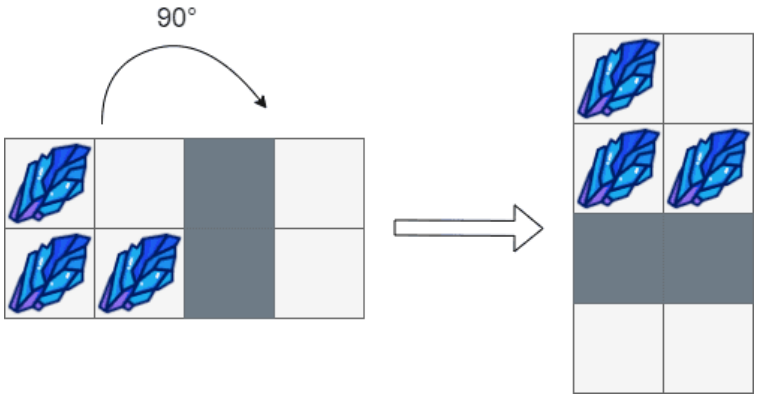
Return *an* `n x m` *matrix representing the box after the rotation described above*.

Example 1:



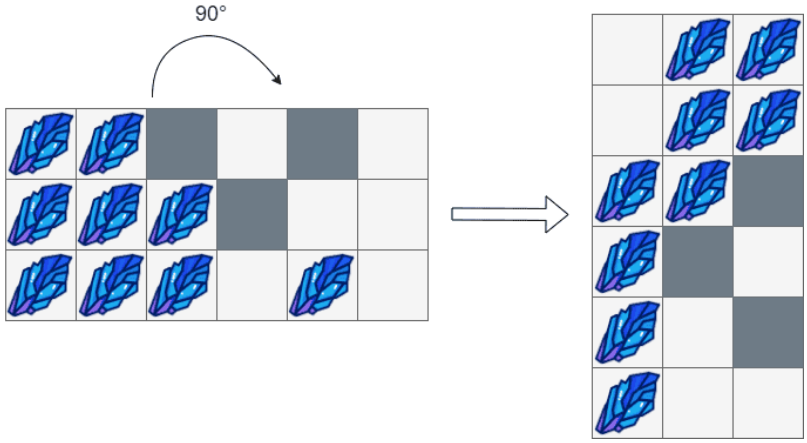

```
Input: boxGrid = [["#",".","#"]]
Output: [["."],
         ["#"],
         ["#"]]
```

Example 2:



```
Input: boxGrid = [["#",".","*","."],
                  ["#","#","*","."]]
Output: [["#","."],
         ["#","#"],
         ["*","*"],
         [".","."]]
```

Example 3:




```
Input: boxGrid = [["#","#","*",".","*","."],
                  ["#","#","#","*",".","."],
                  ["#","#","#",".","#","."]]
Output: [[".","#","#"],
         [".","#","#"],
         ["#","#","*"],
         ["#","*","."],
         ["#",".","*"],
         ["#",".","."]]
```

Constraints:

- `m == boxGrid.length`
- `n == boxGrid[i].length`
- `1 <= m, n <= 500`
- `boxGrid[i][j]` is either `'#'`, `'*'`, or `'.'`.

SOLUTION (PYTHON 3)

Solution 1: Queue Simulation

> **How to think about it**
>
> Rotate 90° clockwise, then let stones fall until they hit an obstacle or the floor. Moving each stone step by step can revisit the same cells. After the rotation, scan each column from the bottom: enqueue empty rows, swap a stone with the first empty slot, and clear the queue at an obstacle. Each cell is handled once and stones settle in the lowest reachable empties.

We first rotate the matrix 90 degrees clockwise, then simulate the falling process of stones in each column.

Specifically, we use a queue q to store the row indices of empty positions in the current column. When traversing each column, we scan from bottom to top. If we encounter a stone, we drop it to the first empty position in q, remove that empty position from q, and add the current position's row index to q since it becomes empty. If we encounter an obstacle, we clear q because stones cannot pass through obstacles. If we encounter an empty position, we add its row index to q.

The time complexity is $O(m \times n)$, and the space complexity is $O(m \times n)$, where m and n are the number of rows and columns of the matrix respectively.

Python 3

```python
class Solution:
    def rotateTheBox(self, boxGrid: List[List[str]]) -> List[List[str]]:
        m, n = len(boxGrid), len(boxGrid[0])
        ans = [[None] * m for _ in range(n)]
        for i in range(m):
            for j in range(n):
                ans[j][m - i - 1] = boxGrid[i][j]
        for j in range(m):
            q = deque()
            for i in range(n - 1, -1, -1):
                if ans[i][j] == "*":
                    q.clear()
                elif ans[i][j] == ".":
                    q.append(i)
                elif q:
                    ans[q.popleft()][j] = "#"
                    ans[i][j] = "."
                    q.append(i)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Rotate counterclockwise instead (gravity pulls down
# in the rotated box)
# 2. How many stones end up in the last column after
# falling to the right


# ============================================================
# FOLLOW-UP 1: Rotate counterclockwise instead (gravity
# pulls down in the rotated box)
# ============================================================

# ---- follow-up 1: function ----
# Answer: After a counterclockwise turn the old left side
# is the bottom, so the stones fall toward the left in the
# old rows; then rotate.
# Time: O(m * n).  Space: O(m * n).
def rotate_box_counterclockwise(box):
    rows, cols = len(box), len(box[0])
    for row in box:
        free = 0
        for c in range(cols):
            if row[c] == "*":
                free = c + 1
            elif row[c] == "#":
                row[c] = "."
                row[free] = "#"
                free += 1
    return [[box[r][cols - 1 - c] for r in range(rows)] for c in range(cols)]


# ============================================================
# FOLLOW-UP 2: How many stones end up in the last column
# after falling to the right
# ============================================================

# ---- follow-up 2: function ----
# Answer: Let the stones fall to the right in every row
# and count the stones in the last column.
# Time: O(m * n).  Space: O(1).
def stones_touching_right_wall(box):
    count = 0
    for row in box:
```

```
        free = len(row) - 1
        for c in range(len(row) - 1, -1, -1):
            if row[c] == "*":
                free = c - 1
            elif row[c] == "#":
                row[c] = "."
                row[free] = "#"
                free -= 1
        if row[-1] == "#":
            count += 1
    return count
```

2132. Stamping the Grid

Hard · Greedy, Array, Matrix, Prefix Sum

leetcode.com/problems/stamping-the-grid

PROBLEM

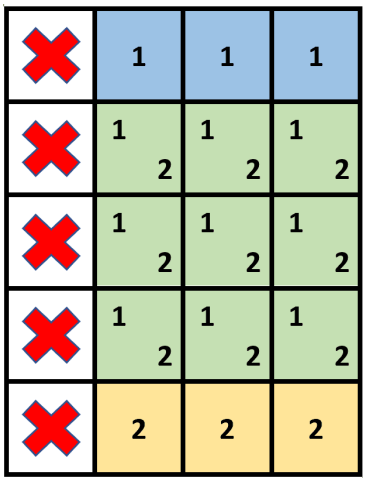
You are given an `m x n` binary matrix `grid` where each cell is either `0` (empty) or `1` (occupied).

You are then given stamps of size `stampHeight x stampWidth`. We want to fit the stamps such that they follow the given **restrictions** and **requirements**:

1. Cover all the **empty** cells.
2. Do not cover any of the **occupied** cells.
3. We can put as **many** stamps as we want.
4. Stamps can **overlap** with each other.
5. Stamps are not allowed to be **rotated**.
6. Stamps must stay completely **inside** the grid.

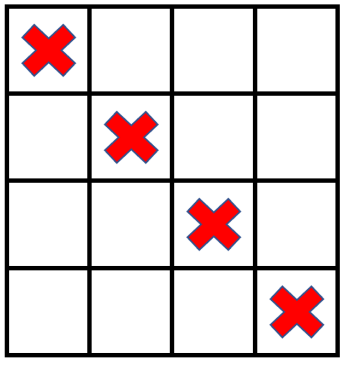
Return `true` *if it is possible to fit the stamps while following the given restrictions and requirements. Otherwise, return* `false`.

Example 1:




```
Input: grid = [[1,0,0,0],[1,0,0,0],[1,0,0,0],[1,0,0,0],[1,0,0,0]], stampHeight = 4, stampWidth = 3
Output: true
Explanation: We have two overlapping stamps (labeled 1 and 2 in the image) that are able to cover all the empty cells.
```

Example 2:



```
Input: grid = [[1,0,0,0],[0,1,0,0],[0,0,1,0],[0,0,0,1]], stampHeight = 2, stampWidth = 2
Output: false
Explanation: There is no way to fit the stamps onto all the empty cells without the stamps going
outside the grid.
```

Constraints:

- `m == grid.length`
- `n == grid[r].length`
- `1 <= m, n <= 10^5`
- `1 <= m * n <= 2 * 10^5`
- `grid[r][c]` is either `0` or `1`.
- `1 <= stampHeight, stampWidth <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: Two-Dimensional Prefix Sum + Two-Dimensional Difference

How to think about it

Every stamp must sit on empty cells only, and every empty cell must be covered at least once. Marking each legal placement cell by cell is O(mn· hw) and too slow. A 2-D prefix sum tests an h× w block in O(1); successful placements are recorded in a 2-D difference array, which is then reconstructed to test coverage of every empty cell. Build the occupied prefix s, increment the difference d for each valid top-left, reconstruct d, and reject an empty cell whose coverage is still 0.

According to the problem description, every empty cell must be covered by a stamp, and no occupied cell can be covered. Therefore, we can traverse the two-dimensional matrix, and for each cell, if all cells in the area of stampHeight × stampWidth with this cell as the upper left corner are empty (i.e., not occupied), then we can place a stamp at this cell.

To quickly determine whether all cells in an area are empty, we can use a two-dimensional prefix sum. We use $s_{i,j}$ to represent the number of occupied cells in the sub-matrix from (1,1) to (i,j) in the two-dimensional matrix. That is, $s_{i, j} = s_{i - 1, j} + s_{i, j - 1} - s_{i - 1, j - 1} + grid_{i-1, j-1}$.

Then, with (i, j) as the upper left corner, and the height and width are stampHeight and stampWidth respectively, the lower right coordinate of the sub-matrix is (x, y) = (i + stampHeight - 1, j + stampWidth - 1). We can calculate the number of occupied cells in this sub-matrix through $s_{x, y} - s_{x, j - 1} - s_{i - 1, y} + s_{i - 1, j - 1}$. If the number of occupied cells in this sub-matrix is 0, then we can place a stamp at (i, j). After placing the stamp, all cells in this stampHeight × stampWidth area will become occupied cells. We can use a two-dimensional difference array d to record this change. That is:

$$\begin{aligned} d_{i, j} &\leftarrow d_{i, j} + 1 \\ d_{i, y + 1} &\leftarrow d_{i, y + 1} - 1 \\ d_{x + 1, j} &\leftarrow d_{x + 1, j} - 1 \\ d_{x + 1, y + 1} &\leftarrow d_{x + 1, y + 1} + 1 \end{aligned}$$

Finally, we perform a prefix sum operation on the two-dimensional difference array d to find out the number of times each cell is covered by a stamp. If a cell is not occupied and the number of times it is covered by a stamp is 0, then we cannot place a stamp at this cell, so we need to return false. If all "unoccupied cells" are successfully covered by stamps, return true.

The time complexity is O(m × n), and the space complexity is O(m × n). Here, m and n are the height and width of the two-dimensional matrix, respectively.

Python 3

```python
class Solution:
    def possibleToStamp(
        self, grid: List[List[int]], stampHeight: int, stampWidth: int
    ) -> bool:
        m, n = len(grid), len(grid[0])
        s = [[0] * (n + 1) for _ in range(m + 1)]
        for i, row in enumerate(grid, 1):
            for j, v in enumerate(row, 1):
                s[i][j] = s[i - 1][j] + s[i][j - 1] - s[i - 1][j - 1] + v
        d = [[0] * (n + 2) for _ in range(m + 2)]
        for i in range(1, m - stampHeight + 2):
            for j in range(1, n - stampWidth + 2):
                x, y = i + stampHeight - 1, j + stampWidth - 1
                if s[x][y] - s[x][j - 1] - s[i - 1][y] + s[i - 1][j - 1] == 0:
                    d[i][j] += 1
                    d[i][y + 1] -= 1
                    d[x + 1][j] -= 1
                    d[x + 1][y + 1] += 1
        for i, row in enumerate(grid, 1):
            for j, v in enumerate(row, 1):
                d[i][j] += d[i - 1][j] + d[i][j - 1] - d[i - 1][j - 1]
                if v == 0 and d[i][j] == 0:
                    return False
        return True
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the top left corners of the stamps that are
# placed
# 2. How many empty cells can NOT be covered by any stamp


# ============================================================
# FOLLOW-UP 1: Return the top left corners of the stamps
# that are placed
# ============================================================

# ---- follow-up 1: function ----
# Answer: Prefix sums find every place where a stamp fits;
# a difference array covers all the cells they cover.
# Time: O(m * n).  Space: O(m * n).
def stamp_positions(grid, stamp_height, stamp_width):
    rows, cols = len(grid), len(grid[0])
    blocked = [[0] * (cols + 1) for _ in range(rows + 1)]
    for r in range(rows):
        for c in range(cols):
            blocked[r + 1][c + 1] = grid[r][c] + blocked[r][c + 1] + blocked[r + 1][c] - blocked[r][c]
    positions = []
    for r in range(rows - stamp_height + 1):
        for c in range(cols - stamp_width + 1):
            total = (blocked[r + stamp_height][c + stamp_width] - blocked[r][c + stamp_width]
                     - blocked[r + stamp_height][c] + blocked[r][c])
            if total == 0:
                positions.append((r, c))
    return positions


# ============================================================
# FOLLOW-UP 2: How many empty cells can NOT be covered by
# any stamp
# ============================================================

# ---- follow-up 2: function ----
# Answer: Mark all cells covered by fitting stamps, then
```

```
# count the empty cells left unmarked.
# Time: O(m * n + P * h * w), P = valid stamp positions.
# Space: O(m * n).
def uncoverable_cells(grid, stamp_height, stamp_width):
    rows, cols = len(grid), len(grid[0])
    covered = [[False] * cols for _ in range(rows)]
    for r, c in stamp_positions(grid, stamp_height, stamp_width):
        for i in range(stamp_height):
            for j in range(stamp_width):
                covered[r + i][c + j] = True
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 0 and not covered[r][c]:
                count += 1
    return count
```

PATTERN 37 OF 41

Two Heaps

20 problems · 3 Easy · 10 Medium · 7 Hard

THE IDEA

A max-heap for the smaller half and a min-heap for the larger half give the median (or any split point) in O(1) with O(log n) updates.

Signals: find median from a data stream, sliding window median, IPO, maximize capital.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
import heapq
small, large = [], []                    # small is a max-heap (store negatives)
def add(x):
    heapq.heappush(small, -x)
    heapq.heappush(large, -heapq.heappop(small))
    if len(large) > len(small):
        heapq.heappush(small, -heapq.heappop(large))
def median():
    if len(small) > len(large):
        return -small[0]
    return (-small[0] + large[0]) / 2
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Take Gifts From the Richest Pile (see page)	Easy	LC 2558	1046
2	Kth Largest Element in a Stream (see page)	Easy	LC 703	1040
3	Last Stone Weight (see page)	Easy	LC 1046	1038
4	Minimum Cost to Hire K Workers — substitute: Total Cost to Hire K Workers (see page)	Medium	LC 2462	1077
5	★ **Furthest Building You Can Reach** (see page)	Medium	LC 1642	1060
6	★ **Single-Threaded CPU** (see page)	Medium	LC 1834	1063
7	Seat Reservation Manager (see page)	Medium	LC 1845	1074
8	Maximum Subsequence Score (see page)	Medium	LC 2542	1066
9	The Number of the Smallest Unoccupied Chair	Medium	LC 1942	1905
10	Maximum Average Pass Ratio	Medium	LC 1792	1908
11	Find Right Interval (see page)	Medium	LC 436	1120
12	Reorganize String (see page)	Medium	LC 767	335
13	Task Scheduler (see page)	Medium	LC 621	1052
14	★ **Find Median from Data Stream** (see page)	Hard	LC 295	1586
15	★ **Sliding Window Median**	Hard	LC 480	1911
16	★ **IPO**	Hard	LC 502	1916
17	Constrained Subsequence Sum (see page)	Hard	LC 1425	474
18	★ **Maximum Performance of a Team** (see page)	Hard	LC 1383	1080
19	Process Tasks Using Servers (see page)	Hard	LC 1882	1083
20	★ **Minimize Deviation in Array**	Hard	LC 1675	1919

1942. The Number of the Smallest Unoccupied Chair

Medium · Array, Hash Table, Heap (Priority Queue)

leetcode.com/problems/the-number-of-the-smallest-unoccupied-chair

PROBLEM

There is a party where `n` friends numbered from `0` to `n - 1` are attending. There is an **infinite** number of chairs in this party that are numbered from `0` to `infinity`. When a friend arrives at the party, they sit on the unoccupied chair with the **smallest number**.

- For example, if chairs `0`, `1`, and `5` are occupied when a friend comes, they will sit on chair number `2`.

When a friend leaves the party, their chair becomes unoccupied at the moment they leave. If another friend arrives at that same moment, they can sit in that chair.

You are given a **0-indexed** 2D integer array `times` where `times[i] = [arrival`$_i$`, leaving`$_i$`]`, indicating the arrival and leaving times of the `i`th friend respectively, and an integer `targetFriend`. All arrival times are **distinct**.

Return *the **chair number** that the friend numbered* `targetFriend` *will sit on*.

Example 1:

```
Input: times = [[1,4],[2,3],[4,6]], targetFriend = 1
Output: 1
Explanation:
- Friend 0 arrives at time 1 and sits on chair 0.
- Friend 1 arrives at time 2 and sits on chair 1.
- Friend 1 leaves at time 3 and chair 1 becomes empty.
- Friend 0 leaves at time 4 and chair 0 becomes empty.
- Friend 2 arrives at time 4 and sits on chair 0.
Since friend 1 sat on chair 1, we return 1.
```

Example 2:

```
Input: times = [[3,10],[1,5],[2,6]], targetFriend = 0
Output: 2
Explanation:
- Friend 1 arrives at time 1 and sits on chair 0.
- Friend 2 arrives at time 2 and sits on chair 1.
- Friend 0 arrives at time 3 and sits on chair 2.
- Friend 1 leaves at time 5 and chair 0 becomes empty.
- Friend 2 leaves at time 6 and chair 1 becomes empty.
- Friend 0 leaves at time 10 and chair 2 becomes empty.
Since friend 0 sat on chair 2, we return 2.
```

Constraints:

- `n == times.length`
- `2 <= n <= 10`^{4}
- `times[i].length == 2`
- `1 <= arrival`$_i$ `< leaving`$_i$ `<= 10`^{5}
- `0 <= targetFriend <= n - 1`
- Each `arrival`$_i$ time is **distinct**.

SOLUTION (PYTHON 3)

Solution 1: Priority Queue (Min-Heap)

How to think about it

Friends take the smallest idle chair on arrival and release it on leaving. Scanning all chairs each time adds a linear factor. A min-heap of idle indices and a min-heap of (leaving,chair) handle reuse. On arrival we free every expired chair, then pop the smallest idle index. We stop when the target friend is seated. Sorting plus heaps is O(nlog n).

First, we create a tuple for each friend consisting of their arrival time, leaving time, and index, then sort these tuples by arrival time.

We use a min-heap idle to store the currently available chair numbers. Initially, we add 0, 1, …, n-1 to idle. We also use a min-heap busy to store tuples (leaving, chair), where leaving represents the leaving time and chair represents the chair number.

We iterate through each friend's arrival time, leaving time, and index. For each friend, we first remove all friends from busy whose leaving time is less than or equal to the current friend's arrival time, and add their chair numbers back to idle. Then we pop a chair number from idle, assign it to the current friend, and add (leaving, chair) to busy. If the current friend's index is equal to targetFriend, we return the assigned chair number.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the number of friends.

Python 3

```python
class Solution:
    def smallestChair(self, times: List[List[int]], targetFriend: int) -> int:
        n = len(times)
        for i in range(n):
            times[i].append(i)
        times.sort()
        idle = list(range(n))
        heapify(idle)
        busy = []
        for arrival, leaving, i in times:
            while busy and busy[0][0] <= arrival:
                heappush(idle, heappop(busy)[1])
            j = heappop(idle)
            if i == targetFriend:
                return j
            heappush(busy, (leaving, j))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the chair used by every friend
# 2. How many chairs are needed in total


# ============================================================
# FOLLOW-UP 1: Return the chair used by every friend
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same sweep over arrivals with a heap of free
# chairs and a heap of (leaving time, chair); record the
# chair when each friend sits.
# Time: O(n log n).  Space: O(n).
import heapq


def chair_of_every_friend(times):
    order = sorted(range(len(times)), key=lambda i: times[i][0])
    free = list(range(len(times)))
```

```
    heapq.heapify(free)
    busy = []
    chair_of = [0] * len(times)
    for friend in order:
        arrive, leave = times[friend]
        while busy and busy[0][0] <= arrive:
            heapq.heappush(free, heapq.heappop(busy)[1])
        chair = heapq.heappop(free)
        chair_of[friend] = chair
        heapq.heappush(busy, (leave, chair))
    return chair_of


# ============================================================
# FOLLOW-UP 2: How many chairs are needed in total
# ============================================================

# ---- follow-up 2: function ----
# Answer: The biggest chair number used, plus one (equal
# to the largest number of friends at the party at once).
# Time: O(n log n).  Space: O(n).
def chairs_needed(times):
    return max(chair_of_every_friend(times)) + 1
```

1792. Maximum Average Pass Ratio

Medium · Greedy, Array, Heap (Priority Queue)

leetcode.com/problems/maximum-average-pass-ratio

PROBLEM

There is a school that has classes of students and each class will be having a final exam. You are given a 2D integer array `classes`, where `classes[i] = [pass_i, total_i]`. You know beforehand that in the i^{th} class, there are `total_i` total students, but only `pass_i` number of students will pass the exam.

You are also given an integer `extraStudents`. There are another `extraStudents` brilliant students that are **guaranteed** to pass the exam of any class they are assigned to. You want to assign each of the `extraStudents` students to a class in a way that **maximizes** the **average** pass ratio across **all** the classes.

The **pass ratio** of a class is equal to the number of students of the class that will pass the exam divided by the total number of students of the class. The **average pass ratio** is the sum of pass ratios of all the classes divided by the number of the classes.

Return *the **maximum** possible average pass ratio after assigning the* `extraStudents` *students.* Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

```
Input: classes = [[1,2],[3,5],[2,2]], extraStudents = 2
Output: 0.78333
Explanation: You can assign the two extra students to the first class. The average pass ratio will be equal to (3/4 + 3/5 + 2/2) / 3 = 0.78333.
```

Example 2:

```
Input: classes = [[2,4],[3,9],[4,5],[2,10]], extraStudents = 4
Output: 0.53485
```

Constraints:

- `1 <= classes.length <= 10^5`
- `classes[i].length == 2`
- `1 <= pass_i <= total_i <= 10^5`
- `1 <= extraStudents <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: Priority Queue (Max-Heap of Increment)

> **How to think about it**
>
> Each extra passing student joins one class; we maximize the average of pass ratios. The gain (a+1)/(b+1)-a/b decreases as a class grows, so always give the next student to the class with the largest current gain. A heap keyed by that gain (or its negation) pops a class, increments both counts, and pushes it back. Average the final ratios.

Suppose a class currently has a pass rate of a/b. If we arrange a smart student into this class, then the pass rate of the class will become (a+1)/(b+1). We can find that the increment of the pass rate is (a+1)/(b+1) - a/b.

We maintain a max-heap, which stores the increment of the pass rate for each class.

Perform `extraStudents` operations, each time taking a class from the top of the heap, adding 1 to both the number of students and the number of passes in this class, then recalculating the increment of the pass rate of this class and putting it back into the heap. Repeat this process until all students are allocated.

Finally, we sum up the pass rates of all classes, and then divide by the number of classes to get the answer.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the number of classes.

Python 3

```python
class Solution:
    def maxAverageRatio(self, classes: List[List[int]], extraStudents: int) -> float:
        h = [(a / b - (a + 1) / (b + 1), a, b) for a, b in classes]
        heapify(h)
        for _ in range(extraStudents):
            _, a, b = heappop(h)
            a, b = a + 1, b + 1
            heappush(h, (a / b - (a + 1) / (b + 1), a, b))
        return sum(v[1] / v[2] for v in h) / len(classes)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return how many extra students each class receives
# 2. The students arrive one at a time and must be placed
# at once; report the average after each


# ============================================================
# FOLLOW-UP 1: Return how many extra students each class
# receives
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same heap of best gains, counting how often each
# class is chosen.
# Time: O((c + e) log c), c = classes, e = extra students.
# Space: O(c).
import heapq


def extra_students_per_class(classes, extra_students):
    def gain(passed, total):
        return (passed + 1) / (total + 1) - passed / total

    given = [0] * len(classes)
    heap = [(-gain(p, t), i) for i, (p, t) in enumerate(classes)]
    heapq.heapify(heap)
    for _ in range(extra_students):
        negative, i = heapq.heappop(heap)
        given[i] += 1
        p, t = classes[i][0] + given[i], classes[i][1] + given[i]
        heapq.heappush(heap, (-gain(p, t), i))
    return given


# ============================================================
# FOLLOW-UP 2: The students arrive one at a time and must
# be placed at once; report the average after each
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same greedy choice, made when each student
# arrives; keep the running sum of the ratios.
# Time: O((c + e) log c).  Space: O(c).
def averages_after_each_student(classes, extra_students):
    def gain(passed, total):
        return (passed + 1) / (total + 1) - passed / total

    state = [list(c) for c in classes]
    heap = [(-gain(p, t), i) for i, (p, t) in enumerate(classes)]
    heapq.heapify(heap)
    total_ratio = sum(p / t for p, t in classes)
```

```
    averages = []
    for _ in range(extra_students):
        negative, i = heapq.heappop(heap)
        total_ratio += -negative
        state[i][0] += 1
        state[i][1] += 1
        heapq.heappush(heap, (-gain(state[i][0], state[i][1]), i))
        averages.append(total_ratio / len(classes))
    return averages
```

PATTERN 37 · TWO HEAPS · PROBLEM 15 OF 20

480. Sliding Window Median

Hard · Array, Hash Table, Treap, Sliding Window, Heap (Priority Queue)

leetcode.com/problems/sliding-window-median

★ **TOP 7 PROBLEM** in Two Heaps: master this one first

PROBLEM

The **median** is the middle value in an ordered integer list. If the size of the list is even, there is no middle value. So the median is the mean of the two middle values.

- For examples, if `arr = [2,3,4]`, the median is `3`.
- For examples, if `arr = [1,2,3,4]`, the median is `(2 + 3) / 2 = 2.5`.

You are given an integer array `nums` and an integer `k`. There is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Return *the median array for each window in the original array*. Answers within 10^{-5} of the actual value will be accepted.

Example 1:

```
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [1.00000,-1.00000,-1.00000,3.00000,5.00000,6.00000]
Explanation:
Window position Median
--------------- -----
[1 3 -1] -3 5 3 6 7 1
 1 [3 -1 -3] 5 3 6 7 -1
 1 3 [-1 -3 5] 3 6 7 -1
 1 3 -1 [-3 5 3] 6 7 3
 1 3 -1 -3 [5 3 6] 7 5
 1 3 -1 -3 5 [3 6 7] 6
```

Example 2:

```
Input: nums = [1,2,3,4,2,3,1,4,2], k = 3
Output: [2.00000,3.00000,3.00000,3.00000,2.00000,3.00000,2.00000]
```

Constraints:

- `1 <= k <= nums.length <=` 10^5
- -2^{31} `<= nums[i] <=` $2^{31} - 1$

SOLUTIONS (PYTHON 3)

Solution 1: Dual Priority Queues (Min-Heap and Max-Heap) + Lazy Deletion

How to think about it

Median of every window. Sorting each window is O(nklog k). Heaps give the median but cannot delete an arbitrary outgoing value. Two heaps: a max-heap for the lower half, a min-heap for the upper half, plus a lazy-deletion map. After insert or erase, only expired heap tops are popped, then the live sizes are rebalanced. Lazy deletion turns an arbitrary erase into a top pop; live sizes are counted separately so the tops remain the median candidates.

We can use two priority queues (min-heap and max-heap) to maintain the elements in the current window. One priority queue stores the smaller half of the elements, and the other priority queue stores the larger half of the elements. This way, the median of the current window is either the average of the top elements of the two heaps or one of the top elements.

We design a class MedianFinder to maintain the elements in the current window. This class includes the following methods:

- `add_num(num)`: Adds num to the current window.
- `find_median()`: Returns the median of the elements in the current window.
- `remove_num(num)`: Removes num from the current window.
- `prune(pq)`: If the top element of the heap is in the lazy deletion dictionary delayed, it pops the top element from the heap and decrements its lazy deletion count. If the lazy deletion count of the element becomes zero, it removes the element from the lazy deletion dictionary.
- `rebalance()`: If the number of elements in the smaller half exceeds the number of elements in the larger half by 2, it moves the top element of the larger half to the smaller half. If the number of elements in the smaller half is less than the number of elements in the larger half, it moves the top element of the larger half to the smaller half.

In the `add_num(num)` method, we first consider adding num to the smaller half. If num is greater than the top element of the larger half, we add num to the larger half. Then we call the `rebalance()` method to ensure that the size difference between the two priority queues does not exceed 1.

In the `remove_num(num)` method, we increment the lazy deletion count of num. Then we compare num with the top element of the smaller half. If num is less than or equal to the top element of the smaller half, we update the size of the smaller half and call the `prune()` method to ensure that the top element of the smaller half is not in the lazy deletion dictionary. Otherwise, we update the size of the larger half and call the `prune()` method to ensure that the top element of the larger half is not in the lazy deletion dictionary.

In the `find_median()` method, if the current window size is odd, we return the top element of the smaller half; otherwise, we return the average of the top elements of the smaller half and the larger half.

In the `prune(pq)` method, if the top element of the heap is in the lazy deletion dictionary, it pops the top element from the heap and decrements its lazy deletion count. If the lazy deletion count of the element becomes zero, it removes the element from the lazy deletion dictionary.

In the `rebalance()` method, if the number of elements in the smaller half exceeds the number of elements in the larger half by 2, it moves the top element of the larger half to the smaller half. If the number of elements in the smaller half is less than the number of elements in the larger half, it moves the top element of the larger half to the smaller half.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the length of the array nums.

Python 3

```
class MedianFinder:
    def __init__(self, k: int):
        self.k = k
        self.small = []
        self.large = []
        self.delayed = defaultdict(int)
        self.small_size = 0
        self.large_size = 0

    def add_num(self, num: int):
        if not self.small or num <= -self.small[0]:
            heappush(self.small, -num)
            self.small_size += 1
        else:
            heappush(self.large, num)
            self.large_size += 1
        self.rebalance()

    def find_median(self) -> float:
        return -self.small[0] if self.k & 1 else (-self.small[0] + self.large[0]) / 2

    def remove_num(self, num: int):
        self.delayed[num] += 1
        if num <= -self.small[0]:
            self.small_size -= 1
            if num == -self.small[0]:
                self.prune(self.small)
        else:
            self.large_size -= 1
            if num == self.large[0]:
                self.prune(self.large)
        self.rebalance()

    def prune(self, pq: List[int]):
        sign = -1 if pq is self.small else 1
        while pq and sign * pq[0] in self.delayed:
            self.delayed[sign * pq[0]] -= 1
            if self.delayed[sign * pq[0]] == 0:
                self.delayed.pop(sign * pq[0])
            heappop(pq)

    def rebalance(self):
        if self.small_size > self.large_size + 1:
            heappush(self.large, -heappop(self.small))
            self.small_size -= 1
            self.large_size += 1
            self.prune(self.small)
        elif self.small_size < self.large_size:
            heappush(self.small, -heappop(self.large))
            self.large_size -= 1
            self.small_size += 1
            self.prune(self.large)


class Solution:
    def medianSlidingWindow(self, nums: List[int], k: int) -> List[float]:
        finder = MedianFinder(k)
        for x in nums[:k]:
            finder.add_num(x)
        ans = [finder.find_median()]
```

```
        for i in range(k, len(nums)):
            finder.add_num(nums[i])
            finder.remove_num(nums[i - k])
            ans.append(finder.find_median())
        return ans
```

Solution 2: Ordered Set

> **How to think about it**
>
> Lazy heaps are long to implement. An ordered set inserts and deletes in O(log k). The left set holds the lower half, the right the upper half: insert into the right, move its minimum left, and rebalance. When the window is full, read the ends and remove the outgoing value. The code is shorter if an ordered-set type is available.

We can use two ordered sets to maintain the elements in the current window. The ordered set l stores the smaller half of the elements in the current window, and the ordered set r stores the larger half of the elements.

We traverse the array nums. For each element x, we add it to the ordered set r, then move the smallest element in the ordered set r to the ordered set l. If the size of the ordered set l is greater than the size of the ordered set r by more than 1, we move the largest element in the ordered set l to the ordered set r.

If the total number of elements in the current window is k and the size is odd, the maximum value in the ordered set l is the median. If the size of the current window is even, the average of the maximum value in the ordered set l and the minimum value in the ordered set r is the median. Then, we remove the leftmost element of the window and continue traversing the array.

The time complexity is O(n log k), and the space complexity is O(k). Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def medianSlidingWindow(self, nums: List[int], k: int) -> List[float]:
        l = SortedList()
        r = SortedList()
        ans = []
        for i, x in enumerate(nums):
            r.add(x)
            l.add(r.pop(0))
            while len(l) - len(r) > 1:
                r.add(l.pop())
            j = i - k + 1
            if j >= 0:
                ans.append(l[-1] if k & 1 else (l[-1] + r[0]) / 2)
                if nums[j] in l:
                    l.remove(nums[j])
                else:
                    r.remove(nums[j])
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Use one sorted list instead of two heaps
# 2. The k-th smallest number of every window instead of
# the median


# ============================================================
# FOLLOW-UP 1: Use one sorted list instead of two heaps
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the window in a sorted list: insert the new
# number with binary search and remove the old one the
# same way; the median is in the middle.
# Time: O(n * k) (insertion into a sorted list).  Space:
# O(k).
import bisect


def sliding_median_sorted_list(nums, k):
```

```
    window = sorted(nums[:k])
    medians = []
    for i in range(k, len(nums) + 1):
        if k % 2 == 1:
            medians.append(float(window[k // 2]))
        else:
            medians.append((window[k // 2 - 1] + window[k // 2]) / 2)
        if i == len(nums):
            break
        window.pop(bisect.bisect_left(window, nums[i - k]))
        bisect.insort(window, nums[i])
    return medians


# ============================================================
# FOLLOW-UP 2: The k-th smallest number of every window
# instead of the median
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same sorted window, reading index r - 1.
# Time: O(n * k).  Space: O(k).
def sliding_kth_smallest(nums, k, r):
    window = sorted(nums[:k])
    answers = []
    for i in range(k, len(nums) + 1):
        answers.append(window[r - 1])
        if i == len(nums):
            break
        window.pop(bisect.bisect_left(window, nums[i - k]))
        bisect.insort(window, nums[i])
    return answers
```

PATTERN 37 · TWO HEAPS · PROBLEM 16 OF 20

502. IPO

Hard · Greedy, Array, Sorting, Heap (Priority Queue)

leetcode.com/problems/ipo

★ **TOP 7 PROBLEM** in Two Heaps: master this one first

PROBLEM

Suppose LeetCode will start its **IPO** soon. In order to sell a good price of its shares to Venture Capital, LeetCode would like to work on some projects to increase its capital before the **IPO**. Since it has limited resources, it can only finish at most `k` distinct projects before the **IPO**. Help LeetCode design the best way to maximize its total capital after finishing at most `k` distinct projects.

You are given `n` projects where the i^{th} project has a pure profit `profits[i]` and a minimum capital of `capital[i]` is needed to start it.

Initially, you have `w` capital. When you finish a project, you will obtain its pure profit and the profit will be added to your total capital.

Pick a list of **at most** `k` distinct projects from given projects to **maximize your final capital**, and return *the final maximized capital*.

The answer is guaranteed to fit in a 32-bit signed integer.

Example 1:

```
Input: k = 2, w = 0, profits = [1,2,3], capital = [0,1,1]
Output: 4
Explanation: Since your initial capital is 0, you can only start the project indexed 0.
After finishing it you will obtain profit 1 and your capital becomes 1.
With capital 1, you can either start the project indexed 1 or the project indexed 2.
Since you can choose at most 2 projects, you need to finish the project indexed 2 to get the maximum
capital.
Therefore, output the final maximized capital, which is 0 + 1 + 3 = 4.
```

Example 2:

```
Input: k = 3, w = 0, profits = [1,2,3], capital = [0,1,2]
Output: 6
```

Constraints:

- `1 <= k <=` 10^5
- `0 <= w <=` 10^9
- `n == profits.length`
- `n == capital.length`
- `1 <= n <=` 10^5
- `0 <= profits[i] <=` 10^4
- `0 <= capital[i] <=` 10^9

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Each of the k choices should take the most profitable project we can afford. Scanning all n projects every time is O(kn), which is heavy for n,k ≤ 10^5. Capital never decreases, so keep locked projects in a min-heap by capital and move those we can afford into a max-heap by profit. Each round pops the best profit and updates capital. Both heaps expose the next unlock and the best pick at the top.

Python 3

```python
class Solution:
    def findMaximizedCapital(
        self, k: int, w: int, profits: List[int], capital: List[int]
    ) -> int:
        h1 = [(c, p) for c, p in zip(capital, profits)]
        heapify(h1)
        h2 = []
        while k:
            while h1 and h1[0][0] <= w:
                heappush(h2, -heappop(h1)[1])
            if not h2:
                break
            w -= heappop(h2)
            k -= 1
        return w
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return which projects were picked
# 2. Projects may be done again and again (k jobs in
# total)


# ========================================================
# FOLLOW-UP 1: Return which projects were picked
# ========================================================

# ---- follow-up 1: function ----
# Answer: Same two heaps, saving the index of every
# project taken.
# Time: O((n + k) log n).  Space: O(n).
import heapq


def projects_picked(k, w, profits, capital):
    projects = sorted(range(len(profits)), key=lambda i: capital[i])
    available = []
    picked = []
    index = 0
    for _ in range(k):
        while index < len(projects) and capital[projects[index]] <= w:
            heapq.heappush(available, (-profits[projects[index]], projects[index]))
            index += 1
        if not available:
            break
        negative, project = heapq.heappop(available)
        w += -negative
        picked.append(project)
    return picked
```

```
# ============================================================
# FOLLOW-UP 2: Projects may be done again and again (k
# jobs in total)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Each round, do the most profitable project you
# can afford; its capital requirement is only a threshold,
# so the best one keeps being repeated.
# Time: O(k * n).  Space: O(1).
def capital_with_repeats(k, w, profits, capital):
    for _ in range(k):
        best = 0
        for i in range(len(profits)):
            if capital[i] <= w and profits[i] > best:
                best = profits[i]
        if best == 0:
            break
        w += best
    return w
```

PATTERN 37 · TWO HEAPS · PROBLEM 20 OF 20

1675. Minimize Deviation in Array

Hard · Greedy, Array, Ordered Set, Heap (Priority Queue)

leetcode.com/problems/minimize-deviation-in-array

★ **TOP 7 PROBLEM** in Two Heaps: master this one first

PROBLEM

You are given an array `nums` of `n` positive integers.

You can perform two types of operations on any element of the array any number of times:

- If the element is **even**, **divide** it by `2`.
  - For example, if the array is `[1,2,3,4]`, then you can do this operation on the last element, and the array will be `[1,2,3,2]`.
- If the element is **odd**, **multiply** it by `2`.
  - For example, if the array is `[1,2,3,4]`, then you can do this operation on the first element, and the array will be `[2,2,3,4]`.

The **deviation** of the array is the **maximum difference** between any two elements in the array.

Return *the **minimum deviation** the array can have after performing some number of operations.*

Example 1:

```
Input: nums = [1,2,3,4]
Output: 1
Explanation: You can transform the array to [1,2,3,2], then to [2,2,3,2], then the deviation will be 3 - 2 = 1.
```

Example 2:

```
Input: nums = [4,1,5,20,3]
Output: 3
Explanation: You can transform the array after two operations to [4,2,5,5,3], then the deviation will be 5 - 2 = 3.
```

Example 3:

```
Input: nums = [2,10,8]
Output: 3
```

Constraints:

- `n == nums.length`
- `2 <= n <= 5 * 10^4`
- `1 <= nums[i] <= 10^9`

SOLUTION (PYTHON 3)

Solution 1: Greedy + Priority Queue

How to think about it

An odd value may be doubled once; an even value may be halved repeatedly. We want the smallest max-min. n can be 5× 10^4, so we cannot list every reachable value of every element. Double every odd first so only halving remains. A max-heap holds the current maximum and we track the global minimum; repeatedly halve the top and update the deviation until the top is odd and cannot shrink.

Intuitively, to get the minimum offset of the array, we need to decrease the maximum value of the array and increase the minimum value of the array.

Since there are two operations that can be performed each time: multiply an odd number by 2; divide an even number by 2, the situation is more complex. We can multiply all odd numbers by 2 to convert them into even numbers, which is equivalent to having only one division operation. The division operation can only reduce a certain number, and only by reducing the maximum value can the result be more optimal.

Therefore, we use a priority queue (max heap) to maintain the maximum value of the array. Each time we take out the top element of the heap for division operation, put the new value into the heap, and update the minimum value and the minimum value of the difference between the top element of the heap and the minimum value.

When the top element of the heap is an odd number, the operation stops.

The time complexity is O(nlog n × log m). Where n and m are the length of the array `nums` and the maximum element of the array, respectively. Since the maximum element in the array is divided by 2 at most O(log m) times, all elements are divided by 2 at most O(nlog m) times. Each time the heap is popped and put into operation, the time complexity is O(log n). Therefore, the total time complexity is O(nlog n × log m).

Python 3

```python
class Solution:
    def minimumDeviation(self, nums: List[int]) -> int:
        h = []
        mi = inf
        for v in nums:
            if v & 1:
                v <<= 1
            h.append(-v)
            mi = min(mi, v)
        heapify(h)
        ans = -h[0] - mi
        while h[0] % 2 == 0:
            x = heappop(h) // 2
            heappush(h, x)
            mi = min(mi, -x)
            ans = min(ans, -h[0] - mi)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the final array that has the smallest
# deviation
# 2. How many operations does the heap simulation use


# ========================================================
# FOLLOW-UP 1: Return the final array that has the
# smallest deviation
# ========================================================

# ---- follow-up 1: function ----
# Answer: Same heap simulation; remember the array when
# the best deviation is found.
# Time: O(n log M * n log n) in the worst case, since a
# sorted copy is made on every improvement, M = largest
# number.  Space: O(n).
import heapq


def best_array_for_deviation(nums):
    heap = []
    smallest = float("inf")
    for x in nums:
```

```
        if x % 2 == 1:
            x *= 2
        heap.append(-x)
        smallest = min(smallest, x)
    heapq.heapify(heap)
    best = float("inf")
    best_array = None
    while True:
        biggest = -heap[0]
        if biggest - smallest < best:
            best = biggest - smallest
            best_array = sorted(-v for v in heap)
        if biggest % 2 == 1:
            break
        heapq.heappop(heap)
        heapq.heappush(heap, -(biggest // 2))
        smallest = min(smallest, biggest // 2)
    return best, best_array


# ============================================================
# FOLLOW-UP 2: How many operations does the heap
# simulation use
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count every halving of the biggest even number.
# Time: O(n log n + n log M * log n).  Space: O(n).
def operations_used(nums):
    heap = []
    for x in nums:
        if x % 2 == 1:
            x *= 2
        heap.append(-x)
    heapq.heapify(heap)
    count = 0
```

```
    while -heap[0] % 2 == 0:
        biggest = -heapq.heappop(heap)
        heapq.heappush(heap, -(biggest // 2))
        count += 1
    return count
```

PATTERN 38 OF 41

K-way Merge

20 problems · 3 Easy · 11 Medium · 6 Hard

THE IDEA

Merge K sorted sources with a heap that always holds the next item from each source; pop the smallest and push its successor.

Signals: merge k sorted lists, kth smallest in a sorted matrix, smallest range covering K lists, find K pairs with smallest sums.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
import heapq
heap = [(lists[i][0], i, 0) for i in range(len(lists)) if lists[i]]
heapq.heapify(heap)
while heap:
    value, i, j = heapq.heappop(heap)
    result.append(value)
    if j + 1 < len(lists[i]):
        heapq.heappush(heap, (lists[i][j + 1], i, j + 1))
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Merge Two Sorted Lists (see page)	Easy	LC 21	1328
2	Merge Sorted Array (see page)	Easy	LC 88	1230
3	The K Weakest Rows in a Matrix (see page)	Easy	LC 1337	1043
4	★ **Kth Smallest Element in a Sorted Matrix** (see page)	Medium	LC 378	217
5	★ **Find K Pairs with Smallest Sums**	Medium	LC 373	1923
6	★ **Ugly Number II** (see page)	Medium	LC 264	1827
7	★ **Super Ugly Number**	Medium	LC 313	1926
8	Sort the Matrix Diagonally (see page)	Medium	LC 1329	1264
9	Find K Closest Elements (see page)	Medium	LC 658	228
10	K Closest Points to Origin (see page)	Medium	LC 973	1048
11	Minimum Cost to Connect Sticks (see page)	Medium	LC 1167	1057
12	Maximum Number of Eaten Apples	Medium	LC 1705	1929
13	Top K Frequent Elements (see page)	Medium	LC 347	34
14	Kth Largest Element in an Array (see page)	Medium	LC 215	737
15	★ **Merge k Sorted Lists** (see page)	Hard	LC 23	1389
16	★ **Smallest Range Covering Elements from K Lists**	Hard	LC 632	1932
17	Kth Smallest Number in Multiplication Table	Hard	LC 668	1935
18	★ **Find K-th Smallest Pair Distance**	Hard	LC 719	1938
19	K-th Smallest Prime Fraction	Hard	LC 786	1941
20	Find the Kth Smallest Sum of a Matrix With Sorted Rows	Hard	LC 1439	1943

PATTERN 38 · K-WAY MERGE · PROBLEM 5 OF 20

373. Find K Pairs with Smallest Sums

Medium · Array, Heap (Priority Queue)

leetcode.com/problems/find-k-pairs-with-smallest-sums

★ **TOP 7 PROBLEM** in K-way Merge: master this one first

PROBLEM

You are given two integer arrays `nums1` and `nums2` sorted in **non-decreasing order** and an integer `k`.

Define a pair `(u, v)` which consists of one element from the first array and one element from the second array.

Return *the* `k` *pairs* $(u_1, v_1), (u_2, v_2), \ldots, (u_k, v_k)$ *with the smallest sums*.

Example 1:

```
Input: nums1 = [1,7,11], nums2 = [2,4,6], k = 3
Output: [[1,2],[1,4],[1,6]]
Explanation: The first 3 pairs are returned from the sequence:
[1,2],[1,4],[1,6],[7,2],[7,4],[11,2],[7,6],[11,4],[11,6]
```

Example 2:

```
Input: nums1 = [1,1,2], nums2 = [1,2,3], k = 2
Output: [[1,1],[1,1]]
Explanation: The first 2 pairs are returned from the sequence:
[1,1],[1,1],[1,2],[2,1],[1,2],[2,2],[1,3],[1,3],[2,3]
```

Constraints:

- `1 <= nums1.length, nums2.length <=` 10^5
- -10^9 `<= nums1[i], nums2[i] <=` 10^9
- `nums1` and `nums2` both are sorted in **non-decreasing order**.
- `1 <= k <=` 10^4
- `k <= nums1.length * nums2.length`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

The k pairs of smallest sums from two sorted arrays. Sorting all pairs is O(mnlog(mn)). For fixed i, increasing j increases the sum, so a heap pops the current minimum and only advances j. Push (nums1[i]+nums2[0], i, 0) for i<k. Each pop, if j+1 exists, pushes the next column of the same i. At most k pops.

Python 3

```
class Solution:
    def kSmallestPairs(
        self, nums1: List[int], nums2: List[int], k: int
    ) -> List[List[int]]:
        q = [[u + nums2[0], i, 0] for i, u in enumerate(nums1[:k])]
        heapify(q)
        ans = []
        while q and k > 0:
            _, i, j = heappop(q)
            ans.append([nums1[i], nums2[j]])
            k -= 1
            if j + 1 < len(nums2):
                heappush(q, [nums1[i] + nums2[j + 1], i, j + 1])
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The k LARGEST sums
# 2. Three sorted lists: the k smallest triples


# ============================================================
# FOLLOW-UP 1: The k LARGEST sums
# ============================================================

# ---- follow-up 1: function ----
# Answer: Start from the last numbers of both lists and
# move toward the front, using a max-heap.
# Time: O(k log k).  Space: O(k).
import heapq


def k_largest_pair_sums(nums1, nums2, k):
    if not nums1 or not nums2:
        return []
    heap = [(-(nums1[-1] + nums2[-1]), len(nums1) - 1, len(nums2) - 1)]
    seen = {(len(nums1) - 1, len(nums2) - 1)}
    result = []
    while heap and len(result) < k:
        negative, i, j = heapq.heappop(heap)
        result.append([nums1[i], nums2[j]])
        for ni, nj in ((i - 1, j), (i, j - 1)):
            if ni >= 0 and nj >= 0 and (ni, nj) not in seen:
                seen.add((ni, nj))
                heapq.heappush(heap, (-(nums1[ni] + nums2[nj]), ni, nj))
    return result


# ============================================================
# FOLLOW-UP 2: Three sorted lists: the k smallest triples
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same idea with three indices; each pop pushes
# the three neighbours.
# Time: O(k log k).  Space: O(k).
def k_smallest_triples(a, b, c, k):
    if not a or not b or not c:
        return []
    heap = [(a[0] + b[0] + c[0], 0, 0, 0)]
    seen = {(0, 0, 0)}
    result = []
    while heap and len(result) < k:
        total, i, j, l = heapq.heappop(heap)
```

```
        result.append([a[i], b[j], c[l]])
        for ni, nj, nl in ((i + 1, j, l), (i, j + 1, l), (i, j, l + 1)):
            if ni < len(a) and nj < len(b) and nl < len(c) and (ni, nj, nl) not in seen:
                seen.add((ni, nj, nl))
                heapq.heappush(heap, (a[ni] + b[nj] + c[nl], ni, nj, nl))
    return result
```

PATTERN 38 · K-WAY MERGE · PROBLEM 7 OF 20

313. Super Ugly Number

Medium · Array, Math, Dynamic Programming

leetcode.com/problems/super-ugly-number

★ **TOP 7 PROBLEM** in K-way Merge: master this one first

PROBLEM

A **super ugly number** is a positive integer whose prime factors are in the array `primes`.

Given an integer `n` and an array of integers `primes`, return *the* n^{th} ***super ugly number***.

The n^{th} **super ugly number** is **guaranteed** to fit in a **32-bit** signed integer.

Example 1:

```
Input: n = 12, primes = [2,7,13,19]
Output: 32
Explanation: [1,2,4,7,8,13,14,16,19,26,28,32] is the sequence of the first 12 super ugly numbers given primes = [2,7,13,19].
```

Example 2:

```
Input: n = 1, primes = [2,3,5]
Output: 1
Explanation: 1 has no prime factors, therefore all of its prime factors are in the array primes = [2,3,5].
```

Constraints:

- `1 <= n <=` 10^5
- `1 <= primes.length <= 100`
- `2 <= primes[i] <= 1000`
- `primes[i]` is **guaranteed** to be a prime number.
- All the values of `primes` are **unique** and sorted in **ascending order**.

SOLUTIONS (PYTHON 3)

Solution 1: Priority Queue (Min Heap)

How to think about it

A super ugly number uses only the given primes. Trial-dividing every integer is too slow for large n. Start from 1, pop the heap minimum x, and push x× p when it does not overflow. If x is divisible by the current prime, skip later primes (Euler-sieve style). The n-th pop is the answer.

We use a priority queue (min heap) to maintain all possible super ugly numbers, initially putting 1 into the queue.

Each time we take the smallest super ugly number x from the queue, multiply x by each number in the array `primes`, and put the product into the queue. Repeat the above operation n times to get the nth super ugly number.

Since the problem guarantees that the nth super ugly number is within the range of a 32-bit signed integer, before we put the product into the queue, we can first check whether the product exceeds 2^{31} - 1. If it does, there is no need to put the product into the queue. In addition, the Euler sieve can be used for optimization.

The time complexity is O(n × m × log (n × m)), and the space complexity is O(n × m). Where m and n are the length of the array `primes` and the given integer n respectively.

Python 3

```python
class Solution:
    def nthSuperUglyNumber(self, n: int, primes: List[int]) -> int:
        q = [1]
        x = 0
        mx = (1 << 31) - 1
        for _ in range(n):
            x = heappop(q)
            for k in primes:
                if x <= mx // k:
                    heappush(q, k * x)
                if x % k == 0:
                    break
        return x
```

Solution 2: Dynamic Programming + Multi-pointer Heap

> **How to think about it**
>
> Method 1 fans each ugly number across every prime and the heap grows large. Keep one pointer per prime: the heap top is the next candidate; after writing it, push that prime's next multiple. The heap stays O(m) and the time is O(nlog m).

Store the first n super ugly numbers in ugly[1..n], and keep a min-heap. Each heap entry belongs to one prime p and records the next candidate p × ugly[index].

Initialize ugly[1] = 1 and push (p, p, 2) for every prime. Repeatedly pop the heap minimum into ugly, then advance that prime's pointer and push it back. Each prime keeps a single pointer, so we do not expand every generated ugly number against the whole prime list.

The time complexity is O(n × log m), and the space complexity is O(n + m), where m is the length of primes.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```python
from heapq import heapify, heappop, heappush


class Solution:
    def nthSuperUglyNumber(self, n: int, primes: List[int]) -> int:
        ugly = [0] * (n + 1)
        ugly[1] = 1
        # every prime keeps one entry: (next candidate, prime, index of the ugly number it multiplies next)
        heap = [(p, p, 2) for p in primes]
        heapify(heap)
        count = 2
        while count <= n:
            value, prime, index = heappop(heap)
            if ugly[count - 1] != value:
                ugly[count] = value
                count += 1
            heappush(heap, (ugly[index] * prime, prime, index + 1))
        return ugly[n]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the first n super ugly numbers
# 2. How many super ugly numbers are at most a limit


# ========================================================
# FOLLOW-UP 1: Return the first n super ugly numbers
# ========================================================
```

```
# ---- follow-up 1: function ----
# Answer: One pointer per prime, always taking the
# smallest next product.
# Time: O(n * p), p = primes.  Space: O(n + p).
def first_n_super_ugly(n, primes):
    ugly = [1]
    pointer = [0] * len(primes)
    while len(ugly) < n:
        candidate = min(ugly[pointer[i]] * primes[i] for i in range(len(primes)))
        ugly.append(candidate)
        for i in range(len(primes)):
            if ugly[pointer[i]] * primes[i] == candidate:
                pointer[i] += 1
    return ugly


# ============================================================
# FOLLOW-UP 2: How many super ugly numbers are at most a
# limit
# ============================================================

# ---- follow-up 2: function ----
# Answer: Generate in order and stop when the next number
# is over the limit.
# Time: O(c * p), c = how many are at most the limit.
# Space: O(c + p).
def count_super_ugly_up_to(limit, primes):
    ugly = [1]
    pointer = [0] * len(primes)
    while True:
        candidate = min(ugly[pointer[i]] * primes[i] for i in range(len(primes)))
        if candidate > limit:
            return len(ugly)
        ugly.append(candidate)
        for i in range(len(primes)):
            if ugly[pointer[i]] * primes[i] == candidate:
                pointer[i] += 1
```

1705. Maximum Number of Eaten Apples

Medium · Greedy, Array, Heap (Priority Queue)

leetcode.com/problems/maximum-number-of-eaten-apples

PROBLEM

There is a special kind of apple tree that grows apples every day for `n` days. On the `i`th day, the tree grows `apples[i]` apples that will rot after `days[i]` days, that is on day `i + days[i]` the apples will be rotten and cannot be eaten. On some days, the apple tree does not grow any apples, which are denoted by `apples[i] == 0` and `days[i] == 0`.

You decided to eat **at most** one apple a day (to keep the doctors away). Note that you can keep eating after the first `n` days.

Given two integer arrays `days` and `apples` of length `n`, return *the maximum number of apples you can eat.*

Example 1:

```
Input: apples = [1,2,3,5,2], days = [3,2,1,4,2]
Output: 7
Explanation: You can eat 7 apples:
- On the first day, you eat an apple that grew on the first day.
- On the second day, you eat an apple that grew on the second day.
- On the third day, you eat an apple that grew on the second day. After this day, the apples that grew
on the third day rot.
- On the fourth to the seventh days, you eat apples that grew on the fourth day.
```

Example 2:

```
Input: apples = [3,0,0,0,0,2], days = [3,0,0,0,0,2]
Output: 5
Explanation: You can eat 5 apples:
- On the first to the third day you eat apples that grew on the first day.
- Do nothing on the fouth and fifth days.
- On the sixth and seventh days you eat apples that grew on the sixth day.
```

Constraints:

- `n == apples.length == days.length`
- `1 <= n <= 2 * 10`^{4}
- `0 <= apples[i], days[i] <= 2 * 10`^{4}
- `days[i] = 0` if and only if `apples[i] = 0`.

SOLUTION (PYTHON 3)

Solution 1: Greedy + Priority Queue

> **How to think about it**
>
> At most one apple may be eaten per day, and it must not have rotted. Eating long-lived apples first wastes those about to expire and reduces the total. Among still-edible apples, always eat the soonest to rot. That policy is a min-heap keyed by expiry. On day i, push any new batch as (expiry,count). After dropping expired batches, eat one from the heap top and push the remainder back. Continue until both production and the heap are empty.

We can greedily choose the apples that are closest to rotting among the unrotten apples, so that we can eat as many apples as possible.

Therefore, we can use a priority queue (min-heap) to store the rotting time of the apples and the corresponding number of apples. Each time, we take out the apples with the smallest rotting time from the priority queue, then decrement their quantity by one. If the quantity is not zero after decrementing, we put them back into the priority queue. If the apples have

already rotted, we remove them from the priority queue.

The time complexity is O(n × log n + M), and the space complexity is O(n). Here, n is the length of the array days, and M = max(days).

Python 3

```
class Solution:
    def eatenApples(self, apples: List[int], days: List[int]) -> int:
        n = len(days)
        i = ans = 0
        q = []
        while i < n or q:
            if i < n and apples[i]:
                heappush(q, (i + days[i] - 1, apples[i]))
            while q and q[0][0] < i:
                heappop(q)
            if q:
                t, v = heappop(q)
                v -= 1
                ans += 1
                if v and t > i:
                    heappush(q, (t, v))
            i += 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. On which days are apples eaten
# 2. You can eat two apples a day


# ============================================================
# FOLLOW-UP 1: On which days are apples eaten
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same heap of (rot day, count); record every
# day an apple is eaten.
# Time: O((n + D) log n), D = largest expiry gap.  Space:
# O(n).
import heapq


def days_with_eating(apples, days):
    heap = []
    eaten_on = []
    day = 0
    while day < len(apples) or heap:
        if day < len(apples) and apples[day] > 0:
            heapq.heappush(heap, [day + days[day], apples[day]])
        while heap and (heap[0][0] <= day or heap[0][1] == 0):
            heapq.heappop(heap)
        if heap:
            heap[0][1] -= 1
            eaten_on.append(day)
        day += 1
    return eaten_on


# ============================================================
# FOLLOW-UP 2: You can eat two apples a day
# ============================================================

# ---- follow-up 2: function ----
# Answer: Each day, take up to two apples, from the
```

```
# earliest-rotting batches first.
# Time: O((n + D) log n).  Space: O(n).
def eaten_two_per_day(apples, days):
    heap = []
    total = 0
    day = 0
    while day < len(apples) or heap:
        if day < len(apples) and apples[day] > 0:
            heapq.heappush(heap, [day + days[day], apples[day]])
        for _ in range(2):
            while heap and (heap[0][0] <= day or heap[0][1] == 0):
                heapq.heappop(heap)
            if heap:
                heap[0][1] -= 1
                total += 1
        day += 1
    return total
```

632. Smallest Range Covering Elements from K Lists

Hard · Greedy, Array, Hash Table, Sorting, Sliding Window, Heap (Priority Queue)

leetcode.com/problems/smallest-range-covering-elements-from-k-lists

★ **TOP 7 PROBLEM** in K-way Merge: master this one first

PROBLEM

You have `k` lists of sorted integers in **non-decreasing order**. Find the **smallest** range that includes at least one number from each of the `k` lists.

We define the range `[a, b]` is smaller than range `[c, d]` if `b - a < d - c` **or** `a < c` if `b - a == d - c`.

Example 1:

```
Input: nums = [[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]
Output: [20,24]
Explanation:
List 1: [4, 10, 15, 24,26], 24 is in range [20,24].
List 2: [0, 9, 12, 20], 20 is in range [20,24].
List 3: [5, 18, 22, 30], 22 is in range [20,24].
```

Example 2:

```
Input: nums = [[1,2,3],[1,2,3],[1,2,3]]
Output: [1,1]
```

Constraints:

- `nums.length == k`
- `1 <= k <= 3500`
- `1 <= nums[i].length <= 50`
- -10^5 `<= nums[i][j] <=` 10^5
- `nums[i]` is sorted in **non-decreasing** order.

SOLUTIONS (PYTHON 3)

Solution 1: Sorting + Sliding Window

How to think about it

We need one element from each of k sorted lists so the covering range is shortest. Index combinations do not scale. Flatten (value, group) pairs and sort. The task is the shortest window that covers all k groups; expand the right end and shrink the left while a counter still sees k groups.

We construct a data item (x, i) for each number x and its group i, and store these items in a new array t. Then, we sort t by the value of the numbers (similar to merging multiple sorted arrays into a new sorted array).

Next, we traverse each data item in t, focusing on the group to which each number belongs. We use a hash table to record the groups of numbers within the sliding window. If the number of groups is k, it means the current window meets the problem's requirements. At this point, we calculate the start and end positions of the window and update the answer.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the total number of numbers in all arrays.

Python 3

```python
class Solution:
    def smallestRange(self, nums: List[List[int]]) -> List[int]:
        t = [(x, i) for i, v in enumerate(nums) for x in v]
        t.sort()
        cnt = Counter()
        ans = [-inf, inf]
        j = 0
        for i, (b, v) in enumerate(t):
            cnt[v] += 1
            while len(cnt) == len(nums):
                a = t[j][0]
                x = b - a - (ans[1] - ans[0])
                if x < 0 or (x == 0 and a < ans[0]):
                    ans = [a, b]
                w = t[j][1]
                cnt[w] -= 1
                if cnt[w] == 0:
                    cnt.pop(w)
                j += 1
        return ans
```

Solution 2: Priority Queue (Heap)

> **How to think about it**
>
> Method 1 materializes every value. Because each list is sorted, a min-heap of the k current pointers plus a running maximum already describes a candidate range; advance the list that produced the minimum. Space is O(k).

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```python
from heapq import heapify, heappop, heappush


class Solution:
    def smallestRange(self, nums: List[List[int]]) -> List[int]:
        k = len(nums)
        heap = [(row[0], j, 0) for j, row in enumerate(nums)]
        heapify(heap)
        biggest = max(row[0] for row in nums)
        left, right = 0, float("inf")
        while len(heap) == k:                    # the heap holds one number of every list
            smallest, j, i = heappop(heap)
            if biggest - smallest < right - left:
                left, right = smallest, biggest
            if i + 1 < len(nums[j]):
                nxt = nums[j][i + 1]
                heappush(heap, (nxt, j, i + 1))
                biggest = max(biggest, nxt)
        return [left, right]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Sliding window version (merge all numbers with their
# list id)
# 2. The range must contain numbers from at least m of the
# lists


# ============================================================
# FOLLOW-UP 1: Sliding window version (merge all numbers
# with their list id)
# ============================================================
```

```
# ---- follow-up 1: function ----
# Answer: Sort every (value, list) pair and slide a window
# until it contains every list.
# Time: O(N log N), N = total numbers.  Space: O(N).
def smallest_range_by_window(nums):
    pairs = []
    for i in range(len(nums)):
        for x in nums[i]:
            pairs.append((x, i))
    pairs.sort()
    need = len(nums)
    have = {}
    best = None
    left = 0
    for right in range(len(pairs)):
        have[pairs[right][1]] = have.get(pairs[right][1], 0) + 1
        while len(have) == need:
            if best is None or pairs[right][0] - pairs[left][0] < best[1] - best[0]:
                best = [pairs[left][0], pairs[right][0]]
            have[pairs[left][1]] -= 1
            if have[pairs[left][1]] == 0:
                del have[pairs[left][1]]
            left += 1
    return best


# ============================================================
# FOLLOW-UP 2: The range must contain numbers from at
# least m of the lists
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same sliding window; the condition becomes 'at
# least m different lists'.
# Time: O(N log N).  Space: O(N).
def smallest_range_at_least_m_lists(nums, m):
    pairs = []
    for i in range(len(nums)):
        for x in nums[i]:
            pairs.append((x, i))
    pairs.sort()
    have = {}
    best = None
    left = 0
    for right in range(len(pairs)):
        have[pairs[right][1]] = have.get(pairs[right][1], 0) + 1
        while len(have) >= m:
            if best is None or pairs[right][0] - pairs[left][0] < best[1] - best[0]:
                best = [pairs[left][0], pairs[right][0]]
            have[pairs[left][1]] -= 1
            if have[pairs[left][1]] == 0:
                del have[pairs[left][1]]
            left += 1
    return best
```

668. Kth Smallest Number in Multiplication Table

Hard · Math, Binary Search

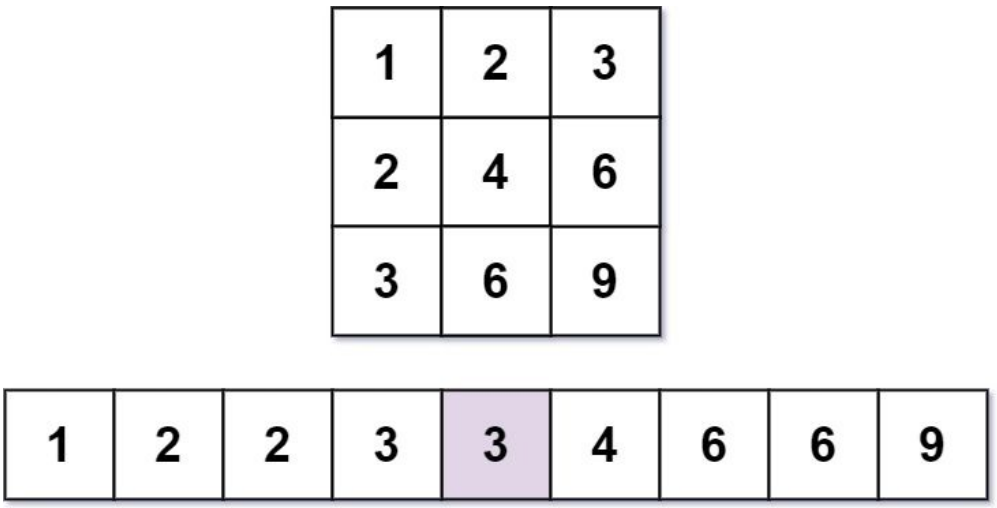
leetcode.com/problems/kth-smallest-number-in-multiplication-table

PROBLEM

Nearly everyone has used the Multiplication Table. The multiplication table of size `m x n` is an integer matrix `mat` where `mat[i][j] == i * j` (**1-indexed**).

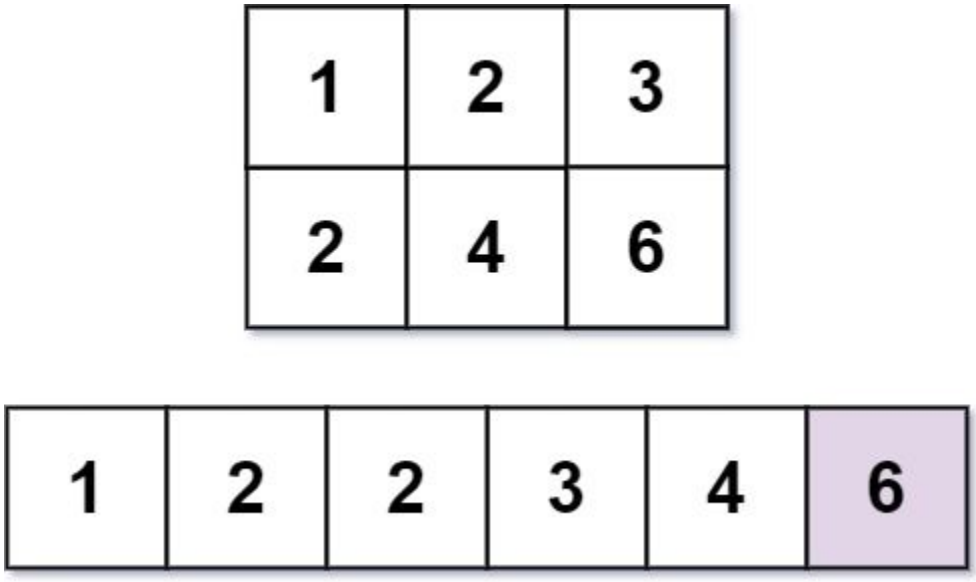
Given three integers `m`, `n`, and `k`, return *the* k^{th} *smallest element in the* `m x n` *multiplication table*.

Example 1:




```
Input: m = 3, n = 3, k = 5
Output: 3
Explanation: The 5th smallest number is 3.
```

Example 2:




```
Input: m = 2, n = 3, k = 6
Output: 6
Explanation: The 6th smallest number is 6.
```

Constraints:

- `1 <= m, n <= 3 * 10^4`
- `1 <= k <= m * n`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

The k-th smallest entry of an m× n multiplication table cannot be materialized when sides reach 3· 10^4. Binary-search the value x. Row i contributes min(floor(x/i), n) entries ≤ x. The answer is the smallest x with count ≥ k.

Python 3

```python
class Solution:
    def findKthNumber(self, m: int, n: int, k: int) -> int:
        left, right = 1, m * n
        while left < right:
            mid = (left + right) >> 1
            cnt = 0
            for i in range(1, m + 1):
                cnt += min(mid // i, n)
            if cnt >= k:
                right = mid
            else:
                left = mid + 1
        return left
```

Complexity. Time: O(m * log(m * n)). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The k-th LARGEST number in the table
# 2. How many table entries are at most x


# ============================================================
# FOLLOW-UP 1: The k-th LARGEST number in the table
# ============================================================

# ---- follow-up 1: function ----
# Answer: The k-th largest is the (m * n - k + 1)-th
# smallest.
# Time: O(m log (m * n)).  Space: O(1).
def kth_largest_in_table(m, n, k):
    def count_at_most(x):
        total = 0
        for row in range(1, m + 1):
            total += min(n, x // row)
        return total

    target = m * n - k + 1
    low, high = 1, m * n
    while low < high:
        middle = (low + high) // 2
        if count_at_most(middle) >= target:
            high = middle
        else:
            low = middle + 1
    return low


# ============================================================
```

```
# FOLLOW-UP 2: How many table entries are at most x
# ========================================================

# ---- follow-up 2: function ----
# Answer: Row i has min(n, x // i) entries that are at
# most x.
# Time: O(m).  Space: O(1).
def count_entries_at_most(m, n, x):
    total = 0
    for row in range(1, m + 1):
        total += min(n, x // row)
    return total
```

PATTERN 38 · K-WAY MERGE · PROBLEM 18 OF 20

719. Find K-th Smallest Pair Distance

Hard · Array, Two Pointers, Binary Search, Sorting

leetcode.com/problems/find-k-th-smallest-pair-distance

★ **TOP 7 PROBLEM** in K-way Merge: master this one first

PROBLEM

The **distance of a pair** of integers `a` and `b` is defined as the absolute difference between `a` and `b`.

Given an integer array `nums` and an integer `k`, return *the* k^{th} *smallest* ***distance among all the pairs*** `nums[i]` *and* `nums[j]` *where* `0 <= i < j < nums.length`.

Example 1:

```
Input: nums = [1,3,1], k = 1
Output: 0
Explanation: Here are all the pairs:
(1,3) -> 2
(1,1) -> 0
(3,1) -> 2
Then the 1st smallest distance pair is (1,1), and its distance is 0.
```

Example 2:

```
Input: nums = [1,1,1], k = 2
Output: 0
```

Example 3:

```
Input: nums = [1,6,1], k = 3
Output: 5
```

Constraints:

- `n == nums.length`
- `2 <= n <=` 10^4
- `0 <= nums[i] <=` 10^6
- `1 <= k <= n * (n - 1) / 2`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

We want the k-th smallest pairwise distance. n≤ 10^4 forbids listing all O(n^2) pairs. Distances lie in [0,max-min] and the count of pairs ≤ d is monotone in d. Binary-search d and count pairs with distance ≤ d. After sorting, each right value b has a leftmost partner b-d; a lower-bound search counts them in O(nlog n). The smallest d whose count is at least k is the answer, searched in [0, nums[-1]-nums[0]].

Python 3

```python
class Solution:
    def smallestDistancePair(self, nums: List[int], k: int) -> int:
        def count(dist):
            cnt = 0
            for i, b in enumerate(nums):
                a = b - dist
                j = bisect_left(nums, a, 0, i)
                cnt += i - j
            return cnt

        nums.sort()
        return bisect_left(range(nums[-1] - nums[0]), k, key=count)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return one pair that has the k-th smallest distance
# 2. The k-th LARGEST pair distance


# ============================================================
# FOLLOW-UP 1: Return one pair that has the k-th smallest
# distance
# ============================================================

# ---- follow-up 1: function ----
# Answer: Find the distance by binary search, then scan
# for a pair with exactly that distance.
# Time: O(n log n + n log D + n^2) in the worst case, D =
# largest gap (the last scan is quadratic).  Space: O(n).
def pair_with_kth_distance(nums, k):
    nums = sorted(nums)

    def count_at_most(d):
        total = 0
        left = 0
        for right in range(len(nums)):
            while nums[right] - nums[left] > d:
                left += 1
            total += right - left
        return total

    low, high = 0, nums[-1] - nums[0]
    while low < high:
        middle = (low + high) // 2
        if count_at_most(middle) >= k:
            high = middle
        else:
            low = middle + 1
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[j] - nums[i] == low:
                return nums[i], nums[j]
    return None


# ============================================================
# FOLLOW-UP 2: The k-th LARGEST pair distance
# ============================================================

# ---- follow-up 2: function ----
# Answer: The k-th largest is the (pairs - k + 1)-th
# smallest.
# Time: O(n log n + n log D).  Space: O(n).
def kth_largest_pair_distance(nums, k):
```

```
    nums = sorted(nums)
    pairs = len(nums) * (len(nums) - 1) // 2
    target = pairs - k + 1

    def count_at_most(d):
        total = 0
        left = 0
        for right in range(len(nums)):
            while nums[right] - nums[left] > d:
                left += 1
```

```
            total += right - left
        return total

    low, high = 0, nums[-1] - nums[0]
    while low < high:
        middle = (low + high) // 2
        if count_at_most(middle) >= target:
            high = middle
        else:
            low = middle + 1
    return low
```

PATTERN 38 · K-WAY MERGE · PROBLEM 19 OF 20

786. K-th Smallest Prime Fraction

Medium · Array, Two Pointers, Binary Search, Sorting, Heap (Priority Queue)

leetcode.com/problems/k-th-smallest-prime-fraction

PROBLEM

You are given a sorted integer array `arr` containing `1` and **prime** numbers, where all the integers of `arr` are unique. You are also given an integer `k`.

For every `i` and `j` where `0 <= i < j < arr.length`, we consider the fraction `arr[i] / arr[j]`.

Return *the* k^{th} *smallest fraction considered*. Return your answer as an array of integers of size `2`, where `answer[0] == arr[i]` and `answer[1] == arr[j]`.

Example 1:

```
Input: arr = [1,2,3,5], k = 3
Output: [2,5]
Explanation: The fractions to be considered in sorted order are:
1/5, 1/3, 2/5, 1/2, 3/5, and 2/3.
The third fraction is 2/5.
```

Example 2:

```
Input: arr = [1,7], k = 1
Output: [1,7]
```

Constraints:

- `2 <= arr.length <= 1000`
- `1 <= arr[i] <= 3 * 10^4`
- `arr[0] == 1`
- `arr[i]` is a **prime** number for `i > 0`.
- All the numbers of `arr` are **unique** and sorted in **strictly increasing** order.
- `1 <= k <= arr.length * (arr.length - 1) / 2`

Follow up:

Can you solve the problem with better than

$O(n^2)$

complexity?

SOLUTION (PYTHON 3)

Solution 1

How to think about it

The k-th smallest proper fraction arr[i]/arr[j] over a sorted prime list. n≤ 1000; listing all pairs is $O(n^2 \log n)$. For fixed j the fractions increase with i. A min-heap merges these n-1 chains. Start with 1/arr[j] for each j, pop k-1 times while pushing the next numerator. The heap top is the answer.

Python 3

```
class Solution:
    def kthSmallestPrimeFraction(self, arr: List[int], k: int) -> List[int]:
        h = [(1 / y, 0, j + 1) for j, y in enumerate(arr[1:])]
        heapify(h)
        for _ in range(k - 1):
            _, i, j = heappop(h)
            if i + 1 < j:
                heappush(h, (arr[i + 1] / arr[j], i + 1, j))
        return [arr[h[0][1]], arr[h[0][2]]]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The k-th LARGEST fraction
# 2. How many fractions are less than or equal to a value
# x


# ============================================================
# FOLLOW-UP 1: The k-th LARGEST fraction
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use a max-heap that holds, for every
# denominator, its biggest remaining fraction (the
# numerator just before it).
# Time: O(n + k log n).  Space: O(n).
import heapq


def kth_largest_fraction(arr, k):
    n = len(arr)
    heap = [(-arr[i - 1] / arr[i], i, i - 1) for i in range(1, n)]
    heapq.heapify(heap)
    for _ in range(k - 1):
        value, i, j = heapq.heappop(heap)
        if j - 1 >= 0:
            heapq.heappush(heap, (-arr[j - 1] / arr[i], i, j - 1))
    value, i, j = heap[0]
    return [arr[j], arr[i]]


# ============================================================
# FOLLOW-UP 2: How many fractions are less than or equal
# to a value x
# ============================================================

# ---- follow-up 2: function ----
# Answer: Two pointers: for each denominator move the
# numerator pointer forward while the fraction is small
# enough.
# Time: O(n^2) in the worst case.  Space: O(1).
def count_fractions_at_most(arr, x):
    total = 0
    j = 0
    for i in range(1, len(arr)):
        j = 0
        while j < i and arr[j] / arr[i] <= x:
            j += 1
        total += j
    return total
```

PATTERN 38 · K-WAY MERGE · PROBLEM 20 OF 20

1439. Find the Kth Smallest Sum of a Matrix With Sorted Rows

Hard · Array, Binary Search, Matrix, Heap (Priority Queue)

leetcode.com/problems/find-the-kth-smallest-sum-of-a-matrix-with-sorted-rows

PROBLEM

You are given an `m x n` matrix `mat` that has its rows sorted in non-decreasing order and an integer `k`.

You are allowed to choose **exactly one element** from each row to form an array.

Return *the* k^{th} *smallest array sum among all possible arrays*.

Example 1:

```
Input: mat = [[1,3,11],[2,4,6]], k = 5
Output: 7
Explanation: Choosing one element from each row, the first k smallest sum are:
[1,2], [1,4], [3,2], [3,4], [1,6]. Where the 5th sum is 7.
```

Example 2:

```
Input: mat = [[1,3,11],[2,4,6]], k = 9
Output: 17
```

Example 3:

```
Input: mat = [[1,10,10],[1,4,5],[2,3,6]], k = 7
Output: 9
Explanation: Choosing one element from each row, the first k smallest sum are:
[1,1,2], [1,1,3], [1,4,2], [1,4,3], [1,1,6], [1,5,2], [1,5,3]. Where the 7th sum is 9.
```

Constraints:

- `m == mat.length`
- `n == mat.length[i]`
- `1 <= m, n <= 40`
- `1 <= mat[i][j] <= 5000`
- `1 <= k <= min(200, n^m)`
- `mat[i]` is a non-decreasing array.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

One value per row yields n^m sums, but k≤ 200. We only need the k smallest partial sums. Rows are sorted, so combining with the next row uses at most its first k entries. Let pre be the k smallest sums so far. Cartesian-add the current row, sort, and keep k values. After the last row, pre[k-1] is the answer.

Python 3

```python
class Solution:
    def kthSmallest(self, mat: List[List[int]], k: int) -> int:
        pre = [0]
        for cur in mat:
            pre = sorted(a + b for a in pre for b in cur[:k])[:k]
        return pre[-1]
```

Complexity. Time: O(m * k^2 * log k). Space: O(k^2).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The k-th LARGEST sum
# 2. How many ways give a sum of at most s


# ============================================================
# FOLLOW-UP 1: The k-th LARGEST sum
# ============================================================

# ---- follow-up 1: function ----
# Answer: Merge row by row, keeping the k largest sums at
# each step.
# Time: O(m * k * n log (k * n)).  Space: O(k * n).
def kth_largest_matrix_sum(mat, k):
    sums = [0]
    for row in mat:
        combined = sorted((s + x for s in sums for x in row), reverse=True)
        sums = combined[:k]
    return sums[k - 1]


# ============================================================
# FOLLOW-UP 2: How many ways give a sum of at most s
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count with a table over rows: ways[t] is how
# many choices give total t so far.
# Time: O(m * n * limit) in the worst case.  Space:
# O(limit).
def count_sums_at_most(mat, limit):
    ways = {0: 1}
    for row in mat:
        new_ways = {}
        for total, count in ways.items():
            for x in row:
                if total + x <= limit:
                    new_ways[total + x] = new_ways.get(total + x, 0) + count
        ways = new_ways
    return sum(ways.values())
```

PATTERN 39 OF 41

Game Theory / Minimax

20 problems · 3 Easy · 12 Medium · 5 Hard

THE IDEA

Two players play optimally. Define win(state) = true if the player to move can force a win: it is true when some move leads to a state where the opponent loses. Cache states.

Signals: nim game, stone game, can I win, predict the winner, flip game, guess number higher or lower II.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
from functools import lru_cache
@lru_cache(None)
def can_win(state):
    for move in moves(state):
        if not can_win(apply(state, move)):   # opponent loses: I win
            return True
    return False
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Nim Game	Easy	LC 292	1946
2	Divisor Game	Easy	LC 1025	1948
3	Optimal Strategy — substitute: Flip Game	Easy	LC 293	1950
4	★ **Stone Game**	Medium	LC 877	1953
5	★ **Stone Game II**	Medium	LC 1140	1956
6	Stone Game VI	Medium	LC 1686	1959
7	Stone Game VII	Medium	LC 1690	1962
8	★ **Predict the Winner**	Medium	LC 486	1966
9	★ **Can I Win**	Medium	LC 464	1969
10	★ **Guess Number Higher or Lower II**	Medium	LC 375	1972
11	Minimax Game — substitute: Min and Max (see page)	Medium	LC 1673	455
12	Find the Winner of the Circular Game (see page)	Medium	LC 1823	522
13	Maximum Number of Coins You Can Get	Medium	LC 1561	1976
14	Stone Game IX	Medium	LC 2029	1978
15	Maximum Score from Performing Multiplication Operations	Medium	LC 1770	1982
16	★ **Stone Game III**	Hard	LC 1406	1985
17	Stone Game IV	Hard	LC 1510	1989
18	★ **Cat and Mouse**	Hard	LC 913	1992
19	Minimum Number of Moves to Make Palindrome	Hard	LC 2193	1998
20	Stone Game VIII	Hard	LC 1872	2001

PATTERN 39 · GAME THEORY / MINIMAX · PROBLEM 1 OF 20

292. Nim Game

Easy · Brainteaser, Minimax, Math, Game Theory, Nim Game, Impartial Game

leetcode.com/problems/nim-game

PROBLEM

You are playing the following Nim Game with your friend:

- Initially, there is a heap of stones on the table.
- You and your friend will alternate taking turns, and **you go first**.
- On each turn, the person whose turn it is will remove 1 to 3 stones from the heap.
- The one who removes the last stone is the winner.

Given `n`, the number of stones in the heap, return `true` *if you can win the game assuming both you and your friend play optimally, otherwise return* `false`.

Example 1:

```
Input: n = 4
Output: false
Explanation: These are the possible outcomes:
1. You remove 1 stone. Your friend removes 3 stones, including the last stone. Your friend wins.
2. You remove 2 stones. Your friend removes 2 stones, including the last stone. Your friend wins.
3. You remove 3 stones. Your friend removes the last stone. Your friend wins.
In all outcomes, your friend wins.
```

Example 2:

```
Input: n = 1
Output: true
```

Example 3:

```
Input: n = 2
Output: true
```

Constraints:

- $1 <= n <= 2^{31} - 1$

SOLUTION (PYTHON 3)

Solution 1: Finding the Pattern

> **How to think about it**
>
> Each turn takes 1 to 3 stones. A multiple of 4 cannot be cleared in one move, and the opponent can always restore a multiple of 4. The first player wins iff n is not divisible by 4.

The first player who gets a multiple of 4 (i.e., n can be divided by 4) will lose the game.

Proof:

1. When n < 4, the first player can directly take all the stones, so the first player will win the game.
1. When n = 4, no matter whether the first player chooses 1, 2, 3, the second player can always choose the remaining number, so the first player will lose the game.
1. When 4 < n < 8, i.e., n = 5, 6, 7, the first player can correspondingly reduce the number to 4, then the "death number" 4 is given to the second player, and the second player will lose the game.
1. When n = 8, no matter whether the first player chooses 1, 2, 3, it will leave a number between 4 < n < 8 to the second player, so the first player will lose the game.

1. ...
1. By induction, when a player gets the number n, and n can be divided by 4, he will lose the game, otherwise, he will win the game.

The time complexity is O(1), and the space complexity is O(1).

Python 3

```
class Solution:
    def canWinNim(self, n: int) -> bool:
        return n % 4 != 0
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Misère version: the player who takes the LAST stone
# loses
# 2. You may take 1 to k stones


# ============================================================
# FOLLOW-UP 1: Misère version: the player who takes the
# LAST stone loses
# ============================================================

# ---- follow-up 1: function ----
# Answer: With 1 to 3 stones per turn, you want to leave
# the opponent exactly 1 stone, so you lose only when n -
# 1 is a multiple of 4.
# Time: O(1).  Space: O(1).
def can_win_misere(n):
    return (n - 1) % 4 != 0


# ============================================================
# FOLLOW-UP 2: You may take 1 to k stones
# ============================================================

# ---- follow-up 2: function ----
# Answer: Leave the opponent a multiple of k + 1; you lose
# only when n already is one.
# Time: O(1).  Space: O(1).
def can_win_take_up_to(n, k):
    return n % (k + 1) != 0
```

PATTERN 39 · GAME THEORY / MINIMAX · PROBLEM 2 OF 20

1025. Divisor Game

Easy · Brainteaser, Math, Dynamic Programming, Game Theory, Impartial Game

leetcode.com/problems/divisor-game

PROBLEM

Alice and Bob take turns playing a game, with Alice starting first.

Initially, there is a number `n` on the chalkboard. On each player's turn, that player makes a move consisting of:

- Choosing any integer `x` with `0 < x < n` and `n % x == 0`.
- Replacing the number `n` on the chalkboard with `n - x`.

Also, if a player cannot make a move, they lose the game.

Return `true` *if and only if Alice wins the game, assuming both players play optimally*.

Example 1:

```
Input: n = 2
Output: true
Explanation: Alice chooses 1, and Bob has no more moves.
```

Example 2:

```
Input: n = 3
Output: false
Explanation: Alice chooses 1, Bob chooses 1, and Alice has no more moves.
```

Constraints:

- `1 <= n <= 1000`

SOLUTION (PYTHON 3)

Solution 1: Mathematical Induction

How to think about it

With n≤ 1000 we could DP each remaining value: the starter wins if some x leaves a losing position. Small cases already suggest that odd n loses and even n wins, which induction confirms. A proper divisor of an odd integer is odd, so the opponent receives an even number. From an even integer we can subtract 1 and leave an odd number. The outcome depends only on the parity of n. It is enough to return whether n is even.

- When n=1, the first player loses.
- When n=2, the first player takes 1, leaving 1, the second player loses, the first player wins.
- When n=3, the first player takes 1, leaving 2, the second player wins, the first player loses.
- When n=4, the first player takes 1, leaving 3, the second player loses, the first player wins.
- ...

We conjecture that when n is odd, the first player loses; when n is even, the first player wins.

Proof:

1. If n=1 or n=2, the conclusion holds.
1. If n > 2, assume that the conclusion holds when n ≤ k, then when n=k+1:
   - If k+1 is odd, since x is a divisor of k+1, then x can only be odd, so k+1-x is even, the second player wins, the first player loses.
   - If k+1 is even, now x can be either odd 1 or even. If x is odd, then k+1-x is odd, the second player loses, the first player wins.

In conclusion, when n is odd, the first player loses; when n is even, the first player wins. The conclusion is correct.

The time complexity is O(1), and the space complexity is O(1).

Python 3

```
class Solution:
    def divisorGame(self, n: int) -> bool:
        return n % 2 == 0
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return a winning first move (or None)
# 2. The chosen x must be a prime divisor of n (or 1)


# ============================================================
# FOLLOW-UP 1: Return a winning first move (or None)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Build the win table up to n; a move x wins when
# n - x is a losing number.
# Time: O(n^2).  Space: O(n).
def winning_first_move(n):
    win = [False] * (n + 1)
    for i in range(2, n + 1):
        for x in range(1, i):
            if i % x == 0 and not win[i - x]:
                win[i] = True
                break
    for x in range(1, n):
        if n % x == 0 and not win[n - x]:
            return x
    return None


# ============================================================
# FOLLOW-UP 2: The chosen x must be a prime divisor of n
# (or 1)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same table, only trying moves x that are 1 or
# prime.
# Time: O(n^2 + n * D * sqrt n), D = divisors.  Space:
# O(n).
def can_win_with_prime_moves(n):
    def is_prime(x):
        if x < 2:
            return False
        for d in range(2, int(x ** 0.5) + 1):
            if x % d == 0:
                return False
        return True

    win = [False] * (n + 1)
    for i in range(2, n + 1):
        for x in range(1, i):
            if i % x == 0 and (x == 1 or is_prime(x)) and not win[i - x]:
                win[i] = True
                break
    return win[n]
```

293. Flip Game

Easy · String

leetcode.com/problems/flip-game

PROBLEM

You are playing a Flip Game with your friend.

You are given a string `currentState` that contains only `'+'` and `'-'`. You and your friend take turns to flip **two consecutive** `"++"` into `"--"`. The game ends when a person can no longer make a move, and therefore the other person will be the winner.

Return all possible states of the string `currentState` after **one valid move**. You may return the answer in **any order**. If there is no valid move, return an empty list `[]`.

Example 1:

```
Input: currentState = "++++"
Output: ["--++","+--+","++--"]
```

Example 2:

```
Input: currentState = "+"
Output: []
```

Constraints:

- `1 <= currentState.length <= 500`
- `currentState[i]` is either `'+'` or `'-'`.

SOLUTION (PYTHON 3)

Solution 1: Traversal + Simulation

> **How to think about it**
>
> A move flips one occurrence of ++ to --. Scan adjacent pairs, flip each ++, record the string, and restore.

We traverse the string. If the current character and the next character are both `+`, we change these two characters to `-`, add the result to the result array, and then change these two characters back to `+`.

After the traversal ends, we return the result array.

The time complexity is $O(n^2)$, where n is the length of the string. Ignoring the space complexity of the result array, the space complexity is O(n) or O(1).

Python 3

```python
class Solution:
    def generatePossibleNextMoves(self, currentState: str) -> List[str]:
        s = list(currentState)
        ans = []
        for i, (a, b) in enumerate(pairwise(s)):
            if a == b == "+":
                s[i] = s[i + 1] = "-"
                ans.append("".join(s))
                s[i] = s[i + 1] = "+"
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Can the first player guarantee a win (players keep
# flipping '++' to '--', the one who cannot move loses)
# 2. Count the first moves that win


# ============================================================
# FOLLOW-UP 1: Can the first player guarantee a win
# (players keep flipping '++' to '--', the one who cannot
# move loses)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Try every '++', and win if the resulting string
# is a loss for the opponent; remember the strings already
# solved.
# Time: O(2^n * n^2) (string slicing for every state).
# Space: O(2^n * n).
def can_first_player_win(state):
    memo = {}

    def win(s):
        if s in memo:
            return memo[s]
        result = False
        for i in range(len(s) - 1):
            if s[i] == "+" and s[i + 1] == "+":
                if not win(s[:i] + "--" + s[i + 2:]):
                    result = True
                    break
        memo[s] = result
        return result

    return win(state)


# ============================================================
# FOLLOW-UP 2: Count the first moves that win
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the positions i whose flip leaves a losing
# string for the opponent.
# Time: O(2^n * n^2).  Space: O(2^n * n).
def count_winning_first_moves(state):
    memo = {}

    def win(s):
        if s in memo:
            return memo[s]
        result = False
        for i in range(len(s) - 1):
            if s[i] == "+" and s[i + 1] == "+":
                if not win(s[:i] + "--" + s[i + 2:]):
                    result = True
                    break
        memo[s] = result
        return result

    count = 0
```

```
    for i in range(len(state) - 1):
        if state[i] == "+" and state[i + 1] == "+":
            if not win(state[:i] + "--" + state[i + 2:]):
                count += 1
```

```
    return count
```

PATTERN 39 · GAME THEORY / MINIMAX · PROBLEM 4 OF 20

877. Stone Game

Medium · Minimax, Array, Math, Dynamic Programming, Game Theory, Zero-Sum Game

leetcode.com/problems/stone-game

★ **TOP 7 PROBLEM** in Game Theory / Minimax: master this one first

PROBLEM

Alice and Bob play a game with piles of stones. There are an **even** number of piles arranged in a row, and each pile has a **positive** integer number of stones `piles[i]`.

The objective of the game is to end with the most stones. The **total** number of stones across all the piles is **odd**, so there are no ties.

Alice and Bob take turns, with **Alice starting first**. Each turn, a player takes the entire pile of stones either from the **beginning** or from the **end** of the row. This continues until there are no more piles left, at which point the person with the **most stones wins**.

Assuming Alice and Bob play optimally, return `true` *if Alice wins the game, or* `false` *if Bob wins*.

Example 1:

```
Input: piles = [5,3,4,5]
Output: true
Explanation:
Alice starts first, and can only take the first 5 or the last 5.
Say she takes the first 5, so that the row becomes [3, 4, 5].
If Bob takes 3, then the board is [4, 5], and Alice takes 5 to win with 10 points.
If Bob takes the last 5, then the board is [3, 4], and Alice takes 4 to win with 9 points.
This demonstrated that taking the first 5 was a winning move for Alice, so we return true.
```

Example 2:

```
Input: piles = [3,7,2,3]
Output: true
```

Constraints:

- `2 <= piles.length <= 500`
- `piles.length` is **even**.
- `1 <= piles[i] <= 500`
- `sum(piles[i])` is **odd**.

SOLUTIONS (PYTHON 3)

Solution 1: Memoization Search

How to think about it

Players take from either end of a zero-sum pile array; we ask whether Alice scores more. With ≤ 500 piles, the state is the net score on interval [i,j]. dfs(i,j) is the max of piles[i]-dfs(i+1,j) and piles[j]-dfs(i,j-1). A positive net score wins. Alice always wins under the constraints, but the code still computes the interval DP.

We design a function dfs(i, j) that represents the maximum difference in the number of stones between the current player and the other player when considering piles from the i-th to the j-th. The answer is then dfs(0, n - 1) > 0.

The function dfs(i, j) is computed as follows:

- If i > j, there are no stones left, so the current player cannot take any stones and the difference is 0, i.e., dfs(i, j) = 0.

- Otherwise, the current player has two choices: if they take the i-th pile, the difference between the current player and the other player is piles[i] - dfs(i + 1, j); if they take the j-th pile, the difference is piles[j] - dfs(i, j - 1). The current player will choose the option with the larger difference, so dfs(i, j) = max(piles[i] - dfs(i + 1, j), piles[j] - dfs(i, j - 1)).

Finally, we only need to check whether dfs(0, n - 1) > 0.

To avoid redundant computation, we can use memoization: we use an array f to record all values of dfs(i, j), so that when the function is called again, we can directly retrieve the answer from f without recomputing.

The time complexity is $O(n^2)$ and the space complexity is $O(n^2)$, where n is the number of stone piles.

Python 3

```
class Solution:
    def stoneGame(self, piles: List[int]) -> bool:
        @cache
        def dfs(i: int, j: int) -> int:
            if i > j:
                return 0
            return max(piles[i] - dfs(i + 1, j), piles[j] - dfs(i, j - 1))

        return dfs(0, len(piles) - 1) > 0
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> The memoized recurrence tabulates without recursion. f[i][j] is the net score on [i,j], filled from the length-1 diagonal outward. The transition matches Method 1. The answer is whether f[0][n-1] is positive.

We can also use dynamic programming. Define f[i][j] as the maximum difference in the number of stones the current player can obtain over the other player from piles piles[i..j]. The final answer is then f[0][n - 1] > 0.

Initially, f[i][i] = piles[i], because with only one pile, the current player can only take that pile, and the difference is piles[i].

For f[i][j] where i < j, there are two cases:

- If the current player takes pile piles[i], the remaining piles are piles[i + 1..j], and it becomes the other player's turn, so f[i][j] = piles[i] - f[i + 1][j].
- If the current player takes pile piles[j], the remaining piles are piles[i..j - 1], and it becomes the other player's turn, so f[i][j] = piles[j] - f[i][j - 1].

Therefore, the final state transition equation is f[i][j] = max(piles[i] - f[i + 1][j], piles[j] - f[i][j - 1]).

Finally, we only need to check whether f[0][n - 1] > 0.

The time complexity is $O(n^2)$ and the space complexity is $O(n^2)$, where n is the number of stone piles.

Similar problems:

- 486. Predict the Winner

Python 3

```
class Solution:
    def stoneGame(self, piles: List[int]) -> bool:
        n = len(piles)
        f = [[0] * n for _ in range(n)]
        for i, x in enumerate(piles):
            f[i][i] = x
        for i in range(n - 2, -1, -1):
            for j in range(i + 1, n):
                f[i][j] = max(piles[i] - f[i + 1][j], piles[j] - f[i][j - 1])
        return f[0][n - 1] > 0
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The exact best score difference (Alice minus Bob)
# with optimal play
# 2. How many stones does each player end with


# ============================================================
# FOLLOW-UP 1: The exact best score difference (Alice
# minus Bob) with optimal play
# ============================================================

# ---- follow-up 1: function ----
# Answer: diff[i][j] is the best advantage for the player
# to move on piles i..j: take an end and subtract what the
# opponent then gets.
# Time: O(n^2).  Space: O(n^2).
def best_difference(piles):
    n = len(piles)
    diff = [[0] * n for _ in range(n)]
    for i in range(n):
        diff[i][i] = piles[i]
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            diff[i][j] = max(piles[i] - diff[i + 1][j], piles[j] - diff[i][j - 1])
    return diff[0][n - 1]


# ============================================================
# FOLLOW-UP 2: How many stones does each player end with
# ============================================================

# ---- follow-up 2: function ----
# Answer: With total T and difference D, Alice has (T + D)
# / 2 and Bob has (T - D) / 2.
# Time: O(n^2).  Space: O(n^2).
def final_totals(piles):
    difference = best_difference(piles)
    total = sum(piles)
    return (total + difference) // 2, (total - difference) // 2
```

1140. Stone Game II

Medium · Minimax, Array, Math, Dynamic Programming, Game Theory, Prefix Sum, Zero-Sum Game

leetcode.com/problems/stone-game-ii

★ **TOP 7 PROBLEM** in Game Theory / Minimax: master this one first

PROBLEM

Alice and Bob continue their games with piles of stones. There are a number of piles **arranged in a row**, and each pile has a positive integer number of stones `piles[i]`. The objective of the game is to end with the most stones.

Alice and Bob take turns, with Alice starting first.

On each player's turn, that player can take **all the stones** in the **first** `X` remaining piles, where `1 <= X <= 2M`. Then, we set `M = max(M, X)`. Initially, M = 1.

The game continues until all the stones have been taken.

Assuming Alice and Bob play optimally, return the maximum number of stones Alice can get.

Example 1:

Input: piles = [2,7,9,4,4]

Output: 10

Explanation:

- If Alice takes one pile at the beginning, Bob takes two piles, then Alice takes 2 piles again. Alice can get `2 + 4 + 4 = 10` stones in total.
- If Alice takes two piles at the beginning, then Bob can take all three piles left. In this case, Alice get `2 + 7 = 9` stones in total.

So we return 10 since it's larger.

Example 2:

Input: piles = [1,2,3,4,5,100]

Output: 104

Constraints:

- `1 <= piles.length <= 100`
- `1 <= piles[i] <= 10^4`

SOLUTIONS (PYTHON 3)

Solution 1: Prefix Sum + Memoization Search

How to think about it

Players take 1..2M piles from the remaining prefix; the state is the start index and M. Naive search repeats subproblems. A prefix sum is the suffix total from i; the current player gets that total minus the opponent's optimum. If 2M covers the rest, take it all; otherwise try each X with memoization. Alice starts at (0,1).

Since the player can take all the stones from the first X piles each time, that is, they can take the stones from an interval, we can first preprocess a prefix sum array s of length n+1, where s[i] represents the sum of the first i elements of the array `piles`.

Then we design a function dfs(i, m), which represents the maximum number of stones that the current player can take when they can start from index i of the array `piles`, and the current M is m. Initially, Alice starts from index 0, and M=1, so the answer we need to find is dfs(0, 1).

The calculation process of the function dfs(i, m) is as follows:

- If the current player can take all the remaining stones, the maximum number of stones they can take is s[n] - s[i];
- Otherwise, the current player can take all the stones from the first x piles of the remaining ones, where $1 \le x \le 2m$, and the maximum number of stones they can take is s[n] - s[i] - dfs(i + x, max(m, x)). That is to say, the number of stones that the current player can take is the number of all the remaining stones minus the number of stones that the opponent can take in the next round. We need to enumerate all x, and take the maximum value as the return value of the function dfs(i, m).

To avoid repeated calculations, we can use memoization search.

Finally, we return dfs(0, 1) as the answer.

The time complexity is $O(n^3)$, and the space complexity is $O(n^2)$. Here, n is the length of the array `piles`.

Python 3

```
class Solution:
    def stoneGameII(self, piles: List[int]) -> int:
        @cache
        def dfs(i, m):
            if m * 2 >= n - i:
                return s[n] - s[i]
            return max(
                s[n] - s[i] - dfs(i + x, max(m, x)) for x in range(1, m << 1 | 1)
            )

        n = len(piles)
        s = list(accumulate(piles, initial=0))
        return dfs(0, 1)
```

Solution 2

> **How to think about it**
>
> Method 1 returns the suffix sum when everything can be taken. Method 2 always enumerates X and writes the same optimum as suffix minus the opponent's minimum, with an out-of-range base case of 0. The state is unchanged; the code is more uniform.

Python 3

```
class Solution:
    def stoneGameII(self, piles: List[int]) -> int:
        @cache
        def dfs(i: int, m: int = 1) -> int:
            if i >= len(piles):
                return 0
            t = inf
            for x in range(1, m << 1 | 1):
                t = min(t, dfs(i + x, max(m, x)))
            return s[-1] - s[i] - t

        s = list(accumulate(piles, initial=0))
        return dfs(0)
```

Complexity. Time: O(n^3). Space: O(n^2).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Alice may take up to 3M piles instead of 2M
# 2. How many stones does Bob end with


# ============================================================
# FOLLOW-UP 1: Alice may take up to 3M piles instead of 2M
# ============================================================
```

```
# ---- follow-up 1: function ----
# Answer: The same memoised search with the limit changed
# to factor * M.
# Time: O(f * n^3) in the worst case, f = factor.  Space:
# O(n^2).
def alice_stones_with_factor(piles, factor):
    n = len(piles)
    suffix = [0] * (n + 1)
    for i in range(n - 1, -1, -1):
        suffix[i] = suffix[i + 1] + piles[i]
    memo = {}

    def best(i, m):
        if i >= n:
            return 0
        if (i, m) in memo:
            return memo[(i, m)]
        if i + factor * m >= n:
            memo[(i, m)] = suffix[i]
            return suffix[i]
        result = 0
        for x in range(1, factor * m + 1):
            result = max(result, suffix[i] - best(i + x, max(m, x)))
        memo[(i, m)] = result
        return result

    return best(0, 1)


# ============================================================
# FOLLOW-UP 2: How many stones does Bob end with
# ============================================================

# ---- follow-up 2: function ----
# Answer: Bob gets everything Alice does not.
# Time: O(n^3).  Space: O(n^2).
def bob_stones(piles):
    return sum(piles) - alice_stones_with_factor(piles, 2)
```

1686. Stone Game VI

Medium · Greedy, Minimax, Array, Math, Game Theory, Sorting, Heap (Priority Queue), Zero-Sum Game

leetcode.com/problems/stone-game-vi

PROBLEM

Alice and Bob take turns playing a game, with Alice starting first.

There are `n` stones in a pile. On each player's turn, they can **remove** a stone from the pile and receive points based on the stone's value. Alice and Bob may **value the stones differently**.

You are given two integer arrays of length `n`, `aliceValues` and `bobValues`. Each `aliceValues[i]` and `bobValues[i]` represents how Alice and Bob, respectively, value the i^{th} stone.

The winner is the person with the most points after all the stones are chosen. If both players have the same amount of points, the game results in a draw. Both players will play **optimally**. Both players know the other's values.

Determine the result of the game, and:

- If Alice wins, return `1`.
- If Bob wins, return `-1`.
- If the game results in a draw, return `0`.

Example 1:

```
Input: aliceValues = [1,3], bobValues = [2,1]
Output: 1
Explanation:
If Alice takes stone 1 (0-indexed) first, Alice will receive 3 points.
Bob can only choose stone 0, and will only receive 2 points.
Alice wins.
```

Example 2:

```
Input: aliceValues = [1,2], bobValues = [3,1]
Output: 0
Explanation:
If Alice takes stone 0, and Bob takes stone 1, they will both have 1 point.
Draw.
```

Example 3:

```
Input: aliceValues = [2,4,3], bobValues = [1,6,7]
Output: -1
Explanation:
Regardless of how Alice plays, Bob will be able to have more points than Alice.
For example, if Alice takes stone 1, Bob can take stone 2, and Alice takes stone 0, Alice will have 6
points to Bob's 7.
Bob wins.
```

Constraints:

- `n == aliceValues.length == bobValues.length`
- `1 <= n <= 10^5`
- `1 <= aliceValues[i], bobValues[i] <= 100`

SOLUTION (PYTHON 3)

Solution 1: Greedy + Sorting

> **How to think about it**
>
> Players alternate taking a stone, scoring their own value and denying the opponent's. The comparison should count both “I gain a_i” and “they lose b_i”, so take stones in decreasing a_i+b_i. After that sort Alice takes even indices and Bob odd ones; compare the two scores for 1, 0, or -1.

The optimal strategy for picking stones is to maximize one's own score while making the opponent lose as much as possible. Therefore, we create an array vals, where vals[i] = (aliceValues[i] + bobValues[i], i) represents the total value and index of the i-th stone. Then we sort vals in descending order by total value.

Next, we let Alice and Bob pick stones alternately according to the order of vals. Alice picks the stones at even positions in vals, and Bob picks the stones at odd positions in vals. Finally, we compare the scores of Alice and Bob and return the corresponding result.

The time complexity is O(n × log n), and the space complexity is O(n), where n is the length of the arrays `aliceValues` and `bobValues`.

Python 3

```python
class Solution:
    def stoneGameVI(self, aliceValues: List[int], bobValues: List[int]) -> int:
        vals = [(a + b, i) for i, (a, b) in enumerate(zip(aliceValues, bobValues))]
        vals.sort(reverse=True)
        a = sum(aliceValues[i] for _, i in vals[::2])
        b = sum(bobValues[i] for _, i in vals[1::2])
        if a > b:
            return 1
        if a < b:
            return -1
        return 0
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the final scores of Alice and Bob
# 2. Bob moves first


# ============================================================
# FOLLOW-UP 1: Return the final scores of Alice and Bob
# ============================================================

# ---- follow-up 1: function ----
# Answer: Both take the stone with the biggest sum of
# values, alternately; add the value each really gets.
# Time: O(n log n).  Space: O(n).
def final_scores(alice_values, bob_values):
    order = sorted(range(len(alice_values)), key=lambda i: -(alice_values[i] + bob_values[i]))
    alice = 0
    bob = 0
    for turn in range(len(order)):
        if turn % 2 == 0:
            alice += alice_values[order[turn]]
        else:
            bob += bob_values[order[turn]]
    return alice, bob


# ============================================================
# FOLLOW-UP 2: Bob moves first
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Same ordering, but Bob takes the even turns.
# Time: O(n log n).  Space: O(n).
def final_scores_bob_first(alice_values, bob_values):
    order = sorted(range(len(alice_values)), key=lambda i: -(alice_values[i] + bob_values[i]))
    alice = 0
    bob = 0
    for turn in range(len(order)):
        if turn % 2 == 0:
            bob += bob_values[order[turn]]
        else:
            alice += alice_values[order[turn]]
    return alice, bob
```

PATTERN 39 · GAME THEORY / MINIMAX · PROBLEM 7 OF 20

1690. Stone Game VII

Medium · Minimax, Array, Math, Dynamic Programming, Game Theory, Zero-Sum Game

leetcode.com/problems/stone-game-vii

PROBLEM

Alice and Bob take turns playing a game, with **Alice starting first**.

There are `n` stones arranged in a row. On each player's turn, they can **remove** either the leftmost stone or the rightmost stone from the row and receive points equal to the **sum** of the remaining stones' values in the row. The winner is the one with the higher score when there are no stones left to remove.

Bob found that he will always lose this game (poor Bob, he always loses), so he decided to **minimize the score's difference**. Alice's goal is to **maximize the difference** in the score.

Given an array of integers `stones` where `stones[i]` represents the value of the i^{th} stone **from the left**, return *the **difference** in Alice and Bob's score if they both play **optimally**.*

Example 1:

```
Input: stones = [5,3,1,4,2]
Output: 6
Explanation:
- Alice removes 2 and gets 5 + 3 + 1 + 4 = 13 points. Alice = 13, Bob = 0, stones = [5,3,1,4].
- Bob removes 5 and gets 3 + 1 + 4 = 8 points. Alice = 13, Bob = 8, stones = [3,1,4].
- Alice removes 3 and gets 1 + 4 = 5 points. Alice = 18, Bob = 8, stones = [1,4].
- Bob removes 1 and gets 4 points. Alice = 18, Bob = 12, stones = [4].
- Alice removes 4 and gets 0 points. Alice = 18, Bob = 12, stones = [].
The score difference is 18 - 12 = 6.
```

Example 2:

```
Input: stones = [7,90,5,1,100,10,10,2]
Output: 122
```

Constraints:

- `n == stones.length`
- `2 <= n <= 1000`
- `1 <= stones[i] <= 1000`

SOLUTIONS (PYTHON 3)

Solution 1: Memorization Search

> **How to think about it**
>
> Each turn removes one end and scores the sum of what remains. The first-player minus second-player gap on an interval is a memoized game: dfs(i,j) is the current player's advantage. Prefix sums s give the score after dropping the left or right end; take the larger of s[j+1]-s[i+1]-dfs(i+1,j) and s[j]-s[i]-dfs(i,j-1).

First, we preprocess to get the prefix sum array s, where s[i] represents the total sum of the first i stones.

Next, we design a function dfs(i, j), which represents the score difference between the first and second players when the remaining stones are stones[i], stones[i + 1], …, stones[j]. The answer is dfs(0, n - 1).

The calculation process of the function dfs(i, j) is as follows:

- If i > j, it means there are no stones currently, so return 0;
- Otherwise, the first player has two choices, which are to remove stones[i] or stones[j], and then calculate the score difference, i.e., a = s[j + 1] - s[i + 1] - dfs(i + 1, j) and b = s[j] - s[i] - dfs(i, j - 1). We take the larger of the two as the return

value of dfs(i, j).

During the process, we use memorization search, i.e., use an array f to record the return value of the function dfs(i, j), to avoid repeated calculations.

The time complexity is $O(n^2)$, and the space complexity is $O(n^2)$. Here, n is the number of stones.

Python 3

```
class Solution:
    def stoneGameVII(self, stones: List[int]) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            if i > j:
                return 0
            a = s[j + 1] - s[i + 1] - dfs(i + 1, j)
            b = s[j] - s[i] - dfs(i, j - 1)
            return max(a, b)

        s = list(accumulate(stones, initial=0))
        ans = dfs(0, len(stones) - 1)
        dfs.cache_clear()
        return ans
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> Solution 1 already fills this interval with decreasing i and increasing j. The f[i][j] below is that same table.

Solution 1 already fills this transition with decreasing i and increasing j. Define f[i][j] as the score difference between the first and second players when the remaining stones are stones[i], stones[i + 1], …, stones[j]. Therefore, the answer is f[0][n - 1].

The state transition equation is as follows:

$$f[i][j] = \max(s[j + 1] - s[i + 1] - f[i + 1][j], s[j] - s[i] - f[i][j - 1])$$

When calculating f[i][j], we need to ensure that f[i + 1][j] and f[i][j - 1] have been calculated. Therefore, we need to enumerate i in descending order and j in ascending order.

Finally, the answer is f[0][n - 1].

The time complexity is $O(n^2)$, and the space complexity is $O(n^2)$. Here, n is the number of stones.

Python 3

```
class Solution:
    def stoneGameVII(self, stones: List[int]) -> int:
        s = list(accumulate(stones, initial=0))
        n = len(stones)
        f = [[0] * n for _ in range(n)]
        for i in range(n - 2, -1, -1):
            for j in range(i + 1, n):
                a = s[j + 1] - s[i + 1] - f[i + 1][j]
                b = s[j] - s[i] - f[i][j - 1]
                f[i][j] = max(a, b)
        return f[0][-1]
```

Solution 3: Dynamic Programming

> **How to think about it**
>
> Enumerating both players' choices is exponential. There can be 1000 stones, and the branch that removes the left stone first always advances the left endpoint, so the search has depth n. The current player's advantage on [i,j] depends only on shorter intervals. Let f[i][j] store that difference, with a single stone worth 0, and fill decreasing i and increasing j: removing the left end scores s[j+1]-s[i+1]-f[i+1][j], removing the right end scores s[j]-s[i]-f[i][j-1], and the state keeps the larger one.

Precompute the prefix sums s, where s[i] is the sum of the first i stones. Let f[i][j] be the score difference between the current player and the opponent on stones[i], stones[i + 1], …, stones[j]. A single stone has difference 0. For i from n-2 down to 0, and for each i with j from i+1 to n-1, the shorter states f[i+1][j] and f[i][j-1] are already known. Removing the left stone scores s[j+1]-s[i+1]-f[i+1][j], and removing the right stone scores s[j]-s[i]-f[i][j-1]. f[i][j] is the larger of the two. The answer is f[0][n-1].

The time complexity is $O(n^2)$, and the space complexity is $O(n^2)$. Here, n is the number of stones.

Python 3

```
class Solution:
    def stoneGameVII(self, stones: List[int]) -> int:
        s = list(accumulate(stones, initial=0))
        n = len(stones)
        f = [[0] * n for _ in range(n)]
        for i in range(n - 2, -1, -1):
            for j in range(i + 1, n):
                a = s[j + 1] - s[i + 1] - f[i + 1][j]
                b = s[j] - s[i] - f[i][j - 1]
                f[i][j] = max(a, b)
        return f[0][-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Use only O(n) memory
# 2. Which end should Alice remove first


# ============================================================
# FOLLOW-UP 1: Use only O(n) memory
# ============================================================

# ---- follow-up 1: function ----
# Answer: Row i of the table only needs row i + 1, so one
# list is enough when i goes from the right to the left.
# Time: O(n^2).  Space: O(n).
def stone_game_vii_one_row(stones):
    n = len(stones)
    prefix = [0]
    for x in stones:
        prefix.append(prefix[-1] + x)
    row = [0] * n
    for i in range(n - 2, -1, -1):
        for j in range(i + 1, n):
            remove_left = prefix[j + 1] - prefix[i + 1] - row[j]
            remove_right = prefix[j] - prefix[i] - row[j - 1]
            row[j] = max(remove_left, remove_right)
    return row[n - 1]


# ============================================================
# FOLLOW-UP 2: Which end should Alice remove first
# ============================================================

# ---- follow-up 2: function ----
# Answer: Compare the two options of the first move: left
# (best score minus what the opponent then gets) and
# right.
# Time: O(n^2).  Space: O(n^2).
def alice_first_move(stones):
    n = len(stones)
    prefix = [0]
    for x in stones:
        prefix.append(prefix[-1] + x)
    memo = {}
```

```
    def best(i, j):
        if i >= j:
            return 0
        if (i, j) in memo:
            return memo[(i, j)]
        left = prefix[j + 1] - prefix[i + 1] - best(i + 1, j)
        right = prefix[j] - prefix[i] - best(i, j - 1)
        memo[(i, j)] = max(left, right)
        return memo[(i, j)]

    left = prefix[n] - prefix[1] - best(1, n - 1)
    right = prefix[n - 1] - prefix[0] - best(0, n - 2)
    if left >= right:
        return "left"
    return "right"
```

486. Predict the Winner

Medium · Recursion, Minimax, Array, Math, Dynamic Programming, Game Theory, Zero-Sum Game

leetcode.com/problems/predict-the-winner

★ **TOP 7 PROBLEM** in Game Theory / Minimax: master this one first

PROBLEM

You are given an integer array `nums`.

Two players are playing a game with this array: Player 1 and Player 2.

Player 1 and Player 2 take turns, with Player 1 starting first. Both players start the game with a score of 0.

At each turn, the current player takes the number at **either end** of the array (i.e., `nums[0]` or `nums[nums.length - 1]`), **removing** it from the array and adding it to their own score. The game ends when there are no more elements in the array.

Return `true` if Player 1's final score is **greater than or equal** to Player 2's final score, and `false` otherwise.

Note that a tie counts as a win for Player 1. You may assume that both players play optimally.

Example 1:

Input: nums = [1,5,2]

Output: false

Explanation:

- Player 1 must open by taking either 1 or 2, and both choices lead to the same result.
- If Player 1 takes 1, the array becomes `[5,2]`. Player 2 takes 5, leaving 2 for Player 1.
- If Player 1 takes 2, the array becomes `[1,5]`. Player 2 takes 5, leaving 1 for Player 1.
- Either way, Player 1 finishes with `1 + 2 = 3` and Player 2 finishes with `5`. Player 1 can never win, so return `false`.

Example 2:

Input: nums = [1,5,233,7]

Output: true

Explanation:

- Player 1 first takes 1, leaving `[5,233,7]`. Player 2 must then choose between 5 and 7, and no matter which number Player 2 chooses, Player 1 can take 233 on the next turn.
- Player 1 finishes with a higher score (234) than Player 2 (12), so return `true`.

Constraints:

- `1 <= nums.length <= 20`
- `0 <= nums[i] <= 10^7`

SOLUTIONS (PYTHON 3)

Solution 1: Memoization Search

How to think about it

Players take from the two ends; the first player wins if her score is at least the second's. The game tree repeats the same interval. dfs(i,j) is the best score gap on [i,j]: take the left end for nums[i]-dfs(i+1,j), or the right end symmetrically. The first player wins iff dfs(0,n-1)≥ 0. There are $O(n^2)$ intervals; memoization makes every leftover interval optimal for the player to move.

We design a function dfs(i, j), which represents the maximum difference in scores between the current player and the other player from the i-th number to the j-th number. The answer is dfs(0, n - 1) ≥ 0.

The function dfs(i, j) is calculated as follows:

- If i > j, it means there are no numbers left, so the current player cannot take any points, and the difference is 0, i.e., dfs(i, j) = 0.
- Otherwise, the current player has two choices. If they choose the i-th number, the difference in scores between the current player and the other player is nums[i] - dfs(i + 1, j). If they choose the j-th number, the difference in scores between the current player and the other player is nums[j] - dfs(i, j - 1). The current player will choose the option with the larger difference, so dfs(i, j) = max(nums[i] - dfs(i + 1, j), nums[j] - dfs(i, j - 1)).

Finally, we only need to check if dfs(0, n - 1) ≥ 0.

To avoid repeated calculations, we can use memoization. We use an array f to record all the values of dfs(i, j). When the function is called again, we can directly retrieve the answer from f without recalculating it.

The time complexity is $O(n^2)$, and the space complexity is $O(n^2)$. Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def predictTheWinner(self, nums: List[int]) -> bool:
        @cache
        def dfs(i: int, j: int) -> int:
            if i > j:
                return 0
            return max(nums[i] - dfs(i + 1, j), nums[j] - dfs(i, j - 1))

        return dfs(0, len(nums) - 1) >= 0
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> The same recurrence fills a table: f[i][j] starts from the length-1 diagonal, then grows by decreasing i and increasing j. No call stack, same asymptotics.

We can also use dynamic programming. Define f[i][j] to represent the maximum score difference the current player can achieve in the range nums[i..j]. The final answer is f[0][n - 1] ≥ 0.

Initially, f[i][i] = nums[i], because with only one number, the current player can only take that number, and the score difference is nums[i].

Consider f[i][j] where i < j, there are two cases:

- If the current player takes nums[i], the remaining numbers are nums[i + 1..j], and it is the other player's turn. So, f[i][j] = nums[i] - f[i + 1][j].
- If the current player takes nums[j], the remaining numbers are nums[i..j - 1], and it is the other player's turn. So, f[i][j] = nums[j] - f[i][j - 1].

Therefore, the state transition equation is f[i][j] = max(nums[i] - f[i + 1][j], nums[j] - f[i][j - 1]).

Finally, we only need to check if f[0][n - 1] ≥ 0.

The time complexity is $O(n^2)$, and the space complexity is $O(n^2)$. Here, n is the length of the array nums.

Similar problem:

- 877. Stone Game

Python 3

```python
class Solution:
    def predictTheWinner(self, nums: List[int]) -> bool:
        n = len(nums)
        f = [[0] * n for _ in range(n)]
        for i, x in enumerate(nums):
            f[i][i] = x
        for i in range(n - 2, -1, -1):
            for j in range(i + 1, n):
                f[i][j] = max(nums[i] - f[i + 1][j], nums[j] - f[i][j - 1])
        return f[0][n - 1] >= 0
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the winning margin (Player 1 minus Player 2)
# 2. Each player's total score


# ============================================================
# FOLLOW-UP 1: Return the winning margin (Player 1 minus
# Player 2)
# ============================================================

# ---- follow-up 1: function ----
# Answer: diff[i][j] is the best advantage for the player
# to move on nums[i..j].
# Time: O(n^2).  Space: O(n^2).
def winning_margin(nums):
    n = len(nums)
    diff = [[0] * n for _ in range(n)]
    for i in range(n):
        diff[i][i] = nums[i]
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            diff[i][j] = max(nums[i] - diff[i + 1][j], nums[j] - diff[i][j - 1])
    return diff[0][n - 1]


# ============================================================
# FOLLOW-UP 2: Each player's total score
# ============================================================

# ---- follow-up 2: function ----
# Answer: With sum S and margin D, player 1 gets (S + D) /
# 2.
# Time: O(n^2).  Space: O(n^2).
def both_totals(nums):
    margin = winning_margin(nums)
    total = sum(nums)
    return (total + margin) // 2, (total - margin) // 2
```

464. Can I Win

Medium · Bit Manipulation, Memoization, Math, Dynamic Programming, Bitmask, Game Theory

leetcode.com/problems/can-i-win

★ **TOP 7 PROBLEM** in Game Theory / Minimax: master this one first

PROBLEM

In the "100 game" two players take turns adding, to a running total, any integer from `1` to `10`. The player who first causes the running total to **reach or exceed** 100 wins.

What if we change the game so that players **cannot** re-use integers?

For example, two players might take turns drawing from a common pool of numbers from 1 to 15 without replacement until they reach a total >= 100.

Given two integers `maxChoosableInteger` and `desiredTotal`, return `true` if the first player to move can force a win, otherwise, return `false`. Assume both players play **optimally**.

Example 1:

```
Input: maxChoosableInteger = 10, desiredTotal = 11
Output: false
Explanation:
No matter which integer the first player choose, the first player will lose.
The first player can choose an integer from 1 up to 10.
If the first player choose 1, the second player can only choose integers from 2 up to 10.
The second player will win by choosing 10 and get a total = 11, which is >= desiredTotal.
Same with other integers chosen by the first player, the second player will always win.
```

Example 2:

```
Input: maxChoosableInteger = 10, desiredTotal = 0
Output: true
```

Example 3:

```
Input: maxChoosableInteger = 10, desiredTotal = 1
Output: true
```

Constraints:

- `1 <= maxChoosableInteger <= 20`
- `0 <= desiredTotal <= 300`

SOLUTION (PYTHON 3)

Solution 1: State Compression + Memoization

How to think about it

Players draw without replacement from 1..n and the first to reach the target wins. n≤ 20, so the state is the used set; a raw game tree repeats positions. If the total is too small, nobody can win. Otherwise dfs(mask,s) tries each unused i: a win if s+i already meets the target or the opponent loses afterwards. Memoize on mask. A bit mask packs the set; at most 2^n subproblems. The total check skips a hopeless search.

First, we check if the sum of all selectable integers is less than the target value. If so, it means that we cannot win no matter what, so we directly return `false`.

Then, we design a function `dfs(mask, s)`, where `mask` represents the current state of the selected integers, and `s` represents the current cumulative sum. The return value of the function is whether the current player can win.

The execution process of the function `dfs(mask, s)` is as follows:

We iterate through each integer `i` from `1` to `maxChoosableInteger`. If `i` has not been selected, we can choose `i`. If the cumulative sum `s + i` after choosing `i` is greater than or equal to the target value `desiredTotal`, or if the result of the opponent choosing `i` is losing, then the current player is winning, return `true`.

If no choice can make the current player win, then the current player is losing, return `false`.

To avoid repeated calculations, we use a hash table `f` to record the calculated states, where the key is `mask`, and the value is whether the current player can win.

The time complexity is $O(2^n)$, and the space complexity is $O(2^n)$. Where n is `maxChoosableInteger`.

Python 3

```
class Solution:
    def canIWin(self, maxChoosableInteger: int, desiredTotal: int) -> bool:
        @cache
        def dfs(mask: int, s: int) -> bool:
            for i in range(1, maxChoosableInteger + 1):
                if mask >> i & 1 ^ 1:
                    if s + i >= desiredTotal or not dfs(mask | 1 << i, s + i):
                        return True
            return False

        if (1 + maxChoosableInteger) * maxChoosableInteger // 2 < desiredTotal:
            return False
        return dfs(0, 0)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. List all numbers that win as a first move
# 2. Numbers can be chosen again and again


# ============================================================
# FOLLOW-UP 1: List all numbers that win as a first move
# ============================================================

# ---- follow-up 1: function ----
# Answer: Try each x: you win at once if x reaches the
# total, or if the opponent loses from the remaining
# numbers.
# Time: O(2^M * M), M = max choosable.  Space: O(2^M).
def winning_first_moves(max_choosable, desired_total):
    memo = {}

    def win(used, remaining):
        if used in memo:
            return memo[used]
        result = False
        for x in range(1, max_choosable + 1):
            if not (used >> x) & 1:
                if x >= remaining or not win(used | (1 << x), remaining - x):
                    result = True
                    break
        memo[used] = result
        return result

    moves = []
    for x in range(1, max_choosable + 1):
        if x >= desired_total or not win(1 << x, desired_total - x):
            moves.append(x)
    return moves
```

```python
# ============================================================
# FOLLOW-UP 2: Numbers can be chosen again and again
# ============================================================

# ---- follow-up 2: function ----
# Answer: Now the state is only the remaining total.
# Time: O(T * M), T = desired total.  Space: O(T).
def can_win_with_reuse(max_choosable, desired_total):
    win = [False] * (max(desired_total, 0) + 1)
    for remaining in range(1, desired_total + 1):
        for x in range(1, max_choosable + 1):
            if x >= remaining or not win[remaining - x]:
                win[remaining] = True
                break
    return desired_total <= 0 or win[desired_total]
```

PATTERN 39 · GAME THEORY / MINIMAX · PROBLEM 10 OF 20

375. Guess Number Higher or Lower II

Medium · Minimax, Math, Dynamic Programming, Game Theory

leetcode.com/problems/guess-number-higher-or-lower-ii

★ **TOP 7 PROBLEM** in Game Theory / Minimax: master this one first

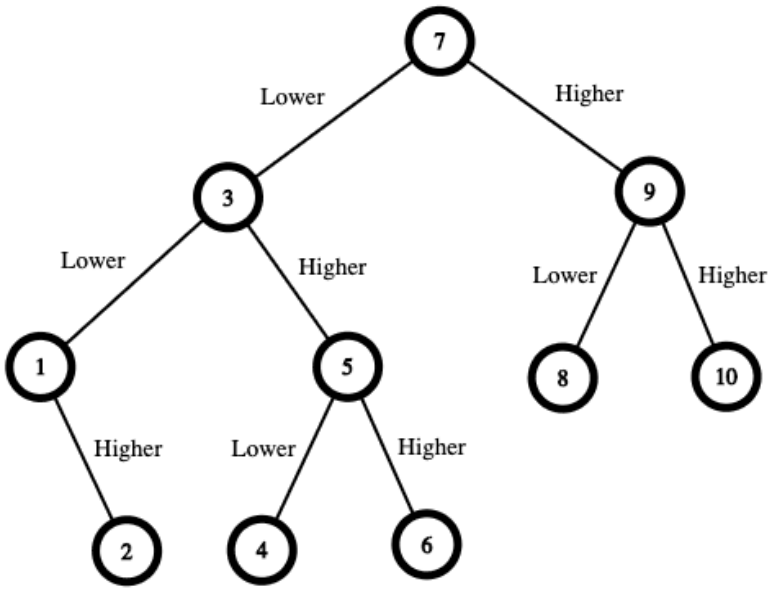
PROBLEM

We are playing the Guessing Game. The game will work as follows:

1. I pick a number between `1` and `n`.
2. You guess a number.
3. If you guess the right number, **you win the game**.
4. If you guess the wrong number, then I will tell you whether the number I picked is **higher or lower**, and you will continue guessing.
5. Every time you guess a wrong number `x`, you will pay `x` dollars. If you run out of money, **you lose the game**.

Given a particular `n`, return *the minimum amount of money you need to **guarantee a win regardless of what number I pick***.

Example 1:



```
Input: n = 10
Output: 16
Explanation: The winning strategy is as follows:
- The range is [1,10]. Guess 7.
    - If this is my number, your total is $0. Otherwise, you pay $7.
    - If my number is higher, the range is [8,10]. Guess 9.
        - If this is my number, your total is $7. Otherwise, you pay $9.
        - If my number is higher, it must be 10. Guess 10. Your total is $7 + $9 = $16.
        - If my number is lower, it must be 8. Guess 8. Your total is $7 + $9 = $16.
    - If my number is lower, the range is [1,6]. Guess 3.
        - If this is my number, your total is $7. Otherwise, you pay $3.
        - If my number is higher, the range is [4,6]. Guess 5.
            - If this is my number, your total is $7 + $3 = $10. Otherwise, you pay $5.
            - If my number is higher, it must be 6. Guess 6. Your total is $7 + $3 + $5 = $15.
            - If my number is lower, it must be 4. Guess 4. Your total is $7 + $3 + $5 = $15.
        - If my number is lower, the range is [1,2]. Guess 1.
            - If this is my number, your total is $7 + $3 = $10. Otherwise, you pay $1.
            - If my number is higher, it must be 2. Guess 2. Your total is $7 + $3 + $1 = $11.
The worst case in all these scenarios is that you pay $16. Hence, you only need $16 to guarantee a win.
```

Example 2:

```
Input: n = 1
Output: 0
Explanation: There is only one possible number, so you can guess 1 and not have to pay anything.
```

Example 3:

```
Input: n = 2
Output: 1
Explanation: There are two possible numbers, 1 and 2.
- Guess 1.
    - If this is my number, your total is $0. Otherwise, you pay $1.
    - If my number is higher, it must be 2. Guess 2. Your total is $1.
The worst case is that you pay $1.
```

Constraints:

- `1 <= n <= 200`

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

> **How to think about it**
>
> A wrong guess costs that number; we want the min-max cost to guarantee a win. Searching guess orders is a game tree. The optimum on [i,j] depends only on shorter intervals. f[i][j] is that min-max cost. Guessing k costs k+max(left,right); minimize over k. Fill by interval length; the answer is f[1][n].

We define f[i][j] as the minimum cost required to guess any number in the interval [i, j]. Initially, f[i][i] = 0 because there is no cost to guess the only number, and for i > j, we also have f[i][j] = 0. The answer is f[1][n].

For f[i][j], we can enumerate any number k in [i, j], divide the interval [i, j] into two parts, [i, k - 1] and [k + 1, j], choose the larger value of the two parts plus the cost of k,

Python 3

```
class Solution:
    def getMoneyAmount(self, n: int) -> int:
        f = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(n - 1, 0, -1):
            for j in range(i + 1, n + 1):
                f[i][j] = j + f[i][j - 1]
                for k in range(i, j):
                    f[i][j] = min(f[i][j], max(f[i][k - 1], f[k + 1][j]) + k)
        return f[1][n]
```

Complexity. Time: O(n^3). Space: O(n^2).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Which number should be guessed first
# 2. The money needed to be safe on any range low..high


# ============================================================
# FOLLOW-UP 1: Which number should be guessed first
# ============================================================

# ---- follow-up 1: function ----
# Answer: Return the smallest guess whose worst case
# equals the answer.
# Time: O(n^3).  Space: O(n^2).
def best_first_guess(n):
    cost = [[0] * (n + 2) for _ in range(n + 2)]
    for length in range(2, n + 1):
        for low in range(1, n - length + 2):
            high = low + length - 1
            best = float("inf")
            for guess in range(low, high):
                best = min(best, guess + max(cost[low][guess - 1], cost[guess + 1][high]))
            cost[low][high] = best
    best_cost = float("inf")
    best_guess = 1
    for guess in range(1, n + 1):
        worst = guess + max(cost[1][guess - 1] if guess > 1 else 0, cost[guess + 1][n] if guess < n else 0)
        if worst < best_cost:
            best_cost = worst
            best_guess = guess
    return best_guess


# ============================================================
# FOLLOW-UP 2: The money needed to be safe on any range
# low..high
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same table, read for the range.
# Time: O(r^3), r = high - low.  Space: O(r^2).
def cost_of_range(low, high):
    memo = {}

    def go(a, b):
        if a >= b:
            return 0
        if (a, b) in memo:
            return memo[(a, b)]
        best = float("inf")
        for guess in range(a, b):
            best = min(best, guess + max(go(a, guess - 1), go(guess + 1, b)))
        memo[(a, b)] = best
```

```
        return best
    return go(low, high)
```

1561. Maximum Number of Coins You Can Get

Medium · Greedy, Array, Math, Game Theory, Sorting

leetcode.com/problems/maximum-number-of-coins-you-can-get

PROBLEM

There are `3n` piles of coins of varying size, you and your friends will take piles of coins as follows:

- In each step, you will choose **any** `3` piles of coins (not necessarily consecutive).
- Of your choice, Alice will pick the pile with the maximum number of coins.
- You will pick the next pile with the maximum number of coins.
- Your friend Bob will pick the last pile.
- Repeat until there are no more piles of coins.

Given an array of integers `piles` where `piles[i]` is the number of coins in the i^{th} pile.

Return the maximum number of coins that you can have.

Example 1:

```
Input: piles = [2,4,1,2,7,8]
Output: 9
Explanation: Choose the triplet (2, 7, 8), Alice Pick the pile with 8 coins, you the pile with 7 coins and Bob the last one.
Choose the triplet (1, 2, 4), Alice Pick the pile with 4 coins, you the pile with 2 coins and Bob the last one.
The maximum number of coins which you can have are: 7 + 2 = 9.
On the other hand if we choose this arrangement (1, 2, 8), (2, 4, 7) you only get 2 + 4 = 6 coins which is not optimal.
```

Example 2:

```
Input: piles = [2,4,5]
Output: 4
```

Example 3:

```
Input: piles = [9,8,7,6,5,1,2,3,4]
Output: 18
```

Constraints:

- `3 <= piles.length <= 10^5`
- `piles.length % 3 == 0`
- `1 <= piles[i] <= 10^4`

SOLUTION (PYTHON 3)

Solution 1: Greedy + Sorting

> **How to think about it**
>
> Three players each take one pile per round: Alice the current largest, we the next, Bob the smallest. n reaches 10^5, so we should not simulate comparisons. After sorting, Bob always receives the smallest third. The remaining piles alternate between Alice and us; ours are every other value starting at index n/3. Sum them.

To maximize the number of coins we get, we can greedily let Bob take the smallest n piles of coins. Each time, we let Alice take the largest pile of coins, then we take the second largest pile of coins, and so on, until there are no more coins to take.

The time complexity is O(n × log n), and the space complexity is O(log n). Here, n is the number of piles of coins.

Python 3

```python
class Solution:
    def maxCoins(self, piles: List[int]) -> int:
        piles.sort()
        return sum(piles[len(piles) // 3 :][::2])
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. How many coins does Bob end with
# 2. Groups of size s: Alice takes the largest, you the
# second largest, Bob the rest


# ============================================================
# FOLLOW-UP 1: How many coins does Bob end with
# ============================================================

# ---- follow-up 1: function ----
# Answer: Bob takes the smallest pile of every round; the
# number of rounds is len / 3.
# Time: O(n log n).  Space: O(n).
def bobs_coins(piles):
    piles = sorted(piles)
    rounds = len(piles) // 3
    return sum(piles[:rounds])


# ============================================================
# FOLLOW-UP 2: Groups of size s: Alice takes the largest,
# you the second largest, Bob the rest
# ============================================================

# ---- follow-up 2: function ----
# Answer: Take pairs from the top for you, and the
# smallest piles go to Bob.
# Time: O(n log n).  Space: O(n).
def my_coins_in_groups(piles, size):
    piles = sorted(piles)
    rounds = len(piles) // size
    total = 0
    position = len(piles) - 2
    for _ in range(rounds):
        total += piles[position]
        position -= 2
    return total
```

2029. Stone Game IX

Medium · Greedy, Minimax, Array, Math, Counting, Game Theory, Nim Game, Zero-Sum Game

leetcode.com/problems/stone-game-ix

PROBLEM

Alice and Bob continue their games with stones. There is a row of n stones, and each stone has an associated value. You are given an integer array `stones`, where `stones[i]` is the **value** of the i^{th} stone.

Alice and Bob take turns, with **Alice** starting first. On each turn, the player may remove any stone from `stones`. The player who removes a stone **loses** if the **sum** of the values of **all removed stones** is divisible by `3`. Bob will win automatically if there are no remaining stones (even if it is Alice's turn).

Assuming both players play **optimally**, return `true` *if Alice wins and* `false` *if Bob wins*.

Example 1:

```
Input: stones = [2,1]
Output: true
Explanation: The game will be played as follows:
- Turn 1: Alice can remove either stone.
- Turn 2: Bob removes the remaining stone.
The sum of the removed stones is 1 + 2 = 3 and is divisible by 3. Therefore, Bob loses and Alice wins the game.
```

Example 2:

```
Input: stones = [2]
Output: false
Explanation: Alice will remove the only stone, and the sum of the values on the removed stones is 2.
Since all the stones are removed and the sum of values is not divisible by 3, Bob wins the game.
```

Example 3:

```
Input: stones = [5,1,2,4,3]
Output: false
Explanation: Bob will always win. One possible way for Bob to win is shown below:
- Turn 1: Alice can remove the second stone with value 1. Sum of removed stones = 1.
- Turn 2: Bob removes the fifth stone with value 3. Sum of removed stones = 1 + 3 = 4.
- Turn 3: Alices removes the fourth stone with value 4. Sum of removed stones = 1 + 3 + 4 = 8.
- Turn 4: Bob removes the third stone with value 2. Sum of removed stones = 1 + 3 + 4 + 2 = 10.
- Turn 5: Alice removes the first stone with value 5. Sum of removed stones = 1 + 3 + 4 + 2 + 5 = 15.
Alice loses the game because the sum of the removed stones (15) is divisible by 3. Bob wins the game.
```

Constraints:

- `1 <= stones.length <= 10^5`
- `1 <= stones[i] <= 10^4`

SOLUTIONS (PYTHON 3)

Solution 1: Greedy + Case Discussion

How to think about it

Only residues modulo 3 matter; n ≤ 10^5 forbids full play. Residue 0 does not change the running sum and can be inserted; Alice cannot open with 0. Opening with 1 yields the optimal stream 1,1,2,1,2,… (symmetric for 2). After folding in the zeros, Alice wins if the length is odd and the two nonzero piles are not tied empty. Test cnt and the swapped (cnt[0],cnt[2],cnt[1]); either opening suffices.

Since the player's goal is to ensure the total value of the removed stones is not divisible by 3, we only need to consider the remainder of each stone's value when divided by 3.

We use an array cnt of length 3 to maintain the count of the current remaining stones' values modulo 3, where cnt[0] represents the count of stones with a remainder of 0, and cnt[1] and cnt[2] respectively represent the counts of stones with remainders of 1 and 2.

In the first round, Alice cannot remove stones with a remainder of 0, as this would make the total value of the removed stones divisible by 3. Therefore, Alice can only remove stones with a remainder of 1 or 2.

First, let's consider the case where Alice removes a stone with a remainder of 1. If Alice removes a stone with a remainder of 1, the remainder of the total value of stones 0 against 3 will not change, so stones with a value remainder of 0 can be removed in any round, which we will not consider for now. Thus, Bob can only remove stones with a remainder of 1, followed by Alice removing stones with a remainder of 2, and so on, in the sequence 1, 1, 2, 1, 2, …. In this scenario, if the final round is odd and there are still remaining stones, then Alice wins; otherwise, Bob wins.

For the case where Alice removes a stone with a remainder of 2 in the first round, we can draw a similar conclusion.

The time complexity is O(n), where n is the length of the array stones. The space complexity is O(1).

Python 3

```python
class Solution:
    def stoneGameIX(self, stones: List[int]) -> bool:
        def check(cnt: List[int]) -> bool:
            if cnt[1] == 0:
                return False
            cnt[1] -= 1
            r = 1 + min(cnt[1], cnt[2]) * 2 + cnt[0]
            if cnt[1] > cnt[2]:
                cnt[1] -= 1
                r += 1
            return r % 2 == 1 and cnt[1] != cnt[2]

        c1 = [0] * 3
        for x in stones:
            c1[x % 3] += 1
        c2 = [c1[0], c1[2], c1[1]]
        return check(c1) or check(c2)
```

Solution 2: Simulation

> **How to think about it**
>
> Solution 1 uses a closed-form parity test. The same counts can instead decrement along the residue sequence and, when the needed residue runs out, ask whether zeros make the turn index odd. It remains an O(n) count plus a constant simulation, easier to check at the boundaries.

As in Solution 1, count the residues of stone values modulo 3. Alice's first move can only take a residue-1 or residue-2 stone.

For each opening, both players follow the optimal residue sequence: starting with 1 gives 1, 1, 2, 1, 2, …; starting with 2 is symmetric. When the required residue runs out, Alice wins if the remaining residue-0 stones make the current turn count odd.

Return true if either opening lets Alice win.

The time complexity is O(n) and the space complexity is O(1), where n is the length of stones.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```python
class Solution:
    def stoneGameIX(self, stones: List[int]) -> bool:
        if len(stones) == 1:
            return False
        count = [0] * 3
        for x in stones:
            count[x % 3] += 1

        def check(first: int, count: List[int]) -> bool:
            turn = 1
            count[first] -= 1
            if count[first] < 0:
                return False
            x = first
            while count[1] or count[2]:
                if count[x]:
                    count[x] -= 1
                    x = 2 if x == 1 else 1
                else:
                    return (turn + count[0]) % 2 == 1
                turn += 1
            return False

        return check(1, count[:]) or check(2, count[:])
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Plain search over the counts (to check the formula)
# 2. What if Bob starts


# ============================================================
# FOLLOW-UP 1: Plain search over the counts (to check the
# formula)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Only the counts of stones with remainder 0, 1, 2
# and the removed sum modulo 3 matter, so search that
# small state space.
# Time: O(3^n) without memoisation (it only checks the
# formula on tiny inputs).  Space: O(n).
def winner_by_search(stones, first="Alice"):
    counts = [0, 0, 0]
    for x in stones:
        counts[x % 3] += 1
    second = "Bob" if first == "Alice" else "Alice"

    # turn 0 is the first player, turn 1 the second player
    def first_wins(c0, c1, c2, total, turn):
        left = [c0, c1, c2]
        if sum(left) == 0:
            return False                # everything removed: the second player wins
        movable = False
        for r in range(3):
            if left[r] > 0:
                if (total + r) % 3 == 0:
                    continue            # taking it would make the mover lose
                movable = True
                left[r] -= 1
                outcome = first_wins(left[0], left[1], left[2], (total + r) % 3, 1 - turn)
                left[r] += 1
                if (turn == 0 and outcome) or (turn == 1 and not outcome):
                    return outcome
```

```
        if not movable:
            return turn == 1            # the mover cannot avoid losing
        return turn == 1

    wins = first_wins(counts[0], counts[1], counts[2], 0, 0)
    return first if wins else second


# ============================================================
# FOLLOW-UP 2: What if Bob starts
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same search with the roles of the players
# swapped.
# Time: O(3^n).  Space: O(n).
def winner_if_bob_starts(stones):
    return winner_by_search(stones, "Bob")
```

1770. Maximum Score from Performing Multiplication Operations

Hard · Array, Dynamic Programming

leetcode.com/problems/maximum-score-from-performing-multiplication-operations

PROBLEM

You are given two **0-indexed** integer arrays `nums` and `multipliers` of size `n` and `m` respectively, where `n >= m`.

You begin with a score of `0`. You want to perform **exactly** `m` operations. On the `i`th operation (**0-indexed**) you will:

- Choose one integer `x` from **either the start or the end** of the array `nums`.
- Add `multipliers[i] * x` to your score.
  - Note that `multipliers[0]` corresponds to the first operation, `multipliers[1]` to the second operation, and so on.
- Remove `x` from `nums`.

Return *the **maximum** score after performing* `m` *operations.*

Example 1:

```
Input: nums = [1,2,3], multipliers = [3,2,1]
Output: 14
Explanation: An optimal solution is as follows:
- Choose from the end, [1,2,3], adding 3 * 3 = 9 to the score.
- Choose from the end, [1,2], adding 2 * 2 = 4 to the score.
- Choose from the end, [1], adding 1 * 1 = 1 to the score.
The total score is 9 + 4 + 1 = 14.
```

Example 2:

```
Input: nums = [-5,-3,-3,-2,7,1], multipliers = [-10,-5,3,4,6]
Output: 102
Explanation: An optimal solution is as follows:
- Choose from the start, [-5,-3,-3,-2,7,1], adding -5 * -10 = 50 to the score.
- Choose from the start, [-3,-3,-2,7,1], adding -3 * -5 = 15 to the score.
- Choose from the start, [-3,-2,7,1], adding -3 * 3 = -9 to the score.
- Choose from the end, [-2,7,1], adding 1 * 4 = 4 to the score.
- Choose from the end, [-2,7], adding 7 * 6 = 42 to the score.
The total score is 50 + 15 - 9 + 4 + 42 = 102.
```

Constraints:

- `n == nums.length`
- `m == multipliers.length`
- `1 <= m <= 300`
- `m <= n <= 10`^{5}
- `-1000 <= nums[i], multipliers[i] <= 1000`

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Each step multiplies one end of nums by multipliers[k]. After m moves only the two ends change, so a state is how many were taken from the left and from the right. Memoize f(i,j,k) with left index i, right index j, and multiplier k. Recur on taking left or right; stop at k=m. There are $O(m^2)$ states.

Python 3

```python
class Solution:
    def maximumScore(self, nums: List[int], multipliers: List[int]) -> int:
        @cache
        def f(i, j, k):
            if k >= m or i >= n or j < 0:
                return 0
            a = f(i + 1, j, k + 1) + nums[i] * multipliers[k]
            b = f(i, j - 1, k + 1) + nums[j] * multipliers[k]
            return max(a, b)

        n = len(nums)
        m = len(multipliers)
        return f(0, n - 1, 0)
```

Solution 2

How to think about it

Tabulate the same idea: f[i][j] is the best after taking i from the front and j from the back. Fill by increasing i+j and record answers at i+j=m. Same $O(m^2)$ without recursion.

Python 3

```python
class Solution:
    def maximumScore(self, nums: List[int], multipliers: List[int]) -> int:
        n, m = len(nums), len(multipliers)
        f = [[-inf] * (m + 1) for _ in range(m + 1)]
        f[0][0] = 0
        ans = -inf
        for i in range(m + 1):
            for j in range(m - i + 1):
                k = i + j - 1
                if i > 0:
                    f[i][j] = max(f[i][j], f[i - 1][j] + multipliers[k] * nums[i - 1])
                if j > 0:
                    f[i][j] = max(f[i][j], f[i][j - 1] + multipliers[k] * nums[n - j])
                if i + j == m:
                    ans = max(ans, f[i][j])
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the chosen sides (L or R) of an optimal play
# 2. The MINIMUM score instead


# ========================================================
# FOLLOW-UP 1: Return the chosen sides (L or R) of an
# optimal play
# ========================================================
```

```
# ---- follow-up 1: function ----
# Answer: Remember the better choice of every state and
# replay it from the start.
# Time: O(m^2).  Space: O(m^2).
def sides_of_best_play(nums, multipliers):
    m = len(multipliers)
    memo = {}

    def best(i, left):
        if i == m:
            return 0
        if (i, left) in memo:
            return memo[(i, left)]
        right = len(nums) - 1 - (i - left)
        take_left = multipliers[i] * nums[left] + best(i + 1, left + 1)
        take_right = multipliers[i] * nums[right] + best(i + 1, left)
        memo[(i, left)] = max(take_left, take_right)
        return memo[(i, left)]

    sides = []
    left = 0
    for i in range(m):
        right = len(nums) - 1 - (i - left)
        take_left = multipliers[i] * nums[left] + best(i + 1, left + 1)
        take_right = multipliers[i] * nums[right] + best(i + 1, left)
        if take_left >= take_right:
            sides.append("L")
            left += 1
        else:
            sides.append("R")
    return best(0, 0), "".join(sides)


# ============================================================
# FOLLOW-UP 2: The MINIMUM score instead
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same table with min in place of max.
# Time: O(m^2).  Space: O(m^2).
def minimum_score(nums, multipliers):
    m = len(multipliers)
    memo = {}

    def best(i, left):
        if i == m:
            return 0
        if (i, left) in memo:
            return memo[(i, left)]
        right = len(nums) - 1 - (i - left)
        take_left = multipliers[i] * nums[left] + best(i + 1, left + 1)
        take_right = multipliers[i] * nums[right] + best(i + 1, left)
        memo[(i, left)] = min(take_left, take_right)
        return memo[(i, left)]

    return best(0, 0)
```

1406. Stone Game III

Hard · Minimax, Array, Math, Dynamic Programming, Game Theory, Zero-Sum Game

leetcode.com/problems/stone-game-iii

★ **TOP 7 PROBLEM** in Game Theory / Minimax: master this one first

PROBLEM

Alice and Bob continue their games with piles of stones. There are several stones **arranged in a row**, and each stone has an associated value which is an integer given in the array `stoneValue`.

Alice and Bob take turns, with Alice starting first. On each player's turn, that player can take `1`, `2`, or `3` stones from the **first** remaining stones in the row.

The score of each player is the sum of the values of the stones taken. The score of each player is `0` initially.

The objective of the game is to end with the highest score, and the winner is the player with the highest score and there could be a tie. The game continues until all the stones have been taken.

Assume Alice and Bob **play optimally**.

Return `"Alice"` *if Alice will win,* `"Bob"` *if Bob will win, or* `"Tie"` *if they will end the game with the same score*.

Example 1:

```
Input: stoneValue = [1,2,3,7]
Output: "Bob"
Explanation: Alice will always lose. Her best move will be to take three piles and the score become 6.
Now the score of Bob is 7 and Bob wins.
```

Example 2:

```
Input: stoneValue = [1,2,3,-9]
Output: "Alice"
Explanation: Alice must choose all the three piles at the first move to win and leave Bob with negative
score.
If Alice chooses one pile her score will be 1 and the next move Bob's score becomes 5. In the next
move, Alice will take the pile with value = -9 and lose.
If Alice chooses two piles her score will be 3 and the next move Bob's score becomes 3. In the next
move, Alice will take the pile with value = -9 and also lose.
Remember that both play optimally so here Alice will choose the scenario that makes her win.
```

Example 3:

```
Input: stoneValue = [1,2,3,6]
Output: "Tie"
Explanation: Alice cannot win this game. She can end the game in a draw if she decided to choose all
the first three piles, otherwise she will lose.
```

Constraints:

- `1 <= stoneValue.length <= 5 * 10^4`
- `-1000 <= stoneValue[i] <= 1000`

SOLUTIONS (PYTHON 3)

Solution 1: Memoization Search

> **How to think about it**
>
> Each player takes 1– 3 piles. Plain recursion on n≤ 5× 10^4 recomputes the same suffixes many times. The current player maximizes “stones taken this turn minus the opponent's best difference on the rest”. Let dfs(i) be that value from index i, trying the three prefixes. Memoizing dfs(i) evaluates each start once. The sign of dfs(0) decides Alice, Bob, or a tie.

We design a function dfs(i), which represents the maximum score difference that the current player can obtain when playing the game in the range [i, n). If dfs(0) > 0, it means that the first player Alice can win; if dfs(0) < 0, it means that the second player Bob can win; otherwise, it means that the two players tie.

The execution logic of the function dfs(i) is as follows:

- If i ≥ n, it means that there are no stones to take now, so we can directly return 0;
- Otherwise, we enumerate the index j of the last pile taken by the current player, where i ≤ j < min(i + 3, n), i.e., the current player takes all the piles in the index range [i, j] and gets a score of $\sum_{k=i}^{j}$ stoneValue[k]. Then the score difference that the other player can get in the next round is dfs(j + 1), so the score difference that the current player can get is $\sum_{k=i}^{j}$ stoneValue[k] - dfs(j + 1). We want to maximize the score difference of the current player, so we can use the max function to get the maximum score difference, that is:

$$dfs(i) = \max_{i \le j < \min(i+3,\, n)} \{\sum_{k=i}^{j} stoneValue[k] - dfs(j + 1)\}$$

To prevent repeated calculations, we can use memoization search.

The time complexity is O(n), and the space complexity is O(n). Where n is the number of piles of stones.

Python 3

```python
class Solution:
    def stoneGameIII(self, stoneValue: List[int]) -> str:
        @cache
        def dfs(i: int) -> int:
            if i >= len(stoneValue):
                return 0
            ans = -inf
            s = 0
            for j in range(i, i + 3):
                if j >= len(stoneValue):
                    break
                s += stoneValue[j]
                ans = max(ans, s - dfs(j + 1))
            return ans

        res = dfs(0)
        if res == 0:
            return 'Tie'
        return 'Alice' if res > 0 else 'Bob'
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> Each player takes 1–3 piles. Trying every line of play is exponential, and n≤ 5× 10^4. The current player maximizes the stones taken this turn minus the opponent's best difference on the rest. That difference from index i depends only on the three later starts i+1, i+2, and i+3. Searching the recurrence still calls the next index before it returns, so the chain has depth n and overflows the stack. Those later values are known if we walk from the end. Let f[i] be the best difference from i, fill i from n-1 down to 0, and read the sign of f[0].

We define f[i] as the maximum score difference that the current player can obtain when playing the game in the range [i, n). If f[0] > 0, Alice wins; if f[0] < 0, Bob wins; otherwise the two players tie.

f[i] is calculated as follows:

- If i ≥ n, there are no stones left, so the value is 0;
- Otherwise, we enumerate the index j of the last pile taken by the current player, where i ≤ j < min(i + 3, n). The current player takes every pile in [i, j] and scores $\sum_{k=i}^{j}$ stoneValue[k]. The opponent's best difference on the rest is f[j + 1], so the current difference is that sum minus f[j + 1]. The current player takes the maximum:

$$f[i] = \max_{i \le j < \min(i+3,\, n)} \{\sum_{k=i}^{j} \text{stoneValue}[k] - f[j + 1]\}$$

We calculate f from i = n - 1 down to 0.

The time complexity is O(n), and the space complexity is O(n). Where n is the number of piles of stones.

Python 3

```python
class Solution:
    def stoneGameIII(self, stoneValue: List[int]) -> str:
        n = len(stoneValue)
        f = [0] * (n + 1)
        for i in range(n - 1, -1, -1):
            ans = -inf
            s = 0
            for j in range(i, min(i + 3, n)):
                s += stoneValue[j]
                ans = max(ans, s - f[j + 1])
            f[i] = ans
        res = f[0]
        if res == 0:
            return 'Tie'
        return 'Alice' if res > 0 else 'Bob'
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. A player may take 1 to k stones
# 2. Each player's final score (taking 1 to 3 stones)


# ============================================================
# FOLLOW-UP 1: A player may take 1 to k stones
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same table from the right, with k choices
# instead of 3.
# Time: O(n * k).  Space: O(n).
def difference_with_limit(stone_value, k):
    n = len(stone_value)
    best = [0] * (n + 1)
    for i in range(n - 1, -1, -1):
        total = 0
        best[i] = float("-inf")
        for take in range(1, k + 1):
            if i + take > n:
                break
            total += stone_value[i + take - 1]
            best[i] = max(best[i], total - best[i + take])
    return best[0]


# ============================================================
# FOLLOW-UP 2: Each player's final score (taking 1 to 3
# stones)
# ============================================================

# ---- follow-up 2: function ----
# Answer: From the difference D and the total T: Alice has
```

```
# (T + D) / 2.
# Time: O(n).  Space: O(n).
def final_scores_three(stone_value):
    difference = difference_with_limit(stone_value, 3)
    total = sum(stone_value)
    return (total + difference) // 2, (total - difference) // 2
```

1510. Stone Game IV

Hard · Minimax, Math, Dynamic Programming, Game Theory, Nim Game, Sprague–Grundy, Zero-Sum Game

leetcode.com/problems/stone-game-iv

PROBLEM

Alice and Bob take turns playing a game, with Alice starting first.

Initially, there are `n` stones in a pile. On each player's turn, that player makes a *move* consisting of removing **any** non-zero **square number** of stones in the pile.

Also, if a player cannot make a move, he/she loses the game.

Given a positive integer `n`, return `true` if and only if Alice wins the game otherwise return `false`, assuming both players play optimally.

Example 1:

```
Input: n = 1
Output: true
Explanation: Alice can remove 1 stone winning the game because Bob doesn't have any moves.
```

Example 2:

```
Input: n = 2
Output: false
Explanation: Alice can only remove 1 stone, after that Bob removes the last one winning the game (2 -> 1 -> 0).
```

Example 3:

```
Input: n = 4
Output: true
Explanation: n is already a perfect square, Alice can win with one move, removing 4 stones (4 -> 0).
```

Constraints:

- `1 <= n <= 10^5`

SOLUTIONS (PYTHON 3)

Solution 1: Memoization

How to think about it

Players alternately remove a square number of stones from a pile of n; the one who takes the last stone wins. $n \le 10^5$, so expanding the full game tree would recompute the same remainder many times. A position depends only on the remaining count: it is winning if some move leaves the opponent in a losing position. Memoized dfs(i) answers whether the player to move wins with i stones, trying every $j^2 \le i$. Each state has $O(\sqrt{i})$ transitions.

We design a function dfs(i), which represents whether the current player can win the game when there are i stones in the pile. If the current player can win, it returns true; otherwise, it returns false. The answer is dfs(n).

The calculation process of the function dfs(i) is as follows:

- If $i \le 0$, it means the current player cannot make any move, so the current player loses the game, return false;
- Otherwise, enumerate the number of stones j that the current player can take away, where j is a square number. If the other player cannot win the game after the current player takes away j stones, then the current player wins the game, return true. If all j enumerated cannot satisfy the above condition, then the current player loses the game, return false.

To avoid repeated calculations, we can use memoization, i.e., use an array f to record the calculation results of the function dfs(i).

The time complexity is O(n × √n), and the space complexity is O(n). Where n is the number of stones in the pile.

Python 3

```python
@cache
def dfs(i: int) -> bool:
    if i <= 0:
        return False
    k = isqrt(i)
    return any(not dfs(i - j * j) for j in range(1, k + 1))


class Solution:
    def winnerSquareGame(self, n: int) -> bool:
        return dfs(n)
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> Solution 1 already fills this win/loss table from fewer stones to more. The f[i] below is that same table.

Solution 1 already fills this table from fewer stones to more. Define an array f, where f[i] represents whether the current player can win the game when there are i stones in the pile. If the current player can win, then f[i] is true, otherwise it is false. The answer is f[n].

We enumerate i in the range [1,..n], and enumerate j in the range [1,..i], where j is a square number. If the other player cannot win the game after the current player takes away j stones, then the current player wins the game, i.e., f[i] = true. If all j enumerated cannot satisfy the above condition, then the current player loses the game, i.e., f[i] = false. Therefore, we can get the state transition equation:

$$f[i] = \begin{cases} \text{true}, & \text{if } \exists\, j \in [1,..i],\ j^2 \le i \text{ and } f[i-j^2] = \text{false} \\ \text{false}, & \text{otherwise} \end{cases}$$

Finally, we return f[n].

The time complexity is O(n × √n), and the space complexity is O(n). Where n is the number of stones in the pile.

Python 3

```python
class Solution:
    def winnerSquareGame(self, n: int) -> bool:
        f = [False] * (n + 1)
        for i in range(1, n + 1):
            j = 1
            while j <= i // j:
                if not f[i - j * j]:
                    f[i] = True
                    break
                j += 1
        return f[n]
```

Solution 3: Dynamic Programming

> **How to think about it**
>
> Expanding the whole game tree is exponential. There can be 10^5 stones, and the first move always removes one stone, so the search has depth n. Whether a position is winning depends only on the remaining count, and only on smaller counts. Let f[i] record a win with i stones, with f[0] losing, and scan i upward. A square j^2 that leaves a losing position makes f[i] winning.

Let f[i] be whether the player to move wins with i stones. f[0] is false, since no move remains. For i from 1 to n, enumerate squares $j^2 \le i$. If $f[i-j^2]$ is false, the opponent loses from there, so f[i] is true and the remaining squares can be skipped. If every square leaves a winning position for the opponent, f[i] stays false. The answer is f[n].

The time complexity is O(n × √n), and the space complexity is O(n). Where n is the number of stones in the pile.

Python 3

```python
class Solution:
    def winnerSquareGame(self, n: int) -> bool:
        f = [False] * (n + 1)
        for i in range(1, n + 1):
            j = 1
            while j <= i // j:
                if not f[i - j * j]:
                    f[i] = True
                    break
                j += 1
        return f[n]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Players may remove a perfect cube instead of a square
# 2. How many n up to N are winning for the first player
# (perfect squares)


# ============================================================
# FOLLOW-UP 1: Players may remove a perfect cube instead
# of a square
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same table with cubes as the allowed moves.
# Time: O(n^(4/3)).  Space: O(n).
def alice_wins_with_cubes(n):
    win = [False] * (n + 1)
    for stones in range(1, n + 1):
        k = 1
        while k ** 3 <= stones:
            if not win[stones - k ** 3]:
                win[stones] = True
                break
            k += 1
    return win[n]


# ============================================================
# FOLLOW-UP 2: How many n up to N are winning for the
# first player (perfect squares)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Fill the table once and count the True values.
# Time: O(n * sqrt n).  Space: O(n).
def count_winning_positions(n):
    win = [False] * (n + 1)
    for stones in range(1, n + 1):
        k = 1
        while k * k <= stones:
            if not win[stones - k * k]:
                win[stones] = True
                break
            k += 1
    return sum(win)
```

913. Cat and Mouse

Hard · Graph, Topological Sort, Memoization, Minimax, Math, Dynamic Programming, Game Theory, Zero-Sum Game

leetcode.com/problems/cat-and-mouse

★ **TOP 7 PROBLEM** in Game Theory / Minimax: master this one first

PROBLEM

A game on an **undirected** graph is played by two players, Mouse and Cat, who alternate turns.

The graph is given as follows: `graph[a]` is a list of all nodes `b` such that `ab` is an edge of the graph.

The mouse starts at node `1` and goes first, the cat starts at node `2` and goes second, and there is a hole at node `0`.

During each player's turn, they **must** travel along one edge of the graph that meets where they are. For example, if the Mouse is at node 1, it **must** travel to any node in `graph[1]`.

Additionally, it is not allowed for the Cat to travel to the Hole (node `0`).

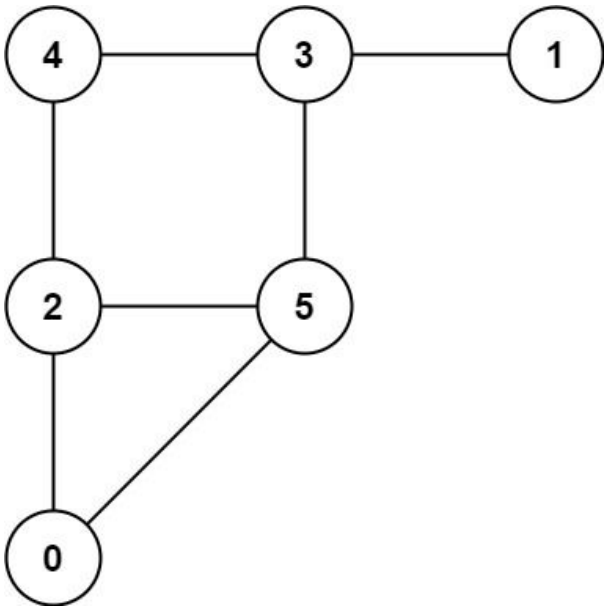
Then, the game can end in three ways:

- If ever the Cat occupies the same node as the Mouse, the Cat wins.
- If ever the Mouse reaches the Hole, the Mouse wins.
- If ever a position is repeated (i.e., the players are in the same position as a previous turn, and it is the same player's turn to move), the game is a draw.

Given a `graph`, and assuming both players play optimally, return

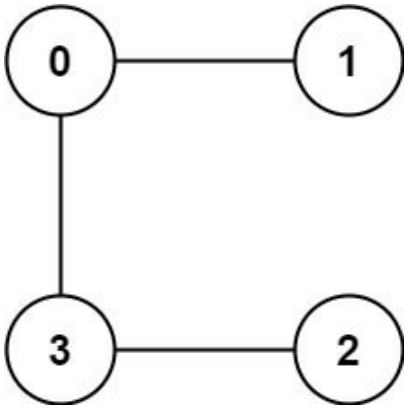
- `1` if the mouse wins the game,
- `2` if the cat wins the game, or
- `0` if the game is a draw.

Example 1:




```
Input: graph = [[2,5],[3],[0,4,5],[1,4,5],[2,3],[0,2,3]]
Output: 0
```

Example 2:




```
Input: graph = [[1,3],[0],[3],[0,2]]
Output: 1
```

Constraints:

- `3 <= graph.length <= 50`
- `1 <= graph[i].length < graph.length`
- `0 <= graph[i][j] < graph.length`
- `graph[i][j] != i`
- `graph[i]` is unique.
- The mouse and the cat can always move.

SOLUTION (PYTHON 3)

Solution 1: Topological Sorting

How to think about it

A state is (mouse, cat, whose turn). n≤ 50, and searching forward struggles to classify draws. Work backward from terminals: the mouse wins in the hole, the cat wins on a coincidence. If the player to move in a previous state can step into their own winning state, mark it a win; otherwise decrease the out-degree and, when it hits zero, mark a loss. After this topological sweep, read the start state (1,2,mouse turn).

According to the problem description, the state of the game is determined by the position of the mouse, the position of the cat, and the player who is moving. The outcome can be directly determined in the following situations:

- When the positions of the cat and the mouse are the same, the cat wins. This is a winning state for the cat and a losing state for the mouse.
- When the mouse is at the hole, the mouse wins. This is a winning state for the mouse and a losing state for the cat.

To determine the result of the initial state, we need to traverse all states starting from the boundary states. Each state includes the position of the mouse, the position of the cat, and the player who is moving. Based on the current state, we can determine all possible states from the previous round. The player who moved in the previous round is the opposite of the player who is moving in the current state, and the positions of the players in the previous round are different from their positions in the current state.

We use the tuple (m, c, t) to represent the current state and (pm, pc, pt) to represent a possible state from the previous round. The possible states from the previous round are:

- If the player moving in the current round is the mouse, then the player moving in the previous round is the cat. The position of the mouse in the previous round is the same as the current position of the mouse, and the position of the cat in the previous round is any adjacent node of the current position of the cat.
- If the player moving in the current round is the cat, then the player moving in the previous round is the mouse. The position of the cat in the previous round is the same as the current position of the cat, and the position of the mouse in the previous round is any adjacent node of the current position of the mouse.

Initially, all states except the boundary states are unknown. Starting from the boundary states, for each state, we determine all possible states from the previous round and update the results. The update logic is as follows:

1. If the player moving in the previous round is the same as the winner in the current round, then the player moving in the previous round can reach the current state and win. We directly update the state of the previous round to the winner of the current round.
2. If the player moving in the previous round is different from the winner in the current round, and all states that the player moving in the previous round can reach are losing states for that player, then we update the state of the previous round to the winner of the current round.

For the second update logic, we need to record the degree of each state. Initially, the degree of each state represents the number of nodes the player moving in that state can move to, which is the number of adjacent nodes of the node where the player is located. If the player is the cat and the node is adjacent to the hole, the degree of that state is reduced by 1.

When all states have been updated, the result of the initial state is the final result.

The time complexity is $O(n^3)$, and the space complexity is $O(n^2)$. Here, n is the number of nodes in the graph.

Python 3

```
HOLE, MOUSE_START, CAT_START = 0, 1, 2
MOUSE_TURN, CAT_TURN = 0, 1
MOUSE_WIN, CAT_WIN, TIE = 1, 2, 0


class Solution:
    def catMouseGame(self, graph: List[List[int]]) -> int:
        def get_prev_states(state):
            m, c, t = state
            pt = t ^ 1
            pre = []
            if pt == CAT_TURN:
                for pc in graph[c]:
                    if pc != HOLE:
                        pre.append((m, pc, pt))
            else:
                for pm in graph[m]:
                    pre.append((pm, c, pt))
            return pre

        n = len(graph)
        ans = [[[0, 0] for _ in range(n)] for _ in range(n)]
        degree = [[[0, 0] for _ in range(n)] for _ in range(n)]
        for i in range(n):
            for j in range(1, n):
                degree[i][j][MOUSE_TURN] = len(graph[i])
                degree[i][j][CAT_TURN] = len(graph[j])
            for j in graph[HOLE]:
                degree[i][j][CAT_TURN] -= 1
        q = deque()
        for j in range(1, n):
            ans[0][j][MOUSE_TURN] = ans[0][j][CAT_TURN] = MOUSE_WIN
            q.append((0, j, MOUSE_TURN))
            q.append((0, j, CAT_TURN))
        for i in range(1, n):
            ans[i][i][MOUSE_TURN] = ans[i][i][CAT_TURN] = CAT_WIN
            q.append((i, i, MOUSE_TURN))
            q.append((i, i, CAT_TURN))
        while q:
            state = q.popleft()
            t = ans[state[0]][state[1]][state[2]]
            for prev_state in get_prev_states(state):
                pm, pc, pt = prev_state
                if ans[pm][pc][pt] == TIE:
                    win = (t == MOUSE_WIN and pt == MOUSE_TURN) or (
                        t == CAT_WIN and pt == CAT_TURN
                    )
                    if win:
                        ans[pm][pc][pt] = t
                        q.append(prev_state)
                    else:
                        degree[pm][pc][pt] -= 1
                        if degree[pm][pc][pt] == 0:
                            ans[pm][pc][pt] = t
                            q.append(prev_state)
        return ans[MOUSE_START][CAT_START][MOUSE_TURN]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Call it a draw when the game has not ended after a
# given number of moves
# 2. Find the smallest move limit with which the mouse can
# force a win


# ============================================================
# FOLLOW-UP 1: Call it a draw when the game has not ended
# after a given number of moves
# ============================================================

# ---- follow-up 1: function ----
# Answer: Search over (mouse, cat, moves left): the mover
# wins if some move leads to a win for them within the
# limit.
# Time: O(L * n * E), L = move limit.  Space: O(L * n^2).
def result_within_moves(graph, limit):
    # returns 1 if the mouse wins, 2 if the cat wins, 0 for a draw (nobody wins within the limit)
    memo = {}

    def play(mouse, cat, moves_left, mouse_turn):
        if mouse == 0:
            return 1
        if mouse == cat:
            return 2
        if moves_left == 0:
            return 0
        key = (mouse, cat, moves_left, mouse_turn)
        if key in memo:
            return memo[key]
        me = 1 if mouse_turn else 2
        best = 2 if mouse_turn else 1          # the worst result for the mover
        draw = False
        if mouse_turn:
            options = [(nxt, cat) for nxt in graph[mouse]]
        else:
            options = [(mouse, nxt) for nxt in graph[cat] if nxt != 0]
        for new_mouse, new_cat in options:
            outcome = play(new_mouse, new_cat, moves_left - 1, not mouse_turn)
            if outcome == me:
                memo[key] = me
                return me
            if outcome == 0:
                draw = True
        memo[key] = 0 if draw else best
        return memo[key]

    return play(1, 2, limit, True)


# ============================================================
# FOLLOW-UP 2: Find the smallest move limit with which the
# mouse can force a win
# ============================================================

# ---- follow-up 2: function ----
# Answer: Try limits 1, 2, 3, ... until the mouse wins (or
# give up after the largest sensible limit).
# Time: O(n^5 * E), because every limit up to 4n^2
```

```
# restarts the search.  Space: O(n^4).
def fewest_moves_for_mouse_win(graph):
    n = len(graph)
    for limit in range(1, 4 * n * n + 1):
```

```
        if result_within_moves(graph, limit) == 1:
            return limit
    return -1
```

2193. Minimum Number of Moves to Make Palindrome

Hard · Greedy, Binary Indexed Tree, Two Pointers, String

leetcode.com/problems/minimum-number-of-moves-to-make-palindrome

PROBLEM

You are given a string `s` consisting only of lowercase English letters.

In one **move**, you can select any two **adjacent** characters of `s` and swap them.

Return *the **minimum number of moves** needed to make `s` a palindrome*.

Note that the input will be generated such that `s` can always be converted to a palindrome.

Example 1:

```
Input: s = "aabb"
Output: 2
Explanation:
We can obtain two palindromes from s, "abba" and "baab".
- We can obtain "abba" from s in 2 moves: "aabb" -> "abab" -> "abba".
- We can obtain "baab" from s in 2 moves: "aabb" -> "abab" -> "baab".
Thus, the minimum number of moves needed to make s a palindrome is 2.
```

Example 2:

```
Input: s = "letelt"
Output: 2
Explanation:
One of the palindromes we can obtain from s in 2 moves is "lettel".
One of the ways we can obtain it is "letelt" -> "letetl" -> "lettel".
Other palindromes such as "tleelt" can also be obtained in 2 moves.
It can be shown that it is not possible to obtain a palindrome in less than 2 moves.
```

Constraints:

- `1 <= s.length <= 2000`
- `s` consists only of lowercase English letters.
- `s` can be converted to a palindrome using a finite number of moves.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Adjacent swaps must turn the string into a palindrome; a solution is guaranteed. The cost is the total distance that paired letters travel to symmetric positions. Searching all swap sequences is too large; n≤ 2000 allows an $O(n^2)$ greedy. Fix the leftmost letter a, find the nearest a from the right, swap it to the right end, and recurse on the inner substring. If a has no right match (the unique odd letter), move it to the center at cost equal to the distance to the midpoint. Pairing the outermost letter is never worse than pairing an inner one first, so the greedy is optimal.

Python 3

```python
class Solution:
    def minMovesToMakePalindrome(self, s: str) -> int:
        cs = list(s)
        ans, n = 0, len(s)
        i, j = 0, n - 1
        while i < j:
            even = False
            for k in range(j, i, -1):
                if cs[i] == cs[k]:
                    even = True
                    while k < j:
                        cs[k], cs[k + 1] = cs[k + 1], cs[k]
                        k += 1
                        ans += 1
                    j -= 1
                    break
            if not even:
                ans += n // 2 - i
            i += 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the adjacent swaps (positions) that build the
# palindrome
# 2. Can the letters form a palindrome at all


# ============================================================
# FOLLOW-UP 1: Return the adjacent swaps (positions) that
# build the palindrome
# ============================================================

# ---- follow-up 1: function ----
# Answer: Greedy from both ends: bring the matching letter
# to the right end with adjacent swaps; a letter without a
# partner is moved one step toward the middle.
# Time: O(n^2).  Space: O(n).
def palindrome_swaps(s):
    chars = list(s)
    swaps = []
    left, right = 0, len(chars) - 1
    while left < right:
        k = right
        while k > left and chars[k] != chars[left]:
            k -= 1
        if k == left:
            chars[left], chars[left + 1] = chars[left + 1], chars[left]
            swaps.append((left, left + 1))
            continue
        for j in range(k, right):
            chars[j], chars[j + 1] = chars[j + 1], chars[j]
            swaps.append((j, j + 1))
        left += 1
        right -= 1
    return swaps


# ============================================================
# FOLLOW-UP 2: Can the letters form a palindrome at all
# ============================================================

# ---- follow-up 2: function ----
# Answer: At most one letter may have an odd count.
```

```
# Time: O(n).  Space: O(1).
def can_form_palindrome(s):
    counts = {}
    for ch in s:
        counts[ch] = counts.get(ch, 0) + 1
    odd = 0
    for count in counts.values():
        if count % 2 == 1:
            odd += 1
    return odd <= 1
```

1872. Stone Game VIII

Hard · Minimax, Array, Math, Dynamic Programming, Game Theory, Prefix Sum, Zero-Sum Game

leetcode.com/problems/stone-game-viii

PROBLEM

Alice and Bob take turns playing a game, with **Alice starting first**.

There are `n` stones arranged in a row. On each player's turn, while the number of stones is **more than one**, they will do the following:

1. Choose an integer `x > 1`, and **remove** the leftmost `x` stones from the row.
2. Add the **sum** of the **removed** stones' values to the player's score.
3. Place a **new stone**, whose value is equal to that sum, on the left side of the row.

The game stops when **only one** stone is left in the row.

The **score difference** between Alice and Bob is `(Alice's score - Bob's score)`. Alice's goal is to **maximize** the score difference, and Bob's goal is the **minimize** the score difference.

Given an integer array `stones` of length `n` where `stones[i]` represents the value of the i^{th} stone **from the left**, return *the **score difference** between Alice and Bob if they both play **optimally**.*

Example 1:

```
Input: stones = [-1,2,-3,4,-5]
Output: 5
Explanation:
- Alice removes the first 4 stones, adds (-1) + 2 + (-3) + 4 = 2 to her score, and places a stone of
  value 2 on the left. stones = [2,-5].
- Bob removes the first 2 stones, adds 2 + (-5) = -3 to his score, and places a stone of value -3 on
  the left. stones = [-3].
The difference between their scores is 2 - (-3) = 5.
```

Example 2:

```
Input: stones = [7,-6,5,10,5,-2,-6]
Output: 13
Explanation:
- Alice removes all stones, adds 7 + (-6) + 5 + 10 + 5 + (-2) + (-6) = 13 to her score, and places a
  stone of value 13 on the left. stones = [13].
The difference between their scores is 13 - 0 = 13.
```

Example 3:

```
Input: stones = [-10,-12]
Output: -22
Explanation:
- Alice can only make one move, which is to remove both stones. She adds (-10) + (-12) = -22 to her
  score and places a stone of value -22 on the left. stones = [-22].
The difference between their scores is (-22) - 0 = -22.
```

Constraints:

- `n == stones.length`
- $2 \le n \le 10^5$
- $-10^4 \le stones[i] \le 10^4$

SOLUTIONS (PYTHON 3)

Solution 1: Prefix Sum + Memoization Search

How to think about it

Each move merges a prefix of at least two piles and scores the new prefix sum; both players maximize the score difference. n≤ 10^5 forbids searching every cut. Merging leaves the prefix-sum array s unchanged. From index i the current player may take s[i] and leave i+1 to the opponent, or postpone the cut. dfs(i)=max(dfs(i+1),s[i]-dfs(i+1)), starting at i=1. Memoization yields linearly many states.

According to the problem description, each time we take the leftmost x stones, add their sum to our score, and then put a stone with this sum value on the leftmost side, it is equivalent to merging these x stones into a stone with this sum value, and the prefix sum remains unchanged.

We can use a prefix sum array s of length n to represent the prefix sum of the array stones, where s[i] represents the sum of the elements stones[0..i].

Next, we design a function dfs(i), which means that we currently take stones from stones[i:], and return the maximum score difference that the current player can get.

The execution process of the function dfs(i) is as follows:

- If i ≥ n - 1, it means that we can only take all the stones at present, so we return s[n - 1].
- Otherwise, we can choose to take all the stones from stones[i + 1:], and the score difference obtained is dfs(i + 1); we can also choose to take the stones stones[:i], and the score difference obtained is s[i] - dfs(i + 1). We take the maximum of the two situations, which is the maximum score difference that the current player can get.

Finally, we can get the score difference between Alice and Bob as dfs(1), that is, Alice must start the game by taking stones from stones[1:].

To avoid repeated calculations, we can use memoization search.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array stones.

Python 3

```
class Solution:
    def stoneGameVIII(self, stones: List[int]) -> int:
        @cache
        def dfs(i: int) -> int:
            if i >= len(stones) - 1:
                return s[-1]
            return max(dfs(i + 1), s[i] - dfs(i + 1))

        s = list(accumulate(stones))
        return dfs(1)
```

Solution 2: Prefix Sum + Dynamic Programming

How to think about it

Solution 1 fills a linear table from the end, and each cell only reads the next one. That table rolls into one scalar. f starts as s[n-1], then f=max(f,s[i]-f) for i=n-2,…,1. Extra space becomes constant.

We can also use dynamic programming to solve this problem.

Similar to Solution 1, we first use a prefix sum array s of length n to represent the prefix sum of the array stones, where s[i] represents the sum of the elements stones[0..i].

We define f[i] to represent the maximum score difference that the current player can get when taking stones from stones[i:].

If the player chooses to take the stones stones[:i], then the score obtained is s[i]. At this time, the other player will take stones from stones[i+1:], and the maximum score difference that the other player can get is f[i+1]. Therefore, the maximum score difference that the current player can get is s[i] - f[i+1].

If the player chooses to take stones from stones[i+1:], then the maximum score difference obtained is f[i+1].

Therefore, we can get the state transition equation:

$$f[i] = \max\{s[i] - f[i+1], f[i+1]\}$$

Finally, we can get the score difference between Alice and Bob as f[1], that is, Alice must start the game by taking stones from stones[1:].

We notice that f[i] is only related to f[i+1], so we only need to use a variable f to represent f[i].

The time complexity is O(n), and the space complexity is O(1). Here, n is the length of the array stones.

Python 3

```
class Solution:
    def stoneGameVIII(self, stones: List[int]) -> int:
        s = list(accumulate(stones))
        f = s[-1]
        for i in range(len(s) - 2, 0, -1):
            f = max(f, s[i] - f)
        return f
```

Solution 3: Prefix Sum + Dynamic Programming

How to think about it

Each move merges a prefix of at least two piles and scores the new prefix sum; both players maximize the score difference. $n \le 10^5$ forbids searching every cut. From index i the current player takes s[i] and leaves i+1, or postpones the cut. That recurrence always calls i+1 first, so the chain has length n and overflows the stack. The next index is known if we walk from the end. Let f[i] be the best difference on stones[i:], with f[n-1]=s[n-1], and fill i from n-2 down to 1.

According to the problem description, each time we take the leftmost x stones, add their sum to our score, and then put a stone with this sum value on the leftmost side, it is equivalent to merging these x stones into a stone with this sum value, and the prefix sum remains unchanged.

We can use a prefix sum array s of length n to represent the prefix sum of the array stones, where s[i] represents the sum of the elements stones[0..i].

Let f[i] be the maximum score difference the current player can get on stones[i:]. The answer is f[1], because Alice must start by taking at least the first two stones. Set f[n - 1] = s[n - 1]: only one pile remains, so the player takes it.

Fill i from n - 2 down to 1. Postponing the cut keeps f[i + 1]. Taking stones[0..i] scores s[i] and leaves the opponent with f[i + 1], so the difference is s[i] - f[i + 1]. Take the larger one:

$$f[i] = \max(f[i + 1], s[i] - f[i + 1])$$

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array stones.

Python 3

```
class Solution:
    def stoneGameVIII(self, stones: List[int]) -> int:
        s = list(accumulate(stones))
        n = len(s)
        f = [0] * n
        f[-1] = s[-1]
        for i in range(n - 2, 0, -1):
            f[i] = max(f[i + 1], s[i] - f[i + 1])
        return f[1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Which number of stones should Alice take on her first
# move
# 2. A move must take at least m stones (not just 2)


# ============================================================
# FOLLOW-UP 1: Which number of stones should Alice take on
# her first move
# ============================================================

# ---- follow-up 1: function ----
# Answer: Compute the table from the right and pick the
# first move with the best prefix - table value.
# Time: O(n^2).  Space: O(n).
def first_move_size(stones, minimum=2):
    n = len(stones)
    prefix = [0] * n
    prefix[0] = stones[0]
    for i in range(1, n):
        prefix[i] = prefix[i - 1] + stones[i]
    after = [0] * n                          # after[j]: best advantage when the last move ended at j
    for j in range(n - 2, -1, -1):
        after[j] = max(prefix[k] - after[k] for k in range(j + 1, n))
    best_value = None
    best_take = None
    for j in range(minimum - 1, n):
        value = prefix[j] - after[j]
        if best_value is None or value > best_value:
            best_value = value
            best_take = j + 1
    return best_take


# ============================================================
# FOLLOW-UP 2: A move must take at least m stones (not
# just 2)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Only Alice's first choice and the later options
# change: a move ending at index k is allowed for any k
# after the last one.
# Time: O(n).  Space: O(n).
def difference_with_minimum(stones, minimum):
    n = len(stones)
    prefix = [0] * n
    prefix[0] = stones[0]
    for i in range(1, n):
        prefix[i] = prefix[i - 1] + stones[i]
    after = [0] * n
    best_ahead = None
    for j in range(n - 2, -1, -1):
        value = prefix[j + 1] - after[j + 1]
        if best_ahead is None or value > best_ahead:
            best_ahead = value
        after[j] = best_ahead
    return max(prefix[j] - after[j] for j in range(minimum - 1, n))
```

PATTERN 40 OF 41

Segment Tree

20 problems · 0 Easy · 7 Medium · 13 Hard

THE IDEA

A tree over array ranges: each node stores the answer (sum, min, max) of its range, so a range query or a point update costs O(log n).

Signals: range sum with updates, range minimum, count of smaller numbers after self, rectangle area union, my calendar.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
size = 1
while size < n:
    size *= 2
tree = [0] * (2 * size)
def update(i, value):
    i += size
    tree[i] = value
    while i > 1:
        i //= 2
        tree[i] = tree[2 * i] + tree[2 * i + 1]
def query(l, r):                        # sum of [l, r)
    total, l, r = 0, l + size, r + size
    while l < r:
        if l & 1:
            total += tree[l]; l += 1
        if r & 1:
            r -= 1; total += tree[r]
        l //= 2; r //= 2
    return total
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	★ **Range Sum Query - Mutable** (see page)	Medium	LC 307	274
2	My Calendar I (see page)	Medium	LC 729	1102
3	My Calendar II (see page)	Medium	LC 731	1105
4	Create Sorted Array through Instructions	Medium	LC 1649	2006
5	Number of Longest Increasing Subsequence (see page)	Medium	LC 673	712
6	Range Frequency Queries	Medium	LC 2080	2011
7	Longest Uploaded Prefix	Medium	LC 2424	2014
8	★ **Count of Smaller Numbers After Self** (see page)	Hard	LC 315	753
9	Count of Range Sum (see page)	Hard	LC 327	776
10	★ **Reverse Pairs** (see page)	Hard	LC 493	759
11	★ **The Skyline Problem** (see page)	Hard	LC 218	750
12	★ **Falling Squares** (see page)	Hard	LC 699	1142
13	★ **Range Module** (see page)	Hard	LC 715	1131
14	★ **My Calendar III** (see page)	Hard	LC 732	1124
15	Rectangle Area II	Hard	LC 850	2017
16	Longest Increasing Subsequence II	Hard	LC 2407	2021
17	Handling Sum Queries After Update	Hard	LC 2569	2025
18	Booking Concert Tickets in Groups	Hard	LC 2286	2030
19	Find Building Where Alice and Bob Can Meet	Hard	LC 2940	2035
20	Number of Pairs Satisfying Inequality	Hard	LC 2426	2039

PATTERN 40 · SEGMENT TREE · PROBLEM 4 OF 20

1649. Create Sorted Array through Instructions

Hard · Binary Indexed Tree, Segment Tree, Array, Binary Search, Divide and Conquer, Ordered Set, Merge Sort

leetcode.com/problems/create-sorted-array-through-instructions

★ **TOP 7 PROBLEM** in Binary Indexed Tree (Fenwick Tree): master this one first

Also appears in pattern 41 Binary Indexed Tree (Fenwick Tree)

PROBLEM

Given an integer array `instructions`, you are asked to create a sorted array from the elements in `instructions`. You start with an empty container `nums`. For each element from **left to right** in `instructions`, insert it into `nums`. The **cost** of each insertion is the **minimum** of the following:

- The number of elements currently in `nums` that are **strictly less than** `instructions[i]`.
- The number of elements currently in `nums` that are **strictly greater than** `instructions[i]`.

For example, if inserting element `3` into `nums = [1,2,3,5]`, the **cost** of insertion is `min(2, 1)` (elements `1` and `2` are less than `3`, element `5` is greater than `3`) and `nums` will become `[1,2,3,3,5]`.

Return *the **total cost** to insert all elements from* `instructions` *into* `nums`. Since the answer may be large, return it **modulo** $10^9 + 7$

Example 1:

```
Input: instructions = [1,5,6,2]
Output: 1
Explanation: Begin with nums = [].
Insert 1 with cost min(0, 0) = 0, now nums = [1].
Insert 5 with cost min(1, 0) = 0, now nums = [1,5].
Insert 6 with cost min(2, 0) = 0, now nums = [1,5,6].
Insert 2 with cost min(1, 2) = 1, now nums = [1,2,5,6].
The total cost is 0 + 0 + 0 + 1 = 1.
```

Example 2:

```
Input: instructions = [1,2,3,6,5,4]
Output: 3
Explanation: Begin with nums = [].
Insert 1 with cost min(0, 0) = 0, now nums = [1].
Insert 2 with cost min(1, 0) = 0, now nums = [1,2].
Insert 3 with cost min(2, 0) = 0, now nums = [1,2,3].
Insert 6 with cost min(3, 0) = 0, now nums = [1,2,3,6].
Insert 5 with cost min(3, 1) = 1, now nums = [1,2,3,5,6].
Insert 4 with cost min(3, 2) = 2, now nums = [1,2,3,4,5,6].
The total cost is 0 + 0 + 0 + 0 + 1 + 2 = 3.
```

Example 3:

```
Input: instructions = [1,3,3,3,2,4,2,1,2]
Output: 4
Explanation: Begin with nums = [].
Insert 1 with cost min(0, 0) = 0, now nums = [1].
Insert 3 with cost min(1, 0) = 0, now nums = [1,3].
Insert 3 with cost min(1, 0) = 0, now nums = [1,3,3].
Insert 3 with cost min(1, 0) = 0, now nums = [1,3,3,3].
Insert 2 with cost min(1, 3) = 1, now nums = [1,2,3,3,3].
Insert 4 with cost min(5, 0) = 0, now nums = [1,2,3,3,3,4].
Insert 2 with cost min(1, 4) = 1, now nums = [1,2,2,3,3,3,4].
Insert 1 with cost min(0, 6) = 0, now nums = [1,1,2,2,3,3,3,4].
Insert 2 with cost min(2, 4) = 2, now nums = [1,1,2,2,2,3,3,3,4].
The total cost is 0 + 0 + 0 + 0 + 1 + 0 + 1 + 0 + 2 = 4.
```

Constraints:

- `1 <= instructions.length <= 10^5`
- `1 <= instructions[i] <= 10^5`

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

The cost of inserting x is the min of how many existing values are strictly smaller and strictly larger. 10^5 insertions make a linear scan per step too slow. The value domain is also 10^5, so a Fenwick tree queries a prefix count and adds one in O(log M). Before inserting x, add min(query(x-1), i-query(x)), then update(x,1), and reduce modulo.

Python 3

```python
class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x: int, v: int):
        while x <= self.n:
            self.c[x] += v
            x += x & -x

    def query(self, x: int) -> int:
        s = 0
        while x:
            s += self.c[x]
            x -= x & -x
        return s


class Solution:
    def createSortedArray(self, instructions: List[int]) -> int:
        m = max(instructions)
        tree = BinaryIndexedTree(m)
        ans = 0
        mod = 10**9 + 7
        for i, x in enumerate(instructions):
            cost = min(tree.query(x - 1), i - tree.query(x))
            ans += cost
            tree.update(x, 1)
        return ans % mod
```

Solution 2

> **How to think about it**
>
> Solution 1 stores frequencies in a Fenwick tree. A segment tree with range sums answers the same prefix counts. Point add plus queries on [1,x) and (x,M] match the Fenwick version. The Python port is marked TLE here; Java and C++ pass.

Python 3

```python
class Node:
    def __init__(self):
        self.l = 0
        self.r = 0
        self.v = 0


class SegmentTree:
    def __init__(self, n):
        self.tr = [Node() for _ in range(4 * n)]
        self.build(1, 1, n)

    def build(self, u, l, r):
        self.tr[u].l = l
        self.tr[u].r = r
        if l == r:
            return
        mid = (l + r) >> 1
        self.build(u << 1, l, mid)
        self.build(u << 1 | 1, mid + 1, r)

    def modify(self, u, x, v):
        if self.tr[u].l == x and self.tr[u].r == x:
            self.tr[u].v += v
            return
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        if x <= mid:
            self.modify(u << 1, x, v)
        else:
            self.modify(u << 1 | 1, x, v)
        self.pushup(u)

    def pushup(self, u):
        self.tr[u].v = self.tr[u << 1].v + self.tr[u << 1 | 1].v

    def query(self, u, l, r):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            return self.tr[u].v
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        v = 0
        if l <= mid:
            v = self.query(u << 1, l, r)
        if r > mid:
            v += self.query(u << 1 | 1, l, r)
        return v


class Solution:
    def createSortedArray(self, instructions: List[int]) -> int:
        n = max(instructions)
        tree = SegmentTree(n)
        ans = 0
        for num in instructions:
            a = tree.query(1, 1, num - 1)
            b = tree.query(1, 1, n) - tree.query(1, 1, num)
            ans += min(a, b)
            tree.modify(1, num, 1)
        return ans % int((1e9 + 7))
```

Complexity. Time: O(m * log M), m = len(instructions), M = largest value. Space: O(M).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Use a segment tree of counts
# 2. Return the cost of every single insertion


# ============================================================
# FOLLOW-UP 1: Use a segment tree of counts
# ============================================================

# ---- follow-up 1: function ----
# Answer: A tree over the values: add one at the value,
# and ask for the count of values below or above by a
# range query.
# Time: O(n log M), M = largest value.  Space: O(M).
class CountTree:
    def __init__(self, size):
        self.size = 1
        while self.size < size + 2:
            self.size *= 2
        self.tree = [0] * (2 * self.size)

    def add(self, position):
        i = position + self.size
        while i >= 1:
            self.tree[i] += 1
            i //= 2

    def count(self, low, high):
        # how many added positions lie in low..high
        if low > high:
            return 0
        total = 0
        left, right = low + self.size, high + self.size + 1
        while left < right:
            if left % 2 == 1:
                total += self.tree[left]
                left += 1
            if right % 2 == 1:
                right -= 1
                total += self.tree[right]
            left //= 2
            right //= 2
        return total


def insertion_cost_with_segment_tree(instructions):
    mod = 10 ** 9 + 7
    top = max(instructions)
    tree = CountTree(top)
    cost = 0
    for x in instructions:
        smaller = tree.count(0, x - 1)
        larger = tree.count(x + 1, top)
        cost += min(smaller, larger)
        tree.add(x)
    return cost % mod


# ============================================================
# FOLLOW-UP 2: Return the cost of every single insertion
```

```
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Keep a sorted list; for each number, binary
# search the count of smaller and larger numbers.
# Time: O(n^2) (insort shifts the list).  Space: O(n).
import bisect


def cost_of_every_insertion(instructions):
    sorted_list = []
    costs = []
    for x in instructions:
        smaller = bisect.bisect_left(sorted_list, x)
        larger = len(sorted_list) - bisect.bisect_right(sorted_list, x)
        costs.append(min(smaller, larger))
        bisect.insort(sorted_list, x)
    return costs
```

PATTERN 40 · SEGMENT TREE · PROBLEM 6 OF 20

2080. Range Frequency Queries

Medium · Design, Segment Tree, Array, Hash Table, Binary Search

leetcode.com/problems/range-frequency-queries

Also appears in pattern 41 Binary Indexed Tree (Fenwick Tree)

PROBLEM

Design a data structure to find the **frequency** of a given value in a given subarray.

The **frequency** of a value in a subarray is the number of occurrences of that value in the subarray.

Implement the `RangeFreqQuery` class:

- `RangeFreqQuery(int[] arr)` Constructs an instance of the class with the given **0-indexed** integer array `arr`.
- `int query(int left, int right, int value)` Returns the **frequency** of `value` in the subarray `arr[left...right]`.

A **subarray** is a contiguous sequence of elements within an array. `arr[left...right]` denotes the subarray that contains the elements of `nums` between indices `left` and `right` (**inclusive**).

Example 1:

```
Input
["RangeFreqQuery", "query", "query"]
[[[12, 33, 4, 56, 22, 2, 34, 33, 22, 12, 34, 56]], [1, 2, 4], [0, 11, 33]]
Output
[null, 1, 2]

Explanation
RangeFreqQuery rangeFreqQuery = new RangeFreqQuery([12, 33, 4, 56, 22, 2, 34, 33, 22, 12, 34, 56]);
rangeFreqQuery.query(1, 2, 4); // return 1. The value 4 occurs 1 time in the subarray [33, 4]
rangeFreqQuery.query(0, 11, 33); // return 2. The value 33 occurs 2 times in the whole array.
```

Constraints:

- $1 <= arr.length <= 10^5$
- $1 <= arr[i], value <= 10^4$
- $0 <= left <= right < arr.length$
- At most 10^5 calls will be made to `query`

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Binary Search

> **How to think about it**
>
> Many range frequency queries on a static array. n and the query count are 10^5; a segment tree works, but one value only needs its index list. A hash map stores increasing indices; two binary searches on [left,right] yield the count.

We use a hash table g to store the array of indices corresponding to each value. In the constructor, we traverse the array arr, adding the index corresponding to each value to the hash table.

In the query function, we first check whether the given value exists in the hash table. If it does not exist, it means that the value does not exist in the array, so we directly return 0. Otherwise, we get the index array idx corresponding to the value. Then we use binary search to find the first index l that is greater than or equal to left, and the first index r that is greater than right. Finally, we return r - l.

In terms of time complexity, the time complexity of the constructor is O(n), and the time complexity of the query function is O(log n). The space complexity is O(n). Where n is the length of the array.

Python 3

```python
class RangeFreqQuery:

    def __init__(self, arr: List[int]):
        self.g = defaultdict(list)
        for i, x in enumerate(arr):
            self.g[x].append(i)

    def query(self, left: int, right: int, value: int) -> int:
        idx = self.g[value]
        l = bisect_left(idx, left)
        r = bisect_left(idx, right + 1)
        return r - l


# Your RangeFreqQuery object will be instantiated and called as such:
# obj = RangeFreqQuery(arr)
# param_1 = obj.query(left,right,value)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Values can change: set(index, value)
# 2. Count the numbers whose value is between lo and hi
# inside an index range


# ============================================================
# FOLLOW-UP 1: Values can change: set(index, value)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep a sorted list of positions for each value;
# an update removes the index from the old list and
# inserts it into the new one.
# Time: set O(n), query O(log n).  Space: O(n).
import bisect


class RangeFreqWithUpdates:
    def __init__(self, arr):
        self.arr = list(arr)
        self.positions = {}
        for i in range(len(arr)):
            self.positions.setdefault(arr[i], []).append(i)

    def set(self, index, value):
        old = self.arr[index]
        spots = self.positions[old]
        spots.pop(bisect.bisect_left(spots, index))
        bisect.insort(self.positions.setdefault(value, []), index)
        self.arr[index] = value

    def query(self, left, right, value):
        spots = self.positions.get(value, [])
        return bisect.bisect_right(spots, right) - bisect.bisect_left(spots, left)


# ============================================================
# FOLLOW-UP 2: Count the numbers whose value is between lo
# and hi inside an index range
# ============================================================

# ---- follow-up 2: function ----
# Answer: Add up the queries of every distinct value in
# that value range.
```

```
# Time: O(D log n), D = distinct values.  Space: O(1)
# extra.
def count_values_between(container, left, right, low, high):
    total = 0
    for value in container.positions:
        if low <= value <= high:
            total += container.query(left, right, value)
    return total
```

2424. Longest Uploaded Prefix

Medium · Union Find, Design, Binary Indexed Tree, Segment Tree, Hash Table, Binary Search, Ordered Set, Heap (Priority Queue)

leetcode.com/problems/longest-uploaded-prefix

PROBLEM

You are given a stream of `n` videos, each represented by a **distinct** number from `1` to `n` that you need to "upload" to a server. You need to implement a data structure that calculates the length of the **longest uploaded prefix** at various points in the upload process.

We consider `i` to be an uploaded prefix if all videos in the range `1` to `i` (**inclusive**) have been uploaded to the server. The longest uploaded prefix is the **maximum** value of `i` that satisfies this definition.

Implement the `LUPrefix` class:

- `LUPrefix(int n)` Initializes the object for a stream of `n` videos.
- `void upload(int video)` Uploads `video` to the server.
- `int longest()` Returns the length of the **longest uploaded prefix** defined above.

Example 1:

```
Input
["LUPrefix", "upload", "longest", "upload", "longest", "upload", "longest"]
[[4], [3], [], [1], [], [2], []]
Output
[null, null, 0, null, 1, null, 3]

Explanation
LUPrefix server = new LUPrefix(4); // Initialize a stream of 4 videos.
server.upload(3); // Upload video 3.
server.longest(); // Since video 1 has not been uploaded yet, there is no prefix.
                                    // So, we return 0.
server.upload(1); // Upload video 1.
server.longest(); // The prefix [1] is the longest uploaded prefix, so we return 1.
server.upload(2); // Upload video 2.
server.longest(); // The prefix [1,2,3] is the longest uploaded prefix, so we return 3.
```

Constraints:

- `1 <= n <= 10^5`
- `1 <= video <= n`
- All values of `video` are **distinct**.
- At most `2 * 10^5` calls **in total** will be made to `upload` and `longest`.
- At least one call will be made to `longest`.

SOLUTION (PYTHON 3)

Solution 1: Simulation

> **How to think about it**
>
> Up to 2× 10^5 uploads and queries forbid scanning [1,n] each time. The longest prefix is the largest r such that 1..r are all uploaded; only uploading r+1 can extend it. Store uploaded ids in a set and keep r. After an upload, increment r while r+1 is present. Each id advances r at most once, so the extra work is linear overall.

We use a variable r to record the current longest prefix of uploaded videos, and an array or hash table s to record the videos that have been uploaded.

Each time a video is uploaded, we set `s[video]` to `true`, then loop to check whether `s[r + 1]` is `true`. If it is, we update r.

The time complexity is O(n), and the space complexity is O(n). Here, n is the total number of videos.

Python 3

```
class LUPrefix:
    def __init__(self, n: int):
        self.r = 0
        self.s = set()

    def upload(self, video: int) -> None:
        self.s.add(video)
        while self.r + 1 in self.s:
            self.r += 1

    def longest(self) -> int:
        return self.r


# Your LUPrefix object will be instantiated and called as such:
# obj = LUPrefix(n)
# obj.upload(video)
# param_2 = obj.longest()
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Videos can also be removed again
# 2. Segment tree version: keep counts of uploaded videos
# per range


# ============================================================
# FOLLOW-UP 1: Videos can also be removed again
# ============================================================

# ---- follow-up 1: function ----
# Answer: If the removed video is inside the prefix, the
# prefix shrinks to just before it.
# Time: upload O(n) in the worst case (the pointer can
# walk far again after a remove), remove O(1), longest
# O(1).  Space: O(n).
class LUPrefixWithRemove:
    def __init__(self, n):
        self.uploaded = set()
        self.prefix = 0

    def upload(self, video):
        self.uploaded.add(video)
        while self.prefix + 1 in self.uploaded:
            self.prefix += 1

    def remove(self, video):
        if video in self.uploaded:
            self.uploaded.discard(video)
            if video <= self.prefix:
                self.prefix = video - 1

    def longest(self):
        return self.prefix


# ============================================================
# FOLLOW-UP 2: Segment tree version: keep counts of
# uploaded videos per range
```

```
# ============================================================

# ---- follow-up 2: function ----
# Answer: A node is full when its whole range is uploaded;
# walking down from the root finds the first video that is
# not uploaded.
# Time: upload O(log n), longest O(log n).  Space: O(n).
class LUPrefixTree:
    def __init__(self, n):
        self.size = 1
        while self.size < n:
            self.size *= 2
        self.full = [0] * (2 * self.size)
        self.n = n

    def upload(self, video):
        i = video - 1 + self.size
        self.full[i] = 1
        i //= 2
        while i >= 1:
            self.full[i] = self.full[2 * i] & self.full[2 * i + 1]
            i //= 2

    def longest(self):
        if self.full[1] == 1 and self.size == self.n:
            return self.n
        node = 1
        while node < self.size:
            if self.full[2 * node] == 1:
                node = 2 * node + 1
            else:
                node = 2 * node
        first_missing = node - self.size
        return min(first_missing, self.n)
```

850. Rectangle Area II

Hard · Segment Tree, Array, Ordered Set, Sweep Line

leetcode.com/problems/rectangle-area-ii

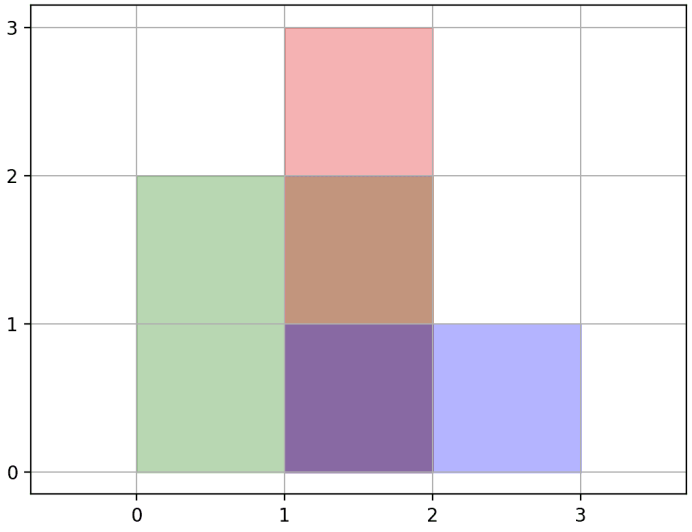
PROBLEM

You are given a 2D array of axis-aligned `rectangles`. Each `rectangle[i] = [$x_{i1}$, $y_{i1}$, $x_{i2}$, $y_{i2}$]` denotes the i^{th} rectangle where (x_{i1}, y_{i1}) are the coordinates of the **bottom-left corner**, and (x_{i2}, y_{i2}) are the coordinates of the **top-right corner**.

Calculate the **total area** covered by all `rectangles` in the plane. Any area covered by two or more rectangles should only be counted **once**.

Return *the **total area***. Since the answer may be too large, return it **modulo** $10^9 + 7$.

Example 1:




```
Input: rectangles = [[0,0,2,2],[1,0,2,3],[1,0,3,1]]
Output: 6
Explanation: A total area of 6 is covered by all three rectangles, as illustrated in the picture.
From (1,1) to (2,2), the green and red rectangles overlap.
From (1,0) to (2,3), all three rectangles overlap.
```

Example 2:

```
Input: rectangles = [[0,0,1000000000,1000000000]]
Output: 49
Explanation: The answer is 1018 modulo (109 + 7), which is 49.
```

Constraints:

- `1 <= rectangles.length <= 200`
- `rectanges[i].length == 4`
- `0 <=` $x_{i1}, y_{i1}, x_{i2}, y_{i2} <= 10^9$
- $x_{i1} <= x_{i2}$
- $y_{i1} <= y_{i2}$
- All rectangles have non zero area.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

We need the area of a union of rectangles with coordinates up to 10^9 and at most 200 rectangles. Inclusion-exclusion explodes with intersections. A vertical sweep is stabler: left and right edges are events, and the measure between them is the covered y-length. Discretize y-coordinates and let a segment tree store the length where the cover count is positive. Sorting events by x, each gap Δ x times the tree length is the area of that slab.

Python 3

```python
class Node:
    def __init__(self):
        self.l = self.r = 0
        self.cnt = self.length = 0


class SegmentTree:
    def __init__(self, nums):
        n = len(nums) - 1
        self.nums = nums
        self.tr = [Node() for _ in range(n << 2)]
        self.build(1, 0, n - 1)

    def build(self, u, l, r):
        self.tr[u].l, self.tr[u].r = l, r
        if l != r:
            mid = (l + r) >> 1
            self.build(u << 1, l, mid)
            self.build(u << 1 | 1, mid + 1, r)

    def modify(self, u, l, r, k):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            self.tr[u].cnt += k
        else:
            mid = (self.tr[u].l + self.tr[u].r) >> 1
            if l <= mid:
                self.modify(u << 1, l, r, k)
            if r > mid:
                self.modify(u << 1 | 1, l, r, k)
        self.pushup(u)

    def pushup(self, u):
        if self.tr[u].cnt:
            self.tr[u].length = self.nums[self.tr[u].r + 1] - self.nums[self.tr[u].l]
        elif self.tr[u].l == self.tr[u].r:
            self.tr[u].length = 0
        else:
            self.tr[u].length = self.tr[u << 1].length + self.tr[u << 1 | 1].length

    @property
    def length(self):
        return self.tr[1].length


class Solution:
    def rectangleArea(self, rectangles: List[List[int]]) -> int:
        segs = []
        alls = set()
        for x1, y1, x2, y2 in rectangles:
            segs.append((x1, y1, y2, 1))
            segs.append((x2, y1, y2, -1))
            alls.update([y1, y2])

        segs.sort()
        alls = sorted(alls)
        tree = SegmentTree(alls)
        m = {v: i for i, v in enumerate(alls)}
        ans = 0
        for i, (x, y1, y2, k) in enumerate(segs):
            if i:
                ans += tree.length * (x - segs[i - 1][0])
            tree.modify(1, m[y1], m[y2] - 1, k)
        ans %= int(1e9 + 7)
        return ans
```

Complexity. Time: O(n log n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Area covered by at least k rectangles
# 2. The largest number of rectangles that overlap at a
# single point


# ============================================================
# FOLLOW-UP 1: Area covered by at least k rectangles
# ============================================================

# ---- follow-up 1: function ----
# Answer: Compress the coordinates into a small grid,
# count the rectangles that cover each grid cell, and add
# up the cells covered at least k times.
# Time: O(n^3).  Space: O(n).
def area_covered_at_least(rectangles, k):
    xs = sorted(set([r[0] for r in rectangles] + [r[2] for r in rectangles]))
    ys = sorted(set([r[1] for r in rectangles] + [r[3] for r in rectangles]))
    total = 0
    for i in range(len(xs) - 1):
        for j in range(len(ys) - 1):
            covering = 0
            for x1, y1, x2, y2 in rectangles:
                if x1 <= xs[i] and xs[i + 1] <= x2 and y1 <= ys[j] and ys[j + 1] <= y2:
                    covering += 1
            if covering >= k:
                total += (xs[i + 1] - xs[i]) * (ys[j + 1] - ys[j])
    return total


# ============================================================
# FOLLOW-UP 2: The largest number of rectangles that
# overlap at a single point
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the covering rectangles of every
# compressed cell and take the maximum.
# Time: O(n^3).  Space: O(n).
def deepest_overlap(rectangles):
    xs = sorted(set([r[0] for r in rectangles] + [r[2] for r in rectangles]))
    ys = sorted(set([r[1] for r in rectangles] + [r[3] for r in rectangles]))
    best = 0
    for i in range(len(xs) - 1):
        for j in range(len(ys) - 1):
            covering = 0
            for x1, y1, x2, y2 in rectangles:
                if x1 <= xs[i] and xs[i + 1] <= x2 and y1 <= ys[j] and ys[j + 1] <= y2:
                    covering += 1
            best = max(best, covering)
    return best
```

2407. Longest Increasing Subsequence II

Hard · Binary Indexed Tree, Segment Tree, Queue, Array, Divide and Conquer, Dynamic Programming, Monotonic Queue

leetcode.com/problems/longest-increasing-subsequence-ii

PROBLEM

You are given an integer array `nums` and an integer `k`.

Find the longest subsequence of `nums` that meets the following requirements:

- The subsequence is **strictly increasing** and
- The difference between adjacent elements in the subsequence is **at most** `k`.

Return *the length of the **longest subsequence** that meets the requirements.*

A **subsequence** is an array that can be derived from another array by deleting some or no elements without changing the order of the remaining elements.

Example 1:

```
Input: nums = [4,2,1,4,3,4,5,8,15], k = 3
Output: 5
Explanation:
The longest subsequence that meets the requirements is [1,3,4,5,8].
The subsequence has a length of 5, so we return 5.
Note that the subsequence [1,3,4,5,8,15] does not meet the requirements because 15 - 8 = 7 is larger than 3.
```

Example 2:

```
Input: nums = [7,4,5,1,8,12,4,7], k = 5
Output: 4
Explanation:
The longest subsequence that meets the requirements is [4,5,8,12].
The subsequence has a length of 4, so we return 4.
```

Example 3:

```
Input: nums = [1,5], k = 1
Output: 1
Explanation:
The longest subsequence that meets the requirements is [1].
The subsequence has a length of 1, so we return 1.
```

Constraints:

- `1 <= nums.length <= 10^5`
- `1 <= nums[i], k <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: Segment Tree

> **How to think about it**
>
> Classic LIS is $O(n^2)$, which fails at $n \le 10^5$, and adjacent values may differ by at most k. The best length ending at v is one plus the maximum f on [v-k,v-1], so we need range-max queries and point updates on the value domain. Values lie in $[1,10^5]$. A segment tree stores f[v]; for each v query then update, in O(nlog V).

We assume that f[v] represents the length of the longest increasing subsequence ending with the number v.

We traverse each element v in the array nums, with the state transition equation: f[v] = max(f[v], f[x]), where the range of x is [v-k, v-1].

Therefore, we need a data structure to maintain the maximum value of the interval. It is not difficult to think of using a segment tree.

The segment tree divides the entire interval into multiple discontinuous subintervals, and the number of subintervals does not exceed log(width). To update the value of an element, only log(width) intervals need to be updated, and these intervals are all contained in a large interval that contains the element.

- Each node of the segment tree represents an interval;
- The segment tree has a unique root node, which represents the entire statistical range, such as [1,N];
- Each leaf node of the segment tree represents an elementary interval of length 1, [x, x];
- For each internal node [l,r], its left child is [l,mid], and the right child is [mid+1,r], where mid = floor((l+r)/2).

For this problem, the information maintained by the segment tree node is the maximum value within the interval range.

The time complexity is O(n × log n), where n is the length of the array nums.

Python 3

```python
class Node:
    def __init__(self):
        self.l = 0
        self.r = 0
        self.v = 0


class SegmentTree:
    def __init__(self, n):
        self.tr = [Node() for _ in range(4 * n)]
        self.build(1, 1, n)

    def build(self, u, l, r):
        self.tr[u].l = l
        self.tr[u].r = r
        if l == r:
            return
        mid = (l + r) >> 1
        self.build(u << 1, l, mid)
        self.build(u << 1 | 1, mid + 1, r)

    def modify(self, u, x, v):
        if self.tr[u].l == x and self.tr[u].r == x:
            self.tr[u].v = v
            return
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        if x <= mid:
            self.modify(u << 1, x, v)
        else:
            self.modify(u << 1 | 1, x, v)
        self.pushup(u)

    def pushup(self, u):
        self.tr[u].v = max(self.tr[u << 1].v, self.tr[u << 1 | 1].v)

    def query(self, u, l, r):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            return self.tr[u].v
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        v = 0
        if l <= mid:
            v = self.query(u << 1, l, r)
        if r > mid:
            v = max(v, self.query(u << 1 | 1, l, r))
        return v


class Solution:
    def lengthOfLIS(self, nums: List[int], k: int) -> int:
        tree = SegmentTree(max(nums))
        ans = 1
        for v in nums:
            t = tree.query(1, v - k, v - 1) + 1
            ans = max(ans, t)
            tree.modify(1, v, t)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subsequence itself
# 2. Equal neighbours are also allowed (difference between
# 0 and k)


# ============================================================
# FOLLOW-UP 1: Return the subsequence itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Table over the elements: the best length ending
# at each element and the previous element that gave it.
# Time: O(n^2).  Space: O(n).
def lis_with_difference_limit(nums, k):
    n = len(nums)
    length = [1] * n
    previous = [-1] * n
    for i in range(n):
        for j in range(i):
            if 0 < nums[i] - nums[j] <= k and length[j] + 1 > length[i]:
                length[i] = length[j] + 1
                previous[i] = j
    end = max(range(n), key=lambda i: length[i])
    answer = []
    while end != -1:
        answer.append(nums[end])
        end = previous[end]
    return answer[::-1]


# ============================================================
# FOLLOW-UP 2: Equal neighbours are also allowed
# (difference between 0 and k)
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same table with the condition 0 <=
# difference <= k.
# Time: O(n^2).  Space: O(n).
def lis_non_strict_with_limit(nums, k):
    n = len(nums)
    length = [1] * n
    for i in range(n):
        for j in range(i):
            if 0 <= nums[i] - nums[j] <= k:
                length[i] = max(length[i], length[j] + 1)
    return max(length)
```

2569. Handling Sum Queries After Update

Hard · Segment Tree, Array

leetcode.com/problems/handling-sum-queries-after-update

PROBLEM

You are given two **0-indexed** arrays `nums1` and `nums2` and a 2D array `queries` of queries. There are three types of queries:

1. For a query of type 1, `queries[i] = [1, l, r]`. Flip the values from `0` to `1` and from `1` to `0` in `nums1` from index `l` to index `r`. Both `l` and `r` are **0-indexed**.
2. For a query of type 2, `queries[i] = [2, p, 0]`. For every index `0 <= i < n`, set `nums2[i] = nums2[i] + nums1[i] * p`.
3. For a query of type 3, `queries[i] = [3, 0, 0]`. Find the sum of the elements in `nums2`.

Return *an array containing all the answers to the third type queries.*

Example 1:

```
Input: nums1 = [1,0,1], nums2 = [0,0,0], queries = [[1,1,1],[2,1,0],[3,0,0]]
Output: [3]
Explanation: After the first query nums1 becomes [1,1,1]. After the second query, nums2 becomes [1,1,1], so the answer to the third query is 3. Thus, [3] is returned.
```

Example 2:

```
Input: nums1 = [1], nums2 = [5], queries = [[2,0,0],[3,0,0]]
Output: [5]
Explanation: After the first query, nums2 remains [5], so the answer to the second query is 5. Thus, [5] is returned.
```

Constraints:

- `1 <= nums1.length,nums2.length <= 10^5`
- `nums1.length = nums2.length`
- `1 <= queries.length <= 10^5`
- `queries[i].length = 3`
- `0 <= l <= r <= nums1.length - 1`
- `0 <= p <= 10^6`
- `0 <= nums1[i] <= 1`
- `0 <= nums2[i] <= 10^9`

SOLUTION (PYTHON 3)

Solution 1: Segment Tree

How to think about it

Three operations: flip a range of nums1; add p×∑ nums1 to every element of nums2; report ∑ nums2. n,q≤ 10^5 forbids walking a range on every flip. ∑ nums2 is a single running total; only the range sum of nums1 matters for the second operation. A segment tree stores how many ones sit in each interval, with a lazy flip that replaces a sum s by length-s. The third operation just records the current total.

According to the problem description:

- Operation 1 is to reverse all numbers in the index range [l,..r] of array `nums1`, that is, change 0 to 1 and 1 to 0.
- Operation 3 is to sum all numbers in array `nums2`.

- Operation 2 is to add the sum of all numbers in array `nums2` with p times the sum of all numbers in array `nums1`, that is, sum(nums2) = sum(nums2) + p * sum(nums1).

Therefore, we actually only need to maintain the segment sum of array `nums1`, which can be implemented through a segment tree.

We define each node of the segment tree as `Node`, each node contains the following attributes:

- `l`: The left endpoint of the node, the index starts from 1.
- `r`: The right endpoint of the node, the index starts from 1.
- `s`: The segment sum of the node.
- `lazy`: The lazy tag of the node.

The segment tree mainly has the following operations:

- `build(u, l, r)`: Build the segment tree.
- `pushdown(u)`: Propagate the lazy tag.
- `pushup(u)`: Update the information of the parent node with the information of the child nodes.
- `modify(u, l, r)`: Modify the segment sum. In this problem, it is to reverse each number in the segment, so the segment sum s = r - l + 1 - s.
- `query(u, l, r)`: Query the segment sum.

First, calculate the sum of all numbers in array `nums2`, denoted as s.

When executing operation 1, we only need to call `modify(1, l + 1, r + 1)`.

When executing operation 2, we update s = s + p × query(1, 1, n).

When executing operation 3, we just need to add s to the answer array.

The time complexity is O(n + m × log n), and the space complexity is O(n). Where n and m are the lengths of arrays `nums1` and `queries` respectively.

Python 3

```python
class Node:
    def __init__(self):
        self.l = self.r = 0
        self.s = self.lazy = 0


class SegmentTree:
    def __init__(self, nums):
        self.nums = nums
        n = len(nums)
        self.tr = [Node() for _ in range(n << 2)]
        self.build(1, 1, n)

    def build(self, u, l, r):
        self.tr[u].l, self.tr[u].r = l, r
        if l == r:
            self.tr[u].s = self.nums[l - 1]
            return
        mid = (l + r) >> 1
        self.build(u << 1, l, mid)
        self.build(u << 1 | 1, mid + 1, r)
        self.pushup(u)

    def modify(self, u, l, r):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            self.tr[u].lazy ^= 1
            self.tr[u].s = self.tr[u].r - self.tr[u].l + 1 - self.tr[u].s
            return
        self.pushdown(u)
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        if l <= mid:
            self.modify(u << 1, l, r)
        if r > mid:
            self.modify(u << 1 | 1, l, r)
        self.pushup(u)

    def query(self, u, l, r):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            return self.tr[u].s
        self.pushdown(u)
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        res = 0
        if l <= mid:
            res += self.query(u << 1, l, r)
        if r > mid:
            res += self.query(u << 1 | 1, l, r)
        return res

    def pushup(self, u):
        self.tr[u].s = self.tr[u << 1].s + self.tr[u << 1 | 1].s

    def pushdown(self, u):
        if self.tr[u].lazy:
            mid = (self.tr[u].l + self.tr[u].r) >> 1
            self.tr[u << 1].s = mid - self.tr[u].l + 1 - self.tr[u << 1].s
            self.tr[u << 1].lazy ^= 1
            self.tr[u << 1 | 1].s = self.tr[u].r - mid - self.tr[u << 1 | 1].s
            self.tr[u << 1 | 1].lazy ^= 1
            self.tr[u].lazy ^= 1
```

```python
class Solution:
    def handleQuery(
        self, nums1: List[int], nums2: List[int], queries: List[List[int]]
    ) -> List[int]:
        tree = SegmentTree(nums1)
        s = sum(nums2)
```

```
        ans = []
        for op, a, b in queries:
            if op == 1:
                tree.modify(1, a + 1, b + 1)
            elif op == 2:
                s += a * tree.query(1, 1, len(nums1))
            else:
                ans.append(s)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. A segment tree that flips a range and counts the ones
# 2. Answer all three kinds of queries with that tree


# ============================================================
# FOLLOW-UP 1: A segment tree that flips a range and
# counts the ones
# ============================================================

# ---- follow-up 1: function ----
# Answer: Each node stores its number of ones; a lazy flag
# says that the whole node is waiting to be flipped (ones
# become length minus ones).
# Time: build O(n), flip O(log n), count_ones O(log n).
# Space: O(n).
class FlipTree:
    def __init__(self, bits):
        self.n = len(bits)
        self.ones = [0] * (4 * self.n)
        self.lazy = [False] * (4 * self.n)
        self.build(1, 0, self.n - 1, bits)

    def build(self, node, lo, hi, bits):
        if lo == hi:
            self.ones[node] = bits[lo]
            return
        mid = (lo + hi) // 2
        self.build(2 * node, lo, mid, bits)
        self.build(2 * node + 1, mid + 1, hi, bits)
        self.ones[node] = self.ones[2 * node] + self.ones[2 * node + 1]

    def apply_flip(self, node, lo, hi):
        self.ones[node] = (hi - lo + 1) - self.ones[node]
        self.lazy[node] = not self.lazy[node]

    def push_down(self, node, lo, hi):
        if self.lazy[node]:
            mid = (lo + hi) // 2
            self.apply_flip(2 * node, lo, mid)
            self.apply_flip(2 * node + 1, mid + 1, hi)
            self.lazy[node] = False

    def flip(self, left, right, node=1, lo=0, hi=None):
        if hi is None:
            hi = self.n - 1
        if right < lo or hi < left:
            return
        if left <= lo and hi <= right:
            self.apply_flip(node, lo, hi)
            return
        self.push_down(node, lo, hi)
        mid = (lo + hi) // 2
        self.flip(left, right, 2 * node, lo, mid)
```

```
        self.flip(left, right, 2 * node + 1, mid + 1, hi)
        self.ones[node] = self.ones[2 * node] + self.ones[2 * node + 1]

    def count_ones(self, left, right, node=1, lo=0, hi=None):
        if hi is None:
            hi = self.n - 1
```

```
        if right < lo or hi < left:
            return 0
        if left <= lo and hi <= right:
            return self.ones[node]
        self.push_down(node, lo, hi)
        mid = (lo + hi) // 2
        return self.count_ones(left, right, 2 * node, lo, mid) + self.count_ones(left, right, 2 * node +
1, mid + 1, hi)


# ============================================================
# FOLLOW-UP 2: Answer all three kinds of queries with that
# tree
# ============================================================

# ---- follow-up 2: function ----
# Answer: Type 1 flips a range, type 2 adds p times the
# number of ones to the running sum, and type 3 reports
# the sum.
# Time: O(n + q log n).  Space: O(n).
def handle_queries(nums1, nums2, queries):
    tree = FlipTree(nums1)
    total = sum(nums2)
    answers = []
    for kind, a, b in queries:
        if kind == 1:
            tree.flip(a, b)
        elif kind == 2:
            total += a * tree.count_ones(0, len(nums1) - 1)
        else:
            answers.append(total)
    return answers
```

2286. Booking Concert Tickets in Groups

Hard · Design, Binary Indexed Tree, Segment Tree, Binary Search

leetcode.com/problems/booking-concert-tickets-in-groups

PROBLEM

A concert hall has `n` rows numbered from `0` to `n - 1`, each with `m` seats, numbered from `0` to `m - 1`. You need to design a ticketing system that can allocate seats in the following cases:

- If a group of `k` spectators can sit **together** in a row.
- If **every** member of a group of `k` spectators can get a seat. They may or **may not** sit together.

Note that the spectators are very picky. Hence:

- They will book seats only if each member of their group can get a seat with row number **less than or equal** to `maxRow`. `maxRow` can **vary** from group to group.
- In case there are multiple rows to choose from, the row with the **smallest** number is chosen. If there are multiple seats to choose in the same row, the seat with the **smallest** number is chosen.

Implement the `BookMyShow` class:

- `BookMyShow(int n, int m)` Initializes the object with `n` as number of rows and `m` as number of seats per row.
- `int[] gather(int k, int maxRow)` Returns an array of length `2` denoting the row and seat number (respectively) of the **first seat** being allocated to the `k` members of the group, who must sit **together**. In other words, it returns the smallest possible `r` and `c` such that all `[c, c + k - 1]` seats are valid and empty in row `r`, and `r <= maxRow`. Returns `[]` in case it is **not possible** to allocate seats to the group.
- `boolean scatter(int k, int maxRow)` Returns `true` if all `k` members of the group can be allocated seats in rows `0` to `maxRow`, who may or **may not** sit together. If the seats can be allocated, it allocates `k` seats to the group with the **smallest** row numbers, and the smallest possible seat numbers in each row. Otherwise, returns `false`.

Example 1:

```
Input
["BookMyShow", "gather", "gather", "scatter", "scatter"]
[[2, 5], [4, 0], [2, 0], [5, 1], [5, 1]]
Output
[null, [0, 0], [], true, false]

Explanation
BookMyShow bms = new BookMyShow(2, 5); // There are 2 rows with 5 seats each
bms.gather(4, 0); // return [0, 0]
                  // The group books seats [0, 3] of row 0.
bms.gather(2, 0); // return []
                  // There is only 1 seat left in row 0,
                  // so it is not possible to book 2 consecutive seats.
bms.scatter(5, 1); // return True
                   // The group books seat 4 of row 0 and seats [0, 3] of row 1.
bms.scatter(5, 1); // return False
                   // There is only one seat left in the hall.
```

Constraints:

- $1 <= n <= 5 * 10^4$
- $1 <= m, k <= 10^9$
- `0 <= maxRow <= n - 1`
- At most $5 * 10^4$ calls **in total** will be made to `gather` and `scatter`.

SOLUTION (PYTHON 3)

Solution 1: Segment Tree

How to think about it

gather needs k consecutive seats in the lowest possible row; scatter only needs any k seats in the lowest rows. There are 5× 10^4 rows of size 10^9, so a seat array or a linear scan per call is too slow. A segment tree on rows stores remaining-seat sums s and per-row maxima mx. gather finds the leftmost row with mx≥ k and subtracts k. scatter checks the range sum, then walks rows from the left deducting seats.

From the problem description, we can deduce the following:

- For the `gather(k, maxRow)` operation, the goal is to seat k people on the same row with consecutive seats. In other words, we need to find the smallest row where the remaining seats are greater than or equal to k.
- For the `scatter(k, maxRow)` operation, we just need to find k seats in total, but we want to minimize the row number. Therefore, we need to find the first row that has more than 0 seats remaining, allocate seats there, and continue searching for the rest.

We can implement this using a segment tree. Each segment tree node contains the following information:

- `l`: The left endpoint of the node's interval
- `r`: The right endpoint of the node's interval
- `s`: The total remaining seats in the interval corresponding to the node
- `mx`: The maximum remaining seats in the interval corresponding to the node

Note that the index range for the segment tree starts from 1.

The operations of the segment tree are as follows:

- `build(u, l, r)`: Builds node u, corresponding to the interval [l, r], and recursively builds its left and right children.
- `modify(u, x, v)`: Starting from node u, finds the first node corresponding to the interval [l, r] where l = r = x, and modifies the `s` and `mx` values of this node to v, then updates the tree upwards.
- `query_sum(u, l, r)`: Starting from node u, calculates the sum of `s` values in the interval [l, r].
- `query_idx(u, l, r, k)`: Starting from node u, finds the first node in the interval [l, r] where `mx` is greater than or equal to k, and returns the left endpoint `l` of this node. When searching, we start from the largest interval [1, maxRow]. Since we need to find the leftmost node with `mx` greater than or equal to k, we check whether the `mx` of the first half of the interval meets the condition. If so, the answer is in the first half, and we recursively search that half. Otherwise, the answer is in the second half, and we search that half recursively.
- `pushup(u)`: Updates the information of node u using the information from its children.

For the `gather(k, maxRow)` operation, we first use `query_idx(1, 1, n, k)` to find the first row where the remaining seats are greater than or equal to k, denoted as i. Then, we use `query_sum(1, i, i)` to get the remaining seats in this row, denoted as s. Next, we use `modify(1, i, s - k)` to modify the remaining seats of this row to s - k, and update the tree upwards. Finally, we return the result [i - 1, m - s].

For the `scatter(k, maxRow)` operation, we first use `query_sum(1, 1, maxRow)` to calculate the total remaining seats in the first maxRow rows, denoted as s. If s < k, there are not enough seats, so we return `false`. Otherwise, we use `query_idx(1, 1, maxRow, 1)` to find the first row where the remaining seats are greater than or equal to 1, denoted as i. Starting from this row, we use `query_sum(1, i, i)` to get the remaining seats in row i, denoted as s_i. If $s_i \geq k$, we directly use `modify(1, i, s_i - k)` to modify the remaining seats of this row to s_i - k, update the tree upwards, and return `true`. Otherwise, we update k = k - s_i, modify the remaining seats of this row to 0, and update the tree upwards. Finally, we return `true`.

Time complexity:

- The initialization time complexity is O(n).
- The time complexity of `gather(k, maxRow)` is O(log n).
- The time complexity of `scatter(k, maxRow)` is O((n + q) × log n).

The overall time complexity is O(n + q × log n), and the space complexity is O(n). Here, n is the number of rows, and q is the number of operations.

Python 3

```
class Node:
    __slots__ = "l", "r", "s", "mx"

    def __init__(self):
        self.l = self.r = 0
        self.s = self.mx = 0


class SegmentTree:
    def __init__(self, n, m):
        self.m = m
        self.tr = [Node() for _ in range(n << 2)]
        self.build(1, 1, n)

    def build(self, u, l, r):
        self.tr[u].l, self.tr[u].r = l, r
        if l == r:
            self.tr[u].s = self.tr[u].mx = self.m
            return
        mid = (l + r) >> 1
        self.build(u << 1, l, mid)
        self.build(u << 1 | 1, mid + 1, r)
        self.pushup(u)

    def modify(self, u, x, v):
        if self.tr[u].l == x and self.tr[u].r == x:
            self.tr[u].s = self.tr[u].mx = v
            return
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        if x <= mid:
            self.modify(u << 1, x, v)
        else:
            self.modify(u << 1 | 1, x, v)
        self.pushup(u)

    def query_sum(self, u, l, r):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            return self.tr[u].s
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        v = 0
        if l <= mid:
            v += self.query_sum(u << 1, l, r)
        if r > mid:
            v += self.query_sum(u << 1 | 1, l, r)
        return v

    def query_idx(self, u, l, r, k):
        if self.tr[u].mx < k:
            return 0
        if self.tr[u].l == self.tr[u].r:
            return self.tr[u].l
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        if self.tr[u << 1].mx >= k:
            return self.query_idx(u << 1, l, r, k)
        if r > mid:
            return self.query_idx(u << 1 | 1, l, r, k)
        return 0

    def pushup(self, u):
        self.tr[u].s = self.tr[u << 1].s + self.tr[u << 1 | 1].s
```

```
        self.tr[u].mx = max(self.tr[u << 1].mx, self.tr[u << 1 | 1].mx)
```

```python
class BookMyShow:
    def __init__(self, n: int, m: int):
        self.n = n
        self.tree = SegmentTree(n, m)

    def gather(self, k: int, maxRow: int) -> List[int]:
        maxRow += 1
        i = self.tree.query_idx(1, 1, maxRow, k)
        if i == 0:
            return []
        s = self.tree.query_sum(1, i, i)
        self.tree.modify(1, i, s - k)
        return [i - 1, self.tree.m - s]

    def scatter(self, k: int, maxRow: int) -> bool:
        maxRow += 1
        if self.tree.query_sum(1, 1, maxRow) < k:
            return False
        i = self.tree.query_idx(1, 1, maxRow, 1)
        for j in range(i, self.n + 1):
            s = self.tree.query_sum(1, j, j)
            if s >= k:
                self.tree.modify(1, j, s - k)
                return True
            k -= s
            self.tree.modify(1, j, 0)
        return True


# Your BookMyShow object will be instantiated and called as such:
# obj = BookMyShow(n, m)
# param_1 = obj.gather(k,maxRow)
# param_2 = obj.scatter(k,maxRow)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Cancel a booking: free seats of a row again
# 2. How many rows still have at least k free seats


# ============================================================
# FOLLOW-UP 1: Cancel a booking: free seats of a row again
# ============================================================

# ---- follow-up 1: function ----
# Answer: Increase the free seat count of the row (and
# move the row's first free seat back).
# Time: gather O(max_row), scatter O(max_row), cancel
# O(1).  Space: O(n).
class BookMyShowWithCancel:
    def __init__(self, n, m):
        self.rows = n
        self.seats = m
        self.taken = [0] * n

    def gather(self, k, max_row):
        for row in range(max_row + 1):
            if self.seats - self.taken[row] >= k:
                start = self.taken[row]
                self.taken[row] += k
                return [row, start]
        return []

    def scatter(self, k, max_row):
        free = 0
```

```
        for row in range(max_row + 1):
            free += self.seats - self.taken[row]
        if free < k:
            return False
        for row in range(max_row + 1):
            use = min(k, self.seats - self.taken[row])
            self.taken[row] += use
            k -= use
            if k == 0:
                break
        return True

    def cancel(self, row, count):
        self.taken[row] = max(0, self.taken[row] - count)


# ============================================================
# FOLLOW-UP 2: How many rows still have at least k free
# seats
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the rows whose free seats reach k.
# Time: O(n).  Space: O(1).
def rows_with_at_least(show, k):
    count = 0
    for row in range(show.rows):
        if show.seats - show.taken[row] >= k:
            count += 1
    return count
```

2940. Find Building Where Alice and Bob Can Meet

Hard · Stack, Binary Indexed Tree, Segment Tree, Array, Binary Search, Monotonic Stack, Heap (Priority Queue)

leetcode.com/problems/find-building-where-alice-and-bob-can-meet

PROBLEM

You are given a **0-indexed** array `heights` of positive integers, where `heights[i]` represents the height of the i^{th} building.

If a person is in building `i`, they can move to any other building `j` if and only if `i < j` and `heights[i] < heights[j]`.

You are also given another array `queries` where `queries[i] = [a_i, b_i]`. On the i^{th} query, Alice is in building a_i while Bob is in building b_i.

Return *an array* `ans` *where* `ans[i]` *is* ***the index of the leftmost building*** *where Alice and Bob can meet on the* i^{th} *query. If Alice and Bob cannot move to a common building on query* `i`, *set* `ans[i]` *to* `-1`.

Example 1:

```
Input: heights = [6,4,8,5,2,7], queries = [[0,1],[0,3],[2,4],[3,4],[2,2]]
Output: [2,5,-1,5,2]
Explanation: In the first query, Alice and Bob can move to building 2 since heights[0] < heights[2]
and heights[1] < heights[2].
In the second query, Alice and Bob can move to building 5 since heights[0] < heights[5] and heights[3]
< heights[5].
In the third query, Alice cannot meet Bob since Alice cannot move to any other building.
In the fourth query, Alice and Bob can move to building 5 since heights[3] < heights[5] and heights[4]
< heights[5].
In the fifth query, Alice and Bob are already in the same building.
For ans[i] != -1, It can be shown that ans[i] is the leftmost building where Alice and Bob can meet.
For ans[i] == -1, It can be shown that there is no building where Alice and Bob can meet.
```

Example 2:

```
Input: heights = [5,3,8,2,6,1,4,6], queries = [[0,7],[3,5],[5,2],[3,0],[1,6]]
Output: [7,6,-1,4,6]
Explanation: In the first query, Alice can directly move to Bob's building since heights[0] <
heights[7].
In the second query, Alice and Bob can move to building 6 since heights[3] < heights[6] and heights[5]
< heights[6].
In the third query, Alice cannot meet Bob since Bob cannot move to any other building.
In the fourth query, Alice and Bob can move to building 4 since heights[3] < heights[4] and heights[0]
< heights[4].
In the fifth query, Alice can directly move to Bob's building since heights[1] < heights[6].
For ans[i] != -1, It can be shown that ans[i] is the leftmost building where Alice and Bob can meet.
For ans[i] == -1, It can be shown that there is no building where Alice and Bob can meet.
```

Constraints:

- $1 \le heights.length \le 5 * 10^4$
- $1 \le heights[i] \le 10^9$
- $1 \le queries.length \le 5 * 10^4$
- $queries[i] = [a_i, b_i]$
- $0 \le a_i, b_i \le heights.length - 1$

SOLUTION (PYTHON 3)

Solution 1: Binary Indexed Tree

> **How to think about it**
>
> They meet at some j ≥ max(a,b) strictly taller than the shorter one (or simply at the right building when it is already taller). Scanning the suffix per query is too slow for n,q ≤ 5×10^4. Process queries by decreasing right endpoint and insert buildings to the right of r into a Fenwick tree keyed by compressed height and storing the smallest index. A query for the smallest index taller than heights[l] is the meeting building.

Let's denote queries[i] = $[l_i, r_i]$, where $l_i \le r_i$. If $l_i = r_i$ or heights[l_i] < heights[r_i], then the answer is r_i. Otherwise, we need to find the smallest j among all $j > r_i$ and heights[j] > heights[l_i].

We can sort queries in descending order of r_i, and use a pointer j to point to the current index of heights being traversed.

Next, we traverse each query queries[i] = (l, r). For the current query, if j > r, then we loop to insert heights[j] into the binary indexed tree. The binary indexed tree maintains the minimum index of the suffix height (after discretization). Then, we judge whether l = r or heights[l] < heights[r]. If it is, then the answer to the current query is r. Otherwise, we query the minimum index of heights[l] in the binary indexed tree, which is the answer to the current query.

The time complexity is O((n + m) × log n + m × log m), and the space complexity is O(n + m). Where n and m are the lengths of heights and queries respectively.

Similar problems:

- 2736. Maximum Sum Queries

Python 3

```python
class BinaryIndexedTree:
    __slots__ = ["n", "c"]

    def __init__(self, n: int):
        self.n = n
        self.c = [inf] * (n + 1)

    def update(self, x: int, v: int):
        while x <= self.n:
            self.c[x] = min(self.c[x], v)
            x += x & -x

    def query(self, x: int) -> int:
        mi = inf
        while x:
            mi = min(mi, self.c[x])
            x -= x & -x
        return -1 if mi == inf else mi


class Solution:
    def leftmostBuildingQueries(
        self, heights: List[int], queries: List[List[int]]
    ) -> List[int]:
        n, m = len(heights), len(queries)
        for i in range(m):
            queries[i] = [min(queries[i]), max(queries[i])]
        j = n - 1
        s = sorted(set(heights))
        ans = [-1] * m
        tree = BinaryIndexedTree(n)
        for i in sorted(range(m), key=lambda i: -queries[i][1]):
            l, r = queries[i]
            while j > r:
                k = n - bisect_left(s, heights[j]) + 1
                tree.update(k, j)
                j -= 1
            if l == r or heights[l] < heights[r]:
                ans[i] = r
            else:
                k = n - bisect_left(s, heights[l])
                ans[i] = tree.query(k)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Answer the queries with a monotonic stack and binary
# search (offline, from the right)
# 2. Find the rightmost meeting building instead


# ============================================================
# FOLLOW-UP 1: Answer the queries with a monotonic stack
# and binary search (offline, from the right)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Process queries by their larger index from the
# right; keep the stack of buildings that are taller than
# everything to their right and binary search it.
# Time: O((n + q) log n).  Space: O(n + q).
def leftmost_meeting_buildings(heights, queries):
    answers = [-1] * len(queries)
    waiting = {}
```

```
    for q in range(len(queries)):
        a, b = queries[q]
        if a > b:
            a, b = b, a
        if a == b or heights[a] < heights[b]:
            answers[q] = b
        else:
            waiting.setdefault(b, []).append((heights[a], q))
    stack = []                                  # indices from the far right, heights growing toward the
bottom
    for i in range(len(heights) - 1, -1, -1):
        for need, q in waiting.get(i, []):
            low, high = 0, len(stack) - 1
            found = -1
            while low <= high:
                middle = (low + high) // 2
                if heights[stack[middle]] > need:
                    found = stack[middle]
                    low = middle + 1
                else:
                    high = middle - 1
            answers[q] = found
        while stack and heights[stack[-1]] <= heights[i]:
            stack.pop()
        stack.append(i)
    return answers


# ============================================================
# FOLLOW-UP 2: Find the rightmost meeting building instead
# ============================================================

# ---- follow-up 2: function ----
# Answer: Scan from the right end for a building taller
# than both.
# Time: O(n).  Space: O(1).
def rightmost_meeting_building(heights, a, b):
    limit = max(heights[a], heights[b])
    start = max(a, b)
    if a == b:
        return a
    for j in range(len(heights) - 1, start - 1, -1):
        if heights[j] > limit:
            return j
    if a < b and heights[a] < heights[b]:
        return b
    if b < a and heights[b] < heights[a]:
        return a
    return -1
```

PATTERN 40 · SEGMENT TREE · PROBLEM 20 OF 20

2426. Number of Pairs Satisfying Inequality

Hard · Binary Indexed Tree, Segment Tree, Array, Binary Search, Divide and Conquer, Ordered Set, Merge Sort

leetcode.com/problems/number-of-pairs-satisfying-inequality

Also appears in pattern 41 Binary Indexed Tree (Fenwick Tree)

PROBLEM

You are given two **0-indexed** integer arrays `nums1` and `nums2`, each of size `n`, and an integer `diff`. Find the number of **pairs** `(i, j)` such that:

- `0 <= i < j <= n - 1` **and**
- `nums1[i] - nums1[j] <= nums2[i] - nums2[j] + diff`.

Return *the **number of pairs** that satisfy the conditions.*

Example 1:

```
Input: nums1 = [3,2,5], nums2 = [2,2,1], diff = 1
Output: 3
Explanation:
There are 3 pairs that satisfy the conditions:
1. i = 0, j = 1: 3 - 2 <= 2 - 2 + 1. Since i < j and 1 <= 1, this pair satisfies the conditions.
2. i = 0, j = 2: 3 - 5 <= 2 - 1 + 1. Since i < j and -2 <= 2, this pair satisfies the conditions.
3. i = 1, j = 2: 2 - 5 <= 2 - 1 + 1. Since i < j and -3 <= 2, this pair satisfies the conditions.
Therefore, we return 3.
```

Example 2:

```
Input: nums1 = [3,-1], nums2 = [-2,2], diff = -1
Output: 0
Explanation:
Since there does not exist any pair that satisfies the conditions, we return 0.
```

Constraints:

- `n == nums1.length == nums2.length`
- `2 <= n <= 10^5`
- `-10^4 <= nums1[i], nums2[i] <= 10^4`
- `-10^4 <= diff <= 10^4`

SOLUTION (PYTHON 3)

Solution 1: Binary Indexed Tree

> **How to think about it**
>
> The inequality nums1[i]-nums2[i]≤ nums1[j]-nums2[j]+diff for i<j cannot be double-looped at n≤ 10^5. With v=a-b we count prior v_i≤ v_j+diff. After a shift, a Fenwick tree stores seen v. For each j from left to right, query the prefix up to v_j+diff, then insert v_j.

We can transform the inequality in the problem to nums1[i] - nums2[i] ≤ nums1[j] - nums2[j] + diff. Therefore, if we calculate the difference between the corresponding elements of the two arrays and get another array nums, the problem is transformed into finding the number of pairs in nums that satisfy nums[i] ≤ nums[j] + diff.

We can enumerate j from small to large, find out how many numbers before it satisfy nums[i] ≤ nums[j] + diff, and thus calculate the number of pairs. We can use a binary indexed tree to maintain the prefix sum, so we can find out how many numbers before it satisfy nums[i] ≤ nums[j] + diff in O(log n) time.

The time complexity is O(n × log n).

Python 3

```python
class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x

    def update(self, x, delta):
        while x <= self.n:
            self.c[x] += delta
            x += BinaryIndexedTree.lowbit(x)

    def query(self, x):
        s = 0
        while x:
            s += self.c[x]
            x -= BinaryIndexedTree.lowbit(x)
        return s


class Solution:
    def numberOfPairs(self, nums1: List[int], nums2: List[int], diff: int) -> int:
        tree = BinaryIndexedTree(10**5)
        ans = 0
        for a, b in zip(nums1, nums2):
            v = a - b
            ans += tree.query(v + diff + 40000)
            tree.update(v + 40000, 1)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Merge sort version
# 2. Strict inequality: nums1[i] - nums1[j] < nums2[i] -
# nums2[j] + diff


# ============================================================
# FOLLOW-UP 1: Merge sort version
# ============================================================

# ---- follow-up 1: function ----
# Answer: Let d[i] = nums1[i] - nums2[i]; count pairs i <
# j with d[i] <= d[j] + diff while merge sorting d.
# Time: O(n log n).  Space: O(n).
def count_pairs_merge_sort(nums1, nums2, diff):
    d = [a - b for a, b in zip(nums1, nums2)]

    def sort(values):
        if len(values) <= 1:
            return values, 0
        middle = len(values) // 2
        left, count_left = sort(values[:middle])
        right, count_right = sort(values[middle:])
        count = count_left + count_right
        j = 0
        for x in left:
            while j < len(right) and right[j] + diff < x:
                j += 1
            count += len(right) - j
        merged = []
        i = j = 0
```

```
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                merged.append(left[i])
                i += 1
            else:
                merged.append(right[j])
                j += 1
        merged += left[i:] + right[j:]
        return merged, count

    return sort(d)[1]


# ============================================================
# FOLLOW-UP 2: Strict inequality: nums1[i] - nums1[j] <
# nums2[i] - nums2[j] + diff
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the earlier differences in a sorted list
# and count with binary search (bisect_left gives strictly
# smaller).
# Time: O(n^2) (insort shifts the list).  Space: O(n).
import bisect


def count_pairs_strict(nums1, nums2, diff):
    earlier = []
    count = 0
```

```
    for a, b in zip(nums1, nums2):
        d = a - b
        count += bisect.bisect_left(earlier, d + diff)
        bisect.insort(earlier, d)
    return count
```

PATTERN 41 OF 41

Binary Indexed Tree (Fenwick Tree)

20 problems · 0 Easy · 10 Medium · 10 Hard

THE IDEA

A compact array that supports prefix sums and point updates in O(log n); the lowest set bit of an index says how many items it covers.

Signals: range sum query with updates, count of smaller numbers after self, reverse pairs, inversion counts.

A TEMPLATE TO KEEP IN YOUR HANDS (PYTHON)

```
tree = [0] * (n + 1)
def update(i, delta):                # 1-based index
    while i <= n:
        tree[i] += delta
        i += i & -i                  # jump to the next range that covers i
def prefix_sum(i):
    total = 0
    while i > 0:
        total += tree[i]
        i -= i & -i                  # strip the lowest set bit
    return total
```

THE 20 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	★ **Range Sum Query - Mutable** (see page)	Medium	LC 307	274
2	Range Sum Query 2D - Mutable — substitute: Range Sum Query 2D - Immutable (see page)	Medium	LC 304	248
3	★ **Create Sorted Array through Instructions** (see page)	Medium	LC 1649	2006
4	Queue Reconstruction by Height (see page)	Medium	LC 406	1164
5	Count Number of Teams	Medium	LC 1395	2043
6	Longest Increasing Subsequence (see page)	Medium	LC 300	686
7	Number of Longest Increasing Subsequence (see page)	Medium	LC 673	712
8	The Number of Weak Characters in the Game	Medium	LC 1996	2047
9	Best Team With No Conflicts	Medium	LC 1626	2049
10	Range Frequency Queries (see page)	Medium	LC 2080	2011
11	★ **Count of Smaller Numbers After Self** (see page)	Hard	LC 315	753
12	★ **Count of Range Sum** (see page)	Hard	LC 327	776
13	★ **Reverse Pairs** (see page)	Hard	LC 493	759
14	★ **Russian Doll Envelopes**	Hard	LC 354	2052
15	Number of Pairs Satisfying Inequality (see page)	Hard	LC 2426	2039
16	Make Array Empty	Hard	LC 2659	2054
17	★ **Count Good Triplets in an Array**	Hard	LC 2179	2058
18	Maximum Balanced Subsequence Sum	Hard	LC 2926	2064
19	Sum of Floored Pairs	Hard	LC 1862	2068
20	Peaks in Array	Hard	LC 3187	2071

1395. Count Number of Teams

Medium · Binary Indexed Tree, Segment Tree, Array, Dynamic Programming

leetcode.com/problems/count-number-of-teams

PROBLEM

There are `n` soldiers standing in a line. Each soldier is assigned a **unique** `rating` value.

You have to form a team of 3 soldiers amongst them under the following rules:

- Choose 3 soldiers with index (`i`, `j`, `k`) with rating (`rating[i]`, `rating[j]`, `rating[k]`).
- A team is valid if: (`rating[i] < rating[j] < rating[k]`) or (`rating[i] > rating[j] > rating[k]`) where (`0 <= i < j < k < n`).

Return the number of teams you can form given the conditions. (soldiers can be part of multiple teams).

Example 1:

```
Input: rating = [2,5,3,4,1]
Output: 3
Explanation: We can form three teams given the conditions. (2,3,4), (5,4,1), (5,3,1).
```

Example 2:

```
Input: rating = [2,1,3]
Output: 0
Explanation: We can't form any team given the conditions.
```

Example 3:

```
Input: rating = [1,2,3,4]
Output: 4
```

Constraints:

- `n == rating.length`
- `3 <= n <= 1000`
- `1 <= rating[i] <= 10^5`
- All the integers in `rating` are **unique**.

SOLUTIONS (PYTHON 3)

Solution 1: Enumerate Middle Element

How to think about it

Count strictly increasing or decreasing triples. n ≤ 1000 makes a triple loop cubic. Fix the middle rating b: l smaller values on the left times r larger on the right are the increasing teams; the complementary counts are the decreasing ones. Two linear scans per midpoint suffice.

We can enumerate each element rating[i] in the array rating as the middle element, then count the number of elements l that are smaller than it on the left, and the number of elements r that are larger than it on the right. The number of combat units with this element as the middle element is l × r + (i - l) × (n - i - 1 - r). We can add this to the answer.

The time complexity is $O(n^2)$, and the space complexity is O(1). Where n is the length of the array rating.

Python 3

```python
class Solution:
    def numTeams(self, rating: List[int]) -> int:
        ans, n = 0, len(rating)
        for i, b in enumerate(rating):
            l = sum(a < b for a in rating[:i])
            r = sum(c > b for c in rating[i + 1 :])
            ans += l * r
            ans += (i - l) * (n - i - 1 - r)
        return ans
```

Solution 2: Binary Indexed Tree

> **How to think about it**
>
> Scanning both sides per midpoint is $O(n^2)$. After discretizing, two Fenwick trees query "how many smaller on the left / larger on the right" in O(log n): every value starts in the right tree and moves left as i advances.

We can use two binary indexed trees to maintain the number of elements l that are smaller than each element on the left in the array rating, and the number of elements r that are larger than it on the right. Then count the number of combat units with this element as the middle element as l × r + (i - l) × (n - i - 1 - r), and add this to the answer.

The time complexity is O(n × log n), and the space complexity is O(n). Where n is the length of the array rating.

Python 3

```python
class BinaryIndexedTree:
    def __init__(self, n: int):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x: int, v: int):
        while x <= self.n:
            self.c[x] += v
            x += x & -x

    def query(self, x: int) -> int:
        s = 0
        while x:
            s += self.c[x]
            x -= x & -x
        return s


class Solution:
    def numTeams(self, rating: List[int]) -> int:
        nums = sorted(set(rating))
        m = len(nums)
        tree1 = BinaryIndexedTree(m)
        tree2 = BinaryIndexedTree(m)
        for v in rating:
            x = bisect_left(nums, v) + 1
            tree2.update(x, 1)
        n = len(rating)
        ans = 0
        for i, v in enumerate(rating):
            x = bisect_left(nums, v) + 1
            tree1.update(x, 1)
            tree2.update(x, -1)
            l = tree1.query(x - 1)
            r = n - i - 1 - tree2.query(x)
            ans += l * r
            ans += (i - l) * (n - i - 1 - r)
        return ans
```

Solution 3: Recursion + Memoization

How to think about it

Fenwick trees need discretization. Memoizing by index, remaining picks, and direction counts increasing or decreasing completions: from i we try every later j that respects the order. O(n) states and O(n) transitions match the first method's bound, expressed as recursion.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```python
from functools import cache


class Solution:
    def numTeams(self, rating: List[int]) -> int:
        n = len(rating)

        @cache
        def go(i: int, left: int, ascending: bool) -> int:
            # chains of `left` more soldiers after soldier i, all going up (or all going down)
            if left == 0:
                return 1
            total = 0
            for j in range(i + 1, n):
                if (rating[j] > rating[i]) == ascending:
                    total += go(j, left - 1, ascending)
            return total

        return sum(go(i, 2, True) + go(i, 2, False) for i in range(n))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Teams of size k (each strictly increasing or strictly
# decreasing)
# 2. Count the teams of three with a Fenwick tree (smaller
# on the left times larger on the right)


# ============================================================
# FOLLOW-UP 1: Teams of size k (each strictly increasing
# or strictly decreasing)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Table: ways[length][i] counts the increasing
# chains of that length ending at i; do the same for
# decreasing and add both.
# Time: O(k * n^2).  Space: O(k * n).
def count_teams_of_size(rating, k):
    n = len(rating)

    def chains(better):
        ways = [[0] * n for _ in range(k + 1)]
        for i in range(n):
            ways[1][i] = 1
        for length in range(2, k + 1):
            for i in range(n):
                for j in range(i):
                    if better(rating[j], rating[i]):
                        ways[length][i] += ways[length - 1][j]
        return sum(ways[k])

    if k == 1:
        return n
    return chains(lambda a, b: a < b) + chains(lambda a, b: a > b)
```

```
# ============================================================
# FOLLOW-UP 2: Count the teams of three with a Fenwick
# tree (smaller on the left times larger on the right)
# ============================================================

# ---- follow-up 2: function ----
# Answer: For every middle soldier, count the smaller and
# larger soldiers on both sides with a Fenwick tree over
# the ranks.
# Time: O(n log n).  Space: O(n).
class Fenwick:
    def __init__(self, size):
        self.tree = [0] * (size + 1)

    def add(self, position, amount=1):
        while position < len(self.tree):
            self.tree[position] += amount
            position += position & -position

    def prefix(self, position):
        total = 0
        while position > 0:
            total += self.tree[position]
            position -= position & -position
```

```
        return total


def count_teams_fenwick(rating):
    ranks = {v: i + 1 for i, v in enumerate(sorted(set(rating)))}
    m = len(ranks)
    left = Fenwick(m)
    right = Fenwick(m)
    for x in rating:
        right.add(ranks[x])
    total = 0
    for x in rating:
        r = ranks[x]
        right.add(r, -1)
        smaller_left = left.prefix(r - 1)
        larger_left = left.prefix(m) - left.prefix(r)
        smaller_right = right.prefix(r - 1)
        larger_right = right.prefix(m) - right.prefix(r)
        total += smaller_left * larger_right + larger_left * smaller_right
        left.add(r)
    return total
```

PATTERN 41 · BINARY INDEXED TREE (FENWICK TREE) · PROBLEM 8 OF 20

1996. The Number of Weak Characters in the Game

Medium · Stack, Greedy, Array, Sorting, Monotonic Stack

leetcode.com/problems/the-number-of-weak-characters-in-the-game

PROBLEM

You are playing a game that contains multiple characters, and each of the characters has **two** main properties: **attack** and **defense**. You are given a 2D integer array `properties` where `properties[i] = [attack_i, defense_i]` represents the properties of the `i^th` character in the game.

A character is said to be **weak** if any other character has **both** attack and defense levels **strictly greater** than this character's attack and defense levels. More formally, a character `i` is said to be **weak** if there exists another character `j` where `attack_j > attack_i` and `defense_j > defense_i`.

Return *the number of **weak** characters*.

Example 1:

```
Input: properties = [[5,5],[6,3],[3,6]]
Output: 0
Explanation: No character has strictly greater attack and defense than the other.
```

Example 2:

```
Input: properties = [[2,2],[3,3]]
Output: 1
Explanation: The first character is weak because the second character has a strictly greater attack and defense.
```

Example 3:

```
Input: properties = [[1,5],[10,4],[4,3]]
Output: 1
Explanation: The third character is weak because the second character has a strictly greater attack and defense.
```

Constraints:

- `2 <= properties.length <= 10^5`
- `properties[i].length == 2`
- `1 <= attack_i, defense_i <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: Sorting + Traversal

> **How to think about it**
>
> A character is weak if another beats it in both attack and defense. Pairwise checks are quadratic. Sort by attack descending and defense ascending so earlier characters have strictly larger attack, or equal attack that cannot be a strict win. Track the maximum defense seen; a smaller current defense is weak.

We can sort all characters in descending order of attack power and ascending order of defense power.

Then, traverse all characters. For the current character, if its defense power is less than the previous maximum defense power, it is a weak character, and we increment the answer by one. Otherwise, update the maximum defense power.

After the traversal, we get the answer.

The time complexity is O(n × log n), and the space complexity is O(log n). Here, n is the number of characters.

Python 3

```
class Solution:
    def numberOfWeakCharacters(self, properties: List[List[int]]) -> int:
        properties.sort(key=lambda x: (-x[0], x[1]))
        ans = mx = 0
        for _, x in properties:
            ans += x < mx
            mx = max(mx, x)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return which characters are weak
# 2. Count the strong characters (no character is stronger
# in both ways, but one is weaker in both)


# ========================================================
# FOLLOW-UP 1: Return which characters are weak
# ========================================================

# ---- follow-up 1: function ----
# Answer: Sort by attack descending (defence ascending for
# ties) and keep the biggest defence seen; a character
# with a smaller defence is weak.
# Time: O(n log n).  Space: O(n).
def weak_character_indices(properties):
    order = sorted(range(len(properties)), key=lambda i: (-properties[i][0], properties[i][1]))
    biggest_defence = 0
    weak = []
    for i in order:
        if properties[i][1] < biggest_defence:
            weak.append(i)
        biggest_defence = max(biggest_defence, properties[i][1])
    return sorted(weak)


# ========================================================
# FOLLOW-UP 2: Count the strong characters (no character
# is stronger in both ways, but one is weaker in both)
# ========================================================

# ---- follow-up 2: function ----
# Answer: Mirror image: a character is strong when some
# other character has both a lower attack and a lower
# defence.
# Time: O(n log n).  Space: O(n).
def strong_character_count(properties):
    order = sorted(range(len(properties)), key=lambda i: (properties[i][0], -properties[i][1]))
    smallest_defence = float("inf")
    count = 0
    for i in order:
        if properties[i][1] > smallest_defence:
            count += 1
        smallest_defence = min(smallest_defence, properties[i][1])
    return count
```

PATTERN 41 · BINARY INDEXED TREE (FENWICK TREE) · PROBLEM 9 OF 20

1626. Best Team With No Conflicts

Medium · Array, Dynamic Programming, Sorting, Longest Increasing Subsequence

leetcode.com/problems/best-team-with-no-conflicts

PROBLEM

You are the manager of a basketball team. For the upcoming tournament, you want to choose the team with the highest overall score. The score of the team is the **sum** of scores of all the players in the team.

However, the basketball team is not allowed to have **conflicts**. A **conflict** exists if a younger player has a **strictly higher** score than an older player. A conflict does **not** occur between players of the same age.

Given two lists, `scores` and `ages`, where each `scores[i]` and `ages[i]` represents the score and age of the i^{th} player, respectively, return *the highest overall score of all possible basketball teams*.

Example 1:

```
Input: scores = [1,3,5,10,15], ages = [1,2,3,4,5]
Output: 34
Explanation: You can choose all the players.
```

Example 2:

```
Input: scores = [4,5,6,5], ages = [2,1,2,1]
Output: 16
Explanation: It is best to choose the last 3 players. Notice that you are allowed to choose multiple people of the same age.
```

Example 3:

```
Input: scores = [1,2,3,5], ages = [8,9,10,1]
Output: 6
Explanation: It is best to choose the first 3 players.
```

Constraints:

- `1 <= scores.length, ages.length <= 1000`
- `scores.length == ages.length`
- `1 <= scores[i] <= 10^6`
- `1 <= ages[i] <= 1000`

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

A conflict is an older player with a strictly lower score. Sorting by score, then age, reduces the constraint to a nondecreasing age subsequence. With n ≤ 1000, the best score ending at a player is an $O(n^2)$ LIS-style DP: take the best f[j] among earlier players who are no older, then add the current score. The answer is the maximum f[i].

Python 3

```python
class Solution:
    def bestTeamScore(self, scores: List[int], ages: List[int]) -> int:
        arr = sorted(zip(scores, ages))
        n = len(arr)
        f = [0] * n
        for i, (score, age) in enumerate(arr):
            for j in range(i):
                if age >= arr[j][1]:
                    f[i] = max(f[i], f[j])
            f[i] += score
        return max(f)
```

Solution 2

How to think about it

Solution 1 scans every predecessor in $O(n^2)$. After the same sort we only need the best score among ages up to the current one — a prefix maximum. A Fenwick tree stores those maxima: query [1,age], add score, and update that age, for O(nlog m) time.

Python 3

```python
class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x, val):
        while x <= self.n:
            self.c[x] = max(self.c[x], val)
            x += x & -x

    def query(self, x):
        s = 0
        while x:
            s = max(s, self.c[x])
            x -= x & -x
        return s


class Solution:
    def bestTeamScore(self, scores: List[int], ages: List[int]) -> int:
        m = max(ages)
        tree = BinaryIndexedTree(m)
        for score, age in sorted(zip(scores, ages)):
            tree.update(age, score + tree.query(age))
        return tree.query(m)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the players of the best team
# 2. The team may have at most m players


# ============================================================
# FOLLOW-UP 1: Return the players of the best team
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same table of best scores, with the previous
# player remembered.
# Time: O(n^2).  Space: O(n).
```

```
def best_team_players(scores, ages):
    players = sorted(zip(ages, scores))
    n = len(players)
    best = [players[i][1] for i in range(n)]
    previous = [-1] * n
    for i in range(n):
        for j in range(i):
            if players[j][1] <= players[i][1] and best[j] + players[i][1] > best[i]:
                best[i] = best[j] + players[i][1]
                previous[i] = j
    end = max(range(n), key=lambda i: best[i])
    team = []
    while end != -1:
        team.append(players[end])
        end = previous[end]
    return team[::-1]


# ============================================================
# FOLLOW-UP 2: The team may have at most m players
# ============================================================

# ---- follow-up 2: function ----
# Answer: Add the team size to the table.
# Time: O(n^2 * m).  Space: O(n * m).
def best_team_with_limit(scores, ages, m):
    players = sorted(zip(ages, scores))
    n = len(players)
    best = [[float("-inf")] * (m + 1) for _ in range(n)]
    answer = 0
    for i in range(n):
        best[i][1] = players[i][1]
        for j in range(i):
            if players[j][1] <= players[i][1]:
                for size in range(1, m):
                    if best[j][size] != float("-inf"):
                        best[i][size + 1] = max(best[i][size + 1], best[j][size] + players[i][1])
        answer = max(answer, max(best[i]))
    return answer
```

PATTERN 41 · BINARY INDEXED TREE (FENWICK TREE) · PROBLEM 14 OF 20

354. Russian Doll Envelopes

Hard · Array, Binary Search, Dynamic Programming, Sorting, Longest Increasing Subsequence

leetcode.com/problems/russian-doll-envelopes

★ **TOP 7 PROBLEM** in Binary Indexed Tree (Fenwick Tree): master this one first

PROBLEM

You are given a 2D array of integers `envelopes` where `envelopes[i] = [w_i, h_i]` represents the width and the height of an envelope.

One envelope can fit into another if and only if both the width and height of one envelope are greater than the other envelope's width and height.

Return *the maximum number of envelopes you can Russian doll (i.e., put one inside the other)*.

Note: You cannot rotate an envelope.

Example 1:

```
Input: envelopes = [[5,4],[6,4],[6,7],[2,3]]
Output: 3
Explanation: The maximum number of envelopes you can Russian doll is 3 ([2,3] => [5,4] => [6,7]).
```

Example 2:

```
Input: envelopes = [[1,1],[1,1],[1,1]]
Output: 1
```

Constraints:

- `1 <= envelopes.length <= 10^5`
- `envelopes[i].length == 2`
- `1 <= w_i, h_i <= 10^5`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

An envelope nests only if both width and height increase; we want the longest chain. A 2D $O(n^2)$ LIS fails for $n \le 10^5$. After sorting by width, LIS on height is the answer; equal widths must not nest. Sort by width ascending and height descending, then greedy LIS on heights: append if larger than the tail, else binary-replace. Descending heights keep at most one envelope per width.

Python 3

```python
class Solution:
    def maxEnvelopes(self, envelopes: List[List[int]]) -> int:
        envelopes.sort(key=lambda x: (x[0], -x[1]))
        d = [envelopes[0][1]]
        for _, h in envelopes[1:]:
            if h > d[-1]:
                d.append(h)
            else:
                idx = bisect_left(d, h)
                d[idx] = h
        return len(d)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the chain of envelopes
# 2. Boxes with three sizes


# ============================================================
# FOLLOW-UP 1: Return the chain of envelopes
# ============================================================

# ---- follow-up 1: function ----
# Answer: Table of the longest chain ending at each
# envelope, with the previous one remembered.
# Time: O(n^2).  Space: O(n).
def envelope_chain(envelopes):
    items = sorted(envelopes, key=lambda e: (e[0], -e[1]))
    n = len(items)
    length = [1] * n
    previous = [-1] * n
    for i in range(n):
        for j in range(i):
            if items[j][0] < items[i][0] and items[j][1] < items[i][1] and length[j] + 1 > length[i]:
                length[i] = length[j] + 1
                previous[i] = j
    end = max(range(n), key=lambda i: length[i])
    chain = []
    while end != -1:
        chain.append(items[end])
        end = previous[end]
    return chain[::-1]


# ============================================================
# FOLLOW-UP 2: Boxes with three sizes
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same table; a box fits in another one when
# all three sizes are strictly smaller.
# Time: O(n^2).  Space: O(n).
def most_nested_boxes(boxes):
    items = sorted(boxes)
    n = len(items)
    length = [1] * n
    for i in range(n):
        for j in range(i):
            if all(items[j][d] < items[i][d] for d in range(3)):
                length[i] = max(length[i], length[j] + 1)
    return max(length)
```

2659. Make Array Empty

Hard · Greedy, Binary Indexed Tree, Segment Tree, Array, Binary Search, Ordered Set, Sorting

leetcode.com/problems/make-array-empty

PROBLEM

You are given an integer array `nums` containing **distinct** numbers, and you can perform the following operations **until the array is empty**:

- If the first element has the **smallest** value, remove it
- Otherwise, put the first element at the **end** of the array.

Return *an integer denoting the number of operations it takes to make* `nums` *empty.*

Example 1:

```
Input: nums = [3,4,-1]
Output: 5
```

Operation	Array
1	[4, -1, 3]
2	[-1, 3, 4]
3	[3, 4]
4	[4]
5	[]

Example 2:

```
Input: nums = [1,2,4,3]
Output: 5
```

Operation	Array
1	[2, 4, 3]
2	[4, 3]
3	[3, 4]
4	[4]
5	[]

Example 3:

```
Input: nums = [1,2,3]
Output: 3
```

Operation	Array
1	[2, 3]
2	[3]
3	[]

Constraints:

- $1 <= nums.length <= 10^5$
- $-10^9 <= nums[i] <= 10^9$
- All values in `nums` are **distinct**.

SOLUTIONS (PYTHON 3)

Solution 1: Hash Table + Sorting + Fenwick Tree

> **How to think about it**
>
> Each step rotates left or deletes the current minimum. Simulating rotations fails for $n \le 10^5$. Deletions follow sorted order; the cost is the circular gap between consecutive minima minus already-deleted indices in between. Store original positions, sort the values, and keep deleted indices in an ordered list. The gap j-i drops the deleted count between them; if j wraps before i, add the remaining length of this round.

First, we use a hash table pos to record the position of each element in array nums. Then, we sort array nums. The initial answer is the position of the minimum element in array nums plus 1, which is ans = pos[nums[0]] + 1.

Next, we traverse the sorted array nums, the indexes of the two adjacent elements a and b are i = pos[a], j = pos[b]. The number of operations needed to move the second element b to the first position of the array and delete it is equal to the interval between the two indexes, minus the number of indexes deleted between the two indexes, and add the number of operations to the answer. We can use a Fenwick tree or an ordered list to maintain the deleted indexes between two indexes, so that we can find the number of deleted indexes between two indexes in O(log n) time. Note that if i > j, then we need to increase n - k operations, where k is the current position.

After the traversal is over, return the number of operations ans.

The time complexity is O(n × log n), and the space complexity is O(n). Where n is the length of array nums.

Python 3

```python
class Solution:
    def countOperationsToEmptyArray(self, nums: List[int]) -> int:
        pos = {x: i for i, x in enumerate(nums)}
        nums.sort()
        sl = SortedList()
        ans = pos[nums[0]] + 1
        n = len(nums)
        for k, (a, b) in enumerate(pairwise(nums)):
            i, j = pos[a], pos[b]
            d = j - i - sl.bisect(j) + sl.bisect(i)
            ans += d + (n - k) * int(i > j)
            sl.add(i)
        return ans
```

Solution 2

> **How to think about it**
>
> Solution 1 queries deleted counts in an ordered set. A Fenwick tree offers the same prefix counts: after shifting indices by one, `query(j+1)-query(i+1)` is the deleted amount, and an update is a point add. The wrap-around logic is unchanged.

Python 3

```
class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x, delta):
        while x <= self.n:
            self.c[x] += delta
            x += x & -x

    def query(self, x):
        s = 0
        while x:
            s += self.c[x]
            x -= x & -x
        return s


class Solution:
    def countOperationsToEmptyArray(self, nums: List[int]) -> int:
        pos = {x: i for i, x in enumerate(nums)}
        nums.sort()
        ans = pos[nums[0]] + 1
        n = len(nums)
        tree = BinaryIndexedTree(n)
        for k, (a, b) in enumerate(pairwise(nums)):
            i, j = pos[a], pos[b]
            d = j - i - tree.query(j + 1) + tree.query(i + 1)
            ans += d + (n - k) * int(i > j)
            tree.update(i + 1, 1)
        return ans
```

Complexity. Time: O(n log n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Count the operations with a Fenwick tree (no
# simulation)
# 2. Remove the LARGEST number instead (rotate the front
# to the back until it is the largest)


# ============================================================
# FOLLOW-UP 1: Count the operations with a Fenwick tree
# (no simulation)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remove the numbers from the smallest; moving
# from one removed position to the next costs one move for
# every number still present in between (going around the
# end if needed), counted with a Fenwick tree of the
# removed positions.
# Time: O(n log n).  Space: O(n).
def operations_with_fenwick(nums):
    n = len(nums)
    tree = [0] * (n + 1)

    def add(position):
        position += 1
        while position <= n:
            tree[position] += 1
            position += position & -position

    def removed_up_to(position):
```

```
        position += 1
        total = 0
        while position > 0:
            total += tree[position]
            position -= position & -position
        return total

    def removed_between(low, high):
        if low > high:
            return 0
        return removed_up_to(high) - removed_up_to(low - 1)

    operations = n                                  # every number is removed once
    current = -1                                    # index of the last removed number
    for index in sorted(range(n), key=lambda i: nums[i]):
        if index > current:
            operations += (index - current - 1) - removed_between(current + 1, index - 1)
        else:
            operations += (n - 1 - current) - removed_between(current + 1, n - 1)
            operations += index - removed_between(0, index - 1)
        add(index)
        current = index
    return operations


# ============================================================
# FOLLOW-UP 2: Remove the LARGEST number instead (rotate
# the front to the back until it is the largest)
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Simulate with the same counting, going through
# the numbers from the largest.
# Time: O(n^2).  Space: O(n).
def operations_for_largest_first(nums):
    items = list(nums)
    operations = 0
    while items:
        biggest = max(items)
        if items[0] == biggest:
            items.pop(0)
        else:
            items.append(items.pop(0))
        operations += 1
    return operations
```

2179. Count Good Triplets in an Array

Hard · Binary Indexed Tree, Segment Tree, Array, Binary Search, Divide and Conquer, Ordered Set, Merge Sort

leetcode.com/problems/count-good-triplets-in-an-array

★ **TOP 7 PROBLEM** in Binary Indexed Tree (Fenwick Tree): master this one first

PROBLEM

You are given two **0-indexed** arrays `nums1` and `nums2` of length `n`, both of which are **permutations** of `[0, 1, ..., n - 1]`.

A **good triplet** is a set of `3` **distinct** values which are present in **increasing order** by position both in `nums1` and `nums2`. In other words, if we consider $pos1_v$ as the index of the value `v` in `nums1` and $pos2_v$ as the index of the value `v` in `nums2`, then a good triplet will be a set `(x, y, z)` where `0 <= x, y, z <= n - 1`, such that $pos1_x < pos1_y < pos1_z$ and $pos2_x < pos2_y < pos2_z$.

Return *the **total number** of good triplets*.

Example 1:

```
Input: nums1 = [2,0,1,3], nums2 = [0,1,2,3]
Output: 1
Explanation:
There are 4 triplets (x,y,z) such that pos1x < pos1y < pos1z. They are (2,0,1), (2,0,3), (2,1,3), and (0,1,3).
Out of those triplets, only the triplet (0,1,3) satisfies pos2x < pos2y < pos2z. Hence, there is only 1 good triplet.
```

Example 2:

```
Input: nums1 = [4,0,1,3,2], nums2 = [4,1,0,2,3]
Output: 4
Explanation: The 4 good triplets are (4,0,3), (4,0,2), (4,1,3), and (4,1,2).
```

Constraints:

- `n == nums1.length == nums2.length`
- `3 <= n <= 10^5`
- `0 <= nums1[i], nums2[i] <= n - 1`
- `nums1` and `nums2` are permutations of `[0, 1, ..., n - 1]`.

SOLUTIONS (PYTHON 3)

Solution 1: Binary Indexed Tree (Fenwick Tree)

How to think about it

A good triplet has the same relative order in both permutations. Enumerating triples is $O(n^3)$ and fails for $n \le 10^5$. With the middle value fixed, the left count is how many already seen in nums1 sit earlier in nums2; the right count is the symmetric unseen suffix. Walk nums1, query a Fenwick tree at the nums2 position for the prefix count and the unused suffix, and add the product. Positions are 1-based; update after each value.

For this problem, we first use `pos` to record the position of each number in `nums2`, and then process each element in `nums1` sequentially.

Consider the number of good triplets **with the current number as the middle number**. The first number must have already been traversed and must appear earlier than the current number in `nums2`. The third number must not yet have been traversed and must appear later than the current number in `nums2`.

Take `nums1 = [4,0,1,3,2]` and `nums2 = [4,1,0,2,3]` as an example. Consider the traversal process:

1. First, process `4`. At this point, the state of `nums2` is `[4,X,X,X,X]`. The number of values before `4` is `0`, and the number of values after `4` is `4`. Therefore, `4` as the middle number forms `0` good triplets.
2. Next, process `0`. The state of `nums2` becomes `[4,X,0,X,X]`. The number of values before `0` is `1`, and the number of values after `0` is `2`. Therefore, `0` as the middle number forms `2` good triplets.
3. Next, process `1`. The state of `nums2` becomes `[4,1,0,X,X]`. The number of values before `1` is `1`, and the number of values after `1` is `2`. Therefore, `1` as the middle number forms `2` good triplets.
4. ...
5. Finally, process `2`. The state of `nums2` becomes `[4,1,0,2,3]`. The number of values before `2` is `4`, and the number of values after `2` is `0`. Therefore, `2` as the middle number forms `0` good triplets.

We can use a **Binary Indexed Tree (Fenwick Tree)** to update the occurrence of numbers at each position in `nums2`, and quickly calculate the number of `1`s to the left of each number and the number of `0`s to the right of each number.

A Binary Indexed Tree, also known as a Fenwick Tree, efficiently supports the following operations:

1. **Point Update** `update(x, delta)`: Add a value `delta` to the number at position `x` in the sequence.
2. **Prefix Sum Query** `query(x)`: Query the sum of the sequence in the range `[1, ..., x]`, i.e., the prefix sum at position `x`.

Both operations have a time complexity of O(log n). Therefore, the overall time complexity is O(n log n), where n is the length of the array nums1. The space complexity is O(n).

Python 3

```python
class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x

    def update(self, x, delta):
        while x <= self.n:
            self.c[x] += delta
            x += BinaryIndexedTree.lowbit(x)

    def query(self, x):
        s = 0
        while x > 0:
            s += self.c[x]
            x -= BinaryIndexedTree.lowbit(x)
        return s


class Solution:
    def goodTriplets(self, nums1: List[int], nums2: List[int]) -> int:
        pos = {v: i for i, v in enumerate(nums2, 1)}
        ans = 0
        n = len(nums1)
        tree = BinaryIndexedTree(n)
        for num in nums1:
            p = pos[num]
            left = tree.query(p)
            right = n - p - (tree.query(n) - tree.query(p))
            ans += left * right
            tree.update(p, 1)
        return ans
```

Solution 2: Segment Tree

> **How to think about it**
>
> Solution 1 stores prefix insertion counts in a Fenwick tree. A segment tree with point adds and range sums supports the same queries at the same asymptotic cost. Each node stores how many values in its interval have appeared; query [1,p] and the unused count of (p,n], add the product, then increment that point. The write-up records the segment-tree implementation.

We can also use a segment tree to solve this problem. A segment tree is a data structure that efficiently supports range queries and updates. The basic idea is to divide an interval into multiple subintervals, with each subinterval represented by a node.

The segment tree divides the entire interval into multiple non-overlapping subintervals, with the number of subintervals not exceeding `log(width)`. To update the value of an element, we only need to update `log(width)` intervals, all of which are contained within a larger interval that includes the element.

- Each node of the segment tree represents an interval.
- The segment tree has a unique root node, representing the entire range, such as `[1, N]`.
- Each leaf node of the segment tree represents a unit interval `[x, x]`.
- For each internal node `[l, r]`, its left child represents `[l, mid]`, and its right child represents `[mid + 1, r]`, where `mid = ⌊(l + r) / 2⌋` (floor division).

The time complexity is O(n log n), where n is the length of the array nums1. The space complexity is O(n).

Python 3

```python
class Node:
    __slots__ = ("l", "r", "v")

    def __init__(self):
        self.l = 0
        self.r = 0
        self.v = 0


class SegmentTree:
    def __init__(self, n):
        self.tr = [Node() for _ in range(4 * n)]
        self.build(1, 1, n)

    def build(self, u, l, r):
        self.tr[u].l = l
        self.tr[u].r = r
        if l == r:
            return
        mid = (l + r) >> 1
        self.build(u << 1, l, mid)
        self.build(u << 1 | 1, mid + 1, r)

    def modify(self, u, x, v):
        if self.tr[u].l == x and self.tr[u].r == x:
            self.tr[u].v += v
            return
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        if x <= mid:
            self.modify(u << 1, x, v)
        else:
            self.modify(u << 1 | 1, x, v)
        self.pushup(u)

    def pushup(self, u):
        self.tr[u].v = self.tr[u << 1].v + self.tr[u << 1 | 1].v

    def query(self, u, l, r):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            return self.tr[u].v
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        v = 0
        if l <= mid:
            v += self.query(u << 1, l, r)
        if r > mid:
            v += self.query(u << 1 | 1, l, r)
        return v


class Solution:
    def goodTriplets(self, nums1: List[int], nums2: List[int]) -> int:
        pos = {v: i for i, v in enumerate(nums2, 1)}
        ans = 0
        n = len(nums1)
        tree = SegmentTree(n)
        for num in nums1:
            p = pos[num]
            left = tree.query(1, 1, p)
            right = n - p - (tree.query(1, 1, n) - tree.query(1, 1, p))
            ans += left * right
            tree.modify(1, p, 1)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Count the pairs that appear in the same order in both
# permutations
# 2. Triplets that are in the same order in nums1 but in
# REVERSED order in nums2


# ============================================================
# FOLLOW-UP 1: Count the pairs that appear in the same
# order in both permutations
# ============================================================

# ---- follow-up 1: function ----
# Answer: Map every value to its position in the second
# array and count the increasing pairs of those positions
# with a Fenwick tree.
# Time: O(n log n).  Space: O(n).
def count_pairs_same_order(nums1, nums2):
    n = len(nums1)
    where = {v: i + 1 for i, v in enumerate(nums2)}
    tree = [0] * (n + 1)
    total = 0
    for v in nums1:
        position = where[v]
        i = position - 1
        while i > 0:
            total += tree[i]
            i -= i & -i
        i = position
        while i <= n:
            tree[i] += 1
            i += i & -i
    return total


# ============================================================
# FOLLOW-UP 2: Triplets that are in the same order in
# nums1 but in REVERSED order in nums2
# ============================================================

# ---- follow-up 2: function ----
# Answer: Reverse the second permutation and count normal
# good triplets.
# Time: O(n log n).  Space: O(n).
def count_triplets_same_order(nums1, nums2):
    n = len(nums1)
    where = {v: i + 1 for i, v in enumerate(nums2)}

    class Tree:
        def __init__(self):
            self.t = [0] * (n + 1)

        def add(self, i):
            while i <= n:
                self.t[i] += 1
                i += i & -i

        def prefix(self, i):
            s = 0
            while i > 0:
```

```
                s += self.t[i]
                i -= i & -i
            return s
```

```
    tree = Tree()
    total = 0
    positions = [where[v] for v in nums1]
    for i in range(n):
        p = positions[i]
        before = tree.prefix(p - 1)
        after = (n - p) - (tree.prefix(n) - tree.prefix(p))
        total += before * after
        tree.add(p)
    return total


def count_triplets_reversed_in_second(nums1, nums2):
    return count_triplets_same_order(nums1, nums2[::-1])
```

2926. Maximum Balanced Subsequence Sum

Hard · Binary Indexed Tree, Segment Tree, Array, Binary Search, Dynamic Programming

leetcode.com/problems/maximum-balanced-subsequence-sum

PROBLEM

You are given a **0-indexed** integer array `nums`.

A **subsequence** of `nums` having length `k` and consisting of **indices** $i_0 < i_1 < \ldots < i_{k-1}$ is **balanced** if the following holds:

- $nums[i_j] - nums[i_{j-1}] >= i_j - i_{j-1}$, for every j in the range `[1, k - 1]`.

A **subsequence** of `nums` having length `1` is considered balanced.

Return *an integer denoting the **maximum** possible **sum of elements** in a **balanced** subsequence of* `nums`.

A **subsequence** of an array is a new **non-empty** array that is formed from the original array by deleting some (**possibly none**) of the elements without disturbing the relative positions of the remaining elements.

Example 1:

```
Input: nums = [3,3,5,6]
Output: 14
Explanation: In this example, the subsequence [3,5,6] consisting of indices 0, 2, and 3 can be
selected.
nums[2] - nums[0] >= 2 - 0.
nums[3] - nums[2] >= 3 - 2.
Hence, it is a balanced subsequence, and its sum is the maximum among the balanced subsequences of
nums.
The subsequence consisting of indices 1, 2, and 3 is also valid.
It can be shown that it is not possible to get a balanced subsequence with a sum greater than 14.
```

Example 2:

```
Input: nums = [5,-1,-3,8]
Output: 13
Explanation: In this example, the subsequence [5,8] consisting of indices 0 and 3 can be selected.
nums[3] - nums[0] >= 3 - 0.
Hence, it is a balanced subsequence, and its sum is the maximum among the balanced subsequences of
nums.
It can be shown that it is not possible to get a balanced subsequence with a sum greater than 13.
```

Example 3:

```
Input: nums = [-2,-1]
Output: -1
Explanation: In this example, the subsequence [-1] can be selected.
It is a balanced subsequence, and its sum is the maximum among the balanced subsequences of nums.
```

Constraints:

- $1 <= nums.length <= 10^5$
- $-10^9 <= nums[i] <= 10^9$

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming + Binary Indexed Tree

How to think about it

Balance nums[i]-nums[j] ≥ i-j rewrites as nums[i]-i ≥ nums[j]-j. On arr[t]=nums[t]-t we want a nondecreasing index sequence maximizing the sum of nums. The naive f[i]=nums[i]+max$_{j<i,\ arr[j]\le arr[i]}$ f[j] (or just nums[i]) is O(n^2) for n ≤ 10^5. After compressing arr, a Fenwick tree stores the best f among values up to a threshold. Scan in index order, query, then insert; a final prefix maximum is the answer.

According to the problem description, we can transform the inequality nums[i] - nums[j] ≥ i - j into nums[i] - i ≥ nums[j] - j. Therefore, we consider defining a new array arr, where arr[i] = nums[i] - i. A balanced subsequence satisfies that for any j < i, arr[j] ≤ arr[i]. The problem is transformed into selecting an increasing subsequence in arr such that the corresponding sum in nums is maximized.

Suppose i is the index of the last element in the subsequence, then we consider the index j of the second to last element in the subsequence. If arr[j] ≤ arr[i], we can consider whether to add j to the subsequence.

Therefore, we define f[i] as the maximum sum of nums when the index of the last element in the subsequence is i. The answer is max$_{i=0}^{n-1}$ f[i].

The state transition equation is:

$$f[i] = \max(\max_{j=0}^{i-1} f[j], 0) + nums[i]$$

where j satisfies arr[j] ≤ arr[i].

We can use a Binary Indexed Tree to maintain the maximum value of the prefix, i.e., for each arr[i], we maintain the maximum value of f[i] in the prefix arr[0..i].

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the length of the array nums.

Python 3

```python
class BinaryIndexedTree:
    def __init__(self, n: int):
        self.n = n
        self.c = [-inf] * (n + 1)

    def update(self, x: int, v: int):
        while x <= self.n:
            self.c[x] = max(self.c[x], v)
            x += x & -x

    def query(self, x: int) -> int:
        mx = -inf
        while x:
            mx = max(mx, self.c[x])
            x -= x & -x
        return mx


class Solution:
    def maxBalancedSubsequenceSum(self, nums: List[int]) -> int:
        arr = [x - i for i, x in enumerate(nums)]
        s = sorted(set(arr))
        tree = BinaryIndexedTree(len(s))
        for i, x in enumerate(nums):
            j = bisect_left(s, x - i) + 1
            v = max(tree.query(j), 0) + x
            tree.update(j, v)
        return tree.query(len(s))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the indices of the best balanced subsequence
# 2. Fenwick tree for prefix maximums (fast version of the
# table)


# ============================================================
# FOLLOW-UP 1: Return the indices of the best balanced
# subsequence
# ============================================================

# ---- follow-up 1: function ----
# Answer: Table over positions: best sum ending at each
# index, with the previous index remembered. Index j can
# precede i when nums[i] - nums[j] >= i - j.
# Time: O(n^2).  Space: O(n).
def best_balanced_indices(nums):
    n = len(nums)
    best = list(nums)
    previous = [-1] * n
    for i in range(n):
        for j in range(i):
            if nums[i] - nums[j] >= i - j and best[j] + nums[i] > best[i]:
                best[i] = best[j] + nums[i]
                previous[i] = j
    end = max(range(n), key=lambda i: best[i])
    indices = []
    while end != -1:
        indices.append(end)
        end = previous[end]
    return indices[::-1]


# ============================================================
# FOLLOW-UP 2: Fenwick tree for prefix maximums (fast
# version of the table)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Key every index by nums[i] - i; the best sum for
# a key at most the current key is a prefix maximum.
# Time: O(n log n).  Space: O(n).
def max_balanced_sum_fenwick(nums):
    keys = sorted(set(nums[i] - i for i in range(len(nums))))
    rank = {k: i + 1 for i, k in enumerate(keys)}
    size = len(keys)
    tree = [0] * (size + 1)

    def update(position, value):
        while position <= size:
            if value > tree[position]:
                tree[position] = value
            position += position & -position

    def query(position):
        best = 0
        while position > 0:
            if tree[position] > best:
                best = tree[position]
            position -= position & -position
        return best

    answer = max(nums)
    for i in range(len(nums)):
```

```
        r = rank[nums[i] - i]
        value = nums[i] + query(r)
        answer = max(answer, value)
        update(r, value)
    return answer
```

1862. Sum of Floored Pairs

Hard · Array, Math, Binary Search, Counting, Enumeration, Prefix Sum

leetcode.com/problems/sum-of-floored-pairs

PROBLEM

Given an integer array `nums`, return the sum of `floor(nums[i] / nums[j])` for all pairs of indices `0 <= i, j < nums.length` in the array. Since the answer may be too large, return it **modulo** $10^9 + 7$.

The `floor()` function returns the integer part of the division.

Example 1:

```
Input: nums = [2,5,9]
Output: 10
Explanation:
floor(2 / 5) = floor(2 / 9) = floor(5 / 9) = 0
floor(2 / 2) = floor(5 / 5) = floor(9 / 9) = 1
floor(5 / 2) = 2
floor(9 / 2) = 4
floor(9 / 5) = 1
We calculate the floor of the division for every pair of indices in the array then sum them up.
```

Example 2:

```
Input: nums = [7,7,7,7,7,7,7]
Output: 49
```

Constraints:

- `1 <= nums.length <=` 10^5
- `1 <= nums[i] <=` 10^5

SOLUTION (PYTHON 3)

Solution 1: Prefix Sum of Value Range + Optimized Enumeration

How to think about it

We need $\sum_{i,j}$floor(nums[i]/nums[j]). Pair enumeration is $O(n^2)$ and too slow for n≤ 10^5. Values are at most 10^5, so build a prefix of frequencies. For each denominator y and quotient d, the count of numerators in [dy,dy+y) is a prefix difference, multiplied by cnt[y]· d. The harmonic enumeration is O(Mlog M).

First, we count the occurrences of each element in the array nums and record them in the array cnt. Then, we calculate the prefix sum of the array cnt and record it in the array s, i.e., s[i] represents the count of elements less than or equal to i.

Next, we enumerate the denominator y and the quotient d. Using the prefix sum array, we can calculate the count of the numerator s[min(mx, d × y + y - 1)] - s[d × y - 1], where mx represents the maximum value in the array nums. Then, we multiply the count of the numerator by the count of the denominator cnt[y], and then multiply by the quotient d. This gives us the value of all fractions that meet the conditions. By summing these values, we can get the answer.

The time complexity is O(M × log M), and the space complexity is O(M). Here, M is the maximum value in the array nums.

Python 3

```
class Solution:
    def sumOfFlooredPairs(self, nums: List[int]) -> int:
        mod = 10**9 + 7
        cnt = Counter(nums)
        mx = max(nums)
        s = [0] * (mx + 1)
        for i in range(1, mx + 1):
            s[i] = s[i - 1] + cnt[i]
        ans = 0
        for y in range(1, mx + 1):
            if cnt[y]:
                d = 1
                while d * y <= mx:
                    ans += cnt[y] * d * (s[min(mx, d * y + y - 1)] - s[d * y - 1])
                    ans %= mod
                    d += 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Sum of CEILING(nums[i] / nums[j]) over all pairs
# 2. Only pairs with i < j: sum of floor(nums[i] /
# nums[j])


# ============================================================
# FOLLOW-UP 1: Sum of CEILING(nums[i] / nums[j]) over all
# pairs
# ============================================================

# ---- follow-up 1: function ----
# Answer: For a block of values between two multiples of a
# number, the ceiling is the same; count the values in
# each block with prefix counts.
# Time: O(n + M log M), M = largest value.  Space: O(M).
def sum_of_ceiled_pairs(nums):
    biggest = max(nums)
    count = [0] * (biggest + 2)
    for x in nums:
        count[x] += 1
    prefix = [0] * (biggest + 2)
    for v in range(1, biggest + 2):
        prefix[v] = prefix[v - 1] + count[v]
    total = 0
    for y in range(1, biggest + 1):
        if count[y] == 0:
            continue
        k = 1
        while (k - 1) * y < biggest:
            low = (k - 1) * y + 1
            high = min(k * y, biggest)
            total += count[y] * k * (prefix[high] - prefix[low - 1])
            k += 1
    return total


# ============================================================
# FOLLOW-UP 2: Only pairs with i < j: sum of floor(nums[i]
# / nums[j])
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk left to right with a Fenwick tree of the
# earlier values; for each divisor count the earlier
```

```
# values in every block of multiples.
# Time: O(n * (M / y) * log M) in the worst case, y = the
# smallest value.  Space: O(M).
def sum_of_floors_in_order(nums):
    biggest = max(nums)
    tree = [0] * (biggest + 2)

    def add(v):
        while v <= biggest:
            tree[v] += 1
            v += v & -v

    def prefix(v):
        s = 0
        v = min(v, biggest)
```

```
        while v > 0:
            s += tree[v]
            v -= v & -v
        return s

    total = 0
    for y in nums:
        k = 1
        while k * y <= biggest:
            total += k * (prefix((k + 1) * y - 1) - prefix(k * y - 1))
            k += 1
        add(y)
    return total
```

3187. Peaks in Array

Hard · Binary Indexed Tree, Segment Tree, Array

leetcode.com/problems/peaks-in-array

PROBLEM

A **peak** in an array `arr` is an element that is **greater** than its previous and next element in `arr`.

You are given an integer array `nums` and a 2D integer array `queries`.

You have to process queries of two types:

- `queries[i] = [1, l_i, r_i]`, determine the count of **peak** elements in the subarray `nums[l_i..r_i]`. notionvc: 73b20b7c-e1ab-4dac-86d0-13761094a9ae
- `queries[i] = [2, index_i, val_i]`, change `nums[index_i]` to `val_i`.

Return an array `answer` containing the results of the queries of the first type in order. notionvc: a9ccef22-4061-4b5a-b4cc-a2b2a0e12f30

Notes:

- The **first** and the **last** element of an array or a subarray notionvc: fcffef72-deb5-47cb-8719-3a3790102f73 **cannot** be a peak.

Example 1:

Input: nums = [3,1,4,2,5], queries = [[2,3,4],[1,0,4]]

Output: [0]

Explanation:

First query: We change `nums[3]` to 4 and `nums` becomes `[3,1,4,4,5]`.

Second query: The number of peaks in the `[3,1,4,4,5]` is 0.

Example 2:

Input: nums = [4,1,4,2,1,5], queries = [[2,2,4],[1,0,2],[1,0,4]]

Output: [0,1]

Explanation:

First query: `nums[2]` should become 4, but it is already set to 4.

Second query: The number of peaks in the `[4,1,4]` is 0.

Third query: The second 4 is a peak in the `[4,1,4,2,1]`.

Constraints:

- `3 <= nums.length <= 10^5`
- `1 <= nums[i] <= 10^5`
- `1 <= queries.length <= 10^5`
- `queries[i][0] == 1` or `queries[i][0] == 2`
- For all `i` that:
  - `queries[i][0] == 1: 0 <= queries[i][1] <= queries[i][2] <= nums.length - 1`
  - `queries[i][0] == 2: 0 <= queries[i][1] <= nums.length - 1, 1 <= queries[i][2] <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: Binary Indexed Tree

How to think about it

A peak is an interior index strictly above both neighbors. Range peak counts plus point updates make rebuilding O(nq). Peaks are 0/1 indicators, so a Fenwick tree range-sums them. An assignment at idx only changes peaks at idx-1,idx,idx+1. Insert the initial peaks. A query sums (l+1,r-1). An update subtracts the three old flags, writes the new value, and adds the new flags.

According to the problem description, for $0 < i < n - 1$, if it satisfies $nums[i - 1] < nums[i]$ and $nums[i] > nums[i + 1]$, we can consider $nums[i]$ as 1, otherwise as 0. Thus, for operation 1, i.e., querying the number of peak elements in the subarray $nums[l..r]$, it is equivalent to querying the number of 1s in the interval $[l + 1, r - 1]$. We can use a binary indexed tree to maintain the number of 1s in the interval $[1, n - 1]$.

For operation 1, i.e., updating $nums[idx]$ to val, it will only affect the values at positions $idx - 1$, idx, and $idx + 1$, so we only need to update these three positions. Specifically, we can first remove the peak elements at these three positions, then update the value of $nums[idx]$, and finally add back the peak elements at these three positions.

The time complexity is $O((n + q) \times \log n)$, and the space complexity is $O(n)$. Here, n and q are the lengths of the array `nums` and the query array `queries`, respectively.

Python 3

```python
class BinaryIndexedTree:
    __slots__ = "n", "c"

    def __init__(self, n: int):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x: int, delta: int) -> None:
        while x <= self.n:
            self.c[x] += delta
            x += x & -x

    def query(self, x: int) -> int:
        s = 0
        while x:
            s += self.c[x]
            x -= x & -x
        return s


class Solution:
    def countOfPeaks(self, nums: List[int], queries: List[List[int]]) -> List[int]:
        def update(i: int, val: int):
            if i <= 0 or i >= n - 1:
                return
            if nums[i - 1] < nums[i] and nums[i] > nums[i + 1]:
                tree.update(i, val)

        n = len(nums)
        tree = BinaryIndexedTree(n - 1)
        for i in range(1, n - 1):
            update(i, 1)
        ans = []
        for q in queries:
            if q[0] == 1:
                l, r = q[1] + 1, q[2] - 1
                ans.append(0 if l > r else tree.query(r) - tree.query(l - 1))
            else:
                idx, val = q[1:]
                for i in range(idx - 1, idx + 2):
                    update(i, -1)
                nums[idx] = val
                for i in range(idx - 1, idx + 2):
                    update(i, 1)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count the valleys instead of the peaks
# 2. Peaks and valleys together (zigzag points)


# ============================================================
# FOLLOW-UP 1: Count the valleys instead of the peaks
# ============================================================

# ---- follow-up 1: function ----
# Answer: A valley is smaller than both neighbours; the
# same Fenwick tree over the 0/1 flags.
# Time: build O(n log n), update O(log n), count O(log n).
# Space: O(n).
class ValleyCounter:
    def __init__(self, nums):
        self.nums = list(nums)
```

```
        self.n = len(nums)
        self.tree = [0] * (self.n + 1)
        self.flag = [0] * self.n
        for i in range(1, self.n - 1):
            if self.is_valley(i):
                self.flag[i] = 1
                self.add(i, 1)

    def is_valley(self, i):
        return 0 < i < self.n - 1 and self.nums[i] < self.nums[i - 1] and self.nums[i] < self.nums[i + 1]

    def add(self, i, amount):
        i += 1
        while i <= self.n:
            self.tree[i] += amount
            i += i & -i

    def prefix(self, i):
        i += 1
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & -i
        return total

    def update(self, index, value):
        self.nums[index] = value
        for i in (index - 1, index, index + 1):
            if 0 < i < self.n - 1:
                now = 1 if self.is_valley(i) else 0
                if now != self.flag[i]:
                    self.add(i, now - self.flag[i])
                    self.flag[i] = now

    def count(self, left, right):
        if right - left < 2:
            return 0
        return self.prefix(right - 1) - self.prefix(left)


# ============================================================
# FOLLOW-UP 2: Peaks and valleys together (zigzag points)
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: A point is a zigzag point when it is a peak or a
# valley; keep one flag per index and the same Fenwick
# tree.
# Time: build O(n log n), update O(log n), count O(log n).
# Space: O(n).
class ZigzagCounter:
    def __init__(self, nums):
        self.nums = list(nums)
        self.n = len(nums)
        self.tree = [0] * (self.n + 1)
        self.flag = [0] * self.n
        for i in range(1, self.n - 1):
            self.refresh(i)

    def is_zigzag(self, i):
        a, b, c = self.nums[i - 1], self.nums[i], self.nums[i + 1]
        return (b > a and b > c) or (b < a and b < c)

    def add(self, i, amount):
        i += 1
        while i <= self.n:
            self.tree[i] += amount
            i += i & -i
```

```
    def prefix(self, i):
        i += 1
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & -i
        return total

    def refresh(self, i):
        now = 1 if self.is_zigzag(i) else 0
        if now != self.flag[i]:
            self.add(i, now - self.flag[i])
            self.flag[i] = now

    def update(self, index, value):
        self.nums[index] = value
        for i in (index - 1, index, index + 1):
            if 0 < i < self.n - 1:
                self.refresh(i)

    def count(self, left, right):
        if right - left < 2:
            return 0
        return self.prefix(right - 1) - self.prefix(left)
```

PART 2

Extra Topics and More Practice

Fill the gaps: the rest of AlgoMaster's 600 problems

- 50 topic groups with 210 more problems that are not in Part 1, each on its own page with the statement, figures, Python 3 solutions and follow-up code.
- The groups follow AlgoMaster's 600 list (arrays, strings, linked lists, trees, graphs, shortest path, spanning trees, many kinds of DP, tries, line sweep ...).
- Each group page has a short note and lists the group's problems; it also shows which problems of the group are already in Part 1, with a link.
- Use it for the topics that Part 1 does not teach (minimum spanning tree, Eulerian paths, bitmask / digit / probability DP, tree DP) and for extra reps on the rest.

#	Topic	Problems	Page
42	Arrays	9	2078
43	Strings	8	2099
44	Bit Manipulation	2	2119
45	Hash Tables	8	2124
46	Two Pointers	5	2145
47	Prefix Sum	5	2160
48	Sliding Window - Fixed Size	2	2174
49	Sliding Window - Dynamic Size	3	2181
50	Kadane's Algorithm	5	2189
51	Matrix (2D Array)	1	2202
52	Linked List	3	2206
53	Stacks	5	2217
54	Monotonic Stack	6	2234
55	Queues	2	2251
56	Monotonic Queue	3	2257
57	Bucket Sort	1	2267
58	Recursion	2	2271
59	Divide and Conquer	3	2278
60	Binary Search	9	2293
61	Backtracking	1	2318
62	Tree Traversal - Level Order	1	2322

#	Topic	Problems	Page
63	Tree Traversal - Pre Order	7	2327
64	Tree Traversal - In Order	1	2352
65	Tree Traversal - Post-Order	4	2356
66	BST / Ordered Set	4	2371
67	Tries	4	2384
68	Heaps	1	2397
69	Intervals	6	2401
70	Data Structure Design	9	2418
71	Greedy	8	2453
72	Depth First Search (DFS)	4	2477
73	Breadth First Search (BFS)	4	2494
74	Topological Sort	1	2511
75	Union Find	4	2515
76	Minimum Spanning Tree	5	2532
77	Shortest Path	9	2553
78	Eulerian Circuit	3	2592
79	1-D DP	6	2601
80	Longest Increasing Subsequence (LIS)	2	2621
81	2D Grid DP	10	2628
82	String DP	5	2662
83	Tree / Graph DP	3	2678
84	Bitmask DP	4	2692
85	Digit DP	3	2707
86	Probability DP	3	2717
87	State Machine DP	3	2728
88	Maths / Geometry	6	2740
89	String Matching	3	2758
90	Binary Indexed Tree / Segment Tree	3	2767
91	Line Sweep	1	2782

PART 2 · TOPIC 1 OF 50

Arrays

9 problems · 5 Easy · 4 Medium · 0 Hard

THE IDEA

In-place array tricks: one pass with a counter, two or three running values, or a swap. Try to finish in O(n) time and O(1) extra space.

THE 9 PROBLEMS, EASY TO HARD | ★ = TOP 4: DO THESE FIRST

1	Missing Ranges	Easy	LC 163	2079
2	Majority Element	Easy	LC 169	2081
3	Third Maximum Number	Easy	LC 414	2083
4	Max Consecutive Ones	Easy	LC 485	2086
5	Shuffle the Array	Easy	LC 1470	2088
6	★ **Next Permutation**	Medium	LC 31	2090
7	★ **Majority Element II**	Medium	LC 229	2092
8	★ **Increasing Triplet Subsequence**	Medium	LC 334	2094
9	★ **Number of Zero-Filled Subarrays**	Medium	LC 2348	2097

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Move Zeroes (Part 1, pattern 3, page 133)

Remove Element (Part 1, pattern 3, page 135)

Rotate Array (Part 1, pattern 3, page 159)

Remove Duplicates from Sorted Array II (Part 1, pattern 3, page 153)

First Missing Positive (Part 1, pattern 1, page 65)

Remove Duplicates from Sorted Array (Part 1, pattern 3, page 141)

Best Time to Buy and Sell Stock (Part 1, pattern 2, page 69)

Product of Array Except Self (Part 1, pattern 5, page 255)

Best Time to Buy and Sell Stock II (Part 1, pattern 21, page 1159)

163. Missing Ranges

Easy · Array

leetcode.com/problems/missing-ranges

PROBLEM

You are given an inclusive range `[lower, upper]` and a **sorted unique** integer array `nums`, where all elements are within the inclusive range.

A number `x` is considered **missing** if `x` is in the range `[lower, upper]` and `x` is not in `nums`.

Return *the **shortest sorted** list of ranges that **exactly covers all the missing numbers***. That is, no element of `nums` is included in any of the ranges, and each missing number is covered by one of the ranges.

Example 1:

```
Input: nums = [0,1,3,50,75], lower = 0, upper = 99
Output: [[2,2],[4,49],[51,74],[76,99]]
Explanation: The ranges are:
[2,2]
[4,49]
[51,74]
[76,99]
```

Example 2:

```
Input: nums = [-1], lower = -1, upper = -1
Output: []
Explanation: There are no missing ranges since there are no missing numbers.
```

Constraints:

- -10^9 <= lower <= upper <= 10^9
- 0 <= nums.length <= 100
- lower <= nums[i] <= upper
- All the values of `nums` are **unique**.

SOLUTION (PYTHON 3)

Solution 1: Simulation

> **How to think about it**
>
> Report the ranges in [lower,upper] that nums does not cover. The array is sorted, distinct, and of length at most 100, so we simulate the gaps: before the first value, between neighbors, and after the last. A difference greater than 1 is a missing range.

We can simulate the problem directly according to the requirements.

The time complexity is O(n), where n is the length of the array nums. Ignoring the space consumption of the answer, the space complexity is O(1).

Python 3

```python
class Solution:
    def findMissingRanges(
        self, nums: List[int], lower: int, upper: int
    ) -> List[List[int]]:
        n = len(nums)
        if n == 0:
            return [[lower, upper]]
        ans = []
        if nums[0] > lower:
            ans.append([lower, nums[0] - 1])
        for a, b in pairwise(nums):
            if b - a > 1:
                ans.append([a + 1, b - 1])
        if nums[-1] < upper:
            ans.append([nums[-1] + 1, upper])
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Write the ranges as text: '5' for one number and
# '7->9' for several
# 2. How many numbers are missing in total


# ============================================================
# FOLLOW-UP 1: Write the ranges as text: '5' for one
# number and '7->9' for several
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same sweep; format each gap with a small helper.
# Time: O(n).  Space: O(n).
def missing_ranges_as_text(nums, lower, upper):
    def text(a, b):
        if a == b:
            return str(a)
        return str(a) + "->" + str(b)

    result = []
    previous = lower - 1
    for x in nums + [upper + 1]:
        if x - previous >= 2:
            result.append(text(previous + 1, x - 1))
        previous = x
    return result


# ============================================================
# FOLLOW-UP 2: How many numbers are missing in total
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the gap lengths between neighbours.
# Time: O(n).  Space: O(1).
def count_missing_numbers(nums, lower, upper):
    total = 0
    previous = lower - 1
    for x in nums + [upper + 1]:
        total += max(0, x - previous - 1)
        previous = x
    return total
```

PART 2 · ARRAYS · PROBLEM 2 OF 9

169. Majority Element

Easy · Array, Hash Table, Divide and Conquer, Counting, Sorting, Boyer-Moore Voting

leetcode.com/problems/majority-element

PROBLEM

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-10^9 <= nums[i] <= 10^9`
- The input is generated such that a majority element will exist in the array.

Follow-up:

Could you solve the problem in linear time and in

`O(1)`

space?

SOLUTION (PYTHON 3)

Solution 1: Moore Voting Algorithm

> **How to think about it**
>
> The majority value appears more than floor(n/2) times and is guaranteed. Counting or sorting works; the follow-up wants O(n) time and O(1) space. n≤ 5× 10^4. Boyer–Moore cancels distinct values in pairs. When the counter hits zero we change candidate. The majority cannot be fully cancelled, so the candidate after one pass is the answer; a second pass is unnecessary.

The basic steps of the Moore voting algorithm are as follows:

Initialize the element m and initialize the counter cnt = 0. Then, for each element x in the input list:

1. If cnt = 0, then m = x and cnt = 1;
1. Otherwise, if m = x, then cnt = cnt + 1, otherwise cnt = cnt - 1.

In general, the Moore voting algorithm requires **two passes** over the input list. In the first pass, we generate the candidate value m, and if there is a majority, the candidate value is the majority value. In the second pass, we simply compute the frequency of the candidate value to confirm whether it is the majority value. Since this problem has clearly stated that there is a majority value, we can directly return m after the first pass, without the need for a second pass to confirm whether it is the majority value.

The time complexity is O(n), where n is the length of the array nums. The space complexity is O(1).

Python 3

```python
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        cnt = m = 0
        for x in nums:
            if cnt == 0:
                m, cnt = x, 1
            else:
                cnt += 1 if m == x else -1
        return m
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The majority element may not exist: return None then
# 2. Elements that appear more than n / k times


# ============================================================
# FOLLOW-UP 1: The majority element may not exist: return
# None then
# ============================================================

# ---- follow-up 1: function ----
# Answer: Boyer-Moore finds a candidate; a second pass
# checks that it really appears more than n / 2 times.
# Time: O(n).  Space: O(1).
def majority_or_none(nums):
    candidate = None
    count = 0
    for x in nums:
        if count == 0:
            candidate = x
        count += 1 if x == candidate else -1
    if nums.count(candidate) > len(nums) // 2:
        return candidate
    return None


# ============================================================
# FOLLOW-UP 2: Elements that appear more than n / k times
# ============================================================

# ---- follow-up 2: function ----
# Answer: Boyer-Moore generalised (Misra-Gries): keep at
# most k - 1 counters, and decrease all of them when there
# is no room.
# Time: O(n * k).  Space: O(k).
def more_than_n_over_k(nums, k):
    counters = {}
    for x in nums:
        if x in counters:
            counters[x] += 1
        elif len(counters) < k - 1:
            counters[x] = 1
        else:
            for key in list(counters):
                counters[key] -= 1
                if counters[key] == 0:
                    del counters[key]
    result = []
    for candidate in counters:
        if nums.count(candidate) > len(nums) // k:
            result.append(candidate)
    return result
```

PART 2 · ARRAYS · PROBLEM 3 OF 9

414. Third Maximum Number

Easy · Array, Sorting

leetcode.com/problems/third-maximum-number

PROBLEM

You are given an integer array `nums`.

Return the **third distinct maximum** number in this array. If the third **maximum** does not exist, return the **maximum** number.

Example 1:

```
Input: nums = [3,2,1]
Output: 1
Explanation:
The first distinct maximum is 3.
The second distinct maximum is 2.
The third distinct maximum is 1.
```

Example 2:

```
Input: nums = [1,2]
Output: 2
Explanation:
The first distinct maximum is 2.
The second distinct maximum is 1.
The third distinct maximum does not exist, so the maximum (2) is returned instead.
```

Example 3:

```
Input: nums = [2,2,3,1]
Output: 1
Explanation:
The first distinct maximum is 3.
The second distinct maximum is 2 (both 2's are counted together since they have the same value).
The third distinct maximum is 1.
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

Follow up:

Can you find an

`O(n)`

solution?

SOLUTION (PYTHON 3)

Solution 1: Single Pass

How to think about it

We need the third distinct maximum. Sorting and unique-ing works, but the follow-up asks for a linear scan and constant extra memory. Keep $m_1 > m_2 > m_3$, skip a value already stored, and shift the triple when inserting. If m_3 stays at the sentinel, fewer than three distinct values exist and the maximum is the answer. Duplicates must be skipped, or one number would occupy all three slots.

We can use three variables m_1, m_2, and m_3 to represent the first, second, and third largest numbers in the array respectively. Initially, we set these three variables to negative infinity.

Then, we iterate through each number in the array. For each number:

- If it equals any of m_1, m_2, or m_3, we skip this number.
- If it is greater than m_1, we update the values of m_1, m_2, and m_3 to m_2, m_3, and this number respectively.
- If it is greater than m_2, we update the values of m_2 and m_3 to m_3 and this number respectively.
- If it is greater than m_3, we update the value of m_3 to this number.

Finally, if the value of m_3 has not been updated, it means that there is no third largest number in the array, so we return m_1. Otherwise, we return m_3.

The time complexity is O(n), where n is the length of the array `nums`. The space complexity is O(1).

Python 3

```
class Solution:
    def thirdMax(self, nums: List[int]) -> int:
        m1 = m2 = m3 = -inf
        for num in nums:
            if num in [m1, m2, m3]:
                continue
            if num > m1:
                m3, m2, m1 = m2, m1, num
            elif num > m2:
                m3, m2 = m2, num
            elif num > m3:
                m3 = num
        return m3 if m3 != -inf else m1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The k-th distinct maximum (or the maximum if there
# are fewer than k)
# 2. The third DISTINCT minimum (or the minimum), with
# O(1) memory


# ============================================================
# FOLLOW-UP 1: The k-th distinct maximum (or the maximum
# if there are fewer than k)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep a min-heap of the k biggest distinct
# values.
# Time: O(n log k).  Space: O(k).
import heapq


def kth_distinct_maximum(nums, k):
    heap = []
    seen = set()
    for x in nums:
        if x in seen:
            continue
        seen.add(x)
        heapq.heappush(heap, x)
        if len(heap) > k:
            seen.discard(heapq.heappop(heap))
    if len(heap) < k:
        return max(nums)
    return heap[0]
```

```
# ============================================================
# FOLLOW-UP 2: The third DISTINCT minimum (or the
# minimum), with O(1) memory
# ============================================================

# ---- follow-up 2: function ----
# Answer: Three variables, the mirror image of the
# maximum.
# Time: O(n).  Space: O(1).
def third_minimum(nums):
    first = second = third = None
    for x in nums:
        if x in (first, second, third):
            continue
        if first is None or x < first:
            first, second, third = x, first, second
        elif second is None or x < second:
            second, third = x, second
        elif third is None or x < third:
            third = x
    if third is None:
        return first
    return third
```

PART 2 · ARRAYS · PROBLEM 4 OF 9

485. Max Consecutive Ones

Easy · Array

leetcode.com/problems/max-consecutive-ones

PROBLEM

Given a binary array `nums`, return *the maximum number of consecutive* `1`*'s in the array*.

Example 1:

```
Input: nums = [1,1,0,1,1,1]
Output: 3
Explanation: The first two digits or the last three digits are consecutive 1s. The maximum number of consecutive 1s is 3.
```

Example 2:

```
Input: nums = [1,0,1,1,0,1]
Output: 2
```

Constraints:

- `1 <= nums.length <= 10^5`
- `nums[i]` is either `0` or `1`.

SOLUTION (PYTHON 3)

Solution 1: Single Pass

> **How to think about it**
>
> Longest run of ones. Checking every subarray is $O(n^2)$; one scan is enough. A 1 grows the current run and updates the answer; a 0 resets the counter. Zeros separate runs, so the counter never joins two blocks.

We can iterate through the array, using a variable cnt to record the current number of consecutive 1s, and another variable ans to record the maximum number of consecutive 1s.

When we encounter a 1, we increment cnt by one, and then update ans to be the maximum of cnt and ans itself, i.e., ans = max(ans, cnt). Otherwise, we reset cnt to 0.

After the iteration ends, we return the value of ans.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Python 3

```python
class Solution:
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
        ans = cnt = 0
        for x in nums:
            if x:
                cnt += 1
                ans = max(ans, cnt)
            else:
                cnt = 0
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return where the longest run of ones starts
# 2. You may flip at most one zero


# ============================================================
# FOLLOW-UP 1: Return where the longest run of ones starts
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember the start of the current run and the
# best run.
# Time: O(n).  Space: O(1).
def longest_run_of_ones(nums):
    best_length = 0
    best_start = -1
    start = 0
    for i in range(len(nums) + 1):
        if i == len(nums) or nums[i] == 0:
            if i - start > best_length:
                best_length = i - start
                best_start = start
            start = i + 1
    return best_length, best_start


# ============================================================
# FOLLOW-UP 2: You may flip at most one zero
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep a window with at most one zero.
# Time: O(n).  Space: O(1).
def longest_run_with_one_flip(nums):
    best = 0
    left = 0
    zeros = 0
    for right in range(len(nums)):
        if nums[right] == 0:
            zeros += 1
        while zeros > 1:
            if nums[left] == 0:
                zeros -= 1
            left += 1
        best = max(best, right - left + 1)
    return best
```

1470. Shuffle the Array

Easy · Array

leetcode.com/problems/shuffle-the-array

PROBLEM

Given the array `nums` consisting of `2n` elements in the form $[x_1,x_2,...,x_n,y_1,y_2,...,y_n]$.

Return the array in the form $[x_1,y_1,x_2,y_2,...,x_n,y_n]$.

Example 1:

```
Input: nums = [2,5,1,3,4,7], n = 3
Output: [2,3,5,4,1,7]
Explanation: Since x1=2, x2=5, x3=1, y1=3, y2=4, y3=7 then the answer is [2,3,5,4,1,7].
```

Example 2:

```
Input: nums = [1,2,3,4,4,3,2,1], n = 4
Output: [1,4,2,3,3,2,4,1]
```

Example 3:

```
Input: nums = [1,1,2,2], n = 2
Output: [1,2,1,2]
```

Constraints:

- `1 <= n <= 500`
- `nums.length == 2n`
- `1 <= nums[i] <= 10^3`

SOLUTION (PYTHON 3)

Solution 1: Simulation

> **How to think about it**
>
> Interleave $[x_1,...,x_n]$ with $[y_1,...,y_n]$. Zip the two halves and flatten.

We traverse the indices i in the range [0, n). Each time, we take nums[i] and nums[i+n] and place them sequentially into the answer array.

After the traversal is complete, we return the answer array.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def shuffle(self, nums: List[int], n: int) -> List[int]:
        return [x for pair in zip(nums[:n], nums[n:]) for x in pair]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Shuffle in place with O(1) extra memory
# 2. The reverse operation: unshuffle [x1,y1,x2,y2,...]
# back to [x1,x2,...,y1,y2,...]


# ============================================================
# FOLLOW-UP 1: Shuffle in place with O(1) extra memory
# ============================================================

# ---- follow-up 1: function ----
# Answer: Pack two numbers into one: nums[i] holds old
# value in the low bits and the new value in the high bits
# (numbers are below 1024).
# Time: O(n).  Space: O(1) extra.
def shuffle_in_place(nums, n):
    for i in range(n, 2 * n):
        nums[i] |= (nums[i - n] & 1023) << 10
    index = 0
    for i in range(n, 2 * n):
        nums[index] = (nums[i] >> 10) & 1023
        nums[index + 1] = nums[i] & 1023
        index += 2
    return nums


# ============================================================
# FOLLOW-UP 2: The reverse operation: unshuffle
# [x1,y1,x2,y2,...] back to [x1,x2,...,y1,y2,...]
# ============================================================

# ---- follow-up 2: function ----
# Answer: Take the even positions, then the odd positions.
# Time: O(n).  Space: O(n).
def unshuffle(nums):
    return nums[0::2] + nums[1::2]
```

31. Next Permutation

Medium · Array, Two Pointers

leetcode.com/problems/next-permutation

★ **TOP 7 PROBLEM** in Arrays: master this one first

PROBLEM

A **permutation** of an array of integers is an arrangement of its members into a sequence or linear order.

- For example, for `arr = [1,2,3]`, the following are all the permutations of `arr`: `[1,2,3]`, `[1,3,2]`, `[2, 1, 3]`, `[2, 3, 1]`, `[3,1,2]`, `[3,2,1]`.

The **next permutation** of an array of integers is the next lexicographically greater permutation of its integer. More formally, if all the permutations of the array are sorted in one container according to their lexicographical order, then the **next permutation** of that array is the permutation that follows it in the sorted container. If such arrangement is not possible, the array must be rearranged as the lowest possible order (i.e., sorted in ascending order).

- For example, the next permutation of `arr = [1,2,3]` is `[1,3,2]`.
- Similarly, the next permutation of `arr = [2,3,1]` is `[3,1,2]`.
- While the next permutation of `arr = [3,2,1]` is `[1,2,3]` because `[3,2,1]` does not have a lexicographical larger rearrangement.

Given an array of integers `nums`, *find the next permutation of* `nums`.

The replacement must be **in place** and use only constant extra memory.

Example 1:

```
Input: nums = [1,2,3]
Output: [1,3,2]
```

Example 2:

```
Input: nums = [3,2,1]
Output: [1,2,3]
```

Example 3:

```
Input: nums = [1,1,5]
Output: [1,5,1]
```

Constraints:

- `1 <= nums.length <= 100`
- `0 <= nums[i] <= 100`

SOLUTION (PYTHON 3)

Solution 1: Two traversals

How to think about it

The first idea is to generate every permutation and take the one after the current sequence. Correct, but n! is impossible at n ≤ 100, and the problem requires an in-place, constant-extra-space update. We do not need the full set, only the next sequence in lexicographic order: strictly larger, changing as far right as possible, and by the smallest amount. The longest non-increasing suffix is already the largest arrangement of that tail; the element just before it, nums[i], is the pivot we must raise. The smallest larger replacement sits at the right end of that suffix. After the swap, reverse the suffix so it becomes increasing. If the whole array is non-increasing, no larger permutation exists and we reverse everything.

We first traverse the array from back to front and find the first position i where nums[i] < nums[i + 1].

Then traverse the array from back to front again and find the first position j where nums[j] > nums[i]. Swap nums[i] and nums[j], and then reverse the elements from nums[i + 1] to nums[n - 1], the next permutation can be obtained.

The time complexity is O(n) and the space complexity is O(1). Where n is the length of the array.

Python 3

```
class Solution:
    def nextPermutation(self, nums: List[int]) -> None:
        n = len(nums)
        i = next((i for i in range(n - 2, -1, -1) if nums[i] < nums[i + 1]), -1)
        if ~i:
            j = next((j for j in range(n - 1, i, -1) if nums[j] > nums[i]))
            nums[i], nums[j] = nums[j], nums[i]
        nums[i + 1 :] = nums[i + 1 :][::-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The previous permutation
# 2. The k-th permutation of 1..n in order


# ============================================================
# FOLLOW-UP 1: The previous permutation
# ============================================================

# ---- follow-up 1: function ----
# Answer: Mirror image: find the last place where the
# number is bigger than its right neighbour, swap it with
# the biggest smaller number on its right, and reverse the
# tail.
# Time: O(n).  Space: O(1).
def previous_permutation(nums):
    i = len(nums) - 2
    while i >= 0 and nums[i] <= nums[i + 1]:
        i -= 1
    if i >= 0:
        j = len(nums) - 1
        while nums[j] >= nums[i]:
            j -= 1
        nums[i], nums[j] = nums[j], nums[i]
    nums[i + 1:] = reversed(nums[i + 1:])
    return nums


# ============================================================
# FOLLOW-UP 2: The k-th permutation of 1..n in order
# ============================================================

# ---- follow-up 2: function ----
# Answer: Factorial number system: the first digit is k //
# (n - 1)!, and so on.
# Time: O(n^2).  Space: O(n).
def kth_permutation(n, k):
    numbers = list(range(1, n + 1))
    factorial = [1] * (n + 1)
    for i in range(1, n + 1):
        factorial[i] = factorial[i - 1] * i
    k -= 1
    result = []
    for i in range(n, 0, -1):
        index, k = divmod(k, factorial[i - 1])
        result.append(numbers.pop(index))
    return result
```

PART 2 · ARRAYS · PROBLEM 7 OF 9

229. Majority Element II

Medium · Array, Hash Table, Counting, Sorting, Boyer-Moore Voting

leetcode.com/problems/majority-element-ii

★ **TOP 7 PROBLEM** in Arrays: master this one first

PROBLEM

Given an integer array of size `n`, find all elements that appear more than `⌊n / 3⌋` times.

Example 1:

```
Input: nums = [3,2,3]
Output: [3]
```

Example 2:

```
Input: nums = [1]
Output: [1]
```

Example 3:

```
Input: nums = [1,2]
Output: [1,2]
```

Constraints:

- `1 <= nums.length <= 5 * 10^4`
- `-10^9 <= nums[i] <= 10^9`

Follow up: Could you solve the problem in linear time and in `O(1)` space?

SOLUTION (PYTHON 3)

Solution 1

How to think about it

At most two values can appear more than floor(n/3) times. Hashing works but uses extra memory. Boyer–Moore can keep two candidates and their offsetting counts. A match increments a count; otherwise both counts decrement when they are positive. A second pass verifies the true frequencies.

Python 3

```python
class Solution:
    def majorityElement(self, nums: List[int]) -> List[int]:
        n1 = n2 = 0
        m1, m2 = 0, 1
        for m in nums:
            if m == m1:
                n1 += 1
            elif m == m2:
                n2 += 1
            elif n1 == 0:
                m1, n1 = m, 1
            elif n2 == 0:
                m2, n2 = m, 1
            else:
                n1, n2 = n1 - 1, n2 - 1
        return [m for m in [m1, m2] if nums.count(m) > len(nums) // 3]
```

Complexity. Time: O(n). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Elements that appear at least n / k times (not just
# more than)
# 2. Streaming version: after every number, the candidates
# that could be the answer


# ============================================================
# FOLLOW-UP 1: Elements that appear at least n / k times
# (not just more than)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same counters, but the final check uses >=; k -
# 1 counters are not enough in general, so keep k
# counters.
# Time: O(n * k).  Space: O(k).
def at_least_n_over_k(nums, k):
    counters = {}
    for x in nums:
        if x in counters:
            counters[x] += 1
        elif len(counters) < k:
            counters[x] = 1
        else:
            for key in list(counters):
                counters[key] -= 1
                if counters[key] == 0:
                    del counters[key]
    threshold = len(nums) / k
    return sorted(c for c in counters if nums.count(c) >= threshold)


# ============================================================
# FOLLOW-UP 2: Streaming version: after every number, the
# candidates that could be the answer
# ============================================================

# ---- follow-up 2: function ----
# Answer: Run the two-counter vote and report the current
# candidates that really have more than n / 3 so far (a
# sliding count dictionary).
# Time: O(n * d), d = distinct values.  Space: O(d).
def candidates_after_each(nums):
    seen = {}
    answers = []
    for i, x in enumerate(nums, 1):
        seen[x] = seen.get(x, 0) + 1
        answers.append(sorted(v for v, c in seen.items() if c > i // 3))
    return answers
```

PART 2 · ARRAYS · PROBLEM 8 OF 9

334. Increasing Triplet Subsequence

Medium · Greedy, Array, Longest Increasing Subsequence

leetcode.com/problems/increasing-triplet-subsequence

★ **TOP 7 PROBLEM** in Arrays: master this one first

PROBLEM

Given an integer array `nums`, return `true` *if there exists a triple of indices* `(i, j, k)` *such that* `i < j < k` *and* `nums[i] < nums[j] < nums[k]`. If no such indices exists, return `false`.

Example 1:

```
Input: nums = [1,2,3,4,5]
Output: true
Explanation: Any triplet where i < j < k is valid.
```

Example 2:

```
Input: nums = [5,4,3,2,1]
Output: false
Explanation: No triplet exists.
```

Example 3:

```
Input: nums = [2,1,5,0,4,6]
Output: true
Explanation: One of the valid triplet is (1, 4, 5), because nums[1] == 1 < nums[4] == 4 < nums[5] == 6.
```

Constraints:

- `1 <= nums.length <= 5 * 10^5`
- `-2^31 <= nums[i] <= 2^31 - 1`

Follow up:

Could you implement a solution that runs in

`O(n)`

time complexity and

`O(1)`

space complexity?

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Decide whether an increasing triplet of indices exists. Trying every middle index is O(n^2). We only need a smaller left value and a candidate second value. Keep mi<mid. A value above mid finishes the triplet; otherwise update mi if it is no larger, else update mid. The old index of mid need not be stored: it already sits after a smaller mi.

Python 3

```
class Solution:
    def increasingTriplet(self, nums: List[int]) -> bool:
        mi, mid = inf, inf
        for num in nums:
            if num > mid:
                return True
            if num <= mi:
                mi = num
            else:
                mid = num
        return False
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Increasing subsequence of length k
# 2. Return one increasing triplet (the values)


# ============================================================
# FOLLOW-UP 1: Increasing subsequence of length k
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the smallest possible ending for every
# length (patience sorting); length k exists when the list
# grows to k.
# Time: O(n log k).  Space: O(k).
import bisect


def has_increasing_subsequence(nums, k):
    tails = []
    for x in nums:
        position = bisect.bisect_left(tails, x)
        if position == len(tails):
            tails.append(x)
        else:
            tails[position] = x
        if len(tails) >= k:
            return True
    return k <= 0


# ============================================================
# FOLLOW-UP 2: Return one increasing triplet (the values)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep first, second and for each second the first
# that came before; when a bigger number arrives, we have
# the triplet.
# Time: O(n).  Space: O(1).
def find_increasing_triplet(nums):
    first = None
    second = None
    second_start = None
    for x in nums:
        if first is None or x <= first:
            first = x
        elif second is None or x <= second:
            second = x
            second_start = first
        else:
            return [second_start, second, x]
```

```
    return None
```

PART 2 · ARRAYS · PROBLEM 9 OF 9

2348. Number of Zero-Filled Subarrays

Medium · Array, Math

leetcode.com/problems/number-of-zero-filled-subarrays

★ **TOP 7 PROBLEM** in Arrays: master this one first

PROBLEM

Given an integer array `nums`, return *the number of **subarrays** filled with* `0`.

A **subarray** is a contiguous non-empty sequence of elements within an array.

Example 1:

```
Input: nums = [1,3,0,0,2,0,0,4]
Output: 6
Explanation:
There are 4 occurrences of [0] as a subarray.
There are 2 occurrences of [0,0] as a subarray.
There is no occurrence of a subarray with a size more than 2 filled with 0. Therefore, we return 6.
```

Example 2:

```
Input: nums = [0,0,0,2,0,0]
Output: 9
Explanation:
There are 5 occurrences of [0] as a subarray.
There are 3 occurrences of [0,0] as a subarray.
There is 1 occurrence of [0,0,0] as a subarray.
There is no occurrence of a subarray with a size more than 3 filled with 0. Therefore, we return 9.
```

Example 3:

```
Input: nums = [2,10,2019]
Output: 0
Explanation: There is no subarray filled with 0. Therefore, we return 0.
```

Constraints:

- `1 <= nums.length <= 10^5`
- `-10^9 <= nums[i] <= 10^9`

SOLUTION (PYTHON 3)

Solution 1: Traversal and Counting

How to think about it

Zero-filled subarrays live inside runs of zeros. $n \le 10^5$, so we count by run instead of endpoints. Keep the current run length cnt. A zero increments cnt and adds it to the answer (subarrays ending here); a nonzero resets cnt.

We traverse the array nums and use a variable cnt to record the current number of consecutive 0s. For the current element x we are traversing, if x is 0, then cnt is incremented by 1, and the number of all-zero subarrays ending with the current x is cnt, which we add to the answer. Otherwise, we set cnt to 0.

After the traversal, we return the answer.

Time complexity O(n), where n is the length of the array nums. Space complexity O(1).

Similar problems:

- 413. Arithmetic Slices

- 1513. Number of Substrings With Only 1s

Python 3

```python
class Solution:
    def zeroFilledSubarray(self, nums: List[int]) -> int:
        ans = cnt = 0
        for x in nums:
            if x == 0:
                cnt += 1
                ans += cnt
            else:
                cnt = 0
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count the subarrays whose numbers are all equal (any
# value)
# 2. The longest run of zeros and where it starts


# ============================================================
# FOLLOW-UP 1: Count the subarrays whose numbers are all
# equal (any value)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same run counting: a run of length L adds 1 + 2
# + ... + L.
# Time: O(n).  Space: O(1).
def count_constant_subarrays(nums):
    total = 0
    run = 0
    for i in range(len(nums)):
        if i > 0 and nums[i] == nums[i - 1]:
            run += 1
        else:
            run = 1
        total += run
    return total


# ============================================================
# FOLLOW-UP 2: The longest run of zeros and where it
# starts
# ============================================================

# ---- follow-up 2: function ----
# Answer: Track the start of the current zero run.
# Time: O(n).  Space: O(1).
def longest_zero_run(nums):
    best = (0, -1)
    start = None
    for i in range(len(nums) + 1):
        if i < len(nums) and nums[i] == 0:
            if start is None:
                start = i
        else:
            if start is not None and i - start > best[0]:
                best = (i - start, start)
            start = None
    return best
```

PART 2 · TOPIC 2 OF 50

Strings

8 problems · 3 Easy · 4 Medium · 1 Hard

THE IDEA

Scan once with a pointer or a small state, compare characters directly, and only reach for a map or a trie when the problem needs counts or prefixes.

THE 8 PROBLEMS, EASY TO HARD | ★ = TOP 5: DO THESE FIRST

1	Length of Last Word	Easy	LC 58	2100
2	Is Subsequence	Easy	LC 392	2102
3	Rotate String	Easy	LC 796	2104
4	★ **Reverse Words in a String**	Medium	LC 151	2106
5	★ **One Edit Distance**	Medium	LC 161	2109
6	★ **Add Bold Tag in String**	Medium	LC 616	2111
7	★ **Determine if Two Strings Are Close**	Medium	LC 1657	2114
8	★ **Guess the Word**	Hard	LC 843	2116

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Reverse String (Part 1, pattern 3, page 131)

Valid Palindrome II (Part 1, pattern 3, page 144)

Longest Common Prefix (Part 1, pattern 6, page 295)

Implement strStr() — substitute: Find the Index of the First Occurrence in a String (Part 1, pattern 6, page 299)

Count and Say (Part 1, pattern 6, page 330)

Valid Palindrome (Part 1, pattern 3, page 128)

Valid Anagram (Part 1, pattern 1, page 14)

Longest Palindrome (Part 1, pattern 1, page 25)

Zigzag Conversion (Part 1, pattern 6, page 307)

Text Justification (Part 1, pattern 6, page 341)

58. Length of Last Word

Easy · String

leetcode.com/problems/length-of-last-word

PROBLEM

Given a string `s` consisting of words and spaces, return *the length of the **last** word in the string.*

A **word** is a maximal substring consisting of non-space characters only.

Example 1:

```
Input: s = "Hello World"
Output: 5
Explanation: The last word is "World" with length 5.
```

Example 2:

```
Input: s = " fly me to the moon "
Output: 4
Explanation: The last word is "moon" with length 4.
```

Example 3:

```
Input: s = "luffy is still joyboy"
Output: 6
Explanation: The last word is "joyboy" with length 6.
```

Constraints:

- `1 <= s.length <= 10^4`
- `s` consists of only English letters and spaces `' '`.
- There will be at least one word in `s`.

SOLUTION (PYTHON 3)

Solution 1: Reverse Traversal + Two Pointers

How to think about it

The first idea is to split on spaces and take the last token. n ≤ 10^4 would pass, but we scan the whole string and allocate substrings. The waste is every earlier word. The last word sits at the end (possibly behind trailing spaces). So scan from the right: skip trailing spaces to get i, then walk to the word's left edge j; the length is i-j.

We start traversing from the end of the string s, find the first character that is not a space, which is the last character of the last word, and mark the index as i. Then continue to traverse forward, find the first character that is a space, which is the character before the first character of the last word, and mark it as j. Then the length of the last word is i - j.

The time complexity is O(n), where n is the length of the string s. The space complexity is O(1).

Python 3

```python
class Solution:
    def lengthOfLastWord(self, s: str) -> int:
        i = len(s) - 1
        while i >= 0 and s[i] == ' ':
            i -= 1
        j = i
        while j >= 0 and s[j] != ' ':
            j -= 1
        return i - j
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Length of the longest word
# 2. The last word itself, found by scanning from the end
# (no split)


# ============================================================
# FOLLOW-UP 1: Length of the longest word
# ============================================================

# ---- follow-up 1: function ----
# Answer: Split on spaces and take the longest.
# Time: O(n).  Space: O(n).
def longest_word_length(s):
    best = 0
    for word in s.split():
        best = max(best, len(word))
    return best


# ============================================================
# FOLLOW-UP 2: The last word itself, found by scanning
# from the end (no split)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Skip trailing spaces, then walk back to the
# previous space.
# Time: O(n).  Space: O(1) extra besides the returned
# word.
def last_word(s):
    end = len(s) - 1
    while end >= 0 and s[end] == " ":
        end -= 1
    start = end
    while start >= 0 and s[start] != " ":
        start -= 1
    return s[start + 1:end + 1]
```

PART 2 · STRINGS · PROBLEM 2 OF 8

392. Is Subsequence

Easy · Two Pointers, String, Dynamic Programming

leetcode.com/problems/is-subsequence

PROBLEM

Given two strings `s` and `t`, return `true` *if* `s` *is a* ***subsequence*** *of* `t`*, or* `false` *otherwise*.

A **subsequence** of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., `"ace"` is a subsequence of `"abcde"` while `"aec"` is not).

Example 1:

```
Input: s = "abc", t = "ahbgdc"
Output: true
```

Example 2:

```
Input: s = "axc", t = "ahbgdc"
Output: false
```

Constraints:

- `0 <= s.length <= 100`
- `0 <= t.length <= 10^4`
- `s` and `t` consist only of lowercase English letters.

Follow up:

Suppose there are lots of incoming

`s`

, say

`s_1, s_2, ..., s_k`

where

`k >= 10^9`

, and you want to check one by one to see if

`t`

has its subsequence. In this scenario, how would you change your code?

SOLUTION (PYTHON 3)

Solution 1: Two Pointers

> **How to think about it**
>
> Is s a subsequence of t? Each character of s needs a later match in t. Two pointers in one scan. i walks s, j walks t; on a match advance i. s is a subsequence iff i reaches the end.

We define two pointers i and j to point to the initial position of the string s and t respectively. Each time we compare the two characters pointed to by the two pointers, if they are the same, both pointers move right at the same time; if they are not the same, only j moves right. When the pointer i moves to the end of the string s, it means that s is the subsequence of t.

The time complexity is O(m + n), where m and n are the lengths of the strings s and t respectively. The space complexity is O(1).

Python 3

```python
class Solution:
    def isSubsequence(self, s: str, t: str) -> bool:
        i = j = 0
        while i < len(s) and j < len(t):
            if s[i] == t[j]:
                i += 1
            j += 1
        return i == len(s)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Many strings s are checked against the same long t
# 2. Return the indices of t that match s


# ============================================================
# FOLLOW-UP 1: Many strings s are checked against the same
# long t
# ============================================================

# ---- follow-up 1: function ----
# Answer: Store the positions of every letter of t; for
# each s, binary search the next position after the last
# match.
# Time: build O(|t|), each check O(|s| log |t|).  Space:
# O(|t|).
import bisect


class SubsequenceChecker:
    def __init__(self, t):
        self.positions = {}
        for i, ch in enumerate(t):
            self.positions.setdefault(ch, []).append(i)

    def is_subsequence(self, s):
        previous = -1
        for ch in s:
            spots = self.positions.get(ch, [])
            k = bisect.bisect_right(spots, previous)
            if k == len(spots):
                return False
            previous = spots[k]
        return True


# ============================================================
# FOLLOW-UP 2: Return the indices of t that match s
# ============================================================

# ---- follow-up 2: function ----
# Answer: Greedy walk through t, recording the index of
# each matched letter.
# Time: O(|t|).  Space: O(|s|).
def matching_indices(s, t):
    indices = []
    i = 0
    for j in range(len(t)):
        if i < len(s) and s[i] == t[j]:
            indices.append(j)
            i += 1
    if i == len(s):
        return indices
    return None
```

PART 2 · STRINGS · PROBLEM 3 OF 8

796. Rotate String

Easy · String, String Matching

leetcode.com/problems/rotate-string

PROBLEM

Given two strings `s` and `goal`, return `true` *if and only if* `s` *can become* `goal` *after some number of* ***shifts*** *on* `s`.

A **shift** on `s` consists of moving the leftmost character of `s` to the rightmost position.

- For example, if `s = "abcde"`, then it will be `"bcdea"` after one shift.

Example 1:

Input: s = "abcde", goal = "cdeab"

Output: true

Explanation:

Rotating `s` to the left by 2 positions moves `"ab"` to the end, resulting in `"cdeab"`, which is equal to `goal`.

Example 2:

Input: s = "abcde", goal = "abced"

Output: false

Explanation:

No sequence of rotations of `s` can produce `"abced"`. The characters appear in a different relative order, so `goal` is not a rotation of `s`.

Constraints:

- `1 <= s.length, goal.length <= 100`
- `s` and `goal` consist of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1

> **How to think about it**
>
> Is goal a rotation of s? Unequal lengths fail; otherwise goal is a substring of s+s.

Python 3

```python
class Solution:
    def rotateString(self, s: str, goal: str) -> bool:
        return len(s) == len(goal) and goal in s + s
```

Complexity. Time: O(n) (substring search on 2n characters). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The smallest number of left rotations that turn s
# into goal
# 2. How many different rotations does s have


# ============================================================
# FOLLOW-UP 1: The smallest number of left rotations that
# turn s into goal
```

```python
# ============================================================

# ---- follow-up 1: function ----
# Answer: Look for goal inside s + s; the position of the
# first match is the number of rotations.
# Time: O(n).  Space: O(n).
def min_rotations(s, goal):
    if len(s) != len(goal):
        return -1
    return (s + s).find(goal)


# ============================================================
# FOLLOW-UP 2: How many different rotations does s have
# ============================================================

# ---- follow-up 2: function ----
# Answer: The smallest period p of s with p dividing n
# gives p different rotations.
# Time: O(n).  Space: O(n).
def distinct_rotations(s):
    n = len(s)
    position = (s + s).find(s, 1)
    return position if position != -1 and position < n else n
```

PART 2 · STRINGS · PROBLEM 4 OF 8

151. Reverse Words in a String

Medium · Two Pointers, String

leetcode.com/problems/reverse-words-in-a-string

★ **TOP 7 PROBLEM** in Strings: master this one first

PROBLEM

Given an input string `s`, reverse the order of the **words**.

A **word** is defined as a sequence of non-space characters. The **words** in `s` will be separated by at least one space.

Return *a string of the words in reverse order concatenated by a single space.*

Note that `s` may contain leading or trailing spaces or multiple spaces between two words. The returned string should only have a single space separating the words. Do not include any extra spaces.

Example 1:

```
Input: s = "the sky is blue"
Output: "blue is sky the"
```

Example 2:

```
Input: s = " hello world "
Output: "world hello"
Explanation: Your reversed string should not contain leading or trailing spaces.
```

Example 3:

```
Input: s = "a good example"
Output: "example good a"
Explanation: You need to reduce multiple spaces between two words to a single space in the reversed
string.
```

Constraints:

- `1 <= s.length <= 10^4`
- `s` contains English letters (upper-case and lower-case), digits, and spaces `' '`.
- There is **at least one** word in `s`.

Follow-up: If the string data type is mutable in your language, can you solve it **in-place** with `O(1)` extra space?

SOLUTIONS (PYTHON 3)

Solution 1: Two Pointers

How to think about it

Reverse the word order and collapse extra spaces. n≤ 10^4. Two pointers skip spaces, cut out each word, then reverse the list and join. Leading, trailing, and repeated spaces disappear.

We can use two pointers i and j to find each word, add it to the result list, then reverse the result list, and finally concatenate it into a string.

The time complexity is O(n), and the space complexity is O(n). Where n is the length of the string.

Python 3

```python
class Solution:
    def reverseWords(self, s: str) -> str:
        words = []
        i, n = 0, len(s)
        while i < n:
            while i < n and s[i] == " ":
                i += 1
            if i < n:
                j = i
                while j < n and s[j] != " ":
                    j += 1
                words.append(s[i:j])
                i = j
        return " ".join(words[::-1])
```

Solution 2: String Split

> **How to think about it**
>
> Solution 1 splits words by hand. A built-in split on whitespace already drops extra spaces; reverse that list and join. Shorter code, same complexity.

We can use the built-in string split function to split the string into a list of words by spaces, then reverse the list, and finally concatenate it into a string.

The time complexity is O(n), and the space complexity is O(n). Where n is the length of the string.

Python 3

```python
class Solution:
    def reverseWords(self, s: str) -> str:
        return " ".join(reversed(s.split()))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Do it in place on a list of characters
# 2. Reverse the letters of every word but keep the word
# order


# ============================================================
# FOLLOW-UP 1: Do it in place on a list of characters
# ============================================================

# ---- follow-up 1: function ----
# Answer: Reverse the whole list, then reverse every word
# back; collapse the spaces.
# Time: O(n).  Space: O(1).
def reverse_words_in_place(chars):
    def reverse(lo, hi):
        while lo < hi:
            chars[lo], chars[hi] = chars[hi], chars[lo]
            lo += 1
            hi -= 1

    reverse(0, len(chars) - 1)
    start = 0
    for i in range(len(chars) + 1):
        if i == len(chars) or chars[i] == " ":
            reverse(start, i - 1)
            start = i + 1
    return chars
```

```
# ============================================================
# FOLLOW-UP 2: Reverse the letters of every word but keep
# the word order
# ============================================================

# ---- follow-up 2: function ----
# Answer: Split on spaces and reverse each word.
# Time: O(n).  Space: O(n).
def reverse_each_word(s):
    return " ".join(word[::-1] for word in s.split(" "))
```

161. One Edit Distance

Medium · Two Pointers, String

leetcode.com/problems/one-edit-distance

★ **TOP 7 PROBLEM** in Strings: master this one first

PROBLEM

Given two strings `s` and `t`, return `true` if they are both one edit distance apart, otherwise return `false`.

A string `s` is said to be one distance apart from a string `t` if you can:

- Insert **exactly one** character into `s` to get `t`.
- Delete **exactly one** character from `s` to get `t`.
- Replace **exactly one** character of `s` with **a different character** to get `t`.

Example 1:

```
Input: s = "ab", t = "acb"
Output: true
Explanation: We can insert 'c' into s to get t.
```

Example 2:

```
Input: s = "", t = ""
Output: false
Explanation: We cannot get t from s by only one step.
```

Constraints:

- `0 <= s.length, t.length <= 10^4`
- `s` and `t` consist of lowercase letters, uppercase letters, and digits.

SOLUTION (PYTHON 3)

Solution 1: Discuss Different Cases

How to think about it

Exactly one insert, delete, or replace should turn s into t. A length gap above 1 is impossible. n≤ 10^4. Assume s is at least as long as t. At the first mismatch: equal lengths compare the suffixes (replace); unequal lengths compare s without that character to the rest of t (delete). If they match throughout, s must be exactly one character longer.

Let m represent the length of string s, and n represent the length of string t. We can assume that m is always greater than or equal to n.

If m-n > 1, return false directly;

Otherwise, iterate through s and t, if s[i] is not equal to t[i]:

- If m ≠ n, compare s[i+1:] with t[i:], return true if they are equal, otherwise return false;
- If m = n, compare s[i:] with t[i:], return true if they are equal, otherwise return false.

If the iteration ends, it means that all the characters of s and t that have been iterated are equal, at this time it needs to satisfy m=n+1.

The time complexity is O(m), where m is the length of string s. The space complexity is O(1).

Python 3

```
class Solution:
    def isOneEditDistance(self, s: str, t: str) -> bool:
        if len(s) < len(t):
            return self.isOneEditDistance(t, s)
        m, n = len(s), len(t)
        if m - n > 1:
            return False
        for i, c in enumerate(t):
            if c != s[i]:
                return s[i + 1 :] == t[i + 1 :] if m == n else s[i + 1 :] == t[i:]
        return m == n + 1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. At most k edits (insert, delete, replace)
# 2. Say which single edit turns s into t


# ============================================================
# FOLLOW-UP 1: At most k edits (insert, delete, replace)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Edit distance, but only the diagonal band of
# width 2k + 1 matters.
# Time: O(k * |s|).  Space: O(|t|).
def within_k_edits(s, t, k):
    if abs(len(s) - len(t)) > k:
        return False
    big = k + 1
    previous = [min(j, big) for j in range(len(t) + 1)]
    for i in range(1, len(s) + 1):
        current = [big] * (len(t) + 1)
        current[0] = min(i, big)
        for j in range(max(1, i - k), min(len(t), i + k) + 1):
            cost = 0 if s[i - 1] == t[j - 1] else 1
            current[j] = min(previous[j - 1] + cost, previous[j] + 1, current[j - 1] + 1, big)
        previous = current
    return previous[len(t)] <= k


# ============================================================
# FOLLOW-UP 2: Say which single edit turns s into t
# ============================================================

# ---- follow-up 2: function ----
# Answer: Find the first position where they differ and
# test the three kinds of edit.
# Time: O(n).  Space: O(n).
def the_single_edit(s, t):
    if s == t or abs(len(s) - len(t)) > 1:
        return None
    i = 0
    while i < min(len(s), len(t)) and s[i] == t[i]:
        i += 1
    if len(s) == len(t) and s[i + 1:] == t[i + 1:]:
        return ("replace", i, t[i])
    if len(s) + 1 == len(t) and s[i:] == t[i + 1:]:
        return ("insert", i, t[i])
    if len(s) == len(t) + 1 and s[i + 1:] == t[i:]:
        return ("delete", i, s[i])
    return None
```

616. Add Bold Tag in String

Medium · Trie, Array, Hash Table, String, String Matching, Aho-Corasick

leetcode.com/problems/add-bold-tag-in-string

★ **TOP 7 PROBLEM** in Strings: master this one first

PROBLEM

You are given a string `s` and an array of strings `words`.

You should add a closed pair of bold tag `<b>` and `</b>` to wrap the substrings in `s` that exist in `words`.

- If two such substrings overlap, you should wrap them together with only one pair of closed bold-tag.
- If two substrings wrapped by bold tags are consecutive, you should combine them.

Return `s` *after adding the bold tags*.

Example 1:

```
Input: s = "abcxyz123", words = ["abc","123"]
Output: "<b>abc</b>xyz<b>123</b>"
Explanation: The two strings of words are substrings of s as following: "abcxyz123".
We add <b> before each substring and </b> after each substring.
```

Example 2:

```
Input: s = "aaabbb", words = ["aa","b"]
Output: "<b>aaabbb</b>"
Explanation:
"aa" appears as a substring two times: "aaabbb" and "aaabbb".
"b" appears as a substring three times: "aaabbb", "aaabbb", and "aaabbb".
We add <b> before each substring and </b> after each substring:
"<b>a<b>a</b>a</b><b>b</b><b>b</b><b>b</b>".
Since the first two <b>'s overlap, we merge them: "<b>aaa</b><b>b</b><b>b</b><b>b</b>".
Since now the four <b>'s are consecutive, we merge them: "<b>aaabbb</b>".
```

Constraints:

- `1 <= s.length <= 1000`
- `0 <= words.length <= 100`
- `1 <= words[i].length <= 1000`
- `s` and `words[i]` consist of English letters and digits.
- All the values of `words` are **unique**.

Note: This question is the same as 758. Bold Words in String.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Every occurrence of a word in s must be wrapped, and intervals may overlap. Naive matching per word repeats work and still needs a merge pass. Insert words into a trie and walk from every start index to collect end positions. Merge overlapping intervals, then splice `<b>` around them.

Python 3

```
class Trie:
    def __init__(self):
        self.children = [None] * 128
        self.is_end = False

    def insert(self, word):
        node = self
        for c in word:
            idx = ord(c)
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True


class Solution:
    def addBoldTag(self, s: str, words: List[str]) -> str:
        trie = Trie()
        for w in words:
            trie.insert(w)
        n = len(s)
        pairs = []
        for i in range(n):
            node = trie
            for j in range(i, n):
                idx = ord(s[j])
                if node.children[idx] is None:
                    break
                node = node.children[idx]
                if node.is_end:
                    pairs.append([i, j])
        if not pairs:
            return s
        st, ed = pairs[0]
        t = []
        for a, b in pairs[1:]:
            if ed + 1 < a:
                t.append([st, ed])
                st, ed = a, b
            else:
                ed = max(ed, b)
        t.append([st, ed])

        ans = []
        i = j = 0
        while i < n:
            if j == len(t):
                ans.append(s[i:])
                break
            st, ed = t[j]
            if i < st:
                ans.append(s[i:st])
            ans.append('<b>')
            ans.append(s[st : ed + 1])
            ans.append('</b>')
            j += 1
            i = ed + 1

        return ''.join(ans)
```

Complexity. Time: O(n * m + L), m = longest word, L = total letters. Space: O(128 * L).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the merged bold intervals instead of the
# string
# 2. Use any pair of tags (for example <u> and </u>)


# ============================================================
# FOLLOW-UP 1: Return the merged bold intervals instead of
# the string
# ============================================================

# ---- follow-up 1: function ----
# Answer: Mark every covered position, then read the runs.
# Time: O(W * n * L) in the worst case.  Space: O(n).
def bold_intervals(s, words):
    covered = [False] * len(s)
    for word in words:
        start = s.find(word)
        while start != -1:
            for i in range(start, start + len(word)):
                covered[i] = True
            start = s.find(word, start + 1)
    intervals = []
    i = 0
    while i < len(s):
        if covered[i]:
            j = i
            while j + 1 < len(s) and covered[j + 1]:
                j += 1
            intervals.append((i, j + 1))
            i = j + 1
        else:
            i += 1
    return intervals


# ============================================================
# FOLLOW-UP 2: Use any pair of tags (for example <u> and
# </u>)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Insert the opening and closing tag text at the
# interval ends.
# Time: O(W * n * L).  Space: O(n).
def add_tags(s, words, open_tag, close_tag):
    result = []
    last = 0
    for start, end in bold_intervals(s, words):
        result.append(s[last:start])
        result.append(open_tag + s[start:end] + close_tag)
        last = end
    result.append(s[last:])
    return "".join(result)
```

1657. Determine if Two Strings Are Close

Medium · Hash Table, String, Counting, Sorting

leetcode.com/problems/determine-if-two-strings-are-close

★ **TOP 7 PROBLEM** in Strings: master this one first

PROBLEM

Two strings are considered **close** if you can attain one from the other using the following operations:

- Operation 1: Swap any two **existing** characters.
  - For example, `abcde -> aecdb`
- Operation 2: Transform **every** occurrence of one **existing** character into another **existing** character, and do the same with the other character.
  - For example, `aacabb -> bbcbaa` (all `a`'s turn into `b`'s, and all `b`'s turn into `a`'s)

You can use the operations on either string as many times as necessary.

Given two strings, `word1` and `word2`, return `true` *if* `word1` *and* `word2` *are* ***close****, and* `false` *otherwise.*

Example 1:

```
Input: word1 = "abc", word2 = "bca"
Output: true
Explanation: You can attain word2 from word1 in 2 operations.
Apply Operation 1: "abc" -> "acb"
Apply Operation 1: "acb" -> "bca"
```

Example 2:

```
Input: word1 = "a", word2 = "aa"
Output: false
Explanation: It is impossible to attain word2 from word1, or vice versa, in any number of operations.
```

Example 3:

```
Input: word1 = "cabbba", word2 = "abbccc"
Output: true
Explanation: You can attain word2 from word1 in 3 operations.
Apply Operation 1: "cabbba" -> "caabbb"
Apply Operation 2: "caabbb" -> "baaccc"
Apply Operation 2: "baaccc" -> "abbccc"
```

Constraints:

- `1 <= word1.length, word2.length <= 10^5`
- `word1` and `word2` contain only lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Counting + Sorting

How to think about it

Swapping two characters and renaming one letter globally preserve the set of letters and the multiset of frequencies (the latter up to permuting values among those letters). Hence the strings are close iff they use the same letters and their frequency lists match after sorting. Count both strings and compare key sets plus sorted value lists.

According to the problem description, two strings are close if they meet the following two conditions simultaneously:

1. The strings `word1` and `word2` must contain the same types of letters.

2. The arrays obtained by sorting the counts of all characters in `word1` and `word2` must be the same.

Therefore, we can first use an array or hash table to count the occurrences of each letter in `word1` and `word2` respectively, and then compare whether they are the same. If they are not the same, return `false` early.

Otherwise, we sort the corresponding counts, and then compare whether the counts at the corresponding positions are the same. If they are not the same, return `false`.

At the end of the traversal, return `true`.

The time complexity is O(m + n + C × log C), and the space complexity is O(C). Here, m and n are the lengths of the strings `word1` and `word2` respectively, and C is the number of letter types. In this problem, C=26.

Python 3

```
class Solution:
    def closeStrings(self, word1: str, word2: str) -> bool:
        cnt1, cnt2 = Counter(word1), Counter(word2)
        return sorted(cnt1.values()) == sorted(cnt2.values()) and set(
            cnt1.keys()
        ) == set(cnt2.keys())
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Only the swap-two-letters operation is allowed
# 2. Only operation 2 is allowed (swap the names of two
# letters that both occur, as often as you like)


# ============================================================
# FOLLOW-UP 1: Only the swap-two-letters operation is
# allowed
# ============================================================

# ---- follow-up 1: function ----
# Answer: Then the strings must be anagrams of each other.
# Time: O(n log n).  Space: O(n).
def close_with_swaps_only(word1, word2):
    return sorted(word1) == sorted(word2)


# ============================================================
# FOLLOW-UP 2: Only operation 2 is allowed (swap the names
# of two letters that both occur, as often as you like)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Positions never move, so word2 must be word1
# with its letters consistently renamed, and both must use
# the same set of letters.
# Time: O(n).  Space: O(1).
def close_with_renames_only(word1, word2):
    if len(word1) != len(word2) or set(word1) != set(word2):
        return False
    forward = {}
    backward = {}
    for a, b in zip(word1, word2):
        if forward.setdefault(a, b) != b or backward.setdefault(b, a) != a:
            return False
    return True
```

843. Guess the Word

Hard · Minimax, Array, Math, String, Game Theory, Interactive

leetcode.com/problems/guess-the-word

★ **TOP 7 PROBLEM** in Strings: master this one first

PROBLEM

You are given an array of unique strings `words` where `words[i]` is six letters long. One word of `words` was chosen as a secret word.

You are also given the helper object `Master`. You may call `Master.guess(word)` where `word` is a six-letter-long string, and it must be from `words`. `Master.guess(word)` returns:

- `-1` if `word` is not from `words`, or
- an integer representing the number of exact matches (value and position) of your guess to the secret word.

There is a parameter `allowedGuesses` for each test case where `allowedGuesses` is the maximum number of times you can call `Master.guess(word)`.

For each test case, you should call `Master.guess` with the secret word without exceeding the maximum number of allowed guesses. You will get:

- `"Either you took too many guesses, or you did not find the secret word."` if you called `Master.guess` more than `allowedGuesses` times or if you did not call `Master.guess` with the secret word, or
- `"You guessed the secret word correctly."` if you called `Master.guess` with the secret word with the number of calls to `Master.guess` less than or equal to `allowedGuesses`.

The test cases are generated such that you can guess the secret word with a reasonable strategy (other than using the bruteforce method).

Example 1:

```
Input: secret = "acckzz", words = ["acckzz","ccbazz","eiowzz","abcczz"], allowedGuesses = 10
Output: You guessed the secret word correctly.
Explanation:
master.guess("aaaaaa") returns -1, because "aaaaaa" is not in words.
master.guess("acckzz") returns 6, because "acckzz" is secret and has all 6 matches.
master.guess("ccbazz") returns 3, because "ccbazz" has 3 matches.
master.guess("eiowzz") returns 2, because "eiowzz" has 2 matches.
master.guess("abcczz") returns 4, because "abcczz" has 4 matches.
We made 5 calls to master.guess, and one of them was the secret, so we pass the test case.
```

Example 2:

```
Input: secret = "hamada", words = ["hamada","khaled"], allowedGuesses = 10
Output: You guessed the secret word correctly.
Explanation: Since there are two words, you can guess both.
```

Constraints:

- `1 <= words.length <= 100`
- `words[i].length == 6`
- `words[i]` consist of lowercase English letters.
- All the strings of `words` are **unique**.
- `secret` exists in `words`.
- `10 <= allowedGuesses <= 30`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

The secret must be found in at most 30 guesses, each returning a match count. There are only 100 length-6 words, yet random guesses need not shrink the candidate set. Keep the words still consistent with all answers, and filter by the match count after each guess. Even without implementation code here, the core is using that count to reduce the search to a manageable set.

Python 3 (written for this guide: the doocs page has an empty Python tab)

```
class Solution:
    def findSecretWord(self, words: List[str], master: "Master") -> None:
        def match(a: str, b: str) -> int:
            return sum(x == y for x, y in zip(a, b))

        candidates = words[:]
        for _ in range(10):
            # Choose the guess that splits the remaining candidates best. Any word of the list may be
guessed,
            # even one that is already ruled out: compare the groups of candidates by how many letters
            # they share with the guess, prefer the smallest biggest group, then the smallest sum of
            # squares, then a word that can still be the secret.
            still_possible = set(candidates)
            best_word, best_key = None, None
            for guess in words:
                groups = {}
                for other in candidates:
                    k = match(guess, other)
                    groups[k] = groups.get(k, 0) + 1
                key = (max(groups.values()), sum(v * v for v in groups.values()), guess not in
still_possible)
                if best_key is None or key < best_key:
                    best_word, best_key = guess, key
            hits = master.guess(best_word)
            if hits == 6:
                return
            # the secret shares exactly `hits` letters with the guess, so keep only such words
            candidates = [w for w in candidates if match(best_word, w) == hits]
```

Complexity. Time: O(10 * N^2) (each of the 10 rounds compares every word with every candidate). Space: O(N).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Pick a random candidate instead of the best one
# 2. How many candidates are left after a guess and its
# score


# ============================================================
# FOLLOW-UP 1: Pick a random candidate instead of the best
# one
# ============================================================

# ---- follow-up 1: function ----
# Answer: Guess any remaining word and keep the words that
# match its score; usually fine but less safe.
# Time: O(10 * n) for n words.  Space: O(n).
import random


def guess_with_random_choice(words, master):
    def match(a, b):
```

```
        return sum(x == y for x, y in zip(a, b))

    candidates = words[:]
    for _ in range(10):
        guess = random.choice(candidates)
        hits = master.guess(guess)
        if hits == 6:
            return
        candidates = [w for w in candidates if match(guess, w) == hits]


# ============================================================
# FOLLOW-UP 2: How many candidates are left after a guess
# and its score
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the words that share exactly that many
# letters with the guess.
# Time: O(n).  Space: O(n).
def candidates_left(words, guess, hits):
    return [w for w in words if sum(x == y for x, y in zip(guess, w)) == hits]
```

PART 2 · TOPIC 3 OF 50

Bit Manipulation

2 problems · 2 Easy · 0 Medium · 0 Hard

THE IDEA

Think of the number as a row of bits: x & (x - 1) clears the lowest set bit, x ^ y shows where bits differ, and x & -x isolates the lowest set bit.

THE 2 PROBLEMS, EASY TO HARD | ★ = TOP 0: DO THESE FIRST

1	Power of Two	Easy	LC 231	2120
2	Hamming Distance	Easy	LC 461	2122

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Single Number (Part 1, pattern 27, page 1459)
Set Mismatch (Part 1, pattern 33, page 1762)
Counting Bits (Part 1, pattern 27, page 1463)
Bitwise AND of Numbers Range (Part 1, pattern 27, page 1485)
Single Number III (Part 1, pattern 27, page 1480)
Missing Number (Part 1, pattern 27, page 1465)
Number of 1 Bits (Part 1, pattern 27, page 1461)
Reverse Bits (Part 1, pattern 27, page 1468)
Single Number II (Part 1, pattern 27, page 1476)
Sum of Two Integers (Part 1, pattern 27, page 1483)

PART 2 · BIT MANIPULATION · PROBLEM 1 OF 2

231. Power of Two

Easy · Bit Manipulation, Recursion, Math

leetcode.com/problems/power-of-two

PROBLEM

Given an integer `n`, return `true` *if it is a power of two. Otherwise, return* `false`.

An integer `n` is a power of two, if there exists an integer `x` such that $n == 2^x$.

Example 1:

```
Input: n = 1
Output: true
Explanation: 20 = 1
```

Example 2:

```
Input: n = 16
Output: true
Explanation: 24 = 16
```

Example 3:

```
Input: n = 3
Output: false
```

Constraints:

- $-2^{31} <= n <= 2^{31} - 1$

Follow up:

Could you solve it without loops/recursion?

SOLUTIONS (PYTHON 3)

Solution 1: Bit Manipulation

> **How to think about it**
>
> Repeated division by 2 works, but one bit test is enough. A power of two has a single 1 bit, i.e. n>0 and n&(n-1)=0.

According to the properties of bit manipulation, executing n&(n-1) can eliminate the last bit 1 in the binary form of n. Therefore, if n > 0 and n&(n-1) results in 0, then n is a power of 2.

The time complexity is O(1), and the space complexity is O(1).

Python 3

```python
class Solution:
    def isPowerOfTwo(self, n: int) -> bool:
        return n > 0 and (n & (n - 1)) == 0
```

Solution 2: Lowbit

> **How to think about it**
>
> Clearing the lowest set bit is equivalent to taking lowbit. If n>0 and n=n&(-n), that single bit is the whole number.

According to the definition of lowbit, we know that lowbit(x) = x & (-x), which can get the decimal number represented by the last bit 1 of n. Therefore, if n > 0 and lowbit(n) equals n, then n is a power of 2.

The time complexity is O(1), and the space complexity is O(1).

Python 3

```python
class Solution:
    def isPowerOfTwo(self, n: int) -> bool:
        return n > 0 and n == n & (-n)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Power of four
# 2. The smallest power of two that is at least n


# ============================================================
# FOLLOW-UP 1: Power of four
# ============================================================

# ---- follow-up 1: function ----
# Answer: It must be a power of two, and its single set
# bit must sit at an even position (mask 0x55555555).
# Time: O(1).  Space: O(1).
def is_power_of_four(n):
    return n > 0 and n & (n - 1) == 0 and n & 0x55555555 != 0


# ============================================================
# FOLLOW-UP 2: The smallest power of two that is at least
# n
# ============================================================

# ---- follow-up 2: function ----
# Answer: Spread the highest bit of n - 1 to all lower
# bits and add one.
# Time: O(1).  Space: O(1).
def next_power_of_two(n):
    if n <= 1:
        return 1
    n -= 1
    n |= n >> 1
    n |= n >> 2
    n |= n >> 4
    n |= n >> 8
    n |= n >> 16
    n |= n >> 32
    return n + 1
```

PART 2 · BIT MANIPULATION · PROBLEM 2 OF 2

461. Hamming Distance

Easy · Bit Manipulation

leetcode.com/problems/hamming-distance

PROBLEM

The Hamming distance between two integers is the number of positions at which the corresponding bits are different.

Given two integers `x` and `y`, return *the **Hamming distance** between them*.

Example 1:

```
Input: x = 1, y = 4
Output: 2
Explanation:
1 (0 0 0 1)
4 (0 1 0 0)
       ↑   ↑
The above arrows point to positions where the corresponding bits are different.
```

Example 2:

```
Input: x = 3, y = 1
Output: 1
```

Constraints:

- $0 <= x, y <= 2^{31} - 1$

Note: This question is the same as 2220: Minimum Bit Flips to Convert Number.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Hamming distance is the number of differing bits. Comparing bit by bit needs a 32-step loop. The 1-bits of xxor y are exactly those positions; bit_count returns their number. XOR folds the comparison into one word operation; popcount is then a hardware instruction or a short loop.

Python 3

```python
class Solution:
    def hammingDistance(self, x: int, y: int) -> int:
        return (x ^ y).bit_count()
```

Complexity. Time: O(1). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Hamming distance of two strings of equal length
# 2. All numbers (within the given number of bits) at
# Hamming distance 1 from x


# ============================================================
# FOLLOW-UP 1: Hamming distance of two strings of equal
# length
# ============================================================
```

```
# ---- follow-up 1: function ----
# Answer: Count the positions where the characters differ.
# Time: O(n).  Space: O(1).
def string_hamming(a, b):
    return sum(1 for x, y in zip(a, b) if x != y)


# ============================================================
# FOLLOW-UP 2: All numbers (within the given number of
# bits) at Hamming distance 1 from x
# ============================================================

# ---- follow-up 2: function ----
# Answer: Flip each bit in turn.
# Time: O(bits).  Space: O(bits).
def neighbours_at_distance_one(x, bits):
    return [x ^ (1 << i) for i in range(bits)]
```

PART 2 · TOPIC 4 OF 50

Hash Tables

8 problems · 4 Easy · 4 Medium · 0 Hard

THE IDEA

Store what you have already seen (a count, an index, a group key) so every later question is one lookup instead of another scan.

THE 8 PROBLEMS, EASY TO HARD | ★ = TOP 4: DO THESE FIRST

1	Contains Duplicate II	Easy	LC 219	2125
2	Intersection of Two Arrays II	Easy	LC 350	2127
3	First Unique Character in a String	Easy	LC 387	2129
4	Maximum Number of Balloons	Easy	LC 1189	2131
5	★ **Encode and Decode TinyURL**	Medium	LC 535	2133
6	★ **Split Array into Consecutive Subsequences**	Medium	LC 659	2136
7	★ **Number of Good Ways to Split a String**	Medium	LC 1525	2139
8	★ **Minimum Deletions to Make Character Frequencies Unique**	Medium	LC 1647	2142

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Design HashMap (Part 1, pattern 29, page 1545)
Word Pattern (Part 1, pattern 1, page 20)
Number of Good Pairs (Part 1, pattern 33, page 1769)
Ransom Note (Part 1, pattern 1, page 23)
Reorganize String (Part 1, pattern 6, page 335)
Number of Matching Subsequences (Part 1, pattern 7, page 357)
Contains Duplicate (Part 1, pattern 1, page 11)
Find All Numbers Disappeared in an Array (Part 1, pattern 33, page 1760)
Isomorphic Strings (Part 1, pattern 1, page 17)
Group Anagrams (Part 1, pattern 1, page 31)
Longest Consecutive Sequence (Part 1, pattern 1, page 37)
Group Shifted Strings (Part 1, pattern 6, page 333)

219. Contains Duplicate II

Easy · Array, Hash Table, Sliding Window

leetcode.com/problems/contains-duplicate-ii

PROBLEM

Given an integer array `nums` and an integer `k`, return `true` *if there are two* ***distinct indices*** `i` *and* `j` *in the array such that* `nums[i] == nums[j]` ***and*** `abs(i - j) <= k`.

Example 1:

```
Input: nums = [1,2,3,1], k = 3
Output: true
```

Example 2:

```
Input: nums = [1,0,1,1], k = 1
Output: true
```

Example 3:

```
Input: nums = [1,2,3,1,2,3], k = 2
Output: false
```

Constraints:

- $1 <= nums.length <= 10^5$
- $-10^9 <= nums[i] <= 10^9$
- $0 <= k <= 10^5$

SOLUTION (PYTHON 3)

Solution 1: Hash Table

> **How to think about it**
>
> We need equal values whose indices differ by at most k. Sorting would scramble indices. A hash map stores the last index of each value; on a repeat we only check the gap.

We use a hash table d to store the recently traversed numbers and their corresponding indices.

Traverse the array nums. For the current element nums[i], if it exists in the hash table and the difference between the indices is no more than k, return true. Otherwise, add the current element to the hash table.

After traversing, return false.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def containsNearbyDuplicate(self, nums: List[int], k: int) -> bool:
        d = {}
        for i, x in enumerate(nums):
            if x in d and i - d[x] <= k:
                return True
            d[x] = i
        return False
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the closest pair of equal numbers (as indices)
# 2. Count the pairs of equal numbers that are at most k
# apart


# ============================================================
# FOLLOW-UP 1: Return the closest pair of equal numbers
# (as indices)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember the last index of each value and keep
# the smallest gap.
# Time: O(n).  Space: O(n).
def closest_equal_pair(nums):
    last = {}
    best = None
    for i, x in enumerate(nums):
        if x in last and (best is None or i - last[x] < best[1] - best[0]):
            best = (last[x], i)
        last[x] = i
    return best


# ============================================================
# FOLLOW-UP 2: Count the pairs of equal numbers that are
# at most k apart
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep counts for the last k numbers in a sliding
# window.
# Time: O(n).  Space: O(k).
def count_close_equal_pairs(nums, k):
    window = {}
    total = 0
    for i, x in enumerate(nums):
        total += window.get(x, 0)
        window[x] = window.get(x, 0) + 1
        if i >= k:
            old = nums[i - k]
            window[old] -= 1
    return total
```

350. Intersection of Two Arrays II

Easy · Array, Hash Table, Two Pointers, Binary Search, Sorting

leetcode.com/problems/intersection-of-two-arrays-ii

PROBLEM

Given two integer arrays `nums1` and `nums2`, return *an array of their intersection*. Each element in the result must appear as many times as it shows in both arrays and you may return the result in **any order**.

Example 1:

```
Input: nums1 = [1,2,2,1], nums2 = [2,2]
Output: [2,2]
```

Example 2:

```
Input: nums1 = [4,9,5], nums2 = [9,4,9,8,4]
Output: [4,9]
Explanation: [9,4] is also accepted.
```

Constraints:

- `1 <= nums1.length, nums2.length <= 1000`
- `0 <= nums1[i], nums2[i] <= 1000`

Follow up:

- What if the given array is already sorted? How would you optimize your algorithm?
- What if `nums1`'s size is small compared to `nums2`'s size? Which algorithm is better?
- What if elements of `nums2` are stored on disk, and the memory is limited such that you cannot load all elements into the memory at once?

SOLUTION (PYTHON 3)

Solution 1: Hash Table

> **How to think about it**
>
> The intersection must keep multiplicities, the min of the two counts. Unlike 349, we cannot unique the values. Count nums1, then scan nums2: while a remainder exists, emit it and decrement. The length is the sum of those minima.

We can use a hash table cnt to count the occurrences of each element in the array nums1. Then, we iterate through the array nums2. If an element x is in cnt and the occurrence of x is greater than 0, we add x to the answer and then decrement the occurrence of x by one.

After the iteration is finished, we return the answer array.

The time complexity is O(m + n), and the space complexity is O(m). Here, m and n are the lengths of the arrays nums1 and nums2, respectively.

Python 3

```python
class Solution:
    def intersect(self, nums1: List[int], nums2: List[int]) -> List[int]:
        cnt = Counter(nums1)
        ans = []
        for x in nums2:
            if cnt[x]:
                ans.append(x)
                cnt[x] -= 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Both arrays are sorted: use no extra memory
# 2. The intersection of k arrays (with multiplicity)


# ============================================================
# FOLLOW-UP 1: Both arrays are sorted: use no extra memory
# ============================================================

# ---- follow-up 1: function ----
# Answer: Two pointers: advance the smaller one, collect
# equal numbers.
# Time: O(a + b).  Space: O(1) extra besides the result.
def intersect_sorted(a, b):
    i = j = 0
    result = []
    while i < len(a) and j < len(b):
        if a[i] < b[j]:
            i += 1
        elif a[i] > b[j]:
            j += 1
        else:
            result.append(a[i])
            i += 1
            j += 1
    return result


# ============================================================
# FOLLOW-UP 2: The intersection of k arrays (with
# multiplicity)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the minimum count of every number across
# the arrays.
# Time: O(total + m log m).  Space: O(distinct).
from collections import Counter


def intersect_many(arrays):
    common = Counter(arrays[0])
    for arr in arrays[1:]:
        common &= Counter(arr)
    return sorted(common.elements())
```

387. First Unique Character in a String

Easy · Queue, Hash Table, String, Counting

leetcode.com/problems/first-unique-character-in-a-string

PROBLEM

Given a string `s`, find the **first** non-repeating character in it and return its index. If it **does not** exist, return `-1`.

Example 1:

Input: s = "leetcode"

Output: 0

Explanation:

The character `'l'` at index 0 is the first character that does not occur at any other index.

Example 2:

Input: s = "loveleetcode"

Output: 2

Example 3:

Input: s = "aabb"

Output: -1

Constraints:

- `1 <= s.length <= 10^5`
- `s` consists of only lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Counting

> **How to think about it**
>
> Index of the first unique character. One left-to-right pass cannot know about later repeats. Count, then scan for frequency 1; else -1. Two linear passes, alphabet space.

We use a hash table or an array of length 26 cnt to store the frequency of each character. Then, we traverse each character s[i] from the beginning. If cnt[s[i]] is 1, we return i.

If no such character is found after the traversal, we return -1.

The time complexity is O(n), where n is the length of the string. The space complexity is O(|Σ|), where Σ is the character set. In this problem, the character set consists of lowercase letters, so |Σ|=26.

Python 3

```python
class Solution:
    def firstUniqChar(self, s: str) -> int:
        cnt = Counter(s)
        for i, c in enumerate(s):
            if cnt[c] == 1:
                return i
        return -1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Characters arrive one at a time: the first unique
# character at any moment
# 2. All indices of unique characters


# ============================================================
# FOLLOW-UP 1: Characters arrive one at a time: the first
# unique character at any moment
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep counts and a queue of candidates; drop
# repeated ones from the front.
# Time: add O(1), first_unique O(1) amortised.  Space:
# O(n).
from collections import deque


class FirstUniqueStream:
    def __init__(self):
        self.count = {}
        self.queue = deque()

    def add(self, ch):
        self.count[ch] = self.count.get(ch, 0) + 1
        if self.count[ch] == 1:
            self.queue.append(ch)

    def first_unique(self):
        while self.queue and self.count[self.queue[0]] > 1:
            self.queue.popleft()
        return self.queue[0] if self.queue else None


# ============================================================
# FOLLOW-UP 2: All indices of unique characters
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the letters, then list the positions of
# those with count one.
# Time: O(n).  Space: O(1) alphabet.
def all_unique_indices(s):
    count = {}
    for ch in s:
        count[ch] = count.get(ch, 0) + 1
    return [i for i, ch in enumerate(s) if count[ch] == 1]
```

1189. Maximum Number of Balloons

Easy · Hash Table, String, Counting

leetcode.com/problems/maximum-number-of-balloons

PROBLEM

Given a string `text`, you want to use the characters of `text` to form as many instances of the word **"balloon"** as possible.

You can use each character in `text` **at most once**. Return the maximum number of instances that can be formed.

Example 1:

```
Input: text = "nlaebolko"
Output: 1
```

Example 2:

```
Input: text = "loonbalxballpoon"
Output: 2
```

Example 3:

```
Input: text = "leetcode"
Output: 0
```

Constraints:

- `1 <= text.length <= 10^4`
- `text` consists of lower case English letters only.

Note: This question is the same as 2287: Rearrange Characters to Make Target String.

SOLUTION (PYTHON 3)

Solution 1: Counting

> **How to think about it**
>
> `balloon` needs b,a, two l, two o, and n. After counting `text`, halve l and o and take the min over b,a,l,o,n. There is no need to delete letters from the string repeatedly.

We count the frequency of each letter in the string `text`, and then divide the frequency of the letters 'o' and 'l' by 2, because the word `balloon` contains the letters 'o' and 'l' twice.

Next, we traverse each letter in the word `balon`, and find the minimum frequency of each letter in the string `text`. This minimum frequency is the maximum number of times the word `balloon` can appear in the string `text`.

The time complexity is O(n), and the space complexity is O(C). Here, n is the length of the string `text`, and C is the size of the character set. In this problem, C = 26.

Python 3

```python
class Solution:
    def maxNumberOfBalloons(self, text: str) -> int:
        cnt = Counter(text)
        cnt['o'] >>= 1
        cnt['l'] >>= 1
        return min(cnt[c] for c in 'balon')
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Any word instead of 'balloon'
# 2. The letters that are left after making the maximum
# number of copies


# ============================================================
# FOLLOW-UP 1: Any word instead of 'balloon'
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count the letters needed per copy and divide.
# Time: O(t + w).  Space: O(1) alphabet.
def copies_of_word(text, word):
    have = {}
    for ch in text:
        have[ch] = have.get(ch, 0) + 1
    need = {}
    for ch in word:
        need[ch] = need.get(ch, 0) + 1
    return min(have.get(ch, 0) // c for ch, c in need.items())


# ============================================================
# FOLLOW-UP 2: The letters that are left after making the
# maximum number of copies
# ============================================================

# ---- follow-up 2: function ----
# Answer: Subtract the used letters.
# Time: O(t + w).  Space: O(1) alphabet.
def leftover_letters(text, word):
    copies = copies_of_word(text, word)
    left = {}
    for ch in text:
        left[ch] = left.get(ch, 0) + 1
    for ch in word:
        left[ch] = left.get(ch, 0) - copies
    return "".join(sorted(ch * c for ch, c in left.items() if c > 0))
```

535. Encode and Decode TinyURL

Medium · Design, Hash Table, String, Hash Function

leetcode.com/problems/encode-and-decode-tinyurl

★ **TOP 7 PROBLEM** in Hash Tables: master this one first

PROBLEM

Note: This is a companion problem to the

System Design

problem:

Design TinyURL

.

TinyURL is a URL shortening service where you enter a URL such as `https://leetcode.com/problems/design-tinyurl` and it returns a short URL such as `http://tinyurl.com/4e9iAk`. Design a class to encode a URL and decode a tiny URL.

There is no restriction on how your encode/decode algorithm should work. You just need to ensure that a URL can be encoded to a tiny URL and the tiny URL can be decoded to the original URL.

Implement the `Solution` class:

- `Solution()` Initializes the object of the system.
- `String encode(String longUrl)` Returns a tiny URL for the given `longUrl`.
- `String decode(String shortUrl)` Returns the original long URL for the given `shortUrl`. It is guaranteed that the given `shortUrl` was encoded by the same object.

Example 1:

```
Input: url = "https://leetcode.com/problems/design-tinyurl"
Output: "https://leetcode.com/problems/design-tinyurl"

Explanation:
Solution obj = new Solution();
string tiny = obj.encode(url); // returns the encoded tiny url.
string ans = obj.decode(tiny); // returns the original url after decoding it.
```

Constraints:

- `1 <= url.length <= 10^4`
- `url` is guranteed to be a valid URL.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

We need a reversible short URL. Hashing to a fixed string can collide and needs a conflict table. The short code need not be unpredictable, so a monotonic id suffices. A map stores `id -> long URL`; the short form is the domain plus that id, and decode reads the last path segment. Both operations are constant time.

Python 3

```
class Codec:
    def __init__(self):
        self.m = defaultdict()
        self.idx = 0
        self.domain = 'https://tinyurl.com/'

    def encode(self, longUrl: str) -> str:
        """Encodes a URL to a shortened URL."""
        self.idx += 1
        self.m[str(self.idx)] = longUrl
        return f'{self.domain}{self.idx}'

    def decode(self, shortUrl: str) -> str:
        """Decodes a shortened URL to its original URL."""
        idx = shortUrl.split('/')[-1]
        return self.m[idx]


# Your Codec object will be instantiated and called as such:
# codec = Codec()
# codec.decode(codec.encode(url))
```

Complexity. Time: O(1) per call. Space: O(n), n = stored urls.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Short codes from a counter in base 62 (no random
# collisions)
# 2. The user may ask for a custom alias


# ============================================================
# FOLLOW-UP 1: Short codes from a counter in base 62 (no
# random collisions)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Number the URLs and write the number with 62
# digits.
# Time: encode O(log n), decode O(1).  Space: O(n).
class CodecCounter:
    ALPHABET = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"

    def __init__(self):
        self.count = 0
        self.table = {}

    def encode(self, long_url):
        self.count += 1
        n = self.count
        code = ""
        while n > 0:
            n, r = divmod(n, 62)
            code = self.ALPHABET[r] + code
        self.table[code] = long_url
        return "http://tiny.url/" + code

    def decode(self, short_url):
        return self.table[short_url.rsplit("/", 1)[1]]


# ============================================================
# FOLLOW-UP 2: The user may ask for a custom alias
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Accept the alias when it is free; otherwise
# refuse.
# Time: O(1) on average, O(n) if aliases collide.  Space:
# O(n).
class CodecWithAlias:
    def __init__(self):
        self.table = {}

    def encode(self, long_url, alias=None):
        if alias is None:
            alias = str(len(self.table) + 1)
            while alias in self.table:
                alias += "x"
        if alias in self.table and self.table[alias] != long_url:
            return None
        self.table[alias] = long_url
        return "http://tiny.url/" + alias

    def decode(self, short_url):
```

```
        return self.table.get(short_url.rsplit("/", 1)[1])
```

659. Split Array into Consecutive Subsequences

Medium · Greedy, Array, Hash Table, Heap (Priority Queue)

leetcode.com/problems/split-array-into-consecutive-subsequences

★ **TOP 7 PROBLEM** in Hash Tables: master this one first

PROBLEM

You are given an integer array `nums` that is **sorted in non-decreasing order**.

Determine if it is possible to split `nums` into **one or more subsequences** such that **both** of the following conditions are true:

- Each subsequence is a **consecutive increasing sequence** (i.e. each integer is **exactly one** more than the previous integer).
- All subsequences have a length of `3` **or more**.

Return `true` *if you can split* `nums` *according to the above conditions, or* `false` *otherwise*.

A **subsequence** of an array is a new array that is formed from the original array by deleting some (can be none) of the elements without disturbing the relative positions of the remaining elements. (i.e., `[1,3,5]` is a subsequence of `[1,2,3,4,5]` while `[1,3,2]` is not).

Example 1:

```
Input: nums = [1,2,3,3,4,5]
Output: true
Explanation: nums can be split into the following subsequences:
[1,2,3,3,4,5] --> 1, 2, 3
[1,2,3,3,4,5] --> 3, 4, 5
```

Example 2:

```
Input: nums = [1,2,3,3,4,4,5,5]
Output: true
Explanation: nums can be split into the following subsequences:
[1,2,3,3,4,4,5,5] --> 1, 2, 3, 4, 5
[1,2,3,3,4,4,5,5] --> 3, 4, 5
```

Example 3:

```
Input: nums = [1,2,3,4,4,5]
Output: false
Explanation: It is impossible to split nums into consecutive increasing subsequences of length 3 or
more.
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-1000 <= nums[i] <= 1000`
- `nums` is sorted in **non-decreasing** order.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Split the array into consecutive increasing subsequences of length at least 3. Always starting a new sequence can leave short ones stranded. A hash map stores min-heaps of lengths ending at each value. Append v to the shortest sequence ending at v-1, or start a new one. Every heap minimum must be at least 3.

Python 3

```
class Solution:
    def isPossible(self, nums: List[int]) -> bool:
        d = defaultdict(list)
        for v in nums:
            if h := d[v - 1]:
                heappush(d[v], heappop(h) + 1)
            else:
                heappush(d[v], 1)
        return all(not v or v and v[0] > 2 for v in d.values())
```

Complexity. Time: O(n log n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Every subsequence must have length at least k (return
# True or False)
# 2. Return the subsequences themselves


# ========================================================
# FOLLOW-UP 1: Every subsequence must have length at least
# k (return True or False)
# ========================================================

# ---- follow-up 1: function ----
# Answer: Greedy: add each number to the shortest chain
# that ends just before it, or start a new chain.
# Time: O(n log n).  Space: O(n).
import heapq


def can_split_with_min_length(nums, k):
    ends = {}                                   # last number -> heap of chain lengths
    for x in nums:
        chains = ends.get(x - 1)
        if chains:
            length = heapq.heappop(chains)
            heapq.heappush(ends.setdefault(x, []), length + 1)
        else:
            heapq.heappush(ends.setdefault(x, []), 1)
    for chains in ends.values():
        if chains and chains[0] < k:
            return False
    return True


# ========================================================
# FOLLOW-UP 2: Return the subsequences themselves
# ========================================================

# ---- follow-up 2: function ----
# Answer: Same greedy, storing the chains as lists.
# Time: O(n^2 log n) in the worst case (the chains are
# sorted every time).  Space: O(n).
def split_into_chains(nums, k):
    ends = {}                                   # last number -> list of chains ending there
    for x in nums:
        candidates = ends.get(x - 1)
        if candidates:
            candidates.sort(key=len)
            chain = candidates.pop(0)
        else:
            chain = []
        chain.append(x)
        ends.setdefault(x, []).append(chain)
```

```
    chains = [c for group in ends.values() for c in group]
    if any(len(c) < k for c in chains):
        return None
    return chains
```

1525. Number of Good Ways to Split a String

Medium · Bit Manipulation, Hash Table, String, Dynamic Programming, Prefix Sum

leetcode.com/problems/number-of-good-ways-to-split-a-string

★ **TOP 7 PROBLEM** in Hash Tables: master this one first

PROBLEM

You are given a string `s`.

A split is called **good** if you can split `s` into two non-empty strings s_{left} and s_{right} where their concatenation is equal to `s` (i.e., $s_{left} + s_{right} = s$) and the number of distinct letters in s_{left} and s_{right} is the same.

Return *the number of **good splits** you can make in* `s`.

Example 1:

```
Input: s = "aacaba"
Output: 2
Explanation: There are 5 ways to split "aacaba" and 2 of them are good.
("a", "acaba") Left string and right string contains 1 and 3 different letters respectively.
("aa", "caba") Left string and right string contains 1 and 3 different letters respectively.
("aac", "aba") Left string and right string contains 2 and 2 different letters respectively (good
split).
("aaca", "ba") Left string and right string contains 2 and 2 different letters respectively (good
split).
("aacab", "a") Left string and right string contains 3 and 1 different letters respectively.
```

Example 2:

```
Input: s = "abcd"
Output: 1
Explanation: Split the string as follows ("ab", "cd").
```

Constraints:

- `1 <= s.length <= 10^5`
- `s` consists of only lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Count splits where the two sides have equally many distinct characters. n≤ 10^5, so rescanning both sides at every cut is quadratic. The right-hand alphabet starts as a global frequency map; the left-hand set only grows. Moving the cut to the right inserts the current character on the left and decrements it on the right, dropping the key when the count hits zero. Whenever the two maps have the same size, the split is good. One pass suffices.

Python 3

```
class Solution:
    def numSplits(self, s: str) -> int:
        cnt = Counter(s)
        vis = set()
        ans = 0
        for c in s:
            vis.add(c)
            cnt[c] -= 1
            if cnt[c] == 0:
                cnt.pop(c)
            ans += len(vis) == len(cnt)
        return ans
```

Complexity. Time: O(n). Space: O(C), C = 26 letters.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the split positions
# 2. Splits where the left part has MORE distinct letters
# than the right


# ============================================================
# FOLLOW-UP 1: Return the split positions
# ============================================================

# ---- follow-up 1: function ----
# Answer: Prefix and suffix distinct counts; a position is
# good when they are equal.
# Time: O(n).  Space: O(1) alphabet.
def good_split_positions(s):
    right = {}
    for ch in s:
        right[ch] = right.get(ch, 0) + 1
    left = set()
    positions = []
    for i in range(len(s) - 1):
        left.add(s[i])
        right[s[i]] -= 1
        if right[s[i]] == 0:
            del right[s[i]]
        if len(left) == len(right):
            positions.append(i + 1)
    return positions


# ============================================================
# FOLLOW-UP 2: Splits where the left part has MORE
# distinct letters than the right
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same sweep with a different comparison.
# Time: O(n).  Space: O(1) alphabet.
def count_left_has_more_distinct(s):
    right = {}
    for ch in s:
        right[ch] = right.get(ch, 0) + 1
    left = set()
    total = 0
    for i in range(len(s) - 1):
        left.add(s[i])
        right[s[i]] -= 1
        if right[s[i]] == 0:
            del right[s[i]]
```

```
        if len(left) > len(right):
            total += 1
    return total
```

1647. Minimum Deletions to Make Character Frequencies Unique

Medium · Greedy, Hash Table, String, Sorting

leetcode.com/problems/minimum-deletions-to-make-character-frequencies-unique

★ **TOP 7 PROBLEM** in Hash Tables: master this one first

PROBLEM

A string `s` is called **good** if there are no two different characters in `s` that have the same **frequency**.

Given a string `s`, return *the **minimum** number of characters you need to delete to make* `s` ***good***.

The **frequency** of a character in a string is the number of times it appears in the string. For example, in the string `"aab"`, the **frequency** of `'a'` is `2`, while the **frequency** of `'b'` is `1`.

Example 1:

```
Input: s = "aab"
Output: 0
Explanation: s is already good.
```

Example 2:

```
Input: s = "aaabbbcc"
Output: 2
Explanation: You can delete two 'b's resulting in the good string "aaabcc".
Another way it to delete one 'b' and one 'c' resulting in the good string "aaabbc".
```

Example 3:

```
Input: s = "ceabaacb"
Output: 2
Explanation: You can delete both 'c's resulting in the good string "eabaab".
Note that we only care about characters that are still in the string at the end (i.e. frequency of 0 is ignored).
```

Constraints:

- `1 <= s.length <= 10^5`
- `s` contains only lowercase English letters.

SOLUTIONS (PYTHON 3)

Solution 1: Array + Sorting

How to think about it

Frequencies must become unique and we may only delete characters. With 26 letters, sort frequencies decreasingly so each value occupies at most one integer slot. Keep the next free upper bound pre. If v ≥ pre, delete down to pre-1 (or delete all once pre is 0). Otherwise keep v and set pre=v.

First, we use an array cnt of length 26 to count the occurrences of each letter in the string s.

Then, we sort the array cnt in descending order. We define a variable pre to record the current number of occurrences of the letter.

Next, we traverse each element v in the array cnt. If the current pre is 0, we directly add v to the answer. Otherwise, if v ≥ pre, we add v - pre + 1 to the answer and decrement pre by 1. Otherwise, we directly update pre to v. Then, we continue to the next element.

After traversing, we return the answer.

The time complexity is O(n + |Σ| × log |Σ|), and the space complexity is O(|Σ|). Here, n is the length of the string s, and |Σ| is the size of the alphabet. In this problem, |Σ| = 26.

Python 3

```python
class Solution:
    def minDeletions(self, s: str) -> int:
        cnt = Counter(s)
        ans, pre = 0, inf
        for v in sorted(cnt.values(), reverse=True):
            if pre == 0:
                ans += v
            elif v >= pre:
                ans += v - pre + 1
                pre -= 1
            else:
                pre = v
        return ans
```

Solution 2: Greedy (Adjacent Decrement)

> **How to think about it**
>
> Solution 1 enforces a global upper bound. Equivalently, after sorting, force adjacent frequencies to decrease: while the next is not smaller, decrement it and count deletions. The implementation matches “neighbors differ” and has the same complexity.

Count frequencies and sort them in descending order. Walking adjacent frequencies, decrement the current one until it is strictly smaller than the previous.

The time complexity is O(n + |Σ| × log |Σ| + n), and the space complexity is O(|Σ|), where n is the length of s.

Python 3

```python
class Solution:
    def minDeletions(self, s: str) -> int:
        cnt = Counter(s)
        vals = sorted(cnt.values(), reverse=True)
        ans = 0
        for i in range(1, len(vals)):
            while vals[i] >= vals[i - 1] and vals[i] > 0:
                vals[i] -= 1
                ans += 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the frequencies after the deletions
# 2. Make all remaining letters have the SAME frequency
# (delete the fewest)


# ============================================================
# FOLLOW-UP 1: Return the frequencies after the deletions
# ============================================================

# ---- follow-up 1: function ----
# Answer: Process the frequencies from big to small,
# lowering each to the largest free value below it.
# Time: O(26 * n).  Space: O(26).
def frequencies_after_deletions(s):
    counts = sorted((s.count(ch) for ch in set(s)), reverse=True)
    result = []
```

```
    limit = len(s)
    for c in counts:
        f = min(c, limit)
        result.append(f)
        limit = max(f - 1, 0)
    return result


# ============================================================
# FOLLOW-UP 2: Make all remaining letters have the SAME
# frequency (delete the fewest)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Try every target frequency f: every letter with
# count >= f is cut down to f, the others are removed.
# Time: O(26 * n).  Space: O(26).
def min_deletions_equal_frequencies(s):
    counts = [s.count(ch) for ch in set(s)]
    best = len(s)
    for f in range(1, max(counts) + 1):
        kept = sum(f for c in counts if c >= f)
        best = min(best, len(s) - kept)
    return best
```

PART 2 · TOPIC 5 OF 50

Two Pointers

5 problems · 3 Easy · 2 Medium · 0 Hard

THE IDEA

Two indexes walking toward each other or in the same direction replace a nested loop. Sort first when the order does not matter.

THE 5 PROBLEMS, EASY TO HARD | ★ = TOP 2: DO THESE FIRST

1	Valid Word Abbreviation	Easy	LC 408	2146
2	Count Binary Substrings	Easy	LC 696	2149
3	Merge Strings Alternately	Easy	LC 1768	2152
4	★ **3Sum**	Medium	LC 15	2154
5	★ **String Compression**	Medium	LC 443	2157

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Merge Sorted Array (Part 1, pattern 23, page 1230)
Two Sum (Part 1, pattern 1, page 8)
Two Sum II - Input Array Is Sorted (Part 1, pattern 3, page 150)
4Sum (Part 1, pattern 3, page 161)
Longest Palindromic Substring (Part 1, pattern 6, page 327)
Count Subarrays With Fixed Bounds (Part 1, pattern 35, page 1851)
Squares of a Sorted Array (Part 1, pattern 3, page 138)
Backspace String Compare (Part 1, pattern 3, page 147)
Container With Most Water (Part 1, pattern 3, page 166)
Boats to Save People (Part 1, pattern 3, page 172)
Trapping Rain Water (Part 1, pattern 3, page 177)

408. Valid Word Abbreviation

Easy · Two Pointers, String

leetcode.com/problems/valid-word-abbreviation

PROBLEM

A string can be **abbreviated** by replacing any number of **non-adjacent**, **non-empty** substrings with their lengths. The lengths **should not** have leading zeros.

For example, a string such as `"substitution"` could be abbreviated as (but not limited to):

- `"s10n"` (`"s ubstitutio n"`)
- `"sub4u4"` (`"sub stit u tion"`)
- `"12"` (`"substitution"`)
- `"su3i1u2on"` (`"su bst i t u ti on"`)
- `"substitution"` (no substrings replaced)

The following are **not valid** abbreviations:

- `"s55n"` (`"s ubsti tutio n"`, the replaced substrings are adjacent)
- `"s010n"` (has leading zeros)
- `"s0ubstitution"` (replaces an empty substring)

Given a string `word` and an abbreviation `abbr`, return *whether the string **matches** the given abbreviation*.

A **substring** is a contiguous **non-empty** sequence of characters within a string.

Example 1:

```
Input: word = "internationalization", abbr = "i12iz4n"
Output: true
Explanation: The word "internationalization" can be abbreviated as "i12iz4n" ("i nternational iz atio
n").
```

Example 2:

```
Input: word = "apple", abbr = "a2e"
Output: false
Explanation: The word "apple" cannot be abbreviated as "a2e".
```

Constraints:

- `1 <= word.length <= 20`
- `word` consists of only lowercase English letters.
- `1 <= abbr.length <= 10`
- `abbr` consists of lowercase English letters and digits.
- All the integers in `abbr` will fit in a 32-bit integer.

SOLUTION (PYTHON 3)

Solution 1: Simulation

> **How to think about it**
>
> An abbreviation interleaves letters and skip lengths, and a number may not have a leading zero. Expanding the abbreviation into a new string is unnecessary and can be large. Two pointers scan abbr: digits accumulate a skip (rejecting a leading zero); a letter first applies the skip, then compares characters. At the end, the word pointer plus any leftover skip must land exactly on the last index. Skip and compare belong in one pass so overflow, mismatch, and unused length are checked together.

We can directly simulate character matching and replacement.

Assume the lengths of the string word and the string abbr are m and n respectively. We use two pointers i and j to point to the initial positions of the string word and the string abbr respectively, and use an integer variable x to record the current matched number in abbr.

Loop to match each character of the string word and the string abbr:

If the character abbr[j] pointed by the pointer j is a number, if abbr[j] is `'0'` and x is 0, it means that the number in abbr has leading zeros, so it is not a valid abbreviation, return `false`; otherwise, update x to x × 10 + abbr[j] - '0'.

If the character abbr[j] pointed by the pointer j is not a number, then we move the pointer i forward by x positions at this time, and then reset x to 0. If i ≥ m or word[i] ≠ abbr[j] at this time, it means that the two strings cannot match, return `false`; otherwise, move the pointer i forward by 1 position.

Then we move the pointer j forward by 1 position, repeat the above process, until i exceeds the length of the string word or j exceeds the length of the string abbr.

Finally, if i + x equals m and j equals n, it means that the string word can be abbreviated as the string abbr, return `true`; otherwise return `false`.

The time complexity is O(m + n), where m and n are the lengths of the string word and the string abbr respectively. The space complexity is O(1).

Python 3

```python
class Solution:
    def validWordAbbreviation(self, word: str, abbr: str) -> bool:
        m, n = len(word), len(abbr)
        i = j = x = 0
        while i < m and j < n:
            if abbr[j].isdigit():
                if abbr[j] == "0" and x == 0:
                    return False
                x = x * 10 + int(abbr[j])
            else:
                i += x
                x = 0
                if i >= m or word[i] != abbr[j]:
                    return False
                i += 1
            j += 1
        return i + x == m and j == n
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Which words of a list match an abbreviation
# 2. List all the valid abbreviations of a word


# ========================================================
# FOLLOW-UP 1: Which words of a list match an abbreviation
# ========================================================
```

```
# ---- follow-up 1: function ----
# Answer: Run the same two-pointer check against every
# word.
# Time: O(W * L), W words of length L.  Space: O(W) for
# the answer.
def matches_abbreviation(word, abbr):
    i = j = 0
    while i < len(word) and j < len(abbr):
        if abbr[j].isdigit():
            if abbr[j] == "0":
                return False
            number = 0
            while j < len(abbr) and abbr[j].isdigit():
                number = number * 10 + int(abbr[j])
                j += 1
            i += number
        else:
            if word[i] != abbr[j]:
                return False
            i += 1
            j += 1
    return i == len(word) and j == len(abbr)


def words_matching(words, abbr):
    return [w for w in words if matches_abbreviation(w, abbr)]


# ============================================================
# FOLLOW-UP 2: List all the valid abbreviations of a word
# ============================================================

# ---- follow-up 2: function ----
# Answer: Backtracking: for each position either keep the
# letter or start a run of skipped letters, counting the
# run.
# Time: O(2^n * n).  Space: O(2^n * n).
def all_abbreviations(word):
    result = []

    def go(i, current, run):
        if i == len(word):
            result.append(current + (str(run) if run else ""))
            return
        go(i + 1, current, run + 1)                                    # skip this letter
        go(i + 1, current + (str(run) if run else "") + word[i], 0)    # keep this letter

    go(0, "", 0)
    return result
```

PART 2 · TWO POINTERS · PROBLEM 2 OF 5

696. Count Binary Substrings

Easy · Two Pointers, String

leetcode.com/problems/count-binary-substrings

PROBLEM

Given a binary string `s`, return the number of non-empty substrings that have the same number of `0`'s and `1`'s, and all the `0`'s and all the `1`'s in these substrings are grouped consecutively.

Substrings that occur multiple times are counted the number of times they occur.

Example 1:

```
Input: s = "00110011"
Output: 6
Explanation: There are 6 substrings that have equal number of consecutive 1's and 0's: "0011", "01",
"1100", "10", "0011", and "01".
Notice that some of these substrings repeat and are counted the number of times they occur.
Also, "00110011" is not a valid substring because all the 0's (and 1's) are not grouped together.
```

Example 2:

```
Input: s = "10101"
Output: 4
Explanation: There are 4 substrings: "10", "01", "10", "01" that have equal number of consecutive 1's
and 0's.
```

Constraints:

- `1 <= s.length <= 10^5`
- `s[i]` is either `'0'` or `'1'`.

SOLUTION (PYTHON 3)

Solution 1: Iteration and Counting

> **How to think about it**
>
> Count substrings made of a run of 0s next to a run of 1s of equal length. Checking every substring is quadratic. Split into runs. Adjacent runs contribute min(pre,cur) valid substrings. One pass over the runs is enough.

We can iterate through the string s, using a variable pre to record the count of the previous consecutive characters, and another variable cur to record the count of the current consecutive characters. The number of valid substrings ending with the current character is min(pre, cur). We accumulate min(pre, cur) to the answer, assign the value of cur to pre, and continue iterating through string s until the end.

The time complexity is O(n), where n is the length of string s. The space complexity is O(1).

Python 3

```
class Solution:
    def countBinarySubstrings(self, s: str) -> int:
        n = len(s)
        ans = i = 0
        pre = 0
        while i < n:
            j = i + 1
            while j < n and s[j] == s[i]:
                j += 1
            cur = j - i
            ans += min(pre, cur)
            pre = cur
            i = j
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the substrings
# 2. Count ALL substrings with equally many 0s and 1s (the
# digits need not be grouped)


# ============================================================
# FOLLOW-UP 1: Return the substrings
# ============================================================

# ---- follow-up 1: function ----
# Answer: Walk over the groups of equal digits; for each
# pair of neighbouring groups, produce the matching
# substrings.
# Time: O(n^2) in the worst case (the output itself can be
# that large).  Space: O(n^2).
def binary_substrings(s):
    groups = []
    i = 0
    while i < len(s):
        j = i
        while j < len(s) and s[j] == s[i]:
            j += 1
        groups.append((i, j - i))
        i = j
    result = []
    for k in range(1, len(groups)):
        start1, length1 = groups[k - 1]
        start2, length2 = groups[k]
        for size in range(1, min(length1, length2) + 1):
            result.append(s[start2 - size:start2 + size])
    return result


# ============================================================
# FOLLOW-UP 2: Count ALL substrings with equally many 0s
# and 1s (the digits need not be grouped)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Treat 1 as +1 and 0 as -1; count pairs of equal
# prefix balances.
# Time: O(n).  Space: O(n).
def count_balanced_substrings(s):
    seen = {0: 1}
    balance = 0
    total = 0
    for ch in s:
```

```
        balance += 1 if ch == "1" else -1
        total += seen.get(balance, 0)
        seen[balance] = seen.get(balance, 0) + 1
    return total
```

1768. Merge Strings Alternately

Easy · Two Pointers, String

leetcode.com/problems/merge-strings-alternately

PROBLEM

You are given two strings `word1` and `word2`. Merge the strings by adding letters in alternating order, starting with `word1`. If a string is longer than the other, append the additional letters onto the end of the merged string.

Return *the merged string.*

Example 1:

```
Input: word1 = "abc", word2 = "pqr"
Output: "apbqcr"
Explanation: The merged string will be merged as so:
word1: a b c
word2: p q r
merged: a p b q c r
```

Example 2:

```
Input: word1 = "ab", word2 = "pqrs"
Output: "apbqrs"
Explanation: Notice that as word2 is longer, "rs" is appended to the end.
word1: a b
word2: p q r s
merged: a p b q r s
```

Example 3:

```
Input: word1 = "abcd", word2 = "pq"
Output: "apbqcd"
Explanation: Notice that as word1 is longer, "cd" is appended to the end.
word1: a b c d
word2: p q
merged: a p b q c d
```

Constraints:

- `1 <= word1.length, word2.length <= 100`
- `word1` and `word2` consist of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Direct Simulation

> **How to think about it**
>
> Merge by alternating characters and append the leftover of the longer string. One paired walk is enough. zip_longest yields corresponding characters (empty when missing); joining them is the merge.

We traverse the two strings `word1` and `word2`, take out the characters one by one, and append them to the result string. The Python code can be simplified into one line.

The time complexity is O(m + n), where m and n are the lengths of the two strings respectively. Ignoring the space consumption of the answer, the space complexity is O(1).

Python 3

```
class Solution:
    def mergeAlternately(self, word1: str, word2: str) -> str:
        return ''.join(a + b for a, b in zip_longest(word1, word2, fillvalue=''))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Merge k strings, one letter from each in turn
# 2. Take c letters from each string in turn


# ============================================================
# FOLLOW-UP 1: Merge k strings, one letter from each in
# turn
# ============================================================

# ---- follow-up 1: function ----
# Answer: Go through the positions; at every position take
# the letter of each string that is long enough.
# Time: O(total letters).  Space: O(total letters).
def merge_many(strings):
    longest = max(len(s) for s in strings)
    result = []
    for i in range(longest):
        for s in strings:
            if i < len(s):
                result.append(s[i])
    return "".join(result)


# ============================================================
# FOLLOW-UP 2: Take c letters from each string in turn
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same walk, in chunks of c letters.
# Time: O(n + m).  Space: O(n + m).
def merge_in_chunks(word1, word2, c):
    result = []
    i = j = 0
    while i < len(word1) or j < len(word2):
        result.append(word1[i:i + c])
        i += c
        result.append(word2[j:j + c])
        j += c
    return "".join(result)
```

PART 2 · TWO POINTERS · PROBLEM 4 OF 5

15. 3Sum

Medium · Array, Two Pointers, Sorting

leetcode.com/problems/3sum

★ **TOP 7 PROBLEM** in Two Pointers: master this one first

PROBLEM

Given an integer array nums, return all the triplets `[nums[i], nums[j], nums[k]]` such that `i != j`, `i != k`, and `j != k`, and `nums[i] + nums[j] + nums[k] == 0`.

Notice that the solution set must not contain duplicate triplets.

Example 1:

```
Input: nums = [-1,0,1,2,-1,-4]
Output: [[-1,-1,2],[-1,0,1]]
Explanation:
nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.
nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.
nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.
The distinct triplets are [-1,0,1] and [-1,-1,2].
Notice that the order of the output and the order of the triplets does not matter.
```

Example 2:

```
Input: nums = [0,1,1]
Output: []
Explanation: The only possible triplet does not sum up to 0.
```

Example 3:

```
Input: nums = [0,0,0]
Output: [[0,0,0]]
Explanation: The only possible triplet sums up to 0.
```

Constraints:

- `3 <= nums.length <= 3000`
- -10^5 `<= nums[i] <=` 10^5

SOLUTION (PYTHON 3)

Solution 1: Sort + Two Pointers

How to think about it

The first idea is three nested loops plus a set for uniqueness. Correct, but $O(n^3)$. n≤ 3000 will not pass. Hashing two-sum for each i reaches $O(n^2)$, yet duplicates and extra space are messy. The bottleneck is finding two numbers that sum to the opposite of a fixed value, without repeating triplets. Sort first so duplicates sit together and are easy to skip; two-sum on a sorted array is two pointers in O(n). If nums[i]>0, everything after it is positive, so the sum cannot be 0 anymore. So we sort, enumerate the first number, and squeeze the rest with two pointers.

We notice that the problem does not require us to return the triplet in order, so we might as well sort the array first, which makes it easy to skip duplicate elements.

Next, we enumerate the first element of the triplet nums[i], where 0 ≤ i < n - 2. For each i, we can find j and k satisfying nums[i] + nums[j] + nums[k] = 0 by maintaining two pointers j = i + 1 and k = n - 1. In the enumeration process, we need to skip duplicate elements to avoid duplicate triplets.

The specific judgment logic is as follows:

If i > 0 and nums[i] = nums[i - 1], it means that the element currently enumerated is the same as the previous element, we can skip it directly, because it will not produce new results.

If nums[i] > 0, it means that the element currently enumerated is greater than 0, so the sum of three numbers must not be equal to 0, and the enumeration ends.

Otherwise, we let the left pointer j = i + 1, and the right pointer k = n - 1. When j < k, the loop is executed, and the sum of three numbers x = nums[i] + nums[j] + nums[k] is calculated and compared with 0:

- If x < 0, it means that nums[j] is too small, we need to move j to the right.
- If x > 0, it means that nums[k] is too large, we need to move k to the left.
- Otherwise, it means that we have found a valid triplet, add it to the answer, move j to the right, move k to the left, and skip all duplicate elements to continue looking for the next valid triplet.

After the enumeration is over, we can get the answer to the triplet.

The time complexity is $O(n^2)$, and the space complexity is O(log n). The n is the length of the array.

Python 3

```python
class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        n = len(nums)
        ans = []
        for i in range(n - 2):
            if nums[i] > 0:
                break
            if i and nums[i] == nums[i - 1]:
                continue
            j, k = i + 1, n - 1
            while j < k:
                x = nums[i] + nums[j] + nums[k]
                if x < 0:
                    j += 1
                elif x > 0:
                    k -= 1
                else:
                    ans.append([nums[i], nums[j], nums[k]])
                    j, k = j + 1, k - 1
                    while j < k and nums[j] == nums[j - 1]:
                        j += 1
                    while j < k and nums[k] == nums[k + 1]:
                        k -= 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. 3Sum closest to a target
# 2. k-Sum for any k


# ============================================================
# FOLLOW-UP 1: 3Sum closest to a target
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort, fix one number, and walk two pointers;
# keep the sum nearest to the target.
# Time: O(n^2).  Space: O(n).
def three_sum_closest(nums, target):
    nums = sorted(nums)
    best = nums[0] + nums[1] + nums[2]
    for i in range(len(nums) - 2):
        lo, hi = i + 1, len(nums) - 1
```

```
        while lo < hi:
            total = nums[i] + nums[lo] + nums[hi]
            if abs(total - target) < abs(best - target):
                best = total
            if total < target:
                lo += 1
            elif total > target:
                hi -= 1
            else:
                return total
    return best


# ============================================================
# FOLLOW-UP 2: k-Sum for any k
# ============================================================

# ---- follow-up 2: function ----
# Answer: Recursion: fix the smallest number and solve (k
# - 1)-sum on the rest; stop at 2-sum with two pointers.
# Time: O(n^(k-1)).  Space: O(n + k + output).
def k_sum(nums, k, target):
    nums = sorted(nums)

    def go(start, k, target):
        result = []
        if k == 2:
            lo, hi = start, len(nums) - 1
            while lo < hi:
                total = nums[lo] + nums[hi]
                if total < target:
                    lo += 1
                elif total > target:
                    hi -= 1
                else:
                    result.append([nums[lo], nums[hi]])
                    lo += 1
                    hi -= 1
                    while lo < hi and nums[lo] == nums[lo - 1]:
                        lo += 1
            return result
        for i in range(start, len(nums) - k + 1):
```

```
            if i > start and nums[i] == nums[i - 1]:
                continue
            for rest in go(i + 1, k - 1, target - nums[i]):
                result.append([nums[i]] + rest)
        return result

    return go(0, k, target)
```

PART 2 · TWO POINTERS · PROBLEM 5 OF 5

443. String Compression

Medium · Two Pointers, String

leetcode.com/problems/string-compression

★ **TOP 7 PROBLEM** in Two Pointers: master this one first

PROBLEM

Given an array of characters `chars`, compress it using the following algorithm:

Begin with an empty string `s`. For each group of **consecutive repeating characters** in `chars`:

- If the group's length is `1`, append the character to `s`.
- Otherwise, append the character followed by the group's length.

The compressed string `s` **should not be returned separately**, but instead, be stored **in the input character array** `chars`. Note that group lengths that are `10` or longer will be split into multiple characters in `chars`.

After you are done **modifying the input array,** return *the new length of the array*.

You must write an algorithm that uses only constant extra space.

Note: The characters in the array beyond the returned length do not matter and should be ignored.

Example 1:

```
Input: chars = ["a","a","b","b","c","c","c"]
Output: 6
Explanation: The groups are "aa", "bb", and "ccc". This compresses to "a2b2c3".
After modifying the input array in-place, the first 6 characters of chars should be
["a","2","b","2","c","3"].
```

Example 2:

```
Input: chars = ["a"]
Output: 1
Explanation: The only group is "a", which remains uncompressed since it is a single character.
After modifying the input array in-place, the first character of chars should be ["a"].
```

Example 3:

```
Input: chars = ["a","b","b","b","b","b","b","b","b","b","b","b","b"]
Output: 4
Explanation: The groups are "a" and "bbbbbbbbbbbb". This compresses to "ab12".
After modifying the input array in-place, the first 4 characters of chars should be ["a","b","1","2"].
```

Constraints:

- `1 <= chars.length <= 2000`
- `chars[i]` is a lowercase English letter, uppercase English letter, digit, or symbol.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

A run becomes the character plus its count (when the count is greater than 1), written in place. A second buffer would break the space bound. Read pointer i finds a run; write pointer k stores the character and, if needed, the decimal digits. Then i jumps to the next run. The write pointer never passes the read pointer: a compressed prefix is never longer than the original, so overwriting from the left is safe. Return k as the new length.

Python 3

```python
class Solution:
    def compress(self, chars: List[str]) -> int:
        i, k, n = 0, 0, len(chars)
        while i < n:
            j = i + 1
            while j < n and chars[j] == chars[i]:
                j += 1
            chars[k] = chars[i]
            k += 1
            if j - i > 1:
                cnt = str(j - i)
                for c in cnt:
                    chars[k] = c
                    k += 1
            i = j
        return k
```

Complexity. Time: O(n). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Decompress it again
# 2. Return the compressed text as a string (do not change
# the input)


# ============================================================
# FOLLOW-UP 1: Decompress it again
# ============================================================

# ---- follow-up 1: function ----
# Answer: Read a letter and the number after it.
# Time: O(output length).  Space: O(output length).
def decompress(chars):
    result = []
    i = 0
    while i < len(chars):
        letter = chars[i]
        i += 1
        digits = ""
        while i < len(chars) and chars[i].isdigit():
            digits += chars[i]
            i += 1
        result.append(letter * (int(digits) if digits else 1))
    return "".join(result)


# ============================================================
# FOLLOW-UP 2: Return the compressed text as a string (do
# not change the input)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count each run and write the letter and the
# count when it is more than one.
# Time: O(n).  Space: O(n).
def compress_to_string(chars):
    pieces = []
    i = 0
    while i < len(chars):
        j = i
        while j < len(chars) and chars[j] == chars[i]:
            j += 1
        pieces.append(chars[i] + (str(j - i) if j - i > 1 else ""))
        i = j
```

```
    return "".join(pieces)
```

PART 2 · TOPIC 6 OF 50

Prefix Sum

5 problems · 2 Easy · 3 Medium · 0 Hard

THE IDEA

prefix[i] is the sum of the first i numbers, so any range sum is a subtraction. A difference array does the reverse: add on a range in O(1).

THE 5 PROBLEMS, EASY TO HARD | ★ = TOP 3: DO THESE FIRST

1	Running Sum of 1d Array	Easy	LC 1480	2161
2	Maximum Population Year	Easy	LC 1854	2163
3	★ **Range Addition**	Medium	LC 370	2165
4	★ **Subarray Sums Divisible by K**	Medium	LC 974	2169
5	★ **Increment Submatrices by One**	Medium	LC 2536	2171

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Range Sum Query - Immutable (Part 1, pattern 5, page 240)

Subarray Sum Equals K (Part 1, pattern 1, page 40)

Maximum Size Subarray Sum Equals k — substitute: Contiguous Array (Part 1, pattern 5, page 252)

Matrix Block Sum (Part 1, pattern 5, page 282)

Find Pivot Index (Part 1, pattern 5, page 243)

Continuous Subarray Sum (Part 1, pattern 1, page 55)

Range Sum Query 2D - Immutable (Part 1, pattern 5, page 248)

PART 2 · PREFIX SUM · PROBLEM 1 OF 5

1480. Running Sum of 1d Array

Easy · Array, Prefix Sum

leetcode.com/problems/running-sum-of-1d-array

PROBLEM

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0]…nums[i])`.

Return the running sum of `nums`.

Example 1:

```
Input: nums = [1,2,3,4]
Output: [1,3,6,10]
Explanation: Running sum is obtained as follows: [1, 1+2, 1+2+3, 1+2+3+4].
```

Example 2:

```
Input: nums = [1,1,1,1,1]
Output: [1,2,3,4,5]
Explanation: Running sum is obtained as follows: [1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1].
```

Example 3:

```
Input: nums = [3,1,2,10,1]
Output: [3,4,6,16,17]
```

Constraints:

- `1 <= nums.length <= 1000`
- `-10^6 <= nums[i] <= 10^6`

SOLUTION (PYTHON 3)

Solution 1: Prefix Sum

> **How to think about it**
>
> Each output entry is a prefix sum. n≤ 1000, so one `accumulate` pass is enough.

We directly traverse the array. For the current element nums[i], we add it with the prefix sum nums[i-1] to get the prefix sum nums[i] of the current element.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Python 3

```python
class Solution:
    def runningSum(self, nums: List[int]) -> List[int]:
        return list(accumulate(nums))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Recover the original array from its running sum
# 2. Running sum that restarts at every zero


# ============================================================
# FOLLOW-UP 1: Recover the original array from its running
# sum
# ============================================================

# ---- follow-up 1: function ----
# Answer: Subtract neighbours.
# Time: O(n).  Space: O(n).
def original_from_running_sum(running):
    return [running[0]] + [running[i] - running[i - 1] for i in range(1, len(running))]


# ============================================================
# FOLLOW-UP 2: Running sum that restarts at every zero
# ============================================================

# ---- follow-up 2: function ----
# Answer: Reset the sum when the number is zero.
# Time: O(n).  Space: O(n).
def running_sum_with_reset(nums):
    result = []
    total = 0
    for x in nums:
        total = 0 if x == 0 else total + x
        result.append(total)
    return result
```

1854. Maximum Population Year

Easy · Array, Counting, Prefix Sum

leetcode.com/problems/maximum-population-year

PROBLEM

You are given a 2D integer array `logs` where each `logs[i] = [birth_i, death_i]` indicates the birth and death years of the i^{th} person.

The **population** of some year `x` is the number of people alive during that year. The i^{th} person is counted in year `x`'s population if `x` is in the **inclusive** range `[birth_i, death_i - 1]`. Note that the person is **not** counted in the year that they die.

Return *the **earliest** year with the **maximum population***.

Example 1:

```
Input: logs = [[1993,1999],[2000,2010]]
Output: 1993
Explanation: The maximum population is 1, and 1993 is the earliest year with this population.
```

Example 2:

```
Input: logs = [[1950,1961],[1960,1971],[1970,1981]]
Output: 1960
Explanation:
The maximum population is 2, and it had happened in years 1960 and 1970.
The earlier year between them is 1960.
```

Constraints:

- `1 <= logs.length <= 100`
- `1950 <= birth_i < death_i <= 2050`

SOLUTION (PYTHON 3)

Solution 1: Difference Array

> **How to think about it**
>
> We need the earliest year with the maximum population; years lie in [1950,2050]. Recounting every life for every year repeats work. The domain is tiny, so a difference array adds 1 at birth and subtracts 1 at death. A prefix-sum scan yields the population of each year; the first maximum, shifted back by 1950, is the answer.

We notice that the range of years is [1950,..2050]. Therefore, we can map these years to an array d of length 101, where the index of the array represents the value of the year minus 1950.

Next, we traverse logs. For each person, we increment d[birth$_i$ - 1950] by 1 and decrement d[death$_i$ - 1950] by 1. Finally, we traverse the array d, find the maximum value of the prefix sum, which is the year with the most population, and add 1950 to get the answer.

The time complexity is O(n), and the space complexity is O(C). Where n is the length of the array logs, and C is the range size of the years, i.e., 2050 - 1950 + 1 = 101.

Python 3

```python
class Solution:
    def maximumPopulation(self, logs: List[List[int]]) -> int:
        d = [0] * 101
        offset = 1950
        for a, b in logs:
            a, b = a - offset, b - offset
            d[a] += 1
            d[b] -= 1
        s = mx = j = 0
        for i, x in enumerate(d):
            s += x
            if mx < s:
                mx, j = s, i
        return j + offset
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. All years that have the maximum population
# 2. The population in a given year


# ============================================================
# FOLLOW-UP 1: All years that have the maximum population
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same difference array; collect every year that
# reaches the maximum.
# Time: O(n log n + Y), Y = years in the output.  Space:
# O(n + Y).
def max_population_years(logs):
    delta = {}
    for birth, death in logs:
        delta[birth] = delta.get(birth, 0) + 1
        delta[death] = delta.get(death, 0) - 1
    years = []
    best = 0
    people = 0
    previous = None
    for year in sorted(delta):
        if previous is not None and people == best and best > 0:
            years.extend(range(previous, year))
        people += delta[year]
        if people > best:
            best = people
            years = []
        previous = year
    return years


# ============================================================
# FOLLOW-UP 2: The population in a given year
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the people born by that year and not yet
# dead.
# Time: O(n).  Space: O(1).
def population_in_year(logs, year):
    return sum(1 for birth, death in logs if birth <= year < death)
```

370. Range Addition

Medium · Array, Prefix Sum

leetcode.com/problems/range-addition

★ **TOP 7 PROBLEM** in Prefix Sum: master this one first

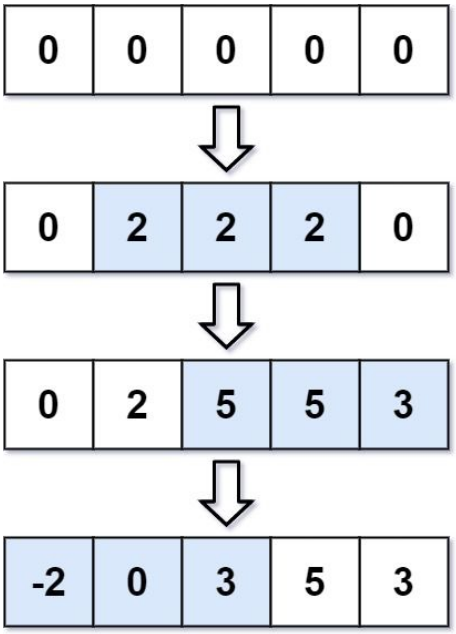
PROBLEM

You are given an integer `length` and an array `updates` where `updates[i] = [startIdx`$_i$`, endIdx`$_i$`, inc`$_i$`]`.

You have an array `arr` of length `length` with all zeros, and you have some operation to apply on `arr`. In the `i`th operation, you should increment all the elements `arr[startIdx`$_i$`], arr[startIdx`$_i$` + 1], ..., arr[endIdx`$_i$`]` by `inc`$_i$.

Return `arr` *after applying all the* `updates`.

Example 1:




```
Input: length = 5, updates = [[1,3,2],[2,4,3],[0,2,-2]]
Output: [-2,0,3,5,3]
```

Example 2:

```
Input: length = 10, updates = [[2,4,6],[5,6,8],[1,9,-4]]
Output: [0,-4,2,2,2,4,4,-4,-4,-4]
```

Constraints:

- `1 <= length <= 10`^{5}
- `0 <= updates.length <= 10`^{4}
- `0 <= startIdx`$_i$ `<= endIdx`$_i$ `< length`
- `-1000 <= inc`$_i$ `<= 1000`

SOLUTIONS (PYTHON 3)

Solution 1: Difference Array

> **How to think about it**
>
> Many range-add updates, then the final array. Touching each range in O(n) does not scale. A difference array turns a range add into two point writes. Add c at l and -c at r+1, then prefix-sum. Each update is O(1).

This is a template problem for difference arrays.

We define d as the difference array. To add c to each number in the interval [l,..r], we set d[l] += c and d[r+1] -= c. Finally, we compute the prefix sum of the difference array to obtain the original array.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array.

Python 3

```python
class Solution:
    def getModifiedArray(self, length: int, updates: List[List[int]]) -> List[int]:
        d = [0] * length
        for l, r, c in updates:
            d[l] += c
            if r + 1 < length:
                d[r + 1] -= c
        return list(accumulate(d))
```

Solution 2: Binary Indexed Tree + Difference Array

> **How to think about it**
>
> The difference array is already linear. A Fenwick tree encodes the same two-point updates; a prefix query recovers the value. An extra log n buys online queries later.

The time complexity is O(n × log n).

A Binary Indexed Tree (BIT), also known as a Fenwick Tree, can efficiently perform the following two operations:

1. **Point Update** `update(x, delta)`: Add a value delta to the number at position x in the sequence.
2. **Prefix Sum Query** `query(x)`: Query the sum of the interval [1, ... , x] in the sequence, i.e., the prefix sum up to position x.

The time complexity for both operations is O(log n).

Python 3

```python
class BinaryIndexedTree:
    __slots__ = "n", "c"

    def __init__(self, n: int):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x: int, delta: int) -> None:
        while x <= self.n:
            self.c[x] += delta
            x += x & -x

    def query(self, x: int) -> int:
        s = 0
        while x:
            s += self.c[x]
            x -= x & -x
        return s


class Solution:
    def getModifiedArray(self, length: int, updates: List[List[int]]) -> List[int]:
        tree = BinaryIndexedTree(length)
        for l, r, c in updates:
            tree.update(l + 1, c)
            tree.update(r + 2, -c)
        return [tree.query(i + 1) for i in range(length)]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The biggest value after all updates, and where it is
# 2. The array does not start at zero


# ============================================================
# FOLLOW-UP 1: The biggest value after all updates, and
# where it is
# ============================================================

# ---- follow-up 1: function ----
# Answer: Build the array from the difference array and
# keep the maximum.
# Time: O(n + u), u = updates.  Space: O(n).
def biggest_after_updates(length, updates):
    diff = [0] * (length + 1)
    for start, end, inc in updates:
        diff[start] += inc
        diff[end + 1] -= inc
    best = None
    best_index = -1
    total = 0
    for i in range(length):
        total += diff[i]
        if best is None or total > best:
            best = total
            best_index = i
    return best, best_index


# ============================================================
# FOLLOW-UP 2: The array does not start at zero
# ============================================================

# ---- follow-up 2: function ----
```

```
# Answer: Add the starting values to the result of the
# difference array.
# Time: O(n + u).  Space: O(n).
def range_addition_from(start_values, updates):
    n = len(start_values)
    diff = [0] * (n + 1)
    for start, end, inc in updates:
        diff[start] += inc
        diff[end + 1] -= inc
    result = []
    total = 0
    for i in range(n):
        total += diff[i]
        result.append(start_values[i] + total)
    return result
```

PART 2 · PREFIX SUM · PROBLEM 4 OF 5

974. Subarray Sums Divisible by K

Medium · Array, Hash Table, Prefix Sum

leetcode.com/problems/subarray-sums-divisible-by-k

★ **TOP 7 PROBLEM** in Prefix Sum: master this one first

PROBLEM

Given an integer array `nums` and an integer `k`, return *the number of non-empty **subarrays** that have a sum divisible by* `k`.

A **subarray** is a **contiguous** part of an array.

Example 1:

```
Input: nums = [4,5,0,-2,-3,1], k = 5
Output: 7
Explanation: There are 7 subarrays with a sum divisible by k = 5:
[4, 5, 0, -2, -3, 1], [5], [5, 0], [5, 0, -2, -3], [0], [0, -2, -3], [-2, -3]
```

Example 2:

```
Input: nums = [5], k = 9
Output: 0
```

Constraints:

- `1 <= nums.length <= 3 * 10^4`
- `-10^4 <= nums[i] <= 10^4`
- `2 <= k <= 10^4`

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Prefix Sum

How to think about it

A subarray sum is divisible by k iff the two prefix sums are congruent modulo k. Enumerating intervals is quadratic. While scanning, a counter of residues tells how many earlier prefixes share the current s, which is the number of good subarrays ending here. Start with cnt[0]=1 for the empty prefix.

Suppose there exists $i \le j$ such that the sum of nums[i,..j] is divisible by k. If we let s_i represent the sum of nums[0,..i] and s_j represent the sum of nums[0,..j], then $s_j - s_i$ is divisible by k, i.e., $(s_j - s_i) \bmod k = 0$, which means $s_j \bmod k = s_i \bmod k$. Therefore, we can use a hash table to count the number of prefix sums modulo k, allowing us to quickly determine if there exists a subarray that meets the condition.

We use a hash table cnt to count the number of prefix sums modulo k, where cnt[i] represents the number of prefix sums with a modulo k value of i. Initially, cnt[0] = 1. We use a variable s to represent the prefix sum, initially s = 0.

Next, we traverse the array nums from left to right. For each element x, we calculate s = (s + x) mod k, then update the answer ans = ans + cnt[s], where cnt[s] represents the number of prefix sums with a modulo k value of s. Finally, we increment the value of cnt[s] by 1 and continue to the next element.

In the end, we return the answer ans.

Note: Since the value of s can be negative, we can add k to the result of s mod k and then take modulo k again to ensure that the value of s is non-negative.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def subarraysDivByK(self, nums: List[int], k: int) -> int:
        cnt = Counter({0: 1})
        ans = s = 0
        for x in nums:
            s = (s + x) % k
            ans += cnt[s]
            cnt[s] += 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The longest subarray whose sum is divisible by k
# 2. Count the subarrays whose sum leaves remainder r


# ============================================================
# FOLLOW-UP 1: The longest subarray whose sum is divisible
# by k
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember the first index of every prefix
# remainder.
# Time: O(n).  Space: O(k).
def longest_divisible_subarray(nums, k):
    first = {0: -1}
    total = 0
    best = 0
    for i, x in enumerate(nums):
        total = (total + x) % k
        if total in first:
            best = max(best, i - first[total])
        else:
            first[total] = i
    return best


# ============================================================
# FOLLOW-UP 2: Count the subarrays whose sum leaves
# remainder r
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count pairs of prefixes whose remainders differ
# by r.
# Time: O(n).  Space: O(k).
def count_with_remainder(nums, k, r):
    seen = {0: 1}
    total = 0
    count = 0
    for x in nums:
        total = (total + x) % k
        count += seen.get((total - r) % k, 0)
        seen[total] = seen.get(total, 0) + 1
    return count
```

2536. Increment Submatrices by One

Medium · Array, Matrix, Prefix Sum

leetcode.com/problems/increment-submatrices-by-one

★ **TOP 7 PROBLEM** in Prefix Sum: master this one first

PROBLEM

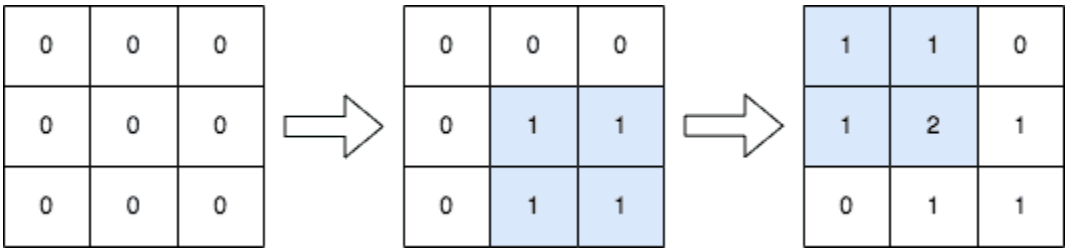
You are given a positive integer `n`, indicating that we initially have an `n x n` **0-indexed** integer matrix `mat` filled with zeroes.

You are also given a 2D integer array `query`. For each `query[i] = [row1_i, col1_i, row2_i, col2_i]`, you should do the following operation:

- Add `1` to **every element** in the submatrix with the **top left** corner `(row1_i, col1_i)` and the **bottom right** corner `(row2_i, col2_i)`. That is, add `1` to `mat[x][y]` for all `row1_i <= x <= row2_i` and `col1_i <= y <= col2_i`.

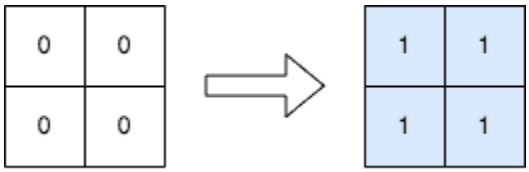
Return *the matrix* `mat` *after performing every query.*

Example 1:




```
Input: n = 3, queries = [[1,1,2,2],[0,0,1,1]]
Output: [[1,1,0],[1,2,1],[0,1,1]]
Explanation: The diagram above shows the initial matrix, the matrix after the first query, and the
matrix after the second query.
- In the first query, we add 1 to every element in the submatrix with the top left corner (1, 1) and
bottom right corner (2, 2).
- In the second query, we add 1 to every element in the submatrix with the top left corner (0, 0) and
bottom right corner (1, 1).
```

Example 2:




```
Input: n = 2, queries = [[0,0,1,1]]
Output: [[1,1],[1,1]]
Explanation: The diagram above shows the initial matrix and the matrix after the first query.
- In the first query we add 1 to every element in the matrix.
```

Constraints:

- `1 <= n <= 500`
- `1 <= queries.length <= 10^4`
- `0 <= row1_i <= row2_i < n`
- `0 <= col1_i <= col2_i < n`

SOLUTION (PYTHON 3)

Solution 1: 2D Difference Array

> **How to think about it**
>
> Several submatrices must be incremented by one. Walking every cell of every query is heavy when both q and n reach 500. A 2D difference array turns one rectangle increment into four O(1) corner updates: +1 at the top-left, +1 just past the bottom-right, and -1 at the other two corners. A 2D prefix sum restores the matrix after all queries.

A 2D difference array is a technique used to efficiently handle range updates on 2D arrays. We can implement fast updates on submatrices by maintaining a difference matrix of the same size as the original matrix.

Suppose we have a 2D difference matrix diff, initially with all elements set to 0. For each query [row1, col1, row2, col2], we can update the difference matrix through the following steps:

1. Increment position (row1, col1) by 1.
2. Decrement position (row2 + 1, col1) by 1, provided that row2 + 1 < n.
3. Decrement position (row1, col2 + 1) by 1, provided that col2 + 1 < n.
4. Increment position (row2 + 1, col2 + 1) by 1, provided that row2 + 1 < n and col2 + 1 < n.

After completing all queries, we need to convert the difference matrix back to the original matrix using prefix sums. That is, for each position (i, j), we calculate:

$$\text{mat}[i][j] = \text{diff}[i][j] + (\text{mat}[i-1][j] \text{ if } i > 0 \text{ else } 0) + (\text{mat}[i][j-1] \text{ if } j > 0 \text{ else } 0) - (\text{mat}[i-1][j-1] \text{ if } i > 0 \text{ and } j > 0 \text{ else } 0)$$

The time complexity is $O(m + n^2)$, where m and n are the length of queries and the given n, respectively. Ignoring the space consumed by the answer, the space complexity is O(1).

Python 3

```python
class Solution:
    def rangeAddQueries(self, n: int, queries: List[List[int]]) -> List[List[int]]:
        mat = [[0] * n for _ in range(n)]
        for x1, y1, x2, y2 in queries:
            mat[x1][y1] += 1
            if x2 + 1 < n:
                mat[x2 + 1][y1] -= 1
            if y2 + 1 < n:
                mat[x1][y2 + 1] -= 1
            if x2 + 1 < n and y2 + 1 < n:
                mat[x2 + 1][y2 + 1] += 1

        for i in range(n):
            for j in range(n):
                if i:
                    mat[i][j] += mat[i - 1][j]
                if j:
                    mat[i][j] += mat[i][j - 1]
                if i and j:
                    mat[i][j] -= mat[i - 1][j - 1]
        return mat
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Each query adds its own amount: [r1, c1, r2, c2, v]
# 2. How many cells end up with a value of at least k


# ========================================================
# FOLLOW-UP 1: Each query adds its own amount: [r1, c1,
# r2, c2, v]
# ========================================================
```

```
# ---- follow-up 1: function ----
# Answer: The same 2D difference array, adding v instead
# of 1.
# Time: O(n^2 + q).  Space: O(n^2).
def add_values_to_submatrices(n, queries):
    diff = [[0] * (n + 1) for _ in range(n + 1)]
    for r1, c1, r2, c2, v in queries:
        diff[r1][c1] += v
        diff[r1][c2 + 1] -= v
        diff[r2 + 1][c1] -= v
        diff[r2 + 1][c2 + 1] += v
    result = [[0] * n for _ in range(n)]
    for r in range(n):
        for c in range(n):
            above = result[r - 1][c] if r else 0
            left = result[r][c - 1] if c else 0
            corner = result[r - 1][c - 1] if r and c else 0
            result[r][c] = diff[r][c] + above + left - corner
    return result


# ========================================================
# FOLLOW-UP 2: How many cells end up with a value of at
# least k
# ========================================================

# ---- follow-up 2: function ----
# Answer: Build the matrix with the difference array and
# count.
# Time: O(n^2 + q).  Space: O(n^2).
def count_cells_at_least(n, queries, k):
    diff = [[0] * (n + 1) for _ in range(n + 1)]
    for r1, c1, r2, c2 in queries:
        diff[r1][c1] += 1
        diff[r1][c2 + 1] -= 1
        diff[r2 + 1][c1] -= 1
        diff[r2 + 1][c2 + 1] += 1
    total = 0
    grid = [[0] * n for _ in range(n)]
    for r in range(n):
        for c in range(n):
            above = grid[r - 1][c] if r else 0
            left = grid[r][c - 1] if c else 0
            corner = grid[r - 1][c - 1] if r and c else 0
            grid[r][c] = diff[r][c] + above + left - corner
            if grid[r][c] >= k:
                total += 1
    return total
```

PART 2 · TOPIC 7 OF 50

Sliding Window - Fixed Size

2 problems · 0 Easy · 1 Medium · 1 Hard

THE IDEA

Move a window of length k one step at a time: add the new element, remove the old one, and update the answer in O(1).

THE 2 PROBLEMS, EASY TO HARD | ★ = TOP 2: DO THESE FIRST

1	★ **Maximum Sum of Distinct Subarrays With Length K**	Medium	LC 2461	2175
2	★ **Substring with Concatenation of All Words**	Hard	LC 30	2178

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Maximum Average Subarray I (Part 1, pattern 35, page 1836)

Permutation in String (Part 1, pattern 2, page 81)

Find All Anagrams in a String (Part 1, pattern 2, page 84)

Maximum Number of Vowels in a Substring of Given Length (Part 1, pattern 2, page 101)

2461. Maximum Sum of Distinct Subarrays With Length K

Medium · Array, Hash Table, Sliding Window

leetcode.com/problems/maximum-sum-of-distinct-subarrays-with-length-k

★ **TOP 7 PROBLEM** in Sliding Window - Fixed Size: master this one first

PROBLEM

You are given an integer array `nums` and an integer `k`. Find the maximum subarray sum of all the subarrays of `nums` that meet the following conditions:

- The length of the subarray is `k`, and
- All the elements of the subarray are **distinct**.

Return *the maximum subarray sum of all the subarrays that meet the conditions.* If no subarray meets the conditions, return `0`.

*A **subarray** is a contiguous non-empty sequence of elements within an array.*

Example 1:

```
Input: nums = [1,5,4,2,9,9,9], k = 3
Output: 15
Explanation: The subarrays of nums with length 3 are:
- [1,5,4] which meets the requirements and has a sum of 10.
- [5,4,2] which meets the requirements and has a sum of 11.
- [4,2,9] which meets the requirements and has a sum of 15.
- [2,9,9] which does not meet the requirements because the element 9 is repeated.
- [9,9,9] which does not meet the requirements because the element 9 is repeated.
We return 15 because it is the maximum subarray sum of all the subarrays that meet the conditions
```

Example 2:

```
Input: nums = [4,4,4], k = 3
Output: 0
Explanation: The subarrays of nums with length 3 are:
- [4,4,4] which does not meet the requirements because the element 4 is repeated.
We return 0 because no subarrays meet the conditions.
```

Constraints:

- `1 <= k <= nums.length <=` 10^5
- `1 <= nums[i] <=` 10^5

SOLUTION (PYTHON 3)

Solution 1: Sliding Window + Hash Table

How to think about it

Maximum sum of a length-k window with distinct values, n≤ 10^5. Slide a window of sum and frequencies; when the map size equals k, update the answer. Drop a key when its count hits zero.

We maintain a sliding window of length k, use a hash table cnt to record the count of each number in the window, and use a variable s to record the sum of all numbers in the window. Each time we slide the window, if all numbers in the window are unique, we update the answer.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array nums.

Python 3

```python
class Solution:
    def maximumSubarraySum(self, nums: List[int], k: int) -> int:
        cnt = Counter(nums[:k])
        s = sum(nums[:k])
        ans = s if len(cnt) == k else 0
        for i in range(k, len(nums)):
            cnt[nums[i]] += 1
            cnt[nums[i - k]] -= 1
            if cnt[nums[i - k]] == 0:
                cnt.pop(nums[i - k])
            s += nums[i] - nums[i - k]
            if len(cnt) == k:
                ans = max(ans, s)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subarray that gives the maximum
# 2. How many windows of length k have all distinct
# numbers


# ============================================================
# FOLLOW-UP 1: Return the subarray that gives the maximum
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same window, remembering where the best window
# starts.
# Time: O(n).  Space: O(k).
def best_distinct_window(nums, k):
    count = {}
    total = 0
    best = None
    best_start = -1
    for i in range(len(nums)):
        count[nums[i]] = count.get(nums[i], 0) + 1
        total += nums[i]
        if i >= k:
            old = nums[i - k]
            count[old] -= 1
            if count[old] == 0:
                del count[old]
            total -= old
        if i >= k - 1 and len(count) == k and (best is None or total > best):
            best = total
            best_start = i - k + 1
    if best is None:
        return None
    return nums[best_start:best_start + k]


# ============================================================
# FOLLOW-UP 2: How many windows of length k have all
# distinct numbers
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the windows whose number of distinct
# values is k.
# Time: O(n).  Space: O(k).
def count_distinct_windows(nums, k):
    count = {}
    windows = 0
```

```
    for i in range(len(nums)):
        count[nums[i]] = count.get(nums[i], 0) + 1
        if i >= k:
            old = nums[i - k]
            count[old] -= 1
            if count[old] == 0:
                del count[old]
        if i >= k - 1 and len(count) == k:
            windows += 1
    return windows
```

30. Substring with Concatenation of All Words

Hard · Hash Table, String, Sliding Window

leetcode.com/problems/substring-with-concatenation-of-all-words

★ **TOP 7 PROBLEM** in Sliding Window - Fixed Size: master this one first

PROBLEM

You are given a string `s` and an array of strings `words`. All the strings of `words` are of **the same length**.

A **concatenated string** is a string that exactly contains all the strings of any permutation of `words` concatenated.

- For example, if `words = ["ab","cd","ef"]`, then `"abcdef"`, `"abefcd"`, `"cdabef"`, `"cdefab"`, `"efabcd"`, and `"efcdab"` are all concatenated strings. `"acdbef"` is not a concatenated string because it is not the concatenation of any permutation of `words`.

Return an array of *the starting indices* of all the concatenated substrings in `s`. You can return the answer in **any order**.

Example 1:

Input: s = "barfoothefoobarman", words = ["foo","bar"]

Output: [0,9]

Explanation:

The substring starting at 0 is `"barfoo"`. It is the concatenation of `["bar","foo"]` which is a permutation of `words`.
The substring starting at 9 is `"foobar"`. It is the concatenation of `["foo","bar"]` which is a permutation of `words`.

Example 2:

Input: s = "wordgoodgoodgoodbestword", words = ["word","good","best","word"]

Output: []

Explanation:

There is no concatenated substring.

Example 3:

Input: s = "barfoofoobarthefoobarman", words = ["bar","foo","the"]

Output: [6,9,12]

Explanation:

The substring starting at 6 is `"foobarthe"`. It is the concatenation of `["foo","bar","the"]`.
The substring starting at 9 is `"barthefoo"`. It is the concatenation of `["bar","the","foo"]`.
The substring starting at 12 is `"thefoobar"`. It is the concatenation of `["the","foo","bar"]`.

Constraints:

- `1 <= s.length <= 10^4`
- `1 <= words.length <= 5000`
- `1 <= words[i].length <= 30`
- `s` and `words[i]` consist of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Sliding Window

How to think about it

The first idea is to try every start and test whether the length-n· k window is some permutation of words. With m ≤ 10^4, n ≤ 5000, k ≤ 30, slicing n words per start is worst-case O(mnk) and will not pass. The bottleneck is recounting heavily overlapping windows from scratch. Words share one length k, so the window slides in steps of k and frequencies can be maintained incrementally in a hash map. There are only k alignments (starts 0,1,…,k-1). In each, the right end eats a length-k token: a word outside cnt resets the window; an over-used word is spat out from the left until counts are legal; exactly n words means we record the left index. Each character enters and leaves a window a constant number of times, so the total is O(mk).

We use a hash table cnt to count the number of times each word appears in words, and use a hash table cnt1 to count the number of times each word appears in the current sliding window. We denote the length of the string s as m, the number of words in the string array words as n, and the length of each word as k.

We can enumerate the starting point i of the sliding window, where 0 < i < k. For each starting point, we maintain a sliding window with the left boundary as l, the right boundary as r, and the number of words in the sliding window as t. Additionally, we use a hash table cnt1 to count the number of times each word appears in the sliding window.

Each time, we extract the string s[r:r+k]. If s[r:r+k] is not in the hash table cnt, it means that the words in the current sliding window are not valid. We update the left boundary l to r, clear the hash table cnt1, and reset the word count t to 0. If s[r:r+k] is in the hash table cnt, it means that the words in the current sliding window are valid. We increase the word count t by 1, and increase the count of s[r:r+k] in the hash table cnt1 by 1. If cnt1[s[r:r+k]] is greater than cnt[s[r:r+k]], it means that s[r:r+k] appears too many times in the current sliding window. We need to move the left boundary l to the right until cnt1[s[r:r+k]] = cnt[s[r:r+k]]. If t = n, it means that the words in the current sliding window are exactly valid, and we add the left boundary l to the answer array.

The time complexity is O(m × k), and the space complexity is O(n × k). Here, m and n are the lengths of the string s and the string array words respectively, and k is the length of the words in the string array words.

Python 3

```python
class Solution:
    def findSubstring(self, s: str, words: List[str]) -> List[int]:
        cnt = Counter(words)
        m, n = len(s), len(words)
        k = len(words[0])
        ans = []
        for i in range(k):
            l = r = i
            cnt1 = Counter()
            while r + k <= m:
                t = s[r : r + k]
                r += k
                if cnt[t] == 0:
                    l = r
                    cnt1.clear()
                    continue
                cnt1[t] += 1
                while cnt1[t] > cnt[t]:
                    rem = s[l : l + k]
                    l += k
                    cnt1[rem] -= 1
                if r - l == n * k:
                    ans.append(l)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Only the first starting index
# 2. The words may have different lengths


# ============================================================
# FOLLOW-UP 1: Only the first starting index
# ============================================================

# ---- follow-up 1: function ----
# Answer: Run the same search and stop at the first match.
# Time: O(n * W * L).  Space: O(W).
def first_concatenation_index(s, words):
    size = len(words[0])
    total = size * len(words)
    need = {}
    for w in words:
        need[w] = need.get(w, 0) + 1
    for start in range(len(s) - total + 1):
        seen = {}
        ok = True
        for i in range(len(words)):
            piece = s[start + i * size:start + (i + 1) * size]
            seen[piece] = seen.get(piece, 0) + 1
            if seen[piece] > need.get(piece, 0):
                ok = False
                break
        if ok:
            return start
    return -1


# ============================================================
# FOLLOW-UP 2: The words may have different lengths
# ============================================================

# ---- follow-up 2: function ----
# Answer: Compare as a multiset of words by trying every
# start and matching words with a recursive search.
# Time: O(n * W!) in the worst case.  Space: O(W).
def concatenations_with_mixed_lengths(s, words):
    total = sum(len(w) for w in words)
    results = []

    def can_match(position, remaining):
        if not remaining:
            return True
        for w in set(remaining):
            if s.startswith(w, position):
                rest = list(remaining)
                rest.remove(w)
                if can_match(position + len(w), rest):
                    return True
        return False

    for start in range(len(s) - total + 1):
        if can_match(start, words):
            results.append(start)
    return results
```

PART 2 · TOPIC 8 OF 50

Sliding Window - Dynamic Size

3 problems · 0 Easy · 3 Medium · 0 Hard

THE IDEA

Grow the right end, and shrink the left end while the window breaks the rule. Each index enters and leaves once, so it is O(n).

THE 3 PROBLEMS, EASY TO HARD | ★ = TOP 3: DO THESE FIRST

1	★ **Longest Substring with At Most Two Distinct Characters**	Medium	LC 159	2182
2	★ **Longest Substring with At Most K Distinct Characters**	Medium	LC 340	2184
3	★ **Longest Substring with At Least K Repeating Characters**	Medium	LC 395	2186

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Longest Substring Without Repeating Characters (Part 1, pattern 2, page 72)

Minimum Size Subarray Sum (Part 1, pattern 2, page 75)

Count Number of Nice Subarrays (Part 1, pattern 2, page 103)

Maximum Points You Can Obtain from Cards (Part 1, pattern 22, page 1199)

Frequency of the Most Frequent Element (Part 1, pattern 2, page 114)

Minimum Window Subsequence (Part 1, pattern 3, page 180)

Longest Repeating Character Replacement (Part 1, pattern 2, page 78)

Max Consecutive Ones III (Part 1, pattern 2, page 91)

Fruit Into Baskets (Part 1, pattern 2, page 87)

Subarray Product Less Than K (Part 1, pattern 2, page 94)

Minimum Window Substring (Part 1, pattern 2, page 121)

Subarrays with K Different Integers (Part 1, pattern 1, page 62)

159. Longest Substring with At Most Two Distinct Characters

Medium · Hash Table, String, Sliding Window

leetcode.com/problems/longest-substring-with-at-most-two-distinct-characters

★ **TOP 7 PROBLEM** in Sliding Window - Dynamic Size: master this one first

PROBLEM

Given a string `s`, return *the length of the longest substring that contains at most **two distinct characters***.

Example 1:

```
Input: s = "eceba"
Output: 3
Explanation: The substring is "ece" which its length is 3.
```

Example 2:

```
Input: s = "ccaabbb"
Output: 5
Explanation: The substring is "aabbb" which its length is 5.
```

Constraints:

- `1 <= s.length <= 10^5`
- `s` consists of English letters.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Longest substring with at most two distinct characters. n≤ 10^5, so enumerating endpoints is O(n^2). The distinct count grows with the right end and shrinks with the left. Extend right while counting; when there are more than two kinds, move left. Update the answer on a valid window.

Python 3

```python
class Solution:
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        cnt = Counter()
        ans = j = 0
        for i, c in enumerate(s):
            cnt[c] += 1
            while len(cnt) > 2:
                cnt[s[j]] -= 1
                if cnt[s[j]] == 0:
                    cnt.pop(s[j])
                j += 1
            ans = max(ans, i - j + 1)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the substring itself
# 2. Count ALL substrings with at most k distinct letters


# ============================================================
# FOLLOW-UP 1: Return the substring itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same window; remember where the longest window
# starts.
# Time: O(n).  Space: O(1).
def longest_two_distinct_substring(s):
    count = {}
    left = 0
    best_start = best_length = 0
    for right in range(len(s)):
        count[s[right]] = count.get(s[right], 0) + 1
        while len(count) > 2:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        if right - left + 1 > best_length:
            best_length = right - left + 1
            best_start = left
    return s[best_start:best_start + best_length]


# ============================================================
# FOLLOW-UP 2: Count ALL substrings with at most k
# distinct letters
# ============================================================

# ---- follow-up 2: function ----
# Answer: For each right end, every start from the
# window's left end works.
# Time: O(n).  Space: O(k).
def count_at_most_k_distinct(s, k):
    count = {}
    left = 0
    total = 0
    for right in range(len(s)):
        count[s[right]] = count.get(s[right], 0) + 1
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        total += right - left + 1
    return total
```

340. Longest Substring with At Most K Distinct Characters

Medium · Hash Table, String, Sliding Window

leetcode.com/problems/longest-substring-with-at-most-k-distinct-characters

★ **TOP 7 PROBLEM** in Sliding Window - Dynamic Size: master this one first

PROBLEM

Given a string `s` and an integer `k`, return *the length of the longest substring of* `s` *that contains at most* `k` ***distinct*** *characters*.

Example 1:

```
Input: s = "eceba", k = 2
Output: 3
Explanation: The substring is "ece" with length 3.
```

Example 2:

```
Input: s = "aa", k = 1
Output: 2
Explanation: The substring is "aa" with length 2.
```

Constraints:

- `1 <= s.length <= 5 * 10^4`
- `0 <= k <= 50`

SOLUTION (PYTHON 3)

Solution 1: Sliding Window + Hash Table

How to think about it

Longest substring with at most k distinct characters. Checking every pair of ends is O(n^2). Distinctness grows with the right end and shrinks when the left advances, so a sliding window works. Admit the right character; while the map has more than k keys, drop the left. The window is always the longest legal suffix of the prefix, and the length is n-l.

We can use the idea of a sliding window, with a hash table cnt to record the occurrence count of each character within the window, and l to denote the left boundary of the window.

Iterate through the string, adding the character at the right boundary to the hash table each time. If the number of distinct characters in the hash table exceeds k, remove the character at the left boundary from the hash table, then update the left boundary l.

Finally, return the length of the string minus the length of the left boundary.

The time complexity is O(n), and the space complexity is O(k). Here, n is the length of the string.

Python 3

```python
class Solution:
    def lengthOfLongestSubstringKDistinct(self, s: str, k: int) -> int:
        l = 0
        cnt = Counter()
        for c in s:
            cnt[c] += 1
            if len(cnt) > k:
                cnt[s[l]] -= 1
                if cnt[s[l]] == 0:
                    del cnt[s[l]]
                l += 1
        return len(s) - l
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. EXACTLY k distinct letters (longest)
# 2. Count the substrings with exactly k distinct letters


# ============================================================
# FOLLOW-UP 1: EXACTLY k distinct letters (longest)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Track the longest window with at most k and make
# sure it has k distinct letters by also keeping the count
# of distinct letters.
# Time: O(n^2).  Space: O(k).
def longest_exactly_k_distinct(s, k):
    best = 0
    for start in range(len(s)):
        seen = set()
        for end in range(start, len(s)):
            seen.add(s[end])
            if len(seen) > k:
                break
            if len(seen) == k:
                best = max(best, end - start + 1)
    return best


# ============================================================
# FOLLOW-UP 2: Count the substrings with exactly k
# distinct letters
# ============================================================

# ---- follow-up 2: function ----
# Answer: at most k minus at most k - 1.
# Time: O(n).  Space: O(k).
def count_exactly_k_distinct(s, k):
    def at_most(m):
        count = {}
        left = 0
        total = 0
        for right in range(len(s)):
            count[s[right]] = count.get(s[right], 0) + 1
            while len(count) > m:
                count[s[left]] -= 1
                if count[s[left]] == 0:
                    del count[s[left]]
                left += 1
            total += right - left + 1
        return total

    return at_most(k) - at_most(k - 1)
```

395. Longest Substring with At Least K Repeating Characters

Medium · Hash Table, String, Divide and Conquer, Sliding Window

leetcode.com/problems/longest-substring-with-at-least-k-repeating-characters

★ **TOP 7 PROBLEM** in Sliding Window - Dynamic Size: master this one first

PROBLEM

Given a string `s` and an integer `k`, return *the length of the longest substring of* `s` *such that the frequency of each character in this substring is greater than or equal to* `k`.

if no such substring exists, return 0.

Example 1:

```
Input: s = "aaabb", k = 3
Output: 3
Explanation: The longest substring is "aaa", as 'a' is repeated 3 times.
```

Example 2:

```
Input: s = "ababbc", k = 2
Output: 5
Explanation: The longest substring is "ababb", as 'a' is repeated 2 times and 'b' is repeated 3 times.
```

Constraints:

- `1 <= s.length <= 10^4`
- `s` consists of only lowercase English letters.
- `1 <= k <= 10^5`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Longest substring in which every present character occurs at least k times. A distinct-count window does not capture "all ≥ k". A character that appears fewer than k times in the whole range cannot appear in any legal substring, so it is a split point. Count the range, split on a rare character, and recurse; if none exists the range is legal. Depth is at most 26.

Python 3

```python
class Solution:
    def longestSubstring(self, s: str, k: int) -> int:
        def dfs(l, r):
            cnt = Counter(s[l : r + 1])
            split = next((c for c, v in cnt.items() if v < k), '')
            if not split:
                return r - l + 1
            i = l
            ans = 0
            while i <= r:
                while i <= r and s[i] == split:
                    i += 1
                if i >= r:
                    break
                j = i
                while j <= r and s[j] != split:
                    j += 1
                t = dfs(i, j - 1)
                ans = max(ans, t)
                i = j
            return ans

        return dfs(0, len(s) - 1)
```

Complexity. Time: O(n * C), C = 26 letters (the recursion splits at most C times deep). Space: O(C).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the substring
# 2. Every letter appears AT MOST k times: the longest
# such substring


# ============================================================
# FOLLOW-UP 1: Return the substring
# ============================================================

# ---- follow-up 1: function ----
# Answer: Split at every letter that is rarer than k, and
# recurse on the pieces; keep the longest.
# Time: O(26^2 * n) = O(n).  Space: O(n).
def longest_substring_text(s, k):
    if len(s) < k:
        return ""
    for ch in set(s):
        if s.count(ch) < k:
            best = ""
            for piece in s.split(ch):
                candidate = longest_substring_text(piece, k)
                if len(candidate) > len(best):
                    best = candidate
            return best
    return s


# ============================================================
# FOLLOW-UP 2: Every letter appears AT MOST k times: the
# longest such substring
# ============================================================

# ---- follow-up 2: function ----
# Answer: Sliding window with a count per letter.
# Time: O(n).  Space: O(alphabet).
def longest_each_at_most_k(s, k):
    count = {}
```

```
    left = 0
    best = 0
    for right in range(len(s)):
        count[s[right]] = count.get(s[right], 0) + 1
        while count[s[right]] > k:
            count[s[left]] -= 1
            left += 1
        best = max(best, right - left + 1)
    return best
```

PART 2 · TOPIC 9 OF 50

Kadane's Algorithm

5 problems · 0 Easy · 5 Medium · 0 Hard

THE IDEA

The best subarray ending here is either the current number alone or the current number added to the best one ending before it. Keep the best seen overall.

THE 5 PROBLEMS, EASY TO HARD | ★ = TOP 5: DO THESE FIRST

1	★ **Maximum Sum Circular Subarray**	Medium	LC 918	2190
2	★ **Longest Turbulent Subarray**	Medium	LC 978	2193
3	★ **Best Sightseeing Pair**	Medium	LC 1014	2195
4	★ **Maximum Subarray Sum with One Deletion**	Medium	LC 1186	2197
5	★ **Maximum Absolute Sum of Any Subarray**	Medium	LC 1749	2200

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Maximum Subarray (Part 1, pattern 5, page 258)

Maximum Product Subarray (Part 1, pattern 13, page 692)

PART 2 · KADANE'S ALGORITHM · PROBLEM 1 OF 5

918. Maximum Sum Circular Subarray

Medium · Queue, Array, Divide and Conquer, Dynamic Programming, Monotonic Queue

leetcode.com/problems/maximum-sum-circular-subarray

★ **TOP 7 PROBLEM** in Kadane's Algorithm: master this one first

PROBLEM

Given a **circular integer array** `nums` of length `n`, return *the maximum possible sum of a non-empty* ***subarray*** *of* `nums`.

A **circular array** means the end of the array connects to the beginning of the array. Formally, the next element of `nums[i]` is `nums[(i + 1) % n]` and the previous element of `nums[i]` is `nums[(i - 1 + n) % n]`.

A **subarray** may only include each element of the fixed buffer `nums` at most once. Formally, for a subarray `nums[i], nums[i + 1], ..., nums[j]`, there does not exist `i <= k1, k2 <= j` with `k1 % n == k2 % n`.

Example 1:

```
Input: nums = [1,-2,3,-2]
Output: 3
Explanation: Subarray [3] has maximum sum 3.
```

Example 2:

```
Input: nums = [5,-3,5]
Output: 10
Explanation: Subarray [5,5] has maximum sum 5 + 5 = 10.
```

Example 3:

```
Input: nums = [-3,-2,-3]
Output: -2
Explanation: Subarray [-2] has maximum sum -2.
```

Constraints:

- `n == nums.length`
- `1 <= n <= 3 * 10^4`
- `-3 * 10^4 <= nums[i] <= 3 * 10^4`

SOLUTION (PYTHON 3)

Solution 1: Maintain Prefix Maximum

How to think about it

A circular maximum subarray is either a normal Kadane segment, or the total sum minus a minimum segment that wraps around. One scan maintains the prefix minimum and prefix maximum to obtain both quantities, then returns the larger of ans and s-smi.

The maximum sum of a circular subarray can be divided into two cases:

- Case 1: The subarray with the maximum sum does not include the circular part, which is the ordinary maximum subarray sum;
- Case 2: The subarray with the maximum sum includes the circular part, which can be transformed into: the total sum of the array minus the minimum subarray sum.

Therefore, we maintain the following variables:

- The minimum prefix sum pmi, initially 0;
- The maximum prefix sum pmx, initially -∞;

- The prefix sum s, initially 0;
- The minimum subarray sum smi, initially ∞;
- The answer ans, initially -∞.

Next, we only need to traverse the array nums. For the current element x we are traversing, we perform the following update operations:

- Update the prefix sum s = s + x;
- Update the answer ans = max(ans, s - pmi), which is the answer for Case 1 (the prefix sum s minus the minimum prefix sum pmi can give the maximum subarray sum);
- Update smi = min(smi, s - pmx), which is the minimum subarray sum for Case 2;
- Update pmi = min(pmi, s), which is the minimum prefix sum;
- Update pmx = max(pmx, s), which is the maximum prefix sum.

After the traversal, we return the maximum value of ans and s - smi as the answer.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Python 3

```python
class Solution:
    def maxSubarraySumCircular(self, nums: List[int]) -> int:
        pmi, pmx = 0, -inf
        ans, s, smi = -inf, 0, inf
        for x in nums:
            s += x
            ans = max(ans, s - pmi)
            smi = min(smi, s - pmx)
            pmi = min(pmi, s)
            pmx = max(pmx, s)
        return max(ans, s - smi)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the start and the length of the best circular
# subarray
# 2. The MINIMUM circular subarray sum


# ============================================================
# FOLLOW-UP 1: Return the start and the length of the best
# circular subarray
# ============================================================

# ---- follow-up 1: function ----
# Answer: Try the best normal subarray and the 'wrap
# around' one (total minus the worst subarray),
# remembering where each begins.
# Time: O(n^2).  Space: O(1).
def best_circular_window(nums):
    n = len(nums)
    best = None
    for start in range(n):
        total = 0
        for length in range(1, n + 1):
            total += nums[(start + length - 1) % n]
            if best is None or total > best[0]:
                best = (total, start, length)
    return best


# ============================================================
# FOLLOW-UP 2: The MINIMUM circular subarray sum
# ============================================================
```

```python

# ---- follow-up 2: function ----
# Answer: Kadane for the smallest sum; it also covers the
# circular case by symmetry (the smallest wrap-around
# equals total minus the biggest middle part).
# Time: O(n).  Space: O(1).
def min_circular_sum(nums):
    total = sum(nums)
    best_here = nums[0]
    best_min = nums[0]
    best_max_here = nums[0]
    best_max = nums[0]
    for x in nums[1:]:
        best_here = min(x, best_here + x)
        best_min = min(best_min, best_here)
        best_max_here = max(x, best_max_here + x)
        best_max = max(best_max, best_max_here)
    if best_max == total:
        return best_min
    return min(best_min, total - best_max)
```

PART 2 · KADANE'S ALGORITHM · PROBLEM 2 OF 5

978. Longest Turbulent Subarray

Medium · Array, Dynamic Programming, Sliding Window

leetcode.com/problems/longest-turbulent-subarray

★ **TOP 7 PROBLEM** in Kadane's Algorithm: master this one first

PROBLEM

Given an integer array `arr`, return *the length of a maximum size turbulent subarray of* `arr`.

A subarray is **turbulent** if the comparison sign flips between each adjacent pair of elements in the subarray.

More formally, a subarray `[arr[i], arr[i + 1], ..., arr[j]]` of `arr` is said to be turbulent if and only if:

- For `i <= k < j`:
  - `arr[k] > arr[k + 1]` when `k` is odd, and
  - `arr[k] < arr[k + 1]` when `k` is even.
- Or, for `i <= k < j`:
  - `arr[k] > arr[k + 1]` when `k` is even, and
  - `arr[k] < arr[k + 1]` when `k` is odd.

Example 1:

```
Input: arr = [9,4,2,10,7,8,8,1,9]
Output: 5
Explanation: arr[1] > arr[2] < arr[3] > arr[4] < arr[5]
```

Example 2:

```
Input: arr = [4,8,12,16]
Output: 2
```

Example 3:

```
Input: arr = [100]
Output: 1
```

Constraints:

- $1 <= arr.length <= 4 * 10^4$
- $0 <= arr[i] <= 10^9$

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

A turbulent subarray alternates strict comparisons. Checking every subarray is quadratic. Ending at i, we only need the longest rise-ending and fall-ending lengths, each extending the opposite state of i-1, or resetting to 1 on a tie. Two rolling variables suffice.

We define f[i] as the length of the longest turbulent subarray ending at nums[i] with an increasing state, and g[i] as the length of the longest turbulent subarray ending at nums[i] with a decreasing state. Initially, f[0] = 1, g[0] = 1. The answer is max(f[i], g[i]).

For i > 0, if nums[i] > nums[i - 1], then f[i] = g[i - 1] + 1, otherwise f[i] = 1; if nums[i] < nums[i - 1], then g[i] = f[i - 1] + 1, otherwise g[i] = 1.

Since f[i] and g[i] are only related to f[i - 1] and g[i - 1], two variables can be used instead of arrays.

The time complexity is O(n), where n is the length of the array. The space complexity is O(1).

Python 3

```python
class Solution:
    def maxTurbulenceSize(self, arr: List[int]) -> int:
        ans = f = g = 1
        for a, b in pairwise(arr):
            ff = g + 1 if a < b else 1
            gg = f + 1 if a > b else 1
            f, g = ff, gg
            ans = max(ans, f, g)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the longest turbulent subarray itself
# 2. Count all turbulent subarrays (length at least 2)


# ============================================================
# FOLLOW-UP 1: Return the longest turbulent subarray
# itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember where the current alternating run
# began.
# Time: O(n).  Space: O(L) for the returned slice.
def longest_turbulent_window(arr):
    best_start, best_length = 0, 1
    start = 0
    for i in range(1, len(arr)):
        if arr[i] == arr[i - 1]:
            start = i
        elif i >= 2 and (arr[i] - arr[i - 1]) * (arr[i - 1] - arr[i - 2]) >= 0:
            start = i - 1
        if i - start + 1 > best_length:
            best_length = i - start + 1
            best_start = start
    return arr[best_start:best_start + best_length]


# ============================================================
# FOLLOW-UP 2: Count all turbulent subarrays (length at
# least 2)
# ============================================================

# ---- follow-up 2: function ----
# Answer: For every right end add the length of the
# turbulent run ending there minus one.
# Time: O(n).  Space: O(1).
def count_turbulent_subarrays(arr):
    total = 0
    run = 1
    for i in range(1, len(arr)):
        if arr[i] == arr[i - 1]:
            run = 1
        elif i >= 2 and (arr[i] - arr[i - 1]) * (arr[i - 1] - arr[i - 2]) < 0:
            run += 1
        else:
            run = 2
        total += run - 1
    return total
```

PART 2 · KADANE'S ALGORITHM · PROBLEM 3 OF 5

1014. Best Sightseeing Pair

Medium · Array, Dynamic Programming

leetcode.com/problems/best-sightseeing-pair

★ **TOP 7 PROBLEM** in Kadane's Algorithm: master this one first

PROBLEM

You are given an integer array `values` where values[i] represents the value of the i^{th} sightseeing spot. Two sightseeing spots `i` and `j` have a **distance** `j - i` between them.

The score of a pair (`i < j`) of sightseeing spots is `values[i] + values[j] + i - j`: the sum of the values of the sightseeing spots, minus the distance between them.

Return *the maximum score of a pair of sightseeing spots*.

Example 1:

```
Input: values = [8,1,5,2,6]
Output: 11
Explanation: i = 0, j = 2, values[i] + values[j] + i - j = 8 + 5 + 0 - 2 = 11
```

Example 2:

```
Input: values = [1,2]
Output: 2
```

Constraints:

- `2 <= values.length <= 5 * 10^4`
- `1 <= values[i] <= 1000`

SOLUTION (PYTHON 3)

Solution 1: Enumeration

How to think about it

Evaluating values[i]+values[j]+i-j for every i<j is quadratic. Rewriting it as (values[i]+i)+(values[j]-j) shows that a fixed j only needs the best values[i]+i to its left. Scanning j from left to right, one variable stores that prefix maximum, giving the best pair at j in constant time before we fold in values[j]+j. Each index is used once as a right endpoint and once as a left candidate, so the scan is linear.

We can enumerate j from left to right while maintaining the maximum value of values[i] + i for elements to the left of j, denoted as mx. For each j, the maximum score is mx + values[j] - j. The answer is the maximum of these maximum scores for all positions.

The time complexity is O(n), where n is the length of the array values. The space complexity is O(1).

Python 3

```python
class Solution:
    def maxScoreSightseeingPair(self, values: List[int]) -> int:
        ans = mx = 0
        for j, x in enumerate(values):
            ans = max(ans, mx + x - j)
            mx = max(mx, x + j)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the indices of the best pair
# 2. The two spots may be at most d apart


# ============================================================
# FOLLOW-UP 1: Return the indices of the best pair
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember where the best values[i] + i came from.
# Time: O(n).  Space: O(1).
def best_pair_indices(values):
    best_left = 0
    best = None
    pair = None
    for j in range(1, len(values)):
        if values[best_left] + best_left < values[j - 1] + j - 1:
            best_left = j - 1
        score = values[best_left] + best_left + values[j] - j
        if best is None or score > best:
            best = score
            pair = (best_left, j)
    return pair


# ============================================================
# FOLLOW-UP 2: The two spots may be at most d apart
# ============================================================

# ---- follow-up 2: function ----
# Answer: A deque keeps the best values[i] + i among the
# last d positions.
# Time: O(n).  Space: O(d).
from collections import deque


def best_pair_within(values, d):
    window = deque()
    best = None
    for j in range(len(values)):
        while window and j - window[0] > d:
            window.popleft()
        if window:
            i = window[0]
            score = values[i] + i + values[j] - j
            if best is None or score > best:
                best = score
        while window and values[window[-1]] + window[-1] <= values[j] + j:
            window.pop()
        window.append(j)
    return best
```

PART 2 · KADANE'S ALGORITHM · PROBLEM 4 OF 5

1186. Maximum Subarray Sum with One Deletion

Medium · Array, Dynamic Programming

leetcode.com/problems/maximum-subarray-sum-with-one-deletion

★ **TOP 7 PROBLEM** in Kadane's Algorithm: master this one first

PROBLEM

Given an array of integers, return the maximum sum for a **non-empty** subarray (contiguous elements) with at most one element deletion. In other words, you want to choose a subarray and optionally delete one element from it so that there is still at least one element left and the sum of the remaining elements is maximum possible.

Note that the subarray needs to be **non-empty** after deleting one element.

Example 1:

```
Input: arr = [1,-2,0,3]
Output: 4
Explanation: Because we can choose [1, -2, 0, 3] and drop -2, thus the subarray [1, 0, 3] becomes the maximum value.
```

Example 2:

```
Input: arr = [1,-2,-2,3]
Output: 3
Explanation: We just choose [3] and it's the maximum sum.
```

Example 3:

```
Input: arr = [-1,-1,-1,-1]
Output: -1
Explanation: The final subarray needs to be non-empty. You can't choose [-1] and delete -1 from it, then get an empty subarray to make the sum equals to 0.
```

Constraints:

- $1 <= arr.length <= 10^5$
- $-10^4 <= arr[i] <= 10^4$

SOLUTION (PYTHON 3)

Solution 1: Preprocessing + Enumeration

How to think about it

A subarray may drop at most one element. Kadane from each end yields the best segment ending or starting at an index; with no deletion the answer is their maximum. Deleting arr[i] adds the best left ending at i-1 to the best right starting at i+1. After filling left and right, enumerate interior deletion points.

We can preprocess the array arr to find the maximum subarray sum ending and starting with each element, storing them in arrays left and right, respectively.

If we do not delete any element, then the maximum subarray sum is the maximum value in left[i] or right[i]; if we delete an element, we can enumerate each position i in [1..n-2], calculate the value of left[i-1] + right[i+1], and take the maximum value.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array arr.

Python 3

```python
class Solution:
    def maximumSum(self, arr: List[int]) -> int:
        n = len(arr)
        left = [0] * n
        right = [0] * n
        s = 0
        for i, x in enumerate(arr):
            s = max(s, 0) + x
            left[i] = s
        s = 0
        for i in range(n - 1, -1, -1):
            s = max(s, 0) + arr[i]
            right[i] = s
        ans = max(left)
        for i in range(1, n - 1):
            ans = max(ans, left[i - 1] + right[i + 1])
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Exactly one element MUST be deleted from the chosen
# subarray
# 2. Delete up to k elements


# ========================================================
# FOLLOW-UP 1: Exactly one element MUST be deleted from
# the chosen subarray
# ========================================================

# ---- follow-up 1: function ----
# Answer: Best sum ending just before the deleted element
# plus best sum starting just after it (at least one side
# must be non-empty).
# Time: O(n).  Space: O(n).
def max_sum_must_delete(arr):
    n = len(arr)
    left = [0] * n                      # best sum of a subarray ending at i
    right = [0] * n                     # best sum of a subarray starting at i
    for i in range(n):
        left[i] = arr[i] + (max(left[i - 1], 0) if i else 0)
    for i in range(n - 1, -1, -1):
        right[i] = arr[i] + (max(right[i + 1], 0) if i < n - 1 else 0)
    best = None
    for i in range(n):
        options = []
        if i > 0:
            options.append(left[i - 1])
        if i < n - 1:
            options.append(right[i + 1])
        if 0 < i < n - 1:
            options.append(left[i - 1] + right[i + 1])
        for value in options:
            if best is None or value > best:
                best = value
    return best


# ========================================================
# FOLLOW-UP 2: Delete up to k elements
# ========================================================

# ---- follow-up 2: function ----
# Answer: Table g[i][d]: best sum of a window ending at i
```

```
# that used at most d deletions (the window keeps at least
# one element).
# Time: O(n * k).  Space: O(n * k).
def max_sum_with_k_deletions(arr, k):
    n = len(arr)
    neg = float("-inf")
    g = [[neg] * (k + 1) for _ in range(n)]
    answer = neg
    for i in range(n):
        for d in range(k + 1):
            best = arr[i]                                  # start a new window at i
            if i > 0:
                best = max(best, g[i - 1][d] + arr[i])     # keep arr[i]
                if d > 0:
                    best = max(best, g[i - 1][d - 1])      # delete arr[i]
```

```
            g[i][d] = best
        answer = max(answer, g[i][k])
    return answer
```

1749. Maximum Absolute Sum of Any Subarray

Medium · Array, Dynamic Programming

leetcode.com/problems/maximum-absolute-sum-of-any-subarray

★ **TOP 7 PROBLEM** in Kadane's Algorithm: master this one first

PROBLEM

You are given an integer array `nums`. The **absolute sum** of a subarray $[nums_l, nums_{l+1}, ..., nums_{r-1}, nums_r]$ is $abs(nums_l + nums_{l+1} + ... + nums_{r-1} + nums_r)$.

Return *the **maximum** absolute sum of any **(possibly empty)** subarray of* `nums`.

Note that `abs(x)` is defined as follows:

- If `x` is a negative integer, then `abs(x) = -x`.
- If `x` is a non-negative integer, then `abs(x) = x`.

Example 1:

```
Input: nums = [1,-3,2,3,-4]
Output: 5
Explanation: The subarray [2,3] has absolute sum = abs(2+3) = abs(5) = 5.
```

Example 2:

```
Input: nums = [2,-5,1,-4,3,-2]
Output: 8
Explanation: The subarray [-5,1,-4] has absolute sum = abs(-5+1-4) = abs(-8) = 8.
```

Constraints:

- $1 <= nums.length <= 10^5$
- $-10^4 <= nums[i] <= 10^4$

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

The maximum absolute subarray sum is the larger of the maximum subarray sum and the absolute minimum subarray sum. Kadane tracks the best and worst sums f,g ending here; the answer is the global max of f and |g|.

We define f[i] to represent the maximum value of the subarray ending with nums[i], and define g[i] to represent the minimum value of the subarray ending with nums[i]. Then the state transition equation of f[i] and g[i] is as follows:

aligned f[i] &= max(f[i - 1], 0) + nums[i] \ g[i] &= min(g[i - 1], 0) + nums[i] aligned

The final answer is the maximum value of max(f[i], |g[i]|).

Since f[i] and g[i] are only related to f[i - 1] and g[i - 1], we can use two variables to replace the array, reducing the space complexity to O(1).

Time complexity O(n), space complexity O(1), where n is the length of the array nums.

Python 3

```python
class Solution:
    def maxAbsoluteSum(self, nums: List[int]) -> int:
        f = g = 0
        ans = 0
        for x in nums:
            f = max(f, 0) + x
            g = min(g, 0) + x
            ans = max(ans, f, abs(g))
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subarray that gives the maximum absolute
# sum
# 2. The smallest absolute sum of a non-empty subarray


# ============================================================
# FOLLOW-UP 1: Return the subarray that gives the maximum
# absolute sum
# ============================================================

# ---- follow-up 1: function ----
# Answer: Prefix sums: the best subarray goes from the
# position of the smallest prefix to the biggest one (or
# the other way round).
# Time: O(n).  Space: O(n).
def best_absolute_window(nums):
    prefix = [0]
    for x in nums:
        prefix.append(prefix[-1] + x)
    low = prefix.index(min(prefix))
    high = prefix.index(max(prefix))
    start, end = min(low, high), max(low, high)
    if start == end:                  # every prefix is equal: all numbers are zero
        return nums[:1]
    return nums[start:end]


# ============================================================
# FOLLOW-UP 2: The smallest absolute sum of a non-empty
# subarray
# ============================================================

# ---- follow-up 2: function ----
# Answer: Sort the prefix sums; the closest two are the
# best pair.
# Time: O(n log n).  Space: O(n).
def smallest_absolute_sum(nums):
    prefix = [0]
    for x in nums:
        prefix.append(prefix[-1] + x)
    prefix.sort()
    return min(prefix[i + 1] - prefix[i] for i in range(len(prefix) - 1))
```

PART 2 · TOPIC 10 OF 50

Matrix (2D Array)

1 problem · 0 Easy · 1 Medium · 0 Hard

THE IDEA

Work with row and column indexes, layers, or directions; for checks, use sets keyed by row, column and box.

THE 1 PROBLEMS, EASY TO HARD | ★ = TOP 1: DO THESE FIRST

1	★ **Valid Sudoku**	Medium	LC 36	2203

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Transpose Matrix (Part 1, pattern 18, page 1003)
Spiral Matrix II (Part 1, pattern 18, page 1009)
Rotate Image (Part 1, pattern 18, page 1012)
Candy Crush (Part 1, pattern 36, page 1866)
Spiral Matrix (Part 1, pattern 18, page 1005)
Diagonal Traverse (Part 1, pattern 18, page 1019)
Set Matrix Zeroes (Part 1, pattern 18, page 1015)
Game of Life (Part 1, pattern 18, page 1022)

36. Valid Sudoku

Medium · Array, Hash Table, Matrix

leetcode.com/problems/valid-sudoku

★ **TOP 7 PROBLEM** in Matrix (2D Array): master this one first

PROBLEM

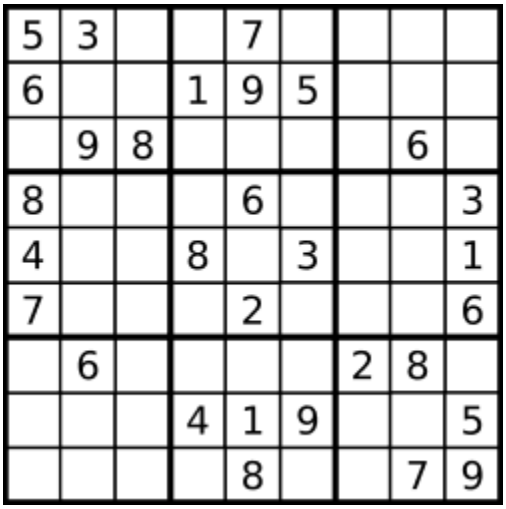
Determine if a `9 x 9` Sudoku board is valid. Only the filled cells need to be validated **according to the following rules**:

1. Each row must contain the digits `1-9` without repetition.
2. Each column must contain the digits `1-9` without repetition.
3. Each of the nine `3 x 3` sub-boxes of the grid must contain the digits `1-9` without repetition.

Note:

- A Sudoku board (partially filled) could be valid but is not necessarily solvable.
- Only the filled cells need to be validated according to the mentioned rules.

Example 1:




```
Input: board =
[["5","3",".",".","7",".",".",".","."]
,["6",".",".","1","9","5",".",".","."]
,[".","9","8",".",".",".",".","6","."]
,["8",".",".",".","6",".",".",".","3"]
,["4",".",".","8",".","3",".",".","1"]
,["7",".",".",".","2",".",".",".","6"]
,[".","6",".",".",".",".",".","2","8","."]
,[".",".",".","4","1","9",".",".","5"]
,[".",".",".",".","8",".",".","7","9"]]
Output: true
```

Example 2:

```
Input: board =
[["8","3",".",".","7",".",".",".","."]
,["6",".",".","1","9","5",".",".","."]
,[".","9","8",".",".",".",".","6","."]
,["8",".",".",".","6",".",".",".","3"]
,["4",".",".","8",".","3",".",".","1"]
,["7",".",".",".","2",".",".",".","6"]
,[".","6",".",".",".",".","2","8","."]
,[".",".",".","4","1","9",".",".","5"]
,[".",".",".",".","8",".",".","7","9"]]
Output: false
Explanation: Same as Example 1, except with the 5 in the top left corner being modified to 8. Since there are two 8's in the top left 3x3 sub-box, it is invalid.
```

Constraints:

- `board.length == 9`
- `board[i].length == 9`
- `board[i][j]` is a digit `1-9` or `'.'`.

SOLUTION (PYTHON 3)

Solution 1: Traversal once

How to think about it

The first idea is, for each filled digit, to rescan its row, column, and box. The board is only 9 × 9, so that would pass. The bottleneck is rescanning the same row, column, and box. We only need to remember whether digit v has already appeared in that unit. One pass is enough: three boolean tables row[i][v], col[j][v], and sub[k][v], with box index k = floor(i/3) × 3 + floor(j/3). A second hit of the same digit returns false immediately.

The valid sudoku satisfies the following three conditions:

- The digits are not repeated in each row;
- The digits are not repeated in each column;
- The digits are not repeated in each 3 × 3 box.

Traverse the sudoku, for each digit, check whether the row, column and 3 × 3 box it is in have appeared the digit. If it is, return `false`. If the traversal is over, return `true`.

The time complexity is O(C) and the space complexity is O(C), where C is the number of empty spaces in the sudoku. In this question, C=81.

Python 3

```python
class Solution:
    def isValidSudoku(self, board: List[List[str]]) -> bool:
        row = [[False] * 9 for _ in range(9)]
        col = [[False] * 9 for _ in range(9)]
        sub = [[False] * 9 for _ in range(9)]
        for i in range(9):
            for j in range(9):
                c = board[i][j]
                if c == '.':
                    continue
                num = int(c) - 1
                k = i // 3 * 3 + j // 3
                if row[i][num] or col[j][num] or sub[k][num]:
                    return False
                row[i][num] = True
                col[j][num] = True
                sub[k][num] = True
        return True
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Report the first conflicting cell instead of True /
# False
# 2. Which digits could still go into an empty cell


# ============================================================
# FOLLOW-UP 1: Report the first conflicting cell instead
# of True / False
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same row, column and box sets; return the cell
# where a repeat is found.
# Time: O(81) = O(1).  Space: O(81) = O(1).
def first_conflict(board):
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [set() for _ in range(9)]
    for r in range(9):
        for c in range(9):
            d = board[r][c]
            if d == ".":
                continue
            b = r // 3 * 3 + c // 3
            if d in rows[r] or d in cols[c] or d in boxes[b]:
                return (r, c)
            rows[r].add(d)
            cols[c].add(d)
            boxes[b].add(d)
    return None


# ============================================================
# FOLLOW-UP 2: Which digits could still go into an empty
# cell
# ============================================================

# ---- follow-up 2: function ----
# Answer: Remove the digits already in its row, column and
# box.
# Time: O(1).  Space: O(1).
def candidates_for_cell(board, r, c):
    used = set(board[r]) | {board[i][c] for i in range(9)}
    br, bc = r // 3 * 3, c // 3 * 3
    for i in range(br, br + 3):
        for j in range(bc, bc + 3):
            used.add(board[i][j])
    return sorted(set("123456789") - used)
```

PART 2 · TOPIC 11 OF 50

Linked List

3 problems · 0 Easy · 3 Medium · 0 Hard

THE IDEA

Dummy head nodes, two pointers with a gap, and rewiring next pointers carefully in the right order.

THE 3 PROBLEMS, EASY TO HARD | ★ = TOP 3: DO THESE FIRST

1	★ **Remove Duplicates from Sorted List II**	Medium	LC 82	2207
2	★ **Delete Node in a Linked List**	Medium	LC 237	2210
3	★ **Insert into a Sorted Circular Linked List**	Medium	LC 708	2214

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Intersection of Two Linked Lists (Part 1, pattern 24, page 1279)

Design Linked List (Part 1, pattern 25, page 1360)

Remove Nth Node From End of List (Part 1, pattern 24, page 1290)

Swap Nodes in Pairs (Part 1, pattern 25, page 1341)

Partition List (Part 1, pattern 24, page 1318)

Reorder List (Part 1, pattern 24, page 1293)

Add Two Numbers II (Part 1, pattern 25, page 1347)

Merge In Between Linked Lists (Part 1, pattern 25, page 1377)

Remove Duplicates from Sorted List (Part 1, pattern 24, page 1283)

Delete the Middle Node of a Linked List (Part 1, pattern 24, page 1303)

Odd Even Linked List (Part 1, pattern 24, page 1315)

Copy List with Random Pointer (Part 1, pattern 25, page 1351)

Rotate List (Part 1, pattern 24, page 1306)

Add Two Numbers (Part 1, pattern 25, page 1344)

Flatten a Multilevel Doubly Linked List (Part 1, pattern 25, page 1356)

82. Remove Duplicates from Sorted List II

Medium · Linked List, Two Pointers

leetcode.com/problems/remove-duplicates-from-sorted-list-ii

★ **TOP 7 PROBLEM** in Linked List: master this one first

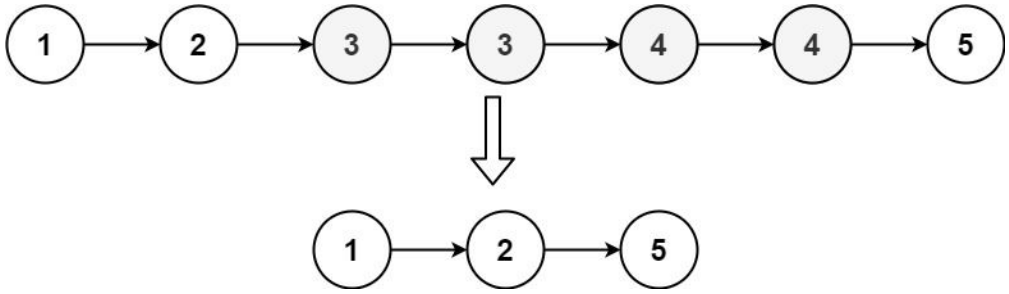
PROBLEM

You are given the `head` of a **sorted** linked list.

Delete all nodes that have **duplicate** numbers, leaving only **distinct** numbers from the original list.

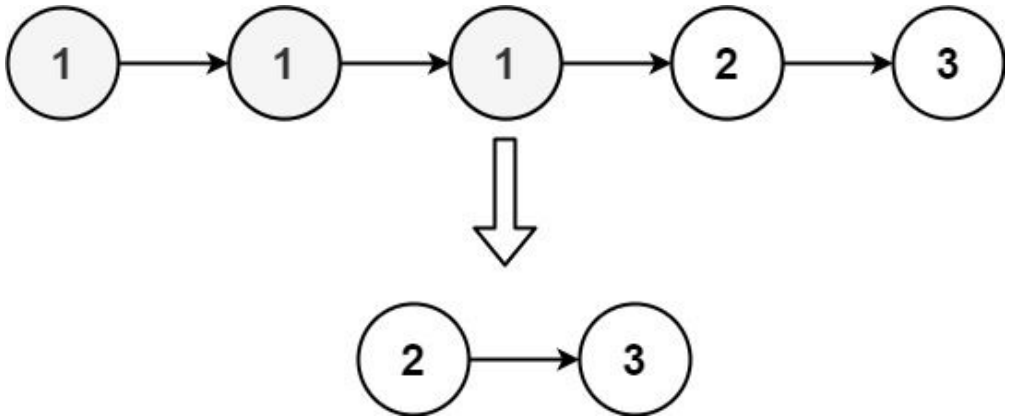
Return the linked list **sorted** as well.

Example 1:




```
Input: head = [1,2,3,3,4,4,5]
Output: [1,2,5]
```

Example 2:




```
Input: head = [1,1,1,2,3]
Output: [2,3]
```

Constraints:

- The number of nodes in the list is in the range `[0, 300]`.
- `-100 <= Node.val <= 100`
- The list is guaranteed to be **sorted** in ascending order.

SOLUTION (PYTHON 3)

Solution 1: Single Pass

How to think about it

The first idea is count frequencies, then rebuild a list of values that appear once. Correct, and n ≤ 300 is tiny, but that wastes “already sorted, so duplicates are adjacent” and uses extra O(n) space. The bottleneck is not using adjacency. Equal values form a contiguous run; one pass decides whether the whole run stays. The head may be deleted, so we need a dummy. And when a run is dropped, the predecessor must not advance into it. Keep pre on the last kept node and slide cur over the current value: if pre.next is still cur, the run has length one and pre moves; otherwise skip the whole run.

First, we create a dummy head node dummy, and set dummy.next = head. Then we create a pointer pre pointing to dummy, and a pointer cur pointing to head, and start traversing the linked list.

When the node value pointed by cur is the same as the node value pointed by cur.next, we let cur keep moving forward until the node value pointed by cur is different from the node value pointed by cur.next. At this point, we check whether pre.next is equal to cur. If they are equal, it means there are no duplicate nodes between pre and cur, so we move pre to the position of cur; otherwise, it means there are duplicate nodes between pre and cur, so we set pre.next to cur.next. Then we continue to move cur forward. Continue the above operation until cur is null, and the traversal ends.

Finally, return dummy.next.

The time complexity is O(n), and the space complexity is O(1). Here, n is the length of the linked list.

Python 3

```python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def deleteDuplicates(self, head: Optional[ListNode]) -> Optional[ListNode]:
        dummy = pre = ListNode(next=head)
        cur = head
        while cur:
            while cur.next and cur.next.val == cur.val:
                cur = cur.next
            if pre.next == cur:
                pre = cur
            else:
                pre.next = cur.next
            cur = cur.next
        return dummy.next
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Keep each value at most twice
# 2. The list is NOT sorted: remove every value that
# appears more than once


# ========================================================
# FOLLOW-UP 1: Keep each value at most twice
# ========================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

```
# ---- follow-up 1: function ----
# Answer: Count the run length of every value and copy at
# most two nodes of the run.
# Time: O(n).  Space: O(n).
def keep_at_most_two(head):
    dummy = ListNode(0)
    tail = dummy
    node = head
    while node:
        run = 0
        value = node.val
        while node and node.val == value:
            if run < 2:
                tail.next = ListNode(value)
                tail = tail.next
            run += 1
            node = node.next
    return dummy.next


# ============================================================
# FOLLOW-UP 2: The list is NOT sorted: remove every value
# that appears more than once
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the values first, then keep only the nodes
# whose value appears once.
# Time: O(n).  Space: O(n).
def remove_all_repeated_unsorted(head):
    count = {}
    node = head
    while node:
        count[node.val] = count.get(node.val, 0) + 1
        node = node.next
    dummy = ListNode(0)
    tail = dummy
    node = head
    while node:
        if count[node.val] == 1:
            tail.next = ListNode(node.val)
            tail = tail.next
        node = node.next
    return dummy.next
```

237. Delete Node in a Linked List

Medium · Linked List

leetcode.com/problems/delete-node-in-a-linked-list

★ **TOP 7 PROBLEM** in Linked List: master this one first

PROBLEM

There is a singly-linked list `head` and we want to delete a node `node` in it.

You are given the node to be deleted `node`. You will **not be given access** to the first node of `head`.

All the values of the linked list are **unique**, and it is guaranteed that the given node `node` is not the last node in the linked list.

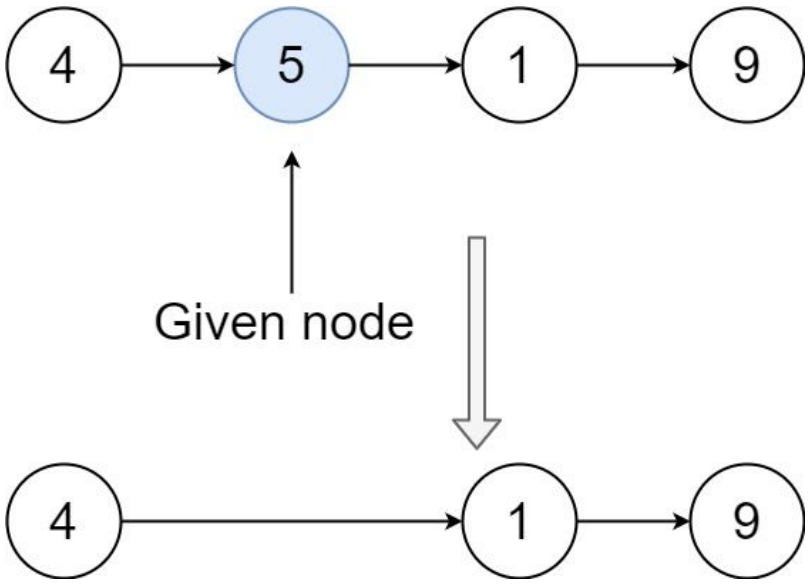
Delete the given node. Note that by deleting the node, we do not mean removing it from memory. We mean:

- The value of the given node should not exist in the linked list.
- The number of nodes in the linked list should decrease by one.
- All the values before `node` should be in the same order.
- All the values after `node` should be in the same order.

Custom testing:

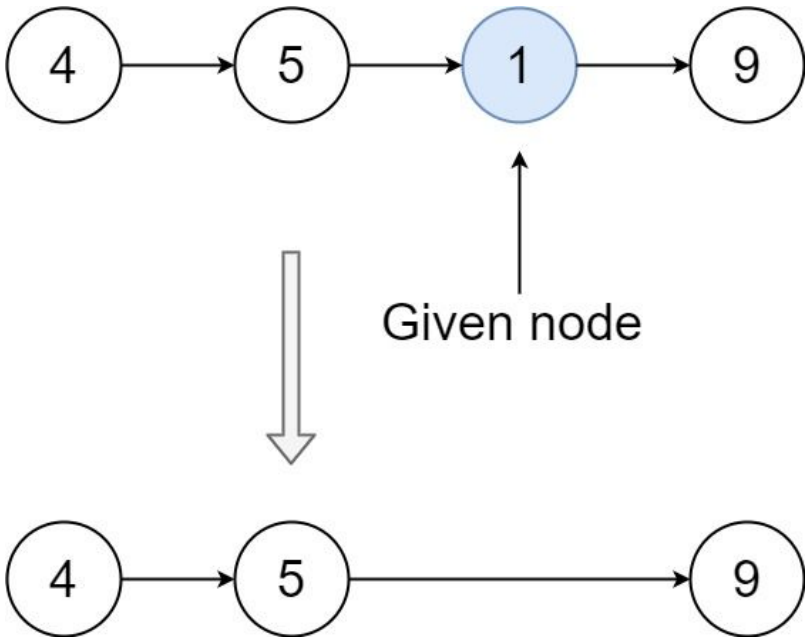
- For the input, you should provide the entire linked list `head` and the node to be given `node`. `node` should not be the last node of the list and should be an actual node in the list.
- We will build the linked list and pass the node to your function.
- The output will be the entire list after calling your function.

Example 1:




```
Input: head = [4,5,1,9], node = 5
Output: [4,1,9]
Explanation: You are given the second node with value 5, the linked list should become 4 -> 1 -> 9
after calling your function.
```

Example 2:




```
Input: head = [4,5,1,9], node = 1
Output: [4,5,9]
Explanation: You are given the third node with value 1, the linked list should become 4 -> 5 -> 9 after calling your function.
```

Constraints:

- The number of the nodes in the given list is in the range `[2, 1000]`.
- `-1000 <= Node.val <= 1000`
- The value of each node in the list is **unique**.
- The `node` to be deleted is **in the list** and is **not a tail** node.

SOLUTION (PYTHON 3)

Solution 1: Node assignment

> **How to think about it**
>
> We are given only the node to delete, so the predecessor's next cannot be rewritten. Copy the successor's value into the current node and skip the successor, which is equivalent to deleting it.

We can replace the value of the current node with the value of the next node, and then delete the next node. This can achieve the purpose of deleting the current node.

Time complexity O(1), space complexity O(1).

Python 3

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None


class Solution:
    def deleteNode(self, node):
        """
        :type node: ListNode
        :rtype: void Do not return anything, modify node in-place instead.
        """
        node.val = node.next.val
        node.next = node.next.next
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. You have the head of the list and the node to delete
# 2. Delete every node with a given value


# ============================================================
# FOLLOW-UP 1: You have the head of the list and the node
# to delete
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 1: function ----
# Answer: Walk to the node before it and skip it (a head
# node needs special care).
# Time: O(n).  Space: O(1).
def delete_node_with_head(head, target):
    if head is target:
        return head.next
    node = head
    while node.next and node.next is not target:
        node = node.next
    if node.next:
        node.next = node.next.next
    return head


# ============================================================
# FOLLOW-UP 2: Delete every node with a given value
# ============================================================

# ---- follow-up 2: function ----
# Answer: Use a dummy head so that the first node is not
# special.
# Time: O(n).  Space: O(1).
def delete_all_with_value(head, value):
    dummy = ListNode(0, head)
    node = dummy
    while node.next:
        if node.next.val == value:
            node.next = node.next.next
        else:
            node = node.next
```

```
    return dummy.next
```

708. Insert into a Sorted Circular Linked List

Medium · Linked List

leetcode.com/problems/insert-into-a-sorted-circular-linked-list

★ **TOP 7 PROBLEM** in Linked List: master this one first

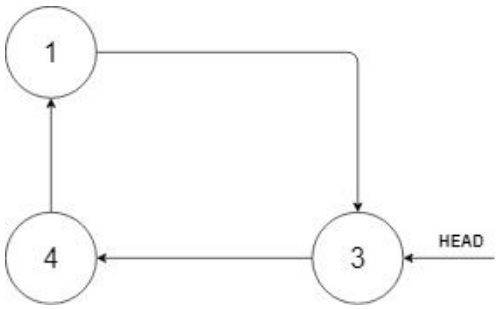
PROBLEM

Given a Circular Linked List node, which is sorted in non-descending order, write a function to insert a value `insertVal` into the list such that it remains a sorted circular list. The given node can be a reference to any single node in the list and may not necessarily be the smallest value in the circular list.

If there are multiple suitable places for insertion, you may choose any place to insert the new value. After the insertion, the circular list should remain sorted.

If the list is empty (i.e., the given node is `null`), you should create a new single circular list and return the reference to that single node. Otherwise, you should return the originally given node.

Example 1:




```
Input: head = [3,4,1], insertVal = 2
Output: [3,4,1,2]
Explanation: In the figure above, there is a sorted circular list of three elements. You are given a reference to the node with value 3, and we need to insert 2 into the list. The new node should be inserted between node 1 and node 3. After the insertion, the list should look like this, and we should still return node 3.
```

Example 2:

```
Input: head = [], insertVal = 1
Output: [1]
Explanation: The list is empty (given head is null). We create a new single circular list and return the reference to that single node.
```

Example 3:

```
Input: head = [1], insertVal = 0
Output: [1,0]
```

Constraints:

- The number of nodes in the list is in the range $[0, 5 * 10^4]$.
- $-10^6 <= Node.val, insertVal <= 10^6$

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Insert into a non-decreasing circular list. Up to 5×10^4 nodes, so we must find the gap in one lap. An empty list becomes a self-loop. A sorted ring has a single drop from the maximum back to the minimum. The insert either sits inside a non-decreasing run, or straddles that drop when the value is at least the max or at most the min. Walk adjacent prev and curr until one of those holds; if the list is globally non-decreasing we land on the last pair. Link the new node and return the original head.

Python 3

```
"""
# Definition for a Node.
class Node:
    def __init__(self, val=None, next=None):
        self.val = val
        self.next = next
"""


class Solution:
    def insert(self, head: 'Optional[Node]', insertVal: int) -> 'Node':
        node = Node(insertVal)
        if head is None:
            node.next = node
            return node
        prev, curr = head, head.next
        while curr != head:
            if prev.val <= insertVal <= curr.val or (
                prev.val > curr.val and (insertVal >= prev.val or insertVal <= curr.val)
            ):
                break
            prev, curr = curr, curr.next
        prev.next = node
        node.next = curr
        return head
```

Complexity. Time: O(n). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Delete a value from a sorted circular list
# 2. Read the circular list as a plain list starting from
# its smallest value


# ============================================================
# FOLLOW-UP 1: Delete a value from a sorted circular list
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- follow-up 1: function ----
# Answer: Find the node before the value, skip it, and
# keep the circle (watch the case of a single node).
# Time: O(n).  Space: O(1).
def remove_from_circular(head, value):
```

```
    if head is None:
        return None
    if head.next is head:
        return None if head.val == value else head
    previous = head
    while previous.next is not head:             # the last node of the circle
        previous = previous.next
    node = head
    while True:
        if node.val == value:
            previous.next = node.next
            return node.next if node is head else head
        previous, node = node, node.next
        if node is head:
            return head


# ============================================================
# FOLLOW-UP 2: Read the circular list as a plain list
# starting from its smallest value
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk once around, then rotate the list to the
# place where it drops.
# Time: O(n).  Space: O(n).
def circular_to_sorted_list(head):
    if head is None:
        return []
    values = [head.val]
    node = head.next
    while node is not head:
        values.append(node.val)
        node = node.next
    start = 0
    for i in range(1, len(values)):
        if values[i] < values[i - 1]:              # the place where the circle wraps from the biggest to
the smallest
            start = i
    return values[start:] + values[:start]
```

PART 2 · TOPIC 12 OF 50

Stacks

5 problems · 1 Easy · 3 Medium · 1 Hard

THE IDEA

A stack remembers the unfinished work: opening brackets, pending operands, or the last candidate you may still undo.

THE 5 PROBLEMS, EASY TO HARD | ★ = TOP 4: DO THESE FIRST

1	Maximum Nesting Depth of the Parentheses	Easy	LC 1614	2218
2	★ **Valid Parenthesis String**	Medium	LC 678	2221
3	★ **Minimum Remove to Make Valid Parentheses**	Medium	LC 1249	2225
4	★ **Removing Stars From a String**	Medium	LC 2390	2228
5	★ **Longest Valid Parentheses**	Hard	LC 32	2230

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Valid Parentheses (Part 1, pattern 6, page 292)

Remove All Adjacent Duplicates in String (Part 1, pattern 8, page 388)

Asteroid Collision (Part 1, pattern 8, page 409)

Validate Stack Sequences (Part 1, pattern 8, page 429)

Simplify Path (Part 1, pattern 6, page 320)

Evaluate Reverse Polish Notation (Part 1, pattern 8, page 401)

Basic Calculator (Part 1, pattern 6, page 344)

Build a String From Another String — substitute: Baseball Game (Part 1, pattern 8, page 395)

Min Stack (Part 1, pattern 8, page 398)

Car Fleet (Part 1, pattern 9, page 449)

Remove Duplicate Letters (Part 1, pattern 9, page 438)

Exclusive Time of Functions (Part 1, pattern 8, page 422)

Basic Calculator II (Part 1, pattern 8, page 412)

PART 2 · STACKS · PROBLEM 1 OF 5

1614. Maximum Nesting Depth of the Parentheses

Easy · Stack, String, Parentheses

leetcode.com/problems/maximum-nesting-depth-of-the-parentheses

PROBLEM

Given a **valid parentheses string** `s`, return the **nesting depth** of `s`. The nesting depth is the **maximum** number of nested parentheses.

Example 1:

Input: s = "(1+(2*3)+((8)/4))+1"

Output: 3

Explanation:

Digit 8 is inside of 3 nested parentheses in the string.

Example 2:

Input: s = "(1)+((2))+(((3)))"

Output: 3

Explanation:

Digit 3 is inside of 3 nested parentheses in the string.

Example 3:

Input: s = "()(())((()()))"

Output: 3

Constraints:

- `1 <= s.length <= 100`
- `s` consists of digits `0-9` and characters `'+'`, `'-'`, `'*'`, `'/'`, `'('`, and `')'`.
- It is guaranteed that parentheses expression `s` is a VPS.

SOLUTION (PYTHON 3)

Solution 1: Traversal

> **How to think about it**
>
> The string is valid and at most length 100. Nesting depth is the maximum number of unmatched opening parentheses during a scan. Digits and operators do not affect depth; only parentheses matter. A counter d increases on `(`, updates the answer, and decreases on `)`. One pass suffices.

We use a variable d to record the current depth, initially d = 0.

Traverse the string s. When encountering a left parenthesis, increment the depth d by one and update the answer to be the maximum of the current depth d and the answer. When encountering a right parenthesis, decrement the depth d by one.

Finally, return the answer.

The time complexity is O(n), where n is the length of the string s. The space complexity is O(1).

Python 3

```python
class Solution:
    def maxDepth(self, s: str) -> int:
        ans = d = 0
        for c in s:
            if c == '(':
                d += 1
                ans = max(ans, d)
            elif c == ')':
                d -= 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Three kinds of brackets (), [], {}: the depth, or -1
# if the text is not valid
# 2. Positions of all the places where the deepest nesting
# is reached


# ============================================================
# FOLLOW-UP 1: Three kinds of brackets (), [], {}: the
# depth, or -1 if the text is not valid
# ============================================================

# ---- follow-up 1: function ----
# Answer: A stack of opening brackets; its largest size is
# the depth.
# Time: O(n).  Space: O(n).
def max_depth_three_kinds(s):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    best = 0
    for ch in s:
        if ch in "([{":
            stack.append(ch)
            best = max(best, len(stack))
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return -1
    return best if not stack else -1


# ============================================================
# FOLLOW-UP 2: Positions of all the places where the
# deepest nesting is reached
# ============================================================

# ---- follow-up 2: function ----
# Answer: Track the depth and record the index of every
# '(' that reaches the maximum.
# Time: O(n).  Space: O(p), p = deepest positions.
def deepest_positions(s):
    depth = 0
    best = 0
    positions = []
    for i, ch in enumerate(s):
        if ch == "(":
            depth += 1
            if depth > best:
                best = depth
                positions = []
            if depth == best and depth > 0:
                positions.append(i)
        elif ch == ")":
```

```
            depth -= 1
    return best, positions
```

PART 2 · STACKS · PROBLEM 2 OF 5

678. Valid Parenthesis String

Medium · Stack, Greedy, String, Dynamic Programming, Parentheses

leetcode.com/problems/valid-parenthesis-string

★ **TOP 7 PROBLEM** in Stacks: master this one first

PROBLEM

Given a string `s` containing only three types of characters: `'('`, `')'` and `'*'`, return `true` *if* `s` *is* ***valid***.

The following rules define a **valid** string:

- Any left parenthesis `'('` must have a corresponding right parenthesis `')'`.
- Any right parenthesis `')'` must have a corresponding left parenthesis `'('`.
- Left parenthesis `'('` must go before the corresponding right parenthesis `')'`.
- `'*'` could be treated as a single right parenthesis `')'` or a single left parenthesis `'('` or an empty string `""`.

Example 1:

```
Input: s = "()"
Output: true
```

Example 2:

```
Input: s = "(*)"
Output: true
```

Example 3:

```
Input: s = "(*))"
Output: true
```

Example 4:

```
Input: s = "("
Output: false
```

Constraints:

- `1 <= s.length <= 100`
- `s[i]` is `'('`, `')'` or `'*'`.

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

`*` can be a left, right, or empty token. Backtracking three choices is heavy for length 100. Interval DP: dp[i][j] is whether s[i..j] is valid. A single `*` is valid; a longer span is a matching pair around a valid inside, or a split into two valid pieces.

Let `dp[i][j]` be true if and only if the interval `s[i], s[i+1], ..., s[j]` can be made valid. Then `dp[i][j]` is true only if:

- `s[i]` is `'*'`, and the interval `s[i+1], s[i+2], ..., s[j]` can be made valid;
- or, `s[i]` can be made to be `'('`, and there is some `k` in `[i+1, j]` such that `s[k]` can be made to be `')'`, plus the two intervals cut by `s[k]` (`s[i+1: k] and s[k+1: j+1]`) can be made valid;
- Time Complexity: $O(n^3)$, where n is the length of the string. There are $O(n^2)$ states corresponding to entries of dp, and we do an average of O(n) work on each state.

- Space Complexity: O(n^2).

Python 3

```python
class Solution:
    def checkValidString(self, s: str) -> bool:
        n = len(s)
        dp = [[False] * n for _ in range(n)]
        for i, c in enumerate(s):
            dp[i][i] = c == '*'
        for i in range(n - 2, -1, -1):
            for j in range(i + 1, n):
                dp[i][j] = (
                    s[i] in '(*' and s[j] in '*)' and (i + 1 == j or dp[i + 1][j - 1])
                )
                dp[i][j] = dp[i][j] or any(
                    dp[i][k] and dp[k + 1][j] for k in range(i, j)
                )
        return dp[0][-1]
```

Solution 2: Greedy

> **How to think about it**
>
> Cubic DP is more than we need. Scan left treating `(` and `*` as stock for `)`; scan right treating `)` and `*` as stock for `(`. Both passes must succeed.

Scan twice, first from left to right to make sure that each of the closing brackets is matched successfully, and second from right to left to make sure that each of the opening brackets is matched successfully.

- Time Complexity: O(n), where n is the length of the string.
- Space Complexity: O(1).

Python 3

```python
class Solution:
    def checkValidString(self, s: str) -> bool:
        x = 0
        for c in s:
            if c in '(*':
                x += 1
            elif x:
                x -= 1
            else:
                return False
        x = 0
        for c in s[::-1]:
            if c in '*)':
                x += 1
            elif x:
                x -= 1
            else:
                return False
        return True
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return one valid string (replace every '*' by '(',
# ')' or nothing)
# 2. The minimum number of parentheses to add to make a
# string with stars valid


# ========================================================
# FOLLOW-UP 1: Return one valid string (replace every '*'
```

```
# by '(', ')' or nothing)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Match every ')' with an earlier '(' or, if none
# is free, with an earlier '*' that becomes '('; then
# close each left-over '(' with a later '*' that becomes
# ')'. The other stars become empty.
# Time: O(n^2) in the worst case (pop(0) and the star
# search).  Space: O(n).
def make_valid(s):
    opens, stars = [], []
    role = {}
    for i, ch in enumerate(s):
        if ch == "(":
            opens.append(i)
        elif ch == "*":
            stars.append(i)
        else:
            if opens:
                opens.pop()
            elif stars:
                role[stars.pop(0)] = "("           # a star before the ')' becomes '('; take the earliest,
 later ones are more useful below
            else:
                return None
    while opens:
        o = opens.pop()
        star = None
        for k in range(len(stars) - 1, -1, -1):
            if stars[k] > o:
                star = stars.pop(k)
                break
        if star is None:
            return None
        role[star] = ")"
    result = []
    for i, ch in enumerate(s):
        if ch == "*":
            result.append(role.get(i, ""))
        else:
            result.append(ch)
    return "".join(result)


# ============================================================
# FOLLOW-UP 2: The minimum number of parentheses to add to
# make a string with stars valid
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the range [low, high] of possible open
```

```
# counts; whenever high would be negative we must add an
# opening bracket, and what is left of low must be closed.
# Time: O(n).  Space: O(1).
def min_additions_with_stars(s):
    low = high = 0
    added = 0
    for ch in s:
        if ch == "(":
            low += 1
            high += 1
        elif ch == ")":
            low -= 1
            high -= 1
        else:
            low -= 1
            high += 1
```

```
        if high < 0:
            added += 1
            high = 0
            low = 0
        low = max(low, 0)
    return added + low
```

PART 2 · STACKS · PROBLEM 3 OF 5

1249. Minimum Remove to Make Valid Parentheses

Medium · Stack, String

leetcode.com/problems/minimum-remove-to-make-valid-parentheses

★ **TOP 7 PROBLEM** in Stacks: master this one first

PROBLEM

Given a string s of `'('` , `')'` and lowercase English characters.

Your task is to remove the minimum number of parentheses (`'('` or `')'`, in any positions) so that the resulting *parentheses string* is valid and return **any** valid string.

Formally, a *parentheses string* is valid if and only if:

- It is the empty string, contains only lowercase characters, or
- It can be written as `AB` (`A` concatenated with `B`), where `A` and `B` are valid strings, or
- It can be written as `(A)`, where `A` is a valid string.

Example 1:

```
Input: s = "lee(t(c)o)de)"
Output: "lee(t(c)o)de"
Explanation: "lee(t(co)de)" , "lee(t(c)ode)" would also be accepted.
```

Example 2:

```
Input: s = "a)b(c)d"
Output: "ab(c)d"
```

Example 3:

```
Input: s = "))(("
Output: ""
Explanation: An empty string is also valid.
```

Constraints:

- `1 <= s.length <= 10^5`
- `s[i]` is either `'('` , `')'`, or lowercase English letter.

SOLUTION (PYTHON 3)

Solution 1: Two Passes

How to think about it

We delete the fewest parentheses to make the string valid, n ≤ 10^5. A stack can mark illegal brackets; a counter also works: left to right we drop unmatched), then right to left we drop extra (. The first pass keeps every prefix from having more rights than lefts; the second pass is symmetric for lefts. Remaining brackets match; letters stay.

First, we scan from left to right and remove the extra right parentheses. Then, we scan from right to left and remove the extra left parentheses.

The time complexity is O(n), and the space complexity is O(n). Where n is the length of the string s.

Similar problems:

- 678. Valid Parenthesis String
- 2116. Check if a Parentheses String Can Be Valid

Python 3

```python
class Solution:
    def minRemoveToMakeValid(self, s: str) -> str:
        stk = []
        x = 0
        for c in s:
            if c == ')' and x == 0:
                continue
            if c == '(':
                x += 1
            elif c == ')':
                x -= 1
            stk.append(c)
        x = 0
        ans = []
        for c in stk[::-1]:
            if c == '(' and x == 0:
                continue
            if c == ')':
                x += 1
            elif c == '(':
                x -= 1
            ans.append(c)
        return ''.join(ans[::-1])
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Only count how many characters must be removed
# 2. Return the positions that are removed


# ============================================================
# FOLLOW-UP 1: Only count how many characters must be
# removed
# ============================================================

# ---- follow-up 1: function ----
# Answer: Unmatched ')' plus unmatched '(' left on the
# stack.
# Time: O(n).  Space: O(1).
def count_removals(s):
    opens = 0
    removals = 0
    for ch in s:
        if ch == "(":
            opens += 1
        elif ch == ")":
            if opens:
                opens -= 1
            else:
                removals += 1
    return removals + opens


# ============================================================
# FOLLOW-UP 2: Return the positions that are removed
# ============================================================

# ---- follow-up 2: function ----
# Answer: A stack of indices of '('; an unmatched ')' is
# removed at once, and the indices left on the stack at
# the end are removed too.
# Time: O(n log n).  Space: O(n).
def removed_positions(s):
    stack = []
```

```
    removed = []
    for i, ch in enumerate(s):
        if ch == "(":
            stack.append(i)
        elif ch == ")":
            if stack:
                stack.pop()
            else:
                removed.append(i)
    return sorted(removed + stack)
```

PART 2 · STACKS · PROBLEM 4 OF 5

2390. Removing Stars From a String

Medium · Stack, String, Simulation

leetcode.com/problems/removing-stars-from-a-string

★ **TOP 7 PROBLEM** in Stacks: master this one first

PROBLEM

You are given a string `s`, which contains stars `*`.

In one operation, you can:

- Choose a star in `s`.
- Remove the closest **non-star** character to its **left**, as well as remove the star itself.

Return *the string after **all** stars have been removed*.

Note:

- The input will be generated such that the operation is always possible.
- It can be shown that the resulting string will always be unique.

Example 1:

```
Input: s = "leet**cod*e"
Output: "lecoe"
Explanation: Performing the removals from left to right:
- The closest character to the 1st star is 't' in "leet**cod*e". s becomes "lee*cod*e".
- The closest character to the 2nd star is 'e' in "lee*cod*e". s becomes "lecod*e".
- The closest character to the 3rd star is 'd' in "lecod*e". s becomes "lecoe".
There are no more stars, so we return "lecoe".
```

Example 2:

```
Input: s = "erase*****"
Output: ""
Explanation: The entire string is removed, so we return an empty string.
```

Constraints:

- `1 <= s.length <= 10^5`
- `s` consists of lowercase English letters and stars `*`.
- The operation above can be performed on `s`.

SOLUTION (PYTHON 3)

Solution 1: Stack Simulation

> **How to think about it**
>
> A star deletes the closest letter to its left. n ≤ 10^5, so repeated scans would reshuffle the same characters. A stack keeps letters that still survive: push a letter, pop on a star. The join of the stack is the answer.

We can use a stack to simulate the operation process. Traverse the string s, and if the current character is not an asterisk, push it onto the stack; if the current character is an asterisk, pop the top element from the stack.

Finally, concatenate the elements in the stack into a string and return it.

The time complexity is O(n), where n is the length of the string s. Ignoring the space consumption of the answer string, the space complexity is O(1).

Python 3

```
class Solution:
    def removeStars(self, s: str) -> str:
        ans = []
        for c in s:
            if c == '*':
                ans.pop()
            else:
                ans.append(c)
        return ''.join(ans)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. A star removes the closest non-star letter to its
# RIGHT
# 2. Which original positions survive


# ============================================================
# FOLLOW-UP 1: A star removes the closest non-star letter
# to its RIGHT
# ============================================================

# ---- follow-up 1: function ----
# Answer: Mirror image: walk from the right with a stack.
# Time: O(n).  Space: O(n).
def remove_stars_to_the_right(s):
    stack = []
    for ch in reversed(s):
        if ch == "*":
            if stack:
                stack.pop()
        else:
            stack.append(ch)
    return "".join(reversed(stack))


# ============================================================
# FOLLOW-UP 2: Which original positions survive
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep indices on the stack instead of letters.
# Time: O(n).  Space: O(n).
def surviving_positions(s):
    stack = []
    for i, ch in enumerate(s):
        if ch == "*":
            if stack:
                stack.pop()
        else:
            stack.append(i)
    return stack
```

PART 2 · STACKS · PROBLEM 5 OF 5

32. Longest Valid Parentheses

Hard · Stack, String, Dynamic Programming, Parentheses

leetcode.com/problems/longest-valid-parentheses

★ **TOP 7 PROBLEM** in Stacks: master this one first

PROBLEM

Given a string containing just the characters `'('` and `')'`, return *the length of the longest valid (well-formed) parentheses substring*.

Example 1:

```
Input: s = "(()"
Output: 2
Explanation: The longest valid parentheses substring is "()".
```

Example 2:

```
Input: s = ")()())"
Output: 4
Explanation: The longest valid parentheses substring is "()()".
```

Example 3:

```
Input: s = ""
Output: 0
```

Constraints:

- `0 <= s.length <= 3 * 10^4`
- `s[i]` is `'('`, or `')'`.

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

The first idea is to test every substring with a stack. Correct, but $n \le 3 \times 10^4$ makes $O(n^2)$ or $O(n^3)$ too slow. The bottleneck is overlap: the longest valid run ending at a position can be built from shorter valid runs. A valid substring can only end with `)`. If it pairs with the previous character, the length is that of the prefix before this pair plus 2. If the previous character is also `)`, we skip the already-valid chunk ending there and check whether a `(` sits just before that chunk. So we let f[i] be the longest valid length ending at s[i-1], transfer on those two pairing cases, and take max f[i].

We define f[i] to be the length of the longest valid parentheses that ends with s[i-1], and the answer is max(f[i]).

When i < 2, the length of the string is less than 2, and there is no valid parentheses, so f[i] = 0.

When i ≥ 2, we consider the length of the longest valid parentheses that ends with s[i-1], that is, f[i]:

- If s[i-1] is a left parenthesis, then the length of the longest valid parentheses that ends with s[i-1] must be 0, so f[i] = 0.
- If s[i-1] is a right parenthesis, there are the following two cases:
  - If s[i-2] is a left parenthesis, then the length of the longest valid parentheses that ends with s[i-1] is f[i-2] + 2.
  - If s[i-2] is a right parenthesis, then the length of the longest valid parentheses that ends with s[i-1] is f[i-1] + 2, but we also need to consider whether s[i-f[i-1]-2] is a left parenthesis. If it is a left parenthesis, then the length of the longest valid parentheses that ends with s[i-1] is f[i-1] + 2 + f[i-f[i-1]-2].

Therefore, we can get the state transition equation:

cases f[i] = 0, & if s[i-1] = '(',\ f[i] = f[i-2] + 2, & if s[i-1] = ')' and s[i-2] = '(',\ f[i] = f[i-1] + 2 + f[i-f[i-1]-2], & if s[i-1] = ')' and s[i-2] = ')' and s[i-f[i-1]-2] = '(',\ cases

Finally, we only need to return max(f).

The time complexity is O(n), and the space complexity is O(n), where n is the length of the string.

Python 3

```
class Solution:
    def longestValidParentheses(self, s: str) -> int:
        n = len(s)
        f = [0] * (n + 1)
        for i, c in enumerate(s, 1):
            if c == ")":
                if i > 1 and s[i - 2] == "(":
                    f[i] = f[i - 2] + 2
                else:
                    j = i - f[i - 1] - 1
                    if j and s[j - 1] == "(":
                        f[i] = f[i - 1] + 2 + f[j - 1]
        return max(f)
```

Solution 2: Using Stack

How to think about it

Method 1 is already O(n), but the recurrence splits on whether the previous character is `(` and on pairing across an already-valid chunk, so the boundaries are easy to miss. What we still lack is not a better complexity, but a record that matches pairing more directly: indices of unmatched `(`, and where the last broken prefix starts. Seed the stack with -1 as a baseline, push on `(`, pop on `)`. An empty stack means the current `)` is a new baseline; otherwise the top of the stack updates the length. Time and space stay O(n); the matching process is just easier to see.

- Maintain a stack to store the indices of left parentheses. Initialize the bottom element of the stack with the value -1 to facilitate the calculation of the length of valid parentheses.
- Iterate through each element of the string:
  - If the character is a left parenthesis, push the index of the character onto the stack.
  - If the character is a right parenthesis, pop an element from the stack to represent that we have found a valid pair of parentheses.
  - If the stack is empty, it means we couldn't find a left parenthesis to match the right parenthesis. In this case, push the index of the character as a new starting point.
  - If the stack is not empty, calculate the length of the valid parentheses and update it.

Summary:

The key to this algorithm is to maintain a stack to store the indices of left parentheses and then update the length of the valid substring of parentheses by pushing and popping elements.

The time complexity is O(n), and the space complexity is O(n), where n is the length of the string.

Python 3

```python
class Solution:
    def longestValidParentheses(self, s: str) -> int:
        stack = [-1]
        ans = 0
        for i in range(len(s)):
            if s[i] == '(':
                stack.append(i)
            else:
                stack.pop()
                if not stack:
                    stack.append(i)
                else:
                    ans = max(ans, i - stack[-1])
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the longest valid substring
# 2. Count all valid parentheses substrings


# ============================================================
# FOLLOW-UP 1: Return the longest valid substring
# ============================================================

# ---- follow-up 1: function ----
# Answer: Stack of indices; the longest valid stretch ends
# at the current index and starts after the index below
# the top.
# Time: O(n).  Space: O(n).
def longest_valid_substring(s):
    stack = [-1]
    best_start, best_length = 0, 0
    for i, ch in enumerate(s):
        if ch == "(":
            stack.append(i)
        else:
            stack.pop()
            if not stack:
                stack.append(i)
            elif i - stack[-1] > best_length:
                best_length = i - stack[-1]
                best_start = stack[-1] + 1
    return s[best_start:best_start + best_length]


# ============================================================
# FOLLOW-UP 2: Count all valid parentheses substrings
# ============================================================

# ---- follow-up 2: function ----
# Answer: A stack of counts: each time a bracket pair
# closes, add the number of valid stretches that end here.
# Time: O(n).  Space: O(n).
def count_valid_substrings(s):
    stack = []
    ending_at = [0] * len(s)
    total = 0
    for i, ch in enumerate(s):
        if ch == "(":
            stack.append(i)
        elif stack:
            j = stack.pop()
            ending_at[i] = 1 + (ending_at[j - 1] if j > 0 else 0)
```

```
            total += ending_at[i]
    return total
```

PART 2 · TOPIC 13 OF 50

Monotonic Stack

6 problems · 1 Easy · 3 Medium · 2 Hard

THE IDEA

Keep the stack in increasing or decreasing order. When a new element breaks the order, the popped elements have just found their answer (next greater, next smaller, a span).

THE 6 PROBLEMS, EASY TO HARD | ★ = TOP 5: DO THESE FIRST

1	Final Prices With a Special Discount in a Shop	Easy	LC 1475	2235
2	★ **Shortest Unsorted Continuous Subarray**	Medium	LC 581	2237
3	★ **Max Chunks To Make Sorted**	Medium	LC 769	2240
4	★ **Buildings With an Ocean View**	Medium	LC 1762	2243
5	★ **Create Maximum Number**	Hard	LC 321	2245
6	★ **Number of Visible People in a Queue**	Hard	LC 1944	2248

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Next Greater Element I (Part 1, pattern 8, page 392)
Daily Temperatures (Part 1, pattern 8, page 407)
132 Pattern (Part 1, pattern 9, page 446)
Maximum Width Ramp (Part 1, pattern 9, page 457)
Sum of Subarray Ranges (Part 1, pattern 9, page 459)
Largest Rectangle in Histogram (Part 1, pattern 9, page 465)
Next Greater Element II (Part 1, pattern 9, page 433)
Online Stock Span (Part 1, pattern 9, page 443)
Remove K Digits (Part 1, pattern 9, page 436)
Sum of Subarray Minimums (Part 1, pattern 9, page 440)
Next Greater Node In Linked List (Part 1, pattern 9, page 452)

1475. Final Prices With a Special Discount in a Shop

Easy · Stack, Array, Monotonic Stack

leetcode.com/problems/final-prices-with-a-special-discount-in-a-shop

PROBLEM

You are given an integer array `prices` where `prices[i]` is the price of the i^{th} item in a shop.

There is a special discount for items in the shop. If you buy the i^{th} item, then you will receive a discount equivalent to `prices[j]` where `j` is the minimum index such that `j > i` and `prices[j] <= prices[i]`. Otherwise, you will not receive any discount at all.

Return an integer array `answer` where `answer[i]` is the final price you will pay for the i^{th} item of the shop, considering the special discount.

Example 1:

```
Input: prices = [8,4,6,2,3]
Output: [4,2,4,2,3]
Explanation:
For item 0 with price[0]=8 you will receive a discount equivalent to prices[1]=4, therefore, the final price you will pay is 8 - 4 = 4.
For item 1 with price[1]=4 you will receive a discount equivalent to prices[3]=2, therefore, the final price you will pay is 4 - 2 = 2.
For item 2 with price[2]=6 you will receive a discount equivalent to prices[3]=2, therefore, the final price you will pay is 6 - 2 = 4.
For items 3 and 4 you will not receive any discount at all.
```

Example 2:

```
Input: prices = [1,2,3,4,5]
Output: [1,2,3,4,5]
Explanation: In this case, for all items, you will not receive any discount at all.
```

Example 3:

```
Input: prices = [10,1,1,6]
Output: [9,0,1,6]
```

Constraints:

- `1 <= prices.length <= 500`
- `1 <= prices[i] <= 1000`

SOLUTION (PYTHON 3)

Solution 1: Monotonic Stack

> **How to think about it**
>
> The discount is the next price to the right that is at most the current one. n≤ 500 would allow a double loop; a monotonic stack finds the next smaller-or-equal in linear time. Scan right to left on an increasing stack and subtract in place.

The problem is essentially to find the first element on the right side that is smaller than each element. We can use a monotonic stack to solve this.

We traverse the array prices in reverse order, using the monotonic stack to find the nearest smaller element on the left side of the current element, and then calculate the discount.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array prices.

Python 3

```python
class Solution:
    def finalPrices(self, prices: List[int]) -> List[int]:
        stk = []
        for i in reversed(range(len(prices))):
            x = prices[i]
            while stk and x < stk[-1]:
                stk.pop()
            if stk:
                prices[i] -= stk[-1]
            stk.append(x)
        return prices
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Which item gave the discount to each item
# 2. The discount comes from the item to the LEFT instead


# ============================================================
# FOLLOW-UP 1: Which item gave the discount to each item
# ============================================================

# ---- follow-up 1: function ----
# Answer: The stack holds indices of items still waiting
# for a discount.
# Time: O(n).  Space: O(n).
def discount_givers(prices):
    giver = [-1] * len(prices)
    stack = []
    for i, p in enumerate(prices):
        while stack and prices[stack[-1]] >= p:
            giver[stack.pop()] = i
        stack.append(i)
    return giver


# ============================================================
# FOLLOW-UP 2: The discount comes from the item to the
# LEFT instead
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk left to right; the stack keeps earlier
# items that are not bigger.
# Time: O(n).  Space: O(n).
def final_prices_left_discount(prices):
    result = []
    stack = []
    for p in prices:
        while stack and stack[-1] > p:
            stack.pop()
        result.append(p - stack[-1] if stack else p)
        stack.append(p)
    return result
```

PART 2 · MONOTONIC STACK · PROBLEM 2 OF 6

581. Shortest Unsorted Continuous Subarray

Medium · Stack, Greedy, Array, Two Pointers, Sorting, Monotonic Stack

leetcode.com/problems/shortest-unsorted-continuous-subarray

★ **TOP 7 PROBLEM** in Monotonic Stack: master this one first

PROBLEM

Given an integer array `nums`, you need to find one **continuous subarray** such that if you only sort this subarray in non-decreasing order, then the whole array will be sorted in non-decreasing order.

Return *the shortest such subarray and output its length*.

Example 1:

```
Input: nums = [2,6,4,8,10,9,15]
Output: 5
Explanation: You need to sort [6, 4, 8, 10, 9] in ascending order to make the whole array sorted in ascending order.
```

Example 2:

```
Input: nums = [1,2,3,4]
Output: 0
```

Example 3:

```
Input: nums = [1]
Output: 0
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^5 <= nums[i] <= 10^5`

Follow up:

Can you solve it in

`O(n)`

time complexity?

SOLUTIONS (PYTHON 3)

Solution 1: Sorting

How to think about it

The shortest unsorted subarray is the one that, once sorted, makes the whole array sorted. Sort a copy and compare: the first and last mismatches bound that interval. $n \le 10^4$ allows a sort. If nothing mismatches, the array is already sorted and the length is 0.

We can first sort the array, and then compare the sorted array with the original array to find the leftmost and rightmost positions where they differ. The length between them is the length of the shortest unsorted continuous subarray.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the length of the array.

Python 3

```python
class Solution:
    def findUnsortedSubarray(self, nums: List[int]) -> int:
        arr = sorted(nums)
        l, r = 0, len(nums) - 1
        while l <= r and nums[l] == arr[l]:
            l += 1
        while l <= r and nums[r] == arr[r]:
            r -= 1
        return r - l + 1
```

Solution 2: Maintaining the Maximum Value on the Left and the Minimum Value on the Right

> **How to think about it**
>
> Sorting adds a log factor. A left-to-right scan: whenever a value is below the prefix maximum it belongs in the unsorted interval, so the right end grows. A right-to-left scan uses the suffix minimum for the left end. Both ends start at -1; if the right end never moves, return 0. Two linear passes, constant space.

We can traverse the array from left to right and maintain a maximum value mx. If the current value is less than mx, it means that the current value is not in the correct position, and we update the right boundary r to the current position. Similarly, we can traverse the array from right to left and maintain a minimum value mi. If the current value is greater than mi, it means that the current value is not in the correct position, and we update the left boundary l to the current position. At initialization, we set l and r to -1. If l and r are not updated, it means that the array is already sorted, and we return 0. Otherwise, we return r - l + 1.

The time complexity is O(n), and the space complexity is O(1). Here, n is the length of the array.

Python 3

```python
class Solution:
    def findUnsortedSubarray(self, nums: List[int]) -> int:
        mi, mx = inf, -inf
        l = r = -1
        n = len(nums)
        for i, x in enumerate(nums):
            if mx > x:
                r = i
            else:
                mx = x
            if mi < nums[n - i - 1]:
                l = n - i - 1
            else:
                mi = nums[n - i - 1]
        return 0 if r == -1 else r - l + 1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the start and end indices
# 2. The array must be sorted in DESCENDING order


# ============================================================
# FOLLOW-UP 1: Return the start and end indices
# ============================================================

# ---- follow-up 1: function ----
# Answer: Left boundary: the first place where a bigger
# number appears before a smaller one; right boundary
# mirrored.
# Time: O(n).  Space: O(1).
```

```
def unsorted_bounds(nums):
    n = len(nums)
    left, right = -1, -2
    biggest = nums[0]
    smallest = nums[-1]
    for i in range(1, n):
        biggest = max(biggest, nums[i])
        if nums[i] < biggest:
            right = i
    for i in range(n - 2, -1, -1):
        smallest = min(smallest, nums[i])
        if nums[i] > smallest:
            left = i
    if right == -2:
        return None
    return left, right


# ============================================================
# FOLLOW-UP 2: The array must be sorted in DESCENDING
# order
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same scan with the comparisons reversed.
# Time: O(n).  Space: O(1).
def unsorted_length_descending(nums):
    n = len(nums)
    left, right = -1, -2
    smallest = nums[0]
    biggest = nums[-1]
    for i in range(1, n):
        smallest = min(smallest, nums[i])
        if nums[i] > smallest:
            right = i
    for i in range(n - 2, -1, -1):
        biggest = max(biggest, nums[i])
        if nums[i] < biggest:
            left = i
    return right - left + 1 if right >= 0 else 0
```

PART 2 · MONOTONIC STACK · PROBLEM 3 OF 6

769. Max Chunks To Make Sorted

Medium · Stack, Greedy, Array, Sorting, Monotonic Stack

leetcode.com/problems/max-chunks-to-make-sorted

★ **TOP 7 PROBLEM** in Monotonic Stack: master this one first

PROBLEM

You are given an integer array `arr` of length `n` that represents a permutation of the integers in the range `[0, n - 1]`.

We split `arr` into some number of **chunks** (i.e., partitions), and individually sort each chunk. After concatenating them, the result should equal the sorted array.

Return *the largest number of chunks we can make to sort the array*.

Example 1:

```
Input: arr = [4,3,2,1,0]
Output: 1
Explanation:
Splitting into two or more chunks will not return the required result.
For example, splitting into [4, 3], [2, 1, 0] will result in [3, 4, 0, 1, 2], which isn't sorted.
```

Example 2:

```
Input: arr = [1,0,2,3,4]
Output: 4
Explanation:
We can split into two chunks, such as [1, 0], [2, 3, 4].
However, splitting into [1, 0], [2], [3], [4] is the highest number of chunks possible.
```

Constraints:

- `n == arr.length`
- `1 <= n <= 10`
- `0 <= arr[i] < n`
- All the elements of `arr` are **unique**.

SOLUTIONS (PYTHON 3)

Solution 1: Greedy + One Pass

How to think about it

The array is a permutation of 0..n-1. A cut after i is legal iff the prefix is exactly {0,…,i}. That is the prefix maximum equaling i. One pass counting those hits.

Since arr is a permutation of [0,..,n-1], if the maximum value mx among the numbers traversed so far is equal to the current index i, it means a split can be made, and the answer is incremented.

Time complexity is O(n), and space complexity is O(1). Where n is the length of the array arr.

Python 3

```python
class Solution:
    def maxChunksToSorted(self, arr: List[int]) -> int:
        mx = ans = 0
        for i, v in enumerate(arr):
            mx = max(mx, v)
            if i == mx:
                ans += 1
        return ans
```

Solution 2: Monotonic Stack

> **How to think about it**
>
> Solution 1 uses value=index, which fails with duplicates. The monotonic stack of chunk maxima is the same as problem 768. A smaller new value merges earlier chunks; the stack length is the answer.

The solution of method one has certain limitations. If there are duplicate elements in the array, the correct answer cannot be obtained.

According to the problem, we can find that from left to right, each chunk has a maximum value, and these maximum values are monotonically increasing. We can use a stack to store these maximum values of the chunks. The size of the final stack is the maximum number of chunks that can be sorted.

This solution can not only solve this problem but also solve the problem 768. Max Chunks To Make Sorted II. You can try it yourself.

Time complexity is O(n), and space complexity is O(n). Where n is the length of the array arr.

Python 3

```python
class Solution:
    def maxChunksToSorted(self, arr: List[int]) -> int:
        stk = []
        for v in arr:
            if not stk or v >= stk[-1]:
                stk.append(v)
            else:
                mx = stk.pop()
                while stk and stk[-1] > v:
                    stk.pop()
                stk.append(mx)
        return len(stk)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The array may contain duplicates (not a permutation
# of 0..n-1)
# 2. Return the chunks (as start and end indices)


# ============================================================
# FOLLOW-UP 1: The array may contain duplicates (not a
# permutation of 0..n-1)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Stack of chunk maximums: a new number smaller
# than the top merges chunks.
# Time: O(n).  Space: O(n).
def max_chunks_with_duplicates(arr):
    stack = []
    for x in arr:
        biggest = x
        while stack and stack[-1] > x:
            biggest = max(biggest, stack.pop())
        stack.append(biggest)
    return len(stack)


# ============================================================
# FOLLOW-UP 2: Return the chunks (as start and end
# indices)
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Chunks end where the sorted prefix equals the
# prefix as a multiset.
# Time: O(n log n).  Space: O(n).
def chunk_boundaries(arr):
    expected = sorted(arr)
    chunks = []
    start = 0
    count = {}
    differences = 0
    for i in range(len(arr)):
        for value, delta in ((arr[i], 1), (expected[i], -1)):
            before = count.get(value, 0)
            count[value] = before + delta
            if before == 0 and count[value] != 0:
                differences += 1
            elif before != 0 and count[value] == 0:
                differences -= 1
        if differences == 0:
            chunks.append((start, i))
            start = i + 1
    return chunks
```

1762. Buildings With an Ocean View

Medium · Stack, Array, Monotonic Stack

leetcode.com/problems/buildings-with-an-ocean-view

★ **TOP 7 PROBLEM** in Monotonic Stack: master this one first

PROBLEM

There are `n` buildings in a line. You are given an integer array `heights` of size `n` that represents the heights of the buildings in the line.

The ocean is to the right of the buildings. A building has an ocean view if the building can see the ocean without obstructions. Formally, a building has an ocean view if all the buildings to its right have a **smaller** height.

Return a list of indices **(0-indexed)** of buildings that have an ocean view, sorted in increasing order.

Example 1:

```
Input: heights = [4,2,3,1]
Output: [0,2,3]
Explanation: Building 1 (0-indexed) does not have an ocean view because building 2 is taller.
```

Example 2:

```
Input: heights = [4,3,2,1]
Output: [0,1,2,3]
Explanation: All the buildings have an ocean view.
```

Example 3:

```
Input: heights = [1,3,2,4]
Output: [3]
Explanation: Only building 3 has an ocean view.
```

Constraints:

- `1 <= heights.length <= 10^5`
- `1 <= heights[i] <= 10^9`

SOLUTION (PYTHON 3)

Solution 1: Reverse Traversal to Find the Maximum on the Right

How to think about it

A building sees the ocean iff nothing to its right is at least as tall. A right-to-left scan keeping the right-hand maximum decides this. If the height exceeds mx, record the index and update mx. Reverse the collected indices to left-to-right order.

We traverse the array height in reverse order for each element v, comparing v with the maximum element mx on the right. If mx < v, it means all elements to the right are smaller than the current element, so the current position can see the ocean and is added to the result array ans. Then we update mx to v.

After the traversal, return ans in reverse order.

The time complexity is O(n), where n is the length of the array. Ignoring the space consumption of the answer array, the space complexity is O(1).

Python 3

```python
class Solution:
    def findBuildings(self, heights: List[int]) -> List[int]:
        ans = []
        mx = 0
        for i in range(len(heights) - 1, -1, -1):
            if heights[i] > mx:
                ans.append(i)
                mx = heights[i]
        return ans[::-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The ocean is on the LEFT
# 2. Oceans on both sides: which buildings see at least
# one ocean


# ============================================================
# FOLLOW-UP 1: The ocean is on the LEFT
# ============================================================

# ---- follow-up 1: function ----
# Answer: Walk left to right keeping the tallest so far.
# Time: O(n).  Space: O(1) extra besides the answer.
def ocean_view_left(heights):
    result = []
    tallest = 0
    for i, h in enumerate(heights):
        if h > tallest:
            result.append(i)
            tallest = h
    return result


# ============================================================
# FOLLOW-UP 2: Oceans on both sides: which buildings see
# at least one ocean
# ============================================================

# ---- follow-up 2: function ----
# Answer: Union of the two results.
# Time: O(n log n).  Space: O(n).
def see_either_ocean(heights):
    seen = set()
    tallest = 0
    for i in range(len(heights)):
        if heights[i] > tallest:
            seen.add(i)
            tallest = heights[i]
    tallest = 0
    for i in range(len(heights) - 1, -1, -1):
        if heights[i] > tallest:
            seen.add(i)
            tallest = heights[i]
    return sorted(seen)
```

321. Create Maximum Number

Hard · Stack, Greedy, Array, Two Pointers, Monotonic Stack

leetcode.com/problems/create-maximum-number

★ **TOP 7 PROBLEM** in Monotonic Stack: master this one first

PROBLEM

You are given two integer arrays `nums1` and `nums2` of lengths `m` and `n` respectively. `nums1` and `nums2` represent the digits of two numbers. You are also given an integer `k`.

Create the maximum number of length `k <= m + n` from digits of the two numbers. The relative order of the digits from the same array must be preserved.

Return an array of the `k` digits representing the answer.

Example 1:

```
Input: nums1 = [3,4,6,5], nums2 = [9,1,2,5,8,3], k = 5
Output: [9,8,6,5,3]
```

Example 2:

```
Input: nums1 = [6,7], nums2 = [6,0,4], k = 5
Output: [6,7,6,0,4]
```

Example 3:

```
Input: nums1 = [3,9], nums2 = [8,9], k = 3
Output: [9,8,9]
```

Constraints:

- `m == nums1.length`
- `n == nums2.length`
- `1 <= m, n <= 500`
- `0 <= nums1[i], nums2[i] <= 9`
- `1 <= k <= m + n`
- `nums1` and `nums2` do not have leading zeros.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Pick a total of k digits from two arrays, preserving order in each. One monotone stack cannot split the quota across both. Enumerate taking x digits from nums1 (in the feasible range), extract the best subsequences of lengths x and k-x with monotone stacks, and merge by lexicographic compare. Keep the best merge. Equal prefixes must look ahead so the merge is not locally greedy.

Python 3

```python
class Solution:
    def maxNumber(self, nums1: List[int], nums2: List[int], k: int) -> List[int]:
        def f(nums: List[int], k: int) -> List[int]:
            n = len(nums)
            stk = [0] * k
            top = -1
            remain = n - k
            for x in nums:
                while top >= 0 and stk[top] < x and remain > 0:
                    top -= 1
                    remain -= 1
                if top + 1 < k:
                    top += 1
                    stk[top] = x
                else:
                    remain -= 1
            return stk

        def compare(nums1: List[int], nums2: List[int], i: int, j: int) -> bool:
            if i >= len(nums1):
                return False
            if j >= len(nums2):
                return True
            if nums1[i] > nums2[j]:
                return True
            if nums1[i] < nums2[j]:
                return False
            return compare(nums1, nums2, i + 1, j + 1)

        def merge(nums1: List[int], nums2: List[int]) -> List[int]:
            m, n = len(nums1), len(nums2)
            i = j = 0
            ans = [0] * (m + n)
            for k in range(m + n):
                if compare(nums1, nums2, i, j):
                    ans[k] = nums1[i]
                    i += 1
                else:
                    ans[k] = nums2[j]
                    j += 1
            return ans

        m, n = len(nums1), len(nums2)
        l, r = max(0, k - n), min(k, m)
        ans = [0] * k
        for x in range(l, r + 1):
            arr1 = f(nums1, x)
            arr2 = f(nums2, k - x)
            arr = merge(arr1, arr2)
            if ans < arr:
                ans = arr
        return ans
```

Complexity. Time: O(k * (m + n + k^2)). Space: O(k).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The smallest subsequence of length k (as digits)
# 2. The SMALLEST number of length k from two arrays


# ============================================================
# FOLLOW-UP 1: The smallest subsequence of length k (as
# digits)
# ============================================================
```

```
# ---- follow-up 1: function ----
# Answer: Greedy with an increasing stack, the mirror
# image of the maximum.
# Time: O(n).  Space: O(n).
def smallest_subsequence(nums, k):
    drop = len(nums) - k
    stack = []
    for x in nums:
        while drop and stack and stack[-1] > x:
            stack.pop()
            drop -= 1
        stack.append(x)
    return stack[:k]


# ============================================================
# FOLLOW-UP 2: The SMALLEST number of length k from two
# arrays
# ============================================================

# ---- follow-up 2: function ----
# Answer: Take the smallest subsequence of every length
# from each array and merge them into the smallest result.
# Time: O(k * (n + k^2)).  Space: O(k).
def min_number_from_two(nums1, nums2, k):
    def merge(a, b):
        a, b = list(a), list(b)
        result = []
        while a or b:
            if not b or (a and a + [float("inf")] < b + [float("inf")]):   # the end of a list counts as
the biggest value
                result.append(a.pop(0))
            else:
                result.append(b.pop(0))
        return result

    best = None
    for i in range(max(0, k - len(nums2)), min(k, len(nums1)) + 1):
        candidate = merge(smallest_subsequence(nums1, i), smallest_subsequence(nums2, k - i))
        if best is None or candidate < best:
            best = candidate
    return best
```

1944. Number of Visible People in a Queue

Hard · Stack, Array, Monotonic Stack

leetcode.com/problems/number-of-visible-people-in-a-queue

★ **TOP 7 PROBLEM** in Monotonic Stack: master this one first

PROBLEM

There are `n` people standing in a queue, and they numbered from `0` to `n - 1` in **left to right** order. You are given an array `heights` of **distinct** integers where `heights[i]` represents the height of the i^{th} person.

A person can **see** another person to their right in the queue if everybody in between is **shorter** than both of them. More formally, the i^{th} person can see the j^{th} person if `i < j` and `min(heights[i], heights[j]) > max(heights[i+1], heights[i+2], ..., heights[j-1])`.

Return *an array* `answer` *of length* `n` *where* `answer[i]` *is the* ***number of people*** *the* i^{th} *person can* ***see*** *to their right in the queue*.

Example 1:

```
Input: heights = [10,6,8,5,11,9]
Output: [3,1,2,1,1,0]
Explanation:
Person 0 can see person 1, 2, and 4.
Person 1 can see person 2.
Person 2 can see person 3 and 4.
Person 3 can see person 4.
Person 4 can see person 5.
Person 5 can see no one since nobody is to the right of them.
```

Example 2:

```
Input: heights = [5,1,2,3,10]
Output: [4,1,1,1,0]
```

Constraints:

- `n == heights.length`
- `1 <= n <= 10^5`
- `1 <= heights[i] <= 10^5`
- All the values of `heights` are **unique**.

SOLUTION (PYTHON 3)

Solution 1: Monotonic Stack

> **How to think about it**
>
> Person i sees j iff everyone in between is shorter. A rightward scan per i is $O(n^2)$. Visible heights to the right of i are strictly increasing. A stack increasing from top to bottom pops every shorter person (each counts) and, if someone remains, adds one more blocked by a taller person. Each height enters and leaves the stack once.

We observe that for the i-th person, the people he can see must be strictly increasing in height from left to right.

Therefore, we can traverse the array heights in reverse order, using a stack stk that is monotonically increasing from top to bottom to record the heights of the people we have traversed.

For the i-th person, if the stack is not empty and the top element of the stack is less than heights[i], we increment the count of people the i-th person can see, then pop the top element of the stack, until the stack is empty or the top element of the stack is greater than or equal to heights[i]. If the stack is not empty at this point, it means the top element of the stack is greater than or equal to heights[i], so we increment the count of people the i-th person can see by 1.

Next, we push heights[i] onto the stack and continue to the next person.

After traversing, we return the answer array ans.

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array heights.

Python 3

```python
class Solution:
    def canSeePersonsCount(self, heights: List[int]) -> List[int]:
        n = len(heights)
        ans = [0] * n
        stk = []
        for i in range(n - 1, -1, -1):
            while stk and stk[-1] < heights[i]:
                ans[i] += 1
                stk.pop()
            if stk:
                ans[i] += 1
            stk.append(heights[i])
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. How many people can each person see to their LEFT
# 2. How many pairs of people can see each other


# ============================================================
# FOLLOW-UP 1: How many people can each person see to
# their LEFT
# ============================================================

# ---- follow-up 1: function ----
# Answer: Mirror image: walk left to right with the same
# stack.
# Time: O(n).  Space: O(n).
def visible_to_the_left(heights):
    result = [0] * len(heights)
    stack = []
    for i, h in enumerate(heights):
        while stack and heights[stack[-1]] < h:
            stack.pop()
            result[i] += 1
        if stack:
```

```
            result[i] += 1
        stack.append(i)
    return result


# ========================================================
# FOLLOW-UP 2: How many pairs of people can see each other
# ========================================================

# ---- follow-up 2: function ----
# Answer: A pair is counted once, for the person on the
# left: the sum of the answers to the question itself.
# Time: O(n).  Space: O(n).
def visible_pairs(heights):
    result = [0] * len(heights)
    stack = []
    for i in range(len(heights) - 1, -1, -1):
        while stack and heights[stack[-1]] < heights[i]:
            stack.pop()
            result[i] += 1
        if stack:
            result[i] += 1
        stack.append(i)
    return sum(result)
```

PART 2 · TOPIC 14 OF 50

Queues

2 problems · 1 Easy · 1 Medium · 0 Hard

THE IDEA

A queue gives first-in-first-out order: simulate the line, drop old entries from the front, or count with a window of time.

THE 2 PROBLEMS, EASY TO HARD | ★ = TOP 1: DO THESE FIRST

1	Moving Average from Data Stream	Easy	LC 346	2252
2	★ **Number of People Aware of a Secret**	Medium	LC 2327	2255

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Moving Average from Data Stream — substitute: Number of Recent Calls (Part 1, pattern 10, page 484)

Reveal Cards In Increasing Order (Part 1, pattern 10, page 502)

Find the Winner of the Circular Game (Part 1, pattern 10, page 522)

Time Needed to Buy Tickets (Part 1, pattern 10, page 487)

First Unique Number (Part 1, pattern 10, page 511)

PART 2 · QUEUES · PROBLEM 1 OF 2

346. Moving Average from Data Stream

Easy · Design, Queue, Array, Data Stream

leetcode.com/problems/moving-average-from-data-stream

PROBLEM

Given a stream of integers and a window size, calculate the moving average of all integers in the sliding window.

Implement the `MovingAverage` class:

- `MovingAverage(int size)` Initializes the object with the size of the window `size`.
- `double next(int val)` Returns the moving average of the last `size` values of the stream.

Example 1:

```
Input
["MovingAverage", "next", "next", "next", "next"]
[[3], [1], [10], [3], [5]]
Output
[null, 1.0, 5.5, 4.66667, 6.0]

Explanation
MovingAverage movingAverage = new MovingAverage(3);
movingAverage.next(1); // return 1.0 = 1 / 1
movingAverage.next(10); // return 5.5 = (1 + 10) / 2
movingAverage.next(3); // return 4.66667 = (1 + 10 + 3) / 3
movingAverage.next(5); // return 6.0 = (10 + 3 + 5) / 3
```

Constraints:

- $1 <= size <= 1000$
- $-10^5 <= val <= 10^5$
- At most 10^4 calls will be made to `next`.

SOLUTIONS (PYTHON 3)

Solution 1: Circular Array

How to think about it

A moving average over a fixed window. Resumming the window is O(size). Keep the running sum for O(1) updates. A circular buffer overwrites index cnt mod size: subtract the old value, add the new one. The average is s/min(cnt,size).

We define a variable s to calculate the sum of the last size elements, and a variable cnt to record the total number of current elements. Additionally, we use an array data of length size to record the value of each element at each position.

When calling the next function, we first calculate the index i where val should be stored, then update the sum s, set the value at index i to val, and increment the element count by one. Finally, we return the value of s/(min(cnt, size)).

The time complexity is O(1), and the space complexity is O(n), where n is the integer size given in the problem.

Python 3

```
class MovingAverage:

    def __init__(self, size: int):
        self.s = 0
        self.data = [0] * size
        self.cnt = 0

    def next(self, val: int) -> float:
        i = self.cnt % len(self.data)
        self.s += val - self.data[i]
        self.data[i] = val
        self.cnt += 1
        return self.s / min(self.cnt, len(self.data))


# Your MovingAverage object will be instantiated and called as such:
# obj = MovingAverage(size)
# param_1 = obj.next(val)
```

Solution 2: Queue

> **How to think about it**
>
> Modular indexing is easy to get wrong. A queue stores the window: when full, pop the front and subtract, then append. Same meaning, clearer code.

We can use a queue q to store the last size elements, and a variable s to record the sum of these size elements.

When calling the next function, we first check if the length of the queue q is equal to size. If it is, we dequeue the front element of the queue q and update the value of s. Then we enqueue val and update the value of s. Finally, we return the value of s/len(q).

The time complexity is O(1), and the space complexity is O(n), where n is the integer size given in the problem.

Python 3

```
class MovingAverage:
    def __init__(self, size: int):
        self.n = size
        self.s = 0
        self.q = deque()

    def next(self, val: int) -> float:
        if len(self.q) == self.n:
            self.s -= self.q.popleft()
        self.q.append(val)
        self.s += val
        return self.s / len(self.q)


# Your MovingAverage object will be instantiated and called as such:
# obj = MovingAverage(size)
# param_1 = obj.next(val)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Moving median instead of the average
# 2. Average of the numbers of the last t seconds (each
# number has a time)


# ========================================================
# FOLLOW-UP 1: Moving median instead of the average
```

```
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the last `size` numbers in a sorted list
# and read its middle.
# Time: O(w) per call (insort and pop).  Space: O(w).
import bisect
from collections import deque


class MovingMedian:
    def __init__(self, size):
        self.size = size
        self.window = deque()
        self.sorted = []

    def next(self, val):
        self.window.append(val)
        bisect.insort(self.sorted, val)
        if len(self.window) > self.size:
            old = self.window.popleft()
            self.sorted.pop(bisect.bisect_left(self.sorted, old))
        m = len(self.sorted)
        if m % 2:
            return float(self.sorted[m // 2])
        return (self.sorted[m // 2 - 1] + self.sorted[m // 2]) / 2


# ============================================================
# FOLLOW-UP 2: Average of the numbers of the last t
# seconds (each number has a time)
# ============================================================

# ---- follow-up 2: function ----
# Answer: A queue of (time, value); drop the ones older
# than t.
# Time: O(1) amortised per call.  Space: O(w).
class TimeWindowAverage:
    def __init__(self, seconds):
        self.seconds = seconds
        self.queue = deque()
        self.total = 0

    def next(self, time, val):
        self.queue.append((time, val))
        self.total += val
        while self.queue and self.queue[0][0] <= time - self.seconds:
            self.total -= self.queue.popleft()[1]
        return self.total / len(self.queue)
```

2327. Number of People Aware of a Secret

Medium · Queue, Dynamic Programming, Simulation

leetcode.com/problems/number-of-people-aware-of-a-secret

★ **TOP 7 PROBLEM** in Queues: master this one first

PROBLEM

On day `1`, one person discovers a secret.

You are given an integer `delay`, which means that each person will **share** the secret with a new person **every day**, starting from `delay` days after discovering the secret. You are also given an integer `forget`, which means that each person will **forget** the secret `forget` days after discovering it. A person **cannot** share the secret on the same day they forgot it, or on any day afterwards.

Given an integer `n`, return *the number of people who know the secret at the end of day* `n`. Since the answer may be very large, return it **modulo** $10^9 + 7$.

Example 1:

```
Input: n = 6, delay = 2, forget = 4
Output: 5
Explanation:
Day 1: Suppose the first person is named A. (1 person)
Day 2: A is the only person who knows the secret. (1 person)
Day 3: A shares the secret with a new person, B. (2 people)
Day 4: A shares the secret with a new person, C. (3 people)
Day 5: A forgets the secret, and B shares the secret with a new person, D. (3 people)
Day 6: B shares the secret with E, and C shares the secret with F. (5 people)
```

Example 2:

```
Input: n = 4, delay = 1, forget = 3
Output: 6
Explanation:
Day 1: The first person is named A. (1 person)
Day 2: A shares the secret with B. (2 people)
Day 3: A and B share the secret with 2 new people, C and D. (4 people)
Day 4: A forgets the secret. B, C, and D share the secret with 3 new people. (6 people)
```

Constraints:

- `2 <= n <= 1000`
- `1 <= delay < forget <= n`

SOLUTION (PYTHON 3)

Solution 1: Difference Array

How to think about it

A person shares once per day on [delay, forget) after learning, then forgets. n ≤ 1000, so a day-by-day simulation is fine. Let cnt[i] be new learners on day i; they copy themselves on each later share day. A difference array d tracks who still remembers; the prefix through day n is the answer.

We use a difference array d[i] to record the change in the number of people who know the secret on day i, and an array cnt[i] to record the number of people who newly learn the secret on day i.

For the cnt[i] people who newly learn the secret on day i, they can share the secret with another cnt[i] people each day during the interval [i+delay, i+forget).

The answer is $\sum_{i=1}^{n} d[i]$.

The time complexity is $O(n^2)$, and the space complexity is $O(n)$, where n is the given integer.

Python 3

```
class Solution:
    def peopleAwareOfSecret(self, n: int, delay: int, forget: int) -> int:
        m = (n << 1) + 10
        d = [0] * m
        cnt = [0] * m
        cnt[1] = 1
        for i in range(1, n + 1):
            if cnt[i]:
                d[i] += cnt[i]
                d[i + forget] -= cnt[i]
                nxt = i + delay
                while nxt < i + forget:
                    cnt[nxt] += cnt[i]
                    nxt += 1
        mod = 10**9 + 7
        return sum(d[: n + 1]) % mod
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. How many people learn the secret on each day
# 2. How many people still remember the secret at the end
# of each day


# ========================================================
# FOLLOW-UP 1: How many people learn the secret on each
# day
# ========================================================

# ---- follow-up 1: function ----
# Answer: Keep the number of new people per day; day i
# shares to the people who learned between i - forget + 1
# and i - delay.
# Time: O(n * forget).  Space: O(n).
def new_people_per_day(n, delay, forget):
    new = [0] * (n + 1)
    new[1] = 1
    for day in range(2, n + 1):
        for earlier in range(max(1, day - forget + 1), day - delay + 1):
            new[day] += new[earlier]
    return new[1:]


# ========================================================
# FOLLOW-UP 2: How many people still remember the secret
# at the end of each day
# ========================================================

# ---- follow-up 2: function ----
# Answer: On day d, people who learned on days d - forget
# + 1 .. d still remember.
# Time: O(n * forget).  Space: O(n).
def remembering_each_day(n, delay, forget):
    new = new_people_per_day(n, delay, forget)
    result = []
    for day in range(1, n + 1):
        result.append(sum(new[max(0, day - forget):day]))
    return result
```

PART 2 · TOPIC 15 OF 50

Monotonic Queue

3 problems · 0 Easy · 2 Medium · 1 Hard

THE IDEA

A deque kept in order holds the best candidate for the current window at its front; drop worse candidates from the back and expired ones from the front.

THE 3 PROBLEMS, EASY TO HARD | ★ = TOP 3: DO THESE FIRST

1	★ **Continuous Subarrays**	Medium	LC 2762	2258
2	★ **Count Partitions With Max-Min Difference at Most K**	Medium	LC 3578	2261
3	★ **Max Value of Equation**	Hard	LC 1499	2264

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Jump Game VI (Part 1, pattern 10, page 519)

Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit (Part 1, pattern 35, page 1843)

Constrained Subsequence Sum (Part 1, pattern 9, page 474)

Find the Most Competitive Subsequence (Part 1, pattern 9, page 455)

Sliding Window Maximum (Part 1, pattern 2, page 124)

Shortest Subarray with Sum at Least K (Part 1, pattern 9, page 472)

PART 2 · MONOTONIC QUEUE · PROBLEM 1 OF 3

2762. Continuous Subarrays

Medium · Queue, Array, Ordered Set, Sliding Window, Monotonic Queue, Heap (Priority Queue)

leetcode.com/problems/continuous-subarrays

★ **TOP 7 PROBLEM** in Monotonic Queue: master this one first

PROBLEM

You are given a **0-indexed** integer array `nums`. A subarray of `nums` is called **continuous** if:

- Let `i`, `i + 1`, ..., `j` be the indices in the subarray. Then, for each pair of indices `i <= i1, i2 <= j`, `0 <= |nums[i1] - nums[i2]| <= 2`.

Return *the total number of **continuous** subarrays.*

A subarray is a contiguous **non-empty** sequence of elements within an array.

Example 1:

```
Input: nums = [5,4,2,4]
Output: 8
Explanation:
Continuous subarray of size 1: [5], [4], [2], [4].
Continuous subarray of size 2: [5,4], [4,2], [2,4].
Continuous subarray of size 3: [4,2,4].
There are no subarrys of size 4.
Total continuous subarrays = 4 + 3 + 1 = 8.
It can be shown that there are no more continuous subarrays.
```

Example 2:

```
Input: nums = [1,2,3]
Output: 6
Explanation:
Continuous subarray of size 1: [1], [2], [3].
Continuous subarray of size 2: [1,2], [2,3].
Continuous subarray of size 3: [1,2,3].
Total continuous subarrays = 3 + 2 + 1 = 6.
```

Constraints:

- `1 <= nums.length <= 10^5`
- `1 <= nums[i] <= 10^9`

SOLUTIONS (PYTHON 3)

Solution 1: Ordered List + Two Pointers

How to think about it

A subarray is continuous iff its maximum and minimum differ by at most 2. Checking every pair of ends is quadratic and too slow at length 10^5. After the right end inserts a value, drop left-end elements from a sorted list until the max–min gap is at most 2. The number of legal subarrays that end here equals the list size.

We can use two pointers, i and j, to maintain the left and right endpoints of the current subarray, and use an ordered list to maintain all elements in the current subarray.

Iterate through the array nums. For the current number nums[i] we're iterating over, we add it to the ordered list. If the difference between the maximum and minimum values in the ordered list is greater than 2, we then loop to move the pointer i to the right, continuously removing nums[i] from the ordered list, until the list is empty or the maximum difference between elements in the ordered list is not greater than 2. At this point, the number of uninterrupted subarrays is j - i + 1, which we

add to the answer.

After the iteration, return the answer.

The time complexity is O(n × log n) and the space complexity is O(n), where n is the length of the array nums.

Python 3

```python
class Solution:
    def continuousSubarrays(self, nums: List[int]) -> int:
        ans = i = 0
        sl = SortedList()
        for x in nums:
            sl.add(x)
            while sl[-1] - sl[0] > 2:
                sl.remove(nums[i])
                i += 1
            ans += len(sl)
        return ans
```

Solution 2: Monotonic queue + Two Pointers

> **How to think about it**
>
> A sorted list pays a logarithm per update. Only the extrema matter, so two monotonic deques of indices suffice. When the gap exceeds 2, pop the older extremum and advance the left end. The counting is unchanged, and each index enters and leaves a deque once.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```python
class Solution:
    def continuousSubarrays(self, nums: List[int]) -> int:
        min_q, max_q = deque(), deque()
        total = 0
        left = 0
        for right, x in enumerate(nums):
            while min_q and nums[min_q[-1]] > x:
                min_q.pop()
            while max_q and nums[max_q[-1]] < x:
                max_q.pop()
            min_q.append(right)
            max_q.append(right)
            while min_q and max_q and nums[max_q[0]] - nums[min_q[0]] > 2:
                if max_q[0] < min_q[0]:
                    left = max_q[0] + 1
                    max_q.popleft()
                else:
                    left = min_q[0] + 1
                    min_q.popleft()
            total += right - left + 1
        return total
```

Complexity. Time: O(n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The difference between maximum and minimum may be up
# to d
# 2. The longest such subarray


# ============================================================
# FOLLOW-UP 1: The difference between maximum and minimum
# may be up to d
# ============================================================
```

```python
# ---- follow-up 1: function ----
# Answer: The same two deques, with the limit d instead of
# 2.
# Time: O(n).  Space: O(n).
from collections import deque


def count_subarrays_within(nums, d):
    big, small = deque(), deque()
    left = 0
    total = 0
    for right in range(len(nums)):
        while big and nums[big[-1]] <= nums[right]:
            big.pop()
        big.append(right)
        while small and nums[small[-1]] >= nums[right]:
            small.pop()
        small.append(right)
        while nums[big[0]] - nums[small[0]] > d:
            left += 1
            if big[0] < left:
                big.popleft()
            if small[0] < left:
                small.popleft()
        total += right - left + 1
    return total


# ============================================================
# FOLLOW-UP 2: The longest such subarray
# ============================================================

# ---- follow-up 2: function ----
# Answer: Track the longest window instead of counting.
# Time: O(n).  Space: O(n).
def longest_subarray_within(nums, d):
    big, small = deque(), deque()
    left = 0
    best = 0
    for right in range(len(nums)):
        while big and nums[big[-1]] <= nums[right]:
            big.pop()
        big.append(right)
        while small and nums[small[-1]] >= nums[right]:
            small.pop()
        small.append(right)
        while nums[big[0]] - nums[small[0]] > d:
            left += 1
            if big[0] < left:
                big.popleft()
            if small[0] < left:
                small.popleft()
        best = max(best, right - left + 1)
    return best
```

PART 2 · MONOTONIC QUEUE · PROBLEM 2 OF 3

3578. Count Partitions With Max-Min Difference at Most K

Medium · Queue, Array, Dynamic Programming, Prefix Sum, Sliding Window, Monotonic Queue

leetcode.com/problems/count-partitions-with-max-min-difference-at-most-k

★ **TOP 7 PROBLEM** in Monotonic Queue: master this one first

PROBLEM

You are given an integer array `nums` and an integer `k`. Your task is to partition `nums` into one or more **non-empty** contiguous segments such that in each segment, the difference between its **maximum** and **minimum** elements is **at most** `k`.

Return the total number of ways to partition `nums` under this condition.

Since the answer may be too large, return it **modulo** $10^9 + 7$.

Example 1:

Input: nums = [9,4,1,3,7], k = 4

Output: 6

Explanation:

There are 6 valid partitions where the difference between the maximum and minimum elements in each segment is at most `k = 4`:

- `[[9], [4], [1], [3], [7]]`
- `[[9], [4], [1], [3, 7]]`
- `[[9], [4], [1, 3], [7]]`
- `[[9], [4, 1], [3], [7]]`
- `[[9], [4, 1], [3, 7]]`
- `[[9], [4, 1, 3], [7]]`

Example 2:

Input: nums = [3,3,4], k = 0

Output: 2

Explanation:

There are 2 valid partitions that satisfy the given conditions:

- `[[3], [3], [4]]`
- `[[3, 3], [4]]`

Constraints:

- `2 <= nums.length <=` $5 * 10^4$
- `1 <= nums[i] <=` 10^9
- `0 <= k <=` 10^9

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming + Two Pointers + Ordered Set

How to think about it

Every part of a partition must have max-min ≤ k. f[r] is the number of ways to partition the prefix of length r; the last part is some [l,r], so f[r] sums f[j] over legal last parts. The legal left end only moves right: an ordered set stores the window extrema and advances l when the gap exceeds k. A prefix sum g evaluates the range sum in O(1).

We define f[i] as the number of ways to partition the first i elements. If an array satisfies that the difference between its maximum and minimum values does not exceed k, then any of its subarrays also satisfies this condition. Therefore, we can use two pointers to maintain a sliding window representing the current subarray.

When we reach the r-th element, we need to find the left pointer l such that the subarray from l to r satisfies that the difference between the maximum and minimum values does not exceed k. We can use an ordered set to maintain the elements in the current window, so that we can quickly get the maximum and minimum values.

Each time we add a new element, we insert it into the ordered set and check the difference between the maximum and minimum values in the current window. If it exceeds k, we move the left pointer l until the condition is satisfied. The number of partition ways ending at the r-th element is f[l - 1] + f[l] + … + f[r - 1]. We can use a prefix sum array to quickly calculate this value.

The answer is f[n], where n is the length of the array.

The time complexity is O(n × log n) and the space complexity is O(n), where n is the length of the array nums.

Python 3

```python
class Solution:
    def countPartitions(self, nums: List[int], k: int) -> int:
        mod = 10**9 + 7
        sl = SortedList()
        n = len(nums)
        f = [1] + [0] * n
        g = [1] + [0] * n
        l = 1
        for r, x in enumerate(nums, 1):
            sl.add(x)
            while sl[-1] - sl[0] > k:
                sl.remove(nums[l - 1])
                l += 1
            f[r] = (g[r - 1] - (g[l - 2] if l >= 2 else 0) + mod) % mod
            g[r] = (g[r - 1] + f[r]) % mod
        return f[n]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The minimum number of parts (every part has max - min
# <= k)
# 2. Return that smallest partition as lists


# ============================================================
# FOLLOW-UP 1: The minimum number of parts (every part has
# max - min <= k)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Greedy: extend the current part as far as
# possible.
# Time: O(n).  Space: O(1).
def min_parts(nums, k):
    parts = 1
    low = high = nums[0]
    for x in nums[1:]:
        low2, high2 = min(low, x), max(high, x)
        if high2 - low2 > k:
            parts += 1
            low = high = x
        else:
            low, high = low2, high2
    return parts
```

```python
# ============================================================
# FOLLOW-UP 2: Return that smallest partition as lists
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same greedy, cutting the list.
# Time: O(n).  Space: O(n).
def smallest_partition(nums, k):
    parts = [[nums[0]]]
    low = high = nums[0]
    for x in nums[1:]:
        low2, high2 = min(low, x), max(high, x)
        if high2 - low2 > k:
            parts.append([x])
            low = high = x
        else:
            parts[-1].append(x)
            low, high = low2, high2
    return parts
```

PART 2 · MONOTONIC QUEUE · PROBLEM 3 OF 3

1499. Max Value of Equation

Hard · Queue, Array, Sliding Window, Monotonic Queue, Heap (Priority Queue)

leetcode.com/problems/max-value-of-equation

★ **TOP 7 PROBLEM** in Monotonic Queue: master this one first

PROBLEM

You are given an array `points` containing the coordinates of points on a 2D plane, sorted by the x-values, where `points[i] = [x_i, y_i]` such that `x_i < x_j` for all `1 <= i < j <= points.length`. You are also given an integer `k`.

Return *the maximum value of the equation* `y_i + y_j + |x_i - x_j|` where `|x_i - x_j| <= k` and `1 <= i < j <= points.length`.

It is guaranteed that there exists at least one pair of points that satisfy the constraint `|x_i - x_j| <= k`.

Example 1:

```
Input: points = [[1,3],[2,0],[5,10],[6,-10]], k = 1
Output: 4
Explanation: The first two points satisfy the condition |xi - xj| <= 1 and if we calculate the equation we get 3 + 0 + |1 - 2| = 4. Third and fourth points also satisfy the condition and give a value of 10 + -10 + |5 - 6| = 1.
No other pairs satisfy the condition, so we return the max of 4 and 1.
```

Example 2:

```
Input: points = [[0,0],[3,0],[9,2]], k = 3
Output: 3
Explanation: Only the first two points have an absolute difference of 3 or less in the x-values, and give the value of 0 + 0 + |0 - 3| = 3.
```

Constraints:

- `2 <= points.length <= 10^5`
- `points[i].length == 2`
- `-10^8 <= x_i, y_i <= 10^8`
- `0 <= k <= 2 * 10^8`
- `x_i < x_j` for all `1 <= i < j <= points.length`
- `x_i` form a strictly increasing sequence.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

x is strictly increasing and n≤ 10^5. Rewrite $y_i+y_j+|x_i-x_j|=(y_i-x_i)+(x_j+y_j)$. For each j we need the max y_i-x_i among $x_j-x_i\le k$. A heap of (x_i-y_i,x_i) pops expired points (min-heap on x-y is max y-x) and adds the current x+y.

Python 3

```python
class Solution:
    def findMaxValueOfEquation(self, points: List[List[int]], k: int) -> int:
        ans = -inf
        pq = []
        for x, y in points:
            while pq and x - pq[0][1] > k:
                heappop(pq)
            if pq:
                ans = max(ans, x + y - pq[0][0])
            heappush(pq, (x - y, x))
        return ans
```

Complexity. Time: O(n log n). Space: O(n).

Solution 2

> **How to think about it**
>
> Method 1 is O(log n) per point. A decreasing deque maintains the window max of y-x in amortized O(1): drop expired heads and dominated tails before pushing.

Python 3

```python
class Solution:
    def findMaxValueOfEquation(self, points: List[List[int]], k: int) -> int:
        ans = -inf
        q = deque()
        for x, y in points:
            while q and x - q[0][0] > k:
                q.popleft()
            if q:
                ans = max(ans, x + y + q[0][1] - q[0][0])
            while q and y - x >= q[-1][1] - q[-1][0]:
                q.pop()
            q.append((x, y))
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the pair of points that gives the maximum
# 2. At most m points apart (by position in the list) as
# well


# ============================================================
# FOLLOW-UP 1: Return the pair of points that gives the
# maximum
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same deque of the best yi - xi, remembering the
# index.
# Time: O(n).  Space: O(n).
from collections import deque


def best_pair_of_points(points, k):
    queue = deque()
    best = None
    pair = None
    for j in range(len(points)):
        x, y = points[j]
        while queue and x - points[queue[0]][0] > k:
```

```
            queue.popleft()
        if queue:
            i = queue[0]
            value = points[i][1] - points[i][0] + y + x
            if best is None or value > best:
                best = value
                pair = (i, j)
        while queue and points[queue[-1]][1] - points[queue[-1]][0] <= y - x:
            queue.pop()
        queue.append(j)
    return pair


# ============================================================
# FOLLOW-UP 2: At most m points apart (by position in the
# list) as well
# ============================================================

# ---- follow-up 2: function ----
# Answer: Drop from the front of the deque the points that
# are more than m places back.
# Time: O(n).  Space: O(n).
def max_value_close_points(points, k, m):
    queue = deque()
    best = None
    for j in range(len(points)):
        x, y = points[j]
        while queue and (x - points[queue[0]][0] > k or j - queue[0] > m):
            queue.popleft()
        if queue:
            i = queue[0]
            value = points[i][1] - points[i][0] + y + x
            if best is None or value > best:
                best = value
        while queue and points[queue[-1]][1] - points[queue[-1]][0] <= y - x:
```

```
            queue.pop()
        queue.append(j)
    return best
```

PART 2 · TOPIC 16 OF 50

Bucket Sort

1 problem · 0 Easy · 0 Medium · 1 Hard

THE IDEA

When values or counts are limited, put items into buckets by key and read the buckets in order: O(n) instead of O(n log n).

THE 1 PROBLEMS, EASY TO HARD | ★ = TOP 1: DO THESE FIRST

1	★ **Contains Duplicate III**	Hard	LC 220	2268

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Sort Characters By Frequency (Part 1, pattern 19, page 1055)

Top K Frequent Words (Part 1, pattern 19, page 1071)

Maximum Gap (Part 1, pattern 23, page 1249)

PART 2 · BUCKET SORT · PROBLEM 1 OF 1

220. Contains Duplicate III

Hard · Array, Bucket Sort, Ordered Set, Sorting, Sliding Window

leetcode.com/problems/contains-duplicate-iii

★ **TOP 7 PROBLEM** in Bucket Sort: master this one first

PROBLEM

You are given an integer array `nums` and two integers `indexDiff` and `valueDiff`.

Find a pair of indices `(i, j)` such that:

- `i != j`,
- `abs(i - j) <= indexDiff`.
- `abs(nums[i] - nums[j]) <= valueDiff`, and

Return `true` *if such pair exists or* `false` *otherwise*.

Example 1:

```
Input: nums = [1,2,3,1], indexDiff = 3, valueDiff = 0
Output: true
Explanation: We can choose (i, j) = (0, 3).
We satisfy the three conditions:
i != j --> 0 != 3
abs(i - j) <= indexDiff --> abs(0 - 3) <= 3
abs(nums[i] - nums[j]) <= valueDiff --> abs(1 - 1) <= 0
```

Example 2:

```
Input: nums = [1,5,9,1,5,9], indexDiff = 2, valueDiff = 3
Output: false
Explanation: After trying all the possible pairs (i, j), we cannot satisfy the three conditions, so we
return false.
```

Constraints:

- $2 \le nums.length \le 10^5$
- $-10^9 \le nums[i] \le 10^9$
- $1 \le indexDiff \le nums.length$
- $0 \le valueDiff \le 10^9$

SOLUTION (PYTHON 3)

Solution 1: Sliding Window + Ordered Set

How to think about it

We need a pair whose indices differ by at most indexDiff and values by at most valueDiff. A hash set only tests equality, not nearest neighbors. Keep an ordered set of the last indexDiff values. For v, look up the least element ≥ v-valueDiff; if it is also ≤ v+valueDiff, we are done. Then insert v and drop the value that leaves the window.

We maintain a sliding window of size k, and the elements in the window are kept in order.

We traverse the array `nums`. For each element nums[i], we look for the first element in the ordered set that is greater than or equal to nums[i] - t. If the element exists, and this element is less than or equal to nums[i] + t, it means we have found a pair of elements that meet the conditions, and we return `true`. Otherwise, we insert nums[i] into the ordered set, and if the size of the ordered set exceeds k, we need to remove the earliest added element from the ordered set.

The time complexity is O(n × log k), where n is the length of the array `nums`. For each element, we need O(log k) time to find the element in the ordered set, and there are n elements in total, so the total time complexity is O(n × log k).

Python 3

```
class Solution:
    def containsNearbyAlmostDuplicate(
        self, nums: List[int], indexDiff: int, valueDiff: int
    ) -> bool:
        s = SortedSet()
        for i, v in enumerate(nums):
            j = s.bisect_left(v - valueDiff)
            if j < len(s) and s[j] <= v + valueDiff:
                return True
            s.add(v)
            if i >= indexDiff:
                s.remove(nums[i - indexDiff])
        return False
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the pair of indices
# 2. Count all such pairs


# ============================================================
# FOLLOW-UP 1: Return the pair of indices
# ============================================================

# ---- follow-up 1: function ----
# Answer: Buckets of width t + 1 keyed by value; remember
# the index in the bucket.
# Time: O(n).  Space: O(min(n, w)).
def near_duplicate_pair(nums, index_diff, value_diff):
    width = value_diff + 1
    buckets = {}
    for i, x in enumerate(nums):
        b = x // width
        for key in (b - 1, b, b + 1):
            if key in buckets and abs(buckets[key][1] - x) <= value_diff:
                return (buckets[key][0], i)
        buckets[b] = (i, x)
        if i >= index_diff:
            old = nums[i - index_diff]
            if buckets.get(old // width, (None,))[0] == i - index_diff:
                del buckets[old // width]
    return None


# ============================================================
# FOLLOW-UP 2: Count all such pairs
# ============================================================

# ---- follow-up 2: function ----
# Answer: Check every pair inside the index window (simple
# version), or use a sorted list of the window.
# Time: O(n * w), w = index difference.  Space: O(w).
import bisect


def count_near_duplicates(nums, index_diff, value_diff):
    window = []
    total = 0
    for i, x in enumerate(nums):
        low = bisect.bisect_left(window, x - value_diff)
        high = bisect.bisect_right(window, x + value_diff)
```

```
        total += high - low
        bisect.insort(window, x)
        if i >= index_diff:
            window.pop(bisect.bisect_left(window, nums[i - index_diff]))
    return total
```

PART 2 · TOPIC 17 OF 50

Recursion

2 problems · 0 Easy · 0 Medium · 2 Hard

THE IDEA

Define the answer for the whole in terms of smaller parts, and state the base case first.

THE 2 PROBLEMS, EASY TO HARD | ★ = TOP 2: DO THESE FIRST

1	★ **Integer to English Words**	Hard	LC 273	2272
2	★ **Special Binary String**	Hard	LC 761	2276

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Merge Two Sorted Lists (Part 1, pattern 25, page 1328)
Decode String (Part 1, pattern 6, page 338)
Pow(x, n) (Part 1, pattern 14, page 732)

PART 2 · RECURSION · PROBLEM 1 OF 2

273. Integer to English Words

Hard · Recursion, Math, String

leetcode.com/problems/integer-to-english-words

★ **TOP 7 PROBLEM** in Recursion: master this one first

PROBLEM

Convert a non-negative integer `num` to its English words representation.

Example 1:

```
Input: num = 123
Output: "One Hundred Twenty Three"
```

Example 2:

```
Input: num = 12345
Output: "Twelve Thousand Three Hundred Forty Five"
```

Example 3:

```
Input: num = 1234567
Output: "One Million Two Hundred Thirty Four Thousand Five Hundred Sixty Seven"
```

Constraints:

- $0 <= num <= 2^{31} - 1$

SOLUTION (PYTHON 3)

Solution 1

How to think about it

English names group by thousands: Billion, Million, Thousand. Each block has at most three digits and is built from the teens table, the tens table, and Hundred. 0 is Zero. From high to low, convert each nonempty block with transfer and append the scale word.

Python 3

```
class Solution:
    def numberToWords(self, num: int) -> str:
        if num == 0:
            return 'Zero'

        lt20 = [
            '',
            'One',
            'Two',
            'Three',
            'Four',
            'Five',
            'Six',
            'Seven',
            'Eight',
            'Nine',
            'Ten',
            'Eleven',
            'Twelve',
            'Thirteen',
            'Fourteen',
            'Fifteen',
            'Sixteen',
            'Seventeen',
            'Eighteen',
            'Nineteen',
        ]
        tens = [
            '',
            'Ten',
            'Twenty',
            'Thirty',
            'Forty',
            'Fifty',
            'Sixty',
            'Seventy',
            'Eighty',
            'Ninety',
        ]
        thousands = ['Billion', 'Million', 'Thousand', '']

        def transfer(num):
            if num == 0:
                return ''
            if num < 20:
                return lt20[num] + ' '
            if num < 100:
                return tens[num // 10] + ' ' + transfer(num % 10)
            return lt20[num // 100] + ' Hundred ' + transfer(num % 100)

        res = []
        i, j = 1000000000, 0
        while i > 0:
            if num // i != 0:
                res.append(transfer(num // i))
                res.append(thousands[j])
                res.append(' ')
                num %= i
            j += 1
            i //= 1000

        return ''.join(res).strip()
```

Complexity. Time: O(1) (at most 10 digits). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. British style with 'and' (One Hundred and Twenty
# Three)
# 2. The reverse: English words to the number


# ============================================================
# FOLLOW-UP 1: British style with 'and' (One Hundred and
# Twenty Three)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Put 'and' after hundreds when something follows,
# and before the last part of bigger numbers.
# Time: O(1) (the number has at most 10 digits).  Space:
# O(1).
def to_words_british(num):
    if num == 0:
        return "Zero"
    ones = ["", "One", "Two", "Three", "Four", "Five", "Six", "Seven", "Eight", "Nine", "Ten", "Eleven", "
Twelve", "Thirteen",
            "Fourteen", "Fifteen", "Sixteen", "Seventeen", "Eighteen", "Nineteen"]
    tens = ["", "", "Twenty", "Thirty", "Forty", "Fifty", "Sixty", "Seventy", "Eighty", "Ninety"]

    def below_thousand(n):
        parts = []
        if n >= 100:
            parts.append(ones[n // 100] + " Hundred")
            n %= 100
            if n:
                parts.append("and")
        if n >= 20:
            parts.append(tens[n // 10] + (" " + ones[n % 10] if n % 10 else ""))
        elif n:
            parts.append(ones[n])
        return " ".join(parts)

    pieces = []
    for value, name in ((10 ** 9, "Billion"), (10 ** 6, "Million"), (1000, "Thousand")):
        if num >= value:
            pieces.append(below_thousand(num // value) + " " + name)
            num %= value
    if num:
        if pieces and num < 100:
            pieces.append("and")
        pieces.append(below_thousand(num))
    return " ".join(pieces)


# ============================================================
# FOLLOW-UP 2: The reverse: English words to the number
# ============================================================

# ---- follow-up 2: function ----
# Answer: Read the words, collecting a small value and
# multiplying at Hundred, Thousand, Million, Billion.
# Time: O(length of the text).  Space: O(1).
def words_to_number(text):
    small = {"Zero": 0, "One": 1, "Two": 2, "Three": 3, "Four": 4, "Five": 5, "Six": 6, "Seven": 7, "
Eight": 8, "Nine": 9, "Ten": 10,
             "Eleven": 11, "Twelve": 12, "Thirteen": 13, "Fourteen": 14, "Fifteen": 15, "Sixteen": 16, "
Seventeen": 17, "Eighteen": 18,
             "Nineteen": 19, "Twenty": 20, "Thirty": 30, "Forty": 40, "Fifty": 50, "Sixty": 60, "Seventy":
70, "Eighty": 80, "Ninety": 90}
```

```
    scale = {"Thousand": 1000, "Million": 10 ** 6, "Billion": 10 ** 9}
    total = 0
    current = 0
    for word in text.split():
        if word in small:
            current += small[word]
        elif word == "Hundred":
            current *= 100
        elif word in scale:
            total += current * scale[word]
            current = 0
    return total + current
```

761. Special Binary String

Hard · String, Divide and Conquer, Sorting

leetcode.com/problems/special-binary-string

★ **TOP 7 PROBLEM** in Recursion: master this one first

PROBLEM

Special binary strings are binary strings with the following two properties:

- The number of `0`'s is equal to the number of `1`'s.
- Every prefix of the binary string has at least as many `1`'s as `0`'s.

You are given a **special binary** string `s`.

A move consists of choosing two consecutive, non-empty, special substrings of `s`, and swapping them. Two strings are consecutive if the last character of the first string is exactly one index before the first character of the second string.

Return *the lexicographically largest resulting string possible after applying the mentioned operations on the string*.

Example 1:

```
Input: s = "11011000"
Output: "11100100"
Explanation: The strings "10" [occuring at s[1]] and "1100" [at s[3]] are swapped.
This is the lexicographically largest string possible after some number of swaps.
```

Example 2:

```
Input: s = "10"
Output: "10"
```

Constraints:

- `1 <= s.length <= 50`
- `s[i]` is either `'0'` or `'1'`.
- `s` is a special binary string.

SOLUTION (PYTHON 3)

Solution 1: Recursion + Sorting

How to think about it

A special binary string is a valid parentheses string with 1/0. Swapping adjacent special pieces increases the lexicographic order. n≤ 50, but the recursive shape is the real lever. The string is a concatenation of 1+special+0 blocks. Maximize each interior, then sort those blocks descending. A balance counter isolates blocks; recurse on the inside, sort, and join.

We can treat the special binary sequence as "valid parentheses", where 1 represents an opening parenthesis and 0 represents a closing parenthesis. For example, "11011000" can be viewed as "((()(()))".

Swapping two consecutive non-empty special substrings is equivalent to swapping two adjacent valid parentheses. We can use recursion to solve this problem.

We treat each "valid parenthesis" in string s as a part, process it recursively, and finally sort them to get the final answer.

The time complexity is $O(n^2)$, and the space complexity is $O(n)$, where n is the length of string s.

Python 3

```
class Solution:
    def makeLargestSpecial(self, s: str) -> str:
        if s == '':
            return ''
        ans = []
        cnt = 0
        i = j = 0
        while i < len(s):
            cnt += 1 if s[i] == '1' else -1
            if cnt == 0:
                ans.append('1' + self.makeLargestSpecial(s[j + 1 : i]) + '0')
                j = i + 1
            i += 1
        ans.sort(reverse=True)
        return ''.join(ans)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Check that a string is special
# 2. The smallest string (instead of the largest) you can
# reach by swapping neighbouring special parts


# ============================================================
# FOLLOW-UP 1: Check that a string is special
# ============================================================

# ---- follow-up 1: function ----
# Answer: Equal numbers of 0 and 1, and no prefix has more
# 0s than 1s.
# Time: O(n).  Space: O(1).
def is_special(s):
    balance = 0
    for ch in s:
        balance += 1 if ch == "1" else -1
        if balance < 0:
            return False
    return balance == 0


# ============================================================
# FOLLOW-UP 2: The smallest string (instead of the
# largest) you can reach by swapping neighbouring special
# parts
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same recursion, but sort the parts in increasing
# order.
# Time: O(n^2).  Space: O(n).
def smallest_special(s):
    parts = []
    balance = 0
    start = 0
    for i, ch in enumerate(s):
        balance += 1 if ch == "1" else -1
        if balance == 0:
            parts.append("1" + smallest_special(s[start + 1:i]) + "0")
            start = i + 1
    return "".join(sorted(parts))
```

PART 2 · TOPIC 18 OF 50

Divide and Conquer

3 problems · 1 Easy · 2 Medium · 0 Hard

THE IDEA

Split the input, solve the halves, and combine. Pick the split point that makes the combine step easy.

THE 3 PROBLEMS, EASY TO HARD | ★ = TOP 2: DO THESE FIRST

1	Longest Nice Substring	Easy	LC 1763	2279
2	★ **Construct Quad Tree**	Medium	LC 427	2282
3	★ **Maximum Binary Tree**	Medium	LC 654	2286

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Convert Sorted List to Binary Search Tree (Part 1, pattern 24, page 1309)

PART 2 · DIVIDE AND CONQUER · PROBLEM 1 OF 3

1763. Longest Nice Substring

Easy · Bit Manipulation, Hash Table, String, Divide and Conquer, Sliding Window

leetcode.com/problems/longest-nice-substring

PROBLEM

A string `s` is **nice** if, for every letter of the alphabet that `s` contains, it appears **both** in uppercase and lowercase. For example, `"abABB"` is nice because `'A'` and `'a'` appear, and `'B'` and `'b'` appear. However, `"abA"` is not because `'b'` appears, but `'B'` does not.

Given a string `s`, return *the longest **substring** of `s` that is **nice**. If there are multiple, return the substring of the **earliest** occurrence. If there are none, return an empty string*.

Example 1:

```
Input: s = "YazaAay"
Output: "aAa"
Explanation: "aAa" is a nice string because 'A/a' is the only letter of the alphabet in s, and both 'A' and 'a' appear.
"aAa" is the longest nice substring.
```

Example 2:

```
Input: s = "Bb"
Output: "Bb"
Explanation: "Bb" is a nice string because both 'B' and 'b' appear. The whole string is a substring.
```

Example 3:

```
Input: s = "c"
Output: ""
Explanation: There are no nice substrings.
```

Constraints:

- `1 <= s.length <= 100`
- `s` consists of uppercase and lowercase English letters.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

A nice substring contains both cases of every letter that appears. n is small enough to try all substrings. Fix the left end i, extend rightward, and store characters in a set. When every letter has both cases and the window is longer, record it. Ties keep the earlier substring.

Python 3

```python
class Solution:
    def longestNiceSubstring(self, s: str) -> str:
        n = len(s)
        ans = ''
        for i in range(n):
            ss = set()
            for j in range(i, n):
                ss.add(s[j])
                if (
                    all(c.lower() in ss and c.upper() in ss for c in ss)
                    and len(ans) < j - i + 1
                ):
                    ans = s[i : j + 1]
        return ans
```

Complexity. Time: O(n^2 * C), C = 52 letters. Space: O(C).

Solution 2

> **How to think about it**
>
> Solution 1 scans the set for each window, paying an extra C factor. With 26 letters, two bitmasks for lower and upper case suffice; the window is nice iff the masks are equal.

Python 3

```python
class Solution:
    def longestNiceSubstring(self, s: str) -> str:
        n = len(s)
        ans = ''
        for i in range(n):
            lower = upper = 0
            for j in range(i, n):
                if s[j].islower():
                    lower |= 1 << (ord(s[j]) - ord('a'))
                else:
                    upper |= 1 << (ord(s[j]) - ord('A'))
                if lower == upper and len(ans) < j - i + 1:
                    ans = s[i : j + 1]
        return ans
```

Complexity. Time: O(n^2). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count the nice substrings
# 2. The longest nice substring, found by scanning with
# masks (no recursion)


# ============================================================
# FOLLOW-UP 1: Count the nice substrings
# ============================================================

# ---- follow-up 1: function ----
# Answer: Check every substring with two letter masks
# (lower and upper case).
# Time: O(n^2).  Space: O(1).
def count_nice_substrings(s):
    count = 0
    for i in range(len(s)):
        lower = upper = 0
        for j in range(i, len(s)):
```

```
            ch = s[j]
            if ch.islower():
                lower |= 1 << (ord(ch) - 97)
            else:
                upper |= 1 << (ord(ch) - 65)
            if lower == upper:
                count += 1
    return count


# ============================================================
# FOLLOW-UP 2: The longest nice substring, found by
# scanning with masks (no recursion)
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same masks, remembering the longest.
# Time: O(n^2).  Space: O(n).
def longest_nice_by_masks(s):
    best = ""
    for i in range(len(s)):
        lower = upper = 0
        for j in range(i, len(s)):
            ch = s[j]
            if ch.islower():
                lower |= 1 << (ord(ch) - 97)
            else:
                upper |= 1 << (ord(ch) - 65)
            if lower == upper and j - i + 1 > len(best):
                best = s[i:j + 1]
    return best
```

427. Construct Quad Tree

Medium · Tree, Array, Divide and Conquer, Matrix

leetcode.com/problems/construct-quad-tree

★ **TOP 7 PROBLEM** in Divide and Conquer: master this one first

PROBLEM

Given a `n * n` matrix `grid` of `0's` and `1's` only. We want to represent `grid` with a Quad-Tree.

Return *the root of the Quad-Tree representing* `grid`.

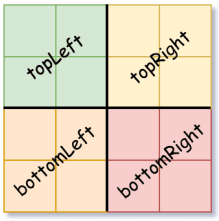
A Quad-Tree is a tree data structure in which each internal node has exactly four children. Besides, each node has two attributes:

- `val`: True if the node represents a grid of 1's or False if the node represents a grid of 0's. Notice that you can assign the `val` to True or False when `isLeaf` is False, and both are accepted in the answer.
- `isLeaf`: True if the node is a leaf node on the tree or False if the node has four children.

```
class Node {
    public boolean val;
    public boolean isLeaf;
    public Node topLeft;
    public Node topRight;
    public Node bottomLeft;
    public Node bottomRight;
}
```

We can construct a Quad-Tree from a two-dimensional area using the following steps:

1. If the current grid has the same value (i.e all `1's` or all `0's`) set `isLeaf` True and set `val` to the value of the grid and set the four children to Null and stop.
2. If the current grid has different values, set `isLeaf` to False and set `val` to any value and divide the current grid into four sub-grids as shown in the photo.
3. Recurse for each of the children with the proper sub-grid.



If you want to know more about the Quad-Tree, you can refer to the wiki.

Quad-Tree format:

You don't need to read this section for solving the problem. This is only if you want to understand the output format here. The output represents the serialized format of a Quad-Tree using level order traversal, where `null` signifies a path terminator where no node exists below.

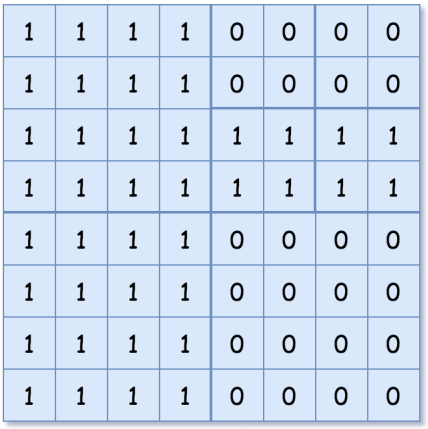
It is very similar to the serialization of the binary tree. The only difference is that the node is represented as a list `[isLeaf, val]`.

If the value of `isLeaf` or `val` is True we represent it as **1** in the list `[isLeaf, val]` and if the value of `isLeaf` or `val` is False we represent it as **0**.

Example 1:

```
Input: grid = [[0,1],[1,0]]
Output: [[0,1],[1,0],[1,1],[1,1],[1,0]]
Explanation: The explanation of this example is shown below:
Notice that 0 represents False and 1 represents True in the photo representing the Quad-Tree.
```

Example 2:




```
Input: grid = [[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0],[1,1,1,1,1,1,1,1],[1,1,1,1,1,1,1,1],[1,1,1,1,0,0,0,
0],[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0]]
Output: [[0,1],[1,1],[0,1],[1,1],[1,0],null,null,null,null,[1,0],[1,0],[1,1],[1,1]]
Explanation: All values in the grid are not the same. We divide the grid into four sub-grids.
The topLeft, bottomLeft and bottomRight each has the same value.
The topRight have different values so we divide it into 4 sub-grids where each has the same value.
Explanation is shown in the photo below:
```

Constraints:

- `n == grid.length == grid[i].length`
- `n == 2^x` where `0 <= x <= 6`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

A quad tree stores a uniform block as a leaf and otherwise splits into four. Rescanning every sub-rectangle repeats work, yet n≤ 64 and a four-way split keep the total scan acceptable. DFS inspects whether the current rectangle contains both 0 and 1. One value yields a leaf; otherwise recurse on the four quadrants. A leaf's val is that block's color. Testing uniformity before splitting avoids cutting a solid region.

Python 3

```
"""
# Definition for a QuadTree node.
class Node:
    def __init__(self, val, isLeaf, topLeft, topRight, bottomLeft, bottomRight):
        self.val = val
        self.isLeaf = isLeaf
        self.topLeft = topLeft
        self.topRight = topRight
        self.bottomLeft = bottomLeft
        self.bottomRight = bottomRight
"""


class Solution:
    def construct(self, grid: List[List[int]]) -> 'Node':
        def dfs(a, b, c, d):
            zero = one = 0
            for i in range(a, c + 1):
                for j in range(b, d + 1):
                    if grid[i][j] == 0:
                        zero = 1
                    else:
                        one = 1
            isLeaf = zero + one == 1
            val = isLeaf and one
            if isLeaf:
                return Node(grid[a][b], True)
            topLeft = dfs(a, b, (a + c) // 2, (b + d) // 2)
            topRight = dfs(a, (b + d) // 2 + 1, (a + c) // 2, d)
            bottomLeft = dfs((a + c) // 2 + 1, b, c, (b + d) // 2)
            bottomRight = dfs((a + c) // 2 + 1, (b + d) // 2 + 1, c, d)
            return Node(val, isLeaf, topLeft, topRight, bottomLeft, bottomRight)

        return dfs(0, 0, len(grid) - 1, len(grid[0]) - 1)
```

Complexity. Time: O(n^2 log n) for an n by n grid (every level scans the whole grid). Space: O(log n) recursion (plus the O(n^2) tree).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Turn a quad tree back into the grid
# 2. Count the nodes of the quad tree for a grid


# ========================================================
# FOLLOW-UP 1: Turn a quad tree back into the grid
# ========================================================

# ---- follow-up 1: helper functions ----
class QuadNode:
    def __init__(self, val, isLeaf, topLeft=None, topRight=None, bottomLeft=None, bottomRight=None):
        self.val = val
        self.isLeaf = isLeaf
        self.topLeft = topLeft
        self.topRight = topRight
        self.bottomLeft = bottomLeft
        self.bottomRight = bottomRight

# ---- follow-up 1: function ----
# Answer: Fill the four quarters recursively; a leaf fills
# its whole square.
# Time: O(n^2).  Space: O(n^2).
def quad_tree_to_grid(root, n):
    grid = [[0] * n for _ in range(n)]

    def fill(node, r, c, size):
```

```
        if node.isLeaf:
            for i in range(r, r + size):
                for j in range(c, c + size):
                    grid[i][j] = 1 if node.val else 0
            return
        half = size // 2
        fill(node.topLeft, r, c, half)
        fill(node.topRight, r, c + half, half)
        fill(node.bottomLeft, r + half, c, half)
        fill(node.bottomRight, r + half, c + half, half)

    fill(root, 0, 0, n)
    return grid


# ============================================================
# FOLLOW-UP 2: Count the nodes of the quad tree for a grid
# ============================================================

# ---- follow-up 2: function ----
# Answer: Build the tree recursively and count; a uniform
# square is a single leaf.
# Time: O(n^2 log n), the uniformity check repeats on
# every level.  Space: O(log n).
def count_quad_nodes(grid):
    def go(r, c, size):
        first = grid[r][c]
        if all(grid[i][j] == first for i in range(r, r + size) for j in range(c, c + size)):
            return 1
        half = size // 2
        return 1 + go(r, c, half) + go(r, c + half, half) + go(r + half, c, half) + go(r + half, c + half,
 half)

    return go(0, 0, len(grid))
```

PART 2 · DIVIDE AND CONQUER · PROBLEM 3 OF 3

654. Maximum Binary Tree

Medium · Stack, Tree, Array, Divide and Conquer, Binary Tree, Cartesian Tree, Monotonic Stack

leetcode.com/problems/maximum-binary-tree

★ **TOP 7 PROBLEM** in Divide and Conquer: master this one first

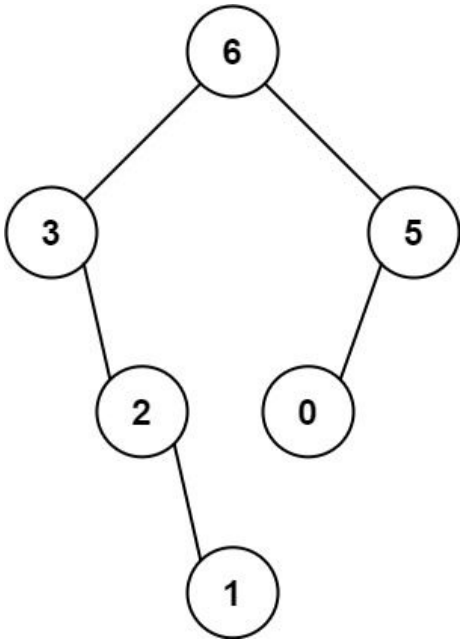
PROBLEM

You are given an integer array `nums` with no duplicates. A **maximum binary tree** can be built recursively from `nums` using the following algorithm:

1. Create a root node whose value is the maximum value in `nums`.
2. Recursively build the left subtree on the **subarray prefix** to the **left** of the maximum value.
3. Recursively build the right subtree on the **subarray suffix** to the **right** of the maximum value.

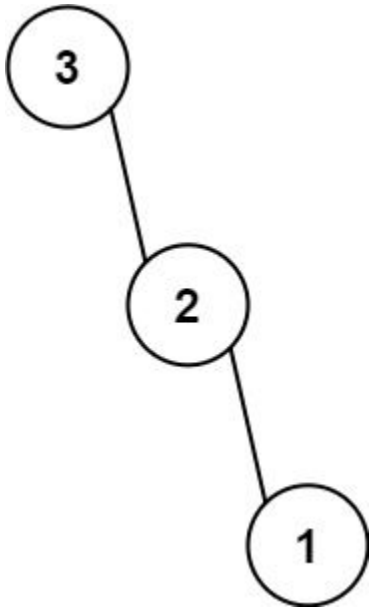
Return *the **maximum binary tree** built from* `nums`.

Example 1:




```
Input: nums = [3,2,1,6,0,5]
Output: [6,3,5,null,2,0,null,null,1]
Explanation: The recursive calls are as follow:
- The largest value in [3,2,1,6,0,5] is 6. Left prefix is [3,2,1] and right suffix is [0,5].
    - The largest value in [3,2,1] is 3. Left prefix is [] and right suffix is [2,1].
        - Empty array, so no child.
        - The largest value in [2,1] is 2. Left prefix is [] and right suffix is [1].
            - Empty array, so no child.
            - Only one element, so child is a node with value 1.
    - The largest value in [0,5] is 5. Left prefix is [0] and right suffix is [].
        - Only one element, so child is a node with value 0.
        - Empty array, so no child.
```

Example 2:




```
Input: nums = [3,2,1]
Output: [3,null,2,null,1]
```

Constraints:

- `1 <= nums.length <= 1000`
- `0 <= nums[i] <= 1000`
- All integers in `nums` are **unique**.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

The root is the interval maximum; children follow the same rule. A linear scan for the max is $O(n^2)$ worst-case, acceptable for $n \le 10^3$. Take `max` and its index, then recurse on the two sides.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def constructMaximumBinaryTree(self, nums: List[int]) -> Optional[TreeNode]:
        def dfs(nums):
            if not nums:
                return None
            val = max(nums)
            i = nums.index(val)
            root = TreeNode(val)
            root.left = dfs(nums[:i])
            root.right = dfs(nums[i + 1 :])
            return root

        return dfs(nums)
```

Solution 2

> **How to think about it**
>
> Each max query in method 1 is linear. A segment tree answers range maxima in O(log n), so the same recursion becomes O(nlog n).

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def constructMaximumBinaryTree(self, nums: List[int]) -> Optional[TreeNode]:
        def dfs(l, r):
            if l > r:
                return None
            val = tree.query(1, l, r)
            root = TreeNode(val)
            root.left = dfs(l, d[val] - 1)
            root.right = dfs(d[val] + 1, r)
            return root

        d = {v: i for i, v in enumerate(nums, 1)}
        tree = SegmentTree(nums)
        return dfs(1, len(nums))


class Node:
    def __init__(self):
        self.l = 0
        self.r = 0
        self.v = 0


class SegmentTree:
    def __init__(self, nums):
        self.nums = nums
        n = len(nums)
        self.tr = [Node() for _ in range(n << 2)]
        self.build(1, 1, n)

    def build(self, u, l, r):
        self.tr[u].l, self.tr[u].r = l, r
        if l == r:
            self.tr[u].v = self.nums[l - 1]
            return
        mid = (l + r) >> 1
        self.build(u << 1, l, mid)
        self.build(u << 1 | 1, mid + 1, r)
        self.pushup(u)

    def query(self, u, l, r):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            return self.tr[u].v
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        v = 0
        if l <= mid:
            v = max(v, self.query(u << 1, l, r))
        if r > mid:
            v = max(v, self.query(u << 1 | 1, l, r))
        return v

    def pushup(self, u):
        self.tr[u].v = max(self.tr[u << 1].v, self.tr[u << 1 | 1].v)
```

Solution 3

> **How to think about it**
>
> The segment tree is extra structure. A decreasing stack finds the nearest greater neighbor on the left: the last popped node becomes the left child, and the new top takes the current node as its right child. One pass builds the tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def constructMaximumBinaryTree(self, nums: List[int]) -> Optional[TreeNode]:
        stk = []
        for v in nums:
            node = TreeNode(v)
            last = None
            while stk and stk[-1].val < v:
                last = stk.pop()
            node.left = last
            if stk:
                stk[-1].right = node
            stk.append(node)
        return stk[0]
```

Complexity. Time: O(n). Space: O(n).

Solution 4: Explicit Stack

> **How to think about it**
>
> The root is the maximum of an interval, and both children follow the same rule. n ≤ 1000, so a linear scan for each maximum is acceptable in time. On an increasing array the maximum is always at the right end, so the first call always handles the remaining n-1 elements on the left and the depth is n, past the default recursion limit. The root of a segment depends only on its two endpoints, so the subarray does not need to be copied. Store pending half-open intervals [l, r) on an explicit stack, together with which side of the parent they hang from. Pop an interval, scan for its maximum, create the node, then push the right interval and the left interval. An empty interval is skipped.

Scan the half-open interval [l, r) for the index i of the maximum, and make nums[i] the root. The left child is the interval [l, i) and the right child is [i + 1, r). Pending intervals live on an explicit stack: after a root is created, push the right interval and then the left interval. Empty intervals are skipped. The first interval is [0, n).

The time complexity is $O(n^2)$, and the space complexity is O(n), where n is the length of the array.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def constructMaximumBinaryTree(self, nums: List[int]) -> Optional[TreeNode]:
        n = len(nums)
        root = None
        stk = [(0, n, None, 0)]
        while stk:
            l, r, parent, side = stk.pop()
            if l >= r:
                continue
            i = l
            for j in range(l + 1, r):
                if nums[j] > nums[i]:
                    i = j
            node = TreeNode(nums[i])
            if parent is None:
                root = node
            elif side == 0:
                parent.left = node
            else:
                parent.right = node
            stk.append((i + 1, r, node, 1))
            stk.append((l, i, node, 0))
        return root
```

Solution 5: Segment Tree + Explicit Stack

How to think about it

Solution 1 already builds the tree with an explicit stack, but each maximum is still a linear scan. A segment tree answers a range maximum in O(log n). The left interval can still form a chain of length n, so those two child intervals go on the same explicit stack. The total time is O(n log n).

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def constructMaximumBinaryTree(self, nums: List[int]) -> Optional[TreeNode]:
        d = {v: i for i, v in enumerate(nums, 1)}
        tree = SegmentTree(nums)
        root = None
        stk = [(1, len(nums), None, 0)]
        while stk:
            l, r, parent, side = stk.pop()
            if l > r:
                continue
            val = tree.query(1, l, r)
            node = TreeNode(val)
            i = d[val]
            if parent is None:
                root = node
            elif side == 0:
                parent.left = node
            else:
                parent.right = node
            stk.append((i + 1, r, node, 1))
            stk.append((l, i - 1, node, 0))
        return root


class Node:
    def __init__(self):
        self.l = 0
        self.r = 0
        self.v = 0


class SegmentTree:
    def __init__(self, nums):
        self.nums = nums
        n = len(nums)
        self.tr = [Node() for _ in range(n << 2)]
        self.build(1, 1, n)

    def build(self, u, l, r):
        self.tr[u].l, self.tr[u].r = l, r
        if l == r:
            self.tr[u].v = self.nums[l - 1]
            return
        mid = (l + r) >> 1
        self.build(u << 1, l, mid)
        self.build(u << 1 | 1, mid + 1, r)
        self.pushup(u)

    def query(self, u, l, r):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            return self.tr[u].v
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        v = 0
        if l <= mid:
            v = max(v, self.query(u << 1, l, r))
        if r > mid:
            v = max(v, self.query(u << 1 | 1, l, r))
        return v

    def pushup(self, u):
```

```
        self.tr[u].v = max(self.tr[u << 1].v, self.tr[u << 1 | 1].v)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Append a new value at the end of the array and update
# the tree
# 2. The MINIMUM binary tree (the root is the smallest)


# ============================================================
# FOLLOW-UP 1: Append a new value at the end of the array
# and update the tree
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: If the value is bigger than the root it becomes
# the new root with the old tree on its left; otherwise it
# goes down the right spine.
# Time: O(h), up to O(n).  Space: O(h).
def insert_into_max_tree(root, val):
    if root is None or val > root.val:
        return TreeNode(val, root)
    root.right = insert_into_max_tree(root.right, val)
    return root


# ============================================================
# FOLLOW-UP 2: The MINIMUM binary tree (the root is the
# smallest)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same construction with the smallest value as the
# root.
# Time: O(n^2) in the worst case.  Space: O(n).
def construct_minimum_tree(nums):
    if not nums:
        return None
    index = nums.index(min(nums))
    return TreeNode(nums[index], construct_minimum_tree(nums[:index]), construct_minimum_tree(nums[index +
1:]))
```

PART 2 · TOPIC 19 OF 50

Binary Search

9 problems · 4 Easy · 4 Medium · 1 Hard

THE IDEA

Search on a sorted array or on an answer that flips from false to true. Decide what lo and hi mean, and what the loop keeps true.

THE 9 PROBLEMS, EASY TO HARD | ★ = TOP 5: DO THESE FIRST

1	First Bad Version	Easy	LC 278	2294
2	Valid Perfect Square	Easy	LC 367	2296
3	Guess Number Higher or Lower	Easy	LC 374	2298
4	Find Smallest Letter Greater Than Target	Easy	LC 744	2300
5	★ **Search a 2D Matrix II**	Medium	LC 240	2302
6	★ **Heaters**	Medium	LC 475	2306
7	★ **Random Pick with Weight**	Medium	LC 528	2309
8	★ **Magnetic Force Between Two Balls**	Medium	LC 1552	2312
9	★ **Find in Mountain Array**	Hard	LC 1095	2315

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Binary Search (Part 1, pattern 4, page 184)

Find First and Last Position of Element in Sorted Array (Part 1, pattern 4, page 192)

Search in Rotated Sorted Array II (Part 1, pattern 4, page 197)

Find K Closest Elements (Part 1, pattern 4, page 228)

Capacity To Ship Packages Within D Days (Part 1, pattern 4, page 209)

Find Minimum in Rotated Sorted Array (Part 1, pattern 4, page 200)

Split Array Largest Sum (Part 1, pattern 4, page 236)

Find K-th Smallest Pair Distance (Part 1, pattern 38, page 1938)

Search Insert Position (Part 1, pattern 4, page 187)

Search in Rotated Sorted Array (Part 1, pattern 4, page 194)

Find Peak Element (Part 1, pattern 4, page 212)

Koko Eating Bananas (Part 1, pattern 4, page 206)

Minimum Number of Days to Make m Bouquets (Part 1, pattern 4, page 225)

Search a 2D Matrix (Part 1, pattern 4, page 202)

Median of Two Sorted Arrays (Part 1, pattern 4, page 233)

278. First Bad Version

Easy · Binary Search, Interactive

leetcode.com/problems/first-bad-version

PROBLEM

You are a product manager and currently leading a team to develop a new product. Unfortunately, the latest version of your product fails the quality check. Since each version is developed based on the previous version, all the versions after a bad version are also bad.

Suppose you have `n` versions `[1, 2, ..., n]` and you want to find out the first bad one, which causes all the following ones to be bad.

You are given an API `bool isBadVersion(version)` which returns whether `version` is bad. Implement a function to find the first bad version. You should minimize the number of calls to the API.

Example 1:

```
Input: n = 5, bad = 4
Output: 4
Explanation:
call isBadVersion(3) -> false
call isBadVersion(5) -> true
call isBadVersion(4) -> true
Then 4 is the first bad version.
```

Example 2:

```
Input: n = 1, bad = 1
Output: 1
```

Constraints:

- $1 <= bad <= n <= 2^{31} - 1$

SOLUTION (PYTHON 3)

Solution 1: Binary Search

> **How to think about it**
>
> Once a version is bad, every later one is bad. If isBadVersion(mid) then the first bad version lies in the left half including mid; otherwise it lies to the right. The search stops at l=r, the first bad version.

We define the left boundary of the binary search as l = 1 and the right boundary as r = n.

While l < r, we calculate the middle position mid = floor((l + r)/2), then call the `isBadVersion(mid)` API. If it returns true, it means the first bad version is between [l, mid], so we set r = mid; otherwise, the first bad version is between [mid + 1, r], so we set l = mid + 1.

Finally, we return l.

The time complexity is O(log n), and the space complexity is O(1).

Python 3

```
# The isBadVersion API is already defined for you.
# def isBadVersion(version: int) -> bool:


class Solution:
    def firstBadVersion(self, n: int) -> int:
        l, r = 1, n
        while l < r:
            mid = (l + r) >> 1
            if isBadVersion(mid):
                r = mid
            else:
                l = mid + 1
        return l
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Also report how many times isBadVersion was called
# 2. Versions can go bad and then good again: find ANY
# version that is bad right after a good one


# ============================================================
# FOLLOW-UP 1: Also report how many times isBadVersion was
# called
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same binary search with a counter.
# Time: O(log n).  Space: O(1).
def first_bad_with_calls(n, is_bad):
    calls = 0
    low, high = 1, n
    while low < high:
        middle = (low + high) // 2
        calls += 1
        if is_bad(middle):
            high = middle
        else:
            low = middle + 1
    return low, calls


# ============================================================
# FOLLOW-UP 2: Versions can go bad and then good again:
# find ANY version that is bad right after a good one
# ============================================================

# ---- follow-up 2: function ----
# Answer: Binary search for a boundary: keep a good left
# end and a bad right end.
# Time: O(log n).  Space: O(1).
def find_bad_after_good(n, is_bad):
    # needs version 1 good and version n bad
    low, high = 1, n
    while high - low > 1:
        middle = (low + high) // 2
        if is_bad(middle):
            high = middle
        else:
            low = middle
    return high
```

PART 2 · BINARY SEARCH · PROBLEM 2 OF 9

367. Valid Perfect Square

Easy · Math, Binary Search

leetcode.com/problems/valid-perfect-square

PROBLEM

Given a positive integer num, return `true` *if* `num` *is a perfect square or* `false` *otherwise*.

A **perfect square** is an integer that is the square of an integer. In other words, it is the product of some integer with itself.

You must not use any built-in library function, such as `sqrt`.

Example 1:

```
Input: num = 16
Output: true
Explanation: We return true because 4 * 4 = 16 and 4 is an integer.
```

Example 2:

```
Input: num = 14
Output: false
Explanation: We return false because 3.742 * 3.742 = 14 and 3.742 is not an integer.
```

Constraints:

- $1 <= num <= 2^{31} - 1$

SOLUTIONS (PYTHON 3)

Solution 1: Binary Search

> **How to think about it**
>
> Is num a perfect square? Scanning to √num is O(√n). Binary search on x^2 is enough. Find the least x in [1,num] with $x^2 \geq$ num, then test equality. The code uses `bisect_left` with $x \mapsto x^2$.

We can use binary search to solve this problem. Define the left boundary l = 1 and the right boundary r = num of the binary search, then find the smallest integer x that satisfies $x^2 \geq$ num in the range [l, r]. Finally, if x^2 = num, then num is a perfect square.

The time complexity is O(log n), where n is the given number. The space complexity is O(1).

Python 3

```python
class Solution:
    def isPerfectSquare(self, num: int) -> bool:
        l = bisect_left(range(1, num + 1), num, key=lambda x: x * x) + 1
        return l * l == num
```

Solution 2: Mathematics

> **How to think about it**
>
> Binary search needs the monotone squares. The odd sum 1+3+…+(2n-1)=n^2, so subtract increasing odds until num hits 0. Shorter code in O(√n).

Since 1 + 3 + 5 + … + (2n - 1) = n^2, we can gradually subtract 1, 3, 5, … from num. If num finally equals 0, then num is a perfect square.

The time complexity is O(√n), and the space complexity is O(1).

Python 3

```
class Solution:
    def isPerfectSquare(self, num: int) -> bool:
        i = 1
        while num > 0:
            num -= i
            i += 2
        return num == 0
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The integer square root (rounded down)
# 2. The smallest perfect square that is at least n


# ============================================================
# FOLLOW-UP 1: The integer square root (rounded down)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Binary search for the biggest x with x * x <= n.
# Time: O(log n).  Space: O(1).
def integer_sqrt(n):
    low, high = 0, n
    while low < high:
        middle = (low + high + 1) // 2
        if middle * middle <= n:
            low = middle
        else:
            high = middle - 1
    return low


# ============================================================
# FOLLOW-UP 2: The smallest perfect square that is at
# least n
# ============================================================

# ---- follow-up 2: function ----
# Answer: Take the integer square root of n - 1, add one,
# and square it.
# Time: O(log n).  Space: O(1).
def next_perfect_square(n):
    if n <= 0:
        return 0
    root = integer_sqrt(n - 1) + 1
    return root * root
```

374. Guess Number Higher or Lower

Easy · Binary Search, Interactive

leetcode.com/problems/guess-number-higher-or-lower

PROBLEM

We are playing the Guess Game. The game is as follows:

I pick a number from `1` to `n`. You have to guess which number I picked (the number I picked stays the same throughout the game).

Every time you guess wrong, I will tell you whether the number I picked is higher or lower than your guess.

You call a pre-defined API `int guess(int num)`, which returns three possible results:

- `-1`: Your guess is higher than the number I picked (i.e. `num > pick`).
- `1`: Your guess is lower than the number I picked (i.e. `num < pick`).
- `0`: your guess is equal to the number I picked (i.e. `num == pick`).

Return *the number that I picked*.

Example 1:

```
Input: n = 10, pick = 6
Output: 6
```

Example 2:

```
Input: n = 1, pick = 1
Output: 1
```

Example 3:

```
Input: n = 2, pick = 1
Output: 1
```

Constraints:

- `1 <= n <= 2^31 - 1`
- `1 <= pick <= n`

SOLUTION (PYTHON 3)

Solution 1: Binary Search

> **How to think about it**
>
> Guess a number in [1,n]; `guess` says higher or lower. Linear probes take n tries. The range is sorted, so binary search. Find the first x with `guess(x)\le 0`. The code uses key -guess(x) and a single `bisect`.

We perform a binary search in the interval [1,..n], and find the first number that satisfies `guess(x) <= 0`, which is the answer.

The time complexity is O(log n), where n is the upper limit given in the problem. The space complexity is O(1).

Python 3

```
# The guess API is already defined for you.
# @param num, your guess
# @return -1 if num is higher than the picked number
#          1 if num is lower than the picked number
#          otherwise return 0
# def guess(num: int) -> int:


class Solution:
    def guessNumber(self, n: int) -> int:
        return bisect.bisect(range(1, n + 1), 0, key=lambda x: -guess(x))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. How many guesses are needed in the worst case for
# 1..n
# 2. Return the guess count as well as the number


# ============================================================
# FOLLOW-UP 1: How many guesses are needed in the worst
# case for 1..n
# ============================================================

# ---- follow-up 1: function ----
# Answer: Every guess halves the range: ceil(log2(n + 1))
# guesses.
# Time: O(log n).  Space: O(1).
def worst_case_guesses(n):
    guesses = 0
    while (1 << guesses) - 1 < n:
        guesses += 1
    return guesses


# ============================================================
# FOLLOW-UP 2: Return the guess count as well as the
# number
# ============================================================

# ---- follow-up 2: function ----
# Answer: Binary search with a counter; the helper returns
# -1, 1 or 0.
# Time: O(log n).  Space: O(1).
def guess_with_count(n, compare):
    # compare(x) is -1 when the secret is lower, 1 when higher, 0 when equal
    low, high = 1, n
    count = 0
    while low <= high:
        middle = (low + high) // 2
        count += 1
        answer = compare(middle)
        if answer == 0:
            return middle, count
        if answer < 0:
            high = middle - 1
        else:
            low = middle + 1
    return None, count
```

744. Find Smallest Letter Greater Than Target

Easy · Array, Binary Search

leetcode.com/problems/find-smallest-letter-greater-than-target

PROBLEM

You are given an array of characters `letters` that is sorted in **non-decreasing order**, and a character `target`. There are **at least two different** characters in `letters`.

Return *the smallest character in* `letters` *that is lexicographically greater than* `target`. If such a character does not exist, return the first character in `letters`.

Example 1:

```
Input: letters = ["c","f","j"], target = "a"
Output: "c"
Explanation: The smallest character that is lexicographically greater than 'a' in letters is 'c'.
```

Example 2:

```
Input: letters = ["c","f","j"], target = "c"
Output: "f"
Explanation: The smallest character that is lexicographically greater than 'c' in letters is 'f'.
```

Example 3:

```
Input: letters = ["x","x","y","y"], target = "z"
Output: "x"
Explanation: There are no characters in letters that is lexicographically greater than 'z' so we
return letters[0].
```

Constraints:

- `2 <= letters.length <= 10^4`
- `letters[i]` is a lowercase English letter.
- `letters` is sorted in **non-decreasing** order.
- `letters` contains at least two different characters.
- `target` is a lowercase English letter.

SOLUTION (PYTHON 3)

Solution 1: Binary Search

> **How to think about it**
>
> Among non-decreasing letters, find the smallest one strictly greater than target, wrapping to the front. The array is sorted, so we binary-search. That is an upper bound. If it falls at n, the answer is letters[0]—index modulo n. bisect_right on the code points, then letters[i mod n]. O(log n).

Since `letters` is sorted in non-decreasing order, we can use binary search to find the smallest character that is larger than `target`.

We define the left boundary of the binary search as l = 0, and the right boundary as r = n. For each binary search, we calculate the middle position mid = (l + r) / 2. If letters[mid] > target, it means we need to continue searching in the left half, so we set r = mid. Otherwise, we need to continue searching in the right half, so we set l = mid + 1.

Finally, we return letters[l mod n].

The time complexity is O(log n), where n is the length of `letters`. The space complexity is O(1).

Python 3

```
class Solution:
    def nextGreatestLetter(self, letters: List[str], target: str) -> str:
        i = bisect_right(letters, ord(target), key=lambda c: ord(c))
        return letters[i % len(letters)]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The smallest letter that is greater than OR EQUAL to
# the target (wrap around)
# 2. The largest letter that is smaller than the target
# (wrap around)


# ============================================================
# FOLLOW-UP 1: The smallest letter that is greater than OR
# EQUAL to the target (wrap around)
# ============================================================

# ---- follow-up 1: function ----
# Answer: bisect_left instead of bisect_right.
# Time: O(log n).  Space: O(1).
import bisect


def smallest_letter_at_least(letters, target):
    i = bisect.bisect_left(letters, target)
    return letters[i % len(letters)]


# ============================================================
# FOLLOW-UP 2: The largest letter that is smaller than the
# target (wrap around)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Step back from the position where the target
# would be inserted.
# Time: O(log n).  Space: O(1).
def largest_letter_below(letters, target):
    i = bisect.bisect_left(letters, target)
    return letters[i - 1]
```

240. Search a 2D Matrix II

Medium · Array, Binary Search, Divide and Conquer, Matrix

leetcode.com/problems/search-a-2d-matrix-ii

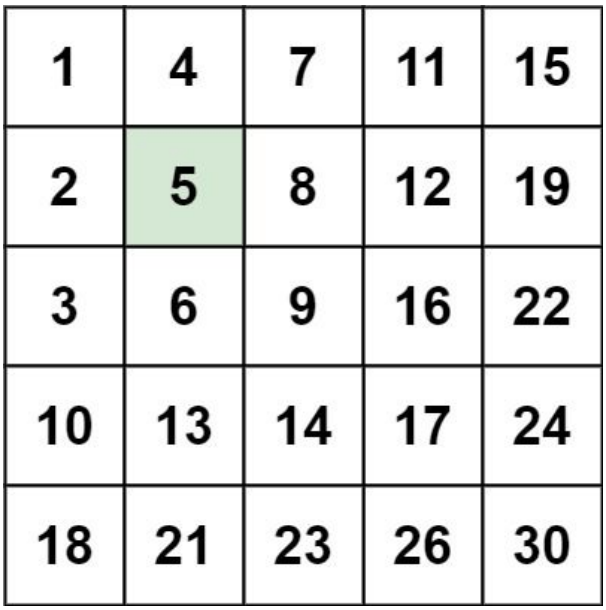
★ **TOP 7 PROBLEM** in Binary Search: master this one first

PROBLEM

Write an efficient algorithm that searches for a value `target` in an `m x n` integer matrix `matrix`. This matrix has the following properties:

- Integers in each row are sorted in ascending from left to right.
- Integers in each column are sorted in ascending from top to bottom.

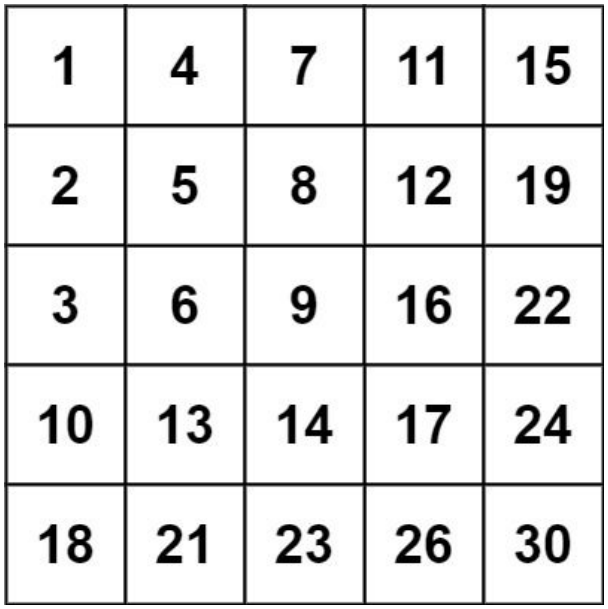
Example 1:



1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

```
Input: matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]], target = 5
Output: true
```

Example 2:



1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

```
Input: matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]], target
= 20
Output: false
```

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= n, m <= 300`
- -10^9 `<= matrix[i][j] <=` 10^9
- All the integers in each row are **sorted** in ascending order.
- All the integers in each column are **sorted** in ascending order.
- -10^9 `<= target <=` 10^9

SOLUTIONS (PYTHON 3)

Solution 1: Binary Search

> **How to think about it**
> Each row (and column) is sorted, so we can binary-search target in every row. That is O(mlog n) with no extra memory.

Since all elements in each row are sorted in ascending order, for each row, we can use binary search to find the first element greater than or equal to target, and then check if that element is equal to target. If it is equal to target, it means the target value is found, and we return true. If it is not equal to target, it means all elements in this row are less than target, and we should continue searching the next row.

If all rows have been searched and the target value is not found, it means the target value does not exist, and we return false.

The time complexity is O(m × log n), where m and n are the number of rows and columns of the matrix, respectively. The space complexity is O(1).

Python 3

```python
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        for row in matrix:
            j = bisect_left(row, target)
            if j < len(matrix[0]) and row[j] == target:
                return True
        return False
```

Solution 2: Search from Bottom-Left or Top-Right

> **How to think about it**
>
> Per-row binary search ignores column order. From the bottom-left, a too-large value moves up (dropping that column above) and a too-small value moves right (dropping that row to the right). Each step discards a row or a column, in O(m+n) time.

We start the search from the bottom-left or top-right corner and move towards the top-right or bottom-left direction. Compare the current element matrix[i][j] with target:

- If matrix[i][j] = target, it means the target value is found, and we return true.
- If matrix[i][j] > target, it means all elements in this column from the current position upwards are greater than target, so we move the i pointer upwards, i.e., i ← i - 1.
- If matrix[i][j] < target, it means all elements in this row from the current position to the right are less than target, so we move the j pointer to the right, i.e., j ← j + 1.

If the search ends and the target is not found, return false.

The time complexity is O(m + n), where m and n are the number of rows and columns of the matrix, respectively. The space complexity is O(1).

Python 3

```python
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        i, j = m - 1, 0
        while i >= 0 and j < n:
            if matrix[i][j] == target:
                return True
            if matrix[i][j] > target:
                i -= 1
            else:
                j += 1
        return False
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count how many times the target appears
# 2. How many numbers are at most x


# ============================================================
# FOLLOW-UP 1: Count how many times the target appears
# ============================================================

# ---- follow-up 1: function ----
# Answer: Every row is sorted, so two binary searches per
# row give the number of copies.
# Time: O(m log n).  Space: O(1).
import bisect
```

```
def count_in_matrix(matrix, target):
    return sum(bisect.bisect_right(row, target) - bisect.bisect_left(row, target) for row in matrix)


# =========================================================
# FOLLOW-UP 2: How many numbers are at most x
# =========================================================

# ---- follow-up 2: function ----
# Answer: Staircase walk: in every row count the numbers
# <= x with a pointer that only moves left.
# Time: O(m + n).  Space: O(1).
def count_at_most(matrix, x):
    total = 0
    col = len(matrix[0]) - 1
    for row in range(len(matrix)):
        while col >= 0 and matrix[row][col] > x:
            col -= 1
        total += col + 1
    return total
```

PART 2 · BINARY SEARCH · PROBLEM 6 OF 9

475. Heaters

Medium · Array, Two Pointers, Binary Search, Sorting

leetcode.com/problems/heaters

★ **TOP 7 PROBLEM** in Binary Search: master this one first

PROBLEM

Winter is coming! During the contest, your first job is to design a standard heater with a fixed warm radius to warm all the houses.

Every house can be warmed, as long as the house is within the heater's warm radius range.

Given the positions of `houses` and `heaters` on a horizontal line, return *the minimum radius standard of heaters so that those heaters could cover all houses.*

Notice that all the `heaters` follow your radius standard, and the warm radius will be the same.

Example 1:

```
Input: houses = [1,2,3], heaters = [2]
Output: 1
Explanation: The only heater was placed in the position 2, and if we use the radius 1 standard, then all the houses can be warmed.
```

Example 2:

```
Input: houses = [1,2,3,4], heaters = [1,4]
Output: 1
Explanation: The two heaters were placed at positions 1 and 4. We need to use a radius 1 standard, then all the houses can be warmed.
```

Example 3:

```
Input: houses = [1,5], heaters = [2]
Output: 3
```

Constraints:

- `1 <= houses.length, heaters.length <= 3 * 10^4`
- `1 <= houses[i], heaters[i] <= 10^9`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Heaters must cover every house; we want the smallest common radius. Trying every radius is huge, yet feasibility grows with the radius. Sort both arrays and binary-search r. A two-pointer check lets each heater cover [h-r,h+r]: a house to the left fails, a house to the right needs the next heater. Both sequences are sorted, so the pointers only move forward and one check is linear.

Python 3

```python
class Solution:
    def findRadius(self, houses: List[int], heaters: List[int]) -> int:
        houses.sort()
        heaters.sort()

        def check(r):
            m, n = len(houses), len(heaters)
            i = j = 0
            while i < m:
                if j >= n:
                    return False
                mi = heaters[j] - r
                mx = heaters[j] + r
                if houses[i] < mi:
                    return False
                if houses[i] > mx:
                    j += 1
                else:
                    i += 1
            return True

        left, right = 0, int(1e9)
        while left < right:
            mid = (left + right) >> 1
            if check(mid):
                right = mid
            else:
                left = mid + 1
        return left
```

Complexity. Time: O(m log m + n log n + (m + n) log R), R = largest radius searched. Space: O(log n) for the sorts.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Which heater warms each house
# 2. You may switch on only k of the heaters: the smallest
# radius


# ============================================================
# FOLLOW-UP 1: Which heater warms each house
# ============================================================

# ---- follow-up 1: function ----
# Answer: Binary search the closest heater for every
# house.
# Time: O((h + n) log h).  Space: O(h + n).
import bisect


def nearest_heater(houses, heaters):
    heaters = sorted(heaters)
    result = []
    for h in houses:
        i = bisect.bisect_left(heaters, h)
        options = []
        if i > 0:
            options.append(heaters[i - 1])
        if i < len(heaters):
            options.append(heaters[i])
        result.append(min(options, key=lambda p: abs(p - h)))
    return result


# ============================================================
```

```
# FOLLOW-UP 2: You may switch on only k of the heaters:
# the smallest radius
# ============================================================

# ---- follow-up 2: function ----
# Answer: Binary search on the radius; greedily place the
# next heater that covers the first uncovered house.
# Time: O((k * h + n) log R), R = largest distance.
# Space: O(h + n).
def min_radius_with_k_heaters(houses, heaters, k):
    houses = sorted(houses)
    heaters = sorted(heaters)

    def enough(radius):
        used = 0
        i = 0
        while i < len(houses):
            used += 1
            if used > k:
                return False
            # the heater farthest to the right that still covers houses[i]
            best = None
            for p in heaters:
                if abs(p - houses[i]) <= radius:
                    best = p
            if best is None:
                return False
            while i < len(houses) and abs(best - houses[i]) <= radius:
```

```
                i += 1
        return True

    low, high = 0, max(houses[-1], heaters[-1]) - min(houses[0], heaters[0])
    while low < high:
        middle = (low + high) // 2
        if enough(middle):
            high = middle
        else:
            low = middle + 1
    return low
```

528. Random Pick with Weight

Medium · Array, Math, Binary Search, Prefix Sum, Randomized

leetcode.com/problems/random-pick-with-weight

★ **TOP 7 PROBLEM** in Binary Search: master this one first

PROBLEM

You are given a **0-indexed** array of positive integers `w` where `w[i]` describes the **weight** of the i^{th} index.

You need to implement the function `pickIndex()`, which **randomly** picks an index in the range `[0, w.length - 1]` (**inclusive**) and returns it. The **probability** of picking an index `i` is `w[i] / sum(w)`.

- For example, if `w = [1, 3]`, the probability of picking index `0` is `1 / (1 + 3) = 0.25` (i.e., `25%`), and the probability of picking index `1` is `3 / (1 + 3) = 0.75` (i.e., `75%`).

Example 1:

```
Input
["Solution","pickIndex"]
[[[1]],[]]
Output
[null,0]

Explanation
Solution solution = new Solution([1]);
solution.pickIndex(); // return 0. The only option is to return 0 since there is only one element in w.
```

Example 2:

```
Input
["Solution","pickIndex","pickIndex","pickIndex","pickIndex","pickIndex"]
[[[1,3]],[],[],[],[],[]]
Output
[null,1,1,1,1,0]

Explanation
Solution solution = new Solution([1, 3]);
solution.pickIndex(); // return 1. It is returning the second element (index = 1) that has a
probability of 3/4.
solution.pickIndex(); // return 1
solution.pickIndex(); // return 1
solution.pickIndex(); // return 1
solution.pickIndex(); // return 0. It is returning the first element (index = 0) that has a
probability of 1/4.

Since this is a randomization problem, multiple answers are allowed.
All of the following outputs can be considered correct:
[null,1,1,1,1,0]
[null,1,1,1,1,1]
[null,1,1,1,0,0]
[null,1,1,1,0,1]
[null,1,0,1,0,0]
......
and so on.
```

Constraints:

- `1 <= w.length <= 10^4`
- `1 <= w[i] <= 10^5`
- `pickIndex` will be called at most `10^4` times.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

We must sample index i with probability proportional to w[i], many times, so a linear scan per call is wasteful. Prefix sums turn weights into intervals on [1, S]. Draw a uniform integer and binary-search the interval. Build is O(n) and each pick is O(log n). Subtract one because the prefix array is padded with a leading 0.

Python 3

```
class Solution:
    def __init__(self, w: List[int]):
        self.s = [0]
        for c in w:
            self.s.append(self.s[-1] + c)

    def pickIndex(self) -> int:
        x = random.randint(1, self.s[-1])
        left, right = 1, len(self.s) - 1
        while left < right:
            mid = (left + right) >> 1
            if self.s[mid] >= x:
                right = mid
            else:
                left = mid + 1
        return left - 1


# Your Solution object will be instantiated and called as such:
# obj = Solution(w)
# param_1 = obj.pickIndex()
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Weights can change: update(i, w)
# 2. Pick k different indices (no repeats), each pick
# still weighted


# ======================================================
# FOLLOW-UP 1: Weights can change: update(i, w)
# ======================================================

# ---- follow-up 1: function ----
# Answer: Keep the prefix sums in a Fenwick tree and pick
# by searching the tree.
# Time: build O(n log n), update O(log n), pick_index O(n)
# for the total plus O(log n).  Space: O(n).
import random


class WeightedPickerWithUpdate:
    def __init__(self, w):
        self.w = list(w)
        self.n = len(w)
        self.tree = [0] * (self.n + 1)
        for i, x in enumerate(w):
            self._add(i, x)

    def _add(self, i, delta):
        i += 1
        while i <= self.n:
```

```
            self.tree[i] += delta
            i += i & -i

    def update(self, i, weight):
        self._add(i, weight - self.w[i])
        self.w[i] = weight

    def pick_by(self, target):
        # the first index whose prefix sum is greater than target (target in 0 .. total - 1)
        position = 0
        step = 1
        while step * 2 <= self.n:
            step *= 2
        while step:
            nxt = position + step
            if nxt <= self.n and self.tree[nxt] <= target:
                position = nxt
                target -= self.tree[nxt]
            step //= 2
        return position

    def pick_index(self):
        total = sum(self.w)
        return self.pick_by(random.randrange(total))


# ============================================================
# FOLLOW-UP 2: Pick k different indices (no repeats), each
# pick still weighted
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Pick one, set its weight to zero in a copy, and
# repeat.
# Time: O(k * n).  Space: O(n).
def pick_k_distinct(weights, k):
    weights = list(weights)
    chosen = []
    for _ in range(k):
        total = sum(weights)
        if total == 0:
            break
        target = random.randrange(total)
        for i, x in enumerate(weights):
            if target < x:
                chosen.append(i)
                weights[i] = 0
                break
            target -= x
    return chosen
```

1552. Magnetic Force Between Two Balls

Medium · Array, Binary Search, Sorting

leetcode.com/problems/magnetic-force-between-two-balls

★ **TOP 7 PROBLEM** in Binary Search: master this one first

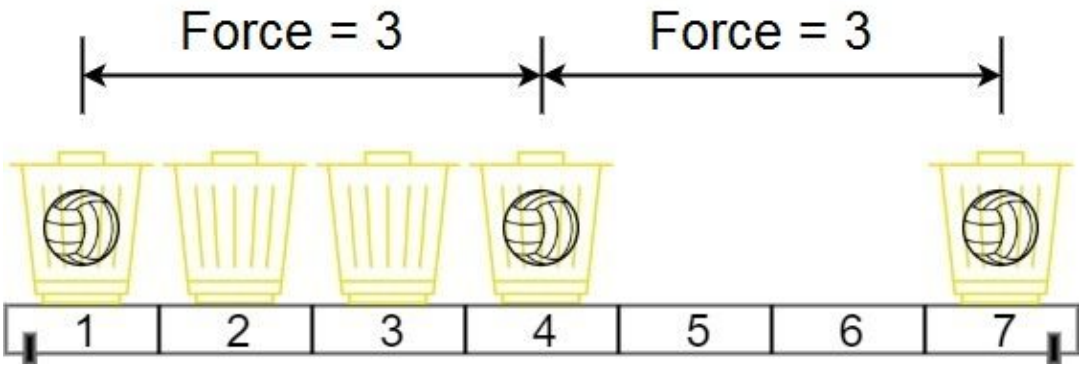
PROBLEM

In the universe Earth C-137, Rick discovered a special form of magnetic force between two balls if they are put in his new invented basket. Rick has `n` empty baskets, the `i`th basket is at `position[i]`, Morty has `m` balls and needs to distribute the balls into the baskets such that the **minimum magnetic force** between any two balls is **maximum**.

Rick stated that magnetic force between two different balls at positions `x` and `y` is `|x - y|`.

Given the integer array `position` and the integer `m`. Return *the required force*.

Example 1:




```
Input: position = [1,2,3,4,7], m = 3
Output: 3
Explanation: Distributing the 3 balls into baskets 1, 4 and 7 will make the magnetic force between ball pairs [3, 3, 6]. The minimum magnetic force is 3. We cannot achieve a larger minimum magnetic force than 3.
```

Example 2:

```
Input: position = [5,4,3,2,1,1000000000], m = 2
Output: 999999999
Explanation: We can use baskets 1 and 1000000000.
```

Constraints:

- `n == position.length`
- `2 <= n <= 10`^{5}
- `1 <= position[i] <= 10`^{9}
- All integers in `position` are **distinct**.
- `2 <= m <= position.length`

SOLUTION (PYTHON 3)

Solution 1: Binary Search

How to think about it

Place m balls to maximize the minimum gap. Positions reach 10^9, so we cannot enumerate layouts by gap; n≤ 10^5 still allows a fast feasibility test. Larger gaps admit fewer balls, which is monotone. After sorting, binary-search a gap f and scan left to right, placing a ball only when it is at least f from the previous one. Treat “cannot place m balls” as the search key; the predecessor of the first failing f is the largest feasible gap.

We notice that the greater the minimum magnetic force between any two balls, the fewer balls can be placed, which exhibits monotonicity. We can use binary search to find the maximum minimum magnetic force that allows the number of balls not less than m to be placed.

First, we sort the positions of the baskets, and then use binary search with the left boundary l = 1 and the right boundary r = position[n - 1], where n is the number of baskets. In each binary search iteration, we calculate the midpoint m = (l + r + 1) / 2, and then determine if there is a way to place the balls such that the number of balls placed is not less than m.

The problem is transformed into determining whether a given minimum magnetic force f can place m balls. We can traverse the positions of the baskets from left to right, and if the distance between the position of the last ball and the current basket's position is greater than or equal to f, it indicates that a ball can be placed in the current basket. Finally, we check if the number of balls placed is not less than m.

The time complexity is O(n × log n + n × log M), and the space complexity is O(log n). Here, n and M respectively represent the number of baskets and the maximum value of the basket positions.

Python 3

```python
class Solution:
    def maxDistance(self, position: List[int], m: int) -> int:
        def check(f: int) -> bool:
            prev = -inf
            cnt = 0
            for curr in position:
                if curr - prev >= f:
                    prev = curr
                    cnt += 1
            return cnt < m

        position.sort()
        l, r = 1, position[-1]
        return bisect_left(range(l, r + 1), True, key=check)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the positions of the balls for the best answer
# 2. How many balls fit with every two at least d apart


# ============================================================
# FOLLOW-UP 1: Return the positions of the balls for the
# best answer
# ============================================================

# ---- follow-up 1: function ----
# Answer: Binary search the distance, then replay the
# greedy placement.
# Time: O(n log n + n log D), D = largest gap.  Space:
# O(n).
def best_positions(position, m):
    position = sorted(position)
```

```
    def place(d):
        chosen = [position[0]]
        for p in position[1:]:
            if p - chosen[-1] >= d:
                chosen.append(p)
        return chosen

    low, high = 1, position[-1] - position[0]
    while low < high:
        middle = (low + high + 1) // 2
        if len(place(middle)) >= m:
            low = middle
        else:
            high = middle - 1
    return low, place(low)[:m]


# ============================================================
# FOLLOW-UP 2: How many balls fit with every two at least
# d apart
# ============================================================

# ---- follow-up 2: function ----
# Answer: Greedy placement from the left.
# Time: O(n log n).  Space: O(n).
def balls_that_fit(position, d):
    position = sorted(position)
    count = 1
    last = position[0]
    for p in position[1:]:
        if p - last >= d:
            count += 1
            last = p
    return count
```

1095. Find in Mountain Array

Hard · Array, Binary Search, Interactive, Ternary Search

leetcode.com/problems/find-in-mountain-array

★ **TOP 7 PROBLEM** in Binary Search: master this one first

PROBLEM

*(This problem is an **interactive problem**.)*

You may recall that an array `arr` is a **mountain array** if and only if:

- `arr.length >= 3`
- There exists some `i` with `0 < i < arr.length - 1` such that:
  - `arr[0] < arr[1] < ... < arr[i - 1] < arr[i]`
  - `arr[i] > arr[i + 1] > ... > arr[arr.length - 1]`

Given a mountain array `mountainArr`, return the **minimum** `index` such that `mountainArr.get(index) == target`. If such an `index` does not exist, return `-1`.

You cannot access the mountain array directly. You may only access the array using a `MountainArray` interface:

- `MountainArray.get(k)` returns the element of the array at index `k` (0-indexed).
- `MountainArray.length()` returns the length of the array.

Submissions making more than `100` calls to `MountainArray.get` will be judged *Wrong Answer*. Also, any solutions that attempt to circumvent the judge will result in disqualification.

Example 1:

```
Input: mountainArr = [1,2,3,4,5,3,1], target = 3
Output: 2
Explanation: 3 exists in the array, at index=2 and index=5. Return the minimum index, which is 2.
```

Example 2:

```
Input: mountainArr = [0,1,2,4,2,1], target = 3
Output: -1
Explanation: 3 does not exist in the array, so we return -1.
```

Constraints:

- `3 <= mountainArr.length() <= 10^4`
- `0 <= target <= 10^9`
- `0 <= mountainArr.get(index) <= 10^9`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

The mountain is available only through `get`, and $n \le 10^4$ requires logarithmic queries. Find the peak, then binary-search the increasing left and the decreasing right. Comparing get(mid) with get(mid+1) locates the peak. `search` uses $k \pm 1$ so the same lower-bound loop works on both sides. The left half is tried first so the smallest index is returned.

Python 3

```
# """
# This is MountainArray's API interface.
# You should not implement it, or speculate about its implementation
# """
# class MountainArray:
#    def get(self, index: int) -> int:
#    def length(self) -> int:


class Solution:
    def findInMountainArray(self, target: int, mountain_arr: 'MountainArray') -> int:
        def search(l: int, r: int, k: int) -> int:
            while l < r:
                mid = (l + r) >> 1
                if k * mountain_arr.get(mid) >= k * target:
                    r = mid
                else:
                    l = mid + 1
            return -1 if mountain_arr.get(l) != target else l

        n = mountain_arr.length()
        l, r = 0, n - 1
        while l < r:
            mid = (l + r) >> 1
            if mountain_arr.get(mid) > mountain_arr.get(mid + 1):
                r = mid
            else:
                l = mid + 1
        ans = search(0, l, 1)
        return search(l + 1, n - 1, -1) if ans == -1 else ans
```

Complexity. Time: O(log n) calls to get. Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Only find the peak index
# 2. Find ALL the indices of the target


# ============================================================
# FOLLOW-UP 1: Only find the peak index
# ============================================================

# ---- follow-up 1: function ----
# Answer: Binary search on the slope.
# Time: O(log n).  Space: O(1).
def peak_of_mountain(mountain):
    low, high = 0, len(mountain) - 1
    while low < high:
        middle = (low + high) // 2
        if mountain[middle] < mountain[middle + 1]:
            low = middle + 1
        else:
            high = middle
    return low


# ============================================================
# FOLLOW-UP 2: Find ALL the indices of the target
# ============================================================

# ---- follow-up 2: function ----
# Answer: Binary search the rising side and the falling
# side separately.
# Time: O(log n).  Space: O(1).
```

```
def all_indices_in_mountain(mountain, target):
    peak = peak_of_mountain(mountain)
    found = []
    low, high = 0, peak
    while low <= high:
        middle = (low + high) // 2
        if mountain[middle] == target:
            found.append(middle)
            break
        if mountain[middle] < target:
            low = middle + 1
        else:
            high = middle - 1
    low, high = peak + 1, len(mountain) - 1
    while low <= high:
        middle = (low + high) // 2
        if mountain[middle] == target:
            found.append(middle)
            break
        if mountain[middle] > target:
            low = middle + 1
        else:
            high = middle - 1
    return sorted(found)
```

PART 2 · TOPIC 20 OF 50

Backtracking

1 problem · 0 Easy · 0 Medium · 1 Hard

THE IDEA

Build the answer one choice at a time, undo the choice after the recursive call, and cut branches that cannot lead to a valid answer.

THE 1 PROBLEMS, EASY TO HARD | ★ = TOP 1: DO THESE FIRST

1	★ **Remove Invalid Parentheses**	Hard	LC 301	2319

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Generate Parentheses (Part 1, pattern 8, page 404)
Permutations (Part 1, pattern 15, page 796)
Permutations II (Part 1, pattern 15, page 799)
Subsets (Part 1, pattern 15, page 780)
Subsets II (Part 1, pattern 15, page 793)
Combination Sum (Part 1, pattern 15, page 789)
Combination Sum II (Part 1, pattern 15, page 802)
Combination Sum III (Part 1, pattern 15, page 805)
Combinations (Part 1, pattern 15, page 786)
Letter Combinations of a Phone Number (Part 1, pattern 15, page 809)
Restore IP Addresses (Part 1, pattern 15, page 822)
Palindrome Partitioning (Part 1, pattern 15, page 819)
Unique Paths III (Part 1, pattern 18, page 1033)
N-Queens (Part 1, pattern 15, page 834)
Sudoku Solver (Part 1, pattern 15, page 837)

301. Remove Invalid Parentheses

Hard · Breadth-First Search, String, Backtracking

leetcode.com/problems/remove-invalid-parentheses

★ **TOP 7 PROBLEM** in Backtracking: master this one first

PROBLEM

Given a string `s` that contains parentheses and letters, remove the minimum number of invalid parentheses to make the input string valid.

Return *a list of **unique strings** that are valid with the minimum number of removals*. You may return the answer in **any order**.

Example 1:

```
Input: s = "()())()"
Output: ["(())()","()()()"]
```

Example 2:

```
Input: s = "(a)())()"
Output: ["(a())()","(a)()()"]
```

Example 3:

```
Input: s = ")("
Output: [""]
```

Constraints:

- `1 <= s.length <= 25`
- `s` consists of lowercase English letters and parentheses `'('` and `')'`.
- There will be at most `20` parentheses in `s`.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

We must delete as few parentheses as possible and list every optimal string. Choosing delete-or-keep at each parenthesis branches too widely, and most paths are not minimal. A scan first yields lower bounds l and r on deletions. The search tracks remaining quotas and the current left/right counts; prune when leftover characters cannot finish the deletions or a prefix has more right parentheses than left. Try deleting (if quota remains) then keeping, and deduplicate with a set. Only paths that delete exactly the minimum survive.

Python 3

```python
class Solution:
    def removeInvalidParentheses(self, s: str) -> List[str]:
        def dfs(i, l, r, lcnt, rcnt, t):
            if i == n:
                if l == 0 and r == 0:
                    ans.add(t)
                return
            if n - i < l + r or lcnt < rcnt:
                return
            if s[i] == '(' and l:
                dfs(i + 1, l - 1, r, lcnt, rcnt, t)
            elif s[i] == ')' and r:
                dfs(i + 1, l, r - 1, lcnt, rcnt, t)
            dfs(i + 1, l, r, lcnt + (s[i] == '('), rcnt + (s[i] == ')'), t + s[i])

        l = r = 0
        for c in s:
            if c == '(':
                l += 1
            elif c == ')':
                if l:
                    l -= 1
                else:
                    r += 1
        ans = set()
        n = len(s)
        dfs(0, l, r, 0, 0, '')
        return list(ans)
```

Complexity. Time: O(2^n) in the worst case (heavily pruned in practice). Space: O(n) recursion (plus the answers).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Only the minimum number of removals
# 2. How many different valid strings are there


# ============================================================
# FOLLOW-UP 1: Only the minimum number of removals
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count unmatched ')' and unmatched '(' with one
# scan.
# Time: O(n).  Space: O(1).
def min_removals(s):
    opens = closes = 0
    for ch in s:
        if ch == "(":
            opens += 1
        elif ch == ")":
            if opens:
                opens -= 1
            else:
                closes += 1
    return opens + closes


# ============================================================
# FOLLOW-UP 2: How many different valid strings are there
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same search as the question, counting the
# distinct results.
```

```python
# Time: O(2^n * n) in the worst case.  Space: O(2^n * n).
def count_valid_results(s):
    def valid(t):
        balance = 0
        for ch in t:
            if ch == "(":
                balance += 1
            elif ch == ")":
                balance -= 1
                if balance < 0:
                    return False
        return balance == 0

    level = {s}
    while True:
        good = {t for t in level if valid(t)}
        if good:
            return len(good)
        level = {t[:i] + t[i + 1:] for t in level for i in range(len(t)) if t[i] in "()"}
```

PART 2 · TOPIC 21 OF 50

Tree Traversal - Level Order

1 problem · 0 Easy · 1 Medium · 0 Hard

THE IDEA

Breadth-first search with a queue, processing one level at a time (len(queue) tells you how many nodes are on the current level).

THE 1 PROBLEMS, EASY TO HARD | ★ = TOP 1: DO THESE FIRST

1	★ **Binary Tree Vertical Order Traversal**	Medium	LC 314	2323

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Maximum Depth of Binary Tree (Part 1, pattern 11, page 545)
Binary Tree Level Order Traversal (Part 1, pattern 12, page 611)
Binary Tree Zigzag Level Order Traversal (Part 1, pattern 12, page 617)
Populating Next Right Pointers in Each Node II (Part 1, pattern 12, page 628)
Check Completeness of a Binary Tree (Part 1, pattern 12, page 666)
Average of Levels in Binary Tree (Part 1, pattern 12, page 596)
Binary Tree Right Side View (Part 1, pattern 12, page 620)
Populating Next Right Pointers in Each Node (Part 1, pattern 12, page 624)
Maximum Width of Binary Tree (Part 1, pattern 12, page 632)

314. Binary Tree Vertical Order Traversal

Medium · Tree, Depth-First Search, Breadth-First Search, Hash Table, Binary Tree, Sorting

leetcode.com/problems/binary-tree-vertical-order-traversal

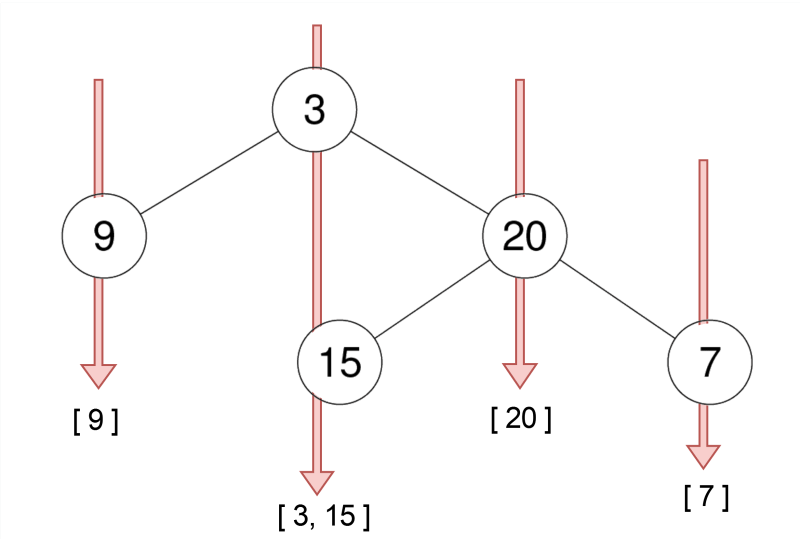
★ **TOP 7 PROBLEM** in Tree Traversal - Level Order: master this one first

PROBLEM

Given the `root` of a binary tree, return ***the vertical order traversal*** *of its nodes' values*. (i.e., from top to bottom, column by column).

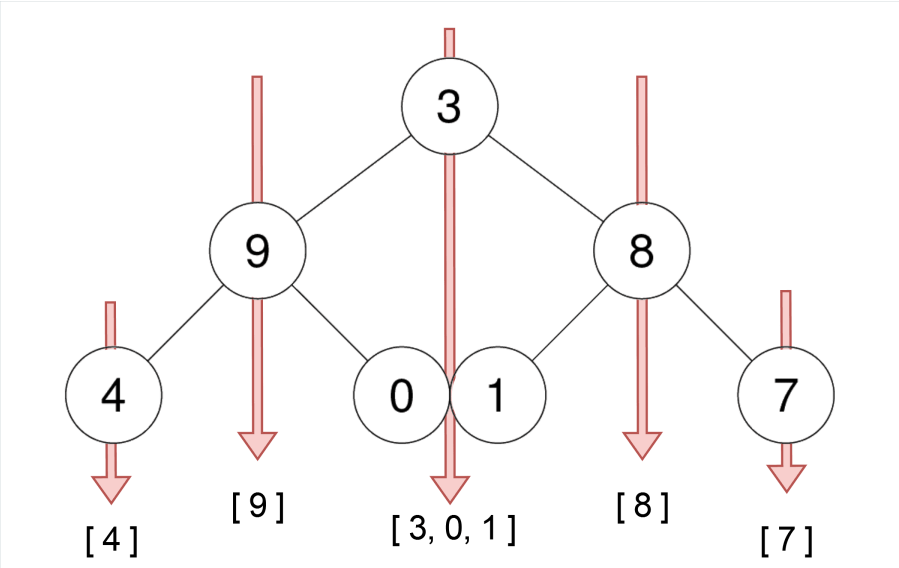
If two nodes are in the same row and column, the order should be from **left to right**.

Example 1:



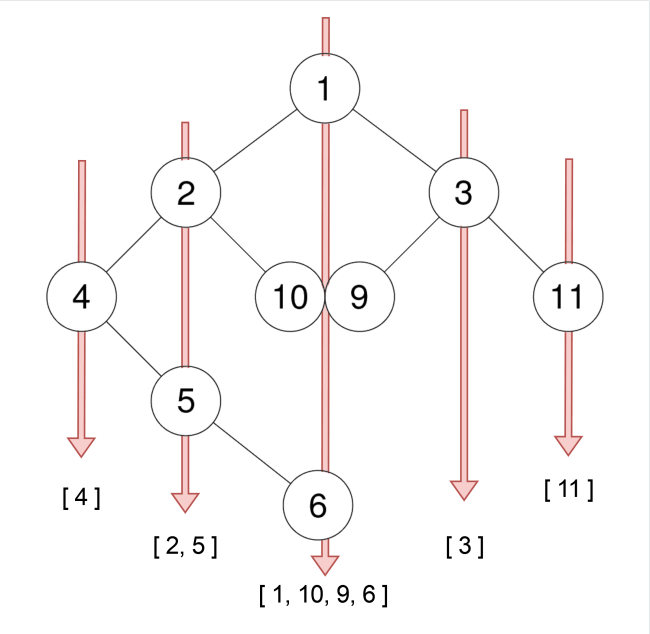

```
Input: root = [3,9,20,null,null,15,7]
Output: [[9],[3,15],[20],[7]]
```

Example 2:




```
Input: root = [3,9,8,4,0,1,7]
Output: [[4],[9],[3,0,1],[8],[7]]
```

Example 3:




```
Input: root = [1,2,3,4,10,9,11,null,5,null,null,null,null,null,null,null,6]
Output: [[4],[2,5],[1,10,9,6],[3],[11]]
```

Constraints:

- The number of nodes in the tree is in the range `[0, 100]`.
- `-100 <= Node.val <= 100`

SOLUTIONS (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> Vertical order lists columns left to right and, within a column, top to bottom. DFS is not level order, so a column may see a deep node first. Record depth and horizontal offset, bucket by column, then sort each bucket by depth. Sorting column keys yields left-to-right order.

DFS traverses the binary tree, recording the value, depth, and horizontal offset of each node. Then sort all nodes by horizontal offset from small to large, then by depth from small to large, and finally group by horizontal offset.

The time complexity is O(nlog log n), and the space complexity is O(n). Where n is the number of nodes in the binary tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def verticalOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        def dfs(root, depth, offset):
            if root is None:
                return
            d[offset].append((depth, root.val))
            dfs(root.left, depth + 1, offset - 1)
            dfs(root.right, depth + 1, offset + 1)

        d = defaultdict(list)
        dfs(root, 0, 0)
        ans = []
        for _, v in sorted(d.items()):
            v.sort(key=lambda x: x[0])
            ans.append([x[1] for x in v])
        return ans
```

Solution 2: BFS

> **How to think about it**
>
> Method 1 still sorts each column by depth. BFS visits shallower nodes first, so a column is already top-down; only column keys need sorting. Enqueue left/right with offsets -1 and +1.

A better approach to this problem should be BFS, traversing from top to bottom level by level.

The time complexity is O(nlog n), and the space complexity is O(n). Where n is the number of nodes in the binary tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def verticalOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        if root is None:
            return []
        q = deque([(root, 0)])
        d = defaultdict(list)
        while q:
            for _ in range(len(q)):
                root, offset = q.popleft()
                d[offset].append(root.val)
                if root.left:
                    q.append((root.left, offset - 1))
                if root.right:
                    q.append((root.right, offset + 1))
        return [v for _, v in sorted(d.items())]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Nodes in the same column and row are sorted by value
# (the version that sorts ties)
# 2. The width of the tree in columns


# ============================================================
# FOLLOW-UP 1: Nodes in the same column and row are sorted
# by value (the version that sorts ties)
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: Collect (column, row, value), sort, and group by
# column.
# Time: O(n log n).  Space: O(n).
def vertical_order_sorted(root):
    cells = []
    stack = [(root, 0, 0)]
    while stack:
        node, row, col = stack.pop()
        if node:
            cells.append((col, row, node.val))
            stack.append((node.left, row + 1, col - 1))
            stack.append((node.right, row + 1, col + 1))
    cells.sort()
    result = []
    last = None
    for col, row, val in cells:
        if col != last:
            result.append([])
            last = col
        result[-1].append(val)
    return result


# ============================================================
# FOLLOW-UP 2: The width of the tree in columns
# ============================================================

# ---- follow-up 2: function ----
# Answer: The largest column minus the smallest column,
# plus one.
# Time: O(n).  Space: O(h).
def width_in_columns(root):
    low = high = 0
    stack = [(root, 0)]
    while stack:
        node, col = stack.pop()
        if node:
            low, high = min(low, col), max(high, col)
            stack.append((node.left, col - 1))
            stack.append((node.right, col + 1))
    return high - low + 1
```

PART 2 · TOPIC 22 OF 50

Tree Traversal - Pre Order

7 problems · 3 Easy · 4 Medium · 0 Hard

THE IDEA

Visit the node first, then the children, passing information down (a path, a running sum, a bound). Good for paths and for building trees.

THE 7 PROBLEMS, EASY TO HARD | ★ = TOP 4: DO THESE FIRST

1	Binary Tree Paths	Easy	LC 257	2328
2	Sum of Left Leaves	Easy	LC 404	2331
3	Subtree of Another Tree	Easy	LC 572	2334
4	★ **Construct Binary Tree from Inorder and Postorder Traversal**	Medium	LC 106	2338
5	★ **Count Complete Tree Nodes**	Medium	LC 222	2341
6	★ **Longest Univalue Path**	Medium	LC 687	2344
7	★ **Maximum Difference Between Node and Ancestor**	Medium	LC 1026	2348

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Binary Tree Preorder Traversal (Part 1, pattern 11, page 535)

Same Tree (Part 1, pattern 11, page 556)

Symmetric Tree (Part 1, pattern 11, page 560)

Convert Sorted Array to Binary Search Tree (Part 1, pattern 14, page 725)

Path Sum II (Part 1, pattern 11, page 565)

Path Sum III (Part 1, pattern 5, page 262)

Construct Binary Tree from Preorder and Inorder Traversal (Part 1, pattern 11, page 571)

Serialize and Deserialize Binary Tree (Part 1, pattern 29, page 1589)

257. Binary Tree Paths

Easy · Tree, Depth-First Search, String, Backtracking, Binary Tree

leetcode.com/problems/binary-tree-paths

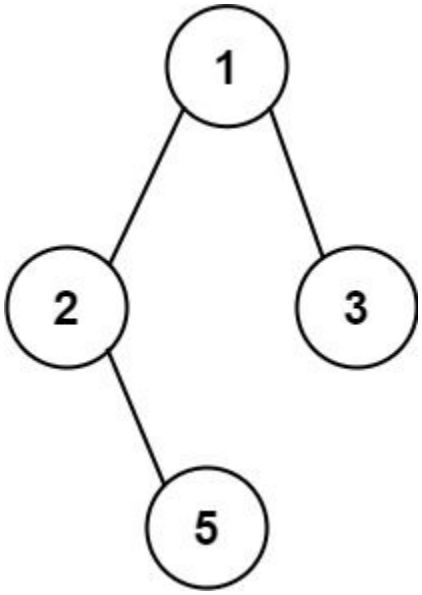
PROBLEM

You are given the `root` of a binary tree.

Return all **root-to-leaf** paths in **any order**.

A **leaf** is a node with no children.

Example 1:




```
Input: root = [1,2,3,null,5]
Output: ["1->2->5","1->3"]
```

Example 2:

```
Input: root = [1]
Output: ["1"]
```

Constraints:

- The number of nodes in the tree is in the range `[1, 100]`.
- `-100 <= Node.val <= 100`

SOLUTION (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> Root-to-leaf paths are recorded while we walk. DFS appends the current value, joins the path at a leaf, and pops on the way back.

We can use depth-first search to traverse the entire binary tree. Each time, we add the current node to the path. If the current node is a leaf node, we add the entire path to the answer. Otherwise, we continue to recursively traverse the child nodes of the node. Finally, when the recursion ends and returns to the current node, we need to remove the current node from the path.

The time complexity is $O(n^2)$, and the space complexity is O(n). Where n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def binaryTreePaths(self, root: Optional[TreeNode]) -> List[str]:
        def dfs(root: Optional[TreeNode]):
            if root is None:
                return
            t.append(str(root.val))
            if root.left is None and root.right is None:
                ans.append("->".join(t))
            else:
                dfs(root.left)
                dfs(root.right)
            t.pop()

        ans = []
        t = []
        dfs(root)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Root-to-leaf paths whose values add up to a target
# 2. The longest root-to-leaf path (as text)


# ============================================================
# FOLLOW-UP 1: Root-to-leaf paths whose values add up to a
# target
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: Walk down subtracting; at a leaf, check what is
# left.
# Time: O(n * h) for building the path texts.  Space: O(h)
# plus the output.
def paths_with_sum(root, target):
    result = []

    def go(node, left, path):
        if node is None:
            return
        path.append(node.val)
        if node.left is None and node.right is None and left == node.val:
            result.append("->".join(map(str, path)))
        go(node.left, left - node.val, path)
        go(node.right, left - node.val, path)
        path.pop()

    go(root, target, [])
    return result
```

```
# ============================================================
# FOLLOW-UP 2: The longest root-to-leaf path (as text)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the deepest path seen at a leaf.
# Time: O(n + h^2).  Space: O(h).
def longest_path_text(root):
    best = []

    def go(node, path):
        if node is None:
            return
        path.append(node.val)
        if node.left is None and node.right is None and len(path) > len(best):
            best[:] = path
        go(node.left, path)
        go(node.right, path)
        path.pop()

    go(root, [])
```

```
    return "->".join(map(str, best))
```

PART 2 · TREE TRAVERSAL - PRE ORDER · PROBLEM 2 OF 7

404. Sum of Left Leaves

Easy · Tree, Depth-First Search, Breadth-First Search, Binary Tree

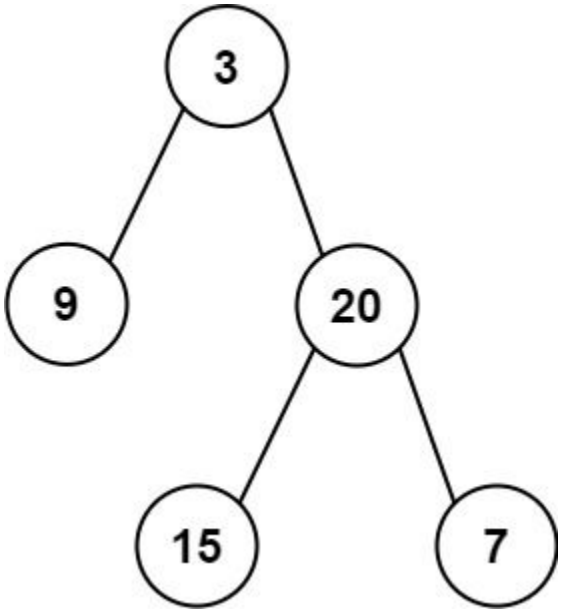
leetcode.com/problems/sum-of-left-leaves

PROBLEM

Given the `root` of a binary tree, return *the sum of all left leaves.*

A **leaf** is a node with no children. A **left leaf** is a leaf that is the left child of another node.

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: 24
Explanation: There are two left leaves in the binary tree, with values 9 and 15 respectively.
```

Example 2:

```
Input: root = [1]
Output: 0
```

Constraints:

- The number of nodes in the tree is in the range `[1, 1000]`.
- `-1000 <= Node.val <= 1000`

SOLUTIONS (PYTHON 3)

Solution 1: Recursion

How to think about it

A left leaf is a left child with no children, not an arbitrary left subtree. Scanning a subtree from every node would revisit nodes. One recursion suffices: always walk the right child; if the left child is a leaf, add its value, otherwise recurse. An empty tree contributes 0. Treating a leaf as both children being null avoids an extra walk.

First, we check if `root` is null. If it is, we return 0.

Otherwise, we recursively call the `sumOfLeftLeaves` function to calculate the sum of all left leaves in `root`'s right subtree, and assign the result to the answer variable ans. Then we check if `root`'s left child exists. If it does, we check if it

is a leaf node. If it is a leaf node, we add its value to the answer variable ans. Otherwise, we recursively call the `sumOfLeftLeaves` function to calculate the sum of all left leaves in `root`'s left subtree, and add the result to the answer variable ans.

Finally, we return the answer.

The time complexity is O(n), and the space complexity is O(n), where n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sumOfLeftLeaves(self, root: Optional[TreeNode]) -> int:
        if root is None:
            return 0
        ans = self.sumOfLeftLeaves(root.right)
        if root.left:
            if root.left.left == root.left.right:
                ans += root.left.val
            else:
                ans += self.sumOfLeftLeaves(root.left)
        return ans
```

Solution 2: Stack

> **How to think about it**
>
> The same classification, implemented with an explicit stack: pop a node, add a left leaf or push the left child, and push a right child when it exists. Asymptotics are unchanged; the call stack is no longer implicit.

We can also convert the recursion in Solution 1 to iteration, using a stack to simulate the recursion process.

Similar to Solution 1, we first check if `root` is null. If it is, we return 0.

Otherwise, we initialize the answer variable ans to 0, and then initialize a stack stk and add `root` to the stack.

While the stack is not empty, we pop the top element `root` from the stack. If `root`'s left child exists, we check if it is a leaf node. If it is a leaf node, we add its value to the answer variable ans. Otherwise, we add its left child to the stack. Then we check if `root`'s right child exists. If it does, we add it to the stack.

Finally, we return the answer.

The time complexity is O(n), and the space complexity is O(n), where n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sumOfLeftLeaves(self, root: Optional[TreeNode]) -> int:
        if root is None:
            return 0
        ans = 0
        stk = [root]
        while stk:
            root = stk.pop()
            if root.left:
                if root.left.left == root.left.right:
                    ans += root.left.val
                else:
                    stk.append(root.left)
            if root.right:
                stk.append(root.right)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Sum of the RIGHT leaves
# 2. Sum of the leaves on the deepest level


# ============================================================
# FOLLOW-UP 1: Sum of the RIGHT leaves
# ============================================================

# ---- follow-up 1: function ----
# Answer: Mirror image: a leaf that is a right child.
# Time: O(n).  Space: O(h).
def sum_of_right_leaves(root):
    def go(node, is_right):
        if node is None:
            return 0
        if node.left is None and node.right is None:
            return node.val if is_right else 0
        return go(node.left, False) + go(node.right, True)

    return go(root, False)


# ============================================================
# FOLLOW-UP 2: Sum of the leaves on the deepest level
# ============================================================

# ---- follow-up 2: function ----
# Answer: Breadth-first search; the last level's leaves.
# Time: O(n).  Space: O(w), w = widest level.
def sum_of_deepest_leaves(root):
    level = [root]
    last = [root]
    while level:
        last = level
        level = [child for node in level for child in (node.left, node.right) if child]
    return sum(node.val for node in last)
```

572. Subtree of Another Tree

Easy · Tree, Depth-First Search, Binary Tree, String Matching, Hash Function

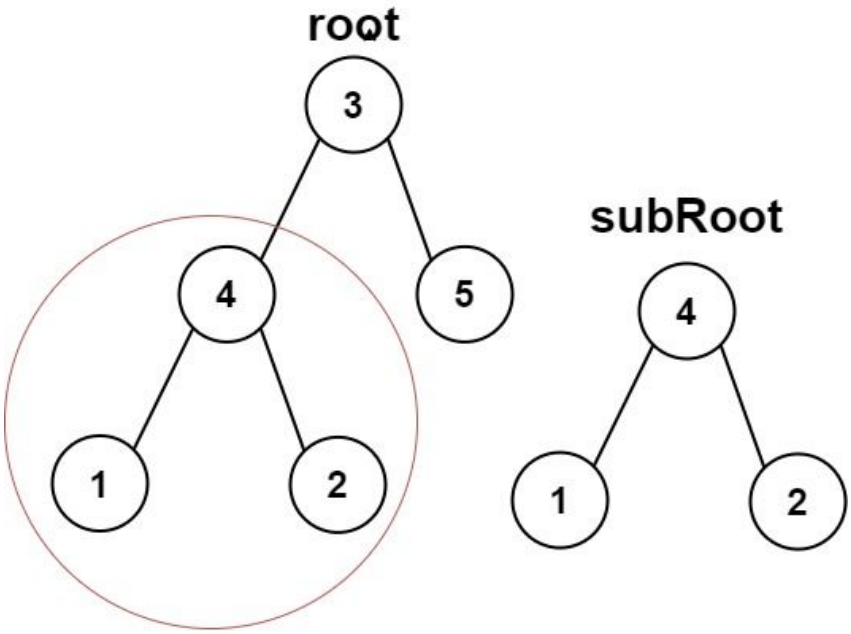
leetcode.com/problems/subtree-of-another-tree

PROBLEM

Given the roots of two binary trees `root` and `subRoot`, return `true` if there is a subtree of `root` with the same structure and node values of `subRoot` and `false` otherwise.

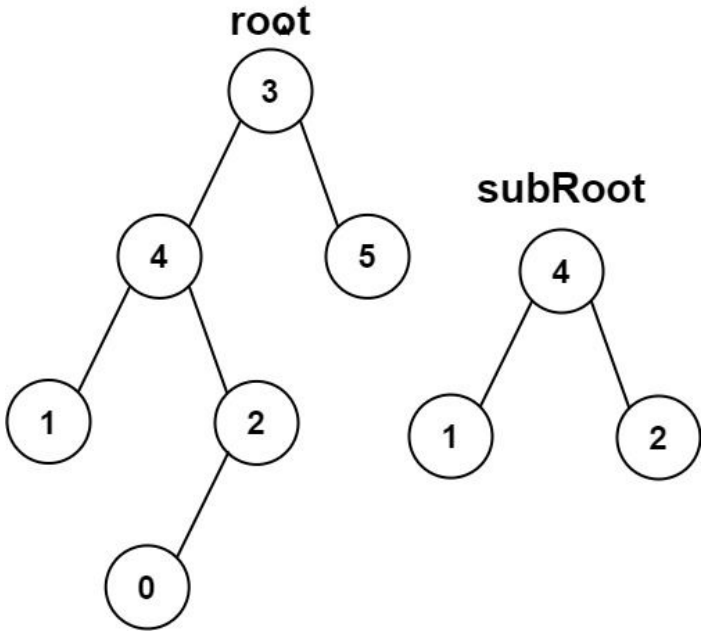
A subtree of a binary tree `tree` is a tree that consists of a node in `tree` and all of this node's descendants. The tree `tree` could also be considered as a subtree of itself.

Example 1:




```
Input: root = [3,4,5,1,2], subRoot = [4,1,2]
Output: true
```

Example 2:




```
Input: root = [3,4,5,1,2,null,null,null,null,0], subRoot = [4,1,2]
Output: false
```

Constraints:

- The number of nodes in the `root` tree is in the range `[1, 2000]`.
- The number of nodes in the `subRoot` tree is in the range `[1, 1000]`.
- -10^4 <= root.val <= 10^4
- -10^4 <= subRoot.val <= 10^4

SOLUTION (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> Ask whether subRoot equals some subtree of root. Comparing at every node is O(mn) in the worst case and fits the limits. `same` walks two trees together. The outer DFS invokes `same` at each node and returns on the first hit. Compare the root first so mismatches exit early.

We define a helper function same(p, q) to determine whether the tree rooted at p and the tree rooted at q are identical. If the root values of the two trees are equal, and their left and right subtrees are also respectively equal, then the two trees are identical.

In the isSubtree(root, subRoot) function, we first check if root is null. If it is, we return false. Otherwise, we check if root and subRoot are identical. If they are, we return true. Otherwise, we recursively check if the left or right subtree of root contains subRoot.

The time complexity is O(n × m), and the space complexity is O(n). Here, n and m are the number of nodes in the trees root and subRoot, respectively.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSubtree(self, root: Optional[TreeNode], subRoot: Optional[TreeNode]) -> bool:
        def same(p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
            if p is None or q is None:
                return p is q
            return p.val == q.val and same(p.left, q.left) and same(p.right, q.right)

        if root is None:
            return False
        return (
            same(root, subRoot)
            or self.isSubtree(root.left, subRoot)
            or self.isSubtree(root.right, subRoot)
        )
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Count how many times the subtree occurs
# 2. Use serialisation: both trees as strings


# ============================================================
# FOLLOW-UP 1: Count how many times the subtree occurs
# ============================================================

# ---- follow-up 1: function ----
# Answer: Compare at every node.
# Time: O(n * m).  Space: O(h).
def count_occurrences(root, sub):
    def same(a, b):
        if a is None or b is None:
            return a is b
        return a.val == b.val and same(a.left, b.left) and same(a.right, b.right)

    def go(node):
        if node is None:
            return 0
        return (1 if same(node, sub) else 0) + go(node.left) + go(node.right)

    return go(root)


# ============================================================
# FOLLOW-UP 2: Use serialisation: both trees as strings
# ============================================================

# ---- follow-up 2: function ----
# Answer: Write each tree in preorder with markers for
# missing children; the subtree's text must appear inside
# the big text (with a separator before every value).
# Time: O(n * h + n * m) with the text search.  Space: O(n
# + m).
def is_subtree_by_text(root, sub):
    def text(node):
        if node is None:
            return "#"
        return "," + str(node.val) + " " + text(node.left) + " " + text(node.right)
```

```
    return text(sub) in text(root)
```

106. Construct Binary Tree from Inorder and Postorder Traversal

Medium · Tree, Array, Hash Table, Divide and Conquer, Binary Tree

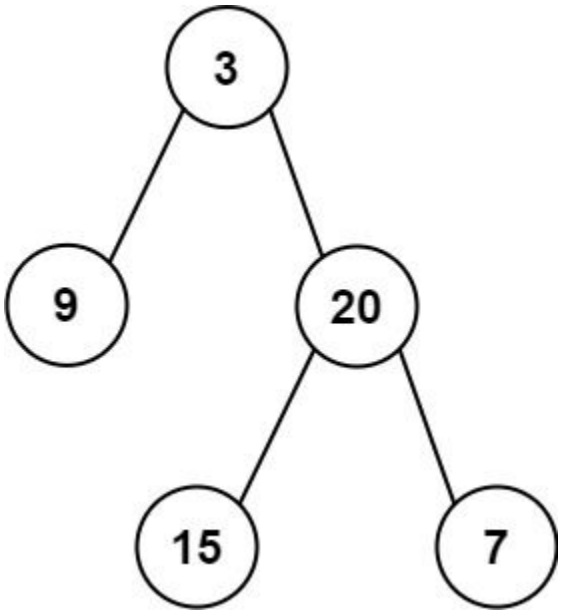
leetcode.com/problems/construct-binary-tree-from-inorder-and-postorder-traversal

★ **TOP 7 PROBLEM** in Tree Traversal - Pre Order: master this one first

PROBLEM

Given two integer arrays `inorder` and `postorder` where `inorder` is the inorder traversal of a binary tree and `postorder` is the postorder traversal of the same tree, construct and return *the binary tree*.

Example 1:




```
Input: inorder = [9,3,15,20,7], postorder = [9,15,7,20,3]
Output: [3,9,20,null,null,15,7]
```

Example 2:

```
Input: inorder = [-1], postorder = [-1]
Output: [-1]
```

Constraints:

- `1 <= inorder.length <= 3000`
- `postorder.length == inorder.length`
- `-3000 <= inorder[i], postorder[i] <= 3000`
- `inorder` and `postorder` consist of **unique** values.
- Each value of `postorder` also appears in `inorder`.
- `inorder` is **guaranteed** to be the inorder traversal of the tree.
- `postorder` is **guaranteed** to be the postorder traversal of the tree.

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Recursion

How to think about it

The last postorder value is the root; inorder again splits left and right. Linear search for the root each time can be $O(n^2)$. Values are unique, so we index inorder first. The last value of the current postorder slice is the root; the count of inorder values to its left is the left-subtree size, which splits the remaining postorder into two recursive ranges.

The last node in the post-order traversal is the root node. We can find the position of the root node in the in-order traversal, and then recursively construct the left and right subtrees.

Specifically, we first use a hash table d to store the position of each node in the in-order traversal. Then we design a recursive function dfs(i, j, n), where i and j represent the starting positions of the in-order and post-order traversals, respectively, and n represents the number of nodes in the subtree. The function logic is as follows:

- If n ≤ 0, it means the subtree is empty, return a null node.
- Otherwise, take out the last node v of the post-order traversal, and then find the position k of v in the in-order traversal using the hash table d. Then the number of nodes in the left subtree is k - i, and the number of nodes in the right subtree is n - k + i - 1.
- Recursively construct the left subtree dfs(i, j, k - i) and the right subtree dfs(k + 1, j + k - i, n - k + i - 1), connect them to the root node, and finally return the root node.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def buildTree(self, inorder: List[int], postorder: List[int]) -> Optional[TreeNode]:
        def dfs(i: int, j: int, n: int) -> Optional[TreeNode]:
            if n <= 0:
                return None
            v = postorder[j + n - 1]
            k = d[v]
            l = dfs(i, j, k - i)
            r = dfs(k + 1, j + k - i, n - k + i - 1)
            return TreeNode(v, l, r)

        d = {v: i for i, v in enumerate(inorder)}
        return dfs(0, 0, len(inorder))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Build the tree from PREORDER and POSTORDER (every
# node has 0 or 2 children)
# 2. Check that a tree has the given inorder and postorder


# ============================================================
# FOLLOW-UP 1: Build the tree from PREORDER and POSTORDER
# (every node has 0 or 2 children)
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
```

```
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: The second item of the preorder is the root of
# the left subtree; find it in the postorder to know the
# left size.
# Time: O(n).  Space: O(n).
def build_from_pre_post(preorder, postorder):
    where = {v: i for i, v in enumerate(postorder)}

    def build(pre_lo, pre_hi, post_lo, post_hi):
        if pre_lo > pre_hi:
            return None
        root = TreeNode(preorder[pre_lo])
        if pre_lo == pre_hi:
            return root
        left_root = preorder[pre_lo + 1]
        size = where[left_root] - post_lo + 1
        root.left = build(pre_lo + 1, pre_lo + size, post_lo, post_lo + size - 1)
        root.right = build(pre_lo + size + 1, pre_hi, post_lo + size, post_hi - 1)
        return root

    return build(0, len(preorder) - 1, 0, len(postorder) - 1)


# ============================================================
# FOLLOW-UP 2: Check that a tree has the given inorder and
# postorder
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk the tree and compare the two lists.
# Time: O(n).  Space: O(n).
def has_traversals(root, inorder, postorder):
    ino, post = [], []

    def go(node):
        if node is None:
            return
        go(node.left)
        ino.append(node.val)
        go(node.right)
        post.append(node.val)
```

```
    go(root)
    return ino == inorder and post == postorder
```

222. Count Complete Tree Nodes

Medium · Bit Manipulation, Tree, Binary Search, Binary Tree

leetcode.com/problems/count-complete-tree-nodes

★ **TOP 7 PROBLEM** in Tree Traversal - Pre Order: master this one first

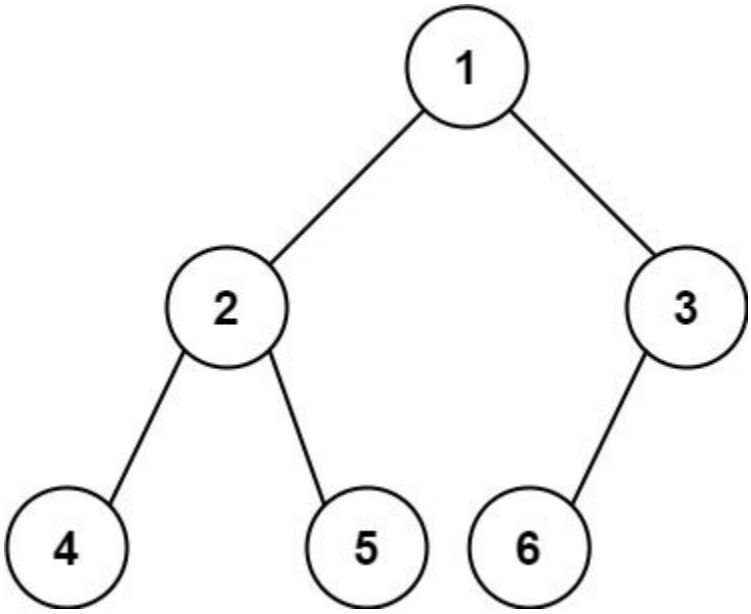
PROBLEM

Given the `root` of a **complete** binary tree, return the number of the nodes in the tree.

According to **Wikipedia**, every level, except possibly the last, is completely filled in a complete binary tree, and all nodes in the last level are as far left as possible. It can have between `1` and 2^h nodes inclusive at the last level `h`.

Design an algorithm that runs in less than `O(n)` time complexity.

Example 1:




```
Input: root = [1,2,3,4,5,6]
Output: 6
```

Example 2:

```
Input: root = []
Output: 0
```

Example 3:

```
Input: root = [1]
Output: 1
```

Constraints:

- The number of nodes in the tree is in the range `[0, 5 * 10^4]`.
- `0 <= Node.val <= 5 * 10^4`
- The tree is guaranteed to be **complete**.

SOLUTIONS (PYTHON 3)

Solution 1: Recursion

> **How to think about it**
>
> A complete tree can still be counted by ordinary recursion: the size is 1 plus the sizes of the two subtrees, visiting every node once.

We recursively traverse the entire tree and count the number of nodes.

The time complexity is O(n), and the space complexity is O(n), where n is the number of nodes in the tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def countNodes(self, root: Optional[TreeNode]) -> int:
        if root is None:
            return 0
        return 1 + self.countNodes(root.left) + self.countNodes(root.right)
```

Solution 2: Binary Search

> **How to think about it**
>
> The linear walk ignores that the last level is packed left. If the left and right heights match, the left subtree is perfect and contributes 2^{left} nodes (including the root), so we recurse only on the right; otherwise the right subtree is perfect and we recurse on the left. Each height walk is O(log n) and the recursion depth is O(log n), for $O(\log^2 n)$ time.

For this problem, we can also take advantage of the characteristics of a complete binary tree to design a faster algorithm.

Characteristics of a complete binary tree: leaf nodes can only appear on the bottom and second-to-bottom layers, and the leaf nodes on the bottom layer are concentrated on the left side of the tree. It should be noted that a full binary tree is definitely a complete binary tree, but a complete binary tree is not necessarily a full binary tree.

If the number of layers in a full binary tree is h, then the total number of nodes is $2^h - 1$.

We first count the heights of the left and right subtrees of root, denoted as left and right.

1. If left = right, it means that the left subtree is a full binary tree, so the total number of nodes in the left subtree is $2^{left} - 1$. Plus the root node, it is 2^{left}. Then we recursively count the right subtree.
1. If left > right, it means that the right subtree is a full binary tree, so the total number of nodes in the right subtree is $2^{right} - 1$. Plus the root node, it is 2^{right}. Then we recursively count the left subtree.

The time complexity is $O(\log^2 n)$.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def countNodes(self, root: Optional[TreeNode]) -> int:
        def depth(root):
            d = 0
            while root:
                d += 1
                root = root.left
            return d

        if root is None:
            return 0
        left, right = depth(root.left), depth(root.right)
        if left == right:
            return (1 << left) + self.countNodes(root.right)
        return (1 << right) + self.countNodes(root.left)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The height of a complete tree in O(log n)
# 2. The path (L / R) from the root to the n-th node of a
# complete tree


# ============================================================
# FOLLOW-UP 1: The height of a complete tree in O(log n)
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: Walk down the left side only.
# Time: O(log n).  Space: O(1).
def height_of_complete_tree(root):
    height = 0
    while root:
        height += 1
        root = root.left
    return height


# ============================================================
# FOLLOW-UP 2: The path (L / R) from the root to the n-th
# node of a complete tree
# ============================================================

# ---- follow-up 2: function ----
# Answer: Write n in binary; after the leading 1, each bit
# is a turn (0 left, 1 right).
# Time: O(log n).  Space: O(log n).
def path_to_nth_node(n):
    bits = bin(n)[3:]
    return "".join("L" if b == "0" else "R" for b in bits)
```

687. Longest Univalue Path

Medium · Tree, Depth-First Search, Binary Tree, Tree DP

leetcode.com/problems/longest-univalue-path

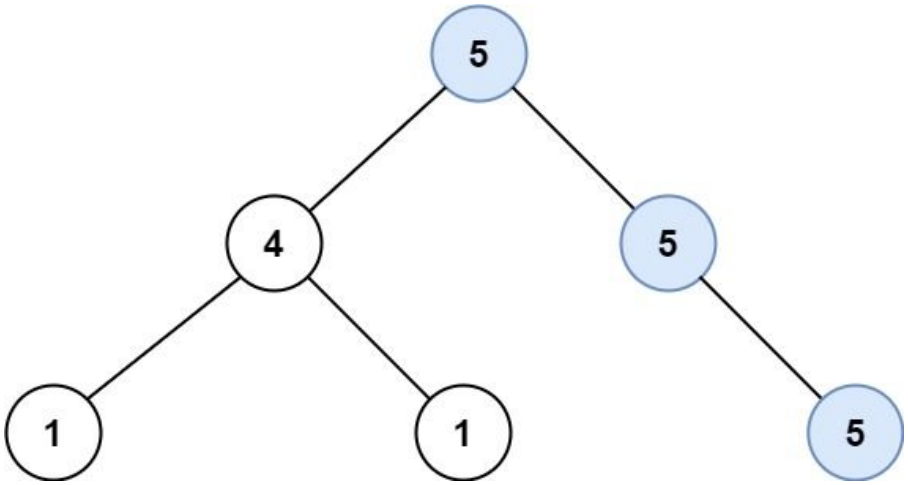
★ **TOP 7 PROBLEM** in Tree Traversal - Pre Order: master this one first

PROBLEM

Given the `root` of a binary tree, return *the length of the longest path, where each node in the path has the same value*. This path may or may not pass through the root.

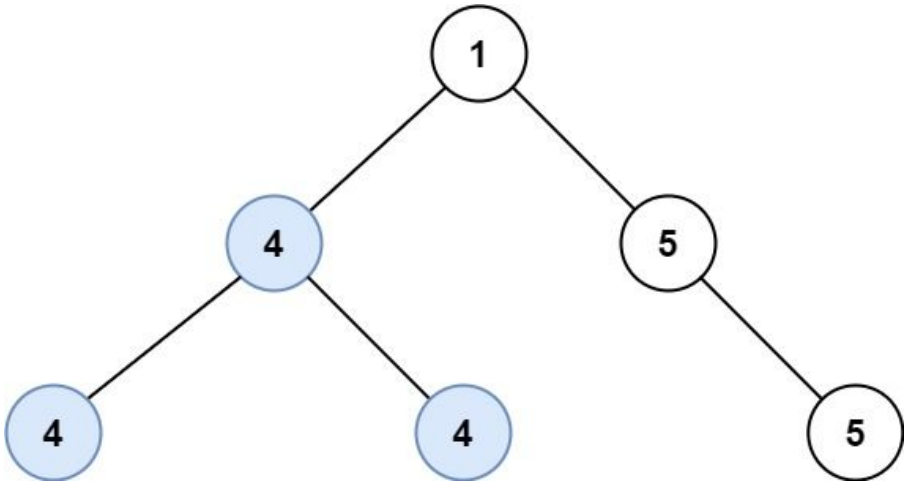
The length of the path between two nodes is represented by the number of edges between them.

Example 1:




```
Input: root = [5,4,5,1,1,null,5]
Output: 2
Explanation: The shown image shows that the longest path of the same value (i.e. 5).
```

Example 2:




```
Input: root = [1,4,5,4,4,null,5]
Output: 2
Explanation: The shown image shows that the longest path of the same value (i.e. 4).
```

Constraints:

- The number of nodes in the tree is in the range `[0, 10^4]`.

- `-1000 <= Node.val <= 1000`
- The depth of the tree will not exceed `1000`.

SOLUTION (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> A univalue path may bend at a node and is counted in edges. Root-to-leaf walks miss a left-plus-right combination. DFS returns the longest same-value downward run: increment only if the child matches. Update the answer with l+r and return max(l,r).

We design a function dfs(root), which represents the longest univalue path length extending downward with the root node as one endpoint of the path.

In dfs(root), we first recursively call dfs(root.left) and dfs(root.right) to get two return values l and r. These two return values represent the longest univalue path lengths extending downward with the left and right children of the root node as one endpoint of the path, respectively.

If the root has a left child and root.val = root.left.val, then the longest univalue path length extending downward with the left child of the root as one endpoint of the path should be l + 1; otherwise, this length is 0. If the root has a right child and root.val = root.right.val, then the longest univalue path length extending downward with the right child of the root as one endpoint of the path should be r + 1; otherwise, this length is 0.

After recursively calling the left and right children, we update the answer to max(ans, l + r), which is the longest univalue path length passing through the root with the root as one endpoint of the path.

Finally, the dfs(root) function returns the longest univalue path length extending downward with the root as one endpoint of the path, which is max(l, r).

In the main function, we call dfs(root) to get the answer.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def longestUnivaluePath(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]) -> int:
            if root is None:
                return 0
            l, r = dfs(root.left), dfs(root.right)
            l = l + 1 if root.left and root.left.val == root.val else 0
            r = r + 1 if root.right and root.right.val == root.val else 0
            nonlocal ans
            ans = max(ans, l + r)
            return max(l, r)

        ans = 0
        dfs(root)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Count the subtrees in which every node has the same
# value
# 2. The longest downward path where each value is exactly
# one more than its parent


# ============================================================
# FOLLOW-UP 1: Count the subtrees in which every node has
# the same value
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: Post-order: a subtree is uniform when both
# children are uniform and match the node.
# Time: O(n).  Space: O(h).
def count_univalue_subtrees(root):
    count = 0

    def go(node):
        nonlocal count
        if node is None:
            return True
        left = go(node.left)
        right = go(node.right)
        if left and right and (node.left is None or node.left.val == node.val) and (node.right is None or
node.right.val == node.val):
            count += 1
            return True
        return False

    go(root)
    return count


# ============================================================
# FOLLOW-UP 2: The longest downward path where each value
# is exactly one more than its parent
# ============================================================

# ---- follow-up 2: function ----
# Answer: Post-order: the best chain starting at each
# node.
# Time: O(n).  Space: O(h).
def longest_consecutive_path(root):
    best = 0

    def go(node):
        nonlocal best
        if node is None:
            return 0
        left, right = go(node.left), go(node.right)
        length = 1
        if node.left and node.left.val == node.val + 1:
            length = max(length, left + 1)
        if node.right and node.right.val == node.val + 1:
            length = max(length, right + 1)
```

```
        best = max(best, length)
        return length

    go(root)
    return best
```

1026. Maximum Difference Between Node and Ancestor

Medium · Tree, Depth-First Search, Binary Tree

leetcode.com/problems/maximum-difference-between-node-and-ancestor

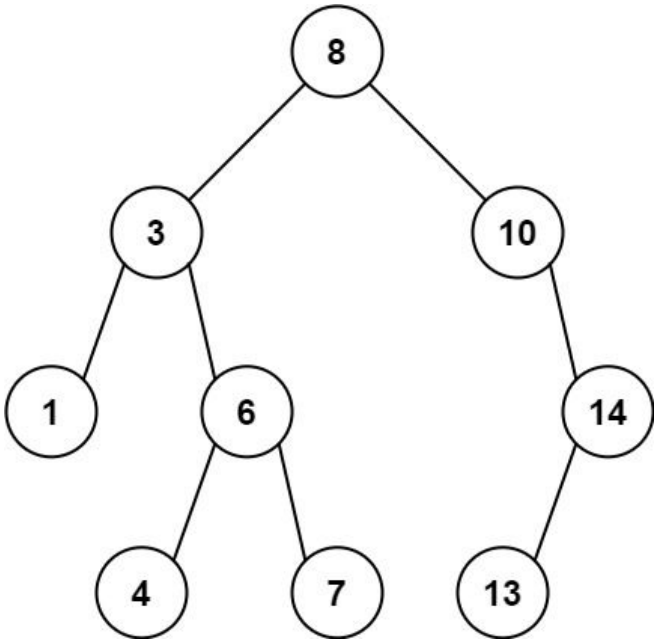
★ **TOP 7 PROBLEM** in Tree Traversal - Pre Order: master this one first

PROBLEM

Given the `root` of a binary tree, find the maximum value `v` for which there exist **different** nodes `a` and `b` where `v = |a.val - b.val|` and `a` is an ancestor of `b`.

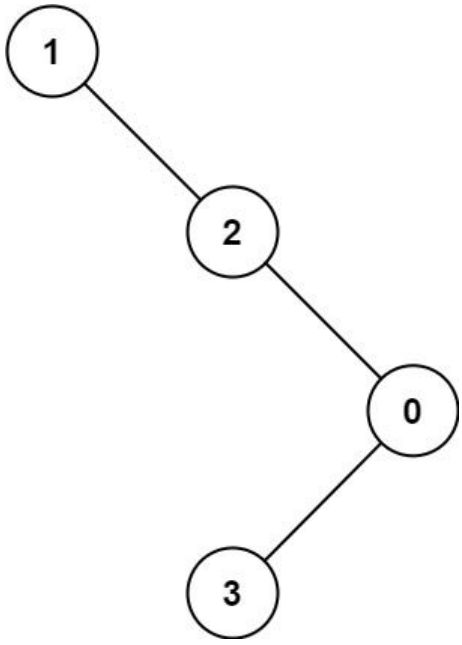
A node `a` is an ancestor of `b` if either: any child of `a` is equal to `b` or any child of `a` is an ancestor of `b`.

Example 1:




```
Input: root = [8,3,10,1,6,null,14,null,null,4,7,13]
Output: 7
Explanation: We have various ancestor-node differences, some of which are given below :
|8 - 3| = 5
|3 - 7| = 4
|8 - 1| = 7
|10 - 13| = 3
Among all possible differences, the maximum value of 7 is obtained by |8 - 1| = 7.
```

Example 2:




```
Input: root = [1,null,2,null,0,3]
Output: 3
```

Constraints:

- The number of nodes in the tree is in the range `[2, 5000]`.
- `0 <= Node.val <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> Collecting every ancestor at each node repeats paths and is heavy for n≤ 5000. Only the minimum and maximum on the path matter. DFS carries (mi,mx) downward, updates the answer with |val-mi| and |val-mx|, then includes the current value in the range. One traversal of the tree is enough.

For each node, to find the maximum difference with its ancestor nodes, we only need to find the difference between the maximum and minimum values of the ancestor nodes. The maximum difference among all nodes and their ancestor nodes is the answer.

Therefore, we design a function dfs(root, mi, mx), where the current node being searched is root, the maximum value of its ancestor nodes is mx, and the minimum value is mi. The function updates the maximum difference ans.

The logic of the function dfs(root, mi, mx) is as follows:

- If root is null, return directly.
- Otherwise, we update ans = max(ans, |mi - root.val|, |mx - root.val|).
- Then update mi = min(mi, root.val), mx = max(mx, root.val), and recursively search the left and right subtrees.

In the main function, we call dfs(root, root.val, root.val), and finally return ans.

The time complexity is O(n), and the space complexity is O(n). Where n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maxAncestorDiff(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode], mi: int, mx: int):
            if root is None:
                return
            nonlocal ans
            ans = max(ans, abs(mi - root.val), abs(mx - root.val))
            mi = min(mi, root.val)
            mx = max(mx, root.val)
            dfs(root.left, mi, mx)
            dfs(root.right, mi, mx)

        ans = 0
        dfs(root, root.val, root.val)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the pair (ancestor, node) with the biggest
# difference
# 2. The SMALLEST difference between a node and one of its
# ancestors


# ============================================================
# FOLLOW-UP 1: Return the pair (ancestor, node) with the
# biggest difference
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: Pass the smallest and biggest ancestor down and
# test every node.
# Time: O(n).  Space: O(h).
def best_ancestor_pair(root):
    best = [-1, None]

    def go(node, low, high):
        if node is None:
            return
        for ancestor in (low, high):
            if abs(ancestor - node.val) > best[0]:
                best[0] = abs(ancestor - node.val)
                best[1] = (ancestor, node.val)
        go(node.left, min(low, node.val), max(high, node.val))
        go(node.right, min(low, node.val), max(high, node.val))

    if root.left:
        go(root.left, root.val, root.val)
    if root.right:
        go(root.right, root.val, root.val)
    return best[1]
```

```
# ============================================================
# FOLLOW-UP 2: The SMALLEST difference between a node and
# one of its ancestors
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the set of ancestor values on the path and
# compare.
# Time: O(n * h).  Space: O(h).
def smallest_ancestor_difference(root):
    best = float("inf")

    def go(node, ancestors):
        nonlocal best
        if node is None:
            return
        for a in ancestors:
            best = min(best, abs(a - node.val))
        ancestors.append(node.val)
        go(node.left, ancestors)
        go(node.right, ancestors)
        ancestors.pop()

    go(root, [])
    return best
```

PART 2 · TOPIC 23 OF 50

Tree Traversal - In Order

1 problem · 1 Easy · 0 Medium · 0 Hard

THE IDEA

Left, node, right. On a binary search tree this visits the values in sorted order.

THE 1 PROBLEMS, EASY TO HARD | ★ = TOP 0: DO THESE FIRST

1	Minimum Distance Between BST Nodes	Easy	LC 783	2353

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

- Binary Tree Inorder Traversal (Part 1, pattern 11, page 530)
- Minimum Absolute Difference in BST (Part 1, pattern 26, page 1400)
- Validate Binary Search Tree (Part 1, pattern 26, page 1419)
- Kth Smallest Element in a BST (Part 1, pattern 26, page 1428)
- Binary Search Tree Iterator (Part 1, pattern 26, page 1451)
- Balance a Binary Search Tree (Part 1, pattern 26, page 1437)

783. Minimum Distance Between BST Nodes

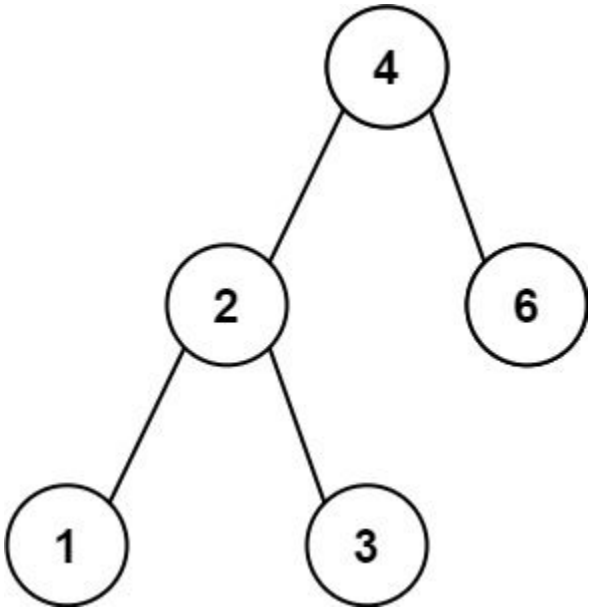
Easy · Tree, Depth-First Search, Breadth-First Search, Binary Search Tree, Binary Tree

leetcode.com/problems/minimum-distance-between-bst-nodes

PROBLEM

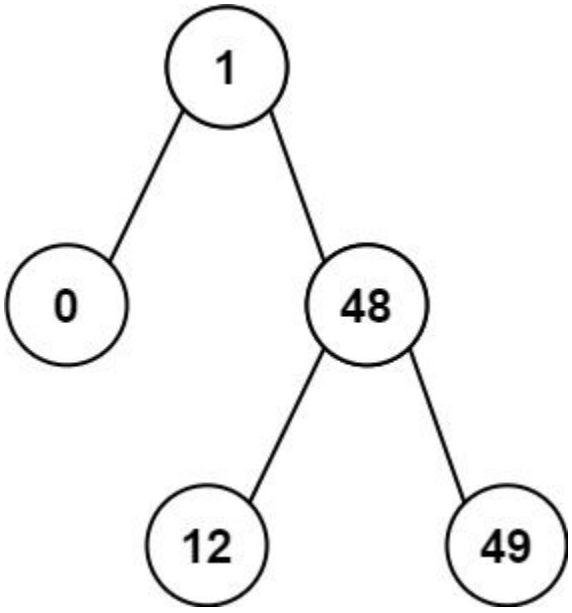
Given the `root` of a Binary Search Tree (BST), return *the minimum difference between the values of any two different nodes in the tree*.

Example 1:




```
Input: root = [4,2,6,1,3]
Output: 1
```

Example 2:




```
Input: root = [1,0,48,null,null,12,49]
Output: 1
```

Constraints:

- The number of nodes in the tree is in the range `[2, 100]`.
- `0 <= Node.val <= 10^5`

Note: This question is the same as 530: https://leetcode.com/problems/minimum-absolute-difference-in-bst/

SOLUTION (PYTHON 3)

Solution 1: Inorder Traversal

> **How to think about it**
>
> Minimum difference between any two BST values. Inorder is sorted, so the minimum lies between consecutive inorder values. Walk inorder, keep pre, and update ans by root.val-pre.

The problem requires us to find the minimum difference between the values of any two nodes. Since the inorder traversal of a binary search tree is an increasing sequence, we only need to find the minimum difference between the values of two adjacent nodes in the inorder traversal.

We can use a recursive method to implement the inorder traversal. During the process, we use a variable pre to save the value of the previous node. This way, we can calculate the minimum difference between the values of two adjacent nodes during the traversal.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the binary search tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def minDiffInBST(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]):
            if root is None:
                return
            dfs(root.left)
            nonlocal pre, ans
            ans = min(ans, root.val - pre)
            pre = root.val
            dfs(root.right)

        pre = -inf
        ans = inf
        dfs(root)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the two values that are closest
# 2. The biggest difference between two nodes


# ============================================================
# FOLLOW-UP 1: Return the two values that are closest
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
```

```
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: In-order walk: remember the previous value and
# the best pair.
# Time: O(n).  Space: O(h).
def closest_pair_in_bst(root):
    best = [float("inf"), None]
    previous = [None]

    def go(node):
        if node is None:
            return
        go(node.left)
        if previous[0] is not None and node.val - previous[0] < best[0]:
            best[0] = node.val - previous[0]
            best[1] = (previous[0], node.val)
        previous[0] = node.val
        go(node.right)

    go(root)
    return best[1]


# ============================================================
# FOLLOW-UP 2: The biggest difference between two nodes
# ============================================================

# ---- follow-up 2: function ----
# Answer: The maximum minus the minimum: walk to the far
# left and the far right.
# Time: O(h).  Space: O(1).
def biggest_difference_in_bst(root):
    low = root
    while low.left:
        low = low.left
    high = root
    while high.right:
        high = high.right
    return high.val - low.val
```

PART 2 · TOPIC 24 OF 50

Tree Traversal - Post-Order

4 problems · 0 Easy · 4 Medium · 0 Hard

THE IDEA

Solve the children first, then combine their results at the node: heights, diameters, and anything that returns data upward.

THE 4 PROBLEMS, EASY TO HARD | ★ = TOP 4: DO THESE FIRST

1	★ **Boundary of Binary Tree**	Medium	LC 545	2357
2	★ **Find Duplicate Subtrees**	Medium	LC 652	2361
3	★ **Distribute Coins in Binary Tree**	Medium	LC 979	2364
4	★ **Delete Nodes And Return Forest**	Medium	LC 1110	2367

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Binary Tree Postorder Traversal (Part 1, pattern 11, page 540)

Balanced Binary Tree (Part 1, pattern 11, page 553)

Count Good Nodes in Binary Tree (Part 1, pattern 11, page 581)

Lowest Common Ancestor of a Binary Tree (Part 1, pattern 11, page 568)

Step-By-Step Directions From a Binary Tree Node to Another (Part 1, pattern 12, page 658)

Invert Binary Tree (Part 1, pattern 11, page 563)

Diameter of Binary Tree (Part 1, pattern 11, page 550)

Sum Root to Leaf Numbers (Part 1, pattern 11, page 578)

Flatten Binary Tree to Linked List (Part 1, pattern 11, page 575)

Binary Tree Maximum Path Sum (Part 1, pattern 11, page 591)

545. Boundary of Binary Tree

Medium · Tree, Depth-First Search, Binary Tree

leetcode.com/problems/boundary-of-binary-tree

★ **TOP 7 PROBLEM** in Tree Traversal - Post-Order: master this one first

PROBLEM

The **boundary** of a binary tree is the concatenation of the **root**, the **left boundary**, the **leaves** ordered from left-to-right, and the **reverse order** of the **right boundary**.

The **left boundary** is the set of nodes defined by the following:

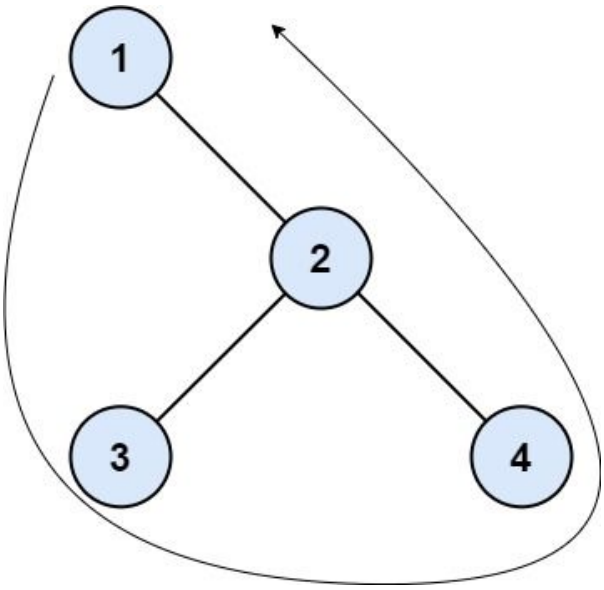
- The root node's left child is in the left boundary. If the root does not have a left child, then the left boundary is **empty**.
- If a node is in the left boundary and has a left child, then the left child is in the left boundary.
- If a node is in the left boundary, has **no** left child, but has a right child, then the right child is in the left boundary.
- The leftmost leaf is **not** in the left boundary.

The **right boundary** is similar to the **left boundary**, except it is the right side of the root's right subtree. Again, the leaf is **not** part of the **right boundary**, and the **right boundary** is empty if the root does not have a right child.

The **leaves** are nodes that do not have any children. For this problem, the root is **not** a leaf.

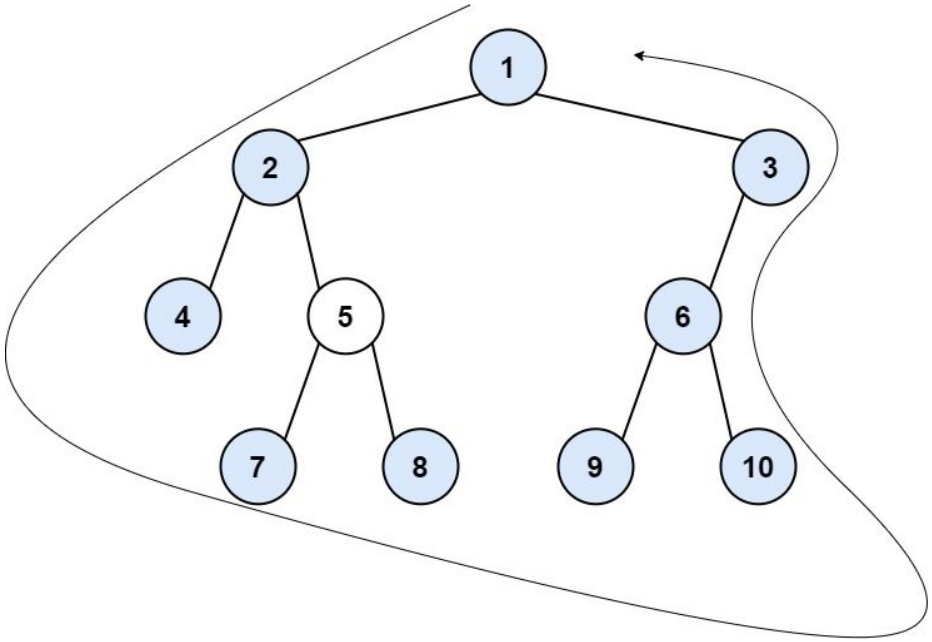
Given the `root` of a binary tree, return *the values of its **boundary***.

Example 1:




```
Input: root = [1,null,2,3,4]
Output: [1,3,4,2]
Explanation:
- The left boundary is empty because the root does not have a left child.
- The right boundary follows the path starting from the root's right child 2 -> 4.
  4 is a leaf, so the right boundary is [2].
- The leaves from left to right are [3,4].
Concatenating everything results in [1] + [] + [3,4] + [2] = [1,3,4,2].
```

Example 2:




```
Input: root = [1,2,3,4,5,6,null,null,null,7,8,9,10]
Output: [1,2,4,7,8,9,10,6,3]
Explanation:
- The left boundary follows the path starting from the root's left child 2 -> 4.
  4 is a leaf, so the left boundary is [2].
- The right boundary follows the path starting from the root's right child 3 -> 6 -> 10.
  10 is a leaf, so the right boundary is [3,6], and in reverse order is [6,3].
- The leaves from left to right are [4,7,8,9,10].
Concatenating everything results in [1] + [2] + [4,7,8,9,10] + [6,3] = [1,2,4,7,8,9,10,6,3].
```

Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.
- $-1000 <= Node.val <= 1000$

SOLUTION (PYTHON 3)

Solution 1: DFS

How to think about it

The boundary is the left side, the leaves, and the right side in counterclockwise order. The root is listed once and leaves must not repeat. One mixed walk struggles with direction and deduplication. Three DFS passes: left edge prefers left (else right) and skips leaves; a second pass collects leaves; the right edge prefers right and is reversed. The root is emitted first if it is not a leaf. A flag i selects the role.

First, if the tree has only one node, we directly return a list with the value of that node.

Otherwise, we can use depth-first search (DFS) to find the left boundary, leaf nodes, and right boundary of the binary tree.

Specifically, we can use a recursive function dfs to find these three parts. In the dfs function, we need to pass in a list nums, a node root, and an integer i, where nums is used to store the current part's node values, and root and i represent the current node and the type of the current part (left boundary, leaf nodes, or right boundary), respectively.

The function implementation is as follows:

- If root is null, then directly return.
- If i = 0, we need to find the left boundary. If root is not a leaf node, we add the value of root to nums. If root has a left child, we recursively call the dfs function, passing in nums, the left child of root, and i. Otherwise, we recursively call the dfs function, passing in nums, the right child of root, and i.

- If i = 1, we need to find the leaf nodes. If root is a leaf node, we add the value of root to nums. Otherwise, we recursively call the dfs function, passing in nums, the left child of root and i, as well as nums, the right child of root and i.
- If i = 2, we need to find the right boundary. If root is not a leaf node, we add the value of root to nums. If root has a right child, we recursively call the dfs function, passing in nums, the right child of root, and i. Otherwise, we recursively call the dfs function, passing in nums, the left child of root, and i.

We call the dfs function separately to find the left boundary, leaf nodes, and right boundary, and then concatenate these three parts to get the answer.

The time complexity is O(n) and the space complexity is O(n), where n is the number of nodes in the binary tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def boundaryOfBinaryTree(self, root: Optional[TreeNode]) -> List[int]:
        def dfs(nums: List[int], root: Optional[TreeNode], i: int):
            if root is None:
                return
            if i == 0:
                if root.left != root.right:
                    nums.append(root.val)
                    if root.left:
                        dfs(nums, root.left, i)
                    else:
                        dfs(nums, root.right, i)
            elif i == 1:
                if root.left == root.right:
                    nums.append(root.val)
                else:
                    dfs(nums, root.left, i)
                    dfs(nums, root.right, i)
            else:
                if root.left != root.right:
                    nums.append(root.val)
                    if root.right:
                        dfs(nums, root.right, i)
                    else:
                        dfs(nums, root.left, i)

        ans = [root.val]
        if root.left == root.right:
            return ans
        left, leaves, right = [], [], []
        dfs(left, root.left, 0)
        dfs(leaves, root, 1)
        dfs(right, root.right, 2)
        ans += left + leaves + right[::-1]
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Only the left boundary (top to bottom, without the
# leaves)
# 2. The leaves from left to right


# ========================================================
# FOLLOW-UP 1: Only the left boundary (top to bottom,
# without the leaves)
```

```
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: Go down, preferring the left child, skipping
# leaves.
# Time: O(h).  Space: O(h).
def left_boundary(root):
    result = []
    node = root.left if root else None
    while node:
        if node.left or node.right:
            result.append(node.val)
        node = node.left if node.left else node.right
    return result


# ============================================================
# FOLLOW-UP 2: The leaves from left to right
# ============================================================

# ---- follow-up 2: function ----
# Answer: Depth-first search, collecting nodes without
# children.
# Time: O(n).  Space: O(h + leaves).
def leaves_left_to_right(root):
    result = []

    def go(node):
        if node is None:
            return
        if node.left is None and node.right is None:
            result.append(node.val)
        go(node.left)
        go(node.right)

    go(root)
    return result
```

652. Find Duplicate Subtrees

Medium · Tree, Depth-First Search, Hash Table, Binary Tree

leetcode.com/problems/find-duplicate-subtrees

★ **TOP 7 PROBLEM** in Tree Traversal - Post-Order: master this one first

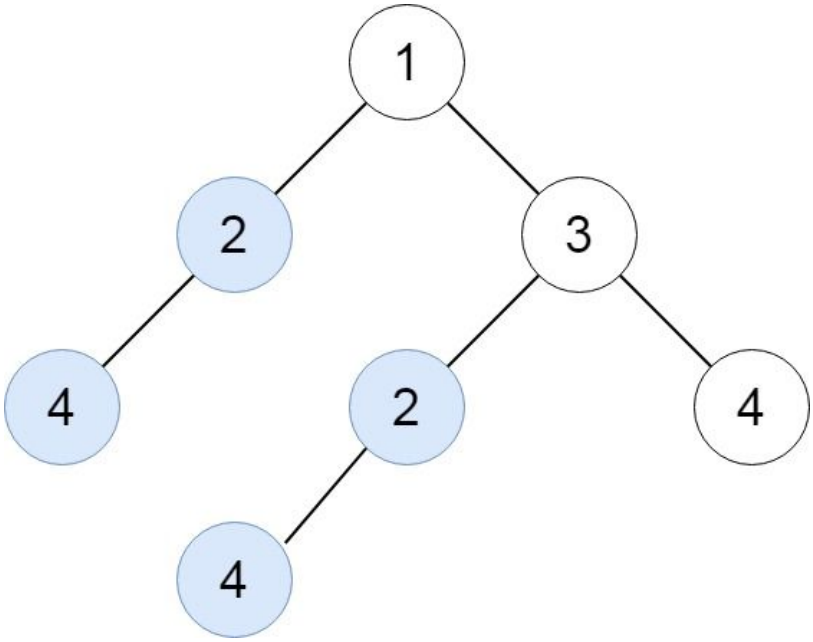
PROBLEM

Given the `root` of a binary tree, return all **duplicate subtrees**.

For each kind of duplicate subtrees, you only need to return the root node of any **one** of them.

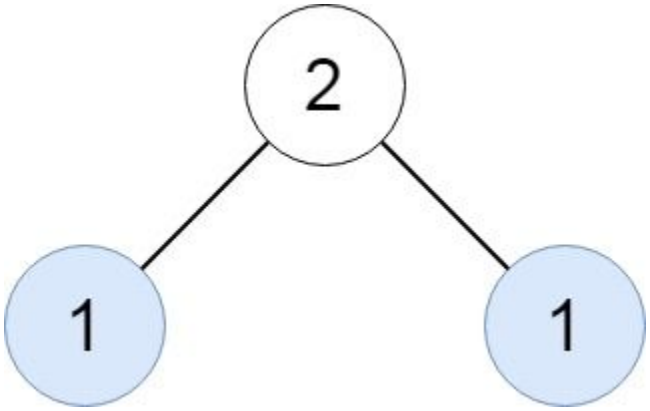
Two trees are **duplicate** if they have the **same structure** with the **same node values**.

Example 1:



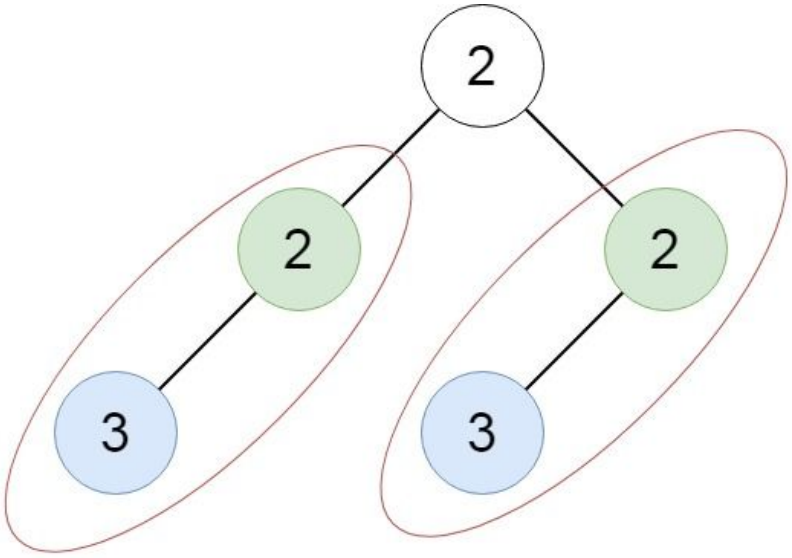

```
Input: root = [1,2,3,4,null,2,4,null,null,4]
Output: [[2,4],[4]]
```

Example 2:




```
Input: root = [2,1,1]
Output: [[1]]
```

Example 3:




```
Input: root = [2,2,2,3,null,3,null]
Output: [[2,3],[3]]
```

Constraints:

- The number of the nodes in the tree will be in the range `[1, 5000]`
- `-200 <= Node.val <= 200`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Duplicate subtrees share structure and values. Pairwise comparison is quadratic. Serialize each subtree in postorder and count the strings. When a serialization first reaches 2, record that root so each shape is reported once.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findDuplicateSubtrees(
        self, root: Optional[TreeNode]
    ) -> List[Optional[TreeNode]]:
        def dfs(root):
            if root is None:
                return '#'
            v = f'{root.val},{dfs(root.left)},{dfs(root.right)}'
            counter[v] += 1
            if counter[v] == 2:
                ans.append(root)
            return v

        ans = []
        counter = Counter()
        dfs(root)
        return ans
```

Complexity. Time: O(n^2) (each node builds a string of size up to n). Space: O(n^2).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. How many distinct subtree shapes are duplicated
# 2. The serialisation strings of the duplicated subtrees


# ============================================================
# FOLLOW-UP 1: How many distinct subtree shapes are
# duplicated
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: Same serialisation and counting; count the texts
# that appear twice or more.
# Time: O(n^2) (the subtree texts are long).  Space:
# O(n^2).
def count_duplicated_shapes(root):
    count = {}

    def text(node):
        if node is None:
            return "#"
        s = str(node.val) + "(" + text(node.left) + "," + text(node.right) + ")"
        count[s] = count.get(s, 0) + 1
        return s

    text(root)
    return sum(1 for c in count.values() if c >= 2)


# ============================================================
# FOLLOW-UP 2: The serialisation strings of the duplicated
# subtrees
# ============================================================

# ---- follow-up 2: function ----
# Answer: Collect the texts seen more than once.
# Time: O(n^2).  Space: O(n^2).
def duplicated_subtree_texts(root):
    count = {}

    def text(node):
        if node is None:
            return "#"
        s = str(node.val) + "(" + text(node.left) + "," + text(node.right) + ")"
        count[s] = count.get(s, 0) + 1
        return s

    text(root)
    return sorted(s for s, c in count.items() if c >= 2)
```

979. Distribute Coins in Binary Tree

Medium · Tree, Depth-First Search, Binary Tree, Tree DP

leetcode.com/problems/distribute-coins-in-binary-tree

★ **TOP 7 PROBLEM** in Tree Traversal - Post-Order: master this one first

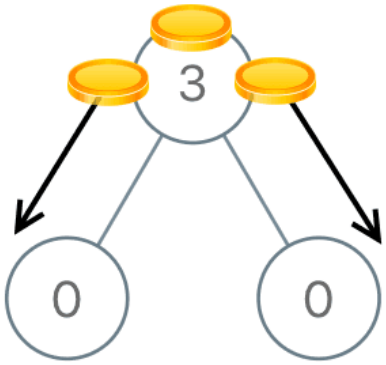
PROBLEM

You are given the `root` of a binary tree with `n` nodes where each `node` in the tree has `node.val` coins. There are `n` coins in total throughout the whole tree.

In one move, we may choose two adjacent nodes and move one coin from one node to another. A move may be from parent to child, or from child to parent.

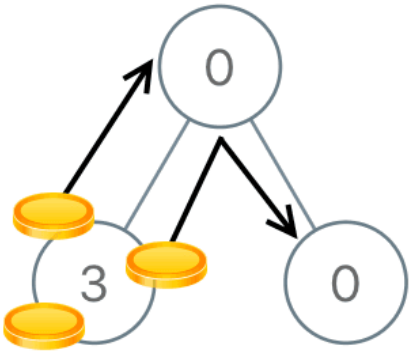
Return *the **minimum** number of moves required to make every node have **exactly** one coin*.

Example 1:




```
Input: root = [3,0,0]
Output: 2
Explanation: From the root of the tree, we move one coin to its left child, and one coin to its right child.
```

Example 2:



```
Input: root = [0,3,0]
Output: 3
Explanation: From the left child of the root, we move two coins to the root [taking two moves]. Then,
we move one coin from the root of the tree to the right child.
```

Constraints:

- The number of nodes in the tree is `n`.
- `1 <= n <= 100`
- `0 <= Node.val <= n`
- The sum of all `Node.val` is `n`.

SOLUTION (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> Moving one coin across an edge counts as one step, until every node has exactly one coin. Routing globally is messy, yet a subtree's surplus must cross the edge to its parent: surplus =coins-nodes, and its absolute value is the traffic on that edge. DFS adds the absolute left and right surpluses and returns the combined surplus.

We define a function dfs(node), which represents the coin overload in the subtree rooted at node, i.e., the number of coins minus the number of nodes. If dfs(node) is positive, it means the subtree has more coins than nodes, and the excess coins need to be moved out of the subtree; if dfs(node) is negative, it means the subtree has fewer coins than nodes, and the shortfall needs to be moved into the subtree.

In the function dfs(node), we first traverse the left and right subtrees to obtain the coin overload left and right of the left and right subtrees, respectively. Then, the current number of moves needs to be increased by |left| + |right|, which means moving the coins from the left and right subtrees to the current node. After that, we return the coin overload of the entire subtree, which is left + right + node.val - 1.

Finally, we return the number of moves.

The time complexity is O(n), and the space complexity is O(h). Here, n and h respectively represent the number of nodes and the height of the binary tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def distributeCoins(self, root: Optional[TreeNode]) -> int:
        def dfs(root):
            if root is None:
                return 0
            left, right = dfs(root.left), dfs(root.right)
            nonlocal ans
            ans += abs(left) + abs(right)
            return left + right + root.val - 1

        ans = 0
        dfs(root)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. How many coins cross the edge above every node
# (positive: going down)
# 2. The number of moves when every node needs k coins
# instead of one


# ============================================================
# FOLLOW-UP 1: How many coins cross the edge above every
# node (positive: going down)
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: Post-order: the surplus of a subtree is the
# number of coins that must leave it.
# Time: O(n).  Space: O(n).
def flow_above_each_node(root):
    flows = []

    def go(node):
        if node is None:
            return 0
        surplus = go(node.left) + go(node.right) + node.val - 1
        flows.append((node.val, surplus))
        return surplus

    go(root)
    return flows


# ============================================================
# FOLLOW-UP 2: The number of moves when every node needs k
# coins instead of one
# ============================================================

# ---- follow-up 2: function ----
# Answer: A subtree with m nodes needs m * k coins; the
# flow is its coins minus that.
# Time: O(n).  Space: O(h).
def moves_for_k_each(root, k):
    moves = 0

    def go(node):
        nonlocal moves
        if node is None:
            return 0, 0
        left_coins, left_size = go(node.left)
        right_coins, right_size = go(node.right)
        coins = left_coins + right_coins + node.val
        size = left_size + right_size + 1
        moves += abs(left_coins - left_size * k) + abs(right_coins - right_size * k)
        return coins, size
```

```
    go(root)
    return moves
```

1110. Delete Nodes And Return Forest

Medium · Tree, Depth-First Search, Array, Hash Table, Binary Tree

leetcode.com/problems/delete-nodes-and-return-forest

★ **TOP 7 PROBLEM** in Tree Traversal - Post-Order: master this one first

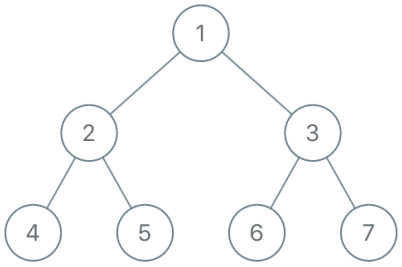
PROBLEM

Given the `root` of a binary tree, each node in the tree has a distinct value.

After deleting all nodes with a value in `to_delete`, we are left with a forest (a disjoint union of trees).

Return the roots of the trees in the remaining forest. You may return the result in any order.

Example 1:




```
Input: root = [1,2,3,4,5,6,7], to_delete = [3,5]
Output: [[1,2,null,4],[6],[7]]
```

Example 2:

```
Input: root = [1,2,4,null,3], to_delete = [3]
Output: [[1,2,4]]
```

Constraints:

- The number of nodes in the given tree is at most `1000`.
- Each node has a distinct value between `1` and `1000`.
- `to_delete.length <= 1000`
- `to_delete` contains distinct values between `1` and `1000`.

SOLUTIONS (PYTHON 3)

Solution 1: DFS

How to think about it

Deleting nodes splits the tree into a forest. A new root is a surviving node whose parent was deleted (or the original root if it survives). A post-order DFS processes both children first, then deletes the current node if required, promoting any surviving children to new roots. A hash set makes the delete-test expected constant time. If the original root survives, it is appended last.

First, we use a hash table or an array of length 1001, `s`, to record all nodes that need to be deleted.

Next, we design a function `dfs(root)` that returns the root of the subtree with `root` as the root after deleting all nodes that need to be deleted. The execution logic of the function `dfs(root)` is as follows:

- If `root` is null, we return null;
- Otherwise, we recursively execute `dfs(root.left)` and `dfs(root.right)`, and assign the return values to `root.left` and `root.right` respectively. If `root` does not need to be deleted, we return `root`; if `root` needs to be

deleted, we check whether `root.left` and `root.right` are null. If they are not null, we add them to the answer array; finally, we return null.

In the main function, we call `dfs(root)`. If the result is not null, it means that the root node does not need to be deleted, and we add the root node to the answer array. Finally, we return the answer array.

The time complexity is O(n), and the space complexity is O(n). Here, n is the number of nodes in the tree.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def delNodes(
        self, root: Optional[TreeNode], to_delete: List[int]
    ) -> List[TreeNode]:
        def dfs(root: Optional[TreeNode]) -> Optional[TreeNode]:
            if root is None:
                return None
            root.left, root.right = dfs(root.left), dfs(root.right)
            if root.val not in s:
                return root
            if root.left:
                ans.append(root.left)
            if root.right:
                ans.append(root.right)
            return None

        s = set(to_delete)
        ans = []
        if dfs(root):
            ans.append(root)
        return ans
```

Solution 2: BFS

> **How to think about it**
>
> Method 1 relies on the recursion stack for post-order deletion. BFS walks top-down: sever a child that must be deleted, and if the current node is deleted, record its still-linked children as new roots. Level order finishes the cuts without a return value; the original root is appended if it survives.

Python 3 (written for this guide: the doocs page has no Python 3 version of this solution)

```python
class Solution:
    def delNodes(self, root: Optional[TreeNode], to_delete: List[int]) -> List[TreeNode]:
        if not root:
            return []
        gone = set(to_delete)
        forest = []
        level = [root]
        while level:
            next_level = []
            for node in level:
                if node.left:
                    next_level.append(node.left)
                    if node.left.val in gone:
                        node.left = None
                if node.right:
                    next_level.append(node.right)
                    if node.right.val in gone:
                        node.right = None
                if node.val in gone:                  # the children of a deleted node become roots
                    if node.left:
                        forest.append(node.left)
                    if node.right:
                        forest.append(node.right)
            level = next_level
        if root.val not in gone:
            forest.append(root)
        return forest
```

Complexity. Time: O(n). Space: O(w), w = widest level.

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. How many trees are in the forest
# 2. The number of nodes in every tree of the forest


# ============================================================
# FOLLOW-UP 1: How many trees are in the forest
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: Count the roots that survive.
# Time: O(n + d).  Space: O(h + d).
def forest_size(root, to_delete):
    gone = set(to_delete)
    count = 0

    def go(node, is_root):
        nonlocal count
        if node is None:
            return None
        deleted = node.val in gone
        if is_root and not deleted:
            count += 1
        node.left = go(node.left, deleted)
        node.right = go(node.right, deleted)
        return None if deleted else node
```

```
    go(root, True)
    return count


# ============================================================
# FOLLOW-UP 2: The number of nodes in every tree of the
# forest
# ============================================================

# ---- follow-up 2: function ----
# Answer: Collect the roots and count nodes below each.
# Time: O(n + d).  Space: O(h + f), f = trees.
def forest_sizes(root, to_delete):
    gone = set(to_delete)
    roots = []

    def go(node, is_root):
        if node is None:
            return None
        deleted = node.val in gone
        if is_root and not deleted:
            roots.append(node)
        node.left = go(node.left, deleted)
        node.right = go(node.right, deleted)
        return None if deleted else node
```

```
    def size(node):
        return 0 if node is None else 1 + size(node.left) + size(node.right)

    go(root, True)
    return sorted(size(r) for r in roots)
```

PART 2 · TOPIC 25 OF 50

BST / Ordered Set

4 problems · 0 Easy · 4 Medium · 0 Hard

THE IDEA

Use the ordering rule of a binary search tree to discard half of the tree at every step, or keep an ordered structure of values.

THE 4 PROBLEMS, EASY TO HARD | ★ = TOP 4: DO THESE FIRST

1	★ **Inorder Successor in BST**	Medium	LC 285	2372
2	★ **Largest BST Subtree**	Medium	LC 333	2375
3	★ **Convert Binary Search Tree to Sorted Doubly Linked List**	Medium	LC 426	2378
4	★ **Inorder Successor in BST II**	Medium	LC 510	2381

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Search in a Binary Search Tree (Part 1, pattern 26, page 1393)

Two Sum IV - Input is a BST (Part 1, pattern 26, page 1407)

Insert into a Binary Search Tree (Part 1, pattern 26, page 1422)

Trim a Binary Search Tree (Part 1, pattern 26, page 1441)

Recover Binary Search Tree (Part 1, pattern 26, page 1445)

My Calendar I (Part 1, pattern 20, page 1102)

Stock Price Fluctuation (Part 1, pattern 29, page 1570)

Closest Binary Search Tree Value (Part 1, pattern 26, page 1416)

Range Sum of BST (Part 1, pattern 26, page 1404)

Delete Node in a BST (Part 1, pattern 26, page 1425)

Lowest Common Ancestor of a Binary Search Tree (Part 1, pattern 26, page 1396)

Construct Binary Search Tree from Preorder Traversal (Part 1, pattern 26, page 1455)

My Calendar II (Part 1, pattern 20, page 1105)

285. Inorder Successor in BST

Medium · Tree, Depth-First Search, Binary Search Tree, Binary Tree

leetcode.com/problems/inorder-successor-in-bst

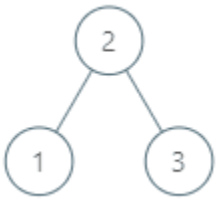
★ **TOP 7 PROBLEM** in BST / Ordered Set: master this one first

PROBLEM

Given the `root` of a binary search tree and a node `p` in it, return *the in-order successor of that node in the BST*. If the given node has no in-order successor in the tree, return `null`.

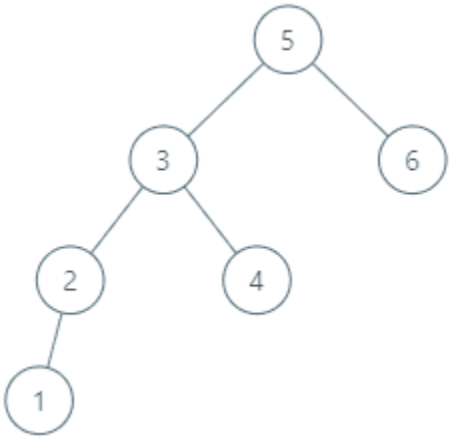
The successor of a node `p` is the node with the smallest key greater than `p.val`.

Example 1:




```
Input: root = [2,1,3], p = 1
Output: 2
Explanation: 1's in-order successor node is 2. Note that both p and the return value is of TreeNode type.
```

Example 2:




```
Input: root = [5,3,6,2,4,null,null,1], p = 6
Output: null
Explanation: There is no in-order successor of the current node, so the answer is null.
```

Constraints:

- The number of nodes in the tree is in the range `[1, 10^4]`.
- `-10^5 <= Node.val <= 10^5`
- All Nodes will have unique values.

SOLUTION (PYTHON 3)

Solution 1: Binary Search

> **How to think about it**
>
> The inorder successor is the smallest key greater than p. On a BST we walk as in binary search: if the current key is larger it is a candidate and we go left; otherwise we go right. The last candidate on that path is the successor, in time proportional to the height.

The in-order traversal of a binary search tree is an ascending sequence, so we can use the binary search method.

The in-order successor node of a binary search tree node p satisfies:

1. The value of the in-order successor node is greater than the value of node p.
2. The in-order successor is the node with the smallest value among all nodes greater than p.

Therefore, for the current node root, if root.val > p.val, then root could be the in-order successor of p. We record root as ans and then search the left subtree, i.e., root = root.left. If root.val ≤ p.val, then root cannot be the in-order successor of p, and we search the right subtree, i.e., root = root.right.

The time complexity is O(h), where h is the height of the binary search tree. The space complexity is O(1).

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None


class Solution:
    def inorderSuccessor(self, root: TreeNode, p: TreeNode) -> Optional[TreeNode]:
        ans = None
        while root:
            if root.val > p.val:
                ans = root
                root = root.left
            else:
                root = root.right
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The inorder PREDECESSOR
# 2. The k-th successor of a node


# ============================================================
# FOLLOW-UP 1: The inorder PREDECESSOR
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: Mirror image: the last node on the way down
# whose value is smaller.
# Time: O(h).  Space: O(1).
```

```python
def inorder_predecessor(root, p):
    candidate = None
    while root:
        if root.val < p.val:
            candidate = root
            root = root.right
        else:
            root = root.left
    return candidate


# ============================================================
# FOLLOW-UP 2: The k-th successor of a node
# ============================================================

# ---- follow-up 2: function ----
# Answer: Repeat the successor search k times.
# Time: O(k * h).  Space: O(1).
def kth_successor(root, p, k):
    def successor(node_value):
        candidate = None
        node = root
        while node:
            if node.val > node_value:
                candidate = node
                node = node.left
            else:
                node = node.right
        return candidate

    current = p
    for _ in range(k):
        current = successor(current.val)
        if current is None:
            return None
    return current
```

333. Largest BST Subtree

Medium · Tree, Depth-First Search, Binary Search Tree, Dynamic Programming, Binary Tree, Tree DP

leetcode.com/problems/largest-bst-subtree

★ **TOP 7 PROBLEM** in BST / Ordered Set: master this one first

PROBLEM

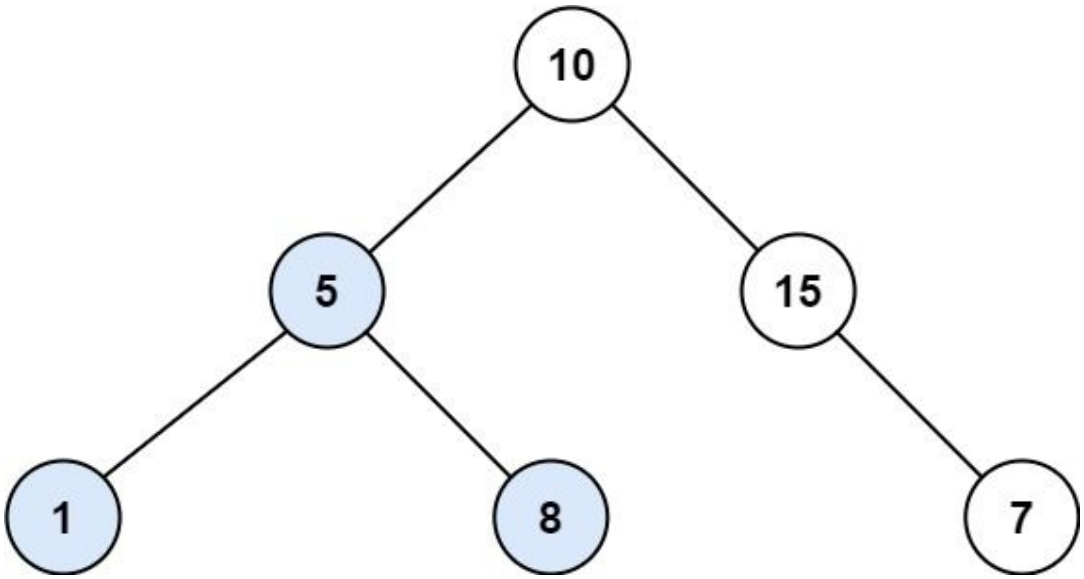
Given the root of a binary tree, find the largest subtree, which is also a Binary Search Tree (BST), where the largest means subtree has the largest number of nodes.

A **Binary Search Tree (BST)** is a tree in which all the nodes follow the below-mentioned properties:

- The left subtree values are less than the value of their parent (root) node's value.
- The right subtree values are greater than the value of their parent (root) node's value.

Note: A subtree must include all of its descendants.

Example 1:




```
Input: root = [10,5,15,1,8,null,7]
Output: 3
Explanation: The Largest BST Subtree in this case is the highlighted one. The return value is the
subtree's size, which is 3.
```

Example 2:

```
Input: root = [4,2,7,2,3,5,null,2,null,null,null,null,null,1]
Output: 2
```

Constraints:

- The number of nodes in the tree is in the range `[0, 10^4]`.
- `-10^4 <= Node.val <= 10^4`

Follow up: Can you figure out ways to solve it with `O(n)` time complexity?

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Find the size of the largest BST subtree. Validating a BST at every node repeats work. A subtree is a BST iff both sides are BSTs and the root sits between their extrema. Postorder returns (min,max,size). If left-max < root < right-min, merge sizes and update the answer; otherwise return sentinels (-∞,∞,0) so ancestors cannot absorb it. An empty tree returns (∞,-∞,0) so a leaf succeeds.

Python 3

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def largestBSTSubtree(self, root: Optional[TreeNode]) -> int:
        def dfs(root):
            if root is None:
                return inf, -inf, 0
            lmi, lmx, ln = dfs(root.left)
            rmi, rmx, rn = dfs(root.right)
            nonlocal ans
            if lmx < root.val < rmi:
                ans = max(ans, ln + rn + 1)
                return min(lmi, root.val), max(rmx, root.val), ln + rn + 1
            return -inf, inf, 0

        ans = 0
        dfs(root)
        return ans
```

Complexity. Time: O(n). Space: O(h).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the root of the largest BST subtree
# 2. How many subtrees are binary search trees


# ============================================================
# FOLLOW-UP 1: Return the root of the largest BST subtree
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: Same post-order summary, remembering the root of
# the best one.
# Time: O(n).  Space: O(h).
def largest_bst_root(root):
    best = [0, None]

    def go(node):
        # returns (is_bst, size, smallest, biggest)
        if node is None:
```

```
            return True, 0, float("inf"), float("-inf")
        lb, ls, lmin, lmax = go(node.left)
        rb, rs, rmin, rmax = go(node.right)
        if lb and rb and lmax < node.val < rmin:
            size = ls + rs + 1
            if size > best[0]:
                best[0], best[1] = size, node
            return True, size, min(lmin, node.val), max(rmax, node.val)
        return False, 0, 0, 0

    go(root)
    return best[1]


# ========================================================
# FOLLOW-UP 2: How many subtrees are binary search trees
# ========================================================

# ---- follow-up 2: function ----
# Answer: Count every node whose summary says BST.
# Time: O(n).  Space: O(h).
def count_bst_subtrees(root):
    count = 0

    def go(node):
        nonlocal count
        if node is None:
            return True, float("inf"), float("-inf")
        lb, lmin, lmax = go(node.left)
        rb, rmin, rmax = go(node.right)
        if lb and rb and lmax < node.val < rmin:
            count += 1
            return True, min(lmin, node.val), max(rmax, node.val)
        return False, 0, 0
```

```
    go(root)
    return count
```

426. Convert Binary Search Tree to Sorted Doubly Linked List

Medium · Stack, Tree, Depth-First Search, Binary Search Tree, Linked List, Binary Tree, Doubly-Linked List

leetcode.com/problems/convert-binary-search-tree-to-sorted-doubly-linked-list

★ **TOP 7 PROBLEM** in BST / Ordered Set: master this one first

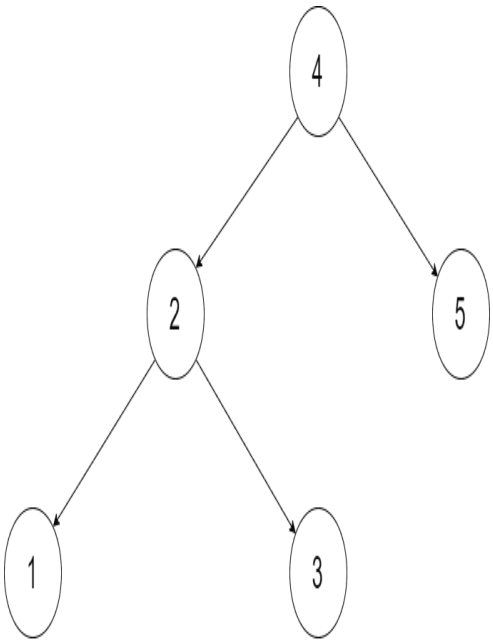
PROBLEM

Convert a **Binary Search Tree** to a sorted **Circular Doubly-Linked List** in place.

You can think of the left and right pointers as synonymous to the predecessor and successor pointers in a doubly-linked list. For a circular doubly linked list, the predecessor of the first element is the last element, and the successor of the last element is the first element.

We want to do the transformation **in place**. After the transformation, the left pointer of the tree node should point to its predecessor, and the right pointer should point to its successor. You should return the pointer to the smallest element of the linked list.

Example 1:




```
Input: root = [4,2,5,1,3]


Output: [1,2,3,4,5]

Explanation: The figure below shows the transformed BST. The solid line indicates the successor
relationship, while the dashed line means the predecessor relationship.
```

Example 2:

```
Input: root = [2,1,3]
Output: [1,2,3]
```

Constraints:

- The number of nodes in the tree is in the range `[0, 2000]`.
- `-1000 <= Node.val <= 1000`
- All the values of the tree are **unique**.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

In-order traversal of a BST is sorted. Collecting nodes into an array then wiring them uses extra memory. The problem asks for an in-place circular list, reusing left/right as pred/succ. During in-order, link prev with the current node and remember the first node as head. Afterwards connect head with the last node to close the circle. Left–root–right makes the link order match the sorted sequence; a null prev identifies the head.

Python 3

```
"""
# Definition for a Node.
class Node:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
"""


class Solution:
    def treeToDoublyList(self, root: 'Optional[Node]') -> 'Optional[Node]':
        def dfs(root):
            if root is None:
                return
            nonlocal prev, head
            dfs(root.left)
            if prev:
                prev.right = root
                root.left = prev
            else:
                head = root
            prev = root
            dfs(root.right)

        if root is None:
            return None
        head = prev = None
        dfs(root)
        prev.right = head
        head.left = prev
        return head
```

Complexity. Time: O(n). Space: O(h).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Make the list NOT circular (the ends stay empty)
# 2. The reverse: a sorted doubly linked list back into a
# balanced BST


# ============================================================
# FOLLOW-UP 1: Make the list NOT circular (the ends stay
# empty)
# ============================================================

# ---- follow-up 1: helper functions ----
class DNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
```

```
        self.right = right

# ---- follow-up 1: function ----
# Answer: Same in-order walk, without joining first and
# last.
# Time: O(n).  Space: O(h).
def bst_to_linear_list(root):
    first = last = None

    def go(node):
        nonlocal first, last
        if node is None:
            return
        go(node.left)
        if last:
            last.right = node
            node.left = last
        else:
            first = node
        last = node
        go(node.right)

    go(root)
    if last:
        last.right = None
    if first:
        first.left = None
    return first


# ============================================================
# FOLLOW-UP 2: The reverse: a sorted doubly linked list
# back into a balanced BST
# ============================================================

# ---- follow-up 2: function ----
# Answer: Collect the nodes into an array, pick the middle
# as the root, and recurse.
# Time: O(n).  Space: O(n).
def list_to_balanced_bst(head):
    nodes = []
    node = head
    while node:
        nodes.append(node)
```

```
        node = node.right

    def build(lo, hi):
        if lo > hi:
            return None
        middle = (lo + hi) // 2
        root = nodes[middle]
        root.left = build(lo, middle - 1)
        root.right = build(middle + 1, hi)
        return root

    return build(0, len(nodes) - 1)
```

510. Inorder Successor in BST II

Medium · Tree, Binary Search Tree, Binary Tree

leetcode.com/problems/inorder-successor-in-bst-ii

★ **TOP 7 PROBLEM** in BST / Ordered Set: master this one first

PROBLEM

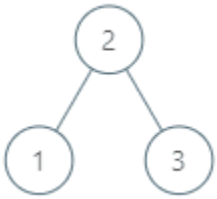
Given a `node` in a binary search tree, return *the in-order successor of that node in the BST*. If that node has no in-order successor, return `null`.

The successor of a `node` is the node with the smallest key greater than `node.val`.

You will have direct access to the node but not to the root of the tree. Each node will have a reference to its parent node. Below is the definition for `Node`:

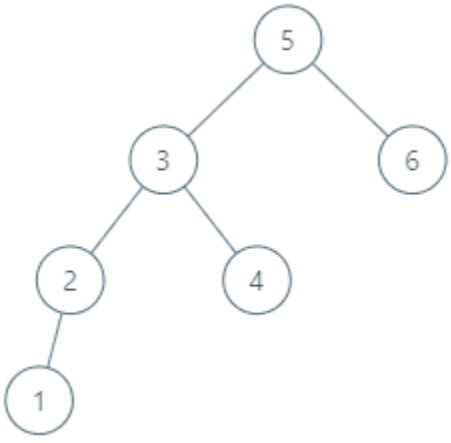
```
class Node {
    public int val;
    public Node left;
    public Node right;
    public Node parent;
}
```

Example 1:




```
Input: tree = [2,1,3], node = 1
Output: 2
Explanation: 1's in-order successor node is 2. Note that both the node and the return value is of Node type.
```

Example 2:




```
Input: tree = [5,3,6,2,4,null,null,1], node = 6
Output: null
Explanation: There is no in-order successor of the current node, so the answer is null.
```

Constraints:

- The number of nodes in the tree is in the range `[1, 10^4]`.
- `-10^5 <= Node.val <= 10^5`
- All Nodes will have unique values.

Follow up: Could you solve it without looking up any of the node's values?

SOLUTION (PYTHON 3)

Solution 1: Case Discussion

> **How to think about it**
>
> The usual inorder successor walks from the root. Here we only have a parent pointer, not the tree root. There are two cases: if a right child exists, the successor is the leftmost node of that subtree; otherwise climb parents until the node is a left child, and that parent is the successor. Both walks follow a single chain.

If the `node` has a right subtree, then the in-order successor of `node` is the leftmost node in the right subtree.

If the `node` does not have a right subtree, then if `node` is the right child of its parent, we continue to search upwards until the parent of the node is null, or the node is the left child of its parent. In this case, the parent node is the in-order successor.

The time complexity is O(h), where h is the height of the binary tree. The space complexity is O(1).

Python 3

```
"""
# Definition for a Node.
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
        self.parent = None
"""

class Solution:
    def inorderSuccessor(self, node: 'Node') -> 'Optional[Node]':
        if node.right:
            node = node.right
            while node.left:
                node = node.left
            return node
        while node.parent and node.parent.right is node:
            node = node.parent
        return node.parent
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The inorder predecessor of a node that has a parent
# pointer
# 2. The k-th successor, using parent pointers only


# ========================================================
# FOLLOW-UP 1: The inorder predecessor of a node that has
# a parent pointer
# ========================================================

# ---- follow-up 1: function ----
# Answer: The rightmost node of the left subtree, or the
# first ancestor of which we are in the right subtree.
```

```
# Time: O(h).  Space: O(1).
def predecessor_with_parent(node):
    if node.left:
        node = node.left
        while node.right:
            node = node.right
        return node
    while node.parent and node is node.parent.left:
        node = node.parent
    return node.parent


# ============================================================
# FOLLOW-UP 2: The k-th successor, using parent pointers
# only
# ============================================================

# ---- follow-up 2: function ----
# Answer: Apply the successor step k times.
# Time: O(k * h).  Space: O(1).
def successor_with_parent(node):
    if node.right:
        node = node.right
        while node.left:
            node = node.left
        return node
    while node.parent and node is node.parent.right:
        node = node.parent
    return node.parent


def kth_successor_with_parent(node, k):
    for _ in range(k):
        node = successor_with_parent(node)
        if node is None:
            return None
    return node
```

PART 2 · TOPIC 26 OF 50

Tries

4 problems · 0 Easy · 3 Medium · 1 Hard

THE IDEA

A tree of characters: each path from the root spells a prefix. Ideal for prefix queries, word dictionaries, and XOR over bits.

THE 4 PROBLEMS, EASY TO HARD | ★ = TOP 4: DO THESE FIRST

1	★ **Remove Sub-Folders from the Filesystem**	Medium	LC 1233	2385
2	★ **Words Within Two Edits of Dictionary**	Medium	LC 2452	2389
3	★ **Find the Length of the Longest Common Prefix**	Medium	LC 3043	2391
4	★ **Word Squares**	Hard	LC 425	2394

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Implement Trie (Prefix Tree) (Part 1, pattern 29, page 1548)

Search Suggestions System (Part 1, pattern 32, page 1739)

Replace Words (Part 1, pattern 32, page 1733)

Word Search II (Part 1, pattern 32, page 1745)

Concatenated Words (Part 1, pattern 7, page 384)

Design Add and Search Words Data Structure (Part 1, pattern 29, page 1567)

Longest Word in Dictionary (Part 1, pattern 32, page 1726)

Maximum XOR of Two Numbers in an Array (Part 1, pattern 27, page 1487)

Stream of Characters (Part 1, pattern 7, page 376)

Palindrome Pairs (Part 1, pattern 7, page 380)

PART 2 · TRIES · PROBLEM 1 OF 4

1233. Remove Sub-Folders from the Filesystem

Medium · Depth-First Search, Trie, Array, String

leetcode.com/problems/remove-sub-folders-from-the-filesystem

★ **TOP 7 PROBLEM** in Tries: master this one first

PROBLEM

Given a list of folders `folder`, return *the folders after removing all **sub-folders** in those folders*. You may return the answer in **any order**.

If a `folder[i]` is located within another `folder[j]`, it is called a **sub-folder** of it. A sub-folder of `folder[j]` must start with `folder[j]`, followed by a `"/"`. For example, `"/a/b"` is a sub-folder of `"/a"`, but `"/b"` is not a sub-folder of `"/a/b/c"`.

The format of a path is one or more concatenated strings of the form: `'/'` followed by one or more lowercase English letters.

- For example, `"/leetcode"` and `"/leetcode/problems"` are valid paths while an empty string and `"/"` are not.

Example 1:

```
Input: folder = ["/a","/a/b","/c/d","/c/d/e","/c/f"]
Output: ["/a","/c/d","/c/f"]
Explanation: Folders "/a/b" is a subfolder of "/a" and "/c/d/e" is inside of folder "/c/d" in our
filesystem.
```

Example 2:

```
Input: folder = ["/a","/a/b/c","/a/b/d"]
Output: ["/a"]
Explanation: Folders "/a/b/c" and "/a/b/d" will be removed because they are subfolders of "/a".
```

Example 3:

```
Input: folder = ["/a/b/c","/a/b/ca","/a/b/d"]
Output: ["/a/b/c","/a/b/ca","/a/b/d"]
```

Constraints:

- `1 <= folder.length <= 4 * 10^4`
- `2 <= folder[i].length <= 100`
- `folder[i]` contains only lowercase letters and `'/'`.
- `folder[i]` always starts with the character `'/'`.
- Each folder name is **unique**.

SOLUTIONS (PYTHON 3)

Solution 1: Sorting

How to think about it

A subfolder is lexicographically after its parent and starts with the parent path plus /. n reaches 4× 10^4, so pairwise prefix tests are too slow. After sorting we only compare with the last kept folder: a descendant is dropped, otherwise the path is a new root. Sorting turns parent-child into an adjacent-prefix relation, so one scan suffices.

First, we sort the array `folder` in lexicographical order, then traverse the array. For the current folder f we are traversing, if its length is greater than or equal to the length of the last folder in the answer array, and its prefix includes the last folder in the answer array plus a `/`, then f is a subfolder of the last folder in the answer array, and we don't need to add it to the

answer array. Otherwise, we add f to the answer array.

After the traversal ends, the folders in the answer array are the answer required by the problem.

The time complexity is O(n × log n × m), and the space complexity is O(m). Where n and m are the length of the array `folder` and the maximum length of the strings in the array `folder`, respectively.

Python 3

```python
class Solution:
    def removeSubfolders(self, folder: List[str]) -> List[str]:
        folder.sort()
        ans = [folder[0]]
        for f in folder[1:]:
            m, n = len(ans[-1]), len(f)
            if m >= n or not (ans[-1] == f[:m] and f[m] == '/'):
                ans.append(f)
        return ans
```

Solution 2: Trie

> **How to think about it**
>
> Sorting relies on global lexicographic order and pays string-prefix comparisons. A trie inserts path segments and marks folder ends; a search that hits an end can skip the whole subtree as nested folders. Time follows total path length and avoids sorting.

We can use a trie to store all the folders in the array `folder`. Each node of the trie contains a `children` field, used to store the child nodes of the current node, and a `fid` field, used to store the index of the folder corresponding to the current node in the array `folder`.

For each folder f in the array `folder`, we first split f into several substrings according to `/`, then start from the root node and add the substrings to the trie in turn. Next, we start from the root node to search the trie. If the `fid` field of the current node is not `-1`, it means that the folder corresponding to the current node is a folder in the answer array. We add it to the answer array and return. Otherwise, we recursively search all child nodes of the current node and finally return the answer array.

The time complexity is O(n × m), and the space complexity is O(n × m). Where n and m are the length of the array `folder` and the maximum length of the strings in the array `folder`, respectively.

Python 3

```python
class Trie:
    def __init__(self):
        self.children = {}
        self.fid = -1

    def insert(self, i, f):
        node = self
        ps = f.split('/')
        for p in ps[1:]:
            if p not in node.children:
                node.children[p] = Trie()
            node = node.children[p]
        node.fid = i

    def search(self):
        def dfs(root):
            if root.fid != -1:
                ans.append(root.fid)
                return
            for child in root.children.values():
                dfs(child)

        ans = []
        dfs(self)
        return ans


class Solution:
    def removeSubfolders(self, folder: List[str]) -> List[str]:
        trie = Trie()
        for i, f in enumerate(folder):
            trie.insert(i, f)
        return [folder[i] for i in trie.search()]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. How many folders are removed
# 2. The same task with a trie of path parts


# ============================================================
# FOLLOW-UP 1: How many folders are removed
# ============================================================

# ---- follow-up 1: function ----
# Answer: The number of folders minus the number that are
# kept.
# Time: O(n * L log n), L = path length.  Space: O(n * L).
def count_removed_folders(folder):
    folder = sorted(folder)
    kept = []
    for path in folder:
        if not kept or not path.startswith(kept[-1] + "/"):
            kept.append(path)
    return len(folder) - len(kept)


# ============================================================
# FOLLOW-UP 2: The same task with a trie of path parts
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk the parts of every path; stop when a folder
# end is met on the way.
```

```
# Time: O(n * L + r log r).  Space: O(n * L).
def remove_subfolders_with_trie(folder):
    root = {}
    for path in folder:
        node = root
        for part in path.split("/")[1:]:
            node = node.setdefault(part, {})
        node["#"] = path
    result = []

    def go(node):
        if "#" in node:
            result.append(node["#"])
            return
        for key, child in node.items():
            go(child)

    go(root)
    return sorted(result)
```

PART 2 · TRIES · PROBLEM 2 OF 4

2452. Words Within Two Edits of Dictionary

Medium · Trie, Array, String

leetcode.com/problems/words-within-two-edits-of-dictionary

★ **TOP 7 PROBLEM** in Tries: master this one first

PROBLEM

You are given two string arrays, `queries` and `dictionary`. All words in each array comprise of lowercase English letters and have the same length.

In one **edit** you can take a word from `queries`, and change any letter in it to any other letter. Find all words from `queries` that, after a **maximum** of two edits, equal some word from `dictionary`.

Return *a list of all words from* `queries`*, that match with some word from* `dictionary` *after a maximum of* ***two edits***. Return the words in the **same order** they appear in `queries`.

Example 1:

```
Input: queries = ["word","note","ants","wood"], dictionary = ["wood","joke","moat"]
Output: ["word","note","wood"]
Explanation:
- Changing the 'r' in "word" to 'o' allows it to equal the dictionary word "wood".
- Changing the 'n' to 'j' and the 't' to 'k' in "note" changes it to "joke".
- It would take more than 2 edits for "ants" to equal a dictionary word.
- "wood" can remain unchanged (0 edits) and match the corresponding dictionary word.
Thus, we return ["word","note","wood"].
```

Example 2:

```
Input: queries = ["yes"], dictionary = ["not"]
Output: []
Explanation:
Applying any two edits to "yes" cannot make it equal to "not". Thus, we return an empty array.
```

Constraints:

- `1 <= queries.length, dictionary.length <= 100`
- `n == queries[i].length == dictionary[j].length`
- `1 <= n <= 100`
- All `queries[i]` and `dictionary[j]` are composed of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Brute Force Enumeration

How to think about it

At most 100 queries and dictionary words of length ≤ 100. For each query, count mismatches against each dictionary word; fewer than three is allowed. Accept the query on the first hit.

We directly traverse each word s in the array queries, and then traverse each word t in the array dictionary. If there exists a word t whose edit distance from s is less than 3, we add s to the answer array and then exit the inner loop. If there is no such word t, we continue to traverse the next word s.

The time complexity is O(m × n × l), where m and n are the lengths of the arrays queries and dictionary respectively, and l is the length of the word.

Python 3

```
class Solution:
    def twoEditWords(self, queries: List[str], dictionary: List[str]) -> List[str]:
        ans = []
        for s in queries:
            for t in dictionary:
                if sum(a != b for a, b in zip(s, t)) < 3:
                    ans.append(s)
                    break
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Within k edits (not just two) of some dictionary word
# 2. Return the dictionary word that matches every query
# (or None)


# ============================================================
# FOLLOW-UP 1: Within k edits (not just two) of some
# dictionary word
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count the differing positions for each pair of
# equal length words.
# Time: O(q * d * L).  Space: O(1) extra besides the
# answer.
def words_within_k_edits(queries, dictionary, k):
    result = []
    for q in queries:
        for d in dictionary:
            if len(q) == len(d) and sum(1 for a, b in zip(q, d) if a != b) <= k:
                result.append(q)
                break
    return result


# ============================================================
# FOLLOW-UP 2: Return the dictionary word that matches
# every query (or None)
# ============================================================

# ---- follow-up 2: function ----
# Answer: The first dictionary word within two
# substitutions.
# Time: O(q * d * L).  Space: O(q).
def matching_dictionary_words(queries, dictionary):
    result = []
    for q in queries:
        match = None
        for d in dictionary:
            if sum(1 for a, b in zip(q, d) if a != b) <= 2:
                match = d
                break
        result.append(match)
    return result
```

3043. Find the Length of the Longest Common Prefix

Medium · Trie, Array, Hash Table, String

leetcode.com/problems/find-the-length-of-the-longest-common-prefix

★ **TOP 7 PROBLEM** in Tries: master this one first

PROBLEM

You are given two arrays with **positive** integers `arr1` and `arr2`.

A **prefix** of a positive integer is an integer formed by one or more of its digits, starting from its **leftmost** digit. For example, `123` is a prefix of the integer `12345`, while `234` is **not**.

A **common prefix** of two integers `a` and `b` is an integer `c`, such that `c` is a prefix of both `a` and `b`. For example, `5655359` and `56554` have common prefixes `565` and `5655` while `1223` and `43456` **do not** have a common prefix.

You need to find the length of the **longest common prefix** between all pairs of integers `(x, y)` such that `x` belongs to `arr1` and `y` belongs to `arr2`.

Return *the length of the **longest** common prefix among all pairs. If no common prefix exists among them*, *return* `0`.

Example 1:

```
Input: arr1 = [1,10,100], arr2 = [1000]
Output: 3
Explanation: There are 3 pairs (arr1[i], arr2[j]):
- The longest common prefix of (1, 1000) is 1.
- The longest common prefix of (10, 1000) is 10.
- The longest common prefix of (100, 1000) is 100.
The longest common prefix is 100 with a length of 3.
```

Example 2:

```
Input: arr1 = [1,2,3], arr2 = [4,4,4]
Output: 0
Explanation: There exists no common prefix for any pair (arr1[i], arr2[j]), hence we return 0.
Note that common prefixes between elements of the same array do not count.
```

Constraints:

- `1 <= arr1.length, arr2.length <= 5 * 10^4`
- `1 <= arr1[i], arr2[i] <= 10^8`

SOLUTION (PYTHON 3)

Solution 1: Hash Table

How to think about it

Integers are read as decimal strings; we want the longest common prefix between the two arrays. $n \le 5 \times 10^4$ forbids comparing every pair. All prefixes of a number are the values obtained by repeated division by 10. After storing arr_1 prefixes in a hash set, each value in arr_2 searches from itself toward shorter prefixes. The largest hit is the integer that encodes the longest prefix; its number of digits is the answer.

We can use a hash table to store all the prefixes of the numbers in `arr1`. Then, we traverse all the numbers x in `arr2`. For each number x, we start from the highest bit and gradually decrease, checking whether it exists in the hash table. If it does, we have found a common prefix, and we can update the answer accordingly.

The time complexity is $O(m \times \log M + n \times \log N)$, and the space complexity is $O(m \times \log M)$. Here, m and n are the lengths of `arr1` and `arr2` respectively, and M and N are the maximum values in `arr1` and `arr2` respectively.

Python 3

```
class Solution:
    def longestCommonPrefix(self, arr1: List[int], arr2: List[int]) -> int:
        s = set()
        for x in arr1:
            while x:
                s.add(x)
                x //= 10
        mx = 0
        for x in arr2:
            while x:
                if x in s:
                    mx = max(mx, x)
                    break
                x //= 10
        return len(str(mx)) if mx else 0
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the common prefix itself
# 2. The longest common prefix of two DIFFERENT numbers of
# one array


# ============================================================
# FOLLOW-UP 1: Return the common prefix itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Put the prefixes of arr1 in a set; for every
# number in arr2 look for the longest prefix in the set.
# Time: O((a + b) * D^2), D = digits per number (prefixes
# are sliced).  Space: O(a * D^2).
def longest_common_prefix_text(arr1, arr2):
    prefixes = set()
    for x in arr1:
        s = str(x)
        for i in range(1, len(s) + 1):
            prefixes.add(s[:i])
    best = ""
    for y in arr2:
        s = str(y)
        for i in range(len(s), 0, -1):
            if s[:i] in prefixes:
                if i > len(best):
                    best = s[:i]
                break
    return best


# ============================================================
# FOLLOW-UP 2: The longest common prefix of two DIFFERENT
# numbers of one array
# ============================================================

# ---- follow-up 2: function ----
# Answer: Insert each number into a trie before testing: a
# prefix seen twice is shared.
# Time: O(n * D^2).  Space: O(n * D^2).
def longest_shared_prefix_in_array(nums):
    seen = set()
    best = 0
    for x in nums:
        s = str(x)
        mine = set(s[:i] for i in range(1, len(s) + 1))
```

```
        for p in mine:
            if p in seen:
                best = max(best, len(p))
        seen |= mine
    return best
```

PART 2 · TRIES · PROBLEM 4 OF 4

425. Word Squares

Hard · Trie, Array, String, Backtracking

leetcode.com/problems/word-squares

★ **TOP 7 PROBLEM** in Tries: master this one first

PROBLEM

Given an array of **unique** strings `words`, return *all the* **word squares** *you can build from* `words`. The same word from `words` can be used **multiple times**. You can return the answer in **any order**.

A sequence of strings forms a valid **word square** if the k^{th} row and column read the same string, where `0 <= k < max(numRows, numColumns)`.

- For example, the word sequence `["ball","area","lead","lady"]` forms a word square because each word reads the same both horizontally and vertically.

Example 1:

```
Input: words = ["area","lead","wall","lady","ball"]
Output: [["ball","area","lead","lady"],["wall","area","lead","lady"]]
Explanation:
The output consists of two word squares. The order of output does not matter (just the order of words
in each word square matters).
```

Example 2:

```
Input: words = ["abat","baba","atan","atal"]
Output: [["baba","abat","baba","atal"],["baba","abat","baba","atan"]]
Explanation:
The output consists of two word squares. The order of output does not matter (just the order of words
in each word square matters).
```

Constraints:

- `1 <= words.length <= 1000`
- `1 <= words[i].length <= 4`
- All `words[i]` have the same length.
- `words[i]` consists of only lowercase English letters.
- All `words[i]` are **unique**.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Row k of a word square is forced to start with the k-th letters of the rows already chosen. Scanning the whole list for that prefix repeats the same comparisons. Insert every word into a trie and store indices on the path. DFS fills rows: the prefix is those k-th letters, and a trie walk returns every candidate at once. A square is complete when the number of rows equals the word length. Building the trie first turns prefix search into a descent.

Python 3

```
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.v = []

    def insert(self, w, i):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
            node.v.append(i)

    def search(self, w):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                return []
            node = node.children[idx]
        return node.v


class Solution:
    def wordSquares(self, words: List[str]) -> List[List[str]]:
        def dfs(t):
            if len(t) == len(words[0]):
                ans.append(t[:])
                return
            idx = len(t)
            pref = [v[idx] for v in t]
            indexes = trie.search(''.join(pref))
            for i in indexes:
                t.append(words[i])
                dfs(t)
                t.pop()

        trie = Trie()
        ans = []
        for i, w in enumerate(words):
            trie.insert(w, i)
        for w in words:
            dfs([w])
        return ans
```

Complexity. Time: exponential in the word length L: about O(N * 26^L) in the worst case. Space: O(N * L).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Count the word squares
# 2. Stop at the first word square


# ========================================================
# FOLLOW-UP 1: Count the word squares
# ========================================================

# ---- follow-up 1: function ----
# Answer: Same search, counting the finished squares.
# Time: O(N * L^2) to build the prefix table, then the
# search is exponential in L (at most N^L branches).
# Space: O(N * L^2).
def count_word_squares(words):
    n = len(words[0])
```

```
    prefix_words = {}
    for w in words:
        for i in range(n + 1):
            prefix_words.setdefault(w[:i], []).append(w)
    count = 0

    def go(square):
        nonlocal count
        if len(square) == n:
            count += 1
            return
        prefix = "".join(row[len(square)] for row in square)
        for w in prefix_words.get(prefix, []):
            go(square + [w])

    for w in words:
        go([w])
    return count


# ============================================================
# FOLLOW-UP 2: Stop at the first word square
# ============================================================

# ---- follow-up 2: function ----
# Answer: Return as soon as one square is complete.
# Time: the same worst case, but it stops at the first
# square.  Space: O(N * L^2).
def first_word_square(words):
    n = len(words[0])
    prefix_words = {}
    for w in words:
        for i in range(n + 1):
            prefix_words.setdefault(w[:i], []).append(w)

    def go(square):
        if len(square) == n:
            return square
        prefix = "".join(row[len(square)] for row in square)
        for w in prefix_words.get(prefix, []):
            found = go(square + [w])
            if found:
                return found
        return None
```

```
    for w in words:
        found = go([w])
        if found:
            return found
    return None
```

PART 2 · TOPIC 27 OF 50

Heaps

1 problem · 0 Easy · 1 Medium · 0 Hard

THE IDEA

A heap gives the smallest (or biggest) item in O(log n). Use it when you repeatedly need the best candidate so far.

THE 1 PROBLEMS, EASY TO HARD | ★ = TOP 1: DO THESE FIRST

1	★ **Longest Happy String**	Medium	LC 1405	2398

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Last Stone Weight (Part 1, pattern 19, page 1038)
Minimum Cost to Connect Sticks (Part 1, pattern 19, page 1057)
Furthest Building You Can Reach (Part 1, pattern 19, page 1060)
Single-Threaded CPU (Part 1, pattern 19, page 1063)
Process Tasks Using Servers (Part 1, pattern 19, page 1083)
Maximum Performance of a Team (Part 1, pattern 19, page 1080)

PART 2 · HEAPS · PROBLEM 1 OF 1

1405. Longest Happy String

Medium · Greedy, String, Heap (Priority Queue)

leetcode.com/problems/longest-happy-string

★ **TOP 7 PROBLEM** in Heaps: master this one first

PROBLEM

A string `s` is called **happy** if it satisfies the following conditions:

- `s` only contains the letters `'a'`, `'b'`, and `'c'`.
- `s` does not contain any of `"aaa"`, `"bbb"`, or `"ccc"` as a substring.
- `s` contains **at most** `a` occurrences of the letter `'a'`.
- `s` contains **at most** `b` occurrences of the letter `'b'`.
- `s` contains **at most** `c` occurrences of the letter `'c'`.

Given three integers `a`, `b`, and `c`, return *the **longest possible happy** string*. If there are multiple longest happy strings, return *any of them*. If there is no such string, return *the empty string* `""`.

A **substring** is a contiguous sequence of characters within a string.

Example 1:

```
Input: a = 1, b = 1, c = 7
Output: "ccaccbcc"
Explanation: "ccbccacc" would also be a correct answer.
```

Example 2:

```
Input: a = 7, b = 1, c = 0
Output: "aabaa"
Explanation: It is the only correct answer in this case.
```

Constraints:

- `0 <= a, b, c <= 100`
- `a + b + c > 0`

SOLUTION (PYTHON 3)

Solution 1: Greedy + Priority Queue

How to think about it

a+b+c≤ 300 would allow search, but the optimum follows a local rule: always spend the letter that remains most often, without three identical characters in a row. If the last two characters are already that letter, take the second-most instead. A max-heap by remaining count implements this: pop, append if legal, otherwise pop the next, then push leftovers back. Stop when no letter can be appended, which yields the longest happy string.

The greedy strategy is to prioritize the selection of characters with the most remaining occurrences. By using a priority queue or sorting, we ensure that the character selected each time is the one with the most remaining occurrences (to avoid having three consecutive identical characters, in some cases, we need to select the character with the second most remaining occurrences).

Python 3

```
class Solution:
    def longestDiverseString(self, a: int, b: int, c: int) -> str:
        h = []
        if a > 0:
            heappush(h, [-a, 'a'])
        if b > 0:
            heappush(h, [-b, 'b'])
        if c > 0:
            heappush(h, [-c, 'c'])

        ans = []
        while len(h) > 0:
            cur = heappop(h)
            if len(ans) >= 2 and ans[-1] == cur[1] and ans[-2] == cur[1]:
                if len(h) == 0:
                    break
                nxt = heappop(h)
                ans.append(nxt[1])
                if -nxt[0] > 1:
                    nxt[0] += 1
                    heappush(h, nxt)
                heappush(h, cur)
            else:
                ans.append(cur[1])
                if -cur[0] > 1:
                    cur[0] += 1
                    heappush(h, cur)

        return ''.join(ans)
```

Complexity. Time: O(n), n = a + b + c (a heap of at most 3 letters). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. No more than k equal letters in a row (instead of 2)
# 2. The same rule with any number of letters


# ============================================================
# FOLLOW-UP 1: No more than k equal letters in a row
# (instead of 2)
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same greedy: take the most frequent letter
# unless it already ended with k equal letters.
# Time: O(n * k), n = total letters.  Space: O(n).
import heapq


def longest_string_with_runs(a, b, c, k):
    heap = [(-n, ch) for n, ch in ((a, "a"), (b, "b"), (c, "c")) if n > 0]
    heapq.heapify(heap)
    result = []
    while heap:
        count, ch = heapq.heappop(heap)
        if len(result) >= k and all(x == ch for x in result[-k:]):
            if not heap:
                break
            count2, ch2 = heapq.heappop(heap)
            result.append(ch2)
            if count2 + 1 < 0:
                heapq.heappush(heap, (count2 + 1, ch2))
            heapq.heappush(heap, (count, ch))
        else:
```

```
            result.append(ch)
            if count + 1 < 0:
                heapq.heappush(heap, (count + 1, ch))
    return "".join(result)


# ============================================================
# FOLLOW-UP 2: The same rule with any number of letters
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same heap with one entry per letter.
# Time: O(n * (k + log c)), c = letters in the alphabet.
# Space: O(n).
def longest_string_any_letters(counts, k=2):
    heap = [(-n, ch) for ch, n in counts.items() if n > 0]
    heapq.heapify(heap)
    result = []
    while heap:
        count, ch = heapq.heappop(heap)
        if len(result) >= k and all(x == ch for x in result[-k:]):
            if not heap:
                break
            count2, ch2 = heapq.heappop(heap)
            result.append(ch2)
            if count2 + 1 < 0:
                heapq.heappush(heap, (count2 + 1, ch2))
            heapq.heappush(heap, (count, ch))
```

```
        else:
            result.append(ch)
            if count + 1 < 0:
                heapq.heappush(heap, (count + 1, ch))
    return "".join(result)
```

PART 2 · TOPIC 28 OF 50

Intervals

6 problems · 2 Easy · 3 Medium · 1 Hard

THE IDEA

Sort by start (or end) and sweep once, keeping the last merged interval or the number of overlapping meetings.

THE 6 PROBLEMS, EASY TO HARD | ★ = TOP 4: DO THESE FIRST

1	Summary Ranges	Easy	LC 228	2402
2	Meeting Rooms	Easy	LC 252	2404
3	★ **Meeting Rooms II**	Medium	LC 253	2406
4	★ **Meeting Scheduler**	Medium	LC 1229	2409
5	★ **Two Best Non-Overlapping Events**	Medium	LC 2054	2412
6	★ **Meeting Rooms III**	Hard	LC 2402	2415

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Interval List Intersections (Part 1, pattern 20, page 1099)

Merge Intervals (Part 1, pattern 20, page 1087)

Remove Covered Intervals (Part 1, pattern 20, page 1118)

Minimum Number of Arrows to Burst Balloons (Part 1, pattern 20, page 1097)

Non-overlapping Intervals (Part 1, pattern 20, page 1094)

Car Pooling (Part 1, pattern 20, page 1109)

Insert Interval (Part 1, pattern 20, page 1090)

Partition Labels (Part 1, pattern 3, page 169)

Maximum Number of Events That Can Be Attended (Part 1, pattern 20, page 1112)

PART 2 · INTERVALS · PROBLEM 1 OF 6

228. Summary Ranges

Easy · Array

leetcode.com/problems/summary-ranges

PROBLEM

You are given a **sorted unique** integer array `nums`.

A **range** `[a,b]` is the set of all integers from `a` to `b` (inclusive).

Return *the **smallest sorted** list of ranges that **cover all the numbers in the array exactly***. That is, each element of `nums` is covered by exactly one of the ranges, and there is no integer `x` such that `x` is in one of the ranges but not in `nums`.

Each range `[a,b]` in the list should be output as:

- `"a->b"` if `a != b`
- `"a"` if `a == b`

Example 1:

```
Input: nums = [0,1,2,4,5,7]
Output: ["0->2","4->5","7"]
Explanation: The ranges are:
[0,2] --> "0->2"
[4,5] --> "4->5"
[7,7] --> "7"
```

Example 2:

```
Input: nums = [0,2,3,4,6,8,9]
Output: ["0","2->4","6","8->9"]
Explanation: The ranges are:
[0,0] --> "0"
[2,4] --> "2->4"
[6,6] --> "6"
[8,9] --> "8->9"
```

Constraints:

- `0 <= nums.length <= 20`
- -2^{31} `<= nums[i] <=` $2^{31} - 1$
- All the values of `nums` are **unique**.
- `nums` is sorted in ascending order.

SOLUTION (PYTHON 3)

Solution 1: Two Pointers

> **How to think about it**
>
> The array is sorted and unique, so a range is a maximal run of neighbors that differ by 1. One scan can cut the runs. Pointers i,j mark a segment until nums[j+1]≠ nums[j]+1, then format a singleton or a→b.

We can use two pointers i and j to find the left and right endpoints of each interval.

Traverse the array, when j + 1 < n and nums[j + 1] = nums[j] + 1, move j to the right, otherwise the interval [i, j] has been found, add it to the answer, then move i to the position of j + 1, and continue to find the next interval.

Time complexity O(n), where n is the length of the array. Space complexity O(1).

Python 3

```python
class Solution:
    def summaryRanges(self, nums: List[int]) -> List[str]:
        def f(i: int, j: int) -> str:
            return str(nums[i]) if i == j else f'{nums[i]}->{nums[j]}'

        i = 0
        n = len(nums)
        ans = []
        while i < n:
            j = i
            while j + 1 < n and nums[j + 1] == nums[j] + 1:
                j += 1
            ans.append(f(i, j))
            i = j + 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Join numbers whose gap is at most g into one range
# 2. Expand the text ranges back into the numbers


# ============================================================
# FOLLOW-UP 1: Join numbers whose gap is at most g into
# one range
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same sweep with the condition 'next - current <=
# g'.
# Time: O(n).  Space: O(n).
def summary_with_gap(nums, g):
    ranges = []
    for x in nums:
        if ranges and x - ranges[-1][1] <= g:
            ranges[-1][1] = x
        else:
            ranges.append([x, x])
    return [str(a) if a == b else str(a) + "->" + str(b) for a, b in ranges]


# ============================================================
# FOLLOW-UP 2: Expand the text ranges back into the
# numbers
# ============================================================

# ---- follow-up 2: function ----
# Answer: Split on '->' and generate the numbers.
# Time: O(total numbers).  Space: O(total numbers).
def expand_summary(ranges):
    nums = []
    for text in ranges:
        if "->" in text:
            a, b = text.split("->")
            nums.extend(range(int(a), int(b) + 1))
        else:
            nums.append(int(text))
    return nums
```

PART 2 · INTERVALS · PROBLEM 2 OF 6

252. Meeting Rooms

Easy · Array, Sorting, Quick Sort

leetcode.com/problems/meeting-rooms

PROBLEM

You are given an array of meeting times `intervals` where `intervals[i] = [start_i, end_i]`.

A person can attend all meetings if no two meeting intervals overlap. Meetings ending at time `t` and starting at time `t` **do not** overlap.

Return `true` if a person can attend all meetings. Otherwise, return `false`.

Example 1:

```
Input: intervals = [[0,30],[5,10],[15,20]]
Output: false
```

Example 2:

```
Input: intervals = [[7,10],[2,4]]
Output: true
```

Constraints:

- `0 <= intervals.length <= 10^4`
- `intervals[i].length == 2`
- `0 <= start_i < end_i <= 10^6`

SOLUTION (PYTHON 3)

Solution 1: Sorting

> **How to think about it**
>
> One person cannot attend two overlapping meetings. After sorting by start time, it is enough that each meeting ends no later than the next one starts.

We sort the meetings based on their start times, and then iterate through the sorted meetings. If the start time of the current meeting is less than the end time of the previous meeting, it indicates that there is an overlap between the two meetings, and we return `false`. Otherwise, we continue iterating.

If no overlap is found by the end of the iteration, we return `true`.

The time complexity is O(n × log n), and the space complexity is O(log n), where n is the number of meetings.

Python 3

```python
class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort()
        return all(a[1] <= b[0] for a, b in pairwise(intervals))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the first two meetings that clash
# 2. Return the free slots between the meetings


# ============================================================
# FOLLOW-UP 1: Return the first two meetings that clash
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort by start; a clash is a meeting that starts
# before the previous one ends.
# Time: O(n log n).  Space: O(n).
def first_clash(intervals):
    ordered = sorted(intervals)
    for i in range(1, len(ordered)):
        if ordered[i][0] < ordered[i - 1][1]:
            return ordered[i - 1], ordered[i]
    return None


# ============================================================
# FOLLOW-UP 2: Return the free slots between the meetings
# ============================================================

# ---- follow-up 2: function ----
# Answer: After sorting, the gaps between a meeting's end
# and the next start.
# Time: O(n log n).  Space: O(n).
def free_slots(intervals):
    ordered = sorted(intervals)
    slots = []
    end = ordered[0][1]
    for start, finish in ordered[1:]:
        if start > end:
            slots.append((end, start))
        end = max(end, finish)
    return slots
```

PART 2 · INTERVALS · PROBLEM 3 OF 6

253. Meeting Rooms II

Medium · Greedy, Array, Two Pointers, Prefix Sum, Sorting, Heap (Priority Queue)

leetcode.com/problems/meeting-rooms-ii

★ **TOP 7 PROBLEM** in Intervals: master this one first

PROBLEM

Given an array of meeting time intervals `intervals` where `intervals[i] = [start_i, end_i]`, return *the minimum number of conference rooms required*.

Example 1:

```
Input: intervals = [[0,30],[5,10],[15,20]]
Output: 2
```

Example 2:

```
Input: intervals = [[7,10],[2,4]]
Output: 1
```

Constraints:

- `1 <= intervals.length <= 10^4`
- `0 <= start_i < end_i <= 10^6`

SOLUTIONS (PYTHON 3)

Solution 1: Difference Array

How to think about it

The number of rooms is the peak number of meetings underway. Add 1 at each start and -1 at each end; the prefix sum is the occupancy. A difference array up to the latest end time, then one scan, yields that peak.

We can implement this using a difference array.

First, we find the maximum end time of all the meetings, denoted as m. Then, we create a difference array d of length m + 1. For each meeting, we add to the corresponding positions in the difference array: d[l] = d[l] + 1 for the start time, and d[r] = d[r] - 1 for the end time.

Next, we calculate the prefix sum of the difference array and find the maximum value of the prefix sum, which represents the minimum number of meeting rooms required.

The time complexity is O(n + m) and the space complexity is O(m), where n is the number of meetings and m is the maximum end time.

Python 3

```python
class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        m = max(e[1] for e in intervals)
        d = [0] * (m + 1)
        for l, r in intervals:
            d[l] += 1
            d[r] -= 1
        ans = s = 0
        for v in d:
            s += v
            ans = max(ans, s)
        return ans
```

Solution 2: Difference (Hash Map)

> **How to think about it**
> A large time horizon wastes an O(m) array. A hash map stores updates only at endpoints; sorting the keys and taking a prefix-sum peak is equivalent.

If the meeting times span a large range, we can use a hash map instead of a difference array.

First, we create a hash map d, where we add to the corresponding positions for each meeting's start time and end time: d[l] = d[l] + 1 for the start time, and d[r] = d[r] - 1 for the end time.

Then, we sort the hash map by its keys, calculate the prefix sum of the hash map, and find the maximum value of the prefix sum, which represents the minimum number of meeting rooms required.

The time complexity is O(n × log n) and the space complexity is O(n), where n is the number of meetings.

Python 3

```
class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        d = defaultdict(int)
        for l, r in intervals:
            d[l] += 1
            d[r] -= 1
        ans = s = 0
        for _, v in sorted(d.items()):
            s += v
            ans = max(ans, s)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Say which room every meeting uses
# 2. The time at which the most meetings are on at once


# ============================================================
# FOLLOW-UP 1: Say which room every meeting uses
# ============================================================

# ---- follow-up 1: function ----
# Answer: A heap of (end time, room); reuse the room that
# frees up first, otherwise open a new one.
# Time: O(n log n).  Space: O(n).
import heapq


def room_of_each_meeting(intervals):
    order = sorted(range(len(intervals)), key=lambda i: intervals[i][0])
    busy = []
    free = []
    assignment = [0] * len(intervals)
    rooms = 0
    for i in order:
        start, end = intervals[i]
        while busy and busy[0][0] <= start:
            heapq.heappush(free, heapq.heappop(busy)[1])
        if free:
            room = heapq.heappop(free)
        else:
            room = rooms
            rooms += 1
        assignment[i] = room
        heapq.heappush(busy, (end, room))
    return assignment
```

```
# ============================================================
# FOLLOW-UP 2: The time at which the most meetings are on
# at once
# ============================================================

# ---- follow-up 2: function ----
# Answer: Sweep over the start and end events.
# Time: O(n log n).  Space: O(n).
def busiest_time(intervals):
    events = []
    for start, end in intervals:
        events.append((start, 1))
        events.append((end, -1))
    events.sort()
    current = best = 0
    when = None
    for time, delta in events:
        current += delta
        if current > best:
            best = current
            when = time
    return when, best
```

1229. Meeting Scheduler

Medium · Array, Two Pointers, Sorting

leetcode.com/problems/meeting-scheduler

★ **TOP 7 PROBLEM** in Intervals: master this one first

PROBLEM

Given the availability time slots arrays `slots1` and `slots2` of two people and a meeting duration `duration`, return the **earliest time slot** that works for both of them and is of duration `duration`.

If there is no common time slot that satisfies the requirements, return an **empty array**.

The format of a time slot is an array of two elements `[start, end]` representing an inclusive time range from `start` to `end`.

It is guaranteed that no two availability slots of the same person intersect with each other. That is, for any two time slots `[start1, end1]` and `[start2, end2]` of the same person, either `start1 > end2` or `start2 > end1`.

Example 1:

```
Input: slots1 = [[10,50],[60,120],[140,210]], slots2 = [[0,15],[60,70]], duration = 8
Output: [60,68]
```

Example 2:

```
Input: slots1 = [[10,50],[60,120],[140,210]], slots2 = [[0,15],[60,70]], duration = 12
Output: []
```

Constraints:

- `1 <= slots1.length, slots2.length <=` 10^4
- `slots1[i].length, slots2[i].length == 2`
- `slots1[i][0] < slots1[i][1]`
- `slots2[i][0] < slots2[i][1]`
- `0 <= slots1[i][j], slots2[i][j] <=` 10^9
- `1 <= duration <=` 10^6

SOLUTION (PYTHON 3)

Solution 1: Sorting + Two Pointers

How to think about it

Each person has up to 10^4 slots; checking every pair is O(mn). After sorting by start, an overlap can only involve the two current slots: start is the max of starts, end the min of ends. If the overlap is long enough, we return the earliest such interval. Otherwise the slot that ends first cannot form an earlier feasible overlap with later slots of the other person, so we advance that pointer. Sorting walks candidates in time order; two pointers discard a hopeless slot each step, for a linear number of comparisons.

We can sort the free time intervals of both people, then use two pointers to traverse the two arrays and find the intersection of the free time intervals of both people. If the length of the intersection is greater than or equal to `duration`, return the start time of the intersection and the start time plus `duration`. Otherwise, if the end time of the first person's free time interval is less than the end time of the second person's free time interval, move the first person's pointer; otherwise, move the second person's pointer. Continue traversing until a suitable time interval is found or the traversal ends.

The time complexity is O(m × log m + n × log n), and the space complexity is O(log m + log n). Here, m and n are the lengths of the two arrays, respectively.

Python 3

```python
class Solution:
    def minAvailableDuration(
        self, slots1: List[List[int]], slots2: List[List[int]], duration: int
    ) -> List[int]:
        slots1.sort()
        slots2.sort()
        m, n = len(slots1), len(slots2)
        i = j = 0
        while i < m and j < n:
            start = max(slots1[i][0], slots2[j][0])
            end = min(slots1[i][1], slots2[j][1])
            if end - start >= duration:
                return [start, start + duration]
            if slots1[i][1] < slots2[j][1]:
                i += 1
            else:
                j += 1
        return []
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Any number of people: the earliest common slot of
# length d
# 2. All the possible start times (one per slot) for a
# meeting of length d


# ========================================================
# FOLLOW-UP 1: Any number of people: the earliest common
# slot of length d
# ========================================================

# ---- follow-up 1: function ----
# Answer: Intersect the schedules one after another with
# two pointers, then pick the first long enough slot.
# Time: O(K * S log S), S = slots per person.  Space:
# O(S).
def common_slot(schedules, d):
    def intersect(a, b):
        a, b = sorted(a), sorted(b)
        i = j = 0
        result = []
        while i < len(a) and j < len(b):
            start = max(a[i][0], b[j][0])
            end = min(a[i][1], b[j][1])
            if start < end:
                result.append([start, end])
            if a[i][1] < b[j][1]:
                i += 1
            else:
                j += 1
        return result

    common = schedules[0]
    for other in schedules[1:]:
        common = intersect(common, other)
    for start, end in sorted(common):
        if end - start >= d:
            return [start, start + d]
    return []


# ========================================================
# FOLLOW-UP 2: All the possible start times (one per slot)
```

```
# for a meeting of length d
# ============================================================

# ---- follow-up 2: function ----
# Answer: Take every slot that is long enough and list the
# first start of each.
# Time: O(a log a + b log b).  Space: O(a + b).
def all_slot_starts(a, b, d):
    a, b = sorted(a), sorted(b)
    i = j = 0
    starts = []
    while i < len(a) and j < len(b):
        start = max(a[i][0], b[j][0])
        end = min(a[i][1], b[j][1])
        if end - start >= d:
            starts.append(start)
```

```
        if a[i][1] < b[j][1]:
            i += 1
        else:
            j += 1
    return starts
```

2054. Two Best Non-Overlapping Events

Medium · Array, Binary Search, Dynamic Programming, Sorting, Heap (Priority Queue)

leetcode.com/problems/two-best-non-overlapping-events

★ **TOP 7 PROBLEM** in Intervals: master this one first

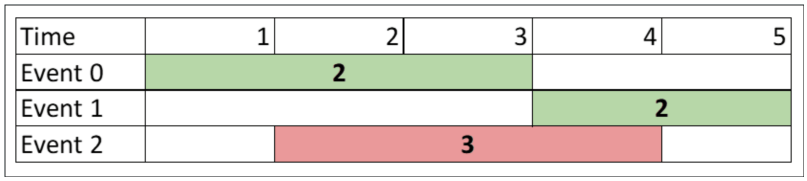
PROBLEM

You are given a **0-indexed** 2D integer array of `events` where `events[i] = [startTime`$_i$`, endTime`$_i$`, value`$_i$`]`. The i^{th} event starts at `startTime`$_i$ and ends at `endTime`$_i$, and if you attend this event, you will receive a value of `value`$_i$. You can choose **at most two non-overlapping** events to attend such that the sum of their values is **maximized**.

Return *this* ***maximum*** *sum.*

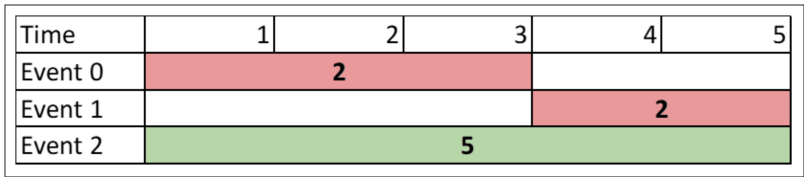
Note that the start time and end time is **inclusive**: that is, you cannot attend two events where one of them starts and the other ends at the same time. More specifically, if you attend an event with end time `t`, the next event must start at or after `t + 1`.

Example 1:



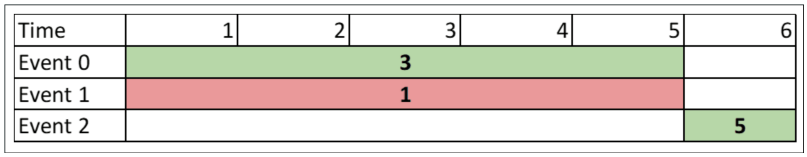

```
Input: events = [[1,3,2],[4,5,2],[2,4,3]]
Output: 4
Explanation: Choose the green events, 0 and 1 for a sum of 2 + 2 = 4.
```

Example 2:




```
Input: events = [[1,3,2],[4,5,2],[1,5,5]]
Output: 5
Explanation: Choose event 2 for a sum of 5.
```

Example 3:




```
Input: events = [[1,5,3],[1,5,1],[6,6,5]]
Output: 8
Explanation: Choose events 0 and 2 for a sum of 3 + 5 = 8.
```

Constraints:

- `2 <= events.length <= 10`^{5}
- `events[i].length == 3`
- `1 <= startTime`$_i$ `<= endTime`$_i$ `<= 10`^{9}
- `1 <= value`$_i$ `<= 10`^{6}

SOLUTION (PYTHON 3)

Solution 1: Sorting + Binary Search

How to think about it

Pick at most two non-overlapping events of maximum value. Pairwise search is quadratic for n ≤ 10^5. After sorting by start, the best single event in a suffix is a static array. f[i] is that suffix maximum. For each event as the first, binary-search the first start after its end and add f[idx] (or take the event alone). Sort plus binary search is O(n log n).

We can sort the events by their start times, and then preprocess the maximum value starting from each event, i.e., f[i] represents the maximum value of choosing one event from the i-th event to the last event.

Then we enumerate each event. For each event, we use binary search to find the first event whose start time is greater than the end time of the current event, denoted as idx. The maximum value starting from the current event is f[idx] plus the value of the current event, which is the maximum value that can be obtained by choosing the current event as the first event. We take the maximum value among all these values.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the number of events.

Python 3

```python
class Solution:
    def maxTwoEvents(self, events: List[List[int]]) -> int:
        events.sort()
        n = len(events)
        f = [events[-1][2]] * n
        for i in range(n - 2, -1, -1):
            f[i] = max(f[i + 1], events[i][2])
        ans = 0
        for _, e, v in events:
            idx = bisect_right(events, e, key=lambda x: x[0])
            if idx < n:
                v += f[idx]
            ans = max(ans, v)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. At most k events
# 2. Return the events that give the best pair


# ========================================================
# FOLLOW-UP 1: At most k events
# ========================================================

# ---- follow-up 1: function ----
# Answer: Sort by start; dp[j][i] is the best value using
# j events from event i on, with binary search for the
# next event that can follow.
# Time: O(n log n + n * k).  Space: O(n * k).
import bisect


def best_k_events(events, k):
    events = sorted(events)
    n = len(events)
    starts = [e[0] for e in events]
    nxt = [bisect.bisect_right(starts, events[i][1]) for i in range(n)]
    dp = [[0] * (n + 2) for _ in range(k + 1)]
    for j in range(1, k + 1):
        for i in range(n - 1, -1, -1):
            dp[j][i] = max(dp[j][i + 1], events[i][2] + dp[j - 1][nxt[i]])
```

```
    return dp[k][0]


# ============================================================
# FOLLOW-UP 2: Return the events that give the best pair
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same sweep, remembering which event gave the
# best earlier value.
# Time: O(n log n).  Space: O(n).
def best_pair_events(events):
    by_end = sorted(events, key=lambda e: e[1])
    ends = [e[1] for e in by_end]
    prefix_best = []
    best = None
    for e in by_end:
        if best is None or e[2] > best[2]:
            best = e
        prefix_best.append(best)
    answer = (0, None)
    for e in events:
        total = e[2]
        pair = [e]
        i = bisect.bisect_left(ends, e[0]) - 1
        if i >= 0:
            total += prefix_best[i][2]
            pair.append(prefix_best[i])
        if total > answer[0]:
            answer = (total, pair)
    return answer
```

2402. Meeting Rooms III

Hard · Array, Hash Table, Sorting, Simulation, Heap (Priority Queue)

leetcode.com/problems/meeting-rooms-iii

★ **TOP 7 PROBLEM** in Intervals: master this one first

PROBLEM

You are given an integer `n`. There are `n` rooms numbered from `0` to `n - 1`.

You are given a 2D integer array `meetings` where `meetings[i] = [start_i, end_i]` means that a meeting will be held during the **half-closed** time interval `[start_i, end_i)`. All the values of `start_i` are **unique**.

Meetings are allocated to rooms in the following manner:

1. Each meeting will take place in the unused room with the **lowest** number.
2. If there are no available rooms, the meeting will be delayed until a room becomes free. The delayed meeting should have the **same** duration as the original meeting.
3. When a room becomes unused, meetings that have an earlier original **start** time should be given the room.

Return *the **number** of the room that held the most meetings.* If there are multiple rooms, return *the room with the **lowest** number.*

A **half-closed interval** `[a, b)` is the interval between `a` and `b` **including** `a` and **not including** `b`.

Example 1:

```
Input: n = 2, meetings = [[0,10],[1,5],[2,7],[3,4]]
Output: 0
Explanation:
- At time 0, both rooms are not being used. The first meeting starts in room 0.
- At time 1, only room 1 is not being used. The second meeting starts in room 1.
- At time 2, both rooms are being used. The third meeting is delayed.
- At time 3, both rooms are being used. The fourth meeting is delayed.
- At time 5, the meeting in room 1 finishes. The third meeting starts in room 1 for the time period
[5,10).
- At time 10, the meetings in both rooms finish. The fourth meeting starts in room 0 for the time
period [10,11).
Both rooms 0 and 1 held 2 meetings, so we return 0.
```

Example 2:

```
Input: n = 3, meetings = [[1,20],[2,10],[3,5],[4,9],[6,8]]
Output: 1
Explanation:
- At time 1, all three rooms are not being used. The first meeting starts in room 0.
- At time 2, rooms 1 and 2 are not being used. The second meeting starts in room 1.
- At time 3, only room 2 is not being used. The third meeting starts in room 2.
- At time 4, all three rooms are being used. The fourth meeting is delayed.
- At time 5, the meeting in room 2 finishes. The fourth meeting starts in room 2 for the time period
[5,10).
- At time 6, all three rooms are being used. The fifth meeting is delayed.
- At time 10, the meetings in rooms 1 and 2 finish. The fifth meeting starts in room 1 for the time
period [10,12).
Room 0 held 1 meeting while rooms 1 and 2 each held 2 meetings, so we return 1.
```

Constraints:

- `1 <= n <= 100`
- `1 <= meetings.length <= 10^5`
- `meetings[i].length == 2`

- $0 <= start_i < end_i <= 5 * 10^5$
- All the values of $start_i$ are **unique**.

SOLUTION (PYTHON 3)

Solution 1: Priority Queue (Min-Heap)

How to think about it

With $n \leq 100$ and $m \leq 10^5$, scanning every room for each meeting is O(mn) and tight. Start times are distinct, so meetings must be assigned in time order. Idle rooms are chosen by the smallest index; busy rooms leave by earliest end time. Two heaps maintain these sets. After sorting meetings by start, return finished rooms to the idle heap; take the smallest idle index if any, otherwise delay the earliest-ending room by the meeting length. A delayed end time is the previous end plus the duration, and a long chain exceeds 2^{31}, so that timestamp is 64-bit.

We define two priority queues to represent idle meeting rooms and busy meeting rooms respectively. The idle meeting rooms idle are sorted by **index**; the busy meeting rooms busy are sorted by **end time and index**.

First, sort the meetings by start time, then iterate through the meetings. For each meeting:

- If there are busy meeting rooms with end time less than or equal to the current meeting's start time, add them to the idle meeting rooms queue idle;
- If there are idle meeting rooms available, take the room with the smallest index from the idle queue idle and add it to the busy queue busy;
- If there are no idle meeting rooms, find the room with the earliest end time and smallest index from the busy queue busy, and add it back to the busy queue busy.

The time complexity is O(m (log m + log n)), and the space complexity is O(n + m), where n and m are the number of meeting rooms and meetings respectively.

Similar problems:

- 1882. Process Tasks Using Servers

Python 3

```python
class Solution:
    def mostBooked(self, n: int, meetings: List[List[int]]) -> int:
        meetings.sort(key=lambda x: x[0])
        busy = []
        idle = list(range(n))
        heapify(idle)
        cnt = [0] * n
        for s, e in meetings:
            while busy and busy[0][0] <= s:
                heappush(idle, heappop(busy)[1])
            i = 0
            if idle:
                i = heappop(idle)
                heappush(busy, (e, i))
            else:
                time_end, i = heappop(busy)
                heappush(busy, (time_end + e - s, i))
            cnt[i] += 1
        ans = 0
        for i in range(n):
            if cnt[ans] < cnt[i]:
                ans = i
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The total time each room is busy
# 2. How many meetings were delayed (could not start on
# time)


# ============================================================
# FOLLOW-UP 1: The total time each room is busy
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same simulation, adding the length of each
# meeting to its room.
# Time: O(m log m + m log n).  Space: O(n).
import heapq


def busy_time_per_room(n, meetings):
    meetings = sorted(meetings)
    free = list(range(n))
    heapq.heapify(free)
    busy = []
    time = [0] * n
    for start, end in meetings:
        while busy and busy[0][0] <= start:
            heapq.heappush(free, heapq.heappop(busy)[1])
        if free:
            room = heapq.heappop(free)
            finish = end
        else:
            finish0, room = heapq.heappop(busy)
            finish = finish0 + (end - start)
        time[room] += end - start
        heapq.heappush(busy, (finish, room))
    return time


# ============================================================
# FOLLOW-UP 2: How many meetings were delayed (could not
# start on time)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the meetings that had to wait for a room.
# Time: O(m log m + m log n).  Space: O(n).
def delayed_meetings(n, meetings):
    meetings = sorted(meetings)
    free = list(range(n))
    heapq.heapify(free)
    busy = []
    delayed = 0
    for start, end in meetings:
        while busy and busy[0][0] <= start:
            heapq.heappush(free, heapq.heappop(busy)[1])
        if free:
            room = heapq.heappop(free)
            finish = end
        else:
            finish0, room = heapq.heappop(busy)
            finish = finish0 + (end - start)
            delayed += 1
        heapq.heappush(busy, (finish, room))
    return delayed
```

PART 2 · TOPIC 29 OF 50

Data Structure Design

9 problems · 1 Easy · 2 Medium · 6 Hard

THE IDEA

Pick the data structures so that every operation meets the required time: a hash map plus a list, a map plus a linked list, a heap with lazy deletion.

THE 9 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	Logger Rate Limiter	Easy	LC 359	2419
2	★ **Encode and Decode Strings**	Medium	LC 271	2422
3	★ **Design Hit Counter**	Medium	LC 362	2425
4	★ **All O`one Data Structure**	Hard	LC 432	2428
5	★ **Design In-Memory File System**	Hard	LC 588	2432
6	★ **Design Search Autocomplete System**	Hard	LC 642	2437
7	Max Stack	Hard	LC 716	2441
8	★ **Maximum Frequency Stack**	Hard	LC 895	2445
9	★ **Design a Text Editor**	Hard	LC 2296	2449

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Implement Queue using Stacks (Part 1, pattern 10, page 478)
Implement Stack using Queues (Part 1, pattern 10, page 481)
Design Circular Queue (Part 1, pattern 10, page 490)
Design Circular Deque (Part 1, pattern 10, page 494)
Design Browser History (Part 1, pattern 29, page 1560)
Time Based Key-Value Store (Part 1, pattern 4, page 222)
Snapshot Array (Part 1, pattern 29, page 1564)
Design Twitter (Part 1, pattern 29, page 1552)
LRU Cache (Part 1, pattern 25, page 1380)
Insert Delete GetRandom O(1) (Part 1, pattern 1, page 58)
Design a Food Rating System (Part 1, pattern 29, page 1574)
LFU Cache (Part 1, pattern 29, page 1581)
Range Module (Part 1, pattern 20, page 1131)

359. Logger Rate Limiter

Easy · Design, Hash Table, Data Stream

leetcode.com/problems/logger-rate-limiter

PROBLEM

Design a logger system that receives a stream of messages along with their timestamps. Each **unique** message should only be printed **at most every 10 seconds** (i.e. a message printed at timestamp `t` will prevent other identical messages from being printed until timestamp `t + 10`).

All messages will come in chronological order. Several messages may arrive at the same timestamp.

Implement the `Logger` class:

- `Logger()` Initializes the `logger` object.
- `bool shouldPrintMessage(int timestamp, string message)` Returns `true` if the `message` should be printed in the given `timestamp`, otherwise returns `false`.

Example 1:

```
Input
["Logger", "shouldPrintMessage", "shouldPrintMessage", "shouldPrintMessage", "shouldPrintMessage",
"shouldPrintMessage", "shouldPrintMessage"]
[[], [1, "foo"], [2, "bar"], [3, "foo"], [8, "bar"], [10, "foo"], [11, "foo"]]
Output
[null, true, true, false, false, false, true]

Explanation
Logger logger = new Logger();
logger.shouldPrintMessage(1, "foo"); // return true, next allowed timestamp for "foo" is 1 + 10 = 11
logger.shouldPrintMessage(2, "bar"); // return true, next allowed timestamp for "bar" is 2 + 10 = 12
logger.shouldPrintMessage(3, "foo"); // 3 < 11, return false
logger.shouldPrintMessage(8, "bar"); // 8 < 12, return false
logger.shouldPrintMessage(10, "foo"); // 10 < 11, return false
logger.shouldPrintMessage(11, "foo"); // 11 >= 11, return true, next allowed timestamp for "foo" is 11
+ 10 = 21
```

Constraints:

- `0 <= timestamp <= 10^9`
- Every `timestamp` will be passed in non-decreasing order (chronological order).
- `1 <= message.length <= 30`
- At most `10^4` calls will be made to `shouldPrintMessage`.

SOLUTION (PYTHON 3)

Solution 1: Hash Table

> **How to think about it**
>
> The same message prints at most once per 10 seconds; timestamps are nondecreasing. Scanning a log each time is unnecessary. A map stores the next allowed time. Reject if `timestamp` is still before that; otherwise set it to t+10 and print.

We use a hash table ts to store the next available print timestamp for each message. When the `shouldPrintMessage` method is called, we check whether the current timestamp is greater than or equal to the next available print timestamp for the message. If so, we update the next available print timestamp to the current timestamp plus 10 and return `true`; otherwise, we return `false`.

The time complexity is O(1). The space complexity is O(m), where m is the number of distinct messages.

Python 3

```
class Logger:

    def __init__(self):
        self.ts = {}

    def shouldPrintMessage(self, timestamp: int, message: str) -> bool:
        t = self.ts.get(message, 0)
        if t > timestamp:
            return False
        self.ts[message] = timestamp + 10
        return True


# Your Logger object will be instantiated and called as such:
# obj = Logger()
# param_1 = obj.shouldPrintMessage(timestamp,message)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. At most k copies of a message in every window of 10
# seconds
# 2. Forget old messages so memory stays small


# ========================================================
# FOLLOW-UP 1: At most k copies of a message in every
# window of 10 seconds
# ========================================================

# ---- follow-up 1: function ----
# Answer: Keep the times of the last k prints of every
# message.
# Time: O(1) amortised per call.  Space: O(M * k), M =
# distinct messages.
from collections import deque


class LoggerAtMostK:
    def __init__(self, k, window=10):
        self.k = k
        self.window = window
        self.times = {}

    def should_print(self, timestamp, message):
        queue = self.times.setdefault(message, deque())
        while queue and queue[0] <= timestamp - self.window:
            queue.popleft()
        if len(queue) < self.k:
            queue.append(timestamp)
            return True
        return False


# ========================================================
# FOLLOW-UP 2: Forget old messages so memory stays small
# ========================================================

# ---- follow-up 2: function ----
# Answer: Keep a queue in time order; before every call
# pop the messages older than the window.
# Time: O(1) amortised per call.  Space: O(messages in the
# window).
class LoggerWithCleanup:
    def __init__(self, window=10):
```

```
        self.window = window
        self.queue = deque()
        self.last = {}

    def should_print(self, timestamp, message):
        while self.queue and self.queue[0][0] <= timestamp - self.window:
            old_time, old_message = self.queue.popleft()
            if self.last.get(old_message) == old_time:
                del self.last[old_message]
        if message in self.last:
            return False
        self.last[message] = timestamp
        self.queue.append((timestamp, message))
        return True
```

PART 2 · DATA STRUCTURE DESIGN · PROBLEM 2 OF 9

271. Encode and Decode Strings

Medium · Design, Array, String

leetcode.com/problems/encode-and-decode-strings

★ **TOP 7 PROBLEM** in Data Structure Design: master this one first

PROBLEM

Design an algorithm to encode **a list of strings** to **a string**. The encoded string is then sent over the network and is decoded back to the original list of strings.

Machine 1 (sender) has the function:

```
string encode(vector<string> strs) {
  // ... your code
  return encoded_string;
}
```

Machine 2 (receiver) has the function:

```
vector<string> decode(string s) {
  //... your code
  return strs;
}
```

So Machine 1 does:

```
string encoded_string = encode(strs);
```

and Machine 2 does:

```
vector<string> strs2 = decode(encoded_string);
```

`strs2` in Machine 2 should be the same as `strs` in Machine 1.

Implement the `encode` and `decode` methods.

You are not allowed to solve the problem using any serialize methods (such as `eval`).

Example 1:

```
Input: dummy_input = ["Hello","World"]
Output: ["Hello","World"]
Explanation:
Machine 1:
Codec encoder = new Codec();
String msg = encoder.encode(strs);
Machine 1 ---msg---> Machine 2

Machine 2:
Codec decoder = new Codec();
String[] strs = decoder.decode(msg);
```

Example 2:

```
Input: dummy_input = [""]
Output: [""]
```

Constraints:

- `1 <= strs.length <= 200`
- `0 <= strs[i].length <= 200`

- `strs[i]` contains any possible characters out of `256` valid ASCII characters.

Follow up: Could you write a generalized algorithm to work on any possible set of characters?

SOLUTION (PYTHON 3)

Solution 1: Encode String Length

> **How to think about it**
>
> A raw delimiter can appear inside a string. Prefixing a fixed-width length makes the split unambiguous. Encode writes a 4-character length then the payload; decode reads that length and slices.

During encoding, we convert the length of the string into a fixed 4-digit string, add the string itself, and append it to the result string in sequence.

During decoding, we first take the first four digits of the string to get the length, and then cut the following string according to the length. We cut it in sequence until we get the list of strings.

The time complexity is O(n).

Python 3

```
class Codec:
    def encode(self, strs: List[str]) -> str:
        """Encodes a list of strings to a single string."""
        ans = []
        for s in strs:
            ans.append('{:4}'.format(len(s)) + s)
        return ''.join(ans)

    def decode(self, s: str) -> List[str]:
        """Decodes a single string to a list of strings."""
        ans = []
        i, n = 0, len(s)
        while i < n:
            size = int(s[i : i + 4])
            i += 4
            ans.append(s[i : i + size])
            i += size
        return ans


# Your Codec object will be instantiated and called as such:
# codec = Codec()
# codec.decode(codec.encode(strs))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Encode a list of lists of strings
# 2. Use an escape character instead of lengths


# ========================================================
# FOLLOW-UP 1: Encode a list of lists of strings
# ========================================================

# ---- follow-up 1: function ----
# Answer: Encode every inner list into one string with the
# length-prefix scheme, then encode those strings the same
# way.
# Time: O(total length).  Space: O(total length).
def encode_list(strs):
    return "".join(str(len(s)) + "#" + s for s in strs)
```

```
def decode_list(text):
    result = []
    i = 0
    while i < len(text):
        j = text.index("#", i)
        size = int(text[i:j])
        result.append(text[j + 1:j + 1 + size])
        i = j + 1 + size
    return result


def encode_nested(lists):
    return encode_list([encode_list(inner) for inner in lists])


def decode_nested(text):
    return [decode_list(inner) for inner in decode_list(text)]


# ============================================================
# FOLLOW-UP 2: Use an escape character instead of lengths
# ============================================================

# ---- follow-up 2: function ----
# Answer: Escape the separator inside the strings; split
# on unescaped separators.
# Time: O(total length).  Space: O(total length).
def encode_with_escape(strs):
    return "".join(s.replace("\\", "\\\\").replace(",", "\\,") + "," for s in strs)


def decode_with_escape(text):
    result = []
    current = []
    i = 0
    while i < len(text):
        if text[i] == "\\":
            current.append(text[i + 1])
            i += 2
        elif text[i] == ",":
            result.append("".join(current))
            current = []
            i += 1
        else:
            current.append(text[i])
            i += 1
    return result
```

PART 2 · DATA STRUCTURE DESIGN · PROBLEM 3 OF 9

362. Design Hit Counter

Medium · Design, Queue, Array, Binary Search, Data Stream

leetcode.com/problems/design-hit-counter

★ **TOP 7 PROBLEM** in Data Structure Design: master this one first

PROBLEM

Design a hit counter which counts the number of hits received in the past `5` minutes (i.e., the past `300` seconds).

Your system should accept a `timestamp` parameter (**in seconds** granularity), and you may assume that calls are being made to the system in chronological order (i.e., `timestamp` is monotonically increasing). Several hits may arrive roughly at the same time.

Implement the `HitCounter` class:

- `HitCounter()` Initializes the object of the hit counter system.
- `void hit(int timestamp)` Records a hit that happened at `timestamp` (**in seconds**). Several hits may happen at the same `timestamp`.
- `int getHits(int timestamp)` Returns the number of hits in the past 5 minutes from `timestamp` (i.e., the past `300` seconds).

Example 1:

```
Input
["HitCounter", "hit", "hit", "hit", "getHits", "hit", "getHits", "getHits"]
[[], [1], [2], [3], [4], [300], [300], [301]]
Output
[null, null, null, null, 3, null, 4, 3]

Explanation
HitCounter hitCounter = new HitCounter();
hitCounter.hit(1); // hit at timestamp 1.
hitCounter.hit(2); // hit at timestamp 2.
hitCounter.hit(3); // hit at timestamp 3.
hitCounter.getHits(4); // get hits at timestamp 4, return 3.
hitCounter.hit(300); // hit at timestamp 300.
hitCounter.getHits(300); // get hits at timestamp 300, return 4.
hitCounter.getHits(301); // get hits at timestamp 301, return 3.
```

Constraints:

- `1 <= timestamp <= 2 * 10^9`
- All the calls are being made to the system in chronological order (i.e., `timestamp` is monotonically increasing).
- At most `300` calls will be made to `hit` and `getHits`.

Follow up: What if the number of hits per second could be huge? Does your design scale?

SOLUTION (PYTHON 3)

Solution 1: Binary Search

How to think about it

Count hits in the last 300 seconds. Timestamps increase. A queue that drops stale hits works; a list plus binary search is shorter. `hit` appends; `getHits` bisects the first index ≥ timestamp-299 and returns the suffix length.

Since `timestamp` is monotonically increasing, we can use an array `ts` to store all `timestamp`s. Then in the `getHits` method, we use binary search to find the first position that is greater than or equal to `timestamp - 300 + 1`, and then return the length of `ts` minus this position.

In terms of time complexity, the time complexity of the `hit` method is O(1), and the time complexity of the `getHits` method is O(log n). Where n is the length of `ts`.

Python 3

```python
class HitCounter:

    def __init__(self):
        self.ts = []

    def hit(self, timestamp: int) -> None:
        self.ts.append(timestamp)

    def getHits(self, timestamp: int) -> int:
        return len(self.ts) - bisect_left(self.ts, timestamp - 300 + 1)


# Your HitCounter object will be instantiated and called as such:
# obj = HitCounter()
# obj.hit(timestamp)
# param_2 = obj.getHits(timestamp)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Hits in the last t seconds for any t
# 2. Constant memory: 300 buckets (one per second of the
# last five minutes)


# ============================================================
# FOLLOW-UP 1: Hits in the last t seconds for any t
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep all the hit times in order and binary
# search the first one inside the window.
# Time: hit O(1), get_hits O(log n).  Space: O(n).
import bisect


class HitCounterAnyWindow:
    def __init__(self):
        self.times = []

    def hit(self, timestamp):
        self.times.append(timestamp)

    def get_hits(self, timestamp, window):
        return len(self.times) - bisect.bisect_right(self.times, timestamp - window)


# ============================================================
# FOLLOW-UP 2: Constant memory: 300 buckets (one per
# second of the last five minutes)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Bucket i holds the hits of the second whose time
# mod 300 is i; reset a bucket when its second is old.
# Time: hit O(1), get_hits O(300) = O(1).  Space: O(300).
class HitCounterBuckets:
    def __init__(self):
        self.times = [0] * 300
        self.hits = [0] * 300

    def hit(self, timestamp):
```

```
        i = timestamp % 300
        if self.times[i] != timestamp:
            self.times[i] = timestamp
            self.hits[i] = 0
        self.hits[i] += 1

    def get_hits(self, timestamp):
        return sum(h for t, h in zip(self.times, self.hits) if timestamp - t < 300)
```

PART 2 · DATA STRUCTURE DESIGN · PROBLEM 4 OF 9

432. All O`one Data Structure

Hard · Design, Hash Table, Linked List, Doubly-Linked List

leetcode.com/problems/all-oone-data-structure

★ **TOP 7 PROBLEM** in Data Structure Design: master this one first

PROBLEM

Design a data structure to store the strings' count with the ability to return the strings with minimum and maximum counts.

Implement the `AllOne` class:

- `AllOne()` Initializes the object of the data structure.
- `inc(String key)` Increments the count of the string `key` by `1`. If `key` does not exist in the data structure, insert it with count `1`.
- `dec(String key)` Decrements the count of the string `key` by `1`. If the count of `key` is `0` after the decrement, remove it from the data structure. It is guaranteed that `key` exists in the data structure before the decrement.
- `getMaxKey()` Returns one of the keys with the maximal count. If no element exists, return an empty string `""`.
- `getMinKey()` Returns one of the keys with the minimum count. If no element exists, return an empty string `""`.

Note that each function must run in `O(1)` average time complexity.

Example 1:

```
Input
["AllOne", "inc", "inc", "getMaxKey", "getMinKey", "inc", "getMaxKey", "getMinKey"]
[[], ["hello"], ["hello"], [], [], ["leet"], [], []]
Output
[null, null, null, "hello", "hello", null, "hello", "leet"]

Explanation
AllOne allOne = new AllOne();
allOne.inc("hello");
allOne.inc("hello");
allOne.getMaxKey(); // return "hello"
allOne.getMinKey(); // return "hello"
allOne.inc("leet");
allOne.getMaxKey(); // return "hello"
allOne.getMinKey(); // return "leet"
```

Constraints:

- `1 <= key.length <= 10`
- `key` consists of lowercase English letters.
- It is guaranteed that for each call to `dec`, `key` is existing in the data structure.
- At most $5 * 10^4$ calls will be made to `inc`, `dec`, `getMaxKey`, and `getMinKey`.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

inc, dec, getMaxKey and getMinKey all need amortized O(1). A hash map updates a count in O(1) but cannot report the extremes; an ordered set would take logarithmic time. Keys that share a count sit in one doubly linked bucket, and buckets form a ring ordered by count. A map sends each key to its bucket. Increment and decrement move a key only to an adjacent count, inserting or deleting an empty bucket when needed. The ring's successor and predecessor are the min and max counts. Keys never jump over buckets, so the list is never scanned.

Python 3

```python
class Node:
    def __init__(self, key='', cnt=0):
        self.prev = None
        self.next = None
        self.cnt = cnt
        self.keys = {key}

    def insert(self, node):
        node.prev = self
        node.next = self.next
        node.prev.next = node
        node.next.prev = node
        return node

    def remove(self):
        self.prev.next = self.next
        self.next.prev = self.prev


class AllOne:
    def __init__(self):
        self.root = Node()
        self.root.next = self.root
        self.root.prev = self.root
        self.nodes = {}

    def inc(self, key: str) -> None:
        root, nodes = self.root, self.nodes
        if key not in nodes:
            if root.next == root or root.next.cnt > 1:
                nodes[key] = root.insert(Node(key, 1))
            else:
                root.next.keys.add(key)
                nodes[key] = root.next
        else:
            curr = nodes[key]
            next = curr.next
            if next == root or next.cnt > curr.cnt + 1:
                nodes[key] = curr.insert(Node(key, curr.cnt + 1))
            else:
                next.keys.add(key)
                nodes[key] = next
            curr.keys.discard(key)
            if not curr.keys:
                curr.remove()

    def dec(self, key: str) -> None:
        root, nodes = self.root, self.nodes
        curr = nodes[key]
        if curr.cnt == 1:
            nodes.pop(key)
        else:
            prev = curr.prev
            if prev == root or prev.cnt < curr.cnt - 1:
                nodes[key] = prev.insert(Node(key, curr.cnt - 1))
            else:
                prev.keys.add(key)
                nodes[key] = prev
        curr.keys.discard(key)
        if not curr.keys:
```

```python
            curr.remove()

    def getMaxKey(self) -> str:
        return next(iter(self.root.prev.keys))

    def getMinKey(self) -> str:
```

```
        return next(iter(self.root.next.keys))


# Your AllOne object will be instantiated and called as such:
# obj = AllOne()
# obj.inc(key)
# obj.dec(key)
# param_3 = obj.getMaxKey()
# param_4 = obj.getMinKey()
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the k keys with the biggest counts
# 2. Every key with a given count


# ============================================================
# FOLLOW-UP 1: Return the k keys with the biggest counts
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort the keys by count (simple), or keep the
# buckets linked and walk from the biggest.
# Time: inc and dec O(1), get_top_k O(n log n).  Space:
# O(n).
class AllOneTopK:
    def __init__(self):
        self.count = {}

    def inc(self, key):
        self.count[key] = self.count.get(key, 0) + 1

    def dec(self, key):
        self.count[key] -= 1
        if self.count[key] == 0:
            del self.count[key]

    def get_top_k(self, k):
        return sorted(self.count, key=lambda x: (-self.count[x], x))[:k]


# ============================================================
# FOLLOW-UP 2: Every key with a given count
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep a dictionary from count to the set of keys.
# Time: inc and dec O(1), keys_with_count O(m log m).
# Space: O(n).
class AllOneByCount:
    def __init__(self):
        self.count = {}
        self.by_count = {}

    def _move(self, key, old, new):
        if old:
            self.by_count[old].discard(key)
            if not self.by_count[old]:
                del self.by_count[old]
        if new:
            self.by_count.setdefault(new, set()).add(key)

    def inc(self, key):
        old = self.count.get(key, 0)
        self.count[key] = old + 1
```

```
        self._move(key, old, old + 1)

    def dec(self, key):
        old = self.count[key]
        if old == 1:
            del self.count[key]
```

```
        else:
            self.count[key] = old - 1
        self._move(key, old, old - 1)

    def keys_with_count(self, c):
        return sorted(self.by_count.get(c, set()))
```

588. Design In-Memory File System

Hard · Design, Trie, Hash Table, String, Sorting

leetcode.com/problems/design-in-memory-file-system

★ **TOP 7 PROBLEM** in Data Structure Design: master this one first

PROBLEM

Design a data structure that simulates an in-memory file system.

Implement the FileSystem class:

- `FileSystem()` Initializes the object of the system.
- `List<String> ls(String path)` The answer should in **lexicographic order**.
  - If `path` is a file path, returns a list that only contains this file's name.
  - If `path` is a directory path, returns the list of file and directory names **in this directory**.
- `void mkdir(String path)` Makes a new directory according to the given `path`. The given directory path does not exist. If the middle directories in the path do not exist, you should create them as well.
- `void addContentToFile(String filePath, String content)`
  - If `filePath` does not exist, creates that file containing given `content`.
  - If `filePath` already exists, appends the given `content` to original content.
- `String readContentFromFile(String filePath)` Returns the content in the file at `filePath`.

Example 1:

Operation	Output	Explanation
FileSystem fs = new FileSystem()	null	The constructor returns nothing.
fs.ls("/")	[]	Initially, directory `/` has nothing. So return empty list.
fs.mkdir("/a/b/c")	null	Create directory `a` in directory `/`. Then create directory `b` in directory `a`. Finally, create directory `c` in directory `b`.
fs.addContentToFile("/a/b/c/d","hello")	null	Create a file named `d` with content `"hello"` in directory `/a/b/c`.
fs.ls("/")	["a"]	Only directory `a` is in directory `/`.
fs.readContentFromFile("/a/b/c/d")	"hello"	Output the file content.

```
Input
["FileSystem", "ls", "mkdir", "addContentToFile", "ls", "readContentFromFile"]
[[], ["/"], ["/a/b/c"], ["/a/b/c/d", "hello"], ["/"], ["/a/b/c/d"]]
Output
[null, [], null, null, ["a"], "hello"]

Explanation
FileSystem fileSystem = new FileSystem();
fileSystem.ls("/"); // return []
fileSystem.mkdir("/a/b/c");
fileSystem.addContentToFile("/a/b/c/d", "hello");
fileSystem.ls("/"); // return ["a"]
fileSystem.readContentFromFile("/a/b/c/d"); // return "hello"
```

Constraints:

- `1 <= path.length, filePath.length <= 100`
- `path` and `filePath` are absolute paths which begin with `'/'` and do not end with `'/'` except that the path is just `"/"`.
- You can assume that all directory names and file names only contain lowercase letters, and the same names will not exist in the same directory.
- You can assume that all operations will be passed valid parameters, and users will not attempt to retrieve file content or list a directory or file that does not exist.
- You can assume that the parent directory for the file in `addContentToFile` will exist.
- `1 <= content.length <= 50`
- At most `300` calls will be made to `ls`, `mkdir`, `addContentToFile`, and `readContentFromFile`.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

A file system is hierarchical: list, mkdir, append, read. A flat map of full paths works, but sharing prefixes and listing a directory is awkward. A trie keyed by path segments stores children, a file flag, and content chunks. `insert` creates nodes; `search` walks to the target. `ls` returns a file name or the sorted child names. Cost follows path depth.

Python 3

```python
class Trie:
    def __init__(self):
        self.name = None
        self.isFile = False
        self.content = []
        self.children = {}

    def insert(self, path, isFile):
        node = self
        ps = path.split('/')
        for p in ps[1:]:
            if p not in node.children:
                node.children[p] = Trie()
            node = node.children[p]
        node.isFile = isFile
        if isFile:
            node.name = ps[-1]
        return node

    def search(self, path):
        node = self
        if path == '/':
            return node
        ps = path.split('/')
        for p in ps[1:]:
            if p not in node.children:
                return None
            node = node.children[p]
        return node


class FileSystem:
    def __init__(self):
        self.root = Trie()

    def ls(self, path: str) -> List[str]:
        node = self.root.search(path)
        if node is None:
            return []
        if node.isFile:
            return [node.name]
        return sorted(node.children.keys())

    def mkdir(self, path: str) -> None:
        self.root.insert(path, False)

    def addContentToFile(self, filePath: str, content: str) -> None:
        node = self.root.insert(filePath, True)
        node.content.append(content)

    def readContentFromFile(self, filePath: str) -> str:
        node = self.root.search(filePath)
        return ''.join(node.content)


# Your FileSystem object will be instantiated and called as such:
# obj = FileSystem()
# param_1 = obj.ls(path)
# obj.mkdir(path)
# obj.addContentToFile(filePath,content)
```

```python
# param_4 = obj.readContentFromFile(filePath)
```

Complexity. Time: O(L) per path operation (L = path length); ls adds O(k log k) for k entries. Space: O(total length of all paths and contents).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Delete a file or a whole folder
# 2. Find every file with a given name


# ============================================================
# FOLLOW-UP 1: Delete a file or a whole folder
# ============================================================

# ---- follow-up 1: function ----
# Answer: Walk to the parent folder and remove the name.
# Time: each operation O(path length), ls adds O(k log k).
# Space: O(total content).
class FileSystemWithDelete:
    def __init__(self):
        self.root = {}

    def _walk(self, path, create=False):
        node = self.root
        for part in [p for p in path.split("/") if p]:
            if part not in node:
                if not create:
                    return None
                node[part] = {}
            node = node[part]
        return node

    def mkdir(self, path):
        self._walk(path, True)

    def add_content_to_file(self, path, content):
        parent = self._walk(path.rsplit("/", 1)[0] or "/", True)
        name = path.rsplit("/", 1)[1]
        if not isinstance(parent.get(name), str):
            parent[name] = ""
        parent[name] += content

    def read_content_from_file(self, path):
        parent = self._walk(path.rsplit("/", 1)[0] or "/")
        return parent[path.rsplit("/", 1)[1]]

    def ls(self, path):
        parent = self._walk(path.rsplit("/", 1)[0] or "/")
        name = path.rsplit("/", 1)[1]
        if name and isinstance(parent.get(name), str):
            return [name]
        node = self._walk(path)
        return sorted(node.keys())

    def rm(self, path):
        parent = self._walk(path.rsplit("/", 1)[0] or "/")
        name = path.rsplit("/", 1)[1]
        if parent and name in parent:
            del parent[name]
            return True
        return False


# ============================================================
# FOLLOW-UP 2: Find every file with a given name
```

```
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk the whole tree and collect the paths of
```

```
# files with that name.
# Time: O(n + f log f), f = matches.  Space: O(depth + f).
def find_files(fs, name):
    result = []

    def go(node, path):
        for key, child in node.items():
            if isinstance(child, str):
                if key == name:
                    result.append(path + "/" + key)
            else:
                go(child, path + "/" + key)

    go(fs.root, "")
    return sorted(result)
```

642. Design Search Autocomplete System

Hard · Depth-First Search, Design, Trie, String, Data Stream, Sorting, Heap (Priority Queue)

leetcode.com/problems/design-search-autocomplete-system

★ **TOP 7 PROBLEM** in Data Structure Design: master this one first

PROBLEM

Design a search autocomplete system for a search engine. Users may input a sentence (at least one word and end with a special character `'#'`).

You are given a string array `sentences` and an integer array `times` both of length `n` where `sentences[i]` is a previously typed sentence and `times[i]` is the corresponding number of times the sentence was typed. For each input character except `'#'`, return the top `3` historical hot sentences that have the same prefix as the part of the sentence already typed.

Here are the specific rules:

- The hot degree for a sentence is defined as the number of times a user typed the exactly same sentence before.
- The returned top `3` hot sentences should be sorted by hot degree (The first is the hottest one). If several sentences have the same hot degree, use ASCII-code order (smaller one appears first).
- If less than `3` hot sentences exist, return as many as you can.
- When the input is a special character, it means the sentence ends, and in this case, you need to return an empty list.

Implement the `AutocompleteSystem` class:

- `AutocompleteSystem(String[] sentences, int[] times)` Initializes the object with the `sentences` and `times` arrays.
- `List<String> input(char c)` This indicates that the user typed the character `c`.
  - Returns an empty array `[]` if `c == '#'` and stores the inputted sentence in the system.
  - Returns the top `3` historical hot sentences that have the same prefix as the part of the sentence already typed. If there are fewer than `3` matches, return them all.

Example 1:

```
Input
["AutocompleteSystem", "input", "input", "input", "input"]
[[["i love you", "island", "iroman", "i love leetcode"], [5, 3, 2, 2]], ["i"], [" "], ["a"], ["#"]]
Output
[null, ["i love you", "island", "i love leetcode"], ["i love you", "i love leetcode"], [], []]

Explanation
AutocompleteSystem obj = new AutocompleteSystem(["i love you", "island", "iroman", "i love leetcode"],
[5, 3, 2, 2]);
obj.input("i"); // return ["i love you", "island", "i love leetcode"]. There are four sentences that
have prefix "i". Among them, "ironman" and "i love leetcode" have same hot degree. Since ' ' has ASCII
code 32 and 'r' has ASCII code 114, "i love leetcode" should be in front of "ironman". Also we only
need to output top 3 hot sentences, so "ironman" will be ignored.
obj.input(" "); // return ["i love you", "i love leetcode"]. There are only two sentences that have
prefix "i ".
obj.input("a"); // return []. There are no sentences that have prefix "i a".
obj.input("#"); // return []. The user finished the input, the sentence "i a" should be saved as a
historical sentence in system. And the following input will be counted as a new search.
```

Constraints:

- `n == sentences.length`
- `n == times.length`
- `1 <= n <= 100`

- `1 <= sentences[i].length <= 100`
- `1 <= times[i] <= 50`
- `c` is a lowercase English letter, a hash `'#'`, or space `' '`.
- Each tested sentence will be a sequence of characters `c` that end with the character `'#'`.
- Each tested sentence will have a length in the range `[1, 200]`.
- The words in each input sentence are separated by single spaces.
- At most `5000` calls will be made to `input`.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Each typed character must yield the three hottest sentences with that prefix. Scanning every sentence on every keystroke is slow. A trie stores sentence and heat. After walking the current prefix, DFS the subtree, sort by (-heat, lex), and keep three. `#` inserts the finished sentence back.

Python 3

```python
class Trie:
    def __init__(self):
        self.children = [None] * 27
        self.v = 0
        self.w = ''

    def insert(self, w, t):
        node = self
        for c in w:
            idx = 26 if c == ' ' else ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.v += t
        node.w = w

    def search(self, pref):
        node = self
        for c in pref:
            idx = 26 if c == ' ' else ord(c) - ord('a')
            if node.children[idx] is None:
                return None
            node = node.children[idx]
        return node


class AutocompleteSystem:
    def __init__(self, sentences: List[str], times: List[int]):
        self.trie = Trie()
        for a, b in zip(sentences, times):
            self.trie.insert(a, b)
        self.t = []

    def input(self, c: str) -> List[str]:
        def dfs(node):
            if node is None:
                return
            if node.v:
                res.append((node.v, node.w))
            for nxt in node.children:
                dfs(nxt)

        if c == '#':
            s = ''.join(self.t)
            self.trie.insert(s, 1)
            self.t = []
            return []

        res = []
        self.t.append(c)
        node = self.trie.search(''.join(self.t))
        if node is None:
            return res
        dfs(node)
        res.sort(key=lambda x: (-x[0], x[1]))
        return [v[1] for v in res[:3]]


# Your AutocompleteSystem object will be instantiated and called as such:
# obj = AutocompleteSystem(sentences, times)
```

```python
# param_1 = obj.input(c)
```

Complexity. Time: constructor O(total letters); input O(p + S log S), p = typed length, S = sentences under the prefix. Space: O(27 * total letters).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the top k suggestions (instead of 3)
# 2. Ignore upper and lower case when matching


# ============================================================
# FOLLOW-UP 1: Return the top k suggestions (instead of 3)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep a trie with a count of every sentence at
# the end; collect the sentences under the typed prefix,
# sort by (-count, text), take k.
# Time: O(S * L + S log S) per typed letter.  Space: O(S *
# L).
class AutocompleteTopK:
    def __init__(self, sentences, times, k=3):
        self.k = k
        self.counts = {}
        self.typed = ""
        for s, t in zip(sentences, times):
            self.counts[s] = self.counts.get(s, 0) + t

    def input(self, c):
        if c == "#":
            self.counts[self.typed] = self.counts.get(self.typed, 0) + 1
            self.typed = ""
            return []
        self.typed += c
        matches = [s for s in self.counts if s.startswith(self.typed)]
        matches.sort(key=lambda s: (-self.counts[s], s))
        return matches[:self.k]


# ============================================================
# FOLLOW-UP 2: Ignore upper and lower case when matching
# ============================================================

# ---- follow-up 2: function ----
# Answer: Compare in lower case but return the original
# sentence.
# Time: O(S * L + S log S) per typed letter.  Space: O(S *
# L).
class AutocompleteIgnoreCase:
    def __init__(self, sentences, times, k=3):
        self.k = k
        self.counts = {}
        self.typed = ""
        for s, t in zip(sentences, times):
            self.counts[s] = self.counts.get(s, 0) + t

    def input(self, c):
        if c == "#":
            self.counts[self.typed] = self.counts.get(self.typed, 0) + 1
            self.typed = ""
            return []
        self.typed += c
        low = self.typed.lower()
        matches = [s for s in self.counts if s.lower().startswith(low)]
        matches.sort(key=lambda s: (-self.counts[s], s))
```

```
        return matches[:self.k]
```

716. Max Stack

Hard · Stack, Design, Linked List, Doubly-Linked List, Ordered Set

leetcode.com/problems/max-stack

PROBLEM

Design a max stack data structure that supports the stack operations and supports finding the stack's maximum element.

Implement the `MaxStack` class:

- `MaxStack()` Initializes the stack object.
- `void push(int x)` Pushes element `x` onto the stack.
- `int pop()` Removes the element on top of the stack and returns it.
- `int top()` Gets the element on the top of the stack without removing it.
- `int peekMax()` Retrieves the maximum element in the stack without removing it.
- `int popMax()` Retrieves the maximum element in the stack and removes it. If there is more than one maximum element, only remove the **top-most** one.

You must come up with a solution that supports `O(1)` for each `top` call and `O(logn)` for each other call.

Example 1:

```
Input
["MaxStack", "push", "push", "push", "top", "popMax", "top", "peekMax", "pop", "top"]
[[], [5], [1], [5], [], [], [], [], [], []]
Output
[null, null, null, null, 5, 5, 1, 5, 1, 5]

Explanation
MaxStack stk = new MaxStack();
stk.push(5); // [5] the top of the stack and the maximum number is 5.
stk.push(1); // [5, 1] the top of the stack is 1, but the maximum is 5.
stk.push(5); // [5, 1, 5] the top of the stack is 5, which is also the maximum, because it is the top most one.
stk.top(); // return 5, [5, 1, 5] the stack did not change.
stk.popMax(); // return 5, [5, 1] the stack is changed now, and the top is different from the max.
stk.top(); // return 1, [5, 1] the stack did not change.
stk.peekMax(); // return 5, [5, 1] the stack did not change.
stk.pop(); // return 1, [5] the top of the stack and the max element is now 5.
stk.top(); // return 5, [5] the stack did not change.
```

Constraints:

- -10^7 <= x <= 10^7
- At most 10^5 calls will be made to `push`, `pop`, `top`, `peekMax`, and `popMax`.
- There will be **at least one element** in the stack when `pop`, `top`, `peekMax`, or `popMax` is called.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Besides a normal stack we must peek the maximum and pop the most recent occurrence of it. One stack cannot delete in the middle; dual stacks find the max but cannot unlink an arbitrary node. A doubly linked list preserves push order and unlinks in O(1). An ordered set of those nodes, keyed by value, finds the current max in logarithmic time. Both structures share nodes, so every delete updates both. push appends and inserts; pop removes the tail; popMax takes the last ordered node and unlinks it. Writes are O(log n).

Python 3

```
class Node:
    def __init__(self, val=0):
        self.val = val
        self.prev: Union[Node, None] = None
        self.next: Union[Node, None] = None


class DoubleLinkedList:
    def __init__(self):
        self.head = Node()
        self.tail = Node()
        self.head.next = self.tail
        self.tail.prev = self.head

    def append(self, val) -> Node:
        node = Node(val)
        node.next = self.tail
        node.prev = self.tail.prev
        self.tail.prev = node
        node.prev.next = node
        return node

    @staticmethod
    def remove(node) -> Node:
        node.prev.next = node.next
        node.next.prev = node.prev
        return node

    def pop(self) -> Node:
        return self.remove(self.tail.prev)

    def peek(self):
        return self.tail.prev.val


class MaxStack:
    def __init__(self):
        self.stk = DoubleLinkedList()
        self.sl = SortedList(key=lambda x: x.val)

    def push(self, x: int) -> None:
        node = self.stk.append(x)
        self.sl.add(node)

    def pop(self) -> int:
        node = self.stk.pop()
        self.sl.remove(node)
        return node.val

    def top(self) -> int:
        return self.stk.peek()

    def peekMax(self) -> int:
        return self.sl[-1].val

    def popMax(self) -> int:
        node = self.sl.pop()
        DoubleLinkedList.remove(node)
        return node.val
```

```
# Your MaxStack object will be instantiated and called as such:
# obj = MaxStack()
# obj.push(x)
# param_2 = obj.pop()
# param_3 = obj.top()
# param_4 = obj.peekMax()
```

```
# param_5 = obj.popMax()
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. peekMax and popMax in O(log n) (heap with lazy
# deletion)
# 2. A stack that also gives the minimum in O(1)


# ============================================================
# FOLLOW-UP 1: peekMax and popMax in O(log n) (heap with
# lazy deletion)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Give every pushed item an id; keep a stack and a
# max-heap of (value, id); mark ids removed and skip them
# when they surface.
# Time: push O(log n), pop, top, peek_max and pop_max
# O(log n) amortised.  Space: O(n).
import heapq


class MaxStackHeap:
    def __init__(self):
        self.stack = []
        self.heap = []
        self.removed = set()
        self.next_id = 0

    def push(self, x):
        self.stack.append((x, self.next_id))
        heapq.heappush(self.heap, (-x, -self.next_id))
        self.next_id += 1

    def _clean_stack(self):
        while self.stack and self.stack[-1][1] in self.removed:
            self.stack.pop()

    def _clean_heap(self):
        while self.heap and -self.heap[0][1] in self.removed:
            heapq.heappop(self.heap)

    def pop(self):
        self._clean_stack()
        x, i = self.stack.pop()
        self.removed.add(i)
        return x

    def top(self):
        self._clean_stack()
        return self.stack[-1][0]

    def peek_max(self):
        self._clean_heap()
        return -self.heap[0][0]

    def pop_max(self):
        self._clean_heap()
        x, i = heapq.heappop(self.heap)
        self.removed.add(-i)
        return -x
```

```
# ============================================================
```

```
# FOLLOW-UP 2: A stack that also gives the minimum in O(1)
# ========================================================

# ---- follow-up 2: function ----
# Answer: Store the smallest value so far next to every
# element.
# Time: O(1) for every operation.  Space: O(n).
class MinMaxStack:
    def __init__(self):
        self.items = []

    def push(self, x):
        low = min(x, self.items[-1][1]) if self.items else x
        high = max(x, self.items[-1][2]) if self.items else x
        self.items.append((x, low, high))

    def pop(self):
        return self.items.pop()[0]

    def top(self):
        return self.items[-1][0]

    def get_min(self):
        return self.items[-1][1]

    def get_max(self):
        return self.items[-1][2]
```

PART 2 · DATA STRUCTURE DESIGN · PROBLEM 8 OF 9

895. Maximum Frequency Stack

Hard · Stack, Design, Hash Table, Ordered Set

leetcode.com/problems/maximum-frequency-stack

★ **TOP 7 PROBLEM** in Data Structure Design: master this one first

PROBLEM

Design a stack-like data structure to push elements to the stack and pop the most frequent element from the stack.

Implement the `FreqStack` class:

- `FreqStack()` constructs an empty frequency stack.
- `void push(int val)` pushes an integer `val` onto the top of the stack.
- `int pop()` removes and returns the most frequent element in the stack.
  - If there is a tie for the most frequent element, the element closest to the stack's top is removed and returned.

Example 1:

```
Input
["FreqStack", "push", "push", "push", "push", "push", "push", "pop", "pop", "pop", "pop"]
[[], [5], [7], [5], [7], [4], [5], [], [], [], []]
Output
[null, null, null, null, null, null, null, 5, 7, 5, 4]

Explanation
FreqStack freqStack = new FreqStack();
freqStack.push(5); // The stack is [5]
freqStack.push(7); // The stack is [5,7]
freqStack.push(5); // The stack is [5,7,5]
freqStack.push(7); // The stack is [5,7,5,7]
freqStack.push(4); // The stack is [5,7,5,7,4]
freqStack.push(5); // The stack is [5,7,5,7,4,5]
freqStack.pop(); // return 5, as 5 is the most frequent. The stack becomes [5,7,5,7,4].
freqStack.pop(); // return 7, as 5 and 7 is the most frequent, but 7 is closest to the top. The stack
becomes [5,7,5,4].
freqStack.pop(); // return 5, as 5 is the most frequent. The stack becomes [5,7,4].
freqStack.pop(); // return 4, as 4, 5 and 7 is the most frequent, but 4 is closest to the top. The
stack becomes [5,7].
```

Constraints:

- $0 <= val <= 10^9$
- At most $2 * 10^4$ calls will be made to `push` and `pop`.
- It is guaranteed that there will be at least one element in the stack before calling `pop`.

SOLUTIONS (PYTHON 3)

Solution 1: Hash Table + Priority Queue (Max Heap)

How to think about it

Pop the current most frequent value, breaking ties by recency. Up to $2 \cdot 10^4$ operations, so scanning the stack is too slow. We need both frequency and a timestamp. A map stores frequencies; a heap stores (-freq,-ts,val). After popping, decrement that value's frequency. Stale heap entries are left behind and will not be chosen while a newer, higher frequency exists.

According to the problem description, we need to design a data structure that supports popping out the element with the highest frequency. If multiple elements have the same frequency, the element closest to the top of the stack should be

popped out.

We can use a hash table cnt to record the frequency of each element, and a priority queue (max heap) q to maintain the frequency of elements and their corresponding timestamps.

When performing a push operation, we first increment the current timestamp, i.e., ts ← ts + 1; then we increment the frequency of the element val, i.e., cnt[val] ← cnt[val] + 1, and finally, we add the triplet (cnt[val], ts, val) to the priority queue q. The time complexity of the push operation is O(log n).

When performing a pop operation, we directly pop an element from the priority queue q. Since the elements in the priority queue q are sorted in descending order of frequency, the popped element is definitely the one with the highest frequency. If multiple elements have the same frequency, the element closest to the top of the stack is popped out, i.e., the element with the largest timestamp is popped out. After popping, we decrement the frequency of the popped element, i.e., cnt[val] ← cnt[val] - 1. The time complexity of the pop operation is O(log n).

Python 3

```python
class FreqStack:
    def __init__(self):
        self.cnt = defaultdict(int)
        self.q = []
        self.ts = 0

    def push(self, val: int) -> None:
        self.ts += 1
        self.cnt[val] += 1
        heappush(self.q, (-self.cnt[val], -self.ts, val))

    def pop(self) -> int:
        val = heappop(self.q)[2]
        self.cnt[val] -= 1
        return val


# Your FreqStack object will be instantiated and called as such:
# obj = FreqStack()
# obj.push(val)
# param_2 = obj.pop()
```

Solution 2: Double Hash Tables

> **How to think about it**
>
> The heap retains stale frequency records. Bucketing by frequency is cleaner: level f is a stack of values whose current frequency is f. Push appends to level f+1 and updates the maximum frequency; pop takes the top of the highest level and decreases the maximum when that level empties. Each operation is amortized O(1).

In Solution 1, in order to pop out the required element, we maintained a priority queue and had to operate on it each time, which has a time complexity of O(log n). If we can find the required element in O(1) time, then the time complexity of each operation of the entire data structure can be reduced to O(1).

In fact, we can use a variable mx to record the current maximum frequency, a hash table d to record the list of elements corresponding to each frequency, and as in Solution 1, a hash table cnt to record the frequency of each element.

When performing a push operation, we increment the frequency of the element, i.e., cnt[val] ← cnt[val] + 1, and then add the element val to the corresponding frequency list in the hash table d, i.e., d[cnt[val]].push(val). If the current element's frequency is greater than mx, then update mx, i.e., mx ← cnt[val]. The time complexity of the push operation is O(1).

When performing a pop operation, we take the list of elements with frequency mx from the hash table d, pop out the last element val in the list, and then remove val from the hash table d, i.e., d[mx].pop(). Finally, we decrement the frequency of val, i.e., cnt[val] ← cnt[val] - 1. If the list d[mx] is empty, it means that all elements with the current maximum frequency have been popped out, and we need to decrement mx, i.e., mx ← mx - 1. The time complexity of the pop operation is O(1).

Python 3

```python
class FreqStack:
    def __init__(self):
        self.cnt = defaultdict(int)
        self.d = defaultdict(list)
        self.mx = 0

    def push(self, val: int) -> None:
        self.cnt[val] += 1
        self.d[self.cnt[val]].append(val)
        self.mx = max(self.mx, self.cnt[val])

    def pop(self) -> int:
        val = self.d[self.mx].pop()
        self.cnt[val] -= 1
        if not self.d[self.mx]:
            self.mx -= 1
        return val


# Your FreqStack object will be instantiated and called as such:
# obj = FreqStack()
# obj.push(val)
# param_2 = obj.pop()
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Pop the LEAST frequent element (the most recent one
# among ties)
# 2. Ask how many times an element is in the stack


# ============================================================
# FOLLOW-UP 1: Pop the LEAST frequent element (the most
# recent one among ties)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the pushes in a list and the count of every
# value; pop the latest push whose value has the smallest
# count.
# Time: push O(1), pop O(n).  Space: O(n).
class LeastFrequencyStack:
    def __init__(self):
        self.items = []
        self.count = {}

    def push(self, x):
        self.items.append(x)
        self.count[x] = self.count.get(x, 0) + 1

    def pop(self):
        low = min(self.count.values())
        for i in range(len(self.items) - 1, -1, -1):
            if self.count[self.items[i]] == low:
                x = self.items.pop(i)
                self.count[x] -= 1
                if self.count[x] == 0:
                    del self.count[x]
                return x


# ============================================================
# FOLLOW-UP 2: Ask how many times an element is in the
# stack
```

```
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the count dictionary and read it.
# Time: push, pop and occurrences are each O(1).  Space:
# O(n).
class FreqStackWithCount:
    def __init__(self):
        self.count = {}
        self.groups = {}
        self.most = 0

    def push(self, x):
        c = self.count.get(x, 0) + 1
        self.count[x] = c
        self.most = max(self.most, c)
        self.groups.setdefault(c, []).append(x)

    def pop(self):
        x = self.groups[self.most].pop()
        if not self.groups[self.most]:
            self.most -= 1
        self.count[x] -= 1
        return x

    def occurrences(self, x):
        return self.count.get(x, 0)
```

PART 2 · DATA STRUCTURE DESIGN · PROBLEM 9 OF 9

2296. Design a Text Editor

Hard · Stack, Design, Linked List, String, Doubly-Linked List, Simulation

leetcode.com/problems/design-a-text-editor

★ **TOP 7 PROBLEM** in Data Structure Design: master this one first

PROBLEM

Design a text editor with a cursor that can do the following:

- **Add** text to where the cursor is.
- **Delete** text from where the cursor is (simulating the backspace key).
- **Move** the cursor either left or right.

When deleting text, only characters to the left of the cursor will be deleted. The cursor will also remain within the actual text and cannot be moved beyond it. More formally, we have that `0 <= cursor.position <= currentText.length` always holds.

Implement the `TextEditor` class:

- `TextEditor()` Initializes the object with empty text.
- `void addText(string text)` Appends `text` to where the cursor is. The cursor ends to the right of `text`.
- `int deleteText(int k)` Deletes `k` characters to the left of the cursor. Returns the number of characters actually deleted.
- `string cursorLeft(int k)` Moves the cursor to the left `k` times. Returns the last `min(10, len)` characters to the left of the cursor, where `len` is the number of characters to the left of the cursor.
- `string cursorRight(int k)` Moves the cursor to the right `k` times. Returns the last `min(10, len)` characters to the left of the cursor, where `len` is the number of characters to the left of the cursor.

Example 1:

```
Input
["TextEditor", "addText", "deleteText", "addText", "cursorRight", "cursorLeft", "deleteText",
"cursorLeft", "cursorRight"]
[[], ["leetcode"], [4], ["practice"], [3], [8], [10], [2], [6]]
Output
[null, null, 4, null, "etpractice", "leet", 4, "", "practi"]

Explanation
TextEditor textEditor = new TextEditor(); // The current text is "|". (The '|' character represents
the cursor)
textEditor.addText("leetcode"); // The current text is "leetcode|".
textEditor.deleteText(4); // return 4
                          // The current text is "leet|".
                          // 4 characters were deleted.
textEditor.addText("practice"); // The current text is "leetpractice|".
textEditor.cursorRight(3); // return "etpractice"
                           // The current text is "leetpractice|".
                           // The cursor cannot be moved beyond the actual text and thus did not move.
                           // "etpractice" is the last 10 characters to the left of the cursor.
textEditor.cursorLeft(8); // return "leet"
                          // The current text is "leet|practice".
                          // "leet" is the last min(10, 4) = 4 characters to the left of the cursor.
textEditor.deleteText(10); // return 4
                           // The current text is "|practice".
                           // Only 4 characters were deleted.
textEditor.cursorLeft(2); // return ""
                          // The current text is "|practice".
                          // The cursor cannot be moved beyond the actual text and thus did not move.
                          // "" is the last min(10, 0) = 0 characters to the left of the cursor.
textEditor.cursorRight(6); // return "practi"
                           // The current text is "practi|ce".
                           // "practi" is the last min(10, 6) = 6 characters to the left of the
cursor.
```

Constraints:

- `1 <= text.length, k <= 40`
- `text` consists of lowercase English letters.
- At most `2 * 10^4` calls **in total** will be made to `addText`, `deleteText`, `cursorLeft` and `cursorRight`.

Follow-up: Could you find a solution with time complexity of `O(k)` per call?

SOLUTION (PYTHON 3)

Solution 1: Left and Right Stacks

How to think about it

The editor inserts, deletes, and moves a cursor; there are 2× 10^4 calls and each text is short. Concatenating a whole string on every move would copy too much. The cursor splits the text into two stacks with amortized O(1) edits at the split. left is left of the cursor, right is right of it (tops face the cursor). Insert and delete touch only left; moves pour up to k characters between the stacks. The last ten characters of left are returned.

We can use two stacks, left and right, where the stack left stores the characters to the left of the cursor, and the stack right stores the characters to the right of the cursor.

- When calling the addText method, we push the characters in text onto the left stack one by one. The time complexity is O(|text|).
- When calling the deleteText method, we pop characters from the left stack up to k times. The time complexity is O(k).
- When calling the cursorLeft method, we pop characters from the left stack up to k times, then push the popped characters onto the right stack one by one, and finally return up to 10 characters from the left stack. The time complexity

is O(k).

- When calling the cursorRight method, we pop characters from the right stack up to k times, then push the popped characters onto the left stack one by one, and finally return up to 10 characters from the left stack. The time complexity is O(k).

Python 3

```
class TextEditor:
    def __init__(self):
        self.left = []
        self.right = []

    def addText(self, text: str) -> None:
        self.left.extend(list(text))

    def deleteText(self, k: int) -> int:
        k = min(k, len(self.left))
        for _ in range(k):
            self.left.pop()
        return k

    def cursorLeft(self, k: int) -> str:
        k = min(k, len(self.left))
        for _ in range(k):
            self.right.append(self.left.pop())
        return ''.join(self.left[-10:])

    def cursorRight(self, k: int) -> str:
        k = min(k, len(self.right))
        for _ in range(k):
            self.left.append(self.right.pop())
        return ''.join(self.left[-10:])


# Your TextEditor object will be instantiated and called as such:
# obj = TextEditor()
# obj.addText(text)
# param_2 = obj.deleteText(k)
# param_3 = obj.cursorLeft(k)
# param_4 = obj.cursorRight(k)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Undo the last editing action
# 2. Move the cursor left and right and report the 10
# characters before it


# ============================================================
# FOLLOW-UP 1: Undo the last editing action
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep a list of actions; undoing re-applies the
# opposite action.
# Time: add O(len), delete O(k), undo O(len or k),
# text_before_cursor O(10).  Space: O(n).
class TextEditorWithUndo:
    def __init__(self):
        self.left = []
        self.right = []
        self.history = []

    def add_text(self, text):
        self.left.extend(text)
```

```
        self.history.append(("add", len(text)))

    def delete_text(self, k):
        removed = []
        while k and self.left:
            removed.append(self.left.pop())
            k -= 1
        self.history.append(("delete", removed))
        return len(removed)

    def undo(self):
        if not self.history:
            return
        kind, data = self.history.pop()
        if kind == "add":
            for _ in range(data):
                self.left.pop()
        else:
            self.left.extend(reversed(data))

    def text_before_cursor(self):
        return "".join(self.left[-10:])


# ============================================================
# FOLLOW-UP 2: Move the cursor left and right and report
# the 10 characters before it
# ============================================================

# ---- follow-up 2: function ----
# Answer: Two stacks: the characters left of the cursor
# and those on its right.
# Time: each call O(k).  Space: O(n).
class TextEditorCursor:
    def __init__(self):
        self.left = []
        self.right = []
```

```
    def add_text(self, text):
        self.left.extend(text)

    def delete_text(self, k):
        n = min(k, len(self.left))
        del self.left[len(self.left) - n:]
        return n

    def cursor_left(self, k):
        while k and self.left:
            self.right.append(self.left.pop())
            k -= 1
        return "".join(self.left[-10:])

    def cursor_right(self, k):
        while k and self.right:
            self.left.append(self.right.pop())
            k -= 1
        return "".join(self.left[-10:])
```

PART 2 · TOPIC 30 OF 50

Greedy

8 problems · 2 Easy · 4 Medium · 2 Hard

THE IDEA

Make the locally best choice and prove it never hurts later. Sorting by the right key often makes the greedy choice obvious.

THE 8 PROBLEMS, EASY TO HARD | ★ = TOP 6: DO THESE FIRST

1	Assign Cookies	Easy	LC 455	2454
2	Apple Redistribution into Boxes	Easy	LC 3074	2457
3	★ **Most Profit Assigning Work**	Medium	LC 826	2459
4	★ **Avoid Flood in The City**	Medium	LC 1488	2462
5	★ **Maximum Score From Removing Substrings**	Medium	LC 1717	2465
6	★ **Jump Game VII**	Medium	LC 1871	2468
7	★ **Set Intersection Size At Least Two**	Hard	LC 757	2471
8	★ **Minimum Cost to Hire K Workers**	Hard	LC 857	2474

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Jump Game (Part 1, pattern 13, page 689)

Minimum Add to Make Parentheses Valid (Part 1, pattern 8, page 426)

Task Scheduler (Part 1, pattern 19, page 1052)

Queue Reconstruction by Height (Part 1, pattern 21, page 1164)

Candy (Part 1, pattern 21, page 1166)

Jump Game II (Part 1, pattern 21, page 1153)

Gas Station (Part 1, pattern 21, page 1156)

Maximum Swap (Part 1, pattern 33, page 1781)

Maximum Matrix Sum (Part 1, pattern 36, page 1890)

Matrix Block Sum — substitute: Minimum Number of Refueling Stops (Part 1, pattern 28, page 1531)

PART 2 · GREEDY · PROBLEM 1 OF 8

455. Assign Cookies

Easy · Greedy, Array, Two Pointers, Sorting, Quick Sort

leetcode.com/problems/assign-cookies

PROBLEM

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child `i` has a greed factor `g[i]`, which is the minimum size of a cookie that the child will be content with; and each cookie `j` has a size `s[j]`. If `s[j] >= g[i]`, we can assign the cookie `j` to the child `i`, and the child `i` will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

```
Input: g = [1,2,3], s = [1,1]
Output: 1
Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.
And even though you have 2 cookies, since their size is both 1, you could only make the child whose
greed factor is 1 content.
You need to output 1.
```

Example 2:

```
Input: g = [1,2], s = [1,2,3]
Output: 2
Explanation: You have 2 children and 3 cookies. The greed factors of 2 children are 1, 2.
You have 3 cookies and their sizes are big enough to gratify all of the children,
You need to output 2.
```

Constraints:

- `1 <= g.length <= 3 * 10^4`
- `0 <= s.length <= 3 * 10^4`
- `1 <= g[i], s[j] <= 2^31 - 1`

Note: This question is the same as 2410: Maximum Matching of Players With Trainers.

SOLUTION (PYTHON 3)

Solution 1: Sorting + Two Pointers

> **How to think about it**
>
> Each cookie feeds at most one child; we want as many content children as possible. Small cookies should go to small appetites, otherwise a large cookie is wasted. Sort both arrays. A cookie pointer skips sizes that are too small and advances together with the child when the size is enough. Exhausting the cookies returns how many children were fed. After sorting, the two pointers always try the current cookie on the easiest remaining child.

According to the problem description, we should prioritize giving cookies to children with smaller appetites, so as to satisfy as many children as possible.

Therefore, we first sort the two arrays, and then use two pointers i and j to point to the head of arrays g and s respectively. Each time we compare the size of g[i] and s[j]:

- If s[j] < g[i], it means that the current cookie s[j] cannot satisfy the current child g[i]. We should allocate a larger cookie to the current child, so j should move to the right by one. If j goes out of bounds, it means that the current child cannot be satisfied. At this time, the number of successfully allocated children is i, and we can return directly.

- If s[j] ≥ g[i], it means that the current cookie s[j] can satisfy the current child g[i]. We allocate the current cookie to the current child, so both i and j should move to the right by one.

If we have traversed the array g, it means that all children have been allocated cookies, and we can return the total number of children.

The time complexity is O(m × log m + n × log n), and the space complexity is O(log m + log n). Where m and n are the lengths of arrays g and s respectively.

Python 3

```
class Solution:
    def findContentChildren(self, g: List[int], s: List[int]) -> int:
        g.sort()
        s.sort()
        j = 0
        for i, x in enumerate(g):
            while j < len(s) and s[j] < g[i]:
                j += 1
            if j >= len(s):
                return i
            j += 1
        return len(g)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the pairs (child, cookie) of the best
# assignment
# 2. Every cookie can be split into two equal halves, each
# given to a different child


# ============================================================
# FOLLOW-UP 1: Return the pairs (child, cookie) of the
# best assignment
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort both, give each child the smallest cookie
# that satisfies the child.
# Time: O(n log n + m log m).  Space: O(n + m).
def assign_pairs(g, s):
    children = sorted(range(len(g)), key=lambda i: g[i])
    cookies = sorted(range(len(s)), key=lambda j: s[j])
    pairs = []
    j = 0
    for i in children:
        while j < len(cookies) and s[cookies[j]] < g[i]:
            j += 1
        if j == len(cookies):
            break
        pairs.append((i, cookies[j]))
        j += 1
    return pairs


# ============================================================
# FOLLOW-UP 2: Every cookie can be split into two equal
# halves, each given to a different child
# ============================================================

# ---- follow-up 2: function ----
# Answer: Treat every cookie as two cookies of half the
# size, then run the usual greedy.
# Time: O((n + m) log (n + m)).  Space: O(m).
import bisect
```

```
def satisfied_with_split_cookies(g, s):
    halves = sorted(x // 2 for x in s for _ in range(2))
    children = sorted(g)
    i = 0
    for cookie in halves:
        if i < len(children) and cookie >= children[i]:
            i += 1
    return i
```

3074. Apple Redistribution into Boxes

Easy · Greedy, Array, Sorting

leetcode.com/problems/apple-redistribution-into-boxes

PROBLEM

You are given an array `apple` of size `n` and an array `capacity` of size `m`.

There are `n` packs where the `i`th pack contains `apple[i]` apples. There are `m` boxes as well, and the `i`th box has a capacity of `capacity[i]` apples.

Return *the **minimum** number of boxes you need to select to redistribute these* `n` *packs of apples into boxes*.

Note that, apples from the same pack can be distributed into different boxes.

Example 1:

```
Input: apple = [1,3,2], capacity = [4,3,1,5,2]
Output: 2
Explanation: We will use boxes with capacities 4 and 5.
It is possible to distribute the apples as the total capacity is greater than or equal to the total
number of apples.
```

Example 2:

```
Input: apple = [5,5,5], capacity = [2,4,2,7]
Output: 4
Explanation: We will need to use all the boxes.
```

Constraints:

- `1 <= n == apple.length <= 50`
- `1 <= m == capacity.length <= 50`
- `1 <= apple[i], capacity[i] <= 50`
- The input is generated such that it's possible to redistribute packs of apples into boxes.

SOLUTION (PYTHON 3)

Solution 1: Greedy + Sorting

How to think about it

Apples may be reboxed arbitrarily; we want the fewest boxes. n,m ≤ 50. Only the sum of capacities versus the total number of apples matters. Larger boxes reach that sum sooner. Sort capacities decreasingly, subtract from the apple total, and return how many boxes were used.

To minimize the number of boxes needed, we should prioritize using boxes with larger capacities. Therefore, we can sort the boxes in descending order of capacity, and then use the boxes one by one until all the apples are packed. We return the number of boxes used at this point.

The time complexity is O(m × log m + n) and the space complexity is O(log m), where m and n are the lengths of the arrays capacity and apple respectively.

Python 3

```python
class Solution:
    def minimumBoxes(self, apple: List[int], capacity: List[int]) -> int:
        capacity.sort(reverse=True)
        s = sum(apple)
        for i, c in enumerate(capacity, 1):
            s -= c
            if s <= 0:
                return i
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the boxes that are used
# 2. How many boxes are used and how much capacity is left
# unused


# ============================================================
# FOLLOW-UP 1: Return the boxes that are used
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort the capacities from big to small and take
# boxes until all apples fit.
# Time: O(m log m).  Space: O(m).
def boxes_used(apple, capacity):
    total = sum(apple)
    chosen = []
    for c in sorted(capacity, reverse=True):
        if total <= 0:
            break
        chosen.append(c)
        total -= c
    return chosen


# ============================================================
# FOLLOW-UP 2: How many boxes are used and how much
# capacity is left unused
# ============================================================

# ---- follow-up 2: function ----
# Answer: Take the biggest boxes first; the unused
# capacity is the total of the chosen boxes minus the
# apples.
# Time: O(m log m).  Space: O(m).
def boxes_and_unused(apple, capacity):
    total = sum(apple)
    used = 0
    held = 0
    for c in sorted(capacity, reverse=True):
        if held >= total:
            break
        used += 1
        held += c
    return used, held - total
```

PART 2 · GREEDY · PROBLEM 3 OF 8

826. Most Profit Assigning Work

Medium · Greedy, Array, Two Pointers, Binary Search, Sorting

leetcode.com/problems/most-profit-assigning-work

★ **TOP 7 PROBLEM** in Greedy: master this one first

PROBLEM

You have `n` jobs and `m` workers. You are given three arrays: `difficulty`, `profit`, and `worker` where:

- `difficulty[i]` and `profit[i]` are the difficulty and the profit of the i^{th} job, and
- `worker[j]` is the ability of j^{th} worker (i.e., the j^{th} worker can only complete a job with difficulty at most `worker[j]`).

Every worker can be assigned **at most one job**, but one job can be **completed multiple times**.

- For example, if three workers attempt the same job that pays `$1`, then the total profit will be `$3`. If a worker cannot complete any job, their profit is `$0`.

Return the maximum profit we can achieve after assigning the workers to the jobs.

Example 1:

```
Input: difficulty = [2,4,6,8,10], profit = [10,20,30,40,50], worker = [4,5,6,7]
Output: 100
Explanation: Workers are assigned jobs of difficulty [4,4,6,6] and they get a profit of [20,20,30,30]
separately.
```

Example 2:

```
Input: difficulty = [85,47,57], profit = [24,66,99], worker = [40,25,25]
Output: 0
```

Constraints:

- `n == difficulty.length`
- `n == profit.length`
- `m == worker.length`
- `1 <= n, m <= 10^4`
- `1 <= difficulty[i], profit[i], worker[i] <= 10^5`

SOLUTIONS (PYTHON 3)

Solution 1: Sorting + Two Pointers

How to think about it

A worker takes the most profitable job whose difficulty is at most their ability. Scanning every job per worker is slow for $n,m \le 10^4$. Sort workers by ability and jobs by difficulty, then advance a pointer: ability is monotone, so we can maintain the best profit so far. Each worker reads that maximum in O(1).

We can sort the jobs in ascending order of ability, and then sort the jobs in ascending order of difficulty.

Then we traverse the workers. For each worker, we find the job with the maximum profit that he can complete, and then add this profit to the answer.

The time complexity is $O(n \times \log n + m \times \log m)$, and the space complexity is $O(n)$. Where n and m are the lengths of the arrays `profit` and `worker` respectively.

Python 3

```python
class Solution:
    def maxProfitAssignment(
        self, difficulty: List[int], profit: List[int], worker: List[int]
    ) -> int:
        worker.sort()
        jobs = sorted(zip(difficulty, profit))
        ans = mx = i = 0
        for w in worker:
            while i < len(jobs) and jobs[i][0] <= w:
                mx = max(mx, jobs[i][1])
                i += 1
            ans += mx
        return ans
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> The two-pointer method sorts both arrays. Difficulty is at most 10^5, so f[d] can store the best profit of exact difficulty d, then a prefix maximum gives the best job of difficulty ≤ i. Each worker is a table lookup. Workers need not be sorted, which is convenient when the difficulty range is modest.

Let's denote m = max(difficulty) and define an array f of length m + 1, where f[i] represents the maximum profit among jobs with difficulty less than or equal to i, initially f[i] = 0.

Then, we iterate over the jobs, and for each job (d, p), if d ≤ m, we update f[d] = max(f[d], p).

Next, we iterate from 1 to m, and for each i, we update f[i] = max(f[i], f[i - 1]).

Finally, we iterate over the workers, and for each worker w, we add f[w] to the answer.

The time complexity is O(n + M), and the space complexity is O(M). Here, n is the length of the `profit` array, and M is the maximum value in the `difficulty` array.

Python 3

```python
class Solution:
    def maxProfitAssignment(
        self, difficulty: List[int], profit: List[int], worker: List[int]
    ) -> int:
        m = max(difficulty)
        f = [0] * (m + 1)
        for d, p in zip(difficulty, profit):
            f[d] = max(f[d], p)
        for i in range(1, m + 1):
            f[i] = max(f[i], f[i - 1])
        return sum(f[min(w, m)] for w in worker)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Say which job every worker takes
# 2. Every job can be done by only ONE worker


# ============================================================
# FOLLOW-UP 1: Say which job every worker takes
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same sweep over sorted jobs, remembering the
# best job so far.
# Time: O(n log n + w log w).  Space: O(n + w).
def job_of_each_worker(difficulty, profit, worker):
```

```
    jobs = sorted(range(len(difficulty)), key=lambda i: difficulty[i])
    order = sorted(range(len(worker)), key=lambda w: worker[w])
    choice = [None] * len(worker)
    best = None
    k = 0
    for w in order:
        while k < len(jobs) and difficulty[jobs[k]] <= worker[w]:
            if best is None or profit[jobs[k]] > profit[best]:
                best = jobs[k]
            k += 1
        choice[w] = best
    return choice


# ========================================================
# FOLLOW-UP 2: Every job can be done by only ONE worker
# ========================================================

# ---- follow-up 2: function ----
# Answer: Greedy matching: take the most profitable jobs
# first and give each to the weakest worker who can do it.
# Time: O(n log n + n * w) (workers.pop shifts the list).
# Space: O(w).
import bisect


def profit_each_job_once(difficulty, profit, worker):
    workers = sorted(worker)
    total = 0
    for p, d in sorted(zip(profit, difficulty), reverse=True):
        k = bisect.bisect_left(workers, d)
        if k < len(workers):
            workers.pop(k)
            total += p
    return total
```

PART 2 · GREEDY · PROBLEM 4 OF 8

1488. Avoid Flood in The City

Medium · Greedy, Array, Hash Table, Binary Search, Heap (Priority Queue)

leetcode.com/problems/avoid-flood-in-the-city

★ **TOP 7 PROBLEM** in Greedy: master this one first

PROBLEM

Your country has 10^9 lakes. Initially, all the lakes are empty, but when it rains over the n^{th} lake, the n^{th} lake becomes full of water. If it rains over a lake that is **full of water**, there will be a **flood**. Your goal is to avoid floods in any lake.

Given an integer array `rains` where:

- `rains[i] > 0` means there will be rains over the `rains[i]` lake.
- `rains[i] == 0` means there are no rains this day and you **must** choose **one lake** this day and **dry it**.

Return *an array* `ans` where:

- `ans.length == rains.length`
- `ans[i] == -1` if `rains[i] > 0`.
- `ans[i]` is the lake you choose to dry in the `ith` day if `rains[i] == 0`.

If there are multiple valid answers return **any** of them. If it is impossible to avoid flood return **an empty array**.

Notice that if you chose to dry a full lake, it becomes empty, but if you chose to dry an empty lake, nothing changes.

Example 1:

```
Input: rains = [1,2,3,4]
Output: [-1,-1,-1,-1]
Explanation: After the first day full lakes are [1]
After the second day full lakes are [1,2]
After the third day full lakes are [1,2,3]
After the fourth day full lakes are [1,2,3,4]
There's no day to dry any lake and there is no flood in any lake.
```

Example 2:

```
Input: rains = [1,2,0,0,2,1]
Output: [-1,-1,2,1,-1,-1]
Explanation: After the first day full lakes are [1]
After the second day full lakes are [1,2]
After the third day, we dry lake 2. Full lakes are [1]
After the fourth day, we dry lake 1. There is no full lakes.
After the fifth day, full lakes are [2].
After the sixth day, full lakes are [1,2].
It is easy that this scenario is flood-free. [-1,-1,1,2,-1,-1] is another acceptable scenario.
```

Example 3:

```
Input: rains = [1,2,0,1,2]
Output: []
Explanation: After the second day, full lakes are [1,2]. We have to dry one lake in the third day.
After that, it will rain over lakes [1,2]. It's easy to prove that no matter which lake you choose to
dry in the 3rd day, the other one will flood.
```

Constraints:

- `1 <= rains.length <= 10^5`
- `0 <= rains[i] <= 10^9`

SOLUTION (PYTHON 3)

Solution 1: Greedy + Binary Search

How to think about it

n≤ 10^5. A lake that rains a second time must have been dried after the previous rain. A dry day should be spent on the lake that will rain again soonest among those still full. Store dry days in a sorted list and the last rain day of each lake in a map. On a repeat rain, binary-search the first dry day after that last rain; if none exists, fail. Unused dry days become 1.

We store all sunny days in the sunny array or a sorted set, and use the hash table rainy to record the last rainy day for each lake. We initialize the answer array ans with each element set to -1.

Next, we traverse the rains array. For each rainy day i, if rainy[rains[i]] exists, it means that the lake has rained before, so we need to find the first date in the sunny array that is greater than rainy[rains[i]], and replace it with the rainy day. Otherwise, it means that the flood cannot be prevented, and we return an empty array. For each non-rainy day i, we store i in the sunny array and set ans[i] to 1.

After the traversal, we return the answer array.

The time complexity is O(n × log n), and the space complexity is O(n). Here, n is the length of the rains array.

Python 3

```
class Solution:
    def avoidFlood(self, rains: List[int]) -> List[int]:
        n = len(rains)
        ans = [-1] * n
        sunny = SortedList()
        rainy = {}
        for i, v in enumerate(rains):
            if v:
                if v in rainy:
                    idx = sunny.bisect_right(rainy[v])
                    if idx == len(sunny):
                        return []
                    ans[sunny[idx]] = v
                    sunny.discard(sunny[idx])
                rainy[v] = i
            else:
                sunny.add(i)
                ans[i] = 1
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. If a flood cannot be avoided, report the day of the
# first flood
# 2. How many dry days were left unused


# ============================================================
# FOLLOW-UP 1: If a flood cannot be avoided, report the
# day of the first flood
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same greedy; return the day when a full lake
# rains again with no dry day available.
# Time: O(n^2) in the worst case (list pop), otherwise O(n
# log n).  Space: O(n).
import bisect
```

```
def first_flood_day(rains):
    full = {}
    dry_days = []
    for day, lake in enumerate(rains):
        if lake == 0:
            dry_days.append(day)
        else:
            if lake in full:
                k = bisect.bisect_right(dry_days, full[lake])
                if k == len(dry_days):
                    return day
                dry_days.pop(k)
            full[lake] = day
    return None


# ============================================================
# FOLLOW-UP 2: How many dry days were left unused
# ============================================================

# ---- follow-up 2: function ----
# Answer: Run the greedy and count the dry days nobody
# needed.
# Time: O(n^2) in the worst case.  Space: O(n).
def unused_dry_days(rains):
    full = {}
    dry_days = []
    used = 0
    for day, lake in enumerate(rains):
        if lake == 0:
            dry_days.append(day)
        else:
            if lake in full:
                k = bisect.bisect_right(dry_days, full[lake])
                if k == len(dry_days):
                    return None
                dry_days.pop(k)
                used += 1
            full[lake] = day
    total_dry = sum(1 for x in rains if x == 0)
    return total_dry - used
```

1717. Maximum Score From Removing Substrings

Medium · Stack, Greedy, String

leetcode.com/problems/maximum-score-from-removing-substrings

★ **TOP 7 PROBLEM** in Greedy: master this one first

PROBLEM

You are given a string `s` and two integers `x` and `y`. You can perform two types of operations any number of times.

- Remove substring `"ab"` and gain `x` points.
  - For example, when removing `"ab"` from `"cabxbae"` it becomes `"cxbae"`.
- Remove substring `"ba"` and gain `y` points.
  - For example, when removing `"ba"` from `"cabxbae"` it becomes `"cabxe"`.

Return *the maximum points you can gain after applying the above operations on* `s`.

Example 1:

```
Input: s = "cdbcbbaaabab", x = 4, y = 5
Output: 19
Explanation:
- Remove the "ba" underlined in "cdbcbbaaabab". Now, s = "cdbcbbaaab" and 5 points are added to the score.
- Remove the "ab" underlined in "cdbcbbaaab". Now, s = "cdbcbbaa" and 4 points are added to the score.
- Remove the "ba" underlined in "cdbcbbaa". Now, s = "cdbcba" and 5 points are added to the score.
- Remove the "ba" underlined in "cdbcba". Now, s = "cdbc" and 5 points are added to the score.
Total score = 5 + 4 + 5 + 5 = 19.
```

Example 2:

```
Input: s = "aabbaaxybbaabb", x = 5, y = 4
Output: 20
```

Constraints:

- `1 <= s.length <= 10^5`
- `1 <= x, y <= 10^4`
- `s` consists of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Greedy

How to think about it

Deleting ab scores x and ba scores y; each deletion changes adjacency. Searching deletion orders is infeasible on long strings. A run of only a and b always ends with a single letter; the number of operations is fixed by the two counts. Higher-scoring pairs should be taken first. Swap letters and scores when x<y. While scanning, match the high-score pair against a counter; at a separator or the end, settle the low-score pair with $\min(cnt_a, cnt_b)$.

We can assume that the score of substring "ab" is always no less than the score of substring "ba". If not, we can swap "a" and "b", and simultaneously swap x and y.

Next, we only need to consider the case where the string contains only "a" and "b". If the string contains other characters, we can treat them as split points, dividing the string into several substrings that contain only "a" and "b", and then calculate the score for each substring separately.

We observe that for a substring containing only "a" and "b", no matter what operations we take, we will eventually be left with only one type of character, or an empty string. Since each operation removes one "a" and one "b" simultaneously, the total number of operations is fixed. We can greedily remove "ab" first, then remove "ba", which ensures the maximum score.

Therefore, we can use two variables cnt1 and cnt2 to record the counts of "a" and "b" respectively, then traverse the string and update cnt1 and cnt2 according to different cases of the current character, while calculating the score.

For the current character c being traversed:

- If c is "a", since we want to remove "ab" first, we don't eliminate this character at this time, only increment cnt1;
- If c is "b", if cnt1 > 0 at this time, we can eliminate one "ab" and add x points; otherwise, we can only increment cnt2;
- If c is another character, then for this substring, we have cnt2 "b"s and cnt1 "a"s left. We can eliminate min(cnt1, cnt2) "ba"s and add several y points.

After traversal, we need to handle the remaining "ba"s and add several y points.

The time complexity is O(n), where n is the length of string s. The space complexity is O(1).

Python 3

```python
class Solution:
    def maximumGain(self, s: str, x: int, y: int) -> int:
        a, b = "a", "b"
        if x < y:
            x, y = y, x
            a, b = b, a
        ans = cnt1 = cnt2 = 0
        for c in s:
            if c == a:
                cnt1 += 1
            elif c == b:
                if cnt1:
                    ans += x
                    cnt1 -= 1
                else:
                    cnt2 += 1
            else:
                ans += min(cnt1, cnt2) * y
                cnt1 = cnt2 = 0
        ans += min(cnt1, cnt2) * y
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the string that is left after all the removals
# 2. Only 'ab' can be removed: how many times


# ============================================================
# FOLLOW-UP 1: Return the string that is left after all
# the removals
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same stack passes; the stack is what
# remains.
# Time: O(n).  Space: O(n).
def left_after_removals(s, x, y):
    def remove(text, first, second):
        stack = []
        for ch in text:
            if stack and stack[-1] == first and ch == second:
                stack.pop()
            else:
```

```
                stack.append(ch)
        return "".join(stack)

    if x >= y:
        return remove(remove(s, "a", "b"), "b", "a")
    return remove(remove(s, "b", "a"), "a", "b")


# ============================================================
# FOLLOW-UP 2: Only 'ab' can be removed: how many times
# ============================================================

# ---- follow-up 2: function ----
# Answer: A single stack pass counting the removals.
# Time: O(n).  Space: O(n).
def count_ab_removals(s):
    stack = []
    count = 0
    for ch in s:
        if stack and stack[-1] == "a" and ch == "b":
            stack.pop()
            count += 1
        else:
            stack.append(ch)
    return count
```

PART 2 · GREEDY · PROBLEM 6 OF 8

1871. Jump Game VII

Medium · String, Dynamic Programming, Prefix Sum, Sliding Window

leetcode.com/problems/jump-game-vii

★ **TOP 7 PROBLEM** in Greedy: master this one first

PROBLEM

You are given a **0-indexed** binary string `s` and two integers `minJump` and `maxJump`. In the beginning, you are standing at index `0`, which is equal to `'0'`. You can move from index `i` to index `j` if the following conditions are fulfilled:

- `i + minJump <= j <= min(i + maxJump, s.length - 1)`, and
- `s[j] == '0'`.

Return `true` *if you can reach index* `s.length - 1` *in* `s`, *or* `false` *otherwise.*

Example 1:

```
Input: s = "011010", minJump = 2, maxJump = 3
Output: true
Explanation:
In the first step, move from index 0 to index 3.
In the second step, move from index 3 to index 5.
```

Example 2:

```
Input: s = "01101110", minJump = 2, maxJump = 3
Output: false
```

Constraints:

- `2 <= s.length <= 10^5`
- `s[i]` is either `'0'` or `'1'`.
- `s[0] == '0'`
- `1 <= minJump <= maxJump < s.length`

SOLUTION (PYTHON 3)

Solution 1: Prefix Sum + Dynamic Programming

How to think about it

From index 0 we may jump to a `'0'` whose distance lies in [minJump,maxJump]. Checking the jump window from every index is too slow for n≤ 10^5. f[i] is true iff s[i]='0' and some reachable index sits in [i-maxJump,i-minJump]. A prefix sum of f answers that range in O(1), so we fill f from left to right.

We define a prefix sum array pre of length n+1, where pre[i] represents the number of reachable positions in the first i positions of s. We define a boolean array f of length n, where f[i] indicates whether s[i] is reachable. Initially, pre[1] = 1 and f[0] = true.

Consider $i \in [1, n)$, if s[i] = 0, then we need to determine whether there exists a position j in the first i positions of s, such that j is reachable and the distance from j to i is within [minJump, maxJump]. If such a position j exists, then we have f[i] = true, otherwise f[i] = false. When determining whether j exists, we can use the prefix sum array pre to determine whether such a position j exists in O(1) time.

The final answer is f[n-1].

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the string s.

Python 3

```python
class Solution:
    def canReach(self, s: str, minJump: int, maxJump: int) -> bool:
        n = len(s)
        pre = [0] * (n + 1)
        pre[1] = 1
        f = [True] + [False] * (n - 1)
        for i in range(1, n):
            if s[i] == "0":
                l, r = max(0, i - maxJump), i - minJump
                f[i] = l <= r and pre[r + 1] - pre[l] > 0
            pre[i + 1] = pre[i] + f[i]
        return f[-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The minimum number of jumps to reach the end
# 2. Return one path of positions


# ============================================================
# FOLLOW-UP 1: The minimum number of jumps to reach the
# end
# ============================================================

# ---- follow-up 1: function ----
# Answer: Breadth-first search with a window of reachable
# positions (a pointer that never moves back).
# Time: O(n).  Space: O(n).
from collections import deque


def min_jumps(s, min_jump, max_jump):
    n = len(s)
    if s[-1] != "0":
        return -1
    distance = [-1] * n
    distance[0] = 0
    queue = deque([0])
    farthest = 0
    while queue:
        i = queue.popleft()
        for j in range(max(i + min_jump, farthest + 1), min(i + max_jump, n - 1) + 1):
            if s[j] == "0":
                distance[j] = distance[i] + 1
                queue.append(j)
        farthest = max(farthest, min(i + max_jump, n - 1))
    return distance[-1]


# ============================================================
# FOLLOW-UP 2: Return one path of positions
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same search, remembering the previous position.
# Time: O(n).  Space: O(n).
def one_path(s, min_jump, max_jump):
    n = len(s)
    previous = [-1] * n
    seen = [False] * n
    seen[0] = True
    queue = deque([0])
    farthest = 0
    while queue:
```

```
        i = queue.popleft()
        for j in range(max(i + min_jump, farthest + 1), min(i + max_jump, n - 1) + 1):
            if s[j] == "0":
                seen[j] = True
                previous[j] = i
                queue.append(j)
        farthest = max(farthest, min(i + max_jump, n - 1))
    if not seen[n - 1]:
        return None
    path = []
```

```
    i = n - 1
    while i != -1:
        path.append(i)
        i = previous[i]
    return path[::-1]
```

PART 2 · GREEDY · PROBLEM 7 OF 8

757. Set Intersection Size At Least Two

Hard · Greedy, Array, Sorting

leetcode.com/problems/set-intersection-size-at-least-two

★ **TOP 7 PROBLEM** in Greedy: master this one first

PROBLEM

You are given a 2D integer array `intervals` where `intervals[i] = [start_i, end_i]` represents all the integers from `start_i` to `end_i` inclusively.

A **containing set** is an array `nums` where each interval from `intervals` has **at least two** integers in `nums`.

- For example, if `intervals = [[1,3], [3,7], [8,9]]`, then `[1,2,4,7,8,9]` and `[2,3,4,8,9]` are **containing sets**.

Return *the minimum possible size of a containing set*.

Example 1:

```
Input: intervals = [[1,3],[3,7],[8,9]]
Output: 5
Explanation: let nums = [2, 3, 4, 8, 9].
It can be shown that there cannot be any containing array of size 4.
```

Example 2:

```
Input: intervals = [[1,3],[1,4],[2,5],[3,5]]
Output: 3
Explanation: let nums = [2, 3, 4].
It can be shown that there cannot be any containing array of size 2.
```

Example 3:

```
Input: intervals = [[1,2],[2,3],[2,4],[4,5]]
Output: 5
Explanation: let nums = [1, 2, 3, 4, 5].
It can be shown that there cannot be any containing array of size 4.
```

Constraints:

- `1 <= intervals.length <= 3000`
- `intervals[i].length == 2`
- `0 <= start_i < end_i <= 10^8`

SOLUTION (PYTHON 3)

Solution 1: Sorting + Greedy

How to think about it

Pick as few points as possible so every interval contains at least two. 3000 intervals rule out subsets. Points should sit as far right as possible to be reused. Sort by end, then by start descending, so tighter intervals come first. Keep the last two placed points s<e. Skip if both lie in the interval; if only e does, add b; if neither does, add b-1 and b. Each step adds the fewest, rightmost points.

We want to select as few integer points as possible on the number line such that each interval contains at least two points. A classic and effective strategy is to sort intervals by their right endpoints and try to place selected points towards the right side of intervals, so that these points can cover more subsequent intervals.

First, sort all intervals by the following rules:

1. Sort by right endpoint in ascending order;
2. If right endpoints are equal, sort by left endpoint in descending order.

The reason for this sorting is: intervals with smaller right endpoints have less "operational space" and should be satisfied first; when right endpoints are equal, intervals with larger left endpoints are narrower and should be prioritized.

Next, we use two variables s and e to record the **second-to-last point** and **last point** that are common to all currently processed intervals. Initially, s = e = -1, indicating that no points have been placed yet.

Then we process the sorted intervals [a, b] one by one, and discuss three cases based on their relationship with {s, e}:

1. **If a ≤ s**: The current interval already contains both points s and e, so no additional points are needed.
2. **If s < a ≤ e**: The current interval only contains one point (i.e., e), and needs one more point. To make the new point most helpful for subsequent intervals, we choose the rightmost point b in the interval. Update ans = ans + 1, and set the new two points to {e, b}.
3. **If a > e**: The current interval does not contain either of the existing two points, so two points need to be added. The optimal choice is to place {b - 1, b} at the rightmost side of the interval. Update ans = ans + 2, and set the new two points to {b - 1, b}.

Finally, return the total number of points placed, ans.

The time complexity is O(n × log n) and the space complexity is O(log n), where n is the number of intervals.

Python 3

```python
class Solution:
    def intersectionSizeTwo(self, intervals: List[List[int]]) -> int:
        intervals.sort(key=lambda x: (x[1], -x[0]))
        s = e = -1
        ans = 0
        for a, b in intervals:
            if a <= s:
                continue
            if a > e:
                ans += 2
                s, e = b - 1, b
            else:
                ans += 1
                s, e = e, b
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. At least k numbers from every interval
# 2. Return the chosen set (at least two per interval)


# ============================================================
# FOLLOW-UP 1: At least k numbers from every interval
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort by end (then start, descending); for each
# interval count the chosen numbers inside it and add the
# largest missing ones from its right end.
# Time: O(n log n + n^2 * k^2) (the list `chosen` is
# scanned repeatedly).  Space: O(n * k).
def smallest_set_at_least_k(intervals, k):
    intervals = sorted(intervals, key=lambda x: (x[1], -x[0]))
    chosen = []
    for start, end in intervals:
        inside = sum(1 for x in chosen if start <= x <= end)
        value = end
        while inside < k:
            while value in chosen:
```

```
                value -= 1
            chosen.append(value)
            inside += 1
    return len(chosen)


# ========================================================
# FOLLOW-UP 2: Return the chosen set (at least two per
# interval)
# ========================================================

# ---- follow-up 2: function ----
# Answer: The same greedy with k = 2, listing the numbers.
# Time: O(n log n + n^2).  Space: O(n).
def smallest_set_numbers(intervals):
    intervals = sorted(intervals, key=lambda x: (x[1], -x[0]))
    chosen = []
    for start, end in intervals:
        inside = sum(1 for x in chosen if start <= x <= end)
        value = end
        while inside < 2:
            while value in chosen:
                value -= 1
            chosen.append(value)
            inside += 1
    return sorted(chosen)
```

PART 2 · GREEDY · PROBLEM 8 OF 8

857. Minimum Cost to Hire K Workers

Hard · Greedy, Array, Sorting, Heap (Priority Queue)

leetcode.com/problems/minimum-cost-to-hire-k-workers

★ **TOP 7 PROBLEM** in Greedy: master this one first

PROBLEM

There are `n` workers. You are given two integer arrays `quality` and `wage` where `quality[i]` is the quality of the i^{th} worker and `wage[i]` is the minimum wage expectation for the i^{th} worker.

We want to hire exactly `k` workers to form a **paid group**. To hire a group of `k` workers, we must pay them according to the following rules:

1. Every worker in the paid group must be paid at least their minimum wage expectation.
2. In the group, each worker's pay must be directly proportional to their quality. This means if a worker's quality is double that of another worker in the group, then they must be paid twice as much as the other worker.

Given the integer `k`, return *the least amount of money needed to form a paid group satisfying the above conditions*. Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

```
Input: quality = [10,20,5], wage = [70,50,30], k = 2
Output: 105.00000
Explanation: We pay 70 to 0th worker and 35 to 2nd worker.
```

Example 2:

```
Input: quality = [3,1,10,10,1], wage = [4,8,2,2,7], k = 3
Output: 30.66667
Explanation: We pay 4 to 0th worker, 13.33333 to 2nd and 3rd workers separately.
```

Constraints:

- `n == quality.length == wage.length`
- `1 <= k <= n <=` 10^4
- `1 <= quality[i], wage[i] <=` 10^4

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Everyone is paid at one ratio, at least their wage expectation; we pick k workers to minimize total pay. The ratio is the max wage/quality in the group. Subsets are impossible for n≤ 10^4. Insert workers by increasing ratio so the current ratio covers the group. A max-heap drops the largest quality to keep size k. Whenever the group has k people, update with current ratio times total quality.

Python 3

```python
class Solution:
    def mincostToHireWorkers(
        self, quality: List[int], wage: List[int], k: int
    ) -> float:
        t = sorted(zip(quality, wage), key=lambda x: x[1] / x[0])
        ans, tot = inf, 0
        h = []
        for q, w in t:
            tot += q
            heappush(h, -q)
            if len(h) == k:
                ans = min(ans, w / q * tot)
                tot += heappop(h)
        return ans
```

Complexity. Time: O(n log n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the workers that are hired
# 2. The same answer without a heap (slower, easier to
# check)


# ============================================================
# FOLLOW-UP 1: Return the workers that are hired
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same heap sweep; remember the best group.
# Time: O(n log n + n * k log k) in the worst case.
# Space: O(n).
import heapq


def hired_workers(quality, wage, k):
    workers = sorted(range(len(quality)), key=lambda i: wage[i] / quality[i])
    heap = []
    total = 0
    best = None
    best_group = None
    for i in workers:
        heapq.heappush(heap, (-quality[i], i))
        total += quality[i]
        if len(heap) > k:
            q, j = heapq.heappop(heap)
            total += q
        if len(heap) == k:
            cost = total * wage[i] / quality[i]
            if best is None or cost < best:
                best = cost
                best_group = sorted(j for q, j in heap)
    return best, best_group


# ============================================================
# FOLLOW-UP 2: The same answer without a heap (slower,
# easier to check)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Sort by wage / quality; for each possible last
# worker, the group is the k smallest qualities among
# those so far, paid at that worker's ratio.
# Time: O(n^2 log n).  Space: O(n).
```

```
def cheapest_group_cost(quality, wage, k):
    best = None
    workers = sorted(range(len(quality)), key=lambda i: wage[i] / quality[i])
    for end in range(k - 1, len(workers)):
        ratio = wage[workers[end]] / quality[workers[end]]
        qualities = sorted(quality[i] for i in workers[:end + 1])
        cost = sum(qualities[:k]) * ratio
        if best is None or cost < best:
            best = cost
    return best
```

PART 2 · TOPIC 31 OF 50

Depth First Search (DFS)

4 problems · 0 Easy · 2 Medium · 2 Hard

THE IDEA

Follow one path as far as it goes, mark cells or nodes as visited, then backtrack. Use it for regions, paths, cycles and connected components.

THE 4 PROBLEMS, EASY TO HARD | ★ = TOP 4: DO THESE FIRST

1	★ **Employee Importance**	Medium	LC 690	2478
2	★ **Is Graph Bipartite**	Medium	LC 785	2482
3	★ **Critical Connections in a Network**	Hard	LC 1192	2486
4	★ **Longest Path With Different Adjacent Characters**	Hard	LC 2246	2490

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Flood Fill (Part 1, pattern 16, page 842)

Number of Distinct Islands (Part 1, pattern 18, page 1026)

Time Needed to Inform All Employees (Part 1, pattern 16, page 899)

Clone Graph (Part 1, pattern 16, page 854)

Surrounded Regions (Part 1, pattern 16, page 862)

Number of Enclaves (Part 1, pattern 16, page 873)

Number of Islands (Part 1, pattern 16, page 846)

Word Search (Part 1, pattern 15, page 815)

All Paths From Source to Target (Part 1, pattern 16, page 888)

All Nodes Distance K in Binary Tree (Part 1, pattern 11, page 588)

Pacific Atlantic Water Flow (Part 1, pattern 16, page 858)

Making A Large Island (Part 1, pattern 16, page 920)

690. Employee Importance

Medium · Tree, Depth-First Search, Breadth-First Search, Array, Hash Table

leetcode.com/problems/employee-importance

★ **TOP 7 PROBLEM** in Depth First Search (DFS): master this one first

PROBLEM

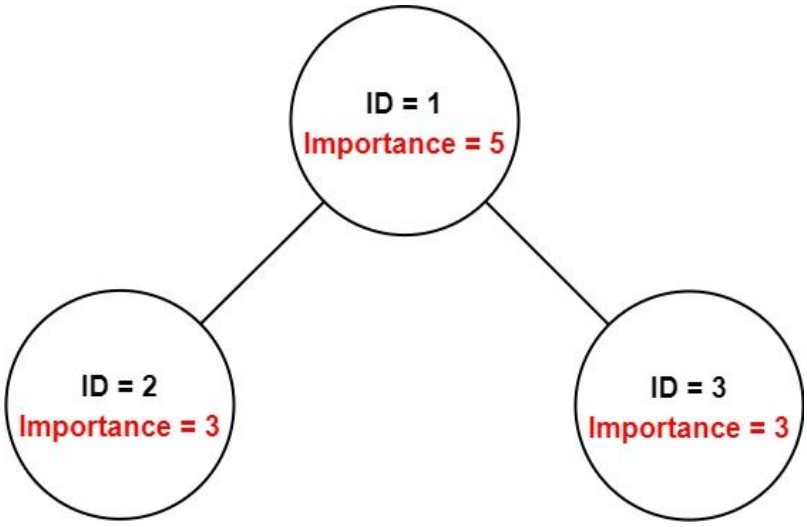
You have a data structure of employee information, including the employee's unique ID, importance value, and direct subordinates' IDs.

You are given an array of employees `employees` where:

- `employees[i].id` is the ID of the i^{th} employee.
- `employees[i].importance` is the importance value of the i^{th} employee.
- `employees[i].subordinates` is a list of the IDs of the direct subordinates of the i^{th} employee.

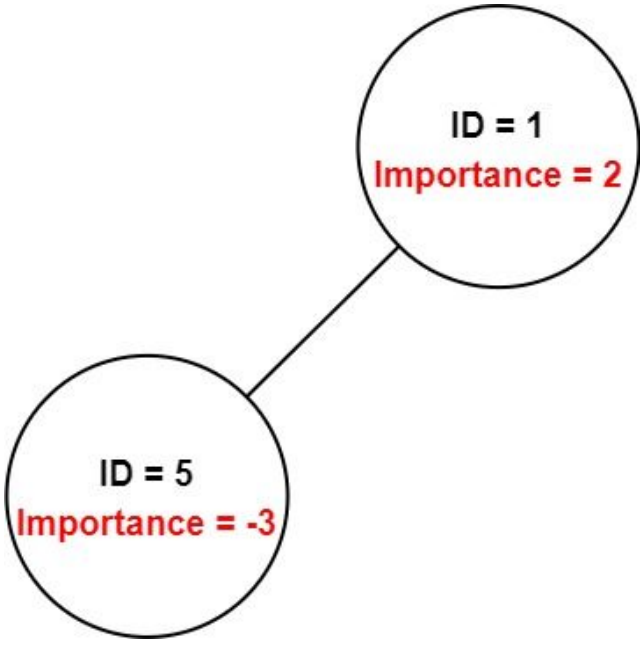
Given an integer `id` that represents an employee's ID, return *the **total** importance value of this employee and all their direct and indirect subordinates*.

Example 1:




```
Input: employees = [[1,5,[2,3]],[2,3,[]],[3,3,[]]], id = 1
Output: 11
Explanation: Employee 1 has an importance value of 5 and has two direct subordinates: employee 2 and
employee 3.
They both have an importance value of 3.
Thus, the total importance value of employee 1 is 5 + 3 + 3 = 11.
```

Example 2:




```
Input: employees = [[1,2,[5]],[5,-3,[]]], id = 5
Output: -3
Explanation: Employee 5 has an importance value of -3 and has no direct subordinates.
Thus, the total importance value of employee 5 is -3.
```

Constraints:

- `1 <= employees.length <= 2000`
- `1 <= employees[i].id <= 2000`
- All `employees[i].id` are **unique**.
- `-100 <= employees[i].importance <= 100`
- One employee has at most one direct leader and may have several subordinates.
- The IDs in `employees[i].subordinates` are valid IDs.

SOLUTION (PYTHON 3)

Solution 1: Hash Table + DFS

> **How to think about it**
>
> Importance is the employee plus every subordinate. Looking up ids in the raw list is linear per hop. Map id to the object, then DFS from the given id: `importance` plus the sum of recursive calls on subordinates.

We use a hash table d to store all employee information, where the key is the employee's ID, and the value is the employee object. Then we start a depth-first search from the given employee ID. Each time we traverse to an employee, we add the employee's importance to the answer, and recursively traverse all the subordinates of the employee, adding the importance of the subordinates to the answer as well.

The time complexity is O(n), and the space complexity is O(n). Where n is the number of employees.

Python 3

```
"""
# Definition for Employee.
class Employee:
    def __init__(self, id: int, importance: int, subordinates: List[int]):
        self.id = id
        self.importance = importance
        self.subordinates = subordinates
"""


class Solution:
    def getImportance(self, employees: List["Employee"], id: int) -> int:
        def dfs(i: int) -> int:
            return d[i].importance + sum(dfs(j) for j in d[i].subordinates)

        d = {e.id: e for e in employees}
        return dfs(id)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The importance of the whole team below an employee,
# but only down to depth d
# 2. Which employee has the most important team


# ============================================================
# FOLLOW-UP 1: The importance of the whole team below an
# employee, but only down to depth d
# ============================================================

# ---- follow-up 1: function ----
# Answer: Depth-first search with a depth counter
# (employees are [id, importance, subordinates]).
# Time: O(n).  Space: O(h).
def team_importance_to_depth(employees, boss, d):
    by_id = {e[0]: e for e in employees}

    def go(i, depth):
        _, importance, subs = by_id[i]
        total = importance
        if depth < d:
            for s in subs:
                total += go(s, depth + 1)
        return total

    return go(boss, 0)


# ============================================================
# FOLLOW-UP 2: Which employee has the most important team
# ============================================================

# ---- follow-up 2: function ----
# Answer: Compute the team total of every employee once
# (memoised) and take the maximum.
# Time: O(n).  Space: O(n).
def most_important_team(employees):
    by_id = {e[0]: e for e in employees}
    memo = {}

    def total(i):
        if i not in memo:
            memo[i] = by_id[i][1] + sum(total(s) for s in by_id[i][2])
        return memo[i]
```

```
    best = max(by_id, key=lambda i: (total(i), -i))
    return best, total(best)
```

785. Is Graph Bipartite

Medium · Depth-First Search, Breadth-First Search, Union Find, Graph, Graph Coloring, Bipartite Graph

leetcode.com/problems/is-graph-bipartite

★ **TOP 7 PROBLEM** in Depth First Search (DFS): master this one first

PROBLEM

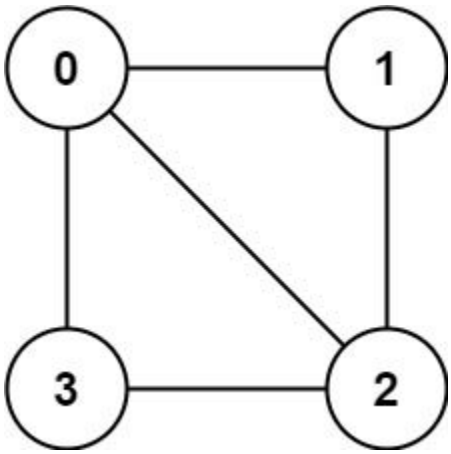
There is an **undirected** graph with `n` nodes, where each node is numbered between `0` and `n - 1`. You are given a 2D array `graph`, where `graph[u]` is an array of nodes that node `u` is adjacent to. More formally, for each `v` in `graph[u]`, there is an undirected edge between node `u` and node `v`. The graph has the following properties:

- There are no self-edges (`graph[u]` does not contain `u`).
- There are no parallel edges (`graph[u]` does not contain duplicate values).
- If `v` is in `graph[u]`, then `u` is in `graph[v]` (the graph is undirected).
- The graph may not be connected, meaning there may be two nodes `u` and `v` such that there is no path between them.

A graph is **bipartite** if the nodes can be partitioned into two independent sets `A` and `B` such that **every** edge in the graph connects a node in set `A` and a node in set `B`.

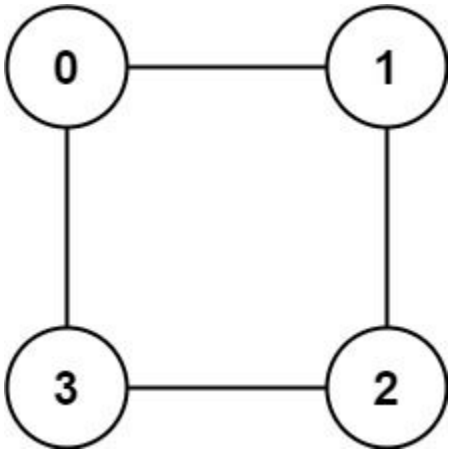
Return `true` *if and only if it is* ***bipartite***.

Example 1:




```
Input: graph = [[1,2,3],[0,2],[0,1,3],[0,2]]
Output: false
Explanation: There is no way to partition the nodes into two independent sets such that every edge
connects a node in one and a node in the other.
```

Example 2:




```
Input: graph = [[1,3],[0,2],[1,3],[0,2]]
Output: true
Explanation: We can partition the nodes into two sets: {0, 2} and {1, 3}.
```

Constraints:

- `graph.length == n`
- `1 <= n <= 100`
- `0 <= graph[u].length < n`
- `0 <= graph[u][i] <= n - 1`
- `graph[u]` does not contain `u`.
- All the values of `graph[u]` are **unique**.
- If `graph[u]` contains `v`, then `graph[v]` contains `u`.

SOLUTIONS (PYTHON 3)

Solution 1: Coloring Method to Determine Bipartite Graph

How to think about it

Decide whether an undirected graph is bipartite. n≤ 100. DFS from each uncolored vertex with two colors; a neighbor already colored the same fails. The graph may be disconnected, so every component is colored.

Traverse all nodes for coloring. For example, initially color them white, and use DFS to color the adjacent nodes with another color. If the target color to be colored is different from the color that the node has already been colored, it means that it cannot form a bipartite graph.

The time complexity is O(n), and the space complexity is O(n). Where n is the number of nodes.

Python 3

```python
class Solution:
    def isBipartite(self, graph: List[List[int]]) -> bool:
        def dfs(a: int, c: int) -> bool:
            color[a] = c
            for b in graph[a]:
                if color[b] == c or (color[b] == 0 and not dfs(b, -c)):
                    return False
            return True

        n = len(graph)
        color = [0] * n
        for i in range(n):
            if color[i] == 0 and not dfs(i, 1):
                return False
        return True
```

Solution 2: Union-Find

> **How to think about it**
>
> Coloring is equivalent to union-find: all neighbors of a belong to one part, not a's. If a and a neighbor share a root, fail; otherwise union the neighbors together.

For this problem, if it is a bipartite graph, then all adjacent nodes of each vertex in the graph should belong to the same set and not be in the same set as the vertex. Therefore, we can use the union-find method. Traverse each vertex in the graph, and if it is found that the current vertex and its corresponding adjacent nodes are in the same set, it means that it is not a bipartite graph. Otherwise, merge the adjacent nodes of the current node.

The time complexity is O(n × log n), and the space complexity is O(n). Where n is the number of nodes.

Python 3

```
class Solution:
    def isBipartite(self, graph: List[List[int]]) -> bool:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(len(graph)))
        for a, bs in enumerate(graph):
            for b in bs:
                pa, pb = find(a), find(b)
                if pa == pb:
                    return False
                p[pb] = find(bs[0])
        return True
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the two sides
# 2. If it is not bipartite, return an odd cycle


# ============================================================
# FOLLOW-UP 1: Return the two sides
# ============================================================

# ---- follow-up 1: function ----
# Answer: Colour with two colours by search; the colours
# are the sides.
# Time: O(V + E).  Space: O(V).
def two_sides(graph):
    colour = {}
    for start in range(len(graph)):
        if start in colour:
            continue
        colour[start] = 0
        stack = [start]
        while stack:
            node = stack.pop()
            for nxt in graph[node]:
                if nxt not in colour:
                    colour[nxt] = 1 - colour[node]
                    stack.append(nxt)
                elif colour[nxt] == colour[node]:
                    return None
    return [i for i in colour if colour[i] == 0], [i for i in colour if colour[i] == 1]


# ============================================================
```

```
# FOLLOW-UP 2: If it is not bipartite, return an odd cycle
# ========================================================

# ---- follow-up 2: function ----
# Answer: Search with parents; an edge between two nodes
# of the same colour closes an odd cycle.
# Time: O(V + E).  Space: O(V).
def odd_cycle(graph):
    colour = {}
    parent = {}
    for start in range(len(graph)):
        if start in colour:
            continue
        colour[start] = 0
        parent[start] = None
        stack = [start]
        while stack:
            node = stack.pop()
            for nxt in graph[node]:
                if nxt not in colour:
                    colour[nxt] = 1 - colour[node]
                    parent[nxt] = node
                    stack.append(nxt)
                elif colour[nxt] == colour[node]:
                    left, right = [node], [nxt]
                    while parent[left[-1]] is not None:
                        left.append(parent[left[-1]])
                    while parent[right[-1]] is not None:
```

```
                        right.append(parent[right[-1]])
                    while len(left) > 1 and len(right) > 1 and left[-2] == right[-2]:
                        left.pop()
                        right.pop()
                    return left + right[-2::-1]
    return None
```

1192. Critical Connections in a Network

Hard · Depth-First Search, Graph, Bridge, Biconnected Component

leetcode.com/problems/critical-connections-in-a-network

★ **TOP 7 PROBLEM** in Depth First Search (DFS): master this one first

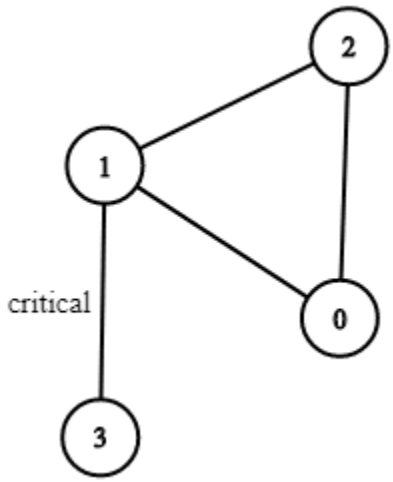
PROBLEM

There are `n` servers numbered from `0` to `n - 1` connected by undirected server-to-server `connections` forming a network where `connections[i] = [a_i, b_i]` represents a connection between servers a_i and b_i. Any server can reach other servers directly or indirectly through the network.

A *critical connection* is a connection that, if removed, will make some servers unable to reach some other server.

Return all critical connections in the network in any order.

Example 1:




```
Input: n = 4, connections = [[0,1],[1,2],[2,0],[1,3]]
Output: [[1,3]]
Explanation: [[3,1]] is also accepted.
```

Example 2:

```
Input: n = 2, connections = [[0,1]]
Output: [[0,1]]
```

Constraints:

- `2 <= n <= 10^5`
- `n - 1 <= connections.length <= 10^5`
- `0 <= a_i, b_i <= n - 1`
- `a_i != b_i`
- There are no repeated connections.

SOLUTION (PYTHON 3)

Solution 1: Tarjan Algorithm

How to think about it

A critical connection is a bridge. Testing each edge with union-find or BFS is too slow on a large edge set. Tarjan's DFS keeps discovery time dfn and earliest ancestor low; a tree edge with low[v]>dfn[u] is a bridge. Skip the parent in an undirected graph so the tree edge is not treated as a back edge.

The "critical connections" in this problem can be considered as "bridges".

"Bridges": In a connected undirected graph, if removing a certain edge makes the graph disconnected, then this edge can be considered as a "bridge".

Correspondingly, there is also the concept of "articulation points".

"Articulation Points": In a connected undirected graph, if removing a certain point and all edges connected to it makes the graph disconnected, then this point can be considered as an "articulation point".

There is an algorithm called the Tarjan algorithm for finding "bridges" and "articulation points" in a graph. This algorithm uses a depth-first search (DFS) method that first recursively visits adjacent nodes and then visits the node itself. By recording the "order of visit: DFN" and updating the "earliest backtrackable node: low" when visiting the node itself after recursion ends, it can find the "bridges" and "articulation points" of the graph in O(n) time. Also, this algorithm can be used to find strongly connected components in directed graphs.

Python 3

```
class Solution:
    def criticalConnections(
        self, n: int, connections: List[List[int]]
    ) -> List[List[int]]:
        def tarjan(a: int, fa: int):
            nonlocal now
            now += 1
            dfn[a] = low[a] = now
            for b in g[a]:
                if b == fa:
                    continue
                if not dfn[b]:
                    tarjan(b, a)
                    low[a] = min(low[a], low[b])
                    if low[b] > dfn[a]:
                        ans.append([a, b])
                else:
                    low[a] = min(low[a], dfn[b])

        g = [[] for _ in range(n)]
        for a, b in connections:
            g[a].append(b)
            g[b].append(a)

        dfn = [0] * n
        low = [0] * n
        now = 0
        ans = []
        tarjan(0, -1)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The articulation points (servers whose removal
# disconnects the network)
# 2. How many groups remain if every critical connection
# is cut (2-edge-connected components)


# ============================================================
# FOLLOW-UP 1: The articulation points (servers whose
# removal disconnects the network)
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same low-link search: a node is critical if
# a child cannot reach above it, or it is a root with two
# children.
# Time: O(V + E).  Space: O(V + E).
def articulation_points(n, connections):
    graph = [[] for _ in range(n)]
    for a, b in connections:
        graph[a].append(b)
        graph[b].append(a)
    order = [-1] * n
    low = [0] * n
    points = set()
    timer = 0

    def go(node, parent):
        nonlocal timer
        order[node] = low[node] = timer
        timer += 1
        children = 0
        for nxt in graph[node]:
            if nxt == parent:
                continue
            if order[nxt] == -1:
                children += 1
                go(nxt, node)
                low[node] = min(low[node], low[nxt])
                if parent != -1 and low[nxt] >= order[node]:
                    points.add(node)
            else:
                low[node] = min(low[node], order[nxt])
        if parent == -1 and children >= 2:
            points.add(node)

    for i in range(n):
        if order[i] == -1:
            go(i, -1)
    return sorted(points)


# ============================================================
# FOLLOW-UP 2: How many groups remain if every critical
# connection is cut (2-edge-connected components)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Remove the bridges, then count the connected
# components.
```

```
# Time: O(V + E).  Space: O(V + E).
def components_after_cutting_bridges(n, connections):
    graph = [[] for _ in range(n)]
    for a, b in connections:
```

```
        graph[a].append(b)
        graph[b].append(a)
    order = [-1] * n
    low = [0] * n
    bridges = set()
    timer = 0

    def go(node, parent):
        nonlocal timer
        order[node] = low[node] = timer
        timer += 1
        for nxt in graph[node]:
            if nxt == parent:
                continue
            if order[nxt] == -1:
                go(nxt, node)
                low[node] = min(low[node], low[nxt])
                if low[nxt] > order[node]:
                    bridges.add((min(node, nxt), max(node, nxt)))
            else:
                low[node] = min(low[node], order[nxt])

    for i in range(n):
        if order[i] == -1:
            go(i, -1)
    seen = [False] * n
    groups = 0
    for s in range(n):
        if seen[s]:
            continue
        groups += 1
        seen[s] = True
        stack = [s]
        while stack:
            x = stack.pop()
            for y in graph[x]:
                if not seen[y] and (min(x, y), max(x, y)) not in bridges:
                    seen[y] = True
                    stack.append(y)
    return groups
```

2246. Longest Path With Different Adjacent Characters

Hard · Tree, Depth-First Search, Graph, Topological Sort, Array, String

leetcode.com/problems/longest-path-with-different-adjacent-characters

★ **TOP 7 PROBLEM** in Depth First Search (DFS): master this one first

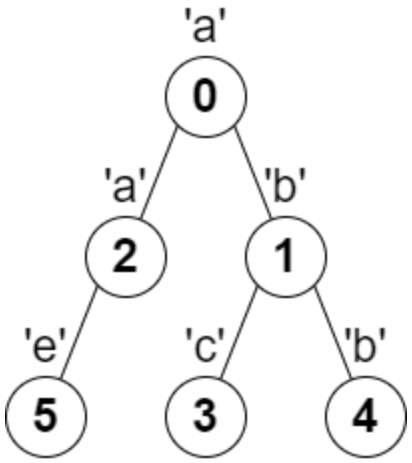
PROBLEM

You are given a **tree** (i.e. a connected, undirected graph that has no cycles) **rooted** at node `0` consisting of `n` nodes numbered from `0` to `n - 1`. The tree is represented by a **0-indexed** array `parent` of size `n`, where `parent[i]` is the parent of node `i`. Since node `0` is the root, `parent[0] == -1`.

You are also given a string `s` of length `n`, where `s[i]` is the character assigned to node `i`.

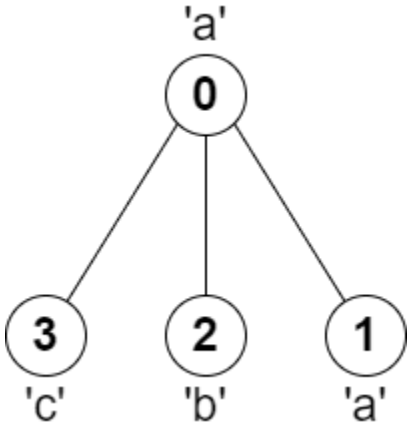
Return *the length of the **longest path** in the tree such that no pair of **adjacent** nodes on the path have the same character assigned to them.*

Example 1:




```
Input: parent = [-1,0,0,1,1,2], s = "abacbe"
Output: 3
Explanation: The longest path where each two adjacent nodes have different characters in the tree is
the path: 0 -> 1 -> 3. The length of this path is 3, so 3 is returned.
It can be proven that there is no longer path that satisfies the conditions.
```

Example 2:



```
Input: parent = [-1,0,0,0], s = "aabc"
Output: 3
Explanation: The longest path where each two adjacent nodes have different characters is the path: 2
-> 0 -> 3. The length of this path is 3, so 3 is returned.
```

Constraints:

- `n == parent.length == s.length`
- `1 <= n <= 10^5`
- `0 <= parent[i] <= n - 1` for all `i >= 1`
- `parent[0] == -1`
- `parent` represents a valid tree.
- `s` consists of only lowercase English letters.

SOLUTIONS (PYTHON 3)

Solution 1: Tree-shaped DP

How to think about it

We want the longest tree path whose adjacent labels differ. $n \le 10^5$ forbids pairing endpoints. A path either stays in one subtree or is the join of two downward chains at a vertex. DFS returns the longest downward chain whose first step has a different character. For each child, join the current best chain with the new one when labels differ, and keep the best downward length. Adding one at the end counts the node itself.

First, we construct an adjacency list g based on the array parent, where g[i] represents all child nodes of node i.

Then we start DFS from the root node. For each node i, we traverse each child node j in g[i]. If s[i] ≠ s[j], then we can start from node i, pass through node j, and reach a leaf node. The length of this path is x = 1 + dfs(j). We use mx to record the longest path length starting from node i. At the same time, we update the answer ans = max(ans, mx + x) during the traversal process.

Finally, we return ans + 1.

The time complexity is O(n), and the space complexity is O(n), where n is the number of nodes.

Python 3

```python
class Solution:
    def longestPath(self, parent: List[int], s: str) -> int:
        def dfs(i: int) -> int:
            mx = 0
            nonlocal ans
            for j in g[i]:
                x = dfs(j) + 1
                if s[i] != s[j]:
                    ans = max(ans, mx + x)
                    mx = max(mx, x)
            return mx

        g = defaultdict(list)
        for i in range(1, len(parent)):
            g[parent[i]].append(i)
        ans = 0
        dfs(0)
        return ans + 1
```

Solution 2: Tree DP + Explicit Stack

How to think about it

We want the longest tree path whose adjacent labels differ. With $n \le 10^5$, recursing from the root for every downward chain is too deep: a chain makes the call depth n. Such a path either stays inside one subtree or joins two downward chains at one vertex. When a node is left, each child's downward length is already known. The chain can be attached only when s[i] ≠ s[j], and its length is the child's length plus one. Join that chain with the best one already seen, then keep the longer of the two as this node's downward chain. An explicit stack of (node, state) runs the postorder. On entry we push the exit marker and then the children, and on exit we update the answer and the downward length. Adding one at the end counts the nodes on the path.

First, we construct an adjacency list g based on the array parent, where g[i] represents all child nodes of node i.

An explicit stack walks from the root in postorder. When node i is left, each child j contributes x, one more than the longest chain downward from j. If s[i] ≠ s[j], update the answer with ans = max(ans, mx + x) and then set mx = max(mx, x). Here mx is the longest downward chain from i whose first step has a different character.

Finally, we return ans + 1.

The time complexity is O(n), and the space complexity is O(n), where n is the number of nodes.

Python 3

```python
class Solution:
    def longestPath(self, parent: List[int], s: str) -> int:
        n = len(parent)
        g = [[] for _ in range(n)]
        for i in range(1, n):
            g[parent[i]].append(i)
        down = [0] * n
        ans = 0
        stk = [(0, 0)]
        while stk:
            i, state = stk.pop()
            if state == 0:
                stk.append((i, 1))
                for j in g[i]:
                    stk.append((j, 0))
            else:
                mx = 0
                for j in g[i]:
                    x = down[j] + 1
                    if s[i] != s[j]:
                        ans = max(ans, mx + x)
                        mx = max(mx, x)
                down[i] = mx
        return ans + 1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the path itself
# 2. The longest path where all neighbouring characters
# are the SAME


# ============================================================
# FOLLOW-UP 1: Return the path itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep, for every node, the longest downward path
# as a list.
# Time: O(n^2) in the worst case (whole paths are copied
```

```
# and sorted).  Space: O(n).
def longest_path_nodes(parent, s):
    n = len(parent)
    children = [[] for _ in range(n)]
    for i in range(1, n):
        children[parent[i]].append(i)
    best = []

    def go(node):
        nonlocal best
        down = []
        for c in children[node]:
            sub = go(c)
            if s[c] != s[node]:
                down.append(sub)
        down.sort(key=len, reverse=True)
        first = down[0] if down else []
        second = down[1] if len(down) > 1 else []
        whole = second[::-1] + [node] + first
        if len(whole) > len(best):
            best = whole
        return [node] + first

    go(0)
    return best


# ============================================================
# FOLLOW-UP 2: The longest path where all neighbouring
# characters are the SAME
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same recursion with the condition reversed.
# Time: O(n).  Space: O(h).
def longest_path_same_chars(parent, s):
    n = len(parent)
    children = [[] for _ in range(n)]
    for i in range(1, n):
        children[parent[i]].append(i)
    best = 1

    def go(node):
        nonlocal best
        first = second = 0
        for c in children[node]:
```

```
            sub = go(c)
            if s[c] == s[node]:
                if sub > first:
                    first, second = sub, first
                elif sub > second:
                    second = sub
        best = max(best, first + second + 1)
        return first + 1

    go(0)
    return best
```

PART 2 · TOPIC 32 OF 50

Breadth First Search (BFS)

4 problems · 0 Easy · 3 Medium · 1 Hard

THE IDEA

Explore level by level: the first time you reach a node is by the fewest steps. Add a state (keys, obstacles removed) when steps are not the whole story.

THE 4 PROBLEMS, EASY TO HARD | ★ = TOP 4: DO THESE FIRST

1	★ **Walls and Gates**	Medium	LC 286	2495
2	★ **Path With Maximum Minimum Value**	Medium	LC 1102	2498
3	★ **Minimum Knight Moves**	Medium	LC 1197	2503
4	★ **Minimum Obstacle Removal to Reach Corner**	Hard	LC 2290	2507

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Rotting Oranges (Part 1, pattern 17, page 924)
Open the Lock (Part 1, pattern 17, page 934)
Minimum Genetic Mutation (Part 1, pattern 17, page 956)
Shortest Path in a Grid with Obstacles Elimination (Part 1, pattern 17, page 989)
Word Ladder (Part 1, pattern 17, page 974)
01 Matrix (Part 1, pattern 17, page 927)
Snakes and Ladders (Part 1, pattern 17, page 939)
Shortest Path in Binary Matrix (Part 1, pattern 17, page 931)
Bus Routes (Part 1, pattern 17, page 978)

286. Walls and Gates

Medium · Breadth-First Search, Array, Matrix

leetcode.com/problems/walls-and-gates

★ **TOP 7 PROBLEM** in Breadth First Search (BFS): master this one first

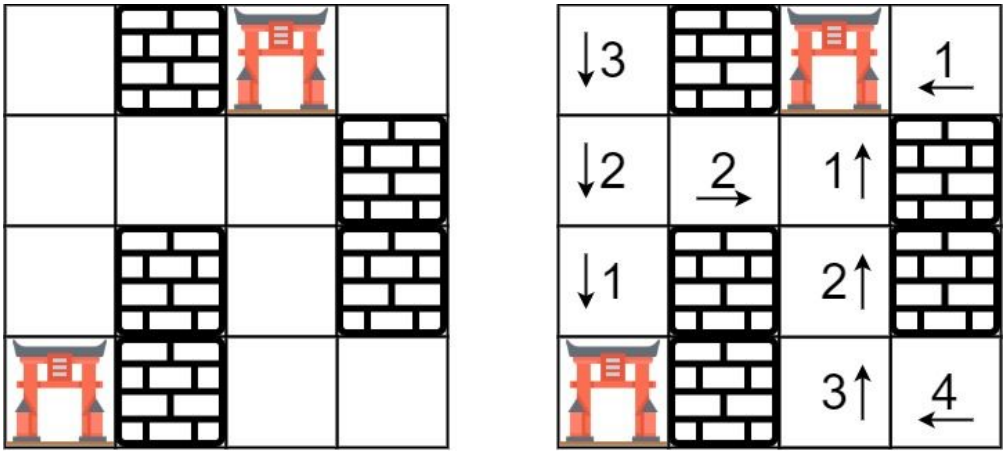
PROBLEM

You are given an `m x n` grid `rooms` initialized with these three possible values.

- `-1` A wall or an obstacle.
- `0` A gate.
- `INF` Infinity means an empty room. We use the value $2^{31} - 1 = 2147483647$ to represent `INF` as you may assume that the distance to a gate is less than `2147483647`.

Fill each empty room with the distance to *its nearest gate*. If it is impossible to reach a gate, it should be filled with `INF`.

Example 1:




```
Input: rooms = [[2147483647,-1,0,2147483647],[2147483647,2147483647,2147483647,-1],[2147483647,-1,2147483647,-1],[0,-1,2147483647,2147483647]]
Output: [[3,-1,0,1],[2,2,1,-1],[1,-1,2,-1],[0,-1,3,4]]
```

Example 2:

```
Input: rooms = [[-1]]
Output: [[-1]]
```

Constraints:

- `m == rooms.length`
- `n == rooms[i].length`
- `1 <= m, n <= 250`
- `rooms[i][j]` is `-1`, `0`, or $2^{31} - 1$.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Each empty room wants the distance to the nearest gate. A BFS from every room repeats work. A multi-source BFS from all gates reaches each room first at its shortest distance. Enqueue every gate and expand level by level, writing only into rooms that are still empty.

Python 3

```
class Solution:
    def wallsAndGates(self, rooms: List[List[int]]) -> None:
        """
        Do not return anything, modify rooms in-place instead.
        """
        m, n = len(rooms), len(rooms[0])
        inf = 2**31 - 1
        q = deque([(i, j) for i in range(m) for j in range(n) if rooms[i][j] == 0])
        d = 0
        while q:
            d += 1
            for _ in range(len(q)):
                i, j = q.popleft()
                for a, b in [[0, 1], [0, -1], [1, 0], [-1, 0]]:
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n and rooms[x][y] == inf:
                        rooms[x][y] = d
                        q.append((x, y))
```

Complexity. Time: O(m * n). Space: O(m * n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. For every room, which gate is the nearest (its
# position)
# 2. How many empty rooms cannot reach any gate


# ============================================================
# FOLLOW-UP 1: For every room, which gate is the nearest
# (its position)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Multi-source search that carries the gate with
# each cell.
# Time: O(m * n).  Space: O(m * n).
from collections import deque


def nearest_gate(rooms):
    rows, cols = len(rooms), len(rooms[0])
    gate = [[None] * cols for _ in range(rows)]
    queue = deque()
    for r in range(rows):
        for c in range(cols):
            if rooms[r][c] == 0:
                gate[r][c] = (r, c)
                queue.append((r, c))
    while queue:
        r, c = queue.popleft()
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
```

```
            if 0 <= nr < rows and 0 <= nc < cols and rooms[nr][nc] > 0 and gate[nr][nc] is None:
                gate[nr][nc] = gate[r][c]
                queue.append((nr, nc))
    return gate


# ============================================================
# FOLLOW-UP 2: How many empty rooms cannot reach any gate
# ============================================================

# ---- follow-up 2: function ----
# Answer: Run the search and count the rooms left at
# infinity.
# Time: O(m * n).  Space: O(m * n).
def unreachable_rooms(rooms):
    INF = 2147483647
    rows, cols = len(rooms), len(rooms[0])
    seen = [[False] * cols for _ in range(rows)]
    queue = deque()
    for r in range(rows):
        for c in range(cols):
            if rooms[r][c] == 0:
                seen[r][c] = True
                queue.append((r, c))
    while queue:
        r, c = queue.popleft()
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and rooms[nr][nc] == INF and not seen[nr][nc]:
```

```
                seen[nr][nc] = True
                queue.append((nr, nc))
    return sum(1 for r in range(rows) for c in range(cols) if rooms[r][c] == INF and not seen[r][c])
```

1102. Path With Maximum Minimum Value

Medium · Depth-First Search, Breadth-First Search, Union Find, Array, Binary Search, Matrix, Dijkstra, Heap (Priority Queue)

leetcode.com/problems/path-with-maximum-minimum-value

★ **TOP 7 PROBLEM** in Breadth First Search (BFS): master this one first

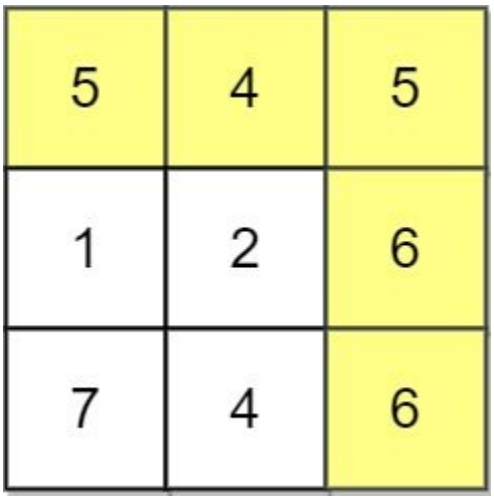
PROBLEM

Given an `m x n` integer matrix `grid`, return *the maximum **score** of a path starting at* `(0, 0)` *and ending at* `(m - 1, n - 1)` moving in the 4 cardinal directions.

The **score** of a path is the minimum value in that path.

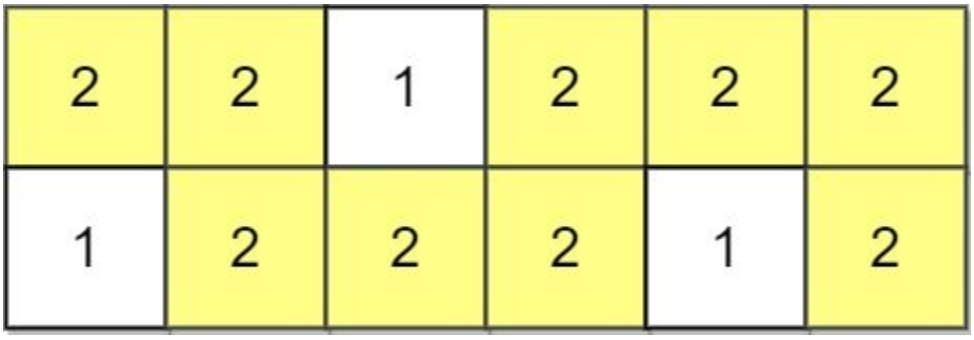
- For example, the score of the path `8 → 4 → 5 → 9` is `4`.

Example 1:



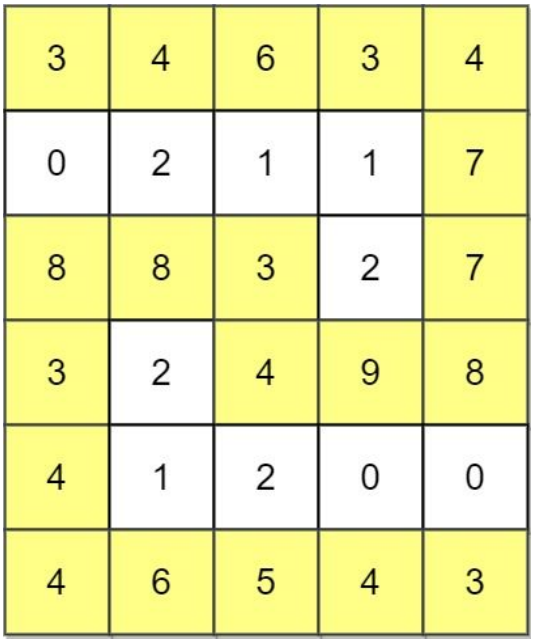

```
Input: grid = [[5,4,5],[1,2,6],[7,4,6]]
Output: 4
Explanation: The path with the maximum score is highlighted in yellow.
```

Example 2:




```
Input: grid = [[2,2,1,2,2,2],[1,2,2,2,1,2]]
Output: 2
```

Example 3:




```
Input: grid = [[3,4,6,3,4],[0,2,1,1,7],[8,8,3,2,7],[3,2,4,9,8],[4,1,2,0,0],[4,6,5,4,3]]
Output: 3
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 100`
- `0 <= grid[i][j] <= 10^9`

SOLUTIONS (PYTHON 3)

Solution 1: Sorting + Union-Find

> **How to think about it**
>
> The score of a path is its minimum cell, and we want the maximum such score from the top-left to the bottom-right. Enumerating all paths is not viable. Insert cells in decreasing value and union each with already-inserted neighbors. The value just inserted is the lowest score allowed so far; the first time the start and end share a component, no smaller cell can raise that minimum, so that value is the answer.

First, we construct a triplet (v, i, j) for each element in the matrix, where v represents the element value, and i and j represent the row and column of the element in the matrix, respectively. Then we sort these triplets in descending order by element value and store them in a list q.

Next, we take out the triplets from q in order, use the corresponding element value as the score of the path, and mark the position as visited. Then we check the four adjacent positions (up, down, left, and right) of this position. If an adjacent position has been visited, we merge this position with the current position. If we find that the position (0, 0) and the position (m - 1, n - 1) have been merged, we can directly return the score of the current path as the answer.

The time complexity is $O(m \times n \times (\log (m \times n) + \alpha(m \times n)))$, where m and n are the number of rows and columns of the matrix, respectively.

Python 3

```python
class Solution:
    def maximumMinimumPath(self, grid: List[List[int]]) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        m, n = len(grid), len(grid[0])
        p = list(range(m * n))
        q = [(v, i, j) for i, row in enumerate(grid) for j, v in enumerate(row)]
        q.sort()
        ans = 0
        dirs = (-1, 0, 1, 0, -1)
        vis = set()
        while find(0) != find(m * n - 1):
            v, i, j = q.pop()
            ans = v
            vis.add((i, j))
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if (x, y) in vis:
                    p[find(i * n + j)] = find(x * n + y)
        return ans
```

Solution 2

How to think about it

Method 1 path-compresses on a raw parent array. Method 2 uses a union-by-size `UnionFind`, which keeps trees shallower and the API regular. Cells are still inserted in decreasing order, stopping when start and end become connected.

Python 3

```python
class UnionFind:
    __slots__ = ("p", "size")

    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x: int) -> int:
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True


class Solution:
    def maximumMinimumPath(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        uf = UnionFind(m * n)
        q = [(v, i, j) for i, row in enumerate(grid) for j, v in enumerate(row)]
        q.sort()
        ans = 0
        vis = set()
        dirs = (-1, 0, 1, 0, -1)
        while uf.find(0) != uf.find(m * n - 1):
            v, i, j = q.pop()
            ans = v
            vis.add((i, j))
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if (x, y) in vis:
                    uf.union(x * n + y, i * n + j)
        return ans
```

Complexity. Time: O(m * n * log(m * n)). Space: O(m * n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the path of cells
# 2. The path whose MAXIMUM value is as small as possible


# ============================================================
# FOLLOW-UP 1: Return the path of cells
# ============================================================

# ---- follow-up 1: function ----
# Answer: Best-first search (heap) remembering the
# previous cell.
# Time: O(m * n log (m * n)).  Space: O(m * n).
import heapq


def best_path_cells(grid):
    rows, cols = len(grid), len(grid[0])
```

```
    heap = [(-grid[0][0], 0, 0)]                # the heap key is minus the smallest value on the path so
 far
    previous = {(0, 0): None}
    while heap:
        key, r, c = heapq.heappop(heap)
        if (r, c) == (rows - 1, cols - 1):
            break
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and (nr, nc) not in previous:
                previous[(nr, nc)] = (r, c)
                heapq.heappush(heap, (-min(-key, grid[nr][nc]), nr, nc))
    path = []
    cell = (rows - 1, cols - 1)
    while cell is not None:
        path.append(cell)
        cell = previous[cell]
    path.reverse()
    return min(grid[r][c] for r, c in path), path


# ========================================================
# FOLLOW-UP 2: The path whose MAXIMUM value is as small as
# possible
# ========================================================

# ---- follow-up 2: function ----
# Answer: Same best-first search with the smallest maximum
# so far on top of the heap.
# Time: O(m * n log (m * n)).  Space: O(m * n).
def min_of_max_on_path(grid):
    rows, cols = len(grid), len(grid[0])
    heap = [(grid[0][0], 0, 0)]
    seen = {(0, 0)}
    while heap:
        value, r, c = heapq.heappop(heap)
        if (r, c) == (rows - 1, cols - 1):
            return value
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and (nr, nc) not in seen:
                seen.add((nr, nc))
                heapq.heappush(heap, (max(value, grid[nr][nc]), nr, nc))
```

1197. Minimum Knight Moves

Medium · Breadth-First Search, Bidirectional Search, A* Search, Heuristic Search

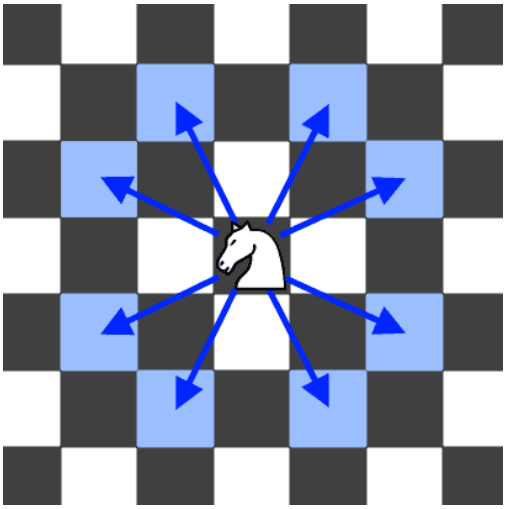
leetcode.com/problems/minimum-knight-moves

★ **TOP 7 PROBLEM** in Breadth First Search (BFS): master this one first

PROBLEM

In an **infinite** chess board with coordinates from `-infinity` to `+infinity`, you have a **knight** at square `[0, 0]`.

A knight has 8 possible moves it can make, as illustrated below. Each move is two squares in a cardinal direction, then one square in an orthogonal direction.



Return *the minimum number of steps needed to move the knight to the square* `[x, y]`. It is guaranteed the answer exists.

Example 1:

```
Input: x = 2, y = 1
Output: 1
Explanation: [0, 0] → [2, 1]
```

Example 2:

```
Input: x = 5, y = 5
Output: 4
Explanation: [0, 0] → [2, 1] → [4, 2] → [3, 4] → [5, 5]
```

Constraints:

- `-300 <= x, y <= 300`
- `0 <= |x| + |y| <= 300`

SOLUTIONS (PYTHON 3)

Solution 1: BFS

How to think about it

A knight has eight jumps; the minimum number of moves is a grid shortest path. BFS from (0,0) returns the layer that first hits (x,y). The coordinate range is bounded; a visited set prevents requeueing. Quadrant symmetry can shrink the search, but plain BFS already passes.

This problem can be solved using the BFS shortest path model. The search space for this problem is not large, so we can directly use the naive BFS. The solution below also provides the code for bidirectional BFS for reference.

Bidirectional BFS is a common optimization method for BFS. The main implementation ideas are as follows:

1. Create two queues, q1 and q2, for "start -> end" and "end -> start" search directions, respectively.
2. Create two hash maps, m1 and m2, to record the visited nodes and their corresponding expansion times (steps).
3. During each search, prioritize the queue with fewer elements for search expansion. If a node visited from the other direction is found during the expansion, it means the shortest path has been found.
4. If one of the queues is empty, it means that the search in the current direction cannot continue, indicating that the start and end points are not connected, and there is no need to continue the search.

Python 3

```python
class Solution:
    def minKnightMoves(self, x: int, y: int) -> int:
        q = deque([(0, 0)])
        ans = 0
        vis = {(0, 0)}
        dirs = ((-2, 1), (-1, 2), (1, 2), (2, 1), (2, -1), (1, -2), (-1, -2), (-2, -1))
        while q:
            for _ in range(len(q)):
                i, j = q.popleft()
                if (i, j) == (x, y):
                    return ans
                for a, b in dirs:
                    c, d = i + a, j + b
                    if (c, d) not in vis:
                        vis.add((c, d))
                        q.append((c, d))
            ans += 1
        return -1
```

Complexity. Time: O(max(|x|, |y|)^2). Space: O(max(|x|, |y|)^2).

Solution 2: Bidirectional BFS

> **How to think about it**
>
> Method 1 grows from one side and the frontier widens far from the origin. Bidirectional BFS expands from both ends, always stepping the smaller queue; the first shared cell is the shortest path and typically visits fewer cells.

Search knight jumps from both ends until the two sides meet.

Python 3

```
class Solution:
    def minKnightMoves(self, x: int, y: int) -> int:
        def extend(m1, m2, q):
            for _ in range(len(q)):
                i, j = q.popleft()
                step = m1[(i, j)]
                for a, b in (
                    (-2, 1),
                    (-1, 2),
                    (1, 2),
                    (2, 1),
                    (2, -1),
                    (1, -2),
                    (-1, -2),
                    (-2, -1),
                ):
                    x, y = i + a, j + b
                    if (x, y) in m1:
                        continue
                    if (x, y) in m2:
                        return step + 1 + m2[(x, y)]
                    q.append((x, y))
                    m1[(x, y)] = step + 1
            return -1

        if (x, y) == (0, 0):
            return 0
        q1, q2 = deque([(0, 0)]), deque([(x, y)])
        m1, m2 = {(0, 0): 0}, {(x, y): 0}
        while q1 and q2:
            t = extend(m1, m2, q1) if len(q1) <= len(q2) else extend(m2, m1, q2)
            if t != -1:
                return t
        return -1
```

Complexity. Time: O(max(|x|, |y|)^2) in the worst case, but much faster in practice (two searches meet in the middle). Space: O(max(|x|, |y|)^2).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the sequence of squares
# 2. The board is limited to an n by n square starting at
# (0, 0)


# ============================================================
# FOLLOW-UP 1: Return the sequence of squares
# ============================================================

# ---- follow-up 1: function ----
# Answer: Breadth-first search remembering where every
# square came from.
# Time: O(|x| * |y|).  Space: O(|x| * |y|).
from collections import deque


def knight_path(x, y):
    moves = ((1, 2), (2, 1), (-1, 2), (-2, 1), (1, -2), (2, -1), (-1, -2), (-2, -1))
    previous = {(0, 0): None}
    queue = deque([(0, 0)])
    while queue:
        cx, cy = queue.popleft()
        if (cx, cy) == (x, y):
            break
        for dx, dy in moves:
            nxt = (cx + dx, cy + dy)
```

```
            if nxt not in previous and abs(nxt[0]) <= abs(x) + 2 and abs(nxt[1]) <= abs(y) + 2:
                previous[nxt] = (cx, cy)
                queue.append(nxt)
    path = []
    cell = (x, y)
    while cell is not None:
        path.append(cell)
        cell = previous[cell]
    return path[::-1]


# ============================================================
# FOLLOW-UP 2: The board is limited to an n by n square
# starting at (0, 0)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same search, staying inside the board (return -1
# when it is unreachable).
# Time: O(n^2).  Space: O(n^2).
def knight_moves_on_board(n, tx, ty):
    moves = ((1, 2), (2, 1), (-1, 2), (-2, 1), (1, -2), (2, -1), (-1, -2), (-2, -1))
    distance = {(0, 0): 0}
    queue = deque([(0, 0)])
    while queue:
        x, y = queue.popleft()
        if (x, y) == (tx, ty):
            return distance[(x, y)]
        for dx, dy in moves:
            nx, ny = x + dx, y + dy
            if 0 <= nx < n and 0 <= ny < n and (nx, ny) not in distance:
                distance[(nx, ny)] = distance[(x, y)] + 1
                queue.append((nx, ny))
```

```
    return -1
```

2290. Minimum Obstacle Removal to Reach Corner

Hard · Breadth-First Search, Graph, Array, Matrix, Shortest Path, Dijkstra, Heap (Priority Queue), 0-1 BFS

leetcode.com/problems/minimum-obstacle-removal-to-reach-corner

★ **TOP 7 PROBLEM** in Breadth First Search (BFS): master this one first

PROBLEM

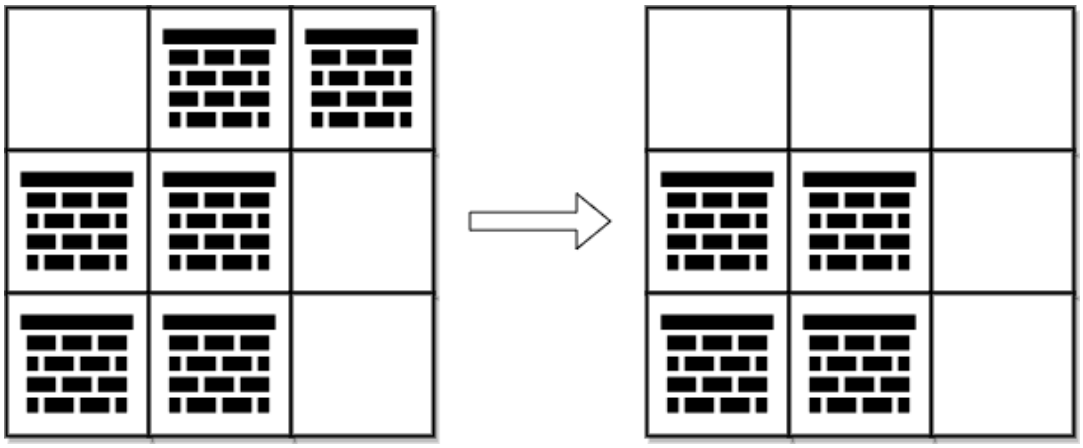
You are given a **0-indexed** 2D integer array `grid` of size `m x n`. Each cell has one of two values:

- `0` represents an **empty** cell,
- `1` represents an **obstacle** that may be removed.

You can move up, down, left, or right from and to an empty cell.

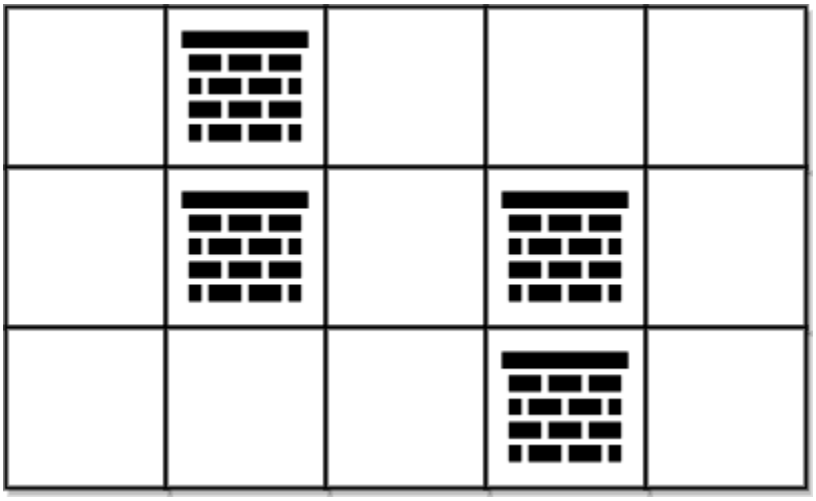
Return *the **minimum** number of **obstacles** to **remove** so you can move from the upper left corner* `(0, 0)` *to the lower right corner* `(m - 1, n - 1)`.

Example 1:



```
Input: grid = [[0,1,1],[1,1,0],[1,1,0]]
Output: 2
Explanation: We can remove the obstacles at (0, 1) and (0, 2) to create a path from (0, 0) to (2, 2).
It can be shown that we need to remove at least 2 obstacles, so we return 2.
Note that there may be other ways to remove 2 obstacles to create a path.
```

Example 2:



```
Input: grid = [[0,1,0,0,0],[0,1,0,1,0],[0,0,0,1,0]]
Output: 0
Explanation: We can move from (0, 0) to (2, 4) without removing any obstacles, so we return 0.
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 10^5`
- `2 <= m * n <= 10^5`
- `grid[i][j]` is either `0` **or** `1`.
- `grid[0][0] == grid[m - 1][n - 1] == 0`

SOLUTION (PYTHON 3)

Solution 1: Double-Ended Queue BFS

How to think about it

We walk from the top-left to the bottom-right; entering an obstacle costs one removal. mn ≤ 10^5 forbids searching subsets. Empty cells have weight 0 and obstacles weight 1, so this is a 0-1 shortest path for a deque BFS. Push a neighbor to the front when the cell is empty, and to the back when it is an obstacle. The first time we reach the exit is the minimum number of removals.

This problem is essentially a shortest path model, but we need to find the minimum number of obstacles to remove.

In an undirected graph with edge weights of only 0 and 1, we can use a double-ended queue to perform BFS. The principle is that if the weight of the current point that can be expanded is 0, it is added to the front of the queue; if the weight is 1, it is added to the back of the queue.

If the weight of an edge is 0, then the newly expanded node has the same weight as the current front node, and it can obviously be used as the starting point for the next expansion.

The time complexity is O(m × n), and the space complexity is O(m × n). Where m and n are the number of rows and columns of the grid, respectively.

Similar problems:

- 1368. Minimum Cost to Make at Least One Valid Path in a Grid

Python 3

```python
class Solution:
    def minimumObstacles(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        q = deque([(0, 0, 0)])
        vis = set()
        dirs = (-1, 0, 1, 0, -1)
        while 1:
            i, j, k = q.popleft()
            if i == m - 1 and j == n - 1:
                return k
            if (i, j) in vis:
                continue
            vis.add((i, j))
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n:
                    if grid[x][y] == 0:
                        q.appendleft((x, y, k))
                    else:
                        q.append((x, y, k + 1))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the cells whose obstacle is removed
# 2. Diagonal moves are also allowed


# ============================================================
# FOLLOW-UP 1: Return the cells whose obstacle is removed
# ============================================================

# ---- follow-up 1: function ----
# Answer: 0-1 search remembering the previous cell, then
# list the obstacle cells on the path.
# Time: O(m * n).  Space: O(m * n).
from collections import deque


def removed_obstacles(grid):
    rows, cols = len(grid), len(grid[0])
    INF = float("inf")
    cost = [[INF] * cols for _ in range(rows)]
    previous = {}
    cost[0][0] = grid[0][0]
    queue = deque([(0, 0)])
    while queue:
        r, c = queue.popleft()
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols:
                new = cost[r][c] + grid[nr][nc]
                if new < cost[nr][nc]:
                    cost[nr][nc] = new
                    previous[(nr, nc)] = (r, c)
                    if grid[nr][nc] == 0:
                        queue.appendleft((nr, nc))
                    else:
                        queue.append((nr, nc))
    cell = (rows - 1, cols - 1)
    removed = []
    while cell != (0, 0):
        if grid[cell[0]][cell[1]] == 1:
            removed.append(cell)
        cell = previous[cell]
    return sorted(removed)


# ============================================================
# FOLLOW-UP 2: Diagonal moves are also allowed
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same 0-1 search with eight directions.
# Time: O(m * n).  Space: O(m * n).
def min_removals_eight_directions(grid):
    rows, cols = len(grid), len(grid[0])
    INF = float("inf")
    cost = [[INF] * cols for _ in range(rows)]
    cost[0][0] = grid[0][0]
    queue = deque([(0, 0)])
    while queue:
        r, c = queue.popleft()
```

```
        for dr in (-1, 0, 1):
            for dc in (-1, 0, 1):
                nr, nc = r + dr, c + dc
                if (dr or dc) and 0 <= nr < rows and 0 <= nc < cols:
```

```
                    new = cost[r][c] + grid[nr][nc]
                    if new < cost[nr][nc]:
                        cost[nr][nc] = new
                        if grid[nr][nc] == 0:
                            queue.appendleft((nr, nc))
                        else:
                            queue.append((nr, nc))
    return cost[rows - 1][cols - 1]
```

Topological Sort

1 problem · 0 Easy · 0 Medium · 1 Hard

THE IDEA

Order the nodes of a directed acyclic graph so every edge goes forward. Repeatedly take nodes with no remaining incoming edges.

THE 1 PROBLEMS, EASY TO HARD | ★ = TOP 1: DO THESE FIRST

1	★ **Alien Dictionary**	Hard	LC 269	2512

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Course Schedule II (Part 1, pattern 30, page 1595)

Parallel Courses (Part 1, pattern 30, page 1609)

Minimum Height Trees (Part 1, pattern 30, page 1602)

Sort Items by Groups Respecting Dependencies (Part 1, pattern 30, page 1637)

Course Schedule IV (Part 1, pattern 30, page 1598)

Find Eventual Safe States (Part 1, pattern 16, page 891)

Find All Possible Recipes from Given Supplies (Part 1, pattern 30, page 1613)

269. Alien Dictionary

Hard · Depth-First Search, Breadth-First Search, Graph, Topological Sort, Array, String, Directed Acyclic Graph

leetcode.com/problems/alien-dictionary

★ **TOP 7 PROBLEM** in Topological Sort: master this one first

PROBLEM

There is a new alien language that uses the English alphabet. However, the order of the letters is unknown to you.

You are given a list of strings `words` from the alien language's dictionary. Now it is claimed that the strings in `words` are **sorted lexicographically** by the rules of this new language.

If this claim is incorrect, and the given arrangement of string in `words` cannot correspond to any order of letters, return `""`.

Otherwise, return *a string of the unique letters in the new alien language sorted in **lexicographically increasing order** by the new language's rules.* If there are multiple solutions, return ***any of them***.

Example 1:

```
Input: words = ["wrt","wrf","er","ett","rftt"]
Output: "wertf"
```

Example 2:

```
Input: words = ["z","x"]
Output: "zx"
```

Example 3:

```
Input: words = ["z","x","z"]
Output: ""
Explanation: The order is invalid, so return "".
```

Constraints:

- `1 <= words.length <= 100`
- `1 <= words[i].length <= 100`
- `words[i]` consists of only lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

The alien order is a DAG whose edges come from the first differing letters of adjacent words. A later word that is a prefix of an earlier one, or a reverse edge, makes the order impossible. Topologically sort the letters that appear: enqueue indegree 0, and the dequeue order is a valid alphabet. Too few letters means a cycle.

Python 3

```
class Solution:
    def alienOrder(self, words: List[str]) -> str:
        g = [[False] * 26 for _ in range(26)]
        s = [False] * 26
        cnt = 0
        n = len(words)
        for i in range(n - 1):
            for c in words[i]:
                if cnt == 26:
                    break
                o = ord(c) - ord('a')
                if not s[o]:
                    cnt += 1
                    s[o] = True
            m = len(words[i])
            for j in range(m):
                if j >= len(words[i + 1]):
                    return ''
                c1, c2 = words[i][j], words[i + 1][j]
                if c1 == c2:
                    continue
                o1, o2 = ord(c1) - ord('a'), ord(c2) - ord('a')
                if g[o2][o1]:
                    return ''
                g[o1][o2] = True
                break
        for c in words[n - 1]:
            if cnt == 26:
                break
            o = ord(c) - ord('a')
            if not s[o]:
                cnt += 1
                s[o] = True

        indegree = [0] * 26
        for i in range(26):
            for j in range(26):
                if i != j and s[i] and s[j] and g[i][j]:
                    indegree[j] += 1
        q = deque()
        ans = []
        for i in range(26):
            if s[i] and indegree[i] == 0:
                q.append(i)
        while q:
            t = q.popleft()
            ans.append(chr(t + ord('a')))
            for i in range(26):
                if s[i] and i != t and g[t][i]:
                    indegree[i] -= 1
                    if indegree[i] == 0:
                        q.append(i)
        return '' if len(ans) < cnt else ''.join(ans)
```

Complexity. Time: O(L + 26^2), L = total number of letters. Space: O(26^2).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Check whether a letter order is consistent with the
# sorted words
# 2. How many different letter orders are possible


# ============================================================
# FOLLOW-UP 1: Check whether a letter order is consistent
# with the sorted words
# ============================================================

# ---- follow-up 1: function ----
# Answer: Compare every pair of neighbouring words at
# their first different letter using the order's ranks.
# Time: O(total letters).  Space: O(1) alphabet.
def is_sorted_in_order(words, order):
    rank = {ch: i for i, ch in enumerate(order)}
    for a, b in zip(words, words[1:]):
        for x, y in zip(a, b):
            if x != y:
                if rank[x] > rank[y]:
                    return False
                break
        else:
            if len(a) > len(b):
                return False
    return True


# ============================================================
# FOLLOW-UP 2: How many different letter orders are
# possible
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the topological orders with a bitmask DP
# over the letters that appear.
# Time: O(2^L * L), L = distinct letters.  Space: O(2^L).
def count_alien_orders(words):
    letters = sorted(set("".join(words)))
    index = {ch: i for i, ch in enumerate(letters)}
    before = [0] * len(letters)                      # bitmask of the letters that must come earlier
    for a, b in zip(words, words[1:]):
        for x, y in zip(a, b):
            if x != y:
                before[index[y]] |= 1 << index[x]
                break
        else:
            if len(a) > len(b):
                return 0
    ways = {0: 1}
    for mask in range(1 << len(letters)):
        if mask not in ways:
            continue
        for i in range(len(letters)):
            if not (mask >> i) & 1 and before[i] & mask == before[i]:
                ways[mask | (1 << i)] = ways.get(mask | (1 << i), 0) + ways[mask]
    return ways.get((1 << len(letters)) - 1, 0)
```

PART 2 · TOPIC 34 OF 50

Union Find

4 problems · 0 Easy · 3 Medium · 1 Hard

THE IDEA

Merge groups and ask whether two items are in the same group in almost O(1), using path compression and union by size.

THE 4 PROBLEMS, EASY TO HARD | ★ = TOP 4: DO THESE FIRST

1	★ **Graph Valid Tree**	Medium	LC 261	2516
2	★ **Number of Connected Components in an Undirected Graph**	Medium	LC 323	2520
3	★ **Evaluate Division**	Medium	LC 399	2525
4	★ **Number of Islands II**	Hard	LC 305	2528

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Number of Provinces (Part 1, pattern 16, page 869)

Accounts Merge (Part 1, pattern 31, page 1671)

Minimize Malware Spread (Part 1, pattern 31, page 1690)

Redundant Connection (Part 1, pattern 31, page 1667)

Most Stones Removed with Same Row or Column (Part 1, pattern 16, page 903)

PART 2 · UNION FIND · PROBLEM 1 OF 4

261. Graph Valid Tree

Medium · Depth-First Search, Breadth-First Search, Union Find, Graph

leetcode.com/problems/graph-valid-tree

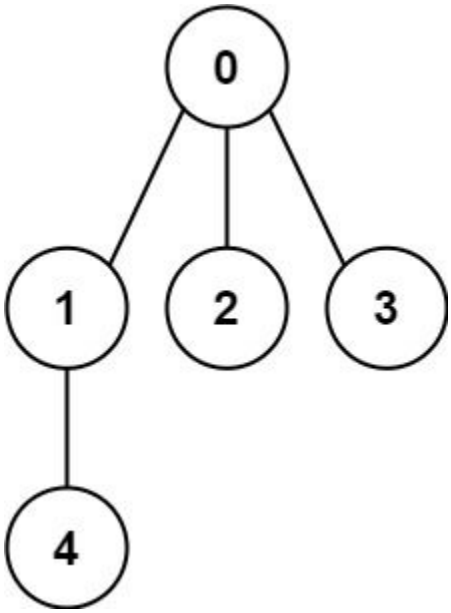
★ **TOP 7 PROBLEM** in Union Find: master this one first

PROBLEM

You have a graph of `n` nodes labeled from `0` to `n - 1`. You are given an integer n and a list of `edges` where `edges[i] = [a_i, b_i]` indicates that there is an undirected edge between nodes `a_i` and `b_i` in the graph.

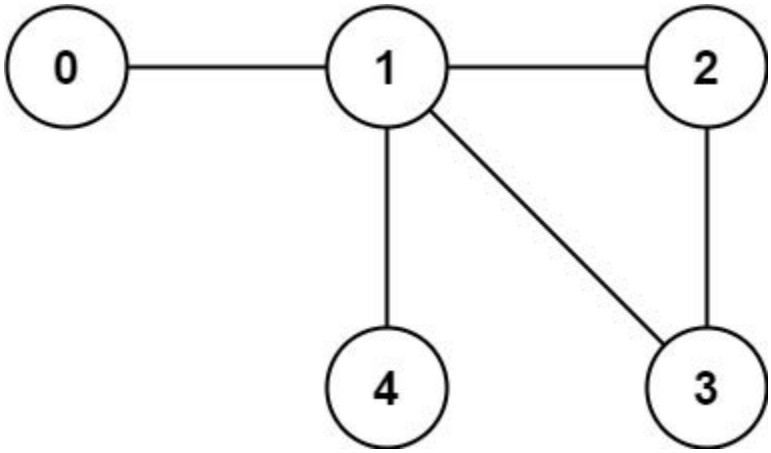
Return `true` *if the edges of the given graph make up a valid tree, and* `false` *otherwise*.

Example 1:




```
Input: n = 5, edges = [[0,1],[0,2],[0,3],[1,4]]
Output: true
```

Example 2:




```
Input: n = 5, edges = [[0,1],[1,2],[2,3],[1,3],[1,4]]
Output: false
```

Constraints:

- `1 <= n <= 2000`
- `0 <= edges.length <= 5000`
- `edges[i].length == 2`
- $0 \le a_i, b_i < n$
- $a_i \ne b_i$
- There are no self-loops or repeated edges.

SOLUTIONS (PYTHON 3)

Solution 1: Union-Find

> **How to think about it**
>
> An undirected graph is a tree iff it is connected and acyclic, so it has exactly n-1 edges. Union-find reports a cycle when an edge joins two nodes already in the same set. Each successful union decreases the component count; it must be 1 at the end.

To determine whether it is a tree, the following two conditions must be met:

1. The number of edges is equal to the number of nodes minus one;
2. There is no cycle.

We can use a union-find set to determine whether there is a cycle. We traverse the edges, if two nodes are already in the same set, it means there is a cycle. Otherwise, we merge the two nodes into the same set. Then we decrease the number of connected components by one, and finally check whether the number of connected components is 1.

The time complexity is $O(n \times \log n)$, and the space complexity is $O(n)$, where n is the number of nodes.

Python 3

```python
class Solution:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(n))
        for a, b in edges:
            pa, pb = find(a), find(b)
            if pa == pb:
                return False
            p[pa] = pb
            n -= 1
        return n == 1
```

Solution 2: DFS

> **How to think about it**
>
> Union-find checks cycles on edges. We can instead require |E|=n-1 and DFS from 0: visiting n nodes means the graph is connected and has no spare edge.

We can also use depth-first search to determine whether there is a cycle. We can use an array vis to record the visited nodes. During the search, we first mark the node as visited, then traverse the nodes adjacent to this node. If the adjacent node has been visited, we skip it, otherwise we recursively visit the adjacent node. Finally, we check whether all nodes have been visited. If there are nodes that have not been visited, it means that it cannot form a tree, so we return `false`.

The time complexity is $O(n)$, and the space complexity is $O(n)$, where n is the number of nodes.

Python 3

```python
class Solution:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        def dfs(i: int):
            vis.add(i)
            for j in g[i]:
                if j not in vis:
                    dfs(j)

        if len(edges) != n - 1:
            return False
        g = [[] for _ in range(n)]
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)
        vis = set()
        dfs(0)
        return len(vis) == n
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the edge that breaks the tree (the one that
# closes a cycle)
# 2. How many edges must be added to turn the graph into a
# connected graph


# ============================================================
# FOLLOW-UP 1: Return the edge that breaks the tree (the
# one that closes a cycle)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Union-find; the first edge whose ends are
# already joined.
# Time: O(E * alpha(n)).  Space: O(n).
def cycle_edge(n, edges):
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for a, b in edges:
        ra, rb = find(a), find(b)
        if ra == rb:
            return [a, b]
        parent[ra] = rb
    return None


# ============================================================
# FOLLOW-UP 2: How many edges must be added to turn the
# graph into a connected graph
# ============================================================

# ---- follow-up 2: function ----
# Answer: The number of components minus one.
# Time: O((n + E) * alpha(n)).  Space: O(n).
def edges_to_connect(n, edges):
    parent = list(range(n))

    def find(x):
```

```
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    groups = n
    for a, b in edges:
        ra, rb = find(a), find(b)
        if ra != rb:
            parent[ra] = rb
            groups -= 1
    return groups - 1
```

323. Number of Connected Components in an Undirected Graph

Medium · Depth-First Search, Breadth-First Search, Union Find, Graph

leetcode.com/problems/number-of-connected-components-in-an-undirected-graph

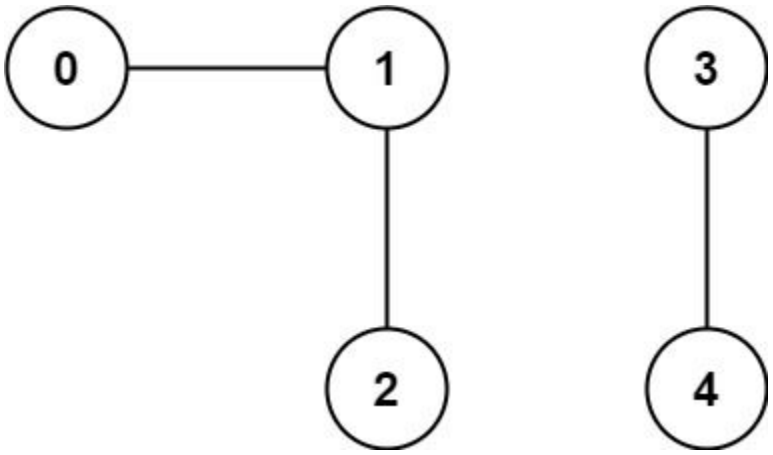
★ **TOP 7 PROBLEM** in Union Find: master this one first

PROBLEM

You have a graph of `n` nodes labeled from 0 to `n - 1`. You are given an integer `n` and an array `edges` where `edges[i] = [a_i, b_i]` indicates that there is an edge between `a_i` and `b_i` in the graph.

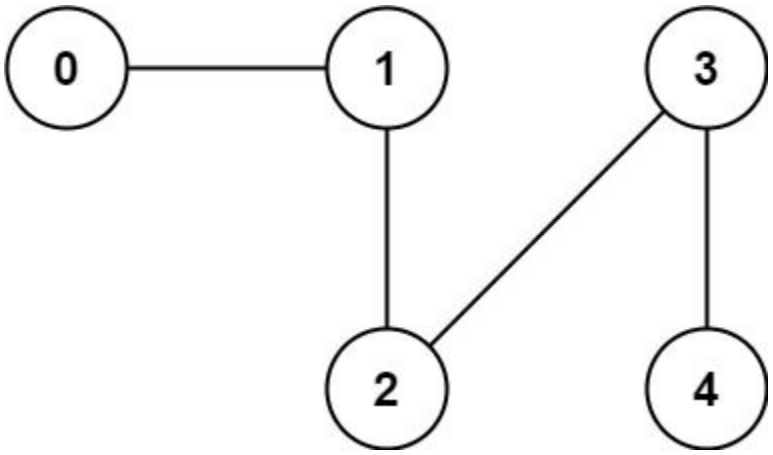
Return *the number of connected components in the graph*.

Example 1:




```
Input: n = 5, edges = [[0,1],[1,2],[3,4]]
Output: 2
```

Example 2:




```
Input: n = 5, edges = [[0,1],[1,2],[2,3],[3,4]]
Output: 1
```

Constraints:

- `1 <= n <= 2000`
- `1 <= edges.length <= 5000`

- `edges[i] = [a_i, b_i]`
- `a_i != b_i`
- There are no repeated edges.

SOLUTIONS (PYTHON 3)

Solution 1: DFS

> **How to think about it**
>
> Count connected components in an undirected graph. Start a search at each unseen node and increment once per new component. O(n+m) fits the limits. Build the adjacency list and DFS: return 0 if seen, otherwise mark, recurse on neighbors, and return 1. Summing over all nodes is the count.

First, we construct an adjacency list g based on the given edges, where g[i] represents all neighbor nodes of node i.

Then we traverse all nodes. For each node, we use DFS to traverse all its adjacent nodes and mark them as visited until all its adjacent nodes have been visited. In this way, we have found a connected component, and the answer is incremented by one. Then we continue to traverse the next unvisited node until all nodes have been visited.

The time complexity is O(n + m), and the space complexity is O(n + m). Where n and m are the number of nodes and edges, respectively.

Python 3

```python
class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        def dfs(i: int) -> int:
            if i in vis:
                return 0
            vis.add(i)
            for j in g[i]:
                dfs(j)
            return 1

        g = [[] for _ in range(n)]
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)
        vis = set()
        return sum(dfs(i) for i in range(n))
```

Solution 2: Union-Find

> **How to think about it**
>
> DFS needs the graph and a call stack. Union-find merges endpoints; a successful union decrements the component count from n. No explicit adjacency list is required.

We can use a union-find set to maintain the connected components in the graph.

First, we initialize a union-find set, then traverse all the edges. For each edge (a, b), we merge nodes a and b into the same connected component. If the merge is successful, it means that nodes a and b were not in the same connected component before, and the number of connected components decreases by one.

Finally, we return the number of connected components.

The time complexity is O(n + m × α(n)), and the space complexity is O(n). Where n and m are the number of nodes and edges, respectively, and α(n) is the inverse of the Ackermann function, which can be regarded as a very small constant.

Python 3

```python
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True


class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        uf = UnionFind(n)
        for a, b in edges:
            n -= uf.union(a, b)
        return n
```

Solution 3: BFS

> **How to think about it**
>
> Solution 1 expands each component with an explicit stack. A queue expands the same component in breadth-first order, and each new search still increments the count. Time stays O(n + m).

We can also use BFS (Breadth-First Search) to count the number of connected components in the graph.

Similar to Solution 1, we first construct an adjacency list g based on the given edges. Then we traverse all nodes. For each node, if it has not been visited, we start BFS traversal from this node, marking all its adjacent nodes as visited, until all its adjacent nodes have been visited. In this way, we have found a connected component, and the answer is incremented by one.

After traversing all nodes, we get the number of connected components in the graph.

The time complexity is O(n + m), and the space complexity is O(n + m). Where n and m are the number of nodes and edges, respectively.

Python 3

```python
class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        g = [[] for _ in range(n)]
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)
        vis = set()
        ans = 0
        for i in range(n):
            if i in vis:
                continue
            vis.add(i)
            q = deque([i])
            while q:
                a = q.popleft()
                for b in g[a]:
                    if b not in vis:
                        vis.add(b)
                        q.append(b)
            ans += 1
        return ans
```

Solution 4: Explicit Stack

> **How to think about it**
>
> The number of connected components is the number of searches that start at an unvisited node and cover its component. With n ≤ 2000, one O(n + m) traversal is enough. Recursing along a chain uses a call depth equal to the node count and overflows Python once the chain reaches length 1000. Each new component is therefore expanded on a stack: a node is marked when it is pushed, and unvisited neighbors are pushed after it is popped. Marking on push keeps every node out of the stack more than once. Each new start increments the answer.

Build an adjacency list g from the given edges. g[i] holds every neighbor of node i.

Scan the nodes. When node i is still unvisited, increment the answer, push i, and mark it visited. While the stack is not empty, pop node u and push every neighbor that has not been visited, marking it at the same time. That search covers the component of i. Continue with the next unvisited node until every node has been seen. The order of the neighbors does not change the number of components.

The time complexity is O(n + m), and the space complexity is O(n + m). Where n and m are the number of nodes and edges, respectively.

Python 3

```python
class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        g = [[] for _ in range(n)]
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)
        vis = [False] * n
        ans = 0
        for i in range(n):
            if vis[i]:
                continue
            ans += 1
            stk = [i]
            vis[i] = True
            while stk:
                u = stk.pop()
                for v in g[u]:
                    if not vis[v]:
                        vis[v] = True
                        stk.append(v)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The size of every component
# 2. Edges arrive one at a time: the number of components
# after each


# ============================================================
# FOLLOW-UP 1: The size of every component
# ============================================================

# ---- follow-up 1: function ----
# Answer: Union-find with sizes; read the sizes at the
# end.
# Time: O(n log n + E * alpha(n)).  Space: O(n).
def component_sizes_uf(n, edges):
    parent = list(range(n))
    size = [1] * n

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for a, b in edges:
        ra, rb = find(a), find(b)
        if ra != rb:
            if size[ra] < size[rb]:
                ra, rb = rb, ra
            parent[rb] = ra
            size[ra] += size[rb]
    return sorted((size[x] for x in range(n) if find(x) == x), reverse=True)


# ============================================================
# FOLLOW-UP 2: Edges arrive one at a time: the number of
# components after each
# ============================================================

# ---- follow-up 2: function ----
# Answer: Start with n and subtract one for every union
# that really merges two groups.
# Time: O(n + E * alpha(n)).  Space: O(n + E).
def components_over_time(n, edges):
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    groups = n
    history = []
    for a, b in edges:
        ra, rb = find(a), find(b)
        if ra != rb:
            parent[ra] = rb
            groups -= 1
        history.append(groups)
```

```
    return history
```

399. Evaluate Division

Medium · Depth-First Search, Breadth-First Search, Union Find, Graph, Array, String, Shortest Path, Floyd–Warshall, Bellman–Ford

leetcode.com/problems/evaluate-division

★ **TOP 7 PROBLEM** in Union Find: master this one first

PROBLEM

You are given an array of variable pairs `equations` and an array of real numbers `values`, where `equations[i] = [A_i, B_i]` and `values[i]` represent the equation $A_i / B_i = values[i]$. Each A_i or B_i is a string that represents a single variable.

You are also given some `queries`, where `queries[j] = [C_j, D_j]` represents the j^{th} query where you must find the answer for $C_j / D_j = ?$.

Return *the answers to all queries*. If a single answer cannot be determined, return `-1.0`.

Note: The input is always valid. You may assume that evaluating the queries will not result in division by zero and that there is no contradiction.

Note: The variables that do not occur in the list of equations are undefined, so the answer cannot be determined for them.

Example 1:

```
Input: equations = [["a","b"],["b","c"]], values = [2.0,3.0], queries =
[["a","c"],["b","a"],["a","e"],["a","a"],["x","x"]]
Output: [6.00000,0.50000,-1.00000,1.00000,-1.00000]
Explanation:
Given: a / b = 2.0, b / c = 3.0
queries are: a / c = ?, b / a = ?, a / e = ?, a / a = ?, x / x = ?
return: [6.0, 0.5, -1.0, 1.0, -1.0 ]
note: x is undefined => -1.0
```

Example 2:

```
Input: equations = [["a","b"],["b","c"],["bc","cd"]], values = [1.5,2.5,5.0], queries =
[["a","c"],["c","b"],["bc","cd"],["cd","bc"]]
Output: [3.75000,0.40000,5.00000,0.20000]
```

Example 3:

```
Input: equations = [["a","b"]], values = [0.5], queries = [["a","b"],["b","a"],["a","c"],["x","y"]]
Output: [0.50000,2.00000,-1.00000,-1.00000]
```

Constraints:

- `1 <= equations.length <= 20`
- `equations[i].length == 2`
- `1 <= A_i.length, B_i.length <= 5`
- `values.length == equations.length`
- `0.0 < values[i] <= 20.0`
- `1 <= queries.length <= 20`
- `queries[i].length == 2`
- `1 <= C_j.length, D_j.length <= 5`
- A_i, B_i, C_j, D_j consist of lower case English letters and digits.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Given a/b values, answer other ratios. Graph search multiplies along a path; weighted union-find is lighter with many variables. w[x] is x relative to its root. `find` multiplies weights on compression. Union of a/b=v sets w[pa]=w[b]· v/w[a]. A query is -1 if the variables are missing or in different components, else w[c]/w[d].

Python 3

```
class Solution:
    def calcEquation(
        self, equations: List[List[str]], values: List[float], queries: List[List[str]]
    ) -> List[float]:
        def find(x):
            if p[x] != x:
                origin = p[x]
                p[x] = find(p[x])
                w[x] *= w[origin]
            return p[x]

        w = defaultdict(lambda: 1)
        p = defaultdict()
        for a, b in equations:
            p[a], p[b] = a, b
        for i, v in enumerate(values):
            a, b = equations[i]
            pa, pb = find(a), find(b)
            if pa == pb:
                continue
            p[pa] = pb
            w[pa] = w[b] * v / w[a]
        return [
            -1 if c not in p or d not in p or find(c) != find(d) else w[c] / w[d]
            for c, d in queries
        ]
```

Complexity. Time: O((e + q) * alpha), e = equations, q = queries. Space: O(e).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Weighted union-find version
# 2. Check whether the equations contradict each other


# ============================================================
# FOLLOW-UP 1: Weighted union-find version
# ============================================================

# ---- follow-up 1: function ----
# Answer: Every variable stores a ratio to its root; find
# compresses the path and multiplies the ratios.
# Time: O((E + Q) * alpha(V)).  Space: O(V).
def calc_with_union_find(equations, values, queries):
    parent = {}
    ratio = {}                                  # value of x divided by value of parent[x]

    def find(x):
        if parent[x] != x:
            root = find(parent[x])
            ratio[x] *= ratio[parent[x]]
            parent[x] = root
        return parent[x]
```

```
    for (a, b), v in zip(equations, values):
        for x in (a, b):
            if x not in parent:
                parent[x] = x
                ratio[x] = 1.0
        ra, rb = find(a), find(b)
        if ra != rb:
            parent[ra] = rb
            ratio[ra] = v * ratio[b] / ratio[a]
    answers = []
    for a, b in queries:
        if a not in parent or b not in parent or find(a) != find(b):
            answers.append(-1.0)
        else:
            answers.append(ratio[a] / ratio[b])
    return answers


# ============================================================
# FOLLOW-UP 2: Check whether the equations contradict each
# other
# ============================================================

# ---- follow-up 2: function ----
# Answer: Search with ratios; a second route to a variable
# with a different ratio is a contradiction.
# Time: O(V + E).  Space: O(V + E).
def equations_consistent(equations, values):
    graph = {}
    for (a, b), v in zip(equations, values):
        graph.setdefault(a, []).append((b, 1 / v))
        graph.setdefault(b, []).append((a, v))
    value = {}
    for start in graph:
        if start in value:
            continue
```

```
        value[start] = 1.0
        stack = [start]
        while stack:
            x = stack.pop()
            for y, factor in graph[x]:
                expected = value[x] * factor
                if y not in value:
                    value[y] = expected
                    stack.append(y)
                elif abs(value[y] - expected) > 1e-9 * max(1.0, abs(expected)):
                    return False
    return True
```

305. Number of Islands II

Hard · Union Find, Array, Hash Table

leetcode.com/problems/number-of-islands-ii

★ **TOP 7 PROBLEM** in Union Find: master this one first

PROBLEM

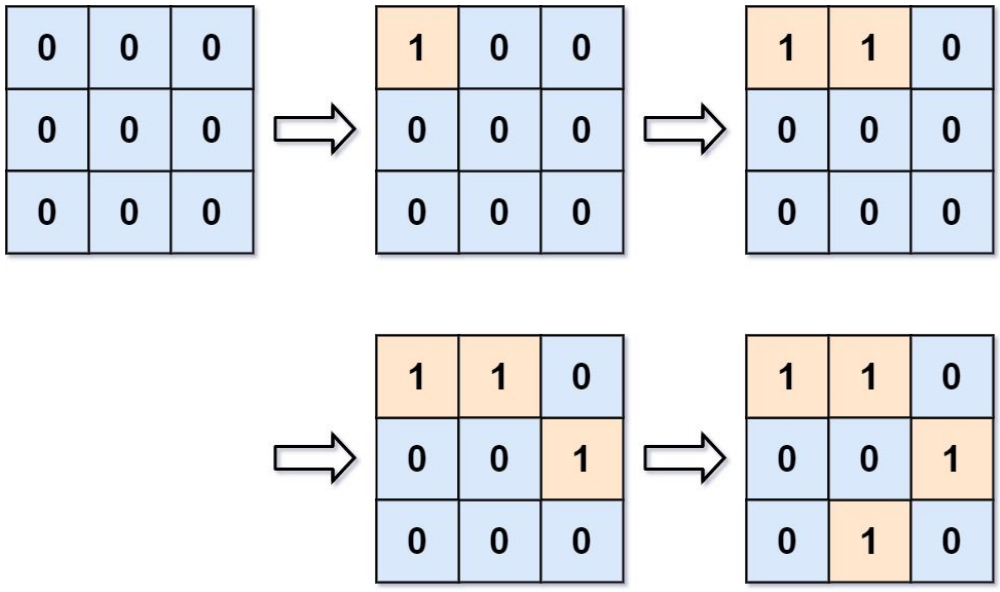
You are given an empty 2D binary grid `grid` of size `m x n`. The grid represents a map where `0`'s represent water and `1`'s represent land. Initially, all the cells of `grid` are water cells (i.e., all the cells are `0`'s).

We may perform an add land operation which turns the water at position into a land. You are given an array `positions` where `positions[i] = [r_i, c_i]` is the position `(r_i, c_i)` at which we should operate the `i^th` operation.

Return *an array of integers* `answer` *where* `answer[i]` *is the number of islands after turning the cell* `(r_i, c_i)` *into a land*.

An **island** is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:




```
Input: m = 3, n = 3, positions = [[0,0],[0,1],[1,2],[2,1]]
Output: [1,1,2,3]
Explanation:
Initially, the 2d grid is filled with water.
- Operation #1: addLand(0, 0) turns the water at grid[0][0] into a land. We have 1 island.
- Operation #2: addLand(0, 1) turns the water at grid[0][1] into a land. We still have 1 island.
- Operation #3: addLand(1, 2) turns the water at grid[1][2] into a land. We have 2 islands.
- Operation #4: addLand(2, 1) turns the water at grid[2][1] into a land. We have 3 islands.
```

Example 2:

```
Input: m = 1, n = 1, positions = [[0,0]]
Output: [1]
```

Constraints:

- `1 <= m, n, positions.length <= 10^4`
- `1 <= m * n <= 10^4`
- `positions[i].length == 2`

- $0 <= r_i < m$
- $0 <= c_i < n$

Follow up: Could you solve it in time complexity $O(k \log(mn))$, where $k == positions.length$?

SOLUTION (PYTHON 3)

Solution 1: Union-Find

How to think about it

Land cells arrive over time and we must report the island count after each one. Restarting DFS/BFS on the whole grid per operation multiplies grid size by the number of updates. A new land cell starts as its own island, then merges with land in the four neighbors. Union-find tracks components: merge only when a neighbor is already land and in a different set, and decrement the count. Re-adding the same cell is a no-op. Each operation is nearly constant, so the total grows linearly with k.

We use a two-dimensional array grid to represent a map, where 0 and 1 represent water and land respectively. Initially, all cells in grid are water cells (i.e., all cells are 0), and we use a variable cnt to record the number of islands. The connectivity between islands can be maintained by a union-find set uf.

Next, we traverse each position (i, j) in the array positions. If grid[i][j] is 1, it means that this position is already land, and we directly add cnt to the answer; otherwise, we change the value of grid[i][j] to 1, and increase the value of cnt by 1. Then, we traverse the four directions of up, down, left, and right of this position. If a certain direction is 1, and this position does not belong to the same connected component as (i, j), then we merge this position with (i, j), and decrease the value of cnt by 1. After traversing the four directions of up, down, left, and right of this position, we add cnt to the answer.

The time complexity is O(k × α(m × n)) or O(k × log(m × n)), where k is the length of positions, and α is the inverse function of the Ackermann function. In this problem, α(m × n) can be considered as a very small constant.

Python 3

```python
class UnionFind:
    def __init__(self, n: int):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x: int):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a - 1), self.find(b - 1)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True


class Solution:
    def numIslands2(self, m: int, n: int, positions: List[List[int]]) -> List[int]:
        uf = UnionFind(m * n)
        grid = [[0] * n for _ in range(m)]
        ans = []
        dirs = (-1, 0, 1, 0, -1)
        cnt = 0
        for i, j in positions:
            if grid[i][j]:
                ans.append(cnt)
                continue
            grid[i][j] = 1
            cnt += 1
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if (
                    0 <= x < m
                    and 0 <= y < n
                    and grid[x][y]
                    and uf.union(i * n + j, x * n + y)
                ):
                    cnt -= 1
            ans.append(cnt)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The size of the largest island after every operation
# 2. Land touches in eight directions (diagonals count)


# ============================================================
# FOLLOW-UP 1: The size of the largest island after every
# operation
# ============================================================

# ---- follow-up 1: function ----
# Answer: Union-find with sizes; track the biggest size.
# Time: O(k * alpha(k)), k = positions.  Space: O(k).
def largest_island_over_time(m, n, positions):
    parent = {}
```

```
    size = {}

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    biggest = 0
    result = []
    for r, c in positions:
        cell = (r, c)
        if cell not in parent:
            parent[cell] = cell
            size[cell] = 1
            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                nxt = (r + dr, c + dc)
                if nxt in parent:
                    ra, rb = find(cell), find(nxt)
                    if ra != rb:
                        parent[ra] = rb
                        size[rb] += size[ra]
            biggest = max(biggest, size[find(cell)])
        result.append(biggest)
    return result


# ============================================================
# FOLLOW-UP 2: Land touches in eight directions (diagonals
# count)
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same union-find with eight neighbours.
# Time: O(k * alpha(k)).  Space: O(k).
def islands_over_time_eight(m, n, positions):
    parent = {}

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    count = 0
```

```
    result = []
    for r, c in positions:
        cell = (r, c)
        if cell not in parent:
            parent[cell] = cell
            count += 1
            for dr in (-1, 0, 1):
                for dc in (-1, 0, 1):
                    nxt = (r + dr, c + dc)
                    if (dr or dc) and nxt in parent:
                        ra, rb = find(cell), find(nxt)
                        if ra != rb:
                            parent[ra] = rb
                            count -= 1
        result.append(count)
    return result
```

PART 2 · TOPIC 35 OF 50

Minimum Spanning Tree

5 problems · 0 Easy · 2 Medium · 3 Hard

THE IDEA

Connect all nodes with the least total cost: Kruskal (sort edges, add if they join two groups) or Prim (grow with a heap).

THE 5 PROBLEMS, EASY TO HARD | ★ = TOP 5: DO THESE FIRST				
1	★ **Connecting Cities With Minimum Cost**	Medium	LC 1135	2533
2	★ **Min Cost to Connect All Points**	Medium	LC 1584	2536
3	★ **Optimize Water Distribution in a Village**	Hard	LC 1168	2540
4	★ **Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree**	Hard	LC 1489	2544
5	★ **Maximize Spanning Tree Stability with Upgrades**	Hard	LC 3600	2548

1135. Connecting Cities With Minimum Cost

Medium · Union Find, Graph, Minimum Spanning Tree, Heap (Priority Queue)

leetcode.com/problems/connecting-cities-with-minimum-cost

★ **TOP 7 PROBLEM** in Minimum Spanning Tree: master this one first

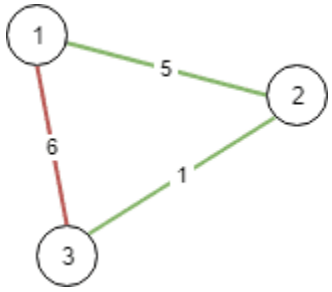
PROBLEM

There are `n` cities labeled from `1` to `n`. You are given the integer `n` and an array `connections` where `connections[i] = [x_i, y_i, cost_i]` indicates that the cost of connecting city `x_i` and city `y_i` (bidirectional connection) is `cost_i`.

Return *the minimum **cost** to connect all the `n` cities such that there is at least one path between each pair of cities*. If it is impossible to connect all the `n` cities, return `-1`,

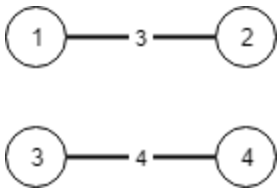
The **cost** is the sum of the connections' costs used.

Example 1:




```
Input: n = 3, connections = [[1,2,5],[1,3,6],[2,3,1]]
Output: 6
Explanation: Choosing any 2 edges will connect all cities so we choose the minimum 2.
```

Example 2:




```
Input: n = 4, connections = [[1,2,3],[3,4,4]]
Output: -1
Explanation: There is no way to connect all cities even if all edges are used.
```

Constraints:

- `1 <= n <= 10^4`
- `1 <= connections.length <= 10^4`
- `connections[i].length == 3`
- `1 <= x_i, y_i <= n`
- `x_i != y_i`
- `0 <= cost_i <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: Kruskal's Algorithm

> **How to think about it**
>
> The cheapest way to connect n cities is a minimum spanning tree. Sort edges by cost and union endpoints that are still apart, adding the cost; when one component remains we have a tree. If edges run out first, the graph is disconnected and the answer is -1.

Kruskal's algorithm is a greedy algorithm used to compute the minimum spanning tree.

The basic idea of Kruskal's algorithm is to select the smallest edge from the edge set each time. If the two vertices connected by this edge are not in the same connected component, then add this edge to the minimum spanning tree, otherwise discard this edge.

For this problem, we can sort the edges in ascending order of connection cost, use a union-find set to maintain connected components, select the smallest edge each time, and if the two vertices connected by this edge are not in the same connected component, then merge these two vertices and accumulate the connection cost. If a situation occurs where the number of connected components is 1, it means that all vertices are connected, and we return the accumulated connection cost, otherwise, we return -1.

The time complexity is $O(m \times \log m)$, and the space complexity is $O(n)$. Here, m and n are the number of edges and vertices, respectively.

Python 3

```python
class Solution:
    def minimumCost(self, n: int, connections: List[List[int]]) -> int:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        connections.sort(key=lambda x: x[2])
        p = list(range(n))
        ans = 0
        for x, y, cost in connections:
            x, y = x - 1, y - 1
            if find(x) == find(y):
                continue
            p[find(x)] = find(y)
            ans += cost
            n -= 1
            if n == 1:
                return ans
        return -1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the connections that are used
# 2. The same answer with Prim's algorithm (grow from city
# 1 with a heap)


# ============================================================
# FOLLOW-UP 1: Return the connections that are used
# ============================================================

# ---- follow-up 1: function ----
# Answer: Kruskal: keep the cheap edges that join two
# groups.
# Time: O(E log E).  Space: O(n).
```

```
def chosen_connections(n, connections):
    parent = list(range(n + 1))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    chosen = []
    for a, b, cost in sorted(connections, key=lambda e: e[2]):
        ra, rb = find(a), find(b)
        if ra != rb:
            parent[ra] = rb
            chosen.append([a, b, cost])
    if len(chosen) != n - 1:
        return None
    return chosen


# ============================================================
# FOLLOW-UP 2: The same answer with Prim's algorithm (grow
# from city 1 with a heap)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Always take the cheapest edge that leaves the
# connected part.
# Time: O(E log E).  Space: O(n + E).
import heapq


def prim_total(n, connections):
    graph = [[] for _ in range(n + 1)]
    for a, b, cost in connections:
        graph[a].append((cost, b))
        graph[b].append((cost, a))
    seen = set()
    heap = [(0, 1)]
    total = 0
    while heap and len(seen) < n:
        cost, node = heapq.heappop(heap)
        if node in seen:
            continue
        seen.add(node)
        total += cost
        for edge in graph[node]:
            if edge[1] not in seen:
                heapq.heappush(heap, edge)
    return total if len(seen) == n else -1
```

PART 2 · MINIMUM SPANNING TREE · PROBLEM 2 OF 5

1584. Min Cost to Connect All Points

Medium · Union Find, Graph, Array, Kruskal, Minimum Spanning Tree, Prim, Borůvka

leetcode.com/problems/min-cost-to-connect-all-points

★ **TOP 7 PROBLEM** in Minimum Spanning Tree: master this one first

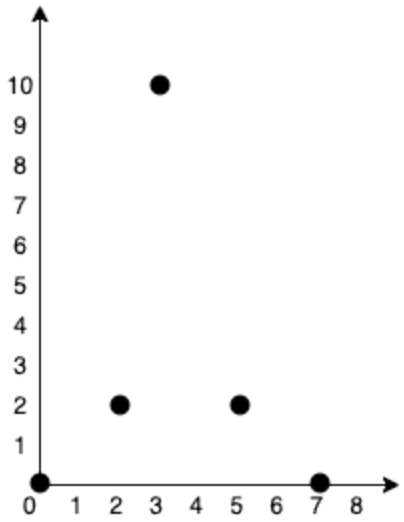
PROBLEM

You are given an array `points` representing integer coordinates of some points on a 2D-plane, where `points[i] = [x_i, y_i]`.

The cost of connecting two points `[x_i, y_i]` and `[x_j, y_j]` is the **manhattan distance** between them: `|x_i - x_j| + |y_i - y_j|`, where `|val|` denotes the absolute value of `val`.

Return *the minimum cost to make all points connected.* All points are connected if there is **exactly one** simple path between any two points.

Example 1:




```
Input: points = [[0,0],[2,2],[3,10],[5,2],[7,0]]
Output: 20
Explanation:

We can connect the points as shown above to get the minimum cost of 20.
Notice that there is a unique path between every pair of points.
```

Example 2:

```
Input: points = [[3,12],[-2,5],[-4,1]]
Output: 18
```

Constraints:

- `1 <= points.length <= 1000`
- `-10^6 <= x_i, y_i <= 10^6`
- All pairs `(x_i, y_i)` are distinct.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Connect all points by Manhattan distance and take a minimum spanning tree. n≤ 1000 yields a dense complete graph, which suits a dense MST algorithm. Plain Prim keeps each vertex's distance to the current tree, repeatedly adds the closest unused vertex, and relaxes the others. n rounds of an n-scan cost $O(n^2)$.

Python 3

```python
class Solution:
    def minCostConnectPoints(self, points: List[List[int]]) -> int:
        n = len(points)
        g = [[0] * n for _ in range(n)]
        dist = [inf] * n
        vis = [False] * n
        for i, (x1, y1) in enumerate(points):
            for j in range(i + 1, n):
                x2, y2 = points[j]
                t = abs(x1 - x2) + abs(y1 - y2)
                g[i][j] = g[j][i] = t
        dist[0] = 0
        ans = 0
        for _ in range(n):
            i = -1
            for j in range(n):
                if not vis[j] and (i == -1 or dist[j] < dist[i]):
                    i = j
            vis[i] = True
            ans += dist[i]
            for j in range(n):
                if not vis[j]:
                    dist[j] = min(dist[j], g[i][j])
        return ans
```

Solution 2

How to think about it

Prim is organized around a cut and an adjacency matrix. Kruskal sorts every edge and unions endpoints until the graph is connected. With $O(n^2)$ edges the sort dominates; the style is convenient when we prefer to iterate edges.

Python 3

```python
class Solution:
    def minCostConnectPoints(self, points: List[List[int]]) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(points)
        g = []
        for i, (x1, y1) in enumerate(points):
            for j in range(i + 1, n):
                x2, y2 = points[j]
                t = abs(x1 - x2) + abs(y1 - y2)
                g.append((t, i, j))
        p = list(range(n))
        ans = 0
        for cost, i, j in sorted(g):
            pa, pb = find(i), find(j)
            if pa == pb:
                continue
            p[pa] = pb
            ans += cost
            n -= 1
            if n == 1:
                break
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Prim's algorithm in O(n squared) without a heap
# 2. Return the connections that are used


# ============================================================
# FOLLOW-UP 1: Prim's algorithm in O(n squared) without a
# heap
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the cheapest distance of every point to the
# connected part and take the smallest each time.
# Time: O(n^2).  Space: O(n).
def prim_points(points):
    n = len(points)
    best = [float("inf")] * n
    done = [False] * n
    best[0] = 0
    total = 0
    for _ in range(n):
        u = -1
        for i in range(n):
            if not done[i] and (u == -1 or best[i] < best[u]):
                u = i
        done[u] = True
        total += best[u]
        for v in range(n):
            if not done[v]:
                d = abs(points[u][0] - points[v][0]) + abs(points[u][1] - points[v][1])
                if d < best[v]:
                    best[v] = d
    return total


# ============================================================
```

```python
# FOLLOW-UP 2: Return the connections that are used
# ========================================================

# ---- follow-up 2: function ----
# Answer: Kruskal over all pairs, collecting the chosen
# edges.
# Time: O(n^2 log n).  Space: O(n^2).
def chosen_point_edges(points):
    n = len(points)
    edges = []
    for i in range(n):
        for j in range(i + 1, n):
            edges.append((abs(points[i][0] - points[j][0]) + abs(points[i][1] - points[j][1]), i, j))
    edges.sort()
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    chosen = []
    for d, i, j in edges:
```

```python
        ri, rj = find(i), find(j)
        if ri != rj:
            parent[ri] = rj
            chosen.append((i, j, d))
    return chosen
```

1168. Optimize Water Distribution in a Village

Hard · Union Find, Graph, Kruskal, Minimum Spanning Tree, Prim, Heap (Priority Queue), Borůvka

leetcode.com/problems/optimize-water-distribution-in-a-village

★ **TOP 7 PROBLEM** in Minimum Spanning Tree: master this one first

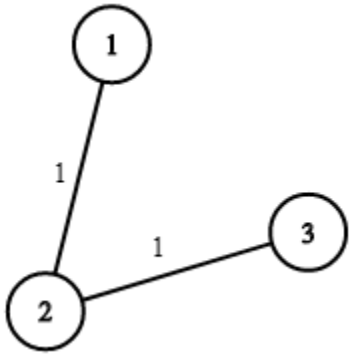
PROBLEM

There are `n` houses in a village. We want to supply water for all the houses by building wells and laying pipes.

For each house `i`, we can either build a well inside it directly with cost `wells[i - 1]` (note the `-1` due to **0-indexing**), or pipe in water from another well to it. The costs to lay pipes between houses are given by the array `pipes` where each `pipes[j] = [house1`$_j$`, house2`$_j$`, cost`$_j$`]` represents the cost to connect `house1`$_j$ and `house2`$_j$ together using a pipe. Connections are bidirectional, and there could be multiple valid connections between the same two houses with different costs.

Return *the minimum total cost to supply water to all houses*.

Example 1:




```
Input: n = 3, wells = [1,2,2], pipes = [[1,2,1],[2,3,1]]
Output: 3
Explanation: The image shows the costs of connecting houses using pipes.
The best strategy is to build a well in the first house with cost 1 and connect the other houses to it
with cost 2 so the total cost is 3.
```

Example 2:

```
Input: n = 2, wells = [1,1], pipes = [[1,2,1],[1,2,2]]
Output: 2
Explanation: We can supply water with cost two using one of the three options:
Option 1:
  - Build a well inside house 1 with cost 1.
  - Build a well inside house 2 with cost 1.
The total cost will be 2.
Option 2:
  - Build a well inside house 1 with cost 1.
  - Connect house 2 with house 1 with cost 1.
The total cost will be 2.
Option 3:
  - Build a well inside house 2 with cost 1.
  - Connect house 1 with house 2 with cost 1.
The total cost will be 2.
Note that we can connect houses 1 and 2 with cost 1 or with cost 2 but we will always choose the
cheapest option.
```

Constraints:

- $2 <= n <= 10^4$
- $wells.length == n$
- $0 <= wells[i] <= 10^5$
- $1 <= pipes.length <= 10^4$
- $pipes[j].length == 3$
- $1 <= house1_j, house2_j <= n$
- $0 <= cost_j <= 10^5$
- $house1_j != house2_j$

SOLUTIONS (PYTHON 3)

Solution 1: Kruskal's Algorithm (Minimum Spanning Tree)

How to think about it

A house may sink its own well or connect through pipes. Add a virtual node 0 whose edges to houses are well costs; the instance is an MST on n+1 vertices. Sort edges by cost and union them until n merges remain (all houses touch 0), which is the minimum cost.

We assume that there is a well with the number 0. Then we can consider the connectivity between each house and the well 0 as an edge, and the weight of each edge is the cost of building a well for that house. At the same time, we consider the connectivity between each house as an edge, and the weight of each edge is the cost of laying a pipe. In this way, we can transform this problem into finding the minimum spanning tree of an undirected graph.

We can use Kruskal's algorithm to find the minimum spanning tree of the undirected graph. First, we add an edge between the well 0 and the house to the pipes array, and then sort the pipes array in ascending order of edge weights. Then, we traverse each edge. If this edge connects different connected components, we choose this edge and merge the corresponding connected components. If the current connected component is exactly 1, then we have found the minimum spanning tree. The answer at this time is the current edge weight, and we return it.

The time complexity is O((m + n) × log (m + n)), and the space complexity is O(m + n). Here, m and n are the lengths of the pipes array and the wells array, respectively.

Python 3

```
class Solution:
    def minCostToSupplyWater(
        self, n: int, wells: List[int], pipes: List[List[int]]
    ) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        for i, w in enumerate(wells, 1):
            pipes.append([0, i, w])
        pipes.sort(key=lambda x: x[2])
        p = list(range(n + 1))
        ans = 0
        for a, b, c in pipes:
            pa, pb = find(a), find(b)
            if pa != pb:
                p[pa] = pb
                n -= 1
                ans += c
                if n == 0:
                    return ans
```

Solution 2

How to think about it

Method 1 inlines union-find. Method 2 wraps union-by-size; `union` reports a real merge, and the loop only adds cost and counts. The graph and Kruskal order stay the same.

Python 3

```python
class UnionFind:
    __slots__ = ("p", "size")

    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x: int) -> int:
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True


class Solution:
    def minCostToSupplyWater(
        self, n: int, wells: List[int], pipes: List[List[int]]
    ) -> int:
        for i, w in enumerate(wells, 1):
            pipes.append([0, i, w])
        pipes.sort(key=lambda x: x[2])
        uf = UnionFind(n + 1)
        ans = 0
        for a, b, c in pipes:
            if uf.union(a, b):
                ans += c
                n -= 1
                if n == 0:
                    return ans
```

Complexity. Time: O((n + m) log (n + m)), m = pipes. Space: O(n + m).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Which houses get a well
# 2. The pipes that are built (not the wells)


# ========================================================
# FOLLOW-UP 1: Which houses get a well
# ========================================================

# ---- follow-up 1: function ----
# Answer: Add a node 0 joined to every house with the well
# cost; houses joined to node 0 in the tree have a well.
# Time: O((n + p) log (n + p)).  Space: O(n + p).
```

```python
def houses_with_wells(n, wells, pipes):
    edges = [(wells[i], 0, i + 1) for i in range(n)] + [(c, a, b) for a, b, c in pipes]
    edges.sort()
    parent = list(range(n + 1))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    with_wells = []
    for cost, a, b in edges:
        ra, rb = find(a), find(b)
        if ra != rb:
            parent[ra] = rb
            if a == 0:
                with_wells.append(b)
    return sorted(with_wells)


# ============================================================
# FOLLOW-UP 2: The pipes that are built (not the wells)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same Kruskal, keeping the pipe edges.
# Time: O((n + p) log (n + p)).  Space: O(n + p).
def pipes_built(n, wells, pipes):
    edges = [(wells[i], 0, i + 1) for i in range(n)] + [(c, a, b) for a, b, c in pipes]
    edges.sort()
    parent = list(range(n + 1))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    built = []
    for cost, a, b in edges:
        ra, rb = find(a), find(b)
        if ra != rb:
            parent[ra] = rb
            if a != 0:
                built.append([a, b, cost])
    return built
```

1489. Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree

Hard · Union Find, Graph, Kruskal, Minimum Spanning Tree, Prim, Sorting, Strongly Connected Component, Borůvka

leetcode.com/problems/find-critical-and-pseudo-critical-edges-in-minimum-spanning-tree

★ **TOP 7 PROBLEM** in Minimum Spanning Tree: master this one first

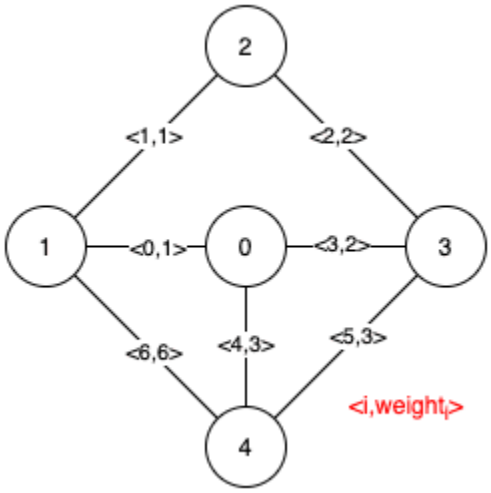
PROBLEM

Given a weighted undirected connected graph with `n` vertices numbered from `0` to `n - 1`, and an array `edges` where `edges[i] = [a_i, b_i, weight_i]` represents a bidirectional and weighted edge between nodes `a_i` and `b_i`. A minimum spanning tree (MST) is a subset of the graph's edges that connects all vertices without cycles and with the minimum possible total edge weight.

Find *all the critical and pseudo-critical edges in the given graph's minimum spanning tree (MST)*. An MST edge whose deletion from the graph would cause the MST weight to increase is called a *critical edge*. On the other hand, a pseudo-critical edge is that which can appear in some MSTs but not all.

Note that you can return the indices of the edges in any order.

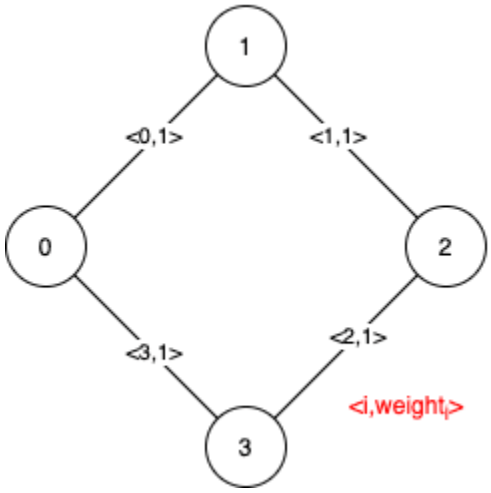
Example 1:




```
Input: n = 5, edges = [[0,1,1],[1,2,1],[2,3,2],[0,3,2],[0,4,3],[3,4,3],[1,4,6]]
Output: [[0,1],[2,3,4,5]]
Explanation: The figure above describes the graph.
The following figure shows all the possible MSTs:

Notice that the two edges 0 and 1 appear in all MSTs, therefore they are critical edges, so we return
them in the first list of the output.
The edges 2, 3, 4, and 5 are only part of some MSTs, therefore they are considered pseudo-critical
edges. We add them to the second list of the output.
```

Example 2:




```
Input: n = 4, edges = [[0,1,1],[1,2,1],[2,3,1],[0,3,1]]
Output: [[],[0,1,2,3]]
Explanation: We can observe that since all 4 edges have equal weight, choosing any 3 edges from the
given 4 will yield an MST. Therefore all 4 edges are pseudo-critical.
```

Constraints:

- `2 <= n <= 100`
- `1 <= edges.length <= min(200, n * (n - 1) / 2)`
- `edges[i].length == 3`
- `0 <= a_i < b_i < n`
- `1 <= weight_i <= 1000`
- All pairs `(a_i, b_i)` are **distinct**.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

n≤ 100 and at most 200 edges, so we can run MST many times. Kruskal first yields the MST weight v. An edge is critical iff deleting it disconnects the graph or raises the weight above v. Otherwise it is pseudo-critical iff forcing it in still achieves v. Test both for every edge.

Python 3

```
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.n = n

    def union(self, a, b):
        if self.find(a) == self.find(b):
            return False
        self.p[self.find(a)] = self.find(b)
        self.n -= 1
        return True

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]


class Solution:
    def findCriticalAndPseudoCriticalEdges(
        self, n: int, edges: List[List[int]]
    ) -> List[List[int]]:
        for i, e in enumerate(edges):
            e.append(i)
        edges.sort(key=lambda x: x[2])
        uf = UnionFind(n)
        v = sum(w for f, t, w, _ in edges if uf.union(f, t))
        ans = [[], []]
        for f, t, w, i in edges:
            uf = UnionFind(n)
            k = sum(z for x, y, z, j in edges if j != i and uf.union(x, y))
            if uf.n > 1 or (uf.n == 1 and k > v):
                ans[0].append(i)
                continue

            uf = UnionFind(n)
            uf.union(f, t)
            k = w + sum(z for x, y, z, j in edges if j != i and uf.union(x, y))
            if k == v:
                ans[1].append(i)
        return ans
```

Complexity. Time: O(E^2 * alpha), E = number of edges. Space: O(n + E).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The weight of a minimum spanning tree (the number
# every other test compares with)
# 2. Is a given edge in SOME minimum spanning tree


# ============================================================
# FOLLOW-UP 1: The weight of a minimum spanning tree (the
# number every other test compares with)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Kruskal's algorithm.
# Time: O(E log E).  Space: O(n).
def mst_weight(n, edges):
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
```

```
            x = parent[x]
        return x

    total = 0
    used = 0
    for a, b, w in sorted(edges, key=lambda e: e[2]):
        ra, rb = find(a), find(b)
        if ra != rb:
            parent[ra] = rb
            total += w
            used += 1
    return total if used == n - 1 else None


# ============================================================
# FOLLOW-UP 2: Is a given edge in SOME minimum spanning
# tree
# ============================================================

# ---- follow-up 2: function ----
# Answer: Force the edge in first, then finish with
# Kruskal; it is in some MST when the weight stays the
# same.
# Time: O(E log E).  Space: O(n).
def edge_in_some_mst(n, edges, index):
    a0, b0, w0 = edges[index]
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    parent[find(a0)] = find(b0)
    total = w0
    used = 1
    for a, b, w in sorted(edges, key=lambda e: e[2]):
        ra, rb = find(a), find(b)
        if ra != rb:
```

```
            parent[ra] = rb
            total += w
            used += 1
    return used == n - 1 and total == mst_weight(n, edges)
```

3600. Maximize Spanning Tree Stability with Upgrades

Hard · Greedy, Union Find, Graph, Binary Search, Minimum Spanning Tree

leetcode.com/problems/maximize-spanning-tree-stability-with-upgrades

★ **TOP 7 PROBLEM** in Minimum Spanning Tree: master this one first

PROBLEM

You are given an integer `n`, representing `n` nodes numbered from 0 to `n - 1` and a list of `edges`, where `edges[i] = [u_i, v_i, s_i, must_i]`:

- `u_i` and `v_i` indicates an undirected edge between nodes `u_i` and `v_i`.
- `s_i` is the strength of the edge.
- `must_i` is an integer (0 or 1). If `must_i == 1`, the edge **must** be included in the **spanning tree**. These edges **cannot** be **upgraded**.

You are also given an integer `k`, the **maximum** number of upgrades you can perform. Each upgrade **doubles** the strength of an edge, and each eligible edge (with `must_i == 0`) can be upgraded **at most** once.

The **stability** of a spanning tree is defined as the **minimum** strength score among all edges included in it.

Return the **maximum** possible stability of any valid spanning tree. If it is impossible to connect all nodes, return `-1`.

Note: A **spanning tree** of a graph with `n` nodes is a subset of the edges that connects all nodes together (i.e. the graph is **connected**) *without* forming any cycles, and uses **exactly** `n - 1` edges.

Example 1:

Input: n = 3, edges = [[0,1,2,1],[1,2,3,0]], k = 1

Output: 2

Explanation:

- Edge `[0,1]` with strength = 2 must be included in the spanning tree.
- Edge `[1,2]` is optional and can be upgraded from 3 to 6 using one upgrade.
- The resulting spanning tree includes these two edges with strengths 2 and 6.
- The minimum strength in the spanning tree is 2, which is the maximum possible stability.

Example 2:

Input: n = 3, edges = [[0,1,4,0],[1,2,3,0],[0,2,1,0]], k = 2

Output: 6

Explanation:

- Since all edges are optional and up to `k = 2` upgrades are allowed.
- Upgrade edges `[0,1]` from 4 to 8 and `[1,2]` from 3 to 6.
- The resulting spanning tree includes these two edges with strengths 8 and 6.
- The minimum strength in the tree is 6, which is the maximum possible stability.

Example 3:

Input: n = 3, edges = [[0,1,1,1],[1,2,1,1],[2,0,1,1]], k = 0

Output: -1

Explanation:

- All edges are mandatory and form a cycle, which violates the spanning tree property of acyclicity. Thus, the answer is -1.

Constraints:

- $2 <= n <= 10^5$

- `1 <= edges.length <= 10^5`
- `edges[i] = [u_i, v_i, s_i, must_i]`
- `0 <= u_i, v_i < n`
- `u_i != v_i`
- `1 <= s_i <= 10^5`
- `must_i` is either `0` or `1`.
- `0 <= k <= n`
- There are no duplicate edges.

SOLUTION (PYTHON 3)

Solution 1: Binary Search + Union-Find

How to think about it

Enumerating spanning trees and subsets of at most k upgrades is exponential and impossible for $n,m \le 10^5$. Stability is the tree's minimum edge strength, so feasibility of a threshold x is monotone and the maximum can be binary-searched. Must-edges cannot be upgraded, so their minimum strength mn is an upper bound. A cycle among must-edges, or a graph that stays disconnected after adding every edge, has no answer. For a candidate lim, union every edge with strength at least lim, then spend at most k upgrades on edges with $2s \ge$ lim. Union-find decides connectivity, so a logarithmic number of checks yields the maximum feasible stability.

According to the problem description, the stability of a spanning tree is determined by the minimum strength edge in it. If a stability x is feasible, then for any y < x, stability y is also feasible. Therefore, we can use binary search to find the maximum stability.

We first add all required edges into the Union-Find and record the minimum strength mn among them. If there is a cycle among the required edges, return -1 directly. Then we add all edges into the Union-Find; if the final number of connected components is greater than 1, it means not all nodes can be connected, and we return -1.

Next, we perform binary search in the range [1, mn]. We define a function check(lim) to check whether there exists a spanning tree with stability at least lim. In the check function, we first add all edges with strength no less than lim into the Union-Find. Then we try to use the upgrade count to connect the remaining edges, with the condition that the edge strength is at least lim/2 (since upgrading doubles the strength). If the final number of connected components in the Union-Find is 1, a spanning tree satisfying the condition exists.

The time complexity is $O((m \times \alpha(n) + n) \times \log M)$, and the space complexity is $O(n)$. Here, m is the number of edges, n is the number of nodes, and M is the maximum edge strength.

Python 3

```python
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n
        self.cnt = n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        self.cnt -= 1
        return True


class Solution:
    def maxStability(self, n: int, edges: List[List[int]], k: int) -> int:
        def check(lim: int) -> bool:
            uf = UnionFind(n)
            for u, v, s, _ in edges:
                if s >= lim:
                    uf.union(u, v)
            rem = k
            for u, v, s, _ in edges:
                if s * 2 >= lim and rem > 0:
                    if uf.union(u, v):
                        rem -= 1
            return uf.cnt == 1

        uf = UnionFind(n)
        mn = 10**6
        for u, v, s, must in edges:
            if must:
                mn = min(mn, s)
                if not uf.union(u, v):
                    return -1
        for u, v, _, _ in edges:
            uf.union(u, v)
        if uf.cnt > 1:
            return -1
        l, r = 1, mn
        while l < r:
            mid = (l + r + 1) >> 1
            if check(mid):
                l = mid
            else:
                r = mid - 1
        return l
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Do the must-use edges contain a cycle (then no
# spanning tree can use them all)
# 2. The best stability when no upgrades are allowed (k =
# 0)


# ============================================================
# FOLLOW-UP 1: Do the must-use edges contain a cycle (then
# no spanning tree can use them all)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Union-find over the must edges only.
# Time: O(E * alpha(n)).  Space: O(n).
def must_edges_make_a_forest(n, edges):
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for u, v, s, must in edges:
        if must:
            ru, rv = find(u), find(v)
            if ru == rv:
                return False
            parent[ru] = rv
    return True


# ============================================================
# FOLLOW-UP 2: The best stability when no upgrades are
# allowed (k = 0)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Take the must edges first, then the strongest
# edges that join two groups; the stability is the weakest
# edge used.
# Time: O(E log E).  Space: O(n).
def stability_without_upgrades(n, edges):
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    weakest = float("inf")
    used = 0
    for u, v, s, must in edges:
        if must:
            ru, rv = find(u), find(v)
            if ru == rv:
                return -1
            parent[ru] = rv
```

```
            weakest = min(weakest, s)
            used += 1
    for u, v, s, must in sorted(edges, key=lambda e: -e[2]):
        if not must:
```

```
            ru, rv = find(u), find(v)
            if ru != rv:
                parent[ru] = rv
                weakest = min(weakest, s)
                used += 1
    return weakest if used == n - 1 else -1
```

PART 2 · TOPIC 36 OF 50

Shortest Path

9 problems · 0 Easy · 6 Medium · 3 Hard

THE IDEA

Dijkstra with a heap for weighted graphs, 0-1 BFS for 0/1 costs, Bellman-Ford when edges are limited; state can include extra parameters (stops, effort).

THE 9 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	★ **The Maze II**	Medium	LC 505	2554
2	★ **Network Delay Time**	Medium	LC 743	2558
3	★ **Path With Minimum Effort**	Medium	LC 1631	2562
4	★ **Minimum Cost to Convert String I**	Medium	LC 2976	2569
5	Find Minimum Time to Reach Last Room I	Medium	LC 3341	2573
6	Minimum Cost Path with Edge Reversals	Medium	LC 3650	2576
7	★ **Swim in Rising Water**	Hard	LC 778	2580
8	★ **Minimum Cost to Make at Least One Valid Path in a Grid**	Hard	LC 1368	2584
9	★ **Minimum Weighted Subgraph With the Required Paths**	Hard	LC 2203	2588

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Cheapest Flights Within K Stops (Part 1, pattern 17, page 959)

Path with Maximum Probability (Part 1, pattern 16, page 916)

505. The Maze II

Medium · Depth-First Search, Breadth-First Search, Graph, Array, Matrix, Shortest Path, Dijkstra, Heap (Priority Queue), A* Search, Heuristic Search

leetcode.com/problems/the-maze-ii

★ **TOP 7 PROBLEM** in Shortest Path: master this one first

PROBLEM

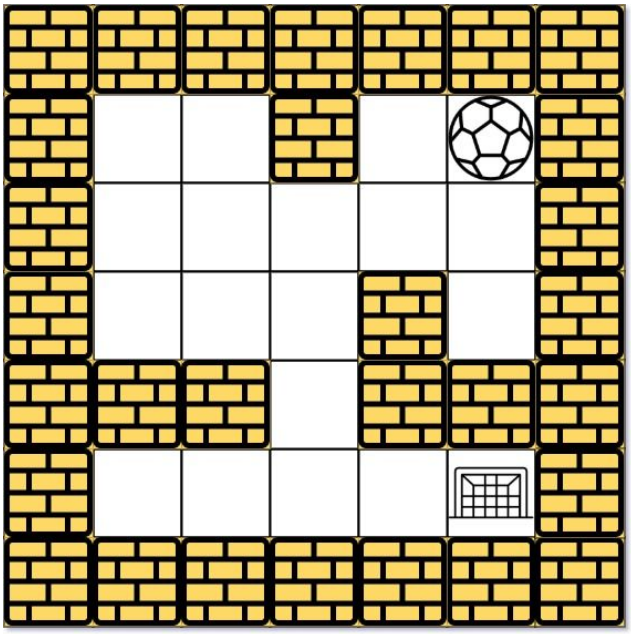
There is a ball in a `maze` with empty spaces (represented as `0`) and walls (represented as `1`). The ball can go through the empty spaces by rolling **up, down, left or right**, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction.

Given the `m x n maze`, the ball's `start` position and the `destination`, where `start = [start`$_{row}$`, start`$_{col}$`]` and `destination = [destination`$_{row}$`, destination`$_{col}$`]`, return *the shortest **distance** for the ball to stop at the destination*. If the ball cannot stop at `destination`, return `-1`.

The **distance** is the number of **empty spaces** traveled by the ball from the start position (excluded) to the destination (included).

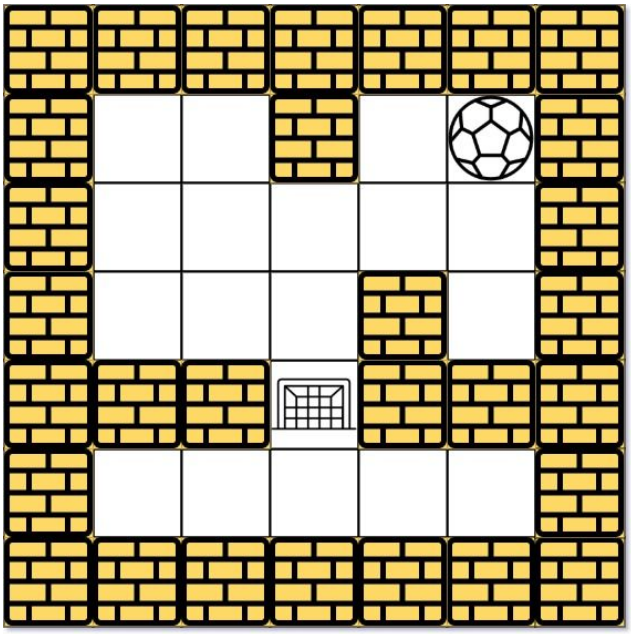
You may assume that **the borders of the maze are all walls** (see examples).

Example 1:



```
Input: maze = [[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]], start = [0,4],
destination = [4,4]
Output: 12
Explanation: One possible way is : left -> down -> left -> down -> right -> down -> right.
The length of the path is 1 + 1 + 3 + 1 + 2 + 2 + 2 = 12.
```

Example 2:



```
Input: maze = [[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]], start = [0,4],
destination = [3,2]
Output: -1
Explanation: There is no way for the ball to stop at the destination. Notice that you can pass through
the destination but you cannot stop there.
```

Example 3:

```
Input: maze = [[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]], start = [4,3],
destination = [0,1]
Output: -1
```

Constraints:

- `m == maze.length`
- `n == maze[i].length`
- `1 <= m, n <= 100`
- `maze[i][j]` is `0` or `1`.
- `start.length == 2`
- `destination.length == 2`
- $0 <= start_{row}, destination_{row} < m$
- $0 <= start_{col}, destination_{col} < n$
- Both the ball and the destination exist in an empty space, and they will not be in the same position initially.
- The maze contains **at least 2 empty spaces**.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

The ball rolls until it hits a wall, so an edge is a full roll rather than a single cell. Stepping cell by cell re-expands the same roll many times. States are stopping cells. From each stop, roll in four directions to the end and relax that cell if the path is shorter. Distances per roll vary, so this is shortest-path relaxation on stop points, not unweighted BFS. The recorded distance at the destination is the answer, or -1 if it stays infinite.

Python 3

```
class Solution:
    def shortestDistance(
        self, maze: List[List[int]], start: List[int], destination: List[int]
    ) -> int:
        m, n = len(maze), len(maze[0])
        dirs = (-1, 0, 1, 0, -1)
        si, sj = start
        di, dj = destination
        q = deque([(si, sj)])
        dist = [[inf] * n for _ in range(m)]
        dist[si][sj] = 0
        while q:
            i, j = q.popleft()
            for a, b in pairwise(dirs):
                x, y, k = i, j, dist[i][j]
                while 0 <= x + a < m and 0 <= y + b < n and maze[x + a][y + b] == 0:
                    x, y, k = x + a, y + b, k + 1
                if k < dist[x][y]:
                    dist[x][y] = k
                    q.append((x, y))
        return -1 if dist[di][dj] == inf else dist[di][dj]
```

Complexity. Time: O(m * n * (m + n)) in the worst case (a cell can be improved several times). Space: O(m * n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the places where the ball stops on the best
# route
# 2. How many different places can the ball stop at


# ============================================================
# FOLLOW-UP 1: Return the places where the ball stops on
# the best route
# ============================================================

# ---- follow-up 1: function ----
# Answer: Dijkstra over stopping points, remembering the
# previous stop.
# Time: O(m * n * (m + n) + m * n log (m * n)).  Space:
# O(m * n).
import heapq


def maze_route(maze, start, destination):
    rows, cols = len(maze), len(maze[0])

    def roll(r, c, dr, dc):
        steps = 0
        while 0 <= r + dr < rows and 0 <= c + dc < cols and maze[r + dr][c + dc] == 0:
            r += dr
            c += dc
            steps += 1
        return r, c, steps

    best = {tuple(start): 0}
    previous = {tuple(start): None}
    heap = [(0, tuple(start))]
    while heap:
        d, (r, c) = heapq.heappop(heap)
        if d > best[(r, c)]:
            continue
        if [r, c] == list(destination):
            break
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
```

```
            nr, nc, steps = roll(r, c, dr, dc)
            if steps and d + steps < best.get((nr, nc), float("inf")):
                best[(nr, nc)] = d + steps
                previous[(nr, nc)] = (r, c)
                heapq.heappush(heap, (d + steps, (nr, nc)))
    goal = tuple(destination)
    if goal not in best:
        return None
    path = []
    cell = goal
    while cell is not None:
        path.append(cell)
        cell = previous[cell]
    return path[::-1]


# ============================================================
# FOLLOW-UP 2: How many different places can the ball stop
# at
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Search over stopping points and count them.
# Time: O(m * n * (m + n)).  Space: O(m * n).
def reachable_stops(maze, start):
    rows, cols = len(maze), len(maze[0])
    seen = {tuple(start)}
    stack = [tuple(start)]
    while stack:
        r, c = stack.pop()
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r, c
            while 0 <= nr + dr < rows and 0 <= nc + dc < cols and maze[nr + dr][nc + dc] == 0:
                nr += dr
                nc += dc
            if (nr, nc) not in seen:
                seen.add((nr, nc))
                stack.append((nr, nc))
    return len(seen)
```

743. Network Delay Time

Medium · Depth-First Search, Breadth-First Search, Graph, Shortest Path, Dijkstra, Heap (Priority Queue)

leetcode.com/problems/network-delay-time

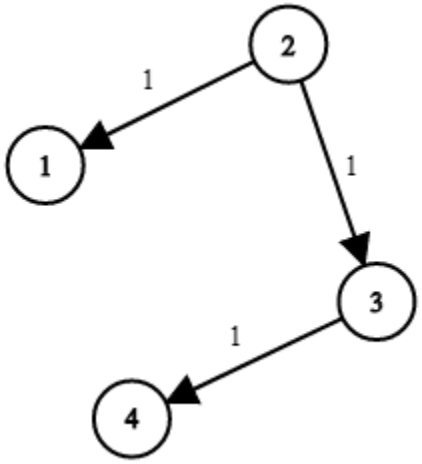
★ **TOP 7 PROBLEM** in Shortest Path: master this one first

PROBLEM

You are given a network of `n` nodes, labeled from `1` to `n`. You are also given `times`, a list of travel times as directed edges `times[i] = (u_i, v_i, w_i)`, where `u_i` is the source node, `v_i` is the target node, and `w_i` is the time it takes for a signal to travel from source to target.

We will send a signal from a given node `k`. Return *the **minimum** time it takes for all the `n` nodes to receive the signal*. If it is impossible for all the `n` nodes to receive the signal, return `-1`.

Example 1:




```
Input: times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2
Output: 2
```

Example 2:

```
Input: times = [[1,2,1]], n = 2, k = 1
Output: 1
```

Example 3:

```
Input: times = [[1,2,1]], n = 2, k = 2
Output: -1
```

Constraints:

- `1 <= k <= n <= 100`
- `1 <= times.length <= 6000`
- `times[i].length == 3`
- `1 <= u_i, v_i <= n`
- `u_i != v_i`
- `0 <= w_i <= 100`
- All the pairs `(u_i, v_i)` are **unique**. (i.e., no multiple edges.)

SOLUTIONS (PYTHON 3)

Solution 1: Naive Dijkstra Algorithm

> **How to think about it**
>
> The time for every node to hear a signal from k is the longest shortest-path distance. n≤ 100 admits naive Dijkstra in $O(n^2)$. Repeatedly pick the unused node with smallest tentative distance and relax its outgoing edges. An adjacency matrix stores weights, missing edges as +∞. The answer is max(dist), or -1 if that value is still infinite.

We define g[u][v] to represent the edge weight from node u to node v. If there is no edge between node u and node v, then g[u][v] = +∞.

We maintain an array dist, where dist[i] represents the shortest path length from node k to node i. Initially, we set all dist[i] to +∞, except for dist[k - 1] = 0. We define an array vis, where vis[i] indicates whether node i has been visited. Initially, we set all vis[i] to false.

Each time, we find the unvisited node t with the smallest distance, and then perform relaxation operations centered on node t. For each node j, if dist[j] > dist[t] + g[t][j], we update dist[j] = dist[t] + g[t][j].

Finally, we return the maximum value in dist as the answer. If the answer is +∞, it means there are unreachable nodes, and we return -1.

The time complexity is $O(n^2 + m)$, and the space complexity is $O(n^2)$. Here, n and m are the number of nodes and edges, respectively.

Python 3

```python
class Solution:
    def networkDelayTime(self, times: List[List[int]], n: int, k: int) -> int:
        g = [[inf] * n for _ in range(n)]
        for u, v, w in times:
            g[u - 1][v - 1] = w
        dist = [inf] * n
        dist[k - 1] = 0
        vis = [False] * n
        for _ in range(n):
            t = -1
            for j in range(n):
                if not vis[j] and (t == -1 or dist[t] > dist[j]):
                    t = j
            vis[t] = True
            for j in range(n):
                dist[j] = min(dist[j], dist[t] + g[t][j])
        ans = max(dist)
        return -1 if ans == inf else ans
```

Solution 2: Heap-Optimized Dijkstra Algorithm

> **How to think about it**
>
> Solution 1 scans all nodes to pick the next vertex; that is wasteful on a sparse graph. A heap pops the current best in O(mlog m). Store an adjacency list. Pop (d,u); skip stale distances; otherwise relax neighbors onto the heap. The answer is still the max finite distance.

We can use a priority queue (heap) to optimize the naive Dijkstra algorithm.

We define g[u] to represent all adjacent edges of node u, and dist[u] to represent the shortest path length from node k to node u. Initially, we set all dist[u] to +∞, except for dist[k - 1] = 0.

We define a priority queue pq, where each element is (d, u), representing the distance d from node u to node k. Each time, we take out the node (d, u) with the smallest distance from pq. If d >\textit{dist}[u], we skip this node. Otherwise, we traverse all adjacent edges of nodeu. For each adjacent edge(v, w), if\textit{dist}[v] > \textit{dist}[u] + w, we update\textit{dist}[v] = \textit{dist}[u] + wand add(\textit{dist}[v], v)to\textit{pq}$.

Finally, we return the maximum value in dist as the answer. If the answer is +∞, it means there are unreachable nodes, and we return -1.

The time complexity is O(m × log m + n), and the space complexity is O(n + m). Here, n and m are the number of nodes and edges, respectively.

Python 3

```python
class Solution:
    def networkDelayTime(self, times: List[List[int]], n: int, k: int) -> int:
        g = [[] for _ in range(n)]
        for u, v, w in times:
            g[u - 1].append((v - 1, w))
        dist = [inf] * n
        dist[k - 1] = 0
        pq = [(0, k - 1)]
        while pq:
            d, u = heappop(pq)
            if d > dist[u]:
                continue
            for v, w in g[u]:
                if (nd := d + w) < dist[v]:
                    dist[v] = nd
                    heappush(pq, (nd, v))
        ans = max(dist)
        return -1 if ans == inf else ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Which node receives the signal last
# 2. The same answer with Bellman-Ford


# ============================================================
# FOLLOW-UP 1: Which node receives the signal last
# ============================================================

# ---- follow-up 1: function ----
# Answer: Dijkstra distances; the node with the biggest
# finite distance.
# Time: O(E log V).  Space: O(V + E).
import heapq


def last_node_to_receive(times, n, k):
    graph = [[] for _ in range(n + 1)]
    for u, v, w in times:
        graph[u].append((v, w))
    dist = {k: 0}
    heap = [(0, k)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist.get(v, float("inf")):
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    if len(dist) < n:
        return None
    node = max(dist, key=lambda x: (dist[x], -x))
    return node, dist[node]


# ============================================================
# FOLLOW-UP 2: The same answer with Bellman-Ford
```

```
# ============================================================

# ---- follow-up 2: function ----
# Answer: Relax every edge n - 1 times.
# Time: O(V * E).  Space: O(V).
def network_delay_bellman_ford(times, n, k):
    INF = float("inf")
    dist = [INF] * (n + 1)
    dist[k] = 0
    for _ in range(n - 1):
        changed = False
        for u, v, w in times:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                changed = True
        if not changed:
            break
    best = max(dist[1:])
    return -1 if best == INF else best
```

1631. Path With Minimum Effort

Medium · Depth-First Search, Breadth-First Search, Union Find, Array, Binary Search, Matrix, Dijkstra, Heap (Priority Queue)

leetcode.com/problems/path-with-minimum-effort

★ **TOP 7 PROBLEM** in Shortest Path: master this one first

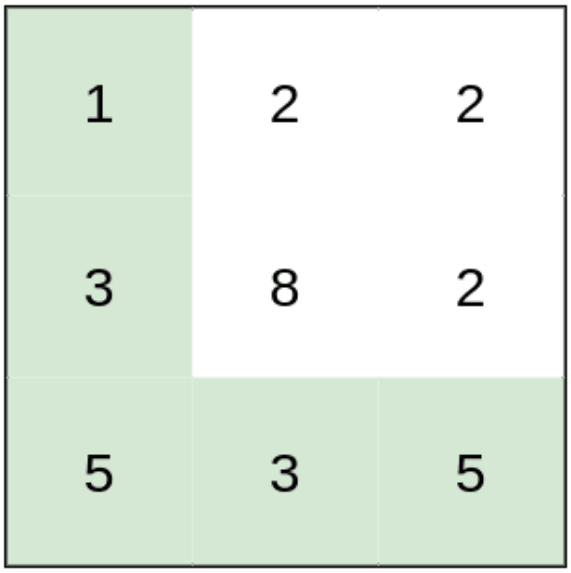
PROBLEM

You are a hiker preparing for an upcoming hike. You are given `heights`, a 2D array of size `rows x columns`, where `heights[row][col]` represents the height of cell `(row, col)`. You are situated in the top-left cell, `(0, 0)`, and you hope to travel to the bottom-right cell, `(rows-1, columns-1)` (i.e., **0-indexed**). You can move **up**, **down**, **left**, or **right**, and you wish to find a route that requires the minimum **effort**.

A route's **effort** is the **maximum absolute difference** in heights between two consecutive cells of the route.

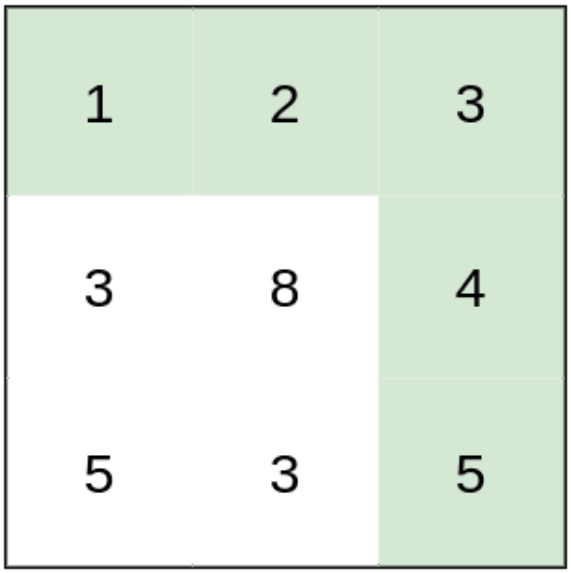
Return *the minimum **effort** required to travel from the top-left cell to the bottom-right cell.*

Example 1:



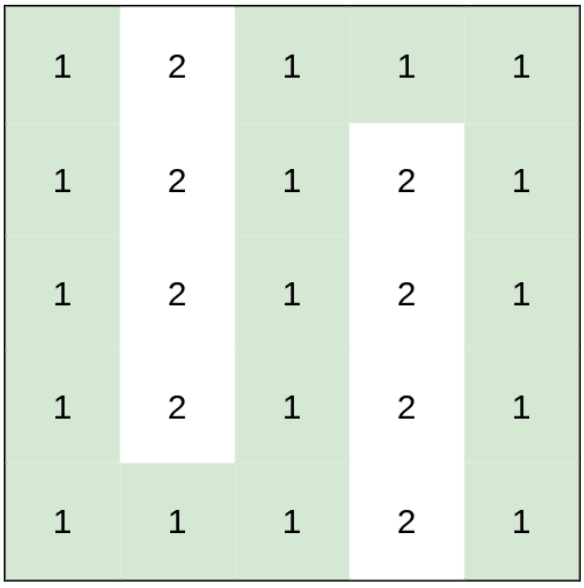

```
Input: heights = [[1,2,2],[3,8,2],[5,3,5]]
Output: 2
Explanation: The route of [1,3,5,3,5] has a maximum absolute difference of 2 in consecutive cells.
This is better than the route of [1,2,2,2,5], where the maximum absolute difference is 3.
```

Example 2:




```
Input: heights = [[1,2,3],[3,8,4],[5,3,5]]
Output: 1
Explanation: The route of [1,2,3,4,5] has a maximum absolute difference of 1 in consecutive cells,
which is better than route [1,3,5,3,5].
```

Example 3:




```
Input: heights = [[1,2,1,1,1],[1,2,1,2,1],[1,2,1,2,1],[1,2,1,2,1],[1,1,1,2,1]]
Output: 0
Explanation: This route does not require any effort.
```

Constraints:

- `rows == heights.length`
- `columns == heights[i].length`
- `1 <= rows, columns <= 100`
- `1 <= heights[i][j] <= 10^6`

Solution 1: Union-Find

How to think about it

A path's cost is the maximum adjacent height difference; we want the minimum such bottleneck. Binary search would still need a connectivity check; equivalently, treat cells as vertices and differences as edge weights and add edges in increasing order. A disjoint-set unions edges by weight; the first time start and end meet, that weight is the min-max effort. Collect 4-neighbor edges, sort them, and union until connected(0,mn-1).

For this problem, we can treat each cell as a node in a graph, and the absolute difference in height between two adjacent cells as the weight of the edge. Therefore, this problem is to solve the connectivity problem from the top-left node to the bottom-right node.

We first construct a set of edges, then sort them in ascending order of edge weight, and add edges one by one until the top-left node and the bottom-right node are connected. At this point, the weight of the edge is the minimum physical consumption value required by the problem.

The time complexity is $O(m \times n \times \log(m \times n))$, and the space complexity is $O(m \times n)$. Here, m and n are the number of rows and columns in the two-dimensional array, respectively.

Python 3

```python
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True

    def connected(self, a, b):
        return self.find(a) == self.find(b)


class Solution:
    def minimumEffortPath(self, heights: List[List[int]]) -> int:
        m, n = len(heights), len(heights[0])
        uf = UnionFind(m * n)
        e = []
        dirs = (0, 1, 0)
        for i in range(m):
            for j in range(n):
                for a, b in pairwise(dirs):
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n:
                        e.append(
                            (abs(heights[i][j] - heights[x][y]), i * n + j, x * n + y)
                        )
        e.sort()
        for h, a, b in e:
            uf.union(a, b)
            if uf.connected(0, m * n - 1):
                return h
        return 0
```

Solution 2: Binary Search + BFS

> **How to think about it**
>
> Solution 1 sorts every edge. A candidate h works iff start reaches end using only differences ≤ h, and that predicate is monotone in h. Binary-search h in $[0,10^6]$ and BFS each mid, without a disjoint-set.

We notice that if the maximum physical consumption value of a path is x, then for any y > x, this path also meets the conditions. This shows monotonicity, so we can use the binary search method to find the minimum physical consumption value that meets the conditions.

We define the left boundary of the binary search as l=0, and the right boundary as $r=10^6$. Each time we take mid=(l+r)/2, then use BFS to determine whether there is a path from the top-left corner to the bottom-right corner, so that the absolute difference in height between adjacent nodes on the path is not greater than mid. If it exists, it means that mid may still be the minimum physical consumption value that meets the conditions, so we set r=mid, otherwise we set l=mid+1.

The time complexity is O(m × n × log M), and the space complexity is O(m × n). Here, m and n are the number of rows and columns in the two-dimensional array, respectively, and M is the maximum value in the two-dimensional array. In this

problem, M=10^6.

Python 3

```python
class Solution:
    def minimumEffortPath(self, heights: List[List[int]]) -> int:
        def check(h: int) -> bool:
            q = deque([(0, 0)])
            vis = {(0, 0)}
            dirs = (-1, 0, 1, 0, -1)
            while q:
                for _ in range(len(q)):
                    i, j = q.popleft()
                    if i == m - 1 and j == n - 1:
                        return True
                    for a, b in pairwise(dirs):
                        x, y = i + a, j + b
                        if (
                            0 <= x < m
                            and 0 <= y < n
                            and (x, y) not in vis
                            and abs(heights[i][j] - heights[x][y]) <= h
                        ):
                            q.append((x, y))
                            vis.add((x, y))
            return False

        m, n = len(heights), len(heights[0])
        return bisect_left(range(10**6), True, key=check)
```

Solution 3: Heap-optimized Dijkstra Algorithm

> **How to think about it**
>
> The first two methods sort globally or search repeatedly. Treating a move as the new bottleneck max(so far, | Δ h |) is a shortest-path problem with max instead of +. Dijkstra with a heap stores the best bottleneck dist to each cell and relaxes four neighbors; the bottom-right entry is the answer.

We can treat each cell as a node in a graph, and the absolute difference in height between two adjacent cells as the weight of the edge. Therefore, this problem is to solve the shortest path problem from the top-left node to the bottom-right node.

We can use the Dijkstra algorithm to solve the shortest path problem, and use a priority queue (heap) to optimize the time complexity. Specifically, we maintain a two-dimensional array dist of size m × n, where dist[i][j] represents the maximum weight of the shortest path from the top-left corner to the node (i,j). Initially, dist[0][0]=0, and all other elements are positive infinity.

We use a priority queue (heap) to store nodes, and each time we take out the node with the smallest weight from the priority queue (heap), then update the weights of its adjacent nodes. If the weight of an adjacent node changes, then we add this node to the priority queue (heap). When the priority queue (heap) is empty, it means that we have found the shortest path.

The time complexity is O(m × n × log(m × n)), and the space complexity is O(m × n). Here, m and n are the number of rows and columns in the two-dimensional array, respectively.

Python 3

```python
class Solution:
    def minimumEffortPath(self, heights: List[List[int]]) -> int:
        m, n = len(heights), len(heights[0])
        dist = [[inf] * n for _ in range(m)]
        dist[0][0] = 0
        dirs = (-1, 0, 1, 0, -1)
        q = [(0, 0, 0)]
        while q:
            t, i, j = heappop(q)
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if (
                    0 <= x < m
                    and 0 <= y < n
                    and (d := max(t, abs(heights[i][j] - heights[x][y]))) < dist[x][y]
                ):
                    dist[x][y] = d
                    heappush(q, (d, x, y))
        return int(dist[-1][-1])
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the path
# 2. How many cells can be reached with effort at most e


# ============================================================
# FOLLOW-UP 1: Return the path
# ============================================================

# ---- follow-up 1: function ----
# Answer: Dijkstra on the effort, remembering the previous
# cell.
# Time: O(m * n log (m * n)).  Space: O(m * n).
import heapq


def minimum_effort_path(heights):
    rows, cols = len(heights), len(heights[0])
    best = {(0, 0): 0}
    previous = {(0, 0): None}
    heap = [(0, 0, 0)]
    while heap:
        effort, r, c = heapq.heappop(heap)
        if effort > best[(r, c)]:
            continue
        if (r, c) == (rows - 1, cols - 1):
            break
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols:
                e = max(effort, abs(heights[nr][nc] - heights[r][c]))
                if e < best.get((nr, nc), float("inf")):
                    best[(nr, nc)] = e
                    previous[(nr, nc)] = (r, c)
                    heapq.heappush(heap, (e, nr, nc))
    path = []
    cell = (rows - 1, cols - 1)
    while cell is not None:
        path.append(cell)
        cell = previous[cell]
    return best[(rows - 1, cols - 1)], path[::-1]
```

```
# ============================================================
# FOLLOW-UP 2: How many cells can be reached with effort
# at most e
# ============================================================

# ---- follow-up 2: function ----
# Answer: Flood fill that only crosses steps of at most e.
# Time: O(m * n).  Space: O(m * n).
def cells_with_effort_at_most(heights, limit):
    rows, cols = len(heights), len(heights[0])
    seen = {(0, 0)}
    stack = [(0, 0)]
    while stack:
        r, c = stack.pop()
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and (nr, nc) not in seen and abs(heights[nr][nc] -
heights[r][c]) <= limit:
```

```
                seen.add((nr, nc))
                stack.append((nr, nc))
    return len(seen)
```

PART 2 · SHORTEST PATH · PROBLEM 4 OF 9

2976. Minimum Cost to Convert String I

Medium · Graph, Array, String, Shortest Path

leetcode.com/problems/minimum-cost-to-convert-string-i

★ **TOP 7 PROBLEM** in Shortest Path: master this one first

PROBLEM

You are given two **0-indexed** strings `source` and `target`, both of length `n` and consisting of **lowercase** English letters. You are also given two **0-indexed** character arrays `original` and `changed`, and an integer array `cost`, where `cost[i]` represents the cost of changing the character `original[i]` to the character `changed[i]`.

You start with the string `source`. In one operation, you can pick a character `x` from the string and change it to the character `y` at a cost of `z` **if** there exists **any** index `j` such that `cost[j] == z`, `original[j] == x`, and `changed[j] == y`.

Return *the **minimum** cost to convert the string* `source` *to the string* `target` *using **any** number of operations. If it is impossible to convert* `source` *to* `target`, *return* `-1`.

Note that there may exist indices `i`, `j` such that `original[j] == original[i]` and `changed[j] == changed[i]`.

Example 1:

```
Input: source = "abcd", target = "acbe", original = ["a","b","c","c","e","d"], changed =
["b","c","b","e","b","e"], cost = [2,5,5,1,2,20]
Output: 28
Explanation: To convert the string "abcd" to string "acbe":
- Change value at index 1 from 'b' to 'c' at a cost of 5.
- Change value at index 2 from 'c' to 'e' at a cost of 1.
- Change value at index 2 from 'e' to 'b' at a cost of 2.
- Change value at index 3 from 'd' to 'e' at a cost of 20.
The total cost incurred is 5 + 1 + 2 + 20 = 28.
It can be shown that this is the minimum possible cost.
```

Example 2:

```
Input: source = "aaaa", target = "bbbb", original = ["a","c"], changed = ["c","b"], cost = [1,2]
Output: 12
Explanation: To change the character 'a' to 'b' change the character 'a' to 'c' at a cost of 1,
followed by changing the character 'c' to 'b' at a cost of 2, for a total cost of 1 + 2 = 3. To change
all occurrences of 'a' to 'b', a total cost of 3 * 4 = 12 is incurred.
```

Example 3:

```
Input: source = "abcd", target = "abce", original = ["a"], changed = ["e"], cost = [10000]
Output: -1
Explanation: It is impossible to convert source to target because the value at index 3 cannot be
changed from 'd' to 'e'.
```

Constraints:

- `1 <= source.length == target.length <= 10^5`
- `source`, `target` consist of lowercase English letters.
- `1 <= cost.length == original.length == changed.length <= 2000`
- `original[i]`, `changed[i]` are lowercase English letters.
- `1 <= cost[i] <= 10^6`
- `original[i] != changed[i]`

SOLUTION (PYTHON 3)

Solution 1: Floyd Algorithm

How to think about it

A letter may be rewritten into another at a given cost; we want the cheapest map from source to target. There are 26 letters and rewrites compose, so we need all-pairs shortest paths. Floyd finishes g[x][y] in 26^3. Sum per position; an unreachable pair fails the instance. Parallel edges keep the lighter cost. The string length is 10^5, so preprocessing is separate from the scan.

According to the problem description, we can consider each letter as a node, and the conversion cost between each pair of letters as a directed edge. We first initialize a 26 × 26 two-dimensional array g, where g[i][j] represents the minimum cost of converting letter i to letter j. Initially, g[i][j] = ∞, and if i = j, then g[i][j] = 0.

Next, we traverse the arrays original, changed, and cost. For each index i, we update the cost cost[i] of converting original[i] to changed[i] to g[original[i]][changed[i]], taking the minimum value.

Then, we use the Floyd algorithm to calculate the minimum cost between any two nodes in g. Finally, we traverse the strings source and target. If source[i] ≠ target[i] and g[source[i]][target[i]] ≥ ∞, it means that the conversion cannot be completed, so we return -1. Otherwise, we add g[source[i]][target[i]] to the answer.

After the traversal ends, we return the answer.

The time complexity is $O(m + n + |\Sigma|^3)$, and the space complexity is $O(|\Sigma|^2)$. Where m and n are the lengths of the arrays original and source respectively; and |Σ| is the size of the alphabet, that is, |Σ| = 26.

Python 3

```python
class Solution:
    def minimumCost(
        self,
        source: str,
        target: str,
        original: List[str],
        changed: List[str],
        cost: List[int],
    ) -> int:
        g = [[inf] * 26 for _ in range(26)]
        for i in range(26):
            g[i][i] = 0
        for x, y, z in zip(original, changed, cost):
            x = ord(x) - ord('a')
            y = ord(y) - ord('a')
            g[x][y] = min(g[x][y], z)
        for k in range(26):
            for i in range(26):
                for j in range(26):
                    g[i][j] = min(g[i][j], g[i][k] + g[k][j])
        ans = 0
        for a, b in zip(source, target):
            if a != b:
                x, y = ord(a) - ord('a'), ord(b) - ord('a')
                if g[x][y] >= inf:
                    return -1
                ans += g[x][y]
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Floyd-Warshall instead of Dijkstra for every letter
# 2. The chain of letters used to turn one letter into
# another


# ============================================================
# FOLLOW-UP 1: Floyd-Warshall instead of Dijkstra for
# every letter
# ============================================================

# ---- follow-up 1: function ----
# Answer: All pairs of the 26 letters at once.
# Time: O(26^3 + n + m).  Space: O(26^2).
def convert_cost_floyd(source, target, original, changed, cost):
    INF = float("inf")
    d = [[INF] * 26 for _ in range(26)]
    for i in range(26):
        d[i][i] = 0
    for a, b, c in zip(original, changed, cost):
        x, y = ord(a) - 97, ord(b) - 97
        d[x][y] = min(d[x][y], c)
    for k in range(26):
        for i in range(26):
            for j in range(26):
                if d[i][k] + d[k][j] < d[i][j]:
                    d[i][j] = d[i][k] + d[k][j]
    total = 0
    for a, b in zip(source, target):
        step = d[ord(a) - 97][ord(b) - 97]
        if step == INF:
            return -1
        total += step
    return total


# ============================================================
# FOLLOW-UP 2: The chain of letters used to turn one
# letter into another
# ============================================================

# ---- follow-up 2: function ----
# Answer: Dijkstra over the 26 letters, remembering the
# previous letter.
# Time: O((m + 26) log 26).  Space: O(26).
import heapq


def letter_chain(original, changed, cost, a, b):
    graph = {}
    for x, y, c in zip(original, changed, cost):
        graph.setdefault(x, []).append((y, c))
    dist = {a: 0}
    previous = {a: None}
    heap = [(0, a)]
    while heap:
        d, x = heapq.heappop(heap)
        if d > dist[x]:
            continue
        for y, c in graph.get(x, []):
            if d + c < dist.get(y, float("inf")):
                dist[y] = d + c
                previous[y] = x
                heapq.heappush(heap, (d + c, y))
```

```
    if b not in dist:
        return None
    chain = []
    while b is not None:
        chain.append(b)
        b = previous[b]
    return chain[::-1]
```

3341. Find Minimum Time to Reach Last Room I

Medium · Graph, Array, Matrix, Shortest Path, Heap (Priority Queue)

leetcode.com/problems/find-minimum-time-to-reach-last-room-i

PROBLEM

There is a dungeon with `n x m` rooms arranged as a grid.

You are given a 2D array `moveTime` of size `n x m`, where `moveTime[i][j]` represents the **minimum** time in seconds **after** which the room opens and can be moved to. You start from the room `(0, 0)` at time `t = 0` and can move to an **adjacent** room. Moving between adjacent rooms takes *exactly* one second.

Return the **minimum** time to reach the room `(n - 1, m - 1)`.

Two rooms are **adjacent** if they share a common wall, either *horizontally* or *vertically*.

Example 1:

Input: moveTime = [[0,4],[4,4]]

Output: 6

Explanation:

The minimum time required is 6 seconds.

- At time `t == 4`, move from room `(0, 0)` to room `(1, 0)` in one second.
- At time `t == 5`, move from room `(1, 0)` to room `(1, 1)` in one second.

Example 2:

Input: moveTime = [[0,0,0],[0,0,0]]

Output: 3

Explanation:

The minimum time required is 3 seconds.

- At time `t == 0`, move from room `(0, 0)` to room `(1, 0)` in one second.
- At time `t == 1`, move from room `(1, 0)` to room `(1, 1)` in one second.
- At time `t == 2`, move from room `(1, 1)` to room `(1, 2)` in one second.

Example 3:

Input: moveTime = [[0,1],[1,2]]

Output: 3

Constraints:

- `2 <= n == moveTime.length <= 50`
- `2 <= m == moveTime[i].length <= 50`
- `0 <= moveTime[i][j] <= 10^9`

SOLUTION (PYTHON 3)

Solution 1: Dijkstra's Algorithm

How to think about it

A move into (x,y) cannot happen before moveTime[x][y], and every step costs 1. Weights are nonnegative but depend on arrival time, so BFS is wrong. With n,m ≤ 50, Dijkstra on time is enough. The relaxation is t=max(moveTime[x][y],dist[i][j])+1. The first time the priority queue pops the destination is the answer.

We define a two-dimensional array dist, where dist[i][j] represents the minimum time required to reach room (i, j) from the starting point. Initially, we set all elements in the dist array to infinity, and then set the dist value of the starting point (0, 0) to 0.

We use a priority queue pq to store each state, where each state consists of three values (d, i, j), representing the time d required to reach room (i, j) from the starting point. Initially, we add the starting point (0, 0, 0) to pq.

In each iteration, we take the front element (d, i, j) from pq. If (i, j) is the endpoint, we return d. If d is greater than dist[i][j], we skip this state. Otherwise, we enumerate the four adjacent positions (x, y) of (i, j). If (x, y) is within the map, we calculate the final time t from (i, j) to (x, y) as t = max(moveTime[x][y], dist[i][j]) + 1. If t is less than dist[x][y], we update the value of dist[x][y] and add (t, x, y) to pq.

The time complexity is O(n × m × log (n × m)), and the space complexity is O(n × m). Here, n and m are the number of rows and columns of the map, respectively.

Python 3

```python
class Solution:
    def minTimeToReach(self, moveTime: List[List[int]]) -> int:
        n, m = len(moveTime), len(moveTime[0])
        dist = [[inf] * m for _ in range(n)]
        dist[0][0] = 0
        pq = [(0, 0, 0)]
        dirs = (-1, 0, 1, 0, -1)
        while 1:
            d, i, j = heappop(pq)
            if i == n - 1 and j == m - 1:
                return d
            if d > dist[i][j]:
                continue
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < n and 0 <= y < m:
                    t = max(moveTime[x][y], dist[i][j]) + 1
                    if dist[x][y] > t:
                        dist[x][y] = t
                        heappush(pq, (t, x, y))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the path of rooms
# 2. The earliest arrival time of EVERY room


# ============================================================
# FOLLOW-UP 1: Return the path of rooms
# ============================================================

# ---- follow-up 1: function ----
# Answer: Dijkstra with the previous room remembered.
# Time: O(m * n log (m * n)).  Space: O(m * n).
import heapq


def fastest_room_path(move_time):
    rows, cols = len(move_time), len(move_time[0])
    best = {(0, 0): 0}
    previous = {(0, 0): None}
    heap = [(0, 0, 0)]
    while heap:
        t, r, c = heapq.heappop(heap)
        if t > best[(r, c)]:
            continue
        if (r, c) == (rows - 1, cols - 1):
            break
```

```
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols:
                arrive = max(t, move_time[nr][nc]) + 1
                if arrive < best.get((nr, nc), float("inf")):
                    best[(nr, nc)] = arrive
                    previous[(nr, nc)] = (r, c)
                    heapq.heappush(heap, (arrive, nr, nc))
    path = []
    cell = (rows - 1, cols - 1)
    while cell is not None:
        path.append(cell)
        cell = previous[cell]
    return best[(rows - 1, cols - 1)], path[::-1]


# ============================================================
# FOLLOW-UP 2: The earliest arrival time of EVERY room
# ============================================================

# ---- follow-up 2: function ----
# Answer: Run the same Dijkstra to the end and print the
# distances.
# Time: O(m * n log (m * n)).  Space: O(m * n).
def arrival_times(move_time):
    rows, cols = len(move_time), len(move_time[0])
    best = [[float("inf")] * cols for _ in range(rows)]
    best[0][0] = 0
    heap = [(0, 0, 0)]
    while heap:
        t, r, c = heapq.heappop(heap)
        if t > best[r][c]:
            continue
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
```

```
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols:
                arrive = max(t, move_time[nr][nc]) + 1
                if arrive < best[nr][nc]:
                    best[nr][nc] = arrive
                    heapq.heappush(heap, (arrive, nr, nc))
    return best
```

3650. Minimum Cost Path with Edge Reversals

Medium · Graph, Shortest Path, Heap (Priority Queue)

leetcode.com/problems/minimum-cost-path-with-edge-reversals

PROBLEM

You are given a directed, weighted graph with `n` nodes labeled from 0 to `n - 1`, and an array `edges` where `edges[i] = [u_i, v_i, w_i]` represents a directed edge from node `u_i` to node `v_i` with cost `w_i`.

Each node `u_i` has a switch that can be used **at most once**: when you arrive at `u_i` and have not yet used its switch, you may activate it on one of its incoming edges `v_i → u_i` reverse that edge to `u_i → v_i` and **immediately** traverse it.

The reversal is only valid for that single move, and using a reversed edge costs `2 * w_i`.

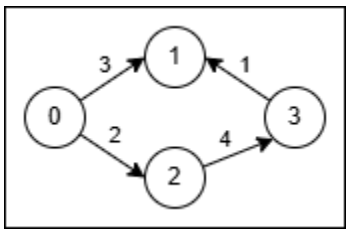
Return the **minimum** total cost to travel from node 0 to node `n - 1`. If it is not possible, return -1.

Example 1:

Input: n = 4, edges = [[0,1,3],[3,1,1],[2,3,4],[0,2,2]]

Output: 5

Explanation:



- Use the path `0 → 1` (cost 3).
- At node 1 reverse the original edge `3 → 1` into `1 → 3` and traverse it at cost `2 * 1 = 2`.
- Total cost is `3 + 2 = 5`.

Example 2:

Input: n = 4, edges = [[0,2,1],[2,1,1],[1,3,1],[2,3,3]]

Output: 3

Explanation:

- No reversal is needed. Take the path `0 → 2` (cost 1), then `2 → 1` (cost 1), then `1 → 3` (cost 1).
- Total cost is `1 + 1 + 1 = 3`.

Constraints:

- $2 \le n \le 5 * 10^4$
- $1 \le edges.length \le 10^5$
- `edges[i] = [u_i, v_i, w_i]`
- $0 \le u_i, v_i \le n - 1$
- $1 \le w_i \le 1000$

SOLUTION (PYTHON 3)

Solution 1: Dijkstra's Algorithm

How to think about it

A forward traversal costs w and a reversal costs 2w. We want the cheapest walk from 0 to n-1. Enumerating which edges to reverse is exponential. Materialize the reverse as an extra arc: keep (u,v,w) and add (v,u,2w). The instance becomes an ordinary nonnegative shortest-path problem. Heap Dijkstra from 0 returns the first time n-1 is popped; an empty heap means unreachable. Each original edge yields two directed arcs, so the complexity tracks mlog m.

According to the problem description, we can construct a directed graph g where each edge (u, v) allows for two types of traversal:

- Direct traversal with cost w, corresponding to edge (u, v).
- Reverse traversal with cost 2w, corresponding to edge (v, u).

Then, we can use Dijkstra's algorithm on graph g to find the shortest path from node 0 to node n-1, which corresponds to the minimum total cost required.

Specifically, we define a priority queue pq, where each element is a tuple (d, u), indicating that the current minimum cost to reach node u is d. We also define an array dist, where dist[u] represents the minimum cost from node 0 to node u. Initially, we set dist[0] = 0, and the costs for all other nodes to infinity, then push (0, 0) into the queue.

In each iteration, we extract the node (d, u) with the minimum cost from the priority queue. If d is greater than dist[u], we skip this node. Otherwise, we traverse all neighbors v of node u, calculating the new cost nd = d + w to reach node v via node u. If nd is less than dist[v], we update dist[v] = nd and push (nd, v) into the queue.

When we extract node n-1, the current d is the minimum total cost from node 0 to node n-1. If the priority queue becomes empty and node n-1 has not been extracted, it implies that node n-1 is unreachable, so we return -1.

The time complexity is O(n + m × log m), and the space complexity is O(n + m). Here, n and m refer to the number of nodes and edges, respectively.

Python 3

```python
class Solution:
    def minCost(self, n: int, edges: List[List[int]]) -> int:
        g = [[] for _ in range(n)]
        for u, v, w in edges:
            g[u].append((v, w))
            g[v].append((u, w * 2))
        pq = [(0, 0)]
        dist = [inf] * n
        dist[0] = 0
        while pq:
            d, u = heappop(pq)
            if d > dist[u]:
                continue
            if u == n - 1:
                return d
            for v, w in g[u]:
                nd = d + w
                if nd < dist[v]:
                    dist[v] = nd
                    heappush(pq, (nd, v))
        return -1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the path of nodes
# 2. How many reversals does the cheapest route use
# (fewest among equal costs)


# ============================================================
# FOLLOW-UP 1: Return the path of nodes
# ============================================================

# ---- follow-up 1: function ----
# Answer: Dijkstra where every directed edge a -> b costs
# w forward, and reversed (b to a) costs 2 * w; remember
# the previous node.
# Time: O((n + E) log n).  Space: O(n + E).
import heapq


def cheapest_route(n, edges):
    graph = [[] for _ in range(n)]
    for a, b, w in edges:
        graph[a].append((b, w))
        graph[b].append((a, 2 * w))
    dist = [float("inf")] * n
    previous = [None] * n
    dist[0] = 0
    heap = [(0, 0)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                previous[v] = u
                heapq.heappush(heap, (d + w, v))
    if dist[n - 1] == float("inf"):
        return None
    path = []
    node = n - 1
    while node is not None:
        path.append(node)
        node = previous[node]
    return dist[n - 1], path[::-1]


# ============================================================
# FOLLOW-UP 2: How many reversals does the cheapest route
# use (fewest among equal costs)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Dijkstra on (cost, reversals).
# Time: O((n + E) log n).  Space: O(n + E).
def reversals_used(n, edges):
    graph = [[] for _ in range(n)]
    for a, b, w in edges:
        graph[a].append((b, w, 0))
        graph[b].append((a, 2 * w, 1))
    best = [(float("inf"), 0)] * n
```

```
    best[0] = (0, 0)
    heap = [(0, 0, 0)]
    while heap:
        d, rev, u = heapq.heappop(heap)
```

```
        if (d, rev) > best[u]:
            continue
        for v, w, r in graph[u]:
            candidate = (d + w, rev + r)
            if candidate < best[v]:
                best[v] = candidate
                heapq.heappush(heap, (candidate[0], candidate[1], v))
    return None if best[n - 1][0] == float("inf") else best[n - 1][1]
```

778. Swim in Rising Water

Hard · Depth-First Search, Breadth-First Search, Union Find, Minimax, Array, Binary Search, Matrix, Dijkstra, Heap (Priority Queue)

leetcode.com/problems/swim-in-rising-water

★ **TOP 7 PROBLEM** in Shortest Path: master this one first

PROBLEM

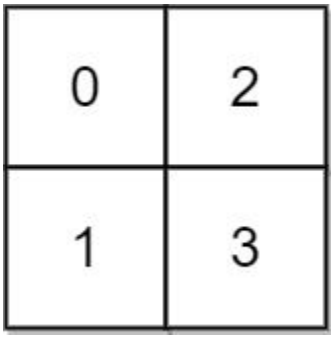
You are given an `n x n` integer matrix `grid` where each value `grid[i][j]` represents the elevation at that point `(i, j)`.

It starts raining, and water gradually rises over time. At time `t`, the water level is `t`, meaning **any** cell with elevation less than equal to `t` is submerged or reachable.

You can swim from a square to another 4-directionally adjacent square if and only if the elevation of both squares individually are at most `t`. You can swim infinite distances in zero time. Of course, you must stay within the boundaries of the grid during your swim.

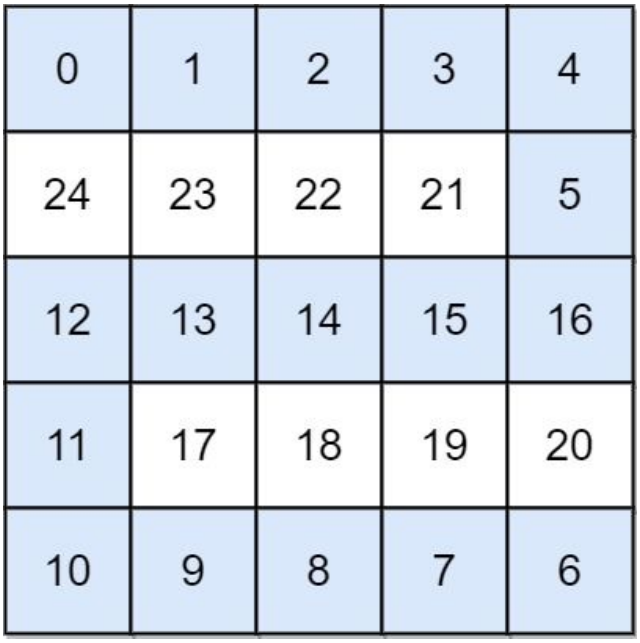
Return *the minimum time until you can reach the bottom right square* `(n - 1, n - 1)` *if you start at the top left square* `(0, 0)`.

Example 1:




```
Input: grid = [[0,2],[1,3]]
Output: 3
Explanation:
At time 0, you are in grid location (0, 0).
You cannot go anywhere else because 4-directionally adjacent neighbors have a higher elevation than t
= 0.
You cannot reach point (1, 1) until time 3.
When the depth of water is 3, we can swim anywhere inside the grid.
```

Example 2:




```
Input: grid = [[0,1,2,3,4],[24,23,22,21,5],[12,13,14,15,16],[11,17,18,19,20],[10,9,8,7,6]]
Output: 16
Explanation: The final route is shown.
We need to wait until time 16 so that (0, 0) and (4, 4) are connected.
```

Constraints:

- `n == grid.length`
- `n == grid[i].length`
- `1 <= n <= 50`
- `0 <= grid[i][j] < n^2`
- Each value `grid[i][j]` is **unique**.

SOLUTION (PYTHON 3)

Solution 1: Union Find

How to think about it

At time t we may walk cells ≤ t. Heights are a permutation of 0..n^2-1, so add cells in height order. When the cell of height t is unioned with already-present neighbors, the first time start meets end is the answer. Map height to id, then union as t grows. O(n^2α).

We can map each position (i, j) to an ID id = i × n + j, and use a union-find data structure to maintain connected components.

First, we use a one-dimensional array hi to record the position ID corresponding to each height, i.e., hi[h] represents the position ID with height h.

Then we traverse from height 0 to height n^2 - 1. For each height t, we merge position hi[t] with its four adjacent positions that have heights not exceeding t. If after merging, position 0 and position n^2 - 1 are connected, then we have found the minimum time t, and we return t.

The time complexity is O(n^2 × log n) and the space complexity is O(n^2), where n is the side length of the matrix.

Python 3

```
class Solution:
    def swimInWater(self, grid: List[List[int]]) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(grid)
        m = n * n
        p = list(range(m))
        hi = [0] * m
        for i, row in enumerate(grid):
            for j, h in enumerate(row):
                hi[h] = i * n + j
        dirs = (-1, 0, 1, 0, -1)
        for t in range(m):
            x, y = divmod(hi[t], n)
            for dx, dy in pairwise(dirs):
                nx, ny = x + dx, y + dy
                if 0 <= nx < n and 0 <= ny < n and grid[nx][ny] <= t:
                    p[find(x * n + y)] = find(nx * n + ny)
            if find(0) == find(m - 1):
                return t
        return 0
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the path of cells
# 2. Binary search on the time


# ============================================================
# FOLLOW-UP 1: Return the path of cells
# ============================================================

# ---- follow-up 1: function ----
# Answer: Best-first search on the highest cell so far,
# remembering the previous cell.
# Time: O(n^2 log n).  Space: O(n^2).
import heapq


def swim_path(grid):
    n = len(grid)
    heap = [(grid[0][0], 0, 0)]
    previous = {(0, 0): None}
    seen = {(0, 0)}
    while heap:
        t, r, c = heapq.heappop(heap)
        if (r, c) == (n - 1, n - 1):
            path = []
            cell = (r, c)
            while cell is not None:
                path.append(cell)
                cell = previous[cell]
            return t, path[::-1]
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < n and 0 <= nc < n and (nr, nc) not in seen:
                seen.add((nr, nc))
                previous[(nr, nc)] = (r, c)
                heapq.heappush(heap, (max(t, grid[nr][nc]), nr, nc))
```

```
# ============================================================
# FOLLOW-UP 2: Binary search on the time
# ============================================================

# ---- follow-up 2: function ----
# Answer: For a time t, search the squares of height at
# most t from the corner.
# Time: O(n^2 log n).  Space: O(n^2).
def swim_by_binary_search(grid):
    n = len(grid)

    def can(t):
        if grid[0][0] > t:
            return False
        seen = {(0, 0)}
        stack = [(0, 0)]
        while stack:
            r, c = stack.pop()
            if (r, c) == (n - 1, n - 1):
                return True
            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                nr, nc = r + dr, c + dc
                if 0 <= nr < n and 0 <= nc < n and (nr, nc) not in seen and grid[nr][nc] <= t:
                    seen.add((nr, nc))
                    stack.append((nr, nc))
        return False

    low, high = 0, n * n - 1
    while low < high:
        middle = (low + high) // 2
        if can(middle):
            high = middle
        else:
            low = middle + 1
    return low
```

1368. Minimum Cost to Make at Least One Valid Path in a Grid

Hard · Breadth-First Search, Graph, Array, Matrix, Shortest Path, Dijkstra, Heap (Priority Queue), 0-1 BFS

leetcode.com/problems/minimum-cost-to-make-at-least-one-valid-path-in-a-grid

★ **TOP 7 PROBLEM** in Shortest Path: master this one first

PROBLEM

Given an `m x n` grid. Each cell of the grid has a sign pointing to the next cell you should visit if you are currently in this cell. The sign of `grid[i][j]` can be:

- `1` which means go to the cell to the right. (i.e go from `grid[i][j]` to `grid[i][j + 1]`)
- `2` which means go to the cell to the left. (i.e go from `grid[i][j]` to `grid[i][j - 1]`)
- `3` which means go to the lower cell. (i.e go from `grid[i][j]` to `grid[i + 1][j]`)
- `4` which means go to the upper cell. (i.e go from `grid[i][j]` to `grid[i - 1][j]`)

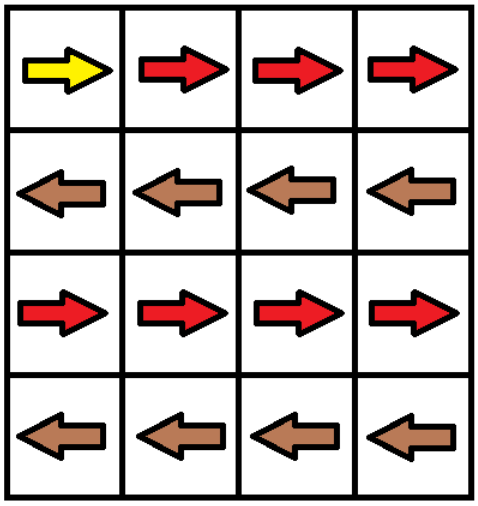
Notice that there could be some signs on the cells of the grid that point outside the grid.

You will initially start at the upper left cell `(0, 0)`. A valid path in the grid is a path that starts from the upper left cell `(0, 0)` and ends at the bottom-right cell `(m - 1, n - 1)` following the signs on the grid. The valid path does not have to be the shortest.

You can modify the sign on a cell with `cost = 1`. You can modify the sign on a cell **one time only**.

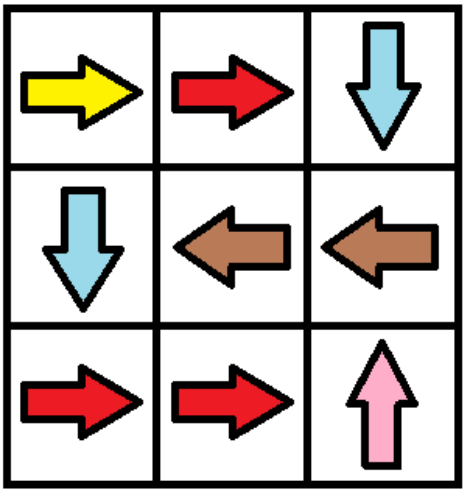
Return *the minimum cost to make the grid have at least one valid path*.

Example 1:



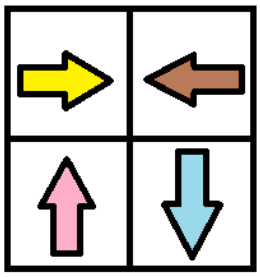
```
Input: grid = [[1,1,1,1],[2,2,2,2],[1,1,1,1],[2,2,2,2]]
Output: 3
Explanation: You will start at point (0, 0).
The path to (3, 3) is as follows. (0, 0) --> (0, 1) --> (0, 2) --> (0, 3) change the arrow to down with cost = 1 --> (1, 3) --> (1, 2) --> (1, 1) --> (1, 0) change the arrow to down with cost = 1 --> (2, 0) --> (2, 1) --> (2, 2) --> (2, 3) change the arrow to down with cost = 1 --> (3, 3)
The total cost = 3.
```

Example 2:



```
Input: grid = [[1,1,3],[3,2,2],[1,1,4]]
Output: 0
Explanation: You can follow the path from (0, 0) to (2, 2).
```

Example 3:



```
Input: grid = [[1,2],[4,3]]
Output: 1
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 100`
- `1 <= grid[i][j] <= 4`

SOLUTION (PYTHON 3)

Solution 1: Double-ended Queue BFS

How to think about it

Each cell points to a neighbor; changing an arrow costs 1. We want the fewest changes to the bottom-right. Dijkstra is O(mnlog). Weights are only 0 (follow the arrow) and 1 (change it), so 0-1 BFS applies: an aligned neighbor goes to the front at the same cost, a changed one to the back at cost plus one. The first visit to the target is optimal.

This problem is essentially a shortest path model, but what we are looking for is the minimum number of direction changes.

In an undirected graph where the edge weights are only 0 and 1, we can use a double-ended queue for BFS. The principle is that when the weight of the point that can be expanded currently is 0, it is added to the front of the queue; when the weight is 1, it is added to the end of the queue.

If the weight of an edge is 0, then the weight of the newly expanded node is the same as the weight of the current queue head node. Obviously, it can be used as the starting point for the next expansion.

Python 3

```python
class Solution:
    def minCost(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        dirs = [[0, 0], [0, 1], [0, -1], [1, 0], [-1, 0]]
        q = deque([(0, 0, 0)])
        vis = set()
        while q:
            i, j, d = q.popleft()
            if (i, j) in vis:
                continue
            vis.add((i, j))
            if i == m - 1 and j == n - 1:
                return d
            for k in range(1, 5):
                x, y = i + dirs[k][0], j + dirs[k][1]
                if 0 <= x < m and 0 <= y < n:
                    if grid[i][j] == k:
                        q.appendleft((x, y, d))
                    else:
                        q.append((x, y, d + 1))
        return -1
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the cells whose arrow is changed
# 2. The ball may also move diagonally if the arrow is a
# diagonal arrow (5 to 8)


# ============================================================
# FOLLOW-UP 1: Return the cells whose arrow is changed
# ============================================================

# ---- follow-up 1: function ----
# Answer: 0-1 search remembering the previous cell; a step
# against the arrow costs one change.
# Time: O(m * n).  Space: O(m * n).
from collections import deque


def changed_arrows(grid):
    rows, cols = len(grid), len(grid[0])
    moves = {1: (0, 1), 2: (0, -1), 3: (1, 0), 4: (-1, 0)}
    cost = {(0, 0): 0}
    previous = {(0, 0): None}
    queue = deque([(0, 0)])
    while queue:
        r, c = queue.popleft()
        for sign, (dr, dc) in moves.items():
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols:
                new = cost[(r, c)] + (0 if grid[r][c] == sign else 1)
                if new < cost.get((nr, nc), float("inf")):
                    cost[(nr, nc)] = new
                    previous[(nr, nc)] = (r, c)
```

```
                    if grid[r][c] == sign:
                        queue.appendleft((nr, nc))
                    else:
                        queue.append((nr, nc))
    cell = (rows - 1, cols - 1)
    changed = []
    while previous[cell] is not None:
        p = previous[cell]
        step = (cell[0] - p[0], cell[1] - p[1])
        if moves[grid[p[0]][p[1]]] != step:
            changed.append(p)
        cell = p
    return sorted(changed)


# ============================================================
# FOLLOW-UP 2: The ball may also move diagonally if the
# arrow is a diagonal arrow (5 to 8)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Add four diagonal directions to the same 0-1
# search.
# Time: O(m * n).  Space: O(m * n).
def min_changes_with_diagonals(grid):
    rows, cols = len(grid), len(grid[0])
    moves = {1: (0, 1), 2: (0, -1), 3: (1, 0), 4: (-1, 0), 5: (1, 1), 6: (1, -1), 7: (-1, 1), 8: (-1, -1)}
    cost = {(0, 0): 0}
```

```
    queue = deque([(0, 0)])
    while queue:
        r, c = queue.popleft()
        for sign, (dr, dc) in moves.items():
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols:
                new = cost[(r, c)] + (0 if grid[r][c] == sign else 1)
                if new < cost.get((nr, nc), float("inf")):
                    cost[(nr, nc)] = new
                    if grid[r][c] == sign:
                        queue.appendleft((nr, nc))
                    else:
                        queue.append((nr, nc))
    return cost[(rows - 1, cols - 1)]
```

2203. Minimum Weighted Subgraph With the Required Paths

Hard · Graph, Shortest Path, Heap (Priority Queue)

leetcode.com/problems/minimum-weighted-subgraph-with-the-required-paths

★ **TOP 7 PROBLEM** in Shortest Path: master this one first

PROBLEM

You are given an integer `n` denoting the number of nodes of a **weighted directed** graph. The nodes are numbered from `0` to `n - 1`.

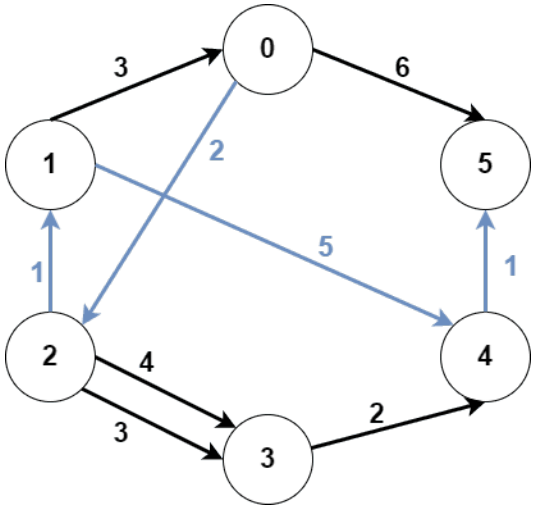
You are also given a 2D integer array `edges` where `edges[i] = [from_i, to_i, weight_i]` denotes that there exists a **directed** edge from `from_i` to `to_i` with weight `weight_i`.

Lastly, you are given three **distinct** integers `src1`, `src2`, and `dest` denoting three distinct nodes of the graph.

Return *the **minimum weight** of a subgraph of the graph such that it is **possible** to reach* `dest` *from both* `src1` *and* `src2` *via a set of edges of this subgraph*. In case such a subgraph does not exist, return `-1`.

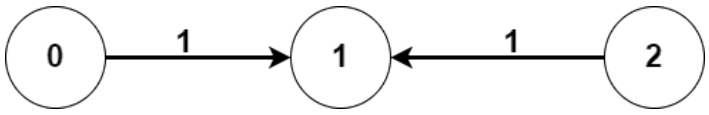
A **subgraph** is a graph whose vertices and edges are subsets of the original graph. The **weight** of a subgraph is the sum of weights of its constituent edges.

Example 1:




```
Input: n = 6, edges = [[0,2,2],[0,5,6],[1,0,3],[1,4,5],[2,1,1],[2,3,3],[2,3,4],[3,4,2],[4,5,1]], src1
= 0, src2 = 1, dest = 5
Output: 9
Explanation:
The above figure represents the input graph.
The blue edges represent one of the subgraphs that yield the optimal answer.
Note that the subgraph [[1,0,3],[0,5,6]] also yields the optimal answer. It is not possible to get a
subgraph with less weight satisfying all the constraints.
```

Example 2:



Input: n = 3, edges = [[0,1,1],[2,1,1]], src1 = 0, src2 = 1, dest = 2
Output: -1
Explanation:
The above figure represents the input graph.
It can be seen that there does not exist any path from node 1 to node 2, hence there are no subgraphs satisfying all the constraints.

Constraints:

- 3 <= n <= 10^5
- 0 <= edges.length <= 10^5
- edges[i].length == 3
- 0 <= $from_i$, to_i, src1, src2, dest <= n - 1
- $from_i$!= to_i
- src1, src2, and dest are pairwise distinct.
- 1 <= weight[i] <= 10^5

SOLUTION (PYTHON 3)

Solution 1

How to think about it

We need a minimum-weight subgraph that contains both src_1 → dest and src_2 → dest. Taking two shortest paths independently double-counts the shared suffix and is not always optimal. n, m ≤ 10^5, so enumerating subgraphs is impossible. Both paths end at dest, so they meet at some vertex p (possibly dest itself). The optimum is the sum of three shortest paths: src_1 → p, src_2 → p, and p → dest. Run Dijkstra from src_1 and src_2 on the original graph, and from dest on the reversed graph. Enumerate p and take min(d_1[p]+d_2[p]+d_3[p]), or -1 if the value stays infinite.

Python 3

```python
class Solution:
    def minimumWeight(
        self, n: int, edges: List[List[int]], src1: int, src2: int, dest: int
    ) -> int:
        def dijkstra(g, u):
            dist = [inf] * n
            dist[u] = 0
            q = [(0, u)]
            while q:
                d, u = heappop(q)
                if d > dist[u]:
                    continue
                for v, w in g[u]:
                    if dist[v] > dist[u] + w:
                        dist[v] = dist[u] + w
                        heappush(q, (dist[v], v))
            return dist

        g = defaultdict(list)
        rg = defaultdict(list)
        for f, t, w in edges:
            g[f].append((t, w))
            rg[t].append((f, w))
        d1 = dijkstra(g, src1)
        d2 = dijkstra(g, src2)
        d3 = dijkstra(rg, dest)
        ans = min(sum(v) for v in zip(d1, d2, d3))
        return -1 if ans >= inf else ans
```

Complexity. Time: O(E log E) (three Dijkstra runs). Space: O(n + E).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Which node is the meeting point
# 2. Three sources must all reach the destination


# ============================================================
# FOLLOW-UP 1: Which node is the meeting point
# ============================================================

# ---- follow-up 1: function ----
# Answer: The node that minimises dist(src1, x) +
# dist(src2, x) + dist(x, dest).
# Time: O((n + E) log n).  Space: O(n + E).
import heapq


def meeting_node(n, edges, src1, src2, dest):
    forward = [[] for _ in range(n)]
    backward = [[] for _ in range(n)]
    for a, b, w in edges:
        forward[a].append((b, w))
        backward[b].append((a, w))

    def dijkstra(graph, start):
        dist = [float("inf")] * n
        dist[start] = 0
        heap = [(0, start)]
        while heap:
            d, u = heapq.heappop(heap)
            if d > dist[u]:
                continue
            for v, w in graph[u]:
                if d + w < dist[v]:
                    dist[v] = d + w
                    heapq.heappush(heap, (d + w, v))
        return dist

    d1, d2, dd = dijkstra(forward, src1), dijkstra(forward, src2), dijkstra(backward, dest)
    best = min(range(n), key=lambda x: (d1[x] + d2[x] + dd[x], x))
    total = d1[best] + d2[best] + dd[best]
    return (best, total) if total != float("inf") else None


# ============================================================
# FOLLOW-UP 2: Three sources must all reach the
# destination
# ============================================================

# ---- follow-up 2: function ----
# Answer: One more Dijkstra from the third source; the
# meeting point sums the four distances.
# Time: O(S * (n + E) log n), S = sources.  Space: O(n +
# E).
def min_weight_three_sources(n, edges, sources, dest):
    forward = [[] for _ in range(n)]
    backward = [[] for _ in range(n)]
    for a, b, w in edges:
        forward[a].append((b, w))
        backward[b].append((a, w))
```

```
    def dijkstra(graph, start):
        dist = [float("inf")] * n
        dist[start] = 0
        heap = [(0, start)]
        while heap:
```

```
            d, u = heapq.heappop(heap)
            if d > dist[u]:
                continue
            for v, w in graph[u]:
                if d + w < dist[v]:
                    dist[v] = d + w
                    heapq.heappush(heap, (d + w, v))
        return dist

    from_sources = [dijkstra(forward, s) for s in sources]
    to_dest = dijkstra(backward, dest)
    best = min(sum(d[x] for d in from_sources) + to_dest[x] for x in range(n))
    return -1 if best == float("inf") else best
```

Eulerian Circuit

3 problems · 0 Easy · 0 Medium · 3 Hard

THE IDEA

Walk every edge exactly once (Hierholzer's algorithm): follow edges with a stack and write nodes in post-order, then reverse.

THE 3 PROBLEMS, EASY TO HARD | ★ = TOP 3: DO THESE FIRST				
1	★ **Reconstruct Itinerary**	Hard	LC 332	2593
2	★ **Cracking the Safe**	Hard	LC 753	2596
3	★ **Valid Arrangement of Pairs**	Hard	LC 2097	2598

PART 2 · EULERIAN CIRCUIT · PROBLEM 1 OF 3

332. Reconstruct Itinerary

Hard · Depth-First Search, Graph, Array, String, Sorting, Eulerian Path, Eulerian Circuit, Heap (Priority Queue), Semi-Eulerian Graph

leetcode.com/problems/reconstruct-itinerary

★ **TOP 7 PROBLEM** in Eulerian Circuit: master this one first

PROBLEM

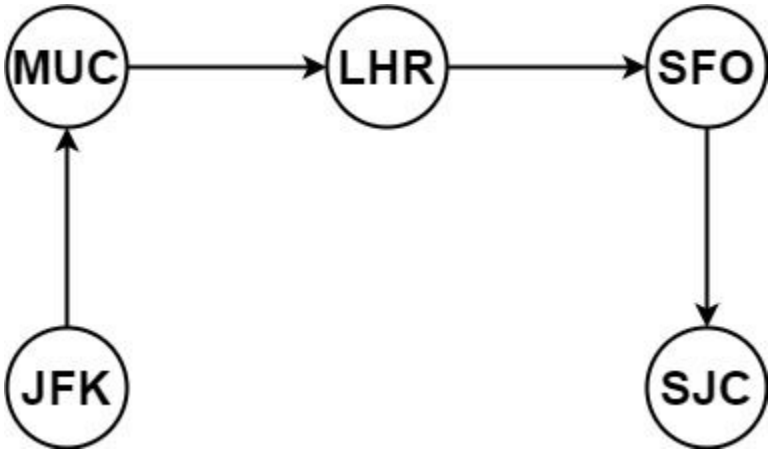
You are given a list of airline `tickets` where `tickets[i] = [from`$_i$`, to`$_i$`]` represent the departure and the arrival airports of one flight. Reconstruct the itinerary in order and return it.

All of the tickets belong to a man who departs from `"JFK"`, thus, the itinerary must begin with `"JFK"`. If there are multiple valid itineraries, you should return the itinerary that has the smallest lexical order when read as a single string.

- For example, the itinerary `["JFK", "LGA"]` has a smaller lexical order than `["JFK", "LGB"]`.

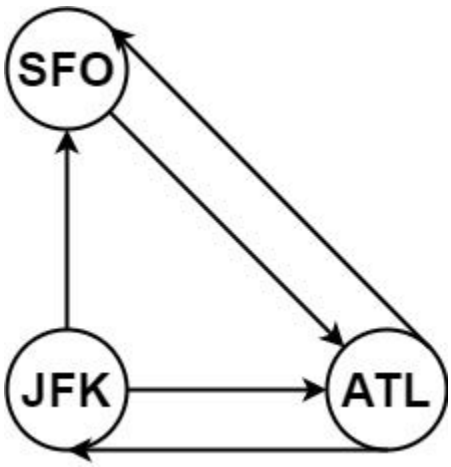
You may assume all tickets form at least one valid itinerary. You must use all the tickets once and only once.

Example 1:




```
Input: tickets = [["MUC","LHR"],["JFK","MUC"],["SFO","SJC"],["LHR","SFO"]]
Output: ["JFK","MUC","LHR","SFO","SJC"]
```

Example 2:




```
Input: tickets = [["JFK","SFO"],["JFK","ATL"],["SFO","ATL"],["ATL","JFK"],["ATL","SFO"]]
Output: ["JFK","ATL","JFK","SFO","ATL","SFO"]
Explanation: Another possible reconstruction is ["JFK","SFO","ATL","JFK","ATL","SFO"] but it is larger in lexical order.
```

Constraints:

- `1 <= tickets.length <= 300`
- `tickets[i].length == 2`
- `from`$_i$`.length == 3`
- `to`$_i$`.length == 3`
- `from`$_i$ and `to`$_i$ consist of uppercase English letters.
- `from`$_i$ `!= to`$_i$

SOLUTION (PYTHON 3)

Solution 1: Eulerian Path

> **How to think about it**
>
> Use every ticket once and produce the lexicographically smallest itinerary: an Eulerian path starting at `JFK`. Store destinations in reverse order so the last pop is the smallest next hop. Append nodes in postorder and reverse; dead ends are recorded first. A tour is guaranteed.

The problem is essentially about finding a path that starts from a specified starting point, passes through all the edges exactly once, and has the smallest lexicographical order among all such paths, given n vertices and m edges. This is a classic Eulerian path problem.

Since the problem guarantees that there is at least one feasible itinerary, we can directly use the Hierholzer algorithm to output the Eulerian path starting from the starting point.

The time complexity is O(m × log m), and the space complexity is O(m). Here, m is the number of edges.

Python 3

```python
class Solution:
    def findItinerary(self, tickets: List[List[str]]) -> List[str]:
        def dfs(f: str):
            while g[f]:
                dfs(g[f].pop())
            ans.append(f)

        g = defaultdict(list)
        for f, t in sorted(tickets, reverse=True):
            g[f].append(t)
        ans = []
        dfs("JFK")
        return ans[::-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Any valid itinerary (not necessarily the smallest)
# 2. Check that a route uses every ticket exactly once


# ============================================================
# FOLLOW-UP 1: Any valid itinerary (not necessarily the
# smallest)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Hierholzer's algorithm with the neighbours in
# any order.
# Time: O(E).  Space: O(E).
def any_itinerary(tickets):
    graph = {}
    for a, b in tickets:
```

```
        graph.setdefault(a, []).append(b)
    stack = ["JFK"]
    route = []
    while stack:
        while graph.get(stack[-1]):
            stack.append(graph[stack[-1]].pop())
        route.append(stack.pop())
    return route[::-1]


# ============================================================
# FOLLOW-UP 2: Check that a route uses every ticket
# exactly once
# ============================================================

# ---- follow-up 2: function ----
# Answer: Compare the multiset of consecutive pairs with
# the tickets.
# Time: O(E log E).  Space: O(E).
def is_valid_itinerary(tickets, route):
    pairs = sorted(zip(route, route[1:]))
    return route[0] == "JFK" and pairs == sorted(map(tuple, tickets))
```

PART 2 · EULERIAN CIRCUIT · PROBLEM 2 OF 3

753. Cracking the Safe

Hard · Depth-First Search, Graph, String, Eulerian Path, Eulerian Circuit, Eulerian Graph

leetcode.com/problems/cracking-the-safe

★ **TOP 7 PROBLEM** in Eulerian Circuit: master this one first

PROBLEM

There is a safe protected by a password. The password is a sequence of `n` digits where each digit can be in the range `[0, k - 1]`.

The safe has a peculiar way of checking the password. When you enter in a sequence, it checks the **most recent** `n` **digits** that were entered each time you type a digit.

- For example, the correct password is `"345"` and you enter in `"012345"`:
  - After typing `0`, the most recent `3` digits is `"0"`, which is incorrect.
  - After typing `1`, the most recent `3` digits is `"01"`, which is incorrect.
  - After typing `2`, the most recent `3` digits is `"012"`, which is incorrect.
  - After typing `3`, the most recent `3` digits is `"123"`, which is incorrect.
  - After typing `4`, the most recent `3` digits is `"234"`, which is incorrect.
  - After typing `5`, the most recent `3` digits is `"345"`, which is correct and the safe unlocks.

Return *any string of* ***minimum length*** *that will unlock the safe* ***at some point*** *of entering it*.

Example 1:

```
Input: n = 1, k = 2
Output: "10"
Explanation: The password is a single digit, so enter each digit. "01" would also unlock the safe.
```

Example 2:

```
Input: n = 2, k = 2
Output: "01100"
Explanation: For each possible password:
- "00" is typed in starting from the 4th digit.
- "01" is typed in starting from the 1st digit.
- "10" is typed in starting from the 3rd digit.
- "11" is typed in starting from the 2nd digit.
Thus "01100" will unlock the safe. "10011", and "11001" would also unlock the safe.
```

Constraints:

- `1 <= n <= 4`
- `1 <= k <= 10`
- `1 <= k^n <= 4096`

SOLUTION (PYTHON 3)

Solution 1: Eulerian Circuit

How to think about it

A shortest string that contains every length-n password over k digits is a de Bruijn sequence, length k^n+n-1. Nodes are (n-1)-mers; an extra digit is an edge, i.e. one password. Every degree is k, so an Eulerian circuit exists and spells the answer. Hierholzer from 0: mark edge u· 10+x, recurse, then append the digit (post-order), and finally add the n-1 zeros of the start.

We can construct a directed graph based on the description in the problem: each point is considered as a length n-1 k-string, and each edge carries a character from 0 to k-1. If there is a directed edge e from point u to point v, and the character carried by e is c, then the last k-1 characters of u+c form the string v. At this point, the edge u+c represents a password of length n.

In this directed graph, there are k^{n-1} points, each point has k outgoing edges and k incoming edges. Therefore, this directed graph has an Eulerian circuit, and the path traversed by the Eulerian circuit is the answer to the problem.

The time complexity is $O(k^n)$, and the space complexity is $O(k^n)$.

Python 3

```
class Solution:
    def crackSafe(self, n: int, k: int) -> str:
        def dfs(u):
            for x in range(k):
                e = u * 10 + x
                if e not in vis:
                    vis.add(e)
                    v = e % mod
                    dfs(v)
                    ans.append(str(x))

        mod = 10 ** (n - 1)
        vis = set()
        ans = []
        dfs(0)
        ans.append("0" * (n - 1))
        return "".join(ans)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Check that a string tries every n-digit code
# 2. The shortest possible length


# ============================================================
# FOLLOW-UP 1: Check that a string tries every n-digit
# code
# ============================================================

# ---- follow-up 1: function ----
# Answer: Collect all windows of length n and compare with
# k ** n.
# Time: O(k^n * n), the windows are sliced.  Space: O(k^n
# * n).
def is_valid_safe_string(n, k, s):
    windows = {s[i:i + n] for i in range(len(s) - n + 1)}
    return len(windows) == k ** n and all(ch in "0123456789"[:k] for ch in s)


# ============================================================
# FOLLOW-UP 2: The shortest possible length
# ============================================================

# ---- follow-up 2: function ----
# Answer: A de Bruijn sequence: every code appears once,
# and the first n - 1 digits are the start of the first
# code.
# Time: O(n).  Space: O(1).
def shortest_safe_length(n, k):
    return k ** n + n - 1
```

PART 2 · EULERIAN CIRCUIT · PROBLEM 3 OF 3

2097. Valid Arrangement of Pairs

Hard · Depth-First Search, Graph, Array, Eulerian Path, Eulerian Circuit, Semi-Eulerian Graph

leetcode.com/problems/valid-arrangement-of-pairs

★ **TOP 7 PROBLEM** in Eulerian Circuit: master this one first

PROBLEM

You are given a **0-indexed** 2D integer array `pairs` where `pairs[i] = [start`$_i$`, end`$_i$`]`. An arrangement of `pairs` is **valid** if for every index `i` where `1 <= i < pairs.length`, we have `end`$_{i-1}$ `== start`$_i$.

Return ***any** valid arrangement of* `pairs`.

Note: The inputs will be generated such that there exists a valid arrangement of `pairs`.

Example 1:

```
Input: pairs = [[5,1],[4,5],[11,9],[9,4]]
Output: [[11,9],[9,4],[4,5],[5,1]]
Explanation:
This is a valid arrangement since endi-1 always equals starti.
end0 = 9 == 9 = start1
end1 = 4 == 4 = start2
end2 = 5 == 5 = start3
```

Example 2:

```
Input: pairs = [[1,3],[3,2],[2,1]]
Output: [[1,3],[3,2],[2,1]]
Explanation:
This is a valid arrangement since endi-1 always equals starti.
end0 = 3 == 3 = start1
end1 = 2 == 2 = start2
The arrangements [[2,1],[1,3],[3,2]] and [[3,2],[2,1],[1,3]] are also valid.
```

Example 3:

```
Input: pairs = [[1,2],[1,3],[2,1]]
Output: [[1,2],[2,1],[1,3]]
Explanation:
This is a valid arrangement since endi-1 always equals starti.
end0 = 2 == 2 = start1
end1 = 1 == 1 = start2
```

Constraints:

- `1 <= pairs.length <= 10`^{5}
- `pairs[i].length == 2`
- `0 <= start`$_i$`, end`$_i$ `<= 10`^{9}
- `start`$_i$ `!= end`$_i$
- No two pairs are exactly the same.
- There **exists** a valid arrangement of `pairs`.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Pairs must chain, i.e. an Eulerian path in the directed graph of edges start → end. Hierholzer runs in linear time for m ≤ 10^5. Start at a vertex with out-degree one higher than in-degree, else anywhere. Push edges in postorder and reverse to obtain the arrangement. The code tabs are empty; the reasoning is the Eulerian-path construction.

Python 3 (written for this guide: the doocs page has an empty Python tab)

```python
class Solution:
    def validArrangement(self, pairs: List[List[int]]) -> List[List[int]]:
        # every pair is a directed edge a -> b: the answer is an Eulerian path (Hierholzer's algorithm)
        graph = defaultdict(list)
        out_minus_in = defaultdict(int)
        for a, b in pairs:
            graph[a].append(b)
            out_minus_in[a] += 1
            out_minus_in[b] -= 1
        start = pairs[0][0]
        for node, d in out_minus_in.items():
            if d == 1:                      # the path must begin where one more edge leaves than enters
                start = node
                break
        stack = [start]
        path = []
        while stack:
            node = stack[-1]
            if graph[node]:
                stack.append(graph[node].pop())
            else:
                path.append(stack.pop())
        path.reverse()
        return [[path[i], path[i + 1]] for i in range(len(path) - 1)]
```

Complexity. Time: O(E), E = number of pairs. Space: O(E).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Check that a list of pairs is a valid arrangement
# 2. Which numbers can the arrangement start with


# ============================================================
# FOLLOW-UP 1: Check that a list of pairs is a valid
# arrangement
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same pairs (as a multiset), and every pair ends
# where the next one starts.
# Time: O(P log P).  Space: O(P).
def is_valid_arrangement(pairs, arrangement):
    if sorted(map(tuple, pairs)) != sorted(map(tuple, arrangement)):
        return False
    return all(arrangement[i][1] == arrangement[i + 1][0] for i in range(len(arrangement) - 1))


# ============================================================
# FOLLOW-UP 2: Which numbers can the arrangement start
# with
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: If one node has one more outgoing edge than
# incoming, it is the only start; if all are balanced,
# every node with an edge can start.
# Time: O(P).  Space: O(V).
def possible_starts(pairs):
    balance = {}
    for a, b in pairs:
        balance[a] = balance.get(a, 0) + 1
        balance[b] = balance.get(b, 0) - 1
    for node, d in balance.items():
        if d == 1:
            return [node]
    return sorted(balance)
```

PART 2 · TOPIC 38 OF 50

1-D DP

6 problems · 2 Easy · 3 Medium · 1 Hard

THE IDEA

dp[i] depends on a few earlier values (dp[i-1], dp[i-2]); often it can be reduced to a couple of variables.

THE 6 PROBLEMS, EASY TO HARD | ★ = TOP 4: DO THESE FIRST

1	Fibonacci Number	Easy	LC 509	2602
2	Min Cost Climbing Stairs	Easy	LC 746	2605
3	★ **Integer Break**	Medium	LC 343	2609
4	★ **Maximum Length of Pair Chain**	Medium	LC 646	2612
5	★ **Greatest Sum Divisible by Three**	Medium	LC 1262	2614
6	★ **Number of Ways to Paint N × 3 Grid**	Hard	LC 1411	2617

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Climbing Stairs (Part 1, pattern 13, page 674)

House Robber (Part 1, pattern 13, page 677)

House Robber II (Part 1, pattern 13, page 680)

Delete and Earn (Part 1, pattern 13, page 719)

509. Fibonacci Number

Easy · Recursion, Memoization, Math, Dynamic Programming

leetcode.com/problems/fibonacci-number

PROBLEM

The **Fibonacci numbers**, commonly denoted `F(n)` form a sequence, called the **Fibonacci sequence**, such that each number is the sum of the two preceding ones, starting from `0` and `1`. That is,

```
F(0) = 0, F(1) = 1
F(n) = F(n - 1) + F(n - 2), for n > 1.
```

Given `n`, calculate `F(n)`.

Example 1:

```
Input: n = 2
Output: 1
Explanation: F(2) = F(1) + F(0) = 1 + 0 = 1.
```

Example 2:

```
Input: n = 3
Output: 2
Explanation: F(3) = F(2) + F(1) = 1 + 1 = 2.
```

Example 3:

```
Input: n = 4
Output: 3
Explanation: F(4) = F(3) + F(2) = 2 + 1 = 3.
```

Constraints:

- `0 <= n <= 30`

SOLUTIONS (PYTHON 3)

Solution 1: Recurrence

How to think about it

Fibonacci obeys $F_n = F_{n-1} + F_{n-2}$. Naive recursion recomputes the same subproblems. Even though n ≤ 30, an iterative recurrence is enough. Roll two variables for consecutive terms and iterate n times. Time is linear and extra space is constant.

We define two variables a and b, initially a = 0 and b = 1.

Next, we perform n iterations. In each iteration, we update the values of a and b to b and a + b, respectively.

Finally, we return a.

The time complexity is O(n), where n is the given integer. The space complexity is O(1).

Python 3

```python
class Solution:
    def fib(self, n: int) -> int:
        a, b = 0, 1
        for _ in range(n):
            a, b = b, a + b
        return a
```

Solution 2: Matrix Exponentiation

> **How to think about it**
>
> Solution 1 is already O(n) and enough here, but the same linear recurrence can be sped up for larger n. F_n is a product of a fixed 2 × 2 transition matrix. Fast exponentiation reduces n multiplications to O(log n). The constant is larger; use it when the same recurrence must scale.

We define Fib(n) as a 1 × 2 matrix bmatrix F_n & F_{n-1} bmatrix, where F_n and F_{n-1} are the n-th and (n - 1)-th Fibonacci numbers, respectively.

We want to derive Fib(n) from Fib(n - 1) = bmatrix F_{n-1} & F_{n-2} bmatrix. In other words, we need a matrix base such that Fib(n - 1) × base = Fib(n), i.e.:

$$\text{bmatrix } F_{n-1} \text{ \& } F_{n-2} \text{ bmatrix} \times \text{base} = \text{bmatrix } F_n \text{ \& } F_{n-1} \text{ bmatrix}$$

Since $F_n = F_{n-1} + F_{n-2}$, the first column of the matrix base is:

$$\text{bmatrix } 1 \text{ \textbackslash } 1 \text{ bmatrix}$$

The second column is:

$$\text{bmatrix } 1 \text{ \textbackslash } 0 \text{ bmatrix}$$

Thus, we have:

$$\text{bmatrix } F_{n-1} \text{ \& } F_{n-2} \text{ bmatrix} \times \text{bmatrix} 1 \text{ \& } 1 \text{ \textbackslash } 1 \text{ \& } 0 \text{bmatrix} = \text{bmatrix } F_n \text{ \& } F_{n-1} \text{ bmatrix}$$

We define the initial matrix res = bmatrix 1 & 0 bmatrix, then F_n is equal to the second element of the first row of the result matrix obtained by multiplying res with base^n. We can solve this using matrix exponentiation.

The time complexity is O(log n), and the space complexity is O(1).

Python 3

```python
import numpy as np


class Solution:
    def fib(self, n: int) -> int:
        factor = np.asmatrix([(1, 1), (1, 0)], np.dtype("O"))
        res = np.asmatrix([(1, 0)], np.dtype("O"))
        while n:
            if n & 1:
                res = res * factor
            factor = factor * factor
            n >>= 1
        return res[0, 1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. O(log n) with fast doubling
# 2. The n-th Fibonacci number modulo m for a huge n


# ============================================================
# FOLLOW-UP 1: O(log n) with fast doubling
# ============================================================

# ---- follow-up 1: function ----
# Answer: F(2k) = F(k) * (2 * F(k + 1) - F(k)) and F(2k +
# 1) = F(k)^2 + F(k + 1)^2.
# Time: O(log n).  Space: O(log n) recursion.
def fib_fast(n):
    def go(k):
        if k == 0:
            return 0, 1
```

```
        a, b = go(k // 2)
        c = a * (2 * b - a)
        d = a * a + b * b
        if k % 2 == 0:
            return c, d
        return d, c + d

    return go(n)[0]


# ============================================================
# FOLLOW-UP 2: The n-th Fibonacci number modulo m for a
# huge n
# ============================================================

# ---- follow-up 2: function ----
# Answer: The remainders repeat (Pisano period): find the
# period and reduce n.
# Time: O(m + n mod period), the Pisano period is at most
# 6m.  Space: O(1).
def fib_mod(n, m):
    if m == 1:
        return 0
    a, b = 0, 1
    length = 0
    while True:                                  # length of the Pisano period
        a, b = b, (a + b) % m
        length += 1
        if a == 0 and b == 1:
            break
    n %= length
    a, b = 0, 1
    for _ in range(n):
        a, b = b, (a + b) % m
    return a
```

746. Min Cost Climbing Stairs

Easy · Array, Dynamic Programming

leetcode.com/problems/min-cost-climbing-stairs

PROBLEM

You are given an integer array `cost` where `cost[i]` is the cost of i^{th} step on a staircase.

Once you pay the cost, you can either climb **one** or **two** steps.

You can either start from the step with index 0, or the step with index 1.

Return the **minimum** cost to reach the top of the staircase, which is the position just past the last step (index `cost.length`).

Example 1:

```
Input: cost = [10,15,20]
Output: 15
Explanation: You will start at index 1.
- Pay 15 and climb two steps to reach the top.
The total cost is 15.
```

Example 2:

```
Input: cost = [1,100,1,1,1,100,1,1,100,1]
Output: 6
Explanation: You will start at index 0.
- Pay 1 and climb two steps to reach index 2.
- Pay 1 and climb two steps to reach index 4.
- Pay 1 and climb two steps to reach index 6.
- Pay 1 and climb one step to reach index 7.
- Pay 1 and climb two steps to reach index 9.
- Pay 1 and climb one step to reach the top.
The total cost is 6.
```

Constraints:

- `2 <= cost.length <= 1000`
- `0 <= cost[i] <= 999`

SOLUTIONS (PYTHON 3)

Solution 1: Memoization Search

How to think about it

Start at index 0 or 1, pay the current stair, and climb one or two steps. n≤ 1000; plain recursion repeats stairs. The cost from i depends only on i+1 and i+2; past the top it is 0. Memoization computes each index once. The answer is min(dfs(0), dfs(1)). Time and space O(n).

We design a function dfs(i), which represents the minimum cost required to climb the stairs starting from the i-th step. Therefore, the answer is min(dfs(0), dfs(1)).

The execution process of the function dfs(i) is as follows:

- If i ≥ len(cost), it means the current position has exceeded the top of the stairs, and there is no need to climb further, so return 0;
- Otherwise, we can choose to climb 1 step with a cost of cost[i], then recursively call dfs(i + 1); or we can choose to climb 2 steps with a cost of cost[i], then recursively call dfs(i + 2);
- Return the minimum cost between these two options.

To avoid repeated calculations, we use memoization search, saving the results that have already been calculated in an array or hash table.

The time complexity is O(n), and the space complexity is O(n), where n is the length of the array cost.

Python 3

```python
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        @cache
        def dfs(i: int) -> int:
            if i >= len(cost):
                return 0
            return cost[i] + min(dfs(i + 1), dfs(i + 2))

        return min(dfs(0), dfs(1))
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> Solution 1 already fills the cost of leaving each stair from the top downward. Here f[i] is the cost of reaching index i, which depends only on i-1 and i-2, so the table is filled upward and f[n] is the top.

We define f[i] as the minimum cost needed to reach the i-th stair. Initially, f[0] = f[1] = 0, and the answer is f[n].

When i ≥ 2, we can reach the i-th stair directly from the (i - 1)-th stair with one step, or from the (i - 2)-th stair with two steps. Therefore, we have the state transition equation:

$$f[i] = \min(f[i - 1] + cost[i - 1], f[i - 2] + cost[i - 2])$$

The final answer is f[n].

The time complexity is O(n), and the space complexity is O(n), where n is the length of the array cost.

Python 3

```python
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        n = len(cost)
        f = [0] * (n + 1)
        for i in range(2, n + 1):
            f[i] = min(f[i - 2] + cost[i - 2], f[i - 1] + cost[i - 1])
        return f[n]
```

Solution 3: Dynamic Programming (Space Optimization)

> **How to think about it**
>
> f[i] reads only the previous two values, so two scalars suffice. Space O(1).

We notice that the state transition equation for f[i] only depends on f[i - 1] and f[i - 2]. Therefore, we can use two variables f and g to alternately record the values of f[i - 2] and f[i - 1], thus optimizing the space complexity to O(1).

Python 3

```python
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        f = g = 0
        for i in range(2, len(cost) + 1):
            f, g = g, min(f + cost[i - 2], g + cost[i - 1])
        return g
```

Solution 4: Dynamic Programming

How to think about it

Expanding every one-step and two-step choice is exponential. There can be 1000 stairs, and the first call always moves to the next stair, so the chain has depth n. The cost of leaving stair i depends only on i+1 and i+2, and past the top the cost is 0. Set f[i] = cost[i] + min(f[i+1], f[i+2]) and fill i from n-1 down to 0. The answer is min(f[0], f[1]).

We let f[i] be the minimum cost of climbing from stair i to the top. When i ≥ n, f[i] = 0. For i < n,

$$f[i] = cost[i] + \min(f[i + 1], f[i + 2])$$

Compute this for i from n - 1 down to 0. The answer is min(f[0], f[1]).

The time complexity is O(n), and the space complexity is O(n), where n is the length of cost.

Python 3

```python
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        n = len(cost)
        f = [0] * (n + 2)
        for i in range(n - 1, -1, -1):
            f[i] = cost[i] + min(f[i + 1], f[i + 2])
        return min(f[0], f[1])
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the steps you stand on
# 2. You may climb one, two or three steps at a time


# ============================================================
# FOLLOW-UP 1: Return the steps you stand on
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same table; walk back from the end through the
# cheaper predecessor.
# Time: O(n).  Space: O(n).
def cheapest_steps(cost):
    n = len(cost)
    best = [0] * (n + 1)
    came = [0] * (n + 1)
    for i in range(2, n + 1):
        one = best[i - 1] + cost[i - 1]
        two = best[i - 2] + cost[i - 2]
        if one <= two:
            best[i], came[i] = one, i - 1
        else:
            best[i], came[i] = two, i - 2
    steps = []
    i = n
    while i >= 2:
        steps.append(came[i])
        i = came[i]
    return best[n], steps[::-1]


# ============================================================
# FOLLOW-UP 2: You may climb one, two or three steps at a
# time
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same table with three predecessors.
```

```python
# Time: O(n).  Space: O(n).
def min_cost_three_steps(cost):
    n = len(cost)
    best = [float("inf")] * (n + 1)
    best[0] = best[1] = 0
    for i in range(2, n + 1):
        for k in (1, 2, 3):
            if i - k >= 0:
                best[i] = min(best[i], best[i - k] + cost[i - k])
    return best[n]
```

PART 2 · 1-D DP · PROBLEM 3 OF 6

343. Integer Break

Medium · Math, Dynamic Programming

leetcode.com/problems/integer-break

★ **TOP 7 PROBLEM** in 1-D DP: master this one first

PROBLEM

Given an integer `n`, break it into the sum of `k` **positive integers**, where `k >= 2`, and maximize the product of those integers.

Return *the maximum product you can get*.

Example 1:

```
Input: n = 2
Output: 1
Explanation: 2 = 1 + 1, 1 × 1 = 1.
```

Example 2:

```
Input: n = 10
Output: 36
Explanation: 10 = 3 + 3 + 4, 3 × 3 × 4 = 36.
```

Constraints:

- `2 <= n <= 58`

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

Split n into at least two positives maximizing the product. Listing partitions is large. If the last part is j, the rest is either kept whole or already optimal. f[i]=max$_j$ max(f[i-j]· j, (i-j)· j) with f[1]=1. Fill by increasing i; the answer is f[n]. n≤ 58 allows O(n^2).

We define f[i] as the maximum product that can be obtained by splitting the positive integer i, with an initial condition of f[1] = 1. The answer is f[n].

Consider the last number j split from i, where j ∈ [1, i). For the number j split from i, there are two cases:

1. Split i into the sum of i - j and j, without further splitting, where the product is (i - j) × j;
2. Split i into the sum of i - j and j, and continue splitting, where the product is f[i - j] × j.

Therefore, we can derive the state transition equation:

$$f[i] = \max(f[i], f[i - j] \times j, (i - j) \times j) \ (j \in [0, i))$$

Finally, returning f[n] will suffice.

The time complexity is O(n^2), and the space complexity is O(n). Here, n is the given positive integer.

Python 3

```python
class Solution:
    def integerBreak(self, n: int) -> int:
        f = [1] * (n + 1)
        for i in range(2, n + 1):
            for j in range(1, i):
                f[i] = max(f[i], f[i - j] * j, (i - j) * j)
        return f[n]
```

Solution 2: Mathematics

> **How to think about it**
>
> Method 1 does not name the optimal parts. For n≥ 4 use as many 3s as possible; a remainder 1 becomes 2+2, a remainder 2 keeps an extra 2. For n<4 the product is n-1. Closed form in O(1).

When n < 4, since the problem requires splitting into at least two integers, n - 1 yields the maximum product. When n ≥ 4, we split into as many 3s as possible. If the last segment remaining is 4, we split it into 2 + 2 for the maximum product.

The time complexity is O(1), and the space complexity is O(1).

Python 3

```
class Solution:
    def integerBreak(self, n: int) -> int:
        if n < 4:
            return n - 1
        if n % 3 == 0:
            return pow(3, n // 3)
        if n % 3 == 1:
            return pow(3, n // 3 - 1) * 4
        return pow(3, n // 3) * 2
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the parts
# 2. At least k parts are required


# ============================================================
# FOLLOW-UP 1: Return the parts
# ============================================================

# ---- follow-up 1: function ----
# Answer: Break into 3s (and one 2 or two 2s) when n is at
# least 5.
# Time: O(n).  Space: O(n).
def break_parts(n):
    if n == 2:
        return [1, 1]
    if n == 3:
        return [1, 2]
    parts = []
    while n > 4:
        parts.append(3)
        n -= 3
    parts.append(n)
    return parts


# ============================================================
# FOLLOW-UP 2: At least k parts are required
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same idea: spread n as evenly as possible
# into parts (the table version handles any k).
# Time: O(n^3).  Space: O(n^2).
def break_into_at_least(n, k):
    if k > n:
        return None
    best = [[0] * (n + 1) for _ in range(n + 1)]      # best[j][m]: biggest product of exactly j positive
parts adding to m
    best[0][0] = 1
```

```
    for j in range(1, n + 1):
        for m in range(j, n + 1):
            for part in range(1, m - j + 2):
                if best[j - 1][m - part]:
                    best[j][m] = max(best[j][m], best[j - 1][m - part] * part)
    return max(best[j][n] for j in range(k, n + 1))
```

PART 2 · 1-D DP · PROBLEM 4 OF 6

646. Maximum Length of Pair Chain

Medium · Greedy, Array, Dynamic Programming, Sorting, Longest Increasing Subsequence

leetcode.com/problems/maximum-length-of-pair-chain

★ **TOP 7 PROBLEM** in 1-D DP: master this one first

PROBLEM

You are given an array of `n` pairs `pairs` where `pairs[i] = [left_i, right_i]` and `left_i < right_i`.

A pair `p2 = [c, d]` **follows** a pair `p1 = [a, b]` if `b < c`. A **chain** of pairs can be formed in this fashion.

Return *the length longest chain which can be formed*.

You do not need to use up all the given intervals. You can select pairs in any order.

Example 1:

```
Input: pairs = [[1,2],[2,3],[3,4]]
Output: 2
Explanation: The longest chain is [1,2] -> [3,4].
```

Example 2:

```
Input: pairs = [[1,2],[7,8],[4,5]]
Output: 3
Explanation: The longest chain is [1,2] -> [4,5] -> [7,8].
```

Constraints:

- `n == pairs.length`
- `1 <= n <= 1000`
- `-1000 <= left_i < right_i <= 1000`

SOLUTION (PYTHON 3)

Solution 1: Sorting + Greedy

How to think about it

A chain is a strictly increasing sequence of pairs. LIS-style DP is $O(n^2)$. Sort by the right end and take a pair whenever it fits. Smaller right ends leave more room, so one greedy pass is optimal.

We sort all pairs in ascending order by the second number, and use a variable pre to maintain the maximum value of the second number of the selected pairs.

We traverse the sorted pairs. If the first number of the current pair is greater than pre, we can greedily select the current pair, increment the answer by one, and update pre to the second number of the current pair.

After the traversal, we return the answer.

The time complexity is O(n × log n), and the space complexity is O(log n). Here, n is the number of pairs.

Python 3

```python
class Solution:
    def findLongestChain(self, pairs: List[List[int]]) -> int:
        pairs.sort(key=lambda x: x[1])
        ans, pre = 0, -inf
        for a, b in pairs:
            if pre < a:
                ans += 1
                pre = b
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the chain
# 2. The smallest number of pairs to remove so that the
# rest can all be chained


# ============================================================
# FOLLOW-UP 1: Return the chain
# ============================================================

# ---- follow-up 1: function ----
# Answer: Greedy by the smallest end: take a pair when its
# start is after the last end.
# Time: O(n log n).  Space: O(n).
def pair_chain(pairs):
    chain = []
    for a, b in sorted(pairs, key=lambda p: p[1]):
        if not chain or a > chain[-1][1]:
            chain.append([a, b])
    return chain


# ============================================================
# FOLLOW-UP 2: The smallest number of pairs to remove so
# that the rest can all be chained
# ============================================================

# ---- follow-up 2: function ----
# Answer: The number of pairs minus the longest chain.
# Time: O(n log n).  Space: O(n).
def pairs_to_remove(pairs):
    return len(pairs) - len(pair_chain(pairs))
```

1262. Greatest Sum Divisible by Three

Medium · Greedy, Array, Dynamic Programming, Sorting

leetcode.com/problems/greatest-sum-divisible-by-three

★ **TOP 7 PROBLEM** in 1-D DP: master this one first

PROBLEM

Given an integer array `nums`, return *the **maximum possible sum** of elements of the array such that it is divisible by three*.

Example 1:

```
Input: nums = [3,6,5,1,8]
Output: 18
Explanation: Pick numbers 3, 6, 1 and 8 their sum is 18 (maximum sum divisible by 3).
```

Example 2:

```
Input: nums = [4]
Output: 0
Explanation: Since 4 is not divisible by 3, do not pick any number.
```

Example 3:

```
Input: nums = [1,2,3,4,4]
Output: 12
Explanation: Pick numbers 1, 3, 4 and 4 their sum is 12 (maximum sum divisible by 3).
```

Constraints:

- `1 <= nums.length <= 4 * 10^4`
- `1 <= nums[i] <= 10^4`

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

We want the maximum subsequence sum divisible by 3. n ≤ 4× 10^4 forbids subsets. A sum has only three remainders modulo 3, and taking or skipping the current value moves among those three. f[i][j] is the best sum of the first i numbers with remainder j. Skip inherits; take adds x from remainder j-x. The answer is f[n][0]. Remainders collapse the constraint to a constant number of states.

We define f[i][j] as the maximum sum of several numbers selected from the first i numbers, such that the sum modulo 3 equals j. Initially, f[0][0]=0, and the rest are -∞.

For f[i][j], we can consider the state of the ith number x:

- If we do not select x, then f[i][j]=f[i-1][j];
- If we select x, then f[i][j]=f[i-1][(j-x mod 3 + 3) mod 3]+x.

Therefore, we can get the state transition equation:

$$f[i][j]=\max\{f[i-1][j],f[i-1][(j-x \bmod 3 + 3) \bmod 3]+x\}$$

The final answer is f[n][0].

The time complexity is O(n), and the space complexity is O(n). Where n is the length of the array nums.

Python 3

```python
class Solution:
    def maxSumDivThree(self, nums: List[int]) -> int:
        n = len(nums)
        f = [[-inf] * 3 for _ in range(n + 1)]
        f[0][0] = 0
        for i, x in enumerate(nums, 1):
            for j in range(3):
                f[i][j] = max(f[i - 1][j], f[i - 1][(j - x) % 3] + x)
        return f[n][0]
```

Solution 2: Dynamic Programming (Rolling Array)

> **How to think about it**
>
> Solution 1 stores O(n) rows. Row i reads only three previous values, so a length-3 rolling array suffices. Extra space is constant; the transition is unchanged.

f[i][j] depends only on the previous row's three residues, so an array of length 3 is enough. The space complexity is O(1).

Python 3

```python
class Solution:
    def maxSumDivThree(self, nums: List[int]) -> int:
        f = [0, -inf, -inf]
        for x in nums:
            g = f[:]
            for j in range(3):
                g[j] = max(f[j], f[(j - x) % 3] + x)
            f = g
        return f[0]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Divisible by any k
# 2. Return the numbers that are chosen


# ============================================================
# FOLLOW-UP 1: Divisible by any k
# ============================================================

# ---- follow-up 1: function ----
# Answer: dp[r] is the biggest sum with remainder r; add
# each number to every state.
# Time: O(n * k).  Space: O(k).
def max_sum_divisible_by(nums, k):
    best = [0] + [float("-inf")] * (k - 1)
    for x in nums:
        new = best[:]
        for r in range(k):
            if best[r] != float("-inf"):
                new[(r + x) % k] = max(new[(r + x) % k], best[r] + x)
        best = new
    return best[0]


# ============================================================
# FOLLOW-UP 2: Return the numbers that are chosen
# ============================================================

# ---- follow-up 2: function ----
# Answer: Remove the smallest numbers that fix the
# remainder (greedy for k = 3).
```

```
# Time: O(n log n).  Space: O(n).
def chosen_for_three(nums):
    total = sum(nums)
    r = total % 3
    if r == 0:
        return sorted(nums)
    ones = sorted(x for x in nums if x % 3 == 1)
    twos = sorted(x for x in nums if x % 3 == 2)
    options = []
    mine, other = (ones, twos) if r == 1 else (twos, ones)
    if mine:
        options.append([mine[0]])
    if len(other) >= 2:
        options.append(other[:2])
    if not options:
        return sorted(nums)
    best = min(options, key=sum)
    rest = list(nums)
    for x in best:
        rest.remove(x)
    return sorted(rest)
```

PART 2 · 1-D DP · PROBLEM 6 OF 6

1411. Number of Ways to Paint N × 3 Grid

Hard · Dynamic Programming, Graph Coloring

leetcode.com/problems/number-of-ways-to-paint-n-3-grid

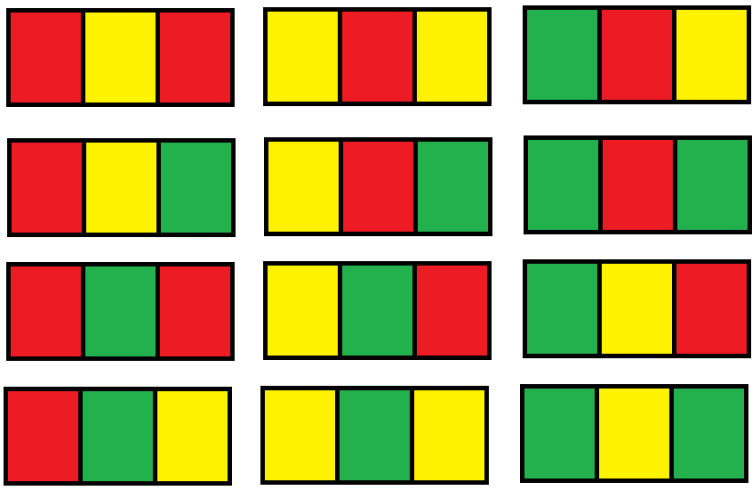
★ **TOP 7 PROBLEM** in 1-D DP: master this one first

PROBLEM

You have a `grid` of size `n x 3` and you want to paint each cell of the grid with exactly one of the three colors: **Red**, **Yellow,** or **Green** while making sure that no two adjacent cells have the same color (i.e., no two cells that share vertical or horizontal sides have the same color).

Given `n` the number of rows of the grid, return *the number of ways* you can paint this `grid`. As the answer may grow large, the answer **must be** computed modulo $10^9 + 7$.

Example 1:



```
Input: n = 1
Output: 12
Explanation: There are 12 possible way to paint the grid as shown.
```

Example 2:

```
Input: n = 5000
Output: 30228214
```

Constraints:

- `n == grid.length`
- `1 <= n <= 5000`

SOLUTIONS (PYTHON 3)

Solution 1: Recursion

How to think about it

Three colors with adjacent cells different and n≤ 5000 make coloring the whole grid infeasible. A row of three cells falls into two symmetry classes: type 010 (two colors) and type 012 (three colors). Counting successors yields $g_0=3f_0+2f_1$ and $g_1=2f_0+2f_1$. Both classes start at 6; after n-1 transitions their sum is the answer.

We classify all possible states for each row. According to the principle of symmetry, when a row only has 3 elements, all legal states are classified as: 010 type, 012 type.

- When the state is 010 type: The possible states for the next row are: 101, 102, 121, 201, 202. These 5 states can be summarized as 3 010 types and 2 012 types.
- When the state is 012 type: The possible states for the next row are: 101, 120, 121, 201. These 4 states can be summarized as 2 010 types and 2 012 types.

In summary, we can get: newf0 = 3 × f0 + 2 × f1, newf1 = 2 × f0 + 2 × f1.

The time complexity is O(n), where n is the number of rows in the grid. The space complexity is O(1).

Python 3

```python
class Solution:
    def numOfWays(self, n: int) -> int:
        mod = 10**9 + 7
        f0 = f1 = 6
        for _ in range(n - 1):
            g0 = (3 * f0 + 2 * f1) % mod
            g1 = (2 * f0 + 2 * f1) % mod
            f0, f1 = g0, g1
        return (f0 + f1) % mod
```

Solution 2: State Compression + Dynamic Programming

> **How to think about it**
>
> Method 1 needs hand-derived transition coefficients. With only three columns there are at most 27 colorings per row, so we can list valid masks and compatible pairs. f[i][j] is the number of ways to paint row i with mask j, summed from compatible previous masks. A rolling array keeps one layer, without closed-form coefficients.

We notice that the grid only has 3 columns, so there are at most 3^3=27 different coloring schemes in a row.

Therefore, we define f[i][j] to represent the number of schemes in the first i rows, where the coloring state of the ith row is j. The state f[i][j] is transferred from f[i - 1][k], where k is the coloring state of the i - 1th row, and k and j meet the requirement of different colors being adjacent. That is:

$$f[i][j] = \sum_{k \in valid(j)} f[i - 1][k]$$

where valid(j) represents all legal predecessor states of state j.

The final answer is the sum of f[n][j], where j is any legal state.

We notice that f[i][j] is only related to f[i - 1][k], so we can use a rolling array to optimize the space complexity.

The time complexity is $O((m + n) \times 3^{2m})$, and the space complexity is $O(3^m)$. Here, m and n are the number of rows and columns of the grid, respectively.

Python 3

```python
class Solution:
    def numOfWays(self, n: int) -> int:
        def f1(x: int) -> bool:
            last = -1
            for _ in range(3):
                if x % 3 == last:
                    return False
                last = x % 3
                x //= 3
            return True

        def f2(x: int, y: int) -> bool:
            for _ in range(3):
                if x % 3 == y % 3:
                    return False
                x //= 3
                y //= 3
            return True

        mod = 10**9 + 7
        m = 27
        valid = {i for i in range(m) if f1(i)}
        d = defaultdict(list)
        for i in valid:
            for j in valid:
                if f2(i, j):
                    d[i].append(j)
        f = [int(i in valid) for i in range(m)]
        for _ in range(n - 1):
            g = [0] * m
            for i in valid:
                for j in d[i]:
                    g[j] = (g[j] + f[i]) % mod
            f = g
        return sum(f) % mod
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Any number of colours k (three columns)
# 2. Two columns instead of three


# ============================================================
# FOLLOW-UP 1: Any number of colours k (three columns)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Enumerate the valid rows (no equal neighbours)
# and count the compatible next rows.
# Time: O(n * k^6).  Space: O(k^3).
import itertools


def paint_ways_k_colours(n, k):
    mod = 10 ** 9 + 7
    rows = [r for r in itertools.product(range(k), repeat=3) if r[0] != r[1] and r[1] != r[2]]
    ways = {r: 1 for r in rows}
    for _ in range(n - 1):
        new = {}
        for r in rows:
            new[r] = sum(ways[q] for q in rows if all(q[i] != r[i] for i in range(3))) % mod
        ways = new
    return sum(ways.values()) % mod
```

```
# ============================================================
# FOLLOW-UP 2: Two columns instead of three
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same table with rows of length two.
# Time: O(n * k^4).  Space: O(k^2).
def paint_ways_two_columns(n, k):
    mod = 10 ** 9 + 7
    rows = [r for r in itertools.product(range(k), repeat=2) if r[0] != r[1]]
    ways = {r: 1 for r in rows}
    for _ in range(n - 1):
        new = {}
        for r in rows:
            new[r] = sum(ways[q] for q in rows if q[0] != r[0] and q[1] != r[1]) % mod
        ways = new
    return sum(ways.values()) % mod
```

PART 2 · TOPIC 39 OF 50

Longest Increasing Subsequence (LIS)

2 problems · 0 Easy · 2 Medium · 0 Hard

THE IDEA

dp[i] is the best chain ending at i (O(n^2)); for O(n log n), keep the smallest tail for each length and binary search.

THE 2 PROBLEMS, EASY TO HARD | ★ = TOP 2: DO THESE FIRST

1	★ **Longest Arithmetic Subsequence**	Medium	LC 1027	2622
2	★ **Longest String Chain**	Medium	LC 1048	2625

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Longest Increasing Subsequence (Part 1, pattern 13, page 686)

Number of Longest Increasing Subsequence (Part 1, pattern 13, page 712)

Russian Doll Envelopes (Part 1, pattern 41, page 2052)

1027. Longest Arithmetic Subsequence

Medium · Array, Hash Table, Binary Search, Dynamic Programming

leetcode.com/problems/longest-arithmetic-subsequence

★ **TOP 7 PROBLEM** in Longest Increasing Subsequence (LIS): master this one first

PROBLEM

Given an array `nums` of integers, return *the length of the longest arithmetic subsequence in* `nums`.

Note that:

- A **subsequence** is an array that can be derived from another array by deleting some or no elements without changing the order of the remaining elements.
- A sequence `seq` is arithmetic if `seq[i + 1] - seq[i]` are all the same value (for `0 <= i < seq.length - 1`).

Example 1:

Input: nums = [3, 6, 9, 12]

Output: 4

Explanation: The whole array is already an arithmetic sequence with a common difference of 3, so the longest arithmetic subsequence is `[3, 6, 9, 12]`, of length 4.

Example 2:

Input: nums = [9, 4, 7, 2, 10]

Output: 3

Explanation: The longest arithmetic subsequence is `[4, 7, 10]` (indices `1`, `2`, `4`), with a common difference of 3. There is no arithmetic subsequence of length 4.

Example 3:

Input: nums = [20, 1, 15, 3, 10, 5, 8]

Output: 4

Explanation: The longest arithmetic subsequence is `[20, 15, 10, 5]` (indices `0`, `2`, `4`, `5`), with a common difference of -5.

Constraints:

- `2 <= nums.length <= 1500`
- `0 <= nums[i] <= 500`

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

Trying every difference and scanning for a subsequence is viable: n≤ 1000 and values lie in [0,500], so differences are a few thousand. The subsequence need not be contiguous, so the state should be “ending at index i with difference j”. f[i][j] is that length. Shifting j by 500 maps differences into [0,1000]. A transition enumerates k<i and sets f[i][j]=max(f[i][j],f[k][j]+1). Every pair of indices updates once; the answer is the maximum over the table.

We define f[i][j] as the maximum length of the arithmetic sequence ending with nums[i] and having a common difference of j. Initially, f[i][j]=1, that is, each element itself is an arithmetic sequence of length 1.

Since the common difference may be negative, and the maximum difference is 500, we can uniformly add 500 to the common difference, so the range of the common difference becomes [0, 1000].

Considering f[i], we can enumerate the previous element nums[k] of nums[i], then the common difference j=nums[i]-nums[k]+500, at this time f[i][j]=max(f[i][j], f[k][j]+1), then we update the answer ans=max(ans, f[i][j]).

Finally, return the answer.

If initially f[i][j]=0, then we need to add 1 to the answer when returning the answer.

The time complexity is O(n × (d + n)), and the space complexity is O(n × d). Where n and d are the length of the array nums and the difference between the maximum and minimum values in the array nums, respectively.

Python 3

```
class Solution:
    def longestArithSeqLength(self, nums: List[int]) -> int:
        n = len(nums)
        f = [[1] * 1001 for _ in range(n)]
        ans = 0
        for i in range(1, n):
            for k in range(i):
                j = nums[i] - nums[k] + 500
                f[i][j] = max(f[i][j], f[k][j] + 1)
                ans = max(ans, f[i][j])
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subsequence
# 2. The difference is given: the longest subsequence with
# exactly that difference


# ============================================================
# FOLLOW-UP 1: Return the subsequence
# ============================================================

# ---- follow-up 1: function ----
# Answer: Table over (index, difference) with the previous
# index remembered.
# Time: O(n^2).  Space: O(n^2).
def arithmetic_subsequence(nums):
    n = len(nums)
    length = {}
    previous = {}
    best = (1, 0, None)
    for i in range(n):
        for j in range(i):
            d = nums[i] - nums[j]
            length[(i, d)] = length.get((j, d), 1) + 1
            previous[(i, d)] = j
            if length[(i, d)] > best[0]:
                best = (length[(i, d)], i, d)
    size, i, d = best
    if d is None:
        return nums[:1]
    result = [nums[i]]
    while (i, d) in previous and len(result) < size:
        i = previous[(i, d)]
        result.append(nums[i])
    return result[::-1]


# ============================================================
# FOLLOW-UP 2: The difference is given: the longest
# subsequence with exactly that difference
# ============================================================

# ---- follow-up 2: function ----
```

```
# Answer: One pass with a dictionary of the best length
# ending at each value.
# Time: O(n).  Space: O(n).
def longest_with_difference(nums, difference):
    best = {}
    answer = 0
    for x in nums:
        best[x] = best.get(x - difference, 0) + 1
        answer = max(answer, best[x])
    return answer
```

PART 2 · LONGEST INCREASING SUBSEQUENCE (LIS) · PROBLEM 2 OF 2

1048. Longest String Chain

Medium · Array, Hash Table, Two Pointers, String, Dynamic Programming, Sorting

leetcode.com/problems/longest-string-chain

★ **TOP 7 PROBLEM** in Longest Increasing Subsequence (LIS): master this one first

PROBLEM

You are given an array of `words` where each word consists of lowercase English letters.

$word_A$ is a **predecessor** of $word_B$ if and only if we can insert **exactly one** letter anywhere in $word_A$ **without changing the order of the other characters** to make it equal to $word_B$.

- For example, `"abc"` is a **predecessor** of `"abac"`, while `"cba"` is not a **predecessor** of `"bcad"`.

A **word chain** is a sequence of words $[word_1, word_2, ..., word_k]$ with $k >= 1$, where $word_1$ is a **predecessor** of $word_2$, $word_2$ is a **predecessor** of $word_3$, and so on. A single word is trivially a **word chain** with $k == 1$.

Return *the **length** of the **longest possible word chain** with words chosen from the given list of* `words`.

Example 1:

```
Input: words = ["a","b","ba","bca","bda","bdca"]
Output: 4
Explanation: One of the longest word chains is ["a","ba","bda","bdca"].
```

Example 2:

```
Input: words = ["xbc","pcxbcf","xb","cxbc","pcxbc"]
Output: 5
Explanation: All the words can be put in a word chain ["xb", "xbc", "cxbc", "pcxbc", "pcxbcf"].
```

Example 3:

```
Input: words = ["abcd","dbqca"]
Output: 1
Explanation: The trivial word chain ["abcd"] is one of the longest word chains.
["abcd","dbqca"] is not a valid word chain because the ordering of the letters is changed.
```

Constraints:

- `1 <= words.length <= 1000`
- `1 <= words[i].length <= 16`
- `words[i]` only consists of lowercase English letters.

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

A chain grows by one character. n≤ 1000 and length ≤ 16 allow DP after sorting by length. The best chain ending at i comes from some predecessor j whose length is one smaller. Two pointers test whether a becomes b by inserting one letter. Each i tries earlier j and sets f[i]=max(f[i],f[j]+1) on a hit. The answer is the maximum of f.

First, sort words by string length in ascending order. Define f[i] as the length of the longest word chain ending with words[i]. Initially, f[i] = 1.

For each i, enumerate j ∈ [0, i). If words[j] is a predecessor of words[i], update f[i] = max(f[i], f[j] + 1). Two strings form a predecessor pair if their lengths differ by 1 and the shorter one can be obtained by deleting exactly one character from the longer one.

The answer is max(f).

The time complexity is $O(n^2 \times L)$ and the space complexity is $O(n)$, where n is the length of the array and L is the maximum length of a string.

Python 3

```python
class Solution:
    def longestStrChain(self, words: List[str]) -> int:
        def check(a: str, b: str) -> bool:
            if len(a) + 1 != len(b):
                return False
            i = 0
            for c in b:
                if i < len(a) and a[i] == c:
                    i += 1
            return i == len(a)

        words.sort(key=len)
        n = len(words)
        f = [1] * n
        for i in range(n):
            for j in range(i):
                if check(words[j], words[i]):
                    f[i] = max(f[i], f[j] + 1)
        return max(f)
```

Solution 2: Dynamic Programming + Hash Table

> **How to think about it**
>
> Solution 1 scans every shorter word even when the length gap is not 1. A predecessor is w with one character deleted, so there are at most L candidates and a hash map can store scores by word. Still sorting by length, we delete each index of w and take f[p]+1. The time becomes $O(nL^2)$.

Sort words by length as well. Use a hash table f to record the longest word chain length ending at each word.

For the current word w, enumerate each predecessor p obtained by deleting one character. If p is already in the hash table, update f[w] with f[p] + 1.

The answer is the maximum value among all f[w].

The time complexity is $O(n \times L^2)$ and the space complexity is $O(n \times L)$.

Python 3

```python
class Solution:
    def longestStrChain(self, words: List[str]) -> int:
        words.sort(key=len)
        f = {}
        ans = 0
        for w in words:
            x = 1
            for i in range(len(w)):
                pred = w[:i] + w[i + 1 :]
                x = max(x, f.get(pred, 0) + 1)
            f[w] = x
            ans = max(ans, x)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the chain
# 2. Which words can start a chain (have no predecessor in
# the list)


# ============================================================
# FOLLOW-UP 1: Return the chain
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort by length; remember the predecessor that
# gave the best length.
# Time: O(n log n + n * L^2).  Space: O(n * L).
def longest_chain_words(words):
    best = {}
    previous = {}
    for w in sorted(words, key=len):
        best[w] = 1
        previous[w] = None
        for i in range(len(w)):
            p = w[:i] + w[i + 1:]
            if p in best and best[p] + 1 > best[w]:
                best[w] = best[p] + 1
                previous[w] = p
    end = max(best, key=lambda w: best[w])
    chain = []
    while end is not None:
        chain.append(end)
        end = previous[end]
    return chain[::-1]


# ============================================================
# FOLLOW-UP 2: Which words can start a chain (have no
# predecessor in the list)
# ============================================================

# ---- follow-up 2: function ----
# Answer: A word starts a chain when none of its one-
# letter deletions is a word.
# Time: O(n * L^2).  Space: O(n * L).
def chain_starters(words):
    present = set(words)
    return sorted(w for w in present if not any(w[:i] + w[i + 1:] in present for i in range(len(w))))
```

PART 2 · TOPIC 40 OF 50

2D Grid DP

10 problems · 0 Easy · 2 Medium · 8 Hard

THE IDEA

dp[r][c] (or an interval dp[i][j]) built from neighbouring cells; decide the order that guarantees dependencies are ready.

THE 10 PROBLEMS, EASY TO HARD | ★ = TOP 7: DO THESE FIRST

1	★ **Paint House**	Medium	LC 256	2629
2	★ **Maximum Number of Points with Cost**	Medium	LC 1937	2631
3	★ **Frog Jump**	Hard	LC 403	2635
4	Optimal Account Balancing	Hard	LC 465	2638
5	★ **Remove Boxes**	Hard	LC 546	2641
6	★ **Maximum Profit in Job Scheduling**	Hard	LC 1235	2644
7	★ **Minimum Difficulty of a Job Schedule**	Hard	LC 1335	2648
8	★ **Minimum Cost to Cut a Stick**	Hard	LC 1547	2651
9	Painting a Grid With Three Different Colors	Hard	LC 1931	2655
10	Minimum Cost Path with Teleportations	Hard	LC 3651	2658

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Pascal's Triangle (Part 1, pattern 36, page 1855)

Unique Paths II (Part 1, pattern 22, page 1175)

Triangle (Part 1, pattern 22, page 1188)

Count Square Submatrices with All Ones (Part 1, pattern 5, page 285)

Burst Balloons (Part 1, pattern 14, page 773)

Cherry Pickup (Part 1, pattern 22, page 1215)

Longest Increasing Path in a Matrix (Part 1, pattern 22, page 1225)

Unique Paths (Part 1, pattern 22, page 1172)

Minimum Path Sum (Part 1, pattern 22, page 1179)

Maximal Square (Part 1, pattern 22, page 1185)

Minimum Falling Path Sum (Part 1, pattern 22, page 1195)

Maximal Rectangle (Part 1, pattern 9, page 468)

Dungeon Game (Part 1, pattern 22, page 1212)

PART 2 · 2D GRID DP · PROBLEM 1 OF 10

256. Paint House

Medium · Array, Dynamic Programming

leetcode.com/problems/paint-house

★ **TOP 7 PROBLEM** in 2D Grid DP: master this one first

PROBLEM

There is a row of `n` houses, where each house can be painted one of three colors: red, blue, or green. The cost of painting each house with a certain color is different. You have to paint all the houses such that no two adjacent houses have the same color.

The cost of painting each house with a certain color is represented by an `n x 3` cost matrix `costs`.

- For example, `costs[0][0]` is the cost of painting house `0` with the color red; `costs[1][2]` is the cost of painting house 1 with color green, and so on...

Return *the minimum cost to paint all houses*.

Example 1:

```
Input: costs = [[17,2,17],[16,16,5],[14,3,19]]
Output: 10
Explanation: Paint house 0 into blue, paint house 1 into green, paint house 2 into blue.
Minimum cost: 2 + 5 + 3 = 10.
```

Example 2:

```
Input: costs = [[7,6,2]]
Output: 2
```

Constraints:

- `costs.length == n`
- `costs[i].length == 3`
- `1 <= n <= 100`
- `1 <= costs[i][j] <= 20`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Adjacent houses cannot share a color, so listing paintings is too large. The best cost of color c on house i depends only on the other two colors on house i-1. Three rolling variables store the best totals for the three colors; the answer is their minimum.

Python 3

```python
class Solution:
    def minCost(self, costs: List[List[int]]) -> int:
        a = b = c = 0
        for ca, cb, cc in costs:
            a, b, c = min(b, c) + ca, min(a, c) + cb, min(a, b) + cc
        return min(a, b, c)
```

Complexity. Time: O(n). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the colour of every house
# 2. k colours and neighbouring houses must differ in
# colour, in O(n k)


# ============================================================
# FOLLOW-UP 1: Return the colour of every house
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same table with the choice remembered; walk back
# from the cheapest last colour.
# Time: O(n * k^2).  Space: O(n * k).
def paint_choices(costs):
    n, k = len(costs), len(costs[0])
    best = [row[:] for row in costs]
    choice = [[-1] * k for _ in range(n)]
    for i in range(1, n):
        for c in range(k):
            options = [(best[i - 1][p], p) for p in range(k) if p != c]
            value, p = min(options)
            best[i][c] = costs[i][c] + value
            choice[i][c] = p
    c = min(range(k), key=lambda x: best[n - 1][x])
    total = best[n - 1][c]
    result = []
    for i in range(n - 1, -1, -1):
        result.append(c)
        c = choice[i][c]
    return total, result[::-1]


# ============================================================
# FOLLOW-UP 2: k colours and neighbouring houses must
# differ in colour, in O(n k)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the two smallest values of the previous
# row.
# Time: O(n * k log k).  Space: O(k).
def paint_house_ii(costs):
    n, k = len(costs), len(costs[0])
    previous = costs[0][:]
    for i in range(1, n):
        order = sorted(range(k), key=lambda c: previous[c])
        first, second = order[0], order[1] if k > 1 else order[0]
        current = []
        for c in range(k):
            other = previous[second] if c == first else previous[first]
            current.append(costs[i][c] + other)
        previous = current
    return min(previous)
```

1937. Maximum Number of Points with Cost

Medium · Array, Dynamic Programming, Matrix

leetcode.com/problems/maximum-number-of-points-with-cost

★ **TOP 7 PROBLEM** in 2D Grid DP: master this one first

PROBLEM

You are given an `m x n` integer matrix `points` (**0-indexed**). Starting with `0` points, you want to **maximize** the number of points you can get from the matrix.

To gain points, you must pick one cell in **each row**. Picking the cell at coordinates `(r, c)` will **add** `points[r][c]` to your score.

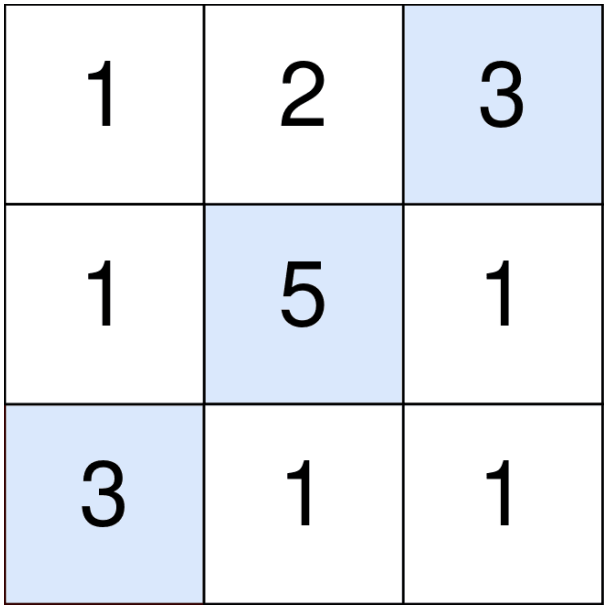
However, you will lose points if you pick a cell too far from the cell that you picked in the previous row. For every two adjacent rows `r` and `r + 1` (where `0 <= r < m - 1`), picking cells at coordinates (r, c_1) and $(r + 1, c_2)$ will **subtract** $\texttt{abs}(c_1 - c_2)$ from your score.

Return *the **maximum** number of points you can achieve*.

`abs(x)` is defined as:

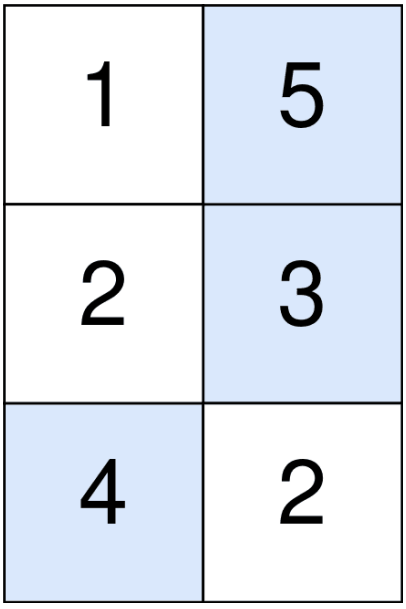
- `x` for `x >= 0`.
- `-x` for `x < 0`.

Example 1:




```
Input: points = [[1,2,3],[1,5,1],[3,1,1]]
Output: 9
Explanation:
The blue cells denote the optimal cells to pick, which have coordinates (0, 2), (1, 1), and (2, 0).
You add 3 + 5 + 3 = 11 to your score.
However, you must subtract abs(2 - 1) + abs(1 - 0) = 2 from your score.
Your final score is 11 - 2 = 9.
```

Example 2:




```
Input: points = [[1,5],[2,3],[4,2]]
Output: 11
Explanation:
The blue cells denote the optimal cells to pick, which have coordinates (0, 1), (1, 1), and (2, 0).
You add 5 + 3 + 4 = 12 to your score.
However, you must subtract abs(1 - 1) + abs(1 - 0) = 1 from your score.
Your final score is 12 - 1 = 11.
```

Constraints:

- `m == points.length`
- `n == points[r].length`
- `1 <= m, n <= 10^5`
- `1 <= m * n <= 10^5`
- `0 <= points[r][c] <= 10^5`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

One cell per row with an absolute-column penalty. Trying every previous column is $O(n^2)$ per row and fails when $mn \le 10^5$. The $k \le j$ part depends on $\max(f[k]+k)$ and the $k \ge j$ part on $\max(f[k]-k)$. A left-to-right prefix max and a right-to-left suffix max evaluate each cell in $O(1)$. Rolling one row of DP uses $O(n)$ extra space and linear time in the number of cells.

Python 3

```python
class Solution:
    def maxPoints(self, points: List[List[int]]) -> int:
        n = len(points[0])
        f = points[0][:]
        for p in points[1:]:
            g = [0] * n
            lmx = -inf
            for j in range(n):
                lmx = max(lmx, f[j] + j)
                g[j] = max(g[j], p[j] + lmx - j)
            rmx = -inf
            for j in range(n - 1, -1, -1):
                rmx = max(rmx, f[j] - j)
                g[j] = max(g[j], p[j] + rmx + j)
            f = g
        return max(f)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the columns that are picked
# 2. The penalty is the SQUARE of the distance


# ============================================================
# FOLLOW-UP 1: Return the columns that are picked
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same row-by-row DP with the best previous column
# remembered.
# Time: O(m * n^2).  Space: O(m * n).
def picked_columns(points):
    m, n = len(points), len(points[0])
    best = points[0][:]
    choice = [[0] * n for _ in range(m)]
    for r in range(1, m):
        new = []
        for c in range(n):
            value, prev = max((best[p] - abs(p - c), p) for p in range(n))
            new.append(points[r][c] + value)
            choice[r][c] = prev
        best = new
    c = max(range(n), key=lambda x: best[x])
    total = best[c]
    columns = []
    for r in range(m - 1, -1, -1):
        columns.append(c)
        c = choice[r][c]
    return total, columns[::-1]


# ============================================================
# FOLLOW-UP 2: The penalty is the SQUARE of the distance
# ============================================================

# ---- follow-up 2: function ----
# Answer: Now the best previous column needs the full scan
# (or a convex hull); the simple scan is O(n squared) per
# row.
# Time: O(m * n^2).  Space: O(n).
def max_points_squared_penalty(points):
    m, n = len(points), len(points[0])
    best = points[0][:]
    for r in range(1, m):
```

```
        best = [points[r][c] + max(best[p] - (p - c) ** 2 for p in range(n)) for c in range(n)]
    return max(best)
```

PART 2 · 2D GRID DP · PROBLEM 3 OF 10

403. Frog Jump

Hard · Array, Dynamic Programming

leetcode.com/problems/frog-jump

★ **TOP 7 PROBLEM** in 2D Grid DP: master this one first

PROBLEM

A frog is crossing a river. The river is divided into some number of units, and at each unit, there may or may not exist a stone. The frog can jump on a stone, but it must not jump into the water.

Given a list of `stones` positions (in units) in sorted **ascending order**, determine if the frog can cross the river by landing on the last stone. Initially, the frog is on the first stone and assumes the first jump must be `1` unit.

If the frog's last jump was `k` units, its next jump must be either `k - 1`, `k`, or `k + 1` units. The frog can only jump in the forward direction.

Example 1:

```
Input: stones = [0,1,3,5,6,8,12,17]
Output: true
Explanation: The frog can jump to the last stone by jumping 1 unit to the 2nd stone, then 2 units to the 3rd stone, then 2 units to the 4th stone, then 3 units to the 6th stone, 4 units to the 7th stone, and 5 units to the 8th stone.
```

Example 2:

```
Input: stones = [0,1,2,3,4,8,9,11]
Output: false
Explanation: There is no way to jump to the last stone as the gap between the 5th and 6th stone is too large.
```

Constraints:

- `2 <= stones.length <= 2000`
- `0 <= stones[i] <= 2^31 - 1`
- `stones[0] == 0`
- `stones` is sorted in a strictly increasing order.

SOLUTIONS (PYTHON 3)

Solution 1: Hash Table + Memoization

How to think about it

Trying jumps of k-1,k,k+1 from every stone revisits the same (stone, last jump). With n≤ 1100 the pair (i,k) has at most $O(n^2)$ states. Map positions to indices, memoize dfs(i,k), and expand only onto stones that exist. The hash table makes the landing test O(1).

We use a hash table pos to record the index of each stone. Next, we design a function dfs(i, k), which means that the frog jumps from the i-th stone and the last jump distance is k. If the frog can reach the end, the function returns `true`, otherwise it returns `false`.

The calculation process of function dfs(i, k) is as follows:

If i is the index of the last stone, the frog has reached the end, and return `true`;

Otherwise, we need to enumerate the frog's next jump distance j, where j ∈ [k-1, k, k+1]. If j is a positive integer and the hash table pos exists the position stones[i] + j, then the frog can choose to jump j units on the i-th stone, if dfs(pos[stones[i] + j], j) returns `true`, the frog can successfully jump to the end from the i-th stone, and we can return `true`.

The enumeration is over, indicating that the frog cannot choose the appropriate jump distance on the i-th stone to jump to the end, so we return `false`.

In order to prevent repeated calculations in the function dfs(i, k), we can use memoization, record the result of dfs(i, k) in an array f, and assign f[i][k] each time the function dfs(i, k) returns result, and return f[i][k] directly when encountering dfs(i, k) next time.

The time complexity is $O(n^2)$, and the space complexity is $O(n^2)$. Where n is the number of stones.

Python 3

```python
class Solution:
    def canCross(self, stones: List[int]) -> bool:
        @cache
        def dfs(i, k):
            if i == n - 1:
                return True
            for j in range(k - 1, k + 2):
                if j > 0 and stones[i] + j in pos and dfs(pos[stones[i] + j], j):
                    return True
            return False

        n = len(stones)
        pos = {s: i for i, s in enumerate(stones)}
        return dfs(0, 0)
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> The recursive search can grow a deep stack. Let f[i][k] mean we can reach stone i with last jump k, transferring from an earlier j with k=stones[i]-stones[j]. If k-1>j, farther stones cannot supply that jump, so the inner loop may stop. Filling the table bottom-up removes the recursion.

We define f[i][k] to be true if and only if it is possible to reach stone i with last jump of size k. Initially f[0][0] = true, and all other elements of f are false.

We can determine the value of f[i][k] for all i and k using a double loop. For each possible jump size k, we look at the stones we could have jumped from: i-k, i-k+1, i-k+2. If any of these stones exist and if we can reach them with a last jump of size k-1, k, or k+1, then we can reach stone i with a last jump of size k.

If we can reach the last stone, the answer is true. Otherwise, the answer is false.

The time complexity is $O(n^2)$, and the space complexity is $O(n^2)$. Where n is the number of stones.

Python 3

```python
class Solution:
    def canCross(self, stones: List[int]) -> bool:
        n = len(stones)
        f = [[False] * n for _ in range(n)]
        f[0][0] = True
        for i in range(1, n):
            for j in range(i - 1, -1, -1):
                k = stones[i] - stones[j]
                if k - 1 > j:
                    break
                f[i][k] = f[j][k - 1] or f[j][k] or f[j][k + 1]
                if i == n - 1 and f[i][k]:
                    return True
        return False
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the jumps (the sizes) that reach the last
# stone
# 2. How many different ways reach the last stone


# ============================================================
# FOLLOW-UP 1: Return the jumps (the sizes) that reach the
# last stone
# ============================================================

# ---- follow-up 1: function ----
# Answer: Search over (stone, last jump) states
# remembering how each was reached.
# Time: O(n^2).  Space: O(n^2).
def frog_route(stones):
    position = {s: i for i, s in enumerate(stones)}
    came = {(0, 0): None}
    stack = [(0, 0)]
    while stack:
        i, k = stack.pop()
        if i == len(stones) - 1:
            route = []
            state = (i, k)
            while came[state] is not None:
                route.append(state[1])
                state = came[state]
            return route[::-1]
        for step in (k - 1, k, k + 1):
            if step > 0 and stones[i] + step in position:
                nxt = (position[stones[i] + step], step)
                if nxt not in came:
                    came[nxt] = (i, k)
                    stack.append(nxt)
    return None


# ============================================================
# FOLLOW-UP 2: How many different ways reach the last
# stone
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count the paths over (stone, last jump) states.
# Time: O(n^2).  Space: O(n^2).
def count_frog_ways(stones):
    position = {s: i for i, s in enumerate(stones)}
    memo = {}

    def go(i, k):
        if i == len(stones) - 1:
            return 1
        if (i, k) in memo:
            return memo[(i, k)]
        total = 0
        for step in (k - 1, k, k + 1):
            if step > 0 and stones[i] + step in position:
                total += go(position[stones[i] + step], step)
        memo[(i, k)] = total
        return total
```

```
    return go(0, 0)
```

465. Optimal Account Balancing

Hard · Bit Manipulation, Array, Dynamic Programming, Backtracking, Bitmask

leetcode.com/problems/optimal-account-balancing

PROBLEM

You are given an array of transactions `transactions` where `transactions[i] = [from_i, to_i, amount_i]` indicates that the person with `ID = from_i` gave `amount_i $` to the person with `ID = to_i`.

Return *the minimum number of transactions required to settle the debt*.

Example 1:

```
Input: transactions = [[0,1,10],[2,0,5]]
Output: 2
Explanation:
Person #0 gave person #1 $10.
Person #2 gave person #0 $5.
Two transactions are needed. One way to settle the debt is person #1 pays person #0 and #2 $5 each.
```

Example 2:

```
Input: transactions = [[0,1,10],[1,0,1],[1,2,5],[2,0,5]]
Output: 1
Explanation:
Person #0 gave person #1 $10.
Person #1 gave person #0 $1.
Person #1 gave person #2 $5.
Person #2 gave person #0 $5.
Therefore, person #1 only need to give person #0 $4, and all debt is settled.
```

Constraints:

- `1 <= transactions.length <= 8`
- `transactions[i].length == 3`
- `0 <= from_i, to_i < 12`
- `from_i != to_i`
- `1 <= amount_i <= 100`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

We want the fewest transfers that zero every balance. People already at 0 drop out. Searching transfer orders among the remaining m≤ 12 people is still large; subset DP is the right grain. A subset whose balances sum to 0 needs at most |S|-1 transfers. f[i] is that minimum for mask i: only a zero-sum mask is feasible, starting from |i|-1 and trying f[j]+f[ixor j] over nonempty proper subsets. Subset enumeration is $O(3^m)$. Collapse transactions into nonzero balances first, and transfer only on zero-sum masks.

Python 3

```python
class Solution:
    def minTransfers(self, transactions: List[List[int]]) -> int:
        g = defaultdict(int)
        for f, t, x in transactions:
            g[f] -= x
            g[t] += x
        nums = [x for x in g.values() if x]
        m = len(nums)
        f = [inf] * (1 << m)
        f[0] = 0
        for i in range(1, 1 << m):
            s = 0
            for j, x in enumerate(nums):
                if i >> j & 1:
                    s += x
            if s == 0:
                f[i] = i.bit_count() - 1
                j = (i - 1) & i
                while j > 0:
                    f[i] = min(f[i], f[j] + f[i ^ j])
                    j = (j - 1) & i
        return f[-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the settling transactions (who pays whom)
# 2. The number of people who end up owing or owed money


# ============================================================
# FOLLOW-UP 1: Return the settling transactions (who pays
# whom)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Search over the balances, remembering the
# payments of the best way.
# Time: O(m!) in the worst case, m = people with a non-
# zero balance.  Space: O(m).
def settle_transactions(transactions):
    balance = {}
    for a, b, amount in transactions:
        balance[a] = balance.get(a, 0) - amount
        balance[b] = balance.get(b, 0) + amount
    debts = [(p, v) for p, v in balance.items() if v != 0]
    best = [None]

    def go(i, current, payments):
        while i < len(current) and current[i][1] == 0:
            i += 1
        if i == len(current):
            if best[0] is None or len(payments) < len(best[0]):
                best[0] = list(payments)
            return
        if best[0] is not None and len(payments) >= len(best[0]):
            return
        for j in range(i + 1, len(current)):
            if current[i][1] * current[j][1] < 0:
                amount = current[i][1]
                new = list(current)
                new[j] = (current[j][0], current[j][1] + amount)
                new[i] = (current[i][0], 0)
                who_pays, who_gets = (current[j][0], current[i][0]) if amount < 0 else (current[i][0],
current[j][0])
```

```
                payments.append((who_pays, who_gets, abs(amount)))
                go(i + 1, new, payments)
                payments.pop()

    go(0, debts, [])
    return best[0] or []


# ============================================================
# FOLLOW-UP 2: The number of people who end up owing or
# owed money
# ============================================================

# ---- follow-up 2: function ----
# Answer: Those whose net balance is not zero.
# Time: O(t).  Space: O(p).
def people_with_open_balance(transactions):
    balance = {}
    for a, b, amount in transactions:
        balance[a] = balance.get(a, 0) - amount
        balance[b] = balance.get(b, 0) + amount
```

```
    return sum(1 for v in balance.values() if v != 0)
```

PART 2 · 2D GRID DP · PROBLEM 5 OF 10

546. Remove Boxes

Hard · Memoization, Array, Dynamic Programming

leetcode.com/problems/remove-boxes

★ **TOP 7 PROBLEM** in 2D Grid DP: master this one first

PROBLEM

You are given several `boxes` with different colors represented by different positive numbers.

You may experience several rounds to remove boxes until there is no box left. Each time you can choose some continuous boxes with the same color (i.e., composed of `k` boxes, `k >= 1`), remove them and get `k * k` points.

Return *the maximum points you can get*.

Example 1:

```
Input: boxes = [1,3,2,2,2,3,4,3,1]
Output: 23
Explanation:
[1, 3, 2, 2, 2, 3, 4, 3, 1]
----> [1, 3, 3, 4, 3, 1] (3*3=9 points)
----> [1, 3, 3, 3, 1] (1*1=1 points)
----> [1, 1] (3*3=9 points)
----> [] (2*2=4 points)
```

Example 2:

```
Input: boxes = [1,1,1]
Output: 9
```

Example 3:

```
Input: boxes = [1]
Output: 1
```

Constraints:

- `1 <= boxes.length <= 100`
- `1 <= boxes[i] <= 100`

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Removing a run scores the square of its length, and order changes later merges. A plain interval DP cannot express "clear the middle, then join the right run". dfs(i,j,k) is the interval [i,j] with k extra boxes already equal to boxes[j]. Absorb the run ending at j into k, then either remove j now or find an earlier h of the same color, clear (h,j), and merge. Memoization caches each triple.

Python 3

```python
class Solution:
    def removeBoxes(self, boxes: List[int]) -> int:
        @cache
        def dfs(i, j, k):
            if i > j:
                return 0
            while i < j and boxes[j] == boxes[j - 1]:
                j, k = j - 1, k + 1
            ans = dfs(i, j - 1, 0) + (k + 1) * (k + 1)
            for h in range(i, j):
                if boxes[h] == boxes[j]:
                    ans = max(ans, dfs(h + 1, j - 1, 0) + dfs(i, h, k + 1))
            return ans

        n = len(boxes)
        ans = dfs(0, n - 1, 0)
        dfs.cache_clear()
        return ans
```

Complexity. Time: O(n^4). Space: O(n^3).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Points are k to the power p for a run of k boxes
# 2. The fewest operations to remove everything (each
# operation removes one run of equal boxes)


# ============================================================
# FOLLOW-UP 1: Points are k to the power p for a run of k
# boxes
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same interval DP with a parameter p instead
# of 2.
# Time: O(n^4).  Space: O(n^3).
def remove_boxes_power(boxes, p):
    memo = {}

    def go(l, r, k):
        if l > r:
            return 0
        if (l, r, k) in memo:
            return memo[(l, r, k)]
        while l < r and boxes[l] == boxes[l + 1]:
            l += 1
            k += 1
        best = (k + 1) ** p + go(l + 1, r, 0)
        for m in range(l + 1, r + 1):
            if boxes[m] == boxes[l]:
                best = max(best, go(l + 1, m - 1, 0) + go(m, r, k + 1))
        memo[(l, r, k)] = best
        return best

    return go(0, len(boxes) - 1, 0)


# ============================================================
# FOLLOW-UP 2: The fewest operations to remove everything
# (each operation removes one run of equal boxes)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Interval DP: remove the first box alone, or
```

```
# remove it together with a later box of the same colour.
# Time: O(n^3).  Space: O(n^2).
def fewest_removals(boxes):
    n = len(boxes)
    if n == 0:
        return 0
    dp = [[0] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):
        for j in range(i, n):
            if i == j:
                dp[i][j] = 1
                continue
            best = 1 + dp[i + 1][j]
            for m in range(i + 1, j + 1):
                if boxes[m] == boxes[i]:
                    inner = dp[i + 1][m - 1] if m - 1 >= i + 1 else 0
                    best = min(best, inner + dp[m][j])
```

```
            dp[i][j] = best
    return dp[0][n - 1]
```

1235. Maximum Profit in Job Scheduling

Hard · Array, Binary Search, Dynamic Programming, Sorting

leetcode.com/problems/maximum-profit-in-job-scheduling

★ **TOP 7 PROBLEM** in 2D Grid DP: master this one first

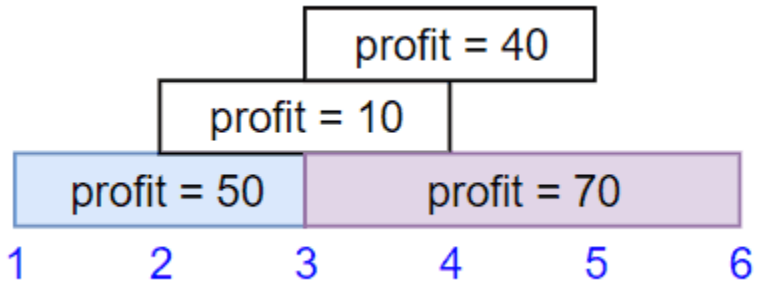
PROBLEM

We have `n` jobs, where every job is scheduled to be done from `startTime[i]` to `endTime[i]`, obtaining a profit of `profit[i]`.

You're given the `startTime`, `endTime` and `profit` arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range.

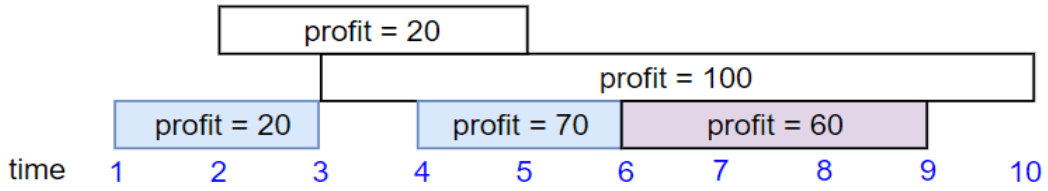
If you choose a job that ends at time `X` you will be able to start another job that starts at time `X`.

Example 1:



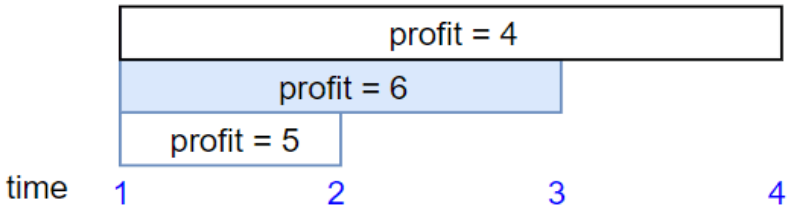

```
Input: startTime = [1,2,3,3], endTime = [3,4,5,6], profit = [50,10,40,70]
Output: 120
Explanation: The subset chosen is the first and fourth job.
Time range [1-3]+[3-6] , we get profit of 120 = 50 + 70.
```

Example 2:




```
Input: startTime = [1,2,3,4,6], endTime = [3,5,10,6,9], profit = [20,20,100,70,60]
Output: 150
Explanation: The subset chosen is the first, fourth and fifth job.
Profit obtained 150 = 20 + 70 + 60.
```

Example 3:




```
Input: startTime = [1,1,1], endTime = [2,3,4], profit = [5,6,4]
Output: 6
```

Constraints:

- `1 <= startTime.length == endTime.length == profit.length <= 5 * 10^4`
- `1 <= startTime[i] < endTime[i] <= 10^9`
- `1 <= profit[i] <= 10^4`

SOLUTIONS (PYTHON 3)

Solution 1: Memoization Search + Binary Search

How to think about it

Jobs must not overlap. n ≤ 5× 10^4 rules out subset search. Whether we take job i only affects jobs that start no earlier than its end. After sorting by start, dfs(i) takes the better of skipping i and taking i then jumping to the first job with start≥ end_i. That index is a binary search on the sorted starts. Memoization evaluates each i once.

First, we sort the jobs by start time in ascending order, then design a function dfs(i) to represent the maximum profit that can be obtained starting from the i-th job. The answer is dfs(0).

The calculation process of function dfs(i) is as follows:

For the i-th job, we can choose to do it or not. If we don't do it, the maximum profit is dfs(i + 1); if we do it, we can use binary search to find the first job that starts after the end time of the i-th job, denoted as j, then the maximum profit is profit[i] + dfs(j). We take the larger of the two. That is:

$$dfs(i)=max(dfs(i+1),profit[i]+dfs(j))$$

Where j is the smallest index that satisfies startTime[j] ≥ endTime[i].

In this process, we can use memoization search to save the answer of each state to avoid repeated calculations.

The time complexity is O(n × log n), where n is the number of jobs.

Python 3

```python
class Solution:
    def jobScheduling(
        self, startTime: List[int], endTime: List[int], profit: List[int]
    ) -> int:
        @cache
        def dfs(i):
            if i >= n:
                return 0
            _, e, p = jobs[i]
            j = bisect_left(jobs, e, lo=i + 1, key=lambda x: x[0])
            return max(dfs(i + 1), p + dfs(j))

        jobs = sorted(zip(startTime, endTime, profit))
        n = len(profit)
        return dfs(0)
```

Solution 2: Dynamic Programming + Binary Search

How to think about it

Solution 1 sorts by start and fills one suffix table. Sorting by end instead, dp[i] is the best profit among the first i jobs: skip inherits dp[i-1]; take adds dp[j] for the last job that ends before this start, with j still from binary search.

Solution 1 sorts by start and fills a suffix table. Here we sort by end and define a prefix table.

First, sort the jobs, this time we sort by end time in ascending order, then define dp[i], which represents the maximum profit that can be obtained from the first i jobs. The answer is dp[n]. Initialize dp[0]=0.

For the i-th job, we can choose to do it or not. If we don't do it, the maximum profit is dp[i]; if we do it, we can use binary search to find the last job that ends before the start time of the i-th job, denoted as j, then the maximum profit is profit[i] + dp[j]. We take the larger of the two. That is:

$$dp[i+1] = \max(dp[i], profit[i] + dp[j])$$

Where j is the largest index that satisfies endTime[j] ≤ startTime[i].

The time complexity is O(n × log n), where n is the number of jobs.

Similar problems:

- 2008. Maximum Earnings From Taxi
- 1751. Maximum Number of Events That Can Be Attended II

Python 3

```python
class Solution:
    def jobScheduling(
        self, startTime: List[int], endTime: List[int], profit: List[int]
    ) -> int:
        jobs = sorted(zip(endTime, startTime, profit))
        n = len(profit)
        dp = [0] * (n + 1)
        for i, (_, s, p) in enumerate(jobs):
            j = bisect_right(jobs, s, hi=i, key=lambda x: x[0])
            dp[i + 1] = max(dp[i], dp[j] + p)
        return dp[n]
```

Solution 3: Dynamic Programming + Binary Search

> **How to think about it**
>
> Jobs must not overlap. n ≤ 5× 10^4 rules out subset search. Whether we take job i only affects jobs that start no earlier than its end. After sorting by start, skipping calls i+1 before it returns, so the chain has length n and overflows the stack. Later indices are known if we walk from the end. Let f[i] be the best profit from job i, with f[n]=0, and binary-search the next compatible job while filling i from n-1 down to 0.

First, we sort the jobs by start time in ascending order. Let f[i] be the maximum profit that can be obtained starting from the i-th job. The answer is f[0], and f[n] = 0.

Fill i from n - 1 down to 0. For the i-th job we can skip it, which keeps f[i + 1], or take it. Binary search finds the first job j that starts at or after end_i. Taking it earns profit[i] + f[j]. Take the larger of the two:

$$f[i]=\max(f[i+1],profit[i]+f[j])$$

Where j is the smallest index that satisfies startTime[j] ≥ endTime[i]. Because j > i, f[j] is already filled.

The time complexity is O(n × log n), where n is the number of jobs.

Python 3

```python
class Solution:
    def jobScheduling(
        self, startTime: List[int], endTime: List[int], profit: List[int]
    ) -> int:
        jobs = sorted(zip(startTime, endTime, profit))
        n = len(profit)
        f = [0] * (n + 1)
        for i in range(n - 1, -1, -1):
            _, e, p = jobs[i]
            j = bisect_left(jobs, e, lo=i + 1, key=lambda x: x[0])
            f[i] = max(f[i + 1], p + f[j])
        return f[0]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the jobs that are chosen
# 2. At most k jobs


# ============================================================
# FOLLOW-UP 1: Return the jobs that are chosen
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same sorted-by-end table; walk back to see which
# jobs were taken.
# Time: O(n log n).  Space: O(n).
import bisect


def chosen_jobs(start, end, profit):
    jobs = sorted(zip(end, start, profit))
    ends = [j[0] for j in jobs]
    best = [0] * (len(jobs) + 1)
    take = [False] * (len(jobs) + 1)
    previous = [0] * (len(jobs) + 1)
    for i, (e, s, p) in enumerate(jobs, 1):
        k = bisect.bisect_right(ends, s, 0, i - 1)
        with_job = best[k] + p
        if with_job > best[i - 1]:
            best[i] = with_job
            take[i] = True
            previous[i] = k
        else:
            best[i] = best[i - 1]
    i = len(jobs)
    result = []
    while i > 0:
        if take[i]:
            result.append(jobs[i - 1])
            i = previous[i]
        else:
            i -= 1
    return best[-1], result[::-1]


# ============================================================
# FOLLOW-UP 2: At most k jobs
# ============================================================

# ---- follow-up 2: function ----
# Answer: dp[j][i]: the best profit using at most j jobs
# among the first i by end time.
# Time: O(n log n + n * k).  Space: O(n * k).
def max_profit_k_jobs(start, end, profit, k):
    jobs = sorted(zip(end, start, profit))
    ends = [j[0] for j in jobs]
    n = len(jobs)
    dp = [[0] * (n + 1) for _ in range(k + 1)]
    for j in range(1, k + 1):
        for i in range(1, n + 1):
            e, s, p = jobs[i - 1]
            idx = bisect.bisect_right(ends, s, 0, i - 1)
            dp[j][i] = max(dp[j][i - 1], dp[j - 1][idx] + p)
```

```
    return dp[k][n]
```

PART 2 · 2D GRID DP · PROBLEM 7 OF 10

1335. Minimum Difficulty of a Job Schedule

Hard · Array, Dynamic Programming

leetcode.com/problems/minimum-difficulty-of-a-job-schedule

★ **TOP 7 PROBLEM** in 2D Grid DP: master this one first

PROBLEM

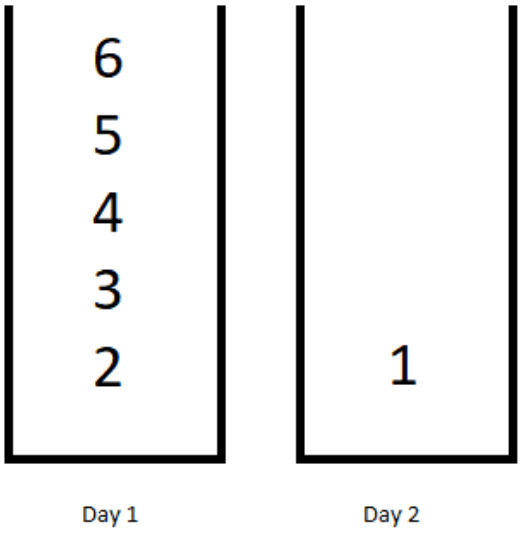
You want to schedule a list of jobs in `d` days. Jobs are dependent (i.e To work on the i^{th} job, you have to finish all the jobs `j` where `0 <= j < i`).

You have to finish **at least** one task every day. The difficulty of a job schedule is the sum of difficulties of each day of the `d` days. The difficulty of a day is the maximum difficulty of a job done on that day.

You are given an integer array `jobDifficulty` and an integer `d`. The difficulty of the i^{th} job is `jobDifficulty[i]`.

Return *the minimum difficulty of a job schedule*. If you cannot find a schedule for the jobs return `-1`.

Example 1:




```
Input: jobDifficulty = [6,5,4,3,2,1], d = 2
Output: 7
Explanation: First day you can finish the first 5 jobs, total difficulty = 6.
Second day you can finish the last job, total difficulty = 1.
The difficulty of the schedule = 6 + 1 = 7
```

Example 2:

```
Input: jobDifficulty = [9,9,9], d = 4
Output: -1
Explanation: If you finish a job per day you will still have a free day. you cannot find a schedule for the given jobs.
```

Example 3:

```
Input: jobDifficulty = [1,1,1], d = 3
Output: 3
Explanation: The schedule is one job per day. total difficulty will be 3.
```

Constraints:

- `1 <= jobDifficulty.length <= 300`
- `0 <= jobDifficulty[i] <= 1000`
- `1 <= d <= 10`

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

> **How to think about it**
>
> Split the job sequence into d contiguous days; a day's difficulty is its max job, and the total should be minimal. Jobs stay in order. n ≤ 300, d ≤ 10 still forbids enumerating all cuts. The best way to finish i jobs in j days depends only on the last day covering [k..i]. f[i][j] stores that optimum. Enumerating k backward maintains the segment max and transitions from f[k-1][j-1]. Fewer jobs than days is impossible.

We define f[i][j] as the minimum difficulty to finish the first i jobs within j days. Initially f[0][0] = 0, and all other f[i][j] are ∞.

For the j-th day, we can choose to finish jobs [k,..i] on this day. Therefore, we have the following state transition equation:

$$f[i][j] = \min_{k \in [1,i]} \{f[k-1][j-1] + \max_{k \le t \le i} \{jobDifficulty[t]\}\}$$

The final answer is f[n][d].

The time complexity is $O(n^2 \times d)$, and the space complexity is $O(n \times d)$. Here n and d are the number of jobs and the number of days respectively.

Python 3

```python
class Solution:
    def minDifficulty(self, jobDifficulty: List[int], d: int) -> int:
        n = len(jobDifficulty)
        f = [[inf] * (d + 1) for _ in range(n + 1)]
        f[0][0] = 0
        for i in range(1, n + 1):
            for j in range(1, min(d + 1, i + 1)):
                mx = 0
                for k in range(i, 0, -1):
                    mx = max(mx, jobDifficulty[k - 1])
                    f[i][j] = min(f[i][j], f[k - 1][j - 1] + mx)
        return -1 if f[n][d] >= inf else f[n][d]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return how many jobs go on each day
# 2. The MAXIMUM total difficulty over all schedules


# ============================================================
# FOLLOW-UP 1: Return how many jobs go on each day
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember the best split point for every state
# and replay it.
# Time: O(d * n^2).  Space: O(d * n).
def jobs_per_day(job_difficulty, d):
    n = len(job_difficulty)
    INF = float("inf")
    dp = [[INF] * (n + 1) for _ in range(d + 1)]
    cut = [[0] * (n + 1) for _ in range(d + 1)]
    dp[0][0] = 0
    for day in range(1, d + 1):
        for i in range(day, n + 1):
            hardest = 0
```

```
            for j in range(i - 1, day - 2, -1):
                hardest = max(hardest, job_difficulty[j])
                if dp[day - 1][j] + hardest < dp[day][i]:
                    dp[day][i] = dp[day - 1][j] + hardest
                    cut[day][i] = j
    if dp[d][n] == INF:
        return None
    sizes = []
    i = n
    for day in range(d, 0, -1):
        sizes.append(i - cut[day][i])
        i = cut[day][i]
    return dp[d][n], sizes[::-1]


# ============================================================
# FOLLOW-UP 2: The MAXIMUM total difficulty over all
# schedules
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same table with max instead of min.
# Time: O(d * n^2).  Space: O(d * n).
def max_total_difficulty(job_difficulty, d):
    n = len(job_difficulty)
    NEG = float("-inf")
    dp = [[NEG] * (n + 1) for _ in range(d + 1)]
    dp[0][0] = 0
    for day in range(1, d + 1):
        for i in range(day, n + 1):
            hardest = 0
            for j in range(i - 1, day - 2, -1):
                hardest = max(hardest, job_difficulty[j])
                if dp[day - 1][j] != NEG:
                    dp[day][i] = max(dp[day][i], dp[day - 1][j] + hardest)
    return dp[d][n] if dp[d][n] != NEG else None
```

1547. Minimum Cost to Cut a Stick

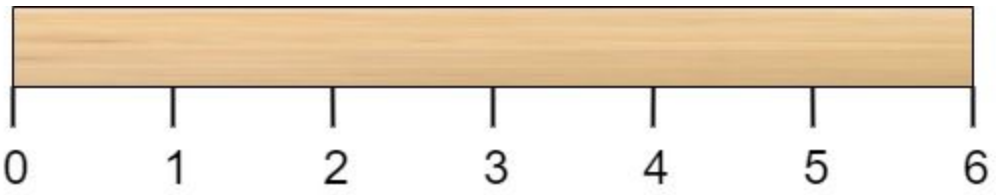
Hard · Array, Dynamic Programming, Sorting

leetcode.com/problems/minimum-cost-to-cut-a-stick

★ **TOP 7 PROBLEM** in 2D Grid DP: master this one first

PROBLEM

Given a wooden stick of length `n` units. The stick is labelled from `0` to `n`. For example, a stick of length **6** is labelled as follows:



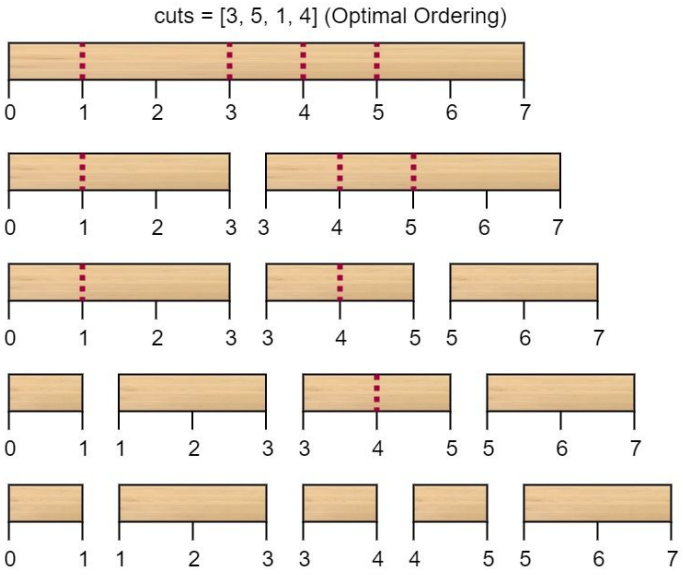
Given an integer array `cuts` where `cuts[i]` denotes a position you should perform a cut at.

You should perform the cuts in order, you can change the order of the cuts as you wish.

The cost of one cut is the length of the stick to be cut, the total cost is the sum of costs of all cuts. When you cut a stick, it will be split into two smaller sticks (i.e. the sum of their lengths is the length of the stick before the cut). Please refer to the first example for a better explanation.

Return *the minimum total cost* of the cuts.

Example 1:




```
Input: n = 7, cuts = [1,3,4,5]
Output: 16
Explanation: Using cuts order = [1, 3, 4, 5] as in the input leads to the following scenario:

The first cut is done to a rod of length 7 so the cost is 7. The second cut is done to a rod of length
6 (i.e. the second part of the first cut), the third is done to a rod of length 4 and the last cut is
to a rod of length 3. The total cost is 7 + 6 + 4 + 3 = 20.
Rearranging the cuts to be [3, 5, 1, 4] for example will lead to a scenario with total cost = 16 (as
shown in the example photo 7 + 4 + 3 + 2 = 16).
```

Example 2:

```
Input: n = 9, cuts = [5,6,1,4,2]
Output: 22
Explanation: If you try the given cuts ordering the cost will be 25.
There are much ordering with total cost <= 25, for example, the order [4, 6, 5, 2, 1] has total cost =
22 which is the minimum possible.
```

Constraints:

- `2 <= n <= 10^6`
- `1 <= cuts.length <= min(n - 1, 100)`
- `1 <= cuts[i] <= n - 1`
- All the integers in `cuts` array are **distinct**.

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming (Interval DP)

How to think about it

Cut a stick of length n at given points; each cut costs the current piece's length. n can be 10^6 but there are at most 100 cuts, so DP belongs on the cut list, not on stick coordinates. Insert 0 and n and sort. f[i][j] is the min cost to finish every cut strictly inside (cuts[i],cuts[j]). The last cut at cuts[k] costs f[i][k]+f[k][j]+cuts[j]-cuts[i]. Filling by increasing interval length makes every subinterval ready.

We can add two elements to the array cuts, namely 0 and n, representing the two ends of the stick. Then we sort the cuts array, so we can divide the entire stick into several intervals, each with two cut points. Let the length of the cuts array be m.

Next, we define f[i][j] to represent the minimum cost to cut the interval [cuts[i], cuts[j]].

If an interval has only two cut points, meaning we do not need to cut this interval, then f[i][j] = 0.

Otherwise, we enumerate the length l of the interval, where l is equal to the number of cut points minus 1. Then we enumerate the left endpoint i of the interval, and the right endpoint j can be obtained by i + l. For each interval, we enumerate its cut point k, where i < k < j. We can then divide the interval [i, j] into [i, k] and [k, j]. The cost at this point is f[i][k] + f[k][j] + cuts[j] - cuts[i]. We take the minimum value among all possible k, which is the value of f[i][j].

Finally, we return f[0][m - 1].

The time complexity is $O(m^3)$, and the space complexity is $O(m^2)$. Here, m is the length of the modified cuts array.

Python 3

```python
class Solution:
    def minCost(self, n: int, cuts: List[int]) -> int:
        cuts.extend([0, n])
        cuts.sort()
        m = len(cuts)
        f = [[0] * m for _ in range(m)]
        for l in range(2, m):
            for i in range(m - l):
                j = i + l
                f[i][j] = inf
                for k in range(i + 1, j):
                    f[i][j] = min(f[i][j], f[i][k] + f[k][j] + cuts[j] - cuts[i])
        return f[0][-1]
```

Solution 2: Dynamic Programming (Another Enumeration Method)

How to think about it

Solution 1 enumerates by length only to compute short intervals first. The same recurrence works if i runs downward and j runs upward, which also evaluates every i<k<j beforehand. The asymptotics are unchanged; only the loop order differs.

We can also enumerate i from large to small and j from small to large. This ensures that when calculating f[i][j], the states f[i][k] and f[k][j] have already been computed, where i < k < j.

The time complexity is $O(m^3)$, and the space complexity is $O(m^2)$. Here, m is the length of the modified cuts array.

Python 3

```
class Solution:
    def minCost(self, n: int, cuts: List[int]) -> int:
        cuts.extend([0, n])
        cuts.sort()
        m = len(cuts)
        f = [[0] * m for _ in range(m)]
        for i in range(m - 1, -1, -1):
            for j in range(i + 2, m):
                f[i][j] = inf
                for k in range(i + 1, j):
                    f[i][j] = min(f[i][j], f[i][k] + f[k][j] + cuts[j] - cuts[i])
        return f[0][-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the order of the cuts
# 2. The MAXIMUM total cost


# ========================================================
# FOLLOW-UP 1: Return the order of the cuts
# ========================================================

# ---- follow-up 1: function ----
# Answer: Interval DP with the best first cut remembered.
# Time: O(m^3), m = cuts + 2.  Space: O(m^2).
def cut_order(n, cuts):
    points = [0] + sorted(cuts) + [n]
    m = len(points)
    cost = [[0] * m for _ in range(m)]
    first = [[0] * m for _ in range(m)]
    for length in range(2, m):
        for i in range(m - length):
            j = i + length
            best = None
            for k in range(i + 1, j):
                value = cost[i][k] + cost[k][j] + points[j] - points[i]
                if best is None or value < best:
                    best = value
                    first[i][j] = k
            cost[i][j] = best

    order = []

    def build(i, j):
        if j - i < 2:
            return
        k = first[i][j]
        order.append(points[k])
        build(i, k)
        build(k, j)

    build(0, m - 1)
    return cost[0][m - 1], order


# ========================================================
# FOLLOW-UP 2: The MAXIMUM total cost
# ========================================================
```

```
# ---- follow-up 2: function ----
# Answer: Same table with max.
# Time: O(m^3).  Space: O(m^2).
def max_cut_cost(n, cuts):
    points = [0] + sorted(cuts) + [n]
    m = len(points)
    cost = [[0] * m for _ in range(m)]
    for length in range(2, m):
        for i in range(m - length):
            j = i + length
            cost[i][j] = max(cost[i][k] + cost[k][j] for k in range(i + 1, j)) + points[j] - points[i]
    return cost[0][m - 1]
```

1931. Painting a Grid With Three Different Colors

Hard · Dynamic Programming, Graph Coloring

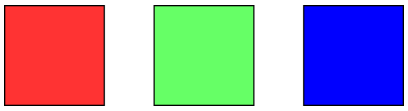
leetcode.com/problems/painting-a-grid-with-three-different-colors

PROBLEM

You are given two integers `m` and `n`. Consider an `m x n` grid where each cell is initially white. You can paint each cell **red**, **green**, or **blue**. All cells **must** be painted.

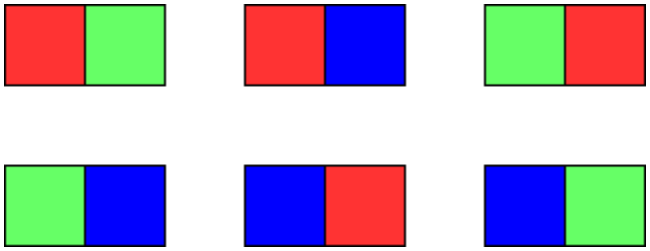
Return *the number of ways to color the grid with* ***no two adjacent cells having the same color***. Since the answer can be very large, return it **modulo** $10^9 + 7$.

Example 1:



```
Input: m = 1, n = 1
Output: 3
Explanation: The three possible colorings are shown in the image above.
```

Example 2:



```
Input: m = 1, n = 2
Output: 6
Explanation: The six possible colorings are shown in the image above.
```

Example 3:

```
Input: m = 5, n = 5
Output: 580986
```

Constraints:

- `1 <= m <= 5`
- `1 <= n <= 1000`

SOLUTION (PYTHON 3)

Solution 1: State Compression + Dynamic Programming

How to think about it

Adjacent cells need different colors. Cell-by-cell coloring is too large, but m≤ 5 so a column has only 3^m colorings. We keep masks whose vertical neighbors differ, then precompute pairs of masks that also differ horizontally. f[j] is the number of ways the previous column equals j, rolled forward. The first column counts valid masks; n-1 transitions follow, and the answer is the sum modulo 10^9+7.

We notice that the number of rows in the grid does not exceed 5, so there are at most 3^5=243 different color schemes in a column.

Therefore, we define f[i][j] to represent the number of schemes in the first i columns, where the coloring state of the ith column is j. The state f[i][j] is transferred from f[i - 1][k], where k is the coloring state of the i - 1th column, and k and j meet the requirement of different colors being adjacent. That is:

$$f[i][j] = \sum_{k \in valid(j)} f[i - 1][k]$$

where valid(j) represents all legal predecessor states of state j.

The final answer is the sum of f[n][j], where j is any legal state.

We notice that f[i][j] is only related to f[i - 1][k], so we can use a rolling array to optimize the space complexity.

The time complexity is $O((m + n) \times 3^{2m})$, and the space complexity is $O(3^m)$. Here, m and n are the number of rows and columns of the grid, respectively.

Python 3

```python
class Solution:
    def colorTheGrid(self, m: int, n: int) -> int:
        def f1(x: int) -> bool:
            last = -1
            for _ in range(m):
                if x % 3 == last:
                    return False
                last = x % 3
                x //= 3
            return True

        def f2(x: int, y: int) -> bool:
            for _ in range(m):
                if x % 3 == y % 3:
                    return False
                x, y = x // 3, y // 3
            return True

        mod = 10**9 + 7
        mx = 3**m
        valid = {i for i in range(mx) if f1(i)}
        d = defaultdict(list)
        for x in valid:
            for y in valid:
                if f2(x, y):
                    d[x].append(y)
        f = [int(i in valid) for i in range(mx)]
        for _ in range(n - 1):
            g = [0] * mx
            for i in valid:
                for j in d[i]:
                    g[i] = (g[i] + f[j]) % mod
            f = g
        return sum(f) % mod
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. k colours instead of three
# 2. Also forbid equal colours on the DIAGONAL neighbours


# ============================================================
# FOLLOW-UP 1: k colours instead of three
# ============================================================

# ---- follow-up 1: function ----
# Answer: Enumerate the valid columns of height m and
# count the compatible neighbours.
# Time: O(n * C^2 * m), C = valid columns (about k * (k -
# 1)^(m - 1)).  Space: O(C).
```

```
import itertools


def paint_grid_k_colours(m, n, k):
    mod = 10 ** 9 + 7
    columns = [c for c in itertools.product(range(k), repeat=m) if all(c[i] != c[i + 1] for i in range(m -
 1))]
    ways = {c: 1 for c in columns}
    for _ in range(n - 1):
        new = {}
        for c in columns:
            new[c] = sum(ways[q] for q in columns if all(q[i] != c[i] for i in range(m))) % mod
        ways = new
    return sum(ways.values()) % mod


# ============================================================
# FOLLOW-UP 2: Also forbid equal colours on the DIAGONAL
# neighbours
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same table, with a compatibility test that
# includes diagonals.
# Time: O(n * C^2 * m).  Space: O(C).
def paint_grid_no_diagonal(m, n, k):
    mod = 10 ** 9 + 7
    columns = [c for c in itertools.product(range(k), repeat=m) if all(c[i] != c[i + 1] for i in range(m -
 1))]

    def compatible(q, c):
        for i in range(m):
            if q[i] == c[i]:
                return False
            if i + 1 < m and q[i] == c[i + 1]:
                return False
            if i > 0 and q[i] == c[i - 1]:
                return False
        return True

    ways = {c: 1 for c in columns}
    for _ in range(n - 1):
        new = {}
        for c in columns:
            new[c] = sum(ways[q] for q in columns if compatible(q, c)) % mod
        ways = new
    return sum(ways.values()) % mod
```

3651. Minimum Cost Path with Teleportations

Hard · Array, Dynamic Programming, Matrix

leetcode.com/problems/minimum-cost-path-with-teleportations

PROBLEM

You are given a `m x n` 2D integer array `grid` and an integer `k`. You start at the top-left cell `(0, 0)` and your goal is to reach the bottom-right cell `(m - 1, n - 1)`.

There are two types of moves available:

- **Normal move**: You can move right or down from your current cell `(i, j)`, i.e. you can move to `(i, j + 1)` (right) or `(i + 1, j)` (down). The cost is the value of the destination cell.
- **Teleportation**: You can teleport from any cell `(i, j)`, to any cell `(x, y)` such that `grid[x][y] <= grid[i][j]`; the cost of this move is 0. You may teleport at most `k` times.

Return the **minimum** total cost to reach cell `(m - 1, n - 1)` from `(0, 0)`.

Example 1:

Input: grid = [[1,3,3],[2,5,4],[4,3,5]], k = 2

Output: 7

Explanation:

Initially we are at (0, 0) and cost is 0.

Current Position	Move	New Position	Total Cost
`(0, 0)`	Move Down	`(1, 0)`	`0 + 2 = 2`
`(1, 0)`	Move Right	`(1, 1)`	`2 + 5 = 7`
`(1, 1)`	Teleport to `(2, 2)`	`(2, 2)`	`7 + 0 = 7`

The minimum cost to reach bottom-right cell is 7.

Example 2:

Input: grid = [[1,2],[2,3],[3,4]], k = 1

Output: 9

Explanation:

Initially we are at (0, 0) and cost is 0.

Current Position	Move	New Position	Total Cost
`(0, 0)`	Move Down	`(1, 0)`	`0 + 2 = 2`
`(1, 0)`	Move Right	`(1, 1)`	`2 + 3 = 5`
`(1, 1)`	Move Down	`(2, 1)`	`5 + 4 = 9`

The minimum cost to reach bottom-right cell is 9.

Constraints:

- `2 <= m, n <= 80`
- `m == grid.length`
- `n == grid[i].length`
- `0 <= grid[i][j] <= 10^4`
- `0 <= k <= 10`

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

Besides paying a cell when moving right or down, we may teleport from a larger value to a smaller one at most k times. The leftover teleport count is part of the state. f[t][i][j] is the cheapest way to reach (i,j) with t teleports. Layer t=0 uses only grid moves. Teleports require a nonincreasing value, so cells are scanned from large to small and the best previous-layer cost among those cells is written onto the current layer. A second right/down sweep after the assignment lets the walk continue. The answer is the minimum destination cost over all t.

We define f[t][i][j] as the minimum cost to reach cell (i, j) using exactly t teleportations. Initially, f[0][0][0] = 0, and all other states are infinity.

First, we need to initialize f[0][i][j]. Without using teleportation, we can only reach cell (i, j) by moving right or down.

If i > 0, we can move from the cell above (i-1, j), updating the state as:

$$f[0][i][j] = \min(f[0][i][j], f[0][i-1][j] + grid[i][j])$$

If j > 0, we can move from the cell to the left (i, j-1), updating the state as:

$$f[0][i][j] = \min(f[0][i][j], f[0][i][j-1] + grid[i][j])$$

To handle teleportation, we need to group the cells in the grid by their values. We use a hash map g, where the key is the cell value and the value is a list of coordinates of cells with that value.

For each teleportation count t from 1 to k, we process each group in descending order of cell values. For each cell (i, j) in a group, we first update a global minimum mn, representing the minimum cost to reach these cells using t-1 teleportations:

$$mn = \min(mn, f[t-1][i][j])$$

Then, we update the state of all cells in the group to mn, representing the minimum cost to reach these cells via teleportation.

Next, we traverse the entire grid again to update f[t][i][j], considering moves from the top or left cells:

If i > 0, then:

$$f[t][i][j] = \min(f[t][i][j], f[t][i-1][j] + grid[i][j])$$

If j > 0, then:

$$f[t][i][j] = \min(f[t][i][j], f[t][i][j-1] + grid[i][j])$$

Finally, our answer is min(f[t][m-1][n-1]), where t ranges from 0 to k.

The time complexity is $O((k + \log mn) \times mn)$, and the space complexity is $O(k \times mn)$. Here, m and n are the number of rows and columns of the grid, respectively, and k is the maximum allowed number of teleportations.

Python 3

```python
class Solution:
    def minCost(self, grid: List[List[int]], k: int) -> int:
        m, n = len(grid), len(grid[0])
        f = [[[inf] * n for _ in range(m)] for _ in range(k + 1)]
        f[0][0][0] = 0
        for i in range(m):
            for j in range(n):
                if i:
                    f[0][i][j] = min(f[0][i][j], f[0][i - 1][j] + grid[i][j])
                if j:
                    f[0][i][j] = min(f[0][i][j], f[0][i][j - 1] + grid[i][j])
        g = defaultdict(list)
        for i, row in enumerate(grid):
            for j, x in enumerate(row):
                g[x].append((i, j))
        keys = sorted(g, reverse=True)
        for t in range(1, k + 1):
            mn = inf
            for key in keys:
                pos = g[key]
                for i, j in pos:
                    mn = min(mn, f[t - 1][i][j])
                for i, j in pos:
                    f[t][i][j] = mn
            for i in range(m):
                for j in range(n):
                    if i:
                        f[t][i][j] = min(f[t][i][j], f[t][i - 1][j] + grid[i][j])
                    if j:
                        f[t][i][j] = min(f[t][i][j], f[t][i][j - 1] + grid[i][j])
        return min(f[t][m - 1][n - 1] for t in range(k + 1))
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. No teleports at all
# 2. A teleport costs c instead of 0


# ============================================================
# FOLLOW-UP 1: No teleports at all
# ============================================================

# ---- follow-up 1: function ----
# Answer: The classic grid DP: every cell adds its own
# value except the start.
# Time: O(m * n).  Space: O(m * n).
def min_cost_no_teleport(grid):
    m, n = len(grid), len(grid[0])
    INF = float("inf")
    dp = [[INF] * n for _ in range(m)]
    dp[0][0] = 0
    for i in range(m):
        for j in range(n):
            if i:
                dp[i][j] = min(dp[i][j], dp[i - 1][j] + grid[i][j])
            if j:
                dp[i][j] = min(dp[i][j], dp[i][j - 1] + grid[i][j])
    return dp[m - 1][n - 1]


# ============================================================
# FOLLOW-UP 2: A teleport costs c instead of 0
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Layer by layer: the best way to arrive by
# teleport is the cheapest cell with a bigger or equal
# value, plus c.
# Time: O(k * m * n log (m * n)).  Space: O(m * n).
def min_cost_teleport_price(grid, k, c):
    m, n = len(grid), len(grid[0])
    INF = float("inf")
    cells = [(grid[i][j], i, j) for i in range(m) for j in range(n)]
    layer = None
    answer = INF
    for used in range(k + 1):
        dp = [[INF] * n for _ in range(m)]
        if used == 0:
            dp[0][0] = 0
        else:
            # best[v] is the cheapest earlier layer cost over cells with value >= v
            best_for_value = {}
            for value, i, j in cells:
                best_for_value[value] = min(best_for_value.get(value, INF), layer[i][j])
            running = INF
            minimum_from_above = {}
            for value in sorted(best_for_value, reverse=True):
                running = min(running, best_for_value[value])
                minimum_from_above[value] = running
            for i in range(m):
                for j in range(n):
                    dp[i][j] = min(layer[i][j], minimum_from_above[grid[i][j]] + c)
        for i in range(m):
```

```
            for j in range(n):
                if i:
                    dp[i][j] = min(dp[i][j], dp[i - 1][j] + grid[i][j])
                if j:
                    dp[i][j] = min(dp[i][j], dp[i][j - 1] + grid[i][j])
        layer = dp
        answer = min(answer, dp[m - 1][n - 1])
    return answer
```

PART 2 · TOPIC 41 OF 50

String DP

5 problems · 0 Easy · 2 Medium · 3 Hard

THE IDEA

Compare prefixes of one or two strings: dp[i][j] describes the first i characters of one and the first j of the other.

THE 5 PROBLEMS, EASY TO HARD | ★ = TOP 5: DO THESE FIRST

1	★ **Longest Palindromic Subsequence**	Medium	LC 516	2663
2	★ **Minimum Deletions to Make String Balanced**	Medium	LC 1653	2666
3	★ **Palindrome Partitioning II**	Hard	LC 132	2669
4	★ **Word Break II**	Hard	LC 140	2672
5	★ **Shortest Common Supersequence**	Hard	LC 1092	2675

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Longest Common Subsequence (Part 1, pattern 22, page 1182)
Decode Ways (Part 1, pattern 6, page 316)
Interleaving String (Part 1, pattern 22, page 1191)
Regular Expression Matching (Part 1, pattern 7, page 367)
Strange Printer (Part 1, pattern 14, page 768)
Edit Distance (Part 1, pattern 22, page 1206)
Word Break (Part 1, pattern 6, page 323)
Wildcard Matching (Part 1, pattern 7, page 363)
Distinct Subsequences (Part 1, pattern 22, page 1209)

PART 2 · STRING DP · PROBLEM 1 OF 5

516. Longest Palindromic Subsequence

Medium · String, Dynamic Programming

leetcode.com/problems/longest-palindromic-subsequence

★ **TOP 7 PROBLEM** in String DP: master this one first

PROBLEM

Given a string `s`, find *the longest palindromic **subsequence**'s length in* `s`.

A **subsequence** is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements.

Example 1:

```
Input: s = "bbbab"
Output: 4
Explanation: One possible longest palindromic subsequence is "bbbb".
```

Example 2:

```
Input: s = "cbbd"
Output: 2
Explanation: One possible longest palindromic subsequence is "bb".
```

Constraints:

- `1 <= s.length <= 1000`
- `s` consists only of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

A palindromic subsequence need not be contiguous, so center expansion is not enough, and listing all subsequences is too many. Let f[i][j] be the LPS length of s[i..j]. Equal ends add 2 to the inner interval; otherwise drop one end. Fill by increasing interval length (i right to left, j to the right) so dependencies are ready. f[0][n-1] is the answer.

We define f[i][j] as the length of the longest palindromic subsequence from the i-th character to the j-th character in string s. Initially, f[i][i] = 1, and the values of other positions are all 0.

If s[i] = s[j], then f[i][j] = f[i + 1][j - 1] + 2; otherwise, f[i][j] = max(f[i + 1][j], f[i][j - 1]).

Since the value of f[i][j] is related to f[i + 1][j - 1], f[i + 1][j], and f[i][j - 1], we should enumerate i from large to small, and enumerate j from small to large.

The answer is f[0][n - 1].

The time complexity is $O(n^2)$, and the space complexity is $O(n^2)$. Where n is the length of the string s.

Python 3

```python
class Solution:
    def longestPalindromeSubseq(self, s: str) -> int:
        n = len(s)
        f = [[0] * n for _ in range(n)]
        for i in range(n):
            f[i][i] = 1
        for i in range(n - 1, -1, -1):
            for j in range(i + 1, n):
                if s[i] == s[j]:
                    f[i][j] = f[i + 1][j - 1] + 2
                else:
                    f[i][j] = max(f[i + 1][j], f[i][j - 1])
        return f[0][-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subsequence itself
# 2. The fewest insertions that make the string a
# palindrome


# ============================================================
# FOLLOW-UP 1: Return the subsequence itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Interval table, then rebuild from both ends.
# Time: O(n^2).  Space: O(n^2).
def longest_palindromic_subsequence_text(s):
    n = len(s)
    dp = [[0] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):
        dp[i][i] = 1
        for j in range(i + 1, n):
            if s[i] == s[j]:
                dp[i][j] = dp[i + 1][j - 1] + 2
            else:
                dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
    left, right = [], []
    i, j = 0, n - 1
    while i <= j:
        if i == j:
            left.append(s[i])
            break
        if s[i] == s[j]:
            left.append(s[i])
            right.append(s[j])
            i += 1
            j -= 1
        elif dp[i + 1][j] >= dp[i][j - 1]:
            i += 1
        else:
            j -= 1
    return "".join(left + right[::-1])


# ============================================================
# FOLLOW-UP 2: The fewest insertions that make the string
# a palindrome
# ============================================================

# ---- follow-up 2: function ----
# Answer: The length minus the longest palindromic
# subsequence.
```

```python
# Time: O(n^2).  Space: O(n^2).
def min_insertions_for_palindrome(s):
    return len(s) - len(longest_palindromic_subsequence_text(s))
```

PART 2 · STRING DP · PROBLEM 2 OF 5

1653. Minimum Deletions to Make String Balanced

Medium · Stack, String, Dynamic Programming

leetcode.com/problems/minimum-deletions-to-make-string-balanced

★ **TOP 7 PROBLEM** in String DP: master this one first

PROBLEM

You are given a string `s` consisting only of characters `'a'` and `'b'`.

You can delete any number of characters in `s` to make `s` **balanced**. `s` is **balanced** if there is no pair of indices `(i,j)` such that `i < j` and `s[i] = 'b'` and `s[j]= 'a'`.

Return *the **minimum** number of deletions needed to make* `s` ***balanced***.

Example 1:

```
Input: s = "aababbab"
Output: 2
Explanation: You can either:
Delete the characters at 0-indexed positions 2 and 6 ("aababbab" -> "aaabbb"), or
Delete the characters at 0-indexed positions 3 and 6 ("aababbab" -> "aabbbb").
```

Example 2:

```
Input: s = "bbaaaaabb"
Output: 2
Explanation: The only solution is to delete the first two characters.
```

Constraints:

- `1 <= s.length <= 10^5`
- `s[i]` is `'a'` or `'b'`.

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

A balanced string is some `a`s followed by some `b`s. n can be 10^5, so we cannot try every deletion set. After a prefix, the number of `b`s seen is the cost of deleting them versus deleting the current `a`. f[i] is the min deletions to balance the first i characters: on `b` keep f[i-1] and increment b; on `a` take min(f[i-1]+1,b).

We define f[i] as the minimum number of characters to be deleted in the first i characters to make the string balanced. Initially, f[0]=0. The answer is f[n].

We traverse the string s, maintaining a variable b, which represents the number of character 'b' in the characters before the current position.

- If the current character is 'b', it does not affect the balance of the first i characters, so f[i]=f[i-1], then we update b ← b+1.
- If the current character is 'a', we can choose to delete the current character, so f[i]=f[i-1]+1; or we can choose to delete the previous character 'b', so f[i]=b. Therefore, we take the minimum of the two, that is, f[i]=min(f[i-1]+1,b).

In summary, we can get the state transition equation:

f[i]=cases f[i-1], & s[i-1]='b'\ min(f[i-1]+1,b), & s[i-1]='a' cases

The final answer is f[n].

We notice that the state transition equation is only related to the previous state and the variable b, so we can just use an answer variable ans to maintain the current f[i], and there is no need to allocate an array f.

The time complexity is O(n), where n is the length of the string s. The space complexity is O(1).

Python 3

```python
class Solution:
    def minimumDeletions(self, s: str) -> int:
        n = len(s)
        f = [0] * (n + 1)
        b = 0
        for i, c in enumerate(s, 1):
            if c == 'b':
                f[i] = f[i - 1]
                b += 1
            else:
                f[i] = min(f[i - 1] + 1, b)
        return f[n]
```

Solution 2: Enumeration + Prefix Sum

> **How to think about it**
>
> Solution 1 is a prefix DP. We can also pick a cut: delete every `b` on the left and every `a` on the right; their sum is the deletion cost. Track left-hand `b`s and right-hand `a`s and take the minimum sum in one pass, still O(1) extra space.

We can enumerate each position i in the string s, dividing the string s into two parts, namely s[0,..,i-1] and s[i+1,..n-1]. To make the string balanced, the number of characters we need to delete at the current position i is the number of character 'b' in s[0,..,i-1] plus the number of character 'a' in s[i+1,..n-1].

Therefore, we maintain two variables lb and ra to represent the number of character 'b' in s[0,..,i-1] and the number of character 'a' in s[i+1,..n-1] respectively. The number of characters we need to delete is lb+ra. During the enumeration process, we update the variables lb and ra.

The time complexity is O(n), and the space complexity is O(1). Here, n is the length of the string s.

Python 3

```python
class Solution:
    def minimumDeletions(self, s: str) -> int:
        lb, ra = 0, s.count('a')
        ans = len(s)
        for c in s:
            ra -= c == 'a'
            ans = min(ans, lb + ra)
            lb += c == 'b'
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the string that is left (no 'b' before an 'a')
# 2. The reverse rule: no 'a' before a 'b'


# ============================================================
# FOLLOW-UP 1: Return the string that is left (no 'b'
# before an 'a')
# ============================================================

# ---- follow-up 1: function ----
# Answer: Pick the split point with the fewest deletions:
# delete the b's before it and the a's after it.
# Time: O(n).  Space: O(n).
def balanced_string_left(s):
    a_after = s.count("a")
    b_before = 0
```

```
    best_cost, split = a_after, 0                 # split 0: every a is deleted
    for i, ch in enumerate(s):
        if ch == "a":
            a_after -= 1
        else:
            b_before += 1
        if b_before + a_after < best_cost:
            best_cost, split = b_before + a_after, i + 1
    return "".join(c for c in s[:split] if c == "a") + "".join(c for c in s[split:] if c == "b")


# ============================================================
# FOLLOW-UP 2: The reverse rule: no 'a' before a 'b'
# ============================================================

# ---- follow-up 2: function ----
# Answer: Mirror image: delete so that all b come first.
# Time: O(n).  Space: O(1).
def min_deletions_b_first(s):
    a_before = 0
    b_after = s.count("b")
    best = b_after
    for ch in s:
        if ch == "a":
            a_before += 1
        else:
            b_after -= 1
        best = min(best, a_before + b_after)
    return best
```

PART 2 · STRING DP · PROBLEM 3 OF 5

132. Palindrome Partitioning II

Hard · String, Dynamic Programming

leetcode.com/problems/palindrome-partitioning-ii

★ **TOP 7 PROBLEM** in String DP: master this one first

PROBLEM

Given a string `s`, partition `s` such that every substring of the partition is a palindrome.

Return *the **minimum** cuts needed for a palindrome partitioning of* `s`.

Example 1:

```
Input: s = "aab"
Output: 1
Explanation: The palindrome partitioning ["aa","b"] could be produced using 1 cut.
```

Example 2:

```
Input: s = "a"
Output: 0
```

Example 3:

```
Input: s = "ab"
Output: 1
```

Constraints:

- `1 <= s.length <= 2000`
- `s` consists of lowercase English letters only.

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

We only need the fewest cuts, not the partitions. n≤ 2000, so enumerating them is impossible. Precompute palindromic spans as before. f[i] is the min cuts of s[0..i]: try a last palindrome s[j..i] and take f[j-1]+1; if the whole prefix is a palindrome, the answer is 0.

First, we preprocess the string s to determine whether each substring s[i..j] is a palindrome, and record this in a 2D array g[i][j], where g[i][j] indicates whether the substring s[i..j] is a palindrome.

Next, we define f[i] to represent the minimum number of cuts needed for the substring s[0..i-1]. Initially, f[i] = i.

Next, we consider how to transition the state for f[i]. We can enumerate the previous cut point j. If the substring s[j..i] is a palindrome, then f[i] can be transitioned from f[j]. If j = 0, it means that s[0..i] itself is a palindrome, and no cuts are needed, i.e., f[i] = 0. Therefore, the state transition equation is as follows:

$$f[i] = \min_{0 \le j \le i} \text{cases } f[j-1] + 1, \text{ \& if } g[j][i] = \text{True } \backslash\ 0, \text{ \& if } g[0][i] = \text{True cases}$$

The answer is f[n], where n is the length of the string s.

The time complexity is $O(n^2)$, and the space complexity is $O(n^2)$. Here, n is the length of the string s.

Python 3

```python
class Solution:
    def minCut(self, s: str) -> int:
        n = len(s)
        g = [[True] * n for _ in range(n)]
        for i in range(n - 1, -1, -1):
            for j in range(i + 1, n):
                g[i][j] = s[i] == s[j] and g[i + 1][j - 1]
        f = list(range(n))
        for i in range(1, n):
            for j in range(i + 1):
                if g[j][i]:
                    f[i] = min(f[i], 1 + f[j - 1] if j else 0)
        return f[-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return one best partition
# 2. How many palindromic substrings are there (by
# position)


# ============================================================
# FOLLOW-UP 1: Return one best partition
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember the split point of every prefix.
# Time: O(n^2).  Space: O(n^2).
def best_partition(s):
    n = len(s)
    is_pal = [[False] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):
        for j in range(i, n):
            is_pal[i][j] = s[i] == s[j] and (j - i < 2 or is_pal[i + 1][j - 1])
    cuts = [0] * n
    start = [0] * n
    for j in range(n):
        if is_pal[0][j]:
            cuts[j] = 0
            start[j] = 0
        else:
            cuts[j] = float("inf")
            for i in range(1, j + 1):
                if is_pal[i][j] and cuts[i - 1] + 1 < cuts[j]:
                    cuts[j] = cuts[i - 1] + 1
                    start[j] = i
    parts = []
    j = n - 1
    while j >= 0:
        parts.append(s[start[j]:j + 1])
        j = start[j] - 1
    return parts[::-1]


# ============================================================
# FOLLOW-UP 2: How many palindromic substrings are there
# (by position)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Expand around every centre.
# Time: O(n^2).  Space: O(1).
def count_palindromic_substrings(s):
    count = 0
```

```
    for centre in range(2 * len(s) - 1):
        i, j = centre // 2, centre // 2 + centre % 2
        while i >= 0 and j < len(s) and s[i] == s[j]:
            count += 1
            i -= 1
            j += 1
    return count
```

PART 2 · STRING DP · PROBLEM 4 OF 5

140. Word Break II

Hard · Trie, Memoization, Array, Hash Table, String, Dynamic Programming, Backtracking

leetcode.com/problems/word-break-ii

★ **TOP 7 PROBLEM** in String DP: master this one first

PROBLEM

Given a string `s` and a dictionary of strings `wordDict`, add spaces in `s` to construct a sentence where each word is a valid dictionary word. Return all such possible sentences in **any order**.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

```
Input: s = "catsanddog", wordDict = ["cat","cats","and","sand","dog"]
Output: ["cats and dog","cat sand dog"]
```

Example 2:

```
Input: s = "pineapplepenapple", wordDict = ["apple","pen","applepen","pine","pineapple"]
Output: ["pine apple pen apple","pineapple pen apple","pine applepen apple"]
Explanation: Note that you are allowed to reuse a dictionary word.
```

Example 3:

```
Input: s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]
Output: []
```

Constraints:

- `1 <= s.length <= 20`
- `1 <= wordDict.length <= 1000`
- `1 <= wordDict[i].length <= 10`
- `s` and `wordDict[i]` consist of only lowercase English letters.
- All the strings of `wordDict` are **unique**.
- Input is generated in a way that the length of the answer doesn't exceed 10^5.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

We must list every sentence, not only decide feasibility. |s|≤ 20, so the number of partitions is manageable. After inserting words into a trie, try every next word ending in the current suffix; on a hit, recurse on the rest and join the pieces. The trie tests prefixes quickly.

Python 3

```
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, word):
        node = self
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, word):
        node = self
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                return False
            node = node.children[idx]
        return node.is_end


class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> List[str]:
        def dfs(s):
            if not s:
                return [[]]
            res = []
            for i in range(1, len(s) + 1):
                if trie.search(s[:i]):
                    for v in dfs(s[i:]):
                        res.append([s[:i]] + v)
            return res

        trie = Trie()
        for w in wordDict:
            trie.insert(w)
        ans = dfs(s)
        return [' '.join(v) for v in ans]
```

Complexity. Time: O(2^n) in the worst case (the number of sentences can be exponential). Space: O(2^n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. How many sentences are possible
# 2. The sentence with the fewest words


# ============================================================
# FOLLOW-UP 1: How many sentences are possible
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count with a table over positions instead of
# listing.
# Time: O(n^3) (every slice costs O(n)).  Space: O(n).
def count_sentences(s, word_dict):
    words = set(word_dict)
    ways = [0] * (len(s) + 1)
    ways[0] = 1
    for end in range(1, len(s) + 1):
        for start in range(end):
            if ways[start] and s[start:end] in words:
```

```
                ways[end] += ways[start]
    return ways[len(s)]


# ============================================================
# FOLLOW-UP 2: The sentence with the fewest words
# ============================================================

# ---- follow-up 2: function ----
# Answer: Table of the fewest words for every prefix, then
# rebuild.
# Time: O(n^3).  Space: O(n^2).
def shortest_sentence(s, word_dict):
    words = set(word_dict)
    best = [None] * (len(s) + 1)
    best[0] = []
    for end in range(1, len(s) + 1):
        for start in range(end):
            if best[start] is not None and s[start:end] in words:
                candidate = best[start] + [s[start:end]]
                if best[end] is None or len(candidate) < len(best[end]):
                    best[end] = candidate
    return " ".join(best[len(s)]) if best[len(s)] is not None else None
```

PART 2 · STRING DP · PROBLEM 5 OF 5

1092. Shortest Common Supersequence

Hard · String, Dynamic Programming, Longest Common Subsequence

leetcode.com/problems/shortest-common-supersequence

★ **TOP 7 PROBLEM** in String DP: master this one first

PROBLEM

Given two strings `str1` and `str2`, return *the shortest string that has both* `str1` *and* `str2` *as* ***subsequences***. If there are multiple valid strings, return **any** of them.

A string `s` is a **subsequence** of string `t` if deleting some number of characters from `t` (possibly `0`) results in the string `s`.

Example 1:

```
Input: str1 = "abac", str2 = "cab"
Output: "cabac"
Explanation:
str1 = "abac" is a subsequence of "cabac" because we can delete the first "c".
str2 = "cab" is a subsequence of "cabac" because we can delete the last "ac".
The answer provided is the shortest such string that satisfies these properties.
```

Example 2:

```
Input: str1 = "aaaaaaaa", str2 = "aaaaaaaa"
Output: "aaaaaaaa"
```

Constraints:

- `1 <= str1.length, str2.length <= 1000`
- `str1` and `str2` consist of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

The SCS length is $|s_1|+|s_2|$-LCS. m,n≤ 1000: build the LCS table, then walk the transitions backward, inserting the characters that are not shared. f[i][j] is the LCS of the two prefixes. From (m,n), a move from above or the left emits the discarded character; an equal pair emits the common character and steps diagonally. The walk collects the reverse of the SCS; leftover characters on one side are appended when the other index hits zero.

Python 3

```python
class Solution:
    def shortestCommonSupersequence(self, str1: str, str2: str) -> str:
        m, n = len(str1), len(str2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if str1[i - 1] == str2[j - 1]:
                    f[i][j] = f[i - 1][j - 1] + 1
                else:
                    f[i][j] = max(f[i - 1][j], f[i][j - 1])
        ans = []
        i, j = m, n
        while i or j:
            if i == 0:
                j -= 1
                ans.append(str2[j])
            elif j == 0:
                i -= 1
                ans.append(str1[i])
            else:
                if f[i][j] == f[i - 1][j]:
                    i -= 1
                    ans.append(str1[i])
                elif f[i][j] == f[i][j - 1]:
                    j -= 1
                    ans.append(str2[j])
                else:
                    i, j = i - 1, j - 1
                    ans.append(str1[i])
        return ''.join(ans[::-1])
```

Complexity. Time: O(m * n). Space: O(m * n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Only the length of the shortest common supersequence
# 2. The length for THREE strings


# ============================================================
# FOLLOW-UP 1: Only the length of the shortest common
# supersequence
# ============================================================

# ---- follow-up 1: function ----
# Answer: len(a) + len(b) - LCS(a, b).
# Time: O(a * b).  Space: O(a * b).
def scs_length(a, b):
    dp = [[0] * (len(b) + 1) for _ in range(len(a) + 1)]
    for i in range(1, len(a) + 1):
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return len(a) + len(b) - dp[len(a)][len(b)]


# ============================================================
# FOLLOW-UP 2: The length for THREE strings
# ============================================================

# ---- follow-up 2: function ----
# Answer: A three-dimensional table: the shortest
# supersequence of the three prefixes.
```

```python
# Time: O(a * b * c).  Space: O(a * b * c).
def scs_length_three(a, b, c):
    INF = float("inf")
    dp = [[[INF] * (len(c) + 1) for _ in range(len(b) + 1)] for _ in range(len(a) + 1)]
    dp[0][0][0] = 0
    for i in range(len(a) + 1):
        for j in range(len(b) + 1):
            for k in range(len(c) + 1):
                if dp[i][j][k] == INF:
                    continue
                for letter in set(a[i:i + 1] + b[j:j + 1] + c[k:k + 1]):
                    ni = i + (1 if i < len(a) and a[i] == letter else 0)
                    nj = j + (1 if j < len(b) and b[j] == letter else 0)
                    nk = k + (1 if k < len(c) and c[k] == letter else 0)
                    dp[ni][nj][nk] = min(dp[ni][nj][nk], dp[i][j][k] + 1)
    return dp[len(a)][len(b)][len(c)]
```

PART 2 · TOPIC 42 OF 50

Tree / Graph DP

3 problems · 0 Easy · 1 Medium · 2 Hard

THE IDEA

Solve each node's subtree first and return a small summary (take / skip, depth, counts); rerooting reuses the parent's answer to get all roots in O(n).

THE 3 PROBLEMS, EASY TO HARD | ★ = TOP 3: DO THESE FIRST

1	★ **House Robber III**	Medium	LC 337	2679
2	★ **Sum of Distances in Tree**	Hard	LC 834	2683
3	★ **Binary Tree Cameras**	Hard	LC 968	2688

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Unique Binary Search Trees II (Part 1, pattern 14, page 742)

Number of Ways to Arrive at Destination (Part 1, pattern 30, page 1625)

PART 2 · TREE / GRAPH DP · PROBLEM 1 OF 3

337. House Robber III

Medium · Tree, Depth-First Search, Dynamic Programming, Binary Tree, Tree DP

leetcode.com/problems/house-robber-iii

★ **TOP 7 PROBLEM** in Tree / Graph DP: master this one first

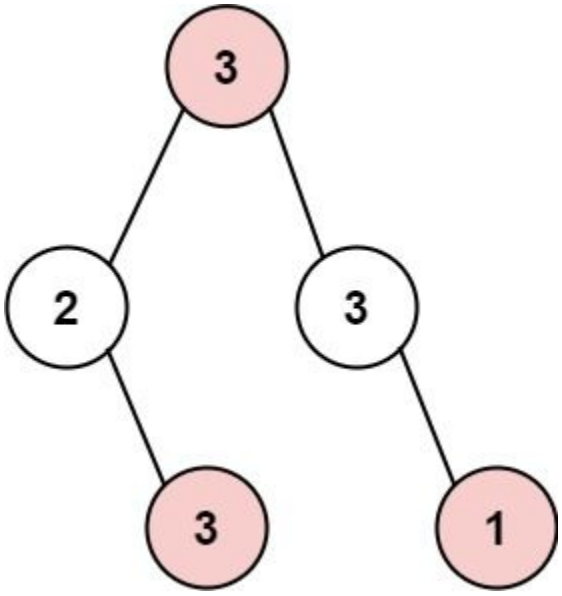
PROBLEM

The thief has found himself a new place for his thievery again. There is only one entrance to this area, called `root`.

Besides the `root`, each house has one and only one parent house. After a tour, the smart thief realized that all houses in this place form a binary tree. It will automatically contact the police if **two directly-linked houses were broken into on the same night**.

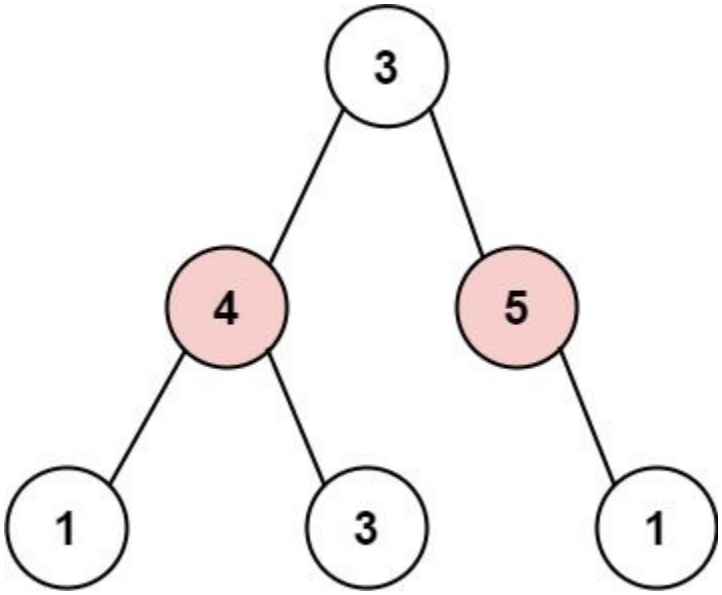
Given the `root` of the binary tree, return *the maximum amount of money the thief can rob **without alerting the police***.

Example 1:




```
Input: root = [3,2,3,null,3,null,1]
Output: 7
Explanation: Maximum amount of money the thief can rob = 3 + 3 + 1 = 7.
```

Example 2:




```
Input: root = [3,4,5,1,3,null,1]
Output: 9
Explanation: Maximum amount of money the thief can rob = 4 + 5 = 9.
```

Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.
- $0 <= Node.val <= 10^4$

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Adjacent nodes cannot both be robbed. Computing steal/skip separately and walking each subtree twice repeats work. Postorder returns (rob root, skip root). Robbing the root forces both children to skip; skipping takes the better of each child. The answer is the max at the root. Each node is visited once.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rob(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]) -> (int, int):
            if root is None:
                return 0, 0
            la, lb = dfs(root.left)
            ra, rb = dfs(root.right)
            return root.val + lb + rb, max(la, lb) + max(ra, rb)

        return max(dfs(root))
```

Complexity. Time: O(n). Space: O(h).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the houses that are robbed
# 2. At most k houses may be robbed


# ============================================================
# FOLLOW-UP 1: Return the houses that are robbed
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: Same (take, skip) summary, remembering the
# robbed nodes for each choice.
# Time: O(n^2) in the worst case (lists are concatenated).
# Space: O(n).
def robbed_houses(root):
    def go(node):
        # returns (best with node taken, nodes), (best with node skipped, nodes)
        if node is None:
            return (0, []), (0, [])
        left_take, left_skip = go(node.left)
        right_take, right_skip = go(node.right)
        take = (node.val + left_skip[0] + right_skip[0], [node.val] + left_skip[1] + right_skip[1])
        l = max(left_take, left_skip, key=lambda x: x[0])
        r = max(right_take, right_skip, key=lambda x: x[0])
        skip = (l[0] + r[0], l[1] + r[1])
        return take, skip

    take, skip = go(root)
    best = max(take, skip, key=lambda x: x[0])
    return best[0], sorted(best[1])


# ============================================================
# FOLLOW-UP 2: At most k houses may be robbed
# ============================================================

# ---- follow-up 2: function ----
# Answer: Table per node: the best value for each count of
# robbed houses, with the node taken or not.
# Time: O(n * k^2).  Space: O(h * k).
def rob_at_most_k(root, k):
    NEG = float("-inf")

    def merge(a, b):
        result = [NEG] * (k + 1)
        for i in range(k + 1):
            if a[i] == NEG:
                continue
            for j in range(k + 1 - i):
                if b[j] != NEG:
                    result[i + j] = max(result[i + j], a[i] + b[j])
        return result
```

```
    def go(node):
        # returns (taken[count], skipped[count])
        if node is None:
            empty = [NEG] * (k + 1)
            empty[0] = 0
```

```
            return empty, empty
        lt, ls = go(node.left)
        rt, rs = go(node.right)
        both_skip = merge(ls, rs)
        taken = [NEG] * (k + 1)
        for count in range(1, k + 1):
            if both_skip[count - 1] != NEG:
                taken[count] = both_skip[count - 1] + node.val
        child_best = [max(a, b) for a, b in zip(lt, ls)]
        child_best_r = [max(a, b) for a, b in zip(rt, rs)]
        skipped = merge(child_best, child_best_r)
        return taken, skipped

    taken, skipped = go(root)
    return max(max(taken), max(skipped))
```

834. Sum of Distances in Tree

Hard · Tree, Depth-First Search, Graph, Dynamic Programming, Tree DP

leetcode.com/problems/sum-of-distances-in-tree

★ **TOP 7 PROBLEM** in Tree / Graph DP: master this one first

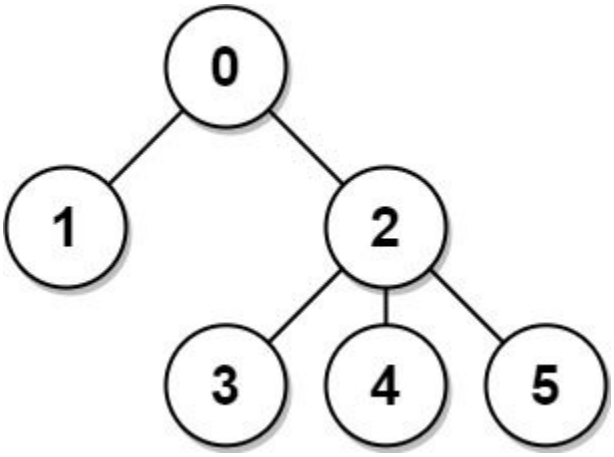
PROBLEM

There is an undirected connected tree with `n` nodes labeled from `0` to `n - 1` and `n - 1` edges.

You are given the integer `n` and the array `edges` where `edges[i] = [a_i, b_i]` indicates that there is an edge between nodes a_i and b_i in the tree.

Return an array `answer` of length `n` where `answer[i]` is the sum of the distances between the i^{th} node in the tree and all other nodes.

Example 1:

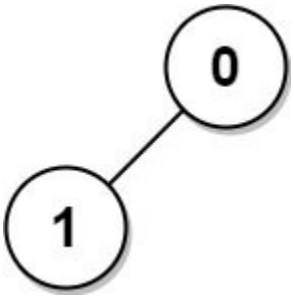



```
Input: n = 6, edges = [[0,1],[0,2],[2,3],[2,4],[2,5]]
Output: [8,12,6,10,10,10]
Explanation: The tree is shown above.
We can see that dist(0,1) + dist(0,2) + dist(0,3) + dist(0,4) + dist(0,5)
equals 1 + 1 + 2 + 2 + 2 = 8.
Hence, answer[0] = 8, and so on.
```

Example 2:

```
Input: n = 1, edges = []
Output: [0]
```

Example 3:




```
Input: n = 2, edges = [[1,0]]
Output: [1,1]
```

Constraints:

- $1 <= n <= 3 * 10^4$
- `edges.length == n - 1`
- `edges[i].length == 2`
- $0 <= a_i, b_i < n$
- $a_i != b_i$
- The given input represents a valid tree.

SOLUTIONS (PYTHON 3)

Solution 1: Tree DP (Re-rooting)

How to think about it

We need the sum of distances from every node. n≤ 3· 10^4, so a DFS from each root is quadratic. After rerooting, neighboring answers differ only by subtree size versus the rest of the tree. Root at 0 to get ans[0] and subtree sizes, then a second DFS pushes t-size[j]+n-size[j] to each child. Two traversals fill every answer.

First, we run a DFS to calculate the size of each node's subtree, recorded in the array size, and compute the sum of distances from node 0 to all other nodes, recorded in ans[0].

Next, we run another DFS to enumerate the sum of distances from each node when it is considered as the root. Suppose the answer for the current node i is t. When we move from node i to node j, the sum of distances changes to t - size[j] + n - size[j], meaning the sum of distances to node j and its subtree nodes decreases by size[j], while the sum of distances to other nodes increases by n - size[j].

The time complexity is O(n), and the space complexity is O(n), where n is the number of nodes in the tree.

Similar problems:

- 2581. Count Number of Possible Root Nodes

Python 3

```python
class Solution:
    def sumOfDistancesInTree(self, n: int, edges: List[List[int]]) -> List[int]:
        def dfs1(i: int, fa: int, d: int):
            ans[0] += d
            size[i] = 1
            for j in g[i]:
                if j != fa:
                    dfs1(j, i, d + 1)
                    size[i] += size[j]

        def dfs2(i: int, fa: int, t: int):
            ans[i] = t
            for j in g[i]:
                if j != fa:
                    dfs2(j, i, t - size[j] + n - size[j])

        g = defaultdict(list)
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)

        ans = [0] * n
        size = [0] * n
        dfs1(0, -1, 0)
        dfs2(0, -1, ans[0])
        return ans
```

Solution 2: Re-rooting DP + Explicit Stack

> **How to think about it**
>
> We need the sum of distances from every node. With $n \le 3 \cdot 10^4$, a walk from each root is quadratic. After rerooting, neighboring answers differ only by subtree size versus the rest of the tree. Recursion along a chain uses a call depth equal to n and overflows Python once the chain reaches length 1000. Subtree sizes are known only after the children finish, so the first walk is a post-order stack: state 0 adds the depth and pushes the children, and state 1 totals size. The second answer depends only on the parent's answer and the finished sizes, so a pre-order stack passes t-size[j]+n-size[j] to each child.

Root the tree at 0 and walk in post-order. The stack holds a node, its parent, a depth, and a state. In state 0, add the depth to ans[0], push the same node with state 1, and push every child. In state 1, every child size is already stored, and size[i] is 1 plus those sizes.

Then reroot with an explicit stack. It starts with (0,-1,ans[0]). When node i is popped, write the current sum t into ans[i]. Moving from i to a child j decreases the distances inside j's subtree by size[j] and increases the distances to every other node by n-size[j], so the pushed sum is t-size[j]+n-size[j].

The time complexity is O(n), and the space complexity is O(n), where n is the number of nodes in the tree.

Similar problems:

- 2581. Count Number of Possible Root Nodes

Python 3

```python
class Solution:
    def sumOfDistancesInTree(self, n: int, edges: List[List[int]]) -> List[int]:
        g = defaultdict(list)
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)
        ans = [0] * n
        size = [0] * n
        stk = [(0, -1, 0, 0)]
        while stk:
            i, fa, d, state = stk.pop()
            if state == 0:
                ans[0] += d
                stk.append((i, fa, d, 1))
                for j in g[i]:
                    if j != fa:
                        stk.append((j, i, d + 1, 0))
            else:
                size[i] = 1
                for j in g[i]:
                    if j != fa:
                        size[i] += size[j]
        walk = [(0, -1, ans[0])]
        while walk:
            i, fa, t = walk.pop()
            ans[i] = t
            for j in g[i]:
                if j != fa:
                    walk.append((j, i, t - size[j] + n - size[j]))
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Which node has the smallest sum of distances
# 2. The farthest distance from every node (its
# eccentricity)


# ============================================================
# FOLLOW-UP 1: Which node has the smallest sum of
# distances
# ============================================================

# ---- follow-up 1: function ----
# Answer: Compute all the sums by rerooting and take the
# smallest.
# Time: O(n).  Space: O(n).
def best_root(n, edges):
    graph = [[] for _ in range(n)]
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)
    size = [1] * n
    answer = [0] * n
    order = []
    parent = [-1] * n
    stack = [0]
    while stack:
        node = stack.pop()
        order.append(node)
        for nxt in graph[node]:
            if nxt != parent[node]:
                parent[nxt] = node
                stack.append(nxt)
```

```
    for node in reversed(order):
        for nxt in graph[node]:
            if nxt != parent[node]:
                size[node] += size[nxt]
                answer[node] += answer[nxt] + size[nxt]
    for node in order:
        for nxt in graph[node]:
            if nxt != parent[node]:
                answer[nxt] = answer[node] - size[nxt] + (n - size[nxt])
    best = min(range(n), key=lambda x: (answer[x], x))
    return best, answer[best]


# ============================================================
# FOLLOW-UP 2: The farthest distance from every node (its
# eccentricity)
# ============================================================

# ---- follow-up 2: function ----
# Answer: The farthest node from any node is one end of
# the diameter: take the larger distance to the two ends.
# Time: O(n).  Space: O(n).
def farthest_from_every_node(n, edges):
    graph = [[] for _ in range(n)]
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)
```

```
    def distances(start):
        d = [-1] * n
        d[start] = 0
        queue = [start]
        for x in queue:
            for y in graph[x]:
                if d[y] == -1:
                    d[y] = d[x] + 1
                    queue.append(y)
        return d

    first = distances(0)
    a = max(range(n), key=lambda x: first[x])
    da = distances(a)
    b = max(range(n), key=lambda x: da[x])
    db = distances(b)
    return [max(da[x], db[x]) for x in range(n)]
```

968. Binary Tree Cameras

Hard · Tree, Depth-First Search, Dynamic Programming, Binary Tree, Tree DP

leetcode.com/problems/binary-tree-cameras

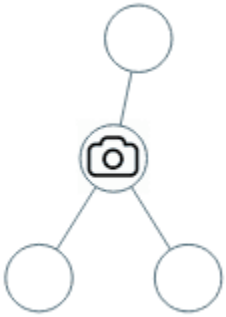
★ **TOP 7 PROBLEM** in Tree / Graph DP: master this one first

PROBLEM

You are given the `root` of a binary tree. We install cameras on the tree nodes where each camera at a node can monitor its parent, itself, and its immediate children.

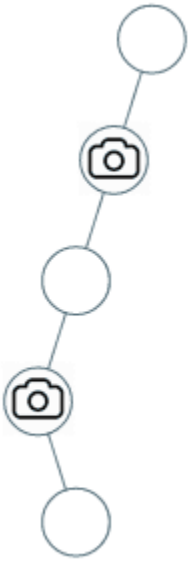
Return *the minimum number of cameras needed to monitor all nodes of the tree*.

Example 1:



```
Input: root = [0,0,null,0,0]
Output: 1
Explanation: One camera is enough to monitor all nodes if placed as shown.
```

Example 2:



```
Input: root = [0,0,null,0,null,0,null,null,0]
Output: 2
Explanation: At least two cameras are needed to monitor all nodes of the tree. The above image shows
one of the valid configurations of camera placement.
```

Constraints:

- The number of nodes in the tree is in the range `[1, 1000]`.
- `Node.val == 0`

SOLUTIONS (PYTHON 3)

Solution 1: Dynamic Programming (Tree DP)

> **How to think about it**
> A camera covers itself, its parent, and its children; we want as few as possible. The optimum at a node depends on the children, so we distinguish "has a camera / covered by a child / uncovered". Tree DP returns the three minima bottom-up; the root may not stay uncovered.

For each node, we define three states:

- `a`: The current node has a camera
- `b`: The current node does not have a camera, but is monitored by its children
- `c`: The current node does not have a camera and is not monitored by its children

Next, we design a function dfs(root), which will return an array of length 3, representing the minimum number of cameras in the subtree rooted at `root` for the three states. The answer is min(dfs(root)[0], dfs(root)[1]).

The calculation process of the function dfs(root) is as follows:

If `root` is null, return [inf, 0, 0], where `inf` represents a very large number, used to indicate an impossible situation.

Otherwise, we recursively calculate the left and right subtrees of `root`, obtaining [la, lb, lc] and [ra, rb, rc] respectively.

- If the current node has a camera, then its left and right children must be in a monitored state, i.e., a = min(la, lb, lc) + min(ra, rb, rc) + 1.
- If the current node does not have a camera but is monitored by its children, then one or both of the children must have a camera, i.e., b = min(la + rb, lb + ra, la + ra).
- If the current node does not have a camera and is not monitored by its children, then the children must be monitored by their children, i.e., c = lb + rb.

Finally, we return [a, b, c].

The time complexity is O(n), and the space complexity is O(n), where n is the number of nodes in the binary tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def minCameraCover(self, root: Optional[TreeNode]) -> int:
        def dfs(root):
            if root is None:
                return inf, 0, 0
            la, lb, lc = dfs(root.left)
            ra, rb, rc = dfs(root.right)
            a = min(la, lb, lc) + min(ra, rb, rc) + 1
            b = min(la + rb, lb + ra, la + ra)
            c = lb + rb
            return a, b, c

        a, b, _ = dfs(root)
        return min(a, b)
```

Solution 2: Tree DP on an Explicit Stack

> **How to think about it**
>
> A camera covers itself, its parent, and its children, so the minimum depends on how each node is covered. Bottom-up, the three states are “has a camera”, “covered by a child”, and “uncovered”. On a short tree, returning those three minima is enough, and the root may not stay uncovered. The tree can contain 1000 nodes. A left chain makes this postorder walk recurse once per node and overflow the call stack. The three states of a node depend only on the three states of its children, so children must be finished before their parent. An explicit stack performs that postorder walk. On entry it pushes an exit marker and the two children; on exit it writes the node’s states with the original three formulas. The answer is the smaller of the root’s first two states.

For each node, we define three states:

- `a`: The current node has a camera
- `b`: The current node does not have a camera, but is monitored by its children
- `c`: The current node does not have a camera and is not monitored by its children

A null node corresponds to (inf, 0, 0), where inf is a very large number used for an impossible situation.

We walk the binary tree in postorder with an explicit stack. On entry we push an exit marker, then the right child and the left child. On exit the children’s states are already stored as [la, lb, lc] and [ra, rb, rc].

- If the current node has a camera, then its left and right children must be in a monitored state, i.e., a = min(la, lb, lc) + min(ra, rb, rc) + 1.
- If the current node does not have a camera but is monitored by its children, then one or both of the children must have a camera, i.e., b = min(la + rb, lb + ra, la + ra).
- If the current node does not have a camera and is not monitored by its children, then the children must be monitored by their children, i.e., c = lb + rb.

The root must not stay uncovered, so the answer is min(a, b).

The time complexity is O(n), and the space complexity is O(n), where n is the number of nodes in the binary tree.

Python 3

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def minCameraCover(self, root: Optional[TreeNode]) -> int:
        if root is None:
            return 0
        stk = [(root, 0)]
        sub = {}
        while stk:
            node, state = stk.pop()
            if node is None:
                continue
            if state == 0:
                stk.append((node, 1))
                stk.append((node.right, 0))
                stk.append((node.left, 0))
                continue
            la, lb, lc = sub[id(node.left)] if node.left is not None else (inf, 0, 0)
            ra, rb, rc = sub[id(node.right)] if node.right is not None else (inf, 0, 0)
            a = min(la, lb, lc) + min(ra, rb, rc) + 1
            b = min(la + rb, lb + ra, la + ra)
            c = lb + rb
            sub[id(node)] = (a, b, c)
        a, b, _ = sub[id(root)]
        return min(a, b)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the nodes that get a camera
# 2. Check that a set of camera nodes covers the whole
# tree


# ============================================================
# FOLLOW-UP 1: Return the nodes that get a camera
# ============================================================

# ---- follow-up 1: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# ---- follow-up 1: function ----
# Answer: Greedy post-order: put a camera on the parent of
# every uncovered node.
# Time: O(n).  Space: O(h).
def camera_nodes(root):
    cameras = []

    # state: 0 = needs cover, 1 = has camera, 2 = covered without camera
    def go(node, parent):
        if node is None:
            return 2
        left = go(node.left, node)
        right = go(node.right, node)
        if left == 0 or right == 0:
            cameras.append(node)
            return 1
        if left == 1 or right == 1:
            return 2
        return 0

    if go(root, None) == 0:
        cameras.append(root)
    return cameras


# ============================================================
# FOLLOW-UP 2: Check that a set of camera nodes covers the
# whole tree
# ============================================================

# ---- follow-up 2: function ----
# Answer: A node is covered when it or a neighbour (parent
# or child) holds a camera.
# Time: O(n).  Space: O(h).
def covers_everything(root, camera_set):
    def go(node, parent_has_camera):
        if node is None:
            return True
        has = node in camera_set
        covered = has or parent_has_camera or (node.left in camera_set) or (node.right in camera_set)
        return covered and go(node.left, has) and go(node.right, has)

    return go(root, False)
```

PART 2 · TOPIC 43 OF 50

Bitmask DP

4 problems · 0 Easy · 3 Medium · 1 Hard

THE IDEA

Use an integer as a set of up to about 20 items; dp[mask] is the best value after using exactly those items.

THE 4 PROBLEMS, EASY TO HARD | ★ = TOP 4: DO THESE FIRST

1	★ **Campus Bikes II**	Medium	LC 1066	2693
2	★ **Minimum Number of Work Sessions to Finish the Tasks**	Medium	LC 1986	2697
3	★ **Fair Distribution of Cookies**	Medium	LC 2305	2700
4	★ **Shortest Path Visiting All Nodes**	Hard	LC 847	2703

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Partition to K Equal Sum Subsets (Part 1, pattern 15, page 829)

PART 2 · BITMASK DP · PROBLEM 1 OF 4

1066. Campus Bikes II

Medium · Bit Manipulation, Array, Dynamic Programming, Backtracking, Bitmask, Hungarian Algorithm, Bipartite Graph, Graph Matching, Min-Cost Flow, SSP, Network Flow

leetcode.com/problems/campus-bikes-ii

★ **TOP 7 PROBLEM** in Bitmask DP: master this one first

PROBLEM

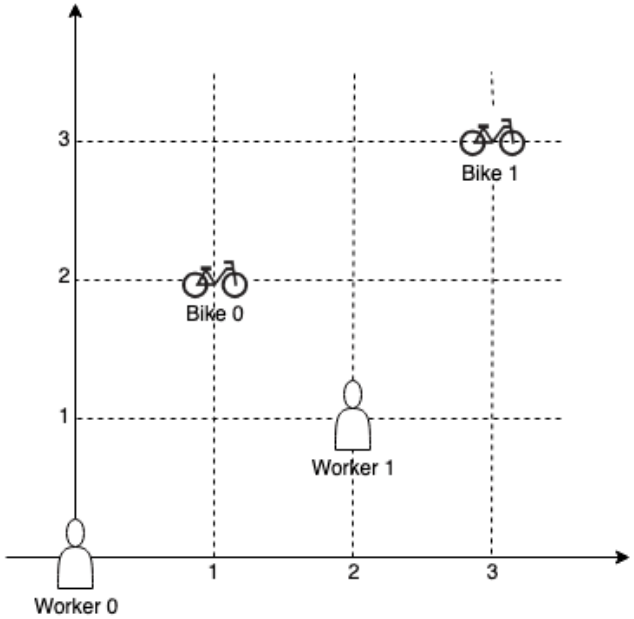
On a campus represented as a 2D grid, there are `n` workers and `m` bikes, with `n <= m`. Each worker and bike is a 2D coordinate on this grid.

We assign one unique bike to each worker so that the sum of the **Manhattan distances** between each worker and their assigned bike is minimized.

Return `the minimum possible sum of Manhattan distances between each worker and their assigned bike`.

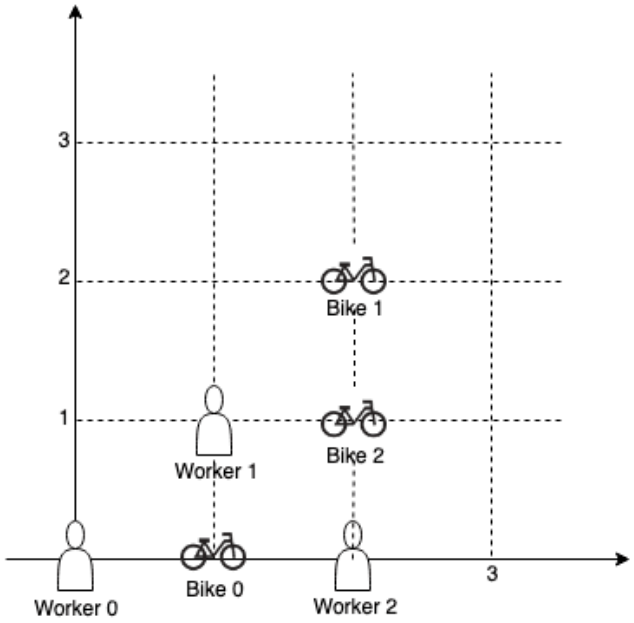
The **Manhattan distance** between two points `p1` and `p2` is `Manhattan(p1, p2) = |p1.x - p2.x| + |p1.y - p2.y|`.

Example 1:




```
Input: workers = [[0,0],[2,1]], bikes = [[1,2],[3,3]]
Output: 6
Explanation:
We assign bike 0 to worker 0, bike 1 to worker 1. The Manhattan distance of both assignments is 3, so the output is 6.
```

Example 2:




```
Input: workers = [[0,0],[1,1],[2,0]], bikes = [[1,0],[2,2],[2,1]]
Output: 4
Explanation:
We first assign bike 0 to worker 0, then assign bike 1 to worker 1 or worker 2, bike 2 to worker 2 or
worker 1. Both assignments lead to sum of the Manhattan distances as 4.
```

Example 3:

```
Input: workers = [[0,0],[1,0],[2,0],[3,0],[4,0]], bikes = [[0,999],[1,999],[2,999],[3,999],[4,999]]
Output: 4995
```

Constraints:

- `n == workers.length`
- `m == bikes.length`
- `1 <= n <= m <= 10`
- `workers[i].length == 2`
- `bikes[i].length == 2`
- `0 <= workers[i][0], workers[i][1], bikes[i][0], bikes[i][1] < 1000`
- All the workers and the bikes locations are **unique**.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

At most ten workers and bikes; listing assignments is P(m,n). The set of used bikes is an m-bit mask, so $n \cdot 2^m$ states fit. f[i][j] is the least total distance after i workers using mask j. For each bit k set in j we come from f[i-1][jxor 2^k] plus that pair's Manhattan distance. The answer is the minimum of row n.

Python 3

```
class Solution:
    def assignBikes(self, workers: List[List[int]], bikes: List[List[int]]) -> int:
        n, m = len(workers), len(bikes)
        f = [[inf] * (1 << m) for _ in range(n + 1)]
        f[0][0] = 0
        for i, (x1, y1) in enumerate(workers, 1):
            for j in range(1 << m):
                for k, (x2, y2) in enumerate(bikes):
                    if j >> k & 1:
                        f[i][j] = min(
                            f[i][j],
                            f[i - 1][j ^ (1 << k)] + abs(x1 - x2) + abs(y1 - y2),
                        )
        return min(f[n])
```

Complexity. Time: O(n * 2^m * m). Space: O(n * 2^m).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return which bike every worker gets
# 2. The MAXIMUM total distance instead


# ============================================================
# FOLLOW-UP 1: Return which bike every worker gets
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same mask table with the choice remembered.
# Time: O(2^m * m * n), m bikes, n workers (plus O(n) list
# copies per state).  Space: O(2^m * n).
def bike_assignment(workers, bikes):
    n, m = len(workers), len(bikes)

    def dist(w, b):
        return abs(workers[w][0] - bikes[b][0]) + abs(workers[w][1] - bikes[b][1])

    memo = {}

    def go(w, mask):
        if w == n:
            return 0, []
        if (w, mask) in memo:
            return memo[(w, mask)]
        best = (float("inf"), [])
        for b in range(m):
            if not (mask >> b) & 1:
                rest, picks = go(w + 1, mask | (1 << b))
                if rest + dist(w, b) < best[0]:
                    best = (rest + dist(w, b), [b] + picks)
        memo[(w, mask)] = best
        return best

    return go(0, 0)


# ============================================================
# FOLLOW-UP 2: The MAXIMUM total distance instead
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same table with max.
# Time: O(2^m * m).  Space: O(2^m).
def max_total_distance(workers, bikes):
    n, m = len(workers), len(bikes)
```

```
    memo = {}

    def go(w, mask):
        if w == n:
            return 0
        if (w, mask) in memo:
            return memo[(w, mask)]
        best = float("-inf")
        for b in range(m):
            if not (mask >> b) & 1:
                d = abs(workers[w][0] - bikes[b][0]) + abs(workers[w][1] - bikes[b][1])
                best = max(best, d + go(w + 1, mask | (1 << b)))
        memo[(w, mask)] = best
        return best

    return go(0, 0)
```

1986. Minimum Number of Work Sessions to Finish the Tasks

Medium · Bit Manipulation, Array, Dynamic Programming, Backtracking, Bitmask

leetcode.com/problems/minimum-number-of-work-sessions-to-finish-the-tasks

★ **TOP 7 PROBLEM** in Bitmask DP: master this one first

PROBLEM

There are `n` tasks assigned to you. The task times are represented as an integer array `tasks` of length `n`, where the i^{th} task takes `tasks[i]` hours to finish. A **work session** is when you work for **at most** `sessionTime` consecutive hours and then take a break.

You should finish the given tasks in a way that satisfies the following conditions:

- If you start a task in a work session, you must complete it in the **same** work session.
- You can start a new task **immediately** after finishing the previous one.
- You may complete the tasks in **any order**.

Given `tasks` and `sessionTime`, return *the **minimum** number of **work sessions** needed to finish all the tasks following the conditions above.*

The tests are generated such that `sessionTime` is **greater** than or **equal** to the **maximum** element in `tasks[i]`.

Example 1:

```
Input: tasks = [1,2,3], sessionTime = 3
Output: 2
Explanation: You can finish the tasks in two work sessions.
- First work session: finish the first and the second tasks in 1 + 2 = 3 hours.
- Second work session: finish the third task in 3 hours.
```

Example 2:

```
Input: tasks = [3,1,3,1,1], sessionTime = 8
Output: 2
Explanation: You can finish the tasks in two work sessions.
- First work session: finish all the tasks except the last one in 3 + 1 + 3 + 1 = 8 hours.
- Second work session: finish the last task in 1 hour.
```

Example 3:

```
Input: tasks = [1,2,3,4,5], sessionTime = 15
Output: 1
Explanation: You can finish all the tasks in one work session.
```

Constraints:

- `n == tasks.length`
- `1 <= n <= 14`
- `1 <= tasks[i] <= 10`
- `max(tasks[i]) <= sessionTime <= 15`

SOLUTION (PYTHON 3)

Solution 1: State Compression Dynamic Programming + Subset Enumeration

How to think about it

Tasks must fill sessions of capacity sessionTime, using as few sessions as possible. n≤ 14 admits subset DP. Mark subsets that fit in one session, then for each mask i try every submask j: if j is feasible, f[i]=min(f[ixor j]+1). f[2^n-1] is the session count.

We note that n does not exceed 14, so we can consider using state compression dynamic programming to solve this problem.

We use a binary number i of length n to represent the current task state, where the j-th bit of i is 1 if and only if the j-th task is completed. We use f[i] to represent the minimum number of work sessions needed to complete all tasks with state i.

We can enumerate all subsets j of i, where each bit of the binary representation of j is a subset of the corresponding bit of the binary representation of i, i.e., j ⊆ i. If the tasks corresponding to j can be completed in one work session, then we can update f[i] using f[i xor j] + 1, where i xor j represents the bitwise XOR of i and j.

The final answer is f[2^n - 1].

The time complexity is O(n × 3^n), and the space complexity is O(2^n). Here, n is the number of tasks.

Python 3

```python
class Solution:
    def minSessions(self, tasks: List[int], sessionTime: int) -> int:
        n = len(tasks)
        ok = [False] * (1 << n)
        for i in range(1, 1 << n):
            t = sum(tasks[j] for j in range(n) if i >> j & 1)
            ok[i] = t <= sessionTime
        f = [inf] * (1 << n)
        f[0] = 0
        for i in range(1, 1 << n):
            j = i
            while j:
                if ok[j]:
                    f[i] = min(f[i], f[i ^ j] + 1)
                j = (j - 1) & i
        return f[-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the tasks of every session
# 2. Each session may also hold at most c tasks


# ============================================================
# FOLLOW-UP 1: Return the tasks of every session
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same subset DP, remembering the best subset for
# every mask.
# Time: O(3^n).  Space: O(2^n).
def session_groups(tasks, session_time):
    n = len(tasks)
    total = [0] * (1 << n)
    for mask in range(1, 1 << n):
        low = (mask & -mask).bit_length() - 1
        total[mask] = total[mask & (mask - 1)] + tasks[low]
    INF = float("inf")
    best = [INF] * (1 << n)
```

```
    pick = [0] * (1 << n)
    best[0] = 0
    for mask in range(1, 1 << n):
        sub = mask
        while sub:
            if total[sub] <= session_time and best[mask ^ sub] + 1 < best[mask]:
                best[mask] = best[mask ^ sub] + 1
                pick[mask] = sub
            sub = (sub - 1) & mask
    groups = []
    mask = (1 << n) - 1
    while mask:
        sub = pick[mask]
        groups.append([tasks[i] for i in range(n) if (sub >> i) & 1])
        mask ^= sub
    return best[(1 << n) - 1], groups


# ============================================================
# FOLLOW-UP 2: Each session may also hold at most c tasks
# ============================================================

# ---- follow-up 2: function ----
# Answer: Add the task count to the subset test.
# Time: O(3^n).  Space: O(2^n).
def min_sessions_with_limit(tasks, session_time, c):
    n = len(tasks)
    total = [0] * (1 << n)
    count = [0] * (1 << n)
    for mask in range(1, 1 << n):
        low = (mask & -mask).bit_length() - 1
        total[mask] = total[mask & (mask - 1)] + tasks[low]
        count[mask] = count[mask & (mask - 1)] + 1
    INF = float("inf")
    best = [INF] * (1 << n)
    best[0] = 0
    for mask in range(1, 1 << n):
        sub = mask
        while sub:
```

```
            if total[sub] <= session_time and count[sub] <= c:
                best[mask] = min(best[mask], best[mask ^ sub] + 1)
            sub = (sub - 1) & mask
    return best[(1 << n) - 1]
```

PART 2 · BITMASK DP · PROBLEM 3 OF 4

2305. Fair Distribution of Cookies

Medium · Bit Manipulation, Array, Dynamic Programming, Backtracking, Bitmask

leetcode.com/problems/fair-distribution-of-cookies

★ **TOP 7 PROBLEM** in Bitmask DP: master this one first

PROBLEM

You are given an integer array `cookies`, where `cookies[i]` denotes the number of cookies in the i^{th} bag. You are also given an integer `k` that denotes the number of children to distribute **all** the bags of cookies to. All the cookies in the same bag must go to the same child and cannot be split up.

The **unfairness** of a distribution is defined as the **maximum total** cookies obtained by a single child in the distribution.

Return *the **minimum** unfairness of all distributions*.

Example 1:

```
Input: cookies = [8,15,10,20,8], k = 2
Output: 31
Explanation: One optimal distribution is [8,15,8] and [10,20]
- The 1st child receives [8,15,8] which has a total of 8 + 15 + 8 = 31 cookies.
- The 2nd child receives [10,20] which has a total of 10 + 20 = 30 cookies.
The unfairness of the distribution is max(31,30) = 31.
It can be shown that there is no distribution with an unfairness less than 31.
```

Example 2:

```
Input: cookies = [6,1,3,2,2,4,1,2], k = 3
Output: 7
Explanation: One optimal distribution is [6,1], [3,2,2], and [4,1,2]
- The 1st child receives [6,1] which has a total of 6 + 1 = 7 cookies.
- The 2nd child receives [3,2,2] which has a total of 3 + 2 + 2 = 7 cookies.
- The 3rd child receives [4,1,2] which has a total of 4 + 1 + 2 = 7 cookies.
The unfairness of the distribution is max(7,7,7) = 7.
It can be shown that there is no distribution with an unfairness less than 7.
```

Constraints:

- `2 <= cookies.length <= 8`
- `1 <= cookies[i] <= 10^5`
- `2 <= k <= cookies.length`

SOLUTION (PYTHON 3)

Solution 1: Backtracking + Pruning

How to think about it

We assign n ≤ 8 bags to k ≤ 8 children and minimize the maximum load. Unconstrained assignment has k^n states, which is too large near the limits. Unfairness does not depend on bag order, but placing large bags first triggers pruning sooner. Sort bags decreasingly, then backtrack: skip a child if adding the bag is already no better than the known answer; treat equal loads on adjacent children as symmetric. After every bag is placed, update the answer with the current maximum.

First, we sort the array cookies in descending order (to reduce the number of searches), and then create an array cnt of length k to store the number of cookies each child gets. Also, we use a variable ans to maintain the current minimum degree of unfairness, initialized to a very large value.

Next, we start from the first snack pack. For the current snack pack i, we enumerate each child j. If the cookies cookies[i] in the current snack pack are given to child j, making the degree of unfairness greater than or equal to ans, or the number of cookies the current child already has is the same as the previous child, then we don't need to consider giving the cookies in the current snack pack to child j, just skip it (pruning). Otherwise, we give the cookies cookies[i] in the current snack pack to child j, and then continue to consider the next snack pack. When we have considered all the snack packs, we update the value of ans, then backtrack to the previous snack pack, and continue to enumerate which child to give the cookies in the current snack pack to.

Finally, we return ans.

Python 3

```
class Solution:
    def distributeCookies(self, cookies: List[int], k: int) -> int:
        def dfs(i):
            if i >= len(cookies):
                nonlocal ans
                ans = max(cnt)
                return
            for j in range(k):
                if cnt[j] + cookies[i] >= ans or (j and cnt[j] == cnt[j - 1]):
                    continue
                cnt[j] += cookies[i]
                dfs(i + 1)
                cnt[j] -= cookies[i]

        ans = inf
        cnt = [0] * k
        cookies.sort(reverse=True)
        dfs(0)
        return ans
```

Complexity. Time: O(k^n) in the worst case (pruned in practice). Space: O(n + k).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return how the bags are shared
# 2. Maximise the SMALLEST share instead


# ============================================================
# FOLLOW-UP 1: Return how the bags are shared
# ============================================================

# ---- follow-up 1: function ----
# Answer: Backtracking with pruning, keeping the best
# assignment.
# Time: O(k^n) in the worst case (the pruning helps in
# practice).  Space: O(n + k).
def fair_distribution(cookies, k):
    best = [float("inf"), None]
    loads = [0] * k
    owner = [0] * len(cookies)

    def go(i):
        if max(loads) >= best[0]:
            return
        if i == len(cookies):
            best[0] = max(loads)
            best[1] = list(owner)
            return
        seen = set()
        for child in range(k):
            if loads[child] in seen:
                continue
```

```
            seen.add(loads[child])
            loads[child] += cookies[i]
            owner[i] = child
            go(i + 1)
            loads[child] -= cookies[i]

    go(0)
    return best[0], best[1]


# ============================================================
# FOLLOW-UP 2: Maximise the SMALLEST share instead
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same search, comparing the minimum load.
# Time: O(k^(n + 1)).  Space: O(n + k).
def best_smallest_share(cookies, k):
    best = [0]
    loads = [0] * k

    def go(i):
        if i == len(cookies):
            best[0] = max(best[0], min(loads))
            return
        for child in range(k):
            loads[child] += cookies[i]
            go(i + 1)
            loads[child] -= cookies[i]
```

```
    go(0)
    return best[0]
```

PART 2 · BITMASK DP · PROBLEM 4 OF 4

847. Shortest Path Visiting All Nodes

Hard · Bit Manipulation, Breadth-First Search, Graph, Dynamic Programming, Bitmask

leetcode.com/problems/shortest-path-visiting-all-nodes

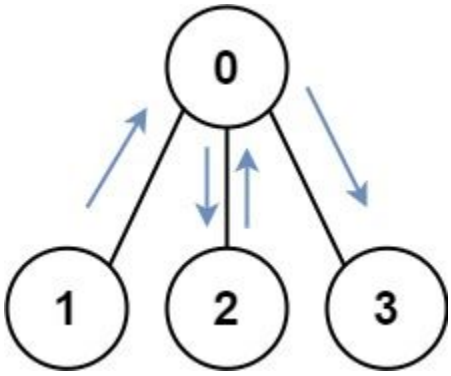
★ **TOP 7 PROBLEM** in Bitmask DP: master this one first

PROBLEM

You have an undirected, connected graph of `n` nodes labeled from `0` to `n - 1`. You are given an array `graph` where `graph[i]` is a list of all the nodes connected with node `i` by an edge.

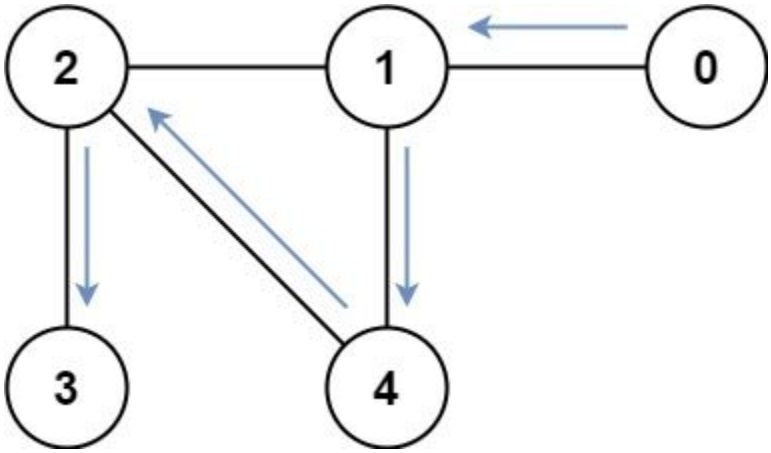
Return *the length of the shortest path that visits every node*. You may start and stop at any node, you may revisit nodes multiple times, and you may reuse edges.

Example 1:




```
Input: graph = [[1,2,3],[0],[0],[0]]
Output: 4
Explanation: One possible path is [1,0,2,0,3]
```

Example 2:




```
Input: graph = [[1],[0,2,4],[1,3,4],[2],[1,2]]
Output: 4
Explanation: One possible path is [0,1,4,2,3]
```

Constraints:

- `n == graph.length`
- `1 <= n <= 12`
- `0 <= graph[i].length < n`

- `graph[i]` does not contain `i`.
- If `graph[a]` contains `b`, then `graph[b]` contains `a`.
- The input graph is always connected.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

We want a shortest walk that visits every node; edges may be reused. n≤ 12, so the walk itself is not the state, but (current node, visited mask) has only n· 2^n pairs. Multi-source BFS from every start on that state space: the first time the mask is full, the layer index is the answer.

Python 3

```
class Solution:
    def shortestPathLength(self, graph: List[List[int]]) -> int:
        n = len(graph)
        q = deque()
        vis = set()
        for i in range(n):
            q.append((i, 1 << i))
            vis.add((i, 1 << i))
        ans = 0
        while 1:
            for _ in range(len(q)):
                i, st = q.popleft()
                if st == (1 << n) - 1:
                    return ans
                for j in graph[i]:
                    nst = st | 1 << j
                    if (j, nst) not in vis:
                        vis.add((j, nst))
                        q.append((j, nst))
            ans += 1
```

Complexity. Time: O(2^n * E), at most O(2^n * n^2). Space: O(n * 2^n).

Solution 2

How to think about it

Plain BFS expands by layer. An admissible heuristic — the number of still-unvisited nodes — lets a priority queue pop dist+h first. Relaxation is still unit-weight shortest paths, so optimality is kept; full masks tend to appear earlier once most nodes are visited.

Python 3

```python
class Solution:
    def shortestPathLength(self, graph: List[List[int]]) -> int:
        n = len(graph)

        def f(state):
            return sum(((state >> i) & 1) == 0 for i in range(n))

        q = []
        dist = [[inf] * (1 << n) for _ in range(n)]
        for i in range(n):
            heappush(q, (f(1 << i), i, 1 << i))
            dist[i][1 << i] = 0
        while q:
            _, u, state = heappop(q)
            if state == (1 << n) - 1:
                return dist[u][state]
            for v in graph[u]:
                nxt = state | (1 << v)
                if dist[v][nxt] > dist[u][state] + 1:
                    dist[v][nxt] = dist[u][state] + 1
                    heappush(q, (dist[v][nxt] + f(nxt), v, nxt))
        return 0
```

Complexity. Time: O(2^n * E * log), at most O(2^n * n^2 * log); the heuristic only reorders the search. Space: O(n * 2^n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the path
# 2. The walk must end where it started (a closed tour)


# ============================================================
# FOLLOW-UP 1: Return the path
# ============================================================

# ---- follow-up 1: function ----
# Answer: Breadth-first search over (node, visited mask)
# remembering the previous state.
# Time: O(2^n * (n + E)).  Space: O(2^n * n).
from collections import deque


def shortest_visiting_path(graph):
    n = len(graph)
    full = (1 << n) - 1
    previous = {}
    queue = deque()
    for i in range(n):
        state = (i, 1 << i)
        previous[state] = None
        queue.append(state)
    while queue:
        node, mask = queue.popleft()
        if mask == full:
            path = []
            state = (node, mask)
            while state is not None:
                path.append(state[0])
                state = previous[state]
            return path[::-1]
        for nxt in graph[node]:
            state = (nxt, mask | (1 << nxt))
            if state not in previous:
                previous[state] = (node, mask)
                queue.append(state)
```

```
    return None


# ============================================================
# FOLLOW-UP 2: The walk must end where it started (a
# closed tour)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Start from one node only and finish at the same
# node with all nodes visited.
# Time: O(2^n * (n + E)).  Space: O(2^n * n).
def shortest_closed_tour(graph, start):
    n = len(graph)
    full = (1 << n) - 1
    distance = {(start, 1 << start): 0}
    queue = deque([(start, 1 << start)])
    best = None
    while queue:
        node, mask = queue.popleft()
        d = distance[(node, mask)]
```

```
        for nxt in graph[node]:
            if nxt == start and mask == full:
                if best is None or d + 1 < best:
                    best = d + 1
            state = (nxt, mask | (1 << nxt))
            if state not in distance:
                distance[state] = d + 1
                queue.append(state)
    return best
```

PART 2 · TOPIC 44 OF 50

Digit DP

3 problems · 0 Easy · 1 Medium · 2 Hard

THE IDEA

Build numbers digit by digit with a state (tight to the limit or free, whether it has started, what has been used) and memoise.

THE 3 PROBLEMS, EASY TO HARD | ★ = TOP 3: DO THESE FIRST				
1	★ **Count Numbers with Unique Digits**	Medium	LC 357	2708
2	★ **Number of Digit One**	Hard	LC 233	2711
3	★ **Numbers At Most N Given Digit Set**	Hard	LC 902	2714

PART 2 · DIGIT DP · PROBLEM 1 OF 3

357. Count Numbers with Unique Digits

Medium · Math, Dynamic Programming, Backtracking

leetcode.com/problems/count-numbers-with-unique-digits

★ **TOP 7 PROBLEM** in Digit DP: master this one first

PROBLEM

Given an integer `n`, return the count of all numbers with unique digits, `x`, where `0 <= x < 10^n`.

Example 1:

```
Input: n = 2
Output: 91
Explanation: The answer should be the total numbers in the range of 0 ≤ x < 100, excluding
11,22,33,44,55,66,77,88,99
```

Example 2:

```
Input: n = 0
Output: 1
```

Constraints:

- `0 <= n <= 8`

SOLUTION (PYTHON 3)

Solution 1: State Compression + Digit DP

How to think about it

Count integers with at most n digits and all distinct digits (including 0). Brute force still works for n=8, but digit DP handles leading zeros and used digits uniformly. dfs(i,mask,lead) is the remaining i+1 positions, the used-digit mask, and whether we are still leading zeros. A leading zero does not occupy a digit; otherwise j enters the mask. Start at i=n-1; the memoized state space is small.

This problem essentially asks for the number of numbers in the given range [l, ..r] that satisfy certain conditions. The conditions are related to the composition of the numbers rather than their size, so we can use the concept of Digit DP to solve it. In Digit DP, the size of the number has little impact on the complexity.

For the range [l, ..r] problem, we generally convert it to the problem of [1, ..r] and then subtract the result of [1, ..l - 1], i.e.:

$$ans = \sum_{i=1}^{r} ans_i - \sum_{i=1}^{l-1} ans_i$$

However, for this problem, we only need to find the value for the range $[1, ..10^n-1]$.

Here, we use memoized search to implement Digit DP. We search from the starting point downwards, and at the lowest level, we get the number of solutions. We then return the answers layer by layer upwards, and finally get the final answer from the starting point of the search.

Based on the problem information, we design a function dfs(i, mask, lead), where:

- The digit i represents the current position being searched, starting from the highest digit, i.e., i = 0 represents the highest digit.
- The digit mask represents the current state of the number, i.e., the j-th bit of mask being 1 indicates that the digit j has been used.
- The boolean lead indicates whether the current number only contains leading 0s.

The function executes as follows:

If i exceeds the length of the number n, i.e., i < 0, it means the search is over, directly return 1.

Otherwise, we enumerate the digits j from 0 to 9 for the position i. For each j:

- If the j-th bit of mask is 1, it means the digit j has been used, so we skip it.
- If lead is true and j = 0, it means the current number only contains leading 0s. When we recurse to the next level, lead remains true.
- Otherwise, we recurse to the next level, update the j-th bit of mask to 1, and set lead to false.

Finally, we sum all the results from the recursive calls to the next level, which is the answer.

The answer is dfs(n - 1, 0, True).

The time complexity is $O(n \times 2^D \times D)$, and the space complexity is $O(n \times 2^D)$. Here, n is the length of the number n, and D = 10.

Similar Problems:

- 233. Number of Digit One
- 600. Non-negative Integers without Consecutive Ones
- 788. Rotated Digits
- 902. Numbers At Most N Given Digit Set
- 1012. Numbers with Repeated Digits
- 2376. Count Special Integers

Python 3

```python
class Solution:
    def countNumbersWithUniqueDigits(self, n: int) -> int:
        @cache
        def dfs(i: int, mask: int, lead: bool) -> int:
            if i < 0:
                return 1
            ans = 0
            for j in range(10):
                if mask >> j & 1:
                    continue
                if lead and j == 0:
                    ans += dfs(i - 1, mask, True)
                else:
                    ans += dfs(i - 1, mask | 1 << j, False)
            return ans

        return dfs(n - 1, 0, True)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count the numbers from lo to hi (any range) that have
# unique digits
# 2. List the numbers below 10^n with unique digits


# ============================================================
# FOLLOW-UP 1: Count the numbers from lo to hi (any range)
# that have unique digits
# ============================================================

# ---- follow-up 1: function ----
# Answer: Digit DP: count up to hi minus count up to lo -
# 1.
# Time: O(L^2), L = digits of the limit.  Space: O(L).
def count_unique_up_to(limit):
    if limit < 0:
        return 0
    if limit == 0:
        return 1
```

```
    digits = list(map(int, str(limit)))
    n = len(digits)
    total = 1                                       # the number 0
    for length in range(1, n):                      # numbers with fewer digits
        ways = 9
        for k in range(1, length):
            ways *= 10 - k
        total += ways
    used = set()
    for i, top in enumerate(digits):                # numbers with n digits that are below the limit
        for d in range(1 if i == 0 else 0, top):
            if d in used:
                continue
            free = 10 - len(used) - 1
            rest = 1
            for t in range(n - i - 1):
                rest *= free - t
            total += rest
        if top in used:
            return total
        used.add(top)
    return total + 1                                # the limit itself has unique digits


def count_unique_in_range(lo, hi):
    return count_unique_up_to(hi) - count_unique_up_to(lo - 1)


# ============================================================
# FOLLOW-UP 2: List the numbers below 10^n with unique
# digits
# ============================================================

# ---- follow-up 2: function ----
# Answer: Build the numbers digit by digit, never reusing
# a digit.
# Time: O(output size).  Space: O(output size).
def list_unique_numbers(n):
    result = [0]
```

```
    def go(value, used, length):
        if length == n:
            return
        for d in range(10):
            if (used >> d) & 1 or (length == 0 and d == 0):
                continue
            result.append(value * 10 + d)
            go(value * 10 + d, used | (1 << d), length + 1)

    go(0, 0, 0)
    return sorted(result)
```

233. Number of Digit One

Hard · Recursion, Math, Dynamic Programming

leetcode.com/problems/number-of-digit-one

★ **TOP 7 PROBLEM** in Digit DP: master this one first

PROBLEM

Given an integer `n`, count *the total number of digit* `1` *appearing in all non-negative integers less than or equal to* `n`.

Example 1:

```
Input: n = 13
Output: 6
```

Example 2:

```
Input: n = 0
Output: 0
```

Constraints:

- `0 <= n <= 10^9`

SOLUTION (PYTHON 3)

Solution 1: Digit DP

How to think about it

Scanning every integer in [1,n] is impossible for large n. The count of digit 1 depends only on the digits and the upper bound, which is a digit-DP setting. Treat n as a string and memoize dfs(i,cnt,limit): position i from the high end, cnt ones so far, and whether we still match the prefix of n. Enumerate the current digit and add the results.

This problem essentially asks for the number of times the digit 1 appears in the given range [l, ..r]. The count is related to the number of digits and the value of each digit. We can use the concept of Digit DP to solve this problem. In Digit DP, the size of the number has little impact on the complexity.

For the range [l, ..r] problem, we generally convert it to the problem of [1, ..r] and then subtract the result of [1, ..l - 1], i.e.:

$$\text{ans} = \sum_{i=1}^{r} \text{ans}_i - \sum_{i=1}^{l-1} \text{ans}_i$$

However, for this problem, we only need to find the value for the range [1, ..r].

Here, we use memoized search to implement Digit DP. We search from the starting point downwards, and at the lowest level, we get the number of solutions. We then return the answers layer by layer upwards, and finally get the final answer from the starting point of the search.

The basic steps are as follows:

First, we convert the number n to a string s. Then we design a function dfs(i, cnt, limit), where:

- The digit i represents the current position being searched, starting from the highest digit, i.e., i = 0 represents the highest digit.
- The digit cnt represents the current count of the digit 1 in the number.
- The boolean limit indicates whether the current number is restricted by the upper bound.

The function executes as follows:

If i exceeds the length of the number n, it means the search is over, directly return cnt. If limit is true, up is the i-th digit of the current number. Otherwise, up = 9. Next, we iterate j from 0 to up. For each j:

- If j equals 1, we increment cnt by one.

- Recursively call dfs(i + 1, cnt, limit ∧ j = up).

The answer is dfs(0, 0, True).

The time complexity is $O(m^2 \times D)$, and the space complexity is $O(m^2)$. Here, m is the length of the number n, and D = 10.

Similar Problems:

Here is the translation of the similar problems into English:

- 357. Count Numbers with Unique Digits
- 600. Non-negative Integers without Consecutive Ones
- 788. Rotated Digits
- 902. Numbers At Most N Given Digit Set
- 1012. Numbers with Repeated Digits
- 2376. Count Special Integers

Python 3

```python
class Solution:
    def countDigitOne(self, n: int) -> int:
        @cache
        def dfs(i: int, cnt: int, limit: bool) -> int:
            if i >= len(s):
                return cnt
            up = int(s[i]) if limit else 9
            ans = 0
            for j in range(up + 1):
                ans += dfs(i + 1, cnt + (j == 1), limit and j == up)
            return ans

        s = str(n)
        return dfs(0, 0, True)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count any digit d (0 to 9) in the numbers 1..n
# 2. Count the digit 1 in the numbers from a to b


# ============================================================
# FOLLOW-UP 1: Count any digit d (0 to 9) in the numbers
# 1..n
# ============================================================

# ---- follow-up 1: function ----
# Answer: For every position, count how often the digit
# appears there (higher part, current digit, lower part).
# Time: O(log n).  Space: O(1).
def count_digit(n, d):
    if n <= 0:
        return 0
    count = 0
    place = 1
    while place <= n:
        higher = n // (place * 10)
        current = (n // place) % 10
        lower = n % place
        if d == 0:
            if higher > 0:
                count += (higher - 1) * place
                if current > 0:
                    count += place
                else:
                    count += lower + 1
```

```
        else:
            count += higher * place
            if current > d:
                count += place
            elif current == d:
                count += lower + 1
        place *= 10
    return count


# ============================================================
# FOLLOW-UP 2: Count the digit 1 in the numbers from a to
# b
# ============================================================

# ---- follow-up 2: function ----
# Answer: The count up to b minus the count up to a - 1.
# Time: O(log b).  Space: O(1).
def count_ones_in_range(a, b):
    return count_digit(b, 1) - count_digit(a - 1, 1)
```

PART 2 · DIGIT DP · PROBLEM 3 OF 3

902. Numbers At Most N Given Digit Set

Hard · Array, Math, String, Binary Search, Dynamic Programming

leetcode.com/problems/numbers-at-most-n-given-digit-set

★ **TOP 7 PROBLEM** in Digit DP: master this one first

PROBLEM

Given an array of `digits` which is sorted in **non-decreasing** order. You can write numbers using each `digits[i]` as many times as we want. For example, if `digits = ['1','3','5']`, we may write numbers such as `'13'`, `'551'`, and `'1351315'`.

Return *the number of positive integers that can be generated* that are less than or equal to a given integer `n`.

Example 1:

```
Input: digits = ["1","3","5","7"], n = 100
Output: 20
Explanation:
The 20 numbers that can be written are:
1, 3, 5, 7, 11, 13, 15, 17, 31, 33, 35, 37, 51, 53, 55, 57, 71, 73, 75, 77.
```

Example 2:

```
Input: digits = ["1","4","9"], n = 1000000000
Output: 29523
Explanation:
We can write 3 one digit numbers, 9 two digit numbers, 27 three digit numbers,
81 four digit numbers, 243 five digit numbers, 729 six digit numbers,
2187 seven digit numbers, 6561 eight digit numbers, and 19683 nine digit numbers.
In total, this is 29523 integers that can be written using the digits array.
```

Example 3:

```
Input: digits = ["7"], n = 8
Output: 1
```

Constraints:

- `1 <= digits.length <= 9`
- `digits[i].length == 1`
- `digits[i]` is a digit from `'1'` to `'9'`.
- All the values in `digits` are **unique**.
- `digits` is sorted in **non-decreasing** order.
- `1 <= n <= 10^9`

SOLUTION (PYTHON 3)

Solution 1: Digit DP

How to think about it

Building every number from digits and testing x ≤ n re-expands the same prefixes near the decimal length of n. The count depends only on the position, whether we still have leading zeros, and whether we are tight against the upper bound. Write n as s and memoize dfs(i, lead, limit): a leading-zero state may still place 0, otherwise only digits from digits are allowed, clipped by limit. A finished number counts as 1 only if it is not all leading zeros.

This problem essentially asks for the number of positive integers that can be generated from the digits in digits within the given range [l, .., r]. The count depends on the number of digits and the value of each digit. We can solve this problem using

the Digit DP approach. In Digit DP, the size of the number has little impact on the complexity.

For the range [l, .., r] problem, we generally convert it to the problem of [1, .., r] and then subtract the result of [1, .., l - 1], i.e.,

$$ans = \sum_{i=1}^{r} ans_i - \sum_{i=1}^{l-1} ans_i$$

However, for this problem, we only need to find the value for the range [1, .., r].

Here, we use memoization to implement Digit DP. We start searching from the top, get the number of solutions at the bottom, and return the answers layer by layer until we get the final answer from the starting point.

The basic steps are as follows:

We convert the number n into a string s and denote the length of the string s as m.

Next, we design a function dfs(i, lead, limit) to represent the number of solutions from the current i-th digit to the last digit of the string. Here:

- The integer i represents the current position in the string s.
- The boolean lead indicates whether the current number contains only leading zeros.
- The boolean limit indicates whether the current position is restricted by the upper bound.

The function executes as follows:

If i is greater than or equal to m, it means we have processed all digits. If lead is true, it means the current number is a leading zero, and we should return 0. Otherwise, we should return 1.

Otherwise, we calculate the upper bound up. If limit is true, then up is the digit corresponding to s[i]. Otherwise, up is 9.

Then, we enumerate the current digit j in the range [0, up]. If j is 0 and lead is true, we recursively calculate dfs(i + 1, true, limit ∧ j = up). Otherwise, if j is in digits, we recursively calculate dfs(i + 1, false, limit ∧ j = up). We accumulate all the results as the answer.

Finally, we return dfs(0, true, true).

The time complexity is O(log n × D), and the space complexity is O(log n). Here, D = 10.

Similar problems:

- 233. Number of Digit One
- 357. Count Numbers with Unique Digits
- 600. Non-negative Integers without Consecutive Ones
- 788. Rotated Digits
- 1012. Numbers With Repeated Digits
- 2376. Count Special Integers

Python 3

```
class Solution:
    def atMostNGivenDigitSet(self, digits: List[str], n: int) -> int:
        @cache
        def dfs(i: int, lead: int, limit: bool) -> int:
            if i >= len(s):
                return lead ^ 1

            up = int(s[i]) if limit else 9
            ans = 0
            for j in range(up + 1):
                if j == 0 and lead:
                    ans += dfs(i + 1, 1, limit and j == up)
                elif j in nums:
                    ans += dfs(i + 1, 0, limit and j == up)
            return ans

        s = str(n)
        nums = {int(x) for x in digits}
        return dfs(0, 1, True)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. How many of them have exactly L digits
# 2. The k-th smallest number that can be written with the
# digits


# ============================================================
# FOLLOW-UP 1: How many of them have exactly L digits
# ============================================================

# ---- follow-up 1: function ----
# Answer: len(digits) ** L (any digit allowed in any
# position).
# Time: O(L).  Space: O(1).
def count_with_exact_length(digits, length):
    return len(digits) ** length


# ============================================================
# FOLLOW-UP 2: The k-th smallest number that can be
# written with the digits
# ============================================================

# ---- follow-up 2: function ----
# Answer: Write k - 1 in base len(digits) (bijective),
# using the sorted digits as the alphabet.
# Time: O(log k).  Space: O(log k).
def kth_smallest_number(digits, k):
    digits = sorted(digits)
    base = len(digits)
    out = []
    while k > 0:
        k -= 1
        out.append(digits[k % base])
        k //= base
    return "".join(reversed(out))
```

PART 2 · TOPIC 45 OF 50

Probability DP

3 problems · 0 Easy · 3 Medium · 0 Hard

THE IDEA

dp[state] is a probability or an expected value, built from the probabilities of the moves out of it.

THE 3 PROBLEMS, EASY TO HARD | ★ = TOP 3: DO THESE FIRST				
1	★ **Knight Probability in Chessboard**	Medium	LC 688	2718
2	★ **Soup Servings**	Medium	LC 808	2721
3	★ **New 21 Game**	Medium	LC 837	2724

688. Knight Probability in Chessboard

Medium · Dynamic Programming

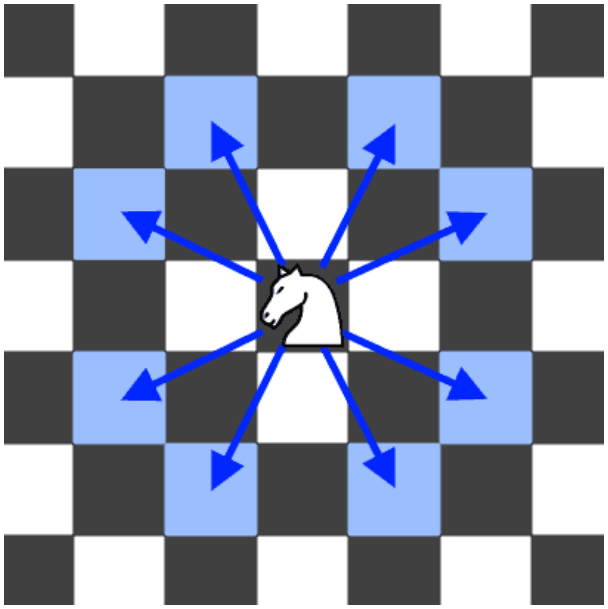
leetcode.com/problems/knight-probability-in-chessboard

★ **TOP 7 PROBLEM** in Probability DP: master this one first

PROBLEM

On an `n x n` chessboard, a knight starts at the cell `(row, column)` and attempts to make exactly `k` moves. The rows and columns are **0-indexed**, so the top-left cell is `(0, 0)`, and the bottom-right cell is `(n - 1, n - 1)`.

A chess knight has eight possible moves it can make, as illustrated below. Each move is two cells in a cardinal direction, then one cell in an orthogonal direction.



Each time the knight is to move, it chooses one of eight possible moves uniformly at random (even if the piece would go off the chessboard) and moves there.

The knight continues moving until it has made exactly `k` moves or has moved off the chessboard.

Return *the probability that the knight remains on the board after it has stopped moving*.

Example 1:

```
Input: n = 3, k = 2, row = 0, column = 0
Output: 0.06250
Explanation: There are two moves (to (1,2), (2,1)) that will keep the knight on the board.
From each of those positions, there are also two moves that will keep the knight on the board.
The total probability the knight stays on the board is 0.0625.
```

Example 2:

```
Input: n = 1, k = 0, row = 0, column = 0
Output: 1.00000
```

Constraints:

- `1 <= n <= 25`
- `0 <= k <= 100`
- `0 <= row, column <= n - 1`

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

> **How to think about it**
>
> The probability a knight stays on an n× n board after k moves. An 8^k search is impossible for k≤ 100. f[h][i][j] is the chance of staying after h more moves from (i,j). Base h=0 is 1; otherwise average f[h-1] over the eight in-bound landings.

We define f[h][i][j] to represent the probability that the knight remains on the board after taking h steps starting from position (i, j). The final answer is f[k][row][column].

When h=0, the knight is definitely on the board, so the probability is 1, i.e., f[0][i][j]=1.

When h > 0, the probability that the knight is at position (i, j) can be derived from the probabilities of the 8 possible positions it could have come from in the previous step, i.e.,

$$f[h][i][j] = \sum_{x,\, y} f[h - 1][x][y] \times 1/8$$

where (x, y) is one of the 8 positions the knight can move to from (i, j).

The final answer is f[k][row][column].

The time complexity is $O(k \times n^2)$, and the space complexity is $O(k \times n^2)$. Here, k and n are the given number of steps and the size of the board, respectively.

Python 3

```python
class Solution:
    def knightProbability(self, n: int, k: int, row: int, column: int) -> float:
        f = [[[0] * n for _ in range(n)] for _ in range(k + 1)]
        for i in range(n):
            for j in range(n):
                f[0][i][j] = 1
        for h in range(1, k + 1):
            for i in range(n):
                for j in range(n):
                    for a, b in pairwise((-2, -1, 2, 1, -2, 1, 2, -1, -2)):
                        x, y = i + a, j + b
                        if 0 <= x < n and 0 <= y < n:
                            f[h][i][j] += f[h - 1][x][y] / 8
        return f[k][row][column]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The probability that the knight is on a given square
# after k moves
# 2. The probability that the knight falls off the board
# at some point in k moves


# ============================================================
# FOLLOW-UP 1: The probability that the knight is on a
# given square after k moves
# ============================================================

# ---- follow-up 1: function ----
# Answer: A table of probabilities for every square after
# every move, without discarding the ones that fall off.
# Time: O(k * n^2).  Space: O(n^2).
def probability_on_square(n, k, row, column, target_row, target_column):
    moves = ((1, 2), (2, 1), (-1, 2), (-2, 1), (1, -2), (2, -1), (-1, -2), (-2, -1))
    current = [[0.0] * n for _ in range(n)]
    current[row][column] = 1.0
```

```
    for _ in range(k):
        new = [[0.0] * n for _ in range(n)]
        for r in range(n):
            for c in range(n):
                if current[r][c]:
                    for dr, dc in moves:
                        nr, nc = r + dr, c + dc
                        if 0 <= nr < n and 0 <= nc < n:
                            new[nr][nc] += current[r][c] / 8
        current = new
    return current[target_row][target_column]


# ============================================================
# FOLLOW-UP 2: The probability that the knight falls off
# the board at some point in k moves
# ============================================================

# ---- follow-up 2: function ----
# Answer: One minus the probability to stay on the board.
# Time: O(k * n^2).  Space: O(n^2).
def probability_falls_off(n, k, row, column):
    moves = ((1, 2), (2, 1), (-1, 2), (-2, 1), (1, -2), (2, -1), (-1, -2), (-2, -1))
    current = [[0.0] * n for _ in range(n)]
    current[row][column] = 1.0
    for _ in range(k):
        new = [[0.0] * n for _ in range(n)]
        for r in range(n):
            for c in range(n):
                if current[r][c]:
                    for dr, dc in moves:
                        nr, nc = r + dr, c + dc
                        if 0 <= nr < n and 0 <= nc < n:
                            new[nr][nc] += current[r][c] / 8
        current = new
    return 1 - sum(sum(r) for r in current)
```

808. Soup Servings

Medium · Math, Dynamic Programming, Probability and Statistics

leetcode.com/problems/soup-servings

★ **TOP 7 PROBLEM** in Probability DP: master this one first

PROBLEM

You have two soups, **A** and **B**, each starting with `n` mL. On every turn, one of the following four serving operations is chosen *at random*, each with probability `0.25` **independent** of all previous turns:

- pour 100 mL from type A and 0 mL from type B
- pour 75 mL from type A and 25 mL from type B
- pour 50 mL from type A and 50 mL from type B
- pour 25 mL from type A and 75 mL from type B

Note:

- There is no operation that pours 0 mL from A and 100 mL from B.
- The amounts from A and B are poured *simultaneously* during the turn.
- If an operation asks you to pour **more than** you have left of a soup, pour all that remains of that soup.

The process stops immediately after any turn in which *one of the soups* is used up.

Return the probability that A is used up *before* B, plus half the probability that both soups are used up in the **same turn**. Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

```
Input: n = 50
Output: 0.62500
Explanation:
If we perform either of the first two serving operations, soup A will become empty first.
If we perform the third operation, A and B will become empty at the same time.
If we perform the fourth operation, B will become empty first.
So the total probability of A becoming empty first plus half the probability that A and B become empty
at the same time, is 0.25 * (1 + 1 + 0.5 + 0) = 0.625.
```

Example 2:

```
Input: n = 100
Output: 0.71875
Explanation:
If we perform the first serving operation, soup A will become empty first.
If we perform the second serving operations, A will become empty on performing operation [1, 2, 3],
and both A and B become empty on performing operation 4.
If we perform the third operation, A will become empty on performing operation [1, 2], and both A and B
become empty on performing operation 3.
If we perform the fourth operation, A will become empty on performing operation 1, and both A and B
become empty on performing operation 2.
So the total probability of A becoming empty first plus half the probability that A and B become empty
at the same time, is 0.71875.
```

Constraints:

- $0 <= n <= 10^9$

SOLUTION (PYTHON 3)

Solution 1: Memoization Search

How to think about it

Four operations deplete the soups with equal probability; we want the probability that A empties first (ties count as 1/2). n can be 10^9, so a millilitre-level simulation is impossible. Each pour removes at least 25 ml, so states shrink to ceil(n/25). dfs(i,j) is the probability with i and j remaining units, with base cases 1, 0, or 1/2. For large n, A almost surely finishes first, so the code returns 1 when n>4800.

In this problem, since each operation is a multiple of 25, we can consider every 25ml of soup as one unit. This reduces the data scale to ceil(n/25).

We design a function dfs(i, j), which represents the probability result when there are i units of soup A and j units of soup B remaining.

When i ≤ 0 and j ≤ 0, it means both soups are finished, and we should return 0.5. When i ≤ 0, it means soup A is finished first, and we should return 1. When j ≤ 0, it means soup B is finished first, and we should return 0.

Next, for each operation, we have four choices:

- Take 4 units from soup A and 0 units from soup B;
- Take 3 units from soup A and 1 unit from soup B;
- Take 2 units from soup A and 2 units from soup B;
- Take 1 unit from soup A and 3 units from soup B.

Each choice has a probability of 0.25, so we can derive:

$$dfs(i, j) = 0.25 \times (dfs(i - 4, j) + dfs(i - 3, j - 1) + dfs(i - 2, j - 2) + dfs(i - 1, j - 3))$$

We use memoization to store the results of the function.

Additionally, we find that when n=4800, the result is 0.999994994426, and the required precision is 10^{-5}. As n increases, the result gets closer to 1. Therefore, when n > 4800, we can directly return 1.

The time complexity is $O(C^2)$, and the space complexity is $O(C^2)$. In this problem, C=200.

Python 3

```python
class Solution:
    def soupServings(self, n: int) -> float:
        @cache
        def dfs(i: int, j: int) -> float:
            if i <= 0 and j <= 0:
                return 0.5
            if i <= 0:
                return 1
            if j <= 0:
                return 0
            return 0.25 * (
                dfs(i - 4, j)
                + dfs(i - 3, j - 1)
                + dfs(i - 2, j - 2)
                + dfs(i - 1, j - 3)
            )

        return 1 if n > 4800 else dfs((n + 24) // 25, (n + 24) // 25)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The probability that both soups are used up in the
# same turn
# 2. The expected number of turns until one soup is used
# up


# ============================================================
# FOLLOW-UP 1: The probability that both soups are used up
# in the same turn
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same memoised search, returning only the
# probability of the tie.
# Time: O(1), because n above 5000 is answered at once and
# the table is at most 200 by 200.  Space: O(1).
def both_empty_probability(n):
    if n > 5000:
        return 0.0
    units = (n + 24) // 25
    memo = {}

    def go(a, b):
        if a <= 0 and b <= 0:
            return 1.0
        if a <= 0 or b <= 0:
            return 0.0
        if (a, b) in memo:
            return memo[(a, b)]
        value = 0.25 * (go(a - 4, b) + go(a - 3, b - 1) + go(a - 2, b - 2) + go(a - 1, b - 3))
        memo[(a, b)] = value
        return value

    return go(units, units)


# ============================================================
# FOLLOW-UP 2: The expected number of turns until one soup
# is used up
# ============================================================

# ---- follow-up 2: function ----
# Answer: Expected value: one turn plus the average of the
# expected values of the four results.
# Time: O((n / 25)^2).  Space: O((n / 25)^2).
def expected_turns(n):
    units = (n + 24) // 25
    memo = {}

    def go(a, b):
        if a <= 0 or b <= 0:
            return 0.0
        if (a, b) in memo:
            return memo[(a, b)]
        value = 1 + 0.25 * (go(a - 4, b) + go(a - 3, b - 1) + go(a - 2, b - 2) + go(a - 1, b - 3))
        memo[(a, b)] = value
        return value

    return go(units, units)
```

837. New 21 Game

Medium · Math, Dynamic Programming, Sliding Window, Probability and Statistics

leetcode.com/problems/new-21-game

★ **TOP 7 PROBLEM** in Probability DP: master this one first

PROBLEM

Alice plays the following game, loosely based on the card game **"21"**.

Alice starts with `0` points and draws numbers while she has less than `k` points. During each draw, she gains an integer number of points randomly from the range `[1, maxPts]`, where `maxPts` is an integer. Each draw is independent and the outcomes have equal probabilities.

Alice stops drawing numbers when she gets `k` **or more points**.

Return the probability that Alice has `n` or fewer points.

Answers within 10^{-5} of the actual answer are considered accepted.

Example 1:

```
Input: n = 10, k = 1, maxPts = 10
Output: 1.00000
Explanation: Alice gets a single card, then stops.
```

Example 2:

```
Input: n = 6, k = 1, maxPts = 10
Output: 0.60000
Explanation: Alice gets a single card, then stops.
In 6 out of 10 possibilities, she is at or below 6 points.
```

Example 3:

```
Input: n = 21, k = 17, maxPts = 10
Output: 0.73278
```

Constraints:

- `0 <= k <= n <=` 10^4
- `1 <= maxPts <=` 10^4

SOLUTIONS (PYTHON 3)

Solution 1: Memoized Search

How to think about it

We add a uniform integer in [1,maxPts] until the total reaches k, and want the probability it is still ≤ n. Expanding every draw is impossible for k,n≤ 10^4; memoize on the current total. dfs(i) is the success probability from total i. For i≥ k we stop and test i≤ n; i=k-1 has a closed form. Other states use a difference of neighbors so each transition is O(1) instead of O(maxPts).

We design a function dfs(i), which represents the probability that when the current score is i, the final score does not exceed n when we stop drawing numbers. The answer is dfs(0).

The calculation method of function dfs(i) is as follows:

- If i ≥ k, then we stop drawing numbers. If i ≤ n, return 1, otherwise return 0;
- Otherwise, we can draw the next number j in the range [1,..maxPts], then dfs(i) = 1/maxPts $\sum_{j=1}^{maxPts}$ dfs(i+j).

Here we can use memoized search to accelerate the calculation.

The time complexity of the above method is O(k × maxPts), which will exceed the time limit, so we need to optimize it.

When i < k, the following equation holds:

$$\text{dfs}(i) = (\text{dfs}(i+1) + \text{dfs}(i+2) + \ldots + \text{dfs}(i+\text{maxPts})) / \text{maxPts} \quad (1)$$

When i < k - 1, the following equation holds:

$$\text{dfs}(i+1) = (\text{dfs}(i+2) + \text{dfs}(i+3) + \ldots + \text{dfs}(i+\text{maxPts}+1)) / \text{maxPts} \quad (2)$$

Therefore, when i < k-1, we subtract equation (2) from equation (1) to get:

$$\text{dfs}(i) - \text{dfs}(i+1) = (\text{dfs}(i+1) - \text{dfs}(i+\text{maxPts}+1)) / \text{maxPts}$$

That is:

$$\text{dfs}(i) = \text{dfs}(i+1) + (\text{dfs}(i+1) - \text{dfs}(i+\text{maxPts}+1)) / \text{maxPts}$$

If i=k-1, we have:

$$\text{dfs}(i) = \text{dfs}(k-1) = (\text{dfs}(k) + \text{dfs}(k+1) + \ldots + \text{dfs}(k+\text{maxPts}-1)) / \text{maxPts} \quad (3)$$

We assume there are i numbers not exceeding n, then k+i-1 ≤ n, and since i≤ maxPts, we have i ≤ min(n-k+1, maxPts), so equation (3) can be written as:

$$\text{dfs}(k-1) = \min(n-k+1, \text{maxPts}) / \text{maxPts}$$

In summary, we have the following state transition equation:

$$\text{dfs}(i) = \begin{cases} 1, & i \geq k, i \leq n \\ 0, & i \geq k, i > n \\ \min(n-k+1, \text{maxPts}) / \text{maxPts}, & i = k-1 \\ \text{dfs}(i+1) + (\text{dfs}(i+1) - \text{dfs}(i+\text{maxPts}+1)) / \text{maxPts}, & i < k-1 \end{cases}$$

Time complexity O(k + maxPts), space complexity O(k + maxPts). Where k is the maximum score.

Python 3

```python
class Solution:
    def new21Game(self, n: int, k: int, maxPts: int) -> float:
        @cache
        def dfs(i: int) -> float:
            if i >= k:
                return int(i <= n)
            if i == k - 1:
                return min(n - k + 1, maxPts) / maxPts
            return dfs(i + 1) + (dfs(i + 1) - dfs(i + maxPts + 1)) / maxPts

        return dfs(0)
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> Solution 1 already fills the probabilities from the high scores downward. Each f[i] comes from f[i+1] and the window end f[i+maxPts+1]. Terminals in [k,min(n,k+maxPts)) are 1. The answer is f[0]. Time stays linear.

Solution 1 already fills this table from the high scores downward. The transitions below are the same ones.

Define f[i] to represent the probability that when the current score is i, the final score does not exceed n when we stop drawing numbers. The answer is f[0].

When k ≤ i ≤ min(n, k + maxPts - 1), we have f[i] = 1.

When i = k - 1, we have f[i] = min(n-k+1, maxPts) / maxPts.

When i < k - 1, we have f[i] = f[i + 1] + (f[i + 1] - f[i + maxPts + 1]) / maxPts.

Time complexity O(k + maxPts), space complexity O(k + maxPts). Where k is the maximum score.

Python 3

```python
class Solution:
    def new21Game(self, n: int, k: int, maxPts: int) -> float:
        f = [0] * (k + maxPts)
        for i in range(k, min(n + 1, k + maxPts)):
            f[i] = 1
        f[k - 1] = min(n - k + 1, maxPts) / maxPts
        for i in range(k - 2, -1, -1):
            f[i] = f[i + 1] + (f[i + 1] - f[i + maxPts + 1]) / maxPts
        return f[0]
```

Solution 3: Dynamic Programming

> **How to think about it**
>
> We add a uniform integer in [1,maxPts] until the total reaches k, and want the probability it is still ≤ n. k can be 10^4, so expanding every draw is impossible. After the one-step difference, every state except the closed form at k-1 reads a larger score first. The chain from 0 to k has length k and overflows the stack. Larger scores are known if we walk downward. Let f[i] be the success probability from total i, write the terminal window as 1, and fill from k-2 down to 0.

Let f[i] be the probability that a current score of i ends at most n. The answer is f[0].

The value of f[i] is as follows:

- If i ≥ k, drawing stops. The value is 1 when i ≤ n, and 0 otherwise;
- Otherwise the next draw j lies in [1,..maxPts], so f[i] = 1/maxPts $\sum_{j=1}^{maxPts}$ f[i+j].

Every state depends only on a larger score, so fill from k - 2 down to 0. When k = 0 no draw has started, and the answer is 1.

The time complexity of the above method is O(k × maxPts), which will exceed the time limit, so we need to optimize it.

When i < k, the following equation holds:

$$f[i] = (f[i + 1] + f[i + 2] + \ldots + f[i + maxPts]) / maxPts \quad (1)$$

When i < k - 1, the following equation holds:

$$f[i+1] = (f[i + 2] + f[i + 3] + \ldots + f[i + maxPts + 1]) / maxPts \quad (2)$$

Therefore, when i < k-1, we subtract equation (2) from equation (1) to get:

$$f[i] - f[i+1] = (f[i + 1] - f[i + maxPts + 1]) / maxPts$$

That is:

$$f[i] = f[i + 1] + (f[i + 1] - f[i + maxPts + 1]) / maxPts$$

If i=k-1, we have:

$$f[i] = f[k - 1] = (f[k] + f[k + 1] + \ldots + f[k + maxPts - 1]) / maxPts \quad (3)$$

We assume there are i numbers not exceeding n, then k+i-1 ≤ n, and since i≤ maxPts, we have i ≤ min(n-k+1, maxPts), so equation (3) can be written as:

$$f[k-1] = \min(n-k+1, maxPts) / maxPts$$

In summary, we have the following state transition equation:

$$f[i] = \begin{cases} 1, & i \geq k, i \leq n \\ 0, & i \geq k, i > n \\ \min(n-k+1, maxPts) / maxPts, & i = k - 1 \\ f[i + 1] + (f[i + 1] - f[i + maxPts + 1]) / maxPts, & i < k - 1 \end{cases}$$

Time complexity O(k + maxPts), space complexity O(k + maxPts). Where k is the maximum score.

Python 3

```python
class Solution:
    def new21Game(self, n: int, k: int, maxPts: int) -> float:
        f = [0] * (k + maxPts)
        for i in range(k, min(n + 1, k + maxPts)):
            f[i] = 1
        f[k - 1] = min(n - k + 1, maxPts) / maxPts
        for i in range(k - 2, -1, -1):
            f[i] = f[i + 1] + (f[i + 1] - f[i + maxPts + 1]) / maxPts
        return f[0]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The probability of ending with exactly score s (k <=
# s < k + maxPts)
# 2. The expected number of draws


# ============================================================
# FOLLOW-UP 1: The probability of ending with exactly
# score s (k <= s < k + maxPts)
# ============================================================

# ---- follow-up 1: function ----
# Answer: dp[i] is the probability to reach exactly i; the
# ending scores are the ones at least k.
# Time: O(k + maxPts).  Space: O(k + maxPts).
def final_score_probability(k, max_pts):
    probability = [0.0] * (k + max_pts)
    probability[0] = 1.0
    window = 0.0
    for i in range(1, k + max_pts):
        if i - 1 < k:
            window += probability[i - 1]
        if i - max_pts - 1 >= 0 and i - max_pts - 1 < k:
            window -= probability[i - max_pts - 1]
        probability[i] = window / max_pts
    return {s: probability[s] for s in range(k, k + max_pts)}


# ============================================================
# FOLLOW-UP 2: The expected number of draws
# ============================================================

# ---- follow-up 2: function ----
# Answer: E(x) = 1 + average of E(x + 1) .. E(x + maxPts),
# and 0 once the score is at least k.
# Time: O(k * maxPts).  Space: O(k + maxPts).
def expected_draws(k, max_pts):
    expected = [0.0] * (k + max_pts + 1)
    for x in range(k - 1, -1, -1):
        expected[x] = 1 + sum(expected[x + 1:x + max_pts + 1]) / max_pts
    return expected[0] if k > 0 else 0.0
```

PART 2 · TOPIC 46 OF 50

State Machine DP

3 problems · 0 Easy · 1 Medium · 2 Hard

THE IDEA

A few states (holding, cooldown, sold ...) with transitions between them; keep the best value for every state at every step.

THE 3 PROBLEMS, EASY TO HARD | ★ = TOP 3: DO THESE FIRST

1	★ **Best Time to Buy and Sell Stock with Transaction Fee**	Medium	LC 714	2729
2	★ **Best Time to Buy and Sell Stock III**	Hard	LC 123	2733
3	★ **Best Time to Buy and Sell Stock IV**	Hard	LC 188	2736

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Best Time to Buy and Sell Stock with Cooldown (Part 1, pattern 13, page 694)

714. Best Time to Buy and Sell Stock with Transaction Fee

Medium · Greedy, Array, Dynamic Programming

leetcode.com/problems/best-time-to-buy-and-sell-stock-with-transaction-fee

★ **TOP 7 PROBLEM** in State Machine DP: master this one first

PROBLEM

You are given an array `prices` where `prices[i]` is the price of a given stock on the i^{th} day, and an integer `fee` representing a transaction fee.

Find the maximum profit you can achieve. You may complete as many transactions as you like, but you need to pay the transaction fee for each transaction.

Note:

- You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).
- The transaction fee is only charged once for each stock purchase and sale.

Example 1:

```
Input: prices = [1,3,2,8,4,9], fee = 2
Output: 8
Explanation: The maximum profit can be achieved by:
- Buying at prices[0] = 1
- Selling at prices[3] = 8
- Buying at prices[4] = 4
- Selling at prices[5] = 9
The total profit is ((8 - 1) - 2) + ((9 - 4) - 2) = 8.
```

Example 2:

```
Input: prices = [1,3,7,5,10,3], fee = 3
Output: 6
```

Constraints:

- `1 <= prices.length <= 5 * 10^4`
- `1 <= prices[i] < 5 * 10^4`
- `0 <= fee < 5 * 10^4`

SOLUTIONS (PYTHON 3)

Solution 1: Memoization

How to think about it

We may trade many times and pay fee on each completed sale. n ≤ 5× 10^4, so an unmemoized search over days repeats states. A day has two modes: flat or holding. From flat we buy or skip; from holding we sell (minus fee) or hold. The optimum from day i in mode j depends only on those two successors. Memoize dfs(i,j); past the last day the profit is 0. The answer is dfs(0,0). There are O(n) states.

We design a function dfs(i, j), which represents the maximum profit that can be obtained starting from day i with state j. Here, j can take the values 0 and 1, representing not holding and holding a stock, respectively. The answer is dfs(0, 0).

The execution logic of the function dfs(i, j) is as follows:

If i ≥ n, there are no more stocks to trade, so we return 0.

Otherwise, we can choose not to trade, in which case dfs(i, j) = dfs(i + 1, j). We can also choose to trade stocks. If j > 0, it means that we currently hold a stock and can sell it. In this case, dfs(i, j) = prices[i] + dfs(i + 1, 0) - fee. If j = 0, it means that

we currently do not hold a stock and can buy one. In this case, dfs(i, j) = -prices[i] + dfs(i + 1, 1). We take the maximum value as the return value of the function dfs(i, j).

The answer is dfs(0, 0).

To avoid redundant calculations, we use memoization to record the return value of dfs(i, j) in an array f. If f[i][j] is not equal to -1, it means that we have already calculated it, so we can directly return f[i][j].

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array prices.

Python 3

```python
class Solution:
    def maxProfit(self, prices: List[int], fee: int) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            if i >= len(prices):
                return 0
            ans = dfs(i + 1, j)
            if j:
                ans = max(ans, prices[i] + dfs(i + 1, 0) - fee)
            else:
                ans = max(ans, -prices[i] + dfs(i + 1, 1))
            return ans

        return dfs(0, 0)
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> Solution 1 fills a suffix table from the end. The same choices can be written forward: f[i][0/1] is the best profit after day i, flat or holding. Flat comes from staying flat or selling today and paying fee; holding comes from staying put or buying today. Fill left to right; the answer is flat on the last day. Time stays O(n).

We define f[i][j] as the maximum profit that can be obtained up to day i with state j. Here, j can take the values 0 and 1, representing not holding and holding a stock, respectively. We initialize f[0][0] = 0 and f[0][1] = -prices[0].

When i ≥ 1, if we do not hold a stock at the current day, then f[i][0] can be obtained by transitioning from f[i - 1][0] and f[i - 1][1] + prices[i] - fee, i.e., f[i][0] = max(f[i - 1][0], f[i - 1][1] + prices[i] - fee). If we hold a stock at the current day, then f[i][1] can be obtained by transitioning from f[i - 1][1] and f[i - 1][0] - prices[i], i.e., f[i][1] = max(f[i - 1][1], f[i - 1][0] - prices[i]). The final answer is f[n - 1][0].

The time complexity is O(n), and the space complexity is O(n). Here, n is the length of the array prices.

Python 3

```python
class Solution:
    def maxProfit(self, prices: List[int], fee: int) -> int:
        n = len(prices)
        f = [[0] * 2 for _ in range(n)]
        f[0][1] = -prices[0]
        for i in range(1, n):
            f[i][0] = max(f[i - 1][0], f[i - 1][1] + prices[i] - fee)
            f[i][1] = max(f[i - 1][1], f[i - 1][0] - prices[i])
        return f[n - 1][0]
```

Solution 3: Dynamic Programming (Space Optimization)

> **How to think about it**
>
> Day i in Solution 2 reads only day i-1, so the full table is unnecessary. Roll two scalars f_0,f_1. Parallel assignment keeps the previous pair while both updates run, so the new holding state still sees the old flat profit. Space is O(1).

The transition only needs the previous day, so two variables are enough and the space complexity is O(1).

Python 3

```python
class Solution:
    def maxProfit(self, prices: List[int], fee: int) -> int:
        f0, f1 = 0, -prices[0]
        for x in prices[1:]:
            f0, f1 = max(f0, f1 + x - fee), max(f1, f0 - x)
        return f0
```

Solution 4: Dynamic Programming

> **How to think about it**
>
> We may trade many times and pay fee on each completed sale. $n \le 5\times 10^4$, so a raw decision tree over days is far too large. The state is the day and whether we hold. Skipping always calls the next day before it returns, so the chain has length n and overflows the stack. Later days are known if we walk backward. Let f[i][j] be the best profit from day i with holding flag j, and fill i from n-1 down to 0. Selling subtracts fee; buying enters the holding state.

Let f[i][j] be the maximum profit starting from day i in state j. The values of j are 0 and 1, meaning we do not hold a stock and we hold a stock. The answer is f[0][0]. Days past the end contribute 0, so f[n][j] = 0.

Fill i from n - 1 down to 0. Doing nothing keeps f[i + 1][j]. If j > 0, we hold a stock and may sell it, earning prices[i] + f[i + 1][0] - fee. If j = 0, we may buy, earning -prices[i] + f[i + 1][1]. Take the larger value:

$$f[i][j] = \max(f[i + 1][j], prices[i] + f[i + 1][0] - fee)$$

when j > 0, and

$$f[i][j] = \max(f[i + 1][j], -prices[i] + f[i + 1][1])$$

when j = 0.

The time complexity is O(n), and the space complexity is O(n), where n is the length of the array prices.

Python 3

```python
class Solution:
    def maxProfit(self, prices: List[int], fee: int) -> int:
        n = len(prices)
        f = [[0] * 2 for _ in range(n + 1)]
        for i in range(n - 1, -1, -1):
            for j in range(2):
                ans = f[i + 1][j]
                if j:
                    ans = max(ans, prices[i] + f[i + 1][0] - fee)
                else:
                    ans = max(ans, -prices[i] + f[i + 1][1])
                f[i][j] = ans
        return f[0][0]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the buy and sell days of the best plan
# 2. A cooldown of one day after every sale as well as the
# fee


# ========================================================
# FOLLOW-UP 1: Return the buy and sell days of the best
# plan
# ========================================================

# ---- follow-up 1: function ----
# Answer: Run the two-state table with the choices
# remembered (small inputs).
```

```
# Time: O(n^2) in the worst case (the trade lists are
# copied).  Space: O(n).
def best_trades(prices, fee):
    cash_value, cash_trades = 0, []
    hold_value, hold_buy, hold_trades = -prices[0], 0, []
    for i in range(1, len(prices)):
        sell_value = hold_value + prices[i] - fee
        buy_value = cash_value - prices[i]
        if sell_value > cash_value:
            new_cash = (sell_value, hold_trades + [(hold_buy, i)])
        else:
            new_cash = (cash_value, cash_trades)
        if buy_value > hold_value:
            new_hold = (buy_value, i, cash_trades)
        else:
            new_hold = (hold_value, hold_buy, hold_trades)
        (cash_value, cash_trades), (hold_value, hold_buy, hold_trades) = new_cash, new_hold
    return cash_value, cash_trades


# ============================================================
# FOLLOW-UP 2: A cooldown of one day after every sale as
# well as the fee
# ============================================================

# ---- follow-up 2: function ----
# Answer: A third state: just sold.
# Time: O(n).  Space: O(1).
def max_profit_cooldown_fee(prices, fee):
    hold = float("-inf")
    sold = 0
    rest = 0
    for p in prices:
        new_hold = max(hold, rest - p)
        new_sold = hold + p - fee
        new_rest = max(rest, sold)
        hold, sold, rest = new_hold, new_sold, new_rest
    return max(sold, rest)
```

PART 2 · STATE MACHINE DP · PROBLEM 2 OF 3

123. Best Time to Buy and Sell Stock III

Hard · Array, Dynamic Programming

leetcode.com/problems/best-time-to-buy-and-sell-stock-iii

★ **TOP 7 PROBLEM** in State Machine DP: master this one first

PROBLEM

You are given an array `prices` where `prices[i]` is the price of a given stock on the i^{th} day.

Find the maximum profit you can achieve. You may complete **at most two transactions**.

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

```
Input: prices = [3,3,5,0,0,3,1,4]
Output: 6
Explanation: Buy on day 4 (price = 0) and sell on day 6 (price = 3), profit = 3-0 = 3.
Then buy on day 7 (price = 1) and sell on day 8 (price = 4), profit = 4-1 = 3.
```

Example 2:

```
Input: prices = [1,2,3,4,5]
Output: 4
Explanation: Buy on day 1 (price = 1) and sell on day 5 (price = 5), profit = 5-1 = 4.
Note that you cannot buy on day 1, buy on day 2 and sell them later, as you are engaging multiple
transactions at the same time. You must sell before buying again.
```

Example 3:

```
Input: prices = [7,6,4,3,1]
Output: 0
Explanation: In this case, no transaction is done, i.e. max profit = 0.
```

Constraints:

- `1 <= prices.length <=` 10^5
- `0 <= prices[i] <=` 10^5

SOLUTION (PYTHON 3)

Solution 1: Dynamic Programming

How to think about it

At most two transactions, so the unlimited-trade state of the previous problem is not enough. A 3D DP over day, trades finished, and hold is heavier than we need. The legal order is buy1, sell1, buy2, sell2. Four variables update in that order, each day using the previous stage. Buying and selling on the same day yields 0 and does not hurt the optimum.

We define the following variables:

- `f1` represents the maximum profit after the first purchase of the stock;
- `f2` represents the maximum profit after the first sale of the stock;
- `f3` represents the maximum profit after the second purchase of the stock;
- `f4` represents the maximum profit after the second sale of the stock.

During the traversal, we directly calculate `f1`, `f2`, `f3`, `f4`. We consider that buying and selling on the same day will result in a profit of 0, which will not affect the answer.

Finally, return `f4`.

The time complexity is O(n), where n is the length of the `prices` array. The space complexity is O(1).

Python 3

```python
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        # 第一次买入，第一次卖出，第二次买入，第二次卖出
        f1, f2, f3, f4 = -prices[0], 0, -prices[0], 0
        for price in prices[1:]:
            f1 = max(f1, -price)
            f2 = max(f2, f1 + price)
            f3 = max(f3, f2 - price)
            f4 = max(f4, f3 + price)
        return f4
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Return the two trades
# 2. At most k transactions (k given)


# ============================================================
# FOLLOW-UP 1: Return the two trades
# ============================================================

# ---- follow-up 1: function ----
# Answer: Try every split day: the best single trade
# before it and the best after it, remembering the days.
# Time: O(n^2).  Space: O(1).
def best_two_trades(prices):
    n = len(prices)

    def best_single(lo, hi):
        best = (0, None)
        low = lo
        for i in range(lo, hi):
            if prices[i] < prices[low]:
                low = i
            if prices[i] - prices[low] > best[0]:
                best = (prices[i] - prices[low], (low, i))
        return best

    answer = (0, [])
    for split in range(n + 1):
        first = best_single(0, split)
        second = best_single(split, n)
        total = first[0] + second[0]
        if total > answer[0]:
            answer = (total, [t for t in (first[1], second[1]) if t])
    return answer


# ============================================================
# FOLLOW-UP 2: At most k transactions (k given)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Two tables for k transactions: the cash after j
# sells and the stock held during the j-th buy.
# Time: O(n * k).  Space: O(k).
def max_profit_k_transactions(k, prices):
    if not prices or k == 0:
        return 0
    buy = [float("-inf")] * (k + 1)
    sell = [0] * (k + 1)
    for p in prices:
```

```
        for j in range(1, k + 1):
            buy[j] = max(buy[j], sell[j - 1] - p)
            sell[j] = max(sell[j], buy[j] + p)
    return sell[k]
```

PART 2 · STATE MACHINE DP · PROBLEM 3 OF 3

188. Best Time to Buy and Sell Stock IV

Hard · Array, Dynamic Programming

leetcode.com/problems/best-time-to-buy-and-sell-stock-iv

★ **TOP 7 PROBLEM** in State Machine DP: master this one first

PROBLEM

You are given an integer array `prices` where `prices[i]` is the price of a given stock on the i^{th} day, and an integer `k`.

Find the maximum profit you can achieve. You may complete at most `k` transactions: i.e. you may buy at most `k` times and sell at most `k` times.

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

```
Input: k = 2, prices = [2,4,1]
Output: 2
Explanation: Buy on day 1 (price = 2) and sell on day 2 (price = 4), profit = 4-2 = 2.
```

Example 2:

```
Input: k = 2, prices = [3,2,6,5,0,3]
Output: 7
Explanation: Buy on day 2 (price = 2) and sell on day 3 (price = 6), profit = 6-2 = 4. Then buy on day
5 (price = 0) and sell on day 6 (price = 3), profit = 3-0 = 3.
```

Constraints:

- `1 <= k <= 100`
- `1 <= prices.length <= 1000`
- `0 <= prices[i] <= 1000`

SOLUTIONS (PYTHON 3)

Solution 1: Memoization Search

How to think about it

At most k transactions generalizes the two-trade problem. n≤ 1000, k≤ 100, so a 3D state is fine. dfs(i,j,hold) starts on day i with j buys left. Skipping is always legal; holding may sell (buy count unchanged); a flat position with remaining buys may buy and consume one. Memoize (i,j,hold).

We design a function dfs(i, j, k) to represent the maximum profit that can be obtained when starting from day i, completing at most j transactions, and holding the stock with the current state k (not holding the stock is represented by 0, and holding the stock is represented by 1). The answer is dfs(0, k, 0).

The execution logic of the function dfs(i, j, k) is as follows:

- If i is greater than or equal to n, return 0 directly.
- The i-th day can choose not to do anything, then dfs(i, j, k) = dfs(i + 1, j, k).
- If k > 0, the i-th day can choose to sell the stock, then dfs(i, j, k) = max(dfs(i + 1, j - 1, 0) + prices[i], dfs(i + 1, j, k)).
- Otherwise, if j > 0, the i-th day can choose to buy the stock, then dfs(i, j, k) = max(dfs(i + 1, j - 1, 1) - prices[i], dfs(i + 1, j, k)).

The value of dfs(i, j, k) is the maximum value of the above three cases.

During the process, we can use memoization search to save the results of each calculation to avoid repeated calculations.

The time complexity is O(n × k), and the space complexity is O(n × k), where n and k are the length of the prices array and the value of k, respectively.

Python 3

```python
class Solution:
    def maxProfit(self, k: int, prices: List[int]) -> int:
        @cache
        def dfs(i: int, j: int, k: int) -> int:
            if i >= len(prices):
                return 0
            ans = dfs(i + 1, j, k)
            if k:
                ans = max(ans, prices[i] + dfs(i + 1, j, 0))
            elif j:
                ans = max(ans, -prices[i] + dfs(i + 1, j - 1, 1))
            return ans

        return dfs(0, k, 0)
```

Solution 2: Dynamic Programming

> **How to think about it**
>
> Solution 1 is memoized recursion. Bottom-up, f[i][j][0/1] is the best profit after day i with j buys used. Day 0 buy is -prices[0]; later transitions are sell, stay flat, buy, or keep holding — the same state space.

We can also use dynamic programming to define f[i][j][k] as the maximum profit that can be obtained when completing at most j transactions (here we define the number of transactions as the number of purchases), and holding the stock with the current state k on the i-th day. The initial value of f[i][j][k] is 0. The answer is f[n - 1][k][0].

When i = 0, the stock price is prices[0]. For any j \in [1, k], we havef[0][j][1] = -prices[0], which means buying the stock on the 0-th day with a profit of-prices[0]$.

When i > 0:

- If the i-th day does not hold the stock, it may be that the stock was held on the i-1-th day and sold on the i-th day, or the stock was not held on the i-1-th day and no operation was performed on the i-th day. Therefore, f[i][j][0] = max(f[i - 1][j][1] + prices[i], f[i - 1][j][0]).
- If the i-th day holds the stock, it may be that the stock was not held on the i-1-th day and bought on the i-th day, or the stock was held on the i-1-th day and no operation was performed on the i-th day. Therefore, f[i][j][1] = max(f[i - 1][j - 1][0] - prices[i], f[i - 1][j][1]).

Therefore, when i > 0, we can get the state transition equation:

aligned f[i][j][0] &= max(f[i - 1][j][1] + prices[i], f[i - 1][j][0]) \ f[i][j][1] &= max(f[i - 1][j - 1][0] - prices[i], f[i - 1][j][1]) aligned

The final answer is f[n - 1][k][0].

The time complexity is O(n × k), and the space complexity is O(n × k), where n and k are the length of the prices array and the value of k, respectively.

Python 3

```python
class Solution:
    def maxProfit(self, k: int, prices: List[int]) -> int:
        n = len(prices)
        f = [[[0] * 2 for _ in range(k + 1)] for _ in range(n)]
        for j in range(1, k + 1):
            f[0][j][1] = -prices[0]
        for i, x in enumerate(prices[1:], 1):
            for j in range(1, k + 1):
                f[i][j][0] = max(f[i - 1][j][1] + x, f[i - 1][j][0])
                f[i][j][1] = max(f[i - 1][j - 1][0] - x, f[i - 1][j][1])
        return f[n - 1][k][0]
```

Solution 3: Dynamic Programming (Space Optimization)

> **How to think about it**
>
> Day i in Solution 2 depends only on day i-1. Drop the day dimension and roll f[j][2] in O(k) space.

f[i][j][k] depends only on the previous day, so a table of size (k+1) × 2 is enough. The space complexity is O(k).

Python 3

```
class Solution:
    def maxProfit(self, k: int, prices: List[int]) -> int:
        f = [[0] * 2 for _ in range(k + 1)]
        for j in range(1, k + 1):
            f[j][1] = -prices[0]
        for x in prices[1:]:
            for j in range(k, 0, -1):
                f[j][0] = max(f[j][1] + x, f[j][0])
                f[j][1] = max(f[j - 1][0] - x, f[j][1])
        return f[k][0]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the transactions (buy day, sell day)
# 2. Unlimited transactions with a one-day cooldown


# ============================================================
# FOLLOW-UP 1: Return the transactions (buy day, sell day)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same table, and replay the trades with a small
# brute search on the days (for short price lists).
# Time: O(n^(2k)) (brute force over all trades).  Space:
# O(k).
def best_k_trades(k, prices):
    n = len(prices)
    best = [0, []]

    def go(day, left, profit, trades):
        if profit > best[0]:
            best[0] = profit
            best[1] = list(trades)
        if left == 0:
            return
        for buy in range(day, n):
            for sell in range(buy + 1, n):
                if prices[sell] > prices[buy]:
                    trades.append((buy, sell))
                    go(sell + 1, left - 1, profit + prices[sell] - prices[buy], trades)
                    trades.pop()

    go(0, k, 0, [])
    return best[0], best[1]


# ============================================================
# FOLLOW-UP 2: Unlimited transactions with a one-day
# cooldown
# ============================================================

# ---- follow-up 2: function ----
# Answer: Three states: holding, just sold, resting.
# Time: O(n).  Space: O(1).
def max_profit_cooldown(prices):
```

```
    hold = float("-inf")
    sold = 0
    rest = 0
    for p in prices:
        new_hold = max(hold, rest - p)
        new_sold = hold + p
        new_rest = max(rest, sold)
        hold, sold, rest = new_hold, new_sold, new_rest
    return max(sold, rest)
```

PART 2 · TOPIC 47 OF 50

Maths / Geometry

6 problems · 0 Easy · 5 Medium · 1 Hard

THE IDEA

Look for a formula, a gcd or modulo trick, or a way to represent the geometry (slopes, vectors, hashed points) so that a simple count solves it.

THE 6 PROBLEMS, EASY TO HARD | ★ = TOP 6: DO THESE FIRST

1	★ **Valid Square**	Medium	LC 593	2741
2	★ **Valid Triangle Number**	Medium	LC 611	2744
3	★ **Minimum Area Rectangle**	Medium	LC 939	2746
4	★ **Minimum Area Rectangle II**	Medium	LC 963	2749
5	★ **Count Good Numbers**	Medium	LC 1922	2753
6	★ **Max Points on a Line**	Hard	LC 149	2755

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Palindrome Number (Part 1, pattern 34, page 1793)

Roman to Integer (Part 1, pattern 34, page 1796)

Greatest Common Divisor of Strings (Part 1, pattern 34, page 1803)

Ugly Number II (Part 1, pattern 34, page 1827)

Multiply Strings (Part 1, pattern 6, page 310)

Count Primes (Part 1, pattern 34, page 1818)

Plus One (Part 1, pattern 36, page 1864)

Excel Sheet Column Title (Part 1, pattern 34, page 1801)

Ugly Number (Part 1, pattern 34, page 1808)

Reverse Integer (Part 1, pattern 34, page 1812)

Find the Duplicate Number (Part 1, pattern 1, page 43)

Factorial Trailing Zeroes (Part 1, pattern 34, page 1821)

PART 2 · MATHS / GEOMETRY · PROBLEM 1 OF 6

593. Valid Square

Medium · Geometry, Math

leetcode.com/problems/valid-square

★ **TOP 7 PROBLEM** in Maths / Geometry: master this one first

PROBLEM

Given the coordinates of four points in 2D space `p1`, `p2`, `p3` and `p4`, return `true` *if the four points construct a square*.

The coordinate of a point p_i is represented as $[x_i, y_i]$. The input is **not** given in any order.

A **valid square** has four equal sides with positive length and four equal angles (90-degree angles).

Example 1:

```
Input: p1 = [0,0], p2 = [1,1], p3 = [1,0], p4 = [0,1]
Output: true
```

Example 2:

```
Input: p1 = [0,0], p2 = [1,1], p3 = [1,0], p4 = [0,12]
Output: false
```

Example 3:

```
Input: p1 = [1,0], p2 = [-1,0], p3 = [0,1], p4 = [0,-1]
Output: true
```

Constraints:

- `p1.length == p2.length == p3.length == p4.length == 2`
- $-10^4 <= x_i, y_i <= 10^4$

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Four points form a square if the sides are equal and adjacent sides are perpendicular — equivalently, every triple is an isosceles right triangle. Testing permutations of a quadrilateral is messy. For each triple, check that two squared sides are equal and the Pythagorean identity holds, rejecting zero length. If all four triples pass, the points form a square.

Python 3

```
class Solution:
    def validSquare(
        self, p1: List[int], p2: List[int], p3: List[int], p4: List[int]
    ) -> bool:
        def check(a, b, c):
            (x1, y1), (x2, y2), (x3, y3) = a, b, c
            d1 = (x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2)
            d2 = (x1 - x3) * (x1 - x3) + (y1 - y3) * (y1 - y3)
            d3 = (x2 - x3) * (x2 - x3) + (y2 - y3) * (y2 - y3)
            return any(
                [
                    d1 == d2 and d1 + d2 == d3 and d1,
                    d2 == d3 and d2 + d3 == d1 and d2,
                    d1 == d3 and d1 + d3 == d2 and d1,
                ]
            )

        return (
            check(p1, p2, p3)
            and check(p2, p3, p4)
            and check(p1, p3, p4)
            and check(p1, p2, p4)
        )
```

Complexity. Time: O(1). Space: O(1).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Is it a rectangle (four right angles)
# 2. Is it a rhombus (four equal sides)


# ============================================================
# FOLLOW-UP 1: Is it a rectangle (four right angles)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Two diagonals of a rectangle have the same
# centre and the same length; the area must not be zero.
# Time: O(1).  Space: O(1).
def is_rectangle(p1, p2, p3, p4):
    points = [p1, p2, p3, p4]
    for k in (1, 2, 3):
        c = points[k]
        a, b = [points[i] for i in (1, 2, 3) if i != k]
        same_centre = p1[0] + c[0] == a[0] + b[0] and p1[1] + c[1] == a[1] + b[1]
        equal_diagonals = (p1[0] - c[0]) ** 2 + (p1[1] - c[1]) ** 2 == (a[0] - b[0]) ** 2 + (a[1] - b[1])
** 2
        area = (a[0] - p1[0]) * (b[1] - p1[1]) - (a[1] - p1[1]) * (b[0] - p1[0])
        if same_centre and equal_diagonals and area != 0:
            return True
    return False


# ============================================================
# FOLLOW-UP 2: Is it a rhombus (four equal sides)
# ============================================================

# ---- follow-up 2: function ----
# Answer: Two diagonals of a rhombus have the same centre
# and are perpendicular; the area must not be zero.
# Time: O(1).  Space: O(1).
def is_rhombus(p1, p2, p3, p4):
    points = [p1, p2, p3, p4]
    for k in (1, 2, 3):
```

```
        c = points[k]
        a, b = [points[i] for i in (1, 2, 3) if i != k]
        same_centre = p1[0] + c[0] == a[0] + b[0] and p1[1] + c[1] == a[1] + b[1]
        perpendicular = (c[0] - p1[0]) * (b[0] - a[0]) + (c[1] - p1[1]) * (b[1] - a[1]) == 0
        area = (a[0] - p1[0]) * (b[1] - p1[1]) - (a[1] - p1[1]) * (b[0] - p1[0])
        if same_centre and perpendicular and area != 0:
            return True
    return False
```

611. Valid Triangle Number

Medium · Greedy, Array, Two Pointers, Binary Search, Sorting

leetcode.com/problems/valid-triangle-number

★ **TOP 7 PROBLEM** in Maths / Geometry: master this one first

PROBLEM

Given an integer array `nums`, return *the number of triplets chosen from the array that can make triangles if we take them as side lengths of a triangle*.

Example 1:

```
Input: nums = [2,2,3,4]
Output: 3
Explanation: Valid combinations are:
2,3,4 (using the first 2)
2,3,4 (using the second 2)
2,2,3
```

Example 2:

```
Input: nums = [4,2,3,4]
Output: 4
```

Constraints:

- `1 <= nums.length <= 1000`
- `0 <= nums[i] <= 1000`

SOLUTION (PYTHON 3)

Solution 1: Sorting + Binary Search

How to think about it

A triple loop is acceptable for $n \le 10^3$ but sorting reduces the check: after $a \le b \le c$ only $a+b>c$ remains. Fix the two smaller sides i,j and binary-search the first index ≥ nums[i]+nums[j]; every k in between is a valid third side.

A valid triangle must satisfy: **the sum of any two sides is greater than the third side**. That is:

$$a + b > c \text{ tag1}$$

$$a + c > b \text{ tag2}$$

$$b + c > a \text{ tag3}$$

If we arrange the sides in ascending order, i.e., $a \le b \le c$, then obviously conditions (2) and (3) are satisfied. We only need to ensure that condition (1) is also satisfied to form a valid triangle.

We enumerate i in the range [0, n - 3], enumerate j in the range [i + 1, n - 2], and perform binary search in the range [j + 1, n - 1] to find the first index left that is greater than or equal to nums[i] + nums[j]. Then, the values of k in the range [j + 1, left - 1] satisfy the condition, and we add them to the result ans.

The time complexity is $O(n^2 \log n)$, and the space complexity is $O(\log n)$, where n is the length of the array.

Python 3

```
class Solution:
    def triangleNumber(self, nums: List[int]) -> int:
        nums.sort()
        ans, n = 0, len(nums)
        for i in range(n - 2):
            for j in range(i + 1, n - 1):
                k = bisect_left(nums, nums[i] + nums[j], lo=j + 1) - 1
                ans += k - j
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the triples (as index triples) that can form a
# triangle
# 2. Count triangles whose three sides are all different


# ============================================================
# FOLLOW-UP 1: Return the triples (as index triples) that
# can form a triangle
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort, then for every pair find the third by two
# pointers.
# Time: O(n^3).  Space: O(T), T = triples found.
def triangle_triples(nums):
    nums = sorted(nums)
    triples = []
    for k in range(len(nums) - 1, 1, -1):
        for i in range(k):
            for j in range(i + 1, k):
                if nums[i] + nums[j] > nums[k]:
                    triples.append((nums[i], nums[j], nums[k]))
    return sorted(triples)


# ============================================================
# FOLLOW-UP 2: Count triangles whose three sides are all
# different
# ============================================================

# ---- follow-up 2: function ----
# Answer: Count all triangles and subtract the ones with
# equal sides.
# Time: O(n^3).  Space: O(n).
def count_scalene(nums):
    nums = sorted(nums)
    total = 0
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            for k in range(j + 1, len(nums)):
                if nums[i] + nums[j] > nums[k] and nums[i] != nums[j] and nums[j] != nums[k]:
                    total += 1
    return total
```

939. Minimum Area Rectangle

Medium · Geometry, Array, Hash Table, Math, Sorting

leetcode.com/problems/minimum-area-rectangle

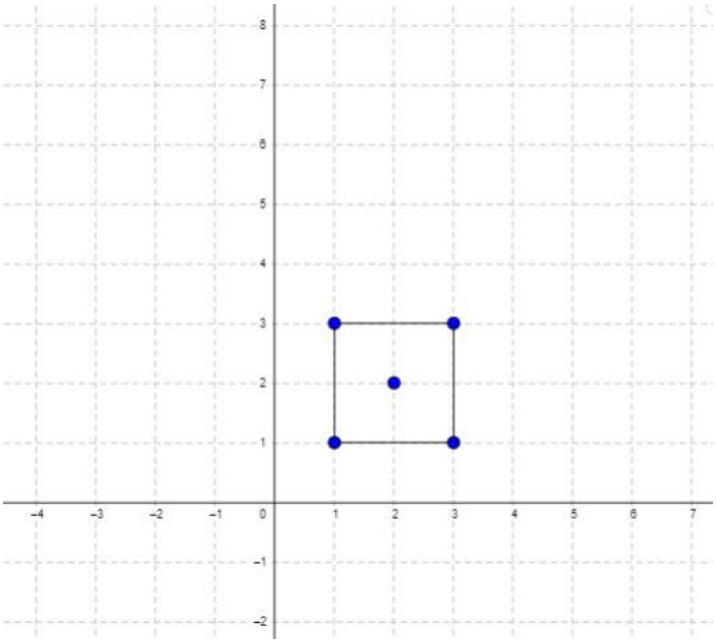
★ **TOP 7 PROBLEM** in Maths / Geometry: master this one first

PROBLEM

You are given an array of points in the **X-Y** plane `points` where `points[i] = [x_i, y_i]`.

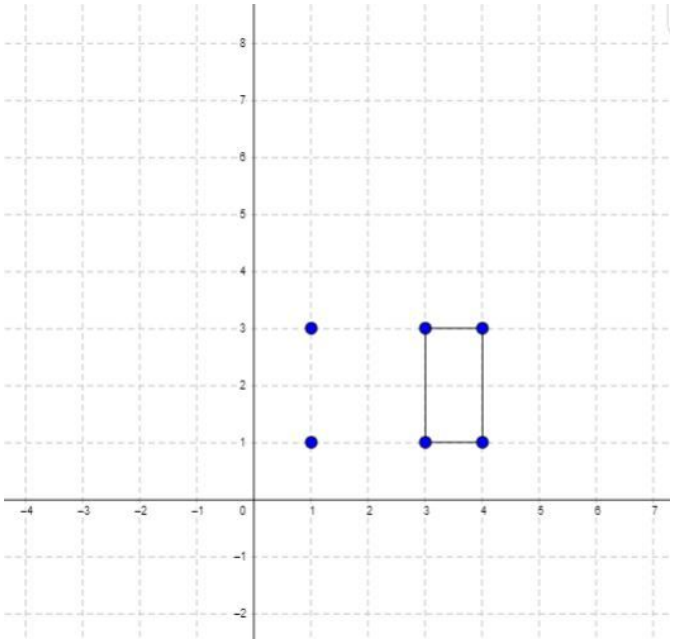
Return *the minimum area of a rectangle formed from these points, with sides parallel to the X and Y axes*. If there is not any such rectangle, return `0`.

Example 1:



```
Input: points = [[1,1],[1,3],[3,1],[3,3],[2,2]]
Output: 4
```

Example 2:



Input: points = [[1,1],[1,3],[3,1],[3,3],[4,1],[4,3]]
Output: 2

Constraints:

- 1 <= points.length <= 500
- points[i].length == 2
- 0 <= x_i, y_i <= 4 * 10^4
- All the given points are **unique**.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

An axis-aligned rectangle is spanned by two pairs of matching y-coordinates. n≤ 500, so four-nested loops are too slow, but pairs on the same vertical line are manageable. Group by x and scan left to right. For each pair (y_1,y_2) on the current line, a previous occurrence of that pair yields a rectangle. A map stores the latest x of every y-pair.

Python 3

```
class Solution:
    def minAreaRect(self, points: List[List[int]]) -> int:
        d = defaultdict(list)
        for x, y in points:
            d[x].append(y)
        pos = {}
        ans = inf
        for x in sorted(d):
            ys = d[x]
            ys.sort()
            n = len(ys)
            for i, y1 in enumerate(ys):
                for y2 in ys[i + 1 :]:
                    if (y1, y2) in pos:
                        ans = min(ans, (x - pos[(y1, y2)]) * (y2 - y1))
                    pos[(y1, y2)] = x
        return 0 if ans == inf else ans
```

Complexity. Time: O(n^2). Space: O(n^2).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the four corners of the smallest rectangle
# 2. The LARGEST rectangle area


# ============================================================
# FOLLOW-UP 1: Return the four corners of the smallest
# rectangle
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same pair-of-columns search, remembering the
# corners.
# Time: O(n^2).  Space: O(n).
def smallest_rectangle_corners(points):
    seen = set(map(tuple, points))
    best = None
    pts = sorted(seen)
    for i, (x1, y1) in enumerate(pts):
        for x2, y2 in pts[i + 1:]:
            if x1 < x2 and y1 != y2 and (x1, y2) in seen and (x2, y1) in seen:
                area = (x2 - x1) * abs(y2 - y1)
                if best is None or area < best[0]:
                    best = (area, sorted([(x1, y1), (x2, y2), (x1, y2), (x2, y1)]))
    return best


# ============================================================
# FOLLOW-UP 2: The LARGEST rectangle area
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same search keeping the maximum.
# Time: O(n^2).  Space: O(n).
def largest_rectangle_area(points):
    seen = set(map(tuple, points))
    best = 0
    for (x1, y1) in seen:
        for (x2, y2) in seen:
            if x1 < x2 and y1 < y2 and (x1, y2) in seen and (x2, y1) in seen:
                best = max(best, (x2 - x1) * (y2 - y1))
    return best
```

963. Minimum Area Rectangle II

Medium · Geometry, Array, Hash Table, Math

leetcode.com/problems/minimum-area-rectangle-ii

★ **TOP 7 PROBLEM** in Maths / Geometry: master this one first

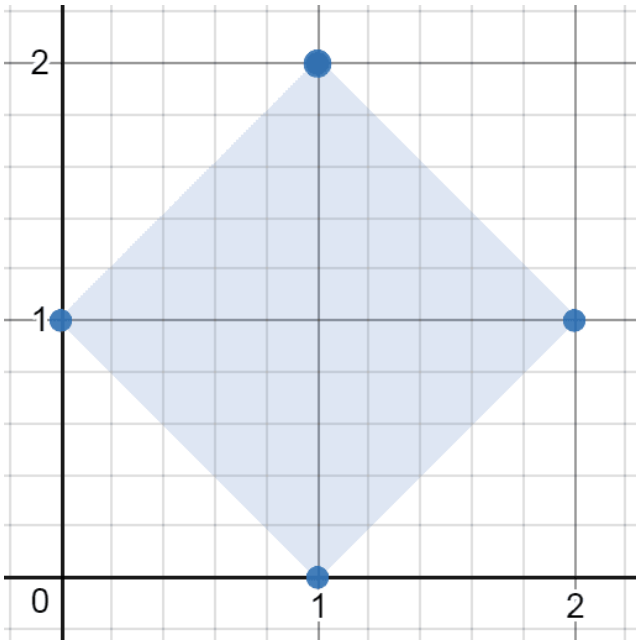
PROBLEM

You are given an array of points in the **X-Y** plane `points` where `points[i] = [x_i, y_i]`.

Return *the minimum area of any rectangle formed from these points, with sides **not necessarily parallel** to the X and Y axes*. If there is not any such rectangle, return `0`.

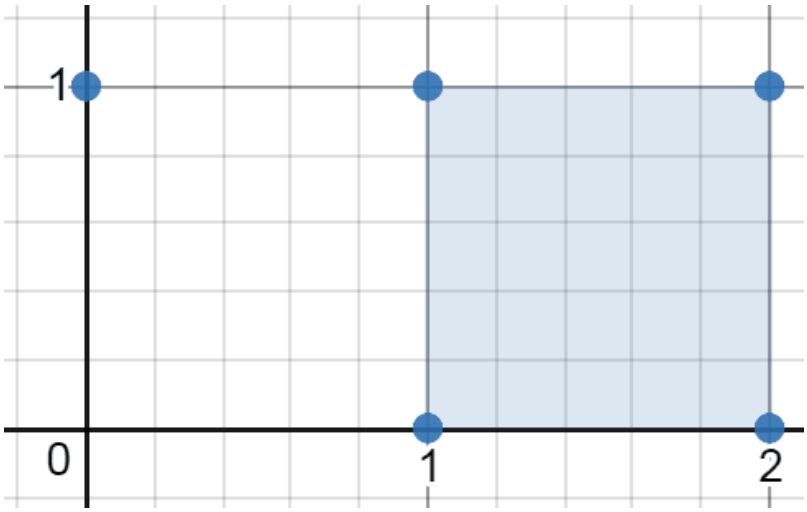
Answers within 10^{-5} of the actual answer will be accepted.

Example 1:



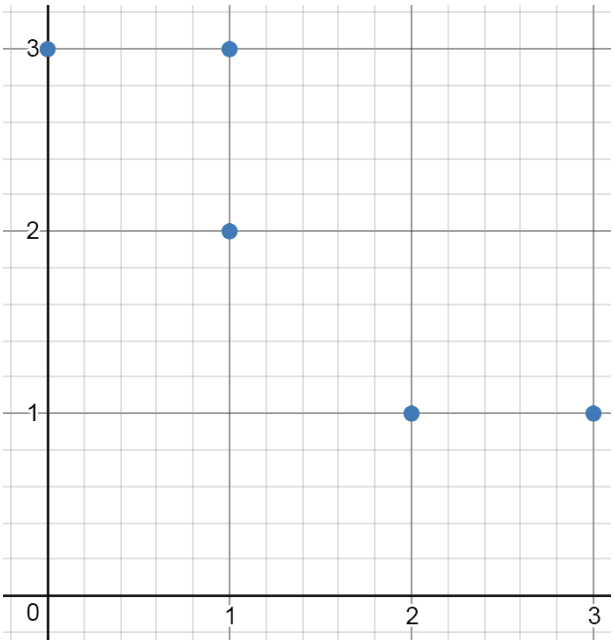

```
Input: points = [[1,2],[2,1],[1,0],[0,1]]
Output: 2.00000
Explanation: The minimum area rectangle occurs at [1,2],[2,1],[1,0],[0,1], with an area of 2.
```

Example 2:



```
Input: points = [[0,1],[2,1],[1,1],[1,0],[2,0]]
Output: 1.00000
Explanation: The minimum area rectangle occurs at [1,0],[1,1],[2,1],[2,0], with an area of 1.
```

Example 3:




```
Input: points = [[0,3],[1,2],[3,1],[1,3],[2,1]]
Output: 0
Explanation: There is no possible rectangle to form from these points.
```

Constraints:

- `1 <= points.length <= 50`
- `points[i].length == 2`
- `0 <= x_i, y_i <= 4 * 10^4`
- All the given points are **unique**.

SOLUTION (PYTHON 3)

Solution 1: Hash Table + Enumeration

How to think about it

The smallest area of a (possibly rotated) rectangle, n≤ 50, allows enumerating three vertices. With a right angle at p_1, the fourth corner is the vector sum of the two edges. The packed key x × 40001 + y repeats once a coordinate leaves [0, 4 × 10^4], and the sum of two squared edges exceeds 2^{31}-1. Inside that square the key is unique, and those squares fit in a 64-bit integer. The fourth corner is accepted only when both coordinates lie in range and the key is present. Squared edge lengths are accumulated in 64 bits before the square root.

We use a hash table to store all the points, then enumerate three points $p_1 = (x_1, y_1)$, $p_2 = (x_2, y_2)$, $p_3 = (x_3, y_3)$. If overrightarrowp_1p_2 and overrightarrowp_1p_3 are perpendicular, the fourth corner is $(x_4, y_4)=(x_2 - x_1 + x_3, y_2 - y_1 + y_3)$. Coordinates lie in [0, 4 × 10^4], and the packed key x × 40001 + y is unique only inside that square, so this corner counts only when both coordinates fall in range and the key is present. The squared edge lengths are summed in 64-bit integers, since each sum can exceed 2^{31}-1, and the area is the square root of their product.

Finally, if a rectangle that satisfies the conditions is found, return the minimum area among them. Otherwise, return 0.

The time complexity is $O(n^3)$ and the space complexity is O(n), where n is the length of the array points.

Python 3

```python
class Solution:
    def minAreaFreeRect(self, points: List[List[int]]) -> float:
        s = {(x, y) for x, y in points}
        n = len(points)
        ans = inf
        for i in range(n):
            x1, y1 = points[i]
            for j in range(n):
                if j != i:
                    x2, y2 = points[j]
                    for k in range(j + 1, n):
                        if k != i:
                            x3, y3 = points[k]
                            x4 = x2 - x1 + x3
                            y4 = y2 - y1 + y3
                            if 0 <= x4 <= 40000 and 0 <= y4 <= 40000 and (x4, y4) in s:
                                v21 = (x2 - x1, y2 - y1)
                                v31 = (x3 - x1, y3 - y1)
                                if v21[0] * v31[0] + v21[1] * v31[1] == 0:
                                    w = sqrt(v21[0] ** 2 + v21[1] ** 2)
                                    h = sqrt(v31[0] ** 2 + v31[1] ** 2)
                                    ans = min(ans, w * h)
        return 0 if ans == inf else ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. The LARGEST area of a rectangle (any orientation)
# 2. How many rectangles are there (any orientation)


# ============================================================
# FOLLOW-UP 1: The LARGEST area of a rectangle (any
# orientation)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Group diagonals by centre and length; every two
# diagonals of a group form a rectangle.
# Time: O(n^2 + sum of g^2 over the groups of equal
# midpoint and length).  Space: O(n^2).
def largest_any_rectangle(points):
    groups = {}
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            (x1, y1), (x2, y2) = points[i], points[j]
            key = (x1 + x2, y1 + y2, (x1 - x2) ** 2 + (y1 - y2) ** 2)
            groups.setdefault(key, []).append((points[i], points[j]))
    best = 0
    for diagonals in groups.values():
        for a in range(len(diagonals)):
            for b in range(a + 1, len(diagonals)):
                p1, p2 = diagonals[a]
                p3, p4 = diagonals[b]
                area = abs((p3[0] - p1[0]) * (p4[1] - p1[1]) - (p3[1] - p1[1]) * (p4[0] - p1[0]))
                best = max(best, area)
    return best


# ============================================================
# FOLLOW-UP 2: How many rectangles are there (any
# orientation)
# ============================================================

# ---- follow-up 2: function ----
```

```
# Answer: Every pair of diagonals with the same centre and
# length makes one rectangle.
# Time: O(n^2).  Space: O(n^2).
def count_any_rectangles(points):
    groups = {}
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            (x1, y1), (x2, y2) = points[i], points[j]
            key = (x1 + x2, y1 + y2, (x1 - x2) ** 2 + (y1 - y2) ** 2)
            groups[key] = groups.get(key, 0) + 1
    return sum(c * (c - 1) // 2 for c in groups.values())
```

1922. Count Good Numbers

Medium · Recursion, Math

leetcode.com/problems/count-good-numbers

★ **TOP 7 PROBLEM** in Maths / Geometry: master this one first

PROBLEM

A digit string is **good** if the digits **(0-indexed)** at **even** indices are **even** and the digits at **odd** indices are **prime** (`2`, `3`, `5`, or `7`).

- For example, `"2582"` is good because the digits (`2` and `8`) at even positions are even and the digits (`5` and `2`) at odd positions are prime. However, `"3245"` is **not** good because `3` is at an even index but is not even.

Given an integer `n`, return *the **total** number of good digit strings of length* `n`. Since the answer may be large, **return it modulo** $10^9 + 7$.

A **digit string** is a string consisting of digits `0` through `9` that may contain leading zeros.

Example 1:

```
Input: n = 1
Output: 5
Explanation: The good numbers of length 1 are "0", "2", "4", "6", "8".
```

Example 2:

```
Input: n = 4
Output: 400
```

Example 3:

```
Input: n = 50
Output: 564908303
```

Constraints:

- $1 <= n <= 10^{15}$

SOLUTION (PYTHON 3)

Solution 1: Fast Exponentiation

How to think about it

Even indices have 5 choices and odd indices have 4, independently. Looping $n \le 10^{15}$ times is impossible. The count is $5^{\text{ceil}(n/2)} \cdot 4^{\text{floor}(n/2)}$, evaluated with modular fast exponentiation in O(log n).

For a "good number" of length n, the even-indexed positions have ceil(n/2) = floor((n + 1)/2) digits, and these positions can be filled with 5 different digits (0, 2, 4, 6, 8). The odd-indexed positions have floor(n/2) digits, and these positions can be filled with 4 different digits (2, 3, 5, 7). Therefore, the total number of "good numbers" of length n is:

$$\text{ans} = 5^{\text{ceil}(n/2)} \times 4^{\text{floor}(n/2)}$$

We can use fast exponentiation to compute $5^{\text{ceil}(n/2)}$ and $4^{\text{floor}(n/2)}$. The time complexity is O(log n), and the space complexity is O(1).

Python 3

```python
class Solution:
    def countGoodNumbers(self, n: int) -> int:
        mod = 10**9 + 7
        return pow(5, (n + 1) >> 1, mod) * pow(4, n >> 1, mod) % mod
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The number of good numbers with no leading zero (n
# digits)
# 2. The k-th smallest good number of n digits (digits may
# start with 0)


# ============================================================
# FOLLOW-UP 1: The number of good numbers with no leading
# zero (n digits)
# ============================================================

# ---- follow-up 1: function ----
# Answer: The first position (an even index) has four
# choices instead of five.
# Time: O(log n).  Space: O(1).
def count_good_no_leading_zero(n):
    mod = 10 ** 9 + 7
    even_positions = (n + 1) // 2
    odd_positions = n // 2
    return 4 * pow(5, even_positions - 1, mod) * pow(4, odd_positions, mod) % mod


# ============================================================
# FOLLOW-UP 2: The k-th smallest good number of n digits
# (digits may start with 0)
# ============================================================

# ---- follow-up 2: function ----
# Answer: A mixed number system: even positions use
# [0,2,4,6,8] (base 5), odd positions [2,3,5,7] (base 4).
# Time: O(n).  Space: O(n).
def kth_good_number(n, k):
    k -= 1
    digits = []
    for position in range(n - 1, -1, -1):
        options = [0, 2, 4, 6, 8] if position % 2 == 0 else [2, 3, 5, 7]
        k, r = divmod(k, len(options))
        digits.append(options[r])
    return "".join(map(str, reversed(digits)))
```

149. Max Points on a Line

Hard · Geometry, Array, Hash Table, Math, Greatest Common Divisor, Euclidean Algorithm

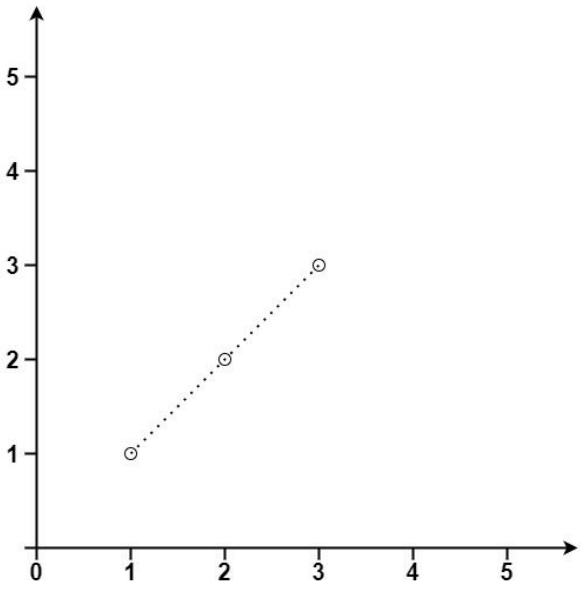
leetcode.com/problems/max-points-on-a-line

★ **TOP 7 PROBLEM** in Maths / Geometry: master this one first

PROBLEM

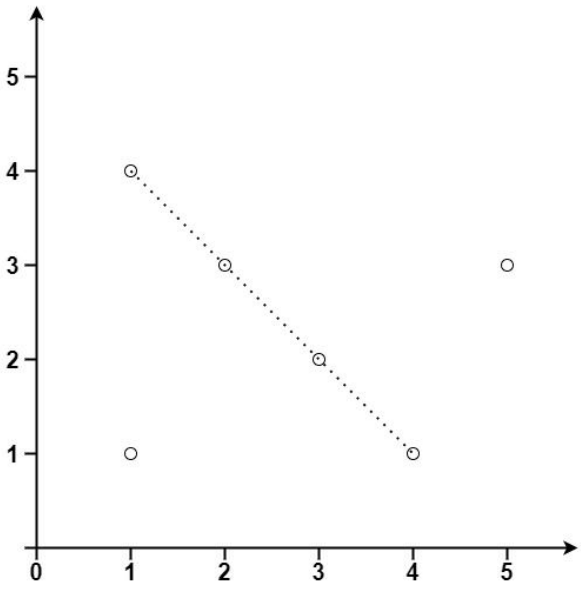
Given an array of `points` where `points[i] = [x_i, y_i]` represents a point on the **X-Y** plane, return *the maximum number of points that lie on the same straight line*.

Example 1:




```
Input: points = [[1,1],[2,2],[3,3]]
Output: 3
```

Example 2:



```
Input: points = [[1,1],[3,2],[5,3],[4,1],[2,3],[1,4]]
Output: 4
```

Constraints:

- `1 <= points.length <= 300`
- `points[i].length == 2`
- -10^4 <= x_i, y_i <= 10^4
- All the `points` are **unique**.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

Maximum number of collinear points. Points are distinct and n is small. Enumerate two points to fix a line, then test the rest with a cross product so we never divide. O(n^3) is acceptable.

Python 3

```python
class Solution:
    def maxPoints(self, points: List[List[int]]) -> int:
        n = len(points)
        ans = 1
        for i in range(n):
            x1, y1 = points[i]
            for j in range(i + 1, n):
                x2, y2 = points[j]
                cnt = 2
                for k in range(j + 1, n):
                    x3, y3 = points[k]
                    a = (y2 - y1) * (x3 - x1)
                    b = (y3 - y1) * (x2 - x1)
                    cnt += a == b
                ans = max(ans, cnt)
        return ans
```

Solution 2

How to think about it

Solution 1 is O(n^3). Fix one point and group the others by slope; equal slopes are collinear. Store a reduced (Δ y,Δ x) pair as the key to avoid floats. O(n) work per point, O(n^2) overall.

Python 3

```python
class Solution:
    def maxPoints(self, points: List[List[int]]) -> int:
        def gcd(a, b):
            return a if b == 0 else gcd(b, a % b)

        n = len(points)
        ans = 1
        for i in range(n):
            x1, y1 = points[i]
            cnt = Counter()
            for j in range(i + 1, n):
                x2, y2 = points[j]
                dx, dy = x2 - x1, y2 - y1
                g = gcd(dx, dy)
                k = (dx // g, dy // g)
                cnt[k] += 1
                ans = max(ans, cnt[k] + 1)
        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the points of the best line
# 2. How many different lines contain at least k points


# ============================================================
# FOLLOW-UP 1: Return the points of the best line
# ============================================================

# ---- follow-up 1: function ----
# Answer: For every pair keep the points with the same
# slope from the first point; return the largest set.
# Time: O(n^2 log C), C = coordinate size.  Space: O(n).
from math import gcd


def points_on_best_line(points):
    best = points[:1]
    for i in range(len(points)):
        groups = {}
        for j in range(i + 1, len(points)):
            dx, dy = points[j][0] - points[i][0], points[j][1] - points[i][1]
            g = gcd(dx, dy)
            dx, dy = dx // g, dy // g
            if dx < 0 or (dx == 0 and dy < 0):
                dx, dy = -dx, -dy
            groups.setdefault((dx, dy), []).append(points[j])
        for group in groups.values():
            if len(group) + 1 > len(best):
                best = [points[i]] + group
    return best


# ============================================================
# FOLLOW-UP 2: How many different lines contain at least k
# points
# ============================================================

# ---- follow-up 2: function ----
# Answer: Collect each line as a set of points
# (normalised) and count the distinct ones.
# Time: O(n^3 log n).  Space: O(n^3) in the worst case.
def lines_with_at_least(points, k):
    lines = set()
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            a, b = points[i], points[j]
            on_line = tuple(sorted(
                tuple(p) for p in points
                if (b[0] - a[0]) * (p[1] - a[1]) == (b[1] - a[1]) * (p[0] - a[0])))
            if len(on_line) >= k:
                lines.add(on_line)
    return len(lines)
```

PART 2 · TOPIC 48 OF 50

String Matching

3 problems · 0 Easy · 2 Medium · 1 Hard

THE IDEA

Find patterns in text with a rolling hash or the prefix function (KMP): preprocess the pattern or hash windows, then compare in O(1).

THE 3 PROBLEMS, EASY TO HARD | ★ = TOP 3: DO THESE FIRST

1	★ **Repeated String Match**	Medium	LC 686	2759
2	★ **Number of Distinct Substrings in a String**	Medium	LC 1698	2761
3	★ **Longest Duplicate Substring**	Hard	LC 1044	2764

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Repeated Substring Pattern (Part 1, pattern 6, page 297)

Longest Happy Prefix (Part 1, pattern 7, page 371)

Shortest Palindrome (Part 1, pattern 7, page 360)

PART 2 · STRING MATCHING · PROBLEM 1 OF 3

686. Repeated String Match

Medium · String, String Matching, KMP, Boyer–Moore, Extended KMP

leetcode.com/problems/repeated-string-match

★ **TOP 7 PROBLEM** in String Matching: master this one first

PROBLEM

Given two strings `a` and `b`, return *the minimum number of times you should repeat string* `a` *so that string* `b` *is a substring of it*. If it is impossible for `b` to be a substring of `a` after repeating it, return `-1`.

Notice: string `"abc"` repeated 0 times is `""`, repeated 1 time is `"abc"` and repeated 2 times is `"abcabc"`.

Example 1:

```
Input: a = "abcd", b = "cdabcdab"
Output: 3
Explanation: We return 3 because by repeating a three times "abcdabcdabcd", b is a substring of it.
```

Example 2:

```
Input: a = "a", b = "aa"
Output: 2
```

Constraints:

- `1 <= a.length, b.length <= 10^4`
- `a` and `b` consist of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

b must appear inside some repetition of a. At least ceil(|b|/|a|) copies are required, and a few extra copies cover a wrap-around match. Start from that lower bound and append a at most three more times. If `b` is a substring, return the count; otherwise -1.

Python 3

```python
class Solution:
    def repeatedStringMatch(self, a: str, b: str) -> int:
        m, n = len(a), len(b)
        ans = ceil(n / m)
        t = [a] * ans
        for _ in range(3):
            if b in ''.join(t):
                return ans
            ans += 1
            t.append(a)
        return -1
```

Complexity. Time: O(m + n). Space: O(m + n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the repeated string itself
# 2. How many times does b appear inside the first L
# characters of the endless repetition of a


# ============================================================
# FOLLOW-UP 1: Return the repeated string itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Repeat enough times that b fits, and check one
# more copy.
# Time: O(|a| + |b|) on average.  Space: O(|a| + |b|).
def shortest_repetition(a, b):
    times = -(-len(b) // len(a))
    for extra in (0, 1):
        candidate = a * (times + extra)
        if b in candidate:
            return candidate
    return None


# ============================================================
# FOLLOW-UP 2: How many times does b appear inside the
# first L characters of the endless repetition of a
# ============================================================

# ---- follow-up 2: function ----
# Answer: Build the prefix and count (overlapping)
# occurrences.
# Time: O(length * |b|) in the worst case.  Space:
# O(length).
def occurrences_in_prefix(a, b, length):
    text = (a * (length // len(a) + 1))[:length]
    count = 0
    start = text.find(b)
    while start != -1:
        count += 1
        start = text.find(b, start + 1)
    return count
```

1698. Number of Distinct Substrings in a String

Medium · Trie, String, Suffix Array, Hash Function, Rolling Hash

leetcode.com/problems/number-of-distinct-substrings-in-a-string

★ **TOP 7 PROBLEM** in String Matching: master this one first

PROBLEM

Given a string `s`, return *the number of **distinct** substrings of* `s`.

A **substring** of a string is obtained by deleting any number of characters (possibly zero) from the front of the string and any number (possibly zero) from the back of the string.

Example 1:

```
Input: s = "aabbaba"
Output: 21
Explanation: The set of distinct strings is ["a","b","aa","bb","ab","ba","aab","abb","bab","bba","aba"
,"aabb","abba","bbab","baba","aabba","abbab","bbaba","aabbab","abbaba","aabbaba"]
```

Example 2:

```
Input: s = "abcdefg"
Output: 28
```

Constraints:

- `1 <= s.length <= 500`
- `s` consists of lowercase English letters.

Follow up:

Can you solve this problem in

`O(n)`

time complexity?

SOLUTIONS (PYTHON 3)

Solution 1: Brute Force Enumeration

How to think about it

Count distinct substrings. n is small, so put all O(n^2) slices into a set. Slicing makes the time about O(n^3), which is acceptable here.

Enumerate all substrings and use a hash table to record the count of different substrings.

The time complexity is O(n^3), and the space complexity is O(n^2). Here, n is the length of the string.

Python 3

```python
class Solution:
    def countDistinct(self, s: str) -> int:
        n = len(s)
        return len({s[i:j] for i in range(n) for j in range(i + 1, n + 1)})
```

Solution 2: String Hashing

> **How to think about it**
>
> Solution 1 compares whole slices. Polynomial hashes give any substring fingerprint in O(1); enumerating endpoints and storing hashes compares integers in $O(n^2)$.

String hashing is a method to map a string of any length to a non-negative integer, and the probability of collision is almost zero. String hashing is used to calculate the hash value of a string, which can quickly determine whether two strings are equal.

We take a fixed value BASE, treat the string as a number in BASE radix, and assign a value greater than 0 to represent each character. Generally, the values we assign are much smaller than BASE. For example, for a string composed of lowercase letters, we can set a=1, b=2, ..., z=26. We take a fixed value MOD, calculate the remainder of the BASE radix number to MOD, and use it as the hash value of the string.

Generally, we take BASE=131 or BASE=13331, at which point the probability of collision of the hash value is extremely low. As long as the hash values of two strings are the same, we consider the two strings to be equal. Usually, MOD is taken as 2^64. In C++, we can directly use the unsigned long long type to store this hash value. When calculating, we do not handle arithmetic overflow. When overflow occurs, it is equivalent to automatically taking the modulus of 2^64, which can avoid inefficient modulus operations.

Except for extremely specially constructed data, the above hash algorithm is unlikely to cause collisions. In general, the above hash algorithm can appear in the standard answer of the problem. We can also take some appropriate BASE and MOD values (such as large prime numbers), perform several groups of hash operations, and only consider the original strings equal when the results are all the same, making it even more difficult to construct data that causes this hash to produce errors.

The time complexity is $O(n^2)$, and the space complexity is $O(n^2)$. Here, n is the length of the string.

Python 3

```python
class Solution:
    def countDistinct(self, s: str) -> int:
        base = 131
        n = len(s)
        p = [0] * (n + 10)
        h = [0] * (n + 10)
        p[0] = 1
        for i, c in enumerate(s):
            p[i + 1] = p[i] * base
            h[i + 1] = h[i] * base + ord(c)
        ss = set()
        for i in range(1, n + 1):
            for j in range(i, n + 1):
                t = h[j] - h[i - 1] * p[j - i + 1]
                ss.add(t)
        return len(ss)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Count them with a suffix array and the longest common
# prefixes
# 2. Distinct substrings of one given length k


# ============================================================
# FOLLOW-UP 1: Count them with a suffix array and the
# longest common prefixes
# ============================================================

# ---- follow-up 1: function ----
# Answer: Distinct substrings = total substrings minus the
```

```
# sum of LCP of neighbouring sorted suffixes.
# Time: O(n^2 log n).  Space: O(n^2).
def distinct_substrings_by_suffixes(s):
    n = len(s)
    suffixes = sorted(range(n), key=lambda i: s[i:])
    total = n * (n + 1) // 2
    for a, b in zip(suffixes, suffixes[1:]):
        common = 0
        while a + common < n and b + common < n and s[a + common] == s[b + common]:
            common += 1
        total -= common
    return total


# ============================================================
# FOLLOW-UP 2: Distinct substrings of one given length k
# ============================================================

# ---- follow-up 2: function ----
# Answer: Put the windows of length k in a set.
# Time: O(n * k).  Space: O(n * k).
def distinct_of_length(s, k):
    return len({s[i:i + k] for i in range(len(s) - k + 1)})
```

PART 2 · STRING MATCHING · PROBLEM 3 OF 3

1044. Longest Duplicate Substring

Hard · String, Binary Search, Suffix Array, Suffix Tree, Sliding Window, Hash Function, Rolling Hash, Boyer–Moore, Extended KMP, Suffix Automato

leetcode.com/problems/longest-duplicate-substring

★ **TOP 7 PROBLEM** in String Matching: master this one first

PROBLEM

Given a string `s`, consider all *duplicated substrings*: (contiguous) substrings of s that occur 2 or more times. The occurrences may overlap.

Return **any** duplicated substring that has the longest possible length. If `s` does not have a duplicated substring, the answer is `""`.

Example 1:

```
Input: s = "banana"
Output: "ana"
```

Example 2:

```
Input: s = "abcd"
Output: ""
```

Constraints:

- `2 <= s.length <= 3 * 10^4`
- `s` consists of lowercase English letters.

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Inserting every substring into a set is quadratic and fails for $n \leq 3 \times 10^4$. If a duplicate of length L exists, shorter duplicates exist as well, so length is monotonic. Binary search the length. The check inserts every slice of length mid into a set and returns on the first collision. A hit tries a longer length; a miss shortens it. The implementation uses ordinary Python slices and a hash set, remembering the last hit as the answer.

Python 3

```python
class Solution:
    def longestDupSubstring(self, s: str) -> str:
        def check(l: int) -> str:
            vis = set()
            for i in range(1, n - l + 2):
                j = i + l - 1
                t = h[j] - h[i - 1] * p[j - i + 1]
                if t in vis:
                    return s[i - 1 : j]
                vis.add(t)
            return ''

        base, n = 131, len(s)
        p = [0] * (n + 10)
        h = [0] * (n + 10)
        p[0] = 1
        for i, c in enumerate(s):
            p[i + 1] = p[i] * base
            h[i + 1] = h[i] * base + ord(c)
        left, right = 0, n
        ans = ''
        while left < right:
            mid = (left + right + 1) >> 1
            t = check(mid)
            if t:
                left = mid
                ans = t
            else:
                right = mid - 1
        return ans
```

Complexity. Time: O(n log n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Every longest duplicate substring (all of them)
# 2. The longest substring that appears at least k times


# ============================================================
# FOLLOW-UP 1: Every longest duplicate substring (all of
# them)
# ============================================================

# ---- follow-up 1: function ----
# Answer: Find the longest length with binary search, then
# collect all windows of that length that appear twice.
# Time: O(n^2 log n).  Space: O(n^2).
def all_longest_duplicates(s):
    def duplicates(length):
        seen = {}
        found = set()
        for i in range(len(s) - length + 1):
            w = s[i:i + length]
            if w in seen:
                found.add(w)
            seen[w] = i
        return found

    low, high = 0, len(s) - 1
    while low < high:
        middle = (low + high + 1) // 2
        if duplicates(middle):
            low = middle
        else:
```

```
            high = middle - 1
    return sorted(duplicates(low)) if low else []


# ============================================================
# FOLLOW-UP 2: The longest substring that appears at least
# k times
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same binary search with a count per window.
# Time: O(n^2 log n).  Space: O(n^2).
def longest_repeated_k_times(s, k):
    def has(length):
        count = {}
        for i in range(len(s) - length + 1):
            w = s[i:i + length]
            count[w] = count.get(w, 0) + 1
            if count[w] >= k:
                return w
        return None

    low, high = 0, len(s)
    while low < high:
        middle = (low + high + 1) // 2
        if has(middle):
            low = middle
        else:
            high = middle - 1
```

```
    return has(low) if low else ""
```

PART 2 · TOPIC 49 OF 50

Binary Indexed Tree / Segment Tree

3 problems · 0 Easy · 1 Medium · 2 Hard

THE IDEA

Update a value and query a range sum (or min, max) in O(log n) each: a Fenwick tree for prefix sums, a segment tree for general ranges.

THE 3 PROBLEMS, EASY TO HARD | ★ = TOP 3: DO THESE FIRST

1	★ **Range Sum Query 2D - Mutable**	Medium	LC 308	2768
2	★ **Block Placement Queries**	Hard	LC 3161	2774
3	★ **Longest Balanced Subarray II**	Hard	LC 3721	2778

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Range Sum Query - Mutable (Part 1, pattern 5, page 274)

Count of Smaller Numbers After Self (Part 1, pattern 14, page 753)

Falling Squares (Part 1, pattern 20, page 1142)

Number of Pairs Satisfying Inequality (Part 1, pattern 40, page 2039)

308. Range Sum Query 2D - Mutable

Medium · Design, Binary Indexed Tree, Segment Tree, Array, Matrix, Sqrt Decomposition

leetcode.com/problems/range-sum-query-2d-mutable

★ **TOP 7 PROBLEM** in Binary Indexed Tree / Segment Tree: master this one first

PROBLEM

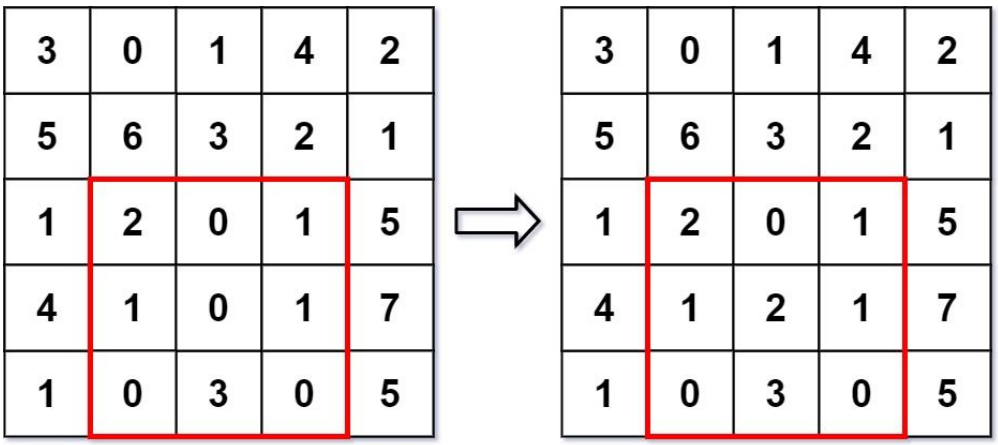
Given a 2D matrix `matrix`, handle multiple queries of the following types:

1. **Update** the value of a cell in `matrix`.
2. Calculate the **sum** of the elements of `matrix` inside the rectangle defined by its **upper left corner** `(row1, col1)` and **lower right corner** `(row2, col2)`.

Implement the NumMatrix class:

- `NumMatrix(int[][] matrix)` Initializes the object with the integer matrix `matrix`.
- `void update(int row, int col, int val)` **Updates** the value of `matrix[row][col]` to be `val`.
- `int sumRegion(int row1, int col1, int row2, int col2)` Returns the **sum** of the elements of `matrix` inside the rectangle defined by its **upper left corner** `(row1, col1)` and **lower right corner** `(row2, col2)`.

Example 1:




```
Input
["NumMatrix", "sumRegion", "update", "sumRegion"]
[[[[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]]], [2, 1, 4, 3],
[3, 2, 2], [2, 1, 4, 3]]
Output
[null, 8, null, 10]

Explanation
NumMatrix numMatrix = new NumMatrix([[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1,
7], [1, 0, 3, 0, 5]]);
numMatrix.sumRegion(2, 1, 4, 3); // return 8 (i.e. sum of the left red rectangle)
numMatrix.update(3, 2, 2); // matrix changes from left image to right image
numMatrix.sumRegion(2, 1, 4, 3); // return 10 (i.e. sum of the right red rectangle)
```

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 200`
- `-1000 <= matrix[i][j] <= 1000`
- `0 <= row < m`

- `0 <= col < n`
- `-1000 <= val <= 1000`
- `0 <= row1 <= row2 < m`
- `0 <= col1 <= col2 < n`
- At most `5000` calls will be made to `sumRegion` and `update`.

SOLUTIONS (PYTHON 3)

Solution 1

How to think about it

The matrix needs both point updates and sub-rectangle sums. Scanning a rectangle per query, or rebuilding a 2D prefix after every write, cannot keep both cheap. The code builds one Fenwick tree per row: a row range is a prefix difference, then those rows in [row1,row2] are added. An update touches only that row. With a modest number of rows this is simpler than a true 2D Fenwick tree.

Python 3

```python
class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x

    def update(self, x, delta):
        while x <= self.n:
            self.c[x] += delta
            x += BinaryIndexedTree.lowbit(x)

    def query(self, x):
        s = 0
        while x > 0:
            s += self.c[x]
            x -= BinaryIndexedTree.lowbit(x)
        return s


class NumMatrix:
    def __init__(self, matrix: List[List[int]]):
        self.trees = []
        n = len(matrix[0])
        for row in matrix:
            tree = BinaryIndexedTree(n)
            for j, v in enumerate(row):
                tree.update(j + 1, v)
            self.trees.append(tree)

    def update(self, row: int, col: int, val: int) -> None:
        tree = self.trees[row]
        prev = tree.query(col + 1) - tree.query(col)
        tree.update(col + 1, val - prev)

    def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:
        return sum(
            tree.query(col2 + 1) - tree.query(col1)
            for tree in self.trees[row1 : row2 + 1]
        )


# Your NumMatrix object will be instantiated and called as such:
# obj = NumMatrix(matrix)
# obj.update(row,col,val)
# param_2 = obj.sumRegion(row1,col1,row2,col2)
```

Complexity. Time: build O(m * n log n), update O(log n), sumRegion O(m log n). Space: O(m * n).

Solution 2

> **How to think about it**
>
> Method 1 still walks every involved row. Replacing each Fenwick tree with a segment tree keeps point updates and in-row ranges at O(log n) with the same meaning. A sub-rectangle is still the sum of per-row interval sums. Explicit interval nodes make the column-range aggregation easier to read.

Python 3

```python
class Node:
    def __init__(self):
        self.l = 0
        self.r = 0
        self.v = 0


class SegmentTree:
    def __init__(self, nums):
        n = len(nums)
        self.nums = nums
        self.tr = [Node() for _ in range(4 * n)]
        self.build(1, 1, n)

    def build(self, u, l, r):
        self.tr[u].l = l
        self.tr[u].r = r
        if l == r:
            self.tr[u].v = self.nums[l - 1]
            return
        mid = (l + r) >> 1
        self.build(u << 1, l, mid)
        self.build(u << 1 | 1, mid + 1, r)
        self.pushup(u)

    def modify(self, u, x, v):
        if self.tr[u].l == x and self.tr[u].r == x:
            self.tr[u].v = v
            return
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        if x <= mid:
            self.modify(u << 1, x, v)
        else:
            self.modify(u << 1 | 1, x, v)
        self.pushup(u)

    def query(self, u, l, r):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            return self.tr[u].v
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        v = 0
        if l <= mid:
            v += self.query(u << 1, l, r)
        if r > mid:
            v += self.query(u << 1 | 1, l, r)
        return v

    def pushup(self, u):
        self.tr[u].v = self.tr[u << 1].v + self.tr[u << 1 | 1].v


class NumMatrix:
    def __init__(self, matrix: List[List[int]]):
        self.trees = [SegmentTree(row) for row in matrix]

    def update(self, row: int, col: int, val: int) -> None:
        tree = self.trees[row]
        tree.modify(1, col + 1, val)

    def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:
        return sum(
            self.trees[row].query(1, col1 + 1, col2 + 1)
            for row in range(row1, row2 + 1)
        )
```

```
# Your NumMatrix object will be instantiated and called as such:
# obj = NumMatrix(matrix)
# obj.update(row,col,val)
# param_2 = obj.sumRegion(row1,col1,row2,col2)
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. A 2D Fenwick tree
# 2. Add an amount to a cell instead of setting it


# ============================================================
# FOLLOW-UP 1: A 2D Fenwick tree
# ============================================================

# ---- follow-up 1: function ----
# Answer: A tree over rows and columns: an update touches
# log m * log n cells, and so does a prefix query.
# Time: build O(m * n log m log n), update and sum_region
# O(log m log n).  Space: O(m * n).
class NumMatrixBIT:
    def __init__(self, matrix):
        self.m = len(matrix)
        self.n = len(matrix[0])
        self.matrix = [[0] * self.n for _ in range(self.m)]
        self.tree = [[0] * (self.n + 1) for _ in range(self.m + 1)]
        for r in range(self.m):
            for c in range(self.n):
                self.update(r, c, matrix[r][c])

    def update(self, row, col, val):
        delta = val - self.matrix[row][col]
        self.matrix[row][col] = val
        i = row + 1
        while i <= self.m:
            j = col + 1
            while j <= self.n:
                self.tree[i][j] += delta
                j += j & -j
            i += i & -i

    def _prefix(self, row, col):
        total = 0
        i = row + 1
        while i > 0:
            j = col + 1
            while j > 0:
                total += self.tree[i][j]
                j -= j & -j
            i -= i & -i
        return total

    def sum_region(self, r1, c1, r2, c2):
        return self._prefix(r2, c2) - self._prefix(r1 - 1, c2) - self._prefix(r2, c1 - 1) +
self._prefix(r1 - 1, c1 - 1)


# ============================================================
# FOLLOW-UP 2: Add an amount to a cell instead of setting
# it
# ============================================================

# ---- follow-up 2: function ----
# Answer: Pass the amount straight to the tree.
# Time: add O(log m log n).  Space: O(m * n).
```

```
class NumMatrixAdd(NumMatrixBIT):
    def add(self, row, col, delta):
```

```
        self.update(row, col, self.matrix[row][col] + delta)
```

3161. Block Placement Queries

Hard · Binary Indexed Tree, Segment Tree, Array, Binary Search, Ordered Set

leetcode.com/problems/block-placement-queries

★ **TOP 7 PROBLEM** in Binary Indexed Tree / Segment Tree: master this one first

PROBLEM

There exists an infinite number line, with its origin at 0 and extending towards the **positive** x-axis.

You are given a 2D array `queries`, which contains two types of queries:

1. For a query of type 1, `queries[i] = [1, x]`. Build an obstacle at distance `x` from the origin. It is guaranteed that there is **no** obstacle at distance `x` when the query is asked.
2. For a query of type 2, `queries[i] = [2, x, sz]`. Check if it is possible to place a block of size `sz` *anywhere* in the range `[0, x]` on the line, such that the block **entirely** lies in the range `[0, x]`. A block **cannot** be placed if it intersects with any obstacle, but it may touch it. Note that you do **not** actually place the block. Queries are separate.

Return a boolean array `results`, where `results[i]` is `true` if you can place the block specified in the i^{th} query of type 2, and `false` otherwise.

Example 1:

Input: queries = [[1,2],[2,3,3],[2,3,1],[2,2,2]]

Output: [false,true,true]

Explanation:

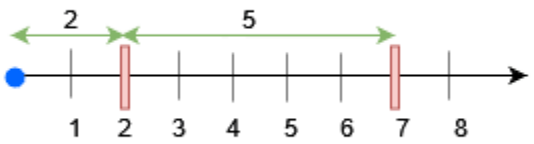
For query 0, place an obstacle at `x = 2`. A block of size at most 2 can be placed before `x = 3`.

Example 2:

Input: queries = [[1,7],[2,7,6],[1,2],[2,7,5],[2,7,6]] notionvc: 4a471445-5af1-4d72-b11b-94d351a2c8e9

Output: [true,true,false]

Explanation:



- Place an obstacle at `x = 7` for query 0. A block of size at most 7 can be placed before `x = 7`.
- Place an obstacle at `x = 2` for query 2. Now, a block of size at most 5 can be placed before `x = 7`, and a block of size at most 2 before `x = 2`.

Constraints:

- `1 <= queries.length <= 15 * 10^4`

- `2 <= queries[i].length <= 3`
- `1 <= queries[i][0] <= 2`
- `1 <= x, sz <= min(5 * 10^4, 3 * queries.length)`
- The input is generated such that for queries of type 1, no obstacle exists at distance `x` when the query is asked.
- The input is generated such that there is at least one query of type 2.

SOLUTION (PYTHON 3)

Solution 1: Binary Indexed Tree + Ordered Set

> **How to think about it**
>
> Obstacles are inserted only, and a query asks whether a block of size sz fits in [0,x]. Scanning gaps online is $O(q^2)$. Reversed time turns insertions into deletions, which only enlarge gaps. A Fenwick tree can store prefix maxima of gap lengths keyed by the right endpoint. Load every obstacle plus sentinels, write gaps, then go backward: a query checks the prefix maximum up to pre and the tail x-pre; a deletion updates the successor gap to nxt-pre. Reverse the collected answers.

Obstacles are only inserted, so we can process the queries offline in reverse and turn "insert an obstacle" into "delete an obstacle". After a deletion the adjacent gap only grows, and a Fenwick tree can maintain prefix maxima.

Put all obstacles into an ordered set, and add sentinels 0 and m+1, where m is the maximum coordinate. For every pair of neighboring obstacles x_1, x_2, update index x_2 in the Fenwick tree with the gap $x_2 - x_1$.

Then scan the queries from back to front:

- Type 2: find the last obstacle pre ≤ x. The block can be placed if the maximum gap in [0, pre] or the tail gap (pre, x] is at least sz.
- Type 1: delete obstacle x, and update the gap at its successor nxt to nxt - pre.

The time complexity is O(q × log m), and the space complexity is O(m), where q is the number of queries and m is the maximum coordinate.

Python 3

```
class BinaryIndexedTree:
    def __init__(self, n: int):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x: int, v: int):
        while x <= self.n:
            self.c[x] = max(self.c[x], v)
            x += x & -x

    def query(self, x: int) -> int:
        mx = 0
        while x:
            mx = max(mx, self.c[x])
            x -= x & -x
        return mx


class Solution:
    def getResults(self, queries: List[List[int]]) -> List[bool]:
        m = max(q[1] for q in queries)
        sl = SortedList([0, m + 1])
        for q in queries:
            if q[0] == 1:
                sl.add(q[1])
        tree = BinaryIndexedTree(m + 1)
        for x1, x2 in pairwise(sl):
            tree.update(x2, x2 - x1)
        ans = []
        for q in reversed(queries):
            x = q[1]
            if q[0] == 1:
                i = sl.index(x)
                tree.update(sl[i + 1], sl[i + 1] - sl[i - 1])
                sl.remove(x)
            else:
                i = sl.bisect_right(x)
                pre = sl[i - 1]
                ans.append(tree.query(pre) >= q[2] or x - pre >= q[2])
        return ans[::-1]
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. The length of the largest free stretch inside [0, x]
# 2. Answer a whole list of queries


# ============================================================
# FOLLOW-UP 1: The length of the largest free stretch
# inside [0, x]
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the obstacles sorted; the free stretches
# are the gaps between neighbours, cut at x.
# Time: build O(n) (insort), largest_gap O(n).  Space:
# O(n).
import bisect


class ObstacleLine:
    def __init__(self):
        self.obstacles = [0]
```

```
    def build(self, x):
        bisect.insort(self.obstacles, x)

    def largest_gap(self, x):
        cut = bisect.bisect_right(self.obstacles, x)
        points = self.obstacles[:cut] + [x]
        return max(b - a for a, b in zip(points, points[1:]))

    def can_place(self, x, size):
        return self.largest_gap(x) >= size


# ============================================================
# FOLLOW-UP 2: Answer a whole list of queries
# ============================================================

# ---- follow-up 2: function ----
# Answer: Handle the building and the checking queries in
# order with the class above.
# Time: O(q * n).  Space: O(q).
def answer_queries(queries):
    line = ObstacleLine()
    results = []
    for q in queries:
        if q[0] == 1:
            line.build(q[1])
        else:
            results.append(line.can_place(q[1], q[2]))
    return results
```

3721. Longest Balanced Subarray II

Hard · Segment Tree, Array, Hash Table, Divide and Conquer, Prefix Sum

leetcode.com/problems/longest-balanced-subarray-ii

★ **TOP 7 PROBLEM** in Binary Indexed Tree / Segment Tree: master this one first

PROBLEM

You are given an integer array `nums`.

A **subarray** is called **balanced** if the number of **distinct even** numbers in the subarray is equal to the number of **distinct odd** numbers.

Return the length of the **longest** balanced subarray.

Example 1:

Input: nums = [2,5,4,3]

Output: 4

Explanation:

- The longest balanced subarray is `[2, 5, 4, 3]`.
- It has 2 distinct even numbers `[2, 4]` and 2 distinct odd numbers `[5, 3]`. Thus, the answer is 4.

Example 2:

Input: nums = [3,2,2,5,4]

Output: 5

Explanation:

- The longest balanced subarray is `[3, 2, 2, 5, 4]`.
- It has 2 distinct even numbers `[2, 4]` and 2 distinct odd numbers `[3, 5]`. Thus, the answer is 5.

Example 3:

Input: nums = [1,2,3,2]

Output: 3

Explanation:

- The longest balanced subarray is `[2, 3, 2]`.
- It has 1 distinct even number `[2]` and 1 distinct odd number `[3]`. Thus, the answer is 3.

Constraints:

- `1 <= nums.length <= 10^5`
- `1 <= nums[i] <= 10^5`

SOLUTION (PYTHON 3)

Solution 1: Segment Tree + Prefix Sum + Hash Table

How to think about it

The quadratic scan of the previous problem does not scale. Mapping odds to +1 and evens to -1, balance becomes a zero-sum segment; a repeated value must undo its earlier contribution. Those updates are range adds, and we query the earliest prefix equal to the current sum, which a segment tree with min/max and lazy tags supports.

We can transform the problem into a prefix sum problem. Define a prefix sum variable now, representing the difference between odd and even numbers in the current subarray:

now = distinct odd numbers - distinct even numbers

For odd elements, record as +1, and for even elements, record as -1. Use a hash table last to record the last occurrence position of each number. If a number appears repeatedly, we need to revoke its previous contribution.

To efficiently calculate the subarray length each time a right endpoint element is added, we use a segment tree to maintain the minimum and maximum values of the interval prefix sum, while supporting interval addition operations and binary search queries on the segment tree. When iterating to right endpoint i, first update the contribution of the current element, then use the segment tree to query the earliest position pos where the current prefix sum now appears. The current subarray length is i - pos, and we update the answer:

$$\text{ans} = \max(\text{ans}, i - \text{pos})$$

The time complexity is O(n log n), where n is the length of the array. Each segment tree modification and query operation takes O(log n), and enumerating the right endpoint n times gives a total time complexity of O(n log n). The space complexity is O(n), where segment tree nodes and the hash table each occupy O(n) space.

Python 3

```python
class Solution:
    def longestBalanced(self, nums: List[int]) -> int:
        n = len(nums)

        class Node:
            __slots__ = ("l", "r", "mn", "mx", "lazy")
            def __init__(self):
                self.l = self.r = 0
                self.mn = self.mx = 0
                self.lazy = 0

        tr = [Node() for _ in range((n + 1) * 4)]

        def build(u: int, l: int, r: int):
            tr[u].l, tr[u].r = l, r
            tr[u].mn = tr[u].mx = tr[u].lazy = 0
            if l == r:
                return
            mid = (l + r) >> 1
            build(u << 1, l, mid)
            build(u << 1 | 1, mid + 1, r)

        def apply(u: int, v: int):
            tr[u].mn += v
            tr[u].mx += v
            tr[u].lazy += v

        def pushdown(u: int):
            if tr[u].lazy:
                apply(u << 1, tr[u].lazy)
                apply(u << 1 | 1, tr[u].lazy)
                tr[u].lazy = 0

        def pushup(u: int):
            tr[u].mn = min(tr[u << 1].mn, tr[u << 1 | 1].mn)
            tr[u].mx = max(tr[u << 1].mx, tr[u << 1 | 1].mx)

        def modify(u: int, l: int, r: int, v: int):
            if tr[u].l >= l and tr[u].r <= r:
                apply(u, v)
                return
            pushdown(u)
            mid = (tr[u].l + tr[u].r) >> 1
            if l <= mid:
                modify(u << 1, l, r, v)
            if r > mid:
                modify(u << 1 | 1, l, r, v)
            pushup(u)

        def query(u: int, target: int) -> int:
            if tr[u].l == tr[u].r:
                return tr[u].l
            pushdown(u)
            if tr[u << 1].mn <= target <= tr[u << 1].mx:
                return query(u << 1, target)
            return query(u << 1 | 1, target)

        build(1, 0, n)

        last = {}
        now = ans = 0

        for i, x in enumerate(nums, start=1):
            det = 1 if (x & 1) else -1
            if x in last:
                modify(1, last[x], n, -det)
```

```
                now -= det
            last[x] = i
            modify(1, i, n, det)
            now += det
            pos = query(1, now)
            ans = max(ans, i - pos)

        return ans
```

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the longest balanced subarray itself
# 2. Count all the balanced subarrays


# ============================================================
# FOLLOW-UP 1: Return the longest balanced subarray itself
# ============================================================

# ---- follow-up 1: function ----
# Answer: Simple version: for every start, extend and keep
# the sets of distinct even and odd numbers.
# Time: O(n^2).  Space: O(n).
def longest_balanced_subarray(nums):
    best = (0, 0, 0)
    for i in range(len(nums)):
        evens, odds = set(), set()
        for j in range(i, len(nums)):
            (evens if nums[j] % 2 == 0 else odds).add(nums[j])
            if len(evens) == len(odds) and j - i + 1 > best[0]:
                best = (j - i + 1, i, j)
    return nums[best[1]:best[2] + 1] if best[0] else []


# ============================================================
# FOLLOW-UP 2: Count all the balanced subarrays
# ============================================================

# ---- follow-up 2: function ----
# Answer: Same sweep counting every balanced end.
# Time: O(n^2).  Space: O(n).
def count_balanced_subarrays(nums):
    total = 0
    for i in range(len(nums)):
        evens, odds = set(), set()
        for j in range(i, len(nums)):
            (evens if nums[j] % 2 == 0 else odds).add(nums[j])
            if len(evens) == len(odds):
                total += 1
    return total
```

PART 2 · TOPIC 50 OF 50

Line Sweep

1 problem · 0 Easy · 0 Medium · 1 Hard

THE IDEA

Turn the picture into events sorted by position (start, end, building edges) and sweep once while keeping the active set.

THE 1 PROBLEMS, EASY TO HARD | ★ = TOP 1: DO THESE FIRST

1	★ **Perfect Rectangle**	Hard	LC 391	2783

ALSO IN THIS ALGOMASTER GROUP, ALREADY IN PART 1

Employee Free Time (Part 1, pattern 20, page 1122)

Minimum Interval to Include Each Query (Part 1, pattern 20, page 1128)

The Skyline Problem (Part 1, pattern 14, page 750)

Rectangle Area II (Part 1, pattern 40, page 2017)

PART 2 · LINE SWEEP · PROBLEM 1 OF 1

391. Perfect Rectangle

Hard · Geometry, Array, Hash Table, Math, Sweep Line

leetcode.com/problems/perfect-rectangle

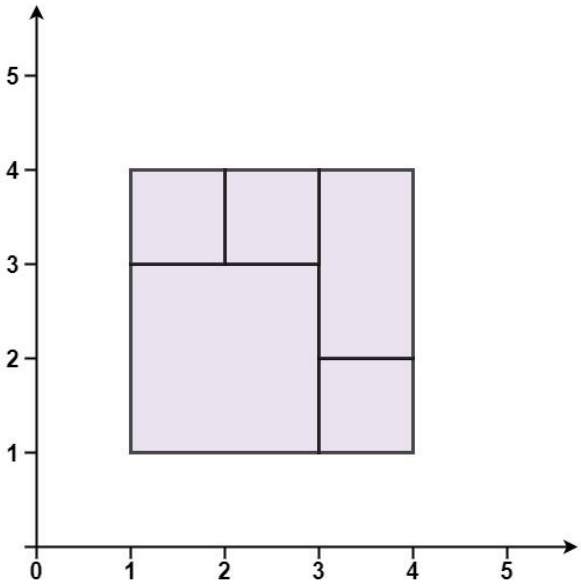
★ **TOP 7 PROBLEM** in Line Sweep: master this one first

PROBLEM

Given an array `rectangles` where `rectangles[i] = [x_i, y_i, a_i, b_i]` represents an axis-aligned rectangle. The bottom-left point of the rectangle is `(x_i, y_i)` and the top-right point of it is `(a_i, b_i)`.

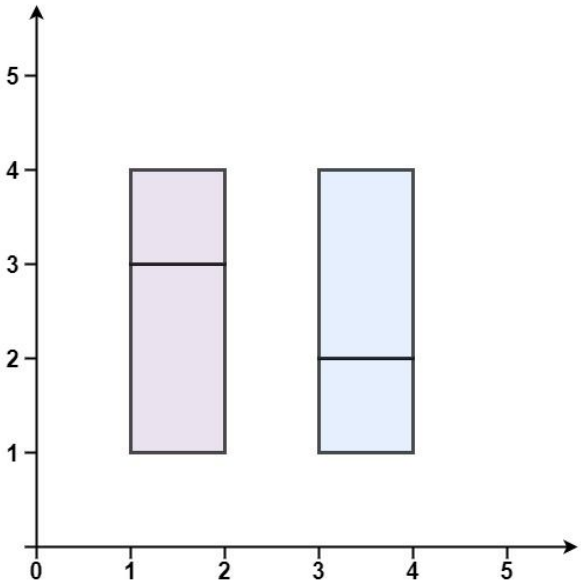
Return `true` *if all the rectangles together form an exact cover of a rectangular region*.

Example 1:



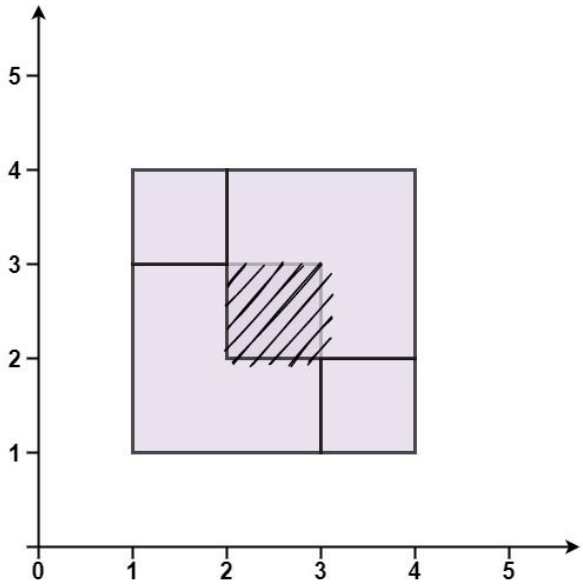

```
Input: rectangles = [[1,1,3,3],[3,1,4,2],[3,2,4,4],[1,3,2,4],[2,3,3,4]]
Output: true
Explanation: All 5 rectangles together form an exact cover of a rectangular region.
```

Example 2:



```
Input: rectangles = [[1,1,2,3],[1,3,2,4],[3,1,4,2],[3,2,4,4]]
Output: false
Explanation: Because there is a gap between the two rectangular regions.
```

Example 3:




```
Input: rectangles = [[1,1,3,3],[3,1,4,2],[1,3,2,4],[2,2,4,4]]
Output: false
Explanation: Because two of the rectangles overlap with each other.
```

Constraints:

- $1 <= rectangles.length <= 2 * 10^4$
- $rectangles[i].length == 4$
- $-10^5 <= x_i < a_i <= 10^5$
- $-10^5 <= y_i < b_i <= 10^5$

SOLUTION (PYTHON 3)

Solution 1

How to think about it

Can the small rectangles tile a large one without gaps or overlaps? Sweep lines are heavy. A perfect cover has equal area, and every vertex except the four outer corners appears an even number of times (2 or 4). Sum areas and count vertices. The bounding-box corners must appear once; every other vertex 2 or 4 times; area equals the bounding box.

Python 3

```python
class Solution:
    def isRectangleCover(self, rectangles: List[List[int]]) -> bool:
        area = 0
        minX, minY = rectangles[0][0], rectangles[0][1]
        maxX, maxY = rectangles[0][2], rectangles[0][3]
        cnt = defaultdict(int)

        for r in rectangles:
            area += (r[2] - r[0]) * (r[3] - r[1])

            minX = min(minX, r[0])
            minY = min(minY, r[1])
            maxX = max(maxX, r[2])
            maxY = max(maxY, r[3])

            cnt[(r[0], r[1])] += 1
            cnt[(r[0], r[3])] += 1
            cnt[(r[2], r[3])] += 1
            cnt[(r[2], r[1])] += 1

        if (
            area != (maxX - minX) * (maxY - minY)
            or cnt[(minX, minY)] != 1
            or cnt[(minX, maxY)] != 1
            or cnt[(maxX, maxY)] != 1
            or cnt[(maxX, minY)] != 1
        ):
            return False

        del cnt[(minX, minY)], cnt[(minX, maxY)], cnt[(maxX, maxY)], cnt[(maxX, minY)]

        return all(c == 2 or c == 4 for c in cnt.values())
```

Complexity. Time: O(n). Space: O(n).

FOLLOW-UPS AN INTERVIEWER MAY ASK NEXT (CODE ONLY)

Built on the solutions above: helper functions, then the function for each follow-up. No tests and no restating of the question.

```python
# FOLLOW-UPS (answered below, in this order):
# 1. Is there any overlap between two rectangles
# 2. The total area that is NOT covered inside the
# bounding box


# ============================================================
# FOLLOW-UP 1: Is there any overlap between two rectangles
# ============================================================

# ---- follow-up 1: function ----
# Answer: Compare the pairs (simple) or sweep the x-axis.
# Time: O(n^2).  Space: O(1).
def any_overlap(rectangles):
    for i in range(len(rectangles)):
        x1, y1, a1, b1 = rectangles[i]
        for j in range(i + 1, len(rectangles)):
            x2, y2, a2, b2 = rectangles[j]
            if max(x1, x2) < min(a1, a2) and max(y1, y2) < min(b1, b2):
                return True
    return False


# ============================================================
# FOLLOW-UP 2: The total area that is NOT covered inside
# the bounding box
# ============================================================

# ---- follow-up 2: function ----
```

```
# Answer: The area of the bounding box minus the sum of
# the areas (valid when there is no overlap).
# Time: O(n).  Space: O(1).
def uncovered_area(rectangles):
    x_low = min(r[0] for r in rectangles)
    y_low = min(r[1] for r in rectangles)
    x_high = max(r[2] for r in rectangles)
    y_high = max(r[3] for r in rectangles)
    box = (x_high - x_low) * (y_high - y_low)
    return box - sum((a - x) * (b - y) for x, y, a, b in rectangles)
```